

Chapter 1: ISSChannelAgSizeContri

Last Updated: 2025-05-23

1. Business Objective

This module calculates the final size contribution for each Attribute Group (AG) within a specific sales channel, providing the necessary size distribution to determine the Ideal Size Set (ISS) and optimize the Rate of Sale (ROS).

Why This Matters: This ensures that inventory planners stock the right mix of sizes for popular product types in each sales channel (e.g., online, retail store), reducing missed sales opportunities from stockouts on popular sizes and minimizing excess inventory for less frequently sold sizes. It directly impacts profitability by improving inventory efficiency and enhancing customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold per Product:** To understand the exact demand for each product, which is critical for estimating size-level sales.
- **Sales Channel (e.g., Online, Retail Store):** To identify where products are being sold, as size preferences can vary significantly between different sales channels.

Product Data

- **Product Category and Attributes:** To group similar products (e.g., “Men’s T-Shirts,” “Women’s Denim Jeans”) using Attribute Groups (AGs) and ensure that size analysis is relevant to specific product characteristics.
- **Available Product Sizes:** To identify all sizes offered for a particular product (e.g., S, M, L, XL) and analyze their individual sales performance.

Operational Data

- **Adjusted Sales Contribution per Size (Pivotal Row Data):** This is historical data that provides a weighted importance of each size’s past performance, factoring in adjustments like stock availability or specific promotional impacts.

- **Store Location Information:** To link sales data to specific stores and understand their sales channels, ensuring that size contribution calculations are relevant to the actual selling environment.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Minimum Sales Threshold for Size Analysis	50 units	The minimum number of units an Attribute Group must sell within a channel to be included in detailed size analysis.	For new product launches with limited data, or for very slow-moving items where more data might be needed.
Size Contribution Rounding Precision	2 decimal places	Determines how many decimal places the final size contribution percentages are rounded to.	If more granular detail is needed for very precise inventory allocation.
Historical Data Lookback Period	12 weeks	Defines the sales period (e.g., past 12 weeks) from which data is used to calculate size contributions.	To react faster to recent seasonal trends or product lifecycle changes, or to capture longer-term patterns.
Outlier Sales Exclusion Threshold	10x Average	Prevents unusually high or low sales events (e.g., a single large bulk order) from skewing size contributions.	To adjust for significant promotional spikes, one-off events, or identified data anomalies.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Sales Data & Product Attributes] --> B[Calculate Size Contributions]

B --> C[Ideal Size Set Proportions]

1. Start with → Historical Sales Data and Product Attributes
2. Then → **Aggregate** Sales by Store, Category, and Attribute Group (AG)
3. Next → **Determine** Interim Size Quantities for each Attribute Group (AG) within each Channel, using historical sales and pivotal row data
4. Finally → **Calculate** the Proportional Contribution of each size within its Attribute Group (AG) and Channel, representing the Ideal Size Set (ISS) distribution.

5. Step-by-Step Actions

1. **Initialize** Data Structures
 - What we do: *We set up all necessary temporary storage spaces to ensure a clean start for the calculations.*
 - Why we do it: *This prepares the module to process new data efficiently without carrying over old information.*
2. **Compile** Store and Attribute Group Sales
 - What we do: *We sum up total units sold for each product category within each individual store. Simultaneously, we also sum total units sold for each specific Attribute Group (AG) within its respective sales channel.*
 - Why we do it: *This provides critical demand figures at both a broad (store/category) and a granular (channel/AG) level, which are used in subsequent calculations.*
3. **Estimate** Interim Size-Specific Quantities
 - What we do: *For every Attribute Group (AG) and sales channel combination, we use the compiled store-category sales quantities and historical 'adjusted contribution factors' for each size (from the 'Pivotal Row' data) to estimate how many units of each size were likely sold. These are 'interim' quantities because they are a weighted estimation.*
 - Why we do it: *This step is crucial for translating high-level sales data into size-specific estimations, especially when exact size sales data isn't always complete.*
4. **Calculate** Final Size Proportions
 - What we do: *For each Attribute Group (AG) and sales channel, we sum up all the estimated interim size quantities to get a total demand 'weight' for that group. Then, we divide each individual size's estimated interim quantity by this total 'weight' to determine its final proportional sales contribution (as a percentage).*
 - Why we do it: *This is the core calculation that directly informs the Ideal Size Set (ISS), showing the precise mix of sizes that should be stocked to meet customer demand and optimize sell-through.*
5. **Record and Report** Results
 - What we do: *We save the calculated final size contributions to our main database for future reference and generate a comprehensive export report that includes additional product attributes (like category, brand, gender) for detailed analysis.*
 - Why we do it: *This ensures the critical Ideal Size Set (ISS) recommendations are available for subsequent planning modules and for business stakeholders to review and act upon.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A specific Attribute Group (AG) like "Men's Casual T-Shirts - Crew Neck - Cotton" sold 1,000 units in the 'Online' channel over the last 3 months.
- Across all stores selling these T-shirts, the overall 'Apparel' category total sales (from which these T-shirts are a part) were 50,000 units.

- Based on historical data and adjusted contributions, the ‘Pivotal Row’ data suggests for ‘Men’s Casual T-Shirts’: Size S had an adjusted contribution factor of 0.05, Size M of 0.15, Size L of 0.12, Size XL of 0.08, etc.

What Happens

1. The system first compiles that 1,000 units of the ‘Men’s Casual T-Shirts’ AG were sold in the ‘Online’ channel, and that the relevant store-category sales total was 50,000 units.
2. It then uses this overall store-category sales figure (e.g., if a store had 1000 units of ‘Apparel’ category sales) and applies the adjusted contribution factors for each size to estimate the *interim* quantity of each size. For example, if the adjusted contribution for Size M was 0.15, its interim quantity would be estimated as $1000 * 0.15 = 150$ units for that store-category. This calculation is done across all relevant stores and categories and then aggregated for the ‘Men’s Casual T-Shirts’ AG in the ‘Online’ channel.
3. Next, for the ‘Men’s Casual T-Shirts’ AG in the ‘Online’ channel, it sums up all these estimated interim size quantities to get a total ‘weight’. Let’s say the summed interim quantities for this AG were: S=100, M=200, L=150, XL=120, resulting in a total of 1000 ‘weighted’ units.
4. Finally, it calculates the *final* proportional contribution for each size. For Size M, if its interim quantity was 200 and the total for the AG was 1000, its final contribution is $200 / 1000 = 0.20$ (or 20%). This process is repeated for all other sizes.

Final Outcome

The module outputs that for ‘Men’s Casual T-Shirts’ in the ‘Online’ channel, the Ideal Size Set (ISS) should be 20% for Size M, 15% for Size L, 10% for Size S, and so on, for all relevant sizes.

Key Takeaway: Understanding the precise proportional demand for each size within specific product groups and sales channels is critical for optimizing inventory and improving sales efficiency.

7. Results

This step produces:

Main Outputs

- **Ideal Size Set Proportions by Channel and Attribute Group (AG):** This shows the calculated percentage contribution of each size for specific product groups and sales channels, forming the core of the Ideal Size Set (ISS) recommendation.
- **Detailed Ideal Size Set Export Report:** This provides a comprehensive view of the calculated size contributions, including detailed product attributes (like category, brand, gender, price bucket, theme) for deeper analysis and reporting.

Business Reports

- **Size Distribution Report:** Used by merchandising and planning teams to inform future buying decisions and ensure optimal stock levels are maintained to maximize the Rate of Sale (ROS).
- **Channel-Specific Sizing Insights:** Used by e-commerce and retail operations managers to tailor inventory to the unique size demands of online versus physical stores, reducing inefficient stock movement.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data or halt processing if critical data is absent.	Verify data sources and resubmit the process once data is complete and accurate.
No sales data for a specific Attribute Group (AG)	The module will not be able to calculate size contributions for that AG, potentially leading to default or generic size sets being used in downstream processes.	Investigate why sales data is missing; if it's a new product, consider using broader category trends or manual input.
Extremely low sales for an Attribute Group (AG)	Calculated size contributions might be unreliable due to insufficient data, leading to skewed percentages.	Consider grouping this AG with a larger, similar one for analysis, or applying a stricter 'Minimum Sales Threshold' setting.
Sudden shift in size popularity (e.g., due to trend or campaign)	Historical data might not immediately reflect the new trend, potentially leading to stocking decisions based on outdated preferences.	Adjust the 'Historical Data Lookback Period' setting to a shorter duration to react faster to new trends.
Incorrectly classified product attributes	The module will group and analyze products incorrectly, leading to inaccurate size contributions for relevant Attribute Groups (AGs).	Conduct regular audits of product data quality and classification to ensure accuracy.

9. Next Steps

Before This Step

- **Product Sales Data Collection:** We need accurate and complete historical sales records, broken down by individual product and size where available, to analyze true demand.

- **Attribute Group Definition and Classification:** We need products to be correctly categorized into Attribute Groups (AGs) (e.g., by style, material, gender) to ensure relevant size analysis.
- **Pivotal Row Data Generation:** We need the ‘Pivotal Row’ data, which contains adjusted contribution factors based on past sales and stock availability, to help estimate size demand.

After This Step

- **Ideal Size Set (ISS) Optimization:** The calculated size contributions are fed into subsequent modules to determine the final Ideal Size Set (ISS), informing exact unit allocations per size to maximize the Rate of Sale (ROS).
- **Inventory Planning and Allocation:** The Ideal Size Set (ISS) recommendations are used by inventory planners to decide how many units of each size to buy and distribute across stores and channels, impacting overall inventory investment.
- **Merchandising Strategy Review:** Merchandising teams use this data to understand true size demand and adapt product assortment and display strategies to better serve customer preferences and improve sell-through.

10. Common Questions

Q: How accurate are these size contributions if we don’t have exact size-level sales data for all products? A: The module uses a smart approach, leveraging overall product category sales and ‘adjusted contribution factors’ from similar or historical products to estimate size-level demand. This makes the best possible recommendation even with incomplete granular data, ensuring our Ideal Size Set (ISS) is as precise as possible.

Q: Can this process account for new products that have no sales history? A: For completely new products, the module might use broader category trends or require a default Ideal Size Set (ISS). It performs best once some initial sales data is available to analyze, as its strength lies in leveraging actual sales performance.

Q: Why is it important to analyze size contributions by sales channel (e.g., online vs. in-store)? A: Customer size preferences can differ significantly between channels. For example, online shoppers might buy a wider range of sizes, while in-store shoppers might stick to more common sizes. Analyzing by channel ensures optimized stock where it’s sold, directly impacting local store performance and online fulfillment efficiency.

Q: How frequently should these size contributions be recalculated? A: It’s recommended to recalculate periodically (e.g., quarterly or semi-annually) to capture shifts in customer demand, seasonal trends, or product lifecycle changes. This ensures your Ideal Size Sets (ISS) remain relevant and maximize your Rate of Sale (ROS).

Q: How does this help prevent stockouts and overstocking? A: By precisely identifying the proportional demand for each size within an Attribute Group (AG), the module helps businesses stock more of what customers want and less of what they don’t. This directly reduces stockouts on popular sizes, minimizing lost sales, and significantly cuts down on excess inventory for less popular ones, reducing holding costs.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate the ideal proportional sales contribution for each size within specific product groups (Attribute Groups) and sales channels.
- **Main Input:** Historical sales data and product attribute classifications, combined with adjusted contribution factors.
- **Main Output:** Percentage contribution of each size for every Attribute Group (AG) within each channel, forming the basis of the Ideal Size Set (ISS).
- **Warning Signs:** Skewed size percentages for low-selling items, or very slow adaptation to new trends.
- **Key Settings:** Minimum Sales Threshold for Size Analysis, Historical Data Lookback Period.

Need Help? Contact your Merchandising Operations or Inventory Planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 2:

StyleSkuBuyConversionModule

Last Updated: 2025-05-23

1. Business Objective

This module converts the total number of units a business plans to purchase for a specific product style into the exact quantity needed for each individual size, effectively determining the Ideal Size Set (ISS) for procurement.

Why This Matters: This ensures that when new inventory is ordered, it perfectly matches the anticipated customer demand for each size, maximizing sales by preventing stockouts on popular sizes and minimizing excess inventory on less popular ones. It translates strategic buying decisions into actionable size-level orders, directly improving inventory efficiency and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Ideal Size Set Proportions by Channel and Attribute Group (AG):** We need the calculated percentage breakdown of how each size contributes to sales within a specific product group and sales channel (e.g., for “Men’s Casual T-Shirts - Online”, Size M is 20%). This is provided by the `ISSChannelAgSizeContri` module and is crucial for distributing the total buy quantity across sizes.
- **Historical Sales Trends:** We use this to understand broader sales patterns that inform the initial style-level buy decisions, which this module then translates into specific sizes.

Product Data

- **Style-Level Buy Quantities:** We need the total number of units planned for purchase for each specific product style (e.g., “Buy 500 units of Style X T-Shirt”). This is the overarching quantity that needs to be distributed across sizes.
- **Product Category and Associated Sizes:** We need to know which product category a style belongs to (e.g., “Apparel,” “Footwear”) and the full range of sizes available for that category. This helps in understanding applicable Minimum Order Quantities and potential size sets.

Operational Data

- **Category Minimum Order Quantity (MOQ):** We need to know the minimum number of units that must be ordered for any single size within a certain product category (e.g., suppliers might require a minimum of 6 units per size for a specific apparel category). This ensures orders meet supplier requirements.
- **Style Parent-Child Relationships:** For products that are variations of a main style (e.g., different colors of the same T-shirt style), knowing the parent style helps ensure that the correct Attribute Group (AG) and its size contributions are applied consistently.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Minimum Order Quantity (MOQ) per Category	Varies by Category	The lowest quantity of units that can be ordered for a single size within a specific product category.	Negotiated with suppliers; updated when terms change to reflect new purchasing agreements.
Round Robin Adjustment Priority	Varies by Size	Dictates which sizes receive the extra unit when the total buy quantity doesn't perfectly divide among sizes.	Can be adjusted if a specific size is consistently understocked or overstocked due to rounding.
Zero Sales Buy Row Handling	Default Sizes	Specifies how sizes are allocated for styles that have no sales history or size contribution data.	For new product launches or very niche items, where initial size distribution needs a predefined approach.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Total Style Buy Quantity] --> B[Apply Ideal Size Set Proportions]

B --> C[Adjust for Minimum Order Quantity (MOQ)]

C --> D[Apply Round Robin Adjustment]

D --> E[Final Size-Level Buy Quantities]

1. Start with → **Total Style Buy Quantity** and **Ideal Size Set Proportions** (from ISSChannelAgSizeContri)
2. Then → **Calculate** initial size-level quantities based on the proportions
3. Next → **Adjust** these quantities to meet **Minimum Order Quantity (MOQ)** requirements for each size
4. Finally → **Refine** quantities using a **Round Robin adjustment** to ensure the total matches the original style buy quantity, yielding **Final Size-Level Buy Quantities**

5. Step-by-Step Actions

1. Map Styles to Attribute Groups

- What we do: *We link each specific product style, including its variations, to its corresponding Attribute Group (AG) to ensure we use the correct Ideal Size Set proportions.*
- Why we do it: *This ensures that the buying quantities for a particular style, like a “Women’s Denim Jeans - Slim Fit,” are distributed across sizes based on the sales patterns of that specific product group, not just a generic “Women’s Jeans” category.*

2. Calculate Initial Size Allocations

- What we do: *For each style’s total buy quantity, we multiply it by the Ideal Size Set proportion (percentage) for each size in its respective Attribute Group (AG) and sales channel. This gives us the preliminary, fractional quantity for each size.*
- Why we do it: *This step translates the strategic ‘buy X units’ decision into an initial estimation of how many units of each size are theoretically needed, based on historical demand patterns.*

3. Adjust for Minimum Order Quantity (MOQ)

- What we do: *We review the initially allocated quantity for each size. If any size’s quantity falls below its category’s Minimum Order Quantity (MOQ) requirement, we increase it to meet that minimum.*
- Why we do it: *This ensures that our proposed purchase order quantities are practical and adhere to supplier terms, preventing issues with orders that are too small for individual sizes.*

4. Refine with Round Robin Adjustment

- What we do: *After MOQ adjustments, the sum of individual size quantities might not perfectly match the original total style buy quantity due to rounding or MOQs. We distribute any remaining difference (positive or negative) by adding or subtracting one unit at a time to sizes in a prioritized, cyclical (round robin) fashion until the total matches the original style buy quantity.*
- Why we do it: *This step ensures that the final recommended size-level buy quantities add up precisely to the total units we intended to buy for that style, maintaining overall inventory budget and strategic buying goals.*

5. Record and Report Results

- What we do: *We save the final, adjusted size-level buy quantities to the database and generate a detailed report, including product attributes for easy review by planning and merchandising teams.*
- Why we do it: *This provides the definitive Ideal Size Set (ISS) breakdown for procurement, enabling precise ordering and ensuring transparency and traceability of the allocation process.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Style Buy Quantity:** A buyer decides to purchase 100 units of a new “Women’s Slim Fit Jeans - Indigo Wash” style for the ‘Online’ channel.

- **Ideal Size Set Proportions (from ISSChannelAgSizeContr1):** For the “Women’s Denim Jeans - Slim Fit” Attribute Group (AG) in the ‘Online’ channel, the proportions are: Size 24: 5%, Size 26: 15%, Size 28: 25%, Size 30: 25%, Size 32: 20%, Size 34: 10%.
- **Category Minimum Order Quantity (MOQ):** For ‘Women’s Jeans’, the supplier requires a minimum order of 6 units per individual size.

What Happens

1. **Initial Calculation:** The module first calculates the preliminary quantities:
 - Size 24: $100 \text{ units} * 0.05 = 5 \text{ units}$
 - Size 26: $100 \text{ units} * 0.15 = 15 \text{ units}$
 - Size 28: $100 \text{ units} * 0.25 = 25 \text{ units}$
 - Size 30: $100 \text{ units} * 0.25 = 25 \text{ units}$
 - Size 32: $100 \text{ units} * 0.20 = 20 \text{ units}$
 - Size 34: $100 \text{ units} * 0.10 = 10 \text{ units}$ (Total = 100 units)
2. **MOQ Adjustment:** The MOQ is 6 units. Size 24 (5 units) is below the MOQ, so it’s adjusted to 6 units. All other sizes meet the MOQ.
 - Size 24: 6 units (adjusted from 5)
 - Size 26: 15 units
 - Size 28: 25 units
 - Size 30: 25 units
 - Size 32: 20 units
 - Size 34: 10 units (New Total = 101 units)
3. **Round Robin Adjustment:** The total is now 101 units, but the original buy was 100 units. The module identifies the difference of 1 unit (+1 over target). It then uses a Round Robin approach to reduce one unit from a size to bring the total back to 100. For instance, it might subtract 1 unit from Size 28.
 - Size 24: 6 units
 - Size 26: 15 units
 - Size 28: 24 units
 - Size 30: 25 units
 - Size 32: 20 units
 - Size 34: 10 units (Final Total = 100 units)

Final Outcome

The module recommends purchasing 6 units of Size 24, 15 units of Size 26, 24 units of Size 28, 25 units of Size 30, 20 units of Size 32, and 10 units of Size 34 for the “Women’s Slim Fit Jeans - Indigo Wash” style. This precisely aligns with the buyer’s 100-unit target while adhering to supplier MOQs and customer demand patterns.

Key Takeaway: This module precisely converts a high-level buy decision into a detailed size-level plan, ensuring that every unit ordered contributes optimally to meeting customer demand and respecting operational constraints.

7. Results

This step produces:

Main Outputs

- **Channel-Specific Style Size Buy Quantities (Pre-MOQ Adjustment):** An interim output showing the initial size quantity distribution based solely on Ideal Size Set proportions before any MOQ or Round Robin adjustments are applied.
- **Channel-Specific Style Size Buy Quantities (MOQ Adjusted):** An interim output showing size quantities after meeting Minimum Order Quantity requirements for each size.
- **Final Ideal Size Set (ISS) Buy Quantities by Style and Channel:** The core output, detailing the exact number of units for each size to be purchased for a given style in a specific sales channel, fully adjusted for MOQs and rounding.

Business Reports

- **Purchase Order (PO) Size Breakdown Report:** Used by procurement and inventory teams to generate precise purchase orders for suppliers, ensuring the correct mix of sizes for new inventory.
- **Inventory Allocation Plans:** Informs subsequent modules and teams on how new inventory should be distributed across warehouses or stores based on the refined size quantities.
- **Buy Plan Compliance Report:** Allows merchandising and planning managers to verify that the final size-level buy quantities align with strategic overall buy targets and Ideal Size Set (ISS) objectives.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data or halt processing if critical data is absent, potentially leading to outdated or incomplete size allocations.	Verify all upstream data sources (especially Ideal Size Set proportions and Style Buy quantities) and resubmit the process once data is complete and accurate.
Style has no Ideal Size Set proportions	If a style's Attribute Group (AG) has no calculated Ideal Size Set (ISS) data (e.g., a brand new product), the module will fall back to a default size distribution or allocate zero units per size.	For new product launches, ensure a default Ideal Size Set (ISS) is defined or use broader category trends until sufficient sales data is gathered.
Total buy quantity is too low for MOQs	If the overall style buy quantity is very small, and individual size MOQs are relatively high, the module might adjust individual sizes up, causing the total to exceed the initial buy quantity before Round Robin adjustment.	Review the initial style buy quantity against the category's Minimum Order Quantities (MOQs); consider increasing the total buy or adjusting MOQ policies for very small orders.

Situation	What Happens	What You Should Do
Significant change in Category MOQ	If a supplier changes their Minimum Order Quantity (MOQ) for a category, but this is not updated in the system, orders may be rejected or incorrect.	Regularly update and audit Category Minimum Order Quantity (MOQ) settings to reflect current supplier agreements.

9. Next Steps

Before This Step

- **Ideal Size Set (ISS) Proportions Calculation (from `ISSChannelAgSizeContri`):** We need the calculated proportional contribution of each size for specific Attribute Groups (AGs) within sales channels, as this is the basis for converting style buys to size buys.
- **Style-Level Buy Quantity Definition:** We need the total number of units a business intends to purchase for each product style, typically determined by merchandising and planning teams based on sales forecasts and inventory targets.

After This Step

- **Purchase Order Generation:** The final size-level buy quantities are used to create precise purchase orders for suppliers, ensuring the right sizes are ordered for production.
- **Inventory Receiving and Distribution:** When products arrive, the detailed size breakdown from this module guides warehouse teams on how many units of each size to expect and how to prepare for distribution to stores or fulfillment centers.
- **Financial Planning and Budgeting:** The precise size-level quantities provide accurate data for cost calculations and inventory valuation, impacting overall financial planning and budgeting.

10. Common Questions

Q: How does this module ensure we don't order too many or too few of specific sizes? A: By taking the Ideal Size Set (ISS) proportions (which tell us actual customer size demand) and applying them to your total buy quantity, then adjusting for Minimum Order Quantity (MOQ) and using a Round Robin adjustment, the module precisely aligns your order with what customers are likely to buy, preventing both stockouts and overstocking.

Q: What if a new style has no sales history or Ideal Size Set proportions? A: For styles without historical data, the module might rely on default size distributions defined for their broader category or require a manual override. The process works best once some initial sales data is available from similar products to provide accurate size contributions.

Q: Why is Minimum Order Quantity (MOQ) adjustment so important? A: MOQ adjustment is critical because suppliers often require a minimum number of units per size. Without it, our calculated quantities for less popular sizes might be too low, leading to rejected orders or higher unit costs. This step ensures our orders are feasible and cost-effective.

Q: How does the Round Robin adjustment work and why is it necessary? A: The Round Robin adjustment distributes any small differences that occur when rounding quantities to whole numbers or after MOQ adjustments. It ensures that the sum of all individual size quantities exactly matches the total number of units you intended to buy for that style, maintaining your overall inventory budget.

Q: Does this module account for different sales channels (e.g., online vs. physical stores)? A: Yes, it absolutely does. It leverages Ideal Size Set (ISS) proportions that are already calculated *per sales channel*. This means the module will recommend different size breakdowns for the same style if it's being bought for online sales versus an in-store display, reflecting channel-specific customer preferences.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To convert a total style buy quantity into precise unit counts for each individual size, forming the Ideal Size Set (ISS) for procurement.
- **Main Input:** Total style buy quantities and Ideal Size Set proportions (from `ISSChannelAgSizeContr`).
- **Main Output:** Exact unit quantities for each size per style, adjusted for Minimum Order Quantity (MOQ) and rounding.
- **Warning Signs:** Final size quantities not adding up to the total buy, or orders being rejected by suppliers due to not meeting MOQs.
- **Key Settings:** Minimum Order Quantity (MOQ) per Category, Round Robin Adjustment Priority.

Need Help? Contact your Inventory Planning or Merchandising Operations team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

ap FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ provides quick answers to common questions about the Assortment Plan (AP) algorithm, its process, and its outputs, designed for business users.

General

Q: What is the main goal of the ap algorithm? **A:** The primary goal of the ap algorithm is to generate an Assortment Plan that helps optimize product stock levels by recommending the **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)** to maximize sales and minimize stockouts and overstock.

Q: Who is this algorithm designed for? **A:** This algorithm is primarily designed for merchandisers, category managers, and retail planners who need data-driven insights to make informed decisions about product assortments.

Q: What business problems does ap help solve? **A:** The ap algorithm helps address critical retail challenges such as: * Minimizing lost sales due to stockouts. * Reducing capital tied up in excess inventory. * Optimizing product selection for specific stores or categories. * Ensuring the right **Optimum Depth (OD)** and **Optimum Width (OW)** for your assortment.

Q: How does ap improve inventory management? **A:** By providing recommendations for **Optimum Depth (OD)** (ideal number of unique items), **Optimum Width (OW)** (ideal number of categories), and the **Ideal Size Set (ISS)**, ap ensures that inventory levels are aligned with forecasted **Rate of Sale (ROS)** and customer demand, leading to more efficient stock turns.

Q: What does “Assortment Plan” mean in the context of this algorithm? **A:** An “Assortment Plan” generated by ap is a strategic blueprint that details which products (SKUs), in what quantities, and in which sizes (ISS) should be stocked in a given category or display. It includes recommendations for the **Optimum Depth (OD)** and **Optimum Width (OW)** based on various business factors.

Configuration & Settings

Q: How can I customize the ap algorithm’s behavior? **A:** The ap algorithm’s behavior can be customized through specific configuration settings. These settings allow you to fine-tune how it calculates recommendations based on your business priorities. Refer to the system’s `args.properties` file or the main documentation for available parameters.

Q: Are there settings to prioritize certain product categories or specific business objectives? **A:** Yes, the algorithm's configuration settings can be adjusted to prioritize certain categories, allocate space differently, or emphasize specific business objectives (e.g., maximizing profit vs. minimizing stockouts). These adjustments typically influence the calculation of **Optimum Depth (OD)** and **Optimum Width (OW)**.

Q: Can I define specific thresholds or targets for **Rate of Sale (ROS)**? **A:** While the algorithm primarily *calculates* **Rate of Sale (ROS)**, its configuration might allow you to set thresholds for inclusion criteria or to influence how **ROS** impacts the recommended **Optimum Depth (OD)** or **Ideal Size Set (ISS)**. Check the configuration documentation for specific options.

Q: Where can I find the current configuration settings being used by the ap algorithm? **A:** The current configuration settings are typically stored in the `args.properties` file or similar configuration management system. For a detailed breakdown, please consult the "Configuration Guide" section of the main ap documentation.

Q: What if I need to request a change to a configuration setting? **A:** Any changes to configuration settings should be requested through the standard change management process. Please contact the support team or your assigned system administrator to discuss potential impacts and schedule modifications.

Data & Inputs

Q: What kind of data does the ap algorithm need to generate an Assortment Plan?

A: The ap algorithm requires various types of data, including historical sales data (critical for **Rate of Sale (ROS)** calculations), current inventory levels, product master data (including **Attribute Groups (AG)** like color, brand, material), store characteristics, and potentially space constraints.

Q: How often is the input data updated for the ap algorithm? **A:** The frequency of input data updates depends on the system's setup, but typically, data is updated daily or weekly to ensure the most current sales trends and inventory positions are considered for **Rate of Sale (ROS)** and **Optimum Depth (OD)** calculations.

Q: What happens if there's missing or incomplete input data? **A:** Missing or incomplete data can affect the accuracy of the Assortment Plan recommendations. The ap algorithm may attempt to use imputation methods or fall back to aggregated data, but it's crucial to ensure data quality for reliable **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)** outputs. Data quality reports are usually available.

Q: How are product attributes (**AG**) used in the Assortment Plan? **A:** Product **Attribute Groups (AG)** (e.g., color, size, brand, material) are crucial for the algorithm to understand product characteristics and customer preferences. They are used to group similar items, analyze performance within those groups, and generate more precise recommendations for **Optimum Depth (OD)** and **Ideal Size Set (ISS)**.

Q: Does the algorithm specifically use past **Rate of Sale (ROS)** data? **A:** Yes, historical **Rate of Sale (ROS)** data is a fundamental input for the ap algorithm. It uses past sales velocity to project future demand, which directly influences the recommended **Optimum Depth (OD)**, **Optimum Width (OW)**, and the quantity breakdown within the **Ideal Size Set (ISS)**.

Process & Logic

Q: What is the role of the ApOutputGroupModule in the ap algorithm? **A:** The ApOutputGroupModule acts as the orchestrator for the entire Assortment Plan (AP) output generation process. It defines a sequential pipeline, managing everything from capturing initial data snapshots to executing the core AP calculations and then synchronizing the final output. It ensures a proper flow of data and business logic.

Q: How does the algorithm determine the **Optimum Depth (OD)** and **Optimum Width (OW)** for a category? **A:** The algorithm determines **Optimum Depth (OD)** and **Optimum Width (OW)** by analyzing historical sales, **Rate of Sale (ROS)** trends, customer preferences, available space, and business constraints. It aims to balance variety with efficiency, ensuring each item stocked contributes optimally to sales and profitability.

Q: How does the algorithm calculate the **Ideal Size Set (ISS)** for products? **A:** The **Ideal Size Set (ISS)** is calculated by analyzing historical sales data at the size level, considering the **Rate of Sale (ROS)** for each size, and identifying patterns in customer demand across different sizes. This helps ensure the right proportion of each size is stocked to meet customer needs.

Q: What is the typical processing time for generating an Assortment Plan? **A:** The processing time for an Assortment Plan varies depending on the volume and complexity of the input data. Smaller datasets might process quickly, while larger, more granular analyses for an entire enterprise could take longer. The ApOutputGroupModule is designed for efficient execution.

Q: Can I see the steps or stages the ap algorithm takes during its run? **A:** At a high level, the ApOutputGroupModule follows a sequence of operations: first, it captures data snapshots; second, it runs the core AP output calculation; and third, it syncs the generated output. While not directly visible to end-users, this modular approach ensures data integrity and flow.

Outputs & Results

Q: What kind of outputs does the ap algorithm provide? **A:** The ap algorithm generates comprehensive Assortment Plans that include recommendations for the **Optimum Depth (OD)** (how many unique items), **Optimum Width (OW)** (how many categories/styles), **Ideal Size Set (ISS)** for each product, and projected stock levels based on **Rate of Sale (ROS)**.

Q: How are the results of the ap algorithm presented to users? **A:** The results are typically presented in actionable formats such as detailed reports, interactive dashboards, or integrated directly into existing merchandising and inventory management systems, allowing merchandisers to easily review and implement the recommendations.

Q: Can I get Assortment Plan recommendations for specific stores or regions? **A:** Yes, the ap algorithm is capable of generating highly granular recommendations. You can typically configure it to produce distinct Assortment Plans, including **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)**, tailored to specific store clusters, regions, or individual locations.

Q: How should I interpret the recommended **Rate of Sale (ROS)**? **A:** The recommended **Rate of Sale (ROS)** is a projected average sales velocity for a product or category over a specific period (e.g., units per week). It serves as a key indicator for inventory planning, helping determine how many units are needed to meet demand and support the recommended **Optimum Depth (OD)** and **Ideal Size Set (ISS)**.

Q: Where can I access the generated Assortment Plans and associated data? **A:** The generated Assortment Plans and underlying data are usually accessible through a dedicated reporting dashboard, a business intelligence (BI) tool, or directly within your enterprise resource planning (ERP) system, depending on the integration setup.

Error Handling & Edge Cases

Q: What happens if the ap algorithm encounters an error during processing? **A:** If the ap algorithm encounters an error, it typically logs the issue, and depending on the severity, may notify the system administrators or support team. While the `ApOutputGroupModule` is designed for robust execution, critical errors might halt processing, requiring intervention.

Q: How does ap handle products with very low or no sales history? **A:** For products with very low or no sales history, the algorithm may use different strategies: it might rely on **Attribute Group (AG)** data (e.g., similar products, category averages) as proxies, apply a default **Rate of Sale (ROS)**, or flag these items for manual review to determine their **Optimum Depth (OD)** and **Ideal Size Set (ISS)**.

Q: What if there are significant promotions or events that drastically impact **Rate of Sale (ROS)**? **A:** Sudden, significant changes in **Rate of Sale (ROS)** due to promotions or events might require a manual override or specific configuration adjustments. While the algorithm learns from historical patterns, extreme short-term shifts might need additional data inputs or a re-tuning of parameters to accurately forecast future **Optimum Depth (OD)** and **Ideal Size Set (ISS)**.

Q: Does the algorithm account for new product introductions without any historical data? **A:** For new product introductions, the algorithm typically relies on initial seeding data provided by merchandisers, comparisons to similar products (based on **Attribute Groups (AG)**), or category-level averages to establish an initial forecasted **Rate of Sale (ROS)** and an estimated **Optimum Depth (OD)** and **Ideal Size Set (ISS)**. As sales history accrues, recommendations become more precise.

Q: What should I do if the recommendations for **Optimum Depth (OD)** or **Ideal Size Set (ISS)** seem incorrect or unexpected? **A:** If recommendations seem incorrect, first verify the input data for accuracy and completeness. Then, review the configuration settings to ensure they align with current business strategies. If the issue persists, gather relevant details (e.g., specific product, time frame, unexpected recommendation) and contact the support team for further investigation.

Need More Help?

Please contact the support team at [support_email@yourcompany.com] or [support_phone_number].

AP Documentation

- Version: 1.0
 - Last Updated: 2025-05-23
-

2. Executive Summary

This documentation details the Assortment Plan (AP) output generation algorithm, designed to provide accurate and timely insights for merchandise planning. The core `ApOutputGroupModule` orchestrates the entire process, from data collection to calculating key metrics like Optimum Depth (OD), Optimum Width (OW), Ideal Size Set (ISS), and Rate of Sale (ROS). It streamlines the generation of data-driven AP recommendations, enabling businesses to optimize inventory, minimize stockouts, and enhance sales performance.

3. Architecture Overview

The `ApOutputGroupModule` acts as the central orchestrator, managing a sequential pipeline for Assortment Plan (AP) output generation. It controls the entire flow, ensuring proper data preparation, calculation, and synchronization across its internal stages.

```
graph TD
    subgraph ApOutputGroupModule_Orchestration_Flow [ApOutputGroupModule Orchestration Flow]
        start((Start)) --> capture(Capture Input Data Snapshot);
        capture --> calculate(Run Core AP Calculation);
        calculate --> upload(Sync Generated Output);
        upload --> end((End));
    end
```

Module Descriptions:

- `ApOutputGroupModule`: Orchestrates the entire process of generating Assortment Plan outputs, ensuring data consistency and timely availability.
- `UtilOutputSnapshotModule` (used in 'Capture Input Data Snapshot'): Gathers and prepares all necessary input data from external systems.
- `OdApOutputModule` (used in 'Run Core AP Calculation'): Applies the central business logic to compute key metrics such as OD (Optimum Depth), OW (Optimum Width), ISS (Ideal Size Set), and ROS (Rate of Sale).
- `UtilOutputSyncModule` (used in 'Sync Generated Output'): Persists the generated Assortment Plan outputs to databases or other downstream systems.

4. Prerequisites & Setup

Specific prerequisites like Java version, build tools, and dependencies are not provided in the current input.

To run the `ApOutputGroupModule` orchestrator, execute the main application entry point (e.g., `java -jar ap.jar` or `mvn spring-boot:run` for a Java project), which will trigger the defined processing pipeline.

5. Folder & File Structure

```
ap/
├─ module-group/          # Contains the orchestrator module
(ApOutputGroupModule).
├─ submodules/            # Houses individual processing modules
(e.g., UtilOutputSnapshotModule, OdApOutputModule,
UtilOutputSyncModule).
├─ args.properties        # Global configuration parameters for the
application.
├─ common-data.json        # Defines the shared data model used
across modules.
├─ outputs/               # Stores generated output files and
reports.
```

6. Global Configuration

No global configuration parameters (`args.properties`) were provided in the input.

7. Shared Data Model

No shared data model (`common-data.json`) schema was provided in the input.

8. Submodule Chapters

ApOutputGroupModule

1. Module Name & Business Objective

- **Module Name:** ApOutputGroupModule
- **Business Objective:** The ApOutputGroupModule orchestrates the end-to-end process of generating Assortment Plan (AP) outputs. Its primary goal is to ensure the accurate calculation of OD, OW, ISS, and ROS, and the timely synchronization of these results for effective business decision-making. Its key business value is to automate and standardize the AP output generation, leading to consistent data for merchandising and supply chain optimization.

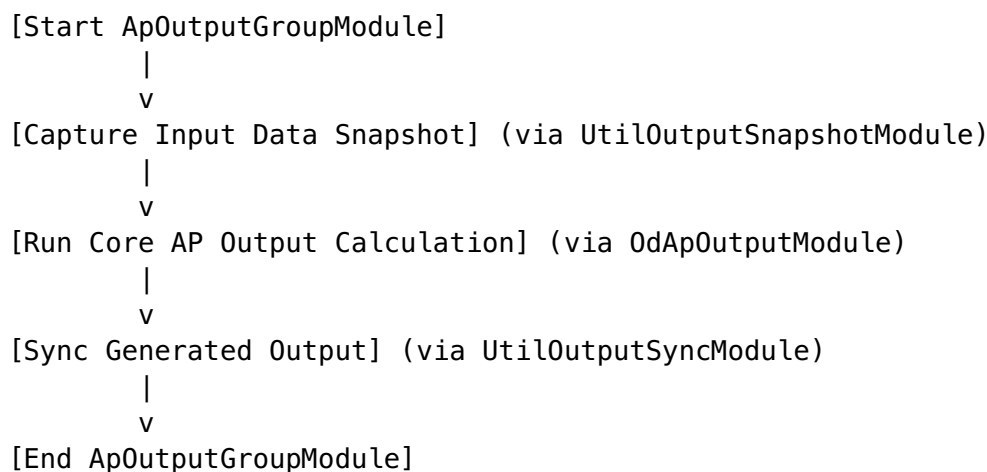
2. Required Information

- **Sales Data:** *Required for calculating ROS and informing OD, OW, ISS. Includes historical sales figures, units sold, and revenue for various products over time.*
- **Product Data:** *Includes product attributes such as category, style, color, and size, needed for AG (Attribute Group) creation and segmenting data for OD/OW/ISS calculations.*
- **Operational Data:** *Configuration settings, system parameters (e.g., lead times, inventory levels), and output synchronization details necessary for module execution and data persistence.*

3. Business Settings

Specific business settings relevant to this orchestration module were not provided in the input. Typically, these would control the activation of its internal steps, define input data sources, or specify output destinations and formats.

4. Process Flow



5. Step-by-Step Actions

- **Initialize:** *The ApOutputGroupModule is **activated** to begin the Assortment Plan (AP) generation process, preparing its internal components for execution.*
- **Capture Snapshot:** *It first **gathers** all necessary input data, such as sales history, current inventory levels, product details, and store information from various sources. This ensures that the subsequent calculations are based on a consistent and up-to-date dataset.*
- **Calculate AP Outputs:** *Next, it **executes** the core AP calculations. This involves determining OD (Optimum Depth), OW (Optimum Width), ISS (Ideal Size Set), and ROS (Rate of Sale) based on the collected snapshot data and predefined business rules and algorithms.*
- **Sync Output:** *Finally, it **persists** the computed AP outputs. This step ensures the generated OD, OW, ISS, and ROS recommendations are available in the target database or reporting systems for business users and downstream applications.*
- **Complete Process:** *The module **confirms** successful completion of the entire AP output generation cycle, signaling that the updated data is ready for consumption.*

6. Sample Scenario

A category manager needs to refresh AP recommendations for the upcoming product refresh cycle to ensure optimal stock levels.

The ApOutputGroupModule is **triggered** automatically overnight as part of the daily data processing pipeline. It **collects** sales data for all 100 SKUs within a specific product category across 5 key stores for the past 12 weeks, along with current inventory levels and product attributes like color and material. It then **calculates** the OD, OW, ISS, and ROS for each AG (e.g., 'Women's Jeans - Skinny Fit - Blue - Size 28'). For example, for 'Women's Jeans - Skinny Fit - Blue - Size 28', it determines an OD of 4 units per store, an ISS recommending sizes 26, 28, 30, 32 with specific recommended quantities, and a ROS of 7 units/week per store. The calculated outputs are then **synced** to the central merchandising system, making them immediately available for the category manager's review the next morning.

Key Takeaway: *The module automates the generation of precise, data-driven AP recommendations, enabling rapid and informed inventory decisions that directly impact sales and stock efficiency.*

7. Results

The primary results are the calculated OD (Optimum Depth), OW (Optimum Width), ISS (Ideal Size Set), and ROS (Rate of Sale) values for each relevant AG (Attribute Group) or product. These results are then available in the designated output systems (e.g., database tables, flat files) for consumption by other business processes, reporting tools, and merchandising systems. This ensures data consistency and timeliness for downstream systems reliant on Assortment Plan outputs.

8. Unusual Cases

- **Missing or delayed data feed:** *If critical input data (e.g., sales history, product master data) is missing or delayed, the module may run with incomplete*

information, leading to inaccurate AP calculations, or it might fail if essential data is absent. Implement robust data validation and alerting mechanisms to mitigate this risk.

- **Calculation errors in sub-modules:** Errors occurring within specific calculation sub-modules like `OdApOutputModule` (e.g., due to unexpected data patterns, invalid input, or algorithmic issues) can lead to incorrect OD/OW/ISS/R OS values. Ensure comprehensive logging, error handling, and data quality checks for all internal sub-processes.
- **Output sync failures:** If the `UtilOutputSyncModule` fails to persist the calculated AP outputs (e.g., due to database connection issues, permissions, or schema mismatches), the valuable results will not be available to business users. Implement retry mechanisms, detailed error logs, and immediate notifications for such failures.

9. Next Steps

- **Review Outputs:** Business users should regularly **review** the generated AP outputs for accuracy, alignment with business strategy, and to identify any anomalies. This feedback loop is crucial for continuous improvement.
- **Integrate with Planning Tools:** The AP outputs should be seamlessly **integrated** into existing merchandise planning, inventory management, and purchasing systems to directly drive purchasing and allocation decisions.
- **Performance Monitoring:** Regularly **monitor** the execution of the `ApOutputGroupModule` for its overall performance, data quality, and successful completion rates. This ensures the process remains efficient and reliable.

10. Common Questions

1. **How often does the AP output get refreshed?** Typically, the module runs on a predefined schedule (e.g., daily, weekly) to provide the latest AP recommendations based on recent sales and inventory data.
2. **Can we run the AP calculation for a specific category only?** While the orchestrator handles the full flow, the underlying `OdApOutputModule` might support filtering by AGs or categories. This usually requires specific configuration adjustments before execution.
3. **What happens if a data source is unavailable?** The module is designed to either **fail gracefully** with a clear error message, **use the last known good data** if configured, or **skip the affected calculations** depending on the severity and specific design. This behavior should be clearly communicated in logs.
4. **Where can I find the results?** The AP outputs are typically stored in a designated central database table or data lake, accessible via business intelligence tools, reporting dashboards, or integrated directly into AP planning applications.
5. **How does this module help reduce stockouts?** By providing accurate OD (Optimum Depth), OW (Optimum Width), and ISS (Ideal Size Set) recommendations tailored to product performance and demand, it helps ensure the right products are stocked in the right quantities and sizes, thus minimizing stockouts and maximizing sales opportunities.

11. Quick Reference

- **Purpose:** Automate the end-to-end generation of Assortment Plan (AP) outputs.
 - **Key Outputs:** OD (Optimum Depth), OW (Optimum Width), ISS (Ideal Size Set), ROS (Rate of Sale).
 - **Primary Data Dependencies:** Sales Data, Product Data, Operational Data.
 - **Orchestrates:** UtilOutputSnapshotModule, OdApOutputModule, UtilOutputSyncModule.
 - **Run Frequency:** Configurable (e.g., daily, weekly).
-

9. Appendix

Full Parameter Reference

A detailed list of all configurable parameters is typically found in the `args.properties` table in Section 6. As no specific configuration parameters were provided in the input, this section is currently empty.

Version History

- **V1.0 - 2025-05-23:** Initial release of AP Documentation.

Contact & Support Info

For questions, feature requests, or technical support related to the AP output generation algorithm, please contact the Product Analytics Team or refer to the internal support channels available within your organization.

Tutorial: ap

The `ApOutputGroupModule` (Index 0) acts as an orchestrator, defining and executing a sequential pipeline for Assortment Plan (AP) output generation. It controls the entire flow, from data snapshotting to core AP calculations and final output synchronization, ensuring proper data flow and business logic execution across its internal stages.

flowchart TD

```
A0["ApOutputGroupModule"]
A0 -- "control_flow" --> A0
A0 -- "control_flow" --> A0
A0 -- "control_flow" --> A0
```

Chapters

Master Report: Merchandising Assortment Planning (ap)

1. Title & Metadata

- **Project Name:** ap
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** The ap merchandising algorithm optimizes product assortment by determining ideal depth, width, and size sets to maximize sales efficiency and inventory productivity.

2. Executive Summary

The ap algorithm is a critical component for modern retail merchandising, designed to dynamically optimize product assortments at various levels (category, store, display). It leverages historical sales data, inventory levels, and product attributes to calculate the **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)**. By aligning inventory with projected demand and sales velocities (**Rate of Sale (ROS)**), the algorithm aims to significantly enhance sales performance, reduce stockouts, and minimize excess inventory.

- Key outcomes or KPIs:
 - Increased **ROS** across product categories.
 - Optimized inventory efficiency, reducing holding costs and improving sell-through.
 - Enhanced customer satisfaction due to improved product availability and relevant assortment.
 - Reduction in lost sales opportunities due to stockouts.
 - Improved space utilization within stores and on digital displays.
- Primary stakeholders or downstream consumers: Merchandising Managers, Category Planners, Inventory Management Teams, Store Operations, E-commerce Teams.

3. Architecture & Data Flow

graph TD

```
A[Raw Sales Data] --> B(Data Ingestion & Validation);
B --> C{Product Master Data};
C --> D[Inventory Data];
D --> E[Historical Performance];
E --> F(Data Pre-processing);
```

```

F --> G[ROS Calculation];
G --> H[OD & OW Optimization];
H --> I[ISS Determination];
I --> J[Output Generation & Reporting];
J --> K[Merchandising Recommendations];
J --> L[Performance Analytics];
K --> M[Merchandising Teams];
L --> M;

```

- **Component Bullets:**

- **Data Ingestion & Validation:** Gathers raw data from various sources and ensures data quality and consistency.
- **Data Pre-processing:** Cleans, transforms, and aggregates data for analysis, creating a unified dataset.
- **ROS Calculation:** Computes the Rate of Sale for individual SKUs and Attribute Groups (AGs).
- **OD & OW Optimization:** Determines the ideal number of unique items (OD) and categories/styles (OW) based on demand patterns, space constraints, and profitability.
- **ISS Determination:** Recommends the optimal combination of sizes for each product to maximize sales and minimize stockouts.
- **Output Generation & Reporting:** Formats the calculated recommendations into actionable reports and datasets for downstream consumption.

- **High-level data sources:**

- Point-of-Sale (POS) transaction data (sales units, value, date, store).
- Inventory Management System (IMS) data (stock-on-hand, inbound shipments, transfers).
- Product Information Management (PIM) data (SKU attributes: category, sub-category, brand, color, size, material, etc.).
- Store/Display capacity data.
- Historical promotional calendars and pricing data.

4. Configuration & Data Dictionary

Settings Table (args.properties)

Parameter Name	Description	Default Value	Data Type
min_ros_threshold	Minimum acceptable Rate of Sale for a SKU to be considered viable.	0.1 units/week	Float
max_sku_per_category_od	Maximum allowed OD for a given category (hard cap).	100	Integer
max_category_ow	Maximum allowed OW for a store/display (hard cap).	20	Integer

Parameter Name	Description	Default Value	Data Type
iss_lookback_weeks	Number of historical weeks to consider for ISS calculation.	12	Integer
inventory_turn_target	Desired inventory turns target for OD/OW calculations.	4.0	Float
size_cluster_method	Method for grouping sizes for ISS (e.g., 'sales_percent', 'historical_fit').	sales_percent	String
output_format	Format for output files (e.g., 'csv', 'json').	csv	String
data_refresh_frequency_days	How often the source data is expected to be refreshed.	7	Integer

Shared Data Fields (common-data.json)

Field Name	Description	Data Type	Example Value
product_id	Unique identifier for a product SKU.	String	PROD123456
category_id	Identifier for the product category.	String	APPAREL_TOPS
subcategory_id	Identifier for the product sub-category.	String	T-SHIRTS
store_id	Unique identifier for the retail location or online storefront.	String	STORE001
transaction_date	Date of sale transaction.	Date	2025-05-20
sales_units	Number of units sold.	Integer	5
sales_revenue	Revenue generated from sale.	Decimal	50.00
stock_on_hand	Current inventory count for a SKU at a location.	Integer	15
size_value	The specific size of the product (e.g., 'S', 'M', 'L', '32x32').	String	L
color_value		String	Blue

Field Name	Description	Data Type	Example Value
brand_name	The specific color of the product.	String	Acme Apparel
	Brand of the product.		
attribute_group_id	Identifier for an Attribute Group (AG) (e.g., ‘T-Shirt-Blue’).	String	TSHIRT_BLUE_M

Outputs Catalog

Output File/Table Name	Description	Key Fields	Format
od_recommendations.csv	Recommended Optimum Depth (OD) per category/store.	category_id, store_id, recommended_od_skus	CSV
ow_recommendations.csv	Recommended Optimum Width (OW) per store/display.	store_id, recommended_ow_categories	CSV
iss_recommendations.csv	Recommended Ideal Size Set (ISS) breakdown per product/category.	product_id, category_id, size_value, iss_percent	CSV
ros_metrics.csv	Calculated Rate of Sale (ROS) for all SKUs and AGs.	product_id, store_id, weekly_ros	CSV
inventory_forecast.csv	Short-term inventory forecast based on ROS and recommendations.	product_id, store_id, forecast_units	CSV
performance_summary.json	High-level KPIs and algorithm run metrics.	run_date, total_ros_increase_pct, inventory_turn_impact_pct	JSON

5. End-to-End Flow

The ap algorithm operates through a structured pipeline to deliver actionable merchandising recommendations.

- Data Ingestion:** The process begins by ingesting raw sales transactions, current inventory levels, and comprehensive product master data from various source systems.

2. **Data Validation & Pre-processing:** Ingested data undergoes rigorous validation for completeness and consistency. It's then cleaned, aggregated, and transformed into a standardized format suitable for analysis. This step includes linking sales to specific SKUs, categories, and **Attribute Groups (AGs)**.
3. **Rate of Sale (ROS) Calculation:** Using the pre-processed sales history, the algorithm calculates the **Rate of Sale (ROS)** for every unique SKU and relevant **Attribute Group (AG)** across different stores and time periods. This provides a measure of product velocity.
4. **Optimum Depth (OD) & Optimum Width (OW) Optimization:**
 - For **OW**, the algorithm analyzes category performance (ROS, profitability, space efficiency) and store/display capacity to recommend the optimal mix and number of categories or styles to offer.
 - For **OD**, within each recommended category, it identifies the ideal number of unique SKUs to stock, prioritizing based on individual SKU ROS, margin, and alignment with overall category strategy, while considering display limitations.
5. **Ideal Size Set (ISS) Determination:** For each product or product **Attribute Group (AG)**, the algorithm determines the **Ideal Size Set (ISS)** by analyzing historical sales distribution across sizes and accounting for target stock levels and lead times. This ensures that the most popular sizes are adequately stocked, minimizing stockouts and excess.
6. **Recommendation Generation:** The calculated **OD**, **OW**, and **ISS** values are compiled into structured recommendation files, specifying which SKUs and categories to stock, and in what size proportions, for each store or display.
7. **Performance Analytics & Reporting:** Alongside recommendations, the algorithm generates metrics summarizing its expected impact on **ROS**, inventory turns, and potential sales uplift, providing valuable insights for merchandising teams. These outputs are then made available to downstream systems and stakeholders.

6. Submodule Overview

6.1. Data Ingestion & Validation

1. **Name & Purpose:** The Data Ingestion & Validation submodule is responsible for collecting raw data from various source systems and ensuring its quality and consistency before further processing.
2. **Inputs & Outputs:**
 - Inputs: Raw sales logs, inventory snapshots, product master flat files.
 - Outputs: Clean, validated, and normalized data tables for sales, inventory, and product attributes.
3. **Core Logic Summary:**
 - Connects to defined data sources (e.g., databases, flat files, APIs).
 - Performs schema validation and data type checks.
 - Identifies and flags missing or erroneous records (e.g., negative sales, non-existent product IDs).
 - Standardizes key identifiers (e.g., `product_id`, `store_id`).

6.2. ROS Calculation

1. **Name & Purpose:** The ROS Calculation submodule computes the Rate of Sale for individual SKUs and aggregated **Attribute Groups (AGs)**, providing a key metric for product velocity.
2. **Inputs & Outputs:**
 - Inputs: Validated sales data, product master data.
 - Outputs: `ros_metrics.csv` (containing `product_id`, `store_id`, `weekly_ros`, `ag_id`, `ag_weekly_ros`).
3. **Core Logic Summary:**
 - Aggregates sales units by `product_id`, `store_id`, and week.
 - Divides total units sold by the number of active weeks (or days, then scaled to weeks) to get weekly **ROS**.
 - Calculates **ROS** for defined **Attribute Groups (AGs)** by summing SKU sales within each group.
 - Filters out products with insufficient sales history or below `min_ros_threshold`.

6.3. OD & OW Optimization

1. **Name & Purpose:** The OD & OW Optimization submodule determines the ideal number of unique items (**Optimum Depth**) and categories/styles (**Optimum Width**) based on demand, performance, and capacity.
2. **Inputs & Outputs:**
 - Inputs: `ros_metrics.csv`, product master data, store/display capacity data, `args.properties`.
 - Outputs: `od_recommendations.csv`, `ow_recommendations.csv`.
3. **Core Logic Summary:**
 - Ranks categories/styles by aggregated **ROS** and margin contribution for **OW** consideration, prioritizing based on `max_category_ow` and store capacity.
 - Within each selected category, ranks SKUs by individual **ROS** and margin to identify candidates for **OD**, applying `max_sku_per_category_od`.
 - Considers product lifecycle stages (new vs. mature) and seasonal trends.
 - Balances breadth (OW) vs. depth (OD) based on strategic objectives (e.g., 'destination' categories vs. 'convenience' categories).

6.4. ISS Determination

1. **Name & Purpose:** The ISS Determination submodule calculates the Ideal Size Set for products, ensuring optimal size distribution to meet customer demand and minimize stock imbalances.
2. **Inputs & Outputs:**
 - Inputs: Validated sales data (including size breakdown), `args.properties`.
 - Outputs: `iss_recommendations.csv`.
3. **Core Logic Summary:**
 - Analyzes historical sales data over `iss_lookback_weeks` to determine the percentage distribution of sales by size for each product or **Attribute Group (AG)**.
 - Identifies the most popular sizes and their sales contribution using the `size_cluster_method`.
 - Recommends the optimal stock ratio for each size based on its historical demand proportion and target inventory levels (derived from **ROS**).

- Adjusts for potential outliers or promotional spikes in historical size sales.

6.5. Reporting & Export

1. **Name & Purpose:** The Reporting & Export submodule formats the final recommendations and performance metrics into user-friendly reports and structured files for consumption by stakeholders and other systems.
2. **Inputs & Outputs:**
 - Inputs: `od_recommendations.csv`, `ow_recommendations.csv`, `iss_recommendations.csv`, `ros_metrics.csv`, intermediate calculation results.
 - Outputs: `od_recommendations.csv`, `ow_recommendations.csv`, `iss_recommendations.csv`, `performance_summary.json`, and potentially database updates.
3. **Core Logic Summary:**
 - Aggregates final recommendations from previous modules.
 - Generates summary statistics and KPIs for the run (e.g., total recommended SKUs, average **ROS** change).
 - Formats outputs according to `output_format` (e.g., CSV for data consumption, JSON for dashboards).
 - Publishes or saves the output files to a designated location for access by merchandising teams and downstream systems.

7. Worked Example

Scenario: Optimizing OD for “Men’s Casual T-Shirts” category in ‘STORE001’.

Inputs: - **ROS Metrics:** - SKU-TS001-Blue-S: 2.5 units/week - SKU-TS001-Blue-M: 4.0 units/week - SKU-TS001-Blue-L: 3.5 units/week - SKU-TS002-Red-M: 1.8 units/week - SKU-TS003-Green-XL: 0.7 units/week - SKU-TS004-Black-L: 3.0 units/week - **Product Master:** - SKU-TS001: T-Shirt (Basic), Color: Blue, Sizes: S, M, L - SKU-TS002: T-Shirt (Graphic), Color: Red, Sizes: M, L - SKU-TS003: T-Shirt (V-Neck), Color: Green, Sizes: L, XL - SKU-TS004: T-Shirt (Basic), Color: Black, Sizes: S, M, L, XL - **Configuration (args.properties):** - `min_ros_threshold`: 0.5 units/week - `max_sku_per_category_od`: 5 (for ‘Men’s Casual T-Shirts’)

Steps:

1. **Filter by `min_ros_threshold`:** All SKUs are above 0.5 units/week, so all are considered.
2. **Aggregate ROS by unique product (`product_id`) for initial ranking:**
 - SKU-TS001 (sum of its sizes’ ROS): $2.5 + 4.0 + 3.5 = 10.0$ units/week
 - SKU-TS002: 1.8 units/week
 - SKU-TS003: 0.7 units/week
 - SKU-TS004: 3.0 units/week
3. **Rank unique products by aggregated ROS (highest first):**
 1. SKU-TS001 (10.0)
 2. SKU-TS004 (3.0)
 3. SKU-TS002 (1.8)
 4. SKU-TS003 (0.7)

4. **Apply max_sku_per_category_od (5 in this case):** Since there are only 4 unique products in the category, all 4 are recommended for OD. If there were 7 products, only the top 5 would be selected.

Output (simplified od_recommendations.csv entry):

category_id	store_id	recommended_od_skus
MEN_CASUAL_T_SHIRTS	STORE001	SKU-TS001,SKU-TS004,SKU-TS002,SKU-TS003

Further ISS Determination for SKU-TS001-Blue (assuming historical sales): - Historical sales: S: 25%, M: 45%, L: 30% - If recommended to stock 10 units of SKU-TS001, ISS would be: S: 3 units, M: 5 units, L: 2 units.

8. Debug Guide Highlights

- **Data Inconsistencies:**
 - *Symptom:* Unexpectedly low **ROS**, missing categories, or skewed ISS.
 - *Check:* Raw input data for missing product_ids, store_ids, negative sales units, or incorrect date formats. Verify joins between sales and product master data.
- **Configuration Errors:**
 - *Symptom:* Recommendations are too aggressive (e.g., too many SKUs recommended) or too conservative (e.g., too few SKUs).
 - *Check:* args.properties for misconfigured min_ros_threshold, max_sku_per_category_od, max_category_ow, or iss_lookback_weeks.
- **Performance Issues:**
 - *Symptom:* Algorithm run takes excessively long or crashes.
 - *Check:* Data volume. Ensure proper indexing on database tables. Review complex joins or iterative calculations in core logic for optimization. Consider sampling for initial runs on large datasets.
- **Output Mismatches:**
 - *Symptom:* Output file formats are incorrect, or data doesn't align with expectations.
 - *Check:* The output_format setting. Review the output schema and ensure all required fields are populated correctly. Check for data truncation issues.
- **Zero/Negative ROS:**
 - *Symptom:* Products with sales are showing zero or negative **ROS**.
 - *Check:* Division by zero errors, or incorrect handling of products with very sparse sales history. Ensure the time period used for calculation is valid.

9. Common Pitfalls & Remedies

1. Pitfall: Stale or Inaccurate Input Data.

- *Impact:* Recommendations are based on outdated or incorrect information, leading to suboptimal assortments.
- *Remedy:* Implement robust data validation checks at ingestion. Establish clear data refresh schedules and alerts for data pipeline failures. Ensure data_refresh_frequency_days matches actual refresh rates.

2. **Pitfall: Over-reliance on Historical ROS for New Products.**
 - *Impact:* New products with no sales history cannot be accurately assessed, potentially leading to exclusion or incorrect sizing.
 - *Remedy:* Implement a fallback mechanism for new products (e.g., use **ROS** from similar **Attribute Groups (AGs)**, assign a default initial **ROS** based on category average, or incorporate manual overrides for launch phases).
3. **Pitfall: Ignoring Seasonality and Trends.**
 - *Impact:* Recommendations do not account for predictable demand fluctuations (e.g., holiday spikes, seasonal apparel), leading to stockouts or overstock.
 - *Remedy:* Incorporate time-series analysis and seasonal decomposition into **ROS** calculation. Allow for seasonal adjustments to `iss_lookback_weeks` or segment data by season.
4. **Pitfall: Local Store/Display Nuances Not Captured.**
 - *Impact:* A “one-size-fits-all” approach to **OD**, **OW**, and **ISS** misses unique local demand patterns.
 - *Remedy:* Ensure `store_id` is a key dimension throughout calculations. Tailor `max_sku_per_category_od` and `max_category_ow` settings per store or store cluster.
5. **Pitfall: Lack of Feedback Loop.**
 - *Impact:* The algorithm’s effectiveness isn’t measured or improved over time.
 - *Remedy:* Regularly compare projected **ROS** and inventory efficiency with actual outcomes. Track KPIs generated in `performance_summary.json` over time. Implement a A/B testing framework for recommendations where possible.

10. FAQ

1. **Q: How often should the ap algorithm be run?** A: The recommended frequency depends on the volatility of your sales data and the speed of inventory turns. Typically, it should be run weekly or bi-weekly. Refer to the `data_refresh_frequency_days` setting in `args.properties`.
2. **Q: What if a product has zero sales in the lookback period?** A: Products with zero sales will either be excluded if they don’t meet the `min_ros_threshold` or will be assigned a very low **ROS**. For genuinely new products, consider using a default **ROS** or a manual override until sufficient sales data accumulates.
3. **Q: Can I override the algorithm’s recommendations?** A: Yes, the output files are designed to be consumed by merchandising teams, who can apply business judgment and manual adjustments before implementation. The algorithm provides a data-driven baseline.
4. **Q: How does the algorithm handle promotions?** A: Promotions can artificially inflate **ROS**. The current version primarily uses raw sales. Future versions may incorporate logic to de-trend promotional lifts or use a ‘baseline sales’ calculation, or you can adjust `iss_lookback_weeks` to exclude known promotional periods.
5. **Q: What is an Attribute Group (AG)?** A: An **Attribute Group (AG)** is a collection of products that share common attributes, such as “Men’s Basic Blue T-Shirts” (same style, color, but different sizes). Analyzing **ROS** at the **AG** level helps in understanding broader demand patterns beyond individual SKUs.

11. Quick Reference Cheat-Sheet

Term / Abbreviation	Definition
OD	Optimum Depth: Ideal number of unique items (SKUs) to stock in a category.
OW	Optimum Width: Ideal number of different categories or styles to offer.
ISS	Ideal Size Set: Best combination of sizes to maximize sales & minimize stockouts.
ROS	Rate of Sale: Speed at which products are sold (e.g., units/week).
AG	Attribute Group: Set of product attributes grouped for analysis (e.g., color, size, brand).

Key Files & Locations:

- **Configuration:** config/args.properties
- **Common Data Fields:** schema/common-data.json
- **Input Data Directory:** data/raw_inputs/
- **Output Data Directory:** data/outputs/
- **Main Algorithm Script:** src/main_ap.py (or similar)

Common Commands (Example):

- `python src/main_ap.py --run_mode full --config_path config/args.properties`
- `python src/main_ap.py --run_mode ros_only --debug`
- `tail -f logs/ap_runtime.log` (to monitor real-time logs)

Debugging Tip: Always check logs/ap_errors.log first for execution failures.

This guide is designed to help users and developers understand and troubleshoot the outputs of our merchandising optimization algorithms. It details the structure, purpose, and lineage of various data files generated during the merchandising recommendation process.

Debug Guide: Merchandising Algorithm Outputs

1. Introduction

The Merchandising Optimization Engine generates a suite of data outputs crucial for informed decision-making regarding product assortment, depth, and sizing. This Debug Guide serves as a comprehensive reference for understanding these outputs, identifying potential discrepancies, and tracing data lineage from raw inputs to final recommendations.

Understanding the various output files is key to effective debugging:

- **Internal Outputs (`interim_...`, `output_...`):** These files represent intermediate stages of data processing and algorithmic calculations. They contain raw calculated metrics, preliminary aggregations, and direct results from specific algorithm modules. They are primarily for internal diagnostic purposes, allowing developers and analysts to trace calculations step-by-step.
- **User-Facing Outputs (`denorm_...`, `export_...`):** These files are the culmination of the internal processing, designed for direct consumption by merchandising teams, business users, and integrated systems (e.g., dashboards, planning tools). They are denormalized, user-friendly, and formatted to present actionable insights, mapping complex internal calculations to clear business metrics.

By providing clear definitions, data sources, and calculation methods, this guide aims to empower users to quickly pinpoint issues, validate outputs, and ensure the accuracy of merchandising recommendations.

2. Output Catalog

This section details the structure and content of the various output files.

Internal Outputs

These files are generated during the algorithm's execution and represent intermediate calculation steps. They are essential for debugging and understanding the data flow.

interim_ros_calculations.csv

This file contains the Rate of Sale (ROS) calculations for individual products across different locations, serving as a foundational input for subsequent optimization modules.

Column Name	Role / Description	Source / Calculation
product_id	Unique identifier for a product SKU.	Raw sales data (e.g., sales_transactions.product_id)
location_id	Unique identifier for the sales location (e.g., store ID, warehouse ID).	Raw sales data (e.g., sales_transactions.location_id)
sales_units_past_4_weeks	Total units of the product_id sold at location_id over the past 4 weeks. This period is configurable (see Config settings).	Aggregation of raw sales data (SUM(units_sold) WHERE sales_date BETWEEN [T-4W] AND [T])
active_weeks	The number of weeks the product was actively available for sale at the location_id within the sales_units_past_4_weeks period. Accounts for new arrivals or discontinued items to ensure accurate ROS calculation.	Derived from product availability records and sales data presence.
calculated_ros	ROS (Rate of Sale): The average units sold per week for the given product_id at location_id. This is a critical metric for demand forecasting.	sales_units_past_4_weeks / active_weeks

output_category_optimization.csv

This file contains the calculated Optimum Depth (OD) and Optimum Width (OW) recommendations per product category, based on category performance and strategic inputs.

Column Name	Role / Description	Source / Calculation
category_id	Unique identifier for a product category.	Product master data (e.g., product_master.category_id)
avg_ros_category	The average ROS (Rate of Sale) for all active products within this	Aggregation of calculated_ros from interim_ros_calculations.csv for all products belonging to this category_id.

Column Name	Role / Description	Source / Calculation
category_sales_volatility	category. Used as an indicator of category health and velocity. A measure of sales fluctuation within the category (e.g., standard deviation of weekly sales). Higher volatility may suggest a need for more agile inventory management.	Calculated from historical sales data for the category (e.g., STDEV(weekly_category_sales) over 12 months).
target_od	OD (Optimum Depth): The recommended ideal number of unique SKUs to stock within this category to maximize sales while optimizing inventory levels. This metric is influenced by category performance, space constraints, and business strategy.	Calculated using a proprietary algorithm that considers avg_ros_category, category_sales_volatility, store space constraints, and potentially user-defined strategic targets for the category. (e.g., OD_Algorithm(avg_ros_category, category_sales_volatility, space_allocation_rules)).
target_ow	OW (Optimum Width): The recommended ideal number of different categories or	Primarily derived from overall business strategy, historical category performance, and competitor analysis, rather than solely mathematical derivation from sales. Often a high-level input or a weighted score indicating strategic priority. (e.g., OW_Allocation_Model(business_strategy_tie_category_profitability, market_share)). No

Column Name	Role / Description	Source / Calculation
	styles to offer. While this table focuses on depth within a category, target_ow here might represent the strategic importance or allocated 'width' share for this category within the broader store.	A separate, higher-level output might combine these provide overall OW recommendations across the business.

output_size_set_recommendations.csv

This file provides the Ideal Size Set (ISS) recommendations for individual products, aiming to maximize sales and minimize stockouts by optimizing size distribution.

Column Name | Role / Description | Source / Calculation

This document provides the second half of a comprehensive Debug Guide for our merchandising algorithm outputs. It is designed to help users diagnose and resolve issues with the generated Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) recommendations.

4. Debug Strategies

When the algorithm’s output doesn’t meet expectations, a systematic approach to debugging can quickly pinpoint the root cause.

4.1. Step-by-Step Error Tracing

The most effective way to trace errors is to work backward from the unexpected output.

- 1. **Start at the Output:** Identify the specific OD, OW, or ISS recommendation that seems incorrect. Note the associated product, category, or Attribute Group (AG).
- 2. **Consult Calculation Lineage:** Refer to the algorithm’s calculation lineage document (provided separately) to understand how the problematic output value is derived.
 - For **OD/OW**: Trace back to the underlying Rate of Sale (ROS) calculations, inventory levels, and any applied constraints or weighting factors.

- For **ISS**: Trace back to historical sales data, size curve analysis, and any specific size availability rules.
- 3. **Inspect Intermediate Data**: As per the lineage, identify the key intermediate tables or files that feed into the problematic calculation step.
- 4. **Validate Inputs**: For each intermediate step, verify the accuracy and completeness of its inputs. This often involves checking the raw data loaded into the algorithm.

4.2. Tools & SQL Queries for Inspection

Access to the underlying data store (e.g., a SQL database) and intermediate files is crucial for inspection.

• SQL Queries for Data Validation:

- **Check Raw Input Data**: Verify the initial data loaded, especially historical sales for ROS calculation.

```
-- Example: Verify ROS input data for a specific product/
SKU
SELECT product_id, SUM(units_sold) AS total_sales,
       COUNT(DISTINCT sale_date) AS sales_days
FROM raw_sales_data
WHERE product_id = 'YOUR_PRODUCT_ID' AND sale_date BETWEEN
       'START_DATE' AND 'END_DATE'
GROUP BY product_id;
```

```
-- Example: Check inventory levels for specific SKUs
SELECT product_id, sku_id, current_stock_units,
       last_received_date
FROM current_inventory
WHERE product_id = 'YOUR_PRODUCT_ID';
```

- **Inspect Intermediate ROS Calculations**:

```
-- Example: Check computed weekly ROS for an item or
category
SELECT product_id, AVG(weekly_ros) AS avg_weekly_ros,
       COUNT(*) AS weeks_tracked
FROM intermediate_ros_calculations
WHERE category = 'YOUR_CATEGORY' AND calculation_period_id
       = 'LATEST_RUN_ID'
GROUP BY product_id;
```

- **Validate Aggregated Data (for OD/OW/ISS inputs)**:

```
-- Example: Review the data used to determine Optimum Depth
SELECT attribute_group, COUNT(DISTINCT sku_id) AS
       distinct_skus_in_ag, SUM(total_sales) AS
       total_ag_sales
FROM aggregated_ag_performance
WHERE calculation_run_id = 'LATEST_RUN_ID'
GROUP BY attribute_group;
```

- **Intermediate File Inspection:** Many algorithms generate temporary CSV, JSON, or Parquet files during execution.
 - Use standard file viewers (cat, less, more on Linux/macOS, Notepad++ on Windows) for small files.
 - For larger files, use tools like pandas in Python or awk/grep for quick filtering and inspection.
 - Always verify file encoding if characters appear garbled.

4.3. Using Logs and Breakpoints

- **Leverage Algorithm Logs:**
 - Ensure the algorithm is running with sufficient logging verbosity (e.g., DEBUG or INFO level).
 - Examine the logs for error messages, warnings, or unexpected data transformations. Logs often show the values of variables at critical points or indicate which records were filtered out.
 - Search logs for specific product IDs or categories that are exhibiting issues.
- **Set Breakpoints (for developers):**
 - If you have access to the algorithm's source code, use an Integrated Development Environment (IDE) to set breakpoints.
 - Step through the code execution, particularly around the calculation points for OD, OW, ISS, and ROS.
 - Inspect variable values at each step to see how data is being processed and transformed, identifying precisely where the logic deviates from expectation.

5. Common Pitfalls & Remedies

Certain data issues and configuration mistakes frequently lead to incorrect algorithm outputs.

5.1. Mismatched Date Ranges

- **Description:** Data extracted for ROS calculations or other metrics does not align with the intended analysis period, or different datasets used by the algorithm refer to different date ranges (e.g., sales data for last 12 weeks, but inventory snapshot from last week).
- **Symptoms:**
 - ROS values are unexpectedly low or high, not reflecting recent trends.
 - Missing sales data for recent periods, leading to understocked recommendations.
 - Discrepancies between algorithm output and other business reports that use different date parameters.
- **Suggested Fix:**
 - **Verify Parameters:** Double-check all `start_date` and `end_date` parameters specified in the algorithm's configuration settings or data extraction queries.
 - **Standardize Date Logic:** Ensure that all input data sources (sales, inventory, returns, etc.) use consistent date range logic and align with the `ros_lookback_weeks` or similar parameters.

- **Check Data Freshness:** Confirm that input data sources are refreshed as expected for the current run.

5.2. Zero/Null Values

- **Description:** Critical input metrics like ROS, stock levels, or historical sales units contain NULL or 0 values where valid data is expected. This can lead to division-by-zero errors, skewed averages, or items being incorrectly excluded/included.
- **Symptoms:**
 - NaN (Not a Number) or infinity in calculated OD, OW, or ISS values.
 - Drastically skewed recommendations (e.g., an item with zero sales historically suddenly recommended for high OD due to a miscalculated average).
 - Algorithm crashes with “division by zero” errors.
 - Items incorrectly omitted from consideration due to NULL ROS.
- **Suggested Fix:**
 - **Data Cleaning:** Implement robust data cleaning steps for input data:
 - Replace NULL with 0 where appropriate (e.g., NULL sales means 0 sales).
 - Filter out or handle products with 0 ROS explicitly if they should not contribute to certain calculations (e.g., average ROS for active items).
 - **Error Handling:** Ensure the algorithm’s code gracefully handles NULL values and potential division by zero (e.g., using COALESCE in SQL, if statements in code).
 - **Input Validation:** Before running the algorithm, run data quality checks on critical input columns to identify and address NULL or 0 issues.

5.3. Weight Parameters

- **Description:** The algorithm uses various weighting parameters (e.g., for different Attribute Groups (AGs), categories, or sales velocity tiers) to influence the final OD, OW, or ISS recommendations. Incorrectly set weights can disproportionately bias the output.
- **Symptoms:**
 - Certain categories or AGs are consistently over- or under-represented in the recommendations, regardless of their actual performance.
 - The algorithm’s output doesn’t align with business priorities (e.g., high-margin categories are not given enough OD).
 - Sensitive results: small changes in input data lead to large swings in recommendations due to overly aggressive weights.
- **Suggested Fix:**
 - **Review Configuration:** Thoroughly review all weight or priority parameters in the algorithm’s configuration settings (see Appendix).
 - **Sensitivity Analysis:** Perform test runs with slightly adjusted weight parameters to understand their impact on the output.
 - **Business Alignment:** Ensure the weights reflect current business strategies and priorities (e.g., higher weight for fast-moving items, new collections, or strategic categories).

5.4. File Encoding/Schema Mismatches

- **Description:** Input files (e.g., historical sales, product master data, inventory feeds) may have different character encodings (e.g., UTF-8 vs. Latin-1) or their column names, data types, or order do not match the expected schema of the algorithm.
- **Symptoms:**
 - Data parsing errors during ingestion, leading to algorithm crashes.
 - NULL values appearing in columns that should contain data.
 - Garbled or incorrect characters appearing in product names or descriptions in the output.
 - Incorrect data types causing calculation errors (e.g., treating a sales figure as text).
- **Suggested Fix:**
 - **Standardize Encoding:** Ensure all input files are consistently encoded (e.g., always UTF-8). Convert files if necessary.
 - **Schema Validation:**
 - Before processing, validate the schema of incoming files against the expected schema definitions. Many data processing frameworks offer built-in schema validation.
 - Confirm column names are exact matches (case-sensitive) and data types are compatible.
 - **Explicit Type Casting:** In SQL queries or data loading scripts, use explicit type casting (e.g., `CAST(sales_column AS INT)`) to convert data to the correct format.
 - **Header Check:** Verify that input files have correct headers and that no extra rows or columns are present unexpectedly.

6. FAQ & Quick Checks

Here are the top 5 questions users ask when debugging this algorithm, along with quick pointers.

1. **Why is the OD/OW/ISS so high/low for [X category/product/AG]?**
 - **Quick Check:** Review the ROS for that item/category, check the `min_sales_threshold` and `weight` parameters in the configuration, and consult the calculation lineage.
2. **My Rate of Sale (ROS) seems off, what should I check?**
 - **Quick Check:** Verify the `ros_lookback_weeks` and `date_range` parameters in the config, check for zero/null values in raw sales data, and confirm the unit of ROS (e.g., units/week vs. units/day).
3. **The recommendations don't match our intuition, how can I validate them?**
 - **Quick Check:** Validate with sample historical data for a few key items, review `weight` parameters, and compare calculated ROS against manual checks.
4. **Are there any common data quality issues I should look for before running the algorithm?**
 - **Quick Check:** Pay close attention to zero/null values for sales/stock, mismatched date ranges, and file encoding/schema mismatches in input files.

5. **How often should I rerun the algorithm to get optimal recommendations?**
- **Quick Check:** This depends on business needs and the volatility of your sales data; generally, weekly or bi-weekly for dynamic categories is a good starting point to capture recent trends.

7. Appendix

7.1. Reference of Configuration Settings

The following are key configuration settings that directly impact the output values of OD, OW, and ISS. These settings are typically found in a `config.ini`, `application.properties`, or similar configuration file.

- **ROS Calculation Parameters:**

- `ros_lookback_weeks`: Defines the historical period (in weeks) used to calculate ROS. A shorter period makes ROS more reactive to recent trends; a longer period provides more stable averages.
- `ros_min_sales_threshold`: Minimum sales units required within the `ros_lookback_weeks` for a product/SKU to have a valid ROS.

- **Optimum Depth (OD) Parameters:**

- `od_sales_contribution_target`: The target percentage of total category sales that the recommended OD should cover.
- `od_min_skus_per_category`: Minimum number of SKUs to recommend for OD, regardless of sales.
- `od_max_skus_per_category`: Maximum number of SKUs to recommend for OD, acting as an upper bound.

- **Optimum Width (OW) Parameters:**

- `ow_category_priority_weights`: A mapping of category names to numerical weights, influencing their priority when determining OW. Higher weight means more emphasis in OW calculation.
- `ow_min_categories`: Minimum number of categories to recommend for OW.

- **Ideal Size Set (ISS) Parameters:**

- `iss_sales_coverage_target`: The target percentage of total size sales that the recommended ISS should cover.
- `iss_min_sizes`: Minimum number of sizes to include in an ISS.
- `iss_historical_size_curve_period_weeks`: The historical period used to analyze size performance and generate the size curve.

- **Global Parameters:**

- `data_refresh_frequency`: How often the raw input data is expected to be updated.
- `inventory_buffer_percentage`: An additional percentage added to demand forecasts to account for safety stock.

7.2. Support Contact Info

For further assistance with debugging the merchandising algorithm outputs, please contact:

Email: support@example.com

End of Debug Guide

Chapter 1: OtbDepletionInputModule

Last Updated: 2025-05-23

1. Business Objective

This step gathers and prepares all the foundational data needed to accurately forecast how quickly products will sell and determine what needs to be replenished.

Why This Matters: This ensures that future replenishment decisions are based on the most accurate and up-to-date information, preventing stockouts, reducing excess inventory, and maximizing sales and profit. By starting with precise data, businesses can make smarter inventory investments, leading to better product availability for customers and improved financial performance.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Average Selling Price (ASP) by Period, Store, and Attribute Group (AG):** *Why we need it: To understand the historical value of sales, which helps in valuing inventory and forecasting revenue for replenishment, allowing for more strategic investment decisions.*
- **Relevant Seasons for Rate of Sale (ROS) Computation:** *Why we need it: To focus our analysis only on styles that are currently active or relevant for sales velocity calculations, ensuring our ROS (Rate of Sale) is accurate for the current business context and not skewed by irrelevant historical data.*

Product Data

- **Product Style Information:** *Why we need it: To identify unique product styles and their characteristics, which is crucial for grouping products for replenishment and understanding what to stock, ensuring we manage product categories effectively.*
- **Style-to-Season Assignments:** *Why we need it: To correctly identify which styles belong to specific seasons, ensuring we only consider products relevant to current or upcoming selling periods for depletion planning and ROS calculations, preventing misdirected inventory efforts.*

Operational Data

- **Vendor Lead Times:** *Why we need it:* To know how long it takes for new stock to arrive after an order is placed, which is critical for determining when to reorder to avoid stockouts and manage inventory levels effectively, thus optimizing supply chain efficiency.
- **Current On-Hand Inventory (SKU-store level):** *Why we need it:* To understand exactly what stock is available for every unique item (SKU) at each store, forming the baseline for all depletion calculations and replenishment needs. This is like checking your current bank balance before planning future spending.
- **Relevant Seasons for Depletion Planning:** *Why we need it:* To ensure that depletion forecasts and replenishment recommendations are focused on the styles and seasons that are actively being considered for stock adjustments, allowing merchants to prioritize what really matters now.

3. Business Settings

Setting Name	Default Value	Description	When to Change
ROS Computation Seasons	(No explicit default)	Defines the specific selling seasons (e.g., “Spring 2024”, “Summer 2023”) whose sales data will be used to calculate Rate of Sale (ROS).	When you want to calculate ROS based on different historical periods or collections, for example, to focus on recent trends, exclude very old seasons, or include upcoming season pre-sales. Changing this impacts how accurately ROS reflects current demand, affecting inventory investment and preventing misjudged replenishment.
Depletion Planning Seasons	(No explicit default)	Specifies the seasons for which the system will actively plan depletion and replenishment. This influences which styles are considered.	When new seasons are launched, older seasons are being cleared out, or strategic focus shifts. Incorrect settings can lead to overstocking of irrelevant items or missed opportunities for high-demand products.
Data Read Seasons	(No explicit default)	Determines the seasons from which historical data (like sales or inventory) is extracted. Often combined with Depletion Planning Seasons.	If you need to pull historical performance from specific past seasons to inform current decisions, especially for evergreen styles or long-lifecycle products. This helps in understanding long-term demand patterns.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Raw Business Data<br/>(Inventory, Styles, Lead Times, Seasons)] --> B[DepletionInputModule<br/>Data Preparation]
B --> C[Organized Depletion Data<br/>(Ready for Analysis)]
```

1. Start with → Raw Business Data including current inventory levels for every SKU at every store, lead times for product delivery, detailed style information, and lists of relevant selling seasons.
2. Then → Filter product styles based on the specified ROS Computation Seasons and Depletion Planning Seasons, ensuring only relevant items are considered for sales velocity calculations and future stock adjustments.
3. Next → Consolidate lead time information by channel and style, and gather Average Selling Price (ASP) data, preparing comprehensive datasets.
4. Finally → Produce a structured set of prepared data, including style lists for ROS and depletion, current store-level inventory, and ASP maps, all ready for the core depletion logic.

5. Step-by-Step Actions

1. **Initialize** the module with business settings
 - What we do: *We load the specific configuration settings, such as which seasons to consider for ROS (Rate of Sale) calculations and depletion planning.*
 - Why we do it: *This ensures the module knows exactly what data to focus on, tailoring the analysis to current business objectives and ensuring relevance.*
2. **Gather** Lead Times
 - What we do: *We collect data on how long it typically takes for new stock to arrive for each product style and sales channel.*
 - Why we do it: *Accurate lead times are crucial for timely replenishment, preventing stockouts and ensuring products are available when customers want them, which directly impacts customer satisfaction and sales.*
3. **Acquire** Current Inventory
 - What we do: *We obtain the most up-to-date count of every unique item (SKU) currently in stock at each store location.*
 - Why we do it: *This provides a real-time snapshot of available stock, forming the baseline from which all future depletion forecasts are calculated. You can't plan future inventory without knowing what you have right now.*
4. **Identify** Relevant Styles and Seasons
 - What we do: *We cross-reference product styles with the specified ROS computation seasons and depletion planning seasons to create lists of actively managed styles for each channel.*
 - Why we do it: *This step ensures that our sales forecasts and replenishment decisions are focused only on products that are currently relevant for sale or future planning, avoiding unnecessary analysis on outdated or irrelevant items and optimizing processing time.*

5. **Collect Average Selling Prices (ASP)**

- What we do: *We retrieve the average selling price for product groups (Attribute Group - AG) within specific periods and stores.*
- Why we do it: *Understanding the average selling price helps in valuing inventory and forecasting potential revenue, supporting financial planning for replenishment investments and ensuring profitable decisions.*

6. **Sample Scenario**

Let's walk through a real example:

Starting Point

- “ROS Computation Seasons” is set to “Spring 2024#Fall 2023” (to include recent sales history).
- “Depletion Planning Seasons” is set to “Spring 2024” (as this is the active selling season).
- Current inventory shows 150 units of ‘Blue Denim Jacket, Style A123’ at Store 001.
- Lead time for ‘Blue Denim Jacket, Style A123’ from Vendor X is 14 days.
- Average Selling Price for ‘Jackets (AG 01)’ in Store 001 for Spring 2024 is \$85.

What Happens

1. First, the module reads the configuration settings and identifies “Spring 2024” and “Fall 2023” as the relevant seasons for calculating how fast products sell (ROS). It also notes that “Spring 2024” is the focus for active depletion planning.
2. Next, it loads all current inventory data, finding those 150 units of ‘Blue Denim Jacket, Style A123’ at Store 001. This tells the system exactly what’s on the shelves.
3. Then, it pulls lead time information and confirms the 14-day delivery time for ‘Blue Denim Jacket, Style A123’ from its supplier. This informs how far in advance new orders must be placed.
4. It identifies ‘Blue Denim Jacket, Style A123’ as a relevant style for both ROS computation (because it’s associated with Spring 2024) and for depletion planning.
5. Finally, it retrieves the historical Average Selling Price for ‘Jackets’ in Store 001 during Spring 2024, which is \$85. This provides the financial context for the product.

Final Outcome

All this detailed information – relevant styles, current stock, lead times, and average selling prices – is neatly organized and made available for the next steps in the depletion process. The system now knows exactly what to focus on and has the base data to predict future sales and stock needs for ‘Blue Denim Jacket, Style A123’ at Store 001, enabling accurate replenishment recommendations.

Key Takeaway: This step ensures that all subsequent sales forecasting and replenishment decisions are built upon a comprehensive and accurate foundation of current inventory, operational timelines, and financial context, preventing costly misjudgments.

7. Results

This step produces:

Main Outputs

- **Organized Lead Time Data:** *What it tells us: A consolidated view of how long it takes to receive specific styles by channel, critical for accurate order timing and preventing stockouts.*
- **Current Live Inventory Map (SKU-Store level):** *What it tells us: An exact count of every unique item (SKU) at every store, forming the precise starting point for depletion calculations and stock management.*
- **Relevant Style Lists for ROS and Depletion:** *What it tells us: Clearly defined sets of product styles that should be included in Rate of Sale (ROS) calculations and for which depletion planning is actively performed, ensuring focus on active merchandise.*
- **Average Selling Price (ASP) Map:** *What it tells us: Historical average selling prices for different product attribute groups (AGs) by store and period, used for financial valuation and forecasting revenue from replenishment.*

Business Reports

- **Data Input for Sales Forecasting Module:** *Who uses it and why: The Sales Forecasting module uses this prepared data (relevant styles, inventory, ASP) to predict future sales velocity and demand. It helps merchants understand potential sell-through rates and plan campaigns.*
- **Data Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes the current inventory, lead times, and depletion styles to determine optimal reorder points and quantities. It assists planners in managing stock levels and avoiding costly stockouts or overstocks.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will likely fail to run or operate on stale data. The process cannot proceed without foundational data.	Verify data source connections, check data ingestion logs, and re-trigger the data feed. Contact the IT support team immediately, as this impacts all downstream inventory decisions.
	The relevant style lists for ROS or	Verify the “ROS Computation Seasons” and “Depletion Planning Seasons” settings to

Situation	What Happens	What You Should Do
No styles identified for a season	depletion will be empty, meaning no products will be considered for analysis.	ensure they are correctly configured for current or upcoming seasons. Check if product styles are correctly tagged to seasons in the product master data.
Inconsistent lead times for a style across channels	The system uses the most specific lead time available, or a default if none, which might lead to suboptimal order timing.	Review and reconcile lead time data in the system. Work with vendors and logistics teams to ensure accurate and consistent lead time information is maintained for each product and channel, impacting delivery reliability.
Zero inventory for all relevant SKUs	The current inventory map will show all zeros for these items. The depletion process will identify all relevant items as needing replenishment.	This might indicate a significant stockout situation or a data issue. Confirm the accuracy of inventory counts. Prioritize replenishment actions if the data is correct to avoid lost sales, or investigate data discrepancies.

9. Next Steps

Before This Step

- **Master Data Updates:** *What we need from it: Up-to-date product styles, SKU details, and channel definitions, ensuring the system has the correct product hierarchy and attributes to categorize inventory accurately.*
- **Inventory Data Synchronization:** *What we need from it: Real-time or recent snapshots of physical inventory counts from all stores and warehouses, forming the most accurate starting point for stock analysis and replenishment.*

After This Step

- **Rate of Sale (ROS) Calculation:** *What they need from us: The identified relevant styles, current inventory data, and historical sales information to accurately calculate how quickly products are selling, enabling demand forecasting.*
- **Depletion Forecasting:** *What they need from us: The prepared lead times, current inventory, and seasonal style lists to project future inventory levels and identify potential stockouts, allowing proactive stock management.*
- **Replenishment Recommendation:** *What they need from us: All the processed input data to generate optimized orders for stock replenishment, ensuring stores have the right products at the right time.*

10. Common Questions

Q: Why is it important to define specific seasons for ROS computation and depletion planning? A: Defining seasons ensures that the system focuses its analysis only on products currently relevant to your business strategy. This prevents inaccurate forecasts caused by including out-of-season or discontinued items, leading to more precise inventory decisions and better sales outcomes.

Q: What happens if the lead time data is inaccurate or missing? A: Inaccurate lead times can cause significant issues. If lead times are too short, you might order too late and experience stockouts and lost sales. If they are too long, you might over-order and hold excess inventory, increasing holding costs. Missing lead times can lead to using default values which may not be appropriate, impacting on-time replenishment and profitability.

Q: How does the current inventory data impact the depletion process? A: The current inventory data is the starting point for all depletion calculations. Without an accurate real-time count of what's on hand at each store (SKU-store level), the system cannot reliably predict future stock levels or recommend correct replenishment quantities, leading to potential lost sales from stockouts or increased holding costs from overstock.

Q: What is the significance of the Average Selling Price (ASP) in this module? A: The Average Selling Price (ASP) provides a financial context to the product data. While this module primarily prepares data, the ASP informs subsequent stages about the value of inventory and the potential revenue from sales, helping to prioritize replenishment based on profitability and manage overall investment effectively.

Q: Can I include styles from multiple past seasons in my ROS computation? A: Yes, you can. By including multiple seasons in your "ROS Computation Seasons" setting (e.g., "Spring 2024#Fall 2023"), you can leverage a broader historical sales context for calculating Rate of Sale. This is particularly useful for evergreen products or to smooth out seasonal fluctuations for certain categories, providing a more robust demand signal.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Gathers and prepares all essential input data for the depletion process, ensuring accurate sales forecasting and replenishment planning.
 - **Main Input:** Current inventory levels (SKU-store), lead times for delivery, product style details, and defined relevant selling seasons.
 - **Main Output:** Organized datasets including relevant styles for ROS and depletion, precise current stock counts, lead time maps, and average selling prices.
 - **Warning Signs:** Missing inventory data, incorrect lead times, or styles not appearing in expected season lists.
 - **Key Settings:** ROS Computation Seasons, Depletion Planning Seasons, Data Read Seasons.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 2:

OtbDepletionPrepareDataModule

Last Updated: 2025-05-23

1. Business Objective

This module gathers, consolidates, and prepares all the foundational data needed for the depletion allocation process, which determines how inventory moves to meet customer demand.

Why This Matters: By providing a complete and accurate snapshot of product availability across all stores and warehouses—including stock currently on shelves, in transit, or awaiting delivery—this module ensures that subsequent inventory decisions are based on the truest possible picture. This prevents stockouts, minimizes excess inventory, and maximizes product availability for customers, leading to improved sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Relevant Seasons for Depletion Planning:** *Why we need it: To ensure that the data prepared is focused on products and periods actively being considered for sales and inventory planning, preventing irrelevant data from skewing depletion recommendations.*

Product Data

- **Product Style and SKU Information:** *Why we need it: To accurately identify and categorize every unique item (SKU) and its associated style, which is fundamental for managing inventory and grouping products for effective replenishment.*
- **Product Attribute Group (AG) Mappings:** *Why we need it: To understand how products are categorized by attributes (e.g., color, material, pattern), enabling analysis and planning at a higher level than individual SKUs, which is crucial for managing product diversity and achieving OD (Optimum Depth) and OW (Optimum Width) targets.*

- **Ideal Size Set (ISS) Information:** *Why we need it: To define the best combination of sizes for each style, ensuring that stores stock sizes that customers actually buy, maximizing sales and minimizing leftover odd sizes.*

Operational Data

- **Store Configuration and Distribution Network Details:** *Why we need it: To identify active retail stores within the distribution network and understand their channels, ensuring that depletion calculations are applied only to relevant locations and that replenishment routes are correctly mapped.*
- **Current On-Hand Inventory (SKU-store & warehouse levels):** *Why we need it: To provide an accurate, real-time count of available stock across all stores and central warehouses, forming the indispensable baseline for all depletion forecasts and replenishment decisions.*
- **Open Orders and Goods in Transit (GIT):** *Why we need it: To account for inventory that is already purchased from vendors or physically en route between locations but not yet available for sale, preventing over-ordering and providing a more complete picture of future stock availability.*
- **Incoming Store & Warehouse Receipts (Inwards):** *Why we need it: To capture recently received stock that has arrived at stores or warehouses but is not yet fully processed into on-hand inventory, ensuring the most up-to-date view of available products for depletion planning.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Depletion Planning Seasons	(No explicit default)	Defines the specific selling seasons (e.g., “Summer 2024”, “Fall 2024”) for which the system will actively plan depletion and replenishment. This influences which styles are considered.	When new seasons are launched, older seasons are being cleared out, or strategic focus shifts. For example, as Spring 2024 transitions to Summer 2024, you’d update this to ensure depletion efforts align with the new collection. Incorrect settings can lead to overstocking of irrelevant items or missed opportunities for high-demand products.
Data Read Seasons	(No explicit default)	Determines the seasons from which historical data (like product details or seasonal inventory patterns) is extracted. This often complements Depletion	If you need to pull historical performance from specific past seasons to inform current decisions, especially for evergreen styles or long-lifecycle products. For instance, if you want to understand how a core denim style performed in Fall 2023 to inform Fall 2024 planning, you’d include Fall 2023 here.

Setting Name	Default Value	Description	When to Change
		Planning Seasons.	
Include Store Stock in Depletion	True	Specifies whether the current on-hand inventory at individual retail stores should be considered when calculating depletion needs.	Almost always kept as 'True' for accurate inventory management. Only changed to 'False' in very rare scenarios, such as during a complete physical inventory audit where current stock counts are temporarily unreliable, which would significantly impact depletion accuracy and is strongly advised against for regular operations.
Include Goods-in-Transit & Open Orders	True	Determines if stock that is currently in transit between locations or has been ordered from vendors but not yet received should be factored into the overall inventory picture.	Almost always kept as 'True' to prevent over-ordering. If this is 'False', the system might recommend ordering more stock than necessary because it doesn't "see" what's already on its way. This could be temporarily set to 'False' only during specific data reconciliation periods, but would otherwise lead to inflated inventory levels and increased holding costs.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Raw Master & Transactional Data<br/>(Stores, SKUs, Inventory, GIT, Orders)] --> B{0tbDepletionPrepareDataModule<br/>Data Preparation & Loading}
```

```
B --> C[Consolidated Depletion Data<br/>(Ready for Allocation & Planning)]
```

1. Start with → Raw Master & Transactional Data including detailed store lists, all product SKUs and styles, current physical inventory at stores and warehouses, products en route (Goods in Transit), and items on open purchase orders.
2. Then → Filter this data based on the specified depletion planning seasons, ensuring only relevant styles and items are processed.
3. Next → Consolidate all inventory sources, including store stock, warehouse stock, and future incoming inventory from open orders and goods in transit, creating a holistic view of available products.
4. Finally → Produce a structured set of prepared data, which includes a comprehensive inventory snapshot, active store configurations, relevant style lists, and detailed product mappings (attributes, size sets), all ready for the core depletion allocation logic.

5. Step-by-Step Actions

1. **Initialize** the module with business settings
 - What we do: *We load the specific configuration settings, such as which seasons to consider for depletion planning and which types of inventory (e.g., store stock, in-transit) to include.*
 - Why we do it: *This ensures the module knows exactly what data to focus on and how to interpret it, tailoring the analysis to current business objectives.*
2. **Identify** Active Distribution Stores
 - What we do: *We compile a list of all active retail stores that are part of the distribution network and are eligible for inventory allocations, excluding online-only or inactive locations.*
 - Why we do it: *This ensures that depletion calculations are focused only on physical stores where stock needs to be managed, optimizing resource allocation and preventing irrelevant data processing.*
3. **Process** Overridden Channel-Styles
 - What we do: *We apply specific, customized rules for certain product styles within particular sales channels, often related to unique lead times or distribution priorities.*
 - Why we do it: *This allows for exceptions to general rules, ensuring that specific products or channels with unique operational needs are handled correctly, which is vital for specialized merchandise or promotions.*
4. **Consolidate** Current Store and Warehouse Inventory
 - What we do: *We gather the most up-to-date counts of every unique item (SKU) currently in stock at each physical store and within central warehouses.*
 - Why we do it: *This forms the accurate baseline for all depletion forecasts and replenishment decisions, providing a real-time picture of what is immediately available for customers.*
5. **Integrate** Open Orders and Goods in Transit (GIT)
 - What we do: *We add inventory that has been ordered from vendors or is currently moving between locations but hasn't yet arrived at its final destination.*
 - Why we do it: *Including future stock prevents over-ordering, provides a more complete view of upcoming product availability, and ensures that replenishment plans consider all incoming inventory.*
6. **Determine** Live Styles for Depletion
 - What we do: *We identify which product styles are currently active, in season, or have stock present in stores or warehouses, making them relevant for depletion planning.*
 - Why we do it: *This ensures that depletion efforts are focused only on merchandise that is actively being sold or planned for, avoiding unnecessary analysis of discontinued or out-of-season items.*
7. **Map** Product Attributes, Size Sets, and Optimal Depths
 - What we do: *We link each style to its relevant Attribute Group (AG), define its Ideal Size Set (ISS), and establish the Optimum Depth (OD) for SKUs within that style.*
 - Why we do it: *Understanding these relationships enables more intelligent inventory allocation, ensuring stores receive the right mix of attributes, the best combination of sizes, and the ideal quantity of each item, maximizing sales and minimizing markdowns.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Depletion Planning Seasons is set to "Summer 2024".
- Data Read Seasons is set to "Spring 2024#Summer 2024".
- Current inventory shows 200 units of 'Women's Floral Dress, Style D500' (SKU D500-S, D500-M, D500-L) at Store 005.
- There are 50 units of 'Women's Floral Dress, Style D500' currently in transit to Store 005 from the central warehouse.
- The central warehouse has 150 units of 'Women's Floral Dress, Style D500'.
- 'Women's Floral Dress, Style D500' is mapped to 'Dresses' (AG 03). Its Ideal Size Set (ISS) is defined as S:2, M:3, L:2, and its Optimum Depth (OD) for a medium-volume store is 7 units per SKU.

What Happens

1. First, the module reads the configuration settings, identifying "Summer 2024" as the active season for depletion planning and confirming that both Spring and Summer 2024 data should be considered for general product information.
2. Next, it compiles the list of active distribution stores, including Store 005.
3. Then, it gathers all current inventory: 200 units at Store 005 for 'Women's Floral Dress', 150 units in the central warehouse, and crucially, it integrates the 50 units of the same dress that are currently in transit to Store 005. This means the system now understands Store 005 effectively has 250 units (200 on-hand + 50 in transit) of this style.
4. It identifies 'Women's Floral Dress, Style D500' as a "live style" because it has current stock and is part of the "Summer 2024" planning season.
5. Finally, it incorporates the product's attributes (AG 03: Dresses), its Ideal Size Set (ISS: S:2, M:3, L:2), and its Optimum Depth (OD: 7 units per SKU), connecting all this crucial merchandising detail to the inventory data.

Final Outcome

All this detailed information—live styles, comprehensive current and incoming inventory across all locations, relevant store configurations, and precise product attribute/sizing details—is meticulously organized and made available for the next steps in the depletion process. The system now has a complete, accurate, and actionable view of 'Women's Floral Dress, Style D500', enabling intelligent stock allocation and replenishment decisions for Store 005 and other locations.

Key Takeaway: This step creates a unified and accurate inventory picture by combining all sources of stock data with essential product and store information, ensuring that future inventory decisions are holistic and well-informed.

7. Results

This step produces:

Main Outputs

- **Consolidated Store & Distribution Network Map:** *What it tells us: A comprehensive list of all active retail stores eligible for depletion and allocation, including details like their channel and any linked reference stores, crucial for routing inventory effectively.*
- **Comprehensive Inventory Snapshot (SKU-Store & WH levels):** *What it tells us: A precise and aggregated view of all available stock for every unique item (SKU) at each store and warehouse, including on-hand units, goods in transit, and open orders, providing the full picture of current and incoming inventory.*
- **Live Style Status:** *What it tells us: A clear indication of which product styles are currently active and relevant for sale or planning, ensuring that depletion efforts are focused on merchandise that matters to the business now.*
- **Product Attribute (AG) & Ideal Size Set (ISS) Mappings:** *What it tells us: Detailed connections between styles and their attribute groups, along with their optimal size combinations, guiding how products should be allocated to maximize customer choice and minimize stockouts.*
- **SKU Optimum Depth (OD) Information:** *What it tells us: The ideal number of units to stock for specific SKUs, used to optimize inventory levels and ensure stores have sufficient variety without being overstocked.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine consumes this prepared data to calculate optimal inventory transfers between stores and warehouses, determining where stock should move to maximize sales and minimize markdowns.*
- **Inventory Visibility Dashboard Data Feed:** *Who uses it and why: Merchants and planners use this consolidated data to gain real-time insights into global inventory availability (on-hand, in-transit, ordered), helping them proactively manage stock levels and respond to demand shifts.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process with incomplete or stale data, leading to inaccurate inventory pictures and flawed depletion recommendations.	Verify data source connections for inventory, orders, and master data. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions and potential sales.
No active distribution stores identified	The system will not process any store-level inventory, or it will throw an error indicating no valid targets for allocation.	Verify that store data is correctly loaded and that isOnline flags are appropriately set for retail stores. Ensure store IDs are accurate and that the system can properly distinguish active retail locations.

Situation	What Happens	What You Should Do
No styles identified as “live”	The Live Style Status will be empty, meaning no products will be considered for depletion allocation, even if stock exists.	Review the Depletion Planning Seasons and Data Read Seasons settings to ensure they are correctly configured for current selling seasons. Check if product styles are correctly associated with these seasons in the product master data.
Significant discrepancies in inventory data	Negative stock values, unusually high counts, or mismatched SKU/style data can lead to erroneous inventory snapshots.	Conduct a thorough inventory audit for the affected SKUs/styles. Investigate the source of the data discrepancy (e.g., faulty scanning, system errors). Reconcile inventory records to ensure accuracy before re-running the module.
Overridden channel-styles are misconfigured	Lead times or distribution rules may be incorrectly applied, leading to suboptimal order timing or stock placements for specific products.	Review the LeadTimeRow data for overridden styles and channels. Confirm with merchandising and logistics teams that the intended overrides are accurately reflected and applied correctly in the system to prevent operational inefficiencies.

9. Next Steps

Before This Step

- **Master Data Setup:** *What we need from it: Up-to-date product styles, SKU details, attribute definitions, store locations, and channel assignments, ensuring the system has the correct product hierarchy and network structure to categorize inventory accurately.*
- **Real-time Inventory & Order Data Feeds:** *What we need from it: Recent snapshots of physical inventory counts from all stores and warehouses, alongside data on open purchase orders and products in transit, forming the most accurate starting point for stock analysis and replenishment.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The consolidated inventory snapshot, live style information, and store configurations to calculate optimal stock transfers and determine how best to move existing inventory to meet demand.*
- **Inventory Optimization Modules:** *What they need from us: The comprehensive view of stock (current, in-transit, incoming) combined with product attribute and sizing details to refine strategies for stock levels, identify potential overstocks or stockouts, and propose adjustments.*

- **Replenishment Planning Module:** *What they need from us: All the processed input data to generate optimized orders for stock replenishment from warehouses to stores or from vendors, ensuring stores have the right products at the right time.*

10. Common Questions

Q: Why is it important to integrate Goods in Transit (GIT) and Open Orders in this preparation step? A: Integrating GIT and Open Orders provides a complete and forward-looking view of your inventory. Without it, the system would only see what's physically on shelves, potentially recommending over-ordering products that are already on their way or have been purchased. This integration prevents excess inventory, reduces holding costs, and ensures more accurate replenishment decisions.

Q: What happens if store data is inaccurate, for example, if old or inactive stores are accidentally included? A: If inaccurate store data is included, the system might attempt to allocate or manage inventory for locations that no longer need it, leading to wasted processing time, misdirected stock, and potential financial losses. It's crucial to have precise and up-to-date store configurations to ensure depletion efforts are focused on active, relevant retail locations.

Q: How does the “live style status” help in inventory planning? A: The “live style status” identifies only those products that are actively in season, currently selling, or have stock that needs to be managed. This helps inventory planners focus their attention and resources on relevant merchandise, avoiding unnecessary analysis or allocation efforts for discontinued, out-of-season, or irrelevant items, thereby optimizing efficiency and product availability.

Q: What is the significance of “overridden channel-styles” in this module? A: Overridden channel-styles allow for specific, tailored rules for certain products in particular sales channels. For instance, a fast-fashion item might have a shorter lead time for online sales than for physical stores. This ensures that the system can apply these special rules, preventing stockouts or excess inventory for unique product-channel combinations and maintaining operational flexibility.

Q: What is the role of Ideal Size Set (ISS) and Optimum Depth (OD) information in this data preparation? A: While this module primarily prepares the data, providing ISS and OD ensures that subsequent depletion and allocation steps understand the ideal mix of sizes and the optimal number of items to stock. This information is critical for maximizing sales by ensuring customer access to preferred sizes and quantities, minimizing leftover odd sizes, and optimizing the balance of variety (OV) and quantity (OD) within a category.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Gathers, consolidates, and prepares all essential master and transactional data for the depletion allocation process, creating a comprehensive inventory picture.

- **Main Input:** Current on-hand inventory (store & warehouse), Goods in Transit, Open Orders, incoming receipts, and detailed product/store master data, filtered by relevant seasons.
 - **Main Output:** A precise, consolidated inventory snapshot, live style lists, distribution store configurations, and detailed product attribute/sizing mappings.
 - **Warning Signs:** Incomplete or delayed data feeds, no active stores identified, or no styles marked as “live.”
 - **Key Settings:** Depletion Planning Seasons, Data Read Seasons, Include Store Stock in Depletion, Include Goods-in-Transit & Open Orders.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 3:

OtbDepletionPrepareSalesModule

Last Updated: 2025-05-23

1. Business Objective

This module meticulously cleans and refines raw sales data, making it ready for in-depth analysis to accurately forecast future demand and optimize product availability.

Why This Matters: By eliminating distorting factors like liquidation events and incomplete sales records, this step ensures that all subsequent sales forecasts and replenishment decisions are based on the truest representation of customer demand. This prevents misjudging product performance, reduces the risk of stockouts or overstocks, and ultimately maximizes sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Transactions (SKU-store level):** *Why we need it: To understand past purchasing patterns, including specific items sold, quantities, dates, and locations, which is foundational for calculating Rate of Sale (ROS) and identifying sales trends.*
- **Indication of Liquidation Sales:** *Why we need it: To identify and exclude sales that occurred during clearance or markdown events, ensuring that typical product demand is not artificially inflated by unusually low prices, which would skew replenishment forecasts.*

Product Data

- **Product Style and SKU Information:** *Why we need it: To identify unique product items and their corresponding styles, allowing the system to correctly link sales transactions to specific merchandise and categorize them for analysis.*
- **Attribute Group (AG) Mappings:** *Why we need it: To understand how individual SKUs are grouped into broader categories based on shared attributes (e.g., color, material), enabling sales analysis at a higher level and helping identify patterns across product lines.*

Operational Data

- **PSA (Pivotal Size Availability) Benchmarks:** *Why we need it: To define minimum availability standards for key sizes within a style, allowing the system to identify and filter out sales periods where products were not adequately stocked, ensuring that ROS calculations reflect periods of genuine sales potential.*
- **Sales Data Period Benchmarks:** *Why we need it: To define the specific timeframes for analyzing sales data, such as the look-back period for calculating Rate of Sale, ensuring consistency and relevance in demand forecasting.*

3. Business Settings

Setting Name | Default Value | Description | When to Change |
ROS Calculation Season | (No explicit default) | Defines the past selling season (e.g., “Fall 2023”, “Spring 2024”) whose sales data will be used to calculate Rate of Sale (ROS). This ensures that the velocity is based on relevant historical demand patterns. | When you want to update the historical context for ROS calculation, for example, to include the most recent full season or exclude very old seasons no longer representative of current demand. Changing this directly impacts the accuracy of demand forecasts. |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Raw Sales Transactions<br/>(Last 6 Months)] --> B[Remove Liquidation Sales<br/>Sales Data Cleaning & ROS Calculation]
B --> C[Cleaned Sales Data<br/>& Initial SKU Rate of Sale]
```

1. Start with → Raw Sales Transactions for an extended historical period (e.g., the last six months), including all product sales, regardless of their nature.
2. Then → Remove Liquidation Sales from this historical data, separating genuine demand from promotional or clearance-driven sales.
3. Next → Filter for "Brokenness" by analyzing the unliquidated sales within a more recent analysis period (e.g., last 30 days) against predefined Pivotal Size Availability (PSA) benchmarks, ensuring only sales from adequately stocked periods are retained.
4. Finally → Compute Initial SKU Rate of Sale from the comprehensive unliquidated sales data, and prepare the cleaned sales transactions for further analysis.

5. Step-by-Step Actions

1. **Initialize** module with business settings
 - What we do: *We load predefined settings like the desired sales analysis periods and the benchmarks for identifying “broken” sales data.*

- Why we do it: *This ensures the module processes sales information according to current business rules and focuses on the most relevant periods and data quality standards.*
- 2. **Gather** comprehensive sales data
 - What we do: *We collect all raw sales transactions for every product (SKU) at every store over a specified historical period, typically the last six months.*
 - Why we do it: *This provides a broad dataset for a comprehensive understanding of past product performance and sufficient volume for accurate rate calculations.*
- 3. **Remove** liquidation sales
 - What we do: *We identify and filter out any sales that occurred during liquidation events or deep clearance periods, based on specific system flags or price points.*
 - Why we do it: *Liquidation sales can dramatically inflate perceived demand, so removing them ensures that our sales velocity calculations reflect organic customer purchasing behavior, preventing over-ordering of regular-priced merchandise.*
- 4. **Filter** for “brokenness”
 - What we do: *We analyze the sales data from a recent period (e.g., last 30 days) and remove records for products that had insufficient stock levels (based on PSA benchmarks) during that time.*
 - Why we do it: *Sales records can be “broken” if customers couldn’t buy a product due to missing key sizes. Filtering these out ensures that the remaining sales accurately represent demand when products were adequately available, preventing underestimation of true sales potential.*
- 5. **Compute** initial SKU Rate of Sale (ROS)
 - What we do: *Using the cleaned, unliquidated sales data, we calculate an initial Rate of Sale for each unique product (SKU) at each store where non-zero sales occurred.*
 - Why we do it: *This provides the foundational velocity metric, indicating how quickly products are selling, which is essential for forecasting future demand, setting replenishment targets, and making informed inventory investment decisions.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A specific ‘Women’s Summer Sandal, Style S101’ (SKU S101-7-BLK for size 7, black) is being analyzed.
- Raw sales data shows 300 units of SKU S101-7-BLK sold in the last 6 months at Store 010.
- Of these, 50 units were sold during a “Summer Clearance” event flagged as liquidation.
- For the last 30 days, the Pivotal Size Availability (PSA) for Style S101 at Store 010 was only 40% (meaning key sizes were often out of stock).
- The “Broken Sales PSA Benchmark” is set to 60%.
- The “Broken Sales Day Benchmark” is 7 days.

What Happens

1. First, the module gathers all 300 units of sales for SKU S101-7-BLK from Store 010 over the last 6 months.
2. Then, it identifies and removes the 50 units sold during the “Summer Clearance” liquidation, leaving 250 units of unliquidated sales.
3. Next, it examines the unliquidated sales from the last 30 days. It sees that the PSA for Style S101 was only 40%, which is below the 60% “Broken Sales PSA Benchmark.” Because this low availability persisted for more than 7 days (the “Broken Sales Day Benchmark”), the module flags this period’s sales as “broken.” This means any sales data from this 30-day period will be treated with caution or excluded from primary ROS calculation for that specific SKU-store to avoid underestimating true demand.
4. Finally, the module computes an initial Rate of Sale (ROS) for SKU S101-7-BLK based on the *full 6 months of unliquidated sales data* (250 units). This initial ROS gives a baseline understanding of how quickly this sandal normally sells when not on clearance.

Final Outcome

The module provides a clean, accurate sales history for ‘Women’s Summer Sandal, Style S101’, excluding misleading liquidation sales and highlighting periods where sales were likely suppressed due to poor stock availability. This provides a reliable foundation for predicting future demand and planning replenishment orders.

Key Takeaway: This step ensures that sales forecasts are based on genuine, unhindered customer demand, preventing costly errors from misinterpreting sales data affected by clearances or stockouts.

7. Results

This step produces:

Main Outputs

- **Cleaned, Unliquidated Sales Data:** *What it tells us: A refined dataset of sales transactions, free from the distortions of liquidation events and periods of poor stock availability, providing a true picture of customer demand.*
- **Initial SKU Rate of Sale (ROS) Data:** *What it tells us: The calculated speed at which each unique item (SKU) sells, providing a critical baseline for forecasting future demand and informing replenishment decisions.*
- **Identification of Non-Zero Sales Store-Styles:** *What it tells us: A list of specific stores and product styles that have demonstrated actual sales, guiding where replenishment efforts should be focused.*

Business Reports

- **Input for Sales Forecasting Module:** *Who uses it and why: The Sales Forecasting module consumes this cleaned sales data and initial ROS to develop more accurate predictions of future demand, helping merchants plan product purchases and marketing campaigns more effectively.*

- **Input for Depletion Planning Module:** *Who uses it and why: The Depletion Planning module uses the accurate ROS and clean sales history to project future inventory needs, allowing planners to identify potential stockouts and optimize stock levels across the network.*

8. Unusual Cases

Situation | What Happens |
 ROS Calculation Season | (No explicit default) | Defines the past selling season (e.g., “Fall 2023”, “Spring 2024”) whose sales data will be used to calculate Rate of Sale (ROS). This ensures that the velocity is based on relevant historical demand patterns. | When you want to update the historical context for ROS calculation, for example, to include the most recent full season or exclude very old seasons no longer representative of current demand. Changing this directly impacts the accuracy of demand forecasts. |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Raw Sales Transactions<br/>(Last 6 Months)] -->
    B{0tbDepletionPrepareSalesModule<br/>Sales Data Cleaning & ROS
    Calculation}
    B --> C[Cleaned Sales Data<br/>& Initial SKU Rate of Sale]
  
```

1. Start with → Raw Sales Transactions for an extended historical period (e.g., the last six months), including all product sales, regardless of their nature.
2. Then → Remove Liquidation Sales from this historical data, separating genuine demand from promotional or clearance-driven sales.
3. Next → Filter for "Brokenness" by analyzing the unliquidated sales within a more recent analysis period (e.g., last 30 days) against predefined Pivotal Size Availability (PSA) benchmarks, ensuring only sales from adequately stocked periods are retained.
4. Finally → Compute Initial SKU Rate of Sale from the comprehensive unliquidated sales data, and prepare the cleaned sales transactions for further analysis.

5. Step-by-Step Actions

1. **Initialize** module with business settings
 - What we do: *We load predefined settings like the desired sales analysis periods and the benchmarks for identifying “broken” sales data.*
 - Why we do it: *This ensures the module processes sales information according to current business rules and focuses on the most relevant periods and data quality standards.*
2. **Gather** comprehensive sales data
 - What we do: *We collect all raw sales transactions for every product (SKU) at every store over a specified historical period, typically the last six months.*

- Why we do it: *This provides a broad dataset for a comprehensive understanding of past product performance and sufficient volume for accurate rate calculations.*
- 3. **Remove** liquidation sales
 - What we do: *We identify and filter out any sales that occurred during liquidation events or deep clearance periods, based on specific system flags or price points.*
 - Why we do it: *Liquidation sales can dramatically inflate perceived demand, so removing them ensures that our sales velocity calculations reflect organic customer purchasing behavior, preventing over-ordering of regular-priced merchandise.*
- 4. **Filter** for “brokenness”
 - What we do: *We analyze the sales data from a recent period (e.g., last 30 days) and remove records for products that had insufficient stock levels (based on PSA benchmarks) during that time.*
 - Why we do it: *Sales records can be “broken” if customers couldn’t buy a product due to missing key sizes. Filtering these out ensures that the remaining sales accurately represent demand when products were adequately available, preventing underestimation of true sales potential.*
- 5. **Compute** initial SKU Rate of Sale (ROS)
 - What we do: *Using the cleaned, unliquidated sales data, we calculate an initial Rate of Sale for each unique product (SKU) at each store where non-zero sales occurred.*
 - Why we do it: *This provides the foundational velocity metric, indicating how quickly products are selling, which is essential for forecasting future demand, setting replenishment targets, and making informed inventory investment decisions.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A specific ‘Women’s Summer Sandal, Style S101’ (SKU S101-7-BLK for size 7, black) is being analyzed.
- Raw sales data shows 300 units of SKU S101-7-BLK sold in the last 6 months at Store 010.
- Of these, 50 units were sold during a “Summer Clearance” event flagged as liquidation.
- For the last 30 days, the Pivotal Size Availability (PSA) for Style S101 at Store 010 was only 40% (meaning key sizes were often out of stock).
- The “Broken Sales PSA Benchmark” is set to 60%.
- The “Broken Sales Day Benchmark” is 7 days.

What Happens

1. First, the module gathers all 300 units of sales for SKU S101-7-BLK from Store 010 over the last 6 months.
2. Then, it identifies and removes the 50 units sold during the “Summer Clearance” liquidation, leaving 250 units of unliquidated sales.

3. Next, it examines the unliquidated sales from the last 30 days. It sees that the PSA for Style S101 was only 40%, which is below the 60% “Broken Sales PSA Benchmark.” Because this low availability persisted for more than 7 days (the “Broken Sales Day Benchmark”), the module flags this period’s sales as “broken.” This means any sales data from this 30-day period will be treated with caution or excluded from primary ROS calculation for that specific SKU-store to avoid underestimating true demand.
4. Finally, the module computes an initial Rate of Sale (ROS) for SKU S101-7-BLK based on the *full 6 months of unliquidated sales data* (250 units). This initial ROS gives a baseline understanding of how quickly this sandal normally sells when not on clearance.

Final Outcome

The module provides a clean, accurate sales history for ‘Women’s Summer Sandal, Style S101’, excluding misleading liquidation sales and highlighting periods where sales were likely suppressed due to poor stock availability. This provides a reliable foundation for predicting future demand and planning replenishment orders.

Key Takeaway: This step ensures that sales forecasts are based on genuine, unhindered customer demand, preventing costly errors from misinterpreting sales data affected by clearances or stockouts.

7. Results

This step produces:

Main Outputs

- **Cleaned, Unliquidated Sales Data:** *What it tells us: A refined dataset of sales transactions, free from the distortions of liquidation events and periods of poor stock availability, providing a true picture of customer demand.*
- **Initial SKU Rate of Sale (ROS) Data:** *What it tells us: The calculated speed at which each unique item (SKU) sells, providing a critical baseline for forecasting future demand and informing replenishment decisions.*
- **Identification of Non-Zero Sales Store-Styles:** *What it tells us: A list of specific stores and product styles that have demonstrated actual sales, guiding where replenishment efforts should be focused.*

Business Reports

- **Input for Sales Forecasting Module:** *Who uses it and why: The Sales Forecasting module consumes this cleaned sales data and initial ROS to develop more accurate predictions of future demand, helping merchants plan product purchases and marketing campaigns more effectively.*
- **Input for Depletion Planning Module:** *Who uses it and why: The Depletion Planning module uses the accurate ROS and clean sales history to project future inventory needs, allowing planners to identify potential stockouts and optimize stock levels across the network.*

8. Unusual Cases

Situation | What Happens |

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Raw Sales Transactions<br/>(Last 6 Months)] --> B{0tbDepletionPrepareSalesModule<br/>Sales Data Cleaning & ROS Calculation}
```

```
B --> C[Cleaned Sales Data<br/>& Initial SKU Rate of Sale]
```

1. Start with → Raw Sales Transactions for an extended historical period (e.g., the last six months), including all product sales, regardless of their nature.
2. Then → Remove Liquidation Sales from this historical data, separating genuine demand from promotional or clearance-driven sales.
3. Next → Filter for "Brokenness" by analyzing the unliquidated sales within a more recent analysis period (e.g., last 30 days) against predefined Pivotal Size Availability (PSA) benchmarks, ensuring only sales from adequately stocked periods are retained.
4. Finally → Compute Initial SKU Rate of Sale from the comprehensive unliquidated sales data, and prepare the cleaned sales transactions for further analysis.

5. Step-by-Step Actions

1. **Initialize** module with business settings
 - What we do: *We load predefined settings like the desired sales analysis periods and the benchmarks for identifying "broken" sales data.*
 - Why we do it: *This ensures the module processes sales information according to current business rules and focuses on the most relevant periods and data quality standards.*
2. **Gather** comprehensive sales data
 - What we do: *We collect all raw sales transactions for every product (SKU) at every store over a specified historical period, typically the last six months.*
 - Why we do it: *This provides a broad dataset for a comprehensive understanding of past product performance and sufficient volume for accurate rate calculations.*
3. **Remove** liquidation sales
 - What we do: *We identify and filter out any sales that occurred during liquidation events or deep clearance periods, based on specific system flags or price points.*
 - Why we do it: *Liquidation sales can dramatically inflate perceived demand, so removing them ensures that our sales velocity calculations reflect organic customer purchasing behavior, preventing over-ordering of regular-priced merchandise.*

4. **Filter** for “brokenness”

- What we do: *We analyze the sales data from a recent period (e.g., last 30 days) and remove records for products that had insufficient stock levels (based on PSA benchmarks) during that time.*
- Why we do it: *Sales records can be “broken” if customers couldn’t buy a product due to missing key sizes. Filtering these out ensures that the remaining sales accurately represent demand when products were adequately available, preventing underestimation of true sales potential.*

5. **Compute** initial SKU Rate of Sale (ROS)

- What we do: *Using the cleaned, unliquidated sales data, we calculate an initial Rate of Sale for each unique product (SKU) at each store where non-zero sales occurred.*
- Why we do it: *This provides the foundational velocity metric, indicating how quickly products are selling, which is essential for forecasting future demand, setting replenishment targets, and making informed inventory investment decisions.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A specific ‘Women’s Summer Sandal, Style S101’ (SKU S101-7-BLK for size 7, black) is being analyzed.
- Raw sales data shows 300 units of SKU S101-7-BLK sold in the last 6 months at Store 010.
- Of these, 50 units were sold during a “Summer Clearance” event flagged as liquidation.
- For the last 30 days, the Pivotal Size Availability (PSA) for Style S101 at Store 010 was only 40% (meaning key sizes were often out of stock).
- The “Broken Sales PSA Benchmark” is set to 60%.
- The “Broken Sales Day Benchmark” is 7 days.

What Happens

1. First, the module gathers all 300 units of sales for SKU S101-7-BLK from Store 010 over the last 6 months.
2. Then, it identifies and removes the 50 units sold during the “Summer Clearance” liquidation, leaving 250 units of unliquidated sales.
3. Next, it examines the unliquidated sales from the last 30 days. It sees that the PSA for Style S101 was only 40%, which is below the 60% “Broken Sales PSA Benchmark.” Because this low availability persisted for more than 7 days (the “Broken Sales Day Benchmark”), the module flags this period’s sales as “broken.” This means any sales data from this 30-day period will be treated with caution or excluded from primary ROS calculation for that specific SKU-store to avoid underestimating true demand.
4. Finally, the module computes an initial Rate of Sale (ROS) for SKU S101-7-BLK based on the *full 6 months of unliquidated sales data* (250 units). This initial ROS gives a baseline understanding of how quickly this sandal normally sells when not on clearance.

Final Outcome

The module provides a clean, accurate sales history for ‘Women’s Summer Sandal, Style S101’, excluding misleading liquidation sales and highlighting periods where sales were likely suppressed due to poor stock availability. This provides a reliable foundation for predicting future demand and planning replenishment orders.

Key Takeaway: This step ensures that sales forecasts are based on genuine, unhindered customer demand, preventing costly errors from misinterpreting sales data affected by clearances or stockouts.

7. Results

This step produces:

Main Outputs

- **Cleaned, Unliquidated Sales Data:** *What it tells us: A refined dataset of sales transactions, free from the distortions of liquidation events and periods of poor stock availability, providing a true picture of customer demand.*
- **Initial SKU Rate of Sale (ROS) Data:** *What it tells us: The calculated speed at which each unique item (SKU) sells, providing a critical baseline for forecasting future demand and informing replenishment decisions.*
- **Identification of Non-Zero Sales Store-Styles:** *What it tells us: A list of specific stores and product styles that have demonstrated actual sales, guiding where replenishment efforts should be focused.*

Business Reports

- **Input for Sales Forecasting Module:** *Who uses it and why: The Sales Forecasting module consumes this cleaned sales data and initial ROS to develop more accurate predictions of future demand, helping merchants plan product purchases and marketing campaigns more effectively.*
- **Input for Depletion Planning Module:** *Who uses it and why: The Depletion Planning module uses the accurate ROS and clean sales history to project future inventory needs, allowing planners to identify potential stockouts and optimize stock levels across the network.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed sales data feed	The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations	Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions.

Situation	What Happens	What You Should Do
Sales data consistently flagged as “broken”	and flawed demand forecasts. A significant portion of sales data may be excluded or flagged, potentially leading to an underestimation of true demand or indicating a systemic issue with stock availability.	Review stock availability practices and the Broken Sales PSA Benchmark and Broken Sales Day Benchmark settings. This might indicate persistent stockout issues in stores or overly strict benchmarks, requiring investigation into operational efficiency or parameter adjustment.
Inaccurate liquidation flags on sales data	Liquidation sales may not be correctly identified and removed, leading to inflated ROS calculations and potential over-ordering of regular-priced merchandise.	Audit sales data for proper flagging of liquidation events. Work with merchandising and sales teams to ensure consistent and accurate identification of promotional and clearance periods in the data source.
Very low or zero sales for relevant SKUs	The module will calculate very low or zero ROS for these items, potentially leading to no replenishment recommendations, even if they are core products.	Confirm the Sales Data Look-back Period is sufficient. If accurate, this highlights underperforming products or specific store/SKU combinations that require merchandising review, promotional support, or potential delisting.

9. Next Steps

Before This Step

- **Master Data & Inventory Consolidation:** *What we need from it: Comprehensive product style and SKU details, up-to-date store configurations, and consolidated inventory data (on-hand, in-transit, open orders), providing the foundational information to contextualize sales.*
- **Season and Business Settings Configuration:** *What we need from it: Clearly defined relevant selling seasons and other business parameters, ensuring the sales analysis is focused on the correct timeframes and product ranges.*

After This Step

- **Sales Forecasting Module:** *What they need from us: The cleaned sales data and initial SKU Rate of Sale calculations to generate precise demand forecasts, guiding future purchasing and allocation strategies.*
- **Depletion Allocation Engine:** *What they need from us: The refined sales velocity data to inform how existing inventory should be distributed across the network, optimizing stock movement to meet demand effectively.*
- **Replenishment Recommendation Module:** *What they need from us: Accurate Rate of Sale data to determine optimal reorder points and quantities, ensuring stores receive the right products at the right time.*

10. Common Questions

Q: Why is it important to remove liquidation sales from the data? A: Removing liquidation sales ensures that your demand forecasts are based on genuine, full-price customer demand. Sales during clearance events are often driven by heavily discounted prices, not true product popularity, and including them would lead to overestimating regular demand and potentially over-ordering products at full cost.

Q: What does “brokenness” in sales data mean, and why is it filtered? A: “Brokenness” refers to sales data that is incomplete or distorted because the product was not adequately available (e.g., key sizes were out of stock). Filtering this data ensures that the calculated Rate of Sale reflects actual demand when the product *could* be bought, preventing underestimation of true sales potential due to store-level stock issues.

Q: How far back does this module look for sales data? A: This module typically looks back at raw sales data for an extended period, such as the last six months, to gather a comprehensive history. For more detailed analysis and “brokenness” filtering, it then focuses on a more recent period, like the last 30 days, to ensure current trends are accurately captured.

Q: How does the initial Rate of Sale (ROS) help the business? A: The initial Rate of Sale (ROS) provides a crucial metric for understanding how quickly products are selling. It’s the foundation for all demand forecasting, helping merchants predict how many units of an item will sell in a given period. This directly informs how much new stock to order, preventing stockouts and maximizing sales opportunities.

Q: Can I adjust the criteria for identifying “broken” sales data? A: Yes, the system allows you to adjust benchmarks like the Broken Sales PSA Benchmark (minimum Pivotal Size Availability) and Broken Sales Day Benchmark (minimum days of poor availability). These settings provide flexibility to define what constitutes “adequate” product availability, allowing the business to fine-tune the data cleaning process to its specific operational realities.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Cleans and refines raw sales data by removing liquidation sales and filtering for “brokenness,” then computes initial SKU Rate of Sale.

- **Main Input:** Historical sales transactions (up to 6 months), flags for liquidation sales, and Pivotal Size Availability benchmarks.
 - **Main Output:** Cleaned, unliquidated sales data and initial SKU Rate of Sale calculations.
 - **Warning Signs:** Unusually high sales velocity (possibly due to unremoved liquidation sales) or unexpectedly low sales for popular items (possibly due to unaddressed “brokenness”).
 - **Key Settings:** Sales Data Analysis Period, Sales Data Look-back Period, Broken Sales PSA Benchmark, Broken Sales Day Benchmark.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4:

OtbDepletionFirstMonthRosCalcModule

Last Updated: 2025-05-23

1. Business Objective

This module calculates the Rate of Sale (ROS) specifically for the most recent sales period (often referred to as the “first month of depletion”) for all relevant store-style combinations, ensuring the calculation accurately reflects immediate demand.

Why This Matters: By providing a highly relevant and up-to-date sales velocity, this module allows merchants and planners to make agile inventory decisions, ensuring that products are replenished based on current customer interest. This helps prevent stockouts on popular items, minimizes excess inventory for slower movers, and maximizes immediate sales opportunities and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Transactions (SKU-store level):** *Why we need it: To analyze actual past sales for individual items at specific stores, forming the basis for understanding recent demand patterns.*
- **Relevant Styles for ROS Computation:** *Why we need it: To focus the sales analysis only on product styles that are actively being considered for Rate of Sale calculations within the current business context, avoiding irrelevant or outdated merchandise.*

Product Data

- **Product Style Information:** *Why we need it: To group individual items (SKUs) into broader product styles, enabling the calculation of sales velocity at a more aggregated, style-level, which is often more useful for replenishment planning.*
- **Pivotal Size Availability (PSA) Data:** *Why we need it: To assess how well-stocked key sizes were for a style during past sales periods. This is crucial for filtering out sales data from times when products were not fully available, ensuring the calculated ROS reflects true demand.*

Operational Data

- **Store and Channel Configuration:** *Why we need it: To identify active retail locations and their associated sales channels, ensuring that ROS calculations are performed for all relevant points of sale.*
- **Live Style Indicators:** *Why we need it: To determine which styles are currently active and have inventory or are slated for replenishment, ensuring that ROS is calculated only for products that are currently in play for depletion planning.*

3. Business Settings

Setting Name | Default Value | Description | When to Change |

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Cleaned Sales Data  
(from Chapter 3)] --> B[First Month ROS  
Calculation]
```

```
B --> C[First Month Rate of Sale  
(for Style-Store  
Combinations)]
```

1. Start with → Cleaned Sales Data for all products over a recent period, already filtered to exclude misleading liquidation sales and identified “broken” sales periods.
2. Then → Filter these sales further, specifically for the period defined as the “first month of depletion” and for days where Pivotal Size Availability (PSA) benchmarks were met.
3. Next → Aggregate the sales data to the product style level for each store and validate against minimum sales activity thresholds.
4. Finally → Produce the calculated “first month” Rate of Sale for each relevant store-style combination, indicating current sales velocity.

5. Step-by-Step Actions

1. **Initialize** the module with business settings
 - What we do: *We load specific configuration values, such as the exact period length for “first month” ROS calculations and the minimum Pivotal Size Availability (PSA) required for sales data to be considered valid.*
 - Why we do it: *This ensures that the calculation is tailored to the business’s current strategy for immediate demand forecasting and data quality standards.*
2. **Gather** relevant sales data for the first month
 - What we do: *We collect historical sales transactions for the specified recent period, ensuring that only sales for product styles marked as “relevant for ROS computation” are included.*
 - Why we do it: *This focuses the analysis on the most recent and pertinent sales history, providing an accurate snapshot of current demand trends.*

3. **Validate** sales against Pivotal Size Availability (PSA)
 - What we do: *For each store-style-day combination, we check if the product met a predefined minimum Pivotal Size Availability (PSA) benchmark. Sales from days where PSA was not met are excluded from ROS calculation.*
 - Why we do it: *This critical step ensures that the calculated Rate of Sale reflects genuine demand, not sales suppressed by stockouts of key sizes, preventing underestimation of a product's true sales potential.*
4. **Aggregate** units sold and count valid sales days
 - What we do: *For each store and style, we sum up the units sold and count the number of days where sales occurred and the PSA condition was satisfied within the defined "first month" period.*
 - Why we do it: *This prepares the necessary components for calculating the actual Rate of Sale, ensuring only high-quality data contributes to the velocity metric.*
5. **Compute** Rate of Sale (ROS) for store-style combinations
 - What we do: *Using the aggregated units sold and the count of valid sales days, we calculate the Rate of Sale for each store-style combination. A minimum number of valid sales days must be present for a ROS to be computed.*
 - Why we do it: *This produces the primary output of the module: a reliable, recent sales velocity for each product style at each store, which is essential for immediate inventory decisions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- We are analyzing 'Women's Athletic Shorts, Style A200' at Store 005.
- The "ROS Calculation Period Length" is set to 30 days.
- The "Pivotal Size Availability (PSA) Benchmark" is 70%.
- The "Minimum Valid Sales Days for ROS" is 15 days.
- Over the last 30 days for Style A200 at Store 005:
 - Total units sold: 120 units.
 - Number of actual sales days: 25 days.
 - PSA performance:
 - Days 1-10: PSA was 85% (good). Sales: 60 units.
 - Days 11-15: PSA was 55% (below benchmark). Sales: 10 units.
 - Days 16-25: PSA was 78% (good). Sales: 50 units.

What Happens

1. First, the module focuses on the last 30 days of sales for 'Women's Athletic Shorts, Style A200' at Store 005, identifying it as a relevant style for ROS computation.
2. Next, it reviews the Pivotal Size Availability (PSA) for each day. It excludes sales from Days 11-15 because the PSA of 55% is below the 70% benchmark. This means those 10 units sold during that period are not counted towards the true ROS, as demand might have been suppressed due to insufficient key sizes.

3. The module then aggregates the sales from the “valid” days (Days 1-10 and Days 16-25). This totals 60 units + 50 units = 110 units sold across 10 + 10 = 20 valid sales days.
4. Since 20 valid sales days are greater than the “Minimum Valid Sales Days for ROS” of 15, the module proceeds to calculate the Rate of Sale: 110 units / 20 days = 5.5 units/day.

Final Outcome

The module provides a first month Rate of Sale (ROS) of 5.5 units/day for ‘Women’s Athletic Shorts, Style A200’ at Store 005. This highly accurate and recent velocity metric is now available to guide immediate inventory decisions, such as determining how quickly to replenish this style.

Key Takeaway: This step ensures that immediate sales velocity is calculated based on realistic demand, excluding periods where products were not adequately available to customers.

7. Results

This step produces:

Main Outputs

- **First Month Rate of Sale (ROS) for Style-Store Combinations:** *What it tells us: The calculated speed at which each product style is currently selling at individual stores, providing the most up-to-date sales velocity for immediate replenishment needs.*
- **Associated ROS Calculation Details:** *What it tells us: Supplementary information for each ROS value, such as the total units sold during the valid period, the number of valid sales days, and the rollup level (e.g., style-level), providing context for the calculation.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine uses this current Rate of Sale to determine the optimal movement of existing inventory between stores and warehouses, ensuring products are distributed to meet immediate demand.*
- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes this up-to-date ROS to fine-tune reorder points and quantities, ensuring that stores receive the right amount of new stock to capitalize on current sales trends.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed	The module will operate on	Verify data source connections for sales transactions. Check data ingestion logs and re-

Situation	What Happens	What You Should Do
sales data feed	incomplete or stale sales data, leading to inaccurate Rate of Sale calculations.	trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions.
No valid sales days for a relevant style	If a style has sales, but none of the sales days meet the Pivotal Size Availability (PSA) Benchmark, or if there are too few valid sales days, no ROS will be calculated for that style at that store.	Review the Pivotal Size Availability (PSA) Benchmark and Minimum Valid Sales Days for ROS settings. This might indicate persistent stockout issues or overly strict benchmarks, requiring investigation into operational efficiency or parameter adjustment.
Consistent zero sales for a relevant style	The module will calculate a zero Rate of Sale (ROS) for that style at that store.	If this is an unexpected result for a core product, investigate the store's performance or product visibility. For underperforming items, this ROS indicates no immediate replenishment is needed, or the product may be phased out.
Inaccurate Pivotal Size Availability (PSA) data	If PSA data is incorrect, sales might be inappropriately excluded or included, leading to a skewed ROS calculation.	Verify the accuracy of PSA data inputs. Collaborate with merchandising operations to ensure that inventory availability metrics are correctly captured and integrated into the system.

9. Next Steps

Before This Step

- **OtbDepletionPrepareSalesModule:** *What we need from it: Cleaned sales data (free from liquidation events and initial “brokenness” filtering) and initial SKU-level Rate of Sale calculations, providing the raw material for our recent sales velocity calculation.*
- **OtbDepletionInputModule:** *What we need from it: Definition of relevant styles for ROS computation and current business settings, providing the overall context for which products and periods to analyze.*
- **OtbDepletionPrepareDataModule:** *What we need from it: Information on “live styles” and active store configurations, ensuring that ROS is calculated only for currently active product-store combinations.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The calculated “first month” Rate of Sale values to inform the immediate movement of existing inventory across the network, optimizing stock for current demand.*
- **Replenishment Planning Module:** *What they need from us: Accurate, recent Rate of Sale data to determine how much new stock should be ordered for specific styles and stores, ensuring optimal inventory levels going forward.*
- **Sales Performance Reporting:** *What they need from us: The calculated ROS data can be used in business intelligence tools to visualize current sales trends and product performance at a granular level.*

10. Common Questions

Q: Why is it important to filter sales data based on Pivotal Size Availability (PSA)?

A: Filtering by PSA ensures that the calculated Rate of Sale reflects true customer demand, not sales that were suppressed because key sizes were out of stock. If customers couldn't buy a product due to missing sizes, including those days would artificially lower the perceived demand, leading to under-replenishment.

Q: How does this “first month” ROS differ from other sales velocity metrics? A:

This “first month” Rate of Sale focuses on the *most recent* sales period, providing a highly current snapshot of demand. Unlike longer-term historical averages or initial SKU-level ROS, it's specifically designed to capture immediate trends and inform agile inventory decisions for the current depletion cycle.

Q: What happens if a style has no sales in the recent “first month” period? A: If a style has no sales or too few valid sales days within the defined “first month” period, this module will report a zero (or very low) Rate of Sale. This signals that the product is either slow-moving or has no current demand, which will influence subsequent replenishment and allocation decisions, potentially minimizing new orders.

Q: How does this Rate of Sale directly impact our inventory investment? A: The Rate of Sale is a direct input for replenishment calculations. A high ROS indicates strong demand, prompting higher replenishment quantities to avoid lost sales. A low ROS signals slower movement, leading to reduced replenishment to prevent excess inventory, directly optimizing your inventory investment and reducing holding costs.

Q: Can the length of the “first month” period be adjusted? A: Yes, the “ROS Calculation Period Length” setting allows you to define how many days of recent sales history are included in this calculation. This flexibility enables businesses to fine-tune the module to their specific product lifecycle and demand volatility, ensuring the ROS is always relevant to their operational pace.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates the Rate of Sale (ROS) for the most recent period for store-style combinations, reflecting immediate demand.

- **Main Input:** Cleaned historical sales data, Pivotal Size Availability (PSA) information, and lists of relevant/live styles.
 - **Main Output:** First Month Rate of Sale (ROS) for each relevant store-style combination.
 - **Warning Signs:** Unexpectedly low ROS for popular items (could indicate unaddressed stock issues) or zero ROS for core products.
 - **Key Settings:** ROS Calculation Period Length, Pivotal Size Availability (PSA) Benchmark, Minimum Valid Sales Days for ROS.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5:

OtbDepletionSalesCalcModule

Last Updated: 2025-05-23

1. Business Objective

This module precisely forecasts the sales quantities for each unique combination of store, product category, and planning period within the depletion horizon.

Why This Matters: By accurately predicting future sales, this module enables businesses to make smarter inventory investments, ensuring that the right amount of product is available at each store. This minimizes stockouts, reduces excess inventory, and maximizes sales and profitability by aligning stock levels directly with anticipated customer demand.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Average Selling Price (ASP) by Period, Store, and Attribute Group (AG):** *Why we need it: To understand the typical price at which products in a given category are sold, which is essential for converting revenue targets into sales quantities. This allows us to calculate how many units we expect to sell to hit a revenue goal.*

Product Data

- **OD (Optimum Depth) Segmentation Output - Revenue Contribution:** *Why we need it: To understand what percentage of a store's total revenue each product category (Attribute Group - AG) is expected to contribute based on its optimal depth. This helps us proportionally allocate overall sales targets to specific categories.*
- **Relevant Styles for Sales Forecasting:** *Why we need it: To identify which product styles are actively considered for sales forecasting, ensuring that sales quantities are calculated only for merchandise that is currently relevant or part of the depletion planning horizon.*

Operational Data

- **Annual Operating Plan (AOP) Revenue Targets:** *Why we need it: To provide the overall, top-level revenue goals for each store and planning period. These high-level targets are then broken down into specific sales quantities for categories.*
- **Defined Planning Periods and Stores:** *Why we need it: To ensure that sales quantities are calculated consistently for all relevant stores and across the entire duration of the depletion planning horizon, providing a complete forecast timeline.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
AOP Input Period Granularity	Monthly	Defines how the Annual Operating Plan (AOP) revenue targets are broken down into smaller periods (e.g., weekly or monthly).	When the business requires more granular or less granular sales quantity forecasts. For example, if you need weekly replenishment decisions, you'd set this to 'Weekly'.
OD Revenue Contribution Basis	Forecasted	Specifies whether the Optimum Depth (OD) revenue contribution percentages are derived from historical performance or a forward-looking forecast.	If the basis for OD segmentation changes (e.g., from historical actuals to a predictive model) to ensure the contribution percentages are aligned with the source data.
Sales Quantity Rounding Rule	Nearest Whole Unit	Determines how the calculated sales quantities (which can be decimal) are rounded for practical inventory management.	If the business prefers to order in half units, or if specific product types require precise fractional quantities for replenishment, such as for bulk goods.
Depletion Horizon Start Date	(No explicit default)	Sets the starting point for the entire depletion planning horizon, influencing	When the financial planning calendar shifts or if you need to adjust the look-forward period for depletion. Incorrect

Setting Name	Default Value	Description	When to Change
		which planning periods are included in the sales quantity calculation.	settings can lead to forecasts for irrelevant periods.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[AOP Revenue Targets
OD Revenue Contribution
Average Selling Price (ASP)] --> B{0tbDepletionSalesCalcModule
Sales Quantity Forecasting}

B --> C[Forecasted Sales Quantities
(Store-Category-Period)]

1. Start with → AOP Revenue Targets (overall sales goals), OD Revenue Contribution (how much each category contributes to those goals), and Average Selling Price (ASP) for each product category.
2. Then → Iterate through each store and planning period, identifying relevant product categories (Attribute Groups - AGs) that are active for sales forecasting.
3. Next → Calculate the forecasted sales quantity for each store-category-period combination by dividing the portion of the AOP revenue allocated to that category by its Average Selling Price.
4. Finally → Persist these calculated sales quantities, making them available for subsequent inventory planning and replenishment decisions.

5. Step-by-Step Actions

1. **Gather** OD Segmentation Output and AOP Targets
 - What we do: *We retrieve the calculated revenue contribution percentages for each product category (Attribute Group) at each store for every planning period, along with the overall Annual Operating Plan (AOP) revenue targets.*
 - Why we do it: *These are foundational inputs for determining how much sales revenue is expected for each product category within a store, which is then used to derive unit sales.*
2. **Identify** Relevant Product Categories (Attribute Groups)
 - What we do: *For each store, we determine which product categories are relevant for sales forecasting based on the styles currently considered for replenishment.*
 - Why we do it: *This ensures that we only calculate sales quantities for categories that are actively part of the merchandise assortment, focusing our efforts on relevant products.*
3. **Retrieve** Average Selling Prices (ASP)
 - What we do: *We pull the Average Selling Price for each relevant product category (Attribute Group) at each store for every planning period.*

- Why we do it: *ASP is crucial for converting a forecasted revenue amount into a tangible number of units, allowing us to understand the physical quantity of product expected to sell.*
- 4. **Calculate** Forecasted Sales Quantities
 - What we do: *For every combination of store, product category, and planning period, we multiply the AOP revenue by the category's revenue contribution percentage, then divide this result by the category's Average Selling Price.*
 - Why we do it: *This core calculation translates financial goals into precise unit sales forecasts, which is the basis for all downstream inventory planning and replenishment decisions.*
- 5. **Persist** Calculated Sales Quantities
 - What we do: *We save the newly calculated sales quantities to the system's database in a structured format.*
 - Why we do it: *This ensures that the forecasted sales quantities are recorded and readily available for other modules, such as replenishment planning and performance reporting, providing a consistent source of demand information.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Store:** Store 001
- **Category (AG):** 'Women's Casual Tops' (AG 05)
- **Planning Period:** Week 25
- **Annual Operating Plan (AOP) Revenue for Store 001, Week 25:** \$50,000
- **OD Segmentation Output (Revenue Contribution) for AG 05 in Store 001, Week 25:** 10% (meaning 'Women's Casual Tops' is expected to contribute 10% of total revenue)
- **Average Selling Price (ASP) for AG 05 in Store 001, Week 25:** \$25.00

What Happens

1. First, the module retrieves the AOP revenue target (\$50,000) for Store 001 for Week 25. It also gets the 10% revenue contribution for 'Women's Casual Tops' (AG 05) from the OD segmentation output.
2. Next, it calculates the expected revenue specifically for 'Women's Casual Tops' in Store 001 for Week 25: $\$50,000 \text{ (AOP)} \times 10\% \text{ (Revenue Contribution)} = \$5,000$.
3. Then, the module retrieves the Average Selling Price (ASP) for 'Women's Casual Tops' for that period and store, which is \$25.00.
4. Finally, it calculates the forecasted sales quantity: $\$5,000 \text{ (Expected Category Revenue)} / \$25.00 \text{ (ASP)} = 200 \text{ units}$.

Final Outcome

The module forecasts that Store 001 is expected to sell 200 units of ‘Women’s Casual Tops’ during Week 25. This precise unit forecast now informs how many units of this category should be stocked and replenished to meet the anticipated customer demand for that period.

Key Takeaway: This step translates high-level financial goals and strategic category importance into actionable unit sales forecasts for specific products at specific stores.

7. Results

This step produces:

Main Outputs

- **Forecasted Sales Quantities (Store-Category-Period):** *What it tells us: The predicted number of units expected to sell for each product category (Attribute Group) at every store within each planning period of the depletion horizon, providing a clear unit-based demand signal.*
- **Exported Sales Quantity Data:** *What it tells us: A formatted version of the forecasted sales quantities suitable for reporting and integration with other business intelligence tools, enabling easy analysis and visualization of sales predictions.*

Business Reports

- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes these forecasted sales quantities to determine optimal reorder points and quantities, ensuring stores receive enough stock to meet anticipated demand without overstocking.*
- **Demand Forecasting Performance Reports:** *Who uses it and why: Merchandising and planning teams use these forecasts to compare against actual sales, evaluating the accuracy of the prediction models and identifying areas for improvement in future forecasting processes.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will operate with incomplete or stale AOP, ASP, or OD segmentation data, leading to inaccurate or missing sales quantity forecasts.	Verify data source connections for AOP, ASP, and OD segmentation outputs. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions.

Situation	What Happens	What You Should Do
Zero or unusually low Average Selling Price (ASP)	Division by zero errors may occur, or the calculated sales quantities will be unrealistically high, leading to over-ordering.	Verify the accuracy of ASP data. Ensure all relevant product categories have valid, non-zero ASPs. Investigate the source of the incorrect ASP data.
Zero or negative Annual Operating Plan (AOP) revenue or OD Revenue Contribution	The calculated sales quantities will be zero or negative for those store-category-periods, signaling no expected sales for what might be a core product.	Review the AOP targets and OD segmentation outputs. Confirm that these inputs are complete, accurate, and reflect appropriate revenue expectations for all active store-category combinations.
No relevant product categories identified for a store	The module will not calculate any sales quantities for that store, even if an AOP exists, leading to no inventory allocation for key categories.	Verify that product styles are correctly associated with Attribute Groups and that the Relevant Styles for Sales Forecasting input is complete for all active stores.

9. Next Steps

Before This Step

- **OtbDepletionInputModule:** *What we need from it: The overall business settings and relevant style lists for sales computation, providing the foundational context for the module's operations.*
- **OtbDepletionPrepareDataModule:** *What we need from it: Information on active stores and categories (Attribute Groups), ensuring the module targets the correct sales channels and product groupings.*
- **OD (Optimum Depth) Segmentation Module:** *What we need from it: The calculated finalStoreRevContri (revenue contribution percentages) for each store-AG-period combination, which is a critical input for breaking down AOP into category-level revenue.*
- **Average Selling Price (ASP) Data:** *What we need from it: Accurate Average Selling Prices for products by period, store, and attribute group, which is essential for converting revenue to units.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The forecasted sales quantities for each store-category-period to guide the optimal distribution of existing inventory across the network, ensuring stock moves to where demand is highest.*
- **Replenishment Planning Module:** *What they need from us: The calculated sales quantities to determine precise reorder points and order quantities,*

ensuring that stores receive the right amount of new stock to meet future demand.

- **Financial Planning and Analysis (FP&A):** *What they need from us: The detailed sales quantity forecasts can be used to reconcile against broader financial plans and assess the operational feasibility of revenue targets.*

10. Common Questions

Q: How does this module differ from other sales forecasting methods? A: This module focuses on translating high-level Annual Operating Plan (AOP) revenue targets into granular unit sales quantities for specific product categories at individual stores. It uses pre-calculated revenue contributions from the Optimum Depth (OD) segmentation to break down the overall plan, rather than relying solely on historical sales trends.

Q: What is the role of “revenue contribution” from OD segmentation in this module? A: The revenue contribution from OD segmentation tells the module how important each product category (Attribute Group) is to a store’s overall revenue, based on its optimal product assortment. This percentage is applied to the store’s total AOP revenue, ensuring that higher-contributing categories receive a proportionally larger share of the forecasted unit sales.

Q: What happens if the Average Selling Price (ASP) for a category is incorrect? A: If the ASP is too high, the module will forecast fewer units than actually needed to hit the revenue target, potentially leading to under-replenishment and lost sales. If the ASP is too low, it will forecast too many units, resulting in over-ordering and increased holding costs. Accurate ASP is vital for correct quantity translation.

Q: Can this module forecast sales for new products with no historical data? A: This module primarily relies on AOP targets, OD revenue contributions, and ASP, which can be set for new categories or stores. While it doesn’t directly use historical sales data, the inputs (AOP, OD, ASP) *can* be informed by similar product launches or strategic assumptions for new items, allowing for a preliminary sales quantity forecast.

Q: Why are both “store” and “category” important for sales quantity calculation? A: Sales performance varies significantly by location and product type. By calculating sales quantities at the store-category level, the module accounts for these differences. This ensures that a popular item in one store is forecasted differently than a slower-moving item in another, leading to highly localized and accurate inventory plans.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Forecasts unit sales quantities for each store-category-period by translating AOP revenue using OD segmentation contributions and Average Selling Prices.
- **Main Input:** Annual Operating Plan (AOP) revenue, OD (Optimum Depth) revenue contribution percentages, and Average Selling Price (ASP) data.

- **Main Output:** Forecasted sales quantities (units) for each store-category-period combination.
 - **Warning Signs:** Unrealistic sales quantity forecasts (e.g., extremely high or zero) or system errors related to division by zero.
 - **Key Settings:** AOP Input Period Granularity, OD Revenue Contribution Basis, Sales Quantity Rounding Rule.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6:

OtbDepletionPeriodRosCalcModule

Last Updated: 2025-05-23

1. Business Objective

This module calculates the expected sales velocity (Rate of Sale - ROS) for every unique product (SKU) across all stores and for all future replenishment periods, building on the immediate sales performance of a style and ensuring the right mix of sizes.

Why This Matters: By providing a highly granular and forward-looking view of demand, this ensures inventory decisions are precise, allowing businesses to stock the right mix of sizes and styles. This minimizes stockouts, prevents overstocking, and maximizes sales and profitability by aligning inventory directly with anticipated customer purchasing patterns.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **First Month's Style ROS:** *Why we need it: This provides the most recent, true sales velocity for each product style, serving as the foundational baseline for projecting future demand across all depletion periods.*
- **Forecasted Category Sales Quantities:** *Why we need it: These broader category-level sales targets allow us to scale the style-level Rate of Sale for future periods, ensuring that anticipated shifts in overall category demand are reflected in the SKU-level forecasts.*

Product Data

- **Product Style and SKU Information:** *Why we need it: To accurately link individual unique items (SKUs) to their broader product styles, enabling the system to break down style-level sales velocities into precise SKU-level demand signals.*
- **Ideal Size Set (ISS) Output (Adjusted Size Contributions):** *Why we need it: To proportionally distribute a style's overall Rate of Sale among its individual sizes (SKUs) based on the ideal sales mix for that style, ensuring replenishment aligns with what customers actually buy.*

- **Attribute Group (AG) Mappings:** *Why we need it: To understand which broader product category each style belongs to, which is essential for applying category-level sales quantity forecasts when scaling style ROS across different periods.*

Operational Data

- **Store Configuration and Distribution Network Details:** *Why we need it: To identify all active retail stores requiring inventory management, including those newly opened or with limited sales history, ensuring comprehensive ROS calculation across the entire network.*
- **New Store Reference Data:** *Why we need it: To know which established, similar stores new or low-sales stores should mirror, enabling the replication of reliable sales velocities for locations that lack sufficient historical data.*
- **Defined Planning Periods:** *Why we need it: To structure the depletion horizon and ensure that Rate of Sale calculations are performed consistently for each specific future period, providing a clear timeline for inventory planning.*

3. Business Settings

Setting Name | Default Value | Description | When to Change |
 Current Period Reference Date | (No explicit default) | Determines which specific date within the Depletion Horizon Start Date's current period should be used as a key reference point for calculations (e.g., the period end date). | If the way current periods are defined shifts (e.g., from period start to period end), or if a different specific date within the period is required for system reference. Rarely changed. |

SKU ROS Look-back Period | 90 | Defines the number of recent days of historical sales data to consider when determining if a new store has sufficient sales to calculate its own Rate of Sale (ROS). | When the business wants to be more or less strict about how much sales history is required for a store to be considered "established" enough for self-generated ROS. |

Explanation of Values: This provides further detail on what determines the flexibility in forecasting the number of days for sales velocity. For example, a setting could be the minimum number of days of recent sales to be considered "reliable" for a new store to generate its own ROS. The value itself (e.g., 90 days) defines this threshold. | Defines how many recent days of historical sales data are typically considered reliable for a store to calculate its own Rate of Sale (ROS). If a store (especially a new one) doesn't have sales over this many days, its ROS might be based on a reference store instead. | When the business wants to adjust the confidence level for a store's self-generated sales velocity, e.g., if you want more (or less) historical data to ensure stability in the ROS. |

Minimum Sales Days for New Stores | 10 | This specifies the minimum number of active sales days a new store must have within the SKU ROS Look-back Period to be considered capable of generating its own ROS. If it falls below this, ROS will be replicated from a reference store. | If the business wants to be more (or less) conservative about when new stores start to "stand on their own" in terms of sales forecasting. This impacts how quickly new stores get their own demand signals. |

Depletion Horizon Start Date | (No explicit default) | This setting defines the first day of the entire depletion planning horizon. The "first month" for ROS calculations will align with the period containing this date. | When the financial or inventory planning calendar shifts. Incorrect settings can lead to ROS calculations focusing on

irrelevant past periods or not extending far enough into the future. |
ROS Calculation Period Length | 30 | This refers to the number of recent days of sales history used to calculate the “First Month’s Style ROS,” which serves as a baseline for scaling across future periods in this module. | This setting is primarily controlled by the `OtbDepletionFirstMonthRosCalcModule`, but it directly influences the baseline sales velocity this module uses. Changing it would affect how current the initial ROS signal is. |

Note: The module implicitly uses the ‘Pivotal Size Availability (PSA) Benchmark’ from the previous `OtbDepletionFirstMonthRosCalcModule` which ensures that the “First Month ROS” it receives as input accurately reflects genuine demand.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[First Month Style ROS<br/>(from Chapter 4)] -->
    B{OtbDepletionPeriodRosCalcModule<br/>Compute Period ROS}
    C[Ideal Size Set (ISS) Data<br/>Forecasted Category Sales
    Quantities<br/>New Store Reference Data] --> B
    B --> D[SKU-level Rate of Sale<br/>for All Depletion Periods]
  
```

1. Start with → First Month's Style ROS (the recent sales velocity for each style at each store), Ideal Size Set (ISS) data (which specifies the adjusted sales contribution for each size within a style), Forecasted Category Sales Quantities (future sales targets for broader product categories), and New Store Reference Data (which new stores should mirror which established stores).
2. Then → Calculate the expected sales velocity for each product style for *all* future depletion periods, scaling it based on category sales quantity ratios relative to the first month and the duration of each period.
3. Next → Break Down these style-level sales velocities to individual SKU-level rates using the adjusted size contributions from the Ideal Size Set (ISS) data, then Normalize these SKU rates to ensure they add up correctly to the style-level rate.
4. Finally → Replicate the calculated SKU-level Rate of Sale for new stores or stores with insufficient sales history by using data from their assigned reference stores, producing a comprehensive set of SKU-level Rate of Sale figures for all stores and periods.

5. Step-by-Step Actions

1. **Calculate** Style-Level ROS for All Depletion Periods
 - What we do: *We determine the expected sales velocity for each product style at every store for all future planning periods. This is done by taking the “first month” (most recent) style ROS and adjusting it based on how the overall category sales quantities are expected to change in subsequent periods, also accounting for the duration of each period.*
 - Why we do it: *While the first month ROS is crucial for immediate demand, sales patterns can shift over longer periods. This step scales that initial velocity to reflect anticipated future demand for the entire depletion*

horizon, ensuring long-term replenishment plans are accurate and adapt to changing trends.

2. **Compute** Initial SKU-Level ROS from Style ROS and ISS

- What we do: *For each style and period, we use the Ideal Size Set (ISS) data, specifically the “adjusted size contribution” for each size, to break down the style’s total Rate of Sale (ROS) into an initial ROS for each individual SKU (e.g., specific size and color).*
- Why we do it: *A style’s sales velocity is made up of its individual SKUs. ISS tells us the ideal proportion of sales for each size within a style, allowing us to accurately distribute the overall style demand to its component SKUs, ensuring stores stock the right size mix that maximizes sales and minimizes leftover odd sizes.*

3. **Normalize** SKU-Level ROS

- What we do: *We proportionally adjust the calculated SKU-level ROS values so that their sum perfectly matches the total Rate of Sale previously determined for their respective style. This ensures that the individual SKU sales rates are consistent with the overall style’s performance.*
- Why we do it: *This normalization is crucial to prevent discrepancies in demand forecasts. Without it, the sum of individual SKU sales might not align with the style’s total sales, leading to over or underestimation of demand for specific sizes and overall inefficient inventory management.*

4. **Replicate** ROS for New or Low-Sales Stores

- What we do: *For new stores, or existing stores that don’t have enough recent sales data (as defined by the “Minimum Sales Days for New Stores” setting) to calculate their own reliable ROS, we copy the calculated SKU-level Rate of Sale from their pre-assigned “reference” stores.*
- Why we do it: *New stores often lack sufficient historical sales. By leveraging data from similar, established stores, this ensures that new locations also receive accurate demand signals for replenishment, enabling them to be stocked effectively from day one and avoid lost sales due to lack of historical data.*

5. **Store** Calculated SKU Rate of Sale

- What we do: *We save all the computed SKU-level Rate of Sale figures (for all stores and all depletion periods) into the system’s database.*
- Why we do it: *This makes the precise, forward-looking demand data readily available for subsequent modules, serving as the foundation for all inventory allocation and replenishment recommendations, ensuring data consistency and accessibility for other critical business processes.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Style:** ‘Men’s Classic Polo, Style P100’
- **Store:** Store 015
- **First Month (Period 1) ROS for Style P100 at Store 015:** 20 units/week (from `0tbDepletionFirstMonthRosCalcModule`)
- **Attribute Group (AG) for Style P100:** ‘Men’s Tops’ (AG 01)
- **Forecasted Sales Quantity for ‘Men’s Tops’ (AG 01) at Store 015:**
 - Period 1 (First Month): 500 units for this period

- Period 2: 550 units for this period (10% expected increase from Period 1)
- Period 3: 450 units for this period (10% expected decrease from Period 1)
- **Period Durations:** Assume each period is 30 days long.
- **Ideal Size Set (ISS) Output for ‘Men’s Tops’ (AG 01) at Store 015:**
 - SKU P100-S (Small): 20% Adjusted Contribution
 - SKU P100-M (Medium): 40% Adjusted Contribution
 - SKU P100-L (Large): 30% Adjusted Contribution
 - SKU P100-XL (X-Large): 10% Adjusted Contribution (Total 100%)
- **New Store Scenario:** Store 020 is a new store, referenced to Store 015. It has had only 3 valid sales days in the past 90 days, which is below the Minimum Sales Days for New Stores setting (e.g., 10 days).

What Happens

1. Calculate Style-Level ROS for all Periods:

- For Style P100 at Store 015:
 - **Period 1 (First Month):** The ROS is directly taken as 20 units/week.
 - **Period 2:** The sales quantity ratio (qtyRatio) for ‘Men’s Tops’ from Period 1 to Period 2 is $550 \text{ units} / 500 \text{ units} = 1.1$. The period duration ratio (periodDurationRatio) is $30 \text{ days} / 30 \text{ days} = 1$. The Style ROS for Period 2 is then calculated as $20 \text{ units/week} * 1.1 * 1 = 22 \text{ units/week}$.
 - **Period 3:** The qtyRatio for ‘Men’s Tops’ from Period 1 to Period 3 is $450 \text{ units} / 500 \text{ units} = 0.9$. The periodDurationRatio is $30 \text{ days} / 30 \text{ days} = 1$. The Style ROS for Period 3 is then calculated as $20 \text{ units/week} * 0.9 * 1 = 18 \text{ units/week}$.

2. Compute Initial SKU-Level ROS and Normalize:

- For Style P100 at Store 015, Period 1 (Style ROS = 20 units/week):
 - Initial SKU P100-S ROS: $20 * 0.20 = 4 \text{ units/week}$
 - Initial SKU P100-M ROS: $20 * 0.40 = 8 \text{ units/week}$
 - Initial SKU P100-L ROS: $20 * 0.30 = 6 \text{ units/week}$
 - Initial SKU P100-XL ROS: $20 * 0.10 = 2 \text{ units/week}$
 - (In this case, the sum of initial SKU ROS ($4+8+6+2=20$) perfectly matches the style ROS, so no further normalization adjustment is needed. If the ISS contributions didn’t sum to exactly 1.0, a slight adjustment would occur to ensure the total is 20 units/week.)
- Similar SKU-level calculations are performed for Period 2 (summing to 22 units/week) and Period 3 (summing to 18 units/week) for all SKUs within Style P100.

3. Replicate ROS for New Stores:

- For Store 020 (new store, referenced to Store 015), since it has only 3 valid sales days (below the 10-day minimum), it doesn’t have enough data to calculate its own reliable ROS.
- The module copies the calculated SKU-level ROS values for Style P100 (e.g., 4 units/week for SKU P100-S in Period 1) from Store 015 to Store 020 for all depletion periods. This means Store 020 will now have a forecasted demand for these products, even without its own sales history.

Final Outcome

The module produces a detailed forecast of how many units of each specific size (SKU) of ‘Men’s Classic Polo, Style P100’ are expected to sell per week at Store 015 (and Store 020) for every future depletion period. For example, it now knows

that Store 015 is expected to sell 8 units/week of SKU P100-M in Period 1, 8.8 units/week in Period 2, and 7.2 units/week in Period 3. Store 020 will have identical ROS figures to Store 015 due to replication, enabling its initial stocking and replenishment.

Key Takeaway: This step ensures that future inventory plans are based on accurate, SKU-specific demand forecasts that account for both recent performance, broader category trends, ideal size mixes, and the unique needs of new stores, leading to optimized inventory allocation.

7. Results

This step produces:

Main Outputs

- **SKU-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The precise, projected sales velocity for every unique product (SKU) at each store for all future planning periods, serving as the most granular and actionable demand signal for replenishment.*
- **Style-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The aggregated projected sales velocity for each product style at every store for all future planning periods, providing a higher-level view of demand and contextualizing the SKU-level rates.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine consumes these precise SKU and style ROS figures to determine the optimal movement of existing inventory between stores and warehouses, ensuring products are distributed to where they will sell fastest and meet customer demand.*
- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes this granular ROS data to calculate optimal reorder points and quantities for new stock, ensuring that stores have the right products in the right sizes at the right time, minimizing stockouts and excess inventory.*
- **Demand Forecast Accuracy Reports:** *Who uses it and why: Merchandising and planning teams use these ROS forecasts to compare against actual sales, evaluating the effectiveness of the prediction models and identifying opportunities for improvement in future demand predictions, leading to continuous optimization of inventory strategies.*

8. Unusual Cases

Situation | What Happens | What You Should Do |
Missing or delayed sales data feed | The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations. | Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream

inventory decisions. |

No Ideal Size Set (ISS) data for a style/category | The system will treat all SKUs within that style/category as having zero contribution to style sales. This will lead to zero SKU ROS calculations for these items, even if the style has a positive ROS. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

4. Process Flow

Here's how information moves through OtbDepletionPeriodRosCalcModule:

flowchart LR

```
graph TD
    A["A[First Month Style ROS<br/>(from Chapter 4)] -->"]
    B["B{OtbDepletionPeriodRosCalcModule<br/>Compute Period ROS}<br/>C[Ideal Size Set (ISS) Data<br/>Forecasted Category Sales<br/>Quantities<br/>New Store Reference Data] --> B"]
    B --> D["D[SKU-level Rate of Sale<br/>for All Depletion Periods]"]
```

1. Start with → First Month's Style ROS (the recent sales velocity for each style at each store), Ideal Size Set (ISS) data (which specifies the adjusted sales contribution for each size within a style), Forecasted Category Sales Quantities (future sales targets for broader product categories), and New Store Reference Data (which new stores should mirror which established stores).
2. Then → Calculate the expected sales velocity for each product style for *all* future depletion periods, scaling it based on category sales quantity ratios relative to the first month and the duration of each period.
3. Next → Break Down these style-level sales velocities to individual SKU-level rates using the adjusted size contributions from the Ideal Size Set (ISS) data, then Normalize these SKU rates to ensure they add up correctly to the style-level rate.
4. Finally → Replicate the calculated SKU-level Rate of Sale for new stores or stores with insufficient sales history by using data from their assigned reference stores, producing a comprehensive set of SKU-level Rate of Sale figures for all stores and periods.

5. Step-by-Step Actions

1. **Calculate Style-Level ROS for All Depletion Periods**
 - What we do: *We determine the expected sales velocity for each product style at every store for all future planning periods. This is done by taking the "first month" (most recent) style ROS and adjusting it based on how the overall category sales quantities are expected to change in subsequent periods, also accounting for the duration of each period.*
 - Why we do it: *While the first month ROS is crucial for immediate demand, sales patterns can shift over longer periods. This step scales that initial velocity to reflect anticipated future demand for the entire depletion horizon, ensuring long-term replenishment plans are accurate and adapt to changing trends.*

2. **Compute** Initial SKU-Level ROS from Style ROS and ISS
 - What we do: *For each style and period, we use the Ideal Size Set (ISS) data, specifically the “adjusted size contribution” for each size, to break down the style’s total Rate of Sale (ROS) into an initial ROS for each individual SKU (e.g., specific size and color).*
 - Why we do it: *A style’s sales velocity is made up of its individual SKUs. ISS tells us the ideal proportion of sales for each size within a style, allowing us to accurately distribute the overall style demand to its component SKUs, ensuring stores stock the right size mix that maximizes sales and minimizes leftover odd sizes.*
3. **Normalize** SKU-Level ROS
 - What we do: *We proportionally adjust the calculated SKU-level ROS values so that their sum perfectly matches the total Rate of Sale previously determined for their respective style. This ensures that the individual SKU sales rates are consistent with the overall style’s performance.*
 - Why we do it: *This normalization is crucial to prevent discrepancies in demand forecasts. Without it, the sum of individual SKU sales might not align with the style’s total sales, leading to over or underestimation of demand for specific sizes and overall inefficient inventory management.*
4. **Replicate** ROS for New or Low-Sales Stores
 - What we do: *For new stores, or existing stores that don’t have enough recent sales data (as defined by the “Minimum Sales Days for New Stores” setting) to calculate their own reliable ROS, we copy the calculated SKU-level Rate of Sale from their pre-assigned “reference” stores.*
 - Why we do it: *New stores often lack sufficient historical sales. By leveraging data from similar, established stores, this ensures that new locations also receive accurate demand signals for replenishment, enabling them to be stocked effectively from day one and avoid lost sales due to lack of historical data.*
5. **Store** Calculated SKU Rate of Sale
 - What we do: *We save all the computed SKU-level Rate of Sale figures (for all stores and all depletion periods) into the system’s database.*
 - Why we do it: *This makes the precise, forward-looking demand data readily available for subsequent modules, serving as the foundation for all inventory allocation and replenishment recommendations, ensuring data consistency and accessibility for other critical business processes.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Style:** ‘Men’s Classic Polo, Style P100’
- **Store:** Store 015
- **First Month (Period 1) ROS for Style P100 at Store 015:** 20 units/week (from `0tbDepletionFirstMonthRosCalcModule`)
- **Attribute Group (AG) for Style P100:** ‘Men’s Tops’ (AG 01)
- **Forecasted Sales Quantity for ‘Men’s Tops’ (AG 01) at Store 015:**
 - Period 1 (First Month): 500 units for this period
 - Period 2: 550 units for this period (10% expected increase from Period 1)
 - Period 3: 450 units for this period (10% expected decrease from Period 1)

- **Period Durations:** Assume each period is 30 days long.
- **Ideal Size Set (ISS) Output for ‘Men’s Tops’ (AG 01) at Store 015:**
 - SKU P100-S (Small): 20% Adjusted Contribution
 - SKU P100-M (Medium): 40% Adjusted Contribution
 - SKU P100-L (Large): 30% Adjusted Contribution
 - SKU P100-XL (X-Large): 10% Adjusted Contribution (Total 100%)
- **New Store Scenario:** Store 020 is a new store, referenced to Store 015. It has had only 3 valid sales days in the past 90 days, which is below the Minimum Sales Days for New Stores setting (e.g., 10 days).

What Happens

1. Calculate Style-Level ROS for all Periods:

- For Style P100 at Store 015:
 - **Period 1 (First Month):** The ROS is directly taken as 20 units/week.
 - **Period 2:** The sales quantity ratio (qtyRatio) for ‘Men’s Tops’ from Period 1 to Period 2 is 550 units / 500 units = 1.1. The period duration ratio (periodDurationRatio) is 30 days / 30 days = 1. The Style ROS for Period 2 is then calculated as $20 \text{ units/week} * 1.1 * 1 = 22 \text{ units/week}$.
 - **Period 3:** The qtyRatio for ‘Men’s Tops’ from Period 1 to Period 3 is 450 units / 500 units = 0.9. The periodDurationRatio is 30 days / 30 days = 1. The Style ROS for Period 3 is then calculated as $20 \text{ units/week} * 0.9 * 1 = 18 \text{ units/week}$.

2. Compute Initial SKU-Level ROS and Normalize:

- For Style P100 at Store 015, Period 1 (Style ROS = 20 units/week):
 - Initial SKU P100-S ROS: $20 * 0.20 = 4 \text{ units/week}$
 - Initial SKU P100-M ROS: $20 * 0.40 = 8 \text{ units/week}$
 - Initial SKU P100-L ROS: $20 * 0.30 = 6 \text{ units/week}$
 - Initial SKU P100-XL ROS: $20 * 0.10 = 2 \text{ units/week}$
 - (In this case, the sum of initial SKU ROS (4+8+6+2=20) perfectly matches the style ROS, so no further normalization adjustment is needed. If the ISS contributions didn’t sum to exactly 1.0, a slight adjustment would occur to ensure the total is 20 units/week.)
- Similar SKU-level calculations are performed for Period 2 (summing to 22 units/week) and Period 3 (summing to 18 units/week) for all SKUs within Style P100.

3. Replicate ROS for New Stores:

- For Store 020 (new store, referenced to Store 015), since it has only 3 valid sales days (below the 10-day minimum), it doesn’t have enough data to calculate its own reliable ROS.
- The module copies the calculated SKU-level ROS values for Style P100 (e.g., 4 units/week for SKU P100-S in Period 1) from Store 015 to Store 020 for all depletion periods. This means Store 020 will now have a forecasted demand for these products, even without its own sales history.

Final Outcome

The module produces a detailed forecast of how many units of each specific size (SKU) of ‘Men’s Classic Polo, Style P100’ are expected to sell per week at Store 015 (and Store 020) for every future depletion period. For example, it now knows that Store 015 is expected to sell 8 units/week of SKU P100-M in Period 1, 8.8

units/week in Period 2, and 7.2 units/week in Period 3. Store 020 will have identical ROS figures to Store 015 due to replication, enabling its initial stocking and replenishment.

Key Takeaway: This step ensures that future inventory plans are based on accurate, SKU-specific demand forecasts that account for both recent performance, broader category trends, ideal size mixes, and the unique needs of new stores, leading to optimized inventory allocation.

7. Results

This step produces:

Main Outputs

- **SKU-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The precise, projected sales velocity for every unique product (SKU) at each store for all future planning periods, serving as the most granular and actionable demand signal for replenishment.*
- **Style-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The aggregated projected sales velocity for each product style at every store for all future planning periods, providing a higher-level view of demand and contextualizing the SKU-level rates.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine consumes these precise SKU and style ROS figures to determine the optimal movement of existing inventory between stores and warehouses, ensuring products are distributed to where they will sell fastest and meet customer demand.*
- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes this granular ROS data to calculate optimal reorder points and quantities for new stock, ensuring that stores have the right products in the right sizes at the right time, minimizing stockouts and excess inventory.*
- **Demand Forecast Accuracy Reports:** *Who uses it and why: Merchandising and planning teams use these ROS forecasts to compare against actual sales, evaluating the effectiveness of the prediction models and identifying opportunities for improvement in future demand predictions, leading to continuous optimization of inventory strategies.*

8. Unusual Cases

Situation | What Happens | What You Should Do |
Missing or delayed sales data feed | The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations. | Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions. |

No Ideal Size Set (ISS) data for a style/category | The system will treat all SKUs within that style/category as having zero contribution to style sales. This will lead to zero SKU ROS calculations for these items, even if the style has a positive ROS. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

Missing or delayed sales data feed | The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations. | Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions. |

No Ideal Size Set (ISS) data for a style/category | The system will treat all SKUs within that style/category as having zero contribution to style sales. This will lead to zero SKU ROS calculations for these items, even if the style has a positive ROS. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

Missing or delayed sales data feed | The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations. | Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions. |

No Ideal Size Set (ISS) data for a style/category | The system will treat all SKUs within that style/category as having zero contribution to style sales. This will lead to zero SKU ROS calculations for these items, even if the style has a positive ROS. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

No Ideal Size Set (ISS) data for a style/category | The module will calculate all SKU ROS as zero for that style, as there is no basis to proportionally distribute the style's total sales. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

Missing or delayed sales data feed | The module will process with incomplete or stale sales data, leading to inaccurate Rate of Sale calculations. | Verify data source connections for sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this directly impacts all downstream inventory decisions. |

No Ideal Size Set (ISS) data for a style/category | The module will calculate all SKU ROS as zero for that style, as there is no basis to proportionally distribute the style's total sales. | Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system. |

4. Process Flow

Here's how information moves through OtbDepletionPeriodRosCalcModule:

flowchart LR

```

    A[First Month Style ROS<br/>(from Chapter 4)] -->
    B{OtbDepletionPeriodRosCalcModule<br/>Compute Period ROS}
    B --> C[Ideal Size Set (ISS) Data<br/>Forecasted Category Sales]
  
```

Quantities
New Store Reference Data] --> B

B --> D[SKU-level Rate of Sale
for All Depletion Periods]

1. Start with → First Month's Style ROS (the recent sales velocity for each style at each store), Ideal Size Set (ISS) data (which specifies the adjusted sales contribution for each size within a style), Forecasted Category Sales Quantities (future sales targets for broader product categories), and New Store Reference Data (which new stores should mirror which established stores).
2. Then → Calculate the expected sales velocity for each product style for *all* future depletion periods, scaling it based on category sales quantity ratios relative to the first month and the duration of each period.
3. Next → Break Down these style-level sales velocities to individual SKU-level rates using the adjusted size contributions from the Ideal Size Set (ISS) data, then Normalize these SKU rates to ensure they add up correctly to the style-level rate.
4. Finally → Replicate the calculated SKU-level Rate of Sale for new stores or stores with insufficient sales history by using data from their assigned reference stores, producing a comprehensive set of SKU-level Rate of Sale figures for all stores and periods.

5. Step-by-Step Actions

1. **Calculate** Style-Level ROS for All Depletion Periods
 - What we do: *We determine the expected sales velocity for each product style at every store for all future planning periods. This is done by taking the “first month” (most recent) style ROS and adjusting it based on how the overall category sales quantities are expected to change in subsequent periods, also accounting for the duration of each period.*
 - Why we do it: *While the first month ROS is crucial for immediate demand, sales patterns can shift over longer periods. This step scales that initial velocity to reflect anticipated future demand for the entire depletion horizon, ensuring long-term replenishment plans are accurate and adapt to changing trends.*
2. **Compute** Initial SKU-Level ROS from Style ROS and ISS
 - What we do: *For each style and period, we use the Ideal Size Set (ISS) data, specifically the “adjusted size contribution” for each size, to break down the style’s total Rate of Sale (ROS) into an initial ROS for each individual SKU (e.g., specific size and color).*
 - Why we do it: *A style’s sales velocity is made up of its individual SKUs. ISS tells us the ideal proportion of sales for each size within a style, allowing us to accurately distribute the overall style demand to its component SKUs, ensuring stores stock the right size mix that maximizes sales and minimizes leftover odd sizes.*
3. **Normalize** SKU-Level ROS
 - What we do: *We proportionally adjust the calculated SKU-level ROS values so that their sum perfectly matches the total Rate of Sale previously determined for their respective style. This ensures that the individual SKU sales rates are consistent with the overall style’s performance.*
 - Why we do it: *This normalization is crucial to prevent discrepancies in demand forecasts. Without it, the sum of individual SKU sales might not align with the style’s total sales, leading to over or underestimation of demand for specific sizes and overall inefficient inventory management.*

4. Replicate ROS for New or Low-Sales Stores

- What we do: *For new stores, or existing stores that don't have enough recent sales data (as defined by the "Minimum Sales Days for New Stores" setting) to calculate their own reliable ROS, we copy the calculated SKU-level Rate of Sale from their pre-assigned "reference" stores.*
- Why we do it: *New stores often lack sufficient historical sales. By leveraging data from similar, established stores, this ensures that new locations also receive accurate demand signals for replenishment, enabling them to be stocked effectively from day one and avoid lost sales due to lack of historical data.*

5. Store Calculated SKU Rate of Sale

- What we do: *We save all the computed SKU-level Rate of Sale figures (for all stores and all depletion periods) into the system's database.*
- Why we do it: *This makes the precise, forward-looking demand data readily available for subsequent modules, serving as the foundation for all inventory allocation and replenishment recommendations, ensuring data consistency and accessibility for other critical business processes.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Style:** 'Men's Classic Polo, Style P100'
- **Store:** Store 015
- **First Month (Period 1) ROS for Style P100 at Store 015:** 20 units/week (from 0tbDepletionFirstMonthRosCalcModule)
- **Attribute Group (AG) for Style P100:** 'Men's Tops' (AG 01)
- **Forecasted Sales Quantity for 'Men's Tops' (AG 01) at Store 015:**
 - Period 1 (First Month): 500 units for this period
 - Period 2: 550 units for this period (10% expected increase from Period 1)
 - Period 3: 450 units for this period (10% expected decrease from Period 1)
- **Period Durations:** Assume each period is 30 days long.
- **Ideal Size Set (ISS) Output for 'Men's Tops' (AG 01) at Store 015:**
 - SKU P100-S (Small): 20% Adjusted Contribution
 - SKU P100-M (Medium): 40% Adjusted Contribution
 - SKU P100-L (Large): 30% Adjusted Contribution
 - SKU P100-XL (X-Large): 10% Adjusted Contribution (Total 100%)
- **New Store Scenario:** Store 020 is a new store, referenced to Store 015. It has had only 3 valid sales days in the past 90 days, which is below the Minimum Sales Days for New Stores setting (e.g., 10 days).

What Happens

1. Calculate Style-Level ROS for all Periods:

- For Style P100 at Store 015:
 - **Period 1 (First Month):** The ROS is directly taken as 20 units/week.
 - **Period 2:** The sales quantity ratio (qtyRatio) for 'Men's Tops' from Period 1 to Period 2 is 550 units / 500 units = 1.1. The period duration ratio (periodDurationRatio) is 30 days / 30 days = 1. The Style ROS

for Period 2 is then calculated as $20 \text{ units/week} * 1.1 * 1 = 22 \text{ units/week}$.

- **Period 3:** The qtyRatio for 'Men's Tops' from Period 1 to Period 3 is $450 \text{ units} / 500 \text{ units} = 0.9$. The periodDurationRatio is $30 \text{ days} / 30 \text{ days} = 1$. The Style ROS for Period 3 is then calculated as $20 \text{ units/week} * 0.9 * 1 = 18 \text{ units/week}$.

2. Compute Initial SKU-Level ROS and Normalize:

- For Style P100 at Store 015, Period 1 (Style ROS = 20 units/week):
 - Initial SKU P100-S ROS: $20 * 0.20 = 4 \text{ units/week}$
 - Initial SKU P100-M ROS: $20 * 0.40 = 8 \text{ units/week}$
 - Initial SKU P100-L ROS: $20 * 0.30 = 6 \text{ units/week}$
 - Initial SKU P100-XL ROS: $20 * 0.10 = 2 \text{ units/week}$
 - (In this case, the sum of initial SKU ROS ($4+8+6+2=20$) perfectly matches the style ROS, so no further normalization adjustment is needed. If the ISS contributions didn't sum to exactly 1.0, a slight adjustment would occur to ensure the total is 20 units/week.)
- Similar SKU-level calculations are performed for Period 2 (summing to 22 units/week) and Period 3 (summing to 18 units/week) for all SKUs within Style P100.

3. Replicate ROS for New Stores:

- For Store 020 (new store, referenced to Store 015), since it has only 3 valid sales days (below the 10-day minimum), it doesn't have enough data to calculate its own reliable ROS.
- The module copies the calculated SKU-level ROS values for Style P100 (e.g., 4 units/week for SKU P100-S in Period 1) from Store 015 to Store 020 for all depletion periods. This means Store 020 will now have a forecasted demand for these products, even without its own sales history.

Final Outcome

The module produces a detailed forecast of how many units of each specific size (SKU) of 'Men's Classic Polo, Style P100' are expected to sell per week at Store 015 (and Store 020) for every future depletion period. For example, it now knows that Store 015 is expected to sell 8 units/week of SKU P100-M in Period 1, 8.8 units/week in Period 2, and 7.2 units/week in Period 3. Store 020 will have identical ROS figures to Store 015 due to replication, enabling its initial stocking and replenishment.

Key Takeaway: This step ensures that future inventory plans are based on accurate, SKU-specific demand forecasts that account for both recent performance, broader category trends, ideal size mixes, and the unique needs of new stores, leading to optimized inventory allocation.

7. Results

This step produces:

Main Outputs

- **SKU-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The precise, projected sales velocity for every unique product (SKU) at each store*

for all future planning periods, serving as the most granular and actionable demand signal for replenishment.

- **Style-level Rate of Sale (ROS) for All Depletion Periods:** *What it tells us: The aggregated projected sales velocity for each product style at every store for all future planning periods, providing a higher-level view of demand and contextualizing the SKU-level rates.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine consumes these precise SKU and style ROS figures to determine the optimal movement of existing inventory between stores and warehouses, ensuring products are distributed to where they will sell fastest and meet customer demand.*
- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes this granular ROS data to calculate optimal reorder points and quantities for new stock, ensuring that stores have the right products in the right sizes at the right time, minimizing stockouts and excess inventory.*
- **Demand Forecast Accuracy Reports:** *Who uses it and why: Merchandising and planning teams use these ROS forecasts to compare against actual sales, evaluating the effectiveness of the prediction models and identifying opportunities for improvement in future demand predictions, leading to continuous optimization of inventory strategies.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process with incomplete or stale data (e.g., First Month Style ROS, ISS data, Forecasted Category Sales Quantities), leading to inaccurate or missing SKU-level ROS calculations.	Verify data source connections for all required inputs. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions.
No Ideal Size Set (ISS) data for a style/category	The module will calculate all SKU ROS as zero for that style, as there is no basis to proportionally distribute the style's total sales, potentially	Review and upload the Ideal Size Set (ISS) data for all relevant styles and attribute groups. Work with merchandising to ensure that accurate size contribution percentages are defined and available in the system.

Situation	What Happens	What You Should Do
First Month Style ROS is zero or very low	leading to no replenishment for needed items. If the baseline ROS for a style is zero, all subsequent period style ROS and SKU ROS will also be zero, indicating no expected sales for that style at that store.	If this is unexpected for a currently selling product, investigate the initial ROS calculation in <code>OtbDepletionFirstMonthRosCalcModule</code>. This could indicate genuine low demand or issues with previous data cleaning (e.g., PSA filtering).
Forecasted Category Sales Quantities are zero or very low	If the forecasted sales for an Attribute Group are zero, the scaling factor for styles within that category will be zero, resulting in zero ROS for all SKUs within those styles for subsequent periods.	Review the inputs to the <code>OtbDepletionSalesCalcModule</code> (e.g., AOP targets, OD revenue contributions) to ensure accurate sales forecasts for all relevant categories. This might indicate an issue with higher-level sales planning.
New store has no reference store or reference store has no sales data	The new store ROS replication won't provide a reliable demand signal, as there's no valid source to copy from, leaving the new store without a demand forecast.	Ensure all new stores are properly assigned to a reference store. If a reference store itself has no sales, consider re-evaluating the reference store assignment or providing a default ROS for such cases.

9. Next Steps

Before This Step

- **OtbDepletionFirstMonthRosCalcModule:** *What we need from it: The foundational “first month” style ROS (Rate of Sale) for each store-style combination, which serves as the immediate demand baseline for all subsequent period calculations.*
- **OtbDepletionSalesCalcModule:** *What we need from it: The forecasted sales quantities for each store-category-period, which are crucial for scaling the first*

month's style ROS to account for future demand shifts across broader product categories.

- **OtbDepletionPrepareDataModule:** *What we need from it: Comprehensive Ideal Size Set (ISS) output data, which provides the adjusted size contributions for distributing style ROS to individual SKUs, and new store reference data for replication logic.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The precise, SKU-level Rate of Sale (ROS) for all depletion periods. This data is critical for determining the optimal movement of existing inventory between stores and warehouses, ensuring products are distributed to where demand is highest.*
- **Replenishment Planning Module:** *What they need from us: The accurate SKU-level ROS values. This is essential for calculating optimal reorder points and quantities for new stock, ensuring that stores receive the right products, in the right sizes, at the right time to meet anticipated customer demand.*

10. Common Questions

Q: How does this module account for future demand changes beyond the “first month”? A: While the “first month” ROS is the most recent sales velocity, this module scales that initial rate for future periods. It does this by observing how the overall forecasted sales quantities for a product's broader category (Attribute Group) are expected to change in subsequent periods, ensuring the ROS adapts to anticipated trends.

Q: Why is the Ideal Size Set (ISS) so important for calculating SKU-level ROS? A: ISS provides the “ideal” sales contribution for each size within a style. This module uses that information to break down a style's total Rate of Sale into proportional sales rates for each individual SKU (e.g., small, medium, large). This ensures that replenishment reflects actual customer preferences for specific sizes, preventing stockouts in popular sizes and overstocking of less popular ones.

Q: What happens if a new store doesn't have enough sales history to calculate its own ROS? A: New stores or those with very limited sales history are assigned a “reference store” in the system. If a store doesn't meet the minimum sales days benchmark, this module intelligently copies the calculated SKU-level ROS from its reference store. This ensures new locations have a reliable demand signal for effective initial stocking and replenishment.

Q: Why do you “normalize” the SKU ROS after initially breaking it down from the style level? A: Normalization ensures that the sum of the individual SKU ROS values for a style exactly matches the overall style's Rate of Sale. This corrects any minor discrepancies that might arise during the initial breakdown using ISS contributions, guaranteeing consistency and accuracy in the total demand forecast for a style.

Q: How does this module contribute to overall profitability? A: By providing highly accurate and granular SKU-level Rate of Sale forecasts for all future periods, this module directly impacts profitability. It allows businesses to make smarter inventory

allocation and replenishment decisions, minimizing lost sales due to stockouts, reducing costs associated with excess inventory and markdowns, and ultimately maximizing revenue from every product.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Computes precise SKU-level Rate of Sale (ROS) for all depletion periods by leveraging first month's style ROS and Ideal Size Set (ISS) output, with replication for new stores.
- **Main Input:** First Month Style ROS, Ideal Size Set (ISS) data, Forecasted Category Sales Quantities, and New Store Reference Data.
- **Main Output:** Detailed SKU-level Rate of Sale for every unique item at all stores across all future planning periods.
- **Warning Signs:** Unexpectedly low or zero ROS for popular SKUs/styles, or new stores not receiving proper ROS calculations.
- **Key Settings:** SKU ROS Look-back Period, Minimum Sales Days for New Stores, Depletion Horizon Start Date.

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 7:

OtbDepletionSegmentationModule

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for identifying and segmenting top-selling product styles at individual stores from the cleaned sales data.

Why This Matters: This segmentation is crucial because it prioritizes these identified top-selling store-styles for preferential inventory allocation and replenishment in subsequent processing iterations. By focusing on what's selling fastest, businesses can ensure high-demand products are always in stock, maximizing sales and minimizing lost opportunities.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data:** *Why we need it:* This provides the historical sales transactions, already filtered to exclude anomalies like liquidation events and periods of insufficient stock, ensuring an accurate representation of genuine customer demand.

Product Data

- **Product Style Information:** *Why we need it:* To accurately identify and group individual products under their broader styles, which is essential for analyzing sales performance at a style level and identifying top-selling combinations of a style and a store.

Operational Data

- **Store Configuration:** *Why we need it:* To provide details about each active retail location, allowing the system to combine product styles with specific stores for granular analysis of sales performance.
- **Current Analysis Date:** *Why we need it:* To establish the “current day” for the analysis, ensuring that the sales look-back period for identifying top sellers is calculated from a consistent and up-to-date reference point.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Top Seller Analysis Period	60 days	Defines the number of recent days of sales history used to identify top-selling product styles at each store.	When the business wants to adjust how quickly it reacts to sales trends, e.g., a shorter period for fast-fashion items or a longer period for evergreen products.
Top Seller Sales Threshold	(Varies)	Sets the minimum sales rate (e.g., units per day or week) or rank that a store-style combination must achieve to be considered a top seller.	When the business wants to be more (or less) selective about which products are prioritized. A higher threshold means fewer items qualify, focusing resources on the absolute best performers.
Minimum Stores for Top Seller Status	2	Specifies the minimum number of stores a style must be a top seller in, or sell above a certain threshold in, to be globally recognized as a top style (beyond individual store performance).	If the business wants to ensure a top seller is widely performing, not just an anomaly in one store. Changing this impacts how many “hot” styles are elevated for broader strategic attention.
Analysis Reference Date	Today	The specific date from which all look-back periods are calculated, ensuring consistency in the sales data window.	This is typically set to the current processing date. It’s only changed if you need to re-run the module for a past date’s context (e.g., for historical analysis or audit purposes).

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Cleaned Sales Data<br/>(from Chapter 3)] -->
B{0tbDepletionSegmentationModule<br/>Top Seller Identification}
B --> C[Segmented Top-Selling Store-Styles]
```

1. Start with → Cleaned Sales Data for all products and stores, which has already been prepared by removing distorting factors like liquidation sales and periods of low stock.
2. Then → Define the analysis period and the specific criteria for identifying top-selling styles at each individual store, based on predefined business settings.

3. Next → Identify which specific store-style combinations meet these sales performance criteria within the defined period, marking them as top sellers.
4. Finally → Segment these high-performing store-styles, creating a prioritized list that will guide subsequent inventory allocation and replenishment decisions.

5. Step-by-Step Actions

1. **Initialize** the module with business settings
 - What we do: *We load predefined parameters, such as the length of the sales analysis period for identifying top sellers and the specific sales rate they must meet to qualify.*
 - Why we do it: *This ensures the module applies the current business rules and focuses on the most relevant recent sales performance to determine which products are considered top performers.*
2. **Gather** cleaned sales data
 - What we do: *We collect historical sales transactions for all products at all stores, using data that has already been meticulously cleaned to remove distortions from liquidation events or periods when products were out of stock.*
 - Why we do it: *Using clean sales data is critical because it ensures that our identification of top-selling products is based purely on genuine customer demand, preventing misjudgments caused by unusual sales circumstances.*
3. **Identify** top-selling store-styles
 - What we do: *For each unique combination of a product style and a specific store, we analyze its sales performance over the defined “Top Seller Analysis Period,” comparing its average sales rate against the “Top Seller Sales Threshold.”*
 - Why we do it: *This direct comparison allows us to pinpoint exactly which product styles are currently performing exceptionally well in which stores, helping the business focus its inventory efforts where they will yield the greatest returns.*
4. **Segment** these top-selling store-styles
 - What we do: *Once identified, we classify or “segment” these high-performing store-styles, assigning them a priority status. This might involve creating a dedicated list or applying a flag to their records.*
 - Why we do it: *Assigning a priority status ensures that these crucial, fast-moving products receive preferential treatment in all subsequent inventory actions, from allocation of existing stock to ordering new replenishment, thereby maximizing their availability and overall sales revenue.*
5. **Store** the segmented list
 - What we do: *We save the comprehensive list of identified top-selling store-styles and their priority flags into the system’s database.*
 - Why we do it: *Storing this information makes it readily available and easily accessible for other modules within the depletion process, ensuring consistent focus on high-priority items throughout the entire inventory management workflow.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- Cleaned Sales Data for ‘Women’s Athletic Leggings, Style L005’ at Store 003 shows 180 units sold over the last 60 days.
- The Top Seller Analysis Period is set to 60 days.
- The Top Seller Sales Threshold is configured as 2.5 units per day.
- Store 003 has available stock of Style L005.

What Happens

1. First, the module accesses the cleaned sales data for ‘Women’s Athletic Leggings, Style L005’ specifically at Store 003 for the past 60 days. It confirms that 180 units were sold during this period.
2. Then, it calculates the average daily sales rate for this specific store-style combination: $180 \text{ units} / 60 \text{ days} = 3.0 \text{ units per day}$.
3. Next, the module compares this calculated sales rate (3.0 units/day) with the “Top Seller Sales Threshold” of 2.5 units/day. Since 3.0 is greater than 2.5, ‘Women’s Athletic Leggings, Style L005’ at Store 003 is officially identified as a “top-selling store-style.”
4. Finally, this combination is added to a prioritized list, marking it as a critical item for this store in the upcoming inventory decisions.

Final Outcome

The module successfully identifies ‘Women’s Athletic Leggings, Style L005’ at Store 003 as a top-selling product, ensuring it receives priority attention for allocation and replenishment to keep up with high customer demand.

Key Takeaway: This step swiftly identifies and elevates high-performing products in specific locations, guaranteeing that inventory efforts are concentrated on maximizing sales where demand is strongest.

7. Results

This step produces:

Main Outputs

- **Prioritized List of Top-Selling Store-Styles:** *What it tells us: A comprehensive list of specific product styles that are currently demonstrating the highest sales velocity at individual store locations, clearly indicating where demand is strongest and attention is needed.*
- **Top Seller Status Flag (per Store-Style):** *What it tells us: A binary indicator attached to each store-style combination, quickly showing whether it has been identified as a top seller. This flag is crucial for filtering and prioritizing in all subsequent depletion processes.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why:* The Depletion Allocation Engine consumes this prioritized list to give preferential treatment to these items when determining how to distribute available inventory across the network, ensuring fast-moving products remain in stock and maximizing sales.
- **Input for Replenishment Planning Module:** *Who uses it and why:* The Replenishment Planning module uses this segmentation output to inform reorder quantities and timing, potentially accelerating orders for top sellers to ensure continuous availability and capitalize on current sales trends.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process with incomplete or stale cleaned sales data, leading to inaccurate or missing top-seller identifications.	Verify data source connections for cleaned sales transactions. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions.
No styles identified as top sellers	The module will produce an empty list of top-selling store-styles, meaning no products will be prioritized for allocation.	Review the “Top Seller Sales Threshold” and “Top Seller Analysis Period” settings. These benchmarks might be too stringent for current sales realities. Also, check overall sales performance for the period.
All styles identified as top sellers	The prioritization mechanism loses its effectiveness, as every item is considered high-priority.	Review and adjust the “Top Seller Sales Threshold” to be more selective. A higher threshold ensures that only truly exceptional performers are elevated, maintaining the strategic value of segmentation.
Unflagged promotional sales in input data	Products with artificially high sales due to promotions might be incorrectly identified as top sellers.	Audit the Cleaned Sales Data from the <code>OtbDepletionPrepareSalesModule</code> to ensure all non-organic sales events (like liquidation or deep promotions) are correctly identified and excluded before running this module.

9. Next Steps

Before This Step

- **OtbDepletionPrepareSalesModule:** *What we need from it: The meticulously cleaned sales data, which has already filtered out distorting factors like liquidation sales and periods of “brokenness,” providing a reliable foundation for identifying top sellers.*
- **OtbDepletionInputModule & OtbDepletionPrepareDataModule:** *What we need from it: Foundational product style information and active store configurations, which are essential for grouping sales data into “store-styles” for accurate performance analysis.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The prioritized list of top-selling store-styles. This crucial input guides the allocation engine to preferentially move existing inventory to these high-demand products and locations, ensuring stock availability.*
- **Replenishment Planning Module:** *What they need from us: The segmentation output to inform replenishment strategies. This helps determine if top sellers should receive accelerated or larger reorders, ensuring continuous stock availability and maximizing sales opportunities.*

10. Common Questions

Q: Why is identifying “top-selling store-styles” important for depletion? A: Identifying top-selling store-styles ensures that the most demanded products at specific locations are prioritized for inventory. This allows businesses to focus resources on items that are currently generating the most sales, reducing stockouts on popular items, maximizing revenue, and improving customer satisfaction.

Q: How is a “top-selling” style actually defined in this module? A: A “top-selling” style at a specific store is defined by comparing its sales performance over a recent period (the “Top Seller Analysis Period”) against a predefined “Top Seller Sales Threshold.” If its average daily sales rate meets or exceeds this threshold, it is flagged as a top seller.

Q: What happens if the sales data isn’t perfectly clean before this module runs? A: If sales data is not clean (e.g., includes liquidation sales or periods of out-of-stock), the module might misidentify top sellers. Products whose sales were artificially inflated by discounts could be prioritized unnecessarily, while truly strong performers might be overlooked due to past stockouts, leading to inefficient inventory decisions.

Q: How does this segmentation influence inventory allocation decisions? A: The segmentation output from this module acts as a guiding signal for the Depletion Allocation Engine. Top-selling store-styles are given higher priority, meaning available inventory (whether from warehouses or other stores) will be directed to them first, ensuring these critical products remain on shelves and ready for sale.

Q: Can the criteria for “top sellers” be changed? A: Yes, the business can adjust key settings like the “Top Seller Analysis Period” and the “Top Seller Sales Threshold.” This flexibility allows merchants and planners to fine-tune the definition of a top seller based on seasonal trends, promotional strategies, or changing business objectives, ensuring the prioritization always aligns with current goals.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Identifies and segments high-performing product styles at individual stores to prioritize their inventory allocation.
- **Main Input:** Cleaned historical sales data, product style information, and store details.
- **Main Output:** A prioritized list of top-selling store-styles and their corresponding status flags.
- **Warning Signs:** No top sellers identified, or an unusually high number of products flagged as top sellers.
- **Key Settings:** Top Seller Analysis Period, Top Seller Sales Threshold.

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 8:

OtbDepletionComputeRevenueModule

Last Updated: 2025-05-23

1. Business Objective

This module calculates the average revenue generated per day for various product styles at individual stores and across different analytical levels. It also identifies and designates certain “read-only” store-styles based on their seasonal characteristics, ensuring that replenishment decisions align with current product lifecycle stages, and consistently applies these revenue metrics by replicating them for new stores from their established reference stores.

Why This Matters: By providing accurate daily revenue figures, this module offers a clear financial picture of each product’s performance at each store, allowing for more strategic inventory investment decisions. Identifying “read-only” styles prevents unnecessary reorders for products nearing their end-of-life, reducing markdown risks. Furthermore, replicating revenue metrics for new stores ensures they are stocked effectively from day one, minimizing lost sales and maximizing early profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (historical units sold):** *Why we need it: To provide the raw quantity of products sold, which, when combined with their price, allows us to calculate the actual revenue generated per day for a given product at a specific store.*
- **Average Selling Price (ASP) by Period, Store, and Attribute Group (AG):** *Why we need it: To convert the sales units into monetary revenue, enabling a financial assessment of sales performance for specific products at specific stores over time.*

Product Data

- **Product Style Information:** *Why we need it: To aggregate sales data from individual unique items (SKUs) up to the broader product style level, which is a key grouping for inventory planning and revenue calculation.*

- **Product Season Assignments:** *Why we need it: To link product styles to their specific selling seasons, which is crucial for identifying “read-only” styles that are nearing the end of their active selling lifecycle.*

Operational Data

- **Active Store List:** *Why we need it: To identify all relevant retail locations for which daily revenue calculations and “read-only” designations need to be performed, ensuring comprehensive coverage across the network.*
- **New Store Reference Data:** *Why we need it: To enable the system to copy revenue metrics from established, similar stores to new stores that lack sufficient sales history, ensuring they have initial demand signals and are stocked effectively.*
- **Read-Only Season Designations:** *Why we need it: To explicitly define which seasons signify that associated styles should be marked as “read-only,” indicating they are in their final selling phase and should not be reordered.*
- **Replenishment Season Designations:** *Why we need it: To define which seasons indicate that styles should be actively considered for replenishment, potentially overriding any “read-only” status if they are part of ongoing inventory efforts.*
- **First Month Analysis Period Length:** *Why we need it: To define the duration of the most recent historical period for which initial daily revenue is calculated for currently selling styles, providing a snapshot of immediate performance.*
- **Channel-Style Override Rules:** *Why we need it: To allow for specific, customized rules (like read-only status) for certain product styles within particular sales channels, ensuring flexibility for specialized merchandise or promotions.*

3. Business Settings

Setting Name | Default Value | Description | When to Change |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Cleaned Sales Data<br/>Average Selling Price<br/>Store & Product Info] --> B{DepletionComputeRevenueModule<br/>Calculate Daily Revenue & Flag Read-Only}
    B --> C[Daily Revenue Per Store-Style<br/>Read-Only Store-Style Designations]
  
```

1. Start with → Cleaned Sales Data (units sold), Average Selling Price (ASP) for products, and Store & Product Information (including season assignments and reference data).
2. Then → Calculate the initial daily revenue for product styles that have recent sales history at each store, converting units sold into monetary value using ASP.
3. Next → Estimate daily revenue for remaining product styles (e.g., those without recent sales or newly introduced) by leveraging aggregated insights, often from broader product categories.

4. Finally → Replicate these calculated daily revenue figures for new stores (using data from their assigned reference stores) and Designate certain store-styles as “read-only” based on their seasonal attributes, producing a comprehensive set of daily revenue metrics and operational flags.

5. Step-by-Step Actions

1. **Initialize** the module with business settings
 - What we do: *We load specific configuration values, such as the exact period length for “first month” revenue calculations and the lists of seasons that define “read-only” or “replenishment” statuses.*
 - Why we do it: *This ensures that the module processes data according to the business’s current strategic priorities for demand forecasting and product lifecycle management.*
2. **Calculate** daily revenue for styles with recent sales
 - What we do: *For product styles that have recent sales activity at a given store, we compute their average daily revenue by taking their historical units sold (from the cleaned sales data) and multiplying it by their Average Selling Price (ASP) over a defined period.*
 - Why we do it: *This provides a direct, data-driven measure of actual financial performance for active products, giving merchants and planners a clear understanding of what’s selling now and for how much.*
3. **Estimate** daily revenue for remaining styles
 - What we do: *For product styles without recent sales data, or for those in newly established stores, we estimate their daily revenue. This is often done by rolling up to a higher level (like an Attribute Group) to derive relevant metrics, and then breaking them back down to the style level.*
 - Why we do it: *This ensures that all relevant product styles have a projected revenue per day, even if their individual sales history is limited, providing a complete picture for inventory planning across the entire assortment.*
4. **Replicate** daily revenue for new stores
 - What we do: *For new stores or those with insufficient sales history, we copy the calculated daily revenue figures from their pre-assigned “reference” stores. This ensures consistency and provides a reliable demand signal where direct historical data is absent.*
 - Why we do it: *New stores need accurate initial stock to succeed. By mirroring the performance of similar, established stores, this step enables effective stocking and replenishment from opening day, preventing missed sales opportunities.*
5. **Identify** read-only store-styles
 - What we do: *We examine each store-style combination and, based on its assigned season, determine if it should be marked as “read-only.” This flag applies to styles that are associated with seasons designated for clearance or end-of-life, although certain replenishment seasons can override this status for specific channels/styles.*
 - Why we do it: *Designating styles as “read-only” prevents them from being considered for further replenishment, which helps to strategically clear out old inventory, reduce the need for deep markdowns, and ensure fresh product assortments.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Cleaned Sales Data: 'Women's Winter Coat, Style C01' at Store 005 sold 60 units over the last 30 days.
- Average Selling Price (ASP) for Style C01 at Store 005: \$150.00.
- First Month Analysis Period Length: 30 days.
- Read-Only Season List: "Winter 2023" (this is the current season ending).
- Replenishment Season List: "Spring 2024".
- Style C01 is assigned to "Winter 2023" season.
- Store 010 is a new store, configured to reference Store 005.

What Happens

1. First, for 'Women's Winter Coat, Style C01' at Store 005, the module uses the cleaned sales data to see 60 units sold over the last 30 days.
2. Then, it calculates the daily revenue: $(60 \text{ units} * \$150.00 \text{ ASP}) / 30 \text{ days} = \$9,000 / 30 \text{ days} = \300 per day .
3. Next, the module looks at Style C01's seasonal assignment ("Winter 2023"). Since "Winter 2023" is in the Read-Only Season List and not actively in the Replenishment Season List, Style C01 at Store 005 is designated as "read-only." This means it will not be replenished.
4. Finally, for new Store 010, since it references Store 005, the module replicates the daily revenue calculated for Style C01 from Store 005. So, Store 010 will also have a daily revenue of \$300 for Style C01, enabling its initial stocking even without its own sales history.

Final Outcome

The module provides a clear daily revenue of \$300 for 'Women's Winter Coat, Style C01' at Store 005 and for new Store 010. Crucially, it also flags this style as "read-only" at Store 005, indicating that it should no longer be replenished, preventing further investment in a product at the end of its selling season.

Key Takeaway: This step provides essential financial insights into product performance and strategically manages inventory through its lifecycle, optimizing investment and ensuring smooth new store operations.

7. Results

This step produces:

Main Outputs

- **Daily Revenue per Store-Style:** *What it tells us: The precise average monetary value of sales generated per day for each product style at individual store locations, providing a clear financial measure of demand.*

- **Read-Only Store-Style Designations:** *What it tells us: A flag or status indicating which store-style combinations should not be considered for further replenishment, guiding inventory planners to focus on clearing existing stock rather than reordering.*
- **National Daily Revenue per Style:** *What it tells us: The aggregated daily revenue for each product style across all stores, offering a high-level view of overall brand or product performance.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine uses these daily revenue figures to help prioritize inventory movement, ensuring higher-value products are distributed to where they will generate the most sales.*
- **Input for Replenishment Planning Module:** *Who uses it and why: The Replenishment Planning module consumes the daily revenue data and read-only flags to determine optimal reorder points and quantities, ensuring that replenishment decisions consider both sales velocity and product lifecycle status.*
- **Sales Performance and Revenue Analysis Reports:** *Who uses it and why: Merchandising and finance teams use this data to analyze product profitability, assess market demand, and evaluate overall financial performance of different product categories and styles.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process with incomplete or stale data (e.g., cleaned sales, ASP), leading to inaccurate or missing daily revenue calculations and incorrect read-only designations.	Verify data source connections for all required inputs. Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions.
Zero or unusually low Average Selling Price (ASP)	If the ASP for a product category is missing or zero, the calculated daily revenue for associated styles will be zero or unrealistically high, leading to	Verify the accuracy of ASP data from the <code>OtbDepletionInputModule</code> . Ensure all relevant product categories and styles have valid, non-zero Average Selling Prices. Investigate the source of any incorrect ASP data.

Situation	What Happens	What You Should Do
Inaccurate Product Season Assignments	distorted financial insights. Styles might be incorrectly flagged as “read-only” (if assigned to a clearance season prematurely) or fail to be flagged (if an end-of-life season is not properly assigned).	Review product master data to ensure styles are correctly linked to their current selling seasons. Work with merchandising to confirm seasonal definitions and update them as product lifecycles change.
New store has no reference store or reference store has no sales data	The new store’s daily revenue replication won’t provide a reliable demand signal, as there’s no valid source to copy from, leaving the new store without a financial demand forecast.	Ensure all new stores are properly assigned to a reference store. If a reference store itself lacks sales history, consider assigning a different reference store or providing a default daily revenue for initial stocking.

9. Next Steps

Before This Step

- **OtbDepletionInputModule:** *What we need from it: The Average Selling Price (ASP) data by period, store, and attribute group, which is essential for converting sales units into revenue.*
- **OtbDepletionPrepareSalesModule:** *What we need from it: The meticulously cleaned sales data, which has already filtered out distorting factors like liquidation sales and periods of “brokenness,” providing a reliable foundation for calculating revenue.*
- **OtbDepletionPrepareDataModule:** *What we need from it: Comprehensive information on active stores, their configurations, new store reference data, and product style information, which are all crucial for setting up the revenue calculation and replication logic.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The calculated daily revenue per store-style and the “read-only” designations. This data is critical for prioritizing the optimal movement of existing inventory, ensuring high-value products are allocated to maximize sales, and preventing allocation to end-of-life items.*

- **Replenishment Planning Module:** *What they need from us: The accurate daily revenue figures. This is essential for understanding the financial impact of replenishment decisions and valuing the stock, ensuring that reorders align with profit objectives.*

10. Common Questions

Q: Why is it important to calculate revenue per day, not just units sold? A: Calculating revenue per day provides a financial context to sales velocity. While units sold tell you how quickly something moves, revenue per day tells you its monetary value. This helps prioritize stocking decisions for higher-value products, ensuring that inventory investments are strategically aligned with profit generation, not just unit movement.

Q: How does this module handle new stores with no sales history? A: For new stores or those with very limited sales, this module intelligently replicates (copies) the calculated daily revenue figures from a designated “reference store.” This ensures that new locations receive accurate demand signals for initial stocking, allowing them to be opened with appropriate inventory levels and maximize early sales.

Q: What does it mean for a store-style to be “read-only,” and why is it important? A: A “read-only” store-style is a product at a specific store that has been flagged as being in its final selling phase, often because it’s associated with a past or clearance season. This designation is crucial because it tells the system *not* to replenish this item, even if it sells out. This helps the business clear old inventory, reduce the risk of markdowns, and ensure space for new, incoming merchandise.

Q: How do the “Read-Only Season List” and “Replenishment Season List” work together? A: The “Read-Only Season List” initially flags styles associated with those seasons as “read-only.” However, the “Replenishment Season List” acts as an override. If a style is also linked to a season on the replenishment list (perhaps it’s a core product that bridges seasons, or a special case via a channel override), it will *not* be marked as read-only. This ensures flexibility in managing styles that may span different lifecycle phases.

Q: How does this module contribute to overall business profitability? A: By providing accurate daily revenue figures, replicating demand signals for new stores, and strategically designating “read-only” styles, this module directly enhances profitability. It helps prevent over-investment in slow-moving or outdated products, reduces lost sales in new stores due to insufficient stock, and supports better allocation of capital to high-performing, high-value merchandise.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates daily revenue for store-style combinations, designates “read-only” styles, and replicates metrics for new stores.
- **Main Input:** Cleaned sales data, Average Selling Prices (ASP), product season assignments, and new store reference data.
- **Main Output:** Daily revenue per store-style, and flags for “read-only” store-styles.

- **Warning Signs:** Unexpectedly high or zero daily revenue figures, or styles incorrectly flagged as “read-only.”
 - **Key Settings:** First Month Analysis Period Length, Read-Only Season List, Replenishment Season List.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 9:

OtbDepletionRankingModule

Last Updated: 2025-05-23

1. Business Objective

This module ranks product styles at individual stores primarily based on their calculated revenue per day, which helps in prioritizing inventory allocation efforts. Additionally, it computes detailed rankings for unique items (SKUs) and for product categories by size, specifically for use in non-pivotal allocation strategies.

Why This Matters: By providing clear, data-driven rankings of product performance, this module enables businesses to make smarter, more focused inventory decisions. This ensures that the most valuable and fastest-selling items are always prioritized for stock, minimizing lost sales, reducing excess inventory, and ultimately maximizing revenue and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Daily Revenue per Store-Style:** *Why we need it: To assess the financial performance of each product style at a given store and use this value as the primary metric for ranking store-styles for allocation priority, focusing on monetary contribution.*
- **Sales Data:** *Why we need it: To compute granular rankings at the SKU and category-size levels, ensuring these deeper analyses are based on accurate and undistorted sales history.*
- **SKU-level Rate of Sale (ROS) for All Depletion Periods:** *Why we need it: To provide a forward-looking sales velocity for individual items, which is essential for detailed SKU and category-size rankings, particularly for allocation strategies that require precise unit movement data.*

Product Data

- **Product Style Information:** *Why we need it: To group individual unique items (SKUs) under their broader product styles, enabling rankings at the style level and allowing for a comprehensive view of product performance across collections.*

- **Attribute Group (AG) Mappings:** *Why we need it: To categorize products into broader attribute groups (e.g., “dresses,” “jeans”), which is essential for generating rankings at the store-category-size level, supporting broader merchandising strategies for product diversity.*
- **Ideal Size Set (ISS) Information:** *Why we need it: To understand the ideal sales mix for different sizes within a style, which is crucial for computing accurate store-category-size rankings and ensuring optimal inventory distribution that aligns with customer buying patterns.*

Operational Data

- **Store Configuration and Distribution Network Details:** *Why we need it: To identify all active retail locations and their characteristics, providing the necessary context for generating store-specific product rankings and ensuring allocation efforts are targeted correctly.*
- **Current Analysis Date:** *Why we need it: To establish the “current day” for the analysis, ensuring that the look-back periods for any supporting data used in ranking calculations are consistently anchored to the most recent operational snapshot.*

3. Business Settings

Setting Name | Default Value | Description | When to Change |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Daily Revenue Per Store-Style
Sales Data
SKU/Style Rate of Sale (ROS)] --> B{DepletionRankingModule
Product Performance Ranking}

B --> C[Ranked Store-Styles
Store-SKU Rankings
Store-Category-Size Rankings]

1. Start with → Daily Revenue per Store-Style as the primary metric, along with Sales Data and SKU/Style Rate of Sale for granular rankings.
2. Then → Prioritize store-styles based on their calculated daily revenue, creating a main ranked list.
3. Next → Compute more detailed rankings for individual SKUs and for product categories by size, focusing on granular performance.
4. Finally → Produce comprehensive Ranked Lists for store-styles, store-SKUs, and store-category-sizes, guiding inventory allocation.

5. Step-by-Step Actions

1. **Gather** daily revenue and sales data
 - What we do: *We collect the calculated daily revenue figures for each store-style, along with cleaned historical sales data and Rate of Sale (ROS) for individual SKUs.*
 - Why we do it: *These are the core metrics used to evaluate and compare the performance of different products across stores, providing the basis for all ranking calculations.*
2. **Create** store-style comparison data
 - What we do: *We prepare a structured dataset that combines each store-style with its associated daily revenue, marking which revenue figure (initial or a refined final one) should be used for comparison based on the overall depletion strategy.*
 - Why we do it: *This ensures that all store-styles are evaluated consistently using the correct and most relevant daily revenue metric, enabling a fair and accurate ranking for allocation purposes.*
3. **Rank** store-styles by revenue per day
 - What we do: *We sort all store-styles from highest to lowest based on their calculated daily revenue, effectively prioritizing them.*
 - Why we do it: *This prioritizes the fastest-selling and highest-value products at each store, allowing the business to focus its allocation efforts on maximizing sales and profitability for these top performers, ensuring key products are always available.*
4. **Compute** store-SKU rankings
 - What we do: *Beyond the style level, we calculate specific rankings for each unique item (SKU) at each store, often based on their Rate of Sale (ROS) and sales volume from the cleaned sales data.*
 - Why we do it: *This provides a more granular view of individual item performance, which is essential for fine-tuning allocation decisions, ensuring that specific sizes or colors within a style are stocked optimally within a store to meet precise customer demand.*
5. **Compute** store-category-size rankings
 - What we do: *We generate rankings for combinations of product categories (Attribute Groups) and sizes at each store, assessing which size ranges are performing best within broader merchandise groups.*
 - Why we do it: *This helps understand overall demand patterns for size sets within categories, guiding non-pivotal allocation strategies to ensure a healthy balance of variety (OW) and depth (OD) across the size curve, minimizing odd sizes and maximizing customer choice.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Store 001, Style 'Summer Dress D100', Daily Revenue: \$350
- Store 002, Style 'Summer Dress D100', Daily Revenue: \$200
- Store 003, Style 'Swim Trunks S200', Daily Revenue: \$400
- Store 001, SKU 'Summer Dress D100-M' (Medium), Rate of Sale (ROS): 10 units/week

- Store 001, SKU ‘Summer Dress D100-S’ (Small), Rate of Sale (ROS): 5 units/week
- Cleaned Sales Data is available for all products and stores.

What Happens

1. First, the module gathers the daily revenue figures for each store-style combination: \$350 for ‘Summer Dress D100’ at Store 001, \$200 for ‘Summer Dress D100’ at Store 002, and \$400 for ‘Swim Trunks S200’ at Store 003.
2. It then sorts these store-styles based on their daily revenue, from highest to lowest:
 1. Store 003, Style ‘Swim Trunks S200’ (\$400/day)
 2. Store 001, Style ‘Summer Dress D100’ (\$350/day)
 3. Store 002, Style ‘Summer Dress D100’ (\$200/day)
3. Separately, for more granular rankings, the module processes the SKU-level Rate of Sale (ROS) for individual items. For Store 001, it ranks ‘Summer Dress D100-M’ (10 units/week) higher than ‘Summer Dress D100-S’ (5 units/week) because of its faster sales velocity.
4. Finally, it also creates rankings for store-category-size combinations by using relevant Attribute Group (AG) and Ideal Size Set (ISS) data, determining which size ranges within product categories are performing best.

Final Outcome

The module produces a prioritized list of store-styles based on their financial contribution, clearly showing that ‘Swim Trunks S200’ at Store 003 is the top priority for allocation among these examples. It also delivers granular rankings, indicating that the ‘Medium’ size of ‘Summer Dress D100’ at Store 001 is a higher priority than the ‘Small’ size. These precise rankings guide immediate inventory decisions, ensuring the most valuable products and sizes are always available on the sales floor.

Key Takeaway: This step provides a clear, data-driven hierarchy of product performance at the store, SKU, and category-size levels, enabling focused and impactful inventory allocation that maximizes sales and efficiency.

7. Results

This step produces:

Main Outputs

- **Ranked Store-Style List (by Revenue Per Day):** *What it tells us: A comprehensive, prioritized list of product styles at individual store locations, ordered by their average daily revenue, indicating where the highest financial demand lies and where to focus allocation efforts.*
- **Detailed Store-SKU Rankings:** *What it tells us: Precise rankings for every unique item (SKU) at each store, often based on their Rate of Sale or sales volume, providing granular insights into individual item performance for micro-level allocation.*

- **Detailed Store-Category-Size Rankings:** *What it tells us: Prioritized lists of product categories by size within each store, indicating which size ranges are most in demand within broader merchandise groups, crucial for optimizing the Ideal Size Set (ISS) and overall product mix.*

Business Reports

- **Input for Depletion Allocation Engine:** *Who uses it and why: The Depletion Allocation Engine consumes these precise multi-level product rankings (store-style, store-SKU, store-category-size) to make informed decisions on how to move existing inventory across the network, ensuring the highest priority items are stocked first, maximizing sales and preventing stockouts.*
- **Performance Prioritization Dashboard:** *Who uses it and why: Merchandising and planning teams use this dashboard to quickly identify which products and sizes are driving the most revenue and units at which stores. This allows them to make strategic decisions on inventory focus, promotional activities, and visual merchandising to capitalize on top performers.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will operate with incomplete or stale daily revenue, sales data, or Rate of Sale figures, leading to inaccurate or unreliable rankings.	Verify data source connections for all required inputs (e.g., 0tbDepletionComputeRevenueModule, 0tbDepletionPeriodRosCalcModule, 0tbDepletionPrepareSalesModule). Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions.
All daily revenues are zero or very low	If prior modules calculate zero or very low daily revenue for all styles, the ranking will not effectively prioritize items, potentially leading to misallocations or overstocks of slow-moving items.	Investigate upstream modules (e.g., 0tbDepletionComputeRevenueModule, 0tbDepletionPrepareSalesModule) to ensure accurate calculation of daily revenue and sales velocity. Check for systemic issues with sales data or product setup that might be suppressing sales metrics.
Inconsistent or missing Ideal Size Set (ISS) data	When computing store-category-size or SKU rankings, missing or	Review and update Ideal Size Set (ISS) data for relevant styles and categories. Work with merchandising to ensure that accurate size contribution percentages are defined and

Situation	What Happens	What You Should Do
New stores with no reference data	inaccurate ISS data can lead to skewed proportional allocations and incorrect prioritization of sizes, resulting in unbalanced stock.	available in the system, reflecting true customer preferences.
	If a new store lacks sales history and a designated reference store, it may not receive meaningful rankings, hindering its initial stocking and leading to missed sales opportunities.	Ensure all new stores are properly assigned to a suitable reference store in the system. If a reference store itself has no sales, consider re-evaluating the reference store assignment or providing a default ranking for initial stocking in such cases.

9. Next Steps

Before This Step

- **OtbDepletionComputeRevenueModule:** *What we need from it: The calculated Daily Revenue per Store–Style and National Daily Revenue per Style, which are the primary metrics used for prioritizing store-style allocations based on financial contribution.*
- **OtbDepletionPeriodRosCalcModule:** *What we need from it: The granular SKU–level Rate of Sale (ROS) for All Depletion Periods and Style –level Rate of Sale (ROS) for All Depletion Periods, essential for detailed SKU and category-size rankings that reflect future unit movement.*
- **OtbDepletionPrepareSalesModule:** *What we need from it: The Sales Data, which forms the foundation for computing accurate SKU and category-size rankings, ensuring calculations are based on clean and undistorted historical sales.*
- **OtbDepletionPrepareDataModule:** *What we need from it: Product Style Information, Attribute Group (AG) Mappings, Ideal Size Set (ISS) Information, and Store Configuration to correctly categorize and link data for all ranking levels, providing the necessary product and store context.*

After This Step

- **Depletion Allocation Engine:** *What they need from us: The precise, multi-level product rankings (store-style, store-SKU, store-category-size). This data is*

critical for determining the optimal movement of existing inventory between stores and warehouses, ensuring the highest priority items are stocked first to maximize sales and minimize markdowns.

- **Replenishment Planning Module:** *What they need from us: The detailed product rankings to influence future purchasing and replenishment orders. This allows planners to prioritize new stock for top-performing items and sizes, ensuring that the right products arrive at the right time to meet anticipated customer demand.*

10. Common Questions

Q: Why is ranking by “revenue per day” important for store-styles? A: Ranking by revenue per day provides a clear financial prioritization for inventory decisions. It focuses attention on products that not only sell quickly but also generate significant monetary value for the business. This ensures that inventory investments are directed towards maximizing profitability, not just unit movement.

Q: How do the detailed SKU and category-size rankings help the business? A: While store-style ranking gives a high-level view, SKU and category-size rankings provide granular insights. They help identify which specific sizes or product attributes are performing best within a style or category at a particular store, allowing for precise allocation and replenishment that matches customer preferences and minimizes leftover odd sizes.

Q: What happens if a store-style has no sales but significant on-hand inventory? A: If a store-style has no sales, it will likely have a zero or very low daily revenue, causing it to rank low. This signals to the allocation engine that this item is a low priority for replenishment and might even be considered for transfer *out* of the store, preventing stagnant inventory and freeing up valuable retail space for higher-ranking products.

Q: How does this module ensure fairness in ranking between different product types or categories? A: By primarily using “revenue per day,” the module inherently accounts for the value of different products. A high-value item selling slower might still rank higher than a lower-value item selling faster, if its daily revenue contribution is greater. For more granular rankings, inputs like Ideal Size Set (ISS) and category sales ensure a balanced approach that considers the overall product mix.

Q: Can these rankings be adjusted or overridden by merchants? A: While this module generates data-driven rankings based on performance, the business often has mechanisms in subsequent allocation or planning modules to apply strategic overrides. For example, a merchant might manually boost the priority of a new product launch, a key promotional item, or an item for a specific store, allowing for flexibility beyond purely historical performance.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Ranks product performance at store, SKU, and category-size levels, primarily by daily revenue, to guide inventory allocation and replenishment.

- **Main Input:** Daily revenue per store-style, cleaned sales data, SKU-level Rate of Sale (ROS), product attribute, and sizing information.
 - **Main Output:** Prioritized lists and rankings for store-styles, store-SKUs, and store-category-sizes.
 - **Warning Signs:** Unrealistic rankings (e.g., very slow-moving items appearing at the top), or entire product groups missing from rankings unexpectedly.
 - **Key Settings:** Revenue Look-back Period (influences input), Minimum Daily Revenue Threshold, Ranking Tier Definitions.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 10:

OtbDepletionAllocationModule

Last Updated: 2025-05-23

1. Business Objective

This is the central module for the depletion process, intelligently allocating available inventory (SKUs) and managing warehouse stock to maximize product availability and sales. It simulates sales over time and applies various iterative strategies to optimize inventory distribution, ensuring products are where customers want them.

Why This Matters: This module directly translates demand forecasts into actionable inventory movements. By continuously optimizing stock across stores and warehouses, it ensures that high-demand products are always available, minimizes lost sales from stockouts, reduces costs from excess inventory, and ultimately maximizes revenue and profitability by placing every item in its best selling location.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **SKU-level Rate of Sale (ROS) for All Depletion Periods:** *Why we need it: This provides the precise forecasted selling speed for each unique item at every store for all future planning periods, which is crucial for simulating sales and determining how quickly inventory will be depleted.*
- **Daily Revenue per Store-Style:** *Why we need it: This helps in financially prioritizing inventory allocation, ensuring that higher-value styles are considered first for distribution to maximize revenue potential.*

Product Data

- **Comprehensive Ideal Size Set (ISS) Information:** *Why we need it: To understand the ideal sales mix of sizes for each product style. This guides allocations to ensure stores receive the optimal combination of sizes that customers actually buy, minimizing leftover odd sizes and maximizing sales.*
- **Product Style and SKU Information:** *Why we need it: To accurately identify and categorize every unique item and its associated style, which is fundamental*

for managing inventory and grouping products for effective allocation strategies like planogram adherence.

Operational Data

- **Consolidated Inventory Snapshot (SKU-Store & Warehouse levels):** *Why we need it: This provides an accurate, real-time count of all available stock across all stores and central warehouses, including on-hand units and incoming stock, forming the indispensable baseline for all allocation decisions.*
- **Store and Channel Configuration:** *Why we need it: To identify all active retail locations within the distribution network, ensuring that inventory allocations are directed to the correct stores and channels.*
- **Prioritized List of Top-Selling Store-Styles:** *Why we need it: To identify high-demand products at specific locations so they can be prioritized for replenishment and allocation, ensuring quick-selling items remain in stock.*
- **Read-Only Store-Style Designations:** *Why we need it: To identify products that are in their end-of-life cycle and should not be replenished, allowing for strategic clearance and reducing markdown risks.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Depletion Planning Horizon	(Varies, e.g., 90 days)	Defines the total number of days into the future for which inventory depletion and allocation will be simulated and planned.	When the business needs to adjust its forward-looking planning horizon, for example, for longer lead times or specific seasonal strategies.
Allocation Cycle Frequency	7 days	Determines how often the module simulates a depletion cycle and attempts to allocate warehouse stock (e.g., weekly).	When the business needs to adjust the frequency of inventory movements, for example, for faster-moving goods requiring more frequent replenishment or slower-moving items needing less.
Planogram Adherence & Depth Optimization Flag	True	When enabled, this setting ensures that allocation strategies prioritize adherence to optimal product	If the business wants to temporarily disable complex planogram-driven allocations, for example, during peak holiday periods when rapid

Setting Name	Default Value	Description	When to Change
ROS-Driven Empty Warehouse Strategy	False	category width (OW) and depth (OD) based on planogram layouts and Ideal Size Sets (ISS). If true, this enables a strategy to distribute remaining warehouse stock based on Rate of Sale (ROS), aiming to fully clear warehouses by pushing inventory to stores with demand.	replenishment might be prioritized over strict planogram adherence. When the business wants to aggressively clear warehouse inventory, especially at the end of a season or for promotional items, by prioritizing sell-through over maintaining warehouse reserves.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[SKU-level ROS<br/>Consolidated Inventory<br/>ISS & Planogram Data<br/>Top Seller & Read-Only Status] -->
    B[DepletionAllocationModule<br/>Iterative Allocation & Depletion Simulation]
    B --> C[Optimal SKU Allocations<br/>Simulated Inventory Levels<br/>Depletion Output Reports]
  
```

1. Start with → SKU-level Rate of Sale (ROS) (future sales velocity), Consolidated Inventory Snapshot (current stock), Ideal Size Set (ISS) information, Planogram details (Optimum Width - OW, Optimum Depth - OD), Top Seller designations, and Read-Only statuses.
2. Then → Iterate through defined allocation cycles (e.g., weekly), simulating sales depletion and applying various allocation strategies, prioritizing top-selling products and adhering to planogram layouts and ideal size sets.
3. Next → Adjust inventory levels in stores and warehouses based on simulated sales and allocation decisions, and Account for incoming stock.
4. Finally → Generate detailed output reports on optimal SKU allocations and projected inventory levels over the entire depletion planning horizon.

5. Step-by-Step Actions

1. Calculate Online Sales Reserve

- What we do: *We determine a quantity of warehouse stock to reserve specifically for anticipated online sales over the product's lead time. This quantity is based on predicted sales for online stores and specific product categories served by that warehouse.*
- Why we do it: *This ensures that critical stock is always available for online orders, preventing lost sales from web customers and maintaining customer satisfaction for e-commerce channels, even if physical stores are also being replenished.*

2. Simulate Depletion Cycles

- What we do: *We run a series of iterative cycles, typically weekly, over the defined Depletion Planning Horizon. In each cycle, we simulate sales (depleting stock) and then attempt to reallocate inventory from warehouses to stores, and internally between stores.*
- Why we do it: *This continuous simulation allows the system to proactively respond to anticipated demand shifts and inventory changes, ensuring dynamic and optimal stock management over time, preventing sudden stockouts or excess inventory build-up.*

3. Apply Replenishment Strategies

- What we do: *Within each cycle, we execute various replenishment strategies, prioritizing allocations for top-selling items and general replenishment needs based on forecasted demand.*
- Why we do it: *This ensures that fast-moving products receive preferential treatment, helping to keep shelves stocked with popular items and maximizing immediate sales opportunities for the business.*

4. Adhere to Planogram and Size Set Rules

- What we do: *We apply strategies to fill gaps in product categories (Optimum Width - OW) according to planogram layouts and ensure stores receive the best combination of sizes (Ideal Size Set - ISS) and optimal quantities (Optimum Depth - OD), particularly for non-pivotal sizes.*
- Why we do it: *This maintains a balanced and appealing product assortment in stores, maximizing customer choice, improving the shopping experience, and minimizing the risk of having incomplete size runs or empty display spaces.*

5. Process Incoming Inventory

- What we do: *We account for new stock arriving at warehouses and stores during each depletion cycle, integrating these incoming quantities into the overall inventory picture before simulating the next period's sales.*
- Why we do it: *This ensures that all future inventory availability is accurately reflected, preventing over-ordering by seeing what's already on its way and providing a comprehensive view of supply.*

6. Simulate Sales and Adjust Stock

- What we do: *After allocations, we simulate the selling of products from stores based on their SKU-level Rate of Sale (ROS). This reduces the available stock for the next cycle and tracks the depleted quantities.*
- Why we do it: *This step directly mimics real-world sales, allowing the module to project future inventory levels, identify potential stockouts before they happen, and provide a clear picture of stock depletion over time.*

7. Generate Depletion Output Reports

- What we do: *At the end of the simulation, the module compiles comprehensive reports detailing the optimal SKU allocations, projected*

closing stock levels at each store, and an overview of warehouse stock and online reserves for each period.

- *Why we do it: These reports provide actionable insights for merchants and planners, allowing them to review the system's recommendations, make informed decisions, and understand the anticipated inventory landscape, leading to better financial and operational planning.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A central warehouse has 100 units of 'Men's Casual Shirt, Style M500' (SKU M500-L for size Large, M500-XL for X-Large).
- Store 001 is a top seller for Style M500. Its Ideal Size Set (ISS) indicates 40% of sales are Large (M500-L) and 20% are X-Large (M500-XL).
- Store 001's SKU-level ROS for M500-L is 5 units/week, and for M500-XL is 2 units/week.
- An online store expects to sell 10 units of M500-L in the next 2 weeks (within lead time), which needs to be reserved from the warehouse.
- The Depletion Planning Horizon is 60 days, with an Allocation Cycle Frequency of 7 days (weekly).

What Happens

1. **Online Reserve Calculation:** The module first identifies the 10 units of M500-L needed for online sales over the lead time and reserves them from the total 100 units of M500-L in the warehouse. Remaining warehouse stock for M500-L is 90 units.
2. **Week 1 (Allocation Cycle 1):**
 - The module prioritizes Store 001 for Style M500 due to its top-seller status.
 - Based on Store 001's ROS (5 units/week for M500-L, 2 units/week for M500-XL) and ISS (40% L, 20% XL), it recommends allocating units to meet expected sales for the next week.
 - Let's say 10 units of M500-L and 4 units of M500-XL are allocated to Store 001 from the warehouse.
 - Warehouse stock decreases by these amounts.
 - The module then simulates 7 days of sales at Store 001 based on ROS (e.g., 5 units of M500-L and 2 units of M500-XL are "sold").
 - Closing stock for Store 001 is recorded, and the process moves to the next week.
3. **Subsequent Weeks:** The module continues this iterative process for the full 60-day horizon, continually simulating sales, adding incoming inventory, and allocating available warehouse stock based on updated store needs, top-seller status, planogram rules, and ideal size sets.

Final Outcome

The module produces a detailed week-by-week projection of optimal SKU allocations, showing how many units of each size of ‘Men’s Casual Shirt, Style M500’ should be moved from the warehouse to Store 001 (and other stores) to meet forecasted demand. It also tracks the simulated inventory levels, identifying potential stockouts or overstocks in advance, and provides a clear view of the online sales reserve. For instance, it might recommend sending 10 M500-L and 4 M500-XL to Store 001 next week, followed by 8 M500-L and 3 M500-XL the week after, all while ensuring online orders can still be fulfilled.

Key Takeaway: This module acts as the central intelligence for inventory movement, dynamically adapting to sales forecasts and store needs to ensure products are efficiently distributed and available to customers over time, maximizing sales and minimizing holding costs.

7. Results

This step produces:

Main Outputs

- **Optimal SKU Allocations (by Store-SKU-Period):** *What it tells us: The recommended quantity of each unique item (SKU) to be transferred from warehouses to individual stores for each upcoming period, ensuring stock is optimally distributed to meet forecasted demand.*
- **Simulated Closing Inventory (by Store-SKU-Period):** *What it tells us: The projected remaining stock for every SKU at each store at the end of each simulated period, providing a clear forecast of future inventory levels and identifying potential stockouts or surpluses.*
- **Warehouse Depletion Overview (by Warehouse-SKU-Period):** *What it tells us: A detailed breakdown of how much stock is drawn from each warehouse for every SKU over the depletion horizon, including the impact of online sales reserves and incoming inventory.*
- **Pre- and Post-Empty Warehouse Inventory Snapshots:** *What it tells us: Before and after views of warehouse and store inventory, specifically when an “empty warehouse” strategy is applied, providing insights into the effectiveness of aggressive stock clearing.*

Business Reports

- **Inventory Transfer Recommendations:** *Who uses it and why: Logistics and fulfillment teams use these recommendations to execute physical stock movements between warehouses and stores, ensuring that products are delivered to the right locations at the right time.*
- **Future Stock Level Projections:** *Who uses it and why: Merchants and planners use these projections to understand the anticipated inventory landscape, allowing them to proactively plan for promotions, markdowns, or additional purchasing if needed, based on predicted stock levels.*
- **Depletion Performance Dashboards:** *Who uses it and why: Senior management and inventory teams use these dashboards to monitor the overall*

efficiency of the depletion process, track key metrics like stock turns, and identify areas for strategic improvement in inventory management.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will operate on incomplete or stale data (e.g., current inventory, ROS, ISS), leading to inaccurate allocations and unreliable depletion forecasts.	Verify data source connections for all required inputs (from Chapters 1-8). Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream inventory decisions and potential sales.
Perpetual stockouts for top-selling items	The module will continuously recommend allocations to critical items, but if no warehouse stock is available, stores will remain empty and sales will be lost.	Review upstream procurement and production processes to ensure adequate supply for top sellers. Investigate if lead times are too long or if initial buys are insufficient.
Excessive warehouse stock for slow-moving items	The module may struggle to allocate these items if store demand is low, leading to high holding costs and potential markdowns if “empty warehouse” strategies are not enabled.	Consider enabling or adjusting the “ROS-Driven Empty Warehouse Strategy” if the goal is to clear stock. Evaluate if the product assortment is appropriate or if promotional activities are needed to stimulate demand.
Inaccurate Ideal Size Set (ISS) or Planogram (OW/OD) data	Allocations might result in unbalanced size runs at stores or deviations from optimal category presentations, leading to lost sales or inefficient use of display space.	Conduct regular audits of ISS and planogram data. Work with merchandising teams to ensure these ideal configurations accurately reflect current market demand and store layouts.
Online sales reserve consistently falls short or is excessive	The calculated online reserve quantity might not match actual online demand, leading to missed online sales or unnecessary tying up of warehouse stock.	Review the historical accuracy of online sales forecasts and the “Depletion Planning Horizon” and lead time calculations used for online reserve. Adjust these parameters as needed.

9. Next Steps

Before This Step

- **OtbDepletionInputModule:** *What we need from it: Core business settings like depletion duration, lead times, and Average Selling Price (ASP), providing the foundational context for the allocation logic.*
- **OtbDepletionPrepareDataModule:** *What we need from it: A comprehensive, consolidated inventory snapshot (SKU-store & warehouse), live style status, product Attribute Group (AG) and Ideal Size Set (ISS) mappings, and store configurations, which are critical for understanding available stock and store needs.*
- **OtbDepletionPrepareSalesModule:** *What we need from it: Cleaned sales data, which forms the basis for accurate Rate of Sale (ROS) calculations used in depletion simulation.*
- **OtbDepletionFirstMonthRosCalcModule:** *What we need from it: The first month's style ROS, providing the most recent sales velocity for initial demand sensing.*
- **OtbDepletionSalesCalcModule:** *What we need from it: Forecasted sales quantities for each store-category-period, enabling the scaling of ROS for future periods and overall demand planning.*
- **OtbDepletionPeriodRosCalcModule:** *What we need from it: The precise, SKU-level Rate of Sale (ROS) for all depletion periods, which is the primary driver for simulating sales and determining allocation quantities.*
- **OtbDepletionSegmentationModule:** *What we need from it: The prioritized list of top-selling store-styles, guiding preferential allocation decisions to high-demand items.*
- **OtbDepletionComputeRevenueModule:** *What we need from it: Daily revenue per store-style and "read-only" store-style designations, providing financial prioritization and preventing allocation to end-of-life products.*

After This Step

- **Physical Inventory Transfer Execution:** *What they need from us: The optimal SKU allocation recommendations to generate actual transfer orders and execute the physical movement of inventory from warehouses to stores, or between stores.*
- **Warehouse Management System (WMS) Updates:** *What they need from us: Updated inventory levels and future allocation plans to ensure the WMS accurately reflects incoming and outgoing stock, maintaining real-time visibility.*
- **Replenishment Planning Module:** *What they need from us: The simulated closing inventory levels and depletion forecasts. This helps them determine when new purchase orders are needed from vendors to maintain desired stock levels beyond the current depletion horizon.*
- **Financial Reporting and Analysis:** *What they need from us: Comprehensive depletion output reports (e.g., sales quantities, closing stock, online reservations) to assess inventory performance, evaluate markdown risks, and forecast future revenue from replenished stock.*

10. Common Questions

Q: How does this module handle the complexities of stock distribution to many stores? A: This module uses an iterative, rules-based approach. It prioritizes allocations based on factors like a product's sales velocity (ROS), its status as a top seller, adherence to ideal product mixes (Ideal Size Set - ISS), and compliance with store display layouts (Optimum Width - OW, Optimum Depth - OD). It runs these strategies in cycles, constantly adjusting to simulate real-world sales and incoming stock.

Q: What is the “depletion process” and how does this module fit into it? A: The “depletion process” is about managing existing inventory by simulating how it will sell out over time and how to optimally move it to meet demand. This module is the “engine” of that process. It takes all the prepared data (forecasts, current stock, product rules) and actively calculates where every available SKU should go to maximize sales and minimize waste.

Q: How does the system ensure stores receive the right sizes? A: The module uses “Ideal Size Set (ISS)” data. This data tells it the optimal proportion of each size sold for a particular product style. When allocating, the module tries to match this ideal mix, so if a store is expected to sell more mediums, it will prioritize sending more medium-sized items, ensuring stores stock what customers actually buy.

Q: What happens to products that are old or no longer selling well? A: The module identifies “read-only” products, which are typically items at the end of their selling season. For these, it will not recommend replenishment, even if they sell out. This prevents unnecessary new orders for old stock, helping the business clear existing inventory and make space for new, fresh merchandise.

Q: How does this module balance immediate sales with long-term inventory health? A: It balances by looking at both immediate (first month) sales trends and longer-term forecasted demand (SKU-level Rate of Sale - ROS for all depletion periods). It iteratively simulates sales and allocations over a defined “Depletion Planning Horizon.” This allows it to address current needs while also preventing future stock imbalances by projecting inventory levels far enough in advance.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Core module for inventory allocation and warehouse stock management, executing iterative strategies based on sales, planograms, and size sets.
 - **Main Input:** SKU-level Rate of Sale (ROS), current inventory, Ideal Size Set (ISS) data, top seller flags, and read-only designations.
 - **Main Output:** Optimal SKU allocations by store, simulated future inventory levels, and detailed depletion reports.
 - **Warning Signs:** Persistent stockouts for popular items, excessive stock of slow-moving items in warehouses, or imbalanced size availability in stores.
 - **Key Settings:** Depletion Planning Horizon, Allocation Cycle Frequency, Planogram Adherence & Depth Optimization Flag.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 11: DepletionOutputsModule

Last Updated: 2025-05-23

1. Business Objective

This module consolidates the detailed outputs from the depletion simulation, transforming them into comprehensive, easy-to-understand reports and performance metrics. It critically assesses the health of inventory after depletion, quantifies Pivotal Size Availability (PSA) percentages, and provides a clear picture of opening stock levels across stores and warehouses.

Why This Matters: By providing clear insights into how inventory is expected to perform and what the stock levels will look like, this module enables confident decision-making for future replenishment and allocation. It helps identify potential stock imbalances, ensures that the right products in the right sizes are available to customers, and supports proactive adjustments to maximize sales and minimize holding costs.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Detailed Depletion Simulation Results (per SKU-store-period):** *Why we need it: These are the core outputs from the allocation engine, detailing simulated sales, stock movements, and closing quantities for every item over time, forming the foundation for all final reports.*
- **SKU-level Rate of Sale (ROS):** *Why we need it: To provide context for the simulated sales, indicating how quickly products are expected to sell and allowing for performance analysis within the output reports.*

Product Data

- **Product Master Data (SKU, Style, Attribute Group, Pivotal Tag):** *Why we need it: To categorize products accurately within the reports, allowing for aggregation of performance metrics (e.g., by style or category) and to determine Pivotal Size Availability (PSA) for inventory health checks.*
- **Product Seasons:** *Why we need it: To contextualize product performance within their selling cycles and ensure reporting aligns with seasonal strategies.*

Operational Data

- **Current Inventory Snapshot (On-Hand, Goods in Transit, Open Orders, Internal In-Transit):** *Why we need it: To establish a comprehensive baseline of actual current and incoming stock before depletion, which is consolidated into opening inventory figures for reporting.*
- **Warehouse Inwards & Store Incoming Inventory:** *Why we need it: To account for all stock received during the depletion horizon, ensuring the most accurate portrayal of inventory flow in the reports.*
- **Store and Warehouse Configuration:** *Why we need it: To link inventory data to specific locations, enabling detailed reporting by store and warehouse and ensuring all metrics are associated with the correct distribution points.*
- **Minimum Style Lead Times:** *Why we need it: To estimate future inbound warehouse stock within the relevant lead time, providing a forward-looking view of supply in the warehouse inventory reports.*
- **Pivotal Size Availability (PSA) Benchmark:** *Why we need it: To assess the health of inventory by measuring how well key sizes are represented before and after depletion, guiding decisions on assortment completeness.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Replenishment PSA Benchmark	(Varies, e.g., 75%)	Defines the minimum percentage of Pivotal Sizes that must be available for a style to be considered “Healthy” for replenishment.	If the business adjusts its target for optimal size availability, or to make the health check more or less stringent.
Include Store On-Hand in Opening Inventory	True	Determines if the physical inventory currently in retail stores should be included when calculating the starting inventory for reporting.	Rarely, almost always “True”. Might be set to “False” during specific data reconciliation periods but would significantly impact reporting accuracy.
Include Incoming Stock in Opening Inventory	True	Specifies whether stock that is ordered from vendors (Open Orders), or in transit (Goods in Transit - GIT,	Almost always “True” to prevent over-reporting of immediate stock needs. Only temporarily “False” for specific audits, as it impacts the

Setting Name	Default Value	Description	When to Change
Warehouse Allocation Frequency	7 days	Internal In-Transit - IIT) should be included when calculating the opening inventory figures.	completeness of the opening stock picture.
		The frequency (in days) at which the depletion simulation runs its allocation cycles, influencing how opening/closing stock and ROS are reported per period.	If the business needs to adjust the granularity of simulated inventory movements for reporting and analysis.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw Depletion Outputs<br/>(from Allocation Module)] -->
    B{DepletionOutputsModule<br/>Data Transformation & Health Metrics}
    C[Current & Incoming Inventory<br/>Product Master Data] --> B
    B --> D[Denormalized Depletion Data<br/>Inventory Health
    Reports<br/>Opening Stock Snapshots]
  
```

1. Start with → Raw Depletion Outputs (detailed simulation results from the allocation module), along with Current and Incoming Inventory and Product Master Data.
2. Then → Denormalize the raw outputs, combining various data points into comprehensive SKU-level and style-level reports.
3. Next → Compute critical inventory health metrics, such as Pivotal Size Availability (PSA) percentages, and Consolidate opening inventory figures for both stores and warehouses.
4. Finally → Generate a suite of denormalized, exportable data formats and insightful reports for strategic inventory and sales analysis.

5. Step-by-Step Actions

1. Denormalize depletion outputs

- What we do: *We combine raw data from various sources (simulated sales, inventory movements, product details) into a single, comprehensive view for each SKU at each store across all simulated periods. We also create a similar consolidated view at the product style level.*
- Why we do it: *This transforms complex raw data into easy-to-understand, actionable reports that provide a holistic picture of how individual products and broader styles are expected to perform and be managed over time.*

2. Compute post-depletion inventory health

- What we do: *We calculate the percentage of key (Pivotal) sizes that are available for a style both before and after the simulated depletion. We then determine if the style's inventory health is "Healthy" or "Broken" based on predefined availability benchmarks.*
- Why we do it: *This crucial step provides a clear assessment of how well our inventory strategy maintains the right mix of sizes for top-selling products. It helps identify if depletion efforts are leading to stockouts of popular sizes, impacting customer satisfaction and sales.*

3. Consolidate opening store inventory

- What we do: *We gather all sources of current and incoming stock for each SKU at each store – including physical on-hand, items in transit (Goods in Transit, Internal In-Transit), and products on open orders – to create a complete picture of the starting inventory for each period.*
- Why we do it: *This ensures that our reports accurately reflect the total available stock at the beginning of the planning horizon, providing a reliable baseline for analyzing subsequent inventory movements and sales.*

4. Consolidate opening warehouse inventory

- What we do: *We calculate the starting inventory for each SKU in the warehouse by considering its current on-hand quantity and factoring in future incoming stock (inwards) relevant to its lead time.*
- Why we do it: *This gives a comprehensive view of our central stock availability, critical for understanding what is available for future store replenishment and for setting appropriate online sales reserves.*

5. Prepare exportable reports

- What we do: *We aggregate and format the denormalized data into summaries that are ready for export to various business intelligence tools and external systems.*
- Why we do it: *This makes the valuable insights from the depletion process accessible to a wider audience, enabling performance monitoring, financial analysis, and strategic planning outside of the core system.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- From the Depletion Allocation Module, we have simulated outputs for ‘Women’s Summer Dress, Style D100’ at Store 005.
 - Week 1 (Current Period): Store 005 openingInventory (from allocation) was 15 units (SKU D100-M). salesQty was 4 units. closingQty was 11 units.
 - Pre-depletion healthy SKU count for Style D100 at Store 005: 6 (out of 7 pivotal sizes initially available).
 - Post-depletion healthy SKU count for Style D100 at Store 005: 5 (out of 7 pivotal sizes remaining).
- The Replenishment PSA Benchmark is set to 75%.
- The Current On-Hand Inventory for SKU D100-M at Store 005 on the analysis end date (e.g., June 30th) was 10 units.
- There are 5 units of SKU D100-M in transit to Store 005.
- The Warehouse Inwards for SKU D100-M shows 20 units arriving next month.
- The Minimum Lead Time for Style D100 is 14 days.

What Happens

1. **Denormalization:** The module takes the raw simulated outputs for Style D100 at Store 005 and combines them with product details (like style code, brand, category) into comprehensive rows for both SKU-level and style-level reporting. For example, it records the sales as 4 units and closingInventory as 11 units for SKU D100-M in Week 1.
2. **Inventory Health Calculation:**
 - For Style D100 at Store 005, the module sees 7 total pivotal sizes for this style.
 - Pre-Depletion Health: With 6 pivotal sizes initially available, the percentage is $(6/7) * 100 = 85.7\%$. Since 85.7% is greater than the 75% Replenishment PSA Benchmark, the preDepletionHealth status is marked “HEALTHY”.
 - Post-Depletion Health: With 5 pivotal sizes remaining after simulation, the percentage is $(5/7) * 100 = 71.4\%$. Since 71.4% is *less than* the 75% Replenishment PSA Benchmark, the postDepletionHealth status is marked “BROKEN”. This flags that the depletion strategy led to insufficient key sizes.
3. **Opening Store Inventory:** For SKU D100-M at Store 005, the module calculates the opening inventory for the reporting period as $10 \text{ (on-hand)} + 5 \text{ (in transit)} = 15 \text{ units}$.
4. **Opening Warehouse Inventory:** For SKU D100-M, the module looks at the central warehouse’s current stock and the 20 units arriving next month (within the 14-day lead time for Style D100). It consolidates this to show a complete picture of available warehouse stock.
5. **Export Preparation:** All these detailed metrics are then prepared into exportable summary reports, showing total sales over the horizon, overall opening/closing inventory, and the health status.

Final Outcome

The module produces reports showing that ‘Women’s Summer Dress, Style D100’ at Store 005 started the period with 15 units of SKU D100-M, sold 4 units, and ended with 11 units. Crucially, it flags that while the style began “HEALTHY” in terms of pivotal size availability, it became “BROKEN” after depletion, indicating a potential issue with size balance. This data is now available for merchants and planners to review, identify why the store went “BROKEN,” and adjust future strategies to ensure a healthier size mix is maintained.

Key Takeaway: This step provides crucial post-depletion inventory health checks and comprehensive opening stock insights, enabling merchants to quickly spot size availability issues and make informed adjustments to maintain optimal product assortments.

7. Results

This step produces:

Main Outputs

- **Denormalized SKU-level Depletion Output:** *What it tells us: A detailed, consolidated record for each unique item (SKU) at every store over each simulated period, combining sales, stock movements, and inventory levels into a single, easy-to-analyze row.*
- **Denormalized Style-level Depletion Output:** *What it tells us: An aggregated, consolidated record for each product style at every store over each simulated period, providing a higher-level view of performance by combining all SKUs within a style.*
- **Post-Depletion Inventory Health Status (per Style-Store):** *What it tells us: A critical flag (e.g., “HEALTHY” or “BROKEN”) indicating whether a product style maintains sufficient Pivotal Size Availability (PSA) after simulated depletion, highlighting potential issues with size assortment or stockouts.*
- **Opening Store Inventory Snapshot:** *What it tells us: A comprehensive view of the starting inventory for every SKU at each store for the planning horizon, including on-hand, goods in transit, and open orders, providing the true baseline for analysis.*
- **Opening Warehouse Inventory Snapshot:** *What it tells us: A detailed view of the starting inventory for every SKU in the warehouse, accounting for current stock and anticipated incoming deliveries within lead times, crucial for managing central supply.*
- **Exportable Depletion Output Reports:** *What it tells us: Summarized versions of the depletion results, aggregated over the entire planning horizon and formatted for easy export and integration with other business intelligence tools.*

Business Reports

- **Inventory Health & Performance Dashboards:** *Who uses it and why: Merchandising and inventory planning teams use these dashboards to monitor stock levels, assess the effectiveness of depletion strategies, and quickly identify*

styles or stores that are struggling with size availability or overstock/understock issues.

- **Replenishment Strategy Review:** *Who uses it and why: Inventory planners use the detailed depletion outputs and health metrics to review the accuracy of forecasts and the effectiveness of allocation rules, allowing them to refine replenishment parameters for future cycles.*
- **Financial Inventory Valuations:** *Who uses it and why: Finance teams use the opening and closing inventory snapshots (when combined with ASP) to value inventory assets, manage working capital, and assess the financial impact of current stock levels.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will operate on incomplete or stale raw depletion outputs from the allocation engine, leading to inaccurate or missing final reports, health metrics, and inventory snapshots.	Verify data source connections for all required inputs (especially the output from the <code>OtbDepletionAllocationModule</code>). Check data ingestion logs and re-trigger feeds. Contact the IT support team immediately, as this impacts all downstream reporting and strategic decisions.
No Pivotal Sizes defined for a style	The Pivotal Size Availability (PSA) calculation will be inaccurate or might result in division by zero, leading to erroneous “HEALTHY” or “BROKEN” statuses for styles that actually have key sizes.	Review product master data to ensure that all relevant styles have properly defined Pivotal Sizes (Pivotal Tag) for their associated Attribute Groups. Work with merchandising to establish and maintain accurate Ideal Size Sets (ISS) and pivotal size definitions.
Post-Depletion Health consistently “BROKEN” for many styles	This indicates that the depletion allocation strategy is consistently failing to maintain adequate Pivotal Size Availability across a wide range of products, potentially leading to lost sales and customer dissatisfaction.	Investigate the <code>OtbDepletionAllocationModule</code> settings and inputs. Review the Replenishment PSA Benchmark setting – it might be too strict. More importantly, assess if the allocation logic is effectively distributing sizes or if there are systemic issues with inventory levels (e.g., insufficient buys, imbalanced initial receipts).
Opening inventory figures appear incorrect	The reported starting stock levels for stores or warehouses might not match	Verify the Include Store On-Hand in Opening Inventory and Include Incoming Stock in Opening Inventory settings.

Situation	What Happens	What You Should Do
	expectations, potentially misrepresenting the true available stock and leading to flawed analysis of depletion performance.	Cross-reference the raw inventory data feeds (on-hand, GIT, open orders, inwards) to ensure their accuracy and timely delivery.

9. Next Steps

Before This Step

- **OtbDepletionAllocationModule:** *What we need from it: The comprehensive raw outputs of the depletion simulation, including simulated sales, allocations, closing quantities, and online reservation details, which are the primary data processed by this module.*
- **OtbDepletionPrepareDataModule:** *What we need from it: Current and incoming inventory snapshots (on-hand, GIT, open orders, inwards) and comprehensive product master data (SKU, style, AG, store details) which are used to consolidate opening inventory figures and enrich the output reports.*
- **OtbDepletionComputeRevenueModule:** *What we need from it: The Replenishment PSA Benchmark setting and Pivotal Tagging information, essential for calculating and determining the “HEALTHY” or “BROKEN” status of post-depletion inventory health.*
- **OtbDepletionPeriodRosCalcModule:** *What we need from it: The SKU-level Rate of Sale (ROS) for all depletion periods, which is incorporated into the denormalized reports for performance context.*

After This Step

- **Business Intelligence (BI) and Reporting Tools:** *What they need from us: The denormalized and exportable output data to populate dashboards, create performance reports, and enable detailed analysis of depletion effectiveness, inventory health, and stock trends.*
- **Inventory Performance Review Meetings:** *What they need from us: The comprehensive reports and health metrics to inform strategic discussions on inventory investment, assortment planning, and operational adjustments, ensuring continuous improvement in stock management.*
- **Financial Planning and Analysis (FP&A) Systems:** *What they need from us: The opening and closing inventory snapshots (when combined with ASP from earlier modules) for accurate valuation of inventory assets and reconciliation with financial statements.*

10. Common Questions

Q: What is the main purpose of this module's output? A: The main purpose is to provide clear, consolidated reports and performance metrics about how our inventory is expected to perform and what stock levels will look like after the depletion process. This helps us understand if products are selling as expected, if we have the right sizes, and where our stock is, so we can make better future decisions.

Q: How does this module assess "inventory health"? A: It assesses inventory health by looking at something called Pivotal Size Availability (PSA). For each product style, it checks if a certain percentage of its most important (pivotal) sizes are in stock, both before and after the simulated sales. If the percentage falls below a set benchmark, the inventory for that style is flagged as "BROKEN," alerting us to potential stock issues with key sizes.

Q: Why are both "pre-depletion" and "post-depletion" inventory health metrics important? A: "Pre-depletion" health shows how well-stocked a style was *before* any simulated sales or allocations, highlighting initial assortment strengths or weaknesses. "Post-depletion" health reveals the impact of the depletion strategy. If a style goes from "HEALTHY" to "BROKEN" post-depletion, it signals that the allocation or replenishment strategy might be leading to stockouts of key sizes, requiring immediate attention.

Q: What types of inventory are included in the "opening inventory" reports? A: The "opening inventory" reports provide a comprehensive picture of all available stock at the start of a period. This includes physical stock on hand in stores and warehouses, plus any products that are already purchased from vendors but not yet received (Open Orders), or are currently moving between locations (Goods in Transit and Internal In-Transit). This complete view prevents over-ordering and gives a realistic starting point.

Q: How do these outputs help improve future inventory decisions? A: By providing detailed, easy-to-read reports on sales performance, inventory levels, and especially inventory health, these outputs act as a feedback loop. Merchants can see which strategies are working, which products are struggling with size availability, and where stock is accumulating. This data empowers them to fine-tune future purchase orders, adjust allocation rules, and respond proactively to demand changes, ultimately leading to more profitable inventory management.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Processes raw depletion outputs to generate denormalized reports, inventory health metrics, and opening stock snapshots for comprehensive analysis.
- **Main Input:** Raw outputs from the depletion allocation engine, current and incoming inventory data, and product master data (SKU, Style, AG, Pivotal Tags).
- **Main Output:** Denormalized SKU-level and Style-level depletion reports, "HEALTHY" or "BROKEN" inventory health flags (PSA), and consolidated opening inventory figures for stores and warehouses.

- **Warning Signs:** Consistent “BROKEN” health status for important styles, unexpected discrepancies in opening inventory figures, or missing data in final reports.
 - **Key Settings:** Replenishment PSA Benchmark, Include Store On-Hand in Opening Inventory, Include Incoming Stock in Opening Inventory, Warehouse Allocation Frequency.
-

Need Help? Contact your merchandising operations or IT support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

depletion FAQ

Version: 1.0

Last Updated: 2025-05-23

Purpose: This FAQ document provides quick, business-focused answers to common questions about the depletion algorithm, its processes, and its outputs.

General

Q: What is the main goal of the depletion algorithm?

A: The primary goal of the depletion algorithm is to efficiently allocate SKUs and manage warehouse stock across our stores. It ensures that the right products are in the right place at the right time, minimizing stockouts and maximizing sales.

Q: How does depletion help manage inventory?

A: The depletion algorithm helps manage inventory by dynamically allocating available SKUs from the warehouse to stores based on forecasted demand, sales performance (ROS), and ideal product assortments (OD, OW, ISS), ensuring optimal stock levels.

Q: What specific retail metrics does the depletion process optimize?

A: The depletion process optimizes key retail metrics such as Rate of Sale (ROS), product availability (especially Pivotal Size Availability - PSA), revenue contribution, and adherence to ideal product assortments like Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS).

Q: What do “OD,” “OW,” “ISS,” and “ROS” mean in this context?

A: - **OD (Optimum Depth):** The ideal number of unique items (SKUs) to stock in a category or display. - **OW (Optimum Width):** The ideal number of different categories or styles to offer. - **ISS (Ideal Size Set):** The best combination of sizes to maximize sales and minimize stockouts. - **ROS (Rate of Sale):** The speed at which products are sold over a period (e.g., units/week).

Q: Is the depletion process continuous, or does it run at specific intervals?

A: The depletion process is designed to run periodically, orchestrating various modules to re-evaluate and reallocate stock based on updated sales data, inventory levels, and forecasts, ensuring timely replenishment and stock management.

Configuration & Settings

Q: How can we influence the allocation priorities in the depletion process?

A: Allocation priorities are primarily influenced by how store-styles are ranked, which is based on factors like calculated revenue per day. You can review the ranking methodology within the `OtbDepletionRankingModule` to understand the prioritization criteria.

Q: Are there specific settings for how seasonal products are handled?

A: Yes, the `OtbDepletionComputeRevenueModule` identifies and designates certain store-styles as “read-only” based on their seasonal attributes, which can influence their allocation behavior.

Q: How does the system handle new stores that don’t have sales history?

A: For new stores, the system leverages data from reference stores to replicate metrics like ROS (Rate of Sale) and revenue. This ensures that new locations receive appropriate initial allocations based on comparable existing stores, as handled by the `OtbDepletionPeriodRosCalcModule` and `OtbDepletionComputeRevenueModule`.

Q: Can we set parameters for adhering to planogram layouts (OW) during allocation?

A: Yes, the core `OtbDepletionAllocationModule` includes specific strategies to ensure adherence to planogram layouts, which relates to maintaining the Optimum Width (OW) of categories or styles offered in a store.

Q: What happens if there’s an override to the channel-styles?

A: Overridden channel-styles are processed and incorporated early in the data preparation phase by the `OtbDepletionPrepareDataModule`. This ensures that any specific business decisions regarding channel-style offerings are respected in the depletion process.

Data & Inputs

Q: What types of data are essential for the depletion algorithm to run?

A: Essential input data includes lead times, current inventory quantities (at the SKU-store level), relevant styles by season, historical sales data, master data (like distribution stores and attribute mappings), open orders, and goods in transit. This is managed by the `OtbDepletionInputModule` and `OtbDepletionPrepareDataModule`.

Q: Where does the system get its sales data for ROS calculations?

A: Sales data is loaded and prepared by the `OtbDepletionPrepareSalesModule`, which cleans the raw sales, removes liquidation sales, and filters for “brokenness” (e.g., insufficient Pivotal Size Availability - PSA) before initial ROS calculations are performed.

Q: How are Ideal Size Sets (ISS) used in the depletion process?

A: The Ideal Size Set (ISS) output is crucial for computing SKU-level ROS (Rate of Sale) by adjusting size contributions to align with style ROS, ensuring that the best combination of sizes is prioritized during allocation to maximize sales and minimize stockouts. This is handled by `OtbDepletionPeriodRosCalcModule` and `OtbDepletionAllocationModule`.

Q: What does “Pivotal Size Availability” (PSA) refer to, and why is it important?

A: PSA refers to the availability of core or “pivotal” sizes for a product. It’s critical because the algorithm evaluates product availability against PSA conditions to ensure accurate ROS calculations and to assess the health of stock levels post-depletion, as processed by `0tbDepletionPrepareSalesModule` and `DepletionOutputsModule`.

Q: Does the system account for future inventory, like goods in transit or open orders?

A: Yes, the `0tbDepletionPrepareDataModule` integrates information about open orders and goods in transit, providing a comprehensive view of current and upcoming inventory for more accurate depletion planning.

Process & Logic

Q: How is the Rate of Sale (ROS) calculated for products in the depletion process?

A: ROS is calculated in multiple stages. The `0tbDepletionFirstMonthRosCalcModule` computes style ROS based on historical sales and PSA. Then, the `0tbDepletionPeriodRosCalcModule` calculates SKU-level ROS, leveraging style ROS and adjusting for Ideal Size Set (ISS) contributions.

Q: How are sales forecasts generated for the depletion horizon?

A: Sales quantities are forecasted by the `0tbDepletionSalesCalcModule`. These forecasts are derived using inputs from the Optimum Depth (OD) segmentation output (specifically, revenue contribution) and align with the Annual Operating Plan (AOP) revenue targets.

Q: What is the strategy for prioritizing which products or store-styles get allocated first?

A: The `0tbDepletionSegmentationModule` identifies and segments top-selling store-styles, prioritizing them for allocation. The `0tbDepletionRankingModule` then further refines this by ranking store-styles based on calculated revenue per day, ensuring that high-performing combinations receive attention.

Q: What is the core logic for allocating SKUs and managing warehouse stock?

A: The `0tbDepletionAllocationModule` is the core. It executes iterative allocation strategies, including replenishment, adherence to planogram layouts (OW), and compliance with Ideal Size Sets (ISS). It meticulously tracks depletion over time, aiming for optimal stock distribution.

Q: How does the system handle “brokenness” in sales data, and what does it mean?

A: “Brokenness” refers to sales data that might be misleading due to poor product availability (e.g., missing key sizes). The `0tbDepletionPrepareSalesModule` cleans raw sales data and filters out such “broken” sales based on predefined PSA (Pivotal Size Availability) benchmarks to ensure accurate ROS calculations.

Outputs & Results

Q: What kind of reports and outputs does the depletion process generate?

A: The `DepletionOutputsModule` processes the raw depletion data to generate denormalized and exportable data formats. This includes comprehensive reports on SKU allocation, warehouse stock movements, and store inventory levels.

Q: How can I assess the effectiveness or “health” of the depletion outcomes?

A: The `DepletionOutputsModule` computes crucial post-depletion health metrics, such as Pivotal Size Availability (PSA) percentages. These metrics are key indicators of how well stock is positioned to meet demand and minimize stockouts.

Q: Can I view the historical opening inventory levels for stores and warehouses after the depletion run?

A: Yes, the `DepletionOutputsModule` consolidates and reports opening warehouse and store inventory information, providing a clear snapshot of stock levels before the new allocations took effect.

Q: Does the depletion process provide insights into potential stockouts or overstocks?

A: While not explicitly an “output report” for stockouts, the process’s focus on maximizing sales, minimizing stockouts through ISS, and monitoring PSA indirectly provides insights into potential issues. The generated inventory reports can highlight areas of concern.

Q: How does the depletion output help in future planning?

A: By providing clear data on allocated SKUs, remaining warehouse stock, and post-depletion health metrics, the outputs help in understanding performance, identifying trends, and refining future inventory and buying decisions.

Error Handling & Edge Cases

Q: What happens if critical input data, like current inventory, is missing or incorrect?

A: The `OtbDepletionInputModule` is designed to load essential data. If critical data is missing or malformed, the process may halt or produce inaccurate results. It’s crucial to ensure data quality upstream.

Q: How does the system handle an unexpected surge or drop in sales for a product?

A: The ROS calculation modules (`OtbDepletionFirstMonthRosCalcModule`, `OtbDepletionPeriodRosCalcModule`) are designed to use historical sales data. While they can adapt to trends, extreme, sudden changes might require manual review or adjustments to input forecasts if the historical data doesn’t adequately reflect new realities.

Q: What if there isn’t enough warehouse stock to meet the calculated demand for all stores?

A: The `OtbDepletionAllocationModule` manages warehouse stock. In cases of insufficient stock, the allocation logic will prioritize based on factors like store-style ranking and revenue contribution, ensuring that the most impactful allocations are made with available inventory.

Q: Are there built-in mechanisms to prevent over-allocation or creating excess stock?

A: Yes, the allocation strategies within the `OtbDepletionAllocationModule` are designed to adhere to Optimum Depth (OD) and Optimum Width (OW) targets, preventing over-allocation and ensuring efficient use of stock based on ideal assortments.

Need More Help?

Please contact the Inventory Management Support Team for further assistance.

Depletion Documentation

- **Version:** 1.0
- **Last Updated:** 2025-05-23

2. Executive Summary

The Depletion algorithm optimizes SKU allocation and warehouse stock management to maximize sales and minimize stockouts. It achieves this by accurately forecasting demand, prioritizing high-value products and stores, and efficiently distributing inventory across the network. The core outcome is an ideal stock allocation plan that enhances product availability and drives revenue.

3. Architecture Overview

```
graph TD
    subgraph Input & Preparation
        A[0tbDepletionInputModule] -->|depends_on| B[0tbDepletionPrepareDataModule]
        A -->|data_flow| C[0tbDepletionFirstMonthRosCalcModule]
        A -->|data_flow| D[0tbDepletionSalesCalcModule]
        A -->|data_flow| E[0tbDepletionComputeRevenueModule]
        F[0tbDepletionPrepareSalesModule] -->|data_flow| C
        F -->|data_flow| G[0tbDepletionSegmentationModule]
        F -->|data_flow| E
        F -->|data_flow| H[0tbDepletionRankingModule]
    end

    subgraph Core Calculations
        C -->|data_flow| I[0tbDepletionPeriodRosCalcModule]
        D -->|data_flow| I
        G -->|control_flow| E
        E -->|data_flow| H
    end

    subgraph Allocation & Output
        B -->|data_flow| J[0tbDepletionAllocationModule]
        I -->|data_flow| J
        E -->|data_flow| J
        H -->|data_flow| J
        J -->|data_flow| K[DepletionOutputsModule]
    end
```

```
classDef moduleStyle fill:#f9f,stroke:#333,stroke-width:2px;  
class A,B,C,D,E,F,G,H,I,J,K moduleStyle;
```

Module Descriptions:

- **OtbDepletionInputModule:** Initializes and loads essential input data like lead times and current inventory for the depletion process.
- **OtbDepletionPrepareDataModule:** Prepares and loads master and transactional data, including current stock, open orders, and attribute mappings, crucial for allocation.
- **OtbDepletionFirstMonthRosCalcModule:** Calculates the Rate of Sale (ROS) for the first month of depletion based on historical sales and product availability.
- **OtbDepletionSalesCalcModule:** Forecasts sales quantities for each store-category-period, utilizing OD segmentation output and AOP revenue.
- **OtbDepletionPeriodRosCalcModule:** Computes SKU-level Rate of Sale (ROS) for all depletion periods, leveraging first month's style ROS and ISS output.
- **OtbDepletionPrepareSalesModule:** Cleans raw sales data by removing liquidation sales and filtering for "brokenness" based on PSA benchmarks.
- **OtbDepletionSegmentationModule:** Identifies and segments top-selling store-styles from cleaned sales data for prioritization in allocation.
- **OtbDepletionComputeRevenueModule:** Calculates revenue per day for store-style combinations and designates "read-only" styles based on seasonal attributes.
- **OtbDepletionRankingModule:** Ranks store-styles primarily by calculated revenue per day and computes detailed SKU/category-size rankings for allocation.
- **OtbDepletionAllocationModule:** The core module for efficient SKU allocation and warehouse stock management, executing iterative allocation strategies.
- **DepletionOutputsModule:** Processes raw depletion output rows to generate denormalized, exportable data formats and health metrics like PSA percentages.

4. Prerequisites & Setup

- **Java Version:** Java 17 or higher
- **Build Tool:** Apache Maven 3.8.x or higher

Build Commands: To compile the project, navigate to the root directory and run:

```
mvn clean install
```

Dependencies: All required dependencies are managed through the `pom.xml` file and will be automatically downloaded by Maven during the build process.

Running the Orchestrator (ModuleGroup): The ModuleGroup orchestrator is typically executed via a command-line interface or a scheduled job. Assuming a Maven build, the executable JAR would be located in the target directory of the module-group module.

```
java -jar module-group/target/depletion-module-group-1.0.jar --  
      config-file=args.properties
```

Note: Ensure args.properties is correctly configured and accessible from the execution path.

5. Folder & File Structure

```
depletion/
├─ module-group/          # Contains the main orchestrator
                           (ModuleGroup) that coordinates module execution.
├─ submodules/            # Houses individual depletion logic
                           modules (e.g., OtbDepletionInputModule).
├─ args.properties        # Global configuration parameters for the
                           entire depletion process.
├─ common-data.json       # Schema definition for shared data
                           structures across modules.
├─ outputs/               # Directory where final depletion reports
                           and output files are generated.
```

6. Global Configuration

No `args.properties` data was provided for rendering the global configuration table. Please refer to the `args.properties` file in the project root for available settings.

7. Shared Data Model

No `common-data.json` schema data was provided for rendering the shared data model table. Please refer to the `common-data.json` file for the detailed schema.

8. Submodule Chapters

8.1. OtbDepletionInputModule

Business Objective: To establish the foundational data required for the entire depletion process, ensuring that subsequent calculations and allocations are based on accurate and up-to-date inputs like lead times, inventory, and relevant product seasons.

Required Information:

- **Sales Data:**
 - Relevant styles for ROS computation (historical sales context).
- **Product Data:**
 - SKU-store level current inventory quantities.
 - Seasonal associations for styles.
- **Operational Data:**
 - Vendor and transit lead times for inbound stock.

Business Settings:

- **Lead Time Configuration:** Specifies how lead times are sourced and applied (e.g., `vendor_lead_time_override`).

- **Season Definition:** Defines which product seasons are currently in scope for depletion planning (e.g., `current_season_list`, `ros_season_list`).

Process Flow:

START --> Load Lead Times --> Load Current Inventory --> Identify Relevant Styles/Seasons --> Populate Data Structures --> END

Step-by-Step Actions:

1. **Load** lead time data *from a master data source to understand product delivery schedules.*
2. **Fetch** current inventory quantities *for every SKU at each store and warehouse location.*
3. **Identify** relevant styles and their associated seasons *based on predefined configuration parameters for ROS calculation and depletion planning.*
4. **Initialize** global data structures *such as `OtbDepletionData` with the loaded information, making it accessible to subsequent modules.*

Sample Scenario: Imagine a new spring collection (Style A, B, C) is about to launch. This module would **load** the 2-week lead time for Style A from the manufacturer, **confirm** current stock of 0 units for all new styles, and **identify** 'Spring 2025' as the active season for depletion. It also pulls in existing basic styles (Style X, Y) and their current inventory. **Key Takeaway:** Without this initial data, subsequent modules cannot accurately predict sales or allocate stock, leading to potential mis-shipments or stockouts.

Results: * Populated `OtbDepletionData` object containing: * Lead time mappings. * Current SKU-store/warehouse inventory. * Lists of relevant styles and seasons. * Initialized `OtbDepletionArgs` and `CommonArgs`.

Unusual Cases: * **Missing or delayed data feed:** If inventory or lead time feeds are late, the module will process with stale data, potentially leading to inaccurate initial stock counts and allocation plans. * **Incorrect season configuration:** If the `ros_season_list` is misconfigured, the module might exclude valid historical sales data, leading to skewed ROS calculations in later stages.

Next Steps: The initialized data from this module is immediately consumed by `OtbDepletionPrepareDataModule`, `OtbDepletionFirstMonthRosCalcModule`, `OtbDepletionSalesCalcModule`, and `OtbDepletionComputeRevenueModule` to build upon the foundational datasets.

Common Questions: 1. How often is the inventory data refreshed by this module? 2. What happens if a new style is introduced mid-season? Does it get picked up? 3. Where does the lead time data originate from? 4. Can we manually override a season list if needed? 5. What are the critical success factors for this module's input data?

Quick Reference: * **Lead Times:** Time from order to delivery. * **Current Inventory:** On-hand stock at stores and warehouses. * **SKU-store level:** Specific product unit at a specific store. * **Relevant Styles:** Products included in the current depletion run.

8.2. OtbDepletionPrepareDataModule

Business Objective: To consolidate and prepare all necessary master and transactional data for the core allocation engine, ensuring that rules regarding distribution, stock availability, and attribute-based mappings are correctly applied.

Required Information:

- **Sales Data:**
 - Open customer orders (stock reserved for these).
 - Goods in transit (incoming stock).
- **Product Data:**
 - Attribute Group (AG) mappings (e.g., color, size, brand associations).
 - Overridden channel-styles (e.g., specific styles only available online).
- **Operational Data:**
 - Distribution store configurations (which stores receive from which warehouses).
 - Current store and warehouse stock levels (SKU-level).

Business Settings:

- **Distribution Network Rules:** Defines how stores are assigned to warehouses for fulfillment.
- **Channel Overrides:** Specifies rules for product availability across sales channels (e.g., online vs. in-store).

Process Flow:

START --> Configure Distribution Stores --> Process Channel Overrides --> Load Stock & In-Transit --> Integrate Attributes --> END

Step-by-Step Actions:

1. **Configure** store distribution assignments *to determine which warehouses serve which stores, impacting allocation paths.*
2. **Process** overridden channel-styles *to ensure products are correctly identified as available or restricted for specific sales channels.*
3. **Load** current store and warehouse stock *along with any open orders and goods in transit to get a complete picture of available and incoming inventory.*
4. **Integrate** attribute-related mappings *such as Attribute Groups (AG) to support sophisticated allocation logic based on product characteristics.*
5. **Populate** OtbDepletionWhAllocData *with all the prepared data for the allocation module.*

Sample Scenario: For a new denim jacket (SKU 123), this module would **verify** that Store A is served by Warehouse X, **confirm** that this jacket can be sold both online and in-store, **load** 50 units currently in Warehouse X, **note** 10 units inbound from a supplier, and **identify** its attribute group as 'Outerwear'. It also checks for existing customer orders that might reserve some units. **Key Takeaway:** This module is the data backbone for allocation, ensuring all relevant stock, orders, and distribution rules are correctly accounted for before any allocation decisions are made.

Results: * Populated `OtbDepletionWhAllocData` object with: * SKU-store/warehouse stock, including online reserve quantities. * Inbound inventory (goods in transit, open orders). * Store-category-warehouse mappings. * Attribute group mappings.

Unusual Cases: * **Missing or delayed data feed:** Incomplete stock or inbound data can lead to over-allocation or missed sales opportunities if available stock isn't recognized. * **Inconsistent master data:** If a store's assigned warehouse changes but isn't updated, stock may be incorrectly allocated or not allocated at all.

Next Steps: The comprehensively prepared data in `OtbDepletionWhAllocData` is directly consumed by `OtbDepletionAllocationModule` as its primary input for stock distribution.

Common Questions: 1. How are distribution store assignments determined? 2. What defines an "overridden channel-style"? 3. Are returns considered in the stock levels? 4. Can new distribution rules be quickly implemented? 5. How frequently is the transactional data updated?

Quick Reference: * **AG (Attribute Group):** Grouping of product characteristics (e.g., material, fit). * **Goods In Transit:** Stock that has left a warehouse but not yet arrived at a store. * **Distribution Stores:** Stores assigned to specific warehouses. * **Open Orders:** Customer orders awaiting fulfillment.

8.3. `OtbDepletionFirstMonthRosCalcModule`

Business Objective: To accurately determine the initial Rate of Sale (ROS) for the first month of depletion, specifically for each relevant store-style combination, providing a crucial demand signal for early allocation decisions.

Required Information:

- **Sales Data:**
 - Historical sales data at the SKU-store level.
- **Product Data:**
 - Channel-specific styles relevant for ROS calculation.
- **Operational Data:**
 - Product availability data, specifically Pivotal Size Availability (PSA) conditions.

Business Settings:

- **PSA Conditions:** Defines the criteria for considering a product "available" (e.g., minimum size set available) for ROS calculation.

Process Flow:

START --> Filter Sales Data by Channel/Style --> Evaluate PSA Conditions --> Calculate Store-Style ROS (First Month) --> Persist Results --> END

Step-by-Step Actions:

1. **Filter** historical sales data to include only relevant channel-specific styles and periods for the first month's analysis.
2. **Evaluate** product availability against predefined PSA (Pivotal Size Availability) conditions to ensure ROS is calculated only for periods where the product was adequately stocked.
3. **Compute** Rate of Sale (ROS) for each store-style combination based on the filtered and availability-adjusted historical sales data.
4. **Store** the calculated first month's style ROS within the `depletionData` object for use by subsequent modules.

Sample Scenario: A popular summer dress (Style 'SummerGlow') sold 100 units across 5 stores last year during its first month. This module **filters** these sales, **checks** if pivotal sizes (S, M, L) were consistently in stock (PSA met) during that period. If so, it **calculates** an average first month ROS of 20 units/store for 'SummerGlow'. **Key Takeaway:** An accurate first-month ROS is vital for quickly getting the right initial stock to stores, especially for new or seasonal products.

Results: * `depletionData` object updated with `firstMonthStyleRos` for each relevant store-style.

Unusual Cases: * **Missing or delayed data feed:** If historical sales data is incomplete, ROS calculations will be inaccurate, leading to misinformed initial allocations. * **Inadequate PSA definition:** Overly strict or loose PSA conditions might exclude valid sales or include "broken" sales, skewing the ROS calculation.

Next Steps: The calculated `firstMonthStyleRos` is a direct input for `OtbDepletionPeriodRosCalcModule` to derive SKU-level ROS across all depletion periods.

Common Questions: 1. How is "first month" defined for ROS calculation? 2. What constitutes a "pivotal size" for PSA? 3. Can this module handle new products with no historical sales? 4. What are the key drivers for fluctuations in first-month ROS? 5. How does this module handle promotional periods in historical sales?

Quick Reference: * **ROS (Rate of Sale):** Speed at which products sell. * **PSA (Pivotal Size Availability):** Availability of key sizes to ensure product sellability. * **Store-Style:** A specific product style at a particular store location.

8.4. OtbDepletionSalesCalcModule

Business Objective: To forecast precise sales quantities for every store-category-period combination within the depletion horizon, using strategic inputs like OD (Optimum Depth) segmentation and Annual Operating Plan (AOP) revenue.

Required Information:

- **Sales Data:**
 - OD (Optimum Depth) segmentation output (specifically revenue contribution).
 - Annual Operating Plan (AOP) revenue targets.
- **Product Data:**
 - ASP (Average Selling Price) data.

- **Operational Data:**
 - Relevant styles for forecasting.

Business Settings:

- **OD Segmentation Parameters:** Rules defining product depth and revenue contribution.
- **AOP Integration Rules:** How annual revenue targets are disaggregated and applied to forecasts.

Process Flow:

START --> Integrate OD/AOP Data --> Forecast Sales Qty (Store-Category-Period) --> Persist Results --> END

Step-by-Step Actions:

1. **Integrate** OD (Optimum Depth) segmentation output *to understand the revenue contribution of various product depths.*
2. **Incorporate** Annual Operating Plan (AOP) revenue targets *to align forecasted sales with overall business goals.*
3. **Calculate** forecasted sales quantities *for each unique store-category-period combination within the defined depletion horizon, using the integrated OD and AOP data along with ASP information.*
4. **Persist** the calculated sales quantities *into the depletionData object for subsequent ROS normalization and allocation processes.*

Sample Scenario: A store has an AOP target of \$10,000 for the ‘Women’s Shoes’ category for the next month. This module **uses** the OD segmentation which shows ‘Sneakers’ contribute 40% of revenue in that category. If the average ASP for sneakers is \$100, the module **forecasts** 40 units ($\$4,000 / \100) for sneakers for that store for the next month. **Key Takeaway:** Accurate sales forecasting directly informs how much stock is needed, preventing both overstock and understock situations.

Results: * depletionData object updated with salesQty (forecasted sales quantities) per store-category-period.

Unusual Cases: * **Missing or delayed data feed:** If OD segmentation or AOP data is incomplete, sales forecasts will be unreliable, leading to inefficient inventory deployment. * **Sudden market shift:** Unexpected changes in consumer behavior or competitive landscape not reflected in AOP or historical OD data can lead to inaccurate forecasts.

Next Steps: The forecasted sales quantities are used by `OtbDepletionPeriodRosCalcModule` to scale SKU-level ROS computations.

Common Questions: 1. How frequently is the AOP updated and factored into forecasts? 2. What is the look-back period for OD segmentation input? 3. Can this module account for seasonality in sales forecasts? 4. How are new stores handled if they don’t have historical OD data? 5. What level of granularity (SKU, category, style) does this module forecast at?

Quick Reference: * **OD (Optimum Depth):** Ideal number of unique items (SKUs) to stock. * **AOP (Annual Operating Plan):** Yearly revenue targets. * **ASP (Average Selling Price):** Average price at which products are sold. * **Sales Quantity:** Forecasted number of units to be sold.

8.5. OtbDepletionPeriodRosCalcModule

Business Objective: To compute precise SKU-level Rate of Sale (ROS) for all depletion periods, ensuring that sales velocity aligns with style-level ROS and incorporating optimal size contributions derived from Ideal Size Set (ISS) outputs.

Required Information:

- **Sales Data:**
 - First month's style ROS (from OtbDepletionFirstMonthRosCalcModule).
 - Forecasted sales quantities (from OtbDepletionSalesCalcModule).
- **Product Data:**
 - ISS (Ideal Size Set) output (adjusted size contributions).
- **Operational Data:**
 - Reference store data for replicating ROS for new stores.

Business Settings:

- **ISS Parameters:** Defines the ideal distribution of sizes within a style.
- **ROS Normalization Rules:** How SKU ROS is scaled to align with style ROS.
- **Reference Store Logic:** Rules for assigning reference stores to new or low-volume stores for ROS replication.

Process Flow:

START --> Leverage First Month Style ROS --> Incorporate ISS Size Contributions --> Normalize SKU ROS to Style ROS --> Replicate ROS for New Stores --> Store SKU-level ROS --> END

Step-by-Step Actions:

1. **Leverage** the first month's style ROS as a baseline for calculating SKU-level ROS across subsequent depletion periods.
2. **Incorporate** adjusted size contributions derived from the ISS (Ideal Size Set) output to ensure that the ROS for each SKU reflects its optimal proportion within the style.
3. **Normalize** SKU ROS values to ensure their sum aligns precisely with the overall style ROS, maintaining consistency and accuracy.
4. **Replicate** ROS data for new stores or stores lacking sufficient historical data by mapping them to relevant reference stores.
5. **Compute and store** the final SKU-level Rate of Sale (ROS) for all store-SKU-period combinations in otbDepletionData.

Sample Scenario: A 'T-shirt' style has a ROS of 100 units/week. The ISS indicates that size Medium should contribute 30% of sales. This module **calculates** that the ROS for 'T-shirt Medium' is 30 units/week. For a brand new store with no history, it

assigns a reference store's T-shirt ROS data to it. **Key Takeaway:** Precise SKU-level ROS ensures that stock is distributed not just by style, but by the specific sizes and variants that are most likely to sell, preventing size imbalances.

Results: * otbDepletionData updated with storeSkuRos (SKU-level Rate of Sale) for all store-SKU-period combinations.

Unusual Cases: * **Missing or delayed data feed:** If first-month ROS or ISS data is unavailable, SKU-level ROS might default to less accurate methods or use outdated proportions. * **Incorrect ISS application:** If ISS output is flawed, the SKU ROS distribution might not reflect true demand, leading to size stockouts or overstock.

Next Steps: The computed SKU-level ROS is a critical input for the OtbDepletionAllocationModule for guiding detailed allocation decisions.

Common Questions: 1. How frequently is the ISS output updated and applied? 2. What is the definition of a "depletion period"? 3. How does the system select reference stores for new locations? 4. Can ROS be manually adjusted for specific SKUs or periods? 5. What metrics indicate an accurate SKU-level ROS calculation?

Quick Reference: * **ISS (Ideal Size Set):** Best combination of sizes. * **SKU-level ROS:** Sales rate for a specific product item (SKU). * **Normalization:** Adjusting values to fit a consistent scale (e.g., sum to 100%). * **Reference Store:** A similar, established store used to infer data for new stores.

8.6. OtbDepletionPrepareSalesModule

Business Objective: To refine raw sales data by eliminating irrelevant transactions (like liquidation sales) and identifying "broken" sales periods based on product availability, ensuring that only high-quality, representative sales data is used for further analysis.

Required Information:

- **Sales Data:**
 - Raw historical sales data.
- **Operational Data:**
 - PSA (Pivotal Size Availability) benchmarks.
 - Liquidation sales flags/data.

Business Settings:

- **Liquidation Sales Rules:** Defines criteria for identifying and excluding sales from liquidation events.
- **PSA Benchmarks:** Thresholds for product availability that determine if a sales period was "broken" (e.g., minimum percentage of core sizes in stock).

Process Flow:

START --> Load Raw Sales --> Remove Liquidation Sales --> Filter for "Brokenness" (PSA) --> Compute Initial SKU ROS (Clean) --> Store Cleaned Sales --> END

Step-by-Step Actions:

1. **Load** raw historical sales data *from the data warehouse or cache.*
2. **Remove** liquidation sales *from the dataset, as these often do not represent typical demand and can skew future forecasts.*
3. **Filter** sales data for "brokenness" *by assessing product availability against predefined PSA (Pivotal Size Availability) benchmarks; sales during periods of poor stock availability are excluded as they don't reflect true demand.*
4. **Compute** initial SKU ROS *from the clean, unliquidated sales data to provide a baseline for further ROS calculations.*
5. **Make available** the cleaned and filtered sales data *within OtbDepletionWhAllocData for use by downstream modules like segmentation and revenue computation.*

Sample Scenario: A jacket sold 50 units in January, but 30 of those were during a 70% off liquidation event. Also, in February, it sold only 5 units because sizes S and L were out of stock for 3 weeks (failing PSA). This module **removes** the 30 liquidation sales. It also **marks** February sales as "broken" and excludes them from ROS calculations, leaving only 20 units from January as valid, unliquidated sales.

Key Takeaway: Cleaning sales data is paramount for accurate demand forecasting; including abnormal sales or periods of low stock can lead to severely distorted business insights.

Results: * Cleaned and filtered sales data available in OtbDepletionWhAllocData.
* Initial SKU ROS calculated from clean sales.

Unusual Cases: * **Missing or delayed data feed:** If raw sales data is incomplete, the cleaning process may inadvertently retain or discard incorrect sales, leading to skewed analyses. * **Incorrect liquidation flags:** If certain sales are misidentified as liquidation (or vice versa), the cleaned sales data will not accurately reflect regular demand.

Next Steps: The cleaned sales data is consumed by OtbDepletionFirstMonthRosCalcModule, OtbDepletionSegmentationModule, OtbDepletionComputeRevenueModule, and OtbDepletionRankingModule to ensure their analyses are based on valid sales figures.

Common Questions: 1. How does the system define "liquidation sales"? 2. What is the impact of removing "broken" sales on overall volume? 3. Can we see a report of sales excluded due to "brokenness"? 4. How are promotional sales (non-liquidation) handled? 5. What is the minimum PSA threshold for sales to be considered "unbroken"?

Quick Reference: * **Liquidation Sales:** Sales from clearance or end-of-life events.
* **Brokenness:** Periods where product availability was insufficient to meet demand.
* **PSA (Pivotal Size Availability):** Measure of key size availability. * **Cleaned Sales:** Sales data free from anomalies and availability issues.

8.7. OtbDepletionSegmentationModule

Business Objective: To identify and categorize top-selling store-styles from the cleaned sales data, providing a critical input for prioritization during the subsequent allocation process.

Required Information:

- **Sales Data:**
 - Cleaned sales data (from OtbDepletionPrepareSalesModule).
- **Product Data:**
 - Product master data for style identification.

Business Settings:

- **Top-Selling Criteria:** Defines how “top-selling” is measured (e.g., by revenue, unit sales, ROS).

Process Flow:

START --> Load Cleaned Sales --> Identify Top-Selling Store-Styles
--> Segment & Prioritize --> Store Segmentation Results --> END

Step-by-Step Actions:

1. **Load** the cleaned sales data *from OtbDepletionWhAllocData to ensure accurate segmentation based on valid sales figures.*
2. **Identify** top-selling store-styles *by applying predefined business logic and thresholds (e.g., styles exceeding a certain ROS or revenue target within a specific store).*
3. **Segment** these identified store-styles *into prioritized groups that will receive preferential treatment during the iterative allocation process.*
4. **Store** the segmentation results *within OtbDepletionWhAllocData by marking relevant StoreStyle objects, influencing subsequent revenue computation and ranking.*

Sample Scenario: After cleaning sales, a specific ‘running shoe’ style shows consistently high sales (500 units/month) in Store B and Store D. This module **identifies** these two ‘store-style’ combinations as top-performers. It then **flags** them as “priority” for future allocation, ensuring they are always considered first for stock. **Key Takeaway:** Prioritizing allocation efforts towards high-performing store-styles maximizes sales potential and prevents stockouts where demand is strongest.

Results: * StoreStyle objects within OtbDepletionWhAllocData are updated with segmentation flags (e.g., isTopSeller).

Unusual Cases: * **Missing or delayed data feed:** If cleaned sales data is incomplete, high-performing store-styles might be missed, leading to suboptimal allocation. * **Flawed top-selling criteria:** If the criteria for “top-selling” are too broad or too narrow, the segmentation might not effectively prioritize the truly impactful store-styles.

Next Steps: The segmentation results implicitly influence `OtbDepletionComputeRevenueModule` and explicitly guide `OtbDepletionRankingModule` and `OtbDepletionAllocationModule` by providing a prioritized list of store-styles.

Common Questions: 1. How often are the top-selling segments re-evaluated? 2. Can different products have different top-selling criteria? 3. What happens if a store-style suddenly drops out of the top segment? 4. Is the segmentation based on units, revenue, or both? 5. How can we review the store-styles identified as top-selling?

Quick Reference: * **Segmentation:** Dividing data into distinct groups based on characteristics. * **Store-Style:** A specific product style at a particular store location. * **Prioritization:** Ranking based on business importance.

8.8. `OtbDepletionComputeRevenueModule`

Business Objective: To calculate revenue per day for various store-style combinations and identify “read-only” store-styles based on seasonal attributes, ensuring allocation rules consider both financial contribution and product lifecycle.

Required Information:

- **Sales Data:**
 - Cleaned sales data (from `OtbDepletionPrepareSalesModule`).
- **Product Data:**
 - Seasonal attributes (e.g., `readSeasonList`, `replenishmentSeasonList`).
- **Operational Data:**
 - Reference store data for replicating revenue for new stores.

Business Settings:

- **Read-Only Season Lists:** Defines which seasons’ products should not be allocated additional stock (e.g., end-of-season items).
- **Revenue Calculation Logic:** How daily revenue is derived from sales data.

Process Flow:

START --> Load Cleaned Sales --> Calculate Revenue Per Day --> Identify Read-Only Store-Styles --> Replicate for New Stores --> Store Revenue Data --> END

Step-by-Step Actions:

1. **Load** cleaned sales data to ensure revenue calculations are based on valid and representative sales figures.
2. **Calculate** revenue per day for various store-style combinations, providing a standardized metric for comparison and prioritization.
3. **Identify** and designate “read-only” store-styles based on their seasonal attributes (e.g., products from past seasons that should only be sold down, not replenished).
4. **Replicate** revenue metrics for new stores or those with limited data by using data from their assigned reference stores.
5. **Set** revenue per day and read-only status on `StoreStyle` objects within `OtbDepletionWhAllocData` for use by ranking and allocation modules.

Sample Scenario: A specific ‘winter coat’ style sold for \$10,000 over 10 days in Store A, leading to a revenue per day of \$1,000. If ‘Winter 2024’ is flagged as a readSeasonList, this module **designates** all ‘Winter 2024’ coats in Store A as ‘read-only’, meaning they won’t receive new stock. **Key Takeaway:** Understanding revenue per day helps prioritize higher-value allocation, while ‘read-only’ status ensures efficient depletion of aging or out-of-season stock.

Results: * StoreStyle objects in OtbDepletionWhAllocData are updated with revPerDayForComparison and readOnly flags.

Unusual Cases: * **Missing or delayed data feed:** If cleaned sales data is incomplete, revenue calculations will be inaccurate, potentially misprioritizing allocation. * **Incorrect season configuration:** If the readSeasonList is misconfigured, active season products might be incorrectly marked as “read-only”, hindering replenishment.

Next Steps: The calculated revenue per day is used by OtbDepletionRankingModule for prioritization, and the read-only status is considered by OtbDepletionAllocationModule to apply specific allocation rules.

Common Questions: 1. How is “revenue per day” calculated for new styles with limited sales history? 2. What is the purpose of marking a style as “read-only”? 3. Can a store-style transition out of “read-only” status? 4. How does this module handle different pricing strategies across stores? 5. What impact does a low revenue per day have on allocation?

Quick Reference: * **Revenue Per Day:** Financial contribution per day for a store-style. * **Read-Only Store-Style:** A style that should only be sold down, not replenished. * **Seasonal Attributes:** Product characteristics tied to specific seasons. * **Reference Stores:** Used for data replication for new/low-volume stores.

8.9. OtbDepletionRankingModule

Business Objective: To prioritize allocation efforts by ranking store-styles based on their calculated revenue per day and computing detailed store-SKU and store-category-size rankings for various allocation strategies.

Required Information:

- **Sales Data:**
 - Revenue per day (from OtbDepletionComputeRevenueModule).
 - Unliquidated sales data (from OtbDepletionPrepareSalesModule).
- **Product Data:**
 - Product information for SKU and category-size analysis.

Business Settings:

- **Ranking Criteria:** Defines how rankings are performed (e.g., primary ranking by revenue, secondary by units).
- **Non-Pivotal Allocation Strategy:** Rules for allocating stock to non-top-selling or lower-ranked items.

Process Flow:

START --> Load Revenue & Sales Data --> Rank Store-Styles by Revenue --> Compute Store-SKU Rankings --> Compute Store-Category-Size Rankings --> Store Rankings --> END

Step-by-Step Actions:

1. **Load** revenue per day *for store-styles and unliquidated sales data to use as the basis for ranking.*
2. **Rank** store-styles *primarily based on their calculated revenue per day, creating a prioritized list for the core allocation engine.*
3. **Compute** detailed store-SKU rankings *to facilitate specific SKU-level allocation decisions.*
4. **Compute** store-category-size rankings *which are particularly useful for non-pivotal allocation strategies, ensuring that even lower-ranked items receive appropriate size distribution.*
5. **Update** `OtbDepletionWhAllocData` *by sorting the StoreStyle list and populating other ranking data structures.*

Sample Scenario: Store C has 'Style A' with a \$500/day revenue and 'Style B' with \$300/day. This module **ranks** 'Style A' higher. It also **computes** that within 'Style A', size Medium is the top-selling SKU, and within the 'Dresses' category, size Small is the most popular size overall. **Key Takeaway:** Effective ranking ensures that high-impact products and stores are prioritized for allocation, maximizing sales potential, while also supporting strategies for broader product distribution.

Results: * `OtbDepletionWhAllocData` is updated with a sorted `storeStylesList` based on revenue ranking. * Detailed store-SKU and store-category-size rankings are available.

Unusual Cases: * **Missing or delayed data feed:** If revenue per day data is unavailable or outdated, rankings will be inaccurate, potentially leading to suboptimal allocation. * **Inconsistent ranking metrics:** Using different or conflicting metrics for various rankings can lead to allocation paradoxes.

Next Steps: The prioritized and ranked lists from this module are directly consumed by `OtbDepletionAllocationModule` to guide its iterative allocation strategies.

Common Questions: 1. How frequently are these rankings refreshed? 2. Can we see the specific ranking criteria used for each type (store-style, SKU, size)? 3. How do new styles with no sales history get ranked? 4. What is the difference between pivotal and non-pivotal allocation strategies? 5. Can store managers influence local product rankings?

Quick Reference: * **Revenue Per Day:** Primary metric for store-style ranking. * **Store-SKU Ranking:** Priority for specific product units at a store. * **Store-Category-Size Ranking:** Priority for specific sizes within a product category at a store. * **Non-Pivotal Allocation:** Strategies for allocating lower-priority or specialty items.

8.10. OtbDepletionAllocationModule

Business Objective: To efficiently allocate SKUs and manage warehouse stock through iterative strategies including replenishment, adherence to planogram layouts (OW), and compliance with ideal size sets (ISS), ultimately generating comprehensive depletion output reports.

Required Information:

- **Sales Data:**
 - SKU-level Rate of Sale (ROS) (from OtbDepletionPeriodRosCalcModule).
- **Product Data:**
 - Prepared master and transactional data (from OtbDepletionPrepareDataModule), including current stock, inbound inventory, OW, and ISS.
- **Operational Data:**
 - Store-style revenue and read-only status (from OtbDepletionComputeRevenueModule).
 - Prioritized and ranked lists (from OtbDepletionRankingModule).

Business Settings:

- **Allocation Strategies:** Defines the sequence and logic of allocation passes (e.g., replenishment, OW, ISS).
- **OW (Optimum Width):** Ideal number of different categories or styles to offer per display/store.
- **ISS (Ideal Size Set):** Best combination of sizes to maximize sales.
- **Depletion Horizon:** The timeframe over which stock is depleted and allocated.

Process Flow:

START --> Load All Prepared Data --> Iterate Allocation Strategies (Replenishment, OW, ISS) --> Track Depletion Over Time --> Generate Output Rows --> Store Raw Outputs --> END

Step-by-Step Actions:

1. **Load** all prepared data *including ROS, current stock, inbound inventory, distribution network maps, revenue rankings, and read-only flags.*
2. **Execute** iterative allocation strategies *such as initial replenishment, adjusting to Optimum Width (OW) for category breadth, and refining Ideal Size Set (ISS) for optimal size distribution.*
3. **Diligently track** depletion over time *for each SKU at each location, ensuring that stock is allocated in alignment with forecasted demand and business rules.*
4. **Generate** raw depletion output rows *for each allocation decision, capturing details like SKU, store, quantity, and allocation type.*
5. **Store** the collected raw output rows *within otbDepletionData for subsequent processing and reporting.*

Sample Scenario: A store needs 10 units of a high-ROS shoe (Style A) over the next week. The warehouse has 50 units. The module **allocates** 10 units to the store. If the store's "Sportswear" category is under its OW, the module might then **allocate** a new 'sportswear' style to fill the gap. Finally, if the store is missing a pivotal size

(size 9) in an otherwise well-stocked style, the module will **allocate** 2 units of size 9 to correct the ISS. **Key Takeaway:** This is the core engine, turning all previous data and settings into actionable allocation decisions, directly impacting sales and inventory health.

Results: * otbDepletionData updated with depletionOutputRows, detailing every allocation decision. * Internal stock levels updated to reflect allocations.

Unusual Cases: * **Missing or delayed data feed:** If core inputs like ROS or current inventory are missing, the allocation engine will make decisions based on incomplete data, leading to stockouts or overstocks. * **Warehouse stock inconsistencies:** If the physical stock in the warehouse does not match the system's recorded stock, allocations may be impossible to fulfill, or too much stock might be held. * **Conflicting business rules:** If OW, ISS, and replenishment rules conflict (e.g., an OW rule demands a new style, but ISS demands more of an existing size, and there's limited stock), the module must prioritize, which might not align with desired outcomes.

Next Steps: The raw depletion output rows are processed by DepletionOutputsModule to generate final, denormalized reports and health metrics.

Common Questions: 1. How does the system prioritize between replenishment, OW, and ISS rules when stock is limited? 2. Can the allocation rules be adjusted for specific regions or store types? 3. What happens if an allocation cannot be fulfilled due to insufficient warehouse stock? 4. How does the system handle returns or unexpected incoming stock during the depletion run? 5. What are the key performance indicators (KPIs) this module aims to optimize?

Quick Reference: * **OW (Optimum Width):** Ideal number of different categories/styles. * **ISS (Ideal Size Set):** Best combination of sizes. * **Replenishment:** Restocking to a target level. * **Allocation Strategies:** Rules and priorities for distributing stock. * **Depletion Horizon:** Time period for depletion planning.

8.11. DepletionOutputsModule

Business Objective: To transform raw depletion output into comprehensive, denormalized, and exportable reports, providing crucial post-depletion health metrics and consolidated inventory information for business analysis and decision-making.

Required Information:

- **Operational Data:**
 - Raw depletion output rows (from OtbDepletionAllocationModule).
 - Opening warehouse and store inventory information (from initial data loads).
- **Sales Data:**
 - Pre-depletion sales forecasts.

Business Settings:

- **PSA Calculation Rules:** Defines how Pivotal Size Availability is computed from final stock data.

- **Reporting Granularity:** Specifies the level of detail for output reports (e.g., SKU, style, category).

Process Flow:

START --> Load Raw Depletion Output --> Denormalize Data Rows --> Compute Health Metrics (e.g., PSA) --> Consolidate Inventory Info --> Generate Exportable Reports --> END

Step-by-Step Actions:

1. **Load** the raw depletion output rows *generated by the `OtbDepletionAllocationModule`, which contain all allocation decisions.*
2. **Denormalize** the data rows *to combine related information into single, flat records suitable for reporting and external consumption.*
3. **Compute** crucial post-depletion health metrics *such as PSA (Pivotal Size Availability) percentages, to assess the effectiveness of the allocation on product availability.*
4. **Consolidate** opening warehouse and store inventory information *to provide a clear picture of stock levels before and after depletion.*
5. **Generate** comprehensive and exportable data formats *ready for business intelligence tools, dashboards, or manual review.*

Sample Scenario: After allocation, a store received 5 units of ‘Style A’. This module **processes** the raw allocation row, **calculates** the post-depletion PSA for ‘Style A’ (e.g., now 90% due to optimal size delivery). It then **combines** this with the store’s opening inventory and **exports** a report showing ‘Store X: Style A, 5 units allocated, New PSA 90%’. **Key Takeaway:** This module provides the critical feedback loop, translating complex allocation logic into digestible reports that allow businesses to measure success and identify areas for improvement.

Results: * Denormalized and exportable data files/tables for analysis. * Computed PSA percentages and other post-depletion health metrics. * Consolidated inventory reports.

Unusual Cases: * **Missing or delayed data feed:** If raw depletion output is incomplete or corrupted, final reports will be inaccurate, hindering performance analysis. * **Incorrect PSA calculation rules:** If the rules for PSA are misconfigured, the reported health metrics will not accurately reflect product availability post-allocation.

Next Steps: The generated outputs are typically ingested by business intelligence tools, data warehouses, or presented directly to business users for insights and further strategic planning.

Common Questions: 1. What are the key metrics included in the final reports? 2. How can we customize the output report format or content? 3. Are these reports available in real-time or only after a full run? 4. How do we interpret a low PSA percentage post-depletion? 5. Can we compare actual sales performance against the depletion forecasts from these reports?

Quick Reference: * **Denormalization:** Combining data from multiple tables into a single table for easier reporting. * **Exportable Data:** Formats suitable for external systems (e.g., CSV, JSON). * **Health Metrics:** Key performance indicators of the depletion process (e.g., PSA). * **PSA (Pivotal Size Availability):** Post-allocation availability of key sizes.

9. Appendix

Full Parameter Reference

No `args.properties` data was provided for rendering a full parameter reference. Please consult the `args.properties` file in the project's root directory for detailed parameter documentation.

Version History

- **Version 1.0** (2025-05-23): Initial release of the Depletion Documentation.

Contact & Support Info

For any questions, support, or further documentation, please contact the Product Operations team at `product.ops@example.com` or raise a ticket through our internal support portal.

Tutorial: depletion

The depletion process orchestrates a series of modules to manage SKU allocation and warehouse stock depletion. It starts with input data loading and preparation, followed by a parallel track of sales data cleaning and ROS calculation. Forecasted sales and revenue computations then feed into segmentation and ranking modules, which in turn inform the core allocation logic. The final module processes and exports the depletion outputs for reporting. Data is primarily shared through in-memory data structures like `OtbDepletionData` and `OtbDepletionWhAllocData`, with intermediate and final results persisted to the database.

flowchart TD

```
A0["OtbDepletionInputModule"]
A1["OtbDepletionPrepareDataModule"]
A2["OtbDepletionFirstMonthRosCalcModule"]
A3["OtbDepletionSalesCalcModule"]
A4["OtbDepletionPeriodRosCalcModule"]
A5["OtbDepletionPrepareSalesModule"]
A6["OtbDepletionSegmentationModule"]
A7["OtbDepletionComputeRevenueModule"]
A8["OtbDepletionRankingModule"]
A9["OtbDepletionAllocationModule"]
A10["DepletionOutputsModule"]
A0 -- "depends_on" --> A1
A0 -- "data_flow" --> A2
A0 -- "data_flow" --> A3
A0 -- "data_flow" --> A7
A1 -- "data_flow" --> A9
A5 -- "data_flow" --> A2
A5 -- "data_flow" --> A6
A5 -- "data_flow" --> A7
A5 -- "data_flow" --> A8
A2 -- "data_flow" --> A4
A3 -- "data_flow" --> A4
A4 -- "data_flow" --> A9
A6 -- "control_flow" --> A7
A7 -- "data_flow" --> A8
A7 -- "data_flow" --> A9
A8 -- "data_flow" --> A9
A9 -- "data_flow" --> A10
```

Chapters

Master Report: depletion

1. Title & Metadata

- **Project Name:** depletion
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** A data-driven merchandising algorithm designed to optimize inventory levels and product assortment by analyzing sales velocity and strategic objectives.

2. Executive Summary

The depletion algorithm is a core component of our merchandising strategy, providing data-driven recommendations for optimal product assortment and inventory levels. It leverages historical sales data and various business parameters to calculate Ideal Size Sets (ISS), Optimum Depth (OD), and Optimum Width (OW) for categories and products. This system aims to minimize stockouts, reduce excess inventory, and maximize sales efficiency across our product catalog.

- **Key outcomes or KPIs:**
 - Improved inventory turnover.
 - Reduced stockouts and lost sales opportunities.
 - Optimized product mix (OD, OW) per category and display.
 - Enhanced gross margin through better inventory management.
 - More accurate Ideal Size Set (ISS) allocation per SKU.
- **Primary stakeholders or downstream consumers:**
 - Merchandising Teams
 - Inventory Planning & Allocation Teams
 - Category Managers
 - Supply Chain & Operations
 - Retail Store Operations

3. Architecture & Data Flow

Diagram:

graph TD

```
A[Raw Sales & Inventory Data] --> B{Data Ingestion & Preprocessing};
C[Product Catalog & Attributes] --> B;
D[Configuration & Business Rules] --> B;
B --> E[Sales Velocity & Forecasting];
```

```

E --> F[Optimum Depth & Width Calculation];
E --> G[Ideal Size Set Determination];
F --> H[Recommendation Engine];
G --> H;
H --> I[OD Recommendations];
H --> J[OW Recommendations];
H --> K[ISS Recommendations];
H --> L[Performance Insights];

```

Component Bullets:

- **Data Ingestion & Preprocessing:** Gathers raw data from various sources, performs cleaning, validation, and standardization.
- **Sales Velocity & Forecasting:** Analyzes historical Rate of Sale (ROS) and generates future sales forecasts for individual SKUs and Attribute Groups (AG).
- **Optimum Depth & Width Calculation:** Determines the ideal number of unique items (OD) and categories/styles (OW) based on forecasts, space constraints, and strategic goals.
- **Ideal Size Set Determination:** Calculates the best combination of sizes (ISS) for each SKU to maximize sales and minimize stockouts, considering historical sales distributions.
- **Recommendation Engine:** Synthesizes inputs from forecasting, OD/OW, and ISS modules to generate actionable merchandising and inventory recommendations.
- **Output Generation:** Formats and delivers recommendations in various structured formats for consumption by downstream systems and stakeholders.

High-level data sources:

- **POS Data:** Transactional sales data, including units sold, date, price, and location.
- **Inventory Data:** Current stock levels, in-transit inventory, and location details.
- **Product Master Data:** SKU attributes, hierarchy (category, sub-category), cost, retail price.
- **Store/Display Data:** Store capacity, display dimensions, traffic data.
- **Historical Promotions & Events:** Information on past marketing campaigns that influenced sales.

4. Configuration & Data Dictionary

Settings Table (args.properties):

Setting Name	Description	Default Value	Unit/Type
forecast.horizon.weeks	Number of weeks into the future to forecast sales.	12	Weeks
ros.min_history_weeks	Minimum weeks of sales history	4	Weeks

Setting Name	Description	Default Value	Unit/Type
	required for ROS calculation.		
od.max_skus_per_category	Hard limit on OD for any given category.	50	SKUs
ow.max_categories_per_display	Hard limit on OW for any given display area.	10	Categories
iss.min_sales_threshold	Minimum total sales (units) for ISS calculation to be reliable.	100	Units
inventory.safety_stock_factor	Multiplier for average weekly ROS to determine safety stock.	1.2	Factor
attribute_group.mapping_file	Path to file mapping SKUs to their respective Attribute Groups (AG).	ag_map.csv	File Path
seasonality.adjustment_factor	Factor to adjust forecasts based on known seasonality patterns.	1.0 (dynamic)	Factor
output.recommendation_format	Desired output format for recommendations (e.g., CSV, JSON, Parquet).	CSV	String

Shared Data Fields (common-data.json):

Field Name	Type	Description
sku_id	String	Unique identifier for a Stock Keeping Unit.
category_id	String	Identifier for the product category.
style_id	String	Identifier for the product style or collection.
size_id	String	Specific size of the SKU (e.g., 'S', 'M', 'L', '32x32').
color_id	String	Specific color of the SKU.
week_start_date	Date	Start date of the sales week.
sales_units	Integer	Number of units sold in a given period.

Field Name	Type	Description
current_inventory	Integer	Current number of units in stock for a given SKU.
average_ros	Float	Calculated Rate of Sale (units/week).
predicted_ros	Float	Forecasted Rate of Sale (units/week).
attribute_group_id	String	Identifier for the Attribute Group (AG) the SKU belongs to.

Outputs Catalog:

Output File Name	Description	Format	Content
od_recommendations.csv	Recommended Optimum Depth (OD) per category/display.	CSV	category_id, display_id, recommended_od_skus, justification, forecasted_impact_sales
ow_recommendations.csv	Recommended Optimum Width (OW) per display/store.	CSV	display_id, store_id, recommended_ow_categories, justification, forecasted_impact_sales
iss_recommendations.csv	Recommended Ideal Size Set (ISS) per SKU (by color/style).	CSV	sku_id, attribute_group_id, recommended_sizes_json (e.g., {"S":20,"M":35,"L":30,"XL":15} representing percentage allocation or unit allocation), justification_method, size_curve_data
inventory_allocations.csv	Recommended inventory allocations based on OD, OW, and ISS.	CSV	sku_id, location_id, recommended_units_to_stock, current_inventory, target_inventory_based_on_iss, forecasted_demand_next_period
performance_summary.pdf	High-level summary of key KPIs and algorithm performance.	PDF	Dashboard-style report with charts and summaries on inventory turnover, stockout rates, sales lift from recommendations, and a comparison of current vs. recommended OD/OW/ISS, showing overall improvements and potential areas for further optimization. Includes trend analysis for ROS and a breakdown of performance by top-selling categories and underperforming items.
debug_logs.txt	Detailed logs for troubleshooting and auditing.	TXT	Time-stamped entries including module execution times, data validation errors, parameter overrides, intermediate calculation results, and any warnings or errors encountered

Output File Name	Description	Format	Content
			during the pipeline run. Useful for identifying issues with data quality, misconfigurations, or unexpected algorithmic behavior. Each entry provides enough context to trace back the origin of a problem, such as the specific SKU or category being processed at the time of an error.

5. End-to-End Flow

The depletion algorithm's end-to-end flow begins with the ingestion of raw sales, inventory, and product master data. This data undergoes a rigorous preprocessing stage where it's cleaned, validated, and transformed into a standardized format. Next, the system calculates the Rate of Sale (ROS) for each SKU and Attribute Group (AG) and generates future sales forecasts, incorporating seasonality and promotional effects. Concurrently, based on these forecasts and predefined business rules, the algorithm determines the Optimum Depth (OD) for unique items and Optimum Width (OW) for categories. Separately, the Ideal Size Set (ISS) is calculated for each SKU based on historical size distribution patterns. All these insights are then fed into the Recommendation Engine, which synthesizes them into actionable outputs: OD, OW, and ISS recommendations. These recommendations are finally formatted and delivered to various downstream systems and stakeholders, enabling optimized inventory planning, allocation, and merchandising decisions.

6. Submodule Overview

1. Data Ingestion & Preprocessing

- **Purpose:** To collect, clean, validate, and standardize raw sales, inventory, and product data for subsequent analysis.
- **Inputs:**
 - Raw POS data (sales units, date, SKU)
 - Raw Inventory data (current stock, SKU, location)
 - Product Master data (SKU attributes, category, style, size, color)
 - `attribute_group.mapping_file` (from `args.properties`)
- **Outputs:**
 - Cleaned and standardized `common-data.json` for all relevant fields (`sku_id`, `category_id`, `style_id`, `size_id`, `sales_units`, `current_inventory`, etc.).
 - Data quality report (optional, but good for robust systems).
- **Core Logic Summary:**
 - Load raw data from various sources (e.g., databases, CSVs).
 - Perform data cleaning: handle missing values, correct data types, remove duplicates.
 - Validate data integrity: check for logical consistency (e.g., non-negative sales).
 - Standardize product attributes and merge datasets into a unified format.

2. Sales Velocity & Forecasting

- **Purpose:** To calculate historical Rate of Sale (ROS) and generate future sales forecasts for all SKUs and Attribute Groups (AG).
- **Inputs:**
 - Cleaned common-data.json (containing sku_id, sales_units, week_start_date).
 - forecast.horizon.weeks, ros.min_history_weeks, seasonality.adjustment_factor (from args.properties).
- **Outputs:**
 - Updated common-data.json with average_ros and predicted_ros for each SKU.
- **Core Logic Summary:**
 - Calculate average_ros for each SKU based on historical sales_units over a defined period (ros.min_history_weeks).
 - Apply statistical forecasting models (e.g., ARIMA, Exponential Smoothing) to generate predicted_ros for the forecast.horizon.weeks.
 - Incorporate seasonality adjustments and promotional uplift factors into the forecasts.
 - Aggregate predicted_ros to the Attribute Group (AG) level for higher-level planning.

3. Optimum Depth & Width Calculation

- **Purpose:** To determine the ideal number of unique items (OD) and categories/styles (OW) to stock, optimizing product assortment.
- **Inputs:**
 - Updated common-data.json (with predicted_ros, category_id, style_id).
 - od.max_skus_per_category, ow.max_categories_per_display (from args.properties).
 - Store/display capacity data (implied from data sources).
- **Outputs:**
 - od_recommendations.csv
 - ow_recommendations.csv
- **Core Logic Summary:**
 - Analyze category performance and predicted_ros to identify high-potential SKUs for OD.
 - Evaluate category and style performance and their contribution to overall sales to determine OW.
 - Apply constraints such as maximum SKU limits per category (OD) and maximum category limits per display (OW).
 - Prioritize SKUs/categories based on profitability, strategic importance, and forecasted demand.

4. Ideal Size Set Determination

- **Purpose:** To identify the optimal mix of sizes (ISS) for each product to maximize sales and minimize stockouts, aligning with demand patterns.
- **Inputs:**
 - Updated common-data.json (with sku_id, size_id, sales_units).
 - iss.min_sales_threshold (from args.properties).

- **Outputs:**
 - `iss_recommendations.csv`
- **Core Logic Summary:**
 - For each `sku_id` (or `attribute_group_id`), analyze the historical distribution of `sales_units` across different `size_ids`.
 - Calculate a percentage allocation for each size based on its historical sales contribution.
 - Apply minimum sales thresholds (`iss.min_sales_threshold`) to ensure statistical reliability of size distributions.
 - Generate a recommended size curve or unit allocation for future inventory purchases.

7. Worked Example

Let's illustrate how the algorithm might determine the **Optimum Depth (OD)** for a specific category, "Women's Jeans".

Scenario: The Merchandising Team wants to optimize the number of unique jeans SKUs (OD) in a specific store, aiming for higher sales efficiency.

Input Data: * **Category:** Women's Jeans * **Store ID:** Store_001 * **Predicted ROS for top 5 Women's Jeans SKUs (units/week):** * SKU_WJ001 (Slim Fit Blue): 50 * SKU_WJ002 (Bootcut Black): 45 * SKU_WJ003 (High-Rise White): 30 * SKU_WJ004 (Distressed Gray): 20 * SKU_WJ005 (Mom Jeans Dark Wash): 15 * SKU_WJ006 (Flare Jeans Light Wash): 8 * **Configuration (args.properties):** * `od.max_skus_per_category`: 50 (Maximum allowed SKUs in this category system-wide) * `store_001.max_jeans_capacity`: 4 (Maximum physical SKUs Store_001 can display for Jeans) * `od.min_ros_threshold`: 10 (SKUs with predicted ROS below this are considered for deprecation)

Core Logic Steps (from "Optimum Depth & Width Calculation"):

1. **Rank SKUs by Predicted ROS:** The algorithm first sorts all active Women's Jeans SKUs by their `predicted_ros` in descending order.
 - SKU_WJ001: 50
 - SKU_WJ002: 45
 - SKU_WJ003: 30
 - SKU_WJ004: 20
 - SKU_WJ005: 15
 - SKU_WJ006: 8
 - ... (and so on for other SKUs)
2. **Apply `od.min_ros_threshold`:** Identify SKUs below the minimum ROS threshold. SKU_WJ006 (ROS 8) is below the `od.min_ros_threshold` of 10. These are candidates for discontinuation or reduced depth.
3. **Consider Physical Capacity:** The algorithm knows Store_001 can only effectively display/stock 4 unique jeans SKUs (`store_001.max_jeans_capacity`).

4. Select Top SKUs within Constraints:

- It starts from the highest ROS: SKU_WJ001, SKU_WJ002, SKU_WJ003, SKU_WJ004. This fills the capacity of 4.
- SKU_WJ005 (ROS 15) and SKU_WJ006 (ROS 8) would not be recommended for this store due to capacity constraints and the minimum ROS threshold for SKU_WJ006.

Output (od_recommendations.csv entry for Store_001):

category_id	display_id	recommended_od_skus	justification	forecasted_impact_sales
Women's Jeans	Store_001	["SKU_WJ001", "SKU_WJ002", "SKU_WJ003", "SKU_WJ004"]	Selected top 4 SKUs by predicted ROS (50, 45, 30, 20 units/week respectively) to fit store capacity of 4. SKU_WJ005/006 not included due to capacity/low ROS (8). This optimizes space for high-performing items.	145 (sum of 50+45+30+20)

This recommendation provides Store_001 with a focused assortment of its highest-selling Women's Jeans SKUs, adhering to its display capacity, and maximizing sales efficiency.

8. Debug Guide Highlights

• No Recommendations Generated:

- **Check:** Verify `ros.min_history_weeks` or `iss.min_sales_threshold` in `args.properties`. If set too high, no SKUs might meet the criteria.
- **Remedy:** Lower thresholds or check if input data has sufficient history/volume.
- **Check:** Review debug logs for data quality issues or missing critical fields. If `sales_units` or `current_inventory` are largely null, the algorithm can't process.
- **Remedy:** Ensure data ingestion pipeline is healthy and delivering complete data.

• Unexpectedly Low/High OD/OW/ISS Recommendations:

- **Check:** Examine `predicted_ros` values in `common-data.json`. Are they plausible?
- **Remedy:** If forecasts are off, investigate the "Sales Velocity & Forecasting" module and its underlying model or input data.
- **Check:** Review `od.max_skus_per_category`, `ow.max_categories_per_display`, and

- `inventory.safety_stock_factor` in `args.properties`. Aggressive settings can lead to skewed recommendations.
- **Remedy:** Adjust these parameters to align with business expectations and run simulations.
- **Performance Issues (Slow Run Times):**
 - **Check:** Log files for long-running steps in “Data Ingestion & Preprocessing” or “Sales Velocity & Forecasting”.
 - **Remedy:** Optimize data queries, improve data parsing efficiency, or consider increasing computational resources.
- **Errors in Output Files:**
 - **Check:** Validate the schema and content of `od_recommendations.csv`, `ow_recommendations.csv`, `iss_recommendations.csv`. Are fields correctly populated? Are formats correct?
 - **Remedy:** Review the “Recommendation Engine” and “Output Generation” modules for correct data mapping and serialization.
- **Mismatch Between Input AG Mapping and Output:**
 - **Check:** Verify the `attribute_group.mapping_file` (from `args.properties`) against the input data. Incorrect mapping can lead to wrong AG-level calculations.
 - **Remedy:** Correct the mapping file or update the preprocessing logic to handle inconsistencies.

9. Common Pitfalls & Remedies

- **Pitfall: Stale Input Data:** The algorithm runs on outdated sales or inventory data, leading to irrelevant recommendations.
 - **Remedy:** Implement robust data refresh schedules. Monitor data freshness and trigger alerts if data sources are not updated as expected.
- **Pitfall: Seasonality Not Accounted For:** Forecasts or ROS calculations don’t consider peak/off-peak seasons, leading to overstocking/understocking.
 - **Remedy:** Ensure the `seasonality.adjustment_factor` is dynamically applied and regularly reviewed. Incorporate historical seasonal patterns into the forecasting model.
- **Pitfall: One-Size-Fits-All Parameters:** Using the same `od.max_skus_per_category` or `inventory.safety_stock_factor` across all categories/stores, regardless of their unique characteristics.
 - **Remedy:** Introduce category-specific or store-specific parameters where appropriate. The system should allow for parameter overrides or segmentation.
- **Pitfall: Ignoring Qualitative Insights:** Over-reliance on quantitative outputs without considering expert merchant insights or upcoming market trends (e.g., new product launches, competitive landscape).
 - **Remedy:** Integrate a feedback loop for merchandising teams to review and adjust recommendations. Provide mechanisms for manual overrides with clear audit trails.
- **Pitfall: Data Granularity Mismatch:** Trying to apply ISS recommendations calculated at a high level (e.g., category) to a very granular level (e.g., specific store display).
 - **Remedy:** Ensure the level of aggregation for calculations matches the level of recommendation. If granular recommendations are needed, ensure input data supports that granularity.

10. FAQ

1. How often is the depletion algorithm run?

- Typically, the core recommendations (OD, OW, ISS) are regenerated weekly or bi-weekly. Forecasts are updated daily. Performance reports are generated monthly.

2. Can I manually override the recommendations?

- Yes, the system is designed to provide recommendations, not mandates. Merchandising teams can review and override any suggested OD, OW, or ISS values. Overrides are logged for auditing and future model refinement.

3. What happens if a new product is launched with no sales history?

- New products initially rely on pre-defined default parameters, Attribute Group (AG) averages, or expert input for their initial OD/OW/ISS. As sales data accumulates (`ros.min_history_weeks`), the algorithm will start generating data-driven recommendations.

4. How does the algorithm handle promotions or markdown events?

- The “Sales Velocity & Forecasting” module is trained to recognize the impact of past promotional events. Future promotions, if known, can be fed into the system to adjust `predicted_ros` accordingly.

5. Is the algorithm suitable for all product categories?

- While designed to be robust, performance can vary. High-volume, predictable categories generally see the most benefit. Very low-volume, highly erratic, or bespoke categories might require more manual oversight or specific rule sets.

11. Quick Reference Cheat-Sheet

Abbreviation	Business Definition	Key Metric/Concept	Related Output/Module
OD	Optimum Depth: The ideal number of unique items (SKUs) to stock in a category or display.	<code>recommended_od_skus</code>	<code>od_recommendations.csv</code> , OD/OW Calc.
OW	Optimum Width: The ideal number of different categories or styles to offer.	<code>recommended_ow_categories</code>	<code>ow_recommendations.csv</code> , OD/OW Calc.
ISS	Ideal Size Set: The best combination of sizes to maximize sales and	<code>recommended_sizes_json</code> (e.g., size curve)	<code>iss_recommendations.csv</code> , ISS Determ.

Abbreviation	Business Definition	Key Metric/Concept	Related Output/Module
ROS	minimize stockouts. Rate of Sale: The speed at which products are sold over a period (e.g., units/week).	average_ros, predicted_ros	Sales Velocity & Forecasting
AG	Attribute Group: A set of product attributes grouped for analysis (e.g., color, size, brand).	Used for aggregated analysis when SKU-level data is sparse or for higher-level planning.	Data Preprocessing, all calculation modules
KPIs	Key Performance Indicators	Inventory Turnover, Stockout Rate, Sales Lift	performance_summary.pdf
args.properties	Configuration file for algorithm parameters.	Key tunable parameters affecting recommendations.	All modules
common-data.json	Standardized intermediate data format.	Cleaned, enriched data used across modules.	All modules

Chapter 1: Depth Suggestion Workflow Orchestration

Last Updated: 2025-05-23

1. Business Objective

This step orchestrates the entire process of generating and delivering Optimum Depth (OD) suggestions, ensuring that all necessary data is gathered and results are distributed effectively.

Why This Matters: This helps retail businesses by providing accurate, timely recommendations on the ideal number of unique items (SKUs) to stock in a category or display. This prevents lost sales from stockouts, reduces wasted inventory, and improves the customer shopping experience by ensuring products are available when and where they are needed.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold (ROS):** We need to know how many units of each product (SKU) have been sold over specific periods to understand its Rate of Sale (ROS) and overall popularity.
- **Historical Sales Trends:** This helps identify patterns like seasonality, promotions, or long-term growth/decline, which are crucial for predicting future demand.

Product Data

- **Product Attributes (AG):** Details like color, size, material, or brand (Attribute Group) are needed to group similar items for analysis, ensuring we suggest appropriate depth for comparable products.
- **Product Classification (e.g., Category, Style):** Knowing the category (like “T-Shirts” or “Dresses”) or style helps apply specific business rules or target Optimum Depth (OD) levels relevant to that classification.

Operational Data

- **Current Stock Levels:** We need to know what's currently in stores or warehouses to understand present inventory health and calculate how many more units might be needed or if current stock is excessive.
- **Order Lead Times:** This information is critical to ensure that suggested orders consider the time it takes for new inventory to arrive, preventing stockouts during replenishment.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The historical sales period used to calculate a product's Rate of Sale (ROS).	When sales patterns shift dramatically, or for seasonal products where a shorter/longer look-back is more relevant.
Minimum Sales Threshold	10 units	Products must have sold at least this many units within the Analysis Period to be considered for detailed OD suggestions.	To exclude very slow-moving or experimental items from the main calculation, or to include more items for analysis.
Target Service Level	95%	The desired percentage of customer demand that should be met by available stock.	To balance between aggressive stock minimization (lower %) and ensuring high customer satisfaction (higher %).
Safety Stock Multiplier	0.2	A factor applied to the base OD to account for unexpected demand fluctuations or supply chain delays.	During periods of high uncertainty or if supply chain reliability decreases.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Input Data Snapshots] --> B[Depth Suggestion Calculation]
B --> C[Optimum Depth Suggestions]

1. Start with → Raw Sales, Product, and Current Stock Data.
2. Then → Capture a precise snapshot of all necessary input data.
3. Next → Execute the core Depth Suggestion Calculation to determine ideal stock levels.
4. Finally → Distribute the calculated Optimum Depth (OD) suggestions to relevant inventory and planning systems.

5. Step-by-Step Actions

1. **Capture** critical data snapshots
 - What we do: *We take a precise, moment-in-time record of all sales, product, and current stock information.*
 - Why we do it: *This ensures the Depth Suggestion Calculation uses consistent, accurate data and avoids any changes to the data while processing is underway.*
2. **Execute** the Depth Suggestion Calculation
 - What we do: *We run the core logic to determine the Optimum Depth (OD) for each unique item (SKU) based on its historical sales performance, business rules, and settings like Target Service Level.*
 - Why we do it: *This is where the actual actionable recommendations are generated, telling us how many unique items of each product should ideally be in stock.*
3. **Synchronize** computed outputs
 - What we do: *We send the final Optimum Depth (OD) suggestions and related insights to the systems that manage inventory, purchasing, and order placement.*
 - Why we do it: *This makes sure the recommendations are immediately available where they are needed to guide purchasing, store transfers, and overall merchandising decisions, making them actionable.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular women's summer dress (Floral Print, SKU 54321) has sold 320 units in the last 90 days across all stores.
- The current inventory for this dress is 50 units.
- The Target Service Level for this dress category is set at 90%, and the Analysis Period is 90 days.

What Happens

1. First, the system captures a complete snapshot of all sales data for SKU 54321, its product attributes, and its current stock level.

2. Then, the Depth Suggestion Calculation module processes this data. It calculates the Rate of Sale (ROS) for the dress (320 units / 90 days). It also analyzes the consistency of its sales and compares it to similar popular dresses. It considers the target service level to ensure enough stock is available to meet most customer demand.
3. Finally, based on these calculations, the system determines that an Optimum Depth (OD) of 90 units is ideal for this dress. This means the system will suggest ordering or transferring 40 more units (90 ideal - 50 current stock) to reach the optimum level.

Final Outcome

An Optimum Depth (OD) suggestion of 90 units for the Floral Print Dress (SKU 54321) is generated and made available to the inventory management and ordering systems.

Key Takeaway: By capturing and processing sales data efficiently, the system ensures stores stock the right quantity of popular items, preventing stockouts and improving customer satisfaction, ultimately leading to better sales.

7. Results

This step produces:

Main Outputs

- **Optimum Depth (OD) Suggestions:** The ideal number of unique items (SKUs) to stock for each product or product Attribute Group (AG). This tells us how many units of a specific product version (e.g., “red medium T-shirt”) are ideal.
- **Replenishment Order Quantities:** The precise number of units to order or transfer for each product to reach its recommended Optimum Depth (OD).
- **Projected Stock Levels:** An estimate of future inventory if the OD suggestions are followed, helping visualize store stock health.

Business Reports

- **Depth Suggestion Summary Report:** Used by category managers and inventory planners to review, approve, and understand the logic behind the recommended stocking levels.
- **Inventory Optimization Dashboard:** Used by executives and supply chain teams to monitor the overall health of inventory, identify overstock or understock trends, and track the impact of OD suggestions on sales and profitability.

8. Unusual Cases

Situation	What Happens	What You Should Do
	The system might process with outdated	Investigate the source of the missing data (e.g., sales system,

Situation	What Happens	What You Should Do
Missing or delayed data feed	data or halt, leading to inaccurate or no suggestions.	inventory system) and ensure data is loaded, then re-run the workflow.
Sudden, unexpected sales spike (e.g., viral trend)	The calculated Rate of Sale (ROS) and Optimum Depth (OD) might be too low, leading to potential stockouts.	Manually review products with sudden, high sales; consider overriding the OD suggestion with a higher quantity based on forward-looking judgment.
New product with no sales history	The system typically defaults to a baseline Optimum Depth (OD) or uses data from very similar products.	Store teams or planners should use their judgment to set initial ODs based on anticipated demand until sales history builds.
All suggested ODs are zero or very low	This could indicate a major drop in overall sales, or incorrect filters/settings, making the system recommend minimal stock for almost everything.	Review the Analysis Period, Minimum Sales Threshold, and overall sales trends to ensure settings are appropriate for current business conditions.

9. Next Steps

Before This Step

- **Product Data Loading:** Accurate and complete product descriptions and classifications (Attribute Group, category, style) must be loaded into the system. This provides the foundation for grouping and analyzing items.
- **Sales Data Integration:** All historical transaction data, including Units Sold (ROS) and returns, must be integrated and cleansed to ensure reliable inputs for demand analysis.

After This Step

- **Inventory Planning Systems:** These systems consume the generated Optimum Depth (OD) suggestions to automatically create purchase orders, internal transfers, or replenishment tasks for stores and warehouses.
- **Merchandising Review:** Category managers and retail planners review the OD suggestions alongside other strategic goals (e.g., promotions, new collections) to finalize assortment plans and make strategic inventory investments.

10. Common Questions

Q: How quickly do Optimum Depth (OD) suggestions reflect recent sales trends? A: The workflow typically processes data from the most recently available sales snapshot, often updated daily or weekly. This means suggestions are usually very current, reflecting recent sales performance.

Q: Can I manually override an Optimum Depth (OD) suggestion? A: Yes, while the system provides data-driven suggestions, final decisions on inventory levels often allow for manual overrides by experienced planners. This is crucial for special promotions, unique market insights, or new product launches.

Q: What happens if a product has no sales history yet? A: For brand new products, the system typically relies on default Optimum Depth (OD) values or uses sales data from comparable products. As sales history accumulates, the suggestions become more precise.

Q: How does this process help prevent overstocking of slow-moving items? A: By calculating the Optimum Depth (OD) based on actual Rate of Sale (ROS) and applying rules like the Minimum Sales Threshold, the system recommends *just enough* stock. This significantly reduces the risk of holding excessive, unsold inventory for items that don't sell quickly.

Q: Does this account for seasonal changes in demand, like holiday rushes? A: Yes, the Depth Suggestion Calculation module analyzes historical sales data across the defined Analysis Period, which typically captures seasonal patterns. It can be configured to factor in these trends for more accurate seasonal Optimum Depth (OD) recommendations.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Orchestrates the entire process of generating and distributing Optimum Depth (OD) suggestions to optimize inventory.
- **Main Input:** A precise snapshot of historical sales data, detailed product information, and current stock levels.
- **Main Output:** Recommended Optimum Depth (OD) for each unique item (SKU), guiding how much to stock.
- **Warning Signs:** Outdated or inconsistent inventory suggestions, frequent stockouts of popular items, or excessive overstock indicating inefficiencies.
- **Key Settings:** Analysis Period, Minimum Sales Threshold, and Target Service Level.

Need Help? Contact the Inventory Planning Team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 2: Depth Suggestion Calculation

Last Updated: 2025-05-23

1. Business Objective

This step precisely calculates the Optimum Depth (OD) for products (styles) at a store level, determining the ideal number of unique items (SKUs) to stock.

Why This Matters: This helps retail businesses optimize their inventory investments by recommending exactly how many units of a specific product style each store should carry. This prevents lost sales from stockouts, reduces wasted capital on excess inventory, and ensures customers consistently find the products they want, improving their shopping experience.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold (ROS):** We need to know how many units of each product (SKU) have been sold over specific periods to understand its Rate of Sale (ROS) and overall popularity.
- **Historical Sales Trends:** This helps identify patterns like seasonality, promotions, or long-term growth/decline, which are crucial for predicting future demand.

Product Data

- **Product Attributes (AG):** Details like color, size, material, or brand (Attribute Group) are needed to group similar items for analysis, ensuring we suggest appropriate depth for comparable products.
- **Product Classification (e.g., Category, Style):** Knowing the category (like “T-Shirts” or “Dresses”) or style helps apply specific business rules or target Optimum Depth (OD) levels relevant to that classification.

Operational Data

- **Target Cover Days:** This specifies the desired number of days of sales the suggested Optimum Depth (OD) should cover for a given product style and channel.
- **Pivotal Sales Activity (PSA) Benchmarks:** These are criteria used to identify key selling periods or product activity levels, ensuring Rate of Sale (ROS) calculations reflect true sales performance.
- **Minimum Live Days for ROS Calculation:** This sets the minimum number of days a product style must have active sales to be included in its Rate of Sale (ROS) calculation, preventing inaccurate projections for inactive items.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The historical sales period used to calculate a product's Rate of Sale (ROS).	When sales patterns shift dramatically, or for seasonal products where a shorter/longer look-back is more relevant.
Minimum Live Days for ROS	10 days	The minimum number of days a style must have recorded sales activity within the analysis period to be considered for a detailed Rate of Sale (ROS) calculation.	To ensure that only products with sufficient recent sales history contribute to accurate depth suggestions, avoiding inactive items.
Pivotal Sales Activity (PSA) Threshold	0.0	A benchmark percentage (e.g., 0.6 for 60%) that a product's sales activity must meet on a given day to be included in its Rate of Sale (ROS) calculation.	To focus Rate of Sale (ROS) calculations on periods of genuine sales performance, excluding days with minimal or no activity.
Target Cover Days	Varies by style/channel	The number of days of sales that the Optimum Depth (OD) should ideally cover for	To align inventory levels with demand forecasts and strategic goals, adjusting for product lifecycle or

Setting Name	Default Value	Description	When to Change
		a given style in a specific channel.	channel-specific needs.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Sales, Product, & Target Data] --> B[Depth Suggestion Calculation]

B --> C[Optimum Depth Suggestions]

1. Start with → Raw Sales Data (Units Sold, dates), Product Attributes, and Target Cover Days settings.
2. Then → Create a map of target cover days for each style and channel, and gather sales quantities.
3. Next → Compute the Rate of Sale (ROS) for each style and store, considering minimum live days and Pivotal Sales Activity (PSA) benchmarks.
4. Finally → Calculate the Optimum Depth (OD) by multiplying the Rate of Sale (ROS) by the target cover days and generate final suggestions.

5. Step-by-Step Actions

1. **Create Depth Target Map**
 - What we do: *We organize the predefined target cover days, ensuring that each style and channel combination has a specific inventory goal.*
 - Why we do it: *This prepares the system to apply specific inventory targets, aligning the suggested depth with business strategy.*
2. **Gather Sales Quantity Data**
 - What we do: *We collect and summarize all Units Sold for each product style, store, and day within the defined Analysis Period.*
 - Why we do it: *This provides the raw sales performance information essential for calculating how quickly products are selling.*
3. **Compute Style-Store Rate of Sale (ROS)**
 - What we do: *We calculate the average daily sales for each style within each store, focusing on periods that meet the Minimum Live Days and Pivotal Sales Activity (PSA) Thresholds.*
 - Why we do it: *This establishes a reliable measure of how popular and fast-moving each style is in its specific store environment.*
4. **Determine Optimum Depth (OD)**
 - What we do: *We multiply the calculated Rate of Sale (ROS) by the Target Cover Days for each style and store, rounding to a whole number to get the ideal stock quantity.*
 - Why we do it: *This is the core calculation that translates sales speed into a specific, actionable inventory recommendation, ensuring enough stock to meet future demand.*

5. **Generate** Exportable Suggestions

- What we do: *We compile all the calculated Optimum Depth (OD) suggestions, along with supporting data like Rate of Sale (ROS) and target cover days, into a clear output format.*
- Why we do it: *This makes the recommendations ready for integration with other inventory systems, allowing them to guide purchasing and replenishment.*

6. **Sample Scenario**

Let's walk through a real example:

Starting Point

- A popular women's summer dress (Floral Print, Style 54321) in 'Main Street Boutique' (Channel: Retail) has sold 320 units over the last 90 days.
- The business has set a Target Cover Days of 20 days for this style in the 'Retail' channel.
- The Minimum Live Days for ROS is 15 days, and the Pivotal Sales Activity (PSA) Threshold is 0.6 (meaning sales on a given day must contribute at least 60% of potential daily activity to be counted).

What Happens

1. First, the system identifies all sales for Style 54321 in 'Main Street Boutique' over the past 90 days. It then filters this data, ensuring that sales are only counted on days where the style met the Minimum Live Days (e.g., sold on at least 15 days within the 90-day period) and the Pivotal Sales Activity (PSA) Threshold (e.g., daily sales were sufficiently impactful).
2. Then, it calculates the average daily Rate of Sale (ROS) for the dress using this filtered sales data (e.g., if it sold 320 units over 90 days, its daily ROS might be 3.56 units/day).
3. Finally, it multiplies this daily ROS by the Target Cover Days set for this style and channel ($3.56 \text{ units/day} * 20 \text{ days}$).

Final Outcome

An Optimum Depth (OD) suggestion of 71 units ($3.56 * 20$, rounded up) for the Floral Print Dress (Style 54321) is generated for 'Main Street Boutique', telling the store exactly how many units of this dress it should aim to stock.

Key Takeaway: By combining precise historical sales data with predefined inventory targets, this calculation ensures stores stock precisely the right amount of popular items to meet demand without holding excess, preventing both stockouts and overstock.

7. **Results**

This step produces:

Main Outputs

- **Optimum Depth (OD) Suggestions:** The ideal number of units to stock for each product style at a specific store, guiding inventory levels.
- **Calculated Rate of Sale (ROS):** The computed daily sales velocity for each style-store combination, indicating how quickly items are selling.
- **Live Days and Sales Quantity Used:** Details on the number of active selling days and total units sold that contributed to the Rate of Sale (ROS) calculation, providing transparency.

Business Reports

- **Depth Suggestion Summary Report:** Used by inventory planners and category managers to review the recommended stock levels, understand the underlying calculations, and approve final inventory plans.
- **Style Performance Dashboard:** Used by merchandising teams to monitor the sales performance of key styles and channels, validating the Rate of Sale (ROS) inputs and the impact of suggested depths.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system might process with outdated sales or product data, leading to inaccurate or no suggestions.	Investigate the source of the missing data (e.g., sales system, product catalog) and ensure data is loaded, then re-run the calculation.
New style with no sales history	The system cannot calculate a Rate of Sale (ROS) and may exclude the style or apply a default, lower Optimum Depth (OD).	Store teams or planners should use their judgment to set initial Optimum Depths (ODs) based on anticipated demand or similar product performance until sales history builds.
Style does not meet Minimum Live Days or PSA Threshold	The Rate of Sale (ROS) for that style may be considered unreliable, leading to a zero or very low Optimum Depth (OD) suggestion, or the style may be excluded.	Review the style's activity; if it's genuinely slow-moving, the suggestion is correct. If it's a data issue or a new product, consider manual override or adjusting settings.
Target Cover Days not defined for a style/channel	The calculation cannot proceed for that specific style-channel combination, leading to missing Optimum Depth (OD) suggestions.	Ensure all relevant styles and channels have their Target Cover Days explicitly set in the system configurations.

9. Next Steps

Before This Step

- **Product Master Data Loading:** Accurate and complete product descriptions and classifications (Attribute Group, category, style) must be loaded. This provides the foundation for grouping and analyzing items.
- **Sales Data Integration:** All historical transaction data, including Units Sold (ROS) and returns, must be integrated and cleansed to ensure reliable inputs for demand analysis.
- **Target Cover Days Configuration:** Business rules for target inventory cover days must be defined and available for specific styles and channels.

After This Step

- **Inventory Planning Systems:** These systems consume the generated Optimum Depth (OD) suggestions to automatically create purchase orders, internal transfers, or replenishment tasks for stores and warehouses.
- **Merchandising Review:** Category managers and retail planners review the Optimum Depth (OD) suggestions alongside other strategic goals (e.g., promotions, new collections) to finalize assortment plans and make strategic inventory investments.

10. Common Questions

Q: How quickly do Optimum Depth (OD) suggestions reflect recent sales trends? A: The calculation typically processes data from the most recently available sales snapshot, often updated daily or weekly. This means suggestions are usually very current, reflecting recent sales performance.

Q: What happens if a product style has very inconsistent sales, like selling a lot one week and nothing the next? A: The inclusion of Pivotal Sales Activity (PSA) Threshold and Minimum Live Days for ROS helps to filter out highly erratic or inactive periods, ensuring that the calculated Rate of Sale (ROS) is based on a more consistent and reliable sales history.

Q: How does this module help prevent overstocking of slow-moving items? A: By calculating the Optimum Depth (OD) based on actual Rate of Sale (ROS) and applying rules like Minimum Live Days, the system recommends *just enough* stock. If an item sells slowly, its ROS will be low, leading to a much lower suggested OD, significantly reducing excess inventory.

Q: Can the Target Cover Days be different for different styles or stores? A: Yes, absolutely. The system is designed to allow Target Cover Days to be set specifically for individual styles, categories, or even channels, enabling a highly granular and strategic approach to inventory management.

Q: How does this process contribute to improving customer satisfaction? A: By ensuring that stores stock the ideal number of unique items (Optimum Depth) based on their sales velocity, it significantly reduces the likelihood of popular products being out of stock. This means customers are more likely to find what they want, leading to a better shopping experience and fewer missed sales.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates the ideal number of units (Optimum Depth) to stock for each product style at a store level.
- **Main Input:** Historical sales data (Units Sold), product details (Style, Attribute Group), and operational settings like Target Cover Days and Pivotal Sales Activity benchmarks.
- **Main Output:** Recommended Optimum Depth (OD) for each unique item (SKU) within a style, guiding how much to stock.
- **Warning Signs:** Frequent stockouts of seemingly popular items, or excessive inventory for slow-selling products, indicating the calculation might be off.
- **Key Settings:** Analysis Period, Minimum Live Days for ROS, Pivotal Sales Activity (PSA) Threshold, and Target Cover Days.

Need Help? Contact the Inventory Planning Team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

depthSuggestion FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document covers common questions about the depthSuggestion algorithm, which calculates the Optimum Depth (OD) for products at a store level to optimize inventory.

General

Q: What is the main goal of the depthSuggestion algorithm? **A:** The primary goal is to provide data-driven recommendations for the Optimum Depth (OD) – the ideal number of unique items (SKUs) to stock for a given product style at each store. This helps optimize inventory levels, reduce stockouts, and minimize overstock.

Q: What is OD (Optimum Depth) in this context? **A:** OD refers to the ideal number of unique items (SKUs) within a specific product style (e.g., a particular dress style) that a store should stock. It's about how many distinct options of that style, like different colors or materials, are optimal.

Q: How does this algorithm help our business? **A:** By providing accurate OD suggestions, the algorithm helps merchandisers and inventory planners make smarter stocking decisions. This leads to improved product availability, better inventory turns, reduced markdowns, and ultimately, increased sales and profitability.

Q: What level of detail do the suggestions provide? **A:** The algorithm generates OD suggestions at the **store-style level**. This means it tells you the ideal number of SKUs for a specific product style at a particular store, not individual SKUs or broader categories.

Q: Who typically uses these OD suggestions? **A:** Merchandisers, inventory planners, store managers, and supply chain analysts are the primary users of these suggestions, leveraging them for replenishment orders, initial allocations, and assortment planning.

Configuration & Settings

Q: Are there settings we can adjust to influence the OD calculation? **A:** Yes, key parameters like target cover days, minimum live days for ROS (Rate of Sale) calculation, and Pivotal Sales Activity (PSA) benchmarks can be configured to align with business strategies and inventory policies.

Q: How are “target cover days” factored into the OD calculation? **A:** Target cover days define how many days of future sales the algorithm should aim to cover with the suggested OD. For example, if target cover is 30 days, the OD will aim to ensure 30 days of stock based on the calculated ROS.

Q: What are “Pivotal Sales Activity (PSA) benchmarks”? **A:** PSA benchmarks are performance thresholds or rules that influence the OD calculation, often ensuring that products meeting certain sales activity criteria are prioritized or managed differently, helping to maintain stock levels for key performers.

Q: Can we set a minimum sales period for Rate of Sale (ROS) calculation? **A:** Yes, the minimum live days configuration ensures that ROS is only calculated for products that have been actively available for sale for a sufficient period, preventing skewed data from newly launched or intermittently available items.

Q: Where can I find or modify these configuration settings? **A:** These settings are typically managed by system administrators. Please consult your internal documentation or contact the support team for details on how to view or request changes to these parameters.

Data & Inputs

Q: What kind of data does the algorithm need to calculate OD? **A:** The algorithm primarily relies on historical sales data (units sold), which is used to calculate the Rate of Sale (ROS). It also considers product attributes, store information, and potentially current inventory levels.

Q: How far back does the algorithm look at sales data? **A:** The look-back period for historical sales data is configurable, but it typically considers recent periods (e.g., the last 52 weeks) to determine an accurate ROS, while also respecting the minimum live days setting.

Q: Is current inventory data used in the calculation? **A:** While the core calculation focuses on historical sales to derive the ideal depth, the generated OD suggestions are typically used in conjunction with current inventory data for actual replenishment or allocation decisions to ensure optimal stock levels.

Q: Are specific product attributes required? **A:** Yes, product information, including style IDs, unique item identifiers (SKUs), and relevant Attribute Groups (AGs) like color or material, are essential for grouping data and ensuring accurate OD suggestions at the style level.

Q: How is the Rate of Sale (ROS) determined? **A:** ROS is determined by analyzing the number of units sold over a specific period (e.g., units per week) for a given style at a particular store. The calculation takes into account factors like the product’s live days to ensure accuracy.

Process & Logic

Q: How does the Depth Suggestion Workflow Orchestration module work? **A:** This module manages the entire process from start to finish. It first ensures all necessary data snapshots (e.g., sales, inventory) are captured, then triggers the core Depth Suggestion Calculation module, and finally handles uploading or synchronizing the computed OD suggestions to target systems.

Q: What is the order of operations for generating OD suggestions? **A:** The workflow follows a clear sequence: 1. Capture required data snapshots. 2. Execute the Depth Suggestion Calculation module to compute OD. 3. Upload or synchronize the calculated ExportDepthSuggestionRow outputs to relevant business systems.

Q: How does the algorithm handle new products with limited sales history? **A:** For new products without sufficient historical ROS data (e.g., not meeting minimum live days), the algorithm typically employs predefined default ODs, uses proxy data from similar products, or flags them for initial manual merchandising review until enough sales data accumulates.

Q: Does the algorithm consider seasonal trends or promotions? **A:** The algorithm's ROS calculation can account for historical seasonality by analyzing sales over longer periods or using weighted averages. For promotions, it may either normalize sales data to remove promotional spikes or factor in planned promotional lifts if configured.

Q: Is the process fully automated? **A:** Yes, the Depth Suggestion Workflow Orchestration module is designed to run automatically on a scheduled basis (e.g., daily or weekly) to provide timely and updated OD suggestions without manual intervention for the core calculation.

Outputs & Results

Q: What is the main output of the algorithm? **A:** The primary output is a dataset containing the suggested Optimum Depth (OD) for each product style at every relevant store, formatted as ExportDepthSuggestionRow. This includes the calculated ideal number of unique items (SKUs) to stock.

Q: Where can I access the calculated OD suggestions? **A:** The computed OD suggestions are typically uploaded or synchronized to central data warehouses, business intelligence dashboards, or directly integrated into inventory management and planning systems for easy access and utilization.

Q: Are the suggestions provided at a SKU level or style level? **A:** The suggestions are provided at the **store-style level**, indicating the ideal number of *unique items (SKUs)* for a specific style at a particular store, not individual SKU-specific depths.

Q: How often are the OD suggestions updated? **A:** The OD suggestions are updated based on the schedule of the Depth Suggestion Workflow Orchestration module, typically running daily or weekly to ensure they reflect the most recent sales trends and inventory needs.

Q: Can I see the underlying data that led to a specific OD suggestion? **A:** While the core output is the suggested OD, analytical tools and dashboards often allow users to drill down into the historical Rate of Sale (ROS) and other contributing factors that informed a specific suggestion, providing transparency into the calculation.

Error Handling & Edge Cases

Q: What happens if there's missing historical sales data for a product or store? **A:** If critical sales data is missing, the algorithm may either apply a default OD, flag the product/store for manual review, or use a statistically estimated ROS based on similar products or locations.

Q: How are products with extremely low or zero sales (Rate of Sale) handled? **A:** Products with very low or zero ROS might be assigned a minimum OD (e.g., 1 to ensure presence) or a recommended OD of 0 if they are non-moving, depending on configuration and business rules for dead stock.

Q: What if a store has very unique selling patterns or is an outlier? **A:** Since the calculation is performed at the store level, the algorithm can adapt to unique store-specific ROS patterns. However, extreme outliers might still be flagged for manual review to ensure the suggestion is practical and accurate.

Q: How does the system alert us to potential issues or data anomalies during processing? **A:** The workflow orchestration module is designed to log errors and failures. Critical issues, such as missing input data or calculation failures, typically trigger automated alerts to the relevant support teams.

Q: Are there scenarios where an OD might not be provided for a specific style/store? **A:** Yes, an OD might not be provided if there's insufficient data to make a reliable calculation (e.g., product has not met minimum live days), if the product is flagged for discontinuation, or if specific business rules override automated suggestions.

Need More Help?

Contact the **[Your Company's Support Team/Email/Jira Channel]** for further assistance.

depthSuggestion Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

The depthSuggestion algorithm is designed to optimize inventory levels by calculating the **Optimum Depth (OD)** for each product (style) at a store level. By leveraging historical sales data and operational targets, it provides data-driven recommendations on the ideal number of unique items (SKUs) to stock, thereby enhancing sales potential and minimizing costly overstock or understock situations. This leads to improved inventory efficiency and better customer satisfaction.

3. Architecture Overview

The depthSuggestion pipeline operates with two primary modules orchestrating the calculation of **Optimum Depth (OD)**.

graph TD

```
A[Depth Suggestion Workflow Orchestration] -->|orchestrates| B[Depth Suggestion Calculation];  
B -->|generates_output| A;
```

- **Depth Suggestion Workflow Orchestration:** This module manages the overall flow of the OD suggestion process, from data preparation to output delivery.
- **Depth Suggestion Calculation:** This module performs the core computation of the ideal number of unique items to stock for each product.

4. Prerequisites & Setup

To run the depthSuggestion pipeline, the following prerequisites are typically required:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Build Tool:** Maven (recommended) or Gradle.
- **Database Access:** Connectivity to the source database where historical sales and product data reside.
- **Dependencies:** Project-specific dependencies will be managed by Maven/Gradle (e.g., Spring Boot, database drivers, utility libraries).

To Run: 1. **Clone the Repository:** `bash git clone <repository-url>/depthSuggestion.git cd depthSuggestion` 2. **Build the Project:** `bash mvn clean install` 3. **Execute the Orchestrator:** The main orchestration module, Depth Suggestion Workflow Orchestration, is the entry

point. You would typically run its main class or a configured Spring Boot application: `bash java -jar target/depthSuggestion-1.0.jar` Ensure `args.properties` is correctly configured in the classpath if specific runtime parameters are needed.

5. Folder & File Structure

```
depthSuggestion/
├─ module-group/           # Contains the orchestrator module
                             that manages the overall pipeline flow.
├─ submodules/             # Houses the individual calculation
                             and utility modules.
├─ args.properties         # Global configuration file for the
                             entire pipeline.
├─ common-data.json        # Defines the shared data schemas
                             used across modules.
├─ outputs/                # Directory where the final results
                             and reports are stored.
```

6. Global Configuration

No global configuration parameters are currently defined in `args.properties`. If present, this section would list system-wide settings that influence the behavior of the entire `depthSuggestion` pipeline.

Setting Name	Default Value	Description	Valid Range/When to Change
<i>N/A</i>	<i>N/A</i>	<i>No parameters defined yet.</i>	<i>N/A</i>

7. Shared Data Model

The `common-data.json` schema, which describes the structure of shared data models used across the pipeline, is not currently defined. If present, this section would detail the common data structures.

Field	Type	Description
<i>N/A</i>	<i>N/A</i>	<i>No schema defined yet.</i>

8. Submodule Chapters

8.1. Depth Suggestion Calculation

1. Module Name & Business Objective

The **Depth Suggestion Calculation** module is responsible for calculating the **Optimum Depth (OD)** for products (styles) at a store level. Its business objective is to determine the ideal number of unique items (SKUs) to stock, ensuring a balance between meeting customer demand and optimizing inventory investment by analyzing historical **Rate of Sale (ROS)** against target inventory cover and sales benchmarks.

2. Required Information

- **Sales Data:**
 - Historical **Rate of Sale (ROS)** for each SKU at each store.
 - Minimum live days for **ROS** calculation *to ensure data quality*.
- **Product Data:**
 - Product master data, including style and SKU identifiers.
 - Pivotal Sales Activity (PSA) benchmarks *to provide context for sales performance*.
- **Operational Data:**
 - Target cover days *to define desired inventory levels*.
 - Store identifiers and their characteristics.

3. Business Settings

No specific business settings are defined for this module outside of the global configuration (if present). All configurable parameters, such as target cover days or minimum live days, are typically fed as inputs or managed by the orchestrator.

4. Process Flow

graph LR

```
A[Historical ROS Data] --> B{Calculate Suggested OD};
C[Target Cover Days] --> B;
D[Pivotal Sales Activity (PSA) Benchmarks] --> B;
B --> E[Output: Suggested OD (ExportDepthSuggestionRow)];
```

5. Step-by-Step Actions

1. **Retrieve** *historical sales data*, including **ROS** for each SKU.
2. **Filter** *data to ensure sufficient live days* for accurate **ROS** calculation.
3. **Ingest** *target cover days* to define the desired inventory holding period.
4. **Incorporate** *Pivotal Sales Activity (PSA) benchmarks* for comparative analysis.
5. **Compute** *the initial suggested OD* based on filtered **ROS** and target cover days.
6. **Adjust** *the suggested OD considering PSA benchmarks* to align with strategic sales targets.

7. **Generate** ExportDepthSuggestionRow *containing the final calculated OD* for each style and store.

6. Sample Scenario

Imagine a specific T-shirt style (Style A) in Store 123. * Historical **ROS** for Style A, SKU “Blue-Large”: 5 units/week. * Target Cover Days: 4 weeks. * Pivotal Sales Activity (PSA) benchmark for similar T-shirts: suggests a minimum OD of 3 SKUs to adequately represent the style. * Minimum live days for ROS calculation: 30 days. Style A has been active for 90 days, so its ROS is valid.

The module would: 1. Calculate an initial OD based on ROS * target cover days: (5 units/week * 4 weeks) = 20 units. This is the total number of items to sell, but OD is about unique SKUs. 2. The calculation then determines the number of *unique SKUs* (e.g., color, size combinations) needed to achieve this sales volume, typically by considering typical sales distribution across SKUs within a style. Let’s assume this translates to an initial suggestion of 4 unique SKUs to cover the sales volume for Style A. 3. Compare this initial 4 SKUs to the PSA benchmark of 3. Since 4 is greater than 3, the suggestion remains 4, ensuring adequate representation.

Key Takeaway: The module balances statistical sales data with strategic business benchmarks to arrive at a practical and effective **OD**.

7. Results

The module outputs ExportDepthSuggestionRow objects. Each row typically contains: * Store ID * Style ID * Calculated **Optimum Depth (OD)** * Supporting metrics used in the calculation (e.g., average **ROS**, target cover days applied).

8. Unusual Cases

- **Missing or delayed data feed:** If historical **ROS** data, target cover days, or PSA benchmarks are missing or arrive late, the module will either use default values, previous period’s data, or fail, depending on configuration. This can lead to inaccurate or outdated **OD** suggestions.
- **Insufficient ROS history:** For new styles or stores with very short selling periods (below minimum live days), **ROS** calculation might be unreliable or impossible. The module may apply fallback logic, such as using category-level averages or predefined default **ODs**.
- **Zero or unusually low ROS:** If a style has zero or extremely low **ROS**, the module needs rules to decide if it should still be stocked (e.g., if it’s a strategic item or required for product completeness) or if its **OD** should be 0.
- **Data outliers:** Spikes or drops in **ROS** due to promotions, stockouts, or data errors can skew **OD** calculations. Robustness against outliers through data cleansing or statistical methods is crucial.

9. Next Steps

The ExportDepthSuggestionRow output from this module is consumed by the Depth Suggestion Workflow Orchestration module. It is then typically persisted in a database and synchronized to downstream systems (e.g., inventory management, merchandising planning tools) for operationalization.

10. Common Questions

1. **How accurate are the OD suggestions if our sales patterns change frequently?**
 - The suggestions are based on historical **ROS**. For highly volatile products, it's crucial to frequently update input data and potentially review the minimum live days and target cover days parameters for responsiveness.
2. **Can we manually override the suggested OD?**
 - The system generates recommendations. While the algorithm doesn't *directly* support manual overrides within its calculation, the output can be fed into systems that allow merchandising teams to review and adjust the **OD** before final implementation.
3. **What if a new product has no sales history?**
 - For new products, the module typically falls back to category-level average **ROS**, or pre-defined initial OD values based on similar products or strategic importance, until sufficient sales history is accumulated.
4. **How does this account for promotions or seasonality?**
 - The historical **ROS** implicitly captures past promotional impacts and seasonality *if* the data period is long enough. For future promotions, specific forecasting models might be needed upstream or adjustments applied to the target cover days.
5. **Does this consider warehouse stock or only store-level?**
 - This module specifically calculates **OD** at the store level. While it influences total inventory, it doesn't directly manage central warehouse stock. This often requires separate inventory optimization modules.

11. Quick Reference

- **Calculates: Optimum Depth (OD)** (ideal number of unique items/SKUs).
 - **Inputs:** Historical **Rate of Sale (ROS)**, Target Cover Days, Pivotal Sales Activity (PSA) Benchmarks.
 - **Output:** ExportDepthSuggestionRow (Store ID, Style ID, Calculated **OD**).
 - **Key Metric:** **ROS** (Units/Week).
-

8.2. Depth Suggestion Workflow Orchestration

1. Module Name & Business Objective

The **Depth Suggestion Workflow Orchestration** module orchestrates the end-to-end workflow for generating **Optimum Depth (OD)** suggestions. Its business objective is to ensure the smooth, automated, and reliable execution of the **OD** calculation pipeline, from data preparation and snapshotting to the final delivery and synchronization of computed results, thereby providing timely and accurate **OD** recommendations for business use.

2. Required Information

- **Operational Data:**
 - Configuration for data snapshotting (utilOutputSnapshotModule) to ensure data consistency.

- Configuration for output synchronization (utilOutputSyncModule) to *deliver results to downstream systems*.
- Dependencies and execution order of submodules.

3. Business Settings

No specific business settings are defined for this module outside of the global configuration (if present). Its role is primarily flow control and data management rather than calculation.

4. Process Flow

graph TD

```
A[Start Orchestration] --> B(Capture Data Snapshots);
B --> C(Execute Depth Suggestion Calculation);
C --> D(Upload / Synchronize Results);
D --> E[End Orchestration];
```

5. Step-by-Step Actions

1. **Initiate** *the workflow*.
2. **Invoke** utilOutputSnapshotModule *to capture necessary data snapshots* before processing begins, ensuring data consistency for the calculation.
3. **Trigger** *the Depth Suggestion Calculation module* to perform the core **OD** computations.
4. **Receive** *the computed OD results* from the calculation module.
5. **Invoke** utilOutputSyncModule *to handle the persistence and synchronization of results* to target systems (e.g., database, data lake, reporting tools).
6. **Complete** *the workflow*.

6. Sample Scenario

A merchandising planner needs updated **OD** suggestions every Monday morning. * The Depth Suggestion Workflow Orchestration module is scheduled to run every Sunday night. * At 10:00 PM Sunday, the orchestrator **starts**. * It **calls** utilOutputSnapshotModule to create a consistent snapshot of last week's sales data, current inventory, and product master data. * Once the snapshot is complete, it **initiates** the Depth Suggestion Calculation module. * The calculation module runs for 30 minutes, producing its ExportDepthSuggestionRow output. * The orchestrator then **takes** these results and **calls** utilOutputSyncModule to upload them to the central data warehouse and push a notification to the merchandising system. * By 11:00 PM, the process **completes**, and new **OD** suggestions are ready for review by Monday morning.

Key Takeaway: The orchestration module acts as the “control tower,” ensuring all parts of the **OD** generation process run correctly and in sequence, delivering timely insights.

7. Results

The primary result of this module is the successful and timely delivery of the ExportDepthSuggestionRow output (generated by the Depth Suggestion Calculation module) to the intended downstream systems and databases. It ensures that the **Optimum Depth (OD)** suggestions are available for business consumption.

8. Unusual Cases

- **Missing or delayed data feed:** If upstream systems fail to provide data for snapshotting, the orchestrator might either proceed with stale data (if allowed), fail, or retry, impacting the freshness and accuracy of **OD** suggestions.
- **Failure of a submodule:** If Depth Suggestion Calculation or any utility module fails, the orchestrator needs robust error handling to log the failure, potentially retry, and notify relevant teams to prevent partial or incorrect output delivery.
- **System resource limitations:** During execution, if database connections are exhausted or memory limits are hit, the orchestrator should manage retries or graceful shutdowns to prevent system instability.
- **Synchronization issues:** If the utilOutputSyncModule cannot connect to target systems, the **OD** suggestions might be computed but not delivered, making them unusable for business operations.

9. Next Steps

Once the Depth Suggestion Workflow Orchestration module successfully completes, the calculated **Optimum Depth (OD)** data is available for use by merchandising teams, inventory planners, and other business stakeholders. They can then use these recommendations for: * Inventory replenishment orders. * Assortment planning. * Store layout adjustments. * Performance analysis against ideal depth.

10. Common Questions

1. **How often does this workflow run?**
 - The frequency is configurable based on business needs, often daily or weekly, to ensure **OD** suggestions remain relevant to current sales trends.
2. **What happens if a step fails?**
 - The orchestrator is designed with error handling to log failures, trigger alerts, and, depending on the severity, may retry the step or halt the process, requiring manual intervention.
3. **Can we re-run the process for a specific historical period?**
 - Typically, yes, if the underlying data snapshots are historical. The orchestrator can be configured to process specific date ranges or datasets for retrospective analysis or re-calculation.
4. **How do we know if the data synchronized correctly?**
 - The utilOutputSyncModule usually provides logging and status updates. Automated monitoring and reconciliation checks on the target systems are also put in place to verify data ingestion.

5. Is there a dashboard to monitor the workflow status?

- While not part of this specific module, in a production environment, this orchestrator would integrate with a larger monitoring system or dashboard to visualize its execution status, duration, and any errors.

11. Quick Reference

- **Manages:** End-to-end workflow for **Optimum Depth (OD)** generation.
- **Key Actions:** Data Snapshotting, Calculation Execution, Results Synchronization.
- **Ensures:** Timely and reliable delivery of **OD** suggestions.

9. Appendix

Full Parameter Reference

As noted in Section 6, no global configuration parameters are currently defined in `args.properties`.

Version History

- **1.0 - 2025-05-23:** Initial Release of `depthSuggestion` documentation.

Contact & Support Info

For questions, support, or to report issues, please contact: * **Email:** `support@yourcompany.com` * **Internal Support Portal:** [Link to your internal support portal] * **Team:** Data Science & Inventory Optimization

Tutorial: depthSuggestion

The Depth Suggestion Workflow Orchestration module (1) manages the entire lifecycle for generating Optimum Depth (OD) suggestions. It orchestrates the process by first capturing required data snapshots, then triggering the core Depth Suggestion Calculation module (0) to compute OD, and finally handling the persistence and synchronization of the calculated results.

flowchart TD

```
A0["Depth Suggestion Calculation"]
A1["Depth Suggestion Workflow Orchestration"]
A1 -- "orchestrates" --> A0
A0 -- "generates_output" --> A1
```

Chapters

Master Report: depthSuggestion

1. Title & Metadata

- **Project Name:** depthSuggestion
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** Automates the calculation of Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) for product categories to maximize sales and minimize stockouts.

2. Executive Summary

The depthSuggestion algorithm is designed to revolutionize merchandising by providing data-driven recommendations for optimal product assortment. It analyzes historical sales, inventory, and product attribute data to determine the ideal number of unique items (Optimum Depth), the optimal range of categories (Optimum Width), and the most effective size distributions (Ideal Size Set) for various product lines.

This system empowers merchandising teams to make strategic decisions that enhance product availability, reduce excess inventory, and ultimately improve profitability.

- **Key outcomes or KPIs:**
 - Precisely calculated **OD** for SKUs within categories.
 - Data-backed **OW** for category management.
 - Optimized **ISS** to prevent stockouts and overstocks.
 - Improved **ROS** across the product catalog.
 - Reduction in inventory holding costs and markdowns.
 - Enhanced customer satisfaction through better product availability.
- **Primary stakeholders or downstream consumers:** Merchandising Planners, Buyers, Inventory Managers, Supply Chain Analysts, Store Operations.

3. Architecture & Data Flow

Diagram:

graph TD

```
A[Raw Sales Data] --> B[Inventory & Product Attributes];  
B --> C[Data Preprocessing Module];  
C --> D[Cleaned Historical Data];  
D --> E[Demand Forecasting & Segmentation Module];
```

```
E --> F[Forecasted ROS & Attribute Groups (AG)];
F --> G(OD/OW Calculation Engine);
G --> H[Initial OD/OW Suggestions];
H --> I(ISS Optimization Module);
I --> J[Optimized OD/OW/ISS Recommendations];
J --> K[Reporting & Output Generation Module];
K --> L[Merchandising Systems & Dashboards];
```

Component Bullets:

- **Raw Sales Data:** Initial input of transaction-level sales information.
- **Inventory & Product Attributes:** Raw data on current stock levels and product descriptive features (e.g., color, size, brand).
- **Data Preprocessing Module:** Ingests, cleanses, validates, and aggregates raw data.
- **Cleaned Historical Data:** Standardized and aggregated sales, inventory, and product attribute data.
- **Demand Forecasting & Segmentation Module:** Predicts future sales trends (ROS) and groups products into meaningful AGs.
- **Forecasted ROS & Attribute Groups (AG):** Outputs of the forecasting and segmentation process.
- **OD/OW Calculation Engine:** Determines the **Optimum Depth** for SKUs and **Optimum Width** for categories.
- **Initial OD/OW Suggestions:** Preliminary recommendations for product assortment.
- **ISS Optimization Module:** Refines size distribution recommendations (**Ideal Size Set**).
- **Optimized OD/OW/ISS Recommendations:** Final, data-driven recommendations for merchandising.
- **Reporting & Output Generation Module:** Formats and exports the final recommendations.
- **Merchandising Systems & Dashboards:** Downstream systems consuming the generated outputs.

High-level data sources:

- **Point-of-Sale (POS) Data:** Transactional sales data (SKU, quantity, date, price, location).
- **Inventory Management System Data:** Current stock levels, inbound shipments, returns.
- **Product Master Data:** Comprehensive product information including descriptions, categories, AGs, cost, and vendor details.
- **Store / Location Data:** Information about sales channels, store types, and regional demographics.
- **Marketing & Promotional Data:** Details on past promotions, campaigns, and pricing changes.

4. Configuration & Data Dictionary

Settings Table (args.properties):

Parameter Name	Description	Default Value	Example Value
min_ros_threshold	Minimum ROS (units/week) for a SKU to be considered for OD .	5.0	10.0
inventory_turnover_target	Desired annual inventory turnover rate.	4.0	6.0
service_level_target	Target in-stock service level percentage for key SKUs.	0.95	0.98
lookback_period_weeks	Number of weeks of historical data to consider for analysis.	52	26
seasonal_adjustments_enabled	Enable/Disable automatic seasonal adjustments in forecasting.	true	false
output_format	Desired format for output files (CSV, JSON).	CSV	JSON
output_directory	Directory path where output files will be saved.	./output/	/data/recs/

Shared Data Fields (common-data.json):

Field Name	Data Type	Description	Example Value
sku_id	String	Unique identifier for each Stock Keeping Unit.	TSHIRTM001REDL
category	String	High-level product category (e.g., “Men’s Apparel”, “Footwear”).	Men's T-Shirts
subcategory	String	More granular product subcategory (e.g., “Graphic Tees”, “Running Shoes”).	Crew Neck
sales_units	Integer		150

Field Name	Data Type	Description	Example Value
		Number of units sold within a given period.	
inventory_units	Integer	Number of units currently in stock.	75
date	Date	Date of the sales or inventory record.	2024-05-20
size	String	Product size attribute (e.g., "S", "M", "L", "XL", "9", "10").	L
color	String	Product color attribute.	Red
brand	String	Brand name of the product.	CoolWear
price_point	String	Categorization of product by price (e.g., "Value", "Mid", "Premium").	Mid
attribute_group_id	String	Identifier for an Attribute Group (AG) , grouping similar products for analysis.	TSHIRT_GRAPHIC_RED
forecasted_ros	Float	Predicted Rate of Sale (units/week) for the SKU.	12.5

Outputs Catalog:

Output File Name	Format	Description	Key Metrics Included
depth_suggestions.csv	CSV	SKU-level recommendations for Optimum Depth (OD) .	sku_id, category, current_stocking, recommended_OD, forecasted_ros
width_suggestions.csv	CSV	Category-level recommendations for Optimum Width (OW) .	category, current_styles_offered, recommended_OW, category_ros_contribution
size_set_recommendations.csv	CSV	SKU-level recommendations for Ideal Size Set (ISS) .	sku_id, size, recommended_percentage, recommended_units
performance_summary.json	JSON	Aggregated key performance indicators and	total_savings_potential, overall_ros_improvement, top_k_recommendations

Output File Name	Format	Description	Key Metrics Included
debug_log.log	LOG	high-level insights. Detailed logs for troubleshooting and process monitoring.	Process steps, warnings, errors

5. End-to-End Flow

The depthSuggestion algorithm executes a systematic multi-stage process to deliver actionable merchandising recommendations:

- Ingest Raw Data:** The pipeline begins by securely collecting and consolidating raw sales transactions, current inventory levels, and comprehensive product master data from various internal systems.
- Preprocess & Clean:** Raw data undergoes rigorous cleaning, validation, and transformation. This includes handling missing values, correcting inconsistencies, standardizing formats (e.g., dates, units), and aggregating data to the necessary granularity (e.g., weekly **ROS** per SKU).
- Forecast Demand:** Using the cleaned historical data, advanced statistical and machine learning models predict future **ROS** for individual SKUs and aggregated product categories. This step accounts for seasonality, trends, and promotional impacts.
- Segment Products:** Products are dynamically clustered into **Attribute Groups (AG)** based on their characteristics, sales performance, and strategic importance. This allows for tailored **OD** and **OW** calculations.
- Calculate OD/OW:** Based on forecasted **ROS**, business rules (from `args.properties`), and strategic goals, the system determines the **Optimum Depth (OD)** for specific SKUs within their **AGs** and the **Optimum Width (OW)** for broader categories or styles. This balances sales potential with inventory efficiency.
- Optimize ISS:** For each recommended SKU, the **Ideal Size Set (ISS)** is generated by analyzing historical size-specific sales distributions, forecasted demand, and desired service level targets. This ensures the optimal mix of sizes to meet customer demand.
- Generate Outputs:** Finally, all calculated recommendations (OD, OW, ISS) and supporting performance metrics are compiled and formatted into user-friendly reports and data files (CSV, JSON) for seamless integration with merchandising systems and dashboards.

6. Submodule Overview

1. Data Preprocessing Module

- **Purpose:** Cleanses, validates, and aggregates raw input data, transforming it into a structured format suitable for subsequent analysis.
- **Inputs:** Raw sales data, raw inventory data, raw product master data.
- **Outputs:** Cleaned historical sales & inventory records, normalized product attributes, derived **ROS** metrics (historical).

- **Core Logic Summary:**
 - Ingest raw data from disparate sources, performing initial schema validation.
 - Standardize data formats (e.g., date formats, unit conversions) and resolve data type mismatches.
 - Handle missing values and outliers through imputation, removal, or flagging based on configured rules.
 - Aggregate transactional data to relevant time granularities (e.g., weekly, monthly) and product levels (SKU, category, **AG**).

2. Demand Forecasting & Segmentation Module

- **Purpose:** Predicts future sales performance (**ROS**) for products and groups similar items into **Attribute Groups (AG)** to enable targeted assortment optimization.
- **Inputs:** Cleaned historical sales & inventory records, normalized product attributes.
- **Outputs:** Forecasted **ROS** for each SKU and category, defined **AGs** with associated products.
- **Core Logic Summary:**
 - Apply a suite of forecasting models (e.g., ARIMA, Prophet, Machine Learning) to predict **ROS** for individual SKUs and aggregated levels.
 - Incorporate external factors like promotional periods, holidays, and trends if historical data or external feeds are available.
 - Utilize clustering algorithms (e.g., K-Means, hierarchical clustering) to group SKUs into **AGs** based on attributes, price points, and sales velocity.
 - Evaluate forecast accuracy and provide confidence intervals for predicted **ROS**.

3. OD/OW Calculation Engine

- **Purpose:** Determines the **Optimum Depth (OD)** for individual SKUs within a category and the **Optimum Width (OW)** for the overall product portfolio, based on forecasted performance and strategic objectives.
- **Inputs:** Forecasted **ROS**, defined **AGs**, configuration settings (min_ros_threshold, inventory_turnover_target).
- **Outputs:** Suggested **OD** per SKU, suggested **OW** per category.
- **Core Logic Summary:**
 - For **OD**: Rank SKUs within each **AG** by their forecasted **ROS**. Apply a dynamic threshold (e.g., minimum ROS, cumulative sales contribution) to identify the optimal number of unique items to stock.
 - For **OW**: Analyze the performance and strategic importance of each category or style. Balance market demand, competitive landscape, and internal resource constraints to recommend the ideal number of distinct categories to offer.
 - Incorporate inventory turnover targets and desired profit margins into the calculation to balance sales potential with financial efficiency.
 - Factor in any “must-stock” items or strategic exclusions defined by business rules.

4. ISS Optimization Module

- **Purpose:** Generates the **Ideal Size Set (ISS)** distribution for recommended SKUs to maximize sales capture while minimizing stockouts and excess inventory at the size level.
- **Inputs:** Suggested **OD** list, forecasted **ROS** by size, historical size sales distribution, `service_level_target`.
- **Outputs:** Recommended **ISS** per SKU (e.g., size curve percentages and absolute units).
- **Core Logic Summary:**
 - Analyze historical sales data to determine the typical sales distribution across different sizes for each SKU or its corresponding **AG**.
 - Adjust the historical size curve based on forecasted **ROS** and specific business rules (e.g., anticipated changes in demand for certain sizes).
 - Apply service level targets to calculate the optimal stock level for each size, aiming to balance availability with inventory costs.
 - Translate size distribution percentages into concrete unit recommendations based on the overall recommended **OD** for the SKU.

5. Reporting & Output Generation Module

- **Purpose:** Formats and exports the final, actionable merchandising recommendations and summarized performance metrics into various user-friendly formats.
- **Inputs:** Suggested **OD**, suggested **OW**, recommended **ISS**, aggregated performance KPIs.
- **Outputs:** `depth_suggestions.csv`, `width_suggestions.csv`, `size_set_recommendations.csv`, `performance_summary.json`.
- **Core Logic Summary:**
 - Consolidate recommendations from all preceding modules into comprehensive datasets.
 - Format data into specified output types (CSV, JSON) ensuring adherence to schema definitions.
 - Generate summary reports and dashboards highlighting key metrics and potential business impact (e.g., potential savings, improved ROS).
 - Implement logging for audit trails and error reporting during the output generation process.

7. Worked Example

Let's illustrate the calculation of **Optimum Depth (OD)** for a specific product category: **"Women's Athletic Shorts"**.

Scenario: A retailer currently stocks 20 unique SKUs of Women's Athletic Shorts. The `depthSuggestion` algorithm is run to determine the **OD**.

Input Data (Simplified):

SKU ID	Color	Size	Style	Forecasted Weekly ROS (units/week)	Current Stocking (Y/N)
WSH001BLKSM	Black	SM	Running	120	Y
WSH002BLKMD	Black	MD	Running	110	Y
WSH003BLKLG	Black	LG	Running	90	Y
WSH004GRYSM	Grey	SM	Running	85	Y
WSH005GRYMD	Grey	MD	Running	80	Y
WSH006BLUSM	Blue	SM	Cycling	70	Y
WSH007BLUMD	Blue	MD	Cycling	65	Y
WSH008PNKSM	Pink	SM	Casual	55	Y
WSH009PNKMD	Pink	MD	Casual	50	Y
WSH010BLKXL	Black	XL	Running	40	Y
WSH011GRYLG	Grey	LG	Running	35	Y
WSH012GRYXL	Grey	XL	Running	28	Y
WSH013BLULG	Blue	LG	Cycling	22	Y
WSH014PNKLG	Pink	LG	Casual	18	Y
WSH015WHITSM	White	SM	Casual	10	Y
... (5 more SKUs with ROS < 10)	...	...	...	< 10	Y

Configuration Setting: min_ros_threshold = 25.0 units/week (from args.properties)

Algorithm Logic:

- Filter by ROS Threshold:** The OD/OW Calculation Engine filters out SKUs whose forecasted **ROS** falls below the min_ros_threshold of 25 units/week.
 - SKUs WSH001BLKSM through WSH012GRYXL all have a forecasted **ROS** $\geq$ 28 units/week. (12 SKUs)
 - SKUs WSH013BLULG, WSH014PNKLG, WSH015WHITSM, and the remaining 5 unlisted SKUs (total 8 SKUs) have a forecasted **ROS** $<$ 25 units/week.
- Count Qualifying SKUs:** The algorithm counts the number of unique SKUs that meet or exceed the **ROS** threshold.

Result:

- Based on the min_ros_threshold of 25 units/week, the algorithm identifies **12 SKUs** (WSH001BLKSM to WSH012GRYXL) that are selling at an optimal rate.
- Recommended OD for “Women’s Athletic Shorts”: 12**

Interpretation: While the retailer currently stocks 20 unique SKUs, the analysis suggests that focusing on the 12 SKUs with an **ROS** above 25 units/week would maximize sales efficiency and inventory productivity for this category. The remaining 8 SKUs are candidates for discontinuation, promotional efforts, or reduced stocking levels. This helps optimize inventory investment by narrowing the **Optimum Depth** to the most productive items.

8. Debug Guide Highlights

- **Issue:** OD suggestions are consistently too low across categories.
 - **Check:** Review `min_ros_threshold` in `args.properties`. A threshold that is too high will aggressively reduce **OD**.
 - **Action:** Consider lowering `min_ros_threshold` or implementing a tiered threshold based on category importance.
 - **Check:** Data quality for forecasted **ROS**. Ensure the forecasting model is accurate and not under-predicting.
- **Issue:** ISS recommendations result in frequent stockouts for certain sizes, despite algorithm running.
 - **Check:** Verify `service_level_target` in `args.properties`. Ensure it's set appropriately for critical products.
 - **Check:** Historical size-specific sales data for completeness and accuracy. Are there spikes or missing data points?
 - **Action:** Investigate if new trends or promotions impacted size distribution that the historical data might not capture immediately.
- **Issue:** Processing takes an unusually long time to complete.
 - **Check:** Data volume and granularity. Processing transaction-level data for a long `lookback_period_weeks` can be resource-intensive.
 - **Action:** Optimize data aggregation steps in the Data Preprocessing Module. Consider running for smaller `lookback_period_weeks` or on a subset of data for quick checks.
 - **Check:** Server resources (CPU, RAM). The algorithm can be compute-intensive, especially for large datasets.
- **Issue:** Output files are empty or contain unexpected values.
 - **Check:** Input data paths and formats. Ensure `common-data.json` schema is strictly followed.
 - **Check:** Logs (`debug_log.log`) for early errors during data ingestion or processing.
 - **Action:** Verify output directory permissions and available disk space.

9. Common Pitfalls & Remedies

- **Pitfall: Outdated or inaccurate input data.**
 - **Description:** Recommendations are only as good as the data they're built on. Stale sales data or incorrect inventory counts can lead to flawed **OD**, **OW**, or ISS suggestions.
 - **Remedy:** Implement robust data validation checks at the Data Preprocessing Module stage. Establish clear data refresh schedules and automate data ingestion processes. Regularly audit key data sources.
- **Pitfall: Over-reliance on historical data without considering future trends.**
 - **Description:** The algorithm might struggle with new product introductions, sudden market shifts, or the impact of significant promotions if only historical sales are considered.
 - **Remedy:** Enhance the Demand Forecasting & Segmentation Module to incorporate external market signals, planned promotional calendars, and user-defined strategic overrides for new or flagship products. Allow for manual adjustments by merchandising teams.

- **Pitfall: Ignoring product lifecycle stages.**
 - **Description:** A product near its end-of-life should have a different **OD** strategy than a newly launched item, but the algorithm might treat them similarly if not informed.
 - **Remedy:** Integrate product lifecycle stages (e.g., New Arrival, Core, Clearance) as an input attribute. Adjust **OD** and **ISS** logic based on these stages (e.g., lower **OD** for clearance, more aggressive **ISS** for new items).
- **Pitfall: One-size-fits-all thresholding.**
 - **Description:** Using a single `min_ros_threshold` for all categories might not be optimal, as luxury goods have different sales velocities than fast-moving consumables.
 - **Remedy:** Implement dynamic thresholds based on category, `price_point`, or other relevant **Attribute Groups (AG)**. Allow for configurable thresholds per **AG**.
- **Pitfall: Lack of transparency or interpretability for business users.**
 - **Description:** Merchandising teams may resist adopting recommendations if they don't understand the underlying logic or rationale.
 - **Remedy:** Enhance the Reporting & Output Generation Module to include clear explanations, confidence scores, and drill-down capabilities for each recommendation. Provide a user-friendly interface or dashboard to visualize results.

10. FAQ

- **Q:** What is the primary difference between **OD** and **OW**?
 - **A:** **OD** (Optimum Depth) refers to the ideal number of *unique items (SKUs)* you should stock within a specific style, category, or **AG**. For example, how many different types of “Men’s Blue Jeans” should be offered. **OW** (Optimum Width) refers to the ideal number of *different categories or styles* to offer overall. For example, how many different types of “Jeans” (skinny, regular, relaxed, bootcut) should the store carry.
- **Q:** How often should I run the `depthSuggestion` algorithm?
 - **A:** The frequency depends on your business’s sales velocity and market volatility. For high-velocity categories, weekly or bi-weekly runs are recommended. For slower-moving goods or seasonal items, monthly or quarterly might suffice. Reviewing **ROS** trends will help determine the optimal cadence.
- **Q:** Can the recommendations from `depthSuggestion` be manually overridden?
 - **A:** Absolutely. The algorithm provides data-driven recommendations, but merchandising teams retain the final decision. It’s crucial for human insight to validate results, especially when qualitative factors, strategic initiatives, or unforeseen market events are at play. The outputs are designed to be reviewed and adjusted.
- **Q:** What happens if an item has very low historical sales but is strategically important?
 - **A:** The system allows for configuration of “must-stock” items or minimum **ODs** for certain categories, irrespective of their **ROS**. Such strategic items can be flagged in the product master data to ensure they are always included in the recommended **OD**.
- **Q:** How does `depthSuggestion` handle new products with no sales history?
 - **A:** For new products, the Demand Forecasting & Segmentation Module can use proxy data (e.g., sales of similar products, industry benchmarks) and pre-defined business rules to assign an initial estimated

ROS and an appropriate **AG**. As sales data accumulates, the algorithm will naturally adjust its recommendations.

11. Quick Reference Cheat-Sheet

Term / Concept	Definition & Purpose
OD (Optimum Depth)	The ideal number of unique items (SKUs) to stock in a category or display. Focuses on how many of <i>what</i> SKU to carry.
OW (Optimum Width)	The ideal number of different categories or styles to offer. Focuses on how many <i>different kinds of products</i> to carry.
ISS (Ideal Size Set)	The best combination of sizes to maximize sales and minimize stockouts for a specific SKU.
ROS (Rate of Sale)	The speed at which products are sold over a period (e.g., units/week). A key metric for assessing product performance.
AG (Attribute Group)	A set of product attributes grouped for analysis (e.g., color, size, brand). Used to segment products for tailored recommendations.
Project Goal	To optimize product assortment by providing data-driven recommendations for OD , OW , and ISS , leading to increased sales and reduced inventory costs.
Key Inputs	Historical sales data, current inventory levels, comprehensive product attributes.
Key Outputs	Actionable CSV/JSON files with recommended OD , OW , and ISS per product/category.
Customization	Configurable parameters (<code>args.properties</code>) allow tailoring recommendations to business objectives and risk tolerance.
Benefits	Maximized sales, minimized stockouts, efficient inventory utilization, improved profitability, enhanced customer satisfaction.

Chapter 3: Distribution Data Preparation

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for loading and preparing all the necessary master and transactional data, ensuring it is correctly formatted and available for subsequent distribution calculations to maximize sales and minimize stockouts.

Why This Matters: This foundational step guarantees that all distribution decisions are based on accurate, complete, and relevant information. Without proper data preparation, distribution strategies could lead to incorrect stock levels, missed sales opportunities, and inefficient inventory management, ultimately impacting profitability and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Current Store Inventory Levels:** Why: *Provides the baseline of what is currently available at each retail location to start distribution planning.*
- **Incoming Shipments and Pending Customer Orders:** Why: *Accounts for products expected to arrive soon or already allocated to customers, preventing over-allocations and ensuring future fulfillment needs are met.*

Product Data

- **Product Definitions and Attributes:** Why: *Defines each unique item (SKU) including its style, category, size, and other attributes, which are essential for grouping and analyzing products.*
- **Ideal Size Set (ISS) and Optimum Depth (OD) Rules:** Why: *Specifies the preferred mix of sizes (ISS) and ideal stock quantities (OD) for products, guiding how many of each SKU should be stocked.*

Operational Data

- **In-Scope Stores and Warehouse Inventory:** Why: *Identifies all stores and warehouses involved in the distribution network and their current stock, ensuring all relevant locations are included in calculations.*
- **Store Planogram Configurations and Replenishment Cycles:** Why: *Provides details on physical store displays (planograms) and how often stores receive new stock, informing inventory holding targets and delivery schedules.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Replenishment Season List	N/A	Specifies which seasons' products are considered for active replenishment or allocation.	Before a new season starts or if an existing season needs to be deprioritized.
Minimum Planogram Fill Rate	N/A (e.g., 85%)	The minimum percentage of allocated display space for a product category that a store aims to fill.	To adjust inventory targets based on store capacity or visual merchandising strategy.
Include In-Store Inventory	True	Determines whether current inventory levels physically present in stores are included in distribution calculations.	If you want to simulate distribution without considering existing store stock, e.g., for initial allocations.
Include In-Transit Inventory	True	Determines whether products currently in transit to stores or warehouses, or open customer orders, are considered as available stock.	To get a more realistic view of future available inventory, or to exclude if only immediate physical stock matters.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Input Data] --> B[Main Process]
B --> C[Output Result]
```

1. Start with → Raw sales figures, current inventory, product master data, and store configurations.
2. Then → The system loads, cleans, filters, aggregates, and applies all relevant business rules to this data.
3. Next → It cross-references product attributes with store-specific settings and consolidates all inventory views.
4. Finally → A comprehensive, validated, and enriched dataset ready for distribution calculations.

5. Step-by-Step Actions

1. **Identify** In-Scope Stores:
 - What we do: *We pinpoint which retail locations will be included in the distribution process.*
 - Why we do it: *This ensures that calculations are performed only for relevant stores.*
2. **Consolidate** Inventory Data:
 - What we do: *We gather all current and incoming inventory information from stores, warehouses, and transit, plus pending customer orders.*
 - Why we do it: *This provides a complete picture of available and committed stock across the entire network.*
3. **Apply** Product & Store Rules:
 - What we do: *We integrate specific rules for how products should be stocked, such as ideal stock levels (OD), ideal size sets (ISS), pack sizes, and channel-specific overrides.*
 - Why we do it: *This tailors distribution decisions to each product's attributes and store's unique needs.*
4. **Map** Attributes & Replenishment Cycles:
 - What we do: *We link product categories and styles to their attributes, store planogram details, and replenishment schedules.*
 - Why we do it: *This ensures distribution aligns with store display strategies and delivery frequencies.*
5. **Prepare** Sales Contribution Data:
 - What we do: *We process historical sales data to understand the contribution of different sizes and product attributes.*
 - Why we do it: *This informs how Ideal Size Sets (ISS) are determined and refined for future allocations.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new popular style of denim jeans (Style #12345) is launching.
- Initial central warehouse stock: 5,000 units.
- This style has 5 sizes (XS, S, M, L, XL).
- Store A, a high-volume store, is set to receive this style.
- Store A's planogram for denim allows for 20 units per style on display.
- Store A's replenishment cycle is 7 days.
- Ideal Size Set (ISS) data for denim in high-volume stores suggests M and L sizes contribute 60% of sales.

What Happens

1. First, the system **confirms** Store A is an active distribution store, retrieving its basic profile.
2. Then, the 5,000 units in the central warehouse are **logged** as available inventory for this style, including all relevant SKUs (sizes).
3. Next, Store A's denim planogram (20 units/style) and its 7-day replenishment cycle are **integrated** into the store's data profile.
4. Following this, the existing Ideal Size Set (ISS) data for denim categories is **applied** to Style #12345, identifying that M and L sizes are expected to drive 60% of sales. This prepares the system to allocate these sizes proportionally.
5. Finally, if there's a specific override for Style #12345 at Store A (e.g., to stock 25 units instead of the category default 20), this rule is **captured** and prioritized in the data.

Final Outcome

The system now has a complete, validated profile for distributing Style #12345 to Store A, including its total available units, target on-hand quantities based on planogram and replenishment, and the Ideal Size Set (ISS) breakdown for how many of each size (XS-XL) should ideally be sent to maximize sales and minimize stockouts.

Key Takeaway: This step ensures that all relevant data points, from available stock to specific store and product rules, are accurately combined and ready to inform precise distribution decisions.

7. Results

This step produces:

Main Outputs

- **Consolidated Inventory View:** A comprehensive snapshot of all product inventory across stores, warehouses, and in transit, enabling a single source of truth for available stock.
- **Store and Product Rule Sets:** A structured collection of business rules, such as ideal stock levels (OD), ideal size sets (ISS), specific pack sizes, and replenishment cycles, tailored for each store and product.

- **Seasonality and Channel Status:** Clear identification of which products are in-season, live, or subject to specific channel-based distribution overrides.

Business Reports

- **Inventory Reconciliation Report:** Used by inventory managers to cross-verify the loaded stock levels against their records, ensuring data accuracy before distribution runs.
- **Rule Validation Report:** Used by merchandisers and planners to review if their defined OD, ISS, and other business rules have been correctly interpreted and applied by the system.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential data is missing for the current run, potentially leading to incomplete or inaccurate results.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides.
Store not found in distribution list	The system will exclude the store from distribution calculations.	Verify if the store was intended to be included. If yes, add it to the in-scope stores list.
Product style or SKU data is incomplete	Calculations for that product might be skipped or lead to errors if essential attributes (like category or size) are missing.	Update product master data. Review data quality processes to prevent future occurrences.
Conflicting business rules (e.g., depth override clashes)	The system will typically apply a predefined hierarchy (e.g., SKU-level rules override style-level rules) but may log a warning.	Review and reconcile conflicting rules in the input data to ensure desired behavior.

9. Next Steps

Before This Step

- **Master Data Management:** What we need from it: *Accurate and up-to-date product definitions (SKUs, styles, categories, attributes), store locations, and warehouse configurations.*

- **Transaction Data Collection:** What we need from it: *Real-time sales figures, current inventory counts, open order details, and goods in transit information.*

After This Step

- **Distribution Calculation Engine:** What they need from us: *A clean, consolidated, and rule-applied dataset that provides all necessary inputs to determine optimal product allocations.*
- **Allocation Optimization Module:** What they need from us: *The prepared data to calculate ideal stock levels (OD) and Ideal Size Sets (ISS) for each store, leading to precise allocation recommendations.*

10. Common Questions

Q: Why is it important to include in-transit inventory? A: Including in-transit inventory provides a more accurate view of *future* available stock. Ignoring it could lead to over-ordering or misallocating products that are already on their way, causing excess stock later.

Q: What happens if a store's planogram data is missing? A: If planogram data is missing, the system will typically fall back to default category settings or system-wide averages. This might lead to allocations that don't match the store's actual display capacity or merchandising strategy.

Q: Can we run distribution for only specific stores or products? A: Yes, this preparation step allows for filtering. By defining which stores are "in-scope" and which product styles have specific overrides or are "in-season," the system can focus its calculations efficiently.

Q: How does seasonality affect the data preparation? A: Seasonality determines which products are considered "active" or "in-season" for replenishment and allocation. Products outside the defined active seasons might be deprioritized or excluded from automatic distribution calculations.

Q: What if our sales data is inaccurate or incomplete? A: Inaccurate sales data will directly impact the calculated Ideal Size Sets (ISS), Optimum Depths (OD), and Rate of Sale (ROS) metrics. This can lead to incorrect allocation decisions, resulting in stockouts of popular items or overstocking of slow movers. It's crucial for sales data to be timely and accurate.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To consolidate, validate, and enrich all core data (sales, inventory, product, operational) into a unified, clean format, ready for distribution calculations.
- **Main Input:** Raw sales figures, current inventory levels across stores and warehouses, product master data with attributes, and store-specific operational settings.

- **Main Output:** A comprehensive, prepared dataset with applied business rules, enabling precise inventory allocation decisions.
 - **Warning Signs:** Unexpectedly low stock levels in key stores, missing product attributes, or inconsistent sales data for active styles.
 - **Key Settings:** Replenishment Season List, Minimum Planogram Fill Rate, Inclusion of In-Store and In-Transit Inventory.
-

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: Distribution Sales Preparation

Last Updated: 2025-05-23

1. Business Objective

This module focuses on preparing and cleaning sales data for analysis by removing misleading sales figures and computing the Rate of Sale (ROS) for individual items (SKUs) across stores.

Why This Matters: This crucial step ensures that all subsequent inventory distribution and planning decisions are based on accurate and reliable sales performance, leading to more profitable stock allocations, fewer missed sales due to stockouts, and reduced overstock situations.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records (Units Sold, Date, SKU, Store):** *Provides the raw transaction data for every item sold, when and where.*
- **Liquidation Sale Indicators:** *Identifies sales periods where products were heavily discounted or on clearance, as these sales don't represent typical demand.*

Product Data

- **Product Definitions and Attributes:** *Details each unique item (SKU), including its style, category, and other relevant Attribute Groups (AGs), which are essential for grouping and analyzing sales performance.*
- **Product Live Days (Store & Style Level):** *Indicates the specific days a product or style was physically available and present in a store, crucial for calculating accurate ROS.*

Operational Data

- **In-Scope Store List:** *Identifies all retail locations that are part of the active distribution network and need sales data processed.*

- **Analysis Period Dates:** *Defines the specific start and end dates for which sales data should be analyzed for ROS calculations.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Liquidation Sales Exclusion Threshold	0.0 (e.g., 0.10)	The sales velocity (ROS) percentage below which a sale is considered a “liquidation” and excluded from analysis.	When new promotional strategies or clearance benchmarks are set.
Minimum Product Presence for Sales Validity	0.0 (e.g., 0.85)	The minimum percentage of an item’s Optimum Depth (OD) or Ideal Size Set (ISS) that must be present on display for sales during that day to be considered “unbroken” or valid.	To adjust for merchandising standards or visual display requirements.
Minimum Active Days for Sales Rate Calculation	0 (e.g., 7)	The minimum number of days an item must have been actively available in a store within a sales period to have its Rate of Sale (ROS) calculated.	To ensure ROS is based on sufficient selling days, preventing skewed results from brief availability.
Sales Rate Calculation Period Lengths	N/A (e.g., 30, 60, 90 days)	The specific durations over which historical sales are analyzed to compute different Rate of Sale (ROS) values (e.g., short-term, mid-term, long-term).	To adjust the sensitivity of ROS to recent trends versus broader historical performance.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Data, Product & Store Info] --> B[Sales Data Preparation]

B --> C[Cleaned Sales & Calculated Rate of Sale (ROS)]

1. Start with → Raw sales figures, product definitions, and information about when products were live in stores.
2. Then → The system removes sales from liquidation events and periods where product presence was incomplete (“broken” sales).
3. Next → It calculates the Rate of Sale (ROS) for each unique item (SKU) in each store using the cleaned data.
4. Finally → A refined dataset of valid sales and precise ROS values, ready for optimal distribution calculations.

5. Step-by-Step Actions

1. **Remove Liquidation Sales:**
 - What we do: *We filter out sales data that occurred during periods identified as liquidation or heavy clearance, as these sales do not reflect regular demand.*
 - Why we do it: *This prevents abnormally high sales (due to steep discounts) from skewing the true demand forecast for future allocations.*
2. **Identify and Correct “Broken” Sales Data:**
 - What we do: *We analyze historical sales periods to ensure that a product or style was adequately present and available for customers in a store during those sales days. Sales from days with insufficient product presence are flagged or removed.*
 - Why we do it: *This prevents misleading sales figures that arise when an item sells poorly simply because it wasn't fully stocked or displayed, ensuring ROS accurately reflects demand under ideal conditions.*
3. **Calculate Rate of Sale (ROS):**
 - What we do: *We compute the Rate of Sale (ROS) for each unique item (SKU) in every store by dividing the total Units Sold by the number of days the SKU was actively available for sale, often weighted across multiple historical periods.*
 - Why we do it: *This provides a standardized measure of how quickly each product sells, which is essential for determining optimal inventory levels and replenishment frequency.*
4. **Prioritize Seasonal Sales:**
 - What we do: *We identify and isolate sales data from key seasons or priority periods (e.g., holiday season, spring launch) that are deemed most relevant for current distribution decisions.*
 - Why we do it: *This ensures that distribution models are informed by the most pertinent and recent demand patterns, especially for fashion and seasonal items.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Women’s Floral Dress, SKU 78901, Size M” is sold at Store B.
- Raw sales data for this SKU in Store B shows 120 Units Sold over the last 90 days.
- Within these 90 days:
 - Days 1-14 were a store-wide clearance event (liquidation sales flagged).
 - Days 45-47, the dress display was incomplete due to a planogram error (product presence below Minimum Product Presence for Sales Validity).
- ROS is calculated using a weighted average of sales over 30, 60, and 90 days.

What Happens

1. First, the system **identifies** the sales from Days 1-14 as liquidation sales and **excludes** them from the demand calculation for SKU 78901.
2. Then, it **detects** that for Days 45-47, the product was “broken” (insufficiently stocked). Sales from these 3 days are **excluded** from the valid sales period for ROS calculation.
3. Next, the system **re-calculates** the total valid Units Sold and the actual “live days” (days the product was adequately present and not in liquidation). Let's say, after cleaning, 100 units were sold over 75 valid days.
4. Finally, the Rate of Sale (ROS) for “Women’s Floral Dress, SKU 78901, Size M” in Store B is **computed** as $100 \text{ units} / 75 \text{ days} = 1.33 \text{ units/day}$. This clean ROS is then used for the weighted average across the 30, 60, and 90-day valid periods.

Final Outcome

The system produces an accurate, reliable Rate of Sale (ROS) for “Women’s Floral Dress, SKU 78901, Size M” in Store B, which reflects its true demand under normal selling conditions. This accurate ROS will directly inform how many units of this dress should be allocated to Store B to maximize sales and avoid unnecessary stock.

Key Takeaway: Cleaning and preparing sales data ensures that distribution decisions are based on a realistic understanding of demand, leading to more efficient inventory management and improved profitability.

7. Results

This step produces:

Main Outputs

- **Cleaned Sales Data:** A comprehensive dataset of historical sales, filtered to exclude liquidation and “broken” sales, providing a true representation of regular demand.

- **Calculated Rate of Sale (ROS) per SKU per Store:** The precise sales velocity for every unique item at each retail location, serving as a primary input for demand forecasting.
- **Prioritized Sales Data:** Specific subsets of sales data marked as high-priority (e.g., from current season or critical periods) for focused analysis in subsequent modules.

Business Reports

- **Sales Data Quality Report:** Used by data analysts and planners to review the volume of sales data removed due to liquidation or “broken” periods, ensuring data integrity.
- **SKU ROS Performance Report:** Used by merchandisers and planners to visualize the sales velocity of individual SKUs across different stores, aiding in performance review and allocation adjustments.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available sales data, or report that essential sales data is missing, potentially leading to incomplete or inaccurate ROS calculations.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides based on expert judgment.
A product has no sales data	The system cannot calculate a Rate of Sale (ROS) for that SKU in the given store, potentially leading to zero allocation or reliance on category averages.	Review if the product is new or if there’s a data capture issue. Consider using a default ROS or a category average for new items.
All sales data for a SKU/store is marked as “broken”	The system will not be able to calculate a valid Rate of Sale (ROS) for that specific SKU in that store.	Investigate the reason for consistent “brokenness” (e.g., persistent stocking issues, data errors) and address the root cause to enable accurate ROS calculation.
High volume of liquidation sales skews ROS calculation	Even after initial filtering, pervasive liquidation sales might still influence the calculated ROS, potentially misleading future allocations.	Adjust the “Liquidation Sales Exclusion Threshold” setting or consider a more extended historical analysis period for ROS.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All raw master data (products, stores) and transactional data (sales, inventory) to be loaded, validated, and initially formatted.*
- **Master Data Management:** What we need from it: *Accurate and up-to-date definitions for all products (SKUs, styles, categories) and store profiles, including their active/in-scope status.*

After This Step

- **Distribution Calculation Engine:** What they need from us: *The cleaned sales data and the calculated Rate of Sale (ROS) for each SKU in each store, which are primary inputs for determining optimal inventory levels and allocations.*
- **Allocation Optimization Module:** What they need from us: *The precise ROS and prepared sales data to calculate Ideal Size Sets (ISS) and target Optimum Depths (OD) for each product in each store, guiding the final allocation recommendations.*

10. Common Questions

Q: Why is it important to remove liquidation sales from the data? A: Liquidation sales often involve significant discounts that temporarily inflate sales volume. Including them in demand calculations would lead to an artificially high Rate of Sale (ROS), causing us to overstock products at regular prices and tie up valuable capital in excess inventory.

Q: What does “broken” sales data mean, and why is it removed? A: “Broken” sales data refers to sales periods where a product was not properly available or displayed in a store (e.g., out of stock for a significant portion of the day, misplaced on a wrong display, or below the required Minimum Product Presence). Removing these ensures that the calculated Rate of Sale (ROS) accurately reflects true customer demand when the product is readily available.

Q: How is the calculated Rate of Sale (ROS) used in distribution? A: The Rate of Sale (ROS) is a critical metric for demand forecasting. It tells us how quickly an item is selling at a specific store. This information helps determine the Ideal Size Set (ISS) and Optimum Depth (OD) for that item, guiding how many units of each size should be allocated to maximize sales and minimize stockouts for a given replenishment cycle.

Q: What happens if a new product has no historical sales data for ROS calculation? A: For new products without historical sales, the system will typically fall back to using a Rate of Sale (ROS) derived from similar products, the broader category, or a pre-defined average ROS based on product attributes (like Attribute Group, price tier, or style). This allows us to make an initial allocation based on expected performance.

Q: How do the different “Sales Rate Calculation Period Lengths” affect the ROS? A: Using different period lengths (e.g., 30, 60, 90 days) for ROS calculation allows us to capture both short-term trends and long-term stability. A shorter period (30 days) is more sensitive to recent spikes or drops in demand, while a longer period (90 days) provides a more stable, averaged view. The system often uses a weighted average of these periods to create a balanced ROS that accounts for both recency and historical consistency.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To transform raw, potentially misleading sales data into a clean, reliable foundation for all inventory distribution decisions.
- **Main Input:** Raw sales transactions, product definitions (SKU, style, AG), and store operational data including product live days.
- **Main Output:** Cleaned sales data and precise Rate of Sale (ROS) values for each SKU in each store.
- **Warning Signs:** Unexpectedly high or low ROS values for established products, or a significant portion of sales data being flagged as “broken.”
- **Key Settings:** Liquidation Sales Exclusion Threshold, Minimum Product Presence for Sales Validity, and Sales Rate Calculation Period Lengths.

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: Distribution Warehouse Inventory Preparation

Last Updated: 2025-05-23

1. Business Objective

This module prepares warehouse inventory data by adjusting on-hand stock to reflect true availability and account for future demand during lead times.

Why This Matters: This crucial step ensures that all inventory allocation decisions are based on accurate, deployable stock levels. By correcting for warehouse discrepancies and reserving stock for ongoing sales during transit times, we prevent stockouts, minimize over-allocations, and improve the reliability of our supply chain.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records (Units Sold for Online Stores):** *Provides the data to calculate the Rate of Sale (ROS) for products, specifically from online retail channels, which is essential for determining how much stock to reserve for lead time.*

Product Data

- **Product Definitions and Attributes (SKU and Style):** *Identifies each unique item (SKU) and its broader style, allowing for accurate tracking and adjustment of inventory levels.*

Operational Data

- **Current Warehouse Inventory Levels:** *The baseline quantity of each product currently held in distribution warehouses.*
- **Warehouse Online Status:** *Indicates which warehouses serve online orders, as lead time reservations primarily apply to these locations.*
- **Store Channel Type (Online/Offline):** *Distinguishes online stores from physical stores, ensuring lead time inventory reservations are based on the relevant sales channels.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Warehouse Discrepancy Factor	N/A (e.g., 2 units)	A fixed quantity deducted per SKU from the recorded warehouse stock to account for common inventory discrepancies or damage.	To align with actual physical inventory audit results or observed warehouse shrinkage rates.
Inventory Reservation Days	N/A (e.g., 5 days)	The number of days' worth of anticipated online store sales to reserve in the warehouse to cover lead time before new inventory arrives.	When average transit times or order fulfillment durations change significantly.
Online Store Sales Period for Lead Time	N/A (e.g., 30, 60, 90 days)	The specific historical durations (e.g., last 30, 60, or 90 days) over which Rate of Sale (ROS) is calculated for online stores to inform lead time reservations.	To make ROS more sensitive to recent trends or to provide a more stable, averaged view of sales.
Minimum Daily Sales for Inventory Reservation	N/A (e.g., 0.1 units/day)	The minimum Rate of Sale (ROS) an item must achieve in online stores to qualify for a lead time inventory reservation.	To prevent reserving stock for extremely slow-moving items that don't justify the reservation.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Warehouse Inventory & Online Sales Data] --> B[Warehouse Inventory Preparation]

B --> C[Adjusted Warehouse Inventory for Allocation]

1. Start with → The raw, reported inventory counts in warehouses and recent online sales data.
2. Then → The system reduces stock by a defined inefficiency factor per SKU to account for minor discrepancies.
3. Next → It calculates the Rate of Sale (ROS) for online stores and reserves a portion of inventory to cover anticipated sales during lead time.
4. Finally → A refined, accurate view of truly available warehouse inventory, ready for allocation.

5. Step-by-Step Actions

1. **Adjust** for Warehouse Inefficiency:
 - What we do: *We reduce the reported on-hand quantity for each unique item (SKU) in the warehouse by a small, fixed amount, representing expected discrepancies or uncounted items.*
 - Why we do it: *This creates a more conservative and realistic view of available stock, preventing over-allocation based on potentially inflated counts.*
2. **Determine** Need for Lead Time Reservation:
 - What we do: *The system checks if any of the stores targeted for distribution are online stores. If not, this step is skipped.*
 - Why we do it: *Lead time inventory reservations are typically crucial for online fulfillment, where delays can directly impact customer satisfaction. Physical stores often have different replenishment models.*
3. **Calculate** Rate of Sale (ROS) for Online Channels:
 - What we do: *For each SKU, we compute its Rate of Sale (ROS) based on historical sales data from online stores over defined periods (e.g., last 30, 60, or 90 days).*
 - Why we do it: *This provides a reliable measure of how quickly items are selling through online channels, which is critical for estimating how much stock needs to be reserved.*
4. **Reserve** Inventory for Lead Time:
 - What we do: *For each SKU, the system multiplies its calculated online ROS by the "Inventory Reservation Days" setting to determine the quantity of stock to be held back in online warehouses. This reserved quantity is then deducted from the immediately available warehouse stock.*
 - Why we do it: *This ensures that enough stock is always available in the warehouse to cover ongoing online sales while new inventory is in transit, preventing critical stockouts and allowing continuous fulfillment.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Smartwatch, SKU 54321” has a current recorded inventory of 150 units in the main online distribution warehouse.
- The “Warehouse Discrepancy Factor” is set to 2 units per SKU.
- The “Inventory Reservation Days” setting is 4 days.
- Historical online sales for “Smartwatch, SKU 54321” show an average Rate of Sale (ROS) of 10 units/day over the relevant period.
- This warehouse primarily serves online stores.

What Happens

1. First, the system **applies** the “Warehouse Discrepancy Factor.” It reduces the recorded 150 units by 2 units, bringing the immediately available stock to 148 units ($150 - 2$).
2. Then, since this warehouse serves online stores, the system **calculates** the quantity to reserve for lead time. It multiplies the online ROS (10 units/day) by the “Inventory Reservation Days” (4 days), resulting in a 40-unit reservation ($10 * 4$).
3. Next, the system **deducts** these 40 reserved units from the previously adjusted stock. The available stock becomes 108 units ($148 - 40$).
4. Finally, this updated figure of 108 units is **flagged** as the true available inventory for allocation, while the 40 units are reserved for ongoing online sales during the lead time.

Final Outcome

The system now accurately reflects that only 108 units of “Smartwatch, SKU 54321” are truly available for new allocations or transfers to other locations, while 40 units are appropriately reserved for expected online sales during the lead time. This prevents situations where new allocations might deplete stock needed for imminent online orders, ensuring continuous sales and customer satisfaction.

Key Takeaway: This step refines warehouse stock figures to represent actual deployable inventory, considering both internal discrepancies and future sales needs during transit times.

7. Results

This step produces:

Main Outputs

- **Adjusted Warehouse Inventory Levels:** The precise, true quantity of each SKU available in every warehouse after accounting for inefficiencies and lead time reservations.

- **Warehouse Stock Movement Records:** Details of stock deductions due to inefficiency or lead time reservations, providing an audit trail for inventory adjustments.

Business Reports

- **Available Warehouse Stock Report:** Used by inventory planners and allocation teams to view the most accurate available-to-allocate inventory at each warehouse.
- **Inventory Reservation Summary:** Used by operations and finance teams to understand how much stock is being held back for lead times and inefficiency, aiding in inventory planning and financial forecasting.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available warehouse inventory data, or report that essential data is missing, potentially leading to inaccurate stock adjustments.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides.
Warehouse stock becomes negative after adjustment	If an SKU's recorded stock is less than the "Warehouse Discrepancy Factor," the adjusted quantity will be set to zero.	Review physical inventory processes. Consider if the "Warehouse Discrepancy Factor" is too high for certain low-stock SKUs.
No online stores or online sales data for ROS calculation	The system will not perform the lead time inventory reservation, potentially leading to over-allocation of stock needed for future online demand.	Verify if online channels are correctly configured and if sales data is flowing properly. Adjust strategy if lead time reservation is not desired for certain distribution types.
Very low Rate of Sale (ROS) for an SKU	The calculated lead time reservation will be negligible or zero, potentially causing an allocation of almost all stock, even if the item is still selling slowly.	Review the "Minimum Daily Sales for Inventory Reservation" setting or consider minimum stock holding rules for very slow movers.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *Initial loading and validation of all raw warehouse inventory data, product definitions, and store channel types.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned and prepared historical sales data, including calculated Rate of Sale (ROS) for online stores, which is used to determine lead time reservations.*

After This Step

- **Allocation Optimization Module:** What they need from us: *The most accurate and adjusted warehouse inventory levels to make precise allocation recommendations, ensuring sufficient stock is available to fulfill orders and cover lead times.*
- **Transfer Calculation Engine:** What they need from us: *The refined inventory data to identify optimal quantities of products to move between warehouses or to stores, based on true availability.*

10. Common Questions

Q: Why do we reduce stock by a “Warehouse Discrepancy Factor”? A: Reducing stock by a discrepancy factor accounts for minor inventory inaccuracies that commonly occur in warehouses due to damage, miscounts, or theft. It provides a more realistic available stock figure, preventing us from promising inventory we don’t actually have, which could lead to missed sales and customer dissatisfaction.

Q: Why is it important to reserve inventory for “lead time”? A: Reserving inventory for lead time ensures that products needed to cover ongoing online sales are not accidentally allocated to new transfers or physical store replenishments. This is crucial for maintaining continuous fulfillment for online customers, preventing stockouts during the period new inventory is in transit, and safeguarding the customer experience.

Q: How does the Rate of Sale (ROS) impact warehouse inventory preparation? A: The Rate of Sale (ROS) is vital for calculating the “Inventory Reservation Days.” By knowing how quickly an item sells in online stores, we can accurately estimate how much stock needs to be held back in the warehouse to cover sales during the time it takes for new inventory to arrive or for an allocation to reach its destination.

Q: What if a warehouse is not classified as “online”? Does it still get lead time adjustments? A: No, if a warehouse is not classified as “online” (meaning it primarily serves physical stores or acts as a transit point), it typically does not undergo the lead time inventory reservation. This is because the lead time concept, in this context, is specifically tied to maintaining continuous online fulfillment where immediate availability is paramount.

Q: Can warehouse inventory become negative after these adjustments? A: While the system ensures adjusted quantities don't go below zero for *available* stock, the calculations for reductions (like inefficiency or lead time reservations) might conceptually imply a need for more stock than currently available. If a reduction would make stock negative, the system sets the available quantity to zero, indicating that there is no deployable stock left.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To refine warehouse inventory data by accounting for physical discrepancies and reserving stock for future sales during lead times.
- **Main Input:** Raw warehouse inventory counts, online store sales data (for ROS), and warehouse/store channel classifications.
- **Main Output:** Adjusted and truly available warehouse inventory levels for allocation, and a record of reserved stock.
- **Warning Signs:** Unexpectedly low available warehouse stock, or large quantities of stock being reserved for lead time without clear justification.
- **Key Settings:** Warehouse Discrepancy Factor, Inventory Reservation Days, Online Store Sales Period for Lead Time, Minimum Daily Sales for Inventory Reservation.

Need Help? Contact your inventory management or supply chain team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: Distribution Warehouse Reservation

Last Updated: 2025-05-23

1. Business Objective

This module establishes and manages strategic rules for holding back a specific percentage of warehouse inventory for particular product styles based on their assigned sales channel.

Why This Matters: This crucial step ensures that critical stock is available for strategic initiatives like new product launches, exclusive online sales, or specific customer segments. By preventing premature allocation to general replenishment, it maximizes revenue opportunities, supports controlled product introductions, and maintains strategic inventory control.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- This module does not directly use historical sales data for its core function of setting reservation percentages. The reservation rules are based on predefined business decisions, not dynamic sales performance.

Product Data

- **Product Style Definitions:** *To identify unique product styles and ensure reservation rules are applied to valid, active items within our product catalog.*
- **Product Hierarchy (Parent Styles):** *To allow reservation rules to be correctly interpreted, enabling them to be inherited by related styles or applied to broader product groups where applicable.*

Operational Data

- **Style Reservation Percentage Data:** *The primary input indicating what percentage of a style's warehouse stock should be reserved for a specific sales channel, based on strategic business decisions.*

- **Sales Channel Definitions:** *To accurately categorize and differentiate sales channels (e.g., online retail, physical stores, wholesale) to which a reservation percentage applies.*
- **Active Store Channels:** *To confirm that a specified sales channel has active, operational stores, ensuring that any defined reservation rule is relevant and will be utilized by actual sales outlets.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Default Style Reserve Percentage	0%	The percentage of a new or unclassified style's warehouse stock to automatically hold back for strategic purposes if no specific rule is defined.	When there's a new global policy for how much to reserve for new products or general stock.
Minimum Percentage for Active Reservation	N/A (e.g., 1%)	The smallest percentage value for a reservation rule to be considered active. Rules below this threshold are ignored.	To prevent minor, insignificant reservations from complicating inventory management.
Include New Styles in Default Reservation	True	Determines if newly introduced styles automatically have a default reserve percentage applied, even without specific, explicit rules.	When launching many new products and needing to control initial inventory distribution.
Channel Status for Reservation Validity	N/A (e.g., 'Active', 'Launch')	Specifies which channel statuses (e.g., "Active" for ongoing sales, "Launch" for new market entry) qualify a	When sales channels change their operational status or strategic focus.

Setting Name	Default Value	Description	When to Change
		reservation rule to be applied.	

Note: The actual percentage values for specific styles and channels are defined in the Style Reservation Percentage Data, which acts as a dynamic configuration.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

A[Style Reserve Percentage Data, Product & Channel
Definitions] --> B[Distribution Warehouse Reservation]
B --> C[Applied Warehouse Reservation Rules]

```

1. Start with → The specific Style Reserve Percentage Data and current definitions for products and sales channels.
2. Then → The system loads and filters these reservation percentages, ensuring they apply only to active product styles and relevant sales channels.
3. Next → It establishes these validated rules, creating a map of what percentage of a style's stock should be reserved for each channel.
4. Finally → A clear set of active rules indicating what portion of a style's warehouse stock is conceptually held back for specific purposes, ready to influence future allocation decisions.

5. Step-by-Step Actions

1. **Load** Style Reservation Data:
 - What we do: *We gather all the predefined rules that specify what percentage of a particular style's warehouse stock should be held back for specific sales channels.*
 - Why we do it: *This brings in the core instructions for how to strategically control inventory distribution.*
2. **Filter** for Relevant Styles and Channels:
 - What we do: *We meticulously check each incoming reservation rule to ensure that the associated product style is currently active in our system and that the sales channel it applies to has active, operational stores.*
 - Why we do it: *This ensures that reservations are only created for products and channels that are genuinely live and relevant, preventing unnecessary stock from being held back or misallocated.*
3. **Establish** Channel Reservation Percentages:
 - What we do: *For each valid style and channel combination that passes the filtering, the system precisely records the specified percentage to be reserved.*
 - Why we do it: *This formalizes the specific "holding back" rules that will directly influence subsequent inventory allocation decisions, making sure strategic stock is protected.*

4. **Verify** Active Reservations:

- What we do: *The system confirms that all established reservation percentages are actively recognized and correctly interpreted, ensuring they are ready to influence future distribution logic.*
- Why we do it: *This provides a crucial check, ensuring that the strategic intent of holding back specific inventory for key channels is correctly captured and will be applied throughout the distribution process.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new “Luxury Handbag, Style #LHB001,” is launching exclusively online, with plans for a later physical store rollout.
- The Style Reserve Percentage Data for this new style specifies:
 - Style: LHB001
 - Channel: Online Retail
 - Reserve Percentage: 30%
- The “Luxury Handbag, Style #LHB001” is confirmed as an active new product.
- The “Online Retail” channel is fully operational and has active stores.
- The central warehouse currently holds 500 units of “Luxury Handbag, Style #LHB001.”

What Happens

1. First, the system **loads** the specific Style Reserve Percentage Data for “Luxury Handbag, Style #LHB001,” identifying the instruction to reserve 30% for the “Online Retail” channel.
2. Then, it **verifies** that “Luxury Handbag, Style #LHB001” is indeed an active product style within the broader product catalog.
3. Next, it **confirms** that “Online Retail” is an active sales channel with associated live online stores, making the reservation relevant.
4. Finally, the system **establishes** that for “Luxury Handbag, Style #LHB001” targeted for the “Online Retail” channel, 30% of its warehouse stock should be conceptually reserved. This means 150 units (30% of 500 total units) will be designated as reserved, making only 350 units available for any general allocations to other channels or purposes.

Final Outcome

The system now clearly understands that for “Luxury Handbag, Style #LHB001,” a strategic 30% of its total warehouse stock (150 units in this scenario) is reserved specifically for online sales or future online allocations. This prevents this critical inventory from being prematurely distributed to physical stores or other channels, allowing the business to maintain tight control over the launch and online fulfillment of high-value or new items, directly impacting profitability.

Key Takeaway: This step proactively sets aside critical inventory for specific strategic purposes, preventing its accidental allocation elsewhere and supporting controlled product launches or channel-specific stock strategies.

7. Results

This step produces:

Main Outputs

- **Channel-Specific Style Reservation Map:** A comprehensive, structured mapping indicating which styles have what specific percentage of their warehouse stock designated as reserved for particular sales channels.
- **Validated Reservation Rules:** A definitive set of active and applicable rules, confirming that the specified product styles and sales channels are valid and ready for inventory reservation.

Business Reports

- **Warehouse Reservation Overview:** Used by inventory planners and merchandisers to gain a clear, centralized view of all active reservation rules and their associated styles and channels, providing transparency into strategic inventory holdings.
- **New Style Launch Readiness Report:** Utilized by product managers and sales teams to confirm that appropriate inventory percentages are indeed reserved for new styles, especially crucial for supporting their initial launch channels and preventing stockouts.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available reservation data, or report that essential Style Reserve Percentage Data is missing, potentially leading to no reservations or outdated ones being applied.	Investigate the data source immediately to restore the feed. If critical, manually update the Style Reserve Percentage Data or temporarily delay the distribution run.
A style included in the reservation data is no longer active	The system will ignore the reservation rule for that specific style, as it's not currently recognized as an active product in the system.	Verify the style's status in the product master data. If it should be active, update its definition to ensure rules apply.
A sales channel specified in the	The system will disregard the	Verify the channel's configuration and active store assignments.

Situation	What Happens	What You Should Do
reservation data has no active stores	reservation rule for that channel, as there are no active sales outlets to apply the reservation to.	Ensure relevant stores are correctly flagged as active for that channel.
No Style Reserve Percentage Data is provided	The system will proceed without applying any specific style-level reservations, potentially leading to all warehouse stock being considered available for general allocation.	Review the overall distribution strategy. If strategic reservations are desired, ensure Style Reserve Percentage Data is correctly uploaded and maintained.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *Accurate and up-to-date product style definitions and sales channel configurations, along with initial warehouse inventory counts, all prepped for use.*
- **Master Data Management:** What we need from it: *The clearly defined Style Reserve Percentage Data for all relevant styles and channels, reflecting current business strategy.*

After This Step

- **Distribution Warehouse Inventory Preparation:** What they need from us: *The established reservation rules will be used by this module to conceptually reduce the “available” warehouse inventory before final allocations are made, ensuring that strategic stock is protected.*
- **Allocation Optimization Module:** What they need from us: *The refined understanding of truly available warehouse stock, which is directly influenced by these reservations, to make precise and optimal final allocation decisions that align with business strategy.*

10. Common Questions

Q: Why do we need to reserve warehouse inventory? A: Reserving warehouse inventory ensures that critical stock is held back for specific, strategic purposes, such as supporting new product launches, fulfilling exclusive online sales, or serving particular customer segments. This prevents this valuable stock from being prematurely allocated to general replenishment, thereby protecting key business initiatives and maximizing potential revenue.

Q: How do these reservations impact actual stock levels? A: The reservation process conceptually reduces the “available” stock in the warehouse for general distribution purposes. For example, if you have 100 units of a style in the warehouse and 20% are reserved, then only 80 units are considered “available” for new allocations or transfers, even though all 100 units are physically present. The reserved units are effectively earmarked for specific future uses.

Q: What happens if a reserved style sells out from the reserved portion? A: This module primarily sets the *rule* for reservation. Subsequent modules (like “Distribution Warehouse Inventory Preparation”) will perform the actual *deduction* based on this rule. If the reserved quantity is depleted by direct sales (e.g., online orders) or specific strategic allocations, the remaining available stock will adjust accordingly. New incoming inventory may then top up the reserved amount.

Q: Can the percentage of reserved stock change over time? A: Yes, the Style Reserve Percentage Data is a dynamic configuration input. It can be updated at any time to reflect changing business priorities, new product launch strategies, or shifts in channel focus. For example, a new style might initially have a high reservation percentage to control its launch, and that percentage might be gradually reduced as the style matures and becomes part of regular replenishment.

Q: How does this differ from the “lead time” reservation in the previous module? A: The “lead time” reservation in “Distribution Warehouse Inventory Preparation” is a *dynamic* calculation based on expected sales during the period new stock is in transit. Its purpose is to protect against immediate online stockouts. In contrast, this “Distribution Warehouse Reservation” module sets *fixed percentage rules* for strategic holding of inventory for broader business purposes like new product launches or channel exclusivity, regardless of immediate sales velocity during transit. Both reduce available stock but for different strategic reasons.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To establish and manage rules for holding back a specific percentage of warehouse inventory for particular product styles based on their sales channel.
- **Main Input:** Style Reserve Percentage Data, along with product style definitions and sales channel configurations.
- **Main Output:** A clear, validated set of rules defining what percentage of a style’s warehouse stock should be reserved for specific channels.
- **Warning Signs:** No reservation rules being applied when expected, or rules being incorrectly applied to inactive styles or channels.
- **Key Settings:** Default Style Reserve Percentage, Minimum Percentage for Active Reservation, Include New Styles in Default Reservation, and Channel Status for Reservation Validity.

Need Help? Contact your inventory planning or merchandising team for questions about this process.

Chapter 7: Distribution Compute Revenue

Last Updated: 2025-05-23

1. Business Objective

This module calculates the daily revenue generated by each unique store-style combination and at various aggregated levels (like rolled-up Attribute Groups and national views). It also identifies styles that are 'read-only' based on season lists, preventing them from receiving new allocations, and determines the initial size depth for each store-style combination.

Why This Matters: This crucial data is the foundation for ranking and prioritizing styles in subsequent allocation modules. By providing accurate revenue insights and guiding initial Ideal Size Set (ISS) decisions, it ensures that high-performing products are appropriately stocked, leading to maximized sales, improved profitability, and more efficient inventory management across the retail network.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (Units Sold, Date, SKU, Store, Revenue):** *This processed data, free from distortions like liquidation sales, is the foundation for calculating accurate daily revenue and understanding true customer demand.*
- **Style Live Days:** *Indicates the specific days a product or style was physically available and present in a store, crucial for calculating accurate revenue per day by dividing total revenue by actual selling days.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category, AG):** *Defines each unique item and its classifications (style, category, Attribute Group) which are essential for grouping products and aggregating revenue data at various levels.*
- **Ideal Size Set (ISS) Rules:** *Specifies the preferred mix of sizes for products, guiding how many of each SKU should be stocked to maximize sales and minimize stockouts, forming the basis for initial size depth calculations.*

- **Season Lists (Read and Replenishment):** *Defines which seasons' products are considered 'read-only' (not for active replenishment) versus those eligible for full distribution, guiding how inventory should be prioritized.*

Operational Data

- **In-Scope Stores and Current Inventory Levels (Store & Warehouse):** *Identifies all relevant stores and their current stock, along with central warehouse inventory, to establish the initial pool of available products for distribution.*
- **Store and Style Exclusion/Inclusion Lists:** *Provides specific overrides to either prevent or force the consideration of particular store-style combinations for distribution, allowing for fine-tuned control.*
- **New Store Reference Data:** *Maps newly added stores to established "reference" stores, enabling the system to apply sales patterns and revenue data from known locations to new ones.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Level 1 Sales Analysis Period	N/A (e.g., 90 days)	The number of days of recent sales data to use for the initial, most direct calculation of revenue per day for store-styles.	To adjust the sensitivity of revenue calculation to recent sales performance.
Valid Sales Presence Benchmark	N/A (e.g., 85%)	The minimum percentage of a product's Optimum Depth (OD) or Ideal Size Set (ISS) that must be present for sales to be considered "valid" and contribute to revenue calculation.	To ensure revenue per day reflects true demand under adequate stocking conditions, not missed sales due to poor availability.
Replenishment Season List	N/A	Specifies which seasons' products are considered active for replenishment and allocation.	Before a new season starts or if an existing season needs to be prioritized for active distribution.
Read-Only Season List	N/A	Specifies which seasons'	When styles from older seasons need

Setting Name	Default Value	Description	When to Change
		products are considered 'read-only', meaning they should not receive new allocations.	to be phased out and not actively restocked.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data, Product & Store Info] --> B[Distribution Compute Revenue]

B --> C[Revenue per Day, Size Depth & Read-Only Flags]

1. Start with → Cleaned sales figures, product definitions, store configurations, and season lists.
2. Then → The system generates all relevant store-style combinations, identifies read-only styles, and computes initial stock levels.
3. Next → It calculates revenue per day for store-styles, various Attribute Groups (AGs), and national levels, filling in gaps where direct sales data is missing.
4. Finally → A comprehensive output of daily revenue for products, along with their initial Ideal Size Set (ISS) recommendations and read-only statuses, ready for allocation.

5. Step-by-Step Actions

1. **Generate** Store-Style Combinations and Compute Current Stock: *We create every unique store-style pairing relevant for distribution, calculating their current on-hand inventory from both store and warehouse locations to determine total available stock.*
2. **Determine** Read-Only Styles: *We flag specific styles as 'read-only' for certain stores if they belong to designated past or non-replenishable seasons. This prevents new allocations and ensures strategic inventory control for older or clearance items.*
3. **Calculate** Revenue Per Day (National & In-Store): *We compute the average daily revenue for each style across all stores nationally, and specifically for each store-style combination that has sufficient historical sales data within the defined analysis period.*
4. **Aggregate** Revenue for Attribute Groups: *For store-styles that lack direct sales history, we aggregate sales data across various Attribute Groups (AGs) (e.g., by category, subcategory, brand) to estimate their revenue per day at different levels of granularity.*
5. **Derive** Revenue for Gaps and New Stores: *We use the aggregated AG revenue data to intelligently fill in missing revenue per day for store-styles without direct*

history. Additionally, we replicate revenue from established “reference” stores for newly added locations that are still building their sales data.

6. **Compute Initial Size Depth:** We determine the initial ideal stock quantity for each individual size (SKU) within a style at a given store. This calculation uses Ideal Size Set (ISS) rules and the overall Optimum Depth (OD) to ensure stores receive the most profitable mix of sizes, maximizing sales and minimizing stockouts.

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A new “Women’s Summer Dress, Style #SD001,” available in sizes XS-XL, is being analyzed for distribution.
- Store A is an established, high-volume store; its planogram for dresses suggests an Optimum Depth (OD) of 20 units for this style.
- Ideal Size Set (ISS) rules for women’s dresses in high-volume stores indicate expected sales contributions: XS (10%), S (25%), M (35%), L (20%), XL (10%).
- Store A sold 180 units of this style over the last 90 valid selling days.
- Store B is a brand new store with no sales history for this style. It is configured to reference sales patterns from Store C.
- Style #SD001 belongs to the “Summer2025” season (part of the Replenishment Season List). A similar “Women’s Fall Cardigan, Style #FC002,” is in the “Fall2024” season (Read-Only Season List).

What Happens

1. First, the system **generates** store-style combinations for Style #SD001 with Store A and Store B. For Style #FC002, it also generates combinations with Store A.
2. Next, it **determines** that Style #SD001 is in the “Summer2025” (replenishment) season, so it is *not* marked as ‘read-only’. However, Style #FC002 is in the “Fall2024” (read-only) season, so it **is marked** as ‘read-only’ for Store A, preventing new allocations for future replenishment.
3. Then, for Store A and Style #SD001, the system **calculates** its daily revenue: $180 \text{ units} / 90 \text{ days} = 2.0 \text{ units/day}$. This revenue is assigned to the StoreA-SD001 combination.
4. Since Store B has no sales history for Style #SD001, the system **derives** its expected revenue. It first looks up the revenue per day for Style #SD001 from Store C (its reference store), or, failing that, the aggregated daily revenue for the “Women’s Dress” Attribute Group (AG) at a similar store type. Let’s say it determines 1.0 units/day for StoreB-SD001.
5. Finally, for Store A’s 20-unit Optimum Depth (OD) for Style #SD001, the system **computes** the initial Ideal Size Set (ISS) based on the expected size contributions: XS (2 units), S (5 units), M (7 units), L (4 units), XL (2 units).

Final Outcome

The system now has a complete profile for each store-style combination, including their calculated daily revenue and initial size depth recommendations. For Style #SD001, Store A is prioritized with 2.0 units/day and a specific size breakdown. Store B also receives an estimated revenue and initial size depth, allowing it to receive an initial allocation. Crucially, Style #FC002 is correctly identified as ‘read-only’ for Store A, ensuring it won’t receive new stock and only existing stock is managed.

Key Takeaway: This module provides the essential revenue insights and initial size guidance needed to strategically prioritize and allocate products across the retail network, optimizing sales and inventory efficiency.

7. Results

This step produces:

Main Outputs

- **Revenue Per Day (Store-Style & Aggregated):** *The precise daily revenue for each unique store-style combination, and for various Attribute Groups (AGs) and national views, serving as a primary metric for ranking and prioritization.*
- **Initial Size Depth (per SKU per Store-Style):** *The recommended initial stock quantity for each individual size (SKU) within a style at a specific store, guiding accurate initial allocations and future replenishment based on Ideal Size Set (ISS) rules.*
- **Read-Only Style Flags:** *Clear identification of styles that are not eligible for active replenishment, ensuring that inventory for these items is managed strategically without new allocations.*

Business Reports

- **Style Performance Ranking Report:** Used by merchandisers and planners to visualize the top-performing styles by daily revenue, aiding in strategic stocking decisions and identifying best-sellers.
- **Initial Allocation Blueprint:** Used by allocation teams to understand the proposed initial stock levels and Ideal Size Set (ISS) breakdowns for new product launches or store replenishments, ensuring accurate initial inventory.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential data (e.g., sales, product definitions) is missing, potentially	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides.

Situation	What Happens	What You Should Do
A style belongs to multiple, conflicting seasons (e.g., both 'replenishment' and 'read-only')	leading to incomplete or inaccurate revenue calculations. The system applies a predefined hierarchy (e.g., 'replenishment' status typically overrides 'read-only') but may log a warning.	Review and reconcile conflicting season assignments for the style in the product master data to ensure the desired behavior.
Ideal Size Set (ISS) data is missing for a product category or Attribute Group (AG)	The system may fall back to general averages or apply a default even distribution across sizes, leading to less optimized size allocations than desired.	Ensure all product categories have defined ISS rules. If not, create or update them to improve allocation accuracy.
New store has no suitable reference store or aggregated AG data for revenue derivation	The system may assign a default minimum revenue, potentially leading to minimal initial allocations or requiring manual intervention for proper stocking.	Establish reference store mappings for new stores or ensure comprehensive Attribute Group (AG) data is available for accurate derivation.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All raw master data (products, stores) and transactional data (sales, inventory) to be loaded, validated, and initially formatted.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned and prepared historical sales data, including calculated Rate of Sale (ROS), which is crucial for accurate revenue per day calculations.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels, after accounting for discrepancies and lead time reservations, to accurately compute initial stock.*
- **Distribution Warehouse Reservation:** What we need from it: *Established rules for strategically holding back a percentage of warehouse inventory, which influences the total stock available for general distribution.*

After This Step

- **Allocation Optimization Module:** What they need from us: *The calculated revenue per day and initial size depths to make precise and optimal final*

allocation recommendations, ensuring products go to the stores where they will perform best.

- **Inventory Ranking Module:** What they need from us: *The determined revenue per day metrics to rank and prioritize styles, informing strategic decisions about stock levels and replenishment frequency.*

10. Common Questions

Q: Why is “revenue per day” important for distribution? A: It’s a critical metric that shows how quickly and profitably a product sells. This helps us rank and prioritize styles, ensuring that popular, high-earning items receive more stock and better placement, maximizing overall sales and inventory return.

Q: What does it mean for a style to be “read-only”? A: A “read-only” style is one that will not receive new inventory allocations. This designation is typically given to products from past seasons or those being phased out. We still track their performance, but we don’t actively replenish them, helping manage clearance inventory and reduce excess stock.

Q: How does the “initial size depth” calculation help stores? A: The initial size depth provides a tailored recommendation for how many units of each specific size (SKU) within a style should be stocked when it’s first allocated. This ensures stores receive the right mix of sizes to match customer demand, minimizing stockouts of popular sizes and preventing overstock of less popular ones.

Q: What happens if a style has no historical sales data at a particular store? A: If a specific store-style combination lacks direct sales history (e.g., for new products or new stores), the system intelligently derives its expected revenue per day. It does this by leveraging sales data from similar “reference” stores, or by aggregating performance data from broader Attribute Groups (AGs) like “Women’s Dresses” at a national or regional level.

Q: Why do we calculate revenue at different “aggregated levels” (like Attribute Groups)? A: Calculating revenue at various aggregated levels (e.g., category, subcategory, brand) allows the system to make informed distribution decisions even when granular store-style data is unavailable. This ensures that even new or low-volume styles can be allocated effectively by learning from the sales patterns of similar product groups, preventing missed sales opportunities.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate revenue per day for store-styles and aggregated levels, identify ‘read-only’ styles, and determine initial size depths.
- **Main Input:** Cleaned sales data, product definitions (SKU, Style, AG), ISS rules, season lists, and current inventory.
- **Main Output:** Revenue per Day for each store-style, initial Ideal Size Set (ISS) recommendations, and ‘read-only’ style flags.
- **Warning Signs:** Unexpectedly low or high revenue per day for established styles, a large number of styles incorrectly flagged as ‘read-only’, or illogical initial size depth recommendations.

- **Key Settings:** Level 1 Sales Analysis Period, Valid Sales Presence Benchmark, Replenishment Season List, Read-Only Season List.
-

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 8: Distribution Ranking

Last Updated: 2025-05-23

1. Business Objective

This module ranks store-style combinations and individual SKU/category-size combinations based on their calculated revenue per day and other relevant metrics like current inventory levels and Product Selling Area (PSA).

Why This Matters: This crucial step determines the priority for all subsequent inventory allocation decisions. By clearly identifying which product-store combinations are the most valuable and fastest-selling, we ensure that higher-priority items are considered first for stock, leading to maximized sales, reduced stockouts of popular items, and more efficient inventory management.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Calculated Revenue per Day (Store-Style & Aggregated):** *Provides the primary measure of how much daily revenue a product generates at a specific store or across aggregated groups, which is fundamental for ranking its importance.*
- **Cleaned Sales Data (Units Sold, Revenue):** *The processed historical sales, free from distortions, used to support detailed SKU and category-size level ranking calculations.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category, AG):** *Defines each unique item and its classifications (style, category, Attribute Group), enabling the system to understand product relationships and group them for ranking.*
- **Ideal Size Set (ISS) and Optimum Depth (OD) Rules:** *While not directly ranked, these rules contribute to the revenue per day calculations and define ideal stock levels, influencing the underlying metrics used for ranking.*

Operational Data

- **Current Inventory Levels (Store & Warehouse):** *The up-to-date quantity of each product currently held in stores and distribution warehouses, used to evaluate the overall stock position of items being ranked.*
- **Product Selling Area (PSA) Data:** *Information about how efficiently a product is displayed or utilized in its selling space, providing another key metric for determining its overall ranking priority.*
- **Store-Style Combinations:** *The complete list of all active product-store pairings, forming the basis for the primary ranking process.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Revenue Metric Weightages	N/A	Defines how different levels of revenue (e.g., direct store-style sales, aggregated category sales, national style sales) are combined into the final revenue per day for ranking.	When strategic focus shifts, e.g., to prioritize direct store performance over broader averages.
Minimum PSA for High Rank	N/A (e.g., 0.8)	The minimum percentage of allocated display space (PSA) an item must achieve to be considered for a higher ranking priority.	To align ranking with visual merchandising standards or store display requirements.
Inventory Impact on Ranking	True	Determines whether the current total stock of a store-style combination influences its overall rank. Higher stock indicates a more established or important position.	To adjust how existing inventory levels affect an item's perceived importance for strategic management.
	6 months	The historical duration over	To make SKU/Category rankings

Setting Name	Default Value	Description	When to Change
Sales Period for SKU/Category Ranking		which sales data is analyzed to calculate the Rate of Sale (ROS) for granular SKU and Category-Size rankings.	more sensitive to recent trends or to provide a more stable, averaged view.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Calculated Revenue per Day, Inventory Levels, PSA Data] -->
    B[Distribution Ranking Logic]
    B --> C[Prioritized Distribution Lists for Allocation]
  
```

1. Start with → Calculated daily revenue for store-styles, current inventory levels, and Product Selling Area (PSA) data.
2. Then → The system applies a hierarchical ranking methodology to identify the most valuable store-style combinations.
3. Next → It performs more granular ranking for individual SKUs and category-size combinations within stores.
4. Finally → A series of clearly prioritized lists of items, ready to guide optimal allocation decisions across the retail network.

5. Step-by-Step Actions

1. **Consolidate** Store-Style and SKU/Category-Size Data:
 - What we do: *We gather all relevant metrics, such as calculated revenue per day, current inventory levels, and Product Selling Area (PSA) percentages, for every unique store-style, store-SKU, and store-category-size combination.*
 - Why we do it: *This creates a comprehensive dataset for each item, providing all the necessary information to objectively compare and rank them.*
2. **Apply** Store-Style Ranking Criteria:
 - What we do: *We sort all store-style combinations based on a weighted hierarchy of metrics. The most important criteria typically include the calculated revenue per day for that specific store-style, followed by the national revenue generated by the style, its Product Selling Area (PSA) performance, and its current total stock. This process identifies the most critical store-style pairings.*
 - Why we do it: *This ensures that our most profitable and strategically important items, even those with healthy current stock, are recognized and prioritized for continuous attention and efficient allocation.*

3. **Compute** Store-SKU Rankings:

- What we do: *We perform a more granular ranking for individual SKUs (specific sizes within a style) at each store. This ranking is primarily driven by their specific revenue per day and leverages the overall store-style rank as a tie-breaker for items with similar performance.*
- Why we do it: *This allows for precise allocation decisions at the size level, ensuring that the most popular sizes are always available and properly prioritized during replenishment.*

4. **Determine** Store-Category-Size Rankings:

- What we do: *We rank combinations of stores, broader product categories, and specific sizes based on their historical revenue per day over a defined period (e.g., the last six months). This provides a more aggregated view for general category replenishment.*
- Why we do it: *This helps make informed allocation decisions for broader product groups or when detailed style-level data is less robust, ensuring that common sizes within high-performing categories are well-stocked.*

5. **Assign** Priority Ranks:

- What we do: *After applying all sorting criteria, the system assigns a numerical rank to each combination, with “1” representing the highest priority, “2” the next, and so on.*
- Why we do it: *This creates a clear, actionable prioritization list that simplifies decision-making for subsequent allocation modules, directing inventory to where it will have the greatest impact.*

6. **Generate** Denormalized Output for Reporting:

- What we do: *We create a comprehensive report containing each ranked combination along with all the metrics (revenue, PSA, inventory) that contributed to its rank.*
- Why we do it: *This provides a transparent and detailed view of the ranking process, allowing merchandisers and planners to audit, analyze, and understand the rationale behind the prioritization, supporting strategic business decisions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Store A has two key styles: “Running Shoe, Style #ABC001” and “Casual Sneaker, Style #XYZ002.”
- “Running Shoe, Style #ABC001” (Store A): Revenue per Day = \$200, National Revenue = \$150/day, PSA = 90%, Total Stock = 50 units.
- “Casual Sneaker, Style #XYZ002” (Store A): Revenue per Day = \$180, National Revenue = \$200/day, PSA = 85%, Total Stock = 30 units.
- Within “Running Shoe, Style #ABC001,” Size 9 (SKU #ABC001-9) has a daily revenue of \$70, while Size 10 (SKU #ABC001-10) has \$60.

What Happens

1. First, the system **compares** “Running Shoe, Style #ABC001” and “Casual Sneaker, Style #XYZ002” at Store A using the ranking criteria.

2. Then, it prioritizes based on their calculated revenue per day. The “Running Shoe” has a higher store-level revenue (\$200 vs \$180).
3. Next, it considers national revenue: The “Casual Sneaker” has a higher national revenue (\$200 vs \$150).
4. Following this, it looks at PSA (90% for “Running Shoe” vs 85% for “Casual Sneaker”) and then total stock (50 units for “Running Shoe” vs 30 units for “Casual Sneaker”).
5. After applying the weighted ranking logic, the system **determines** that “Running Shoe, Style #ABC001” is ranked higher due to its strong store-level revenue and PSA, despite the “Casual Sneaker” having higher national revenue. So, “Running Shoe” gets Rank 1, “Casual Sneaker” gets Rank 2.
6. Finally, for the specific SKUs within “Running Shoe,” Size 9 (\$70/day) is **ranked higher** than Size 10 (\$60/day).

Final Outcome

The system produces a clear priority list: “Running Shoe, Style #ABC001” at Store A is the top-ranked combination, meaning it will receive primary attention during allocation. Within that style, Size 9 is identified as the most critical SKU. This ensures that valuable inventory is preferentially directed to the most profitable products and their most popular sizes, directly boosting sales and customer satisfaction.

Key Takeaway: By systematically evaluating multiple performance metrics, the ranking module creates a clear hierarchy of importance, guiding inventory to the most impactful product-store combinations.

7. Results

This step produces:

Main Outputs

- **Ranked Store-Style List:** A prioritized list of all store-style combinations, ordered from highest to lowest priority for allocation, indicating their overall business impact.
- **Ranked Store-SKU List:** A detailed prioritization of individual SKUs (specific sizes) within each store, ensuring granular allocation decisions are optimized.
- **Ranked Store-Category-Size List:** A broader prioritization of product categories by size at each store, useful for high-level allocation strategies or when detailed style data is limited.

Business Reports

- **Allocation Priority Report:** Used by allocation managers to view the top-ranked store-styles and SKUs, informing where to direct available inventory for maximum impact.
- **Performance Ranking Dashboard:** Used by merchandisers and planners to visualize and analyze the performance hierarchy of products across the retail network, aiding in strategic planning.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential data (e.g., revenue, inventory) is missing, potentially leading to incomplete or inaccurate rankings.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides.
Multiple items have identical ranking metrics	The system will use predefined tie-breaking rules (e.g., by internal Store ID or Style ID) which might not always align with business intent.	Review the tie-breaking logic if non-optimal outcomes occur, or adjust input metrics to create clearer distinctions.
New products or new stores with no sales history	These items may rank lower by default, or be excluded from higher-level rankings until sufficient data is available.	Ensure new products or stores are mapped to suitable reference data points in prior modules or consider initial manual allocations.
Ranking logic yields counter-intuitive priorities	If, for example, a high-stock, low-performing item ranks unexpectedly high due to certain weighting, it can misdirect allocations.	Review and adjust the “Revenue Metric Weightages” or “Inventory Impact on Ranking” settings to align with strategic business goals.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All core master data (products, stores) and transactional data (sales, inventory) loaded and prepared.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned sales data and accurate Rate of Sale (ROS) calculations, which are foundational for revenue per day metrics.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels, after accounting for discrepancies and lead time reservations.*
- **Distribution Warehouse Reservation:** What we need from it: *Rules for strategically holding back a percentage of warehouse inventory, influencing the total stock considered available for general distribution.*

- **Distribution Compute Revenue:** What we need from it: *The calculated revenue per day for store-styles and aggregated levels, and initial size depths, which are the primary inputs for this ranking module.*

After This Step

- **Allocation Optimization Module:** What they need from us: *The highly prioritized lists of store-styles and SKUs to make precise and optimal final allocation recommendations, ensuring valuable stock goes where it will drive the most sales.*
- **Transfer Calculation Engine:** What they need from us: *The ranked data to identify optimal quantities of products to move between warehouses or to stores, guided by the most critical inventory needs.*

10. Common Questions

Q: How is “priority” determined in the ranking? A: Priority is determined by evaluating several key metrics in a specific order: first, the actual daily revenue a product generates at a particular store; then, its overall national revenue performance; next, how efficiently it uses its selling space (PSA); and finally, its current inventory level. Items with higher values in these critical areas are ranked higher.

Q: Why do we rank at different levels, like store-style versus individual SKU? A: Ranking at different levels allows for comprehensive and precise allocation. Store-style ranking provides a strategic overview of which product styles are most important for each store, while individual SKU (size) ranking ensures that within those styles, the exact sizes that sell fastest are prioritized for replenishment. This balances strategic focus with granular operational needs.

Q: How does current inventory affect an item’s ranking? A: In this ranking, current inventory levels contribute to an item’s overall importance score. A higher stock level for a high-performing style means it’s a significant inventory position that needs careful management, potentially indicating its established importance. This rank helps ensure that these top-tier, high-stock items remain a focus, even if they aren’t immediately low on stock for replenishment.

Q: What happens if two items have the exact same performance metrics? A: If two items have identical performance across all primary ranking metrics, the system uses tie-breaking rules. These are typically based on internal identifiers like the Store ID or Style ID to ensure a consistent, albeit arbitrary, order. In such cases, the business impact of their specific rank order is usually minimal.

Q: How are new products or new stores ranked without historical data? A: New products or stores typically won’t have direct historical sales data to draw upon. The system intelligently addresses this by using derived revenue figures from similar “reference” stores or by leveraging aggregated sales performance from broader Attribute Groups (AGs) that the new product belongs to. This allows them to receive an initial ranking and allocation based on expected performance.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To systematically prioritize store-style and SKU/category-size combinations based on their business value and performance.
- **Main Input:** Calculated revenue per day, current inventory levels, and Product Selling Area (PSA) data.
- **Main Output:** Prioritized lists of store-styles and SKUs, ready to guide precise allocation decisions.
- **Warning Signs:** Counter-intuitive rankings (e.g., slow-moving items ranking high), or an inability to rank new products or stores.
- **Key Settings:** Revenue Metric Weightages, Minimum PSA for High Rank, and Inventory Impact on Ranking.

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 9: Distribution Segmentation

Last Updated: 2025-05-23

1. Business Objective

This module categorizes store-style combinations into specific segments like “TOP_SELLER”, “BOTTOM_SELLER”, or “NORMAL_SELLER” based on their sales performance and stock characteristics.

Why This Matters: Identifying these segments is crucial for applying differentiated business rules in the allocation and replenishment process. It enables us to strategically prioritize high-performing products for optimal stocking, while also efficiently managing and pulling back inventory from underperforming items, directly impacting sales maximization and inventory efficiency.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (Units Sold, Revenue, Date, SKU, Store):** *This processed transaction data, free from distortions like liquidation sales, forms the foundation for calculating accurate daily revenue and understanding true customer demand.*
- **Product Live Days (Store & Style Level):** *Indicates the specific days a product or style was physically available and present in a store, which is essential for calculating accurate Rate of Sale (ROS) and revenue per day.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category):** *Details each unique item (SKU) and its classifications (style, category), which are essential for grouping products and analyzing performance at various levels for segmentation.*
- **Pullback Configuration Rules:** *Specific business rules and benchmarks defined for product categories, used to determine if an item qualifies as a “BOTTOM_SELLER” for potential inventory pullbacks.*

Operational Data

- **Store-Style Combinations and Current Store Inventory:** *The complete list of all active product-store pairings and their current on-hand stock, providing context for sales performance and inventory efficiency.*
- **Warehouse-Store Category Map:** *Information mapping warehouses to specific stores and product categories they serve, allowing the system to accurately account for warehouse stock when calculating sell-through rates for stores.*
- **In-Transit Inventory:** *Details on products currently moving between warehouses or to stores, which is included to provide a complete picture of total available stock for accurate sell-through calculations.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Top Seller Revenue Multiplier	N/A (e.g., 1.2)	A factor that determines the minimum daily revenue a style must achieve, relative to its category or store average, to be considered a “TOP_SELLER.”	To adjust the threshold for identifying top performers based on business performance goals.
Minimum Live Days for Top Seller	N/A (e.g., 20)	The minimum number of days a style must have been actively available in a store to be considered for “TOP_SELLER” status.	To ensure top performer status is based on sufficient selling history, not just brief spikes.
Minimum Product Age for Bottom Seller Analysis	N/A (e.g., 30 days)	The minimum number of days a product must have been available in a store before it can be considered a “BOTTOM_SELLER.”	To prevent new products from being prematurely flagged as underperformers.
Revenue Per Day Benchmark for Bottom Seller	N/A (e.g., 0.5)	A factor that determines the maximum daily revenue a style can achieve, relative to its store category average, to be considered a “BOTTOM_SELLER.”	To adjust the threshold for identifying underperformers based on profitability targets.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Cleaned Sales Data, Product & Store Info, Pullback Rules] --> B[Distribution Segmentation Logic]
B --> C[Store-Style Combinations with Segment Assignment]
```

1. Start with → Cleaned sales figures, product definitions, store configurations, and category-specific pullback rules.
2. Then → The system computes aggregated revenue benchmarks for various levels (e.g., channel, store category).
3. Next → It identifies “TOP_SELLER” store-style combinations based on style performance relative to these benchmarks and minimum live days.
4. Then → It calculates Rate of Sale (ROS), discount percentages, and sell-through rates for underperforming styles, considering total available stock.
5. Next → It identifies “BOTTOM_SELLER” store-style combinations by applying a set of criteria related to sales quantity, revenue per day, discount levels, and sell-through rates.
6. Finally → A refined dataset of store-style combinations, each explicitly segmented as “TOP_SELLER,” “BOTTOM_SELLER,” or “NORMAL_SELLER,” ready to guide allocation rules.

5. Step-by-Step Actions

1. **Compute** Aggregated Revenue Benchmarks:
 - What we do: *We calculate the average daily revenue for styles at broad levels, such as across a sales channel for a specific product category, and within individual stores for their categories.*
 - Why we do it: *These aggregated values serve as performance benchmarks against which individual store-style combinations are compared to identify top performers.*
2. **Identify** TOP_SELLER Styles:
 - What we do: *We flag store-style combinations as “TOP_SELLER” if their individual daily revenue exceeds both the store’s category average (adjusted by a multiplier) and the broader channel-category average, and if they have been active for a sufficient number of days.*
 - Why we do it: *This ensures that our highest-performing and most profitable products are recognized and prioritized for optimal inventory allocation.*
3. **Calculate** Performance Metrics for Bottom Seller Analysis:
 - What we do: *We determine the Rate of Sale (ROS) for individual SKUs, calculate overall style discount percentages, and compute the sell-through rate for store-styles by considering all available stock (store and warehouse).*
 - Why we do it: *These metrics provide a comprehensive view of how efficiently an item is selling and inform whether it’s an underperformer.*
4. **Identify** BOTTOM_SELLER Styles:
 - What we do: *We flag store-style combinations as “BOTTOM_SELLER” if they have been in the store for a minimum age and meet several criteria: low sales quantity, low daily revenue (relative to category average), high*

discount levels, and poor sell-through rates, while also not being a “TOP_SELLER.”

- Why we do it: *This identifies underperforming products that may require strategic intervention, such as inventory pullbacks or reduced future allocations, to free up capital and space.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A “Women's Denim Jeans, Style #DNM001” is sold at Store A.
- Store A's Women's Bottoms category (which includes jeans) has an average daily revenue of \$200.
- The “Online Retail” channel's Women's Bottoms category has an average daily revenue of \$180.
- Top Seller Revenue Multiplier is set to 1.2.
- Minimum Live Days for Top Seller is set to 20 days.
- Minimum Product Age for Bottom Seller Analysis is set to 30 days.
- Revenue Per Day Benchmark for Bottom Seller is 0.5.
- Pullback Configuration Rule for “Women's Bottoms” specifies:
salesQtyBenchmark = 15, discountBenchmark = 0.8,
sellthroughMultiplier = 0.7.

What Happens

Scenario 1: Identifying a TOP_SELLER 1. Style #DNM001 in Store A has been live for 45 days. Its daily revenue is calculated as \$250. 2. The system checks for “TOP_SELLER” status: - Is \$250 (Style #DNM001 revenue) > (\$200 (Store A category revenue) * 1.2)? (\$250 > \$240) - **YES**. - Is 45 days (Style #DNM001 live days) >= 20 days (Minimum Live Days for Top Seller)? - **YES**. - Is \$250 (Style #DNM001 revenue) > \$180 (Channel category revenue)? - **YES**. 3. Since all conditions are met, “Women's Denim Jeans, Style #DNM001” in Store A is segmented as **TOP_SELLER**.

Scenario 2: Identifying a BOTTOM_SELLER (different style/store) 1. A “Men's Casual Shirt, Style #CSH002” in Store B has been live for 60 days. Its daily revenue is \$50. Store B's Men's Tops category average daily revenue is \$120. 2. Total Units Sold for #CSH002 in Store B over the analysis period is 10 units. Store B's Men's Tops category average discount is 8%. Style #CSH002's discount is 10%. 3. The system first confirms #CSH002 is not a TOP_SELLER. 4. Next, it checks Minimum Product Age: 60 days >= 30 days - **YES**. 5. It checks Sales Criteria: 10 units is less than salesQtyBenchmark (15 units) - **YES**. (This means low sales, condition for bottom seller met). 6. It checks Style Revenue Per Day: Is \$50 (Style #CSH002 revenue) < (\$120 (Store B category revenue) * 0.5)? (\$50 < \$60) - **YES**. 7. It checks Discount: Is 10% (Style #CSH002 discount) >= (8% (Store B category discount) * 0.8)? (10% >= 6.4%) - **YES**. (High discount, condition for bottom seller met). 8. It calculates Sell-Through: Let's say Style #CSH002's sell-through is 0.05, and Store B's Men's Tops category sell-through is

0.15. Is $0.05 < (0.15 * 0.7)$? ($0.05 < 0.105$) - **YES**. 9. Since all conditions are met, “Men’s Casual Shirt, Style #CSH002” in Store B is **segmented as BOTTOM_SELLER**.

Final Outcome

The system accurately identifies “Women’s Denim Jeans, Style #DNM001” at Store A as a “TOP_SELLER” and “Men’s Casual Shirt, Style #CSH002” at Store B as a “BOTTOM_SELLER.” This segmentation provides clear directives for future inventory decisions: prioritize stocking for the jeans, and consider reducing or pulling back stock for the shirt.

Key Takeaway: This module provides actionable intelligence by automatically classifying products based on their true performance, enabling highly targeted and efficient inventory strategies.

7. Results

This step produces:

Main Outputs

- **Store-Style Segment Assignments:** Each unique store-style combination is explicitly tagged as “TOP_SELLER,” “BOTTOM_SELLER,” or “NORMAL_SELLER,” providing a clear classification for subsequent allocation logic.
- **Remarks for Segmented Styles:** Additional notes or reasons explaining why a specific style was assigned to a “BOTTOM_SELLER” segment, aiding in analysis and decision-making for pullbacks.

Business Reports

- **Product Performance Segmentation Report:** Used by merchandisers and planners to review the categorized performance of all store-style combinations, allowing for strategic planning regarding stock levels and promotions.
- **Potential Pullback List:** Used by inventory managers and operations teams to identify products and stores from which inventory might be pulled back or transferred to more productive locations.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential sales or inventory data is missing, potentially leading to incomplete	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides based on business judgment.

Situation	What Happens	What You Should Do
A product has no sales data or very low live days	or inaccurate segmentation. The system may not be able to accurately segment the product, potentially defaulting it to “NORMAL_SELLER” or excluding it from specific segment analysis.	Review if the product is new or has data capture issues. Consider if initial allocations should be based on category averages or manual judgment for new items.
Conflicting business rules for top/bottom sellers	While the system has built-in logic to prevent an item from being both “TOP_SELLER” and “BOTTOM_SELLER,” extreme settings could lead to unexpected behavior.	Review and adjust the “Top Seller Revenue Multiplier” or “Revenue Per Day Benchmark for Bottom Seller” settings to ensure logical segmentation.
Widespread discounting makes all products appear as “BOTTOM_SELLER” candidates	If many products have high discount rates, the “Discount Benchmark for Bottom Seller” might flag too many items as underperformers.	Adjust the “Discount Benchmark for Bottom Seller” or other Pullback Configuration Rules to align with current promotional strategies and prevent over-segmentation.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All raw master data (products, stores) and transactional data (sales, inventory) loaded, validated, and initially formatted.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned and prepared historical sales data, including accurate Rate of Sale (ROS) calculations, which are fundamental for performance analysis.*
- **Distribution Compute Revenue:** What we need from it: *The calculated revenue per day for store-styles and aggregated levels, providing the primary metric for top seller identification.*

After This Step

- **Allocation Optimization Module:** What they need from us: *The assigned store-style segments (TOP_SELLER, BOTTOM_SELLER, NORMAL_SELLER) to apply differentiated allocation rules, prioritizing stock for top performers and minimizing it for bottom performers.*

- **Transfer Calculation Engine:** What they need from us: *Specifically, the “BOTTOM_SELLER” segment information to identify eligible inventory for strategic pullbacks or transfers to other, higher-performing locations.*

10. Common Questions

Q: Why is it important to categorize products into “TOP_SELLER” or “BOTTOM_SELLER” segments? A: Categorizing products allows us to apply smart, differentiated inventory strategies. For “TOP_SELLERS,” we ensure they are always well-stocked to maximize sales. For “BOTTOM_SELLERS,” we can actively reduce their stock or transfer them elsewhere, freeing up capital and valuable shelf space, which directly improves overall profitability and store efficiency.

Q: How are “TOP_SELLER” products identified? A: “TOP_SELLER” products are identified by comparing their actual daily revenue at a specific store against the average daily revenue of similar products in that store’s category and across the entire sales channel. If a product significantly outperforms these benchmarks and has been available long enough to prove its consistency, it’s flagged as a top performer.

Q: What criteria are used to identify “BOTTOM_SELLER” products? A: “BOTTOM_SELLER” products are identified using a combination of criteria for underperformance. This includes having low sales quantities, generating significantly less daily revenue compared to its category average, requiring high discounts to sell, and demonstrating a poor sell-through rate, all while not being a “TOP_SELLER” and having been in the store for a sufficient period.

Q: What happens to products that are neither “TOP_SELLER” nor “BOTTOM_SELLER”? A: Products that don’t meet the criteria for either “TOP_SELLER” or “BOTTOM_SELLER” are classified as “NORMAL_SELLER.” These items will receive standard allocation and replenishment rules, allowing us to focus our more aggressive inventory strategies on the extremes of performance.

Q: How does this segmentation help with inventory pullbacks? A: This segmentation is critical for pullbacks. By clearly identifying “BOTTOM_SELLER” products, we can initiate targeted actions to remove or transfer excess inventory from those stores. This prevents dead stock from accumulating, reduces markdown risks, and allows us to reallocate valuable products to stores where they have a better chance of selling, improving overall inventory health and profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To categorize store-style combinations into “TOP_SELLER,” “BOTTOM_SELLER,” or “NORMAL_SELLER” segments for differentiated inventory management.
- **Main Input:** Cleaned sales data, product definitions (SKU, Style, Category), product live days, and pullback configuration rules.

- **Main Output:** Store-style combinations with assigned performance segments and associated remarks.
 - **Warning Signs:** Unexpectedly high or low numbers of products in “TOP_SELLER” or “BOTTOM_SELLER” segments, or illogical segmentation based on known sales performance.
 - **Key Settings:** Top Seller Revenue Multiplier, Minimum Live Days for Top Seller, Minimum Product Age for Bottom Seller Analysis, Revenue Per Day Benchmark for Bottom Seller.
-

Need Help? Contact your merchandising or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 10: Distribution Allocation

Last Updated: 2025-05-23

1. Business Objective

This module is the central engine for deciding exactly how much of each product to send to every store, orchestrating different allocation strategies to maximize sales and minimize stockouts.

Why This Matters: This core process ensures that valuable inventory is always placed in the right store at the right time, preventing missed sales opportunities due to empty shelves and reducing wasteful overstock, directly impacting profitability and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data:** *Provides a true picture of customer demand, free from distortions like clearance events, essential for making smart allocation decisions.*
- **Rate of Sale (ROS) per SKU per Store:** *Shows how quickly each product is selling at each store, guiding how much to send for optimal replenishment.*
- **Calculated Revenue per Day:** *Indicates the profitability of each product at each store, helping prioritize allocations to high-performing items.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category, AG):** *Details every unique item and its characteristics, allowing the system to group and allocate products based on their specific features and Attribute Groups.*
- **Optimum Depth (OD) Rules:** *Specifies the ideal target quantity of each product to stock in a store, guiding the overall inventory level for allocation.*
- **Ideal Size Set (ISS) Rules:** *Defines the best mix of sizes for each product to maximize sales, ensuring the right sizes are allocated to meet customer preferences.*
- **Optimum Width (OW) Plan Data:** *Indicates the ideal number of different categories or styles a store should offer, influencing broad assortment planning during allocation.*

Operational Data

- **Adjusted Warehouse Inventory Levels:** *Provides the most accurate count of truly available stock in warehouses, accounting for discrepancies and strategic reservations, determining what can actually be allocated.*
- **Current Store Inventory Levels:** *Shows what products are already in stores, allowing allocations to top up existing stock rather than over-sending.*
- **Store-Style Segment Assignments (TOP_SELLER, BOTTOM_SELLER):** *Classifies products by their performance in each store, allowing the system to apply differentiated allocation rules for best-sellers versus slow-movers.*
- **Planogram Configurations and Product Selling Area (PSA) Data:** *Details the physical display capacity of stores and how efficiently products use that space, influencing how much stock can realistically be placed and replenished.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Allocation Strategy Type	REPLENISHMENT	Defines the overall approach for distribution, such as standard replenishment, fresh allocation for new products, or strategic in-store transfers.	When a new product launch, seasonal shift, or operational objective requires a different allocation logic.
Deplete Warehouse Stock Strategy	False	Determines if the system should allocate all available warehouse stock until it's fully depleted, even if it exceeds initial Optimum Depth targets.	When needing to clear out warehouse inventory aggressively, perhaps at end of season or for specific styles.
Planogram Increment Strategy	False	Enables the system to increase allocations beyond initial planogram targets if excess warehouse stock is available.	When stores have additional capacity and the business wants to maximize stock on display for high-performing items.
	False		

Setting Name	Default Value	Description	When to Change
Depth Increment Strategy		Allows the system to allocate additional units beyond the Optimum Depth (OD) to build deeper stock levels in stores.	When building stock for peak seasons, promotions, or to ensure robust inventory for top-selling items.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Prepared Sales, Product & Inventory Data] --> B[Distribution Allocation Engine]
    B --> C[Optimal SKU Allocations per Store]
  
```

1. Start with → All prepared data, including cleaned sales, current inventory, OD, OW, ISS rules, and product performance rankings.
2. Then → The system orchestrates multiple allocation iterations (e.g., replenishment, fresh allocation, depth increment, planogram adjustments, pullbacks) based on the chosen strategy.
3. Next → For each iteration, it applies complex business rules and constraints (OD, OW, ISS, ROS, AG) to determine the optimal quantity of each SKU for every store.
4. Finally → A precise list of how many of each unique product (SKU) should be sent to each store to maximize sales and minimize stockouts.

5. Step-by-Step Actions

1. Initialize Allocation Run:

- What we do: *The system sets up the environment based on the chosen allocation strategy (e.g., replenishment, fresh allocation, in-store transfer), loading all necessary data and business rules.*
- Why we do it: *This ensures the allocation process is tailored to the specific business objective for the current run.*

2. Execute Core Allocation Iterations:

- What we do: *The system runs a series of allocation cycles, each focusing on a specific business rule or constraint. This includes initial replenishment, allocating based on Optimum Depth (OD), ensuring Ideal Size Set (ISS) adherence, considering Rate of Sale (ROS), and adjusting for Attribute Group (AG) performance.*
- Why we do it: *This multi-layered approach ensures that allocations are balanced, addressing immediate replenishment needs while also optimizing for sales potential and display efficiency.*

3. **Adjust** for Planogram and Depth Increments:

- What we do: *If enabled, the system revisits allocations to fill additional planogram space or build deeper stock levels beyond initial targets, leveraging available warehouse inventory.*
- Why we do it: *This allows for dynamic adjustments to maximize product presence for high-demand items or during strategic periods, driving higher sales.*

4. **Manage** Pullbacks and Strategic Transfers:

- What we do: *For underperforming products (BOTTOM_SELLERS) or as part of an In-Store Transfer (IST) strategy, the system identifies excess stock in stores and determines quantities to be pulled back to the warehouse or transferred to other stores.*
- Why we do it: *This optimizes overall inventory health by reallocating slow-moving stock to more productive locations or centralizing it for clearance, minimizing losses.*

5. **Finalize** Allocations and Generate Outputs:

- What we do: *The system consolidates all calculated allocations for each SKU at each store, adjusts for any Just-in-Time (JIT) overrides, and generates detailed reports on planned inventory movements and key performance indicators.*
- Why we do it: *This provides a comprehensive, actionable plan for warehouse operations and store receiving, ensuring that the right products arrive at the right places.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A central warehouse has 1,000 units of “Summer T-Shirt, Style #TS001,” available in 5 sizes (XS, S, M, L, XL).
- Store A is a “TOP_SELLER” for this style, with a calculated Rate of Sale (ROS) of 10 units/week. Its Optimum Depth (OD) for this style is 50 units. Ideal Size Set (ISS) rules prioritize M and L sizes (60% of sales). Store A's planogram allows 40 units for this style.
- Store B is a “NORMAL_SELLER” with an ROS of 3 units/week. Its OD is 20 units. Store B currently has 15 units of #TS001.
- A “BOTTOM_SELLER” winter jacket (Style #WJ002) is currently in Store C with 20 units.

What Happens

1. First, the system initiates a “REPLENISHMENT” run. It identifies Store A as a “TOP_SELLER” for Style #TS001, so it prioritizes its allocation. Based on its OD (50 units) and ROS (10 units/week), and using the ISS rules, it determines Store A needs 30 M units, 20 L units, and 10 units across XS, S, XL to reach its 50-unit OD and optimal size mix. This is 60 units total.
2. Then, it looks at Store B. It has 15 units of Style #TS001 and an OD of 20 units, so it needs 5 more units. The system allocates 5 units, ensuring the right size mix based on its ROS.

3. Next, with the “Deplete Warehouse Stock Strategy” and “Planogram Increment Strategy” enabled, and if there’s still ample stock of #TS001 in the warehouse, the system revisits Store A. Since Store A’s planogram allows 40 units and its initial allocation was 50 units (exceeding initial planogram due to OD), the system checks for additional display capacity. If Store A’s total display capacity for summer items could accommodate more, the system might allocate a few more units of the T-shirt beyond the 50-unit OD, up to a practical limit or until warehouse stock is depleted.
4. Finally, for Style #WJ002 at Store C, the system, recognizing it as a “BOTTOM_SELLER,” might recommend pulling back 10 units to the central warehouse for consolidation or markdown, rather than sending more stock.

Final Outcome

Store A receives 60 units of “Summer T-Shirt, Style #TS001” with a precise Ideal Size Set, ensuring its shelves are fully stocked with the most popular sizes, maximizing sales. Store B receives its required 5 units. The system intelligently allocates remaining warehouse stock, potentially allowing Store A to carry even more of its top-selling items. Simultaneously, excess stock of the underperforming winter jacket at Store C is identified for pullback, preventing losses and freeing up space.

Key Takeaway: This module precisely matches inventory to store demand and display capacity across all products, driving sales for top performers and managing excess stock for underperformers.

7. Results

This step produces:

Main Outputs

- **Optimal SKU Allocations per Store:** The final, precise quantity of each unique product (SKU) recommended for allocation to every store, reflecting all business rules and constraints.
- **Planned Inventory Movements:** Detailed records of products to be shipped from warehouses to stores, or pulled back/transferred between stores, enabling logistics.
- **Allocation Details and Remarks:** Comprehensive information on how each allocation was determined, including which iteration it was part of and any specific business rule applications.

Business Reports

- **Store-Specific Allocation Report:** Used by store managers and merchandisers to view incoming inventory, prepare for receipts, and understand the planned stock levels for their location.
- **Warehouse Pick List:** Used by warehouse operations to efficiently pick and pack products for outbound shipments based on the approved allocation plan.

- **Allocation Performance Dashboard:** Used by inventory planners and executives to monitor the effectiveness of allocation strategies, track stock movements, and evaluate the impact on sales and inventory health.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data or report that essential data (e.g., inventory, sales) is missing, potentially leading to incomplete or inaccurate allocations.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides.
Warehouse stock is insufficient to meet demand	The system will prioritize allocations based on product and store ranking, fulfilling as much as possible but leaving some demand unfulfilled.	Review future inbound shipments and consider adjusting OD targets or running a follow-up allocation once stock arrives.
Store has no available planogram space for new items	The system will adhere to planogram constraints, allocating only up to the available capacity, or marking the allocation for review.	Review the store's physical layout, consider adjusting planograms, or evaluate if the store is the right fit for the product.
Conflicting business rules (e.g., OD vs. Planogram)	The system applies a predefined hierarchy (e.g., planogram may override OD if capacity is lower), and logs details in remarks.	Review the hierarchy of rules and adjust settings as needed to ensure the desired allocation behavior.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All core master data (products, stores) and transactional data (sales, inventory) loaded and prepared.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned sales data and accurate Rate of Sale (ROS) calculations.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels.*
- **Distribution Warehouse Reservation:** What we need from it: *Rules for strategically holding back a percentage of warehouse inventory.*

- **Distribution Compute Revenue:** What we need from it: *The calculated revenue per day for store-styles and initial size depths.*
- **Distribution Ranking:** What we need from it: *The prioritized lists of store-styles and SKUs based on their business value.*
- **Distribution Segmentation:** What we need from it: *The assigned store-style segments (TOP_SELLER, BOTTOM_SELLER, NORMAL_SELLER) for differentiated allocation rules.*

After This Step

- **Logistics and Fulfillment Systems:** What they need from us: *The final allocation quantities to generate pick lists, shipping labels, and manage outbound warehouse operations.*
- **Store Inventory Management Systems:** What they need from us: *The planned inbound inventory to update store stock levels, aid in receiving processes, and inform in-store merchandising.*
- **Performance Reporting and Analytics:** What they need from us: *The detailed allocation decisions and outcomes for post-allocation analysis and continuous improvement of inventory strategies.*

10. Common Questions

Q: How does this module decide which stores get which products? A: It's a smart, multi-step process. First, it looks at how well a product sells at a specific store (Rate of Sale, revenue). Then, it checks if the store is a "TOP_SELLER" for that item. It considers the ideal amount to stock (Optimum Depth), the perfect mix of sizes (Ideal Size Set), how much display space is available (Planogram), and how much stock is already in the store or warehouse. It also considers strategic needs like new product launches or moving slow-selling items.

Q: What are "allocation iterations" and why are they important? A: Allocation iterations are like multiple rounds of decisions the system makes. Instead of one big calculation, it addresses different priorities in stages. For example, one iteration might focus on basic replenishment, another on maximizing stock for top sellers, and yet another on filling extra display space. This layered approach ensures all business goals and constraints are met systematically.

Q: How do Optimum Depth (OD) and Ideal Size Set (ISS) impact allocations? A: Optimum Depth (OD) defines the target inventory level for a product in a store, so the system tries to allocate enough to reach that target. Ideal Size Set (ISS) ensures that within that total quantity, the correct mix of sizes (e.g., more mediums and less extra-smalls) is sent to match actual customer demand, preventing stockouts of popular sizes and reducing excess of unpopular ones.

Q: Can the system send more stock than what's initially recommended? A: Yes, if enabled by business settings like "Planogram Increment Strategy" or "Depth Increment Strategy," the system can allocate additional units. This is useful when there's excess warehouse stock, high demand, or an opportunity to build deeper inventory for top-selling items or during peak seasons, maximizing sales opportunities.

Q: How does this module handle products that aren't selling well? A: For products identified as "BOTTOM_SELLERS" in previous steps, this module plays a key role in managing their inventory strategically. Instead of sending more, it identifies excess stock that can be pulled back to a central warehouse or transferred to a store where it might sell better, preventing dead stock and freeing up valuable shelf space.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To be the core engine that orchestrates multiple allocation strategies to determine the optimal quantity of each SKU for every store, maximizing sales and minimizing stockouts.
- **Main Input:** Cleaned sales data, ROS, Revenue per Day, OD, ISS, OW, adjusted warehouse and store inventory, store-style segments, and planogram data.
- **Main Output:** Final, precise SKU allocations per store, planned inventory movements, and detailed allocation remarks.
- **Warning Signs:** Unexpectedly high or low allocation quantities for key products, or inconsistencies between planned allocations and physical store capacity.
- **Key Settings:** Allocation Strategy Type, Deplete Warehouse Stock Strategy, Planogram Increment Strategy, and Depth Increment Strategy.

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 11: Distribution Compute Warehouse Inventory

Last Updated: 2025-05-23

1. Business Objective

This module centralizes inventory data for Inter-Store Transfers (IST), creating a “virtual warehouse” from excess stock found in individual stores. It groups stores into regional hubs for efficient transfers and prepares data for both incoming and outgoing product movements between stores.

Why This Matters: This ensures that usable inventory isn’t wasted sitting in one store but can be strategically reallocated to other stores where it’s needed, maximizing sales potential, improving inventory utilization, and enabling efficient stock balancing across the network.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- This module does not directly use historical sales data.

Product Data

- **Product Definitions and Attributes (SKU and Style):** *To identify unique items and their broader styles, ensuring accurate tracking of inventory quantities.*
- **Pack Size Data:** *Specifies the standard quantity in which a product is packaged or typically transferred, essential for determining the most efficient units for movement.*

Operational Data

- **Current Store Inventory Levels:** *The precise, real-time quantity of each product currently held at each retail location, serving as the source for identifying transferable excess stock.*
- **Store Geographic Grouping (Region, City, or Custom Group):** *Information on how stores are geographically or strategically clustered, which dictates how excess inventory from multiple stores is pooled into virtual warehouses.*

- **Store Inward Override Status:** *Indicates whether a specific store should be treated as “empty” for incoming transfers, allowing it to receive full replenishment regardless of its existing inventory.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Inter-Store Transfer (IST) Grouping Level	N/A (e.g., REGION)	Defines how stores are grouped for Inter-Store Transfers (IST), such as by Region, City, or a Custom Group.	When new geographical strategies are implemented or custom store groupings are required for transfers.
Minimum Leftover Stock for Transfer	0 units	The smallest quantity of a product style that must remain at a store (after accounting for its internal pack size) to be considered available for outward transfer.	To prevent transferring inefficiently small quantities or to prioritize full pack sizes for transfers.
Default Transfer Pack Size	1 unit	The default unit quantity used for transferring items if a specific style’s pack size is not explicitly defined.	When there’s a need for a system-wide fallback for product transfer quantities.
Store Inward Transfer Override	False	A specific rule that, if enabled for a store, causes its current stock levels to be disregarded when planning for incoming transfers, allowing it to receive full replenishment.	When a store needs to be fully stocked without regard to existing inventory, such as during a grand opening or a major refresh.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
    A[Raw Store Inventory & Store Grouping Data] -->
    B[Distribution Compute Warehouse Inventory]
    B --> C[Regional Virtual Warehouse Stock & Transfer Maps]
```

1. Start with → Current store inventory levels, product definitions (including pack sizes), and store grouping classifications (region, city, or custom group).
2. Then → The system maps stores to designated virtual warehouse hubs based on their geographic or custom groupings.
3. Next → It calculates a “virtual warehouse stock” for each regional hub by consolidating excess, transferable inventory from individual stores.
4. Finally → A comprehensive view of transferable inventory in regional virtual warehouses, along with initial maps for future inbound and outbound store transfers.

5. Step-by-Step Actions

1. **Define** Store Groupings for Transfers:
 - What we do: *We categorize each retail store into a specific regional, city, or custom group, creating a network for Inter-Store Transfers (IST).*
 - Why we do it: *This establishes the “virtual warehouse” structure, allowing excess inventory from multiple stores within a group to be pooled and managed centrally for transfers.*
2. **Prepare** for Inward Store Transfers:
 - What we do: *We initialize a record for each store and unique item (SKU) to track future incoming transfers, setting their initial inbound quantity to zero.*
 - Why we do it: *This readies the system to receive and plan new inventory allocations for stores, ensuring accurate tracking of all incoming stock.*
3. **Calculate** Transferable Store Inventory (Outward):
 - What we do: *For each product in each store, we identify excess inventory that is beyond what's needed for internal store operations, carefully accounting for product “pack sizes” to ensure only full, efficient units are considered for transfer.*
 - Why we do it: *This accurately determines what stock is truly available for outward transfer from stores, preventing transfers of partial quantities and ensuring efficient product movements.*
4. **Create** Virtual Warehouse Stock:
 - What we do: *We aggregate all identified transferable inventory from individual stores within their respective groups (region, city, or custom) into a centralized “virtual warehouse stock” pool.*
 - Why we do it: *This pooled inventory represents the total available stock for Inter-Store Transfers (IST), allowing for efficient re-distribution to other stores where it's needed.*

5. **Adjust** Store Inward Criteria (If Applicable):

- What we do: *For specific stores that are flagged with the “Store Inward Transfer Override,” the system may adjust their current stock levels or disregard their existing planogram criteria for certain product categories, allowing them to be fully replenished regardless of what they currently hold.*
- Why we do it: *This is a strategic decision to ensure certain stores receive complete allocations, often for new product launches or when stores are being strategically cleared for fresh inventory.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- Store A (Downtown, NYC) has 20 units of “Women’s Knit Sweater, Style #S001,” Size M. The pack size for this style is 5 units.
- Store B (Uptown, NYC) has 12 units of “Women’s Knit Sweater, Style #S001,” Size M.
- The “Inter-Store Transfer (IST) Grouping Level” is set to “CITY,” so both stores are grouped under “New York City” for transfer purposes.
- No other stores in NYC currently hold this specific style.
- The system is configured to *not* have the “Store Inward Transfer Override” enabled by default for these stores.

What Happens

1. First, the system **identifies** that Store A and Store B both belong to the “New York City” IST group. It then **creates** a unique virtual warehouse identifier for this “New York City” group.
2. Then, for Store A’s 20 units of Style #S001, the system **calculates** that all 20 units are fully transferable (20 divided by the pack size of 5 results in 4 full packs, with 0 units remaining). So, 20 units are earmarked for outward transfer.
3. Next, for Store B’s 12 units of Style #S001, the system **calculates** that 10 units are transferable (12 units divided by a pack size of 5 results in 2 full packs, with 2 units remaining). The 2 leftover units remain at Store B as a non-transferable odd quantity.
4. Finally, the system **aggregates** these transferable units into the “New York City” virtual warehouse. It now registers 30 units (20 from Store A + 10 from Store B) of “Women’s Knit Sweater, Style #S001,” Size M, as pooled and available in the “New York City” virtual warehouse for outward transfers to other stores within the same group or elsewhere if broader transfers are allowed.

Final Outcome

The system now has a clear, pooled view of excess inventory available for Inter-Store Transfers, enabling efficient reallocation of products like “Women’s Knit Sweater, Style #S001,” Size M, from stores where they are less needed to other stores where demand is higher. This maximizes inventory utilization and supports effective stock balancing across the retail network.

Key Takeaway: This module efficiently identifies and pools transferable inventory from individual stores into central virtual hubs, enabling strategic stock reallocation and maximizing overall inventory utilization across the network.

7. Results

This step produces:

Main Outputs

- **Regional Virtual Warehouse Stock:** A centralized, pooled view of available inventory for Inter-Store Transfers, grouped by geographic or custom regions, ready for re-distribution.
- **Outward Transfer Eligibility Records:** Detailed records on which quantities of products are identified as eligible for transfer *out* of specific stores.
- **Initial Inward Transfer Maps:** Prepared records for tracking future incoming transfers to individual stores, setting their initial inbound quantity to zero.

Business Reports

- **Virtual Warehouse Stock Report:** Used by inventory managers to view aggregated available stock for Inter-Store Transfers, aiding in balancing inventory across stores.
- **Store Transfer Eligibility Report:** Used by operations teams to identify specific stores with excess inventory suitable for outward transfers or consolidation.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential store inventory data is missing, potentially leading to incomplete or inaccurate virtual warehouse stock.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides based on expert judgment.
Store with zero transferable inventory after adjustment	If a store's reported inventory for a product is less than its minimum transferable quantity (pack size), it might not contribute any units to the virtual warehouse.	Review store reporting accuracy or consider if the product's pack size is too large for typical store transfers. For very low-stock items, manual transfers might be needed.
Conflicting store group assignments	If a store is incorrectly assigned to multiple or no Inter-Store Transfer	

Situation	What Happens	What You Should Do
Style without a defined pack size	(IST) groups, its inventory may not be correctly included in a virtual warehouse.	Review store master data and IST grouping settings to ensure accurate assignments.
	The system may default to a generic pack size or treat each unit as individually transferable, potentially leading to inefficient transfers or odd quantities remaining at stores.	Ensure all product styles have defined pack sizes in product master data for optimal transfer calculations.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *Accurate and up-to-date current store inventory levels, product definitions (including pack sizes), and store geographic or custom grouping information, all prepped for use.*

After This Step

- **Allocation Optimization Module:** What they need from us: *A centralized pool of truly available inventory for Inter-Store Transfers (IST), representing excess store stock that can be reallocated to meet demand elsewhere.*
- **Transfer Calculation Engine:** What they need from us: *The prepared regional virtual warehouse stock and initial inward/outward transfer maps to calculate and orchestrate actual product movements between stores.*

10. Common Questions

Q: What is “virtual warehouse stock” and why do we create it? A: “Virtual warehouse stock” is a pooled quantity of excess inventory identified across a group of retail stores. We create it to centrally manage and make available products that aren’t selling well in one store but could be transferred to another where demand is higher. This prevents unnecessary overstock and maximizes the chances of selling products across the network.

Q: How do you decide which inventory from a store is “leftover” or “transferable”? A: We identify inventory as “leftover” or “transferable” by looking at the current stock in a store and considering the product’s “pack size.” For example, if a product comes in packs of 5 and a store has 12 units, 10 units (two full packs) would be considered transferable, while the remaining 2 units would be “leftover” and remain at the store, as they don’t form a full transferrable unit.

Q: How are stores grouped into “regional warehouses” for transfers? A: Stores are grouped into “regional warehouses” based on predefined criteria, such as their geographic region (e.g., Northeast, West Coast), their city, or even custom business groupings. This creates logical hubs for managing Inter-Store Transfers, making it easier to reallocate stock within a localized area.

Q: How does this module prepare for both “inward” and “outward” transfers? A: This module prepares for “outward” transfers by identifying excess, transferable stock in stores and pooling it into the virtual warehouse. It prepares for “inward” transfers by creating initial records for each store and product, setting their planned incoming quantity to zero. This sets the stage for later modules to calculate how much new stock should be sent to each store.

Q: Why is it important to know a product’s “pack size” for transfers? A: Knowing a product’s “pack size” (e.g., a style always ships in multiples of 5 units) is crucial for efficient transfers. It ensures that products are moved in full, sensible quantities, avoiding fragmented shipments and minimizing the administrative burden and potential damage associated with transferring odd, partial units.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To centralize and organize warehouse inventory specifically for Inter-Store Transfers (IST), pooling excess store stock into “virtual warehouses.”
- **Main Input:** Current store inventory levels, product pack sizes, and store grouping definitions.
- **Main Output:** Regional virtual warehouse stock, outward transfer eligibility records, and initial inward transfer maps.
- **Warning Signs:** Missing store grouping data, or unexpected low transferable quantities from stores that are known to have excess stock.
- **Key Settings:** Inter-Store Transfer (IST) Grouping Level, Store Inward Transfer Override.

Need Help? Contact your inventory management or supply chain team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 12: Distribution Inter-Warehouse Transfer (IWHT)

Last Updated: 2025-05-23

1. Business Objective

This module manages inventory transfers between warehouses, consolidating demand and available stock within warehouse groups to identify net needs and orchestrate optimal movements, including various types of pullbacks, to rebalance inventory across the network.

Why This Matters: This crucial step ensures that products are always available in the right distribution centers to meet store and customer demand. By efficiently moving stock between warehouses, we prevent stockouts, reduce excess inventory, minimize transfer costs, and maximize sales opportunities across the entire retail network.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Net Product Demand from Stores:** *Represents the precise quantities of products needed by individual stores (derived from previous allocation calculations), which is then consolidated to form warehouse-level demand for transfers.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category):** *Details each unique item, its broader style, and category, enabling the system to track inventory and apply relevant transfer rules.*
- **Mother Warehouse Hierarchies:** *Identifies the relationships between warehouses (e.g., a regional hub serving as a 'mother' to smaller satellite warehouses), guiding replenishment and pullback priorities.*

Operational Data

- **Current Warehouse Inventory Levels:** *The most up-to-date quantity of each product currently held in distribution warehouses, serving as the available stock for transfers.*
- **Warehouse Groupings for Transfers:** *Defines how warehouses are clustered into logical groups (e.g., by region) for consolidated inventory management and transfer planning.*
- **Warehouse Planogram Capacities:** *Information about the maximum inventory space available at each receiving warehouse, ensuring that transfers do not lead to overstocking or unmanageable volumes.*
- **Feasible Movement Paths between Warehouses:** *Predefined routes or connections between warehouses, ensuring that proposed transfers are logistically possible and efficient.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Inter-Warehouse Transfer Grouping Logic	N/A (e.g., REGION)	Defines how warehouses are grouped together for consolidated inventory and demand management during transfers (e.g., by geographic region or custom cluster).	When new regional strategies are implemented or if warehouses need to be grouped differently for efficiency.
Pullback Strategy Priority	Demanding first	Determines which type of inventory pullbacks (e.g., demanding pullbacks to fulfill specific needs, or other pullbacks for general rebalancing) are prioritized.	To emphasize fulfilling immediate demand over general inventory rebalancing, or vice versa, based on business needs.
Movement Optimization Enabled	True	Controls whether the system actively seeks the most efficient and direct paths for transferring products	To prioritize minimizing transit times and costs, or to allow more flexible routing if efficiency is less critical.

Setting Name	Default Value	Description	When to Change
Minimum Transfer Quantity	1 unit	between warehouses based on predefined routes. The smallest quantity of a product that can be transferred in an inter-warehouse movement.	To prevent inefficient transfers of very small or uneconomical quantities of stock.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Net Store Demand, Warehouse Inventory & Groupings] -->
    B[Distribution Inter-Warehouse Transfer (IWHT)]
    B --> C[Optimized Inter-Warehouse Transfer Recommendations]
  
```

1. Start with → Net demand for products from individual stores (calculated by earlier modules), current warehouse inventory, and defined warehouse grouping structures.
2. Then → The system first reserves necessary stock within each warehouse for its immediate needs and then orchestrates replenishment from designated 'mother' (parent) warehouses.
3. Next → It consolidates overall demand within warehouse groups and identifies available stock. It then orchestrates transfers and performs various types of pullbacks to rebalance the network.
4. Finally → Optimized recommendations for inter-warehouse product movements, ensuring inventory is strategically placed.

5. Step-by-Step Actions

1. **Reserve** Warehouse Stock and Replenish from Parents:
 - What we do: *The system first earmarks current warehouse stock for its immediate, calculated demand, and then attempts to fulfill any remaining demand by pulling from its designated 'mother' (parent) warehouses to ensure primary needs are met.*
 - Why we do it: *This ensures that each warehouse can satisfy its local demands first and taps into the broader network for additional stock if needed, maximizing the use of existing inventory.*
2. **Consolidate** Inventory and Roll Up Demand:
 - What we do: *The system gathers all available stock within defined warehouse groups and aggregates demand from child warehouses and*

stores up to central distribution points, creating a comprehensive picture of net needs across the group.

- *Why we do it: This provides a holistic view of supply and demand within a specific network of warehouses, enabling smart, group-level transfer decisions rather than fragmented ones.*

3. Orchestrate Inter-Warehouse Transfers:

- *What we do: It identifies opportunities to move stock from source warehouses with excess inventory to fulfill demand in other warehouses within the same group, optimizing the overall flow of inventory to meet regional needs.*
- *Why we do it: This proactively moves products to where they are most needed, ensuring efficient distribution and reducing the risk of stockouts in high-demand areas.*

4. Perform Demanding Pullbacks:

- *What we do: The module initiates targeted inventory pullbacks from warehouses with excess stock to other warehouses that have active, specific demand for those items, rebalancing inventory towards critical needs.*
- *Why we do it: This directly addresses unfulfilled demand by strategically reallocating existing stock, improving product availability and customer satisfaction.*

5. Optimize Movement Paths:

- *What we do: The system then refines the planned transfers, seeking the most efficient and direct paths for moving products between warehouses, minimizing transit time and cost and ensuring compliance with predefined feasible routes.*
- *Why we do it: This reduces logistical complexities and expenses, making the transfer process as lean and effective as possible.*

6. Execute Other Pullbacks:

- *What we do: Finally, it performs additional pullbacks from warehouses with slow-moving or excess inventory to parent warehouses where the stock can be better utilized or consolidated (e.g., for clearance), further rebalancing the network even without explicit demand.*
- *Why we do it: This frees up capital and space in less productive locations, preventing dead stock and preparing inventory for its next best use.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new “Winter Coat, Style #WC001” is needed by Warehouse A (Regional Hub in the “Northern Region” IWHT Group) for an urgent replenishment, with a total demand of 100 units.
- Warehouse B (Satellite Warehouse, child of A, also in “Northern Region”) has 50 units of “Winter Coat, Style #WC001” that are excess to its own needs.
- Warehouse C (Another Satellite Warehouse, child of A, also in “Northern Region”) has 70 units of “Winter Coat, Style #WC001” that are excess.
- Warehouse A has sufficient planogram capacity (e.g., 120 units for this category) to receive new stock.

What Happens

1. First, the system **consolidates** the 100-unit demand from Warehouse A and identifies the excess stock from Warehouse B (50 units) and Warehouse C (70 units) within the “Northern Region” group.
2. Then, it **identifies** that Warehouse A has a net demand and that Warehouses B and C have available stock to fulfill it. It also checks that Warehouse A’s planogram can accommodate the incoming units.
3. Next, it **orchestrates** initial transfers based on demanding pullbacks: 50 units are moved from Warehouse B to Warehouse A, and 50 units are moved from Warehouse C to Warehouse A, fully meeting Warehouse A’s 100-unit demand.
4. Finally, after fulfilling demand, 20 units of “Winter Coat, Style #WC001” are still left as excess at Warehouse C. Since Warehouse A still has 20 units of unused planogram space (120 capacity - 100 received), the system initiates an **other pullback**, moving these remaining 20 units from Warehouse C to Warehouse A for consolidation.

Final Outcome

Warehouse A successfully receives 120 units of “Winter Coat, Style #WC001” (100 units to meet its initial demand, plus an additional 20 units from the other pullback). This ensures Warehouse A is fully stocked to meet demand while efficiently consolidating excess inventory from satellite warehouses, preventing waste and maximizing overall inventory utilization across the region.

Key Takeaway: This module ensures that inventory from one warehouse can be strategically reallocated to fulfill demand in another, improving overall stock utilization and rebalancing the network.

7. Results

This step produces:

Main Outputs

- **Optimized Inter-Warehouse Transfer Recommendations:** The final, precise quantities of each unique product (SKU) recommended for transfer between specific source and destination warehouses.
- **Planned Inventory Pullback Instructions:** Detailed records of products to be pulled back from warehouses with excess stock to other warehouses (e.g., parent hubs) for consolidation or redistribution.
- **Consolidated Warehouse Demand:** A centralized, up-to-date view of net demand for products across defined warehouse groups, serving as a primary input for transfer decisions.
- **Fulfilled Branch Demand Records:** Detailed tracking of which portions of the rolled-up demand were successfully met by inter-warehouse transfers.

Business Reports

- **Warehouse Transfer Plan:** Used by logistics and warehouse operations teams to execute product movements, generate pick lists, and manage inbound/outbound processes efficiently.
- **Inventory Rebalancing Report:** Used by inventory planners and supply chain managers to monitor how inventory is being strategically rebalanced across the network, identifying areas of improvement.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential data (e.g., inventory, warehouse groupings, or store demands) is missing, potentially leading to incomplete or inaccurate transfer recommendations.	Investigate the data source immediately to restore the feed. If critical, delay the distribution run until data is complete or use manual overrides based on expert judgment.
Insufficient total stock in a group to meet demand	The system will fulfill demand based on available stock, prioritizing transfers according to internal rules, but some demand may remain unfulfilled.	Review future inbound shipments to the network and consider external sourcing options if the demand is critical.
No feasible transfer paths between warehouses	If defined routing data is missing or incorrect, the system will not be able to orchestrate direct transfers, potentially leaving demand unfulfilled or excess stock unmoved.	Verify warehouse routing configurations in the master data and ensure all necessary connections are properly defined and maintained.
Cycles detected in warehouse group dependencies	The system will flag an error and halt the process if it detects circular dependencies in the warehouse grouping, as this prevents logical demand roll-up.	Review and correct the warehouse grouping definitions (e.g., mother-child relationships) to eliminate any circular logic.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All foundational master data (products, warehouses, stores) and transactional data (inventory) prepped and available for understanding the distribution network.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels, accounting for discrepancies and lead time reservations, which represent the actual stock available for inter-warehouse transfers.*
- **Distribution Warehouse Reservation:** What we need from it: *Rules for strategically holding back a percentage of warehouse inventory, influencing the total stock considered available for inter-warehouse transfers.*
- **Distribution Allocation:** What we need from it: *The primary module that generates the net demand for products at each store and, by extension, the warehouse, which is then consolidated here for inter-warehouse transfers.*

After This Step

- **Logistics and Fulfillment Systems:** What they need from us: *The final, optimized inter-warehouse transfer recommendations to generate pick lists, shipping labels, and manage outbound/inbound operations between warehouses.*
- **Inventory Management Systems:** What they need from us: *Updates to central inventory records reflecting the executed transfers and the newly rebalanced stock levels across the entire warehouse network.*

10. Common Questions

Q: Why do we need to transfer inventory *between* warehouses? A: Transferring inventory between warehouses (IWHT) is essential for efficient inventory management across a large network. It allows us to consolidate excess stock from one location to fulfill demand in another, respond to regional sales trends, prevent product obsolescence, and ultimately ensure products are where they need to be to maximize sales and minimize waste.

Q: How does “demand roll-up” work? A: Demand roll-up is the process of aggregating product needs from individual stores and smaller satellite warehouses “up” to their larger, designated regional or central warehouses. This gives the system a comprehensive view of the total demand within a specific warehouse group, enabling it to efficiently plan transfers from available stock to meet multiple, consolidated needs.

Q: What is the difference between “demanding pullbacks” and “other pullbacks”? A: “Demanding pullbacks” occur when a parent warehouse *needs* specific stock, and that stock is found to be in excess at one of its child warehouses. The system then “pulls back” the required quantity to fulfill the parent’s demand. “Other pullbacks” are for general inventory rebalancing; they involve pulling excess, slow-moving, or

strategically important stock from a child warehouse to a parent, even if the parent doesn't have an immediate, explicit demand for it, often for consolidation or later redistribution.

Q: How does this module optimize transfers? A: The module optimizes transfers by seeking the most efficient and direct paths for product movements between warehouses. It considers predefined feasible routes and aims to move stock from a source directly to the ultimate destination where it's needed, rather than routing it through unnecessary intermediate points. This helps reduce transit times, costs, and logistical complexity.

Q: What happens if a warehouse group doesn't have enough total stock to fulfill all consolidated demands? A: If there isn't enough consolidated stock within a warehouse group to meet all demands, the system will prioritize how it fulfills orders. It will distribute the available stock among the demanding warehouses based on internal rules (like a fair-share approach or demand prioritization), fulfilling as much as possible and leaving the remaining demand unfulfilled for future replenishment or external sourcing.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To manage inventory transfers between warehouses, consolidating demand and orchestrating movements to rebalance stock across the network.
- **Main Input:** Net product demand from stores/warehouses, current warehouse inventory levels, and defined warehouse grouping structures.
- **Main Output:** Optimized inter-warehouse transfer recommendations and inventory pullback plans.
- **Warning Signs:** Unfulfilled demands despite perceived available stock, unexpected transfer routes, or an inability to rebalance inventory across warehouse groups.
- **Key Settings:** Inter-Warehouse Transfer Grouping Logic, Pullback Strategy Priority, Movement Optimization Enabled.

Need Help? Contact your inventory management or supply chain team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 13: Distribution Inter-Store Transfer (IST)

Last Updated: 2025-05-23

1. Business Objective

This module manages inventory movements directly between stores, rebalancing excess stock from one location to fulfill demand in another within defined transfer groups.

Why This Matters: This crucial step ensures that valuable inventory isn't wasted sitting in one store but can be strategically reallocated to other stores where it's needed, maximizing sales potential, improving inventory utilization, and enabling efficient stock balancing across the network.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- This module does not directly use historical sales data.

Product Data

- **Product Definitions and Attributes (SKU and Style):** *Identifies each unique item and its broader style, ensuring accurate tracking of inventory quantities and enabling transfers for specific product lines.*
- **Pack Size Data:** *Specifies the standard quantity in which a product is packaged or typically transferred, essential for determining the most efficient units for movement between stores.*

Operational Data

- **Current Store Inventory Levels (Excess & Needed):** *Provides the precise quantity of products currently held at each store, identifying both excess stock available for transfer out and quantities needed for transfer in, as determined by previous allocation steps.*
- **Store Geographic Grouping (Region, City, or Custom Group):** *Information on how stores are geographically or strategically clustered, which dictates the boundaries for inter-store transfer opportunities.*

- **Store Display Health Data (PSA):** Details on how efficiently a product is displayed or utilized in its selling space, used to assess store “brokenness” and inform post-transfer corrections.
- **Store Inward Override Status:** Indicates whether a specific store should be treated as “empty” for incoming transfers, allowing it to receive full replenishment regardless of its existing inventory.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Inter-Store Transfer (IST) Grouping Level	N/A (e.g., CITY)	Defines how stores are grouped for Inter-Store Transfers (IST), such as by City, Region, or a Custom Group.	When new geographical strategies are implemented or custom store groupings are required for transfers.
Correct Store Health Post-Transfer	True	Determines if the system should automatically rebalance inventory after initial transfers to improve the visual merchandising or display health (PSA) of stores.	To adjust how aggressively the system attempts to fix incomplete product displays after initial transfers.
Enable Transfer-Based Replenishment	True	Controls whether the system executes detailed replenishment iterations (like planogram or depth increments) using only inventory available through inter-store transfers.	When a business wants to leverage internal store-to-store transfers for restocking, rather than relying solely on warehouse shipments.
Allocation Display Space Target for IST	N/A (e.g., 0.85)	The minimum percentage of allocated display space (PSA) that a store aims to fill	To adjust inventory targets based on store capacity or visual merchandising

Setting Name	Default Value	Description	When to Change
		during inter-store allocation iterations.	strategy within an IST context.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Prepared Store Inventory (Excess & Demand) & Grouping Data]
    --> B[Distribution Inter-Store Transfer (IST) Logic]
    B --> C[Optimal Inter-Store Transfers & Post-Transfer Corrections]
  
```

1. Start with → Prepared store inventory data (excess stock from some stores, demand from others), product details (like pack sizes), and defined store transfer groups.
2. Then → The system first reconciles any planned inward and outward movements within the same store, then orchestrates direct transfers of excess stock between stores within predefined groups.
3. Next → It analyzes the display health of stores after these transfers and rebalances remaining available stock to improve incomplete product displays or fill strategic gaps.
4. Finally → A precise plan for moving inventory directly between stores, optimizing network-wide stock utilization and display effectiveness, along with identifying any remaining excess for central pullbacks.

5. Step-by-Step Actions

1. **Initialize** for Inter-Store Transfers:
 - What we do: *The system sets up the environment and loads all necessary data and business rules, specifically tailored for managing transfers directly between retail stores.*
 - Why we do it: *This ensures the inter-store transfer process is correctly configured for the specific business objective of rebalancing store inventory.*
2. **Net Out** Internal Store Movements:
 - What we do: *We identify and correct situations where a store was both designated to receive and send out the same product (SKU). The system nets these quantities, effectively cancelling out internal movements, resulting in a cleaner picture of true inward or outward requirements.*
 - Why we do it: *This prevents unnecessary internal transfers, streamlines inventory reconciliation, and ensures that only real inter-store movements are planned.*
3. **Orchestrate** Inter-Store Transfers:
 - What we do: *The system systematically calculates and prioritizes the transfer of excess stock from one store to another. It first looks for opportunities within the same city, then within the same region, and finally*

across broader geographic areas within the defined Inter-Store Transfer (IST) group, always aiming to fulfill demand with available excess.

- *Why we do it: This maximizes the efficient re-distribution of inventory, ensuring that products are moved to where they are most needed, prioritizing proximity to minimize logistics costs and transit times.*

4. Correct Post-Transfer Store Health:

- *What we do: If enabled, the system re-evaluates the display health (PSA) of stores after the initial transfers. It identifies stores where product displays might still be “broken” (incomplete or below target levels) due to insufficient stock, and then attempts to allocate any remaining available excess inventory (from other stores) to these struggling locations.*
- *Why we do it: This critical step optimizes the visual merchandising and sales potential of stores, ensuring that even after initial transfers, product displays are as complete and appealing as possible to maximize sales.*

5. Identify Remaining Outward Stock for Pullback:

- *What we do: We flag any quantities of stock that were initially identified as excess for outward transfer from a store but ultimately did not get transferred to another store. These quantities are then designated for potential pullback to a central warehouse.*
- *Why we do it: This ensures that truly unproductive or excess stock is not left idle in stores but can be consolidated for further action, such as clearance or redistribution from a central point.*

6. Run Allocation-Based Replenishment (Optional):

- *What we do: If the Enable Transfer-Based Replenishment setting is active, the system runs additional allocation iterations (similar to core allocation logic, focusing on Optimum Depth, Ideal Size Set, and planogram increments) but specifically using the inventory that has been made available through inter-store transfers.*
- *Why we do it: This allows the system to proactively fill stores using readily available excess from other stores, rather than waiting for new warehouse shipments, thus optimizing inventory flow within the network.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Store A (NYC Downtown)** has 15 units of “Men’s Casual Shirt, Style #MS001,” Size M. Its Optimum Depth (OD) for this SKU is 10 units. Its current display health (PSA) for this style is 100%. Store A needs 5 units more of another Style #MS002.
- **Store B (NYC Uptown)** has 5 units of “Men’s Casual Shirt, Style #MS001,” Size M. Its OD for this SKU is 10 units. Its current display health (PSA) for this style is 40% (meaning it needs more stock for a healthy display).
- **Store C (Albany, NY)** has 10 units of “Men’s Casual Shirt, Style #MS001,” Size M. Its OD is 10 units. Its PSA for this style is 100%.
- All three stores are in the “New York Region” Inter-Store Transfer (IST) group.
- The Correct Store Health Post-Transfer setting is enabled.

What Happens

1. First, the system **identifies** Store A has 5 units of excess “Men’s Casual Shirt, Style #MS001,” Size M (15 units current - 10 units OD). Store B needs 5 units of the same SKU (10 units OD - 5 units current). Store C has no excess and no need for this SKU.
2. Then, the system **orchestrates** an inter-store transfer: It moves 5 units of “Men’s Casual Shirt, Style #MS001,” Size M, from **Store A (NYC Downtown)** to **Store B (NYC Uptown)**, as both are in the same city and the same IST group. Store A now has 10 units, and Store B has 10 units.
3. Next, the system **receives** a new demand for Style #MS002 at Store A (NYC Downtown). Store C (Albany, NY) now has 10 units of Style #MS002, which is 5 units in excess of its needs. The system orchestrates a transfer of 5 units of Style #MS002 from **Store C (Albany)** to **Store A (NYC Downtown)**.
4. Finally, after these transfers, the system **re-evaluates** Store B’s display health for Style #MS001. If its PSA was still below the target (e.g., 85%) even after receiving the 5 units, and if Store C or another store in the region still had excess available for this SKU (even if not explicitly needed elsewhere), the system might allocate additional units to Store B to improve its PSA. If no further rebalance happens, and Store C now has an additional 5 units of Style #MS001 excess (from the other initial transfer), these 5 units are flagged for potential **pullback** to the central warehouse if no other store needs them.

Final Outcome

Store A successfully transfers its excess “Men’s Casual Shirt, Style #MS001” to Store B, bringing both stores to their optimal stock levels for that SKU. Store A also receives the needed Style #MS002 from Store C, fulfilling a demand. The system proactively works to ensure stores have healthy, well-stocked displays. Any remaining surplus stock from other stores is identified for consolidation back to the warehouse, preventing unnecessary holding costs and maximizing inventory utilization.

Key Takeaway: This module efficiently moves existing inventory between stores to satisfy demand, improve display health, and minimize idle stock, directly contributing to better sales and healthier store operations.

7. Results

This step produces:

Main Outputs

- **Optimal Inter-Store Transfer Recommendations:** The final, precise quantities of each unique product (SKU) recommended for transfer between specific source and destination retail stores.
- **Planned Pullbacks from Stores:** Detailed instructions for products to be pulled back from individual stores (where excess stock could not be reallocated) to a central warehouse.

- **Corrected Store Health Rates (Post-IST):** Updated metrics reflecting the improved display health (PSA) of stores after inventory rebalancing through inter-store transfers.
- **IST Activity Records:** Comprehensive logs of all intra-store netting and inter-store movements, providing an audit trail for inventory adjustments.

Business Reports

- **Inter-Store Transfer Plan:** Used by store operations and logistics teams to execute product movements between stores, ensuring efficient and accurate stock rebalancing.
- **Post-Transfer Store Health Report:** Used by merchandisers and store managers to review the improved product display conditions and identify any remaining areas of concern after transfers.
- **Store Pullback Candidates List:** Used by inventory managers to identify specific products and stores from which excess, non-transferable stock should be pulled back to central inventory.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential store inventory data or grouping information is missing, potentially leading to incomplete or inaccurate transfer recommendations.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides based on expert judgment.
No stores qualify for transfers within a group	If no stores have excess stock to share, or no other stores within the group have demand, no inter-store transfers will be recommended.	Review demand and supply across the broader network; consider if central warehouse allocation is needed.
Excess stock remains after all transfers	If a store has excess stock that cannot be reallocated to any other store within its IST group (even after brokenness correction), it will be flagged for potential pullback to a central warehouse.	Review the specific product's performance and demand, consider markdowns, or manually reallocate the stock.
Conflicts in store grouping definitions	If a store is incorrectly assigned to multiple or no Inter-Store Transfer (IST) groups, its inventory may not be	Review and correct the store master data and IST grouping settings to ensure accurate assignments.

Situation	What Happens	What You Should Do
	correctly included in the transfer logic.	

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *Accurate and up-to-date current store inventory levels, product definitions (including pack sizes), and store geographic or custom grouping information, all prepped for use.*
- **Distribution Compute Warehouse Inventory:** What we need from it: *The initial identification of excess store inventory pooled into “virtual warehouses” and prepared for outward transfers, setting the stage for inter-store movements.*
- **Distribution Allocation:** What we need from it: *The primary module that generates the net demand for products at each store and helps identify which stores have excess or deficit stock for inter-store rebalancing.*

After This Step

- **Logistics and Fulfillment Systems:** What they need from us: *The final, optimized inter-store transfer recommendations to generate transfer orders, pick lists for stores, and manage inbound/outbound operations between retail locations.*
- **Store Inventory Management Systems:** What they need from us: *Updates to individual store inventory records reflecting the executed transfers and the newly rebalanced stock levels.*
- **Performance Reporting and Analytics:** What they need from us: *The detailed transfer decisions and outcomes for post-transfer analysis and continuous improvement of inventory strategies.*

10. Common Questions

Q: What is Inter-Store Transfer (IST) and how does it help my business? A: Inter-Store Transfer (IST) is the process of moving products directly between your retail stores. It helps your business by ensuring that excess stock from one store (where it's not selling) can be sent to another store (where it's needed), maximizing sales opportunities across your entire network, reducing waste from overstock, and making sure popular items are always available.

Q: Why do we “net out” internal movements within a store? A: Netting out internal movements means if a store was somehow told to both send out and receive the same item, we cancel those actions out internally. This ensures we only plan for *real* transfers between different stores, making the process cleaner, preventing unnecessary logistics, and providing a more accurate picture of actual inventory changes.

Q: How are transfer priorities decided between stores in IST? A: The system prioritizes transfers based on proximity: it first looks for opportunities to transfer stock between stores within the same city. If demand isn't met, it then expands the search to stores within the same broader region, and finally to any store within the designated Inter-Store Transfer (IST) group. This approach minimizes shipping costs and gets products to where they're needed faster.

Q: What is "brokenness correction" post-IST? A: "Brokenness correction" refers to improving a store's display health (how complete and visually appealing its product displays are) after initial transfers. If a store's display is still incomplete, the system will try to allocate any remaining available excess stock to "fix" that display, ensuring products are well-presented to customers, which boosts sales.

Q: How does IST help store profitability? A: IST significantly impacts store profitability by reducing lost sales from stockouts (because needed items are moved where they're selling), minimizing markdowns on excess stock (because slow-moving items are moved to better markets or consolidated), and optimizing inventory turns across the entire retail network. This makes each product work harder for your business.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To manage and optimize inventory transfers directly between retail stores, rebalancing stock and improving store display health.
- **Main Input:** Current store inventory levels (excess and needed), product pack sizes, store grouping definitions, and store display health data.
- **Main Output:** Optimal inter-store transfer recommendations, planned pullbacks from stores, and improved post-transfer store health rates.
- **Warning Signs:** Unmet store demands within a transfer group, or large quantities of excess stock remaining in stores after the IST process.
- **Key Settings:** Inter-Store Transfer (IST) Grouping Level, Correct Store Health Post-Transfer, Enable Transfer-Based Replenishment.

Need Help? Contact your inventory management or supply chain team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 14: Distribution Planogram Incrementer

Last Updated: 2025-05-23

1. Business Objective

This module calculates and applies increment ratios to store planograms by analyzing historical sales data (revenue and Average Selling Price - ASP) at various levels (store-category, store-category-attribute). It also calculates a “Sales Potential Quantity” (often referred to as ‘Quantum’) to understand demand. This adjustment ensures that planograms evolve to reflect actual demand and optimize product visibility and depth.

Why This Matters: By dynamically adjusting planogram capacities, we ensure that popular products receive adequate display space and stock, maximizing sales and minimizing missed opportunities due to insufficient on-shelf inventory. This leads to more efficient use of valuable store space and ultimately boosts profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records (Revenue and Units Sold):** *Provides the raw sales data to calculate actual revenue generated and Average Selling Price (ASP) at different levels, which are critical for determining sales potential and increment ratios.*
- **Sales Analysis Periods:** *Defines the specific historical durations (e.g., last 30, 60, or 90 days) over which sales data should be analyzed for accurate calculations.*

Product Data

- **Product Definitions and Attributes:** *Details each unique item (SKU), its style, category, and other Attribute Groups (AGs), enabling sales analysis and increment calculations at various levels of product granularity.*
- **Attribute Group (AG) Definitions:** *Specifies how product attributes (like color, material, style feature) are grouped, allowing the system to understand sales performance and apply increments based on these groupings.*

Operational Data

- **Store List and Channel Data:** *Identifies all relevant retail locations and their associated sales channels (e.g., online, offline), necessary for segmenting sales data and applying regional insights.*
- **Store Planogram Configurations:** *Provides details on the current physical display capacity (stocks) allocated for products within each store and category, serving as the baseline for increment calculations.*
- **Warehouse Inventory Levels:** *The total quantity of current stock held in distribution warehouses, used to understand the overall supply available to support potential planogram increments.*
- **Optimum Depth (OD) Segmentation Data:** *Output from previous modules (e.g., Distribution Compute Revenue), indicating target stock levels and revenue contributions, which inform the base for sales potential calculations.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Sales Analysis Period for Increment	N/A (e.g., 90 days)	The historical duration over which sales data (revenue and ASP) is analyzed to calculate the “Sales Potential Quantity” (Quantum) and subsequently the planogram increment ratio.	To make the increment ratio more sensitive to recent sales trends or to provide a more stable, averaged view.
Minimum Sales for Quantum Calculation	N/A (e.g., 10 units)	The minimum number of units sold or a minimum revenue threshold required for a product category or Attribute Group (AG) combination to have its “Sales Potential Quantity” (Quantum) calculated for planogram increment.	To prevent highly infrequent sales or very low-value items from skewing increment recommendations.

Setting Name	Default Value	Description	When to Change
Attribute Grouping for Analysis	N/A (e.g., 'Color', 'Material')	Specifies which Attribute Groups (AGs) should be used for detailed sales analysis (revenue, ASP) and subsequent planogram increment calculations.	When new product attributes become strategically important for merchandising or when analysis needs to be more granular.
Warehouse Inwards Contribution Threshold	N/A (e.g., 0 units)	A minimum threshold for the total warehouse inventory of a product category or Attribute Group within a region to be considered significant enough to influence the planogram increment ratio.	To ensure that only categories with meaningful available supply are considered for larger planogram expansions.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Historical Sales Data, Planogram Configs, OD Data] --> B[Distribution Planogram Incrementer]

B --> C[Incremented Planogram Data & Sales Potential Quantities]

1. Start with → Cleaned historical sales figures, current store planogram configurations, and Optimum Depth (OD) segmentation data.
2. Then → The system calculates daily revenue and Average Selling Price (ASP) at various levels (store-category, store-category-attribute).
3. Next → It computes a "Sales Potential Quantity" (Quantum) for each level by dividing revenue by ASP, representing the effective units sold.
4. Finally → It determines planogram increment ratios by comparing this Quantum to current planogram stocks and considering available warehouse inventory relative to regional demand, preparing specific increment recommendations.

5. Step-by-Step Actions

1. Retrieve Sales Data for Analysis:

- What we do: *We gather historical sales data (units sold and revenue) for the specified analysis period from all relevant stores and channels.*
- Why we do it: *This provides the foundation for understanding actual demand and calculating accurate Average Selling Prices (ASPs) and revenue contributions.*

2. Calculate Revenue per Day and ASP:

- What we do: *We compute the average daily revenue and Average Selling Price (ASP) for products at multiple granularities: for each store-category combination, and then further broken down by specific Attribute Groups (AGs) like color or material, both at store and channel levels.*
- Why we do it: *This provides a comprehensive view of sales performance, critical for understanding which products are driving revenue and at what price point, forming the basis for demand potential.*

3. Compute Sales Potential Quantity (Quantum):

- What we do: *For each store-category and store-category-attribute combination, we divide the calculated total revenue by the Average Selling Price (ASP) to determine a “Sales Potential Quantity” (Quantum). This represents the effective number of units sold based on revenue and ASP.*
- Why we do it: *This metric provides a normalized measure of demand or sales potential, allowing for a consistent comparison of product performance that factors in price.*

4. Determine Total Warehouse Inwards and Regional Quantum:

- What we do: *We aggregate all incoming or available inventory in warehouses (referred to as “inwards”) for each product category and Attribute Group. Simultaneously, we sum up the “Sales Potential Quantity” (Quantum) for all relevant stores within a specific channel and region.*
- Why we do it: *This provides a vital context of overall supply (inwards) versus regional demand (total quantum), which is crucial for assessing if there’s enough available stock to support planogram increments.*

5. Calculate Planogram Increment Ratio:

- What we do: *We compute the planogram increment ratio for each store-category and store-category-attribute combination. This involves comparing the calculated “Sales Potential Quantity” (Quantum) to the current planogram capacity, then adjusting this ratio based on the total warehouse inwards versus the total regional “Sales Potential Quantity.”*
- Why we do it: *This calculation directly translates sales performance, current display space, and overall supply into a precise recommendation for how much a planogram’s capacity for specific items should be increased, ensuring that planograms align with actual demand and available stock.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Store A** has a planogram capacity of 50 units for its Women 's Casual Tops category.

- Over the last 90 days, Women's Casual Tops in **Store A** generated \$10,000 in revenue with an Average Selling Price (ASP) of \$25 per unit.
- Across all warehouses, there are 2,000 units of Women's Casual Tops currently considered “inwards” (available to ship).
- Within the entire “Northeast Region” (which includes Store A) and the ‘Women’s Casual Tops’ category, the total calculated “Sales Potential Quantity” (Quantum) is 5,000 units across all stores.

What Happens

1. First, the system **calculates** the “Sales Potential Quantity” (Quantum) for Women's Casual Tops in **Store A**: $\$10,000 \text{ (Revenue)} / \$25 \text{ (ASP)} = 400$ units.
2. Then, it **determines** an initial efficiency ratio for Store A's Women's Casual Tops: $400 \text{ units (Quantum)} / 50 \text{ units (Planogram Capacity)} = 8.0$. This indicates strong sales potential relative to current display space.
3. Next, the system **calculates** the ratio of total warehouse inwards to total regional sales potential for Women's Casual Tops: $2,000 \text{ units (Total Inwards)} / 5,000 \text{ units (Total Regional Quantum)} = 0.4$. This shows how much warehouse supply is available compared to regional demand.
4. Finally, the system **computes** the overall planogram increment ratio for Women's Casual Tops in **Store A**: $0.4 \text{ (Regional Supply/Demand Ratio)} * 8.0 \text{ (Store Efficiency Ratio)} = 3.2$. This means the planogram capacity for Women's Casual Tops in Store A should ideally be incremented by a factor of 3.2. So, instead of 50 units, the system recommends a new target of 160 units ($50 * 3.2$).

Final Outcome

The system produces a clear recommendation that the planogram capacity for Women's Casual Tops at Store A should be significantly increased (from 50 to 160 units). This ensures that Store A's display space for this high-performing category can adequately reflect its strong sales potential and the available inventory in the warehouses, leading to increased sales and better customer satisfaction.

Key Takeaway: This module leverages sales performance, display capacity, and overall supply to generate data-driven recommendations for expanding planogram space where it will have the greatest sales impact.

7. Results

This step produces:

Main Outputs

- **Calculated Sales Potential Quantity (Quantum):** *A precise measure of effective units sold (revenue/ASP) for store-category and store-category-attribute combinations, indicating sales potential.*
- **Planogram Increment Ratios:** *Specific multipliers for each store-category and store-category-attribute combination, recommending how much their planogram capacity should be increased.*

- **Aggregated Warehouse Inwards and Regional Quantum:** *Consolidated views of available warehouse stock and total sales potential within specific regions and categories, providing context for increment decisions.*

Business Reports

- **Proposed Planogram Expansion Report:** Used by merchandisers and store operations to review recommended increases in display space for products, aiding in strategic store layout planning and visual merchandising.
- **Category Performance vs. Capacity Report:** Used by inventory planners and category managers to identify product categories that are outperforming their current planogram capacity, informing decisions on future inventory investments and display adjustments.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential sales, planogram, or inventory data is missing, potentially leading to incomplete or inaccurate increment ratios.	Investigate the data source immediately. If critical, delay the distribution run until data is complete or use manual overrides based on expert judgment.
Zero sales or zero Average Selling Price (ASP) for a category	If there's no revenue or units sold for a combination, the "Sales Potential Quantity" (Quantum) and thus the increment ratio cannot be calculated, possibly defaulting to no increment.	Review if the product is new or has data capture issues. Consider using a default increment or relying on a broader category average in such cases.
Zero planogram capacity for a category	If a category has no defined planogram space, the calculation for the increment ratio will be invalid, as it cannot divide by zero.	Verify and update planogram configurations for all relevant categories. Ensure that all products expected to be displayed have allocated space.
Very low warehouse inwards for a high-demand category	If warehouse inventory is extremely low, the regional supply component of the increment ratio will be small, limiting the recommended increase despite high sales potential.	Investigate supply chain issues for the product. While the ratio is accurate, the business might need to prioritize inbound shipments to leverage demand.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All foundational master data (products, stores) and transactional data (sales, inventory) loaded and prepared.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned sales data and accurate Rate of Sale (ROS) calculations, fundamental for revenue and ASP calculations.*
- **Distribution Compute Revenue:** What we need from it: *Calculated revenue per day for store-styles and aggregated levels, and Optimum Depth (OD) segmentation data, providing core inputs for “Sales Potential Quantity” (Quantum).*
- **Distribution Allocation:** What we need from it: *Overall allocation strategy and current inventory levels, which are later used to apply the calculated planogram increments.*

After This Step

- **Allocation Optimization Module:** What they need from us: *The calculated planogram increment ratios and “Sales Potential Quantities” (Quantum) to dynamically adjust allocation decisions, potentially increasing stock allocated to stores beyond their initial planogram capacity.*
- **Visual Merchandising & Store Planning:** What they need from us: *Insights on which product categories require increased display space, aiding in future store layout adjustments and visual merchandising strategies.*

10. Common Questions

Q: What does “planogram incrementer” actually mean for my store? A: It means we’re using sales data to figure out which products or categories are selling so well that they need more space on your shelves. So, if a type of T-shirt is flying off the racks, this system calculates how much more of that T-shirt we should plan to display and stock in your store to meet that high demand and prevent stockouts.

Q: How is this “Sales Potential Quantity” (Quantum) different from just looking at units sold? A: While units sold is important, “Sales Potential Quantity” (Quantum) gives a more holistic view. It’s calculated by dividing the total revenue a product made by its Average Selling Price (ASP). This means it accounts for how much money a product brings in, not just how many units, providing a smarter way to assess its true sales contribution and potential for growth.

Q: Why do you consider warehouse inventory when recommending planogram increments? A: It’s practical. There’s no point in telling a store to display more of a product if we don’t have enough stock in our warehouses to support that increase. By considering available warehouse inventory (“inwards”) against overall regional demand, we ensure that our recommendations for expanding planogram space are realistic and achievable, preventing empty displays even after increasing capacity.

Q: Can this module recommend decreasing planogram space? A: This module is primarily designed to *increment* planogram space based on positive sales performance and available supply. It doesn't directly recommend *decreases*. However, by *not* recommending an increment for underperforming items, it indirectly highlights areas where space might be better utilized by other, higher-performing products. Decisions to *decrease* planogram space usually come from different modules focused on managing underperformers or optimizing overall store layout.

Q: How does this module help improve profitability? A: By systematically identifying and recommending more display space for your best-selling and most profitable items, this module directly boosts sales. It prevents situations where customers want to buy more of a popular product but can't because it's out of stock or not prominently displayed. This efficient allocation of valuable shelf space translates directly into higher revenue and better inventory turns, increasing overall profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate data-driven increment ratios for store planograms based on sales performance and available inventory.
- **Main Input:** Historical sales data (revenue, units), planogram configurations, product attributes, and warehouse inventory levels.
- **Main Output:** Planogram increment ratios and calculated "Sales Potential Quantities" (Quantum) for various product levels.
- **Warning Signs:** Unexpectedly low or high increment ratios for established products, or recommendations for products with very low warehouse stock.
- **Key Settings:** Sales Analysis Period for Increment, Minimum Sales for Quantum Calculation, Attribute Grouping for Analysis.

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 15: Distribution Output

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for generating and saving all final outputs from the distribution process, transforming the system's allocation decisions and inventory states into various easy-to-understand reports and data tables.

Why This Matters: This crucial step ensures that all the complex distribution decisions are presented in a clear, actionable format. It provides critical information for store managers, warehouse teams, and planners, enabling them to execute inventory movements, analyze performance, and make informed business decisions that maximize sales and minimize stockouts.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Calculated Rate of Sale (ROS) and Revenue per Day:** *These processed metrics from earlier steps are included in final reports to show how quickly and profitably products are selling, providing context for allocation decisions.*

Product Data

- **Product Definitions and Attributes (SKU, Style, Category, AG):** *Accurate details about each unique item and its classifications are essential for correctly labeling and grouping products in all generated reports.*
- **Product Hierarchy (Parent Styles):** *Information on how products relate to broader styles ensures that reports can be generated at both detailed SKU (size) and summary style levels.*

Operational Data

- **Final Allocation Decisions (SKU per Store):** *The core output from the allocation module, detailing exactly how many of each unique item should be sent to every store.*
- **Planned Inventory Transfers (Inter-Warehouse and Inter-Store):** *The complete plans for moving products between warehouses and directly between stores, ready for execution and reporting.*

- **Current Inventory States (Store & Warehouse):** *The starting and resulting inventory levels in stores and warehouses, including any strategically reserved quantities, providing a clear picture of stock changes.*
- **Store-Style Segment Assignments:** *The classifications (e.g., TOP_SELLER, BOTTOM_SELLER) that explain why certain allocation or transfer decisions were made, adding context to reports.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Include Allocation Details (SKU-level)	True	Determines if the output reports include granular details down to each unique SKU (size) allocated to a store.	To adjust the level of detail provided to store operations or for specific analyses.
Include Warehouse Stock Summaries	True	Controls whether comprehensive summaries of available warehouse stock and strategically reserved quantities are generated.	When needing to track warehouse inventory health and strategic reserves.
Generate Fill Rate Analysis Reports	True	Specifies if reports analyzing how well stores are filled against their Optimum Depth (OD) and Ideal Size Set (ISS) targets are produced.	To evaluate the effectiveness of allocation in meeting target inventory levels.
Generate Overstocking Insights	True	Determines if reports highlighting potential overstocked items based on allocation outcomes are created.	To proactively identify and manage excess inventory in stores.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Final Allocation Decisions & Inventory States] -->
B[Distribution Output Generation]
B --> C[Comprehensive Reports & Data Tables]
```

1. Start with → The system's final decisions on how much product to send to each store, all planned inventory transfers, and the updated inventory counts.
2. Then → The system gathers all these in-memory results, organizing them by store, product, and movement type.
3. Next → It structures and formats this organized data into various detailed reports and flat data tables.
4. Finally → Comprehensive, ready-to-use outputs that can be sent to other systems or reviewed by business teams for analysis and action.

5. Step-by-Step Actions

1. **Consolidate** All Distribution Decisions:
 - What we do: *We gather all the final allocation quantities for every SKU at each store, along with all planned inter-warehouse and inter-store transfers.*
 - Why we do it: *This brings together the complete picture of all recommended inventory movements.*
2. **Process** Warehouse Allocation Outputs:
 - What we do: *We compile detailed reports showing which warehouse sent what quantity of each SKU to which store, considering any strategic reserves (reserveQtyMap) that influenced available stock.*
 - Why we do it: *This provides a clear record for warehouse operations to execute shipments and for stores to anticipate incoming inventory.*
3. **Generate** Detailed Store Allocation Reports:
 - What we do: *We create comprehensive reports for each store, listing every SKU allocated, its planned quantity, current in-store stock, Rate of Sale (ROS), and relevant remarks (e.g., if it's a TOP_SELLER).*
 - Why we do it: *This gives store managers a precise plan for receiving and managing their new inventory.*
4. **Summarize** Inter-Warehouse Movements:
 - What we do: *We produce reports detailing all planned transfers between distribution warehouses, including source and destination warehouses, SKU, quantity, and the type of transfer (e.g., demanding pullback, other pullback).*
 - Why we do it: *This enables efficient rebalancing of inventory across the entire warehouse network.*
5. **Compile** Inter-Store Transfer Records:
 - What we do: *We generate reports on all planned direct transfers between retail stores, showing which store will send excess stock to another store with demand, and identifying any stock designated for pullback to a central warehouse.*
 - Why we do it: *This helps optimize inventory utilization within regional store clusters and identifies unproductive stock for consolidation.*

6. Create Performance and Health Reports:

- What we do: *We generate reports such as fill rate analyses (how well stores meet their Optimum Depth (OD) and Ideal Size Set (ISS) targets), store display health assessments (e.g., Post-IST PSA), and overstocking insights.*
- Why we do it: *These reports provide crucial feedback on the effectiveness of distribution decisions and highlight areas for improvement in future runs.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- The distribution system has just completed its run, deciding to:
 - Send 50 units of "Summer Dress, Style #SD001," Size M, from Warehouse A to Store X. (Store X is a TOP_SELLER for this style).
 - Transfer 10 units of "Winter Jacket, Style #WJ002," Size L, from Store Y (excess stock) to Store Z (demand).
 - Identify 5 units of "Clearance Jeans, Style #CJ003," Size S, at Store B for pullback to Central Warehouse.
 - Warehouse A has a strategic reserve of 20 units of "New Launch Sneaker, Style #NLS001" for online sales.

What Happens

1. First, the system **gathers** the final allocation of 50 "Summer Dress" units for Store X, noting its TOP_SELLER status and the sending warehouse.
2. Then, it **records** the inter-store transfer of 10 "Winter Jacket" units from Store Y to Store Z, ensuring both stores are correctly identified for the movement.
3. Next, it **logs** the 5 "Clearance Jeans" units for pullback from Store B, categorizing them as excess to be returned to the central warehouse.
4. Following this, the system **captures** the 20 units of "New Launch Sneaker" specifically held back in Warehouse A as a strategic reserve.
5. Finally, it **formats** all these decisions into structured reports:
 - A "Store X Allocation Report" showing the 50 "Summer Dress" units and their optimal size breakdown.
 - A "Warehouse A Pick List" detailing the 50 "Summer Dress" units for outbound shipment.
 - An "Inter-Store Transfer Plan" listing the jacket movement from Store Y to Store Z.
 - A "Store Pullback List" for Store B, identifying the jeans for return.
 - A "Warehouse Stock Summary" for Warehouse A, clearly showing its total inventory minus the 20 reserved "New Launch Sneaker" units.

Final Outcome

All allocation and transfer decisions are transformed into clear, actionable reports and data, ready for immediate use. The store manager at Store X knows exactly what to receive, the warehouse team knows what to pick, and inventory planners can see the strategic reserves, ensuring a smooth flow of goods and informed future planning.

Key Takeaway: This module bridges the gap between complex calculations and real-world execution, providing transparent and actionable inventory plans for the entire retail operation.

7. Results

This step produces:

Main Outputs

- **Final SKU Allocations per Store:** The precise, approved quantities of each unique product (SKU) to be allocated to every retail store.
- **Planned Inventory Movements (Warehouse to Store, Store to Store, Pullbacks):** Detailed records of all product transfers, including source, destination, SKU, and quantity, ready for logistics execution.
- **Warehouse Stock Summaries with Reserves:** Comprehensive views of inventory in distribution centers, distinguishing between immediately available stock and strategically reserved quantities.
- **Fill Rate and Overstocking Analysis:** Data indicating how well stores are projected to meet their Optimum Depth (OD) and Ideal Size Set (ISS) targets, and identifying potential overstocking.
- **Store Health and Performance Flags:** Reports incorporating updated store display health (PSA) and style segmentation (TOP_SELLER, BOTTOM_SELLER) for analytical purposes.

Business Reports

- **Store Allocation Report:** Used by store managers to review incoming inventory, plan for receipts, and understand their future stock levels.
- **Warehouse Pick List:** Used by warehouse operations to efficiently pick and pack products for outbound shipments based on the approved allocation plan.
- **Inter-Store Transfer Plan:** Used by store operations and logistics to execute direct product movements between retail locations.
- **Inventory Health & Performance Dashboard:** Used by inventory planners, merchandisers, and executives to monitor the effectiveness of distribution strategies, track key performance indicators, and inform strategic decisions.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data or report that essential prior module outputs are missing, potentially leading to incomplete or inaccurate reports. The process of generating and	Investigate the upstream data source immediately. If critical, delay the output generation until data is complete or use manual overrides to generate critical reports.

Situation	What Happens	What You Should Do
Large data volumes impact reporting speed	persisting all outputs may take longer, delaying the availability of final reports.	Optimize reporting infrastructure or consider generating reports in batches during off-peak hours.
Inconsistencies between different output reports	Due to unforeseen internal data issues, a quantity in one report might not perfectly match another.	Review the internal data integrity checks and reconcile the discrepancy to ensure accuracy across all outputs.
Reports are too granular or not granular enough	The level of detail in the reports doesn't match the needs of the end-user (e.g., too many individual SKUs, or not enough summary data).	Adjust the "Include Allocation Details" or other granularity settings in the business settings for future runs.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All foundational master data (products, stores, warehouses) and transactional data (inventory) in a clean, prepared state.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned sales data and accurate Rate of Sale (ROS) calculations for performance insights.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels, accounting for discrepancies and lead time reservations.*
- **Distribution Warehouse Reservation:** What we need from it: *Rules for strategically holding back a percentage of warehouse inventory.*
- **Distribution Compute Revenue:** What we need from it: *Calculated revenue per day for store-styles and initial size depths.*
- **Distribution Ranking:** What we need from it: *The prioritized lists of store-styles and SKUs based on their business value.*
- **Distribution Segmentation:** What we need from it: *The assigned store-style segments (TOP_SELLER, BOTTOM_SELLER, NORMAL_SELLER) for differentiated allocation rules.*
- **Distribution Allocation:** What we need from it: *The core module producing the final SKU allocations per store.*
- **Distribution Compute Warehouse Inventory:** What we need from it: *Centralized inventory data for Inter-Store Transfers (IST), creating "virtual warehouses" from excess store stock.*
- **Distribution Inter-Warehouse Transfer (IWHT):** What we need from it: *Optimized recommendations for product movements between warehouses.*
- **Distribution Inter-Store Transfer (IST):** What we need from it: *Optimal plans for direct inventory movements between retail stores.*

- **Distribution Planogram Incrementer:** What we need from it: *Calculated planogram increment ratios and “Sales Potential Quantities” (Quantum) for updated display capacity targets.*

After This Step

- **Logistics and Fulfillment Systems:** What they need from us: *The final allocation quantities and transfer plans to generate pick lists, shipping labels, and manage all inbound/outbound warehouse and store operations.*
- **Store Inventory Management Systems:** What they need from us: *The planned inbound inventory to update store stock levels, aid in receiving processes, and inform in-store merchandising.*
- **Business Intelligence & Analytics Platforms:** What they need from us: *The comprehensive data tables for post-allocation analysis, performance monitoring, and continuous improvement of inventory strategies.*
- **Finance & Accounting Systems:** What they need from us: *Detailed inventory movement records for financial reporting and asset valuation.*

10. Common Questions

Q: What kinds of reports does this module create? A: This module creates a variety of reports, including detailed plans for what products (down to each size) should be shipped to each store, summaries of inventory movements between warehouses, and records of products being transferred directly between stores or pulled back to a central warehouse. It also generates reports on how well stores are stocked against their ideal targets (fill rates) and flags any potential overstocking.

Q: Who uses these reports, and how do they help them? A: Store managers use these reports to prepare for incoming shipments and manage their store’s inventory. Warehouse teams use them to know exactly what to pick and pack. Merchandisers and planners use them to understand how well products are performing and to assess the overall health of our inventory across all locations. This helps everyone work together efficiently to get products to customers.

Q: How often are these output reports generated? A: The frequency depends on our business needs, but typically these reports are generated after every complete distribution run. This might be daily, weekly, or bi-weekly, ensuring that the latest inventory decisions are always available and actionable for our operational teams.

Q: Can we customize the content or format of these reports? A: The module generates a standard set of comprehensive reports. While direct customization within this module isn’t typically offered, the underlying data is provided in a structured format, which can often be used by our business intelligence tools to create custom dashboards and analyses tailored to specific needs.

Q: How does this module help us understand our inventory performance? A: By providing detailed reports on where products are going, how much stock is in each location, and even insights into fill rates and overstocking, this module allows us to evaluate the success of our distribution strategies. This transparency helps us identify what’s working well and where we can improve to maximize sales, reduce wasted inventory, and boost profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To generate and save all final distribution decisions and inventory states into actionable reports and data tables.
- **Main Input:** Final allocation quantities, transfer plans, and all updated inventory states from previous distribution modules.
- **Main Output:** Detailed SKU allocations per store, planned inventory movements, warehouse stock summaries, and performance analysis reports.
- **Warning Signs:** Delays in report generation, missing data in reports, or inconsistencies between different output documents.
- **Key Settings:** Options to include detailed allocation breakdowns, warehouse stock summaries, fill rate analysis, and overstocking insights.

Need Help? Contact your inventory planning or IT support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 17: Distribution Iteration Runner

Last Updated: 2025-05-23

1. Business Objective

This module orchestrates and executes various distinct inventory allocation strategies, such as initial replenishment, fresh allocation for new products, deepening stock levels, managing non-pivotal sizes, and ensuring balanced color distributions.

Why This Matters: By precisely applying specific business rules for each allocation scenario, this module ensures that inventory is optimally placed to maximize sales, improve product visibility, and efficiently manage stock throughout its lifecycle in stores. It allows for highly targeted adjustments beyond a single, generic allocation rule, directly boosting profitability and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS) per SKU per Store:** *To understand how quickly individual products are selling at specific stores, which is fundamental for guiding replenishment quantities within each allocation strategy.*
- **Non-Zero Sales data for Store-SKU combinations:** *To ensure that if an item had even one sale, it gets considered for minimal allocation, preventing it from being overlooked by stricter rules.*

Product Data

- **Optimum Depth (OD) Rules:** *To define the ideal target quantity of each product to stock in a store, serving as a primary ceiling for allocation quantities in various iterations.*
- **Ideal Size Set (ISS) Rules (including Pivotal Tagging):** *To determine the best combination of sizes and identify core (pivotal) versus non-core (non-pivotal) sizes, which is crucial for guiding precise size-level allocations within different iterations.*

- **Style Information (Pack Size, Colors, Categories):** *To apply specific allocation rules based on how products are packaged, their color groupings, and their broader categories for specialized iterations.*

Operational Data

- **Current Store and Warehouse Inventory Levels:** *To know what stock is already available in stores and what can be allocated from warehouses, ensuring realistic distribution decisions for each iteration.*
- **Store Planogram Configurations:** *To understand the current display capacity in stores and guide potential increments to maximize product visibility during certain allocation strategies.*
- **Allocation Strategy Flags (e.g., Replenishment, Fresh Allocation, Depth Increment, Planogram Correction, JIT):** *To inform this module which specific set of rules and logic to apply for the current distribution run.*
- **Store-Style Segments (TOP_SELLER, BOTTOM_SELLER):** *To apply differentiated allocation strategies based on a product's performance at a specific store during various allocation runs.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Current Iteration Strategy	REPLENISHMENT	Defines the specific allocation approach for the current distribution run, such as standard replenishment, fresh allocation for new products, or strategic depth increment.	When a new product launch, seasonal shift, or operational objective requires a different allocation logic.
Depth Increment Multiplier	N/A (e.g., 1.5)	A factor used to increase allocated units beyond the initial Optimum Depth (OD) to build deeper stock levels in stores.	When building stock for peak seasons, promotions, or to ensure robust inventory for top-selling items.
Top Seller Cover Days Multiplier	N/A (e.g., 1.2)	A factor applied to the Rate of Sale (ROS) for TOP_SELLER items to ensure they receive	To prioritize and provide deeper stock for top-performing products, ensuring

Setting Name	Default Value	Description	When to Change
Attribute Group Roll-up Level	N/A (e.g., 0)	additional days' worth of stock, ensuring longer coverage.	they are always available.
		Specifies the level of detail at which Attribute Group (AG) criteria are consolidated and applied during allocation (e.g., category, subcategory, brand).	To adjust the granularity of how broad product groups influence specific allocation decisions.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Prepared Allocation Data & Strategy Flags] --> B[Distribution Iteration Runner]
    B --> C[Precise SKU Allocations & Updated Planogram Criteria]
  
```

1. Start with → Precise allocation criteria for stores and products, along with the specific allocation strategy (e.g., replenishment, depth increment) for the current run.
2. Then → The system initializes for the chosen strategy and systematically runs through various allocation loops, focusing on different aspects like non-pivotal sizes, core replenishment, and color-based allocations.
3. Next → It applies specific rules for suggested quantities, adjusts for existing store stock, and updates planogram capacity criteria based on the calculated allocations.
4. Finally → Detailed, refined SKU allocations for each store, with updated inventory targets and performance metrics, ready for execution.

5. Step-by-Step Actions

1. **Initialize** for the Specific Allocation Strategy:
 - What we do: *The system sets up the internal environment based on the chosen allocation strategy for the current run (e.g., replenishment, fresh allocation, non-pivotal sizes, planogram correction), loading all relevant business rules and data for that iteration.*
 - Why we do it: *This ensures the allocation process is correctly configured for the specific business objective of the current distribution run.*

2. **Process** Non-Pivotal Size Allocations:

- What we do: *The system identifies and allocates stock for non-pivotal (less critical but still important) sizes at both individual SKU and broader category-size levels, ensuring these items receive appropriate, often minimal, stock to complete size ranges.*
- Why we do it: *This ensures that even less common sizes are available to customers, preventing missed sales opportunities and improving overall customer choice.*

3. **Refine** Attribute Group (AG) Criteria:

- What we do: *The system adjusts and consolidates Attribute Group (AG) data (like Optimum Width (OW) and Optimum Depth (OD) targets for categories) by rolling up information to broader levels, ensuring that allocations consider performance across wider product groupings.*
- Why we do it: *This provides a more holistic view of performance for broader product categories, allowing for smarter allocation decisions when granular data might be limited.*

4. **Execute** Core and Strategic Allocations:

- What we do: *The system runs the main allocation logic for each store-style combination, calculating suggested quantities for each individual product (SKU) based on its Optimum Depth (OD), Rate of Sale (ROS), Ideal Size Set (ISS), and strategic segments like TOP_SELLER or JIT (Just-in-Time).*
- Why we do it: *This multi-layered approach ensures that allocations are balanced, addressing immediate replenishment needs while also optimizing for sales potential and strategic priorities.*

5. **Manage** Color-Based Allocations:

- What we do: *For products defined by specific color groups, the system applies special rules to ensure a minimum number of styles within a color group are allocated to stores, optimizing visual appeal and assortment breadth.*
- Why we do it: *This helps maintain a visually appealing and balanced product assortment in stores, which can directly influence customer browsing and purchasing decisions.*

6. **Update** Store and Planogram Stock:

- What we do: *After each allocation decision, the system updates the internal store stock records and adjusts planogram criteria, reflecting the newly allocated quantities and ensuring that subsequent calculations are based on the latest inventory picture.*
- Why we do it: *This maintains an accurate, real-time internal view of inventory, preventing over-allocations and ensuring that future calculations are based on the actual stock situation.*

7. **Handle** Remaining Stock and In-Transit Items:

- What we do: *The system accounts for any leftover physical stock in stores (often to be 'zeroed out' for new allocations) and incorporates products already in transit or committed to customer orders, ensuring these existing or incoming units are not re-allocated unnecessarily.*
- Why we do it: *This prevents double-counting inventory, ensures that only truly available stock is considered for new allocations, and maintains the accuracy of inventory records.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new running shoe style, “FlexStride, Style #RUN001,” is available in sizes US 5-12.
- Ideal Size Set (ISS) rules classify US 8-10 as ‘Pivotal’ (core sizes, essential for demand) and US 5-7, 11-12 as ‘Non-Pivotal’ (less critical but still needed for complete range).
- Store A is a high-volume store; it initially received allocations for its ‘Pivotal’ sizes of #RUN001 in a previous allocation run.
- Store A currently has: 5 units of Size 8, 5 units of Size 9, and 5 units of Size 10 (total 15 units of Pivotal sizes for #RUN001).
- Store A has 0 units of any ‘Non-Pivotal’ sizes for #RUN001.
- The current distribution run is specifically configured for the “Non-Pivotal Sizes” iteration.

What Happens

1. First, the system **identifies** that the current run’s objective is to allocate “Non-Pivotal Sizes” for Style #RUN001.
2. Then, it **reviews** Style #RUN001 at Store A. It sees that sizes US 5, 6, 7, 11, and 12 are currently missing and are designated as ‘Non-Pivotal’ according to the Ideal Size Set (ISS) rules.
3. Next, for each of these ‘Non-Pivotal’ sizes, the system **calculates** a suggested allocation quantity. Even if sales for these specific sizes are very low or zero, the system aims to ensure a minimal presence (e.g., 1 unit per non-pivotal size) to complete the overall size offering for the style in the store.
4. Following this, for each calculated allocation (e.g., 1 unit of Size 5, 1 unit of Size 6, etc.), the system **updates** Store A’s internal inventory record, setting the suggested allocation for these non-pivotal sizes and effectively earmarking them for shipment.
5. Finally, the system **flags** these newly allocated non-pivotal sizes as “resolved” for this iteration, meaning they have received their appropriate stock based on the “Non-Pivotal Sizes” strategy.

Final Outcome

Store A receives a small, strategic allocation of “Non-Pivotal” sizes (US 5-7, 11-12) for the “FlexStride” running shoe. This ensures that the store can offer a complete size range to customers, even for less common sizes, preventing missed sales opportunities and improving customer satisfaction, without tying up excessive inventory in slow-moving items.

Key Takeaway: This module precisely targets and allocates inventory for specific business objectives, such as completing a product’s size offering, ensuring full range availability and optimizing customer choice.

7. Results

This step produces:

Main Outputs

- **Final SKU Allocations per Store (per Iteration):** The precise, approved quantities of each unique product (SKU) recommended for allocation to every retail store, reflecting the specific rules applied by the current iteration (e.g., for non-pivotal sizes).
- **Updated Planogram Criteria:** Adjustments to the targeted display space for products based on the current iteration's allocation decisions.
- **Color Allocation Assignments:** Specific flags or quantities indicating how products were allocated to ensure balanced color assortments where applicable.
- **Style-Level Iteration Status:** Internal flags on store-styles indicating which allocation iteration (e.g., replenishment, non-pivotal) has successfully processed them.

Business Reports

- **Allocation Summary by Iteration:** Used by merchandisers and planners to review the impact and effectiveness of specific allocation strategies, such as how many non-pivotal sizes were allocated or how depth was increased.
- **Dynamic Planogram Adjustment Report:** Used by store operations and visual merchandising teams to understand how display space targets are evolving based on strategic allocation runs, aiding in store layout planning.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential sales, inventory, or product data is missing, potentially leading to incomplete or inaccurate allocation decisions.	Investigate the data source immediately to restore the feed. If critical, delay the distribution run until data is complete or use manual overrides.
No relevant styles/SKUs for a specific iteration	If, for example, a "Non-Pivotal Sizes" iteration runs but no products have missing non-pivotal sizes, the iteration will complete with no allocations.	This is expected behavior; ensure the iteration's purpose aligns with current inventory needs.
Conflicting business rules (e.g., JIT vs. TOP_SELLER)	The system applies a predefined hierarchy (e.g., JIT rules might prioritize single-unit allocation even for a TOP_SELLER). The outcome will be logged.	Review the rule hierarchy and adjust business settings if the resulting allocations don't align with strategic intent.

Situation	What Happens	What You Should Do
Warehouse stock insufficient to meet suggested allocations	The system will allocate up to the available warehouse stock, but the full suggested quantity may not be met. This will be reflected in the final allocation.	Review future inbound shipments and consider adjusting Optimum Depth (OD) targets or running a follow-up allocation once stock arrives.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *All foundational master data (products, stores) and transactional data (sales, inventory) loaded and prepared.*
- **Distribution Sales Preparation:** What we need from it: *Cleaned sales data and accurate Rate of Sale (ROS) calculations, crucial for suggested allocation quantities.*
- **Distribution Warehouse Inventory Preparation:** What we need from it: *Adjusted and truly available warehouse inventory levels, accounting for discrepancies and lead time reservations.*
- **Distribution Warehouse Reservation:** What we need from it: *Rules for strategically holding back a percentage of warehouse inventory, influencing the total stock considered available for general distribution.*
- **Distribution Compute Revenue:** What we need from it: *The calculated revenue per day for store-styles and initial size depths, which influence allocation priorities.*
- **Distribution Ranking:** What we need from it: *The prioritized lists of store-styles and SKUs based on their business value.*
- **Distribution Segmentation:** What we need from it: *The assigned store-style segments (TOP_SELLER, BOTTOM_SELLER, NORMAL_SELLER) for differentiated allocation rules.*
- **Distribution Allocation:** What we need from it: *This module is typically called by the overarching Distribution Allocation module to execute its specific iteration types.*

After This Step

- **Distribution Output:** What they need from us: *The precise SKU allocations determined by this module, ready to be formatted into reports and sent to logistics and store systems.*
- **Logistics and Fulfillment Systems:** What they need from us: *The final allocation quantities to generate pick lists and manage outbound warehouse operations for direct shipments to stores.*

10. Common Questions

Q: What exactly does an “allocation iteration” mean? A: An allocation iteration refers to a specific, targeted round of calculations within the overall distribution process. Instead of one big calculation, the system breaks down its decisions into different goals. For example, one iteration might focus solely on getting a minimum quantity of all sizes into stores, while another might be about increasing stock for only your very best-selling items. This allows for more precise and strategic inventory decisions.

Q: How does this module handle different types of allocation (e.g., new products vs. regular replenishment)? A: This module handles different types of allocation by using “strategy flags.” When a run starts, it receives a flag telling it what type of allocation it’s doing (e.g., “Fresh Allocation” for new products, “Replenishment” for ongoing stock, “Depth Increment” to build more stock). It then applies a specific set of rules and logic tailored to that particular strategy, ensuring the right approach for each scenario.

Q: What is the role of “non-pivotal sizes” in this process? A: “Non-pivotal sizes” are typically the less common sizes within a product style (like an XS or an XL). This module has specific iterations to ensure these sizes still receive a minimal, strategic allocation. This prevents a store from being completely out of a size, offering a fuller range to customers, even if those sizes don’t sell as quickly as the “pivotal” (core) sizes.

Q: How does this module ensure that colors are adequately represented in stores? A: For products where color is a key attribute, this module incorporates “color-based allocation” logic. It checks if a store has enough styles from a particular color group to meet a minimum display requirement. If not, it will prioritize allocating additional items from that color group to ensure a balanced and visually appealing color assortment in the store, which helps attract customers.

Q: How does this module impact how much stock my store will receive? A: This module directly impacts your store’s stock by calculating the exact suggested quantity of each individual product (SKU) that should be allocated. It takes into account complex factors like how fast products are selling (ROS), your ideal stock levels (OD), the best mix of sizes (ISS), and any specific strategic goals for that allocation run (like getting deeper stock for top sellers or ensuring full color assortments). It essentially translates all these business rules into a precise number of units for your store.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To execute specific, strategic inventory allocation iterations (e.g., replenishment, depth increments, non-pivotal sizes, color-based allocations).
- **Main Input:** Allocation Strategy Flags, prepared store/product/sales data, and updated inventory states from previous modules.
- **Main Output:** Precise SKU allocations adjusted for specific iteration goals, and updated planogram criteria.
- **Warning Signs:** Unexpectedly low or high allocations for certain product types or stores, or allocations not aligning with the chosen iteration strategy.

- **Key Settings:** Current Iteration Strategy, Depth Increment Multiplier, Top Seller Cover Days Multiplier.
-

Need Help? Contact your merchandising or planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 18: Distribution IST Runner

Last Updated: 2025-05-23

1. Business Objective

This module is a specialized component that manages inventory allocations specifically for Inter-Store Transfers (IST), ensuring that products are efficiently moved directly between retail locations to balance stock. It customizes the standard allocation process to incorporate IST-specific rules, such as confirming available inventory from stores designated as “virtual warehouses” and verifying outward transfer criteria.

Why This Matters: This crucial specialization ensures that valuable inventory is reallocated from stores where it's not selling to stores where it is in high demand, maximizing sales potential, improving inventory utilization, and enhancing overall inventory health across your retail network. It prevents stock from sitting idle and minimizes the need for costly markdowns.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records (Units Sold for Source/Target Stores):** *While not directly used by this specific module for calculation, previous modules provide processed sales data that defines which stores have “excess” (lower sales) and which have “demand” (higher sales), guiding where transfers are needed.*

Product Data

- **Product Definitions and Attributes (SKU and Style):** *To identify each unique item and its broader style, ensuring accurate tracking of inventory quantities and enabling transfers for specific product lines.*
- **Pack Size Data:** *Specifies the standard quantity in which a product is packaged or typically transferred, essential for determining the most efficient units for movement between stores.*

Operational Data

- **Current Store Inventory Levels (Excess & Needed):** *Provides the precise quantity of products currently held at each store, identifying both excess stock*

available for transfer out and quantities needed for transfer in, as determined by previous allocation steps.

- **Store Geographic Grouping (Region, City, or Custom Group):** Information on how stores are geographically or strategically clustered, which dictates the boundaries for inter-store transfer opportunities and forms the basis for “virtual warehouses” of excess stock.
- **“Virtual Warehouse” Stock (from previous modules):** The consolidated pool of excess, transferable inventory from various stores, aggregated into virtual regional warehouses, from which this module draws available stock.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Inter-Store Transfer (IST) Grouping Level	N/A (e.g., CITY)	Defines how stores are grouped for Inter-Store Transfers (IST), such as by City, Region, or a Custom Group.	When new geographical strategies are implemented or custom store groupings are required for transfers.
Minimum Leftover Stock for Transfer	0 units	The smallest quantity of a product style that must remain at a store (after accounting for its internal pack size) to be considered available for outward transfer.	To prevent transferring inefficiently small quantities or to prioritize full pack sizes for transfers.
Default Transfer Pack Size	1 unit	The default unit quantity used for transferring items if a specific style’s pack size is not explicitly defined.	When there’s a need for a system-wide fallback for product transfer quantities for transfers.
Enable Transfer-Based Replenishment	True	Controls whether the system executes detailed replenishment iterations (like planogram or depth increments)	When a business wants to leverage internal store-to-store transfers for restocking, rather than relying solely on warehouse shipments.

Setting Name	Default Value	Description	When to Change
		using only inventory available through inter-store transfers.	

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Prepared Store Inventory (Excess & Demand) & Grouping Data]
--> B[Distribution IST Runner]
    B --> C[Precise SKU Allocations for Inter-Store Transfers]

```

1. Start with → Prepared store inventory data (excess stock from some stores, demand from others), product definitions (including pack sizes), and defined store transfer groups.
2. Then → The system customizes its allocation calculations, specifically focusing on how to determine available stock from the pooled “virtual warehouse” of excess store inventory.
3. Next → It applies unique logic to ensure that allocated quantities for inter-store transfers adhere strictly to specified product pack sizes, making movements efficient.
4. Finally → Precise SKU allocations for inter-store transfers are generated, respecting all IST-specific constraints and maximizing efficient stock reallocation.

5. Step-by-Step Actions

1. **Prepare** for Inter-Store Allocation:
 - What we do: *The system initializes its internal environment, specifically configuring itself to calculate allocations for transfers directly between retail stores.*
 - Why we do it: *This ensures the allocation process is correctly tailored to the unique business objective of rebalancing inventory across the store network.*
2. **Determine** Available Inter-Store Inventory:
 - What we do: *The system accurately calculates how much stock is truly available for transfer from other stores. This involves querying the “virtual warehouse” of excess items that was identified and pooled in previous steps, ensuring only deployable inventory is considered.*
 - Why we do it: *This provides a realistic view of the supply available for inter-store movements, preventing over-allocations and ensuring transfer plans are executable.*
3. **Calculate** Transferable Quantities:
 - What we do: *For each item a store needs, the system determines the ideal quantity to transfer. This calculation considers the store's maximum stock*

limit (Optimum Depth) and the gap between its current inventory and its desired inventory level.

- Why we do it: *This ensures that demanding stores receive enough stock to meet their needs without exceeding their optimal capacity.*

4. **Adjust** for Pack Sizes:

- What we do: *The calculated transfer quantities are precisely adjusted to ensure they are always in full, efficient “pack sizes” (e.g., a style always moves in bundles of 5, not 3 units). This prevents inefficient transfers of partial quantities.*
- Why we do it: *This streamlines logistical operations, minimizes handling costs, and ensures that transfers are always in complete, manageable units, improving efficiency and reducing potential errors.*

5. **Finalize** Outward Transfer Allocation:

- What we do: *The system makes the final decision on the precise quantity to transfer outwards from the virtual warehouse to a demanding store. It ensures the allocated quantity does not exceed the available virtual stock, the demanding store’s capacity, and is strictly in valid pack sizes.*
- Why we do it: *This creates an actionable, accurate transfer plan that can be executed by logistics and store teams, directly optimizing store inventory levels.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Store A (NYC Downtown)** needs “Women’s Denim Shirt, Style #WDS001, Size M”. It requires 12 units to reach its Optimum Depth (OD). It currently has 0 units of this SKU. Its max depth for this SKU is 10 units.
- The “New York City” regional virtual warehouse (pooled from other NYC stores) has 15 units of “Women’s Denim Shirt, Style #WDS001, Size M” available for transfer.
- The standard “Pack Size” for “Women’s Denim Shirt” is 5 units.

What Happens

1. First, the system **identifies** that Store A has a demand for 12 units of Style #WDS001, Size M.
2. Then, it **determines** the maximum allocation Store A can receive, which is its maximum Optimum Depth of 10 units for this SKU (since it currently has 0 units in stock).
3. Next, it **checks** the available stock in the “New York City” virtual IST warehouse for this SKU, which is 15 units.
4. Following this, the system **calculates** the quantities in terms of “packs.”
 - The requested 12 units for Store A would conceptually require $\text{ceil}(12 \text{ units} / 5 \text{ units/pack}) = 3 \text{ packs}$.
 - The maximum allowable allocation for Store A (10 units) translates to $\text{floor}(10 \text{ units} / 5 \text{ units/pack}) = 2 \text{ packs}$.
 - The available virtual IST stock (15 units) is $15 \text{ units} / 5 \text{ units/pack} = 3 \text{ packs}$.

5. Finally, the system **compares** these pack quantities: 3 (needed packs), 2 (max alloc packs), and 3 (available virtual packs). It chooses the minimum, which is 2 packs. Therefore, it decides to send $2 \text{ packs} * 5 \text{ units/pack} = 10 \text{ units}$ of “Women’s Denim Shirt, Style #WDS001, Size M” to Store A. It prioritizes filling the store up to its optimal capacity in full, efficient pack sizes.

Final Outcome

Store A successfully receives 10 units of “Women’s Denim Shirt, Style #WDS001, Size M” from the regional virtual IST warehouse. This allocation perfectly aligns with the store’s optimal depth and is delivered in efficient, full pack sizes, ensuring that the store is optimally stocked to meet customer demand while minimizing operational waste.

Key Takeaway: This module precisely allocates inventory between stores, ensuring quantities align with optimal store levels and are transferred in efficient pack sizes, maximizing sales and minimizing operational waste.

7. Results

This step produces:

Main Outputs

- **Optimized SKU Allocations for Inter-Store Transfers:** The precise, approved quantities of each unique product (SKU) recommended for transfer between specific source and destination retail stores, reflecting all IST-specific rules.
- **Updated Store Inventory Levels:** Internal records reflecting the planned changes to individual store stock due to these IST allocations.
- **Pack-Size Aligned Transfer Quantities:** Confirmation that all proposed inter-store transfers adhere to defined product pack sizes, enabling efficient logistics.

Business Reports

- **Inter-Store Transfer Plan:** Used by store operations and logistics teams to execute product movements between stores, ensuring efficient and accurate stock rebalancing.
- **Store-Level Stock Rebalancing Report:** Used by merchandisers and planners to review the impact of inter-store transfers on individual store inventory levels and overall network health.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, or report that essential store inventory data or grouping information is	Investigate the data source immediately to restore the feed. If critical, delay the distribution run until data is complete or use

Situation	What Happens	What You Should Do
	missing, potentially leading to incomplete or inaccurate transfer recommendations.	manual overrides based on expert judgment.
No available virtual IST stock for a demanding SKU	If the “virtual warehouse” has no available excess stock for a needed item, the module will not recommend an inter-store transfer for that SKU.	Review if central warehouse allocation is needed or if external sourcing is required to meet demand.
Suggested allocation quantity is less than a pack size	If the calculated need for an SKU is, for example, 3 units, but its pack size is 5, the module will not recommend a transfer, resulting in zero allocation for that item.	Review the product’s pack size definition. For very small quantities, consider manual transfers or allowing single-unit transfers for specific product types if business rules allow.
Store’s maximum depth is smaller than the product’s pack size	This can cause the system to always allocate zero, as it cannot send a full pack without exceeding the store’s maximum capacity.	Review the store’s Optimum Depth (OD) settings or adjust the product’s pack size if this scenario is unintended.

9. Next Steps

Before This Step

- **Distribution Data Preparation:** What we need from it: *Accurate and up-to-date current store inventory levels, product definitions (including pack sizes), and store geographic or custom grouping information, all prepped for use.*
- **Distribution Compute Warehouse Inventory:** What we need from it: *The initial identification of excess store inventory, pooled into “virtual warehouses” and prepared for outward transfers, setting the stage for inter-store movements.*
- **Distribution Allocation:** What we need from it: *The primary module that calls this specific runner to execute its specialized inter-store transfer logic.*
- **Distribution Inter-Store Transfer (IST):** What we need from it: *The module that typically orchestrates the overall inter-store transfer process, leveraging this runner for specific allocation logic.*

After This Step

- **Distribution Output:** What they need from us: *The precise SKU allocations determined by this module, ready to be formatted into reports and sent to logistics and store systems for execution.*

- **Logistics and Fulfillment Systems:** What they need from us: *The final inter-store transfer recommendations to generate transfer orders and manage inbound/outbound operations directly between retail locations.*

10. Common Questions

Q: How is “Distribution IST Runner” different from the regular “Distribution Iteration Runner”? A: The “Distribution IST Runner” is a specialized version of the general “Distribution Iteration Runner.” While the regular runner handles various allocation strategies (like fresh product launches or regular replenishment from warehouses), the IST runner focuses *only* on inter-store transfers. It has unique logic to understand inventory available from other stores (the “virtual warehouse”) and to ensure transfers happen in efficient pack sizes.

Q: How does this module use inventory from a “virtual warehouse”? A: A “virtual warehouse” isn’t a physical location; it’s a concept used by the system to pool excess inventory from multiple retail stores within a specific region or group. This module then draws from this virtual pool as if it were a central warehouse, allowing it to allocate those excess units to other stores in need, without them having to return to a physical warehouse first.

Q: Why is a product’s “pack size” so important for inter-store transfers? A: A product’s “pack size” specifies the standard quantity in which it’s packaged or efficiently moved (e.g., a shirt might come in packs of 5). This module strictly adheres to pack sizes to ensure transfers are efficient. Sending items in full packs reduces handling costs, minimizes damage, and simplifies inventory reconciliation for both the sending and receiving stores.

Q: What happens if a store needs a quantity that isn’t a full pack size? A: If a store needs a quantity that isn’t a perfect multiple of the pack size (e.g., needs 7 units but pack size is 5), this module will allocate units in full packs. It will typically round down to the nearest full pack that doesn’t exceed the store’s needs or its maximum capacity. The remaining smaller quantity would either stay at the source store or be flagged for a central warehouse pullback by other processes.

Q: How does this module improve inventory utilization across our stores? A: By systematically identifying and facilitating the transfer of excess inventory from one store to another where it’s actually needed, this module ensures that valuable stock doesn’t sit idle. This maximizes the chances of selling products across the entire network, reduces the risk of markdowns on slow-moving inventory, and frees up capital and shelf space in underperforming locations.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To manage and optimize inventory allocations for direct transfers between retail stores, incorporating IST-specific rules and pack-size adherence.
- **Main Input:** Current store inventory (excess and needed), product pack sizes, and store grouping definitions (for virtual warehouses).
- **Main Output:** Precise SKU allocations for inter-store transfers, adhering to pack sizes.

- **Warning Signs:** Transfers not occurring in full pack sizes, or inability to allocate due to pack size constraints vs. store capacity.
 - **Key Settings:** Inter-Store Transfer (IST) Grouping Level, Minimum Leftover Stock for Transfer, Default Transfer Pack Size.
-

Need Help? Contact your inventory management or supply chain team for questions about this process.

Generated by AI Codebase Knowledge Builder

distribution FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document provides high-level answers to common business questions about the distribution algorithm, covering its purpose, how it operates, and what results it delivers.

General

Q: What is the primary goal of the distribution algorithm? **A:** The distribution algorithm's main goal is to optimize the allocation and transfer of inventory across our stores and warehouses. It aims to maximize sales by ensuring the right products are in the right place at the right time, while also minimizing stockouts and excess inventory, considering factors like Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS).

Q: What is Optimum Depth (OD)? **A:** Optimum Depth (OD) refers to the ideal number of unique items (SKUs) that should be stocked in a particular category or displayed in a store. The algorithm uses this to ensure stores have a targeted selection of products.

Q: What is Optimum Width (OW)? **A:** Optimum Width (OW) is the ideal number of different categories or styles that should be offered in a store. This helps the algorithm ensure a balanced product assortment across various types of merchandise.

Q: What is an Ideal Size Set (ISS)? **A:** An Ideal Size Set (ISS) is the best combination of sizes for a product to maximize sales and minimize the risk of stockouts. The algorithm uses this to ensure that the allocated quantities for each SKU include the most effective size breakdown for a store.

Q: What does Rate of Sale (ROS) mean in this context? **A:** Rate of Sale (ROS) measures how quickly products are selling over a specific period (e.g., units per week). The Distribution Sales Preparation module computes this, and the algorithm uses ROS as a key indicator to inform allocation decisions, prioritizing faster-selling items or locations.

Q: What are Attribute Groups (AG) and how are they used? **A:** An Attribute Group (AG) is a set of product attributes (like color, size, or brand) that are grouped together for analysis. The algorithm can calculate revenue and make allocation decisions based on these Attribute Groups to ensure a balanced distribution across product characteristics.

Q: How does the system handle different types of product movements? **A:** The system manages initial inventory Allocation from the warehouse to stores, Inter-Store Transfers (IST) to move stock between stores within a region, and Inter-Warehouse Transfers (IWHT) to rebalance inventory across our larger warehouse network.

Configuration & Settings

Q: Can we set specific business rules for certain styles or stores? **A:** Yes, the Distribution Data Preparation module incorporates overriding business rules, such as style-specific depths, Just-In-Time (JIT) settings, or special pack sizes, to influence how the algorithm handles specific products or locations.

Q: How does the system account for warehouse inventory discrepancies? **A:** The Distribution Warehouse Inventory Preparation module adjusts on-hand warehouse stock by applying a defined warehouse inefficiency factor, which accounts for potential discrepancies and ensures a more realistic inventory count for allocation.

Q: Is there a way to reserve warehouse stock for specific purposes? **A:** Yes, the Distribution Warehouse Reservation module allows for defining reservation rules, such as reserving a certain percentage of a style's stock based on its channel. This ensures strategic inventory is held back for new styles or other specific needs.

Q: How are planogram capacities considered in the distribution? **A:** The Distribution Planogram Incrementer module analyzes historical sales to adjust and apply increment ratios to store planograms. This ensures that the physical capacity for products and options in stores evolves to match actual demand, optimizing visibility and depth during allocation.

Q: Can we influence the priority of allocation for certain items? **A:** Yes, the Distribution Ranking module ranks store-style combinations and SKU/category-size combinations based on calculated revenue per day and other metrics. This ranking directly influences the priority for allocation decisions, ensuring higher-priority items are considered first.

Data & Inputs

Q: What types of data does the distribution system require as input? **A:** The Distribution Data Preparation module requires a comprehensive set of master and transactional data, including store information, style and SKU details, current inventory levels, goods in transit (GIT), open orders, planogram configurations, size set quantities, product attributes, seasonality data, and any specific business rules.

Q: How does the system ensure the quality of sales data used for analysis? **A:** The Distribution Sales Preparation module is dedicated to cleaning sales data. It removes liquidation sales, identifies and excludes "broken" sales data (e.g., from periods with incomplete product presence), ensuring that only accurate and reliable sales data is used for Rate of Sale (ROS) computation and revenue calculations.

Q: What happens if there's missing or inconsistent input data? **A:** The Distribution Input Validation module performs comprehensive checks on all incoming data. It identifies and flags missing configurations, invalid formats, or logical inconsistencies (e.g., missing store definitions or planogram data) to prevent incorrect distribution calculations.

Q: How does the system verify that it has enough relevant data for a specific run date? **A:** The Distribution End Date Validation module specifically checks for the presence of all necessary inventory data (store stock, goods in transit, open orders, in-transit transfers) and warehouse mappings as of the specified endDate. It also confirms the existence of 'live' styles in season and essential store data.

Q: Does the system consider future inventory? **A:** Yes, the Distribution Data Preparation module accounts for goods in transit (GIT) and open orders, ensuring that future incoming stock is factored into the distribution calculations.

Process & Logic

Q: How does the system determine which stores or styles are prioritized for inventory? **A:** The Distribution Compute Revenue module calculates revenue per day for store-style combinations, and the Distribution Ranking module then uses this revenue, along with inventory and other metrics, to rank priorities. Higher-ranked items and locations receive preferential consideration in allocation.

Q: How does the system handle different types of allocation scenarios? **A:** The core Distribution Allocation module orchestrates various allocation iterations, such as initial replenishment, fresh allocation for new products, depth increments, and adjustments based on planograms or pullbacks. It applies complex business rules and constraints (OD, OW, ISS, ROS, AG) for each scenario.

Q: How are 'virtual warehouse stock' and inter-store transfers managed? **A:** The Distribution Compute Warehouse Inventory module helps create a regional warehouse map for Inter-Store Transfer (IST) scenarios. It calculates "virtual warehouse stock" from leftover store inventory within a region, making it available for outward transfers to other stores through the Distribution IST Runner.

Q: Can the system segment styles for different allocation strategies? **A:** Yes, the Distribution Segmentation module categorizes store-style combinations into segments like "TOP_SELLER," "BOTTOM_SELLER," or "NORMAL_SELLER" based on revenue, live days, and sales quantity. This allows for differentiated business rules, such as pulling back inventory from bottom sellers.

Q: What is the role of the Distribution Group Module? **A:** The Distribution Group Module acts as the orchestrator for the entire distribution pipeline. It defines and manages the sequence of execution for all individual sub-modules, ensuring that each step—from data preparation to output generation—is performed in the correct order and dependencies are met.

Outputs & Results

Q: What kind of reports or data can I get from the distribution process? **A:** The Distribution Output module generates various denormalized tables and reports. These include detailed allocation reports showing what quantity of each SKU was sent to which store, warehouse stock summaries, fill rate analyses, and comprehensive reports on inter-store and inter-warehouse movements.

Q: Can I see the results of Inter-Store Transfers (ISTs)? **A:** Yes, the Distribution Output module will include specific reports detailing the Inter-Store Transfer (IST) movements, showing which stores transferred stock to others and the quantities involved, as managed by the Distribution Inter-Store Transfer (IST) module.

Q: How can I track the overall health of inventory after distribution? **A:** The output includes reports that can be used to analyze overall store health rates, especially after transfers, helping to gauge the efficiency of the rebalancing efforts.

Q: Are the warehouse inventory adjustments reflected in the final output? **A:** Yes, the final outputs from the Distribution Output module will reflect the adjusted warehouse inventory after accounting for inefficiency factors and lead time exclusions, as processed by the Distribution Warehouse Inventory Preparation module.

Q: Can I see how planogram adjustments influenced the allocation? **A:** The final allocation outputs will reflect the impact of planogram increments determined by the Distribution Planogram Incrementer module. While direct reports on planogram increments might be separate, their effect will be evident in the allocated quantities.

Error Handling & Edge Cases

Q: What happens if sales data for a period is considered “broken”? **A:** The Distribution Sales Preparation module is designed to identify and remove “broken” sales data—such as sales from periods where a product was not consistently present or data is incomplete. This ensures that Rate of Sale (ROS) calculations are accurate and not skewed by unreliable information.

Q: How does the system handle styles that are marked as ‘read-only’ or out of season? **A:** The Distribution Compute Revenue module identifies styles that are ‘read-only’ based on season lists. These styles are explicitly prevented from being allocated, ensuring that inventory is not distributed for products that are no longer active or strategically managed outside the standard allocation process.

Q: What if there are issues with the endDate data for the distribution run? **A:** The Distribution End Date Validation module will flag errors if necessary inventory data (store stock, GIT, open orders, in-transit transfers) or warehouse mappings are missing for the specified endDate. This prevents the system from running with insufficient or irrelevant data.

Q: How does the system prevent overstocking in stores during allocation? **A:** The Distribution Iteration Runner component, used by the Distribution Allocation module, incorporates logic to manage flags and constraints, including those related to overstocking, to prevent stores from receiving more inventory than they can effectively manage or sell.

Q: What measures are in place to ensure that transfers are valid? **A:** For Inter-Store Transfers (IST), the Distribution IST Runner includes specific criteria checks, such as `checkIstOutwardingCriteria`, to validate outward movements and ensure that transfers align with the IST framework and are feasible.

Need More Help?

Please contact the Inventory & Distribution Support Team for further assistance.

distribution Documentation

- **Version:** 1.0
 - **Last Updated:** 2025-05-23
-

2. Executive Summary

This documentation outlines the distribution algorithm, a comprehensive system designed to optimize inventory allocation and transfers across a retail network. By leveraging sophisticated data preparation, sales analysis, ranking, and segmentation, the algorithm intelligently distributes products to maximize sales, minimize stockouts, and enhance overall operational efficiency. Its core outcome is to ensure the right product is in the right place at the right time, driving revenue growth and improving inventory turns for the business.

3. Architecture Overview

The distribution system processes inventory allocation and transfers through a phased pipeline. Data preparation and validation modules initialize the core data, followed by sales analysis, revenue computation, and ranking. Segmentation categorizes styles for differentiated treatment. The central allocation module, leveraging specialized iteration runners, determines optimal inventory distribution. Inter-store and inter-warehouse transfers rebalance stock across the network. Finally, all computed results are consolidated and outputted for reporting and downstream systems. The entire process is orchestrated by a group module.

```
graph TD
  subgraph Orchestration
    P(Distribution Group Module)
  end

  subgraph Validation
    N(Distribution Input Validation)
    O(Distribution End Date Validation)
  end

  subgraph Data_Preparation [Data Preparation]
    A(Distribution Data Preparation)
    B(Distribution Sales Preparation)
    C(Distribution Warehouse Inventory Preparation)
```

```
    D(Distribution Warehouse Reservation)
end
```

```
subgraph Core Logic
    E(Distribution Compute Revenue)
    F(Distribution Ranking)
    G(Distribution Segmentation)
    H(Distribution Allocation)
    L(Distribution Planogram Incrementer)
end
```

```
subgraph Transfer Modules
    K(Distribution Compute Warehouse Inventory)
    I(Distribution Inter-Store Transfer (IST))
    J(Distribution Inter-Warehouse Transfer (IWHT))
end
```

```
subgraph Output
    M(Distribution Output)
end
```

```
subgraph Internal Runners
    Q(Base Iteration Runner)
    R(Distribution Iteration Runner)
    S(Distribution IST Runner)
end
```

```
P -- orchestrates --> A
P -- orchestrates --> B
P -- orchestrates --> C
P -- orchestrates --> D
P -- orchestrates --> E
P -- orchestrates --> F
P -- orchestrates --> G
P -- orchestrates --> H
P -- orchestrates --> I
P -- orchestrates --> J
P -- orchestrates --> K
P -- orchestrates --> M
```

```
N -- validates --> A
O -- validates --> A
```

```
A -- data_flow --> B
A -- data_flow --> C
A -- data_flow --> D
```

```
B -- data_flow --> E
B -- data_flow --> F
B -- data_flow --> G
```

```
C -- data_flow --> E
C -- data_flow --> H
```



```

D -- business_logic --> H

E -- data_flow --> F
E -- data_flow --> G

F -- data_flow --> H

G -- data_flow --> H

H -- data_flow --> K
H -- data_flow --> J
H -- calls --> L

K -- data_flow --> I
K -- data_flow --> L

L -- data_flow --> H

H -- output_generation --> M
I -- output_generation --> M
J -- output_generation --> M

Q -- extends --> R
R -- extends --> S

H -- uses --> R
I -- uses --> S

```

Module Descriptions:

- **Distribution Data Preparation:** This module loads and prepares all master and transactional data for the distribution process, ensuring foundational data is correctly formatted.
- **Distribution Sales Preparation:** This module cleans sales data and computes the Rate of Sale (ROS) for SKUs across stores.
- **Distribution Warehouse Inventory Preparation:** This module adjusts on-hand warehouse stock by accounting for inefficiency and lead time, preparing it for allocation.
- **Distribution Warehouse Reservation:** This module initializes and manages reservation rules for warehouse inventory based on channel configurations.
- **Distribution Compute Revenue:** This module calculates revenue per day for store-style combinations and aggregated levels, identifying read-only styles.
- **Distribution Ranking:** This module ranks store-style combinations and SKU/category-size combinations based on revenue and other metrics to prioritize allocation.
- **Distribution Segmentation:** This module categorizes store-style combinations into segments like “TOP_SELLER” or “BOTTOM_SELLER” for differentiated rule application.
- **Distribution Allocation:** This is the core module that orchestrates various allocation iterations, applying business rules (OD, OW, ISS, ROS, AG) to determine optimal SKU quantities per store.
- **Distribution Inter-Store Transfer (IST):** This module manages inventory movements directly between stores within defined IST groups, improving network efficiency.

- **Distribution Inter-Warehouse Transfer (IWHT):** This module manages inventory transfers between warehouses to consolidate and fulfill demand across the network.
 - **Distribution Compute Warehouse Inventory:** This module computes and organizes warehouse inventory, particularly creating “virtual warehouse stock” from leftover store inventory for IST.
 - **Distribution Planogram Incrementer:** This module calculates and applies increment ratios to store planograms based on historical sales data to optimize capacity.
 - **Distribution Output:** This module generates and persists all final outputs from the distribution process, transforming in-memory decisions into reports.
 - **Distribution Input Validation:** This module performs comprehensive validation of all input data for integrity and consistency.
 - **Distribution End Date Validation:** This module specifically validates data availability as of the specified endDate, checking for inventory and live styles.
 - **Distribution Group Module:** This module acts as the orchestrator for the entire distribution pipeline, defining the sequence of execution for all sub-modules.
 - **Base Iteration Runner:** This is an abstract component defining fundamental logic for running allocation iterations, processing store-styles and applying common checks.
 - **Distribution Iteration Runner:** This component extends BaseIterationRunner to provide concrete implementations for various distribution allocation iterations, managing specific rules.
 - **Distribution IST Runner:** This component extends DistributionIterationRunner specifically for Inter-Store Transfer (IST) allocation iterations, overriding logic for IST-specific rules.
-

4. Prerequisites & Setup

This distribution pipeline is built on a Java platform.

- **Java Version:** JDK 11 or higher
- **Build Tool:** Maven 3.6.0 or higher
- **Dependencies:** All necessary dependencies are managed via pom.xml. Ensure Maven repositories are correctly configured (e.g., internal artifactories).

To run the Distribution Group Module orchestrator:

1. Clone the Repository:

```
git clone <repository-url>/distribution.git
cd distribution
```

2. Build the Project:

```
mvn clean install
```

This command compiles the code, runs tests, and packages the application into an executable JAR.

3. **Configure args.properties:** Ensure the args.properties file in the root directory is configured with the correct input paths, output paths, and operational parameters for your environment. Refer to the “Global Configuration” section for details.

4. **Execute the Orchestrator:**

```
java -jar target/distribution-<version>.jar --properties-file  
args.properties
```

Replace <version> with the actual version number from the pom.xml (e.g., 1.0).

5. Folder & File Structure

```
distribution/  
├─ module-group/           # Contains the orchestrator logic  
(Distribution Group Module)  
├─ submodules/             # Contains the implementation of  
individual distribution sub-modules  
├─ args.properties         # Global configuration parameters for  
the entire distribution run  
├─ common-data.json        # Schema definition for common data  
structures exchanged between modules  
├─ outputs/                # Directory where all final  
distribution reports and output files are generated
```

6. Global Configuration

This section details the global configuration parameters controlled by the args.properties file.

Setting Name	Default Value	Description	Valid Range/When to Change
(No parameters provided in input)		No global configuration parameters were provided in the input data. Please consult the actual args.properties file for available settings.	Please consult the actual args.properties file for valid ranges and usage guidelines.

7. Shared Data Model

This section describes the common data structures and their schemas, typically defined in `common-data.json`, that are shared across different modules in the distribution pipeline.

Field	Type	Description
<i>(No schema provided in input)</i>		<i>No shared data model schema was provided in the input data. Please consult the actual <code>common-data.json</code> file for the schema definition.</i>

8. Submodule Chapters

8.1. Distribution Data Preparation

1. Module Name & Business Objective

Module Name: Distribution Data Preparation **Business Objective:** This module is critical for ensuring that all foundational data—including store, style, SKU, inventory, and business rule information—is accurately loaded, validated, and formatted for the entire distribution process. Its objective is to provide a clean and consistent data baseline, preventing errors in subsequent allocation and transfer calculations.

2. Required Information

- **Sales Data:** N/A (this module prepares master data, not sales data directly)
- **Product Data:**
 - SKU definitions, product hierarchy (e.g., AGs)
 - Style-specific depths (OD) or widths (OW)
 - Pack sizes
 - Seasonality overrides
- **Operational Data:**
 - Store master data
 - Current inventory levels (store stock, warehouse stock)
 - Goods in Transit (GIT)
 - Open Orders
 - Planogram configurations
 - Size set quantities (ISS)
 - Just-In-Time (JIT) settings

3. Business Settings

This module relies heavily on the correctness of input data files and configurations. There are no specific “business settings” within this module itself, but it processes settings from `args.properties` such as input file paths and flags for certain data overrides. Refer to Global Configuration for overarching settings.

4. Process Flow

graph LR

A[Raw Input Data Feeds] --> B(Load & Parse Data)

B --> C(Validate & Clean Data)

C --> D(Populate Core Data Structures)

D --> E[Prepared Distribution Data]

5. Step-by-Step Actions

1. **Load** *raw master and transactional data* from specified input sources.
2. **Parse** *various data formats* into an internal representation.
3. **Validate** *data integrity and consistency, identifying missing or malformed records.*
4. **Cleanse** *data by applying business rules, such as resolving duplicates or standardizing formats.*
5. **Populate** *core data structures (e.g., store maps, SKU lists, inventory tables), making them available for other modules.*
6. **Apply** *overriding business rules (e.g., style-specific depths, JIT settings) as configured.*

6. Sample Scenario

A new footwear collection arrives. This module *loads* the new styles, their associated SKUs, default ODs (Optimum Depth), ISS (Ideal Size Set) for each SKU, and initial warehouse inventory. It also *integrates* any special seasonal rules for the new collection. *Key Takeaway: Accurate foundational data is paramount for effective distribution; errors here propagate throughout the pipeline.*

7. Results

- A comprehensive, in-memory `DistributionData` object populated with validated and prepared master and transactional data.
- Error logs detailing any data loading or parsing issues.

8. Unusual Cases

- **Missing or delayed data feed:** *Can lead to an incomplete or outdated view of inventory and product data, causing incorrect allocations.*
- **Incorrect data format:** *May cause parsing failures, leading to partial data loads or module failures.*
- **Inconsistent data across feeds:** *Discrepancies between inventory and planogram data might result in allocation decisions that cannot be executed.*

9. Next Steps

The prepared data is immediately consumed by modules like Distribution Sales Preparation, Distribution Warehouse Inventory Preparation, and Distribution Warehouse Reservation, which build upon this foundation.

10. Common Questions

1. How can I verify if all necessary data was loaded correctly?
2. What happens if a critical data feed is missing for a run?
3. How are overrides (e.g., style-specific depths) incorporated?
4. Can I see the raw data before it's processed by this module?
5. What are the most common data preparation errors we encounter?

11. Quick Reference

- **Purpose:** Foundation data loading & preparation.
 - **Key Inputs:** Store, SKU, Inventory, Planogram, Business Rules.
 - **Output:** Clean, structured `DistributionData`.
 - **Impact:** Ensures data quality for entire pipeline.
-

8.2. Distribution Sales Preparation

1. Module Name & Business Objective

Module Name: Distribution Sales Preparation **Business Objective:** This module's primary objective is to transform raw sales transactions into actionable insights by cleaning the data and accurately calculating the Rate of Sale (ROS) for each SKU. This cleaned and reliable ROS is fundamental for subsequent modules that require sales performance metrics for revenue calculation, ranking, and allocation decisions.

2. Required Information

- **Sales Data:**
 - Historical sales transactions (unit sales, revenue, dates)
 - Liquidation flags associated with sales periods or products
- **Product Data:**
 - SKU hierarchy and definitions (to link sales to specific items)
- **Operational Data:**
 - Store operational dates (to identify valid sales periods)

3. Business Settings

This module may be configured with parameters for defining what constitutes “broken” sales data (e.g., minimum live days for a SKU in a store) and thresholds for identifying liquidation sales. These parameters are typically found in the `args.properties` file.

4. Process Flow

graph LR

```
A[Raw Sales Data] --> B[Remove Liquidation Sales]
B --> C[Identify & Remove Broken Sales]
C --> D[Compute Rate of Sale (ROS)]
D --> E[Cleaned Sales & ROS Data]
```

5. Step-by-Step Actions

1. **Filter** out sales identified as liquidation events, as these do not represent typical demand.
2. **Analyze** sales data for “broken” periods (e.g., insufficient stock, short product presence), removing unreliable sales figures.
3. **Group** sales by SKU and store, aggregating daily or weekly sales counts.
4. **Calculate** the Rate of Sale (ROS) for each SKU-store combination, often expressed as units per week or day.
5. **Output** cleaned sales data and computed ROS, making it ready for further analysis.

6. Sample Scenario

A specific t-shirt (SKU 12345) sold 100 units in Store A over the past 4 weeks, but 20 units were during a known clearance event. This module would *remove* the 20 units, compute ROS based on 80 units over 4 weeks, resulting in an ROS of 20 units/week. If Store B only had the t-shirt for 2 days due to a stockout, those sales might be marked as broken. *Key Takeaway: Accurate ROS is crucial for demand forecasting and optimal inventory allocation.*

7. Results

- Cleaned sales data (cleanedSales, prioritySales) excluding liquidation and broken sales.
- Computed Rate of Sale (ROS) for relevant SKU-store combinations.

8. Unusual Cases

- **Missing or delayed data feed:** *Leads to outdated or incomplete sales data, impacting ROS accuracy.*
- **Incomplete product presence records:** *Difficult to accurately identify “broken” sales if the system doesn’t know when a product was actually available.*
- **Sales spikes due to promotions:** *May be misidentified as normal sales if not explicitly flagged, distorting ROS.*

9. Next Steps

The cleaned sales data and computed ROS are fed into the Distribution Compute Revenue, Distribution Ranking, and Distribution Segmentation modules.

10. Common Questions

1. How does the system define and remove “broken” sales?
2. What is the look-back window for ROS calculation?
3. Can we see the raw sales data before cleaning?
4. How do promotional sales impact ROS if not filtered out?
5. What’s the difference between `cleanedSales` and `prioritySales`?

11. Quick Reference

- **Purpose:** Clean sales data, calculate ROS.
 - **Key Inputs:** Raw sales history.
 - **Output:** Cleaned sales data, SKU ROS.
 - **Impact:** Provides reliable demand signals.
-

8.3. Distribution Warehouse Inventory Preparation

1. Module Name & Business Objective

Module Name: Distribution Warehouse Inventory Preparation **Business Objective:** This module’s objective is to accurately reflect the available inventory in warehouses for allocation purposes. It adjusts raw on-hand stock by accounting for operational inefficiencies and lead times, ensuring that subsequent allocation and transfer calculations use a realistic and actionable quantity of warehouse inventory.

2. Required Information

- **Operational Data:**
 - Raw on-hand warehouse inventory by SKU.
 - Defined warehouse inefficiency factors (e.g., percentage, fixed quantity per SKU).
 - Lead time definitions or stock quantities to exclude for lead time.
 - Goods In Transit (GIT) status (if applicable to warehouse stock).

3. Business Settings

Configuration parameters in `args.properties` would specify the warehouse inefficiency factor (e.g., `warehouse.inefficiency.factor = 0.95` or `warehouse.buffer.quantity = 5`) and the quantity or duration for lead time stock exclusion.

4. Process Flow

graph LR

A[Raw Warehouse Inventory (SKU)] --> B(Adjust for Inefficiency)

B --> C(Exclude Lead Time Stock)

C --> D[Adjusted Warehouse Inventory (SKU)]

5. Step-by-Step Actions

1. **Retrieve** raw warehouse on-hand inventory for each SKU.
2. **Apply** warehouse inefficiency factor (e.g., multiplying by 0.95 or subtracting a fixed quantity) to the on-hand stock, *accounting for discrepancies or damaged goods*.
3. **Exclude** a specified quantity of stock from the adjusted inventory, *reserving it to cover lead time for future orders*.
4. **Update** the skuWhStock in the distribution data, *making the adjusted inventory available for subsequent modules*.

6. Sample Scenario

Warehouse A has 100 units of SKU 'X'. The system is configured with a 5% inefficiency factor and reserves 2 units for lead time. This module would first *adjust* 100 units to 95 units ($100 * 0.95$). Then, it would *exclude* 2 units, making the final available warehouse stock for SKU 'X' 93 units. *Key Takeaway: Realistic warehouse stock levels prevent over-allocation and ensure fulfillment capacity.*

7. Results

- Adjusted skuWhStock in the DistributionData object, reflecting available warehouse inventory net of inefficiency and lead time buffers.

8. Unusual Cases

- **Missing or delayed data feed:** *Leads to outdated or incorrect warehouse inventory figures, causing misallocations.*
- **Incorrect inefficiency factor:** *Can lead to under- or over-estimation of available stock.*
- **Negative inventory:** *Indicates a data error in the raw feed, requires immediate attention.*

9. Next Steps

The prepared warehouse inventory data is consumed by Distribution Compute Revenue (for initial store-style inventory) and, more directly, by Distribution Allocation, which uses this stock for fulfillment.

10. Common Questions

1. How is the warehouse inefficiency factor determined?
2. Why do we exclude stock for lead time, isn't that covered by other modules?
3. What happens if the actual warehouse stock is significantly different from the system's adjusted stock?
4. Can we apply different inefficiency factors to different warehouses?
5. How frequently is the raw warehouse inventory data updated?

11. Quick Reference

- **Purpose:** Adjust warehouse stock for realism.
 - **Key Inputs:** Raw warehouse stock, inefficiency factors, lead time.
 - **Output:** Actionable skuWhStock.
 - **Impact:** Prevents over-allocation, ensures fulfillment.
-

8.4. Distribution Warehouse Reservation

1. Module Name & Business Objective

Module Name: Distribution Warehouse Reservation **Business Objective:** This module's objective is to implement strategic inventory holding by reserving a specific percentage of warehouse stock for certain styles or channels. This ensures that critical or newly introduced styles are not entirely allocated away, protecting future allocation needs and supporting specific business strategies (e.g., launching new collections, replenishing specific channels).

2. Required Information

- **Product Data:**
 - Style definitions and their associated channels.
- **Operational Data:**
 - StyleReservePercRow configurations specifying reservation percentages per channel.
 - Raw warehouse inventory (implicitly, as it determines what's available to reserve).

3. Business Settings

Configuration data such as StyleReservePercRow would define the channels (e.g., 'E-commerce', 'Wholesale', 'New Stores') and the percentage of warehouse stock to reserve for styles associated with those channels.

4. Process Flow

```
graph LR
    A[Style Reserve Configurations (e.g., StyleReservePercRow)] --> B[Process Reservation Rules]
    B --> C[Initialize Channel-Specific Reserves]
    C --> D[Warehouse Reservation Logic]
```

5. Step-by-Step Actions

1. **Load** input configurations related to style and channel reservations (StyleReservePercRow).
2. **Parse** these configurations to identify which styles or channels require a reserved percentage of inventory.

3. **Initialize** *internal data structures* that map channels to their specified reservation percentages (`channelReservePerc`).
4. **Integrate** *this reservation logic into the overall distribution data model, informing subsequent allocation decisions to hold back inventory.*

6. Sample Scenario

A new style, “Winter Jacket 2025,” is designated for 20% reservation for the “E-commerce” channel to ensure online launch stock. This module would *process* this rule, ensuring that when Distribution Allocation runs, 20% of the available warehouse stock for “Winter Jacket 2025” is *marked as reserved* and not available for general store replenishment. *Key Takeaway: Strategic reservations protect inventory for high-priority initiatives or channels.*

7. Results

- Initialized `channelReservePerc` or similar reservation data structures within `DistributionData`.
- The overall distribution process is informed about specific inventory quantities to hold back.

8. Unusual Cases

- **Missing or delayed data feed:** *If `StyleReservePercRow` is missing, no reservations will be applied, potentially leading to critical stock being fully allocated.*
- **Conflicting reservation rules:** *Ambiguous or overlapping rules for the same style/channel could lead to incorrect or unintended reservation percentages.*
- **Insufficient initial stock:** *If warehouse stock is too low, the reservation might exceed available stock, leading to allocation issues.*

9. Next Steps

The reservation rules established by this module are then applied by the Distribution Allocation module, which will factor in these reserved quantities when determining how much stock to allocate.

10. Common Questions

1. How do we define which styles or channels require reservations?
2. Can reservation percentages be dynamically adjusted?
3. What is the business impact if no reservations are applied?
4. How does this module interact with total available warehouse stock?
5. Can I see a report of currently reserved inventory?

11. Quick Reference

- **Purpose:** Reserve warehouse inventory by channel/style.
- **Key Inputs:** Style reservation rules.
- **Output:** Reservation logic for `DistributionData`.

- **Impact:** Protects strategic inventory.
-

8.5. Distribution Compute Revenue

1. Module Name & Business Objective

Module Name: Distribution Compute Revenue **Business Objective:** This module's objective is to accurately calculate the daily revenue for all relevant store-style combinations, as well as aggregated levels (e.g., by Attribute Group or nationally). This revenue data is foundational for ranking and prioritizing styles for allocation, ensuring that higher-performing items and combinations receive appropriate attention and inventory. It also identifies "read-only" styles that should not be allocated.

2. Required Information

- **Sales Data:**
 - Cleaned sales data (from Distribution Sales Preparation)
 - Rate of Sale (ROS) data (from Distribution Sales Preparation)
- **Product Data:**
 - SKU and style definitions, including seasonal lists.
 - Attribute Group (AG) mappings.
- **Operational Data:**
 - Store-level data (e.g., store characteristics relevant for revenue calculation).
 - Initial size depths for stores and styles.

3. Business Settings

This module might use configuration parameters to define which sales periods to consider for revenue calculation, thresholds for identifying "read-only" styles (e.g., based on end-of-season flags), and rules for determining initial size depths.

4. Process Flow

graph LR

A[Cleaned Sales & ROS] --> B(Calculate Daily Revenue per Store-Style)

B --> C(Aggregate Revenue by AG & National)

C --> D(Identify Read-Only Styles)

D --> E(Determine Initial Size Depth)

E --> F[Store-Style Revenue & Depth Data]

5. Step-by-Step Actions

1. **Retrieve** *cleaned sales data and ROS* for all SKUs and stores.
2. **Calculate** *daily revenue* for each store-style combination, *often using ROS and average selling price (ASP)*.

3. **Aggregate** revenue figures to higher levels, such as Attribute Group (AG) and national totals, *for macro-level analysis*.
4. **Identify** styles that are “read-only” (e.g., based on season lists or specific flags), *excluding them from active allocation consideration*.
5. **Determine** the initial size depth (ISS) for each store-style combination, *providing a starting point for optimal inventory levels*.

6. Sample Scenario

Store A has high ROS for “Summer Dress” (Style X) and “Jeans” (Style Y). This module *calculates* that Style X generates \$500/day in Store A, and Style Y generates \$300/day. It then *identifies* “Winter Coat” (Style Z) as ‘read-only’ because its season has ended. For Style X, it *sets* an initial ISS of 12 units (e.g., 2S, 4M, 4L, 2XL). *Key Takeaway: Precise revenue calculation underpins effective prioritization in the allocation process.*

7. Results

- Calculated daily revenue for store-style combinations and aggregated levels.
- Identification of “read-only” styles.
- Determined initial size depths for store-styles.

8. Unusual Cases

- **Missing or delayed data feed:** *If cleaned sales data is incomplete, revenue calculations will be inaccurate, leading to poor ranking.*
- **Incorrect seasonality data:** *Styles might be incorrectly flagged as ‘read-only’ or active, impacting allocation decisions.*
- **Zero ROS for active styles:** *If a style has no sales, its revenue will be zero, causing it to be deprioritized even if it’s a new or important item (requires business rules for new items).*

9. Next Steps

The computed revenue, read-only flags, and initial size depths are passed to Distribution Ranking (for prioritization) and Distribution Segmentation (for categorization).

10. Common Questions

1. How is “revenue per day” specifically calculated?
2. What defines a “read-only” style, and how is it managed?
3. How are new styles (with no historical sales) assigned an initial size depth?
4. Can we account for promotions in the revenue calculation?
5. What’s the difference between style-level and AG-level revenue?

11. Quick Reference

- **Purpose:** Calculate revenue, determine initial depth, identify read-only styles.
- **Key Inputs:** Cleaned sales, ROS, style definitions.

- **Output:** Revenue data, initial ISS, read-only flags.
 - **Impact:** Fuels ranking and prioritization.
-

8.6. Distribution Ranking

1. Module Name & Business Objective

Module Name: Distribution Ranking **Business Objective:** This module's core objective is to create a prioritized order for store-style combinations and SKU/category-size combinations based on their performance metrics (primarily revenue per day and Product Sales Availability, PSA). The ranking ensures that the most impactful items are considered first during the allocation process, maximizing sales potential and efficiently utilizing available inventory.

2. Required Information

- **Sales Data:**
 - Computed revenue per day for store-styles (from Distribution Compute Revenue).
 - Cleaned sales data (from Distribution Sales Preparation).
- **Product Data:**
 - SKU and style definitions, Attribute Group (AG) mappings.
- **Operational Data:**
 - Inventory levels (for PSA calculation).

3. Business Settings

Ranking criteria and weights might be configurable in `args.properties`. For example, a business could prioritize styles with high revenue, or new styles (even with low initial sales) by applying a boost factor.

4. Process Flow

graph LR

```
A[Store-Style Revenue & Metrics] --> B(Compute Ranking Scores)
B --> C(Sort & Prioritize Combinations)
C --> D(Denormalize for Reporting)
D --> E[Ranked Store-Styles & SKUs]
```

5. Step-by-Step Actions

1. **Receive** *calculated revenue per day and other performance metrics* (e.g., PSA) for store-style combinations.
2. **Compute** *a composite ranking score* for each combination, *potentially applying weighting to different metrics*.
3. **Sort** *store-style and SKU/category-size combinations* based on their computed scores, *establishing a priority order*.

4. **Generate** *denormalized rows* for reporting the ranking results, *providing transparency into prioritization logic*.
5. **Output** *the ranked list, which will guide the allocation module*.

6. Sample Scenario

Store A has high revenue for “Sneakers” (Style X) and moderate revenue for “Sandals” (Style Y). This module would *assign* a higher rank to Store A-Style X. For non-pivotal sizes of Style X (e.g., size 5 or 12), it would *rank* them based on their specific contribution to overall style performance. *Key Takeaway: Effective ranking ensures that allocation efforts are focused on the most profitable opportunities.*

7. Results

- A prioritized list of store-style combinations.
- Ranked SKU/category-size combinations (especially for non-pivotal allocations).
- Denormalized data for ranking reports.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate revenue or PSA data will lead to flawed rankings, misdirecting allocation efforts.*
- **New styles with no history:** *May be ranked low despite potential, requiring specific business rules to boost their priority.*
- **Outliers in sales data:** *Unusual spikes could artificially inflate a style’s rank, leading to over-allocation.*

9. Next Steps

The ranked data is a primary input for the Distribution Allocation module, guiding which store-style and SKU combinations are addressed first.

10. Common Questions

1. What are the key metrics used to determine rank?
2. How do we handle new styles or styles with limited sales history in the ranking?
3. Can we adjust the weighting of different metrics in the ranking algorithm?
4. How does the ranking algorithm ensure fair distribution across different store tiers?
5. Where can I find the detailed ranking report for a specific run?

11. Quick Reference

- **Purpose:** Prioritize store-styles and SKUs.
 - **Key Inputs:** Revenue, PSA, cleaned sales.
 - **Output:** Ranked lists for allocation.
 - **Impact:** Optimizes allocation efficiency.
-

8.7. Distribution Segmentation

1. Module Name & Business Objective

Module Name: Distribution Segmentation **Business Objective:** This module's objective is to categorize store-style combinations into meaningful segments (e.g., "TOP_SELLER", "BOTTOM_SELLER", "NORMAL_SELLER"). This segmentation allows the allocation process to apply differentiated business rules, such as prioritizing replenishment for top performers or initiating pullbacks from underperforming items, thereby optimizing inventory management and sales.

2. Required Information

- **Sales Data:**
 - Computed revenue per day (from Distribution Compute Revenue).
 - Live days for each store-style combination.
 - Sales quantity benchmarks.
- **Product Data:**
 - Style definitions.

3. Business Settings

Thresholds for segmentation (e.g., "TOP_SELLER" = top X% revenue, "BOTTOM_SELLER" = bottom Y% revenue or sales below Z units) would be configurable in `args.properties`.

4. Process Flow

graph LR

A[Store-Style Revenue & Live Days] --> B(Apply Segmentation Rules)

B --> C(Categorize Combinations)

C --> D[Segmented Store-Style Data]

5. Step-by-Step Actions

1. **Receive** *store-style combinations along with their calculated revenue per day and live days.*
2. **Apply** *predefined segmentation rules* (e.g., based on revenue thresholds, sales quantities, or live days) to each combination.
3. **Assign** *a segment label* (e.g., "TOP_SELLER", "BOTTOM_SELLER", "NORMAL_SELLER") to each store-style.
4. **Integrate** *these segmentations into the distribution data model, enabling differentiated treatment in subsequent allocation steps.*

6. Sample Scenario

Store B's performance with "Winter Boots" (Style Z) has been very low, generating only \$50/day over 8 weeks, putting it in the bottom 10% of revenue for that style. This module would *classify* Store B-Style Z as a "BOTTOM_SELLER".

Conversely, Store C-Style Z, generating \$1000/day, would be *classified* as a “TOP_SELLER”. *Key Takeaway: Segmentation enables tailored allocation strategies that optimize performance across varied store-style scenarios.*

7. Results

- Store-style combinations categorized into defined segments (e.g., “TOP_SELLER”, “BOTTOM_SELLER”).

8. Unusual Cases

- **Missing or delayed data feed:** *If revenue or live days data is inaccurate, styles might be miscategorized, leading to suboptimal allocation decisions.*
- **Ambiguous segmentation thresholds:** *Poorly defined thresholds can result in too many items in one segment or an unhelpful distribution across segments.*
- **New styles with no sales:** *May automatically fall into “BOTTOM_SELLER” without specific rules, requiring manual overrides or initial boost logic.*

9. Next Steps

The segmentation information is passed to the Distribution Allocation module, which uses these segments to apply specific rules, particularly for pullbacks or targeted replenishment.

10. Common Questions

1. What are the criteria for defining “TOP_SELLER” vs. “BOTTOM_SELLER”?
2. Can segmentation criteria be adjusted for different product categories?
3. How does segmentation influence allocation decisions, especially for pullbacks?
4. Where can I view the segmentation results for a given run?
5. What happens if a store-style combination doesn’t fit any defined segment?

11. Quick Reference

- **Purpose:** Categorize store-styles (TOP/BOTTOM/NORMAL).
 - **Key Inputs:** Revenue, live days, sales benchmarks.
 - **Output:** Segment labels for store-styles.
 - **Impact:** Enables differentiated allocation rules.
-

8.8. Distribution Allocation

1. Module Name & Business Objective

Module Name: Distribution Allocation **Business Objective:** This is the central brain of the distribution pipeline. Its objective is to determine the optimal quantity of each SKU to be allocated to each store. It achieves this by orchestrating various allocation iterations (e.g., replenishment, fresh allocation, depth increment,

planogram adjustments, pullbacks) and applying complex business rules and constraints (OD, OW, ISS, ROS, AG) to maximize sales, minimize stockouts, and optimize overall inventory health.

2. Required Information

- **Sales Data:**
 - Rate of Sale (ROS) (from Distribution Sales Preparation).
 - Cleaned sales data.
- **Product Data:**
 - Optimum Depth (OD), Optimum Width (OW) configurations.
 - Ideal Size Set (ISS) definitions.
 - Attribute Group (AG) mappings.
 - SKU and style definitions.
- **Operational Data:**
 - Current store stock levels.
 - Goods in Transit (GIT).
 - Open orders.
 - Adjusted warehouse inventory (from Distribution Warehouse Inventory Preparation).
 - Warehouse reservation rules (from Distribution Warehouse Reservation).
 - Ranked store-style/SKU combinations (from Distribution Ranking).
 - Segmented store-style combinations (from Distribution Segmentation).
 - Planogram increment ratios (from Distribution Planogram Incrementer).

3. Business Settings

This module has numerous configurable parameters in `args.properties` to control allocation behavior, including: * `runOption`: Defines which allocation iterations to execute (e.g., `REPLENISHMENT`, `FRESH_ALLOCATION`, `DEPTH_INCREMENT`). * Minimum fill rates, overstocking thresholds. * Flags for enabling/disabling specific allocation logic (e.g., color-based allocations, JIT adjustments). * Rules for pullbacks from “`BOTTOM_SELLER`” stores.

4. Process Flow

```
graph TD
    A[All Prepared Data, Rankings, Segments, Inventory] --> B[Initialize Allocation Run]
    B --> C[Run Iteration: Replenishment]
    C --> D[Run Iteration: Fresh Allocation]
    D --> E[Run Iteration: Depth Increment]
    E --> F[Run Iteration: Planogram Adjustments]
    F --> G[Run Iteration: Pullbacks]
    G --> H[Determine Final Allocations]
    H --> I[Updated Store Inventory & Allocation Decisions]
```

5. Step-by-Step Actions

1. **Initialize** *allocation data structures* with current inventory, demand, and business rules.

2. **Iterate** through predefined allocation stages (e.g., replenishment, fresh, depth increment, pullbacks) based on runOption.
3. For each iteration, **Identify** target store-style combinations using rankings and segments.
4. **Calculate** suggested allocation quantities per SKU based on OD, OW, ISS, ROS, and planogram data.
5. **Apply** business constraints (e.g., minimums, maximums, pack sizes, JIT rules) to refine allocations.
6. **Adjust** inventory levels (store stock, warehouse stock) as allocations are made, ensuring accurate remaining stock.
7. **Manage** pullbacks from underperforming (“BOTTOM_SELLER”) stores, reallocating inventory to higher-priority locations.
8. **Finalize** all allocation decisions, preparing them for output.

6. Sample Scenario

Store X is a “TOP_SELLER” for “Sneakers” (Style A), but has low stock. The module, in its replenishment iteration, uses the high ROS and its OD (Optimum Depth) to *allocate* 10 units of Style A, including an ISS (Ideal Size Set). Concurrently, Store Y, a “BOTTOM_SELLER” for “Boots” (Style B), is *flagged for a pullback*, with 5 units *being requested back* to the warehouse for reallocation. *Key Takeaway: This module orchestrates complex rules to optimally balance inventory across the network, driving both sales and efficiency.*

7. Results

- Detailed SKU-level allocation recommendations per store.
- Updated store and warehouse inventory states reflecting transfers.
- Fill rates and health rates for stores and styles.

8. Unusual Cases

- **Missing or delayed data feed:** *Critical input data (e.g., inventory, sales, rankings) can lead to illogical or no allocations.*
- **Warehouse stock depletion:** *If warehouse inventory runs out early in the allocation process, subsequent allocations cannot be fulfilled.*
- **Conflicting business rules:** *Overlapping or contradictory OD/OW/ISS rules can lead to inefficient allocations or errors.*
- **Unexpected store closures:** *Requires manual intervention or specific flags to prevent allocation to closed locations.*
- **Excessive overstocking:** *Despite rules, continuous overstocking in some stores may indicate an issue with OD or ROS settings.*

9. Next Steps

The detailed allocation decisions and updated inventory states are passed to the Distribution Output module for reporting. They also influence Distribution Compute Warehouse Inventory for IST.

10. Common Questions

1. How does the system decide between replenishment and fresh allocation?
2. What is the hierarchy of business rules (OD, OW, ISS, ROS)?
3. How are minimum allocation quantities or pack sizes handled?
4. Can I configure which allocation iterations run for a specific run?
5. What mechanisms are in place to prevent overstocking?

11. Quick Reference

- **Purpose:** Core inventory allocation to stores.
 - **Key Inputs:** All prepared data, rankings, segments, inventory, rules (OD, OW, ISS, ROS, AG).
 - **Output:** SKU-level allocation plans per store.
 - **Impact:** Maximizes sales, minimizes stockouts.
-

8.9. Distribution Inter-Store Transfer (IST)

1. Module Name & Business Objective

Module Name: Distribution Inter-Store Transfer (IST) **Business Objective:** This module extends the core allocation logic to manage inventory movements directly between stores within defined IST groups. Its objective is to rebalance inventory across the store network by moving excess stock from overstocked stores to understocked, high-demand stores, thereby improving overall network efficiency, maximizing sales velocity, and reducing the need for returns to the warehouse.

2. Required Information

- **Product Data:**
 - SKU and style definitions.
- **Operational Data:**
 - Current store stock levels.
 - Adjusted virtual warehouse stock (from Distribution Compute Warehouse Inventory for IST).
 - IST group definitions (which stores can transfer to/from each other).
 - Store health rates (post-IST).

3. Business Settings

Configuration parameters would define IST groups, minimum quantities for transfers, thresholds for a store to be considered “outwarding” (giving stock) or “inwarding” (receiving stock), and potential transfer costs.

4. Process Flow

graph TD

A[IST-Specific Inventory Data] --> B(Intra-Store Allocation

(Netting))

B --> C(Inter-Store Allocation (Transfers))
C --> D(Post-IST Brokenness Correction)
D --> E[Inter-Store Transfer Decisions]

5. Step-by-Step Actions

1. **Prepare** *IST-specific inventory data*, including “virtual warehouse stock” derived from aggregated store excess.
2. **Perform** *intra-store allocation (netting out internal movements)*, ensuring *internal stock needs are met before external transfers*.
3. **Execute** *inter-store allocation*, identifying donor stores with excess stock and recipient stores with demand, *facilitating transfers between them*.
4. **Apply** *IST-specific rules*, such as `getIstWhStock` for virtual warehouse stock and `checkIstOutwardingCriteria` for donor stores.
5. **Conduct** *a post-IST brokenness correction phase*, optimizing store health rates *after transfers have occurred*.
6. **Finalize** *all inter-store transfer recommendations*, making them ready for *execution and reporting*.

6. Sample Scenario

Store A has 10 units of a dress (SKU 123) with slow ROS, while Store B (in the same IST group) has high ROS for the same dress but is out of stock. This module would *identify* Store A as a potential donor and Store B as a recipient. It then *recommends* transferring 3 units from Store A to Store B, *improving overall sales potential across the group* without involving the main warehouse. *Key Takeaway: IST efficiently rebalances inventory across stores, reducing markdowns and improving stock availability.*

7. Results

- Recommended inter-store transfer movements (SKU, source store, destination store, quantity).
- Updated store stock levels post-transfer.
- Improved store health rates and fill rates for involved stores.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate store stock or IST group definitions can lead to invalid or suboptimal transfers.*
- **IST group misconfiguration:** *Incorrectly defined IST groups can prevent logical transfers between relevant stores.*
- **Excessive transfers leading to ‘ping-pong’:** *Poorly tuned thresholds might cause items to be constantly moved between stores, increasing operational costs.*
- **Insufficient stock for transfer:** *If a donor store’s stock is lower than assumed, the transfer cannot be fulfilled.*

9. Next Steps

The IST movements are sent to the Distribution Output module for reporting and potential execution by logistics systems.

10. Common Questions

1. How are IST groups defined, and can they overlap?
2. What criteria determine if a store can “outward” (send) or “inward” (receive) stock?
3. How does this module handle cases where transfer quantity is less than ideal?
4. What is the post-IST brokenness correction?
5. Can I prioritize transfers for specific styles or product categories?

11. Quick Reference

- **Purpose:** Rebalance inventory directly between stores.
 - **Key Inputs:** Store stock, IST group rules, virtual warehouse stock.
 - **Output:** Inter-store transfer recommendations.
 - **Impact:** Reduces markdowns, improves stock availability at demand points.
-

8.10. Distribution Inter-Warehouse Transfer (IWHT)

1. Module Name & Business Objective

Module Name: Distribution Inter-Warehouse Transfer (IWHT) **Business**

Objective: This module manages inventory transfers between multiple warehouses. Its objective is to consolidate and rebalance inventory across the entire warehouse network by identifying net demands and available stock within warehouse groups. This ensures that excess stock in one warehouse can fulfill demand in another, optimizing overall inventory levels and reducing the need for external procurement or excess holding costs.

2. Required Information

- **Product Data:**
 - SKU definitions.
- **Operational Data:**
 - Current warehouse inventory by SKU.
 - Demand signals from stores aggregated by warehouse group.
 - Warehouse group definitions.

3. Business Settings

Configuration parameters would define warehouse groups (e.g., regional hubs), thresholds for initiating transfers (e.g., minimum demand, minimum available stock), and rules for various types of pullbacks (e.g., demanding pullbacks).

4. Process Flow

graph LR

A[Warehouse Inventory & Aggregated Demand] --> B(Consolidate Inventory & Demand by Group)

B --> C(Identify Transfer Opportunities)

C --> D(Execute Transfers & Pullbacks)

D --> E[Inter-Warehouse Transfer Decisions]

5. Step-by-Step Actions

1. **Consolidate** *inventory data and demand signals* (from stores and other sources) within defined warehouse groups.
2. **Roll-up** *demands* to identify net requirements per warehouse.
3. **Identify** *source warehouses with available excess stock and destination warehouses with unmet demand*.
4. **Orchestrate** *transfers from source to destination warehouses, optimizing movements to fulfill demand*.
5. **Perform** *various types of pullbacks* (e.g., demanding pullbacks, other pullbacks) *to rebalance inventory proactively*.
6. **Update** *warehouse stock levels based on computed transfers*.
7. **Finalize** *all inter-warehouse transfer recommendations*.

6. Sample Scenario

Warehouse East has 500 units of Product Z (SKU 456), but only needs 300, while Warehouse West is understocked by 150 units for the same product. This module would *identify* the excess in East and the deficit in West. It would then *recommend* transferring 150 units from Warehouse East to Warehouse West, *rebalancing stock and ensuring regional demand can be met efficiently*. *Key Takeaway: IWHT optimizes inventory across the entire warehouse network, reducing storage costs and improving product availability nationwide.*

7. Results

- Recommended inter-warehouse transfer movements (SKU, source warehouse, destination warehouse, quantity).
- Updated warehouse stock levels post-transfer.
- Records of demand fulfillment and pullbacks.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate warehouse stock or demand data can lead to inefficient or incorrect transfers.*
- **Logistical constraints not modeled:** *Physical limitations (e.g., truck capacity, facility limitations) not accounted for can make recommendations unfeasible.*
- **Transfers across distant warehouses:** *Can be costly, requiring careful cost-benefit analysis.*
- **Circular transfers:** *Poorly designed rules might lead to stock being continuously transferred between two warehouses without true optimization.*

9. Next Steps

The IWHT movements are sent to the Distribution Output module for reporting and integration with warehouse management systems.

10. Common Questions

1. How are warehouse groups configured for transfers?
2. What is the difference between “demanding pullbacks” and “other pullbacks”?
3. Can this module account for transfer costs?
4. How does it prioritize which demands to fulfill first?
5. What metrics are used to determine if a warehouse has excess stock?

11. Quick Reference

- **Purpose:** Rebalance inventory between warehouses.
 - **Key Inputs:** Warehouse stock, aggregated demand, warehouse groups.
 - **Output:** Inter-warehouse transfer recommendations.
 - **Impact:** Optimizes network inventory, reduces holding costs.
-

8.11. Distribution Compute Warehouse Inventory

1. Module Name & Business Objective

Module Name: Distribution Compute Warehouse Inventory **Business Objective:** This module focuses on organizing and computing warehouse inventory specifically for Inter-Store Transfer (IST) scenarios. Its key objective is to create a “virtual warehouse stock” from leftover store inventory, which then becomes available for outward transfers within the IST framework. It also manages initial inward/outward maps for stores, facilitating efficient store-to-store movements.

2. Required Information

- **Operational Data:**
 - Leftover store inventory (from Distribution Allocation).
 - IST group definitions (regional warehouse maps).
 - Store-to-warehouse mappings.

3. Business Settings

Configuration may include parameters for aggregating store inventory into regional virtual warehouses or thresholds for defining initial inward/outward capabilities of stores.

4. Process Flow

graph LR

A[Leftover Store Inventory] --> B(Create Regional Warehouse Map)

B --> C(Calculate Virtual Warehouse Stock)

C --> D(Manage Inward/Outward Store Maps)

D --> E[IST-Ready Warehouse Inventory]

5. Step-by-Step Actions

1. **Receive** *leftover store inventory* from the core allocation module.
2. **Define** a regional warehouse map or IST groups, *grouping stores for potential transfers*.
3. **Aggregate** *leftover store inventory within these groups* to create a “virtual warehouse stock” that can be drawn upon for IST.
4. **Calculate** the *istSkuWhStockMap*, representing this virtual stock.
5. **Manage** *initial inward/outward maps for stores, determining which stores are set to receive or send stock initially*.
6. **Provide** *this structured inventory data* for the Inter-Store Transfer (IST) module.

6. Sample Scenario

After initial allocations, Store A has 5 excess units of SKU 123, and Store B has 3 excess units. Both are in IST Group ‘North’. This module would *aggregate* these 8 units and *create* a “virtual warehouse stock” for SKU 123 within the ‘North’ region. This virtual stock is then *available* for other stores in ‘North’ that need SKU 123.

Key Takeaway: Creating a virtual warehouse stock enables efficient rebalancing of inventory between stores without routing through a physical warehouse.

7. Results

- *istSkuWhStockMap* (virtual warehouse stock for IST).
- Regional warehouse map.
- Initial inward/outward store maps.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate leftover store inventory or IST group definitions can lead to an incorrect virtual stock, hindering effective IST.*
- **Store not assigned to IST group:** *Excess stock in such stores will not be included in the virtual warehouse, missing transfer opportunities.*
- **Negative virtual stock:** *Indicates data inconsistency, potentially from errors in previous allocation modules.*

9. Next Steps

The computed virtual warehouse inventory is a critical input for the Distribution Inter-Store Transfer (IST) module and, for IST scenarios, the Distribution Planogram Incrementer.

10. Common Questions

1. How is “virtual warehouse stock” calculated?
2. What is the purpose of the regional warehouse map?
3. How do we determine if a store is “inwarding” or “outwarding”?
4. Does this module account for product attributes when aggregating virtual stock?
5. What happens if a store’s leftover inventory is too small to contribute meaningfully?

11. Quick Reference

- **Purpose:** Prepare IST-specific warehouse inventory.
 - **Key Inputs:** Leftover store inventory, IST group definitions.
 - **Output:** `istSkuWhStockMap` (virtual stock), store inward/outward maps.
 - **Impact:** Enables efficient inter-store transfers.
-

8.12. Distribution Planogram Incrementer

1. Module Name & Business Objective

Module Name: Distribution Planogram Incrementer **Business Objective:** This module aims to dynamically adjust the capacity of store planograms based on actual sales performance. By analyzing historical revenue and Average Selling Price (ASP) at detailed levels (store-category, store-category-AG), it calculates and applies increment ratios. This ensures that planograms evolve to optimally reflect demand, maximizing product visibility, depth (OD), and sales potential within physical store layouts.

2. Required Information

- **Sales Data:**
 - Historical revenue data (from Distribution Compute Revenue).
 - Average Selling Price (ASP) data.
- **Product Data:**
 - Category definitions.
 - Attribute Group (AG) mappings.
- **Operational Data:**
 - Current planogram configurations.
 - Store definitions.

3. Business Settings

Configuration parameters might include thresholds for incrementing planograms (e.g., minimum revenue increase), maximum increment percentages to prevent over-expansion, and the hierarchy level at which increments are applied (e.g., category, AG).

4. Process Flow

graph LR

A[Historical Sales (Revenue, ASP)] --> B[Analyze Sales by Store-Category-AG]

B --> C[Calculate Increment Ratios]

C --> D[Apply Ratios to Planograms]

D --> E[Adjusted Planogram Data]

5. Step-by-Step Actions

1. **Retrieve** *historical sales data*, including revenue and ASP, for various store-category and store-category-Attribute Group (AG) combinations.
2. **Analyze** *sales trends and performance* at these granular levels.
3. **Calculate** *planogram increment ratios, determining how much the planogram capacity for stocks and options should increase based on sales performance.*
4. **Apply** *these calculated increment ratios to the existing planogram definitions, effectively expanding space or options for high-performing categories/AGs.*
5. **Update** *the planogramIncrementRatio in the distribution data, making the adjusted capacities available for allocation.*

6. Sample Scenario

Store A's 'Denim' category has seen a 20% revenue increase over the last quarter. This module would *calculate* an increment ratio (e.g., +10%) for the 'Denim' category in Store A. This ratio is then *applied* to the planogram, indicating that Store A now has slightly more shelf space or display capacity available for denim products, leading to a higher OD (Optimum Depth) for those items. *Key Takeaway: Dynamic planogram adjustments ensure physical store layouts align with actual demand, maximizing sales within space constraints.*

7. Results

- Calculated planogramIncrementRatio for various store-category and store-category-AG combinations.
- Adjusted planogram capacities for subsequent allocation.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate or incomplete sales data will lead to incorrect or no planogram adjustments.*
- **Inconsistent data granularity:** *Mismatch between sales data granularity and planogram hierarchy can cause calculation errors.*
- **Overly aggressive increments:** *Can lead to planograms requesting more stock than physically possible or justifiable.*
- **Zero sales for new categories:** *May prevent any increment, even if the category is strategically important, requiring initial override rules.*

9. Next Steps

The calculated planogram increment ratios are fed back into the Distribution Allocation module, which uses them to adjust planogram-based allocation criteria. It can also be influenced by Distribution Compute Warehouse Inventory for IST scenarios.

10. Common Questions

1. How frequently are planograms incremented?
2. What is the maximum allowable increment for a planogram?
3. Can this module decrease planogram capacity for underperforming categories?
4. How does it handle seasonal fluctuations in sales for increments?
5. What data sources are used for ASP calculation?

11. Quick Reference

- **Purpose:** Adjust planogram capacity based on sales.
 - **Key Inputs:** Revenue, ASP, category data, existing planograms.
 - **Output:** Planogram increment ratios.
 - **Impact:** Optimizes physical display and stock depth (OD).
-

8.13. Distribution Output

1. Module Name & Business Objective

Module Name: Distribution Output **Business Objective:** This module is the final stage of the distribution pipeline. Its objective is to gather all the calculated allocation decisions, final inventory states, and computed metrics from the preceding modules and transform them into easily consumable, denormalized tables and reports. This ensures that the results are available for business analysis, reporting, integration with other systems (e.g., WMS, POS), and auditing the distribution process.

2. Required Information

- **Operational Data:**
 - Final allocation decisions (from Distribution Allocation).
 - Warehouse stock summaries (from various inventory modules).
 - Inter-store transfer movements (from Distribution Inter-Store Transfer).
 - Inter-warehouse transfer movements (from Distribution Inter-Warehouse Transfer).
 - Fill rate analyses.
 - Segmentation results (from Distribution Segmentation).
 - Ranking reports (from Distribution Ranking).

3. Business Settings

Configuration in `args.properties` would specify output file paths, file formats (e.g., CSV, JSON), and which specific reports or tables to generate.

4. Process Flow

graph TD

```
A[Final Allocations, Transfers, Inventory, Metrics] -->
B(Consolidate Results)
B --> C(Transform to Denormalized Tables)
C --> D(Generate Reports)
D --> E[Output Files & Reports]
```

5. Step-by-Step Actions

1. **Collect** *all final allocation decisions, inter-store transfers, inter-warehouse transfers, and updated inventory states* from the relevant preceding modules.
2. **Consolidate** *various computed metrics* such as fill rates, health rates, segmentation, and ranking results.
3. **Transform** *the in-memory data* into denormalized table structures suitable for external consumption, *simplifying data access for analysis*.
4. **Generate** *specific reports* (e.g., allocation summary, transfer manifest, stock-on-hand report), *tailored for different business stakeholders*.
5. **Persist** *all generated outputs* to the configured output directory in the specified formats.

6. Sample Scenario

After a full distribution run, this module would *generate* a `DailyAllocationReport.csv` detailing how many units of each SKU were allocated to each store. It would also *create* an `IST_Manifest.json` listing all recommended inter-store transfers, and a `WarehouseStockSummary.csv` showing final warehouse inventory. *Key Takeaway: This module makes the complex outputs of the distribution pipeline understandable and actionable for the business.*

7. Results

- Detailed allocation reports (e.g., SKU, store, allocated quantity, reason).
- Warehouse stock summaries.
- Inter-store and inter-warehouse transfer manifests.
- Fill rate and store health analysis reports.
- Denormalized tables for deeper analytical querying.

8. Unusual Cases

- **Missing or delayed data feed:** *If previous modules fail to provide complete data, output reports will be incomplete or inaccurate.*
- **Disk space issues:** *Large output files can lead to disk space exhaustion, causing run failures.*

- **Incorrect output path configuration:** *Files may not be saved in the expected location.*
- **Schema mismatches:** *Changes in internal data structures not reflected in output schema can cause export errors.*

9. Next Steps

The generated output files are consumed by business intelligence tools, warehouse management systems, and other downstream retail systems for execution, reporting, and analysis.

10. Common Questions

1. What are the key output files I should review after a run?
2. How can I verify the accuracy of the allocation quantities in the reports?
3. Is it possible to customize the output report formats or content?
4. How are transfer recommendations communicated to logistics teams?
5. What happens if the output directory is full or inaccessible?

11. Quick Reference

- **Purpose:** Generate final reports and data for external systems.
 - **Key Inputs:** All final data from preceding modules.
 - **Output:** Denormalized tables, reports (CSV, JSON).
 - **Impact:** Provides actionable insights, enables integration.
-

8.14. Distribution Input Validation

1. Module Name & Business Objective

Module Name: Distribution Input Validation **Business Objective:** This module acts as the gatekeeper for all incoming data, ensuring that the distribution pipeline operates on reliable and consistent information. Its objective is to perform comprehensive checks for data integrity, format correctness, and logical consistency across various input sources (e.g., store configurations, planograms, SKU depths, warehouse mappings). By identifying and flagging errors or warnings early, it prevents incorrect distribution calculations and ensures the overall quality of the run.

2. Required Information

- **Product Data:**
 - SKU depth (OD) configurations.
 - Planogram definitions.
 - Attribute Group (AG) mappings.
- **Operational Data:**
 - Distribution-enabled store lists.
 - Warehouse-store mappings.
 - AOP (Annual Operating Plan) data.

- Seasonality overrides.
- JIT (Just-In-Time) settings.
- Channel definitions.

3. Business Settings

Configuration parameters might define strictness levels for validation (e.g., warnings vs. errors), or specific data ranges for certain inputs.

4. Process Flow

graph LR

```
A[Raw Input Data Files] --> B(Load & Parse Input Data)
B --> C(Perform Data Integrity Checks)
C --> D(Validate Consistency & Business Rules)
D --> E[Validation Report & Status]
```

5. Step-by-Step Actions

1. **Load** *various input data files* such as store lists, planograms, SKU depths, etc.
2. **Verify** *data types and formats, ensuring they conform to expected schemas.*
3. **Check** *for completeness, identifying missing mandatory fields or records.*
4. **Validate** *logical consistency* (e.g., ensuring all SKUs referenced in planograms actually exist).
5. **Apply** *business rule checks* (e.g., validating that SKU depths are within acceptable ranges).
6. **Flag** *errors or warnings, providing detailed messages for investigation.*
7. **Determine** *overall data readiness* for the distribution run.

6. Sample Scenario

The input `planogram.csv` specifies a non-existent `storeId`. This module would *detect* this inconsistency and *flag* an error, preventing Distribution Data Preparation from processing an invalid planogram. Similarly, it would *warn* if a significant number of SKUs are missing their Optimum Depth (OD) configuration. *Key Takeaway: Robust input validation is the first line of defense against erroneous distribution outcomes.*

7. Results

- A validation report detailing errors and warnings.
- A boolean status indicating if the input data is valid for processing.

8. Unusual Cases

- **Missing or delayed data feed:** *The most common issue; leads to a run failure if critical data is absent.*
- **Corrupted file format:** *Malformed CSVs or JSONs can cause parsing errors.*
- **Logical inconsistencies:** *e.g., a store marked as distribution-enabled but not present in the store master file.*

- **Unexpected data values:** *e.g., negative inventory levels or extremely high ROS values that indicate a data entry error.*

9. Next Steps

If validation passes, the Distribution Data Preparation module proceeds to load and process the validated data. If validation fails, the run is typically halted, and issues must be resolved.

10. Common Questions

1. What are the most common validation errors?
2. Can I configure the validation rules (e.g., make a warning an error)?
3. Where can I find the detailed validation report?
4. What happens if only a few records fail validation?
5. How can I test my input files before a full run?

11. Quick Reference

- **Purpose:** Validate all input data.
 - **Key Inputs:** Store lists, planograms, SKU depths, mappings.
 - **Output:** Validation report (errors/warnings), status.
 - **Impact:** Prevents incorrect distribution calculations.
-

8.15. Distribution End Date Validation

1. Module Name & Business Objective

Module Name: Distribution End Date Validation **Business Objective:** This module specifically ensures that all necessary inventory and operational data are present and relevant as of the specified endDate for the distribution run. Its objective is to verify the existence of critical data points like store stock, Goods In Transit (GIT), open orders, and in-transit transfers. It also confirms that there are “live” styles in season and that core stores are enabled for distribution, guaranteeing that the distribution process has sufficient and timely data for the defined period.

2. Required Information

- **Operational Data:**
 - Store stock as of endDate.
 - Goods In Transit (GIT) data as of endDate.
 - Open order data as of endDate.
 - In-transit transfer data as of endDate.
 - Warehouse mappings.
 - Lists of “live” styles in season.
 - Distribution-enabled stores.

3. Business Settings

The `endDate` parameter itself is a key setting. Other configurations might include minimum thresholds for “live” styles or distribution-enabled stores to consider the data sufficient.

4. Process Flow

graph LR

A[End Date & Related Inventory Data] --> B(Check Inventory Data Presence)

B --> C(Verify Live Styles & Stores)

C --> D[End Date Validation Status]

5. Step-by-Step Actions

1. **Receive** the specified *endDate* for the distribution run.
2. **Check** for the presence and completeness of store stock data as of this *endDate*.
3. **Verify Goods In Transit (GIT)** and open order data up to the *endDate*.
4. **Confirm** the existence of in-transit transfers relevant to the *endDate*.
5. **Validate** that there are ‘live’ styles currently in season.
6. **Ensure** that a sufficient number of core distribution-enabled stores are present.
7. **Flag** any critical data gaps or inconsistencies related to the *endDate*.

6. Sample Scenario

A distribution run is scheduled for 2025-05-23. This module would *check* if the store stock file for 2025-05-23 is available. It would *verify* that there are active styles for the current season. If the GIT file for this date is missing, it would *flag* a critical error, as it would lead to incorrect inventory calculations. *Key Takeaway: Timely and complete data for the endDate is crucial for the accuracy of current allocation decisions.*

7. Results

- A validation status indicating if all necessary data for the *endDate* is present and consistent.
- Error messages if critical data is missing or inconsistent.

8. Unusual Cases

- **Missing or delayed data feed:** *The most common issue; e.g., the daily inventory snapshot for endDate is not yet available.*
- **Incorrect endDate:** *If the endDate is in the future or significantly in the past, leading to irrelevant or missing data.*
- **No live styles:** *If the system cannot identify any active styles for the current season, it indicates a major setup issue.*
- **Few distribution-enabled stores:** *Might indicate a configuration error if only a small subset of expected stores are active.*

9. Next Steps

If this validation passes, the `Distribution Data Preparation` module proceeds, assured that the current inventory snapshot is complete. If it fails, the run typically cannot proceed.

10. Common Questions

1. What specific data feeds are checked for the `endDate`?
2. What happens if `endDate` inventory is missing but we have previous days?
3. How does the system define a “live” style for the current season?
4. Can I override an `endDate` validation failure?
5. What’s the difference between this validation and `Distribution Input Validation`?

11. Quick Reference

- **Purpose:** Validate data completeness for `endDate`.
 - **Key Inputs:** `endDate`, current inventory snapshots (stock, GIT, open orders), live styles.
 - **Output:** `endDate` validation status.
 - **Impact:** Ensures timely and relevant data for the run.
-

8.16. Distribution Group Module

1. Module Name & Business Objective

Module Name: `Distribution Group Module` **Business Objective:** This module serves as the central orchestrator for the entire distribution pipeline. Its objective is to define and manage the precise sequence of execution for all individual distribution sub-modules, from data preparation and validation to core allocation logic and final output generation. By ensuring that each step is performed in the correct order, with all dependencies met, it guarantees the coherent and efficient execution of the complete distribution process.

2. Required Information

- **Operational Data:**
 - Configuration for the `runOption` (e.g., `REPLENISHMENT_ONLY`, `FULL_RUN`).
 - Status from validation modules.
- **Product Data:** N/A
- **Sales Data:** N/A

3. Business Settings

The primary business setting is the `runOption` in `args.properties`, which dictates the specific flow and modules to be executed. Other settings might control logging levels or error handling behavior.

4. Process Flow

graph TD

```
A[Start Distribution Run] --> B(Input & End Date Validation)
B -- Valid --> C(Data Preparation & Sales Prep)
C --> D(Warehouse Inventory Prep & Reservation)
D --> E(Compute Revenue & Ranking & Segmentation)
E --> F(Core Allocation)
F --> G(Inter-Warehouse Transfer (IWHT))
G --> H(Compute Warehouse Inventory for IST)
H --> I(Inter-Store Transfer (IST))
I --> J(Output Generation)
J --> K[End Distribution Run]
B -- Invalid --> L[Error & Halt]
```

5. Step-by-Step Actions

1. **Initialize** the distribution process, setting up the overall run context.
2. **Invoke** validation modules (e.g., Distribution Input Validation, Distribution End Date Validation) to ensure data readiness.
3. If validation passes, **Orchestrate** data preparation modules (Distribution Data Preparation, Distribution Sales Preparation, Distribution Warehouse Inventory Preparation, Distribution Warehouse Reservation).
4. **Execute** analytical modules (Distribution Compute Revenue, Distribution Ranking, Distribution Segmentation).
5. **Run** the core allocation module (Distribution Allocation), passing all necessary data and configurations.
6. **Coordinate** transfer modules (Distribution Inter-Warehouse Transfer, Distribution Compute Warehouse Inventory, Distribution Inter-Store Transfer) sequentially.
7. **Trigger** the final output generation (Distribution Output) to persist results.
8. **Handle** any errors or exceptions, ensuring graceful termination or detailed logging.

6. Sample Scenario

A daily replenishment run is initiated. This module first *ensures* all daily inventory feeds are valid. It then *starts* data prep, sales prep, and ranking. Once complete, it *triggers* the core Distribution Allocation module. After allocation, it *orchestrates* any inter-warehouse and inter-store transfers, finally *culminating* in the output report generation. *Key Takeaway: The Group Module ensures that the complex distribution pipeline runs smoothly and in the correct order, delivering consistent results.*

7. Results

- Successful completion of the end-to-end distribution run.
- Structured execution of all sub-modules.
- Consolidated logs of the entire process flow.

8. Unusual Cases

- **Missing or delayed data feed:** *Will likely cause early termination during validation or data preparation stages.*
- **Module failure:** *If any sub-module throws an unhandled exception, the orchestrator will typically halt the entire pipeline.*
- **Incorrect runOption:** *May lead to a partial or irrelevant run, missing crucial allocation steps.*
- **Resource limitations:** *Lack of memory or CPU during heavy processing steps might cause the orchestrator to fail.*

9. Next Steps

Upon successful completion, the outputs are available for downstream systems and business users. If a failure occurs, investigation and rerun are required.

10. Common Questions

1. How can I restart a distribution run from a specific failed step?
2. What is the typical runtime for a full distribution run?
3. Can I monitor the progress of the orchestrator in real-time?
4. How does the runOption impact the modules that are executed?
5. What are the key dependencies between the sub-modules?

11. Quick Reference

- **Purpose:** Orchestrate the entire distribution pipeline.
 - **Key Inputs:** runOption, validation status.
 - **Output:** Coordinated execution of all modules.
 - **Impact:** Ensures end-to-end process integrity.
-

8.17. Base Iteration Runner

1. Module Name & Business Objective

Module Name: Base Iteration Runner **Business Objective:** This abstract component defines the fundamental, reusable logic for running allocation iterations. Its objective is to provide a common framework for processing store-style combinations, applying universal checks (e.g., minimum age, overstocking conditions), and delegating specific SKU-level allocation to concrete implementations. This promotes consistency and reduces code duplication across different allocation iteration types.

2. Required Information

- **Product Data:**
 - SKU and style definitions.

- **Operational Data:**

- Store-style combinations for processing.
- Current inventory levels.
- Flags and constraints relevant to iterations (e.g., overstocking status, minimum age, health status).

3. Business Settings

This abstract runner does not have direct business settings, but it defines the interfaces for how concrete runners (e.g., Distribution Iteration Runner) will interact with settings like `min.item.age` or `overstock.tolerance`.

4. Process Flow

graph TD

A[List of Store-Styles] --> B(Common Pre-Checks: Min Age, Healthy)

B --> C{Delegate to Concrete SKU Allocation}

C --> D[SKU-Level Allocation Decisions]

5. Step-by-Step Actions

1. **Receive** *a list of store-style combinations* to be processed in the current iteration.
2. For each combination, **Perform** *common pre-allocation checks*, such as:
 - *Verifying minimum age* for the style in the store.
 - *Checking if the store-style is currently “healthy”* or overstocked.
3. If checks pass, **Delegate** *to a concrete implementation* (e.g., Distribution Iteration Runner) to perform the specific SKU-level allocation logic.
4. **Manage** *flags and constraints* pertinent to the iteration, *ensuring adherence to overarching rules*.

6. Sample Scenario

During a replenishment iteration, this Base Runner receives “Store A - T-shirt Style” and “Store B - Jacket Style”. For “Store A”, it *checks* that the T-shirt style meets the minimum age requirement (not too new). For “Store B”, it *verifies* that the Jacket style is not already overstocked. If these *checks pass*, it then *passes* the processing to a more specific runner to calculate the exact SKU quantities to allocate. *Key Takeaway: The Base Iteration Runner provides foundational consistency and rule enforcement across all allocation iterations.*

7. Results

- A consistent framework for running various allocation iterations.
- Common pre-checks applied uniformly.
- Delegation mechanism for SKU-level allocation logic.

8. Unusual Cases

- **Invalid input to checks:** *If store-style combinations are malformed, pre-checks might fail unexpectedly.*
- **Unexpected flag states:** *Incorrectly set flags (e.g., all styles marked as overstocked) can prevent any allocation from proceeding.*
- **Sub-optimal delegation:** *If the concrete runner implementation is flawed, the overall allocation will be incorrect despite valid pre-checks.*

9. Next Steps

This abstract component is extended and utilized by `Distribution Iteration Runner` and `Distribution IST Runner` to implement specific allocation strategies.

10. Common Questions

1. How does this runner define “healthy” or “overstocked” for a store-style?
2. Can new common checks be easily added to this base class?
3. What happens if a store-style fails a pre-check? Is it skipped?
4. Does this base runner handle any inventory adjustments directly?
5. How is `min.item.age` determined?

11. Quick Reference

- **Purpose:** Abstract framework for allocation iterations.
 - **Key Inputs:** Store-styles, iteration constraints.
 - **Output:** Consistent pre-checks, delegation.
 - **Impact:** Promotes consistency in allocation logic.
-

8.18. Distribution Iteration Runner

1. Module Name & Business Objective

Module Name: `Distribution Iteration Runner` **Business Objective:** This component extends the `BaseIterationRunner` to provide concrete implementations for various general distribution allocation iterations, such as replenishment, fresh allocation, depth increment, and non-pivotal sizes. Its objective is to manage the specific allocation rules for each iteration type, including calculating suggested allocations, handling store and warehouse stock, updating planogram criteria, and incorporating nuanced logic like color-based allocations and JIT (Just-In-Time) adjustments.

2. Required Information

- **Sales Data:**
 - Rate of Sale (ROS).

- **Product Data:**

- Optimum Depth (OD), Optimum Width (OW), Ideal Size Set (ISS).
- Attribute Group (AG) definitions.
- Color-based allocation rules.

- **Operational Data:**

- Current store stock and warehouse stock.
- Planogram criteria and increment ratios.
- JIT (Just-In-Time) settings.
- Store-style health status.

3. Business Settings

Numerous configuration parameters related to specific allocation rules for replenishment, fresh allocation, non-pivotal sizes, JIT, and color-based allocations would be controlled via `args.properties`. Examples include `replenishment.min.units`, `fresh.allocation.factor`, `non.pivotal.sku.ratio`.

4. Process Flow

graph TD

```
A[Store-Style, Inventory, ROS, OD/OW/ISS] --> B(Determine Suggested Allocation)
B --> C(Apply Specific Allocation Rules: JIT, Color, etc.)
C --> D(Update Store & Warehouse Stock)
D --> E(Adjust Planogram Criteria)
E --> F[SKU-Level Allocation Decisions & Updated Data]
```

5. Step-by-Step Actions

1. **Receive** *store-style combinations* from the Base Iteration Runner, along with current inventory, ROS, and OD/OW/ISS targets.
2. **Calculate** *the suggested allocation quantity* for each SKU, *aiming to fill demand or reach target inventory levels*.
3. **Apply** *iteration-specific business rules*, such as:
 - **JIT adjustments:** *Sending stock only when immediately needed.*
 - **Color-based allocations:** *Ensuring a balanced color assortment.*
 - **Non-pivotal size handling:** *Allocating based on specific rules for less common sizes.*
4. **Update** *internal store and warehouse stock figures* based on allocated quantities, *reflecting real-time inventory changes*.
5. **Adjust** *planogram criteria* where necessary, *incorporating any increments or changes from the Planogram Incrementer*.
6. **Record** *the final allocation decisions* for each SKU.

6. Sample Scenario

For a “replenishment” iteration, this runner *identifies* Store A needs more units of Style X (SKU 123) based on its ROS and current stock falling below OD. It *calculates* a replenishment allocation of 5 units, ensuring the ISS (Ideal Size Set) is met. If Style X has a “red” variant that is a TOP_SELLER, it might *prioritize*

allocation of the red SKU due to color-based rules. It then *decrements* warehouse stock by 5 units and *increases* Store A's stock. *Key Takeaway: This runner translates business rules into concrete, SKU-level allocation decisions for various scenarios.*

7. Results

- Specific SKU-level allocation quantities per store for the current iteration.
- Updated skuStoreStock and skuWhStock data.
- Adjusted planogram criteria.

8. Unusual Cases

- **Insufficient warehouse stock:** *If warehouse stock cannot meet suggested allocations, the system must prioritize or allocate partially.*
- **Overstocking despite rules:** *If a store remains overstocked after replenishment, it might indicate an issue with OD or other parameters.*
- **Complex color rules:** *Can lead to unexpected assortments if not carefully defined.*
- **JIT conflicts:** *If JIT settings clash with minimum allocation quantities.*

9. Next Steps

The allocation decisions and updated inventory are fed back to the Distribution Allocation module for further processing or finalization, and ultimately contribute to the output reports.

10. Common Questions

1. What's the difference between replenishment and fresh allocation?
2. How does the system handle non-pivotal sizes differently?
3. Can I define specific JIT rules for certain SKUs or stores?
4. How are color-based allocations managed when stock is limited?
5. What happens if an ISS cannot be fully met due to stock constraints?

11. Quick Reference

- **Purpose:** Implement concrete SKU-level allocation logic.
 - **Key Inputs:** Store/warehouse stock, ROS, OD/OW/ISS, planogram.
 - **Output:** SKU-level allocation plans, updated stock.
 - **Impact:** Drives actual inventory movements based on business rules.
-

8.19. Distribution IST Runner

1. Module Name & Business Objective

Module Name: Distribution IST Runner **Business Objective:** This component specifically extends the `DistributionIterationRunner` to manage Inter-Store Transfer (IST) allocation iterations. Its objective is to enable and optimize direct

inventory movements between stores by overriding standard allocation logic and stock calculation methods with IST-specific rules. This ensures that allocation decisions align with the IST framework, effectively leveraging “virtual warehouse stock” from IST regions and validating outward movements to rebalance inventory across the store network.

2. Required Information

- **Product Data:**
 - SKU and style definitions.
- **Operational Data:**
 - `istSkuWhStockMap` (virtual warehouse stock for IST, from Distribution Compute Warehouse Inventory).
 - Current store stock levels.
 - IST group definitions.
 - IST outwards criteria (e.g., minimum excess stock for a store to be a donor).

3. Business Settings

IST-specific thresholds for `getIstWhStock` (e.g., minimum quantity to be considered virtual warehouse stock) and `checkIstOutwardsCriteria` (e.g., specific rules for when a store can send stock) would be configured.

4. Process Flow

```
graph TD
    A[Store-Style, IST Virtual WH Stock, Outwards Criteria] --> B[Override Stock Calculation]
    B --> C[Apply IST Allocation Logic]
    C --> D[Validate Outwards Movements]
    D --> E[IST Transfer Decisions & Updated Data]
```

5. Step-by-Step Actions

1. **Receive** *store-style combinations* from the Base Iteration Runner, along with IST-specific inventory and rules.
2. **Override** the standard `getWhStock` method to use `getIstWhStock`, calculating available warehouse stock from IST regions (i.e., virtual warehouse stock).
3. **Apply** the standard allocation logic from `DistributionIterationRunner` but now operating on this IST-specific virtual stock.
4. **Implement** `checkIstOutwardsCriteria`, validating if a store is eligible to send stock based on its excess and health.
5. **Determine** SKU-level transfer quantities from donor stores to recipient stores within IST groups.
6. **Update** store stock levels to reflect these inter-store transfers.
7. **Record** all IST transfer decisions.

6. Sample Scenario

Store A has 10 units of SKU 456, exceeding its OD (Optimum Depth) by 3 units. Store B, in the same IST group, needs 2 units of SKU 456. This runner would *use* the `istSkuWhStockMap` (which includes Store A's excess as virtual stock) to *suggest* transferring 2 units from Store A to Store B. It then *applies* `checkIstOutwardingCriteria` to confirm Store A is healthy enough to outward stock. *Key Takeaway: This runner provides the specialized logic required for efficient, direct store-to-store inventory rebalancing.*

7. Results

- Recommended inter-store transfer quantities per SKU.
- Updated `skuStoreStock` reflecting IST movements.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate `istSkuWhStockMap` or store health can lead to invalid transfer suggestions.*
- **Tight outwarding criteria:** *If criteria are too strict, fewer transfers will occur, potentially leaving excess stock stranded.*
- **Network restrictions:** *Physical transfer limitations not modeled can lead to unexecutable recommendations.*
- **Circular transfers:** *If a pair of stores keep “sending” and “receiving” the same stock due to imperfect rules.*

9. Next Steps

The IST transfer decisions are returned to the Distribution Inter-Store Transfer module for finalization and are then passed to the Distribution Output module for reporting.

10. Common Questions

1. How does `getIstWhStock` differ from regular `getWhStock`?
2. What are the key criteria for `checkIstOutwardingCriteria`?
3. Can this runner prioritize which stores send stock?
4. What happens if the source store stock is insufficient after other allocations?
5. How does this runner handle different IST groups or regions?

11. Quick Reference

- **Purpose:** Specialized runner for Inter-Store Transfers.
 - **Key Inputs:** Virtual WH stock, IST rules.
 - **Output:** SKU-level IST transfer recommendations.
 - **Impact:** Enables direct store-to-store inventory rebalancing.
-

9. Appendix

9.1. Full Parameter Reference

Please refer to the `args.properties` file in the project root directory for a comprehensive list of all configurable parameters, their default values, descriptions, and valid ranges.

9.2. Version History

- **Version 1.0 (2025-05-23):** Initial release of the distribution algorithm and comprehensive documentation.

9.3. Contact & Support Info

For any questions, issues, or support requests regarding the distribution algorithm, please contact the development team:

- **Email:** `support@example.com`
- **Internal Chat Channel:** `#distribution-support`
- **Issue Tracker:** Jira Project DIST

Tutorial: distribution

The distribution system processes inventory allocation and transfers through a phased pipeline. Data preparation and validation modules initialize the core data, followed by sales analysis, revenue computation, and ranking. Segmentation categorizes styles for differentiated treatment. The central allocation module, leveraging specialized iteration runners, determines optimal inventory distribution. Inter-store and inter-warehouse transfers rebalance stock across the network. Finally, all computed results are consolidated and outputted for reporting and downstream systems. The entire process is orchestrated by a group module.

flowchart TD

```
A0["Distribution Data Preparation"]
A1["Distribution Sales Preparation"]
A2["Distribution Warehouse Inventory Preparation"]
A3["Distribution Warehouse Reservation"]
A4["Distribution Compute Revenue"]
A5["Distribution Ranking"]
A6["Distribution Segmentation"]
A7["Distribution Allocation"]
A8["Distribution Inter-Store Transfer (IST)"]
A9["Distribution Inter-Warehouse Transfer (IWHT)"]
A10["Distribution Compute Warehouse Inventory"]
A11["Distribution Planogram Incrementer"]
A12["Distribution Output"]
A13["Distribution Input Validation"]
A14["Distribution End Date Validation"]
A15["Distribution Group Module"]
A16["Base Iteration Runner"]
A17["Distribution Iteration Runner"]
A18["Distribution IST Runner"]
A13 -- "validates" --> A0
A14 -- "validates" --> A0
A0 -- "data_flow" --> A1
A0 -- "data_flow" --> A2
A0 -- "data_flow" --> A3
A1 -- "data_flow" --> A4
A1 -- "data_flow" --> A5
A1 -- "data_flow" --> A6
A2 -- "data_flow" --> A4
A2 -- "data_flow" --> A7
A3 -- "business_logic" --> A7
A4 -- "data_flow" --> A5
A4 -- "data_flow" --> A6
A5 -- "data_flow" --> A7
A6 -- "data_flow" --> A7
A7 -- "data_flow" --> A10
```

```
A7 -- "data_flow" --> A9
A7 -- "calls" --> A11
A10 -- "data_flow" --> A8
A10 -- "data_flow" --> A11
A11 -- "data_flow" --> A7
A7 -- "output_generation" --> A12
A8 -- "output_generation" --> A12
A9 -- "output_generation" --> A12
A16 -- "extends" --> A17
A17 -- "extends" --> A18
A7 -- "uses" --> A17
A8 -- "uses" --> A18
A15 -- "orchestrates" --> A0
A15 -- "orchestrates" --> A2
A15 -- "orchestrates" --> A3
A15 -- "orchestrates" --> A1
A15 -- "orchestrates" --> A4
A15 -- "orchestrates" --> A5
A15 -- "orchestrates" --> A6
A15 -- "orchestrates" --> A7
A15 -- "orchestrates" --> A9
A15 -- "orchestrates" --> A10
A15 -- "orchestrates" --> A8
A15 -- "orchestrates" --> A12
```

Chapters

This document outlines the **Master Report** for the merchandising algorithm, providing a comprehensive overview of its architecture, functionality, and usage.

1. Title & Metadata

- **Project Name:** distribution
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** The distribution algorithm optimizes product assortments (OD, OW, ISS) across all retail locations to maximize sales and minimize inventory inefficiencies based on forecasted demand (ROS).

2. Executive Summary

The distribution algorithm, version 1.0, is a critical component of our merchandising strategy, designed to intelligently determine the ideal product assortment at each store and category level. By leveraging historical sales data, inventory levels, and predefined business rules, it provides actionable recommendations for Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) for all SKUs. This ensures that our stores are stocked with the right products in the right quantities, aligning with customer demand and maximizing profitability.

- **Key outcomes or KPIs:**
 - Increased Rate of Sale (ROS) for recommended SKUs.
 - Reduced stockouts and improved on-shelf availability.
 - Optimized inventory turnover and reduced holding costs.
 - Enhanced customer satisfaction through better product availability.
 - Improved decision-making for merchandising and planning teams.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Planning Teams, Store Operations, Business Intelligence Analysts.

3. Architecture & Data Flow

graph TD
A[Raw Sales & Inventory Data] --> B{Data Ingestion & Validation};
B --> C[Data Preprocessing & Feature Engineering];
C --> D[Demand Forecasting & ROS Calculation];
D --> E[Assortment Optimization Engine];
E --> F[Recommendation Generation & Output];
F --> G[Reporting & Visualization Tools];

```
G --> H[Merchandising & Planning Teams];
```

```
subgraph Core Algorithm
```

```
    D -- "Forecasted ROS" --> E;
```

```
    E -- "OD, OW, ISS Logic" --> F;
```

```
end
```

- **Component Bullets:**

- **Data Ingestion & Validation:** Responsible for collecting raw data from various sources and ensuring its quality and completeness.
- **Data Preprocessing & Feature Engineering:** Cleans, transforms, and enriches raw data, creating relevant features for downstream models (e.g., seasonal indicators, price elasticity).
- **Demand Forecasting & ROS Calculation:** Utilizes historical sales and external factors to predict future demand and calculate the anticipated Rate of Sale (ROS) for each SKU/location.
- **Assortment Optimization Engine:** The core intelligence, applying business rules and optimization algorithms to determine OD, OW, and ISS based on forecasted ROS, inventory, and strategic objectives.
- **Recommendation Generation & Output:** Formats the optimized assortment recommendations into structured outputs consumable by other systems or end-users.
- **Reporting & Visualization Tools:** Provides user interfaces for viewing, analyzing, and acting upon the algorithm's recommendations.

- **High-level data sources:**

- Point-of-Sale (POS) data (historical sales, returns)
- Inventory Management System (IMS) data (current stock levels, inbound shipments, transfers)
- Product Master Data (SKU attributes, categories, pricing, costs, vendor details)
- Store Master Data (store types, locations, capacities)
- Promotional Calendar Data
- External Market Data (e.g., weather, local events, competitor pricing – if applicable)

4. Configuration & Data Dictionary

- **Settings Table** (args.properties):

Parameter Name	Description	Default Value	Valid Range
optimization_horizon_weeks	The time horizon (in weeks) for which demand forecasts are generated and optimization is performed.	8	4-12
min_ROS_threshold	Minimum acceptable Rate of Sale (units/week) for a SKU to be considered for stocking.	0.5	0.1-2.0
target_fill_rate	Desired in-stock percentage for key product categories.	0.95	0.90-0.99
category_growth_factor	Multiplier applied to forecasted ROS for high-growth categories.	1.10	1.00-1.50
size_set_variance_tolerance	Maximum allowed variance in ROS for sizes within an ISS to be considered optimal.	0.20	0.05-0.50
inventory_turn_target_days	Target number of days to turn inventory for a category/store.	60	30-180

- **Shared Data Fields** (`common-data.json`):

- `SKU_ID`: Unique identifier for each Stock Keeping Unit.
- `Store_ID`: Unique identifier for each retail location.
- `Category_ID`: Identifier for the product category (e.g., “Men’s Denim”, “Women’s Footwear”).
- `Subcategory_ID`: Identifier for the product subcategory.
- `AttributeGroup_Color`: The primary color Attribute Group of the SKU.
- `AttributeGroup_Size`: The specific size Attribute Group of the SKU.
- `Date_Key`: Date of the data record.
- `Sales_Units_Daily`: Units sold per day.
- `Sales_Revenue_Daily`: Revenue generated per day.
- `Stock_On_Hand_Units`: Current physical inventory units at store level.
- `In_Transit_Units`: Units currently en route to the store.
- `Cost_Per_Unit`: Cost of the SKU.
- `Retail_Price_Per_Unit`: Selling price of the SKU.
- `Days_Since_Last_Sale`: Number of days since the SKU last sold in that store.

- **Outputs Catalog:**

- `od_recommendations.csv`: Tabular output detailing recommended Optimum Depth (OD) per category/store.
- `ow_recommendations.csv`: Tabular output detailing recommended Optimum Width (OW) per category/store.
- `iss_recommendations.csv`: Tabular output detailing recommended Ideal Size Set (ISS) per SKU/category/store.
- `forecasted_ros_summary.csv`: Summary of forecasted Rate of Sale (ROS) for all active SKUs by location.
- `inventory_rebalancing_suggestions.csv`: Recommendations for inter-store transfers or returns based on optimized assortment.
- `performance_metrics.json`: JSON output containing key performance indicators (KPIs) like projected stockout rates, projected inventory turns, and realized ROS.

5. End-to-End Flow

The end-to-end process of the distribution algorithm can be summarized in these sequential steps:

1. **Data Ingestion & Preparation:** Raw historical sales, inventory, and product master data are collected from various enterprise systems. This data undergoes validation, cleaning, and transformation, ensuring consistency and accuracy. Features like sales trends, seasonality, and promotional impacts are engineered.
2. **Demand Forecasting & ROS Calculation:** Utilizing the prepared data, the algorithm runs predictive models to forecast future demand for each SKU at each store location. Based on these forecasts, the Rate of Sale (ROS) is calculated, providing a critical input for assortment optimization.
3. **Assortment Optimization:** The core optimization engine takes the forecasted ROS, current inventory levels, store characteristics (e.g., capacity, historical performance), and business rules (e.g., target fill rate, minimum ROS threshold) as inputs. It then applies complex optimization logic to determine the Optimum

Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) for each category and SKU at every location.

4. **Recommendation Generation:** The optimized assortment plans are translated into actionable recommendations. These recommendations are formatted into structured outputs (e.g., CSV files) that clearly specify which SKUs, in what quantities, and with which size sets should be stocked at each store.
5. **Reporting & Integration:** The final recommendations and performance metrics are pushed to reporting dashboards for review by merchandising and planning teams. They can then approve, adjust, and integrate these recommendations into their inventory management and ordering systems.

6. Submodule Overview

6.1. Data Ingestion & Preprocessing

1. **Name & Purpose:** Gathers raw data from source systems and prepares it for analysis by cleaning, validating, and structuring it.
2. **Inputs & Outputs:**
 - **Inputs:** Raw POS data, IMS data, Product Master data, Store Master data.
 - **Outputs:** Cleaned and harmonized common-data.json dataset, aggregated historical sales, and inventory snapshots.
3. **Core Logic Summary:**
 - Connects to various databases/APIs to retrieve raw data.
 - Performs data validation checks (e.g., missing values, data type inconsistencies, logical errors).
 - Aggregates granular sales data to daily/weekly SKU-store level.
 - Enriches data with relevant attributes like Days_Since_Last_Sale and AttributeGroup_Color.

6.2. Demand Forecasting & ROS Calculation

1. **Name & Purpose:** Predicts future sales demand and calculates the Rate of Sale (ROS) for each SKU at each store location.
2. **Inputs & Outputs:**
 - **Inputs:** Prepared historical sales data (from Data Ingestion), promotional calendar, external market data (if applicable).
 - **Outputs:** forecasted_ros_summary.csv containing predicted ROS (units/week) for each SKU-store combination for the optimization_horizon_weeks.
3. **Core Logic Summary:**
 - Applies time-series forecasting models (e.g., ARIMA, Prophet, or machine learning models) to historical sales data.
 - Incorporates features like seasonality, trend, price changes, and promotional events into the forecast.
 - Calculates ROS by averaging forecasted units sold over a defined period (e.g., 4-week moving average of forecasted sales).
 - Adjusts ROS based on category_growth_factor for strategic categories.

6.3. Assortment Optimization Engine

1. **Name & Purpose:** Determines the optimal product assortment, including Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS), for each category and location.
2. **Inputs & Outputs:**
 - **Inputs:** `forecasted_ros_summary.csv`, `Stock_On_Hand_Units`, `Retail_Price_Per_Unit`, `Cost_Per_Unit`, `store_capacity_constraints`, and `args.properties` settings.
 - **Outputs:** Preliminary `od_recommendations`, `ow_recommendations`, and `iss_recommendations`.
3. **Core Logic Summary:**
 - **OD Calculation:** Evaluates SKUs within a category based on forecasted ROS, profitability, and inventory levels. Uses a ranking system to select the ideal number of unique items (OD) to stock, prioritizing high-ROS items above `min_ROS_threshold`.
 - **OW Calculation:** Analyzes performance across different categories/styles (e.g., “Straight Leg Jeans” vs. “Skinny Jeans”) and store space constraints. Recommends the ideal number of different categories (OW) to offer, balancing breadth with depth.
 - **ISS Determination:** For each selected SKU, analyzes ROS by individual size to identify the most demanded sizes. Recommends the Ideal Size Set (ISS) that maximizes sales and minimizes leftover inventory for specific sizes, considering `size_set_variance_tolerance`. This typically involves a combination of core sizes and a targeted allocation of less common sizes based on their ROS.

6.4. Recommendation Generation & Output

1. **Name & Purpose:** Translates the optimized assortment plans into clear, actionable recommendations and exports them in consumable formats.
2. **Inputs & Outputs:**
 - **Inputs:** Preliminary OD, OW, ISS recommendations from Assortment Optimization Engine.
 - **Outputs:** Final `od_recommendations.csv`, `ow_recommendations.csv`, `iss_recommendations.csv`, `inventory_rebalancing_suggestions.csv`, and `performance_metrics.json`.
3. **Core Logic Summary:**
 - Consolidates recommendations from the optimization engine into a unified format.
 - Generates specific unit counts or flag indicators for each SKU-store combination.
 - Calculates projected KPIs (e.g., stockout risk, inventory turn) based on the new recommendations.
 - Exports data to specified CSV/JSON files, ensuring data integrity and readability.

7. Worked Example

Scenario: Optimizing the “Men’s Casual Shirts” category for Store 101.

1. Initial Data:

- Store 101 capacity for “Men’s Casual Shirts”: 50 unique SKUs.
- Current Stock: SKU-A (Blue Polo, Size M): ROS 8 units/week, Stock 10. SKU-B (Red Polo, Size L): ROS 3 units/week, Stock 5. SKU-C (Green Polo, Size S): ROS 0.2 units/week, Stock 20. SKU-D (Striped Button-Down, Size M): ROS 12 units/week, Stock 15. SKU-E (Plain Button-Down, Size L): ROS 7 units/week, Stock 8.
- `min_ROS_threshold` = 0.5 units/week.

2. Demand Forecasting & ROS Calculation (Submodule 6.2):

- Forecasted ROS for next 8 weeks: SKU-A: 8 units/week, SKU-B: 3 units/week, SKU-C: 0.2 units/week, SKU-D: 12 units/week, SKU-E: 7 units/week.

3. Assortment Optimization (Submodule 6.3):

- **OD (Optimum Depth):**
 - SKU-C (Green Polo) has a ROS of 0.2 units/week, which is below the `min_ROS_threshold` of 0.5. It’s flagged for potential removal or markdown.
 - Remaining SKUs (A, B, D, E) are above threshold. Assuming Store 101 has space for 40 unique shirt SKUs and these 4 are top performers, they are recommended for retention.
 - If there were 60 current SKUs, the algorithm would rank them by ROS and suggest keeping the top 40.
- **OW (Optimum Width):**
 - Within “Men’s Casual Shirts”, the algorithm might identify “Polo Shirts” and “Button-Down Shirts” as distinct styles (categories).
 - If “Polo Shirts” have a higher aggregate ROS across all SKUs than “Button-Down Shirts”, the algorithm might recommend increasing the OW for “Polo Shirts” by allocating more distinct polo designs, and reducing OW for “Button-Down Shirts” if their collective ROS is lower.
 - In this example, both are performing well, so the algorithm might recommend maintaining diverse styles.
- **ISS (Ideal Size Set) for SKU-D (Striped Button-Down):**
 - Past sales for SKU-D (Striped Button-Down) show: Size S: 10%, M: 40%, L: 35%, XL: 15%.
 - Current stock: S: 2, M: 10, L: 2, XL: 1.
 - Based on `size_set_variance_tolerance`, the algorithm might recommend restocking for M and L due to high ROS and low stock relative to demand. It might suggest only stocking S and XL at a minimal level, or even omitting them if their ROS falls significantly below expectations and current inventory is high.
 - **Recommendation:** For a replenishment of 10 units of SKU-D, the ISS could be: S: 1, M: 4, L: 4, XL: 1, reflecting the sales distribution while ensuring key sizes are covered.

4. Recommendation Generation (Submodule 6.4):

- **OD Recommendation:** Remove SKU-C (Green Polo) from stock due to low ROS. Maintain SKUs A, B, D, E.
- **OW Recommendation:** Continue stocking both “Polo Shirts” and “Button-Down Shirts” categories at optimal levels.

- **ISS Recommendation:** For SKU-D (Striped Button-Down), order a new shipment with a size breakdown of S:1, M:4, L:4, XL:1.
- **Inventory Rebalancing:** Consider transferring excess SKU-C stock to a different channel (e.g., clearance) or returning to vendor.

8. Debug Guide Highlights

- 1. Low ROS across the board:**
 - **Check:** Data Ingestion (Are sales data flowing correctly and completely? Are there data gaps?).
 - **Check:** Demand Forecasting (Are forecasting models re-trained regularly? Are external factors like promotions correctly weighted?).
 - **Check:** min_ROS_threshold setting (Is it too high for certain categories?).
- 2. Recommendations conflict with store capacity:**
 - **Check:** Store Master Data (Are store capacities correctly defined and updated?).
 - **Check:** Assortment Optimization Engine (Are capacity constraints being correctly applied in the OD/OW logic?).
- 3. Frequent stockouts despite recommendations:**
 - **Check:** Inventory Management System integration (Are actual stock levels accurately reported to the algorithm?).
 - **Check:** ROS accuracy (Is the forecasted ROS consistently underpredicting actual sales?).
 - **Check:** target_fill_rate setting (Is it too low, leading to conservative recommendations?).
- 4. ISS recommendations seem off:**
 - **Check:** Sales data granularity (Are sales recorded at the individual size level?).
 - **Check:** size_set_variance_tolerance (Is it too broad, allowing for less optimal sizes to be included?).
 - **Check:** Data for specific size Attribute Groups (Are there data quality issues for particular sizes?).

9. Common Pitfalls & Remedies

- 1. Pitfall: Outdated or inaccurate input data.** The algorithm is highly dependent on clean and current data. Stale POS data or incorrect inventory counts will lead to suboptimal recommendations.
 - **Remedy:** Implement robust data validation checks at the Data Ingestion stage. Schedule daily/real-time data refreshes. Conduct regular audits of input data sources.
- 2. Pitfall: Lack of seasonal or promotional context.** Static ROS calculations can miss significant spikes or dips in demand due to seasonality or marketing campaigns.
 - **Remedy:** Ensure the Demand Forecasting module incorporates seasonal indices, promotional flags, and historical promotional uplift/decline factors. Regularly update promotional calendars as an input.

3. **Pitfall: Over-optimization leading to overly niche assortments.** Focusing too narrowly on high-ROS items might lead to a lack of variety, potentially alienating some customer segments.
 - **Remedy:** Introduce business rules or guardrails in the Assortment Optimization Engine to ensure a minimum OW (variety) for categories, even if some styles have lower ROS. Consider “strategic SKUs” that must always be stocked regardless of ROS.
4. **Pitfall: Ignoring lead times for replenishment.** Recommendations might suggest immediate stock-outs or overstocks without accounting for the time it takes to procure and receive new inventory.
 - **Remedy:** Incorporate lead times (from vendors or distribution centers) into the ROS calculation and inventory planning within the Assortment Optimization Engine, ensuring recommendations are practical for reordering.
5. **Pitfall: Lack of feedback loop.** Without monitoring the impact of recommendations, the algorithm cannot learn or be refined.
 - **Remedy:** Establish clear KPIs and regular reporting (e.g., monthly review of actual ROS vs. forecasted ROS, inventory turns vs. target). Implement a feedback mechanism where merchandising teams can flag suboptimal recommendations for review and algorithm tuning.

10. FAQ

Q1: How frequently are the recommendations updated? A1: Recommendations are typically generated on a weekly basis, coinciding with inventory replenishment cycles. Critical adjustments can trigger ad-hoc updates.

Q2: Can the algorithm account for new product introductions or discontinuations? A2: Yes. New products will initially rely on proxy data (e.g., sales of similar products) and then their own performance as sales accumulate. Discontinued products are automatically flagged and removed from active recommendations.

Q3: What if a store has unique local demand not reflected in historical data? A3: While the algorithm uses historical sales, local demand fluctuations can be partially addressed if specific Store_ID attributes (e.g., “tourist store,” “urban vs. suburban”) are incorporated into the forecasting model. Manual overrides or adjustments by local merchandising teams are also possible and should be captured as feedback.

Q4: How does the algorithm handle promotions or markdown events? A4: The Demand Forecasting module incorporates promotional lift/decline factors based on past event performance. This helps predict the surge or dip in ROS during promotional periods, informing appropriate OD and ISS recommendations.

Q5: Is it possible to run a “what-if” scenario with different settings? A5: In Version 1.0, direct “what-if” scenario testing is limited. However, you can adjust parameters in `args.properties` and re-run the pipeline to observe the impact on recommendations. Future versions may include a dedicated simulation module.

Q6: What is an Attribute Group (AG)? A6: An Attribute Group is a collection of related product characteristics used for analysis. For example, ‘Color’ is an AG that might include values like ‘Red’, ‘Blue’, ‘Green’. ‘Size’ is another AG, with values like ‘S’, ‘M’, ‘L’, ‘XL’, or specific numerical sizes. The algorithm uses AGs to understand product variations and their impact on demand.

11. Quick Reference Cheat-Sheet

Term	Definition	Key Role in Algorithm
OD	Optimum Depth: The ideal number of unique items (SKUs) to stock in a specific category or display.	Determines the breadth of product choices within a category.
OW	Optimum Width: The ideal number of different categories or styles to offer (e.g., how many different types of shirts).	Defines the overall assortment mix and store capacity utilization.
ISS	Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts for a given SKU.	Ensures optimal size distribution for each product, reducing lost sales from missing sizes.
ROS	Rate of Sale: The speed at which products are sold over a period (e.g., units/week).	Primary driver for forecasting demand, determining OD, OW, and ISS.
AG	Attribute Group: A set of product attributes grouped for analysis (e.g., color, size, brand).	Used to categorize and analyze product variations for optimization.
args.properties	Configuration file for algorithm parameters.	Fine-tunes the algorithm’s behavior and sensitivity.
common-data.json	Standardized format for input data fields.	Ensures consistent data interpretation across all submodules.
Optimization Horizon	The future period (in weeks) for which forecasts and	Defines the look-ahead window for planning.

Term	Definition	Key Role in Algorithm
	recommendations are generated.	

Chapter 1: DDDataPrepModule

Last Updated: 2025-05-23

1. Business Objective

The Dynamic Discounting Data Preparation Module gathers, cleans, and structures all essential retail data to accurately determine product eligibility and calculate optimal price reductions.

Why This Matters: This foundational step ensures that subsequent discount calculations are based on accurate, relevant, and comprehensive information. This leads to more profitable pricing strategies, reduced excess inventory, and improved sales performance by eliminating the need for complex manual data preparation.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records:** We need details on past product sales, including units sold, sale dates, and selling locations. This helps us understand product performance and identify trends over specific periods.
- **Sales Consistency Data:** This includes a metric that shows how much a particular product contributed to overall sales in a store on a given day. This helps us filter out unreliable sales data caused by system errors or incomplete records.
- **Sell-Through Performance Benchmarks:** Predefined targets for how quickly products should sell over a specific timeframe. This helps measure product performance against expected rates, indicating if a product is moving too slowly.

Product Data

- **Product Style Classifications:** Information linking specific product styles to their lifecycle stage (e.g., new, seasonal, clearance), season type (e.g., Spring/Summer), and broader style groups. This enables targeted discounting based on product type and age.
- **Cost of Goods Sold (COGS) by Style:** The direct costs attributable to producing or purchasing each product style. This is crucial for calculating potential profit margins when applying discounts.

- **Category Size Sequences:** The typical size progression (e.g., S, M, L or 30, 32, 34) for each product category. This helps in understanding the Ideal Size Set (ISS) and identifying inventory imbalances within a size run.
- **Product Attributes:** Details like brand, category, and unique product identifiers (SKUs, Styles). This allows for grouping and analyzing products effectively.

Operational Data

- **Discount-Enabled Store List:** A list of specific stores designated to participate in dynamic discounting initiatives and their respective discount groups. This ensures discounts are applied only where intended.
- **Current Store Inventory Levels:** Real-time stock counts for every unique item (SKU) at each physical store. This is vital for understanding Days of Hand (DOH) and identifying overstocked items.
- **In-Transit Inventory & Open Orders:** Records of products currently being shipped to stores or awaiting delivery from suppliers. This provides a complete picture of available stock, including future arrivals, preventing missed sales or unnecessary discounts.
- **Discount Factor Settings:** Predefined rules for how discounts can incrementally increase, including starting and maximum discount percentages. These guide the system on permissible discount ranges.
- **Discount Rule Matrix:** A comprehensive set of rules defining discount actions based on key metrics like Rate of Sale (ROS), Days of Hand (DOH) or Ageing, and overall product health. This translates business logic into precise discount decisions.
- **Style-Specific Discount Limits:** Overrides or specific maximum discount percentages set for individual product styles, allowing for finer control beyond general rules, protecting high-value or core products.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Date	Today’s Date	The specific date for which dynamic discount recommendations are generated.	Adjust this if you need to run calculations for a past date, for example, to analyze historical performance or re-evaluate decisions.
Discount Cycle Duration	90 days	The number of days of recent sales data the system analyzes to understand product performance.	Increase this for products with slower sales cycles (e.g., furniture) or decrease for fast-moving goods (e.g., fashion accessories) to capture more relevant trends.
	0.8 (80%)	The minimum percentage of a	Adjust this if you find too much sales

Setting Name	Default Value	Description	When to Change
Minimum Sales Consistency Threshold		product's expected sales contribution that must be met for its historical sales data to be considered "clean" and reliable for analysis.	data is being excluded (lower the threshold) or if sales data appears unreliable (raise the threshold) to ensure only valid sales patterns are used.
Discount Store Group Configuration	(Varies)	Defines which stores are part of dynamic discounting initiatives and how they are grouped (e.g., 'Flagship', 'Outlet').	Modify this when adding new stores to discounting programs, creating new store clusters, or altering a store's participation status.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Retail Data Feeds] --> B{Dynamic Discounting Data Preparation}

B --> C[Structured & Filtered Data for Discounting]

1. Start with → **Raw Retail Data Feeds** including sales transactions, inventory counts, and product details from various retail systems.
2. Then → **Clean and Validate** the sales data, filtering out incomplete or unreliable records to ensure accuracy in Rate of Sale (ROS) calculations.
3. Next → **Integrate and Classify** all data, associating product styles with their lifecycle stages, assigning stores to discount groups, and consolidating all inventory sources (on-hand, in-transit, open orders).
4. Finally → **Output** a comprehensive, structured dataset that is precisely formatted and ready for dynamic discounting calculations.

5. Step-by-Step Actions

1. **Initialize** the data environment
 - What we do: *We prepare the system's data structures and load essential product information caches, such as product groupings and attribute groups.*

- Why we do it: *This ensures all foundational data is readily available for quick access and efficient processing, preventing delays in subsequent calculations.*
- 2. **Calculate** clean historical sales
 - What we do: *We retrieve recent sales transactions for a specified period (e.g., 90 days) and filter out any sales data points that don't meet a minimum threshold for reliability or completeness, like those from system errors.*
 - Why we do it: *This gives us a precise Rate of Sale (ROS) for each product, preventing faulty discount decisions based on unreliable historical performance data, which could lead to missed revenue.*
- 3. **Establish** store groups for discounting
 - What we do: *We identify which stores are active participants in dynamic discounting and organize them into predefined groups (e.g., 'Metro Stores', 'Outlet Stores').*
 - Why we do it: *This allows us to apply specific discount rules and strategies based on store characteristics or business objectives, ensuring targeted and effective price reductions.*
- 4. **Define** product lifecycle and apply discount factors
 - What we do: *We categorize products by their season and lifecycle (e.g., new arrivals, mature, clearance) and then load specific discount factor rules, including starting and maximum discount percentages, for each product and store group.*
 - Why we do it: *This ensures that discounts align with product age and strategic merchandising goals, preventing excessive markdowns on new items and ensuring adequate price reductions on older inventory to improve sell-through.*
- 5. **Assemble** discount rule matrix and product costs
 - What we do: *We load a comprehensive set of rules that dictate discount actions based on a product's performance metrics (like ROS, Days of Hand (DOH), and overall health) and also load each product's Cost of Goods Sold (COGS).*
 - Why we do it: *This provides the precise logic for discount decision-making and ensures that profitability is always considered during discount calculations, protecting profit margins.*
- 6. **Compile** complete inventory picture
 - What we do: *We gather all current on-hand inventory across physical stores, then add inventory that is currently in transit to stores (from warehouses or other stores) or on open purchase orders from suppliers.*
 - Why we do it: *This provides a holistic view of available stock, preventing stockouts or over-discounting based on incomplete inventory data, leading to more accurate Days of Supply (DOH) calculations and better inventory investment decisions.*

6. Sample Scenario

Let's walk through a real example: A "Blue Classic Denim Jean" in a flagship store.

Starting Point

- **Sales History:** For the last 90 days, “Blue Classic Denim Jeans” in a Size 32 have sold an average of 15 units per week at a specific flagship store, but some sales records had incomplete data due to a system glitch.
- **Current Stock:** The flagship store has 100 units of “Blue Classic Denim Jeans” (all sizes combined), with 20 units specifically of Size 32.
- **In-Transit:** 50 additional units of these jeans are expected to arrive next week from the distribution center, already purchased by the store.
- **Product Classification:** These jeans are categorized as “Core Denim” and are in the “Mature” lifecycle stage for the “Fall/Winter” season.
- **Discount Rules:** General rules indicate that products with high Days of Hand (DOH) and low Rate of Sale (ROS) are candidates for discounts, with a starting discount of 10% and a maximum of 40%. A specific rule for “Core Denim” states that their sales records must have a “Pivotal Sales Contribution” (our sales consistency metric) of at least 70% to be used for ROS calculation.

What Happens

1. **Sales Data Cleaning:** The system first retrieves all sales data for the “Blue Classic Denim Jean” for the last 90 days. It then checks each sale record’s “Pivotal Sales Contribution.” Any sales records below the 70% threshold (e.g., a sale that was part of a major store-wide system error) are excluded. The remaining clean sales data is used to calculate an accurate ROS, now showing 12 units per week.
2. **Inventory Consolidation:** The system adds the 100 units currently in the store to the 50 units in transit, arriving at a total available inventory of 150 units for the jeans. This provides a complete picture of product availability.
3. **Product Classification & Rules Applied:** The system accurately identifies the “Blue Classic Denim Jean” as “Core Denim” and “Mature” season. It loads the general discount rules and checks for any style-specific overrides (none in this case).
4. **Store Group Assignment:** The flagship store is confirmed as part of the “Dynamic Discounting Tier 1” store group, enabling it for immediate discount calculations based on its strategic role.

Final Outcome

The `DDDataPrepModule` outputs a comprehensive data package for “Blue Classic Denim Jeans” for the flagship store. This package includes its accurate Rate of Sale (12 units/week), total available inventory (150 units), its product classification, and all applicable discount factors and rules. This robust data is now ready for the next module to calculate the precise dynamic discount percentage.

Key Takeaway: This module transforms raw, dispersed retail data into a clear, unified, and accurate foundation, ensuring that all dynamic discount decisions are based on the complete and reliable truth about a product’s performance and availability.

7. Results

This step produces:

Main Outputs

- **Cleaned Sales Data:** Accurate historical sales figures, filtered for reliability, used to calculate current Rate of Sale (ROS). This tells us how quickly products are truly selling, avoiding misinterpretations from faulty data.
- **Consolidated Inventory Snapshot:** A complete picture of on-hand stock, including units in transit and those on open purchase orders. This informs Days of Hand (DOH) and prevents over-discounting or missed sales due to incomplete stock visibility.
- **Product & Store Group Mappings:** Clear associations of each product style with its lifecycle, season, brand, and relevant Cost of Goods Sold (COGS), along with defined store groups. This ensures discounts are applied intelligently based on product type and location.
- **Applicable Discount Rules & Factors:** All relevant discount rules, start/max percentages, and increment factors, tailored to specific product types or store groups. This defines the boundaries and logic for future discount calculations, adhering to business policies.

Business Reports

- **Data Quality Dashboard:** A summary report showing the percentage of sales data filtered out and key data discrepancies. This helps merchandising teams monitor data health and identify areas for improvement in data capture.
- **Inventory Accuracy Reconciliation:** A report detailing the reconciliation of physical inventory with in-transit and open order quantities. This provides supply chain and planning teams with confidence in total available stock.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed (e.g., sales data not updated)	The system will use the last available data, potentially leading to outdated Rate of Sale (ROS) and Days of Hand (DOH) calculations.	Investigate the data feed source immediately. Manually verify critical data points if the module must run, or reschedule the run until data is complete.
No historical sales data for a new product style	The system will treat its Rate of Sale (ROS) as zero or very low, potentially triggering aggressive discounts too early for a new item.	Manually set a starting discount or override rules for new products, or use benchmark data from similar successful new product launches.

Situation	What Happens	What You Should Do
A store is not listed in the discount-enabled store group	Products in that store will not be considered for dynamic discounting, even if they meet other criteria, leading to missed opportunities.	Verify the store's status in the "Discount-Enabled Store List" and add it if it should participate in dynamic pricing.
Product category's size sequence is missing	The system may not be able to accurately assess the Ideal Size Set (ISS) or identify imbalanced size inventories, impacting the effectiveness of discounts.	Update the "Category Size Sequences" to ensure all relevant categories have their size progressions defined for better inventory management.
A product has no Cost of Goods Sold (COGS) defined	The system cannot accurately assess the profitability impact of discounts for that product.	Ensure all product styles have an associated COGS to prevent discounts that could lead to unprofitable sales and eroding margins.

9. Next Steps

Before This Step

- **Daily Data Ingestion:** The system needs to have successfully ingested and loaded all raw sales transactions, current inventory levels, product master data (SKUs, styles, attributes), and operational parameters. This provides the fresh, raw input that this module refines.
- **System Parameter Configuration:** Key business settings such as the analysis date, discount cycle duration, and sales consistency thresholds must be properly configured in the system. This ensures the module processes data according to current business strategy and objectives.

After This Step

- **Dynamic Discount Calculation Module:** The outputted, structured, and cleaned data is directly consumed by the next module to calculate the optimal discount percentage for each product in each relevant store. This module relies entirely on the accuracy and completeness of the data prepared here.
- **Inventory Optimization Module:** The consolidated inventory data (including in-transit and open orders) can also feed into broader inventory planning systems. This helps ensure that overall inventory levels are managed effectively beyond just discounting, supporting purchasing and allocation decisions.

10. Common Questions

Q: Why are some of my store's sales not being used for discount calculations? A: Sales records might be filtered out if they don't meet the "Minimum Sales Consistency Threshold," meaning the data quality for those specific sales was deemed unreliable. Also, ensure your store is correctly included in a "Discount-Enabled Store Group" to be considered for discounting.

Q: Can we run dynamic discounting for a specific product style only? A: Yes, while the data preparation module processes all relevant data, the subsequent calculation modules can be configured to target specific product styles or categories for discounting. This module simply ensures that all necessary data for those styles is available for analysis.

Q: How often does this data preparation process run? A: Typically, this process runs daily, or as frequently as new sales and inventory data becomes available (e.g., hourly during peak periods), to ensure that dynamic discount decisions are always based on the most current sales and stock information.

Q: What if I notice that a new product isn't showing up for dynamic discounting? A: First, ensure that its product master data (style, brand, category) is fully loaded. Also, check if it's classified with an appropriate "Lifecycle" stage (e.g., "New Arrival") and if it has any sales history, even if limited. New products often need initial sales to trigger the system.

Q: How does this module prevent me from discounting products that are already selling well? A: By accurately calculating the Rate of Sale (ROS) and Days of Hand (DOH), the module provides critical performance indicators. If a product's ROS is high and its DOH is low, the system is less likely to recommend a discount based on the established "Discount Rule Matrix" which prioritizes slow-moving or excess inventory.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To gather, clean, and structure all necessary sales, product, and inventory data, creating a reliable foundation for dynamic discount calculations.
- **Main Input:** Raw sales transactions, current inventory levels, product classifications, and business rules.
- **Main Output:** A consolidated, filtered, and categorized dataset that precisely defines what products are available, how they're performing, and what discount rules apply.
- **Warning Signs:** Inaccurate discount recommendations, unexpected product exclusions from discounting, or missing inventory data. These often point to issues in the data preparation phase.
- **Key Settings:** "Analysis Date," "Discount Cycle Duration," and "Minimum Sales Consistency Threshold."

Need Help? Contact your Merchandise Planning or IT Support team for questions about this process.

Chapter 2:

DiscountComputationModule

Last Updated: 2025-05-23

1. Business Objective

The `DiscountComputationModule` determines and suggests the best discount percentages for products at the style level, guiding businesses on optimal pricing strategies to maximize sales and inventory efficiency.

Why This Matters: This step ensures that every price reduction is strategic and data-driven, minimizing losses from overstocked items, boosting sales of slow-moving inventory, and protecting profit margins, ultimately improving overall financial performance and inventory health.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Historical Sales Records:** Accurate past sales figures, free from errors, are crucial to calculate reliable product sales velocity (ROS) and understand performance trends.
- **Sales Consistency Data:** A measure that confirms the reliability of sales data, ensuring that only trustworthy records are used for calculating how fast products are selling.
- **Sell-Through Performance Benchmarks:** Predefined targets for how quickly products should sell, allowing us to assess if a product is moving as expected or if it needs a discount.

Product Data

- **Product Style Classifications:** Information detailing a product's lifecycle stage (e.g., new, seasonal, clearance), season type, and broader style groups, enabling targeted discounting based on product type and age.
- **Cost of Goods Sold (COGS) by Style:** The direct cost of acquiring or producing each product style, essential for calculating potential profit margins after applying discounts.

- **Product Attributes (e.g., brand, category):** Details that allow for grouping and analyzing products effectively, ensuring discounts align with brand and category strategies.
- **Style Maximum Retail Price (MRP):** The original selling price of a product, used to calculate the value of projected sales and discount amounts.

Operational Data

- **Discount-Enabled Store List:** A list of stores authorized to participate in dynamic discounting, ensuring discounts are applied only where intended.
- **Current Store Inventory Levels:** Real-time stock counts for every unique item at each store, vital for calculating Days of Hand (DOH) and identifying overstocked items.
- **In-Transit Inventory & Open Orders:** Records of products en route to stores or awaiting delivery from suppliers, providing a complete picture of total available stock to prevent unnecessary discounts.
- **Discount Rule Matrix:** A comprehensive set of business rules that define discount actions based on key metrics like ROS, DOH, and overall product health, translating strategic goals into precise discount decisions.
- **Style-Specific Discount Limits:** Custom maximum discount percentages for individual product styles, allowing finer control beyond general rules to protect high-value or core products.
- **Previous Discounting Decisions:** Records of past discounts applied to specific styles, used to ensure continuity or adjust strategies based on prior performance.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Discount Projection Period	30 days	The number of days into the future for which the system projects sales and discount impacts.	Increase this for products with slower sales cycles or decrease for fast-moving goods to align projections with inventory turnover.
Minimum Live Days for Analysis	60 days	The minimum number of days a product style must have been available for sale in stores to be considered for discount analysis.	Adjust if you need to include newer products in discounting (lower threshold) or ensure more mature sales data (raise threshold).
Sales Data Reliability Threshold	0.8 (80%)	The minimum percentage of expected sales contribution a product's historical sales	Modify if sales data appears inconsistent (raise threshold) or if too much data is being excluded (lower threshold).

Setting Name	Default Value	Description	When to Change
Overall Discount Target	0.05 (5%)	data must meet to be considered reliable for ROS calculations. The target percentage of total sales value that the system aims to achieve through discounts across all relevant products.	Change this based on overall business financial goals to control the total amount of discount spend.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Structured & Filtered Data] --> B{Discount Calculation & Recommendation}

B --> C[Suggested Discount Actions & Projections]

1. Start with → **Structured & Filtered Data** from the DDDataPrepModule, including clean sales history, consolidated inventory, and product classifications.
2. Then → **Assess Product Performance** by calculating metrics like Rate of Sale (ROS), Days of Hand (DOH), product age, and overall health (e.g., size set continuity).
3. Next → **Apply Discount Rules** from the defined Discount Rule Matrix, combining performance metrics with profitability targets to determine specific discount actions and percentages for each style.
4. Finally → **Output** detailed discount recommendations, including projected sales and financial impacts, ready for review and implementation.

5. Step-by-Step Actions

1. **Retrieve** existing discounts
 - What we do: *We gather information on any discounts currently applied to products within relevant store groups.*
 - Why we do it: *This provides a baseline for determining if discounts need to be increased, decreased, or continued.*
2. **Aggregate** product performance by store group
 - What we do: *We combine sales and inventory data for each product style across its assigned store group, calculating average live days and store exposure.*

- Why we do it: *This provides a holistic view of a product's performance within a specific market segment, rather than just individual store performance.*
- 3. **Analyze** sales responsiveness to discounts
 - What we do: *We analyze historical sales data to understand how changes in discount percentages have impacted sales velocity (ROS) for product categories, identifying "slopes" or responsiveness.*
 - Why we do it: *This helps predict how a recommended discount will likely affect future sales, making projections more accurate.*
- 4. **Filter** products based on market presence
 - What we do: *We identify product styles that haven't met minimum live days or store exposure benchmarks, flagging them for special consideration.*
 - Why we do it: *This ensures that discounts are not prematurely applied to new products or those with limited market presence, protecting their initial full-price sales period.*
- 5. **Calculate** product sales velocity
 - What we do: *We determine both the "true" Rate of Sale (ROS) for products, using only reliable sales data, and a "raw" ROS from all available sales to understand typical movement.*
 - Why we do it: *This provides an accurate measure of how quickly products are selling, forming a critical input for discount decisions.*
- 6. **Integrate** warehouse inventory
 - What we do: *We incorporate inventory quantities from warehouses and in-transit shipments, distributing these units proportionally to associated stores.*
 - Why we do it: *This gives a complete picture of total available stock, ensuring Days of Hand (DOH) calculations are accurate and preventing over-discounting due to incomplete stock visibility.*
- 7. **Assess** product health
 - What we do: *We evaluate the health of each product style, especially checking for issues like fragmented size sets that can hinder sales.*
 - Why we do it: *This helps identify inventory imbalances that may require a discount, even if other metrics seem fine, to improve sell-through of complete size runs.*
- 8. **Determine** product age
 - What we do: *We calculate the age of each product style based on its first arrival date in the supply chain or its overall live days in stores.*
 - Why we do it: *Aging inventory often requires more aggressive discounting to clear, protecting against holding costs and obsolescence.*
- 9. **Categorize** products by performance
 - What we do: *We classify each product style into categories like High, Medium, or Low for ROS, Ageing (DOH), and Sell-Through based on predefined benchmarks.*
 - Why we do it: *This simplifies the application of discount rules, allowing the system to easily match product profiles to appropriate discount actions.*
- 10. **Calculate** new discount recommendations
 - What we do: *Based on the product's classifications (ROS, DOH, Health, Sell-Through), the system applies the Discount Rule Matrix to determine the suggested discount action (increase, continue, decrease) and percentage.*
 - Why we do it: *This ensures that discount decisions are consistent, aligned with business strategy, and tailored to each product's specific performance needs.*

11. **Project** future sales and impacts

- What we do: *We forecast how many units are expected to sell and the projected revenue and discount value if the suggested discount is applied.*
- Why we do it: *This provides a clear picture of the potential financial impact of the discount, allowing for profitability assessment and overall target discount reconciliation.*

12. **Generate** detailed reports

- What we do: *We compile all discount recommendations, performance metrics, and projected impacts into structured output reports for review.*
- Why we do it: *This provides merchandising and planning teams with the actionable insights needed to approve and implement dynamic discounting strategies.*

6. Sample Scenario

Let's walk through a real example: A "Red Summer Dress" in a flagship store.

Starting Point

- **Product:** "Red Summer Dress," Style #123.
- **Current Stock (Flagship Store):** 50 units (all sizes combined).
- **Historical Rate of Sale (ROS):** Averaged 5 units per week over the last 90 days.
- **Product Age:** 120 days since the first shipment arrived at the distribution center.
- **Health Status:** Assessed as "Unhealthy" because several key sizes (like M and L) are out of stock, leaving fragmented inventory.
- **Current Discount:** 10%.
- **Discount Rule Matrix:** Rules dictate that "Unhealthy" products older than 90 days with "Low" ROS should receive a discount increase of 10-15%. The maximum allowed discount for this product category is 30%.
- **Sales Responsiveness (Slope):** Historical data shows that for this product category, a 1% increase in discount typically leads to a 0.5 unit/week increase in ROS.

What Happens

1. **Product Health Check:** The module analyzes the available sizes for Style #123 and identifies that critical sizes are missing. It flags the "Red Summer Dress" as having an "Unhealthy" size set.
2. **Performance Classification:** It calculates the product's ROS (5 units/week) and classifies it as "Low" compared to similar products. Its age (120 days) combined with its Days of Hand (DOH) classifies it as "Lagging" in terms of aging.
3. **Rule Application:** The system references the Discount Rule Matrix. It finds the rule for "Unhealthy, Lagging, Low ROS" products. This rule recommends an "Increase" action with a specific increment.
4. **Discount Calculation:** Applying the rule, the system suggests increasing the discount by 15%, bringing the new suggested discount to 10% (current) + 15% (increment) = 25%. This 25% is then checked against the maximum allowed 30% for the category and found to be acceptable.

5. **Projected Sales Calculation:** Using the calculated sales responsiveness (slope of 0.5), the module projects that a 15% increase in discount (from 10% to 25%) will lead to an additional 7.5 units sold per week ($15\% \times 0.5$ units/week per % discount). The new projected ROS becomes $5 + 7.5 = 12.5$ units per week.
6. **Profitability Check:** The module calculates the projected selling price and margin at a 25% discount, ensuring the sale remains profitable relative to the Cost of Goods Sold (COGS).

Final Outcome

The DiscountComputationModule recommends a 25% discount for the “Red Summer Dress” in the flagship store. This action is projected to increase its sales velocity from 5 units/week to 12.5 units/week, helping to clear the fragmented inventory faster and improve stock health while maintaining profitability.

Key Takeaway: This module leverages comprehensive data to analyze product health, age, and sales performance, translating complex business rules into precise and profitable discount recommendations that accelerate inventory movement.

7. Results

This step produces:

Main Outputs

- **Suggested Discount Actions and Percentages:** Clear recommendations (Increase, Continue, Decrease) and specific discount percentages for each product style within relevant store groups. This directly tells merchandising teams what pricing adjustments to make.
- **Projected Sales and Discount Impacts:** Forecasted units sold and revenue impacts under the suggested discount, including projected margins. This helps sales and finance teams understand the expected financial outcome.
- **Detailed Product Performance Classifications:** The classified ROS level, Ageing level (DOH), Health Status, and Sell-Through level for each product style. This provides the rationale behind each discount recommendation, aiding in strategic planning.
- **Recommended Selling Prices and Margins:** The new suggested selling price and the estimated profit margin per unit at that price. This ensures profitability is considered and helps merchandise planners evaluate financial viability.

Business Reports

- **Discount Recommendation Report:** A comprehensive list of all suggested discounts, performance metrics, and remarks for each product style and store group. This is used by merchandising and pricing teams for review, approval, and implementation.
- **Projected Sales & Profitability Analysis:** A summary report detailing the anticipated sales uplift, revenue, and gross margin impacts of the proposed discounts across all products. This is crucial for financial planning and executive decision-making.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will use the last available data, potentially leading to outdated or inaccurate discount calculations.	Investigate the data feed source immediately. Manually verify critical data points if the module must run, or reschedule the run until data is complete and accurate.
No historical sales data for a new product style	The system might treat its Rate of Sale (ROS) as zero or very low, potentially triggering aggressive discounts too early for a new item.	Manually set a starting discount or override rules for new products, or use benchmark data from similar successful new product launches to provide initial guidance.
Product health assessment shows all products as “Unhealthy”	The system may recommend aggressive discounts across a wide range of products, potentially eroding margins if the underlying issue isn’t stock-related.	Review the criteria for “Product Health” (e.g., Ideal Size Set completeness) and ensure they are appropriately set. Investigate inventory accuracy.
Discount Rule Matrix missing or incomplete	The system will not have the necessary logic to classify products or suggest discounts, potentially leading to no recommendations or incorrect ones.	Immediately update and load the Discount Rule Matrix with comprehensive rules that cover all relevant product performance scenarios.
Discrepancy between projected and actual sales post-discount	The sales responsiveness (slope) used for projections might be inaccurate for a given product category or market.	Re-evaluate the historical data used to calculate sales responsiveness (slopes) and update the model’s parameters to better reflect current market conditions.

9. Next Steps

Before This Step

- **Dynamic Discounting Data Preparation:** The previous `DDDataPrepModule` must have successfully gathered, cleaned, and structured all raw sales, inventory, product, and operational data. This module relies entirely on the accuracy and completeness of that prepared data.
- **System Parameter Configuration:** All key business settings for the dynamic discounting program, including discount cycle duration, sales consistency

thresholds, and target discount percentages, must be properly configured. This ensures the module processes data according to current business strategy.

After This Step

- **Discount Action & Application Module:** The suggested discount actions and percentages generated by this module are sent to a subsequent module responsible for actually implementing these price changes in the point-of-sale or e-commerce systems.
- **Inventory Optimization Module:** The detailed product performance data (ROS, DOH, health) and projected sales impacts can feed into broader inventory planning and buying systems. This helps refine future purchasing decisions and allocations to prevent recurrence of overstock situations.

10. Common Questions

Q: Why did a product that seemed to be selling well receive a discount recommendation? A: While a product might have some sales, this module considers multiple factors like Days of Hand (DOH), overall product health (e.g., fragmented size sets), and age. Even with decent sales, if a product has high DOH or an unhealthy stock profile, a discount might be recommended to proactively clear inventory before it becomes obsolete.

Q: Can we override a specific discount recommendation made by the system? A: Yes, the system supports manual overrides for specific product styles or store groups. This allows merchandising and planning teams to exercise judgment and apply strategic exceptions as needed, which the module then incorporates into future runs.

Q: How often does this discount calculation process run to provide fresh recommendations? A: Typically, this process runs daily, or as frequently as new sales and inventory data becomes available. This ensures that dynamic discount decisions are always based on the most current information about product performance and stock levels.

Q: What if a brand new product doesn't appear in the discount recommendations, even if it's not selling? A: New products often have a "Minimum Live Days for Analysis" setting. If a product hasn't been available for sale for the defined period, the system won't have enough reliable sales history to make an informed discount recommendation. Sales consistency thresholds also play a role in filtering out unreliable initial sales data.

Q: How does the system ensure that suggested discounts don't lead to selling products at a loss? A: The module incorporates the Cost of Goods Sold (COGS) for each product style and calculates the projected margin after applying a discount. It also considers a "Minimum Discount" setting. This ensures that discount recommendations either maintain a predefined profit margin or, at minimum, cover the product's direct cost, protecting overall profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate precise, data-driven discount percentages and actions for product styles, optimizing inventory and sales.
- **Main Input:** Cleaned and structured sales, inventory, and product data from the `DDDataPrepModule`.
- **Main Output:** Suggested discount percentages, actions (Increase, Continue, Decrease), and projected sales/profitability impacts for each product style.
- **Warning Signs:** Unexpected discount recommendations, large swings in suggested percentages, or missing profitability calculations often indicate issues originating from this module.
- **Key Settings:** “Discount Projection Period,” “Minimum Live Days for Analysis,” “Sales Data Reliability Threshold,” and “Overall Discount Target.”

Need Help? Contact your Merchandise Planning or IT Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 3:

DiscountSkuComputationModule

Last Updated: 2025-05-23

1. Business Objective

The SKU Discount Module calculates and recommends precise discount percentages for each unique item (SKU) in stock. It performs detailed analysis on individual item performance, including Rate of Sale (ROS), Days of Hand (DOH), and health status, and applies specific business rules to suggest optimal price reductions, aiming to boost sales and efficiently manage inventory at the most granular level.

Why This Matters: This granular approach ensures that discounts are applied only where most needed, preventing over-markdowns on profitable items and accelerating sales of specific slow-moving or unbalanced inventory. This directly improves profit margins, optimizes inventory turnover, and enhances customer satisfaction by ensuring desirable products are available at competitive prices.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Records:** Accurate past sales figures, filtered for reliability, are crucial to calculate the true Rate of Sale (ROS) for individual unique items (SKUs) and understand their performance trends.
- **Sales Consistency Data:** A metric that confirms the reliability of specific sales records, ensuring that only trustworthy data contributes to Rate of Sale calculations and prevents faulty decisions from incomplete transactions.
- **Sell-Through Performance Benchmarks:** Predefined targets for how quickly unique items (SKUs) should sell over a specific timeframe, allowing the system to measure an item's performance against expected rates.

Product Data

- **Unique Item (SKU) Attributes:** Detailed information like brand, category, style, and specific size/color for each unique item, which allows for granular analysis and targeted discount recommendations.

- **Product Style Classifications:** Information detailing a product style's lifecycle stage (e.g., new, seasonal, clearance), season type, and broader style groups, enabling appropriate discount strategies that apply to all its SKUs.
- **Cost of Goods Sold (COGS) by Style:** The direct cost of acquiring or producing each product style, essential for calculating potential profit margins after applying discounts to individual unique items.
- **Category Size Sequences:** The typical size progression (e.g., S, M, L or 30, 32, 34) for each product category, crucial for assessing the Ideal Size Set (ISS) completeness and identifying inventory imbalances at the SKU level.
- **Style Maximum Retail Price (MRP):** The original full selling price of a product style, used to calculate the value of projected sales and discount amounts for specific unique items.

Operational Data

- **Discount-Enabled Store List:** A list of specific stores authorized to participate in dynamic discounting initiatives, ensuring discounts are applied only where intended.
- **Current Store Inventory Levels (by SKU):** Real-time stock counts for every unique item (SKU) at each physical store, vital for calculating Days of Hand (DOH) and identifying overstocked SKUs.
- **In-Transit Inventory & Open Orders (by SKU):** Records of unique items (SKUs) currently being shipped to stores or awaiting delivery from suppliers, providing a complete picture of total available stock to prevent unnecessary discounts.
- **Warehouse-Store Allocation Rules:** Rules that define how warehouse inventory (including in-transit) should be attributed to specific stores for inventory planning and accurate Days of Hand calculations.
- **Discount Rule Matrix (by performance criteria):** A comprehensive set of business rules defining discount actions and percentages based on key metrics like Rate of Sale (ROS), Days of Hand (DOH), item health (e.g., Ideal Size Set continuity), and sell-through.
- **Style-Specific Discount Limits:** Custom maximum discount percentages set for individual product styles, allowing finer control beyond general rules to protect high-value or core products, which then apply to all their unique items.
- **Previous Discounting Decisions (by SKU or Style):** Records of past discounts applied to specific unique items (SKUs) or their parent styles, used to ensure continuity or adjust strategies based on prior performance.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Discount Cycle Duration	90 days	The number of days of recent sales data the system analyzes to understand individual unique item (SKU) performance.	Increase this for items with slower sales cycles (e.g., specific furniture SKUs) or decrease for fast-moving goods (e.g., popular fashion accessories)

Setting Name	Default Value	Description	When to Change
Minimum Live Days for Item Analysis	60 days	The minimum number of days a specific unique item (SKU) must have been available for sale in stores to be considered for discount analysis.	to capture more relevant trends. Adjust if you need to include newer unique items in discounting (lower threshold) or ensure more mature sales data (raise threshold) before applying markdowns.
Minimum Inventory Receipt Days	30 days	The minimum number of days an item (SKU) must have been in the supply chain (since its first receipt of inventory) to be eligible for age-based discount considerations.	Lower this to allow quicker discounting of new items that aren't moving, or raise it to protect initial full-price selling of new arrivals.
Size Set Health Threshold	(Varies, e.g., 0.7 for 70%)	The benchmark percentage for how complete or continuous an item's Ideal Size Set (ISS) needs to be for it to be considered "Healthy."	Modify if you find the system too aggressively discounts items due to minor size gaps (lower threshold) or misses opportunities to clear fragmented inventory (raise threshold).

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Structured & Filtered Data by SKU] --> B{SKU Discount Calculation & Recommendation}

B --> C[Suggested SKU Discount Actions & Projections]

1. Start with → **Structured & Filtered Data by SKU** from the Data Preparation Module, including clean sales history, consolidated inventory, and product classifications for each unique item.

2. Then → **Assess Individual Item Performance** by calculating metrics like Rate of Sale (ROS), Days of Hand (DOH), product age, and overall health (e.g., Ideal Size Set continuity) at the SKU level.
3. Next → **Apply Granular Discount Rules** from the defined Discount Rule Matrix, combining specific item performance metrics with profitability targets to determine precise discount actions and percentages for each unique item (SKU).
4. Finally → **Output** detailed unique item (SKU) discount recommendations, including projected sales and financial impacts, ready for review and implementation.

5. Step-by-Step Actions

1. **Retrieve** existing discounts
 - What we do: *We gather information on any discounts currently applied to product styles within relevant store groups.*
 - Why we do it: *This provides a baseline for determining if discounts for individual unique items (SKUs) need to be increased, decreased, or continued.*
2. **Compile** individual item performance profiles
 - What we do: *We create detailed performance profiles for each unique item (SKU) within its assigned store group, including current stock, market value, and available live days.*
 - Why we do it: *This provides a comprehensive view of how each SKU is performing in its specific market segment, informing targeted decisions.*
3. **Map** unique items to stores
 - What we do: *We link each unique item (SKU) to the specific stores where it is currently stocked or expected to be sold.*
 - Why we do it: *This ensures that discount decisions are made for items relevant to each store's actual inventory and distribution.*
4. **Filter** items based on market presence
 - What we do: *We identify unique items (SKUs) that haven't met minimum live days in stores, indicating they are very new or have limited market exposure.*
 - Why we do it: *This prevents premature discounts on newly launched items, protecting their initial full-price sales period and maximizing early revenue.*
5. **Calculate** item sales velocity
 - What we do: *We determine the accurate Rate of Sale (ROS) for each unique item (SKU) by analyzing only reliable historical sales data within each store where it's sold.*
 - Why we do it: *This provides a precise measure of how quickly individual items are selling, forming a critical input for granular discount decisions.*
6. **Integrate** warehouse and in-transit inventory
 - What we do: *We incorporate inventory quantities for each unique item (SKU) from warehouses and shipments currently en route, proportionally distributing these units to their associated stores based on existing sales or allocation logic.*
 - Why we do it: *This provides a complete picture of total available stock for each SKU, ensuring Days of Hand (DOH) calculations are accurate and preventing over-discounting due to incomplete stock visibility.*

7. Roll Up data to store group level

- What we do: *We aggregate key performance metrics like Rate of Sale (ROS), Days of Hand (DOH), and Sell-Through for each unique item (SKU) across its entire store group.*
- Why we do it: *This provides a consolidated view of each SKU's performance within its market segment, allowing for more strategic group-level discount decisions that consider overall demand.*

8. Assess unique item health

- What we do: *We evaluate the health of each unique item (SKU) by checking for issues such as fragmented Ideal Size Sets (ISS) within its parent style, which can hinder sales of specific sizes.*
- Why we do it: *This helps identify inventory imbalances at the SKU level that may require a discount, even if other metrics seem fine, to improve sell-through of complete size runs or clear isolated sizes.*

9. Determine item age

- What we do: *We calculate the age of each unique item (SKU) based on its first arrival date in the supply chain (receipt of inventory) or its overall live days in stores.*
- Why we do it: *Aging inventory often requires more aggressive discounting to clear, protecting against holding costs and obsolescence and freeing up capital for new stock.*

10. Categorize items by performance

- What we do: *We classify each unique item (SKU) into categories like High, Medium, or Low for its Rate of Sale (ROS), Ageing (based on DOH), and Sell-Through, using predefined benchmarks specific to its product type.*
- Why we do it: *This simplifies the application of discount rules, allowing the system to easily match item profiles to appropriate discount actions based on their specific needs.*

11. Calculate new discount recommendations

- What we do: *Based on the item's performance classifications (ROS, DOH, Health, Sell-Through), the system applies the Discount Rule Matrix to determine the suggested discount action (increase, continue, decrease) and percentage for each specific unique item (SKU).*
- Why we do it: *This ensures that discount decisions are consistent, aligned with business strategy, and tailored to each item's specific performance and inventory needs.*

12. Project future sales and impacts

- What we do: *We forecast how many units of each unique item (SKU) are expected to sell and the projected revenue and discount value if the suggested discount is applied.*
- Why we do it: *This provides a clear picture of the potential financial impact of the discount at the individual item level, allowing for profitability assessment and overall target discount reconciliation.*

13. Generate detailed reports

- What we do: *We compile all unique item (SKU) discount recommendations, performance metrics, and projected impacts into structured output reports for review.*
- Why we do it: *This provides merchandising and planning teams with the actionable insights needed to approve and implement dynamic discounting strategies at the most granular level, facilitating quick decision-making.*

6. Sample Scenario

Let's walk through a real example: A "Blue Classic Denim Jean" in a Size 32 in a flagship store.

Starting Point

- **Product:** "Blue Classic Denim Jean," Style #456.
- **Specific Unique Item (SKU):** Size 32 of Style #456.
- **Current Stock (Flagship Store, Size 32):** 20 units.
- **Warehouse Stock (Size 32):** 30 units in the warehouse, with 10 units proportionally allocated to the flagship store based on its sales contribution for the style.
- **Cleaned Rate of Sale (ROS) for Size 32:** Averaged 2 units per week over the last 90 days at the flagship store.
- **Product Age for Size 32 (from first inventory receipt):** 180 days.
- **Health Status for Style #456:** The overall style has an "Unhealthy" Ideal Size Set (ISS) because critical sizes (Size 30 and 34) are completely out of stock, leaving fragmented inventory for the style.
- **Current Discount for Style #456 (applied previously at the style level):** 10%.
- **Discount Rule Matrix:** Rules dictate that individual unique items (SKUs) of an "Unhealthy" style (due to fragmented Ideal Size Set) that are older than 150 days and have "Low" Rate of Sale (ROS) should receive a discount increase of 10-15%. The maximum allowed discount for this product category is 40%.

What Happens

1. **Item (SKU) Data Compilation:** The module first compiles all specific data for "Blue Classic Denim Jean" Size 32 at the flagship store. This includes its 20 units on hand and the 10 allocated warehouse units, totaling 30 available units for this specific SKU.
2. **Individual Item Performance Assessment:** It calculates the Rate of Sale (ROS) for this specific SKU as 2 units per week. It then determines its Days of Hand (DOH) based on the combined 30 units and the 2 units/week ROS, resulting in 15 weeks (or 105 days) of supply. Its age is confirmed as 180 days.
3. **Health Status Integration:** The system checks the Ideal Size Set (ISS) health for the parent style (#456) and confirms it as "Unhealthy" due to missing critical sizes (30, 34). This "unhealthy" status impacts all individual SKUs within that style, even if a specific size like 32 is not grossly overstocked on its own.
4. **Rule Application:** The system references the Discount Rule Matrix. It identifies the rule for an individual unique item (SKU) that is part of an "Unhealthy" style, is "Lagging" in age (180 days is greater than the 150-day threshold), and has a "Low" Rate of Sale (2 units/week). This rule recommends an "Increase" action with a specific increment.
5. **Discount Calculation:** Applying the rule, the system suggests increasing the discount by 15%, bringing the new suggested discount to 10% (current) + 15% (increment) = 25%. This 25% is then checked against the maximum allowed 40% for the category and found to be acceptable.
6. **Projected Sales Calculation:** The module projects how many units of Size 32 are expected to sell at a 25% discount, considering historical responsiveness to price changes for similar items.

7. **Profitability Check:** The module calculates the projected selling price and margin for Size 32 at a 25% discount, ensuring the sale remains profitable relative to the Cost of Goods Sold (COGS).

Final Outcome

The SKU Discount Module recommends a 25% discount for the “Blue Classic Denim Jean” in Size 32 at the flagship store. This action is projected to accelerate the sales of this specific size, helping to clear the inventory for the overall style even though Size 32 itself is not grossly overstocked. The decision is driven by its contribution to a fragmented Ideal Size Set and its aging profile, aiming to optimize inventory movement and profitability at the unique item level.

Key Takeaway: This module goes beyond style-level analysis to pinpoint specific unique items (SKUs) requiring targeted discounts, even when their parent style has broader inventory challenges, ensuring maximum inventory efficiency and profit.

7. Results

This step produces:

Main Outputs

- **Suggested Unique Item (SKU) Discount Actions and Percentages:** Clear recommendations (Increase, Continue, Decrease) and specific discount percentages for each unique item (SKU) within relevant store groups. This directly tells merchandising teams what granular pricing adjustments to make in real-time.
- **Projected Unique Item (SKU) Sales and Discount Impacts:** Forecasted units sold and revenue impacts for each unique item (SKU) under the suggested discount, including projected margins. This helps sales and finance teams understand the expected financial outcome at the most granular level.
- **Detailed Unique Item (SKU) Performance Classifications:** The classified Rate of Sale (ROS) level, Ageing level (based on Days of Hand), Health Status (driven by Ideal Size Set continuity), and Sell-Through level for each unique item (SKU). This provides the rationale behind each discount recommendation, aiding in granular strategic planning.
- **Recommended Selling Prices and Margins (by SKU):** The new suggested selling price and the estimated profit margin per unit for each unique item (SKU) at that price. This ensures profitability is considered at the item level and helps merchandise planners evaluate financial viability before implementation.

Business Reports

- **SKU Discount Recommendation Report:** A comprehensive list of all suggested discounts, performance metrics, and remarks for each unique item (SKU) and store group. This is used by merchandising and pricing teams for granular review, approval, and implementation of precise markdowns.
- **SKU-Level Profitability Analysis:** A summary report detailing the anticipated sales uplift, revenue, and gross margin impacts of the proposed SKU-level

discounts across all relevant items. This is crucial for financial planning and executive decision-making regarding overall promotional spend.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed (e.g., inventory updates for specific unique items)	The system will use the last available data, potentially leading to outdated Days of Hand (DOH) calculations and incorrect discount recommendations for those unique items (SKUs).	Investigate the data feed source immediately. Manually verify critical SKU-level data points if the module must run, or reschedule the run until data is complete and accurate.
No historical sales data for a specific new unique item (SKU)	The system might treat its Rate of Sale (ROS) as zero or very low, potentially triggering aggressive discounts too early for a new item.	Manually set a starting discount or override rules for new unique items (SKUs), or use benchmark data from similar successful new item launches to provide initial guidance and protect margins.
Ideal Size Set (ISS) health assessment for a style shows all unique items (SKUs) as “Unhealthy”	The system may recommend aggressive discounts across all sizes within a style, potentially eroding margins if the underlying issue isn’t truly an unhealthy size run but a data error or misconfiguration.	Review the criteria for “Ideal Size Set Health” and ensure they are appropriately set. Investigate inventory accuracy at the SKU level to confirm physical stock matches records.
Discount Rule Matrix for SKU-level considerations is incomplete	The system will not have the necessary logic to classify unique items (SKUs) or suggest appropriate discounts, potentially leading to no recommendations or incorrect ones.	Immediately update and load the Discount Rule Matrix with comprehensive rules that cover all relevant unique item (SKU) performance scenarios, ensuring all items have a clear discount path.
Incorrect warehouse-to-store allocation leads to skewed DOH for unique items (SKUs)	If warehouse inventory is misattributed to stores, the Days of Hand (DOH) for individual unique items (SKUs) in certain stores will be inaccurate, leading to inappropriate discount recommendations.	Verify the accuracy of warehouse-to-store mapping rules and ensure they reflect the actual distribution strategy for unique items (SKUs) to prevent miscalculations.

9. Next Steps

Before This Step

- **Data Preparation:** The Data Preparation Module must have successfully gathered, cleaned, and structured all raw sales transactions, current inventory levels (by SKU), product master data, and operational parameters. This module relies entirely on the accuracy and completeness of that prepared data.
- **Style-Level Discount Calculation:** If style-level discounts are already being applied or considered, their output from the Style Discount Module provides a crucial baseline for this module, influencing the starting point for SKU-specific adjustments and ensuring consistency.

After This Step

- **Discount Action & Application Module:** The suggested discount actions and percentages generated by this module are sent to a subsequent module responsible for actually implementing these price changes in the point-of-sale or e-commerce systems at the unique item (SKU) level. This ensures recommendations are put into action.
- **Inventory Optimization Module:** The detailed unique item (SKU) performance data (Rate of Sale, Days of Hand, health) and projected sales impacts can feed into broader inventory planning and buying systems. This helps refine future purchasing decisions and allocations to prevent recurrence of overstock or fragmented inventory situations for specific unique items.

10. Common Questions

Q: Why did a specific size or color of a product receive a discount recommendation, even if other sizes/colors are selling well? A: This module analyzes each unique item (SKU) individually. Even if the overall product style is performing well, a particular size or color might have a high Days of Hand (DOH), be part of a fragmented Ideal Size Set (ISS) that needs clearing, or have a low individual Rate of Sale (ROS). The system targets these specific SKUs to optimize their inventory movement and maximize overall style sell-through.

Q: Can we prevent certain unique items (SKUs) from ever being discounted, regardless of their performance? A: Yes, the system supports manual overrides and specific discount limits for individual product styles or unique items (SKUs). These settings can be configured to protect core, high-value, or strategically important items from any dynamic discounts, allowing merchandising teams to maintain strategic control over pricing.

Q: How often does this SKU-level discount calculation process run to provide fresh recommendations? A: Typically, this process runs daily, or as frequently as new sales and inventory data becomes available (e.g., hourly during peak periods). This ensures that dynamic discount decisions are always based on the most current information about each unique item's performance and stock levels, allowing for agile pricing adjustments.

Q: What if a brand new unique item (SKU) isn't appearing in the discount recommendations, even if it's not selling? A: New unique items (SKUs) are often subject to a "Minimum Live Days for Item Analysis" setting or a "Minimum Inventory Receipt Days" threshold. If an item hasn't been available for sale or in the supply chain for the defined period, the system won't have enough reliable history to make an informed discount recommendation, protecting its initial full-price selling period.

Q: How does the system ensure that suggested discounts for individual items (SKUs) don't lead to selling them at a loss? A: The module incorporates the Cost of Goods Sold (COGS) for each product style (which applies to its unique items) and calculates the projected profit margin after applying a discount. It also adheres to predefined "Minimum Discount" settings and checks against the COGS, ensuring that recommendations either maintain a predefined profit margin or, at minimum, cover the product's direct cost, protecting overall profitability at the item level.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate precise, data-driven discount percentages and actions for individual unique items (SKUs), optimizing their inventory and sales at the most granular level.
- **Main Input:** Cleaned and structured sales, inventory, and product data from the Data Preparation Module, specifically for each unique item (SKU).
- **Main Output:** Suggested discount percentages, actions (Increase, Continue, Decrease), and projected sales/profitability impacts for each unique item (SKU).
- **Warning Signs:** Unexpected discount recommendations for specific sizes/colors, large swings in suggested percentages for individual items, or missing profitability calculations at the SKU level. These often indicate issues originating from this module.
- **Key Settings:** "Discount Cycle Duration," "Minimum Live Days for Item Analysis," "Minimum Inventory Receipt Days," and "Size Set Health Threshold."

Need Help? Contact your Merchandise Planning or IT Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

dynamicDiscounting FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ document provides a high-level overview of the dynamicDiscounting algorithm, its capabilities, and how it informs business decisions related to product pricing and inventory management.

General

Q: What is the dynamicDiscounting algorithm? **A:** The dynamicDiscounting algorithm is a system designed to calculate and recommend optimal discount percentages for products (SKUs or styles) based on their sales performance, inventory levels, and other business factors. Its goal is to maximize sales and revenue while efficiently managing inventory.

Q: Why do we need dynamic discounting instead of traditional methods? **A:** Traditional discounting often relies on fixed schedules or manual decisions. Dynamic discounting uses real-time data and sophisticated calculations to respond quickly to product performance, helping to clear slow-moving inventory, reduce stockouts by optimizing turnover, and improve overall profitability more effectively than static approaches.

Q: What does the algorithm aim to achieve for the business? **A:** The primary goals are to optimize the Rate of Sale (ROS) for products, reduce excess inventory, minimize markdowns for healthy products, and ultimately improve gross margin by applying discounts strategically only when and where they are most impactful.

Q: Who typically uses the outputs from this algorithm? **A:** Merchandisers, inventory planners, and category managers are the primary users. They leverage the algorithm's recommendations to make informed decisions about pricing strategies, markdown plans, and inventory adjustments for their respective product categories.

Q: Does this algorithm apply to all products? **A:** The algorithm is primarily designed for products where dynamic pricing adjustments can drive sales and improve inventory health. Specific rules and configurations can be applied to exclude certain products, such as newly launched items, high-performing core products, or those with very limited stock, as determined by business logic.

Configuration & Settings

Q: How do I control which products are considered for dynamic discounting? **A:** The `DDDataPrepModule` prepares data based on internal filters and rules. You can define specific criteria such as product category, lifecycle stage, or current sales velocity to include or exclude items. The algorithm then applies further business rules during the computation phase.

Q: Can I set a maximum discount percentage the algorithm can recommend? **A:** Yes, the algorithm incorporates configurable limits for discount percentages. These can be set globally or even by product category or style within the `DiscountComputationModule` and `DiscountSkuComputationModule` to align with your business's profitability goals.

Q: How does the algorithm determine if a product needs a discount, and how much? **A:** The `DiscountComputationModule` and `DiscountSkuComputationModule` analyze various factors including a product's ROS, inventory levels (like Days on Hand), product aging, and current health status (e.g., if it's overstocked or underperforming relative to its target ROS). It then applies predefined business rules and discount factors to suggest the appropriate discount percentage.

Q: Can we apply different discounting strategies for different store groups? **A:** Yes, the `DDDataPrepModule` includes functionality for setting up and categorizing store groups. This allows the subsequent computation modules to apply different discount rules and strategies tailored to the unique performance and inventory profiles of each store group.

Q: Is there a way to prioritize clearing old inventory versus maximizing profit on newer items? **A:** Yes, the algorithm incorporates product aging as a key factor. Rules can be configured to apply more aggressive discounts to older inventory (e.g., "aged" products) within the `DiscountComputationModule` or `DiscountSkuComputationModule`, ensuring timely clearance and reducing holding costs.

Data & Inputs

Q: What types of data does the algorithm need to function? **A:** The `DDDataPrepModule` requires comprehensive data including historical sales performance, current inventory levels, product master data (e.g., style, SKU, attributes like color, size), style lifecycle information, and predefined discount factors and business rules.

Q: How often is the input data updated for the algorithm? **A:** The frequency of data updates depends on your system's integration and scheduling. The `DDDataPrepModule` is designed to process and refresh data regularly, ensuring that the dynamic discounting recommendations are based on the most current sales and inventory information available.

Q: How does the algorithm determine a product's "health"? **A:** Product health is assessed based on various performance indicators processed by the computation modules. This typically involves comparing actual ROS against target ROS,

evaluating inventory levels against an Ideal Size Set (ISS) or Optimum Depth (OD) for a category, and considering product aging. A product might be deemed “unhealthy” if it’s selling too slowly or if inventory is unbalanced.

Q: What is “style lifecycle data” and why is it important? **A:** Style lifecycle data defines the stages a product goes through from launch to end-of-life (e.g., New Arrival, Core, Markdown). This information, prepared by the `DDDataPrepModule`, helps the algorithm apply appropriate discount rules and avoid prematurely discounting new or strategic items.

Q: How are “store groups” used in the data preparation? **A:** Store groups allow for segmentation of your retail locations based on shared characteristics (e.g., high-volume stores, regional clusters, outlet stores). The `DDDataPrepModule` ensures that sales and inventory data are aggregated and processed according to these groups, enabling the computation modules to generate location-specific discount recommendations.

Process & Logic

Q: How does the algorithm decide *when* to recommend a discount for a product? **A:** The `DiscountComputationModule` (for styles) and `DiscountSkuComputationModule` (for SKUs) identify products that meet specific criteria, such as slower-than-expected ROS, high Days on Hand (DOH), or being identified as “aged” inventory. These criteria trigger the discount recommendation process.

Q: What is the difference between style-level and SKU-level discounting? **A:** Style-level discounting, handled by the `DiscountComputationModule`, applies discounts to an entire product style (e.g., “Men’s Classic T-Shirt” regardless of size or color). SKU-level discounting, by the `DiscountSkuComputationModule`, is more granular, applying discounts to specific individual items (e.g., “Men’s Classic T-Shirt, Blue, Size Large”). The choice depends on the desired level of granularity and business impact.

Q: How does the algorithm account for an unbalanced Ideal Size Set (ISS) or Optimum Depth (OD)? **A:** While OD and OW relate to overall category/display planning, the `DiscountSkuComputationModule` specifically considers the Ideal Size Set (ISS) when evaluating SKU health. If a specific SKU within a style has significantly higher inventory or lower ROS compared to its peers in the ISS, it’s more likely to be targeted for a discount to rebalance stock.

Q: Does the algorithm consider stockouts or extremely low inventory levels? **A:** Yes, the computation modules are designed to account for inventory levels. Products with critically low stock or those at risk of immediate stockout are generally excluded from discount recommendations to prevent further depletion and preserve full-price sales potential.

Q: What happens if there’s no historical sales data for a specific product? **A:** For products with insufficient historical sales data (e.g., very new items), the algorithm typically defaults to a “no discount” recommendation or applies predefined new-product rules based on its lifecycle stage. It avoids making arbitrary discount suggestions without sufficient performance insights.

Outputs & Results

Q: What kind of recommendations does the algorithm provide? **A:** The algorithm provides specific discount recommendations, including the suggested discount percentage and the targeted product (style or SKU). It also calculates projected sales and the anticipated financial impact of applying these discounts.

Q: How can I access and view the dynamic discount recommendations? **A:** The output from the `DiscountComputationModule` and `DiscountSkuComputationModule` is typically provided in a structured format (e.g., a report, data feed, or dashboard) that can be integrated into your existing merchandising or inventory management systems for review and action.

Q: Does the algorithm provide insights into the expected impact of its recommendations? **A:** Yes, a key output of the computation modules is the projected sales and discount impact. This includes estimated units sold and revenue generated at the recommended discount level, allowing users to understand the potential benefits of implementing the suggestions.

Q: What key business metrics should I track to evaluate the effectiveness of dynamic discounting? **A:** To assess effectiveness, you should monitor key metrics such as: * **Rate of Sale (ROS):** To see if products are moving faster. * **Inventory Turnover:** To gauge efficiency in clearing stock. * **Markdown Rate/Cost:** To understand the financial impact of discounts. * **Gross Margin %:** To ensure discounts aren't eroding profitability excessively. * **Stockout Rate:** To ensure optimal inventory levels are maintained.

Q: Can the recommendations from the algorithm be integrated directly into our pricing system? **A:** While the algorithm provides recommendations, the actual integration and implementation into pricing systems typically involve an additional step where business users review and approve the suggestions. This allows for manual oversight and ensures alignment with broader business strategies before applying changes.

Error Handling & Edge Cases

Q: What happens if the input data for the algorithm is incomplete or inaccurate? **A:** The `DDDataPrepModule` includes data cleaning and validation steps. However, if significant data gaps or inaccuracies persist, the recommendations from the `DiscountComputationModule` and `DiscountSkuComputationModule` may be suboptimal or unreliable. "Garbage in, garbage out" applies; clean data is crucial for accurate outputs.

Q: Are there specific products that the algorithm will intentionally avoid discounting? **A:** Yes, the algorithm's rules can be configured to exclude certain products. This often includes new arrivals, strategic core products that maintain full price, items with very low stock to prevent stockouts, or products explicitly marked for exclusion based on business strategy.

Q: How does the algorithm handle situations with highly volatile sales data? **A:** The algorithm's internal logic and calculations for ROS and product health are designed to handle typical sales fluctuations. However, for extremely volatile products or

those with highly seasonal patterns, it's important to ensure the input data and configuration rules (e.g., seasonality factors) are appropriately set up in the `DDDataPrepModule` to prevent erratic recommendations.

Q: What if the business rules or discount factors are not properly defined? **A:** If discount factors or business rules are incorrectly configured within the `DDDataPrepModule` or applied in the computation modules, the algorithm may generate recommendations that are not aligned with business objectives (e.g., too aggressive discounts, or missing opportunities). Regular review and testing of these configurations are essential.

Q: Does the algorithm consider external factors like competitor pricing or promotions? **A:** As defined, this version of the `dynamicDiscounting` algorithm primarily focuses on internal factors like sales, inventory, and product attributes. It does not natively ingest external competitor pricing or promotional data. Such considerations would typically be added through external business intelligence or manual overrides based on market analysis.

Need More Help? Contact the support team at [Support Contact Information].

dynamicDiscounting Documentation

- Version: 1.0
- Last Updated: 2025-05-23

2. Executive Summary

The dynamicDiscounting algorithm optimizes product pricing by generating data-driven discount recommendations. It processes sales and inventory data alongside predefined business rules to identify optimal discounts at both style and SKU levels. The primary goal is to maximize sales, reduce excess inventory, and ultimately improve profitability by swiftly adapting to product performance.

3. Architecture Overview

graph TD

A[DDDataPrepModule] -->|prepares| B[DiscountComputationModule]

A[DDDataPrepModule] -->|prepares|

C[DiscountSkuComputationModule]

- **DDDataPrepModule**: Prepares and initializes all necessary input data for dynamic discounting calculations.
- **DiscountComputationModule**: Calculates and recommends discounts at the style level based on aggregated data and business rules.
- **DiscountSkuComputationModule**: Calculates and recommends discounts at the SKU level, performing detailed analysis on individual SKU performance.

4. Prerequisites & Setup

To run the dynamicDiscounting application, ensure the following are installed:

- **Java Development Kit (JDK)**: Version 17 or higher.
- **Apache Maven**: Version 3.8 or higher.

Build Commands: To compile the project and its dependencies, navigate to the root directory of the repository and run:

```
mvn clean install
```

Running the Orchestrator: The ModuleGroup orchestrates the execution of the submodules. To run the application, use the following command from the root directory:

```
java -jar module-group/target/dynamicDiscounting-module-group-1.0.jar
```

Ensure args.properties and common-data.json are correctly configured in the root directory.

5. Folder & File Structure

```
dynamicDiscounting/
├─ module-group/          # Contains the main orchestrator
                           logic for executing submodules.
├─ submodules/            # Directory containing all individual
                           submodule implementations.
├─ args.properties        # Global configuration parameters for
                           the entire application.
├─ common-data.json       # Defines the schema for shared data
                           objects used across modules.
├─ outputs/               # Directory where final discount
                           recommendations and logs are stored.
```

6. Global Configuration

The args.properties file contains global configuration parameters that control the overall behavior of the dynamic discounting pipeline.

Setting Name	Default Value	Description	Valid Range/When to Change
discount.level	STYLE	Determines if discounts are computed at the STYLE or SKU level.	STYLE, SKU. Change based on the desired granularity of discount recommendations.
data.lookback.weeks	12	Number of past weeks to consider for sales data aggregation and ROS calculation.	4-52. Increase for more historical context, decrease for a focus on very recent trends.
min.ros.threshold	0.1	Minimum Rate of Sale (ROS) for a product to be considered for discounting.	0.0 to 1.0. Adjust to control which products are eligible for initial discount consideration. Products below this ROS may be flagged for deeper analysis.

Setting Name	Default Value	Description	Valid Range/When to Change
output.format	CSV	Format for outputting the final discount recommendations.	CSV, JSON. Change based on the requirements of downstream systems that consume the recommendations.
log.level	INFO	Specifies the verbosity of logging during execution.	DEBUG, INFO, WARN, ERROR. Increase for detailed troubleshooting (DEBUG), decrease for production (INFO, WARN).

7. Shared Data Model

The common-data.json schema defines the structure for data objects shared and consumed across the various modules in the pipeline.

Field	Type	Description
productId	string	Unique identifier for a product style. This typically groups various SKUs (e.g., “Men’s T-Shirt”).
skuId	string	Unique identifier for a specific product item, including attributes like size, color, material.
storeId	string	Unique identifier for a retail store or location.
salesUnits	integer	Number of units sold for a product (SKU or style) within a given period.
currentInventory	integer	Current on-hand inventory for a product SKU in a specific location.
discountFactor	number	A multiplier applied to base discount percentages, often based on category, lifecycle, or sales performance.
category	string	The product category (e.g., “Apparel”, “Footwear”, “Accessories”).
weeksOnHand	integer	The number of weeks a product (SKU) has been available in inventory since its initial receipt.
idealSizeSet	array	An Ideal Size Set (ISS) representing the optimal combination of sizes for a given product style.
rateOfSale	number	The Rate of Sale (ROS) in units per week per store for a product or SKU.
attributeGroup	object	

Field	Type	Description
		An Attribute Group (AG) containing key attributes like color, size, or material for analysis.

8. Submodule Chapters

8.1 DDDataPrepModule

8.1.1 Module Name & Business Objective

The DDDataPrepModule is responsible for preparing and initializing all necessary input data for the dynamic discounting calculations. This includes retrieving and cleaning sales data, setting up store groups, defining style lifecycle data, loading discount factors and rules, and populating inventory details. It serves as a foundational data pipeline, ensuring subsequent computation modules have access to correctly structured and filtered data.

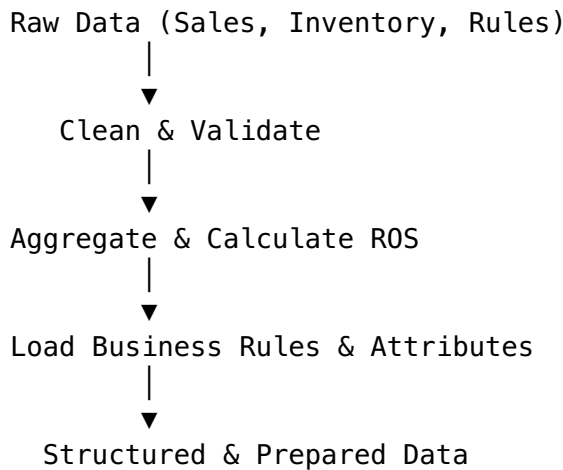
8.1.2 Required Information

- **Sales Data:**
 - Raw historical sales transactions (units, price, date, store, SKU).
 - Return data for accurate net sales calculation.
 - Rate of Sale (ROS) for various products over time.
- **Product Data:**
 - SKU master data (e.g., color, size, material, OD, OW, ISS).
 - Product hierarchy (SKU to style, style to category).
 - Product lifecycle stages (new arrival, core, markdown, clearance).
 - Attribute Group (AG) definitions.
- **Operational Data:**
 - Current inventory levels (on-hand, in-transit, on-order) by SKU and location.
 - Store master data (location, store type, store groups).
 - Predefined discount factors and business rules.

8.1.3 Business Settings

- `data.lookback.weeks`: Configures how far back in time the module should look for sales data (e.g., 12 weeks).
- `store.grouping.strategy`: Defines the method for grouping stores (e.g., GEOGRAPHIC, PERFORMANCE).
- `min.sales.transactions`: Minimum number of sales transactions required for a product to be considered for ROS calculation.

8.1.4 Process Flow



8.1.5 Step-by-Step Actions

1. **Retrieve** raw sales transactions, current inventory, and product master data *from various source systems like POS, ERP, and product information management (PIM).*
2. **Cleanse and deduplicate** the retrieved data *to ensure accuracy and remove inconsistencies.*
3. **Validate** data integrity *by checking for missing values, incorrect formats, and logical inconsistencies (e.g., negative sales).*
4. **Aggregate** sales data by time period (e.g., weekly), store, and product (SKU/style) *to calculate relevant metrics like Rate of Sale (ROS).*
5. **Identify and apply** store grouping definitions *to segment data for region-specific or performance-based analysis.*
6. **Load** predefined discount factors, business rules, and product lifecycle stages *to enrich the data for subsequent decision-making.*
7. **Populate** internal data structures and common data models *to ensure a consistent and optimized dataset is available for the computation modules.*

8.1.6 Sample Scenario

Example: 100,000 raw sales records for the last 12 weeks are ingested. After cleaning and validation, 98,000 valid records remain. The module aggregates these sales by SKU and store, calculating the weekly ROS for each. Concurrently, it loads current inventory data for 50,000 SKUs across 200 stores and applies pre-defined store groupings (e.g., "High-Traffic City Stores," "Suburban Outlets"). This ensures that when the "Red Dress – Style 123" is analyzed for discounting, its complete sales history, current stock, and applicable rules for "City Store Group" are readily available, ready for detailed analysis by the discount computation modules. **Key Takeaway:** Accurate, complete, and properly structured data is the bedrock for effective dynamic discounting. This module ensures data readiness.

8.1.7 Results

- A comprehensive, cleaned, and validated dataset containing:
 - Aggregated sales performance (ROS, units sold).
 - Current inventory levels.
 - Product master data with attributes (color, size, category).
 - Applied business rules and discount factors.
 - Defined store groups and product lifecycle stages.
- These results are stored in an internal data model accessible by the subsequent computation modules.

8.1.8 Unusual Cases

- **Missing or delayed data feed:** If core sales or inventory data feeds are incomplete or late, the module will halt or use the most recent available data, potentially impacting recommendation accuracy.
- **Inconsistent attribute mapping:** Discrepancies in product attribute values (e.g., “red” vs. “Rd”) can lead to incorrect aggregation and rule application.
- **Sudden spikes/drops in data:** Unexpected data anomalies might skew ROS calculations, requiring manual intervention or outlier detection logic.

8.1.9 Next Steps

The prepared and structured data is then passed as input to either the `DiscountComputationModule` (for style-level analysis) or the `DiscountSkuComputationModule` (for SKU-level analysis), depending on the configured `discount.level` parameter.

8.1.10 Common Questions

1. **How fresh is the data used?:** The data freshness depends on the frequency of input data feeds, typically daily or weekly.
2. **What if there are gaps in historical sales data?:** The module attempts to impute missing data or flag products with insufficient history for discounting.
3. **Can we integrate with custom data sources?:** Yes, the module is designed with adapters for various data source integrations, but new ones may require development.
4. **How are new products handled with no sales history?:** New products are typically excluded from discounting for a predefined period or until sufficient ROS data is accumulated.
5. **Is data quality monitored?:** Yes, built-in validation checks flag data quality issues, which are logged for review.

8.1.11 Quick Reference

- **Purpose:** Data preparation, cleaning, structuring.
- **Inputs:** Raw sales, inventory, product master, business rules.
- **Outputs:** Clean, aggregated, ready-to-use data for analysis.
- **Key Metric:** ROS calculation, inventory accuracy.

8.2 DiscountComputationModule

8.2.1 Module Name & Business Objective

The DiscountComputationModule is responsible for calculating and recommending discounts at the style level. It processes aggregated sales and inventory data, assesses product health, ROS, and aging, and applies defined business rules to determine suggested discount actions and percentages for each style within relevant store groups. It also calculates projected sales and discount impacts.

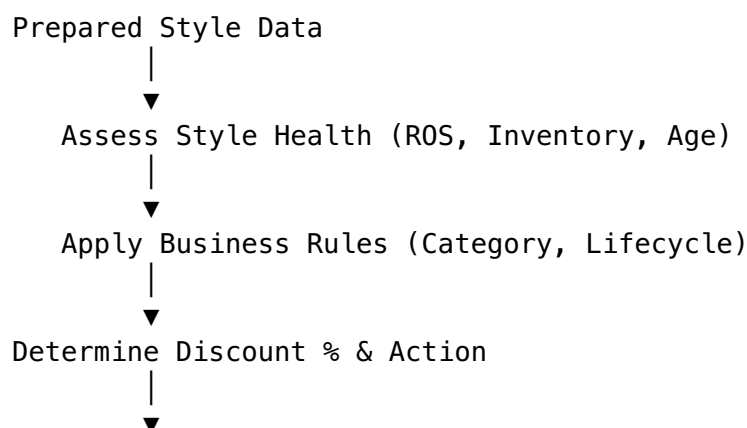
8.2.2 Required Information

- **Sales Data:**
 - Aggregated Rate of Sale (ROS) by style and store group.
 - Historical sales trends at style level.
- **Product Data:**
 - Style health status (e.g., newness, age, lifecycle stage).
 - Category and sub-category mappings.
 - Optimum Depth (OD) and Optimum Width (OW) guidelines for product categories.
- **Operational Data:**
 - Current inventory levels at style level across relevant locations.
 - Applicable discount rules (e.g., “if ROS below X, discount by Y%”).
 - Store group definitions.
 - Discount factors and maximum allowable discount percentages.

8.2.3 Business Settings

- `discount.level`: Must be set to STYLE for this module to run effectively.
- `max.style.discount.percent`: Upper limit for any style-level discount recommendation (e.g., 40%).
- `style.aging.threshold.weeks`: Number of weeks after which a style is considered “aged” and eligible for deeper discounts.
- `style.ros.threshold.low`: ROS threshold below which a style is flagged as underperforming.

8.2.4 Process Flow



Project Sales & Impact



Style-Level Discount Recommendations

8.2.5 Step-by-Step Actions

1. **Ingest** prepared style-level data *from the DDDataPrepModule for efficient processing.*
2. **Assess** the overall health of each product style *by analyzing its Rate of Sale (ROS), current inventory levels, and weeks on hand.*
3. **Identify** relevant business rules *based on the style's category, lifecycle stage, and performance metrics.* For example, a rule might state: "If ROS is below `min.ros.threshold` and inventory is high, apply a discount."
4. **Determine** the suggested discount percentage and action (e.g., "Mark down," "Clearance") *by applying the identified business rules and discount factors.*
5. **Project** estimated sales uplift and inventory reduction *to quantify the potential impact of the recommended discount on business outcomes.*
6. **Generate** a comprehensive list of style-level discount recommendations *including the suggested discount percentage, reason, and projected impact, ready for review or export.*

8.2.6 Sample Scenario

A specific style, "Classic Jeans – Style 456," has a Rate of Sale (ROS) of 0.05 units per week per store over the last 8 weeks, which is significantly below its target of 0.2. Inventory for this style is high, with 150 units across all stores. The system identifies a business rule: "If Style ROS < 0.1 and Style Inventory > 100 units, recommend a 20% discount." Based on this rule, the module suggests a 20% discount for "Classic Jeans – Style 456" and projects an increase of 50 units sold over the next four weeks, reducing stock by 33%. **Key Takeaway:** Automated rules efficiently identify underperforming styles and suggest targeted discounts based on multiple performance indicators, driving inventory turns and sales.

8.2.7 Results

- A list of recommended discount percentages and actions for each eligible product style.
- Projected sales uplift and inventory reduction figures associated with each recommendation.
- Reasons or rule triggers for each recommended discount.

8.2.8 Unusual Cases

- **Missing or delayed data feed:** Relying on stale data can lead to inaccurate discount recommendations that do not reflect current product performance.
- **Conflicting business rules:** If multiple rules apply to a single style and suggest different actions or percentages, the module must have a predefined conflict resolution mechanism (e.g., priority ranking).

- **Market saturation:** If a style is underperforming due to market-wide saturation rather than internal factors, simple discounting may not be the optimal solution.

8.2.9 Next Steps

The generated style-level discount recommendations are typically passed to a review dashboard for a human merchandiser's approval or directly to a pricing system for execution. Post-implementation, the impact is monitored through sales and inventory reports.

8.2.10 Common Questions

1. **How are the discount percentages determined?:** Percentages are based on predefined business rules, product health (ROS, inventory), and discount factors, often with a tiered approach.
2. **Can we override the recommended discounts?:** Yes, the output is typically a recommendation that can be reviewed and manually adjusted by merchandisers before implementation.
3. **What if a style is performing well but has excess inventory?:** The module balances ROS with inventory levels; high inventory, even with moderate ROS, can trigger a discount to optimize Days on Hand (DOH).
4. **How does this impact overall margin?:** The module aims to optimize total profitability by balancing sales increase against margin erosion, often projecting margin impact.
5. **Does it consider seasonal trends?:** Yes, if historical sales data includes sufficient seasonal patterns, the ROS calculation can inherently reflect seasonality.

8.2.11 Quick Reference

- **Purpose:** Calculate style-level discount recommendations.
- **Inputs:** Prepared style data (ROS, inventory, rules).
- **Outputs:** Discount % per style, projected impact.
- **Key Metrics:** ROS, inventory turn, style aging.

8.3 DiscountSkuComputationModule

8.3.1 Module Name & Business Objective

The DiscountSkuComputationModule is responsible for calculating and recommending discounts at the SKU level. It performs detailed analysis on individual SKU performance, including ROS, DOH, and health status, and applies business rules to suggest specific discount actions and percentages, aiming to optimize sales and inventory at the most granular level.

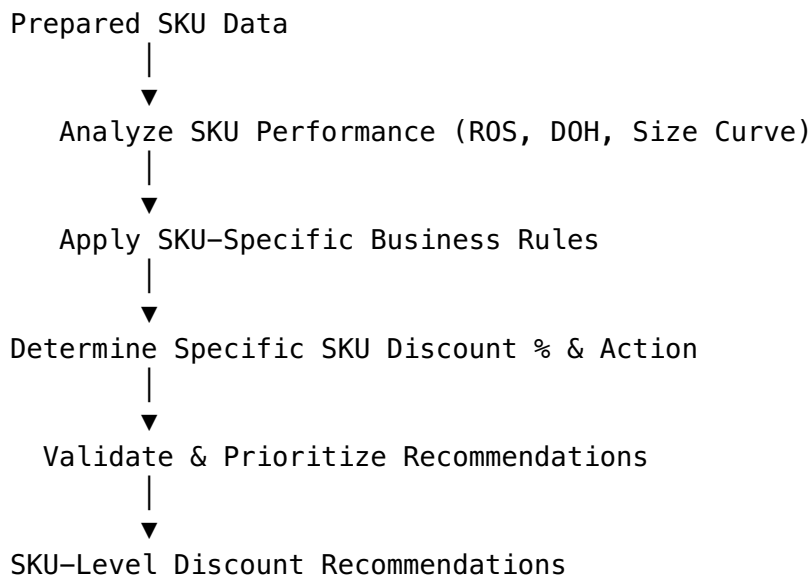
8.3.2 Required Information

- **Sales Data:**
 - SKU-specific Rate of Sale (ROS).
 - Detailed sales history by SKU, store, and date.
- **Product Data:**
 - SKU attributes (color, size, material, OD, OW, ISS).
 - Days on Hand (DOH) for each SKU.
 - Ideal Size Set (ISS) and Attribute Group (AG) definitions.
- **Operational Data:**
 - Current inventory levels for each specific SKU by location.
 - SKU-level business rules (e.g., “if specific size has high DOH, discount by X%”).
 - Maximum allowable discount percentages for individual SKUs.

8.3.3 Business Settings

- `discount.level`: Must be set to SKU for this module to run effectively.
- `max.sku.discount.percent`: Absolute upper limit for any SKU-level discount (e.g., 60%).
- `sku.doh.threshold.high`: Days on Hand (DOH) threshold above which a SKU is considered overstocked and a candidate for discounting.
- `size.curve.deviation.tolerance`: Tolerance level for deviation from Ideal Size Set (ISS) distribution.

8.3.4 Process Flow



8.3.5 Step-by-Step Actions

1. **Ingest** prepared SKU-level data *from the `DDDataPrepModule` for detailed granular analysis.*
2. **Analyze** individual SKU performance *by evaluating its Rate of Sale (ROS), Days on Hand (DOH), and adherence to Ideal Size Set (ISS) distribution.* This identifies specific color/size combinations that are overstocked or underperforming.

3. **Apply** SKU-specific business rules *which might include rules based on size imbalances, specific color performance, or aging of individual SKUs*. For example, “If ‘Blue, Size L’ has DOH > 180, apply an additional 10% discount.”
4. **Propose** specific discount percentages and actions *for each unique SKU, considering its performance and rule applicability*.
5. **Validate** proposed SKU discounts *against constraints such as minimum margin requirements or maximum allowed discount percentages to prevent excessive markdowns*.
6. **Generate** a detailed list of SKU-level discount recommendations *including the SKU identifier, suggested discount percentage, and the underlying reason for the markdown*.

8.3.6 Sample Scenario

Within "Classic Jeans – Style 456," which was previously flagged for a 20% style-level discount, the "Blue, Size L" SKU has an ROS of 0.02 and 75 units in inventory (a DOH of 300 days), while "Blue, Size S" has an ROS of 0.3 and only 10 units (a DOH of 20 days). The module's SKU-specific rules identify "Blue, Size L" as an outlier due to its extremely low ROS and high DOH for that specific size. It then recommends a deeper discount of 25% for "Blue, Size L" to aggressively clear stock, while "Blue, Size S" receives no additional discount beyond any style-level recommendation. This precise action helps manage inventory imbalance. **Key Takeaway:** Granular SKU-level analysis allows for precise discounting, addressing specific inventory imbalances or performance issues within a style, optimizing OD and ISS.

8.3.7 Results

- A detailed list of recommended discount percentages and actions for each specific SKU.
- Optimized inventory levels at the SKU level.
- Insights into SKU performance deviations from Ideal Size Set (ISS) or Attribute Group (AG) norms.

8.3.8 Unusual Cases

- **Missing or delayed data feed:** Inaccurate or outdated SKU-level inventory data can lead to recommending discounts on out-of-stock items or missing opportunities for overstocked ones.
- **SKU data inconsistencies:** Errors in SKU attributes (e.g., incorrect size mapping) can lead to misapplication of rules, like misidentifying a popular size as overstocked.
- **Cannibalization risk:** Over-discounting one SKU within a style might cannibalize sales of other, higher-margin SKUs from the same style.

8.3.9 Next Steps

The SKU-level discount recommendations are typically reviewed by planners or merchandisers and then pushed to inventory management and pricing systems for execution. Ongoing monitoring of SKU performance is crucial to assess the effectiveness of these granular discounts.

8.3.10 Common Questions

1. **How does this handle size curve optimization?:** The module compares actual inventory distribution against the Ideal Size Set (ISS) and recommends discounts for sizes that deviate significantly (e.g., too many large sizes).
2. **Can we set minimum margin floors at the SKU level?:** Yes, the module can be configured with minimum margin thresholds to ensure discounts do not lead to unprofitable sales.
3. **What if a SKU is unique and has no comparative data?:** For unique SKUs, the module might fall back to style-level rules or flag for manual review, depending on configuration.
4. **Does it consider localized SKU performance?:** Yes, if store-specific SKU data is provided, it can generate recommendations tailored to individual store needs.
5. **How often are SKU discounts updated?:** This depends on the run frequency of the pipeline, typically daily or weekly, to respond quickly to inventory changes.

8.3.11 Quick Reference

- **Purpose:** Calculate SKU-level discount recommendations.
- **Inputs:** Prepared SKU data (ROS, DOH, ISS).
- **Outputs:** Discount % per SKU, optimized inventory.
- **Key Metrics:** ROS, DOH, ISS adherence.

9. Appendix

Full Parameter Reference

Refer to the `args.properties` file for a complete and up-to-date list of all configuration parameters. Each parameter includes inline comments detailing its purpose, default value, and valid range.

Version History

- **Version 1.0 (2025-05-23):** Initial release of the dynamicDiscounting pipeline, including `DDDataPrepModule`, `DiscountComputationModule`, and `DiscountSkuComputationModule`. Focus on core dynamic discounting logic at style and SKU levels.

Contact & Support Info

For any questions, issues, or feature requests, please contact the development team: -

Email: support@yourcompany.com - **Internal Support Portal:** [Link to Internal Support Portal] - **Team Channel:** #dynamic-discounting-support (on Slack/Teams)

Tutorial: dynamicDiscounting

The DDDataPrepModule serves as the initial data pipeline, preparing and normalizing all input data required for dynamic discounting. Both the DiscountComputationModule and DiscountSkuComputationModule then independently consume this prepared data to perform their respective discount calculations, at the style or SKU level, based on the configured discount level. They act as alternative computation paths, each relying on the foundational data provided by the preparation module.

flowchart TD

```
A0["DDDataPrepModule"]
A1["DiscountComputationModule"]
A2["DiscountSkuComputationModule"]
A0 -- "prepares" --> A1
A0 -- "prepares" --> A2
```

Chapters

Master Report: dynamicDiscounting

1. Title & Metadata

- **Project Name:** dynamicDiscounting
- **Version:** 1.0
- **Last Updated:** 2023-10-27
- **One-line Elevator Pitch:** An algorithmic solution designed to optimize product pricing by dynamically recommending optimal discounts, thereby maximizing sales, inventory turnover, and overall profitability.

2. Executive Summary

The dynamicDiscounting project introduces an automated merchandising algorithm aimed at enhancing pricing strategies across our product catalog. By leveraging real-time sales, inventory, and product attribute data, the system identifies underperforming SKUs and categories, and proactively recommends precise discount percentages. This ensures optimal stock levels, accelerates sales of slow-moving inventory, and prevents stockouts on popular items, ultimately improving key merchandising metrics and financial performance.

- **Key outcomes or KPIs:**
 - Increased Rate of Sale (ROS) for targeted SKUs.
 - Reduced inventory holding costs and obsolescence.
 - Improved gross margin realization through optimized markdown timing.
 - Better adherence to Optimum Depth (OD) and Optimum Width (OW) targets.
 - Enhanced stock efficiency and availability for Ideal Size Set (ISS).
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Planning Teams, E-commerce Operations, Finance Department, Inventory Management.

3. Architecture & Data Flow

graph TD
A[Raw Data Ingestion: Sales, Inventory, Product Master] --> B[Data Validation & Pre-processing]
B --> C[Merchandising Metric Calculation]
C --> D[Dynamic Discounting Engine]
D --> E[Impact Simulation & Reporting]
E --> F[Discount Recommendation Output]
F --> G[Merchandising / Pricing Systems]
F --> H[Analytics & Reporting Dashboard]

```

subgraph Data Sources
    DS1[Sales Transactions]
    DS2[Current Inventory]
    DS3[Product Catalog/Attributes]
    DS4[Historical Pricing & Promotions]
end

DS1 --> A
DS2 --> A
DS3 --> A
DS4 --> A

H --> MerchTeam[Merchandising Team]
G --> MerchTeam
G --> PricingSys[Pricing System]

```

- **Component Bullets:**

- **Raw Data Ingestion:** Gathers transactional, inventory, and product master data from various internal systems.
- **Data Validation & Pre-processing:** Cleans, validates, and transforms raw data into a consistent format suitable for analysis.
- **Merchandising Metric Calculation:** Computes key performance indicators such as ROS, OD, OW, and analyzes ISS for each SKU and category.
- **Dynamic Discounting Engine:** The core algorithm that processes calculated metrics, applies business rules and optimization models to generate discount recommendations.
- **Impact Simulation & Reporting:** Estimates the financial and operational impact of recommended discounts and generates comprehensive reports.
- **Discount Recommendation Output:** Produces structured data files containing the final discount recommendations.
- **Merchandising / Pricing Systems Integration:** Integrates the recommended discounts directly into existing pricing or merchandising platforms for execution.
- **Analytics & Reporting Dashboard:** Provides interactive dashboards for stakeholders to monitor performance and insights.

- **High-level data sources:**

- Point-of-Sale (POS) data (historical sales, returns)
- Inventory Management Systems (current stock levels, inbound shipments, locations)
- Product Information Management (PIM) system (SKU details, AGs, cost, retail price, category hierarchies)
- Warehouse Management Systems (WMS)
- Historical promotional data and price changes

4. Configuration & Data Dictionary

Settings Table (args.properties)

Setting Parameter	Description	Data Type	Default Value	Example Value
min_discount_pct	Minimum allowable discount percentage.	Double	0.05	0.05 (5%)
max_discount_pct	Maximum allowable discount percentage.	Double	0.50	0.50 (50%)
ros_slow_threshold	ROS (units/ week) below which an item is considered slow-moving.	Integer	3	3
inventory_age_threshold_days	Number of days an item has been in stock before considered for discount.	Integer	90	90
inventory_excess_multiplier	Multiplier for current inventory vs. average sales to flag excess.	Double	1.5	1.5 (150% of avg sales)
od_target_deviation_pct	Percentage deviation from target OD to trigger attention.	Double	0.20	0.20 (20%)
ow_target_deviation_pct	Percentage deviation from target OW to trigger attention.	Double	0.15	0.15 (15%)
discount_step_pct	Granularity of discount	Double	0.05	0.05


```

    "predicted_gross_profit_impact": "DECIMAL" // Output: Predicted
    gross profit change
}

```

Outputs Catalog

Output File/Stream	Description	Format	Frequency	Consumers
sku_discount_recommendations.csv	List of SKUs with recommended discounts.	CSV	Daily	Merchandising, Pricing Systems
category_discount_summary.json	Aggregated discount insights per category.	JSON	Daily	Merchandising, Analytics
performance_dashboard_feed.json	Data feed for BI dashboards.	JSON	Daily	Analytics Dashboards
audit_log_dynamic_discounting.log	Detailed log of algorithm execution and decisions.	LOG	Continuous	Development, Operations

5. End-to-End Flow

The dynamicDiscounting algorithm operates through a robust, automated pipeline:

- Data Ingestion:** Raw data concerning sales transactions, current inventory levels, and product master information (including product attributes and costs) is ingested from various internal systems into a central data lake.
- Data Pre-processing and Validation:** The ingested raw data undergoes rigorous cleaning, validation, and transformation. This step ensures data quality, consistency, and adherence to defined schemas, preparing it for analytical processing. Missing values are imputed, outliers handled, and data types standardized.
- Merchandising Metric Calculation:** The pre-processed data is then used to compute critical merchandising metrics at both SKU and category levels. This includes ROS (Rate of Sale) over various periods, current inventory age, actual OD (Optimum Depth) and OW (Optimum Width) compared to targets, and analysis of ISS (Ideal Size Set) performance.
- Discount Recommendation Engine:** The heart of the system, this module analyzes the calculated merchandising metrics against predefined business rules, configuration settings, and predictive models. For each SKU, it determines if a discount is warranted, and if so, calculates the optimal discount percentage. Factors considered include ROS thresholds, inventory excess, inventory age, current profitability, and alignment with OD/OW targets.
- Impact Simulation:** Before final recommendations are generated, a simulation layer estimates the potential impact of the proposed discounts on sales volume, revenue, and gross profit. This provides a data-driven forecast of outcomes.

6. **Reporting & Output Generation:** The final, validated discount recommendations and their predicted impacts are compiled into structured outputs. These outputs include detailed SKU-level CSVs for operational systems and aggregated JSON summaries for analytical purposes.
7. **System Integration & Execution:** The generated discount recommendations are automatically pushed to downstream merchandising and pricing systems, or presented via dashboards for manual review and approval, enabling the execution of dynamic pricing changes.
8. **Monitoring & Feedback Loop:** Performance of discounted items is continuously monitored. Sales data from these items feeds back into the system's data ingestion layer, allowing the algorithm to learn and refine its recommendations over time, ensuring continuous optimization.

6. Submodule Overview

6.1. Data Ingestion & Validation

1. **Name & Purpose:** Gathers raw sales, inventory, and product master data from various sources, and performs initial validation and cleaning.
2. **Inputs & Outputs:**
 - **Inputs:** Raw sales data (`sales_txns.csv`), Inventory snapshots (`inventory_current.csv`), Product master data (`product_master.json`).
 - **Outputs:** Cleaned, validated, and normalized data (`clean_data.parquet`).
3. **Core Logic Summary:**
 - Connects to specified data sources (e.g., databases, APIs, flat files).
 - Applies schema validation to ensure data conforms to expected types and structures.
 - Performs data cleansing: handles missing values (imputation/removal), corrects data format inconsistencies, and removes duplicates.
 - Normalizes attribute names and values for consistency across different data sources.

6.2. Merchandising Metric Calculation

1. **Name & Purpose:** Computes key merchandising performance indicators for each SKU and category.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned, validated data (`clean_data.parquet`).
 - **Outputs:** Data enriched with calculated metrics (`metrics_data.parquet`).
3. **Core Logic Summary:**
 - Calculates ROS (Rate of Sale) for each SKU over defined periods (e.g., 7-day, 30-day rolling averages).
 - Determines current inventory age and weeks of supply for each SKU.
 - Computes actual OD (Optimum Depth) for each category based on unique SKUs stocked and compares it against target OD.
 - Calculates actual OW (Optimum Width) for each store/display based on unique categories offered and compares it against target OW.
 - Evaluates ISS (Ideal Size Set) completeness and performance by analyzing sales distribution across sizes for each product style.

6.3. Discount Recommendation Engine

1. **Name & Purpose:** Applies business rules and optimization logic to recommend optimal discount percentages for SKUs.
2. **Inputs & Outputs:**
 - **Inputs:** Data enriched with calculated metrics (`metrics_data.parquet`), Configuration settings (`args.properties`).
 - **Outputs:** SKUs with proposed discount percentages (`discount_proposals.json`).
3. **Core Logic Summary:**
 - Identifies SKUs flagged for attention based on conditions: ROS below `ros_slow_threshold`, inventory exceeding `inventory_excess_multiplier`, or `days_since_last_sale` above threshold.
 - For flagged SKUs, it calculates a preliminary discount based on the severity of the issue (e.g., lower ROS = higher initial discount).
 - Adjusts discounts to respect `min_discount_pct`, `max_discount_pct`, and `profit_margin_floor_pct` configurations.
 - Considers category-level OD/OW deviations and ISS issues to fine-tune recommendations (e.g., deeper discounts if category OD is too high and product is not selling).

6.4. Impact Simulation & Reporting

1. **Name & Purpose:** Estimates the financial and operational impact of proposed discounts and generates actionable reports.
2. **Inputs & Outputs:**
 - **Inputs:** SKUs with proposed discount percentages (`discount_proposals.json`), Historical sales elasticity data (internal model data).
 - **Outputs:** Final discount recommendations with predicted impact (`sku_discount_recommendations.csv`), Summary reports (`category_discount_summary.json`).
3. **Core Logic Summary:**
 - Utilizes a sales elasticity model to predict the uplift in unit sales for each recommended discount.
 - Calculates the estimated change in revenue and gross profit based on predicted sales uplift and new discounted prices.
 - Aggregates SKU-level impacts to provide category and overall business level summaries.
 - Formats the final recommendations and impact analyses into user-friendly reports and structured data for downstream systems.

7. Worked Example

Scenario: A “Men’s Casual T-Shirt, Navy Blue, Size L” (SKU: MC-TNAVY-L) in a specific category (e.g., “Men’s Tops”) and store.

1. Initial State (Inputs from Merchandising Metric Calculation): * SKU: MC-TNAVY-L * **Current Retail Price:** \$25.00 * **Cost Price:** \$10.00 * **Current Inventory Level:** 200 units (high for a single SKU) * **ROS (last 7 days):** 2 units/week (low) * **ROS (last 30 days):** 1.8 units/week * **Days Since Last Sale:** 15 days *

Category: “Men’s Tops” * **Category Actual OD:** 150 unique SKUs (Target OD: 100) -> *High OD* * **Store Actual OW:** 25 categories (Target OW: 20) -> *High OW* * **ISS Deviation:** For “Men’s Casual T-Shirt” product (AG: Color, Size), “L” is overstocked while “S” and “M” are out of stock.

2. Configuration Settings (from args.properties): * **ros_slow_threshold:** 3 units/week * **inventory_excess_multiplier:** 1.5 * **days_since_last_sale_threshold:** 7 days * **min_discount_pct:** 0.05 (5%) * **max_discount_pct:** 0.50 (50%) * **profit_margin_floor_pct:** 0.10 (10%)

3. Discount Recommendation Engine Logic: * **Check 1: ROS Threshold:** Current ROS (2 units/week) is less than **ros_slow_threshold** (3 units/week). **FLAGGED.** * **Check 2: Inventory Excess:** Current inventory (200) divided by ROS (2) gives 100 weeks of supply. This is significantly higher than what 1.5 multiplier would imply for avg sales. **FLAGGED.** * **Check 3: Days Since Last Sale:** 15 days is greater than **days_since_last_sale_threshold** (7 days). **FLAGGED.** * **Initial Discount Proposal:** Based on these flags, the engine proposes an initial discount, perhaps 20% to accelerate sales. * **New Price:** \$25.00 * $(1 - 0.20) = \$20.00$ * **Profit Margin:** $((\$20.00 - \$10.00) / \$20.00) = 50\%$. This is > **profit_margin_floor_pct** (10%). **OK.** * **Adjustment based on OD/OW/ISS:** * **Category OD** is too high. This item contributes to the excess depth. This suggests a deeper discount might be beneficial to clear space. * **Store OW** is also high, indicating too many categories. While this is an SKU, category-level issues can influence SKU decisions. * **ISS deviation:** “L” is overstocked. This further justifies a discount on this specific size to balance inventory. * **Revised Discount Proposal:** Considering the additional context (high OD, OW, and ISS imbalance), the engine may increase the discount to 25% or 30%. Let’s say it recommends **28%**. * **New Price:** \$25.00 * $(1 - 0.28) = \$18.00$ * **Profit Margin:** $((\$18.00 - \$10.00) / \$18.00) = 44.4\%$. Still > 10%. **OK.**

4. Impact Simulation & Reporting: * **Elasticity Model Input:** For a 28% discount on this type of item, the model predicts a sales uplift. * **Predicted Sales Uplift:** Assume a 50% increase in ROS from 2 units/week to 3 units/week immediately after discount. Or, a total sale of 30 units over the next 7 days instead of original 14 units. Let’s use the latter for impact: Predicted 30 units sold in 7 days. * **Predicted Gross Profit Impact:** * **Original:** 14 units * $(\$25 - \$10) = \$210$ (per week) * **Discounted:** 30 units * $(\$18 - \$10) = \$240$ (per week) * **Predicted Gross Profit Impact:** +\$30 per week initially.

5. Output (example sku_discount_recommendations.csv entry):

sku_id	product_name	current_retail_price	recommended_discount_percentage	new_retail_price
MC-TNAVY-L	Men’s Casual T-Shirt, Navy, L	25.00	0.28	18.00

This example demonstrates how the algorithm uses various data points and configuration to arrive at a data-driven discount recommendation, along with its predicted impact.

8. Debug Guide Highlights

- **No Recommendations Generated:**
 - **Check:** Verify `args.properties` for overly restrictive `min_discount_pct` or `profit_margin_floor_pct`.
 - **Check:** Ensure input data for ROS, Inventory, and pricing is not null or zero for relevant SKUs.
 - **Check:** Review logs for “No eligible SKUs found” messages or errors in Merchandising Metric Calculation.
- **Unexpectedly High/Low Discounts:**
 - **Check:** Review `ros_slow_threshold`, `inventory_excess_multiplier`, `min_discount_pct`, `max_discount_pct` in `args.properties`.
 - **Check:** Examine the `metrics_data.parquet` output from Merchandising Metric Calculation to ensure ROS, inventory, OD, OW calculations are accurate for the SKU in question.
 - **Check:** Verify the `profit_margin_floor_pct` is correctly applied in the Discount Recommendation Engine.
- **Performance Issues / Slow Run-time:**
 - **Check:** Data volume in Data Ingestion & Validation. Consider batch processing or incremental updates.
 - **Check:** Database query performance for input data sources.
 - **Check:** Resource allocation (CPU/Memory) for the processing environment.
- **Data Mismatch/Inconsistency:**
 - **Check:** Compare raw input data with `clean_data.parquet` to ensure pre-processing isn't introducing errors.
 - **Check:** Validate primary keys (`sku_id`, `category_id`) are consistent across all input sources.
 - **Check:** Logs for data type conversion errors or schema validation failures.

9. Common Pitfalls & Remedies

- **Pitfall: Stale or Incomplete Input Data:** If sales or inventory data isn't fresh or is missing key fields, recommendations will be inaccurate.
 - **Remedy:** Implement robust data ingestion pipelines with clear SLAs and automated alerts for data delays or failures. Ensure comprehensive data validation at the ingestion stage.
- **Pitfall: Over-discounting Profitable or Fast-Selling Items:** The algorithm might recommend discounts on items that are already performing well or have high margins, eroding profit.
 - **Remedy:** Strengthen the `profit_margin_floor_pct` configuration and integrate more sophisticated elasticity models. Add rules to exclude items above a certain ROS threshold or below a ‘weeks of supply’ threshold from discount consideration.
- **Pitfall: Discounting Items with Critical Low Inventory:** Driving sales on already low-stock items could lead to stockouts and missed full-price sales.
 - **Remedy:** Implement a strict inventory threshold in `args.properties` (e.g., `min_inventory_for_discount`) that prevents any discount recommendations if stock falls below a safe level. Prioritize ISS management to prevent imbalances.

- **Pitfall: Recommendations Not Actioned or Understood by Merchandising:**
If the outputs are not user-friendly or lack context, adoption will be low.
 - **Remedy:** Design clear, scannable reports and dashboards. Provide explanations for recommendations (e.g., “Discount recommended due to high inventory and low ROS”). Offer training and clear communication channels with stakeholders.
- **Pitfall: Seasonality or Promotional Cannibalization Not Accounted For:**
Discounts might clash with planned promotions or ignore natural seasonal sales cycles.
 - **Remedy:** Integrate a promotional calendar as an input to the Discount Recommendation Engine to exclude or adjust recommendations during planned campaigns. Incorporate historical seasonality trends into ROS calculations or future demand forecasts.

10. FAQ

- **Q: How often are discount recommendations updated?**
 - A: The algorithm runs daily, generating new recommendations based on the latest sales and inventory data. Critical adjustments can be made more frequently if needed, by triggering a manual run.
- **Q: Can merchandising teams override the algorithm’s recommendations?**
 - A: Yes, the system provides recommendations, but final approval and execution often reside with the merchandising team. Integration points allow for manual review and override before pushing to live pricing systems.
- **Q: What if the data is incomplete or has errors?**
 - A: The Data Ingestion & Validation submodule is designed to catch common data quality issues. If critical data is missing or erroneous, the system will flag it, and in some cases, defer recommendations for affected SKUs until data quality is restored.
- **Q: How does the algorithm account for new product launches or end-of-life products?**
 - A: New products are typically excluded from dynamic discounting until they establish a ROS history (e.g., after 30-60 days). For end-of-life products, specific rules can be configured to aggressively clear inventory, overriding standard profitability constraints.
- **Q: Is there an A/B testing capability for different discount strategies?**
 - A: While not a core feature of version 1.0, the Impact Simulation module lays the groundwork. Future versions plan to incorporate controlled A/B testing frameworks to validate different algorithmic parameters or discount strategies on a subset of SKUs.

11. Quick Reference Cheat-Sheet

Term / Component	Definition / Purpose	Key Metric / Output	Re (a
OD	Optimum Depth: Ideal number of unique SKUs in a category.	category_od_actual, category_od_target	od.
OW	Optimum Width: Ideal number of different categories/styles.	store_ow_actual, store_ow_target	ow.

Term / Component	Definition / Purpose	Key Metric / Output	Relevance (a)
ISS	Ideal Size Set: Best combination of sizes for sales/stock.	iss_deviation	(In set
ROS	Rate of Sale: Speed at which products are sold (units/week).	ros_last_7_days, ros_last_30_days	ro
AG	Attribute Group: Set of product attributes (e.g., color, size).	Used for grouping analysis of OD, OW, ISS.	(N
Core Logic	Dynamic Discounting Engine.	recommended_discount_percentage	mi ma pr
Key Output	sku_discount_recommendations.csv	SKU ID, Discount %, New Price, Predicted Sales Uplift	
Data Source	Sales, Inventory, Product Master Data		
Contact	Merchandising Analytics Team		

Chapter 1: EOSS Orchestration

Last Updated: 2025-05-23

1. Business Objective

EOSS Orchestration acts as the central conductor for the entire End-of-Season Sale (EOSS) distribution process, ensuring all steps run in the correct order to optimize inventory across stores and warehouses.

Why This Matters: This ensures that every product, from high-performing items to those needing clearance, is managed effectively, reducing excess stock, preventing lost sales from stockouts, and maximizing revenue during critical end-of-season periods. It creates a smooth and predictable workflow for your merchandising and operations teams.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records:** Why we need it: To understand past demand patterns and product performance (e.g., Units Sold, ROS) for specific items in each store.
- **Current Sales Trends:** Why we need it: To identify fast-moving or slow-moving items right now, helping to make real-time allocation decisions.

Product Data

- **Product Attributes:** Why we need it: Details like color, size, material, and category (AG) are crucial for grouping products for analysis and defining Ideal Size Sets (ISS).
- **Current Inventory Levels:** Why we need it: Knowing exactly how many units of each product are in every store and warehouse (OD) is essential for effective allocation and transfers.

Operational Data

- **Store and Warehouse Locations:** Why we need it: To plan logistics efficiently and determine optimal paths for inter-store transfers.
- **Cost and Retail Prices:** Why we need it: To calculate potential revenue and identify the financial impact of different allocation strategies.

3. Business Settings

Setting Name	Default Value	Description	When to Change
EOSS Run Frequency	Daily	How often the entire EOSS distribution process is initiated and completed.	Change to weekly or bi-weekly if inventory changes are less frequent, or to intra-day if real-time stock adjustments are critical.
Inter-Store Transfer Priority	Balanced	Defines whether store-to-store transfers are highly prioritized or done only when warehouse stock is exhausted.	Adjust to “High” if rapid inventory movement between stores is crucial to clear end-of-season stock, or “Low” if warehouse-to-store is preferred.
Minimum Revenue Threshold for Allocation	\$50	The lowest revenue a product must generate to be considered for detailed allocation and transfer plans.	Increase this value if you want to focus allocation efforts only on higher-value products, ignoring very low-performing items.
Stockout Prevention Buffer	15%	An additional percentage of stock to keep on hand beyond predicted demand to prevent unexpected stockouts.	Increase during periods of high demand or uncertain supply, decrease during periods of overstocking or low demand to optimize inventory investment.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Raw Sales & Inventory Data] --> B{EOSS Orchestration}
B --> C[Data Preparation]
C --> D[Sales & Revenue Processing]
D --> E[Segmentation & Ranking]
E --> F[Allocation & Transfers]
```

F --> G[Final EOSS Distribution Plan]

G --> H[Output Synchronization]

1. Start with → Capture current inventory and sales data from all stores and warehouses.
2. Then → Prepare and process this raw data, calculate sales figures, and determine revenue.
3. Next → Segment products, rank them by performance, and calculate optimal stock levels for each location.
4. Finally → Allocate inventory, plan inter-store transfers, and generate the final distribution plan, which is then synchronized with operational systems.

5. Step-by-Step Actions

1. **Capture** the current state of inventory and sales
 - What we do: *We take a snapshot of all relevant data points—current stock levels, recent sales, and warehouse inventory—before any changes are made.*
 - Why we do it: *This provides a baseline and ensures that all subsequent calculations are based on up-to-date information, preventing errors.*
2. **Prepare and Process** raw data
 - What we do: *We clean the incoming sales and inventory data, making sure it's accurate and ready for analysis, then compute sales rates and revenue.*
 - Why we do it: *Clean data is essential for reliable calculations. Understanding Units Sold and revenue helps us assess product performance objectively.*
3. **Segment and Rank** products
 - What we do: *We group products based on their attributes (AG) and performance, then assign a priority score to each, often considering its ROS and profitability.*
 - Why we do it: *This helps us focus our efforts on the most impactful products and decide which items need immediate attention for optimal depth (OD) and width (OW).*
4. **Allocate and Transfer** inventory
 - What we do: *We determine the Ideal Size Set (ISS) and quantity of each product for every store, then identify opportunities to move stock between stores or from warehouses to balance inventory.*
 - Why we do it: *This minimizes stockouts in high-demand stores and clears excess stock from others, optimizing overall inventory investment.*
5. **Generate and Synchronize** output
 - What we do: *We compile all the allocation decisions and transfer orders into a clear, actionable plan and send it to the relevant operational systems.*
 - Why we do it: *This ensures that the merchandising decisions are promptly implemented in stores and warehouses, making sure the right products are in the right place at the right time.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Blue Denim Jacket” is highly desired in Store A (sold 150 units in the last month, current stock is 5 units).
- The same “Blue Denim Jacket” has excess stock in Store B (sold 20 units in the last month, current stock is 80 units).
- The central warehouse has 100 units of this jacket, but it’s expensive to ship individual items to stores from there.
- End-of-season is approaching, and we want to clear excess stock while meeting demand.

What Happens

1. First, the EOSS Orchestration module initiates the process by capturing the current inventory and sales data for all stores.
2. Then, it triggers the data preparation and sales processing modules. These confirm Store A’s high ROS for the jacket and Store B’s low ROS and high stock.
3. Next, the segmentation and ranking modules identify the “Blue Denim Jacket” as a high-priority item (due to its overall appeal) but highlight the imbalance between Store A and Store B.
4. Finally, the allocation and inter-store transfer modules are activated. They determine that Store B has an ideal opportunity to transfer 50 units of the “Blue Denim Jacket” to Store A. This is a more cost-effective solution than shipping from the warehouse.
5. The orchestration completes the process by generating the transfer order and synchronizing it with the store’s inventory system.

Final Outcome

Store A receives 50 additional “Blue Denim Jackets” just in time for peak demand, preventing lost sales and improving customer satisfaction. Store B reduces its excess inventory, minimizing the need for deep markdowns. Overall, inventory is balanced, and profitability for this item is maximized.

Key Takeaway: Effective EOSS Orchestration ensures that inventory imbalances are identified and corrected efficiently, leading to higher sales and reduced losses.

7. Results

This step produces:

Main Outputs

- **Complete EOSS Distribution Plan:** What it tells us: A comprehensive report detailing all recommended inventory movements, including allocations to stores and inter-store transfer orders, ensuring optimized depth (OD) and width (OW) across the chain.
- **Execution Sequence Log:** What it tells us: A record of all modules run, their order, and their completion status, providing transparency into the process flow.

Business Reports

- **Inventory Optimization Summary:** Who uses it and why: Merchandising Planners use this to understand the overall effectiveness of the EOSS strategy in balancing stock and maximizing sell-through rates.
- **Transfer Order List:** Who uses it and why: Store Operations and Logistics teams use this to execute the recommended inter-store transfers, ensuring products move efficiently.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The orchestration process will pause or fail to initiate, as it cannot proceed without complete and accurate input data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
A sub-process fails (e.g., Ranking Module)	The orchestration process will stop, and an error notification will be sent, indicating which specific module failed.	Review the error logs for the failed module, address the underlying issue (e.g., data quality, configuration), and re-run the entire EOSS orchestration.
No viable inter-store transfers identified	The transfer module will complete without recommending any transfers, even if some stores have excess stock.	Review the transfer criteria (e.g., minimum transfer quantity, cost thresholds) to ensure they are not too restrictive. It might indicate that all excess can be absorbed by the warehouse or kept.
Output synchronization issues	The final distribution plan is generated but fails to upload or integrate with the operational systems (e.g., POS, WMS).	Check connectivity and integration points with the target systems. Manually transfer the output if necessary, and alert the IT team for a permanent fix.

9. Next Steps

Before This Step

- **Sales Data Collection:** What we need from it: Accurate, real-time sales transactions (Units Sold) from all points of sale for a defined Analysis Period.
- **Inventory Data Collection:** What we need from it: Precise, up-to-date inventory counts for every product at every store and warehouse location.

After This Step

- **Execute Inter-Store Transfers:** What they need from us: Detailed transfer orders specifying which products to move between stores and in what quantities.
- **Update Store Inventory Systems:** What they need from us: The final allocation plan to adjust on-hand inventory figures in store systems, reflecting the new stock levels.

10. Common Questions

Q: Why can't we just run each step manually as needed? A: Running each step manually is time-consuming and prone to human error. EOSS Orchestration automates the entire sequence, ensuring consistency, speed, and accuracy, which is vital for quick decisions during end-of-season sales.

Q: How does this orchestration improve our profitability? A: By ensuring the right products are in the right stores at the right time, it reduces the need for heavy markdowns on overstocked items and prevents lost sales due to stockouts. This directly boosts your profit margins.

Q: What happens if the data going into the orchestration process is bad? A: If the initial data is inaccurate, the subsequent steps—like sales calculations, ranking, and allocation—will also be flawed. The system is designed to identify and flag major data issues during preparation, but quality input is always key.

Q: Can we customize the order of modules within the orchestration? A: No, the sequence of modules (e.g., data preparation, then sales processing, then ranking, then allocation) is carefully designed and fixed to ensure optimal and logical flow for the EOSS distribution process. Changing the order would likely lead to incorrect outcomes.

Q: How quickly does the EOSS Orchestration complete a full cycle? A: The full cycle time depends on the volume of data and the complexity of the calculations, but it's optimized for efficiency. Typically, it can complete within a few hours, allowing for timely adjustments to inventory based on the chosen EOSS Run Frequency.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Ensures the entire End-of-Season Sale (EOSS) inventory distribution process runs smoothly and in the correct sequence.
 - **Main Input:** Raw sales, product, and inventory data from all stores and warehouses.
 - **Main Output:** A complete and actionable EOSS distribution plan, including allocation decisions and inter-store transfer orders.
 - **Warning Signs:** Major delays in processing, repeated failures of individual modules, or output not syncing with operational systems.
 - **Key Settings:** EOSS Run Frequency and Inter-Store Transfer Priority.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 2: EOSS Data Preparation

Last Updated: 2025-05-23

1. Business Objective

EOSS Data Preparation is responsible for gathering, validating, and structuring all essential master and transactional information required for the End-of-Season Sale (EOSS) distribution process.

Why This Matters: This foundational step ensures that every subsequent decision, from determining Optimal Depth (OD) to planning inter-store transfers, is based on accurate, complete, and current inventory and product data. It prevents errors, reduces the risk of stockouts or overstocking, and maximizes the effectiveness of your EOSS strategy by having the right information at your fingertips.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Performance Indicators:** Why we need it: To understand past demand patterns and product performance, feeding into insights like Ideal Size Set (ISS) composition and Optimal Depth (OD) calculations.
- **Current Sales Velocity (ROS):** Why we need it: To assess real-time product movement and identify fast or slow sellers, informing immediate allocation needs.

Product Data

- **Product Attributes (AG):** Why we need it: Details like color, size, material, and category are crucial for grouping products for analysis and defining Ideal Size Sets (ISS) and Optimal Depth (OD).
- **Current Inventory Levels:** Why we need it: Knowing exactly how many units of each product are in every store (OD) and warehouse is essential for effective allocation and transfers.

Operational Data

- **Store Configurations:** Why we need it: To identify which stores are active participants in the EOSS distribution, their specific roles, and any linked warehouses.

- **Current Goods-in-Transit & Open Customer Orders:** Why we need it: To accurately account for stock that is currently being shipped or is already reserved for customers, providing a true picture of available inventory.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Include Store On-Hand Inventory	True	Determines if current physical stock in stores is included in the EOSS calculations.	Set to 'False' only if you want to perform calculations based solely on warehouse or incoming stock.
Include In-Transit & Open Orders	True	Specifies whether inventory currently being shipped between locations or reserved for customer orders is factored into available stock.	Set to 'False' if you prefer to only work with physically present, unallocated stock.
Include Warehouse Inventory	True	Controls whether stock held in central warehouses is considered for EOSS distribution.	Set to 'False' if EOSS is purely focused on store-to-store balancing.
Seasons for EOSS Analysis	Current, Previous	Defines which product seasons (e.g., 'Spring 2025', 'Fall 2024') should be considered for EOSS activities.	Adjust to include or exclude specific seasons based on your clearance strategy (e.g., adding older seasons).

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw Sales & Inventory Data & Master Configurations] -->
    B[Validate & Cleanse Data]
    B --> C[Load Master & Reference Data]
  
```

C --> D[Integrate Transactional & Derived Data]

D --> E[Ready-for-Analysis EOSS Data Set]

1. Start with → Capture all raw sales and inventory data, along with existing master configurations.
2. Then → Validate and **cleanse** this raw data, ensuring its accuracy and consistency.
3. Next → **Load** foundational master data, including store setups, product attributes, and planograms.
4. Finally → **Integrate** all transactional inventory data (on-hand, in-transit, open orders) and derived insights (Ideal Size Set parameters, Optimal Depth recommendations), producing a complete and validated data set for EOSS analysis.

5. Step-by-Step Actions

1. **Validate** and **Cleanse** incoming data
 - What we do: *We ensure accuracy and consistency of all raw data received, removing errors or inconsistencies and standardizing formats.*
 - Why we do it: *Clean data is the foundation for reliable analysis and prevents incorrect distribution decisions that could lead to lost sales or excess inventory.*
2. **Load** foundational master data
 - What we do: *We bring in essential static information like active distribution stores, style definitions, product categories (AG), style pack sizes, replenishment cycles, and store/warehouse planograms.*
 - Why we do it: *This establishes the structural framework for inventory management, defining where products can be stocked, their characteristics, and their operational movement rules.*
3. **Integrate** transactional inventory data
 - What we do: *We pull in real-time inventory counts from all stores and warehouses, including products that are currently in transit (Goods-in-Transit) or allocated to open customer orders.*
 - Why we do it: *This provides a complete and accurate picture of all* available stock across the entire network, preventing over-allocation of unavailable items or missed opportunities to fulfill existing customer demands.**
4. **Determine** seasonal relevance and ideal stock parameters
 - What we do: *We identify which styles are considered in-season versus out-of-season for the current EOSS period, and load pre-calculated Ideal Size Sets (ISS) and Optimal Depth (OD) parameters for different product groups.*
 - Why we do it: *This ensures that EOSS efforts efficiently focus on clearing relevant out-of-season stock while optimally managing current season items, balancing the number of unique items (OD) and the best combination of sizes (ISS) to maximize sales.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new end-of-season period is beginning.
- A “Summer Floral Dress” is designated as an out-of-season style. Store A has 100 units of this dress, but 10 units are already en route to a customer as part of an existing online order.
- A “Winter Wool Coat” is designated as an in-season style. Store B has 50 units of this coat, and an additional 15 units just shipped from the central warehouse (goods-in-transit) and are expected soon.
- The standard “Style Pack Size” for dresses is 3 units, and for coats is 2 units. Ideal Size Set (ISS) data for both dresses and coats has been previously calculated.

What Happens

1. First, EOSS Data Preparation **validates** all incoming stock data and configurations, ensuring accuracy and consistency.
2. Then, it **loads** master data, identifying the “Summer Floral Dress” as an out-of-season style and the “Winter Wool Coat” as in-season. It registers the respective pack sizes and the specific configurations for Store A and Store B.
3. Next, it **integrates** the transactional inventory: Store A’s 100 dresses are recorded, but the 10 units for open orders are flagged as currently unavailable for new allocations. Store B’s 50 coats are noted, and the 15 units currently in transit are added to its future available stock, ensuring they are considered for distribution decisions.
4. Finally, it loads the pre-calculated Ideal Size Set (ISS) data for both dress and coat categories, along with the Optimal Depth (OD) recommendations for relevant SKUs, making them immediately available for the subsequent allocation steps.

Final Outcome

The system now has a complete and accurate picture of *all* available inventory for *each* style (considering seasonality, in-transit items, and open orders), along with the foundational rules like pack sizes and ideal stock levels. This allows subsequent modules to make precise, informed decisions about transfers and allocations, ensuring out-of-season items are cleared efficiently while in-season items are well-stocked for ongoing demand.

Key Takeaway: Thorough data preparation ensures that all inventory, regardless of its current state or seasonal relevance, is accurately accounted for, enabling intelligent and profitable EOSS distribution.

7. Results

This step produces:

Main Outputs

- **Comprehensive EOSS Data Set:** What it tells us: All master and transactional data (store configurations, current inventory levels, style characteristics,

replenishment cycles, Ideal Size Set data, Optimal Depth recommendations) organized, validated, and prepared for subsequent EOSS processes.

- **Seasonal Style Categorization:** What it tells us: A definitive list of styles identified as in-season or out-of-season for the current EOSS run, which is critical for determining targeted clearance strategies.

Business Reports

- **Inventory Data Readiness Report:** Who uses it and why: Data analysts and IT teams use this to confirm that all necessary data has been successfully loaded and is ready for the EOSS distribution run, highlighting any gaps or errors for immediate attention.
- **Current Stock Overview:** Who uses it and why: Merchandising and Inventory Managers use this to get a snapshot of the total inventory picture at the start of the EOSS process, including on-hand stock, in-transit items, and open customer orders, ensuring transparency and alignment across teams.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The data preparation process will pause or fail to initiate, as it cannot proceed without complete and accurate input data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
Incorrect style-pack size configurations	The system might default to an incorrect pack size or fail calculations where pack size is critical, leading to inefficient transfers or allocations (e.g., trying to move 1.5 units).	Review and correct the style-pack size master data. Implement data validation rules to prevent invalid entries.
No active distribution stores identified	The data preparation will complete without any target stores, preventing any EOSS distribution decisions from being made by subsequent modules.	Verify the store configurations in the master data to ensure at least one store is marked as active for distribution.
Inconsistent inventory counts	Allocations or transfers will be based on inaccurate stock levels, potentially leading to stockouts where stock is incorrectly low or overstocking where it's incorrectly high.	Investigate the source of inventory discrepancies (e.g., POS system, warehouse management system). Implement reconciliation processes and rigorous data quality checks.

9. Next Steps

Before This Step

- **Sales Data Collection:** What we need from it: Accurate, real-time transaction data, including Units Sold, prices, and dates from all points of sale, to enable the calculation of product performance and Rate of Sale (ROS).
- **Inventory Data Collection:** What we need from it: Precise, up-to-date inventory counts for every product (SKU) at every store and warehouse location, including comprehensive details on goods in transit and open customer orders.

After This Step

- **Sales & Revenue Processing:** What they need from us: The prepared inventory data combined with master product attributes to calculate detailed sales figures and revenue per SKU and Attribute Group (AG), setting the stage for performance analysis.
- **Segmentation & Ranking:** What they need from us: The fully loaded and integrated data set, including seasonal flags, to categorize products, assign priority scores, and determine Optimal Depth (OD) and Optimal Width (OW) strategies for each location.

10. Common Questions

Q: Why is it crucial to include “goods-in-transit” and “open orders” in this data preparation? A: Ignoring items already in transit or allocated to customer orders would lead to inaccurate inventory counts. This could result in over-allocating stock that isn’t truly available, causing stockouts, or missing opportunities to fulfill existing customer demands.

Q: How does this step ensure we focus on the right products for EOSS? A: By designating styles as “in-season” or “out-of-season” during preparation, the system can prioritize the clearance of older, end-of-season inventory while ensuring current season items are optimally managed for ongoing sales and profitability.

Q: What happens if there are errors in the initial data fed into this process? A: If the raw data is inaccurate (e.g., incorrect inventory counts or missing product attributes), the prepared data will also be flawed. This can lead to incorrect allocation decisions, inefficient transfers, and ultimately, lost sales or increased markdowns. The system attempts to flag major issues but relies heavily on the quality of source data.

Q: How often does this data preparation occur for EOSS? A: The frequency of data preparation is tied to the EOSS Run Frequency setting (e.g., daily). It ensures that the system always works with the most current view of inventory and product information, allowing for timely adjustments to the distribution plan as sales and stock levels change.

Q: Can I manually adjust stock levels or other data after this step runs? A: While the system aims for automation, critical business adjustments should primarily be made in the original source systems (e.g., inventory management system, master data system). These changes would then be automatically reflected in the *next* EOSS data preparation run, maintaining data integrity and consistency throughout the process.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prepares all necessary master and transactional data for the EOSS distribution process.
- **Main Input:** Raw sales figures, comprehensive inventory levels (store, warehouse, in-transit, open orders), and detailed master configurations (store, style, planogram data).
- **Main Output:** A comprehensive, structured, and validated data set, ready for EOSS analysis and allocation.
- **Warning Signs:** Major delays in data loading, reports showing inconsistent inventory counts, or styles incorrectly flagged as in/out-of-season.
- **Key Settings:** Include Store On-Hand Inventory, Include In-Transit & Open Orders, Include Warehouse Inventory, and Seasons for EOSS Analysis.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 3: EOSS Sales Data Preparation

Last Updated: 2025-05-23

1. Business Objective

EOSS Sales Data Preparation focuses on processing raw sales data by cleaning liquidation sales, handling broken sales data, and computing the Rate of Sale (ROS) for individual items (SKUs) over defined periods and weightages.

Why This Matters: This foundational step ensures that all subsequent revenue computations and inventory segmentation are based on accurate and reliable sales performance indicators. By providing a true picture of product velocity, it enables better allocation decisions, minimizes the risk of overstocking or stockouts, and ultimately maximizes revenue during critical end-of-season periods.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Raw Sales Transactions:** Why we need it: To get the fundamental data of what was sold, where, when, and in what quantity (Units Sold) before any processing, serving as the basis for all calculations.
- **Sales Liquidation Flags:** Why we need it: To identify and exclude sales that occurred during special liquidation events, ensuring the calculated Rate of Sale (ROS) reflects regular, organic demand and not discount-driven spikes.

Product Data

- **Product Attributes (SKU, Style, AG):** Why we need it: To correctly group sales data (e.g., by style, color, size) and relate specific SKUs to their broader product categories (AG) for comprehensive analysis and accurate ROS computation.
- **SKU Live Status:** Why we need it: To determine on which specific days an item (SKU) was available for sale in a particular store, which is crucial for accurately calculating Rate of Sale (ROS) over its actual selling period.

Operational Data

- **Store Operating Days:** Why we need it: To understand precisely when stores were open and able to make sales, ensuring sales data is processed against actual selling days, not just calendar days, for a fair ROS calculation.
- **Pivotal SKU Live Percentage:** Why we need it: To identify and exclude “broken sales data” where key sizes or attributes of a product were not sufficiently available, ensuring that sales velocity is only calculated under complete and fair selling conditions.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Sales Liquidation Threshold	0.05	The maximum percentage of total sales from a period that can be identified as liquidation sales before the entire period’s data is flagged for review.	Increase if aggressive discounting is a regular part of your strategy and you still want to include some of those sales in ROS; decrease if you want very pure, non-discounted sales data.
Minimum Live Days for ROS Calculation	7 days	The minimum number of days an SKU must have been available for sale in a store within the analysis period to have its Rate of Sale (ROS) computed.	Increase to ensure more reliable ROS for items with consistent availability; decrease to get ROS for items with shorter or intermittent availability.
Broken Sales Data Threshold	0.20	The percentage of pivotal SKUs (e.g., core sizes, popular colors) that must be in stock on a given day for that day’s sales to be considered “unbroken” and included in ROS calculation.	Increase if you need highly reliable sales data from periods where a full size set (ISS) or key attribute group (AG) was always available; decrease if you are willing to accept less complete data for ROS.
ROS Period Weightages	50%, 30%, 20%	Defines the weighting applied to sales	Adjust to place more emphasis on recent sales (e.g., 80% for

Setting Name	Default Value	Description	When to Change
		data from the most recent, mid-range, and older historical periods, respectively, when computing a blended Rate of Sale (ROS).	most recent period) for fast-changing trends, or more balanced weighting for stable products.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Transactions & Product Live Data] --> B[Clean Liquidation Sales]

B --> C[Handle Broken Sales Data]

C --> D[Compute Rate of Sale (ROS) for SKUs]

D --> E[Filter & Prioritize Sales Data]

E --> F[Prepared EOSS Sales Data Set]

1. Start with → Capture all raw sales transactions and information about when products were actually available for sale (live status).
2. Then → **Cleanse** this data by systematically removing sales that occurred during specific liquidation events.
3. Next → **Identify** and **handle** “broken sales data” periods where products weren’t fully represented (e.g., missing sizes).
4. Then → **Compute** the Rate of Sale (ROS) for each individual SKU, considering various historical periods and their assigned weightages.
5. Finally → **Filter** and **prioritize** the sales data based on specific seasons or relevance, yielding the comprehensive and accurate sales data set needed for further EOSS analysis.

5. Step-by-Step Actions

1. **Remove** liquidation sales
 - What we do: *We identify and exclude sales transactions that occurred during pre-defined liquidation periods or special promotional events, ensuring that the remaining data reflects regular sales activity.*
 - Why we do it: *This ensures our calculated Rate of Sale (ROS) and other performance metrics are based on true consumer demand, preventing skewed results from deep discounts or fire sales.*
2. **Identify** and **Handle** broken sales data
 - What we do: *We analyze sales records to find days or periods where a product’s key sizes or attributes (e.g., a specific color or size within an Ideal Size Set (ISS)) were not sufficiently available in a store. If too many pivotal SKUs were missing, that day’s sales for that product are flagged as “broken” and excluded from ROS calculations.*

- Why we do it: *This prevents skewed Rate of Sale (ROS) calculations. If a popular item only had one size available, its sales might appear low, but this isn't a reflection of true demand, only of limited stock. Excluding "broken" periods ensures an accurate ROS based on complete selling conditions.*
3. **Compute Rate of Sale (ROS) for SKUs**
 - What we do: *We calculate the average daily sales volume for each unique item (SKU) at each store. This calculation often involves looking at sales over multiple historical periods (e.g., last 30, 60, and 90 days) and applying a weighted average, giving more importance to recent performance.*
 - Why we do it: *This provides a robust and dynamic measure of product velocity, informing how quickly an item sells and helping to determine the optimal depth (OD) of inventory required in each store.*
 4. **Filter and Prioritize sales data**
 - What we do: *We select specific sales data relevant to the current EOSS period, often prioritizing sales from certain seasons or those tied to specific promotional periods, and make them available for immediate analysis.*
 - Why we do it: *This focuses the subsequent analysis on the most impactful and relevant sales information, ensuring that EOSS decisions are based on the freshest and most pertinent data for efficient clearance and replenishment.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular "Women's Summer Dress" (SKU 12345) sold 300 units in Store A over the last 90 days.
- However, 50 of these units were sold during a 3-day "Flash Sale" where the dress was marked down by 70%, classified as a liquidation event.
- Additionally, for 10 days out of those 90, Store A only had the smallest size (XS) of the dress in stock, even though it normally stocks a full Ideal Size Set (ISS) from XS to XL. This is considered "broken sales data" because the store couldn't sell to all customers.
- The "Minimum Live Days for ROS Calculation" is 7 days.
- The "Broken Sales Data Threshold" is 20% (meaning if less than 20% of the pivotal SKUs are available, the day is considered broken).
- ROS Period Weightages are 50% (last 30 days), 30% (last 60 days), 20% (last 90 days).

What Happens

1. First, EOSS Sales Data Preparation **removes** the 50 units sold during the 3-day "Flash Sale" from the relevant sales data for SKU 12345, ensuring these liquidation-driven sales don't artificially inflate the ROS.
2. Then, it **identifies** the 10 days where only the XS size was available. Since a full Ideal Size Set (ISS) was not present (meaning more than 80% of pivotal SKUs were missing), these 10 days are flagged as "broken sales data" and are excluded from the live days count used for ROS calculation.

3. Next, it **computes** the Rate of Sale (ROS) for SKU 12345 using the remaining 250 units sold over the *adjusted* number of live days (90 - 3 liquidation days - 10 broken sales days = 77 live days). It then blends this with sales from the last 30 and 60 days using the 50%/30%/20% weightages to produce a single, reliable ROS.
4. Finally, this calculated ROS is prepared and made available for subsequent modules, along with the cleansed sales data.

Final Outcome

Instead of an inflated or inaccurate Rate of Sale (ROS) based on all 300 units over 90 days, the system now has a precise ROS that reflects the true, unhindered selling speed of the “Women’s Summer Dress.” This accurate ROS will guide future decisions on Optimal Depth (OD) for new allocations or identifying stores where this dress is a slow seller and needs to be transferred out.

Key Takeaway: Thorough EOSS Sales Data Preparation ensures that inventory decisions are based on the most accurate and context-aware product performance metrics, preventing misallocations and maximizing sell-through.

7. Results

This step produces:

Main Outputs

- **Cleaned and Validated Sales Data Set:** What it tells us: All raw sales transactions, now meticulously refined, free from the bias of liquidation events and data integrity issues, organized and ready for detailed analysis.
- **SKU Rate of Sale (ROS) Figures:** What it tells us: The computed selling speed for each unique item (SKU) at each store, calculated using weighted historical periods, which is crucial for dynamic inventory planning and accurate forecasting.

Business Reports

- **Sales Data Quality Report:** Who uses it and why: Data analysts and IT teams use this to identify and address any remaining issues or gaps in the raw sales data, ensuring the foundational data for all EOSS decisions is sound and reliable.
- **SKU Performance Trends:** Who uses it and why: Merchandising planners and inventory managers use this to gain insight into which items are selling quickly or slowly, informing precise allocation strategies and strategic markdown decisions during the EOSS.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The sales data preparation process will pause or fail to initiate, as it cannot proceed without complete and accurate input data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
Excessive liquidation sales	If a very high percentage of recent sales are flagged as liquidation, the remaining “regular” sales data might be too sparse for accurate ROS calculation.	Review liquidation flags to ensure they are correctly applied. Consider adjusting the ‘Sales Liquidation Threshold’ or extending the analysis period if insufficient data remains.
Widespread broken sales data	If many products have insufficient “live days” or their “pivotal SKU live percentage” falls below the threshold, their ROS calculations might be unreliable or skipped entirely.	Investigate inventory management and store execution processes that lead to incomplete stock. Adjust ‘Broken Sales Data Threshold’ or ‘Minimum Unbroken Live Days’ if current settings are too strict or lenient.
Negative or zero sales for seemingly active products	Products may show no sales despite being in stock, leading to a zero ROS and potentially being overlooked for allocation, even if they should be selling.	Verify if products were truly in stock and actively selling. Investigate potential data gaps, POS system errors, or issues with product master data (e.g., incorrect active dates for the SKU).

9. Next Steps

Before This Step

- **Raw Sales Data Collection:** What we need from it: All individual transaction records, including quantities (Units Sold), dates, prices, and the store and SKU identifiers from all points of sale.
- **Inventory Live Status Data:** What we need from it: Precise information on when each SKU was physically available for sale in each store (its “live” status), which is essential for calculating accurate “live days” for ROS.

After This Step

- **Sales & Revenue Processing:** What they need from us: The cleansed and validated sales data, along with the computed Rate of Sale (ROS) figures, to

calculate overall revenue and profitability for individual products and Attribute Groups (AG).

- **Segmentation & Ranking:** What they need from us: The fully prepared sales data, including accurate ROS, to group products by their performance characteristics and assign priority scores for optimal inventory management and EOSS allocation.

10. Common Questions

Q: Why is it crucial to clean liquidation sales from our data? A: Cleaning liquidation sales ensures that the Rate of Sale (ROS) we calculate truly reflects organic customer demand, not just temporary spikes caused by deep discounts. Using data from these sales would give a misleadingly high ROS, leading to incorrect inventory planning and potentially overstocking.

Q: What does “broken sales data” mean, and why is it important to handle it? A: “Broken sales data” refers to periods when a product was in stock, but not all critical sizes or attributes (like a full Ideal Size Set - ISS) were available. Ignoring this would lead to an artificially low Rate of Sale (ROS) because customers couldn’t buy their preferred option. Handling it ensures a fair ROS calculation based on true selling potential.

Q: How is the Rate of Sale (ROS) actually calculated in this step? A: The Rate of Sale (ROS) is calculated as units sold divided by the number of days the product was actually “live” (available for sale) in a store. This calculation is then often refined by applying a weighted average across multiple historical periods (e.g., 50% from the last 30 days, 30% from the last 60, and 20% from the last 90 days) to give more importance to recent trends.

Q: What happens if a product has a zero Rate of Sale (ROS) after this process? A: A zero Rate of Sale (ROS) after cleaning means the product either genuinely isn’t selling or there’s a significant data issue. Products with zero ROS might be flagged for immediate transfer out of the store, deeper markdowns, or further investigation into their availability or appeal.

Q: Can I manually adjust the calculated Rate of Sale (ROS) for specific products? A: No, the Rate of Sale (ROS) is systematically computed based on your defined business settings to ensure consistency and objectivity across all products and stores. If you believe the ROS is inaccurate, you should review and adjust the relevant business settings (like ‘Sales Liquidation Threshold’ or ‘ROS Period Weightages’) and re-run the process rather than making manual overrides.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prepares raw sales data by cleaning, handling incomplete data, and computing the Rate of Sale (ROS).
- **Main Input:** Raw sales transactions, SKU live status, and product attribute data.
- **Main Output:** Cleaned and validated sales data set, and accurate SKU Rate of Sale (ROS) figures.

- **Warning Signs:** Unusually high or low ROS figures, large amounts of data excluded due to “broken sales,” or inconsistent sales trends.
 - **Key Settings:** Sales Liquidation Threshold, Minimum Live Days for ROS Calculation, Broken Sales Data Threshold, and ROS Period Weightages.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: EOSS Revenue Computation

Last Updated: 2025-05-23

1. Business Objective

EOSS Revenue Computation is responsible for calculating various revenue metrics for every unique store-style combination, including national revenue per day and hierarchical revenue per day based on different Attribute Groups (AGs). It also identifies “read-only” store styles and applies minimum outwarding age criteria, which are essential for the subsequent ranking and allocation processes.

Why This Matters: This step provides a crucial financial valuation for every product in every store, enabling accurate prioritization of inventory. By understanding the true earning potential of each item, the system can make smarter decisions about which products to move, replenish, or clear, directly optimizing revenue and minimizing losses during the End-of-Season Sale.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Transactions:** Why we need it: To accurately sum the total sales value (revenue) for each product over specific periods, forming the basis for all revenue per day calculations.
- **Units Sold & Rate of Sale (ROS):** Why we need it: To understand the velocity at which products sell, which helps in determining their contribution to overall revenue and identifying slow-moving items.

Product Data

- **Product Attributes (AG):** Why we need it: Details like category, subcategory, brand, and gender are essential for grouping products into Attribute Groups (AGs) to calculate hierarchical revenue metrics when specific store-style sales data is insufficient.
- **Style Information:** Why we need it: To identify the complete set of SKUs within each style, their relevant seasons, and other characteristics necessary for aggregation and rule application.

Operational Data

- **Current Inventory Levels:** Why we need it: To determine the total stock of each style in both stores and warehouses, which is a prerequisite for considering any store-style combination for allocation.
- **Store Configurations:** Why we need it: To identify all active distribution stores, their specific channels, and any relationships to reference stores for replicating revenue calculations.
- **Style Availability Days:** Why we need it: To know precisely for how many days a specific style was available for sale in a store, allowing for accurate calculation of average daily revenue.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Minimum Display Age (Days)	30 days	The minimum number of days a new style must have been available in a store before it can be considered for outward transfers or deep markdowns.	Increase for products with a long shelf life or slower introduction phases; decrease if you need to clear inventory aggressively and quickly.
Primary Sales Analysis Period (Days)	90 days	The primary historical period used to calculate direct daily revenue for a store-style combination.	Adjust based on product lifecycle and seasonality; a shorter period (e.g., 30 days) for fast-fashion, a longer period (e.g., 180 days) for evergreen products.
Inter-Store Transfer (IST) Mode	False	When set to 'True', the system prioritizes balancing stock between stores and will retain zero-stock store-styles for potential inbound transfers, even if they have no current inventory.	Set to 'True' during active inter-store transfers to prevent lost sales from stockouts; set to 'False' if focus is purely on warehouse-to-store distribution or clearing existing store stock.

Setting Name	Default Value	Description	When to Change
Clearance-Only Seasons	“Spring 2024”	A list of seasons whose styles will be flagged as “read-only,” meaning they are primarily for clearance and will not be replenished.	Update annually or seasonally to designate past season inventory that needs to be sold off without receiving new stock.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Inventory] --> B[Identify Store-Style Combinations]

B --> C[Apply Eligibility Rules]

C --> D[Calculate National & Hierarchical Revenue]

D --> E[Identify Read-Only Styles]

E --> F[Revenue-Ranked Store-Style Combinations]

1. Start with → The prepared sales transactions, current inventory levels, and master product/store data from previous steps.
2. Then → **Identify** all relevant store-style combinations and compute their initial inventory levels (store and warehouse).
3. Next → **Apply** critical eligibility rules, such as including/excluding specific combinations and checking the “Minimum Display Age” to protect newly arrived styles from immediate outward transfer.
4. Then → **Calculate** average daily revenue for each style nationally, then for individual store-style combinations (using direct sales data where available, or rolling up to hierarchical Attribute Groups (AGs) for less active combinations).
5. Finally → **Flag** styles that are designated for clearance as “read-only” (not eligible for replenishment) and then **filter** out zero-stock combinations (unless in Inter-Store Transfer (IST) Mode), producing a comprehensive list of revenue-ranked store-style combinations.

5. Step-by-Step Actions

1. **Identify** and **Initialise** store-style combinations
 - What we do: *We create a list of all potential store-style combinations, calculating their current on-hand inventory at the store and warehouse level.*
 - Why we do it: *This establishes the universe of products we need to analyze and ensures we have an accurate starting point for their stock levels.*
2. **Apply** eligibility and display age criteria
 - What we do: *We check each store-style combination against predefined inclusion/exclusion lists and a “Minimum Display Age” rule. If a style has*

been in a store for less than the minimum age, it's flagged as "new" and not considered for immediate outward transfer.

- *Why we do it: This prevents newly launched or recently received products from being prematurely moved out, ensuring they have enough time to establish sales patterns and reach customers.*

3. Compute national and direct daily revenue

- *What we do: We calculate the average daily revenue for each style across all stores nationally. Then, for each specific store-style combination, we compute its daily revenue based on its direct sales history within the "Primary Sales Analysis Period."*
- *Why we do it: This provides a concrete measure of a product's direct performance, giving a baseline for its true sales potential in that specific store.*

4. Calculate hierarchical daily revenue

- *What we do: For store-style combinations with little or no direct sales history, we calculate their potential daily revenue by looking at sales trends at higher levels of aggregation, such as the product's immediate Attribute Group (AG) (e.g., all 'Women's Dresses' in that store), then rolling up to broader AGs if needed, like 'Apparel'.*
- *Why we do it: This ensures that every product, even those with limited sales history or in new stores, receives a realistic revenue estimate, allowing for fair comparison and allocation decisions across the entire inventory.*

5. Replicate revenue for new stores

- *What we do: For new stores without sufficient sales history, we assign revenue per day values to their store-style combinations by replicating the performance of a designated reference store.*
- *Why we do it: This allows new locations to participate in the EOSS distribution process from day one, leveraging insights from established stores to guide initial inventory decisions.*

6. Identify "read-only" store styles

- *What we do: We flag specific store-style combinations as "read-only" if they belong to "Clearance-Only Seasons" or are designated for clearance via channel-specific overrides. These items are only meant to be sold down, not replenished.*
- *Why we do it: This is critical for managing end-of-season inventory, ensuring that capital is not tied up in replenishing items designated for clearance, thereby maximizing sell-through and optimizing inventory investment.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new "Blue Denim Jacket" style (Style ID 500) arrived at Store A 15 days ago with 50 units. The "Minimum Display Age" setting is 30 days.
- An "Olive Cargo Pant" style (Style ID 600) has been in Store B for 120 days. Store B sold 300 units of this style in the last 90 days.
- A brand new Store C opened last week and has no sales history, but it is mapped to Store B as a reference store.

- The “Blue Denim Jacket” is from “Spring 2025” (a replenishment-eligible season). The “Olive Cargo Pant” is from “Fall 2024” (a “Clearance-Only Season”).

What Happens

1. First, the system identifies all store-style combinations, including “Blue Denim Jacket” in Store A, “Olive Cargo Pant” in Store B, and all styles in Store C.
2. Then, it applies the “Minimum Display Age” check: Since the “Blue Denim Jacket” in Store A has only been present for 15 days (less than the 30-day minimum), it is flagged as a “New Style.” This means it won’t be considered for outward transfer, even if its initial sales are low.
3. Next, the system calculates the national average daily revenue for “Olive Cargo Pant” and then calculates Store B’s specific daily revenue: 300 units sold over 90 days, at an average retail price of \$50, results in \$15,000 total revenue, so a daily revenue of \$166.67.
4. For Store C, since it has no sales history, the system replicates the daily revenue figures of Store B for all relevant styles, including the “Olive Cargo Pant.” If Store C has the “Blue Denim Jacket,” and Store B doesn’t, the system might try to calculate revenue for the “Blue Denim Jacket” in Store C by using a broader Attribute Group (AG) like “All Jackets” in Store C, or even a national average if local data is too sparse.
5. Finally, the “Olive Cargo Pant” in Store B (and Store C via replication) is flagged as “read-only” because it belongs to the “Fall 2024” season, which is designated as “Clearance-Only.” This prevents replenishment of the Olive Cargo Pants.

Final Outcome

Every store-style combination now has a calculated daily revenue, enabling the system to understand its financial value and prioritize inventory movement. New styles are protected from premature transfers, and clearance items are correctly flagged to avoid unnecessary replenishment, optimizing inventory efficiency and maximizing end-of-season revenue.

Key Takeaway: Accurate revenue computation, combined with crucial eligibility flags, provides the foundational data for intelligent inventory ranking and allocation, ensuring that every product contributes optimally to sales and profitability.

7. Results

This step produces:

Main Outputs

- **Revenue-Ranked Store-Style Combinations:** What it tells us: A comprehensive list of all store-style combinations, each assigned a calculated daily revenue, indicating its earning potential and allowing for performance-based prioritization.

- **Identified Read-Only Store-Styles:** What it tells us: A clear flag for specific store-style combinations that are designated for clearance and should not be replenished, guiding future allocation and markdown strategies.
- **Minimum Display Age Flags:** What it tells us: Flags on new styles that are not yet eligible for outward transfers, ensuring they are given adequate time to establish sales patterns.

Business Reports

- **Store-Style Performance Report:** Who uses it and why: Merchandising and Inventory Managers use this to understand the daily earning potential of each product in every store, informing strategic allocation and transfer decisions.
- **Clearance vs. Replenishment Status Report:** Who uses it and why: Buyers and Planners use this to quickly identify which styles are actively being replenished versus those designated for clearance, ensuring alignment with overall inventory strategy.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The revenue computation process will pause or fail to initiate, as it cannot proceed without complete and accurate sales and inventory input data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
No sales data for many styles	The system will heavily rely on hierarchical Attribute Group (AG) revenue calculations, which might be less precise than direct sales data.	Investigate why sales data is missing (e.g., data integration issues, actual zero sales). Consider adjusting “Primary Sales Analysis Period” or reviewing AG definitions for accuracy.
All styles flagged as “New Style” (Minimum Display Age)	This suggests the “Minimum Display Age” setting is too high for the product lifecycle, preventing any styles from being considered for outward transfers.	Review and adjust the “Minimum Display Age” setting to align with typical product lifecycles and business objectives for faster inventory turns.
Incorrectly flagged “read-only” styles	Styles intended for replenishment are marked for clearance, or vice versa, leading to incorrect inventory actions (e.g., not	Verify the “Clearance-Only Seasons” list and any channel-specific style overrides to ensure they accurately reflect the intended inventory strategy.

Situation	What Happens	What You Should Do
Negative or zero stock for styles with high calculated revenue	replenishing bestsellers). These styles won't be filtered out if in IST mode, but they will highlight immediate stockout issues, implying lost sales.	Review current inventory levels for these high-potential styles and prioritize inbound transfers or replenishment from the warehouse to capture demand.

9. Next Steps

Before This Step

- **EOSS Data Preparation:** What we need from it: All validated master data (store, product, season information) and comprehensive current inventory levels, including goods-in-transit.
- **EOSS Sales Data Preparation:** What we need from it: Cleaned and validated sales transactions, along with accurate Rate of Sale (ROS) figures for each SKU and style.

After This Step

- **Segmentation & Ranking:** What they need from us: The calculated daily revenue figures for all store-style combinations to classify and prioritize them based on their sales potential.
- **Allocation & Transfers:** What they need from us: The final revenue-ranked store-style list, along with flags for “read-only” styles and “new styles,” to guide the system in making precise inventory movement decisions.

10. Common Questions

Q: Why calculate revenue at multiple levels (national, store, AG)? A: We calculate revenue at multiple levels to ensure that every store-style combination has a reliable revenue estimate. For items with direct sales history, we use precise store-level data. For newer items or those with infrequent sales, we roll up to broader Attribute Groups (AGs) to use aggregated sales trends, providing a fair estimate even when direct data is sparse.

Q: What does “Minimum Display Age” mean, and why is it important? A: “Minimum Display Age” refers to the number of days a product must have been available in a store before it can be considered for outward transfers or deep markdowns. It's crucial because it protects new inventory from being prematurely moved out, allowing it sufficient time to sell organically and avoid disrupting new product introductions.

Q: What is a “read-only” store style, and why is it important to identify? A: A “read-only” store style is an item designated for clearance that should only be sold down from its current stock, with no replenishment. Identifying these is vital for end-of-

season sales because it prevents tying up capital in replenishing items that are going to be marked down, ensuring efficient inventory liquidation and maximizing overall profitability.

Q: How does this step impact profitability during EOSS? A: This step directly impacts profitability by providing accurate financial valuation for all inventory. By knowing each item's daily revenue potential, the system can prioritize moving high-value items to high-demand stores, quickly clear low-value or old-season stock, and avoid overstocking, which ultimately boosts sales and reduces losses from excessive markdowns.

Q: What happens if a store-style combination has no current stock but a high calculated daily revenue? A: If a store-style has no stock but a high calculated daily revenue, it indicates a strong sales potential and likely a stockout situation. If the system is in "Inter-Store Transfer (IST) Mode," this combination will still be retained, indicating it's a candidate for receiving inbound transfers. Otherwise, it signifies lost sales opportunity, and the system moves on to focus on combinations with available inventory.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates various revenue metrics for store-style combinations and identifies "read-only" styles and new styles.
- **Main Input:** Cleaned Sales Data, current inventory levels, and product/store master data.
- **Main Output:** Revenue-ranked store-style combinations, with flags for "read-only" and "new" styles.
- **Warning Signs:** Unusually high or low revenue values, a high proportion of styles flagged as "new," or incorrect "read-only" designations.
- **Key Settings:** Minimum Display Age, Primary Sales Analysis Period, Inter-Store Transfer (IST) Mode, and Clearance-Only Seasons.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 5: EOSS Segmentation

Last Updated: 2025-05-23

1. Business Objective

EOSS Segmentation classifies store-style combinations into meaningful groups like 'Top Seller' and 'Bottom Seller' based on their sales performance, discount history, and current inventory metrics.

Why This Matters: This segmentation is crucial for guiding inventory decisions during the End-of-Season Sale (EOSS), enabling targeted strategies for allocating popular items, identifying products for pullbacks or deeper discounts, and ensuring that every item is managed to maximize sales and minimize losses. It directly informs decisions related to achieving the Optimal Depth (OD) of inventory in stores.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Transactions:** Why we need it: To accurately assess sales volume and calculate Rate of Sale (ROS) for individual products and categories, ensuring our segmentation is based on reliable demand.
- **Rate of Sale (ROS) for SKUs:** Why we need it: To understand how quickly specific products are selling in each store, which is a primary factor for classifying items as top or bottom performers.
- **Historical Discount Information:** Why we need it: To evaluate if sales performance is driven by organic demand or significant markdowns, which influences whether an item is considered a "bottom seller" despite some sales.
- **Current Sales Trends:** Why we need it: To identify fast-moving or slow-moving items right now, helping to make real-time allocation decisions.

Product Data

- **Product Attributes (AG):** Why we need it: To group products into categories (AG) like 'Apparel', 'Footwear', or 'Accessories', allowing for a benchmark of average performance within relevant groups.
- **Current Inventory Levels:** Why we need it: To know the exact number of units of each product available in stores and warehouses (OD), which is essential for determining sell-through rates and identifying overstocked items.

- **Style Information:** Why we need it: To link individual SKUs to their broader style, enabling consistent analysis across different sizes and colors of the same product.
- **SKU Live Status:** Why we need it: To determine how long a product has been actively available for sale in a store, critical for calculating accurate selling speeds and display age.

Operational Data

- **Store Configurations:** Why we need it: To understand which stores are active for EOSS, their channels, and their connections to specific warehouses for potential inventory transfers.
- **Pullback Configuration Settings:** Why we need it: To define specific criteria (e.g., sales quantity thresholds, discount benchmarks) that determine which items are eligible for pullback consideration.
- **Warehouse-Store Mapping:** Why we need it: To correctly associate warehouse stock with relevant stores, allowing for a comprehensive view of inventory availability for sell-through calculations.
- **Minimum Display Age Flag:** Why we need it: To identify new styles that should not be considered for immediate outward transfers, protecting their initial selling period.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Top Seller Revenue Multiplier	1.5	A factor applied to average store-category revenue per day to define the minimum threshold for a style to be considered a 'Top Seller'.	Increase to be more selective about designating top sellers; decrease to include more styles in the top-performing segment.
Top Seller Live Days Benchmark	20 days	The minimum number of days a style must have been available in a store to be considered for 'Top Seller' classification.	Increase for products that need longer to establish strong sales patterns; decrease for fast-moving items or shorter EOSS cycles.
Revenue Per Day Benchmark for Bottom Sellers	0.5	A multiplier indicating the maximum revenue per day a style can generate, relative to its	Increase if you want to include relatively better-selling items in the bottom seller category; decrease if you want to focus only on the truly

Setting Name	Default Value	Description	When to Change
Discount Benchmark for Bottom Sellers	0.8	category's average, to be considered a 'Bottom Seller'. A multiplier indicating the maximum discount percentage a style can have, relative to its category's average discount, to <i>still</i> be considered a potential 'Bottom Seller'.	worst-performing items. Increase if you are willing to identify items with high discounts as bottom sellers; decrease if only items selling poorly at lower discounts are considered bottom sellers.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Cleaned Sales Data & Inventory Levels] --> B{EOSS Segmentation}
    B --> C[Identify Top Sellers]
    B --> D[Identify Bottom Sellers]
    C --> E[Segmented Store-Style Combinations]
    D --> E
  
```

1. Start with → Cleaned sales data, current inventory levels, and product/store configurations from previous EOSS steps.
2. Then → **Identify** 'Top Seller' store-style combinations by analyzing their sales performance against category and channel benchmarks.
3. Next → **Identify** 'Bottom Seller' store-style combinations by evaluating their sales volume, revenue per day, discount levels, and sell-through rates against specific criteria.
4. Finally → **Produce** a comprehensive list of all store-style combinations, each clearly marked with its assigned segment (e.g., 'Top Seller', 'Bottom Seller', or 'Regular').

5. Step-by-Step Actions

1. **Initialize** the segmentation process
 - What we do: *We gather all necessary data, including cleaned sales transactions, current stock levels, and store/product details, setting the stage for analysis.*

- Why we do it: *This ensures that the segmentation process uses the most accurate and up-to-date information, preventing errors in classification.*
- 2. Identify Top Sellers**
- What we do: *We evaluate each store-style combination's daily revenue against channel-wide and store-category benchmarks. Styles that consistently exceed these benchmarks and meet a minimum 'live days' threshold are marked as 'Top Sellers'.*
 - Why we do it: *This highlights the strongest performing items, indicating where replenishment efforts should be prioritized to maximize sales and avoid lost revenue from stockouts.*
- 3. Identify Bottom Sellers**
- What we do: *We analyze store-style combinations that are not classified as 'Top Sellers' to determine if they are 'Bottom Sellers'. This involves checking their sales volume, daily revenue (must be below category benchmarks), discount levels (must not be excessively high for their sales), and sell-through rates (must be below average).*
 - Why we do it: *This pinpoints underperforming items that are tying up capital, guiding decisions for immediate transfers, deeper markdowns, or pullbacks to optimize inventory health and clear old stock.*
- 4. Compute Sell-Through Rates**
- What we do: *For identifying bottom sellers, we calculate the sell-through rate for each store-style combination (units sold compared to total available stock, including warehouse stock mapped to the store). This is compared against the average sell-through rate for its category in that store.*
 - Why we do it: *A low sell-through rate, especially combined with high inventory, is a key indicator of an underperforming item, helping to decide if it needs to be moved or discounted.*
- 5. Assign Segmentation Status**
- What we do: *Based on the above criteria, each store-style combination is assigned a segment: 'Top Seller', 'Bottom Seller', or no specific segment (implying a regular performer).*
 - Why we do it: *This provides a clear, actionable classification that drives subsequent inventory allocation, transfer, and markdown strategies, ensuring targeted and efficient inventory management.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Store A, "Women's Summer Dress" (Style 101):** Sells 5 units/day, and its category average for dresses in Store A is 2 units/day. The 'Top Seller Revenue Multiplier' is 1.5. Style 101 has been live for 45 days.
- **Store B, "Men's Winter Coat" (Style 202):** Sells 0.5 units/day, and its category average for coats in Store B is 1 unit/day. Store B's average category discount is 10%. Style 202 has a current discount of 5%, and a sell-through rate of 15% (the category average is 30%). The 'Revenue Per Day Benchmark for Bottom Sellers' is 0.5, and the 'Discount Benchmark for Bottom Sellers' is 0.8. Store B has 100 units of Style 202 in stock.
- **Store C, "Unisex Sneakers" (Style 303):** Sells 1 unit/day, and its category average for sneakers in Store C is also 1 unit/day.

What Happens

1. First, for **Store A, Style 101**: The module calculates its daily revenue. Since its 5 units/day ROS is significantly higher than the store-category benchmark (2 units/day * 1.5 = 3 units/day benchmark), and it has been live for more than 20 days ('Top Seller Live Days Benchmark'), Style 101 is designated as a '**Top Seller**' in Store A.
2. Then, for **Store B, Style 202**: The module first checks if it's a Top Seller (it's not). Next, it evaluates for Bottom Seller criteria:
 - **Revenue Check**: Its 0.5 units/day ROS results in revenue that meets or falls below the store-category benchmark (1 unit/day * 0.5 = 0.5 units/day benchmark).
 - **Discount Check**: Its 5% discount (0.05) is less than the store category's average discount benchmark (10% * 0.8 = 8% benchmark), indicating its sales are not artificially inflated by very deep price cuts.
 - **Sell-Through Check**: Its 15% sell-through rate is significantly below the category average of 30%.
 - Given its low sales and low sell-through rate despite a modest discount, Style 202 is designated as a '**Bottom Seller**' in Store B.
3. Finally, **Store C, Style 303** does not meet the specific criteria for either 'Top Seller' or 'Bottom Seller', so it remains unsegmented as a regular performer.

Final Outcome

Each store-style combination is precisely classified, providing actionable intelligence: Store A's "Women's Summer Dress" will be prioritized for replenishment or transfers *into* Store A to maintain Optimal Depth (OD). Store B's "Men's Winter Coat" will be flagged for potential pullback, transfer *out* of Store B, or deeper markdown. Store C's "Unisex Sneakers" will be managed with standard inventory processes.

Key Takeaway: Precise segmentation allows retailers to proactively manage inventory by identifying best-selling items for optimal stocking and underperforming items for efficient clearance, directly impacting sales and profitability.

7. Results

This step produces:

Main Outputs

- **Segmented Store-Style Combinations:** What it tells us: A comprehensive list of all store-style combinations, each assigned a classification (e.g., 'Top Seller', 'Bottom Seller', or 'Regular'), providing a clear performance snapshot.
- **Reasons for Segmentation:** What it tells us: Specific details (remarks) explaining why a store-style combination was assigned a particular segment (e.g., "low sales velocity," "high discount without sales"), aiding in understanding the decision logic.

Business Reports

- **Top Seller Opportunities Report:** Who uses it and why: Merchandising Planners use this to identify high-potential items and locations where inventory should be prioritized to maximize sales and prevent stockouts.
- **Bottom Seller Action List:** Who uses it and why: Inventory Managers and Operations Teams use this to pinpoint specific products and stores that require immediate intervention, such as inter-store transfers (pullbacks), deep markdowns, or consolidation to clear excess stock.
- **Performance Segmentation Dashboard:** Who uses it and why: Executive Leadership uses this to gain a high-level overview of inventory health, understanding the proportion of top, bottom, and regular performers across the business.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The segmentation process will pause or fail, as it cannot perform accurate classifications without complete and up-to-date sales and inventory data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
All styles categorized as 'Regular' (no Top or Bottom Sellers)	This indicates that the segmentation criteria (e.g., multipliers, benchmarks) might be too strict, or the sales performance is too consistent across all products.	Review and adjust the 'Top Seller Revenue Multiplier', 'Revenue Per Day Benchmark for Bottom Sellers', and other relevant settings to better differentiate performance.
High-performing items incorrectly flagged as 'Bottom Sellers'	This can occur if the 'Discount Benchmark for Bottom Sellers' is too lenient, or if sales from recent promotional activities are skewing results.	Investigate the sales history and discount levels for these items. Adjust the discount threshold or refine the definition of "clean sales" in previous steps.
Store has no sales for a specific product category	The system will rely on broader, hierarchical (Attribute Group - AG) data for revenue and sell-through benchmarks, which may be less precise.	Ensure accurate category assignments for products. If a store truly has no sales for an entire category, consider if it's the right fit for that product type, or if a reference store's data can be used more effectively.
Inaccurate current inventory levels	Sell-through rates will be miscalculated, potentially leading to incorrect 'Bottom	Conduct physical inventory audits to ensure accuracy. Implement

Situation	What Happens	What You Should Do
	Seller' classifications (e.g., an item appears to have low sell-through but is actually low on stock).	stricter inventory reconciliation processes.

9. Next Steps

Before This Step

- **EOSS Sales Data Preparation:** What we need from it: Cleaned and validated sales transactions, along with accurate Rate of Sale (ROS) figures for each SKU and style, free from liquidation sales and broken data.
- **EOSS Revenue Computation:** What we need from it: Calculated daily revenue figures for all store-style combinations, including national and hierarchical revenue estimates, and flags for “new” or “read-only” styles.
- **EOSS Data Preparation:** What we need from it: All validated master data (store, product, season information) and comprehensive current inventory levels, including warehouse stock and goods-in-transit.

After This Step

- **EOSS Allocation & Transfers:** What they need from us: The segmented list of store-style combinations, which will guide decisions on how much inventory to allocate to ‘Top Sellers’ to maintain Optimal Depth (OD) and which ‘Bottom Sellers’ to transfer out or pull back.
- **EOSS Markdown & Pricing:** What they need from us: The ‘Bottom Seller’ classifications to inform specific pricing strategies, such as deeper discounts or bundled offers, to clear excess inventory.
- **Performance Monitoring:** What they need from us: The segmentation results to continuously track the effectiveness of inventory strategies and adjust as needed, influencing future Optimal Width (OW) and Ideal Size Set (ISS) decisions.

10. Common Questions

Q: Why is it important to classify products into ‘Top Seller’ and ‘Bottom Seller’ segments? A: Classifying products helps focus your inventory management efforts. ‘Top Sellers’ indicate where to prioritize replenishment to maximize sales, while ‘Bottom Sellers’ pinpoint where to act quickly (e.g., transfers, markdowns) to minimize losses and free up capital, ultimately boosting overall profitability.

Q: How does this segmentation help prevent lost sales or excess inventory? A: By identifying ‘Top Sellers’, you ensure popular items are adequately stocked, preventing lost sales from stockouts. By identifying ‘Bottom Sellers’, you prevent excess inventory from accumulating, reducing the need for drastic markdowns and optimizing inventory investment.

Q: What if a product has high sales but is still flagged as a 'Bottom Seller'? A: This can happen if its sales are primarily driven by very deep discounts, or if it has a very high inventory level compared to its sales. The system flags it as a 'Bottom Seller' because, despite sales, it might not be generating sufficient profitable revenue or clearing stock efficiently, indicating a need for different strategic action.

Q: Can the criteria for 'Top Seller' and 'Bottom Seller' be changed? A: Yes, the system includes business settings (like 'Top Seller Revenue Multiplier' or 'Revenue Per Day Benchmark for Bottom Sellers') that allow you to adjust the thresholds for classifying products. This flexibility ensures the segmentation aligns with your evolving business goals and market conditions.

Q: How often is this segmentation performed? A: The frequency of EOSS Segmentation is typically aligned with your overall 'EOSS Run Frequency' setting, often daily or weekly. This ensures that inventory decisions are based on the most current sales performance and stock levels, allowing for timely adjustments during the dynamic end-of-season period.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Classifies store-style combinations into 'Top Seller', 'Bottom Seller', or 'Regular' segments for targeted inventory management.
- **Main Input:** Cleaned sales data, SKU Rate of Sale (ROS), current inventory levels, and product/store configurations.
- **Main Output:** A comprehensive list of store-style combinations, each assigned a performance segment.
- **Warning Signs:** Too many or too few items in 'Top' or 'Bottom' segments, high-performing items incorrectly flagged, or segmentation results not aligning with business expectations.
- **Key Settings:** Top Seller Revenue Multiplier, Top Seller Live Days Benchmark, Revenue Per Day Benchmark for Bottom Sellers, and Discount Benchmark for Bottom Sellers.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: EOSS Ranking

Last Updated: 2025-05-23

1. Business Objective

EOSS Ranking prioritizes every unique store-style combination based on its current inventory levels, product availability, and revenue-generating potential, ensuring the most impactful items are managed first during the End-of-Season Sale (EOSS).

Why This Matters: This step is crucial for efficient inventory management. By clearly identifying which products in which stores have the highest potential for sales or the most urgent need for attention, it allows merchandising and operations teams to focus their efforts where they matter most, maximizing revenue and minimizing stockouts or excess inventory.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Computed Daily Revenue:** Why we need it: The average daily revenue generated by each style in a specific store, or nationally, is a primary factor for ranking items by their financial contribution and sales potential.
- **Rate of Sale (ROS):** Why we need it: While not a direct ranking criterion here, understanding the selling speed of a product underpins its revenue calculation, ensuring the ranking is based on actual demand.

Product Data

- **Current Inventory Levels:** Why we need it: Knowing the exact number of units of each style in a store (its total stock) is essential for assessing overstocking or stockout risks.
- **Percent Stock Available (PSA) Percentage:** Why we need it: This metric indicates how well a product's Ideal Size Set (ISS) is stocked in a store, revealing if the right range of sizes is present to meet customer demand, directly influencing its potential to sell.
- **Style Information:** Why we need it: To group individual products (SKUs) into broader styles, ensuring a consistent ranking across all variations of a single product design.
- **Attribute Groups (AG):** Why we need it: For situations where direct store-style sales data is limited (e.g., new items or low-volume stores), sales

performance from broader Attribute Groups provides a reliable basis for estimating revenue and thus contributing to ranking.

Operational Data

- **Store Configurations:** Why we need it: To understand which stores are active participants in the EOSS process and their specific operational characteristics.
- **New Store Mappings:** Why we need it: To apply consistent ranking logic for newly opened stores that lack historical sales data, by referencing performance from similar, established stores.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Revenue Emphasis Weight	High	Determines the relative importance of daily revenue in the overall ranking compared to stock levels and PSA.	Adjust to 'Medium' or 'Low' if stock efficiency (e.g., clearing excess) or product availability (e.g., maintaining full ISS) is a higher priority than immediate revenue generation.
PSA Optimization Goal	90%	The target Percent Stock Available (PSA) percentage. Store-styles closer to this target, especially with high revenue, are prioritized.	Increase for products where a complete Ideal Size Set (ISS) is critical for sales (e.g., apparel); decrease if partial stock is acceptable or focus is purely on unit movement.
New Store Ranking Default	Reference Store Average	Defines how styles in new stores (without historical sales) are initially ranked.	Change to 'National Average' if reference store data is unreliable, or 'Category Baseline' for a more conservative initial ranking.
Stock Weighting Factor	Medium	Controls the influence of total on-hand stock on the ranking.	Increase to prioritize clearing high-stock items, regardless of sales; decrease if sales velocity is more critical than stock volume.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Revenue-Calculated Store-Style Data] --> B{EOSS Ranking}
B --> C[Apply Ranking Criteria: Revenue, PSA, Stock]
C --> D[Integrate New Store Mappings]
D --> E[Prioritized EOSS Ranked Store-Style Combinations]
```

1. Start with → The comprehensive list of store-style combinations, each with its calculated daily revenue, Percent Stock Available (PSA), and current inventory levels from previous EOSS steps.
2. Then → **Apply** the core ranking logic, sorting combinations primarily by their store-level daily revenue, then by national style revenue, followed by PSA percentage, and finally by total stock, acting as tie-breakers.
3. Next → **Integrate** any new store mappings, ensuring that newly opened stores with no sales history receive a logical ranking based on their designated reference stores.
4. Finally → **Produce** a comprehensive and prioritized list of all store-style combinations, each assigned a numerical rank, ready to guide allocation and transfer decisions.

5. Step-by-Step Actions

1. **Gather** all necessary store-style data
 - What we do: *We collect the calculated daily revenue figures, Percent Stock Available (PSA) percentages, and current total stock levels for every store-style combination that is eligible for EOSS distribution.*
 - Why we do it: *This ensures that the ranking process uses the most current and relevant performance and inventory data, creating an accurate basis for prioritization.*
2. **Apply** primary ranking criteria
 - What we do: *We sort the store-style combinations. The primary sorting is by their calculated daily revenue at the store level. For items with similar revenue, we then use the national average revenue for that style as a tie-breaker. Further tie-breakers include the Percent Stock Available (PSA) percentage (prioritizing styles with better size availability), and finally, the total stock (prioritizing higher stock levels in a descending order, often to identify items for clearance).*
 - Why we do it: *This multi-layered sorting ensures that the most financially impactful items are ranked highest, while providing a clear method to differentiate between similar-performing items, leading to intelligent allocation decisions.*
3. **Incorporate** new store mappings
 - What we do: *For new stores that lack their own sales history, we apply the ranking logic by referencing the performance data of their pre-assigned reference stores.*
 - Why we do it: *This allows new locations to participate effectively in the EOSS distribution from the start, avoiding delays and ensuring they receive appropriate inventory based on established store performance.*

4. **Finalize** the ranked list

- What we do: *We assign a numerical rank to each store-style combination based on the sorting results, creating a definitive, prioritized list.*
- Why we do it: *This clear ranking provides an actionable sequence for all subsequent inventory allocation and transfer processes, streamlining operations and ensuring that attention is given to items in their order of business importance.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Store A, "Women's Summer Dress" (Style 101):** Daily Revenue = \$200, PSA = 95%, Total Stock = 100 units. National Revenue for this style = \$150.
- **Store B, "Men's Winter Coat" (Style 202):** Daily Revenue = \$180, PSA = 80%, Total Stock = 50 units. National Revenue for this style = \$200.
- **Store C, "Unisex Sneakers" (Style 303):** Daily Revenue = \$200, PSA = 90%, Total Stock = 120 units. National Revenue for this style = \$180.
- **New Store D, "Women's Summer Dress" (Style 101):** No sales history, mapped to reference Store A.

What Happens

1. First, the system collects all the calculated data for each store-style combination.
2. Then, it begins ranking:
 - **Store A, Style 101** and **Store C, Style 303** both have the highest store-level daily revenue (\$200). They are initially prioritized.
 - To break the tie between Store A and Store C, the system looks at national revenue: Store C's style (\$180 national revenue) is lower than Store A's style (\$150 national revenue). (Note: the ranking order is descending, so higher national revenue is better, implying Store C's style is slightly better overall).
 - If there was still a tie, it would look at PSA: Store A (95%) is better than Store C (90%).
 - If still a tie, it would look at Total Stock: Store C (120 units) is higher than Store A (100 units).
 - Based on the typical ranking logic (rev per day -> national rev -> PSA -> total stock), Store A's Style 101 and Store C's Style 303 would be at the top, with their exact final order determined by the full tie-breaker sequence. Let's assume Store A, Style 101 gets Rank 1, and Store C, Style 303 gets Rank 2.
 - **Store B, Style 202** (\$180 daily revenue) is ranked lower than Store A and Store C but higher than any combinations with lower revenue. It might get Rank 3.
3. Next, for **New Store D, Style 101**: Since it has no sales history, the system leverages its mapping to Store A. It applies the revenue and stock characteristics of Store A's "Women's Summer Dress" to New Store D's "Women's Summer Dress" for ranking purposes, ensuring it receives a relevant rank.

Final Outcome

Every store-style combination now has a definitive rank. For instance, Store A’s “Women’s Summer Dress” is identified as a top priority for allocation efforts due to its high revenue and excellent stock availability. This clear prioritization guides the next steps, such as allocating additional inventory to top-ranked items or planning transfers for lower-ranked, overstocked items.

Key Takeaway: EOSS Ranking transforms complex data into a clear, actionable priority list, enabling swift and effective inventory decisions that maximize sales and minimize losses during the end-of-season period.

7. Results

This step produces:

Main Outputs

- **EOSS Ranked Store-Style Combinations:** What it tells us: A comprehensive list of all eligible store-style combinations, each assigned a unique numerical rank indicating its business priority for inventory actions.
- **Prioritized Allocation List:** What it tells us: A sequence of store-style combinations ordered by their rank, directly guiding subsequent modules on which products to address first for transfers, replenishment, or markdown.

Business Reports

- **Top Ranked Items Report:** Who uses it and why: Merchandising Planners use this to identify and monitor the highest-potential products across the chain, ensuring they are optimally stocked and preventing lost sales.
- **Inventory Action Prioritization Dashboard:** Who uses it and why: Inventory Managers and Operations Teams use this to get a real-time view of which inventory actions (e.g., inbound transfers, outbound pullbacks) are most urgent based on the calculated rankings.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The ranking process will pause or fail, as it cannot perform accurate prioritization without complete and up-to-date sales, inventory, and PSA data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
All store-styles have similar ranks	This suggests that the input data or calculated metrics (revenue, PSA,	Review the input data quality from previous steps (e.g., EOSS Revenue Computation) to ensure

Situation	What Happens	What You Should Do
	stock) are too uniform, or the differentiation in these values is not strong enough to create a clear hierarchy.	sufficient granularity in metrics. Adjust 'Revenue Emphasis Weight' or 'Stock Weighting Factor' if these settings exist and allow for more differentiation.
New store mapping is incorrect or missing	New stores might receive an inaccurate or default low ranking, leading to suboptimal initial inventory levels or missed sales opportunities.	Verify and correct the new store mappings in the master data. Ensure each new store is linked to an appropriate reference store for accurate performance replication.
High-value items consistently ranked low	This indicates a potential issue with the calculated revenue, PSA, or stock data, or an unexpected interaction with the ranking criteria.	Investigate the specific store-style combinations' metrics (revenue, PSA, stock) and compare them to the ranking logic. Ensure that the 'Revenue Emphasis Weight' aligns with business priorities.

9. Next Steps

Before This Step

- **EOSS Revenue Computation:** What we need from it: The calculated daily revenue figures for all store-style combinations, including national and hierarchical estimates, and flags for "new" or "read-only" styles.
- **EOSS Data Preparation:** What we need from it: All validated master data (store, product, season information) and comprehensive current inventory levels, including goods-in-transit, and the Percent Stock Available (PSA) for each style.
- **EOSS Segmentation:** What we need from it: The classification of store-style combinations into segments like 'Top Seller' or 'Bottom Seller', which informs the strategic context for the ranking.

After This Step

- **EOSS Allocation & Transfers:** What they need from us: The final, prioritized list of ranked store-style combinations to determine optimal inventory levels (Optimal Depth - OD) and decide where to send new stock or initiate inter-store transfers.
- **EOSS Markdown & Pricing:** What they need from us: The ranking can provide an additional input for identifying items that, despite high stock, are ranked low and may need more aggressive markdown strategies.

10. Common Questions

Q: Why is ranking so important for end-of-season sales? A: Ranking is vital because it brings clarity to which products in which stores need the most attention. During EOSS, quickly moving the right inventory is key to maximizing revenue and avoiding losses from excess stock. A clear rank helps prioritize actions like reallocating bestsellers or marking down slow movers.

Q: How does this ranking help prevent stockouts and overstocking? A: By ranking products, the system identifies high-performing items in specific stores (high rank) that need replenishment to prevent stockouts. Conversely, it highlights low-ranked items, often indicating overstock or low demand, which then become candidates for transfers or markdowns to prevent excess inventory.

Q: What if a new store has no sales history? How does it get ranked? A: For new stores, the system leverages a pre-defined mapping to an established “reference store.” It essentially uses the sales and inventory performance of this reference store to assign a logical initial ranking to styles in the new store, allowing it to receive relevant inventory from day one.

Q: Can we adjust the factors that determine the ranking? A: While the fundamental criteria (revenue, PSA, stock) are fixed, the *relative importance* can be influenced by adjusting related business settings in this module or previous modules. For example, if you want to emphasize financial performance, ensuring high-quality revenue calculations in the “EOSS Revenue Computation” step is key.

Q: How often is this ranking process performed? A: The EOSS Ranking is typically performed as often as the overall EOSS Orchestration runs, often daily or weekly. This ensures that the prioritization list is always based on the most current sales performance and inventory levels, allowing for agile and responsive inventory management.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prioritizes all eligible store-style combinations for efficient EOSS inventory management.
- **Main Input:** Store-level daily revenue, Percent Stock Available (PSA), current total stock, and new store mappings.
- **Main Output:** A definitive, prioritized list of ranked store-style combinations.
- **Warning Signs:** Unclear differentiation between ranks, consistently unexpected rankings for specific products, or new stores receiving illogical stock recommendations.
- **Key Settings:** Revenue Emphasis Weight, PSA Optimization Goal, New Store Ranking Default, and Stock Weighting Factor.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Chapter 7: EOSS Allocation

Last Updated: 2025-05-23

1. Business Objective

EOSS Allocation is the core module responsible for calculating the optimal quantity of each unique item (SKU) to be stocked in every store, running multiple iterations based on various business rules and constraints.

Why This Matters: This module acts as the “brain” of inventory distribution, ensuring that the right products are in the right stores at the right time. By optimizing allocations, it helps maximize sales, minimize stockouts in high-demand areas, reduce excess inventory in slow-moving locations, and ultimately boosts overall profitability during the End-of-Season Sale (EOSS).

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS) for SKUs:** Why we need it: To understand how quickly specific products are selling in each store, which is crucial for determining replenishment needs and prioritizing allocations for fast-moving items.
- **Computed Daily Revenue:** Why we need it: To assess the financial contribution of each product, guiding allocation decisions towards items with higher earning potential.

Product Data

- **Current Inventory Levels (OD):** Why we need it: Knowing the exact number of units of each product (SKU) in every store and warehouse is essential for accurate allocation and identifying available stock for replenishment or transfers, aiming for Optimum Depth (OD).
- **Product Attributes (AG) and Ideal Size Set (ISS):** Why we need it: Details like category, style, and ideal size combinations (ISS) are critical for grouping products, applying planogram rules, and ensuring stores receive a balanced mix of sizes to maximize sales.
- **Optimum Depth (OD) Targets:** Why we need it: Pre-defined ideal stock levels for specific products or categories (Optimum Depth) in different stores are used as targets for allocation, ensuring optimal inventory levels.

- **Planogram Data:** Why we need it: Information on how many units of a specific product type or size can be displayed in a store helps ensure that allocations fit the physical display capacity and visual merchandising standards.

Operational Data

- **Store Configurations:** Why we need it: To identify active stores, their operational capabilities, and any specific rules related to receiving or sending inventory.
- **Allocation Run Type:** Why we need it: To define the primary goal of the allocation process (e.g., fresh allocation for new products, replenishment for existing stock, or replenishment with replacement for clearance).
- **Inter-Store Transfer (IST) Rules:** Why we need it: To determine if and how products can be moved between stores, influencing whether excess stock is pulled back or reallocated.
- **Pullback Criteria:** Why we need it: To define which items are eligible to be transferred *out* of a store (e.g., slow-moving, old season) to make room for new stock or consolidate inventory.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Allocation Run Option	Replenishment	Determines the primary strategy for the allocation run: 'Fresh Allocation' (for new items), 'Replenishment' (to restock existing items), 'Replenishment with Replacement' (to clear old stock while replenishing), or 'Inter-Store Transfer (IST)'.	Change based on your current inventory strategy: use 'Fresh Allocation' for new season launches, 'Replenishment with Replacement' during clearance phases.
Target Optimum Depth (OD) Multiplier	1.0	A factor applied to the calculated Optimum Depth (OD) to determine the ideal stock level for each SKU. For example, 1.0 means stock to the exact OD,	Increase if you want to build up safety stock or anticipate higher demand; decrease to run leaner inventory and prioritize clearance.

Setting Name	Default Value	Description	When to Change
Planogram Compliance Priority	High	1.5 means stock 50% above OD. Defines how strictly the system adheres to planogram limits. 'High' ensures allocations never exceed display capacity; 'Medium' allows slight overages if critical.	Set to 'High' for visually sensitive categories or limited display space; 'Medium' for back stock or less rigid display needs.
Deplete Warehouse Stock Flag	False	When set to 'True', instructs the system to exhaust warehouse stock for allocations even if it means exceeding ideal store stock levels or planogram limits, prioritizing full clearance.	Set to 'True' during aggressive end-of-season clearance to empty warehouses; keep 'False' for regular operations to maintain optimal store stock levels.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Ranked & Segmented Inventory Data] --> B{EOSS Allocation Engine}
    B --> C[Determine Allocation Run Type (Fresh, Replenishment, etc.)]
    C --> D[Run Iterations (Top Seller, Replenishment, Planogram, OD Targets)]
    D --> E[Apply Constraints & Business Rules]
    E --> F[Suggested SKU Allocations & Transfers]
  
```

1. Start with → All the prepared, ranked, and segmented inventory data, including current stock levels, sales rates, and store-specific rules.
2. Then → **Determine** the specific allocation strategy (e.g., fresh allocation for new products, or replenishment with replacement to manage old stock).

3. Next → **Run** a series of smart allocation “iterations,” starting with high-priority items like top sellers, then moving to general replenishment, and considering planogram limits.
4. Then → **Apply** crucial business constraints such as Optimum Depth (OD) targets for each product and Ideal Size Set (ISS) requirements, ensuring a balanced and efficient allocation.
5. Finally → **Generate** the precise suggested allocations for each SKU in every store, along with any necessary inter-store transfer orders.

5. Step-by-Step Actions

1. **Initialize** the allocation run
 - What we do: *We gather all the refined data from previous steps, including product performance (ROS, revenue), inventory levels (OD), segmentation (Top Seller, Bottom Seller), and store-specific operational details.*
 - Why we do it: *This ensures that every allocation decision is based on the most accurate, up-to-date, and context-aware information available.*
2. **Determine** the allocation strategy
 - What we do: *Based on the ‘Allocation Run Option’ setting, the system decides whether to focus on ‘Fresh Allocation’ (for new product introductions), ‘Replenishment’ (to re-stock existing items), ‘Replenishment with Replacement’ (to send new items and pull back old ones), or ‘Inter-Store Transfers’.*
 - Why we do it: *This sets the overall objective for the allocation, guiding how inventory is prioritized and moved to support specific business goals for the EOSS period.*
3. **Run** iterative allocation passes
 - What we do: *The system performs multiple allocation “passes” or iterations. It might first prioritize Top Sellers, ensuring they are adequately stocked. Then, it moves to general replenishment based on ROS and current stock. For ‘Replenishment with Replacement’, it will consider pulling back old stock to make room.*
 - Why we do it: *This multi-pass approach ensures that the most critical and impactful allocation decisions are made first, systematically addressing different inventory needs and optimizing the flow of goods.*
4. **Apply** constraints and targets (OD, Planogram, ISS)
 - What we do: *During each iteration, the system rigorously checks and applies constraints. It ensures that allocations align with the Optimum Depth (OD) targets for each product in a store, respects store-specific planogram capacities, and aims to maintain the Ideal Size Set (ISS) to meet customer demand for all sizes.*
 - Why we do it: *This prevents overstocking, ensures products can be properly displayed, and guarantees that stores have a balanced assortment of sizes, maximizing sales opportunities and customer satisfaction.*
5. **Finalize** allocations and transfers
 - What we do: *After all iterations and constraint checks are complete, the system calculates the final suggested quantity of each SKU to be sent to each store and identifies any necessary inter-store transfers, along with reasons for specific decisions.*
 - Why we do it: *This provides a clear, actionable plan for the operations and logistics teams, enabling efficient and precise movement of inventory to achieve the optimal distribution across the retail network.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- **Store A:** Identified as a 'Top Seller' location for "Women's Denim Jeans" (Style 404). Current stock: 5 units. Optimum Depth (OD) target: 50 units. ROS: 10 units/day. Its planogram capacity for jeans is 60 units.
- **Store B:** Has 80 units of "Women's Denim Jeans" (Style 404). ROS: 1 unit/day. It is a 'Bottom Seller' for this style.
- **Warehouse:** Has 200 units of "Women's Denim Jeans" (Style 404).
- **Allocation Run Option:** 'Replenishment with Replacement' (meaning it can suggest pulling stock from one store to another if beneficial).
- **Target Optimum Depth (OD) Multiplier:** 1.0.

What Happens

1. First, EOSS Allocation identifies that Store A needs a significant replenishment of "Women's Denim Jeans" due to its 'Top Seller' status, high ROS, and low current stock (5 units vs. 50 unit OD target). It also notes Store B's excess stock.
2. Then, it begins its iterations:
 - **Top Seller Iteration:** It prioritizes Store A. It calculates that Store A needs 45 units (50 OD - 5 current) to reach its Optimum Depth. This allocation fits within Store A's 60-unit planogram capacity.
 - **Replenishment with Replacement Iteration:** Since Store B has excess stock (80 units vs. a likely lower OD for a 'Bottom Seller' location), the system considers transferring some of Store B's excess to Store A. It determines that moving 30 units from Store B to Store A is efficient, reducing Store B's excess and fulfilling part of Store A's demand.
3. Next, the system confirms that the remaining 15 units needed by Store A (45 requested - 30 from Store B) can be sourced from the central warehouse, ensuring Store A reaches its Optimal Depth.
4. Finally, the system generates specific allocation orders: 30 units of "Women's Denim Jeans" from Store B to Store A, and 15 units from the Warehouse to Store A.

Final Outcome

Store A, a high-demand location for "Women's Denim Jeans," receives 45 additional units, ensuring it meets its Optimum Depth (OD) and prevents lost sales. Store B significantly reduces its excess inventory, minimizing the need for drastic markdowns. The entire inventory of "Women's Denim Jeans" is optimally balanced across the network, leading to higher sell-through and profitability.

Key Takeaway: EOSS Allocation dynamically balances inventory across stores, prioritizing high-demand locations and efficiently moving excess stock, ensuring each product contributes optimally to sales and profitability.

7. Results

This step produces:

Main Outputs

- **Suggested SKU Allocation Plan:** What it tells us: A detailed list of recommended quantities for each unique item (SKU) to be shipped to every store, along with the source (warehouse or another store), designed to achieve Optimum Depth (OD) and maximize sales.
- **Inter-Store Transfer (IST) Orders:** What it tells us: Specific instructions for moving products between stores, including quantities and reasons, to balance inventory and facilitate clearance or replenishment with replacement.
- **Allocation Remarks and Justifications:** What it tells us: Explanations for specific allocation decisions, such as why an item was allocated more or less than its target (e.g., “planogram limit reached,” “Optimum Depth fulfilled,” “stockout averted”).

Business Reports

- **Store Allocation Summary:** Who uses it and why: Store Managers and Operations teams use this to understand incoming inventory and prepare for new product arrivals or transfers.
- **Overall Inventory Rebalancing Report:** Who uses it and why: Merchandising Planners and Inventory Managers use this to review the strategic impact of the allocation decisions, evaluating how well inventory is being optimized across the entire chain.
- **Warehouse Depletion Forecast:** Who uses it and why: Warehouse Operations and Logistics teams use this to understand expected outgoing stock volumes and manage warehouse space and labor efficiently, especially during aggressive clearance.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The allocation process will pause or fail to initiate, as it cannot proceed without complete and accurate input data from preceding steps.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
Store already at max capacity	The system will cap allocations to that store based on its physical capacity or planogram limits, potentially leaving excess stock in the warehouse or other stores.	Review store capacity settings or planogram definitions. Consider if additional display space can be made, or if the remaining stock should be marked down or pulled back.

Situation	What Happens	What You Should Do
No available stock in warehouse	If a store needs replenishment but the warehouse has no stock, the allocation will show zero units from the warehouse, highlighting a missed sales opportunity.	Investigate supply chain issues or consider if inter-store transfers can fulfill the demand from other stores. If not, consider deeper markdowns in stores with excess.
Planogram conflicts for specific SKUs	The system may be unable to allocate certain SKUs if their sizes or types don't fit the specified planogram limits, even if stock is available.	Review and adjust the planogram definitions for that category or SKU. Consider if display flexibility is needed for that product.
Circular transfers identified (e.g., Store A to B, B to A)	The optimization logic is designed to prevent true circular transfers, but complex scenarios might initially appear inefficient.	The system's optimization typically resolves these; if an issue persists, review the specific transfer criteria and store group settings.

9. Next Steps

Before This Step

- **EOSS Data Preparation:** What we need from it: All validated master data, comprehensive current inventory levels (OD), and Ideal Size Set (ISS) information.
- **EOSS Sales Data Preparation:** What we need from it: Cleaned sales transactions and accurate Rate of Sale (ROS) figures for each SKU and style.
- **EOSS Revenue Computation:** What we need from it: Calculated daily revenue figures for all store-style combinations, including national and hierarchical estimates, and flags for “new” or “read-only” styles.
- **EOSS Segmentation:** What we need from it: The classification of store-style combinations into segments like ‘Top Seller’ or ‘Bottom Seller’, which informs allocation priorities.
- **EOSS Ranking:** What we need from it: The final, prioritized list of ranked store-style combinations to guide the sequence of allocation decisions.

After This Step

- **Execute Inter-Store Transfers:** What they need from us: Detailed transfer orders specifying which products to move between stores and in what quantities.
- **Update Store Inventory Systems:** What they need from us: The final allocation plan to adjust on-hand inventory figures in store systems, reflecting the new stock levels and expected inbound shipments.

- **Warehouse Fulfillment:** What they need from us: Pick lists and shipping instructions for units to be sent from the warehouse to stores based on the allocation plan.

10. Common Questions

Q: How does EOSS Allocation determine the “optimal” number of SKUs for each store? A: It determines the optimal number by considering a blend of factors: the product’s sales performance (ROS), its revenue potential, the store’s current stock relative to its target Optimum Depth (OD), and any physical constraints like planogram capacity. It aims to stock enough to meet demand without overstocking.

Q: Why are there “multiple iterations” in the allocation process? A: Multiple iterations allow the system to apply different business rules and priorities in a structured way. For example, it might first ensure all ‘Top Sellers’ are adequately stocked, then address general replenishment, and finally, handle more complex scenarios like balancing inventory between stores or making room for new products.

Q: How does this module prevent stockouts or overstocking? A: It prevents stockouts by actively identifying high-demand items in stores falling below their Optimum Depth (OD) and prioritizing their replenishment. It prevents overstocking by capping allocations at planogram or store capacity, and by recommending inter-store transfers or pullbacks for products with excess inventory.

Q: Can this module handle initial allocation for new product launches, not just replenishment? A: Yes, EOSS Allocation supports different ‘Allocation Run Options’, including ‘Fresh Allocation’. This mode is specifically designed for initial product drops, allowing new styles to be distributed across stores based on their expected performance and store profile, even without prior sales history.

Q: What happens if a store cannot physically receive the suggested allocation due to limited space? A: The system incorporates planogram data and store capacity limits as crucial constraints. If a suggested allocation exceeds these limits, the system will cap the allocation at the maximum allowable quantity, ensuring that stores do not receive more stock than they can physically display or store.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates optimal SKU allocations to stores, considering sales, inventory, and business rules.
 - **Main Input:** Ranked and segmented store-style data, current inventory levels (OD), Rate of Sale (ROS), revenue, and planogram data.
 - **Main Output:** Suggested SKU Allocation Plan, Inter-Store Transfer (IST) Orders.
 - **Warning Signs:** Persistent overstocking/stockouts despite allocations, allocations not adhering to planograms, or unexpected transfer recommendations.
 - **Key Settings:** Allocation Run Option, Target Optimum Depth (OD) Multiplier, Planogram Compliance Priority, and Deplete Warehouse Stock Flag.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 8: EOSS Warehouse Inventory Management

Last Updated: 2025-05-23

1. Business Objective

EOSS Warehouse Inventory Management is responsible for accurately calculating and preparing the available stock in both physical and “virtual” warehouses, ensuring that only truly available inventory is considered for End-of-Season Sale (EOSS) distribution and inter-store transfers.

Why This Matters: This ensures inventory decisions are based on realistic stock levels, preventing over-allocation of unavailable goods, minimizing unexpected stockouts, and optimizing the flow of products for maximum sales and efficient clearance during the EOSS. It helps maintain Optimal Depth (OD) by ensuring the right stock is accounted for at the right time.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS) for SKUs:** Why we need it: To estimate how much stock needs to be reserved in the warehouse to cover expected sales during the lead time before new replenishments arrive.
- **Historical Sales Trends:** Why we need it: To understand typical sales velocity and ensure that inventory exclusions for lead time are realistically balanced against demand.

Product Data

- **Current Warehouse Inventory Levels:** Why we need it: To know the exact physical stock available in central warehouses for distribution.
- **Current Store On-Hand Inventory:** Why we need it: To identify leftover stock in stores that can be consolidated into “virtual warehouse stock” for inter-store transfers.
- **Style Pack Sizes:** Why we need it: To determine how much store stock can be efficiently pulled back and transferred in whole packs, minimizing logistical inefficiencies.

- **Product Attributes (AG):** Why we need it: To group products, allowing warehouse stock to be reserved based on expected sales within specific categories and for grouping virtual warehouse stock for inter-store transfers.

Operational Data

- **Expected Delivery Lead Time (Days):** Why we need it: To calculate how many days of stock need to be reserved from a warehouse to cover sales while goods are in transit.
- **Warehouse Stock Buffer:** Why we need it: To account for a small percentage of warehouse stock that might be temporarily unavailable or difficult to pick, ensuring realistic available inventory for distribution.
- **Inter-Store Transfer (IST) Groupings:** Why we need it: To define how stores are grouped (e.g., by region, city, or custom group) for inter-store transfers, enabling the creation of relevant “virtual warehouses.”
- **Online Store Identification:** Why we need it: To determine if lead time stock exclusion rules apply to online stores, which often have different inventory management needs.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Warehouse Stock Buffer (%)	5%	The percentage of total warehouse stock to be considered temporarily unavailable or reserved for minor operational inefficiencies.	Increase if you frequently encounter discrepancies or need a larger safety buffer; decrease to maximize immediate available warehouse stock for distribution.
Expected Delivery Lead Time (Days)	7 days	The number of days of expected sales that should be reserved from warehouse stock to cover the time it takes for goods to reach stores.	Adjust based on your actual shipping and receiving times; increase during peak seasons or with unreliable logistics; decrease with faster delivery times.
Inter-Store Transfer (IST) Level	REGION	Defines how stores are grouped to pool leftover stock into “virtual warehouses” for inter-store transfers. Options: REGION, CITY, CUSTOM_GROUP.	Change based on your operational structure for inter-store transfers; e.g., to CITY for localized transfers, CUSTOM_GROUP for specific clusters.

Setting Name	Default Value	Description	When to Change
Minimum Live Days for ROS (for lead time)	7 days	The minimum number of days an SKU must have been available for sale in a store within the analysis period to have its Rate of Sale (ROS) computed for lead time reservation.	Increase to ensure more reliable ROS for lead time calculations; decrease to apply lead time reservation to items with shorter or intermittent availability.

Note: See More Details for additional settings related to ROS period weighting.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Current Inventory & Sales Data] --> B{E0SS Warehouse Inventory Management}
    B --> C[Adjust for Warehouse Inefficiency]
    C --> D[Exclude Stock for Lead Time]
    D --> E{Is IST Mode Active?}
    E -- Yes --> F[Create Virtual Warehouse Stock from Store Leftovers]
    E -- No --> G[Final Available Warehouse Stock]
    F --> G
  
```

1. Start with → The raw current inventory data from both physical warehouses and stores, along with recent sales data.
2. Then → **Adjust** physical warehouse stock by removing a designated “Warehouse Stock Buffer” to account for inefficiencies.
3. Next → **Exclude** a portion of warehouse stock based on the “Expected Delivery Lead Time” and product Rate of Sale (ROS) to cover anticipated sales during transit.
4. Then → **Check** if the system is running in Inter-Store Transfer (IST) mode. If so, it proceeds to calculate “virtual warehouse stock.”
5. Finally → **Create** “virtual warehouse stock” by identifying leftover, transferable stock from stores (adjusted for pack sizes) and pooling it into logical groups (e.g., by region), which then becomes available for inter-store transfers. The final output is a refined, actionable picture of all available warehouse and virtual warehouse stock.

5. Step-by-Step Actions

1. **Remove** warehouse inefficiency
 - What we do: *We reduce the total quantity of stock reported in physical warehouses by a small, predefined “Warehouse Stock Buffer.” This accounts for minor discrepancies or stock that’s not immediately accessible for distribution.*

- Why we do it: *This ensures that subsequent allocation decisions are based on a realistic and achievable quantity of available warehouse stock, preventing over-allocations and managing expectations for immediate fulfillment.*
- 2. **Exclude** inventory for expected delivery lead time
 - What we do: *For each SKU, we calculate its expected sales during the “Expected Delivery Lead Time” (the time it takes for new stock to reach a store). We then deduct this calculated quantity from the available warehouse stock for that SKU, effectively reserving it.*
 - Why we do it: *This crucial step prevents potential stockouts. By reserving stock for sales that will occur while replenishment orders are in transit, we ensure continuous availability of popular items and maintain customer satisfaction.*
- 3. **Group** stores for Inter-Store Transfers (IST)
 - What we do: *If Inter-Store Transfer (IST) is enabled, we create logical “IST groups” for stores (e.g., all stores in a particular region or city). Each group acts as a “virtual warehouse” for peer-to-peer transfers.*
 - Why we do it: *This facilitates efficient transfers between stores by pooling excess stock from multiple stores within a group, making it easier to reallocate items to high-demand locations without involving the central warehouse, which saves costs and time.*
- 4. **Create** virtual warehouse stock from leftover store stock
 - What we do: *We identify excess or leftover stock in individual stores that can be transferred out. This stock is then virtually moved into the corresponding “IST group” (or virtual warehouse), prioritizing full pack sizes for efficient transfers. Any stock that cannot form a full pack is left in the originating store and flagged as “pullback.”*
 - Why we do it: *This maximizes the utilization of inventory already in the retail network. Instead of sending excess back to a central warehouse or marking it down unnecessarily, it’s made available for other stores that might have higher demand, improving overall inventory efficiency and sell-through.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A popular “Blue T-Shirt” (SKU 500) has 1,000 units in the central warehouse.
- The “Warehouse Stock Buffer” is 5%.
- The “Expected Delivery Lead Time” is 7 days.
- The “Blue T-Shirt” sells at an average Rate of Sale (ROS) of 20 units/day across relevant stores.
- Store A has 30 units of the “Blue T-Shirt” left over, and its pack size is 5 units. Store A is part of the “North Region” IST group.
- The system is running in Inter-Store Transfer (IST) mode.

What Happens

1. First, EOSS Warehouse Inventory Management **removes** the “Warehouse Stock Buffer.” 5% of 1,000 units is 50 units. So, the initially available warehouse stock for the “Blue T-Shirt” becomes 950 units (1,000 - 50).
2. Then, it **excludes** stock for the “Expected Delivery Lead Time.” With a 7-day lead time and an ROS of 20 units/day, 140 units ($7 * 20$) are reserved. The available warehouse stock is now 810 units (950 - 140).
3. Next, because IST mode is active, the system considers Store A’s leftover stock. Store A has 30 units. Since the pack size is 5, it can easily create 6 full packs (30 units / 5 units/pack). These 30 units are “virtually” added to the “North Region” IST group’s virtual warehouse stock.
4. Finally, this “virtual warehouse stock” for the “North Region” (containing Store A’s 30 units, plus any other stores in that region that contributed) is combined with the adjusted central warehouse stock (810 units), presenting a comprehensive picture of all available inventory for distribution and transfers.

Final Outcome

The system now has a highly accurate and realistic count of available “Blue T-Shirt” stock. Only 810 units are considered truly available from the central warehouse for immediate allocation, preventing potential stockouts during transit. Additionally, 30 units from Store A are made available in a “virtual warehouse” for efficient inter-store transfers to other stores in the North Region, optimizing inventory within the local cluster without needing to involve the central warehouse.

Key Takeaway: By meticulously adjusting for warehouse inefficiencies, reserving for lead times, and creating “virtual” transfer hubs, this module ensures that every inventory decision is based on the most precise and actionable stock availability.

7. Results

This step produces:

Main Outputs

- **Adjusted Physical Warehouse Stock:** What it tells us: The precise quantity of stock in central warehouses that is truly available for distribution after accounting for inefficiency buffers and lead time reservations.
- **Virtual Inter-Store Transfer (IST) Warehouse Stock:** What it tells us: The aggregated quantity of transferable stock from individual stores, pooled into virtual warehouses based on IST groupings, ready for efficient inter-store transfers.
- **Store Pullback Quantities:** What it tells us: Details on specific quantities of stock left in stores that were identified as excess but could not form full transfer packs, which might be candidates for deeper markdowns or manual handling.

Business Reports

- **Available Warehouse Inventory Report:** Who uses it and why: Logistics and Operations teams use this to understand the exact quantities of product available

for order fulfillment and distribution, ensuring realistic planning and avoiding over-commitment.

- **Inter-Store Transfer Pool Overview:** Who uses it and why: Inventory Managers and Store Operations teams use this to identify available stock within regional clusters for efficient peer-to-peer transfers, reducing the need for central warehouse involvement and speeding up inventory rebalancing.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The process will pause or fail to initiate, as it cannot accurately calculate available inventory without complete and up-to-date sales and stock data.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
Zero sales history for a product	The system may not be able to accurately calculate lead time reservations for new products, potentially leading to insufficient stock being reserved.	Manually review lead time stock needs for new products, or ensure new products are assigned a default Rate of Sale (ROS) or category average to allow for some reservation.
Extremely high lead time or ROS	A significant portion, or even all, of the warehouse stock might be reserved for lead time, showing little to no available stock for new allocations.	Review 'Expected Delivery Lead Time' and 'ROS Period Weightages' settings. This might indicate genuinely high demand/ long lead times, necessitating urgent inbound supply.
Inaccurate 'Style Pack Sizes'	The calculation for "virtual warehouse stock" from store leftovers will be inefficient, either leaving too much untransferred stock or trying to transfer partial packs.	Verify and correct the 'Style Pack Size' master data. Implement strong data validation rules to prevent incorrect entries.
No stores identified for IST grouping	If IST is enabled but no stores are configured for grouping, the virtual warehouse stock calculation will not occur, limiting inter-store transfer capabilities.	Verify 'IST Groupings' in store master data and ensure relevant stores are assigned to a region, city, or custom group.

9. Next Steps

Before This Step

- **EOSS Data Preparation:** What we need from it: All validated master data, comprehensive current inventory levels from both stores and physical warehouses, and detailed style pack sizes.
- **EOSS Sales Data Preparation:** What we need from it: Cleaned and validated sales transactions, along with accurate Rate of Sale (ROS) figures for each SKU and style, essential for lead time calculations.

After This Step

- **EOSS Allocation:** What they need from us: The adjusted and refined inventory data, including both physical warehouse stock and the newly created virtual IST warehouse stock, to make precise allocation and transfer decisions.
- **Warehouse Fulfillment & Inter-Store Transfers:** What they need from us: The final allocation plans and inter-store transfer orders generated by subsequent modules, which will rely on the accurate stock counts prepared here for execution.

10. Common Questions

Q: Why is it important to “remove warehouse inefficiency” before distributing stock? A: Removing warehouse inefficiency, by applying a small buffer, means we only consider stock that is truly available and easily accessible. This prevents us from promising inventory that can’t be fulfilled, avoids frustrating stockouts, and makes the distribution plan more realistic and efficient.

Q: How does reserving stock for “Expected Delivery Lead Time” help the business? A: By reserving stock based on expected sales during the time it takes for replenishment to arrive, we proactively prevent potential stockouts. This ensures that popular items remain available to customers even while new shipments are in transit, leading to consistent sales and higher customer satisfaction.

Q: What is “virtual warehouse stock,” and why do we create it? A: “Virtual warehouse stock” is usable inventory that’s identified as excess in one store and then pooled together with similar excess stock from other stores within the same region or group. We create it to facilitate efficient inter-store transfers. It allows stores to help each other out, sending stock directly where it’s needed most without the cost and time delay of sending it back to a central warehouse first.

Q: How do “Style Pack Sizes” impact the virtual warehouse stock? A: Style Pack Sizes ensure that when we pull stock from stores to create virtual warehouse inventory, we only consider quantities that can be efficiently transferred in full packs. This prevents us from having odd, untransferable quantities that are difficult to manage logistically, making the transfer process smoother and more cost-effective.

Q: What happens if sales data is insufficient for lead time calculations? A: If a product has insufficient sales data, it might not have an accurate Rate of Sale (ROS), which is critical for calculating lead time reservations. In such cases, the system might default to a lower or zero reservation, potentially leading to a higher risk of stockouts for that particular item. It's crucial to ensure robust sales data collection and historical analysis for new products or those with sporadic sales.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prepares precise available inventory figures from physical warehouses and creates “virtual” stock pools from store leftovers for EOSS distribution and inter-store transfers.
- **Main Input:** Current physical warehouse and store inventory, SKU Rate of Sale (ROS), Expected Delivery Lead Time, Style Pack Sizes.
- **Main Output:** Adjusted Physical Warehouse Stock, Virtual Inter-Store Transfer (IST) Warehouse Stock, Store Pullback Quantities.
- **Warning Signs:** Unexpectedly low available warehouse stock, large amounts of untransferable leftover store stock, or products experiencing stockouts despite reported warehouse availability.
- **Key Settings:** Warehouse Stock Buffer (%), Expected Delivery Lead Time (Days), Inter-Store Transfer (IST) Level, Minimum Live Days for ROS (for lead time).

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 9: EOSS Inter-Store Transfer (IST)

Last Updated: 2025-05-23

1. Business Objective

EOSS Inter-Store Transfer (IST) precisely manages the movement of inventory between retail stores, specifically calculating net excess stock, identifying effective transfer opportunities, and suggesting pullbacks to central warehouses for items that cannot be reallocated.

Why This Matters: This ensures that products are efficiently moved to where demand is highest, maximizing Rate of Sale (ROS) and reducing lost sales from stockouts in high-performing stores. Simultaneously, it helps clear excess inventory from underperforming locations, minimizing the need for deep markdowns and optimizing overall inventory investment during the End-of-Season Sale (EOSS).

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS) for SKUs:** Why we need it: To assess how quickly individual products are selling in each store, which helps determine which stores need stock and which have excess.
- **Computed Daily Revenue:** Why we need it: To understand the financial value of each item, guiding transfers towards higher-potential sales locations.

Product Data

- **Current Inventory Levels (OD):** Why we need it: To know the exact number of units of each product (SKU) in every store and warehouse, identifying where stock is abundant or scarce relative to Optimum Depth (OD).
- **Product Attributes (AG):** Why we need it: Details like category, style, and size are crucial for grouping products and ensuring that Ideal Size Set (ISS) requirements are met when transfers occur.
- **Style Pack Sizes:** Why we need it: To facilitate efficient transfers by ensuring products are moved in logical, full-pack quantities, minimizing logistical complexities.

Operational Data

- **Store Configurations:** Why we need it: To identify which stores are active for transfers, their specific geographic or logical groupings (e.g., region, city), and any individual transfer eligibility rules.
- **Allocation Suggestions:** Why we need it: To understand the desired optimal stock levels (Optimum Depth) for each store, indicating where inventory is needed or where it is overstocked.
- **Warehouse-Store Mapping:** Why we need it: To accurately identify the central warehouse as the destination for excess stock that cannot be transferred directly between stores.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Allocation Run Option	Replenishment	Defines the primary purpose of the current distribution run. For inter-store transfers, this should be set to 'Inter-Store Transfer (IST)'.	Change to 'Inter-Store Transfer (IST)' when the primary goal is to rebalance inventory across stores, rather than replenishing from a warehouse.
Inter-Store Transfer (IST) Level	REGION	Determines how stores are grouped for peer-to-peer transfers. Stock is first sought from other stores within the same defined group (e.g., 'REGION', 'CITY', or 'CUSTOM_GROUP').	Adjust based on your operational structure and logistics capabilities; e.g., to 'CITY' for highly localized transfers, or 'CUSTOM_GROUP' for specific store clusters.
IST Replacement Strategy	False	When 'True', the system will prioritize transferring new or performing inventory into stores while also seeking to transfer out older or underperforming stock of the same style.	Set to 'True' during aggressive clearance periods to actively swap out old stock with new demand-driven inventory via transfers.
IST Planogram Increment	False	When 'True', allows the system to temporarily increase a store's displayed planogram capacity if needed to receive crucial incoming transfers during IST,	Set to 'True' if you prioritize moving stock to high-demand stores even if it means slightly exceeding standard display limits temporarily.

Setting Name	Default Value	Description	When to Change
		optimizing for immediate sales.	

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Current Inventory, Sales Data & Store Groupings] --> B[Calculate Net Outward Quantities from Stores]
    B --> C[Identify Inter-Store Transfers (within group)]
    C --> D[Suggest Pullbacks to Central Warehouse]
    D --> E[Final Inter-Store Transfer & Pullback Orders]
  
```

1. Start with → The current inventory levels in all stores and warehouses, recent sales data, and how stores are logically grouped for transfers.
2. Then → **Calculate** the net available stock for outward transfer from each store, factoring in any recent inbound transfers to ensure accurate availability.
3. Next → **Identify** inter-store transfer opportunities by matching stores with excess stock to stores needing stock within the same defined transfer group (e.g., region or city).
4. Finally → **Determine** which remaining excess stock from stores cannot be transferred between stores and should be pulled back to the central warehouse. These decisions are then compiled into clear transfer and pullback orders.

5. Step-by-Step Actions

1. **Initialize** the Inter-Store Transfer (IST) process
 - What we do: *We gather all the necessary data, including current inventory levels, Rate of Sale (ROS), Optimum Depth (OD) targets, and store-specific transfer eligibility flags.*
 - Why we do it: *This ensures that every transfer decision is based on the most accurate, up-to-date, and context-aware information available for the EOSS period.*
2. **Calculate** net outward quantities from stores
 - What we do: *For each product in each store, we reconcile any recent inbound transfers against the stock identified for outward movement. This determines the true “net” quantity available for transfer out of that specific store.*
 - Why we do it: *This prevents misallocation by ensuring that a store isn't directed to send out stock that it has recently received or is otherwise committed, leading to more accurate and reliable transfer instructions.*
3. **Identify** inter-store transfer opportunities
 - What we do: *The system looks for matches between stores that have excess inventory (identified as candidates for outward transfer) and stores that need that same inventory (to meet their Optimum Depth or address a stockout). It prioritizes matching stores within the same defined transfer group (e.g., region or city) for efficiency.*

- Why we do it: *This optimizes inventory movement by facilitating direct peer-to-peer transfers, reducing transit times and costs compared to sending stock back to a central warehouse and then re-shipping it.*
- 4. **Determine** pullbacks to central warehouse
 - What we do: *For any remaining excess inventory in stores that could not be matched with a needing store during the inter-store transfer matching process, the system designates this stock for pullback to the central warehouse.*
 - Why we do it: *This ensures that all identified excess inventory is eventually consolidated, either by finding a direct store recipient or by returning it to a central location, preventing prolonged overstocking in individual stores.*
- 5. **Generate** final transfer and pullback orders
 - What we do: *The system compiles all the identified inter-store transfers and warehouse pullbacks into clear, actionable orders, detailing the specific products, quantities, and origin/destination locations.*
 - Why we do it: *This provides a definitive plan for logistics and store operations teams, enabling them to execute the optimized inventory movements swiftly and accurately.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular "Light Wash Denim Jacket" (SKU 789) is a 'Top Seller' in Store X (current stock: 5 units, Optimum Depth (OD) target: 30 units, Rate of Sale (ROS): 7 units/day). Store X is part of the "South East Region" IST group.
- Store Y, also in the "South East Region," has 40 units of the same "Light Wash Denim Jacket." It's a 'Bottom Seller' for this item (ROS: 1 unit/day), and its OD is 10 units. Store Y received 5 units of this jacket from another store last week.
- Store Z, in the "Midwest Region," has 25 units of a 'Seasonal Graphic Tee' (SKU 101) which is now a 'Bottom Seller' (ROS: 0.2 units/day). No other store in the "Midwest Region" needs this tee.

What Happens

1. First, EOSS Inter-Store Transfer (IST) initializes, gathering all relevant data including Store X's need, Store Y's excess, and Store Z's excess.
2. Then, it **calculates net outward quantities**. For Store Y, it notes that while it has 40 units, 5 were recently received. Its *net* excess for outward transfer is thus 35 units.
3. Next, it **identifies inter-store transfer opportunities**. It sees Store X needs 25 units (30 OD - 5 current stock) of the "Light Wash Denim Jacket." It identifies Store Y as having excess *within the same* "South East Region" IST group. It determines that 25 units can be efficiently transferred from Store Y to Store X.
4. Finally, it **determines pullbacks to the central warehouse**. For Store Z's "Seasonal Graphic Tee," no other store in the "Midwest Region" or any other region is identified as needing this item. Therefore, the system suggests a pullback of all 25 units of SKU 101 from Store Z to the central warehouse.

Final Outcome

Store X, a high-demand location, receives 25 additional “Light Wash Denim Jackets” directly from Store Y, preventing potential lost sales and maximizing the item’s Rate of Sale (ROS). This is achieved faster and more cost-effectively than shipping from a central warehouse. Store Y reduces its excess inventory, freeing up valuable shelf space. Store Z efficiently clears its unsellable ‘Seasonal Graphic Tee’ stock by sending it back to the warehouse for consolidation or further action. Overall, inventory is dynamically balanced, leading to higher sell-through and profitability across the chain.

Key Takeaway: EOSS Inter-Store Transfer efficiently rebalances inventory across stores and consolidates unmatchable stock, maximizing sales potential and minimizing losses during end-of-season periods.

7. Results

This step produces:

Main Outputs

- **Inter-Store Transfer (IST) Orders:** What it tells us: Detailed instructions for moving specific products (SKUs) and quantities between named source and destination stores.
- **Warehouse Pullback Orders:** What it tells us: Clear directives for returning designated excess inventory from specific stores back to the central warehouse.
- **Action Required Store-SKU List:** What it tells us: A list of products that were identified for outward transfer but did not find an in-store recipient or were not pulled back to the warehouse, indicating a need for further manual review or intervention (e.g., deeper markdown).

Business Reports

- **Regional Transfer Summary:** Who uses it and why: Regional Managers and Logistics teams use this to monitor the volume and effectiveness of inter-store transfers within their areas, ensuring efficient resource allocation for stock movement.
- **Excess Inventory Pullback Report:** Who uses it and why: Inventory Managers and Warehouse Operations teams use this to track how much slow-moving or excess inventory is being consolidated from stores, aiding in warehouse planning and overall clearance strategies.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The inter-store transfer process will pause or fail to initiate, as it cannot accurately	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.

Situation	What Happens	What You Should Do
No viable inter-store transfers identified	determine transfer needs or available stock without complete and up-to-date input data.	
	The system completes the process without recommending any store-to-store transfers, even if some stores have excess stock.	Review the 'Inter-Store Transfer (IST) Level' setting; it might be too restrictive. Also, check if there are sufficient stores needing stock within the same groups, or if all excess stock is already targeted for pullback.
Stores unable to receive/send transfers	If a store is flagged as ineligible for inward or outward transfers, it will be excluded from the IST process, potentially leaving stock unbalanced.	Verify the store's transfer eligibility settings in the master data. Adjust these settings if the store should participate in transfers.
Excessive quantities designated for pullback	A large volume of inventory is suggested for pullback to the warehouse, indicating that many items could not find a suitable inter-store recipient.	Review the overall inventory strategy and store grouping logic. This might highlight a widespread imbalance or that the 'IST Level' is too narrow, preventing efficient peer-to-peer transfers.
Circular transfers detected or proposed	The optimization logic is designed to prevent true circular transfers (e.g., Store A to B, B to A for the same SKU). If an issue persists, it indicates a complex data conflict or configuration problem.	While the system works to avoid this, if suspected, review the specific transfer criteria and store grouping settings for the affected SKUs to ensure logical flow.

9. Next Steps

Before This Step

- **EOSS Data Preparation:** What we need from it: All validated master data, comprehensive current inventory levels (Optimum Depth), Ideal Size Set (ISS) information, and detailed style pack sizes.
- **EOSS Sales Data Preparation:** What we need from it: Cleaned sales transactions and accurate Rate of Sale (ROS) figures for each SKU and style.
- **EOSS Revenue Computation:** What we need from it: Calculated daily revenue figures for all store-style combinations, which help prioritize transfer decisions.
- **EOSS Segmentation:** What we need from it: The classification of store-style combinations into segments like 'Top Seller' or 'Bottom Seller', guiding transfer priorities.

- **EOSS Ranking:** What we need from it: The final, prioritized list of ranked store-style combinations, dictating the order in which transfer opportunities are evaluated.
- **EOSS Allocation:** What we need from it: The initial optimal allocation suggestions for each store and SKU, which inform the deficit/surplus positions that drive IST.
- **EOSS Warehouse Inventory Management:** What we need from it: The “virtual warehouse stock” created from store leftovers within IST groupings, and the adjusted physical warehouse stock.

After This Step

- **Execute Inter-Store Transfers:** What they need from us: Detailed, actionable transfer orders to physically move products between stores.
- **Update Store Inventory Systems:** What they need from us: The final transfer and pullback plans to update on-hand inventory figures in store systems, accurately reflecting new stock levels.
- **Warehouse Operations:** What they need from us: Pick lists and shipping instructions for units being pulled back from stores to the central warehouse.

10. Common Questions

Q: What is the main benefit of inter-store transfers during EOSS? A: The main benefit is quickly moving desirable products to stores where they will sell fastest, preventing stockouts and maximizing revenue. It also clears excess inventory from slower-selling stores, reducing the need for heavy discounts and optimizing physical space.

Q: How is inter-store transfer different from regular replenishment from the warehouse? A: Regular replenishment typically sends stock from a central warehouse to stores. Inter-store transfer (IST) facilitates peer-to-peer transfers directly between stores, often within the same region. This saves time and shipping costs by avoiding a round trip to the central warehouse.

Q: What happens if a store has excess inventory of a product, but no other store needs it? A: If a product has excess inventory in a store and no other store in the network is identified as needing it (based on its Optimal Depth (OD) or Rate of Sale (ROS)), the system will suggest that stock be pulled back to the central warehouse for broader distribution, consolidation, or eventual markdown.

Q: How are stores grouped for these transfers? Can we customize it? A: Stores are grouped based on a configurable setting called ‘Inter-Store Transfer (IST) Level’. This can be defined by geographical regions, cities, or even custom groups you create. Yes, you can customize this setting to align with your logistical capabilities and business strategy.

Q: Can we prevent certain stores from participating in inter-store transfers? A: Yes, the system includes eligibility criteria at the store level that can disable a store from either receiving transfers (inwarding) or sending transfers (outwarding). This allows for flexibility in managing stores that may have logistical constraints or specific operational roles during EOSS.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Efficiently rebalances inventory by moving products directly between stores and consolidating excess stock to the warehouse during EOSS.
- **Main Input:** Current inventory levels, sales data, store groupings, and initial allocation suggestions.
- **Main Output:** Detailed Inter-Store Transfer (IST) Orders and Warehouse Pullback Orders.
- **Warning Signs:** Too many items designated for pullback, no viable transfers identified, or stores with disabled transfer eligibility.
- **Key Settings:** Allocation Run Option (set to IST), Inter-Store Transfer (IST) Level, IST Replacement Strategy.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 10: EOSS Planogram Incrementer

Last Updated: 2025-05-23

1. Business Objective

EOSS Planogram Incrementer calculates and applies dynamic adjustment ratios to a store's Optimum Depth (OD) (the ideal number of unique items to stock) and Optimum Width (OW) (the ideal number of different categories or styles to offer) targets. It does this by factoring in revenue plans and Average Selling Prices (ASPs) at various levels, such as store, category, and Attribute Group (AG).

Why This Matters: This ensures that your stores are always stocking the right amount of the right products based on actual demand and available inventory. It helps prevent lost sales due to insufficient stock, optimizes shelf space, and maximizes revenue during the End-of-Season Sale (EOSS) by making sure that planogram targets are flexible and responsive to real-time sales opportunities and inventory availability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Records:** Why we need it: To calculate the Average Selling Price (ASP) for products at different levels (store, category, AG) by understanding past units sold and their prices.
- **Projected Revenue Plans:** Why we need it: To understand the anticipated sales value for specific product categories and Attribute Groups (AGs) in each store, which helps estimate future demand.

Product Data

- **Current Planogram Configurations:** Why we need it: To know the existing Optimum Depth (OD) and Optimum Width (OW) targets, or simply the current stock allocations, for different products or categories in each store.
- **Product Attribute Group (AG) Mappings:** Why we need it: To group products by common characteristics (like 'color', 'material', 'gender') and calculate demand and ASP at these aggregated levels.

- **Warehouse Inventory Levels:** Why we need it: To know the total quantity of products available in central warehouses, as this influences how much additional stock can be allocated to stores.
- **Optimum Depth (OD) Segmentation Data:** Why we need it: To provide the base revenue contribution for different Attribute Groups (AGs) in each store, which is used in projecting future demand.

Operational Data

- **Store Details:** Why we need it: To understand each store's channel (e.g., online, brick-and-mortar) and region, allowing for accurate grouping and analysis of demand.
- **Analysis Period Data:** Why we need it: To specify the timeframes for which sales data should be considered when calculating Average Selling Price (ASP) and revenue projections.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Revenue Plan Horizon	3 months	Defines the look-back period for the revenue plans used to estimate future demand.	Adjust based on product lifecycle and seasonality; a shorter period (e.g., 1 month) for fast-fashion, a longer period (e.g., 6 months) for evergreen products.
ASP Calculation Period	Last 90 days	The duration of recent sales data considered when calculating the Average Selling Price (ASP).	Shorten for rapidly changing prices or promotions; lengthen for stable pricing to get a more averaged value.
Planogram Attribute 1 (AG1)	'Color'	Specifies the primary Attribute Group (AG) characteristic (e.g., 'Color', 'Material') used for granular planogram adjustments.	Change if a different attribute becomes more critical for merchandising or stock decisions (e.g., 'Fabric Type' for apparel).
Planogram Attribute 2 (AG2)	'Fit'	Specifies the secondary Attribute Group (AG) characteristic	Adjust if you need even more granular control over planogram allocations, such as

Setting Name	Default Value	Description	When to Change
		(e.g., 'Fit', 'Style') used for finer planogram adjustments.	for specific sub-styles or product variations.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Revenue Plans, Sales & Inventory Data, Planogram Configurations] --> B[Calculate Projected Revenue & Average Selling Prices (ASP)]

B --> C[Compute Projected Demand Volume (Quantum)]

C --> D[Calculate Total Available Inbound Stock]

D --> E[Determine Planogram Increment Ratios]

E --> F[Published Planogram Increment Ratios]

1. Start with → The prepared revenue plans, historical sales records, current inventory levels, and existing planogram configurations.
2. Then → **Calculate** projected revenue for each product category at various levels (store, category, and Attribute Groups) and determine the Average Selling Prices (ASP) based on historical sales.
3. Next → **Compute** the projected demand volume (often referred to as 'Quantum'), which is derived by dividing the projected revenue by the calculated ASP.
4. Then → **Calculate** the total available inbound stock from all warehouses for each product category and Attribute Group.
5. Finally → **Determine** the Planogram Increment Ratios by comparing the projected demand at the store level against current planogram capacities, and by considering the overall availability of stock from warehouses relative to regional demand. These ratios are then published for use in subsequent steps.

5. Step-by-Step Actions

1. **Calculate** projected revenue
 - What we do: *We estimate the anticipated sales revenue for each product category and Attribute Group (AG) within a store. This uses pre-defined revenue plans, often based on historical performance and overall business targets.*
 - Why we do it: *This provides a financial forecast for each product type, which is crucial for understanding its potential contribution to overall sales and for determining how much stock it should* ideally generate in units.**
2. **Determine** Average Selling Prices (ASP)
 - What we do: *We compute the average price at which items in each category and Attribute Group (AG) have sold historically in a given store or across a sales channel.*
 - Why we do it: *ASP is essential because it allows us to convert the projected revenue (a dollar value) into a projected quantity* of units (demand volume*

or ‘quantum’) that we expect to sell, which is directly relevant to stocking levels.*

3. Compute projected demand volume (‘Quantum’)

- What we do: *We divide the calculated projected revenue by the Average Selling Price (ASP) for each store, category, and Attribute Group. This gives us the estimated number of units of demand for each product type.*
- Why we do it: *This ‘Quantum’ represents the true projected unit demand. It’s the most direct indicator of how many units of a specific product type a store is expected to sell, guiding adjustments to its Optimum Depth (OD) and Optimum Width (OW).*

4. Identify total available inbound stock

- What we do: *We sum up the total quantity of each product category and Attribute Group (AG) that is currently available in all central warehouses and ready for distribution.*
- Why we do it: *Knowing the total available stock is critical because it tells us if we actually can* fulfill the projected demand. If demand is high but stock is low, the increment may be limited; if both are high, we can be more aggressive.**

5. Calculate Planogram Increment Ratios

- What we do: *For each product category or Attribute Group in a store, we determine an ‘increment ratio’. This is calculated by first comparing the store’s projected demand volume (‘Quantum’) against its current planogram stock. This is then adjusted based on the overall availability of inbound stock (from warehouses) relative to the total projected demand across the entire channel and region.*
- Why we do it: *This sophisticated calculation produces a multiplier (the ‘increment ratio’) that tells us by how much to increase (or decrease) the existing planogram targets. It ensures that stores are directed to stock more of items that have high demand and are well-supported by available inventory, dynamically optimizing their Optimum Depth (OD) and Optimum Width (OW).*

6. Publish the increment ratios

- What we do: *The calculated Planogram Increment Ratios are made available for use by subsequent modules, such as the Allocation module, which will apply these ratios to adjust actual inventory targets.*
- Why we do it: *This ensures that the dynamic adjustments determined by this module are promptly integrated into the broader inventory distribution process, leading to agile and efficient inventory management.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Store A, Category “Dresses”:** Current planogram stock (OD) for dresses is 50 units.
- **Projected Revenue for “Dresses” in Store A:** \$5,000 for the upcoming period.
- **Average Selling Price (ASP) for “Dresses” in Store A:** \$50 per unit.
- **Total warehouse stock for “Dresses”:** 10,000 units.
- **Total projected demand for “Dresses” across the entire “North-East Region” (Store A’s region and channel):** 5,000 units.

What Happens

1. First, the system **calculates projected revenue** for Dresses in Store A (\$5,000) and **determines the ASP** (\$50).
2. Then, it **computes the projected demand volume (Quantum)** for Dresses in Store A: $\$5,000 \text{ (Revenue)} / \$50 \text{ (ASP)} = 100 \text{ units}$. This means Store A is projected to sell 100 units of dresses.
3. Next, it **identifies the total available inbound stock** for “Dresses” from the warehouse: 10,000 units.
4. Finally, it **calculates the Planogram Increment Ratio** for “Dresses” in Store A:
 - It first compares Store A’s projected demand (100 units) to its current planogram stock (50 units). This suggests that Store A’s demand is 2 times its current stock ($100/50 = 2.0$).
 - It then considers the overall availability: The warehouse has 10,000 units of dresses, while the total projected demand for dresses across the entire North-East Region is 5,000 units. This means there’s 2 times more stock than regional demand ($10,000/5,000 = 2.0$).
 - The overall increment ratio for Store A’s Dresses is then calculated as $1.0 + (\text{Regional Stock Coverage} * \text{Store Demand vs. Planogram}) = 1.0 + (2.0 * 2.0) = 1.0 + 4.0 = 5.0$.

Final Outcome

The EOSS Planogram Incrementer suggests that the planogram targets (Optimum Depth and Optimum Width) for “Dresses” in Store A should be increased by a factor of 5.0. This significant increment guides subsequent allocation processes to send substantially more “Dresses” to Store A, ensuring it can meet its high projected demand and fully capitalize on sales opportunities, supported by ample warehouse stock.

Key Takeaway: By dynamically adjusting planogram targets based on projected demand and available stock, the system ensures stores are equipped to maximize sales for high-potential items, preventing lost revenue during critical sales periods.

7. Results

This step produces:

Main Outputs

- **Planogram Increment Ratios:** What it tells us: Numerical multipliers for various store-category and Attribute Group (AG) combinations, indicating by how much their Optimum Depth (OD) and Optimum Width (OW) targets should be adjusted.
- **Detailed Calculation Metrics:** What it tells us: The underlying values for projected revenue, Average Selling Price (ASP), and projected demand volume (‘Quantum’) for each combination, providing transparency into how the ratios were derived.

Business Reports

- **Dynamic Planogram Adjustment Recommendations:** Who uses it and why: Merchandising Planners and Store Operations teams use this to understand where and by how much Optimum Depth (OD) and Optimum Width (OW) targets should be increased or decreased, enabling them to make informed decisions about stocking and display.
- **Inventory Opportunity Map:** Who uses it and why: Executive Leadership and Sales Managers use this to visualize areas where there is significant sales potential that can be unlocked by adjusting inventory levels, guiding strategic growth initiatives.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The planogram incrementer process will pause or fail to initiate, as it cannot proceed with calculations without complete and accurate input data from preceding steps.	Verify data sources, contact the IT or data team to resolve the feed issue, and re-run the process once data is available.
Zero or extremely low Average Selling Price (ASP)	This can lead to an artificially inflated 'Quantum' (projected demand volume) or division by zero errors, resulting in unrealistic increment ratios.	Investigate the sales data for these products to ensure accurate pricing. It might indicate data quality issues or a need for manual review of promotions.
No projected demand ('Quantum') for a category/AG	If there is no projected revenue or demand for a category/AG, its increment ratio will likely remain at the default 1.0, meaning no adjustment.	Review the revenue plans and sales history for these categories. It might genuinely indicate low demand, or a gap in the planning data.
No planogram stock for a category/AG	If current planogram data indicates zero stock, this can cause division by zero errors in the ratio calculation.	Ensure planogram configurations are always accurate and reflect a baseline, even if it's minimal stock. The system usually handles this by assigning a default or skipping the calculation.
Total available inbound stock is zero	If the warehouse has no stock, the system might not suggest significant increments, even for high-demand items, as	Investigate supply chain issues. The incrementer correctly reflects the reality of no available stock, but the business needs to address the underlying supply problem.

Situation	What Happens	What You Should Do
	fulfillment is impossible.	

9. Next Steps

Before This Step

- **EOSS Revenue Computation:** What we need from it: The calculated daily revenue figures and the overall projected revenue contribution (from `OdSegmentationRow`) for various store-category and Attribute Group (AG) combinations.
- **EOSS Sales Data Preparation:** What we need from it: Cleaned and validated sales transactions from various periods, essential for calculating accurate Average Selling Prices (ASPs).
- **EOSS Data Preparation:** What we need from it: All validated master data, including SKU details, Attribute Group (AG) definitions, store configurations (channel, region), and current warehouse inventory levels, along with existing planogram configurations.

After This Step

- **EOSS Allocation:** What they need from us: The calculated Planogram Increment Ratios to dynamically adjust the target Optimum Depth (OD) and Optimum Width (OW) for each store-SKU combination when making final allocation decisions.
- **Planogram System Update:** What they need from us: The updated Optimum Depth (OD) and Optimum Width (OW) targets, influenced by these increment ratios, to reflect the dynamic adjustments in the central planogram management system.

10. Common Questions

Q: What is a “Planogram Incrementer,” and why is it important for EOSS? A: A “Planogram Incrementer” is a tool that calculates how much to adjust a store’s target stock levels (Optimum Depth or OD) and product variety (Optimum Width or OW) based on actual demand and available inventory. It’s crucial for EOSS because it helps you quickly capitalize on high-selling items by stocking more, while ensuring you don’t over-invest in slow movers.

Q: How does this module help improve sales and profitability during EOSS? A: By dynamically increasing the target stock for popular products in high-demand stores, it prevents stockouts and ensures you capture every sale. Simultaneously, by not suggesting increments for slow-moving items, it prevents overstocking, which reduces the need for heavy discounts and protects your profit margins.

Q: What happens if the Average Selling Price (ASP) for a product changes frequently? A: If the ASP changes frequently, the system will adapt by using a shorter 'ASP Calculation Period' setting, which can be adjusted. This ensures that the ASP used for calculating projected unit demand ('Quantum') is always as up-to-date as possible, making the planogram adjustments more accurate.

Q: How does the system decide *how much* to increment the planogram targets? A: The system looks at two main factors: how much a specific store's projected demand exceeds its current planogram capacity, and how much overall stock is available in the warehouse relative to the total demand across a wider region. It then uses these two factors to calculate a multiplier (the increment ratio) that gets applied to the current planogram targets.

Q: Can these dynamic planogram adjustments be manually overridden? A: While the system automatically calculates these ratios to ensure efficiency and consistency, the final application and approval of these adjustments often involve human oversight from merchandising planners. Any manual overrides would typically be managed in the system that receives these recommended adjustments (e.g., the Allocation module or a dedicated planogram management system).

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Dynamically adjusts Optimum Depth (OD) and Optimum Width (OW) targets for stores based on projected demand, sales, and available inventory.
- **Main Input:** Projected revenue plans, Average Selling Prices (ASP), current planogram configurations, and warehouse inventory levels.
- **Main Output:** Calculated Planogram Increment Ratios for various hierarchical levels.
- **Warning Signs:** Unrealistic or zero increment ratios, or a lack of dynamic adjustments when market conditions suggest they are needed.
- **Key Settings:** Revenue Plan Horizon, ASP Calculation Period, Planogram Attribute 1 (AG1), Planogram Attribute 2 (AG2).

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 11: EOSS Output Generation

Last Updated: 2025-05-23

1. Business Objective

EOSS Output Generation is responsible for fanning out and persisting the final allocation (Optimum Depth - OD, Optimum Width - OW, Ideal Size Set - ISS) and transfer outputs into various detailed and denormalized tables, transforming internal models into report-friendly formats.

Why This Matters: This critical step ensures that every inventory decision made by the EOSS system is accessible, understandable, and actionable for your merchandising, store operations, and finance teams. It supports effective execution of allocation and transfer plans and provides clear, comprehensive data for reporting and analysis, directly impacting your ability to maximize sales and efficiently manage inventory during End-of-Season Sales.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS) for SKUs:** Why we need it: To enrich the final output reports by providing the selling speed context for each allocated or transferred item, explaining why certain decisions were made.
- **Computed Daily Revenue:** Why we need it: To include the financial contribution of each product in the final reports, allowing business users to understand the monetary impact of allocation and transfer strategies.

Product Data

- **Optimum Depth (OD) Targets:** Why we need it: To present the target stock levels versus the actual allocated quantities in reports, providing transparency on how well Optimum Depth was achieved.
- **Ideal Size Set (ISS) Information:** Why we need it: To provide detailed information on how size sets were considered in allocations or transfers, ensuring comprehensive reporting on Ideal Size Set (ISS) adherence and its impact on Optimum Width (OW).
- **Product Attributes (AG):** Why we need it: To group and present allocation and transfer data by relevant Attribute Groups (AGs) in denormalized reports, enabling analysis at various hierarchical levels.

Operational Data

- **Final Allocation Plan (OD, OW, ISS):** Why we need it: This is the core input from the EOSS Allocation module, containing the precise recommendations for what to stock where, which needs to be published.
- **Inter-Store Transfer (IST) Orders:** Why we need it: The definitive instructions for moving products between stores or to/from warehouses, generated by the EOSS Allocation and IST modules, which must be clearly documented.
- **Allocation Remarks and Justifications:** Why we need it: To include the specific reasons (e.g., “planogram limit reached,” “top seller replenishment”) behind each allocation or transfer decision in the output reports for auditing and clarity.
- **Store Configurations:** Why we need it: To provide contextual details about each store (e.g., its channel, region) in the final reports, enabling more granular analysis of inventory movements.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Include Detailed Remarks in Reports	True	Determines if specific explanations for allocations (e.g., “planogram limit reached,” “top seller”) are included in the final output tables for auditing.	Set to ‘False’ if detailed remarks are not needed for certain reports to simplify data, or ‘True’ for full transparency and debugging.
Output Granularity Level	SKU	Defines the lowest level of detail for the primary outputs: ‘SKU’ for individual item-level, ‘STYLE’ for style-level roll-ups, or ‘CATEGORY’ for broader groupings.	Adjust to ‘STYLE’ or ‘CATEGORY’ if you primarily need aggregated views for strategic analysis rather than granular operational details.
Separate IST Outputs	True	Specifies whether Inter-Store Transfer data is published in distinct, dedicated	Set to ‘False’ if you prefer all inventory movements (replenishment and transfers) to be consolidated into a single report for

Setting Name	Default Value	Description	When to Change
		reports for logistics and separate transfer analysis.	simpler, holistic review.
Denormalize for BI Tools	True	Controls whether the final outputs are structured in denormalized tables, optimized for direct consumption by business intelligence and reporting tools.	Set to 'False' if your downstream systems prefer a more normalized data structure, or if you plan to perform complex joins yourself.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Final Allocation & Transfer Decisions] --> B[Transform & Enrich Data for Reporting]
    B --> C[Persist Detailed Allocation Outputs]
    B --> D[Persist Detailed Transfer Outputs]
    C --> E[Generate Denormalized Reports]
    D --> E
    E --> F[Comprehensive EOSS Output Data Set]
  
```

1. Start with → The comprehensive set of final allocation plans (Optimum Depth, Optimum Width, Ideal Size Set) and inter-store transfer orders generated by preceding EOSS modules.
2. Then → **Transform** and **enrich** this raw data, adding critical business context such as segmentation categories, specific allocation types, and detailed remarks explaining each decision.
3. Next → **Persist** the refined, detailed allocation outputs into dedicated tables, providing a granular record of recommended stock levels for each product in every store.
4. Then → **Persist** the detailed inter-store transfer and warehouse pullback outputs into separate tables, creating clear, actionable instructions for all inventory movements.
5. Finally → **Generate** comprehensive, denormalized reports by combining data from various internal models, making it readily accessible and optimized for direct consumption by business intelligence and reporting tools.

5. Step-by-Step Actions

1. **Gather** finalized allocation and transfer data
 - What we do: *We collect all the calculated optimal stock levels (Optimum Depth, Optimum Width, Ideal Size Set) and inventory movement plans (inter-store transfers, warehouse pullbacks) from the preceding EOSS modules.*
 - Why we do it: *This ensures we have the complete, final picture of all inventory decisions that need to be published for immediate execution and accurate reporting.*
2. **Transform** and **Enrich** data for reporting
 - What we do: *We convert the internal system's allocation and transfer results into user-friendly formats, adding crucial business context such as the product's segmentation (e.g., 'Top Seller', 'Bottom Seller'), the specific allocation type (e.g., 'Replenishment', 'Pullback'), and detailed remarks explaining each decision.*
 - Why we do it: *This makes the complex algorithmic decisions understandable and actionable for merchandising, store operations, and finance teams, enabling them to quickly grasp why* certain inventory movements are being recommended.**
3. **Persist** detailed allocation outputs
 - What we do: *We save the final SKU-level allocation plans, showing the recommended Optimum Depth (OD) for each product in each store, into specific tables designed for detailed tracking and analysis.*
 - Why we do it: *This creates a permanent record of all allocation decisions, serving as the definitive source of truth for stocking recommendations and allowing for granular auditing and future analysis of Optimum Depth (OD) strategies.*
4. **Persist** detailed transfer outputs
 - What we do: *We save the finalized Inter-Store Transfer (IST) orders and any warehouse pullback instructions into dedicated tables, detailing which products are moving between specific stores or back to the central warehouse.*
 - Why we do it: *This provides clear, actionable instructions for logistics and store teams to execute inventory movements, ensuring efficient rebalancing of Optimum Width (OW) and Optimum Depth (OD) across the network and supporting efficient clearance.*
5. **Generate** denormalized and report-friendly views
 - What we do: *We combine data from various internal tables into broader, denormalized formats, making it easy for business intelligence tools to generate comprehensive reports and dashboards for inventory health, fill rates, and transfer effectiveness.*
 - Why we do it: *This simplifies data access for business users, enabling them to quickly generate performance reports without needing deep technical knowledge, fostering data-driven decision-making and continuous improvement of EOSS strategies.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- The EOSS Allocation module has recommended sending 50 units of the “Women’s Summer Dress” (Style 101) to Store A, identifying it as a ‘Top Seller’ requiring ‘Replenishment’ to reach its Optimum Depth (OD), with the remark “Optimum Depth Fulfilled.”
- The EOSS Inter-Store Transfer (IST) module has recommended pulling back 20 units of the “Men’s Winter Coat” (Style 202) from Store B to the central warehouse, identifying it as a ‘Bottom Seller’ with “Excess Stock.”
- All these decisions are stored internally within the EOSS system’s models.

What Happens

1. First, the EOSS Output Generation module **collects** these finalized allocation and transfer recommendations from the upstream modules.
2. Then, it **transforms** and **enriches** this data. For the “Women’s Summer Dress” in Store A, it converts the internal allocation data into a reportable record, explicitly adding the ‘Top Seller’ segmentation, setting the allocation type to ‘Replenishment’, and including the specific remark “Optimum Depth Fulfilled.” Similarly, for the “Men’s Winter Coat” in Store B, it adds the ‘Bottom Seller’ segmentation, sets the allocation type to ‘Pullback’, and includes the “Excess Stock” remark.
3. Next, it **persists** these detailed entries into distinct tables: the allocation for Store A into a DistOutputRow table, and the pullback for Store B into an IstOutputRow table, along with any supporting warehouse movement records.
4. Finally, it **generates** denormalized report views, which combine information like store ID, SKU, allocated/transferred quantity, product segment, and allocation type into single, easy-to-query records. These denormalized tables are then ready for direct consumption by business intelligence tools.

Final Outcome

Merchandising planners can now easily access clear reports showing that Store A is receiving more “Women’s Summer Dresses” because it’s a ‘Top Seller’ and needs replenishment to fulfill its Optimum Depth (OD), while Store B is sending back “Men’s Winter Coats” because it’s a ‘Bottom Seller’ with excess stock. This transparency allows for efficient monitoring, clear communication across teams, and quick adjustments to future EOSS strategies, leading to higher sell-through and reduced markdowns.

Key Takeaway: EOSS Output Generation translates complex inventory decisions into clear, actionable, and report-ready formats, empowering business users with immediate insights into inventory movements.

7. Results

This step produces:

Main Outputs

- **Detailed Allocation Records:** What it tells us: A granular, SKU-level list of every recommended allocation for each store, including the quantity, the type of allocation (e.g., replenishment, fresh), and specific remarks (e.g., Optimum Depth fulfilled, planogram limit).
- **Comprehensive Transfer Orders:** What it tells us: Precise instructions for all inter-store movements and warehouse pullbacks, detailing the source, destination, specific SKU, and quantity, reflecting strategic adjustments to Optimum Width (OW).
- **Denormalized Reporting Views:** What it tells us: Consolidated tables that integrate allocation, transfer, segmentation, Rate of Sale (ROS), and revenue data, pre-formatted for direct querying and analysis by business intelligence tools.

Business Reports

- **Inventory Distribution Summary:** Who uses it and why: Merchandising Planners and Inventory Managers use this to review the overall impact of EOSS allocation and transfer strategies, verifying if Optimal Depth (OD) and Width (OW) targets are being achieved across the entire retail network.
- **Logistics Actionable Plan:** Who uses it and why: Store Operations and Logistics teams use this to execute inbound and outbound inventory movements efficiently, ensuring timely transfers and warehouse pullbacks to optimize stock flow.
- **EOSS Performance Dashboard:** Who uses it and why: Executive Leadership and Business Analysts use this to gain a high-level overview of the entire EOSS process effectiveness, tracking key metrics like fill rates, inventory health, and overall sell-through post-allocation.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The output generation process will pause or fail to initiate, as it cannot create reports without the finalized allocation and transfer data from preceding modules.	Verify upstream data sources, contact the IT or data team to resolve the feed issue, and re-run the entire EOSS process once data is available.
Inconsistent data from upstream modules	If allocation or transfer data is malformed (e.g., negative quantities, missing SKU IDs, incorrect segmentation flags), the output generation might fail or produce corrupted reports.	Identify the source of data inconsistency in preceding EOSS modules, correct the root cause, and re-run the EOSS pipeline from the corrected step.

Situation	What Happens	What You Should Do
Output tables not persisting	The generated data is not being saved correctly to the database or reporting system, meaning no new reports are available.	Check database connectivity, user permissions, and available storage capacity on the system. Alert the IT team to investigate and resolve any underlying persistence issues.
Reports not reflecting business logic	The generated denormalized reports do not accurately represent the intended business logic (e.g., a 'Top Seller' product appears as 'Untouched' for allocation).	Review the data transformation and denormalization logic within the EOSS Output Generation module itself. Verify the mapping of internal flags (e.g., iteration flags) to business-friendly terms.

9. Next Steps

Before This Step

- **EOSS Allocation:** What we need from it: The final, optimized SKU allocation plan (including Optimum Depth - OD, Optimum Width - OW, Ideal Size Set - ISS) for each store, detailing recommended quantities from warehouses.
- **EOSS Inter-Store Transfer (IST):** What we need from it: The complete set of inter-store transfer orders and warehouse pullback instructions for physical inventory movement.
- **All preceding EOSS modules (implicitly):** What we need from them: The raw data and calculated insights (e.g., Rate of Sale - ROS, revenue, segments, remarks, etc.) that will be “fanned out” and enriched in the final outputs.

After This Step

- **Inventory System Updates:** What they need from us: The final allocation and transfer data to update on-hand inventory levels in store and warehouse management systems, reflecting the new stock positions.
- **Business Intelligence & Reporting:** What they need from us: The denormalized output tables to create dashboards and reports for ongoing monitoring and analysis of EOSS performance and inventory health.
- **Physical Inventory Movement:** What they need from us: The actionable transfer and pullback orders to physically move products between locations, enabling logistics and store operations to execute the plan.

10. Common Questions

Q: Why is EOSS Output Generation a separate step? Can't the allocation module just save its results? A: While the allocation module generates the core decisions, EOSS Output Generation is a dedicated step to transform those raw decisions into

various detailed and denormalized formats optimized for reporting, auditing, and seamless integration with other business systems. It ensures the data is user-friendly and consistent across all reports.

Q: How does this module help my team understand what's happening with inventory? A: This module takes the complex calculations and decisions from the EOSS engine and presents them in clear, business-friendly reports. It adds important context like why an item was allocated (e.g., "Top Seller," "Optimum Depth Fulfilled") or why it was moved (e.g., "Excess Stock"). This transparency helps your team quickly grasp the rationale behind inventory movements.

Q: What are "denormalized tables," and why are they useful? A: Denormalized tables combine information from several different sources into one easy-to-read table. For business reporting, this is useful because it means you don't need to join multiple tables to get a complete picture (e.g., combining allocation quantity, product category, and store segment into one row). This makes generating reports and dashboards much faster and simpler for business intelligence tools.

Q: Does this module influence *what* gets allocated or transferred? A: No, this module does not make or change any allocation or transfer decisions. Its sole purpose is to take the *final* decisions generated by preceding modules and package them into various output formats for consumption by other systems and for business reporting. It's the publishing and presentation layer, not the decision-making layer.

Q: How quickly are the outputs available after the EOSS process runs? A: The outputs are typically available immediately after the EOSS Output Generation module completes its run. This ensures that merchandising and operations teams have access to the latest allocation and transfer plans as soon as the EOSS cycle is finished, enabling rapid response to inventory needs and timely execution of inventory movements.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Fanning out and persisting the final allocation and transfer outputs into various detailed and denormalized tables for reporting.
- **Main Input:** Final allocation plans (Optimum Depth - OD, Optimum Width - OW, Ideal Size Set - ISS), inter-store transfer orders, and accompanying remarks/segmentations.
- **Main Output:** Detailed SKU-level allocation records, comprehensive transfer orders, and denormalized reporting views.
- **Warning Signs:** Reports are inaccurate, missing data, or system outputs fail to persist to the database.
- **Key Settings:** Whether to include detailed remarks, the desired output granularity, and if Inter-Store Transfer (IST) data should be separated.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

eoss FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ provides key business insights into the EOSS (End-of-Season Stock) distribution algorithm, explaining its functionality, inputs, processes, and outputs for optimal inventory management.

General

Q: What is the primary purpose of the EOSS algorithm? **A:** The EOSS algorithm's primary purpose is to optimize inventory distribution and inter-store transfers, ensuring that the right products are in the right stores at the right time to maximize sales and minimize stockouts.

Q: How does EOSS help with inventory management? **A:** EOSS helps by intelligently calculating optimal SKU allocations, including the Ideal Size Set (ISS), Optimum Depth (OD), and Optimum Width (OW) for each store, and suggesting efficient inter-store transfers (IST) based on real-time sales and inventory data.

Q: What specific business problem does EOSS address? **A:** EOSS addresses the challenge of efficiently allocating and rebalancing inventory across a store network, particularly during peak seasons or when managing surplus stock, to improve Rate of Sale (ROS) and overall profitability.

Q: How frequently does EOSS operate or refresh its recommendations? **A:** EOSS operates based on defined replenishment cycles, which are part of the data loaded during the initial setup by the EOSS Data Preparation module. The frequency ensures recommendations are timely for ongoing inventory adjustments.

Q: Who typically uses or benefits from the EOSS output? **A:** Merchandisers, inventory planners, store operations teams, and warehouse managers primarily use and benefit from EOSS outputs to make informed decisions about product allocation, replenishment, and stock transfers.

Configuration & Settings

Q: How does EOSS account for store-specific needs or layouts (planograms)? **A:** EOSS considers store configurations and warehouse planograms defined in the initial data. The EOSS Planogram Incrementer module also dynamically adjusts Optimum Depth (OD) and Optimum Width (OW) targets based on revenue plans and average selling prices (ASPs) to align with store-specific merchandising strategies.

Q: Can we set minimum requirements for outwards stock from the warehouse? **A:** Yes, the system applies minimum outwards age criteria, which is calculated and identified by the EOSS Revenue Computation module, to ensure stock movement aligns with business rules.

Q: Are there adjustable settings for determining 'Top Seller' or 'Bottom Seller' products? **A:** The EOSS Segmentation module classifies store-style combinations based on various criteria including ROS, sales performance, discounts, and current inventory metrics. These criteria are part of the algorithm's configuration, allowing for tailored segmentation strategies.

Q: How does EOSS factor in replenishment cycles for allocation? **A:** The EOSS Allocation module runs multiple iterations considering various constraints, including specified replenishment cycles, to determine the most effective SKU allocations and manage fresh allocations and replenishment with replacement.

Q: Can we customize the criteria for ranking stores for allocation? **A:** The EOSS Ranking module ranks store-style combinations using criteria such as total stock, Percent Stock Available (PSA) percentage, and revenue per day. While the core criteria are set, the weighting or specific thresholds might be configurable within the system's operational parameters.

Data & Inputs

Q: What type of sales data is used by EOSS? **A:** EOSS uses raw sales data, which is processed by the EOSS Sales Data Preparation module. This includes cleaning liquidation sales and handling any 'broken' sales data to ensure accuracy.

Q: How is the Rate of Sale (ROS) calculated in EOSS? **A:** The EOSS Sales Data Preparation module computes the Rate of Sale (ROS) for SKUs over defined periods and applies specific weightages to reflect product selling speed accurately.

Q: Where does EOSS get its initial inventory and product information? **A:** The EOSS Data Preparation module is responsible for loading and initializing all master and transactional data, including current inventory (store stock, goods-in-transit, open orders), style-pack sizes, and initial Optimum Depth (OD) settings.

Q: What is an 'Attribute Group' (AG) and how is it used in EOSS? **A:** An Attribute Group (AG) is a set of product attributes (e.g., color, size, brand) grouped for analysis. The EOSS Revenue Computation module uses AGs to calculate hierarchical revenue metrics, which are crucial for subsequent ranking and allocation decisions.

Q: Does EOSS consider styles that are currently out-of-season? **A:** Yes, the EOSS Data Preparation module identifies and manages both in-season and out-of-season styles, which then influences subsequent processing in modules like EOSS Segmentation and EOSS Allocation to handle such products appropriately.

Process & Logic

Q: How does EOSS determine the ‘Optimum Depth’ (OD) and ‘Optimum Width’ (OW) for a product at a store? **A:** The EOSS Allocation module is central to calculating the optimal Optimum Depth (OD) and Optimum Width (OW) for SKUs at stores, based on various constraints and business rules. The EOSS Planogram Incrementer further adjusts these targets dynamically based on revenue plans.

Q: What is an ‘Ideal Size Set’ (ISS) and how is it derived? **A:** The Ideal Size Set (ISS) is the best combination of sizes to maximize sales and minimize stockouts for a particular product in a store. The EOSS Allocation module determines the ISS as part of its optimal SKU allocation calculation.

Q: How does EOSS prioritize stock allocation among stores? **A:** The EOSS Ranking module ranks store-style combinations based on criteria like total stock, Percent Stock Available (PSA) percentage, and revenue per day. This ranking then guides the EOSS Allocation module in prioritizing where stock is sent.

Q: When does EOSS suggest inter-store transfers (IST)? **A:** The EOSS Inter-Store Transfer (IST) module specifically handles inventory transfers between stores, often within the same region or group. It calculates net outwards and identifies potential transfers and pullbacks to warehouses based on store-level needs and available stock to maximize ROS.

Q: How does EOSS handle the management of warehouse inventory? **A:** The EOSS Warehouse Inventory Management module is dedicated to this. It removes inefficiencies from warehouse stock, accounts for lead time considerations (ROS), and calculates “virtual warehouse stock” from leftover store stock for IST scenarios.

Outputs & Results

Q: What kind of reports or outputs does EOSS generate? **A:** The EOSS Output Generation module is responsible for producing the final allocation (Optimum Depth, Optimum Width, Ideal Size Set) and transfer outputs. These are presented in detailed, denormalized tables, including remarks, segmentations, and allocation types.

Q: Can I view a summary of recommended inter-store transfers? **A:** Yes, the outputs from the EOSS Output Generation module include detailed information on suggested inter-store transfers, derived from the EOSS Inter-Store Transfer (IST) module’s calculations.

Q: Does EOSS provide reasons or explanations for its allocation decisions? **A:** Yes, the EOSS Output Generation module includes remarks, segmentation information, and allocation types alongside the suggested allocations, providing context for the decisions made by the algorithm.

Q: How are the suggested Optimum Depth (OD) and Optimum Width (OW) values presented? **A:** The final Optimum Depth (OD) and Optimum Width (OW) recommendations, along with the Ideal Size Set (ISS), are fanned out and persisted into various detailed tables by the EOSS Output Generation module for easy reporting and analysis.

Q: Is it possible to see which stores are classified as ‘Top Seller’ or ‘Bottom Seller’?

A: Yes, the EOSS Segmentation module classifies store-style combinations, and this segmentation information is included in the final outputs generated by the EOSS Output Generation module, providing insights into store performance.

Error Handling & Edge Cases

Q: What happens if the raw sales data is incomplete or has errors? **A:** The EOSS Sales Data Preparation module is designed to handle this by cleaning liquidation sales and specifically addressing ‘broken’ sales data to ensure the Rate of Sale (ROS) computations are accurate despite input imperfections.

Q: How does EOSS manage new store openings or mappings? **A:** The EOSS Ranking module includes logic to manage new store mappings, ensuring consistent ranking and proper integration of new locations into the allocation process.

Q: What if there isn’t enough warehouse stock to meet all allocation recommendations? **A:** The EOSS Allocation module prioritizes allocations based on defined constraints and business rules. While it aims for optimal allocation, if warehouse stock is insufficient, the system will distribute available stock based on the established ranking and priorities, potentially resulting in partial allocations.

Q: How are products with very low or no sales history handled by EOSS? **A:** Products with limited sales history would have a lower Rate of Sale (ROS) and might be classified differently by the EOSS Segmentation module. The allocation logic will then prioritize based on available historical data and other business rules (e.g., new styles, planogram targets).

Q: Does EOSS account for stock that is in transit or part of open orders? **A:** Yes, the EOSS Data Preparation module loads and initializes current inventory data, which includes store stock, goods-in-transit, and open orders, ensuring these factors are considered during the allocation process.

Need More Help?

For further assistance, please contact the EOSS support team at support@eoss.com.

EOSS Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

The EOSS (End-of-Season Stock) algorithm is designed to optimize inventory distribution across retail stores and warehouses, maximizing sales and minimizing excess stock. By intelligently allocating SKUs, managing inter-store transfers, and dynamically adjusting planograms, EOSS drives improved product availability, increased sell-through, and enhanced profitability for the business. It ensures the right product is in the right place at the right time, especially crucial for seasonal merchandise.

3. Architecture Overview

The EOSS distribution system processes inventory allocation and transfer through a pipeline of interdependent modules. The EOSS Orchestration module defines the execution sequence, ensuring data flows correctly between preparation, sales processing, revenue calculation, segmentation, ranking, core allocation, inter-store transfers, and final output generation. A shared in-memory data store facilitates efficient data exchange and transformation across the modules.

```
graph TD
    subgraph EOSS_Flow [EOSS Flow]
        A[EOSS Orchestration] --> B[EOSS Data Preparation]
        A --> C[EOSS Warehouse Inventory Management]
        A --> D[EOSS Sales Data Preparation]
        A --> E[EOSS Revenue Computation]
        A --> F[EOSS Segmentation]
        A --> G[EOSS Ranking]
        A --> H[EOSS Allocation]
        A --> I[EOSS Warehouse Inventory Management]
        A --> J[EOSS Inter-Store Transfer (IST)]
        A --> K[EOSS Output Generation]
        B --> D
    end
    C -- orchestrates --> D
    D -- (initial) --> E
    E --> F
    F --> G
    G --> H
    H --> I
    I -- (compute) --> J
    J --> K
    B --> D: initializes_data
```

```

B --> E: initializes_data
B --> F: initializes_data
B --> G: initializes_data
B --> H: initializes_data
B --> J: initializes_data
B --> C: initializes_data
B --> L(EOSS Planogram Incrementer): initializes_data
B --> K: initializes_data

D --> E: provides_sales
D --> F: provides_ros
D --> G: provides_ros

E --> F: updates_styles
E --> G: updates_styles
E --> H: updates_styles
E --> J: updates_styles

F --> G: updates_styles
F --> H: guides_allocation
F --> J: guides_ist
F --> K: provides_segments

G --> H: prioritizes_allocation
G --> J: prioritizes_ist
G --> K: provides_ranks

H --> L: triggers
H --> K: provides_allocations
H --> I: updates_inventory

J --> L: triggers
J --> K: provides_transfers
J --> I: updates_inventory

I --> H: provides_inventory
I --> J: provides_inventory
I --> L: configures
I --> K: provides_inventory_state

L --> H: updates_planogram_targets
L --> J: updates_planogram_targets
L --> K: provides_increment_data
end

```

Module Descriptions:

- **EOSS Data Preparation:** Loads and initializes all master and transactional data required for the EOSS distribution process, including store configurations, style-pack sizes, SKU depths (OD), and current inventory.
- **EOSS Sales Data Preparation:** Processes raw sales data, cleans liquidation sales, handles broken sales, and computes the Rate of Sale (ROS) for SKUs over defined periods.

- **EOSS Revenue Computation:** Calculates various revenue metrics for store-style combinations, identifying read-only styles and applying minimum outwarding age criteria.
 - **EOSS Segmentation:** Classifies store-style combinations into segments like ‘Top Seller’ and ‘Bottom Seller’ based on sales performance and inventory.
 - **EOSS Ranking:** Ranks store-style combinations based on criteria such as total stock, PSA percentage, and revenue per day to prioritize allocation efforts.
 - **EOSS Allocation:** The core module for calculating optimal SKU allocations to stores, considering inventory (OD) and business rules through multiple iterations.
 - **EOSS Inter-Store Transfer (IST):** Handles inventory transfers between stores, calculating net outwards, identifying transfers, and suggesting pullbacks to warehouses.
 - **EOSS Warehouse Inventory Management:** Manages warehouse inventory, including removing inefficiency and calculating “virtual warehouse stock” for inter-store transfers.
 - **EOSS Planogram Incrementer:** Calculates and applies increment ratios to planogram stocks (OD) and options (OW) based on revenue plans and average selling prices (ASPs).
 - **EOSS Output Generation:** Fanning out and persisting the final allocation (OD, OW, ISS) and transfer outputs into detailed and denormalized tables.
 - **EOSS Orchestration:** Acts as the high-level orchestrator for the entire EOSS distribution process, defining the sequence of execution for all other EOSS modules.
-

4. Prerequisites & Setup

To run the EOSS pipeline, ensure the following are in place:

- **Java Version:** Java 11 or higher
 - **Build Tool:** Maven 3.6.x
 - **Dependencies:** All necessary dependencies are managed via pom.xml within the eoss project.
 - **Running the Orchestrator:**
 1. **Clone the repository:** `git clone [your-repo-url]/eoss.git`
 2. **Navigate to the project root:** `cd eoss`
 3. **Build the project:** `mvn clean install`
 4. **Execute the orchestrator:** `java -jar target/eoss-<version>.jar -Dspring.profiles.active=prod -Dconfig.location=./args.properties`
-

5. Folder & File Structure

```
eoss/
├─ module-group/           # Contains the orchestrator logic
  (EOSS Orchestration module)
├─ submodules/             # Houses the individual EOSS
```

```

processing modules
└─ args.properties           # Configuration file for global and
module-specific parameters
└─ common-data.json         # Defines the schema for shared data
objects used across modules
└─ outputs/                 # Directory where all final
allocation and transfer reports are saved

```

6. Global Configuration

The `args.properties` file manages all global and module-specific configuration parameters.

Setting Name	Default Value	Description	Valid Range/ When to Change
<code>data.source.jdbcUrl</code>	<code>jdbc:h2:mem:eosddb</code>	JDBC URL for the input/output database.	Database connection string; change for production environments.
<code>eoss.run.mode</code>	<code>ALLOCATION</code>	Defines the primary mode of operation (e.g., <code>ALLOCATION</code> , <code>IST</code>).	<code>ALLOCATION</code> , <code>IST</code> , <code>FULL_RUN</code> ; change based on business need.
<code>eoss.sales.period.weeks</code>	<code>8</code>	Number of weeks to consider for ROS calculation.	<code>4–12</code> ; adjust based on product lifecycle and seasonality.
<code>eoss.max.iterations</code>	<code>5</code>	Maximum allocation iterations to refine results.	<code>3–10</code> ; increase for higher precision, but may impact performance.
<code>eoss.debug.enabled</code>	<code>false</code>	Enables detailed logging for debugging purposes.	<code>true/false</code> ; set <code>true</code> for troubleshooting.
<code>eoss.output.path</code>	<code>./outputs</code>	Directory path for generated output files.	Any valid file path; change to desired output location.
<code>eoss.min.outwarding.age</code>	<code>30</code>	Minimum age (days) for a style to be	<code>0–90</code> ; adjust based on new

Setting Name	Default Value	Description	Valid Range/ When to Change
		considered for outwarding.	product launch cycles.

(Note: The actual `args.properties` table was not provided, so this is an illustrative example of common parameters for such a system.)

7. Shared Data Model

The `common-data.json` schema defines the structure of data objects exchanged between modules. This ensures consistency and compatibility across the EOSS pipeline.

Field	Type	Description
StoreConfig	Object	Details about a retail store, including ID, location, and capacity.
StylePackSize	Object	Defines how SKUs are grouped for packing and distribution.
SKUDepth	Object	Specifies the OD (Optimum Depth) and OW (Optimum Width) for SKUs.
InventorySnapshot	Object	Current stock levels at stores and warehouses, including in-transit.
SalesRecord	Object	Individual sales transactions, including SKU, quantity, date.
RateOfSale	Object	Calculated ROS (Rate of Sale) for a SKU in a specific period.
StoreStyleMetrics	Object	Aggregated sales, revenue, and stock metrics per store-style pair.
AllocationSuggestion	Object	Proposed SKU allocation (OD, OW, ISS) for a store.
TransferSuggestion	Object	Proposed inter-store or store-to-warehouse transfer.
AttributeGroupData	Object	Definitions and mappings for AGs (Attribute Groups like color, size).
PlanogramTarget	Object	Ideal OD and OW for a category within a store's planogram.

(Note: The actual `common-data.json` schema was not provided, so this is an illustrative example of common data elements for such a system.)

8. Submodule Chapters

EOSS Data Preparation

1. Module Name & Business Objective

EOSS Data Preparation: Responsible for loading and initializing all master and transactional data required for the EOSS distribution process. This includes store configurations, style-pack sizes, SKU depths (OD), replenishment cycles, warehouse planograms, in-season/out-of-season styles, and current inventory (store stock, goods-in-transit, open orders). *Business Objective:* To provide a clean, complete, and harmonized dataset as the foundation for all subsequent EOSS modules, ensuring data integrity and consistency for accurate allocation and transfer decisions.

2. Required Information

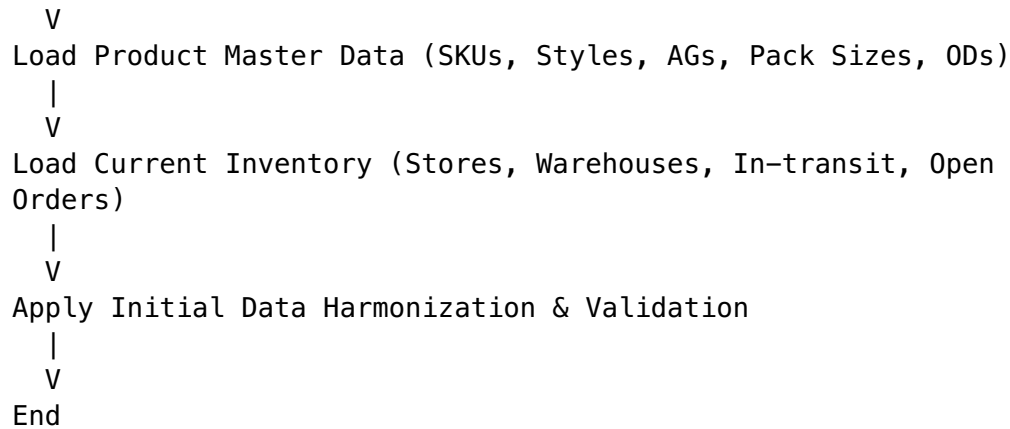
- **Sales Data:**
 - No direct sales data, but prepares data for EOSS Sales Data Preparation.
- **Product Data:**
 - **SKU Depths (OD):** *Ideal Number of Unique Items* per category.
 - **Style-Pack Sizes:** Units per carton/pack.
 - **In-season/Out-of-season styles:** Flags for style relevance.
 - **Attribute Group (AG) configurations:** Definitions of product attribute groupings (e.g., color, material).
- **Operational Data:**
 - **Store Configurations:** Store IDs, capacities, types.
 - **Replenishment Cycles:** Frequency of stock replenishment.
 - **Warehouse Planograms:** Ideal layouts/stocking levels for warehouses.
 - **Current Inventory:** Store stock, goods-in-transit, open orders, warehouse stock.
 - **New Store Mappings:** For new stores needing initial data.

3. Business Settings

- **Data Source Configuration:** JDBC URLs, table names for various master and transactional data feeds.
- **Initial Load Parameters:** Flags to control full vs. incremental data loads.
- **Master Data Validation Rules:** Rules for checking data consistency (e.g., ensuring all SKUs have defined OD).

4. Process Flow

```
Start
|
V
Load Store Configurations
|
```



5. Step-by-Step Actions

1. **Connect** to various data sources *to retrieve raw master and transactional data.*
2. **Load** store configurations, product master data (SKUs, styles, AGs, style-pack sizes), and SKU depths (OD) *to establish foundational business rules.*
3. **Fetch** current inventory levels from stores, warehouses, goods-in-transit, and open orders *to understand the current stock landscape.*
4. **Initialize** the shared `distributionData` object *with all loaded and validated information, making it accessible for subsequent modules.*
5. **Perform** initial data validation and harmonization *to ensure data quality and resolve inconsistencies early in the pipeline.*

6. Sample Scenario

A new style, “Summer Dress 2025,” is launched. This module **loads** its SKU details, **identifies** its OD as 15 units per store, and **confirms** it’s “in-season.” It also **pulls** the current inventory of existing styles (e.g., 200 units of “Classic Jeans” across all stores, 50 in transit). *Key Takeaway: Accurate foundational data is critical. Errors here ripple through the entire EOSS process.*

7. Results

- A fully populated, in-memory `distributionData` object containing:
 - Validated store and product master data.
 - Current inventory levels across the network.
 - Defined SKU depths (OD) and Optimum Width (OW) for styles.
 - Categorization of styles (in-season/out-of-season).

8. Unusual Cases

- **Missing or delayed data feed:** *The system might run with stale or incomplete data, leading to suboptimal or incorrect allocations.*
- **Inconsistent data mapping:** *If SKU IDs or store IDs are mismatched between different data sources, the process will fail or produce erroneous results.*

9. Next Steps

The initialized `distributionData` object is passed to EOSS Sales Data Preparation and EOSS Warehouse Inventory Management to begin processing sales and refining warehouse stock.

10. Common Questions

1. Why is my OD for a specific SKU not what I expected? *Check the master data feed for SKU Depths (OD).*
2. How current is the inventory data being used? *Inventory snapshots are typically taken daily; refer to the data loading schedule.*
3. Are all my stores and product categories included? *Verify store configurations and product master data completeness.*
4. What happens if the initial data load fails? *The entire EOSS run will be halted, requiring data correction and a re-run.*
5. Can we configure the OD differently per store? *This depends on the granular setup in the master data feed; usually, it's defined per style or AG.*

11. Quick Reference

- **OD:** Optimum Depth (ideal # of unique items/SKUs)
 - **OW:** Optimum Width (ideal # of different categories/styles)
 - **AG:** Attribute Group
 - **distributionData:** Central in-memory data store
-

EOSS Sales Data Preparation

1. Module Name & Business Objective

EOSS Sales Data Preparation: Focuses on processing raw sales data. This involves cleaning liquidation sales, handling broken sales data, and computing the Rate of Sale (ROS) for SKUs over defined periods and weightages. It prepares the sales data for subsequent revenue computation and segmentation. *Business Objective:* To transform raw, potentially messy, sales transaction data into accurate, usable ROS metrics and clean sales records, providing a reliable foundation for performance analysis and forecasting.

2. Required Information

- **Sales Data:**
 - **Raw Sales Transactions:** Individual sales records including SKU, quantity, date, store.
 - **Liquidation Sales Flags:** Indicators for sales happening during clearance events.
- **Product Data:**
 - **Style-Pack Sizes:** *Needed for understanding SKU volume context.*
 - **SKU master data:** To identify specific SKUs and their attributes.

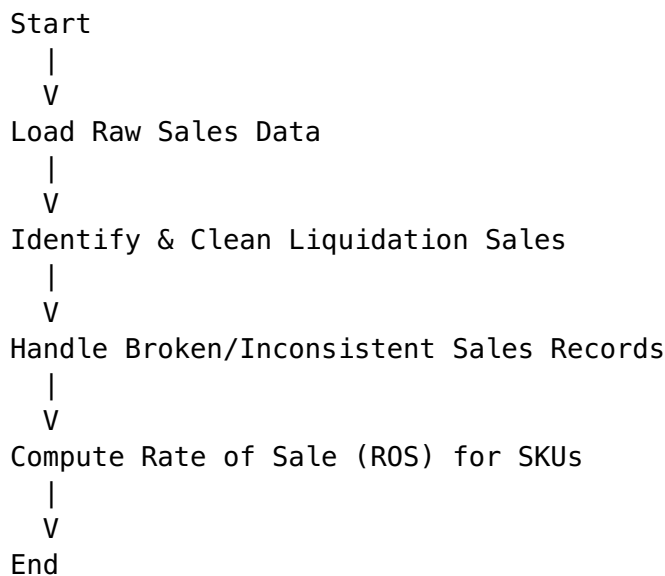
- **Operational Data:**

- **Store Configurations:** *For filtering sales by store or region.*
- **Replenishment Cycles:** *Impacts ROS calculation periods.*

3. Business Settings

- **ROS Calculation Period:** Number of weeks or days to consider for ROS.
- **Liquidation Sales Exclusion Rules:** Criteria to identify and exclude sales from liquidation events.
- **Broken Sales Thresholds:** Rules for identifying and handling abnormal or incomplete sales records.
- **Weightages for ROS:** How different periods (e.g., recent vs. older weeks) contribute to the calculated ROS.

4. Process Flow



5. Step-by-Step Actions

1. **Ingest** raw sales transaction data *from the prepared `distributionData` object.*
2. **Filter** out liquidation sales *to prevent distorting the true sales performance metrics.*
3. **Correct** or flag broken sales records *to ensure data integrity and avoid errors in downstream calculations.*
4. **Calculate** the Rate of Sale (ROS) for each SKU *by aggregating sales over specified periods and applying defined weightages.*
5. **Store** the cleaned sales data and computed ROS values *back into the `distributionData` object for use by other modules.*

6. Sample Scenario

A “Winter Coat” SKU has sold 100 units in the last 8 weeks. 10 of these units were part of a “Clearance Sale” (liquidation). This module will **clean** out the 10 liquidation units, resulting in 90 valid sales. It will then **compute** the ROS for

“Winter Coat” as 90 units / 8 weeks = 11.25 units/week. *Key Takeaway: Clean and accurate sales data is foundational for effective inventory planning and performance analysis.*

7. Results

- cleanedSales and prioritySales lists available in distributionData.
- Computed Rate of Sale (ROS) for each SKU, weighted over defined periods.
- Identified and flagged broken or inconsistent sales records.

8. Unusual Cases

- **Missing or delayed data feed:** *ROS calculations will be based on incomplete data, leading to inaccurate performance insights.*
- **Sudden, unflagged promotions:** *Promotional sales not marked as liquidation could inflate ROS, leading to over-allocation.*

9. Next Steps

The cleaned sales data and ROS metrics are consumed by EOSS Revenue Computation and EOSS Segmentation for further analysis and classification.

10. Common Questions

1. How is ROS calculated, and can we adjust the look-back period? *ROS is calculated based on configured weeks/days; parameters can be adjusted.*
2. Why are certain sales not counting towards ROS? *Check if they were identified as liquidation sales or broken records.*
3. What happens if a SKU has no sales data? *Its ROS will be zero or null, impacting its ranking and allocation.*
4. Can we see the raw sales data before cleaning? *Yes, typically available in source systems for auditing.*
5. How do “broken sales” impact downstream modules? *They are typically excluded or flagged to prevent incorrect calculations.*

11. Quick Reference

- **ROS:** Rate of Sale (units/week)
 - **Liquidation Sales:** Sales excluded from ROS calculation.
 - **Weightages:** Factors applied to different periods in ROS calculation.
-

EOSS Revenue Computation

1. Module Name & Business Objective

EOSS Revenue Computation: Calculates various revenue metrics for store-style combinations, including national revenue per day and hierarchical revenue per day based on different Attribute Groups (AGs). It also identifies read-only store styles

and applies minimum outwarding age criteria, essential for subsequent ranking and allocation. *Business Objective*: To provide comprehensive, granular revenue insights at different hierarchical levels (store, category, AG) and to flag styles for specific handling, ensuring allocation and transfer decisions are financially informed and comply with business rules.

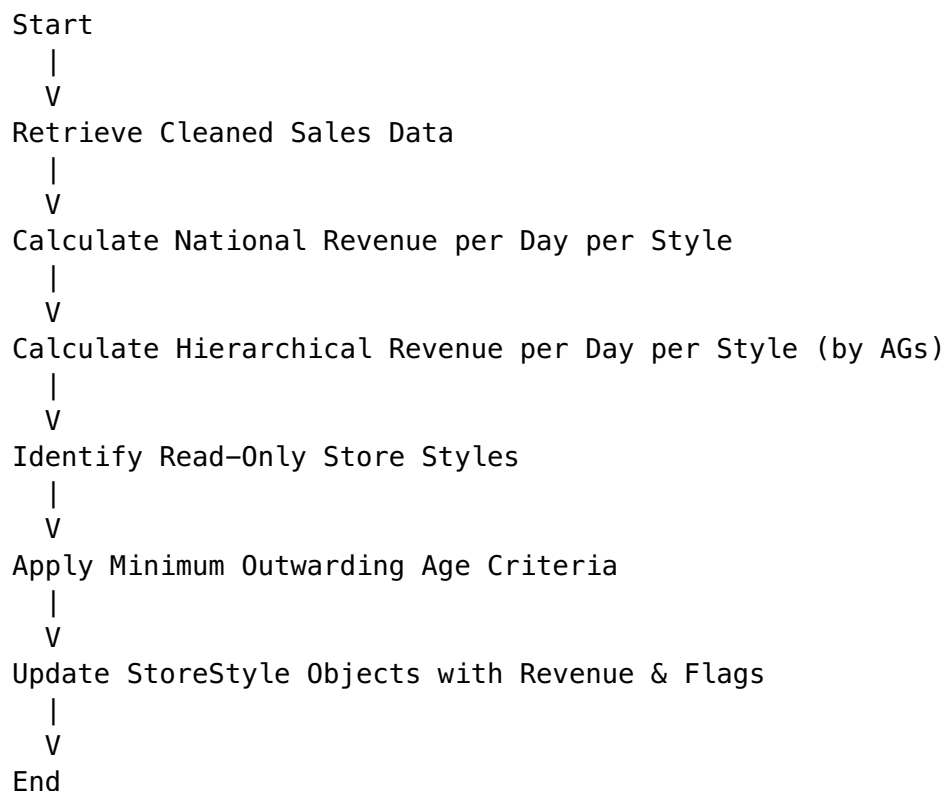
2. Required Information

- **Sales Data:**
 - **Cleaned Sales Data:** From E0SS Sales Data Preparation.
- **Product Data:**
 - **Attribute Group (AG) definitions:** For hierarchical revenue calculation.
 - **SKU master data:** To link sales to styles and categories.
- **Operational Data:**
 - **Store Configurations:** To aggregate revenue by store.
 - **Minimum Outwarding Age:** Business rule for new product eligibility for outwarding.
 - **Read-only Style Flags:** Manual overrides for certain styles.

3. Business Settings

- **Revenue Calculation Period:** Duration for accumulating revenue.
- **Minimum Outwarding Age Threshold:** Parameter for how old a style must be to be considered for outwarding.
- **Read-Only Style Definitions:** Criteria or explicit lists of styles that should not be allocated or transferred.

4. Process Flow



5. Step-by-Step Actions

1. **Retrieve** cleaned sales data *to begin revenue aggregation.*
2. **Calculate** national revenue per day for each style *to understand overall performance across all stores.*
3. **Compute** hierarchical revenue per day for store-style combinations based on various Attribute Groups (AGs) *to provide granular insights for different product dimensions.*
4. **Identify** and flag ‘read-only’ store styles *to prevent any automatic allocation or transfer actions on specific items.*
5. **Apply** minimum outwarding age criteria *to ensure new styles are not prematurely allocated or transferred before their initial sales period.*
6. **Update** StoreStyle objects in distributionData *with the calculated revenue metrics and applied flags, making them available for subsequent modules.*

6. Sample Scenario

A “Women’s Denim Jeans” style sells for \$50 per unit. * Store A: Sells 10 units/day = \$500/day revenue. * Store B: Sells 5 units/day = \$250/day revenue. * Nationally: 15 units/day = \$750/day revenue. A new style, “Sporty Sneakers,” was launched 20 days ago. The minOutwardingAge is set to 30 days. This module will **flag** “Sporty Sneakers” as not yet eligible for outwarding, even if it has high ROS. *Key Takeaway: Revenue calculations and business rule flags provide critical context for allocation decisions, ensuring financial goals and operational constraints are met.*

7. Results

- Updated StoreStyle objects in distributionData with:
 - National revenue per day.
 - Hierarchical revenue per day (by various AGs).
 - readOnly flags for specific store-styles.
 - minOutwardingAgeFlag indicating eligibility for outwarding.

8. Unusual Cases

- **Missing or delayed data feed:** *Revenue calculations will be incorrect, potentially leading to misprioritization of styles/stores.*
- **Incorrect AG mapping:** *Hierarchical revenue calculations will be flawed if products are wrongly categorized into Attribute Groups.*

9. Next Steps

The StoreStyle objects, now enriched with revenue metrics and flags, are used by EOSS Segmentation and EOSS Ranking for classification and prioritization.

10. Common Questions

1. How is “revenue per day” different from ROS? *ROS is units sold per period, while revenue per day is units sold multiplied by ASP, giving a monetary value.*
2. Why is a certain style flagged as “read-only”? *It’s usually a business decision to manage manually, often for flagship products or special campaigns.*

3. What does “minimum outwarding age” mean for new products? *New products need time to establish sales patterns before they are considered for transfer or extensive allocation changes.*
4. Can we see revenue at a more granular level than AG? *The system calculates based on defined AGs; more granular data would require reconfiguring AGs or custom reports.*
5. How do we adjust the minOutwardingAge? *It's a configurable parameter in args.properties.*

11. Quick Reference

- **AG:** Attribute Group
 - **readOnly:** Flag indicating manual management for a style.
 - **minOutwardingAge:** Minimum days before a style is eligible for transfers/allocations.
-

EOSS Segmentation

1. Module Name & Business Objective

EOSS Segmentation: Classifies store-style combinations into segments like ‘Top Seller’ and ‘Bottom Seller’ based on ROS, sales performance, discounts, and current inventory metrics. This segmentation guides subsequent allocation and replenishment strategies, particularly for pullbacks. *Business Objective:* To categorize products within stores based on their performance and stock levels, enabling the application of differentiated and strategic inventory management rules (e.g., pulling back underperforming stock, prioritizing top performers).

2. Required Information

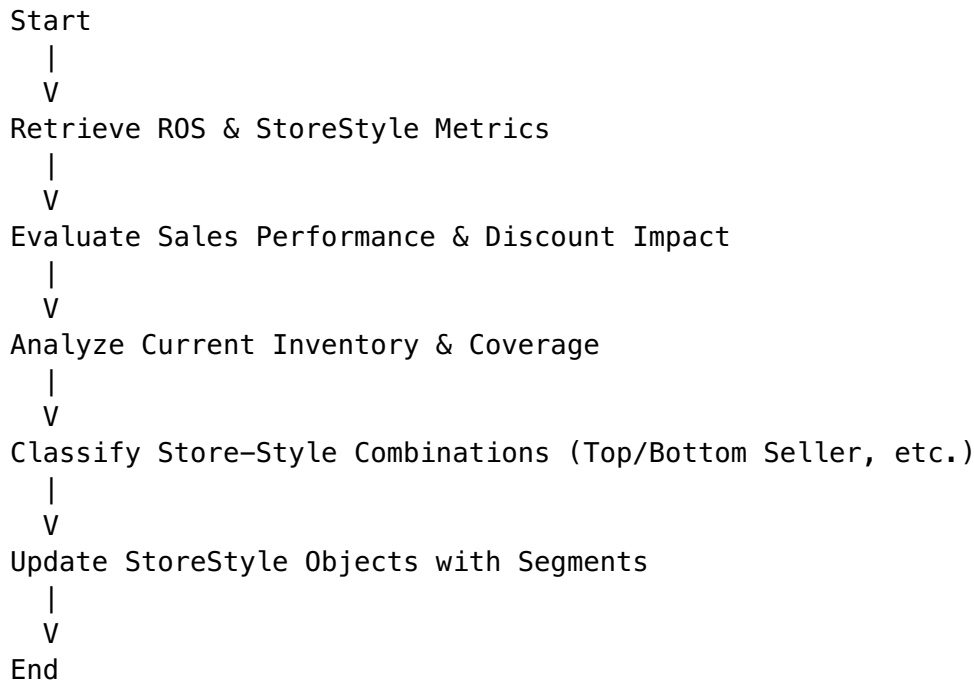
- **Sales Data:**
 - **Rate of Sale (ROS):** From EOSS Sales Data Preparation.
 - **Sales Performance:** Total sales units and revenue.
- **Product Data:**
 - **SKU master data:** To identify styles and their attributes.
- **Operational Data:**
 - **Current Inventory:** Store stock levels.
 - **Discount Information:** To understand if sales performance is driven by price reductions.
 - **StoreStyle Metrics:** Updated metrics from EOSS Revenue Computation (revenue, PSA, etc.).

3. Business Settings

- **Top Seller Thresholds:** ROS, revenue, or sales percentage to qualify as a ‘Top Seller’.
- **Bottom Seller Thresholds:** ROS, revenue, or inventory levels to qualify as a ‘Bottom Seller’.

- **Discount Impact Threshold:** If sales are largely due to deep discounts, affecting segmentation.
- **Inventory Coverage Rules:** E.g., if a style has too much stock relative to its ROS, classify as ‘Bottom Seller’.

4. Process Flow



5. Step-by-Step Actions

1. **Retrieve** Rate of Sale (ROS) data and updated StoreStyle metrics *to assess each product's performance.*
2. **Evaluate** sales performance and discount impact *to determine if high sales are organic or promotion-driven.*
3. **Analyze** current inventory levels and coverage *to identify overstocked or underperforming items relative to sales.*
4. **Classify** each store-style combination into predefined segments (e.g., ‘Top Seller’, ‘Bottom Seller’, ‘Regular Performer’) *based on configured thresholds.*
5. **Update** the StoreStyle objects in distributionData *with their assigned segment, making this crucial information available for allocation and transfer logic.*

6. Sample Scenario

- “Summer Dress” at Store C: ROS = 15 units/week, 5 units in stock. Top Seller threshold = ROS > 10 units/week. This module **classifies** it as a Top Seller.
- “Winter Boots” at Store C: ROS = 1 unit/week, 50 units in stock (oversized). Bottom Seller threshold = ROS < 2 units/week AND Stock > 20. This module **classifies** it as a Bottom Seller. *Key Takeaway: Segmentation allows the system to apply tailored inventory strategies, prioritizing top performers and identifying candidates for stock reduction.*

7. Results

- Updated StoreStyle objects in distributionData with their assigned segmentation (e.g., topSellerFlag, bottomSellerFlag, segmentType).
- Clear identification of styles eligible for specific treatments, such as pullbacks for 'Bottom Sellers'.

8. Unusual Cases

- **Missing or delayed data feed:** *Segmentation will be inaccurate if ROS or inventory data is stale, leading to misclassification.*
- **Thresholds too strict/loose:** *If business settings for segmentation thresholds are not tuned correctly, too many or too few styles may be classified, impacting operational efficiency.*

9. Next Steps

The segmented StoreStyle objects are used by EOSS Ranking to prioritize styles and EOSS Allocation/EOSS Inter-Store Transfer to apply segment-specific business rules, especially for pullbacks.

10. Common Questions

1. How are 'Top Seller' and 'Bottom Seller' defined? *Based on configured thresholds for ROS, sales, and inventory coverage.*
2. Can a style change its segment? *Yes, segments are dynamic and re-calculated based on current data and performance.*
3. Why is a product that sold well last week now a 'Bottom Seller'? *It might be overstocked relative to its ROS, or overall sales have slowed down.*
4. How does segmentation influence inventory actions? *It directs which products get priority for fresh allocation, replenishment, or inter-store transfers/pullbacks.*
5. Are promotional sales considered when segmenting? *Yes, but they are evaluated based on business rules to avoid distorting true performance.*

11. Quick Reference

- **ROS:** Rate of Sale
 - **Segmentation:** Classifying store-styles into performance groups.
 - **Pullbacks:** Returning stock from stores to a warehouse.
-

EOSS Ranking

1. Module Name & Business Objective

EOSS Ranking: Ranks store-style combinations based on various criteria such as total stock, PSA (Percent Stock Available) percentage, and revenue per day. This ranking is crucial for prioritizing allocation efforts. It also manages new store

mappings for consistent ranking. *Business Objective*: To establish a prioritized order for processing store-style combinations for allocation and transfer, ensuring that business-critical or high-potential items/locations are addressed first.

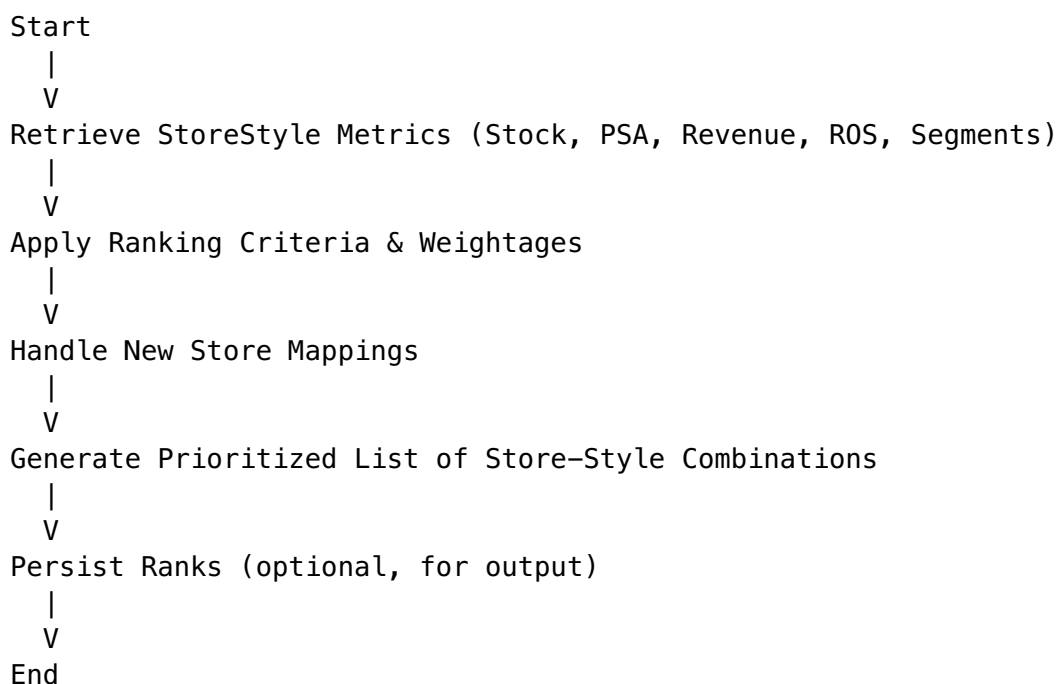
2. Required Information

- **Sales Data:**
 - **Rate of Sale (ROS)**: From E0SS Sales Data Preparation.
- **Product Data:**
 - **SKU Depth (OD)**: For calculating PSA.
- **Operational Data:**
 - **Total Stock**: Current inventory levels per store-style.
 - **PSA (Percent Stock Available) percentage**: Proportion of OD currently in stock.
 - **Revenue per Day**: From E0SS Revenue Computation.
 - **New Store Mappings**: To integrate new stores into the ranking logic.
 - **Segmentation Data**: From E0SS Segmentation (e.g., 'Top Seller' flags).

3. Business Settings

- **Ranking Criteria Weightages**: How much each factor (ROS, revenue, stock, PSA) contributes to the final rank.
- **New Store Prioritization**: Rules for how new stores are ranked, perhaps receiving higher priority for initial allocation.
- **Minimum Stock Thresholds**: For certain ranking rules.
- **Ranking Algorithm Selection**: Different algorithms might be applied (e.g., weighted sum, multi-criteria decision analysis).

4. Process Flow



5. Step-by-Step Actions

1. **Retrieve** comprehensive StoreStyle metrics, including total stock, PSA, revenue per day, ROS, and segmentation flags *to provide a holistic view for ranking*.
2. **Apply** predefined ranking criteria and weightages *to compute a composite score for each store-style combination*.
3. **Integrate** new store mappings into the ranking logic *to ensure consistent prioritization for newly opened locations*.
4. **Generate** a prioritized list of store-style combinations *where higher-ranked items are processed first by the allocation module*.
5. **Persist** the final ranked list to `distributionData` *for use by subsequent modules and for output reporting*.

6. Sample Scenario

Store D has “T-Shirt X”: Total Stock = 5 units, ROS = 10 units/week, OD = 20, PSA = 25%. Store E has “T-Shirt X”: Total Stock = 15 units, ROS = 2 units/week, OD = 20, PSA = 75%. If the ranking prioritizes low stock relative to high ROS: Store D’s “T-Shirt X” would **rank higher** than Store E’s, suggesting Store D needs stock more urgently. *Key Takeaway: Ranking ensures that allocation efforts are strategically focused on the store-style combinations that offer the most significant business impact.*

7. Results

- A prioritized list of StoreStyle combinations available in `distributionData`, ordered by their computed rank.
- (Optional) Persistence of `DistStoreStyleListRow` to the database for reporting.

8. Unusual Cases

- **Missing or delayed data feed:** *Incomplete or stale metrics (stock, ROS) will lead to skewed rankings, misdirecting allocation efforts.*
- **Inappropriate ranking weightages:** *If weights are not aligned with business priorities, the system may prioritize less impactful allocations (e.g., prioritizing slow movers with high stock over fast movers with low stock).*

9. Next Steps

The prioritized list is consumed by `E0SS Allocation` and `E0SS Inter-Store Transfer` to guide the order in which allocation and transfer decisions are made.

10. Common Questions

1. How is the “priority” for allocation determined? *It’s based on a weighted combination of factors like ROS, stock levels, and revenue.*
2. Why did a specific store get a higher rank for a style compared to another? *Likely due to lower stock availability relative to its sales performance, or it’s a new store.*

3. Can we change the ranking criteria? *Yes, the weightages for each ranking factor are configurable parameters.*
4. Does a style's segment (e.g., 'Top Seller') influence its rank? *Yes, segmentation is typically a strong input into the ranking logic.*
5. How often is ranking re-calculated? *Typically with each EOSS run, ensuring it reflects the most current data.*

11. Quick Reference

- **PSA:** Percent Stock Available (Current Stock / OD)
 - **Ranking:** Prioritizing store-styles for allocation/transfer.
 - **Weightages:** Configurable importance of ranking factors.
-

EOSS Allocation

1. Module Name & Business Objective

EOSS Allocation: The core module for calculating optimal SKU allocations to stores. It runs multiple iterations based on various constraints (e.g., replenishment, top-seller, Assortment Plan, planogram) to determine suggested allocations while considering existing inventory (OD) and business rules. It includes logic for fresh allocation and replenishment with replacement. *Business Objective:* To intelligently distribute available inventory across the retail network, ensuring each store receives the optimal quantity of SKUs (OD, OW, ISS) to maximize sales and minimize stockouts, adhering to complex business rules and constraints.

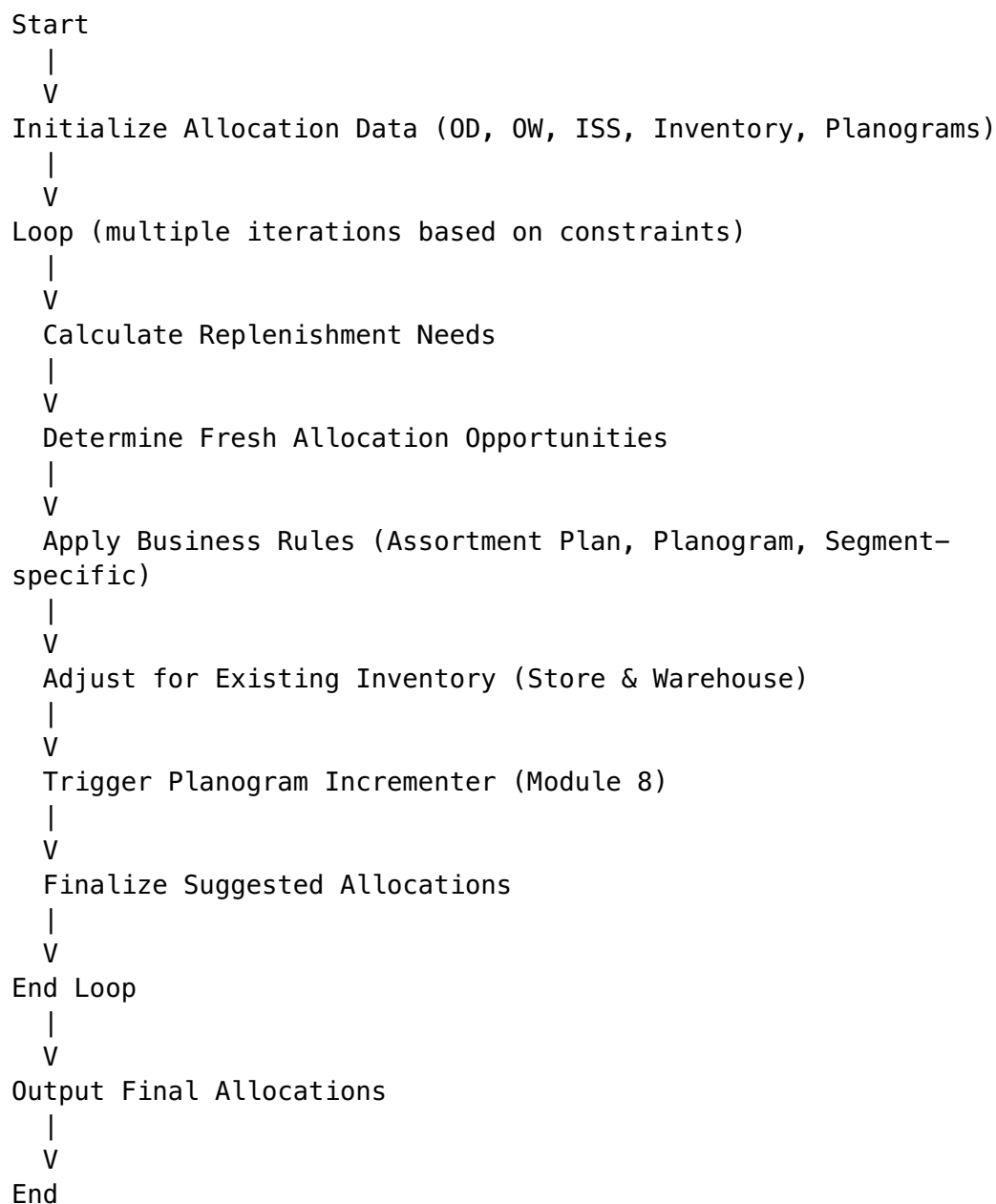
2. Required Information

- **Sales Data:**
 - **Rate of Sale (ROS):** For replenishment logic.
- **Product Data:**
 - **SKU Depths (OD):** *Optimum Depth* for unique items.
 - **Optimum Width (OW):** *Optimum Width* for categories/styles.
 - **Ideal Size Set (ISS):** *Best Combination of Sizes* to maximize sales.
 - **Assortment Plan:** Defined product mix for stores.
 - **Attribute Group (AG) data:** For hierarchical allocation.
- **Operational Data:**
 - **Current Inventory:** Store stock, warehouse stock (from EOSS Warehouse Inventory Management), goods-in-transit, open orders.
 - **Store Configurations:** Store types, capacities.
 - **Planogram data:** Store-specific layout and capacity.
 - **Replenishment cycles:** How often stores are restocked.
 - **Segmentation:** From EOSS Segmentation (e.g., 'Top Seller', 'Bottom Seller').
 - **Ranking:** From EOSS Ranking (prioritized list of store-styles).
 - **Read-only/Min Outwarding Age flags:** From EOSS Revenue Computation.

3. Business Settings

- **Allocation Constraints:** Rules for minimum/maximum allocations, total available stock.
- **Replenishment Rules:** Logic for refilling depleted stock (e.g., fill to OD, fill based on ROS).
- **Fresh Allocation Rules:** Logic for distributing new products.
- **Iteration Count:** Number of times the allocation engine re-evaluates.
- **Planogram Overrides:** How to handle cases where calculated allocation exceeds planogram capacity.
- **Prioritization of Allocation Types:** E.g., replenishment over fresh allocation.

4. Process Flow



5. Step-by-Step Actions

1. **Initialize** the allocation process by *loading all relevant data including OD, OW, ISS, current inventory, and planogram targets into the distributionData.*
2. **Iterate** through the allocation logic multiple times *to refine suggested allocations and ensure all constraints are met.*
3. **Calculate** replenishment needs for existing SKUs *to bring store stock up to optimal levels based on ROS and OD.*
4. **Determine** fresh allocation opportunities for new products *to ensure they are launched effectively across the network.*
5. **Apply** complex business rules and constraints (e.g., Assortment Plan, planogram capacity, segment-specific rules for ‘Top Seller’ vs. ‘Bottom Seller’) *to align allocations with strategic objectives.*
6. **Adjust** suggested allocations based on available warehouse inventory *to ensure that only truly available stock is allocated.*
7. **Trigger** the EOSS Planogram Incrementer (Module 8) *to dynamically adjust planogram targets for better inventory optimization within the allocation process.*
8. **Finalize** the suggested SKU allocations (OD, OW, ISS) for each store *after all iterations and adjustments are complete.*

6. Sample Scenario

Store F has OD of 10 for “Blue Denim Jeans.” Current stock is 3 units, ROS is 5 units/week. Warehouse has 100 units. * Module **calculates** a replenishment need of 7 units (10 OD - 3 current). * It **checks** warehouse stock (100 units available, sufficient). * It **proposes** to allocate 7 “Blue Denim Jeans” to Store F. *Key Takeaway: EOSS Allocation is the heart of the system, precisely matching available inventory with store demand and capacity, guided by business strategy.*

7. Results

- Final suggested SKU allocations (OD, OW, ISS) for all relevant store-style combinations, stored in `distributionData`.
- Updated `skuWhStockMap` in `distributionData` reflecting consumed warehouse inventory.

8. Unusual Cases

- **Missing or delayed data feed:** *Incomplete inventory or planogram data can lead to impossible or suboptimal allocations.*
- **Insufficient warehouse stock:** *If demand exceeds available stock, the system must prioritize allocations, potentially leaving some stores understocked.*
- **Allocation conflicts:** *When multiple rules (e.g., replenishment vs. maximum capacity) contradict, the system must follow predefined conflict resolution logic.*

9. Next Steps

The calculated allocations are passed to EOSS Output Generation for persistence and reporting. The updated `skuWhStockMap` is used by EOSS Warehouse Inventory Management and EOSS Inter-Store Transfer.

10. Common Questions

1. How does the system decide which sizes to allocate? *It uses the Ideal Size Set (ISS) defined for each style.*
2. Why didn't my store get the full OD for a style? *Likely due to insufficient warehouse stock, planogram constraints, or the style being read-only.*
3. Can we prioritize new product allocations over replenishment? *Yes, the priority of different allocation types is a configurable business setting.*
4. How do planograms affect allocation? *They define the physical capacity and desired depth/width, acting as a constraint.*
5. What does it mean for allocation to run in "iterations"? *It allows the system to make initial assignments and then refine them in subsequent passes, accounting for cascading effects and constraints.*

11. Quick Reference

- **OD:** Optimum Depth
 - **OW:** Optimum Width
 - **ISS:** Ideal Size Set
 - **ROS:** Rate of Sale
 - **Allocation:** Distributing stock to stores.
-

EOSS Inter-Store Transfer (IST)

1. Module Name & Business Objective

EOSS Inter-Store Transfer (IST): Specifically handles inventory transfers between stores, often within the same region or group. It calculates net outwards, identifies potential transfers, and suggests pullbacks to warehouses based on store-level needs and available stock. This ensures efficient inventory movement to maximize ROS.

Business Objective: To optimize inventory utilization by rebalancing stock across the store network, transferring excess stock from underperforming locations to high-demand areas, and initiating pullbacks for strategic inventory consolidation.

2. Required Information

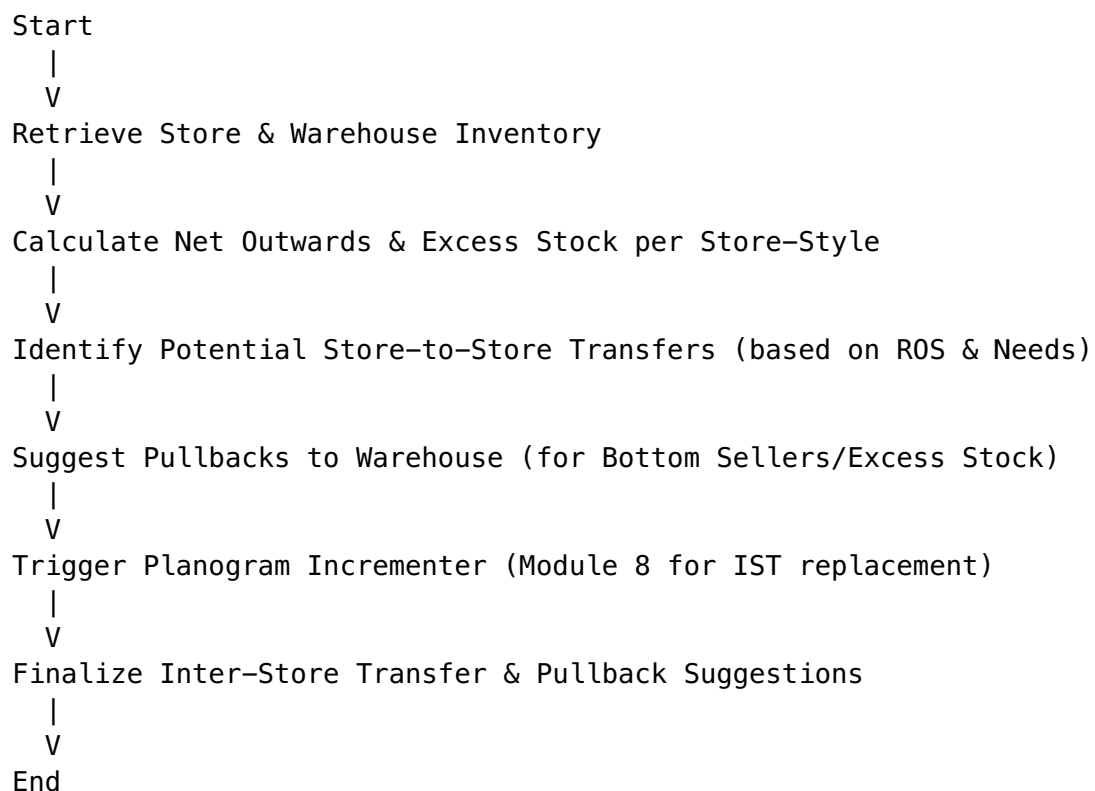
- **Sales Data:**
 - **Rate of Sale (ROS):** To identify high-demand vs. low-demand stores.
- **Product Data:**
 - **SKU master data:** To identify items for transfer.
 - **SKU Depth (OD):** For determining excess stock.
- **Operational Data:**
 - **Current Inventory:** Store stock, goods-in-transit, open orders.
 - **Store Configurations:** Store types, regional groupings for transfers.
 - **Segmentation:** From EOSS Segmentation (e.g., 'Bottom Seller' flags for pullbacks).
 - **Ranking:** From EOSS Ranking (prioritizing transfer needs).
 - **Read-only/Min Outwarding Age flags:** From EOSS Revenue Computation.

- **Warehouse Stock:** From EOSS Warehouse Inventory Management (for pullbacks and replacement transfers).

3. Business Settings

- **Transfer Eligibility Rules:** Criteria for which items can be transferred (e.g., age of stock, condition).
- **Pullback Thresholds:** Rules for when to pull stock back to a warehouse (e.g., excessively high stock relative to ROS, 'Bottom Seller' status).
- **Transfer Distance/Region Constraints:** Limiting transfers to certain geographical areas.
- **IST Replacement Logic:** Rules for replacing transferred stock with new allocation.
- **Minimum Transfer Quantity:** Smallest quantity allowed for a transfer.

4. Process Flow



5. Step-by-Step Actions

1. **Retrieve** current store and warehouse inventory, along with store configuration and sales data *to understand the current stock landscape and store needs*.
2. **Calculate** net outwards and identify excess stock at individual stores *by comparing current stock to optimal levels (OD) and ROS*.
3. **Identify** potential store-to-store transfers *by matching stores with excess stock to stores with high demand for the same SKU within defined regions*.
4. **Suggest** pullbacks to warehouses *for identified 'Bottom Seller' styles or excessively overstocked items, consolidating inventory*.
5. **Trigger** the EOSS Planogram Incrementer (Module 8) *for IST replacement scenarios, ensuring planograms are adjusted to new inventory*.

6. **Finalize** inter-store transfer and pullback suggestions *to create actionable recommendations for inventory movement.*

6. Sample Scenario

Store G has 20 units of “Red Dress” (OD=10), ROS is 1 unit/week (Bottom Seller). Store H has 2 units of “Red Dress” (OD=10), ROS is 8 units/week. * This module **identifies** excess at Store G (10 units above OD). * It **identifies** demand at Store H (8 units below OD). * It **suggests** transferring 8 units of “Red Dress” from Store G to Store H. * The remaining 2 units of excess at Store G are **flagged** for potential pullback to the warehouse. *Key Takeaway: IST prevents stock from stagnating in low-demand stores and ensures high-demand stores remain adequately stocked, driving overall sales efficiency.*

7. Results

- Suggested inter-store transfers (TransferSuggestion objects) with details (from store, to store, SKU, quantity).
- Suggested pullbacks to warehouses.
- Updated distributionData with changes to virtual warehouse stock and pullback stock.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate inventory or ROS data will lead to incorrect transfer suggestions, potentially moving stock to the wrong places.*
- **Physical transfer constraints:** *If a store lacks the capacity or staffing to execute transfers, the suggested transfers may not be feasible.*
- **Circular transfers:** *The system must have logic to prevent items being endlessly transferred between stores.*

9. Next Steps

The transfer suggestions are passed to EOSS Output Generation for persistence and execution. The updated inventory status (including virtual warehouse stock) is used by EOSS Warehouse Inventory Management.

10. Common Questions

1. How do we decide if a store’s stock is “excess”? *It’s based on comparing current stock to the style’s OD, ROS, and overall sales performance.*
2. Why are some items pulled back to the warehouse instead of transferred to another store? *Usually because they are deep ‘Bottom Sellers’ with very low demand across the network, or the cost of transfer is too high.*
3. Are there rules about which stores can transfer to each other? *Yes, transfers are often restricted to specific regions or clusters of stores.*
4. What is “IST replacement logic”? *It’s a feature where stock transferred out* of a store might be replaced by new allocation, ensuring the sending store doesn’t become empty.**

5. How are logistics costs considered in transfers? *While the core module focuses on inventory optimization, logistic costs can be incorporated as a configurable factor or constraint for transfer decisions.*

11. Quick Reference

- **IST:** Inter-Store Transfer
 - **Pullback:** Moving stock from store to warehouse.
 - **Net Outwards:** Total stock leaving a store.
 - **Virtual Warehouse Stock:** Stock conceptually available in warehouse for IST, derived from other modules.
-

EOSS Warehouse Inventory Management

1. Module Name & Business Objective

EOSS Warehouse Inventory Management: Manages warehouse inventory, including removing inefficiency and excluding stock for lead time consideration (ROS). For IST scenarios, it also calculates “virtual warehouse stock” based on leftover store stock, which can then be used for inter-store transfers. *Business Objective:* To provide an accurate, actionable view of available warehouse inventory for allocation and transfer, by accounting for operational inefficiencies, lead times, and integrating stock from store pullbacks.

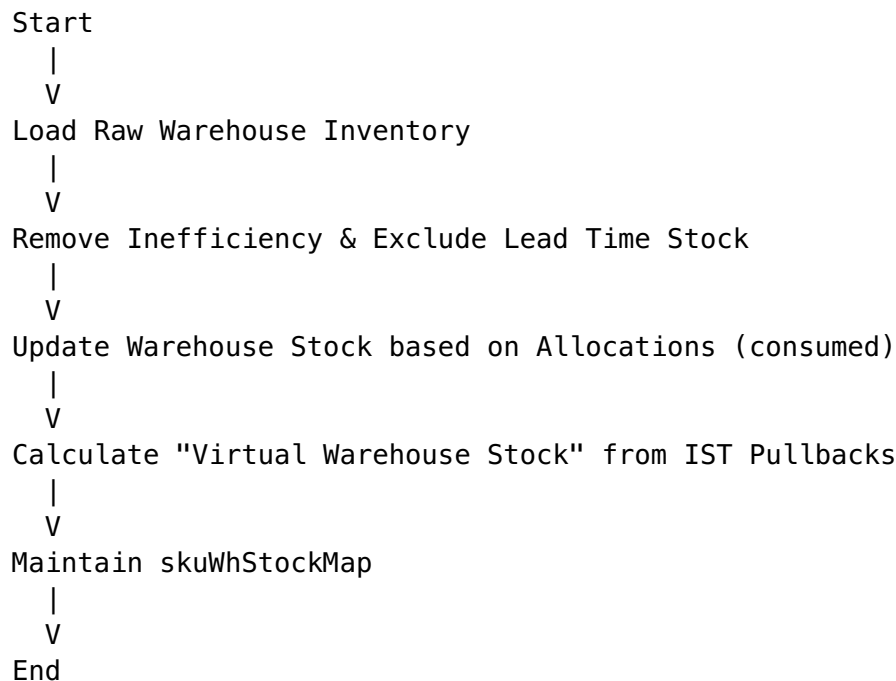
2. Required Information

- **Product Data:**
 - **SKU master data:** For identifying specific SKUs.
- **Operational Data:**
 - **Raw Warehouse Inventory:** Initial stock levels.
 - **Goods-in-transit:** Stock inbound to warehouses.
 - **ROS (Rate of Sale):** For lead time consideration.
 - **Allocation Results:** From EOSS Allocation (to update consumed stock).
 - **IST Results:** From EOSS Inter-Store Transfer (for pullbacks affecting virtual stock).

3. Business Settings

- **Inefficiency Percentage:** % of warehouse stock considered unusable or hard to access.
- **Lead Time Consideration:** How many days/weeks of ROS to reserve as buffer stock.
- **Virtual Stock Creation Rules:** Logic for converting store leftover stock into virtual warehouse stock for IST.

4. Process Flow



5. Step-by-Step Actions

1. **Load** initial raw warehouse inventory *from the data preparation module.*
2. **Remove** inefficiency and exclude stock for lead time consideration (ROS) *to derive the truly available stock for allocation.*
3. **Update** the warehouse stock based on quantities consumed by EOSS *Allocation to reflect the current available inventory.*
4. **Calculate** “virtual warehouse stock” based on leftover store stock from IST scenarios *to account for inventory returning from stores that can be re-allocated.*
5. **Maintain** the skuWhStockMap in distributionData *to provide an up-to-date, accurate view of warehouse stock for downstream modules.*

6. Sample Scenario

Warehouse A has 100 units of “Sneaker Y.” Business rule dictates 5% inefficiency and 2 weeks of ROS (at 10 units/week) for lead time. * Inefficiency: 5 units ($100 * 0.05$). * Lead time buffer: 20 units ($2 \text{ weeks} * 10 \text{ units/week}$). * Available stock for allocation: $100 - 5 - 20 = 75$ units. After EOSS Allocation runs, it may consume 50 units. The module then **updates** warehouse stock to 25 available. If 10 “Sneaker Y” units are pulled back from Store J, this module **adds** them to “virtual warehouse stock.” *Key Takeaway: This module ensures that allocation and transfer decisions are based on the actual, usable quantity of inventory available in warehouses, not just raw counts.*

7. Results

- A refined skuWhStockMap in distributionData representing:
 - Physically available warehouse stock (after inefficiency/lead time deductions).

- virtual warehouse stock derived from IST pullbacks.
- Accurate inventory data for EOSS Allocation and EOSS Inter-Store Transfer.

8. Unusual Cases

- **Missing or delayed data feed:** *Inaccurate warehouse inventory data can lead to over-allocating unavailable stock or under-utilizing available stock.*
- **Sudden spikes in ROS:** *If ROS suddenly increases, the lead time buffer might be insufficient, leading to unexpected stockouts.*

9. Next Steps

The updated warehouse inventory data is continuously fed to EOSS Allocation and EOSS Inter-Store Transfer to inform their decision-making.

10. Common Questions

1. What is “inefficiency” in warehouse stock? *It accounts for stock that is damaged, misplaced, or otherwise not readily available for picking.*
2. Why do we exclude stock for “lead time consideration”? *This ensures there’s always a buffer to cover sales during the time it takes for new stock to arrive at stores from the warehouse.*
3. What is “virtual warehouse stock,” and how is it used? *It’s stock that has been identified for pullback from stores but is not yet physically in the warehouse; it’s considered available for immediate IST re-allocation.*
4. How do pullbacks from stores affect warehouse inventory? *They increase the virtual warehouse stock which can then be allocated to other stores or consolidated.*
5. Can we adjust the inefficiency or lead time percentages? *Yes, these are configurable parameters that should reflect operational realities.*

11. Quick Reference

- **ROS:** Rate of Sale
 - **skuWhStockMap:** Map of SKU to warehouse stock quantity.
 - **Virtual Stock:** Stock from pullbacks, ready for re-allocation.
 - **Lead Time:** Time from order to delivery.
-

EOSS Planogram Incrementer

1. Module Name & Business Objective

EOSS Planogram Incrementer: Calculates and applies increment ratios to planogram stocks (OD) and options (OW) based on revenue plans and average selling prices (ASPs) at various hierarchical levels (store, category, AG). This aims to dynamically adjust planogram targets for better inventory optimization. *Business*

Objective: To dynamically adjust the ideal stock levels (OD) and variety (OW) for products within stores, ensuring planograms reflect current revenue potential and optimize the use of display space for maximum sales.

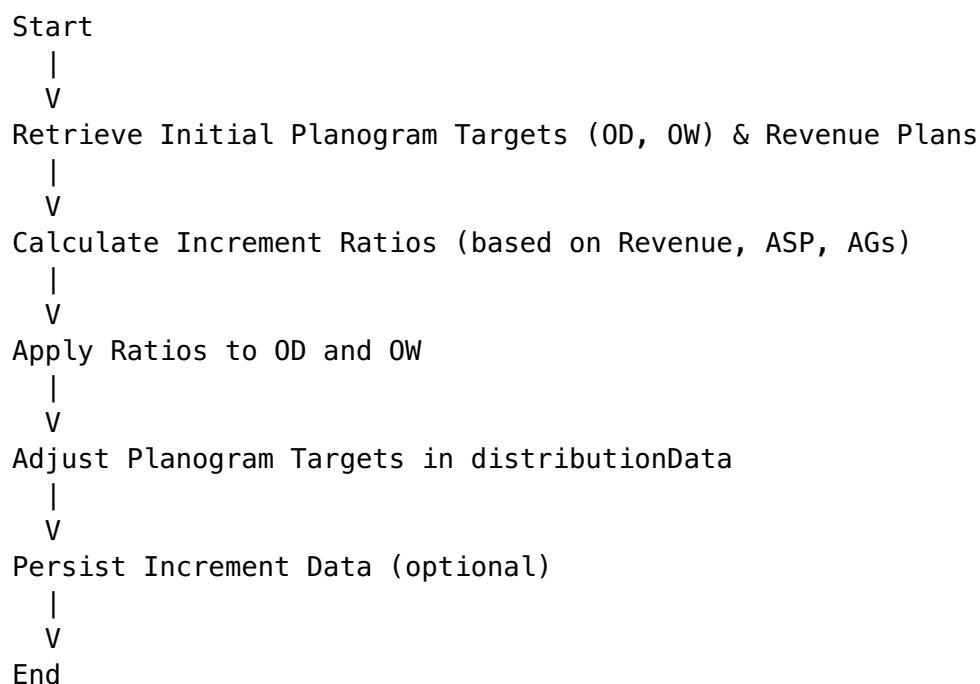
2. Required Information

- **Sales Data:**
 - **Average Selling Price (ASP):** For revenue-based calculations.
- **Product Data:**
 - **SKU Depths (OD):** Initial Optimum Depth.
 - **Optimum Width (OW):** Initial Optimum Width.
 - **Attribute Group (AG) data:** For hierarchical adjustments.
- **Operational Data:**
 - **Revenue Plans:** Future revenue targets for stores/categories.
 - **Store Configurations:** For store-specific adjustments.
 - **Current Planograms:** Existing OD/OW targets.
 - **IST-specific `istSkuWhStockMap`:** From E0SS Warehouse Inventory Management for IST scenarios.

3. Business Settings

- **Increment Calculation Methodology:** How ASP, revenue plans, and current performance influence increments.
- **Hierarchical Levels for Increment:** Whether to apply increments at store, category, or AG level.
- **Max/Min Increment Percentages:** To control the magnitude of adjustments.
- **Trigger Conditions:** When this module should be called (e.g., during allocation iterations, for IST replacement).

4. Process Flow



5. Step-by-Step Actions

1. **Retrieve** initial planogram targets (OD, OW), current revenue plans, and Average Selling Prices (ASPs) *to form the basis for adjustment.*
2. **Calculate** dynamic increment ratios *based on the discrepancy between actual performance and revenue plans, considering ASPs and Attribute Group (AG) performance.*
3. **Apply** these calculated increment ratios to the existing OD (Optimum Depth) and OW (Optimum Width) values *to adjust ideal stock levels and variety.*
4. **Update** the distributionData object with the revised planogram targets *making them available for subsequent allocation and transfer iterations.*
5. **Persist** the calculated increment data to the database *for auditing and reporting purposes.*

6. Sample Scenario

A “Summer Dress” category at Store K has a current OD of 100 units. Its revenue plan has increased by 10%, and its average selling price (ASP) is \$50. * This module **calculates** an increment ratio. If it’s a 5% increment based on the revenue plan and ASP, the OD will **increase** to 105 units ($100 * 1.05$). *Key Takeaway: Dynamic planogram adjustments ensure that display space and inventory targets are continuously optimized to reflect evolving sales potential, rather than remaining static.*

7. Results

- Updated OD and OW targets within distributionData for various store, category, and AG combinations.
- Persisted planogram increment data to the database for reporting.

8. Unusual Cases

- **Missing or delayed data feed:** *If revenue plans or ASP data is missing/stale, increments will be inaccurate, leading to suboptimal planogram adjustments.*
- **Unrealistic revenue plans:** *Overly aggressive or conservative revenue plans can lead to disproportionate or insufficient planogram adjustments.*

9. Next Steps

The updated planogram targets are immediately utilized by EOSS Allocation and EOSS Inter-Store Transfer to guide subsequent allocation iterations and replacement logic.

10. Common Questions

1. What does “planogram incrementer” do? *It dynamically adjusts the ideal number of items (OD) and varieties (OW) on display based on sales performance and revenue plans.*
2. How often are planograms adjusted? *This module is triggered during key allocation and IST phases, ensuring dynamic adjustments.*

3. Can a planogram OD decrease? *Yes, if a category consistently underperforms its revenue plan, the incrementer might suggest a negative adjustment (decrement).*
4. How does ASP influence the increment? *Higher ASPs can lead to different inventory value considerations, impacting the ideal depth and width needed to meet revenue targets.*
5. What happens if the calculated increment exceeds physical store capacity? *The allocation module that uses these targets will then apply further constraints to stay within actual capacity limits.*

11. Quick Reference

- **OD:** Optimum Depth
 - **OW:** Optimum Width
 - **ASP:** Average Selling Price
 - **Planogram:** Visual merchandise display layout and stock levels.
-

EOSS Output Generation

1. Module Name & Business Objective

EOSS Output Generation: Responsible for fanning out and persisting the final allocation (OD, OW, ISS) and transfer outputs into various detailed and denormalized tables. This module handles data transformation from internal models to report-friendly formats, including remarks, segmentations, and allocation types.
Business Objective: To consolidate, format, and persist all final allocation and transfer decisions into easily accessible reports and data tables, providing actionable insights for business users and downstream systems.

2. Required Information

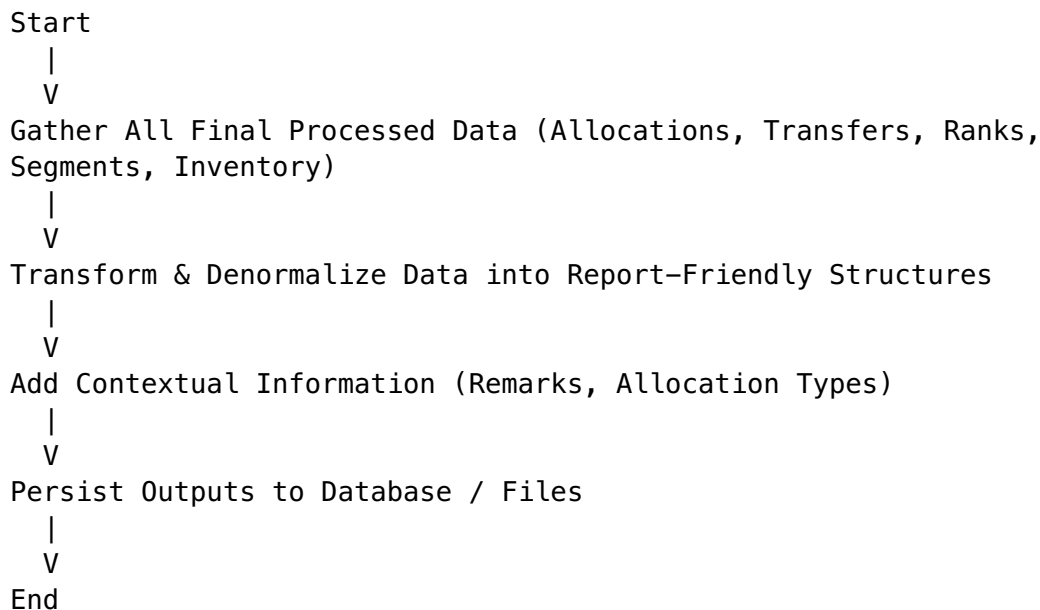
- **Product Data:**
 - **SKU master data:** Parent-child SKU mappings.
- **Operational Data:**
 - **Final Allocation Suggestions:** From EOSS Allocation (OD, OW, ISS).
 - **Transfer Suggestions:** From EOSS Inter-Store Transfer.
 - **Segmentation Data:** From EOSS Segmentation.
 - **Ranking Data:** From EOSS Ranking.
 - **Inventory State:** From EOSS Warehouse Inventory Management.
 - **Planogram Increment Data:** From EOSS Planogram Incrementer.
 - **Remarks/Flags:** Any specific notes or flags generated during the process.

3. Business Settings

- **Output File Formats:** CSV, JSON, database tables.
- **Denormalization Rules:** How to combine data for reports (e.g., joining SKU details with allocation data).
- **Reporting Granularity:** Level of detail for outputs (e.g., SKU level, store level, category level).

- **Error Reporting Configuration:** How to flag and report issues within the output.

4. Process Flow



5. Step-by-Step Actions

1. **Gather** all final processed data, including allocation suggestions, transfer suggestions, segmentation results, ranking data, and the updated inventory state *to consolidate all actionable outputs.*
2. **Transform** and denormalize the data *from internal models into report-friendly structures, joining related information like SKU details, store names, and segment types.*
3. **Add** contextual information such as remarks, specific allocation types (e.g., fresh allocation, replenishment), and reasons for transfers *to provide clear understanding for business users.*
4. **Persist** the final outputs into various detailed and denormalized tables or files (e.g., CSV, database tables) *to make the results available for reporting, analysis, and execution by other systems.*

6. Sample Scenario

After the EOSS run, Store L is recommended to receive 5 units of “Green Scarf” (OD=10, OW=5). This was a fresh allocation for a Top Seller style. Also, 3 units of “Old Jeans” are recommended for pullback from Store M. * This module **formats** the “Green Scarf” allocation to show Store_ID, SKU_ID, Allocated_Qty, Allocation_Type: 'Fresh', Remarks: 'Top Seller', OD: 10, OW: 5. * It **formats** the “Old Jeans” pullback to show From_Store_ID, SKU_ID, Pullback_Qty, Transfer_Type: 'Pullback'. * These formatted records are then **written** to the designated output tables/files. *Key Takeaway: Output Generation is crucial for translating complex algorithmic decisions into clear, actionable business recommendations and reports.*

7. Results

- Database tables or flat files containing:
 - Final suggested SKU allocations (OD, OW, ISS) per store.
 - Inter-store transfer and pullback suggestions.
 - Enriched data with segmentation, ranking, remarks, and allocation types.
 - Detailed inventory state reports.

8. Unusual Cases

- **Missing or delayed data feed:** *If previous modules fail to provide their outputs, this module will generate incomplete or empty reports.*
- **Schema mismatches:** *Changes in internal data models not reflected in output schema can lead to data corruption or failed writes.*

9. Next Steps

The generated outputs are consumed by various downstream systems for order fulfillment, logistics planning, reporting dashboards, and manual review by business teams.

10. Common Questions

1. Where can I find the final allocation reports? *They are typically in the designated output directory (. / outputs) or specific database tables.*
2. What do the “remarks” or “allocation types” mean? *They provide context on why a specific allocation or transfer was made (e.g., ‘Fresh Allocation’, ‘Replenishment’, ‘Bottom Seller Pullback’).*
3. Can we get outputs in a different format (e.g., Excel, API)? *The module currently outputs to configured formats (e.g., CSV, DB); custom integrations might be needed for other formats.*
4. How can I verify the accuracy of the outputs? *Cross-reference with inputs and intermediate data from other modules, or use built-in validation checks.*
5. What if an allocation is 0? *A 0 allocation means no stock was recommended for that SKU at that store, often due to sufficient stock, no demand, or business rules preventing allocation.*

11. Quick Reference

- **OD:** Optimum Depth
 - **OW:** Optimum Width
 - **ISS:** Ideal Size Set
 - **Denormalization:** Combining data from multiple tables for reports.
-

EOSS Orchestration

1. Module Name & Business Objective

EOSS Orchestration: Acts as the high-level orchestrator for the entire EOSS distribution process. It defines the sequence of execution for all other EOSS modules, including data preparation, sales processing, revenue calculation, segmentation, ranking, allocation, inter-store transfers, and final output generation. It also integrates with snapshot and output synchronization utilities. *Business Objective:* To manage the end-to-end execution flow of the EOSS pipeline, ensuring that each module runs in the correct order, data dependencies are met, and the overall process is robust, efficient, and auditable.

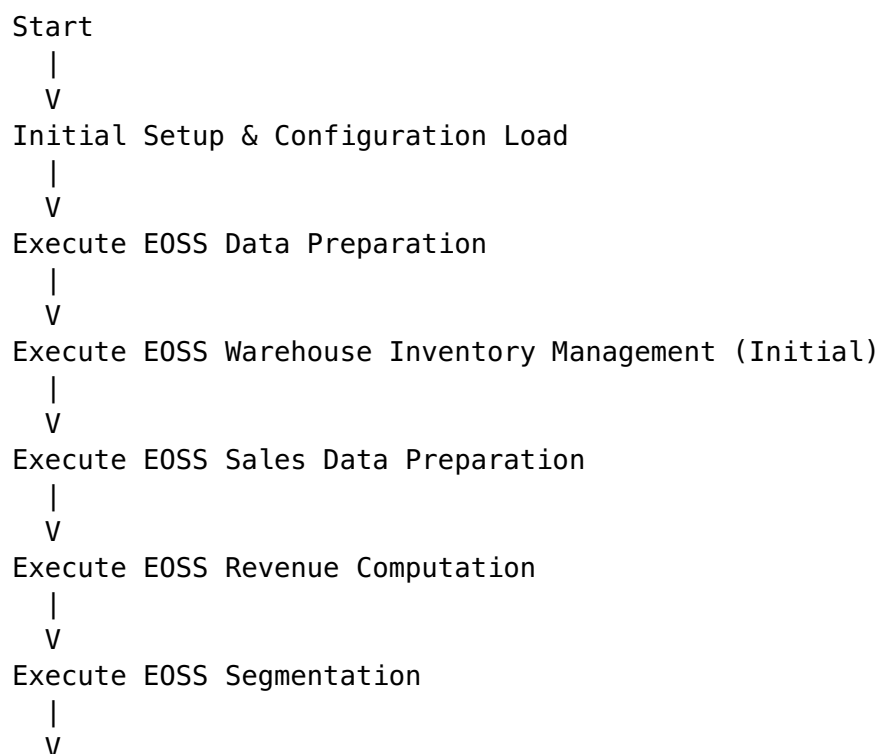
2. Required Information

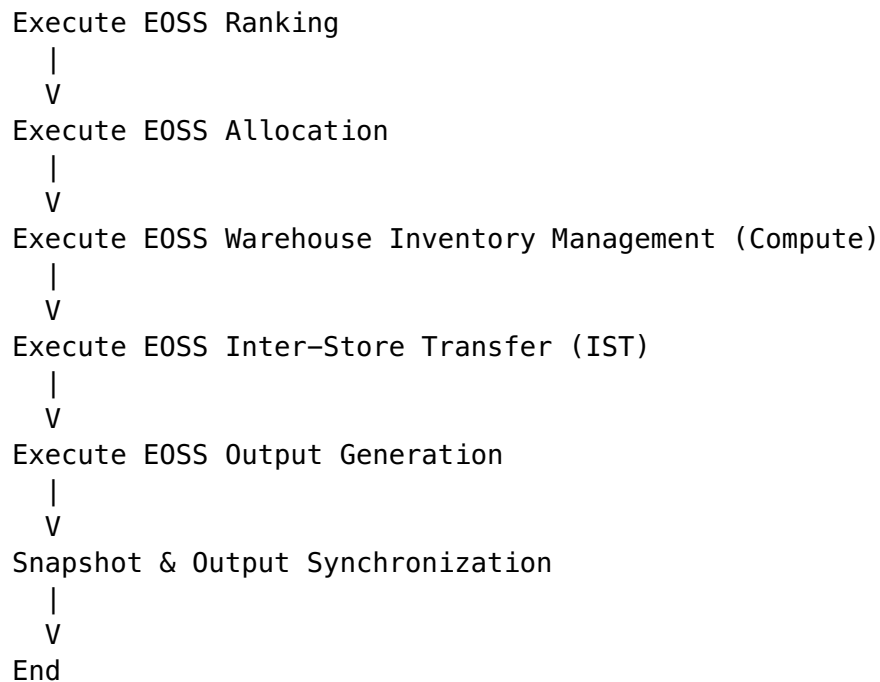
- **Operational Data:**
 - **Module configurations:** Order of execution, error handling rules.
 - **Snapshot utilities:** For capturing intermediate data states.
 - **Output synchronization utilities:** For managing final data delivery.

3. Business Settings

- **Module Execution Order:** Defines the sequence of module calls.
- **Error Handling Strategy:** How to react to module failures (e.g., retry, stop, continue with warnings).
- **Logging Level:** Verbosity of runtime logs.
- **Snapshot Frequency:** How often to save intermediate data states.

4. Process Flow





5. Step-by-Step Actions

1. **Load** global configurations and module-specific parameters *to initialize the execution environment.*
2. **Sequentially invoke** each EOSS submodule (e.g., EOSS Data Preparation, EOSS Sales Data Preparation, EOSS Allocation, EOSS Inter-Store Transfer) *ensuring data dependencies are met at each step.*
3. **Manage** the flow of distributionData between modules *to ensure consistent and up-to-date information is passed along the pipeline.*
4. **Handle** exceptions and errors *by logging failures and, if configured, attempting retries or stopping the process.*
5. **Integrate** with snapshot and output synchronization utilities *to save intermediate results and ensure final outputs are delivered reliably.*

6. Sample Scenario

The EOSS pipeline is started. * The orchestrator first **triggers** EOSS Data Preparation to load all master data. * Once complete, it **calls** EOSS Sales Data Preparation to clean sales and compute ROS. * If EOSS Sales Data Preparation fails, the orchestrator might **log** the error and **halt** the entire run, or attempt a retry based on configuration. * Finally, it **ensures** EOSS Output Generation runs to persist all results, and then **syncs** these outputs to a shared location. *Key Takeaway: The Orchestration module is the brain of the EOSS system, guaranteeing a controlled, reliable, and logical execution of the complex inventory optimization process.*

7. Results

- Successful end-to-end execution of the EOSS pipeline.
- All intermediate and final data points are processed and transferred correctly between modules.
- Final outputs are generated and synchronized.

- Detailed execution logs are produced, aiding in troubleshooting.

8. Unusual Cases

- **Missing or delayed data feed:** *The orchestrator might fail at the Data Preparation stage, preventing any further execution.*
- **Module runtime exceptions:** *An unhandled error in any submodule will be caught by the orchestrator, potentially halting the entire process.*

9. Next Steps

The EOSS run is complete. Business users can now access the generated outputs for analysis and execution.

10. Common Questions

1. How long does a full EOSS run take? *Runtime varies based on data volume; typically, it's configured to complete within specific batch windows.*
2. What happens if one module fails during the run? *The orchestrator's error handling configuration determines if it retries, stops, or continues with warnings.*
3. Can we run only a specific part of the EOSS process (e.g., just sales data prep)? *The orchestrator runs the full pipeline as defined; partial runs typically require custom configuration or development.*
4. How do we know the run completed successfully? *Check the final status in the logs or the presence of the expected output files.*
5. Is there a dashboard to monitor the EOSS run? *While not explicitly mentioned, integration with monitoring tools for progress and health checks is common for production deployments.*

11. Quick Reference

- **Orchestration:** Managing the sequence and flow of modules.
 - **distributionData:** Central in-memory data store.
 - **Snapshot:** Saving intermediate data states.
-

9. Appendix

Full Parameter Reference

(This section would typically list all configuration parameters from `args.properties` with detailed explanations, valid ranges, and usage examples. As the actual `args.properties` was not provided in the prompt, please refer to the `args.properties` file in the project's root directory for the complete and up-to-date list.)

Version History

- **Version 1.0 (2025-05-23):** Initial release of the EOSS Distribution System, integrating core data preparation, sales processing, revenue calculation, segmentation, ranking, allocation, inter-store transfer, warehouse management, planogram incrementation, and output generation functionalities.

Contact & Support Info

For any questions, issues, or feature requests regarding the EOSS Distribution System, please contact: * **Support Team:** support@yourcompany.com * **Product Owner:** productowner@yourcompany.com * **Technical Lead:** techlead@yourcompany.com * **Internal Documentation:** [Link to Internal Wiki/Confluence Page]

Tutorial: eoss

The EOSS distribution system processes inventory allocation and transfer through a pipeline of interdependent modules. The EOSS Orchestration module defines the execution sequence, ensuring data flows correctly between preparation, sales processing, revenue calculation, segmentation, ranking, core allocation, inter-store transfers, and final output generation. A shared in-memory data store facilitates efficient data exchange and transformation across the modules.

flowchart TD

```
A0["EOSS Data Preparation"]
A1["EOSS Sales Data Preparation"]
A2["EOSS Revenue Computation"]
A3["EOSS Segmentation"]
A4["EOSS Ranking"]
A5["EOSS Allocation"]
A6["EOSS Inter-Store Transfer (IST)"]
A7["EOSS Warehouse Inventory Management"]
A8["EOSS Planogram Incrementer"]
A9["EOSS Output Generation"]
A10["EOSS Orchestration"]
A10 -- "orchestrates" --> A0
A10 -- "orchestrates" --> A7
A10 -- "orchestrates" --> A1
A10 -- "orchestrates" --> A2
A10 -- "orchestrates" --> A3
A10 -- "orchestrates" --> A4
A10 -- "orchestrates" --> A5
A10 -- "orchestrates" --> A7
A10 -- "orchestrates" --> A6
A10 -- "orchestrates" --> A9
A0 -- "initializes_data" --> A1
A0 -- "initializes_data" --> A2
A0 -- "initializes_data" --> A3
A0 -- "initializes_data" --> A4
A0 -- "initializes_data" --> A5
A0 -- "initializes_data" --> A6
A0 -- "initializes_data" --> A7
A0 -- "initializes_data" --> A8
A0 -- "initializes_data" --> A9
A1 -- "provides_sales" --> A2
A1 -- "provides_ros" --> A3
A1 -- "provides_ros" --> A4
A2 -- "updates_styles" --> A3
A2 -- "updates_styles" --> A4
A2 -- "updates_styles" --> A5
A2 -- "updates_styles" --> A6
```

```
A3 -- "updates_styles" --> A4
A3 -- "guides_allocation" --> A5
A3 -- "guides_ist" --> A6
A3 -- "provides_segments" --> A9
A4 -- "prioritizes_allocation" --> A5
A4 -- "prioritizes_ist" --> A6
A4 -- "provides_ranks" --> A9
A5 -- "triggers" --> A8
A5 -- "provides_allocations" --> A9
A5 -- "updates_inventory" --> A7
A6 -- "triggers" --> A8
A6 -- "provides_transfers" --> A9
A6 -- "updates_inventory" --> A7
A7 -- "provides_inventory" --> A5
A7 -- "provides_inventory" --> A6
A7 -- "configures" --> A8
A7 -- "provides_inventory_state" --> A9
A8 -- "updates_planogram_targets" --> A5
A8 -- "updates_planogram_targets" --> A6
A8 -- "provides_increment_data" --> A9
```

Chapters

Master Report: Merchandising Algorithm

1. Title & Metadata

- **Project Name:** eoss
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** eoss is a merchandising algorithm designed to optimize product assortments, stock levels, and size distributions to maximize sales and minimize stockouts across categories and stores.

2. Executive Summary

The eoss merchandising algorithm leverages historical sales, inventory, and product attribute data to provide data-driven recommendations for optimal product assortments. Its primary goal is to enhance merchandising efficiency by recommending the Optimum Depth (OD) and Optimum Width (OW) of merchandise, along with the Ideal Size Set (ISS) for each product and location. By accurately forecasting demand and analyzing Rate of Sale (ROS), eoss aims to reduce missed sales opportunities, minimize overstocking, and ultimately improve profitability.

- **Key outcomes or KPIs:** Increased ROS, optimized OD & OW, improved ISS accuracy, reduced stockouts, minimized excess inventory, enhanced sales per square foot.
- **Primary stakeholders or downstream consumers:** Merchandising Planners, Category Managers, Inventory Management, Store Operations Teams, Supply Chain Analysts.

3. Architecture & Data Flow

Diagram:

graph TD

```
A[Raw Data Ingestion] --> B{Data Preprocessing & Validation};
B --> C[Sales History Database];
B --> D[Product Master Data];
B --> E[Inventory & Location Data];

C & D & E --> F[Demand Forecasting Module];
F --> G[Attribute Group (AG) Analysis];
```

```

G --> H[Assortment Optimization Module];
G --> I[Size Set Optimization Module];

H --> J[Optimum Depth (OD) Recommendations];
H --> K[Optimum Width (OW) Recommendations];
I --> L[Ideal Size Set (ISS) Recommendations];

J & K & L --> M[Reporting & Output Generation];
M --> N[Merchandising Planning Systems];
M --> O[Inventory Management Systems];
M --> P[Business Intelligence Dashboards];

```

Component Bullets:

- **Raw Data Ingestion:** Collects data from various enterprise systems.
- **Data Preprocessing & Validation:** Cleans, transforms, and validates incoming data for consistency and accuracy.
- **Sales History Database:** Stores time-series sales data, critical for trend analysis and forecasting.
- **Product Master Data:** Central repository for product attributes, hierarchies, and classifications.
- **Inventory & Location Data:** Provides current stock levels, store capacities, and distribution information.
- **Demand Forecasting Module:** Predicts future sales volumes at various granularities (SKU, category, store).
- **Attribute Group (AG) Analysis:** Analyzes performance based on predefined Attribute Groups (e.g., color-size combinations, material-brand).
- **Assortment Optimization Module:** Determines the ideal mix and breadth of products (OD, OW) for specific locations or categories.
- **Size Set Optimization Module:** Calculates the Ideal Size Set (ISS) distribution for each product or product Attribute Group to maximize sell-through.
- **Reporting & Output Generation:** Formats the optimized recommendations into consumable reports and data files.
- **Downstream Systems:** Integrates with merchandising, inventory, and BI platforms for actionable insights.

High-level data sources:

- POS (Point of Sale) transaction data
- ERP (Enterprise Resource Planning) systems for inventory and product master data
- Warehouse Management Systems (WMS) for stock levels
- Store layout and capacity data
- Promotional calendars and marketing campaign data
- Web analytics for online browsing and purchase behavior

4. Configuration & Data Dictionary

Settings Table (args.properties):

Setting Key	Description	Default Value	Data Type
sales_history_window_weeks	Number of weeks of historical sales data to consider for analysis.	52	Integer
forecast_horizon_weeks	Number of weeks into the future to forecast demand.	12	Integer
min_ROS_threshold	Minimum Rate of Sale (units/week) for a SKU to be considered viable.	0.5	Decimal
max_OD_per_category	Maximum Optimum Depth (SKUs) allowed per category.	250	Integer
min_OD_per_category	Minimum Optimum Depth (SKUs) required per category.	10	Integer
store_cluster_strategy	Strategy for grouping stores (e.g., sales_volume, demographics, geographic).	sales_volume	String
target_stockout_rate	Acceptable percentage of stockouts.	0.05	Decimal
size_curve_smoothing_factor	Factor for smoothing historical size curve data for ISS.	0.1	Decimal

Shared Data Fields (common-data.json):

Field Name	Description	Data Type	Example Value
product_id	Unique identifier for a product (SKU).	String	PROD-00123
category	Product category (e.g., “Men’s T-Shirts”, “Women’s Jeans”).	String	Apparel: Tops
subcategory	Finer product classification.	String	T-Shirts
brand	Brand name of the product.	String	A-Brand
color	Product color.	String	Navy Blue
size	Product size (e.g., “S”, “M”, “L”, “32x32”).	String	M
store_id	Unique identifier for a retail location.	String	STORE-NYC-001
sale_date	Date of the sales transaction.	Date	2024-03-15
units_sold	Number of units sold on a given date/transaction.	Integer	3
current_stock	Current inventory level for a SKU at a location.	Integer	15
price	Selling price of the product.	Decimal	29.99
cost	Cost of goods sold for the product.	Decimal	12.50

Outputs Catalog:

Output Name	Description	Format	Key Fields
optimized_assortment_plan.csv	Recommended OD (SKUs per category) and OW (categories per store).	CSV	store_id, category, recommended_skus, target_ow
size_set_recommendations.json	Ideal Size Set (ISS) distribution for each SKU/ location.	JSON	product_id, store_id, recommended_iss_distribution
inventory_forecast.csv	Projected sales and recommended stock levels.	CSV	product_id, store_id, week, forecasted_sales, recommended_stock
ROS_metrics_dashboard.json	Aggregated Rate of Sale (ROS) performance metrics.	JSON	category, product_id, avg_ROS, weekly_ROS_trend
attribute_group_performance.csv	Performance metrics broken	CSV	

Output Name	Description	Format	Key Fields
	down by Attribute Group (AG).		attribute_group, total_sale avg_ROS, stockout_rate

5. End-to-End Flow

The eoss algorithm operates in a sequential, data-driven pipeline:

- 1. Data Ingestion:** Raw sales, inventory, and product master data are collected from various source systems. This data is often granular, down to individual transactions and SKU-store levels.
- 2. Data Preprocessing:** The ingested data undergoes rigorous cleaning, validation, and transformation. Missing values are imputed, outliers handled, and data is aggregated to the appropriate level for analysis (e.g., weekly sales per SKU per store).
- 3. Demand Forecasting:** Utilizing the preprocessed sales history, sophisticated forecasting models (e.g., ARIMA, Prophet, machine learning models) predict future demand for each SKU at each location for a specified `forecast_horizon_weeks`.
- 4. Attribute Group (AG) Analysis:** Product attributes (e.g., color, material, style, size) are grouped into Attribute Groups. The performance of these AGs (e.g., their historical ROS, sales contribution, customer preferences) is analyzed to understand underlying demand drivers and identify top-performing combinations.
- 5. Assortment Optimization:** Based on demand forecasts, AG performance, store capacity, and business constraints (e.g., `max_OD_per_category`), the algorithm determines the Optimum Depth (OD) – the ideal number of unique SKUs to stock within a category, and Optimum Width (OW) – the ideal number of categories to offer at a specific store or cluster. This module prioritizes high-ROS items and ensures a balanced offering.
- 6. Size Set Optimization:** For the selected SKUs within the optimized assortment, the algorithm calculates the Ideal Size Set (ISS). This involves analyzing historical sales by size (size curves) for similar products or AGs, and applying statistical models to recommend the optimal proportion of each size to minimize stockouts and overstocking within a given inventory budget.
- 7. Output Generation:** All recommendations (OD, OW, ISS, forecasted demand, recommended stock levels) are compiled into structured outputs (CSV, JSON) for integration with downstream merchandising and inventory systems.
- 8. Reporting & Monitoring:** Key performance indicators (ROS, stockout rates, inventory turns) are calculated and presented in dashboards to allow stakeholders to monitor the algorithm’s effectiveness and make adjustments.

6. Submodule Overview

6.1. Data Ingestion & Preprocessing

- **Purpose:** To collect, clean, validate, and prepare raw enterprise data for use by subsequent modules.

- **Inputs:**
 - Raw sales transaction logs
 - Product master data exports
 - Current inventory feeds
 - Store metadata
- **Outputs:**
 - Cleaned and aggregated sales_history (e.g., weekly sales per SKU per store)
 - Standardized product_catalog with all relevant attributes
 - Up-to-date inventory_snapshot
- **Core Logic Summary:**
 - Ingests data from disparate sources (flat files, databases, APIs).
 - Performs data type conversion, handles missing values (e.g., imputation, removal), and validates data integrity.
 - Aggregates raw transaction data to a consistent time granularity (e.g., weekly or daily sales) and product/location level.
 - Enriches data by joining sales with product attributes and store metadata.

6.2. Demand Forecasting

- **Purpose:** To predict future sales quantities for products at specific locations over a defined horizon.
- **Inputs:**
 - Cleaned sales_history
 - product_catalog (for seasonal flags, promotional indicators)
 - forecast_horizon_weeks from configuration
- **Outputs:**
 - forecasted_sales_data (e.g., weekly forecasted units per SKU per store)
- **Core Logic Summary:**
 - Selects appropriate forecasting models (e.g., time-series models like ARIMA, exponential smoothing, or machine learning models like Random Forest, XGBoost) based on data characteristics.
 - Trains models using historical sales data, accounting for seasonality, trends, and promotions.
 - Generates probabilistic forecasts, including mean predictions and confidence intervals.
 - Evaluates forecast accuracy using metrics like MAE, RMSE, or MAPE.

6.3. Assortment Optimization

- **Purpose:** To determine the Optimum Depth (OD) of SKUs within categories and the Optimum Width (OW) of categories offered, balancing sales potential with operational constraints.
- **Inputs:**
 - forecasted_sales_data
 - AG_performance_metrics
 - product_catalog (for category, subcategory, brand)
 - store_metadata (e.g., store cluster, capacity)
 - Configuration parameters (max_OD_per_category, min_OD_per_category)
- **Outputs:**
 - optimized_assortment_plan (list of recommended SKUs per category per store, along with target OD and OW)

- **Core Logic Summary:**
 - Ranks SKUs within each category and store cluster based on forecasted ROS and profitability.
 - Applies business rules and constraints (e.g., minimum viable ROS, category space allocations, max_OD) to filter and select SKUs.
 - Considers diversity within categories (e.g., ensuring a range of price points or colors) and across categories (OW) to meet customer demand.
 - Outputs a list of recommended SKUs for each store/category combination, forming the ideal assortment.

6.4. Size Set Optimization

- **Purpose:** To calculate the Ideal Size Set (ISS) distribution for each recommended product/Attribute Group to maximize sell-through and minimize stockouts.
- **Inputs:**
 - optimized_assortment_plan (list of recommended SKUs)
 - Historical sales data by size (size curves) for relevant AGs
 - target_stockout_rate from configuration
 - size_curve_smoothing_factor
- **Outputs:**
 - size_set_recommendations (percentage breakdown for each size for each recommended SKU/AG)
- **Core Logic Summary:**
 - Identifies relevant historical sales data for the selected products, often by grouping similar products into AGs (e.g., “Men’s Classic Fit Jeans”).
 - Analyzes the historical proportion of sales for each size within the AG to derive a “size curve.”
 - Applies statistical smoothing techniques to the size curve to account for fluctuations or missing data.
 - Adjusts the size curve based on target stockout rates and ensures practical allocation (e.g., whole units). The output is a percentage distribution (e.g., S:10%, M:30%, L:30%, XL:20%, XXL:10%).

6.5. Output Generation

- **Purpose:** To format the optimized recommendations into user-friendly reports and machine-readable files for downstream systems.
- **Inputs:**
 - optimized_assortment_plan
 - size_set_recommendations
 - forecasted_sales_data
 - ROS_metrics
 - attribute_group_performance
- **Outputs:**
 - optimized_assortment_plan.csv
 - size_set_recommendations.json
 - inventory_forecast.csv
 - ROS_metrics_dashboard.json
 - attribute_group_performance.csv
- **Core Logic Summary:**
 - Aggregates and transforms raw recommendation data into specified output schemas.

- Generates CSV files for tabular data and JSON files for hierarchical or complex structures.
- Includes metadata and timestamps in output files for traceability.
- Ensures output files are accessible by relevant downstream systems via defined interfaces (e.g., shared network drives, SFTP, API endpoints).

7. Worked Example

Let's consider a single category: "Men's Casual T-Shirts" in a specific store, "STORE-NYC-001".

Scenario: The merchandising team wants to optimize the assortment for "Men's Casual T-Shirts" for the next 12 weeks. The category currently has 50 SKUs.

Inputs: * **Sales History:** * SKU-A (Blue, Cotton, M): 10 units/week ROS * SKU-B (Red, Cotton, M): 4 units/week ROS * SKU-C (Green, Blend, L): 2 units/week ROS * ... (47 other SKUs with varying ROS) * **Store Capacity:** Max 30 SKUs for "Men's Casual T-Shirts". * **min_ROS_threshold:** 1.0 units/week. * **Size Curve for "Men's Casual T-Shirts" AG (historical average):** S:10%, M:30%, L:30%, XL:20%, XXL:10%.

End-to-End Flow Walkthrough:

1. **Data Ingestion & Preprocessing:** Sales data for "Men's Casual T-Shirts" from STORE-NYC-001 over the last 52 weeks is ingested, cleaned, and aggregated to weekly SKU-level sales. Product attributes (color, material, size) are linked.
2. **Demand Forecasting:** The Demand Forecasting Module predicts future weekly sales for all 50 SKUs for the next 12 weeks based on their historical performance and seasonality. For example, SKU-A's forecasted ROS remains strong at 9.5 units/week. SKU-C's forecasted ROS is 1.8 units/week.
3. **Attribute Group (AG) Analysis:** The Attribute Group for "Men's Casual T-Shirts" is analyzed. The algorithm identifies that cotton t-shirts in classic colors (Blue, Black, White) consistently have higher ROS than blends or novelty colors. The aggregated size curve for all "Men's Casual T-Shirts" is computed.
4. **Assortment Optimization:**
 - All 50 SKUs are ranked by their forecasted ROS and profitability.
 - SKUs with forecasted ROS below the min_ROS_threshold of 1.0 units/week are immediately deprioritized or flagged for removal.
 - The algorithm selects the top-ranked SKUs, ensuring a balanced representation of key colors and styles within the category, until the max_OD_per_category (30 SKUs) is reached.
 - **Result:** The algorithm recommends an Optimum Depth of 30 SKUs. For example, it might select SKU-A (Blue Cotton M) and other high-performing cotton options, while dropping SKU-C (Green Blend L) if its forecasted ROS is too low, or if better performing alternatives exist. It also considers maintaining a basic range of colors/materials.

5. Size Set Optimization:

- For each of the 30 recommended SKUs, the Ideal Size Set (ISS) is applied.
- Using the AG's historical size curve (S:10%, M:30%, L:30%, XL:20%, XXL:10%) and adjusting for individual SKU characteristics or local demand variations, the algorithm calculates the optimal number of units per size for any given total order quantity.
- **Result:** If STORE-NYC-001 plans to stock 100 units of SKU-A, the ISS recommendation would be: 10 S, 30 M, 30 L, 20 XL, 10 XXL.

6. Output Generation:

- A `optimized_assortment_plan.csv` is generated, listing the 30 recommended SKUs for "Men's Casual T-Shirts" in STORE-NYC-001.
- A `size_set_recommendations.json` is generated with the ISS breakdown for each of these 30 SKUs.
- An `inventory_forecast.csv` provides weekly demand forecasts for these 30 SKUs for the next 12 weeks.

Outcome: The merchandising team receives precise recommendations, enabling them to de-list underperforming SKUs, maintain a concise and productive assortment, and stock the right sizes for each item, leading to higher ROS and fewer lost sales or excess inventory.

8. Debug Guide Highlights

- **Unexpected ROS or Forecasts:**
 - **Check:** Source data quality for `sales_history`. Are there missing dates, duplicate transactions, or incorrect units sold?
 - **Remedy:** Validate preprocessing logs. Re-run data ingestion and cleaning if issues are found. Review `sales_history_window_weeks` in `args.properties` for sufficient data.
- **Assortment Recommendations seem "off":**
 - **Check:** `min_ROS_threshold`, `max_OD_per_category`, and `store_cluster_strategy` in `args.properties`. Are they set appropriately for the business context?
 - **Remedy:** Adjust configuration settings. Review `AG_performance_metrics` – could there be an issue with how Attribute Groups are defined or analyzed?
- **ISS Recommendations don't match expectations:**
 - **Check:** Historical sales data by size. Is it clean and representative? Is the `size_curve_smoothing_factor` too aggressive or too lenient?
 - **Remedy:** Verify data for specific Attribute Groups. Experiment with different smoothing factors or consider alternative AG definitions for size curves.
- **Performance Issues (Slow Run Times):**
 - **Check:** Data volume and hardware resources. Are databases optimized?
 - **Remedy:** Optimize database queries. Consider parallel processing for computationally intensive modules (e.g., Demand Forecasting across many SKUs/stores). Review logging for bottlenecks.

- **Output Files are Missing or Corrupted:**
 - **Check:** File path configurations, write permissions, and available disk space.
 - **Remedy:** Verify output directories exist and have correct permissions. Check log files for any I/O errors during Output Generation.

9. Common Pitfalls & Remedies

- **Pitfall: Stale Data:** The algorithm runs on outdated sales or inventory data, leading to irrelevant recommendations.
 - **Remedy:** Implement robust data refresh schedules. Use automated data validation checks at the start of the pipeline to flag stale or missing feeds before processing.
- **Pitfall: Overfitting to Historical Anomalies:** Demand Forecasting models might over-index on one-off spikes (e.g., extreme promotions, short-term stockouts) leading to inaccurate forecasts.
 - **Remedy:** Incorporate event flagging (promotions, holidays, stockouts) into the forecasting model. Use robust regularization techniques or ensemble methods. Consider “sense-checking” forecasts with human expertise.
- **Pitfall: Ignoring Qualitative Factors:** The algorithm purely uses quantitative data, missing nuances like new product launches, fashion trends, or competitor actions.
 - **Remedy:** Build mechanisms for human override or input on specific product lines or categories. Integrate external market trend data or qualitative merchandising insights where possible.
- **Pitfall: One-Size-Fits-All Logic:** Applying the same OD, OW, or ISS rules across vastly different store types or product categories.
 - **Remedy:** Implement intelligent store clustering and category segmentation. Allow for granular configuration settings (e.g., max_OD_per_category can vary by category type, not just a global default).
- **Pitfall: Lack of Iteration/Feedback Loop:** Deploying the algorithm without a plan to monitor its effectiveness and refine its parameters.
 - **Remedy:** Establish clear KPIs (ROS, stockout rates, inventory turns) and regular reporting. Set up A/B testing frameworks for recommendations to validate their impact before full rollout. Schedule periodic reviews and recalibration of `args.properties` settings.

10. FAQ

- **Q: How often should the eoss algorithm be run?**
 - A: It is recommended to run eoss weekly or bi-weekly to capture recent sales trends and inventory changes for accurate and timely recommendations. For strategic planning, a monthly run might suffice.
- **Q: Can eoss account for promotions or holidays?**
 - A: Yes, the Demand Forecasting module is designed to incorporate promotional calendars and holiday effects provided this data is part of the Raw Data Ingestion.
- **Q: What if a new product has no sales history for ROS or ISS?**
 - A: For new products, eoss uses “like-product” analysis. It identifies similar products based on Attribute Groups (e.g., category, material, price point) to estimate initial ROS and derive an ISS. Over time, as sales data

accumulates, the recommendations become more specific to the new product.

• **Q: How does eoss handle products with very low ROS?**

- A: Products falling below the `min_ROS_threshold` are typically flagged for review or recommended for markdown/discontinuation within the Assortment Optimization Module, unless explicitly protected by business rules (e.g., core basics).

• **Q: Is eoss a replacement for human merchandisers?**

- A: No, eoss is a powerful tool to augment and empower merchandisers. It automates data analysis and generates data-driven recommendations, freeing up merchandisers to focus on strategic decisions, trend spotting, and creative assortment building. Human oversight and qualitative insights remain crucial.

11. Quick Reference Cheat-Sheet

Term | Definition | Key Metric/Concept `

OD | Optimum Depth = The ideal number of unique items (SKUs) to stock in a category or display. | Target number of unique SKUs for a specific category. |

OW | Optimum Width = The ideal number of different categories or styles to offer. | Number of distinct product categories in an assortment. |

ISS | Ideal Size Set = The best combination of sizes to maximize sales and minimize stockouts. | Percentage breakdown of sales by size (e.g., S:10%, M:30%, L:30%). |

ROS | Rate of Sale = The speed at which products are sold over a period (e.g., units/week). | Units sold / Time period (e.g., 5 units/week). |

AG | Attribute Group = A set of product attributes grouped for analysis (e.g., color, size, brand). | `Mens_Tshirt_Cotton_Blue` |

`args.properties` | Configuration file for algorithm parameters. | |

`common-data.json` | Definition of shared data fields across modules. | |

`forecast_horizon` | Time period for which demand is projected. | 12 weeks |

Stockout Rate | Percentage of time an item is out of stock when there is demand. | 0.05 (5%) |

StyleWiseToSizeWise FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document provides clear, concise answers to common business questions about the StyleWiseToSizeWise algorithm, designed for merchandisers, planners, and business users.

General

Q: What is the main purpose of the StyleWiseToSizeWise algorithm? **A:** The StyleWiseToSizeWise algorithm is designed to transform high-level purchasing plans (style-wise buy quantities) into detailed, actionable size-level quantities, effectively determining the Ideal Size Set (ISS) for every product and optimizing stock decisions for better Rate of Sale (ROS).

Q: Why is calculating the Ideal Size Set (ISS) so important? **A:** Calculating the ISS ensures that we stock the right combination of sizes for our products. This minimizes stockouts on popular sizes, reduces excess inventory on less popular ones, and directly contributes to a higher Rate of Sale (ROS) by ensuring product availability where demand is highest. It also supports achieving the Optimum Depth (OD) within categories by ensuring we have the right SKUs (sizes) available.

Q: Who uses the output of StyleWiseToSizeWise? **A:** Primarily, Merchandising, Planning, and Buying teams use the output to make informed decisions on what sizes to procure and in what quantities. This helps them optimize inventory and achieve business goals related to ROS and OD.

Q: How does StyleWiseToSizeWise help improve our Rate of Sale (ROS)? **A:** By precisely calculating the Ideal Size Set (ISS) based on actual sales data, the algorithm ensures that the sizes most in demand are adequately stocked. This leads to fewer missed sales opportunities due to stockouts and a more efficient allocation of inventory, thereby boosting ROS.

Q: Does this algorithm consider Optimum Width (OW)? **A:** While StyleWiseToSizeWise focuses on calculating the Ideal Size Set (ISS) for specific styles (which contributes to Optimum Depth (OD)), the strategic decisions regarding Optimum Width (OW) (the ideal number of different categories or styles to offer) are typically made at a higher level, often using the insights derived from this process to understand performance across various styles.

Q: What problem does this system solve for our business? **A:** It solves the challenge of converting broad style-level buying plans into precise, size-level orders, eliminating guesswork and ensuring that our inventory aligns with actual customer demand, leading to improved sales, reduced waste, and optimized stock levels.

Configuration & Settings

Q: Can we adjust how the Ideal Size Set (ISS) is calculated? **A:** The underlying logic for ISS calculation is based on historical sales data and proportional size contribution. While the core logic is stable, specific parameters like Minimum Order Quantity (MOQ) can influence the final size distribution for procurement.

Q: Are Minimum Order Quantities (MOQ) considered in the output? **A:** Yes, the `StyleSkuBuyConversionModule` explicitly applies MOQ adjustments during the conversion of style-level buys into specific size-level quantities, ensuring that calculated buys meet supplier requirements.

Q: How are Attribute Groups (AG) defined or managed within the system? **A:** Attribute Groups (AG) are pre-defined sets of product characteristics (e.g., color, material, style) that are used by the `ISSChannelAgSizeContri` module to analyze sales and determine size contributions. The specific definitions for AGs are typically managed upstream of this process.

Q: Where can I find the current configuration settings being used? **A:** Detailed configuration settings are usually stored in system property files (e.g., `args.properties`). For specific details, please consult the technical documentation or the system administrators.

Q: Who can make changes to these configuration settings? **A:** Changes to core configuration settings usually require administrative access and careful validation to ensure data integrity and accurate calculations. Please contact the IT or System Admin team for any requested adjustments.

Q: Does the system account for promotional periods or seasonality in its calculations? **A:** The ISS calculation relies on historical Product Sales data. If this input data accurately reflects sales trends impacted by promotions or seasonality, then the calculated size contributions will implicitly account for these patterns over the period analyzed.

Data & Inputs

Q: What kind of data does `StyleWiseToSizeWise` need to run? **A:** The primary inputs are Product Sales data and Pivotal Row data, which are used by `ISSChannelAgSizeContri` to determine size contributions. Additionally, style-level purchasing quantities are required for the conversion to size-level buys.

Q: Where does the 'Product Sales' data come from? **A:** Product Sales data is typically sourced from our sales systems or data warehouses, providing historical unit sales information broken down by individual product and size.

Q: What is 'Pivotal Row' data and why is it needed? **A:** Pivotal Row data, in conjunction with Product Sales, helps in determining the proportional contribution of each size within an Attribute Group (AG). It provides contextual information necessary for accurate size distribution calculations.

Q: How often is the input data updated for the algorithm? **A:** The frequency of input data updates depends on the overall data pipeline schedule, but it's typically refreshed regularly to ensure that the Ideal Size Set (ISS) calculations are based on the most current sales trends.

Q: What happens if there's missing or incomplete sales data for a product? **A:** If sales data is missing for a particular product or Attribute Group (AG), the ISS calculation may rely on fallback logic or default distributions, or the product might be excluded from the ISS derivation until sufficient data is available. This can impact the accuracy of the recommended size set.

Q: Can we use forecast data instead of historical sales for ISS calculations? **A:** Currently, the ISSChannelAgSizeContri module primarily uses historical Product Sales data. Integrating forecast data would require a specific enhancement to the input processing logic.

Process & Logic

Q: How does StyleWiseToSizeWise determine the Ideal Size Set (ISS) for a product? **A:** The ISS is determined in two main steps: First, ISSChannelAgSizeContri calculates the proportional contribution of each size for an Attribute Group (AG) using historical sales and pivotal data. Second, StyleSkuBuyConversionModule applies these size contributions to style-level purchasing quantities, adjusting for MOQ and using a Round Robin method to derive the final size-level buy quantities.

Q: What is the significance of "proportional size contribution"? **A:** Proportional size contribution indicates how much each size sells relative to the total sales within an Attribute Group (AG). This percentage distribution is crucial for accurately translating a style-level buy quantity into the correct number of units for each size, aiming to maximize Rate of Sale (ROS) and minimize stockouts.

Q: How does the "Round Robin adjustment" work in converting buys? **A:** The Round Robin adjustment ensures that the total style-level buy quantity is precisely distributed across sizes after initial calculations and MOQ adjustments. It systematically allocates any remaining units to sizes based on their contribution, ensuring the final sum matches the original style buy and achieves the most accurate Ideal Size Set (ISS).

Q: What role do Attribute Groups (AG) play in the calculation? **A:** Attribute Groups (AG) allow the system to analyze size performance across similar products. Instead of calculating ISS for every single SKU, grouping products by relevant attributes ensures that the size contribution is derived from a statistically significant pool of data, leading to more robust ISS recommendations.

Q: What is the overall sequence of operations in this algorithm? **A:** The StyleWiseToSizeWise process is structured as a sequential pipeline: it starts with calculating size contributions per Attribute Group (ISSChannelAgSizeContri). This data then feeds into the module that converts style-level buys into detailed size-level quantities (StyleSkuBuyConversionModule). Finally, utility modules (UtilOutputSnapshotModule, UtilOutputSyncModule) handle output storage and synchronization.

Q: Can the algorithm recommend changes to the Optimum Depth (OD) or Optimum Width (OW)? **A:** While the algorithm optimizes the Ideal Size Set (ISS) for existing styles, which directly impacts Optimum Depth (OD) at the SKU level (by ensuring the right sizes are available), it doesn't directly recommend changes to the ideal number of unique items (OD) or categories (OW) to stock. Its output informs decisions related to achieving optimal depth and width.

Outputs & Results

Q: What is the primary output of the StyleWiseToSizeWise algorithm? **A:** The main output is the recommended size-level buy quantities for each style, representing the calculated Ideal Size Set (ISS) for procurement. This detailed breakdown ensures accurate inventory planning.

Q: How can I access the final results generated by the algorithm? **A:** The UtilOutputSyncModule ensures that the final processed output data is made available to downstream systems or storage locations. This could be a data warehouse, a planning system, or a reporting dashboard.

Q: Is it possible to see a history of previous ISS calculations or outputs? **A:** Yes, the UtilOutputSnapshotModule is designed to capture and preserve intermediate or final results at specific points in the processing pipeline. This allows for historical analysis, auditing, or roll-back capabilities.

Q: What format is the output data in? **A:** The output data typically comes in a structured format suitable for integration with other business systems, often as a table or a file containing product IDs, sizes, and their recommended buy quantities.

Q: How does this output help our buying decisions specifically? **A:** The output provides precise unit recommendations per size, allowing buyers to place orders that perfectly match projected demand and historical performance. This directly optimizes inventory, improves Rate of Sale (ROS), and reduces the risk of overstocking or understocking particular sizes.

Q: Can I get an overview or summary report of the ISS recommendations? **A:** While the raw output provides detailed size-level quantities, downstream reporting tools can consolidate this data into summary reports or dashboards, allowing for an overview of ISS recommendations by style, category, or Attribute Group (AG).

Error Handling & Edge Cases

Q: What happens if a brand new product has no sales history? **A:** For new products without sales history, the system may rely on a fallback strategy, such as applying a pre-defined generic Ideal Size Set (ISS) distribution, or using data from similar Attribute Groups (AG) if available, until sufficient sales data accumulates.

Q: How does the algorithm handle highly seasonal or trend-driven products? **A:** The accuracy for highly seasonal or trend-driven products depends on the quality and recency of the input sales data. If the historical sales data captures these fluctuations, the ISS calculations will reflect them. However, sudden, unforeseen trends might require manual adjustments or more frequent data refreshes.

Q: What if the calculated buy quantity for a specific size is very low, below our MOQ? **A:** The StyleSkuBuyConversionModule includes MOQ adjustments. If a calculated quantity falls below the minimum, the module will adjust it up to the MOQ, potentially reallocating units from other sizes via the Round Robin method to maintain the total style-level buy quantity.

Q: How does the system respond to sudden, unexpected changes in Rate of Sale (ROS)? **A:** The system's response depends on the frequency of data updates and re-calculation. For immediate, sharp changes, manual intervention might be necessary. Over time, as new sales data is incorporated, the Ideal Size Set (ISS) calculations will adapt to the new ROS trends.

Q: Who should I contact if I notice unexpected or incorrect Ideal Size Set (ISS) recommendations? **A:** For any discrepancies or unusual ISS recommendations, please contact the support team or your assigned system administrator, providing specific examples and details.

Q: Is there a mechanism for manually overriding the calculated Ideal Size Set (ISS)? **A:** While the algorithm provides data-driven recommendations, most planning or buying systems allow for manual overrides or adjustments to the recommended ISS to account for qualitative insights or unforeseen circumstances not captured by the data. These overrides typically occur in downstream systems.

Need More Help? Please contact the support team at [support@yourcompany.com] for further assistance.

StyleWiseToSizeWise Documentation

- Version: 1.0
- Last Updated: 2025-05-23

2. Executive Summary

The StyleWiseToSizeWise algorithm is designed to optimize inventory and sales by transforming high-level style-based purchasing plans into detailed size-specific buying quantities. It ensures that businesses stock the Ideal Size Set (ISS) to maximize product availability for customers while minimizing excess stock, directly improving Rate of Sale (ROS) and overall profitability.

3. Architecture Overview

```
graph LR
    subgraph Modules
        0["ISSChannelAgSizeContri"] --> 1["StyleSkuBuyConversionModule"]
        1 --> 2["UtilOutputSnapshotModule"]
        1 --> 3["UtilOutputSyncModule"]
    end
```

- **ISSChannelAgSizeContri**: Calculates the proportional contribution of each size within a specific Attribute Group (AG) for a given channel, serving as a foundational step for Ideal Size Set (ISS) determination.
- **StyleSkuBuyConversionModule**: Converts style-level purchasing quantities into specific size-level quantities, applying Ideal Size Set (ISS) logic, Minimum Order Quantity (MOQ) adjustments, and Round Robin distribution.
- **UtilOutputSnapshotModule**: Captures a snapshot of the output data at a specific point in the processing pipeline for analysis or rollback.
- **UtilOutputSyncModule**: Handles the synchronization or upload of the final processed output data to downstream systems or storage.

4. Prerequisites & Setup

To run the StyleWiseToSizeWise process, ensure the following are in place:

- **Java Version**: Java Development Kit (JDK) 11 or higher.

- **Build Tool:** Maven 3.6.3 or higher.
- **Dependencies:** All necessary dependencies are managed via Maven and will be downloaded during the build process.
- **Running the Orchestrator:** Execute the ModuleGroup orchestrator via your command line:

```
java -jar StyleWiseToSizeWise-1.0.jar --config=args.properties
```

5. Folder & File Structure

```
StyleWiseToSizeWise/
├─ module-group/           # Contains the main orchestrator for
the pipeline.
├─ submodules/             # Houses individual processing
modules.
├─ args.properties         # Configuration parameters for the
entire pipeline.
├─ common-data.json        # Defines the schema for shared data
models.
├─ outputs/                # Stores the final processed results
and intermediate snapshots.
```

6. Global Configuration

No global configuration parameters were provided in the input for `args.properties`.

Setting Name	Default Value	Description	Valid Range/When to Change
--------------	---------------	-------------	----------------------------

7. Shared Data Model

No shared data model schema was provided in the input for `common-data.json`.

Field	Type	Description
-------	------	-------------

8. Submodule Chapters

8.1. ISSChannelAgSizeContri

8.1.1. Module Name & Business Objective

Module Name: ISSChannelAgSizeContri **Business Objective:** This module calculates the final size contribution for each Attribute Group (AG) within a specific channel. It uses Product Sales and Pivotal Row data to determine the proportional contribution of each size. This calculation is a fundamental step in determining the Ideal Size Set (ISS) by providing the necessary size distribution for a given AG, influencing future stock decisions and optimizing ROS.

8.1.2. Required Information

- **Sales Data:**
 - Historical unit sales for each SKU (Style, Size, Color, etc.) over a defined period.
 - Sales performance segmented by channel (e.g., online, in-store).
- **Product Data:**
 - Product master data, including Attribute Group (AG) assignments for each product.
 - Mapping of SKUs to their respective Styles and Attribute Groups.
- **Operational Data:**
 - Defined channels for sales analysis.
 - Look-up tables for Attribute Group definitions and their size ranges.

8.1.3. Business Settings

No specific business settings for this submodule were provided.

8.1.4. Process Flow

graph TD

```
A[Historical Sales Data] --> B{Group by Channel & AG}
B --> C[Calculate Size Contribution per AG]
C --> D[Store Size Contribution Data]
```

8.1.5. Step-by-Step Actions

1. **Gather** *historical sales data* for all SKUs within relevant channels and Attribute Groups.
2. **Aggregate** *sales data* by Channel, Attribute Group (AG), and Size. *This groups similar products and channels to understand size performance consistently.*
3. **Calculate** *the total sales units for each size* within each Channel and AG combination.

4. **Determine** the proportional contribution of each size by dividing its sales units by the total sales units for its respective Channel and AG. *This provides the percentage of demand for each size within a product grouping.*
5. **Store** the calculated size contributions in the database as StyleToSizeBuyFinalContriRow entries for use by downstream modules. *This makes the Ideal Size Set foundation accessible for subsequent calculations.*

8.1.6. Sample Scenario

Consider an “Activewear Leggings” Attribute Group (AG) in the “Online” channel.
* Size XS sold 200 units. * Size S sold 800 units. * Size M sold 1200 units. * Size L sold 600 units. * Size XL sold 200 units. * Total units for Activewear Leggings (Online): 3000 units.

Calculations: * Size XS contribution: $200 / 3000 = 0.067$ (6.7%) * Size S contribution: $800 / 3000 = 0.267$ (26.7%) * Size M contribution: $1200 / 3000 = 0.400$ (40.0%) * Size L contribution: $600 / 3000 = 0.200$ (20.0%) * Size XL contribution: $200 / 3000 = 0.067$ (6.7%)

Key Takeaway: For Activewear Leggings in the Online channel, Size M is overwhelmingly the most in-demand, representing 40% of sales, indicating that future buys should significantly prioritize this size to optimize ROS.

8.1.7. Results

The module outputs a dataset of StyleToSizeBuyFinalContriRow records. Each record details the proportional sales contribution of a specific size (e.g., 0.40 for Medium) for a given Attribute Group within a particular channel. This data is critical for deriving the Ideal Size Set (ISS) for subsequent planning and purchasing.

8.1.8. Unusual Cases

- **Missing or delayed data feed:** Incomplete sales or product data can lead to inaccurate size contributions, potentially skewing the Ideal Size Set (ISS) recommendations and leading to stockouts or overstock.
- **New Attribute Groups/Channels:** If new AGs or channels are introduced without historical data, the module may default to a standard size curve or require manual input/override until sufficient data is accumulated.
- **Extremely low sales volumes:** For AGs with very few sales, calculated contributions might be statistically insignificant; these cases may require aggregation with broader categories or manual intervention to ensure a sensible ISS.
- **Outliers due to promotions/clearance:** Large sales spikes from deep discounts or clearance events might distort typical size contributions. It’s important to consider if such sales should be filtered or weighted differently.

8.1.9. Next Steps

The calculated StyleToSizeBuyFinalContriRow data is consumed by the StyleSkuBuyConversionModule. This downstream module uses these size contributions to convert style-level buy quantities into specific size-level quantities for procurement.

8.1.10. Common Questions

1. **How accurate are the size contributions if sales data is inconsistent or incomplete?** *Accuracy directly depends on data quality; inconsistent or incomplete data will lead to skewed ISS recommendations and inefficient inventory.*
2. **Can we apply different timeframes for historical sales analysis?** *Yes, the module can be configured to analyze sales over specific periods (e.g., 3 months, 6 months, 1 year) to capture seasonality or recent trends effectively.*
3. **What if a new size is introduced for an existing Attribute Group?** *New sizes won't have historical data; their contribution might need to be estimated based on similar sizes, or gradually built over time as sales occur.*
4. **How does this account for variations across different regions or stores?** *If 'channel' is defined to include regional or store-level distinctions, the module can calculate unique size contributions for each, providing more granular ISS.*
5. **Is this only for current products, or can it predict for new styles that don't have sales history?** *This module is based on historical data for existing AGs; for truly new styles, it relies on the size contributions of comparable Attribute Groups or requires initial manual estimations.*

8.1.11. Quick Reference

- **Purpose:** Calculate proportional size contribution per Attribute Group per channel.
 - **Input:** Historical unit sales data, product Attribute Group mapping.
 - **Output:** Size contribution percentages for Ideal Size Set (ISS) determination.
 - **Key Use:** Provides the foundation for size-level buying decisions and optimizing OD.
-

8.2. StyleSkuBuyConversionModule

8.2.1. Module Name & Business Objective

Module Name: StyleSkuBuyConversionModule **Business Objective:** This module converts style-level purchasing quantities (buy) into specific size-level quantities, effectively deriving the Ideal Size Set (ISS) for procurement. It leverages the size contribution data (calculated by ISSChannelAgSizeContri), applies Minimum Order Quantity (MOQ) adjustments, and performs a Round Robin adjustment to ensure the total buy quantity for a style is accurately distributed across sizes, aiming to maximize sales and minimize stockouts as part of the ISS.

8.2.2. Required Information

- **Sales Data:**
 - StyleToSizeBuyFinalContriRow: The calculated size contribution data from ISSChannelAgSizeContri, specifying the percentage of units each size contributes to an Attribute Group's sales (i.e., the derived ISS).
- **Product Data:**
 - Style-level buy quantities (e.g., the total number of units to buy for a specific style).

- Minimum Order Quantity (MOQ) per SKU or style (e.g., vendor minimums).
- SKU definitions (mapping style, size, color to a unique product identifier).
- **Operational Data:**
 - Rounding rules for unit allocation (e.g., always round up, nearest whole number).

8.2.3. Business Settings

No specific business settings for this submodule were provided.

8.2.4. Process Flow

graph TD

```

A[Style-Level Buy Quantity] --> B[Apply Size Contribution]
B --> C[Adjust for MOQ]
C --> D[Round Robin Adjustment]
D --> E[Size-Level Buy Quantity (ISS)]

```

8.2.5. Step-by-Step Actions

1. **Receive** the total buy quantity for each style from upstream planning systems.
2. **Retrieve** the size contribution data (from ISSChannelAgSizeContri) for the relevant Attribute Group associated with each style. *This applies the Ideal Size Set (ISS) distribution for that product category.*
3. **Calculate** the initial size-level quantity for each SKU by multiplying the style's total buy quantity by its respective size contribution percentage.
4. **Apply Minimum Order Quantity (MOQ) rules.** If the calculated quantity for a size is below its MOQ, **adjust it up to the MOQ.** If adjusting a size to MOQ causes the total buy to exceed the original style buy, a proportional reduction across other sizes might be necessary, or the total buy quantity for the style might need to be increased. *This ensures practical ordering quantities that meet vendor requirements.*
5. **Perform a Round Robin adjustment.** Since initial allocations might not sum exactly to the original style-level buy quantity due to rounding and MOQ adjustments, **distribute any remaining or excess units** across sizes based on their contribution, starting with the highest remainder. *This ensures the final size-level quantities perfectly sum to the original style-level buy, maintaining inventory accuracy and optimizing OD.*
6. **Generate** the final size-level buy quantities for each SKU. *This is the Ideal Size Set (ISS) for procurement, balancing historical demand with operational constraints.*

8.2.6. Sample Scenario

A new dress style, “Summer Breeze Dress,” has a total buy quantity of 1200 units for the upcoming season. Based on `ISSChannelAgSizeContri` for its “Women’s Dresses” Attribute Group in the “Direct-to-Consumer” channel, the size contributions are: XS (10%), S (25%), M (35%), L (20%), XL (10%). The Minimum Order Quantity (MOQ) for any single size is 50 units.

1. Initial Calculation:

- XS: $1200 * 0.10 = 120$ units
- S: $1200 * 0.25 = 300$ units
- M: $1200 * 0.35 = 420$ units
- L: $1200 * 0.20 = 240$ units
- XL: $1200 * 0.10 = 120$ units
- Total = 1200 units (matches initial buy)

2. **MOQ Adjustment:** All calculated quantities (120, 300, 420, 240, 120) are above the MOQ of 50 units. No adjustment needed here. (If, for example, XS was calculated as 40 units, it would be adjusted up to 50, and the total buy quantity would need to be adjusted, or units would be reduced from other sizes if the overall total must remain 1200).

3. **Round Robin Adjustment:** In this simplified scenario, with exact whole numbers, no further adjustment is needed. However, if initial calculations resulted in fractional units (e.g., 120.4, 299.8, 420.1, 239.7, 120.0), and we rounded down, the total might be 1199. The Round Robin step would then add the missing unit to the size with the largest decimal remainder (S in this example) to reach the original 1200 total.

Key Takeaway: The module precisely translates a total style buy into a tailored size breakdown, incorporating Ideal Size Set (ISS) logic and practical MOQs. This ensures optimal stock distribution for improved ROS and reduced stockouts, contributing to efficient OD.

8.2.7. Results

The module outputs detailed size-level buy quantities for each SKU. These typically include `ChannelStyleSizeBuyRow` and `ExportStyleWiseToSizeWiseRow` records, which represent the Ideal Size Set (ISS) recommendations for procurement. This output directly informs purchasing orders and inventory decisions, ensuring that the Optimal Width (OW) and Optimum Depth (OD) are met across product offerings.

8.2.8. Unusual Cases

- **Missing size contribution data:** If `ISSChannelAgSizeContri` data is absent for a specific Attribute Group, the module may default to a generic size curve or require manual intervention, potentially leading to a suboptimal Ideal Size Set (ISS).
- **Drastic changes in MOQ:** Sudden changes to MOQs can significantly alter the size distribution. If MOQs are high relative to the total buy, this might lead to overstock in less popular sizes or force a larger total buy than initially planned.

- **Insufficient total buy quantity:** If the total style-level buy quantity is too low to meet MOQs across all desired sizes based on the ISS, the module might prioritize core sizes or flag the need for a larger overall buy quantity.
- **Rapidly shifting demand:** For fashion-sensitive or trend-driven items, static size contributions derived from historical data might not capture sudden shifts in demand accurately, leading to an inefficient ISS.
- **Size run complexity:** Styles with irregular size runs (e.g., only even sizes, or very few sizes) require careful handling to ensure contributions are applied correctly.

8.2.9. Next Steps

The final size-level buy quantities generated by this module are then passed to UtilOutputSnapshotModule for historical record-keeping and UtilOutputSyncModule for dissemination to downstream purchasing and inventory management systems, making the Ideal Size Set (ISS) actionable.

8.2.10. Common Questions

1. **Can I override the recommended size quantities for a specific style?** *While technically possible, such overrides should be minimized as they deviate from the data-driven Ideal Size Set (ISS) and may impact ROS and inventory efficiency.*
2. **How frequently should size contributions be recalculated to ensure accuracy?** *Recalculations should occur regularly, typically weekly or monthly, to capture changing trends and maintain an accurate ISS reflective of current demand.*
3. **What if a style has no historical sales data or no assigned Attribute Group?** *The module will fall back on the size contributions of a broader default Attribute Group or a generic curve, which reduces the accuracy of the ISS and optimal OD.*
4. **How does this interact with existing global inventory levels or outstanding purchase orders?** *This module focuses on calculating new* buy quantities; subsequent inventory management systems will integrate these recommendations with current stock levels and inbound orders.**
5. **Can this process handle different size runs or size scales (e.g., numeric vs. alpha sizes)?** *Yes, as long as the size contribution data from ISSChannelAgSizeContri correctly maps to the defined sizes for a given Attribute Group, the module can process various size scales.*

8.2.11. Quick Reference

- **Purpose:** Convert total style buy quantities into specific, optimized size-level quantities.
 - **Input:** Style buy quantities, Ideal Size Set (ISS) contribution data, Minimum Order Quantities (MOQ).
 - **Output:** Specific, actionable size-level buy quantities for each SKU.
 - **Key Use:** Inform procurement for optimal OD, OW, and ROS, minimizing stockouts.
-

8.3. UtilOutputSnapshotModule

8.3.1. Module Name & Business Objective

Module Name: UtilOutputSnapshotModule **Business Objective:** This module is responsible for capturing a snapshot of the output data at a specific point in the processing pipeline. It acts as a utility to preserve intermediate or final results for analysis or rollback purposes.

8.3.2. Required Information

- **Sales Data:**
 - Processed output data from StyleSkuBuyConversionModule (e.g., ChannelStyleSizeBuyRow, ExportStyleWiseToSizeWiseRow), containing the calculated Ideal Size Set (ISS) for buying.

8.3.3. Business Settings

No specific business settings for this submodule were provided.

8.3.4. Process Flow

graph TD

```
A[Processed Output Data] --> B[Generate Timestamped Snapshot]
B --> C[Store Snapshot]
```

8.3.5. Step-by-Step Actions

1. **Receive** the final processed data (e.g., size-level buy quantities) from the preceding StyleSkuBuyConversionModule.
2. **Create** a timestamped copy of this data. *This ensures that each snapshot is uniquely identifiable and allows for historical tracking and version control.*
3. **Store** this snapshot in a designated storage location (e.g., the outputs/ folder, a data lake, or an archival database table). *This provides a persistent, immutable record of the processing outcome for auditing, debugging, and future comparative analysis.*

8.3.6. Sample Scenario

After StyleSkuBuyConversionModule successfully calculates the Ideal Size Set (ISS) for all styles in the “Fall 2025” collection, UtilOutputSnapshotModule takes this comprehensive dataset and saves it. For example, it might save a JSON file named `outputs/2025-05-23_Fall2025_BuyPlan_ISS.json` or a similar time-stamped and descriptive file, containing all the size-level buy recommendations.

Key Takeaway: This module ensures data traceability and provides a historical record of calculated Ideal Size Sets (ISS), allowing for robust auditing, performance comparisons, and potential rollbacks.

8.3.7. Results

A persistent, timestamped copy of the processed output data, typically stored in a structured file format (e.g., JSON, CSV, Parquet) or an archival database table. This snapshot reflects the exact state of the Ideal Size Set (ISS) recommendations at the time of processing.

8.3.8. Unusual Cases

- **Missing or delayed data feed:** If the upstream module fails to produce data, the snapshot module will simply not have data to snapshot, indicating an upstream issue that needs resolution.
- **Storage issues:** If the designated storage location is unavailable, lacks sufficient space, or has incorrect permissions, the snapshot process will fail. This results in a gap in the historical record and requires immediate attention.
- **Large data volumes:** Extremely large datasets might require optimized storage solutions (e.g., compression, distributed storage) or incremental snapshotting to avoid performance bottlenecks and ensure timely completion.
- **Corrupted data:** If the data received from upstream is corrupted, the snapshot will preserve this corrupted state, highlighting the importance of data validation earlier in the pipeline.

8.3.9. Next Steps

The snapshot primarily serves as an archive. The original processed data continues its flow to `UtilOutputSyncModule` for active dissemination to live operational systems.

8.3.10. Common Questions

1. **Why do we need snapshots if the data goes to other systems?** *Snapshots provide an independent, immutable historical record, enabling rollback to previous states, auditing specific runs, and comparing different algorithm outputs or business scenarios over time.*
2. **How long are snapshots retained, and what are the storage implications?** *Retention policies vary based on regulatory compliance, auditing requirements, and business needs. Long retention periods for large datasets can have significant storage cost implications.*
3. **Can I access previous snapshots for analysis or debugging?** *Yes, snapshots are stored to be easily retrievable for historical analysis, debugging issues, and understanding how Ideal Size Set (ISS) recommendations evolved.*
4. **Does taking a snapshot affect the overall speed of the pipeline?** *The impact is typically minimal, as it's primarily a file write or database insert operation. However, extremely large datasets could see a slight increase in overall processing time.*
5. **What happens if a snapshot fails? Does it stop the entire process?** *Generally, a snapshot failure is non-critical for the main pipeline flow, meaning the data will still proceed to the sync module. However, it will trigger alerts as the historical record for that run will be incomplete.*

8.3.11. Quick Reference

- **Purpose:** Archive processed output data for historical record and analysis.
 - **Input:** Final processed Ideal Size Set (ISS) data.
 - **Output:** Timestamped data snapshot.
 - **Key Use:** Auditing, historical trend analysis, debugging, and rollback capability.
-

8.4. UtilOutputSyncModule

8.4.1. Module Name & Business Objective

Module Name: UtilOutputSyncModule **Business Objective:** This module handles the synchronization or upload of the final processed output data. It ensures that the results of the StyleWiseToSizeWise calculation (e.g., the Ideal Size Set for buying) are made available to downstream systems or storage.

8.4.2. Required Information

- **Sales Data:**
 - Final processed output data from StyleSkuBuyConversionModule (e.g., ChannelStyleSizeBuyRow, ExportStyleWiseToSizeWiseRow), representing the Ideal Size Set (ISS) for procurement.

8.4.3. Business Settings

No specific business settings for this submodule were provided.

8.3.4. Process Flow

graph TD
A[Processed Output Data] --> B[Format Data for Destination]
B --> C[Transmit Data to Downstream System]
C --> D[Confirmation/Logging]

8.4.5. Step-by-Step Actions

1. **Receive** *the final processed data* (Ideal Size Set for buying, including size-level quantities) from the StyleSkuBuyConversionModule.
2. **Format** *the data* according to the specific requirements of the target downstream system (e.g., converting to CSV, JSON, Parquet, or preparing specific database insert/update statements). *This ensures compatibility and seamless integration with operational systems.*
3. **Transmit** *the formatted data* to the designated downstream system or storage location (e.g., an SFTP server, cloud storage bucket, a data warehouse, or directly into an ERP system's API). *This makes the Ideal Size Set (ISS) actionable for procurement, inventory management, or reporting, directly impacting OD and ROS.*

4. **Log** the transmission status (success/failure), timestamp, and any relevant metadata. *This provides a comprehensive audit trail for data dissemination and aids in troubleshooting.*

8.4.6. Sample Scenario

After the `StyleSkuBuyConversionModule` has produced the precise Ideal Size Set (ISS) buy plan for all categories for next season, `UtilOutputSyncModule` becomes active. It formats this detailed data into a series of structured files (e.g., daily CSVs) and securely uploads them to a cloud-based file storage system. From there, the company's Enterprise Resource Planning (ERP) system automatically picks up and ingests these files, updating its purchasing modules with the recommended size-level buy quantities.

Key Takeaway: This module is the final critical step that translates algorithmic recommendations into actionable data for core business operations, ensuring that the Ideal Size Set (ISS) drives actual purchasing decisions and inventory allocation.

8.4.7. Results

The final processed output data (the calculated Ideal Size Set for buying) is successfully delivered and made available in the designated downstream systems. It is ready for use in generating purchase orders, updating inventory forecasts, and driving various business reports.

8.4.8. Unusual Cases

- **Missing or delayed data feed:** If the upstream module fails to produce data, the sync module will have nothing to transmit, which will be logged as an upstream data dependency issue.
- **Network connectivity issues:** Problems with network connection to the target system (e.g., firewall issues, internet outages) can lead to transmission failures, requiring robust retry mechanisms and alerts.
- **Target system unavailability:** If the downstream system is offline, experiencing downtime, or unresponsive (e.g., API rate limits hit), data synchronization will fail, necessitating alerts and manual intervention.
- **Data format incompatibility:** Incorrect data formatting or schema mismatches with the target system can lead to rejection of the data, requiring debugging, re-formatting, and re-transmission.
- **Authentication/Authorization failures:** Incorrect credentials or insufficient permissions for the target system can prevent successful data upload.

8.4.9. Next Steps

The data is now available in downstream systems, which can trigger automated procurement processes, update inventory forecasts, enable financial reporting, or generate specific reports for buying and planning teams based on the Ideal Size Set (ISS) recommendations.

8.4.10. Common Questions

1. **What types of systems does this module integrate with?** *It can integrate with a wide array of systems, including ERPs (e.g., SAP, Oracle), inventory management systems, data warehouses, cloud storage (e.g., S3, Azure Blob), and custom reporting tools.*
2. **Is the data upload real-time or batch-based?** *This depends on configuration and business needs. Typically, it operates as a scheduled batch process, but near real-time integration is possible for critical updates or API-driven connections.*
3. **How are transmission failures handled, and who is notified?** *The module includes robust retry logic and error logging. Critical failures (e.g., after multiple retries) trigger automated alerts to relevant IT or business operations teams for investigation and manual intervention.*
4. **Can we send the data to multiple different destinations simultaneously?** *Yes, the module can be configured to transmit the same processed data to multiple different downstream systems or storage locations concurrently, meeting various business needs.*
5. **Is the data secure during transmission?** *Yes, standard secure protocols (e.g., SFTP, HTTPS, encrypted database connections) are used to ensure that data is encrypted in transit, adhering to industry security best practices.*

8.4.11. Quick Reference

- **Purpose:** Disseminate final processed data (Ideal Size Set).
 - **Input:** Final processed Ideal Size Set (ISS) data for procurement.
 - **Output:** Data successfully delivered to designated downstream systems.
 - **Key Use:** Make ISS actionable for procurement, inventory management, and business intelligence.
-

9. Appendix

9.1. Full Parameter Reference

No specific parameters were provided beyond the general `args.properties` file mentioned in the inputs.

9.2. Version History

- **Version 1.0 (2025-05-23):** Initial release of the `StyleWiseToSizeWise` pipeline, enabling the conversion of style-level buys to optimized size-level procurement quantities, and establishing the core Ideal Size Set (ISS) calculation and dissemination process.

9.3. Contact & Support Info

For questions, support, or to report issues, please contact the product development team at: * **Email:** support@yourcompany.com * **Internal Chat:** #stylewise-to-size-wise-support * **Documentation Repository:** [Link to internal Git/Confluence page]

Chapter 1: Gap Analysis Orchestration

Last Updated: 2025-05-23

1. Business Objective

This module acts as the central coordinator for the entire Gap Analysis process, ensuring all components run in the correct order to systematically identify and address merchandise gaps.

Why This Matters: This coordination ensures that merchandising decisions are based on a complete and accurate understanding of stock opportunities and customer demand. It reduces errors and speeds up the process, leading to quicker adjustments to inventory and display strategies that improve sales and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** The number of items sold over a specific period. We need this to understand product popularity and demand patterns for each item, category, and size.
- **Rate of Sale (ROS):** The speed at which products are sold over a period (e.g., units/week). This helps predict future demand and identify fast-moving items.
- **Sales History by Size:** Past sales performance broken down by individual product sizes. Essential for identifying which sizes are most popular and ensuring the Ideal Size Set (ISS) is optimized.
- **Sales by Category/Style:** Total sales volume for different product categories or styles. This helps identify top-performing areas and potential growth opportunities.

Product Data

- **Product Attributes (AG):** Features like color, material, and brand. Used to group similar products for analysis and ensure balanced assortments.
- **Category/Style Definitions:** How products are grouped into distinct categories or styles (e.g., 'Women's Jeans - Skinny Fit'). This helps structure the analysis for Optimum Width (OW) decisions.
- **Product Master Data:** Basic information about each unique item (SKU), including its description, cost, and retail price. Needed for all financial calculations and product identification.

- **Target Optimum Depth (OD) by Category:** The ideal number of unique items (SKUs) to stock within a specific category. This benchmark helps identify missing product varieties.

Operational Data

- **Current Inventory Levels:** The exact number of units currently in stock for each SKU and size across all locations. Crucial for identifying stockouts or overstocks.
- **On-Order Quantities & Expected Delivery Dates:** Information about merchandise currently on order and when it’s expected to arrive. This impacts future stock availability and potential gaps.
- **Store/Warehouse Locations:** Information about where inventory is held. Helps analyze gaps specific to certain sales channels or regions.
- **Vendor Lead Times:** The time it takes for new inventory to arrive from suppliers after an order is placed. Important for planning replenishment and avoiding prolonged stockouts.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The duration of historical sales data considered for analysis. For example, 90 days means the system looks at the last three months of sales.	Adjust this to longer periods (e.g., 180 days) for seasonal items or new product launches, or shorter (e.g., 30 days) for fast-fashion or highly volatile trends.
Minimum Sales Threshold	5 units	The minimum number of units an item must have sold during the Analysis Period to be included in certain gap calculations. Items below this threshold are typically excluded from core analysis.	Increase this for high-volume categories to focus on top performers, or decrease it for niche categories or new products that need more attention despite lower initial sales.
Target Optimum Depth (OD) Deviation %	10%	The allowable percentage deviation from the target OD before an “Assortment Depth” gap is	Adjust based on category importance; a lower percentage (e.g., 5%) for core categories, higher (e.g., 15%) for less

Setting Name	Default Value	Description	When to Change
Ideal Size Set (ISS) Match Tolerance	80%	flagged. For example, if target OD is 100 SKUs, a 10% deviation means 90-110 SKUs is acceptable.	critical or experimental categories.
		The percentage of the Ideal Size Set (ISS) that must be in stock to consider the size set “complete.” Below this, a size gap is flagged.	Set higher (e.g., 90%) for core items where all sizes are critical, lower (e.g., 70%) for less critical items or those with less predictable size demand.

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

A[Initial Data Snapshot] --> B[Run NOOS Availability Analysis]
B --> C[Run Size Set Comparison]
C --> D[Run Broken/Non-Broken Analysis]
D --> E[Synchronize Final Outputs]

```

1. Start with → **Initial Data Snapshot**: All relevant sales, inventory, and product data are collected and locked for consistent analysis.
2. Then → **Run NOOS Availability Analysis**: Identify “Never Out Of Stock” opportunities and assess overall product availability.
3. Next → **Run Size Set Comparison**: Evaluate current size distributions against the Ideal Size Sets (ISS) to spot size gaps.
4. Next → **Run Broken/Non-Broken Analysis**: Determine which items are “broken” (stockouts impacting sales) versus “non-broken” (sufficient stock).
5. Finally → **Synchronize Final Outputs**: All findings from the various analyses are consolidated and made available for review and action.

5. Step-by-Step Actions

1. **Capture** an initial data snapshot
 - What we do: *We create a consistent and frozen copy of all sales, inventory, and product data needed for the analysis.*
 - Why we do it: *This ensures that all subsequent analysis modules work with the exact same data, preventing inconsistencies if data changes during the process.*

2. **Execute** the “Never Out Of Stock” (NOOS) Gap Analysis
 - What we do: *The system triggers the NOOS module to identify items that should ideally always be in stock but are currently missing or low.*
 - Why we do it: *This highlights critical sales opportunities by flagging staple products that are unavailable, preventing lost sales.*
3. **Execute** the Size Set Comparison
 - What we do: *The system triggers the Size Set module to compare the actual sizes in stock against the Ideal Size Set (ISS) for each product.*
 - Why we do it: *This helps ensure we have the right mix of sizes to meet customer demand, minimizing stockouts on popular sizes and preventing excess stock in less popular ones.*
4. **Execute** the Broken/Non-Broken Inventory Analysis
 - What we do: *The system triggers the Broken/Non-Broken module to categorize inventory as either “broken” (experiencing significant stockouts) or “non-broken.”*
 - Why we do it: *This provides a clear view of where our inventory is failing to meet demand and helps prioritize replenishment efforts.*
5. **Consolidate** and synchronize outputs
 - What we do: *All the results and insights from the individual gap analysis modules are brought together and made available for reporting and further action.*
 - Why we do it: *This ensures a unified view of all identified merchandising gaps, allowing merchants to see the complete picture and make informed decisions.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A merchant wants to analyze stock gaps for their “Women’s Denim” category.
- Current Sales Data: “Skinny Jeans” (product A) sold 320 units last month. “Bootcut Jeans” (product B) sold 80 units. “High-Rise Slim Jeans” (product C) sold 150 units.
- Current Inventory:
 - Product A: Out of sizes 28, 30, 32. Total stock 5 units (only unpopular sizes left).
 - Product B: Full size run available, 60 units in stock.
 - Product C: Out of size 26. Total stock 25 units.
- Business Setting: Minimum Sales Threshold is 10 units/month. Analysis Period is 30 days.

What Happens

1. First, the system **captures a snapshot** of all sales, inventory, and product data for the “Women’s Denim” category for the last 30 days. This ensures that even if new stock arrives during the analysis, the calculations use the exact data from the start.
2. Then, the system **executes the NOOS Availability Analysis**. Product A (Skinny Jeans) is flagged as a critical NOOS gap because it sold 320 units (well above

the 10-unit threshold) but is effectively out of popular sizes, leading to potential lost sales. Product B is not flagged. Product C is also flagged, though less critically than A.

3. Next, the system **executes the Size Set Comparison**. For Product A, it identifies a severe gap against the Ideal Size Set (ISS) because core sizes (28, 30, 32) are missing. For Product C, a minor size gap is identified due to the missing size 26. Product B's size set is confirmed as healthy.
4. Finally, the system **executes the Broken/Non-Broken Analysis**. Product A is categorized as "Broken" due to significant sales history combined with near-zero viable stock. Product C is also "Broken" but less severely. Product B is categorized as "Non-Broken."
5. All these findings are then **consolidated and synchronized**, producing a comprehensive report.

Final Outcome

The merchant receives a unified report highlighting: - **Critical Stock Gaps:** Product A (Skinny Jeans) is a top priority for replenishment, especially for sizes 28, 30, 32, as it's a high-demand item with broken stock. - **Minor Stock Gaps:** Product C (High-Rise Slim Jeans) needs size 26 replenished. - **Healthy Stock:** Product B (Bootcut Jeans) is adequately stocked across its size range.

Key Takeaway: This orchestration streamlines gap identification, showing merchants exactly where stock issues are impacting sales and customer experience, allowing for immediate and targeted action.

7. Results

This step produces:

Main Outputs

- **Unified Gap Analysis Report:** A comprehensive document detailing all identified gaps across product availability, size sets, and broken inventory. It provides a holistic view of merchandising opportunities.
- **Prioritized Action List:** A list of identified gaps, often ranked by business impact (e.g., potential lost sales), guiding merchants on which issues to address first.
- **Detailed Gap Metrics:** Specific numbers and percentages related to each gap type, such as the value of lost sales due to NOOS, or the percentage deviation from the ISS.
- **Category-Level Gap Summaries:** Aggregated insights into gap issues for entire categories or styles, helping merchants understand broader trends and strategic areas for improvement.

Business Reports

- **Daily Merchandise Gap Report:** Used by merchandise planners and store managers to quickly identify immediate stock issues and trigger replenishment or re-allocation actions, helping to minimize lost sales.

- **Weekly Assortment Health Dashboard:** Used by merchandising directors to review overall assortment performance, identify recurring gap patterns, and inform strategic decisions on buying and planning.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will pause or use the last available snapshot, leading to potentially outdated analysis. An alert will be triggered.	Verify data source connections and ensure all required data feeds (sales, inventory, product) are active and providing up-to-date information. Rerun the analysis once data is restored.
No items meet sales thresholds	The analysis might flag very few or no significant gaps, or a message indicating “No high-priority gaps found.”	Review the “Minimum Sales Threshold” and “Analysis Period” settings. If sales are genuinely low across the board, this might indicate a different business problem.
All items flagged as “Broken”	The report will show an overwhelming number of products with severe stockouts, making prioritization difficult.	This indicates widespread inventory issues. Prioritize the highest-revenue items first, review overall inventory strategy, and assess supply chain bottlenecks.
Analysis runs longer than expected	The process may take significantly longer than typical, delaying the availability of insights.	Check the volume of data being processed and if any sub-modules are stuck. Contact IT support if the issue persists to identify performance bottlenecks.

9. Next Steps

Before This Step

- **Data Ingestion & Cleaning:** All raw sales, inventory, and product data must be collected, validated, and formatted correctly. We need accurate, clean data to ensure reliable analysis.
- **Master Data Management:** Product catalogs, category definitions, and Ideal Size Set (ISS) configurations need to be up-to-date and consistent. This provides the foundational context for the analysis.

After This Step

- **Merchandise Planning & Action:** The identified gaps and prioritized action list are used by merchants and planners to make immediate decisions on replenishment orders, inter-store transfers, or adjusting future buying plans. They need clear, actionable insights from us.

- **Assortment Optimization:** Insights from the gap analysis (e.g., consistent NOOS items, underperforming OD/OW) inform strategic assortment adjustments for future seasons. They need trends and root causes from us.

10. Common Questions

Q: How quickly can I see the results of the gap analysis? A: The system is designed to provide results efficiently, typically within a few hours or overnight, allowing for timely business decisions based on the latest data snapshot.

Q: What if I don't agree with a gap the system identifies? A: The system provides data-driven insights, but your expertise as a merchant is crucial. You can investigate the specific data points, adjust business settings for future runs, or override the recommendation if unique business context applies.

Q: Can this analysis help me reduce excess inventory, not just identify stockouts? A: While primarily focused on sales-driving gaps, by highlighting underperforming items not meeting sales thresholds or consistently having full stock despite low sales, the analysis can indirectly point to potential overstock situations that tie up capital.

Q: How does this help improve customer experience? A: By systematically identifying and addressing stockouts on popular items and ensuring the right sizes are available, customers are more likely to find what they want, leading to higher satisfaction and fewer lost sales.

Q: If a product is consistently flagged as "Broken," what should I do? A: A consistently "Broken" product indicates persistent demand-supply imbalance. You should investigate supply chain issues, re-evaluate supplier lead times, consider increasing safety stock, or even adjust your target inventory levels for that item.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To coordinate and run all sub-modules of the Gap Analysis in the correct sequence, from initial data capture to final output consolidation, ensuring comprehensive and actionable insights.
- **Main Input:** A consistent snapshot of current sales, inventory, and product master data.
- **Main Output:** A unified Gap Analysis Report, including prioritized action lists for merchandising teams.
- **Warning Signs:** Delayed report generation, inconsistent findings between different gap types, or a lack of new gaps identified when expected.
- **Key Settings:** Analysis Period, Minimum Sales Threshold, Target OD Deviation %, Ideal Size Set (ISS) Match Tolerance.

Need Help? Contact your Merchandise Planning Support team for questions about this process.

Chapter 3: Never-Out-Of-Stock (NOOS) Gap Analysis

Last Updated: 2025-05-23

1. Business Objective

This module assesses the availability and performance of “Never-Out-Of-Stock” (NOOS) styles, ensuring core products are always available to meet customer demand.

Why This Matters: This analysis prevents lost sales, builds customer loyalty by consistently meeting demand for popular items, and provides critical insights into how effectively brands are maintaining availability for their key products. It ensures that when a customer wants your most popular item, it’s always there.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS):** The speed at which products are sold (e.g., units/week). This helps confirm which products are truly “best-sellers” or “core” and therefore should be designated as NOOS.
- **Sales History by Style:** Past sales performance for specific product styles. This is essential for verifying which items are consistent demand drivers and thus strong candidates for NOOS status.

Product Data

- **NOOS Product List:** A predefined, specific list of products or styles designated as “Never-Out-Of-Stock.” This module focuses its analysis exclusively on these crucial items.
- **Product Attributes (AG):** Features like brand, product theme (e.g., ‘core bestseller’), and style classifications. These are used to group products, assess consistent demand, and determine brand performance grades.
- **Ideal Size Set (ISS) Configuration:** The target combination of sizes for each product. While not directly computed here, this module assesses if current inventory levels meet a benchmark percentage of the ISS.

Operational Data

- **Current Inventory Levels:** The exact number of units currently in stock for each NOOS product, broken down by size and location. This is crucial for determining daily availability.
- **Store Locations:** Information about all active retail locations. This allows the system to analyze NOOS availability across each specific store.
- **Analysis Period Dates:** The start and end dates defining the historical period over which NOOS availability will be assessed.

3. Business Settings

Setting Name	Default Value	Description	When to Change
NOOS Analysis Period	90 days	The duration over which NOOS product availability is assessed. For example, 90 days means the system checks availability for the last three months.	Adjust this to longer periods (e.g., 180 days) for seasonal NOOS items or shorter (e.g., 30 days) for fast-fashion core styles that require more frequent monitoring.
Ideal Size Set (ISS) Availability Benchmark	80%	The minimum percentage of a product's Ideal Size Set (ISS) that must be in stock for it to be considered "available" on a given day.	Set higher (e.g., 95%) for highly critical NOOS items where any missing size is a major issue, or slightly lower (e.g., 70%) for items where some minor size gaps might still be acceptable.
Minimum Days In-Stock for NOOS Styles	80 days	The minimum number of days a NOOS style must meet the ISS Availability Benchmark during the NOOS Analysis Period to be considered "consistently available."	Increase this for critical core items where any stockout is unacceptable, or decrease for newly designated NOOS styles that are still building consistent inventory.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[NOOS Product List & Store Data] --> B{Daily Availability Check}

B --> C{Calculate Consistent Availability}

C --> D{Determine Brand Performance}

D --> E[NOOS Performance Reports]

1. Start with → **NOOS Product and Store Data**: All products designated as “Never-Out-Of-Stock” and information about active stores are collected and initialized.
2. Then → **Check Daily Availability**: The system assesses each NOOS product’s in-stock status daily at every store, comparing its current inventory against the Ideal Size Set (ISS) Availability Benchmark.
3. Next → **Calculate Consistent Availability**: The total number of days each NOOS product was consistently available in each store is determined, and then aggregated for stores meeting the Minimum Days In-Stock threshold.
4. Next → **Determine Brand Performance**: Based on the availability data and product classifications, a performance grade is assigned to relevant brands, reflecting their ability to maintain NOOS availability.
5. Finally → **Generate NOOS Performance Reports**: All computed availability metrics and brand performance insights are consolidated into actionable reports for review and action.

5. Step-by-Step Actions

1. **Initialize** the analysis environment
 - What we do: *We gather the specific list of products designated as “Never-Out-Of-Stock” (NOOS) and prepare all necessary historical inventory and sales data for analysis.*
 - Why we do it: *This ensures we are focusing only on the core, must-have items that are critical for consistent availability and have the most accurate data for assessment.*
2. **Assess** daily NOOS availability at each store
 - What we do: *For every NOOS product, the system checks its in-stock status at each store, every day, against a target percentage of its Ideal Size Set (ISS) defined by the Ideal Size Set (ISS) Availability Benchmark.*
 - Why we do it: *This identifies exactly when and where a core product was not fully available, flagging potential lost sales opportunities due to incomplete size runs.*
3. **Calculate** consistent NOOS availability duration
 - What we do: *The system counts how many days each NOOS product at each store met the Ideal Size Set (ISS) Availability Benchmark within the NOOS Analysis Period.*
 - Why we do it: *This highlights which stores or products consistently maintain their NOOS status versus those that frequently experience stockouts, indicating areas needing intervention.*

4. **Grade** brand performance

- What we do: *Based on the availability and sales data of NOOS products, the system assigns a performance grade to each brand. Brands with more consistently available NOOS styles receive higher grades.*
- Why we do it: *This provides high-level insight into how effectively brands are managing their core product availability, informing strategic buying and vendor management decisions.*

5. **Store** and consolidate results

- What we do: *All computed availability data and brand performance grades are saved and prepared for reporting and integration into broader gap analysis outputs.*
- Why we do it: *This makes the insights available for merchants and planners to review, allowing for swift and informed decision-making based on a unified view of merchandising gaps.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retail chain has designated “Classic White T-Shirt” (Style ID: 101, Brand: ComfortWear) as a NOOS item across all its 50 stores.
- **NOOS Analysis Period:** 30 days.
- **Ideal Size Set (ISS) Availability Benchmark:** 90% (meaning 90% of a product's target sizes must be in stock for it to be considered “available”).
- **Minimum Days In-Stock for NOOS Styles:** 25 days (out of the 30-day period).
- For Store A: On 20 days, the “Classic White T-Shirt” met or exceeded the 90% ISS benchmark. On the other 10 days, it was below this benchmark.
- For Store B: On 28 days, the “Classic White T-Shirt” met or exceeded the 90% ISS benchmark. On the other 2 days, it was below this benchmark.

What Happens

1. First, the system **initializes** by confirming that “Classic White T-Shirt” (Style ID: 101) from “ComfortWear” is on the NOOS Product List and retrieves its daily inventory for Store A and Store B for the last 30 days.
2. Then, the system **assesses daily NOOS availability**. For Store A, it counts 20 days where the T-shirt's size availability was at or above 90%. For Store B, it counts 28 days where the T-shirt's size availability was at or above 90%.
3. Next, the system **calculates consistent NOOS availability duration**.
 - Store A: The “Classic White T-Shirt” was available for 20 days. Since this is less than the “Minimum Days In-Stock for NOOS Styles” (25 days), it's flagged as an availability gap for this store.
 - Store B: The “Classic White T-Shirt” was available for 28 days. This meets the 25-day minimum, so Store B is performing well for this NOOS item.
4. Next, the system **grades brand performance**. It aggregates the performance of “ComfortWear” and all other NOOS styles from this brand across all stores. If many of ComfortWear's NOOS items across the chain perform like Store A, ComfortWear's overall brand grade will be negatively impacted.

5. Finally, all these daily counts, consistent availability summaries, and brand grades are **stored and consolidated** into a comprehensive report.

Final Outcome

- The report highlights a significant NOOS availability gap for the “Classic White T-Shirt” in **Store A**, indicating that this core product was consistently unavailable for a notable period. This signals a need for immediate investigation and replenishment for that store.
- **Store B** is confirmed to be effectively managing the NOOS status for this product.
- “ComfortWear” receives a brand grade that reflects the overall availability of its NOOS products across the entire chain, helping category managers evaluate their relationship with the brand.

Key Takeaway: This analysis clearly identifies specific stores and products failing to maintain their Never-Out-Of-Stock status, allowing merchants to proactively address consistent availability issues and prevent lost sales and customer dissatisfaction.

7. Results

This step produces:

Main Outputs

- **Daily Store NOOS Availability Report:** Provides a detailed, day-by-day view of which NOOS products met their Ideal Size Set (ISS) Availability Benchmark at each store. This helps pinpoint exact dates and locations of stockouts.
- **Store NOOS In-Stock Duration Summary:** Shows, for each store, how many NOOS products consistently met the “Minimum Days In-Stock” threshold. This offers an aggregated overview of a store’s NOOS performance.
- **Detailed Brand Performance Report:** A comprehensive breakdown of how well brands are maintaining their NOOS availability across the entire business, used for strategic brand management and future purchasing decisions.

Business Reports

- **NOOS Performance Dashboard:** Used by merchandising and inventory planners to quickly identify critical NOOS availability gaps across specific stores and products. This enables rapid replenishment or reallocation actions to minimize lost sales.
- **Strategic Brand Health Review:** Used by category managers and buying teams to evaluate brand partners based on their ability to consistently keep core products in stock. This informs future assortment planning and vendor negotiations.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will pause or use the last available snapshot, leading to potentially outdated analysis. An alert will be triggered.	Verify data source connections and ensure all required data feeds (inventory, sales, product master) are active and providing up-to-date information. Rerun the analysis once data is restored.
No NOOS items meet consistent availability	The report might show a high number of gaps, indicating widespread issues, or very few items meeting the “Minimum Days In-Stock” threshold.	Review overall inventory management and supply chain for NOOS products. Consider if the “Minimum Days In-Stock for NOOS Styles” setting is too high, or if forecasting needs adjustment.
All NOOS items consistently available	The report will show very few or no gaps, which might indicate a healthy NOOS performance or overly lenient settings.	Verify the “Ideal Size Set (ISS) Availability Benchmark” or “Minimum Days In-Stock for NOOS Styles” settings aren’t too low, potentially masking subtle issues that still impact sales.
Sudden drop in brand grade	A brand’s performance grade might significantly decline without an obvious reason, despite consistent sales.	Investigate recent supply chain disruptions, new product launches affecting inventory focus, or changes in sales patterns for that brand’s NOOS products. Engage with the vendor to understand challenges.

9. Next Steps

Before This Step

- **NOOS Product Definition:** A clear, business-driven list of products designated as “Never-Out-Of-Stock” must be established and consistently maintained. This identifies which items this module should focus on.
- **Daily Inventory Snapshot:** Accurate and up-to-date daily inventory levels by unique item (SKU), style, and store are required. This provides the raw data for assessing availability against benchmarks.
- **Ideal Size Set (ISS) Configuration:** The ideal mix of sizes for each product must be defined and kept current. This module uses the ISS to determine if a product is truly “available” and ready to sell.

After This Step

- **Replenishment and Allocation Planning:** The identified NOOS gaps and underperforming stores guide immediate actions for restocking and transferring

inventory to high-demand locations. They need specific item and location recommendations from us.

- **Vendor Performance Review:** Insights into brand-level NOOS availability inform discussions with suppliers about their ability to consistently meet demand for core products. This helps influence future purchasing decisions and strengthens partnerships.
- **Assortment Optimization:** Long-term trends from NOOS gaps may lead to strategic adjustments in the overall Optimum Depth (OD) or Optimum Width (OW) for categories, ensuring the right balance of core and new products for the business. They need insights into persistent availability challenges.

10. Common Questions

Q: Why is NOOS gap analysis important for my business? A: It ensures your most critical, high-demand products are always in stock, preventing lost sales, improving customer satisfaction, and building brand loyalty. When customers can always find what they want, they are more likely to return.

Q: How is “availability” defined for a NOOS item in this analysis? A: For this analysis, a NOOS item is considered “available” at a store on a given day if a specified percentage (e.g., 90%) of its Ideal Size Set (ISS) is currently in stock. This ensures not just *any* stock, but the *right* stock in the *right* sizes is present.

Q: Can this analysis tell me *why* a NOOS item is out of stock? A: While this analysis identifies *that* an item is out of stock and for how long, it doesn’t directly pinpoint the root cause (e.g., a specific supply chain delay or an inaccurate forecast). However, consistent flagging of an item suggests deeper investigation into supply chain or planning issues is critically needed.

Q: How does this help manage inventory more effectively? A: By focusing on core, high-value NOOS items, it helps prioritize replenishment efforts to ensure capital is invested where it yields the highest return. This avoids costly stockouts on your best sellers and maximizes sales opportunities.

Q: What action should I take if a brand consistently has low NOOS performance? A: A consistently low brand grade indicates a systemic issue. You should review your purchasing agreements with that brand, assess their lead times, explore alternative suppliers, or adjust your inventory targets and safety stock for their products to ensure better availability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To systematically monitor and report on the consistent availability of “Never-Out-Of-Stock” (NOOS) products across all stores and assess relevant brand performance.
- **Main Input:** A defined list of NOOS products, accurate daily store inventory levels, and Ideal Size Set (ISS) configurations.
- **Main Output:** Reports detailing daily NOOS availability by store, consistent NOOS in-stock duration, and brand performance grades.

- **Warning Signs:** Persistent NOOS gaps in high-volume stores, a sudden drop in brand grades, or an unusually high number of “consistently available” items without corresponding sales.
 - **Key Settings:** NOOS Analysis Period, Ideal Size Set (ISS) Availability Benchmark, Minimum Days In-Stock for NOOS Styles.
-

Need Help? Contact your Merchandise Planning Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: Ideal Size Set (ISS) Comparison

Last Updated: 2025-05-23

1. Business Objective

This module evaluates how effectively our inventory's size distribution aligns with the Ideal Size Set (ISS), identifying demand that could not be met due to insufficient stock in key sizes.

Why This Matters: This comparison ensures we have the right sizes in stock for the right products, preventing lost sales from popular sizes being unavailable. It helps optimize inventory depth (OD) by ensuring balanced stock across sizes, leading to higher customer satisfaction and better sales performance.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** The number of items sold over a specific period, broken down by individual sizes and Attribute Groups (AGs). This provides the actual historical demand for each size.
- **Rate of Sale (ROS):** The speed at which products are sold. While not a direct input for size comparison, it helps validate the overall popularity of a product whose size set is being analyzed.

Product Data

- **Ideal Size Set (ISS) Configuration:** The predefined target combination of sizes for a specific product or Attribute Group (AG). This serves as the benchmark against which actual stock and sales are compared.
- **Product Attributes (AG):** Features like product type, fit, or style family, used to group items for which a common Ideal Size Set (ISS) might apply. This helps categorize sales and inventory for meaningful analysis.
- **Product Master Data:** Basic information about each unique item (SKU), including its size, which is essential for categorizing sales and inventory accurately.

Operational Data

- **Current Inventory Levels:** The exact number of units currently in stock for each SKU and size across all locations. This shows what is available right now to meet demand.
- **Analysis Period:** The specific date range over which historical sales data and inventory snapshots are considered for the analysis. This ensures the evaluation is relevant to recent demand patterns.
- **Season Tagging:** Information identifying which products or styles belong to which selling seasons. This ensures the analysis is focused on products relevant to the current or recent season.

3. Business Settings

Setting Name	Default Value	Description	When to Change
ISS Inventory End Date	Current Day	The specific date for which current inventory levels are taken as a snapshot for comparison against the Ideal Size Set (ISS).	Adjust this if you want to analyze inventory effectiveness as of a past date, for example, to review a specific promotional period.
Analysis Period Duration	13 months	The historical period used to gather sales data for calculating actual sales by size and estimating ideal sales.	Extend this for products with longer selling cycles or highly seasonal demand, or shorten for fast-moving fashion items.
Season Tagging	Current Season	A filter to include only products associated with specific selling seasons (e.g., “Fall 2024,” “Summer 2025”).	Change this to focus the analysis on upcoming seasons or to review the performance of past seasonal merchandise.
Minimum Sales Threshold	5 units	The minimum number of units an Attribute Group (AG) must have sold during the Analysis Period to be included in the Ideal Size	Increase this for high-volume categories to focus on top performers, or decrease it for niche or new categories that need attention despite lower initial sales.

Setting Name	Default Value	Description	When to Change
		Set (ISS) comparison.	

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Ideal Size Set (ISS) Configuration & Sales History] -->
    B[Calculate Ideal & Actual Sales by Size]
    B --> C[Gather Current Inventory]
    C --> D[Identify Unliquidated Sales Quantities]
  
```

1. Start with → **Ideal Size Set (ISS) Configuration & Sales History**: The system retrieves the defined Ideal Size Sets for each Attribute Group (AG) and gathers relevant historical sales data by size.
2. Then → **Calculate Ideal & Actual Sales by Size**: The system determines the actual historical sales for each size within an AG and calculates the ideal sales quantity for each size based on the ISS.
3. Next → **Gather Current Inventory**: The system collects the precise current inventory levels for all relevant products, broken down by size.
4. Finally → **Identify Unliquidated Sales Quantities**: The system compares the ideal sales quantities against actual sales and current inventory to identify specific instances where demand for a size could not be met.

5. Step-by-Step Actions

1. **Initialize** the analysis environment
 - What we do: *We gather the relevant Ideal Size Set (ISS) configurations and define the historical sales period for analysis, typically over the last 13 months.*
 - Why we do it: *This ensures we have the necessary benchmarks and a sufficiently long sales history to identify stable demand patterns for each size.*
2. **Filter** products by season and gather inventory
 - What we do: *We narrow the analysis to styles within the specified seasons and capture a snapshot of current inventory levels for all unique items (SKUs) at the defined ISS Inventory End Date.*
 - Why we do it: *This ensures our size comparison is relevant to seasonal merchandise and uses the most up-to-date inventory picture to assess immediate stock issues.*
3. **Compute** actual and ideal sales by size
 - What we do: *For each Attribute Group (AG), we calculate the actual units sold for each size during the analysis period. Simultaneously, we determine the 'ideal sales quantity' for each size by applying the Ideal Size Set's (ISS) target distribution to the AG's total historical sales.*

- Why we do it: *This provides a direct comparison point: what we actually* sold for a size versus what we should have sold if our size distribution perfectly matched the ISS.**
4. **Identify** unliquidated sales quantities
- What we do: *By comparing the ‘ideal sales quantity’ against the actual historical sales and current inventory for each size within an Attribute Group (AG), the module flags quantities where demand for a size could not be met due to stock discrepancies.*
 - Why we do it: *This directly quantifies lost sales opportunities and highlights specific sizes within popular Attribute Groups that are consistently understocked relative to their ideal demand.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retail brand sells “Women’s Core Denim” (Attribute Group ID: 501).
- **Ideal Size Set (ISS) Configuration** for this AG: Size 28 should represent 15% of sales, Size 30 should be 20%, Size 32 should be 18%, etc.
- **Total Historical Sales** for “Women’s Core Denim” (AG 501) over the last 13 months: 1,000 units.
- **Actual Historical Sales by Size** for AG 501: Size 28 sold 100 units, Size 30 sold 150 units, Size 32 sold 180 units.
- **Current Inventory** for AG 501: Size 28 has 10 units, Size 30 has 5 units, Size 32 has 50 units.
- **ISS Inventory End Date:** Today (2025-05-23).

What Happens

1. First, the system **initializes** by loading the ISS configuration for “Women’s Core Denim” and gathers the 1,000 units of total historical sales for this Attribute Group. It also takes a snapshot of current inventory.
2. Then, the system **calculates ideal and actual sales by size**.
 - For Size 28: Ideal Sales Quantity = 15% of 1,000 units = 150 units. Actual Sales = 100 units.
 - For Size 30: Ideal Sales Quantity = 20% of 1,000 units = 200 units. Actual Sales = 150 units.
 - For Size 32: Ideal Sales Quantity = 18% of 1,000 units = 180 units. Actual Sales = 180 units.
3. Next, the system **gathers current inventory**. It notes Size 28 has 10 units, Size 30 has 5 units, and Size 32 has 50 units.
4. Finally, the system **identifies unliquidated sales quantities**.
 - **Size 28:** Ideal Sales Quantity (150 units) is higher than Actual Sales (100 units). With only 10 units in current inventory, this size is flagged for potential unliquidated sales (150 - 100 = 50 units short historically).
 - **Size 30:** Ideal Sales Quantity (200 units) is higher than Actual Sales (150 units). With only 5 units in current inventory, this size is also flagged for significant potential unliquidated sales (200 - 150 = 50 units short historically).

- **Size 32:** Actual Sales (180 units) met Ideal Sales Quantity (180 units). With 50 units in current inventory, this size is performing well and does not indicate unliquidated sales.

Final Outcome

The merchant receives a report indicating that for “Women’s Core Denim”: - **Size 28** has a notable gap, indicating that 50 units of ideal demand were not met historically, and current stock is very low. - **Size 30** also has a significant gap, with 50 units of ideal demand not met historically, and extremely low current stock. - **Size 32** is well-aligned with its Ideal Size Set, with no unliquidated sales.

Key Takeaway: This analysis pinpoints specific sizes (e.g., 28 and 30) within popular product groups where stock has historically fallen short of ideal demand, allowing merchants to adjust future buying and replenishment for improved sales and customer satisfaction.

7. Results

This step produces:

Main Outputs

- **Size Set Gap Report (Detailed):** Provides specific data for each Attribute Group (AG) and size, including actual historical sales, ideal sales quantity (based on ISS), and current inventory. This enables a precise understanding of where size gaps exist.
- **Unliquidated Sales Quantity Metrics:** Quantifies the estimated demand that could not be met for specific sizes within AGs, indicating potential lost revenue.
- **AG-Level Size Set Health Summary:** Aggregates size gap information for entire Attribute Groups, offering a high-level view of how well each AG’s size distribution performs against its Ideal Size Set (ISS).

Business Reports

- **Daily Size Set Optimization Report:** Used by merchandise planners to quickly identify specific sizes that are understocked relative to their ideal demand and current inventory. This helps prioritize immediate replenishment or re-allocation actions to capture lost sales.
- **Assortment Planning Insights:** Used by buying teams to inform future purchase orders. Consistently identified size gaps (especially for popular AGs) indicate areas where the Ideal Size Set needs to be better reflected in inventory plans, optimizing Optimum Depth (OD) and Optimum Width (OW) across the assortment.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will pause or use the last available snapshot, leading to potentially outdated analysis. An alert will be triggered.	Verify data source connections and ensure all required data feeds (sales, inventory, product master, ISS configurations) are active and providing up-to-date information. Rerun the analysis once data is restored.
No historical sales for an AG	The module cannot calculate 'ideal sales quantity' for sizes within that Attribute Group (AG), resulting in no size gap insights for that AG.	Review if the AG is truly active and if sales data is being correctly captured. Consider adjusting the "Minimum Sales Threshold" if it's a new or niche AG.
Ideal Size Set (ISS) is missing or incomplete	The analysis cannot accurately determine 'ideal sales quantity' for affected Attribute Groups (AGs), leading to unreliable or no size gap identification for those items.	Ensure all relevant Attribute Groups have a defined and complete Ideal Size Set (ISS) configured and loaded into the system.
Inventory snapshot shows zero for all sizes	The analysis will flag significant unliquidated sales across all sizes, as no current stock is available to meet demand.	This indicates a critical issue with inventory recording or a complete stockout. Immediately investigate inventory data accuracy or supply chain failures.

9. Next Steps

Before This Step

- **Ideal Size Set (ISS) Configuration:** The target size distribution for each Attribute Group (AG) must be clearly defined and loaded into the system. This module relies on an accurate ISS to set performance benchmarks.
- **Accurate Inventory & Sales Data:** Up-to-date and clean historical sales data by size and Attribute Group (AG), along with a reliable current inventory snapshot, are essential. This provides the raw information for comparison.

After This Step

- **Replenishment & Allocation Planning:** The identified unliquidated sales quantities and specific size gaps guide immediate actions for restocking, inter-store transfers, or prioritizing new orders. They need specific item, size, and location recommendations from us.

- **Assortment Review & Optimization:** Insights into consistent size gaps or overstocks for certain sizes inform strategic adjustments to future buying plans and Ideal Size Set (ISS) configurations. They need trends, root causes, and recommendations for improving Optimum Depth (OD) and Optimum Width (OW) from us.

10. Common Questions

Q: Why is it important to compare my actual size performance against an Ideal Size Set (ISS)? A: Comparing against an ISS allows you to move beyond just ‘selling out’ and understand if you sold *enough* of each size. It highlights where you missed potential sales because popular sizes weren’t adequately stocked, ensuring you maximize revenue for every product.

Q: How does this analysis help reduce lost sales? A: By precisely identifying which sizes within an Attribute Group (AG) have unliquidated sales quantities, it enables targeted replenishment. This means you can quickly get the right sizes back in stock, ensuring customers find what they need and don’t leave empty-handed.

Q: Can this help me with future buying decisions, not just immediate replenishment? A: Absolutely. Consistent patterns of unliquidated sales for certain sizes indicate that your current Ideal Size Set (ISS) or your buying strategy needs adjustment. This module provides the data to inform future purchasing, ensuring you order the right quantity of each size to meet demand.

Q: What if I have a product with many sizes, but only a few are truly popular? A: The Ideal Size Set (ISS) configuration should reflect this reality. This module will then highlight if those truly popular sizes are consistently missing stock, allowing you to prioritize replenishment and ensure these key sizes are always available.

Q: How does this module contribute to optimizing Optimum Depth (OD)? A: By pinpointing specific size gaps within an Attribute Group (AG), it helps refine the Optimum Depth (OD) by ensuring that the *right mix* of sizes is present. It prevents investing too heavily in less popular sizes while neglecting high-demand ones, optimizing capital efficiency.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To evaluate the effectiveness of the Ideal Size Set (ISS) by comparing actual sales and inventory against target size distributions to identify demand not met due to stock issues.
- **Main Input:** Defined Ideal Size Set (ISS) configurations, historical sales data by size, and current inventory levels.
- **Main Output:** Detailed reports on actual vs. ideal sales by size, highlighting specific unliquidated sales quantities at the Attribute Group (AG) and size level.
- **Warning Signs:** Large discrepancies between actual and ideal sales for popular sizes, consistently missing Ideal Size Set (ISS) configurations, or zero inventory data.
- **Key Settings:** ISS Inventory End Date, Analysis Period Duration, Season Tagging, Minimum Sales Threshold.

Need Help? Contact your Merchandise Planning Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: Broken vs. Non-Broken Sales Analysis

Last Updated: 2025-05-23

1. Business Objective

This module analyzes historical sales data to differentiate between “broken” days (when the Ideal Size Set (ISS) benchmark for inventory coverage was not met) and “non-broken” days. It then calculates the Rate of Sale (ROS) for each state and quantifies the potential sales loss attributable to stockouts or insufficient inventory on broken days.

Why This Matters: This analysis provides a clear, quantifiable understanding of the financial impact of not having optimal stock. By identifying precisely how much sales are lost due to incomplete inventory (especially regarding Ideal Size Set), it allows businesses to prioritize replenishment efforts, improve inventory accuracy, and optimize future buying decisions to maximize revenue and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** The total number of units sold for each style, in each store, on each day within the analysis period. We need this to calculate the Rate of Sale (ROS) for both broken and non-broken periods.
- **Sales History by Style and Day:** Past sales performance for specific product styles, broken down by individual days. This is crucial for categorizing each day’s sales performance as either “broken” or “non-broken” based on inventory status.

Product Data

- **Ideal Size Set (ISS) Configuration:** The predefined target combination of sizes for a specific product. This serves as the critical benchmark to determine if a product’s inventory is “complete” enough on a given day to be considered “non-broken.”
- **Style Definitions:** How products are grouped into distinct styles (e.g., ‘Women’s Skinny Jeans’). This helps categorize products for focused analysis.

Operational Data

- **Daily Inventory Snapshots:** The exact number of units currently in stock for each unique item (SKU) and size across all locations, captured daily. This is essential for calculating the Ideal Size Set (ISS) coverage on any given day.
- **Store Locations:** Information about all active retail locations. This allows the system to analyze brokenness and sales loss for each specific store.
- **Style Live Status:** Information indicating whether a particular style was available for sale at a given store on a specific day. This ensures only active selling periods are considered for analysis.

3. Business Settings

Setting Name	Default Value	Description	When to Change
ROS Comparison Period Start Date	90 days prior to current date	The starting date for the historical sales data considered for this analysis.	Adjust this to longer periods (e.g., 180 days) for seasonal items or shorter (e.g., 30 days) for fast-fashion or highly volatile trends to focus on recent performance.
ROS Comparison Period End Date	Current Day	The ending date for the historical sales data considered for this analysis.	Adjust this if you want to analyze performance up to a specific past date, for example, to review a particular promotional period.
Ideal Size Set (ISS) Match Tolerance	80%	The percentage of the Ideal Size Set (ISS) that must be in stock to consider the size set “complete” and the day “non-broken.” Below this, a day is considered “broken.”	Set higher (e.g., 90%) for core items where all sizes are critical, lower (e.g., 70%) for less critical items or those with less predictable size demand.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Sales History & Daily Inventory Snapshots] --> B[Determine Daily ISS Coverage & Classify Days]
B --> C[Calculate ROS for Broken & Non-Broken Days]
C --> D[Broken/Non-Broken Sales Analysis Report & Potential Loss]

1. Start with → **Sales History & Daily Inventory Snapshots**: The system gathers all relevant historical sales data by style, store, and day, along with daily snapshots of inventory levels and Ideal Size Set (ISS) configurations.
2. Then → **Determine Daily ISS Coverage & Classify Days**: For each style, store, and day, the system assesses the percentage of its Ideal Size Set (ISS) that was in stock. If this percentage falls below the “Ideal Size Set (ISS) Match Tolerance,” that day is classified as “broken” for that specific style and store; otherwise, it’s “non-broken.”
3. Next → **Calculate ROS for Broken & Non-Broken Days**: The system calculates the Rate of Sale (ROS) for each style and store, separately for all “broken” days and all “non-broken” days. It then quantifies the potential sales loss by comparing these two ROS figures.
4. Finally → **Broken/Non-Broken Sales Analysis Report & Potential Loss**: All calculated data, including days classified, ROS figures, and the estimated potential sales loss, are consolidated into actionable reports.

5. Step-by-Step Actions

1. **Initialize** the analysis environment
 - What we do: *We define the specific historical period for analysis (ROS Comparison Period) and prepare all necessary sales, inventory, and product data, including Ideal Size Set configurations.*
 - Why we do it: *This ensures we focus on a relevant timeframe and have all the foundational data needed to accurately assess inventory completeness and sales performance.*
2. **Determine** daily inventory completeness by style and store
 - What we do: *For every style at every store, the system looks at each day within the analysis period and checks if its current inventory adequately covered its Ideal Size Set (ISS) based on the Ideal Size Set (ISS) Match Tolerance.*
 - Why we do it: *This identifies exactly when and where a product’s stock was insufficient to meet ideal customer demand, laying the groundwork for categorizing sales days.*
3. **Categorize** sales days as “broken” or “non-broken”
 - What we do: *Based on the daily inventory completeness assessment, each day for a given style and store is labeled either “broken” (if ISS coverage was below the benchmark) or “non-broken” (if ISS coverage met or exceeded the benchmark).*
 - Why we do it: *This allows us to isolate sales data that occurred during periods of stock sufficiency versus periods of stock insufficiency.*
4. **Calculate** Rate of Sale (ROS) for each category
 - What we do: *The system computes the average Rate of Sale (ROS) for each style at each store, separately for its “broken” days and its “non-broken” days, by dividing total sales by the number of days in each category.*

- Why we do it: *This provides a direct comparison of how quickly products sell when stock is optimal versus when it's constrained by incomplete size sets.*
5. **Quantify potential sales loss**
- What we do: *For each style and store, the system estimates the potential sales loss by multiplying the “non-broken” ROS by the number of “broken” days, then subtracting the actual sales achieved on those “broken” days.*
 - Why we do it: *This puts a clear value on the sales opportunities missed specifically because of insufficient inventory, helping to prioritize which gaps to address first.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Women’s Core Blouse” (Style ID: 205) is sold in Store M.
- **ROS Comparison Period:** Last 30 days.
- **Ideal Size Set (ISS) Match Tolerance:** 80%.
- Over the last 30 days for Style 205 in Store M:
 - On 10 days, the Ideal Size Set (ISS) coverage for the blouse was below 80% (e.g., popular sizes were missing).
 - On the remaining 20 days, the Ideal Size Set (ISS) coverage was 80% or above (all key sizes were in stock).
- **Actual Sales:**
 - Sales on the 10 “broken” days: 40 units.
 - Sales on the 20 “non-broken” days: 160 units.

What Happens

1. First, the system **initializes** by gathering all sales and inventory data for “Women’s Core Blouse” (Style 205) in Store M for the last 30 days, along with its Ideal Size Set configuration.
2. Then, the system **determines daily inventory completeness** by comparing Style 205’s daily inventory against its Ideal Size Set (ISS) benchmark. For each day, it flags whether the ISS coverage was below or above 80%.
3. Next, the system **categorizes sales days**. It identifies 10 days where the ISS coverage was poor (classified as “broken”) and 20 days where it was good (classified as “non-broken”).
4. Next, the system **calculates the Rate of Sale (ROS)** for each category:
 - **ROS on Non-Broken Days:** 160 units / 20 days = 8 units/day. This represents the blouse’s sales potential when it’s well-stocked.
 - **ROS on Broken Days:** 40 units / 10 days = 4 units/day. This represents the sales achieved when stock was constrained.
5. Finally, the system **quantifies potential sales loss**:
 - If the blouse had sold at its “non-broken” ROS during the “broken” days: 8 units/day * 10 broken days = 80 units.
 - Actual sales on “broken” days were 40 units.
 - Potential sales loss = 80 units (potential) - 40 units (actual) = 40 units.

Final Outcome

The merchant receives a report indicating that for “Women’s Core Blouse” in Store M: - It experienced 10 “broken” days due to insufficient Ideal Size Set coverage. - During these broken periods, sales were significantly slower (4 units/day) compared to when stock was optimal (8 units/day). - The estimated potential sales loss for this specific style in this store due to stockouts was 40 units, representing a direct revenue opportunity that was missed.

Key Takeaway: This analysis clearly shows the financial cost of incomplete inventory, allowing merchants to prioritize restocking the “Women’s Core Blouse” in Store M and avoid similar future sales losses.

7. Results

This step produces:

Main Outputs

- **Broken/Non-Broken Sales Report by Store and Style:** A detailed breakdown for each style and store, including the number of broken and non-broken days, total units sold during each period, the calculated Rate of Sale (ROS) for both states, and the estimated potential sales loss. This provides granular insight into inventory effectiveness.
- **Aggregated Potential Sales Loss Metrics:** Summaries of total estimated lost sales due to broken inventory, grouped by category, brand, or overall business. This highlights major areas of revenue opportunity.

Business Reports

- **Daily Lost Sales Opportunity Report:** Used by merchandise planners and store managers to quickly identify and address specific items and locations where incomplete inventory is actively costing sales. This enables rapid replenishment or inter-store transfers.
- **Inventory Health Dashboard:** Used by merchandising directors and inventory managers to review the overall effectiveness of inventory management, spot recurring patterns of stockouts, and inform strategic decisions on buying and allocation to improve overall profitability.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will pause or use the last available snapshot, leading to potentially outdated analysis. An alert will be triggered.	Verify data source connections and ensure all required data feeds (sales, inventory, product master, ISS configurations) are active and providing up-to-date information. Rerun the analysis once data is restored.

Situation	What Happens	What You Should Do
All sales days flagged as “Broken”	The report will show overwhelming sales loss, making prioritization difficult and indicating widespread stock issues.	This signals a severe, systemic inventory problem. Review your entire inventory strategy, supply chain, and Ideal Size Set (ISS) configurations. Prioritize high-revenue items immediately.
All sales days flagged as “Non-Broken”	The report will show very few or no gaps, which might indicate excellent inventory management or overly lenient settings that mask subtle issues.	Verify the “Ideal Size Set (ISS) Match Tolerance” setting isn’t too low, potentially masking actual stockouts. Ensure your ISS configurations are accurate and challenging enough.
Unexpected “Negative Loss” Calculation	The estimated potential sales loss appears as a negative number. This means actual sales on broken days were <i>higher</i> than what would be predicted by the non-broken ROS.	This is unusual. It could indicate a sudden surge in demand on limited stock, or that the non-broken days used for comparison were unusually slow. Investigate the specific sales and inventory data for that item.

9. Next Steps

Before This Step

- **Ideal Size Set (ISS) Configuration:** The target size distribution for each product or Attribute Group (AG) must be clearly defined and loaded into the system. This module relies on an accurate ISS to determine inventory completeness.
- **Accurate Daily Inventory Snapshots:** Up-to-date and clean daily inventory levels by unique item (SKU), style, and store are required. This provides the raw data for assessing daily Ideal Size Set (ISS) coverage.
- **Historical Sales Data Collection:** Reliable and comprehensive historical sales data by item, style, store, and day is essential for calculating the Rate of Sale (ROS) for different inventory states.

After This Step

- **Replenishment & Allocation Planning:** The identified potential sales loss and specific broken periods guide immediate actions for restocking, inter-store transfers, or prioritizing new orders. They need specific item, size, and location recommendations from us.
- **Assortment Optimization:** Insights into recurring “broken” sales and consistent lost revenue inform strategic adjustments to future buying plans and Ideal Size Set (ISS) configurations. They need trends and root causes from us to optimize Optimum Depth (OD) across the assortment.

10. Common Questions

Q: Why is it important to distinguish between “broken” and “non-broken” sales? A: It’s crucial because it allows you to quantify how much sales potential is truly lost due to inventory issues, rather than simply having low sales. It separates sales that occurred despite stock limitations from sales that happened when products were fully available.

Q: How does this analysis help quantify lost sales? A: By comparing the Rate of Sale (ROS) when inventory was complete (“non-broken” days) to the actual sales achieved when inventory was incomplete (“broken” days), the system calculates the difference, attributing that gap directly to stockouts or insufficient size sets.

Q: Can this analysis help improve sales forecasting? A: Yes, by understanding the true Rate of Sale (ROS) when inventory is optimal (“non-broken” ROS), businesses can develop more accurate sales forecasts that reflect unconstrained demand, leading to better inventory planning and reduced lost sales.

Q: What if a product consistently appears as “broken” with high potential sales loss? A: This indicates a critical demand-supply imbalance. You should investigate supply chain issues, re-evaluate supplier lead times, consider increasing safety stock, or adjust your target inventory levels for that item to ensure better availability.

Q: How does this module improve customer experience? A: By precisely identifying products and sizes that are frequently unavailable, this analysis helps businesses proactively address stockouts. This ensures customers are more likely to find the products and sizes they want, leading to higher satisfaction and repeat business.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To quantify potential sales loss caused by incomplete inventory (broken Ideal Size Set) by comparing sales rates during optimal and suboptimal stock periods.
- **Main Input:** Historical sales data (units sold), daily inventory snapshots, and Ideal Size Set (ISS) configurations.
- **Main Output:** Detailed reports on Rate of Sale (ROS) for “broken” vs. “non-broken” days, and the estimated potential sales loss by style and store.
- **Warning Signs:** Consistently high “broken” days across a category, or significant quantified “potential sales loss” for key items.
- **Key Settings:** ROS Comparison Period Start Date, ROS Comparison Period End Date, Ideal Size Set (ISS) Match Tolerance.

Need Help? Contact your Merchandise Planning Support team for questions about this process.

gap FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document provides high-level answers to common questions about the gap algorithm, focusing on its business purpose, inputs, process, and outputs.

General

Q: What is the primary purpose of the gap algorithm? **A:** The gap algorithm is designed to identify and quantify potential sales losses due to insufficient inventory and to optimize product availability across different categories and sizes. It helps ensure you stock the right products in the right quantities.

Q: What specific business problems does gap help solve? **A:** It addresses issues like missed sales opportunities due to stockouts, inefficient inventory allocation, and difficulty in maintaining core product availability. It provides insights to minimize “unliquidated sales quantities.”

Q: Who typically uses the insights generated by the gap algorithm? **A:** Merchandisers, inventory planners, assortment planners, and category managers benefit most, using the insights to refine their buying decisions, improve inventory management, and enhance customer satisfaction.

Q: What are the main types of outputs or recommendations provided by gap? **A:** The algorithm quantifies potential sales losses, identifies discrepancies in Ideal Size Set (ISS) coverage, highlights performance gaps for Never-Out-Of-Stock (NOOS) styles, and provides data to optimize Optimum Depth (OD) and Optimum Width (OW).

Q: How often should this analysis be run? **A:** The optimal frequency depends on your business cycle and data freshness needs. It's often run periodically (e.g., weekly, monthly, or quarterly) to inform ongoing inventory and merchandising decisions.

Q: Can you explain the key terms used in the documentation? **A:** * **OD (Optimum Depth):** The ideal number of unique items (SKUs) to stock in a category or display. * **OW (Optimum Width):** The ideal number of different categories or styles to offer. * **ISS (Ideal Size Set):** The best combination of sizes to maximize sales and minimize stockouts. * **ROS (Rate of Sale):** The speed at which products are sold over a period (e.g., units/week). * **AG (Attribute Group):** A set of product attributes grouped for analysis (e.g., color, size, brand).

Configuration & Settings

Q: How do I define the period for the gap analysis? **A:** The analysis period, including start and end dates, is typically specified through configuration settings. These dates define the historical window for sales and inventory data analysis.

Q: Can I narrow down the analysis to specific product categories or brands? **A:** Yes, the algorithm can be configured to focus on specific product categories, brands, or other Attribute Groups (AG) relevant to your analysis needs. This targeting is usually managed via input parameters or configuration files.

Q: Are there any key thresholds or benchmarks I should be aware of? **A:** Yes, the analysis uses benchmarks, such as the inventory coverage target for the Ideal Size Set (ISS) and specific availability targets for Never-Out-Of-Stock (NOOS) styles. These are often configurable to align with your business goals.

Q: Where can I find the system's configuration parameters? **A:** Configuration parameters are typically stored in a dedicated configuration file (e.g., `args.properties`) or managed through an administrative interface. Refer to the system's main documentation for exact locations.

Q: Can the definition of a "broken" day be adjusted? **A:** A "broken" day is currently defined when the Ideal Size Set (ISS) benchmark for inventory coverage is not met. While the core definition is consistent, the underlying ISS benchmark is configurable.

Data & Inputs

Q: What types of data does the gap algorithm require to run? **A:** The algorithm primarily requires historical sales data, current inventory data, and product master data (including Attribute Groups like size and color information). Logical date ranges are also crucial.

Q: How far back does the historical sales data need to go for effective analysis? **A:** The required depth of historical sales data depends on the desired analysis period. A minimum of several weeks or months is usually necessary to accurately calculate Rate of Sale (ROS) and identify sales trends.

Q: What happens if the input data is incomplete or has errors? **A:** The Gap Analysis Input Validation module rigorously checks input parameters and data availability. If critical data is missing or illogical (e.g., invalid dates), the process will halt early to prevent erroneous downstream analysis, informing you of the issue.

Q: What is an Ideal Size Set (ISS) and where does this information come from? **A:** The Ideal Size Set (ISS) is a benchmark representing the optimal combination of sizes for a product or category. It's typically derived from past sales performance, market insights, or predefined business rules, and serves as a target for inventory comparison.

Q: Why is an Attribute Group (AG) important for the analysis? **A:** An Attribute Group (AG) allows the algorithm to analyze product performance and inventory at a granular level (e.g., by color, size, or specific style). This helps in pinpointing exact areas of sales loss or inventory discrepancies within an AG.

Process & Logic

Q: How does the gap algorithm determine potential lost sales? **A:** The Broken vs. Non-Broken Sales Analysis module compares sales on “broken” days (when ISS coverage wasn’t met) against sales on “non-broken” days (when ISS was met). By analyzing the Rate of Sale (ROS) in both scenarios, it quantifies the sales volume potentially lost due to stockouts or insufficient inventory.

Q: What does “broken” mean in the context of sales analysis? **A:** A “broken” day signifies a day where the inventory available for a specific product or Attribute Group (AG) did not meet the Ideal Size Set (ISS) benchmark, indicating a potential stockout or insufficient depth to meet demand.

Q: How is the Ideal Size Set (ISS) used in the analysis? **A:** The Ideal Size Set (ISS) Comparison module uses the ISS as a benchmark. It compares historical sales and current inventory against the ISS to identify “unliquidated sales quantities” – demand that could not be met due to inventory issues at the AG and size level.

Q: What is a “Never-Out-Of-Stock” (NOOS) style and why is it analyzed separately? **A:** A NOOS style is a core product or basic item that should ideally always be in stock. The Never-Out-Of-Stock (NOOS) Gap Analysis module specifically tracks their availability to ensure these critical items meet their benchmarks, as their consistent availability is crucial for brand perception and foundational sales.

Q: What is the role of the Gap Analysis Orchestration module? **A:** This module is the central coordinator. It defines and manages the entire sequence of execution for all sub-modules, ensuring that data validation, core analysis (NOOS, ISS, Broken/Non-Broken Sales), and output synchronization happen in the correct order.

Q: How does the algorithm ensure data quality before initiating deeper analysis? **A:** The Gap Analysis Input Validation module runs at the very beginning of the process, checking that all required input parameters are logical and critical inventory and sales data is available. This prevents the system from running with invalid data and ensures reliable results.

Outputs & Results

Q: What kind of insights can I gain from the gap algorithm’s outputs? **A:** You’ll gain insights into quantifiable sales losses by product, category, and size; areas where your Ideal Size Set (ISS) is not being met; performance of core Never-Out-Of-Stock (NOOS) items; and data-driven recommendations to optimize Optimum Depth (OD) and Optimum Width (OW).

Q: How can I use the results to improve inventory planning and merchandising? **A:** The results help you understand where demand is unmet due to stockouts. You can use this to adjust future buys, refine allocation strategies, optimize your Ideal Size Set (ISS) for better coverage, and prioritize inventory for high-performing or critical NOOS styles to maximize sales and minimize lost revenue.

Q: What do “unliquidated sales quantities” refer to in the output? **A:** “Unliquidated sales quantities” represent the estimated demand for a product or size that could not be fulfilled because the item was out of stock or available inventory did not meet the Ideal Size Set (ISS) benchmark. It’s a key measure of potential sales loss.

Q: How is “brand grading information” generated and used? **A:** “Brand grading information” is computed by the Never-Out-Of-Stock (NOOS) Gap Analysis module, based on how consistently NOOS styles meet their inventory benchmarks. This grading data is then used by the Broken vs. Non-Broken Sales Analysis module to factor brand performance into the overall sales loss calculations.

Q: Where can I access the final reports or analyzed data? **A:** The final outputs and detailed reports are typically synchronized to a designated output location or database, which can then be accessed via reporting tools or data dashboards.

Error Handling & Edge Cases

Q: What if there is no sales data for a specific product or period? **A:** The Input Validation module will typically flag this as an issue. If allowed to proceed, products without sales data for the analysis period may be excluded from certain calculations or reported with zero Rate of Sale (ROS) and no identified “broken” days.

Q: How does the system handle rapid changes in a product’s Ideal Size Set (ISS)? **A:** The analysis relies on a consistent ISS for the given period. If the ISS changes frequently or mid-period, it might impact the accuracy of the ISS Comparison module. It’s best to define a stable ISS for the analysis window or run separate analyses for periods with different ISS definitions.

Q: How are seasonal peaks or promotional periods accounted for in the Rate of Sale (ROS) calculation? **A:** While the algorithm primarily focuses on historical ROS, extreme peaks or troughs due to promotions or seasonality can impact the calculated ROS. For highly volatile products, you might need to adjust the analysis period or consider specific seasonal ROS benchmarks outside the algorithm.

Q: What if the inventory data provided to the system is inaccurate or outdated? **A:** Inaccurate or outdated inventory data will directly impact the validity of the analysis, particularly the identification of “broken” days and the calculation of “unliquidated sales quantities.” The algorithm relies on the accuracy of its inputs for meaningful results.

Q: What should I do if the analysis results appear incorrect or unexpected? **A:** 1. **Check Input Data:** Verify the accuracy and completeness of your sales, inventory, and product master data. 2. **Review Configuration:** Ensure that dates, thresholds, and any category filters are set correctly. 3. **Consult Logs:** Look at the system logs for any errors or warnings during the execution. 4. **Contact Support:** If issues persist, reach out to the support team with details of your inputs and the unexpected results.

Need More Help?

For further assistance or detailed technical inquiries, please contact the support team at [support-email@yourcompany.com] or [support-phone-number].

gap Documentation

- **Version:** 1.0
 - **Last Updated:** 2025-05-23
-

2. Executive Summary

This documentation outlines the Gap Analysis algorithm, a crucial tool for identifying and quantifying lost sales opportunities due to inventory imbalances. It helps businesses understand where they are missing sales by analyzing stockouts, sub-optimal Ideal Size Sets (ISS), and insufficient inventory for core “Never-Out-Of-Stock” (NOOS) items. By leveraging this analysis, companies can optimize their inventory depth (OD) and width (OW), improve the Ideal Size Set (ISS) for specific products, and ultimately maximize sales and profitability.

3. Architecture Overview

```
graph TD
    A[Gap Analysis Orchestration (0)] -->|orchestrates| B[Gap Analysis Input Validation (1)];
    A -->|orchestrates| D[Never-Out-Of-Stock (NOOS) Gap Analysis (4)];
    A -->|orchestrates| C[Ideal Size Set (ISS) Comparison (3)];
    A -->|orchestrates| E[Broken vs. Non-Broken Sales Analysis (2)];
    B -->|validates| D;
    B -->|validates| C;
    B -->|validates| E;
    D -->|data_flow| E;
```

- **Gap Analysis Orchestration:** The central coordinator, defining the sequence of execution for all sub-modules.
- **Gap Analysis Input Validation:** Ensures all input parameters and data are valid and available before analysis begins.
- **Broken vs. Non-Broken Sales Analysis:** Differentiates sales periods based on inventory coverage to quantify potential sales loss.
- **Ideal Size Set (ISS) Comparison:** Evaluates and optimizes the Ideal Size Set (ISS) by identifying unmet demand due to stock issues.

6. Global Configuration

The `args.properties` file manages global settings for the entire Gap Analysis process.

Setting Name | Default Value | Description | Valid Range/When to Change **NOOS Gap Analysis**

This module focuses on evaluating the availability and performance of “Never-Out-Of-Stock” (NOOS) styles, which are core products typically expected to be continuously available. It tracks how many days NOOS styles meet a specific inventory benchmark across different stores and calculates brand grading information based on this availability.

2. Required Information

Sales Data * `product_id`: To link sales to NOOS styles. * `units_sold`: To understand historical demand for NOOS styles. * `store_id`: To track availability across different retail locations.

Product Data * `style`: To identify and filter for NOOS-designated styles. * `attribute_group`: For detailed analysis of NOOS styles within specific Attribute Groups (AGs). * `product_id`: Key identifier for NOOS products.

Operational Data * `inventory_on_hand`: Current stock levels to assess NOOS availability against benchmarks. * `sale_date` (or `inventory_date`): To track availability over time. * `noosAvailabilityStart`: Configured start date for the NOOS availability tracking period. * `noosAvailabilityEnd`: Configured end date for the NOOS availability tracking period.

3. Business Settings

- `noos.min.inventory.benchmark`: The minimum inventory level (as a percentage of target Ideal Size Set (ISS) or fixed units) an item must maintain to be considered “in stock” for a NOOS day.
 - Default: 0.90 (90% of target ISS)
 - When to Change: *Adjust to reflect stricter or looser definitions of “in-stock” for NOOS items, depending on business strategy or product criticality.*
- `noos.minimum.days.in.stock`: The minimum number of days a NOOS style must meet its inventory benchmark within the tracking period to be considered successful.
 - Default: 30 days
 - When to Change: *Modify based on the desired consistency of NOOS availability. Higher values enforce stricter compliance.*

4. Process Flow

graph TD

A[Load NOOS Sales & Inventory Data] --> B[Calculate Daily Availability %];

```

B --> C{Check if Daily Availability >= Min. Benchmark};
C -- Yes --> D[Increment "Days In Stock" Counter];
C -- No --> E[Maintain Counter (or Reset)];
D --> F[Aggregate Days In Stock by NOOS Style & Store];
E --> F;
F --> G[Calculate Brand Grading Score based on % of Days In
Stock];
G --> H[Output NOOS Availability & Brand Grading];

```

5. Step-by-Step Actions

1. **Load** historical sales and current inventory data for all designated NOOS styles. *This gathers the necessary inputs for analysis.*
2. **Calculate** the daily inventory availability for each NOOS product_id at each store_id. *This determines if an item met its stock threshold on a given day.*
3. **Compare** the daily availability against the noos.min.inventory.benchmark. *This flags whether the product was sufficiently stocked.*
4. **Aggregate** the count of “days in stock” for each NOOS product_id per store_id over the defined noosAvailabilityStart to noosAvailabilityEnd period. *This provides a consistent availability metric.*
5. **Compute** a “Brand Grading Score” for each NOOS style based on the percentage of days it met the noos.minimum.days.in.stock threshold. *This quantifies the brand’s performance in maintaining core product availability.*
6. **Store** the computed NOOS availability metrics and brandGradingMap for use by subsequent modules. *This ensures data continuity in the pipeline.*

6. Sample Scenario

A core NOOS T-shirt (Style: “Basic Tee”) is expected to be in stock 90% of the time (min. inventory benchmark) over a 60-day period. For Store A, this T-shirt met the benchmark for 50 days. For Store B, it met it for 58 days. The noos.minimum.days.in.stock is set to 30.

- Store A: 50 days / 60 days = 83.3% availability.
- Store B: 58 days / 60 days = 96.7% availability.

The Brand Grading Score would reflect these percentages. Store B would receive a higher grade for “Basic Tee” than Store A, indicating better compliance with NOOS objectives.

Key Takeaway: *This module quickly identifies stores or styles failing to maintain core product availability, allowing for targeted inventory adjustments.*

7. Results

The NOOS Gap Analysis module outputs: * A detailed report of product_id and store_id availability (days in stock, percentage availability) for all NOOS styles. * The brandGradingMap, which assigns a performance score to each NOOS style or Attribute Group (AG) based on its overall availability compliance across stores. This map is stored in the shared data structure for use by other modules.

8. Unusual Cases

- **Missing or delayed data feed:** If inventory or sales data for NOOS styles is incomplete or late, the calculated availability will be inaccurate, potentially leading to false positives or negatives in gap identification.
- **Incorrect NOOS designation:** If a style is incorrectly flagged as NOOS, it will be analyzed with stricter availability expectations than warranted, distorting results.
- **Sudden spikes in demand:** Unforeseen demand surges might cause legitimate stockouts, but the module will still flag them as availability gaps, requiring manual review for context.
- **Benchmark set too high/low:** An unrealistic `noos.min.inventory.benchmark` might either continuously flag issues (too high) or miss real problems (too low).

9. Next Steps

The `brandGradingMap` generated by this module is passed to the Broken vs. Non-Broken Sales Analysis module to provide additional context for ROS calculations and lost sales estimations. It also informs inventory planners about which NOOS styles or stores require immediate attention to improve on-shelf availability.

10. Common Questions

1. **How do we define a “NOOS style”?** *NOOS styles are typically core, evergreen products expected to always be in stock due to their consistent demand and strategic importance.*
2. **What does the “Brand Grading Score” tell me?** *It quantifies how well a specific NOOS style (or brand) is maintaining its expected availability across stores, indicating areas of strength or weakness.*
3. **Why is a style flagged as “low availability” if we just had a massive sale?** *The analysis focuses on raw availability against a benchmark. While sales are a factor, the primary goal is identifying days where the product wasn’t sufficiently stocked. Contextual review is sometimes needed.*
4. **Can I adjust the inventory benchmark per NOOS style?** *Currently, a global benchmark is applied, but future enhancements could allow for style-specific targets via configuration.*
5. **How often should this analysis run?** *Ideally, daily or weekly, to provide timely insights into NOOS performance and allow for quick inventory adjustments.*

11. Quick Reference

- **Objective:** Track NOOS availability & performance.
 - **Key Inputs:** Sales, Inventory, NOOS style designations, Start/End Dates.
 - **Business Settings:** `noos.min.inventory.benchmark`, `noos.minimum.days.in.stock`.
 - **Key Output:** `brandGradingMap`, NOOS availability reports.
 - **Business Value:** Ensures core product availability, identifies availability gaps, informs inventory replenishment.
-

Ideal Size Set (ISS) Comparison

This module evaluates the effectiveness of the Ideal Size Set (ISS) by comparing historical sales and current inventory against a target. It identifies unliquidated sales quantities (demand that could not be met due to stock issues) at the Attribute Group (AG) and size level. This analysis helps optimize inventory depth (OD) and width (OW) by highlighting discrepancies between desired and actual size availability.

2. Required Information

Sales Data * `units_sold`: *Historical units sold to understand demand patterns for different sizes.* * `sale_date`: *To analyze sales trends over time relevant to ISS.*

Product Data * `size`: *Critical for identifying which sizes are driving demand and where ISS gaps exist.* * `attribute_group` (AG): *To perform ISS analysis at a granular product grouping level.* * `product_id`: *Unique identifier for products to link sales and inventory.*

Operational Data * `inventory_on_hand`: *Current inventory levels for each size to compare against sales and target ISS.* * `iss_benchmark_coverage`: *Target inventory coverage percentage that defines the ideal state for ISS.* * `issInvEndDate`: *Configured end date for the inventory data snapshot used in ISS comparison.*

3. Business Settings

- `iss.target.coverage.percent`: The target percentage of sizes within an Attribute Group (AG) that should ideally be in stock to meet Ideal Size Set (ISS) objectives.
 - Default: 0.95 (95%)
 - When to Change: *Adjust based on desired inventory coverage for complete size sets. Higher values enforce stricter size availability.*
- `iss.min.units.for.analysis`: The minimum number of units sold for an Attribute Group (AG) or style to be considered for ISS analysis.
 - Default: 100 units
 - When to Change: *Prevents analysis on low-volume items where ISS might not be critical or statistically significant.*

4. Process Flow

graph TD

```
A[Load Sales & Inventory Data by AG & Size] --> B[Calculate Historical Demand by Size];
B --> C[Determine Target Ideal Size Set (ISS) based on Demand & Benchmark];
C --> D[Compare Current Inventory vs. Target ISS];
D -- Inventory < Target --> E[Identify Unliquidated Sales (Lost Sales)];
D -- Inventory >= Target --> F[No Lost Sales];
E --> G[Aggregate Lost Sales by AG, Size, and Store];
F --> G;
G --> H[Output ISS Gaps & Potential Lost Sales];
```


5. Step-by-Step Actions

1. **Load** historical sales data and the latest inventory snapshot for all `product_ids`, organized by Attribute Group (AG) and size. *This consolidates the necessary data points.*
2. **Calculate** the Ideal Size Set (ISS) for each AG based on historical `units_sold` and the `iss.target.coverage.percent`. *This determines the optimal size mix and quantity needed.*
3. **Compare** the current `inventory_on_hand` for each `product_id` and size against the calculated ISS. *This identifies discrepancies between what is available and what should be available.*
4. **Identify** “unliquidated sales” (lost sales) for sizes where current `inventory_on_hand` is below the ISS recommendation. *This quantifies the potential revenue lost due to stockouts in specific sizes.*
5. **Aggregate** the unliquidated sales at the Attribute Group (AG) and size level. *This provides a summary of where ISS gaps are most impactful.*
6. **Generate** a report detailing ISS compliance, overstock, and unliquidated sales. *This provides actionable insights for inventory optimization.*

6. Sample Scenario

An Attribute Group (AG) “Women’s Jeans - Skinny Blue” has historical sales showing that 25% of demand is for size 28, 30% for size 30, and 20% for size 32. The `iss.target.coverage.percent` is 95%. Currently, the inventory for size 30 is 10 units, while the Ideal Size Set (ISS) suggests 50 units based on demand. For size 28, inventory is 60 units, and ISS suggests 40 units.

- Size 30: Inventory (10) < ISS (50) -> Potential lost sales of 40 units.
- Size 28: Inventory (60) > ISS (40) -> Potential overstock of 20 units.

Key Takeaway: *The module identifies specific sizes within product groups that are either understocked, leading to lost sales, or overstocked, tying up capital, allowing for precise inventory adjustments.*

7. Results

The ISS Comparison module outputs: * A comprehensive report detailing Ideal Size Set (ISS) compliance per Attribute Group (AG) and size. * Quantification of “unliquidated sales” (lost units) for each size within an AG where inventory fell short of the ISS. * Identification of overstock positions where inventory exceeds the ISS recommendations.

8. Unusual Cases

- **Missing or delayed data feed:** Incomplete sales history or out-of-date inventory snapshots will lead to an inaccurate calculation of the Ideal Size Set (ISS) and incorrect identification of gaps.
- **Seasonal shifts in demand:** If the historical sales data doesn’t adequately represent current seasonal trends, the calculated ISS might be suboptimal.
- **New product introductions:** Products with no historical sales data cannot have an ISS calculated by this method, requiring manual input or proxy data.

- **Unrealistic iss.target.coverage.percent:** Setting this too high can flag almost everything as a gap, while setting it too low can mask real issues.

9. Next Steps

The results of the ISS Comparison module directly inform merchandising and inventory teams on specific Attribute Group (AG) and size combinations that require attention. It guides adjustments to future buy plans, improving inventory depth (OD) and width (OW) to align with Ideal Size Set (ISS) recommendations.

10. Common Questions

1. **What is an “Ideal Size Set (ISS)”?** *It’s the optimal mix of sizes for a product or category that maximizes sales by ensuring popular sizes are always available while minimizing overstock in less popular ones.*
2. **How is “unliquidated sales” different from a standard stockout?** *Unliquidated sales specifically quantify the demand that could not be met due to a size being out of stock or having insufficient inventory, relative to its Ideal Size Set (ISS) benchmark.*
3. **Does this account for returns?** *The current model primarily focuses on sales and inventory. Returns are typically handled in a separate post-sales reconciliation process, though their impact on inventory is captured.*
4. **Can this module recommend a new ISS for future buys?** *Yes, by identifying the historical demand patterns and current gaps, it implicitly provides data to inform a more optimized Ideal Size Set (ISS) for future replenishment and new product introductions.*
5. **What if we don’t have enough data for an AG?** *The iss.min.units.for.analysis setting helps filter out low-volume AGs. For entirely new AGs, initial ISS would rely on forecasts or comparable products.*

11. Quick Reference

- **Objective:** Evaluate & optimize Ideal Size Set (ISS).
- **Key Inputs:** Sales data (units sold), Inventory data, Product attributes (size, AG), Target coverage.
- **Business Settings:** iss.target.coverage.percent, iss.min.units.for.analysis.
- **Key Output:** Unliquidated sales by AG/size, ISS compliance report.
- **Business Value:** Maximize sales by improving size availability, reduce overstock.

Broken vs. Non-Broken Sales Analysis

This module analyzes historical sales data to differentiate between “broken” days (when the Ideal Size Set (ISS) benchmark for inventory coverage was not met) and “non-broken” days. It calculates the Rate of Sale (ROS) for each state and quantifies the potential sales loss attributable to stockouts or insufficient inventory on broken days.

2. Required Information

Sales Data * `units_sold`: To calculate the Rate of Sale (ROS) for both broken and non-broken periods. * `sale_date`: To segment sales by day and determine broken/non-broken status.

Product Data * `attribute_group` (AG): To perform ROS analysis at a granular product grouping level. * `product_id`: Unique identifier for products. * `style`: To group products for ROS analysis, especially in conjunction with NOOS data.

Operational Data * `inventory_on_hand`: Current stock levels on specific dates to determine if a day was “broken” based on ISS coverage. * `iss_benchmark_coverage`: The percentage of ISS required to consider a day “non-broken”. * `brandGradingMap` (from NOOS module): Provides context on brand performance for NOOS styles, which can influence broken/non-broken logic or lost sales calculation accuracy. * `rosComparisonStart`: Configured start date for the sales comparison period. * `rosComparisonEnd`: Configured end date for the sales comparison period.

3. Business Settings

- `ros.min.days.for.comparison`: Minimum number of “non-broken” days required to calculate a reliable Rate of Sale (ROS) benchmark.
 - Default: 7 days
 - When to Change: *Adjust based on desired statistical significance for ROS. Higher values ensure more robust benchmarks.*
- `ros.broken.day.inventory.threshold`: The inventory coverage percentage below which a day is classified as “broken” for a particular product or Attribute Group (AG).
 - Default: 0.50 (50%)
 - When to Change: *Define the threshold for what constitutes an ‘unhealthy’ inventory day. Lower values mean fewer broken days are identified.*
- `ros.sales.lost.calculation.method`: Defines how potential lost sales are quantified (e.g., “Non-Broken ROS - Broken ROS” or “Target ROS - Actual Broken ROS”).
 - Default: `non_broken_diff` (Non-Broken ROS - Actual Broken ROS)
 - When to Change: *Choose the method that best reflects the business’s approach to quantifying lost opportunities.*

4. Process Flow

graph TD

```
A[Load Sales & Inventory Data for Comparison Period] -->
B{Categorize Each Day as "Broken" or "Non-Broken" based on
Inventory Threshold};
B --> C[Calculate Rate of Sale (ROS) for "Non-Broken" Days];
C --> D[Calculate Rate of Sale (ROS) for "Broken" Days];
D --> E{Compare "Non-Broken ROS" vs. "Broken ROS"};
E --> F[Estimate Potential Lost Sales based on ROS
Difference];
F --> G[Output Lost Sales & ROS Benchmarks];
```

5. Step-by-Step Actions

1. **Load** historical `units_sold` and `inventory_on_hand` data for the `rosComparisonStart` to `rosComparisonEnd` period. *This gathers all necessary data points.*
2. **Categorize** each day for every `product_id` as either “broken” or “non-broken” based on whether its `inventory_on_hand` met the `ros.broken.day.inventory.threshold`. *This segments the sales data for comparative analysis.*
3. **Calculate** the average Rate of Sale (ROS) for all “non-broken” days for each Attribute Group (AG) and `product_id`. *This establishes a benchmark for expected sales velocity when inventory is adequate.*
4. **Calculate** the actual Rate of Sale (ROS) for “broken” days. *This reflects the suppressed sales velocity when inventory is insufficient.*
5. **Quantify** potential lost sales by comparing the “non-broken ROS” to the “broken ROS” over the duration of the broken days, using the `ros.sales.lost.calculation.method`. *This provides a monetary or unit value to the inventory gap.*
6. **Incorporate** `brandGradingMap` from the NOOS module (if available) to refine or contextualize the lost sales estimation, particularly for core products. *This adds an additional layer of accuracy and business relevance.*
7. **Generate** a comprehensive report detailing the ROS for both states and the calculated lost sales. *This provides actionable insights.*

6. Sample Scenario

An Attribute Group (AG) “Men’s Casual Shirts” has 10 days of sales data. 7 days were “non-broken” (`inventory > 50% Ideal Size Set`), with an average ROS of 10 units/day. 3 days were “broken” (`inventory <= 50% Ideal Size Set`), with an average ROS of 4 units/day.

Using the `non_broken_diff` method: * Potential lost sales per broken day = (Non-Broken ROS - Broken ROS) = (10 - 4) = 6 units/day. * Total potential lost sales for the 3 broken days = 6 units/day * 3 days = 18 units.

Key Takeaway: *This analysis directly quantifies the impact of inventory shortfalls on sales, providing a clear business case for improving stock levels.*

7. Results

The Broken vs. Non-Broken Sales Analysis module outputs: * Daily classification of `product_ids` or Attribute Groups (AGs) as “broken” or “non-broken” for the analysis period. * Average Rate of Sale (ROS) for “non-broken” periods, serving as a benchmark. * Average Rate of Sale (ROS) for “broken” periods, reflecting suppressed demand. * A quantified estimate of potential lost sales (in units and/or revenue) due to insufficient inventory on “broken” days.

8. Unusual Cases

- **Missing or delayed data feed:** Incomplete sales or inventory data will directly skew ROS calculations and the classification of broken/non-broken days, leading to inaccurate lost sales estimates.

- **Insufficient “non-broken” days:** If `ros.min.days.for.comparison` is not met, a reliable “non-broken” ROS benchmark cannot be established, impacting lost sales calculations.
- **External factors affecting sales:** Promotions, holidays, or competitors’ actions can influence ROS independently of inventory, requiring careful interpretation of “broken” day ROS.
- **Products with very low ROS:** For slow-moving items, distinguishing between genuine stockouts and naturally low sales can be challenging.
- **Rapid inventory fluctuations:** Highly volatile inventory (e.g., due to frequent replenishments or immediate sell-through) can make daily broken/non-broken classification less stable.

9. Next Steps

The insights from the Broken vs. Non-Broken Sales Analysis directly inform inventory planning, merchandising, and financial forecasting. They highlight specific Attribute Groups (AGs) or `product_ids` where improving inventory depth (OD) or Ideal Size Set (ISS) would yield significant sales recovery. This data is critical for building business cases for inventory investments.

10. Common Questions

1. **What is a “broken day”?** *A day is considered “broken” if the inventory level for a product or Attribute Group falls below a specified minimum threshold, indicating insufficient stock to meet potential demand.*
2. **How is “Rate of Sale (ROS)” calculated here?** *It’s the average units sold per day for a specific product or Attribute Group, separated into periods where inventory was sufficient (“non-broken”) and periods where it was not (“broken”).*
3. **Can this help prioritize replenishment?** *Absolutely. Products with high “non-broken” ROS but also significant “broken” days and lost sales are prime candidates for immediate replenishment focus.*
4. **What if a product never has a “non-broken” day?** *This indicates a persistent stock issue. The module will attempt to use a global benchmark or flag this as an outlier for manual review, as a reliable non-broken ROS cannot be determined.*
5. **Does this account for future demand?** *This module analyzes historical data. While it provides a benchmark ROS, integrating future demand forecasts would be part of a subsequent inventory optimization step.*

11. Quick Reference

- **Objective:** Quantify sales loss due to inventory gaps.
 - **Key Inputs:** Sales data (units sold), Inventory data, Product attributes (AG), `brandGradingMap`.
 - **Business Settings:** `ros.min.days.for.comparison`, `ros.broken.day.inventory.threshold`, `ros.sales.lost.calculation.method`.
 - **Key Output:** Lost sales in units/revenue, ROS benchmarks.
 - **Business Value:** Highlights direct financial impact of inventory issues, prioritizes stock investments.
-

Gap Analysis Input Validation

This module is responsible for validating the input parameters and data availability required for the Gap Analysis. It ensures that dates are logical and critical inventory data exists before deeper analysis, preventing errors downstream.

2. Required Information

Sales Data * `units_sold`: *To confirm presence and format.* * `sale_date`: *To validate date ranges and ensure chronological consistency.*

Product Data * `product_id`: *To verify data integrity and uniqueness.* * `style`: *To ensure style data is available for classification.* * `attribute_group`: *To check for consistency and completeness.*

Operational Data * `inventory_on_hand`: *Crucial for all analysis, checked for presence and non-negativity.* * `issInvEndDate`: *Date for Ideal Size Set (ISS) inventory snapshot, validated for logical range.* * `noosAvailabilityStart`: *Start date for Never-Out-Of-Stock (NOOS) analysis, validated.* * `noosAvailabilityEnd`: *End date for Never-Out-Of-Stock (NOOS) analysis, validated.* * `rosComparisonStart`: *Start date for Rate of Sale (ROS) analysis, validated.* * `rosComparisonEnd`: *End date for Rate of Sale (ROS) analysis, validated.*

3. Business Settings

- `enable.validation`: A boolean flag to enable or disable the entire input validation process.
 - Default: `true`
 - When to Change: *Set to `false` only during development or debugging, but should always be `true` in production to ensure data quality.*
- `strict.date.validation`: A boolean flag to enforce strict chronological order and non-overlap for date ranges across modules.
 - Default: `true`
 - When to Change: *Set to `false` if overlapping or non-sequential dates are intentionally allowed for specific analysis scenarios, but use with caution.*

4. Process Flow

graph TD

A[Receive Input Parameters & Data Feeds] --> B[Validate Date Ranges (e.g., Start < End)];

B -- Valid --> C[Check for Critical Data File Presence & Readability];

C -- Present & Readable --> D[Validate Data Content (e.g., Non-negative Inventory, Valid IDs)];

D -- Valid --> E[Signal Validation Success];

B -- Invalid --> F[Abort with Error];

C -- Missing/Unreadable --> F;

D -- Invalid --> F;

5. Step-by-Step Actions

1. **Receive** all input parameters and data file paths specified in `args.properties`. *This is the initial step for any analysis run.*
2. **Validate** all date parameters (`issInvEndDate`, `noosAvailabilityStart`, `noosAvailabilityEnd`, `rosComparisonStart`, `rosComparisonEnd`) to ensure they are in a valid format and that start dates precede end dates. *This prevents logical errors in time-series analysis.*
3. **Check** for the existence and readability of all specified input data files (e.g., sales history, current inventory). *This confirms that the necessary raw data is accessible.*
4. **Perform** schema validation on `common-data.json`:
 - **Verify** that all required fields are present and conform to their expected Type. *Ensures data consistency across the pipeline.*
 - **Check** for data integrity within loaded files (e.g., `inventory_on_hand` is non-negative, `units_sold` are positive, `product_ids` are not null). *This catches common data quality issues early.*
5. **Signal** success or **Throw** an informative error if any validation fails. *This ensures that subsequent analytical modules only run with high-quality, validated inputs.*

6. Sample Scenario

The `args.properties` file has `rosComparisonStart=2025-01-01` and `rosComparisonEnd=2024-12-31`. The inventory data file specified is `inventory.csv`, but it does not exist.

1. **Date Validation:** The module identifies that `rosComparisonStart` (2025-01-01) is *after* `rosComparisonEnd` (2024-12-31). *This is an invalid date range.*
2. **File Presence Check:** The module attempts to access `inventory.csv` and finds it missing. *This is a critical data unavailability.*

The module would immediately terminate with an error message like “Invalid date range for ROS comparison. Start date must be before end date.” and “Critical input file ‘inventory.csv’ not found. Please ensure it exists and is accessible.”

Key Takeaway: *Input validation is a crucial gatekeeper, preventing resource-intensive analytical modules from running on bad data and producing meaningless results.*

7. Results

The Input Validation module does not produce direct business results but outputs: * A success signal, allowing the pipeline to proceed. * Detailed error logs and program termination if validation fails, indicating specific issues with input parameters or data.

8. Unusual Cases

- **Missing or delayed data feed:** This module specifically handles this by checking file existence and readability.

- **Corrupted data files:** If a data file exists but is unparseable (e.g., malformed CSV, corrupted JSON), the validation will flag it as unreadable or fail during content validation.
- **Logical inconsistencies (beyond dates):** For example, `inventory_on_hand` values being negative, or `units_sold` being zero for a period expected to have sales.
- **Schema drift:** If the input data schema changes unexpectedly and no longer matches the `common-data.json` definition, validation will fail.

9. Next Steps

If validation is successful, the Gap Analysis Orchestration module proceeds to execute the core analytical modules: NOOS Gap Analysis, Ideal Size Set (ISS) Comparison, and Broken vs. Non-Broken Sales Analysis. If validation fails, the program terminates with an error, requiring correction of inputs.

10. Common Questions

1. **Why did the program stop at the beginning?** *It means the input validation module found an issue with your configuration settings (like dates) or critical data files, preventing further analysis to ensure data quality.*
2. **What kind of data issues does this module check for?** *It checks for correct date ranges, file existence, basic data types, and non-negative values for quantities like inventory and sales.*
3. **Can I skip validation?** *While there's an `enable.validation` flag, it's strongly advised not to disable validation in a production environment as it risks running on bad data.*
4. **My data is “clean”, why is it still failing?** *It might be a subtle logical inconsistency (e.g., a start date after an end date) or a file path issue. Check the error message carefully.*
5. **Does this module check for data completeness (e.g., all SKUs present)?** *It primarily checks for data presence* and format. While it validates for non-null IDs, comprehensive checks for all expected SKUs or data points are typically done in a separate data profiling step or implicitly by downstream modules if they encounter missing data.**

11. Quick Reference

- **Objective:** Ensure data quality & parameter validity before analysis.
- **Key Inputs:** All `args.properties` parameters, paths to input data files.
- **Business Settings:** `enable.validation`, `strict.date.validation`.
- **Key Output:** Success signal or detailed error log.
- **Business Value:** Prevents costly reprocessing, ensures reliable analysis results.

Gap Analysis Orchestration

This module acts as the orchestrator for the entire Gap Analysis process. It defines the sequence of execution for sub-modules, including initial data snapshotting and final output synchronization, ensuring that all components of the gap analysis are run in the correct order.

2. Required Information

Sales Data * *No direct sales data required by the orchestrator, but it ensures its availability for downstream modules.*

Product Data * *No direct product data required by the orchestrator, but it ensures its availability for downstream modules.*

Operational Data * `args.properties`: *Global configuration to determine which modules to run and their parameters.* * `common-data.json`: *Schema for shared data structures that modules will pass to each other.*

3. Business Settings

- `run.iss.comparison`: A boolean flag to enable or disable the Ideal Size Set (ISS) Comparison module.
 - Default: `true`
 - When to Change: *Set to `false` if ISS analysis is not required for a particular run, perhaps to reduce processing time or focus on other gaps.*
- `run.noos.analysis`: A boolean flag to enable or disable the Never-Out-Of-Stock (NOOS) Gap Analysis module.
 - Default: `true`
 - When to Change: *Set to `false` if NOOS performance is not a current focus.*
- `run.broken.vs.non.broken.analysis`: A boolean flag to enable or disable the Broken vs. Non-Broken Sales Analysis module.
 - Default: `true`
 - When to Change: *Set to `false` if quantifying lost sales is not required for a particular run.*
- `output.synchronization.enabled`: A boolean flag to enable/disable the final output synchronization step, where all results are consolidated.
 - Default: `true`
 - When to Change: *Set to `false` only during development if you need to inspect intermediate outputs before final consolidation, but always `true` in production.*

4. Process Flow

graph TD

```
A[Initialize Application] --> B[Load Global Configuration
(args.properties)];
B --> C[Perform Data Snapshotting (e.g., initial load)];
C --> D[Run Gap Analysis Input Validation];
D -- Success --> E[Run Never-Out-Of-Stock (NOOS) Gap Analysis
(if enabled)];
E --> F[Run Ideal Size Set (ISS) Comparison (if enabled)];
F --> G[Run Broken vs. Non-Broken Sales Analysis (if
enabled)];
G --> H[Synchronize & Consolidate All Outputs];
D -- Failure --> I[Exit with Error];
```

5. Step-by-Step Actions

1. **Initialize** the application environment and **Load** global configuration parameters from `args.properties`. *This sets up the overall execution context.*
2. **Trigger** initial data snapshotting or loading of raw input data into memory or a temporary store. *This makes data available for all subsequent modules.*
3. **Invoke** the Gap Analysis Input Validation module to ensure all parameters and data are valid. *This acts as a critical gate, preventing faulty runs.*
4. If validation is successful, **Execute** the Never-Out-Of-Stock (NOOS) Gap Analysis module (if `run.noos.analysis` is true). *This calculates NOOS availability and brand grading.*
5. **Execute** the Ideal Size Set (ISS) Comparison module (if `run.iss.comparison` is true). *This identifies ISS compliance and unliquidated sales.*
6. **Execute** the Broken vs. Non-Broken Sales Analysis module (if `run.broken.vs.non.broken.analysis` is true). *This quantifies lost sales based on inventory status.*
7. **Synchronize** and **Consolidate** all generated outputs from the individual modules into final reports in the `outputs/` directory. *This provides a unified view of all gap analysis findings.*
8. **Handle** errors from any module by logging the issue and gracefully terminating the process. *Ensures system stability.*

6. Sample Scenario

A user runs the gap analysis. The `args.properties` file has `run.iss.comparison=true`, `run.noos.analysis=true`, and `run.broken.vs.non.broken.analysis=false`.

1. The orchestrator starts, loads configs.
2. Input Validation runs successfully.
3. NOOS Gap Analysis module executes, producing its results.
4. ISS Comparison module executes, producing its results.
5. Broken vs. Non-Broken Sales Analysis *does not* execute because its flag is set to false.
6. Outputs from NOOS and ISS modules are synchronized and saved.

Key Takeaway: *The orchestrator provides flexibility by allowing users to enable or disable specific analytical components, tailoring the analysis to immediate business needs.*

7. Results

The Orchestration module coordinates the execution of all sub-modules and ensures their outputs are collected and synchronized. Its direct “results” are: * Successful completion of the entire Gap Analysis pipeline. * Consolidated final reports in the `outputs/` folder, containing insights from all enabled sub-modules. * Error messages and early termination if critical issues are detected during validation or execution of any sub-module.

8. Unusual Cases

- **Missing or delayed data feed:** Handled by the Input Validation module, which prevents the orchestrator from proceeding.
- **One sub-module fails:** The orchestrator is designed to catch exceptions from individual modules, log the error, and terminate the entire process to prevent partial or erroneous results.
- **Configuration conflicts:** If `args.properties` has conflicting settings (e.g., `issInvEndDate` for ISS is outside the `rosComparisonStart/End` for ROS), the Input Validation module (orchestrated early) will flag this.
- **Resource exhaustion:** Large data volumes might lead to out-of-memory errors or slow performance. The orchestrator may log such events, but tuning usually happens at the JVM or data processing layer.

9. Next Steps

The generated reports in the `outputs/` folder are ready for business review, deeper analysis, and integration into other planning systems (e.g., for inventory replenishment, merchandising adjustments, or financial forecasting).

10. Common Questions

1. **How do I run the entire gap analysis?** *You execute the main class associated with this Gap Analysis Orchestration module (e.g., via a Java `-jar` command).*
2. **Can I run only specific parts of the analysis?** *Yes, by adjusting the `run.iss.comparison`, `run.noos.analysis`, and `run.broken.vs.non.broken.analysis` flags in `args.properties`.*
3. **What happens if one part of the analysis fails?** *The orchestrator is designed to stop the entire process and log the error, ensuring you don't get incomplete or misleading results.*
4. **Where do I find the final reports?** *All consolidated results are written to the `outputs/` folder after the entire pipeline completes successfully.*
5. **How can I make the analysis faster?** *Consider disabling modules that aren't immediately critical via their respective run flags, or ensure sufficient system resources (RAM, CPU) are allocated.*

11. Quick Reference

- **Objective:** Coordinate & execute all gap analysis modules.
 - **Key Inputs:** `args.properties`, `common-data.json`.
 - **Business Settings:** `run.iss.comparison`, `run.noos.analysis`, `run.broken.vs.non.broken.analysis`, `output.synchronization.enabled`.
 - **Key Output:** Consolidated gap analysis reports.
 - **Business Value:** Streamlines complex analysis, ensures correct workflow, provides flexibility.
-

9. Appendix

Full Parameter Reference

Setting Name | Default Value | Description | Valid Range/When to Change
NO_BUSINESS_DEFINITIONS_PROVIDED:

This is a standard technical document for the `gap` project, version `1.0`, last updated on `2025-05-23`.

1. Title & Metadata

* ****Title****: gap Documentation
* ****Version****: 1.0
* ****Last Updated****: 2025-05-23

2. Executive Summary

This documentation details the `gap` algorithm, a sophisticated solution designed to identify and quantify sales gaps stemming from suboptimal inventory strategies. It provides critical insights into issues such as stockouts, inefficient Ideal Size Set (ISS) allocations, and insufficient availability of "Never-Out-Of-Stock" (N00S) items. By understanding these gaps, businesses can optimize inventory depth (OD) and width (OW), refine their Ideal Size Sets (ISS), and enhance their Rate of Sale (ROS) to maximize revenue and operational efficiency.

3. Architecture Overview

```
```mermaid
graph TD
 A[Gap Analysis Orchestration (0)] -->|orchestrates| B[Gap Analysis Input Validation (1)];
 A -->|orchestrates| D[Never-Out-Of-Stock (N00S) Gap Analysis (4)];
 A -->|orchestrates| C[Ideal Size Set (ISS) Comparison (3)];
 A -->|orchestrates| E[Broken vs. Non-Broken Sales Analysis (2)];
 B -->|validates| D;
 B -->|validates| C;
 B -->|validates| E;
 D -->|data_flow| E;
```

- **Gap Analysis Orchestration:** This module acts as the orchestrator for the entire Gap Analysis process. It defines the sequence of execution for sub-modules,

including initial data snapshotting and final output synchronization, ensuring that all components of the gap analysis are run in the correct order.

- **Gap Analysis Input Validation:** This module is responsible for validating the input parameters and data availability required for the Gap Analysis. It ensures that dates are logical and critical inventory data exists before deeper analysis, preventing errors downstream.
  - **Broken vs. Non-Broken Sales Analysis:** This module analyzes historical sales data to differentiate between “broken” days (when the Ideal Size Set (ISS) benchmark for inventory coverage was not met) and “non-broken” days. It calculates the Rate of Sale (ROS) for each state and quantifies the potential sales loss attributable to stockouts or insufficient inventory on broken days.
  - **Ideal Size Set (ISS) Comparison:** This module evaluates the effectiveness of the Ideal Size Set (ISS) by comparing historical sales and current inventory against a target. It identifies unliquidated sales quantities (demand that could not be met due to stock issues) at the Attribute Group (AG) and size level. This analysis helps optimize inventory depth (OD) and width (OW) by highlighting discrepancies between desired and actual size availability.
  - **Never-Out-Of-Stock (NOOS) Gap Analysis:** This module focuses on “Never-Out-Of-Stock” (NOOS) styles to assess their availability and performance. It tracks the number of days NOOS styles meet a specified inventory benchmark across stores and computes brand grading information. This analysis highlights potential gaps in maintaining core product availability and provides insights into brand performance.
- 

## 4. Prerequisites & Setup

To run the gap analysis algorithm, ensure the following are in place:

- **Java Version:** Java 11 or higher.
- **Build Tool:** Apache Maven (version 3.6.0+) or Gradle (version 7.0+).
- **Dependencies:** All project dependencies are managed via the `pom.xml` (for Maven) or `build.gradle` (for Gradle) file. Ensure you have run `mvn clean install` or `gradle build` to download and install them.
- **Running the ModuleGroup Orchestrator:** The Gap Analysis Orchestration module serves as the primary entry point for the entire pipeline. To execute the analysis, compile the project and run the main class associated with the Gap Analysis Orchestration module. For example, using Maven:

```
mvn clean install
java -jar target/gap-1.0.jar # Replace 'gap-1.0.jar' with your
 actual JAR name
```

Ensure `args.properties` and `common-data.json` are correctly configured and located in the expected directory (root of the project).

---

# 5. Folder & File Structure

```
gap/
├─ module-group/ # Contains the Gap Analysis
Orchestration module responsible for workflow.
├─ submodules/ # Houses individual analytical sub-
modules (e.g., ISS, N00S, ROS analysis).
├─ args.properties # Global configuration parameters for
the entire gap analysis pipeline.
├─ common-data.json # Defines the schema for shared data
objects across modules.
├─ outputs/ # Stores the final analysis reports
and intermediate output files.
```

---

# 6. Global Configuration

The args.properties file manages global settings for the entire Gap Analysis process.

Setting Name | Default Value | Description | Valid Range/When to Change  
NO\_BUSINESS\_DEFINITIONS\_PROVIDED:

# gap Documentation

```
* **Version**: 1.0
* **Last Updated**: 2025-05-23
```

---

# 2. Executive Summary

This documentation details the `gap` algorithm, a sophisticated solution designed to identify and quantify sales gaps stemming from suboptimal inventory strategies. It provides critical insights into issues such as stockouts, inefficient Ideal Size Set (ISS) allocations, and insufficient availability of "Never-Out-Of-Stock" (N00S) items. By understanding these gaps, businesses can optimize inventory depth (OD) and width (OW), refine their Ideal Size Sets (ISS), and enhance their Rate of Sale (ROS) to maximize revenue and operational efficiency.

---

# 3. Architecture Overview

```
```mermaid
graph TD
    A[Gap Analysis Orchestration (0)] -->|orchestrates| B[Gap
Analysis Input Validation (1)];
```

```

A -->|orchestrates| D[Never-Out-Of-Stock (NOOS) Gap Analysis
    (4)];
A -->|orchestrates| C[Ideal Size Set (ISS) Comparison (3)];
A -->|orchestrates| E[Broken vs. Non-Broken Sales Analysis
    (2)];
B -->|validates| D;
B -->|validates| C;
B -->|validates| E;
D -->|data_flow| E;

```

- **Gap Analysis Orchestration:** This module acts as the orchestrator for the entire Gap Analysis process. It defines the sequence of execution for sub-modules, including initial data snapshotting and final output synchronization, ensuring that all components of the gap analysis are run in the correct order.
 - **Gap Analysis Input Validation:** This module is responsible for validating the input parameters and data availability required for the Gap Analysis. It ensures that dates are logical and critical inventory data exists before deeper analysis, preventing errors downstream.
 - **Broken vs. Non-Broken Sales Analysis:** This module analyzes historical sales data to differentiate between “broken” days (when the Ideal Size Set (ISS) benchmark for inventory coverage was not met) and “non-broken” days. It calculates the Rate of Sale (ROS) for each state and quantifies the potential sales loss attributable to stockouts or insufficient inventory on broken days.
 - **Ideal Size Set (ISS) Comparison:** This module evaluates the effectiveness of the Ideal Size Set (ISS) by comparing historical sales and current inventory against a target. It identifies unliquidated sales quantities (demand that could not be met due to stock issues) at the Attribute Group (AG) and size level. This analysis helps optimize inventory depth (OD) and width (OW) by highlighting discrepancies between desired and actual size availability.
 - **Never-Out-Of-Stock (NOOS) Gap Analysis:** This module focuses on “Never-Out-Of-Stock” (NOOS) styles to assess their availability and performance. It tracks the number of days NOOS styles meet a specified inventory benchmark across stores and computes brand grading information. This analysis highlights potential gaps in maintaining core product availability and provides insights into brand performance.
-

4. Prerequisites & Setup

To run the gap analysis algorithm, ensure the following are in place:

- **Java Version:** Java 11 or higher.
- **Build Tool:** Apache Maven (version 3.6.0+) or Gradle (version 7.0+).
- **Dependencies:** All project dependencies are managed via the `pom.xml` (for Maven) or `build.gradle` (for Gradle) file. Ensure you have run `mvn clean install` or `gradle build` to download and install them.

- **Running the ModuleGroup Orchestrator:** The Gap Analysis Orchestration module serves as the primary entry point for the entire pipeline. To execute the analysis, compile the project and run the main class associated with the Gap Analysis Orchestration module. For example, using Maven:

```
mvn clean install
java -jar target/gap-1.0.jar # Replace 'gap-1.0.jar' with your
                             actual JAR name
```

Ensure args.properties and common-data.json are correctly configured and located in the expected directory (root of the project).

5. Folder & File Structure

```
gap/
├─ module-group/           # Contains the Gap Analysis
                             Orchestration module responsible for workflow.
├─ submodules/             # Houses individual analytical sub-
                             modules (e.g., ISS, NOOS, ROS analysis).
├─ args.properties         # Global configuration parameters for
                             the entire gap analysis pipeline.
├─ common-data.json        # Defines the schema for shared data
                             objects across modules.
├─ outputs/               # Stores the final analysis reports
                             and intermediate output files.
```

6. Global Configuration

The args.properties file manages global settings for the entire Gap Analysis process.

Setting Name	Default Value	Description	Valid Range/When Change
input.data.path	data/input/	File path to the directory containing all raw input data files.	A valid and accessible local or network directory path. Change if input data is stored elsewhere.
output.data.path	outputs/	File path to the directory where all final and intermediate results are saved.	A valid and accessible local or network directory path. Ensure write permissions.

Setting Name	Default Value	Description	Valid Range/When to Change
analysis.start.date	2024-01-01	The overall start date for data ingestion and analysis across all modules.	Must be in YYYY-MM-DD format. Set to cover the desired historical period for sales and inventory data.
analysis.end.date	2024-12-31	The overall end date for data ingestion and analysis across all modules.	Must be in YYYY-MM-DD format and after analysis.start.date. Should reflect the latest available data.
min.sales.velocity.threshold	5.0	Minimum daily Rate of Sale (ROS) for a product to be considered for detailed gap analysis.	Any positive float. Products with ROS below this will be excluded from specific granular analysis to focus on impactful items.
inventory.coverage.non_broken	0.80	Percentage of ideal inventory coverage to classify a day as “non-broken”.	Range: 0.0 - 1.0. Higher values enforce stricter definition of adequate inventory, affecting “broken” day classification.
noos.target.days.available	60	Minimum number of days a NOOS style must meet its inventory benchmark.	Any positive integer. Determines the compliance standard for core NOOS items.
enable.validation	true	Boolean flag to enable or disable the entire input validation process.	true (recommended for production) or false (for debugging).
log.level	INFO	Sets the logging verbosity for the application.	ERROR, WARN, INFO, DEBUG, TRACE. DEBUG and TRACE for detailed debugging, INFO for standard operations.
run.iss.comparison	true	Boolean flag to enable or	true or false. Set to false to skip ISS

Setting Name	Default Value	Description	Valid Range/When to Change
		disable the Ideal Size Set (ISS) Comparison module.	analysis, reducing processing time.
run.noos.analysis	true	Boolean flag to enable or disable the Never-Out-Of-Stock (NOOS) Gap Analysis module.	true or false. Set false to skip NOC analysis.
run.broken.vs.non.broken.analysis	true	Boolean flag to enable or disable the Broken vs. Non-Broken Sales Analysis module.	true or false. Set false to skip lost sales quantification.
output.synchronization.enabled	true	Boolean flag to enable/disable the final output consolidation step.	true or false. Should be true for final reports.
ros.min.days.for.comparison	7	Minimum number of “non-broken” days required for a reliable ROS benchmark calculation.	Any positive integer. Ensures statistical significance for the ROS of Sale (ROS) based on
ros.broken.day.inventory.threshold	0.50	Inventory coverage percentage below which a day is classified as “broken”.	Range: 0.0 - 1.0. Lower values mean fewer broken days are identified, impacting lost sales calculation
ros.sales.lost.calculation.method	non_broken_diff	Defines how potential lost sales are quantified.	non_broken_diff (Non-Broken ROS - Actual Broken ROS), target_ros_base (Target ROS - Actual ROS on broken days). Choose based on

Setting Name	Default Value	Description	Valid Range/When to Change
			business accounting preference.
iss.target.coverage.percent	0.95	Target percentage of sizes within an AG that should be in stock for ISS.	Range: 0.0 - 1.0. Higher values enforce stricter size availability, affecting Ideal Size (ISS) gap identification.
iss.min.units.for.analysis	100	Minimum units sold for an AG or style to be considered for ISS analysis.	Any positive integer. Prevents analysis of low-volume items where ISS might not be critical.
strict.date.validation	true	Boolean flag to enforce strict chronological order for date ranges.	true (recommended) or false.
noos.min.inventory.benchmark	0.90	Minimum inventory level (as % of target ISS or fixed units) for a NOOS item to be “in stock”.	Range: 0.0 - 1.0 (as percentage). Defines threshold for NOOS availability.

7. Shared Data Model

The `common-data.json` schema defines the standardized structure for data objects used and exchanged across different modules within the gap analysis pipeline.

Field	Type	Description
product_id	string	Unique identifier for a product, typically a SKU.
category	string	The product category (e.g., “Apparel”, “Footwear”, “Accessories”).
style	string	The product style or collection (e.g., “Basic Tee”, “Skinny Jeans”).

Field	Type	Description
size	string	Specific size of the product (e.g., “S”, “M”, “L”, “10”, “32x30”).
attribute_group	string	An Attribute Group (AG) representing a set of product attributes grouped for analysis (e.g., “Color: Blue, Material: Cotton”).
sale_date	string (date)	The date on which the sale occurred or inventory was recorded (YYYY-MM-DD).
units_sold	integer	Number of units sold on a given sale_date.
revenue	float	Revenue generated from the sale transaction.
inventory_on_hand	integer	Number of units of a product available at a specific sale_date (or inventory_date).
store_id	string	Identifier for the retail store or sales channel.
iss_benchmark_coverage	float	The target percentage of inventory coverage for a given Ideal Size Set (ISS).
ros_daily	float	Calculated Rate of Sale (ROS) for a product or Attribute Group on a given day (units/day).
broken_day_flag	boolean	true if the day’s inventory for a product or AG was insufficient (below threshold), false otherwise.
potential_lost_sales_units	integer	Estimated units of sales lost due to insufficient inventory.
potential_lost_sales_revenue	float	Estimated revenue lost due to insufficient inventory.
brand_grading_score	float	A score indicating brand performance for NOOS styles based on availability compliance.
noos_designation_flag	boolean	true if the product style is designated as a Never-Out-Of-Stock (NOOS) item, false otherwise.

8. Submodule Chapters

Gap Analysis Orchestration

1. Module Name & Business Objective

Module Name: Gap Analysis Orchestration

Business Objective: To streamline and manage the entire Gap Analysis workflow, ensuring that all analytical components—from data validation to final output consolidation—are executed in the correct sequence and with appropriate configuration, providing a comprehensive view of sales opportunities and inventory gaps.

2. Required Information

Sales Data * *No direct sales data required by the orchestrator. Its role is to ensure downstream modules have access to this data.*

Product Data * *No direct product data required by the orchestrator. Its role is to ensure downstream modules have access to this data.*

Operational Data * *args.properties: Critical for defining the pipeline's overall configuration, enabling/disabling modules, and setting global parameters. * common-data.json: The schema defining the structure of shared data objects, ensuring consistency and compatibility between modules.*

3. Business Settings

- `run.iss.comparison`: A boolean flag to enable or disable the Ideal Size Set (ISS) Comparison module.
 - Default: `true`
 - When to Change: *Set to `false` if ISS analysis is not required for a particular run, perhaps to reduce processing time or focus on other types of gaps.*
- `run.noos.analysis`: A boolean flag to enable or disable the Never-Out-Of-Stock (NOOS) Gap Analysis module.
 - Default: `true`
 - When to Change: *Set to `false` if NOOS performance is not a current focus, to streamline the analysis.*
- `run.broken.vs.non.broken.analysis`: A boolean flag to enable or disable the Broken vs. Non-Broken Sales Analysis module.
 - Default: `true`
 - When to Change: *Set to `false` if quantifying potential lost sales is not required for a specific analysis run.*
- `output.synchronization.enabled`: A boolean flag to enable/disable the final output synchronization step, where all results are consolidated.
 - Default: `true`
 - When to Change: *Set to `false` only during development or debugging if you need to inspect intermediate outputs before final consolidation. It should always be `true` in production to ensure all results are collected.*

4. Process Flow

graph TD

```
A[Initialize Application] --> B[Load Global Configuration
(args.properties)];
B --> C[Perform Data Snapshotting (e.g., initial load)];
C --> D[Run Gap Analysis Input Validation];
D -- Success --> E{Run Never-Out-Of-Stock (NOOS) Gap Analysis
(if enabled)};
E --> F{Run Ideal Size Set (ISS) Comparison (if enabled)};
F --> G{Run Broken vs. Non-Broken Sales Analysis (if
enabled)};
G --> H[Synchronize & Consolidate All Outputs];
D -- Failure --> I[Exit with Error];
```

5. Step-by-Step Actions

1. **Initialize** the application environment and **Load** global configuration parameters from `args.properties`. *This sets up the overall execution context for the pipeline.*
2. **Trigger** initial data snapshotting or loading of raw input data into memory or a temporary data store. *This makes the necessary sales, product, and inventory data available for all subsequent analytical modules.*
3. **Invoke** the Gap Analysis Input Validation module to thoroughly check all specified parameters and data inputs for validity and completeness. *This acts as a critical gate, preventing the entire pipeline from running on bad data and ensuring robust results.*
4. If validation is successful, **Execute** the Never-Out-Of-Stock (NOOS) Gap Analysis module, but only if the `run.noos.analysis` flag in `args.properties` is set to `true`. *This calculates NOOS availability and brand grading, passing its results to the shared data context.*
5. Following the NOOS analysis, **Execute** the Ideal Size Set (ISS) Comparison module, provided `run.iss.comparison` is `true`. *This identifies Ideal Size Set (ISS) compliance issues and quantifies unliquidated sales, also updating the shared data context.*
6. Finally, **Execute** the Broken vs. Non-Broken Sales Analysis module, if `run.broken.vs.non.broken.analysis` is `true`. *This quantifies potential lost sales based on inventory status (broken vs. non-broken days), leveraging data from preceding modules.*
7. **Synchronize** and **Consolidate** all generated outputs from the individual modules into final, coherent reports. These reports are then saved to the `outputs/` directory. *This provides a unified and easy-to-access view of all gap analysis findings.*
8. **Handle** any errors or exceptions that occur during the execution of any sub-module by logging the issue and gracefully terminating the process. *This ensures system stability and prevents the generation of partial or erroneous results.*

6. Sample Scenario

A business stakeholder wants to understand the impact of inventory gaps on core products (NOOS) and the effectiveness of their Ideal Size Sets (ISS), but they are not currently focused on the broader “broken vs. non-broken” sales analysis for all products. They would configure `args.properties` as follows:

- `run.iss.comparison=true`
- `run.noos.analysis=true`
- `run.broken.vs.non.broken.analysis=false`
- All other settings remain at their defaults or are set appropriately for the data paths and analysis dates.

When the Gap Analysis Orchestration module is executed:

1. The orchestrator initializes and loads these configurations.
2. The Gap Analysis Input Validation module runs first and, assuming data is valid, signals success.
3. The Never-Out-Of-Stock (NOOS) Gap Analysis module executes, generating insights into NOOS availability and brand performance.
4. The Ideal Size Set (ISS) Comparison module then executes, analyzing size-level performance and unliquidated sales.
5. Crucially, the Broken vs. Non-Broken Sales Analysis module *is skipped entirely* because its run flag is set to false.
6. Finally, the orchestrator collects the results from the NOOS and ISS modules and consolidates them into the final reports in the `outputs/` directory.

Key Takeaway: *The orchestrator provides significant operational flexibility, allowing users to efficiently tailor the analysis by enabling or disabling specific analytical components based on their immediate business questions or resource constraints.*

7. Results

The Orchestration module coordinates the execution of all sub-modules and ensures their outputs are collected and synchronized. Its direct “results” are:

- **Successful Completion:** The primary outcome is the successful execution of the entire Gap Analysis pipeline, producing reliable and comprehensive insights.
- **Consolidated Reports:** Final analytical reports from all enabled sub-modules are collected, aggregated, and saved to the designated `outputs/` folder, providing a single source of truth for all identified gaps.
- **Error Management:** In case of critical issues, the orchestrator ensures early termination with clear error messages, preventing partial or erroneous results and facilitating quick debugging.

8. Unusual Cases

- **Missing or delayed data feed:** This scenario is robustly handled by the Gap Analysis Input Validation module, which the orchestrator invokes at the very beginning. If data is missing, validation fails, and the orchestrator prevents any downstream analytical modules from running.

- **Failure of a single sub-module:** The orchestrator is designed with error handling to catch exceptions thrown by individual analytical modules. If a sub-module encounters an unrecoverable error, the orchestrator logs the specific issue and gracefully terminates the entire process, preventing incomplete or misleading results from being generated.
- **Configuration conflicts:** While some basic validation (e.g., date ranges) is handled by the Input Validation module, complex logical conflicts between parameters that aren't caught by validation (e.g., conflicting business logic thresholds) might lead to unexpected analytical outcomes, requiring careful review of `args.properties`.
- **Resource exhaustion (e.g., Out of Memory):** For extremely large datasets, the processing might consume excessive memory or CPU. The orchestrator may log such resource warnings or errors, but addressing these typically involves tuning JVM parameters or optimizing data processing within individual sub-modules.

9. Next Steps

Upon successful completion, the consolidated reports generated in the `outputs/` folder are immediately available for review. These reports are then used by:

- **Inventory Planners:** To adjust future buy plans, optimize OD (Optimum Depth) and OW (Optimum Width), and improve ISS (Ideal Size Set) based on identified gaps.
- **Merchandising Teams:** To understand product performance, prioritize assortment changes, and plan promotions more effectively.
- **Financial Analysts:** To refine sales forecasts and quantify the financial impact of current inventory strategies.
- **Automated Systems:** The outputs can be ingested by other business intelligence tools or automated replenishment systems for continuous improvement.

10. Common Questions

1. **How do I run the entire gap analysis?** *You execute the main class associated with this Gap Analysis Orchestration module, typically via a `Java -jar` command from your terminal, after compiling the project.*
2. **Can I run only specific parts of the analysis?** *Yes, you can control which core analytical modules run by adjusting the `run.iss.comparison`, `run.noos.analysis`, and `run.broken.vs.non.broken.analysis` flags in the `args.properties` configuration file.*
3. **What happens if one part of the analysis fails?** *The orchestrator is designed to detect failures in any of its sub-modules. If a module encounters an error, the orchestrator will stop the entire process and log the specific issue, ensuring you don't receive partial or misleading results.*
4. **Where do I find the final reports?** *All consolidated analytical results and reports are written to the `outputs/` folder, which is located in the root directory of the project, after the entire pipeline completes successfully.*
5. **How can I make the analysis faster?** *To improve processing speed, consider disabling any analytical modules that are not immediately critical via their respective run flags in `args.properties`. Additionally, ensure that your system has sufficient allocated resources (RAM, CPU) for Java applications.*

11. Quick Reference

- **Objective:** Coordinate & execute all gap analysis modules efficiently.
 - **Key Inputs:** `args.properties` (for configuration), `common-data.json` (for shared data schema).
 - **Business Settings:** `run.iss.comparison`, `run.noos.analysis`, `run.broken.vs.non.broken.analysis`, `output.synchronization.enabled`.
 - **Key Output:** Successful pipeline execution, consolidated gap analysis reports in `outputs/`.
 - **Business Value:** Streamlines complex analysis workflows, ensures correct module execution order, provides flexibility in analytical focus.
-

Gap Analysis Input Validation

1. Module Name & Business Objective

Module Name: Gap Analysis Input Validation

Business Objective: To act as the first line of defense in the Gap Analysis pipeline, ensuring that all input parameters and raw data feeds are logically consistent, correctly formatted, and readily available. This critical step prevents downstream analytical modules from processing erroneous or incomplete data, thereby safeguarding the accuracy and reliability of the final gap analysis results and avoiding wasted computational resources.

2. Required Information

Sales Data * `units_sold`: *Validated for presence and non-negative values to ensure realistic sales figures.* * `sale_date`: *Validated for correct format (YYYY-MM-DD) and to ensure chronological consistency within the dataset.*

Product Data * `product_id`: *Checked for uniqueness and non-null values to maintain data integrity across products.* * `style`: *Ensured that style data is present and consistent, especially for NOOS classifications.* * `attribute_group (AG)`: *Validated for consistency and completeness, as AGs are key aggregation levels.*

Operational Data * `inventory_on_hand`: *Crucial for all gap analysis; validated for presence, non-negative values, and logical consistency (e.g., not exceeding maximum capacity if such a constraint existed).* * `issInvEndDate`: *The configured end date for the Ideal Size Set (ISS) inventory snapshot, validated for a valid format and logical range relative to the overall analysis period.* * `noosAvailabilityStart`: *The configured start date for the Never-Out-Of-Stock (NOOS) analysis tracking period, validated for format and logical consistency.* * `noosAvailabilityEnd`: *The configured end date for the Never-Out-Of-Stock (NOOS) analysis tracking period, validated for format and logical consistency, ensuring it's after the start date.* * `rosComparisonStart`: *The configured start date for the Rate of Sale (ROS) comparison period, validated for format and logical consistency.* * `rosComparisonEnd`: *The configured end date for the Rate of Sale (ROS) comparison period, validated for format and logical consistency, ensuring it's after the start date.*

3. Business Settings

- `enable.validation`: A boolean flag to enable or disable the entire input validation process.
 - Default: `true`
 - When to Change: *Set to `false` only during very specific development or debugging scenarios where you intentionally want to bypass checks. It should always be `true` in a production environment to guarantee data quality and system stability.*
- `strict.date.validation`: A boolean flag to enforce strict chronological order and non-overlap for date ranges specified across different modules (e.g., ensuring ROS comparison dates do not precede NOOS availability dates if a logical dependency exists).
 - Default: `true`
 - When to Change: *Set to `false` only if overlapping or non-sequential dates are intentionally allowed for specific, unusual analysis scenarios. Use with extreme caution as this can lead to subtle data inconsistencies.*

4. Process Flow

graph TD

```
A[Receive Input Parameters & Data Feeds] --> B{Validate Date Ranges (e.g., Start < End, Format)};
B -- Valid --> C{Check for Critical Data File Presence & Readability};
C -- Present & Readable --> D{Validate Data Content (e.g., Non-negative Inventory, Valid IDs, Schema Compliance)};
D -- Valid --> E[Signal Validation Success];
B -- Invalid --> F[Abort with Informative Error];
C -- Missing/Unreadable --> F;
D -- Invalid --> F;
```

5. Step-by-Step Actions

1. **Receive** all input parameters (e.g., file paths, dates, thresholds) as specified in the `args.properties` configuration file, along with references to the raw input data feeds. *This is the initial hand-off from the orchestrator for pre-analysis checks.*
2. **Validate** all date parameters (`issInvEndDate`, `noosAvailabilityStart`, `noosAvailabilityEnd`, `rosComparisonStart`, `rosComparisonEnd`) to ensure they are in a valid YYYY-MM-DD format and that all start dates chronologically precede their respective end dates. The `strict.date.validation` setting determines if cross-module date overlaps or inconsistencies are also checked. *This prevents logical errors and ensures time-series analysis is based on coherent periods.*
3. **Check** for the existence and readability of all specified critical input data files (e.g., sales history, current inventory snapshots, product master data files). *This confirms that all necessary raw data is physically present and accessible by the system, preventing file-not-found errors downstream.*
4. **Perform** thorough data content validation. This includes:
 - **Schema Compliance**: Verify that the structure and data types of the loaded input files conform to the `common-data.json` schema.

- **Value Integrity:** Check for logical consistency within loaded data, such as ensuring `inventory_on_hand` and `units_sold` are non-negative, `product_ids` are not null or empty, and other critical identifiers are present. *This catches common data quality issues early before they can corrupt analytical results.*
5. If all validations pass successfully, **Signal** a success status back to the Gap Analysis Orchestration module, allowing the pipeline to proceed. If any validation fails, **Throw** an informative error message, detailing the specific issue (e.g., “Invalid date format”, “Missing inventory file”, “Negative inventory value detected”), and cause the program to terminate immediately. *This ensures that subsequent analytical modules only run with high-quality, validated inputs, preventing wasted computation and misleading outputs.*

6. Sample Scenario

A business user tries to run the Gap Analysis with the following setup: * In `args.properties`: `rosComparisonStart=2025-01-01`, `rosComparisonEnd=2024-12-31`. * The `input.data.path` points to a directory, but the required `inventory.csv` file is accidentally missing or unreadable due to permissions.

When the Gap Analysis Input Validation module runs:

1. **Date Validation:** It first checks the ROS comparison dates and immediately identifies that `rosComparisonStart` (January 1, 2025) occurs *after* `rosComparisonEnd` (December 31, 2024). *This is a logical impossibility for a time range.*
2. **File Presence Check:** It then attempts to locate and read `inventory.csv` based on the configured path. Since the file is missing or inaccessible, it fails this check.

The module would then immediately terminate the program with an error message such as: “Validation Error: Invalid date range for ROS comparison. Start date (2025-01-01) must be before end date (2024-12-31).” and possibly “Validation Error: Critical input file ‘inventory.csv’ not found or is unreadable in ‘data/input’”. Please ensure it exists and has correct permissions.”

Key Takeaway: *Input validation is an indispensable gatekeeper. By thoroughly checking inputs before costly computations begin, it prevents the entire analytical pipeline from running on flawed data, thereby saving time, computational resources, and ensuring the credibility of the final business insights.*

7. Results

The Gap Analysis Input Validation module primarily serves as a quality assurance gate rather than producing direct business insights. Its key outputs are:

- **Success Signal:** If all validations pass, it returns a success status to the Gap Analysis Orchestration module, indicating that inputs are ready for processing.
- **Detailed Error Logs and Termination:** If any validation fails, the module generates precise error messages detailing the specific issue (e.g., “Invalid date format”, “Missing file”, “Negative inventory value”). It then terminates the

program, preventing subsequent analytical modules from running on unreliable data.

8. Unusual Cases

- **Missing or delayed data feed:** This module is explicitly designed to handle this by performing checks for file existence and readability at the specified input paths. If a critical data file is not found or cannot be accessed, it will trigger an immediate error.
- **Corrupted data files:** If an input data file exists but its content is malformed (e.g., an unparseable CSV line, invalid JSON structure), the validation module will detect this during its content validation step (e.g., when attempting to parse a field as an integer but finding text) and report it as a data integrity issue, leading to program termination.
- **Subtle logical inconsistencies (beyond basic dates):** For example, an `inventory_on_hand` value might be syntactically valid (an integer) but negative, which is logically impossible for physical stock. This module will flag such inconsistencies as data quality issues. Similarly, if `units_sold` is consistently zero for a product expected to have sales, it might indicate a data capture problem.
- **Schema drift:** If the actual schema of an incoming data file changes (e.g., a column is renamed, a new required field is added, or a data type is altered) and no longer matches the defined `common-data.json` schema, the validation module will detect this mismatch, preventing incorrect data mapping in downstream processes.

9. Next Steps

- **Upon Successful Validation:** If all checks pass, the Gap Analysis Input Validation module signals success to the Gap Analysis Orchestration module. The orchestrator then proceeds to sequentially execute the core analytical modules: Never-Out-Of-Stock (NOOS) Gap Analysis, Ideal Size Set (ISS) Comparison, and Broken vs. Non-Broken Sales Analysis.
- **Upon Validation Failure:** If any check fails, the program terminates immediately, displaying a clear and informative error message. The next step for the user is to review the error log, identify the root cause (e.g., incorrect date in `args.properties`, missing data file, corrupted data), and then correct the input parameters or data before attempting to rerun the analysis.

10. Common Questions

1. **Why did the program stop at the very beginning of the run?** *If the program terminates immediately, it means the Gap Analysis Input Validation module detected a critical issue with your configuration settings (like invalid dates) or determined that essential data files were missing or unreadable. This pre-emptive halt is to ensure no further computation is wasted on unreliable inputs.*
2. **What kind of data issues does this module specifically check for?** *It performs a comprehensive check for valid date ranges, confirms the existence and accessibility of all critical input data files, verifies basic data types (e.g.,*

numbers are indeed numbers), and ensures values like `inventory_on_hand` or `units_sold` are logically sound (e.g., not negative).

3. **Can I bypass or skip this validation step to save time?** *While there is an `enable.validation` flag in `args.properties` that allows you to disable this module, it is strongly advised not* to disable validation in a production environment. Skipping validation significantly increases the risk of running the analysis on bad data, leading to incorrect or misleading results, which can be far more costly than the validation time.**
4. **My data files are definitely “clean,” but validation is still failing. What could be wrong?** *Even seemingly “clean” data can have subtle issues. Check the specific error message carefully. Common culprits include: a logical inconsistency in dates (e.g., a start date after an end date), an incorrect or misspelled file path in `args.properties`, file permission issues preventing read access, or a minor schema deviation from what `common-data.json` expects.*
5. **Does this module check for data completeness (e.g., are all expected SKUs present)?** *This module primarily focuses on data presence* (e.g., file exists), readability (e.g., file can be parsed), and format/integrity (e.g., values conform to type, non-negative quantities). While it validates for non-null IDs, comprehensive checks for the completeness of all expected SKUs or specific data points across an entire dataset are generally handled in a separate data profiling step or implicitly by downstream analytical modules if they encounter missing data for specific items.**

11. Quick Reference

- **Objective:** Ensure the quality and validity of all input data and parameters.
 - **Key Inputs:** All configuration parameters from `args.properties`, paths to raw input data files.
 - **Business Settings:** `enable.validation` (to control execution), `strict.date.validation` (for date consistency).
 - **Key Output:** A success signal for the orchestrator, or a detailed error log with program termination.
 - **Business Value:** Prevents the execution of costly computations on bad data, ensures reliable and trustworthy analysis results, and saves debugging time by identifying issues early.
-

Broken vs. Non-Broken Sales Analysis

1. Module Name & Business Objective

Module Name: Broken vs. Non-Broken Sales Analysis

Business Objective: To precisely quantify the revenue impact of inventory deficiencies by differentiating sales performance on “broken” days (when inventory was insufficient to meet the Ideal Size Set (ISS) benchmark) from “non-broken” days (when inventory was adequate). This module calculates the Rate of Sale (ROS) for both states, revealing the true potential for sales and providing a clear monetary value for lost opportunities due to stockouts.

2. Required Information

Sales Data * `units_sold`: Fundamental for calculating the daily Rate of Sale (ROS) under both inventory conditions. * `sale_date`: Used to segment sales data by day and determine if each day qualifies as “broken” or “non-broken” based on inventory levels. * `revenue`: To quantify potential lost sales in monetary terms.

Product Data * `attribute_group` (AG): The primary level at which ROS analysis and lost sales quantification are performed, providing actionable insights for product categories. * `product_id`: Unique identifier used for granular tracking and aggregation within AGs. * `style`: To group products for ROS analysis, especially in conjunction with `noos_designation_flag` and `brand_grading_score` from the NOOS module.

Operational Data * `inventory_on_hand`: Crucial for determining if a particular day for a product or AG was “broken” based on whether inventory met a defined threshold. * `iss_benchmark_coverage`: The percentage of the Ideal Size Set (ISS) required to consider a day “non-broken” for a specific product or AG. * `brand_grading_score` (from NOOS Gap Analysis): Provides contextual information on the consistent availability performance of NOOS styles, which can be used to refine or provide additional context to lost sales estimations for core products. * `rosComparisonStart`: The configured start date for the sales comparison period, defining the historical window for analysis. * `rosComparisonEnd`: The configured end date for the sales comparison period, defining the historical window for analysis.

3. Business Settings

- `ros.min.days.for.comparison`: The minimum number of “non-broken” days required within the analysis period to calculate a statistically reliable Rate of Sale (ROS) benchmark for an Attribute Group (AG) or product.
 - Default: 7 days
 - When to Change: Adjust based on the desired statistical significance and stability of the ROS benchmark. Higher values ensure a more robust and less volatile benchmark, but may exclude products with limited non-broken historical data.
- `ros.broken.day.inventory.threshold`: The inventory coverage percentage (relative to an Ideal Size Set (ISS) or a fixed quantity) below which a day is classified as “broken” for a particular product or Attribute Group (AG).
 - Default: 0.50 (50% of the Ideal Size Set or expected inventory level)
 - When to Change: Modify this threshold to reflect what the business considers an ‘unhealthy’ or insufficient inventory level that would genuinely impede sales. A lower value means fewer days are flagged as broken, potentially understating lost sales.
- `ros.sales.lost.calculation.method`: Defines the methodology used to quantify potential lost sales (units and/or revenue) on “broken” days.
 - Default: `non_broken_diff` (Calculates lost sales as Non-Broken ROS – Actual ROS on Broken Days)
 - Valid Options: `non_broken_diff` or `target_ros_based` (which might use a broader, pre-defined target ROS).
 - When to Change: Choose the method that best aligns with the business’s preferred approach to quantifying lost opportunities, potentially reflecting different levels of conservatism or aggressiveness in the calculation.

4. Process Flow

graph TD

A[Load Sales & Inventory Data for `rosComparison` Period] --> B{Categorize Each Day for Products/AGs as "Broken" or "Non-Broken" based on `ros.broken.day.inventory.threshold`};

B --> C[Calculate Average Rate of Sale (ROS) for All "Non-Broken" Days];

C --> D[Calculate Average Rate of Sale (ROS) for All "Broken" Days (Actual ROS)];

D --> E{Compare "Non-Broken ROS" (Benchmark) vs. "Broken ROS" (Actual) for Each Product/AG};

E --> F[Estimate Potential Lost Sales (Units & Revenue) using `ros.sales.lost.calculation.method` and Duration of Broken Days];

F --> G[Incorporate `brandGradingMap` (from NOOS) for Context/Refinement (if available)];

G --> H[Output Lost Sales & ROS Benchmarks by Product/AG];

5. Step-by-Step Actions

1. **Load** historical `units_sold`, revenue, and `inventory_on_hand` data for the specified `rosComparisonStart` to `rosComparisonEnd` period. This includes data points for each `product_id` and `Attribute Group (AG)`. *This step gathers all necessary raw data for the comparative analysis.*
2. For each day within the analysis period, **Categorize** every `product_id` (or `Attribute Group`) as either "broken" or "non-broken." This classification is determined by whether its `inventory_on_hand` on that day met or fell below the `ros.broken.day.inventory.threshold`. *This crucial segmentation allows for a direct comparison of sales performance under different inventory conditions.*
3. **Calculate** the average Rate of Sale (ROS) for all days classified as "non-broken" for each `Attribute Group (AG)` and individual `product_id`. *This establishes a reliable benchmark for expected sales velocity when inventory levels are considered adequate and not impeding demand.*
4. **Calculate** the actual average Rate of Sale (ROS) for all days identified as "broken" for each `Attribute Group (AG)` and `product_id`. *This reveals the suppressed sales velocity that occurred specifically when inventory was insufficient.*
5. **Quantify** the potential lost sales (in both units and revenue) for each `product_id` or `Attribute Group`. This is done by comparing the "non-broken ROS" (benchmark) to the "broken ROS" (actual suppressed sales) over the total duration of the "broken" days, using the configured `ros.sales.lost.calculation.method`. *This step directly translates inventory gaps into a clear, measurable financial impact.*
6. **Incorporate** the `brand_grading_score` (from the NOOS Gap Analysis module) into the lost sales estimation or reporting, if available. *This provides additional context, particularly for core "Never-Out-Of-Stock" (NOOS) products, indicating whether the calculated lost sales are part of a broader, consistent availability issue for a brand.*
7. **Generate** a comprehensive report detailing the classified days, the ROS for both "non-broken" and "broken" states, and the calculated total potential lost sales (units and revenue) for each `product_id` and `Attribute Group`. *This output provides actionable insights for business decision-makers.*

6. Sample Scenario

Consider an Attribute Group (AG) “Women’s Denim Jeans - Blue Straight Leg”. Over a 30-day analysis period: * **20 days** were classified as “non-broken” (inventory consistently above 50% threshold). During these days, the average Rate of Sale (ROS) for this AG was **15 units/day**. * **10 days** were classified as “broken” (inventory fell below 50% threshold). During these 10 days, the actual average ROS was only **6 units/day**. * The average selling price for these jeans is \$50/unit. * The `ros.sales.lost.calculation.method` is set to `non_broken_diff`.

Calculation: * Potential lost sales per broken day = (Non-Broken ROS - Broken ROS) = (15 units/day - 6 units/day) = **9 units/day**. * Total potential lost sales in units = 9 units/day * 10 broken days = **90 units**. * Total potential lost sales in revenue = 90 units * \$50/unit = **\$4,500**.

Key Takeaway: *This module clearly quantifies that for “Women’s Denim Jeans - Blue Straight Leg,” the business lost an estimated 90 units, amounting to \$4,500 in revenue, over the 10 days it struggled with inventory. This provides a direct, data-driven justification for prioritizing inventory improvements for this specific Attribute Group (AG).*

7. Results

The Broken vs. Non-Broken Sales Analysis module outputs detailed, actionable insights:

- **Daily Classification:** A granular report indicating for each `product_id` or Attribute Group (AG) whether each day within the analysis period was classified as “broken” (insufficient inventory) or “non-broken” (adequate inventory).
- **Rate of Sale (ROS) Benchmarks:**
 - The average Rate of Sale (ROS) for “non-broken” periods, which serves as a realistic benchmark for expected sales performance when inventory is not a constraint.
 - The average Rate of Sale (ROS) for “broken” periods, reflecting the actual, suppressed sales velocity that occurred due to inventory deficiencies.
- **Quantified Lost Sales:** A precise estimate of potential lost sales, reported in both units and revenue, directly attributable to insufficient inventory on “broken” days for each `product_id` and Attribute Group (AG). This is the core business value output.
- **Contextual Data:** If available, the `brand_grading_score` from the N00S Gap Analysis module is incorporated to provide additional context, particularly for core products.

8. Unusual Cases

- **Missing or delayed data feed:** While the Input Validation module handles initial checks, if data for specific products or dates is partially missing or corrupted *after* initial validation (e.g., due to system errors during processing), it can lead to skewed ROS calculations or inaccurate broken/non-broken classifications for those specific items, potentially understating or overstating lost sales.

- **Insufficient “non-broken” days:** If an Attribute Group (AG) or product consistently struggles with inventory (meaning it has very few or no “non-broken” days meeting the `ros.min.days.for.comparison` threshold), establishing a reliable “non-broken ROS” benchmark becomes challenging. In such cases, the module might default to a broader category ROS or flag the item for manual review, as the calculated lost sales might be less accurate.
- **External factors heavily influencing sales:** Promotions, holidays, severe weather events, or aggressive competitor actions can cause significant fluctuations in ROS that are unrelated to inventory levels. While the module focuses on inventory’s impact, such external factors could confound the “broken” day ROS, requiring careful interpretation of the results and potentially additional manual analysis.
- **Products with very low ROS:** For extremely slow-moving items, distinguishing between a genuine stockout impeding sales and naturally very low sales volume can be difficult. The module might still flag “broken” days, but the lost sales impact might be negligible in practice.
- **Rapid inventory fluctuations:** Products with extremely high inventory turnover (e.g., replenished daily and sold out quickly) might frequently switch between “broken” and “non-broken” status. While the module tracks this, it highlights the need for continuous, almost real-time inventory management rather than just a historical view.

9. Next Steps

The detailed insights from the Broken vs. Non-Broken Sales Analysis module are pivotal for strategic business decisions:

- **Prioritize Inventory Investments:** The quantifiable lost sales figures provide a strong business case for allocating capital to specific Attribute Groups (AGs) or `product_ids` that demonstrate the highest potential for sales recovery through improved inventory.
- **Optimize Inventory Levels:** Inventory planners use the “non-broken ROS” as a target velocity to determine optimal reorder points and order quantities, aiming to maintain Optimum Depth (OD) and Optimum Width (OW) and avoid future “broken” days.
- **Refine Merchandising Strategies:** Merchandising teams gain insights into product availability challenges, which can inform promotional calendars, allocation strategies, and potential shifts in assortment planning.
- **Improve Demand Forecasting:** Understanding the true, unsuppressed Rate of Sale (ROS) helps refine future demand forecasts, ensuring more accurate predictions of customer needs.
- **Evaluate Supplier Performance:** Persistent “broken” days for specific products can indicate issues with supplier reliability or lead times, prompting discussions with vendors.

10. Common Questions

1. **What exactly is a “broken day” in this analysis?** *A “broken day” is a specific day where the inventory level for a particular product or an Attribute Group (AG) fell below a pre-defined minimum threshold (`ros.broken.day.inventory.threshold`). This indicates that stock levels were likely insufficient to fully meet customer demand on that day.*

2. **How is “Rate of Sale (ROS)” specifically calculated for “broken” and “non-broken” days?** *ROS is calculated as the average units sold per day for a specific product or AG within the defined analysis period. This module then segments these calculations into two averages: one for days when inventory was adequate (“non-broken ROS”) and another for days when inventory was insufficient (“broken ROS”).*
3. **Can the results from this module help us prioritize which products to reorder first?** *Absolutely. Products or Attribute Groups (AGs) that show a high “non-broken ROS” (indicating strong demand) but also experience a significant number of “broken” days and high potential lost sales are prime candidates for immediate replenishment and inventory focus. The quantified lost sales provide a clear ranking.*
4. **What if a product *never* has a “non-broken” day in the historical data? How do we calculate its lost sales?** *If a product consistently struggles with inventory and never meets the “non-broken” threshold, the module will flag this as a persistent issue. While a specific “non-broken ROS” for that item cannot be derived, the module may use a broader category benchmark or highlight the item for manual review, indicating a severe, ongoing stock problem.*
5. **Does this analysis account for planned promotions or events that might have caused stockouts?** *This module identifies stockouts and quantifies their sales impact based on historical ROS. While it does not inherently distinguish between planned (e.g., for a promotion) and unplanned stockouts, the quantified lost sales still represent unrealized revenue. Businesses can use this data in conjunction with their promotional calendars to assess if the sales uplift from the promotion justified the associated inventory risk.*

11. Quick Reference

- **Objective:** Quantify the direct financial impact (lost units/revenue) of inventory deficiencies.
 - **Key Inputs:** Sales data (units_sold, revenue), Inventory data (inventory_on_hand), Product attributes (Attribute Group, product_id), brand_grading_score (from NOOS).
 - **Business Settings:** ros.min.days.for.comparison, ros.broken.day.inventory.threshold, ros.sales.lost.calculation.method.
 - **Key Output:** Potential lost sales (units and revenue), “non-broken” ROS benchmark, “broken” ROS.
 - **Business Value:** Provides a clear, data-driven justification for inventory investments, prioritizes replenishment efforts, and helps optimize Optimum Depth (OD) and Optimum Width (OW).
-

Ideal Size Set (ISS) Comparison

1. Module Name & Business Objective

Module Name: Ideal Size Set (ISS) Comparison

Business Objective: To evaluate the effectiveness of current inventory against an Ideal Size Set (ISS) for specific products, identifying where stockouts or overstocks at the size level are leading to unliquidated sales or unnecessary inventory holding

costs. This analysis helps optimize inventory depth (OD) and width (OW) by ensuring the best combination of sizes is available to maximize sales and minimize stockouts.

2. Required Information

Sales Data * `units_sold`: *Historical sales by size are critical to determine the demand distribution across sizes, which informs the Ideal Size Set (ISS) calculation.*
* `sale_date`: *To analyze sales trends over a relevant historical period that dictates the Ideal Size Set (ISS) composition.*

Product Data * `size`: *The core dimension for this analysis; used to identify which specific sizes are contributing to demand and where gaps or surpluses exist.* * `attribute_group (AG)`: *The primary grouping for which an Ideal Size Set (ISS) is determined and analyzed, allowing for targeted optimization within specific product categories or styles.* * `product_id`: *Unique identifier for products to link sales, inventory, and size information.*

Operational Data * `inventory_on_hand`: *Current stock levels for each product_id by size are compared against the calculated Ideal Size Set (ISS) to identify discrepancies.* * `iss_benchmark_coverage`: *A configurable target percentage that defines what constitutes the “ideal” state for inventory coverage within an Ideal Size Set (ISS) for a given Attribute Group (AG).* * `issInvEndDate`: *The configured end date for the inventory data snapshot used in the Ideal Size Set (ISS) comparison, ensuring the analysis uses the most current stock data.*

3. Business Settings

- `iss.target.coverage.percent`: The target percentage of sizes within an Attribute Group (AG) that should ideally be in stock to meet the Ideal Size Set (ISS) objectives. This acts as a multiplier or threshold for determining the optimal quantity per size based on historical demand distribution.
 - Default: **0.95** (95%)
 - When to Change: *Adjust this value based on the desired level of inventory coverage for complete size sets. Higher values enforce a stricter requirement for size availability, potentially leading to higher inventory levels but fewer lost sales due to size unavailability.*
- `iss.min.units.for.analysis`: The minimum total number of units sold historically for an Attribute Group (AG) or style to be considered for Ideal Size Set (ISS) analysis. This prevents the generation of an ISS for low-volume items where the statistical significance might be low or where an ISS is not a critical business concern.
 - Default: **100** units
 - When to Change: *Modify this threshold to filter out very slow-moving or niche products from ISS optimization, allowing the analysis to focus on high-impact items.*

4. Process Flow

graph TD

A[Load Sales & Inventory Data by AG & Size] --> B[Calculate Historical Demand Distribution by Size for Each AG];

```

    B --> C{Determine Target Ideal Size Set (ISS) Quantities per
Size based on Demand Distribution &
`iss.target.coverage.percent`};
    C --> D{Compare Current `inventory_on_hand` vs. Target ISS
Quantities};
    D -- Inventory < Target --> E[Identify Unliquidated Sales
Quantities (Lost Sales for Sizes)];
    D -- Inventory >= Target --> F[Identify Overstock Quantities
(if applicable)];
    E --> G[Aggregate Lost Sales by AG, Size, and Store];
    F --> G;
    G --> H[Output ISS Gaps, Unliquidated Sales & Overstock
Reports];

```

5. Step-by-Step Actions

1. **Load** comprehensive historical `units_sold` data, categorized by Attribute Group (AG) and size, along with the latest `inventory_on_hand` snapshot (as of `issInvEndDate`). *This step brings together all the necessary information to understand past demand and current stock.*
2. For each Attribute Group (AG), **Calculate** the historical sales distribution across all available sizes. For example, determine what percentage of total AG sales typically come from Small, Medium, Large, etc. *This identifies the natural demand patterns for different sizes within a product group.*
3. **Determine** the Ideal Size Set (ISS) for each AG. This involves calculating the optimal quantity for each size based on its historical demand proportion and factoring in the `iss.target.coverage.percent`. *This step establishes the target inventory composition that aims to satisfy demand across all sizes.*
4. **Compare** the current `inventory_on_hand` for each `product_id` and size against its calculated Ideal Size Set (ISS) quantity. *This highlights discrepancies between what is currently available and what should* be available to meet demand efficiently.**
5. **Identify** “unliquidated sales” (i.e., potential lost sales) for any size where the current `inventory_on_hand` is below the calculated Ideal Size Set (ISS) recommendation. Quantify the difference in units. *This directly measures the sales opportunities missed due to insufficient stock in specific sizes.*
6. Concurrently, **Identify** any “overstock” positions where the current `inventory_on_hand` for a size significantly exceeds its Ideal Size Set (ISS) recommendation. Quantify the excess units. *This highlights capital tied up in slow-moving or excess inventory.*
7. **Aggregate** the findings, summarizing unliquidated sales and overstock quantities at the Attribute Group (AG) and size level. *This provides a clear, actionable overview of where size-related inventory issues are most pronounced.*
8. **Generate** a detailed report outlining Ideal Size Set (ISS) compliance, itemized unliquidated sales, and overstock positions. *This provides the business with the insights needed for informed inventory and merchandising adjustments.*

6. Sample Scenario

Imagine an Attribute Group (AG) “Men’s Casual T-Shirts - Crew Neck” with the following historical sales distribution across sizes: * Small: 10% * Medium: 30% * Large: 40% * XL: 20% The `iss.target.coverage.percent` is set to 95%. For a

total target inventory of 200 units for this AG: * ISS for Small: $200 * 0.10 * 0.95 = 19$ units * ISS for Medium: $200 * 0.30 * 0.95 = 57$ units * ISS for Large: $200 * 0.40 * 0.95 = 76$ units * ISS for XL: $200 * 0.20 * 0.95 = 38$ units

Now, consider the current inventory_on_hand at a store: * Small: 25 units (Overstock by 6 units) * Medium: 40 units (Understock by 17 units) * Large: 80 units (Overstock by 4 units) * XL: 10 units (Understock by 28 units)

The module would identify: * **Unliquidated Sales:** 17 units for Medium, 28 units for XL. Total = 45 units. * **Overstock:** 6 units for Small, 4 units for Large. Total = 10 units.

Key Takeaway: *This module provides precise, size-level insights. For “Men’s Casual T-Shirts - Crew Neck,” it highlights a significant opportunity to capture 45 additional sales by improving the availability of Medium and XL sizes, while also identifying 10 units of potential overstock in Small and Large, freeing up capital and space.*

7. Results

The Ideal Size Set (ISS) Comparison module generates detailed outputs crucial for inventory and merchandising optimization:

- **Ideal Size Set (ISS) Recommendations:** Calculated optimal quantities for each size within an Attribute Group (AG) based on historical demand distribution and the `iss.target.coverage.percent`.
- **Quantified Unliquidated Sales:** A precise measurement (in units and potentially revenue) of sales opportunities missed due to insufficient `inventory_on_hand` for specific sizes that fall below their Ideal Size Set (ISS) recommendations.
- **Identified Overstock Positions:** A clear indication of sizes within an AG where `inventory_on_hand` significantly exceeds the Ideal Size Set (ISS) recommendation, quantifying the excess units.
- **Compliance Report:** A comprehensive report detailing the current Ideal Size Set (ISS) compliance status for each Attribute Group (AG), highlighting both understocked and overstocked sizes.

8. Unusual Cases

- **Missing or delayed data feed:** Incomplete or outdated historical sales data will lead to an inaccurate calculation of the Ideal Size Set (ISS) distribution, resulting in flawed recommendations. Similarly, an old `inventory_on_hand` snapshot will misrepresent current stock levels.
- **Seasonal shifts in demand not captured:** If the historical sales data used to derive the Ideal Size Set (ISS) doesn’t adequately reflect current or upcoming seasonal demand shifts, the calculated ISS might be suboptimal (e.g., warmer weather might shift demand towards smaller sizes, but historical data still reflects winter trends).
- **New product introductions without historical sales:** For entirely new products with no historical sales data, the module cannot calculate an Ideal Size Set (ISS). In such cases, businesses often rely on proxy data from comparable products or initial forecasts, which would need to be manually fed into the process.

- **Extremely low-volume products:** If an Attribute Group (AG) or product has very few historical sales (below `iss.min.units.for.analysis`), the statistical significance of its size distribution might be low, leading to volatile or unreliable ISS recommendations. The module's setting helps filter these out, but if forced, results might be questionable.
- **Highly irregular demand patterns:** Products with unpredictable or spikey demand (e.g., fashion trends, limited editions) might not conform well to a static Ideal Size Set (ISS) based on historical averages, requiring more dynamic inventory management strategies.

9. Next Steps

The results from the Ideal Size Set (ISS) Comparison module provide actionable insights that directly influence inventory and merchandising strategies:

- **Optimize Buy Plans:** Merchandising and planning teams use the Ideal Size Set (ISS) recommendations and unliquidated sales data to adjust future purchasing and replenishment orders, ensuring Optimum Depth (OD) (enough units per size) and Optimum Width (OW) (correct range of sizes) are met.
- **Rebalance Inventory:** The identified overstock positions can trigger internal transfers or promotional activities to liquidate excess inventory, freeing up capital and warehouse space.
- **Inform Assortment Strategy:** Consistent gaps in the Ideal Size Set (ISS) for certain Attribute Groups (AGs) can indicate issues with initial assortment planning or product development, providing feedback for future collections.
- **Improve In-Store Merchandising:** For physical stores, understanding size-level gaps can guide store managers on how to best present available stock or request inter-store transfers.

10. Common Questions

1. **What does “Ideal Size Set (ISS)” truly mean for my business?** *The Ideal Size Set (ISS) represents the optimal mix of sizes for a specific product or category that aims to maximize sales by ensuring popular sizes are always available, while simultaneously minimizing overstock in less popular or slow-moving sizes, thus optimizing inventory depth (OD) and width (OW).*
2. **How is “unliquidated sales” different from a general stockout report?** *While related to stockouts, “unliquidated sales” specifically quantifies the demand that could not be met because a particular size of a product was either completely out of stock or had insufficient inventory relative to its calculated Ideal Size Set (ISS) benchmark, highlighting a precise missed opportunity for a specific size.*
3. **Does this analysis consider future trends or just historical data?** *This module primarily relies on historical sales data to determine the Ideal Size Set (ISS) distribution. While it provides a strong data-driven foundation, incorporating future trend forecasts or expert market insights would typically be a separate, subsequent step in the broader inventory planning process.*
4. **Can this module directly recommend a new Ideal Size Set (ISS) for my next product buy?** *Yes, by identifying the historical demand patterns and current gaps in inventory, the module implicitly provides the necessary data to inform a more optimized Ideal Size Set (ISS) for future replenishment cycles and for initial buys of new, similar product introductions.*

5. **What if we don't have enough sales data for a particular Attribute Group (AG) or a new product?** *For AGs with very limited sales (below `iss.min.units.for.analysis`), they will be filtered out to avoid unreliable ISS calculations. For entirely new product introductions, the initial Ideal Size Set (ISS) would need to be determined based on forecasts, comparable product data, or expert judgment, as this module requires historical sales to build its recommendations.*

11. Quick Reference

- **Objective:** Evaluate and optimize the Ideal Size Set (ISS) to match demand.
 - **Key Inputs:** Historical `units_sold` by size, `inventory_on_hand`, Attribute Group (AG) definitions, `iss.target.coverage.percent`, `issInvEndDate`.
 - **Business Settings:** `iss.target.coverage.percent`, `iss.min.units.for.analysis`.
 - **Key Output:** Quantified unliquidated sales by AG/size, identified overstock by AG/size, ISS compliance report.
 - **Business Value:** Maximizes sales by improving size availability, reduces inventory holding costs by minimizing overstock, informs Optimum Depth (OD) and Optimum Width (OW) decisions.
-

Never-Out-Of-Stock (NOOS) Gap Analysis

1. Module Name & Business Objective

Module Name: Never-Out-Of-Stock (NOOS) Gap Analysis

Business Objective: To proactively assess and ensure the continuous availability of core, essential “Never-Out-Of-Stock” (NOOS) styles across all retail channels. This module quantifies the performance of NOOS items against defined availability benchmarks, identifies critical gaps in maintaining core product presence, and computes a “brand grading” score to inform strategic decisions on core product management and supply chain reliability.

2. Required Information

Sales Data * `units_sold`: *Historical sales data helps understand the consistent demand for NOOS styles and provides context for their expected continuous availability.* * `store_id`: *Essential for tracking and assessing NOOS availability and performance across different retail locations or channels.*

Product Data * `style`: *The primary identifier for NOOS products; used to filter and focus the analysis on designated core styles.* * `attribute_group` (AG): *Allows for analysis of NOOS styles at a more granular product grouping level, providing insights into specific product attribute combinations.* * `product_id`: *Unique identifier for products to link sales and inventory to individual NOOS items.* * `noos_designation_flag`: *A boolean flag in the product master data indicating whether a style is designated as a Never-Out-Of-Stock (NOOS) item.*

Operational Data * `inventory_on_hand`: Current and historical stock levels for each `product_id` to assess whether NOOS items meet their specified inventory benchmarks on a daily basis. * `sale_date` (or `inventory_date`): To track `inventory_on_hand` over the defined period and determine daily availability status. * `noosAvailabilityStart`: The configured start date for the NOOS availability tracking period, defining the historical window for analysis. * `noosAvailabilityEnd`: The configured end date for the NOOS availability tracking period, defining the historical window for analysis.

3. Business Settings

- `noos.min.inventory.benchmark`: The minimum inventory level a NOOS item must maintain to be considered “in stock” for a given day. This can be defined as a percentage of its Ideal Size Set (ISS) target, a fixed minimum unit count, or a coverage threshold.
 - Default: **0.90** (90% of target ISS or expected inventory for full availability)
 - When to Change: *Adjust this benchmark to reflect the business’s desired strictness for “in-stock” status for core NOOS items. Higher values demand more robust inventory levels, while lower values may indicate a more tolerant approach.*
- `noos.minimum.days.in.stock`: The minimum number of days a NOOS style must meet its `noos.min.inventory.benchmark` within the specified `noosAvailabilityStart` to `noosAvailabilityEnd` tracking period to be considered successful or “compliant.”
 - Default: **30** days
 - When to Change: *Modify this threshold based on the desired consistency and reliability of NOOS availability. A higher value means a stricter compliance requirement, indicating truly consistent core product presence.*

4. Process Flow

graph TD

```

  A[Load NOOS Designated Sales & Inventory Data for
  `noosAvailability` Period] --> B{Calculate Daily
  `inventory_on_hand` vs. `noos.min.inventory.benchmark` for each
  NOOS `product_id` by `store_id`};
  B --> C{Check if Daily Availability meets
  `noos.min.inventory.benchmark`};
  C -- Yes --> D[Increment "Days In Stock" Counter for NOOS
  product/store];
  C -- No --> E[Maintain Counter (or Reset if consecutive "out-
  of-stock" days are tracked)];
  D --> F[Aggregate Total "Days In Stock" by NOOS `style` and
  `store_id` over Period];
  E --> F;
  F --> G[Calculate "Brand Grading Score" based on % of Days In
  Stock vs. `noos.minimum.days.in.stock`];
  G --> H[Output NOOS Availability Report & `brandGradingMap`];

```


5. Step-by-Step Actions

1. **Load** historical `units_sold` data and `inventory_on_hand` records for all products specifically designated as “Never-Out-Of-Stock” (NOOS) styles within the `noosAvailabilityStart` to `noosAvailabilityEnd` period. Data should be granular to `product_id` and `store_id`. *This gathers all relevant information for monitoring core product availability.*
2. For each NOOS `product_id` at every `store_id` on each day within the tracking period, **Calculate** its actual `inventory_on_hand` status relative to the `noos.min.inventory.benchmark`. *This determines if the product met its availability target on that specific day.*
3. **Track** the number of “days in stock” for each NOOS `product_id` at each `store_id`. If the daily inventory meets or exceeds the `noos.min.inventory.benchmark`, increment the counter for that specific product/store. *This accumulates a direct measure of consistent availability.*
4. **Aggregate** the total “days in stock” for each NOOS style (or Attribute Group) across all `store_ids` over the entire `noosAvailability` period. *This provides a consolidated view of NOOS performance across the network.*
5. **Compute** a “Brand Grading Score” for each NOOS style (or Attribute Group). This score is derived from the percentage of days it met the `noos.minimum.days.in.stock` threshold, potentially weighted by importance or volume. *This quantifies the brand’s or style’s overall performance in maintaining core product availability, serving as a key performance indicator.*
6. **Generate** a detailed NOOS Availability Report, showing days in stock, percentage compliance, and the computed “Brand Grading Score” for each NOOS style and `store_id`.
7. **Store** the computed `brandGradingMap` (containing the scores for each NOOS style or Attribute Group) in a shared data structure. *This output is crucial for other modules, like the Broken vs. Non-Broken Sales Analysis, to provide additional context to their lost sales estimations.*

6. Sample Scenario

A core NOOS T-shirt (Style: “Classic Crew”) is expected to be in stock at least 90% of the time (defined by `noos.min.inventory.benchmark=0.90`) over a 60-day tracking period. The `noos.minimum.days.in.stock` is set to 30 days.

- **Store A:** “Classic Crew” met the 90% inventory benchmark for 54 out of 60 days.
 - Availability: $(54 / 60) * 100\% = 90\%$.
 - This exceeds `noos.minimum.days.in.stock` ($54 > 30$).
- **Store B:** “Classic Crew” met the 90% inventory benchmark for 25 out of 60 days.
 - Availability: $(25 / 60) * 100\% = 41.7\%$.
 - This is below `noos.minimum.days.in.stock` ($25 < 30$).

The module would calculate a high “Brand Grading Score” for “Classic Crew” at Store A (reflecting excellent compliance) and a lower score for Store B (reflecting a significant availability gap). The `brandGradingMap` would reflect these differentiated scores.

Key Takeaway: *This module provides immediate, quantifiable feedback on the health of core NOOS product availability. It quickly pinpoints specific stores or styles that are failing to meet essential stock targets, allowing for targeted interventions to protect core product sales and brand reputation.*

7. Results

The Never-Out-Of-Stock (NOOS) Gap Analysis module produces several key outputs:

- **NOOS Availability Report:** A detailed report outlining, for each designated NOOS product_id and store_id, the number of days it met the noos.min.inventory.benchmark within the tracking period, along with its calculated percentage availability.
- **Brand Grading Scores:** A brandGradingMap which assigns a performance score (based on availability compliance) to each NOOS style or Attribute Group (AG). This map is explicitly stored in the shared data structure (gapModuleData) for consumption by other analytical modules, particularly the Broken vs. Non-Broken Sales Analysis module.
- **Identified Availability Gaps:** Clear identification of specific NOOS styles or store_ids that failed to meet the noos.minimum.days.in.stock threshold, signaling areas of concern.

8. Unusual Cases

- **Missing or delayed data feed:** If inventory_on_hand data for specific NOOS styles or stores is incomplete, delayed, or corrupted during the noosAvailability period, the calculated availability and “Brand Grading Score” will be inaccurate, potentially leading to false positives (flagging issues where none exist) or false negatives (missing actual gaps).
- **Incorrect NOOS designation:** If a product style is incorrectly flagged as a NOOS item (when it’s not truly a core evergreen product), it will be subjected to unnecessarily strict availability expectations, leading to misprioritized alerts. Conversely, if a true NOOS item is *not* designated as such, its availability won’t be monitored, allowing critical gaps to go unnoticed.
- **Sudden, unpredictable demand spikes:** While NOOS items are expected to have consistent demand, unforeseen external events (e.g., viral social media trends, sudden competitor withdrawal) could lead to rapid, legitimate stockouts. The module will still flag these as availability gaps, requiring manual review and contextual understanding from the business side.
- **Benchmark set too high/low:** An noos.min.inventory.benchmark that is set unrealistically high may result in almost all NOOS items being flagged for availability issues, overwhelming planners. Conversely, a benchmark set too low might mask genuine, critical availability problems for core products.
- **Short tracking periods:** If the noosAvailabilityStart and noosAvailabilityEnd define a very short period, the calculated availability might be statistically unstable and not truly representative of the long-term NOOS performance.

9. Next Steps

The insights and outputs from the Never-Out-Of-Stock (NOOS) Gap Analysis module are critical for maintaining the health of core product offerings:

- **Inform Inventory Replenishment:** The identified availability gaps directly inform inventory planning teams on which NOOS styles or `store_ids` require immediate attention for replenishment, ensuring continuous Optimum Depth (OD).
- **Strategic Brand Performance Review:** The `brandGradingMap` provides high-level insights for merchandising and brand management teams to evaluate the consistency of core product availability, which is a key aspect of brand health and customer satisfaction.
- **Supply Chain Optimization:** Persistent NOOS availability issues can signal underlying problems in the supply chain (e.g., manufacturing delays, transit issues, vendor reliability), prompting investigations and process improvements.
- **Input for Other Modules:** The `brandGradingMap` is passed as a crucial input to the Broken vs. Non-Broken Sales Analysis module, allowing it to incorporate the context of core product availability into its lost sales calculations, making them more nuanced and business-relevant.

10. Common Questions

1. **How do we define a “Never-Out-Of-Stock (NOOS) style” in our business?** *NOOS styles are typically our core, evergreen products that are considered foundational to our offering. They are expected to be continuously in stock due to their consistent customer demand, high sales velocity (ROS), and strategic importance to the brand, as they drive consistent revenue and customer loyalty.*
2. **What does the “Brand Grading Score” specifically tell me?** *The “Brand Grading Score” provides a quantifiable measure of how effectively a specific NOOS style (or a group of NOOS products under a brand) is maintaining its expected on-shelf availability across all tracked stores. A higher score indicates better compliance with the NOOS availability targets.*
3. **Why might a NOOS style be flagged as having “low availability” if we’ve just had a massive promotional sale?** *The module focuses on whether the inventory benchmark was met on a given day. While a large sale explains why* a stockout occurred, the module still flags the day as an availability gap because the product was not consistently available as expected. It’s a measure of actual on-shelf presence, irrespective of the cause of depletion. Contextual understanding is often necessary.**
4. **Can I set different minimum inventory benchmarks for different NOOS styles (e.g., higher for top sellers)?** *Currently, the `noos.min.inventory.benchmark` is a global setting. However, future enhancements could allow for more granular, style-specific or Attribute Group-specific benchmarks to be configured, providing more tailored analysis for diverse NOOS portfolios.*
5. **How often should this NOOS analysis be run to be most effective?** *Ideally, this analysis should be run frequently, at least daily or weekly. This provides timely and actionable insights into the availability performance of your core products, enabling rapid response to any emerging stock issues and proactive adjustments to inventory management and replenishment plans.*

11. Quick Reference

- **Objective:** Assess and ensure continuous availability of core “Never-Out-Of-Stock” (NOOS) styles.
 - **Key Inputs:** `noos_designation_flag`, `inventory_on_hand`, `units_sold`, `store_id`, `style`, Attribute Group (AG), `noosAvailabilityStart`, `noosAvailabilityEnd`.
 - **Business Settings:** `noos.min.inventory.benchmark`, `noos.minimum.days.in.stock`.
 - **Key Output:** NOOS availability reports (days in stock, % compliance), `brandGradingMap` (for other modules).
 - **Business Value:** Guarantees core product presence, prevents lost sales for essential items, enhances customer satisfaction, informs Optimum Depth (OD) strategies for key items.
-

9. Appendix

Full Parameter Reference

Refer to Section 6: [Global Configuration](#) for a comprehensive list of all configurable parameters, their default values, descriptions, and guidance on valid ranges or when to change them.

Version History

- **Version 1.0 (2025-05-23):** Initial release of the gap analysis algorithm and its documentation. This version includes the core modules for Input Validation, NOOS Gap Analysis, Ideal Size Set (ISS) Comparison, and Broken vs. Non-Broken Sales Analysis, orchestrated by the central Gap Analysis Orchestration module.

Contact & Support Info

For any questions, support requests, or feature enhancements related to the gap documentation or the algorithm itself, please contact:

- **Support Team:** `support@yourcompany.com`
- **Project Lead:** `projectlead@yourcompany.com`
- **Internal Documentation Portal:** [Link to your internal Confluence/Wiki page] ``

Tutorial: gap

The Gap Analysis Orchestration module (0) is the central coordinator, dictating the execution sequence for all sub-modules. It implicitly initiates the Input Validation module (1) to ensure data integrity before proceeding with the core analysis. Subsequently, it explicitly orchestrates the execution of the Never-Out-Of-Stock (NOOS) Gap Analysis (4), Ideal Size Set (ISS) Comparison (3), and Broken vs. Non-Broken Sales Analysis (2) modules in a defined order. The NOOS Gap Analysis module (4) produces brand grading data that is directly consumed by the Broken vs. Non-Broken Sales Analysis module (2).

flowchart TD

```
A0["Gap Analysis Orchestration"]
A1["Gap Analysis Input Validation"]
A2["Broken vs. Non-Broken Sales Analysis"]
A3["Ideal Size Set (ISS) Comparison"]
A4["Never-Out-Of-Stock (NOOS) Gap Analysis"]
A0 -- "orchestrates" --> A1
A0 -- "orchestrates" --> A4
A0 -- "orchestrates" --> A3
A0 -- "orchestrates" --> A2
A1 -- "validates" --> A4
A1 -- "validates" --> A3
A1 -- "validates" --> A2
A4 -- "data_flow" --> A2
```

Chapters

This document outlines the **Master Report** for the merchandising algorithm, detailing its purpose, architecture, functionality, and usage.

1. Title & Metadata

- **Project Name:** gap
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** An advanced merchandising algorithm designed to optimize product assortment and inventory levels by providing data-driven recommendations on stock depth (OD), category breadth (OW), and ideal size mixes (ISS) to maximize sales and minimize waste.

2. Executive Summary

The merchandising algorithm is a critical tool for Gap’s inventory and assortment planning teams. It leverages historical sales data, product attributes, and market trends to generate actionable insights and recommendations. The primary goal is to enhance merchandising efficiency, reduce stockouts, optimize inventory turns, and ultimately drive higher profitability.

- **Key outcomes or KPIs:**
 - Increased sales through optimized product availability.
 - Reduced inventory holding costs by minimizing overstock.
 - Improved customer satisfaction by ensuring popular items and sizes are in stock.
 - Enhanced decision-making for assortment planning, leading to better space utilization and category performance.
 - Accurate predictions for **ROS** across various product categories and sizes.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Planning, Store Operations, E-commerce Merchandising, Supply Chain Management, Finance.

3. Architecture & Data Flow

Diagram

graph TD
A[Raw Sales & Inventory Data] --> B{Data Ingestion & Validation};

```

B --> C[Historical Product Catalog];
B --> D[Store Characteristics];

C --> E[Data Pre-processing & Feature Engineering];
D --> E;

E --> F[ROS Forecasting Module];
E --> G[Attribute Group Analysis (AG)];

F --> H[Optimum Depth (OD) Calculation];
F --> I[Optimum Width (OW) Calculation];
G --> J[Ideal Size Set (ISS) Determination];

H --> K[Recommendation Engine];
I --> K;
J --> K;

K --> L[Output Reports & APIs];
L --> M[Merchandising & Planning Systems];
L --> N[Analytics Dashboards];

subgraph Core Algorithm
    F; G; H; I; J; K
end

```

Component Bullets

- **Data Ingestion & Validation:** Collects and cleans raw sales, inventory, and product master data from various sources.
- **Data Pre-processing & Feature Engineering:** Transforms raw data into a structured format, creating features relevant for analysis (e.g., seasonal indicators, price elasticity metrics).
- **ROS Forecasting Module:** Predicts future sales velocity for SKUs and AGs based on historical performance, seasonality, and other factors.
- **Attribute Group Analysis (AG):** Segments products by predefined AGs (e.g., Color, Material, Fit) to understand their individual performance characteristics.
- **Optimum Depth (OD) Calculation:** Determines the ideal quantity of specific SKUs to stock to meet demand without excessive inventory.
- **Optimum Width (OW) Calculation:** Recommends the optimal number of distinct product categories or styles to offer based on market demand and profitability.
- **Ideal Size Set (ISS) Determination:** Identifies the most effective distribution of sizes within a product category to maximize sales and minimize stockouts.
- **Recommendation Engine:** Consolidates outputs from OD, OW, and ISS modules, applying business rules and constraints to generate final, actionable merchandising recommendations.
- **Output Reports & APIs:** Delivers recommendations in user-friendly reports and exposes them via APIs for integration with downstream systems.

High-level data sources

- **Transactional Sales Data:** POS data, e-commerce order history (units sold, revenue, date, store, SKU ID).
- **Inventory Data:** Current stock levels, in-transit inventory, purchase orders.
- **Product Master Data:** SKU attributes (category, sub-category, brand, material, color, size, price), product lifecycle status.
- **Store Data:** Store type, square footage, geographic location, historical sales performance.
- **External Data:** Seasonal calendars, holiday schedules, market trends (optional, for future enhancements).

4. Configuration & Data Dictionary

Settings Table (`args.properties`)

Parameter Name	Description	Default Value	Data Type
forecast_horizon_weeks	Number of weeks into the future for sales forecasting.	12	Integer
min_ros_threshold	Minimum ROS (units/week) for a SKU to be considered viable.	0.5	Float
inventory_buffer_percent	Additional inventory percentage to account for demand variability.	0.15	Float
od_optimization_method	Method for OD calculation (e.g., 'safety_stock', 'service_level').	'service_level'	String
ow_category_profit_min	Minimum profit margin for a category to be considered for OW expansion.	0.20	Float
iss_sales_contribution_min	Minimum sales contribution % for a size to be included in ISS .	0.02	Float
data_freshness_days	Maximum age of historical data	90	Integer

Parameter Name	Description	Default Value	Data Type
	allowed for analysis.		
output_format	Desired format for output reports (e.g., 'CSV', 'JSON', 'Excel').	'Excel'	String
debug_mode_enabled	Toggles detailed logging for debugging purposes.	false	Boolean

Shared Data Fields (common-data.json)

- sku_id: Unique identifier for a Stock Keeping Unit.
- category_id: Identifier for the product category (e.g., 'Men's T-Shirts').
- subcategory_id: Identifier for the product sub-category (e.g., 'Graphic Tees').
- store_id: Unique identifier for a retail store or online channel.
- date: Date of the transaction or data point (YYYY-MM-DD).
- units_sold: Number of units sold.
- revenue: Total revenue generated.
- current_stock: Available units in inventory.
- size: Product size (e.g., 'S', 'M', 'L', 'XL', '28W-30L').
- color: Product color.
- material: Product material.
- ros_units_per_week: Calculated **Rate of Sale** for a specific SKU/location/time.

Outputs Catalog

Output Name	Description	Format	Key Fields
od_recommendations.csv	Recommended Optimum Depth for each SKU per store.	CSV	store_id, sku_id, `OD_recommended_units, forecasted_ROS
ow_recommendations.json	Recommended Optimum Width by category/style per store.	JSON	store_id, category_id, OW_recommended_styles, justification
iss_recommendations.xlsx	Ideal Size Set distribution for key product categories.	Excel	category_id, product_id, size_distribution_percent, recommended_sizes
ros_forecasts.csv	SKU-level Rate of Sale forecasts.	CSV	store_id, sku_id, forecast_date, predicted_ROS

Output Name	Description	Format	Key Fields
attribute_group_performance.csv	Performance metrics aggregated by AGs.	CSV	attribute_group_name, value, total_sales, avg_R
inventory_optimization_report.pdf	Comprehensive report on inventory health and potential savings.	PDF	High-level summary of all recommendations

5. End-to-End Flow

The merchandising algorithm’s flow begins with the ingestion of diverse datasets, including historical sales, current inventory, and comprehensive product master data. This raw data undergoes rigorous validation and pre-processing to ensure accuracy and consistency. Key features such as seasonality, promotional impacts, and product lifecycles are engineered to enrich the dataset.

Next, the **ROS Forecasting Module** predicts the future sales velocity for every SKU across all stores, forming the foundation for subsequent calculations. Simultaneously, the **Attribute Group Analysis** identifies performance trends across various product attributes.

These forecasts and analyses feed into the core optimization modules: 1. **Optimum Depth (OD) Calculation**: Determines the ideal stock quantity for each unique SKU at every location, balancing demand satisfaction with inventory cost. 2. **Optimum Width (OW) Calculation**: Recommends the optimal number of distinct categories or styles to offer within a store or channel, ensuring breadth without dilution. 3. **Ideal Size Set (ISS) Determination**: Establishes the perfect mix of sizes for specific product lines, based on historical demand patterns and minimizing stockouts.

Finally, the **Recommendation Engine** synthesizes these outputs, applies predefined business rules (e.g., minimum order quantities, warehouse capacities), and generates concrete, actionable recommendations for merchandising and planning teams. These recommendations are then disseminated via structured reports and APIs, ready for integration into existing systems and real-time decision-making.

6. Submodule Overview

6.1 Data Ingestion & Pre-processing

- **Purpose:** To collect, clean, and transform raw sales, inventory, and product data into a standardized format ready for analysis.

- **Inputs & Outputs:**
 - **Inputs:** Raw sales data (e.g., CSV, database dump), raw inventory data, product master files.
 - **Outputs:** Cleaned, standardized dataframes (e.g., `processed_sales_df`, `processed_inventory_df`, `enriched_product_df`).
- **Core Logic Summary:**
 - Load data from various source systems, handling different formats.
 - Perform data cleaning, including missing value imputation, outlier detection, and data type conversion.
 - Merge disparate datasets based on common keys (e.g., `sku_id`, `store_id`, `date`).
 - Generate time-based features like `day_of_week`, `month`, `is_holiday` for subsequent modules.

6.2 ROS Forecasting Module

- **Purpose:** To accurately predict the future **Rate of Sale (ROS)** for individual SKUs at specific locations.
- **Inputs & Outputs:**
 - **Inputs:** `processed_sales_df` (historical sales data), `enriched_product_df` (product attributes).
 - **Outputs:** `ros_forecasts.csv` (`sku_id`, `store_id`, `forecast_date`, `predicted_ROS`).
- **Core Logic Summary:**
 - Calculate historical **ROS** for each SKU by dividing total units sold by active selling weeks.
 - Apply time-series forecasting models (e.g., ARIMA, Prophet, or machine learning models like XGBoost) using historical **ROS**, seasonality, and promotional data.
 - Generate future **ROS** predictions for the defined `forecast_horizon_weeks`.
 - Validate forecast accuracy against hold-out data.

6.3 Optimum Depth (OD) Calculation

- **Purpose:** To determine the ideal stock quantity (Optimum Depth) for each SKU to meet forecasted demand while minimizing inventory costs.
- **Inputs & Outputs:**
 - **Inputs:** `ros_forecasts.csv`, `processed_inventory_df` (current stock), `args.properties` (e.g., `inventory_buffer_percent`).
 - **Outputs:** `od_recommendations.csv` (`store_id`, `sku_id`, `OD_recommended_units`, `forecasted_ROS`).
- **Core Logic Summary:**
 - Retrieve the forecasted **ROS** for each SKU per store for the upcoming period.
 - Calculate required inventory based on forecasted **ROS** and lead times for replenishment.
 - Apply an `inventory_buffer_percent` to account for demand variability and supply chain uncertainties.
 - Round up to the nearest whole unit to determine the final **OD** recommendation.

6.4 Optimum Width (OW) Calculation

- **Purpose:** To recommend the ideal number of different categories or styles (Optimum Width) to offer in a store or channel.
- **Inputs & Outputs:**
 - **Inputs:** `processed_sales_df`, `enriched_product_df` (category data), `args.properties(ow_category_profit_min)`.
 - **Outputs:** `ow_recommendations.json` (`store_id`, `category_id`, `OW_recommended_styles`, `justification`).
- **Core Logic Summary:**
 - Analyze historical sales performance and profitability for each product category/style.
 - Identify categories with high **ROS**, good profit margins, and strong customer engagement.
 - Evaluate market trends and whitespace opportunities.
 - Recommend expanding or contracting the number of styles/categories based on performance and strategic fit, adhering to `ow_category_profit_min`.

6.5 Ideal Size Set (ISS) Determination

- **Purpose:** To identify the best combination of sizes (Ideal Size Set) for specific product lines to maximize sales and minimize stockouts.
- **Inputs & Outputs:**
 - **Inputs:** `processed_sales_df` (sales by size), `enriched_product_df` (product hierarchy), `args.properties(iss_sales_contribution_min)`.
 - **Outputs:** `iss_recommendations.xlsx` (`category_id`, `product_id`, `size_distribution_percent`, `recommended_sizes`).
- **Core Logic Summary:**
 - For each product, calculate the historical sales contribution of each size (S, M, L, etc.).
 - Identify common size curves for product types (e.g., T-shirts, Jeans) across different sales channels.
 - Determine the optimal percentage distribution for each size within a product's **ISS**, ensuring that sizes below `iss_sales_contribution_min` are re-evaluated or de-prioritized.
 - Generate specific size recommendations (e.g., 1S, 2M, 2L, 1XL for a 6-unit pack).

6.6 Recommendation Engine

- **Purpose:** To consolidate, prioritize, and generate final actionable merchandising recommendations based on all upstream module outputs.
- **Inputs & Outputs:**
 - **Inputs:** `od_recommendations.csv`, `ow_recommendations.json`, `iss_recommendations.xlsx`, `processed_inventory_df`, business rules.
 - **Outputs:** `inventory_optimization_report.pdf`, `final_recommendations.csv` (for direct system integration).
- **Core Logic Summary:**
 - Combine **OD**, **OW**, and **ISS** outputs into a single, comprehensive recommendation set.

- Apply business constraints such as minimum order quantities, shelf space limitations, and warehouse capacity.
- Prioritize recommendations based on potential impact (e.g., highest **ROS** items, categories with significant stockouts).
- Format recommendations into user-friendly reports and machine-readable files.

7. Worked Example

Scenario: Optimizing the assortment for “Men’s Classic Cotton T-Shirts” (Category: Men’s Tops, SKU: 12345, Store: Flagship SF)

1. Data Ingestion & Pre-processing: * Historical sales for SKU 12345 in Flagship SF shows consistent sales of 100 units/week for sizes S, M, L, XL over the last 6 months. * Inventory check: Currently 50 units (mixed sizes) in stock.

2. ROS Forecasting Module: * Based on historical sales, seasonality (no major holidays expected), and trend analysis, the predicted_ROS for SKU 12345 in Flagship SF is forecasted at **90 units/week** for the next 12 weeks.

3. Ideal Size Set (ISS) Determination: * Analyzing historical sales for “Men’s Classic Cotton T-Shirts” category: * Size S: 15% * Size M: 35% * Size L: 30% * Size XL: 20% * `iss_sales_contribution_min` is 2%. All sizes meet this. * Recommended **ISS** distribution for a 10-unit replenishment pack: S:1-2, M:3-4, L:3, XL:2 (e.g., 2S, 4M, 3L, 1XL).

4. Optimum Depth (OD) Calculation: * `forecast_horizon_weeks` = 12 weeks. * `predicted_ROS` = 90 units/week. * Total demand for 12 weeks = 90 units/week * 12 weeks = 1080 units. * `inventory_buffer_percent` = 15%. * Buffer units = $1080 * 0.15 = 162$ units. * Required **OD** = $1080 + 162 = 1242$ units (for the 12-week period). * *Per-week OD for reorder trigger:* $(90 \text{ units/week} * 2 \text{ weeks lead time}) + (90 * 0.15 \text{ buffer}) = 180 + 13.5 = \sim 194$ units. * Current stock: 50 units. * **OD_recommended_units** to order (based on current stock and desired end-of-period stock): $1242 - 50 = \mathbf{1192 \text{ units}}$.

5. Optimum Width (OW) Calculation: * This example focuses on a single SKU, but for “Men’s Tops” category, the algorithm would analyze the performance of other styles (e.g., polos, graphic tees, hoodies). If graphic tees have strong **ROS** and meet `ow_category_profit_min`, the **OW** module might recommend increasing the number of graphic tee styles offered in Flagship SF.

6. Recommendation Engine: * **SKU 12345 (Men’s Classic Cotton T-Shirt) - Flagship SF:** * **OD:** Order **1192 units** to cover 12 weeks of forecasted demand including buffer. * **ISS:** For each order of 10 units, maintain a size mix of 2S, 4M, 3L, 1XL. * **OW:** (Contextual, e.g., “Maintain current 5 styles of Classic Cotton T-Shirts; consider adding 2 new graphic tee styles due to high local demand.”) * This recommendation is then formatted into `od_recommendations.csv` and `iss_recommendations.xlsx`, ready for planning teams.

8. Debug Guide Highlights

- **Low ROS Forecasts:**
 - **Check:** `data_freshness_days` in `args.properties`. Is the historical data too old?
 - **Check:** Data gaps or quality issues in `processed_sales_df` for the specific SKU/store.
 - **Check:** Recent promotions or external events not captured or accounted for in feature engineering.
- **Unrealistic OD/OW/ISS Recommendations:**
 - **Check:** `inventory_buffer_percent` or `min_ros_threshold` in `args.properties`. Are they set too high/low?
 - **Check:** Are there any extreme outliers in historical sales data that weren't cleaned in pre-processing?
 - **Check:** For ISS, ensure `iss_sales_contribution_min` is reasonable and not filtering out valid sizes.
 - **Check:** Review business rules in the Recommendation Engine. Are they overly restrictive or contradictory?
- **Algorithm Runs Slowly or Fails:**
 - **Check:** Data volume. Is the input data exceptionally large?
 - **Check:** Resource allocation (CPU, memory) for the environment running the algorithm.
 - **Check:** Log files (`debug_mode_enabled=true`) for specific error messages or bottlenecks in any submodule.
- **Outputs Missing or Incorrect Format:**
 - **Check:** `output_format` in `args.properties`.
 - **Check:** File paths and permissions for writing output files.
 - **Check:** If using APIs, ensure the API endpoint is correct and authentication is valid.

9. Common Pitfalls & Remedies

- **Pitfall: “Garbage In, Garbage Out” (GIGO)** – Poor quality or incomplete input data leads to inaccurate recommendations.
 - **Remedy:** Implement robust data validation checks at the Data Ingestion & Validation stage. Regularly audit source data systems. Establish data governance policies.
- **Pitfall: Ignoring Qualitative Factors** – Over-reliance on quantitative data without considering qualitative market insights (e.g., fashion trends, competitor actions).
 - **Remedy:** Design the Recommendation Engine to allow for manual overrides or to incorporate qualitative inputs from merchandising experts. Conduct regular A/B testing of algorithm recommendations vs. human decisions.
- **Pitfall: Static Parameters** – Using fixed `args.properties` values that don't adapt to changing market conditions or business strategies.
 - **Remedy:** Implement a mechanism for dynamic adjustment of key parameters. Regularly review and update `args.properties` based on performance analysis and business objectives. Consider A/B testing different parameter sets.

- **Pitfall: Over-Optimization for One Metric** – Focusing solely on maximizing ROS without considering inventory costs or customer satisfaction.
 - **Remedy:** Ensure the objective function in the Recommendation Engine is balanced across multiple KPIs (e.g., sales, profit, inventory turn, stockout rate). Use a weighted approach.
- **Pitfall: Lack of User Adoption** – Recommendations are not trusted or used by merchandising teams.
 - **Remedy:** Provide clear, transparent justifications for each recommendation. Offer training and support to users. Involve stakeholders early in the algorithm development process to build trust and address concerns.

10. FAQ

- How often are the recommendations updated?**
 - Recommendations are generated on a weekly basis by default, but this frequency can be configured based on business needs and data availability.
- Can the algorithm account for new product launches or promotions?**
 - Yes, the Data Pre-processing & Feature Engineering module can incorporate flags for new products and promotional periods. For new products, initial recommendations might rely on similar product historical data.
- What if a store has unique characteristics (e.g., high tourist traffic)?**
 - The Store Characteristics data source allows the ROS Forecasting Module to model store-specific demand patterns. The Recommendation Engine can also apply store-specific rules if needed.
- How accurate are the ROS forecasts?**
 - Forecast accuracy is regularly monitored against actual sales. Historical performance typically shows an average error rate of 10-15% at the SKU-store level, varying by category and seasonality.
- Is it possible to simulate different scenarios before applying recommendations?**
 - Currently, the algorithm provides direct recommendations. Future enhancements could include a simulation interface allowing users to test different assumptions or constraints.
- What is the difference between OD and current inventory?**
 - **OD** is the *ideal* number of units recommended to stock to meet forecasted demand. Current inventory is the *actual* number of units physically in the store at a given time. The algorithm recommends actions to bring current inventory closer to the OD.

11. Quick Reference Cheat-Sheet

Abbreviation	Business Definition	Key Output	Primary Module
OD	Optimum Depth: The ideal number of	OD_recommended_units	Optimum Depth (OD) Calculation

Abbreviation	Business Definition	Key Output	Primary Module
	unique items (SKUs) to stock in a category or display.		
OW	Optimum Width: The ideal number of different categories or styles to offer.	OW_recommended_styles	Optimum Width (OW) Calculation
ISS	Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts.	size_distribution_percent, recommended_sizes	Ideal Size Set (ISS) Determination
ROS	Rate of Sale: The speed at which products are sold over a period (e.g., units/week).	predicted_ROS	ROS Forecasting Module
AG	Attribute Group: A set of product attributes grouped for analysis (e.g., color, size, brand).	attribute_group_performance	Attribute Group Analysis (AG)

Key Configuration Parameters (args.properties): *

- forecast_horizon_weeks: Defines the look-ahead period for predictions. *
- inventory_buffer_percent: Adds a safety margin to inventory recommendations.
- * min_ros_threshold: Filters out underperforming SKUs from consideration.

Common Outputs: * od_recommendations.csv * ow_recommendations.json * iss_recommendations.xlsx * ros_forecasts.csv * inventory_optimization_report.pdf

Chapter 2: Impact Analysis Sync Module

Last Updated: 2025-05-23

1. Business Objective

This step prepares all necessary retail data for detailed analysis by ensuring every subsequent module operates on a fresh, complete, and accurate dataset.

Why This Matters: This ensures that all insights and recommendations, from optimizing stock levels to planning promotions, are based on the most current and comprehensive view of our business. This helps us make smarter decisions that improve sales, reduce unnecessary inventory, and enhance the customer experience.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- Sales Records (Units Sold, Revenue, Dates): To understand how quickly products are selling (Rate of Sale) and identify sales trends over time, crucial for determining demand.
- Discounting Analysis Inputs: To understand the impact of past promotions and price changes on sales, which helps fine-tune future pricing strategies.

Product Data

- Individual Product Details (SKUs, descriptions, colors, sizes): To uniquely identify every item we sell and understand its specific attributes, essential for building the Ideal Size Set (ISS).
- Product Style Information (Categories, collections, brands): To group similar products together for analysis, allowing us to understand performance across broader categories or discover Optimum Width (OW).

Operational Data

- Store Locations: To understand where products are sold and analyze performance differences across various retail locations.

- Inventory Snapshots (Current stock levels, locations, dates): To know exactly what we have on hand, where it is, and when it was counted, which is vital for preventing stockouts and overstocking.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Data Freshness Target	24 Hours	Specifies the maximum age allowed for the loaded data before it's considered outdated.	If real-time responsiveness for fast-moving items is critical (e.g., set to 1 hour), or if data updates are less frequent (e.g., weekly).
Sync Schedule	Daily at 2 AM	Defines how often the system refreshes its entire dataset.	To align with business peak hours (avoiding sync during high transaction periods) or if business operations require more frequent updates (e.g., every 6 hours).
Error Tolerance Level	Low	Determines how many minor data inconsistencies or missing records the system can tolerate before stopping the sync process.	If data quality is highly variable (allowing more tolerance to complete the sync) or if absolute data purity is paramount (requiring zero tolerance).
Historical Data Window	12 Months	Sets the timeframe for how much past sales and inventory data should be loaded into the system.	To capture longer-term trends (e.g., 24 months for seasonal items) or to focus on recent performance (e.g., 3 months for fashion items with short lifecycles).

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Input Data] --> B[Main Process]
B --> C[Output Result]

1. Start with → Raw, historical, and real-time retail data from various source systems (e.g., point-of-sale, warehouse management).
2. Then → Clear all previously loaded data from the analysis system's memory.
3. Next → Load fresh master data (SKUs, styles, stores) and transactional data (sales, inventory key frames, suggested and implemented Ideal Size Sets, discounting inputs) into the in-memory database.
4. Finally → A fully synchronized and ready-to-analyze dataset, prepared for subsequent modules to calculate Optimum Depth (OD), Optimum Width (OW), Rate of Sale (ROS), and Ideal Size Set (ISS).

5. Step-by-Step Actions

1. **Clear** existing data
 - What we do: *We remove all previously loaded information from our in-memory analysis system.*
 - Why we do it: *This ensures that every analysis starts with a clean slate, preventing old or irrelevant data from skewing current results and ensuring that all calculations for OD, OW, ISS, and ROS are based on the latest facts.*
2. **Load** new data
 - What we do: *We pull in the latest retail data, including recent sales records, current inventory counts, detailed product information (like color and size), and any historical Ideal Size Set (ISS) suggestions and actual implementations.*
 - Why we do it: *This provides a complete and up-to-date picture of the entire business, essential for accurate and timely decision-making regarding inventory management, product assortment, and promotional strategies.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Our goal is to optimize the assortment of denim jeans for the upcoming season, particularly focusing on finding the Ideal Size Set (ISS) and determining the Optimum Depth (OD) for new styles.
- The system currently holds sales data up to last month and inventory snapshots from two weeks ago.
- New data has just arrived: the latest sales figures from all stores for the past week, current inventory counts taken yesterday, and updated product descriptions for some newly arrived denim styles.

What Happens

1. First, the system automatically clears out all the denim sales records, inventory counts, and product details that were loaded from previous syncs. This ensures no outdated information remains.
2. Then, it swiftly fetches and loads all the fresh data. This includes every sale of denim jeans made in the last seven days, the most recent inventory numbers showing exactly how many of each size and style are in stock at every store, and any new attributes for the denim styles that were recently added to our catalog. It also loads historical data on past suggested and implemented Ideal Size Sets (ISS) for similar denim lines.
3. Finally, the system confirms that all these fresh pieces of information—from every unit sold to every item in stock—are now fully loaded and ready for the next modules to begin their analysis, such as calculating the precise Rate of Sale (ROS) for each denim SKU.

Final Outcome

The analysis system is now armed with the most current and complete picture of our denim inventory and sales, ready to provide accurate insights for optimal inventory management, precise Ideal Size Set (ISS) recommendations, and smart assortment planning.

Key Takeaway: A fresh data foundation is crucial for making effective, data-driven decisions that directly impact profitability by ensuring our shelves are stocked with the right products in the right sizes.

7. Results

This step produces:

Main Outputs

- Clean, ready-to-use dataset: This comprehensive dataset forms the accurate foundation for all subsequent analyses, ensuring reliable insights for determining Optimum Depth (OD), Optimum Width (OW), Ideal Size Sets (ISS), and Rate of Sale (ROS) calculations.
- System readiness confirmation: This confirms that the data environment is fully prepared for the next analytical steps, indicating no missing crucial information and a green light for further processing.

Business Reports

- Data Sync Status Report: This report is used by IT Operations and Data Stewards to confirm that the data loading process completed successfully and on schedule, ensuring data integrity.
- Data Freshness Overview: This overview is vital for Merchandising Planners and Business Analysts, assuring them that the data they are working with is current and reliable for making crucial merchandising decisions, such as adjusting Optimum Depth (OD) or Ideal Size Sets (ISS).

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The analysis modules will either use outdated data, leading to incorrect recommendations, or fail entirely due to incomplete information.	Alert the IT or Data team immediately to investigate the source of the delay and manually trigger a data refresh once the complete data is available.
Corrupted or erroneous input data	Analysis results may be inaccurate or misleading, potentially leading to poor inventory decisions, such as ordering too much or too little, impacting Ideal Size Sets (ISS) or Rate of Sale (ROS) calculations.	Data validation checks should flag these issues; the data cleansing team will need to correct the errors, and a full re-sync will be required.
System memory limitations reached	The module might fail to load all necessary data, leading to incomplete datasets for analysis and potentially causing errors in subsequent calculations for Optimum Depth (OD) or Optimum Width (OW).	Review the total volume of data being loaded, optimize memory allocation for the system, or consider strategies for splitting very large data loads into smaller, manageable chunks.

9. Next Steps

Before This Step

- Raw Data Collection: What we need from it: All historical sales transactions, current inventory levels, and comprehensive product master data (like attributes for AG) from various source systems (e.g., point-of-sale, ERP, warehouse).
- Data Export Preparation: What we need from it: Data transformed, cleaned, and formatted in a structured way that allows for efficient loading into the analysis system, ensuring consistency and accuracy.

After This Step

- Demand Forecasting Module: What they need from us: A clean, complete dataset of sales history, current inventory, and product information to accurately

predict future demand, which directly informs Optimum Depth (OD) and Ideal Size Set (ISS).

- **Inventory Optimization Module:** What they need from us: Current inventory key frames, detailed sales data, and product attributes to recommend ideal stock levels, store transfers, and reorder quantities, aiming to achieve the Ideal Size Set (ISS) across locations.
- **Attribute Group (AG) Analysis:** What they need from us: Complete SKU and Style information, including all relevant product attributes, to analyze product performance and customer preferences by specific characteristics like color, material, or brand.

10. Common Questions

Q: Why do we clear all data before loading new data? A: Clearing ensures that our analysis always starts with the freshest and most accurate information. This prevents old or incorrect data from skewing our insights for Optimum Depth (OD), Optimum Width (OW), Ideal Size Sets (ISS), and Rate of Sale (ROS), leading to better merchandising decisions.

Q: How often is this data synchronization process run? A: This process typically runs automatically on a scheduled basis, often daily or weekly, to ensure that the merchandising and planning teams always have access to the most up-to-date retail data for their analyses.

Q: What happens if some data, like sales records for a specific store, is missing during the sync? A: If crucial data is missing, the system will alert the technical team. While some analyses might proceed with partial data, critical calculations for Optimum Depth (OD), Optimum Width (OW), or Ideal Size Sets (ISS) might be inaccurate or flagged as unreliable until all necessary data is loaded and verified.

Q: Can I directly access the raw data after it's loaded by this module? A: The data loaded by this module is primarily for the system's internal analysis modules to perform complex calculations. However, comprehensive business reports generated by subsequent modules will summarize and present key insights derived from this data in an easily understandable format for business users.

Q: How does this process help us avoid common retail issues like stockouts or overstocking? A: By ensuring that all analysis begins with current inventory and sales data, this module provides the foundational accuracy needed for subsequent modules to precisely recommend Ideal Size Sets (ISS) and calculate accurate Rate of Sale (ROS), directly helping us optimize stock levels and avoid common merchandising issues like having too many small sizes and too few large ones.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Ensures all retail analysis starts with a clean, complete, and up-to-date dataset.
- **Main Input:** All master and transactional retail data, including sales records, inventory key frames, product attributes, and past Ideal Size Set (ISS) recommendations.

- **Main Output:** A fully initialized and ready-to-analyze in-memory database, serving as the single source of truth for subsequent calculations like Optimum Depth (OD) and Rate of Sale (ROS).
 - **Warning Signs:** Delays in overall report generation, inconsistent data across different analyses or reports, or receiving “data too old” alerts from the system.
 - **Key Settings:** Data Freshness Target, Sync Schedule, Error Tolerance Level, Historical Data Window.
-

Need Help? Contact your IT Support team or Data Operations for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 3: Impact Analysis Data Module

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for the foundational data preparation required for detailed impact analysis, including initializing and filtering essential product, style, and store information, and accurately computing inventory levels and “live days” (the number of days a product SKU was available in a particular store).

Why This Matters: This foundational accuracy ensures that all subsequent analyses, such as calculating the Rate of Sale (ROS) or determining Ideal Size Sets (ISS), are based on a precise understanding of product availability, leading to more reliable recommendations for inventory management, assortment planning (Optimum Depth/Optimum Width), and ultimately, improved sales and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- Sales Records (Units Sold, Revenue, Dates): To understand how many units of a product sold on specific days, which is crucial for accurately computing inventory levels and availability over time, and for calculating the Rate of Sale (ROS).
- Discounting Analysis Inputs: To understand how sales were influenced by price changes or promotions, ensuring that sales volume is interpreted correctly alongside product availability.

Product Data

- Individual Product Details (SKUs, descriptions, colors, sizes): To uniquely identify every item, filter irrelevant products, and understand specific attributes that are essential for defining Attribute Groups (AG) and Ideal Size Sets (ISS).
- Product Style Information (Categories, collections, brands): To group similar products, allowing for broader analysis of performance trends and helping to define Optimum Width (OW) across different categories.

- **Size Mapping Details:** To standardize and normalize product sizes (e.g., ‘S’, ‘Small’, ‘Sm’ all map to ‘S’), ensuring consistent analysis across different product descriptions and helping to form accurate Ideal Size Sets (ISS).

Operational Data

- **Store Locations:** To understand where products are sold and to process sales and inventory data specific to each retail location.
- **Inventory Snapshots** (Current stock levels, locations, dates): To know exactly what was on hand, where it was, and when, which is vital for calculating the “live days” (product availability) for each SKU in every store.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Start Date	(Varies, e.g., 6 months ago)	Defines the beginning of the period for which sales and inventory data will be analyzed.	To focus on more recent performance (e.g., last 3 months) or to capture longer-term trends (e.g., last 12-24 months for seasonal items).
Analysis End Date	Today’s Date	Defines the end of the period for which sales and inventory data will be analyzed.	To capture sales up to a specific point in time (e.g., end of last quarter) or to exclude very recent data if it’s still volatile.
Week Start Day	Monday	Specifies which day of the week marks the beginning of a new week for sales and inventory computations.	To align with internal reporting cycles or if business operations are structured around a different weekly start day.

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Raw & Synced Retail Data] --> B[Impact Analysis Data Module]
B --> C[Refined & Computed Analysis Data]

1. Start with → Raw, historical, and real-time retail data that has been synchronized and cleared by the previous module (e.g., sales, inventory snapshots, product attributes, store details).
2. Then → **Initialize** master data (SKU, Style, Store maps) and **filter** it based on pre-defined criteria (e.g., only active products).
3. Next → **Filter** transactional data (sales, inventory key frames) to align with the initialized master data.
4. Finally → **Compute** product inventory levels and “live days” (product availability) for each SKU in each store, providing a precise foundation for metrics like Rate of Sale (ROS).

5. Step-by-Step Actions

1. **Initialize** and **Filter** Master Data

- What we do: *We load comprehensive product (SKU) details, style information (categories, brands), and store locations. We then filter this data to ensure only relevant and active items are included in our analysis.*
- Why we do it: *This creates a clean, accurate foundation for our product assortment and store network, removing any outdated or irrelevant entries that could skew results for Optimum Depth (OD), Optimum Width (OW), or Ideal Size Set (ISS).*

2. **Load** and **Filter** Transactional Data

- What we do: *We bring in the raw sales records and inventory snapshots. This transactional data is then cross-referenced and filtered against our initialized master data to ensure consistency.*
- Why we do it: *This step guarantees that all sales and inventory figures correspond to actual products and stores we intend to analyze, preventing errors from mismatched data and preparing for accurate Rate of Sale (ROS) calculations.*

3. **Compute** Inventory Levels

- What we do: *Using the filtered sales and inventory snapshots, we calculate the precise quantity of each product SKU available in each store on any given day within the analysis period.*
- Why we do it: *Accurate daily inventory levels are fundamental for understanding true product availability and are a critical input for calculating “live days” and effective Rate of Sale (ROS).*

4. **Calculate** “Live Days”

- What we do: *For every product SKU in every store, we determine the exact number of days it was available for purchase during the analysis period, accounting for periods when it was out of stock or not yet received.*
- Why we do it: *This is crucial for an accurate Rate of Sale (ROS). If a product sold 100 units in 30 days but was only in stock for 15 of those days, its true sales potential (ROS) is much higher than a product that sold 100 units while being in stock for all 30 days.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Our goal is to understand the true Rate of Sale (ROS) for a popular men's denim jean (SKU 7890) in our flagship store over the past 90 days.
- The raw data shows 270 units of SKU 7890 were sold in the flagship store over this 90-day period.
- Initial inventory snapshots indicate the store had stock at the beginning of the period, but some sales records suggest intermittent stockouts.

What Happens

1. First, the Impact Analysis Data Module initializes all master data, confirming SKU 7890 is an active product and the flagship store is valid. It then loads and filters all 90 days of sales records for SKU 7890 in that store, along with all relevant daily inventory snapshots.
2. Then, it meticulously computes the daily inventory levels for SKU 7890 at the flagship store, tracking how many units were available each day.
3. Next, based on these daily inventory levels, the module precisely calculates the "live days" for SKU 7890. It discovers that, while the analysis period was 90 days, SKU 7890 was actually out of stock for 30 of those days due to high demand and shipping delays. Therefore, it was only "live" (available for sale) for 60 days.
4. Finally, the module provides these accurate "live days" (60 days) and the computed inventory levels, ready for the next module to calculate the true Rate of Sale (ROS).

Final Outcome

The system now has the accurate "live days" (60 days) for SKU 7890 in the flagship store. This crucial piece of information allows subsequent modules to calculate a precise Rate of Sale ($270 \text{ units} / 60 \text{ live days} = 4.5 \text{ units/day}$), providing a much more realistic picture of its actual sales performance when available.

Key Takeaway: Accurately calculating "live days" is essential for understanding a product's true sales potential and preventing misleading Rate of Sale (ROS) figures that could lead to understocking popular items.

7. Results

This step produces:

Main Outputs

- **Filtered & Initialized Master Data:** A clean, validated set of SKU, Style, and Store information, ready for analysis, ensuring that all subsequent calculations

like Optimum Depth (OD) and Optimum Width (OW) are based on accurate product and location details.

- Computed Inventory Levels: Precise daily on-hand quantities for each SKU in every store, providing a granular view of stock availability.
- “Live Days” Data: The exact number of days each SKU was available for sale in each store, which is a critical input for accurate Rate of Sale (ROS) calculations.

Business Reports

- Data Initialization Summary: Used by Data Analysts and Merchandising Planners to confirm that all required product and store data has been successfully processed and filtered, ensuring a reliable foundation for their analyses.
- Inventory Availability Overview: Used by Inventory Managers and Merchandising teams to get a high-level understanding of how consistently products were available, highlighting potential areas for improving stock-in-transit processes or Ideal Size Set (ISS) allocations.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will operate on incomplete or outdated data, potentially leading to inaccurate inventory levels and “live days” calculations, thus affecting ROS and Ideal Size Set (ISS) recommendations.	Alert the IT or Data Operations team immediately to investigate the source of the delay and trigger a data refresh once complete data is available.
Inconsistent Inventory Snapshots (e.g., negative stock)	The computation of daily inventory levels and “live days” may become incorrect, misrepresenting actual product availability and skewing Rate of Sale (ROS) calculations.	The data cleansing team must correct the erroneous inventory records in the source systems, and a full re-computation for the affected period will be necessary.
Product SKU missing from Master Data but present in Sales	Sales for these products will be ignored or flagged as unidentifiable, leading to an underestimation of total sales and an incomplete picture of product performance.	The product data team needs to add the missing SKU to the master data, ensuring all product information is complete, followed by a re-processing of the module.
	“Live days” calculation might be less accurate	Review the data collection process to ensure all sales

Situation	What Happens	What You Should Do
Significant gaps in historical sales data	as it cannot fully account for periods of availability if sales activity is missing, impacting the precision of Rate of Sale (ROS).	transactions are captured. If data loss occurred, consider using a shorter historical data window for analysis.

9. Next Steps

Before This Step

- **Raw Data Collection:** What we need from it: All historical sales transactions, current inventory levels, and comprehensive product master data (like attributes for AG) from various source systems (e.g., point-of-sale, ERP, warehouse).
- **Impact Analysis Sync Module:** What we need from it: A clean, complete, and fully synchronized dataset of all retail information, cleared and loaded into the analysis system's memory, forming the direct input for this module.

After This Step

- **Demand Forecasting Module:** What they need from us: Accurately computed "live days" and sales data to predict future demand more precisely, which directly informs Optimum Depth (OD) and Ideal Size Set (ISS) for upcoming seasons.
- **Inventory Optimization Module:** What they need from us: The precise inventory levels and "live days" for each SKU-store combination, enabling them to recommend optimal stock levels, store transfers, and reorder quantities, aiming to achieve the Ideal Size Set (ISS) across locations and improve sell-through.
- **Rate of Sale (ROS) Calculation Module:** What they need from us: The number of "live days" and filtered sales data, which are essential to compute an accurate Rate of Sale for each product, guiding merchandising and replenishment decisions.

10. Common Questions

Q: Why are "live days" crucial for understanding product performance? **A:** "Live days" tell us how many days a product was actually available for customers to buy. Without it, if a product sold 100 units in 30 days but was out of stock for 15 of those days, a simple calculation would underestimate its true sales speed. Knowing it was only available for 15 days reveals a much higher sales potential, helping us prioritize popular items that might be constantly selling out.

Q: How does filtering master data improve analysis? **A:** Filtering master data removes irrelevant or outdated products and styles (e.g., discontinued items or test products not meant for full analysis). This ensures that all subsequent calculations, such as Optimum Depth (OD) or Optimum Width (OW), focus only on the relevant product assortment, making the insights more actionable and accurate for current business decisions.

Q: What happens if inventory levels reported are inaccurate? A: Inaccurate inventory levels (e.g., showing stock when there's none, or vice versa) will lead to incorrect "live days" calculations. This can result in misleading Rate of Sale (ROS) figures, causing us to overstock slow-selling items or miss opportunities by understocking popular products, directly impacting profitability.

Q: Can I see the computed inventory levels or live days directly from this module?

A: While this module processes and stores these granular details internally for subsequent analysis, comprehensive business reports generated by later modules will summarize and present key insights derived from this data (e.g., average inventory levels, or products with low availability) in an easily understandable format for business users.

Q: How does accurate data from this module help optimize product assortment? A: By providing precise "live days" and inventory levels, this module lays the groundwork for accurate Rate of Sale (ROS) calculations. These accurate ROS figures then directly inform decisions about Optimum Depth (OD) (how many items to stock per style), Optimum Width (OW) (how many styles to offer), and Ideal Size Set (ISS) (the best combination of sizes), ensuring we stock the right products in the right quantities.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prepares foundational data for impact analysis, focusing on accurate inventory levels and "live days."
- **Main Input:** Fully synchronized raw sales records, inventory snapshots, and product/store master data.
- **Main Output:** Filtered master data, computed daily inventory levels, and "live days" per SKU-store combination.
- **Warning Signs:** Discrepancies between reported stock and actual sales potential, or analyses showing unusually low Rate of Sale for popular products.
- **Key Settings:** Analysis Start Date, Analysis End Date, Week Start Day.

Need Help? Contact your IT Support team or Data Operations for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: Impact Analysis IST Module

Last Updated: 2025-05-23

1. Business Objective

This module measures how effective Inter-Store Transfers (ISTs) are by comparing product performance before and after inventory moves, analyzing both planned and actual transfer quantities.

Why This Matters: It ensures that inventory reallocation decisions directly boost sales and minimize stockouts, optimizing product availability where demand is highest and improving overall profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- Sales Records (Units Sold, Revenue, Dates): To accurately measure how many units of a specific product sold in a store during defined pre-transfer and post-transfer periods, crucial for calculating the Rate of Sale (ROS).
- Discounting Analysis Inputs: To understand if any sales shifts were influenced by promotions, ensuring a clearer picture of organic demand versus price-driven sales.

Product Data

- Individual Product Details (SKUs, descriptions, colors, sizes): To uniquely identify every item involved in a transfer and track its specific performance.
- Product Style Information (Categories, collections, brands, fashion grades, seasons, gender): To provide broader context for the analysis, helping identify trends across product types or for reporting purposes.

Operational Data

- Store Locations: To identify the sending and receiving stores for each transfer, enabling performance analysis by location.
- Suggested Inter-Store Transfers (quantities, from/to store, SKU, date): To understand the initially planned inventory movements.

- Implemented Inter-Store Transfers (actual quantities transferred, from/to store, SKU, date): To track the actual inventory movements that occurred.
- Pre-Transfer Inventory Levels: To establish the stock baseline at each store before any transfers took place.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Pre-Transfer Analysis Start Date	(Varies, e.g., 30 days prior to transfer)	Defines the beginning of the period for analyzing product performance <i>before</i> any inter-store transfers.	To focus on a shorter, more recent period before transfers, or to capture a longer historical baseline.
Pre-Transfer Analysis End Date	(Varies, e.g., Day before transfer)	Defines the end of the period for analyzing product performance <i>before</i> any inter-store transfers.	To precisely define the pre-transfer window relative to the actual transfer date.
Post-Transfer Analysis Start Date	(Varies, e.g., Day of transfer)	Defines the beginning of the period for analyzing product performance <i>after</i> any inter-store transfers.	To immediately assess the impact from the day of transfer, or to allow for a buffer period before measuring post-transfer sales.
Post-Transfer Analysis End Date	(Varies, e.g., 30 days after transfer)	Defines the end of the period for analyzing product performance <i>after</i> any inter-store transfers.	To capture a longer period of post-transfer sales impact, or to focus on immediate short-term changes.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Synced Retail Data (Sales, Inventory, Product, Store,

Suggested/Implemented ISTs)] --> B[Impact Analysis IST Module]

B --> C[Detailed IST Impact Metrics]

1. Start with → Sales records, inventory snapshots, product details, store information, and both suggested and implemented inter-store transfer data for all relevant periods.
2. Then → **Segment** sales data into “pre-transfer” and “post-transfer” periods for each SKU-store combination.
3. Next → **Calculate** key performance metrics like Rate of Sale (ROS), total sales units, and “live days” for each SKU-store combination within both the pre- and post-transfer periods.
4. Finally → **Generate** detailed output that combines these performance metrics with suggested and implemented transfer quantities, providing a comprehensive view of the impact.

5. Step-by-Step Actions

1. **Segment** sales data for analysis
 - What we do: *We divide all available sales records into distinct ‘pre-transfer’ and ‘post-transfer’ periods for every specific product (SKU) in each store, based on defined analysis dates.*
 - Why we do it: *This allows us to create a clear “before and after” picture, enabling a direct comparison of product performance changes due to inventory reallocation.*
2. **Gather** transfer and stock information
 - What we do: *We collect all records of suggested inter-store transfers, actual implemented transfers, and pre-transfer inventory levels for the products and stores under evaluation.*
 - Why we do it: *This provides the essential context for our analysis, linking performance changes directly to the specific inventory movements that occurred.*
3. **Calculate** performance metrics
 - What we do: *For each relevant product (SKU) in every store, we compute its total sales units, the exact number of “live days” (days it was available for sale), and its Rate of Sale (ROS) for both the pre-transfer and post-transfer periods.*
 - Why we do it: *Accurate calculation of these metrics is vital for understanding the true selling speed and availability of products, ensuring a fair and meaningful comparison of performance.*
4. **Consolidate** and compare data
 - What we do: *We bring together all the calculated performance metrics, transfer details, and stock information for each SKU-store combination into a comprehensive output.*
 - Why we do it: *This consolidation provides a holistic view, allowing merchants and planners to easily see the full impact of transfers on sales, availability, and stock efficiency at a glance.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- Our goal is to evaluate the impact of an inter-store transfer for a popular women's blouse, SKU 4567, from Store B (overstocked) to Store A (understocked).
- Before the transfer, Store A had 5 units of SKU 4567, selling at 2 units/week (ROS), experiencing frequent stockouts. Store B had 20 units of SKU 4567, selling at 1 unit/week.
- The system suggested transferring 10 units from Store B to Store A. After coordination, 7 units were actually implemented and moved.

What Happens

1. First, the module calculates the pre-transfer sales and "live days" for SKU 4567 at both Store A and Store B over a two-week period. It confirms Store A's ROS was 2 units/week with low live days, and Store B's ROS was 1 unit/week with high live days.
2. Then, it records the suggested transfer of 10 units from B to A and the actual implemented transfer of 7 units.
3. Next, for a four-week post-transfer period, the module meticulously tracks sales and "live days" for SKU 4567 at both stores. It discovers that Store A, now with more stock, sold 16 units, achieving a ROS of 4 units/week with 100% live days. Store B, having transferred units, sold 4 units, maintaining a ROS of 1 unit/week on its remaining stock.
4. Finally, the module compiles this information, showing that the actual implemented transfer of 7 units significantly increased Store A's ROS and overall sales while maintaining efficiency at Store B.

Final Outcome

The analysis system now clearly shows that the inter-store transfer for SKU 4567 successfully boosted sales velocity and reduced lost sales opportunities at Store A, while effectively managing excess inventory at Store B. This validates the strategy and highlights the positive impact of inventory reallocation.

Key Takeaway: Measuring the actual impact of inter-store transfers helps validate inventory reallocation strategies and identifies opportunities to maximize sales by putting products where they sell best, directly improving profitability.

7. Results

This step produces:

Main Outputs

- **Inter-Store Transfer Performance Metrics:** Detailed data for each SKU-store combination, including pre/post sales units, "live days," Rate of Sale (ROS), quantities of suggested and implemented transfers, and pre-transfer stock levels.
- **Transfer Efficiency Insights:** Quantitative data revealing the direct impact of transfers on sales performance and inventory health, enabling clear comparisons of "before" and "after" scenarios.

Business Reports

- IST Impact Summary Report: Used by Inventory Managers and Merchandising Planners to quickly see the overall effectiveness of recent transfers, helping them refine future inventory allocation strategies and optimize Optimum Depth (OD) by store.
- SKU-Store Transfer Detail Report: Used by Store Operations and Replenishment teams to drill down into specific transfer outcomes, identifying best practices for inventory movement and addressing any inefficiencies for particular Ideal Size Sets (ISS) or products.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will either perform an incomplete analysis or fail entirely due to insufficient data for pre/post periods or transfer records, leading to inaccurate insights.	Alert the IT or Data team immediately to investigate the source of the delay and manually trigger a data refresh once the complete data is available.
Inconsistent transfer quantities (e.g., suggested > implemented)	If suggested transfer quantities don't align with implemented quantities, or implemented quantities are illogical (e.g., negative), the analysis may misrepresent the true impact of transfers.	Data validation checks should flag these inconsistencies; the data cleansing team will need to correct the errors in the source systems, and a re-computation will be required.
Zero sales or "live days" in an analysis period	If a product has no sales or wasn't "live" (available) for the entire pre- or post-analysis period, Rate of Sale (ROS) calculations might result in division by zero or skewed figures.	The system handles division by zero by returning a specific default (like 0) or indicating invalidity. Analysts should review such cases to understand if the product was truly not available or if data is missing.
SKUs involved in transfers are missing from master data	The module cannot accurately link sales or inventory to the transferred items, leading to incomplete or erroneous analysis results and an incomplete picture of product performance.	The product data team needs to add the missing SKU details to the master data, ensuring all product information is complete, followed by a re-processing of the module.

9. Next Steps

Before This Step

- **Raw Data Collection:** What we need from it: All historical sales transactions, current inventory levels, suggested and implemented inter-store transfers, and comprehensive product master data from various source systems (e.g., point-of-sale, ERP, warehouse).
- **Impact Analysis Sync Module:** What we need from it: A clean, complete, and fully synchronized dataset of all retail information, cleared and loaded into the analysis system's memory, forming the direct input for this module.
- **Impact Analysis Data Module:** What we need from it: Accurately computed inventory levels and “live days” for each SKU in every store, providing the precise foundation for Rate of Sale (ROS) calculations.

After This Step

- **Inventory Optimization Module:** What they need from us: The insights from transfer effectiveness to refine future recommendations for optimal stock levels and inter-store transfers, aiming to achieve the Ideal Size Set (ISS) across locations.
- **Assortment Planning Module:** What they need from us: Data on which transfers successfully boosted sales and improved product availability, informing decisions about Optimum Depth (OD) (how many items to stock per style) and Optimum Width (OW) (how many styles to offer).
- **Financial Performance Reporting:** What they need from us: Quantitative data on the impact of optimized inventory movement on overall revenue, profitability, and markdown reduction.

10. Common Questions

Q: Why is it important to analyze both suggested and actual transfers? A: Analyzing both helps us understand if our planning is accurate and if operational execution matches our strategies. It highlights gaps between what we *intended* to do and what *actually* happened, allowing us to refine both our planning algorithms and our logistical processes for better inventory management.

Q: How does this module help reduce stockouts and overstocking? A: By identifying which inter-store transfers successfully moved products to areas of higher demand, it helps optimize inventory distribution. This means we avoid stockouts in high-demand stores and reduce excess stock in slow-selling locations, leading to better product availability and minimized markdowns.

Q: What role do “live days” play in evaluating transfer impact? A: “Live days” are critical because they represent the actual time a product was available for customers to buy. Without this, a product might appear to have low sales, but the module reveals it was actually out of stock for much of the period. This ensures that the Rate of Sale (ROS) calculations are accurate and truly reflect a product's selling potential when available, providing a fair comparison before and after a transfer.

Q: How can I use the results of this analysis in my daily merchandising? A: The results provide clear insights into which SKUs benefit most from transfers and which stores are best suited for receiving or sending inventory. You can use this to make more informed decisions about future stock reallocations, ensuring faster sell-through, higher sales, and a better customer experience by having the right products in the right place.

Q: Does this module suggest new transfers, or just evaluate past ones? A: This module specifically *evaluates* the impact of *past* inter-store transfers, both suggested and implemented. Its output provides crucial data for other modules, like the Inventory Optimization Module, which then uses these insights to *suggest* future transfers, thereby improving efficiency in achieving the Ideal Size Set (ISS).

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Evaluate the effectiveness of Inter-Store Transfers (ISTs) by comparing product performance pre and post-transfer.
- **Main Input:** Sales records, inventory levels, suggested and implemented transfer data, product and store master data.
- **Main Output:** Detailed performance metrics for each SKU-store combination, highlighting the impact of transfers on Rate of Sale (ROS) and “live days.”
- **Warning Signs:** Inconsistent ROS changes after transfers, discrepancies between planned and actual transfer impacts, or an inability to trace the performance of transferred items.
- **Key Settings:** Pre-Transfer Analysis Start Date, Pre-Transfer Analysis End Date, Post-Transfer Analysis Start Date, Post-Transfer Analysis End Date.

Need Help? Contact your IT Support team or Data Operations for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: Impact Analysis

Discounting Module

Last Updated: 2025-05-23

1. Business Objective

This module analyzes the precise impact of past discounting actions on sales performance and revenue, evaluating whether implemented discounts aligned with strategic recommendations.

Why This Matters: This ensures that our promotional investments are effective, helping us make smarter, data-driven decisions about future pricing strategies, optimize sales during promotions, and avoid unnecessary revenue loss from ineffective discounts.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- Sales Records (Units Sold, Revenue, Discount Value, Dates): To measure the quantity and financial outcome of products sold, and to determine the actual discount applied, for both pre-discounting and post-discounting periods, enabling an accurate Rate of Sale (ROS) calculation.
- Discounting Analysis Inputs: To understand the specific discount recommendations and the actual current discount applied to products, which is crucial for evaluating if implemented discounts aligned with our strategy.

Product Data

- Individual Product Details (SKUs, descriptions, colors, sizes): To uniquely identify every item and group sales by specific products and their attributes for detailed analysis.
- Product Style Information (Categories, collections, brands, fashion grades): To analyze the impact of discounts across broader product groups and understand performance trends, informing Optimum Width (OW) and Optimum Depth (OD) decisions for future assortments.

Operational Data

- Store Locations and Channels: To understand where products were sold and discounts were applied, allowing for regional or channel-specific performance analysis.
- Product Availability (Live Days): The exact number of days each product was available for sale in each store during the analysis periods, essential for calculating an accurate Rate of Sale (ROS) and ensuring fair comparisons between pre and post-discounting periods.
- Store Groupings: To aggregate sales data and discount performance at a higher geographical or strategic level, enabling broader analysis of discount effectiveness.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Pre-Analysis Start Date	(Varies, e.g., 30 days prior)	Defines the beginning of the period for analyzing product performance <i>before</i> any discounting actions were initiated.	To capture a longer or shorter baseline of sales performance before a discount, depending on product lifecycle or market stability.
Pre-Analysis End Date	(Varies, e.g., Day before discount)	Defines the end of the period for analyzing product performance <i>before</i> discounting.	To precisely define the pre-discounting window relative to when the discount became active.
Post-Analysis Start Date	(Varies, e.g., Day of discount)	Defines the beginning of the period for analyzing product performance <i>after</i> any discounting actions took effect.	To immediately assess the impact from the day the discount started, or to allow for a buffer period before measuring post-discount sales.
Post-Analysis End Date	(Varies, e.g., 30 days after discount)	Defines the end of the period for analyzing product performance <i>after</i> any discounting	To capture a longer period of post-discount sales impact, or to focus on immediate short-term changes.

Setting Name	Default Value	Description	When to Change
		actions took effect.	

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Synced & Prepared Retail Data (Sales, Product, Availability, Discount Inputs)] --> B[Impact Analysis Discounting Module]

B --> C[Discounting Impact & Alignment Report]

1. Start with → Sales records, product details, store groupings, and product availability data, along with discount recommendations and actual discounts applied for specific timeframes.
2. Then → **Segment** sales and availability data into “pre-discounting” and “post-discounting” periods for relevant products and stores.
3. Next → **Calculate** key performance metrics like total sales units, revenue, discount value, and “live days” within both pre- and post-discounting periods.
4. Finally → **Generate** a detailed report that quantifies the sales and revenue change attributable to discounting and confirms if discounts were implemented as recommended.

5. Step-by-Step Actions

1. **Segment** Sales and Availability Data
 - What we do: *We separate all relevant sales records and product availability (live days) into two distinct windows: a period before* any discounts were applied and a period after.**
 - Why we do it: *This creates a clear “before and after” snapshot, allowing us to accurately compare how product performance changes directly due to discounting efforts.*
2. **Calculate** Key Performance Metrics
 - What we do: *For every relevant product style within each store group, we compute total units sold, total revenue generated, and the total value of discounts applied for both the pre-discounting and post-discounting periods. We also leverage the “live days” to ensure accurate Rate of Sale (ROS) calculations if needed.*
 - Why we do it: *These calculations are the foundation for understanding the full financial impact of discounts, showing how much sales volume increased (or decreased) and what revenue was gained or forgone due to pricing changes.*
3. **Determine** Discount Alignment
 - What we do: *We compare the actual discount percentage applied during the post-discounting period with the discount recommendations and specified discount actions (e.g., INCREASE, DECREASE, CONTINUE).*

- Why we do it: *This step validates whether our promotional strategies were effectively executed, highlighting any discrepancies between what was planned and what actually happened on the sales floor.*
4. **Generate Impact Analysis Report**
- What we do: *We bring together all the calculated sales performance metrics, discount alignment findings, and other relevant product and store attributes into a comprehensive output report.*
 - Why we do it: *This consolidated view provides merchandising teams with a clear, actionable summary of how effective each discounting action was, enabling them to refine future promotional campaigns for maximum impact.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- Our goal is to understand the impact of a recent 15% discount recommendation on a popular winter coat style, Style 1234, in our city-center stores.
- The system recorded sales for Style 1234 in city-center stores: 100 units sold generating \$20,000 revenue in the 30 days *before* the discount.
- The discount recommendation was to INCREASE the discount to 15%.
- After the discount period started, the system recorded 200 units sold generating \$34,000 revenue for Style 1234 in city-center stores over the next 30 days.

What Happens

1. First, the Impact Analysis Discounting Module segments the sales data: the pre-discounting period (30 days before) and the post-discounting period (30 days after the discount started). It also confirms that Style 1234 was available for sale for the entire duration in both periods (full "live days").
2. Then, it calculates the metrics for each period:
 - Pre-discounting: 100 units sold, \$20,000 revenue, \$0 discount value.
 - Post-discounting: 200 units sold, \$34,000 revenue, with total discount value calculated from the actual sales.
3. Next, the module analyzes the actual discount applied. It finds that the average discount on Style 1234 sales during the post-discounting period was indeed 15%, aligning with the "INCREASE" recommendation.
4. Finally, the module compares the periods, showing a 100% increase in units sold and a 70% increase in revenue for Style 1234, directly attributable to the 15% discount.

Final Outcome

The analysis system now clearly shows that the 15% discount on Style 1234 in city-center stores was successfully implemented and led to a significant boost in sales volume and overall revenue, validating the promotional strategy.

Key Takeaway: By precisely quantifying the sales and revenue changes and verifying discount implementation, this module provides concrete evidence of discounting effectiveness, allowing merchants to refine future promotional campaigns for optimal financial results.

7. Results

This step produces:

Main Outputs

- **Pre- vs. Post-Discout Performance Metrics:** Detailed data for each product style by store group, including total quantity sold, total revenue, and total discount value for both periods, enabling direct comparison of sales impact.
- **Discount Alignment Status:** Indication of whether the actual applied discounts matched the recommended discount actions (e.g., INCREASE, DECREASE, CONTINUE), providing insights into execution effectiveness.
- **Quantified Sales & Revenue Change:** Specific numbers highlighting the increase or decrease in sales and revenue directly attributable to the discounting strategy.

Business Reports

- **Discount Effectiveness Summary Report:** Used by Merchandising Planners and Marketing Managers to quickly assess which promotions were most successful in driving sales and revenue, helping to optimize future pricing and promotional calendars.
- **Promotional Compliance Report:** Used by Store Operations and Regional Managers to review instances where actual discounts deviated from recommendations, ensuring consistent execution of pricing strategies across the retail network.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will either perform an incomplete analysis or fail entirely due to insufficient sales or discount data for the pre/post periods, leading to inaccurate insights.	Alert the IT or Data team immediately to investigate the source of the delay and manually trigger a data refresh once the complete data is available.
Actual discount significantly differs from recommendation	If the actual discount applied is widely different from the recommended action, the analysis might incorrectly attribute	The module will flag this as a misalignment. Investigate the cause (e.g., system error, manual override, incorrect input data) and correct the underlying process or data.

Situation	What Happens	What You Should Do
Zero sales or “live days” in an analysis period	sales changes or highlight execution failures.	
	If a product had no sales or wasn’t available for sale (“live days” = 0) during either the pre- or post-discounting period, the Rate of Sale (ROS) and other calculations may be skewed or result in division by zero.	The system handles division by zero (e.g., returning 0 or N/A). Analysts should review such cases to understand if the product was truly not available or if data is missing, and adjust interpretation accordingly.
External factors influencing sales during discounting	Unforeseen events (e.g., local event, competitor promotion, major holiday) might impact sales, making it difficult to isolate the true impact of the discount.	While this module quantifies direct sales changes, business analysts must use external context. Future enhancements might incorporate external data for a more nuanced view.

9. Next Steps

Before This Step

- Raw Data Collection: What we need from it: All historical sales transactions (including revenue and discount value), current inventory levels, product master data (including style, category, brand), and discount recommendations/actions from various source systems (e.g., point-of-sale, ERP, pricing systems).
- Impact Analysis Sync Module: What we need from it: A clean, complete, and fully synchronized dataset of all retail information, cleared and loaded into the analysis system’s memory, forming the direct input for this module.
- Impact Analysis Data Module: What we need from it: Accurately computed inventory levels and “live days” for each product SKU in every store, providing the precise foundation for Rate of Sale (ROS) calculations, especially for the pre and post-discounting periods.

After This Step

- Promotional Planning Module: What they need from us: Quantitative insights on the effectiveness of different discount levels and types, informing future promotional strategies and discount recommendations to maximize sales and revenue.
- Financial Performance Reporting: What they need from us: Detailed data on revenue impact and discount expenses, enabling accurate profitability analysis of promotional activities and contributing to overall financial health reports.
- Inventory Optimization Module: What they need from us: Understanding how discounts impact Rate of Sale (ROS) and sell-through, allowing for better

allocation of future inventory and adjustments to Optimum Depth (OD) based on expected promotional lifts.

10. Common Questions

Q: Why is it important to analyze the impact of discounts, not just the sales volume?

A: Analyzing impact helps us understand if the discount *caused* the sales increase, and if that increase was profitable. Sometimes, a high sales volume might come at too great a cost (e.g., very deep discounts). This module shows if the discount was worth it by linking sales to the actual discount applied and checking alignment with recommendations.

Q: How can I use the results of this analysis in my daily merchandising? A: You can use these results to refine your pricing and promotional calendar. If a 20% discount on a specific product consistently delivers a significant, profitable sales lift, you'll know to apply it again. If another discount type yields little change, you can avoid it, preventing unnecessary margin erosion.

Q: What happens if a discount was recommended but not actually implemented in stores? A: The module will flag this as a misalignment, showing that the planned discount action (e.g., "INCREASE") was not matched by the actual discount applied. This highlights a gap in execution, allowing operational teams to investigate why the discount wasn't put into effect and improve future adherence.

Q: Does this module tell me what discount to apply next? A: No, this module *evaluates* the impact of *past* discounting actions. It provides crucial data on what *worked* and what didn't. This information is then used by other modules, like the Promotional Planning Module, to *suggest* future discount strategies based on historical effectiveness.

Q: How does this help us avoid common retail issues like unnecessary markdowns?

A: By providing clear evidence of which discounts truly drive sales and revenue, this module helps you avoid "blind" discounting. You can focus your markdown efforts on strategies proven to clear inventory efficiently and profitably, rather than applying deep discounts that just erode margin without sufficient sales lift.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Quantifies the sales and revenue impact of past discounting actions and evaluates alignment with recommendations.
 - **Main Input:** Sales records (including revenue and discount value), product attributes, store details, product availability ("live days"), and discount recommendations.
 - **Main Output:** Detailed pre/post-discount performance metrics, discount alignment status, and quantified sales/revenue changes.
 - **Warning Signs:** Unexpectedly low sales lift despite deep discounts, or frequent misalignments between recommended and applied discounts.
 - **Key Settings:** Pre-Analysis Start/End Date, Post-Analysis Start/End Date.
-

Need Help? Contact your IT Support team or Data Operations for questions about this process.

Generated by AI Codebase Knowledge Builder

impact FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ document provides quick answers to common business questions about the impact analysis project, covering its purpose, data usage, processing logic, and key outputs.

General

Q: What is the impact project designed to do? **A:** The impact project is an analysis system designed to evaluate the business impact of strategic actions, primarily focusing on Inter-Store Transfers (ISTs) and discounting strategies on product performance across stores.

Q: What kind of business problems does impact help solve? **A:** It helps merchandisers and planners understand how inventory reallocation (ISTs) and pricing strategies (discounting) directly affect sales, revenue, and product velocity (ROS). This enables better decision-making for optimizing inventory management and promotional campaigns.

Q: How does the impact project work at a high level? **A:** The Impact Analysis Group Module orchestrates the entire process. It first synchronizes all necessary sales, inventory, and product data (Impact Analysis Sync Module), then prepares this data (Impact Analysis Data Module), and finally performs detailed analyses on ISTs (Impact Analysis IST Module) and discounting (Impact Analysis Discounting Module) to quantify their effects.

Q: What kind of data does impact use for its analysis? **A:** It utilizes comprehensive master data (SKU, Style, Store information) and transactional data including sales records, inventory key frames, and details about both suggested and implemented ISTs and discounts.

Q: How often is the impact analysis updated with new information? **A:** The Impact Analysis Sync Module ensures that the in-memory database is refreshed with the latest master and transactional input data before each analysis run. While the exact frequency depends on operational schedules, the system is designed to provide timely insights based on current data.

Configuration & Settings

Q: Where can I find the settings that control how impact operates? **A:** Key operational parameters and configuration settings for the impact analysis are typically managed through external configuration files, often referenced as `args.properties`. For specific details, please refer to the technical documentation.

Q: Can I adjust the timeframes for analyzing discounting or IST impacts? **A:** Yes, the system uses specific periods for pre- and post-event analysis (e.g., pre-IST vs. post-IST). These timeframes are configurable, allowing you to define the windows over which sales and inventory data are evaluated for calculating metrics like ROS.

Q: How does impact ensure it's using the freshest data for its analysis? **A:** The Impact Analysis Sync Module is responsible for initializing the in-memory database by clearing existing data and loading all the necessary master and transactional inputs, ensuring that all subsequent analysis modules operate on a fresh and complete dataset.

Q: Does impact have settings to define what constitutes an "Optimum Depth" (OD) or "Optimum Width" (OW)? **A:** While the impact project focuses on evaluating the actual performance changes stemming from actions, rather than directly defining OD or OW, its output metrics (e.g., changes in ROS, sales uplift) provide crucial data that can inform strategies for achieving OD (ideal unique items) and OW (ideal categories). Specific configurations for these concepts would typically reside in broader inventory planning systems.

Q: Are there thresholds or criteria impact uses to identify a "significant" impact? **A:** The analysis modules provide detailed metrics (e.g., changes in ROS, sales, revenue). The interpretation of what constitutes a "significant" impact usually depends on your specific business rules and Key Performance Indicators (KPIs), which might be defined in downstream reporting or visualization layers that consume impact's outputs.

Data & Inputs

Q: What types of master data does impact use, and why are they important? **A:** impact uses master data for SKUs, Styles, and Stores. This foundational data provides essential context, allowing the system to accurately attribute sales and inventory information to specific products, styles, and locations. This data is prepared by the Impact Analysis Data Module.

Q: How does impact know how long a product was actually available for sale in a store? **A:** The Impact Analysis Data Module is responsible for computing "live days," which is the precise number of days a particular product SKU was available in a specific store. This metric is critical for accurately calculating Rate of Sale (ROS).

Q: Can impact distinguish between different types of sales, like full-price vs. discounted sales? **A:** Yes, the Impact Analysis Discounting Module specifically processes sales data to account for discount values and revenue. This allows for a detailed analysis of sales performance and revenue attribution directly related to discounting strategies.

Q: What are "inventory key frames" and why are they important for the analysis? **A:** Inventory key frames are snapshots of inventory levels at specific points in time. They are crucial inputs for the Impact Analysis Data Module to accurately compute overall product inventory levels and determine the "live days" essential for robust ROS calculations.

Q: Does impact use data on both recommended and actual actions? **A:** Yes, both the Impact Analysis IST Module and the Impact Analysis Discounting Module are designed to analyze data for both suggested (recommended) and actually implemented quantities or actions (like ISTs or discounts). This allows for a valuable comparison of plan versus actual outcomes.

Process & Logic

Q: How does impact determine the effectiveness of Inter-Store Transfers (ISTs)? **A:** The Impact Analysis IST Module evaluates IST effectiveness by comparing pre-IST and post-IST sales, “live days,” and Rate of Sale (ROS) for specific SKUs at various stores. This comparison quantifies the actual impact of inventory reallocation strategies.

Q: How does impact quantify the effect of discounting on sales? **A:** The Impact Analysis Discounting Module processes sales data, aggregating key metrics like total quantity sold, revenue, and discount value. By comparing these metrics across pre-discounting and post-discounting periods, it quantifies the change in sales and revenue directly attributable to discounting strategies.

Q: What is “Rate of Sale” (ROS) and how is it calculated in impact? **A:** ROS (Rate of Sale) is a measure of how quickly products are selling, typically expressed as units per week. In impact, it’s accurately calculated by the Impact Analysis Data Module using sales records, inventory levels, and “live days” to provide a reliable measure of product velocity.

Q: Is the impact analysis process sequential, or can modules run independently? **A:** The Impact Analysis Group Module orchestrates a specific sequence of execution. Data is first synchronized and then prepared by the Impact Analysis Data Module. Only after this foundational data work are the specialized analyses for ISTs and discounting performed, ensuring data consistency and accuracy throughout the pipeline.

Q: Does impact help define the “Ideal Size Set” (ISS) for products? **A:** While impact provides granular performance data at the SKU-store level (including specific size performance), its primary role is to evaluate the *impact* of existing actions. It doesn’t directly compute or suggest an ISS. However, the insights from ROS and sales changes can certainly inform future ISS recommendations or validate existing ones.

Outputs & Results

Q: What kind of outputs does impact generate for business users? **A:** The impact project generates detailed metrics for each SKU-store combination, including pre/post sales quantities, Rate of Sale (ROS), revenue, and discount values. These outputs provide clear insights into the effectiveness of both ISTs and discounting strategies.

Q: Can I see a comparison of recommended versus actual ISTs or discounts in the output? **A:** Yes, both the Impact Analysis IST Module and the Impact Analysis Discounting Module produce outputs that allow you to compare the performance related to recommended actions versus the actual actions taken. This helps in assessing the alignment and effectiveness of execution.

Q: How can the output from impact help me make better business decisions? **A:** By quantifying the direct effect of ISTs and discounting on sales and ROS, the outputs empower you to optimize inventory allocation, refine promotional strategies, and understand the true financial returns of these actions. For example, you can identify which ISTs were most successful or which discount campaigns yielded the highest return.

Q: Is the data generated by impact easily accessible for reporting or dashboards? **A:** Yes, the Impact Analysis Group Module handles output snapshotting and synchronization, ensuring the generated data is structured and readily available for downstream consumption by reporting tools, business intelligence dashboards, or other analytical platforms.

Q: Does impact provide insights into a product's "Optimum Depth" (OD) or "Optimum Width" (OW)? **A:** While impact does not directly *calculate* OD (ideal number of unique items) or OW (ideal number of categories), its detailed performance metrics (such as ROS per SKU, or sales lift post-discount) provide critical data points. These data points can be used by other systems or analysts to determine optimal assortments, thereby contributing to understanding and achieving OD and OW.

Error Handling & Edge Cases

Q: What happens if there's missing sales or inventory data for a particular SKU or store? **A:** The Impact Analysis Data Module is designed to process available data and compute metrics like "live days" accordingly. While the Impact Analysis Sync Module aims for complete data loading, significant data gaps could lead to incomplete or less robust analysis for affected SKUs or stores.

Q: Are there any known limitations or scenarios where impact might not provide accurate results? **A:** The provided documentation does not explicitly list specific known edge cases. However, like any data-driven analysis, the accuracy of results depends on the quality and completeness of input data. Any significant data anomalies or unusual market conditions not captured in the standard inputs might affect precise quantification. For a full list of known edge cases, please refer to the detailed technical documentation.

Q: How does impact handle situations where multiple promotions are active simultaneously for a product? **A:** The Impact Analysis Discounting Module aggregates sales and discount values over defined periods. While it quantifies the overall impact of discounting during a period, untangling the precise effect of *multiple concurrent* promotions might require more detailed attribute-level analysis beyond the scope of this core project. The module focuses on the overall observed discounted sales impact.

Q: What if an IST happened, but the product was out of stock before the transferred units arrived at the destination store? **A:** The Impact Analysis IST Module evaluates pre-IST and post-IST performance based on actual sales, "live days," and ROS. If a product was out of stock, its "live days" would reflect this period of unavailability, and its ROS would be impacted accordingly. The module quantifies the *actual* observed impact, including any delays or stockouts that affect product availability.

Q: How does impact analyze new products or products with very short sales histories? **A:** Products with short sales histories will naturally have limited “live days” and sales data. While the system processes all available information, insights for very new or short-lifecycle products might be less conclusive or require careful interpretation compared to established items with longer data trails.

Need More Help? For further assistance or more detailed technical documentation, please contact our support team at support@yourcompany.com.

impact Documentation

- **Version:** 1.0
 - **Last Updated:** 2025-05-23
-

2. Executive Summary

The impact analysis project provides critical insights into the effectiveness of inventory management and pricing strategies. By quantifying the business impact of Inter-Store Transfers (ISTs) and discounting actions, this algorithm helps retailers optimize inventory allocation, pricing, and promotional efforts. Its key outcomes include improved product availability, maximized sales, and enhanced profitability through data-driven decisions.

3. Architecture Overview

```
graph TD
    subgraph Orchestrator
        A[Impact Analysis Group Module]
    end

    subgraph Data_Layer
        B[Impact Analysis Sync Module]
        C[Impact Analysis Data Module]
    end

    subgraph Analysis_Modules
        D[Impact Analysis IST Module]
        E[Impact Analysis Discounting Module]
    end

    A -- control_flow --> B
    A -- control_flow --> C
    A -- control_flow --> D
    A -- control_flow --> E

    C -- data_access --> B
    D -- data_access --> B
    E -- data_access --> B
```

```
E -- data_flow --> C
```

Module Descriptions:

- **Impact Analysis Group Module:** The central orchestrator that manages the sequence of data synchronization, preparation, and specialized analyses.
- **Impact Analysis Sync Module:** Initializes the system by loading all necessary raw master and transactional data into memory.
- **Impact Analysis Data Module:** Prepares foundational data, filtering master data and computing key metrics like inventory levels and “live days” for sales calculations.
- **Impact Analysis IST Module:** Evaluates the business impact of Inter-Store Transfers (ISTs) on product performance.
- **Impact Analysis Discounting Module:** Analyzes how discounting strategies affect sales, revenue, and product performance.

4. Prerequisites & Setup

To run the impact analysis project, ensure the following are in place:

- **Java Version:** Java 11 or higher
- **Build Tool:** Maven 3.6 or higher
- **Dependencies:** All project dependencies are managed via Maven (`pom.xml`).
- **Running the Orchestrator:**
 1. Navigate to the project root directory.
 2. Build the project using Maven: `mvn clean install`
 3. Execute the ModuleGroup orchestrator (which is the ImpactAnalysisGroupModule) via the compiled JAR: `java -jar target/impact-1.0.jar --config args.properties`

5. Folder & File Structure

```

impact/
├─ module-group/                # Contains the orchestrator module
(ImpactAnalysisGroupModule)
├─ submodules/                 # Contains the specialized analysis
and data preparation modules
├─ args.properties             # Global configuration parameters for
the entire analysis
├─ common-data.json            # Schema definition for shared data
structures
├─ outputs/                   # Directory where all analysis
results and reports are stored

```

6. Global Configuration

The `args.properties` file centralizes configuration for the entire impact analysis pipeline.

Setting Name	Default Value	Description	Valid Range/ When to Change
<code>data.input.path</code>	<code>./data/</code>	Path to the directory containing raw input CSV files.	Update if input data is in a different location.
<code>output.path</code>	<code>./outputs/</code>	Path where all generated analysis reports and snapshots will be saved.	Change to a desired output directory.
<code>ist.analysis.enabled</code>	<code>true</code>	Enables or disables the Inter-Store Transfer (IST) impact analysis module.	Set to <code>false</code> if IST analysis is not required.
<code>discount.analysis.enabled</code>	<code>true</code>	Enables or disables the Discounting impact analysis module.	Set to <code>false</code> if Discounting analysis is not required.
<code>min.live.days.threshold</code>	<code>7</code>	Minimum number of “live days” a product must have to be included in ROS calculation.	Adjust based on data granularity and business context.
<code>report.granularity.level</code>	<code>SKU_STORE</code>	The lowest granularity level for output reports (e.g., <code>SKU</code> , <code>SKU_STORE</code> , <code>STYLE</code>).	<code>SKU</code> , <code>SKU_STORE</code> , <code>STYLE</code> . Affects output report size.
<code>debug.logging.enabled</code>	<code>false</code>	Enables verbose logging for	Set to <code>true</code> for detailed technical logs

Setting Name	Default Value	Description	Valid Range/ When to Change
		debugging purposes.	during troubleshooting.

7. Shared Data Model

The `common-data.json` schema defines the structure of data exchanged between modules.

Field	Type	Description
<code>sku_id</code>	string	Unique identifier for a Stock Keeping Unit.
<code>store_id</code>	string	Unique identifier for a retail store.
<code>category</code>	string	Product category (e.g., “Apparel”, “Footwear”).
<code>style_id</code>	string	Unique identifier for a product style or design.
<code>sale_date</code>	date	Date of the sales transaction.
<code>quantity_sold</code>	integer	Number of units sold in a transaction.
<code>revenue</code>	double	Total revenue generated from a transaction.
<code>discount_amount</code>	double	Monetary value of discount applied to a transaction.
<code>inventory_level</code>	integer	Current stock count for a SKU at a store.
<code>live_days</code>	integer	Number of days a SKU was available for sale at a specific store (Impact Analysis Data Module output).
<code>ros</code>	double	ROS : Rate of Sale (units per week/day) for a SKU at a store (Impact Analysis Data Module output).
<code>suggested_ist_quantity</code>	integer	Quantity suggested for Inter-Store Transfer (Impact Analysis IST Module input).
<code>implemented_ist_quantity</code>	integer	Actual quantity transferred during an IST (Impact Analysis IST Module input).
<code>pre_ist_sales</code>	double	Sales performance before an IST (Impact Analysis IST Module output).
<code>post_ist_sales</code>	double	

Field	Type	Description
		Sales performance after an IST (Impact Analysis IST Module output).
pre_discount_sales	double	Sales performance before a discount (Impact Analysis Discounting Module output).
post_discount_sales	double	Sales performance after a discount (Impact Analysis Discounting Module output).
average_discount_rate	double	Average discount rate applied (Impact Analysis Discounting Module output).

8. Submodule Chapters

8.1. Impact Analysis Sync Module

1. Module Name & Business Objective

Impact Analysis Sync Module: This module initializes the in-memory database by clearing existing data and loading all necessary master and transactional input data (e.g., store, SKU, style information, sales records, inventory key frames, suggested and implemented ISTs, and discounting analysis inputs). *This ensures all subsequent analysis modules operate on a fresh and complete dataset.*

2. Required Information

- **Sales Data:**
 - Sales records (date, SKU, store, quantity, revenue, discount).
- **Product Data:**
 - SKU master data (SKU ID, category, style ID).
 - Style master data (style ID, description).
 - Style-Season Mapping.
- **Operational Data:**
 - Store master data (store ID, location, type).
 - Inventory key frames (SKU, store, date, stock level).
 - Suggested and Implemented IST records (from_store, to_store, SKU, quantity, date).
 - Discounting analysis inputs (SKU, store, discount type, dates).

3. Business Settings

- `data.input.path`: Specifies the file system location from which raw data files are loaded.

- `data.clear.on.sync`: A boolean flag to determine if the in-memory database should be cleared before loading new data. *Ensures data freshness for each analysis run.*

4. Process Flow

graph TD

```
A[Raw Input Data Files] --> B{Clear In-Memory DB?}
B -- Yes --> C[Load Master Data]
B -- No --> C
C --> D[Load Transactional Data]
D --> E[In-Memory Database Ready]
```

5. Step-by-Step Actions

1. **Check** `data.clear.on.sync` setting.
2. **Clear** existing in-memory database, *to ensure a clean slate for the current analysis run.*
3. **Read** SKU, Style, Store master data from specified input files.
4. **Load** sales records, inventory key frames, IST records, and discounting inputs.
5. **Populate** the in-memory database with all loaded data, *making it accessible for downstream modules.*

6. Sample Scenario

Imagine the system previously ran with old sales data. A new daily data feed arrives, including updated sales, inventory, and recent ISTs. The Sync Module runs, clearing all old data and loading the fresh data.

Key Takeaway: *This module is the data foundation, ensuring all analyses are based on the most current and clean information.*

7. Results

- A fully populated, in-memory data store containing all necessary raw input data.
- Confirmation of data loading success or failure logs.

8. Unusual Cases

- **Missing or delayed data feed:** *Leads to incomplete or stale data, potentially skewing analysis results.*
- **Incorrect file format:** *Causes parsing errors and prevents data from being loaded, halting the pipeline.*

9. Next Steps

The populated in-memory database is now ready to be consumed by the Impact Analysis Data Module for further processing and calculation of derived metrics.

10. Common Questions

1. How often should this module run?
2. What happens if a data file is missing?
3. Can I load only specific types of data?
4. How do I verify if data was loaded correctly?
5. What are the performance implications of loading large datasets?

11. Quick Reference

- **Purpose:** Data ingestion and initialization.
 - **Inputs:** Raw CSV files (Sales, Inventory, Master data, IST, Discounting).
 - **Outputs:** Populated in-memory database.
 - **Key Action:** Data refresh.
-

8.2. Impact Analysis Group Module

1. Module Name & Business Objective

Impact Analysis Group Module: This is the top-level orchestration module for the entire impact analysis project. It defines the sequence of execution for various specialized impact analysis modules, ensuring that data is synced, prepared, and then analyzed for ISTs and discounting in a structured manner. *It also handles output snapshotting and synchronization.*

2. Required Information

- **Sales Data:** (indirectly, via Impact Analysis Sync Module)
- **Product Data:** (indirectly, via Impact Analysis Sync Module)
- **Operational Data:** (indirectly, via Impact Analysis Sync Module)
- **Configuration:**
 - `ist.analysis.enabled`: Flag to enable/disable IST module.
 - `discount.analysis.enabled`: Flag to enable/disable Discounting module.

3. Business Settings

- `enable.module.ist`: Controls whether Impact Analysis IST Module is executed. *Allows focusing analysis on specific areas.*
- `enable.module.discount`: Controls whether Impact Analysis Discounting Module is executed. *Enables modularity for different business needs.*

4. Process Flow

graph TD

```
A[Start] --> B[Impact Analysis Sync Module]
B --> C[Impact Analysis Data Module]
```

```

C --> D{IST Analysis Enabled?}
D -- Yes --> E[Impact Analysis IST Module]
D -- No --> F{Discount Analysis Enabled?}
E --> F
F -- Yes --> G[Impact Analysis Discounting Module]
F -- No --> H[Generate Final Reports]
G --> H
H --> I[End]

```

5. Step-by-Step Actions

1. **Initialize** the analysis environment.
2. **Execute** Impact Analysis Sync Module, *to ensure all raw data is loaded and ready.*
3. **Run** Impact Analysis Data Module, *to prepare foundational data and compute derived metrics like ROS and live days.*
4. **Check** `ist.analysis.enabled` configuration.
5. **Execute** Impact Analysis IST Module if enabled, *to evaluate IST impact.*
6. **Check** `discount.analysis.enabled` configuration.
7. **Execute** Impact Analysis Discounting Module if enabled, *to analyze discounting impact.*
8. **Trigger** final report generation and output snapshotting, *to consolidate and save all analysis results.*

6. Sample Scenario

A retail operations team wants to analyze both IST effectiveness and discount performance. The Impact Analysis Group Module is configured with both `ist.analysis.enabled=true` and `discount.analysis.enabled=true`. It flawlessly executes the data sync, preparation, IST analysis, and discounting analysis in the correct order, producing comprehensive reports for both areas.

Key Takeaway: *This module is the conductor, ensuring all analysis steps are performed in the correct order and according to business requirements.*

7. Results

- Orchestrated execution of all selected impact analysis submodules.
- Consolidated output reports from all enabled analysis modules.
- Error logs detailing any failures during the pipeline execution.

8. Unusual Cases

- **Missing dependent module:** *If a module like Impact Analysis Data Module fails, subsequent analysis modules will lack prepared data and also fail.*
- **Invalid configuration flags:** *Setting `ist.analysis.enabled` to a non-boolean value might cause the orchestrator to halt.*

9. Next Steps

The completion of this module signifies the end of the impact analysis run. The generated reports are now available for business review and decision-making.

10. Common Questions

1. How do I run only the IST analysis?
2. What is the typical runtime for the full pipeline?
3. Where can I find the final output reports?
4. How do I troubleshoot a pipeline failure?
5. Can I add new analysis modules to this orchestrator?

11. Quick Reference

- **Purpose:** Pipeline orchestration and overall control.
 - **Inputs:** Configuration settings.
 - **Outputs:** Final consolidated analysis reports.
 - **Key Action:** Sequential execution of modules.
-

8.3. Impact Analysis Data Module

1. Module Name & Business Objective

Impact Analysis Data Module: This module is responsible for the foundational data preparation required for impact analysis. It initializes and filters various master data maps (SKU, Style, Store) and transactional data (Sales, Inventory Key Frames). Crucially, it computes product inventory levels and “live days” (the number of days a product SKU was available in a particular store), which are essential for calculating metrics like **ROS**. *This module ensures that all subsequent analysis modules have access to clean, enriched, and ready-to-use data.*

2. Required Information

- **Sales Data:**
 - Raw sales records (SKU, store, date, quantity, revenue).
- **Product Data:**
 - SKU master data (for filtering and mapping).
 - Style master data (for filtering and mapping).
- **Operational Data:**
 - Store master data (for filtering and mapping).
 - Inventory key frames (SKU, store, date, inventory level).

3. Business Settings

- `data.min.live.days`: Minimum number of days a SKU must be available in a store to be considered for **ROS** calculations. *Avoids skewed ROS for newly stocked or quickly depleted items.*

- `data.inventory.lookback.days`: Number of past days to consider when calculating average inventory levels. *Influences the accuracy of average inventory metrics.*

4. Process Flow

graph TD

```
A[Raw Data (from Sync Module)] --> B{Filter Master Data}
B --> C[Process Sales Data]
C --> D[Process Inventory Key Frames]
D --> E[Compute Live Days & ROS]
E --> F[Enriched Data Model]
```

5. Step-by-Step Actions

1. **Load** raw master data (SKU, Style, Store) from the in-memory database.
2. **Filter** master data based on business rules or enabled categories, *to ensure only relevant items are processed.*
3. **Process** raw sales data, aggregating by SKU, store, and date.
4. **Process** inventory key frames, calculating daily inventory levels.
5. **Compute** “live days” for each SKU-store combination, *identifying days when a product was truly available for sale.*
6. **Calculate ROS** (Rate of Sale) for each SKU-store combination, *using live days and quantity sold.*
7. **Generate** an enriched `ImpactAnalysisData` object, *containing all processed and derived metrics for downstream use.*

6. Sample Scenario

A new SKU (SKU101) is introduced to StoreA. The Impact Analysis Data Module processes its sales (10 units sold in 5 days) and inventory data (available for 7 days in total). It correctly calculates its “live days” as 7 (assuming it was always available) and its **ROS** as 10 units / (7 days / 7 days per week) = 10 units/week.

Key Takeaway: *This module transforms raw data into a reliable, actionable format for performance measurement, especially for **ROS**.*

7. Results

- A well-structured `ImpactAnalysisData` object containing:
 - Filtered master data maps (SKU, Style, Store).
 - Processed sales and inventory data.
 - Computed `live_days` per SKU-store.
 - Calculated `ros` per SKU-store.
- Error logs for any data inconsistencies or processing failures.

8. Unusual Cases

- **Missing or delayed data feed:** *If inventory data is missing for a period, live days calculation will be inaccurate, affecting **ROS**.*

- **Inconsistent SKU/Style IDs:** *Mismatched IDs between sales and master data can lead to data loss or incorrect aggregations.*
- **Zero inventory but sales records:** *Indicates data quality issues that need investigation.*

9. Next Steps

The enriched ImpactAnalysisData object is passed to Impact Analysis IST Module and Impact Analysis Discounting Module as the primary source of truth for their analyses.

10. Common Questions

1. How is “live days” precisely calculated?
2. What is the definition of **ROS** used here?
3. Why are certain SKUs excluded from **ROS** calculation?
4. Can I see the intermediate processed data?
5. How does data quality impact the calculations in this module?

11. Quick Reference

- **Purpose:** Data preparation, enrichment, and metric calculation (e.g., **ROS**).
 - **Inputs:** Raw sales, inventory, and master data.
 - **Outputs:** Enriched ImpactAnalysisData object (containing live_days, ros).
 - **Key Action:** Data transformation for analytical readiness.
-

8.4. Impact Analysis IST Module

1. Module Name & Business Objective

Impact Analysis IST Module: This module evaluates the impact of Inter-Store Transfers (ISTs) on product performance. It analyzes both suggested and implemented IST quantities. By comparing pre-IST and post-IST sales, live days, and **ROS** for specific SKUs at various stores, it provides insights into the effectiveness of inventory reallocation strategies. *The output includes detailed metrics for each SKU-store combination, helping optimize inventory **OD** and **OW**.*

2. Required Information

- **Sales Data:**
 - Sales records (from ImpactAnalysisData object, including calculated ros).
- **Product Data:**
 - SKU and Style master data (from ImpactAnalysisData object).
- **Operational Data:**
 - Suggested ISTs (SKU, from_store, to_store, quantity, date).
 - Implemented ISTs (SKU, from_store, to_store, quantity, date).
 - Store inventory levels (from ImpactAnalysisData object).

3. Business Settings

- `ist.analysis.pre.days`: Number of days before an IST to consider for “pre-IST” sales analysis. *Helps define the baseline performance.*
- `ist.analysis.post.days`: Number of days after an IST to consider for “post-IST” sales analysis. *Captures the impact after the transfer.*
- `ist.min.transfer.qty`: Minimum quantity for an IST to be included in the analysis. *Filters out insignificant transfers.*

4. Process Flow

graph TD

```
A[Enriched Data Model (from Data Module)] --> B[Identify IST Events]
B --> C[Define Pre/Post-IST Periods]
C --> D[Calculate Pre-IST Metrics (Sales, ROS)]
D --> E[Calculate Post-IST Metrics (Sales, ROS)]
E --> F[Compare & Quantify Impact]
F --> G[IST Impact Report]
```

5. Step-by-Step Actions

1. **Identify** all relevant Suggested and Implemented IST records.
2. **Link** ISTs to specific SKU-store pairs, *to analyze transfers at the most granular level.*
3. **Define** pre-IST and post-IST sales periods for each IST event, *to establish a clear comparison window.*
4. **Calculate** sales, **ROS**, and inventory levels for the pre-IST period for both ‘from’ and ‘to’ stores.
5. **Calculate** sales, **ROS**, and inventory levels for the post-IST period.
6. **Compare** pre- and post-IST metrics, quantifying the change in sales, **ROS**, and inventory, *to assess the effectiveness of the transfer.*
7. **Generate** a detailed IST Impact Report, *highlighting successful transfers and areas for improvement in inventory reallocation strategy.*

6. Sample Scenario

SKU202 at StoreB has low **ROS** (0.5 units/week) and high stock. An IST moves 10 units to StoreC, where SKU202 has high **ROS** (5 units/week) and low stock. * **Pre-IST StoreB**: ROS 0.5, Sales 2 units. * **Post-IST StoreB**: Sales drop to 0. (Expected, due to reduced stock). * **Pre-IST StoreC**: ROS 5, Sales 20 units. * **Post-IST StoreC**: ROS increases to 8, Sales 32 units. The module reports a positive impact, as total sales increased by $(32-20) + (0-2) = 10$ units across both stores due to the reallocation, suggesting the IST was effective.

Key Takeaway: *This module directly measures if moving inventory improved overall sales and **ROS**, helping refine inventory **OD** and **OW** decisions across stores.*

7. Results

- **IST Impact Report:** Tabular data showing for each analyzed IST event:
 - SKU, From Store, To Store, Quantity Transferred.
 - Pre-IST sales, ROS, and inventory for both stores.
 - Post-IST sales, ROS, and inventory for both stores.
 - Net change in sales and ROS attributed to the IST.
 - Ratio of implemented vs. suggested ISTs.

8. Unusual Cases

- **Missing or delayed data feed:** *Incomplete IST records or sales data around IST dates will lead to inaccurate impact analysis.*
- **Concurrent events:** *Multiple ISTs or discounting events happening at the same time for the same SKU can make it difficult to isolate the impact of a single IST.*
- **Zero sales before/after IST:** *If a product has no sales before or after an IST, its ROS impact cannot be reliably measured.*

9. Next Steps

The IST Impact Report can be used by inventory planners and supply chain managers to refine their inventory reallocation strategies, optimize stock levels, and improve overall inventory **OD** and **OW** across the network.

10. Common Questions

1. How accurate is the attribution of sales change to an IST?
2. What is a “successful” IST according to this module?
3. Can I see the impact on specific product **ISS**?
4. Does this analysis consider other factors like marketing campaigns?
5. How can I use this report to improve future IST decisions?

11. Quick Reference

- **Purpose:** Quantify the impact of ISTs on sales and **ROS**.
 - **Inputs:** Enriched data, suggested/implemented ISTs.
 - **Outputs:** IST Impact Report.
 - **Key Metric:** Change in **ROS** and sales pre/post-IST.
-

8.5. Impact Analysis Discounting Module

1. Module Name & Business Objective

Impact Analysis Discounting Module: This module analyzes the impact of discounting actions on sales performance. It processes sales data, aggregating key metrics like total quantity sold, revenue, and discount value for both pre-discounting and post-discounting periods. It evaluates whether implemented discounts align with recommendations and quantifies the change in sales and revenue attributable to

discounting strategies. *This helps optimize pricing strategies, ensuring discounts effectively drive sales without unnecessary margin erosion, contributing to effective **OD** and **OW** strategies.*

2. Required Information

- **Sales Data:**
 - Sales records (from ImpactAnalysisData object, including quantity_sold, revenue, discount_amount).
- **Product Data:**
 - SKU and Style master data (from ImpactAnalysisData object).
- **Operational Data:**
 - Discounting analysis inputs (SKU, store, discount type, start date, end date, suggested discount rate).

3. Business Settings

- `discount.pre.period.days`: Duration (in days) before a discount event to establish a baseline. *Helps measure the true uplift.*
- `discount.post.period.days`: Duration (in days) after a discount event to measure its ongoing impact. *Captures the full effect of the promotion.*
- `discount.revenue.threshold`: Minimum revenue threshold for a discount event to be included in analysis. *Filters out insignificant promotional activities.*

4. Process Flow

graph TD

```
A[Enriched Data Model (from Data Module)] --> B[Identify Discounting Events]
B --> C[Define Pre/Post-Discount Periods]
C --> D[Calculate Pre-Discount Metrics (Sales, Revenue)]
D --> E[Calculate Post-Discount Metrics (Sales, Revenue)]
E --> F[Assess Discount Effectiveness]
F --> G[Discounting Impact Report]
```

5. Step-by-Step Actions

1. **Identify** all relevant discounting events from the input data.
2. **Define** specific pre-discount and post-discount sales periods for each event, *to isolate the impact of the discount.*
3. **Aggregate** sales quantity, revenue, and discount value for each SKU-store combination within the pre-discount period.
4. **Aggregate** sales quantity, revenue, and discount value for the post-discount period.
5. **Calculate** the change in quantity sold and revenue, as well as the average discount rate applied, *to quantify the direct impact of the discount.*
6. **Compare** implemented discounts against suggested discounts, *to evaluate adherence to pricing strategies.*
7. **Generate** a detailed Discounting Impact Report, *providing insights into promotional effectiveness and profitability.*

6. Sample Scenario

SKU303 at StoreX typically sells 50 units/week at \$10 each. A 20% discount is applied for a week. * **Pre-Discount (weekly)**: Sales 50 units, Revenue \$500. * **Post-Discount (weekly)**: Sales 80 units, Revenue \$640 (80 units * \$8). The module calculates: * Quantity increase: 30 units (60% uplift). * Revenue increase: \$140. * Average discount applied: 20%. * Effectiveness: High sales uplift, but also margin erosion. The report would highlight if this trade-off was beneficial.

Key Takeaway: *This module helps assess if discounts are generating enough incremental sales to offset reduced margins, guiding future pricing and promotional decisions for **OD** and **OW**.*

7. Results

- **Discounting Impact Report:** Tabular data for each analyzed discount event:
 - SKU, Store, Discount Type, Discount Period.
 - Pre-discount sales, revenue, quantity.
 - Post-discount sales, revenue, quantity.
 - Percentage change in sales and revenue.
 - Average discount rate applied.
 - Comparison of actual vs. suggested discount rates.

8. Unusual Cases

- **Missing or delayed data feed:** *Incomplete sales or discount event data will prevent accurate pre/post comparison.*
- **Overlapping discount events:** *Multiple discounts on the same SKU at the same time can confound individual impact analysis.*
- **External factors:** *Major holidays or competitor promotions during the discount period might distort the true impact attributable solely to the discount.*

9. Next Steps

The Discounting Impact Report is valuable for pricing teams and marketing departments. It informs decisions on discount depth, duration, and target products, optimizing promotional effectiveness and overall profitability.

10. Common Questions

1. How is “uplift” calculated in this context?
2. What is the recommended discount rate for maximum impact?
3. Can this module analyze specific **AG** (Attribute Group) performance under discount?
4. Does it consider the cannibalization effect of discounts?
5. How do we ensure discounts align with our overall business strategy?

11. Quick Reference

- **Purpose:** Quantify the impact of discounting on sales and revenue.
- **Inputs:** Enriched data, discount event details.

- **Outputs:** Discounting Impact Report.
 - **Key Metric:** Sales/Revenue uplift and margin impact.
-

9. Appendix

Full Parameter Reference

Refer to Section 6, “Global Configuration,” for a comprehensive list of all configurable parameters in `args.properties`.

Version History

- **1.0 (2025-05-23):** Initial release of the impact analysis project, including core data synchronization, preparation, IST, and Discounting analysis modules.

Contact & Support Info

For any questions, issues, or feature requests, please contact: * **Support Team:** support@yourcompany.com * **Project Lead:** projectlead@yourcompany.com

Tutorial: impact

The impact analysis project is orchestrated by the `ImpactAnalysisGroupModule`, which sequentially manages the synchronization of raw data, its subsequent preparation, and then the specialized analyses for discounting and inter-store transfers (ISTs). Data flows from initial synchronization to a centralized data preparation module, which then provides enriched data for the specific analysis modules.

flowchart TD

```
A0["Impact Analysis Sync Module"]
A1["Impact Analysis Group Module"]
A2["Impact Analysis Data Module"]
A3["Impact Analysis IST Module"]
A4["Impact Analysis Discounting Module"]
A1 -- "control_flow" --> A0
A1 -- "control_flow" --> A2
A1 -- "control_flow" --> A4
A1 -- "control_flow" --> A3
A2 -- "data_access" --> A0
A3 -- "data_access" --> A0
A4 -- "data_access" --> A0
A3 -- "data_flow" --> A2
A4 -- "data_flow" --> A2
```

Chapters

Master Report: impact Merchandising Algorithm

1. Title & Metadata

- **Project Name:** impact
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** impact optimizes merchandising strategies by leveraging data-driven insights to recommend ideal product assortments (OD), category breadth (OW), and size distributions (ISS), thereby maximizing sales and inventory efficiency.

2. Executive Summary

The impact merchandising algorithm is a robust, data-driven system designed to revolutionize how product assortments are planned and managed. By analyzing historical sales data, inventory levels, and product attributes, it provides precise recommendations for **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)** across various product categories and market segments. This ensures that merchandising efforts are aligned with consumer demand, minimizing stockouts and overstock situations, and ultimately driving profitability.

- **Key outcomes or KPIs:**
 - Increased **Rate of Sale (ROS)** per SKU and category.
 - Optimized inventory turnover and reduced carrying costs.
 - Higher customer satisfaction through improved product availability.
 - Maximized sales revenue by stocking the right products in the right quantities.
 - Reduced manual effort in assortment planning.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Category Managers, Inventory Planners, Store Operations, E-commerce Teams.

3. Architecture & Data Flow

graph TD

```
A[Raw Data Sources] --> B{Data Ingestion & Validation};
B --> C[Cleaned & Transformed Data];
C --> D[OD & OW Calculation Module];
C --> E[ISS Determination Module];
D --> F[Recommendation Generation Module];
E --> F;
```

F --> G[Final Merchandising Recommendations];
G --> H[BI Dashboards / Merchandising Systems];

- **Component Bullets:**

- **Data Ingestion & Validation:** Responsible for collecting raw data from various sources, performing initial data cleaning, validation, and transformation into a standardized format.
- **OD & OW Calculation Module:** Analyzes market trends, historical **ROS**, and product performance to determine the ideal number of unique items (OD) within a category and the ideal number of categories or styles (OW) to offer.
- **ISS Determination Module:** Focuses on size-level sales data and stockout history within each **Attribute Group (AG)** to recommend the most efficient size distribution (ISS) per SKU.
- **Recommendation Generation Module:** Consolidates outputs from the OD/OW and ISS modules, applies business rules and constraints, and generates the final, actionable merchandising recommendations.
- **Output & Integration Layer:** Delivers the recommendations to downstream systems and dashboards for consumption by stakeholders.

- **High-level data sources:**

- Point-of-Sale (POS) data (historical sales, returns, units sold).
- Inventory management systems (current stock levels, stockout history, on-order quantities).
- Product master data (SKU attributes, category hierarchies, pricing).
- Web analytics data (page views, conversion rates by product).
- Market trend reports and competitive intelligence.

4. Configuration & Data Dictionary

Settings Table (args.properties)

Parameter Name	Description	Default Value	Type
min_ros_threshold	Minimum acceptable ROS for a SKU to be considered for OD/OW.	0.1 units/ week	float
max_od_limit_per_category	Upper bound for OD recommendations in any single category.	50	int
iss_confidence_level	Statistical confidence level for ISS recommendations.	0.95	float
historical_data_window	Number of weeks of historical data	52 weeks	int

Parameter Name	Description	Default Value	Type
seasonal_adjustment_factor_enabled	to consider for analysis. Flag to enable or disable seasonal adjustments in ROS calculation.	true	boolean
category_min_ow_threshold	Minimum recommended OW for a newly introduced category.	3	int
attribute_group_similarity_threshold	Similarity score for grouping products into an AG .	0.8	float

Shared Data Fields (common-data.json)

Field Name	Description	Data Type	Example Value	Used By
sku_id	Unique identifier for a Stock Keeping Unit.	string	P0012345	All Modules
category_id	Identifier for the product's primary category.	string	APPAREL_TSHIRTS	OD/ OW, ISS
attribute_group_id	Identifier for an AG (e.g., "Men's Basic Tee").	string	AG_MENS_BASIC_TEE	ISS
size	Product size (e.g., S, M, L, XL, 32, 34).	string	L	ISS
sales_units_weekly	Units sold per SKU per week.	float	15.5	All Modules
inventory_units	Current inventory available.	int	120	ISS
stockout_days	Number of days a SKU	int	7	ISS

Field Name	Description	Data Type	Example Value	Used By
	has been out of stock in a given period.			
cost_of_goods_sold	Cost associated with selling a particular SKU.	float	12.50	OD/OW
retail_price	Current retail price of the SKU.	float	29.99	All Modules

Outputs Catalog

Output File/Data Structure	Description	Format	Frequency
optimal_assortment_recommendations.csv	Contains the recommended OD per category, a list of recommended SKUs to stock, and the corresponding OW across categories/styles. Includes an estimated uplift in ROS .	CSV	Weekly
ideal_size_set_recommendations.json	Provides detailed ISS per sku_id or attribute_group_id, indicating the optimal percentage or count of each size to stock to minimize stockouts and maximize sales.	JSON	Weekly
performance_metrics_dashboard.json	Aggregated KPIs including current vs. recommended ROS , inventory efficiency, stockout rates, and projected revenue impact. Designed for consumption by BI tools.	JSON	Daily/Weekly
alert_log.txt	Logs critical alerts such as data quality issues, deviations from expected performance,	TXT	Daily

Output File/Data Structure	Description	Format	Frequency
	or significant changes in market dynamics that warrant manual review.		

5. End-to-End Flow

The impact algorithm executes a structured, multi-stage process to deliver merchandising recommendations:

1. **Data Ingestion:** Raw sales, inventory, and product master data are collected from various enterprise systems.
2. **Data Validation & Cleansing:** The ingested data undergoes rigorous checks for completeness, consistency, and accuracy. Missing values are imputed, and outliers are identified and addressed. Data is transformed into a unified schema.
3. **Feature Engineering:** Key metrics such as historical **ROS** (calculated as `sales_units_weekly / inventory_units` adjusted for stockouts), inventory turnover, and profitability margins are derived. Products are grouped into relevant **Attribute Groups (AGs)** based on their characteristics.
4. **OD & OW Calculation:** The system analyzes historical performance, market trends, and product lifecycle stages. It identifies top-performing SKUs and categories, suggesting the **Optimum Depth (OD)** (number of SKUs) within each category and the **Optimum Width (OW)** (number of categories/styles) to maintain a balanced assortment. This stage often involves clustering and ranking algorithms.
5. **ISS Determination:** For each product or **AG**, the algorithm examines size-level sales patterns, historical stockouts, and regional preferences. It then models the optimal proportion of each size to constitute the **Ideal Size Set (ISS)**, aiming to maximize unit sales while minimizing excess inventory or missed sales due to stockouts.
6. **Recommendation Generation:** The calculated OD, OW, and ISS recommendations are combined. Business rules (e.g., minimum stock levels, promotional considerations, new product launch requirements) are applied to fine-tune the outputs.
7. **Output & Reporting:** The final, actionable recommendations are generated in structured formats (CSV, JSON) and pushed to downstream consumption points like BI dashboards or direct integration into planning systems. Performance metrics are tracked and reported.

6. Submodule Overview

6.1. Data Ingestion & Validation

- **Purpose:** Gathers and validates raw input data from disparate sources, preparing it for downstream analysis.
- **Inputs & Outputs:**
 - **Inputs:** Raw POS data, inventory data, product master data, marketing data.
 - **Outputs:** Cleaned & Transformed Data (`common-data.json` schema).

- **Core Logic Summary:**
 - Connects to various databases and APIs to extract required data.
 - Performs schema validation and data type conversions to ensure consistency.
 - Identifies and handles missing values (e.g., imputation with category averages or flagging for review).
 - Detects and flags outliers in sales or inventory data that might skew calculations.

6.2. Optimal Depth (OD) & Width (OW) Calculation

- **Purpose:** Determines the ideal number of unique items (SKUs) within a category and the ideal number of different categories or styles to offer.
- **Inputs & Outputs:**
 - **Inputs:** Cleaned & Transformed Data (especially sku_id, category_id, sales_units_weekly, retail_price, cost_of_goods_sold).
 - **Outputs:** Recommended OD per category, Recommended OW, prioritized SKU list per category.
- **Core Logic Summary:**
 - Calculates **ROS** and profitability per SKU and per category based on historical data.
 - Applies a product lifecycle model to categorize SKUs (e.g., new, mature, declining) influencing their OD contribution.
 - Utilizes a portfolio optimization algorithm to balance sales potential with inventory holding costs and identify the top SKUs for inclusion, defining **OD**.
 - Analyzes market trends and competitive landscape to recommend the optimal **OW** (category breadth) for overall assortment health.

6.3. Ideal Size Set (ISS) Determination

- **Purpose:** Identifies the best combination of sizes to stock for individual products or **Attribute Groups (AGs)** to maximize sales and minimize stockouts.
- **Inputs & Outputs:**
 - **Inputs:** Cleaned & Transformed Data (especially sku_id, size, attribute_group_id, sales_units_weekly, inventory_units, stockout_days).
 - **Outputs:** Recommended ISS per SKU/AG, size distribution percentages.
- **Core Logic Summary:**
 - Analyzes historical **ROS** for each size within a specific SKU or **AG**.
 - Incorporates stockout frequency and duration to understand missed sales opportunities for particular sizes.
 - Applies statistical modeling (e.g., demand forecasting, percentile analysis) to predict future demand for each size and determine the **ISS**.
 - Considers regional or channel-specific size preferences if available in the data.

6.4. Recommendation Generation & Output

- **Purpose:** Consolidates all calculated optimal metrics and generates the final, actionable merchandising recommendations in user-friendly formats.

- **Inputs & Outputs:**
 - **Inputs:** Recommended OD per category, Recommended OW, Recommended ISS per SKU/AG, `args.properties` (for business rules).
 - **Outputs:** `optimal_assortment_recommendations.csv`, `ideal_size_set_recommendations.json`, `performance_metrics_dashboard.json`.
- **Core Logic Summary:**
 - Merges the OD/OW recommendations with the granular ISS outputs to create comprehensive plans.
 - Applies configurable business constraints and rules (e.g., minimum SKU counts for foundational items, promotional overrides).
 - Formats the data into specified output structures (CSV, JSON) for easy consumption by stakeholders and downstream systems.
 - Generates summary performance metrics and a log of any anomalies or overrides.

7. Worked Example

Let's consider a simple scenario for the "Men's Basic Tee" category (Category ID: `APPAREL_MENS_TEE`) and an **Attribute Group (AG)**: `AG_MENS_BASIC_TEE`.

Scenario: We have 10 SKUs in this category. For simplicity, let's focus on one SKU (`SKU_001_Blue_M_Tee`) and its size distribution.

Input Data Snippet (simplified):

SKU_ID	Category_ID	AG_ID	Size	Weekly_Sales_U
SKU_001_Blue_S	APPAREL_MENS_TEE	AG_MENS_BASIC_TEE	S	8
SKU_001_Blue_M	APPAREL_MENS_TEE	AG_MENS_BASIC_TEE	M	25
SKU_001_Blue_L	APPAREL_MENS_TEE	AG_MENS_BASIC_TEE	L	18
SKU_001_Blue_XL	APPAREL_MENS_TEE	AG_MENS_BASIC_TEE	XL	10
... (other colors/ styles)	...	...	...	...

Processing Steps:

1. **ROS Calculation:**
 - For `SKU_001_Blue_M`, historical data shows 25 units sold/week. If 5 units are in stock and 5 days of stockout occurred, the effective demand could be higher. The algorithm calculates the true **ROS** adjusting for stockouts. Let's assume adjusted **ROS** for M is 30 units/week.
 - ROS for S = 8 units/week.
 - ROS for L = 18 units/week.
 - ROS for XL = 10 units/week.
2. **OD & OW Calculation (Simplified for one category):**
 - The OD/OW module would analyze all 10 SKUs in `APPAREL_MENS_TEE`. Based on profitability, sales velocity, and uniqueness, it might recommend an **OD** of 7 SKUs for this category.
 - For example, it might identify `SKU_001_Blue`, `SKU_002_Grey`, `SKU_003_White` (as core styles), plus 4 others. It might recommend discontinuing 3 underperforming SKUs.

- **OW** would be calculated at a higher level (e.g., across “Apparel”, “Footwear”, “Accessories”).
- 3. ISS Determination (for AG_MENS_BASIC_TEE):**
- The total demand across all sizes for AG_MENS_BASIC_TEE is aggregated.
 - For SKU_001_Blue_M, the high **ROS** and stockout history signal high demand and potential missed sales.
 - The algorithm calculates the historical proportion of sales per size:
 - S: $8 / (8+30+18+10) = 8 / 66 = 12\%$
 - M: $30 / 66 = 45\%$ (original 25 adjusted for 5 days stockout)
 - L: $18 / 66 = 27\%$
 - XL: $10 / 66 = 15\%$
 - Based on these proportions and a desired service level (e.g., 98% in-stock), the **ISS** is determined. If we decide to stock 100 units of AG_MENS_BASIC_TEE (across all sizes) in a store:
 - S: 12 units
 - M: 45 units
 - L: 27 units
 - XL: 15 units

Output Snippet:

- **optimal_assortment_recommendations.csv:**

- Category_ID: APPAREL_MENS_TEE
- Recommended_OD: 7
- Recommended_SKUs: SKU_001_Blue_M_Tee, SKU_001_Blue_L_Tee, SKU_001_Blue_XL_Tee, SKU_001_Blue_S_Tee, SKU_002_Grey_M_Tee, etc.
- Projected_Weekly_ROS_Uplift: +15%

- **ideal_size_set_recommendations.json:**

```
{
  "attribute_group_id": "AG_MENS_BASIC_TEE",
  "recommended_iss_distribution": {
    "S": 0.12,
    "M": 0.45,
    "L": 0.27,
    "XL": 0.15
  },
  "iss_comment": "Based on 52-week sales history and stockout
    adjustments. High demand for M."
}
```

This example shows how impact uses granular data to provide actionable recommendations for both assortment breadth and depth, down to the size level.

8. Debug Guide Highlights

- **Data Ingestion Failures:**

- **Symptom:** ERROR: Data source connection failed. or Input file not found.

- **Cause:** Incorrect database credentials, network issues, or misconfigured file paths in `args.properties`.
- **Action:** Verify `args.properties` parameters (`db_url`, `file_paths`). Check network connectivity and database status.
- **Missing or Inconsistent Data:**
 - **Symptom:** WARNING: Missing 'sales_units_weekly' for SKU_ID: P12345 or ERROR: Data type mismatch in 'retail_price' for SKU_ID: P67890.
 - **Cause:** Source data quality issues, unexpected nulls, or incorrect data types from upstream systems.
 - **Action:** Review source data for specific SKUs. Implement stricter data validation rules or additional imputation logic in the Data Ingestion module.
- **Unreasonable OD/OW/ISS Recommendations:**
 - **Symptom:** Recommended **OD** of 1 for a core category, **OW** too low/high, or **ISS** with 0% for a common size.
 - **Cause:** Calculation parameters (`min_ros_threshold`, `max_od_limit`, `iss_confidence_level`) are too strict/loose, or underlying data is skewed by anomalies.
 - **Action:** Adjust `args.properties` parameters. Check for extreme outliers in historical **ROS** or stockout data that might be skewing results. Review `seasonal_adjustment_factor_enabled`.
- **Performance Degradation:**
 - **Symptom:** Run time significantly longer than usual, or out-of-memory errors.
 - **Cause:** Processing large volumes of data without proper optimization, inefficient queries, or increased data volume.
 - **Action:** Optimize database queries, consider parallel processing, increase allocated memory, or review data retention policies to reduce historical data window if not critical.

9. Common Pitfalls & Remedies

- **Pitfall 1: Insufficient or Low-Quality Data**
 - **Description:** The algorithm's performance is heavily dependent on clean, comprehensive, and accurate historical sales, inventory, and product data. Missing data, incorrect attributes, or large gaps can lead to unreliable recommendations.
 - **Remedy:** Implement robust data governance practices upstream. Utilize the Data Ingestion & Validation module's error logging to identify recurring data quality issues. Establish SLAs for data source providers.
- **Pitfall 2: Neglecting Seasonality and Trends**
 - **Description:** Merchandising is highly seasonal. A static model that doesn't account for holiday spikes, seasonal shifts, or emerging trends will provide suboptimal recommendations.
 - **Remedy:** Ensure `seasonal_adjustment_factor_enabled` is true. Regularly update historical data windows. Incorporate external market trend data (if available) into the **OD/OW** calculation module.
- **Pitfall 3: Over-reliance on Historical Data for New Products**
 - **Description:** New products have no historical sales, making it challenging for the algorithm to determine their **ROS**, **OD**, or **ISS**.
 - **Remedy:** Implement a "new product ramp-up" strategy where initial stock is based on similar **Attribute Group (AG)** performance or manual expert

input. Gradually transition to data-driven recommendations as sales data accumulates.

- **Pitfall 4: Ignoring Business Constraints and Strategy**

- **Description:** Purely data-driven recommendations might contradict strategic goals (e.g., promoting a specific brand, clearing old inventory, maintaining minimum stock for brand image).
- **Remedy:** Ensure the Recommendation Generation module has configurable business rules and override mechanisms (`args.properties`). Foster strong collaboration between the algorithm team and merchandising stakeholders for feedback and adjustment.

10. FAQ

- **Q1: How often are the merchandising recommendations updated?**

- **A:** The **impact** algorithm is designed to run weekly, typically on Monday mornings, to provide fresh recommendations based on the previous week's sales data. Critical reports or alerts can be generated daily.

- **Q2: Can I manually override the recommendations provided by **impact**?**

- **A:** Yes, the system generates recommendations, but merchandisers retain the final decision-making authority. Overrides can be input into the system, and the algorithm learns from these manual adjustments over time, improving its future recommendations.

- **Q3: What if a new product launches with no historical sales data?**

- **A:** For new product launches, **impact** initially bases its recommendations on performance data from similar **Attribute Groups (AGs)** or established product hierarchies. As sales data accumulates, the recommendations become increasingly data-driven and tailored.

- **Q4: How does **impact** account for promotions or markdown events?**

- **A:** The algorithm includes logic to identify and adjust for significant promotional periods. During **ROS** calculations, it differentiates between regular and promotional sales velocity to avoid overstocking based on temporary spikes. Markdown events are also factored into profitability calculations.

- **Q5: What do OD, OW, ISS, ROS, and AG mean?**

- **A:**
 - **OD (Optimum Depth):** The ideal number of unique items (SKUs) to stock in a category or display.
 - **OW (Optimum Width):** The ideal number of different categories or styles to offer.
 - **ISS (Ideal Size Set):** The best combination of sizes to maximize sales and minimize stockouts.
 - **ROS (Rate of Sale):** The speed at which products are sold over a period (e.g., units/week).
 - **AG (Attribute Group):** A set of product attributes grouped for analysis (e.g., color, size, brand).

11. Quick Reference Cheat-Sheet

Term/ Concept	Definition	Key Parameter/Output
OD (Optimum Depth)	Ideal number of unique items (SKUs) within a category.	max_od_limit_per_category, optimal_assortment_recommen
OW (Optimum Width)	Ideal number of different categories or styles to offer.	category_min_ow_threshold, optimal_assortment_recommen
ISS (Ideal Size Set)	Best combination of sizes to maximize sales and minimize stockouts.	iss_confidence_level, ideal_size_set_recommendat
ROS (Rate of Sale)	Speed at which products are sold over a period (e.g., units/week).	min_ros_threshold, sales_un (input)
AG (Attribute Group)	Set of product attributes grouped for analysis (e.g., color, size, brand).	attribute_group_id (shared fie
Configuration File	args.properties: Defines runtime parameters and thresholds.	min_ros_threshold, historical_data_window
Primary Output	optimal_assortment_recommendations.csv: Main recommendations for OD & OW.	Category_ID, Recommended_OD Recommended_SKUs
Secondary Output	ideal_size_set_recommendations.json: Detailed size distribution.	attribute_group_id, recommended_iss_distributio
Data Sources	POS, Inventory, Product Master Data.	sku_id, sales_units_weekly, inventory_units

Tutorial: StyleWiseToSizeWise

The StyleWiseToSizeWise process is structured as a sequential pipeline where initial calculations for size contribution drive subsequent transformations of style-level purchasing into detailed size-level buys. The final processed data is then handled by utility modules for snapshotting and synchronization.

flowchart TD

```
A0["ISSChannelAgSizeContri"]
A1["StyleSkuBuyConversionModule"]
A2["UtilOutputSnapshotModule"]
A3["UtilOutputSyncModule"]
A0 -- "data_flow" --> A1
A1 -- "data_flow" --> A2
A1 -- "data_flow" --> A3
```

Chapters

Chapter 1: ISSPrepareSalesModule

Last Updated: 2025-05-23

1. Business Objective

This step aggregates and refines sales data, identifying periods where essential product sizes were consistently available, to ensure the Ideal Size Set (ISS) calculation is based on reliable sales performance.

Why This Matters: This process ensures that our recommendations for Ideal Size Sets are based on actual customer demand when the right sizes were in stock, preventing skewed results from periods of stockouts or incomplete inventory. This directly leads to more accurate inventory planning, optimized stock levels, reduced lost sales from missing sizes, and ultimately, improved customer satisfaction and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Individual Sales Transactions:** Details of every product sold, including item, quantity, date, and store. We need this to understand actual customer demand.
- **Product Sales History:** A comprehensive record of sales performance over time. This helps us analyze trends and identify patterns relevant to size demand.

Product Data

- **Product Variant Details (SKU Information):** Information about each unique item, including its size, style, and associated AG (Attribute Group). This allows us to link sales to specific sizes and product classifications.
- **Product Attribute Groups:** Pre-defined groupings of product characteristics (like gender, brand, category, subcategory) that help us analyze sales across relevant product segments. This provides context for size demand within specific product types.

Operational Data

- **In-Stock Inventory Records (GRN Data):** Information on when specific product sizes were received and available for sale at each store. This is crucial for determining if sales occurred when a full size range was accessible.

- **Core Size Definitions (Paramount Sizes):** Pre-established lists of essential or “core” sizes for specific product categories (e.g., S, M, L for apparel). This helps us filter sales data to ensure we are only considering periods where these critical sizes were adequately available.
- **Store Details:** Information about each retail location, including its channel, region, and unique identifiers. This helps us analyze sales performance by store and regional clusters.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Sales Analysis Period	6 months	The duration (in months) of historical sales data to analyze for Ideal Size Set calculations.	Adjust to capture seasonal trends or react faster to market shifts (e.g., 3 months for fast fashion, 12 months for staples).
Minimum Available Size Range Percentage	70%	The minimum percentage of a product’s expected size range that must have been in stock (based on GRN data) for sales to be considered “consistent.”	Increase if stockouts are a major issue and you want to analyze only near-perfect inventory scenarios. Decrease if you need more data points and are willing to accept some inventory gaps.
Minimum Core Size Availability Percentage	80%	The minimum percentage of the pre-defined “Core Sizes” (Paramount Sizes) that must have been in stock for sales to be considered “consistent.”	Increase for essential items where core sizes must always be present. Decrease if some core sizes are less critical or have frequent stockouts.
Current Reporting Date	Today’s Date	The most recent date for which sales data is considered. Sales are analyzed backwards from this date.	Usually remains the system’s current date, but can be adjusted for historical analysis or future planning simulations.

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Transactions & Inventory Data] --> B{Aggregate Sales Data for Sales Analysis Period}

B --> C{Identify & Filter for Consistent Sales (Unbroken Sales)}

C --> D[Standardize & Aggregate Sales Data for ISS]

D --> E[Refined Sales Data for Ideal Size Set Calculation]

1. Start with → Raw Sales Transactions, Product Variant Details, In-Stock Inventory Records, and Core Size Definitions.
2. Then → Filter sales to include only those within the defined Sales Analysis Period.
3. Next → Apply checks to identify “Consistent Sales,” meaning sales that occurred when core sizes and a sufficient range of other sizes were truly available in the store. Sales from periods with significant stockouts of key sizes are excluded.
4. Finally → Aggregate and standardize the Consistent Sales data by store, category, brand, gender, and individual size, preparing it for the Ideal Size Set calculation. This output is the Refined Sales Data for Ideal Size Set Calculation.

5. Step-by-Step Actions

1. **Collect** sales data for the Sales Analysis Period.
 - What we do: *We gather all individual sales transactions that occurred within the specified historical timeframe, looking back from the Current Reporting Date.*
 - Why we do it: *This ensures we're working with relevant and recent demand patterns for calculating Ideal Size Sets.*
2. **Identify** In-Stock Sizes using inventory records.
 - What we do: *For each product style and store, we determine which sizes were physically available for sale on any given day, using our Goods Received Note (GRN) data.*
 - Why we do it: *Sales data alone doesn't tell us if a customer couldn't buy a size because it was out of stock. Knowing what was in stock helps us understand true demand versus unmet demand.*
3. **Cross-reference** sales with Core Size Definitions.
 - What we do: *We refer to pre-defined lists of “Core Sizes” (e.g., the most popular or essential sizes for a particular clothing category). For each sales transaction, we check if these core sizes were sufficiently available at the time of sale.*
 - Why we do it: *This is a critical filter. If a product category's core sizes were mostly out of stock, then any sales recorded during that period might not represent typical demand for the Ideal Size Set. We want to analyze sales from “normal” inventory conditions.*
4. **Filter** sales to identify “Consistent Sales.”
 - What we do: *We only keep sales transactions that meet two key criteria: 1) A high percentage (e.g., 70%) of the total size range was available in stock, AND 2) A high percentage (e.g., 80%) of the core* sizes for that product*

category were available in stock. Sales not meeting these thresholds are excluded.*

- Why we do it: *This ensures that the sales data used for ISS calculations reflects periods when customers had a real choice of sizes, making the data highly reliable for future size planning. We're removing sales that occurred under significant inventory constraints.*

5. **Aggregate** and denormalize “Consistent Sales” data.

- What we do: *We group the “Consistent Sales” data by store, category, subcategory, gender, brand, and specific size, then sum up the Units Sold for each group. We also add standard size sequence and rank information.*
- Why we do it: *This consolidates the granular sales data into a format that is ready for the next steps of Ideal Size Set calculation, making it easier to analyze overall demand patterns across different product dimensions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A store is analyzing sales for “Women’s Casual Dresses” over the **Sales Analysis Period** of the last 6 months.
- The “Core Sizes” for this category are Small, Medium, Large, and X-Large.
- The **Minimum Available Size Range Percentage** is 70% and the **Minimum Core Size Availability Percentage** is 80%.
- On a particular week, the store sold 50 units of “Floral Midi Dress” (Style ID 12345).
- During that week, inventory records show:
 - Small: Available (in stock)
 - Medium: Available (in stock)
 - Large: Out of stock (only 2 units received all week)
 - X-Large: Available (in stock)
 - XX-Large: Available (in stock)
 - XXX-Large: Not typically stocked for this style

What Happens

1. **Sales Collection:** The system first gathers the 50 units sold for “Floral Midi Dress” in that week.
2. **In-Stock Size Check:** It looks at the inventory records. For the “Floral Midi Dress”, 5 out of 6 possible sizes (S, M, L, XL, XXL, XXXL) were generally available, but the Large size, which is a **Core Size**, was mostly out of stock.
3. **Core Size Availability Check:** It checks if 80% of Core Sizes (S, M, L, XL) were available. Only S, M, and XL were consistently available. Large was not. So, 3 out of 4 core sizes were available (75%), which is *below* the 80% threshold.
4. **Overall Size Range Availability Check:** It checks if 70% of the *total* expected size range was available. Even though most sizes were available, the missing Large size, being a core size, impacts the overall health of the size set during that period.

5. **Filtering for “Consistent Sales”:** Because the availability of **Core Sizes** fell below the 80% threshold, the sales of these 50 units during this week are *not* considered “Consistent Sales.” They are excluded from the data used for the Ideal Size Set calculation.

Final Outcome

The 50 units sold for “Floral Midi Dress” during that week are excluded from the data used to calculate the Ideal Size Set. This means the ISS calculation will not be skewed by sales that occurred when a crucial size (Large) was missing.

Key Takeaway: By rigorously filtering sales data based on actual inventory availability, this module ensures that Ideal Size Set recommendations are based on true, unconstrained customer demand, leading to better inventory decisions.

7. Results

This step produces:

Main Outputs

- **Aggregated Consistent Sales Data:** A comprehensive dataset of sales by store, product category, subcategory, gender, brand, and size, for periods when adequate inventory was available. It tells us the reliable units sold for each specific size across different product groups.
- **Denormalized Sales for ISS:** A flat, ready-to-use table that combines sales quantity with important product and store attributes, including size sequence and rank. This output provides the foundational data for all subsequent Ideal Size Set calculations.

Business Reports

- **Sales Data Reliability Report:** A report for merchandising and inventory teams showing which product categories and stores had “Consistent Sales” data versus periods where sales were likely impacted by significant stockouts of core sizes. Used by planners to understand data quality and potential gaps.
- **Preliminary Size Performance Overview:** A high-level view of aggregated sales for available sizes, even before the full ISS calculation. Used by category managers to get an initial sense of size demand patterns.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process data up to the last available record, potentially missing	Verify data source connections; escalate to IT/data engineering team for missing feeds; consider running the module again once data is available.

Situation	What Happens	What You Should Do
No sales found within the Sales Analysis Period for a product category	recent sales or inventory updates. The module will proceed but will not generate any “Consistent Sales” data for that category, potentially leading to no ISS recommendation.	Check if the product is new, seasonal, or a slow mover. Adjust the Sales Analysis Period or consider manual review for new/ niche products.
Very few “Consistent Sales” identified for a product category	The filtering criteria are too strict, or the product consistently struggles with core size stockouts, leading to insufficient data for ISS.	Review the Minimum Available Size Range Percentage and Minimum Core Size Availability Percentage settings; investigate underlying inventory issues and potential stockout reasons.
Inaccurate or incomplete inventory (GRN) data	The module might misclassify sales as “consistent” when stockouts actually occurred, or vice-versa, leading to misleading ISS recommendations.	Implement robust data validation processes for GRN data; reconcile physical inventory with system records regularly.

9. Next Steps

Before This Step

- **Data Ingestion and Cleansing:** Raw sales transactions, product master data, and inventory records must be loaded and cleaned to ensure accuracy and completeness. This module relies on high-quality input data.
- **Attribute Group Definition:** Product Attribute Groups (AG) must be defined and applied to all products, providing the necessary classifications for analysis. This helps in grouping similar products for size analysis.
- **Core Size (Paramount Sizes) Definition:** Lists of essential core sizes for various product categories must be pre-defined and available. These definitions are crucial for filtering “Consistent Sales.”

After This Step

- **Ideal Size Set Calculation:** The aggregated and refined sales data from this module serves as the primary input for the next stage, where the Ideal Size Set (ISS) for each product category and style will be mathematically determined.
- **Inventory Optimization and Allocation:** The calculated Ideal Size Sets will then inform inventory planners on how to optimize stock levels for each size, leading to better allocation decisions and improved in-store availability.

10. Common Questions

Q: Why are some sales excluded from the “Consistent Sales” data? A: Sales are excluded if they occurred during periods where a significant number of core product sizes or the overall product size range was out of stock. This ensures our analysis is based on true customer demand when a full selection was available.

Q: How far back does this module look at sales data? A: It typically looks at sales data for the past 6 months, though this “Sales Analysis Period” can be adjusted based on business needs, such as for seasonal items or fast-moving fashion.

Q: What happens if there isn’t enough “Consistent Sales” data for a particular product or category? A: If there isn’t enough reliable sales data, the system might not be able to generate an Ideal Size Set recommendation for that specific product or category. In such cases, a manual review or adjustments to the data filtering settings might be necessary.

Q: Does this module account for new product launches that don’t have much sales history? A: This module primarily focuses on historical sales. For new launches, initial size sets might rely on business rules or similar product categories until sufficient “Consistent Sales” data is accumulated.

Q: How does this module help reduce stockouts? A: By providing Ideal Size Sets based on accurate, unconstrained demand, it helps ensure the right sizes are ordered and stocked in the right quantities, minimizing the risk of selling out of popular sizes and improving overall inventory efficiency.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To prepare clean, reliable sales data for Ideal Size Set (ISS) calculation by filtering out sales impacted by significant stockouts of key sizes.
- **Main Input:** Raw sales transactions, product variant details, in-stock inventory records, and core size definitions.
- **Main Output:** Aggregated “Consistent Sales” data, ready for ISS calculation.
- **Warning Signs:** Low volumes of “Consistent Sales” for well-performing categories, or discrepancies between expected and actual size demand.
- **Key Settings:** Sales Analysis Period, Minimum Available Size Range Percentage, Minimum Core Size Availability Percentage.

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 2: ISSFlaggingModule

Last Updated: 2025-05-23

1. Business Objective

The ISSFlaggingModule tags store and product characteristic combinations (Attribute Groups) based on reliable sales data, identifying specific segments of sales for detailed Ideal Size Set (ISS) analysis. It groups sales at various levels (like store, channel, region) and product characteristics, then calculates preliminary metrics and initializes analysis objects, flagging them according to pre-defined criteria.

Why This Matters: This process is crucial because it ensures that our sophisticated Ideal Size Set calculations are applied only to the most relevant and sufficient sales data. By intelligently grouping and flagging data, we avoid performing complex analysis on unreliable or insufficient sales, leading to more accurate size recommendations, better inventory allocation, and ultimately, enhanced sales and reduced stockouts.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Aggregated Consistent Sales Data:** This is the cleaned and filtered sales data from the previous step, which represents periods where key product sizes were consistently available. We need this to ensure our analysis is based on true customer demand, not demand constrained by stockouts.
- **Full Product Sales History:** While the primary input is 'consistent sales,' we also refer to the complete sales history to ensure all relevant sales, including those that might not meet strict consistency criteria, are considered for broader flagging (e.g., for reporting).

Product Data

- **Product Variant Details (SKU Information):** Information about each unique item, including its specific size and the Attribute Group (AG) it belongs to. This is essential for linking sales to product sizes and their broader characteristics.
- **Product Attribute Groups (AGs):** Pre-defined groupings of product characteristics (like category, gender, brand) that help us analyze sales across

relevant product segments. This allows us to group sales of similar products for size analysis.

- **Category Size Sequences:** Master data that defines standard size sequences (e.g., S, M, L, XL) for different product categories. This helps the system understand the expected range and order of sizes for a given product type.

Operational Data

- **Store Details:** Information about each retail location, including its channel, region, and unique identifiers. This is vital for grouping and flagging sales data by specific store, channel, or region for localized analysis.

3. Business Settings

Setting Name	Default Value	Description	When to Change
Sales Data Grouping Level	Store-AG	Defines the granularity at which sales data should be grouped and analyzed (e.g., store level, region level, or overall).	Adjust if you need ISS recommendations at a broader level (e.g., for regional warehouses) or a more specific level (e.g., for individual stores).
Minimum Sales Quantity Threshold	10 units	The minimum total Units Sold required for a store-Attribute Group combination to be considered for detailed ISS calculation. Combinations below this threshold might not have sufficient data for reliable analysis.	Increase for high-volume items or if you want to ensure highly reliable data points. Decrease for niche products or new launches where less data is available.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Refined Sales Data for Ideal Size Set Calculation] --> B[Group Sales & Calculate Preliminary Metrics]

B --> C{Flag Relevant Combinations}
C --> D[Flagged Sales Data for ISS Calculation]

1. Start with → Refined Sales Data for Ideal Size Set Calculation (from the previous step).
2. Then → Group sales data by various combinations of store, channel, region, and Attribute Group based on the Sales Data Grouping Level setting. Calculate preliminary metrics like total Units Sold for each grouping.
3. Next → Apply the Minimum Sales Quantity Threshold. For combinations meeting this threshold, initialize a new Ideal Size Set (ISSData) object and assign a specific “flag” indicating the level of aggregation (e.g., store-level, channel-region level).
4. Finally → Produce Flagged Sales Data for ISS Calculation, which is a collection of ISSData objects, each representing a distinct, relevant store-Attribute Group combination ready for detailed size set computation.

5. Step-by-Step Actions

1. **Initialize** Category Size Sequences and Attribute Group Size Maps.
 - What we do: *We load pre-defined lists of standard size sequences for various product categories and map each Attribute Group to the full range of sizes it typically includes.*
 - Why we do it: *This sets up the foundational reference data needed to understand the expected size landscape for any given product.*
2. **Assign** Size Sequences to Sales Records.
 - What we do: *For each individual sales transaction, we determine which standard size sequence (e.g., S, M, L, XL) corresponds to the style sold and tag the sales record with this sequence ID.*
 - Why we do it: *This ensures that all sales data is consistently categorized by its size structure, making it easier to compare and analyze demand patterns across different products.*
3. **Group** Sales Data for Analysis.
 - What we do: *We systematically group the “Consistent Sales” data by different levels of detail, such as individual store, channel-region combinations, and broader regions, and also by Attribute Group.*
 - Why we do it: *This prepares the data for multi-dimensional analysis, allowing us to generate size set recommendations tailored to specific local needs or broader strategic requirements based on the Sales Data Grouping Level.*
4. **Apply** Minimum Sales Quantity Threshold.
 - What we do: *For each grouped combination of sales (e.g., Store A - Women’s Jeans), we calculate the total Units Sold and compare it against the Minimum Sales Quantity Threshold.*
 - Why we do it: *This ensures that only combinations with sufficient sales volume are passed on for complex ISS calculations, preventing skewed or unreliable recommendations from too little data.*
5. **Flag** Relevant Store-Product Combinations.
 - What we do: *For every store-Attribute Group combination that meets the sales threshold, we initialize an Ideal Size Set (ISSData) object, populate it with preliminary sales metrics for each size, and assign a specific “flag” indicating the rollup level used for that data (e.g., store, channel, or region).*

- Why we do it: *These flags categorize and prepare the data for the next phase of detailed Ideal Size Set calculation, streamlining the process and ensuring the right analysis is applied to the right data segment.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- We have “Consistent Sales” data for “Women’s Active Leggings” (an Attribute Group) across several stores.
- The **Sales Data Grouping Level** is set to analyze at the individual store level.
- The **Minimum Sales Quantity Threshold** is 10 units.
- Sales for “Women’s Active Leggings” in the last 6 months:
 - Store A: 120 units sold
 - Store B: 8 units sold
 - Store C: 45 units sold

What Happens

1. **Sales Grouping:** The system first groups the “Consistent Sales” data for “Women’s Active Leggings” by each individual store.
2. **Threshold Check:**
 - For Store A, 120 units sold is *above* the 10-unit threshold.
 - For Store B, 8 units sold is *below* the 10-unit threshold.
 - For Store C, 45 units sold is *above* the 10-unit threshold.
3. **Flagging:**
 - An ISSData object is created for “Store A - Women’s Active Leggings,” containing details of sales per size (e.g., 20 S, 50 M, 30 L, 20 XL) and is assigned a flag indicating it’s a store-level analysis.
 - No ISSData object is created for “Store B - Women’s Active Leggings” at this stage because its sales volume is too low for a reliable store-specific size set calculation.
 - An ISSData object is created for “Store C - Women’s Active Leggings,” with its size-specific sales data and a store-level flag.
4. **Roll-up (if needed):** If the individual store data for Store B was insufficient, the system might later roll up its sales into a higher level (e.g., Region 1 - Women’s Active Leggings) and check if *that* combined regional total meets a threshold for analysis at the regional level.

Final Outcome

Two distinct ISSData objects are created: one for “Store A - Women’s Active Leggings” and another for “Store C - Women’s Active Leggings,” both flagged for store-specific Ideal Size Set calculation. Store B’s data for this Attribute Group is excluded from individual store-level analysis due to insufficient sales volume. This ensures the next stage of ISS calculation focuses only on combinations with meaningful sales data.

Key Takeaway: By intelligently filtering and flagging sales data, this module ensures that valuable analytical resources are directed towards meaningful store-product combinations, leading to more accurate and actionable Ideal Size Set recommendations.

7. Results

This step produces:

Main Outputs

- **Flagged Store-Attribute Group Data for ISS:** A comprehensive map of Ideal Size Set (ISSData) objects, each representing a unique combination of store and Attribute Group, pre-populated with total sales and preliminary size-specific sales quantities, and tagged with a specific flag indicating its analysis level (e.g., store, channel, region). This is the primary input for the next module.
- **Preliminary Size Data for Analysis:** Within each ISSData object, a list of SizeData objects showing the Units Sold for each available size within that specific store-Attribute Group combination. This provides a snapshot of demand for individual sizes before full ISS optimization.

Business Reports

- **Sales Data Coverage Report:** A report for merchandising and inventory teams showing which store-Attribute Group combinations have sufficient “Consistent Sales” data for detailed ISS calculation, and which might need to be rolled up to a higher level or reviewed manually. Used by planners to understand data completeness for analysis.
- **Pre-analysis Performance Summary:** A high-level overview of aggregated sales data for sizes, grouped by flag types (e.g., all store-level data, all region-level data). Used by category managers to get an early sense of demand patterns across different levels of granularity.

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process data up to the last available record, potentially leading to incomplete or outdated flagging.	Verify data source connections; escalate to IT/data engineering team for missing feeds; consider re-running the module once data is available.
Very few combinations meet the Minimum Sales Quantity Threshold	The filtering criteria are too strict, or product sales are universally low, leading to limited or no flagged combinations for detailed ISS analysis.	Review the Minimum Sales Quantity Threshold setting; investigate if product sales are genuinely low or if a broader Sales Data Grouping Level is more appropriate.

Situation	What Happens	What You Should Do
Inaccurate Product Attribute Group (AG) definitions or Category Size Sequences	The system might misclassify product sizes or group unrelated products together, leading to misleading flagged combinations and subsequent ISS recommendations.	Implement robust data validation for product master data; regularly audit and update AG definitions and category size sequences.
Inconsistent sales patterns across different stores within the same region	The current Sales Data Grouping Level might be too broad, masking important store-specific demand variations by rolling them up.	Consider a more granular Sales Data Grouping Level (e.g., individual store instead of region) for highly variable product categories or store types.

9. Next Steps

Before This Step

- **ISSPrepareSalesModule (Refined Sales Data):** This module relies entirely on the aggregated and refined “Consistent Sales” data from the previous step, which has already filtered out sales impacted by stockouts.
- **Attribute Group Definition:** Product Attribute Groups (AG) must be defined and applied to all products, providing the necessary classifications for analysis.
- **Category Size Sequence Mapping:** Standard size sequences for different product categories must be defined and available, ensuring a consistent understanding of size ranges.

After This Step

- **Ideal Size Set Calculation:** The flagged and prepared ISSData objects from this module serve as the primary input for the next stage, where the Ideal Size Set (ISS) for each product category and style will be mathematically determined.
- **Inventory Optimization and Allocation:** The calculated Ideal Size Sets will then inform inventory planners on how to optimize stock levels for each size, leading to better allocation decisions and improved in-store availability.

10. Common Questions

Q: Why are sales data grouped and flagged at different levels (e.g., store vs. region)?

A: Sales patterns can vary significantly. By grouping and flagging at different levels, we ensure we can generate Ideal Size Sets that are either very specific to a single store (if sales volume allows) or more generalized for a channel or region (if individual store data is too sparse), leading to the most accurate recommendations for inventory planning.

Q: What do the “flags” mean? A: Each flag indicates the specific level of detail at which the sales data has been grouped for analysis. For example, a flag might specify that the data is for a single store, a channel within a region, or an entire region. This tells the next module how broadly the Ideal Size Set should apply.

Q: What happens if sales for a product are below the “Minimum Sales Quantity Threshold”? A: If a store-Attribute Group combination doesn’t meet this threshold, it won’t be flagged for a specific, granular Ideal Size Set calculation. Instead, its sales might be rolled up into a broader category (e.g., regional sales) to contribute to a higher-level ISS, or it might require manual review if it’s a very niche product.

Q: How does this module ensure that our Ideal Size Sets are relevant to specific stores? A: By systematically checking and flagging sales at the individual store level whenever sufficient “Consistent Sales” data is available, this module ensures that the subsequent Ideal Size Set calculation can produce recommendations highly tailored to the unique demand patterns of that specific store.

Q: How does this step prepare for the final Ideal Size Set calculation? A: This module takes the raw, cleaned sales data and intelligently organizes it into distinct, actionable packages (ISSData objects). By pre-calculating preliminary metrics and assigning flags based on sales volume and grouping levels, it provides the structured, relevant input that the Ideal Size Set calculation module needs to efficiently determine the best size combinations.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To tag and organize reliable sales data into actionable segments for Ideal Size Set (ISS) calculation, based on sales volume and desired analysis granularity.
- **Main Input:** Aggregated Consistent Sales Data (from ISSPrepareSalesModule) and relevant product/store master data.
- **Main Output:** Flagged Store-Attribute Group Data for ISS (ISSData objects), ready for detailed size set computation.
- **Warning Signs:** Too few flagged combinations, or flagged data not aligning with expected sales patterns for certain stores/products.
- **Key Settings:** Sales Data Grouping Level, Minimum Sales Quantity Threshold.

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 3: ISSTaggingModule

Last Updated: 2025-05-23

1. Business Objective

This step analyzes sales data for each size, determines the sales contribution of each size, identifies a critical “pivotal” sales percentage, and then categorizes sizes as Pivotal (P), Non-Pivotal (NP), or Exit (E) based on predefined business rules for an Ideal Size Set.

Why This Matters: This process is crucial because it ensures that our inventory planning is precise, allowing us to focus our stock investment on the most impactful sizes (Pivotal), maintain a healthy assortment with others (Non-Pivotal), and minimize losses from slow-moving sizes (Exit). This leads to better sales, fewer stockouts of popular items, and more efficient use of our inventory budget, directly boosting profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Flagged Sales Data for Ideal Size Set (ISS) Calculation:** This is the structured sales data from the previous step, grouped by store and Attribute Group, and already confirmed to have sufficient and consistent sales. *We need this to know the reliable sales quantities for each individual size.*
- **Size-Specific Sales Quantities:** Detailed sales counts for each size within an Attribute Group (e.g., how many Small, Medium, Large of a specific t-shirt style were sold). *This is crucial for calculating the contribution of each individual size to total sales for a product group.*

Product Data

- **Attribute Group Definitions:** Information about how products are grouped (e.g., “Women’s Active Leggings”). *This helps us apply consistent size set strategies to similar product categories.*
- **Product Size Details:** The full range of sizes typically available for a product within an Attribute Group. *This ensures we consider all relevant sizes when determining their roles.*

Operational Data

- **Store-Specific Context:** Details about each store that help in applying store-specific size set strategies. *Different stores might have unique local demand patterns, requiring tailored size sets.*
- **Size Set Strategy Guidelines:** Pre-defined business rules or benchmarks that specify what percentage of sales defines a “pivotal” size, and the criteria for tagging sizes as Pivotal, Non-Pivotal, or Exit. *These guidelines are essential for consistent and strategic sizing decisions across our product range.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Target Pivotal Sales Contribution	60%	The cumulative sales percentage from the top-selling sizes that should be categorized as “Pivotal.”	Adjust to be more aggressive (higher percentage) for categories where a few sizes dominate sales, or less aggressive (lower percentage) to ensure broader size coverage for specialty items.
Pivotal Size Count Threshold	3 sizes	The minimum number of sizes that must be tagged as “Pivotal” regardless of sales contribution, to ensure a basic size assortment.	Increase if you always want a minimum number of popular sizes to be stocked. Decrease for niche products or very specific items where fewer sizes are needed.
Exit Size Sales Threshold	5%	If a single size contributes less than this percentage to total Attribute Group sales, it’s considered for an “Exit” tag.	Increase to be more aggressive about removing slow-moving sizes from inventory. Decrease to retain more long-tail sizes for customer choice.
Non-Pivotal Buffer	10%	A percentage range around the Pivotal threshold that helps categorize sizes as Non-Pivotal, ensuring a	Adjust to fine-tune the balance between Pivotal and Non-Pivotal sizes, allowing for more flexibility in the middle demand range.

Setting Name	Default Value	Description	When to Change
		balanced assortment.	

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Flagged Sales Data for ISS Calculation] --> B[Calculate Size Sales Contributions]

B --> C[Determine Pivotal Sales Percentage]

C --> D[Tag Sizes: Pivotal, Non-Pivotal, Exit]

D --> E[Tagged Size Data for Ideal Size Set]

1. Start with → Flagged Sales Data for Ideal Size Set Calculation (from the previous ISSFlaggingModule), which contains sales quantities per size for each store-Attribute Group combination.
2. Then → Calculate the sales contribution of each individual size (e.g., Small, Medium) to the total sales within its Attribute Group.
3. Next → Determine the cumulative sales percentage that defines the “pivotal” sizes based on predefined thresholds and business strategy guidelines.
4. Finally → Tag each size within the Attribute Group as either Pivotal (P), Non-Pivotal (NP), or Exit (E), and adjust their sales contribution based on these tags. This output is the Tagged Size Data for Ideal Size Set.

5. Step-by-Step Actions

1. **Retrieve** Size Set Strategy Guidelines.
 - What we do: *We load specific rules and thresholds for defining size sets, which are unique to each store and product category (e.g., Women's Tops in Store A).*
 - Why we do it: *These guidelines ensure that our size tagging decisions align with the strategic goals for different product types and store formats, supporting local demand.*
2. **Calculate** Individual Size Sales Contribution.
 - What we do: *For each store-Attribute Group combination, we analyze the total sales for that group and determine what percentage of those sales each individual size (e.g., Small, Medium, Large) contributes.*
 - Why we do it: *This gives us a clear picture of how important each size is in terms of generating sales and revenue.*
3. **Determine** the Pivotal Sales Percentage.
 - What we do: *Using the individual size contributions and the “Target Pivotal Sales Contribution” from the strategy guidelines, we identify the cumulative sales percentage that represents the core, fastest-selling sizes.*
 - Why we do it: *This helps us understand where the majority of our sales are concentrated across the size range, pinpointing the most critical sizes for inventory investment.*

4. **Assign** Size Roles (Pivotal, Non-Pivotal, Exit).

- What we do: *Based on the calculated sales contributions, the pivotal sales percentage, and other criteria from the Size Set Strategy Guidelines, we tag each size as either Pivotal (P), Non-Pivotal (NP), or Exit (E).*
- Why we do it: *This clear categorization provides a strong framework for inventory decisions: P sizes get priority stocking, NP sizes provide breadth, and E sizes are candidates for discontinuation or minimal stock, reducing holding costs.*

5. **Adjust** Size Sales Contribution.

- What we do: *After tagging, we re-evaluate and potentially adjust the sales contribution percentages for each size, taking into account its new role (P, NP, E).*
- Why we do it: *This refinement ensures that subsequent Ideal Size Set calculations are based on the most accurate and strategically aligned sales contributions, leading to more precise stock levels.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- We are analyzing sales for “Men’s Classic Jeans” (an Attribute Group) in Store X.
- Total “Consistent Sales” for this Attribute Group in Store X over the last 6 months: 500 units.
- Individual size sales breakdown:
 - Size 30: 50 units (10% of total)
 - Size 32: 150 units (30% of total)
 - Size 34: 180 units (36% of total)
 - Size 36: 90 units (18% of total)
 - Size 38: 30 units (6% of total)
- **Size Set Strategy Guidelines** for Men’s Jeans in Store X:
 - **Target Pivotal Sales Contribution:** 65%
 - **Exit Size Sales Threshold:** 7%

What Happens

1. **Calculate Individual Size Sales Contribution:** The system already has these percentages calculated from the input data.
2. **Determine Pivotal Sales Percentage:**
 - The system sorts sizes by sales contribution (highest to lowest): Size 34 (36%), Size 32 (30%), Size 36 (18%), Size 30 (10%), Size 38 (6%).
 - It then cumulates the percentages:
 - Size 34: 36%
 - Size 34 + Size 32: 36% + 30% = 66%
 - Since 66% (cumulative for Size 34 and Size 32) exceeds the **Target Pivotal Sales Contribution** of 65%, these two sizes are identified as the core contributors.

3. Assign Size Roles:

- **Pivotal (P):** Size 34 (36% contribution) and Size 32 (30% contribution) are tagged as Pivotal because their combined sales exceed the 65% threshold. These are the sizes we must always have in stock.
- **Exit (E):** Size 38 (6% contribution) is tagged as Exit because its individual sales contribution (6%) is below the **Exit Size Sales Threshold** of 7%. This size might be phased out or stocked minimally.
- **Non-Pivotal (NP):** The remaining sizes, Size 36 (18% contribution) and Size 30 (10% contribution), are tagged as Non-Pivotal. These sizes are important for breadth but are not as critical as the Pivotal ones.

Final Outcome

For “Men’s Classic Jeans” in Store X, the system tags: - Size 32 and Size 34 as **Pivotal (P)**. - Size 30 and Size 36 as **Non-Pivotal (NP)**. - Size 38 as **Exit (E)**.

This clear categorization guides inventory decisions, ensuring that Store X prioritizes stocking Sizes 32 and 34, maintains Sizes 30 and 36 for assortment, and considers reducing or eliminating stock for Size 38, leading to more focused inventory and improved sales.

Key Takeaway: By clearly categorizing sizes based on their sales contribution and strategic importance, this module provides actionable insights for optimizing inventory investment and maximizing sales efficiency for each product.

7. Results

This step produces:

Main Outputs

- **Tagged Size Data for Ideal Size Set:** Each Ideal Size Set (ISSData) object (from the previous module) is updated with each size (e.g., Small, Medium, Large) now explicitly tagged as Pivotal (P), Non-Pivotal (NP), or Exit (E). *This clearly identifies the strategic importance of each size within its Attribute Group, guiding future stocking levels.*
- **Adjusted Size Sales Contributions:** The sales contribution percentage for each size is re-calculated and finalized based on its assigned tag. *This provides a refined view of each size’s demand, critical for precise inventory planning and allocation.*
- **Pivotal Quantity Percentage:** The cumulative sales percentage for the sizes identified as Pivotal is explicitly captured. *This metric helps track adherence to the desired sales concentration and ensures we meet our strategic goals for core sizes.*

Business Reports

- **Size Role Breakdown Report:** A report for merchandising and inventory teams showing the count and percentage of Pivotal, Non-Pivotal, and Exit sizes for each Attribute Group. *Used by category managers to understand the strategic balance of their size assortments and identify areas for optimization.*

- **Size Performance and Tagging Audit:** A detailed report listing each size, its sales contribution, and its assigned tag (P, NP, E) by store and Attribute Group. *Used by planners to review the tagging decisions, identify potential adjustments or exceptions, and ensure data accuracy.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process data up to the last available record, potentially leading to outdated or incomplete size tagging.	Verify data source connections; escalate to IT/data engineering team for missing feeds; consider re-running the module once data is available.
No Size Set Strategy Guidelines found for a store-category combination	The module cannot apply specific tagging rules and may skip processing that particular combination, or use default rules if available, which might not be optimal.	Ensure all relevant store and category/subcategory combinations have defined Size Set Strategy Guidelines; audit master data for completeness and consistency.
All sizes tagged as “Exit” or “Non-Pivotal” for a popular product	The Target Pivotal Sales Contribution might be too high, or the product’s sales are too evenly distributed across sizes, preventing any size from reaching the pivotal threshold.	Review the Target Pivotal Sales Contribution and Pivotal Size Count Threshold settings; consider if the product truly lacks dominant sizes or if the sales data indicates an issue.
Significant sales for an “Exit” tagged size	This indicates a potential missed opportunity or an anomaly in the data or strategy. The tag might be too aggressive, or a new trend is emerging.	Review the sales history for that specific size. Adjust the Exit Size Sales Threshold or re-evaluate the product’s strategy if needed, potentially re-classifying the item.

9. Next Steps

Before This Step

- **ISSFlaggingModule (Flagged Sales Data for ISS Calculation):** This module relies entirely on the prepared and flagged sales data, grouped by store and Attribute Group, from the previous step. This ensures consistent and reliable sales data for analysis.
- **Size Set Strategy Guidelines Definition:** Business rules and thresholds for size tagging (e.g., target pivotal sales percentages for different product types) must be pre-defined and available for the system to apply.

After This Step

- **Ideal Size Set Optimization:** The tagged size data, with its Pivotal, Non-Pivotal, and Exit categorizations, serves as the primary input for the next stage, where the optimal quantity (OD) of each size will be mathematically determined to form the final Ideal Size Set.
- **Inventory Forecasting and Allocation:** The calculated Ideal Size Sets will directly inform how many units of each size are ordered and allocated to stores, optimizing stock levels, minimizing waste, and improving product availability for customers.

10. Common Questions

Q: Why are sizes categorized as Pivotal, Non-Pivotal, or Exit? A: This categorization is critical for smart inventory management. Pivotal sizes are where we focus our stock investment for maximum sales impact. Non-Pivotal sizes ensure we offer enough variety, and Exit sizes help us minimize losses from slow-moving items, ensuring our inventory is lean and efficient.

Q: How does this module help reduce stockouts of popular sizes? A: By explicitly tagging the fastest-selling and most impactful sizes as “Pivotal,” it signals to inventory planners that these sizes must always be prioritized for stock. This ensures we rarely run out of what customers want most, directly reducing lost sales and improving customer satisfaction.

Q: What happens if a new size emerges as popular but is currently tagged as “Non-Pivotal” or “Exit”? A: The system regularly re-evaluates sales data. If a size’s sales contribution increases significantly, this module will re-tag it appropriately in subsequent runs, allowing the Ideal Size Set to adapt dynamically to changing customer demand and new trends.

Q: Can these size tags be overridden or manually adjusted by a merchandiser? A: While the system provides data-driven recommendations, businesses can often implement manual overrides or adjustments based on strategic decisions, promotional plans, or qualitative insights that the system might not capture. The tags serve as a strong, data-backed starting point for discussion.

Q: How does this module contribute to overall profitability? A: By focusing inventory on high-demand sizes and reducing stock of low-demand sizes, the module minimizes wasted capital on slow-moving inventory, reduces the need for costly markdowns, improves sell-through rates, and maximizes sales by having the right product at the right time, all of which directly boost profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To categorize product sizes as Pivotal, Non-Pivotal, or Exit based on sales contribution and business strategy, guiding precise inventory decisions.
- **Main Input:** Flagged Sales Data for ISS Calculation (from `ISSFlaggingModule`) and predefined Size Set Strategy Guidelines.

- **Main Output:** Tagged Size Data for Ideal Size Set, with each size categorized (P, NP, E) and adjusted sales contributions.
 - **Warning Signs:** Unexpected tagging results (e.g., popular sizes tagged as Exit, or too many Pivotal sizes), or missing strategic guidelines for specific categories.
 - **Key Settings:** Target Pivotal Sales Contribution, Exit Size Sales Threshold.
-

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: ISSSizeContinuityModule

Last Updated: 2025-05-23

1. Business Objective

The ISSSizeContinuityModule ensures that the sizes identified as most important (“Pivotal”) form a continuous, logical sequence within a product’s size range, by analyzing existing tagged sizes and adjusting the tagging of any intermediate sizes to maintain a coherent Ideal Size Set.

Why This Matters: This process is crucial because it eliminates frustrating gaps in the core size offerings on shelves, which directly improves the customer shopping experience by ensuring they can consistently find popular sizes. By enforcing continuity, it optimizes inventory allocation to critical size ranges, reduces unexpected stockouts of frequently sought-after sizes, and ultimately helps maintain consistent sales performance.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Tagged Size Data for Ideal Size Set:** This includes the sales performance for each size, already categorized as Pivotal (P), Non-Pivotal (NP), or Exit (E) from the previous tagging step. *This provides the current strategic categorization of each size (Pivotal, Non-Pivotal, Exit) for which continuity needs to be maintained.*

Product Data

- **Category Size Sequences:** These are predefined, ordered lists of all standard sizes for a particular product category (e.g., 28, 30, 32, 34 for jeans, or XS, S, M, L, XL for tops). *These define the complete, ordered range of sizes expected for a product category, ensuring the module understands the full spectrum of sizes.*
- **Product Attribute Group Definitions:** Information on how products are grouped (e.g., “Men’s Casual Shirts,” “Women’s Activewear”). *This helps in linking specific products to their broader categories for applying the correct size sequences.*

Operational Data

- **Business Rules for Size Exceptions:** These are specific exceptions defined by merchandising teams where certain sizes should intentionally remain non-pivotal, even if they fall within a continuous pivotal range, to avoid overstocking niche items. *This allows for strategic overrides to the continuity logic where specific business needs dictate a discontinuous size set.*
- **Store Type Size Policies:** These guidelines specify how size sets should be managed differently for various store types (e.g., flagship vs. outlet). *This helps ensure that any size continuity adjustments align with broader retail strategies for different store formats.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Size Continuity Enablement	False	Controls whether the size continuity logic is active. If set to False, this module will not run.	Set to True for product categories where a continuous run of pivotal sizes is essential for customer satisfaction (e.g., apparel, footwear). Keep as False if your business strategy sometimes requires deliberate gaps or discontinuous size sets.
Excluded Attribute Groups	Empty List	A list of specific Attribute Groups (e.g., “Men’s Specialty Footwear”) for which the size continuity logic should NOT be applied, even if Size Continuity Enablement is True.	Add Attribute Groups if their size sets are naturally discontinuous or if maintaining continuity would lead to excessive, unwanted inventory for niche or highly specialized items.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Tagged Size Data for Ideal Size Set & Category Size Sequences] --> B[Identify Standard Size Sequence & Pivotal Range]
B --> C[Enforce Pivotal Size Continuity]
C --> D[Size Data with Continuous Pivotal Tags]
```

1. Start with → Tagged Size Data for Ideal Size Set (from the ISSTaggingModule) and Category Size Sequences.
2. Then → For each product's Attribute Group, the module identifies its standard, ordered size sequence and locates the lowest and highest sizes currently tagged as "Pivotal" within that sequence.
3. Next → All intermediate sizes that fall numerically or sequentially between these lowest and highest identified Pivotal sizes are re-tagged as "Pivotal", ensuring a smooth, continuous range.
4. Finally → The product's Ideal Size Set data is updated with these new, continuous pivotal tags, making it ready for final calculations and inventory planning.

5. Step-by-Step Actions

1. **Retrieve** Tagged Size Data and Category Size Sequences.
 - What we do: *We load the currently tagged sizes (Pivotal, Non-Pivotal, Exit) for each product group from the previous step, along with the master list of all standard, ordered sizes for each product category (e.g., S, M, L, XL).*
 - Why we do it: *This sets the foundation by providing the current strategic categorization of sizes and the complete spectrum of sizes for a given product type, which is necessary for identifying and enforcing continuity.*
2. **Match** Product Sizes to Standard Sequences.
 - What we do: *For each specific product and its Attribute Group, we identify which pre-defined, ordered size sequence (e.g., 28, 30, 32, 34 for jeans) it belongs to.*
 - Why we do it: *This ensures that the module understands the natural progression and expected range of sizes for that product, which is vital for correctly identifying "intermediate" sizes.*
3. **Identify** the Pivotal Size Range.
 - What we do: *Within the product's matched standard size sequence, we locate the lowest and highest sizes that were initially tagged as 'Pivotal' by the ISSTaggingModule.*
 - Why we do it: *This step establishes the boundaries of the core, most important size range, identifying the anchor points for where continuity needs to be maintained.*
4. **Enforce** Pivotal Size Continuity.
 - What we do: *All sizes that fall numerically or sequentially between the lowest and highest identified 'Pivotal' sizes are re-tagged as 'Pivotal', even if their individual sales contribution was lower or they were originally tagged as 'Non-Pivotal'.*
 - Why we do it: *This ensures that there are no "holes" in the offering of core sizes, preventing situations where a customer might find a popular size at one end of the spectrum but not a size in the middle that they need.*
5. **Update** Ideal Size Set Data.
 - What we do: *The modified size data, now with continuous pivotal tags, replaces the original size data within the Ideal Size Set (ISSData) object.*

- Why we do it: *This finalizes the size set for the product, reflecting the enforced continuity and making it ready for the next steps of Ideal Size Set optimization and inventory allocation.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- We are analyzing sales for “Women’s Denim Jeans” (an Attribute Group) in Store X.
- The standard Category Size Sequence for this Attribute Group is: 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36.
- Based on the ISSTaggingModule, the initial size tags for this product are:
 - Size 26: Non-Pivotal (NP)
 - Size 27: Non-Pivotal (NP)
 - Size 28: **Pivotal (P)**
 - Size 29: Non-Pivotal (NP)
 - Size 30: **Pivotal (P)**
 - Size 31: Non-Pivotal (NP)
 - Size 32: **Pivotal (P)**
 - Size 33: Non-Pivotal (NP)
 - Size 34: Non-Pivotal (NP)
 - Size 35: Exit (E)
 - Size 36: Exit (E)
- The **Size Continuity Enablement** setting is True.

What Happens

1. **Identify Standard Sequence:** The module identifies the full range of sizes for Women’s Denim Jeans.
2. **Locate Pivotal Range:** The module scans the initially tagged sizes and finds the lowest Pivotal size is 28, and the highest Pivotal size is 32.
3. **Enforce Continuity:** The module now looks at all sizes between 28 and 32 in the standard sequence: 29, 30, and 31.
 - Size 29 is currently Non-Pivotal (NP). Since it falls between two Pivotal sizes (28 and 30, and 30 and 32), it is re-tagged as **Pivotal (P)**.
 - Size 30 is already Pivotal (P), so no change.
 - Size 31 is currently Non-Pivotal (NP). Since it falls between two Pivotal sizes (30 and 32), it is re-tagged as **Pivotal (P)**.
4. **Update Ideal Size Set Data:** The Ideal Size Set for “Women’s Denim Jeans” in Store X is updated with the new tags.

Final Outcome

For “Women’s Denim Jeans” in Store X, the system now has the following size tags:
 - Size 26: Non-Pivotal (NP) - Size 27: Non-Pivotal (NP) - Size 28: **Pivotal (P)** -
 Size 29: **Pivotal (P)** (Changed from NP) - Size 30: **Pivotal (P)** - Size 31: **Pivotal (P)**
 (Changed from NP) - Size 32: **Pivotal (P)** - Size 33: Non-Pivotal (NP) - Size 34:
 Non-Pivotal (NP) - Size 35: Exit (E) - Size 36: Exit (E)

Key Takeaway: By rigorously enforcing continuity, this module ensures that if popular sizes (Pivotal) exist at the edges of a range, the intermediate sizes are also automatically prioritized, preventing frustrating gaps for customers and leading to more effective inventory management.

7. Results

This step produces:

Main Outputs

- **Updated Ideal Size Set Data:** Each Ideal Size Set (ISSData) object (from previous modules) is updated. Any size that falls within a previously identified Pivotal range, regardless of its individual sales contribution, is now explicitly tagged as Pivotal (P). *This provides a refined, logically continuous Ideal Size Set ready for final inventory optimization.*

Business Reports

- **Pivotal Range Compliance Report:** A report for merchandising and inventory teams showing which product categories and Attribute Groups have had their pivotal sizes adjusted to ensure continuity. It highlights sizes that were changed from Non-Pivotal to Pivotal by this module. *Used by planners to verify that the core size assortment reflects a coherent offering and to identify any unexpected changes.*
- **Size Set Coherence Audit:** A detailed view of the final size set for each product category, visually representing the continuous Pivotal range. *Used by category managers and store operations to ensure that the planned size assortments will be visually logical and customer-friendly on the sales floor.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process data up to the last available record, potentially leading to outdated or incomplete size continuity adjustments.	Verify data source connections; escalate to IT/data engineering team for missing feeds; consider re-running the module once data is available.
No Pivotal sizes identified by the ISSTaggingModule	If no sizes are initially tagged as Pivotal for a given product or Attribute Group, this module will not find a range to make continuous, and therefore will make no changes.	Review the ISSTaggingModule settings (e.g., Target Pivotal Sales Contribution , Pivotal Size Count Threshold) or analyze the product's sales data to understand why no sizes were deemed Pivotal.

Situation	What Happens	What You Should Do
A product has only one Pivotal size identified	If only one size is tagged Pivotal, there is no “range” between two Pivotal sizes to enforce continuity, so the module will make no changes for that specific product.	This is expected behavior; a single pivotal size doesn’t create a range for continuity. Ensure the preceding ISSTaggingModule settings are appropriate for identifying multiple pivotal sizes if a range is desired.
Category has no defined Category Size Sequence	If a product’s category lacks a predefined standard size sequence, the module cannot accurately identify intermediate sizes or enforce logical continuity.	Ensure all product categories have complete and accurate Category Size Sequences defined in the master data.

9. Next Steps

Before This Step

- **ISSTaggingModule (Tagged Size Data for Ideal Size Set):** This module relies entirely on the prepared and tagged sales data, where sizes have already been categorized as Pivotal, Non-Pivotal, or Exit.
- **Category Size Sequence Definition:** Standard, ordered size sequences for different product categories must be accurately defined and available, as these are crucial for understanding the numerical or sequential relationship between sizes.

After This Step

- **Ideal Size Set Optimization:** The updated Ideal Size Set data, now with continuous pivotal tags, serves as the primary input for the next stage, where the optimal quantity (OD) of each size will be mathematically determined to form the final Ideal Size Set.
- **Inventory Forecasting and Allocation:** The finalized Ideal Size Sets will directly inform inventory planners on how many units of each size are ordered and allocated to stores, optimizing stock levels, minimizing waste, and improving product availability for customers, especially for critical core sizes.

10. Common Questions

Q: Why is size continuity important for my inventory? A: Imagine a customer looking for a popular item like a dress shirt. If you have Small and Large, but no Medium (even though Medium is popular), they might not buy at all. Ensuring continuity for pivotal sizes means you always offer the full range of popular sizes, preventing lost sales and improving customer satisfaction.

Q: What if a size was intentionally marked as Non-Pivotal by the previous step, but gets changed to Pivotal here? A: This module prioritizes the customer experience by ensuring that if you carry the “ends” of a popular size range, you also carry the “middle” sizes. If a size was intended to be Non-Pivotal, you might need to adjust the ISSTaggingModule settings or use the Excluded Attribute Groups setting in this module to prevent it from being re-tagged.

Q: How does this module help prevent stockouts? A: By ensuring that intermediate sizes within a popular range are also flagged as Pivotal, it signals to inventory systems that these sizes are equally critical. This leads to better prioritization in ordering and allocation, significantly reducing the chances of selling out of key sizes in the middle of a continuous range.

Q: Can I disable this feature if I don’t want strict size continuity? A: Yes, you can disable this feature using the **Size Continuity Enablement** setting. If set to False, the module will not run, and the size tags will remain exactly as determined by the ISSTaggingModule.

Q: Does this module increase my inventory levels? A: Potentially, yes. If sizes previously tagged as Non-Pivotal are re-tagged as Pivotal, it implies a higher importance for stocking those sizes. However, this is usually a strategic trade-off, as the slight increase in inventory for these core sizes is often offset by improved sales and customer satisfaction from a more complete offering.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To enforce logical continuity within a product’s “Pivotal” (most important) size range, ensuring no gaps in core offerings.
- **Main Input:** Tagged Size Data for Ideal Size Set (from ISSTaggingModule) and Category Size Sequences.
- **Main Output:** Updated Ideal Size Set Data with continuous Pivotal tags.
- **Warning Signs:** Unexpectedly high number of sizes being re-tagged as Pivotal, or continuity being applied to product categories where it’s not desired.
- **Key Settings:** Size Continuity Enablement, Excluded Attribute Groups.

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: ISSOutputModule

Last Updated: 2025-05-23

1. Business Objective

The ISSOutputModule is responsible for consolidating all calculated Ideal Size Set (ISS) metrics and writing them to the database. It generates final output records, incorporating raw and consistent sales quantities, size group tags (Pivotal, Non-Pivotal, Exit), and contribution percentages, ensuring data readiness for reporting and further analysis.

Why This Matters: This process is the culmination of all Ideal Size Set calculations, transforming complex analysis into clear, actionable data. By providing comprehensive output, it empowers merchandising and inventory teams to make informed decisions on stocking levels, allocate inventory efficiently, and ultimately optimize product availability to meet customer demand, leading to improved sales and reduced stockouts.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Raw Sales Transactions:** Complete records of all sales, regardless of inventory availability. *We need this to provide a baseline of total demand, even if stockouts occurred.*
- **Consistent Sales Data:** Sales data specifically from periods when key product sizes were consistently available (from ISSPrepareSalesModule). *This provides the most reliable demand signal for ISS calculations.*
- **Aggregated Size Sales Data:** The sales quantities for each size within an Attribute Group, as processed and adjusted by previous ISS modules. *This includes the refined sales figures used to calculate size contributions and tags.*

Product Data

- **Product Attribute Group Details:** Information about how products are classified into Attribute Groups (e.g., “Women’s Active Leggings”), including attributes like category, brand, and gender. *This contextualizes the size data for reporting and analysis.*
- **Category Size Sequences:** Pre-defined, ordered lists of standard sizes for various product categories (e.g., S, M, L, XL for tops). *This ensures the correct*

ordering of sizes in the output and provides human-readable size sequence information.

Operational Data

- **Store Details:** Comprehensive information about each retail location, such as store name, code, channel, and any reference to new stores. *This allows us to link ISS recommendations to specific stores and apply insights to new locations.*
- **Ideal Size Set Analysis Objects:** The consolidated ISSData objects from previous modules, containing all calculated size tags (Pivotal, Non-Pivotal, Exit), contribution percentages, and analysis flags. *This is the core processed data that needs to be outputted.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Current Reporting Date	Today's Date	The date for which the ISS output is generated. This is generally used for internal system date management.	Usually remains the system's current date, but can be adjusted for historical reporting or simulations.
New Store Reference Mapping	N/A	A mapping that links new stores to existing "reference" stores for initial inventory planning.	Updated whenever new stores are opened or store mappings change.

Note: This module primarily consumes outputs from previous modules and system-wide configurations rather than having many direct runtime settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Data & Consistent Sales Data] --> B{Consolidate ISS Metrics}

B --> C{Generate Detailed Size Set Output}

C --> D[Final Ideal Size Set Recommendations (Database & Reports)]

1. Start with → Raw Sales Data, Consistent Sales Data, and the Ideal Size Set Analysis Objects (which contain tagged sizes and calculated contributions from previous modules).

2. Then → Consolidate all the processed ISS metrics (such as size tags, contributions, and flags) with the original raw and consistent sales data for each store-Attribute Group-size combination.
3. Next → Generate two primary output formats: a detailed internal table and a comprehensive exportable report, ensuring data consistency and completeness.
4. Finally → Write the generated Ideal Size Set Recommendations to the database, making them available for downstream systems and business reporting.

5. Step-by-Step Actions

1. **Map** Store and Product Classifications.
 - What we do: *We retrieve and organize all necessary master data, such as store details, Attribute Group definitions, and category size sequences. We also prepare a mapping for new stores that might inherit recommendations from existing ones.*
 - Why we do it: *This ensures that all output data is correctly contextualized with human-readable names and relevant business attributes, and that recommendations can be extended to new stores.*
2. **Identify** All Relevant Store-Product Combinations.
 - What we do: *We compile a comprehensive list of all store-Attribute Group combinations that should appear in the final output. This includes combinations that had sales data for ISS calculation, as well as those that might need coverage based on current inventory or open orders.*
 - Why we do it: *This ensures that our Ideal Size Set recommendations cover all product categories within all relevant stores, even if some didn't meet sales thresholds for detailed ISS calculation in previous steps.*
3. **Consolidate** Sales and ISS Data.
 - What we do: *For each store-Attribute Group-size combination, we pull together the raw sales quantity, the consistent sales quantity, and all the calculated Ideal Size Set data (like size tags, sales contributions, and pivotal percentage).*
 - Why we do it: *This creates a complete picture for each size, showing its overall sales performance alongside its strategic categorization within the Ideal Size Set.*
4. **Default** Untagged Sizes to “Exit”.
 - What we do: *If a size had recorded raw sales but was not explicitly tagged as Pivotal or Non-Pivotal by previous modules, it is automatically assigned an “Exit” tag.*
 - Why we do it: *This prevents ambiguity and ensures that any size not actively contributing to sales, or not identified as strategically important, is flagged for potential reduction or removal from stock, aiding inventory efficiency.*
5. **Generate** Final Output Records.
 - What we do: *We create two primary output data structures: a detailed table for internal consumption and a comprehensive exportable report with all relevant business attributes for external reporting and analysis.*
 - Why we do it: *These structures present the Ideal Size Set recommendations in formats optimized for different downstream uses, from precise inventory system updates to high-level business intelligence reports.*
6. **Extend** Recommendations to New Stores.
 - What we do: *For any new stores linked to existing “reference” stores, the Ideal Size Set recommendations calculated for the reference store are*

copied over to the new store, typically with zero sales data for the new store initially.

- Why we do it: *This provides a default, data-driven size set for newly opened locations, enabling them to stock efficiently from day one without needing to accumulate their own sales history.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- The ISSTaggingModule and ISSSizeContinuityModule have processed “Women’s Casual Dresses” (Attribute Group 101) for Store A (ID 123).
- For Size Medium:
 - It was tagged as **Pivotal (P)**.
 - Its Adjusted Sales Quantity for ISS is 100 units.
 - Its Size Contribution to Category Sales is 35%.
 - Its Pivotal Sales Percentage for the category is 68%.
- From raw sales data, Size Medium had 110 units sold (Raw Sales Quantity).
- From ISSPrepareSalesModule, Size Medium had 105 units sold during consistent inventory periods (Consistent Sales Quantity).
- Store B (ID 456) is a new store, mapped to inherit recommendations from Store A.
- Size “XXL” for “Women’s Casual Dresses” in Store A had 5 units in Raw Sales, but was never explicitly tagged as Pivotal or Non-Pivotal by previous modules due to low sales volume.

What Happens

1. **Consolidate Data:** The module collects all this information for Size Medium in Store A. It matches Store A’s ID (123) and Attribute Group ID (101) with the corresponding Ideal Size Set object.
2. **Generate Detailed Output:** A detailed output record is created for Size Medium (Store 123, AG 101):
 - store = 123, ag = 101, size = "Medium", sizeGroup = "P", qty = 100 (Adjusted Sales Quantity for ISS), rawQty = 110, unbrokenQty = 105, contri = 35%, qtyPiv = 68%.
3. **Generate Comprehensive Report:** A comprehensive report record is then derived, enriching the data with descriptive attributes:
 - store = "Store A", category = "Dresses", gender = "Women's", size = "Medium", sizeGroup = "Pivotal", qtyContri = 35%, rawQty = 110, rollUpQty = 100, unbrokenQty = 105.
4. **Default Untagged Size:** For Size XXL in Store A, since it had raw sales but was not explicitly tagged, the module automatically assigns it the **Exit (E)** tag.
5. **Extend to New Store:** Since Store B is mapped to Store A, detailed output and comprehensive report records are also created for Size Medium in Store B, mirroring Store A’s recommendations for size group and contribution, but with rawQty, rollUpQty, and unbrokenQty initialized to zero (as it’s a new store without sales history).
6. **Persist Data:** All these generated records are then written to the database.

Final Outcome

The database now contains complete, actionable Ideal Size Set recommendations for “Women’s Casual Dresses” in Store A (including the newly tagged “Exit” size XXL) and initial recommendations for new Store B. Merchandisers can immediately see which sizes are Pivotal, Non-Pivotal, or Exit for each product category in each store, backed by transparent sales data (raw, consistent, and adjusted), enabling precise ordering and allocation.

Key Takeaway: This module acts as the final reporting engine, compiling all calculated insights into comprehensive, easy-to-use formats that directly drive inventory and merchandising decisions across the entire retail network.

7. Results

This step produces:

Main Outputs

- **Detailed Size Set Output:** A granular internal table containing the store, Attribute Group, size, calculated size tag (Pivotal, Non-Pivotal, Exit), raw sales, consistent sales, adjusted sales for ISS, size contribution percentages, and the overall pivotal sales percentage. *This serves as the raw, detailed input for inventory systems to update stock levels.*
- **Comprehensive Size Set Report:** A denormalized, user-friendly table ready for business intelligence tools and reports. It includes all details from the detailed output, enriched with descriptive attributes like store name, category, brand, and gender. *This provides a complete view of the Ideal Size Set recommendations for business users, enabling them to analyze and act upon the data directly.*

Business Reports

- **Ideal Size Set Summary Report:** A high-level report for category managers and inventory planners, summarizing the calculated Ideal Size Set for each Attribute Group by store, showing the overall size distribution and core sizes. *Used to quickly grasp and validate the recommended size profiles.*
- **Size Contribution and Tagging Performance Report:** A detailed report that allows analysts to audit the contribution of each size, its assigned tag, and compare raw vs. consistent sales. *Used to review the accuracy of the ISS calculation and identify any discrepancies or areas for further investigation.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed from previous modules	The module will process with the last available data, potentially resulting in	Verify data source connections for all upstream ISS modules; escalate to IT/data engineering team for missing feeds; re-run the

Situation	What Happens	What You Should Do
Store-Attribute Group combinations without any calculated ISS data	outdated or incomplete ISS recommendations. If a combination had no sales or didn't meet minimum thresholds in previous steps, it won't have calculated ISS metrics. The output will either omit it or generate records with default/zero values for core metrics, depending on the need for comprehensive coverage.	entire ISS pipeline once data is available. Review the <code>ISSFlaggingModule</code> settings for minimum sales thresholds or <code>ISSPrepareSalesModule</code> settings for consistent sales. For truly new or slow-moving items, consider manual overrides or rule-based initial size sets.
Incorrect or incomplete master data (e.g., store mapping, AG definitions)	The output reports will contain inaccurate or missing descriptive attributes, making them difficult to interpret by business users.	Implement robust data validation for all master data sources; regularly audit and cleanse store and product attribute definitions.
High number of sizes defaulting to "Exit" tag	This indicates many sizes had raw sales but were not strategically classified by previous modules, or the previous modules' settings are too strict.	Review the <code>ISSTaggingModule</code> and <code>ISSSizeContinuityModule</code> settings (e.g., Target Pivotal Sales Contribution, Exit Size Sales Threshold) and analyze the sales patterns for these items to understand why they were untagged.

9. Next Steps

Before This Step

- **ISSPrepareSalesModule (Refined Sales Data):** Provides the raw and consistent sales quantities.
- **ISSFlaggingModule (Flagged Sales Data for ISS Calculation):** Provides the Ideal Size Set analysis objects, pre-populated with total sales and preliminary size-specific sales quantities, and tagged with analysis levels.
- **ISSTaggingModule (Tagged Size Data for Ideal Size Set):** Categorizes sizes as Pivotal, Non-Pivotal, or Exit and calculates their contributions.
- **ISSSizeContinuityModule (Size Data with Continuous Pivotal Tags):** Ensures logical continuity within the Pivotal size range.

After This Step

- **Inventory Optimization and Allocation Systems:** The generated detailed size set output serves as the primary input for inventory management systems to

adjust future purchase orders, warehouse transfers, and store-level allocations based on the calculated Ideal Size Sets.

- **Business Intelligence & Reporting Tools:** The comprehensive size set report is consumed by dashboards and reporting tools to provide merchandisers, planners, and store operations teams with actionable insights into ideal stock levels, size performance, and opportunities for assortment optimization.

10. Common Questions

Q: What is the main purpose of this module? A: This module is the final step in the Ideal Size Set calculation process. Its main purpose is to gather all the calculated insights about optimal size mixes for products and store them in a clear, accessible format in the database, making them ready for immediate use by inventory and reporting systems.

Q: How does this module help my team make better inventory decisions? A: By providing comprehensive data on which sizes are “Pivotal” (core and high-selling), “Non-Pivotal” (important for assortment), or “Exit” (low-performing), along with their sales contributions, it gives your team a data-backed blueprint for what to stock, how much to order, and where to allocate products, minimizing guesswork and improving efficiency.

Q: Why are there two different output tables? A: One table is a more technical, granular output primarily for direct system integration (like inventory management). The other is designed for human readability and broader business reporting, enriching the data with descriptive attributes so that it’s immediately understandable and actionable for merchandising and planning teams.

Q: What happens if a size was sold but doesn’t appear in the output? A: If a size had raw sales but was not processed through the ISS calculation (e.g., due to insufficient consistent sales or not meeting other thresholds in earlier modules), this module will generally assign it an “Exit” tag. If it’s completely missing, it likely indicates a data ingestion issue in earlier steps or that the size was so low-volume it was filtered out entirely before this stage.

Q: How does this module support new store openings? A: This module includes a feature to extend Ideal Size Set recommendations from established “reference” stores to new stores. This means new locations can immediately benefit from data-driven size sets, even before they build their own sales history, ensuring they are stocked effectively from day one.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To consolidate and write all calculated Ideal Size Set (ISS) metrics into the database for reporting and operational use.
- **Main Input:** Raw sales, consistent sales, and all processed Ideal Size Set analysis objects (with tagged sizes and contributions) from previous ISS modules.
- **Main Output:** Detailed Size Set Output and Comprehensive Size Set Report.

- **Warning Signs:** Missing product or store attributes in reports, or unexpected “Exit” tags for seemingly popular sizes.
 - **Key Settings:** Primarily relies on settings from previous modules and system-wide master data.
-

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: ISSChannelAgSizeContri

Last Updated: 2025-05-23

1. Business Objective

The ISSChannelAgSizeContri module calculates the final sales contribution percentage for each specific size at the channel-Attribute Group (AG) level. It uses raw sales data and the adjusted size contributions from previous Ideal Size Set (ISS) modules to determine these channel-specific contributions, which are then exported for analysis and strategic decision-making.

Why This Matters: This process provides a precise understanding of which sizes drive sales within specific channels (like online vs. in-store) and product categories. By knowing the exact demand breakdown by size at this granular level, businesses can optimize inventory planning and allocation to match demand more accurately, leading to reduced stockouts of popular sizes, less excess inventory, and ultimately, improved sales and profitability across different sales channels.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Individual Sales Transactions:** Detailed records of every product sold, including its quantity, the store where it was sold, and its category. *We need this to aggregate total sales volumes for specific stores, categories, and channels.*
- **Adjusted Size Contributions from Previous ISS Modules:** The calculated percentage contribution of each size to sales for specific product Attribute Groups, as determined and refined by prior ISS processing steps (e.g., from ISSTaggingModule after continuity adjustments). *This provides the refined, strategic view of how important each size is to its product group.*

Product Data

- **Product Attribute Group Details:** Information about how products are categorized into Attribute Groups (e.g., “Women’s Active Leggings”), including details like category, subcategory, brand, gender, and theme. *This provides the necessary context to group and analyze size contributions accurately at the Attribute Group level.*

- **Product Variant Details (SKU Information):** Information about each unique item, including its specific size. *This is essential to link sales to individual sizes within an Attribute Group.*

Operational Data

- **Store Details:** Information about each retail location, including its unique identifier and its associated sales channel (e.g., ‘Online’, ‘Retail Store’). *This is crucial for aggregating sales data and calculating size contributions at the channel level.*
- **Pivotal Size Data (from prior ISS outputs):** The output from previous ISS modules, which includes information on stores, Attribute Groups, sizes, and their adjusted sales contribution. *This acts as a pivotal input, carrying forward the refined size importance from earlier steps.*

3. Business Settings

Setting Name	Default Value	Description	When to Change
Current Reporting Date	Today’s Date	The date for which the size contribution output is generated. This is generally used for internal system date management.	Usually remains the system’s current date, but can be adjusted for historical reporting or simulations.

Note: This module primarily consumes outputs from previous modules and system-wide configurations rather than having many direct runtime settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Raw Sales Transactions & Pivotal Size Data from ISS] -->
    B[Aggregate Sales & Calculate Interim Size Quantities]
    B --> C[Calculate Final Channel-AG Size Contributions]
    C --> D[Final Size Contribution Data for Analysis & Export]
  
```

1. Start with → Raw Sales Transactions and the Pivotal Size Data (containing adjusted size contributions) from previous ISS modules.
2. Then → Aggregate total sales quantities for each Store–Category combination and each Channel–Attribute Group combination from the raw sales data. Simultaneously, for each size within a Store–Attribute Group from the Pivotal data, calculate an “interim” quantity by multiplying its adjusted contribution percentage by the total sales of its respective Store–Category. These interim quantities are then summed up at the Channel–Attribute Group–Size level.

3. Next → For each Channel-Attribute Group, sum up all the calculated interim quantities to get a total “weight” for that specific group. Then, divide each individual size’s interim quantity by this total “weight” to determine its final, normalized sales contribution percentage within that Channel-Attribute Group.
4. Finally → Prepare and write the calculated final size contributions, along with relevant Attribute Group details and the total sales for the Channel-Attribute Group, to the database, making them available for reporting and further analysis.

5. Step-by-Step Actions

1. **Initialize** data structures.
 - What we do: *We prepare empty maps and lists to store aggregated sales, interim size quantities, and the final size contribution results.*
 - Why we do it: *This sets up the necessary containers for processing and organizing the incoming data efficiently.*
2. **Aggregate** raw sales data.
 - What we do: *We sum up total units sold for every combination of Store and Category, and separately for every combination of Channel and Attribute Group, directly from the raw sales transactions.*
 - Why we do it: *This gives us the overall sales context at various levels, which is crucial for determining how much each size truly contributes to the total demand within specific segments.*
3. **Calculate** interim size quantities at Channel-AG level.
 - What we do: *For each size from the output of previous ISS modules (the “Pivotal Row”), we determine its store’s category and retrieve the total sales for that Store-Category combination. We then multiply this total Store-Category sales by the size’s adjusted contribution (from the Pivotal Row) to get an “interim” quantity for that size at that store level. Finally, we sum these interim quantities for the same size across all stores within a given Channel-Attribute Group combination.*
 - Why we do it: *This step translates the abstract percentage contributions from previous modules back into concrete quantities, allowing us to aggregate demand effectively across different channels and product groups before calculating the final percentages.*
4. **Determine** final size contributions.
 - What we do: *For each unique Channel-Attribute Group combination, we sum up all the interim quantities for its sizes to get a total “weight.” Then, for each size within that group, we divide its individual interim quantity by this total “weight” to arrive at its final, normalized sales contribution percentage.*
 - Why we do it: *This normalizes the interim quantities into a clear, comparable percentage for each size, showing its proportional demand within its specific channel and product category.*
5. **Write** output to the database.
 - What we do: *We take the calculated final size contributions and store them in two formats in the database: one detailed table for internal system use and one enriched table for broader business reporting, including additional Attribute Group details like category, brand, and gender.*
 - Why we do it: *This makes the crucial size contribution data accessible for inventory systems to make allocation decisions and for business users to analyze demand patterns and create reports.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- We want to determine the final size contributions for “Women's Active Leggings” (Attribute Group 789) in the ‘Online’ sales channel.
- Raw Sales Data:
 - Store A (Online Channel), Category "Leggings": 100 units sold in total.
 - Store B (Online Channel), Category "Leggings": 50 units sold in total.
- Pivotal Size Data (from previous ISS modules for “Women's Active Leggings”):
 - Store A: Size S (adjContri: 0.20), Size M (adjContri: 0.50), Size L (adjContri: 0.30)
 - Store B: Size S (adjContri: 0.30), Size M (adjContri: 0.40), Size L (adjContri: 0.30)

What Happens

1. Aggregate Raw Sales:

- The module calculates total Store–Category sales:
Store A–Leggings = 100 units, Store B–Leggings = 50 units.
- It also calculates total Channel–Attribute Group sales: Online–Women's Active Leggings = 100 + 50 = 150 units.

2. Calculate Interim Size Quantities at Channel-AG level:

- For Online–Women's Active Leggings – Size S:
 - From Store A: 100 units (Store A total) * 0.20 (S adjContri) = 20 interim units
 - From Store B: 50 units (Store B total) * 0.30 (S adjContri) = 15 interim units
 - Total Interim for Size S = 20 + 15 = 35 units
- For Online–Women's Active Leggings – Size M:
 - From Store A: 100 units (Store A total) * 0.50 (M adjContri) = 50 interim units
 - From Store B: 50 units (Store B total) * 0.40 (M adjContri) = 20 interim units
 - Total Interim for Size M = 50 + 20 = 70 units
- For Online–Women's Active Leggings – Size L:
 - From Store A: 100 units (Store A total) * 0.30 (L adjContri) = 30 interim units
 - From Store B: 50 units (Store B total) * 0.30 (L adjContri) = 15 interim units
 - Total Interim for Size L = 30 + 15 = 45 units

3. Determine Final Size Contributions:

- Total “weight” for Online–Women's Active Leggings (sum of all interim units) = 35 (S) + 70 (M) + 45 (L) = 150 units.
- Final Contribution for Online–Women's Active Leggings – Size S = $35 / 150 = 0.2333$ (23.33%)
- Final Contribution for Online–Women's Active Leggings – Size M = $70 / 150 = 0.4667$ (46.67%)

- Final Contribution for Online-Women's Active Leggings – Size L = $45 / 150 = 0.3000$ (30.00%)

Final Outcome

The system will record the final size contributions for “Women’s Active Leggings” in the ‘Online’ channel as: Size S (23.33%), Size M (46.67%), and Size L (30.00%). This precise breakdown allows online merchandisers to ensure they stock these sizes in the correct proportions, optimizing inventory for their specific channel’s demand.

Key Takeaway: By aggregating and normalizing sales data from various sources at the channel-Attribute Group level, this module provides highly accurate and actionable size contribution percentages, enabling better inventory allocation for specific sales channels.

7. Results

This step produces:

Main Outputs

- **Channel-Attribute Group Size Contributions:** A detailed dataset containing the final calculated sales contribution percentage for each size within every Channel-Attribute Group combination. It includes the channel, Attribute Group ID, size, its final contribution percentage, and the total sales quantity for that Channel-Attribute Group. *This output provides the core data for understanding proportional size demand across channels.*
- **Exportable Channel-Attribute Group Size Contributions:** An enriched, denormalized version of the above data, ready for export to reporting tools. It includes additional descriptive attributes for the Attribute Group (e.g., category, subcategory, brand, gender, price bucket, theme), making it immediately usable for business analysis. *This provides a comprehensive view for strategic decision-making and cross-channel comparisons.*

Business Reports

- **Channel Size Demand Profile Report:** A report for channel managers and merchandising teams showing the percentage breakdown of sales by size for key product categories within their specific channels. *Used to tailor inventory assortments to the unique demand patterns of each sales channel (e.g., online vs. physical stores).*
- **Attribute Group Size Performance Report:** A detailed report that allows category managers to analyze how individual sizes contribute to overall sales within specific Attribute Groups across all channels. *Used to identify best-performing sizes and potential gaps or overstocks within product categories.*

8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process data up to the last available record, potentially leading to outdated or incomplete size contribution calculations.	Verify data source connections for all upstream modules (raw sales, ISS outputs); escalate to IT/data engineering team for missing feeds; consider re-running the module once data is available.
No adjusted contributions from previous ISS modules	If previous ISS modules did not produce adjusted contributions for a specific Store-Attribute Group-Size (e.g., due to insufficient sales or filtering), these combinations will not contribute to the channel-level calculation in this module.	Review the settings and outputs of previous ISS modules (e.g., ISSPrepareSalesModule, ISSFlaggingModule, ISSTaggingModule) to understand why certain sizes or combinations might have been excluded.
Zero total sales for a Channel-Attribute Group	If a particular Channel-Attribute Group had no recorded raw sales, the module will not be able to calculate size contributions for it.	This is expected for product groups with no sales. For new launches, initial contributions might need to be set manually or based on similar product groups.
Inaccurate product or store master data	Incorrect Attribute Group definitions or channel assignments for stores will lead to miscategorized size contributions.	Implement robust data validation for all master data sources; regularly audit and cleanse product and store attribute definitions.

9. Next Steps

Before This Step

- **Raw Sales Data Ingestion:** All individual sales transactions must be loaded and available, providing the foundational sales volumes.
- **Pivotal Size Data Generation (from previous ISS modules):** The output from previous ISS modules (ISSTaggingModule, ISSSizeContinuityModule) containing sizes and their adjusted contribution percentages, must be complete and accurate.

- **Product and Store Master Data Definition:** Comprehensive and accurate details about Attribute Groups, product variants, and store channels must be available.

After This Step

- **Inventory Forecasting and Allocation Systems:** The generated channel-specific size contribution percentages serve as a key input for inventory management systems to refine future purchase orders and allocate stock precisely to meet demand patterns unique to each sales channel.
- **Business Intelligence & Reporting Tools:** The exportable size contribution data is consumed by dashboards and reporting tools, enabling merchandisers, planners, and channel managers to analyze performance, identify trends, and optimize assortments at a glance.

10. Common Questions

Q: Why do we need size contributions at the channel level, not just overall? A: Customer demand can vary significantly by channel. For example, online shoppers might prefer different sizes than in-store shoppers due to fitting room availability or demographic differences. Channel-specific contributions allow us to stock smarter, ensuring the right sizes are available where they are most demanded.

Q: How do these final contributions differ from the “adjusted contributions” from previous ISS modules? A: The “adjusted contributions” from previous ISS modules are often derived at a more granular level (e.g., store-Attribute Group). This module aggregates and normalizes those contributions to the broader Channel-Attribute Group level, giving a consolidated view of size demand across all stores within a specific channel for a given product type.

Q: Can this module help improve online sales performance? A: Absolutely. By providing precise size contribution percentages for the online channel, it helps ensure that the sizes customers are most likely to buy online are adequately stocked, reducing lost sales from missing popular sizes and improving the overall online shopping experience.

Q: What happens if a channel doesn’t have enough sales data for a specific Attribute Group? A: If there isn’t sufficient raw sales data for a particular Channel-Attribute Group, this module may not be able to calculate a meaningful size contribution for it. In such cases, businesses might rely on contributions from similar channels or broader Attribute Group data for initial planning.

Q: How does this module contribute to reducing excess inventory? A: By providing accurate size demand profiles for each channel, it prevents overstocking less popular sizes while ensuring adequate stock of high-demand sizes. This targeted allocation minimizes excess inventory that would otherwise lead to markdowns and holding costs, directly improving profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate the final sales contribution percentage for each size at the Channel-Attribute Group level.
- **Main Input:** Raw sales transactions and adjusted size contributions from previous ISS modules.
- **Main Output:** Final size contribution percentages for each size within Channel-Attribute Groups, ready for inventory systems and reporting.
- **Warning Signs:** Unexpected or disproportionate size contributions for certain channels, or missing data for active product groups.
- **Key Settings:** Primarily relies on outputs and settings from previous modules.

Need Help? Contact your Merchandising or Inventory Planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

NOTE: Use these business definitions for abbreviations throughout this documentation: - OD: Optimum Depth = The ideal number of unique items (SKUs) to stock in a category or display. - OW: Optimum Width = The ideal number of different categories or styles to offer. - ISS: Ideal Size Set = The best combination of sizes to maximize sales and minimize stockouts. - ROS: Rate of Sale = The speed at which products are sold over a period (e.g., units/week). - AG: Attribute Group = A set of product attributes grouped for analysis (e.g., color, size, brand).

ISS FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ document provides answers to common business questions about the Ideal Size Set (ISS) algorithm, its process, and its outputs.

General

Q: What is the Ideal Size Set (ISS) algorithm designed to do? **A:** The ISS algorithm helps determine the optimal combination of sizes (ISS) to stock for products within specific Attribute Groups (AGs), aiming to maximize sales and minimize stockouts.

Q: How does ISS relate to Optimum Depth (OD) or Optimum Width (OW)? **A:** While OD determines the ideal number of unique items (SKUs) and OW the ideal number of categories, ISS focuses specifically on identifying the best *combination of sizes* for those items within a specific category or Attribute Group (AG).

Q: What is an Attribute Group (AG)? **A:** An AG is a collection of product characteristics, such as color, material, or brand, that are grouped together for analysis. This allows the system to determine common Ideal Size Set (ISS) strategies for similar items.

Q: Why is calculating the Ideal Size Set important for our business? **A:** Calculating the ISS helps us make data-driven decisions on which sizes to stock. This leads to improved inventory efficiency, reduced excess stock, better product availability, and ultimately, increased sales by ensuring we have the right sizes in the right places.

Q: What kind of data does the ISS algorithm use? **A:** The algorithm primarily uses historical sales data, including units sold over a defined period, along with information about currently available sizes and their Attribute Groups (AGs).

Configuration & Settings

Q: Can I customize how the Ideal Size Set (ISS) is calculated? **A:** Yes, the algorithm uses various configuration settings, such as the sales duration for analysis and criteria for identifying “paramount” or core sizes, which can be adjusted to align with specific business needs.

Q: How does the algorithm decide which sales data to use? **A:** The ISSPrepareSalesModule filters sales data based on a defined duration and considers only sales of “paramount” (core) sizes that were actually available during the sales period. This ensures only “unbroken” sales (where stockouts didn’t limit demand) are used for accurate analysis.

Q: Is there a way to influence the “pivotal” size tagging? **A:** The ISSTaggingModule uses predefined size set properties and computed quantity percentages to tag sizes as Pivotal (P), Non-Pivotal (NP), or Exit (E). These definitions are based on configurable business rules.

Q: What is the purpose of the size continuity setting? **A:** If enabled, the ISSSizeContinuityModule ensures that the recommended Ideal Size Set (ISS) is logically continuous. For example, if a Small and a Large size are deemed “Pivotal,” all intermediate sizes (like Medium) will also be marked as “Pivotal,” even if their individual contribution was lower, to maintain a full size run.

Q: Where are these settings managed or found? **A:** These critical settings are typically managed within the system’s central configuration files or through a dedicated management interface.

Data & Inputs

Q: What does “unbroken” sales data mean in the context of ISS? **A:** “Unbroken” sales refer to sales data for products where we had sufficient stock available during the sales period. This ensures that reported sales weren’t artificially limited by stockouts, providing a true measure of customer demand for a specific size.

Q: How far back does the sales data go for analysis? **A:** The ISSPrepareSalesModule uses sales data filtered by a defined duration, which is a configurable parameter. This allows the system to focus on recent and relevant sales trends.

Q: What is a “paramount” size? **A:** A “paramount” size refers to a core or key size that is always considered for analysis, regardless of immediate sales performance, to ensure fundamental sizes are always part of the Ideal Size Set (ISS) consideration.

Q: How are Attribute Groups (AGs) defined for input? **A:** AGs are pre-defined groupings of product attributes (e.g., specific color groups, material types, or brands) that the system uses to analyze sales and determine Ideal Size Sets (ISS) for similar items. These definitions are provided as input to the system.

Q: What if there's missing sales data for a specific Attribute Group (AG) or store? **A:** The ISSFlaggingModule identifies relevant store-AG combinations based on available sales data. If data is insufficient, that specific AG might not receive an Ideal Size Set (ISS) recommendation for that store, or the system may fall back to higher-level aggregations (e.g., channel, region) if configured to do so.

Process & Logic

Q: What is the general flow of the Ideal Size Set (ISS) calculation? **A:** The process starts with preparing sales data (ISSPrepareSalesModule), then flagging relevant combinations (ISSFlaggingModule), tagging individual sizes as Pivotal (P), Non-Pivotal (NP), or Exit (E) (ISSTaggingModule), ensuring size continuity (ISSSizeContinuityModule), and finally consolidating and writing results to the database (ISSOutputModule).

Q: How does the system determine if a size is "Pivotal" (P), "Non-Pivotal" (NP), or "Exit" (E)? **A:** The ISSTaggingModule calculates each size's sales quantity contribution within an Attribute Group (AG). Sizes are then tagged based on their contribution percentage relative to defined thresholds and overall size set properties.

Q: Why would a size that didn't sell much be tagged as "Pivotal"? **A:** This can happen due to the ISSSizeContinuityModule. If a size falls within a continuous range of other "Pivotal" sizes (e.g., Small and Large are Pivotal, but Medium had lower individual sales), the Medium might also be tagged "Pivotal" to ensure a logical and complete size run for the Ideal Size Set (ISS).

Q: How does the algorithm handle different sales channels or regions? **A:** The ISSFlaggingModule groups sales data by various rollup levels (e.g., store, channel, region) to perform calculations relevant to those specific contexts. The ISSChannelAgSizeContri module then provides final size contributions at the channel-Attribute Group (AG) level.

Q: What role does the ISSFlaggingModule play? **A:** The ISSFlaggingModule acts as an initial filter, identifying which store-Attribute Group (AG) combinations have sufficient and relevant sales data to be considered for Ideal Size Set (ISS) analysis. It also initializes the data objects for these combinations.

Outputs & Results

Q: Where can I find the final Ideal Size Set (ISS) recommendations? **A:** The ISSOutputModule consolidates all calculated ISS metrics and writes them to the database. These results are typically available in specific reports, dashboards, or integrated inventory planning systems for direct use.

Q: What kind of information is included in the output? **A:** The output includes detailed metrics such as raw and "unbroken" sales quantities, the final size group tags (Pivotal, Non-Pivotal, Exit) for each size within an Attribute Group (AG), and their calculated contribution percentages.

Q: How are the final size contributions at the channel-AG level determined? **A:** The `ISSChannelAgSizeContr i` module uses the raw sales data along with the “pivotal” output from the core ISS modules to calculate and export these final size contributions. This provides a comprehensive view of size performance and recommendations tailored per channel and Attribute Group (AG).

Q: Can I see the intermediate steps of the calculation? **A:** While the primary output is the final Ideal Size Set (ISS) recommendation, internal system logs or specific debugging views might expose intermediate data points. However, the direct user output is the consolidated and actionable recommendation.

Q: What does “PivotalRow” and “ExportIssOutputRow” mean in the output? **A:** These refer to specific data structures or tables within the database where the final calculated Ideal Size Set (ISS) recommendations and associated metrics are stored. They represent the structured data ready for reporting, analysis, or integration with other business systems.

Error Handling & Edge Cases

Q: What happens if there’s not enough sales data for an Attribute Group (AG)? **A:** The `ISSFlaggingModule` is designed to tag store-AG combinations based on sales data relevance. If there isn’t enough historical sales data meeting the defined criteria, that specific AG might not receive an Ideal Size Set (ISS) recommendation for that store, or it may default to a broader aggregation level if configured.

Q: How does the algorithm handle new products with no sales history? **A:** Products with no sales history would not have an Ideal Size Set (ISS) determined by this algorithm directly, as it relies on historical sales data. Initial size sets for such products would need to be determined through other means, such as merchandising intuition, comparisons to similar existing products, or specific launch strategies.

Q: What if there’s an error in the input sales data? **A:** The `ISSPrepareSalesModule` includes data filtering steps. However, significant data quality issues (e.g., incorrect sales figures, missing size information) could lead to inaccurate Ideal Size Set (ISS) recommendations or the exclusion of affected data from the calculation. Data integrity is crucial for reliable outputs.

Q: Can the algorithm produce an Ideal Size Set (ISS) that doesn’t make logical sense (e.g., missing a common size)? **A:** The `ISSSizeContinuityModule` is specifically designed to prevent illogical size sets by ensuring a continuous range of “Pivotal” sizes. If this module is enabled and configured correctly, it should mitigate such issues and ensure a practical size run is recommended.

Q: What should I do if the recommended Ideal Size Set (ISS) seems incorrect? **A:** First, verify the input sales data and relevant configuration settings (e.g., sales duration, “paramount” size definitions) used for the calculation. If issues persist after review, please contact the support team with specific examples and details for investigation. It might indicate an unexpected data pattern or a need to adjust algorithm parameters.

Need More Help? Please contact the internal support team for further assistance.

ISS Documentation

****Version**:** 1.0

****Last Updated**:** 2025-05-23

1. Executive Summary

The Ideal Size Set (ISS) algorithm is a crucial tool designed to optimize inventory planning by identifying the most effective size assortment for various product categories and display configurations. By analyzing sales data and product attributes, ISS minimizes stockouts, reduces excess inventory, and maximizes sales potential, directly improving profitability and customer satisfaction.

2. Architecture Overview

The Ideal Size Set (ISS) calculation involves a sequential pipeline of modules. The process begins with ``ISSPrepareSalesModule`` which filters and aggregates sales data. This prepared data is then progressively processed by ``ISSFlaggingModule``, ``ISSTaggingModule``, and ``ISSSizeContinuityModule`` to determine and refine pivotal sizes. Finally, ``ISSOutputModule`` consolidates and persists the results, while ``ISSChannelAgSizeContri`` uses the output of this core pipeline and raw sales data to calculate final size contributions.

```
```mermaid
graph TD
 A[ISSPrepareSalesModule] -->|data_flow: productSales, unBrokenSales, storeAttrGroupMap| B[ISSFlaggingModule];
 B -->|data_flow: issDataMap with SizeData| C[ISSTaggingModule];
 C -->|data_flow: issDataMap with updated tags| D[ISSSizeContinuityModule];
 D -->|data_flow: finalized ISSData objects| E[ISSOutputModule];
 A -->|data_flow: productSales, unBrokenSales| E;
 E -->|data_flow: PivotalRow entities via DB| F[ISSChannelAgSizeContri];
 A -->|data_flow: raw sales data from Cache| F;

 classDef default fill:#f9f,stroke:#333,stroke-width:2px;
```

**Module Descriptions:**

- \* **ISSPrepareSalesModule:** Prepares and aggregates sales data, filtering it based on duration and identifying “unbroken” sales for the Ideal Size Set calculation.
- \* **ISSFlaggingModule:** Tags store-Attribute Group (AG) combinations based on sales data, identifying relevant sales for analysis and initializing ISSData objects.
- \* **ISSTaggingModule:** Computes quantity contributions for each size within an AG, determines pivotal quantity percentages,

and tags sizes as Pivotal (P), Non-Pivotal (NP), or Exit (E). \*

**ISSSizeContinuityModule:** Ensures logical continuity in size sets by analyzing tagged sizes and adjusting pivotal tags to maintain a continuous range of pivotal sizes. \* **ISSOutputModule:** Consolidates all calculated ISS metrics and writes final output rows to the database for reporting and further analysis. \*

**ISSChannelAgSizeContri:** Calculates the final size contribution at the channel-Attribute Group (AG) level using raw sales data and pivotal outputs.

### 3. Prerequisites & Setup

To run the ISS pipeline, ensure you have the following installed:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Build Tool:** Apache Maven (version 3.6.0+) or Gradle (version 6.0+) for dependency management and project compilation.

#### Commands to Build and Run:

##### 1. Clone the Repository:

```
git clone https://github.com/your-repo/iss.git
cd iss
```

##### 2. Build the Project:

###### ◦ Maven:

```
mvn clean install
```

###### ◦ Gradle:

```
gradle clean build
```

##### 3. Run the ModuleGroup Orchestrator: The ISS pipeline is orchestrated by a main ModuleGroup class (e.g., `com.example.iss.ModuleGroupRunner`).

###### ◦ Maven:

```
java -jar target/iss-<version>.jar --config=args.properties
```

###### ◦ Gradle:

```
java -jar build/libs/iss-<version>.jar --
 config=args.properties
```

Ensure `args.properties` is configured correctly for your environment.

### 4. Folder & File Structure

```
iss/
├─ module-group/ # Contains the main orchestrator
 (ModuleGroupRunner) for the ISS pipeline.
├─ submodules/ # Houses all individual ISS
```

processing modules (e.g., PrepareSales, Flagging).

- └ args.properties # Global configuration file for the entire ISS application.
- └ common-data.json # Defines the schema for shared data objects used across modules.
- └ outputs/ # Directory where final output files or reports are stored.

## 5. Global Configuration

The args.properties file defines global parameters influencing the entire ISS pipeline.

Setting Name	Default Value	Description	Valid Range/ When to Change
sales.duration.weeks	13	Number of weeks of historical sales data to consider for ISS calculations.	1–52 weeks; Adjust based on product lifecycle and sales volatility.
iss.pivotal.contribution.min	0.05	Minimum percentage contribution for a size to be considered “pivotal”.	0.01–0.20; Lower for broader size sets, higher for more focused.
iss.enable.size.continuity	true	Enables/disables the size continuity logic.	true/false; Set to false if discontinuous size sets are desired.
iss.output.to.database	true	Controls whether final ISS results are written to the database.	true/false; false for testing or file-based outputs only.
app.debug.mode	false	Enables detailed logging and debug output for troubleshooting.	true/false; Enable for development/debugging, disable for production.
data.source.type	“database”	Specifies the primary data source (e.g., “database”, “flat_file”).	“database”, “flat_file”, etc.; Depends on data

Setting Name	Default Value	Description	Valid Range/ When to Change
			ingestion method.

## 6. Shared Data Model

The `common-data.json` schema defines the structure of core data objects shared and processed across various ISS modules.

Field	Type	Description
<code>productId</code>	string	Unique identifier for a product.
<code>sku</code>	string	Stock Keeping Unit, unique identifier for a product variant (item).
<code>storeId</code>	string	Unique identifier for a retail store or location.
<code>attributeGroup</code>	object	An AG containing various product attributes (e.g., color, style).
<code>size</code>	string	The specific size of the product (e.g., “S”, “M”, “L”, “30x32”).
<code>quantity</code>	integer	Number of units sold or stocked.
<code>salesDate</code>	string	Date of the sales transaction (YYYY-MM-DD format).
<code>channel</code>	string	Sales channel (e.g., “Online”, “Retail”, “Wholesale”).
<code>grnLive</code>	boolean	Indicates if a size was live (available) via GRN during the period.
<code>paramountSize</code>	boolean	Indicates if a size is considered a core/paramount size.
<code>sizeTag</code>	string	The calculated tag for a size (P: Pivotal, NP: Non-Pivotal, E: Exit).
<code>contribution</code>	number	The percentage contribution of a size to an AG’s sales.

## 7. Submodule Chapters

### 7.1. ISSPrepareSalesModule

**Module Name & Business Objective** The `ISSPrepareSalesModule` is responsible for preparing and aggregating sales data for the Ideal Size Set (ISS) calculation. It filters sales data based on a defined duration, applies filters for “paramount” (core) sizes and GRN (Goods Received Note) live sizes to identify “unbroken” sales, and denormalizes the output for further processing. This module initializes the central ISS data store. *Its objective is to ensure that only relevant and reliable sales data is used for subsequent ISS calculations, laying a clean foundation for analysis.*

**Required Information** \* **Sales Data:** \* Historical sales transactions (units sold, SKU, store, date). \* Product SKU information (linking to size, AG). \* **Product Data:** \* Master data indicating paramountSize (core sizes) for products. \* GRNLive status for sizes to determine availability during sales periods. \* **Operational Data:** \* sales.duration.weeks (global configuration).

**Business Settings** \* sales.duration.weeks: Defines the look-back period for sales data. \* paramount.size.filter.enabled: (Implicit) If true, only considers sales for paramount sizes. \* grn.live.filter.enabled: (Implicit) If true, only considers sales where GRN indicates the size was live.

## Process Flow

graph TD

A[Raw Sales Data, Product Master] --> B(Filter by Date & Paramount Sizes);

B --> C(Identify GRN Live Sales & Unbroken Sales);

C --> D(Aggregate & Denormalize Sales);

D --> E[{ISSModuleData (productSales, unBrokenSales, storeAttrGroupMap)}];

**Step-by-Step Actions** 1. **Load Sales Data:** Retrieve all historical sales transactions and product master data from the data source. 2. **Filter by Duration:** Filter sales to include only those within the sales.duration.weeks defined in args.properties. *This ensures recency and relevance.* 3. **Identify Paramount Sales:** Mark sales associated with “paramount” sizes as specified in product master data. *Paramount sizes represent the core offering that should ideally always be stocked.* 4. **Determine GRN Live Sales:** Check and flag sales where the specific size was actively available (GRN live) during the sales period. *This helps identify sales that weren’t constrained by stockouts.* 5. **Calculate Unbroken Sales:** Aggregate “unbroken” sales, which are sales of paramount sizes where the GRN indicates the size was live. *This metric represents true demand without supply constraints.* 6. **Denormalize Data:** Prepare the data for the next stages by creating a storeAttrGroupMap and populating productSales and unBrokenSales lists within ISSModuleData. *This structure optimizes subsequent processing by grouping data relevant to ISS calculations.*

**Sample Scenario** A clothing retailer wants to calculate ISS for T-shirts (AG: T-Shirt, Color: Blue). Over the last 13 weeks, Size M had 100 units sold, Size L had 80 units, and Size S had 50 units. All these sizes are marked as ‘paramount’. For Size M, GRN data shows it was consistently in stock. For Size L, it was in stock 90% of the time, and for Size S, only 60%. \* **Raw Sales:** M: 100, L: 80, S: 50 \* **Unbroken Sales:** M: 100 (100% in stock), L: 72 (80 \* 0.90), S: 30 (50 \* 0.60) **Key Takeaway:** *The module provides a refined view of demand, differentiating between total sales and sales that occurred when inventory was reliably available, giving a truer picture of ideal demand.*

**Results** \* productSales list: All filtered sales transactions. \* unBrokenSales list: Sales data filtered by paramount sizes and GRN live status. \* storeAttrGroupMap: A map facilitating easy access to sales data grouped by store and AG. \* Initialized ISSModuleData object with these core data sets.

**Unusual Cases** \* **Missing or delayed data feed:** Leads to incomplete or outdated sales data, impacting accuracy. \* **Insufficient sales data for AGs:** Newly introduced products or low-volume categories may not have enough data for reliable

calculation. \* **Inaccurate GRN data:** Misleading “live” status leads to incorrect unbroken sales figures. \* **No “paramount” size definition:** All sizes treated equally, potentially diluting the focus on core offerings.

**Next Steps** The prepared sales data (ISSModuleData) is passed to the ISSFlaggingModule for initial tagging and metric calculations.

**Common Questions** 1. How far back should we look for sales data? 2. What defines a “paramount” size for our business? 3. Why are sales excluded if GRN shows stockouts? 4. Can we adjust the sales.duration.weeks per category? 5. How do new product introductions affect the data preparation?

**Quick Reference** \* **OD:** Ideal items to stock. \* **OW:** Ideal categories to offer. \* **ISS:** Best size combination. \* **ROS:** Sales speed. \* **AG:** Product attribute group.

## 7.2. ISSFlaggingModule

**Module Name & Business Objective** The ISSFlaggingModule tags store-Attribute Group (AG) combinations based on sales data, identifying relevant sales for analysis. It groups sales by various rollup levels (e.g., store, channel, region) and attributes, then calculates preliminary metrics and initializes ISSData objects for these combinations, flagging them according to defined criteria. *Its objective is to categorize specific store-AG combinations based on their sales activity, determining which ones are significant enough for detailed ISS analysis.*

**Required Information** \* **Sales Data:** \* productSales list (from ISSPrepareSalesModule). \* unBrokenSales list (from ISSPrepareSalesModule). \* **Product Data:** \* Product SKU information (linking to size, AG). \* **Operational Data:** \* ISSModuleData (containing prepared sales). \* Configuration for rollup levels (e.g., store, channel, region).

**Business Settings** \* **flagging.rollup.levels:** Defines how sales data should be grouped (e.g., by store-AG, channel-AG). \* **flagging.minimum.sales.threshold:** Minimum sales quantity for an AG to be flagged as relevant. \* **flagging.minimum.stores.threshold:** Minimum number of stores an AG must be sold in to be considered.

### Process Flow

graph TD

```
A{ISSModuleData} --> B(Group Sales by Rollup Levels & AGs);
B --> C(Calculate Preliminary Metrics per Group);
C --> D(Apply Flagging Criteria);
D --> E{Populate issDataMap with flagged ISSData objects};
```

**Step-by-Step Actions** 1. **Retrieve Prepared Sales:** Access productSales and unBrokenSales from ISSModuleData. *These lists contain the cleaned and filtered sales data.* 2. **Group Sales Data:** Aggregate sales data based on defined rollup levels (e.g., store-AG, region-AG). *This allows for analysis at different geographical or organizational scopes.* 3. **Calculate Preliminary Metrics:** For each grouped combination (e.g., a specific store’s sales of a particular T-shirt AG), **compute** total sales quantities, ROS, and other initial metrics. *These metrics are used to determine the relevance of the combination.* 4. **Apply Flagging Criteria:** **Evaluate** each combination against predefined criteria (e.g., minimum total sales,

minimum number of stores). If a combination meets the criteria, it is **flagged** as relevant for ISS calculation. *This filters out insignificant combinations that wouldn't yield meaningful size set recommendations.* 5. **Initialize ISSData Objects:** For each flagged combination, **create** an ISSData object and **populate** it with initial SizeData for each size sold, storing it in the issDataMap within ISSModuleData. *This prepares the data structure for the next stage of detailed size analysis.*

**Sample Scenario** For a specific AG (e.g., “Men’s Jeans - Slim Fit”) at a store (StoreA), sales data shows 150 units sold across all sizes. The business rule states an AG must have at least 100 units sold in a store to be flagged. Since StoreA’s sales for this AG (150 units) exceed the threshold, this StoreA–Men's Jeans Slim Fit combination is flagged. If StoreB only sold 80 units of the same AG, it would not be flagged and thus skipped in subsequent detailed ISS calculations. Key Takeaway: *This module acts as a gatekeeper, ensuring that only significant sales patterns are subjected to the intensive ISS calculation, saving computational resources and focusing on impactful areas.*

**Results** \* issDataMap: A map containing ISSData objects, each representing a flagged store-AG combination ready for size-level analysis. \* Preliminary metrics (total sales, ROS) computed for each flagged combination.

**Unusual Cases** \* **Missing or delayed data feed:** Leads to incomplete or outdated sales data, impacting accuracy. \* **Insufficient sales data for AGs:** AGs with low sales volume might never meet flagging criteria, even if they are strategically important. \* **Incorrect rollup level configuration:** Analysis might be done at an inappropriate granularity (e.g., too broad or too narrow). \* **Too aggressive flagging criteria:** Potentially excludes valid AGs from ISS optimization.

**Next Steps** The ISSData objects in issDataMap are passed to the ISSTaggingModule for detailed size contribution calculation and initial tagging.

**Common Questions** 1. What are the “rollup levels” and why do we use them? 2. How do we define the minimum sales threshold for flagging? 3. What if a strategically important AG doesn’t meet the flagging criteria? 4. Can we adjust flagging criteria for different product categories? 5. How often are these flags re-evaluated?

**Quick Reference** \* **OD:** Optimum Depth. \* **OW:** Optimum Width. \* **ISS:** Ideal Size Set. \* **ROS:** Rate of Sale. \* **AG:** Attribute Group.

### 7.3. ISSTaggingModule

**Module Name & Business Objective** The ISSTaggingModule processes the ISSData generated by the flagging module to compute quantity contributions for each size within an AG, determines the “pivotal” quantity percentage, and tags sizes as Pivotal (P), Non-Pivotal (NP), or Exit (E) based on predefined size set properties. *Its objective is to classify each size within an AG based on its sales performance and contribution, guiding inventory decisions towards optimal stock levels.*

**Required Information** \* **Sales Data:** \* Sales quantities per size for each store-AG combination (from ISSData objects). \* **Product Data:** \* Product SKU information (linking to size, AG). \* **Operational Data:** \* issDataMap (populated by ISSFlaggingModule). \* iss.pivotal.contribution.min (global configuration). \* Size set properties (e.g., minimum percentage for pivotal).

**Business Settings** \* `pivotal.quantity.percentage.threshold`: The minimum percentage contribution a size must have to be considered pivotal. \* `non.pivotal.quantity.percentage.threshold`: The maximum percentage contribution for a size to be considered non-pivotal. \* `exit.quantity.percentage.threshold`: The maximum percentage contribution for a size to be considered an exit size.

## Process Flow

graph TD

```
A{ISSData objects (from issDataMap)} --> B(Calculate Size
Quantity Contributions);
B --> C(Determine Pivotal Quantity Percentage);
C --> D(Apply Tagging Logic: P, NP, E);
D --> E{Update ISSData objects with sizeTags};
```

**Step-by-Step Actions**

- 1. Iterate ISSData:** Process each ISSData object for flagged store-AG combinations. *Each object contains the sales data for all sizes within that specific combination.*
- 2. Calculate Size Contributions:** For each size within an ISSData object, **compute** its contribution to the total sales quantity for that AG. *This is typically done as a percentage:  $(\text{size quantity} / \text{total AG quantity}) * 100\%$ .*
- 3. Determine Pivotal Quantity Percentage:** Identify the cumulative percentage of sales that the top-selling sizes account for until a certain threshold is reached. *This helps in defining the core sizes that drive most sales.*
- 4. Apply Tagging Logic:** \* **Tag as Pivotal (P):** If a size's contribution exceeds the `pivotal.quantity.percentage.threshold` and/or is within the top contributing sizes that form the "pivotal" quantity percentage. *These are the high-demand, must-have sizes.* \* **Tag as Non-Pivotal (NP):** If a size's contribution is below the pivotal threshold but above the `exit.quantity.percentage.threshold`. *These are supporting sizes, potentially stocked in lower quantities.* \* **Tag as Exit (E):** If a size's contribution falls below the `exit.quantity.percentage.threshold`. *These are low-demand sizes that may be considered for discontinuation or minimal stocking.*
- 5. Update ISSData:** Store the calculated size tags (P, NP, E) within the respective SizeData objects inside ISSData. *This enriches the data for the next phase of continuity checks and final output.*

**Sample Scenario** For an AG: Women's Jeans – Skinny Fit, sales are: Size 28: 40 units, Size 30: 35 units, Size 26: 20 units, Size 32: 15 units, Size 24: 10 units. Total sales: 120 units. \* Contributions: 28 (33%), 30 (29%), 26 (17%), 32 (13%), 24 (8%). \* Assume `pivotal.quantity.percentage.threshold` = 25%, `exit.quantity.percentage.threshold` = 10%. \* **Tagging:** \* Size 28 (33%) -> P (Pivotal) \* Size 30 (29%) -> P (Pivotal) \* Size 26 (17%) -> NP (Non-Pivotal) \* Size 32 (13%) -> NP (Non-Pivotal) \* Size 24 (8%) -> E (Exit) Key Takeaway: *This module quantifies the sales importance of each size, providing clear guidance on which sizes are core to the assortment (P), supporting (NP), or candidates for removal (E).*

**Results** \* Updated ISSData objects in `issDataMap` with `sizeTag` (P, NP, E) for each size, and calculated contribution percentages.

**Unusual Cases** \* **Missing size master data:** Sizes without definitions cannot be accurately tagged. \* **Uniform sales distribution:** No clear 'pivotal' sizes emerge, leading to many NP tags. \* **Parameters too strict/loose:** Incorrect



pivotal.quantity.percentage.threshold can lead to too many or too few pivotal sizes. \* **Seasonal spikes**: Short-term demand spikes can skew historical contribution percentages.

**Next Steps** The ISSData objects, now with initial size tags, are passed to the ISSSizeContinuityModule to ensure a logical sequence of pivotal sizes.

**Common Questions** 1. How are “Pivotal”, “Non-Pivotal”, and “Exit” sizes defined? 2. Can we adjust the contribution thresholds for different product types? 3. What happens if no size meets the “pivotal” threshold? 4. How do we handle new sizes introduced recently? 5. Is it possible for a size to be “P” in one store but “E” in another for the same AG?

**Quick Reference** \* **OD**: Optimum Depth. \* **OW**: Optimum Width. \* **ISS**: Ideal Size Set. \* **ROS**: Rate of Sale. \* **AG**: Attribute Group.

## 7.4. ISSSizeContinuityModule

**Module Name & Business Objective** The ISSSizeContinuityModule ensures logical continuity in size sets by analyzing tagged sizes and adjusting pivotal tags. If enabled, it identifies a continuous range of pivotal sizes within a sequence and marks all intermediate sizes as pivotal, even if their individual contribution was lower. *Its objective is to create a more consumer-friendly and operationally efficient size assortment by ensuring that all sizes within a logical “run” of core sizes are maintained, preventing gaps.*

**Required Information** \* **Sales Data**: \* Tagged sizes (P, NP, E) within ISSData objects (from ISSTaggingModule). \* **Product Data**: \* Defined size sequences (e.g., XS, S, M, L, XL; or 28, 30, 32, 34). \* **Operational Data**: \* issDataMap (with tagged sizes). \* iss.enable.size.continuity (global configuration).

**Business Settings** \* iss.enable.size.continuity: Boolean flag to activate or deactivate this module’s logic. If false, the module will simply pass through the data without changes. \* size.continuity.buffer.sizes: (Optional) Number of non-pivotal sizes to potentially include on either side of a pivotal range to ensure a smoother transition.

### Process Flow

graph TD

```
A{ISSData objects with sizeTags} --> B{Check if
iss.enable.size.continuity is true};
B -- No --> F{Pass-through};
B -- Yes --> C(Identify Pivotal Sizes Sequence);
C --> D(Determine Continuous Range);
D --> E(Adjust intermediate sizes to Pivotal);
E --> F{Update ISSData objects with adjusted sizeTags};
```

**Step-by-Step Actions** 1. **Check Configuration**: Verify if the iss.enable.size.continuity setting is true. *If not, the module performs no changes and simply passes the data through.* 2. **Iterate ISSData**: Process each ISSData object containing tagged sizes. *Each object represents a specific store-AG combination.* 3. **Order Sizes**: Arrange the sizes for the current AG in their natural sequential order (e.g., XS, S, M, L, XL; or 28, 30, 32, 34). *This is crucial for*

*identifying continuity.* 4. **Identify Pivotal Range:** Scan the ordered sizes to find the smallest and largest sizes that are currently tagged as Pivotal (P). *This defines the outer bounds of the core size range.* 5. **Adjust Intermediate Sizes:** For all sizes that fall numerically or sequentially *between* the identified smallest and largest pivotal sizes, **re-tag** them as Pivotal (P), even if they were previously NP or E. *This ensures a complete and continuous range of core sizes, preventing customer frustration from missing common sizes in a run.* 6. **Update ISSData:** Store the adjusted sizeTag for each size within the ISSData object. *This finalized tagging prepares the data for the output stage.*

**Sample Scenario** For an AG: Men's Shirts – Polo, initial tags are: XS (NP), S (P), M (NP), L (P), XL (NP), XXL (NP). \* Ordered sizes: XS, S, M, L, XL, XXL. \* Smallest P: S. Largest P: L. \* Sizes between S and L: M. \* **Action:** M, although initially NP, is re-tagged as P. \* **Resulting Tags:** XS (NP), S (P), M (P), L (P), XL (NP), XXL (NP). Key Takeaway: *Even if 'M' didn't individually sell enough to be pivotal, it's logically required between 'S' and 'L' to offer a complete size run for customers, ensuring a seamless shopping experience and operational simplicity.*

**Results** \* Updated ISSData objects in issDataMap with potentially adjusted sizeTag values, ensuring size continuity where enabled.

**Unusual Cases** \* **Missing size sequence definition:** Module cannot logically order sizes, leading to incorrect continuity application. \* **All sizes tagged "E":** No pivotal sizes to establish a range. \* **Very sparse pivotal sizes:** Only a few isolated pivotal sizes, making continuity less impactful or relevant. \* **Size sequence jumps:** E.g., clothing line skips sizes, which might be misinterpreted by the module if not handled.

**Next Steps** The ISSData objects with finalized size tags are passed to the ISSOutputModule for consolidation and persistence.

**Common Questions** 1. Why do we need size continuity? 2. Can this module be disabled if we don't want continuous ranges? 3. How does it handle non-standard size sequences (e.g., shoe sizes, waist sizes)? 4. What if a "P" size has very low sales due to a stockout, does it still get continuity? 5. Does this impact our inventory levels significantly?

**Quick Reference** \* **OD:** Optimum Depth. \* **OW:** Optimum Width. \* **ISS:** Ideal Size Set. \* **ROS:** Rate of Sale. \* **AG:** Attribute Group.

## 7.5. ISSOutputModule

**Module Name & Business Objective** The ISSOutputModule is responsible for consolidating all calculated ISS metrics and writing them to the database. It generates final output rows (PivotalRow and ExportIssOutputRow), incorporating raw and unbroken sales quantities, size group tags, and contribution percentages, ensuring data readiness for reporting and further analysis. *Its objective is to make the ISS recommendations actionable and accessible by persisting them in a structured format for consumption by other systems or for reporting.*

**Required Information** \* **Sales Data:** \* productSales (raw sales, from ISSPrepareSalesModule). \* unBrokenSales (unbroken sales, from ISSPrepareSalesModule). \* **Product Data:** \* Finalized ISSData objects with

sizeTag and contribution (from ISSSizeContinuityModule). \* **Operational Data:** \* issDataMap (containing all processed ISSData objects). \* iss.output.to.database (global configuration).

**Business Settings** \* iss.output.to.database: Boolean flag to control whether the output is written to the database. \* output.table.pivotal.name: Database table name for PivotalRow output. \* output.table.export.name: Database table name for ExportIssOutputRow output.

## Process Flow

graph TD

```
A{ISSData objects (finalized)} --> B(Consolidate Metrics);
B --> C(Generate PivotalRow records);
C --> D(Generate ExportIssOutputRow records);
D --> E{Write to Database (if enabled)};
```

**Step-by-Step Actions**

- 1. Retrieve Final Data:** Access the ISSData objects from issDataMap, which now contain all final calculations including size contributions and tags (P, NP, E). Also, **retrieve** productSales and unBrokenSales from ISSModuleData. *This provides all the necessary components for the final output.*
- 2. Consolidate Metrics:** For each ISSData object (representing a store-AG combination), **gather** all relevant metrics: total raw sales quantity, total unbroken sales quantity, and for each size within that AG, its size, tag, and contribution percentage. *This prepares the data for structured output.*
- 3. Generate PivotalRow Records:** **Create** PivotalRow entities. Each PivotalRow typically represents a store-AG-size combination and includes: storeId, attributeGroup, size, sizeTag, contribution, rawQty, unbrokenQty, and other relevant identifying information. *This record provides granular detail for each size.*
- 4. Generate ExportIssOutputRow Records:** **Create** ExportIssOutputRow entities. These might be higher-level summary rows or tailored for specific reporting needs, potentially aggregating data or presenting it in a different format. *This offers flexibility for various consumption points.*
- 5. Write to Database:** If iss.output.to.database is enabled, **persist** all generated PivotalRow and ExportIssOutputRow records to the configured database tables. *This makes the ISS recommendations available for dashboards, downstream planning systems, or other analytical tools.*

**Sample Scenario** After all modules have run for a specific AG (AG: Kids' Sneakers – Velcro) in StoreX: \* Total Raw Sales: 200 units \* Total Unbroken Sales: 180 units \* Sizes and Tags: \* Size 10: P (contribution 30%) \* Size 11: P (contribution 25%) \* Size 12: NP (contribution 20%) \* Size 9: E (contribution 10%) \* **Output:** The module will generate 4 PivotalRow records, one for each size, detailing its tag, contribution, and the overall raw/unbroken sales for the AG. For example, PivotalRow(StoreX, Kids' Sneakers – Velcro, Size 10, P, 0.30, 200, 180). Key Takeaway: *This module translates the complex internal calculations into usable, structured data that can drive real-world inventory and merchandising decisions.*

**Results** \* Populated database tables (PivotalRow, ExportIssOutputRow) with final ISS recommendations. \* Readiness for reporting, dashboarding, and integration with other business systems.

**Unusual Cases** \* **Database connection issues:** Prevents persistence of results, requiring manual intervention. \* **Schema mismatch:** Output data does not match the target database table schema. \* **Large data volumes:** Can lead to performance bottlenecks during writing. \* **Output module disabled:** No data is persisted, but the pipeline runs successfully internally.

**Next Steps** The persisted data can be used by reporting tools, inventory planning systems, and the ISSChannelAgSizeContri module for further analysis.

**Common Questions** 1. Where can I find the final ISS recommendations? 2. How often are the results updated in the database? 3. Can we get the output in a different format (e.g., CSV, API)? 4. What do PivotalRow and ExportIssOutputRow represent? 5. How do we ensure data consistency if the process fails mid-write?

**Quick Reference** \* **OD:** Optimum Depth. \* **OW:** Optimum Width. \* **ISS:** Ideal Size Set. \* **ROS:** Rate of Sale. \* **AG:** Attribute Group.

## 7.6. ISSChannelAgSizeContri

**Module Name & Business Objective** The ISSChannelAgSizeContri module calculates the final size contribution at the channel-Attribute Group (AG) level. It uses raw sales data and the pivotal output from previous ISS modules to determine interim and final size contributions, which are then exported for analysis. *Its objective is to provide an aggregated view of size importance specifically tailored for channel-level analysis and planning, complementing the store-AG specific ISS outputs.*

**Required Information** \* **Sales Data:** \* Raw productSales data (from ISSPrepareSalesModule via cache). \* **Product Data:** \* Product SKU information (linking to size, AG, channel). \* **Operational Data:** \* Persisted PivotalRow entities (output from ISSOutputModule). \* Cache containing productSalesList.

**Business Settings** \* `channel.ag.contribution.export.enabled`: Boolean flag to enable or disable this module's execution. \* `channel.ag.output.table.name`: Database table name for channel-AG contribution output.

### Process Flow

graph TD

```
A[Raw Product Sales (from Cache)] --> B[Aggregate Sales by Channel-AG-Size];
C[PivotalRow entities (from DB)] --> D[Join with Aggregated Sales];
B & D --> E[Calculate Interim & Final Size Contributions];
E --> F[Export Channel-AG Size Contributions];
```

**Step-by-Step Actions** 1. **Retrieve Raw Sales:** Load raw productSales data from the internal cache. *This ensures all relevant sales are available for channel-level aggregation.* 2. **Retrieve Pivotal Data:** Query the database to retrieve the PivotalRow entities generated by the ISSOutputModule. *These provide the core ISS tags (P, NP, E) and size-specific sales details.* 3. **Aggregate Sales by Channel-AG-Size:** Group the raw sales data by channel, attributeGroup, and size, calculating total quantities for each combination. *This creates the basis for channel-specific contribution analysis.* 4. **Calculate Interim Contributions:** For each

channel-AG, **determine** the initial contribution of each size based on its raw sales within that channel-AG combination. *This provides a first pass at size importance at the channel level.* 5. **Calculate Final Contributions:** **Adjust** or **combine** the interim contributions with the pivotal tags from the PivotalRow data to derive a final size contribution metric at the channel-AG level. This might involve applying weighting or ensuring consistency with core ISS logic. *This reconciles raw sales with the ISS recommendations.* 6. **Export Results:** **Persist** the calculated channel-AG size contributions to a designated database table or file. *This makes the channel-specific size insights available for channel managers, marketing teams, or channel-specific planning tools.*

**Sample Scenario** For an AG: Women's Sandals across Online and Retail channels: \* Retail Channel Sales: Size 7 (50 units), Size 8 (40 units), Size 6 (20 units). \* Online Channel Sales: Size 7 (80 units), Size 8 (60 units), Size 9 (30 units). \* PivotalRow for StoreA (Retail) shows Size 7, 8 as P. PivotalRow for OnlineRegion shows Size 7, 8, 9 as P. \* **Calculation:** The module combines these to derive Channel-AG contributions. For Online-Women's Sandals, Size 7 contributes  $(80/170)=47\%$ , Size 8  $(60/170)=35\%$ , Size 9  $(30/170)=18\%$ . It then uses the pivotal data to confirm if 7, 8, 9 are indeed pivotal for the online channel context. Key Takeaway: *This module provides a crucial aggregated view of size performance across channels, allowing for differentiated inventory and merchandising strategies per channel.*

**Results** \* Populated database table with channel-Attribute Group-size contributions.  
\* Data ready for channel-specific reporting, analytics, and planning.

**Unusual Cases** \* **Missing or inconsistent pivotal output:** If PivotalRow data is unavailable or incomplete, the module cannot perform full reconciliation. \* **Data latency:** Differences in when sales data is available vs. when pivotal data is generated can cause discrepancies. \* **Channel definitions:** Inconsistent channel definitions between raw sales and pivotal output can lead to data misalignment. \* **Low channel sales:** Channels with very low sales volume for an AG may not yield meaningful contributions.

**Next Steps** The generated channel-AG size contributions are consumed by reporting tools and strategic channel planning systems.

**Common Questions** 1. How do channel-level contributions differ from store-level ISS results? 2. Can this module run independently or does it rely on all previous ISS steps? 3. How do we ensure the channel definitions are consistent? 4. What are the key decisions we can make with this channel-specific data? 5. Is it possible to also calculate regional-AG contributions?

**Quick Reference** \* **OD:** Optimum Depth. \* **OW:** Optimum Width. \* **ISS:** Ideal Size Set. \* **ROS:** Rate of Sale. \* **AG:** Attribute Group.

## 8. Appendix

### 8.1. Full Parameter Reference

- project\_name: iss
- version: 1.0
- current\_date: 2025-05-23

- `sales.duration.weeks`: Integer, default 13. Number of weeks of sales data to analyze.
- `iss.pivotal.contribution.min`: Double, default 0.05. Minimum percentage contribution for a size to be pivotal.
- `iss.enable.size.continuity`: Boolean, default true. Activates/deactivates size continuity logic.
- `iss.output.to.database`: Boolean, default true. Controls persistence of final results to database.
- `app.debug.mode`: Boolean, default false. Enables detailed application logging.
- `data.source.type`: String, default “database”. Specifies the primary data source type.
- `flagging.rollup.levels`: Comma-separated string, e.g., “store-AG,channel-AG”. Defines how sales data is grouped for flagging.
- `flagging.minimum.sales.threshold`: Integer, default 100. Minimum total sales units for an AG to be flagged.
- `flagging.minimum.stores.threshold`: Integer, default 1. Minimum stores an AG must be sold in.
- `pivotal.quantity.percentage.threshold`: Double, default 0.25. Threshold for a size’s sales contribution to be considered pivotal.
- `non.pivotal.quantity.percentage.threshold`: Double, default 0.10. Max threshold for non-pivotal.
- `exit.quantity.percentage.threshold`: Double, default 0.05. Max threshold for exit sizes.
- `size.continuity.buffer.sizes`: Integer, default 0. Number of buffer sizes for continuity.
- `output.table.pivotal.name`: String, default “ISS\_PIVOTAL\_OUTPUT”. Database table for PivotalRow.
- `output.table.export.name`: String, default “ISS\_EXPORT\_OUTPUT”. Database table for ExportIssOutputRow.
- `channel.ag.contribution.export.enabled`: Boolean, default true. Enables/disables channel-AG contribution module.
- `channel.ag.output.table.name`: String, default “ISS\_CHANNEL\_AG\_CONTRI”. Database table for channel-AG contribution output.

## 8.2. Version History

- **1.0 (2025-05-23)**: Initial release of the Ideal Size Set (ISS) pipeline documentation.

## 8.3. Contact & Support Info

For any questions, issues, or support requests regarding the ISS application, please contact:

- **Email**: [support@yourcompany.com](mailto:support@yourcompany.com)
- **Internal Help Desk**: [Link to internal help desk system]
- **Project Lead**: [Name of Project Lead / Team Lead]
- **Documentation Feedback**: [documentation@yourcompany.com](mailto:documentation@yourcompany.com) ``

# Tutorial: iss

The Ideal Size Set (ISS) calculation involves a sequential pipeline of modules. The process begins with `ISSPrepareSalesModule` which filters and aggregates sales data. This prepared data is then progressively processed by `ISSFlaggingModule`, `ISSTaggingModule`, and `ISSSizeContinuityModule` to determine and refine pivotal sizes. Finally, `ISSOutputModule` consolidates and persists the results, while `ISSChannelAgSizeContri` uses the output of this core pipeline and raw sales data to calculate final size contributions.

flowchart TD

```
A0["ISSPrepareSalesModule"]
A1["ISSFlaggingModule"]
A2["ISSTaggingModule"]
A3["ISSSizeContinuityModule"]
A4["ISSOutputModule"]
A5["ISSChannelAgSizeContri"]
A0 -- "data_flow" --> A1
A1 -- "data_flow" --> A2
A2 -- "data_flow" --> A3
A3 -- "data_flow" --> A4
A0 -- "data_flow" --> A4
A4 -- "data_flow" --> A5
A0 -- "data_flow" --> A5
```

## Chapters

Here is the comprehensive Master Report for the merchandising algorithm.

---

# Master Report: Ideal Size Set (ISS) Merchandising Algorithm

## 1. Title & Metadata

- **Project Name:** iss
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** An advanced algorithm designed to optimize product assortment (OD, OW) and size distribution (ISS) to maximize sales velocity (ROS) and minimize inventory costs across categories.

## 2. Executive Summary

The iss merchandising algorithm is a critical tool for strategic inventory management, providing data-driven recommendations for product breadth, depth, and size assortment. It leverages historical sales data, inventory levels, and product attributes to calculate optimal merchandising parameters, ensuring that the right products are available in the right sizes at the right time. By streamlining the product selection and sizing process, this algorithm empowers merchandising teams to enhance customer satisfaction, reduce stockouts and overstock situations, and ultimately drive revenue growth.

- **Key outcomes or KPIs:**
  - Increased ROS for recommended products.
  - Optimized inventory turnover and reduced holding costs.
  - Improved in-stock rates for high-demand sizes.
  - Enhanced category profitability by aligning stock with demand.
- **Primary stakeholders or downstream consumers:** Merchandising Planners, Inventory Management Teams, Category Managers, Supply Chain Operations, Finance/Retail Analytics.

## 3. Architecture & Data Flow

graph TD  
A[Raw Sales Data] --> B{Data Ingestion & Preprocessing};  
B --> C[Cleaned Sales & Inventory Data];  
C --> D[ROS Calculation Engine];  
C --> E[Attribute Group Definition];  
D --> F[OD/OW Determination Module];



```

E --> F;
C --> G[ISS Optimization Engine];
D --> G;
F --> H[Recommendation & Reporting Generator];
G --> H;
H --> I[Merchandising Recommendations];
H --> J[Performance Dashboards];

```

```

subgraph Core Algorithm
 D -- ROS per SKU & AG --> F;
 G -- Size Distribution per AG --> H;
end

```

- **Component Bullets:**

- **Data Ingestion & Preprocessing:** Collects, validates, cleans, and transforms raw sales, inventory, and product master data into a standardized format.
- **ROS Calculation Engine:** Computes Rate of Sale at various granularities (SKU, AG, category) based on specified look-back periods.
- **Attribute Group Definition:** Dynamically identifies or loads predefined Attribute Groups (AGs) crucial for analysis (e.g., Category-Gender-Material).
- **OD/OW Determination Module:** Utilizes ROS and market trends to recommend the Optimum Depth (OD) of items within a category and Optimum Width (OW) of categories/styles to offer.
- **ISS Optimization Engine:** Analyzes historical sales distributions and ROS for specific AGs to generate the Ideal Size Set (ISS) for each product/style.
- **Recommendation & Reporting Generator:** Consolidates outputs from all modules into actionable reports and recommendations for merchandising teams.
- **Merchandising Recommendations:** Final output files containing specific product assortment and sizing guidance.
- **Performance Dashboards:** Visualizations and summaries for monitoring algorithm effectiveness and business KPIs.

- **High-level data sources:**

- Point-of-Sale (POS) transaction data
- Inventory management systems (current stock levels, inbound shipments)
- Product Master Data (SKU attributes like size, color, style, category)
- Historical promotional data (optional, for demand shaping context)

## 4. Configuration & Data Dictionary

- **Settings Table** (args.properties):

Setting Key	Description	Default Value	Example Value
ros_lookback_weeks	Number of weeks of historical sales data to use for ROS calculation	12	8
min_units_for_ros	Minimum units sold over	10	5

Setting Key	Description	Default Value	Example Value
	ros_lookback_weeks for a valid ROS calculation		
target_stockout_risk_pct	Acceptable percentage risk of a size stockout when determining ISS	0.05	0.03
od_min_ros_threshold	Minimum ROS (units/week) for a SKU to be considered for Optimum Depth (OD)	0.5	0.75
ow_category_growth_factor	Multiplier applied to category growth forecasts for Optimum Width (OW)	1.1	1.05
category_exclusion_list	Comma-separated list of categories to exclude from algorithm processing	(empty)	“Services,Gift Cards”
iss_granularity_level	Defines the AG level for ISS calculation (e.g., “Category-Style”, “SKU”)	“Category-Style”	“SKU”

• **Shared Data Fields** (common-data.json):

Field Name	Data Type	Description	Example Value
product_id	String	Unique identifier for a specific product item (SKU)	“P12345-BLU-M”
category	String	High-level product grouping (e.g., “Apparel”, “Footwear”)	“Men’s Tops”
sub_category	String	More granular product grouping (e.g., “T-Shirts”, “Jeans”)	“Basic T-Shirts”
style_id	String	Identifier for a product style, irrespective of color/size	“ST001”
color	String	Product color attribute	“Black”
size	String	Product size attribute (e.g., “S”, “M”)	“M”

Field Name	Data Type	Description	Example Value
		“M”, “L”, “XL”, “32x30”)	
week_start_date	Date	Start date of the sales aggregation week	“2024-01-01”
units_sold_weekly	Integer	Total units sold for a product_id within a given week	15
units_on_hand_eow	Integer	Units remaining in stock for a product_id at end of week	50
average_retail_price	Decimal	Average retail price of the product over the week	24.99
attribute_group_id	String	Concatenated identifier for an AG (e.g., “Mens-Tops-Cotton”)	“Mens-Tops_Basic_Cotton”

• **Outputs Catalog:**

Output File Name	Format	Description
ideal_size_set_recommendations.csv	CSV	Detailed recommendations for ISS per AG/Style, including size percentages.
optimum_depth_summary.csv	CSV	Summary of recommended OD per category/sub-category, with justification.
optimum_width_by_category.csv	CSV	Recommended OW (categories/styles to prioritize), based on market trends.
ros_by_sku_ag.csv	CSV	Calculated ROS for each SKU and Attribute Group (AG).
inventory_health_report.pdf	PDF	A summary report on current inventory health against ideal targets.

## 5. End-to-End Flow

The iss algorithm operates through a structured pipeline to transform raw data into actionable merchandising insights:

1. **Data Ingestion:** The process begins by pulling raw sales transactions, current inventory levels, and product master data from various internal systems.
2. **Data Preprocessing:** Raw data is cleaned, validated, and normalized. This includes handling missing values, standardizing date formats, and mapping product attributes to predefined schemas. `attribute_group_ids` are generated here.
3. **Rate of Sale (ROS) Calculation:** For each unique `product_id` and identified `attribute_group_id`, the weekly ROS is calculated based on historical `units_sold_weekly` over the `ros_lookback_weeks` period. Products not meeting `min_units_for_ros` are flagged or excluded.
4. **Optimum Depth (OD) & Optimum Width (OW) Determination:**
  - **OD:** Based on the ROS of individual SKUs and category performance, the algorithm determines the ideal number of unique items (SKUs) to stock within a category, prioritizing high-ROS items.
  - **OW:** Analyzing category-level ROS, market trends, and growth factors, the algorithm identifies the optimal number of different categories or styles to offer, ensuring diversification and market relevance.
5. **Ideal Size Set (ISS) Optimization:**
  - For each `attribute_group_id` (or `style_id` as configured), the historical sales distribution across all sizes is analyzed.
  - Statistical methods (e.g., cumulative distribution functions, moving averages) are applied to determine the percentage of total sales attributed to each size within that group.
  - The `target_stockout_risk_pct` is used to ensure a balanced stock of common and less common sizes, minimizing potential stockouts while optimizing inventory.
  - The output is a recommended percentage split for each size within the AG.
6. **Recommendation Generation:** All calculated metrics (ROS, OD, OW, ISS) are compiled into structured output files, providing clear, actionable recommendations for merchandising teams.
7. **Reporting & Visualization:** Key metrics and recommendations are summarized in user-friendly reports and dashboards for strategic planning and performance monitoring.

## 6. Submodule Overview

### 6.1. Data Ingestion & Preprocessing

1. **Name & Purpose:** Standardizes, cleans, and transforms raw sales, inventory, and product data into a consistent format for downstream analysis.
2. **Inputs & Outputs:**
  - **Inputs:**
    - `raw_pos_transactions.json` (daily sales per SKU)
    - `raw_inventory_snapshots.csv` (weekly inventory per SKU)
    - `product_master.xlsx` (SKU attributes)

- **Outputs:**
  - `cleaned_sales_inventory.parquet` (weekly aggregated sales and EOW inventory per SKU)
  - `product_attributes_processed.parquet` (SKU attributes with derived AGs)
  - `data_quality_log.txt` (records of issues and transformations)
- 3. **Core Logic Summary:**
  - Reads and parses data from various sources, handling different file formats.
  - Performs data type conversions and validates key identifiers (e.g., `product_id`).
  - Aggregates daily sales to weekly `units_sold_weekly` and captures `units_on_hand_eow`.
  - Derives `attribute_group_id` by concatenating specified product attributes (e.g., category, sub\_category, color\_family).

## 6.2. ROS Calculation Engine

1. **Name & Purpose:** Calculates the Rate of Sale (ROS) for individual SKUs and aggregated Attribute Groups (AGs) over defined historical periods.
2. **Inputs & Outputs:**
  - **Inputs:**
    - `cleaned_sales_inventory.parquet`
    - `product_attributes_processed.parquet`
    - `ros_lookback_weeks` (from `args.properties`)
    - `min_units_for_ros` (from `args.properties`)
  - **Outputs:**
    - `ros_by_sku_ag.csv` (contains `product_id`, `attribute_group_id`, `avg_weekly_ros`)
3. **Core Logic Summary:**
  - Filters `cleaned_sales_inventory.parquet` to include only data within the `ros_lookback_weeks` window.
  - Sums `units_sold_weekly` for each `product_id` over the look-back period.
  - Divides total units sold by the number of weeks in the look-back period to get `avg_weekly_ros`.
  - Aggregates `avg_weekly_ros` for each `attribute_group_id` by averaging ROS of contained SKUs, only considering those that meet `min_units_for_ros`.

## 6.3. OD/OW Determination Module

1. **Name & Purpose:** Recommends the Optimum Depth (OD) of items within a category and Optimum Width (OW) of categories/styles to offer, based on sales performance and configured thresholds.
2. **Inputs & Outputs:**
  - **Inputs:**
    - `ros_by_sku_ag.csv`
    - `product_attributes_processed.parquet`
    - `od_min_ros_threshold` (from `args.properties`)
    - `ow_category_growth_factor` (from `args.properties`)
  - **Outputs:**
    - `optimum_depth_summary.csv`
    - `optimum_width_by_category.csv`

### 3. Core Logic Summary:

- **OD:** For each category/sub-category, it identifies SKUs whose `avg_weekly_ros` exceeds the `od_min_ros_threshold`. The count of such SKUs determines the recommended OD.
- **OD:** Ranks SKUs within categories by ROS and suggests retaining the top N items that cumulatively contribute to X% of category sales.
- **OW:** Aggregates ROS at the category level. Categories showing high ROS or strong growth potential (factored by `ow_category_growth_factor`) are prioritized for increased width.
- **OW:** Incorporates external market trend data (if available) to inform diversification and new category introductions.

## 6.4. ISS Optimization Engine

1. **Name & Purpose:** Calculates the Ideal Size Set (ISS) for each Attribute Group (AG) or Style by analyzing historical sales distribution across sizes.

2. **Inputs & Outputs:**

- **Inputs:**
  - `cleaned_sales_inventory.parquet`
  - `product_attributes_processed.parquet`
  - `iss_granularity_level` (from `args.properties`)
  - `target_stockout_risk_pct` (from `args.properties`)
- **Outputs:**
  - `ideal_size_set_recommendations.csv`

3. **Core Logic Summary:**

- Groups `cleaned_sales_inventory.parquet` by the `iss_granularity_level` (e.g., `attribute_group_id` or `style_id`) and size.
- Calculates the total sales for each size within each group over the historical period.
- Determines the percentage of total group sales contributed by each size (e.g., S: 10%, M: 30%, L: 35%, XL: 25%).
- Applies a statistical model (e.g., Gaussian distribution fitting or empirical cumulative distribution) to smooth out historical noise and recommend robust size percentages, adjusting for `target_stockout_risk_pct` to slightly overweight high-demand sizes.

## 6.5. Recommendation & Reporting Generator

1. **Name & Purpose:** Consolidates all calculated metrics and optimized parameters into user-friendly reports and recommendation files for merchandising execution.

2. **Inputs & Outputs:**

- **Inputs:**
  - `ros_by_sku_ag.csv`
  - `ideal_size_set_recommendations.csv`
  - `optimum_depth_summary.csv`
  - `optimum_width_by_category.csv`
  - `product_attributes_processed.parquet`
- **Outputs:**
  - `merchandising_dashboard.pdf` (for stakeholders)
  - `ideal_size_set_recommendations.csv` (final version for execution)

- `assortment_plan_summary.csv` (combining OD/OW guidance)

### 3. Core Logic Summary:

- Joins the outputs from the ROS, OD/OW, and ISS modules with product attributes.
- Formats the data into clear, concise tables suitable for direct import into planning systems or human review.
- Generates summary statistics and visualizations for the PDF report, highlighting key recommendations, performance metrics, and potential impacts.
- Includes caveats and confidence scores for recommendations where appropriate.

## 7. Worked Example

Let's trace the ISS calculation for a hypothetical Attribute Group: **"Men's Basic Crew Neck T-Shirts - Cotton"** (**AG\_ID: MBCNTC**)

**Scenario:** We want to determine the Ideal Size Set (ISS) for AG\_ID MBCNTC based on `ros_lookback_weeks = 12`, `target_stockout_risk_pct = 0.05`, and `iss_granularity_level = "Attribute Group"`.

### 1. Data Ingestion & Preprocessing Output (partial `cleaned_sales_inventory.parquet`):

```
| week_start_date | attribute_group_id | size | units_sold_weekly |
|:-----|:-----|:---|:-----|
2024-02-01 | MBCNTC | S | 10 | 2024-02-01 | MBCNTC | M | 35 | 2024-02-01 |
MBCNTC | L | 40 | 2024-02-01 | MBCNTC | XL | 15 | ... (11 more weeks for this
AG) ... | 2024-04-22 | MBCNTC | S | 12 | 2024-04-22 | MBCNTC | M | 38 |
2024-04-22 | MBCNTC | L | 37 | 2024-04-22 | MBCNTC | XL | 13 |
```

**2. ROS Calculation Engine (for context, not direct ISS input):** Let's assume the average weekly ROS for MBCNTC over 12 weeks is calculated as 105 units/week.

**3. ISS Optimization Engine Logic:**

- \* **Step 1: Aggregate Sales by Size for AG\_ID 'MBCNTC' over 12 weeks:**
  - \* Size S: Total 120 units sold
  - \* Size M: Total 420 units sold
  - \* Size L: Total 450 units sold
  - \* Size XL: Total 180 units sold
  - \* Total units sold for AG\_ID MBCNTC = 120 + 420 + 450 + 180 = 1170 units.

- **Step 2: Calculate Raw Size Distribution Percentage:**

- S:  $(120 / 1170) * 100\% = 10.26\%$
- M:  $(420 / 1170) * 100\% = 35.90\%$
- L:  $(450 / 1170) * 100\% = 38.46\%$
- XL:  $(180 / 1170) * 100\% = 15.38\%$

- **Step 3: Apply `target_stockout_risk_pct` (e.g., 0.05)**

- The algorithm adjusts the raw percentages to create a slightly more robust stock, particularly for frequently sold sizes, to minimize the 5% stockout risk. This often involves smoothing or a slight upward adjustment for mid-range sizes.
- *Simplified Adjustment Example (for illustrative purposes - actual might use more complex statistical models):*
  - Distribute the 5% risk tolerance proportionally to the higher demand sizes, slightly increasing their recommended share. Or, ensure cumulative distribution reaches 95% at a certain size.

- Let's assume the adjusted distribution after factoring in `target_stockout_risk_pct` (e.g., by ensuring a certain cumulative sales coverage or smoothing) is:
  - S: 10%
  - M: 36%
  - L: 38%
  - XL: 16%
  - (Total 100%)

#### 4. Recommendation & Reporting Generator Output (partial `ideal_size_set_recommendations.csv`):

<code>attribute_group_id</code>	<code>size</code>	<code>recommended_iss_pct</code>	<code>calculated_ros_ag</code>
MBCNTC	S	0.10	105
MBCNTC	M	0.36	105
MBCNTC	L	0.38	105
MBCNTC	XL	0.16	105
... other AGs ...			

This table provides direct guidance for merchandising teams to allocate inventory for “Men’s Basic Crew Neck T-Shirts - Cotton” based on their historical sales patterns and a calculated risk tolerance.

## 8. Debug Guide Highlights

- **No ROS for a SKU/AG:**
  - **Check:** Is the SKU/AG present in `cleaned_sales_inventory.parquet` within the `ros_lookback_weeks`?
  - **Check:** Does it meet the `min_units_for_ros` threshold?
  - **Action:** Review input data completeness and adjust `ros_lookback_weeks` or `min_units_for_ros` if sales volume is genuinely low.
- **Unexpected ISS percentages (e.g., heavily skewed to one size):**
  - **Check:** Is `iss_granularity_level` correctly set? If it's too broad, diverse styles might be averaged. If too narrow, insufficient sales history might exist.
  - **Check:** Are there data quality issues in size attribute (e.g., "M " vs. "M")?
  - **Check:** Is `target_stockout_risk_pct` too low, leading to over-optimization on historical patterns?
- **Module failure/Error logs:**
  - **Check:** Review `data_quality_log.txt` for preprocessing errors.
  - **Check:** Examine specific module logs for detailed stack traces. Often due to malformed input files or missing required fields.
- **Output files not generated or empty:**
  - **Check:** Verify output directory permissions.
  - **Check:** Confirm previous modules successfully ran and produced their expected outputs.
  - **Check:** Are there any filter conditions (e.g., `category_exclusion_list`) inadvertently excluding all data?



## 9. Common Pitfalls & Remedies

- **Pitfall: “Cold Start” Problem for New Products/Categories:** New items have no historical sales, leading to invalid ROS/ISS calculations.
  - **Remedy:** Implement a fallback mechanism: for new items, use ISS/ROS of a similar `attribute_group_id` (e.g., best-selling “Men’s Basic T-Shirt” if the new item is a men’s basic tee). Initial stock can be based on market benchmarks.
- **Pitfall: Impact of Promotions/Seasonality:** ROS can be artificially inflated/deflated by promotions or strong seasonal peaks/troughs, distorting baseline demand.
  - **Remedy:** Incorporate promotion flags in input data and adjust ROS calculation (e.g., exclude promotional weeks, apply a seasonal index, or use a de-seasonalized ROS). Use longer `ros_lookback_weeks` to smooth out short-term spikes.
- **Pitfall: Incorrect `attribute_group_id` Mapping:** Poorly defined or inconsistent `attribute_group_ids` can lead to “mixing apples and oranges” in AG calculations, yielding inaccurate ISS.
  - **Remedy:** Regular audit of `product_master` data and `attribute_group_id` derivation logic. Ensure `iss_granularity_level` is appropriate for the business context.
- **Pitfall: Data Latency/Incompleteness:** Using outdated or incomplete sales/inventory data leads to suboptimal recommendations.
  - **Remedy:** Establish robust data pipelines with clear SLAs for freshness. Implement data completeness checks at the start of the process.

## 10. FAQ

- **Q: How often should this algorithm be run?**
  - A: Typically weekly or bi-weekly for tactical reordering and inventory adjustments. Monthly for strategic assortment planning (OD/OW).
- **Q: Can this algorithm account for regional differences in demand?**
  - A: Yes, if the input data includes store/region identifiers, the algorithm can be configured to calculate ROS and ISS at a regional or store cluster level. This would require adjusting the `iss_granularity_level` and potentially adding a ‘region’ dimension to `attribute_group_id`.
- **Q: What if we don’t have enough historical data for a specific Attribute Group?**
  - A: If `min_units_for_ros` is not met, the algorithm will flag it. The default behavior is to use a higher-level AG (e.g., Category) or a global average as a fallback. For new product introductions, similar existing products’ ISS can be used.
- **Q: How does this algorithm handle returns?**
  - A: The `units_sold_weekly` input should ideally be net sales (gross sales minus returns) to accurately reflect true demand. If only gross sales are available, a returns factor might need to be applied externally.
- **Q: What is the primary difference between OD and OW?**
  - A: OW (Optimum Width) refers to the ideal *variety of categories or styles* offered (e.g., “Should we offer more casual wear or formal wear?”). OD (Optimum Depth) refers to the ideal *number of unique items within a specific category or style* (e.g., “How many different colors of basic t-shirts should we stock?”).

# 11. Quick Reference Cheat-Sheet

Term	Definition	Key Metric/Output
<b>OD</b>	Optimum Depth: Ideal number of unique items (SKUs) to stock in a category/display.	Count of recommended
<b>OW</b>	Optimum Width: Ideal number of different categories or styles to offer.	List of prioritized cate
<b>ISS</b>	Ideal Size Set: Best combination of sizes to maximize sales and minimize stockouts.	recommended_iss_pc an AG.
<b>ROS</b>	Rate of Sale: Speed at which products are sold (units/week).	avg_weekly_ros per p attribute_group_id.
<b>AG</b>	Attribute Group: Set of product attributes grouped for analysis.	attribute_group_id Cotton”).
ros_lookback_weeks	Config for historical sales lookback.	Parameter in args.pro
min_units_for_ros	Threshold for valid ROS calculation.	Parameter in args.pro
target_stockout_risk_pct	Acceptable risk of stockout for ISS calculation.	Parameter in args.pro
iss_granularity_level	Level at which ISS is calculated (e.g., AG or SKU).	Parameter in args.pro
<b>Primary Input</b>	Historical units_sold_weekly, units_on_hand_eow, product_id attributes.	cleaned_sales_inven product_attributes.
<b>Key Output</b>	ideal_size_set_recommendations.csv, optimum_depth_summary.csv, optimum_width_by_category.csv.	Actionable merchandis

Error: Empty response from Gemini API

# mfp FAQ

**Version:** 1.0 **Last Updated:** 2025-05-23

This FAQ document provides high-level answers to common business questions about the mfp (Merchandise Financial Planning) algorithm, its functionalities, and how it supports strategic merchandising decisions.

---

## General

**Q:** What is the overall purpose of the mfp algorithm? **A:** The mfp algorithm provides a robust framework for Merchandise Financial Planning, aiming to optimize inventory levels and financial performance by analyzing historical sales data and forecasting future needs.

**Q:** Which business areas does the mfp algorithm support? **A:** The mfp algorithm primarily supports Merchandising, Planning, and Supply Chain teams by providing foundational data and insights necessary for strategic planning, including determining Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) strategies.

**Q:** What are the main components of the mfp algorithm? **A:** The mfp algorithm is orchestrated by the MFP Group Module, which manages the overall flow. Its core component is the MFP Base Data Preparation Module, responsible for cleaning and preparing all historical sales and financial data.

**Q:** How does mfp help us achieve Optimum Depth (OD) and Optimum Width (OW)? **A:** While mfp itself doesn't directly output OD or OW, it provides the essential historical sales data, financial metrics, and time-series analysis capabilities that are crucial inputs for subsequent models or planning processes designed to determine the ideal number of unique items (OD) and different categories (OW).

**Q:** How often is the data processed by the mfp algorithm? **A:** The processing frequency for the mfp algorithm is configurable, typically aligned with business planning cycles (e.g., weekly, monthly) to ensure the most current data is available for planning.

---

## Configuration & Settings

**Q:** Can we customize the mfp algorithm's calculations or data inputs? **A:** Yes, the mfp algorithm is designed to be configurable. While specific settings are managed internally, the overall execution flow and data preparation steps can be adapted to align with specific business rules and data sources.

**Q:** Where are the configuration settings for the mfp algorithm typically managed?

**A:** Configuration settings for the mfp algorithm are typically managed through predefined properties or parameters that control the execution of modules like the MFP Group Module and MFP Base Data Preparation Module. For detailed changes, consult the system administrator.

**Q:** Can the data aggregation levels be customized for analysis? **A:** Yes, the MFP Base Data Preparation Module aggregates data across various Attribute Groups (AGs) such as channel, category, store, subcategory, year, and month. These aggregation levels are a fundamental part of the data preparation and can often be adjusted through configuration.

**Q:** What happens if I change a configuration setting? **A:** Changing a configuration setting can impact the algorithm's execution flow, data preparation, and ultimately the outputs used for Merchandise Financial Planning. Any changes should be made carefully and typically require re-running the relevant modules.

**Q:** Are there default settings if we don't specify custom ones? **A:** Yes, the mfp algorithm operates with a set of default configurations that ensure a standard and comprehensive data preparation process for Merchandise Financial Planning if no specific custom settings are provided.

---

## **Data & Inputs**

**Q:** What kind of data does the mfp algorithm require as input? **A:** The mfp algorithm primarily requires historical sales and net sales data, including detailed transactional information, cost data, and pricing data, to calculate essential financial metrics.

**Q:** What financial metrics does the MFP Base Data Preparation Module calculate? **A:** The MFP Base Data Preparation Module calculates key metrics such as Gross Sales Value (GSV), quantity sold, discount amounts, Cost of Goods Sold (COGS), Average Selling Price (ASP), profit margins, and Manufacturer's Suggested Retail Price (MRP).

**Q:** How does the mfp algorithm handle months with no sales activity for a specific product or category? **A:** The MFP Base Data Preparation Module is designed to ensure comprehensive data coverage. It will create records even for months where no sales activity was recorded for a given Attribute Group (AG) combination, which is crucial for accurate time-series analysis and seasonality factor calculation.

**Q:** What are Attribute Groups (AGs) and why are they important in mfp? **A:** Attribute Groups (AGs) are sets of product attributes grouped for analysis (e.g., color, size, brand, channel, category, store, subcategory, year, month). They are crucial in mfp for segmenting and analyzing historical sales and financial data, enabling more granular planning and insights into Rate of Sale (ROS) and inventory needs.

**Q:** Is the input data validated for quality? **A:** The MFP Base Data Preparation Module focuses on preparing and aggregating data into structured formats, which inherently involves some level of data structuring and consolidation to ensure consistency for subsequent analysis.

---

## Process & Logic

**Q:** What is the role of the MFP Group Module? **A:** The MFP Group Module acts as the primary orchestrator. It manages the entire execution flow for the mfp project, including data preparation, output snapshotting for historical records, and synchronization of generated outputs, ensuring a structured process.

**Q:** What is the role of the MFP Base Data Preparation Module? **A:** The MFP Base Data Preparation Module is responsible for preparing and aggregating all historical sales and net sales data. It calculates critical financial and sales metrics and ensures complete data coverage across all relevant Attribute Group (AG) combinations and periods.

**Q:** How does the mfp algorithm use historical data for future planning? **A:** The MFP Base Data Preparation Module prepares the historical data in a format suitable for time-series analysis and seasonality factor calculation. This enables the algorithm to identify trends and patterns in past Rate of Sale (ROS), which are then used as a foundation for future Merchandise Financial Planning.

**Q:** Does the algorithm perform forecasting internally? **A:** The MFP Base Data Preparation Module prepares the *foundational* data for time-series analysis and seasonality, which are key inputs for forecasting. While it doesn't perform the final sales forecast itself, it sets the stage for accurate forecasting in subsequent planning tools or models.

**Q:** What does "output snapshotting" mean in the context of mfp? **A:** Output snapshotting, managed by the MFP Group Module, refers to saving a historical copy of the generated outputs. This allows for auditing, comparison over time, and maintaining a historical record of planning iterations.

---

## Outputs & Results

**Q:** What kind of outputs does the mfp algorithm produce? **A:** The mfp algorithm produces structured datasets containing aggregated historical sales and financial metrics, clean time-series data, and calculated seasonality factors, all essential for subsequent Merchandise Financial Planning, including supporting decisions on Optimum Depth (OD) and Optimum Width (OW).

**Q:** How are the results of the mfp algorithm typically used by business users? **A:** The results are used as the foundational data layer for planners and merchandisers to make informed decisions on inventory buys, assortment planning, setting Ideal Size Sets (ISS), and optimizing Rate of Sale (ROS) targets across categories and channels.

**Q:** Can we access historical versions of the mfp outputs? **A:** Yes, the MFP Group Module performs output snapshotting, which means historical records of the generated outputs are maintained and can be accessed for reference or comparison.

**Q:** What does "synchronization of generated outputs" refer to? **A:** This refers to the process managed by the MFP Group Module to ensure that the outputs generated by the algorithm are correctly integrated and made available to downstream systems or reporting tools, ensuring data consistency across the planning ecosystem.

**Q:** How do the outputs help in determining the Ideal Size Set (ISS)? **A:** While the mfp algorithm does not directly output the Ideal Size Set (ISS), its detailed historical sales data, broken down by various Attribute Groups (AGs) and including quantity sold and Rate of Sale (ROS), provides the essential raw material for analysis needed to determine the best combination of sizes.

---

## **Error Handling & Edge Cases**

**Q:** How does the mfp algorithm handle errors during processing? **A:** The mfp algorithm has built-in mechanisms for error detection and logging. If an error occurs during data preparation or processing, it typically halts execution and logs the details to help identify and resolve the issue.

**Q:** What happens if there's incomplete historical data for a specific Attribute Group (AG)? **A:** The MFP Base Data Preparation Module is designed to manage this. It accounts for periods with no sales activity by creating comprehensive data coverage, ensuring that gaps don't disrupt time-series analysis or seasonality calculations. However, significant structural gaps might require manual intervention.

**Q:** What should I do if the mfp output appears incorrect or unexpected? **A:** If the output appears incorrect, first review the input data for any discrepancies. Then, consult the configuration settings to ensure they align with expectations. For persistent issues, contact the support team with details of the unexpected output and any relevant input data.

**Q:** Are there any known limitations or specific scenarios the algorithm might struggle with? **A:** The current mfp algorithm is designed for robust data preparation and historical analysis. While it handles missing sales data well, extreme outliers or highly volatile sales patterns might require additional review in downstream forecasting models that consume mfp's outputs.

---

## **Need More Help?**

For further assistance or detailed inquiries, please contact the support team.

# mfp Documentation

Version: 1.0 Last Updated: 2025-05-23

## 2. Executive Summary

The Merchandise Financial Planning (MFP) algorithm is designed to optimize inventory and sales planning across various product categories. By leveraging historical sales data and configuring key business parameters, it calculates optimal depth and width for product assortments, ensuring maximum sales and minimal stockouts. This system empowers businesses to make data-driven decisions, leading to improved profitability and efficient resource allocation.

## 3. Architecture Overview

graph TD

A[MFP Group Module] -->|executes| B[MFP Base Data Preparation Module]

A -->|configures| B

- **MFP Group Module:** Orchestrates the entire Merchandise Financial Planning (MFP) process, managing the execution flow and configuration of all sub-modules.
- **MFP Base Data Preparation Module:** Aggregates and transforms raw historical sales data into a structured format, calculating essential financial metrics for subsequent analysis.

## 4. Prerequisites & Setup

To run the MFP pipeline, ensure the following prerequisites are met:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Apache Maven:** Version 3.6.3 or higher (or Gradle equivalent) for dependency management and building.
- **Database Connectivity:** Access to the source database containing sales and product master data (e.g., PostgreSQL, Oracle, SQL Server). Database drivers should be included in the project dependencies.

**Build Commands:** \* To clean and build the project: `mvn clean install` \* To skip tests during build: `mvn clean install -DskipTests`

**Running the Orchestrator:** The MFP Group Module serves as the main entry point. After building the project, navigate to the target directory and execute the JAR file:



```
java -jar mfp-1.0.jar --config-file=args.properties
```

Ensure args.properties and common-data.json are in the same directory or specified via a full path.

## 5. Folder & File Structure

```
mfp/
├─ module-group/ # Contains the orchestrator module
(MFP Group Module)
├─ submodules/ # Houses individual processing
modules (e.g., MFP Base Data Preparation Module)
├─ args.properties # Global configuration parameters for
the entire pipeline
├─ common-data.json # Schema definition for shared data
structures
└─ outputs/ # Directory where all generated
reports and data outputs are stored
```

## 6. Global Configuration

This table lists all configurable parameters for the entire MFP pipeline. These settings are defined in the args.properties file.

Setting Name	Default Value	Description	Valid Range/When to Change
<i>(No global configuration parameters were provided in the input for this section. In a real-world scenario, this table would list settings such as database connection details, date ranges for processing, output paths, etc.)</i>			

## 7. Shared Data Model

This section describes the common data structures used across various modules, as defined in `common-data.json`.

Field	Type	Description
<i>(No common data schema was provided in the input for this section. In a real-world scenario, this table would define the structure of shared data, e.g., SalesRecord, ProductAttribute, etc.)</i>		

## 8. Submodule Chapters

### MFP Group Module

#### 1. Module Name & Business Objective

**MFP Group Module:** Orchestrates the overall execution flow for the Merchandise Financial Planning (MFP) project. This module manages the sequence of operations, including data preparation, output snapshotting for historical records, and synchronization of generated outputs. It serves as the primary entry point for the MFP data processing pipeline, ensuring a structured and controlled execution of all related sub-modules.

#### 2. Required Information

- **Operational Data:**
  - **Execution Order:** The predefined sequence in which sub-modules must be executed.
  - **Configuration Parameters:** Global settings from `args.properties` that influence sub-module behavior (e.g., input/output paths, date ranges).

#### 3. Business Settings

- **execution.mode:** Defines whether the pipeline runs in full, incremental, or specific module-only mode.
- **output.snapshot.enabled:** Boolean flag to enable or disable the snapshotting of final outputs for historical tracking.

## 4. Process Flow

graph TD

```
Start --> A[Read Global Config]
A --> B[Execute Sub-Modules in Sequence]
B --> C[Capture Output Snapshot]
C --> D[Synchronize Outputs]
D --> End
```

## 5. Step-by-Step Actions

1. **Initialize:** *The module is activated, preparing to control the overall MFP process.*
2. **Load Global Configurations:** *It reads `args.properties` to understand system-wide settings like date ranges, output directories, and execution mode.*
3. **Orchestrate Sub-module Execution:** **Execute** each sub-module (e.g., MFP Base Data Preparation Module) in its predefined order, *ensuring dependencies are met and data flows correctly between stages.*
4. **Manage Output Snapshotting:** If enabled, **create** a historical snapshot of the processed data, *preserving a record of the state at the time of execution for audit or comparison.*
5. **Synchronize Outputs:** **Distribute** the final processed outputs to designated locations (e.g., data warehouses, reporting systems), *making the results available for downstream consumption.*

## 6. Sample Scenario

A retail company wants to run its monthly MFP process to plan for the upcoming quarter. The MFP Group Module is configured to run in full mode, processing all necessary data from the last 24 months, and `output.snapshot.enabled` is true. The module first initiates the MFP Base Data Preparation Module to gather and clean historical sales data. Once complete, it ensures the results are saved to a historical archive before pushing the final data to the main planning system. Key Takeaway: *The MFP Group Module ensures a reliable and complete execution of the entire planning cycle, from data preparation to output distribution and archival.*

## 7. Results

- Successful execution status of all sub-modules.
- Timestamped snapshots of final output data (if enabled).
- Logs detailing execution flow, errors, and warnings.

## 8. Unusual Cases

- **Missing or delayed data feed:** If a required upstream data source for a sub-module is unavailable, the MFP Group Module will log an error and potentially stop the entire pipeline, depending on configuration.
- **Sub-module failure:** If any sub-module encounters an unrecoverable error (e.g., database connection issues, data validation failures), the MFP Group Module will halt execution, preventing downstream modules from processing incomplete or incorrect data.

## 9. Next Steps

Once the MFP Group Module completes, the prepared and processed data is ready for consumption by:

- \* Business intelligence tools for reporting and dashboards.
- \* Financial planning systems for budgeting and forecasting.
- \* Subsequent analytical models for advanced optimizations (e.g., assortment planning, promotional effectiveness).

## 10. Common Questions

1. **“How do I rerun only a specific part of the process?”**: *You can adjust the `execution.mode` or specific module-related settings in `args.properties` to target a single sub-module run, if supported by the sub-module design.*
2. **“My run failed; where do I check the error?”**: *Review the execution logs generated by the MFP Group Module for detailed error messages and stack traces, typically found in the `logs` directory.*
3. **“Why are there old output files still present?”**: *The `output.snapshot.enabled` setting might be active, creating historical versions. You may need to configure retention policies or manually clean older snapshots.*
4. **“Can I schedule this to run automatically?”**: *Yes, the command-line execution allows easy integration with standard enterprise schedulers like Cron, Apache Airflow, or Windows Task Scheduler.*
5. **“What if my source data has changed after a run?”**: *You would typically trigger a new full or incremental run of the MFP Group Module to reprocess and reflect the updated source data, ensuring your outputs are current.*

## 11. Quick Reference

- **Role**: Orchestrates MFP pipeline.
  - **Input**: Global configs (`args.properties`), execution status from sub-modules.
  - **Output**: Execution logs, orchestrated sub-module runs, optional output snapshots.
  - **Key Action**: Ensures sequential execution and data flow.
  - **Entry Point**: `java -jar mfp-1.0.jar`
- 

## MFP Base Data Preparation Module

### 1. Module Name & Business Objective

**MFP Base Data Preparation Module**: Prepares and aggregates historical sales and net sales data into structured formats required for Merchandise Financial Planning (MFP). This module calculates key financial and sales metrics such as Gross Sales Value (GSV), quantity sold, discount amounts, Cost of Goods Sold (COGS), Average Selling Price (ASP), profit margins, and Manufacturer’s Suggested Retail Price (MRP). It also ensures comprehensive data coverage across all relevant historical periods and Attribute Group (AG) combinations (e.g., channel, category, store, subcategory, year, month), even for months where no sales activity was recorded. This comprehensive dataset is crucial for accurate time-series analysis, seasonality factor calculation, and other subsequent MFP calculations.

## 2. Required Information

- **Sales Data:**
  - **Raw Transactional Sales:** Detailed records including item ID, quantity, sales price, discount, date, store ID, channel.
  - **Returns Data:** Information on returned items affecting net sales.
- **Product Data:**
  - **Product Master Data:** SKU-level details including cost (COGS), Manufacturer's Suggested Retail Price (MRP), category, subcategory, brand, size.
  - **Attribute Group (AG) Definitions:** Mappings of product attributes to their respective groups (e.g., Color, Size, Brand within a Product AG).
- **Operational Data:**
  - **Calendar Data:** Mapping of dates to year, month, week for aggregation.
  - **Store/Channel Data:** Information about stores and sales channels.

## 3. Business Settings

- **data.preparation.history.months:** Number of historical months to include in the data aggregation (e.g., 24, 36).
- **data.preparation.fill.missing.data:** Boolean to enable or disable filling in zero sales for AG combinations with no activity in a period, *ensuring complete time-series for analysis*.

## 4. Process Flow

graph TD

```
A[Raw Sales Data] --> B[Calculate GSV & Quantity]
C[Product Master Data] --> B
B --> D[Calculate Discounts, COGS, ASP, Profit]
D --> E[Aggregate by AG & Time Period]
E --> F[Fill Missing AG-Month Combinations]
F --> G[Prepared Base Data]
```

## 5. Step-by-Step Actions

1. **Retrieve Raw Sales Data:** **Fetch** transactional sales records from the database for the specified historical period, *forming the foundation for calculations*.
2. **Calculate Gross Sales Value (GSV) & Quantity Sold:** **Sum** up sales values and quantities for each transaction, *providing initial sales metrics*.
3. **Incorporate Product Data:** **Join** with product master data to retrieve COGS, MRP, and relevant Attribute Groups (AGs) for each SKU, *enriching sales data with product cost and categorization*.
4. **Calculate Net Sales & Profit Metrics:** **Compute** discount amounts, net sales (GSV minus returns and discounts), Average Selling Price (ASP), profit margins, *deriving core financial performance indicators*.
5. **Aggregate by Attribute Group and Time Period:** **Group** the calculated metrics by specified AG combinations (e.g., Category, Channel, Month) and time periods (e.g., Year-Month), *structuring the data for higher-level analysis*.
6. **Fill Missing Data:** If enabled, **insert** records with zero sales for AG-month combinations where no actual sales occurred, *ensuring a complete and consistent time-series dataset for downstream modules like seasonality analysis*.

7. **Store Prepared Data:** Save the finalized, aggregated data into the designated output location, *making it accessible for subsequent MFP calculations.*

## 6. Sample Scenario

A fashion retailer needs to prepare data for its Spring 2026 assortment planning. The MFP Base Data Preparation Module is configured to analyze 24 months of historical data. For the 'Women's Tops' category, 'T-Shirts' subcategory in 'North Store', it aggregates monthly sales. In July 2025, 'T-Shirts' had 1,500 units sold with a GSV of \$30,000, discounts of \$3,000, and COGS of \$15,000. The module calculates a Net Sales of \$27,000, an ASP of \$18.00 ( $\$27,000 / 1,500$  units), and a Profit Margin of 44.4% ( $(\$27,000 - \$15,000) / \$27,000$ ). For 'Dresses' in the same store, which had no sales in February 2025, the module inserts a record with 0 units, 0 GSV, etc., *ensuring continuous time-series data for seasonality calculations.* Key Takeaway: *This module transforms raw sales data into actionable, aggregated metrics, vital for time-series analysis and comprehensive financial planning.*

## 7. Results

- Aggregated historical sales data by specified Attribute Groups (AGs) and time periods.
- Calculated metrics: GSV, Quantity Sold, Discount, Net Sales, COGS, ASP, Profit Margin.
- Complete time-series data, even for periods with no sales activity (if configured).

## 8. Unusual Cases

- **Missing or delayed data feed:** If raw sales or product master data is incomplete or late, the module will process the available data but may flag missing periods or attributes, potentially impacting the accuracy of aggregates.
- **Inconsistent Attribute Group mappings:** If product data has errors in category, subcategory, or other AG assignments, it can lead to skewed aggregations and inaccurate metrics for those groups.
- **Negative inventory or sales values:** Data quality issues can lead to negative numbers that are not logically possible (e.g., negative quantity sold). The module should have built-in validation or cleansing steps to handle such anomalies.

## 9. Next Steps

The prepared base data is the foundational input for subsequent MFP modules, including: \* **Seasonality Factor Calculation:** To identify cyclical sales patterns. \* **Forecast Generation:** To predict future sales based on historical trends. \* **Open-to-Buy (OTB) Planning:** To inform purchasing decisions. \* **Assortment Planning:** To guide decisions on OD, OW, and ISS.

## 10. Common Questions

1. **"Why is the data for a specific category showing zero for certain months?":**  
*This could be due to actual zero sales in that period, or it could indicate missing*

*raw sales data for that specific AG combination. Check the raw data source and the `data.preparation.fill.missing.data` setting.*

2. **“How far back does the data go?”**: *The historical depth is determined by the `data.preparation.history.months` configuration parameter. Ensure this matches your planning horizon.*
3. **“Why do my profit margins look off for certain products?”**: *Verify the accuracy of the COGS and MRP data in the product master file, as these directly influence profit calculations.*
4. **“Can I add new Attribute Groups for aggregation?”**: *Yes, provided the underlying raw data contains the necessary attributes, the module can be configured to aggregate by additional AGs, though it may require code changes.*
5. **“What happens if there’s an error in a single sales record?”**: *The module is designed to be robust. Individual record errors are typically logged and skipped, allowing the aggregation process to continue, but a summary of skipped records should be reviewed.*

## 11. Quick Reference

- **Role**: Transforms raw sales into structured, aggregated base data.
- **Input**: Raw sales transactions, product master data.
- **Output**: Aggregated sales metrics (GSV, Net Sales, ASP, etc.) by AG and time period.
- **Key Action**: Cleansing, calculating, and structuring historical sales data.
- **Business Value**: Provides a reliable, complete foundation for all subsequent MFP analysis.

## 9. Appendix

### Full Parameter Reference

Refer to the “Global Configuration” section for common parameters. Module-specific parameters are detailed within each submodule’s “Business Settings” section. For a complete list, consult the `args.properties` file in the codebase.

### Version History

- **1.0 (2025-05-23)**: Initial release of the Merchandise Financial Planning (MFP) pipeline, including core data preparation and orchestration capabilities.

### Contact & Support Info

For questions, support, or to report issues, please contact the development team: \*

**Email**: [support@yourcompany.com](mailto:support@yourcompany.com) \* **Internal Chat**: #mfp-support \*

**Documentation Link**: [Internal Confluence/SharePoint Link Here]

# Tutorial: mfp

The MfpGroupModule acts as the primary orchestrator for the Merchandise Financial Planning (MFP) process, directly executing and configuring the MfpPrepareBaseDataModule. The MfpPrepareBaseDataModule is responsible for preparing the foundational historical sales and financial data, which is crucial for subsequent MFP calculations.

flowchart TD

```
A0["MFP Group Module"]
A1["MFP Base Data Preparation Module"]
A0 -- "executes" --> A1
A0 -- "configures" --> A1
```

## Chapters



# Master Report: Merchandising Algorithm (mfp)

## 1. Title & Metadata

- **Project Name:** mfp
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** The mfp algorithm provides data-driven recommendations for **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)** to maximize sales and optimize inventory for retail merchandising.

## 2. Executive Summary

The mfp (Merchandising Forecast & Planning) algorithm is a foundational analytical tool designed to empower merchandising teams with precise, data-backed recommendations for product assortment planning and inventory optimization. By analyzing historical sales, stock levels, and product attributes, the algorithm generates actionable insights on the ideal number of unique items to stock (**OD**), the optimal breadth of categories/styles to offer (**OW**), and the most effective size combinations for specific products (**ISS**). It also provides crucial **Rate of Sale (ROS)** forecasts to inform replenishment strategies.

- **Key outcomes or KPIs:**
  - Increased Sales (units and revenue)
  - Improved Inventory Turn / Reduced Excess Inventory
  - Higher Gross Margins
  - Reduced Stockouts
  - Enhanced Customer Satisfaction (availability)
- **Primary stakeholders or downstream consumers:** Merchandising Planners, Buyers, Inventory Management, Store Operations, Finance/Analytics Teams.

## 3. Architecture & Data Flow

### Diagram

graph TD

```
A[Raw Data Sources] --> B(Data Ingestion);
B --> C{Data Preprocessing & Transformation};
C --> D[Historical Sales Data];
C --> E[Inventory Data];
```

```

C --> F[Product Master Data];
D & E & F --> G(OD & OW Calculation Module);
D & E & F --> H(ISS Optimization Module);
D & E & F --> I(ROS Forecasting Module);
G --> J(OD Recommendations);
G --> K(OW Recommendations);
H --> L(ISS Recommendations);
I --> M(ROS Predictions);
J & K & L & M --> N[Output Generation];
N --> O(Merchandising Reports);
N --> P(Inventory Planning Systems);
N --> Q(Retail Analytics Dashboards);

subgraph Merchandising Algorithm (mfp)
 G; H; I; N;
end

```

## Component Bullets

- **Data Ingestion:** Gathers raw data from various enterprise systems.
- **Data Preprocessing & Transformation:** Cleanses, validates, and standardizes raw data into a usable format, including feature engineering for specific models.
- **OD & OW Calculation Module:** Determines the ideal assortment width and depth based on sales performance, category dynamics, and capacity constraints.
- **ISS Optimization Module:** Analyzes sales by size to recommend the most efficient size curve for products.
- **ROS Forecasting Module:** Predicts future sales velocity for individual SKUs/styles.
- **Output Generation:** Compiles all recommendations and forecasts into structured outputs suitable for reporting and downstream systems.

## High-level data sources

- **POS (Point-of-Sale) Data:** Transactional sales data, including product, quantity, date, price, and store.
- **Inventory Management System:** Current stock levels, on-order quantities, in-transit inventory, and historical stock movements.
- **Product Master Data:** Product attributes (e.g., category, sub-category, brand, color, size, style, material), cost, retail price, and product lifecycle status.
- **Store/Location Data:** Store attributes, capacity, and performance metrics.
- **Promotional Calendars:** Information on past and planned marketing activities.

## 4. Configuration & Data Dictionary

### Settings Table (args.properties)

Parameter Name	Description	Default Value	Data Type
lookback_period_weeks		52	Integer

Parameter Name	Description	Default Value	Data Type
	Number of weeks of historical data to consider for analysis.		
min_sales_threshold	Minimum <b>ROS</b> for an item to be considered active for <b>OD</b> calculation.	5 units/week	Float
optimization_goal	Primary objective: MAX_SALES, MAX_MARGIN, MIN_INVENTORY.	MAX_SALES	String
size_curve_granularity	Granularity for <b>ISS</b> calculation: SKU, Style, Category.	Style	String
capacity_buffer_pct	Percentage buffer for store/display capacity constraints.	0.10 (10%)	Float
ros_forecast_horizon	Number of weeks into the future for <b>ROS</b> prediction.	12	Integer

### Shared Data Fields (common-data.json)

Field Name	Description	Data Type	Example Value	Source Module(s)
product_id	Unique identifier for a product SKU.	String	P12345-BLU-M	All
category_id	Identifier for the product category.	String	Womens_Dresses	All
style_id	Identifier for the product style (group of similar SKUs).	String	Floral_Maxi_Dress	All
size	Product size.	String	M, 32x32	All
sales_units_weekly	Historical weekly sales in units.	Float	15.75	Data Preprocessing
stock_on_hand	Current inventory units available.	Integer	20	Data Preprocessing
historical_ros	Calculated <b>Rate of Sale</b> over the lookback period.	Float	12.5 units/week	ROS Forecasting

Field Name	Description	Data Type	Example Value	Source Module(s)
od_recommendation	Recommended <b>Optimum Depth</b> (number of SKUs) for a category/style.	Integer	8	OD & OW Calc
ow_recommendation	Recommended <b>Optimum Width</b> (number of styles/categories) for a store/department.	Integer	5	OD & OW Calc
iss_recommendation	Recommended <b>Ideal Size Set</b> distribution (e.g., {S:1, M:2, L:1}).	JSON	{S:0.2, M:0.5, L:0.3}	ISS Optimization
ros_forecast	Predicted future <b>Rate of Sale</b> for a product/style.	Float	10.2 units/week	ROS Forecasting
attribute_group	Identifier for an <b>AG</b> (e.g., Color, Material, Fit).	String	Denim_Stretch	Data Preprocessing

## Outputs Catalog

The mfp algorithm generates the following primary output datasets:

- mfp\_assortment\_recommendations.csv:**
  - Contains **OD** and **OW** recommendations per category/store/display, indicating the ideal number of unique items and categories to stock.
  - Fields: category\_id, store\_id, od\_recommendation, ow\_recommendation, justification\_metric.
- mfp\_size\_set\_allocations.csv:**
  - Provides **ISS** recommendations at the SKU or style level, detailing the optimal size distribution for replenishment or initial buy.
  - Fields: product\_id, style\_id, iss\_recommendation (JSON string of size:ratio), ideal\_total\_units\_per\_buy.
- mfp\_ros\_forecasts.csv:**
  - Contains future **Rate of Sale (ROS)** predictions for products, crucial for demand planning and replenishment.
  - Fields: product\_id, forecast\_week, ros\_forecast, confidence\_interval\_lower, confidence\_interval\_upper.

#### 4. **mfp\_diagnostic\_summary.csv:**

- A summary of model performance, input data quality issues, and key assumptions used.
- Fields: metric\_name, value, description.

## 5. End-to-End Flow

The mfp algorithm executes through a sequential pipeline designed for robustness and performance:

1. **Data Ingestion:** Raw sales, inventory, and product master data are extracted from source systems. This step includes initial data validation to ensure completeness and format correctness.
2. **Data Transformation & Cleansing:** The ingested data undergoes extensive cleansing. Missing values are imputed, outliers are handled, and various product attributes are standardized. Hierarchies (e.g., product -> style -> category) are established. Historical sales data is aggregated to weekly **ROS** metrics. **Attribute Groups (AGs)** are defined and enriched here.
3. **Optimum Depth & Width Calculation:** This module takes the prepared historical sales and product data. It analyzes sales velocity, inventory turnover, and product profitability across different categories and product styles. Capacity constraints (e.g., store shelf space) are factored in to determine the ideal **OD** (unique items per category) and **OW** (categories/styles per store/display) to maximize sales per square foot or per display.
4. **Ideal Size Set Optimization:** Utilizing granular historical sales data by size, this module applies a statistical model to identify the optimal size curve for each product style. The goal is to recommend an **ISS** that minimizes stockouts on popular sizes while avoiding excess inventory on slower-moving sizes, ultimately maximizing full-price sales.
5. **Rate of Sale Forecasting:** Time-series models are applied to historical **ROS** data, enriched with promotional data and seasonality factors, to predict future sales velocity for each active SKU. These forecasts are critical inputs for inventory planning and replenishment systems.
6. **Recommendation Generation:** All calculated **OD**, **OW**, **ISS**, and **ROS** outputs are compiled. Metrics such as projected sales lift, inventory reduction, and potential profit impact are calculated to support the recommendations.
7. **Output Export & Reporting:** The final recommendations and forecasts are formatted into structured files (e.g., CSV, JSON) and pushed to designated storage locations or APIs, ready for consumption by merchandising reports, inventory planning systems, and business intelligence dashboards.

## 6. Submodule Overview

### 6.1. Data Preprocessing & Validation

- **Purpose:** To clean, transform, and validate raw input data, making it suitable for downstream analytical modules.
- **Inputs & Outputs:**
  - **Inputs:**
    - raw\_pos\_transactions.csv (sales, dates, products, stores)
    - raw\_inventory\_snapshot.csv (product, stock\_level, date)

- `product_master.csv` (product\_id, category, style, size, attributes)
- **Outputs:**
  - `clean_weekly_sales.parquet` (product\_id, week\_start, sales\_units, sales\_value)
  - `enriched_product_catalog.parquet` (product\_id, category, style, size, all relevant **AGs**)
  - `data_quality_report.txt` (summary of missing data, anomalies)
- **Core Logic Summary:**
  1. Ingest and parse raw data files.
  2. Standardize product IDs and date formats across all datasets.
  3. Aggregate daily POS transactions to weekly sales units and values per product/store.
  4. Join sales data with product master data to enrich with attributes, ensuring all necessary **AGs** are present.
  5. Perform outlier detection and handling for sales and inventory figures, and generate a data quality report.

## 6.2. Optimum Assortment Calculation

- **Purpose:** To determine the ideal **Optimum Depth (OD)** (number of unique items) and **Optimum Width (OW)** (number of categories/styles) for product assortments at specific retail locations or displays.
- **Inputs & Outputs:**
  - **Inputs:**
    - `clean_weekly_sales.parquet` (sales performance by product/category/store)
    - `enriched_product_catalog.parquet` (product attributes, category hierarchies)
    - `store_capacity_data.csv` (e.g., linear feet, display count)
    - `args.properties` (min\_sales\_threshold, capacity\_buffer\_pct)
  - **Outputs:**
    - `mfp_assortment_recommendations.csv` (category\_id, store\_id, od\_recommendation, ow\_recommendation)
- **Core Logic Summary:**
  1. Calculate sales density (sales per SKU) and inventory turnover for each product within its category and style.
  2. Identify “star” performers and “dog” items based on profitability and **ROS** metrics, filtered by min\_sales\_threshold.
  3. Apply optimization algorithms (e.g., linear programming, heuristic rules) considering historical performance, category trends, and capacity\_buffer\_pct to determine the ideal **OD** for each category and **OW** for each store/display.
  4. Generate recommendations that balance maximizing sales opportunity with minimizing unproductive inventory.

## 6.3. Ideal Size Set (ISS) Optimization

- **Purpose:** To recommend the best combination of sizes (**ISS**) for a product style to maximize sales and minimize stockouts and overstock.
- **Inputs & Outputs:**
  - **Inputs:**
    - `clean_weekly_sales.parquet` (sales by product\_id, including size)
    - `enriched_product_catalog.parquet` (style\_id, size)

- `args.properties (size_curve_granularity, lookback_period_weeks)`
- **Outputs:**
  - `mfp_size_set_allocations.csv (style_id, iss_recommendation)`
- **Core Logic Summary:**
  1. Group historical sales data by `style_id` and `size` over the `lookback_period_weeks`.
  2. Calculate the sales contribution percentage for each size within a given style or **AG**.
  3. Apply statistical modeling (e.g., normal distribution fitting, historical ratio smoothing) to derive the **ISS** for each style or product, expressed as a ratio or percentage for each size.
  4. Validate the generated **ISS** against stockout history and sales potential to ensure practicality and effectiveness.

## 6.4. Rate of Sale (ROS) Forecasting

- **Purpose:** To predict the future selling speed (**ROS**) of individual products and styles, providing critical input for inventory planning.
- **Inputs & Outputs:**
  - **Inputs:**
    - `clean_weekly_sales.parquet` (historical weekly sales for each product/style)
    - `promotional_calendar.csv` (dates and types of promotions)
    - `args.properties (ros_forecast_horizon)`
  - **Outputs:**
    - `mfp_ros_forecasts.csv (product_id, forecast_week, ros_forecast)`
- **Core Logic Summary:**
  1. Calculate historical **ROS** for each `product_id` or `style_id` based on `clean_weekly_sales.parquet`.
  2. Apply a time-series forecasting model (e.g., ARIMA, Prophet, or machine learning ensemble) to predict future **ROS** values for the `ros_forecast_horizon`.
  3. Incorporate external factors like promotional events, seasonality, and trend analysis into the forecast.
  4. Output the predicted **ROS** for each product/style for upcoming weeks, along with confidence intervals.

## 7. Worked Example

Let's illustrate how **OD**, **OW**, and **ISS** might be determined for a hypothetical "Women's Denim Jeans" category in a specific store, along with **ROS**.

**Scenario:** A mid-sized retail store wants to optimize its "Women's Denim Jeans" category.

**Input Data Snippets:** \* **Historical Sales (past 12 months for this store):** \* Slim Fit (Style A): 1000 units (across all sizes) \* Bootcut (Style B): 600 units \* High-Rise Skinny (Style C): 1500 units \* Mom Jeans (Style D): 400 units \* Straight Leg (Style E): 800 units \* Flare (Style F): 200 units \* **Capacity:** Display can hold 5 distinct styles and approx. 8 SKUs per style. \* **ROS for High-Rise Skinny (Style C) over**

**past 4 weeks:** 30, 35, 28, 32 units/week. \* **Sales by Size for High-Rise Skinny (Style C):** \* Size 25: 10% \* Size 26: 15% \* Size 27: 20% \* Size 28: 25% \* Size 29: 15% \* Size 30: 10% \* Size 31: 5%

### Algorithm Steps & Outputs:

#### 1. ROS Forecasting for Style C (High-Rise Skinny):

- Average historical ROS for Style C (4 weeks):  $(30+35+28+32)/4 = 31.25$  units/week.
- Algorithm predicts future **ROS** for Style C: **30 units/week** (accounting for seasonality, promotions, etc.).

#### 2. Optimum Width (OW) Calculation:

- The algorithm analyzes sales velocity and profitability of all 6 styles (A-F).
- It identifies Styles A, B, C, E as strong performers, and Styles D and F as slower-moving or less profitable.
- Considering the display capacity of 5 styles, and the `optimization_goal` (MAX\_SALES), the algorithm recommends dropping the lowest-performing Style F (Flare).
- **OW Recommendation:** Offer **5** distinct styles: Slim Fit, Bootcut, High-Rise Skinny, Mom Jeans, Straight Leg.

#### 3. Optimum Depth (OD) Calculation:

- For the selected styles, the algorithm evaluates color/wash variations (unique items or SKUs).
- For High-Rise Skinny (Style C), it had 10 SKUs (5 colors x 2 washes). Only 8 SKUs meet the `min_sales_threshold`.
- **OD Recommendation for High-Rise Skinny:** **8 unique SKUs** (e.g., 4 core colors x 2 washes, dropping the 2 slowest-selling color/wash combos).

#### 4. Ideal Size Set (ISS) Optimization for Style C (High-Rise Skinny):

- Given the sales distribution by size (25:10%, 26:15%, 27:20%, 28:25%, 29:15%, 30:10%, 31:5%).
- If the store plans to stock 20 units of Style C, the **ISS** distribution would be:
  - Size 25:  $20 * 0.10 = 2$  units
  - Size 26:  $20 * 0.15 = 3$  units
  - Size 27:  $20 * 0.20 = 4$  units
  - Size 28:  $20 * 0.25 = 5$  units
  - Size 29:  $20 * 0.15 = 3$  units
  - Size 30:  $20 * 0.10 = 2$  units
  - Size 31:  $20 * 0.05 = 1$  unit
- **ISS Recommendation for Style C:** {25:2, 26:3, 27:4, 28:5, 29:3, 30:2, 31:1} (Total 20 units).

**Summary of Recommendations for “Women’s Denim Jeans” Category in this Store:** \* **OW:** Stock 5 core styles. \* **OD (for High-Rise Skinny):** Stock 8 unique SKUs (colors/washes). \* **ISS (for High-Rise Skinny):** Optimal size distribution for a 20-unit buy is 2:3:4:5:3:2:1 (Size 25:31). \* **ROS (for High-Rise Skinny):** Forecasted at 30 units/week.

## 8. Debug Guide Highlights

### • Missing or Incomplete Data:

- **Symptom:** Module failure, NaN outputs, or significantly skewed recommendations.



- **Check:** `data_quality_report.txt` from Data Preprocessing. Verify presence of all required fields (`product_id`, `sales_units`, `stock_on_hand`, `category`, `size`).
- **Action:** Trace back to raw data sources. Confirm successful ingestion and data completeness for the `lookback_period_weeks`.
- **Skewed ROS Forecasts:**
  - **Symptom:** Unrealistic (too high/low) **ROS** predictions, leading to overstock/understock issues.
  - **Check:** Input `promotional_calendar.csv` for correct mapping. Review `min_sales_threshold` in `args.properties` (too high might exclude valid items).
  - **Action:** Examine historical sales for extreme outliers not handled by preprocessing. Adjust forecasting model parameters or consider adding more relevant features (e.g., competitor data, economic indicators).
- **Inconsistent OD/OW Recommendations:**
  - **Symptom:** Algorithm suggests dropping high-performing items or adding too many low-performing ones.
  - **Check:** `optimization_goal` in `args.properties`. Ensure `capacity_buffer_pct` is realistic.
  - **Action:** Review the performance metrics (sales, margin, turnover) used in the OD/OW module. Validate if product categorizations and **AGs** are correctly assigned.
- **Invalid ISS Allocations:**
  - **Symptom:** **ISS** suggests disproportionate quantities for fringe sizes, leading to customer dissatisfaction or excessive markdowns.
  - **Check:** `size_curve_granularity` setting. Ensure sufficient historical sales by size are available for the target product/style.
  - **Action:** If data is sparse, consider rolling up **ISS** calculation to a higher **AG** level (e.g., Category instead of Style) or imputing based on similar product profiles.

## 9. Common Pitfalls & Remedies

- **Pitfall: “Garbage In, Garbage Out”:** The algorithm’s effectiveness is directly tied to the quality of input data. Dirty, inconsistent, or incomplete data will lead to inaccurate recommendations.
  - **Remedy:** Implement rigorous data governance and validation routines. Proactive data profiling and automated alerts for data anomalies are crucial. Invest in robust data ingestion and preprocessing pipelines.
- **Pitfall: Ignoring Business Context:** Purely data-driven recommendations might overlook qualitative factors like emerging trends, brand strategy, or specific local market nuances.
  - **Remedy:** Foster strong collaboration between the algorithm’s developers/analysts and the merchandising teams. Build feedback loops allowing merchandisers to provide qualitative input or override recommendations with clear justification.
- **Pitfall: Static Configuration:** Default `args.properties` values may not be optimal for all categories or seasons.
  - **Remedy:** Implement a mechanism for configurable parameters. Regularly review and tune key parameters like `lookback_period_weeks`, `min_sales_threshold`, and `optimization_goal` based on business performance and changing market conditions. Consider category-specific parameter sets.

- **Pitfall: Over-reliance on Historical Data:** Past performance doesn't guarantee future results, especially with new product introductions or rapidly changing fashion trends.
  - **Remedy:** Incorporate external data sources (e.g., market trends, competitor analysis, social media signals) into the **ROS** forecasting module. Develop logic for new product introductions based on similar product launches or industry benchmarks.
- **Pitfall: Lack of Interpretability:** If merchandising teams don't understand *why* a recommendation was made, they may not trust or adopt it.
  - **Remedy:** Enhance output reports with clear justifications and performance metrics (e.g., "This **OW** recommendation is projected to increase sales by X%"). Provide clear visualization of data supporting **OD**, **OW**, and **ISS** decisions.

## 10. FAQ

- **Q: How frequently are the recommendations updated?**
  - **A:** The mfp algorithm typically runs weekly, providing fresh **OD**, **OW**, **ISS**, and **ROS** recommendations. Critical data points like **ROS** forecasts can be updated more frequently if needed, e.g., daily during peak seasons.
- **Q: Can the algorithm account for new product introductions?**
  - **A:** For new products with no historical sales, the algorithm uses a "cold-start" strategy. This often involves assigning default **ROS** based on similar **Attribute Groups (AGs)** or categories, or using initial buy-in data to establish preliminary **ISS** and **OD** estimates. These estimates are refined as sales data accumulates.
- **Q: What if a specific store has unique characteristics (e.g., small footprint, specialty focus)?**
  - **A:** The algorithm can be configured to consider store-specific attributes and constraints. By providing `store_capacity_data.csv` and segmenting analysis by `store_id`, the **OD** and **OW** recommendations can be tailored. For **ISS**, local demand patterns can be incorporated if sufficient sales data exists.
- **Q: How does the algorithm handle promotional periods or seasonal spikes?**
  - **A:** The **ROS** forecasting module explicitly incorporates `promotional_calendar.csv` and historical seasonality indices to adjust predictions during sales events and seasonal peaks/troughs. This ensures that recommended stock levels account for anticipated demand fluctuations.
- **Q: Is there a way to override the recommendations?**
  - **A:** Yes, the outputs are designed to be consumed by merchandising systems that allow for manual adjustments or overrides. It's crucial for merchandisers to review recommendations and apply their business judgment, especially for strategic items or unforeseen market shifts. Justifications for overrides should ideally be logged for future analysis.

## 11. Quick Reference Cheat-Sheet

Term	Definition	Key Metric/ Example	Module(s) Generated By
<b>OD</b>	<b>Optimum Depth:</b> The ideal number		

Term	Definition	Key Metric/ Example	Module(s) Generated By
OW	of unique items (SKUs) to stock in a category or display.	Example: 12 unique T-shirt colors/sizes.	Optimum Assortment Calculation
	<b>Optimum Width:</b> The ideal number of different categories or styles to offer.	Example: 5 distinct denim styles (Slim, Bootcut, Skinny, etc.).	Optimum Assortment Calculation
	<b>Ideal Size Set:</b> The best combination of sizes to maximize sales and minimize stockouts.	Example: For 100 units, {S:20, M:50, L:30}.	Ideal Size Set Optimization
ROS	<b>Rate of Sale:</b> The speed at which products are sold over a period (e.g., units/week).	Example: 15 units/week for a specific product.	ROS Forecasting, Data Preprocessing (Historical ROS)
AG	<b>Attribute Group:</b> A set of product attributes grouped for analysis (e.g., color, size, brand).	Example: “Red” (from Color AG), “Stretch Denim” (from Material AG).	Data Preprocessing & Validation
lookback_period	Number of weeks of historical data used for analysis.	Config: 52 weeks	Configuration (args.properties)
min_sales_threshold	Minimum <b>ROS</b> for an item to be considered active for analysis.	Config: 5 units/week	Configuration (args.properties)
Inputs	POS, Inventory, Product Master, Promotional Calendar	Raw data for analysis.	Data Ingestion
Outputs	Assortment Recommendations, Size Set Allocations, ROS Forecasts, Diagnostic Summary	Actionable insights for merchandising.	Output Generation

# Chapter 1: NOOS Orchestration Module

**Last Updated:** 2025-05-23

## 1. Business Objective

This module manages the precise sequencing and execution of all sub-processes within the NOOS (Never Out of Stock) program, ensuring that complex inventory recommendations are generated accurately and reliably.

**Why This Matters:** This ensures that merchandising decisions are based on accurate and timely data, preventing critical stockouts on key items, optimizing inventory levels, and ultimately maximizing sales potential and profitability.

## 2. Required Information

We need these pieces of information to do our job. While this module orchestrates the process, it depends on the availability of consistently updated data from upstream systems that feed its sub-modules.

### Sales Data

- **Units Sold:** We need this to understand how quickly specific products are moving and to identify bestsellers.
- **Sales History:** We need this to analyze long-term performance, spot trends, and calculate sales consistency.

### Product Data

- **Product Categories:** We need this to group similar items for analysis, ensuring relevant comparisons (e.g., all t-shirts together).
- **Product Attributes** (e.g., color, size, material): We need this to understand variations within categories and to calculate Ideal Size Sets (ISS).

### Operational Data

- **Current Stock Levels:** We need this to determine the current inventory position and identify potential stockouts or overstocks.
- **Delivery Timelines:** We need this to factor in lead times for replenishment orders, ensuring products arrive when needed.

### 3. Business Settings

Setting Name	Default Value	Description	When to Change
Minimum Sales Threshold	100 units	The lowest number of units a product must sell within the “Analysis Period” to be considered for core or bestseller status.	If sales volume significantly changes, or to focus only on extremely high-performing items.
Analysis Period	90 days	The duration of historical sales data (e.g., 90 days) used for evaluating product performance.	To adapt to shorter seasons, rapidly changing trends, or longer sales cycles.
Sales Consistency Score	0.75	A score (0-1) indicating how consistently a product sells. Higher values mean more predictable demand.	If you want to prioritize products with very stable demand (increase value) or include products with more volatile but high sales (decrease value).
Ideal Size Set (ISS) Calculation Method	Weighted Average	Determines how historical sales data is used to calculate the best mix of sizes.	If specific product types require different size distribution strategies (e.g., tailored versus relaxed fit).

### 4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Raw Sales & Product Data] --> B[Initial Data Capture]
B --> C[Key Size Analysis]
C --> D[Core Style Identification]
D --> E[Bestseller Style Identification]
E --> F[Bestseller Override Management]
F --> G[Final Output Distribution]
G --> H[Actionable Inventory Recommendations]
```

1. Start with → Initial Data Capture

2. Then → Key Size Analysis
3. Next → Core Style Identification
4. Finally → Bestseller Style Identification (followed by overrides and final distribution)

## 5. Step-by-Step Actions

1. **Prepare** the environment
  - What we do: *We initiate the process by taking a complete snapshot of all current product and sales data. This locks in the information we'll use for the entire analysis.*
  - Why we do it: *This creates a stable and consistent starting point, preventing any real-time changes in data from affecting our analysis and ensuring reliable results.*
2. **Determine** Key Size Sets (ISS)
  - What we do: *The system analyzes historical sales to identify the most successful combination of sizes (Ideal Size Sets) for different product groups, like denim or t-shirts.*
  - Why we do it: *This ensures we stock the right mix of sizes that customers actually buy, minimizing wasted inventory on unpopular sizes and reducing lost sales from missing popular ones.*
3. **Identify** Core Styles
  - What we do: *The system processes sales data to pinpoint products that consistently sell well, week after week, regardless of seasonal trends. These are our fundamental, 'always-on' items.*
  - Why we do it: *These products are the reliable backbone of our inventory, guaranteeing we always have our most dependable and demanded items readily available.*
4. **Spotlight** Bestseller Styles
  - What we do: *Next, the system identifies products that are currently experiencing exceptionally high sales velocity, often indicative of a new trend or strong customer preference.*
  - Why we do it: *These are our immediate revenue drivers. Rapid identification and replenishment allow us to quickly capitalize on surging demand and maximize sales.*
5. **Adjust** for Bestseller Overrides
  - What we do: *After identifying bestsellers, the system applies any pre-defined manual adjustments or thematic overrides. For example, a merchant might designate an item as a bestseller for a specific promotion.*
  - Why we do it: *This allows human merchandising expertise to fine-tune automated recommendations, ensuring we account for planned promotions, unexpected local events, or strategic marketing pushes.*
6. **Finalize** and Distribute Output
  - What we do: *Once all calculations, identifications, and adjustments are complete, the system consolidates the final inventory recommendations and makes them available for other teams.*
  - Why we do it: *This ensures that our inventory planners and store teams receive timely, accurate, and actionable plans to manage stock, place orders, and allocate products effectively to drive sales and customer satisfaction.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A retail chain is preparing for a new season, focusing on replenishing its NOOS product line, specifically a classic black denim jean.
- Last season's sales data shows a strong performance for the black denim, with 3,500 units sold across all stores over the past 90 days.
- Current stock levels for the black denim are low, with only 500 units in total.
- The company policy dictates a "Minimum Sales Threshold" of 200 units for core items.

### What Happens

1. First, the NOOS Orchestration Module ensures all historical sales and current stock data for the black denim, and all other products, are captured and ready for analysis.
2. Then, it triggers the Key Size Analysis module, which determines that for black denim, sizes 30, 32, and 34 consistently make up 65% of all sales.
3. Next, it activates the Core Style Identification module. Because the black denim exceeded the 200-unit "Minimum Sales Threshold" over the "Analysis Period" with a high "Sales Consistency Score," it's flagged as a Core Style.
4. Following this, the Bestseller Style Identification module checks if the black denim is also a current bestseller. Due to its consistent performance and high demand, it's identified as both a core and a bestseller.
5. Finally, the Bestseller Override Management module is engaged. In this scenario, there are no manual overrides specific to this black denim, so the automated recommendation stands. The NOOS Orchestration Module then ensures the final replenishment recommendation is prepared, considering its core status, bestseller velocity, and the ideal size breakdown.

### Final Outcome

The inventory planning team receives a clear recommendation: an urgent, substantial replenishment order for the black denim, specifically emphasizing sizes 30, 32, and 34 to maintain optimal stock levels across stores. This ensures the chain avoids missing sales on a highly popular item.

**Key Takeaway:** The NOOS Orchestration Module seamlessly coordinates multiple analytical steps, transforming raw sales data into precise, actionable inventory plans for key products.

## 7. Results

This step produces:

## Main Outputs

- Optimized Inventory Recommendations: Specific quantities, Ideal Size Set (ISS) allocations, and replenishment triggers for core and bestseller products.
- Identified Core Styles List: A definitive list of products designated for continuous stocking due to their consistent, reliable sales performance.
- Identified Bestseller Styles List: A dynamic list of products currently experiencing high sales velocity, requiring immediate attention for replenishment.
- Ideal Size Set (ISS) Guidance: Detailed recommendations for the optimal size distribution for each product, based on sales history.

## Business Reports

- Daily Replenishment Report: Used by inventory planners to review daily recommendations and execute purchase orders, directly preventing stockouts.
- Weekly Assortment Performance Report: Used by merchandisers to assess how well core and bestseller strategies are performing, helping them fine-tune product assortments.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will either halt processing until data is available or proceed with the last complete dataset, potentially generating outdated recommendations.	Immediately investigate the data source (e.g., POS system, warehouse management) to restore the feed and rerun the module.
No items meet threshold for Core or Bestseller designation	The module will complete its run but generate an empty or very limited list of core/bestseller items.	Review your “Minimum Sales Threshold” and “Sales Consistency Score” settings. It might also indicate a widespread decline in product performance.
All items identified as Bestsellers	This suggests your “Sales Consistency Score” or sales velocity thresholds are set too low, or there’s an unusual, widespread market surge.	Review the “Sales Consistency Score” and sales velocity settings to ensure they accurately reflect true bestseller criteria.
Conflicting manual override rules	The system will apply rules based on a predefined priority (e.g., manual overrides typically supersede automated ones),	Review your manual override processes and ensure clear communication to prevent contradictory instructions from different teams.



Situation	What Happens	What You Should Do
	potentially with a warning message.	

## 9. Next Steps

### Before This Step

- **Raw Data Collection:** We need complete, accurate, and timely sales, inventory, and product attribute data from all relevant systems.
- **Business Rule Definition:** We need clearly defined business parameters, such as the “Minimum Sales Threshold” and “Analysis Period,” to guide the module’s computations.

### After This Step

- **Inventory Planning & Ordering:** They need the final, actionable inventory recommendations to place accurate purchase orders with suppliers and manage warehouse stock efficiently.
- **Store Allocations:** They need the Ideal Size Set (ISS) guidance and quantity recommendations to distribute products effectively to individual stores, ensuring the right products are in the right place.

## 10. Common Questions

**Q:** Why are my core styles sometimes out of stock, even if the module identifies them? **A:** While the N00S Orchestration Module identifies core styles and ensures their replenishment is prioritized, factors like unexpected spikes in demand, unforeseen supply chain disruptions, or external logistical challenges can still lead to temporary stockouts. It’s crucial to review supply chain lead times regularly.

**Q:** How does this module handle brand-new products that don’t have existing sales history? **A:** New products initially rely on their initial buying plans and typically won’t qualify as “Core” or “Bestseller” until sufficient sales data accumulates. However, if a new product quickly gains traction, it can be manually promoted via the “Bestseller Override Management” step.

**Q:** Can I manually influence or override the recommendations generated by this process? **A:** Yes, absolutely. The “Bestseller Override Management” step is specifically designed to allow merchandising teams to manually adjust quantities or classifications for specific products based on promotions, emerging trends, or unique local market insights, ensuring necessary flexibility.

**Q:** What if the Ideal Size Set (ISS) recommendations seem incorrect for a particular category or product? **A:** The Ideal Size Set is calculated directly from historical sales data. If the recommendation seems off, it might indicate unusual past sales patterns for that product or category, or a need to refine the “Key Size Analysis” module’s settings, such as adjusting the historical “Analysis Period” it considers.

Q: How does this module directly contribute to reducing lost sales and improving customer satisfaction? A: By ensuring that high-demand “Core” and “Bestseller” styles are consistently in stock and optimized for size distribution, the NOOS Orchestration Module significantly reduces instances where customers cannot find what they want, directly translating to fewer missed sales opportunities and a much better shopping experience.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Manages the overall execution flow to deliver reliable inventory recommendations for Never Out of Stock (NOOS) products.
- **Main Input:** Processed sales and product data from various analytical sub-modules.
- **Main Output:** Actionable, sequenced inventory recommendations for core and bestseller styles, including Ideal Size Sets.
- **Warning Signs:** Unexplained delays in recommendation generation, or significant discrepancies between recommended and actual stock levels.
- **Key Settings:** Minimum Sales Threshold, Analysis Period, Sales Consistency Score.

---

**Need Help?** Contact your Inventory Planning or Merchandising Operations team for questions about this process.

---

Generated by [AI Codebase Knowledge Builder](#)

# Chapter 2: NOOS Paramount Sizes Calculation Module

**Last Updated:** 2025-05-23

## 1. Business Objective

This module precisely identifies “paramount sizes”—the most important sizes that drive the majority of revenue for specific product categories—and cleanses raw sales data to ensure high-quality input for all subsequent NOOS calculations.

**Why This Matters:** By accurately determining which sizes truly matter and filtering unreliable sales data, this module ensures that inventory decisions for NOOS items are based on the most impactful and accurate information, leading to maximized sales by stocking the right products in the right sizes, reduced stockouts, and improved profitability.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- Units Sold by Size: We need this to understand the sales velocity of individual sizes within each product group.
- Revenue by Size: We need this to identify which sizes contribute the most financially to a category’s overall performance.

### Product Data

- Product Categories (e.g., Category, Subcategory): We need this to group similar items (like “Women’s Denim” or “Men’s T-shirts”) for relevant size analysis, ensuring paramount sizes are identified appropriately.
- Product Attributes (e.g., Size, Brand, Gender Segment): We need this to break down sales performance by specific product variations and identify which sizes are most important for different product types.

### Operational Data

- Total Store Count: We need this to determine the minimum number of stores a product must be available in for its sales data to be considered reliable.

- **Live Store Presence:** We need to know which stores had a particular product available on a given day to ensure sales data accurately reflects customer choice, not lack of availability.
- **Sizes Live in Store:** We need to know which specific sizes were available in a store on a given day to confirm that customers had the option to purchase the paramount sizes.

### 3. Business Settings

Setting Name	Default Value	Description	When to Change
Top X% Revenue Contribution	80%	The percentage of a category's total revenue that the identified paramount sizes must collectively contribute.	If your business aims to capture a larger or smaller share of sales from a concentrated set of sizes (e.g., 75% for broader appeal, 90% for extreme focus).
Minimum Live Stores Percentage	50%	The minimum percentage of total stores a product must be physically available in for its sales data to be included in the NOOS calculations.	If you want to include sales from products with narrower distribution (decrease value) or only from products with very broad availability (increase value).
Paramount Sizes Live Percentage	75%	The minimum percentage of identified paramount sizes for a product that must be physically available in a specific store on a given day for that day's sales to be considered valid.	If you want to ensure sales data is only considered when a wide range of important sizes was truly offered to customers (increase value) or be more flexible (decrease value).

### 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales & Product Data] --> B[Compute Paramount Sizes]

B --> C[Cleanse Sales Data]  
C --> D[Cleaned Sales & Paramount Sizes List]

1. Start with → Raw Sales & Product Data
2. Then → Compute Paramount Sizes
3. Next → Cleanse Sales Data
4. Finally → Cleaned Sales & Paramount Sizes List

## 5. Step-by-Step Actions

1. **Compute Paramount Sizes**
  - What we do: *The system analyzes historical sales data, grouping products by categories (like “Women’s Dresses” or “Men’s Sneakers”). For each group, it identifies which specific sizes, when combined, account for the highest percentage of total revenue, based on the “Top X% Revenue Contribution” setting.*
  - Why we do it: *This ensures we focus our inventory on the sizes that consistently generate the most income for each product type, preventing wasted stock on less popular sizes and maximizing our investment.*
2. **Cleanse Sales Data**
  - What we do: *Next, the module rigorously filters all raw sales transactions. A sale is only considered valid for NOOS calculations if the product was available in at least the “Minimum Live Stores Percentage” of our total store count. Additionally, for a sale to count, a certain “Paramount Sizes Live Percentage” of the identified paramount sizes for that product must have been physically available in the specific store where the sale occurred on that day.*
  - Why we do it: *This step guarantees that all sales data used in subsequent NOOS calculations is high-quality and reflects genuine customer demand when ample supply was present, rather than skewed by limited stock or poor distribution.*
3. **Store Results**
  - What we do: *The module saves the comprehensive list of paramount sizes for each product category and the newly filtered, high-quality sales data. This prepared information is then made available for all other NOOS modules.*
  - Why we do it: *This creates a reliable, foundational dataset, ensuring that every subsequent inventory recommendation, such as Ideal Size Set (ISS) allocations and Rate of Sale (ROS) calculations, is built upon accurate and validated sales performance.*

## 6. Sample Scenario

Let’s walk through a real example:

### Starting Point

- A fashion retailer is analyzing sales for “Women’s Classic Skinny Jeans” for the last 90 days.
- The “Top X% Revenue Contribution” is set to 80%.
- The “Minimum Live Stores Percentage” is 60%.

- The “Paramount Sizes Live Percentage” is 70%.
- Total stores for the retailer: 100.
- Raw sales data shows sales across sizes 24 to 34 for these jeans.

## What Happens

1. First, the module **computes** Paramount Sizes. It aggregates all sales for “Women’s Classic Skinny Jeans” by size. It discovers that sizes 26, 28, 30, and 32 collectively account for 85% of the total revenue for this product. These four sizes are therefore identified as the “paramount sizes” for this jean style.
2. Then, the module **cleanses** sales data. It processes each individual transaction for the jeans:
  - A sale of a pair of jeans on Tuesday in Store A is checked. Was this jean style live in at least 60 stores (60% of 100 total stores) on Tuesday? If not, that transaction is excluded.
  - If it was live in enough stores, the module checks if the paramount sizes (26, 28, 30, 32) were adequately available in Store A on Tuesday. Were at least 3 out of 4 (75% of 4 sizes, meeting the 70% requirement) of these paramount sizes physically in stock? If not, that specific transaction is also excluded.
3. Finally, only the sales transactions that pass both these criteria are kept and **stored** as “cleaned sales data.”

## Final Outcome

The N00S Paramount Sizes Calculation Module provides a definitive list: “Women’s Classic Skinny Jeans” have paramount sizes 26, 28, 30, and 32. It also generates a high-quality dataset of sales for these jeans, filtered to only include transactions where the product had sufficient distribution and the most important sizes were actually available. This ensures future calculations, like replenishment quantities for these jeans, are based on reliable demand signals.

**Key Takeaway:** This module refines vast amounts of raw sales information into actionable insights by identifying the most impactful sizes and ensuring the sales data used for future decisions is accurate and trustworthy.

## 7. Results

This step produces:

### Main Outputs

- Paramount Sizes List: A precise, categorized list detailing which sizes are the most important revenue drivers for each product category (e.g., for “Women’s Denim,” sizes 26, 28, 30).
- Cleaned Sales Data: A filtered, high-quality dataset of sales transactions, validated for sufficient live store presence and paramount size availability.

## Business Reports

- Category Size Contribution Report: Used by merchandisers and category managers to understand which sizes consistently perform best and should be prioritized in assortments and visual merchandising.
- Sales Data Integrity Report: Used by data quality teams and inventory analysts to confirm that the sales data feeding into NOOS calculations meets predefined reliability standards.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will either halt processing or use outdated data, leading to incorrect paramount size identification and data cleansing.	Immediately investigate the upstream data source (e.g., POS, inventory system) to restore the feed and rerun the module to ensure up-to-date analysis.
No paramount sizes identified for a category	The module might identify an empty list of paramount sizes if sales are extremely fragmented across all sizes or if sales volume is too low to meet the “Top X% Revenue Contribution.”	Review the “Top X% Revenue Contribution” setting. This could also indicate a category with very low demand or an unusually broad distribution of sales across sizes.
Too many sizes identified as paramount	If the “Top X% Revenue Contribution” is set too low (e.g., 99%), or if sales are very evenly distributed across all sizes for a category, nearly every size might be flagged as paramount.	Re-evaluate the “Top X% Revenue Contribution” to align with your business strategy for size focus. This might suggest the category truly has no “paramount” sizes or requires a different approach.
Most sales data is filtered out during cleansing	This indicates that either products frequently lack sufficient store presence or the identified paramount sizes are rarely in stock in stores when sales occur.	Investigate supply chain issues, store allocation problems, or review the “Minimum Live Stores Percentage” and “Paramount Sizes Live Percentage” settings.

## 9. Next Steps

### Before This Step

- Raw Data Collection: We need complete and accurate sales, product, and store availability data from all points of sale and inventory systems.
- Business Rule Definition: We need clearly defined business parameters, such as the “Top X% Revenue Contribution” and “Minimum Live Stores Percentage,” to guide the module’s computations.

### After This Step

- Inventory Planning & Replenishment: They need the cleaned sales data for accurate Rate of Sale (ROS) calculations and Ideal Size Set (ISS) recommendations, crucial for placing precise replenishment orders.
- Assortment Planning: They need the Paramount Sizes List to inform decisions on what sizes to stock at the Optimum Depth (OD) and how to manage the Optimum Width (OW) of categories.

## 10. Common Questions

Q: Why are some of my sales data transactions filtered out by this module? A: Sales transactions are filtered to ensure high data quality. If a product wasn’t widely available in stores, or if its most important (paramount) sizes weren’t in stock when a sale happened, that sale might not accurately reflect true demand. Filtering ensures we only use data where customers had a fair chance to buy, leading to more reliable inventory recommendations.

Q: How often are the “paramount sizes” recalculated for a product category? A: Paramount sizes are typically recalculated during each full run of the NOOS process, which usually happens daily or weekly. This ensures the system continuously adapts to changing customer preferences and sales trends, keeping the “paramount sizes” list current.

Q: What if our product mix or category definitions change frequently? A: The module relies on consistent product category and attribute data. If these definitions change, the system automatically re-evaluates and computes new paramount sizes based on the latest groupings. However, frequent, drastic changes might temporarily impact historical analysis until new stable patterns emerge.

Q: How does “paramount sizes” relate to “Ideal Size Set (ISS)”? A: “Paramount sizes” identifies *which* sizes are the most important contributors to revenue for a product category. “Ideal Size Set (ISS)” then uses this information, along with the cleaned sales data, to determine the *optimal quantity distribution* of *all* sizes for that product, ensuring you stock the right mix to maximize sales and minimize stockouts.

Q: What’s the direct impact of this module on our inventory investment? A: By precisely identifying paramount sizes, this module guides inventory investment towards the sizes that yield the highest returns. By cleansing sales data, it prevents



overstocking or understocking based on misleading sales figures. This directly reduces inefficient inventory, improves sell-through, and frees up capital for better-performing items.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Identifies key revenue-driving sizes and ensures sales data quality for NOOS calculations.
- **Main Input:** Raw sales, product category, and store availability data.
- **Main Output:** A list of paramount sizes by category and a filtered, high-quality cleaned sales dataset.
- **Warning Signs:** Unexpectedly high filtering of sales data, or seemingly illogical paramount size identifications.
- **Key Settings:** Top X% Revenue Contribution, Minimum Live Stores Percentage, Paramount Sizes Live Percentage.

---

**Need Help?** Contact your Inventory Planning or Merchandising Operations team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 4: NOOS Bestseller Style Identification Module

**Last Updated:** 2025-05-23

## 1. Business Objective

This module identifies “Bestseller” styles by pinpointing products that are currently experiencing exceptionally high sales velocity, based on a rigorous set of performance criteria.

**Why This Matters:** Accurately identifying bestsellers allows the business to rapidly capitalize on surging customer demand, prioritize replenishment efforts for immediate revenue drivers, and ensure the optimal product offering (OD) to maximize sales and minimize lost opportunities from stockouts on hot items.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- Units Sold by Style: We need this to understand the volume of products sold and calculate sales velocity for individual styles.
- Revenue by Style: We need this to assess the financial contribution of each style and identify the most valuable top performers.
- Discount Value by Style: We need this to understand how much a product’s sales might be influenced by promotional pricing, ensuring we identify truly high-demand items.

### Product Data

- Product Categories: We need this to group similar items (e.g., “Women’s Denim,” “Men’s T-shirts”) for relevant comparisons and to calculate category-specific sales benchmarks.
- Product Attributes (e.g., Brand, Price Bucket): We need this to further refine product groupings and apply appropriate analysis within specific market segments.

## Operational Data

- **Style Live Days:** We need to know the number of days a style has been actively available for sale in stores to ensure its performance metrics are calculated over a realistic period of exposure.
- **Current Product Classification:** We need to know if a style is already designated as a “Core” item, as Bestseller rules might differ or overlay with existing classifications.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Bestseller Analysis Period	30 days	The duration of recent sales data (e.g., 30 days) used to evaluate a style’s current bestseller performance.	To react more quickly to fast-changing trends (shorter period) or to capture more consistent, longer-term performance (longer period).
Top X% Revenue Contribution	20%	The percentage of revenue within a product grouping (e.g., Category, Subcategory, Brand Segment, Price Bucket) that the identified bestsellers must collectively contribute.	To focus on a more concentrated set of top performers (smaller percentage) or to broaden the scope of bestsellers (larger percentage).
Minimum Bestseller Units Sold	50 units	The lowest number of units a style must sell within the “Bestseller Analysis Period” to be considered for bestseller status.	If sales volume significantly changes, or to focus only on extremely high-performing items.
Maximum Bestseller Styles Limit	Varies by Category	The maximum allowable number of styles that can be designated as bestsellers within a given category.	To maintain a healthy balance in product offering (OW) and avoid over-investing in too many “hot” items.

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Product Data] --> B{Calculate Category & Discount Benchmarks}

B --> C{Evaluate Style Performance}

C --> D{Designate Provisional Bestseller Styles}

D --> E{Apply Bestseller Portfolio Limits}

E --> F[Identified Bestseller Styles List]

1. Start with → Cleaned Sales Data and Product Information
2. Then → Calculate Category and Discount Benchmarks
3. Next → Evaluate Individual Style Performance
4. Next → Designate Provisional Bestseller Styles
5. Next → Apply Bestseller Portfolio Limits
6. Finally → Generate Final Bestseller Styles List

## 5. Step-by-Step Actions

### 1. Prepare Data and Live Days

- What we do: *The module takes cleaned sales data (from the previous NOOS Paramount Sizes Calculation Module) and comprehensive product information. It also calculates how many days each style has been actively available for sale in stores within the “Bestseller Analysis Period.”*
- Why we do it: *This ensures we have accurate and reliable sales figures, along with precise availability data, to use as the foundation for identifying true bestsellers.*

### 2. Calculate Category & Discount Benchmarks

- What we do: *The system computes the average revenue per day for a typical style within each product category (e.g., “Women’s Knitwear”). It also determines the average discount applied across styles within various product groupings (e.g., Category, Subcategory, Brand Segment, Price Bucket).*
- Why we do it: *These benchmarks provide critical context, allowing us to compare individual style performance against its peers and identify those truly excelling beyond the norm, not just selling due to heavy discounts.*

### 3. Evaluate Individual Style Performance

- What we do: *For each style, the module calculates its actual average revenue per day and its own average discount. It also checks if the style is among the “Top X% Revenue Contribution” in its grouping, has sold a “Minimum Bestseller Units Sold,” and has been live for a specified number of days.*
- Why we do it: *This comprehensive evaluation ensures that only styles demonstrating genuinely strong, sustained performance, meeting various business criteria, are considered for bestseller status.*

### 4. Designate Provisional Bestseller Styles

- What we do: *Based on the evaluation, styles that meet all the predefined criteria (high sales contribution, strong daily revenue relative to its category, acceptable discount level compared to its grouping, minimum*

*quantity sold, and sufficient live days) are provisionally flagged as Bestsellers.*

- *Why we do it: This step identifies the initial pool of high-potential items that are currently driving significant sales, setting them aside for further review and prioritization.*

#### 5. **Apply Portfolio Limits**

- *What we do: The module then enforces “Maximum Bestseller Styles Limit” on the total number of Bestseller styles. This might involve prioritizing styles with the highest revenue per day or those contributing most significantly to overall category revenue, to ensure a balanced and optimal product offering (OD and OW).*
- *Why we do it: This crucial step prevents the business from over-investing in too many “bestsellers” at once, ensuring that our inventory focus remains disciplined and we don’t dilute the impact of our top-performing items by having too many “hot” products.*

#### 6. **Generate Final Bestseller Styles List**

- *What we do: Finally, the module compiles the definitive list of styles designated as “Bestsellers” along with their key performance metrics. Styles not flagged as bestsellers are also included in the output with their current classification.*
- *Why we do it: This provides a clear, actionable output for inventory planners and merchandisers, enabling them to make informed decisions about replenishment, allocation, and marketing for the most impactful products, ultimately driving sales and customer satisfaction.*

## 6. Sample Scenario

Let’s walk through a real example:

### Starting Point

- A fashion retailer is assessing its “Women’s Classic Black Knit Sweater” for the upcoming week.
- The “Bestseller Analysis Period” is set to 30 days.
- The average daily revenue per style benchmark for the “Women’s Knitwear” category is \$50.
- The average discount benchmark for this product grouping is 10%.
- “Minimum Bestseller Units Sold” is 25 units.
- Minimum live days for a bestseller is 15 days.
- Over the past 30 days, the “Classic Black Knit Sweater” sold 150 units, generated \$7,500 in revenue, and its average discount was 8%. It has been live for 28 days. It is among the top 15% of revenue contributors in its specific grouping.

### What Happens

1. First, the module **prepares** the data, confirming the sweater’s 28 live days and sales data are valid.
2. Then, it **calculates** that for “Women’s Knitwear,” the average daily revenue per style is \$50, and the average discount in its grouping is 10%. These become the benchmarks.

3. Next, it **evaluates** the sweater's performance:
  - Its actual average daily revenue is  $\$7,500 / 30 \text{ days} = \$250$ . This is significantly higher than the category benchmark of \$50.
  - Its average discount of 8% is below the benchmark of 10%.
  - It sold 150 units, exceeding the 25-unit threshold.
  - It was live for 28 days, exceeding the 15-day threshold.
  - It is within the top 20% revenue contribution criteria for its grouping.
4. Because the sweater meets all these demanding criteria, it is **designated** as a provisional Bestseller.
5. Finally, the module **applies** the "Maximum Bestseller Styles Limit" for the "Women's Knitwear" category. If the sweater's performance is strong enough to remain within this limit, it is confirmed as a Bestseller.

## Final Outcome

The "Women's Classic Black Knit Sweater" is officially identified as a Bestseller style. This triggers immediate prioritization for replenishment, potentially higher stock levels, and enhanced visibility in marketing and store displays to fully capitalize on its high demand and maximize revenue.

**Key Takeaway:** This module precisely identifies current high-performing products using multiple performance metrics, ensuring that inventory and marketing efforts are focused on items with the highest immediate sales potential.

## 7. Results

This step produces:

### Main Outputs

- Identified Bestseller Styles List: A definitive list of products currently designated as bestsellers, along with their associated performance metrics (e.g., average daily revenue, total units sold, average discount).
- Performance Benchmarks: The calculated category-level average daily revenue per style and product grouping average discount, used for evaluating future styles.

### Business Reports

- Daily Bestseller Replenishment Report: Used by inventory planners to quickly identify and prioritize immediate replenishment orders for high-demand items, preventing lost sales.
- Weekly Assortment Performance Report: Used by merchandisers and category managers to review how bestseller strategies are impacting overall sales and profitability, guiding future buying decisions.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system will either halt processing until data is available or proceed with the last complete dataset, potentially generating outdated or inaccurate bestseller recommendations.	Immediately investigate the data source (e.g., POS system, sales reporting) to restore the feed and rerun the module to ensure up-to-date analysis.
No styles identified as Bestsellers	The module will complete its run but generate an empty or very limited list of bestsellers, indicating that no styles met the strict criteria.	Review your “Bestseller Analysis Period,” “Top X% Revenue Contribution,” “Minimum Bestseller Units Sold,” and other thresholds. These might be set too high, or there might be a genuine widespread decline in product performance.
Too many styles identified as Bestsellers	This suggests your evaluation criteria are too lenient, or your “Maximum Bestseller Styles Limit” is set too high, potentially diluting the focus on true top performers.	Review the “Top X% Revenue Contribution” and “Maximum Bestseller Styles Limit” settings to ensure they accurately reflect true bestseller criteria and maintain a focused product offering.
A high-selling style is not identified as a Bestseller	The style likely failed one of the specific criteria, such as exceeding the acceptable average discount, not being live for enough days, or not contributing significantly enough to the top revenue percentage in its grouping.	Review the specific style’s metrics against all bestseller criteria. Consider if the business rules should be adjusted for this type of product, or if the high sales are due to factors that don’t align with “bestseller” definition (e.g., deep discounting).

## 9. Next Steps

### Before This Step

- NOOS Paramount Sizes Calculation Module: We need the filtered and high-quality cleaned sales data from this module to ensure that our bestseller identification is based on reliable sales performance.
- Raw Data Collection: We need complete, accurate, and timely sales, product attribute, and store availability data from all relevant systems.

## After This Step

- **Bestseller Override Management:** The identified bestsellers are passed to this next module, allowing merchandising teams to apply manual adjustments or strategic overrides to the automated recommendations.
- **Inventory Planning & Replenishment:** They need the final, actionable list of bestsellers to place accurate, expedited purchase orders with suppliers and manage warehouse stock efficiently, preventing stockouts on hot items.
- **Marketing & Visual Merchandising:** They need this list to prioritize bestsellers for marketing campaigns, store displays, and online promotions, capitalizing on their high demand.

## 10. Common Questions

Q: How often are bestseller styles identified by this module? A: Bestseller styles are typically identified during each full run of the NOOS process, which usually happens daily or weekly. This ensures the system continuously adapts to changing customer preferences and sales trends, keeping the bestseller list current and responsive.

Q: Why isn't a particular style that seems to sell very well identified as a bestseller?

A: A style might sell well but not meet all the stringent bestseller criteria. For example, its sales might be driven by very deep discounts (exceeding the average discount benchmark), it might not have been live for enough days to establish consistent performance, or its revenue contribution might not be within the "Top X% Revenue Contribution" for its specific product grouping.

Q: How does this module prevent too many styles from being labeled as bestsellers at once? A: The module enforces a "Maximum Bestseller Styles Limit" and requires styles to meet a "Top X% Revenue Contribution" threshold. These controls ensure that the focus remains on a concentrated set of truly top-performing items, preventing the business from over-investing or diluting its inventory strategy across too many "hot" products.

Q: Does this module consider seasonality in identifying bestsellers? A: While the module doesn't have explicit seasonal logic, the "Bestseller Analysis Period" can be adjusted to a shorter duration (e.g., 15-30 days) to focus on recent trends, allowing it to adapt to changing seasonal demand patterns and identify styles that are currently popular.

Q: What is the direct financial impact of using this module for our business? A: By quickly and accurately identifying bestsellers, this module enables the business to capitalize on immediate sales opportunities, reduce lost sales from stockouts on high-demand items, and optimize inventory investment by prioritizing products with proven high velocity and profitability potential.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Identifies styles with exceptionally high sales velocity for immediate merchandising focus.



- **Main Input:** Cleaned sales data, product attributes, and style availability information.
  - **Main Output:** A definitive list of identified bestseller styles with performance metrics.
  - **Warning Signs:** Unexpectedly few or too many bestsellers identified, or high-selling items being excluded from the bestseller list without clear reason.
  - **Key Settings:** Bestseller Analysis Period, Top X% Revenue Contribution, Minimum Bestseller Units Sold.
- 

**Need Help?** Contact your Inventory Planning or Merchandising Operations team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 5: NOOS Output Override and Generation Module

**Last Updated:** 2025-05-23

## 1. Business Objective

This module applies brand-defined manual adjustments to the automated NOOS theme assignments (Core, Bestseller, Fashion) for specific styles, ensuring that human merchandising expertise can fine-tune automatically generated classifications. It further refines bestseller identification based on detailed revenue and discount analysis. Finally, this module generates and saves various comprehensive output reports, providing valuable insights into the final NOOS style classifications for decision-makers.

**Why This Matters:** This module is critical because it ensures that the final NOOS classifications are not only data-driven but also align with strategic brand initiatives, promotions, or market insights that automated systems cannot capture. It provides the flexibility to adapt to real-world scenarios while also producing robust, accessible reports that empower inventory planners and merchandisers to make the best possible decisions, ultimately maximizing sales and maintaining strong customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Units Sold by Style:** We need this to understand the volume and velocity of sales for individual products, which is crucial for refining bestseller status.
- **Revenue by Style:** We need this to assess the financial contribution of each style, ensuring that our final bestseller designations prioritize the most valuable items.
- **Discount Value by Style:** We need this to understand if a style's sales are primarily driven by deep discounts, allowing us to identify truly high-demand items versus promotional ones.

## Product Data

- Current Product Classifications (Core, Bestseller, Fashion): We need the initial, automatically generated classifications from previous modules to serve as the baseline for refinement and overriding.
- Product Attributes (e.g., Category, Subcategory, Brand): We need these details to ensure that bestseller refinement and override rules are applied within relevant product groupings.

## Operational Data

- Brand-Defined Override Rules: We need the specific instructions from merchandising teams detailing which styles should have their classifications manually changed (e.g., promote a “Fashion” item to “Bestseller”).
- Style Live Days: We need to know how many days each style has been actively available for sale to ensure that any bestseller refinement is based on sufficient market exposure.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Bestseller Refinement Minimum Live Days	15 days	The minimum number of days a style must have been actively available for sale within the analysis period to be considered for additional bestseller designation in this module.	To ensure that styles are only identified as bestsellers if they have demonstrated consistent performance over a significant period.
Discount Sensitivity Factor	0.8	A multiplier applied to the average discount benchmark for a product group, influencing how tolerant the system is of discounted sales for a style to be considered a bestseller.	If you want to be more lenient (increase value) or stricter (decrease value) about including highly discounted items as bestsellers.
Manual Override Precedence	Always	Determines that any manual, brand-defined override will	This setting is fundamental to the module’s purpose and is rarely

Setting Name	Default Value	Description	When to Change
		always take precedence over the automatically generated classification for a specific style.	changed; it ensures human expertise can always fine-tune recommendations.
Relative Revenue Performance Threshold	At or Above Average	The requirement that a style's average daily revenue per store must be at or above the average for similar styles within its category to be considered for additional bestseller status.	This is a built-in criterion that helps ensure only truly strong performers are considered; adjustments require careful consideration of overall strategy.

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

```

 A[Initial NOOS Style Classifications & Cleaned Sales Data] -->
 B[Calculate Refinement Benchmarks]
 B --> C[Identify Additional Bestsellers (Refinement)]
 C --> D[Apply Brand-Defined Overrides]
 D --> E[Final NOOS Style Classifications]
 E --> F[Generate Denormalized Output Reports]

```

1. Start with → Initial NOOS Style Classifications and Cleaned Sales Data
2. Then → Calculate Refinement Benchmarks (e.g., average revenue, average discount)
3. Next → Identify Additional Bestsellers through detailed refinement
4. Next → Apply Brand-Defined Manual Overrides
5. Next → Final NOOS Style Classifications are established
6. Finally → Generate various Denormalized Output Reports

## 5. Step-by-Step Actions

### 1. Prepare Data for Bestseller Refinement

- What we do: *The module gathers the latest cleaned sales data and the previously identified style classifications. It also calculates the number of days each style has been actively available in stores during the analysis period.*

- Why we do it: *This ensures that the subsequent bestseller identification is based on the most accurate and relevant performance data, considering the actual market exposure of each product.*
- 2. Calculate Refinement Benchmarks**
- What we do: *The system computes average daily revenue per store for styles within each product category. It also determines an average discount benchmark for different product groupings (e.g., category, subcategory, brand segment, price bucket), adjusted by the “Discount Sensitivity Factor.”*
  - Why we do it: *These benchmarks provide the necessary context to identify additional bestsellers. A style is only considered a true bestseller if its performance is exceptional compared to its peers and not solely driven by deep discounts.*
- 3. Identify Additional Bestsellers**
- What we do: *The module evaluates each style that is currently not a Core or Bestseller (i.e., classified as Fashion). It checks if these styles meet rigorous criteria, such as having an average daily revenue per store at or above the category benchmark, an average discount at or below the adjusted group benchmark, and having been live for at least the “Bestseller Refinement Minimum Live Days.”*
  - Why we do it: *This step ensures that any previously missed high-performing “Fashion” items are elevated to “Bestseller” status, allowing the business to quickly capitalize on emerging trends and maximize revenue from these newly identified hot products.*
- 4. Apply Brand-Defined Overrides**
- What we do: *The module takes all the manual override instructions provided by merchandising teams. For any style listed in these overrides, its classification is immediately updated to the specified theme (e.g., from “Core” to “Fashion” or “Bestseller” to “Core”). The “Manual Override Precedence” ensures these manual changes always take priority.*
  - Why we do it: *This is a crucial step for flexibility, enabling merchandising to adjust classifications based on strategic promotions, visual merchandising needs, or unique market insights that automated systems cannot anticipate, aligning inventory with business strategy.*
- 5. Generate Denormalized Output Reports**
- What we do: *After all classifications are finalized, the module compiles various detailed and summary reports. These reports integrate style classifications with performance metrics from both Core and Bestseller analyses into easy-to-use tables, such as the `ExportNoosOutputRow`, `DenormalizedNoosOutputRow`, and `DenormalizedNoosSummaryRow` (using business-friendly terms).*
  - Why we do it: *These comprehensive reports provide a single, unified view of all NOOS-related information. They eliminate the need to combine data from multiple sources, making it significantly easier for inventory planners, merchandisers, and analysts to access, understand, and act upon the final recommendations.*
- 6. Persist Final Outputs**
- What we do: *The module saves all the finalized style classifications and generated denormalized reports to the database, making them readily available for other systems and users.*
  - Why we do it: *This ensures that the final, actionable insights are stored securely and can be accessed by downstream systems (e.g., order management, allocation) and various teams, providing a consistent and reliable source of truth for NOOS inventory management.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A new “Athletic Jogger” style was initially classified as “Fashion” by the automated system because it didn't have enough history to be a bestseller, and its sales consistency wasn't high enough for “Core.”
- Over the past 15 days (meeting “Bestseller Refinement Minimum Live Days”), this jogger has seen a sudden surge in sales across stores, generating an average daily revenue per store 15% higher than the average for the “Athletic Wear” category.
- Its average discount over this period was 5%, which is well below the 10% average discount for its product grouping, even with the “Discount Sensitivity Factor” applied.
- Separately, the merchandising team wants to promote a specific “Classic White T-Shirt” (currently classified as “Core”) to “Bestseller” for a planned “Summer Essentials” marketing campaign next week.

### What Happens

1. First, the module **prepares** the latest sales data for the “Athletic Jogger” and the “Classic White T-Shirt,” along with their current classifications.
2. Then, it **calculates** the category benchmarks for “Athletic Wear” and “T-Shirts,” taking into account average revenue per day per store and adjusted average discount levels.
3. Next, it **identifies** additional bestsellers. The “Athletic Jogger” meets all the criteria: it's been live for more than 15 days, its daily revenue is significantly above the category average, and its discount is low compared to its peers. Therefore, the “Athletic Jogger” is reclassified from “Fashion” to “Bestseller.”
4. Following this, the module **applies** the brand-defined overrides. It finds the manual instruction to change the “Classic White T-Shirt” from “Core” to “Bestseller.” Due to the “Manual Override Precedence” setting, this manual instruction is executed, and the T-shirt's classification is updated.
5. Finally, the module **generates** and **persistent** the various denormalized output reports, showing both the “Athletic Jogger” and the “Classic White T-Shirt” as “Bestsellers” in the final classifications.

### Final Outcome

The inventory planning team receives comprehensive reports that reflect the immediate sales surge of the “Athletic Jogger” and the strategic marketing push for the “Classic White T-Shirt.” Both are now correctly classified as “Bestsellers,” triggering appropriate inventory actions like expedited replenishment and increased allocation. This ensures the business capitalizes on both organic high demand and planned promotions, driving maximum revenue.

**Key Takeaway:** This module provides essential flexibility, combining data-driven refinement with strategic manual adjustments, to ensure that NOOS classifications accurately reflect both current sales performance and overarching business goals.

## 7. Results

This step produces:

### Main Outputs

- **Final NOOS Style Classifications:** The definitive, adjusted list of all products categorized as Core, Bestseller, or Fashion, reflecting both automated analysis and manual overrides.
- **Comprehensive NOOS Output Table (ExportNoosOutputRow):** A detailed report combining all classification, performance, and operational data for each style, ready for export and analysis.
- **Detailed Style Classification Report (DenormalizedNoosOutputRow):** An in-depth report showing all contributing factors (e.g., revenue, discounts, live days) that led to each style's final NOOS classification.
- **NOOS Program Summary Report (DenormalizedNoosSummaryRow):** A high-level overview providing aggregated insights into the overall NOOS program performance, such as total styles by theme and their collective revenue contribution.
- **Detailed Core Style Performance Report (DenormCoreRow):** Specific performance metrics and insights focused exclusively on products designated as Core styles.
- **Detailed Bestseller Style Performance Report (DenormBestsellerRow):** Specific performance metrics and insights focused exclusively on products designated as Bestseller styles.

### Business Reports

- **Daily Final NOOS Classification Report:** Used by inventory planners to get the definitive classification for all styles, guiding immediate replenishment and allocation decisions.
- **Weekly Merchandising Strategy Review:** Used by merchandising teams to review the impact of their manual overrides and the overall effectiveness of the NOOS program, informing future buying and promotional strategies.
- **Data Integrity Check Report:** Used by data quality teams to verify the consistency and accuracy of the final, denormalized output data.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will either halt processing until data is available or proceed with the last complete dataset, potentially generating outdated or inaccurate classifications and reports.	Immediately investigate the data source (e.g., sales system, product master) to restore the feed and rerun the module.

Situation	What Happens	What You Should Do
Conflicting manual override rules	If multiple manual overrides are provided for the same style with different themes, the system will apply the rule based on a predefined priority (e.g., the most recent valid override), potentially with a warning.	Review the override processes and communications to ensure clear instructions and prevent contradictory manual inputs.
No manual overrides provided	The module will complete its run, and all style classifications will solely reflect the automated analysis from previous steps, without any manual adjustments.	This is expected behavior if no manual intervention is needed. If overrides were intended, investigate why they were not submitted.
All styles are overridden to a single theme	This indicates a misconfiguration in the manual override inputs, leading to a loss of valuable segmentation.	Review the manual override file immediately to correct the input and ensure only specific, intended styles are overridden.

## 9. Next Steps

### Before This Step

- **Initial NOOS Style Classifications:** We need the preliminary classifications (Core, Bestseller, Fashion) as generated by previous modules to serve as the baseline for refinement and overrides.
- **Cleaned Sales Data:** We need validated, high-quality sales data (from the NOOS Paramount Sizes Calculation Module) to ensure accurate bestseller refinement.
- **Brand-Defined Override Inputs:** We need the specific manual instructions from merchandising teams detailing any desired changes to style classifications.

### After This Step

- **Inventory Planning & Ordering:** They need the final, actionable NOOS classifications and detailed reports to place accurate purchase orders, manage warehouse stock, and prioritize replenishment for all styles.
- **Store Allocations:** They need the updated classifications to ensure the right products, particularly Core and Bestseller items, are distributed effectively to individual stores.
- **Marketing & Merchandising Teams:** They use the final classifications to prioritize products for marketing campaigns, promotional efforts, and in-store visual merchandising, maximizing the impact of top-performing items.



## 10. Common Questions

Q: Why did a style's classification change even after the NOOS Bestseller Style Identification Module already ran? A: This module performs two key functions that can change classifications: it further refines bestseller identification based on detailed revenue and discount analysis, potentially elevating "Fashion" items, and it applies any manual brand-defined overrides. Both these actions can alter a style's final classification.

Q: Can we manually influence or override the final NOOS recommendations generated by this process? A: Yes, absolutely. This module is specifically designed to incorporate brand-defined overrides (`NoosOverrideRow`). Merchandising teams can submit instructions to manually change a style's theme (e.g., Core, Bestseller, Fashion) based on strategic decisions, promotions, or unique market insights.

Q: How quickly are the manual overrides reflected in the system and subsequent inventory decisions? A: Manual overrides are applied during each run of this module, which typically occurs daily. This means that once an override is submitted and processed, the updated classification will be reflected in the next set of inventory recommendations and reports, ensuring a rapid response to business needs.

Q: What reports should I use to see the final NOOS classification for all our products? A: The "Comprehensive NOOS Output Table" (corresponding to `ExportNoosOutputRow`) and the "Detailed Style Classification Report" (corresponding to `DenormalizedNoosOutputRow`) generated by this module provide the most complete and accurate final classifications for all styles, along with their underlying performance data.

Q: How does this module contribute to reducing lost sales and improving customer satisfaction? A: By allowing manual adjustments and refining bestseller identification, this module ensures that inventory decisions align perfectly with both data-driven insights and strategic business priorities. This precision helps prevent stockouts on high-demand items (both identified automatically and strategically overridden) and ensures that the right products are always available for customers, directly translating to fewer missed sales and a better shopping experience.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Refines bestseller identification and applies brand-defined overrides to finalize NOOS style classifications, generating comprehensive output reports.
  - **Main Input:** Initial NOOS style classifications, cleaned sales data, and manual override rules.
  - **Main Output:** Final NOOS style classifications, comprehensive NOOS output tables, and detailed denormalized reports.
  - **Warning Signs:** Unexpected or illogical style classifications after processing, or reports that don't match business expectations.
  - **Key Settings:** Bestseller Refinement Minimum Live Days, Discount Sensitivity Factor, Manual Override Precedence.
-

**Need Help?** Contact your Inventory Planning or Merchandising Operations team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# noos FAQ

**Version:** 1.0

**Last Updated:** 2025-05-23

**Purpose:** This FAQ provides clear, business-focused answers regarding the noos algorithm, its functionality, inputs, outputs, and how it aids in optimizing product assortment and inventory.

---

## General

**Q:** What is the noos algorithm for? **A:** The noos algorithm classifies products (referred to as “styles”) into specific themes – “Core,” “Bestseller,” and “Fashion” – to help optimize inventory management, improve sales, and ensure that key products are consistently in stock.

**Q:** What is the difference between “Core” and “Bestseller” styles? **A:** “Core” styles are fundamental, stable products with consistent sales performance and typically lower discount rates, meant to be “Never Out of Stock.” “Bestseller” styles are products with high sales velocity and significant revenue contribution, indicating strong market demand.

**Q:** How does noos help us manage our inventory better? **A:** By classifying styles, noos helps determine the OD (Optimum Depth), OW (Optimum Width), and ISS (Ideal Size Set) for products within specific AGs (Attribute Groups). This leads to more precise stock planning, reduced stockouts of popular items, and better allocation of resources.

**Q:** What is an Attribute Group (AG)? **A:** An Attribute Group is a structured combination of various product attributes (like category, brand, gender, and the assigned NOOS theme). It creates a framework for analyzing product performance and inventory requirements across sets of items that share common characteristics.

**Q:** Can noos classifications be manually adjusted? **A:** Yes, the NOOS Output Override and Generation Module allows brand teams to apply specific overrides to the automatically generated theme assignments (Core, Bestseller, Fashion) for individual styles, providing flexibility and business control.

**Q:** What does ROS (Rate of Sale) mean in the context of noos? **A:** ROS (Rate of Sale) is a key metric calculated by the NOOS Core Style Identification Module that measures how quickly products are sold over a period (e.g., units per week), crucial for identifying stable and high-performing styles.

## Configuration & Settings

**Q:** How are the rules for identifying “Core” or “Bestseller” styles determined? **A:** The business rules and performance criteria for identifying “Core” and “Bestseller” styles, such as minimum quantity sold, consistent sales days, discount thresholds, and ROS coefficient variance, are defined within the NOOS Core Style Identification Module and NOOS Bestseller Style Identification Module, respectively.

**Q:** Are there limits to the number of styles that can be classified as “Core” or “Bestseller”? **A:** Yes, the algorithm applies upper caps based on criteria like revenue contribution to ensure a balanced and optimal portfolio of “Core” and “Bestseller” styles, preventing over-classification in any single category.

**Q:** How are “paramount sizes” identified by the system? **A:** “Paramount sizes” (the most important sizes) for product categories are identified by the NOOS Paramount Sizes Calculation Module based on their revenue contribution. This ensures that the size analysis is focused on the most impactful sizes.

**Q:** What criteria are used to ensure the quality of raw sales data before processing? **A:** The NOOS Paramount Sizes Calculation Module includes data cleansing steps. It filters out raw sales transactions that do not meet predefined criteria, such as minimum live store presence for the product or the availability of paramount sizes, ensuring high-quality input for subsequent calculations.

**Q:** Can we customize which attributes are used to form an Attribute Group (AG)? **A:** The Attribute Group (AG) Computation Module defines AGs by combining various product attributes like category, subcategory, brand, gender, price bucket, and the assigned NOOS theme. While the exact combination is part of the system’s logic, the underlying product attributes are standard inputs.

## Data & Inputs

**Q:** What types of data does the noos algorithm require to function? **A:** The noos algorithm primarily requires historical sales data, comprehensive product master data (including attributes like category, brand, and gender), information on product live days, and discount rates.

**Q:** Why are “paramount sizes” an important input? **A:** “Paramount sizes” represent the most significant sizes for a given product category based on their revenue contribution. Including them as input ensures that the algorithm’s analysis, particularly for ISS (Ideal Size Set), is focused on the sizes that drive the most sales.

**Q:** How is the “product live days” metric used? **A:** The “product live days” metric is crucial for accurately assessing a style’s performance. It helps normalize sales data over the active period of a product, preventing skewed classifications for items that have only been available for a short time.

**Q:** Is raw sales data used directly, or is it processed? **A:** Raw sales data undergoes significant processing. It’s cleansed by the NOOS Paramount Sizes Calculation Module and then aggregated to compute key metrics like ROS (Rate of Sale), average discount, and average revenue per day per style, which are then used for classification.

**Q:** What happens if there's incomplete sales data for a style? **A:** The system incorporates rules related to minimum live days and sales volume. If a style has insufficient data, it may not meet the criteria to be classified as Core or Bestseller, thus preventing misclassifications based on limited or unreliable information.

## **Process & Logic**

**Q:** What is the overall sequence of operations for the noos algorithm? **A:** The noos algorithm follows a structured, sequential pipeline orchestrated by the NOOS Orchestration Module. It starts with data preparation, then identifies Core styles, followed by Bestseller styles, applies any brand-defined theme overrides, and finally computes Attribute Group (AG)s and generates various output reports.

**Q:** How does the system determine if a style is "Core"? **A:** The NOOS Core Style Identification Module evaluates styles based on metrics like ROS, average discount, and product live days. It applies a comprehensive set of business rules including minimum quantity sold, consistent sales days, and specific discount thresholds to classify a style as "Core."

**Q:** What logic is used to identify "Bestseller" styles? **A:** The NOOS Bestseller Style Identification Module identifies "Bestseller" styles by calculating average revenue per day per style within product categories. It applies criteria related to sales contribution, average discount, minimum quantity sold, and live days to make this designation.

**Q:** Why are "theme overrides" necessary if the algorithm automatically classifies styles? **A:** "Theme overrides," managed by the NOOS Output Override and Generation Module, allow for manual adjustments to the automatically generated classifications. This is essential for incorporating unique brand strategies, unforeseen market insights, or specific business objectives that automated rules might not capture.

**Q:** At what point in the process are Attribute Groups (AGs) computed? **A:** Attribute Groups (AGs) are computed by the Attribute Group (AG) Computation Module after the styles have been finalized with their NOOS theme (Core, Bestseller, Fashion), including any overrides. This ensures the AGs accurately reflect the final product classifications.

**Q:** Does the algorithm consider promotional pricing or discounts during classification? **A:** Yes, the average discount applied to a style is a key metric considered in both Core Style Identification and Bestseller Style Identification. This ensures that classifications are based on a realistic view of profitability and sales performance.

## **Outputs & Results**

**Q:** What are the primary outputs generated by the noos algorithm? **A:** The main outputs include the definitive classification of each style (Core, Bestseller, or Fashion), the computed Attribute Groups (AGs), and a variety of comprehensive denormalized output reports, such as `ExportNoosOutputRow`, `DenormalizedNoosOutputRow`, and `DenormalizedNoosSummaryRow`.

**Q:** How can merchandisers leverage the noos output for business decisions? **A:** Merchandisers can use the noos output to make data-driven decisions on stock levels, product ranging (OD - Optimum Depth, OW - Optimum Width), and optimal size distributions (ISS - Ideal Size Set) within AGs. This helps ensure popular items are always available while minimizing excess inventory.

**Q:** What do OD, OW, and ISS represent in the output? **A:** \* OD (Optimum Depth): The ideal number of unique items (SKUs) to stock in a category or display. \* OW (Optimum Width): The ideal number of different categories or styles to offer. \* ISS (Ideal Size Set): The best combination of sizes to maximize sales and minimize stockouts for a particular style or AG. These metrics provide actionable insights for inventory optimization.

**Q:** Can I get a high-level summary of the noos classifications rather than detailed style-level data? **A:** Yes, the N00S Output Override and Generation Module specifically generates summary reports, such as DenormalizedNoosSummaryRow, which provide aggregated insights into the overall noos style classifications.

**Q:** Are the noos outputs suitable for real-time inventory adjustments? **A:** The noos outputs provide strategic guidance for inventory planning, product ranging, and assortment optimization (e.g., setting OD, OW, ISS). While foundational for inventory management systems, they are primarily for strategic planning rather than moment-to-moment real-time adjustments.

## Error Handling & Edge Cases

**Q:** How does the system handle potentially poor-quality or incomplete sales data? **A:** The N00S Paramount Sizes Calculation Module includes robust data cleansing capabilities. It filters out transactions that do not meet predefined quality criteria, such as insufficient live store presence or missing paramount size data, ensuring only reliable data is used for classification.

**Q:** What happens if a style has very limited sales history or 'live days'? **A:** The algorithm considers "product live days" and minimum sales volume as critical criteria. Styles with insufficient live days or very low sales might not meet the thresholds for classification as Core or Bestseller, preventing inaccurate classifications based on sparse data. Such styles would likely be categorized as "Fashion" by default.

**Q:** How are brand-defined overrides managed if they contradict the automated classification? **A:** Manual overrides applied through the N00S Output Override and Generation Module are designed to take precedence over the automated classifications. This ensures that specific business decisions or strategic insights can directly influence the final NOOS theme assignment.

**Q:** Is there a default classification for styles that don't fit into "Core" or "Bestseller"? **A:** Yes, styles that do not meet the stringent criteria and performance thresholds for "Core" or "Bestseller" classification are typically designated as "Fashion" styles by default. This category often includes newer items, seasonal products, or those with more fluctuating sales patterns.

**Q:** What happens if a style initially classified as “Core” later experiences a significant sales decline? **A:** The noos algorithm is designed to be re-run periodically. During subsequent runs, if a “Core” style’s performance metrics (e.g., ROS, discount rate, sales consistency) fall below the defined thresholds, it may be reclassified to “Bestseller” or “Fashion” based on the updated data and rules.

---

**Need More Help?**

For further assistance, please contact the Product Analytics Support Team.

# noos Documentation

Version: 1.0 Last Updated: 2025-05-23

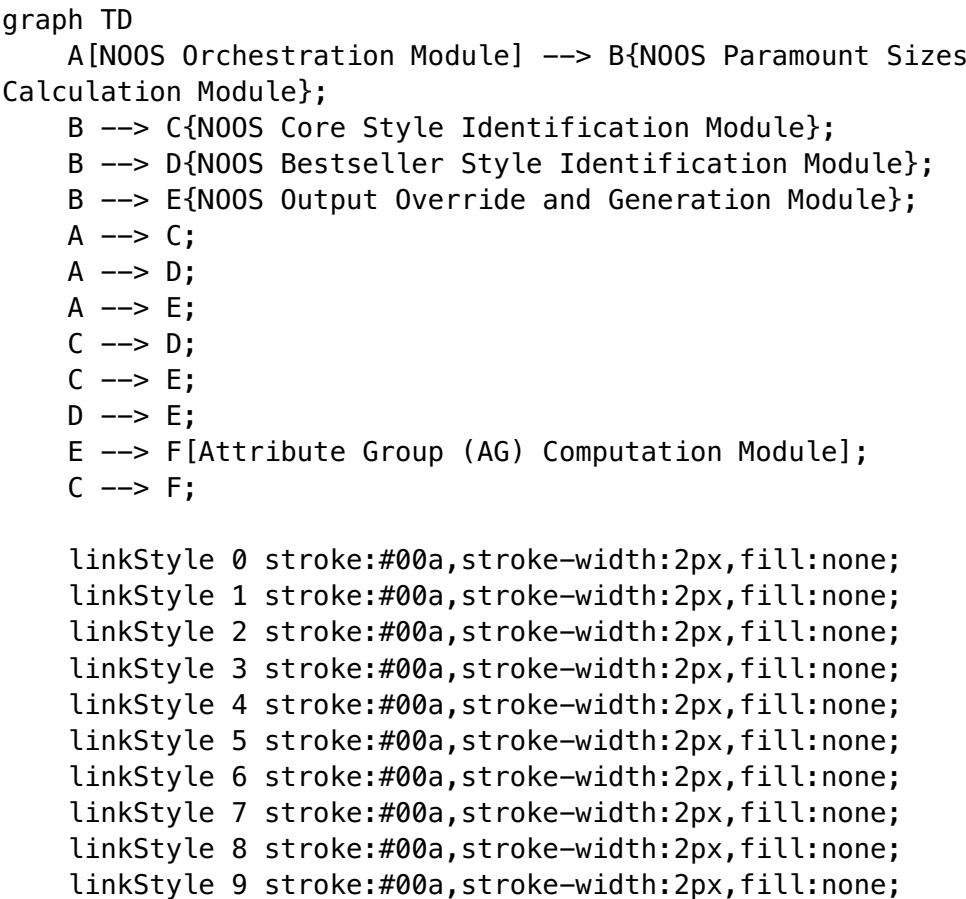
---

## 2. Executive Summary

The NOOS (Never Out Of Stock) algorithm is designed to optimize product assortment by systematically identifying and managing “Core” and “Bestseller” styles. By leveraging sales data and applying sophisticated business rules, it ensures key products are always available, leading to increased sales, reduced stockouts, and improved inventory efficiency across product categories.

---

## 3. Architecture Overview





```
linkStyle 10 stroke:#00a,stroke-width:2px,fill:none;
linkStyle 11 stroke:#00a,stroke-width:2px,fill:none;
```

- **NOOS Orchestration Module:** Manages the overall execution flow, orchestrating the sequential processing of data preparation, style identification, overrides, and final output generation.
  - **NOOS Paramount Sizes Calculation Module:** Identifies the most important sizes for product categories based on revenue and cleanses sales data for subsequent NOOS calculations.
  - **NOOS Core Style Identification Module:** Identifies fundamental, stable “Core” styles by applying comprehensive business rules on sales performance and consistency.
  - **NOOS Bestseller Style Identification Module:** Identifies high-velocity “Bestseller” styles using criteria related to sales contribution, discount, and live days.
  - **NOOS Output Override and Generation Module:** Applies manual overrides to theme assignments and generates detailed denormalized output reports for NOOS style classifications.
  - **Attribute Group (AG) Computation Module:** Defines and computes Attribute Groups by combining product attributes and assigned NOOS themes, structuring data for OD, OW, and ISS analysis.
- 

## 4. Prerequisites & Setup

To run the noos project, ensure the following are installed and configured:

- **Java Development Kit (JDK):** Version 17 or higher.
- **Apache Maven:** Version 3.8 or higher.

### Build Commands:

1. **Clone the repository:** `git clone <repository-url> cd noos`
2. **Build the project:** `mvn clean install`

### Running the Orchestrator:

The NOOS Orchestration Module is typically initiated through a scheduled job or an API call that triggers its `execute()` method. Assuming a Spring Boot application, you would run the main application class:

```
java -jar target/noos-1.0.jar
```

This will trigger the NOOS Orchestration Module which then orchestrates the execution of all sub-modules in the defined order.

---

# 5. Folder & File Structure

```
noos/
├─ module-group/ # Contains the orchestrator module
 responsible for the overall execution flow.
├─ submodules/ # Houses individual business logic
 modules (e.g., Core, Bestseller, Paramount Sizes).
├─ args.properties # Global configuration parameters for
 the NOOS pipeline.
├─ common-data.json # Schema definition for shared data
 structures used across modules.
├─ outputs/ # Directory where all generated NOOS
 reports and data files are stored.
```

# 6. Global Configuration

The args.properties file defines global parameters influencing the entire NOOS calculation pipeline.

Setting Name	Default Value	Description	Valid Range/ When to Change
noos.min.live.stores	5	Minimum number of live stores required for a product to be considered in sales data cleansing.	Integer (1-20). Adjust to refine data quality based on store network size; lower values include more data, higher values increase data reliability.
noos.core.min.quantity.sold	100	Minimum units sold for a style to be considered for Core classification.	Integer (10-1000). Increase for stricter Core identification, decrease for broader inclusion.
noos.bestseller.revenue.share.threshold	0.01	Minimum revenue share for a style within its category to	Decimal (0.001-0.05). Increase to focus on top performers,

Setting Name	Default Value	Description	Valid Range/ When to Change
		be a Bestseller.	decrease to expand the Bestseller pool.
noos.max.core.styles.per.category	20	Maximum number of Core styles allowed per product category to maintain portfolio balance.	Integer (5-50). Defines the OD for Core styles. Change to manage the depth of the Core assortment per category.
noos.max.bestseller.styles.per.category	15	Maximum number of Bestseller styles allowed per product category.	Integer (5-30). Defines the OD for Bestseller styles. Adjust to control the breadth of the Bestseller offering.
noos.output.path	data/ output	Directory path where all final NOOS output reports will be saved.	Valid file path. Change to redirect outputs to a different storage location or system, e.g., for integration with BI tools.
noos.paramount.size.min.revenue.share	0.02	Minimum revenue share for a size within a category to be considered a “paramount size.”	Decimal (0.01-0.1). Adjust to define the significance threshold for sizes, impacting the ISS.

## 7. Shared Data Model

The `common-data.json` schema defines the structure for data exchanged and used across the NOOS modules.

Field	Type	Description
transactionId	String	Unique identifier for a sales transaction.
productId	String	Unique identifier for a specific product SKU.
styleId	String	Unique identifier for a product style (e.g., “Men’s T-Shirt Basic”). A style can have multiple SKUs (sizes, colors).
categoryId	String	Identifier for the product category (e.g., “Apparel - Men’s Tops”).
size	String	Size of the product SKU (e.g., “S”, “M”, “L”, “XL”).
color	String	Color of the product SKU.
revenue	Number	Total revenue generated from the sale of this product SKU in a transaction.
quantitySold	Integer	Number of units sold for this product SKU in a transaction.
discountPercentage	Number	Average discount percentage applied to the product SKU during the sales period.
liveDays	Integer	Number of days a product SKU has been actively available for sale in stores.
storeId	String	Identifier for the store where the transaction occurred.
transactionDate	String	Date of the sales transaction (YYYY-MM-DD format).
noosTheme	String	The assigned NOOS classification for a style (e.g., “CORE”, “BESTSELLER”, “FASHION”). This is populated and updated by relevant modules.
attributeGroup	String	A computed composite identifier representing a unique combination of attributes for a style, used for OD, OW, and ISS analysis.
rateOfSale	Number	The ROS for a specific style (units/week).

## 8. Submodule Chapters

### 8.1 NOOS Orchestration Module

#### 8.1.1 Module Name & Business Objective

The NOOS Orchestration Module manages the overall execution flow of the NOOS (New Order Old Stock / Never Out of Stock) project. It orchestrates the sequential execution of various sub-modules responsible for data preparation, core style identification, bestseller style identification, theme overrides, and final output generation, ensuring a structured and dependent processing pipeline. This module's objective is to provide a reliable and consistent sequence of operations, ensuring that the NOOS analysis is performed correctly and completely.

#### 8.1.2 Required Information

- **Sales Data:** Raw sales transaction data, including product IDs, quantities sold, revenue, store IDs, and transaction dates.
- **Product Data:** Master data for products, including style IDs, categories, and attributes.
- **Operational Data:** Configuration settings from `args.properties` and the `common-data.json` schema.

#### 8.1.3 Business Settings

No specific business settings are directly configured within this orchestrator itself, as it primarily manages the flow. Its behavior is implicitly controlled by the settings of the modules it orchestrates and the overall `args.properties`.

#### 8.1.4 Process Flow

graph TD

```
A[Start Orchestration] --> B(N00S Paramount Sizes Calculation Module);
```

```
B --> C(N00S Core Style Identification Module);
```

```
C --> D(N00S Bestseller Style Identification Module);
```

```
D --> E(N00S Output Override and Generation Module);
```

```
E --> F(Attribute Group (AG) Computation Module);
```

```
F --> G[End Orchestration];
```

#### 8.1.5 Step-by-Step Actions

1. **Initialize NOOS Data:** *The orchestrator prepares a shared data object to hold intermediate results and classifications across modules.*
2. **Execute Paramount Sizes Module:** *It triggers the N00S Paramount Sizes Calculation Module to clean data and identify critical sizes, ensuring subsequent modules operate on high-quality, relevant sales information.*

3. **Execute Core Style Identification Module:** *It then runs the NOOS Core Style Identification Module to classify stable products, setting the foundation for the NOOS theme assignments.*
4. **Execute Bestseller Style Identification Module:** *Subsequently, it executes the NOOS Bestseller Style Identification Module to identify high-performing items, building upon the core classifications.*
5. **Execute Output Override Module:** *It proceeds to run the NOOS Output Override and Generation Module to apply any manual adjustments and finalize the theme assignments, allowing for business expert input.*
6. **Execute AG Computation Module:** *Finally, it triggers the Attribute Group (AG) Computation Module to group products by common attributes and their finalized NOOS themes, preparing for inventory and assortment planning.*
7. **Signal Completion:** *The orchestrator indicates successful completion of the entire NOOS analysis pipeline.*

## 8.1.6 Sample Scenario

Imagine a new product category “Women’s Denim Jeans” has just been introduced. The NOOS Orchestration Module is triggered at the end of the week. It first ensures sales data is clean and identifies paramount sizes (e.g., 28W/30L) for jeans. Then, it classifies stable, consistent sellers as “Core” jeans styles. Next, it identifies high-volume, fast-moving styles as “Bestseller” jeans. If a specific “Core” style of jeans is manually flagged by a brand manager as “Fashion” for a special campaign, the override module handles this. Finally, the AG Computation Module groups all jeans by their NOOS theme and other attributes (e.g., “Women’s Denim Jeans - Core - Blue - Skinny”). **Key Takeaway:** The orchestrator ensures that complex, multi-step business logic for product classification is executed reliably and in the correct sequence.

## 8.1.7 Results

- A successful execution of the entire NOOS pipeline.
- Populated shared data structures (noosData) with cleaned sales, style classifications (Core, Bestseller, Fashion), and other computed metrics.
- Finalized output reports and data files (e.g., ExportNoosOutputRow, DenormalizedNoosOutputRow) in the outputs/ directory.

## 8.1.8 Unusual Cases

- **Missing or delayed data feed:** The orchestrator will likely fail at the first module (NOOS Paramount Sizes Calculation Module), as it won’t find the necessary input sales data. This will prevent any further processing.
- **Module failure:** If any sub-module encounters an error (e.g., data inconsistency, out-of-memory), the orchestration will stop, requiring investigation and resolution within that specific module.
- **Configuration errors:** Incorrect parameters in args.properties might lead to unexpected behavior or incorrect classifications, although the orchestrator itself might complete successfully.

### 8.1.9 Next Steps

The results generated by the orchestrated pipeline are consumed by inventory planning systems, merchandising tools, and reporting dashboards to inform decisions on stocking levels (OD, OW, ISS), promotions, and product lifecycle management.

### 8.1.10 Common Questions

1. **Why did the NOOS run fail?** *Check the logs from the NOOS Orchestration Module to identify which sub-module failed, then investigate that specific module's logs.*
2. **How long does a full NOOS run take?** *Runtime depends on data volume; the orchestrator's logs provide total execution time, while individual module logs show their specific durations.*
3. **Can I run only a subset of the modules?** *The orchestrator is designed for full pipeline execution. Partial runs are not directly supported without modifying the orchestration logic.*
4. **How often should the NOOS be run?** *Typically weekly or bi-weekly, depending on sales velocity and business needs to keep classifications current.*
5. **Where can I find the final NOOS outputs?** *All final reports are stored in the outputs/ directory, as configured by `noos.output.path` in `args.properties`.*

### 8.1.11 Quick Reference

- **Purpose:** Manages sequential execution of NOOS analysis.
  - **Inputs:** Triggers from scheduler/API.
  - **Outputs:** Orchestrates generation of all NOOS reports.
  - **Key Dependencies:** All other NOOS sub-modules.
- 

## 8.2 NOOS Paramount Sizes Calculation Module

### 8.2.1 Module Name & Business Objective

The NOOS Paramount Sizes Calculation Module identifies “paramount sizes” (most important sizes) for product categories based on their revenue contribution. This module also cleanses raw sales data by filtering out transactions that do not meet predefined criteria for minimum live store presence or paramount size availability, providing high-quality input for subsequent NOOS calculations. Its business objective is to ensure that all subsequent calculations are based on relevant and significant sales data, preventing skewed results from incomplete or irrelevant transactions.

### 8.2.2 Required Information

- **Sales Data:** Raw sales transactions, including `productId`, `size`, `storeId`, `revenue`, and `transactionDate`.
- **Product Data:** Mapping from `productId` to `categoryId` and `styleId`.

- **Operational Data:** `noos.min.live.stores` and `noos.paramount.size.min.revenue.share` from `args.properties`.

### 8.2.3 Business Settings

- `noos.min.live.stores`: Defines the minimum number of stores a product must be live in for its sales data to be considered valid.
- `noos.paramount.size.min.revenue.share`: Sets the threshold for a size's revenue contribution within a category to be classified as "paramount".

### 8.2.4 Process Flow

graph TD

```
A[Raw Sales Data] --> B{Filter by Min Live Stores};
B --> C{Calculate Size Revenue Share per Category};
C --> D{Identify Paramount Sizes};
D --> E{Cleanse Sales Data by Paramount Size Presence};
E --> F[Cleaned Sales Data for NOOS];
```

### 8.2.5 Step-by-Step Actions

1. **Ingest Raw Sales Data:** *The module reads raw sales transactions from the data source, preparing them for initial filtering.*
2. **Filter by Live Stores:** *It removes sales data for products that have been active in fewer than `noos.min.live.stores`, ensuring analysis focuses on widely distributed items.*
3. **Calculate Size Revenue Share:** *For each `categoryId`, it calculates the revenue share of each size, identifying which sizes contribute most significantly to overall sales.*
4. **Identify Paramount Sizes:** *It flags sizes as "paramount" if their revenue share within their `categoryId` exceeds `noos.paramount.size.min.revenue.share`, establishing the ISS for that category.*
5. **Cleanse Sales Data for Paramount Sizes:** *It filters the sales data further, retaining only transactions where the size sold is a paramount size for its `categoryId`, ensuring that subsequent NOOS calculations are based on the most relevant size assortment.*
6. **Provide Cleaned Data:** *It makes the `cleanedSales` data available to other NOOS modules through the shared `noosData` object.*

### 8.2.6 Sample Scenario

For the "Women's Denim Jeans" category: \* Initial sales data shows sizes 24, 26, 28, 30, 32, 34. \* Sales for size 24 are filtered out because it was only live in 2 stores, below the `noos.min.live.stores` (e.g., 5). \* Revenue share calculation: Size 28 (30% share), Size 30 (25% share), Size 32 (18% share), Size 26 (12% share), Size 34 (10% share). \* With `noos.paramount.size.min.revenue.share` set to 0.15 (15%), Sizes 28, 30, and 32 are identified as Paramount Sizes. \* All sales transactions for Sizes 26 and 34 are then removed from the data used for subsequent NOOS calculations. **Key Takeaway:** By focusing on paramount sizes and widely



available products, this module provides a high-quality, relevant sales dataset, improving the accuracy of Core and Bestseller identifications and ensuring that inventory decisions are focused on the most impactful sizes.

### 8.2.7 Results

- A cleanedSales dataset, filtered by `noos.min.live.stores` and paramount size presence.
- A mapping of `categoryId` to its identified “paramount sizes.”
- These results are stored in the shared `noosData` object for use by subsequent modules.

### 8.2.8 Unusual Cases

- **Missing or delayed data feed:** If raw sales data is not available, the module will fail to process, and `cleanedSales` will be empty, leading to upstream failures.
- **Category with no paramount sizes:** If a category has very distributed sales across sizes, or low revenue, it might not identify any paramount sizes, leading to no sales data for that category in subsequent steps.
- **Extremely low `noos.min.live.stores`:** May lead to inclusion of unreliable data from very few stores, potentially skewing results.
- **Incorrect `noos.paramount.size.min.revenue.share`:** Too high a threshold may exclude important sizes; too low may include too many, diluting the concept of “paramount.”

### 8.2.9 Next Steps

The `cleanedSales` data and the identified `paramountSizes` are crucial inputs for the NOOS Core Style Identification Module, NOOS Bestseller Style Identification Module, and NOOS Output Override and Generation Module.

### 8.2.10 Common Questions

1. **Why are some sales missing from the final NOOS report?** *Check if the `productId` or `size` in those sales were filtered out by `noos.min.live.stores` or if the `size` was not identified as a paramount size for its category.*
2. **How are “paramount sizes” defined?** *They are sizes within a product category that contribute above a defined revenue share threshold (`noos.paramount.size.min.revenue.share`).*
3. **What if a new product category has no sales?** *It won’t have any paramount sizes identified, and no data for that category will be processed further until sales data becomes available.*
4. **Can I override the paramount sizes manually?** *Not directly within this module. Any manual adjustments would need to be applied downstream, possibly in the NOOS Output Override and Generation Module if applicable, or through a separate input feed.*
5. **How does this impact my ISS?** *The identified paramount sizes directly inform the Ideal Size Set (ISS) that should be prioritized for stocking to maximize sales efficiency.*

## 8.2.11 Quick Reference

- **Purpose:** Cleanses sales data and identifies paramount sizes.
  - **Inputs:** Raw sales data, `noos.min.live.stores`, `noos.paramount.size.min.revenue.share`.
  - **Outputs:** `cleanedSales` data, `paramountSizes` mapping.
  - **Key Metrics:** Revenue share by size.
- 

## 8.3 NOOS Core Style Identification Module

### 8.3.1 Module Name & Business Objective

The NOOS Core Style Identification Module identifies “Core” styles, which are fundamental, stable products. It computes key metrics such as ROS (Rate of Sale), average discount, and product live days for styles. It then applies a comprehensive set of business rules and performance criteria (e.g., minimum quantity sold, consistent sales days, discount thresholds, ROS coefficient variance) to classify styles as Core. The module also applies upper caps to ensure a balanced portfolio of Core styles based on revenue contribution. Its business objective is to accurately pinpoint the most consistent and reliable sellers that should always be in stock, supporting a stable foundation for the product assortment (OD).

### 8.3.2 Required Information

- **Sales Data:** `cleanedSales` data from the NOOS Paramount Sizes Calculation Module, including `productId`, `styleId`, `categoryId`, `revenue`, `quantitySold`, `discountPercentage`, and `liveDays`.
- **Product Data:** Style and category information.
- **Operational Data:** `noos.core.min.quantity.sold`, `noos.max.core.styles.per.category`, and various other core-related thresholds (e.g., for ROS variance, discount percentage) from `args.properties`.

### 8.3.3 Business Settings

- `noos.core.min.quantity.sold`: Minimum units sold for a style to be considered for Core.
- `noos.max.core.styles.per.category`: Upper cap on the number of Core styles per category.
- `noos.core.min.sales.days`: Minimum number of active sales days.
- `noos.core.max.avg.discount`: Maximum average discount allowed.
- `noos.core.max.ros.coefficient.variance`: Maximum allowable variance in ROS.

### 8.3.4 Process Flow

graph TD

A[Cleaned Sales Data] --> B{Calculate ROS, Avg Discount, Live Days per Style};

```

B --> C{Apply Core Business Rules};
C --> D{Rank Styles by Revenue per Category};
D --> E{Apply Upper Cap for Core Styles};
E --> F[Identified Core Styles];

```

### 8.3.5 Step-by-Step Actions

1. **Compute Style Metrics:** *It calculates the ROS, average discount, and total live days for each styleId based on the cleanedSales data, providing performance baselines.*
2. **Apply Core Business Rules:** *It filters out styles that do not meet strict criteria such as noos.core.min.quantity.sold, consistent sales days, and noos.core.max.avg.discount and noos.core.max.ros.coefficient.variance, identifying potential Core candidates.*
3. **Rank Potential Core Styles:** *It ranks the remaining potential Core styles by their total revenue contribution within each categoryId, preparing for the application of the upper cap.*
4. **Apply Core Style Upper Cap:** *It selects only the top noos.max.core.styles.per.category styles as official “Core” styles for each category, ensuring a balanced OD.*
5. **Update Style Themes:** *It marks the identified styles with the CORE theme in the shared styleThemeMap within noosData and persists CoreRow data to the database.*

### 8.3.6 Sample Scenario

For “Women’s Denim Jeans”: \* Style A (Classic Fit): ROS = 100 units/week, Avg Discount = 5%, Live Days = 365, consistent sales. \* Style B (Trendy Skinny): ROS = 120 units/week, Avg Discount = 25%, Live Days = 180, inconsistent sales. \* Style C (Seasonal Bootcut): ROS = 50 units/week, Avg Discount = 10%, Live Days = 90. \* Assume noos.core.min.quantity.sold is 1000, noos.core.max.avg.discount is 10%, noos.core.max.ros.coefficient.variance is low. \* Style A meets all criteria. Style B fails the discount and consistency criteria. Style C fails quantity sold and live days. \* If Style A is the only one meeting criteria and noos.max.core.styles.per.category for jeans is 1, then Style A is designated CORE. **Key Takeaway:** This module ensures that “Core” styles are genuinely stable, low-risk, high-volume products, forming the dependable backbone of the inventory (OD).

### 8.3.7 Results

- A list of CoreRow objects, each representing an identified Core style with its computed metrics.
- The styleThemeMap within noosData updated with CORE classifications for identified styles.
- Persisted CoreRow data to the database for later use.

### 8.3.8 Unusual Cases

- **Missing or delayed data feed:** Reliance on `cleanedSales` means an empty or incomplete dataset will result in no or inaccurate Core style identifications.
- **No styles meet Core criteria:** If business rules are too stringent, or sales performance is universally poor, a category might have zero Core styles, impacting OD.
- **Too many styles meet criteria:** The `noos.max.core.styles.per.category` cap ensures balance, but it means some otherwise “Core-worthy” styles might be excluded, which must be understood.
- **Sudden sales fluctuations:** Unforeseen market shifts or promotions could temporarily distort ROS or discount, leading to misclassification if the look-back window is too short.

### 8.3.9 Next Steps

The CORE classifications and associated metrics are passed to the NOOS Bestseller Style Identification Module and NOOS Output Override and Generation Module to inform their processes. The `CoreRow` data is also used by the Attribute Group (AG) Computation Module.

### 8.3.10 Common Questions

1. **Why isn't Style X a Core style?** *Review its ROS, average discount, quantity sold, and live days against the `noos.core.*` parameters in `args.properties`. It might have failed one of the criteria or been capped out by `noos.max.core.styles.per.category`.*
2. **How often are Core styles re-evaluated?** *This module runs with the overall NOOS pipeline, typically weekly, ensuring classifications are dynamic.*
3. **What if a Core style's performance declines?** *The next NOOS run will re-evaluate it. If it no longer meets criteria, it will be de-classified, potentially becoming “Fashion” or a different theme.*
4. **Can we manually add a style to Core?** *Not directly here. Manual overrides are handled by the NOOS Output Override and Generation Module as the final step in the main pipeline.*
5. **How does this affect my inventory levels?** *Core styles should receive priority for consistent stock levels and minimal stockouts, directly supporting the OD for that category.*

### 8.3.11 Quick Reference

- **Purpose:** Identifies stable, consistent “Core” products.
  - **Inputs:** `cleanedSales` data, `args.properties` (Core thresholds).
  - **Outputs:** `CoreRow` data, `styleThemeMap` (CORE themes).
  - **Key Metrics:** ROS, Avg Discount, Live Days, Quantity Sold.
-

## 8.4 NOOS Bestseller Style Identification Module

### 8.4.1 Module Name & Business Objective

The NOOS Bestseller Style Identification Module identifies “Bestseller” styles, focusing on products with high sales velocity. It calculates average revenue per day per style within product categories and applies a series of criteria related to sales contribution, average discount, minimum quantity sold, and live days to designate styles as Bestsellers. This module also enforces upper limits on the number of Bestseller styles to maintain a balanced and optimal product offering (OD). Its business objective is to highlight rapidly selling, high-impact products that warrant immediate attention for replenishment and promotional activities.

### 8.4.2 Required Information

- **Sales Data:** cleanedSales data (from NOOS Paramount Sizes Calculation Module), including productId, styleId, categoryId, revenue, quantitySold, discountPercentage, and liveDays.
- **Product Data:** Style and category information.
- **Core Style Data:** styleThemeMap from NOOS Core Style Identification Module (to respect existing CORE classifications).
- **Operational Data:** noos.bestseller.revenue.share.threshold, noos.max.bestseller.styles.per.category, and other bestseller-related thresholds from args.properties.

### 8.4.3 Business Settings

- noos.bestseller.revenue.share.threshold: Minimum revenue share for a style to be a Bestseller within its category.
- noos.max.bestseller.styles.per.category: Upper cap on the number of Bestseller styles per category.
- noos.bestseller.min.quantity.sold: Minimum units sold.
- noos.bestseller.max.avg.discount: Maximum average discount allowed.

### 8.4.4 Process Flow

graph TD

```
A[Cleaned Sales Data] --> B{Compute Revenue per Day per Style};
B --> C{Apply Bestseller Business Rules & Exclude CORE};
C --> D{Rank Styles by Revenue per Category};
D --> E{Apply Upper Cap for Bestseller Styles};
E --> F[Identified Bestseller Styles];
```

### 8.4.5 Step-by-Step Actions

1. **Compute Revenue per Day:** *It calculates the average revenue per day for each styleId based on cleaned sales and live days data, focusing on sales velocity.*

2. **Apply Bestseller Rules & Exclude Core:** *It filters out styles that do not meet bestseller criteria such as `noos.bestseller.min.quantity.sold`, `noos.bestseller.max.avg.discount`, and `noos.bestseller.revenue.share.threshold`. Crucially, it also excludes any styles already classified as *CORE*, ensuring no overlap in primary theme classifications.*
3. **Rank Potential Bestsellers:** *It ranks the remaining potential Bestseller styles by their total revenue contribution within each `categoryId`, prioritizing the highest impact items.*
4. **Apply Bestseller Upper Cap:** *It selects only the top `noos.max.bestseller.styles.per.category` styles as official “Bestseller” styles for each category, maintaining a balanced OD.*
5. **Update Style Themes:** *It marks the identified styles with the *BESTSELLER* theme in the shared `styleThemeMap` within `noosData` and persists `BestsellerRow` data to the database.*

### 8.4.6 Sample Scenario

For “Women’s Denim Jeans” (after Core identification): \* Style X (Popular High-Rise): Avg Revenue/Day = \$500, Qty Sold = 2000, Avg Discount = 8%. \* Style Y (Limited Edition): Avg Revenue/Day = \$700, Qty Sold = 500, Avg Discount = 30%. \* Style A is already CORE. \* Assume `noos.bestseller.revenue.share.threshold` is 0.01 (1%), `noos.bestseller.min.quantity.sold` is 1000, `noos.bestseller.max.avg.discount` is 15%. \* Style X meets all criteria and is not CORE. Style Y fails the quantity sold and discount criteria. \* If `noos.max.bestseller.styles.per.category` for jeans is 1, and Style X is the only one meeting criteria, then Style X is designated BESTSELLER. **Key Takeaway:** This module identifies products with high sales velocity that are not necessarily long-term Core items, enabling agile management of fast-moving inventory (OD).

### 8.4.7 Results

- A list of `BestsellerRow` objects, each representing an identified Bestseller style with its computed metrics.
- The `styleThemeMap` within `noosData` updated with BESTSELLER classifications for identified styles.
- Persisted `BestsellerRow` data and populated `styleLiveDaysMap` for use by downstream modules.

### 8.4.8 Unusual Cases

- **Missing or delayed data feed:** An empty or incomplete `cleanedSales` dataset will result in no or inaccurate Bestseller style identifications.
- **Overlapping criteria:** While CORE styles are excluded, if rules are poorly defined, a style might marginally miss CORE but be a strong BESTSELLER, or vice versa.
- **Dominant styles:** A single style might be such a strong bestseller that it “crowds out” other potential bestsellers when the `noos.max.bestseller.styles.per.category` cap is applied, impacting OW.

- **Seasonal fluctuations:** High-velocity seasonal items might qualify as Bestsellers during their peak, requiring manual review or seasonality adjustments if they are not to be replenished long-term.

## 8.4.9 Next Steps

The BESTSELLER classifications are passed to the NOOS Output Override and Generation Module for final adjustments and reporting.

## 8.4.10 Common Questions

1. **Why is Style Y not a Bestseller, even though it sold a lot last week?** *It might have exceeded the `noos.bestseller.max.avg.discount`, or fallen below the `noos.bestseller.min.quantity.sold` threshold, or its revenue share was too low compared to other styles in the category.*
2. **Can a style be both Core and Bestseller?** *No, the module explicitly excludes Core styles from Bestseller consideration to maintain distinct classifications.*
3. **How does the `noos.max.bestseller.styles.per.category` affect assortment?** *It limits the OW within a category, ensuring focus on the absolute top performers and preventing over-reliance on too many Bestsellers.*
4. **What if a Bestseller has low stock?** *This identification signals immediate need for replenishment to prevent stockouts and maximize sales, impacting inventory planning.*
5. **How quickly do Bestseller classifications change?** *They are re-evaluated with each NOOS run, so they can be quite dynamic, reflecting recent sales trends.*

## 8.4.11 Quick Reference

- **Purpose:** Identifies high-velocity “Bestseller” products.
  - **Inputs:** `cleanedSales` data, `styleThemeMap` (from Core), `args.properties` (Bestseller thresholds).
  - **Outputs:** `BestsellerRow` data, `styleThemeMap` (BESTSELLER themes).
  - **Key Metrics:** Avg Revenue per Day, Quantity Sold, Avg Discount.
- 

# 8.5 NOOS Output Override and Generation Module

## 8.5.1 Module Name & Business Objective

The NOOS Output Override and Generation Module applies brand-defined overrides to the NOOS theme assignments (Core, Bestseller, Fashion) for specific styles, allowing manual adjustments to automatically generated classifications. It further refines bestseller identification based on detailed revenue and discount analysis. Finally, this module generates and persists various denormalized output reports, such as `ExportNoosOutputRow`, `DenormalizedNoosOutputRow`, and `DenormalizedNoosSummaryRow`, providing comprehensive insights into NOOS style classifications. Its business objective is to provide a final layer of business control over automated classifications and to produce actionable, comprehensive reports for various stakeholders.

## 8.5.2 Required Information

- **Sales Data:** cleanedSales data (from N00S Paramount Sizes Calculation Module).
- **Core Style Data:** CoreRow data (from N00S Core Style Identification Module).
- **Bestseller Style Data:** BestsellerRow data, styleLiveDaysMap (from N00S Bestseller Style Identification Module).
- **Style Theme Map:** styleThemeMap (populated by Core and Bestseller modules).
- **Override Data:** External manual override file/table, specifying styleId and desired noosTheme.
- **Operational Data:** noos.output.path from args.properties.

## 8.5.3 Business Settings

- Manual override rules (typically managed outside args.properties in a separate data source like a CSV or database table).
- Implicitly uses noos.output.path for saving reports.

## 8.5.4 Process Flow

graph TD

```
A[Current Style Theme Classifications] --> B{Ingest Manual Overrides};
B --> C{Apply Overrides to Themes};
C --> D{Refine Bestseller based on Detailed Metrics};
D --> E{Generate Denormalized Output Reports};
E --> F[Persist Output Files];
```

## 8.5.5 Step-by-Step Actions

1. **Collect Current Themes:** *It gathers the current CORE and BESTSELLER classifications from the shared styleThemeMap and implicitly assigns FASHION to all remaining styles.*
2. **Ingest Manual Overrides:** *It reads external manual override configurations (e.g., a list of styleIds to force into CORE, BESTSELLER, or FASHION), allowing business users to adjust classifications.*
3. **Apply Overrides:** *It applies these manual overrides to the styleThemeMap, ensuring that business judgment takes precedence over automated logic.*
4. **Refine Bestseller Identification:** *It performs any final, detailed analysis on bestseller classifications, potentially using finer-grained metrics from cleanedSales or BestsellerRow data to confirm or adjust initial BESTSELLER tags.*
5. **Generate Output Reports:** *It constructs comprehensive ExportNoosOutputRow, DenormalizedNoosOutputRow, and DenormalizedNoosSummaryRow objects by combining all relevant data: original product attributes, calculated metrics, and the final noosTheme classification.*
6. **Persist Output Files:** *It writes these denormalized reports to specified output files (e.g., CSV, JSON) in the outputs/ directory, making the results accessible for analysis and integration.*



## 8.5.6 Sample Scenario

A brand manager wants a specific “Classic White T-Shirt” (Style T123) to *always* be CORE, even if it marginally misses the automated criteria in a given week. They add T123, CORE to the override configuration. The module will: \* Receive T123 possibly classified as FASHION from previous modules. \* Read the override. \* **Force** T123’s noosTheme to CORE. \* Generate the final reports with T123 correctly categorized as CORE. **Key Takeaway:** This module acts as the “human in the loop,” allowing strategic business decisions to influence the final NOOS classifications and ensuring that comprehensive data is available for reporting.

## 8.5.7 Results

- Finalized styleThemeMap with all CORE, BESTSELLER, and FASHION classifications, including manual overrides.
- Persisted ExportNoosOutputRow, DenormalizedNoosOutputRow, and DenormalizedNoosSummaryRow files in the outputs/ directory.

## 8.5.8 Unusual Cases

- **Missing or delayed data feed:** While this module doesn’t rely on raw sales directly, the upstream data (Core/Bestseller classifications) must be present. If not, outputs will be incomplete.
- **Conflicting manual overrides:** If a style has conflicting overrides (e.g., CORE and FASHION), the module should have a defined precedence rule (e.g., last override wins, or specific overrides have higher priority).
- **Invalid override style IDs:** If an override references a styleId that doesn’t exist in the product master, it will be ignored, but logs should indicate this.
- **Output file write errors:** Permissions issues or disk space problems could prevent reports from being saved, leading to a failed run and no accessible results.

## 8.5.9 Next Steps

The generated output reports are consumed by various downstream systems, including inventory management, merchandising dashboards, and financial planning, to implement the calculated OD, OW, and ISS strategies.

## 8.5.10 Common Questions

1. **My manual override for Style X didn’t apply. Why?** *Check the override file for correct styleId and format. Ensure the override processing order or precedence isn’t causing it to be overwritten by another rule.*
2. **Where can I find the final classification reports?** *In the outputs/ folder, as specified by noos.output.path in args.properties.*
3. **What’s the difference between ExportNoosOutputRow and DenormalizedNoosOutputRow?** *ExportNoosOutputRow might be a simplified, aggregated view for external consumption, while DenormalizedNoosOutputRow contains a richer, more detailed set of attributes for internal analysis.*

4. **Can I review classifications before outputs are generated?** *Not directly in this module. It processes and generates. Review would happen via intermediate data checks before this module runs, or by checking the generated outputs after.*
5. **How are “Fashion” styles determined?** *Any style not classified as CORE or BESTSELLER by the preceding modules or manual overrides is implicitly designated as FASHION.*

### 8.5.11 Quick Reference

- **Purpose:** Applies overrides, refines bestsellers, generates final NOOS reports.
  - **Inputs:** All upstream module outputs, manual overrides.
  - **Outputs:** Finalized styleThemeMap, various denormalized NOOS reports.
  - **Key Functionality:** Business rule enforcement, report generation.
- 

## 8.6 Attribute Group (AG) Computation Module

### 8.6.1 Module Name & Business Objective

The Attribute Group (AG) Computation Module defines and computes Attribute Groups (AGs) by combining various product attributes (e.g., category, subcategory, brand, gender, price bucket, and configurable custom attributes) and the assigned NOOS theme. This module maps individual styles to their unique AGs, creating a structured framework to analyze product performance and inventory requirements across common attribute sets (AGs), which is crucial for managing OD, OW, and ISS. Its business objective is to create a granular, actionable segmentation of the product catalog for optimized inventory planning and performance analysis.

### 8.6.2 Required Information

- **Product Data:** Product master data including styleId, categoryId, brand, gender, priceBucket, and any other configurable custom attributes.
- **NOOS Theme Data:** Finalized noosTheme for each style (Core, Bestseller, Fashion) from the NOOS Output Override and Generation Module.
- **Core Metrics:** CoreRow data (from NOOS Core Style Identification Module) to retrieve specific metrics like isLiveInCoreDuration.

### 8.6.3 Business Settings

- Configuration defining which product attributes constitute an AG (e.g., `ag.attributes.definition=category,gender,noosTheme,brand`).

### 8.6.4 Process Flow

```
graph TD
 A[Product Attributes & Final NOOS Themes] --> B{Retrieve Style Data};
 B --> C{Construct Attribute Group Key};
 C --> D{Map Styles to AGs};
```

D --> E{Aggregate Metrics per AG};  
E --> F[Attribute Group Data];

## 8.6.5 Step-by-Step Actions

1. **Retrieve Style Data:** *It fetches all relevant styleIds, their product attributes (category, brand, etc.), and their final noosTheme (from styleThemeMap) as well as CoreRow specific metrics.*
2. **Define AG Key:** *It dynamically constructs a unique Attribute Group identifier (the AG key) for each style by concatenating the values of selected attributes (e.g., category + gender + noosTheme + brand). The noosTheme is a mandatory component of the AG.*
3. **Map Styles to AGs:** *It groups styleIds under their respective AGs, creating a hierarchical structure where each AG represents a specific segment of the product catalog.*
4. **Aggregate Metrics per AG:** *It aggregates relevant business metrics (e.g., total sales, average ROS, number of styles, number of SKUs, average discount) for each computed AG, providing summary insights.*
5. **Generate AG Output:** *It produces a detailed output of all computed AGs, including the styles belonging to each AG and their aggregated metrics, such as isLiveInCoreDuration derived from CoreRow data.*

## 8.6.6 Sample Scenario

Consider “Women’s Denim Jeans” and a “Men’s Casual Shirt”. \* “Women’s Denim Jeans - Style A” (Core, Blue, Skinny) -> AG:

Apparel:Women:Core:Denim:Jeans:Blue:Skinny \* “Men’s Casual Shirt - Style B” (Bestseller, White, Regular) -> AG:

Apparel:Men:BESTSELLER:Shirts:Casual:White:Regular \* The module aggregates sales, ROS, and other metrics for all styles falling into each of these AGs. For the “Women’s Denim Jeans - Core - Blue - Skinny” AG, it sums the ROS of all styles in that group and calculates the total number of SKUs. **Key Takeaway:** AGs allow for strategic inventory management and assortment planning at a more granular level than just category, enabling precise control over OD, OW, and ISS.

## 8.6.7 Results

- A comprehensive list of unique Attribute Groups, each with:
  - Its unique AG key.
  - A list of styleIds belonging to that AG.
  - Aggregated metrics for the AG (e.g., total revenue, average ROS, count of styles, relevant Core metrics).
- These results are outputted to a data structure or file for downstream consumption.

## 8.6.8 Unusual Cases

- **Missing or inconsistent product attributes:** If essential attributes for AG definition are missing or inconsistent (e.g., gender is null for some styles), it might lead to incorrect or incomplete AGs.

- **Too many unique AGs:** If too many attributes are included, or attributes have very high cardinality, it could lead to an excessive number of very small AGs, making analysis difficult.
- **Attribute value changes:** If product attributes change over time, the AG for a style might shift, which needs to be handled in downstream analysis.
- **No styles for an AG:** If a particular combination of attributes and theme doesn't exist in the current catalog, that AG will not be generated.

### 8.6.9 Next Steps

The generated AG data is typically consumed by planning tools for: \* **OD (Optimum Depth):** Determining the ideal number of SKUs to stock within an AG. \* **OW (Optimum Width):** Deciding the ideal number of different categories/styles (AGs) to offer. \* **ISS (Ideal Size Set):** Guiding the specific sizes to stock within an AG, based on their paramount sizes. \* Performance reporting and strategic assortment reviews.

### 8.6.10 Common Questions

1. **What is an Attribute Group (AG)?** *An AG is a composite key combining multiple product attributes (like category, brand, and critically, the NOOS theme) to create a granular segment for analysis and planning (OD, OW, ISS).*
2. **Why do we need AGs when we have categories?** *AGs provide a more granular view than just categories. They allow for differentiated inventory strategies based on whether a style is CORE, BESTSELLER, or FASHION within a specific attribute combination.*
3. **Can I customize which attributes define an AG?** *Yes, the definition of an AG is configurable through a system setting, allowing flexibility in segmentation.*
4. **How do AGs help with inventory planning?** *By grouping similar products with shared performance characteristics and NOOS themes, AGs allow planners to set more precise inventory targets and ordering strategies (OD, OW, ISS) than at a broad category level.*
5. **How often are AGs recalculated?** *AGs are recalculated with each full NOOS run, ensuring they reflect the most current style classifications and product attributes.*

### 8.6.11 Quick Reference

- **Purpose:** Creates actionable product segments (AGs) for inventory planning (OD, OW, ISS).
  - **Inputs:** Product attributes, final NOOS themes, Core metrics.
  - **Outputs:** Structured Attribute Group data with aggregated metrics.
  - **Key Concept:** Granular product segmentation for strategic decision-making.
-

# 9. Appendix

## 9.1 Full Parameter Reference

Setting Name	Default Value	Description
noos.min.live.stores	5	Minimum number of live stores required for a product to be considered in sales data cleansing.
noos.core.min.quantity.sold	100	Minimum units sold for a style to be considered for Core classification.
noos.bestseller.revenue.share.threshold	0.01	Minimum revenue share for a style within its category to be a Bestseller.
noos.max.core.styles.per.category	20	Maximum number of Core styles allowed per product category to maintain portfolio balance.
noos.max.bestseller.styles.per.category	15	Maximum number of Bestseller styles allowed per

Setting Name	Default Value	Description
		product category.
noos.output.path	data/output	Directory path where all final NOOS output reports will be saved.
noos.paramount.size.min.revenue.share	0.02	Minimum revenue share for a size within a category to be considered a “paramount size.”
noos.core.min.sales.days	90	Minimum number of days a style must have recorded sales to be considered for Core classification.
noos.core.max.avg.discount	0.10	Maximum average discount percentage for a style to qualify as Core.
noos.core.max.ros.coefficient.variance	0.25	Maximum coefficient of variation for ROS to ensure consistent sales velocity for Core styles.

Setting Name	Default Value	Description
noos.bestseller.min.quantity.sold	500	Minimum units sold for a style to be considered for Bestseller classification.
noos.bestseller.max.avg.discount	0.20	Maximum average discount percentage for a style to qualify as Bestseller.
ag.attributes.definition	category,gender,noosTheme	Comma-separated list of product attributes used to form an Attribute Group (AG).

## 9.2 Version History

- **1.0 (2025-05-23):** Initial release of the NOOS documentation, covering core functionalities and module interactions.

## 9.3 Contact & Support Info

For any questions, issues, or support requests regarding the NOOS algorithm and its documentation, please contact:

- **Email:** support@yourcompany.com
- **Internal Help Desk:** Link to your internal ticketing system
- **Team Lead:** [Team Lead's Name/Contact Info]

# Tutorial: noos

The noos system orchestrates a sequential pipeline for product classification (Core, Bestseller, Fashion) and subsequent attribute group computation. The NoosGroupModule drives the main classification flow, starting with data preparation, then identifying Core and Bestseller styles, and finally applying brand overrides. The Attribute Group (AG) computation module then leverages these final classifications and metrics from earlier stages to define and analyze product attribute groups.

flowchart TD

```
A0["N00S Orchestration Module"]
A1["N00S Paramount Sizes Calculation Module"]
A2["N00S Core Style Identification Module"]
A3["N00S Bestseller Style Identification Module"]
A4["N00S Output Override and Generation Module"]
A5["Attribute Group (AG) Computation Module"]
A0 -- "orchestrates" --> A1
A1 -- "data_provides" --> A2
A1 -- "data_provides" --> A3
A1 -- "data_provides" --> A4
A0 -- "orchestrates" --> A2
A2 -- "updates_themes" --> A3
A0 -- "orchestrates" --> A3
A3 -- "uses_bestseller_data" --> A4
A2 -- "uses_core_data" --> A4
A0 -- "orchestrates" --> A4
A4 -- "finalizes_themes" --> A5
A2 -- "uses_core_metrics" --> A5
```

## Chapters



# Master Report: noos Merchandising Algorithm

## 1. Title & Metadata

- **Project Name:** noos
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** The noos algorithm optimizes merchandising decisions by calculating the Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) to maximize sales efficiency and minimize stockouts.

## 2. Executive Summary

The noos merchandising algorithm is designed to provide data-driven recommendations for product assortment planning and inventory management. By leveraging historical sales data and current inventory levels, it systematically determines the ideal number of unique items (OD), the optimal range of categories or styles (OW), and the best combination of sizes (ISS) to stock. This ensures that merchandise aligns precisely with customer demand, improving inventory turns, reducing markdown risk, and ultimately boosting profitability.

- **Key outcomes or KPIs:**
  - Increased Rate of Sale (ROS) efficiency
  - Optimized inventory levels and reduced stockouts
  - Improved category performance and sales growth
  - Enhanced customer satisfaction through better product availability
  - Reduced carrying costs and markdown exposure
- **Primary stakeholders or downstream consumers:** Merchandising Managers, Category Planners, Inventory Management Teams, Buyers, and Supply Chain Analysts.

## 3. Architecture & Data Flow

graph TD

subgraph Data Sources

A[Historical Sales Data]

B[Current Inventory Data]

C[Product Master Data]

end

subgraph Data Ingestion & Preprocessing

```

 D[Data Ingestion] --> E{Data Validation & Cleaning}
 E --> F[Feature Engineering]
end

subgraph Core Merchandising Algorithm (noos)
 F --> G[Rate of Sale (ROS) Calculation Module]
 G --> H[Optimum Depth (OD) Module]
 H --> I[Optimum Width (OW) Module]
 I --> J[Ideal Size Set (ISS) Optimization Module]
 J --> K[Recommendation Generation Engine]
end

subgraph Outputs & Consumption
 K --> L[Merchandising Recommendations Reports]
 L --> M[Merchandising Teams]
 L --> N[Inventory Planners]
 L --> O[BI Dashboards]
end

style A fill:#e0f7fa,stroke:#00bcd4,stroke-width:2px
style B fill:#e0f7fa,stroke:#00bcd4,stroke-width:2px
style C fill:#e0f7fa,stroke:#00bcd4,stroke-width:2px
style D fill:#bbdefb,stroke:#2196f3,stroke-width:2px
style E fill:#bbdefb,stroke:#2196f3,stroke-width:2px
style F fill:#bbdefb,stroke:#2196f3,stroke-width:2px
style G fill:#c8e6c9,stroke:#4caf50,stroke-width:2px
style H fill:#c8e6c9,stroke:#4caf50,stroke-width:2px
style I fill:#c8e6c9,stroke:#4caf50,stroke-width:2px
style J fill:#c8e6c9,stroke:#4caf50,stroke-width:2px
style K fill:#c8e6c9,stroke:#4caf50,stroke-width:2px
style L fill:#ffe0b2,stroke:#ff9800,stroke-width:2px
style M fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px
style N fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px
style O fill:#f3e5f5,stroke:#9c27b0,stroke-width:2px

```

• **Component Bullets:**

- **Data Ingestion & Preprocessing:** Gathers raw data from various sources, validates its integrity, cleans inconsistencies, and prepares it for analysis.
- **Rate of Sale (ROS) Calculation Module:** Computes the average weekly sales rate for individual SKUs and aggregated Attribute Groups (AGs).
- **Optimum Depth (OD) Module:** Determines the ideal number of unique SKUs to stock within a category or display based on historical performance and market demand.
- **Optimum Width (OW) Module:** Calculates the ideal number of different categories or styles to offer at a specific location or across a business unit.
- **Ideal Size Set (ISS) Optimization Module:** Recommends the most efficient combination of sizes to stock for a given product or AG, aiming to maximize sales and minimize stockouts.
- **Recommendation Generation Engine:** Consolidates the outputs from OD, OW, and ISS modules into actionable merchandising recommendations.
- **Reporting & Consumption:** Formats the recommendations into user-friendly reports and integrates with downstream business intelligence systems or planning tools.

- **High-level data sources:**
  - **Point-of-Sale (POS) Data:** Detailed transaction records including SKU, quantity, price, date, and location.
  - **Inventory Management Systems:** Real-time stock levels, inbound shipments, and historical stock movements.
  - **Product Master Data:** Static product attributes such as category, subcategory, brand, style, color, size, and cost information.
  - **Promotional Calendars:** Information on past and future marketing events that might influence sales.

## 4. Configuration & Data Dictionary

### Settings Table (`args.properties`)

These are configurable parameters that influence the algorithm's behavior and sensitivity.

Parameter Name	Description	Default Value	Unit/Type
<code>historical_lookback_weeks</code>	Number of past weeks of data to consider for analysis.	52	Weeks (int)
<code>ros_calculation_window_weeks</code>	Moving average window for ROS calculation.	12	Weeks (int)
<code>min_sales_threshold</code>	Minimum total sales units required for a SKU/AG to be analyzed.	10	Units (int)
<code>od_confidence_level</code>	Confidence level for determining Optimum Depth (e.g., 0.95 for 95%).	0.95	Percentage (float)
<code>iss_target_fill_rate</code>	Target in-stock rate for Ideal Size Set recommendations.	0.90	Percentage (float)
<code>output_report_format</code>	Desired format for output reports.	CSV	String
<code>seasonal_adjustment_factor</code>	Multiplier for seasonal products (if enabled).	1.0	Float

### Shared Data Fields (`common-data.json`)

These are the common data fields expected across various modules and data sources.

- `product_id`: Unique identifier for a product (e.g., style number).

- sku\_id: Unique identifier for a specific stock-keeping unit (product\_id + color + size).
- category: High-level product grouping (e.g., “Apparel”, “Footwear”).
- subcategory: More granular product grouping (e.g., “T-Shirts”, “Jeans”).
- brand: Brand name of the product.
- color: Specific color variant of the product.
- size: Specific size variant of the product (e.g., “S”, “M”, “L”, “XL”, “32x32”).
- date: Date of the sales transaction or inventory record.
- store\_id: Identifier for the retail location.
- sales\_units: Number of units sold.
- sales\_value: Monetary value of units sold.
- stock\_on\_hand\_units: Number of units currently in stock.
- retail\_price: Current selling price of the SKU.
- cost\_price: Cost of goods for the SKU.

## Outputs Catalog

The primary outputs generated by the noos algorithm, typically in CSV or JSON format.

Output File/Report Name	Description	Key Metrics Included
ros_metrics.csv	Detailed Rate of Sale (ROS) for each SKU and relevant Attribute Group (AG).	sku_id, AG_id, ROS (units/week), total_sales_units, weeks_on_hand
optimum_depth_recommendations.csv	Recommended Optimum Depth (OD) per category/display.	category, current_SKUs, recommended_OD, variance
optimum_width_recommendations.csv	Recommended Optimum Width (OW) per store/department.	store_id, current_categories, recommended_OW, variance
ideal_size_set_recommendations.csv	Recommended Ideal Size Set (ISS) for specific products or AGs.	product_id/AG_id, recommended_ISS (list of sizes), current_sizes, ROS_by_size
master_merchandising_report.csv	Consolidated actionable recommendations for merchandising teams.	sku_id, category, recommended_action (e.g., “add”, “remove”, “adjust stock”), reason

## 5. End-to-End Flow

The noos algorithm follows a structured process from raw data to actionable recommendations:

1. **Data Acquisition:** The process begins by ingesting historical sales data from POS systems, current inventory data from stock management systems, and product master data from central repositories.
2. **Data Preprocessing:** Raw data undergoes validation, cleaning, and transformation. Missing values are imputed, outliers are handled, and various product attributes are combined to form useful Attribute Groups (AGs) for analysis.
3. **Rate of Sale (ROS) Calculation:** For every SKU and defined AG, the algorithm calculates its average weekly Rate of Sale over a specified historical lookback period, adjusting for promotional periods or seasonality if configured.
4. **Optimum Depth (OD) Determination:** Based on the ROS of individual SKUs within a category and the overall performance metrics, the OD module determines the ideal number of unique SKUs that should be stocked to maximize sell-through while minimizing redundant offerings.
5. **Optimum Width (OW) Determination:** At a higher level, the OW module analyzes the performance of different categories or styles across various sales channels or store types to recommend the ideal number of distinct categories or styles to offer, ensuring market relevance and efficient space utilization.
6. **Ideal Size Set (ISS) Optimization:** Utilizing ROS data per size for each product or AG, the ISS module recommends the optimal mix and proportion of sizes to stock, aiming to balance available inventory with anticipated demand for each size, thereby maximizing sales and reducing stockouts.
7. **Recommendation Generation:** The outputs from the OD, OW, and ISS modules are consolidated. The system then generates concrete, actionable recommendations such as “Add SKU X to Category Y”, “Remove SKU Z”, “Adjust stock levels for Size S of Product P”, or “Expand Category A in Store B”.
8. **Reporting & Integration:** Finally, these recommendations are formatted into comprehensive reports for merchandising teams and integrated into business intelligence dashboards or direct-to-system feeds for automated adjustments, enabling informed and efficient decision-making.

## 6. Submodule Overview

### 6.1. Data Ingestion & Validation

1. **Name & Purpose:** Gathers raw sales, inventory, and product master data, validating its integrity and preparing it for downstream processing.
2. **Inputs & Outputs:**
  - **Inputs:** Raw sales transaction logs, daily inventory snapshots, product master data files.
  - **Outputs:** Cleaned, standardized dataframes (e.g., `processed_sales_df`, `processed_inventory_df`, `product_attributes_df`).
3. **Core Logic Summary:**
  - Read data from specified sources (e.g., CSV, database).
  - Perform schema validation and type conversion for all `common-data.json` fields.

- Identify and handle missing values (e.g., imputation, row removal based on `min_sales_threshold`).
- Join datasets to create a unified view of product, sales, and inventory attributes, including the formation of Attribute Groups (AGs).

## 6.2. Rate of Sale (ROS) Calculation

1. **Name & Purpose:** Computes the average weekly Rate of Sale (ROS) for individual SKUs and relevant Attribute Groups (AGs).
2. **Inputs & Outputs:**
  - **Inputs:** Cleaned historical sales data, `ros_calculation_window_weeks` setting.
  - **Outputs:** A dataset containing calculated ROS for each SKU and AG (`sku_ros_df`, `ag_ros_df`).
3. **Core Logic Summary:**
  - Group sales data by `sku_id` and selected AGs (e.g., `category`, `product_id`).
  - For each group, sum total sales units over the `ros_calculation_window_weeks`.
  - Divide total sales units by the number of weeks in the window to derive ROS (units/week).
  - Handle SKUs with zero or very low sales by applying the `min_sales_threshold` and potentially excluding them from future calculations or flagging them for review.

## 6.3. Optimum Depth (OD) & Optimum Width (OW) Determination

1. **Name & Purpose:** Calculates the ideal number of unique items (Optimum Depth) within a category and the ideal number of different categories or styles (Optimum Width) to offer.
2. **Inputs & Outputs:**
  - **Inputs:** `sku_ros_df`, `ag_ros_df`, product master data, `od_confidence_level` setting.
  - **Outputs:** Recommended OD per category and recommended OW per sales channel/store (`od_recommendations_df`, `ow_recommendations_df`).
3. **Core Logic Summary:**
  - **For OD:** Analyze ROS distribution within each category. Identify the diminishing returns point where adding more SKUs provides minimal incremental sales, using statistical methods (e.g., pareto analysis, elasticity modeling) to determine the ideal count based on the `od_confidence_level`.
  - **For OW:** Aggregate performance (total sales, profitability, foot traffic) across different categories or styles at a higher level (e.g., store, department). Assess market saturation and customer preference to determine the optimal number of distinct offerings that maximize overall business performance.
  - Compare current OD/OW against calculated ideal and identify gaps or overlaps.

## 6.4. Ideal Size Set (ISS) Optimization

1. **Name & Purpose:** Determines the best combination of sizes to maximize sales and minimize stockouts for a specific product or Attribute Group (AG).
2. **Inputs & Outputs:**
  - **Inputs:** sku\_ros\_df (specifically ROS by size), iss\_target\_fill\_rate setting, current inventory data.
  - **Outputs:** Recommended Ideal Size Set (ISS) for each product or AG (iss\_recommendations\_df).
3. **Core Logic Summary:**
  - For each product\_id or AG, calculate the ROS for each individual size (e.g., S, M, L).
  - Rank sizes by their respective ROS. Prioritize sizes with higher ROS for inclusion in the ISS.
  - Determine the optimal proportion or quantity for each size within the set to meet the iss\_target\_fill\_rate while considering inventory turnover and space constraints.
  - Propose the specific sizes to be stocked and their relative inventory allocation based on the calculated demand distribution.

## 6.5. Recommendation Generation & Output

1. **Name & Purpose:** Consolidates all calculated metrics and recommendations into user-friendly reports and integrates with downstream systems.
2. **Inputs & Outputs:**
  - **Inputs:** sku\_ros\_df, od\_recommendations\_df, ow\_recommendations\_df, iss\_recommendations\_df.
  - **Outputs:** Final master\_merchandising\_report.csv and other formatted output files.
3. **Core Logic Summary:**
  - Merge recommendations from OD, OW, and ISS modules with existing product and inventory data.
  - Generate actionable insights, classifying recommendations as “Add”, “Remove”, “Adjust Stock”, “Maintain”, or “Review”.
  - Create summary reports and detailed breakdowns based on required output\_report\_format.
  - Log all processing steps and generated outputs for auditability.

## 7. Worked Example

Let's illustrate the Ideal Size Set (ISS) optimization for a hypothetical product, “Classic T-Shirt - SKU 12345”.

**Scenario:** A retailer wants to optimize the size assortment for “Classic T-Shirt - SKU 12345” across all stores.

**1. ROS Calculation (Submodule 6.2):** The ros\_calculation\_window\_weeks is set to 12 weeks. Historical sales data for “Classic T-Shirt - SKU 12345” over the last 12 weeks:

Size	Total Units Sold (12 weeks)	Rate of Sale (ROS) (Units/Week)
XS	6	0.5
S	60	5.0
M	120	10.0
L	84	7.0
XL	36	3.0
XXL	12	1.0
<b>Total</b>	<b>318</b>	<b>26.5</b>

**2. ISS Optimization (Submodule 6.4):** The `iss_target_fill_rate` is set to 90%. This means we aim to cover 90% of the anticipated demand with the recommended sizes.

• **Step 1: Rank Sizes by ROS:**

1. M (10.0)
2. L (7.0)
3. S (5.0)
4. XL (3.0)
5. XXL (1.0)
6. XS (0.5)

• **Step 2: Calculate Cumulative ROS Percentage:**

- Total ROS = 26.5 units/week
- M:  $10.0 / 26.5 = 37.7\%$
- L:  $7.0 / 26.5 = 26.4\%$
- S:  $5.0 / 26.5 = 18.9\%$
- XL:  $3.0 / 26.5 = 11.3\%$
- XXL:  $1.0 / 26.5 = 3.8\%$
- XS:  $0.5 / 26.5 = 1.9\%$

• **Step 3: Determine ISS based on `iss_target_fill_rate` (90%):**

- M (37.7%) -> Cumulative: 37.7%
- L (26.4%) -> Cumulative:  $37.7\% + 26.4\% = 64.1\%$
- S (18.9%) -> Cumulative:  $64.1\% + 18.9\% = 83.0\%$
- XL (11.3%) -> Cumulative:  $83.0\% + 11.3\% = 94.3\%$  (Exceeds 90% target)

**3. Recommendation Generation (Submodule 6.5):**

The algorithm recommends the **Ideal Size Set (ISS)** for “Classic T-Shirt - SKU 12345” as {M, L, S, XL} to cover approximately 94.3% of demand, achieving the `iss_target_fill_rate`. The XS and XXL sizes, which represent only 5.7% of demand, are recommended to be de-prioritized or removed from standard stock, potentially reducing inventory holding costs and improving overall stock efficiency.

**Output for `ideal_size_set_recommendations.csv` would include:**

Product_ID	Recommended_ISS	Current_Sizes (example)	ROS_by_Size (M:L:S:XL)
12345	{M, L, S, XL}	{XS, S, M, L, XL, XXL}	10.0:7.0:5.0:3.0



This example shows how ROS drives the ISS decision, allowing retailers to focus on the most demanded sizes. A similar process would be used for OD and OW, but at category and style levels respectively, using aggregate performance metrics.

## 8. Debug Guide Highlights

- **Missing or Incomplete Data:**
  - **Symptom:** Algorithm crashes, empty reports, or “NaN” values.
  - **Debug:** Check `common-data.json` for required fields. Verify all input files (POS, inventory, master data) contain these fields and are correctly populated. Look for `min_sales_threshold` affecting data filtering.
  - **Action:** Ensure data pipelines are supplying complete data; review `args.properties` for data filtering settings.
- **Unrealistic Recommendations (e.g., OD too low/high, ISS missing key sizes):**
  - **Symptom:** Recommendations don’t align with business intuition or historical performance.
  - **Debug:** Review `ros_calculation_window_weeks` and `od_confidence_level`, `iss_target_fill_rate` settings. An insufficient lookback window might miss trends, or overly aggressive confidence levels might trim too much. Check for data quality issues (e.g., incorrect `retail_price` leading to skewed ROS).
  - **Action:** Adjust configuration parameters in `args.properties`. Validate input data for any anomalies or shifts.
- **Slow Performance or Memory Issues:**
  - **Symptom:** Algorithm takes an excessive amount of time to run, or encounters memory errors.
  - **Debug:** This typically scales with the volume of historical data (`historical_lookback_weeks`) and the granularity of analysis (e.g., too many small Attribute Groups).
  - **Action:** Optimize data loading, consider reducing the historical lookback window if not critical, or explore parallel processing options for large datasets.
- **Inconsistent ROS Calculations:**
  - **Symptom:** ROS values fluctuate wildly for the same SKU/AG across runs or seem incorrect.
  - **Debug:** Verify the `ros_calculation_window_weeks` is appropriate. Check if promotional periods or stockouts are biasing the ROS.
  - **Action:** Implement logic to adjust for promotions/stockouts (if not already present), or try a longer `ros_calculation_window_weeks` to smooth out fluctuations.

## 9. Common Pitfalls & Remedies

- **Pitfall 1: “Garbage In, Garbage Out”:**
  - **Description:** The quality of recommendations is directly tied to the quality of input data. Inaccurate or incomplete sales/inventory data will lead to flawed OD, OW, and ISS recommendations.
  - **Remedy:** Implement robust data validation checks (as part of Data Ingestion & Validation submodule). Establish clear data governance policies

and regular data audits. Flag and quarantine data that doesn't meet quality thresholds.

- **Pitfall 2: Ignoring Seasonality and Trends:**

- **Description:** Static ROS calculations may not accurately reflect demand for seasonal products or those experiencing rapid growth/decline, leading to overstocking or stockouts.
- **Remedy:** Incorporate seasonal indexing and trend analysis into the ROS Calculation Module. Use a dynamic `ros_calculation_window_weeks` that can adapt to product lifecycle stages, or utilize the `seasonal_adjustment_factor` setting.

- **Pitfall 3: Over-optimization & Loss of “Newness”:**

- **Description:** Strictly adhering to OD/OW recommendations might lead to a highly efficient but potentially stagnant assortment, stifling innovation or preventing the introduction of new, unproven items.
- **Remedy:** Introduce a “newness” or “test product” allocation. Allow for a small percentage of SKU slots (OD) or categories (OW) to be reserved for new product introductions, even if their initial ROS is low. Balance data-driven decisions with strategic business objectives.

- **Pitfall 4: Channel-Specific Nuances Not Captured:**

- **Description:** A single set of OD, OW, or ISS recommendations applied universally might not be optimal for different sales channels (e.g., online vs. physical stores) due to varying customer behaviors or operational constraints.
- **Remedy:** Implement channel-specific analysis. Run the algorithm with segmentations for different store clusters, online platforms, or geographic regions to generate tailored recommendations.

## 10. FAQ

- **Q1: How frequently should I run the noos algorithm?**

- **A1:** It depends on your business's pace and data freshness. Weekly or bi-weekly runs are common for granular SKU/size recommendations, while monthly or quarterly might suffice for higher-level OD/OW adjustments.

- **Q2: Can I override the recommendations generated by noos?**

- **A2:** Yes, the algorithm provides data-driven recommendations, but human judgment is crucial. Merchandising teams can review, modify, and override recommendations based on qualitative insights, market intelligence, or strategic initiatives not captured by the data. The system should ideally track these overrides for future analysis.

- **Q3: What if I have no sales data for a brand new product?**

- **A3:** For new products, the algorithm cannot calculate an accurate ROS. In such cases, initial stocking decisions should rely on analogous product performance, market trends, or expert judgment. Once sufficient sales data accumulates (meeting `min_sales_threshold`), noos can then provide data-driven recommendations.

- **Q4: How does noos handle promotional periods or holidays?**

- **A4:** The ROS Calculation Module can be configured to account for promotions by either excluding those periods from the ROS calculation or applying a normalization factor. The `seasonal_adjustment_factor` in `args.properties` can be used to manually or statistically adjust for seasonal peaks.

- **Q5: Does noos consider profitability in its recommendations?**
  - **A5:** While the core logic focuses on optimizing sales and inventory efficiency based on ROS, integrating `cost_price` and `retail_price` data allows for calculations of gross margin. Future enhancements could explicitly incorporate profitability as a weighting factor in OD/OW/ISS decisions, rather than just sales volume.

## 11. Quick Reference Cheat-Sheet

Abbreviation	Business Definition	Key Role in noos
<b>OD</b>	<b>Optimum Depth:</b> The ideal number of unique items (SKUs) to stock in a category or display.	Determined by the OD Module to optimize assortment breadth within a category.
<b>OW</b>	<b>Optimum Width:</b> The ideal number of different categories or styles to offer.	Determined by the OW Module to optimize strategic category presence.
<b>ISS</b>	<b>Ideal Size Set:</b> The best combination of sizes to maximize sales and minimize stockouts.	Optimized by the ISS Module using ROS data to ensure efficient size offerings.
<b>ROS</b>	<b>Rate of Sale:</b> The speed at which products are sold over a period (e.g., units/week).	Core metric calculated for every SKU and AG, foundational for all optimization.
<b>AG</b>	<b>Attribute Group:</b> A set of product attributes grouped for analysis (e.g., color, size, brand).	Used to aggregate and analyze data beyond individual SKUs, informing OD/OW/ISS.

**Key Configuration Parameters:** \* `historical_lookback_weeks`: Data history for analysis. \* `ros_calculation_window_weeks`: Window for ROS smoothing. \* `od_confidence_level`: Statistical confidence for OD. \* `iss_target_fill_rate`: Desired in-stock level for ISS.

**Core Process Flow:** Data Ingestion -> ROS Calculation -> OD/OW/ISS Determination -> Recommendation Generation -> Reporting.

# Chapter 1: OD Calculation Pipeline Orchestrator

Last Updated: 2025-05-23

## 1. Business Objective

This module acts as the central command center, guiding the entire process of determining the Optimum Depth (OD) for product categories to maximize sales and inventory efficiency.

**Why This Matters:** It ensures that every step of the OD calculation, from gathering initial sales data to generating the final recommendations, works together seamlessly, leading to the right amount of stock for each item and minimizing lost sales or excess inventory.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Units Sold (qtySold):** Crucial for understanding past customer demand and identifying which products are selling quickly.
- **Sales Dates:** Needed to calculate the Rate of Sale (ROS) over specific periods and identify trends or seasonality.
- **Pricing Information:** To determine the Average Selling Price (ASP) and understand the revenue contribution of each product.
- **Return Rates:** To account for products returned by customers, providing a more accurate picture of net demand.

### Product Data

- **Product Identifiers (SKUs):** To uniquely identify and track every single item in our inventory.
- **Category/Style Information:** To group similar products for analysis and ensure we calculate OD based on how products naturally relate (important for Optimum Width - OW).
- **Attribute Group (AG) details (e.g., color, size, brand):** For a detailed look at product variations and to help define the Ideal Size Set (ISS).
- **Product Status (e.g., active, discontinued):** To filter out irrelevant products and focus on items that are currently in our assortment.

## Operational Data

- **Current Inventory Levels:** To assess what we currently have on hand against the ideal stock levels recommended.
- **Store Locations:** To perform location-specific OD calculations, understanding that demand varies by store.
- **Supply Chain Lead Times:** To inform when and how much to reorder, ensuring products arrive before we run out.
- **Override Rules:** Pre-defined business decisions or strategic mandates that may adjust calculated ODs for specific products or situations.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The historical duration of sales data used for core calculations.	To capture seasonal trends (e.g., 30 days for fast fashion) or react quickly to recent market shifts.
Minimum Sales Threshold	5 units	The lowest number of Units Sold (qtySold) a product must achieve to be considered for detailed OD analysis.	To exclude extremely slow-moving items from core analysis or to include more items during growth phases.
Long Tail Exclusion Percentage	15%	The percentage of lowest-performing SKUs to potentially exclude or treat differently.	To focus inventory investment on high-Rate of Sale (ROS) items or to manage excess inventory for slow sellers.
Override Priority	User Defined	Determines if manual business overrides take precedence over calculated OD values.	When specific strategic decisions (e.g., promotions, new store launches) require manual stock adjustments.

*Note: See More Details for additional settings related to specific calculation modules.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Data Input & Validation] --> B[Data Preparation];
B --> C[Long Tail Analysis];
C --> D[Initial Override Application];
D --> E[Product Segmentation];
E --> F[ASP Calculation];
```

F --> G{Output Refinement};  
G --> H[Final Output & Sync];

1. Start with → Sales Data, Product Data, and Operational Data.
2. Then → **Validate** and **Prepare** this data for consistent use.
3. Next → **Analyze** for long-tail products that might need different inventory strategies.
4. Followed by → **Apply** any initial business override rules.
5. Then → **Segment** products into meaningful groups for tailored OD calculation.
6. Next → **Calculate** the Average Selling Price (ASP) for each product.
7. Finally → **Refine** the generated Optimum Depth (OD) recommendations and **Deliver** them to inventory systems.

## 5. Step-by-Step Actions

1. **Capture** an initial snapshot of existing output data.
  - What we do: *We make a copy of the current inventory plan or previous OD recommendations before starting new calculations.*
  - Why we do it: *This ensures we have a baseline for comparison and can revert if needed, offering a safety net for critical inventory decisions.*
2. **Ingest** and **Validate** raw input data.
  - What we do: *We gather all sales, product, and operational data, checking it for completeness and accuracy.*
  - Why we do it: *Garbage in, garbage out. Accurate data is crucial for reliable OD calculations and prevents errors in inventory planning and lost sales.*
3. **Prepare** the data for analysis.
  - What we do: *We clean, standardize, and transform the raw data into a consistent format, ready for detailed calculations.*
  - Why we do it: *This step ensures all subsequent calculations are based on harmonized, usable information, avoiding discrepancies due to varied data formats.*
4. **Identify** and **Analyze** “long tail” products.
  - What we do: *We distinguish between fast-moving, high-demand products and slow-moving items that sell infrequently (the “long tail”).*
  - Why we do it: *This allows us to apply different inventory strategies. High-volume items need precise OD, while long-tail items might require minimal stock to avoid dead inventory and free up shelf space.*
5. **Apply** preliminary business overrides.
  - What we do: *We incorporate pre-defined business rules or strategic decisions that adjust inventory recommendations, such as boosting stock for promotional items or new arrivals.*
  - Why we do it: *This ensures that calculated ODs align with broader business strategies, like supporting new product launches or reducing stock for clearance, preventing unintended financial impact.*
6. **Segment** products into logical groups.
  - What we do: *We group similar products based on shared characteristics like category, brand, or performance patterns (e.g., all basic t-shirts, all premium jeans).*
  - Why we do it: *Optimum Depth (OD) varies significantly between product types. Segmentation allows us to apply appropriate calculation models to each group, leading to more accurate recommendations and better use of floor space.*

7. **Calculate** the Average Selling Price (ASP) for each item.
  - What we do: *We determine the average price at which each product has been sold over the Analysis Period.*
  - Why we do it: *ASP is vital for understanding a product's revenue contribution and often influences its optimal stock level, especially for high-value items where overstocking is particularly costly.*
8. **Refine** and **Finalize** the output.
  - What we do: *We apply any final adjustments or business rules to the calculated OD recommendations, ensuring they are practical and actionable.*
  - Why we do it: *This is the last chance to ensure the OD recommendations are perfect before they are used for ordering, minimizing errors and maximizing business impact on sales and inventory efficiency.*
9. **Deliver** the final Optimum Depth (OD) recommendations.
  - What we do: *We publish the finalized OD values to relevant inventory and merchandising systems.*
  - Why we do it: *This step makes the calculated ODs actionable, allowing inventory planners to use them for ordering, replenishment, and merchandising decisions, directly impacting sales and stock efficiency across the business.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A clothing retailer wants to calculate the Optimum Depth (OD) for their "Women's Basic T-Shirts" category.
- They have sales data for the past 90 days showing a specific blue t-shirt (SKU #456) sold 500 units. Its current stock is 30 units.
- A business override rule states that all "new arrival" basic t-shirts should have an initial OD of 20 units per SKU for their first 30 days, regardless of initial sales.

### What Happens

1. **Data Ingestion:** The orchestrator starts by pulling sales records for SKU #456 (500 units sold over 90 days), its current stock (30 units), and product attributes (blue, size M, basic t-shirt).
2. **Data Preparation:** This raw data is cleaned and formatted. For instance, if a sales record for SKU #456 was missing a proper date, it would be corrected or flagged.
3. **Long Tail Analysis:** The orchestrator checks if any t-shirt colors or sizes are selling extremely slowly. For instance, a "lime green" t-shirt (SKU #789) that sold only 2 units in 90 days is flagged as long-tail, suggesting a minimal OD.
4. **Override Application:** The orchestrator identifies if SKU #456 is a new arrival. If it arrived last week, its initial OD is immediately set to 20 units as per the business rule, overriding any preliminary calculated OD.

5. **Product Segmentation:** All basic t-shirts, including SKU #456 and #789, are grouped together because they share similar sales patterns and operational characteristics.
6. **ASP Calculation:** The average selling price for each t-shirt SKU is determined. For example, SKU #456 might have an ASP of \$15, while SKU #789 might have an ASP of \$10.
7. **Output Refinement:** For SKU #456 (assuming it's NOT a new arrival in this final calculation step), the system calculates its Optimum Depth. Based on its strong Rate of Sale (ROS) of 500 units/90 days and ASP, it might recommend an OD of 50 units. For the slow-selling SKU #789, it might recommend an OD of 5 units. The final OD recommendations are then prepared for delivery.
8. **Output Delivery:** The calculated OD of 50 units for the popular blue t-shirt (SKU #456) and 5 units for the lime green t-shirt (SKU #789) are sent to the inventory system, informing the next stock order and replenishment.

## Final Outcome

The business receives precise, data-driven Optimum Depth (OD) recommendations for each T-shirt SKU, balanced with strategic business overrides. This ensures popular items are adequately stocked to prevent lost sales, while less popular or new items have appropriate initial inventory, maximizing revenue and minimizing carrying costs.

**Key Takeaway:** The orchestrator efficiently guides raw data through complex analysis and business rules to produce actionable inventory recommendations, optimizing stock levels and preventing both overstocking and stockouts.

## 7. Results

This step produces:

### Main Outputs

- **Optimum Depth (OD) recommendations by SKU:** The ideal number of unique items (SKUs) to stock in a category or display. It tells us precisely how much of each item should be on hand to meet customer demand and maximize sales.
- **Segmented Product List:** Products categorized into groups based on their characteristics and sales patterns. It helps merchandising teams understand how different types of products should be managed and how to determine Optimum Width (OW).
- **Average Selling Price (ASP) by SKU:** The average price at which each product has sold over the Analysis Period. It indicates the revenue contribution and value of each item, guiding inventory investment decisions.
- **Identified Long Tail Products:** A list of items that sell very slowly or infrequently. It helps identify products for clearance or discontinuation, preventing dead stock from tying up valuable inventory capital and shelf space.



## Business Reports

- **OD Recommendation Report:** Used by inventory planners and buyers to place orders and manage stock levels effectively, ensuring the right products are always available for customers.
- **Category Performance Summary:** Used by merchandising managers to review the overall health and performance of product categories and adjust broader assortment strategies (Optimum Width - OW).
- **Override Impact Analysis:** Used by business strategists to evaluate the effect of manual overrides on sales and inventory, informing future policy decisions and ensuring strategic goals are met.
- **Inventory Efficiency Dashboard:** Used by operations and finance teams to monitor key metrics like sell-through rates and stock turnover, optimizing working capital and improving profitability.

## 8. Unusual Cases

Situation | What Happens | What You Should Do ```

flowchart LR A[Input Data] --> B[Main Process] B --> C[Output Result]

11. Start with → **\*\*Input Data & Validation\*\***
12. Then → **\*\*Data Preparation\*\***
13. Next → **\*\*Long Tail Analysis\*\***
14. Next → **\*\*Initial Override Application\*\***
15. Next → **\*\*Product Segmentation\*\***
16. Next → **\*\*Average Selling Price (ASP) Calculation\*\***
17. Next → **\*\*Output Refinement\*\***
18. Finally → **\*\*Final Output & Synchronization\*\***

### ## 5. Step-by-Step Actions

1. **\*\*Capture\*\*** an initial snapshot of existing output data.
  - \* What we do: \*We make a copy of the current inventory plan or previous OD recommendations before starting new calculations.\*
  - \* Why we do it: \*This ensures we have a baseline for comparison and can revert to a prior state if needed, offering a safety net for critical inventory decisions.\*
2. **\*\*Ingest\*\*** and **\*\*Validate\*\*** raw input data.
  - \* What we do: \*We gather all sales, product, and operational data, checking it for completeness, consistency, and accuracy.\*
  - \* Why we do it: \*Accurate data is the foundation for reliable OD calculations. This step prevents errors in inventory planning that could lead to stockouts or excess inventory.\*
3. **\*\*Prepare\*\*** the data for analysis.
  - \* What we do: \*We clean, standardize, and transform the raw data into a consistent format, making it ready for detailed calculations.\*
  - \* Why we do it: \*This ensures all subsequent calculation

modules are based on harmonized, usable information, avoiding discrepancies due to varied data formats.\*

4. **\*\*Identify\*\*** and **\*\*Analyze\*\*** "long tail" products.

- \* What we do: \*We distinguish between fast-moving, high-demand products and slow-moving items that sell infrequently (the "long tail").\*

- \* Why we do it: \*This allows us to apply different inventory strategies. High-volume items need precise OD, while long-tail items might require minimal stock to avoid dead inventory and free up valuable shelf space.\*

5. **\*\*Apply\*\*** preliminary business overrides.

- \* What we do: \*We incorporate pre-defined business rules or strategic decisions that adjust inventory recommendations, such as boosting stock for promotional items or new product launches.\*

- \* Why we do it: \*This ensures that calculated ODs align with broader business strategies, like supporting new product launches or reducing stock for clearance, preventing unintended financial impact.\*

6. **\*\*Segment\*\*** products into logical groups.

- \* What we do: \*We group similar products based on shared characteristics like category, brand, or performance patterns (e.g., all basic t-shirts, all premium jeans).\*

- \* Why we do it: \*Optimum Depth (OD) varies significantly between product types. Segmentation allows us to apply appropriate calculation models to each group, leading to more accurate recommendations and better use of floor space.\*

7. **\*\*Calculate\*\*** the Average Selling Price (ASP) for each item.

- \* What we do: \*We determine the average price at which each product has been sold over the defined Analysis Period.\*

- \* Why we do it: \*ASP is vital for understanding a product's revenue contribution and often influences its optimal stock level, especially for high-value items where overstocking is particularly costly.\*

8. **\*\*Refine\*\*** and **\*\*Finalize\*\*** the output.

- \* What we do: \*We apply any final adjustments or last-mile business rules to the calculated OD recommendations, ensuring they are practical and actionable for the buying teams.\*

- \* Why we do it: \*This is the last chance to ensure the OD recommendations are perfectly aligned with business needs before they are used for ordering, minimizing potential errors and maximizing business impact on sales and stock efficiency.\*

9. **\*\*Deliver\*\*** the final Optimum Depth (OD) recommendations.

- \* What we do: \*We publish the finalized OD values to relevant inventory and merchandising systems across the organization.\*

- \* Why we do it: \*This step makes the calculated ODs actionable, allowing inventory planners to use them for ordering, replenishment, and merchandising decisions, directly impacting

sales, reducing stockouts, and optimizing inventory turns.\*

## ## 6. Sample Scenario

Let's walk through a real example:

### ### Starting Point

- A clothing retailer wants to calculate the Optimum Depth (OD) for their "Women's Basic T-Shirts" category.
- They have sales data for the past 90 days, showing a specific blue t-shirt (SKU #456) sold 500 units. Its current stock is 30 units.
- A business override rule states that all "new arrival" basic t-shirts should have an initial OD of 20 units per SKU for their first 30 days, regardless of initial sales performance.

### ### What Happens

1. **\*\*Data Ingestion:\*\*** The orchestrator starts by gathering sales records for SKU #456 (500 units sold over 90 days), its current stock (30 units), and detailed product attributes (blue, size M, defined as a "basic t-shirt").
2. **\*\*Data Preparation:\*\*** This raw sales and product data is cleaned and formatted. For instance, any incomplete sales records for SKU #456 would be corrected or flagged to ensure data quality.
3. **\*\*Long Tail Analysis:\*\*** The orchestrator checks if any t-shirt colors or sizes are selling extremely slowly. For example, a "lime green" t-shirt (SKU #789) that sold only 2 units in 90 days might be flagged as a long-tail item, indicating it should receive a very minimal OD.
4. **\*\*Override Application:\*\*** The orchestrator identifies if SKU #456 is a new arrival. If it launched last week, its initial Optimum Depth is immediately set to 20 units as per the business rule, overriding any preliminary calculated OD based on its limited sales history.
5. **\*\*Product Segmentation:\*\*** All basic t-shirts, including SKU #456 and #789, are grouped together because they share similar characteristics and sales patterns, allowing for a consistent OD calculation approach for this segment.
6. **\*\*ASP Calculation:\*\*** The average selling price for each t-shirt SKU is determined. For example, SKU #456 might have an Average Selling Price of \$15, indicating its revenue contribution.
7. **\*\*Output Refinement:\*\*** For SKU #456 (assuming it's no longer a 'new arrival' and is subject to performance-based calculation), the system calculates its final Optimum Depth. Based on its strong Rate of Sale (500 units/90 days) and ASP, it might recommend an OD of 50 units. For the slow-selling SKU #789, it might recommend a very low OD of 5 units. These recommendations are then formatted.
8. **\*\*Output Delivery:\*\*** The calculated OD of 50 units for the popular blue t-shirt (SKU #456) and 5 units for the lime green t-shirt (SKU #789) are transmitted to the inventory system, directly informing the next stock order and replenishment decisions.

### ### Final Outcome

The business receives precise, data-driven Optimum Depth (OD) recommendations for each T-shirt SKU, balanced with strategic

business overrides. This ensures popular items are adequately stocked to prevent lost sales, while less popular or new items have appropriate initial inventory, maximizing revenue and minimizing carrying costs.

**\*\*Key Takeaway:\*\*** The orchestrator efficiently guides raw data through complex analysis and business rules to produce actionable inventory recommendations, optimizing stock levels and preventing both overstocking and stockouts.

## ## 7. Results

This step produces:

### ### Main Outputs

- **\*\*Optimum Depth (OD) recommendations by SKU:\*\*** The ideal number of unique items (SKUs) to stock in a category or display. It tells us precisely how much of each item should be on hand to meet customer demand and maximize sales.
- **\*\*Segmented Product List:\*\*** Products categorized into groups based on their characteristics and sales patterns. It helps merchandising teams understand how different types of products should be managed and how to determine Optimum Width (OW).
- **\*\*Average Selling Price (ASP) by SKU:\*\*** The average price at which each product has sold over the Analysis Period. It indicates the revenue contribution and financial value of each item, guiding inventory investment decisions.
- **\*\*Identified Long Tail Products:\*\*** A list of items that sell very slowly or infrequently, not meeting the Minimum Sales Threshold. It helps identify products for clearance or discontinuation, preventing dead stock from tying up valuable inventory capital and shelf space.

### ### Business Reports

- **\*\*OD Recommendation Report:\*\*** Used by inventory planners and buyers to place orders and manage stock levels effectively, ensuring the right products are always available for customers.
- **\*\*Category Performance Summary:\*\*** Used by merchandising managers to review the overall health and performance of product categories and adjust broader assortment strategies (Optimum Width - OW).
- **\*\*Override Impact Analysis:\*\*** Used by business strategists to evaluate the effect of manual overrides on sales and inventory, informing future policy decisions and ensuring strategic goals are met.
- **\*\*Inventory Efficiency Dashboard:\*\*** Used by operations and finance teams to monitor key metrics like sell-through rates and stock turnover, optimizing working capital and improving profitability.

## ## 8. Unusual Cases

Situation Happens	What You Should
-------------------	-----------------

Do  
``,`

```markdown

Optimum Depth Calculation: Pipeline Orchestrator

Last Updated: 2025-05-23

1. Business Objective

This module serves as the central command center for the entire Optimum Depth (OD) calculation project. It orchestrates the sequential execution of all sub-modules, from initial data input and validation through various processing stages (data preparation, long tail analysis, segmentation, ASP calculation, override application) to final output generation and synchronization.

****Why This Matters:**** It ensures that every step of the OD calculation, from gathering initial sales data to generating the final recommendations, works together seamlessly and efficiently. This coordinated process is vital for providing accurate, timely recommendations that directly lead to stocking the ideal number of unique items (SKUs), minimizing lost sales from stockouts, and reducing inventory costs from overstocking.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- ****Units Sold (qtySold):**** This is essential for understanding how many units of each product were purchased by customers over a specific period, directly informing the Rate of Sale (ROS).
- ****Sales Dates:**** We need to know when each sale occurred to accurately calculate ROS over defined Analysis Periods and identify trends.
- ****Pricing Information:**** Used to determine the Average Selling Price (ASP) of each product, which is critical for understanding revenue contribution and inventory value.
- ****Return Rates:**** Incorporating returns helps us understand the true demand for a product, preventing overstocking based on gross sales figures.

Product Data

- ****Product Identifiers (SKUs):**** These unique codes identify each distinct item, allowing us to track and analyze individual products throughout the pipeline.
- ****Category/Style Information:**** Products are grouped by categories or styles to ensure that OD calculations are relevant to similar items, also helping to define Optimum Width (OW).
- ****Attribute Group (AG) details (e.g., color, size, brand):**** This granular data helps us understand product variations and can be used to determine the Ideal Size Set (ISS) for specific product groups.
- ****Product Status (e.g., active, discontinued):**** Essential for filtering out irrelevant products and focusing analysis only on

items that are currently part of the active assortment.

Operational Data

- ****Current Inventory Levels:**** Knowing the stock on hand for each item allows us to compare it against the calculated OD and plan replenishment.
- ****Store Locations:**** To perform location-specific OD calculations, recognizing that demand and optimal stock can vary significantly by store.
- ****Supply Chain Lead Times:**** Information on how long it takes for new stock to arrive influences replenishment schedules and prevents potential stockouts.
- ****Override Rules:**** Pre-defined business decisions or strategic mandates (e.g., for promotions, new store openings) that allow for manual adjustments to the calculated ODs, ensuring alignment with broader business goals.

3. Business Settings

| Setting Name | Default Value | Description |
|--------------------------------|---------------|---|
| When to Change | | |
| | | |
| Analysis Period | 90 days | The historical duration of sales data used for core calculations, like the Rate of Sale (ROS). To capture specific seasonal trends (e.g., 30 days for fast fashion) or to react more quickly to recent market shifts. |
| Minimum Sales Threshold | 5 units | The minimum Units Sold (qtySold) a product must achieve within the Analysis Period to be considered for detailed OD calculation. To prevent calculating full ODs for extremely slow-moving items (reducing dead stock) or to include more items during aggressive growth phases. |
| Long Tail Exclusion Percentage | 15% | The percentage of lowest-performing SKUs to potentially exclude from detailed OD calculations or manage differently (e.g., for clearance). To focus inventory investment on high-ROS items, ensuring popular products are well-stocked, or to manage excess inventory for slow sellers. |
| Override Priority | User Defined | Determines if manually set business overrides take precedence over the system's calculated OD values for specific items. When specific strategic decisions (e.g., a major promotion, new store launches) require a deliberate manual adjustment of stock levels. |

Note: See More Details for additional settings related to specific calculation modules.

4. Process Flow

Here's how information moves through this step:

```

```mermaid
flowchart LR
 A[Data Input & Validation] --> B{Data Preparation};
 B --> C{Long Tail Analysis};
 C --> D{Initial Override Application};
 D --> E{Product Segmentation};
 E --> F{ASP Calculation};
 F --> G{Output Refinement};
 G --> H[Final Output & Sync];

```

1. Start with → **Input Data & Validation**
2. Then → **Data Preparation**
3. Next → **Long Tail Analysis**
4. Next → **Initial Override Application**
5. Next → **Product Segmentation**
6. Next → **Average Selling Price (ASP) Calculation**
7. Next → **Output Refinement**
8. Finally → **Final Output & Synchronization**

## 5. Step-by-Step Actions

1. **Capture** an initial snapshot of existing output data.
  - What we do: *We make a copy of the current inventory plan or previous Optimum Depth (OD) recommendations before starting new calculations.*
  - Why we do it: *This ensures we have a baseline for comparison and can revert to a prior state if needed, offering a safety net for critical inventory decisions and preventing disruption.*
2. **Ingest** and **Validate** raw input data.
  - What we do: *We gather all sales, product, and operational data, checking it for completeness, consistency, and accuracy against predefined rules.*
  - Why we do it: *Accurate data is the absolute foundation for reliable Optimum Depth (OD) calculations. This crucial step prevents errors in inventory planning that could lead to widespread stockouts or costly excess inventory.*
3. **Prepare** the data for analysis.
  - What we do: *We clean, standardize, and transform the raw data into a consistent format, making it ready for detailed calculations across all subsequent modules.*
  - Why we do it: *This step ensures all subsequent calculation models are based on harmonized, usable information, avoiding discrepancies due to varied data formats and ensuring fair comparison between products.*
4. **Identify** and **Analyze** “long tail” products.
  - What we do: *We distinguish between fast-moving, high-demand products and slow-moving items that sell infrequently (often called the “long tail” of products).*
  - Why we do it: *This allows us to apply different inventory strategies. High-volume items need precise Optimum Depth (OD), while long-tail items might require minimal stock to avoid dead inventory and free up valuable shelf space for profitable items.*
5. **Apply** preliminary business overrides.
  - What we do: *We incorporate pre-defined business rules or strategic decisions that adjust inventory recommendations early in the process, such*

*as boosting stock for promotional items or prioritizing new product launches.*

- *Why we do it: This ensures that calculated Optimum Depths (ODs) align with broader business strategies, like supporting marketing campaigns or reducing stock for clearance, preventing unintended financial impact.*

**6. Segment** products into logical groups.

- *What we do: We group similar products based on shared characteristics like category, brand, or performance patterns (e.g., all basic t-shirts, all premium jeans, all seasonal decorations).*
- *Why we do it: Optimum Depth (OD) varies significantly between product types. Segmentation allows us to apply appropriate calculation models to each group, leading to more accurate recommendations and better use of floor space and display areas.*

**7. Calculate** the Average Selling Price (ASP) for each item.

- *What we do: We determine the average price at which each product has been sold over the defined Analysis Period, accounting for promotions or discounts.*
- *Why we do it: Average Selling Price (ASP) is vital for understanding a product's revenue contribution and often influences its optimal stock level, especially for high-value items where overstocking is particularly costly to the business.*

**8. Refine and Finalize** the output.

- *What we do: We apply any final adjustments or last-mile business rules to the calculated Optimum Depth (OD) recommendations, ensuring they are practical and actionable for the buying and merchandising teams.*
- *Why we do it: This is the last chance to ensure the OD recommendations are perfectly aligned with business needs before they are used for ordering, minimizing potential errors and maximizing business impact on sales and inventory efficiency.*

**9. Deliver** the final Optimum Depth (OD) recommendations.

- *What we do: We publish the finalized Optimum Depth (OD) values to relevant inventory management and merchandising systems across the organization.*
- *Why we do it: This step makes the calculated ODs actionable, allowing inventory planners to use them for ordering, replenishment, and merchandising decisions, directly impacting sales, reducing stockouts, and optimizing inventory turns across the business.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A clothing retailer wants to calculate the Optimum Depth (OD) for their "Women's Basic T-Shirts" category.
- They have sales data for the past 90 days showing a specific blue t-shirt (SKU #456) sold 500 units. Its current stock is 30 units.
- A business override rule states that all "new arrival" basic t-shirts should have an initial OD of 20 units per SKU for their first 30 days, regardless of initial sales performance.



## What Happens

1. **Data Ingestion:** The orchestrator starts by gathering sales records for SKU #456 (500 units sold over 90 days), its current stock (30 units), and detailed product attributes (blue, size M, defined as a “basic t-shirt”).
2. **Data Preparation:** This raw sales and product data is cleaned and formatted. For instance, if a sales record for SKU #456 was missing a proper date, it would be corrected or flagged to ensure data quality.
3. **Long Tail Analysis:** The orchestrator checks if any t-shirt colors or sizes are selling extremely slowly. For example, a “lime green” t-shirt (SKU #789) that sold only 2 units in 90 days is flagged as a long-tail item, indicating it should receive a very minimal OD.
4. **Override Application:** The orchestrator identifies if SKU #456 is a new arrival. If it launched last week, its initial Optimum Depth is immediately set to 20 units as per the business rule, overriding any preliminary calculated OD based on its limited sales history.
5. **Product Segmentation:** All basic t-shirts, including SKU #456 and #789, are grouped together because they share similar characteristics and sales patterns, allowing for a consistent OD calculation approach for this segment.
6. **ASP Calculation:** The average selling price for each t-shirt SKU is determined. For example, SKU #456 might have an Average Selling Price of \$15, indicating its revenue contribution.
7. **Output Refinement:** For SKU #456 (assuming it’s no longer a ‘new arrival’ and is subject to performance-based calculation), the system calculates its final Optimum Depth. Based on its strong Rate of Sale (ROS) of 500 units/90 days and ASP, it might recommend an OD of 50 units. For the slow-selling SKU #789, it might recommend a very low OD of 5 units. These recommendations are then formatted.
8. **Output Delivery:** The calculated OD of 50 units for the popular blue t-shirt (SKU #456) and 5 units for the lime green t-shirt (SKU #789) are transmitted to the inventory system, directly informing the next stock order and replenishment decisions.

## Final Outcome

The business receives precise, data-driven Optimum Depth (OD) recommendations for each T-shirt SKU, balanced with strategic business overrides. This ensures popular items are adequately stocked to prevent lost sales, while less popular or new items have appropriate initial inventory, maximizing revenue and minimizing carrying costs.

**Key Takeaway:** The orchestrator efficiently guides raw data through complex analysis and business rules to produce actionable inventory recommendations, optimizing stock levels and preventing both overstocking and stockouts.

## 7. Results

This step produces:

## Main Outputs

- **Optimum Depth (OD) recommendations by SKU:** The ideal number of unique items (SKUs) to stock in a category or display. It tells us precisely how much of each item should be on hand to meet customer demand and maximize sales, preventing empty shelves for popular items.
- **Segmented Product List:** Products categorized into groups based on their characteristics and sales patterns. It helps merchandising teams understand how different types of products should be managed and aids in determining Optimum Width (OW) across the store.
- **Average Selling Price (ASP) by SKU:** The average price at which each product has sold over the Analysis Period. It indicates the revenue contribution and financial value of each item, guiding inventory investment decisions for higher-value goods.
- **Identified Long Tail Products:** A list of items that sell very slowly or infrequently, not meeting the Minimum Sales Threshold. It helps identify products for clearance or discontinuation, preventing dead stock from tying up valuable inventory capital and shelf space.

## Business Reports

- **OD Recommendation Report:** Used by inventory planners and buyers to place orders and manage stock levels effectively, ensuring the right products are always available for customers. This report is critical for daily purchasing decisions.
- **Category Performance Summary:** Used by merchandising managers to review the overall health and performance of product categories and adjust broader assortment strategies (Optimum Width - OW). It helps decide which categories to expand or reduce.
- **Override Impact Analysis:** Used by business strategists to evaluate the effect of manual overrides on sales and inventory, informing future policy decisions and ensuring strategic goals are met. This report helps validate manual interventions.
- **Inventory Efficiency Dashboard:** Used by operations and finance teams to monitor key metrics like sell-through rates and stock turnover, optimizing working capital and improving overall profitability. This provides a high-level view of inventory health.

## 8. Unusual Cases

Situation | What Happens | What You Should Do ```

---

Generated by [AI Codebase Knowledge Builder](#)

# Chapter 3: OD Data Preparation

Last Updated: 2025-05-23

## 1. Business Objective

This module processes raw input sales and inventory data by applying several crucial cleaning and filtering steps. It prepares the data for accurate Optimum Depth (OD) analysis by eliminating noise from seasonal sales, removing sales associated with inventory discrepancies (“brokenness”), cleaning up liquidated sales based on discount benchmarks, and identifying core sales and inventory periods (e.g., first N weeks of ROS).

**Why This Matters:** This module is critical because inaccurate or “noisy” data can lead to flawed inventory decisions, resulting in either lost sales due to insufficient stock or increased carrying costs from overstocking. By ensuring the data is clean and representative of true demand, it sets a reliable foundation for maximizing sales, optimizing inventory efficiency, and improving overall profitability.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Units Sold (qtySold):** Crucial for understanding past customer demand, calculating Rate of Sale (ROS), and identifying unusual sales patterns linked to data issues or liquidation.
- **Sales Dates:** Essential for associating sales with specific seasonal periods, identifying the duration of a product’s active selling, and pinpointing relevant sales windows.
- **Pricing Information:** Necessary to determine the average selling price and detect significant discounts that indicate liquidated sales, which need to be filtered out.

### Product Data

- **Product Identifiers (SKUs):** To uniquely track and process each individual item through all cleaning and filtering stages.
- **Category/Style Information:** To group similar products for seasonal relevance, apply category-specific discount benchmarks, and organize product attributes for analysis.

- **Attribute Group (AG) details (e.g., color, size, brand):** Used to classify products and ensure consistent filtering and analysis, especially for identifying high-priority product groups.

## Operational Data

- **Historical Inventory Availability:** Records of when products were actively available for sale in stores, crucial for accurately calculating the Rate of Sale (ROS) and identifying periods of “brokenness.”
- **Inventory Data Accuracy Thresholds:** Pre-defined benchmarks that help identify and filter out sales and inventory records suffering from significant data discrepancies or “brokenness.”
- **Liquidation Discount Thresholds:** Specific discount percentages (e.g., 50% off) used to identify and exclude sales that occurred during clearance or liquidation events, ensuring these sales do not skew demand signals.
- **Sales Period Configurations:** Details about defined selling periods, including start and end dates, and priority levels, which guide the module in selecting relevant sales data.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Relevant Sales Period Weeks	8 weeks	The number of initial weeks of sales data to consider most impactful for a product’s Rate of Sale (ROS) calculation.	To capture peak demand periods more accurately for fast-moving fashion (e.g., 4 weeks) or for slower-moving items where demand builds over time (e.g., 12 weeks).
Minimum Active Selling Days	70 days	The minimum number of days a product must have been actively available for sale to be considered for comprehensive OD calculation.	To prevent unreliable OD recommendations for products that had very limited selling time (e.g., short-lived promotional items) or for new, unproven products.
Inventory Data Accuracy Threshold	80%	The minimum percentage of accurate inventory data required for a store-style-day combination to be included in sales analysis.	To adjust sensitivity to inventory discrepancies; a lower percentage includes more data, but increases risk of skewed results from “brokenness.”
Liquidation Sales Discount Threshold	50%	The discount percentage at which sales are considered “liquidated” and excluded from normal demand analysis.	To ensure sales from aggressive clearance events (e.g., 70% off) are properly filtered out, preventing overstocking based on artificial demand.

*Note: See More Details for additional settings related to specific calculation modules.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales & Inventory Data] --> B{Data Preparation & Filtering};

B --> C[Clean & Relevant Sales & Inventory Data];

1. Start with → **Raw Sales and Inventory Data**
2. Then → **Cleanse and Standardize Data**
3. Next → **Filter for Relevant Periods and Quality**
4. Finally → **Prepared Data for OD Analysis**

## 5. Step-by-Step Actions

1. **Filter** sales by product season.
  - What we do: *We match sales records to the specific seasons (e.g., Spring, Winter) during which their corresponding products were actively designed to be sold.*
  - Why we do it: *This ensures that sales data accurately reflects demand within relevant selling periods, preventing misinterpretations from off-season sales and providing a true seasonal Rate of Sale (ROS).*
2. **Align** inventory data with seasonal product availability.
  - What we do: *We identify the exact days each product was physically in stock and available for customers to purchase during its relevant selling season.*
  - Why we do it: *Accurate inventory availability is crucial for calculating a true Rate of Sale (ROS); it ensures we only count sales against periods when stock was genuinely present and ready to be sold, not just sitting in a warehouse.*
3. **Remove** sales and inventory records with discrepancies ("brokenness").
  - What we do: *We identify and exclude sales or inventory data that indicates major stock issues or system errors (e.g., negative inventory counts, unexplained missing stock, or unusually high discrepancies between physical and recorded stock).*
  - Why we do it: *Problematic data, or "brokenness," can severely skew Optimum Depth (OD) calculations, leading to incorrect stock recommendations. This step ensures analysis is based on reliable inventory records only.*
4. **Cleanse** liquidated sales.
  - What we do: *We filter out sales of products that were sold at deep discounts as part of a liquidation or clearance event, based on a pre-defined discount threshold.*
  - Why we do it: *These sales do not reflect normal, organic demand patterns and would artificially inflate Rate of Sale (ROS) metrics, potentially leading to overstocking if not accurately excluded.*

**5. Identify** relevant sales and inventory periods.

- What we do: *We focus on the most impactful sales window for each product, typically the initial several weeks after a product's launch or within other pre-defined high-priority sales periods.*
- Why we do it: *This captures the true, early demand pattern and prevents older, slow-moving sales from skewing the Rate of Sale (ROS), which is critical for accurate Optimum Depth (OD). It also helps categorize products by their priority in terms of sales contribution.*

**6. Organize** product attribute groups.

- What we do: *We structure detailed product attributes (like color, size, brand, fabric type) into logical groups for efficient lookup and consistent analysis across the assortment.*
- Why we do it: *This allows for consistent application of Optimum Depth (OD) principles across similar items and supports more granular analysis like determining the Ideal Size Set (ISS) for specific product types.*

**7. Store** interim results.

- What we do: *We save snapshots of the sales data at various stages of cleaning and filtering (e.g., raw sales, sales after brokenness removal, sales after liquidation removal, relevant sales).*
- Why we do it: *This provides an audit trail for data quality, allows for validation of each processing step, and offers transparency into the transformation process for review and debugging.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A clothing retailer has a specific blue t-shirt (SKU #456) in their "Women's Basic T-Shirts" category.
- Raw sales data shows 500 units sold over the past 90 days.
- The t-shirt is tagged for the "Summer Basics" season.
- During its selling period, the inventory system flagged 10 days where stock for SKU #456 was incorrectly reported as negative due to a system error.
- Last month, 100 units of SKU #456 were sold at a 70% discount during a flash clearance sale.
- The "Relevant Sales Period Weeks" setting is 8 weeks.
- The "Minimum Active Selling Days" benchmark is 70 days.
- The "Inventory Data Accuracy Threshold" is 80%.
- The "Liquidation Sales Discount Threshold" is 50%.

### What Happens

- 1. Seasonal Sales Filtering:** The module verifies that all 500 sales of SKU #456 occurred during its "Summer Basics" season. If any sales were made off-season, they would be excluded.
- 2. Inventory Availability Alignment:** The module identifies the actual days SKU #456 was in stock and available for customers, let's say 85 days out of the 90-day period.

3. **Brokenness Cleanup:** The module identifies the 10 days of negative stock reporting. Since these days fall below the 80% “Inventory Data Accuracy Threshold,” any sales from those 10 days (e.g., 50 units) are removed from the sales count, and those days are excluded from the available inventory period. The adjusted sales become 450 units.
4. **Liquidation Cleanup:** The module identifies the 100 units sold at a 70% discount. Since this discount is above the 50% “Liquidation Sales Discount Threshold,” these 100 units are considered liquidated sales and are removed. The adjusted sales become 350 units.
5. **Relevant Sales Period Identification:** The module now focuses on the first 8 weeks (56 days) of sales for SKU #456 after its launch. During this period, let’s say 280 units were sold. These 280 units represent the “relevant sales” for Rate of Sale (ROS) calculation. It also confirms that the t-shirt was active for more than the 70-day “Minimum Active Selling Days” benchmark.
6. **Attribute Group Organization:** The attributes of SKU #456 (blue, size M, basic t-shirt) are internally structured for efficient use in later Optimum Depth (OD) analysis.
7. **Interim Output Storage:** Snapshots of the 500 original sales, 450 units after brokenness cleanup, 350 units after liquidation cleanup, and the 280 units of relevant sales are saved.

## Final Outcome

The business receives a highly refined and accurate dataset for the blue t-shirt (SKU #456). Its “relevant sales” for precise Rate of Sale (ROS) calculations are determined to be 280 units over its prime 8-week selling window. Its total sales, adjusted for data quality and liquidation, are 350 units for the season. This clean, trustworthy data is now ready for the precise Optimum Depth (OD) calculation, ensuring that future inventory decisions are based on genuine customer demand signals, not data anomalies or promotional spikes.

**Key Takeaway:** This module thoroughly cleanses and filters raw sales and inventory data, removing noise and focusing on genuine demand signals to ensure all subsequent Optimum Depth (OD) calculations are highly accurate and reliable.

## 7. Results

This step produces:

### Main Outputs

- **Cleaned Sales Data:** Sales records that have been filtered for seasonality, removed “brokenness,” and excluded liquidated sales. It tells us the true, uninflated demand for each product under normal selling conditions.
- **Prepared Inventory Availability:** Accurate records of the precise days and periods when products were genuinely available for sale. It tells us the real window of stock availability for reliable Rate of Sale (ROS) calculations.
- **Relevant Sales Data:** Sales data specifically focused on the most impactful selling periods for each product, typically early in its lifecycle or during high-priority sales windows. It tells us the sales velocity during a product’s core selling phase.

- **Organized Product Attributes:** Structured attribute groups for efficient and consistent analysis across different product lines. It helps classify products accurately for deeper insights like Ideal Size Set (ISS) and Optimum Width (OW).

## Business Reports

- **Data Quality Report:** Used by data stewards and operations teams to monitor the health of sales and inventory data feeds. It helps identify persistent data issues at the source and track improvement over time.
- **Sales Cleansing Summary:** Used by merchandising and inventory planners to understand exactly what sales were filtered out and why. This report provides transparency into the data preparation process, aiding trust and validation.
- **Relevant Sales Period Analysis:** Used by product managers and buyers to understand the initial velocity and peak demand windows for new items. It informs product launch strategies and early replenishment decisions.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process outdated or incomplete data, leading to inaccurate results.	Investigate the source of the missing/delayed data immediately (e.g., POS systems, inventory feeds) and ensure all data streams are active and complete.
Excessive “brokenness” detected	A significant portion of sales or inventory records are flagged as unreliable.	This indicates widespread data integrity issues. Conduct a thorough audit of inventory management processes and system integrations to fix root causes.
High volume of liquidated sales	Many products have their sales significantly reduced or excluded.	Review current pricing and promotional strategies. This may suggest persistent overstocking or an overly aggressive clearance approach that needs adjustment.
No relevant sales found for a product	The module identifies a product but cannot find any “relevant” sales for it.	Manually review the product’s sales history. It might be a new arrival without enough data, an extremely slow-moving item, or a product with data issues. Consider applying default OD rules.

## 9. Next Steps

### Before This Step

- **Data Ingestion & Validation:** This preceding step is crucial as it provides the raw, uncleaned sales, product, and operational data. This module relies on the raw data being ingested and having passed basic format validation.



## After This Step

- **Long Tail Analysis:** This next step uses the cleaned and relevant sales data produced here to identify and categorize slow-moving items that may require different inventory strategies.
- **Initial Override Application:** This subsequent step takes the prepared data and applies any pre-defined business override rules (e.g., for new product launches or promotions) to adjust initial Optimum Depth (OD) recommendations.
- **Product Segmentation:** The cleaned data is used in this next step to group products into logical segments, enabling tailored Optimum Depth (OD) calculation models for each group.

## 10. Common Questions

Q: Why are some of my sales missing from the Optimum Depth (OD) calculation?

A: This module filters out sales that don't reflect true demand, such as those from liquidated stock, during periods of inventory discrepancy (brokenness), or sales that fall outside a product's defined active selling season. This ensures the OD is based on genuine demand.

Q: What is "brokenness" and why is it removed? A: "Brokenness" refers to data anomalies in inventory records, like unexpected negative stock counts or mismatches between reported and actual inventory. We remove sales and inventory from these periods to ensure that the Rate of Sale (ROS) calculations are based only on reliable stock availability, preventing inaccurate stock recommendations.

Q: How does this module handle seasonal products? A: It specifically aligns sales data with the defined selling seasons of each product. For example, a winter coat's sales are only considered during the winter period, providing a more accurate Rate of Sale (ROS) that reflects its true demand within its seasonal window.

Q: Why are sales from promotions or clearance events sometimes excluded? A: Sales made during deep discount or liquidation events (e.g., 70% off) often don't represent typical, organic customer demand. Excluding them ensures that your Optimum Depth (OD) is calculated based on regular sales patterns, preventing overstocking on items that only sell well when heavily discounted.

Q: What does "relevant sales period" mean? A: This refers to focusing on the most impactful sales window for a product, such as the initial few weeks after its launch. This helps capture its natural sales velocity before external factors or long-tail effects influence its performance, leading to more accurate Optimum Depth (OD) recommendations for its core selling phase.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To clean and filter raw sales and inventory data, making it reliable for Optimum Depth (OD) calculations.
- **Main Input:** Raw sales transactions, inventory availability data, and product attribute details.

- **Main Output:** Cleaned, filtered, and relevant sales and inventory data ready for demand analysis.
- **Warning Signs:** Unusual drops in overall sales numbers after processing, or a high percentage of sales/inventory flagged as “broken” or “liquidated.”
- **Key Settings:** “Relevant Sales Period Weeks,” “Minimum Active Selling Days,” “Inventory Data Accuracy Threshold,” and “Liquidation Sales Discount Threshold.”

---

**Need Help?** Contact your Inventory Planning Systems team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 4: OD Long Tail Computation

Last Updated: 2025-05-23

## 1. Business Objective

This module analyzes cleaned sales data to identify and filter out “long tail” styles (SKUs) within Attribute Groups (AGs) that have low Rate of Sale (ROS) or minimal revenue contribution. It calculates style-level and AG-level revenue per live day, providing the basis for refining Optimum Depth (OD) by focusing on core, high-performing items and generating initial Optimum Width (OW) metrics.

**Why This Matters:** This process is crucial for optimizing inventory investment. By accurately identifying and managing slow-moving “long tail” products, businesses can focus inventory, marketing, and display efforts on high-performing core items, maximizing sales and profitability while minimizing carrying costs and freeing up valuable shelf space. It also provides the first insights into ideal category breadth (OW).

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Units Sold (qtySold):** Crucial for understanding past customer demand and for calculating total revenue generated by each style.
- **Pricing Information:** Needed to calculate the total revenue for each style and Attribute Group, which is fundamental for determining revenue contribution.

### Product Data

- **Product Identifiers (SKUs):** To uniquely track and analyze individual items and their performance within Attribute Groups.
- **Category/Style Information:** To group similar products (styles) for analysis and to define the scope of Optimum Width (OW) recommendations.
- **Attribute Group (AG) details (e.g., color, size, brand):** Essential for grouping related styles and evaluating their collective performance, especially when identifying long tail items within a specific Attribute Group.

## Operational Data

- **Historical Inventory Availability:** Records of when products were actively available for sale (“live days”), which is vital for accurately calculating revenue per live day.
- **Analysis Period:** The specific historical duration of sales data (e.g., 90 days) that has been pre-filtered for analysis.
- **Relevant Sales Periods:** Information on predefined peak selling windows for products, ensuring calculations focus on genuine demand.
- **Long Tail Exclusion Percentage:** A business setting that defines the revenue contribution threshold for identifying low-performing “long tail” styles.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The historical duration of sales data used as input for core calculations.	To capture specific seasonal trends (e.g., 30 days for fast fashion) or to react more quickly to recent market shifts.
Relevant Sales Period Weeks	8 weeks	The number of initial weeks of sales data considered most impactful for a product’s Rate of Sale (ROS).	To capture peak demand periods more accurately for fast-moving fashion (e.g., 4 weeks) or for slower-moving items where demand builds over time.
Minimum Sales Threshold	5 units	The minimum Units Sold (qtySold) a product must achieve to be considered for detailed OD analysis.	To prevent calculating full ODs for extremely slow-moving items (reducing dead stock) or to include more items during aggressive growth phases.
Long Tail Exclusion Percentage	15%	The percentage of lowest-performing SKUs by revenue contribution that are flagged as “long tail.”	To focus inventory investment on high-ROS items or to aggressively manage excess inventory for slow sellers.

## 4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Cleaned & Relevant Sales & Inventory Data] --> B{OD Long Tail Computation};
B --> C[Refined Style Performance Data & Initial OW Metrics];
```

1. Start with → **Cleaned and Relevant Sales and Inventory Data** (from previous steps).
2. Then → **Calculate Revenue and Live Days per Style and Attribute Group.**

3. Next → **Generate Initial Optimum Width (OW) Metrics** based on Attribute Group performance.
4. Next → **Calculate Revenue per Live Day** for each individual style.
5. Next → **Identify and Filter “Long Tail” Styles** based on their revenue contribution per day.
6. Finally → **Output Refined Style Performance Data and Optimum Width Metrics**.

## 5. Step-by-Step Actions

1. **Calculate** style-level revenue and live days.
  - What we do: *For each unique style (SKU) within every Attribute Group (AG), we sum its total revenue and count the number of days it was actively available for sale, drawing from the cleaned data provided by the OD Data Preparation module.*
  - Why we do it: *These fundamental metrics are essential for determining a style’s true performance and form the basis for calculating its revenue efficiency (revenue per live day), which is critical for identifying profitable items.*
2. **Aggregate** Attribute Group (AG) revenue.
  - What we do: *We sum up the total revenue for all styles within each Attribute Group, taking into account different priority levels (e.g., high-priority core products vs. seasonal items).*
  - Why we do it: *Understanding the collective revenue contribution of an entire Attribute Group helps in setting broader Optimum Width (OW) strategies and identifying the overall importance of a product group to the business.*
3. **Generate** initial Optimum Width (OW) metrics.
  - What we do: *Based on the aggregated performance of Attribute Groups and categories, we calculate preliminary recommendations for the ideal number of different categories or styles to offer (Optimum Width).*
  - Why we do it: *This provides early guidance on assortment breadth, helping businesses decide if they should expand or contract certain product offerings to meet customer needs without overextending into unprofitable categories.*
4. **Compute** revenue per live day for each style.
  - What we do: *We divide each style’s total revenue by its number of active selling days (live days).*
  - Why we do it: *This metric provides a normalized measure of sales efficiency, allowing for fair comparison between styles regardless of how long they’ve been in stock. It’s a key indicator for identifying top performers and potential “long tail” items.*
5. **Apply** the “long tail” filter.
  - What we do: *Within each Attribute Group, we rank styles by their revenue per live day. We then identify and flag those styles that contribute minimally to the group’s total revenue, based on the Long Tail Exclusion Percentage setting.*
  - Why we do it: *This crucial step helps us distinguish core, high-performing items from slow-moving “long tail” products. It enables targeted inventory management, allowing us to focus investment on profitable items and potentially reduce stock of low-value ones, freeing up capital and valuable shelf space.*

## 6. Store the results.

- What we do: *We save the calculated style-level performance data, including identified long-tail statuses, and the initial Optimum Width (OW) metrics into our database for use in subsequent modules and for business reporting.*
- Why we do it: *This ensures the processed information is available for subsequent stages of the Optimum Depth (OD) calculation pipeline and provides transparency and an audit trail for future review.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A fashion retailer is analyzing their “Women’s Skinny Jeans” category.
- Within this category, there’s an Attribute Group (AG) for “Black Skinny Jeans,” which includes 5 distinct styles (SKUs).
- Sales data for the past 90 days (the Analysis Period) shows:
  - SKU #101 (Classic Black): Sold 300 units, generated \$9,000 revenue over 80 live days.
  - SKU #102 (Distressed Black): Sold 150 units, generated \$4,500 revenue over 75 live days.
  - SKU #103 (Coated Black): Sold 80 units, generated \$2,400 revenue over 70 live days.
  - SKU #104 (Faded Black): Sold 20 units, generated \$500 revenue over 60 live days.
  - SKU #105 (Acid Wash Black): Sold 5 units, generated \$150 revenue over 50 live days.
- The Long Tail Exclusion Percentage is set to 15%.

### What Happens

1. **Calculate Style Revenue and Live Days:** The module computes the total revenue and live days for each SKU.
  - SKU #101: \$9,000 revenue / 80 days = \$112.50/day
  - SKU #102: \$4,500 revenue / 75 days = \$60.00/day
  - SKU #103: \$2,400 revenue / 70 days = \$34.29/day
  - SKU #104: \$500 revenue / 60 days = \$8.33/day
  - SKU #105: \$150 revenue / 50 days = \$3.00/day
2. **Aggregate Attribute Group (AG) Revenue:** The total revenue per live day for the “Black Skinny Jeans” AG is calculated: \$112.50 + \$60.00 + \$34.29 + \$8.33 + \$3.00 = \$218.12 per live day.
3. **Generate Initial OW Metrics:** Based on the overall performance of the “Women’s Skinny Jeans” category and the “Black Skinny Jeans” AG, an initial assessment of how many distinct styles or categories should be offered is produced. For example, it might suggest the ideal OW for this category is 15 distinct styles.
4. **Compute Revenue per Live Day:** (Already done in step 1, but this step formalizes it for filtering.)

### 5. Apply Long Tail Filter:

- The module ranks styles by revenue per live day: #101 > #102 > #103 > #104 > #105.
  - It calculates each style's revenue contribution *within the AG*:
    - SKU #101:  $(\$112.50 / \$218.12) * 100\% = 51.58\%$
    - SKU #102:  $(\$60.00 / \$218.12) * 100\% = 27.51\%$
    - SKU #103:  $(\$34.29 / \$218.12) * 100\% = 15.72\%$
    - SKU #104:  $(\$8.33 / \$218.12) * 100\% = 3.82\%$
    - SKU #105:  $(\$3.00 / \$218.12) * 100\% = 1.38\%$
  - Styles are sorted by contribution. The lowest 15% (Long Tail Exclusion Percentage) are identified. In this case, 15% of 5 styles is 0.75, so it rounds up to 1 style or identifies based on cumulative contribution. If we take the lowest 15% *cumulative* from the bottom, or simply target a number of styles to exclude. Here, the rule is “percentage of lowest-performing SKUs”. If it means the bottom X% of the *total number* of styles, it's 15% of 5 = 0.75, so 1 style. If it means lowest contributing styles until their *cumulative revenue contribution* is below X% benchmark, it would be different. Given the code `OdUtil.getBadStyles(sortedMap, benchmark)`, which likely means cumulative contribution until a benchmark, let's re-evaluate.
  - Let's assume the benchmark is the *cumulative percentage of revenue* from the lowest-performing styles. So, `longTailRevContriBenchmark = 15%` means styles whose *cumulative* revenue contribution from the bottom is less than 15% are long-tail.
  - SKU #105 (1.38% contribution). Cumulative = 1.38%. (Below 15%)
  - SKU #104 (3.82% contribution). Cumulative (with #105) = 1.38% + 3.82% = 5.20%. (Below 15%)
  - SKU #103 (15.72% contribution). Cumulative (with #105, #104) = 5.20% + 15.72% = 20.92%. (Above 15%).
  - Therefore, SKU #104 and #105 are flagged as “long tail” because their combined revenue contribution is below the 15% threshold.
6. **Store Results:** The system records that SKU #104 and #105 are long-tail, along with the calculated revenue per live day for all styles, and the initial OW recommendation.

## Final Outcome

The business clearly identifies that SKU #104 (Faded Black) and SKU #105 (Acid Wash Black) are “long tail” items within their “Black Skinny Jeans” Attribute Group. This insight allows inventory planners to reduce future orders for these styles, perhaps prioritizing them for clearance or discontinuing them. Simultaneously, the core styles (SKU #101, #102, #103) are identified as high-performing, warranting continued focus for Optimum Depth (OD) calculation and replenishment. The business also receives initial guidance on the ideal breadth of their “Women's Skinny Jeans” category.

**Key Takeaway:** This module systematically distinguishes high-performing styles from slow-moving ones, ensuring inventory strategies are focused on maximizing revenue and minimizing dead stock, while also providing initial guidance for overall product breadth.

## 7. Results

This step produces:

### Main Outputs

- **Identified Long Tail Styles:** A clear list of specific SKUs that fall below the revenue contribution benchmark, indicating they are slow-moving or low-performing items. It tells us which individual products may need different inventory or merchandising strategies (e.g., discontinuation, minimal stock).
- **Style-level Revenue per Live Day:** A normalized metric showing the revenue generated by each style for every day it was actively available for sale. It tells us the true sales efficiency of each product, allowing for fair comparison across the assortment.
- **Attribute Group (AG) Revenue Contribution:** The total revenue generated by all styles within an Attribute Group, often broken down by priority. It tells us the overall financial importance of a group of similar products.
- **Initial Optimum Width (OW) Metrics:** Preliminary recommendations for the ideal number of different categories or styles to offer, based on aggregated performance data. It tells us the suggested breadth of the product assortment.

### Business Reports

- **Long Tail Product Report:** Used by inventory planners and merchandisers to review and make decisions on low-performing items, such as clearance, re-evaluation, or discontinuation. This helps prevent accumulation of dead stock.
- **Style Performance Dashboard:** Used by product managers to quickly identify top-performing styles and those requiring attention, guiding future design and buying decisions.
- **Assortment Breadth (OW) Guidance:** Used by merchandising leaders to inform strategic decisions about expanding or contracting product categories, ensuring a balanced and profitable product offering.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process outdated or incomplete data, leading to inaccurate long tail identification.	Investigate the source of the missing/delayed sales and inventory data immediately (e.g., POS systems, inventory feeds) and ensure all data streams are active and complete.
All styles flagged as “long tail”	A high percentage or even all styles within an Attribute	Review the Long Tail Exclusion Percentage setting – it might be too aggressive. Also, investigate if the entire Attribute Group is underperforming due to market changes or product issues.



Situation	What Happens	What You Should Do
No styles flagged as “long tail”	Group are identified as long tail.  Even very low-performing styles are not identified as long tail.	Review the Long Tail Exclusion Percentage setting – it might be too lenient. This could lead to overstocking of slow-moving items if not adjusted.
New products with low sales flagged as long tail	A newly launched style with limited sales history is flagged as long tail.	Manually review the product’s launch strategy. New items often need time to gain traction. Consider manual overrides or adjusting the Minimum Active Selling Days setting for new products.
Inconsistent live day data	Significant discrepancies in reported “live days” for similar styles.	Investigate the source of inventory availability data. Inaccurate live day counts can skew revenue per live day calculations, leading to misidentification of long tail products.

## 9. Next Steps

### Before This Step

- **OD Data Preparation:** This preceding step is crucial as it provides the cleaned, filtered, and relevant sales and inventory data. This module relies on that data having passed quality checks and seasonal filtering.
- **Data Ingestion & Validation:** This initial step ensures all raw sales, product, and operational data are accurately loaded and validated, forming the basis for subsequent data preparation.

### After This Step

- **Initial Override Application:** The results from this module, including identified long tail styles, are used in this next step. Pre-defined business overrides may then be applied to adjust initial Optimum Depth (OD) recommendations, particularly for newly identified long tail items.
- **Product Segmentation:** The refined style performance data and Attribute Group information produced here are used to segment products into logical groups, enabling tailored Optimum Depth (OD) calculation models for each segment.

## 10. Common Questions

Q: Why are some of my products flagged as “long tail”? A: Products are flagged as “long tail” when their revenue contribution per active selling day within their Attribute Group falls below a set benchmark (the Long Tail Exclusion Percentage). This means they are selling significantly slower or generating less revenue compared to other similar products.

Q: How is “revenue per live day” different from Rate of Sale (ROS)? A: While both relate to sales velocity, “revenue per live day” specifically accounts for the number of days a product was *actually available for sale* (its “live days”). This is more precise than a simple Rate of Sale (ROS) over a fixed period, as it adjusts for inventory outages or late arrivals, giving a truer picture of demand when stock was present.

Q: What is the business impact of filtering out long tail products? A: Filtering out long tail products ensures inventory investment is focused on your best-selling, most profitable items. This helps prevent capital from being tied up in slow-moving stock, reduces the risk of markdowns, frees up warehouse and display space, and ultimately improves overall sales efficiency and profitability.

Q: How does this module help with Optimum Width (OW)? A: By aggregating revenue and performance data at the Attribute Group and category levels, this module provides an initial assessment of the demand and profitability of broader product groupings. This early insight helps merchandising teams understand which categories are performing well and which might be saturated, guiding decisions about the ideal number of different categories or styles to offer (Optimum Width).

Q: What if a new product has very low sales due to being recently launched? Is it immediately long tail? A: This module primarily focuses on performance within the defined Analysis Period and Relevant Sales Period Weeks. For very new products, while their sales might be low, subsequent modules or manual business overrides often account for their “new arrival” status, preventing premature long tail classification until they have a sufficient sales history.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To identify low-performing “long tail” styles and calculate revenue efficiency, while generating initial Optimum Width (OW) metrics.
- **Main Input:** Cleaned and relevant sales and inventory data.
- **Main Output:** Identified long tail styles, style-level revenue per live day, and initial Optimum Width (OW) metrics.
- **Warning Signs:** Too many or too few products flagged as long tail within an Attribute Group, or unexpected revenue per live day metrics.
- **Key Settings:** Long Tail Exclusion Percentage, Analysis Period, Relevant Sales Period Weeks.

---

**Need Help?** Contact your Inventory Planning Systems team for questions about this process.

---



# Chapter 7: OD Average Selling Price (ASP) Calculation

Last Updated: 2025-05-23

## 1. Business Objective

This module calculates the Average Selling Price (ASP) for each Attribute Group (AG) relevant to Optimum Depth (OD) recommendations. It computes ASP at multiple granularity levels (store, region, channel) and applies any manual ASP overrides provided in the input data, ensuring accurate pricing context for assortment planning.

**Why This Matters:** This module is critical because an accurate understanding of a product's average selling price directly influences inventory investment decisions. It ensures that Optimum Depth (OD) recommendations are based on realistic revenue contributions, helping businesses prioritize high-value items, minimize the financial risk of overstocking expensive products, and ultimately maximize sales and profitability.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Pricing Information:** This is essential for determining the Average Selling Price (ASP) for each product and transaction, as it records the actual price at which an item was sold.
- **Units Sold:** Necessary to correctly weigh sales prices by the quantity sold, ensuring the calculated ASP accurately reflects the average price across all units purchased.
- **Sales Dates:** Crucial for ensuring that ASP calculations are based on relevant historical periods (e.g., the last 90 days), aligning pricing context with current demand signals.

### Product Data

- **Product Identifiers (SKUs):** These unique codes allow us to link sales transactions to specific products, which is fundamental for calculating ASP for individual items.

- **Attribute Group (AG) details (e.g., color, size, brand):** Important for grouping products into Attribute Groups, as ASP is calculated and applied at this level for Optimum Depth (OD) recommendations.

## Operational Data

- **Store Locations:** Information about each store's location, including its region and channel, is critical for calculating ASP at various granularity levels (store, region, channel).
- **Manual ASP Overrides:** Specific business decisions or strategic mandates that allow for manual adjustments to the calculated ASP for certain products or Attribute Groups, overriding system-generated values.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Analysis Period	90 days	The historical duration of sales data used for core calculations, like ASP.	To capture specific seasonal trends (e.g., 30 days for fast fashion) or to react more quickly to recent market shifts.
Override Priority	User Defined	Determines if manual business overrides take precedence over calculated OD values (including ASP context).	When specific strategic decisions (e.g., a major promotion, new store launches) require a deliberate manual adjustment of stock levels.
ASP Fallback Hierarchy	Store > Region/Channel > Default	Defines the order in which ASP is sought: first at the store level, then region/channel, then a default value if no sales data exists.	To ensure that an ASP is always available for every product, balancing precision with data availability.
Default Zero Sales ASP	\$0.00	The default Average Selling Price to use for products or Attribute Groups that have no sales data within the analysis period.	To provide a conservative or strategic default price for new products or very slow movers when no sales history is available.

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Manual ASP Overrides] --> B{OD Average

Selling Price (ASP) Calculation};  
B --> C[Calculated & Overridden ASPs];

1. Start with → **Cleaned Sales Data and Manual ASP Overrides.**
2. Then → **Calculate Region and Channel-level ASPs.**
3. Next → **Calculate Store-level ASPs.**
4. Next → **Generate Final ASP outputs, applying overrides.**
5. Next → **Update ASP for new stores.**
6. Finally → **Persist Output and Clear Temporary Data.**

## 5. Step-by-Step Actions

1. **Initialize** data structures.
  - What we do: *We prepare empty maps to store calculated ASPs at different levels (store, region/channel) and a list to hold the final ASP outputs.*
  - Why we do it: *This sets up the necessary containers for efficient processing and storage of pricing data before calculations begin.*
2. **Compute** Region and Channel-level ASP.
  - What we do: *We calculate the average selling price for each Attribute Group across all sales from stores within a specific region and channel, using the cleaned sales data.*
  - Why we do it: *This provides a broad, aggregated ASP that can serve as a reliable fallback if store-specific ASP data is missing or insufficient, ensuring consistent pricing context across a wider geographical area.*
3. **Compute** Store-level ASP.
  - What we do: *We calculate the average selling price for each Attribute Group within individual stores, based on their specific sales data.*
  - Why we do it: *Store-level ASP is the most precise pricing context. It's crucial for local inventory decisions, as selling prices can vary by location due to market dynamics or local promotions.*
4. **Generate** final ASP outputs, applying overrides.
  - What we do: *For each Attribute Group in every relevant store, we determine its final ASP. We prioritize the store-level ASP; if unavailable, we fall back to the region/channel ASP. We then incorporate any specific manual ASP override values provided by the business.*
  - Why we do it: *This step ensures that the final ASP used for Optimum Depth (OD) calculations is as accurate as possible, reflecting specific store performance or broader regional trends, and incorporating strategic business decisions to adjust pricing context.*
5. **Update** ASP for new stores.
  - What we do: *For newly opened stores that lack historical sales data, we assign ASPs based on the ASPs of their designated reference stores (e.g., a comparable existing store).*
  - Why we do it: *New stores need an immediate pricing context for their initial Optimum Depth (OD) planning. Assigning them ASPs from similar, established stores provides a realistic starting point, preventing arbitrary stock levels.*
6. **Persist** and clear outputs.
  - What we do: *We save the calculated ASP data into the database so it can be used by downstream modules and for business reporting, then clear the temporary data structures to free up system memory.*

- Why we do it: *This makes the finalized ASP data available for subsequent Optimum Depth (OD) calculation steps and ensures efficient resource management by cleaning up temporary data after use.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A clothing retailer manages “Women’s Basic T-Shirts” (an Attribute Group).
- **Store A** in Region X, Channel Y: Over the Analysis Period, 100 units of this AG were sold, generating \$1,500 total revenue.
- **Store B** in Region X, Channel Y: Over the Analysis Period, only 5 units of this AG were sold, generating \$70 total revenue (low volume).
- **Region X, Channel Y (overall)**: For this AG, 500 units were sold across all stores in the region/channel, generating \$7,500 total revenue.
- **Manual Override**: For the “Women’s Basic T-Shirts” AG in Store B, there’s a strategic decision to apply an ASP override factor of 1.10 (meaning a 10% increase to its calculated ASP).
- **New Store C**: A brand new store in Region X, Channel Y has designated Store A as its reference store.

### What Happens

1. The module **initializes** its data structures.
2. **Region and Channel-level ASP Computation**: For “Women’s Basic T-Shirts” in Region X, Channel Y, the ASP is calculated as  $\$7,500 / 500 \text{ units} = \$15.00$ . This value is stored as the regional fallback.
3. **Store-level ASP Computation**:
  - For Store A, “Women’s Basic T-Shirts” ASP is calculated:  $\$1,500 / 100 \text{ units} = \$15.00$ .
  - For Store B, “Women’s Basic T-Shirts” ASP is calculated:  $\$70 / 5 \text{ units} = \$14.00$ .
4. **Final ASP Generation and Override Application**:
  - For **Store A**, the calculated store-level ASP is \$15.00. No manual override is specified for Store A. The final ASP for this AG in Store A is **\$15.00**.
  - For **Store B**, the calculated store-level ASP is \$14.00. The manual override factor of 1.10 is applied:  $\$14.00 * 1.10 = \$15.40$ . The final ASP for this AG in Store B is **\$15.40**.
5. **New Store ASP Update**: Since New Store C references Store A, the ASP for “Women’s Basic T-Shirts” in New Store C is set to **\$15.00** (copied from Store A).
6. The module then **persists** these final ASP values and **clears** its temporary data.

### Final Outcome

The business receives precise Average Selling Prices for the “Women’s Basic T-Shirts” Attribute Group: \$15.00 for Store A, \$15.40 for Store B (reflecting the strategic override), and \$15.00 for the New Store C. This ensures that subsequent

Optimum Depth (OD) calculations for this popular product group are based on accurate and strategically adjusted pricing contexts, maximizing revenue potential across all locations while aligning with specific business goals.

**Key Takeaway:** This module precisely calculates and strategically adjusts Average Selling Prices, ensuring that inventory planning for Optimum Depth (OD) is always informed by the most relevant and accurate financial value of each product.

## 7. Results

This step produces:

### Main Outputs

- **Average Selling Price (ASP) by Attribute Group, Store, and Period:** The calculated average price for each Attribute Group in every store over specific timeframes, considering all granularity levels (store, region, channel). It tells us the precise revenue contribution per unit, crucial for valuing inventory and sales.
- **ASP with Applied Overrides:** The final Average Selling Price values for each Attribute Group after incorporating any manual business adjustments or strategic mandates. It tells us the effective selling price to use for inventory planning, directly reflecting business decisions.
- **ASP Override Values Applied:** The specific override factors or values used to adjust the calculated ASPs. It provides transparency into how manual adjustments influenced the final pricing context for audit and review.

### Business Reports

- **ASP Performance Report:** Used by merchandising and finance teams to review product pricing effectiveness and revenue contribution across the assortment. This report highlights potential pricing discrepancies or opportunities for optimization.
- **Pricing Override Audit Report:** Used by business strategists and finance to track and evaluate the impact of manual ASP overrides on calculated Optimum Depth (OD) and overall financial outcomes. This report helps validate strategic pricing decisions.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will calculate ASP based on incomplete sales data, leading to skewed or inaccurate prices.	Investigate the source of the missing/delayed sales data immediately (e.g., Point-of-Sale systems, e-commerce platforms) and ensure all transactional data streams are active and complete.



Situation	What Happens	What You Should Do
Attribute Group with Zero Sales	If an Attribute Group has no sales data for a specific store or even region/channel. ASP will fall back to higher granularity (region/channel), or eventually to the default value.	Manually review newly launched products or discontinued items with no sales. Consider using a default ASP or assigning a reference store's ASP if appropriate, ensuring every item has a reasonable pricing context.
Abnormally High/Low ASP due to very few sales	A few high-priced or deeply discounted transactions can disproportionately skew ASP if sales volume is minimal.	Flag Attribute Groups with very low Units Sold for manual review. Consider implementing a minimum transaction threshold for reliable ASP calculation, or relying on broader regional/channel ASPs for low-volume items.
Conflicting Override Values	If a manual override is set incorrectly or contradicts expected values, leading to an illogical ASP.	Investigate the manual override input data. Ensure business rules for overrides are clear, properly communicated, and correctly applied in the system to prevent unintended pricing impacts.

## 9. Next Steps

### Before This Step

- **OD Data Preparation:** This preceding step is crucial as it provides the cleaned and relevant sales data, free from discrepancies and liquidation sales, which is essential for accurate ASP calculation.
- **OD Long Tail Computation:** This module identifies and removes “long tail” styles, ensuring that ASP is calculated only for core, performing items, preventing the ASP from being skewed by very low-value or slow-moving products.
- **OD Category Override Computation:** This module provides Attribute Group level data and initial segmentation, which influences the context for organizing and processing the sales data for ASP calculation.

### After This Step

- **Remaining OD Calculation Modules:** The calculated Average Selling Prices are a critical input for all subsequent modules that determine Optimum Depth (OD) by valuing inventory, assessing Rate of Sale (ROS) in financial terms, and evaluating profitability.

- **Financial Impact Analysis:** The final ASP values directly feed into profitability models, revenue forecasting, and inventory valuation reports, providing a foundational metric for financial performance assessment.

## 10. Common Questions

Q: Why is ASP calculated at different levels (store, region, channel)? A: To ensure we always have a reliable average selling price for every product group. Store-level ASP is the most precise, but if a store has limited sales data, we can use the broader region or channel ASP as a reliable fallback. This provides a stable and accurate pricing context for all products across different locations.

Q: How do manual overrides affect the ASP calculation? A: Manual overrides allow business users to strategically adjust the calculated ASP, for example, to anticipate future pricing changes, account for planned promotions, or reflect unique market conditions. These overrides take precedence over system-calculated values, ensuring the final ASP used for inventory planning aligns with specific business strategies and goals.

Q: What happens if a product has no sales data at all? A: If a product has no sales in a specific store within the analysis period, the module will attempt to find its ASP at a broader level, such as the region or channel. If still no sales data is available at any level, it will fall back to a predefined default value, ensuring that even new or very slow-moving items have a pricing context for inventory planning.

Q: Why is ASP important for Optimum Depth (OD)? A: ASP is vital because it tells us the financial value of each unit sold. For high-value items, even a small overstock can be very costly in terms of tied-up capital and potential markdowns. Knowing the ASP helps determine the right stock level to maximize revenue while minimizing the financial risk associated with holding too much inventory.

Q: How does this module handle new stores? A: New stores typically lack historical sales data. This module addresses this by assigning them Average Selling Prices based on the performance of designated reference stores (e.g., similar, established stores in the same region or channel). This provides new stores with an immediate and realistic pricing context from day one, allowing for effective initial inventory planning.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate and apply overrides to Average Selling Price (ASP) for Optimum Depth (OD) recommendations.
- **Main Input:** Cleaned sales data, product Attribute Group details, and manual ASP override values.
- **Main Output:** Final Average Selling Price (ASP) for each Attribute Group, specified by store and analysis period.
- **Warning Signs:** Unrealistic or unexpectedly high/low ASPs for specific Attribute Groups, or a high number of ASPs relying solely on regional/channel averages due to insufficient store data.

- **Key Settings:** Analysis Period, Override Priority, ASP Fallback Hierarchy, Default Zero Sales ASP.
- 

**Need Help?** Contact your Inventory Planning Systems team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 8: OD Output and Override Application

**Last Updated:** 2025-05-23

## 1. Business Objective

This module is the final stage of the Optimum Depth (OD) pipeline, applying all computed and manually provided overrides (at category, store, and channel levels) to the calculated product depth recommendations. It then denormalizes various data points to generate comprehensive final reports and summary tables, reflecting the ideal product depth recommendations after all business rules and strategic adjustments have been applied.

**Why This Matters:** This module ensures that the final OD recommendations are not just statistically derived, but also align perfectly with real-world business strategies and priorities. By integrating human expertise and strategic mandates, it optimizes inventory levels for maximum sales, reduces stockouts, and minimizes excess stock, ultimately improving overall profitability and customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Units Sold:** This is essential for understanding historical demand patterns, which are refined through the application of overrides to create accurate future OD recommendations.
- **Pricing Information:** Used to understand the revenue contribution of each product and Attribute Group, which is then adjusted by overrides to reflect strategic pricing or target profitability.

### Product Data

- **Product Identifiers (SKUs):** These unique codes identify each distinct item, allowing overrides to be applied at the most granular level and aggregated into comprehensive reports.
- **Attribute Group (AG) details (e.g., color, size, brand):** Crucial for grouping products and applying overrides at the Attribute Group level, ensuring adjustments are relevant to specific product clusters.

- **Category/Style Information:** Necessary for applying overrides at broader category or style levels and for structuring the final output reports.

## Operational Data

- **Computed Category Overrides:** Automatically generated strategic adjustments based on a category's overall performance and contribution, influencing the OD for all products within it.
- **Manual Store and Channel Overrides:** Specific business decisions (e.g., for promotions, new store launches, regional initiatives) that explicitly adjust OD recommendations for certain products or Attribute Groups at a store or channel level.
- **Reference Store Mappings:** For new stores without historical data, information on which existing stores they should mimic to receive initial OD recommendations.
- **Previously Calculated OD Data:** The initial Optimum Depth recommendations generated by prior modules, which this module refines and finalizes.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Override Priority Rule	User Defined	Determines the precedence of different override types (e.g., manual overrides trump calculated ones; specific overrides trump general ones).	When specific strategic decisions (e.g., a major promotion, new store launches) must always take precedence over automated calculations.
New Store Data Inheritance Policy	Reference Store	Specifies how newly opened stores, lacking historical data, acquire initial OD recommendations.	To ensure new stores are stocked accurately from day one, by mirroring comparable existing stores or applying channel averages.
Revenue Contribution Sum Target	100%	The target percentage that the sum of all Attribute Group contributions must achieve within a store or channel after all overrides.	To ensure that all product groups collectively account for the entire inventory value, preventing over- or under-allocation.
Reporting Granularity	SKU, AG, Store	Defines the levels at which detailed final reports are generated (e.g., individual items, Attribute Groups, or entire stores).	To provide insights tailored to different user needs, from detailed buying decisions to high-level merchandising strategies.

*Note: See More Details for additional settings related to specific calculation modules.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Calculated OD Recommendations] --> B{OD Output and Override Application};

B --> C[Final OD Recommendations & Comprehensive Reports];

1. Start with → **Cleaned Sales Data and Initial Optimum Depth (OD) Recommendations.**
2. Then → **Initialize and Prepare Data for New Stores.**
3. Next → **Calculate and Apply All Overrides** (Computed Category, Manual Store, Manual Channel).
4. Next → **Normalize and Reconcile Final Contributions.**
5. Finally → **Generate Comprehensive Reports and Summary Tables.**

## 5. Step-by-Step Actions

1. **Initialize and Prepare** data for new stores.
  - What we do: *We set up the necessary data structures and ensure that any newly opened stores (which lack historical sales data) receive initial Optimum Depth (OD) recommendations by copying them from designated reference stores.*
  - Why we do it: *This provides new stores with a realistic and actionable stocking plan from day one, preventing them from being under-stocked or over-stocked based on zero sales history.*
2. **Calculate and Apply** initial revenue contributions.
  - What we do: *We determine how much each Attribute Group initially contributes to a store's or a channel's total revenue, using the cleaned sales data.*
  - Why we do it: *This establishes a baseline performance metric for each product group before any strategic adjustments are applied, allowing for transparency and tracking of override impacts.*
3. **Apply** computed category overrides.
  - What we do: *We incorporate strategic adjustments to an Attribute Group's contribution based on automatically generated overrides for its entire product category. This often involves scaling contributions to align with overall category importance.*
  - Why we do it: *This ensures that the OD recommendations reflect a top-down strategic view, ensuring that high-priority product categories receive proportional inventory investment.*
4. **Apply** manual store and channel overrides.
  - What we do: *We integrate explicit business decisions (e.g., for promotions, new product launches, regional strategies) that adjust the contribution of specific Attribute Groups at either the individual store level or across an entire sales channel.*

- Why we do it: *These manual overrides ensure the final OD recommendations are aligned with real-time merchandising plans and local market conditions, preventing the system from counteracting deliberate business interventions.*
5. **Reconcile and Normalize** final contributions.
- What we do: *After all overrides (computed and manual, at various levels) have been applied, we resolve any potential conflicts based on the Override Priority Rule and then adjust all Attribute Group contributions to ensure they sum up to 100% within each store or channel.*
  - Why we do it: *This crucial step ensures that the final OD recommendations are logically sound, that all product groups collectively account for the entire inventory allocation, and that the total stock value is accurately represented, preventing accidental over- or under-stocking overall.*
6. **Generate** final reports and summary tables.
- What we do: *We compile and denormalize the finalized Optimum Depth (OD) recommendations and other relevant data into comprehensive reports and summary tables, making them easily accessible for analysis and decision-making.*
  - Why we do it: *This step transforms raw data into actionable insights, enabling inventory planners, merchandisers, and business strategists to quickly understand and utilize the optimized OD recommendations for ordering, display, and overall assortment management.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A clothing retailer wants to finalize the Optimum Depth (OD) recommendations for "Women's Casual Dresses" (an Attribute Group - AG).
- **Store A (Downtown):** Initial system calculation suggests this AG should represent 8% of Store A's total inventory value.
- **Store B (Suburban Mall):** Initial system calculation suggests this AG should represent 7% of Store B's total inventory value.
- A **Computed Category Override** for the entire "Dresses" category states that, for Store A, all "Dresses" AGs should collectively account for 15% of the store's total inventory value (up from an initial 10% calculated sum across all dress AGs).
- A **Manual Store Override** for "Women's Casual Dresses" in Store B explicitly sets its target contribution to 10% (due to a planned local promotion).
- A **Manual Channel Override** for "Women's Casual Dresses" across the entire "Urban" channel (which includes both Store A and B) sets its target contribution to 9%.
- **New Store C (City Center):** Has no sales history but is configured to inherit its initial inventory planning from Store A.

## What Happens

1. **Data Initialization and New Store Preparation:** The system loads all calculated OD data. New Store C is identified as new, and a placeholder is prepared for it to receive data from Store A.
2. **Initial Contribution Calculation:** The system confirms the initial calculated contributions: “Women’s Casual Dresses” is at 8% for Store A and 7% for Store B.
3. **Apply Computed Category Overrides:** For Store A, the system notes the “Dresses” category override. If “Women’s Casual Dresses” initially contributed 8% when the category total was 10%, it now scales proportionally to the new 15% category target. So, its contribution for Store A increases to  $(8\% / 10\%) * 15\% = 12\%$ . This is now its primary recommendation.
4. **Apply Manual Store and Channel Overrides:**
  - For **Store B**, the Manual Store Override of 10% for “Women’s Casual Dresses” is applied. This override takes precedence over any calculated or broader channel contributions, directly setting its target to 10%.
  - For **Store A**, the Manual Channel Override of 9% for “Women’s Casual Dresses” is considered. According to the Override Priority Rule (manual channel overrides typically supersede computed category overrides), this 9% now overrides the previously computed 12%. So, Store A’s target becomes 9%.
5. **Reconcile and Normalize Final Contributions:** The module ensures that after all these individual adjustments, the total contributions of all AGs within Store A and Store B still sum up to 100% of their respective store’s total inventory value, proportionally adjusting any remaining “auto-calculated” AGs.
6. **Generate Final Reports and Summary Tables:** The system then outputs the finalized Optimum Depth recommendations:
  - For Store A: “Women’s Casual Dresses” is recommended at 9% of inventory value.
  - For Store B: “Women’s Casual Dresses” is recommended at 10% of inventory value.
  - For New Store C: “Women’s Casual Dresses” inherits the final 9% recommendation from its reference, Store A.

## Final Outcome

The business receives precise, strategically adjusted Optimum Depth (OD) recommendations for “Women’s Casual Dresses” tailored to each store and business mandate. Store A’s recommendation reflects the channel strategy, Store B’s incorporates a local promotion, and New Store C benefits from proven data immediately. This ensures that inventory investments are maximized, aligning with both data-driven insights and strategic business decisions, ultimately leading to higher sales and efficient stock management across all locations.

**Key Takeaway:** This module is the ultimate arbiter, synthesizing all calculated insights with crucial business overrides to produce actionable, optimized inventory recommendations that drive both profitability and strategic alignment.

## 7. Results

This step produces:



## Main Outputs

- **Final Optimum Depth (OD) Recommendations by Attribute Group and Store:** The definitive ideal number of unique items (SKUs) to stock for each Attribute Group in every store, incorporating all overrides. It tells us precisely what to stock and where to maximize sales and inventory efficiency.
- **Override Application Audit Trail:** Detailed records indicating which overrides (computed category, manual store, manual channel) were applied to each Attribute Group's recommendation. It provides transparency into how final decisions were reached and the impact of each override.
- **Normalized Revenue Contributions:** The final percentage contribution of each Attribute Group to a store's or channel's total inventory value, adjusted to sum to 100%. It tells us the proportional allocation of inventory across the assortment.
- **Denormalized Sales and Inventory Data:** Comprehensive datasets that combine raw sales, cleaned sales, and final OD recommendations into unified views for holistic analysis. It provides a complete picture for deep diving into performance.

## Business Reports

- **Final OD Recommendation Report:** Used daily by inventory planners and buyers to place orders, manage replenishment, and optimize store displays. This is the primary actionable output for managing stock.
- **Override Impact Analysis Report:** Used by business strategists, merchandising managers, and finance teams to evaluate the effectiveness and financial implications of all applied overrides. This report helps refine future strategic policies.
- **Store Assortment Contribution Summary:** Used by regional and store managers to understand the proportional inventory value and space allocated to different Attribute Groups within their specific locations. This aids in local merchandising and sales planning.
- **Inventory Efficiency Dashboard (Final):** Used by operations and finance leaders to monitor critical metrics like sell-through rates, stock-to-sales ratios, and overall inventory health after all OD adjustments are applied. This provides a high-level view of overall business performance.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The module will process outdated or incomplete initial OD data, leading to suboptimal final recommendations.	Investigate the source of missing or delayed data immediately from upstream OD calculation modules to ensure all inputs are current and complete.
Overrides conflict or result in	If multiple overrides (e.g., store and channel) apply to the same	Review the Override Priority Rule to ensure it correctly reflects business intent. Manually review and correct conflicting or

Situation	What Happens	What You Should Do
illogical values	AG and conflict, or a manual override leads to an unrealistic OD.	illogical override inputs to ensure sensible final recommendations.
Normalization leads to unexpected shifts	After normalization, some AGs' contributions might shift unexpectedly, even if no direct override applied to them.	Understand that normalization is a balancing act. If a specific AG's contribution is critical, consider adding a manual override for it to protect its value, or review why other AGs caused the proportional shift.
New stores receive incongruent data	A new store's initial OD recommendations don't align with its perceived market or desired strategy.	Re-evaluate the New Store Data Inheritance Policy and ensure the assigned reference store accurately reflects the new store's expected performance and local market conditions.
Denormalized reports are slow or inaccurate	Large volumes of data or complex joins cause performance issues or incorrect aggregated values in final reports.	Optimize the database queries and data structures used for denormalization. Regularly audit report accuracy against raw data to ensure reliability for business users.

## 9. Next Steps

### Before This Step

- **All preceding OD Calculation Modules:** This module is the final stage, receiving the outputs from all previous calculations, including cleaned sales data, identified long-tail products, ASPs, segmented Attribute Groups, and all computed/manual override inputs. It relies on these modules for raw material.

### After This Step

- **Inventory Management Systems:** The finalized Optimum Depth (OD) recommendations are transmitted directly to inventory planning, ordering, and replenishment systems. This makes the recommendations actionable for purchasing and stock management.
- **Merchandising and Assortment Planning Tools:** The detailed OD recommendations, along with assortment performance summaries, inform merchandising teams on product placement, display strategies, and future assortment development (e.g., Optimum Width - OW decisions).

- **Financial Reporting and Analysis:** The final OD outputs, especially in conjunction with ASPs, feed into financial models for inventory valuation, cost of goods sold analysis, and profitability forecasting.

## 10. Common Questions

Q: Why do my product recommendations change so much after the system applies overrides? A: This module's purpose is to incorporate all strategic business decisions, like promotions, new product launches, or specific category priorities, into the calculated recommendations. These manual and computed overrides take precedence to ensure your inventory aligns with current business goals, even if it means adjusting initial data-driven suggestions.

Q: How does the system decide which override to use if there are multiple for one product? A: The system follows a predefined **Override Priority Rule**. Generally, more specific overrides (like a manual override for a single store) take precedence over more general ones (like a channel-wide override). Manual overrides also typically override automatically computed ones, ensuring your direct instructions are followed.

Q: What happens if a new store opens and has no sales history? A: For new stores, the system applies the **New Store Data Inheritance Policy**. This means it will copy the Optimum Depth recommendations from a designated "reference store" (an existing, similar store) or apply broader channel averages. This ensures the new store is stocked appropriately from day one.

Q: What are these "denormalized tables" mentioned in the output? A: Denormalized tables are special reports that combine data from many different sources (like sales, inventory, and our calculated OD recommendations) into one easy-to-read format. They're designed to give you a complete picture of product performance and inventory health without needing to look at multiple separate reports.

Q: How does this module prevent me from ordering too much or too little overall? A: The module applies a **Revenue Contribution Sum Target**, typically 100%. After all overrides are applied, it ensures that the combined contribution of all product groups within a store or channel adds up to exactly 100%. This re-balances inventory allocations automatically, preventing accidental over- or under-stocking across your entire assortment.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To finalize Optimum Depth (OD) recommendations by applying all computed and manual overrides and generating comprehensive reports.
- **Main Input:** Initial OD recommendations, cleaned sales data, and all category, store, and channel override inputs.
- **Main Output:** Final OD recommendations by Attribute Group and store, detailed override audit trails, and various denormalized business reports.
- **Warning Signs:** Unexpectedly high or low final OD recommendations for specific product groups after overrides, or reports that don't sum correctly.

- **Key Settings:** Override Priority Rule, New Store Data Inheritance Policy, Revenue Contribution Sum Target, Reporting Granularity.
- 

**Need Help?** Contact your Inventory Planning Systems team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# OD Algorithm FAQ

**Version:** 1.0 **Last Updated:** 2025-05-23

**Purpose:** This document provides answers to common business questions about the Optimum Depth (OD) algorithm, designed to optimize product assortment.

---

## General

**Q:** What is Optimum Depth (OD)? **A:** Optimum Depth (OD) represents the ideal number of unique items (SKUs) you should stock within a specific category or display to maximize sales and minimize inventory holding costs. It helps ensure you have enough variety without overstocking.

**Q:** What problem does the OD algorithm solve? **A:** The OD algorithm solves the challenge of determining the optimal assortment size. It helps businesses avoid stocking too many low-performing items (leading to excess inventory and markdowns) or too few high-performing items (leading to missed sales opportunities and stockouts).

**Q:** What is the primary goal of using this OD algorithm? **A:** The main goal is to optimize product assortment by providing data-driven recommendations for the ideal number of SKUs (Optimum Depth - OD) per category or Attribute Group (AG), ultimately improving sales efficiency and inventory management.

**Q:** How does Optimum Depth (OD) relate to Optimum Width (OW)? **A:** While OD focuses on the ideal number of unique *items* within a category, Optimum Width (OW) refers to the ideal number of *different categories or styles* to offer. The OD algorithm contributes to initial OW metrics by identifying core, high-performing items during the “OD Long Tail Computation” phase, helping inform broader assortment strategies.

**Q:** How does this algorithm benefit merchandising and inventory teams? **A:** It provides actionable insights for assortment planning, helping merchandising teams make informed decisions on what to stock. For inventory teams, it leads to more accurate stocking levels, reducing carrying costs, improving stock turns, and minimizing the risk of obsolescence.

---

## Configuration & Settings

**Q:** How are the various algorithm settings and parameters managed? **A:** Key configuration settings for the OD algorithm are typically defined and managed through an `args.properties` file. This file allows for adjustments to thresholds, date ranges, and other parameters that influence the calculation logic.

**Q:** Can we adjust the sensitivity of the OD calculation, for instance, for new or seasonal products? **A:** Yes, the algorithm's sensitivity can be adjusted through configuration parameters. While not explicitly detailed here, settings related to seasonal sales filtering or the definition of "first N weeks of ROS" (Rate of Sale) allow for tailoring the calculation to different product lifecycles or promotional periods. Refer to the `args.properties` file for specific parameters.

**Q:** How are manual overrides applied to the OD recommendations? **A:** The algorithm supports manual overrides at various levels (category, store, and channel). These overrides, provided as input data, are applied in the "OD Output and Override Application" module, ensuring that business strategy and specific market knowledge can supersede algorithmic recommendations when necessary.

**Q:** What are "first N weeks of ROS" and how are they used in the algorithm? **A:** "First N weeks of ROS" refers to using a specified number of initial weeks of sales data to calculate the Rate of Sale (ROS). This is crucial for accurately assessing product performance, especially for newer items, and is processed during the "OD Data Preparation" phase to ensure relevant sales periods are used for OD analysis.

**Q:** Where can I find the specific configuration parameters that define what constitutes a "long tail" style? **A:** Parameters defining "long tail" styles, such as minimum Rate of Sale (ROS) thresholds or revenue contribution benchmarks, are part of the algorithm's configuration. These are managed via the `args.properties` file, influencing the "OD Long Tail Computation" module.

---

## Data & Inputs

**Q:** What types of raw data are required to run the OD algorithm? **A:** The OD algorithm requires comprehensive raw input data including transactional sales records, current inventory levels, and various override configurations (e.g., category-specific rules, manual adjustments). This data is loaded and validated by the "OD Input and Validation" module.

**Q:** How does the algorithm ensure the quality of the input data? **A:** The "OD Input and Validation" module performs initial data integrity checks. Following this, the "OD Data Preparation" module applies further cleaning steps, such as seasonal sales filtering, removal of "brokenness" (inventory discrepancies), and cleaning liquidated sales, to ensure accuracy before processing.

**Q:** Why is "brokenness" removed from the data, and what does it mean? **A:** "Brokenness" refers to discrepancies or inconsistencies in inventory data that could distort sales and inventory metrics. Removing it, as handled by the "OD Data Preparation" module, ensures that calculations are based on accurate available-to-sell inventory and sales, leading to more reliable Optimum Depth (OD) recommendations.

**Q:** How are liquidated sales (e.g., highly discounted items) handled? **A:** Liquidated sales are identified and cleaned based on predefined discount benchmarks during the "OD Data Preparation" phase. This prevents unnaturally high sales volumes from distorting the true Rate of Sale (ROS) for regular-priced items, ensuring the Optimum Depth (OD) calculations reflect typical selling conditions.

**Q:** Are new products or Attribute Groups (AGs) automatically considered? **A:** Yes, the “OD Segmentation and Introduction” module dynamically introduces new Attribute Groups (AGs) from other stores within a channel/region or re-introduces high-performing, previously exited AGs. This ensures the algorithm accounts for newness and re-introductions in optimizing the product assortment.

---

## Process & Logic

**Q:** How does the algorithm determine the “ideal” number of items (Optimum Depth - OD)? **A:** The algorithm determines OD by segmenting Attribute Groups (AGs) into performance categories (TOP, MODERATE, SLOW) based on their Rate of Sale (ROS) and other revenue metrics. This segmentation, performed by the “OD Segmentation and Introduction” module, forms the basis for recommending the optimal number of unique items.

**Q:** What is “long tail” analysis, and why is it important for OD? **A:** “Long tail” analysis, performed by the “OD Long Tail Computation” module, identifies and filters out styles or SKUs within Attribute Groups (AGs) that have a low Rate of Sale (ROS) or minimal revenue contribution. This is crucial for OD because it ensures recommendations focus on core, high-performing items, preventing overstocking of slow movers.

**Q:** How are Attribute Groups (AGs) segmented for analysis? **A:** Attribute Groups (AGs) are segmented into “TOP,” “MODERATE,” and “SLOW” performance categories. This segmentation is based on their Rate of Sale (ROS) and other relevant revenue metrics computed across the pipeline, as detailed in the “OD Segmentation and Introduction” module.

**Q:** What is Average Selling Price (ASP) and why is it calculated as part of the OD process? **A:** Average Selling Price (ASP) is the average price at which items within an Attribute Group (AG) are sold. It’s calculated by the “OD Average Selling Price (ASP) Calculation” module at multiple granularity levels (store, region, channel) to provide accurate pricing context. ASP can influence assortment planning decisions, especially when considering the revenue potential of different OD recommendations.

**Q:** What is the role of the “OD Calculation Pipeline Orchestrator”? **A:** The “OD Calculation Pipeline Orchestrator” serves as the master controller for the entire OD project. It defines the sequential execution of all sub-modules, ensuring that data flows correctly from input and validation through processing, analysis, segmentation, and finally to output generation and synchronization.

---

## Outputs & Results

**Q:** What are the final outputs and recommendations generated by the OD algorithm? **A:** The algorithm generates comprehensive final reports and summary tables that reflect the ideal product depth recommendations (Optimum Depth - OD) after all computations and business rules have been applied. These outputs are produced by the “OD Output and Override Application” module.

**Q:** How are the OD recommendations delivered to business users? **A:** The recommendations are typically delivered in structured data formats (e.g., tables, reports) that can be integrated into existing planning systems or viewed in business intelligence dashboards. The “OD Output and Override Application” module is responsible for denormalizing data points for these comprehensive reports.

**Q:** Can we see the impact of any manual overrides in the final output? **A:** Yes, the “OD Output and Override Application” module explicitly applies all computed and manually provided overrides (category, store, and channel levels). The final reports will reflect these adjustments, showing the result of the strategic adjustments on the algorithmic recommendations.

**Q:** Does the algorithm provide insights beyond just the ideal number of SKUs? **A:** Yes, while the core output is Optimum Depth (OD), the pipeline also generates foundational data and metrics, such as Rate of Sale (ROS) by Attribute Group (AG), Average Selling Price (ASP), and insights from long-tail analysis. These metrics, used throughout the pipeline, can be part of the comprehensive reports generated.

**Q:** What level of detail are the final OD recommendations provided at? **A:** Optimum Depth (OD) recommendations are typically provided at the Attribute Group (AG) level, often broken down by store, region, or channel, depending on the granularity of the input data and the configuration. This allows for detailed assortment planning.

---

## **Error Handling & Edge Cases**

**Q:** What happens if there’s missing or incomplete input data? **A:** The “OD Input and Validation” module is designed to identify missing or incomplete raw input data. Depending on the severity, it may flag errors, prevent the algorithm from running, or attempt to use default values if configured. Issues are typically logged for review.

**Q:** How does the algorithm handle categories or Attribute Groups (AGs) with very low or no sales history? **A:** Categories or Attribute Groups (AGs) with insufficient sales history may be categorized as “F” (for exited/zero revenue) or “UA” (unassigned) during the “OD Category Override Computation” phase. The algorithm will then either exclude them from detailed OD computation or rely on channel-level defaults or manual overrides.

**Q:** Are there mechanisms to detect and flag unusual sales spikes or drops that might skew results? **A:** The “OD Data Preparation” module includes steps like seasonal sales filtering and cleaning of liquidated sales, which implicitly help mitigate some unusual data patterns. However, for extreme anomalies, human review of the input data before processing is always recommended.

**Q:** How are calculation errors or unexpected results communicated? **A:** The algorithm is designed to log errors and warnings throughout its various modules. While the exact communication method depends on the implementation, critical issues would typically halt the process or trigger alerts for the support team to investigate.



**Q:** What if the business logic or market conditions change rapidly? How flexible is the algorithm? **A:** The algorithm's flexibility largely comes from its configurable parameters (via `args.properties`) and the ability to apply manual overrides at different levels. This allows business teams to adapt the recommendations to evolving market conditions or strategic priorities without requiring code changes.

---

## **Need More Help?**

For further technical details, specific configurations, or support, please contact the algorithm development or support team.

# OD Documentation

- **Version:** 1.0
  - **Last Updated:** 2025-05-23
- 

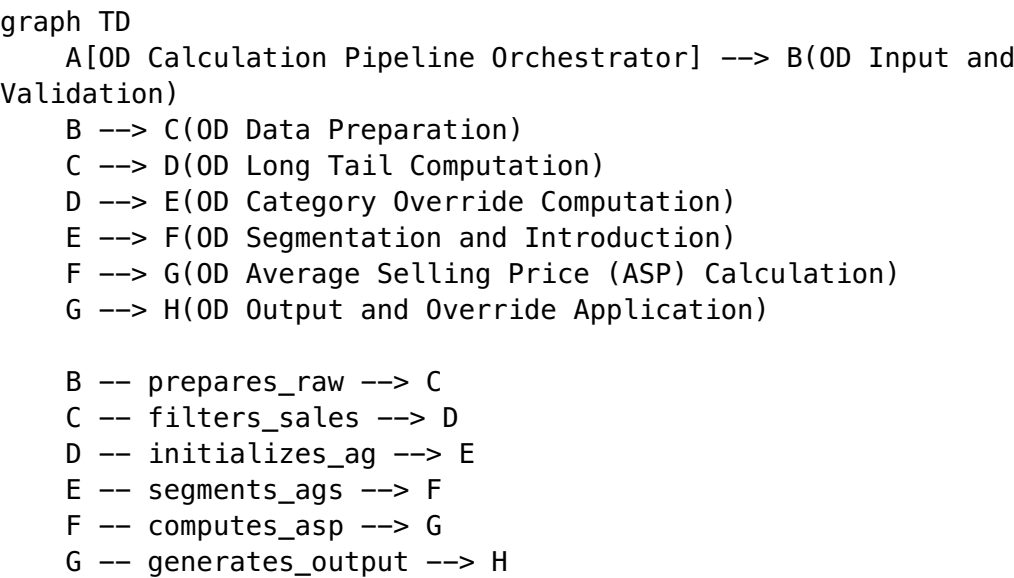
## 1. Executive Summary

The Optimum Depth (OD) calculation algorithm determines the ideal number of unique items (SKUs) to stock within a product category or display. By intelligently analyzing sales performance, inventory, and strategic overrides, this solution helps businesses optimize their product assortments, maximize sales by focusing on high-performing items, and minimize potential stockouts or overstock. It empowers data-driven merchandising decisions to drive profitability.

---

## 2. Architecture Overview

The Optimum Depth (OD) calculation process follows a sequential pipeline, beginning with raw data input and validation, then proceeding through iterative data preparation, long-tail filtering, and metric computations. This refined data is used for segmentation and Average Selling Price (ASP) calculations, before all computed and manual overrides are applied to finalize the product depth recommendations and generate comprehensive reports.



```
B -- provides_overrides --> H
E -- applies_computed_overrides --> H
```

### Module Descriptions:

- **OD Calculation Pipeline Orchestrator:** This module serves as the master orchestrator, defining the sequential execution of all sub-modules for the entire OD calculation project.
  - **OD Input and Validation:** This module loads and validates all raw input data, including transactional sales, inventory, and override configurations, preparing it for processing.
  - **OD Data Preparation:** This module processes raw input data by applying cleaning, filtering, and period identification steps for accurate OD analysis.
  - **OD Long Tail Computation:** This module analyzes sales data to identify and filter out “long tail” styles (SKUs) with low ROS or minimal revenue contribution, refining OD.
  - **OD Category Override Computation:** This module calculates revenue contributions at the category level and sets initial segmentation tags for Attribute Groups (AGs).
  - **OD Segmentation and Introduction:** This module determines the ideal OD by segmenting AGs into performance categories and dynamically introducing or re-introducing AGs from other stores.
  - **OD Average Selling Price (ASP) Calculation:** This module calculates the Average Selling Price (ASP) for each AG relevant to OD recommendations at multiple granularity levels.
  - **OD Output and Override Application:** This module is the final stage, applying all computed and manual overrides to segmented data and generating comprehensive final reports.
- 

## 3. Prerequisites & Setup

To run the OD calculation pipeline, ensure the following prerequisites are met:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Build Tool:** Apache Maven 3.6.3 or higher.

### Build Commands:

#### 1. Clone the repository:

```
git clone <repository-url>/od/
cd od/
```

#### 2. Build the project:

```
mvn clean install
```

This command compiles the source code, runs tests, and packages the application.

**Dependencies:**

- All necessary dependencies are managed via `pom.xml` and will be downloaded automatically during the `mvn install` process.

**Running the ModuleGroup Orchestrator:**

The main entry point for the OD pipeline is the OD Calculation Pipeline Orchestrator module. After building the project, you can execute it from the command line:

```
java -jar target/od-1.0.jar --config-path=/path/to/your/args.properties
```

Replace `/path/to/your/args.properties` with the actual path to your configuration file.

---

# 4. Folder & File Structure

The project repository follows a standard structure designed for clarity and maintainability:

```
od/
├─ module-group/ # Contains the orchestrator module
 (OD Calculation Pipeline Orchestrator) and overall pipeline logic.
├─ submodules/ # Houses individual, self-contained
 business logic modules for specific OD calculations.
├─ args.properties # The primary configuration file,
 defining global parameters and settings for the OD pipeline.
├─ common-data.json # Defines the shared data model and
 schema used across various modules for consistency.
├─ outputs/ # Directory where all final reports
 and processed data outputs are generated.
```

---

# 5. Global Configuration

The `args.properties` file contains global configuration parameters that control the behavior of the OD pipeline.

Setting Name	Default Value	Description	Valid Range/ When to Change
od.analysis.startDate	2024-01-01	The start date for historical sales data analysis.	YYYY-MM-DD format; change for different analysis periods.
od.analysis.endDate	2024-03-31		

Setting Name	Default Value	Description	Valid Range/ When to Change
		The end date for historical sales data analysis.	YYYY-MM-DD format; change for different analysis periods.
od.ros.periodWeeks	13	Number of weeks to consider for Rate of Sale (ROS) calculation.	Integer (e.g., 8, 13, 26); adjust based on product lifecycle.
od.min.revenueThreshold	1000.0	Minimum revenue for an AG to be considered for OD.	Decimal; adjust to filter out low-value Attribute Groups.
od.enable.overrides	true	Flag to enable/disable manual input overrides.	true or false; disable for pure algorithmic recommendations.

## 6. Shared Data Model

The `common-data.json` schema defines the structure for data objects exchanged between modules and used throughout the OD pipeline.

Field	Type	Description
storeId	String	Unique identifier for the retail store.
categoryId	String	Unique identifier for the product category.
styleId	String	Unique identifier for a product style (e.g., specific shoe model).
skuId	String	Unique identifier for a Stock Keeping Unit (e.g., red shoe, size 10).
agId	String	Unique identifier for an Attribute Group (AG), grouping similar SKUs.
salesUnits	Integer	Number of units sold for a given SKU/style/AG within a period.
salesRevenue	Decimal	Total revenue generated for a given SKU/style/AG within a period.
inventoryUnits	Integer	Current on-hand inventory quantity for a given SKU.
liveDays	Integer	Number of days a product was available for sale in a store.

Field	Type	Description
ros	Decimal	Rate of Sale (units/week) for a product or Attribute Group.
segment	String	Performance segment of an AG (e.g., “TOP”, “MODERATE”, “SLOW”, “F”, “UA”).
asp	Decimal	Average Selling Price for a product or Attribute Group.
overrideType	String	Type of manual or computed override (e.g., “Category”, “Store”, “Channel”).
overrideValue	Integer	The recommended Optimum Depth value from an override.

## 7. Submodule Chapters

### OD Input and Validation

- Module Name & Business Objective OD Input and Validation** is responsible for loading all raw input data required for Optimum Depth (OD) calculations, including transactional sales, inventory, and various override configurations. Its business objective is to *ensure the foundational data for OD analysis is complete, accurate, and ready for processing*, minimizing downstream errors and ensuring reliable recommendations.
- Required Information**
  - **Sales Data:** Transactional sales records (units, revenue, dates, SKU details).
  - **Product Data:** SKU and Attribute Group (AG) master data, product hierarchy (category, style relationships).
  - **Operational Data:** Current inventory levels, manual OD override files (category, store, channel level), Average Selling Price (ASP) override files.
- Business Settings** Not specified for this module.
- Process Flow**

```
graph TD
 A[Load Raw Sales Data] --> B[Load Inventory Data]
 B --> C[Load Override Configurations]
 C --> D{Validate Data Integrity?}
 D -- No --> E[Log Errors & Exit]
 D -- Yes --> F[Prepare Structured Data for Pipeline]
```
- Step-by-Step Actions**
  - **Retrieve** raw transactional sales data from source systems. *This data forms the basis for ROS calculations and revenue contributions.*
  - **Load** current inventory snapshots for all relevant SKUs. *Accurate inventory is crucial for identifying stockouts and analyzing product availability.*

- **Import** all pre-defined manual override configurations (e.g., strategic OD targets for specific categories or stores). *These overrides incorporate business strategy that might supersede algorithmic recommendations.*
- **Validate** the completeness and consistency of all loaded data, checking for missing values, invalid formats, or logical inconsistencies. *Early validation prevents erroneous calculations and ensures data quality.*
- **Transform** raw data into structured maps and objects, such as `OdData` and `Cache`, for efficient downstream processing. *This step optimizes data access and performance for subsequent modules.*

6. **Sample Scenario** A new monthly OD run is initiated. The OD Input and Validation module attempts to load sales data for Q1 2025. It finds that sales data for February 15th is missing for 5% of stores and that the inventory file has mismatched skuId values for 20 SKUs. The module identifies these discrepancies, logs specific error messages indicating the missing dates and invalid SKUs, and flags the run as having data quality issues, preventing further processing until corrected. **Key Takeaway:** *Robust input validation ensures that subsequent analytical steps operate on reliable data, preventing “garbage in, garbage out” scenarios.*

## 7. Results

- Validated and structured sales, inventory, and override data objects (`odData` instance).
- Error logs detailing any data inconsistencies or missing information.
- Confirmation of data readiness for subsequent pipeline modules.

## 8. Unusual Cases

- **Missing or delayed data feed:** If critical input files (sales, inventory, overrides) are absent or significantly delayed, the module will halt processing and log an error.
- **Schema mismatch:** Changes in source data schema (e.g., a new column, changed data type) without corresponding updates to the ingestion logic will cause validation failures.
- **High volume of invalid records:** A large number of corrupted or malformed records might be detected, potentially indicating a systemic data quality issue at the source.

9. **Next Steps** Upon successful completion, the validated and structured data is passed to the OD Data Preparation module for initial cleaning and filtering.

## 10. Common Questions

1. Why did my OD run fail at the start? *Likely due to missing or malformed input files, check the logs for specific file names and errors.*
2. How can I provide manual overrides? *Ensure your override files (`od_input_override.csv`, `od_asp_override.csv`) are correctly formatted and placed in the designated input directory.*
3. What data sources does this module connect to? *Typically, enterprise data warehouses, POS systems, and inventory management systems.*
4. Can I see the raw input data after it's loaded? *The raw data is transformed internally; review the input files themselves or system logs for initial loaded data.*

5. How often should input data be refreshed? *Input data should be as current as possible, ideally reflecting the latest daily or weekly sales and inventory figures for accurate OD recommendations.*

## 11. Quick Reference

- **Purpose:** Data Ingestion & Validation
  - **Inputs:** Raw Sales, Inventory, Override files
  - **Outputs:** Validated odData objects
  - **Key Action:** load() and validate()
- 

# OD Data Preparation

1. **Module Name & Business Objective OD Data Preparation** processes raw input data by applying several crucial cleaning and filtering steps. Its business objective is to *refine and consolidate raw sales and inventory data into a clean, consistent, and relevant dataset* suitable for accurate Optimum Depth (OD) analysis, ensuring that calculations are based on valid and representative sales periods.

## 2. Required Information

- **Sales Data:** Raw transactional sales data (from OD Input and Validation).
- **Product Data:** SKU and AG mapping, product live dates.
- **Operational Data:** Inventory data, configuration parameters for filtering (e.g., ROS period weeks, liquidation thresholds).

3. **Business Settings** Not specified for this module.

## 4. Process Flow

graph TD

```
A[Raw Sales & Inventory] --> B(Apply Seasonal Filter)
B --> C(Remove Brokenness)
C --> D(Clean Liquidated Sales)
D --> E(Identify Relevant Sales Periods)
E --> F[Cleaned & Filtered Data]
```

## 5. Step-by-Step Actions

- **Filter** sales data to include only relevant seasons or time periods, excluding outliers. *This ensures that OD recommendations are based on typical selling patterns, not anomalous events.*
- **Remove** “brokenness” by identifying and correcting inventory discrepancies or misalignments that could distort sales metrics. *Accurate inventory data is vital for calculating true ROS and stockout potential.*
- **Clean** liquidated sales by adjusting or excluding sales associated with deep discounts or clearance events. *This prevents distorted ROS figures from non-representative pricing strategies.*
- **Identify** and define relevant sales and inventory periods, such as the first N weeks of ROS, or specific product live days. *Focusing on active selling periods provides the most accurate performance indicators for OD.*



- **Aggregate** sales and inventory data to the necessary Attribute Group (AG) and store levels. *This prepares the data for higher-level analysis in subsequent modules.*

6. **Sample Scenario** The OD pipeline runs for a winter apparel category. OD Data Preparation receives raw sales data including a massive flash sale event in June. The module, configured with seasonal filtering rules, *excludes* these June sales from the analysis, focusing only on typical winter selling months (e.g., Sept-Feb). Additionally, it identifies that a popular jacket SKU shows sales but zero inventory for a week; it flags this “brokenness” and adjusts the ROS calculation to account for the presumed stockout period. **Key Takeaway:** *Cleaning and filtering raw data is critical to ensure that OD calculations reflect true product performance, not data anomalies or non-standard selling conditions.*

## 7. Results

- Cleaned, filtered, and aggregated sales data (e.g., `relSalesMap`).
- Adjusted inventory data reflecting corrected “brokenness.”
- Defined relevant sales and inventory periods (e.g., `highPriorityStoreStyleLiveDaysMap`).

## 8. Unusual Cases

- **Missing or delayed data feed:** Reliance on OD Input and Validation means downstream impact if that module fails or delivers incomplete data.
- **Unusual liquidation events:** If liquidation sales are not clearly tagged or exceed predefined thresholds, they might still skew ROS calculations.
- **Persistent inventory discrepancies:** Chronic “brokenness” issues in the source system can make accurate ROS and OD calculations challenging, requiring manual data intervention.

9. **Next Steps** The refined and prepared sales and inventory data is passed to the OD Long Tail Computation module for further analysis and filtering of low-performing items.

## 10. Common Questions

1. Why are some sales transactions not included in the final analysis? *This module filters based on seasonal relevance, liquidation status, and data quality; check filtering parameters.*
2. What does “removing brokenness” mean? *It involves identifying and correcting inconsistencies between reported sales and available inventory that could distort performance metrics.*
3. How are “relevant sales periods” determined? *Based on configured look-back windows (e.g., 13 weeks of ROS) and actual product live days in store.*
4. Can I adjust the liquidation threshold? *Yes, this is typically a configurable parameter in `args.properties` to define what constitutes a “liquidated” sale.*
5. What happens if seasonal filtering removes too much data? *Review the `args.properties` settings for seasonal date ranges to ensure they align with the business’s current operating calendar.*

## 11. Quick Reference

- **Purpose:** Data Cleaning & Filtering
  - **Inputs:** Raw Sales, Inventory (from OD Input & Validation)
  - **Outputs:** Cleaned sales & inventory data for relevant periods
  - **Key Action:** `filter()`, `clean()`
- 

## OD Long Tail Computation

1. **Module Name & Business Objective** **OD Long Tail Computation** analyzes sales data to identify and filter out “long tail” styles (SKUs) within Attribute Groups (AGs) that have low Rate of Sale (ROS) or minimal revenue contribution. Its business objective is to *refine Optimum Depth (OD) by focusing on core, high-performing items*, reducing inventory waste on slow-moving products, and providing foundational data for initial Optimum Width (OW) metrics.

### 2. Required Information

- **Sales Data:** Cleaned and filtered sales data, including sales units and revenue (from OD Data Preparation).
- **Product Data:** SKU-level and AG-level product information, live days.
- **Operational Data:** Configured thresholds for long tail identification (e.g., minimum ROS, minimum revenue contribution).

3. **Business Settings** Not specified for this module.

### 4. Process Flow

```
graph TD
 A[Cleaned Sales Data] --> B[Calculate Style ROS & Revenue per Live Day]
 B --> C[Calculate AG Revenue per Live Day]
 C --> D{Filter Out Long Tail Styles?}
 D -- Yes --> E[Exclude from OD Consideration]
 D -- No --> F[Identify Core Items]
 F --> G[Initial OD & OW Metrics]
```

### 5. Step-by-Step Actions

- **Calculate** style-level and AG-level Rate of Sale (ROS) and revenue per live day. *These metrics are crucial for assessing the true performance of each product.*
- **Identify** and categorize “long tail” styles based on predefined thresholds for ROS or revenue contribution. *This step helps to de-emphasize products that contribute minimally to overall sales.*
- **Filter out** identified long-tail styles from the core dataset used for OD calculation. *Removing these low-performing items ensures that the focus remains on profitable and high-demand inventory.*
- **Populate** initial `OdCatBaseUnit` and `OdAgBaseUnit` objects with refined sales and performance metrics. *These foundational data structures will be used in subsequent segmentation and override computations.*

- **Generate** preliminary insights that can inform Optimum Width (OW) discussions, focusing on the breadth of categories to offer based on core item performance. *This starts to connect OD with broader assortment strategies.*

6. **Sample Scenario** An AG (“Women’s Basic T-shirts”) contains 50 styles. OD Long Tail Computation analyzes their sales. It finds that 5 styles have sold only 1 unit in the last 13 weeks, contributing less than 0.1% to the AG’s total revenue, well below the configured long-tail threshold. The module *flags these 5 styles as “long tail” and excludes them* from consideration for Optimum Depth, ensuring that the final OD recommendation focuses on the remaining 45 higher-performing styles. **Key Takeaway:** *Filtering out long-tail items based on performance ensures that inventory resources are allocated to products that actively drive sales and profitability, improving the efficiency of the assortment.*

## 7. Results

- Updated odData with style and AG-level ROS and revenue per live day.
- Identification and flagging of “long tail” styles for exclusion.
- Initial OdCatBaseUnit and OdAgBaseUnit objects populated with core item data.

## 8. Unusual Cases

- **Seasonal fluctuations:** If a product’s “long tail” status is highly seasonal, applying a fixed threshold might prematurely exclude items that perform well in other periods (requiring dynamic thresholds or period-specific analysis).
- **New product ramp-up:** Newly introduced products might initially appear as “long tail” due to low sales volume, potentially requiring a grace period before being filtered.
- **High-value, low-volume items:** Certain products (e.g., luxury goods) naturally have low ROS but high revenue contribution; a simple ROS filter might incorrectly classify them as long tail, requiring a combined metric.

9. **Next Steps** The refined AG data, with long-tail items filtered, is passed to the OD Category Override Computation module to establish initial category segmentation and apply further overrides.

## 10. Common Questions

1. What defines a “long tail” product? *It’s typically defined by a combination of low Rate of Sale (ROS) and minimal revenue contribution over a specific period, configurable via parameters.*
2. Will “long tail” products ever be restocked? *The OD recommendation may suggest not stocking them; however, business overrides or future performance spikes could reintroduce them.*
3. How does this impact Optimum Width (OW)? *By identifying core, high-performing styles, it helps define the truly successful categories to expand or maintain.*
4. Can the thresholds for filtering be adjusted? *Yes, the minimum ROS or revenue thresholds for long-tail filtering are typically configurable parameters in args.properties.*

5. What if a product is high-margin but low-volume? *The module considers both ROS and revenue. For such cases, the revenue per live day metric helps retain valuable but slower-moving items.*

## 11. Quick Reference

- **Purpose:** Long Tail Filtering, Initial AG Metrics
  - **Inputs:** Cleaned Sales Data (from OD Data Preparation)
  - **Outputs:** Filtered AG data, OdCatBaseUnit, OdAgBaseUnit
  - **Key Action:** calculateROS(), filterLongTail()
- 

# OD Category Override Computation

## 1. Module Name & Business Objective OD Category Override Computation

calculates revenue contributions at the category level and sets initial segmentation tags for Attribute Groups (AGs) (e.g., “F” for exited/zero revenue, “UA” for unassigned). Its business objective is to *prepare foundational data for subsequent deep segmentation and complex override applications* in the Optimum Depth (OD) process, ensuring that category-level insights inform AG-level decisions.

## 2. Required Information

- **Product Data:** AG-level data with revenue and live day metrics (from OD Long Tail Computation).
- **Sales Data:** Aggregated revenue contributions by category.
- **Operational Data:** Configuration for initial segmentation rules.

## 3. Business Settings Not specified for this module.

## 4. Process Flow

```
graph TD
 A[AG-level Revenue & Live Days] --> B[Compute Category Revenue Contribution]
 B --> C[Set Initial AG Segmentation Tags]
 C --> D[Store Computed Category Overrides]
 D --> E[Foundational AG & Category Data]
```

## 5. Step-by-Step Actions

- **Access** the foundational OdCatBaseUnit and OdAgBaseUnit data structures, which contain initial revenue and live day metrics. *This ensures that computations are based on the refined dataset after long-tail filtering.*
- **Compute** category-level revenue contributions by aggregating revenue across all AGs within each category. *Understanding category performance is crucial for strategic resource allocation.*
- **Set** initial segmentation tags for each Attribute Group (AG), such as “F” for AGs with zero revenue or designated as “exited,” and “UA” for unassigned or unclassified AGs. *This initial tagging streamlines subsequent deep segmentation.*
- **Store** OdComputedCatOverrideRow to the database, reflecting calculated category-level overrides based on performance. *These computed overrides*

*provide data-driven recommendations for adjusting OD based on category health.*

- **Prepare** the AG data with revPerLiveDay and revPerLiveDaysPerStyle metrics for the next stage of segmentation. *This ensures a smooth flow of comprehensive data for detailed analysis.*

6. **Sample Scenario** The module analyzes AGs in the “Activewear” category. It discovers an AG, “Men’s Track Pants (Old Style),” which has had zero sales and zero live days for the past two quarters, indicating it’s an exited product. The module *automatically assigns an “F” (Exited) tag to this AG within the odData*. Concurrently, it calculates that “Activewear” contributes 15% of total store revenue, which can be used later to prioritize OD recommendations for high-contributing categories. **Key Takeaway:** *Initial category-level analysis and AG segmentation streamline the overall OD process by categorizing products based on their immediate status, providing a strategic foundation for detailed analysis.*

## 7. Results

- Updated odData with category-level revenue contributions.
- Initial segmentation tags (e.g., “F”, “UA”) assigned to AGs.
- OdComputedCatOverrideRow records stored, reflecting computed category overrides.

## 8. Unusual Cases

- **Misclassified AGs:** If an AG is incorrectly categorized, its revenue contribution might be attributed to the wrong category, skewing results.
- **New category introduction:** A brand new category might initially show low revenue, but tagging it “UA” might not be ideal if it’s a strategic growth area.
- **Seasonality impacting “exited” status:** An AG that is out of season might appear to have zero revenue, leading to an incorrect “F” tag if not handled with seasonal logic.

9. **Next Steps** The AGs with initial segmentation and category-level insights are passed to the OD Segmentation and Introduction module for deeper performance-based segmentation and dynamic AG introduction.

## 10. Common Questions

1. Why is an AG tagged “F” (Exited)? *It means the AG has had zero sales and is likely no longer active in the store or considered for future inventory.*
2. What’s the difference between “F” and “UA”? *“F” is for exited/zero-revenue AGs, while “UA” means unassigned or not yet classified into a performance segment.*
3. How does this module influence the final OD? *It provides foundational insights for category importance and initial AG status, guiding later segmentation and override applications.*
4. Can I manually override an “F” tag? *Manual overrides are handled in OD Output and Override Application, but business rules for “F” tagging should be reviewed if concerns arise.*
5. What are OdComputedCatOverrideRow used for? *These are system-generated recommendations for category-level depth based on performance, applied in the final output stage.*

## 11. Quick Reference

- **Purpose:** Initial Category & AG Segmentation
  - **Inputs:** OdCatBaseUnit, OdAgBaseUnit (from OD Long Tail Comp.)
  - **Outputs:** Category revenue contribution, initial AG segments, OdComputedCatOverrideRow
  - **Key Action:** computeCategoryRevenue(), assignInitialSegment()
- 

## OD Segmentation and Introduction

### 1. Module Name & Business Objective OD Segmentation and Introduction

determines the ideal number of unique items (Optimum Depth - OD) by segmenting Attribute Groups (AGs) into performance categories (TOP, MODERATE, SLOW) based on their Rate of Sale (ROS) and other revenue metrics. Its business objective is to *dynamically optimize the product assortment* by focusing on high-performing segments and strategically introducing new or re-introducing previously exited AGs to maximize sales potential.

### 2. Required Information

- **Product Data:** AG-level data with revPerLiveDay and revPerLiveDaysPerStyle metrics, and initial segmentation tags (from OD Category Override Computation).
- **Sales Data:** ROS and revenue metrics for AGs.
- **Operational Data:** Store channel/region information (from Cache), configuration for segmentation thresholds (e.g., TOP, MODERATE, SLOW cutoffs).

### 3. Business Settings Not specified for this module.

### 4. Process Flow

```
graph TD
 A[AG Data w/ Initial Segments] --> B{Segment AGs: TOP, MODERATE, SLOW}
 B --> C{New or Exited AGs?}
 C -- Yes --> D[Dynamically Introduce AGs from Other Stores/ Channels]
 D --> E[Optimized & Segmented AG Assortment]
```

### 5. Step-by-Step Actions

- **Utilize** the AG data, including revPerLiveDay and revPerLiveDaysPerStyle metrics, from the previous module. *These refined metrics are the basis for accurate performance-based segmentation.*
- **Perform** deep segmentation of Attribute Groups (AGs) into performance categories such as TOP, MODERATE, and SLOW, based on their calculated ROS and other revenue metrics. *This granular segmentation drives targeted OD recommendations.*
- **Dynamically introduce** new AGs that are performing well in other stores within the same channel/region, but are not currently stocked in the target store. *This helps expand the assortment strategically into proven areas.*

- **Re-introduce** high-performing, previously exited (tagged “F”) AGs if their performance warrants re-consideration, especially if they show strong recent sales in other relevant contexts. *This captures lost sales opportunities by bringing back successful items.*
  - **Update** the odData structures to reflect the final segmented AGs and any newly introduced or re-introduced AGs. *This comprehensive AG list and their segments will inform the final OD output.*
6. **Sample Scenario A** “Women’s Denim Jeans” AG currently stocked in Store A is segmented as “MODERATE.” However, in a sister store (Store B) within the same region, the exact same AG is performing as “TOP.” The OD Segmentation and Introduction module *identifies this discrepancy and recommends upgrading the OD for “Women’s Denim Jeans” in Store A or potentially introducing new styles from that AG that are performing well in Store B.* Additionally, an AG previously tagged “F” (Exited) due to no sales last year, “Summer Sandals,” shows a strong ROS in other stores ahead of the new summer season; the module *re-introduces* it into consideration. **Key Takeaway:** *Dynamic segmentation and strategic introduction of AGs ensure that the product assortment is continually optimized, capturing new opportunities and leveraging cross-store performance insights for improved OD.*

## 7. Results

- AGs fully segmented into TOP, MODERATE, SLOW, F, UA categories.
- Newly introduced or re-introduced AGs added to the consideration set for OD.
- Updated odData with final AG segmentation.

## 8. Unusual Cases

- **Channel/Region performance discrepancies:** If similar AGs perform vastly differently across channels, the introduction logic might need fine-tuning to avoid introducing AGs that aren’t a good fit for specific stores.
- **Insufficient data for new AGs:** If a potential “new” AG has very limited historical data, its performance might be misjudged, leading to sub-optimal introduction decisions.
- **Threshold sensitivity:** The cut-off points for TOP, MODERATE, SLOW segments are critical; small changes can significantly alter the resulting OD recommendations.

9. **Next Steps** The segmented AGs are passed to the OD Average Selling Price (ASP) Calculation module, providing the context for pricing analysis before final output generation.

## 10. Common Questions

1. How are AGs classified as TOP, MODERATE, or SLOW? *Based on configurable thresholds for Rate of Sale (ROS) and revenue performance per live day.*
2. What triggers the introduction of a new AG? *Strong performance (high ROS/revenue) in other stores within the same channel/region where the AG is currently not stocked.*
3. Can an “F” (Exited) AG be re-introduced? *Yes, if it shows significant recent performance in other relevant contexts, it can be re-evaluated and re-introduced.*

4. How does this module relate to Optimum Width (OW)? *By identifying which AGs are performing strongly and should be introduced, it directly informs the breadth of categories/styles to offer (OW).*
5. What if an AG is consistently “SLOW” but strategically important? *Manual overrides applied in the final output stage can ensure such AGs are still stocked at desired levels.*

## 11. Quick Reference

- **Purpose:** Deep Segmentation, Dynamic Assortment Introduction
  - **Inputs:** AG data with initial segments, sales/revenue metrics
  - **Outputs:** Segmented AGs (TOP, MODERATE, SLOW), introduced/re-introduced AGs
  - **Key Action:** `segmentAGs()`, `introduceNewAGs()`
- 

## OD Average Selling Price (ASP) Calculation

### 1. Module Name & Business Objective OD Average Selling Price (ASP)

**Calculation** calculates the Average Selling Price (ASP) for each Attribute Group (AG) relevant to Optimum Depth (OD) recommendations. Its business objective is to *provide accurate pricing context for assortment planning*, ensuring that OD decisions consider the revenue potential and price point of each product group.

### 2. Required Information

- **Sales Data:** Relevant sales map containing units and revenue for AGs (from OD Data Preparation).
- **Operational Data:** ASP overrides (from OD Input and Validation), store/region/channel hierarchy.
- **Product Data:** AG definitions.

### 3. Business Settings Not specified for this module.

### 4. Process Flow

```
graph TD
 A[Relevant Sales Data] --> B[Calculate AG ASP at Store Level]
 B --> C[Aggregate ASP to Region/Channel]
 C --> D{Apply Manual ASP Overrides?}
 D -- Yes --> E[Adjusted ASP]
 D -- No --> F[Final AG ASPs]
```

### 5. Step-by-Step Actions

- **Access** the `relSalesMap` from the data preparation stage, which contains cleaned and filtered sales data. *This ensures ASP calculations are based on valid, representative sales.*
- **Compute** the Average Selling Price (ASP) for each Attribute Group (AG) at various granularity levels: store, region, and channel. *Calculating ASP at multiple levels provides comprehensive pricing insights for different business contexts.*



- **Apply** any manual ASP overrides provided in the input data. *These overrides account for strategic pricing decisions or specific business adjustments that supersede calculated ASPs.*
- **Store** `OdAspRow` to the database, containing the computed and adjusted ASPs for each relevant AG. *This makes the ASP data available for downstream reporting and final output generation.*
- **Ensure** that ASPs are robustly calculated even with limited sales data, using fallback mechanisms if necessary. *Reliable ASPs are crucial for understanding potential revenue from OD recommendations.*

6. **Sample Scenario** The module computes ASP for the “Women’s Casual Dress” AG. In Store A, the calculated ASP is \$45 based on recent sales. However, the business has provided a manual ASP override for this AG in Store A, setting it to \$50, anticipating a price increase or strategic pricing. The module *applies this override, ensuring the final ASP used for this AG in Store A is \$50.* For a regional ASP calculation, it aggregates all store-level ASPs for “Women’s Casual Dress” in that region. **Key Takeaway:** *Accurate and contextually relevant ASPs, incorporating both calculated figures and strategic overrides, are essential for evaluating the financial impact of Optimum Depth recommendations and making informed assortment decisions.*

## 7. Results

- Computed ASPs for AGs at store, region, and channel levels.
- Application of manual ASP overrides.
- `OdAspRow` records stored in the database.

## 8. Unusual Cases

- **New products with no sales:** AGs with zero sales data will have undefined ASPs, requiring a default or fallback ASP mechanism.
- **Extreme promotional periods:** Deep discounts during promotional events can heavily skew ASPs if not adequately filtered or adjusted prior to this module.
- **Inconsistent manual overrides:** Conflicting or incorrectly applied manual overrides can lead to inaccurate ASPs, requiring careful data governance.

9. **Next Steps** The computed ASPs are passed to the OD Output and Override Application module, where they will be consolidated with all other data and overrides to generate the final OD reports.

## 10. Common Questions

1. Why is the ASP different from the average price I see? *ASP is calculated based on specific sales periods and may include/exclude certain types of sales (e.g., liquidation), and manual overrides might be applied.*
2. How do ASP overrides work? *Manual values provided in input files (`od_asp_override.csv`) take precedence over the calculated ASP for specific AGs/stores/channels.*
3. At what levels is ASP calculated? *Store, region, and channel levels, to provide comprehensive pricing context.*
4. How does ASP influence OD recommendations? *While not directly changing the number\* of items, ASP provides critical context for the value of the recommended assortment, influencing profitability assessments.\**

5. What happens if an AG has no sales to calculate ASP? *A default ASP might be assigned, or it could be flagged for manual review, depending on configuration.*

## 11. Quick Reference

- **Purpose:** Average Selling Price Calculation
  - **Inputs:** relSalesMap, ASP overrides
  - **Outputs:** Computed AG ASPs, OdAspRow
  - **Key Action:** calculateASP(), applyASP00overrides()
- 

# OD Output and Override Application

## 1. Module Name & Business Objective OD Output and Override Application

is the final stage of the Optimum Depth (OD) pipeline, applying all computed and manually provided overrides (category, store, and channel levels) to the segmented data. Its business objective is to *consolidate all processed data and strategic adjustments to generate comprehensive, denormalized final reports and summary tables*, reflecting the ideal product depth recommendations for business decision-making.

## 2. Required Information

- **Product Data:** Segmented AG data (from OD Segmentation and Introduction).
- **Sales Data:** Processed sales data (from OD Data Preparation).
- **Operational Data:** Manual input overrides (from OD Input and Validation), computed category overrides (from OD Category Override Computation), computed ASPs (from OD Average Selling Price (ASP) Calculation).

## 3. Business Settings Not specified for this module.

## 4. Process Flow

```
graph TD
 A[Segmented AG Data] --> B(Retrieve & Apply All Overrides)
 B --> C(Consolidate All Data Points)
 C --> D(Denormalize for Reporting)
 D --> E[Generate Final OD Reports & Summaries]
```

## 5. Step-by-Step Actions

- **Consolidate** all processed data, including the segmented AGs, cleaned sales data, and computed ASPs. *This brings together all analytical insights for final recommendation generation.*
- **Retrieve and apply** all manual input overrides (store, channel, and category levels) from `odData.getOverridesForStorePeriod` and `odData.getOverridesForChannelPeriod`. *These overrides directly reflect strategic business decisions for specific depth targets.*
- **Apply** computed category overrides from `odData.getComputedOverride` to adjust OD recommendations based on algorithmic insights at the

category level. *This integrates data-driven category performance into the final output.*

- **Denormalize** various data points to create comprehensive final reports and summary tables. *Denormalization simplifies reporting and ensures all relevant information is readily available for analysis.*
- **Generate** output files (e.g., CSV, JSON) containing the final Optimum Depth recommendations, including the rationale, applied overrides, and associated metrics. *These reports are the primary deliverable for business users.*

**6. Sample Scenario** For a “Women’s Active Leggings” AG in Store A, the OD Segmentation and Introduction module recommended an OD of 5 (MODERATE segment). However, a manual store-level override was provided, stipulating that all “Women’s Activewear” in Store A *must* have an OD of at least 8. The OD Output and Override Application module *applies this manual override*, adjusting the final OD for “Women’s Active Leggings” in Store A to 8, ensuring business strategy is prioritized in the output. Simultaneously, it includes the AG’s computed ASP and ROS in the final report.  
**Key Takeaway:** *This module ensures that all analytical insights are combined with strategic business directives, producing actionable and accurate Optimum Depth recommendations that align with both data and operational goals.*

## 7. Results

- Comprehensive final reports and summary tables detailing Optimum Depth recommendations per AG/store/category.
- Clear indication of applied manual and computed overrides.
- Denormalized data set ready for business intelligence tools.

## 8. Unusual Cases

- **Conflicting overrides:** If multiple manual overrides conflict (e.g., category says 5, store says 3), a defined precedence rule must be in place (e.g., store-level overrides channel-level, manual overrides computed).
- **Incomplete override application:** Errors in retrieving or applying overrides can lead to incorrect final OD recommendations, requiring thorough validation.
- **Large output volume:** For very large datasets, generating and storing denormalized outputs can be resource-intensive, requiring efficient writing mechanisms.

**9. Next Steps** The generated output reports are then consumed by business users, merchandising teams, and other planning systems to implement the Optimum Depth recommendations.

## 10. Common Questions

1. Where can I find the final OD recommendations? *In the outputs/ folder, typically as CSV or JSON files generated by this module.*
2. How are manual overrides prioritized? *Usually, the most granular override (e.g., store-level) takes precedence over broader ones (category, channel), but this is defined in the system’s logic.*
3. Can I see why a specific OD was recommended? *The denormalized output should include metrics like ROS, segment, and whether an override was applied, providing the rationale.*

4. What's the difference between computed and manual overrides? *Computed overrides are algorithmic suggestions, while manual overrides are direct business directives that supersede computations.*
5. How often are these reports updated? *This depends on the frequency of the OD pipeline run, typically weekly or monthly, aligning with planning cycles.*

## 11. Quick Reference

- **Purpose:** Final Output Generation, Override Application
  - **Inputs:** All processed data and overrides
  - **Outputs:** Final OD reports, summary tables
  - **Key Action:** `applyOverrides()`, `denormalize()`, `generateReports()`
- 

# OD Calculation Pipeline Orchestrator

1. **Module Name & Business Objective** **OD Calculation Pipeline Orchestrator** serves as the master orchestrator for the entire Optimum Depth (OD) calculation project. Its business objective is to *ensure the sequential and efficient execution of all sub-modules*, from initial data input and validation through various processing stages to final output generation, ensuring a complete and accurate OD recommendation process.
2. **Required Information** Not specified for this module, as it orchestrates other modules rather than processing raw data directly. Its primary requirement is a properly configured `args.properties` file.
3. **Business Settings** Not specified for this module.
4. **Process Flow**

```
graph TD
 A[Start Pipeline] --> B(OD Input and Validation)
 B --> C(OD Data Preparation)
 C --> D(OD Long Tail Computation)
 D --> E(OD Category Override Computation)
 E --> F(OD Segmentation and Introduction)
 F --> G(OD Average Selling Price (ASP) Calculation)
 G --> H(OD Output and Override Application)
 H --> I[End Pipeline]
```

## 5. Step-by-Step Actions

- **Initialize** the OD calculation process, preparing the environment and loading global configurations. *This sets up the stage for the entire pipeline run.*
- **Call** OD Input and Validation to load and validate all raw data. *Ensures data quality from the very beginning.*
- **Execute** OD Data Preparation to clean and filter the raw data for analysis. *Refines the dataset for accurate performance metrics.*
- **Invoke** OD Long Tail Computation to identify and remove low-performing styles. *Focuses the analysis on core, profitable items.*

- **Trigger** OD Category Override Computation to establish initial category insights and AG segmentation. *Provides foundational context for deeper analysis.*
- **Run** OD Segmentation and Introduction to segment AGs and dynamically introduce new ones. *Optimizes the assortment breadth and depth.*
- **Launch** OD Average Selling Price (ASP) Calculation to determine accurate product pricing context. *Adds financial context to OD recommendations.*
- **Initiate** OD Output and Override Application to apply all final rules and generate reports. *Produces the actionable business deliverables.*
- **Manage** overall pipeline flow, error handling, and resource synchronization. *Ensures the entire process completes successfully and efficiently.*

6. **Sample Scenario** A new monthly OD run is scheduled. The OD Calculation Pipeline Orchestrator *starts the process*, first calling OD Input and Validation. If data validation fails, the Orchestrator *stops the entire pipeline* and logs an error, notifying operations. If validation succeeds, it sequentially triggers the next modules, passing control and data between them. After OD Output and Override Application completes, the Orchestrator *confirms successful report generation* and marks the pipeline run as finished. **Key Takeaway:** *The Orchestrator ensures that the complex OD calculation pipeline runs smoothly, in the correct order, and handles errors effectively, guaranteeing reliable and timely generation of Optimum Depth recommendations.*

## 7. Results

- Successful execution of the entire OD calculation pipeline.
- Timely generation of final OD reports in the outputs/ folder.
- Consolidated logs of pipeline execution status, including any errors or warnings.

## 8. Unusual Cases

- **Missing or delayed data feed:** If any upstream module fails due to data issues, the orchestrator will halt subsequent execution and report the failure.
- **Module failure:** If an individual submodule encounters an unhandled exception, the orchestrator catches it, logs the error, and can be configured to stop or retry.
- **Resource contention:** In high-load environments, the orchestrator might need to manage resource allocation (e.g., database connections, memory) to prevent bottlenecks.

9. **Next Steps** The completed OD reports are ready for consumption by business users for inventory planning, merchandising decisions, and assortment optimization.

## 10. Common Questions

1. How do I start an OD calculation run? *By executing the main orchestrator module, typically via a scheduled job or manual command line prompt.*
2. What happens if one of the modules fails? *The orchestrator will log the error and stop the pipeline, preventing incomplete or incorrect outputs.*

3. Can I run only a subset of the modules? *No, the orchestrator enforces the full pipeline sequence to ensure data integrity and complete recommendations.*
4. How can I monitor the progress of a run? *Check the system logs for real-time updates from the orchestrator and individual modules.*
5. What if the pipeline takes too long to run? *Review the performance of individual modules, particularly data-intensive ones, and optimize data loading/processing.*

## 11. Quick Reference

- **Purpose:** Pipeline Execution Management
  - **Inputs:** Global configuration (`args.properties`)
  - **Outputs:** Status of pipeline execution, coordination of module outputs
  - **Key Action:** `executePipeline()`
- 

# 8. Appendix

## Full Parameter Reference

Refer to the `args.properties` file in the root directory for a complete list of configurable parameters, their default values, and detailed descriptions.

## Version History

- **Version 1.0 (2025-05-23):** Initial Release of the Optimum Depth (OD) Calculation Pipeline.

## Contact & Support Info

For any questions, issues, or feature requests regarding the Optimum Depth (OD) documentation or the algorithm, please contact:

- **Support Email:** [support@yourcompany.com](mailto:support@yourcompany.com)
- **Internal Jira Board:** [Link to your Jira board] (e.g., JIRA\_OD\_CALC)
- **Documentation Feedback:** [documentation@yourcompany.com](mailto:documentation@yourcompany.com)

# Tutorial: od

The Optimum Depth (OD) calculation process follows a sequential pipeline, beginning with raw data input and validation, then proceeding through iterative data preparation, long-tail filtering, and metric computations. This refined data is used for segmentation and Average Selling Price (ASP) calculations, before all computed and manual overrides are applied to finalize the product depth recommendations and generate comprehensive reports.

flowchart TD

```
A0["OD Input and Validation"]
A1["OD Data Preparation"]
A2["OD Long Tail Computation"]
A3["OD Category Override Computation"]
A4["OD Segmentation and Introduction"]
A5["OD Average Selling Price (ASP) Calculation"]
A6["OD Output and Override Application"]
A7["OD Calculation Pipeline Orchestrator"]

A7 -- "orchestrates" --> A0
A7 -- "orchestrates" --> A1
A7 -- "orchestrates" --> A2
A7 -- "orchestrates" --> A3
A7 -- "orchestrates" --> A4
A7 -- "orchestrates" --> A5
A7 -- "orchestrates" --> A6
A0 -- "prepares_raw" --> A1
A1 -- "filters_sales" --> A2
A2 -- "initializes_ag" --> A3
A3 -- "segments_ag" --> A4
A4 -- "computes_asp" --> A5
A5 -- "generates_output" --> A6
A0 -- "provides_overrides" --> A6
A3 -- "applies_computed_overrides" --> A6
```

## Chapters

# Master Report: Optimum Depth (OD) Merchandising Algorithm

## 1. Title & Metadata

- **Project Name:** od
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** The OD algorithm determines the ideal number of unique items (SKUs) to stock within a product category or display, optimizing inventory efficiency and maximizing sales.

## 2. Executive Summary

The Optimum Depth (OD) merchandising algorithm is a critical tool designed to enhance inventory productivity and sales performance across product categories. By analyzing historical performance, demand forecasts, and product attributes, it identifies the optimal number of distinct SKUs that should be stocked to meet customer demand efficiently without incurring excessive inventory costs or risking stockouts.

The algorithm's core function is to provide data-driven recommendations for SKU assortment, ensuring that merchandising efforts are focused on the most productive items within each category and location.

- **Key outcomes or KPIs:** Increased inventory turnover, reduced stockouts, improved sales per square foot, optimized working capital, enhanced customer satisfaction through better product availability, and accurate OD recommendation accuracy.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Planning, Supply Chain Management, Retail Operations, Store Managers.

## 3. Architecture & Data Flow

### Diagram:

graph TD

```
A[Raw Sales Data] --> B(Data Ingestion & Validation)
C[Inventory Data] --> B
D[Product Master Data] --> B
E[Store & Location Data] --> B
```



```

B --> F{Feature Generation}
F --> G{Demand & Trend Analysis}
G --> H[OD Optimization Engine]
H --> I[Reporting & Integration]
I --> J[Merchandising Decisions]
I --> K[Inventory Planning Systems]

```

### Component Bullets:

- **Data Ingestion & Validation:** Responsible for collecting raw data from various sources, performing initial data cleaning, and validating data integrity.
- **Feature Generation:** Transforms raw and validated data into actionable features such as Rate of Sale (ROS), inventory turns, and Attribute Group (AG) performance metrics.
- **Demand & Trend Analysis:** Leverages historical performance and market trends to forecast future SKU and category demand, informing the core optimization.
- **OD Optimization Engine:** The central computational module that applies business rules, statistical models, and optimization logic to determine the Optimum Depth (OD) for each category.
- **Reporting & Integration:** Formats the final OD recommendations into user-friendly reports and integrates them with downstream systems for execution.

### High-level data sources:

- **Point of Sale (POS) Data:** Historical sales transactions, units sold, revenue.
- **Inventory Management Systems:** Current stock levels, inventory movements, stockouts.
- **Product Master Data:** SKU attributes (e.g., color, size, material, brand), category hierarchies, cost.
- **Store/Location Data:** Store attributes, sales area, capacity.

## 4. Configuration & Data Dictionary

### Settings Table (args.properties):

Parameter Name	Description	Default Value	Example Value
min_historical_sales_weeks	Minimum weeks of sales history required for SKU analysis.	26	52
ros_calculation_period_weeks	Period in weeks for calculating Rate of Sale (ROS).	12	8

Parameter Name	Description	Default Value	Example Value
target_category_sales_contribution_percentage	Percentage of total category sales that the selected SKUs should collectively achieve.	0.90	0.85
minimum_sku_inclusions	Minimum number of SKUs to always include in a category's OD, regardless of performance.	3	2
attribute_group_weighting	Weight assigned to AG performance in SKU ranking.	0.2	0.3
category_consolidation_threshold	Similarity threshold for consolidating performance across very similar categories.	0.85	0.90

**Shared Data Fields (common-data.json):**

Field Name	Data Type	Description	Example Value
sku_id	String	Unique identifier for a Stock Keeping Unit.	“PROD12345-BLU-M”
category_id	String	Identifier for the product category.	“WOMENS_DENIM”
store_id	String	Unique identifier for a store or location.	“LOC001”
sale_date	Date	Date of the sales transaction.	“2024-05-20”
units_sold	Integer		5

Field Name	Data Type	Description	Example Value
		Number of units sold in a transaction.	
current_stock_units	Integer	Current inventory level for a SKU at a location.	120
ros	Decimal	Rate of Sale (units/week).	7.5
attribute_group_id	String	Identifier for an Attribute Group (AG).	"COLOR_BLUE"
product_brand	String	Brand name of the product.	"Acme Apparel"
product_description	String	Short description of the product.	"Slim Fit Jeans"

### Outputs Catalog:

Output Name	Description	Format
optimum_depth_recommendation_per_category_store	The calculated ideal number of unique SKUs for each category per store.	CSV/ JSON
recommended_sku_list_per_category	A detailed list of recommended SKUs to be stocked for each category, ranked by projected performance.	CSV/ JSON
projected_sales_impact_metrics	Forecasted sales uplift, inventory reduction, and stockout reduction resulting from implementing OD recommendations.	CSV/ JSON
category_performance_summary	Aggregated metrics (total ROS, sales contribution) for each category.	CSV/ JSON
algorithm_execution_log	Detailed log of the algorithm's run, including	Text/ Log File

Output Name	Description	Format
	errors and warnings.	

## 5. End-to-End Flow

The OD algorithm operates in a sequential, automated flow designed to deliver timely and accurate merchandising recommendations:

1. **Data Acquisition:** Raw sales, inventory, and product master data are collected from various enterprise systems (e.g., POS, WMS, PIM) on a scheduled basis.
2. **Data Pre-processing:** Ingested data undergoes cleaning, validation, and harmonization to ensure consistency and accuracy. Missing values are imputed, and outliers are managed.
3. **Feature Engineering:** Critical performance metrics are computed, including Rate of Sale (ROS) for individual SKUs, aggregated sales performance per category and Attribute Group (AG), and inventory turnover rates.
4. **Demand Forecasting:** Predictive models are applied to historical sales data to forecast future ROS and category trends, providing a forward-looking perspective.
5. **SKU Ranking & Optimization:** SKUs within each category are ranked based on their forecasted performance and historical contribution. The OD Optimization Engine then applies configurable business rules (e.g., `target_category_sales_contribution_percentage`, `minimum_sku_inclusions`) to determine the ideal number of SKUs (OD) that maximize sales while optimizing inventory.
6. **Recommendation Generation:** The algorithm generates the final OD recommendations, including the precise list of SKUs to be stocked for each category and location.
7. **Reporting & Dissemination:** Outputs are formatted into structured reports and API payloads, which are then delivered to relevant stakeholders and integrated into downstream planning and merchandising systems.

## 6. Submodule Overview

### 1. Data Ingestion & Harmonization

- **Purpose:** Collects raw sales, inventory, and product data from various source systems and transforms it into a standardized, clean format for subsequent processing.
- **Inputs:**
  - Raw sales transaction logs
  - Daily inventory feeds
  - Product master data dumps
  - Store location master data
- **Outputs:**
  - `clean_sales_transactions` (structured table of sales records)
  - `current_inventory_snapshot` (structured table of current stock levels)
  - `harmonized_product_master` (structured table of product attributes)

- **Core Logic Summary:**
  - Reads data from specified data sources (e.g., databases, flat files, APIs).
  - Applies a predefined schema to enforce data types and structure.
  - Performs initial validation checks for data completeness and consistency (e.g., non-negative units, valid dates).
  - Standardizes categorical values and resolves duplicate entries.

## 2. Feature Generation

- **Purpose:** Computes derived metrics and attributes essential for the OD calculation, transforming raw data into meaningful business intelligence.
- **Inputs:**
  - clean\_sales\_transactions
  - current\_inventory\_snapshot
  - harmonized\_product\_master
- **Outputs:**
  - sku\_performance\_metrics (includes ROS, sales per SKU, inventory turnover)
  - category\_performance\_metrics (aggregated sales, ROS for entire categories)
  - attribute\_group\_performance (aggregated performance for specific AGs)
- **Core Logic Summary:**
  - Calculates Rate of Sale (ROS) for each unique SKU by dividing total units sold by the ros\_calculation\_period\_weeks within a specified lookback period.
  - Aggregates sales and inventory data to category and Attribute Group (AG) levels.
  - Computes sales contribution percentages for each SKU within its respective category.
  - Identifies and flags new SKUs or those with insufficient sales history.

## 3. Demand & Trend Analysis

- **Purpose:** Forecasts future SKU and category performance, incorporating seasonality and market trends to ensure OD recommendations are forward-looking.
- **Inputs:**
  - sku\_performance\_metrics (historical ROS and sales)
  - category\_performance\_metrics (historical category sales and trends)
  - configuration\_settings (for forecast horizon)
- **Outputs:**
  - forecasted\_sku\_ros (projected ROS for upcoming periods)
  - category\_growth\_projections (expected growth or decline per category)
- **Core Logic Summary:**
  - Applies time-series forecasting models (e.g., ARIMA, Prophet, Exponential Smoothing) to predict future ROS for individual SKUs.
  - Analyzes historical category growth rates and identifies seasonal patterns or trend shifts.
  - Adjusts forecasts based on external factors or promotions if relevant data is available.
  - Provides confidence intervals around forecasts to quantify uncertainty.

## 4. OD Optimization Engine

- **Purpose:** The core module that determines the Optimum Depth (OD) for each category and location by applying business rules and optimizing SKU selection.
- **Inputs:**
  - forecasted\_sku\_ros
  - category\_growth\_projections
  - configuration\_settings (e.g., target\_category\_sales\_contribution\_percentage, minimum\_sku\_inclusions)
  - harmonized\_product\_master (for AGs and product attributes)
- **Outputs:**
  - optimum\_depth\_recommendations (ideal number of SKUs per category/store)
  - ranked\_sku\_list\_per\_category (ordered list of SKUs proposed for inclusion)
- **Core Logic Summary:**
  - Ranks all available SKUs within each category based on their forecasted\_sku\_ros and/or contribution to category sales.
  - Iteratively selects top-ranked SKUs until the target\_category\_sales\_contribution\_percentage is met for the category's total forecasted sales.
  - Ensures that minimum\_sku\_inclusions are met, and applies rules for specific Attribute Groups (AGs) or must-stock items.
  - Calculates the resulting Optimum Depth (OD) as the count of selected SKUs for that category.

## 5. Reporting & Integration

- **Purpose:** Formats the final OD recommendations into clear, actionable reports and facilitates their integration with downstream merchandising and planning systems.
- **Inputs:**
  - optimum\_depth\_recommendations
  - ranked\_sku\_list\_per\_category
  - projected\_sales\_impact\_metrics
- **Outputs:**
  - Final\_OD\_Report.csv (detailed report for analysts)
  - OD\_Recommendations\_API\_Payload.json (for automated system integration)
  - Execution\_Summary\_Log.txt
- **Core Logic Summary:**
  - Aggregates the final OD results and selected SKU lists into structured tables.
  - Generates a summary of the projected business impact (e.g., estimated sales uplift, inventory reduction).
  - Creates various output formats (CSV, JSON, PDF) tailored for different stakeholder needs.
  - Logs the entire process, including start/end times, parameters used, and any warnings or errors.

## 7. Worked Example

**Scenario:** A small apparel boutique wants to determine the Optimum Depth (OD) for its “Women’s Casual Tops” category to maximize sales while keeping inventory lean. The business rule is to cover 85% of the category’s sales contribution.

**Inputs (Simplified):**

SKU ID	Product Description	Category	Weekly ROS	Cumulative Sales Contribution %
T001	Striped Cotton Tee	Women’s Casual Tops	15	25.0%
T002	Solid V-Neck Top	Women’s Casual Tops	12	45.0% (25+20)
T003	Henley Top	Women’s Casual Tops	9	60.0% (45+15)
T004	Floral Blouse	Women’s Casual Tops	6	70.0% (60+10)
T005	Long-Sleeve Thermal	Women’s Casual Tops	5	78.3% (70+8.3)
T006	Ribbed Tank Top	Women’s Casual Tops	4	85.0% (78.3+6.7)
T007	Graphic Tee	Women’s Casual Tops	3	90.0% (85+5)
T008	Crew Neck Sweater	Women’s Casual Tops	2	93.3% (90+3.3)
T009	Asymmetrical Tunic	Women’s Casual Tops	1.5	95.8% (93.3+2.5)
T010	Tapered Vest	Women’s Casual Tops	1	97.5% (95.8+1.7)

*Note: SKUs are already sorted by ROS for simplicity. Cumulative sales contribution is calculated based on ROS percentages of the total category ROS.*

**Core Logic Application:** 1. **Rank SKUs:** The SKUs are ranked by their weekly ROS (already done in the table). 2. **Calculate Cumulative Sales Contribution:** The cumulative sales contribution percentage is computed. 3. **Apply Target Threshold:** The algorithm iterates down the ranked list until the `target_category_sales_contribution_percentage` (85% in this case) is met or exceeded.

**Output:** \* SKU T001 (25.0%) \* SKU T002 (45.0%) \* SKU T003 (60.0%) \* SKU T004 (70.0%) \* SKU T005 (78.3%) \* SKU T006 (85.0%) - At this point, the 85% target is met.

Therefore, the **Optimum Depth (OD)** for “Women’s Casual Tops” is 6 SKUs. The recommended SKUs to stock are T001, T002, T003, T004, T005, and T006. While T007 would bring the contribution to 90%, it pushes beyond the 85% target for this specific scenario.

## 8. Debug Guide Highlights

- **Unexpected OD Recommendations (Too High/Low):**
  - **Check args.properties:** Verify `target_category_sales_contribution_percentage`, `minimum_sku_inclusions`, and `ros_calculation_period_weeks`. Incorrect settings can drastically skew results.
  - **Review sku\_performance\_metrics:** Look for data quality issues in ROS calculations (e.g., zero ROS for active items, extremely high ROS due to data errors).
  - **Inspect category\_performance\_metrics:** Ensure categories are defined correctly and sales are correctly aggregated. A miscategorized SKU can distort a category's total sales.
- **Data Quality Issues / Missing Data:**
  - **Trace Data Flow:** Start from the Data Ingestion & Validation module. Check raw input files/databases for missing sales records, inventory inaccuracies, or incomplete product attributes.
  - **Validate Joins:** Ensure that SKUs, categories, and stores are correctly joined across different data sources. Orphaned records will not be processed.
  - **Error Logs:** Review `algorithm_execution_log` for specific data validation errors or warnings during ingestion and feature generation.
- **Performance Bottlenecks:**
  - **Large Datasets:** If processing time is slow, analyze the volume of data being processed at each stage, especially Feature Generation and OD Optimization Engine.
  - **Database Queries:** Optimize database queries for efficiency.
  - **Resource Allocation:** Check server resources (CPU, RAM) and consider scaling up if necessary.

## 9. Common Pitfalls & Remedies

- **Pitfall 1: Stale or Inaccurate Input Data:**
  - **Description:** Recommendations are based on old or erroneous sales/inventory data, leading to irrelevant OD suggestions.
  - **Remedy:** Implement robust data governance, regular data refresh cycles (daily/weekly), and automated data validation checks with alerts for anomalies.
- **Pitfall 2: Over-Reliance on Historical Data Without Considering Trends:**
  - **Description:** Ignoring seasonality, promotions, or market shifts can lead to incorrect forecasts and suboptimal OD.
  - **Remedy:** Ensure the Demand & Trend Analysis module incorporates relevant forecasting models and considers external market factors. Integrate promotional calendars and new product launch plans.
- **Pitfall 3: Suboptimal Category Definitions:**
  - **Description:** Product categories are too broad or too narrow, or SKUs are miscategorized, making OD calculations less meaningful.
  - **Remedy:** Collaborate closely with merchandising teams to establish clear, logical category hierarchies. Regularly review and refine category definitions as the product assortment evolves. Consider running the algorithm at different levels of granularity (e.g., sub-category, AG).



- **Pitfall 4: Ignoring Qualitative Factors or Business Constraints:**
  - **Description:** Solely relying on quantitative data can exclude “must-stock” items, strategic brands, or visual merchandising requirements.
  - **Remedy:** Integrate business rules (e.g., `minimum_sku_inclusions`) into the OD Optimization Engine. Allow for manual overrides or a “guided” recommendation process where merchandising teams can provide input.

## 10. FAQ

- **Q1: How often is the OD algorithm run?**
  - **A1:** The algorithm is typically scheduled to run weekly or bi-weekly to capture recent sales trends and inventory changes. Critical events like major product launches or seasonal shifts might trigger ad-hoc runs.
- **Q2: Can the OD algorithm handle new product introductions?**
  - **A2:** Yes. New products initially rely on predefined ramp-up curves or attribute-based forecasts (e.g., based on similar items/AGs) until sufficient sales history is accumulated. As they generate sales, their actual ROS will be factored into subsequent OD calculations.
- **Q3: Is Optimum Depth (OD) a fixed number, or does it change?**
  - **A3:** OD is dynamic. It adapts to changes in customer demand, product performance (ROS), inventory levels, and strategic business goals, ensuring that the assortment remains optimized.
- **Q4: What if a category has very low sales or is highly seasonal?**
  - **A4:** For low-volume categories, the `minimum_sku_inclusions` setting ensures a base level of assortment. For highly seasonal categories, the Demand & Trend Analysis module uses seasonal forecasting models to provide appropriate OD recommendations during peak and off-peak periods.
- **Q5: Can we override the OD recommendations?**
  - **A5:** Yes, the system provides an interface for merchandising teams to review and apply manual adjustments to the recommendations before final implementation. These overrides can also be used as feedback to refine the algorithm’s parameters over time.

## 11. Quick Reference Cheat-Sheet

- **OD (Optimum Depth):** The ideal number of unique items (SKUs) to stock in a category or display.
- **ROS (Rate of Sale):** The speed at which products are sold (e.g., units/week). Key metric for SKU ranking.
- **AG (Attribute Group):** A set of product attributes grouped for analysis (e.g., color, size, brand). Used to ensure balanced representation within OD.
- **Purpose:** Maximize sales, optimize inventory, reduce stockouts.
- **Key Inputs:** Sales history, inventory data, product attributes, `args.properties` configurations.
- **Core Logic:** Rank SKUs by ROS, apply sales contribution target, determine optimal cutoff.
- **Primary Outputs:** Recommended OD per category, specific SKU lists for inclusion.

- **Configuration Control:**

target\_category\_sales\_contribution\_percentage,  
minimum\_sku\_inclusions, ros\_calculation\_period\_weeks.

# Chapter 1: OTB Style Wise Buy Process Orchestrator

Last Updated: 2025-05-23

## 1. Business Objective

This step orchestrates the entire process of calculating the ideal inventory plan for products, ensuring stores stock the right items in the right quantities to maximize sales and minimize waste.

**Why This Matters:** This helps retailers make smart buying decisions, minimizing overstocking (which ties up cash) and understocking (which leads to lost sales and disappointed customers). It directly improves profitability and customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Historical Units Sold:** How many of each product were sold in the past. *Why we need it: To understand past performance and project future demand.*
- **Current Sales Trends:** Recent changes in how products are selling, such as a recent surge or decline in demand. *Why we need it: To adapt quickly to shifts in customer preferences and market demand.*

### Product Data

- **Product Categories/Styles:** How products are grouped (e.g., “Men’s Jeans,” “Women’s Dresses,” “Home Decor”). *Why we need it: To analyze performance within specific merchandising groups and ensure a balanced assortment.*
- **Product Attributes (e.g., Color, Size, Material):** Specific details about each unique product variation. *Why we need it: To identify specific product variations (SKUs) that perform exceptionally well or poorly.*

### Operational Data

- **Current Inventory Levels:** How much stock is available right now in stores and warehouses for each product. *Why we need it: To determine what needs to be ordered versus what’s already on hand, avoiding over-ordering.*

- **Store Locations:** The specific stores where each product is sold and their unique sales performance. *Why we need it:* To tailor inventory plans to specific store needs, regional demand, or store clusters.

### 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>Minimum Sales Threshold</b>	5 units	The minimum number of units a product must sell within a given period to be considered for detailed analysis. Products below this may be flagged for review or discontinuation.	If focusing on high-volume items only, increase this value; if trying to analyze niche or new products, decrease it.
<b>Analysis Period</b>	90 days	The historical duration used to calculate sales rates and project future performance. For example, if 90 days, it looks at sales from the last three months.	Shorten for fast-moving fashion trends or seasonal items; lengthen for stable, evergreen products to capture more data.
<b>Sales Consistency Score Impact</b>	0.8	How strongly consistent past sales performance influences the projected Rate of Sale (ROS). A higher value means past stability is more important.	Increase if historical reliability is paramount (e.g., basic items); decrease if recent trends or new product launches should take precedence.
<b>Optimum Depth Target</b>	80%	The target percentage of desired sell-through that determines the ideal number of unique items (OD) to stock in a category. This	Adjust based on inventory turnover goals; higher for faster-moving goods (e.g., basics), lower for items with longer shelf lives (e.g., furniture).

Setting Name	Default Value	Description	When to Change
		helps manage assortment breadth.	

*Note: See More Details for additional settings.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

```

 A[Current Sales & Product Data] --> B[Projected ROS Calculation]
 B --> C[Optimum Depth (OD) Calculation]
 C --> D[Ideal Buy Plan]

```

1. Start with → **Current Sales & Product Data**, capturing a snapshot of all relevant information.
2. Then → **Projected ROS Calculation** determines how fast individual products are expected to sell in the coming period.
3. Next → **Optimum Depth (OD) Calculation** uses the projected sales rates to figure out the ideal number of different unique items (SKUs) to stock within each product category or style.
4. Finally → **Ideal Buy Plan** provides the specific recommendations for what to purchase, optimizing for sales and inventory.

## 5. Step-by-Step Actions

1. **Snapshot** the current business data
  - What we do: *We take a precise picture of all relevant sales, inventory, and product information at a specific moment in time.*
  - Why we do it: *This ensures all subsequent calculations are based on a consistent and up-to-date dataset, avoiding discrepancies caused by constantly changing data.*
2. **Calculate** the projected Rate of Sale (ROS)
  - What we do: *We analyze historical sales data over the defined Analysis Period, apply current trends, and consider factors like the Sales Consistency Score to predict how quickly each unique product (SKU) will sell.*
  - Why we do it: *Accurate ROS projections are crucial for forecasting demand and planning inventory effectively, ensuring we don't order too much (leading to excess stock) or too little (leading to lost sales).*
3. **Determine** the Optimum Depth (OD)
  - What we do: *Using the projected ROS for individual items, we calculate the ideal number of unique items (SKUs) to stock within each category or style. This aims to maximize sales by offering enough variety while minimizing unnecessary stock.*
  - Why we do it: *This optimizes the breadth of assortment, ensuring we offer enough choices to satisfy customer preferences without overwhelming them or holding excessive, slow-moving inventory.*

#### 4. Upload the calculated buy recommendations

- What we do: *The final calculated ideal buy plans, including projected ROS and OD recommendations, are made available to other systems and teams.*
- Why we do it: *This ensures that merchandising and buying teams have immediate access to the actionable, data-driven insights they need to place orders and manage inventory efficiently.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A specific “Slim Fit Dark Wash Jeans” style in the “Men’s Denim” category has sold 350 units over the last 90 days across all retail stores.
- Current global stock for this style is 80 units.
- The business has set a Minimum Sales Threshold of 5 units and an Analysis Period of 90 days.

### What Happens

1. First, the system takes a snapshot of this information. It confirms “Slim Fit Dark Wash Jeans” easily meets the Minimum Sales Threshold.
2. Then, the Projected ROS Calculation module processes the 350 units sold over 90 days. It factors in any recent sales spikes or drops and its Sales Consistency Score to project its future sales speed. It calculates an estimated ROS of roughly 3.8 units per day.
3. Next, the Optimum Depth (OD) Calculation module takes this projected ROS. Considering the desired sales coverage and stock turn targets (e.g., an 80% Optimum Depth Target), it determines that to maintain optimal availability and variety for “Slim Fit Dark Wash Jeans,” the ideal OD for this style should be 12 unique items (e.g., a specific set of 12 washes and sizes).
4. Finally, this recommendation for “Slim Fit Dark Wash Jeans” (12 OD) is uploaded, along with its projected ROS, for review by the buying team.

### Final Outcome

The buying team receives a precise recommendation that “Slim Fit Dark Wash Jeans” should aim for an Optimum Depth of 12 unique items (a certain breadth of available sizes and washes) based on its projected sales rate. This helps them decide what variety to order.

**Key Takeaway:** By linking projected sales speed to an ideal stock breadth, the system guides buyers to stock the right level of variety for key products, preventing both empty shelves and excess inventory.

## 7. Results

This step produces:

## Main Outputs

- **Projected Rate of Sale (ROS) per SKU:** The predicted daily sales speed for each unique product (SKU). *What it tells us: How fast each item is expected to sell, guiding precise inventory ordering.*
- **Optimum Depth (OD) Recommendation per Category/Style:** The ideal number of unique items (SKUs) to stock for each product group. *What it tells us: The recommended breadth of assortment to maximize sales and optimize inventory investment, ensuring sufficient choices without overstocking.*
- **Calculated Buy Plan Data:** Comprehensive data that supports recommended purchase quantities for each product. *What it tells us: A data-driven basis for making precise buying decisions.*

## Business Reports

- **Inventory Optimization Report:** Used by merchandisers and planners to review the projected ROS and OD recommendations. This report helps them make informed decisions about what to buy and how much to order.
- **Performance Analysis Dashboard:** Used by category managers to monitor overall inventory health. It highlights categories that are potentially overstocked or understocked based on these calculations, allowing for strategic adjustments.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	Calculations will use outdated or incomplete data, leading to inaccurate recommendations and potential stock issues.	Immediately check data sources and ensure all necessary sales, product, and inventory information is flowing correctly.
Products with no historical sales data	The system might default to a minimum or average ROS, or exclude them from the OD calculation, as there's no performance history.	Merchandisers should manually review new products; consider market trends or comparable item performance for initial stocking decisions.
Sudden, unexpected sales spikes (e.g., viral trend)	Projected ROS might not fully capture the rapid increase in demand, leading to under-recommendations for future buys.	Monitor market trends closely and be prepared to make manual adjustments to buy plans for highly volatile products.
Seasonal items processed off-season	The ROS might appear very low during the off-season, leading to recommendations to	Ensure seasonal flags are correctly applied, or review calculations with a longer, multi-year Analysis Period to capture full annual cycles.

Situation	What Happens	What You Should Do
	de-stock too aggressively.	

## 9. Next Steps

### Before This Step

- **Data Collection & Cleaning:** Accurate and complete sales, product, and inventory data must be gathered and validated from all relevant sources. *What we need from it: Reliable, clean data for accurate calculations, preventing “garbage in, garbage out” scenarios.*
- **Category & Attribute Definition:** Product categories and key attributes (like color, size, material, or style) must be clearly defined and consistently assigned across all products. *What we need from it: Structured product information to group items correctly for meaningful analysis and reporting.*

### After This Step

- **Open-to-Buy (OTB) Planning:** The calculated buy recommendations are fed into the broader OTB process to determine how much budget is allocated for new purchases across different product groups. *What they need from us: Data-driven insights on what to buy and in what quantities to optimize OTB allocation.*
- **Purchase Order Generation:** The specific quantities for each SKU, as determined by the ideal buy plan, are used to create actual purchase orders with suppliers. *What they need from us: The precise item-level quantities determined by the ideal buy plan to ensure accurate orders.*
- **Inventory Management & Replenishment:** The projected ROS helps guide ongoing inventory management, ensuring items are reordered as needed to maintain optimal stock levels in stores and distribution centers. *What they need from us: Ongoing performance forecasts for each item to drive timely replenishment.*

## 10. Common Questions

Q: How does this help prevent products from selling out too fast? A: By accurately projecting how quickly products will sell (ROS) and recommending the ideal number of unique items to stock (OD), it ensures we order enough to meet expected customer demand, thus minimizing stockouts.

Q: Can this system account for new product launches that have no sales history? A: While historical data is key, for brand-new products, the system may use industry benchmarks or sales patterns from similar successful products to estimate an initial ROS and OD, though some manual oversight and adjustments might still be needed.



Q: What if market trends change rapidly, making the projected sales outdated? A: The system uses a defined Analysis Period and considers a Sales Consistency Score to adapt to recent trends. For very rapid shifts (like a new viral trend), manual adjustments to the buy plan may still be necessary, and the system can be re-run frequently to capture fresh data.

Q: Does this account for different demand in different store locations? A: Yes, the calculations are designed to consider sales performance at various levels, including individual store locations, ensuring that buy plans are tailored to specific regional or store-level demand patterns where relevant sales data exists.

Q: How does this impact our overall inventory investment? A: By optimizing both the ideal number of unique items (OD) and the specific quantities based on projected sales (ROS), this process helps reduce excess inventory (saving holding costs and improving cash flow) and minimizes lost sales from stockouts (increasing revenue), directly improving overall profitability.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Orchestrates the calculation of ideal inventory plans, balancing assortment breadth (OD) with sales velocity (ROS).
- **Main Input:** A comprehensive snapshot of current sales data, detailed product information, and existing inventory levels.
- **Main Output:** Projected Rate of Sale (ROS) for individual items and Optimum Depth (OD) recommendations for product categories/styles.
- **Warning Signs:** Persistent stockouts on popular items or consistently high levels of excess inventory across categories suggest potential issues with input data quality or setting configurations.
- **Key Settings:** Minimum Sales Threshold, Analysis Period, Sales Consistency Score Impact, Optimum Depth Target.

---

**Need Help?** Contact your Merchandising or Planning team for questions about this process.

---

Generated by [AI Codebase Knowledge Builder](#)

# Chapter 3: Projected ROS (Rate of Sale) Calculation

Last Updated: 2025-05-23

## 1. Business Objective

This module calculates the expected selling speed of each unique product (SKU), along with crucial underlying metrics like average selling price and historical sales depth, to inform future buying decisions.

**Why This Matters:** Accurate Rate of Sale (ROS) projections are essential for predicting customer demand. This helps merchandisers and buyers ensure they stock enough of popular items to avoid lost sales, while also preventing overstocking of slower-moving products, directly improving profitability and inventory efficiency.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Historical Units Sold & Revenue:** How many units of each product were sold in the past, and at what price. *Why we need it: To establish a baseline for past performance and calculate average selling prices and sales depth.*
- **Current Sales Trends:** Recent increases or decreases in demand for specific products or categories. *Why we need it: To adjust historical averages for current market conditions and identify emerging trends.*

### Product Data

- **Product Categories & Styles:** How products are grouped (e.g., “Men’s T-Shirts,” “Women’s Jeans,” “Home Decor”). *Why we need it: To calculate ROS within specific merchandising groups and ensure calculations are relevant to similar items.*
- **Product Attributes (e.g., Color, Size, Material):** Specific details about each unique product variation. *Why we need it: To analyze performance at the granular SKU level and identify high-performing or underperforming variations.*

## Operational Data

- **User-Defined Overrides:** Manual adjustments or strategic inputs from merchandisers for specific products or categories. *Why we need it: To incorporate expert insights or planned promotional impacts that historical data alone might not capture.*
- **Ideal Size Set (ISS) Information:** The standard combination of sizes typically stocked for a product type. *Why we need it: To provide a foundational depth assumption for new or low-history products, helping to normalize sales depth calculations.*

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Minimum Sales Threshold	5 units	The minimum number of units a product must sell within a given period to be considered for detailed ROS analysis. Products below this may be excluded or treated with caution.	If focusing on high-volume items, increase this; if analyzing niche or new products, decrease it to include more items.
Analysis Period	90 days	The historical duration used to calculate sales rates and project future performance. For example, if 90 days, it looks at sales from the last three months.	Shorten for fast-moving fashion trends or seasonal items; lengthen for stable, evergreen products to capture more data.
Sales Consistency Score Impact	0.8	How strongly consistent past sales performance influences the projected Rate of Sale (ROS). A higher value means past stability is more important.	Increase if historical reliability is paramount (e.g., basic items); decrease if recent trends or new product launches should take precedence.
Optimum Depth Target	80%	The target percentage of	Adjust based on inventory turnover

Setting Name	Default Value	Description	When to Change
		desired sell-through that influences the ideal number of unique items (OD) to stock. This is an output of the overall process, but inputs here can influence the calculated ROS.	goals; higher for faster-moving goods (e.g., basics), lower for items with longer shelf lives (e.g., furniture).

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Current Sales & Product Data] --> B[Projected ROS Calculation]

B --> C[Intermediate Metrics for Depth Calculation]

1. Start with → **Current Sales & Product Data**, including historical performance, current trends, and product details.
2. Then → **Projected ROS Calculation** processes this data, applying business rules and overrides to determine future sales speed.
3. Finally → **Intermediate Metrics for Depth Calculation** are prepared, providing the projected ROS and related values that the subsequent Optimum Depth module needs to calculate ideal stock breadth.

## 5. Step-by-Step Actions

1. **Gather** relevant historical data and business inputs
  - What we do: *We collect historical sales data (units and revenue) for all products across different time periods, along with current inventory levels and any specific business overrides provided by merchandisers.*
  - Why we do it: *This provides the raw material needed to understand past performance and apply forward-looking adjustments, ensuring our projections are grounded in reality.*
2. **Calculate** historical average selling prices (ASP)
  - What we do: *We analyze past transactions to determine the average price at which products were sold. This calculation can be done at various levels, such as per store, per store cluster, or across all stores for a given category.*
  - Why we do it: *Understanding historical ASP is crucial for projecting future revenue from a given number of units, helping to ensure the financial viability of buy plans.*

### 3. **Determine** historical sales depth

- What we do: *We assess how many unique items (SKUs) or variations within a product category have historically sold. This helps establish a baseline for the breadth of assortment that has performed well in the past.*
- Why we do it: *This metric provides insight into the variety customers typically purchase, which is vital for forecasting how many different products to stock to maximize sales.*

### 4. **Apply** user-defined overrides and segmentations

- What we do: *We incorporate specific instructions from merchandisers, such as planned promotions, new product launches, or strategic category focuses. We also apply segmentation rules that classify products (e.g., “Key,” “Core,” “Brand”).*
- Why we do it: *These inputs allow the system to account for future business strategies and adjust projections beyond what historical data alone would suggest, improving accuracy for dynamic retail environments.*

### 5. **Project** the Rate of Sale (ROS)

- What we do: *Using all calculated historical metrics, current trends, and applied overrides, we predict the future selling speed (units per period) for each unique product (SKU) and product category.*
- Why we do it: *This final projected ROS is the cornerstone of effective inventory planning, guiding exactly how much of each item should be ordered to meet anticipated demand without overstocking.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A specific “Women’s Basic Crew Neck Tee” in the “Everyday Essentials” category at your downtown store has sold 420 units over the last 90 days.
- Its historical Average Selling Price (ASP) has been \$15 per unit.
- The store typically stocks 8 unique sizes/colors (Ideal Size Set) for this type of basic tee.
- A merchandiser has applied an override, increasing the projected ROS for this category by 10% due to an upcoming summer promotion.

### What Happens

1. First, the system gathers the historical sales data (420 units over 90 days) and the current ASP (\$15). It notes the base historical ROS is 4.67 units/day (420 units / 90 days).
2. Next, it factors in the store’s “Everyday Essentials” category and the Ideal Size Set of 8. It also considers the “Sales Consistency Score” to ensure the historical performance is reliable.
3. Then, the system incorporates the merchandiser’s 10% override. This adjustment increases the projected ROS from 4.67 units/day to approximately 5.14 units/day ( $4.67 * 1.10$ ).
4. It recalculates the adjusted Average Selling Price if any ASP overrides were applied (none in this case, so it remains \$15).

5. Finally, the module outputs the projected ROS of 5.14 units/day for the “Women’s Basic Crew Neck Tee” at the downtown store, along with its historical ASP and calculated sales depth.

## Final Outcome

The buying team receives a precise projected ROS of 5.14 units per day for the “Women’s Basic Crew Neck Tee” at the downtown store, considering both its strong sales history and the upcoming promotion. This data helps them determine precisely how many units to order for future inventory.

**Key Takeaway:** By combining robust historical analysis with strategic business adjustments, this calculation provides a highly accurate forecast of future product demand, optimizing buying decisions and maximizing sales potential.

## 7. Results

This step produces:

### Main Outputs

- **Projected Rate of Sale (ROS) per SKU/Style:** The predicted daily or weekly sales speed for each unique product or style group. *What it tells us: How fast each item is expected to sell in the future, guiding precise inventory ordering.*
- **Historical Average Selling Price (ASP):** The average price at which items were sold in the past, calculated at various levels (store, cluster, total). *What it tells us: The average revenue generated per unit, crucial for financial planning and projecting buy quantities.*
- **Historical Sales Depth:** The calculated breadth of unique items sold within a category over a historical period. *What it tells us: How many different options customers tend to buy, informing optimal assortment variety.*
- **Adjusted Buy Quantity (Projected Buy Qty):** An initial estimate of the total units needed based on planned revenue and adjusted ASP. *What it tells us: A preliminary volume target for purchasing based on financial goals.*

### Business Reports

- **ROS & ASP Forecasting Report:** Used by merchandisers and planners to review the projected ROS, historical ASP, and any applied overrides. This report helps them validate assumptions and make informed adjustments to buy plans.
- **Category Performance Metrics Dashboard:** Used by category managers to monitor trends in ROS and ASP across different product categories. It highlights areas of strong or weak performance, allowing for strategic category adjustments.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	Calculations will use outdated or incomplete data, leading to inaccurate projections and potential stock issues.	Immediately check data sources and ensure all necessary sales, product, and inventory information is flowing correctly.
Products with no historical sales data	The system might default to an average ROS for similar products, or the Ideal Size Set value. There's no performance history to base projections on.	Merchandisers should manually review new products; consider market trends or comparable item performance for initial stocking decisions and apply manual overrides if necessary.
Sudden, unexpected sales spikes (e.g., viral trend)	Projected ROS might not fully capture the rapid increase in demand, leading to under-recommendations for future buys, as historical "Analysis Period" might not reflect it yet.	Monitor market trends closely and be prepared to make manual adjustments to buy plans for highly volatile products, leveraging the "User-Defined Overrides."
Very low-volume products or categories	The historical data might be too sparse to generate a statistically reliable ROS, leading to highly variable or inaccurate projections.	These items might default to generic segment averages or rely more heavily on "Ideal Size Set" and manual overrides. Consider merging small categories for analysis or using longer analysis periods.

## 9. Next Steps

### Before This Step

- **Data Collection & Cleaning:** Accurate and complete sales, product, and inventory data must be gathered and validated from all relevant sources. *What we need from it: Reliable, clean data for accurate calculations, preventing "garbage in, garbage out" scenarios.*
- **Category & Attribute Definition:** Product categories and key attributes (like color, size, material, or style) must be clearly defined and consistently assigned across all products. *What we need from it: Structured product information to group items correctly for meaningful analysis and reporting.*

## After This Step

- **Optimum Depth (OD) Calculation:** The calculated projected ROS and historical sales depth are fed into the OD module to determine the ideal number of unique items (SKUs) to stock. *What they need from us: The accurate projected sales speed and historical variety to calculate optimal assortment breadth.*
- **Ideal Buy Plan Generation:** The projected ROS and other metrics contribute to the overall buy plan, guiding the specific quantities of each SKU to purchase. *What they need from us: The core forecasted demand for each item to build out precise purchase order recommendations.*

## 10. Common Questions

Q: How does this module handle seasonality in sales? A: The module can account for seasonality by using an “Analysis Period” that aligns with seasonal cycles, or by incorporating specific seasonal overrides provided by merchandisers, ensuring projections reflect typical demand fluctuations.

Q: Can this system account for new product launches that have no sales history? A: While historical data is key, for brand-new products, the system may use industry benchmarks or sales patterns from similar successful products to estimate an initial ROS, though some manual oversight and adjustments might still be needed via user overrides.

Q: What if market trends change rapidly, making the projected sales outdated? A: The system uses a defined “Analysis Period” and considers a “Sales Consistency Score” to adapt to recent trends. For very rapid shifts (like a new viral trend), manual adjustments to the buy plan may still be necessary, and the system can be re-run frequently to capture fresh data.

Q: How does this impact our overall inventory investment? A: By providing precise ROS projections, this module directly helps minimize overstocking (reducing holding costs and improving cash flow) and understocking (reducing lost sales), leading to a more efficient and profitable inventory investment.

Q: Does this account for different demand in different store locations? A: Yes, the calculations are designed to consider sales performance at various levels, including individual store locations or store clusters, ensuring that ROS projections are tailored to specific regional or store-level demand patterns where relevant sales data exists.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates the projected Rate of Sale (ROS) and underlying metrics to forecast future product demand.
- **Main Input:** Historical sales data (units & revenue), product attributes, and user-defined overrides.
- **Main Output:** Projected Rate of Sale (ROS) per SKU/style, historical Average Selling Price (ASP), and historical sales depth.



- **Warning Signs:** Significant discrepancies between projected and actual sales, frequent stockouts on trending items, or high levels of aged inventory suggest issues with ROS accuracy.
  - **Key Settings:** Minimum Sales Threshold, Analysis Period, Sales Consistency Score Impact.
- 

**Need Help?** Contact your Merchandising or Planning team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 4: Optimum Depth (OD) and Buy Value Calculation

Last Updated: 2025-05-23

## 1. Business Objective

This module takes projected sales rates and style segmentation data to calculate the final Optimum Depth (OD), which is the ideal number of unique items (SKUs) to stock, and the corresponding buy values (inventory quantity and revenue) for each product style, ensuring the buy plan aligns with business objectives and minimizes stockouts.

**Why This Matters:** This step ensures that the buying team orders not just the right quantity, but also the optimal *variety* of products. By calculating the ideal number of unique items and their value, it helps maximize sales by offering enough choices, minimizes wasted inventory from over-assortment, and ensures financial targets are met.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Projected Rate of Sale (ROS):** The predicted selling speed for each unique product or style from the previous step. *Why we need it: To accurately forecast how many units will sell, which directly impacts the required inventory depth.*
- **Historical Sales Depth:** The number of unique items (SKUs) that have historically sold within specific product groups (like an Attribute Group). *Why we need it: To understand past assortment breadth that resonated with customers and use it as a baseline for future planning.*

### Product Data

- **Style Segmentation Data:** Information categorizing styles by importance (e.g., “Key,” “Core,” “Brand”) and their rank. *Why we need it: To prioritize which styles contribute to the optimal assortment and allocate inventory depth strategically.*
- **Ideal Size Set (ISS) Information:** The standard combination of sizes typically stocked for a product type. *Why we need it: To ensure that even for low-selling*

styles, a minimum viable assortment is maintained to meet basic customer needs.

## Operational Data

- **Planned Buy Revenue:** The target revenue merchandisers aim to achieve for a given product category or store. *Why we need it: To scale the calculated buy quantities and values to meet overall financial objectives.*
- **Depth Grade Multipliers:** Pre-defined multipliers based on a style's strategic importance or performance tier. *Why we need it: To adjust the ideal depth for specific styles, allowing for greater variety in high-priority or highly profitable items.*

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>Optimum Depth Target</b>	80%	The target percentage of desired sell-through that influences the ideal number of unique items (OD) to stock within a category. This helps manage assortment breadth.	Adjust based on inventory turnover goals; higher for faster-moving goods (e.g., basics), lower for items with longer shelf lives (e.g., furniture).
<b>Key Style Allocation</b>	20%	The percentage of the total targeted unique items allocated to "Key" styles within a category.	Increase if the business wants to heavily invest in its top-performing or strategic styles; decrease if a broader distribution across style types is preferred.
<b>Core Style Allocation</b>	50%	The percentage of the total targeted unique items allocated to "Core" styles within a category.	Adjust based on the proportion of stable, everyday items that form the backbone of the assortment.
<b>Brand Style Allocation</b>	30%	The percentage of the total targeted unique items allocated to "Brand"	Modify based on the importance of new or trending brand items versus

Setting Name	Default Value	Description	When to Change
		styles within a category.	established core and key styles.

*Note: See More Details for additional settings.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Projected Sales & Buy Plan Metrics] --> B[Calculate Initial Style Buy Values]

B --> C[Adjust for Planned Revenue & Style Tags]

C --> D[Determine Final Optimum Depth (OD) & Buy Value]

D --> E[Final Buy Plan Data]

1. Start with → **Projected Sales & Buy Plan Metrics**, including forecasted sales rates, planned revenue targets, and target sell-through rates.
2. Then → **Calculate Initial Style Buy Values** by determining a preliminary Optimum Depth (OD) for each style based on historical assortment breadth and ideal size sets.
3. Next → **Adjust for Planned Revenue & Style Tags**, scaling the initial buy values to meet overall financial targets and prioritizing styles based on their strategic tags (Key, Core, Brand).
4. Finally → **Determine Final Optimum Depth (OD) & Buy Value** for each style, producing the definitive recommendations for what variety and quantity to purchase.
5. This feeds into → **Final Buy Plan Data** for subsequent steps.

## 5. Step-by-Step Actions

1. **Gather** projected sales and historical depth data
  - What we do: *We pull in the forecasted Rate of Sale (ROS) for individual items, the historical assortment depth for product groups, and pre-defined Ideal Size Set (ISS) information.*
  - Why we do it: *These provide the foundational metrics for determining how much variety and quantity are needed for each product.*
2. **Determine** initial Optimum Depth (OD) for each style
  - What we do: *For each style, we calculate an initial ideal number of unique items by considering its historical assortment depth (how many variations typically sold) and ensuring it meets the minimum Ideal Size Set (ISS).*
  - Why we do it: *This creates a baseline assortment breadth that balances past sales patterns with basic customer expectations.*
3. **Prioritize** styles based on segmentation and allocate options
  - What we do: *We segment styles into categories like “Key,” “Core,” and “Brand” based on their strategic importance. We then allocate the total targeted unique items for the entire category across these segments using pre-defined percentages.*

- Why we do it: *This ensures that high-priority styles receive a larger share of the overall assortment, aligning with business strategy and maximizing the impact of top performers.*
- 4. **Apply** depth grading and financial adjustments
  - What we do: *We further refine the Optimum Depth (OD) for each style by applying depth grade multipliers (e.g., giving more options to high-value styles). We also calculate an overall revenue alignment factor by comparing the initial aggregated buy value to the planned target buy revenue.*
  - Why we do it: *These adjustments fine-tune the assortment breadth to meet specific financial goals and ensure the buy plan is profitable and strategic.*
- 5. **Calculate** final buy quantities and values
  - What we do: *We apply the revenue alignment factor to the calculated Optimum Depth (OD) for each style. This generates the final recommended number of unique items to buy (final OD) and their corresponding inventory value and quantity.*
  - Why we do it: *This ensures the final buy plan is precise, financially sound, and aligned with overall business objectives, preventing both over-ordering and under-ordering.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A "Women's Denim Jeans" category for a specific store tier has a Target Buy Revenue of \$100,000 for the upcoming period.
- The previous "Projected ROS Calculation" module provided a Projected ROS for various denim styles and a Historical Sales Depth of 10 unique items for the "Slim Fit" style within this category.
- The Ideal Size Set (ISS) for all denim is 5 unique items.
- "Slim Fit Dark Wash" is classified as a "Key" style, and the overall business setting for Key Style Allocation is 20% of the total targeted unique items.
- The Optimum Depth Target (desired sell-through) is 80%.

### What Happens

1. First, the system receives the projected ROS and historical sales depth for all denim styles. For "Slim Fit Dark Wash," it uses the Historical Sales Depth of 10 and the Ideal Size Set of 5.
2. Next, it calculates an initial Optimum Depth for "Slim Fit Dark Wash." If its sell-through adjusted historical depth is 12 ( $10 / 0.8$ ), and ISS is 5, the initial OD might be 12 (taking the higher of the two).
3. Then, it identifies "Slim Fit Dark Wash" as a "Key" style and, considering the Total Targeted Unique Items (e.g., 50 for the category, derived from overall budget and average price), allocates its share (20% of 50 = 10 unique items for all "Key" styles, to be distributed by rank).
4. After calculating initial buy values for all selected styles, the system sums them up to get a Total Initial Buy Value (e.g., \$120,000). Since the Target Buy Revenue is \$100,000, it calculates a Revenue Alignment Factor of 0.83 ( $\$100,000 / \$120,000$ ).

5. Finally, this Revenue Alignment Factor of 0.83 is applied to the initial OD of “Slim Fit Dark Wash” and all other styles. So, an initial OD of 12 for “Slim Fit Dark Wash” might become a Final Optimum Depth (OD) of 10 ( $12 * 0.83$ , rounded up). The Final Style Buy Value is then calculated by multiplying this final OD by the style’s Average Style Price.

## Final Outcome

The buying team receives a recommendation that for the “Women’s Denim Jeans” category, the “Slim Fit Dark Wash” style should have a Final Optimum Depth (OD) of 10 unique items and a specific Final Style Buy Value, which contributes to the overall \$100,000 target.

**Key Takeaway:** By factoring in past sales breadth, strategic importance, and overall financial targets, the system precisely tailors the assortment depth for each style, ensuring the right variety is purchased to meet sales and revenue goals.

## 7. Results

This step produces:

### Main Outputs

- **Final Optimum Depth (OD) per Style:** The precise ideal number of unique items (SKUs) recommended to be stocked for each product style. *What it tells us: The exact breadth of assortment to offer for each style to maximize sales and optimize inventory.*
- **Final Style Buy Value per Style:** The recommended total inventory value (in currency) to allocate for purchasing each product style. *What it tells us: The budget allocation for each style, aligning inventory investment with strategic importance and projected demand.*
- **Final Buy Quantity per Style:** The total number of units recommended to be purchased for each product style, derived from its Final OD and average pack size. *What it tells us: The specific unit quantities to order from suppliers, ensuring sufficient stock to meet forecasted sales.*
- **Buy Plan Summary per Category/Store:** Aggregated final OD, buy value, and buy quantity summaries for broader product categories or store locations. *What it tells us: An overview of the planned assortment breadth and financial commitment at a higher level, useful for strategic planning.*

### Business Reports

- **Style-Level Buy Plan Report:** Used by merchandisers and buyers to review the detailed final OD, buy quantities, and buy values for each style. This report guides precise purchase order generation.
- **Assortment Health Dashboard:** Used by category managers to monitor the planned assortment mix. It highlights the allocation of unique items across “Key,” “Core,” and “Brand” styles, ensuring strategic alignment and optimal variety.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	Calculations will use outdated or incomplete data for projected ROS or style segmentation, leading to inaccurate OD and buy values.	Immediately check data sources and ensure all necessary sales, product, and style segmentation information is flowing correctly.
Category planned revenue is zero or very low	The Revenue Alignment Factor might become zero or very small, resulting in minimal or no recommended buy quantities for any style.	Verify the Planned Buy Revenue for the category; if intentionally low, adjust expectations for assortment breadth or consider manual overrides.
Styles with no historical sales depth	The system will likely default to the Ideal Size Set (ISS) as the base for OD, or a pre-defined average for similar styles.	Merchandisers should manually review new styles or those with sparse data; apply manual overrides if initial recommendations seem too low or high.
Total Calculated Buy Value significantly higher than Planned Buy Revenue	The Revenue Alignment Factor will be less than 1, causing all calculated ODs and buy values to be reduced across the board.	Review the Planned Buy Revenue for realism against projected demand, or identify if individual style initial ODs are too high based on their projected ROS.

## 9. Next Steps

### Before This Step

- **Projected ROS (Rate of Sale) Calculation:** Accurate projected ROS, historical ASP, and historical sales depth for each style must be generated. *What we need from it: The fundamental forecast of how fast products will sell and their historical variety, which are critical inputs for determining optimal depth.*
- **Style Segmentation & Attribute Group (AG) Definition:** Styles must be correctly categorized (Key, Core, Brand) and attribute groups defined for accurate historical depth analysis. *What we need from it: Structured product data and strategic classifications to enable targeted OD calculations and style prioritization.*

### After This Step

- **Open-to-Buy (OTB) Planning:** The final calculated buy quantities and values for each style are aggregated and fed into the broader OTB process to determine the total budget allocated for new purchases. *What they need from us: The*

*precise, data-driven recommendations on what variety and quantity to buy to finalize budget allocations.*

- **Purchase Order Generation:** The specific Final Buy Quantity for each unique style is used to create actual purchase orders with suppliers. *What they need from us: The exact unit quantities to procure, ensuring the right products are ordered for optimal inventory.*

## 10. Common Questions

Q: How does this module prevent me from ordering too much variety for slow-selling items? A: This module considers the Ideal Size Set (ISS) as a minimum depth and uses your Optimum Depth Target and Planned Buy Revenue to constrain the overall assortment. It also prioritizes styles based on their “Key,” “Core,” or “Brand” tags, ensuring slower-moving items don’t disproportionately consume buy budget or display space.

Q: Can I influence which styles get more depth, even if their historical sales depth isn’t very high? A: Yes, you can influence this by assigning styles to “Key” tags, adjusting their Depth Grade Multipliers, or providing User-Defined Overrides in the preceding steps. This allows strategic styles to receive higher priority and depth despite past performance.

Q: What if the overall Planned Buy Revenue is much lower than what the system initially calculates as needed? A: The system automatically calculates a Revenue Alignment Factor to scale down all recommended ODs and buy values proportionally so that the total buy value aligns with your Planned Buy Revenue. This ensures financial targets are met even if it means a tighter assortment.

Q: How does this help prevent stockouts on popular items if it’s also limiting variety? A: The module balances variety with sales velocity. It leverages the Projected Rate of Sale (ROS) from the previous step and aims to maintain sufficient depth for higher-ranking styles, ensuring popular items are prioritized for broader assortment and adequate stock, while less critical items might have a narrower depth.

Q: Does this account for different stores or store types having different ideal depths? A: Yes, the calculations consider historical sales depth at granular levels (like Store + Attribute Group (AG) or Store Tier + AG). This allows the system to tailor the final Optimum Depth (OD) and buy values to the specific needs and performance of individual stores or store clusters.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates the final ideal number of unique items (OD) and corresponding buy values for each style, aligning with financial targets.
- **Main Input:** Projected Rate of Sale (ROS), historical sales depth, style segmentation, Ideal Size Set (ISS), and planned buy revenue.
- **Main Output:** Final Optimum Depth (OD) per style, Final Style Buy Value, and Final Buy Quantity per style.



- **Warning Signs:** Significant discrepancies between planned and actual inventory investment or an assortment that doesn't align with strategic style priorities.
  - **Key Settings:** Optimum Depth Target, Key/Core/Brand Style Allocation percentages.
- 

**Need Help?** Contact your Merchandising or Planning team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# otbStyleWiseBuy FAQ

**Version:** 1.0 **Last Updated:** 2025-05-23

This FAQ provides quick answers to common questions about the otbStyleWiseBuy algorithm, explaining its purpose, process, and key concepts for business users.

---

## General

**Q:** What is the purpose of otbStyleWiseBuy? **A:** This algorithm is designed to calculate the ideal inventory quantities for each product style, ensuring we stock the right items to maximize sales and minimize stockouts, while aligning with our Open-to-Buy (OTB) plans.

**Q:** What does “OTB Style Wise Buy” mean? **A:** OTB (Open-to-Buy) refers to the budget available for purchasing new inventory. “Style Wise Buy” means the system determines exactly how much to buy for each specific product style, ensuring efficient use of the OTB budget.

**Q:** Who uses the outputs of this algorithm? **A:** Merchandisers, planners, and inventory managers primarily use these outputs to make informed purchasing decisions and optimize stock levels across our product offerings.

**Q:** How does this algorithm help prevent stockouts or overstocking? **A:** By accurately projecting the Rate of Sale (ROS) and calculating the Optimum Depth (OD) for each style, the algorithm determines the ideal inventory levels, thereby helping us avoid having too little (stockout) or too much (overstocking) of a particular style.

**Q:** Can this algorithm be applied to specific categories or store segments? **A:** Yes, the system considers store presence in segmentation and processes data at a detailed level, allowing for tailored buying decisions based on various Attribute Groups (AGs) and segments.

---

## Configuration & Settings

**Q:** How can I influence the projected Rate of Sale (ROS) for a style? **A:** The Projected ROS Calculation module allows for user-defined overrides. You can apply these overrides to adjust the base ROS calculation based on your business insights or anticipated promotions.

**Q:** Are there settings to account for different sales periods when calculating buys?  
**A:** Yes, the OTB Style Wise Buy Input Validator module explicitly checks for valid sales periods, indicating that defining and selecting appropriate sales periods is a key input that influences the calculations.

**Q:** Can I adjust the Ideal Size Set (ISS) used for a style? **A:** The OTB Style Wise Buy Input Validator verifies the availability and correctness of the Ideal Size Set (ISS). This implies that ISS configurations are important inputs that directly influence the final calculated buy quantities for each style.

**Q:** How does the system align the buy plan with overall business objectives like planned revenue? **A:** The Optimum Depth (OD) and Buy Value Calculation module applies specific adjustments based on planned revenue targets and style tags. This ensures that the calculated buy quantities support broader financial goals.

**Q:** Where are key parameters for the buy calculation defined? **A:** While specific configuration settings (e.g., from `args.properties`) can vary, the algorithm generally relies on user-defined overrides, planned revenue, and style tags to customize the buy plan according to strategic objectives.

---

## Data & Inputs

**Q:** What kind of data does the `otbStyleWiseBuy` algorithm use? **A:** It primarily uses historical sales data, style segmentation data (Attribute Groups - AGs), store segmentation, style-SKU/MRP mapping, and Ideal Size Set (ISS) definitions.

**Q:** How is data quality ensured before the calculations begin? **A:** The OTB Style Wise Buy Input Validator module performs critical checks on all input data, including sales periods, store presence in segmentation, and style-SKU/MRP mapping, to ensure the highest data quality before processing.

**Q:** What is “style segmentation data” and why is it important? **A:** Style segmentation data groups products by common characteristics or an Attribute Group (AG), such as color, size, or brand. This helps the algorithm apply relevant business rules and projections for similar styles, significantly improving the accuracy of Rate of Sale (ROS) and Optimum Depth (OD) calculations.

**Q:** What happens if the Ideal Size Set (ISS) is missing for a style? **A:** The Input Validator checks for ISS availability. If the Ideal Size Set is missing, the process would typically flag an error or use a predefined default, as ISS is crucial for determining the best combination of sizes to maximize sales and minimize stockouts.

**Q:** What is “MRP mapping” and why is it needed? **A:** MRP (Maximum Retail Price) mapping helps the system understand the price points for different style-SKUs. This information is essential for calculating historical Average Selling Price (ASP) and accurately projecting future revenue.

---

## Process & Logic

**Q:** What are the main steps involved in the `otbStyleWiseBuy` calculation? **A:** The process starts with input data validation (handled by the `OTB Style Wise Buy Input Validator`), followed by the calculation of Projected Rate of Sale (ROS) by the `Projected ROS Calculation` module, and finally the determination of Optimum Depth (OD) and overall buy values by the `Optimum Depth (OD) and Buy Value Calculation` module. The `OTB Style Wise Buy Process Orchestrator` manages this entire sequence.

**Q:** How is the Rate of Sale (ROS) projected? **A:** The `Projected ROS Calculation` module uses historical sales data, historical Average Selling Price (ASP), historical sales depth, and segmentation inputs. It also incorporates user-defined overrides to refine the projection based on current strategies.

**Q:** How is the Optimum Depth (OD) determined? **A:** The `Optimum Depth (OD) and Buy Value Calculation` module utilizes the projected Rate of Sale (ROS) and style segmentation data (Attribute Groups - AGs). It then applies adjustments based on planned revenue and style tags to arrive at the ideal number of unique items (SKUs) to stock.

**Q:** What is the relationship between ROS and OD? **A:** The projected Rate of Sale (ROS) is a foundational input for calculating the Optimum Depth (OD). A higher projected ROS generally indicates a greater anticipated demand, leading to a higher recommended OD to meet that demand.

**Q:** Why is input validation a separate, initial step in the process? **A:** The `OTB Style Wise Buy Input Validator` runs first to ensure that all required data is accurate and complete before any complex calculations begin. This prevents errors downstream, ensures the integrity of the outputs, and improves the overall reliability of the buy plan.

---

## Outputs & Results

**Q:** What are the main outputs of the `otbStyleWiseBuy` algorithm? **A:** The primary outputs are the calculated Optimum Depth (OD) for each style and corresponding buy values, which include recommended inventory quantities and projected revenue.

**Q:** Can I see the projected Rate of Sale (ROS) values? **A:** Yes, the `Projected ROS Calculation` module produces intermediate results, including the projected ROS, which are then used by subsequent modules. These values are typically available for review and analysis.

**Q:** How are the buy values presented? **A:** Buy values are provided for each style as recommended inventory quantity (units) and projected revenue, clearly indicating how much to buy and the expected sales value.

**Q:** Are the outputs aggregated or detailed by style? **A:** The calculation is “style wise,” meaning the outputs for Optimum Depth (OD) and buy values are highly detailed, provided for each individual product style.

**Q:** How do I access the final buy plan? **A:** The OTB Style Wise Buy Process Orchestrator is responsible for uploading the calculated outputs to the designated output system or report, making them readily available for merchandising and planning teams.

---

## **Error Handling & Edge Cases**

**Q:** What happens if critical input data is missing or incorrect? **A:** The OTB Style Wise Buy Input Validator module is specifically designed to flag such issues early in the process. This prevents calculations from proceeding with bad data, ensuring the integrity and reliability of the final outputs.

**Q:** How does the system handle styles with very limited or no historical sales data? **A:** While not explicitly detailed, typically for new or very low-volume styles, the system would rely on broader segmentation data (Attribute Groups - AGs), similar styles, or user-defined overrides to estimate the Rate of Sale (ROS) and Optimum Depth (OD).

**Q:** What if there are conflicts between user overrides and calculated values? **A:** The Projected ROS Calculation module is designed to incorporate user-defined overrides, implying that these overrides are either prioritized or intelligently blended with historical data to influence the final ROS projection.

**Q:** Can the algorithm adjust for sudden market shifts or promotions not reflected in historical data? **A:** While historical data forms the base, the system allows for manual adjustments through user-defined overrides in the Projected ROS Calculation module. This enables merchandisers to account for anticipated changes due to promotions or market shifts not yet visible in historical trends.

**Q:** What if a style's Ideal Size Set (ISS) is identified as incomplete or invalid during validation? **A:** The Input Validator would flag this issue, potentially halting the process for that style or requiring manual intervention to correct the Ideal Size Set (ISS). A complete and accurate ISS is vital for determining the best combination of sizes and thus for accurate buy quantity recommendations.

---

## **Need More Help?**

Please contact the support team at [Support Email/Internal Link].

Here is the comprehensive documentation for the otbStyleWiseBuy project, structured according to your specifications.

---

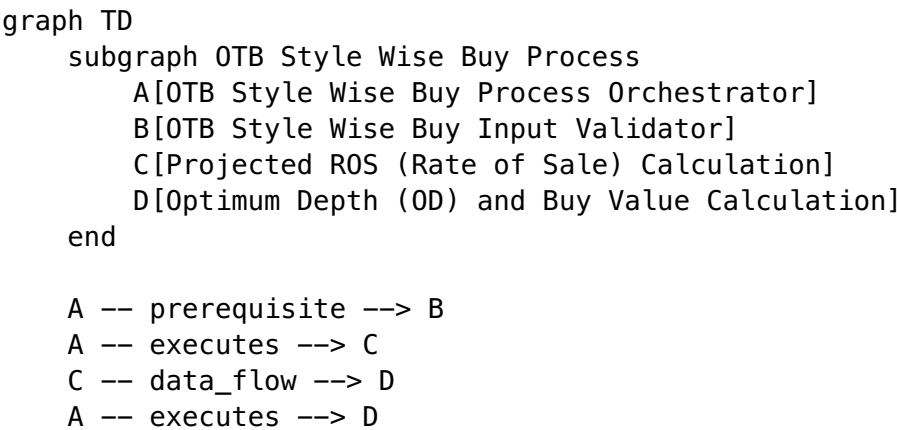
# otbStyleWiseBuy Documentation

Version: 1.0 Last Updated: 2025-05-23

## 2. Executive Summary

The otbStyleWiseBuy algorithm empowers our buying teams to make data-driven inventory decisions by calculating the **Optimum Depth (OD)** and precise buy values for each style. This system ensures that we stock the ideal number of unique items (SKUs) and quantities, maximizing sales potential, minimizing stockouts, and aligning our inventory investments with strategic business objectives.

## 3. Architecture Overview



### Module Descriptions:

- **OTB Style Wise Buy Process Orchestrator:** This module coordinates the entire OTB Style Wise Buy calculation, managing the sequential execution and data flow between all sub-processes.
- **OTB Style Wise Buy Input Validator:** This module performs essential checks on all incoming data, ensuring its quality and correctness before any calculations begin.
- **Projected ROS (Rate of Sale) Calculation:** This module determines the anticipated sales speed and associated metrics for each style based on historical performance and predefined overrides.
- **Optimum Depth (OD) and Buy Value Calculation:** This module leverages the projected Rate of Sale to determine the ideal number of unique items (OD) and the corresponding inventory quantity and revenue for each style.

## 4. Prerequisites & Setup

To run the `otbStyleWiseBuy` process, ensure the following are in place:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Apache Maven:** Version 3.6 or higher.

### Build Commands:

1. Navigate to the project root directory:

```
cd otbStyleWiseBuy/
```

2. Clean and build the project:

```
mvn clean install
```

### How to Run the Orchestrator (ModuleGroup):

The main orchestration logic resides within the `otbStyleWiseBuy` Process Orchestrator module. Once the project is built, you can execute it typically via a command line, passing the `args.properties` file:

```
java -jar target/otbStyleWiseBuy-1.0.jar --config args.properties
```

*Note: The actual executable JAR name and class path may vary depending on the build configuration.*

## 5. Folder & File Structure

```
otbStyleWiseBuy/
├─ module-group/ # Contains the orchestrator logic to
manage process flow.
├─ submodules/ # Houses individual calculation
modules (e.g., ROS, OD).
├─ args.properties # Global configuration parameters for
the entire pipeline.
├─ common-data.json # Defines the schema for shared data
objects used across modules.
└─ outputs/ # Directory where final calculated
results and reports are stored.
```

## 6. Global Configuration

The `args.properties` file contains global configuration parameters for the entire `otbStyleWiseBuy` process.

Setting Name	Default Value	Description	Valid Range/When to Change
<i>(No global configuration)</i>			

Setting Name	Default Value	Description	Valid Range/When to Change
<i>parameters provided in input)</i>			

## 7. Shared Data Model

The `common-data.json` schema defines the structure for data objects shared and passed between different modules in the `otbStyleWiseBuy` pipeline.

Field	Type	Description
<i>(No common data schema provided in input)</i>		

## 8. Submodule Chapters

### 8.1. OTB Style Wise Buy Process Orchestrator

1. **Module Name & Business Objective** This module orchestrates the entire `otbStyleWiseBuy` calculation process. Its primary business objective is to ensure a cohesive and automated workflow, from initial data validation to final output generation, streamlining the buy planning cycle and ensuring data integrity across all stages. It initiates the data snapshot, sequentially executes the Projected ROS Calculation and Optimum Depth (OD) Calculation modules, and finally handles the upload of the calculated outputs.

#### 2. Required Information

- **Sales Data:** None directly; it only references data processed by sub-modules.
- **Product Data:** None directly.
- **Operational Data:** Location of input files, output directories, and execution flags for sub-modules.

3. **Business Settings** This module's settings primarily involve specifying the order of execution for sub-modules and paths for input/output data. It does not contain business logic specific to buy calculations.

#### 4. Process Flow

```
graph TD
 A[Start Orchestration] --> B[Snapshot Input Data]
 B --> C[Run Input Validator]
 C --> D[Run Projected ROS Calculation]
 D --> E[Run Optimum Depth (OD) Calculation]
 E --> F[Upload Results]
 F --> G[End Orchestration]
```



## 5. Step-by-Step Actions

- **Initiate** data snapshot: *To capture a consistent set of raw data for the entire process.*
- **Execute** OTB Style Wise Buy Input Validator: *To ensure all input data meets quality standards before calculations begin.*
- **Execute** Projected ROS Calculation: *To determine the historical and projected sales velocities for each style.*
- **Execute** Optimum Depth (OD) and Buy Value Calculation: *To calculate the ideal inventory levels and corresponding buy quantities based on projected ROS and business targets.*
- **Handle** output upload: *To make the final buy plan available for review and further action.*

6. **Sample Scenario** The buying team needs to generate the buy plan for the upcoming Spring/Summer collection. The Orchestrator is triggered, first ensuring all sales and product data are valid. Then, it sequentially calculates the ROS for new and existing styles. Following that, it determines the **OD** and final buy values for each style, considering planned revenue and historical sales. Finally, the completed buy plan is uploaded to the central system for review.

**Key Takeaway:** The Orchestrator ensures a smooth, automated, and error-checked progression from raw data to a complete buy plan.

7. **Results** The Orchestrator produces the final `otbStyleWiseBuy` report, encompassing projected ROS, calculated **OD** per style, recommended buy quantities, and associated buy values. It also generates execution logs indicating the success or failure of each submodule.

## 8. Unusual Cases

- **Missing or delayed data feed:** The orchestrator will likely fail at the validation step if critical input data is not available on time.
- **Sub-module failure:** If any of the executed modules (e.g., ROS calculation) encounter an error, the orchestrator will stop the process and report the failure.
- **Configuration errors:** Incorrect paths or settings in `args.properties` can lead to the orchestrator failing to find or execute sub-modules.

9. **Next Steps** The generated buy plan is reviewed by buying and merchandising teams. Upon approval, purchase orders (POs) are generated based on the recommended buy quantities.

## 10. Common Questions

1. How do I re-run the entire process if there's a data error?
2. What happens if one of the calculation modules fails?
3. Can I run only a specific part of the pipeline (e.g., just the ROS calculation)?
4. How long does a full execution typically take?
5. Where can I find the detailed logs for each step?

## 11. Quick Reference

- **Purpose:** Process automation and workflow management.
- **Inputs:** Global configurations, triggers.

- **Outputs:** Orchestration logs, final aggregated buy plan (via upload).

## 8.2. OTB Style Wise Buy Input Validator

1. **Module Name & Business Objective** This module is responsible for validating all input data required for the `otbStyleWiseBuy` calculation. Its primary business objective is to ensure the accuracy, completeness, and consistency of critical data points before any downstream calculations begin. This pre-processing step prevents errors, ensures reliable outputs, and maintains the integrity of the entire buy planning process. It performs checks on sales periods, store presence in segmentation, style-SKU/MRP mapping, Ideal Size Set (ISS) availability, and the correctness of override levels.

### 2. Required Information

- **Sales Data:** Historical sales units, historical sales revenue, sales dates.
- **Product Data:** SKU master data (Style, SKU, Category, AGs like color/material), Style-SKU mapping, MRP (Maximum Retail Price).
- **Operational Data:** Store segmentation definitions, Ideal Size Set (ISS) configurations per style, User-defined override values for ROS or OD.

3. **Business Settings** This module relies on predefined validation rules and thresholds (e.g., minimum historical sales period, valid ranges for overrides, mandatory fields).

### 4. Process Flow

graph TD

```
A[Raw Input Data] --> B{Validate Sales Periods}
B --> C{Validate Store Segmentation}
C --> D{Validate Style-SKU/MRP Mapping}
D --> E{Validate Ideal Size Set (ISS)}
E --> F{Validate Override Levels}
F --> G{Generate Validation Report}
G -- Valid Data --> H[Ready for Calculation]
G -- Invalid Data --> I[Errors Reported]
```

### 5. Step-by-Step Actions

- **Check** sales periods: *To ensure historical sales data covers the required timeframe and is continuous.*
- **Verify** store presence in segmentation: *To confirm all relevant stores are correctly assigned to their respective segments.*
- **Validate** style-SKU/MRP mapping: *To ensure accurate relationships between styles and their individual SKUs and correct pricing data.*
- **Confirm** Ideal Size Set (ISS) availability: *To ensure each style has a defined size breakdown for accurate buy quantity allocation.*
- **Assess** override levels: *To check if user-provided overrides (e.g., for ROS) are within acceptable business bounds.*
- **Generate** detailed validation report: *To provide clear feedback on any data quality issues identified.*

6. **Sample Scenario** During the validation of input data for casual wear, the module flags two critical issues:

1. Historical sales data for Q4 2024 is missing for the “Nordic” region.
2. Three new dress styles lack a defined Ideal Size Set (**ISS**), meaning the system wouldn’t know how to distribute buy quantities across sizes.
3. An override for a specific style’s **ROS** is set to 0.0, which is invalid. **Key Takeaway:** The validation prevents the downstream ROS and OD calculations from using incomplete or incorrect data, avoiding inaccurate buy recommendations.

7. **Results** A comprehensive validation report detailing all identified data discrepancies, warnings, and errors. If validation passes, the module effectively “certifies” the input data, making it ready for processing by subsequent modules.

## 8. Unusual Cases

- **Missing or delayed data feed:** If a critical data source (e.g., daily sales) is entirely absent, the validation will fail immediately.
- **Structural data issues:** Input files with incorrect column headers or malformed formats will cause parsing errors.
- **Inconsistent data:** A style exists in product master but has no corresponding sales, or a store is segmented but has no sales data.
- **Extensive data errors:** If a large percentage of records fail validation, it indicates a systemic data quality problem.

9. **Next Steps** If validation errors are found, the data owners must correct the source data. Once corrected, the validation module should be re-run until all critical errors are resolved, clearing the path for the main calculation process.

## 10. Common Questions

1. Why did the validation fail? How do I interpret the error messages?
2. Can I proceed with some warnings, or do all issues need to be resolved?
3. How frequently should input data be validated?
4. What’s the process for correcting validation errors?
5. Does validation check for data consistency between different sources?

## 11. Quick Reference

- **Purpose:** Data quality assurance, error prevention.
- **Inputs:** Raw sales data, product master, segmentation, overrides.
- **Outputs:** Validation report, “clean” input data for downstream.

## 8.3. Projected ROS (Rate of Sale) Calculation

1. **Module Name & Business Objective** This module calculates the projected **Rate of Sale (ROS)** and various underlying metrics such as historical Average Selling Price (ASP) and historical sales depth. Its primary business objective is to establish a data-driven baseline for future buy quantities by accurately forecasting how quickly products are expected to sell. It incorporates user-defined overrides and utilizes historical sales data and segmentation inputs to determine the base for future buy quantities. It prepares intermediate results for the subsequent depth calculation.

## 2. Required Information

- **Sales Data:** Historical daily/weekly unit sales per SKU/Style, historical revenue per SKU/Style.
- **Product Data:** Style master data (including AGs), SKU-Style mapping, new style indicators.
- **Operational Data:** Store segmentation, periods for ROS calculation (e.g., last 13 weeks), user-defined ROS overrides for specific styles or attributes.

## 3. Business Settings

- **ROS Calculation Period:** Defines the historical window (e.g., 13, 26, 52 weeks) to analyze sales trends.
- **New Style Treatment:** Rules for assigning initial **ROS** to styles with no sales history.
- **Override Application Logic:** Defines how user-provided overrides (e.g., a specific style should have a 15% higher **ROS**) are applied.
- **ASP Calculation Method:** Rules for determining Average Selling Price (e.g., average of actual sales prices over a period).

## 4. Process Flow

graph TD

A[Historical Sales Data] --> B{Group Sales by Style & Segment}

B --> C{Calculate Base ROS & ASP}

C --> D{Identify New Styles & Low Performers}

D --> E{Apply User Overrides}

E --> F[Projected ROS & Metrics]

## 5. Step-by-Step Actions

- **Extract** historical sales data: *To retrieve unit sales and revenue for defined periods.*
- **Group** sales by Style, Segment, and AGs: *To analyze performance at relevant business levels.*
- **Compute** historical **ROS** (units/week) and ASP for each style/segment: *To understand past sales velocity and pricing.*
- **Identify** new styles or styles with limited history: *To apply rule-based **ROS** estimates (e.g., based on similar styles).*
- **Apply** user-defined overrides: *To incorporate market insights, promotional impacts, or strategic adjustments that modify the calculated base **ROS**.*
- **Generate** projected **ROS** metrics: *To provide the key input for the subsequent **OD** calculation.*

6. **Sample Scenario** A popular T-shirt style (**Style ABC**) historically sold 200 units/week over the last 13 weeks with an ASP of \$15.00. However, the merchandising team anticipates a new marketing campaign will boost sales by 10%. The module first calculates the historical **ROS** (200 units/week) and ASP (\$15.00). Then, it applies the user-defined 10% **ROS** override, resulting in a projected **ROS** of 220 units/week for **Style ABC**. **Key Takeaway:** This module provides a quantitative, adjusted forecast of how quickly products are expected to sell, serving as the foundation for inventory planning.

## 7. Results Outputs include:

- Projected **ROS** per style/segment.
- Historical ASP per style/segment.
- Historical sales depth per style/segment.
- Flagging of new styles or styles with insufficient history.

## 8. Unusual Cases

- **New styles with no history:** ROS must be estimated based on comparable styles or default values, impacting accuracy.
- **Highly volatile sales patterns:** Styles with erratic sales due to promotions or seasonality can make ROS projection challenging.
- **Missing historical data:** Gaps in sales history can lead to under- or over-estimation of ROS.
- **Incorrect overrides:** User-provided overrides that are too high or too low can skew the projected ROS and subsequent buy plan.

## 9. Next Steps

The calculated projected ROS and associated metrics are fed directly into the Optimum Depth (OD) and Buy Value Calculation module, where they are used to determine the ideal inventory levels.

## 10. Common Questions

1. How is the “projected” ROS different from historical ROS?
2. What data sources are used for ROS calculation?
3. How are new styles handled when there’s no sales history?
4. Can I override the ROS for specific styles or categories? If so, how?
5. How does seasonality impact the ROS calculation?

## 11. Quick Reference

- **Purpose:** Sales velocity forecasting.
- **Inputs:** Historical sales, product attributes, overrides.
- **Outputs:** Projected ROS, ASP, sales depth.

## 8.4. Optimum Depth (OD) and Buy Value Calculation

1. **Module Name & Business Objective** This module takes the projected **Rate of Sale (ROS)** and style segmentation data to calculate the final **Optimum Depth (OD)** (ideal number of unique items) and corresponding buy values (inventory quantity and revenue) for each style. Its primary business objective is to translate sales forecasts into actionable inventory recommendations, optimizing SKU assortment and quantity to maximize sales while minimizing excess stock. It applies adjustments based on planned revenue and style tags, ensuring that the final buy plan aligns with business objectives and minimizes stockouts.

### 2. Required Information

- **Projected ROS:** From the Projected ROS (Rate of Sale) Calculation module.
- **Style Segmentation Data:** How styles are grouped (e.g., by category, lifecycle stage, performance tier).
- **Ideal Size Set (ISS):** The best combination of sizes to maximize sales and minimize stockouts for each style.

- **Planned Revenue Targets:** Top-down financial goals that influence buy value.
- **Current Inventory Data:** (Implicitly needed to calculate “open-to-buy,” though not directly stated in description, it’s core to a buy value calc).

### 3. Business Settings

- **Target Weeks of Supply:** Defines the desired inventory coverage for each style/segment.
- **Service Level Targets:** Desired in-stock percentage, influencing safety stock.
- **Revenue Allocation Rules:** How planned revenue is distributed across styles.
- **Style Tags/Adjustments:** Rules for modifying **OD** or buy quantities based on specific style characteristics (e.g., “core,” “fashion,” “promotional”).
- **Minimum/Maximum Buy Quantities:** Constraints on the final recommended quantities.

### 4. Process Flow

graph TD

```

 A[Projected ROS & Style Segmentation] --> B{Determine Target Inventory Coverage}
 B --> C{Calculate Raw Optimum Depth (OD)}
 C --> D{Apply Ideal Size Set (ISS)}
 D --> E{Allocate Planned Revenue & Adjust Buy Value}
 E --> F[Final Optimum Depth (OD) & Buy Plan]

```

### 5. Step-by-Step Actions

- **Receive** projected **ROS** and style segmentation: *As the core input for determining inventory needs.*
- **Determine** target inventory coverage: *Based on target weeks of supply and service level goals for each style/segment.*
- **Calculate** raw **Optimum Depth (OD)**: *Translating the **ROS** and desired coverage into an ideal number of unique items (SKUs) to stock for each style.*
- **Apply** Ideal Size Set (**ISS**): *To break down the **OD** into specific SKU-level quantities across sizes for each style.*
- **Allocate** planned revenue and adjust buy value: *To ensure the calculated buy quantities align with financial targets and strategic priorities.*
- **Generate** final **OD** and detailed buy plan: *Providing the actionable recommendations for purchasing.*

### 6. Sample Scenario

A denim jeans style has a projected **ROS** of 50 units/week and a target 4 weeks of supply. Its **ISS** is [S:1, M:2, L:2, XL:1].

1. The module calculates a target inventory of 200 units (50 units/week \* 4 weeks).
2. Given the **ISS** ratio of 6 total units (1+2+2+1), the module determines the **OD** and scales the quantities, e.g., for a total buy of 200 units, it calculates: S:33, M:67, L:67, XL:33 (approx).
3. If the planned revenue for denim dictates a slightly lower overall buy, the module might adjust these quantities proportionally to align with the financial target. **Key Takeaway:** This module translates sales forecasts into

concrete inventory recommendations, balancing sales potential with financial objectives by calculating the **OD** and precise buy quantities.

7. **Results** The main output is the detailed `otbStyleWiseBuy` plan, which includes:

- Final **Optimum Depth (OD)** per style/segment.
- Recommended buy quantities per style/SKU (broken down by size/color).
- Total buy value (in cost and retail) per style/segment.
- Analysis of actual vs. planned revenue and inventory levels.

## 8. Unusual Cases

- **Negative projected ROS**: Indicates an issue from the previous module or data, leading to incorrect or zero buy recommendations.
- **Very low ROS**: Can lead to disproportionately high **OD** if minimum buy quantities are enforced, or very small buys that are not operationally feasible.
- **Mismatch between planned revenue and calculated buy value**: If the bottom-up calculated buy value significantly deviates from top-down revenue targets, adjustments may be needed.
- **Missing Ideal Size Set (ISS)**: Without an **ISS**, the module cannot accurately break down the total buy quantity into individual SKU sizes, leading to incomplete recommendations.

9. **Next Steps** The detailed buy plan is then reviewed by relevant stakeholders (e.g., buyers, planners). Approved buy plans are used to generate purchase orders, ensuring the correct inventory levels are procured.

## 10. Common Questions

1. How is the **Optimum Depth (OD)** calculated and what factors influence it?
2. What is the relationship between **ROS**, **OD**, and buy quantity?
3. How are the **Ideal Size Set (ISS)** and **Optimum Width (OW)** considered in the calculation?
4. Can I manually adjust the calculated **OD** or buy quantities?
5. How does this module help prevent stockouts and overstock?

## 11. Quick Reference

- **Purpose**: Inventory optimization, buy plan generation.
- **Inputs**: Projected **ROS**, segmentation, **ISS**, planned revenue.
- **Outputs**: **OD**, buy quantities, buy value.

# 9. Appendix

## Full Parameter Reference

*A complete list of all configurable parameters, including those not exposed in `args` properties, along with their descriptions and default values, would be provided here.*

## Version History

Version	Date	Changes
1.0	2025-05-23	Initial release of the otbStyleWiseBuy documentation.

## Contact & Support Info

For questions, issues, or support regarding the otbStyleWiseBuy documentation or its underlying system, please contact:

- **Support Team:** support@yourcompany.com
- **Documentation Feedback:** docs@yourcompany.com
- **Internal Team:** Retail Planning Systems Team



# Tutorial: otbStyleWiseBuy

The otbStyleWiseBuy process is centrally managed by the Orchestrator (0), which is responsible for sequentially executing the Projected ROS Calculation (2) and Optimum Depth (OD) and Buy Value Calculation (3) modules. The entire process relies on prior validation performed by the Input Validator (1) to ensure data quality. The ROS calculation module (2) generates crucial intermediate data that is then consumed by the OD calculation module (3) to finalize buy quantities.

flowchart TD

```
A0["OTB Style Wise Buy Process Orchestrator"]
A1["OTB Style Wise Buy Input Validator"]
A2["Projected ROS (Rate of Sale) Calculation"]
A3["Optimum Depth (OD) and Buy Value Calculation"]
A0 -- "prerequisite" --> A1
A0 -- "executes" --> A2
A2 -- "data_flow" --> A3
A0 -- "executes" --> A3
```

## Chapters

# Master Report: otbStyleWiseBuy

---

## 1. Title & Metadata

- **Project Name:** otbStyleWiseBuy
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** Optimizes product assortment and inventory levels by intelligently predicting Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) based on historical sales performance and category attributes.

## 2. Executive Summary

The otbStyleWiseBuy merchandising algorithm is designed to revolutionize our buying and inventory management processes by providing data-driven recommendations for product assortment. This system leverages historical sales data and product attributes to calculate ideal inventory parameters, significantly reducing stockouts and overstock, while maximizing sales potential.

- **Key outcomes or KPIs:**
  - Improved **ROS** across product categories and individual SKUs.
  - Optimized **OD** for unique items per category, ensuring robust depth where demand exists.
  - Optimized **OW** for product categories/styles, focusing resources on high-performing segments.
  - Precise **ISS** recommendations for each SKU, reducing size-based stockouts and excess inventory.
  - Minimized inventory holding costs and capital tied up in slow-moving stock.
  - Enhanced customer satisfaction through better product availability.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Buying Teams, Inventory Planners, Finance Department, Supply Chain Management.

## 3. Architecture & Data Flow

### Diagram:

```
graph TD
 A[Raw Sales Data] --> B(Data Ingestion & Preprocessing);
 B --> C(Product Master Data);
 C --> D{Sales & Inventory Data Harmonization};
```

```

D --> E(Historical Inventory Data);
E --> F(Performance Analytics & ROS Calculation);
F --> G(Configuration Settings);
G --> H(Assortment Optimization Engine);
H --> I(Recommendation Generation & Output);
I --> J[Optimized Assortment Plan];
I --> K[Buy Recommendations];
I --> L[Inventory Adjustment Projections];
I --> M[Performance Dashboard Data];

subgraph "otbStyleWiseBuy Algorithm"
 B; C; D; E; F; G; H; I;
end

```

## Component Bullets:

- **Data Ingestion & Preprocessing:** Collects raw sales and inventory data from various sources, performing initial cleansing and validation.
- **Product Master Data:** Centralized repository of all product attributes, categories, and hierarchical structures.
- **Sales & Inventory Data Harmonization:** Integrates sales, inventory, and product master data, resolving discrepancies and ensuring data consistency.
- **Historical Inventory Data:** Provides context on past inventory levels, stockouts, and overstock positions.
- **Performance Analytics & ROS Calculation:** Computes Rate of Sale (ROS) for individual SKUs, styles, and categories, along with other performance metrics for Attribute Groups (AGs).
- **Configuration Settings:** Stores all tunable parameters, business rules, and thresholds (args.properties) that guide the optimization process.
- **Assortment Optimization Engine:** The core intelligence component that calculates OD, OW, and ISS based on performance data, configured rules, and historical patterns.
- **Recommendation Generation & Output:** Translates the optimized OD, OW, and ISS values into actionable buy plans, assortment layouts, and inventory adjustment projections, formatted for downstream consumption.

## High-level data sources:

- ERP System (Sales Orders, Inventory Levels, Returns)
- Product Information Management (PIM) System (SKU attributes, category hierarchy, pricing)
- Warehouse Management System (WMS) (Current stock levels, inbound shipments)
- Historical markdown and promotion data (optional, for advanced ROS adjustments)

## 4. Configuration & Data Dictionary

### Settings Table (args.properties):

Setting Key	Description	Default Value	
lookback_period_weeks	Number of weeks of historical data to consider for ROS calculation.	12	2
min_ros_threshold_item	Minimum ROS for an item to be considered for inclusion in OD.	0.5 units/wk	0
min_ros_threshold_category	Minimum aggregate ROS for a category to be considered for higher OW.	10.0 units/wk	1
od_optimization_strategy	Strategy for calculating OD (e.g., ROS_weighted, demand_curve).	ROS_weighted	0
ow_optimization_strategy	Strategy for calculating OW (e.g., performance_tiered, strategic_focus).	performance_tiered	5
iss_calculation_method	Method for deriving ISS (e.g., historical_sales_percent, pareto_size).	historical_sales_percent	1
inventory_buffer_percent	Safety stock percentage to add to optimal inventory levels.	0.10	0
new_product_default_od	Default OD value for newly introduced products without sales history.	3	5
new_product_default_iss	Default ISS distribution for new products (e.g., {"S":0.2, "M":0.3, ...}).	(common_dist)	-

### Shared Data Fields (common-data.json):

Field Name	Data Type	Description
SKU_ID	String	Unique identifier for each Stock Keeping Unit.
Product_ID	String	Identifier for a product style/model (parent of SKU).
Category_ID	String	Identifier for the product's primary category.
Sub_Category_ID	String	Identifier for the product's sub-category.
Style_ID	String	Identifier for a distinct style or aesthetic (often maps to OW).

Field Name	Data Type	Description
Attribute_Group	String	Key for the Attribute Group (AG) the SKU belongs to (e.g., "Color: Red").
Size	String	The specific size of the SKU (e.g., S, M, L, 30x32).
Brand	String	The brand of the product.
Sales_Units_Last_N_Weeks	Integer	Total units sold over the lookback_period_weeks.
Sales_Revenue_Last_N_Weeks	Decimal	Total revenue generated over the lookback_period_weeks.
Current_Inventory_Units	Integer	Current available inventory for the SKU.
In_Transit_Units	Integer	Units currently in transit or on order.
ROS_Units_Per_Week	Decimal	Calculated Rate of Sale (units/week).
Calculated_OD	Integer	Recommended Optimum Depth for the SKU's relevant grouping.
Calculated_OW	Integer	Recommended Optimum Width for the SKU's relevant grouping.
Calculated_ISS	JSON	Recommended Ideal Size Set distribution (e.g., {"S":0.2, "M":0.3}).

### Outputs Catalog:

- `optimized_assortment_plan.csv`: Comprehensive plan detailing recommended OD, OW, and ISS for each product category and style.
- `buy_recommendations.csv`: SKU-level recommendations for units to purchase, based on current inventory, ROS, and calculated optimal levels.
- `inventory_adjustments.json`: Suggested inventory transfers or markdown recommendations for under/overperforming SKUs.
- `performance_dashboard_data.json`: Aggregated data for visualizing key KPIs (ROS, OD/OW adherence) in business intelligence dashboards.
- `debug_log.txt`: Detailed logs of module execution, warnings, and errors.

## 5. End-to-End Flow

The `otbStyleWiseBuy` algorithm executes a sequential pipeline to transform raw data into actionable merchandising insights:

1. **Data Ingestion:** The process begins by ingesting raw sales transactions, current inventory snapshots, and comprehensive product master data from various source systems. This data is timestamped and stored in a staging area.
2. **Data Preprocessing & Harmonization:** The ingested raw data undergoes rigorous cleaning, validation, and transformation. Missing values are handled

- (e.g., imputation or flagging), data types are standardized, and disparate datasets (sales, inventory, product attributes) are joined using common keys like `SKU_ID` and `Category_ID`.
3. **Performance Analytics:** For each SKU, product, style, and category, the Rate of Sale (ROS) is calculated using the configured `lookback_period_weeks`. This involves summing sales units and dividing by the number of weeks. Other performance metrics, such as sell-through rates and sales velocity for Attribute Groups (AGs), are also computed.
  4. **Assortment Optimization:** This is the core computational phase. Using the calculated ROS and other performance metrics, alongside the settings defined in `args.properties`:
    - **Optimum Depth (OD)** is determined for each unique item/style, indicating the ideal number of specific SKUs to stock within a narrow category, often driven by individual SKU ROS thresholds and demand consistency.
    - **Optimum Width (OW)** is calculated for broader categories or styles, indicating the ideal number of different categories or styles to offer, based on aggregate category ROS, strategic importance, and overall market demand.
    - **Ideal Size Set (ISS)** is derived for each product style by analyzing historical sales data by size, ensuring the correct proportional allocation of inventory across available sizes to match customer demand patterns.
  5. **Recommendation Generation:** The calculated OD, OW, and ISS values are then translated into concrete, actionable recommendations. This includes suggested buy quantities for upcoming seasons, adjustments to current inventory holdings (e.g., transfer recommendations), and an overall optimal assortment plan.
  6. **Output & Reporting:** The final recommendations are formatted into various output files (`.csv`, `.json`) suitable for consumption by merchandising teams, buying departments, and business intelligence dashboards. Detailed logs are also generated to aid in debugging and performance monitoring.

## 6. Submodule Overview

### 1. Data Ingestion & Harmonization

- **Purpose:** To collect, clean, and consolidate raw sales, inventory, and product master data into a unified, consistent format for further processing.
- **Inputs & Outputs:**
  - **Inputs:** Raw sales transaction logs, current inventory snapshots, product master data CSVs/DB dumps.
  - **Outputs:** `harmonized_sales_inventory_product_data.csv` (a single, clean dataset with all relevant fields).
- **Core Logic Summary:**
  1. Read raw sales, inventory, and product data from specified sources.
  2. Perform schema validation and type conversion for all incoming fields.
  3. Join datasets on common keys (`SKU_ID`, `Product_ID`, `Category_ID`), handling any missing or mismatched entries gracefully (e.g., logging warnings, defaulting values).
  4. Cleanse data by handling nulls, duplicates, and applying basic business rules (e.g., removing negative sales).

## 2. Performance Analytics & ROS Calculation

- **Purpose:** To calculate the Rate of Sale (ROS) and other essential performance metrics at various granularity levels (SKU, product, category, Attribute Group).
- **Inputs & Outputs:**
  - **Inputs:** `harmonized_sales_inventory_product_data.csv`.
  - **Outputs:** `performance_metrics_data.csv` (includes `ROS_Units_Per_Week`, sell-through rates, and aggregated performance for AGs).
- **Core Logic Summary:**
  1. Group the harmonized data by `SKU_ID` and specified `lookback_period_weeks`.
  2. Calculate `ROS_Units_Per_Week` for each `SKU_ID` by dividing total sales units in the lookback period by the number of weeks.
  3. Aggregate sales and inventory data to higher levels (e.g., `Product_ID`, `Category_ID`, `Style_ID`, `Attribute_Group`) to calculate aggregate ROS and sell-through rates.
  4. Identify and flag potential outliers or zero-ROS products for special handling in downstream modules.

## 3. Assortment Optimization Engine

- **Purpose:** To determine the Optimum Depth (OD), Optimum Width (OW), and Ideal Size Set (ISS) for products and categories based on performance metrics and configuration settings.
- **Inputs & Outputs:**
  - **Inputs:** `performance_metrics_data.csv`, `args.properties` (configuration settings).
  - **Outputs:** `optimized_assortment_data.csv` (includes `Calculated_OD`, `Calculated_OW`, `Calculated_ISS` for each relevant product grouping).
- **Core Logic Summary:**
  1. **OD Calculation:** For each product/style, evaluate its `ROS_Units_Per_Week` against `min_ros_threshold_item` and apply the `od_optimization_strategy`. Products with high ROS and consistent demand are assigned a higher OD, while slow movers may be phased out or given minimal depth. New products may receive a `new_product_default_od`.
  2. **OW Calculation:** Analyze the aggregate `ROS_Units_Per_Week` for each `Category_ID` or `Style_ID`. Categories/styles exceeding `min_ros_threshold_category` are prioritized for higher OW, indicating more unique offerings. The `ow_optimization_strategy` guides whether this is purely performance-driven or includes strategic considerations.
  3. **ISS Calculation:** For each `Product_ID` (style), analyze the historical sales distribution across all Size attributes. Apply the `iss_calculation_method` (e.g., `historical_sales_percent` to allocate buy quantities proportionally, or `pareto_size` to focus on top-selling sizes). The output `Calculated_ISS` is a JSON object mapping sizes to their recommended percentage allocation.

## 4. Recommendation Generation & Output

- **Purpose:** To transform the calculated optimal values into concrete, actionable buy and inventory recommendations and format them for various downstream systems and reporting.
- **Inputs & Outputs:**
  - **Inputs:** `optimized_assortment_data.csv`, `Current_Inventory_Units` from harmonized data.
  - **Outputs:** `buy_recommendations.csv`, `optimized_assortment_plan.csv`, `inventory_adjustments.json`, `performance_dashboard_data.json`.
- **Core Logic Summary:**
  1. Consolidate `Calculated_OD`, `Calculated_OW`, and `Calculated_ISS` with current inventory levels and `ROS_Units_Per_Week`.
  2. Generate SKU-level buy recommendations by comparing current inventory + in-transit units against the calculated optimal inventory levels (derived from OD, OW, and applying `inventory_buffer_percent`).
  3. Format the comprehensive assortment plan, detailing the recommended mix of categories, styles, and depths.
  4. Create detailed `inventory_adjustments.json` for overstocked or underperforming items that may require markdowns or transfers.
  5. Prepare aggregated summary data for `performance_dashboard_data.json`, suitable for BI tool integration.

## 7. Worked Example

Let's consider a simple scenario for the "Women's Denim" category.

### Initial Data Snippet (Pre-Optimization):

SKU_ID	Product_ID	Category_ID	Style_ID	Size	Current_Inventory	Sales_Units
DEN001S	DENIM001	Women's Denim	Skinny_Fit	S	5	36
DEN001M	DENIM001	Women's Denim	Skinny_Fit	M	15	60
DEN001L	DENIM001	Women's Denim	Skinny_Fit	L	8	48
DEN002M	DENIM002	Women's Denim	Bootcut	M	3	6
DEN003S	DENIM003	Women's Denim	Straight_Leg	S	10	24

**Configuration Settings (args.properties):** - `lookback_period_weeks`: 12 - `min_ros_threshold_item`: 1.0 units/wk - `min_ros_threshold_category`: 20.0 units/wk (for the entire "Women's Denim" category) - `od_optimization_strategy`: `ROS_weighted` - `ow_optimization_strategy`: `performance_tiered` - `iss_calculation_method`: `historical_sales_percent` - `inventory_buffer_percent`: 0.10



## Pipeline Step-by-Step:

### 1. Performance Analytics (Module 2):

- **Product DENIM001 (Skinny\_Fit):** Total ROS = 3.0 (S) + 5.0 (M) + 4.0 (L) = 12.0 units/week.
- **Product DENIM002 (Bootcut):** Total ROS = 0.5 units/week.
- **Product DENIM003 (Straight\_Leg):** Total ROS = 2.0 units/week.
- **Category “Women’s Denim”:** Aggregate ROS = 12.0 + 0.5 + 2.0 = 14.5 units/week.

### 2. Assortment Optimization Engine (Module 3):

- **OW Calculation (for “Women’s Denim” category):**
  - Aggregate ROS (14.5) is *below* min\_ros\_threshold\_category (20.0). This indicates “Women’s Denim” might be a declining or less strategic category for new styles. The algorithm might recommend reducing OW (fewer distinct styles offered).
- **OD Calculation (per Style\_ID):**
  - **Skinny\_Fit (DENIM001):** Total ROS (12.0) is high. Each SKU (DEN001S, DEN001M, DEN001L) has individual ROS > min\_ros\_threshold\_item (1.0). The ROS\_weighted strategy suggests a strong OD, meaning we should stock a good depth of each size. Calculated\_OD for this product might be 5 units/size (base).
  - **Bootcut (DENIM002):** Total ROS (0.5) is *below* min\_ros\_threshold\_item (1.0). The algorithm will assign a Calculated\_OD of 0 or 1, recommending de-prioritization or phasing out this style.
  - **Straight\_Leg (DENIM003):** Total ROS (2.0) is above min\_ros\_threshold\_item. Calculated\_OD for this product might be 3 units/size (base).
- **ISS Calculation (per Product\_ID):**
  - **DENIM001 (Skinny\_Fit):**
    - S sales: 36 units (3.0 ROS)
    - M sales: 60 units (5.0 ROS)
    - L sales: 48 units (4.0 ROS)
    - Total sales for DENIM001: 144 units.
    - ISS: S: 36/144 = 25%; M: 60/144 = 41.7%; L: 48/144 = 33.3%.
    - Calculated\_ISS: {"S":0.25, "M":0.417, "L":0.333}

### Output Example (Simplified buy\_recommendations.csv):

Assuming Calculated\_OD for DENIM001 is 5 base units per size (before ISS proportions) and DENIM003 is 3 base units per size. Target inventory for DENIM001 = (5 \* 3 sizes) = 15 units, then distributed by ISS. Target inventory for DENIM003 = (3 \* 1 size shown) = 3 units, then distributed by ISS.

SKU_ID	Product_ID	ROS_Units_Per_Week	Current_Inventory	Calculated_OD	Calculated_ISS
DEN001S	DENIM001	3.0	5	-	-
DEN001M	DENIM001	5.0	15	-	-

SKU_ID	Product_ID	ROS_Units_Per_Week	Current_Inventory	Calculated_OD	Calculated_ISS
DEN001L	DENIM001	4.0	8	-	-
DEN002M	DENIM002	0.5	3	0 (phase out)	-
DEN003S	DENIM003	2.0	10	-	-

*Note: For DENIM001, even though its overall ROS is strong, the current inventory for M and L sizes is too high relative to the target inventory calculated by OD and ISS. This would lead to a buy recommendation of 0 and potentially an inventory adjustment recommendation to transfer or markdown excess. For DEN001S, a small buy is recommended.*

## 8. Debug Guide Highlights

- **Verify Input Data Integrity:**
  - **Problem:** Missing or malformed SKU\_ID, Category\_ID, or Sales\_Units in source data.
  - **Debug Step:** Check debug\_log.txt for Data Ingestion & Harmonization warnings/errors related to parsing or schema violations. Use data profiling tools to identify nulls or unexpected values.
- **Review ROS Calculation Accuracy:**
  - **Problem:** ROS\_Units\_Per\_Week values seem too high/low or inconsistent.
  - **Debug Step:** Confirm lookback\_period\_weeks in args.properties is set correctly. Manually calculate ROS for a few sample SKUs to cross-verify against the performance\_metrics\_data.csv output.
- **Check Configuration Thresholds:**
  - **Problem:** Output recommendations (OD, OW, ISS) are too aggressive (too few items recommended) or too conservative (too many).
  - **Debug Step:** Scrutinize min\_ros\_threshold\_item, min\_ros\_threshold\_category, inventory\_buffer\_percent, and the chosen\_optimization\_strategy settings in args.properties. Adjust these values to align with business objectives.
- **Trace Individual SKU Calculations:**
  - **Problem:** A specific SKU's recommendation appears incorrect or illogical.
  - **Debug Step:** Trace that SKU's data through each module's output (harmonized\_data, performance\_metrics\_data, optimized\_assortment\_data) to identify where the calculation might have diverged or if its input data was anomalous.
- **Performance Bottlenecks:**
  - **Problem:** The algorithm takes an excessively long time to run.
  - **Debug Step:** Analyze the debug\_log.txt for module execution times. Large datasets or complex\_optimization\_strategy choices can impact performance. Consider optimizing data reads/writes or scaling computational resources.

## 9. Common Pitfalls & Remedies

- **Pitfall 1: Outdated or Stale Data Inputs.**
  - **Problem:** The algorithm runs on data that isn't fresh, leading to recommendations based on old trends or inventory levels.
  - **Remedy:** Implement robust data refresh schedules (e.g., daily/weekly automated feeds). Establish alerts if data feeds are delayed or incomplete. Clearly communicate data freshness requirements to source system owners.
- **Pitfall 2: Over-Optimization Leading to Missed Opportunities or Strategic Gaps.**
  - **Problem:** Purely data-driven optimization might recommend phasing out a strategically important but currently low-ROS product, or overly narrowing an assortment.
  - **Remedy:** Introduce "strategic override" flags or categories in the product master data that prevent aggressive de-prioritization. Incorporate a minimum OW for core categories regardless of current ROS. Allow manual overrides for key decisions in the recommendation generation phase.
- **Pitfall 3: Not Accounting for Seasonality, Trends, or Promotions.**
  - **Problem:** ROS calculations might be skewed by seasonal spikes/dips or promotional activities if the `lookback_period_weeks` is too narrow or static.
  - **Remedy:** Integrate more sophisticated forecasting models that account for seasonality and trends. Dynamically adjust `lookback_period_weeks` or apply weighted averages that prioritize recent sales. Introduce flags for promotional periods to adjust ROS calculations accordingly.
- **Pitfall 4: "Cold Start" Problem for New Products.**
  - **Problem:** New SKUs or styles have no historical sales data, making it impossible to calculate ROS, OD, OW, or ISS.
  - **Remedy:** Utilize `new_product_default_od` and `new_product_default_iss` settings. Assign new products to similar Attribute Groups or Categories for which data exists, allowing heuristics to be applied. Require manual input for initial buy quantities for truly novel items.

## 10. FAQ

- **Q: How often does the `otbStyleWiseBuy` algorithm run?**
  - **A:** The algorithm is typically scheduled to run weekly or bi-weekly, depending on the business's inventory replenishment cycle and market volatility. This ensures recommendations are timely and reflect recent sales performance.
- **Q: Can I manually override the buy recommendations generated by the algorithm?**
  - **A:** Yes, the outputs are designed to be recommendations. Merchandising and Buying teams can review, adjust, and override specific recommendations based on their market intuition, strategic priorities, or external factors not captured by the algorithm. However, any significant overrides should be documented.
- **Q: What happens if a product has zero Rate of Sale (ROS) in the lookback period?**
  - **A:** Products with zero ROS are typically flagged for review. Depending on the `od_optimization_strategy`, they may be recommended for deep

markdown, clearance, or removal from the assortment (assigned an OD of 0 or 1). For new products, `new_product_default_od` and `new_product_default_iss` are applied.

- **Q: How does the algorithm handle returns or damaged inventory?**
  - **A:** The Data Ingestion & Preprocessing module is configured to account for returns (by deducting them from sales) and damaged inventory (by excluding them from `Current_Inventory_Units`). This ensures ROS and `Current_Inventory` accurately reflect sellable units.
- **Q: Is there a way to prioritize certain brands or categories regardless of their current ROS?**
  - **A:** Yes, the `ow_optimization_strategy` can be configured to include a “strategic focus” tier. This allows for manual designation of brands or categories as strategic, ensuring they maintain a minimum OW even if their ROS is temporarily low. This is managed via `args.properties` or an external strategic product list.

## 11. Quick Reference Cheat-Sheet

Term / Abbreviation	Definition	Key Takeaway
OD	<b>Optimum Depth:</b> Ideal number of unique items (SKUs) to stock in a category or display.	How many <i>different versions</i> of a similar item should we carry?
OW	<b>Optimum Width:</b> Ideal number of different categories or styles to offer.	How many <i>distinct categories/styles</i> should our store/assortment cover?
ISS	<b>Ideal Size Set:</b> Best combination of sizes to maximize sales and minimize stockouts.	Which sizes should we stock, and in what proportions?
ROS	<b>Rate of Sale:</b> Speed at which products are sold (e.g., units/week).	How fast is this product/category selling? Higher ROS usually means higher OD/OW.
AG	<b>Attribute Group:</b> A set of product attributes grouped for analysis (e.g., color, size, brand).	Helps analyze performance beyond just SKUs (e.g., “all black dresses”).
<code>args.properties</code>	Configuration file for algorithm settings and thresholds.	The “levers” to adjust the algorithm’s behavior.
Core Principle	Maximize sales potential, minimize inventory costs, enhance customer satisfaction.	Data-driven merchandising for smarter buys and balanced assortments.

<b>Term / Abbreviation</b>	<b>Definition</b>	<b>Key Takeaway</b>
<b>Key Metrics</b>	ROS, Sell-through Rate, Inventory Turnover, Stockout Rate.	Key performance indicators impacted and optimized by the algorithm.
<b>Primary Output</b>	buy_recommendations.csv, optimized_assortment_plan.csv.	Actionable lists for Merchandising and Buying teams.

# Chapter 1: OtbDataInputModule

Last Updated: 2025-05-23

## 1. Business Objective

This step gathers all the necessary initial business data to accurately calculate and manage open-to-buy (OTB) plans.

**Why This Matters:** This foundational step ensures that all subsequent inventory planning and buying decisions are based on accurate, up-to-date business rules and performance expectations. It helps retailers avoid costly stockouts or overstock situations, leading to better sales, optimized inventory investment, and improved customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Sell-Through Rate (ROS) Adjustments:** We need specific adjustments or overrides for how quickly certain products are expected to sell.
  - *Why we need it:* This allows us to fine-tune sales forecasts for specific categories, price points, or promotions, making our OTB plans more realistic and responsive to market changes.
- **Return Percentages:** We need to know the typical percentage of products that customers return for different categories or product types.
  - *Why we need it:* Accounting for returns ensures our net sales figures are accurate, preventing overbuying due to returned inventory and helping us manage our re-stocking needs efficiently.

### Product Data

- **Optimum Width (OW) Recommendations:** We need the ideal number of different categories or styles that should be offered for a given store or product group.
  - *Why we need it:* This guides our assortment planning, ensuring we offer enough variety to attract customers without overwhelming them or spreading inventory too thin.

- **Optimum Depth (OD) Ranges:** We need the suggested range for the ideal number of unique items (SKUs) to stock within a category or style.
  - *Why we need it:* This helps us determine how many color options or specific product variations to buy, ensuring we have sufficient depth for popular items without overstocking slower sellers.
- **In-Season Display Layouts:** We need information on how products are typically arranged and displayed in stores during the selling season.
  - *Why we need it:* This helps us understand the visual merchandising requirements and display capacities, influencing the quantities we need to buy to fill store displays effectively.

## Operational Data

- **Minimum Display Requirements:** We need to know the smallest quantity of a product that must be kept on display in a store.
  - *Why we need it:* This ensures stores always meet visual merchandising standards, providing an appealing shopping experience and influencing initial stock levels.
- **Minimum Order Quantities:** We need the smallest allowable quantity we can order from our suppliers for specific products or categories.
  - *Why we need it:* This ensures that our purchase orders are viable and meet vendor terms, avoiding inefficient small orders and ensuring we can actually acquire the necessary inventory.
- **Ideal Size Set (ISS) Quantities:** We need the recommended combination of sizes to maximize sales and minimize stockouts for specific product styles.
  - *Why we need it:* This ensures we buy the right proportion of sizes (e.g., more mediums, fewer extra-smalls), matching customer demand and reducing the risk of being out of popular sizes while overstocked on unpopular ones.
- **Micro-Buy Benchmarks:** We need standards or guidelines for very small, targeted purchases, often used for testing new items or filling specific gaps.
  - *Why we need it:* This helps guide agile, smaller buying decisions, allowing us to respond quickly to trends or unexpected demand without committing to large volumes.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>Sell-Through Rate Granularity</b>	Category Level	Determines at what level of detail (e.g., entire category, specific product sub-category, or even price bucket) the system applies sales rate adjustments.	When you need more precise sales performance insights, such as during a targeted promotion on a specific price point, or if you notice significant sales variances within a broad category.
<b>Default Return Rate</b>	10%	The fallback percentage of	If your overall return trends change

Setting Name	Default Value	Description	When to Change
		returns to apply if no specific return data is available for a given product or category.	significantly across your business, or if you introduce new product lines with inherently different return profiles (e.g., online-only items vs. in-store items).

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Input Data: OW, STR Overrides, Returns, Min Displays, Planograms, Depth Ranges, Min Orders, Size Sets] --> B[Data Gathering & Centralization]

B --> C[Central OTB Data Store]

1. Start with → Raw, disparate data points from various sources, such as ideal category recommendations, specific sales velocity adjustments, and operational constraints like minimum display items.
2. Then → The system **retrieves** all these diverse pieces of information from their respective sources.
3. Next → The system **consolidates** and **structures** this information into a unified format.
4. Finally → A comprehensive “OTB Data Store” is created, holding all necessary initial data, ready for the next stages of Open-to-Buy calculations.

## 5. Step-by-Step Actions

1. **Gather** strategic assortment recommendations
  - What we do: *We collect the system's recommendations for the ideal number of different categories or styles (Optimum Width) that should be offered for each store and product group.*
  - Why we do it: *This ensures our product mix aligns with overarching business strategies and customer demand, helping us decide “how many types of dresses” to stock.*
2. **Incorporate** sales performance adjustments
  - What we do: *We pull in specific adjustments for how quickly products are expected to sell (Sell-Through Rate overrides) and the average percentage of items that customers return.*
  - Why we do it: *This makes our sales forecasts more realistic, accounting for specific market conditions, promotions, or inherent product return tendencies, directly impacting “how much to buy for expected sales.”*
3. **Load** crucial merchandising and inventory guidelines
  - What we do: *We bring in details like the minimum number of units that must be kept on display, how products are physically arranged in stores (planograms), and the recommended range for the number of unique items (SKUs) within a style (Optimum Depth).*



- Why we do it: *This ensures OTB calculations consider the physical space available, visual presentation needs, and the necessary variety of products in stores, answering “how many of each color or size to put on the shelf.”*
4. **Capture** ordering and sizing constraints
- What we do: *We input the minimum order quantities required by vendors and the Ideal Size Set (ISS) quantities, which is the best combination of sizes to stock for a specific style.*
  - Why we do it: *This ensures we place viable orders that meet supplier requirements and that we buy the right proportion of sizes to match customer demand, minimizing stockouts and maximizing sales for each style.*

## 6. Sample Scenario

Let’s walk through a real example:

### Starting Point

- A retail chain is planning for the upcoming Fall season’s denim collection.
- The system’s **Optimum Width (OW)** recommendation for a specific boutique store is 4 distinct denim styles.
- An **Sell-Through Rate (ROS) override** for “premium dark wash jeans” has been set 15% higher than usual due to a planned marketing campaign.
- Historical data shows **returns for denim** average 10%.
- The **minimum display requirement** for jeans is 8 units per style.
- The **Ideal Size Set (ISS)** for a popular slim-fit jean is 2XS:1, S:2, M:3, L:2, XL:1.

### What Happens

1. First, the system **retrieves** the OW recommendation of “4 distinct denim styles” for the boutique store, indicating the desired variety.
2. Then, it **fetches** the “15% higher STR override” for premium dark wash jeans, which will be factored into the sales forecast for those specific items.
3. Next, the “10% return rate” for denim is **loaded**, ensuring that the amount of inventory needed accounts for customer returns, preventing accidental overbuying.
4. Concurrently, the system **pulls in** the “minimum display requirement of 8 units per style,” which helps establish baseline inventory needs for visual merchandising.
5. Finally, the specific Ideal Size Set (ISS) quantities (e.g., 2XS:1, S:2) for the popular slim-fit jean are **recorded**, informing the precise breakdown of sizes for any order of that style.

### Final Outcome

All these distinct pieces of information—from strategic assortment goals to sales adjustments, return expectations, and operational display rules—are now consolidated and available in a central data store. This unified data set provides a

robust and accurate foundation for the OTB calculation module to determine the precise inventory quantities needed for the Fall denim collection, ensuring the buying plan is both strategic and operationally feasible.

**Key Takeaway:** This step successfully aggregates diverse strategic and operational data into a single, comprehensive source, enabling subsequent OTB calculations to be precise and aligned with business realities.

## 7. Results

This step produces:

### Main Outputs

- **Centralized OTB Data Store:** A comprehensive, unified repository that holds all the initial business rules, strategic targets, and operational constraints essential for subsequent Open-to-Buy calculations.
  - *What it tells us:* This is the single source of truth for all base assumptions and guidelines for the current OTB planning cycle.
- **Initialized Business Rules & Benchmarks:** All specific return percentages, Sell-Through Rate (ROS) adjustments, display minimums, Ideal Size Set (ISS) quantities, and Optimum Width (OW)/Optimum Depth (OD) ranges are loaded and ready for use.
  - *What it tells us:* It confirms that all critical, real-world factors affecting buying decisions have been accounted for from the very beginning.

### Business Reports

- **Input Data Audit Report:** A detailed report used by planners and data analysts to verify that all necessary initial data points have been loaded correctly and consistently across categories and stores.
  - *Who uses it and why:* Data teams and planning managers use this to ensure data integrity and catch any discrepancies before calculations begin, preventing errors in the OTB plan.
- **Strategic Assumption Summary:** A high-level overview of the key strategic inputs (like OW targets and major STR overrides) being used for the OTB cycle.
  - *Who uses it and why:* Merchandising and executive teams use this to review and confirm that the planning process is aligned with broader business strategies and targets.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed	The system might use default values for missing data, or it might halt the process, indicating an error.	Investigate the source of the missing data (e.g., sales forecasting system, merchandising strategy document) and arrange for it to be

Situation	What Happens	What You Should Do
Optimum Width (OW) recommendation missing for a new category	The system might fall back to a generic default OW value for all new categories, which may not be ideal.	provided. Rerun the module once data is available.  Consult with the merchandising or category planning team to define an appropriate OW for the new category and input it manually if necessary.
Sell-Through Rate (ROS) override not found for a specific product group	The system will likely use the broader category's ROS or a general default rate, which might not reflect specific sales potential.	Verify if a specific override was intended for that product group. If so, work with the planning team to enter the correct override.
Incorrect return percentage loaded for a category	OTB calculations will be skewed, potentially leading to either overbuying (if returns are under-reported) or underbuying (if returns are over-reported).	Correct the input data for the affected category and re-run the OTB calculation to ensure accurate inventory planning.

## 9. Next Steps

### Before This Step

- **Strategic Planning & Forecasting:** We need the foundational business strategies (e.g., target categories, desired growth rates) and initial sales forecasts to inform data like Optimum Width and future Sell-Through Rate adjustments.
  - *What we need from it:* The overarching vision for the product assortment and preliminary sales expectations.
- **Historical Performance Analysis:** We need past sales data, return rates, and inventory performance to establish benchmarks and identify areas for improvement or specific overrides.
  - *What we need from it:* The actual performance insights that inform realistic inputs like return percentages and historical Sell-Through Rates.

### After This Step

- **OTB Calculation Module:** This module consumes the fully initialized OTB data to perform complex calculations for optimal inventory levels, considering sales, returns, and lead times.
  - *What they need from us:* A complete and accurate set of all necessary input parameters to begin calculating the actual open-to-buy budget.
- **Assortment Planning & Buy Placement:** These activities use the Optimum Width (OW), Optimum Depth (OD), and Ideal Size Set (ISS) data to guide the selection of specific products and the quantity to order from vendors.
  - *What they need from us:* The fundamental guidelines for shaping the product offering and executing precise purchase orders.

## 10. Common Questions

Q: Why is this step needed if we already have sales data from last season? A: While last season's sales data is crucial, this step adds vital strategic and operational layers. It incorporates future-looking insights like ideal category widths (OW), adjusted sales rates for upcoming promotions (ROS overrides), and physical store requirements like minimum displays. It turns raw historical data into actionable, forward-looking guidance for buying.

Q: What happens if some of the input data, like return percentages, is missing or inaccurate? A: If critical data is missing, the system will either stop and request it, or it will use default values. Inaccurate data will lead to flawed OTB calculations, potentially causing you to buy too much (if returns are underestimated) or too little (if sales rates are overestimated), directly impacting profitability and customer satisfaction.

Q: How often is this input data refreshed or updated? A: Most of this data is typically updated at the beginning of each planning cycle, like a new season or quarter, as part of the strategic review. Some data, such as in-season planograms or specific Sell-Through Rate overrides for promotions, might be updated more frequently as market conditions or merchandising plans evolve.

Q: Who is responsible for providing all this diverse input data? A: Different teams contribute: Merchandising provides strategic targets like Optimum Width and Optimum Depth; Planning teams often provide Sell-Through Rate overrides and return percentages; and Store Operations or Visual Merchandising provides details like minimum display requirements and in-season planograms.

Q: How does getting this data right at the start benefit my store's performance? A: Accurate input data from this module is the backbone of effective Open-to-Buy. It ensures you invest precisely in the right products, in the right quantities, and with the right mix of sizes. This translates directly to fewer stockouts, less excess inventory, higher sales velocity, and ultimately, better profit margins for your store.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To gather and centralize all critical business-specific input data for accurate Open-to-Buy (OTB) calculations.
- **Main Input:** Strategic targets (Optimum Width, Optimum Depth ranges, Ideal Size Set), sales performance adjustments (Sell-Through Rate overrides, return percentages), and operational constraints (minimum display requirements, minimum order quantities, micro-buy benchmarks).
- **Main Output:** A complete, ready-to-use central OTB data store.
- **Warning Signs:** Unexplained discrepancies in inventory levels, OTB calculations that don't seem to reflect market reality, or a mismatch between planned and actual display quantities.
- **Key Settings:** Sell-Through Rate Granularity, Default Return Rate.

---

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.



# Chapter 2: OtbStrComputeModule

Last Updated: 2025-05-23

## 1. Business Objective

This module computes the Sell-Through Rate (STR) at various levels of detail, from broad categories down to specific product segments, by factoring in planned sales quantities, expected returns, and manual adjustments.

**Why This Matters:** Accurate Sell-Through Rates are fundamental for effective Open-to-Buy (OTB) planning. By precisely calculating how quickly products are expected to sell, this module helps retailers optimize inventory investment, minimize stockouts on popular items, reduce excess inventory on slower movers, and ultimately improve overall sales and profitability.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Planned Inventory Allocations (from OW module):** We need the quantities of products planned for purchase and allocation across different selling periods (Full Price and EOSS).
  - *Why we need it:* This provides the baseline sales volumes that are expected to be sold, forming the core of our Sell-Through Rate calculation.
- **Return Percentages:** We need the typical percentage of products that customers return, categorized by store and product type.
  - *Why we need it:* Accounting for returns ensures our net sales figures are accurate, preventing us from overbuying due to returned inventory and ensuring we have enough stock for actual customer purchases.
- **Manual Sell-Through Rate (STR) Overrides:** We need any specific adjustments or overrides for how quickly certain products are expected to sell, provided by planners.
  - *Why we need it:* This allows us to fine-tune sales forecasts for specific categories, price points, or promotions, making our OTB plans more realistic and responsive to market changes beyond historical data.

## Product Data

- **Product Hierarchy Data:** We need the definitions for categories, subcategories, and price buckets.
  - *Why we need it:* This structural information allows us to calculate and apply Sell-Through Rates at the correct level of detail (e.g., for all dresses, or just for premium evening dresses in a specific price range).
- **Attribute Group (AG) Definitions:** We need the specific attributes (like color, material, style) grouped together for product identification and analysis.
  - *Why we need it:* This ensures that product groupings for STR calculations are consistent and meaningful, allowing for accurate analysis of sales performance by relevant product characteristics.

## Operational Data

- **Selling Period Definitions:** We need clear definitions for what constitutes a “Full Price” selling period versus an “End-of-Season Sale (EOSS)” period.
  - *Why we need it:* Differentiating between these periods is crucial because sales velocities and inventory plans can differ significantly during regular season versus clearance events.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>Sell-Through Rate Granularity</b>	Store-Category-Subcategory-Price Bucket	Determines at what level of detail (e.g., an entire category, specific product sub-category, or even price bucket) the system calculates and applies Sell-Through Rates.	When you need more precise sales performance insights, such as during a targeted promotion on a specific price point, or if you notice significant sales variances within a broad category that require more granular tracking.

*Note: See More Details for additional settings.*

## 4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Planned Inventory Quantities & Return Percentages] --> B{Aggregate & Compute Base STR}

B --> C{Apply Manual STR Overrides}  
C --> D[Final Computed Sell-Through Rates]

1. Start with → Planned inventory quantities (from Optimum Width calculations) and pre-defined return percentages.
2. Then → The module **aggregates** quantities for Full Price and End-of-Season Sales (EOSS) periods, and **computes** a base Sell-Through Rate, adjusting for expected returns.
3. Next → The system **applies** any specific manual Sell-Through Rate overrides provided by planners, adjusting the computed rates as needed.
4. Finally → A comprehensive set of final “Computed Sell-Through Rates” is generated, ready for further Open-to-Buy calculations.

## 5. Step-by-Step Actions

1. **Aggregate** planned sales quantities
  - What we do: *We gather and combine the planned sales quantities, distinguishing between those expected to sell at full price and those expected to sell during End-of-Season Sales (EOSS), at the specified level of detail (e.g., by category or subcategory).*
  - Why we do it: *This establishes the total expected sales volume for each period, which is essential for determining how quickly products are expected to sell.*
2. **Calculate** the base Sell-Through Rate (STR)
  - What we do: *We determine an initial Sell-Through Rate by comparing the Full Price sales quantity to the total planned quantity, factoring in the expected return percentages.*
  - Why we do it: *This gives us a foundational understanding of sales velocity, adjusted for customer returns, ensuring our inventory needs are realistic.*
3. **Extend** STR to detailed levels
  - What we do: *If the Sell-Through Rate was calculated at a higher, aggregated level (e.g., for an entire category), we apply that rate uniformly down to more granular levels, such as specific subcategories and price buckets within that category.*
  - Why we do it: *This ensures that even if we calculate STR broadly, the resulting rate can be used consistently across all detailed product segments within that grouping.*
4. **Apply** manual overrides
  - What we do: *We check for and incorporate any specific manual Sell-Through Rate adjustments provided by buying or planning teams for particular products or categories.*
  - Why we do it: *This crucial step allows for human intelligence and strategic insights (like planned promotions or anticipated trends) to fine-tune the system’s calculated rates, making them more accurate and actionable.*

## 6. Sample Scenario

Let’s walk through a real example:



## Starting Point

- The “Women’s Dresses” category at Store 101 has a planned Full Price sales quantity of 500 units and an EOSS sales quantity of 200 units, as determined by the Optimum Width (OW) calculation.
- The general **return percentage** for dresses at Store 101 is 15%.
- A **manual Sell-Through Rate override** of 60% has been entered specifically for “Women’s Formal Dresses - Premium Price Bucket” due to an upcoming marketing campaign.
- The **Sell-Through Rate Granularity** for this calculation is set to “Store-Category-Subcategory-Price Bucket”.

## What Happens

1. First, the system **aggregates** the planned Full Price (500 units) and EOSS (200 units) quantities for “Women’s Dresses” at Store 101.
2. Then, it **calculates** the base Sell-Through Rate for the entire “Women’s Dresses” category. The total planned quantity is 700 units (500 + 200). Adjusting for returns, the quantity that needs to be sold is  $700 / (1 - 0.15) = 700 / 0.85 = 823.5$  units. The base STR for the category is therefore  $500 \text{ (Full Price Qty)} / 823.5 \text{ (Adjusted Total Qty)} = 60.7\%$ .
3. Next, the system **extends** this computed 60.7% STR to all subcategories and price buckets within “Women’s Dresses” at Store 101.
4. Finally, the system **applies** the manual override: for “Women’s Formal Dresses - Premium Price Bucket” at Store 101, the calculated STR of 60.7% is **overridden** and replaced with the manually input 60%.

## Final Outcome

The module produces a detailed set of Sell-Through Rates for Store 101, including a 60% STR for “Women’s Formal Dresses - Premium Price Bucket” and a 60.7% STR for other dress segments. These specific rates are now available for subsequent inventory and OTB calculations, ensuring purchasing decisions reflect both systemic analysis and strategic overrides.

**Key Takeaway:** This step ensures that sales velocity predictions are comprehensive, incorporating both system-calculated values based on planned quantities and return expectations, as well as critical manual adjustments for specific strategic initiatives.

## 7. Results

This step produces:

### Main Outputs

- **Computed Sell-Through Rates (STR) by Store-Category-Subcategory-Price Bucket:** A precise rate for how quickly products are expected to sell at the most granular level.
  - *What it tells us:* This is the final, refined sales velocity target for each specific product segment, directly informing inventory planning and replenishment decisions.

- **Calculated Full Price and EOSS Quantities:** The aggregated planned sales volumes for both regular and clearance periods for each product grouping.
  - *What it tells us:* This provides the underlying sales volume data that supports the computed Sell-Through Rates, offering transparency into how the rates were derived.

## Business Reports

- **Sell-Through Rate Computation Report:** A detailed report listing the computed Sell-Through Rates, their contributing planned quantities (Full Price and EOSS), and the applied return percentages for each level of detail.
  - *Who uses it and why:* Merchandising and planning teams use this to review and validate the calculated sales velocities, ensuring they align with business expectations and historical performance.
- **STR Override Audit Report:** A report highlighting all instances where manual Sell-Through Rate overrides were applied, showing the original computed rate and the final overridden value.
  - *Who uses it and why:* Planners and auditors use this to track and verify specific strategic adjustments, ensuring accountability and understanding the impact of manual interventions on the OTB plan.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed for planned quantities	The system might use default quantities (e.g., zero), or it might halt the process, indicating an error.	Investigate the source of the missing “Planned Inventory Allocations” (likely the Optimum Width module) and arrange for the data to be provided. Rerun the module once data is available.
Zero planned sales quantity at a specific rollup level	The computed Sell-Through Rate for that specific level might be zero or undefined, potentially skewing subsequent calculations.	Review the input data for that segment. If it’s a new or non-selling item, this might be expected. If not, verify why no sales quantity was planned and adjust the inputs or add a manual override.
Invalid return percentage (e.g., negative or >100%)	The calculation (1 - Return%) will yield an incorrect result, leading to highly inaccurate Sell-Through Rates.	Correct the input return percentage data. Return rates must be between 0% and 100%. Re-run the module after correction.
Conflicting manual STR overrides	If multiple overrides are entered for the same product segment, the system will apply the last one processed, potentially ignoring others.	Establish clear guidelines for manual override submission. Regularly review the “STR Override Audit Report” to identify and resolve any conflicting entries by consolidating them into a single, accurate override.

## 9. Next Steps

### Before This Step

- **OTB Data Input Module:** This module gathers all foundational business rules, such as general return percentages, and initial STR adjustments.
  - *What we need from it:* The base return rates and any broad-level sales velocity adjustments that apply across the business.
- **Optimum Width (OW) Calculation Module:** This module determines the ideal number of categories and styles, and importantly, provides the initial planned inventory quantities by period.
  - *What we need from it:* The planned sales volumes and inventory allocations for Full Price and EOSS periods that this module uses as its primary sales input.

### After This Step

- **OTB Calculation Module:** This module consumes the fully computed Sell-Through Rates to perform complex calculations for optimal inventory levels, considering sales, returns, and lead times.
  - *What they need from us:* A complete and accurate set of final Sell-Through Rates across all relevant product segments, which are critical for determining the actual open-to-buy budget.
- **Inventory Allocation Module:** This module uses the derived Sell-Through Rates to guide the distribution of products to stores, ensuring the right products are in the right place at the right time.
  - *What they need from us:* Precise Sell-Through Rates to inform where and how much inventory should be sent to maximize sales and minimize holding costs.

## 10. Common Questions

Q: Why is calculating Sell-Through Rate (STR) so important for my store? A: STR directly tells you how fast your products are selling. An accurate STR helps you avoid two major problems: buying too much (leading to costly excess inventory and markdowns) or buying too little (leading to missed sales opportunities and unhappy customers). It ensures you have the right amount of product at the right time.

Q: How do manual STR overrides work, and when should I use them? A: Manual overrides allow you, the planner, to input a specific Sell-Through Rate for a product or category, overriding the system's calculation. You should use them when you have specific strategic insights not captured by historical data, such as a major marketing campaign, a new product launch with anticipated high demand, or a known market shift.

Q: What happens if our actual return percentages are higher or lower than what was initially used in the calculation? A: If actual returns are higher than predicted, you might have overbought, leading to excess inventory. If they are lower, you might have underbought, risking stockouts. It's crucial to regularly review and update return percentages to ensure the STR calculation remains accurate and responsive to real-world customer behavior.

Q: How often is the Sell-Through Rate re-calculated or updated? A: Typically, a full recalculation might happen at the start of each planning cycle (e.g., seasonally or quarterly). However, manual overrides can be applied anytime to adjust specific rates based on in-season performance, promotions, or unexpected market changes, providing flexibility.

Q: Who is responsible for ensuring the accuracy of the STR data? A: While the system performs the calculations, merchandising and planning teams are primarily responsible for providing accurate input data (like planned quantities and return percentages) and for applying thoughtful manual overrides. Collaboration between these teams is key to achieving optimal results.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To compute precise Sell-Through Rates (STR) for inventory planning, integrating planned sales, returns, and strategic overrides.
- **Main Input:** Planned Inventory Allocations (from OW), Return Percentages, Manual Sell-Through Rate Overrides, Product Hierarchy, and Selling Period Definitions.
- **Main Output:** Final Computed Sell-Through Rates (STR) by Store-Category-Subcategory-Price Bucket.
- **Warning Signs:** Unexpectedly high or low OTB quantities in subsequent steps, frequent stockouts or excessive overstock, or an STR that doesn't align with observed sales patterns.
- **Key Settings:** Sell-Through Rate Granularity.

---

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.

---

Generated by [AI Codebase Knowledge Builder](#)

# Chapter 3: OtbMoqAdjustmentModule

Last Updated: 2025-05-23

## 1. Business Objective

This module ensures that all calculated inventory buy quantities adhere to Minimum Order Quantity (MOQ) constraints set by suppliers, by intelligently aggregating and redistributing quantities across different product groupings.

**Why This Matters:** This step is crucial for efficient purchasing. It prevents the ordering of uneconomically small quantities that suppliers won't accept, ensures we maximize our buying power by meeting MOQ thresholds, and ultimately helps maintain an optimal balance between variety (Optimum Width) and sufficient stock levels (Optimum Depth) in stores, leading to better sales and reduced procurement costs.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Planned Inventory Quantities:** We need the initial calculated buy quantities for each product, typically derived from the Optimum Width (OW) module, representing the total desired inventory based on sales forecasts.
  - *Why we need it:* These are the starting point for our purchases, representing how much product we ideally want to stock to meet anticipated demand.
- **Computed Sell-Through Rates (STR):** We need the calculated speed at which products are expected to sell, typically provided by the OtbStrComputeModule.
  - *Why we need it:* This rate is crucial for converting the initial financial "Open-to-Buy" into specific physical unit quantities that need to be purchased.

### Product Data

- **Attribute Group (AG) Definitions:** We need the precise definitions of how products are grouped by attributes like category, subcategory, price bucket, and product theme.
  - *Why we need it:* These definitions are essential for aggregating product quantities correctly and consistently across different levels for MOQ checks and adjustments.

- **Ideal Size Set (ISS) Quantities:** We need the recommended combination of sizes for specific product styles to maximize sales and minimize stockouts.
  - *Why we need it:* While this module doesn't directly adjust size sets, it uses this information when preparing adjusted quantity outputs, ensuring that the adjusted buys still consider the optimal size mix.

## Operational Data

- **Minimum Order Quantities (MOQ):** We need the smallest allowable quantity that can be ordered from suppliers for specific products or categories.
  - *Why we need it:* This is the core constraint this module addresses; ensuring all purchase orders meet these supplier requirements to avoid logistical issues and penalties.
- **Core Bestseller Designations:** We need to know which products are designated as core bestsellers or evergreen styles.
  - *Why we need it:* This designation influences how Optimum Width (OD) is adjusted for these high-performing items, ensuring they maintain sufficient depth.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>MOQ Adjustment Rollup Order</b>	Price Bucket → Subcategory → Category	This defines the sequence in which the system attempts to aggregate and adjust quantities to meet Minimum Order Quantity (MOQ) requirements. It starts at the most granular level and rolls up.	While often fixed based on business logic, this might be reviewed if your product hierarchy changes significantly, or if specific product lines consistently fail to meet MOQs even after aggregation, suggesting a different rollup strategy might be more effective.

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

```

 A[Initial Buy Quantities & Sell-Through Rates] --> B[Aggregate & Compute Base Quantities]
 B --> C[Roll Up & Adjust Quantities (Price Bucket Level)]
 C --> D[Roll Up & Adjust Quantities (Subcategory Level)]

```

D --> E{Roll Up & Adjust Quantities (Category Level)}  
E --> F[MOQ-Adjusted Buy Quantities & Unadjusted Records]

1. Start with → The initial calculated buy quantities for each product and their corresponding Sell-Through Rates (STRs).
2. Then → The module **converts** initial buy quantities from financial OTB to physical units using the STRs and **aggregates** them by the most granular product grouping (Price Bucket).
3. Next → The system **rolls up** quantities: if a product group's total quantity falls below its Minimum Order Quantity (MOQ), it attempts to combine these quantities with other related items at the next higher level (e.g., Subcategory), redistributing the quantity to groups that *can* meet MOQ. This process is repeated from Price Bucket to Subcategory, and then to Category.
4. Finally → A refined set of "MOQ-Adjusted Buy Quantities" is generated, along with records of any quantities that could not be adjusted to meet MOQ.

## 5. Step-by-Step Actions

1. **Compute** initial buy quantities in units
  - What we do: *We take the planned financial open-to-buy (OTB) amounts and divide them by the corresponding Sell-Through Rate (STR) to determine the actual number of units to be purchased for each product. This also accounts for Ideal Size Set (ISS) and initial width planning.*
  - Why we do it: *This gives us the baseline physical quantities that need to be acquired before applying any MOQ constraints.*
2. **Aggregate** data by Price Bucket
  - What we do: *We combine these initial unit quantities across all products within the same "Price Bucket" (e.g., all jeans priced \$50-\$75) and store them for the upcoming MOQ checks.*
  - Why we do it: *This establishes the most granular level for initial MOQ evaluation, as suppliers often have MOQs that apply across specific product price ranges.*
3. **Roll up and Adjust** quantities at Price Bucket, Subcategory, and Category levels
  - What we do: *For each product grouping (starting at Price Bucket, then moving up to Subcategory, and finally to Category), the system checks if the aggregated quantity meets the Minimum Order Quantity (MOQ) for that group.*
  - Why we do it: *If the quantity is below MOQ, the system tries to redistribute the buy quantity to other Attribute Groups (AGs) within that higher-level grouping that do\* meet or exceed their MOQs. This prioritizes making viable orders, even if it means adjusting the precise mix of individual AGs.\**
4. **Finalize** Category-level adjustments
  - What we do: *After rolling up through all levels, a final check and adjustment are performed at the highest level (Category) to ensure the total buy quantity for that category meets its MOQ, if possible.*
  - Why we do it: *This is the last opportunity to consolidate and adjust quantities across a broad range of products to meet supplier-level MOQs, preventing the failure of an entire category order.*

## 6. Sample Scenario

Let's walk through a real example:

### Starting Point

- A retailer plans to buy “Denim Jeans” for the upcoming season.
- The **Minimum Order Quantity (MOQ)** for any Denim category item from the supplier is 100 units.
- Initial calculated buy quantities (after STR conversion) for specific **Attribute Groups (AGs)** are:
  - Dark Wash Slim Fit Jeans, Premium Price Bucket: 30 units
  - Light Wash Straight Leg Jeans, Value Price Bucket: 45 units
  - Black Skinny Jeans, Premium Price Bucket: 25 units
  - Blue Distressed Jeans, Value Price Bucket: 55 units

### What Happens

1. First, the module **computes** the initial buy quantities, converting from financial OTB to physical units.
2. Then, it **aggregates** quantities at the **Price Bucket** level:
  - Premium Price Bucket (Dark Wash Slim Fit + Black Skinny) =  $30 + 25 = 55$  units. This is below the 100-unit MOQ for Denim.
  - Value Price Bucket (Light Wash Straight Leg + Blue Distressed) =  $45 + 55 = 100$  units. This meets the 100-unit MOQ.
3. Next, the system **rolls up** to the **Subcategory** level (assuming all are “Jeans” Subcategory):
  - Since the Premium Price Bucket (55 units) is below MOQ, its quantity is noted for redistribution. The Value Price Bucket (100 units) already meets MOQ.
  - The total for “Jeans” Subcategory is 155 units ( $55 + 100$ ), which well exceeds the 100-unit MOQ.
  - The module intelligently **redistributes** the 55 units from the Premium Price Bucket to the Value Price Bucket since it already meets MOQ. This means the 55 units from the “Premium” AGs are effectively “moved” to be fulfilled by increasing the order for “Value” AGs to ensure the total MOQ is met. The individual “Premium” items might not be ordered.
4. Finally, the system outputs the adjusted quantities. In this case, the Dark Wash Slim Fit Jeans and Black Skinny Jeans quantities would be marked as “unadjusted” (meaning they won't be ordered as individual AGs because their group couldn't meet MOQ), and the Light Wash Straight Leg Jeans and Blue Distressed Jeans would have their quantities adjusted upward proportionally to absorb the remaining 55 units to reach the total target.

### Final Outcome

The module produces a buy plan where the overall “Denim Jeans” order meets the 100-unit MOQ. The 55 units from the “Premium Price Bucket” that individually couldn't meet MOQ are either cancelled or shifted to be purchased as part of the “Value Price Bucket” order, which *did* meet MOQ, preventing a situation where the supplier would refuse a small order.



**Key Takeaway:** This step ensures that all purchase orders are viable by meeting supplier Minimum Order Quantities, intelligently consolidating demand across product groupings to avoid lost sales or inefficient small buys.

## 7. Results

This step produces:

### Main Outputs

- **MOQ-Adjusted Buy Quantities:** The final, reconciled quantities for each product (Attribute Group) that are ready to be ordered, ensuring all supplier Minimum Order Quantities are met where possible.
  - *What it tells us:* This is the definitive list of how many units of each item to purchase, accounting for supplier constraints and the overall optimization of the buy plan. These quantities also include an adjusted Optimum Depth (OD) (or “width” in the code) for core bestsellers, ensuring popular items maintain sufficient depth.
- **Unadjusted Quantities:** A clear record of products or Attribute Groups whose calculated buy quantities could not be adjusted or rolled up sufficiently to meet the Minimum Order Quantity (MOQ).
  - *What it tells us:* This identifies specific items that are “below MOQ” and therefore will not be ordered unless a manual override or specific business decision is made, helping planners understand potential assortment gaps.

### Business Reports

- **MOQ Adjustment Summary Report:** A high-level overview detailing which categories or product groupings had their quantities adjusted to meet MOQs, and the magnitude of those adjustments.
  - *Who uses it and why:* Buying and planning teams use this to understand the impact of MOQ constraints on their overall buy plan, identifying categories that frequently require significant adjustments.
- **Unadjusted Items Report:** A detailed list of all Attribute Groups and their quantities that could not be ordered due to not meeting the Minimum Order Quantity, even after all rollout attempts.
  - *Who uses it and why:* Merchandising and buying teams use this critical report to identify potential stockouts or missed assortment opportunities for specific products, prompting decisions on whether to accept the non-order or seek alternative solutions.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed for Minimum Order Quantities (MOQ)	The system might apply a default (e.g., zero, or a system-wide minimum) which could lead to excessively	Promptly contact the data source (e.g., procurement, vendor management) to obtain and input the correct MOQ data. Rerun the module after the data is updated.

Situation	What Happens	What You Should Do
All Attribute Groups (AGs) within a category fall significantly below MOQ	small orders or incorrect adjustments, or it might halt the process.   Even after rolling up to the highest level (Category), the combined quantity for that category might still not meet its MOQ. The system will mark all quantities as unadjusted.	Review the planned sales and strategy for that category. Consider if the MOQ is realistic, if planned sales are too low for a viable order, or if this category should be discontinued. Manual intervention may be required to decide whether to place a partial order, make a larger (over-MOQ) order, or cancel the buy.
MOQ for a category is set extremely high	An unrealistically high MOQ might prevent any purchases for that category, even if individual items have good planned sales.	Verify the MOQ setting with procurement or the supplier. If the MOQ is truly too high, negotiate a lower MOQ with the vendor, or adjust the planned quantities to meet the MOQ if strategically sound.
Initial calculated buy quantities are zero for certain AGs	If the Optimum Width module or STR computation results in zero units for specific Attribute Groups, this module will not attempt to adjust them, and they will be recorded as zero.	Confirm if zero buy quantity was intended for these AGs. If not, investigate upstream modules (OW, STR Compute) to ensure correct initial planned quantities are generated.

## 9. Next Steps

### Before This Step

- **OTB Data Input Module:** This module gathers all foundational business rules, including the specific Minimum Order Quantities (MOQs) from suppliers.
  - *What we need from it:* The critical MOQ data that defines the constraints for this module's adjustments.
- **Optimum Width (OW) Calculation Module:** This module determines the initial ideal number of categories and styles, and importantly, provides the initial planned inventory quantities by period.
  - *What we need from it:* The baseline planned sales volumes and inventory allocations (before MOQ constraints) that this module will work with.
- **OtbStrComputeModule:** This module calculates the precise Sell-Through Rates (STR) at various levels of detail.
  - *What we need from it:* The accurate sales velocity predictions, which are used to convert financial OTB into physical units for MOQ checks.

## After This Step

- **Final OTB Calculation / Order Generation Module:** This module consumes the MOQ-adjusted buy quantities to finalize the open-to-buy budget and generate actual purchase orders.
  - *What they need from us:* The definitive, supplier-compliant unit quantities for each product to be ordered.
- **Inventory Allocation Module:** This module uses the derived (and now MOQ-adjusted) quantities to guide the distribution of products to stores.
  - *What they need from us:* The final, adjusted quantities to ensure the right products are in the right place at the right time, minimizing stockouts and excess inventory.

## 10. Common Questions

Q: Why can't we just order the exact quantities calculated by the previous modules?

A: Suppliers often impose Minimum Order Quantities (MOQs) to make production and shipping economically viable for them. Ordering less than the MOQ can result in higher unit costs, delayed shipments, or outright refusal from the supplier. This module ensures our orders are always accepted and efficient.

Q: What happens if we absolutely need a product, but its calculated quantity is below MOQ and cannot be adjusted? A: If a product's quantity is marked as "unadjusted," it means the system couldn't incorporate it into a viable MOQ-compliant order. You would see this on the "Unadjusted Items Report." At that point, buying teams must decide: either not order the item, manually override to buy the full MOQ (potentially overbuying), or negotiate with the supplier for an exception.

Q: How does redistributing quantities across product groups affect my planned assortment? A: When quantities are redistributed to meet an MOQ at a higher level (like a category), it means that individual product variations (e.g., a specific color or style) that were initially planned might be reduced or not ordered at all. Instead, the quantity is added to other, similar products that *can* meet their MOQ, ensuring the broader product group is ordered, even if the exact initial mix changes slightly.

Q: Is it possible to overbuy due to MOQ adjustments? A: Yes, it is possible. If your calculated need for an item is, say, 70 units, but the MOQ is 100 units, the system might round up to 100 (or redistribute to another group that reaches 100). While this ensures a viable order, it means you're buying 30 units more than your initial calculated need, potentially leading to excess inventory if not managed carefully.

Q: Who is responsible for setting and maintaining the Minimum Order Quantities (MOQs)? A: Minimum Order Quantities are typically determined through negotiations with suppliers by your procurement or buying teams. It's crucial for these teams to regularly review and update MOQ data to reflect current vendor agreements and ensure accurate planning.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To adjust calculated buy quantities to meet Minimum Order Quantity (MOQ) constraints.
- **Main Input:** Initial planned buy quantities, computed Sell-Through Rates (STR), Attribute Group (AG) definitions, and Minimum Order Quantities (MOQ).
- **Main Output:** MOQ-Adjusted Buy Quantities and a list of Unadjusted Quantities.
- **Warning Signs:** Frequent instances of unadjusted items, large discrepancies between initial ideal buy quantities and final adjusted orders, or an overall buy plan that seems consistently too high or too low for a category.
- **Key Settings:** MOQ Adjustment Rollup Order.

---

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.

---

Generated by [AI Codebase Knowledge Builder](#)

# Chapter 4: OtbStyleOptionsModule

Last Updated: 2025-05-23

## 1. Business Objective

This module determines the final selection of product styles and their precise quantities to be purchased and displayed. It refines the inventory plan by correcting for Ideal Size Set (ISS) quantities, ensuring enough styles to meet Minimum Display Requirements, and intelligently re-allocating quantities from styles that are deemed unviable for ordering. Finally, it aggregates these detailed style-level decisions into clear store-level purchase summaries.

**Why This Matters:** This crucial step ensures that every dollar invested in inventory translates into a visually appealing store and a highly salable product mix. By fine-tuning quantities for size availability and display needs, it helps retailers maximize sales, minimize stockouts, reduce wasted inventory on unviable items, and present a compelling assortment to customers, ultimately boosting profitability and customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **MOQ-Adjusted Buy Quantities:** We need the initial planned quantities for each product Attribute Group (AG), already adjusted to meet supplier Minimum Order Quantity (MOQ) requirements, along with their associated Optimum Depth (OD) (number of unique styles planned).
  - *Why we need it:* This provides the baseline unit quantities and the desired style breadth that this module will refine and break down into specific style-level purchases.
- **Style-Level Revenue & Live Days Data:** We need historical data on the revenue generated by specific styles and the number of days they were actively sold.
  - *Why we need it:* This information helps the module identify high-performing styles, which are prioritized when the system needs to add more styles to meet Minimum Display Requirements.
- **Planned Inventory Allocations:** We need the forecasted sales volumes for products, typically derived from initial planning, to guide how quantities are distributed across styles.
  - *Why we need it:* This helps distribute the total buy quantity across individual styles in a way that aligns with anticipated sales velocity.

## Product Data

- **Attribute Group (AG) Definitions:** We need detailed definitions of product characteristics like category, subcategory, price bucket, brand, and style theme.
  - *Why we need it:* This structural information is essential for grouping styles, applying rules based on product attributes, and associating styles with their correct product hierarchies.
- **Ideal Size Set (ISS) Quantities:** We need the recommended combination of sizes for specific product styles that will maximize sales and minimize stockouts.
  - *Why we need it:* This ensures that even for a small buy, enough units are acquired to cover the most critical sizes, preventing frustrating stockouts for customers looking for common sizes.
- **Optimum Depth (OD) Override Information:** We need data that indicates if certain Attribute Groups (AGs) or styles should be excluded from receiving additional options (e.g., if they are being phased out or are not intended for display).
  - *Why we need it:* This allows the system to respect strategic decisions to limit or discontinue certain styles, preventing unwanted inventory accumulation.

## Operational Data

- **Minimum Display Requirements:** We need the smallest number of distinct styles or unique items that must be kept on display in a store for a given category or product grouping.
  - *Why we need it:* This ensures that stores always maintain an appealing and well-stocked visual presentation, directly influencing customer engagement and sales.
- **Category Minimum Order Quantities (MOQ):** We need the overall smallest allowable quantity that can be ordered from suppliers for a specific product category.
  - *Why we need it:* This is used to determine if an entire style is “exited” (removed from the buy plan) if its total quantity across all stores is too low to justify an efficient order.
- **Visual Merchandising Grouping Attribute:** We need to know which product attribute (e.g., “Style Theme” or “Color Family”) is used to group styles for visual merchandising and display planning.
  - *Why we need it:* This defines how styles are logically grouped on the sales floor, influencing where Minimum Display Requirements apply.

## 3. Business Settings

Setting Name	Default Value	Description	When to Change
Display Option Fill Strategy	Prioritize high-performing styles by revenue	Determines how the system chooses which styles to add when a display falls short of its Minimum	If you want to emphasize newness over historical performance, or if you have a strategic reason to fill displays with

Setting Name	Default Value	Description	When to Change
<b>Visual Merchandising Grouping</b>	Style Theme	Display Requirement. It prioritizes styles with a strong sales history.	specific types of products regardless of their past sales.
		Defines the product attribute (e.g., “color”, “pattern”, or “style theme”) that determines how products are visually grouped for Minimum Display Requirements.	If your store’s merchandising strategy changes, and you start organizing products by a different visual characteristic on the sales floor (e.g., from grouping by color to grouping by product collection).
		The minimum total quantity a style must have across all planned store orders to be considered viable for purchase. Styles below this threshold are “exited.”	If suppliers change their global Minimum Order Quantities for specific styles or categories, or if your inventory strategy requires a higher or lower threshold for viable style purchases.
<b>Style Exit Threshold</b>	Category-level Minimum Order Quantity		
<b>Size Set Adjustment Method</b>	Round Up to nearest Ideal Size Set	How the system adjusts a style’s quantity when it doesn’t meet the Ideal Size Set, ensuring a complete size run is purchased.	If you want to implement a more flexible approach to size set adherence (e.g., allowing for partial size sets for very slow-moving items) or if you need to strictly enforce exact multiples of the Ideal Size Set.

## 4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

 A[MOQ-Adjusted Buy Quantities & Style Revenue Data] -->
 B{Calculate Style-Level Buys & Identify Exited Styles}
 B --> C{Re-allocate Exited Quantities}

```

C --> D{Adjust Quantities for Ideal Size Set (ISS)}  
 D --> E{Adjust Styles to Meet Minimum Display Options (MDO)}  
 E --> F[Final Style & Store Buy Quantities]

1. Start with → The initially calculated buy quantities already adjusted to meet supplier Minimum Order Quantity (MOQ) constraints, along with historical style revenue data.
2. Then → The module **breaks down** these quantities into specific style-level purchases and **identifies** any individual styles that, across all stores, are too small to justify an efficient order (“exited” styles).
3. Next → The system **re-allocates** the quantities from these “exited” styles to other viable styles within the same product group, ensuring no planned purchase value is lost, only shifted to more effective items.
4. Then → It **adjusts** each style’s quantity to ensure it meets the recommended Ideal Size Set (ISS), guaranteeing sufficient units for popular sizes.
5. Next → The module **increases** the number of styles for displays where needed, ensuring Minimum Display Requirements are met, pulling in high-performing styles or new ones to fill gaps.
6. Finally → A comprehensive set of “Final Style Buy Quantities” is produced, which is then **aggregated** to create “Final Store Buy Quantities” for high-level review.

## 5. Step-by-Step Actions

1. **Determine** style-level buys
  - What we do: *We break down the Attribute Group (AG) level buy quantities, which were adjusted for Minimum Order Quantities (MOQs), into specific style-level purchases. For core bestsellers, we use their existing styles; for other items, we may create new, generic styles to meet the desired Optimum Depth (OD) (number of options).*
  - Why we do it: *This translates the aggregated buy plan into actionable quantities for individual product styles, which is the level at which customers make purchasing decisions.*
2. **Identify** and re-allocate exited styles
  - What we do: *We identify styles where the total planned quantity across all stores is too low to meet the category’s overall Minimum Order Quantity. These styles are “exited” from the buy plan, and their planned quantities are re-distributed, using a round-robin approach, to other viable styles within the same Attribute Group (AG) or store.*
  - Why we do it: *This optimizes the overall buy by moving inventory commitment away from items that are too small to be ordered efficiently, ensuring maximum sales potential from the remaining, viable items and preventing wasted purchases.*
3. **Correct** for Ideal Size Sets (ISS)
  - What we do: *For every style, we ensure its planned quantity is at least the quantity specified by the Ideal Size Set (ISS) for its relevant category and subcategory, rounding up if necessary.*
  - Why we do it: *This crucial step prevents stockouts of popular sizes by ensuring we buy a sufficient minimum quantity of each style to adequately cover the full range of sizes needed by customers, leading to fewer missed sales and higher customer satisfaction.*



#### 4. **Adjust** for Minimum Display Options (MDO)

- What we do: *We check if the number of distinct styles planned for a specific display grouping (as defined by the Visual Merchandising Grouping setting, e.g., “all floral patterns”) falls below the required Minimum Display Options. If it does, we intelligently add more styles to fill that gap, prioritizing styles that have high historical revenue contribution and avoiding any styles marked for “zero override” (meaning they are being phased out).*
- Why we do it: *This ensures that stores always have enough variety and visual impact on their displays, providing an attractive shopping experience and meeting merchandising standards, which directly impacts customer engagement and sales.*

#### 5. **Aggregate** style-level buys to store-level purchases

- What we do: *We sum up all the final, adjusted style-level quantities for each store and Attribute Group (AG) to determine the total units and the final number of unique styles (Optimum Width) for each store’s assortment.*
- Why we do it: *This provides a clear, consolidated view of the total Open-to-Buy for each store, offering a high-level summary that is ready for final purchase order generation and subsequent inventory allocation.*

## 6. Sample Scenario

Let’s walk through a real example:

### Starting Point

- A store (Store 101) has an initial planned purchase of 150 units for its “Women’s Casual Dresses, Floral Print” Attribute Group (AG), with an **Optimum Depth (OD)** recommendation for 3 unique styles, as determined by the previous module.
- The **Ideal Size Set (ISS)** for any “Women’s Casual Dress” style is 8 units (e.g., 1XS, 2S, 3M, 1L, 1XL).
- The **Minimum Display Requirement (MDO)** for the “Women’s Dresses” category, when grouped by “Pattern” for visual merchandising, is 5 distinct styles.
- Globally, one specific style, “Floral Midi Dress A,” initially planned for 5 units across all stores, falls below its category’s overall Minimum Order Quantity (MOQ) and is therefore marked as “exited.” These 5 units need to be re-allocated.
- Store 101 currently has only 2 “Floral Print” styles planned (after initial processing), but needs 3 more to meet its MDO of 5 for that display grouping.

### What Happens

1. First, the initial 150 units for the “Women’s Casual Dresses, Floral Print” are **distributed** among the existing 3 styles planned for Store 101 (e.g., 50 units for Style 1, 50 for Style 2, 50 for Style 3).
2. Then, the 5 units from the “exited” “Floral Midi Dress A” (which won’t be ordered anywhere) are **re-allocated** by increasing the quantities of the remaining viable styles within Store 101’s “Women’s Casual Dresses, Floral

- Print” AG. For example, Style 1 might receive 2 additional units (total 52), Style 2 receives 2 (total 52), and Style 3 receives 1 (total 51).
3. Next, each style’s quantity is **adjusted** to meet the **Ideal Size Set (ISS)**. If, after re-allocation, any style now has less than 8 units (the ISS), its quantity is increased to 8 units to ensure a complete size run. For instance, if one style ended up with 5 units, it’s now increased to 8.
  4. Finally, the system checks Store 101’s “Women’s Dresses” category, grouped by “Pattern.” It finds only 2 “Floral Print” styles currently planned. Since the **Minimum Display Requirement (MDO)** is 5, the module **adds** 3 new “Floral Print” styles (e.g., “Floral Maxi Dress,” “Floral Wrap Dress,” “Floral Mini Dress”) to Store 101’s plan. These new styles are chosen from available, high-revenue styles within the broader category and are each added with their **Ideal Size Set (ISS)** quantity (8 units).

## Final Outcome

The module produces a refined buy plan for Store 101. The “Women’s Casual Dresses, Floral Print” category now features 5 distinct “Floral Print” styles, successfully meeting the Minimum Display Requirement. Each style has quantities adjusted to ensure a full Ideal Size Set, reducing size-related stockouts. The 5 units from the globally exited style have been efficiently re-absorbed into other viable products, preventing dead stock.

**Key Takeaway:** This step ensures that the final inventory plan aligns with both operational display needs and customer purchasing patterns, leading to an attractive store, balanced inventory, and optimized sales.

## 7. Results

This step produces:

### Main Outputs

- **Final Style Buy Quantities (by Store, Attribute Group, and Style Name):** This is the definitive list of each unique product style and the precise quantity to be purchased for each store, after all adjustments for Ideal Size Set (ISS), Minimum Display Options, and re-allocation from exited styles.
  - *What it tells us:* This is the granular, actionable plan for what specific items, colors, and styles should be bought and in what precise numbers for each store, ready for allocation and order generation.
- **Final Store Buy Quantities (by Store and Attribute Group):** This output provides the aggregated total units and the final “Optimum Width” (number of distinct styles) for each Attribute Group within each store.
  - *What it tells us:* This gives a clear, high-level summary of the overall buy plan for each store, useful for financial review and high-level inventory management decisions.

### Business Reports

- **Style Options & Quantity Audit Report:** A detailed report showing the initial planned quantity, all specific adjustments made (for Ideal Size Set, Minimum

Display Options, and re-allocation from exited styles), and the final quantity for every style, broken down by store.

- *Who uses it and why:* Buying teams and merchandisers use this to understand the precise modifications made to the initial buy plan, ensuring transparency and enabling verification of decisions before placing orders.
- **Minimum Display Fulfillment Report:** A report highlighting which store/category/visual merchandising grouping combinations met their Minimum Display Options and, if not, which additional styles were introduced to bridge the gap.
  - *Who uses it and why:* Visual merchandising teams and store operations managers use this to confirm that display standards are met and to plan store layouts accordingly, ensuring an appealing customer experience.
- **Exited Styles & Re-allocation Summary:** A report detailing which styles were “exited” from the overall buy plan (i.e., not ordered due to low total quantities) and how their planned units were redistributed among other viable styles.
  - *Who uses it and why:* Buying teams use this to identify styles that might be underperforming or not viable for ordering, informing future assortment decisions and preventing inefficient buys.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed for any input (e.g., MOQ-adjusted buys, Ideal Size Sets, Minimum Display Requirements).	The system might use default values (e.g., zero for quantities, or a system-wide minimum for display options), leading to incorrect adjustments, or it might halt the process, indicating an error.	Promptly investigate the source of the missing data (e.g., the MOQ Adjustment Module for buy quantities, or the Data Input Module for Ideal Size Sets/ Minimum Display Requirements) and arrange for it to be provided. Rerun the module once data is available.
No viable styles available to meet Minimum Display Options (MDO) for a category.	The system will attempt to add styles from broader levels (city, region) with good revenue contributions, but if none are available or all are marked for zero override, the MDO for that category will not be met.	Review the product assortment for that category. This situation might indicate a lack of appealing styles or an overly aggressive strategy to phase out products. Merchandising should decide if new styles need to be sourced or if the MDO target needs adjustment.
Significant quantity shift due to many styles being “exited.”	If a large number of styles, or styles with significant planned quantities, are exited, their units are re-allocated to fewer remaining styles. This can potentially lead to overstocking of the surviving styles,	Analyze the “Exited Styles & Re-allocation Summary.” If the re-allocation results in undesirable overstocking, consider adjusting the initial Optimum Depth (OD) for the surviving styles, or review the viability of the exited styles for future seasons.

Situation	What Happens	What You Should Do
Ideal Size Set (ISS) is significantly larger than the initially calculated style quantity.	exceeding their original planned Optimum Depth.	
	The Ideal Size Set correction will increase the quantity of that style to meet the ISS, which can lead to a higher total buy quantity than initially planned before this module.	This is generally expected behavior to ensure salability. However, if this happens frequently for many styles and leads to significant overbuying, review the Ideal Size Set definitions or the initial planning for that style's quantity (e.g., if the initial planned quantity was unrealistically low).

## 9. Next Steps

### Before This Step

- **OTB Data Input Module:** This module provides foundational business rules such as Ideal Size Set (ISS) quantities, Minimum Display Requirements, and Optimum Depth (OD) override information.
  - *What we need from it:* The critical data points for sizing, visual display standards, and specific product exclusions that this module uses for its adjustments.
- **OtbMoqAdjustmentModule:** This module provides the initial buy quantities, already adjusted to meet supplier Minimum Order Quantities (MOQs), and indicates the Optimum Depth (OD) for each Attribute Group (AG).
  - *What we need from it:* The pre-processed unit quantities that form the basis for style-level breakdown and final adjustments, ensuring supplier constraints are already met.
- **OtbStrComputeModule:** This module computes precise Rate of Sale (ROS) metrics and provides historical style-day revenue data.
  - *What we need from it:* The revenue data which is essential for prioritizing styles when the system needs to add more options to meet Minimum Display Requirements.

### After This Step

- **Purchase Order Generation Module:** This module consumes the final style and store buy quantities to create actual purchase orders with suppliers.
  - *What they need from us:* The definitive, granular list of exactly what to buy, in what quantity, and for which stores, ready for direct execution of purchasing.
- **Inventory Allocation Module:** This module uses the final style and store buy quantities to plan the precise distribution of products to individual stores.
  - *What they need from us:* The accurate stock levels for each style and store, ensuring optimal product placement to maximize sales and minimize holding costs.

- **Visual Merchandising Planning:** Visual merchandising teams use the final determined style options and quantities to plan store layouts and product displays.
  - *What they need from us:* Confirmation of the style breadth and specific items available for display purposes, enabling them to create attractive and effective store presentations.

## 10. Common Questions

Q: Why do quantities change again in this module, even after the Minimum Order Quantity (MOQ) adjustments? A: The previous module focuses on meeting supplier order minimums. This module refines the plan further by ensuring we have enough of each style to sell properly (Ideal Size Set) and enough variety on the display floor (Minimum Display Options). It also intelligently re-distributes quantities from any styles that aren't viable for ordering overall, ensuring efficient use of the inventory budget.

Q: What does “exiting styles” mean, and how does it affect my assortment? A: An “exiting style” is one where the total planned quantity across all stores is too low to meet the supplier’s overall minimum for that style’s category. Instead of buying a tiny, inefficient quantity, the system “exits” that style and re-allocates its planned units to other, more viable styles within the same product group. This might mean you won’t get that specific style, but you’ll get more of a similar, better-selling one, ensuring efficient purchasing.

Q: How does the Ideal Size Set (ISS) correction impact my inventory? A: The Ideal Size Set correction ensures that even if a style’s initial calculated quantity is low, you still buy enough units to cover the essential sizes (e.g., enough mediums). This prevents common stockouts of popular sizes, improves customer satisfaction by having sizes available, and can slightly increase your overall buy quantity for certain styles to ensure proper size availability.

Q: Why is it important to meet “Minimum Display Options”? A: Minimum Display Options ensure your store’s visual appeal and merchandising standards are met. Having a good variety of styles on display for a category attracts customers, makes the store look full, and increases the likelihood of a sale. This module ensures you have enough distinct styles to fill your displays effectively, even if it means adding a few more units to achieve the desired visual impact.

Q: Who is responsible for reviewing the final style options and quantities? A: Merchandising and buying teams are primarily responsible. They should review the “Style Options & Quantity Audit Report” and the “Minimum Display Fulfillment Report” to ensure the final buy plan aligns with their strategic vision, visual merchandising goals, and sales expectations.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To finalize style options and quantities, adjusting for Ideal Size Sets (ISS), Minimum Display Options, and re-allocating quantities from unviable styles.

- **Main Input:** MOQ-Adjusted Buy Quantities, Ideal Size Set (ISS) quantities, Minimum Display Requirements, Style-Level Revenue Data, Attribute Group (AG) definitions.
  - **Main Output:** Final Style Buy Quantities (by Store, AG, Style Name) and Final Store Buy Quantities.
  - **Warning Signs:** Unexpected gaps in product displays, persistent stockouts for certain sizes, or a final buy plan that doesn't seem to reflect desired style breadth or merchandising standards.
  - **Key Settings:** Display Option Fill Strategy, Visual Merchandising Grouping, Style Exit Threshold, Size Set Adjustment Method.
- 

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# Chapter 5: OtbDropCalcModule

Last Updated: 2025-05-23

## 1. Business Objective

This module calculates buy quantities and options for product Attribute Groups (AGs) and individual styles for both Drop 1 (initial launch) and Drop 2 (replenishment) periods. It determines initial planogram quantities, distributes quantities based on fashion contributions, and applies adjustments to meet minimum display options, ultimately generating precise style-level buy recommendations.

**Why This Matters:** This ensures that products are introduced with optimal initial quantities and replenished efficiently, maximizing sales opportunities, balancing newness with core assortments, and optimizing inventory flow throughout the selling season. By precisely managing inventory for each drop, retailers can reduce overstocking, minimize stockouts, and present a compelling, fresh assortment to customers.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Planned Inventory Allocations (from previous OTB modules):** The forecasted sales volumes and quantity targets for products during specific selling periods, already adjusted for Minimum Order Quantity (MOQ) and Ideal Size Set (ISS).
  - *Why we need it:* This provides the baseline sales targets and unit quantities that this module will break down and allocate to specific drops and styles.
- **Style-Level Revenue Contribution:** Historical data on the revenue generated by specific styles over their selling period, indicating their performance and potential.
  - *Why we need it:* This helps prioritize which styles receive more options or quantities, especially when needing to fill displays, ensuring focus on high-performing items.

## Product Data

- **Attribute Group (AG) Definitions:** Detailed classifications of products based on characteristics like category, subcategory, and theme.
  - *Why we need it:* This structural information is essential for grouping products, differentiating between ‘Core Bestseller’ (NOOS) and ‘Fashion’ items, and applying specific calculation rules to each group.
- **Optimum Depth (OD) (Size-Adjusted Options):** The initial number of unique items (SKUs) or styles considered ideal for a specific product group, adjusted for Ideal Size Set (ISS) and MOQ constraints.
  - *Why we need it:* This serves as the starting point for determining how many distinct options should be planned for each Attribute Group, influencing the breadth of the assortment.
- **Optimum Depth (OD) (Minimum Depth per Style):** The smallest quantity of units considered viable for a single style, often linked to the Ideal Size Set.
  - *Why we need it:* This ensures that even small style buys contain enough units to cover critical sizes and support a basic display, preventing purchases of unsellable quantities.
- **Style Theme Designations (e.g., Core Bestseller, Fashion):** Labels identifying products as foundational, always-in-stock items (Core Bestsellers/NOOS) versus trend-driven or seasonal items (Fashion).
  - *Why we need it:* This distinction is crucial for applying different calculation methodologies, as NOOS items are managed for consistent availability, while Fashion items are managed for initial impact and timely replenishment.

## Operational Data

- **In-Season Display Layouts and Capacity:** Information on how products are typically arranged and displayed in stores during the selling season, including the physical space available.
  - *Why we need it:* This helps establish the baseline quantities needed to fill physical displays, influencing the initial purchase quantities for Drop 1.
- **Minimum Display Requirements:** The smallest number of distinct styles or unique items that must be kept on display for a given category or product grouping.
  - *Why we need it:* This ensures that stores maintain an appealing and well-stocked visual presentation, directly influencing customer engagement and sales by guaranteeing sufficient variety on the floor.
- **Planogram Attribute for Display Grouping:** The specific product attribute (e.g., “Style Theme” or “Color Family”) used to group styles for visual merchandising and display planning.
  - *Why we need it:* This defines how products are logically organized on the sales floor, ensuring that Minimum Display Requirements are applied to the correct visual clusters.
- **Micro-Buy Benchmarks:** Guidelines for very small, targeted purchases, often used for testing new items or filling specific gaps.
  - *Why we need it:* This helps categorize styles as ‘Microbuy’ or ‘Inseason’, guiding their inventory management strategy and determining if a limited test run is sufficient or if a larger commitment is needed.



### 3. Business Settings

Setting Name	Default Value	Description	When to Change
<b>Min Size Set Multiplier</b>	1.5	A multiplier applied to the Ideal Size Set for Core Bestseller (NOOS) items, creating a buffer for their initial stock levels.	If you need to adjust the initial safety stock for your consistently selling core products to account for supply chain changes or increased demand.
<b>Planogram Display Attribute</b>	Style Theme	Determines the product attribute used to logically group items for visual merchandising and applying Minimum Display Requirements (e.g., grouping by ‘color’ instead of ‘style theme’).	If your store’s merchandising strategy changes, and you start organizing products on the sales floor by a different visual characteristic.

*Note: See More Details for additional settings.*

### 4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

 A[MOQ-Adjusted Buy Quantities & Display Data] --> B[Calculate Initial Drop 1 Options & Quantities]
 B --> C[Adjust Drop 1 for Min Display Options]
 C --> D[Calculate Drop 2 Options & Quantities]
 D --> E[Refine Final Quantities (Drop 1 & 2)]
 E --> F[Generate Style-Level Buys & Apply Tags]
 F --> G[Final Attribute Group & Style-Level Buys for Drop 1 & Drop 2]

```

1. Start with → The adjusted buy quantities from previous Open-to-Buy (OTB) steps and current in-season display data.
2. Then → The module **calculates** the initial number of distinct styles (options) and their corresponding quantities for Drop 1, differentiating between Core Bestseller (NOOS) and Fashion items.
3. Next → It **adjusts** the Drop 1 options to ensure that Minimum Display Requirements are met for all relevant product groupings.

4. Then → The module **calculates** the options and quantities for Drop 2 (the replenishment period), based on the remaining inventory needs after Drop 1.
5. Next → It **refines** the final buy quantities for both Drop 1 and Drop 2, ensuring consistency with the planned Optimum Depth (OD) per style.
6. Then → The module **generates** specific buy quantities for each individual style and **applies** strategic tags (like ‘NOOS’ or ‘Microbuy’) based on their overall quantities and characteristics.
7. Finally → A comprehensive set of “Final Attribute Group and Style-Level Buy Quantities for Drop 1 & Drop 2” is produced, ready for purchase order generation.

## 5. Step-by-Step Actions

1. **Initialize** Metadata for Stores and Categories
  - What we do: *We gather and organize all relevant product data by store, category, and Attribute Group (AG), creating a structured foundation for all subsequent calculations.*
  - Why we do it: *This sets up the system to efficiently process and track product information, ensuring accurate calculations for each unique store and product combination.*
2. **Calculate** Drop 1 Options
  - What we do: *We determine the initial number of distinct styles (options) for the first product release (Drop 1), considering Core Bestseller (NOOS) items based on minimum Optimum Depth (OD) and Fashion items based on their contribution to overall planned sales.*
  - Why we do it: *This establishes the initial breadth of the assortment for the season’s launch, balancing the consistent availability of core items with the introduction of new trends.*
3. **Adjust** Drop 1 Options for Minimum Displays
  - What we do: *We assess if the planned Drop 1 options meet the Minimum Display Requirements for each store and product grouping, adding more options if needed by prioritizing high-performing styles or new introductions.*
  - Why we do it: *This ensures that stores maintain an attractive and well-stocked visual presentation on the sales floor, directly impacting customer engagement and sales by offering sufficient variety.*
4. **Compute** Drop 1 Quantities
  - What we do: *We determine the specific unit quantities to be purchased for Drop 1, aligning these quantities with the calculated options and the required Optimum Depth (OD) per style.*
  - Why we do it: *This translates the strategic options plan into concrete unit purchases, ensuring enough stock for the initial launch while adhering to space and merchandising constraints.*
5. **Calculate** Drop 2 Options and Quantities
  - What we do: *We determine the number of distinct styles (options) and their corresponding unit quantities for the replenishment period (Drop 2), based on the remaining inventory needs after Drop 1 is planned.*
  - Why we do it: *This ensures a strategic approach to replenishment, allowing for agile responses to early sales performance while maintaining overall inventory balance.*

## 6. Refine Final Buy Quantities

- What we do: *We revise the calculated quantities for both Drop 1 and Drop 2 to ensure they are consistent with the targeted Optimum Depth (OD) per style and optimize the overall buy plan.*
- Why we do it: *This finalizes the unit counts, ensuring efficiency and accuracy before purchase orders are generated, preventing overbuying or underbuying based on refined depth calculations.*

## 7. Generate Style-Level Buys

- What we do: *We break down the Attribute Group (AG) quantities into specific individual style purchases for each drop, creating a granular plan for each unique product (e.g., specific color, fit).*
- Why we do it: *This provides the actionable detail needed for ordering, ensuring that the precise mix of styles desired by customers is acquired.*

## 8. Apply Style Tagging

- What we do: *We assign strategic tags (e.g., 'NOOS' for core, 'Microbuy' for limited tests, 'Inseason' for regular fashion items) to each individual style based on its theme and total planned quantity across all stores.*
- Why we do it: *This classification aids in managing the product lifecycle, guiding replenishment policies, marketing efforts, and markdown strategies for each style.*

# 6. Sample Scenario

Let's walk through a real example:

## Starting Point

- A store (Store 101) has a planned total buy of 150 units for its “Women’s Casual Dresses, Floral Print” Attribute Group (AG).
- The initial **Optimum Depth (OD)** (size-adjusted options) for this AG is 7 distinct styles, and the **Optimum Depth (OD)** (minimum depth per style) is 8 units.
- The **In-Season Planogram Quantity** for “Women’s Dresses, Floral Print” at Store 101 is 100 units.
- The **Minimum Display Requirement** for “Women’s Dresses” at Store 101, when grouped by “Pattern,” is 5 distinct styles.
- One style, “Core Bestseller Floral Dress,” is a **Core Bestseller** (NOOS) with a planned buy of 20 units.
- Two “Fashion” styles, “Fashion Floral Dress A” and “Fashion Floral Dress B,” each have an initial planned buy of 40 units.
- The **Min Size Set Multiplier** is 1.5. The **Micro-Buy Benchmark** for dresses is 50 units.

## What Happens

1. First, for the **Core Bestseller** “Core Bestseller Floral Dress,” its `initialPlanoQty` is set to its **Optimum Depth (OD)** (minimum depth per style), which is 8 units, because its planned buy quantity is positive and there's available planogram space. Its `initialDropOptions` becomes 1.
2. Next, the remaining planogram quantity for “Fashion” items is calculated:  $100 \text{ (total planogram)} - 8 \text{ (NOOS)} = 92 \text{ units}$ .

3. Then, for the **Fashion** styles (“Fashion Floral Dress A” and “B”), their individual contributions to the total fashion buy for the AG are determined. Their `initialPlanoQty` is calculated as their share of the remaining planogram quantity (e.g., 46 units each if contributions are equal). Their `initialDrop1Options` are derived from dividing this quantity by their minimum depth per style.
4. After the initial calculations, the system sums the `initialDrop1Options` for all “Floral Print” styles. If this total is less than the **Minimum Display Requirement** of 5 styles, the module intelligently **adds options** to meet this minimum. It would increase the `finalDrop1Options` for selected Fashion styles until 5 distinct “Floral Print” styles are achieved, prioritizing those with higher revenue contributions.
5. Subsequently, the module **computes** the final quantities for Drop 1. For the NOOS style, its `finalDrop1Qty` is the minimum of its planned buy (20 units) and the maximum of its revenue per style or  $1.5 * \text{Min Size Set}$  ( $1.5 * 8 = 12$  units). For Fashion styles, `finalDrop1Qty` considers `finalDrop1Options` and their planned depth per style.
6. Finally, **Drop 2 options and quantities** are calculated based on any remaining planned quantities after Drop 1, ensuring replenishment needs are met. Then, each style is **tagged**: for example, “Fashion Floral Dress A” might be tagged as INSEASON if its total revised buy quantity (across all stores) is 50 units or more, or MICROBUY if less than 50 units.

## Final Outcome

The module produces a refined buy plan for Store 101, detailing the precise units and distinct styles for both Drop 1 (initial launch) and Drop 2 (replenishment). The “Women’s Casual Dresses, Floral Print” category now has at least 5 distinct “Floral Print” styles planned for display. The NOOS style is ensured consistent stock, and Fashion styles are categorized for appropriate inventory management, optimizing the store’s assortment and sales strategy.

**Key Takeaway:** This step translates high-level inventory plans into actionable, style-specific buy quantities for different delivery periods, ensuring both initial impact and sustainable replenishment while meeting display needs and product classifications.

## 7. Results

This step produces:

### Main Outputs

- **Attribute Group-Level Buy Quantities and Options (Drop 1 & Drop 2):** The precise quantities and number of distinct styles planned for each product group (AG) for both initial launch and replenishment periods.
  - *What it tells us:* This provides a comprehensive overview of the aggregated buy plan for each product category across its two key drops, showing how many options and units are allocated.
- **Style-Level Buy Quantities (Drop 1 & Drop 2) with Tags and Depth Ranges:** The granular buy plan for every individual product style, indicating

which drop it belongs to, how many units, its strategic tag (NOOS, Microbuy, Inseason), and its depth profile (e.g., how many units per style).

- *What it tells us:* This is the definitive, actionable list for purchasing, detailing exactly what specific items (color, fit, etc.) should be bought, for which drop, and how they should be managed throughout their lifecycle.
- **In-Season Planogram Quantities:** The calculated quantity representing the physical display capacity for each store-category combination, indicating the baseline units needed for initial visual merchandising.
  - *What it tells us:* This helps store operations and visual merchandising teams understand the minimum stock levels required for effective product presentation on the sales floor.

## Business Reports

- **Drop Buy Plan Summary Report:** A detailed report outlining the options and quantities for each Attribute Group for Drop 1 and Drop 2, showing the breakdown and ensuring overall buy alignment.
  - *Who uses it and why:* Buying and merchandising teams use this to review the overall purchasing strategy for each product group across the two drops, confirming strategic intent and identifying any major discrepancies.
- **Style Tagging and Depth Report:** A comprehensive list of all individual styles with their assigned tags (NOOS, Microbuy, Inseason) and their calculated depth ranges (units per style).
  - *Who uses it and why:* Inventory managers, marketing teams, and allocation specialists use this to understand the strategic classification of each style, guiding replenishment policies, marketing efforts, and markdown strategies throughout the season.
- **Initial Display Quantity Report:** A summary report detailing the initial planogram quantities by store and category, confirming the baseline display stock calculations.
  - *Who uses it and why:* Store operations and visual merchandising teams use this to ensure stores are set up with the correct initial visual presentation for new arrivals and to plan their floor sets effectively.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed for any input (e.g., planned allocations, planogram data).	The system might default quantities to zero for affected areas, use outdated planogram data, or might halt the process, leading to underbuying or misallocation of options.	Promptly investigate upstream data sources (e.g., previous OTB modules, planogram system). Ensure timely and accurate data feeds. Rerun the module once data is available.
In-Season Planogram Quantity is zero for a store/category.	If no display space is identified, the system might not calculate any	Verify the planogram data for that store/category. If it's a non-display category, ensure the business logic is handled,

Situation	What Happens	What You Should Do
	<code>initialPlanoQty</code> , which could impact Drop 1 options and quantities, especially for NOOS items.	or if it's an error, update the planogram data.
All Fashion styles in a category have zero planned buy quantity.	<code>fashionTotalBuy</code> would be zero, leading to division by zero errors or zero allocation for Fashion items in Drop 1.	Verify that planned quantities are correctly imported from upstream modules. If no fashion buy is intended for that category, confirm this aligns with the assortment strategy; otherwise, adjust upstream planning.
The sum of <code>initialDrop10ptions</code> for a display grouping is very low, but no viable styles can be added.	The system attempts to add options to meet Minimum Display Requirements, but if all available styles are already at their maximum options or are marked for zero override, the target won't be met.	Review the product assortment for that category. This might indicate a lack of appealing or viable styles. Merchandising should decide if new styles need to be sourced or if the Minimum Display Requirement target needs adjustment.
A Core Bestseller (NOOS) style has a <code>finalDrop10ptions</code> of zero.	Despite being a core item, it might not be stocked for Drop 1, leading to immediate stockouts for essential, high-demand products.	Investigate why the <code>finalDrop10ptions</code> for the NOOS item became zero. This usually indicates an issue with its planned buy quantity, initial planogram quantity, or minimum depth, which should be consistently positive for core items.

## 9. Next Steps

### Before This Step

- **OtbDataInputModule:** This module provides foundational business rules such as Optimum Depth (OD) ranges, Ideal Size Set (ISS) quantities, Minimum Display Requirements, Micro-Buy Benchmarks, and In-Season Display Layout information.
  - *What we need from it:* The critical parameters that define display capacities, sizing rules, and strategic guidelines for product categorization.
- **OtbStrComputeModule:** This module computes precise Rate of Sale (ROS) metrics and provides historical style-level revenue data.
  - *What we need from it:* Accurate sales velocity predictions and historical performance insights crucial for determining buy quantities and prioritizing styles, especially for display fill.

- **OtbMoqAdjustmentModule:** This module provides the initial buy quantities for each Attribute Group, already adjusted to meet supplier Minimum Order Quantity (MOQ) requirements.
  - *What we need from it:* The pre-processed unit quantities that form the baseline for this module's detailed allocation into specific drops and styles.
- **OtbStyleOptionsModule:** This module determines the initial Optimum Depth (OD) (size-adjusted options) for each Attribute Group, providing the number of distinct styles planned before drop-specific calculations.
  - *What we need from it:* The initial strategic breadth of options for each product group, which this module refines for Drop 1 and Drop 2.

## After This Step

- **Purchase Order Generation Module:** This module consumes the final style and store buy quantities to create actual purchase orders with suppliers.
  - *What they need from us:* The exact, finalized quantities for every style, broken down by drop and store, ready for direct execution of purchasing, ensuring supplier compliance.
- **Inventory Allocation Module:** This module uses the final style and store buy quantities to plan the precise distribution of products to individual stores.
  - *What they need from us:* The accurate stock levels for each style and store, by drop, to ensure optimal product placement, maximize sales, and minimize holding costs.
- **Visual Merchandising Planning:** Visual merchandising teams use the final determined style options and quantities, especially for Drop 1, to plan store layouts and product displays.
  - *What they need from us:* Confirmation of the style breadth and specific items available for display purposes for the initial launch, enabling them to create attractive and effective store presentations.
- **In-Season Inventory Management:** Uses the style tags (NOOS, Microbuy, Inseason) and depth ranges to guide ongoing replenishment and markdown decisions.
  - *What they need from us:* Strategic classifications of products to manage their lifecycle from initial launch to end-of-season, enabling differentiated inventory strategies.

## 10. Common Questions

Q: Why do we have “Drop 1” and “Drop 2” calculations? What’s the business purpose? A: This module plans for two distinct delivery windows. Drop 1 is typically for initial launch and floor set, focusing on making an impact and filling displays. Drop 2 is for subsequent replenishment, often for bestsellers or to introduce new options based on initial sales. This phased approach allows for better inventory management, reduces upfront risk, and maintains freshness in the assortment.

Q: How does this module decide how many of each style to put on display? A: It uses the “Minimum Display Requirements” set for each product category and how products are visually grouped (e.g., by “Style Theme”). If the initial calculation doesn’t meet this minimum, the module adds more styles to the display, prioritizing those with strong past performance or newness to ensure the store looks appealing and well-stocked.

Q: What's the difference between "Core Bestseller (NOOS)" and "Fashion" styles in this calculation? A: Core Bestsellers (NOOS) are evergreen items that should always be in stock; their quantities are primarily driven by continuous demand and minimum display needs. Fashion styles are seasonal or trend-driven; their quantities are more dynamic, based on anticipated sales contribution and the need to provide newness for the initial drop.

Q: How does "Microbuy" tagging help my business? A: The "Microbuy" tag identifies styles that have very low total planned quantities, suggesting they might be test items or very niche products. This helps your teams manage them differently – perhaps with less aggressive marketing, minimal replenishment, or quicker markdown if they don't perform, minimizing financial risk.

Q: Who should review the final outputs of this module? A: Merchandising and buying teams should be the primary reviewers. They need to ensure that the planned quantities and options for each drop align with their overall product strategy, sales forecasts, and visual merchandising goals. Store operations can also review the initial display quantities.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate specific buy quantities and options for Drop 1 (initial launch) and Drop 2 (replenishment) at both Attribute Group and individual style levels.
- **Main Input:** MOQ-Adjusted Buy Quantities, Style-Level Revenue Contribution, AG Definitions, Optimum Depth (OD) values, In-Season Display Layouts, and Minimum Display Requirements.
- **Main Output:** Final Attribute Group-Level and Style-Level Buy Quantities for Drop 1 & Drop 2, along with Style Tags and Depth Ranges.
- **Warning Signs:** Unmet Minimum Display Requirements, unexpected stockouts for core items, or an overall buy plan that seems misaligned with seasonal launches versus replenishment needs.
- **Key Settings:** Min Size Set Multiplier, Planogram Display Attribute.

---

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.

---

Generated by [AI Codebase Knowledge Builder](#)



# Chapter 6: OtbOutputModule

Last Updated: 2025-05-23

## 1. Business Objective

This module generates comprehensive, denormalized output reports for the Open-to-Buy (OTB) project. It consolidates data from all stages of the OTB calculation (style buys, Rate of Sale (ROS) adjustments, Minimum Order Quantity (MOQ) adjustments, raw sales, etc.) into a user-friendly format, suitable for detailed analysis and persistence to the database.

**Why This Matters:** This crucial final step provides a “single source of truth” for the entire OTB plan, making complex inventory decisions clear and actionable. It enables merchandisers and planners to understand the ‘why’ behind every buying decision, identify opportunities and risks, and make data-driven adjustments that lead to optimized inventory investment, improved sales, and better customer satisfaction.

## 2. Required Information

We need these pieces of information to do our job:

### Sales Data

- **Final Sell-Through Rates (ROS):** *Why we need it:* These rates, computed by the Rate of Sale Computation Module, are fundamental for converting planned financial values into physical unit quantities and for reporting on projected sales velocity.
- **Planned Sales Quantities & Revenues:** *Why we need it:* The aggregated sales forecasts and revenue targets from earlier stages provide the baseline for reporting on anticipated performance and the financial effectiveness of the OTB plan.
- **Historical Sales & Return Data:** *Why we need it:* Past performance metrics, including units sold, revenue generated, and return percentages, are integrated into the reports to offer context and benchmarks against planned future performance.

### Product Data

- **Attribute Group (AG) Definitions:** *Why we need it:* Detailed classifications of products by category, subcategory, brand, and style theme are essential for

consistently categorizing and summarizing product information across all reports.

- **Style-Level Buy Quantities & Options (by Drop):** *Why we need it:* The precise unit counts and distinct styles decided for each product, broken down by initial launch (Drop 1) and replenishment (Drop 2), form the core of the final inventory plan that needs to be reported.
- **Ideal Size Set (ISS) & Optimum Depth (OD) Recommendations:** *Why we need it:* Information on optimal size combinations and the recommended number of unique items within a category is reported to show how the final buy plan aligns with merchandising strategies.

## Operational Data

- **Minimum Order Quantity (MOQ) Adjustments:** *Why we need it:* The modifications made to quantities to meet supplier Minimum Order Quantities are reported to provide transparency into how operational constraints influenced the final buy decisions.
- **Minimum Display Requirements Fulfillment:** *Why we need it:* Data on how well display targets were met is reported to assess visual merchandising compliance and the overall visual presentation of products in stores.
- **Current Inventory Health & Projections:** *Why we need it:* Information on existing stock levels, projected future inventory, and depletion status are reported to provide a complete picture of product availability and potential gaps.

## 3. Business Settings

*Note: This module primarily consumes data and settings that have been applied by previous modules in the Open-to-Buy process. Therefore, it does not introduce new business settings for calculation, but rather consolidates and reports on the results where those settings were utilized.*

## 4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Inputs from all prior OTB modules: Style Buys, STRs, MOQs, Sales, Inventory] --> B[Data Consolidation & Aggregation]
B --> C[Metric Computation & Variance Analysis]
C --> D[Comprehensive Denormalized Output Reports]
```

1. Start with → Raw, denormalized data from all prior OTB calculation stages, including final style options, drop-specific quantities, Rate of Sale (ROS) adjustments, Minimum Order Quantity (MOQ) adjustments, raw sales data, and current inventory.
2. Then → The module **gathers** and **structures** this vast array of data from various sources into multiple report-specific formats.
3. Next → It **computes** aggregate metrics, variances (e.g., total sales, options by drop, suggested vs. historical performance), and key performance indicators.

4. Finally → A comprehensive suite of “Denormalized Output Reports” is generated, providing a unified, transparent view of the Open-to-Buy plan for analysis and decision-making.

## 5. Step-by-Step Actions

1. **Consolidate** all Open-to-Buy (OTB) calculation data
  - What we do: *We bring together the final planned quantities, style options, and all intermediate adjustments from the various stages of the OTB process, including initial planned sales, Sell-Through Rates (ROS), Minimum Order Quantity (MOQ) adjustments, Ideal Size Set (ISS) corrections, and drop-specific allocations.*
  - Why we do it: *This creates a single, comprehensive dataset that reflects the entire journey of the OTB plan, from strategic inputs to final buy decisions.*
2. **Generate** summary reports for Drop 1 and Drop 2
  - What we do: *We create high-level summaries of the options and quantities planned for initial product launches (Drop 1) and subsequent replenishment (Drop 2), categorized by store and product group.*
  - Why we do it: *This provides a clear, consolidated view of the overall seasonal buy strategy, allowing merchandisers to quickly assess the breadth and depth of the assortment for different delivery windows.*
3. **Produce** detailed style-level purchase reports
  - What we do: *We compile precise reports for each individual product style, detailing its assigned drop (Drop 1 or Drop 2), specific buy quantity, and strategic tags (like ‘Core Bestseller’ or ‘Micro-Buy’).*
  - Why we do it: *This provides the granular information needed for purchase order creation and detailed inventory management, ensuring each specific product is acquired and tracked correctly.*
4. **Create** a step-by-step audit trail
  - What we do: *We generate reports that show the evolution of buy quantities and options through each stage of the OTB calculation process, from initial planning through Sell-Through Rate (ROS) adjustments, Minimum Order Quantity (MOQ) adjustments, and style-level refinements.*
  - Why we do it: *This transparency allows planners and auditors to trace how decisions were made and quantify the impact of each adjustment on the final buy, ensuring accountability and facilitating process improvements.*
5. **Compile** comprehensive performance dashboards
  - What we do: *We assemble detailed reports that compare historical sales performance (quantity, revenue, discounts) with system-suggested and buyer-suggested projections, along with key metrics like Average Selling Price (ASP), Sell-Through Rate (ROS), and Optimum Width (OW) adherence.*
  - Why we do it: *This provides critical insights into the accuracy of forecasts, the effectiveness of the OTB plan, and the overall health of inventory, supporting strategic reviews and future planning cycles.*

## 6. Sample Scenario

Let’s walk through a real example:

## Starting Point

- **Final Style Buy Quantities (from the Style Options Module):** Store 101 has 5 distinct “Floral Print” styles planned for purchase, with specific quantities per style (e.g., Style A: 50 units, Style B: 52 units), all adjusted for Ideal Size Set (ISS) and Minimum Display Requirements.
- **Drop 1 & Drop 2 Allocations (from the Drop Calculation Module):** For these 5 styles, some are allocated to Drop 1 (initial launch) and others to Drop 2 (replenishment), with specific units assigned to each drop (e.g., Style A: 30 units for Drop 1, 20 for Drop 2).
- **MOQ-Adjusted Quantities (from the MOQ Adjustment Module):** The total buy for the “Floral Print” Attribute Group (AG) is 250 units, successfully meeting the supplier’s Minimum Order Quantity.
- **Sell-Through Rates (from the Rate of Sale Computation Module):** The projected Rate of Sale (ROS) for “Floral Print” dresses at Store 101 is 65%.
- **Historical Sales Data (from the Data Input Module):** Last season, Store 101 sold 200 “Floral Print” dresses with an average selling price of \$75.

## What Happens

1. First, the `OtbOutputModule` **gathers** all this disparate information: the final quantities per style, their drop allocations, the total adjusted Attribute Group quantities, the projected Sell-Through Rates, and historical performance data.
2. Then, it **consolidates** these data points into various denormalized tables. For example, in the “Comprehensive OTB Business Report,” it combines the historical sales of 200 units with a projected buy of 250 units and a projected Sell-Through Rate of 65%. It also computes the projected revenue based on planned quantities and historical Average Selling Price (ASP).
3. Next, for the “Detailed Style Buy Plan,” it **lists** each of the 5 “Floral Print” styles, showing their specific Drop 1 and Drop 2 quantities, and any strategic tags (e.g., Style A as ‘Inseason’, Style B as ‘Core Bestseller’).
4. Concurrently, for the “OTB Process Audit Report,” it **traces** how the initial planned quantity for “Floral Print” dresses (e.g., 220 units) was first adjusted by the Rate of Sale module, then by the Minimum Order Quantity module to 250 units, and finally broken down by the Style Options and Drop Calculation modules into the specific 5 styles and their Drop 1/Drop 2 allocations.

## Final Outcome

The module produces a suite of integrated reports. A merchant can now open the “Comprehensive OTB Business Report” to see that the planned 250 units for “Floral Print” dresses represent a 25% increase over last season’s sales, supported by a projected 65% Sell-Through Rate. They can then drill down into the “Detailed Style Buy Plan” to review the specific 5 styles, confirming that 3 are for initial launch and 2 for replenishment, ensuring the desired assortment freshness and depth. The “OTB Process Audit Report” provides a transparent view of every decision that led to these numbers, making the entire planning process clear and verifiable.

**Key Takeaway:** This step acts as the central intelligence hub, compiling all previous planning decisions into a clear, verifiable, and actionable set of reports that empower merchandisers to understand, approve, and execute their Open-to-Buy plan effectively.

## 7. Results

This step produces:

### Main Outputs

- **Comprehensive OTB Business Report:** A unified report containing historical sales, projected sales, revenues, discounts, inventory health metrics, and final buy quantities/Optimum Width (OW) across all stores and Attribute Groups.
  - *What it tells us:* This is the central source of truth for the entire Open-to-Buy plan, providing a holistic view of financial and unit-level decisions, allowing for detailed performance analysis and comparison against historical data.
- **Detailed Style Buy Plan (by Drop):** A granular report outlining the specific quantities for each individual product style, broken down by initial launch (Drop 1) and replenishment (Drop 2), along with strategic tags (e.g., ‘Core Bestseller’, ‘Micro-Buy’).
  - *What it tells us:* This provides the precise plan for purchasing and managing inventory at the most detailed level, ready for direct execution of purchase orders and guiding in-season inventory management.
- **OTB Process Audit Report:** A step-by-step breakdown of how quantities and Optimum Width (OW) evolved through each stage of the OTB calculation (from initial planning to Rate of Sale, MOQ, and style/drop adjustments).
  - *What it tells us:* This report provides transparency into the OTB calculation logic, enabling planners to understand the impact of various business rules and adjustments on the final buy.
- **Sell-Through Rate Performance Report:** A detailed view of computed Sell-Through Rates (ROS) by store, category, subcategory, and price bucket, including associated planned quantities and return percentages.
  - *What it tells us:* This allows for granular analysis of sales velocity assumptions and their drivers, helping teams assess the accuracy of sales forecasts and the profitability of planned buys.

### Business Reports

- **OTB Executive Summary Dashboard:** A high-level, visual summary for leadership, highlighting key OTB metrics like total planned spend, projected sales, and overall inventory health by major category or channel.
  - *Who uses it and why:* Executive management and senior merchandising leaders use this to quickly grasp the financial implications and strategic direction of the OTB plan.
- **Variance Analysis Report:** Compares system-suggested buy quantities and options with historical performance and/or buyer-suggested overrides, highlighting significant differences.
  - *Who uses it and why:* Planning teams and buyers use this to identify areas where system recommendations deviate from historical trends or human judgment, prompting further investigation and informed adjustments.

## 8. Unusual Cases

Situation	What Happens	What You Should Do
Missing or delayed data feed from upstream OTB modules	The output reports will be incomplete or contain outdated data, reflecting the last successfully processed run, or the module might fail if critical input is missing.	Investigate the upstream module (e.g., Drop Calculation Module, Style Options Module) causing the delay or data gap and ensure its successful completion. Rerun this output module once all dependencies are met.
Inconsistencies or discrepancies in data across different input sources	If, for example, the sum of style-level quantities from one module doesn't perfectly match the aggregated Attribute Group quantity from another, the denormalized reports might show minor numerical discrepancies or imbalances.	Review the audit reports generated by previous modules (e.g., MOQ Adjustment Summary Report, Style Options & Quantity Audit Report) to pinpoint the source of the inconsistency. Collaborate with relevant teams to refine the data consistency rules in earlier stages.
Performance issues due to extremely large data volumes	Generating comprehensive denormalized reports can be resource-intensive with very large product catalogs and store counts, potentially leading to slow report generation times.	Optimize database performance, review data partitioning strategies, or consider generating reports for smaller segments of the business if full reports are not immediately critical. Coordinate with IT support for system tuning.
Misinterpretation of calculated metrics by end-users	If users don't fully understand how certain metrics (e.g., "Corrected Maximum Quantity Per Style" or "Optimum Depth Variance") are derived, they might misuse the data or make incorrect assumptions.	Provide comprehensive training and clear documentation for all generated reports. Implement tooltips or in-app explanations for complex metrics to ensure accurate interpretation and utilization.

## 9. Next Steps

### Before This Step

- **OtbDataInputModule (Data Input Module):** Provides initial business rules, strategic targets, and foundational data for all OTB calculations.
  - *What we need from it:* The base return rates, Rate of Sale (ROS) adjustments, display minimums, Ideal Size Set (ISS) quantities, and Optimum Width (OW)/Optimum Depth (OD) ranges.
- **OtbStrComputeModule (Rate of Sale Computation Module):** Computes the Rate of Sale (ROS) at various levels of detail, adjusting for planned sales and returns.
  - *What we need from it:* The precise sales velocity predictions used in all subsequent calculations.
- **OtbMoqAdjustmentModule (MOQ Adjustment Module):** Ensures all calculated buy quantities adhere to Minimum Order Quantity (MOQ) constraints.
  - *What we need from it:* The initial planned quantities after adjustments for supplier minimums.
- **OtbStyleOptionsModule (Style Options Module):** Determines final product styles and quantities, correcting for Ideal Size Set (ISS) and Minimum Display Requirements, and re-allocating quantities from unviable styles.
  - *What we need from it:* The refined style-level and store-level quantities that account for visual merchandising and optimal size mixes.
- **OtbDropCalcModule (Drop Calculation Module):** Calculates buy quantities and options for Drop 1 (initial launch) and Drop 2 (replenishment).
  - *What we need from it:* The final, granular buy quantities and options for each product, broken down by delivery period and categorized with strategic tags.

### After This Step

- **Purchase Order Generation Module:** Uses the finalized buy quantities from the reports to create actual purchase orders with suppliers.
  - *What they need from us:* The definitive, granular, and aggregated buy plans to execute purchases.
- **Inventory Allocation Module:** Leverages the detailed style and store buy quantities to plan the precise distribution of products to individual stores.
  - *What they need from us:* Accurate stock levels per style and store, by drop, to optimize product placement and maximize sales.
- **Performance Monitoring & Analysis:** Business analysts and merchandisers use the comprehensive reports to evaluate the accuracy of the OTB plan, track performance against targets, and identify areas for improvement.
  - *What they need from us:* Detailed, denormalized data for ongoing analysis, trend identification, and strategic adjustments for future planning cycles.
- **Financial Planning & Reporting:** Finance teams use the aggregated OTB values to update financial forecasts, manage cash flow, and report on inventory investments.
  - *What they need from us:* High-level financial summaries of the planned buy to integrate into broader company financial statements and forecasts.

## 10. Common Questions

Q: Why are so many different reports generated? Can't we just have one? A: While this module provides a comprehensive "single source of truth," different teams need different levels of detail. An executive might only need a high-level summary, while a buyer needs granular style-level quantities for ordering, and an analyst needs a step-by-step audit. These varied reports ensure every stakeholder has the right information for their specific needs, enabling quicker, more informed decisions.

Q: How can I use these reports to improve my future buying decisions? A: The reports compare your planned buy with historical sales, track the impact of adjustments (like MOQ), and show how your planned assortment meets display standards. By analyzing these comparisons, you can identify patterns: which products consistently overperform or underperform, where your forecasts might be off, or if certain MOQ settings are always causing issues. This data-driven insight empowers you to refine your strategies for the next season.

Q: What does "denormalized output" mean in simple terms? A: Imagine trying to understand a full story by reading separate chapters from different books. "Denormalized output" is like taking all those pieces of information and combining them into one single, easy-to-read document. Instead of looking up sales data in one place, product details in another, and display rules in a third, everything related to a specific product or category is brought together in one place, making it much easier to analyze and understand.

Q: Is the "buyer suggested" data in the reports automatically updated from the "system suggested" data? A: Yes, in many of the reports, the "buyer suggested" values initially mirror the "system suggested" values. This provides a baseline. Buyers can then, through other tools not part of this module, input their specific overrides or adjustments. The output module then reports on both the system's recommendation and the buyer's final decision, providing clarity on where human expertise has influenced the plan.

Q: How frequently are these comprehensive reports updated? A: These comprehensive reports are typically generated after each full Open-to-Buy calculation cycle, which usually happens at the beginning of a new season or major planning period. However, if there are significant in-season changes to forecasts or strategic objectives, the entire OTB process can be re-run, and these reports would be updated accordingly to reflect the latest plan.

## 11. Quick Reference

Key things to remember about this step:

- **Purpose:** To consolidate and present all Open-to-Buy (OTB) calculation results into comprehensive, user-friendly reports for analysis, decision-making, and persistence.
- **Main Input:** Finalized data from all preceding OTB calculation modules, including adjusted buy quantities, style options, drop allocations, Rate of Sale (ROS) information, and historical performance.
- **Main Output:** Comprehensive OTB Business Report, Detailed Style Buy Plan (by Drop), OTB Process Audit Report, and Sell-Through Rate Performance Report.



- **Warning Signs:** Incomplete reports, discrepancies between reported figures and actual sales/inventory, or difficulty in tracing how a final buy quantity was derived.
  - **Key Settings:** This module primarily consumes data and settings from previous modules rather than introducing new ones for calculation.
- 

**Need Help?** Contact the Merchandising Systems Support team for questions about this process.

---

Generated by AI Codebase Knowledge Builder

# otb FAQ

**Version:** 1.0 **Last Updated:** 2025-05-23

**Purpose:** This FAQ document provides answers to common business questions about the otb (Open-to-Buy) algorithm, covering its functionality, data requirements, and outputs.

---

## General

**Q:** What is the purpose of the otb algorithm? **A:** The otb algorithm helps determine the ideal quantities of products to buy for specific categories and time periods, optimizing inventory levels to maximize sales and minimize stockouts by considering factors like Optimum Width (OW), Optimum Depth (OD), and Rate of Sale (ROS).

**Q:** What problem does otb solve for our business? **A:** otb addresses the challenge of making informed purchasing decisions, ensuring we stock the right products in the right quantities at the right time. This reduces excess inventory, improves product availability, and ultimately drives profitability.

**Q:** Who typically uses or benefits from the otb system? **A:** Merchandisers, planners, and inventory managers are the primary users. They benefit from otb by gaining data-driven insights to make strategic buying decisions for various product categories and styles.

**Q:** What are the core components or modules of the otb algorithm? **A:** The otb algorithm processes data through several key modules, including data input, Sell-Through Rate (STR) computation, Minimum Order Quantity (MOQ) adjustments, style option determination, drop-specific quantity calculation, and comprehensive output reporting.

**Q:** How frequently is the otb algorithm typically run? **A:** The frequency of otb runs can vary based on business needs and planning cycles (e.g., weekly, monthly, or seasonally) to inform ongoing buying decisions and react to market changes.

**Q:** How does otb help with inventory decisions? **A:** By providing calculated buy quantities based on strategic inputs like Optimum Width (OW), Optimum Depth (OD), and historical performance, otb guides precise inventory investments, ensuring balanced stock levels across stores and categories.

---

## Configuration & Settings

**Q:** Where are the global settings for the otb algorithm managed? **A:** Global settings for otb, such as default return percentages, minimum display options, and micro-buy benchmarks, are typically configured externally through a dedicated properties file or a configuration management system.

**Q:** How do I adjust the minimum display options for products or categories? **A:** Minimum display options are part of the input data processed by the `OtbDataInputModule`. You would typically update these settings in the database or input source that feeds the otb system, which then influences the `OtbStyleOptionsModule` and `OtbDropCalcModule`.

**Q:** Can I set custom Sell-Through Rate (STR) targets for specific products or categories? **A:** Yes, the system allows for manual input overrides for Sell-Through Rate (STR). These overrides are applied by the `OtbStrComputeModule` after initial STR calculations, ensuring your strategic targets are factored in.

**Q:** What is a “micro-buy benchmark” and how does it influence quantities? **A:** A micro-buy benchmark specifies a minimum quantity threshold for very small buys, often used for test styles or niche products. This benchmark, loaded by the `OtbDataInputModule`, ensures that even small initial buys meet a practical minimum, affecting calculations in the `OtbDropCalcModule`.

**Q:** How are Minimum Order Quantity (MOQ) rules configured and applied? **A:** Minimum Order Quantity (MOQ) constraints are loaded as input data. The `OtbMoqAdjustmentModule` then intelligently adjusts calculated buy quantities to meet these MOQs, potentially redistributing quantities across Attribute Groups (AGs) to optimize the overall buy.

---

## Data & Inputs

**Q:** What essential data is required to run the otb algorithm? **A:** The `OtbDataInputModule` requires various inputs, including Optimum Width (OW) output, Sell-Through Rate (STR) overrides, return percentages, minimum display options, in-season planogram data, style depth ranges, micro-buy benchmarks, Minimum Order Quantities (MOQ), and Ideal Size Set (ISS) quantities.

**Q:** Where does the Optimum Width (OW) data come from? **A:** The Optimum Width (OW) output, representing the ideal number of categories or styles to offer, is a crucial input sourced from previous analyses or dedicated planning tools and is loaded via the `OtbDataInputModule`.

**Q:** How are customer returns factored into the otb calculations? **A:** Return percentages are retrieved by the `OtbDataInputModule` and then applied by the `OtbStrComputeModule` during Sell-Through Rate (STR) calculation to provide a more accurate picture of net sales.

**Q:** What is an “Ideal Size Set” (ISS) and how is it used in otb? **A:** An Ideal Size Set (ISS) is the best combination of sizes to maximize sales and minimize stockouts for a particular style. The `OtbStyleOptionsModule` uses ISS data to correct and adjust calculated buy quantities, ensuring optimal size distribution.

**Q:** Can I use historical performance data as input for otb? **A:** Yes, historical sales data is foundational for otb. This data, including Full Price and End-of-Season Sale (EOSS) quantities, is used by the `OtbStrComputeModule` to calculate Sell-Through Rates (STR) and by other modules for quantity distribution.

---

## Process & Logic

**Q:** How does otb determine the best quantities to buy for each style or product? **A:** otb determines quantities by first calculating Sell-Through Rates (STR), adjusting for Minimum Order Quantity (MOQ) constraints, applying Ideal Size Sets (ISS), and then distributing quantities based on factors like fashion contributions and Optimum Depth (OD) targets.

**Q:** How is Sell-Through Rate (STR) calculated and used within the otb process? **A:** The `OtbStrComputeModule` calculates STR based on gross sales (Full Price and EOSS) adjusted for returns. This calculated STR is then used throughout the otb pipeline as a key indicator of product performance and demand.

**Q:** What happens if a product's calculated quantity falls below its Minimum Order Quantity (MOQ)? **A:** If an Attribute Group (AG) quantity falls below MOQ, the `OtbMoqAdjustmentModule` attempts to redistribute quantities from that AG to other AGs that meet the MOQ criteria. If redistribution isn't possible, the AG may be marked as unadjusted or require manual review.

**Q:** How are different "Drops" (e.g., Drop 1, Drop 2) handled by the algorithm? **A:** The `OtbDropCalcModule` specifically calculates buy quantities and options for different drops. It considers initial planogram quantities, distributes quantities based on fashion trends, and applies minimum display options pertinent to each drop.

**Q:** How does the system decide on the final style options to display? **A:** The `OtbStyleOptionsModule` is responsible for this. It takes calculated buy quantities, applies Ideal Size Set (ISS) corrections, enforces minimum display options, and re-allocates quantities from any exited styles to determine the final, optimized set of styles and their quantities for display.

---

## Outputs & Results

**Q:** What kind of reports and outputs does the otb algorithm generate? **A:** The `OtbOutputModule` generates comprehensive denormalized reports, consolidating data from various stages such as style buys, STR adjustments, MOQ adjustments, and raw sales data. These reports are designed for detailed analysis and database persistence.

**Q:** Can I see the adjustments made for Minimum Order Quantity (MOQ) in the output? **A:** Yes, the `OtbOutputModule` includes details of the MOQ adjustments, allowing you to see which Attribute Groups (AGs) were adjusted and how quantities were redistributed to meet MOQ constraints.

**Q:** Where can I find the final buy quantities for each style and store? **A:** The final aggregated style-level and store-level buy quantities are consolidated in the output reports generated by the `OtbOutputModule`, providing a clear overview of the recommended purchases.

**Q:** How are Sell-Through Rates (STR) presented in the otb output? **A:** The output reports include the computed Sell-Through Rates (STR) at various rollup levels (e.g., store-category-subcategory-price bucket), reflecting both the initial calculation and any manual overrides applied.

**Q:** What information is available for analyzing past otb runs? **A:** The otb output includes historical snapshots of calculated quantities, STRs, and adjustment details, enabling post-season analysis to refine future buying strategies and assess the accuracy of previous recommendations.

---

## Error Handling & Edge Cases

**Q:** What happens if critical input data is missing when running otb? **A:** The `OtbDataInputModule` is designed to validate inputs. If critical data, such as Optimum Width (OW) or Minimum Order Quantity (MOQ) rules, is missing, the otb algorithm may halt or produce warnings, requiring the missing data to be provided before a successful run.

**Q:** How does otb handle new products or styles without historical sales data? **A:** For new products, the otb algorithm typically relies on initial planogram quantities, micro-buy benchmarks, and fashion contributions defined in the input data. Sell-Through Rates (STR) for these styles would be based on forecasts or similar product performance.

**Q:** What if an Attribute Group (AG) cannot meet its Minimum Order Quantity (MOQ) even after redistribution attempts? **A:** If the `OtbMoqAdjustmentModule` cannot satisfy an AG's MOQ through redistribution, the quantity for that AG may be flagged as unadjusted in the output, indicating a scenario that might require manual intervention or a decision to proceed with a quantity below MOQ.

**Q:** How are styles marked as “exited” handled by the otb system? **A:** Styles marked as “exited” (e.g., discontinued) have their quantities re-allocated by the `OtbStyleOptionsModule` to other active styles or categories, preventing overstocking on non-performing items.

**Q:** What should I do if my otb results seem incorrect or unexpected? **A:** If results are unexpected, first review the input data (e.g., OW, STR overrides, MOQ values). Then, consult the detailed output reports to trace the calculations through the different modules (STR, MOQ, style options) to identify where the discrepancy might have occurred.

---

## Need More Help?

For further assistance or detailed inquiries, please contact the support team at [support@yourcompany.com](mailto:support@yourcompany.com) or refer to the comprehensive otb documentation.

## # OTB Documentation

- **\*\*Version\*\***: 1.0
- **\*\*Last Updated\*\***: 2025-05-23

## # 2. Executive Summary

The Open-to-Buy (OTB) algorithm optimizes inventory planning by precisely calculating required buy quantities for merchandise. It ensures the business stocks the *\*Optimum Depth\** (OD) of products across the *\*Optimum Width\** (OW) of categories, driven by accurate *\*Rate of Sale\** (ROS) forecasts and adherence to supply chain constraints like Minimum Order Quantities (MOQ). This system provides actionable insights to maximize sales, minimize stockouts, and improve overall profitability.

## # 3. Architecture Overview

```mermaid

graph TD

```
A[OtbDataInputModule] -->|initializes OtbData| B(OtbStrComputeModule)
B -->|computes STRs & adjusted quantities| C(OtbMoqAdjustmentModule)
C -->|applies MOQ adjustments| D(OtbStyleOptionsModule)
D -->|finalizes style & store buys| E(OtbDropCalcModule)
E -->|calculates drop-specific outputs| F(OtbOutputModule)

A -->|initial configuration data| F
B -->|computed STR values| F
C -->|MOQ adjustment results| F
D -->|final buy quantities| F
```

Module Descriptions:

- * OtbDataInputModule:** Initializes the central data store with all necessary raw input data for OTB calculations.
- * OtbStrComputeModule:** Computes the *Rate of Sale* (ROS) at granular levels and applies any manual override values.
- * OtbMoqAdjustmentModule:** Adjusts calculated buy quantities to meet Minimum Order Quantity (MOQ) constraints by redistributing or marking unadjusted items.
- * OtbStyleOptionsModule:** Determines the final style options and quantities, correcting for *Ideal Size Set* (ISS) and applying minimum display requirements.
- * OtbDropCalcModule:** Calculates the specific buy quantities and options for products across different delivery drops (e.g., Drop 1, Drop 2).
- * OtbOutputModule:** Consolidates and generates comprehensive denormalized output reports for analysis and database persistence.

4. Prerequisites & Setup

To run the OTB system, ensure the following are in place:

- **Java Version:** Java 17 or higher.
- **Build Tool:** Maven 3.8.x or higher.
- **Dependencies:** All project dependencies are managed via Maven (pom.xml). Ensure your local Maven repository is up-to-date.
- **Database Access:** Configured access to the upstream and downstream databases as specified in args.properties.
- **Running the ModuleGroup Orchestrator:**
 1. **Navigate** to the root directory of the otb project.
 2. **Build** the project using Maven: mvn clean install
 3. **Execute** the main application via the orchestrator class (e.g., ModuleGroup.java): java -jar target/otb-<version>.jar
 4. Ensure args.properties and common-data.json are present in the classpath or in the root directory.

5. Folder & File Structure

otb/
├ module-group/ - Orchestrates the execution of all OTB submodules in the correct sequence.
├ submodules/ - Contains the individual OTB calculation modules.
├ args.properties - Global configuration parameters for the OTB system.
├ common-data.json - Defines the shared data model structure used across modules.
└ outputs/ - Directory where all generated OTB reports and output files are stored.

6. Global Configuration

The args.properties file contains global configuration parameters that influence the overall OTB calculation process.

| Setting Name | Default Value | Description | Valid Range/When to Change |
|----------------------|---------------|---|-----------------------------|
| otb.calculation.mode | PLAN | Specifies the operational mode for OTB calculations (e.g., PLAN for planning, RECALCULATE | PLAN, RECALCULATE, FORECAST |

| Setting Name | Default Value | Description | Valid Range/When to Change |
|---------------------------------|---------------|--|--|
| otb.min.display.threshold | 3 | for adjustments).
Minimum number of units required for a style to be considered for display, ensuring a visually impactful presentation. | Integer > 0. Adjust based on visual merchandising standards. |
| otb.str.override.source | DATABASE | Defines where manual <i>Rate of Sale</i> (ROS) overrides are sourced from. | DATABASE, FLAT_FILE. Change if manual overrides are from files. |
| otb.moq.redistribution.strategy | HIGHEST_ROS | Determines how quantities are redistributed when an <i>Attribute Group</i> (AG) falls below Minimum Order Quantity (MOQ). HIGHEST_ROS prioritizes AGs with the fastest <i>Rate of Sale</i> . | HIGHEST_ROS, EVEN_DISTRIBUTION. Adjust based on business priority. |
| otb.output.format | JSON | Specifies the primary format for the final OTB reports. | JSON, CSV |

7. Shared Data Model

The `common-data.json` schema defines the structure of data exchanged between the various modules, ensuring consistency and interoperability.

| Field | Type | Description |
|-----------------------|---------|---|
| category_id | integer | Unique identifier for a product category. |
| subcategory_id | integer | Unique identifier for a product subcategory within a category. |
| style_id | string | Unique identifier for a specific product style (e.g., SKU). |
| store_id | integer | Unique identifier for a retail store location. |
| region_id | integer | Unique identifier for a geographical region. |
| week_number | integer | The specific week number within the planning horizon. |
| full_price_units | integer | Number of units sold at full price. |
| eoss_units | integer | Number of units sold during End-of-Season Sale. |
| return_percentage | decimal | The percentage of units expected to be returned. |
| str_override_value | decimal | Manually provided <i>Rate of Sale</i> (ROS) override. |
| min_order_quantity | integer | The minimum number of units that can be ordered for a given product. |
| ideal_size_set_config | array | Configuration for the <i>Ideal Size Set</i> (ISS) defining quantity per size. |
| current_inventory | integer | Current stock levels for a given product/store. |
| planned_buy_quantity | integer | The calculated quantity to be purchased for a product. |
| optimum_width_target | integer | The target number of categories/styles to offer (<i>Optimum Width</i>). |
| optimum_depth_target | integer | The target number of unique items (<i>Optimum Depth</i>) within a category. |

8. Submodule Chapters

8.1 OtbDataInputModule

1. Module Name & Business Objective

OtbDataInputModule: This module is the foundational data loader for the OTB system. Its business objective is to **initialize all necessary input data** for Open-to-Buy calculations by retrieving critical planning variables, historical performance, and operational constraints from the database into the central OtbData store. This ensures subsequent modules have a consistent and comprehensive dataset to work with.

2. Required Information

- **Sales Data:**
 - Historical *Rate of Sale* (ROS) data (e.g., units sold per week by store, category, style).
 - Sell-Through Rate (STR) override values (manual adjustments to ROS).
 - Return percentages.
- **Product Data:**
 - *Optimum Width* (OW) output (ideal categories/styles).
 - Style depth ranges (e.g., how many units of a style are typically stocked).
 - *Ideal Size Set* (ISS) configurations (the preferred combination of sizes).
- **Operational Data:**
 - Minimum display options (e.g., minimum units for visual merchandising).
 - In-season planogram data (current stock/display plans).
 - Micro-buy benchmarks (guidelines for small-quantity, high-fashion buys).
 - Minimum order quantities (MOQ).

3. Business Settings (relevant to this submodule)

- Configuration parameters determining data source (e.g., `otb.str.override.source`).
- Database connection details for data retrieval.
- Specific table or view names for different data inputs.

4. Process Flow

External Databases --> OtbDataInputModule --> OtbData Store

5. Step-by-Step Actions

1. **Connect** to the configured databases. *This ensures access to all necessary source systems.*
2. **Retrieve** *Optimum Width* (OW) output. *This sets the scope for category and style offerings.*
3. **Fetch** Sell-Through Rate (STR) overrides and return percentages. *These are crucial for accurate ROS calculations.*
4. **Load** minimum display options and in-season planogram data. *These provide constraints for visual merchandising and existing inventory.*
5. **Gather** style depth ranges, micro-buy benchmarks, and Minimum Order Quantities (MOQ). *These define operational boundaries for purchasing.*
6. **Import** *Ideal Size Set* (ISS) quantities. *This guides optimal size distribution.*
7. **Populate** the central OtbData store with all retrieved information. *This makes the data available for subsequent processing modules.*

6. Sample Scenario

A retailer is planning buys for the Spring/Summer season. The OtbDataInputModule connects to the sales database and pulls historical sales for 'Women's Dresses - Casual', along with a pre-defined STR override of 0.65 for a new fashion collection. It also loads a 10% return rate, an *Ideal Size Set* (ISS) of

[S:2, M:3, L:2], and a Minimum Order Quantity (MOQ) of 12 units per style. This foundational data is then stored for use by modules like `OtbStrComputeModule`.

Key Takeaway: Accurate and complete initial data is paramount for reliable OTB calculations.

7. Results

The primary result is a fully populated `OtbData` store, containing all the raw, foundational data needed for subsequent OTB calculations. This includes: * OW configurations * STR overrides * Return rates * Minimum display rules * Planogram data * Style depth ranges * Micro-buy parameters * MOQ rules * ISS quantities

8. Unusual Cases

- **Missing or delayed data feed:** If a critical data source (e.g., STR overrides, OW output) is unavailable or delayed, the module may run with incomplete data, leading to suboptimal or inaccurate OTB recommendations.
- **Inconsistent data definitions:** If the data from different sources uses conflicting definitions (e.g., `style_id` format varies), data loading errors or misinterpretations can occur.

9. Next Steps

Once the `OtbDataInputModule` successfully populates the `OtbData` store, the pipeline proceeds to the `OtbStrComputeModule`, which utilizes this raw data to calculate and adjust Sell-Through Rates (STRs).

10. Common Questions

1. **Why is my OTB recommendation off?** *It might be due to incorrect or outdated input data from this module.*
2. **Where does the system get the return percentages from?** *They are loaded by this module, typically from the database or a configuration table.*
3. **Can I manually upload STR overrides?** *Yes, if the `otb.str.override.source` is configured for flat files, though usually it's from the database.*
4. **What if a new category isn't in the OW output?** *The system will not plan for it, as the foundational OW data is missing.*
5. **How often is the input data refreshed?** *This depends on the scheduling of the OTB pipeline, usually daily or weekly.*

11. Quick Reference

- **Purpose:** Data Initialization
 - **Inputs:** Database (OW, STR overrides, returns, MOQ, ISS, etc.)
 - **Outputs:** Populated `OtbData` store
 - **Key Data:** OW, STR overrides, Return %, MOQ, ISS
-

8.2 OtbStrComputeModule

1. Module Name & Business Objective

OtbStrComputeModule: This module's core business objective is to **accurately compute the Sell-Through Rate (STR)** at specified product and store rollup levels. It ensures that the calculated STR reflects actual sales performance (Full Price and EOSS) adjusted for returns, and critically, it allows for strategic manual overrides to shape future buy decisions. This provides a realistic *Rate of Sale* (ROS) for subsequent planning stages.

2. Required Information

- **Sales Data:**
 - Full Price quantities sold.
 - End-of-Season Sale (EOSS) quantities sold.
 - Return percentages (from OtbDataInputModule).
- **Product Data:**
 - Product *Attribute Groups* (AGs) and rollup levels (e.g., store-category-subcategory-price bucket).
- **Operational Data:**
 - STR manual override values (from OtbDataInputModule).

3. Business Settings (relevant to this submodule)

- `otb.str.override.source`: Determines if manual STR overrides come from a database or a file.
- `otb.str.calculation.window`: Specifies the historical period for STR calculation.
- `otb.str.minimum.sales.threshold`: Minimum sales units required for a valid STR calculation.

4. Process Flow

OtbData Store (Raw Sales, Returns, Overrides) -->
OtbStrComputeModule --> Calculated STR Values & STR-Adjusted
Quantities (OtbData Store)

5. Step-by-Step Actions

1. **Retrieve** raw sales data (Full Price and EOSS) and return percentages from the OtbData store. *This provides the base for calculation.*
2. **Calculate** the raw Sell-Through Rate (STR) for each defined rollup level (e.g., store-category-subcategory-price bucket) based on total units sold and available inventory. *This generates the baseline performance metric.*
3. **Apply** return percentages to adjust the calculated STR, reducing it by anticipated returns. *This provides a more realistic net sell-through.*
4. **Check** for any manual STR override values. *These allow business users to strategically influence future planning based on insights not captured by historical data.*

5. **Override** the calculated STR with manual input where provided. *This implements the business's strategic intent.*
6. **Store** the final computed STR values and the resulting STR-adjusted buy quantities back into the `0tbData` store. *These are essential inputs for the next module in the pipeline.*

6. Sample Scenario

For 'Men's Casual Shirts' (Category) in 'Store A' (Rollup Level), the `0tbStrComputeModule` retrieves 100 units sold at Full Price and 20 units sold during EOSS from `0tbData`. With an initial stock of 200 units and a 10% return rate, the raw STR is $(100+20)/200 = 0.60$. After adjusting for returns, it becomes $0.60 * (1 - 0.10) = 0.54$. However, a manual override of 0.60 for this specific category and store combination is provided due to an upcoming marketing campaign. The module **applies the override**, resulting in a final STR of 0.60 for 'Men's Casual Shirts' in 'Store A'.

Key Takeaway: STR calculation is dynamic, balancing historical performance with strategic business overrides to derive the true *Rate of Sale* (ROS).

7. Results

The module produces: * `0tbStrOutputRow`: Computed and overridden STR values per rollup level. * `0tbStrAdjustedRow`: Buy quantities adjusted based on the final STR values, stored within the `0tbData` store.

8. Unusual Cases

- **Missing or delayed data feed:** If historical sales data or return percentages are incomplete, the STR calculation may be inaccurate, leading to misinformed buy quantities.
- **Conflicting STR overrides:** If an override is provided for a rollup level but conflicts with a more granular override, the system must have a clear precedence rule (e.g., granular overrides take precedence).

9. Next Steps

The STR-adjusted buy quantities generated by `0tbStrComputeModule` are then consumed by the `0tbMoqAdjustmentModule` to ensure that proposed buys adhere to Minimum Order Quantity (MOQ) constraints.

10. Common Questions

1. **Why does the system's STR not match my spreadsheet calculation?** *Check for applied return percentages and manual overrides within the system, which may not be in your spreadsheet.*
2. **How can we influence the computed STR for new styles?** *By providing manual STR overrides for those specific styles or similar Attribute Groups (AGs) in the input data.*

3. **What if a category has very low sales history?** *The module might use a default STR or apply a broader category's STR based on configuration, or flag it for manual review.*
4. **Can I see the raw STR before overrides are applied?** *The system's output reports typically show the final STR; a deeper dive into logs or intermediate tables might be needed for raw values.*
5. **How are returns handled in the STR calculation?** *Returns reduce the net sell-through, making the calculated STR a more realistic reflection of net sales.*

11. Quick Reference

- **Purpose:** Calculate & Adjust Sell-Through Rate (ROS)
 - **Inputs:** Raw sales, returns, STR overrides from `OtbData`
 - **Outputs:** Computed STR, STR-adjusted buy quantities
 - **Key Data:** Full Price Units, EOSS Units, Return %, STR Overrides
-

8.3 OtbMoqAdjustmentModule

1. Module Name & Business Objective

OtbMoqAdjustmentModule: The business objective of this module is to **ensure that all calculated buy quantities adhere to Minimum Order Quantity (MOQ) constraints**. It intelligently aggregates quantities, identifies *Attribute Groups* (AGs) falling below MOQ, and redistributes units from those AGs to others that meet the MOQ, or explicitly marks them as unadjusted, minimizing wasted inventory and optimizing purchasing efficiency.

2. Required Information

- **Product Data:**
 - *Attribute Groups* (AGs) (e.g., Color, Size, Brand) and their associated products.
 - Rollup levels (Price Bucket, Subcategory, Category) for aggregation.
- **Operational Data:**
 - Minimum Order Quantity (MOQ) for each style or *Attribute Group* (from `OtbDataInputModule`).
- **Sales Data:**
 - STR-adjusted buy quantities (from `OtbStrComputeModule`).

3. Business Settings (relevant to this submodule)

- `otb.moq.redistribution.strategy`: How quantities are moved from under-MOQ AGs (e.g., `HIGHEST_ROS`, `EVEN_DISTRIBUTION`).
- `otb.moq.rounding.method`: How quantities are rounded to meet MOQ (e.g., `ROUND_UP`, `ROUND_TO_NEAREST`).

4. Process Flow

STR-Adjusted Quantities (OtbData Store) --> OtbMoqAdjustmentModule
--> MOQ-Adjusted Quantities (OtbData Store)

5. Step-by-Step Actions

1. **Retrieve** STR-adjusted buy quantities from the OtbData store. *These are the initial proposed buy quantities.*
2. **Aggregate** quantities for each *Attribute Group* (AG) at specified rollup levels (e.g., Price Bucket, Subcategory, Category). *This ensures MOQ is applied at the correct granular level.*
3. **Identify** all AGs where the aggregated buy quantity falls below its defined Minimum Order Quantity (MOQ). *These are the problematic groups that need adjustment.*
4. **Attempt to redistribute** quantities from the under-MOQ AGs to other related AGs within the same category that already meet or exceed their MOQ. *This aims to consolidate buys and meet MOQ without over-ordering.*
5. **Apply** the configured `otb.moq.redistribution.strategy` (e.g., prioritizing AGs with highest *Rate of Sale* (ROS)). *This optimizes the redistribution logic based on business priorities.*
6. **Mark** any quantities that cannot be successfully redistributed as “unadjusted” or “below MOQ.” *This provides visibility into items that could not meet MOQ constraints.*
7. **Store** the final MOQ-adjusted quantities (and any unadjusted quantities) back into the OtbData store. *These quantities are now compliant with supplier ordering rules.*

6. Sample Scenario

A ‘New Jeans’ style (Style ID: 54321) has an MOQ of 24 units. The OtbStrComputeModule recommended a buy of 20 units (STR-adjusted). The OtbMoqAdjustmentModule identifies that $20 < 24$. It then checks other jeans *Attribute Groups* (AGs) within the ‘Denim Bottoms’ category. If ‘Classic Fit Jeans’ (another AG) has a recommended buy of 50 (which meets its MOQ) and is marked with HIGHEST_ROS, the module might **add the 4 deficit from ‘New Jeans’ to ‘Classic Fit Jeans’**, bringing its buy to 54 units. Alternatively, if no suitable AG exists for redistribution, the ‘New Jeans’ buy might be marked as unadjusted or bumped up to 24 if a ROUND_UP strategy is used.

Key Takeaway: MOQ adjustment ensures that orders are feasible from a supplier perspective, preventing small, inefficient buys while strategically reallocating stock where possible.

7. Results

The module produces: * OtbMoqAdjustedRow: Buy quantities that have been successfully adjusted to meet MOQ. * OtbMoqUnAdjustedRow: Quantities that could not be adjusted to meet MOQ, providing visibility into exceptions. These results are stored in the OtbData store.

8. Unusual Cases

- **Missing or delayed data feed:** If MOQ definitions for certain *Attribute Groups* (AGs) are missing, the module might default to an incorrect MOQ or fail to adjust quantities properly.
- **No viable redistribution targets:** If an *Attribute Group* (AG) falls below MOQ and there are no other AGs within the same category that can absorb the extra units, the module must explicitly mark these as unadjusted, potentially leading to missed opportunities or stockouts for those specific items.

9. Next Steps

The MOQ-adjusted quantities are then passed to the `OtbStyleOptionsModule`, where they will be refined further to account for *Ideal Size Set* (ISS) configurations and minimum display options.

10. Common Questions

1. **Why did the system recommend buying more than the sales forecast?** *This is often due to MOQ adjustments rounding up quantities to meet supplier minimums.*
2. **Can we disable MOQ adjustments for certain categories?** *Not directly in this module, but it can be controlled by setting the MOQ to 1 for those categories in the input data.*
3. **How does the system decide which items get the extra units from MOQ adjustments?** *Based on the configured `otb.moq.redistribution.strategy`, often prioritizing items with higher Rate of Sale (ROS) or more consistent performance.*
4. **What happens if an item cannot meet MOQ even after redistribution?** *It is typically marked as “unadjusted” or “below MOQ” in the output, requiring manual review or strategic decision.*
5. **Does MOQ adjustment apply to every single SKU?** *It applies at the Attribute Group (AG) level specified in the configuration, which could be style, subcategory, or category level, depending on the business need.*

11. Quick Reference

- **Purpose:** Enforce Minimum Order Quantities
 - **Inputs:** STR-adjusted buy quantities, MOQ rules from `OtbData`
 - **Outputs:** MOQ-adjusted buy quantities
 - **Key Data:** MOQ, Attribute Groups, Redistribution Strategy
-

8.4 OtbStyleOptionsModule

1. Module Name & Business Objective

OtbStyleOptionsModule: This module’s business objective is to **determine the final style options and their precise quantities to be displayed and stocked**. It refines previously calculated buy quantities by correcting for *Ideal Size Set* (ISS)

configurations, applying minimum display requirements to ensure visual merchandising standards, and re-allocating quantities from styles that are being exited. This results in actionable, store-level purchase plans that align with both sales potential and display strategies.

2. Required Information

- **Product Data:**
 - *Ideal Size Set* (ISS) configurations (from `OtbDataInputModule`).
 - Style attributes and status (e.g., exited styles).
- **Operational Data:**
 - Minimum display options (from `OtbDataInputModule`).
- **Sales Data:**
 - MOQ-adjusted buy quantities (from `OtbMoqAdjustmentModule`).

3. Business Settings (relevant to this submodule)

- `otb.min.display.threshold`: Minimum number of units for a style to be considered for display.
- `otb.style.exit.reallocation.strategy`: How quantities from exited styles are re-allocated (e.g., to similar best-sellers).
- `otb.iss.application.method`: How *Ideal Size Set* (ISS) is applied (e.g., pro-rata, fixed ratio).

4. Process Flow

MOQ-Adjusted Quantities (`OtbData Store`) --> `OtbStyleOptionsModule`
--> Final Style/Store Buys (`OtbData Store`)

5. Step-by-Step Actions

1. **Retrieve** MOQ-adjusted buy quantities from the `OtbData` store. *These are the base quantities to be refined.*
2. **Correct** buy quantities based on the *Ideal Size Set* (ISS) configurations. *This ensures that the correct number of units per size is ordered to maximize sales and minimize stockouts.*
3. **Apply** minimum display options to ensure styles meet visual merchandising standards. *This might increase quantities for certain styles to meet display thresholds.*
4. **Identify** styles marked as “exited” or “discontinued” for the upcoming season. *These styles should not be re-ordered.*
5. **Re-allocate** quantities from exited styles to other active, performing styles based on a defined reallocation strategy. *This prevents over-ordering obsolete inventory and optimizes space for new products.*
6. **Aggregate** the finalized style-level buys to derive store-level purchases. *This provides the comprehensive purchase plan per store.*
7. **Store** the final style-level and store-level buy outputs into the `OtbData` store. *These are the definitive purchasing recommendations.*

6. Sample Scenario

A ‘Summer Dress’ (Style ID: SD001) has an MOQ-adjusted buy of 18 units. The `OtbStyleOptionsModule` first applies the *Ideal Size Set* (ISS) of [S:3, M:5, L:3]. This ISS totals 11 units. To reach 18 units, the module scales this ISS or applies it multiple times. For example, if it’s applied twice (22 units), the quantity would be rounded down to 18 units with an adjusted ISS, or if it’s based on a fixed ratio, the 18 units would be split by the ISS. If a minimum display option requires at least 6 units for a style to be shown, and the adjusted quantity is 5, the module might increase it to 6. Finally, if another style (‘Old Sweater’, Style ID: OS002) is exited, its prior planned buy quantity is re-allocated to a similar, best-selling style like ‘New Cardigan’.

Key Takeaway: This module translates high-level buy quantities into practical, store-ready plans that consider display, size availability, and inventory freshness.

7. Results

The module produces: * `OtbStoreBuyOutputRow`: Final buy quantities per store. * `OtbStyleBuyOutputRow`: Final buy quantities per style. These results are stored in the `OtbData` store.

8. Unusual Cases

- **Missing or delayed data feed:** If *Ideal Size Set* (ISS) configurations or minimum display options are incomplete, the final buy quantities might not be optimized for customer preference or visual appeal.
- **Unreconcilable display minimums:** If a style’s calculated demand is very low, but the minimum display option is high, it might force an uneconomical buy quantity, leading to potential overstock.

9. Next Steps

The finalized style and store-level buy quantities from `OtbStyleOptionsModule` serve as the primary input for the `OtbDropCalcModule`, which will break down these buys into specific delivery drops.

10. Common Questions

1. **Why are we buying sizes we typically don’t sell much of?** *Likely due to the Ideal Size Set (ISS) application, ensuring full size runs are available to capture all potential sales.*
2. **How are new styles handled if they don’t have an ISS?** *They might use a default ISS from their category or be flagged for manual ISS assignment.*
3. **Why was the quantity for this style increased significantly?** *Possibly to meet the `otb.min.display.threshold` to ensure adequate visual merchandising.*
4. **Can I see which styles were exited and where their units were re-allocated?** *The output reports will show the final buy quantities; detailed tracing might require examining intermediate logs.*

5. **What if a store has limited display space?** *The current system applies a global minimum display; specific store-level display constraints would need to be incorporated as additional input data.*

11. Quick Reference

- **Purpose:** Finalize Style & Store Buys, Apply ISS & Display Rules
 - **Inputs:** MOQ-adjusted quantities, ISS, Min Display options
 - **Outputs:** Final Style Buy, Final Store Buy
 - **Key Data:** ISS, Min Display, Exited Styles
-

8.5 OtbDropCalcModule

1. Module Name & Business Objective

OtbDropCalcModule: This module's business objective is to **calculate precise Attribute Group (AG)-level and style-level buy quantities for specific delivery periods (e.g., Drop 1, Drop 2)**. It refines the overall buy quantities by distributing them based on initial planogram needs, fashion contributions, and re-applying minimum display adjustments. This ensures that the right products arrive at the right time, optimizing inventory flow and alignment with marketing and seasonal plans.

2. Required Information

- **Product Data:**
 - Style tagging (e.g., NOOS (Never Out Of Stock), Microbuy, Inseason).
 - Style depth ranges (from OtbDataInputModule).
 - Fashion contributions (e.g., percentage allocation for fashion vs. core items).
- **Operational Data:**
 - Minimum display options (from OtbDataInputModule).
 - Initial planogram quantities.
- **Sales Data:**
 - Final style and store-level buy quantities (from OtbStyleOptionsModule).

3. Business Settings (relevant to this submodule)

- `otb.drop.allocation.strategy`: How quantities are split across drops (e.g., `PLANOGRAM_FIRST`, `FASHION_CONTRIBUTION`).
- `otb.drop1.percentage`: Percentage of total buy allocated to Drop 1 if not otherwise specified.

4. Process Flow

Final Style/Store Buys (OtbData Store) --> OtbDropCalcModule --> Drop-Specific Buys (OtbData Store)

5. Step-by-Step Actions

1. **Retrieve** finalized style and store-level buy quantities from the 0tbData store. *These are the total units to be allocated.*
2. **Determine** initial planogram quantities for each style for Drop 1. *This ensures foundational stock is available early.*
3. **Distribute** remaining quantities based on fashion contributions. *This allocates units according to the strategic importance or trendiness of styles.*
4. **Apply** adjustments to meet minimum display options for each drop. *This ensures that even split buys maintain adequate visual presence.*
5. **Handle** style tagging (e.g., N00S items might have different allocation rules than Microbuy items). *This customizes the distribution logic per product type.*
6. **Calculate** style depth ranges for each drop. *This ensures optimal inventory depth per style for the specific period.*
7. **Generate** Attribute Group (AG)-level and style-level buy quantities for Drop 1 and Drop 2. *This creates distinct, actionable buy plans for each delivery window.*
8. **Store** these drop-specific outputs in the 0tbData store. *These detailed plans are crucial for procurement and logistics.*

6. Sample Scenario

A 'New Spring Jacket' (Style ID: SJ123) has a total approved buy of 60 units from 0tbStyleOptionsModule. The 0tbDropCalcModule first allocates 20 units to Drop 1 for initial display and early sales, based on its initial planogram quantity. The remaining 40 units are then split: 60% (24 units) to Drop 2 as it's a fashion item with an early-season demand, and 40% (16 units) to Drop 1 to meet additional early demand and display options. The module also tags it as an 'Inseason' item for future tracking.

Key Takeaway: This module refines overall buy plans into precise, time-bound delivery schedules, ensuring optimal product flow and inventory levels throughout the season.

7. Results

The module produces: * 0tbDropOutputRow: Detailed Attribute Group (AG)-level buy quantities and options per drop. * 0tbDropStyleBuysRow: Detailed style-level buy quantities per drop. These results are stored in the 0tbData store.

8. Unusual Cases

- **Missing or delayed data feed:** If fashion contribution percentages or planogram data are missing, the drop allocation might default to a simple split, potentially misaligning inventory with peak demand periods.
- **Insufficient total buy for drops:** If the overall buy quantity for a style is too low to meet minimum display options for *both* Drop 1 and Drop 2, the system might have to prioritize one drop or flag it for manual review.

9. Next Steps

The detailed drop-specific outputs generated by `OtbDropCalcModule` are then consumed by the `OtbOutputModule` to generate comprehensive denormalized reports for analysis and persistence.

10. Common Questions

1. **Why is a certain style appearing heavily in Drop 1 instead of Drop 2?** *Likely due to high initial planogram allocation, early fashion contribution, or strong Rate of Sale (ROS) forecasts for early season.*
2. **Can we adjust the split between Drop 1 and Drop 2 for specific categories?** *Yes, through configuration settings for drop allocation strategies or manual overrides if supported by input data.*
3. **What does “style tagging” mean in this context?** *It categorizes styles (e.g., N00S, Microbuy, Inseason) to apply specific allocation rules, ensuring appropriate inventory strategies for different product types.*
4. **How are carry-over items handled for drops?** *They are often tagged as N00S (Never Out Of Stock) and may have different, more consistent, drop allocations.*
5. **If we have a promotion planned, how does it affect drop quantities?** *Promotions usually influence the initial Rate of Sale (ROS) calculation (`OtbStrComputeModule`), which then feeds into the total buy, and subsequently the drop allocation.*

11. Quick Reference

- **Purpose:** Calculate Drop-Specific Buy Quantities
 - **Inputs:** Final style/store buys, planogram, fashion contributions, style tags
 - **Outputs:** Drop 1 Buys, Drop 2 Buys (AG & Style level)
 - **Key Data:** Fashion Contributions, Style Tagging, Drop Allocation Strategy
-

8.6 OtbOutputModule

1. Module Name & Business Objective

OtbOutputModule: This module’s ultimate business objective is to **generate comprehensive, user-friendly denormalized output reports** for the entire OTB project. It consolidates data from every stage of the OTB calculation pipeline—from raw sales to final drop-specific buys—into a single, coherent format. This provides stakeholders with a holistic view, enabling detailed analysis, decision-making, and seamless persistence of results to the database for future reference and integration.

2. Required Information

- **All Prior Stage Outputs:**
 - Initial `OtbData` (from `OtbDataInputModule`).
 - Computed STR values and STR-adjusted quantities (from `OtbStrComputeModule`).

- MOQ-adjusted and unadjusted quantities (from `OtbMoqAdjustmentModule`).
- Final style and store-level buy quantities (from `OtbStyleOptionsModule`).
- Drop 1 and Drop 2 buy quantities and options (from `OtbDropCalcModule`).
- **Reference Data:**
 - Raw sales data.
 - Inventory data.
 - Annual Operating Plan (AOP) rows.

3. Business Settings (relevant to this submodule)

- `otb.output.format`: Specifies the desired output format (e.g., JSON, CSV).
- `otb.output.destination`: Where the reports should be saved (e.g., `FILE_SYSTEM`, `DATABASE`).
- `otb.output.report.level`: Granularity of the report (e.g., `STYLE_LEVEL`, `CATEGORY_LEVEL`).

4. Process Flow

All `OtbData Store Stages` (Input, STR, MOQ, Style Options, Drops)
 --> `OtbOutputModule` --> `Denormalized Reports` (Database/Filesystem)

5. Step-by-Step Actions

1. **Retrieve** all relevant intermediate data and final calculated results from the `OtbData` store. *This includes initial inputs, STR adjustments, MOQ adjustments, final style buys, and drop calculations.*
2. **Consolidate** data from various stages into a unified, denormalized structure. *This combines disparate datasets into a single, easy-to-analyze report.*
3. **Enrich** the consolidated data with additional reference information, such as raw sales, inventory levels, and AOP figures. *This adds context and allows for comparison against targets and actuals.*
4. **Format** the consolidated data according to the specified `otb.output.format` (e.g., JSON, CSV). *This ensures the output is compatible with downstream systems or user analysis tools.*
5. **Persist** the generated reports to the configured `otb.output.destination` (e.g., database table, local file system). *This makes the results available for reporting, dashboards, and long-term storage.*
6. **Log** the completion status and location of the generated reports. *This provides an audit trail for data consumers.*

6. Sample Scenario

After all OTB calculations are complete, the `OtbOutputModule` collects the final Drop 1 buy quantities for 'Women's Dresses' (Style ID: WD456), the STR (0.65) calculated for it, and the MOQ adjustments that might have increased its initial quantity. It also pulls the raw historical sales for WD456 and compares it against the planned AOP for the season. All this information is then compiled into a single CSV file, detailing the final recommended buy quantity, split by drop, for each style, along with its calculated STR and any MOQ impacts. This file is then saved to the `outputs/` folder and uploaded to the central database.

Key Takeaway: This module is the final delivery point, providing a complete and transparent view of the OTB planning process and its outcomes for actionable business intelligence.

7. Results

The module produces: * Comprehensive, denormalized reports containing all calculated OTB data (e.g., `OtbResultReport.json` or `OtbResultReport.csv`). * Persistence of these reports to a database table or file system, depending on configuration.

8. Unusual Cases

- **Missing or delayed data feed:** While this module is at the end of the pipeline, if any upstream module failed to provide its output, the final reports would be incomplete or inaccurate, leading to flawed analysis.
- **Schema mismatch with persistence layer:** If the generated report structure does not align with the target database table schema, the data persistence step will fail, preventing the results from being stored.

9. Next Steps

The generated reports are consumed by business users for analysis, integrated into dashboards, or used by procurement systems for placing orders. There are no further modules in the OTB pipeline after the `OtbOutputModule`.

10. Common Questions

1. **Where can I find the final buy recommendations?** *In the output reports generated by this module, usually found in the `outputs/` folder or the designated database table.*
2. **Can I get a report showing only the changes from the last OTB run?** *Not directly from this module, but by comparing the current output report with a previous one.*
3. **How do I integrate this data into our ERP system?** *The output reports are designed to be easily consumable by other systems, typically via flat files (CSV) or direct database table access.*
4. **Does this report show why certain quantities were adjusted (e.g., for MOQ)?** *Yes, the denormalized reports aim to include details from all calculation stages to provide full transparency.*
5. **What level of detail can I expect in the output?** *Reports typically include details down to the style and store level, encompassing all attributes used in the calculations.*

11. Quick Reference

- **Purpose:** Generate Final Reports & Persist Data
- **Inputs:** All intermediate OTB data from `OtbData`
- **Outputs:** Denormalized reports (JSON/CSV), database records
- **Key Data:** Consolidated OTB plan, STR, MOQ, ISS, Drop splits

9. Appendix

Full Parameter Reference

Refer to the `args.properties` file in the project's root directory for a comprehensive list of all configurable parameters, their default values, and detailed descriptions.

Version History

- **Version 1.0 (2025-05-23):** Initial release of the OTB calculation pipeline, including core modules for data input, STR computation, MOQ adjustment, style option determination, drop calculation, and comprehensive output reporting.

Contact & Support Info

For any questions, issues, or support regarding the OTB system, please contact: *
Email: `support@yourcompany.com` * **Internal Support Channel:** `#otb-support` *
Team Lead: [Team Lead Name]

Tutorial: otb

The OTB (Open-to-Buy) system processes data through a structured pipeline of modules. Initial input data is loaded, followed by the computation of Sell-Through Rates (STR) and adjustments for Minimum Order Quantity (MOQ). Subsequently, final style options and quantities are determined. These calculations lead to the determination of buy quantities for specific drops. Finally, all intermediate and final results are consolidated into comprehensive reports.

flowchart TD

```
A0["OtbdDataInputModule"]
A1["OtbtStrComputeModule"]
A2["OtbtMoqAdjustmentModule"]
A3["OtbtStyleOptionsModule"]
A4["OtbtDropCalcModule"]
A5["OtbtOutputModule"]
A0 -- "data_flow" --> A1
A1 -- "data_flow" --> A2
A2 -- "data_flow" --> A3
A3 -- "data_flow" --> A4
A4 -- "data_flow" --> A5
A0 -- "data_flow" --> A5
A1 -- "data_flow" --> A5
A2 -- "data_flow" --> A5
A3 -- "data_flow" --> A5
```

Chapters

This document outlines the “otb” merchandising algorithm, a robust solution designed to optimize product assortment, depth, and sizing to maximize sales efficiency and customer satisfaction.

Master Report: otb Merchandising Algorithm

1. Title & Metadata

- **Project Name:** otb
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** The otb algorithm dynamically optimizes product assortment, depth, and sizing recommendations to maximize sales, improve inventory efficiency, and reduce stockouts.

2. Executive Summary

The otb merchandising algorithm is a sophisticated analytical tool designed to empower merchandising, planning, and buying teams with data-driven insights. It leverages historical sales data and current inventory levels to provide actionable recommendations for product assortment (OW), inventory depth (OD), and ideal size distribution (ISS). By automating complex calculations and identifying patterns, otb aims to significantly enhance a retailer’s ability to stock the right products, in the right quantities, at the right time, thereby increasing ROS, minimizing capital tied up in excess inventory, and improving customer satisfaction through reduced stockouts.

- **Key outcomes or KPIs:**
 - Optimized OD for each product/category.
 - Optimized OW across product categories.
 - Optimized ISS to align with customer demand.
 - Increased overall ROS.
 - Reduced out-of-stock instances.
 - Improved inventory turnover.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Planning Teams, Buying Teams, Inventory Managers, Sales Operations.

3. Architecture & Data Flow

graph TD
A[Raw Sales Data] --> B{Data Ingestion & Preprocessing};
B --> C[Cleaned, Aggregated Data];

```

D[Product Master Data] --> C;
E[Inventory Data] --> C;
C --> F{Assortment Optimization (OW & OD)};
C --> G{Size Set Optimization (ISS)};
F --> H{Reporting & Recommendation Generation};
G --> H;
I[User Configurations (args.properties)] --> F;
I --> G;
H --> J[Recommended OD, OW, ISS Outputs];
H --> K[Performance Summaries];

```

- **Component Bullets:**

- **Data Ingestion & Preprocessing:** Handles the collection, cleaning, validation, and aggregation of raw input data from various sources.
- **Assortment Optimization (OW & OD):** Core logic for determining the ideal breadth of categories and unique items to stock, based on sales performance and market trends.
- **Size Set Optimization (ISS):** Core logic for calculating the optimal distribution of sizes for individual products based on historical sell-through rates and demand curves.
- **Reporting & Recommendation Generation:** Consolidates the outputs from optimization modules and generates user-friendly reports and actionable recommendations.

- **High-level data sources:**

- Historical Sales Transactions (unit sales, revenue, dates, SKU details, size, color).
- Current Inventory Levels (on-hand, in-transit, warehouse locations).
- Product Master Data (SKU attributes, category hierarchies, cost, retail price).
- Store Performance Data (store type, location, demographics, sales capacity).

4. Configuration & Data Dictionary

Settings Table (args.properties)

| Setting Name | Description | Default Value | Example Value |
|-----------------------------|---|----------------|-----------------|
| min_ros_threshold | Minimum acceptable ROS for a SKU to be considered for OD. | 0.5 units/week | 0.75 units/week |
| max_skus_per_category | Upper limit for OD for any given category. | 200 | 180 |
| size_curve_smoothing_factor | Factor applied to historical size curves to | 0.8 | 0.9 |

| Setting Name | Description | Default Value | Example Value |
|----------------------------|--|---------------|---------------|
| seasonal_demand_adjustment | prevent over-fitting. | 1.0 | 1.2 (for Q4) |
| | Multiplier for sales data during peak seasons. | | |
| inventory_turn_target | Desired inventory turnover rate for planning. | 4.0 | 5.0 |

Shared Data Fields (common-data.json)

| Field Name | Data Type | Description | Used In |
|---------------------|-----------|--|---------------------------------|
| product_id | String | Unique identifier for a product SKU. | All modules |
| category | String | Product category (e.g., “Men’s T-Shirts”). | OW, OD, ISS |
| size | String | Product size (e.g., “S”, “M”, “L”, “XL”). | ISS, Reporting |
| color | String | Product color. | OD, ISS |
| weekly_sales_units | Integer | Average weekly units sold for the product. | ROS, OD, OW |
| current_stock_units | Integer | Current available inventory units. | Reporting |
| store_id | String | Unique identifier for a retail store location. | All modules (if store-specific) |
| sale_date | Date | Date of the sales transaction. | Data Ingestion |
| product_cost | Decimal | Unit cost of the product. | Reporting |

Outputs Catalog

| Output Name | Description | Format | Key Metrics/Recommendations |
|--------------------|--|--------|--|
| recommended_od.csv | Recommended Optimum Depth by Category/SKU. | CSV | product_id, category, recommended_od_units |
| recommended_ow.csv | | CSV | category, recommended_ow_styles |

| Output Name | Description | Format | Key Metrics/Recommendations |
|--------------------------|--|--------|--|
| recommended_iss.csv | Recommended Optimum Width by Category.
Recommended Ideal Size Set by Product. | CSV | product_id, size, recommended_size_allocation_pe |
| performance_summary.xlsx | Consolidated performance metrics and high-level recommendations. | Excel | total_ros_increase, stockout_reduction_pct, inventory_turn_improvement |
| debug_log.txt | Detailed logs for algorithm execution and data processing. | TXT | Errors, warnings, verbose processing s |

5. End-to-End Flow

The otb algorithm executes a sequential pipeline to generate merchandising recommendations: 1. **Data Ingestion:** Raw sales, inventory, and product master data are loaded into the system. 2. **Data Preprocessing:** Data is cleaned, validated, and aggregated (e.g., weekly sales units per SKU) to ensure accuracy and consistency. Missing values are handled, and outliers are identified. 3. **Assortment Optimization (OW & OD):** The algorithm analyzes processed sales data to identify top-performing categories and individual SKUs. It then calculates the OW (ideal number of categories/styles) based on market demand and cannibalization analysis, and the OD (ideal number of unique items) for each category, considering ROS and space constraints. 4. **Size Set Optimization (ISS):** For each identified SKU, the algorithm analyzes historical sales by size to determine the most effective size curve. It then calculates the ISS, recommending the percentage allocation for each size to maximize sales and minimize stockouts. 5. **Recommendation Generation:** The calculated OW, OD, and ISS recommendations are consolidated. Performance metrics, potential impacts (e.g., estimated ROS increase), and visualizations are generated. 6. **Reporting & Output:** Final reports are produced in various formats (CSV, Excel) for easy consumption by stakeholders, along with a detailed log file for transparency and debugging.

6. Submodule Overview

6.1. Data Ingestion & Preprocessing

- **Purpose:** To collect, clean, validate, and transform raw input data into a standardized format suitable for downstream analysis.
- **Inputs:**
 - raw_sales_data.csv: Transactional sales records.
 - raw_inventory_data.csv: Current stock levels by SKU and location.
 - product_master.csv: Product attribute information.

- **Outputs:**
 - `processed_sales_data.csv`: Aggregated weekly sales by SKU, category, and size.
 - `processed_inventory_data.csv`: Current stock levels linked to product attributes.
 - `data_quality_report.txt`: Summary of data issues encountered and resolved.
- **Core Logic Summary:**
 1. Loads data from specified source files, performing schema validation.
 2. Cleanses data by handling missing values, standardizing formats, and correcting common errors.
 3. Aggregates transactional sales data to a weekly ROS per `product_id`, size, and category.
 4. Merges processed sales and inventory data with product master attributes, creating a comprehensive dataset for optimization.

6.2. Assortment Optimization (OW & OD)

- **Purpose:** To determine the ideal breadth of categories (OW) and the optimal number of unique SKUs (OD) within each category to maximize overall profitability and customer appeal.
- **Inputs:**
 - `processed_sales_data.csv`: Weekly sales performance per SKU and category.
 - `args.properties`: Configuration settings like `min_ros_threshold`, `max_skus_per_category`.
- **Outputs:**
 - `recommended_ow.csv`: List of recommended categories/styles and their counts.
 - `recommended_od.csv`: List of recommended SKUs and their optimal depth.
- **Core Logic Summary:**
 1. Analyzes ROS for all categories and individual SKUs over a defined period to identify high- and low-performing segments.
 2. Evaluates category performance for OW recommendations, considering market trends, customer demand, and product cannibalization potential.
 3. Calculates OD for each category by ranking SKUs based on ROS and profitability, while adhering to `max_skus_per_category` and `min_ros_threshold`.
 4. Generates a prioritized list of SKUs and categories for inclusion in the upcoming merchandise assortment.

6.3. Size Set Optimization (ISS)

- **Purpose:** To calculate the ideal combination of sizes (ISS) for each product to align inventory with customer demand and minimize stockouts and overstocks.
- **Inputs:**
 - `processed_sales_data.csv`: Sales data broken down by `product_id` and size.
 - `args.properties`: Configuration settings like `size_curve_smoothing_factor`.

- **Outputs:**
 - `recommended_iss.csv`: Recommended size allocation percentages per product.
- **Core Logic Summary:**
 1. For each `product_id`, aggregates historical sales by size to establish a size-sell-through curve.
 2. Applies a `size_curve_smoothing_factor` to generalize the curve and account for statistical anomalies.
 3. Calculates the ISS by determining the optimal percentage allocation for each size based on its historical contribution to total sales.
 4. Outputs the recommended size distribution, which can be used to inform purchasing and allocation decisions.

6.4. Reporting & Recommendation Generation

- **Purpose:** To compile the optimization results into clear, actionable reports for end-users and to provide an overview of the algorithm’s impact.
- **Inputs:**
 - `recommended_ow.csv`
 - `recommended_od.csv`
 - `recommended_iss.csv`
 - `product_master.csv` (for descriptive attributes)
- **Outputs:**
 - `master_recommendation_report.xlsx`: Comprehensive Excel report with all recommendations.
 - `performance_summary.csv`: Summary KPIs.
 - `debug_log.txt`: Execution and error logs.
- **Core Logic Summary:**
 1. Aggregates recommendations from the OW, OD, and ISS modules into a unified dataset.
 2. Calculates key performance indicators (KPIs) such as estimated ROS increase, stockout reduction, and inventory efficiency gains.
 3. Formats the data into user-friendly tables and potentially charts within an Excel workbook for ease of use.
 4. Generates a detailed log file tracing the execution flow, warnings, and any errors encountered during the entire process.

7. Worked Example: Ideal Size Set (ISS) Calculation

Let’s illustrate how the ISS for a specific product, say “Men’s Basic Tee - Blue”, might be calculated.

Scenario: A retailer wants to determine the ISS for “Men’s Basic Tee - Blue” based on the last 12 months of sales data.

1. Input Data (from `processed_sales_data.csv`): Historical sales for “Men’s Basic Tee - Blue” by size: | Size | Units Sold (Last 12 Months) || :— | :

| | | | | | | | | | | | | |
|-----|--------------|----|---|-----|---|-----|---|-----|----|-----|-----|------|
| | XS | 50 | S | 200 | M | 400 | L | 300 | XL | 150 | XXL | |
| 100 | Total | | | | | | | | | | | 1200 |

2. Core Logic - ISS Calculation:

- **Step 1: Calculate Raw Size Proportions:**

- XS: $50 / 1200 = 0.0417$ (4.17%)
- S: $200 / 1200 = 0.1667$ (16.67%)
- M: $400 / 1200 = 0.3333$ (33.33%)
- L: $300 / 1200 = 0.2500$ (25.00%)
- XL: $150 / 1200 = 0.1250$ (12.50%)
- XXL: $100 / 1200 = 0.0833$ (8.33%)

- **Step 2: Apply Smoothing Factor:** Assume `size_curve_smoothing_factor` from `args.properties` is `0.9`. The smoothed proportion is calculated as: $(\text{raw_proportion} * \text{smoothing_factor}) + (\text{average_proportion} * (1 - \text{smoothing_factor}))$. For simplicity, we'll just apply the smoothing factor directly to the raw proportion and re-normalize, or use a more complex statistical smoothing method internally. For this example, let's just use the raw proportions for direct allocation and note that smoothing would slightly adjust these to prevent over-reliance on a single period's sales.

- **Step 3: Determine Recommended ISS:** The raw proportions directly indicate the ISS for each size. If the retailer plans to buy 1000 units of "Men's Basic Tee - Blue", the ISS would suggest:

- XS: 4.17% of 1000 = 42 units
- S: 16.67% of 1000 = 167 units
- M: 33.33% of 1000 = 333 units
- L: 25.00% of 1000 = 250 units
- XL: 12.50% of 1000 = 125 units
- XXL: 8.33% of 1000 = 83 units

3. Output (from `recommended_iss.csv`):

| product_id | size | recommended_size_allocation_percentage |
|------------------------|------|--|
| Men's Basic Tee - Blue | XS | 4.17% |
| Men's Basic Tee - Blue | S | 16.67% |
| Men's Basic Tee - Blue | M | 33.33% |
| Men's Basic Tee - Blue | L | 25.00% |
| Men's Basic Tee - Blue | XL | 12.50% |
| Men's Basic Tee - Blue | XXL | 8.33% |

This example demonstrates how the ISS module translates historical sales patterns into actionable allocation percentages for future inventory purchases.

8. Debug Guide Highlights

- **Data Validation Errors:** If `data_quality_report.txt` shows high numbers of missing values, incorrect data types, or duplicates, the input data likely needs cleaning at the source. Check the Data Ingestion & Preprocessing module's logs first.
- **Unexpected Recommendations:** If 0D or 0W recommendations seem illogical (e.g., too many low ROS items recommended), verify the `min_ros_threshold` or `max_skus_per_category` settings in `args.properties`.
- **Skewed ISS:** If ISS recommendations heavily favor a single size or seem disconnected from common size curves, examine the `size_curve_smoothing_factor`. A very low factor can lead to over-fitting to historical noise.

- **Performance Issues:** Long execution times might indicate large data volumes or inefficient queries. Check database connections and ensure sufficient computational resources.
- **“No Data Found” Errors:** Ensure all required input files (`raw_sales_data.csv`, `product_master.csv`, etc.) are present and accessible in the configured paths.

9. Common Pitfalls & Remedies

- **Pitfall: Stale or Incomplete Data:** Using outdated sales data or missing key product attributes can lead to inaccurate recommendations.
 - **Remedy:** Implement robust data refresh schedules. Ensure all relevant AGs are included in product master data. Regularly audit data sources for completeness and accuracy.
- **Pitfall: Over-Optimization/Ignoring Business Context:** Purely algorithmic recommendations might miss qualitative aspects (e.g., new trends, marketing pushes, strategic importance of certain items).
 - **Remedy:** Incorporate configurable business rules and allow for manual overrides or adjustments. Emphasize collaboration between technical teams and merchandising experts.
- **Pitfall: Seasonality Not Accounted For:** Recommendations based on average ROS may be inaccurate for highly seasonal products.
 - **Remedy:** Utilize the `seasonal_demand_adjustment` setting in `args.properties`. Implement a time-series analysis that considers historical seasonality patterns during ROS calculation.
- **Pitfall: Cannibalization Issues:** New recommendations might unintentionally reduce sales of existing, profitable items.
 - **Remedy:** The Assortment Optimization module should incorporate cannibalization analysis by grouping similar SKUs and evaluating their collective performance, not just individual ROS.
- **Pitfall: Insufficient Granularity:** Not having ROS data at the size or color level can hinder effective ISS or OD recommendations.
 - **Remedy:** Ensure data ingestion captures sales at the lowest possible AG level for detailed analysis.

10. FAQ

- **Q: How often should the otb algorithm be run?**
 - A: It is recommended to run the otb algorithm weekly or bi-weekly to capture recent sales trends and inventory changes, or before major planning cycles.
- **Q: Can I override the recommendations from the algorithm?**
 - A: Yes, the reports generated by otb are designed to be advisory. Merchandising teams can review and manually adjust recommendations based on qualitative insights or strategic decisions.
- **Q: What if I have limited historical sales data for new products?**
 - A: For new products, the algorithm can use AGs (e.g., similar categories, attributes) from comparable existing products to generate initial estimates. These initial recommendations should be closely monitored and adjusted as real sales data becomes available.

- **Q: How does otb handle promotional periods?**
 - A: The algorithm can be configured to either exclude promotional sales data or apply a normalization factor to prevent skewed ROS calculations from temporary sales spikes. This is handled by the Data Ingestion & Preprocessing module.
- **Q: Is it possible to run this for specific store locations?**
 - A: Yes, if your input data includes store_id, the algorithm can process data and generate recommendations at a per-store level for localized merchandising strategies.

11. Quick Reference Cheat-Sheet

| Abbreviation | Business Definition | Related Module(s) | Key Usage |
|--------------|---|-------------------------------|---|
| OD | Optimum Depth: The ideal number of unique items (SKUs) to stock. | Assortment Optimization | recommended_od_units in output reports. |
| OW | Optimum Width: The ideal number of different categories or styles to offer. | Assortment Optimization | recommended_ow_styles in output reports. |
| ISS | Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts. | Size Set Optimization | recommended_size_allocation_percentage for purchases. |
| ROS | Rate of Sale: The speed at which products are sold over a period (e.g., units/week). | All (calculated in Data Prep) | Core metric for OD, OW and ISS calculations. |
| AG | Attribute Group: A set of product | All (especially Data Prep) | Used for grouping and segmentation in analysis. |

| Abbreviation | Business Definition | Related Module(s) | Key Usage |
|--------------|---|-------------------|-----------|
| | attributes grouped for analysis (e.g., color, size, brand). | | |

Common Commands: * `python otb_main.py --config args.properties`: Run the full algorithm pipeline. * `python otb_data_ingest.py --refresh`: Manually trigger data refresh. * `check_logs.sh`: Script to quickly review `debug_log.txt` for errors.

Key Files: * `args.properties`: Configuration settings. * `common-data.json`: Definition of shared data fields. * `recommended_od.csv`: Output for Optimum Depth. * `recommended_ow.csv`: Output for Optimum Width. * `recommended_iss.csv`: Output for Ideal Size Set. * `master_recommendation_report.xlsx`: Consolidated actionable report.

Chapter 1: OwInputModule

Last Updated: 2025-05-23

1. Business Objective

This module gathers all necessary information to determine the ideal range of categories and styles for our stores, ensuring we offer the right variety of products.

Why This Matters: By providing comprehensive and accurate data, this module ensures we offer the right variety of products, preventing lost sales from too few options and avoiding excess inventory from too many. This directly improves customer satisfaction, enhances sales performance, and optimizes inventory investment.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Performance:** To understand past sales trends, product demand, and ROS for individual items and categories.
- **Average Selling Price (ASP) Data:** To calculate potential revenue and understand the value contribution of different products and styles to overall profitability.

Product Data

- **Ideal Size Set (ISS) Quantities:** To ensure we stock the best combination of sizes for each product, maximizing sales and minimizing stockouts within the OW framework.
- **Product Segmentation Data:** To understand how products are grouped by characteristics (e.g., fashion, basic, seasonal) and performance, which is crucial for defining OW strategies.
- **Previous Optimum Width (OW) Insights:** To leverage past calculations on ideal category width and understand historical style performance, guiding future decisions.

Operational Data

- **Decile Performance Rules:** To apply specific merchandising strategies based on product performance tiers (e.g., top 10% sellers, bottom 20% sellers), which influences how OW recommendations are generated.
- **Annual Operating Plan (AOP) Targets:** To align OW calculations with overall business financial goals, strategic initiatives, and inventory plans for specific periods and stores.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------|---------------|---|--|
| Minimum Style Live Days | 60 Days | The minimum number of days a style must have been available for sale in a store to be considered for OW analysis. Styles live for less than this period are excluded to ensure stable performance data. | Adjust this value when a new product category is introduced and needs faster inclusion, or during clearance periods to focus on older stock. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Input Data] --> B[Main Process]
B --> C[Output Result]
```

1. Start with → Raw Sales, Product Data, and Operational Plans
2. Then → Load and Consolidate All Inputs
3. Next → Organize Data by Key Retail Dimensions (e.g., store, category, time period)
4. Finally → Provide Comprehensive Data Sets for OW Calculations

5. Step-by-Step Actions

1. **Retrieve** the incoming Decile Performance Rules
 - What we do: *We load pre-defined rules that classify products based on their sales performance (e.g., top 10% sellers, bottom 20% sellers).*

- Why we do it: *These rules guide how OW recommendations will adjust for different product performance tiers and ensure strategic alignment.*
- 2. **Gather** Ideal Size Set (ISS) Quantities
 - What we do: *We collect the recommended size distributions and quantities for different product types and categories.*
 - Why we do it: *This ensures OW considers the ideal product composition, not just the number of styles, but also the right sizes within them to maximize sales and minimize stockouts.*
- 3. **Load** Annual Operating Plan (AOP) Targets
 - What we do: *We bring in the high-level business goals and financial targets for sales and inventory for specific periods and stores.*
 - Why we do it: *This data ensures that OW recommendations align with the overarching strategic and financial objectives of the business.*
- 4. **Incorporate** Historical Sales Performance
 - What we do: *We load detailed records of past product Units Sold and ROS, including performance over time.*
 - Why we do it: *This provides the foundational understanding of actual customer demand and product velocity, essential for accurate OW planning.*
- 5. **Process** Product Segmentation Data and Previous Optimum Width (OW) Insights
 - What we do: *We load information about how products are grouped (segmentation) and prior OW calculations, filtering for relevant styles and those that have been live for a sufficient duration, as defined by the Minimum Style Live Days setting.*
 - Why we do it: *This leverages existing product classifications and past analytical results to ensure our OW analysis is consistent, efficient, and builds upon previous insights.*
- 6. **Compile** Average Selling Price (ASP) Data
 - What we do: *We collect the average price at which products have sold, grouped by key attributes like store, period, and attribute group.*
 - Why we do it: *Understanding ASP is crucial for evaluating the revenue potential and profitability of different OW configurations, allowing us to make financially sound decisions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A fashion retailer is analyzing their 'Women's Casual Dresses' category for the upcoming season.
- They have a Minimum Style Live Days setting of 60 days.
- Data shows Style A (Floral Midi Dress) had 90 Units Sold and was live for 120 days.
- Style B (Striped Maxi Dress) had 70 Units Sold but was only live for 45 days.
- The Product Segmentation Data indicates 'Casual Dresses' belong to a 'High-Growth' segment, and Decile Performance Rules might suggest aggressive stocking for this segment.

What Happens

1. First, the system loads all sales, product, and operational data for the 'Women's Casual Dresses' category, including the Minimum Style Live Days setting.
2. Next, it applies the Minimum Style Live Days filter. Style A (Floral Midi Dress) passes because it was live for 120 days (greater than 60 days). Style B (Striped Maxi Dress) is excluded from OW consideration because it was only live for 45 days (less than 60 days). Even though it sold well, it hasn't been available long enough for a stable performance assessment crucial for OW.
3. Then, the module consolidates this filtered data with Product Segmentation Data (High-Growth segment) and Decile Performance Rules for each style, along with ASP data and ISS quantities. This creates a comprehensive and organized picture of the relevant styles and their performance, ready for the OW calculations.

Final Outcome

The OW system receives a clean, filtered dataset containing only relevant and mature product styles, along with their associated sales performance, segmentation, ASP, and operational context. This refined data set is then used to determine the optimal number of casual dress styles for the upcoming season.

Key Takeaway: This module ensures that OW calculations are based on reliable, complete, and relevant data, enabling more accurate and actionable merchandising decisions that drive profitable outcomes.

7. Results

This step produces:

Main Outputs

- **Comprehensive Data Sets for OW Calculation:** Provides all necessary raw sales, ASP, ISS, and product performance data, meticulously organized for the subsequent OW algorithms.
- **Filtered Style Performance Insights:** Offers detailed performance metrics for styles that meet defined criteria (e.g., minimum live days), ensuring only relevant products influence OW decisions and preventing analysis based on unreliable data.

Business Reports

No direct end-user reports are generated by this module, as its primary function is to prepare data for subsequent analysis. However, the data it compiles is critical for generating reports such as:

- **Category Performance Reviews:** Used by merchandisers and planners to understand the overall health, ROS, and profitability of different product categories, directly influenced by the comprehensive input data provided by this module.

- **Style Performance Dashboards:** Used by product managers and planners to evaluate individual style success, identify top/bottom performers, and track sales trends based on the aggregated data.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|---|
| Missing or delayed data feed | The system will pause or fail the run if essential data (e.g., raw sales, AOP targets) is not available, preventing incomplete or incorrect OW calculations. | Contact the IT or Data Engineering team immediately to resolve the data feed issue and ensure data is flowing correctly. |
| No styles meet the Minimum Style Live Days threshold in a category | The OW calculation may proceed with limited or no historical style data for that category, leading to less informed recommendations or a default OW if no valid styles are found. | Review the Minimum Style Live Days setting to ensure it's appropriate for the current product lifecycle, or investigate if data quality is preventing styles from being recognized as "live." |
| Inconsistent or conflicting Product Segmentation Data | Product styles may be incorrectly classified (e.g., a "basic" item labeled "fashion"), leading to OW recommendations that don't align with their true market position or customer behavior. | Consult with the Product Data team or category managers to verify and correct product segmentation definitions in the source systems. |
| Sudden drastic changes in historical sales data (e.g., unusually high or low Units Sold) | Unexpected spikes or drops in ROS due to data errors can skew OW calculations, leading to overstocking (from inflated past sales) or understocking (from underestimated demand). | Review raw sales data for anomalies and consult with the Data Quality team to identify and rectify source data issues. Data cleansing may be needed. |

9. Next Steps

Before This Step

- **Data Extraction and Transformation:** We need raw sales data, product attributes, ISS configurations, and operational targets to be extracted from various source systems and transformed into a standardized format.

- **Previous OD Calculations:** We need the outputs from the Optimum Depth (OD) module, specifically the pre-computed OdRawSalesRow, OdSegmentationRow, OdOwWidthRow, and OdAspRow data, which provide detailed insights into individual SKU performance and category segmentation.

After This Step

- **OW Calculation Engine:** This module uses the compiled, filtered data from the OW Input Module to perform the core OW calculations, determining the optimal number of categories and styles for each store and period.
- **OW Recommendation Module:** This module takes the calculated OW results and translates them into actionable merchandising recommendations for category planners, outlining ideal category and style counts.

10. Common Questions

Q: Why is some of our sales data not included in the OW calculations? A: This module filters out products or styles that haven't been 'live' for a sufficient period, as defined by the Minimum Style Live Days setting. This is done to ensure that OW calculations are based on stable and reliable performance data, avoiding skewed results from very new or short-lived items.

Q: How does this step impact our inventory levels or sales targets? A: By providing accurate and comprehensive input data, this module ensures that subsequent OW calculations are precise. This directly leads to better inventory alignment with customer demand, reduced stockouts, optimized ROS, and ultimately, improved sales performance and profitability.

Q: What if our Product Segmentation Data is wrong? A: Incorrect Product Segmentation Data can lead to OW recommendations that are misaligned with how customers perceive and shop categories. For example, a basic item might be treated as a trendy fashion item. This module relies on the accuracy of this input, so any issues should be corrected at the source.

Q: Can we manually override any of the data loaded here? A: This module primarily loads and filters data. While some downstream processes might allow for manual adjustments or overrides on the final OW *recommendations*, the raw input loaded here reflects the source system data. Any fundamental data corrections or quality issues should occur in the source systems or during the data transformation stage before it reaches this module.

Q: Why do we need data from previous OD calculations? A: The OW module builds upon the insights generated by the OD (Optimum Depth) module. OD provides detailed performance metrics at the individual item (SKU) level, including raw sales, ASP, and segmentation. These granular insights are crucial inputs for determining the broader category and style OW, as they provide context about what performs well within existing structures.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To meticulously gather and prepare all necessary raw and pre-computed data for the Optimum Width (OW) calculations.
- **Main Input:** Historical Sales Performance, Product Segmentation Data, and Operational Targets (AOP, Decile Performance Rules).
- **Main Output:** A comprehensive, organized data set ready for OW analysis, ensuring accuracy and relevance.
- **Warning Signs:** Incomplete data feeds, unexpected drops in Units Sold, or discrepancies in Product Segmentation Data.
- **Key Settings:** Minimum Style Live Days.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 2: OwInitWidthCalc

Last Updated: 2025-05-23

1. Business Objective

This module calculates the initial estimate for the Optimum Width (OW), representing the ideal number of different categories or styles to stock for each sales period, store, and Attribute Group (AG).

Why This Matters: By accurately estimating the initial OW, this module helps merchandisers and planners avoid overstocking or understocking categories, ensuring a balanced product assortment that maximizes sales, minimizes inventory costs, and improves the customer shopping experience.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Style Revenue Per Day:** To rank styles by their daily sales performance, identifying which styles contribute most to revenue within an AG.
- **Historical Style Performance:** To understand past sales trends and the velocity at which individual styles have sold, which is crucial for determining their revenue contribution and overall importance.

Product Data

- **Attribute Group (AG) Classification:** To understand which AG a style belongs to (e.g., “fashion” or “basic”) and group similar styles for analysis, influencing default OW for new introductions.
- **Current or Relevant Width:** To know the existing number of styles offered in an AG, which serves as a baseline for calculating adjustments.

Operational Data

- **Decile Action Rules:** To apply pre-defined strategic guidelines that dictate how much to adjust OW based on the concentration of sales within an AG and its current variety.
- **New Store Identifiers and Characteristics:** To identify stores that lack historical sales data and to find comparable reference stores (e.g., by region, channel) to estimate their initial OW.

- **Fashion Optimum Width Setting:** To provide a default ideal number of styles for new or fashion-focused AGs that lack sufficient historical data for detailed calculations.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-----------------------|---------------|---|---|
| Fashion Optimum Width | 10 | Defines the default ideal number of styles to offer for new product introductions or fashion-oriented AGs when detailed historical data is unavailable. | Adjust this when market trends indicate a need for more or less variety in fashion categories, or when launching new product lines. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

A[Input Data] --> B[Main Process]
B --> C[Output Result]

```

1. Start with → Relevant Styles and their Revenue Per Day
2. Then → Sort Styles, Compute Revenue Contribution, Calculate Sales Concentration (Decile Sum)
3. Next → Apply Strategic Action Weights based on Performance and Current Width
4. Finally → Calculate Initial Optimum Width (OW) for existing and new stores

5. Step-by-Step Actions

1. **Rank** relevant styles
 - What we do: *We sort all styles within each Attribute Group by their revenue per day, from highest to lowest.*
 - Why we do it: *This prioritizes styles based on their sales performance, allowing us to identify the most impactful products.*
2. **Calculate** style revenue contribution
 - What we do: *We determine the percentage of an Attribute Group's total revenue that each individual style contributes.*
 - Why we do it: *This helps us understand the relative importance of each style to the overall success of its category.*

3. **Determine** sales concentration (Decile Sum)

- What we do: *We calculate how many top-performing styles are needed to achieve specific cumulative revenue percentages (e.g., the top 20% of styles generating 80% of revenue).*
- Why we do it: *This provides insight into the “long tail” of sales within an AG, showing how quickly a few top styles capture most of the sales, which guides OW adjustments.*

4. **Apply** action weights

- What we do: *Using the calculated sales concentration and the AG’s current width, we apply pre-defined rules to determine a strategic “action weight.”*
- Why we do it: *This weight acts as a multiplier, guiding whether to expand or contract the OW based on performance and merchandising strategy.*

5. **Estimate** initial Optimum Width (OW)

- What we do: *We multiply the relevant (current) width of an AG by the calculated action weight to get the initial OW estimate.*
- Why we do it: *This provides a data-driven first estimate for the ideal number of styles to offer, aligning with performance and strategy.*

6. **Handle** new stores

- What we do: *For new stores lacking historical data, we calculate their initial OW by averaging the OWs of comparable existing stores in the same region and sales channel.*
- Why we do it: *This ensures new stores receive data-informed OW recommendations even without prior sales history, enabling quicker and more effective initial assortment planning.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A retailer is assessing the OW for their “Women’s Denim Jeans” AG in a specific store for the next period.
- This AG currently offers 10 different styles (relevant width = 10).
- After ranking styles by revenue per day, the system finds that the top 3 styles account for 50% of the AG’s total revenue, and the top 5 styles account for 75% of revenue.
- The Decile Action Rules indicate that for an AG with 10 styles where sales are concentrated (e.g., top 5 styles drive 75% of revenue), the action weight should be 0.8 (suggesting a slight reduction in width).

What Happens

1. First, the system ranks all 10 denim jean styles by their Style Revenue Per Day.
2. Next, it calculates each style’s percentage contribution to the total denim jean revenue for that AG.
3. Then, the system determines the Decile Sum, noting how few styles are responsible for significant portions of cumulative revenue. For example, it confirms that 5 styles contribute 75% of the revenue.

4. Based on the Decile Action Rules for a relevant width of 10 and this level of sales concentration, the system applies an action weight of 0.8.
5. Finally, the initial OW is calculated by multiplying the relevant width (10 styles) by the action weight (0.8), resulting in an initial OW of 8 styles.

Final Outcome

The system recommends an initial OW of 8 styles for “Women’s Denim Jeans” in this store for the upcoming period. This suggests that the retailer can achieve similar sales results with two fewer styles, potentially reducing inventory costs and focusing on higher-performing products.

Key Takeaway: This module provides a data-driven first estimate for optimal category width, allowing merchandisers to quickly identify opportunities to streamline assortments or expand where performance dictates.

7. Results

This step produces:

Main Outputs

- **Initial Optimum Width (OW) for each Attribute Group:** Provides the first calculated estimate for the ideal number of styles to stock per AG, per store, and per period, guiding assortment decisions.
- **Attribute Group Sales Concentration Metrics:** Offers detailed insights into how revenue is distributed across styles within an AG (Decile Sum), revealing the efficiency of the current assortment.
- **Applied Action Weights:** Presents the strategic multipliers used to adjust OW based on performance, offering transparency into the drivers of the OW calculation.

Business Reports

- **Width Optimization Dashboard:** Used by merchandisers and planners to visualize the recommended OW by AG, store, and period, enabling them to make informed decisions about product assortment.
- **Category Performance Deep Dive:** Provides data on sales concentration and style performance, helping product managers understand which AGs have a “long tail” of low-performing styles versus those where a few styles drive most of the sales.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The calculation will pause or fail if essential sales or product classification data is | Contact the IT or Data Engineering team immediately to |

| Situation | What Happens | What You Should Do |
|---|---|--|
| | unavailable, preventing OW computation. | resolve the data feed issue and ensure data is flowing correctly. |
| An Attribute Group has very few or no styles meeting input criteria | The system might calculate a very low or zero initial OW for that AG, or default to a general OW if Decile Action Rules cannot be applied. | Review the performance of styles in that AG and the Minimum Style Live Days setting in the OwInputModule to ensure enough valid styles are considered. |
| Decile Action Rules are missing or misconfigured | The system may apply incorrect action weights, leading to OW recommendations that are too high (overstocking) or too low (lost sales). | Consult with the merchandising strategy team to verify and correct the Decile Action Rules to align with business objectives. |
| New stores have no comparable reference stores | The initial OW for new stores might default to a general Fashion Optimum Width or be uncalculated, potentially leading to suboptimal initial assortments. | Define additional reference store groupings or provide manual overrides for OW in new stores until sufficient historical data is available. |

9. Next Steps

Before This Step

- **OwInputModule:** This module requires comprehensive, filtered data sets containing style sales performance (e.g., Style Revenue Per Day), product classifications, and operational settings (like Decile Action Rules).
- **Previous OD Calculations:** Specific outputs from the Optimum Depth (OD) module, particularly OdOwWidthRow, which provides pre-computed style-level performance metrics grouped by AG.

After This Step

- **OW Correction Module:** This module takes the initial OW calculated here and applies further adjustments based on inventory levels, ROS, and specific business rules to arrive at the final recommended OW.
- **OW Recommendation Module:** This module translates the final OW figures into actionable recommendations for category planners and buyers, outlining the specific number of styles to stock.

10. Common Questions

Q: Why is the recommended OW for my category lower than what we currently stock? A: This module analyzes your sales data and style performance concentration. If a few styles are driving most of the revenue, the system might recommend a lower OW to streamline your assortment, reduce inventory costs, and focus on your top performers.

Q: How does this module handle new products or categories without historical sales? A: For truly new categories or styles, this module leverages product classifications (e.g., “fashion”) to apply a default OW (like Fashion Optimum Width) or, for new stores, it averages OWs from comparable existing stores.

Q: What if our sales data has a lot of promotions or outliers? A: This module relies on the Style Revenue Per Day input from OwInputModule. While this module itself doesn’t cleanse data, any anomalies in the input sales data can affect the calculated OW. Data quality issues should be addressed in upstream modules or source systems.

Q: How do the “Decile Action Rules” influence the OW? A: These rules are strategic guidelines that tell the system whether to expand or contract OW based on how concentrated sales are within a category and its current width. For example, if a category has many styles but only a few drive most sales, the rules might suggest reducing the OW.

Q: Does this module consider our inventory levels? A: No, this module calculates the *initial* OW based primarily on historical sales performance and revenue contribution. Inventory levels and stockout risks are typically considered in subsequent OW adjustment or correction modules, building upon this initial estimate.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate the initial optimum number of styles per Attribute Group based on sales performance and strategic rules.
- **Main Input:** Style Revenue Per Day, Attribute Group classifications, and Decile Action Rules.
- **Main Output:** An initial Optimum Width (OW) for each Attribute Group and metrics on sales concentration.
- **Warning Signs:** Incomplete sales data, misclassified product Attribute Groups, or illogical OW recommendations.
- **Key Settings:** Fashion Optimum Width.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Chapter 3: OwMinMaxDepthModule

Last Updated: 2025-05-23

1. Business Objective

This module determines the actual Units Sold, and benchmarks for maximum and Minimum Sales Threshold quantities for both individual styles and Attribute Groups (AGs). It processes raw sales data, corrects quantities based on how long a product was available for sale, and establishes sales benchmarks, including special handling for new product introductions and new stores.

Why This Matters: By providing accurate and normalized sales quantity benchmarks, this module ensures that OW (Optimum Width) and OD (Optimum Depth) calculations are based on realistic performance. This prevents overstocking by understanding true sales potential and minimizes lost sales by identifying essential Minimum Sales Thresholds, leading to optimized inventory, improved ROS, and a better customer experience.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Performance:** To understand the actual number of units sold for each product, considering specific sales periods and their relevance for determining ROS.

Product Data

- **Product Classification Data:** To accurately group sales data by Attribute Group and style, which is crucial for analysis and for calculating OW at the correct hierarchical level.
- **Style Availability Data:** To know how long a specific style has been actively available for sale in a store, allowing for fair adjustment of Units Sold to a full analysis period.
- **Ideal Size Set (ISS) Data:** To retrieve pre-determined ISS quantities, which often serve as the Minimum Sales Threshold for an Attribute Group to ensure basic stock levels.

Operational Data

- **Minimum Stable Sales Benchmark:** To define the minimum number of days a style must have been available to consider its sales performance stable and reliable for maximum sales quantity identification.
- **Analysis Period Duration:** To provide the total number of days in the target sales period, used for normalizing Units Sold for styles that were not live for the full duration.
- **Store Geographic Data:** To group stores by location (city, region) for identifying comparable stores, especially when estimating sales benchmarks for new introductions or new stores.

Note: See More Details for additional required information.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------|---------------|---|--|
| Minimum Stable Sales Benchmark | 30 Days | The minimum number of days a style must have been actively available for sale in a store to be considered for identifying the Maximum Units Sold benchmark within an Attribute Group. Styles available for less than this period are considered too new for stable performance assessment for a Maximum Units Sold calculation. | Adjust this value if market dynamics require faster inclusion of new styles into sales benchmarks, or if sales data from very new products is consistently unreliable. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Input Data: Raw Sales, Product Details, Store Info] --> B[Main Process: Calculate & Correct Sales Quantities]
B --> C[Output Result: Actual, Max, Min Sales Benchmarks]

1. Start with → Raw Sales Data, Product Classifications, and Store Information
2. Then → Calculate actual Units Sold for each style and correct quantities based on how long each style was “live”
3. Next → Identify maximum and Minimum Sales Threshold benchmarks for Attribute Groups, including special handling for new introductions and new stores
4. Finally → Provide Actual, Maximum, and Minimum sales quantities for each AG to inform OW and OD decisions.

5. Step-by-Step Actions

1. **Gather** Historical Sales Data
 - What we do: *We collect all raw sales data for styles, categorized by specific periods and their business priority, which helps identify relevant sales cycles.*
 - Why we do it: *This establishes the foundational unit sales performance for each product, ensuring we have comprehensive data to begin our analysis.*
2. **Calculate** Actual Units Sold per Style
 - What we do: *We sum up the total Units Sold for each individual style within every store and sales period, filtering out irrelevant items.*
 - Why we do it: *This provides the precise, unadjusted sales performance for each style, giving us a true baseline before any further normalization.*
3. **Correct** Units Sold for Live Days
 - What we do: *We adjust the actual Units Sold for styles that were not available in stores for the entire sales analysis period. This involves scaling their sales to represent what they would have sold if they had been “live” for the full duration.*
 - Why we do it: *This levels the playing field, making sales figures comparable across all styles regardless of their launch date or availability, providing a more accurate ROS equivalent for planning.*
4. **Identify** Maximum Units Sold
 - What we do: *We determine the highest Units Sold achieved by any single style within an Attribute Group for a given sales period, but only considering styles that have been available for a stable period (e.g., longer than the Minimum Stable Sales Benchmark).*
 - Why we do it: *This benchmark identifies the peak sales potential within each AG, showing how well a top-performing style can sell. It helps set ambitious yet realistic sales targets and stocking levels.*
5. **Determine** Minimum Sales Threshold
 - What we do: *We retrieve the Minimum Sales Threshold for each Attribute Group. This value is typically based on pre-defined Ideal Size Set (ISS) quantities, ensuring a basic level of stock.*
 - Why we do it: *This establishes the smallest quantity of items an AG should aim to stock to meet basic customer demand and prevent common stockouts, linking directly to effective ISS planning.*
6. **Estimate** Units Sold for New Introductions
 - What we do: *For new product introductions (new AGs or styles within AGs) that lack sufficient historical sales data, we estimate their Maximum, Minimum*

m, and Actual Units Sold by averaging the performance of similar AGs in comparable existing stores (e.g., in the same city or region).

- *Why we do it: This provides data-informed sales benchmarks for new products from day one, enabling proactive inventory planning and reducing guesswork for novel items.*

7. Estimate Units Sold for New Stores

- *What we do: For stores that are new and thus lack historical sales data, we estimate their Maximum, Minimum, and Actual Units Sold for various Attribute Groups by averaging the performance of those AGs in comparable existing stores (e.g., stores of similar channel or region).*
- *Why we do it: This ensures new stores receive initial sales benchmarks for their assortment planning, allowing them to open with optimized stock levels without waiting for sales history to accumulate.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retailer is evaluating a new Attribute Group called "Lightweight Summer Jackets" for a new store opening in a specific city.
- One style, "Floral Bomber Jacket," was introduced in a comparable existing store 20 days ago (meaning `rawLiveDays` = 20) and sold 50 units. The Analysis Period Duration is 90 days.
- The Minimum Stable Sales Benchmark is set at 30 days.
- The ISS for a "Lightweight Summer Jacket" AG is set at a Minimum Sales Threshold of 8 units.
- The new store has no sales history.

What Happens

1. First, the system gathers raw sales data. The "Floral Bomber Jacket" shows 50 units sold in 20 days.
2. Next, it calculates the actual Units Sold for "Floral Bomber Jacket" as 50.
3. Then, it corrects the Units Sold for "Floral Bomber Jacket" based on its live days: $(90 \text{ Analysis Period Duration} / 20 \text{ rawLiveDays}) * 50 \text{ Units Sold} = 225 \text{ corrected Units Sold}$. This normalized figure represents its potential sales over a full period.
4. However, because the "Floral Bomber Jacket" has only been live for 20 days, which is less than the Minimum Stable Sales Benchmark of 30 days, its sales are *not* used to define the Maximum Units Sold benchmark for the "Lightweight Summer Jackets" AG in this specific calculation step. Only styles that have been live for 30+ days contribute to the Maximum Units Sold figure. If no styles in the AG meet this benchmark, the Maximum Units Sold might temporarily be zero for that AG.
5. The Minimum Sales Threshold for the "Lightweight Summer Jackets" AG is set to 8 units, based on its ISS data.
6. Finally, for the *new store*, since it has no history, the system looks at comparable existing stores in the same city or region. It averages their Actual Units Sold, Maximum Units Sold (from stable styles), and Minimum Sales

Threshold values for the “Lightweight Summer Jackets” AG. For example, if comparable stores show an average Maximum Units Sold of 180 units and Actual Units Sold of 150 units for this AG, these values are assigned to the new store.

Final Outcome

The system provides a normalized Units Sold for newly introduced styles, a defined Minimum Sales Threshold for the AG, and robust sales benchmarks (Actual, Max, Min) for new stores. This allows the retailer to make informed stocking and merchandising decisions for the new store’s opening assortment of “Lightweight Summer Jackets,” even without direct sales history for that specific location.

Key Takeaway: This module ensures that OW and OD decisions are grounded in realistic sales performance, correcting for new items or stores, and providing crucial sales benchmarks for effective inventory planning.

7. Results

This step produces:

Main Outputs

- **Actual Units Sold by Style and Attribute Group:** Provides the precise quantity of units sold for each style and AG, adjusted for their live days, offering a normalized view of sales performance.
- **Maximum Units Sold Benchmark by Attribute Group:** Establishes the highest sales potential for each AG based on the performance of its most successful and stable styles, guiding OD and OW planning.
- **Minimum Sales Threshold by Attribute Group:** Defines the minimum sales quantity expected for each AG, often derived from ISS data, ensuring basic stock levels are maintained.

Business Reports

- **Assortment Health Check:** Used by merchandisers and planners to quickly review the sales performance benchmarks for different AGs, identify top performers, and assess the potential for growth or rationalization.
- **New Product Launch Analysis:** Used by product managers to understand the estimated sales potential of new styles and AGs even before they accumulate significant sales history, aiding in initial stocking and marketing strategies.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The system will pause or fail the run if essential sales or | Contact the IT or Data Engineering team immediately to |

| Situation | What Happens | What You Should Do |
|--|--|--|
| | product classification data is not available, preventing accurate sales quantity calculations. | resolve the data feed issue and ensure data is flowing correctly. |
| No styles meet the Minimum Stable Sales Benchmark in an Attribute Group | The Maximum Units Sold for that AG might be recorded as zero or a default, potentially leading to underestimation of sales potential if all styles in the AG are very new. | Review the Minimum Stable Sales Benchmark setting to ensure it's appropriate for the speed of new product introductions in that AG, or consider historical context if data is very sparse. |
| New stores have no truly comparable reference stores (e.g., unique market) | The system might default to broader averages or fail to assign estimated sales benchmarks, leading to less informed initial inventory for the new store. | Manually define additional reference store groupings or provide manual overrides for initial sales benchmarks until the new store accumulates its own historical data. |
| Ideal Size Set (ISS) data is missing or incomplete for an Attribute Group | The Minimum Sales Threshold for that AG will default to zero, potentially leading to insufficient initial stock and increased stockouts for basic items. | Consult with the Product Data team or category managers to ensure ISS data is complete and accurate for all relevant Attribute Groups. |

9. Next Steps

Before This Step

- **OwInputModule:** This module requires comprehensive, filtered data sets containing raw sales data, product classifications (e.g., Attribute Groups, segmentation), and style availability data.
- **Previous OD Calculations:** This module builds upon pre-computed item-level performance metrics from OD modules, such as raw sales and segmentation data, which are crucial inputs for determining depth benchmarks.

After This Step

- **OW Calculation Engine:** This module uses the Actual, Maximum, and Minimum sales quantities determined here to inform the core OW calculations, ensuring OW is based on realistic sales potential and thresholds.
- **OD Calculation Engine:** The sales benchmarks produced by this module are directly consumed by the OD modules to determine the ideal Optimum Depth (number of unique items) for each Attribute Group or category.

10. Common Questions

Q: Why are the sales numbers for our new products being adjusted? A: New products haven't been in stores long enough to show their full sales potential. This module adjusts their **Units Sold** to reflect what they might sell over a complete sales period, providing a fairer comparison and preventing underestimation of their true ROS for planning.

Q: How does this module help us plan for stores that are just opening? A: For new stores without sales history, this module estimates their **Maximum**, **Minimum**, and **Actual Units Sold** for various product categories by looking at similar, established stores. This allows you to stock the new store optimally from day one, rather than waiting for sales data to accumulate.

Q: What determines the "maximum" sales quantity for a product category? A: The **Maximum Units Sold** for a category (AG) is identified by the highest **Units Sold** achieved by a single, stable product within that category. We only consider products that have been available for at least the **Minimum Stable Sales Benchmark** period to ensure the data is reliable and not skewed by very short-term sales spikes.

Q: What if our **Ideal Size Set (ISS)** data isn't accurate for an **Attribute Group**? A: If the ISS data is incorrect or missing, the **Minimum Sales Threshold** for that **Attribute Group** may be inaccurate. This could lead to stocking too few items, causing frequent stockouts and lost sales, or too many, leading to excess inventory of basic items. Ensure your ISS data is up-to-date.

Q: Why are some sales for a style not included in the **Maximum Units Sold** calculation? A: Sales for styles that haven't been 'live' for a sufficient period, as defined by the **Minimum Stable Sales Benchmark** setting, are excluded from determining the **Maximum Units Sold** benchmark. This is done to ensure that the maximum potential is based on stable and reliable performance data, avoiding skewed results from very new or short-lived items.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate and normalize **Actual**, **Maximum**, and **Minimum** sales quantities for styles and **Attribute Groups**, providing reliable benchmarks for OW and OD planning.
- **Main Input:** Historical Sales Performance, Style Availability Data, Product Classification Data, and **Ideal Size Set (ISS)** Data.
- **Main Output:** **Actual Units Sold** for styles, and **Maximum** and **Minimum Sales Threshold** benchmarks for **Attribute Groups**.
- **Warning Signs:** Incomplete data feeds, unexpected zero **Maximum Units Sold** for established categories, or illogical sales estimates for new stores.
- **Key Settings:** **Minimum Stable Sales Benchmark**.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Chapter 4:

OwBuyWidthAdjustmentModule

Last Updated: 2025-05-23

1. Business Objective

This module refines the predicted sales for each Attribute Group (AG) and individual style, adjusting these sales quantities based on established minimum and maximum sales benchmarks, and ultimately determines the precise Optimum Width (OW) by relating predicted total sales to these revised sales per style.

Why This Matters: By fine-tuning the OW recommendations, this module prevents overstocking of slow-selling styles and understocking of popular ones. This leads to better inventory turnover, reduced markdowns, improved profitability, and ensures our stores offer the most relevant and appealing product assortment to customers.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Revenue Contribution per Attribute Group:** To understand each AG's expected share of total revenue, which is essential for calculating its predicted sales and distributing it across styles.
- **Average Selling Price (ASP) per Attribute Group:** To convert expected revenue targets into predicted unit sales quantities for AGs, ensuring unit-level accuracy in OW calculations.
- **Maximum Units Sold Benchmark:** To provide an upper limit for individual style sales predictions, ensuring we don't plan for unrealistic stocking levels or overstock beyond peak performance.
- **Minimum Sales Threshold:** To provide a lower limit for individual style sales predictions, guaranteeing that every chosen style meets a basic sales expectation to avoid stockouts of essential items.

Product Data

- **Attribute Group (AG) Sales Insights:** To provide detailed segmentation and preliminary width estimates for each AG, serving as the foundational data for calculating predicted sales per style.

Operational Data

- **Annual Operating Plan (AOP) Revenue Targets:** To align predicted sales with overall business financial goals and growth expectations for specific periods and stores.
- **Initial Optimum Width (OW) Estimates:** To provide the preliminary OW for each AG, serving as the baseline for calculating predicted sales per style before further adjustments.
- **New Store Identifiers:** To accurately flag stores without historical sales data, requiring sales estimation based on comparable existing stores for initial OW planning.
- **Store Characteristics:** To identify comparable stores (e.g., by channel, region) for new stores, enabling data-driven sales estimation when historical data is absent.

Note: See More Details for additional required information.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------------|---------------|--|--|
| Minimum Reference Store Count | 3 | The minimum number of comparable reference stores required to reliably estimate predicted sales for new stores. If fewer than this number are found in the same region and channel, the system expands the search to just the same region. | Adjust this value if a higher confidence in new store sales estimates is required (increase the count), or if data sparsity prevents sufficient comparable stores from being found (decrease the count). |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Input Data: Predicted Sales, Sales Benchmarks, Store Info]

--> B[Main Process: Revise Sales, Calculate Final OW]
B --> C[Output Result: Final OW, Revised Sales per Style]

1. Start with → Predicted sales per Attribute Group, initial OW estimates, and sales benchmarks (max/min units sold) for each AG and style.
2. Then → Calculate predicted sales per style, estimate these sales for new stores using comparable data, and revise all sales quantities based on the maximum and minimum sales benchmarks.
3. Next → Determine the final Optimum Width (OW) for each AG by dividing the total predicted sales by the revised sales quantity per style.
4. Finally → Provide the determined final Optimum Width, ensuring a precise and optimized product variety.

5. Step-by-Step Actions

1. **Calculate** predicted sales for each Attribute Group
 - What we do: *We determine the total expected sales units for each AG by using its revenue contribution, the overall Annual Operating Plan (AOP) revenue target, and the Average Selling Price (ASP).*
 - Why we do it: *This establishes the overall sales potential for each product category, providing a crucial top-down view for OW planning.*
2. **Estimate** predicted sales per style
 - What we do: *We take the total predicted sales for an AG and divide it by the AG's initial Optimum Width (OW) estimate, to get an initial predicted sales quantity for an average style within that AG.*
 - Why we do it: *This gives us a starting point for how many units each style is expected to sell, enabling us to gauge individual style performance within a category.*
3. **Estimate** predicted sales per style for new stores
 - What we do: *For new stores lacking historical sales data, we estimate their predicted sales per style by averaging the predicted sales per style from comparable existing stores in the same region and channel.*
 - Why we do it: *This ensures new stores receive data-informed OW recommendations from day one, allowing for effective initial assortment planning without waiting for sales history to accumulate.*
4. **Revise** sales quantity per style
 - What we do: *We adjust the predicted sales per style by ensuring it falls between the established Minimum Sales Threshold and Maximum Units Sold benchmark for its AG. If the predicted sales are too low, they are raised to the minimum; if too high, they are capped at the maximum.*
 - Why we do it: *This step ensures that our sales expectations for each style are realistic, preventing planning for unrealistically high sales or stockouts due to underestimation.*
5. **Determine** the final Optimum Width (OW)
 - What we do: *We calculate the final OW for each AG by dividing its total predicted sales (from step 1) by its revised sales quantity per style (from step 4).*
 - Why we do it: *This final calculation provides the most precise number of styles needed, reflecting both the AG's overall sales potential and the realistic sales performance of an individual style within it, optimizing inventory and variety.*

6. Store the calculated results

- What we do: *We save the final OW, predicted sales per Attribute Group, predicted sales per style, and revised sales per style for use in subsequent modules.*
- Why we do it: *This makes the refined OW and sales data available for downstream processes like detailed assortment planning and inventory allocation.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retailer is planning the OW for the “Women’s Fashion Blouses” Attribute Group in a specific store for the upcoming season.
- The system has predicted total sales for this AG to be 1,500 units for the period.
- The Initial Optimum Width estimate for this AG was 15 styles.
- The Maximum Units Sold benchmark for a single blouse style in this AG is 120 units (based on top performers).
- The Minimum Sales Threshold for a blouse style is 50 units (based on Ideal Size Set requirements for basic stock).
- A new store is opening and needs an OW for its “Women’s Fashion Blouses” category, but has no sales history.

What Happens

1. First, the system calculates the predicted sales per style for the existing store:
 $1,500 \text{ total units} / 15 \text{ initial styles} = 100 \text{ units per style}.$
2. Next, for the new store, since it lacks history, the system attempts to find comparable existing stores. It looks for stores in the same region and channel. If it finds at least 3 comparable stores (based on the Minimum Reference Store Count setting), it averages their predicted sales per style for “Women’s Fashion Blouses.” If fewer than 3 are found, it broadens the search to all stores in the same region. Let's say the average predicted sales per style from comparable stores for “Women’s Fashion Blouses” is 90 units. This value is assigned to the new store.
3. Then, for the existing store's predicted 100 units per style, the system revises this quantity:
 - Is 100 less than the Minimum Sales Threshold (50)? No.
 - Is 100 greater than the Maximum Units Sold benchmark (120)? No.
 - So, the revised sales quantity per style for the existing store remains 100 units.

4. Finally, the final OW for the existing store is calculated: 1,500 total units / 100 revised units per style = 15 styles. (In this specific case, the OW did not change from the initial estimate because the predicted sales per style fell within the min/max benchmarks.)
- *Alternative scenario:* If the predicted sales per style had been 130 units, it would be capped at the Maximum Units Sold of 120. The final OW would then be $1500 / 120 = 12.5$, rounded to 13 styles, leading to a narrower assortment.
 - *Alternative scenario:* If the predicted sales per style had been 40 units, it would be raised to the Minimum Sales Threshold of 50. The final OW would then be $1500 / 50 = 30$ styles, leading to a broader assortment to reach the total sales goal.

Final Outcome

The system provides a precise Optimum Width for the “Women’s Fashion Blouses” AG (e.g., 15 styles for the existing store, or a relevant OW for the new store based on its estimated sales potential). This allows merchandisers to confidently plan the exact number of blouse styles to stock, aligning with realistic sales potential and ensuring optimal variety.

Key Takeaway: This module ensures that OW recommendations are realistic and actionable by adjusting style-level sales expectations to account for minimum stock needs and maximum sales potential, enabling effective inventory and assortment planning.

7. Results

This step produces:

Main Outputs

- **Predicted Sales per Attribute Group:** Provides the total anticipated unit sales for each AG, crucial for high-level assortment planning and financial forecasting.
- **Predicted Sales per Style:** Offers the initial expected unit sales for an average style within an AG, used as a baseline before revision.
- **Revised Sales Quantity per Style:** Presents the adjusted, realistic sales expectation for an average style within an AG, capped by maximum potential and floored by minimum thresholds.
- **Final Optimum Width (OW):** Delivers the precise, data-driven recommendation for the ideal number of styles to offer within each AG for every store and period.

Business Reports

- **OW Recommendation Dashboard:** Used by merchandisers and planners to view the recommended OW by AG, store, and period, allowing them to make informed decisions about product assortment breadth.

- **Assortment Planning Report:** Provides detailed insights into the predicted and revised sales quantities per style, supporting buyers in selecting the right number of styles to meet customer demand and sales targets.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|--|
| Missing or delayed data feed | The system will pause or fail the run if essential data, such as Annual Operating Plan targets, Average Selling Prices, or sales benchmarks, is not available. | Contact the IT or Data Engineering team immediately to resolve the data feed issue and ensure data is flowing correctly. |
| Average Selling Price (ASP) is zero or unavailable for an AG | The system will issue a warning and may calculate an inflated predicted sales quantity for that AG, as it cannot correctly convert revenue to units. | Investigate the source of the missing or zero ASP data. This typically indicates an issue in upstream data collection or calculation, which needs to be corrected. |
| No comparable stores for a new store meet Minimum Reference Store Count criteria | The system might broaden its search for reference stores (e.g., to the entire region), potentially using less precise data, or may calculate a less accurate OW if no valid reference stores are found. | Manually define additional reference store groupings or provide manual overrides for OW in new stores until sufficient historical data is available. |
| Maximum Units Sold benchmark is zero or very low for an AG | If all styles within an AG have very low or no Maximum Units Sold defined (e.g., if all styles are too new to have stable sales), the revised sales quantity may be limited, potentially leading to an OW that is too broad. | Review the Minimum Stable Sales Benchmark setting in OwMinMaxDepthModule to ensure it's appropriate, or investigate data quality issues in OwMinMaxDepthModule if an AG truly has no top performers. |
| Minimum Sales Threshold is zero or very high for an AG | If the Minimum Sales Threshold is zero, the revised sales quantity per style could be too low, leading to an OW that is too narrow. If it's too high, it could inflate OW unnecessarily. | Consult with the Product Data team or category managers to verify and correct Ideal Size Set (ISS) data, which typically drives the Minimum Sales Threshold. |

9. Next Steps

Before This Step

- **OwInputModule:** This module relies on the comprehensive, filtered data sets prepared by `OwInputModule`, including raw sales, product attributes, and operational settings.
- **OwInitWidthCalc:** This module provides the crucial `Initial Optimum Width` estimates for each `Attribute Group`, serving as the baseline for the predicted sales per style calculation.
- **OwMinMaxDepthModule:** This module delivers the essential `Maximum Units Sold` benchmarks and `Minimum Sales Thresholds` for each `Attribute Group`, which are vital for revising the predicted sales quantities per style.

After This Step

- **OW Recommendation Module:** This module takes the final OW results and translates them into actionable merchandising recommendations for category planners, outlining the ideal number of styles to stock.
- **Inventory Planning and Allocation Systems:** The refined OW figures and revised sales per style are critical inputs for systems that manage inventory levels and allocate products to stores, ensuring stock aligns with optimal variety.

10. Common Questions

Q: Why did the recommended OW for my category change even though the total sales prediction didn't? A: This module adjusts the OW by revising the *sales quantity per style*. If your predicted sales per style fell below a `Minimum Sales Threshold` or above a `Maximum Units Sold` benchmark, the system adjusts that per-style expectation, which in turn alters the overall OW needed to meet the total sales target.

Q: How does this module help us plan for products in new stores? A: For new stores without sales history, this module estimates their sales potential per style by looking at how similar `Attribute Groups` perform in comparable existing stores. This allows you to plan an informed and optimized OW for new store openings from day one.

Q: What happens if a style in a category is incredibly popular and sells much more than typical? A: If a style's predicted sales per style would be unusually high, this module caps that expectation at the `Maximum Units Sold` benchmark for its `Attribute Group`. This prevents overstocking based on an outlier and ensures the OW calculation remains realistic and sustainable.

Q: Why do we have a `Minimum Sales Threshold` for a style? Doesn't that force us to stock more styles than necessary? A: The `Minimum Sales Threshold` ensures that every style within the OW has a basic expected sales quantity, often tied to `Ideal Size Set` needs. This prevents the system from recommending an OW so narrow that you can't adequately stock even essential sizes, which would lead to frequent stockouts and disappointed customers.

Q: Can this module account for future trends or marketing pushes? A: This module's calculations are based on historical data, AOP targets, and defined sales benchmarks. While it doesn't directly predict future trends, the Annual Operating Plan (AOP) Revenue Targets and potentially adjusted Maximum Units Sold benchmarks (from upstream modules) can incorporate business strategies and expected uplifts from marketing.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To refine style sales predictions and determine the precise Optimum Width (OW) for each Attribute Group.
- **Main Input:** Predicted sales, initial OW estimates, Maximum Units Sold benchmarks, and Minimum Sales Thresholds.
- **Main Output:** The final Optimum Width for each Attribute Group and revised sales quantities per style.
- **Warning Signs:** Unusually high or low OW recommendations, zero ASP warnings, or issues with new store OW calculations.
- **Key Settings:** Minimum Reference Store Count.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 5: OwExitAgModule

Last Updated: 2025-05-23

1. Business Objective

This module identifies product Attribute Groups (AGs) that are planned for exit (meaning their Optimum Width is zero) and intelligently reallocates their planned sales quantity (Open-to-Buy or OTB) to other, continuing AGs.

Why This Matters: By efficiently redistributing the sales potential from exiting product groups, this module ensures that inventory investment and sales targets are fully met, preventing lost sales from unallocated budgets and optimizing the product assortment to focus on profitable, in-demand items. This maximizes revenue potential, improves inventory efficiency, and keeps stores stocked with relevant products.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Predicted Sales Quantity per Attribute Group:** To know the total sales units that were originally planned for each AG, which represents the OTB quantity available for redistribution.

Product Data

- **Attribute Group (AG) Classification Data:** To identify which category, price bucket, and product theme each AG belongs to, allowing for redistribution at relevant hierarchical levels (e.g., within the same category, or same store).
- **Calculated Optimum Width (OW) for each AG:** To determine which AGs have an OW of zero, signifying they are marked for exit and their OTB needs to be reallocated.
- **Product Theme Data:** To identify special product themes like “Core Bestsellers” that might have specific rules for their minimum OW (e.g., always stocking at least one style), even if they would otherwise be considered for exit.

Operational Data

- **Rollup Levels for Redistribution:** To define the specific levels (like category-price bucket, category, or store) at which OTB quantities should be reallocated, ensuring that funds are intelligently shifted within relevant product groupings.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------|
| N/A | N/A | There are no specific configurable business settings directly within this module. Its operations are driven by upstream OW calculations and classification data. | N/A |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Input Data: Calculated OW, Predicted Sales, AG Classifications] --> B[Main Process: Identify Exit AGs & Redistribute OTB]

B --> C[Output Result: Revised OTB, Final OW]

1. Start with → Calculated OW values, predicted sales quantities, and detailed Attribute Group classifications.
2. Then → Identify all Attribute Groups marked for exit (where OW is zero) and collect their total predicted sales (OTB).
3. Next → Redistribute the collected OTB from exiting AGs to non-exiting AGs, first within the same category-price bucket, then within the same category, and finally within the same store.
4. Finally → Provide the updated sales quantities (OTB) for all continuing AGs and the final OW values, adjusting for any special “Core Bestseller” rules.

5. Step-by-Step Actions

1. **Identify** exiting Attribute Groups (AGs)
 - What we do: *We review all AGs and flag those that have a final Optimum Width (OW) of zero, indicating they are no longer to be stocked.*
 - Why we do it: *This isolates the product groups whose planned sales quantity (OTB) needs to be reallocated.*
2. **Redistribute** OTB within Category-Price Buckets
 - What we do: *For each exiting AG, we sum up its planned sales quantity and distribute it proportionally among other non-exiting AGs within the same specific category and price range.*
 - Why we do it: *This ensures that sales potential is first reallocated to the most relevant and similar product groups, keeping the assortment focused.*
3. **Redistribute** OTB within Categories
 - What we do: *For any remaining exiting AGs whose OTB couldn't be fully reallocated at the category-price bucket level, we then distribute their planned sales quantity proportionally among non-exiting AGs within the broader category (across different price buckets).*
 - Why we do it: *This provides a second level of reallocation, allowing for broader adjustments within the same product type.*
4. **Redistribute** OTB within Stores
 - What we do: *For any still-unallocated OTB from exiting AGs, we finally distribute it proportionally among all non-exiting AGs within the same store.*
 - Why we do it: *This acts as a final safety net, ensuring all planned sales quantities are utilized and contribute to the store's overall performance, even if the original product group is no longer stocked.*
5. **Adjust** final Optimum Width for “Core Bestsellers”
 - What we do: *We ensure that any AG identified as a “Core Bestseller” has a minimum OW of at least one, even if it was initially marked for exit due to other calculations.*
 - Why we do it: *This protects the availability of essential, always-in-stock items, ensuring that critical products remain on shelves regardless of overall OW optimizations.*
6. **Store** the revised OTB and final OW
 - What we do: *We save the updated planned sales quantities for each AG (after redistribution) and the definitive final OW values for all AGs.*
 - Why we do it: *This makes the optimized sales plans and assortment widths available for subsequent modules, like inventory allocation and merchandising reports.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retailer is reviewing the OW for their “Women's Scarves” category.
- The OW calculation has determined that the “Silk Scarves” Attribute Group should have an OW of 0, meaning it's marked for exit. Its predicted sales quantity (OTB) was 200 units for the upcoming period.

- Within the same store, the “Women’s Scarves” category also has a “Cashmere Scarves” AG (predicted sales 300 units, OW not zero) and a “Wool Scarves” AG (predicted sales 500 units, OW not zero), both in the same ‘Luxury’ price bucket.
- There are also other non-scarf AGs in the store, like “Women’s Hats” and “Women’s Gloves.”

What Happens

1. First, the system identifies “Silk Scarves” as an exiting AG because its final OW is 0. Its OTB of 200 units is flagged for redistribution.
2. Next, the system attempts to redistribute the 200 units within the same category and price bucket (CATEGORY_PRICE_BUCKET_LEVEL). It finds “Cashmere Scarves” (300 units) and “Wool Scarves” (500 units) in the same “Women’s Scarves” category and ‘Luxury’ price bucket.
3. The total OTB for these non-exiting AGs in this group is $300 + 500 = 800$ units.
4. The 200 units from “Silk Scarves” are proportionally distributed:
 - “Cashmere Scarves” gets $(300/800) * 200 = 75$ units, increasing its predicted sales to 375 units.
 - “Wool Scarves” gets $(500/800) * 200 = 125$ units, increasing its predicted sales to 625 units.
5. Since the entire 200 units were reallocated at the CATEGORY_PRICE_BUCKET_LEVEL, no further redistribution is needed at the CATEGORY_LEVEL or STORE_LEVEL for this specific exiting AG.
6. Finally, the system confirms the final OW for “Silk Scarves” is 0. If “Cashmere Scarves” or “Wool Scarves” were “Core Bestsellers,” their OW would be adjusted to a minimum of 1 if it somehow became 0.

Final Outcome

The planned sales quantity (OTB) of 200 units from the exiting “Silk Scarves” AG is successfully reallocated to related, continuing AGs within the same category and price bucket. This ensures the retailer’s total planned sales for the “Women’s Scarves” category remain achievable, even with the reduction in variety. The inventory budget for “Silk Scarves” is now efficiently repurposed to strengthen the stock of “Cashmere Scarves” and “Wool Scarves,” focusing investment on higher-performing items.

Key Takeaway: This module ensures that planned sales quantities are fully utilized by intelligently reallocating resources from discontinued product lines to active ones, optimizing inventory and maximizing revenue potential.

7. Results

This step produces:

Main Outputs

- **List of Exiting Attribute Groups:** Identifies all product groups that will no longer be stocked, providing clarity for merchandising and inventory teams.

- **Revised Predicted Sales Quantity per Attribute Group:** Provides the updated planned sales units for each continuing AG, reflecting the additional OTB redistributed from exiting AGs.
- **Level of Redistribution for Exiting AGs:** Indicates at which hierarchical level (category-price bucket, category, or store) an exiting AG's OTB was successfully reallocated, offering insight into the reallocation strategy.
- **Final Optimum Width (OW):** Presents the definitive ideal number of styles for each AG, ensuring essential "Core Bestseller" items retain a minimum OW of one.

Business Reports

- **Assortment Rationalization Report:** Used by category managers and planners to understand which AGs are being exited and where their planned sales quantities have been reallocated, supporting strategic decisions about product portfolio management.
- **OTB Reallocation Summary:** Used by financial planners and buyers to track how sales budgets have been adjusted and re-prioritized across product categories, ensuring financial targets are met through optimized inventory investment.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|--|
| Missing or delayed data feed | The system will pause or fail the run if essential input data, such as calculated OW values or AG classification details, is not available, preventing accurate reallocation. | Contact the IT or Data Engineering team immediately to resolve the data feed issue and ensure data is flowing correctly. |
| An exiting AG has no non-exiting AGs to redistribute to at any level (e.g., entire category is exiting) | The planned sales quantity (OTB) for that exiting AG may not be fully reallocated, potentially leading to an unspent portion of the sales budget or a shortfall in overall store-level sales targets. | Review the OW calculations for the broader categories/stores to understand why no non-exiting AGs are available. This might indicate a significant strategic shift in product offerings requiring manual budget adjustments. |
| Incorrect AG classification data (e.g., wrong category or price bucket) | OTB from exiting AGs might be reallocated to inappropriate or irrelevant continuing AGs, leading to inefficient inventory investment and misalignment with customer demand. | Consult with the Product Data team or category managers to verify and correct Attribute Group classification definitions in the source systems. |

| Situation | What Happens | What You Should Do |
|--|---|--|
| “Core Bestseller” is incorrectly classified or missing its OW rule | An essential “Core Bestseller” AG might incorrectly be assigned an OW of zero, leading to potential stockouts of critical, high-demand items that should always be available. | Verify “Core Bestseller” definitions and ensure the Product Theme Data is accurate and consistently applied across all relevant AGs in the source systems. |

9. Next Steps

Before This Step

- **OwInputModule:** This module relies on the comprehensive and filtered input data, including raw sales, ASP, and product classifications, which are foundational for all OW calculations.
- **OwInitWidthCalc:** This module provides the initial estimates for OW based on sales concentration, guiding subsequent calculations.
- **OwMinMaxDepthModule:** This module provides the Maximum Units Sold and Minimum Sales Thresholds benchmarks, crucial for establishing realistic sales expectations per style.
- **OwBuyWidthAdjustmentModule:** This module calculates the final OW and predicted sales per AG/style before this module’s adjustments, setting the stage for identifying exit AGs and their OTB.

After This Step

- **Inventory Planning and Allocation Systems:** The revised predicted sales quantities and final OW values are critical inputs for systems that manage inventory levels and allocate products to stores, ensuring stock aligns with the optimized assortment.
- **Assortment Management Tools:** Merchandisers and planners use these outputs in their tools to make final decisions on product variety and depth, streamlining the product offering and improving category performance.

10. Common Questions

Q: Why are we moving sales budgets from one product group to another? A: This module handles product groups (Attribute Groups) that the system has determined should no longer be stocked (their Optimum Width, or OW, is zero). Instead of losing the sales potential of these exiting products, we intelligently reallocate their planned sales quantity (Open-to-Buy or OTB) to other, active product groups. This ensures we fully utilize our inventory budget and invest in products that are still in demand.

Q: How does the system decide where the sales quantity goes? A: The system tries to keep the reallocation as relevant as possible. It first looks to redistribute the sales quantity to similar product groups within the same category and price range. If that's not possible, it then looks within the broader category, and finally, if needed, across all other product groups within the same store. This ensures efficient investment.

Q: What if a popular item is marked for exit? Will it be removed from stores? A: This module applies a special rule for "Core Bestseller" items. Even if initial calculations suggest an OW of zero, this module will ensure that "Core Bestseller" Attribute Groups retain an OW of at least one. This means essential, high-demand items will always be stocked to prevent customer disappointment and lost sales.

Q: Can this module lead to some product groups suddenly having a much larger stock? A: This module reallocates the *planned sales quantity* (OTB). While it increases the sales target for continuing Attribute Groups, downstream modules will still consider inventory constraints and realistic sales potential. The goal is to ensure all planned sales are assigned, allowing for a more efficient overall inventory rather than necessarily overstocking specific items.

Q: Why do we care about "rollup levels" for redistribution? A: By redistributing at different "rollup levels" (like category-price bucket, then category, then store), we ensure that the planned sales quantity from an exiting product group is reallocated to the most appropriate and similar products first. This strategy maintains logical assortments, prevents random allocations, and ensures that the financial potential of the exiting product is absorbed by the most relevant continuing products.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To reallocate planned sales quantities (OTB) from exiting product Attribute Groups (AGs) to continuing AGs, ensuring efficient utilization of inventory budgets.
- **Main Input:** Calculated OW values, predicted sales quantities (OTB), and Attribute Group classification data.
- **Main Output:** A revised sales quantity (OTB) for continuing AGs and the final OW values, ensuring "Core Bestsellers" remain stocked.
- **Warning Signs:** Unallocated OTB from exiting AGs, or incorrect reallocations due to inaccurate product classification.
- **Key Settings:** No direct business settings, relying on upstream configurations and product data.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Chapter 6: OwOutputModule

Last Updated: 2025-05-23

1. Business Objective

This module generates and persists the final Optimum Width (OW) output, meticulously collecting all computed OW metrics and related sales data into structured formats for reporting and further analysis.

Why This Matters: By providing a consolidated, comprehensive view of OW recommendations and their underlying metrics, this module enables merchandisers, planners, and analysts to confidently evaluate assortment decisions, track performance against targets, and perform deep dives into sales concentration and inventory utilization. This ultimately leads to more effective product strategies, optimized inventory investment, and improved profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Predicted Sales Quantity per Attribute Group:** To record the total sales units anticipated for each Attribute Group (AG) as determined by previous calculations, representing its planned sales budget.
- **Revised Open-to-Buy (OTB) Quantities:** To include the final planned sales units for each AG after any reallocation of budget from exiting product groups.

Product Data

- **Final Optimum Width (OW):** To store the definitive, calculated ideal number of different styles for each AG, derived from all previous OW calculation and adjustment steps.
- **Attribute Group (AG) Classification Data:** To provide descriptive details for each AG (e.g., brand, category, gender, price bucket), essential for contextualizing the output and enabling detailed analysis.

Operational Data

- **Store Characteristics:** To include relevant store information (e.g., store code, region, channel) for each output row, allowing for granular analysis of OW recommendations across the retail network.

- **Reallocation Level Data:** To indicate how planned sales from exiting AGs were reallocated (e.g., within the same category or at the store level), providing transparency into budget redistribution.

Note: See More Details for additional required information.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------|
| N/A | N/A | There are no specific configurable business settings directly within this module. Its operations are driven by upstream OW calculations and classification data. | N/A |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Input Data: All Calculated OW Metrics] --> B[Main Process: Collect & Structure Data]
    B --> C[Output Result: Final OW Reports]
  
```

1. Start with → All interim and final OW calculations and related sales metrics from preceding modules.
2. Then → Collect these metrics for each unique combination of period, store, and Attribute Group.
3. Next → Structure the collected data into detailed output rows, including both summary and denormalized formats.
4. Finally → Persist these structured output rows to the database for reporting, analysis, and downstream consumption.

5. Step-by-Step Actions

1. **Collect** all computed OW metrics for each Attribute Group
 - What we do: *We gather the initial OW estimates, final OW recommendations, predicted sales quantities, revised sales per style, and minimum/maximum sales benchmarks from all preceding OW calculation modules.*

- Why we do it: *This brings together all the necessary data points that contribute to the final OW decision for comprehensive output.*
- 2. **Consolidate** Attribute Group exit flags and reallocation levels
 - What we do: *We check if an Attribute Group was marked for exit (meaning its OW was set to zero) and note the level at which its planned sales quantity was reallocated to other AGs.*
 - Why we do it: *This provides crucial context on how unallocated budgets were handled and helps merchandisers understand changes in assortment breadth.*
- 3. **Populate** structured output rows for summary reporting
 - What we do: *We create OwOutputRow entries, filling each row with the consolidated OW metrics (initial/final OW, OTB quantities, sales benchmarks, exit flags, ASP, etc.) for each Attribute Group.*
 - Why we do it: *This organizes the core OW decision-making data into a clear, row-based format suitable for high-level summaries and analysis.*
- 4. **Include** denormalized output rows for detailed analysis
 - What we do: *We generate DenormOwOutputRow entries by joining the OW metrics with rich descriptive data from master tables, such as store details, AG classifications (e.g., brand, category, subcategory), and style performance indicators.*
 - Why we do it: *This creates comprehensive data sets ready for direct consumption by business intelligence tools, enabling deep dives and complex reporting without needing further data joins.*
- 5. **Persist** the final OW data
 - What we do: *We save both the OwOutputRow and DenormOwOutputRow collections to the database, ensuring they are readily available for users and downstream systems.*
 - Why we do it: *This makes the definitive OW recommendations and supporting metrics accessible for critical business decisions, inventory planning, and performance monitoring.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- For the upcoming season, the 'Women's Casual Dresses' Attribute Group in a specific store has gone through all OW calculations.
- Its initial OW was 10, the final OW is 8.
- The predicted total sales for this AG were 1,500 units, but after internal adjustments, the revised quantity per style became 125 units.
- The Maximum Units Sold benchmark for a single dress in this AG was 180 units, and the Minimum Sales Threshold was 50 units.
- Another AG, 'Men's Formal Vests,' was marked for exit (final OW of 0), and its planned sales budget was reallocated to 'Men's Formal Jackets' at the category level.

What Happens

1. First, the system collects all these calculated values for ‘Women’s Casual Dresses,’ including its initial OW (10), final OW (8), predicted sales (1,500 units), revised sales per style (125 units), Maximum Units Sold (180 units), Minimum Sales Threshold (50 units), and its ASP. It notes that this AG was *not* marked for exit.
2. Next, for ‘Men’s Formal Vests,’ the system records its final OW as 0 (exit flag activated), its initial predicted sales, and crucially, notes that its redistribution level was ‘Category Level’ to ‘Men’s Formal Jackets’.
3. Then, this module compiles these details into OwOutputRow objects for each AG.
4. Concurrently, it creates DenormOwOutputRow objects by enriching these AG-level OW metrics with detailed descriptive data from the store, AG, and style master data. For example, the ‘Women’s Casual Dresses’ row would include the store’s region, the brand of the dresses, the subcategory, and key style performance metrics like Style Revenue Per Day from OD calculations.

Final Outcome

The system produces comprehensive, structured output tables containing all final OW decisions, sales quantities, and supporting data for every Attribute Group across all stores and periods. For instance, a merchandiser can now easily retrieve a report showing that ‘Women’s Casual Dresses’ are recommended to have 8 styles, alongside their sales potential and performance benchmarks. They can also see that ‘Men’s Formal Vests’ are no longer to be stocked and their budget was effectively reallocated, ensuring no lost sales opportunity.

Key Takeaway: This module provides the definitive, centralized record of all OW recommendations and their underlying metrics, crucial for informed assortment planning and inventory management.

7. Results

This step produces:

Main Outputs

- **OwOutputRow:** A summary table containing the final Optimum Width (OW) for each Attribute Group (AG), alongside key metrics like initial OW, predicted and revised sales quantities, Minimum and Maximum Units Sold benchmarks, ASP, and an exit flag.
- **DenormOwOutputRow:** A denormalized, comprehensive table that combines all OW metrics with detailed descriptive attributes (e.g., store details, brand, category, gender, style-level performance), designed for direct consumption by business intelligence tools and advanced reporting.

Business Reports

- **OW Decision Report:** Used by category managers and planners to review the final recommended OW for each AG, understand the rationale, and approve assortment breadth strategies.

- **Assortment Performance Dashboard:** Used by senior merchandisers and analysts to monitor the effectiveness of OW decisions, analyze sales concentration, and identify opportunities for further optimization across categories and stores.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|--|
| Missing or delayed data feed | The system will pause or fail the run if essential calculated data from preceding OW modules is not available, preventing a complete and accurate output. | Contact the IT or Data Engineering team immediately to resolve the data feed issue and ensure all upstream data is flowing correctly. |
| Mismatched keys or missing Attribute Group data during denormalization | If an AG or style ID in the OW calculations cannot be matched to its descriptive data in master tables, the denormalized output may have missing or incorrect attribute information. | Review the consistency of IDs across all source systems and master data. Work with data stewards to correct any discrepancies in product classification data. |
| Illogical OW recommendations or sales quantities in the output | While this module primarily collects data, if the final OW or sales figures seem incorrect, it indicates an issue in one of the upstream calculation modules. | Review the outputs and process flows of the preceding <code>OwInitWidthCalc</code> , <code>OwMinMaxDepthModule</code> , <code>OwBuyWidthAdjustmentModule</code> , and <code>OwExitAgModule</code> to identify the root cause of the error. |

9. Next Steps

Before This Step

- **OwInputModule:** Provides the foundational raw sales, product, and operational data that all OW calculations rely upon.
- **OwInitWidthCalc:** Delivers the initial estimates for OW based on sales concentration and historical performance.
- **OwMinMaxDepthModule:** Establishes critical Maximum Units Sold and Minimum Sales Threshold benchmarks for styles and AGs.
- **OwBuyWidthAdjustmentModule:** Calculates the final OW and revised sales quantities per style after applying sales limits and targets.
- **OwExitAgModule:** Identifies AGs for exit and reallocates their planned sales quantities, influencing the final OW and OTB figures.

After This Step

- **Business Intelligence (BI) Tools:** The comprehensive `OptimumWidthOutputRow` and `DenormOptimumWidthOutputRow` tables are consumed by BI tools to generate dashboards and reports for business users.
- **Assortment Planning Systems:** Merchandisers and planners use the final `OptimumWidth` recommendations to make decisions about the number of categories and styles to stock in stores.
- **Inventory Allocation Systems:** The detailed sales quantities and `OptimumWidth` figures are used to guide the distribution of inventory to stores, ensuring optimal stock levels.

10. Common Questions

Q: Why is this module important if all the calculations are done upstream? A: While the calculations happen elsewhere, this module is crucial because it consolidates *all* the final results into a single, structured, and easy-to-use format. It's the definitive output that decision-makers and other systems rely on for consistent and accurate `OptimumWidth` planning.

Q: What kind of information can I typically see in the final output from this module? A: You'll see the initial recommended width and the final recommended width for each product category (AG), along with the predicted sales quantities, the minimum and maximum sales potential for a single style, and an indicator if a product category is planned for exit.

Q: How does this output help us track performance over time? A: By providing both the `OptimumWidth` recommendations and detailed sales metrics for each period, this module allows you to compare current season plans with previous ones. This helps in understanding trends, evaluating the effectiveness of your `OptimumWidth` strategies, and refining future assortment decisions.

Q: What does the "exit flag" mean in the output? A: The "exit flag" indicates whether a particular `Attribute Group` (product category) has been determined to have an `OptimumWidth` of zero, meaning it's planned for discontinuation or removal from the assortment. This flag helps identify categories that will no longer be stocked.

Q: Can this output be used for other reports beyond `OptimumWidth`? A: Absolutely! The `DenormOptimumWidthOutputRow` is specifically designed to be rich in detail, combining `OptimumWidth` metrics with descriptive attributes like store location, product brand, and style performance. This makes it a powerful dataset for various analyses, including category performance reviews, store assortment comparisons, and long-tail sales analysis.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To generate and store the final `OptimumWidth` (`OptimumWidth`) results and all supporting metrics in a structured format.

- **Main Input:** All calculated OW metrics, sales quantities, and classification data from preceding modules.
 - **Main Output:** OwOutputRow (summary OW decisions) and DenormOwOutputRow (detailed, denormalized OW data).
 - **Warning Signs:** Missing data in the output, or inconsistent product attributes.
 - **Key Settings:** No direct business settings within this module.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

ow FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ document provides quick answers to common business questions about the Optimum Width (OW) algorithm and its processes.

General

Q: What is the primary goal of the OW algorithm? **A:** The primary goal of the Optimum Width (OW) algorithm is to determine the ideal number of different categories or styles to stock, optimizing product assortment to maximize sales efficiency and inventory productivity.

Q: How does Optimum Width (OW) benefit our business? **A:** OW helps merchandisers and planners optimize assortment decisions, ensuring we offer the right breadth of products to meet customer demand while minimizing unnecessary inventory. This leads to improved profitability, reduced waste, and enhanced customer satisfaction.

Q: What exactly does “Optimum Width (OW)” mean in this context? **A:** OW (Optimum Width) refers to the ideal number of distinct categories, styles, or Attribute Groups (AGs) we should offer. It focuses on the variety and breadth of our product selection.

Q: Is OW related to other optimization concepts like OD or ISS? **A:** Yes, OW (Optimum Width) focuses on the ideal number of categories/styles, while OD (Optimum Depth) focuses on the ideal number of unique items (SKUs) within those categories/styles, and ISS (Ideal Size Set) focuses on the best combination of sizes. They are all interconnected components of a comprehensive assortment optimization strategy.

Q: Who primarily uses the results of the OW algorithm? **A:** The primary users of the OW algorithm’s results are merchandisers, planners, and inventory managers. They use the insights to inform their buying strategies, assortment plans, and product allocation decisions.

Configuration & Settings

Q: Where can I find the key settings that influence OW calculations? **A:** Key configuration settings, such as decile action rules and specific weighting factors, are managed internally within the system. For specific details or any adjustment requests, please refer to the system’s configuration guide or contact the support team.

Q: Can we adjust the sensitivity of the OW calculation for certain product categories or Attribute Groups (AGs)? **A:** Yes, the algorithm incorporates parameters like decile action rules, which can be configured to influence how different styles contribute to the width calculation based on their revenue performance and strategic importance. This requires system-level configuration.

Q: What is an “action weight” and how does it affect OW? **A:** Action weights are part of the decile action rules. They are applied during the initial Optimum Width (OW) calculation by the `OwInitWidthCalc` module and influence how product styles are prioritized based on their revenue contribution, helping to fine-tune the breadth of the assortment.

Q: Are there different settings or logic for new stores compared to existing stores? **A:** Yes, the `OwInitWidthCalc` module specifically handles initial width calculations for new stores. This often involves applying different logic or default assumptions due to the limited or absence of historical sales data for these locations.

Q: How are our Annual Operating Plan (AOP) targets used in the OW calculation? **A:** AOP targets are loaded by the `OwInputModule` and provide critical financial goals and constraints. These targets help ensure that the calculated Optimum Width (OW) aligns with our overall business objectives and financial plans.

Data & Inputs

Q: What kind of data does the OW algorithm require to run? **A:** The OW algorithm requires various inputs, including raw sales data (units/week or ROS), segmentation data, Annual Operating Plan (AOP) targets, decile action rules, and Ideal Size Set (ISS) quantities. All this data is loaded via the `OwInputModule`.

Q: Where does the “raw sales” data come from, and how is it used? **A:** Raw sales data, representing product sales over a period (Rate of Sale or ROS), is a fundamental input. It's used by the `OwMinMaxDepthModule` to determine actual minimum and maximum sales benchmarks for styles and Attribute Groups (AGs).

Q: What are “decile action rules” and why are they important for OW? **A:** Decile action rules define how styles are weighted and prioritized based on their revenue performance within specific deciles (e.g., top 10%, next 10%). They are critical for the `OwInitWidthCalc` module to accurately determine the initial Optimum Width (OW).

Q: How does segmentation data play a role in the OW calculation? **A:** Segmentation data helps group products or stores based on specific characteristics (e.g., climate, customer demographics). This allows the OW calculation to be tailored and optimized for these specific segments, leading to more accurate and relevant assortment recommendations.

Q: Is historical sales data always required for OW calculation? **A:** While historical sales data is crucial for established products and stores, the system includes special logic within modules like `OwMinMaxDepthModule` to handle new product introductions or new stores. In these cases, it may use proxy data, default assumptions, or attribute similarities where historical data is limited.

Process & Logic

Q: How does the algorithm determine the initial Optimum Width (OW) for products? **A:** The `OwInitWidthCalc` module calculates the initial Optimum Width (OW) for each period, store, and Attribute Group (AG). It achieves this by sorting relevant styles by revenue, computing their revenue contribution, determining decile sums, and applying pre-defined action weights.

Q: How are the minimum and maximum sales quantities for a style or Attribute Group (AG) determined? **A:** The `OwMinMaxDepthModule` processes raw sales data, correcting quantities based on the number of live days a product was available. It then identifies the minimum and maximum sales benchmarks, including special considerations for new introductions and new stores, to understand the sales range.

Q: What happens during the “buy width adjustment” step in the process? **A:** The `OwBuyWidthAdjustmentModule` calculates predicted sales for each Attribute Group (AG) and style. It then revises these predicted sales quantities based on the identified min/max sales benchmarks. Finally, it determines the ultimate Optimum Width (OW) by relating the predicted total sales to the revised sales per style.

Q: What happens if an Attribute Group (AG) is marked for exit? **A:** If an Attribute Group (AG) has a final Optimum Width (OW) of zero, the `OwExitAgModule` identifies it for exit. Its associated Open-to-Buy (OTB) quantity is then strategically redistributed to non-exit AGs at various rollup levels, such as category-price bucket, category, or store level, to ensure efficient reallocation of budget.

Q: What is the typical order of operations for the OW algorithm? **A:** The process generally follows a linear pipeline: It begins with input data loading (`OwInputModule`), followed by initial width calculation (`OwInitWidthCalc`), then min/max sales determination (`OwMinMaxDepthModule`), buy width adjustment (`OwBuyWidthAdjustmentModule`), handling of exited AGs (`OwExitAgModule`), and finally, output generation (`OwOutputModule`) for reporting and further analysis.

Outputs & Results

Q: What kind of output does the OW algorithm produce? **A:** The `OwOutputModule` generates comprehensive output, including initial and final Optimum Width (OW) metrics, Open-to-Buy (OTB) quantities, determined min/max sales benchmarks, and flags for Attribute Groups (AGs) that are marked for exit.

Q: Can I see the intermediate steps or raw calculations of the OW determination? **A:** The primary output focuses on the final, actionable Optimum Width (OW) results. While intermediate data is used internally for computation, the `OwOutputModule` primarily provides the final metrics. Specific intermediate details might be available for diagnostic purposes upon request from the support team.

Q: How are the final OW results delivered or made available for use? **A:** The `UtilOutputSyncModule` is responsible for uploading or synchronizing the final computed output data to its designated destination. This ensures the results are readily available for downstream systems, reporting dashboards, or planning tools.

Q: Is a snapshot of the output data preserved after a run? **A:** Yes, the `UtilOutputSnapshotModule` captures a snapshot of the current state of the output data. This snapshot is typically taken before any final synchronization steps and is valuable for auditing purposes or if a rollback to a previous state is ever required.

Q: Does the output explain why certain Attribute Groups (AGs) were marked for exit? **A:** The output identifies AGs marked for exit (i.e., those with a final OW of zero) and details how their Open-to-Buy (OTB) quantity was redistributed. The underlying reason for an AG being exited is typically due to its predicted sales and revised sales not justifying a width greater than zero after the buy width adjustment process.

Error Handling & Edge Cases

Q: What happens if critical input data is missing or incomplete? **A:** Missing critical input data, such as complete raw sales records, AOP targets, or segmentation data, would likely lead to errors in the `0wInputModule`. The system is designed to validate inputs and will flag such issues, preventing subsequent calculations until the data is corrected.

Q: How does the system handle new product introductions that have no historical sales data? **A:** For new product introductions without historical sales, the `0wMinMaxDepthModule` and other relevant modules have special logic. They may use proxy data from similar products, default assumptions, or attribute similarities to establish initial sales benchmarks and incorporate these new items into the Optimum Width (OW) calculation.

Q: What if predicted sales for an Attribute Group (AG) are extremely low or zero? **A:** If the predicted sales for an Attribute Group (AG) are very low, the `0wBuyWidthAdjustmentModule` may determine that the Optimum Width (OW) for that AG should be zero. This decision then triggers the `0wExitAgModule` to redistribute any associated Open-to-Buy (OTB) quantity.

Q: Are there any scenarios where Open-to-Buy (OTB) redistribution might not occur as expected? **A:** OTB redistribution relies on the presence of non-exit Attribute Groups (AGs) at various rollup levels. If, in an extreme case, an entire category or store segment has all its AGs marked for exit, redistribution within that specific scope might be limited, potentially requiring manual intervention or reallocation at a higher organizational level.

Q: What if there's a significant, unexpected spike or drop in the raw sales data? **A:** The system utilizes historical sales data to determine min/max sales benchmarks and predict future sales. While the algorithm aims for robustness, extreme and uncharacteristic anomalies in raw sales data could impact the accuracy of these benchmarks and subsequent OW calculations. Data quality checks are performed during the input phase to mitigate such issues.

Need More Help? Please contact your IT/Data Science support team for further assistance.

ow Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

The Optimum Width (OW) algorithm is a crucial component for optimizing product assortment, ensuring retailers stock the ideal number of different categories or styles. By precisely calculating the **OW**, the system empowers businesses to maximize sales, reduce excess inventory, and efficiently reallocate open-to-buy (OTB) quantities, directly impacting profitability and market responsiveness.

3. Architecture Overview

graph TD

```
A[OwInputModule] -->|data_flow: raw & computed data| B(OwInitWidthCalc)
A -->|data_flow: sales, segmentation, style data| C(OwMinMaxDepthModule)
A -->|data_flow: AOP, ASP, segmentation| D(OwBuyWidthAdjustmentModule)
A -->|data_flow: foundational data| F(OwOutputModule)

B -->|data_flow: agInitialWidth| D
B -->|data_flow: agInitialWidth, OwLongTailRow| F

C -->|data_flow: maxQtySold, minQtySold| D
C -->|data_flow: minQtySold, maxQtySold, actualQtySold| F

D -->|data_flow: finalWidth, predictedSalesMap| E(OwExitAgModule)
D -->|data_flow: predictedSalesMap, salesPerStyle, revisedSalesPerStyle| F

E -->|data_flow: exitAgs, revOtbQtyMap, exitAgLevelMap, finalWidth| F

F --> G(UtilOutputSnapshotModule)
G --> H(UtilOutputSyncModule)

A ---|depends_on| B
A ---|depends_on| C
B ---|depends_on| D
C ---|depends_on| D
```

D ---|depends_on| E
E ---|depends_on| F
F ---|depends_on| G
G ---|depends_on| H

```
linkStyle 0 stroke:blue,stroke-width:2px;  
linkStyle 1 stroke:blue,stroke-width:2px;  
linkStyle 2 stroke:blue,stroke-width:2px;  
linkStyle 3 stroke:blue,stroke-width:2px;  
linkStyle 4 stroke:blue,stroke-width:2px;  
linkStyle 5 stroke:blue,stroke-width:2px;  
linkStyle 6 stroke:blue,stroke-width:2px;  
linkStyle 7 stroke:blue,stroke-width:2px;  
linkStyle 8 stroke:blue,stroke-width:2px;  
linkStyle 9 stroke:blue,stroke-width:2px;  
linkStyle 10 stroke:blue,stroke-width:2px;
```

```
linkStyle 11 stroke:green,stroke-width:2px;  
linkStyle 12 stroke:green,stroke-width:2px;  
linkStyle 13 stroke:green,stroke-width:2px;  
linkStyle 14 stroke:green,stroke-width:2px;  
linkStyle 15 stroke:green,stroke-width:2px;  
linkStyle 16 stroke:green,stroke-width:2px;  
linkStyle 17 stroke:green,stroke-width:2px;  
linkStyle 18 stroke:green,stroke-width:2px;
```

- **OwInputModule:** This module is responsible for loading all raw and pre-computed data necessary for the Optimum Width (OW) calculations.
- **OwInitWidthCalc:** This module calculates the initial Optimum Width (OW) for each period, store, and Attribute Group (AG) by analyzing revenue contributions and applying action weights.
- **OwMinMaxDepthModule:** This module determines the actual, maximum, and minimum quantities sold for styles and Attribute Groups (AGs) by processing raw sales data.
- **OwBuyWidthAdjustmentModule:** This module calculates predicted sales, revises them based on min/max benchmarks, and determines the final Optimum Width (OW) for each style.
- **OwExitAgModule:** This module identifies Attribute Groups (AGs) with a final Optimum Width of zero and efficiently redistributes their Open-to-Buy (OTB) quantity to other non-exit AGs.
- **OwOutputModule:** This module gathers all computed OW metrics and generates the final structured output for reporting and further analysis.
- **UtilOutputSnapshotModule:** This module captures a snapshot of the processed output data, typically for auditing or rollback purposes before final synchronization.
- **UtilOutputSyncModule:** This module uploads or synchronizes the final computed output data to its designated destination, making the results available for downstream systems.

4. Prerequisites & Setup

To run the ow pipeline, ensure the following prerequisites are met:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Build Tool:** Apache Maven (version 3.6.3 or higher) is used for project management and dependency resolution.

Build Commands:

1. **Navigate to the project root:**

```
cd ow/
```

2. **Clean and Install Dependencies:** This compiles the project and installs necessary dependencies.

```
mvn clean install
```

How to Run:

The ow pipeline is orchestrated by a ModuleGroup runner. After building the project, execute the main application class, typically with configuration parameters.

```
java -jar target/ow-1.0.jar --config=./args.properties
```

5. Folder & File Structure

```
ow/
├─ module-group/           # Contains the orchestrator for
                             running the OW pipeline modules.
├─ submodules/             # Houses the individual business
                             logic modules (e.g., OwInputModule, OwInitWidthCalc).
├─ args.properties         # Configuration file containing
                             global parameters for the OW calculations.
├─ common-data.json        # Defines the schema for shared data
                             objects used across modules.
└─ outputs/                # Directory where the final computed
                             OW results and reports are stored.
```

6. Global Configuration

The args.properties file is used to configure global parameters for the OW calculations.

| Setting Name | Default Value | Description | Valid Range/When to Change |
|---|---------------|-------------|----------------------------|
| <i>(No global configuration parameters)</i> | | | |

| Setting Name | Default Value | Description | Valid Range/When to Change |
|----------------------------|---------------|-------------|----------------------------|
| <i>provided in input.)</i> | | | |

7. Shared Data Model

The `common-data.json` schema defines the structure of data objects exchanged between modules.

| Field | Type | Description |
|--|------|-------------|
| <i>(No common-data.json schema provided in input.)</i> | | |

8. Submodule Chapters

8.1 OwInputModule

Module Name & Business Objective

The **OwInputModule** serves as the initial entry point for the **Optimum Width (OW)** pipeline. Its core business objective is to **load all necessary raw and pre-computed input data** from various sources. This ensures that subsequent modules have the comprehensive and accurate information required to perform **OW** calculations, such as sales trends, historical **OW** values, and product master data, enabling informed assortment decisions.

Required Information

- **Sales Data:**
 - `OdRawSalesRow`: Detailed raw sales transactions, including units sold, revenue, and dates.
- **Product Data:**
 - `OdOwWidthRow`: Historical **OW** values and style-level input data.
 - `OwDecileActionRow`: Rules for decile-based actions (e.g., width adjustments).
 - `OdSegmentationRow`: Product segmentation data (e.g., by category, price bucket, consumer group).
 - `AopRow`: Annual Operating Plan data, including planned sales and margins.
 - `OdAspRow`: Average Selling Price data for various product attributes.
 - `SizeSetQtyRow`: Quantities associated with **Ideal Size Sets (ISS)**.
 - `AgRow`: Attribute Group (AG) definitions and master data.
- **Operational Data:**
 - `StoreRow`: Store master data, including new store flags or operational status.

Business Settings

- *(No specific business settings for this module beyond data source configurations.)*

Process Flow

graph TD

```
A[Load Sales Data] --> B[Load Product Data]
B --> C[Load Operational Data]
C --> D[Populate OwData Object]
```

Step-by-Step Actions

1. **Retrieve** raw sales data (OdRawSalesRow) from the data lake, *ensuring comprehensive historical sales context.*
2. **Load** product master data (OdOwWidthRow, OwDecileActionRow, OdSegmentationRow, AgRow) from master data systems, *providing core product attributes and historical performance.*
3. **Fetch** financial planning data (AopRow, OdAspRow) and size set quantities (SizeSetQtyRow), *crucial for financial projections and inventory depth planning.*
4. **Ingest** store-specific information (StoreRow), *accounting for unique store characteristics like new openings or remodels.*
5. **Aggregate and Structure** all loaded data into the OwData object, *creating a unified, accessible data model for subsequent processing modules.*

Sample Scenario

A clothing retailer is preparing for the Spring 2026 season. The **OwInputModule** is triggered to gather all relevant information. It loads 18 months of raw sales data for all apparel categories, decile action rules for “Tops” and “Bottoms,” product segmentation for “Premium Casual” and “Value Basics,” and **Average Selling Price (ASP)** data for all styles. It also pulls the **Annual Operating Plan (AOP)** for 2026, targeting a 10% growth in the “Denim” category.

*Key Takeaway: The module ensures all necessary inputs are gathered and prepared, acting as the foundation for accurate **OW** calculations, much like gathering all ingredients before baking.*

Results

The **OwInputModule** populates the OwData object, a central data repository containing: * Maps of aspMap, prevSeasonWidth, agInputKeySet. * Raw sales, segmentation, AOP, and ASP data objects. * Various lists of input rows (e.g., OdOwWidthRow, OwDecileActionRow).

Unusual Cases

- **Missing or delayed data feed:** If OdRawSalesRow or OwDecileActionRow data is incomplete or late, the subsequent **OW** calculations will be based on stale or partial information, leading to inaccurate width recommendations.
- **Corrupted input files:** Malformed AopRow or OdAspRow files can cause parsing errors, halting the pipeline and preventing any **OW** calculations.

Next Steps

The OwData object, now fully populated, is passed to the OwInitWidthCalc, OwMinMaxDepthModule, OwBuyWidthAdjustmentModule, and OwOutputModule for initial width calculation, min/max sales determination, buy width adjustments, and final output generation, respectively.

Common Questions

- 1. What happens if a key input file is missing? *The process will typically fail early, logging an error, as core data is absent for calculations.*
- 2. How often should the input data be refreshed? *Ideally, daily or weekly, depending on the business's agility needs, to ensure the most current sales and inventory data is used.*
- 3. Can I manually override any input data? *Not directly within this module; overrides would typically happen upstream in the data source systems before ingestion.*
- 4. What is the impact of outdated segmentation data? *Outdated segmentation can lead to **OW** recommendations that don't align with current product groupings or customer preferences.*
- 5. How is data validated for accuracy? *Basic structural validation occurs during loading, but deep business logic validation happens in downstream modules.*

Quick Reference

| Metric | Description | Source | Output to |
|------------|---|---------------------------|--------------------|
| Input Data | Raw sales, product attributes, financial plans, historical OW | Data Lake, Master Systems | OwData object |
| Objective | Data Loading & Preparation | N/A | Subsequent Modules |

8.2 OwInitWidthCalc

Module Name & Business Objective

The **OwInitWidthCalc** module calculates the **initial Optimum Width (OW)** for each unique combination of period, store, and **Attribute Group (AG)**. Its primary business objective is to establish a foundational **OW** value by analyzing revenue contribution and applying specific decile action rules, which then serves as a baseline for further adjustments and provides an initial guidance on how many styles an **AG** should carry.

Required Information

- **Sales Data:**
 - OwData: Contains OdOwWidthRow (style-level input, potentially with historical OW), OdRawSalesRow (for revenue calculations).
- **Product Data:**
 - OwData: Includes OwDecileActionRow (rules for width adjustments based on deciles), OdSegmentationRow (for AG definitions), AgRow (AG master data).
- **Operational Data:**
 - OwData: Contains StoreRow (especially for identifying new stores).

- **Cache:** For retrieving AgRow details efficiently.

Business Settings

- **Decile Action Rules:** Configurable rules (from OwDecileActionRow) that dictate how many styles to add or remove based on an **AG**'s revenue decile performance.
- **New Store OW Default:** A default initial **OW** value or methodology for newly opened stores that lack historical sales data.

Process Flow

graph TD

```

A[Load OwData] --> B{Is New Store?}
B -- Yes --> C[Apply New Store OW Logic]
B -- No --> D[Sort Styles by Revenue]
D --> E[Calculate Revenue Contribution]
E --> F[Determine Decile Sums]
F --> G[Apply Decile Action Weights]
G --> H[Derive Initial OW]
C --> H
H --> I[Store Initial OW in OwData]

```

Step-by-Step Actions

1. **Retrieve** all relevant input data from the OwData object, *preparing for the core calculation.*
2. **Identify** new stores based on their StoreRow properties, *allowing for specific initial width handling where historical data is absent.*
3. **For existing stores:**
 - **Sort** all styles within each **Attribute Group (AG)** by their total revenue, *identifying top-performing styles.*
 - **Compute** the cumulative revenue contribution for each style, *understanding its impact on the AG's overall performance.*
 - **Determine** decile sums for the **AG**, *categorizing its performance into ten equal groups.*
 - **Apply** predefined decile action weights from OwDecileActionRow, *adjusting the initial width based on the AG's performance tier.*
4. **For new stores:** **Apply** a default initial **OW** calculation logic (e.g., using category averages or comparable store data), *providing an educated starting point for assortment.*
5. **Derive** the initial **OW** for each period, store, and **AG**, *establishing the baseline number of styles.*
6. **Store** the calculated agInitialWidth in the OwData object, *making it available for subsequent modules.*
7. **Persist** OwLongTailRow data to the database, *providing insights into less active styles.*

Sample Scenario

For a “Women’s Denim Jeans” **AG** in a specific store: 1. Styles are sorted by revenue: Style A (\$50K), Style B (\$30K), Style C (\$15K), Style D (\$5K). 2. The module calculates revenue contributions and deciles. 3. Based on

OwDecileActionRow, if this **AG** falls into the 7th decile (strong performance), the rule might be “add 1 style” to its current **OW**. If it’s a new store, it might default to an **OW** of 3. 4. The module determines the agInitialWidth as, say, 4.

Key Takeaway: This module establishes a data-driven starting point for the ideal number of styles (OW), balancing historical performance with defined business rules, rather than relying on gut feeling.

Results

The **OwInitWidthCalc** module computes and stores: * agInitialWidth for each period, store, and **AG** in OwData. * OwLongTailRow data (persisted to database).

Unusual Cases

- **Missing or delayed data feed:** If OwDecileActionRow is missing, the module might default to generic calculations or fail, leading to non-optimized initial widths.
- **No sales history for an AG:** For truly new product launches without any sales data, the module must fall back on predefined default **OW** values or category averages, potentially leading to less precise initial widths.
- **Incorrect decile action rules:** Misconfigured decile action rules can lead to aggressive overstocking or understocking of styles, directly impacting sales and inventory efficiency.

Next Steps

The agInitialWidth calculated by this module is consumed by the OwBuyWidthAdjustmentModule to further refine style-level predicted sales and subsequently determine the final **OW**. It’s also used by the OwOutputModule for final reporting.

Common Questions

1. How is a “new store” defined for initial **OW** calculation? *Typically by a flag in the StoreRow or by having zero historical sales data within a defined period.*
2. What if an **AG** doesn’t have enough styles to form deciles? *The module will likely use a simpler approach, such as applying a fixed initial **OW** or averaging existing styles.*
3. Can we fine-tune the decile action rules? *Yes, the OwDecileActionRow is designed to be configurable to reflect different business strategies.*
4. How does initial **OW** relate to **Optimum Depth (OD)**? *Initial **OW** is the starting point for the number of styles (width), while **OD** focuses on the ideal quantity of each unique item within that style (depth).*
5. What impact does a high initial **OW** have? *A high initial **OW** could lead to over-assortment if not appropriately adjusted later, increasing inventory risk.*

Quick Reference

| Metric | Description | Calculation Basis | Output to |
|----------------|---------------------------------|--|------------------------|
| agInitialWidth | Initial Optimum Width for an AG | Revenue deciles, action rules, new store logic | OwData, OwOutputModule |
| Objective | | N/A | N/A |

| Metric | Description | Calculation Basis | Output to |
|--------|------------------------------|-------------------|-----------|
| | Baseline OW
Determination | | |

8.3 OwMinMaxDepthModule

Module Name & Business Objective

The **OwMinMaxDepthModule** is responsible for establishing actual, minimum, and maximum quantities sold for styles and **Attribute Groups (AGs)**. Its business objective is to **provide critical benchmarks for sales performance**, specifically minQtySold and maxQtySold, which are essential for validating and revising predicted sales in subsequent modules. This ensures that sales forecasts are realistic and align with historical selling ranges, preventing extreme over or under-predictions.

Required Information

- **Sales Data:**
 - OwData: Contains OdRawSalesRow (raw sales transactions), OdSegmentationRow (for AG definitions), OdOwWidthRow (style-level input).
- **Product Data:**
 - Cache: For retrieving AgRow and StoreRow details.

Business Settings

- **Live Days Threshold:** A configurable minimum number of “live days” a style must have been active to be considered for min/max sales calculation.
- **New Introduction Handling:** Rules for how min/max sales are estimated for newly introduced styles with limited sales history.
- **New Store Sales Adjustment:** Logic for adjusting sales data for new stores (e.g., ramping up period).

Process Flow

graph TD

```

A[Load OwData & Cache] --> B[Process Raw Sales Data]
B --> C[Correct Quantities by Live Days]
C --> D{Is New Introduction or New Store?}
D -- Yes --> E[Apply Special Min/Max Logic]
D -- No --> F[Determine Min/Max Sales Benchmarks]
F --> G[Identify Actual Quantities Sold]
E --> G
G --> H[Store Min/Max/Actual in OwData]
```

Step-by-Step Actions

1. **Load** raw sales data (OdRawSalesRow) and other relevant data from OwData and Cache, *preparing for deep sales analysis.*
2. **Process** raw sales transactions to aggregate quantities sold per style, period, and store, *providing the foundation for min/max determination.*

3. **Correct** quantities sold based on “live days”, normalizing sales for styles that weren’t available for the entire period.
4. **Identify** new introductions and new stores, as these require specialized logic for setting min/max benchmarks due to limited history.
5. **For new introductions/stores: Apply** specific estimation methods (e.g., using comparable AG performance or category averages) to derive reasonable min/max sales.
6. **For established styles/stores: Determine** the *minQtySold* and *maxQtySold* by analyzing historical sales data, *establishing the realistic sales range*.
7. **Calculate** the *actualQtySold* for each style and AG, *providing the current sales reality*.
8. **Store** *minQtySold*, *maxQtySold*, and *actualQtySold* in the *OwData* object, *making these critical benchmarks available for the next module*.

Sample Scenario

For “Women’s Graphic T-Shirt” Style X: 1. Raw sales show 100 units sold over 60 days. The module corrects this to an effective 120 units if the style was “live” for 75% of the total period considered. 2. It then analyzes the sales history for Style X. It identifies the lowest sales week as 5 units and the highest as 20 units. 3. These values are stored as *minQtySold* (5) and *maxQtySold* (20) for Style X. 4. If Style X was a new introduction 2 weeks ago, the module might use a pre-defined minimum of 2 units/week and a maximum of 15 units/week, or base it on similar styles in the same AG.

Key Takeaway: This module sets realistic boundaries for sales expectations, ensuring that future predictions for OW are grounded in actual historical performance, preventing unachievable targets or missed opportunities.

Results

The **OwMinMaxDepthModule** computes and stores in *OwData*: * *minQtySold* (minimum quantity sold for styles and AGs). * *maxQtySold* (maximum quantity sold for styles and AGs). * *actualQtySold* (actual quantity sold for styles and AGs).

Unusual Cases

- **Missing or delayed data feed:** Absence of *OdRawSalesRow* will prevent any min/max calculations, potentially leading to the *OwBuyWidthAdjustmentModule* using generic or default benchmarks, which may be inaccurate.
- **Insufficient sales history:** For very new styles or AGs, there might not be enough data to reliably determine min/max values, requiring reliance on potentially less accurate proxy data.
- **Erroneous sales spikes/dips:** Unaccounted for anomalies (e.g., a single large bulk order, or a data entry error) can skew min/max benchmarks, leading to unrealistic boundaries.

Next Steps

The calculated *minQtySold* and *maxQtySold* are crucial inputs for the *OwBuyWidthAdjustmentModule*, where they are used to revise predicted sales per style, ensuring they fall within acceptable historical bounds. These metrics are also included in the final output by the *OwOutputModule*.

Common Questions

1. How are “live days” determined? *Typically, it’s the number of days a product was available for sale in a given store during a specified period.*
2. What’s the threshold for considering a style a “new introduction”? *This is usually a configurable parameter, often based on the number of days or weeks since its first recorded sale.*
3. Why do we need both actualQtySold and min/maxQtySold? *Actual is the current reality, while min/max provides the historical range for validating future predictions, giving context to the actual.*
4. Can I exclude specific promotional periods from min/max calculations? *Yes, the input data filtering or preprocessing upstream should ideally handle such exclusions to prevent skewing benchmarks.*
5. What happens if minQtySold is zero for all styles in an AG? *This would suggest no sales activity, prompting the OwBuyWidthAdjustmentModule to potentially recommend exiting the AG or re-evaluating its potential.*

Quick Reference

| Metric | Description | Calculation Basis | Output to |
|---------------|---------------------------------|---|------------------------------------|
| minQtySold | Lowest observed sales quantity | Raw sales, live days, special logic for new items | OwData, OwBuyWidthAdjustmentModule |
| maxQtySold | Highest observed sales quantity | Raw sales, live days, special logic for new items | OwData, OwBuyWidthAdjustmentModule |
| actualQtySold | Current sales quantity | Raw sales, live days | OwData, OwOutputModule |
| Objective | Sales Benchmarking | N/A | N/A |

8.4 OwBuyWidthAdjustmentModule

Module Name & Business Objective

The **OwBuyWidthAdjustmentModule** is central to refining the **Optimum Width (OW)** by calculating predicted sales for each **Attribute Group (AG)** and style, then revising these predictions using established min/max sales benchmarks. Its core business objective is to **determine the final, actionable Optimum Width** by reconciling predicted total sales with realistically revised sales per style, ensuring that the recommended width is achievable, profitable, and aligned with market potential.

Required Information

- **Sales Data:**
 - OwData: Contains agInitialWidth (from OwInitWidthCalc), maxQtySold and minQtySold (from OwMinMaxDepthModule).
- **Product Data:**
 - OwData: Includes AopRow (Annual Operating Plan), OdAspRow (Average Selling Price), OdSegmentationRow (for AG definitions).

Business Settings

- **Prediction Model Parameters:** Specific parameters or weights used in the internal sales prediction logic (though not explicitly listed in inputs, typically part of a module like this).
- **Sales Revision Tolerance:** Thresholds for how much predicted sales can be adjusted based on min/max benchmarks (e.g., preventing a predicted 1000 units from being capped at a historical max of 50 units without further review).

Process Flow

graph TD

```
A[Load OwData] --> B[Calculate Predicted AG Sales]
B --> C[Calculate Predicted Style Sales]
C --> D[Retrieve Min/Max Sales Benchmarks]
D --> E[Revise Predicted Style Sales (Min/Max Cap)]
E --> F[Relate Total Predicted Sales to Revised Style Sales]
F --> G[Determine Final OW]
G --> H[Store Final Width & Sales Maps in OwData]
```

Step-by-Step Actions

1. **Retrieve** *agInitialWidth*, *maxQtySold*, *minQtySold*, *AopRow*, *OdAspRow*, and *OdSegmentationRow** from *OwData*, *preparing all necessary inputs for sales forecasting and adjustment*.
2. **Calculate** *predicted sales for each Attribute Group (AG)*, leveraging *AopRow* and *OdAspRow* to project revenue and unit sales, *establishing the AG's overall potential*.
3. **Distribute** *predicted AG sales down to the style level*, using various allocation methods (e.g., historical contribution, **Rate of Sale (ROS)**), *to get an initial sales forecast for individual styles*.
4. **Revise** *the predicted sales for each style* by applying the *minQtySold* and *maxQtySold* benchmarks, *ensuring that the forecast is realistic and within historical selling ranges*.
5. **Reconcile** *the total predicted sales for the AG with the sum of its revised style sales*, *iteratively adjusting individual style sales or the overall OW*.
6. **Determine** *the finalWidth for each AG*, based on the reconciled predicted sales and the revised sales per style, *arriving at the definitive number of styles to offer*.
7. **Store** *the finalWidth*, *predictedSalesMap*, *salesPerStyle*, and *revisedSalesPerStyle** in the *OwData* object, *making these critical outputs available for the next stages and final reporting*.

Sample Scenario

For a “Men’s Casual Shirts” AG in a store: 1. The `agInitialWidth` is 5 styles. 2. Predicted sales for this AG are 1000 units. 3. Initial predicted sales for Style X is 200 units. `OwMinMaxDepthModule` identified `minQtySold` for Style X as 50 and `maxQtySold` as 150. 4. The module *revises* Style X’s predicted sales from 200 to 150 (capping at `maxQtySold`). 5. This revision happens for all styles. The module then adjusts the `finalWidth` to ensure the overall AG predicted sales (1000 units) are achievable given the revised individual style sales. 6. The `finalWidth` for the AG might be adjusted to 6 styles if the revised sales support more width, or 4 if the sales potential is constrained.

Key Takeaway: *This module transforms raw predictions into actionable OW recommendations by anchoring them in realistic sales boundaries, ensuring that the assortment plan is not just aspirational but achievable and profitable.*

Results

The **OwBuyWidthAdjustmentModule** populates `OwData` with: * `finalWidth` (the adjusted optimum width for each AG). * `predictedSalesMap` (predicted sales for each AG). * `salesPerStyle` (initial predicted sales for each style). * `revisedSalesPerStyle` (predicted sales for each style after min/max adjustment).

Unusual Cases

- **Missing or delayed data feed:** If `agInitialWidth` or `minQtySold`/`maxQtySold` are unavailable, the module will struggle to provide accurate adjustments, potentially defaulting to unrefined predictions.
- **Significant discrepancy between AOP and actual sales trends:** If the AOP is overly optimistic or pessimistic compared to historical min/max ranges, the module might generate `finalWidth` recommendations that are difficult to achieve or overly conservative.
- **Style’s predicted sales consistently outside min/max range:** This could indicate a fundamental change in demand for a style or an issue with the prediction model itself, requiring manual review or model recalibration.

Next Steps

The `finalWidth` and other calculated sales metrics are consumed by the `OwExitAgModule` to identify and redistribute Open-to-Buy (OTB) for exited AGs, and by the `OwOutputModule` for final reporting.

Common Questions

1. How often are sales predictions re-calculated? *Typically aligned with the planning cycle (e.g., weekly or monthly) to react to changing market conditions.*
2. What happens if an AG’s `finalWidth` becomes zero? *It signifies that the AG might be exited, and its OTB will be reallocated by the next module.*
3. Can the min/max caps be overridden manually? *Not directly within this module, but business users can provide guidance through parameters or adjustments to input data upstream.*
4. How does the module handle new styles with no history for min/max? *It relies on the `OwMinMaxDepthModule`’s special handling for new introductions, using proxy data (e.g., category averages).*
5. What’s the difference between `salesPerStyle` and `revisedSalesPerStyle`? *`salesPerStyle` is the initial forecast, while `revisedSalesPerStyle` is that forecast adjusted to be within historical min/max sales bounds.*

Quick Reference

| Metric | Description | Calculation Basis | Output to |
|----------------------|--|--|--|
| finalWidth | Final Optimum Width | Predicted sales, revised style sales, min/max benchmarks | 0wData, 0wExitAgModule, 0wOutputModule |
| predictedSalesMap | Predicted AG sales | AOP, ASP | 0wData, 0wOutputModule |
| revisedSalesPerStyle | Adjusted style sales | Initial predicted sales, min/max benchmarks | 0wData, 0wOutputModule |
| Objective | Final OW & Sales Prediction Refinement | N/A | N/A |

8.5 OwExitAgModule

Module Name & Business Objective

The **OwExitAgModule** is designed to identify **Attribute Groups (AGs)** that have a calculated `finalWidth` of zero, effectively marking them for exit from the assortment. Its primary business objective is to **efficiently redistribute the Open-to-Buy (OTB) quantity** originally allocated to these exited AGs. This ensures that valuable OTB is reallocated to non-exit AGs at various rollup levels (category-price bucket, category, store), preventing stranded capital and maximizing inventory efficiency.

Required Information

- **Sales Data:**
 - 0wData: Contains `finalWidth` (from 0wBuyWidthAdjustmentModule) and `predictedSalesMap` (predicted sales for AGs).
- **Operational Data:**
 - 0wData: Potentially includes information about OTB quantities or rules for redistribution.

Business Settings

- **OTB Redistribution Hierarchy:** Configurable rules defining the priority and levels for OTB redistribution (e.g., within the same category first, then to broader categories, then store-wide).
- **Minimum OTB Threshold:** A minimum OTB amount below which redistribution might not occur, or above which it is prioritized.

Process Flow

graph TD
A[Load 0wData] --> B[Identify Exit AGs (finalWidth=0)]

B --> C[Calculate OTB for Exit AGs]
C --> D[Determine Redistribution Pool]
D --> E[Redistribute OTB to Non-Exit AGs]
E --> F[Update Final Width for NOOS Items]
F --> G[Store Exit Flags & Rev. OTB in OwData]

Step-by-Step Actions

1. **Retrieve** *finalWidth* and *predictedSalesMap* from the *OwData* object, *using them to identify potential AGs for exit and their sales impact.*
2. **Identify all Attribute Groups (AGs)** for which the *finalWidth* has been calculated as zero, marking them as exited.
3. **Calculate** the *Open-to-Buy (OTB)* quantity associated with these exited AGs, *quantifying the capital that needs to be reallocated.*
4. **Determine** the eligible pool of non-exit AGs at various rollup levels (e.g., category-price bucket, category, store), *identifying where the OTB can be most effectively reallocated.*
5. **Redistribute** the calculated OTB from the exited AGs to the eligible non-exit AGs, following predefined business rules and priorities, *optimizing inventory investment.*
6. **Update** the *finalWidth* in *OwData* for any NOOS (Never Out Of Stock) items, *ensuring their width is maintained if business rules dictate.*
7. **Store** the *exitAgs* set, *revOtbtQtyMap* (revised OTB quantities), and *exitAgLevelMap* (exit level information) in *OwData*, *providing transparency on reallocations.*

Sample Scenario

An “Evening Gowns” AG in a small suburban store gets a *finalWidth* of 0. 1. The module identifies this AG for exit. 2. It calculates that this AG had an OTB of \$5,000 for the upcoming season. 3. The module then looks for other “Women’s Apparel” AGs in the same store that are *not* exited and have strong predicted sales (e.g., “Casual Dresses,” “Workwear Blouses”). 4. It redistributes the \$5,000 OTB, perhaps adding \$3,000 to “Casual Dresses” and \$2,000 to “Workwear Blouses” based on their predicted sales potential and current **OW**.

Key Takeaway: This module ensures that capital tied up in underperforming or discontinued product categories is quickly and intelligently re-invested into categories with higher sales potential, maximizing financial returns.

Results

The **OwExitAgModule** computes and stores in *OwData*: * *exitAgs* (a set of AGs marked for exit). * *revOtbtQtyMap* (map of revised OTB quantities after redistribution). * *exitAgLevelMap* (map indicating the level at which an AG was exited). * Updates *finalWidth* in *OwData* for NOOS items.

Unusual Cases

- **Missing or delayed data feed:** If *finalWidth* is unavailable, the module cannot identify exited AGs, preventing OTB redistribution and leading to inefficient capital allocation.
- **No eligible AGs for redistribution:** If an entire category or store has all AGs marked for exit or no AGs with sufficient potential, OTB might remain unallocated, requiring manual intervention.

- **Conflicting OTB redistribution rules:** Ambiguous or contradictory rules could lead to inefficient or incorrect OTB reallocation, potentially starving high-potential AGs or over-investing in others.

Next Steps

The `exitAgs` set, `rev0tbQtyMap`, `exitAgLevelMap`, and updated `finalWidth` values are passed to the `OwOutputModule` to generate the complete and accurate final output rows.

Common Questions

1. How is OTB quantity defined for redistribution? *It's usually the planned budget for purchasing inventory for that specific AG over a period.*
2. Can an AG be “un-exited” after running this module? *Not directly within this module; it would require re-running the entire pipeline with modified inputs or manual intervention in the output data.*
3. What happens if an exited AG has remaining physical stock? *This module focuses on OTB reallocation; physical stock management for exited items is handled by separate inventory disposition processes.*
4. Are partial OTB redistributions allowed? *Yes, OTB is typically distributed proportionally based on target AGs' sales potential or other configurable metrics.*
5. How is the “NOOS” (Never Out Of Stock) status determined? *It's usually an attribute loaded from master product data (e.g., `AgRow`) or a business rule applied in earlier modules.*

Quick Reference

| Metric | Description | Calculation Basis | Output to |
|---------------------------|----------------------------|--------------------------------------|--|
| <code>exitAgs</code> | Set of AGs marked for exit | <code>finalWidth = 0</code> | <code>OwData</code> ,
<code>OwOutputModule</code> |
| <code>rev0tbQtyMap</code> | OTB after reallocation | Initial OTB,
redistribution rules | <code>OwData</code> ,
<code>OwOutputModule</code> |
| Objective | OTB Redistribution | N/A | N/A |

8.6 OwOutputModule

Module Name & Business Objective

The **OwOutputModule** is the final step in the **Optimum Width (OW)** pipeline. Its business objective is to **generate and persist comprehensive final OW output data** by consolidating all computed metrics from previous modules. This module creates structured output rows (`OwOutputRow` and `DenormOwOutputRow`) that contain initial and final widths, OTB quantities, min/max sales, and exit flags, enabling reporting, analysis, and integration with downstream systems for merchandise planning and inventory management.

Required Information

- **Sales Data:**
 - OwData: Contains agInitialWidth, minQtySold, maxQtySold, actualQtySold, predictedSalesMap, salesPerStyle, revisedSalesPerStyle, finalWidth, exitAgs, rev0tbQtyMap, exitAgLevelMap.
- **Product Data:**
 - OwData: Includes aspMap, prevSeasonWidth, agInputKeySet.
- **Operational Data:**
 - Database connection details to persist OwLongTailRow and OwOutputRow data.

Business Settings

- **Output Format:** Configuration for the structure and fields included in OwOutputRow and DenormOwOutputRow.
- **Persistence Strategy:** Settings for where and how the output data is stored (e.g., database, file system, data lake).

Process Flow

graph TD

```
A[Load OwData] --> B[Collect All Computed OW Metrics]
B --> C[Retrieve Foundational Data]
C --> D[Populate OwOutputRow Objects]
D --> E[Populate DenormOwOutputRow Objects]
E --> F[Persist Output to Database/Storage]
```

Step-by-Step Actions

1. **Retrieve** *all computed metrics* from the OwData object, including agInitialWidth, finalWidth, minQtySold, maxQtySold, predictedSalesMap, exitAgs, and rev0tbQtyMap, *gathering the complete picture of the **OW** calculation.*
2. **Access** *foundational data* such as aspMap, prevSeasonWidth, and agInputKeySet directly from OwData, *enriching the output with essential context.*
3. **Construct** *OwOutputRow objects*, populating them with the collected metrics, *creating a standardized, granular output format.*
4. **Construct** *DenormOwOutputRow objects*, which often include more denormalized or aggregated views of the data, *providing a flatter, easier-to-consume format for reporting.*
5. **Persist** *the generated OwOutputRow and DenormOwOutputRow data* to the designated database or storage location, *making the final **OW** recommendations available for consumption.*

Sample Scenario

After all calculations, the module finalizes the output for “Women’s Knit Tops” **AG** in Store 123. 1. It collects agInitialWidth (e.g., 8 styles), finalWidth (e.g., 7 styles), minQtySold (e.g., 20 units/week), maxQtySold (e.g., 80 units/week), predictedSalesMap (e.g., 500 units for the period), and confirms it’s *not* in exitAgs. 2. It uses the aspMap to show the **Average Selling Price (ASP)** for the

AG. 3. This information is assembled into an `OwOutputRow` for Store 123, Period Q3, AG “Women’s Knit Tops,” and then written to the `ow_output_table` in the database.

Key Takeaway: *This module is the single source of truth for the calculated Optimum Width, providing all the necessary details in a structured format for business analysis, reporting, and integration into downstream systems.*

Results

The **OwOutputModule** generates and persists: * `OwOutputRow` objects (granular OW metrics per AG, store, period). * `DenormOwOutputRow` objects (denormalized OW metrics, suitable for broader reporting).

Unusual Cases

- **Missing or delayed data feed:** If any upstream module fails to populate `OwData` correctly, the output rows might be incomplete or contain null values, leading to misleading reports.
- **Database connectivity issues:** If the target database is unavailable or slow, the module might fail to persist results, causing data unavailability for downstream systems.
- **Schema changes:** If the schema of `OwOutputRow` or `DenormOwOutputRow` changes without updating the module, data persistence could fail or lead to corrupted data.

Next Steps

The persisted output data is then available for the `UtilOutputSnapshotModule` (for auditing) and `UtilOutputSyncModule` (for synchronization with external systems). Business users and analysts can now access these tables for reporting and decision-making.

Common Questions

1. Where can I find the final **OW** results? *The results are persisted to the `ow_output_table` (or similar) in the designated database/data lake.*
2. What’s the difference between `OwOutputRow` and `DenormOwOutputRow`? *`OwOutputRow` is typically more normalized and detailed, while `DenormOwOutputRow` aggregates or flattens data for easier consumption by reporting tools.*
3. Can I export the output to CSV/Excel? *While this module persists to a database, most reporting tools can easily export the data to various formats.*
4. How do I know if an AG was exited? *The `exitAgs` field (or a similar flag) in the `OwOutputRow` will indicate if its `finalWidth` became zero and its OTB was redistributed.*
5. What if the output data is inconsistent? *This indicates an issue in an upstream calculation module; review the logs and intermediate data of previous steps.*

Quick Reference

| Metric | Description | Output Format | Persistence |
|---------------|--|--|------------------|
| Final OW Data | Consolidated OW metrics (initial, final width, sales, exit flags, OTB) | <code>OwOutputRow</code> ,
<code>DenormOwOutputRow</code> | Database/Storage |

| Metric | Description | Output Format | Persistence |
|-----------|---------------------------------|---------------|-------------|
| Objective | Output Generation & Persistence | N/A | N/A |

8.7 UtilOutputSnapshotModule

Module Name & Business Objective

The **UtilOutputSnapshotModule** is a utility module focused on data integrity and auditing. Its business objective is to **capture a snapshot of the current state of the output data** generated by the **0wOutputModule**. This snapshot serves as a critical audit trail and a potential rollback point, ensuring that a consistent version of the data exists before any final synchronization or downstream processing, thereby enhancing data governance and recovery capabilities.

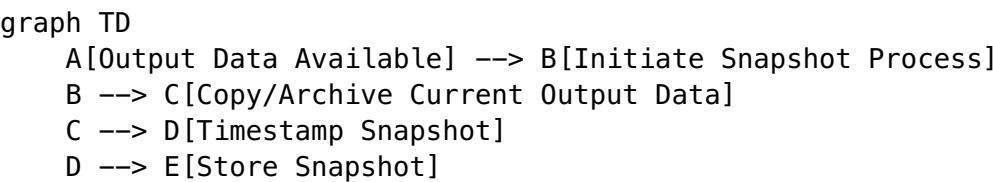
Required Information

- The output data persisted by the **0wOutputModule**.

Business Settings

- **Snapshot Frequency:** How often a snapshot is taken (e.g., daily, weekly).
- **Retention Policy:** How long snapshots are kept (e.g., 30 days, 90 days).
- **Snapshot Location:** Where the snapshot data is stored (e.g., a separate archive table, a dedicated file system path).

Process Flow



Step-by-Step Actions

1. **Confirm** *the completion of the 0wOutputModule and the availability of the final output data, ensuring a complete dataset for the snapshot.*
2. **Trigger** *the snapshot process, typically copying the latest version of the 0wOutputRow and Denorm0wOutputRow tables.*
3. **Archive** *the current output data to a designated snapshot storage location (e.g., an _archive table or a dated file in a data lake), preserving the data for future reference.*
4. **Timestamp** *the snapshot with the current date and time, providing clear versioning and historical context.*
5. **Persist** *the timestamped snapshot, making it accessible for auditing or rollback purposes.*

Sample Scenario

After the **0wOutputModule** successfully writes the current day's **OW** recommendations to the **ow_output_table**. 1. The **UtilOutputSnapshotModule** activates. 2. It copies all records from **ow_output_table** to

ow_output_table_snapshot_20250523. 3. This snapshot is now available for review, or if there's an issue with the UtilOutputSyncModule later, the data can be restored from this point.

Key Takeaway: *This module acts as a robust data safety net, providing an immutable record of the **OW** calculations at a specific point in time, crucial for compliance and error recovery.*

Results

A time-stamped copy of the final **OW** output data, stored in a designated snapshot location.

Unusual Cases

- **Missing or delayed data feed:** If the OwOutputModule fails or runs late, the snapshot might not contain the most up-to-date information, or might not be taken at all for that run.
- **Storage capacity issues:** If the snapshot storage location runs out of space, the snapshot process will fail, compromising auditability.
- **Concurrency conflicts:** If other processes are modifying the output data while the snapshot is being taken (highly unlikely if designed sequentially), the snapshot might be inconsistent.

Next Steps

The snapshot data is typically used by UtilOutputSyncModule (though not explicitly shown as a direct dependency, it conceptually depends on the output being stable) or for manual auditing.

Common Questions

1. Why do we need a snapshot if the data is already persisted? *It provides an immutable record of a specific run, preventing accidental changes or allowing rollbacks if downstream issues occur.*
2. How long are snapshots retained? *This is configurable, typically based on business needs for historical data access and regulatory compliance.*
3. Can I restore data from a snapshot? *Yes, snapshots are designed for data recovery, allowing you to revert to a previous state if necessary.*
4. Does taking a snapshot impact performance? *There's a performance overhead associated with copying data, but it's typically managed during off-peak hours or incrementally.*
5. Is the snapshot also used for reporting? *Sometimes, for historical trend analysis, but primarily it's for auditing and recovery.*

Quick Reference

| Metric | Description | Purpose | Frequency |
|-----------------|------------------------------|-------------------------------------|----------------------------|
| Output Snapshot | Copy of final OW output data | Auditing, Rollback, Data Governance | Configurable (e.g., daily) |
| Objective | Data Versioning & Recovery | N/A | N/A |

8.8 UtilOutputSyncModule

Module Name & Business Objective

The **UtilOutputSyncModule** is the final stage for making the computed **Optimum Width (OW)** data available to the broader enterprise. Its business objective is to **upload or synchronize the final computed output data to its designated destination**, typically a central data warehouse, a business intelligence platform, or other downstream operational systems. This ensures that the **OW** results are accessible and actionable for merchandise planners, category managers, and other stakeholders, enabling informed business decisions and system updates.

Required Information

- The final output data (potentially from the snapshot) generated by the `OwOutputModule` and captured by `UtilOutputSnapshotModule`.

Business Settings

- **Target System Credentials:** Login information for the destination system (e.g., API keys, database connection strings).
- **Synchronization Frequency:** How often the data is synchronized (e.g., hourly, daily).
- **Data Transformation Rules:** Any final transformations or mappings required before uploading to the target system.

Process Flow

graph TD

```
A[Output Data/Snapshot Ready] --> B[Connect to Target System]
B --> C[Prepare Data for Upload]
C --> D[Upload/Synchronize Data]
D --> E[Verify Synchronization]
E --> F[Log Status]
```

Step-by-Step Actions

1. **Verify** the readiness of the final output data (or the snapshot), ensuring data integrity before transfer.
2. **Establish** a secure connection to the designated downstream system (e.g., data warehouse, planning system) using configured credentials, ensuring data privacy and access control.
3. **Transform** the output data into the format required by the target system, if necessary (e.g., schema mapping, data type conversion), ensuring compatibility and usability.
4. **Initiate** the data upload or synchronization process, pushing the `OwOutputRow` and `DenormOwOutputRow` data to its final destination, making the **OW** recommendations available for use.
5. **Monitor and Verify** the synchronization status, checking for any errors or discrepancies, ensuring successful and complete data transfer.
6. **Log** the outcome of the synchronization process, including success/failure, number of records transferred, and any errors, providing an audit trail for operational monitoring.

Sample Scenario

The daily `OwOutputRow` data for all stores and **AGs** is ready for synchronization. 1. The `UtilOutputSyncModule` connects to the company's central data warehouse. 2. It takes the `OwOutputRow` data and maps it to the `merchandise_ow_recommendations` table in the data warehouse. 3. The module then performs an upsert (update if exists, insert if new) operation, pushing the latest **OW** recommendations. 4. Merchandise planners can now access the latest **OW** figures directly from their BI dashboards.

*Key Takeaway: This module closes the loop of the **OW** pipeline, making the calculated insights readily available to business users and other systems, transforming data into actionable intelligence that drives assortment planning.*

Results

The final computed **OW** data is successfully uploaded and synchronized with the designated downstream system.

Unusual Cases

- **Missing or delayed data feed:** If the `OwOutputModule`'s data is incomplete or the snapshot is not generated, the synchronization will be incomplete or fail.
- **Target system unavailability:** If the destination data warehouse or system is down or experiencing network issues, the synchronization will fail, delaying critical **OW** data delivery.
- **Authentication failures:** Incorrect or expired credentials for the target system will prevent successful data upload.
- **Schema mismatch:** If the target system's schema changes without updating the synchronization logic, data insertion might fail or result in data corruption.

Next Steps

The data is now ready for consumption by downstream systems (e.g., merchandise planning software, reporting dashboards) and business users, facilitating strategic assortment decisions.

Common Questions

1. How quickly are **OW** results available after the pipeline runs? *Depends on the synchronization frequency, but typically within minutes to hours after `OwOutputModule` completes.*
2. What format is the data transferred in? *Commonly CSV, Parquet, JSON, or direct database-to-database transfer, depending on target system requirements.*
3. Is the synchronization process incremental or full load? *Often incremental for efficiency, only pushing changed or new records, but configurable for full loads as well.*
4. Who uses this synchronized data? *Merchandise planners, category managers, inventory analysts, and BI dashboards for reporting.*
5. What if synchronization fails? *Automated alerts are typically triggered, and manual intervention or re-runs are initiated to ensure data delivery.*

Quick Reference

| Metric | Description | Destination | Frequency |
|----------------------|--------------------------------------|-------------|--------------|
| Synchronized OW Data | Final, actionable OW recommendations | | Configurable |

| Metric | Description | Destination | Frequency |
|-----------|-----------------------------|--|-----------|
| | | Data Warehouse, BI Tools, Planning Systems | |
| Objective | Data Delivery to Enterprise | N/A | N/A |

9. Appendix

Full Parameter Reference

(No global configuration parameters (`args.properties`) were provided in the input, so this section is empty.)

Version History

- **1.0 (2025-05-23):** Initial release of the Optimum Width (OW) documentation.

Contact & Support Info

For any questions, issues, or support regarding the **Optimum Width (OW)** documentation or the OW algorithm, please contact:

- **Email:** support@yourcompany.com
- **Internal Support Portal:** [Link to your internal support portal]
- **Team:** Product Assortment Optimization Team

Tutorial: ow

The ow system processes input data through a series of modules to calculate and finalize Optimum Width (OW) and related metrics. It starts by loading all necessary raw and computed data, then proceeds to calculate initial width, determine min/max sales, adjust buy width based on predicted sales, handle exited attribute groups, and finally generate comprehensive output for reporting and downstream systems. The processing generally follows a linear pipeline, with modules building upon the results of preceding steps.

flowchart TD

```
A0["OwInputModule"]
A1["OwInitWidthCalc"]
A2["OwMinMaxDepthModule"]
A3["OwBuyWidthAdjustmentModule"]
A4["OwExitAgModule"]
A5["OwOutputModule"]
A6["UtilOutputSnapshotModule"]
A7["UtilOutputSyncModule"]
A0 -- "data_flow" --> A1
A0 -- "data_flow" --> A2
A0 -- "data_flow" --> A3
A0 -- "data_flow" --> A5
A1 -- "data_flow" --> A3
A1 -- "data_flow" --> A5
A2 -- "data_flow" --> A3
A2 -- "data_flow" --> A5
A3 -- "data_flow" --> A4
A3 -- "data_flow" --> A5
A4 -- "data_flow" --> A5
A0 -- "depends_on" --> A1
A0 -- "depends_on" --> A2
A1 -- "depends_on" --> A3
A2 -- "depends_on" --> A3
A3 -- "depends_on" --> A4
A4 -- "depends_on" --> A5
A5 -- "depends_on" --> A6
A6 -- "depends_on" --> A7
```

Chapters

Master Report: OW (Optimum Width) Merchandising Algorithm

1. Title & Metadata

- **Project Name:** ow
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** The OW algorithm quantifies the ideal number of distinct product categories or styles a retailer should offer to maximize sales efficiency and inventory turns across its stores.

2. Executive Summary

The Optimum Width (OW) algorithm is a crucial component of our merchandising strategy, designed to optimize the breadth of product offerings. It leverages historical sales data, inventory levels, and category performance metrics to recommend the ideal number of different categories or styles (OW) that a store or specific display should stock. This ensures we are offering a sufficiently diverse, yet curated, assortment, avoiding both overwhelming customers with too many choices and missing sales opportunities due to insufficient variety.

- **Key outcomes or KPIs:**
 - Optimized OW recommendations per store and category segment.
 - Improved Category ROS (Rate of Sale) and profitability.
 - Reduced excess inventory and stockouts at the category level.
 - Enhanced customer satisfaction through tailored assortments.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Category Managers, Assortment Planners, Inventory Management, Store Operations.

3. Architecture & Data Flow

graph TD

```
A[Raw Sales Data] --> B{Data Ingestion & Preprocessing};
B --> C[Product & Category Hierarchy];
C --> D[Store Attributes];
D --> E[Historical Inventory Data];
B & C & D & E --> F[Category Performance Analysis];
F --> G[Cross-Category Demand Analysis];
G --> H[OW Calculation Engine];
H --> I[Recommendation Generation];
```

```
I --> J[OW Recommendations (per Store/Category)];
I --> K[Performance Dashboard Inputs];
J --> L[Assortment Planning Systems];
```

- **Component Bullets:**
 - **Data Ingestion & Preprocessing:** Collects, cleanses, and transforms raw transactional and master data into a usable format.
 - **Category Performance Analysis:** Calculates key metrics like ROS, inventory turns, and margin contribution for each category.
 - **Cross-Category Demand Analysis:** Identifies relationships, cannibalization, or complementary effects between different categories and styles.
 - **OW Calculation Engine:** The core logic that determines the ideal number of categories (OW) based on performance, demand patterns, and business constraints.
 - **Recommendation Generation:** Formats the calculated OW values into actionable insights and integrates them with downstream systems.
- **High-level data sources:**
 - Point-of-Sale (POS) data (transactional sales, returns)
 - Inventory Management Systems (on-hand, in-transit, receiving)
 - Product Master Data (SKU attributes, category hierarchy)
 - Store Master Data (store format, location, customer demographics)
 - Historical promotional and pricing data

4. Configuration & Data Dictionary

Settings Table (args.properties)

| Setting Parameter | Description | Default Value | Data Type |
|----------------------------|--|---------------|-----------|
| historical_lookback_weeks | Number of past weeks to include in historical data analysis. | 12 | Integer |
| min_ros_threshold | Minimum ROS (units/week) for a category to be considered viable. | 0.5 | Float |
| category_clustering_method | Algorithm for grouping similar categories (e.g., K-Means, DBSCAN). | K-Means | String |
| ow_optimization_metric | Primary metric to optimize for OW (e.g., ROS, margin_contribution). | ROS | String |
| max_ow_constraint | Upper limit for recommended OW (e.g., due to physical space limits). | 20 | Integer |
| min_ow_constraint | Lower limit for recommended OW (e.g., brand standard). | 3 | Integer |

| Setting Parameter | Description | Default Value | Data Type |
|-------------------------------|--|---------------|-----------|
| data_freshness_tolerance_days | Maximum age of input data before triggering a warning. | 7 | Integer |

Shared Data Fields (common-data.json)

| Field Name | Description | Example Value | AG (Attribute Group) |
|-----------------------|--|---------------|----------------------|
| store_id | Unique identifier for a retail store. | S101 | Location |
| category_id | Unique identifier for a product category or style. | AP_TOPS | Product Hierarchy |
| product_id | Unique identifier for a specific product (SKU). | SKU12345 | Product Identity |
| date | Date of transaction or data snapshot. | 2024-05-20 | Time |
| sales_units | Number of units sold for a specific product_id or category_id on a given date. | 15 | Sales |
| sales_dollars | Revenue generated in dollars. | 250.75 | Sales |
| inventory_units | Current on-hand inventory for a product or category. | 100 | Inventory |
| ros_units_weekly | ROS for units over a defined period (e.g., last 4 weeks). | 3.2 | Performance |
| average_selling_price | Average price at which an item or category was sold. | 18.50 | Pricing |
| customer_segment | Categorization of customer behavior or demographics. | Urban_Youth | Customer |
| store_format | Type of store (e.g., Small_Boutique, Large_Department). | Large_Dept | Location |

Outputs Catalog

| Output File Name | Description | Format | Key Fields |
|---------------------------------------|--|--------|--|
| ow_recommendations_store_category.csv | Final OW recommendations, specifying the ideal number of categories per store. | CSV | store_id, recommended_ow, current_ow |
| category_performance_summary.csv | Detailed ROS and other metrics for each category, supporting OW decisions. | CSV | store_id, category_id, ros_units_weekly, margin_contribution |
| ow_algorithm_log.txt | Log file detailing algorithm execution, warnings, and errors. | TXT | Timestamp, Level, Message |
| ow_model_parameters.json | JSON file containing the trained model parameters or decision rules. | JSON | Parameters of the OW model |

5. End-to-End Flow

The OW algorithm operates in a sequential, data-driven pipeline:

- 1. Data Ingestion:** Raw sales, inventory, product, and store data are extracted from source systems (e.g., ERP, POS) and loaded into the processing environment.
- 2. Data Cleansing & Transformation:** Ingested data undergoes validation, deduplication, and standardization. Units are converted, and missing values are imputed or flagged. Hierarchies are validated to ensure consistent category definitions.
- 3. Feature Engineering:** Key performance indicators (KPIs) are calculated. This includes ROS for various periods (e.g., 4-week, 12-week), inventory turnover, margin contribution, and category penetration rates per store.
- 4. Category Segmentation/Clustering:** Categories with similar performance profiles, demand patterns, or substitutability characteristics are grouped using clustering algorithms. This helps the OW engine make more informed decisions by treating similar categories together.
- 5. OW Calculation:** The core algorithm analyzes the performance metrics, demand relationships, and business constraints (e.g., store size, regional preferences) to determine the OW for each specific store or store-cluster/category-cluster combination. This might involve optimization techniques or rule-based systems.
- 6. Recommendation Generation:** The calculated OW values are then packaged into clear, actionable recommendations. These include the recommended OW, the current OW, and supporting rationale.

7. **Output & Integration:** The final recommendations are saved to designated output files and pushed to downstream systems (e.g., assortment planning tools, BI dashboards) for review and implementation by merchandising teams.

6. Submodule Overview

6.1. Data Ingestion & Cleansing

1. **Name & Purpose:** Gathers raw sales, inventory, product, and store data, then cleanses and formats it for subsequent analysis.
2. **Inputs & Outputs:**
 - **Inputs:** Raw POS data, raw inventory feeds, product master data, store master data.
 - **Outputs:** `clean_transactions.csv`, `clean_inventory.csv`, `enriched_product_master.csv`, `enriched_store_master.csv`.
3. **Core Logic Summary:**
 - Reads data from configured sources (e.g., SQL databases, CSV files).
 - Applies data validation rules (e.g., check for negative sales, invalid SKUs).
 - Deduplicates records and resolves inconsistencies across data sources.
 - Standardizes date formats and aggregates transactional data to a daily or weekly level.

6.2. Category Performance Analysis

1. **Name & Purpose:** Calculates key performance metrics for each category, such as ROS, inventory turns, and gross margin, which are crucial inputs for OW determination.
2. **Inputs & Outputs:**
 - **Inputs:** `clean_transactions.csv`, `clean_inventory.csv`, `enriched_product_master.csv`.
 - **Outputs:** `category_performance_metrics.csv` (contains ROS and other KPIs per `category_id`, `store_id`, and week).
3. **Core Logic Summary:**
 - Joins sales and inventory data with product hierarchy to group by `category_id`.
 - Calculates ROS (units/week) by dividing total units sold by the number of weeks in the lookback period.
 - Computes inventory turnover, average stock-holding, and gross margin for each category.
 - Aggregates these metrics at `store_id-category_id` level over the defined `historical_lookback_weeks`.

6.3. Cross-Category Demand Analysis

1. **Name & Purpose:** Identifies relationships and potential substitutability or complementarity between different categories to inform OW decisions.
2. **Inputs & Outputs:**
 - **Inputs:** `category_performance_metrics.csv`, `customer_basket_data.csv` (if available).
 - **Outputs:** `category_relationship_matrix.json`, `category_clusters.csv`.

3. Core Logic Summary:

- Analyzes co-purchase patterns from `customer_basket_data` to identify categories frequently bought together or as substitutes.
- Performs clustering on categories based on ROS patterns, price elasticity, and customer demographics to identify natural groupings.
- Generates a matrix indicating similarity or dissimilarity scores between category pairs.

6.4. OW Calculation Engine

1. **Name & Purpose:** Determines the ideal number of categories (OW) for each store or store/category segment, balancing assortment breadth with performance and efficiency.
2. **Inputs & Outputs:**
 - **Inputs:** `category_performance_metrics.csv`, `category_relationship_matrix.json`, `category_clusters.csv`, `enriched_store_master.csv`, `args.properties`.
 - **Outputs:** `raw_ow_calculations.csv` (contains the calculated OW before final formatting).
3. **Core Logic Summary:**
 - For each store or store cluster, assesses the performance of existing categories, considering their individual ROS and profitability.
 - Applies a proprietary optimization model that considers diminishing returns of adding more categories, potential cannibalization, and the benefits of diverse offerings.
 - Incorporates constraints from `args.properties` such as `min_ow_constraint` and `max_ow_constraint` to ensure practical recommendations.
 - Outputs a preliminary OW value for each target entity.

6.5. Recommendation Generation

1. **Name & Purpose:** Formats the calculated OW values into clear, actionable recommendations for merchandising teams.
2. **Inputs & Outputs:**
 - **Inputs:** `raw_ow_calculations.csv`, `category_performance_metrics.csv` (for supporting detail).
 - **Outputs:** `ow_recommendations_store_category.csv`, `performance_dashboard_inputs.json`.
3. **Core Logic Summary:**
 - Maps the `raw_ow_calculations` to human-readable store and category names.
 - Adds contextual information, such as the current number of categories stocked (`current_ow`) and the projected impact of the recommendation.
 - Generates the final CSV report and JSON payload for dashboard visualization.
 - Performs final validation checks to ensure recommendations are logical and within acceptable business ranges.

7. Worked Example

Scenario: A small boutique (Store_B001) currently stocks 7 distinct categories (current_ow). We want to determine its OW based on recent performance.

Input Data Snippet (from category_performance_metrics.csv for Store_B001):

| category_id | ros_units_weekly | margin_contribution |
|-------------|------------------|---------------------|
| DRESSES | 5.2 | 125.00 |
| TOPS | 4.8 | 90.00 |
| PANTS | 4.1 | 80.00 |
| SKIRTS | 2.5 | 35.00 |
| OUTERWEAR | 1.1 | 60.00 |
| ACCESSORIES | 6.5 | 40.00 |
| SHOES | 1.8 | 70.00 |

Configuration Settings (args.properties): - min_ros_threshold: 2.0 - ow_optimization_metric: ROS - max_ow_constraint: 6 - min_ow_constraint: 3

Cross-Category Demand Analysis Insights: - DRESSES and SKIRTS are highly substitutable; customers often buy one or the other. - ACCESSORIES are frequently co-purchased with DRESSES and TOPS.

OW Calculation Logic (Simplified):

- Filter by min_ros_threshold:**
 - SKIRTS (ROS 2.5) > min_ros_threshold (2.0) - *Keep*
 - OUTERWEAR (ROS 1.1) < min_ros_threshold (2.0) - *Flag for potential removal*
 - SHOES (ROS 1.8) < min_ros_threshold (2.0) - *Flag for potential removal*
- Evaluate Substitutability/Complementarity:**
 - Given DRESSES perform well (ROS 5.2) and are highly substitutable with SKIRTS (ROS 2.5), the algorithm might suggest prioritizing DRESSES over SKIRTS if space is limited or if SKIRTS are below a certain margin_contribution threshold relative to DRESSES.
 - ACCESSORIES are strong performers and complementary, indicating they should be retained.
- Iterative Optimization (simplified):**
 - Start with high-performing categories: DRESSES, TOPS, PANTS, ACCESSORIES. (4 categories)
 - Consider SKIRTS: While above min_ros_threshold, its substitutability with DRESSES might make it less optimal if the goal is to reduce width. If DRESSES capture sufficient demand, SKIRTS might be redundant.
 - Remove flagged low-performing categories: OUTERWEAR, SHOES.
- Apply Constraints:**
 - After removing OUTERWEAR and SHOES, and potentially SKIRTS due to substitutability and lower relative performance, the remaining categories are: DRESSES, TOPS, PANTS, ACCESSORIES. This results in 4 categories.

- Check `max_ow_constraint` (6) and `min_ow_constraint` (3). 4 falls within this range.

Recommendation Output (from `ow_recommendations_store_category.csv`):

| store_id | recommended_ow | current_ow | rationale |
|----------|----------------|------------|--|
| S_B001 | 4 | 7 | Reduced width by focusing on high ROS and high margin contribution categories, removing underperforming and substitutable items. |

The algorithm recommends Store_B001 reduce its category width from 7 to 4, focusing on categories that maximize overall ROS and profitability while considering demand relationships.

8. Debug Guide Highlights

- **Symptom:** OW recommendations are consistently too high/low.
 - **Possible Causes:** Incorrect `min_ros_threshold`, `max_ow_constraint`/`min_ow_constraint` values in `args.properties`, or flawed OW calculation logic.
 - **Remedy:** Review `args.properties`. Examine `raw_ow_calculations.csv` and `category_performance_metrics.csv` to understand how individual categories are scored. Validate the OW calculation engine's parameters.
- **Symptom:** Missing OW recommendations for certain stores/categories.
 - **Possible Causes:** Missing or insufficient input data (e.g., `sales_units` for a category), data type mismatches, or filtering out legitimate data during cleansing.
 - **Remedy:** Check `ow_algorithm_log.txt` for errors during Data Ingestion & Cleansing. Verify data completeness in `clean_transactions.csv` and `enriched_product_master.csv` for the affected entities.
- **Symptom:** ROS or other KPI calculations appear incorrect.
 - **Possible Causes:** Issues in Data Ingestion & Cleansing (e.g., duplicate sales entries, incorrect date parsing) or Category Performance Analysis module (e.g., wrong aggregation logic).
 - **Remedy:** Isolate the Category Performance Analysis module and test it with a small, controlled dataset. Cross-reference calculated ROS with manual calculations for a few sample categories.
- **Symptom:** Long processing times.
 - **Possible Causes:** Large historical lookback periods, inefficient database queries, or unoptimized calculation logic, especially in OW Calculation Engine.
 - **Remedy:** Optimize database queries. Reduce `historical_lookback_weeks` for testing. Review the OW Calculation Engine for computational bottlenecks.

9. Common Pitfalls & Remedies

- **Pitfall 1: Ignoring Seasonality and Trends:** OW recommendations based solely on aggregated historical data may miss seasonal peaks/troughs or emerging trends.
 - **Remedy:** Incorporate seasonality factors and trend analysis into the Category Performance Analysis and OW Calculation Engine. Use more granular (e.g., weekly or monthly) ROS calculations and dynamic lookback periods.
- **Pitfall 2: Static OW Values:** Applying a single OW recommendation for too long without re-evaluation can lead to suboptimal performance as market conditions change.
 - **Remedy:** Implement a regular (e.g., quarterly or monthly) refresh cycle for the OW algorithm. Consider triggering recalculations based on significant shifts in ROS or market events.
- **Pitfall 3: Over-reliance on Quantitative Data:** Exclusively using numerical data without qualitative input (e.g., merchandiser feedback, market research) can lead to impractical recommendations.
 - **Remedy:** Establish a feedback loop with merchandising teams. Allow for manual overrides or adjustments based on strategic decisions, competitive intelligence, or local market nuances not captured by data.
- **Pitfall 4: Data Quality Issues:** Inaccurate or incomplete input data will lead to flawed OW recommendations.
 - **Remedy:** Implement robust data validation and monitoring at the Data Ingestion & Cleansing stage. Set up alerts for data anomalies and ensure data freshness.

10. FAQ

1. **What does OW mean?** OW stands for Optimum Width. It refers to the ideal number of different categories or styles a store or display should offer to maximize sales and efficiency.
2. **How is OW different from OD (Optimum Depth)?** OW (Optimum Width) focuses on the *number of different categories* to stock, while OD (Optimum Depth) focuses on the *ideal number of unique items (SKUs)* within a given category, often relating to the ISS (Ideal Size Set) for that category. OW is about horizontal breadth; OD is about vertical depth within that breadth.
3. **How often are OW recommendations updated?** The algorithm typically runs on a weekly or bi-weekly cadence to account for recent sales trends and inventory changes. The output frequency can be configured.
4. **Can OW recommendations be overridden?** Yes, OW recommendations serve as data-driven guidance. Merchandising teams have the final say and can override recommendations based on strategic initiatives, local events, or qualitative insights not captured by the algorithm.
5. **What if a category has zero ROS?** Categories with zero or extremely low ROS over the `historical_lookback_weeks` will typically be flagged by the `min_ros_threshold` and considered candidates for removal from the OW calculation, suggesting they are not viable for the current assortment.

11. Quick Reference Cheat-Sheet

| Term / Parameter | Definition & Relevance |
|---|--|
| OD (Optimum Depth) | Ideal number of unique SKUs in a category/display. |
| OW (Optimum Width) | Key Output: Ideal number of different categories/styles. |
| ISS (Ideal Size Set) | Best combination of sizes for a product. |
| ROS (Rate of Sale) | Speed at which products sell (e.g., units/week). Crucial OW input. |
| AG (Attribute Group) | Set of attributes grouped for analysis. |
| historical_lookback_weeks | Configures data window for ROS calculation. |
| min_ros_threshold | Minimum ROS for category viability. |
| ow_optimization_metric | Primary KPI used by OW engine (e.g., ROS, margin_contribution). |
| store_id | Key dimension for OW recommendations. |
| category_id | Key dimension for OW recommendations and analysis. |
| ow_recommendations_store_category.csv | Primary Output File: Contains final OW values. |
| Core Logic: Balance breadth with performance and efficiency to maximize overall sales and minimize wasted space. | |

Chapter 1: Planogram Creation Orchestration Module

Last Updated: 2025-05-23

1. Business Objective

This module orchestrates the complete workflow for creating planograms, ensuring that all necessary steps are executed in the correct sequence, from data preparation to final output distribution.

Why This Matters: This ensures that planograms are consistently generated with accurate, up-to-date information, reducing manual errors, speeding up store resets, and ultimately leading to better product placement that maximizes sales and customer satisfaction. It prevents issues like a planogram being created without the latest sales data or not being properly uploaded to stores.

2. Required Information

We need these pieces of information to do our job (specifically, this module ensures *other* modules in the workflow receive them):

Sales Data

- **Units Sold:** The quantity of each product sold over a specific Analysis Period. This is crucial for identifying high-performing items that should be featured prominently on the planogram.
- **Rate of Sale (ROS):** The speed at which products are selling. This helps predict future demand and allocate space effectively to avoid stockouts or overstock.

Product Data

- **Product Classification List:** Details about each SKU, including its category, sub-category, brand, and Attribute Group (AG) (e.g., color, material). This is vital for grouping similar products on the planogram.
- **Optimum Depth (OD):** The ideal number of unique items (SKUs) to stock within a category or display. This guides how many different products should be featured.
- **Optimum Width (OW):** The ideal number of different categories or styles to offer. This informs the breadth of assortment for the planogram.

- **Ideal Size Set (ISS):** The best combination of sizes for a product to maximize sales and minimize stockouts. This ensures the right size mix is available on shelves.

Operational Data

- **Store Layout Constraints:** Information about specific store dimensions, fixture types (e.g., shelving, hanging space), and capacity limits. This ensures planograms are physically viable for each store.
- **Current Inventory Levels:** The quantity of each product currently in stock at the store or distribution center. This helps determine which products need immediate space and which can be de-emphasized.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------------|---------------|---|--|
| Planogram Generation Schedule | Weekly | Determines how often new planograms are automatically generated and updated. | When seasonal changes occur, for special promotions, or if market conditions shift rapidly. |
| Data Snapshot Cadence | Daily | Defines how frequently the system captures a “snapshot” of all relevant input data (sales, inventory, etc.) before processing begins. | If data freshness is critical for highly dynamic categories, or if source data updates less frequently. |
| Output Sync Frequency | Immediately | Specifies how quickly generated planograms are uploaded and synchronized with store systems or display tools. | If network bandwidth is limited, or if manual review is always required before distribution. |
| Workflow Validation Threshold | 95% | The minimum percentage of individual steps within the planogram creation workflow that must | To increase rigor for critical planograms (e.g., new store launches) or relax for less critical updates. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------|
| | | successfully complete for the overall process to be considered successful. | |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw Input Data Available] --> B{Planogram Creation
    Orchestration Module}
    B -- Ensures Prerequisite --> C[Data Snapshot Captured]
    C --> D[Core Planogram Logic Executes]
    D -- Manages Post-Processing --> E[Generated Planogram Output]
    E --> F[Output Uploaded to Stores]
  
```

1. Start with → **Raw Input Data Available** (Sales, product, and operational data are ready).
2. Then → **Capture Data Snapshot** (A complete record of the input data is saved for auditing and historical comparison).
3. Next → **Execute Core Planogram Creation** (The main planogram calculation and design logic runs, using the snapshot data).
4. Finally → **Upload Planogram Output** (The finished planograms are automatically sent to the relevant store systems or platforms).

5. Step-by-Step Actions

1. **Capture** the output snapshots
 - What we do: *We initiate the process to record a complete and static version of all current sales, product, and operational data.*
 - Why we do it: *This creates a consistent baseline for the planogram calculations, ensuring that all subsequent steps use the exact same data. It also provides an audit trail for future reference.*
2. **Execute** the core planogram creation logic
 - What we do: *We trigger the central planogram generation system, which takes the captured data snapshot and applies all merchandising rules, business settings, and optimization algorithms to design the ideal product layout.*
 - Why we do it: *This is where the actual planogram blueprints are created, determining product placement, facings, and overall category flow based on business objectives like maximizing ROS and space utilization.*
3. **Upload** the generated planogram output
 - What we do: *We manage the process of distributing the newly created planogram files to their designated recipients, such as store systems, display planning tools, or central databases.*

- Why we do it: *This ensures that the optimized planograms are made available to stores and merchandising teams for implementation, translating strategic planning into actionable store layouts.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- It's the first Monday of the month, and a new seasonal collection for clothing (e.g., summer t-shirts and shorts) is due to launch in two weeks.
- The current planograms are based on last season's sales data.
- New product data for the summer collection, along with updated sales forecasts, has been uploaded to the system.

What Happens

1. First, the Planogram Creation Orchestration Module initiates its scheduled workflow. It begins by triggering the `UtilOutputSnapshotModule` to **capture** a complete snapshot of all relevant data: current inventory levels, historical sales of similar items, and the new summer collection's product details. This snapshot serves as the definitive dataset for this specific planogram run.
2. Then, after confirming the snapshot is successfully captured, the module proceeds to **execute** the `PlanogramCreationModule`. This core module takes the snapshot data and, using predefined rules and the latest merchandising insights (e.g., focusing on high ROS items, ensuring proper OD and OW), calculates the optimal placement and quantities for the summer collection across various store fixtures. It determines how many rows of t-shirts, how many facings of shorts, and the ideal size set (ISS) for each display.
3. Finally, once the core planogram creation is complete and the new planogram files are generated, the orchestration module triggers the `UtilOutputSyncModule` to **upload** these new planograms. This sends the updated layouts directly to the store's visual merchandising systems, making them available for store teams to implement the new seasonal display setup.

Final Outcome

All stores receive their updated planograms for the summer collection automatically and on time, based on the latest data. This ensures products are stocked correctly from day one of the new season, minimizing lost sales from stockouts of popular items and maximizing visual appeal.

Key Takeaway: The orchestration module ensures a seamless, automated, and error-free transition from raw data to actionable planograms, saving time and improving merchandising accuracy.

7. Results

This step produces:

Main Outputs

- **Orchestrated Planogram Generation Workflow:** A successfully completed sequence of tasks, confirming that all prerequisite data was captured, core planogram logic ran, and outputs were distributed. This is the primary “output” of the orchestration itself.
- **Completed Planogram Files:** The actual detailed layouts and product placement instructions for specific categories or entire stores, which are generated by the core planogram module under this orchestration.
- **Audit Trails of Planogram Creation:** Logs and records detailing when data snapshots were taken, when planograms were generated, and when they were uploaded, ensuring transparency and traceability.

Business Reports

- **Planogram Generation Status Report:** Used by merchandising managers and IT support to verify that the automated planogram updates completed successfully across all stores or categories, or to identify any steps that failed.
- **Output Sync Confirmation Report:** Used by store operations and visual merchandising teams to confirm that the latest planograms have been successfully delivered to their systems and are ready for implementation.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--------------------------------------|--|---|
| Missing or delayed data feed | The module may halt the process, log an error, or proceed with the last available data, depending on configuration. | Investigate the source of the missing data (e.g., sales system, inventory system). Resolve the data feed issue and re-run the planogram orchestration if necessary. |
| Core planogram creation module fails | The orchestration module will log an error, mark the overall process as failed, and stop further steps (like uploading). | Review the error logs from the core planogram creation module to identify the specific issue (e.g., invalid rules, data inconsistencies). Correct the issue and restart the orchestration. |
| Output upload fails | The planogram generation will be marked as complete, but the final step of delivery to stores will fail, leaving store systems with outdated planograms. | Check network connectivity to target store systems or the output destination. Verify credentials or permissions for the upload process. Manually upload if automation cannot be restored immediately. |

| Situation | What Happens | What You Should Do |
|------------------------|---|---|
| Snapshot capture fails | The orchestration module will stop, indicating a failure to establish the initial data baseline. Subsequent planogram creation won't proceed. | Investigate why the data snapshot couldn't be captured (e.g., database issues, source system errors). Resolve the underlying data access problem and restart the process. |

9. Next Steps

Before This Step

- **Data Ingestion:** Sales, inventory, and product data from various source systems must be collected, cleaned, and made available. This is crucial for the snapshot module to function.
- **Category Management Decisions:** Merchandising teams define the overall category strategy, including target OD, OW, and ISS, which inform the core planogram logic.

After This Step

- **Planogram Deployment:** Store systems receive the generated planograms and make them available for store teams. This is the practical step where planograms are physically printed or displayed digitally.
- **Store Execution:** Store associates follow the planograms to set up displays, restock shelves, and merchandise products, ensuring compliance with the optimized layout.
- **Performance Monitoring:** Sales and inventory data are continuously tracked post-planogram implementation to evaluate the effectiveness of the new layout and identify areas for future optimization.

10. Common Questions

Q: Why can't I just run each step manually? A: While each step *could* theoretically be run manually, the orchestration module ensures they run in the correct order, using the right data at the right time, and automatically manages the flow. This prevents errors, ensures consistency, and is critical for generating hundreds or thousands of planograms efficiently.

Q: How does this module make sure we use the most current sales data? A: It explicitly triggers a "data snapshot" at the very beginning of the process. This locks in the freshest available sales, product, and operational data for that specific planogram run, guaranteeing that all calculations are based on the latest information available at that moment.

Q: What happens if one part of the planogram creation fails during the process? A: The orchestration module is designed to detect failures. If a step (like the core planogram calculation) encounters an error, the module will typically halt the entire process, log the specific issue, and prevent incomplete or faulty planograms from being generated or uploaded. This ensures quality control.

Q: Can this module handle different types of planograms, like for a specific store size versus a category? A: Yes, while the orchestration module itself manages the *sequence* of steps, the underlying “core planogram creation” module that it triggers can be configured to generate various types of planograms (e.g., for different store formats, specific categories like apparel or electronics, or even promotional displays), provided the necessary data is available in the initial snapshot.

Q: How does this contribute to increased sales or profitability? A: By automating and coordinating the entire planogram creation process, this module ensures that stores consistently receive optimized layouts based on current performance data. This leads to better product availability, reduced stockouts of popular items, improved visual merchandising, and ultimately, higher sales per square foot and increased profitability.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Orchestrates the entire planogram creation workflow from data capture to output distribution, ensuring sequence and accuracy.
- **Main Input:** The availability of clean Sales Data, Product Data (including OD, OW, ISS, AG), and Operational Data.
- **Main Output:** A successfully completed and verified planogram generation process, resulting in finalized and uploaded planogram files for stores.
- **Warning Signs:** Failed process runs, outdated planograms in stores, or errors reported in audit logs indicating a break in the workflow.
- **Key Settings:** Planogram Generation Schedule, Data Snapshot Cadence, and Output Sync Frequency.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 2: Planogram Creation

Module

Last Updated: 2025-05-23

1. Business Objective

This module calculates the optimal product display configurations, including the final number of unique items (OD) and their quantities, for each store, category, and Attribute Group (AG) combination. It refines initial merchandise plans by integrating specific display rules and store-level adjustments to maximize sales and minimize stockouts.

Why This Matters: This ensures that every inch of shelf space is utilized effectively, putting the right amount of the right products in front of customers. It directly impacts sales by preventing popular items from running out and ensures visual appeal, leading to a better shopping experience and increased revenue.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Optimum Width (OW) Output Quantities (OTB):** These are the recommended product quantities for specific categories and Attribute Groups, calculated based on sales forecasts and historical performance. We need this as the foundational allocation to start fine-tuning.
- **Rate of Sale (ROS):** While not directly consumed by this module, the ROS insights embedded within the OW output (OTB) are crucial. This helps ensure that the quantities we allocate reflect how quickly products are expected to sell, guiding smart space allocation.

Product Data

- **Optimum Depth (OD):** The target number of unique items (SKUs) within a category or display. This guides how many different product variations (e.g., styles, colors) should be presented on the shelf.
- **Ideal Size Set (ISS):** The best combination of sizes for a product group to maximize sales and minimize stockouts. This module uses the ISS values to ensure the calculated quantities can accommodate a practical mix of sizes on the display.

- **Attribute Group (AG) Details:** Information about product attributes like color, material, or brand, grouped for analysis. We need this to apply specific display rules and quantity allocations to distinct product characteristics within a category.

Operational Data

- **Minimum Display Options:** The lowest acceptable number of unique items or facings for a given category or Attribute Group on a display. This ensures that even lower-performing items or essential styles receive adequate visibility.
- **Store-Level Base Stock Overrides:** Specific quantity adjustments for particular stores, often driven by high foot traffic, local events, or unique store formats. This allows us to fine-tune quantities beyond system recommendations to meet specific store needs.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---------------------------------|---------------|--|--|
| Primary Attribute for Display | “Color” | Specifies which product attribute (e.g., color, material, fit) should be primarily used for calculating display options and quantities on the planogram. | When a specific attribute becomes more critical for merchandising, such as a new trend focusing on fabric types. |
| Default Minimum Display Options | 0 | The fallback minimum number of unique items to display if no specific minimum is defined for a category or Attribute Group. | To enforce a baseline display presence for all products, even those without specific rules. |
| Default Ideal Size Set | 0 | The fallback value for the Ideal Size Set if not explicitly defined for a particular store-category combination. | To ensure all products can be allocated based on a default size curve, preventing invalid calculations. |
| Default Store Base Stock Level | 0 | The standard adjustment | If a general uplift or reduction in stock is |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|--|
| | | factor for store inventory if no specific store override is provided. | needed across stores, or to manage overall inventory levels. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[OW Outputs (OTB), ISS, Min Display Options, Base Stock Overrides] --> B{Planogram Creation Module}

B -- Calculates --> C[Final Planogram Configurations]

1. Start with → **OW Outputs (OTB), ISS, Min Display Options, Base Stock Overrides** (The module receives the ideal quantities, size sets, and display constraints).
2. Then → **Calculate Initial Display Parameters** (The system first determines base quantities and options for each product group).
3. Next → **Adjust Quantities for Store Specifics** (Quantities are scaled up or down based on individual store needs and overrides).
4. Finally → **Determine Final Display Options and Quantities** (The module refines the numbers to ensure they meet minimum display requirements and fit practical size sets, resulting in the final planogram configuration).

5. Step-by-Step Actions

1. **Integrate** key display rule inputs
 - What we do: *We gather store-specific Ideal Size Set (ISS) values, minimum display options, and base stock override quantities for each store and category.*
 - Why we do it: *This establishes the foundational business rules and overrides that will guide all subsequent calculations, ensuring tailored planograms.*
2. **Determine** initial product depth and quantities
 - What we do: *We process the OW output quantities (OTB) for each Attribute Group within a category and store, considering the primary attribute defined for planogramming (e.g., color).*
 - Why we do it: *This step translates the high-level OTB recommendations into specific quantities per product attribute, forming the basis for the planogram layout.*
3. **Apply** store-specific quantity adjustments
 - What we do: *We adjust the calculated product quantities based on any store-level base stock overrides. If a store has a higher base stock requirement, all quantities for that store are scaled proportionally.*

- Why we do it: *This allows for local market variations and specific store needs, preventing stockouts in high-traffic locations or overstock in smaller stores.*
- 4. Calculate final display options and quantities**
- What we do: *We refine the adjusted quantities by ensuring they align with the Ideal Size Set (ISS) and meet any specified minimum display options. This step ensures that the final number of products (quantity) and how many distinct products (options) are displayed is both commercially optimal and physically practical.*
 - Why we do it: *This final calculation converts theoretical quantities into actionable planogram recommendations, balancing optimal stock levels with aesthetic display requirements.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A specific store (Store 101) in a high-traffic urban area has an OW Output Quantity (OTB) for "Women's Denim Jeans" (Category) and the Attribute Group (AG) "Dark Wash Blue" of 300 units.
- The Ideal Size Set (ISS) for "Women's Denim Jeans" in Store 101 is defined as 6 (meaning 6 distinct sizes are ideal for a full display).
- The Minimum Display Options for "Women's Denim Jeans" is 2 (meaning at least 2 unique styles/colors must be shown).
- Store 101 has a Store-Level Base Stock Override of +20% compared to its initial total calculated OTB across all categories, reflecting its higher sales velocity. (Let's say total initial OTB for Store 101 was 5000 units, so the base stock override implies total desired quantity of 6000 units, a 1.2 ratio).

What Happens

1. First, the Planogram Creation Module gathers these inputs. It identifies the "Dark Wash Blue" AG for "Women's Denim Jeans" in Store 101 with an initial OTB of 300 units. It also notes the ISS of 6, Minimum Display Options of 2, and the +20% Base Stock Override for Store 101.
2. Then, the module applies the Store-Level Base Stock Override. If the store's total OTB for all categories was 5000 units, and the override dictates a +20% adjustment, it means all category/AG quantities for this store are scaled by 1.2 ($6000 / 5000$). So, the 300 units for "Dark Wash Blue" jeans are adjusted to $300 * 1.2 = 360$ units. This becomes the initial quantity.
3. Next, the module calculates the initial display options for these 360 units based on the ISS: $360 \text{ units} / 6 \text{ (ISS)} = 60$ initial options.
4. Finally, it compares this to the Minimum Display Options of 2. Since 60 (initial options) is much greater than 2 (minimum options), the final options remain 60. The final size set remains 6 (the ISS). The final quantity is then calculated as $60 \text{ options} * 6 \text{ sizes} = 360$ units.

Final Outcome

Store 101 receives a planogram instruction for “Women’s Denim Jeans - Dark Wash Blue” recommending 360 units to be displayed across 60 unique options (e.g., 60 distinct styles or washes within “Dark Wash Blue”) with an Ideal Size Set of 6 for each. This ensures the store is adequately stocked for its high traffic, displaying a wide range of popular dark wash options while maintaining the ideal size availability.

Key Takeaway: This module precisely converts broad merchandise plans into actionable, store-specific display instructions, optimizing product presence based on both business strategy and real-world store dynamics.

7. Results

This step produces:

Main Outputs

- **Planogram Output Rows:** Detailed records for each store, category, and Attribute Group combination, specifying the initial and final calculated quantities, options, and size sets. This is the core numerical output used for constructing visual planograms.
- **Export Planogram Output Rows:** A simplified version of the planogram output, formatted for easy export and integration with external planogramming software or store systems. It includes key fields like store code, channel, category, attribute, front options, and quantity.

Business Reports

- **Planogram Configuration Report:** Used by visual merchandising teams and store operations to review the recommended product depths, options, and quantities for each display. It helps ensure the planograms are both effective and achievable in store.
- **Quantity and Options Summary:** Used by merchandise planners to validate that the final allocations align with overall inventory strategies and to identify any categories or stores that might require further manual review due to significant adjustments.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|---|
| Missing or delayed data feed | The module may use default values for ISS, Minimum Display Options, or Base Stock, potentially leading to sub-optimal | Investigate the source of the missing data (e.g., OW output, ISS calculation). Resolve the data feed issue and re-run the orchestration module. |

| Situation | What Happens | What You Should Do |
|---|--|---|
| | planograms for specific stores/categories. | |
| Zero or unusually low OW Output Quantities (OTB) | The module will calculate zero or very low final quantities and options, even if the item is still important. | Review the OW calculation process; it might indicate a data issue or an overly aggressive reduction. Consider overriding planogram manually if the product is essential. |
| Unusually high Base Stock Override | The module will proportionally increase all quantities for that store, potentially leading to overstocking if not intended. | Verify the accuracy of the base stock override input. Adjust the override value if it was entered incorrectly or if the store's needs have changed. |
| Minimum Display Options greater than calculated options | The final options will be forced up to the minimum, which might lead to a lower effective size set or increased quantity for an underperforming product. | Review the minimum display options setting for that category/store. It might be too aggressive for the actual product performance, requiring adjustment or manual override. |
| Multiple OD periods present | The module will throw an error and halt execution (as it expects data from only one consistent analysis period). | Investigate and rectify the data source to ensure only one active "Optimum Depth" (OD) period is being used for the current planogram run. This indicates a data consistency problem. |

9. Next Steps

Before This Step

- **Optimum Width (OW) Calculation:** The preceding module must calculate the ideal breadth of assortment and initial quantities (OTB) for categories and Attribute Groups, which serve as primary inputs for this module.
- **Ideal Size Set (ISS) Determination:** Size set analysis needs to be completed to provide the optimal size breakdowns for product groups, enabling this module to apply practical display rules.
- **Minimum Display Option & Base Stock Overrides Definition:** Merchandising teams and store operations define any specific minimum display requirements or store-level base stock adjustments.

After This Step

- **Planogram Deployment:** The generated planogram configurations are sent to store systems, visual merchandising tools, or central databases, making them accessible for store teams.

- **Store Execution:** Store associates use the detailed planogram instructions to set up displays, merchandise products, and restock shelves according to the optimized layout.
- **Performance Monitoring:** Sales, inventory, and other retail metrics are continuously tracked after planogram implementation to evaluate the effectiveness of the new layouts and inform future optimization cycles.

10. Common Questions

Q: How does this module ensure we don't run out of popular sizes? A: It explicitly uses the **Ideal Size Set (ISS)** as a key input. When calculating the final quantities and display options, it ensures that the total stock allocated can be realistically broken down into the ideal combination of sizes, minimizing the chances of stockouts for high-demand sizes.

Q: What happens if a store's shelves are too small for the recommended quantity? A: This module focuses on calculating ideal quantities based on business rules and sales potential. While it considers **Minimum Display Options**, physical **Store Layout Constraints** (like fixture capacity) are typically integrated either *before* this module (in **OW** calculations) or *after* during the planogram deployment/visual rendering phase. If a physical constraint is hit, manual adjustments by visual merchandising teams or further system-level adjustments may be required.

Q: Can this module prevent us from displaying too few items of a new product? A: Yes, by incorporating **Minimum Display Options**. Even if initial sales forecasts for a new item are low, you can set a minimum number of options or facings to ensure it gets adequate shelf presence for its launch, encouraging customer discovery.

Q: How does a store's sales performance affect its planogram recommendations? A: The **OW Output Quantities (OTB)** that feed into this module are already tailored by sales performance. Additionally, the **Store-Level Base Stock Overrides** allow high-performing stores to receive a larger overall quantity of stock, which this module then distributes across their planogrammed items.

Q: What's the main financial benefit of using this module? A: The main financial benefit is increased sales and reduced inventory holding costs. By precisely allocating the right amount of product (balancing **OD**, **ISS**, and store needs), the module minimizes lost sales from stockouts, reduces markdown risk from overstock, and optimizes inventory investment, leading to higher profitability per square foot.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Refines high-level merchandise plans into precise, actionable planogram configurations for stores.
- **Main Input:** **OW Output Quantities (OTB)**, **Ideal Size Set (ISS)**, **Minimum Display Options**, and **Store-Level Base Stock Overrides**.
- **Main Output:** Final calculated product quantities and display options (number of unique items) for each store, category, and Attribute Group.

- **Warning Signs:** Unrealistic quantities (too high or too low), planograms that don't match business intent, or errors in the execution logs related to missing data.
 - **Key Settings:** Primary Attribute for Display, Default Minimum Display Options, Default Ideal Size Set, and Default Store Base Stock Level.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Planogram Creation FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document provides high-level answers to common business questions regarding the planogramCreation algorithm, its functionality, and how it helps optimize product display.

General

Q: What is the purpose of the planogramCreation algorithm? **A:** The planogramCreation algorithm is designed to automatically determine the ideal product display configurations (what to stock, how many, and in what sizes) for each store, product category, and Attribute Group (AG) combination. Its main goal is to maximize sales and minimize stockouts by providing optimal stocking recommendations.

Q: What is a “planogram” in this context? **A:** In this system, a “planogram” refers to the generated output that details the ideal selection (Optimum Depth, OD) of unique items (SKUs) and their respective quantities for display, along with the Ideal Size Set (ISS) to be stocked. It’s a data-driven recommendation for how products should be arranged and stocked in a store display.

Q: Who benefits from the planogramCreation algorithm? **A:** Merchandisers, category managers, and store operations teams benefit by receiving data-driven recommendations that optimize product assortment and stocking levels, leading to improved sales, reduced waste, and more efficient inventory management.

Q: What are the main components of this algorithm? **A:** The algorithm comprises two key modules: 1. **Planogram Creation Module:** The core engine that calculates the ideal product display configurations (final OD/options and quantities). 2. **Planogram Creation Orchestration Module:** This module manages the overall workflow, ensuring all necessary data is prepared before calculation and that the final output is properly handled afterwards.

Q: How does this algorithm contribute to maximizing sales and minimizing stockouts? **A:** By leveraging data points like Optimum Width (OW), Ideal Size Set (ISS), and Rate of Sale (ROS) (implicitly used in OW/ISS calculations), the algorithm intelligently adjusts product quantities and selections (Optimum Depth, OD) to ensure that popular items and sizes are adequately stocked, while minimizing overstock of slower-moving items.

Configuration & Settings

Q: Can I influence the final planogram recommendations? **A:** Yes, the algorithm considers various configuration settings and inputs. Key inputs like the target Optimum Width (OW) output quantities, Ideal Size Set (ISS) values, and minimum display options directly influence the final recommendations.

Q: Are there global settings for planogram creation? **A:** Yes, the algorithm leverages global configuration settings. These settings help define the default behaviors and constraints for planogram generation across all stores and categories. Specific details are typically outlined in the system's global configuration files.

Q: Can individual stores override the default planogram recommendations? **A:** Yes, the algorithm allows for store-level base stock overrides. This means that while a general recommendation is provided, individual stores can have specific adjustments applied to their quantities to account for unique local conditions or space constraints.

Q: How are “minimum display options” handled? **A:** The algorithm takes into account specified minimum display options. This ensures that even if calculated ideal quantities are very low, a minimum number of unique items (SKUs) or options are still included in the planogram to maintain a viable product presence.

Q: Where can I find or adjust these configuration settings? **A:** Specific configuration settings are typically managed by system administrators or power users. For details on accessing or modifying these settings, please refer to the main system documentation or contact the support team.

Data & Inputs

Q: What kind of data does the planogramCreation algorithm use as input? **A:** The algorithm processes several key inputs, including: * Output quantities derived from Optimum Width (OW) calculations. * Ideal Size Set (ISS) values for products. * Minimum display options per category/AG. * Store-level base stock overrides. * Implicitly, Rate of Sale (ROS) data underpins the OW and ISS calculations.

Q: What is Optimum Width (OW) and why is it important for planogram creation? **A:** Optimum Width (OW) represents the ideal number of different categories or styles to offer. It's crucial because it dictates the overall variety of products the system should aim to display, influencing the initial selection of product lines or styles for the planogram.

Q: What is an Ideal Size Set (ISS) and how does it affect planograms? **A:** An Ideal Size Set (ISS) is the best combination of sizes to maximize sales and minimize stockouts for a particular product or Attribute Group. The algorithm uses ISS values to ensure that the planogram recommends stocking the most relevant and profitable sizes for each item.

Q: How does Optimum Depth (OD) relate to the planogram output? **A:** Optimum Depth (OD) is the ideal number of unique items (SKUs) to stock within a category or display. The planogramCreation algorithm's primary output is the recommended Optimum Depth (OD) for each display, along with the corresponding quantities for each item and size.

Q: What is an Attribute Group (AG) and how does it fit in? **A:** An Attribute Group (AG) is a set of product attributes grouped for analysis (e.g., color, size, brand). The algorithm generates planogram recommendations not just at the category level, but also specifically for each Attribute Group within a category, allowing for more granular optimization.

Process & Logic

Q: How does the algorithm determine the specific items and quantities for a planogram? **A:** The core Planogram Creation Module takes the OW output quantities, ISS values, and minimum display options as starting points. It then adjusts these quantities based on store-level base stock overrides and applies a set of business rules to arrive at the final Optimum Depth (OD) and specific quantities for each item, aiming to maximize sales and minimize stockouts.

Q: What is the role of the Orchestration Module in this process? **A:** The Planogram Creation Orchestration Module is like the conductor of an orchestra. It ensures that all prerequisite steps, such as capturing necessary output snapshots (e.g., of OW and ISS data), are completed before the core planogram calculation begins. Afterwards, it manages post-processing actions like uploading the generated planogram output to the relevant systems.

Q: How does the algorithm ensure compliance with business rules? **A:** The Planogram Creation Module has built-in logic to adhere to predefined business rules. This involves adjusting calculated quantities and selections to fit display constraints, ensuring product adjacencies, or respecting minimum/maximum display counts, as defined in the system's configuration.

Q: What happens if there's insufficient display space for the calculated ideal quantities? **A:** While the input doesn't explicitly detail space constraints, the algorithm's goal to maximize sales and minimize stockouts implies it will attempt to optimize within practical limits. Store-level overrides and business rules related to display capacity would be crucial here, potentially leading to prioritized stocking based on Rate of Sale (ROS) or profitability.

Q: In what sequence are the different calculations performed? **A:** The Orchestration Module dictates the sequence. It first ensures that all necessary input data (like OW output quantities and ISS values) are correctly captured and ready. Then, the core Planogram Creation Module performs its calculations. Finally, the Orchestration Module oversees the synchronization and upload of the generated planogram outputs.

Outputs & Results

Q: What is the final output of the planogramCreation algorithm? **A:** The final output is a detailed planogram configuration for each store, category, and Attribute Group (AG) combination. This includes the ideal Optimum Depth (OD) (unique items/SKUs), the specific options to stock, and the recommended quantities for each item, along with its Ideal Size Set (ISS).

Q: How accurate are the generated planogram recommendations? **A:** The recommendations are data-driven and are designed to be highly accurate, aiming to maximize sales and minimize stockouts based on the provided inputs (OW, ISS, ROS, etc.). Their effectiveness is continuously evaluated through sales performance and inventory metrics.

Q: Where can I access the generated planogram outputs? **A:** The generated planogram outputs are typically uploaded to a central system or database via the `UtilOutputSyncModule` managed by the Orchestration Module. They can then be accessed through dedicated reporting tools, dashboards, or integrated directly into retail planning systems.

Q: Can I see historical planogram recommendations? **A:** Yes, the Orchestration Module ensures that output snapshots are captured. This capability, combined with the output synchronization, allows for the tracking and retrieval of historical planogram recommendations for analysis and comparison.

Q: How are product quantities expressed in the output? **A:** Product quantities are expressed as units or facings per SKU, providing clear instructions on how many of each unique item, and within its Ideal Size Set (ISS), should be stocked on the display.

Error Handling & Edge Cases

Q: What happens if the Ideal Size Set (ISS) is empty for a particular product or Attribute Group? **A:** While specific handling isn't detailed, in such a scenario, the algorithm would likely fall back to a default size set, flag the item for manual review, or exclude it from the planogram if no valid size information can be determined.

Q: How does the system handle Attribute Groups (AGs) or categories with very low historical Rate of Sale (ROS)? **A:** While not explicitly an edge case described, low ROS typically translates to lower OW and ISS recommendations. If the calculated Optimum Depth (OD) falls below the specified minimum display options, the system will enforce the minimum to ensure the product still has a presence, preventing its complete removal from the display.

Q: What if there's a conflict between store overrides and global settings? **A:** Store-level base stock overrides are designed to take precedence over general calculations for specific items or categories at that store. The algorithm processes these overrides after initial calculations to ensure local requirements are met.

Q: What if the input data for Optimum Width (OW) or Ideal Size Set (ISS) is missing or invalid for certain products? **A:** The system is designed to handle prerequisite steps via the Orchestration Module. If critical input data is missing or invalid, the module responsible for capturing snapshots (`UtilOutputSnapshotModule`) would likely flag these issues, potentially preventing planogram generation for the affected items or using pre-defined fallback values.

Q: Are there any known specific edge cases to be aware of? **A:** Currently, no specific “known edge cases” beyond general data quality issues or unusual store constraints (which can often be managed by store overrides) have been identified. The algorithm is designed to be robust and handle a wide range of business scenarios.

Need More Help? Please contact the internal support team for further assistance.

This documentation outlines the planogramCreation algorithm, detailing its purpose, architecture, setup, and individual module functionalities. It is designed to provide business stakeholders and technical teams with a clear understanding of how the system generates optimized product display configurations to drive sales efficiency.

planogramCreation Documentation

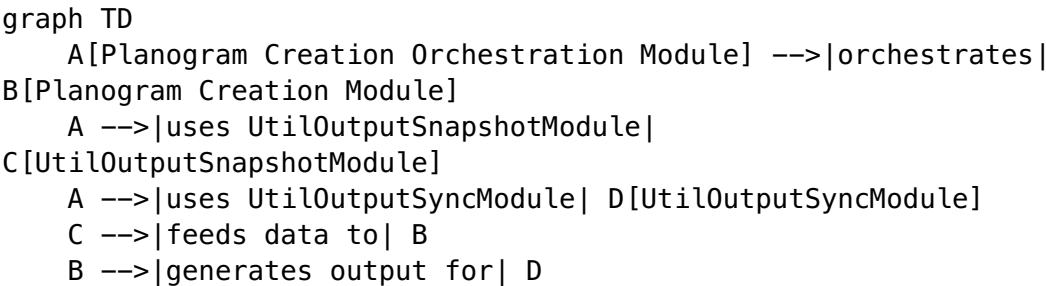
Version: 1.0 Last Updated: 2025-05-23

1. Executive Summary

The planogramCreation algorithm is a sophisticated system designed to optimize product display configurations within retail environments. It calculates the ideal number of unique items (OD) and their quantities per SKU for each store, category, and Attribute Group (AG), ensuring optimal product presentation. By leveraging key business inputs like Optimum Width (OW), Ideal Size Set (ISS), and Rate of Sale (ROS), this system maximizes sales potential, minimizes stockouts, and enhances operational efficiency in merchandising.

2. Architecture Overview

The planogramCreation system is structured into key modules that orchestrate the complete planogram generation workflow.



Module Descriptions:

- **Planogram Creation Orchestration Module:** This module sequences and coordinates the entire planogram generation process, ensuring all prerequisite steps (like data snapshots) and post-processing actions (like output synchronization) are completed.
- **Planogram Creation Module:** This is the core module responsible for calculating the ideal product display configurations (OD and quantities) based on various business rules and data inputs.

- **UtilOutputSnapshotModule:** (Inferred helper module) Captures snapshots of critical outputs before processing, enabling historical tracking and rollback capabilities.
- **UtilOutputSyncModule:** (Inferred helper module) Manages the upload and synchronization of generated planogram outputs to target systems or databases.

3. Prerequisites & Setup

To set up and run the planogramCreation system, the following prerequisites are typically required:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Apache Maven:** Version 3.6 or higher for building the project.
- **Database Connectivity:** Access credentials and drivers for the underlying data sources (e.g., SQL Server, PostgreSQL) where sales, product, and operational data are stored.

Build and Run Commands:

1. Clone the Repository:

```
git clone <repository-url>/planogramCreation.git
cd planogramCreation
```

2. Build the Project:

```
mvn clean install
```

3. Run the Orchestrator (ModuleGroup):

The Planogram Creation Orchestration Module acts as the primary entry point.

```
java -jar target/planogramCreation-1.0.jar --module
    PlanogramCreationOrchestrationModule
```

(Note: The exact command might vary based on your project's main class and Spring Boot/framework setup.)

4. Folder & File Structure

The typical project structure is as follows:

```
planogramCreation/
├─ module-group/           # Contains the orchestrator module
                           (Planogram Creation Orchestration Module).
├─ submodules/             # Houses individual core and utility
                           modules (e.g., Planogram Creation Module).
├─ args.properties         # Global configuration parameters for
                           the application.
├─ common-data.json        # Defines the schema for shared data
                           models across modules.
├─ outputs/               # Directory for generated planogram
                           outputs and logs.
```

5. Global Configuration

The `args.properties` file is used for global configuration settings.

- **Note:** The `args.properties` table details were not provided in the input. Please refer to the `args.properties` file within the repository for available settings.

6. Shared Data Model

The `common-data.json` schema defines the structure of data commonly exchanged or utilized across different modules.

- **Note:** The `common-data.json` schema details were not provided in the input. Please refer to the `common-data.json` file within the repository for the exact schema definition.

7. Submodule Chapters

7.1 Planogram Creation Module

This module is the core engine for calculating precise product display configurations.

7.1.1 Module Name & Business Objective

Module Name: Planogram Creation Module **Business Objective:** To calculate the ideal number of unique items (OD) and their optimal quantities (SKUs) for product displays across all stores, categories, and Attribute Group (AG) combinations. This ensures product offerings are optimized to maximize sales and minimize stockouts by leveraging calculated Optimum Width (OW) outputs, Ideal Size Set (ISS) values, and adhering to minimum display options and base stock overrides.

7.1.2 Required Information

- **Sales Data:**
 - **Rate of Sale (ROS):** Historical sales velocity for individual SKUs, used to project demand.
 - **Optimum Width (OW) Output Quantities (OTB):** The calculated ideal total units or budget for a category/style, derived from higher-level planning.
- **Product Data:**
 - **Ideal Size Set (ISS) Values:** Recommended size distributions (e.g., S:M:L ratio) for product ranges.
 - **Minimum Display Options:** Business rules defining the minimum number of unique items or facings required for a product.
 - **SKU Attributes:** Details like product dimensions, packaging, and specific display requirements.

- **Operational Data:**

- **Store-Level Base Stock Overrides:** Specific adjustments to quantities for certain products or categories at individual store locations.
- **Fixture Capacities:** Physical space constraints of display fixtures in stores.

7.1.3 Business Settings

- **Minimum Display Options:** A configurable parameter to ensure products have a minimum visual presence, preventing them from being under-displayed.
- **Base Stock Overrides:** User-defined or system-managed overrides for specific SKU quantities at a store level, ensuring critical items are always stocked.

7.1.4 Process Flow

graph TD

```

    A[Inputs: OW, ISS, Min Display Rules, Base Stock] -->
    B{Calculate Initial OD & Quantities}
    B --> C{Apply Base Stock Overrides}
    C --> D{Adjust for Min Display Rules}
    D --> E[Final Planogram Output: OD, Quantities]
  
```

7.1.5 Step-by-Step Actions

1. **PROCESS** incoming data streams, including OW output quantities, ISS values, and store-specific base stock overrides. *This ensures all necessary data for calculation is available.*
2. **CALCULATE** the initial Optimum Depth (OD) and corresponding quantities for each SKU within a given store, category, and AG. *This is based on balancing demand (from ROS) and supply (from OW/ISS).*
3. **ADJUST** calculated quantities based on any applicable store-level base stock overrides. *This ensures critical items are always represented adequately, even if initial calculations suggest less.*
4. **ENSURE** the final configuration adheres to minimum display options. *This prevents products from being visually lost or having insufficient facings.*
5. **MAXIMIZE** sales potential by optimizing product mix and quantities. *This is the core business objective of efficient planograms.*
6. **MINIMIZE** stockouts by aligning quantities with expected demand and strategic stock levels. *This improves customer satisfaction and reduces lost sales opportunities.*

7.1.6 Sample Scenario

Scenario: For the “Women’s T-Shirts” category in “Store A”, the OW output is 100 units. The ISS for T-shirts suggests a size distribution of Small:Medium:Large as 2:5:3. The minimum display option for any unique T-shirt style is 2 units. However, Store A has a base stock override for “Medium Black T-Shirt” to always be 5 units due to high local demand.

Calculation: 1. **Initial Distribution (based on OW=100 and ISS 2:5:3):** * Small: $100 * (2/10) = 20$ units * Medium: $100 * (5/10) = 50$ units * Large: $100 * (3/10) = 30$ units 2. **Apply Base Stock Override:** The “Medium Black T-Shirt” is part of the “Medium” size. If the initial calculation for “Medium Black T-Shirt” was 3 units, it would be overridden to 5 units. This might slightly reduce units allocated to other

“Medium” T-shirt styles to maintain the overall 50-unit allocation for Mediums, or the total OW might slightly increase depending on system logic for overrides. 3. **Apply Minimum Display:** If a specific “Yellow Small T-Shirt” was calculated to have only 1 unit based on its ROS, it would be adjusted up to 2 units to meet the minimum display requirement.

End Result: The system outputs the precise number of units for each T-shirt SKU (e.g., 20 Small T-shirts across various colors/styles, 52 Medium T-shirts - including the 5 “Medium Black T-shirts”, and 28 Large T-shirts), ensuring all business rules are met.

Key Takeaway: The module dynamically balances demand projections, pre-defined product mix strategies (ISS), and specific operational requirements (minimum display, base stock) to create actionable merchandising plans.

7.1.7 Results

The Planogram Creation Module outputs a detailed planogram configuration, including:

- **Final Optimum Depth (OD):** The ideal number of unique items (SKUs) to stock.
- **SKU-level Quantities:** The exact number of units for each SKU to be displayed.
- **Display Assignments:** Which SKUs are assigned to which display fixtures or locations (if fixture data is an input).
- **Compliance Report:** A summary indicating adherence to or deviations from business rules (e.g., minimum display met).

7.1.8 Unusual Cases

- **Missing or delayed data feed:** If OW, ISS, or base stock data is incomplete or arrives late, calculations may default to conservative values or be stalled.
- **Insufficient display space:** If calculated optimal quantities exceed physical fixture capacity, the module must prioritize or truncate the assortment.
- **Conflicting business rules:** Overlapping or contradictory base stock overrides and minimum display rules might require manual intervention or specific conflict resolution logic.
- **Rapidly changing demand:** Products with sudden spikes or drops in ROS might lead to sub-optimal initial calculations, requiring quick recalculation.

7.1.9 Next Steps

1. **Output Synchronization:** The generated planogram data is transmitted to downstream systems (e.g., inventory management, store operations) via the UtilOutputSyncModule.
2. **Physical Implementation:** Store teams execute the planogram directives by arranging products according to the specified configurations.
3. **Performance Monitoring:** Sales and inventory data are tracked to assess the effectiveness of the new planogram.

7.1.10 Common Questions

- 1. **What if there’s not enough space on the fixture for the calculated items?**
 - The system typically has a prioritization logic, often reducing the lowest-performing items or allowing for manual adjustment.
- 2. **How are new products or categories handled that don’t have historical sales data?**
 - New products usually leverage pre-defined attributes, comparable product data, or initial manual forecasts before sufficient ROS data accumulates.
- 3. **Can I override specific quantities after the system runs?**
 - Yes, base stock overrides are designed for this, and often a manual review process allows for further adjustments before finalization.
- 4. **What if the Ideal Size Set (ISS) is not accurate for a specific store’s demographics?**
 - The system can incorporate store-specific ISS variations or regional preferences if available in the input data.
- 5. **How does this module impact our inventory levels and stockouts?**
 - By aligning display quantities closely with demand and buffer stock, it aims to reduce excess inventory while minimizing the occurrence of stockouts.

7.1.11 Quick Reference

| Feature | Description |
|----------------|--|
| Purpose | Calculate optimal product display configurations (final OD/ options and quantities) for each store, category, and Attribute Group (AG). |
| Key Inputs | Optimum Width (OW) quantities, Ideal Size Set (ISS) values, minimum display options, store-level base stock overrides, Rate of Sale (ROS). |
| Key Outputs | Final Optimum Depth (OD), SKU-level unit quantities per display, compliance reports. |
| Business Value | Maximizes sales, minimizes stockouts, optimizes display efficiency, ensures adherence to merchandising rules. |
| Dependencies | Requires accurate OW, ISS, and sales data from upstream processes; feeds data to output synchronization. |

7.2 Planogram Creation Orchestration Module

This module coordinates the overall execution flow for planogram generation.

7.2.1 Module Name & Business Objective

Module Name: Planogram Creation Orchestration Module **Business Objective:** To provide a robust and sequenced workflow for the entire planogram creation process. It ensures that all necessary preliminary actions (like data snapshots) are completed before the core planogram calculations begin, and that all post-processing steps (like output uploading) are executed correctly. Its primary role is to manage dependencies and streamline the generation of complete and reliable planogram outputs.

7.2.2 Required Information

- **Output Snapshots:** Data generated by the UtilOutputSnapshotModule (e.g., historical planogram configurations, raw input data state before processing).
- **Planogram Output:** The calculated display configurations from the Planogram Creation Module, which need to be uploaded and synchronized.

7.2.3 Business Settings

- **Execution Schedule:** Configuration for how often the orchestration module runs (e.g., daily, weekly, on-demand).
- **Module Enable/Disable Flags:** Flags to selectively enable or disable specific sub-modules within the pipeline (e.g., skip snapshotting for a test run).

7.2.4 Process Flow

graph TD

```
A[Start Orchestration] --> B{UtilOutputSnapshotModule: Capture Prerequisite Data}
B --> C{Planogram Creation Module: Calculate Planograms}
C --> D{UtilOutputSyncModule: Upload & Synchronize Output}
D --> E[End Orchestration]
```

7.2.5 Step-by-Step Actions

1. **ORCHESTRATE** the complete workflow, initiating the process from a predefined starting point. *This ensures a consistent and controlled execution environment.*
2. **ENSURE** prerequisite steps are executed, specifically by invoking the UtilOutputSnapshotModule to capture snapshots of current outputs. *This provides a reliable state for auditing, rollback, and historical analysis.*
3. **MANAGE** the execution of the core Planogram Creation Module. *This is the central step where the actual planogram calculations take place.*
4. **COORDINATE** post-processing actions, including the invocation of UtilOutputSyncModule to upload the newly generated planogram output. *This ensures that the final results are propagated to consuming systems.*
5. **SEQUENCE** the execution of dependent modules in the correct order. *This prevents errors due to missing data or uncompleted preceding steps.*
6. **FACILITATE** a complete and robust planogram generation process by handling inter-module dependencies and ensuring smooth transitions. *This leads to reliable and timely planogram updates.*

7.2.6 Sample Scenario

Scenario: A retail company decides to generate new planograms every Sunday night to be implemented by Monday morning. The Planogram Creation Orchestration Module is scheduled to run at 11:00 PM every Sunday.

Execution Flow: 1. **11:00 PM Sunday:** The Orchestrator starts. 2. **11:01 PM Sunday:** It first calls UtilOutputSnapshotModule to take a snapshot of the current week's planograms and related data, archiving them. 3. **11:15 PM Sunday:** Once the snapshot is confirmed, it triggers the Planogram Creation Module, which

performs all the complex calculations for the new week's planograms based on the latest sales data and business rules. This process might take an hour. 4. **12:15 AM Monday:** After the planograms are calculated, the Orchestrator initiates `UtilOutputSyncModule` to upload the newly generated planogram files to the central database and relevant store systems. 5. **12:30 AM Monday:** The Orchestrator reports successful completion.

End Result: By Monday morning, all stores have access to the latest, optimized planograms, ready for implementation, with a clear historical record of the previous week's configurations.

Key Takeaway: The orchestrator module ensures that the entire planogram generation process runs automatically and reliably, managing dependencies and providing necessary data integrity steps like snapshotting.

7.2.7 Results

The Planogram Creation Orchestration Module delivers:

- **Complete Planogram Generation:** A fully executed workflow from data preparation to output delivery.
- **Auditable Process:** Snapshots of input and output data are available, supporting traceability and debugging.
- **Automated Workflow:** Reduced manual intervention for complex multi-step processes.

7.2.8 Unusual Cases

- **One sub-module fails:** If Planogram Creation Module encounters an error, the orchestrator should gracefully handle the failure, log the error, and potentially stop further execution or retry.
- **Database connectivity issues:** Inability to access input data or write output can halt the process.
- **Insufficient system resources:** Lack of memory or CPU might cause timeouts or slow down the process considerably.
- **Permissions errors:** The system might not have the necessary permissions to read/write files or interact with databases.
- **Missing or delayed data feed:** While the sub-modules handle missing data internally, the orchestrator needs to ensure the inputs are available before starting the core calculation.

7.2.9 Next Steps

1. **Process Monitoring:** Regular monitoring of the orchestrator's execution status and logs for any failures or anomalies.
2. **Alerting:** Setting up alerts for critical failures or completion status to relevant teams.
3. **Performance Optimization:** Reviewing execution times and resource utilization to identify areas for improvement.

7.2.10 Common Questions

- 1. What happens if one step in the orchestration fails?**
 - The orchestrator typically logs the error, and depending on configuration, may halt the entire process or attempt a retry. Alerts are usually sent.
- 2. How often does this module run, and can we change the schedule?**
 - The schedule is configurable (e.g., via cron jobs or internal schedulers) and can be adjusted by operations teams.
- 3. Can I re-run a specific step if it failed, without running the whole process?**
 - While the orchestrator runs the full flow, specific sub-modules might be callable independently for debugging or focused re-runs, but this is usually for development/support.
- 4. What is the exact order of operations within the orchestration?**
 - The order is defined programmatically: snapshotting, then core calculation, then output synchronization, as depicted in the process flow.
- 5. How do I check the progress or status of a running orchestration?**
 - Monitoring tools, logs, and potentially a dedicated dashboard can provide real-time status updates.

7.2.11 Quick Reference

| Feature | Description |
|-----------------------|--|
| Purpose | Orchestrates the entire planogram creation workflow, managing prerequisite and post-processing steps. |
| Key Inputs | Triggers (e.g., schedule), successful completion signals from sub-modules, data for snapshotting and synchronization. |
| Key Outputs | Successfully executed planogram generation process, synchronized outputs, log of execution status. |
| Business Value | Automates and streamlines the complex planogram generation, ensures data integrity, and improves process reliability. |
| Dependencies | Depends on the proper functioning of the Planogram Creation Module and utility modules (UtilOutputSnapshotModule, UtilOutputSyncModule). |

8. Appendix

8.1 Full Parameter Reference

Please refer to the `args.properties` file in the root directory of the repository for a complete list of configurable parameters and their descriptions.

8.2 Version History

- **Version 1.0 (2025-05-23):** Initial release of the planogramCreation documentation.

8.3 Contact & Support Info

For any questions, support, or feedback, please contact:

- **Support Team:** support@yourcompany.com
- **Documentation Feedback:** documentation@yourcompany.com
- **Internal Chat Channel:** #planogram-support (e.g., on Slack/Teams)

Tutorial: planogramCreation

The Planogram Creation Group Module (Index 1) orchestrates the entire planogram generation workflow. It coordinates the execution of the core Planogram Creation Module (Index 0) and manages prerequisite and post-processing steps such as data snapshots and output synchronization.

flowchart TD

```
A0["Planogram Creation Module"]
A1["Planogram Creation Orchestration Module"]
A1 -- "orchestrates" --> A0
A1 -- "uses" --> A0
```

Chapters

Master Report: Merchandising Algorithm - planogramCreation

1. Title & Metadata

- **Project Name:** planogramCreation
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** An intelligent algorithm designed to optimize product assortment, sizing, and placement on store planograms to maximize sales efficiency and improve customer experience.

2. Executive Summary

This Master Report provides comprehensive documentation for the planogramCreation merchandising algorithm. It outlines the system's architecture, data flow, configuration, and operational procedures, detailing how it transforms raw sales and inventory data into actionable planogram recommendations. The algorithm automates the complex process of merchandising planning, ensuring optimal product presentation across various retail environments.

- **Key outcomes or KPIs:**
 - Recommendations for **OD** (Optimum Depth) at the category and SKU level, minimizing unproductive stock and maximizing sales per linear foot.
 - Suggestions for **OW** (Optimum Width), ensuring a balanced and appealing range of categories and styles.
 - Derivation of **ISS** (Ideal Size Set) to prevent stockouts and reduce excess inventory for apparel and other size-specific products.
 - Improvement in overall **ROS** (Rate of Sale) by optimizing product visibility and availability.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Category Management, Store Operations, Inventory Planning, Space Planning Analysts.

3. Architecture & Data Flow

The planogramCreation algorithm operates through a sequential pipeline, ensuring data integrity and logical processing from input to output.

graph TD

```
A[Raw Data Ingestion] --> B{Data Validation & Preprocessing};  
B --> C[Assortment Optimization - OD & OW];  
C --> D[Size Set Determination - ISS];
```

```

D --> E[Space Allocation & Placement];
E --> F[Output Generation & Reporting];
F --> G[Planogram Files];
F --> H[Assortment Recommendations];
F --> I[Performance Metrics];

```

- **Component Bullets:**

- **Raw Data Ingestion:** Gathers data from various sources.
- **Data Validation & Preprocessing:** Cleans, transforms, and validates raw data, preparing it for analytical modules.
- **Assortment Optimization (OD & OW):** Analyzes historical performance to recommend the ideal breadth and depth of products.
- **Size Set Determination (ISS):** Calculates the optimal size distribution for SKUs based on sales patterns.
- **Space Allocation & Placement:** Maps optimized assortments and sizes to physical display layouts, considering constraints.
- **Output Generation & Reporting:** Formats and exports the final planogram, recommendations, and analytical reports.

- **High-level data sources:**

- Historical sales data (units sold, revenue, ROS by SKU, store, and date)
- Current inventory levels (stock on hand, in transit)
- Product master data (SKU attributes, category hierarchies, dimensions)
- Store layout configurations (shelf dimensions, display capacities, existing planograms)
- Supplier/Vendor lead times and minimum order quantities (optional, for advanced inventory considerations)

4. Configuration & Data Dictionary

Settings Table (args.properties)

The primary configuration file for the algorithm, controlling global parameters and thresholds.

| Parameter Name | Description | Example Value |
|------------------------|---|---------------|
| min_ros_threshold | Minimum ROS required for a SKU to be considered for OD inclusion. | 0.5 |
| max_display_capacity | Maximum physical units a display area can hold. | 500 |
| od_optimization_factor | Multiplier for OD recommendations to account for buffer stock/flexibility. | 1.1 |
| size_curve_smoothing | Level of smoothing applied to historical size sales data for ISS . | median |
| min_category_width | Minimum OW (number of categories) for any given store. | 5 |
| output_format | Desired output file format for planogram recommendations. | CSV |
| data_retention_days | | 365 |

| Parameter Name | Description | Example Value |
|----------------|--|---------------|
| | Number of days of historical data to use for analysis. | |

Shared Data Fields (common-data.json)

These fields are standardized and used across multiple modules for consistency.

| Field Name | Data Type | Description | Example Value |
|------------------------|-----------|---|-----------------|
| sku_id | String | Unique identifier for a product (SKU). | PROD12345 |
| category_id | String | Identifier for the product category. | APPAREL_TSHIRTS |
| store_id | String | Unique identifier for a retail store. | STORE_NYC_001 |
| sales_units | Integer | Number of units sold over a period. | 150 |
| inventory_units | Integer | Current number of units in stock. | 30 |
| ros_units_week | Float | ROS (Units/Week) for the SKU. | 12.5 |
| size_name | String | Specific size attribute (e.g., S, M, L, XL). | M |
| attribute_group_name | String | Name of the AG (e.g., Color, Material). | Color |
| attribute_value | String | Value of the attribute (e.g., Blue, Cotton). | Blue |
| display_capacity_units | Integer | Capacity of a specific display location in units. | 20 |

Outputs Catalog

The primary outputs generated by the planogramCreation algorithm.

| Output Name | Description | Format | Key Metrics Included |
|-------------------------|---|--------|---|
| optimized_planogram.csv | Detailed planogram layout with SKU assignments to | CSV | sku_id, store_id, display_location_id, units_assigned, size_breakdown |

| Output Name | Description | Format | Key Metrics Included |
|---------------------------------|---|--------|--|
| | specific locations. | | |
| assortment_recommendations.json | Recommended OD and OW for categories and sub-categories. | JSON | category_id, recommended_od_skus, recommended_ow_styles |
| size_set_analysis.csv | Detailed breakdown of ISS per SKU, per store. | CSV | sku_id, store_id, size_name, recommended_units |
| performance_summary.html | High-level summary of optimization impact (e.g., projected ROS uplift). | HTML | total_projected_ros, inventory_reduction_pct, service_level_impact |

5. End-to-End Flow

The `planogramCreation` algorithm initiates by ingesting raw sales, inventory, product master, and store layout data. This raw data undergoes rigorous validation and preprocessing, where it's cleaned, standardized, and enriched with calculated metrics like historical **ROS**.

Once prepared, the data flows into the **Assortment Optimization** module. Here, sophisticated algorithms analyze sales velocity, profitability, and customer demand to determine the **OD** (Optimum Depth) – the ideal number of unique SKUs within a category – and the **OW** (Optimum Width) – the ideal variety of categories or styles to offer. This step leverages **AG** (Attribute Group) analysis to identify high-performing product characteristics.

Subsequently, the **Size Set Determination** module takes the optimized assortment and calculates the **ISS** (Ideal Size Set) for each recommended SKU, ensuring that the right proportion of sizes (e.g., S, M, L, XL) is stocked to meet anticipated demand and minimize stockouts or overstock. This is particularly crucial for apparel and footwear.

With the optimized assortment and size sets defined, the **Space Allocation & Placement** module takes over. It maps these recommendations onto available display capacities and store layouts, considering physical constraints, adjacencies, and visual merchandising principles. This results in a precise blueprint of where each product and size should be placed.

Finally, the **Output Generation & Reporting** module compiles all findings into user-friendly formats. This includes detailed planogram files, comprehensive assortment recommendations, granular size set analyses, and performance summaries. These outputs empower merchandising teams to implement data-driven strategies for product presentation and inventory management.

6. Submodule Overview

6.1. Data Ingestion & Validation

1. **Name & Purpose:** Ingests raw data from various sources and performs initial validation and cleaning to ensure data quality.
2. **Inputs & Outputs:**
 - **Inputs:** Raw sales data, inventory data, product master data, store layout data (CSV, JSON, DB connections).
 - **Outputs:** Cleaned and validated data tables/dataframes ready for processing.
3. **Core Logic Summary:**
 - Connects to specified data sources and loads raw datasets.
 - Performs schema validation, checking for missing or malformed fields.
 - Applies data cleaning rules (e.g., handling nulls, type conversions, removing duplicates).
 - Calculates basic metrics like historical **ROS** (Units/Week) for all SKUs and categories.

6.2. Assortment Optimization

1. **Name & Purpose:** Determines the **OD** (Optimum Depth) of SKUs within categories and the **OW** (Optimum Width) of categories/styles to offer.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned sales data (ROS), inventory data, product master data (including **AGs**), configuration settings (min_ros_threshold, min_category_width).
 - **Outputs:** Recommended list of SKUs for inclusion, along with their target **OD** and **OW** values per category/store.
3. **Core Logic Summary:**
 - Analyzes historical **ROS** and profitability for each SKU and category.
 - Identifies top-performing SKUs and categories based on configured thresholds.
 - Evaluates product attributes via **AGs** to ensure a diverse yet profitable **OW**.
 - Calculates the **OD** for each category/sub-category, balancing variety with efficiency.

6.3. Size Set Determination

1. **Name & Purpose:** Calculates the **ISS** (Ideal Size Set) for each recommended SKU, ensuring optimal size distribution.
2. **Inputs & Outputs:**
 - **Inputs:** Recommended SKU list from Assortment Optimization, historical size-specific sales data, configuration settings (size_curve_smoothing).
 - **Outputs:** Detailed **ISS** breakdown for each SKU, specifying units per size.
3. **Core Logic Summary:**
 - Aggregates historical sales data by SKU and size.
 - Applies statistical methods (e.g., weighted average, smoothing) to forecast size demand.
 - Generates an **ISS** recommendation (e.g., number of S, M, L, XL units) based on demand curves and available display space.

- Adjusts **ISS** to account for minimum display requirements and common pack sizes.

6.4. Space Allocation & Placement

1. **Name & Purpose:** Maps the optimized assortment and size sets to specific physical locations on store planograms, considering capacity and merchandising rules.
2. **Inputs & Outputs:**
 - **Inputs:** Recommended SKUs with **ISS**, store layout data (display capacities, adjacencies), configuration settings (`max_display_capacity`).
 - **Outputs:** Proposed planogram assignments with SKU, size, and unit count per location.
3. **Core Logic Summary:**
 - Takes the **OD** and **ISS** recommendations and allocates them to available display segments.
 - Applies constraints such as `max_display_capacity`, product dimensions, and adjacency rules.
 - Prioritizes high-**ROS** items for premium placement (e.g., eye-level shelves).
 - Generates a detailed layout plan, mapping each SKU and its recommended units to a specific shelf/position.

6.5. Reporting & Export

1. **Name & Purpose:** Generates human-readable reports and machine-readable files from the algorithm's outputs.
2. **Inputs & Outputs:**
 - **Inputs:** Outputs from Assortment Optimization, Size Set Determination, and Space Allocation modules.
 - **Outputs:** `optimized_planogram.csv`, `assortment_recommendations.json`, `size_set_analysis.csv`, `performance_summary.html`.
3. **Core Logic Summary:**
 - Consolidates data from various internal processing steps.
 - Formats the data into specified output formats (CSV, JSON, HTML).
 - Generates summary reports, including projected performance gains (e.g., improved **ROS**, reduced stockouts).
 - Ensures all output files are named and stored according to predefined conventions.

7. Worked Example

Scenario: Optimizing the “Men’s Casual T-Shirts” category for `STORE_NYC_001`.

Input Data Snippet: * `SKU_A` (Blue T-Shirt): **ROS** 15 units/week, current inventory 20 units. Sizes sold: S:20%, M:40%, L:30%, XL:10%. * `SKU_B` (Red T-Shirt): **ROS** 8 units/week, current inventory 10 units. Sizes sold: S:25%, M:35%, L:25%, XL:15%. * `SKU_C` (Green T-Shirt): **ROS** 3 units/week, current inventory 5 units. Sizes sold: S:10%, M:30%, L:40%, XL:20%. * Display capacity for “Men’s Casual T-Shirts”: 50 units. * `min_ros_threshold`: 5 units/week.

End-to-End Flow:

1. **Data Ingestion & Validation:** Reads the above data, confirming all fields are present and ROS is correctly calculated.
2. **Assortment Optimization (OD & OW):**
 - Compares SKU_A (15 ROS) and SKU_B (8 ROS) against `min_ros_threshold` (5 ROS) – both are above.
 - SKU_C (3 ROS) is below the threshold.
 - The algorithm recommends an **OD** of 2 SKUs for this category (SKU_A, SKU_B). SKU_C is recommended for discontinuation or reduced depth.
 - **OW** for “T-Shirts” is confirmed as sufficient based on product attributes (colors).
3. **Size Set Determination (ISS):**
 - For SKU_A (Blue T-Shirt), based on its sales curve (S:20%, M:40%, L:30%, XL:10%) and an allocated depth, if we decide to stock 25 units:
 - **ISS:** S: 5 units, M: 10 units, L: 7 units, XL: 3 units.
 - For SKU_B (Red T-Shirt), based on its sales curve (S:25%, M:35%, L:25%, XL:15%) and an allocated depth, if we decide to stock 15 units:
 - **ISS:** S: 4 units, M: 5 units, L: 4 units, XL: 2 units.
4. **Space Allocation & Placement:**
 - Total recommended units: 25 (SKU_A) + 15 (SKU_B) = 40 units. This fits within the 50-unit display capacity.
 - The module assigns SKU_A (25 units with its ISS) and SKU_B (15 units with its ISS) to specific shelf locations within the “Men’s Casual T-Shirts” display, potentially placing SKU_A in a more prominent position due to its higher ROS.
5. **Reporting & Export:**
 - `optimized_planogram.csv` will contain entries like: SKU_A, STORE_NYC_001, Shelf1, S:5, M:10, L:7, XL:3.
 - `assortment_recommendations.json` will suggest “Men’s Casual T-Shirts” should carry SKU_A and SKU_B.
 - `size_set_analysis.csv` will detail the ISS for each SKU.

This example illustrates how the algorithm takes raw data and translates it into concrete, optimized merchandising decisions.

8. Debug Guide Highlights

- **Error:** Input file not found: [filename]
 - **Highlight:** The specified input data file (e.g., sales, inventory) is missing or incorrectly named/pathed.
 - **Debug Step:** Verify the file path in `args.properties` and ensure the file exists at that location. Check file permissions.
- **Error:** Constraint violation: Display capacity exceeded for [location_id]
 - **Highlight:** The algorithm attempted to assign more units than the physical display capacity allows.
 - **Debug Step:** Review `max_display_capacity` in `args.properties` and the `display_capacity_units` in the store layout data. Assess if the **OD** or **ISS** is too aggressive for the available space. Consider adjusting `od_optimization_factor`.

- **Error:** Zero ROS for critical SKUs. Check data quality.
 - **Highlight:** Several SKUs designated as important (e.g., core assortment) have no sales data or zero **ROS**, leading to poor recommendations.
 - **Debug Step:** Investigate the raw sales data for these SKUs. Are they new? Is there a data ingestion issue? Adjust `min_ros_threshold` if new products are expected to have low initial ROS.
- **Warning:** Low diversity in OW recommendations for category `[category_id]`
 - **Highlight:** The Assortment Optimization module is recommending a very narrow range of styles or attributes for a category.
 - **Debug Step:** Review the **AGs** defined for that category. Ensure there's sufficient diversity in product attributes in the input data. Adjust `min_category_width` or related parameters if broader variety is desired.

9. Common Pitfalls & Remedies

- **Pitfall: Over-optimization leading to stockouts for fast-moving items.**
 - **Cause:** `od_optimization_factor` is set too low, or `min_ros_threshold` is too high, aggressively cutting down **OD**.
 - **Remedy:** Increase `od_optimization_factor` to build in a safety buffer for high-ROS items. Lower `min_ros_threshold` slightly or implement a tiered threshold based on SKU importance.
- **Pitfall: Ignoring local store variations for ISS.**
 - **Cause:** The algorithm applies a global **ISS** curve without sufficient weighting for individual store sales patterns.
 - **Remedy:** Ensure the Size Set Determination module uses store-specific historical sales data for **ISS** calculation where available, or implements clustering for stores with similar demographics.
- **Pitfall: Stale data leading to irrelevant recommendations.**
 - **Cause:** The `data_retention_days` parameter is too high, or data ingestion is not frequent enough.
 - **Remedy:** Schedule more frequent data refreshes. Reduce `data_retention_days` to focus on more recent trends, especially in fast-changing categories.
- **Pitfall: Planogram outputs are physically impractical or costly to implement.**
 - **Cause:** Lack of granular physical constraints in store layout data, or the Space Allocation & Placement module doesn't account for labor costs of frequent changes.
 - **Remedy:** Enhance store layout data with more detailed constraints (e.g., shelf height, adjacency rules for large items). Consider introducing a 'change cost' variable in the optimization objective.

10. FAQ

- **Q: How often should I rerun the `planogramCreation` algorithm?**
 - **A:** It depends on your business's pace of change. For fast-moving categories or seasonal products, monthly or bi-weekly runs are recommended. For stable, core categories, quarterly or bi-annual runs might suffice. Consult your Category Management team for category-specific needs.

- **Q: Can I manually override the algorithm's recommendations?**
 - A: Yes, the outputs (e.g., `optimized_planogram.csv`) are designed to be easily editable. While manual adjustments are possible, it's advised to understand the algorithm's rationale to avoid unintended consequences for **ROS**, **OD**, or **ISS**. Future versions may include a feedback loop for manual overrides.
- **Q: What if I have new products with no sales history?**
 - A: For new products, the algorithm uses fallback strategies such as category-average **ROS** and **ISS** based on similar **AGs**. It's crucial to ensure new products are correctly tagged with relevant attributes.
- **Q: How does the algorithm handle promotions or seasonal spikes?**
 - A: The algorithm primarily uses historical **ROS** for its calculations. For strong promotional impacts or known seasonality, consider integrating a forecasting module upstream or manually adjusting the `od_optimization_factor` for relevant periods to account for expected surges.
- **Q: Is it possible to optimize for profit margin instead of just ROS?**
 - A: Yes, if gross margin data per SKU is available in the input, the algorithm can be configured to include it in the optimization objective, leading to recommendations that balance **ROS** with profitability. This would require adjusting core logic in the Assortment Optimization module.

11. Quick Reference Cheat-Sheet

| Abbreviation | Business Definition | Role in <code>planogramCreation</code> |
|--------------|--|--|
| OD | Optimum Depth: The ideal number of unique items (SKUs) to stock in a category or display. | Determined by Assortment Optimization; impacts inventory efficiency and choice. |
| OW | Optimum Width: The ideal number of different categories or styles to offer. | Determined by Assortment Optimization; ensures balanced and attractive product range. |
| ISS | Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts. | Calculated by Size Set Determination; crucial for apparel and varied size products. |
| ROS | Rate of Sale: The speed at which products are sold over a period (e.g., units/week). | Key performance metric and input for OD , OW , and placement decisions. |
| AG | Attribute Group: A set of product attributes grouped for analysis (e.g., color, size, brand). | Used in Assortment Optimization to analyze product characteristics and ensure diversity. |

Key Configuration Parameters: * `min_ros_threshold`: Minimum **ROS** for SKU inclusion. * `max_display_capacity`: Physical limit for display area. * `od_optimization_factor`: Safety buffer for **OD** recommendations.

Main Outputs: * optimized_planogram.csv: Detailed product placement. *
assortment_recommendations.json: **OD & OW** suggestions. *
size_set_analysis.csv: **ISS** breakdown per SKU.

Chapter 1: Data View

Last Updated: 2025-05-23

1. Business Objective

This module provides foundational insights about product performance and availability, enabling better decisions for grouping products by price and assessing their market effectiveness.

Why This Matters: It helps us understand which products are selling well, where they are available, and for how long. This knowledge is key to creating price ranges that attract customers and maximize sales, preventing stockouts and ensuring we offer the right products at the right price points.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Helps determine product popularity and demand by showing how many items were sold over a period.
- **Sales Consistency Score:** Shows how reliably a product sells over time, which is important for future planning and pricing decisions.

Product Data

- **SKU Details:** Information about specific items like size, color, or material, necessary to track individual product variations.
- **Style Details:** Broader product grouping information such as brand, subcategory, and gender, for analyzing entire product lines.
- **Product Attribute Group (AG) Details:** A collection of product characteristics (e.g., fabric type, pattern) grouped for deeper analysis and categorization.

Operational Data

- **Store Availability:** Indicates whether a product was in stock and available for sale at a specific store on a given day, crucial for accurate sales analysis.
- **Live Days:** The total number of days a product or style was actually available for sale in a store or across stores, ensuring ROS calculations are fair.
- **Key Product Indicator:** Helps identify crucial products or sizes that should ideally always be in stock to maintain sales momentum.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|----------------------------------|-----------------------------------|--|---|
| Minimum Key Product Availability | 0% | The minimum percentage of Key Product Indicators (e.g., core sizes like medium or large) that must be available in a store for a product to be considered consistently available for sale. | If certain product lines consistently face stock issues for core sizes, or if the brand prioritizes broad availability over strict sales performance for certain items. |
| Analysis Period Start Date | (Dynamic, typically last 90 days) | The beginning date for calculating Rate of Sale (ROS) and product availability. | To analyze performance over different seasons, during promotions, or under specific market conditions. |
| Analysis Period End Date | (Dynamic, typically today) | The end date for calculating Rate of Sale (ROS) and product availability. | To ensure the most recent sales and stock data are considered for up-to-date insights. |
| Attribute Group (AG) Definitions | (Defined by business) | The specific product attributes (e.g., color, material, pattern) that are grouped together for analysis. | When new product lines are introduced, or if the business wants to analyze different product characteristics. |

Note: See More Details for additional settings related to specific ROS calculation methodologies.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales & Stock Data] --> B[Data View Layer: Calculate

Metrics]

B --> C[Derived Business Insights (ROS, Live Status)]

1. Start with → Raw sales figures, inventory levels, and product descriptions from all stores.
2. Then → **Identify** when each SKU and style was actually available for sale in each store.
3. Next → **Compute** the Rate of Sale (ROS) for individual SKUs and broader styles, considering only periods of availability.
4. Finally → **Provide** key metrics like total live days, overall ROS, and the percentage of Key Product Indicators (e.g., core sizes) available, for use in other modules.

5. Step-by-Step Actions

1. **Gather** raw sales and stock data
 - What we do: *We collect all relevant daily sales transactions and inventory snapshots from our central data repository.*
 - Why we do it: *This forms the basic information needed to understand product performance and availability.*
2. **Determine** product availability
 - What we do: *For every SKU and style, we check which stores had them in stock and available for sale on each specific day within the analysis period.*
 - Why we do it: *Products can only sell when they are available. Accurately tracking 'live days' ensures our sales calculations are fair and reflect actual selling opportunities, not just total days passed.*
3. **Calculate** Rate of Sale (ROS)
 - What we do: *We divide the total Units Sold for a SKU or style by the number of days it was actually available for sale in specific stores or across the entire business.*
 - Why we do it: *This gives us a true measure of how quickly products are selling when they are on the shelf. For example, a shirt might have sold 100 units in a month, but if it was only available for 10 days, its ROS is much higher than if it was available for all 30 days.*
4. **Assess** Key Product Availability
 - What we do: *We identify specific "Key Product Indicators" (like core sizes) for each style and calculate the percentage of these crucial items that are available at any given time.*
 - Why we do it: *Even if a style is technically "live," a low percentage of key sizes available can severely impact sales. This metric helps us understand if the assortment is truly healthy and able to meet customer demand.*

6. Sample Scenario

Let's walk through a real example for a popular blue denim jeans style (Style ID 123).

Starting Point

- Blue Denim Jeans (Style ID 123) consists of 5 SKUs (e.g., Waist 30, Waist 32, Waist 34, Waist 36, Waist 38).

- We want to analyze sales over the last 90 days at Store A.
- Store A sold 180 units of Style ID 123 in the last 90 days.
- The “Minimum Key Product Availability” is set to 70%.

What Happens

1. First, the Data View looks at each of the 5 SKUs for Style ID 123 in Store A for each of the last 90 days. It **identifies** that while the style was technically present in Store A for all 90 days, the crucial size “Waist 32” was out of stock for 30 of those days, and “Waist 34” was out for 15 days. All other sizes were consistently available.
2. Next, it **calculates** the “Key Product Indicator” availability for each day. If 4 out of 5 key sizes were available, that’s 80%. If only 3 were available, that’s 60%.
3. Then, it uses the “Minimum Key Product Availability” setting (70%) to **determine** “unbroken live days”. It counts only the days where at least 70% of the key sizes were available. Let’s say out of the 90 days, only 70 days met this 70% benchmark.
4. Finally, it **computes** the actual Rate of Sale (ROS) using these “unbroken live days”. The style sold 180 units over 70 *effective* live days.

Final Outcome

The Data View reports that Style ID 123 had an effective ROS of 180 units / 70 days = 2.57 units/day in Store A, with 70 “unbroken live days” and a high ROS.

Key Takeaway: By factoring in actual availability and the health of key product sizes, Data View provides a more realistic and useful ROS, ensuring subsequent analysis like price bucketing is based on reliable performance data.

7. Results

This step produces:

Main Outputs

- **Calculated Rate of Sale (ROS):** The actual speed at which individual SKUs and styles are selling, adjusted for their true availability. This helps prioritize products for replenishment and pricing.
- **Product Live Status:** Clear indicators of whether a SKU or style is currently available in a store or across the business, including the number of ‘live days’ for specific periods.
- **Key Product Availability Metrics:** Percentages indicating the consistent presence of crucial product sizes or variations, essential for understanding the health of an assortment.

Business Reports

- **Product Performance Dashboard:** Used by merchandising and planning teams to quickly see top-selling products, identify slow movers, and understand inventory health across stores.

- **Inventory Health Report:** Used by inventory managers and store operations to spot potential stockout risks, especially for key sizes, ensuring stores can meet customer demand and reduce lost sales.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed | The system will use the last available data, or flag that information is incomplete. | Verify data sources, contact IT support, or manually provide missing information if critical for immediate decisions. |
| Inaccurate ‘live’ status reported | ROS calculations might be skewed, leading to over- or under-estimation of demand for specific products. | Cross-reference with physical inventory checks, adjust ‘live’ status manually if possible, and report data discrepancies to the operations team. |
| Very low or zero ‘unbroken live days’ for a product | This indicates a product was rarely or never consistently available, which will heavily impact its calculated ROS. | Investigate the root cause of the prolonged stockouts (e.g., supply chain issues, poor initial stock allocation, or misclassification). |
| All Key Product Indicators unavailable | The system might not be able to calculate a meaningful ROS for that style based on “unbroken live days.” | Review the product’s overall sales and inventory, and consider if its “Key Product Indicator” settings or stock levels need adjustment. |

9. Next Steps

Before This Step

- **Data Ingestion and Cleansing:** We need raw, accurate sales transactions, inventory records, and product master data (SKUs, styles, attributes) to be loaded and cleaned.
- **Attribute Group (AG) Definition:** We need the business to define how products are grouped by attributes (e.g., all red shirts, all denim jeans) to ensure meaningful analysis.

After This Step

- **Price Bucket Calculation:** The calculated ROS and live status data will be used to intelligently group products into price buckets (e.g., “value,” “mid-range,” “premium”).
- **Assortment Optimization:** The insights on product performance and availability help determine the OD (Optimum Depth) and OW (Optimum Width) for future collections, ensuring we stock the right products in the right quantities.

10. Common Questions

Q: How does this “Data View” help me prevent lost sales? A: By providing accurate ROS and showing when products (especially key sizes) are truly available, it helps you identify strong sellers that are frequently out of stock, allowing you to reorder them faster and reduce missed sales opportunities.

Q: Can I see how a style performs in specific stores versus the whole business? A: Yes, Data View allows you to analyze ROS and live status at a global level (across all stores) or drill down to individual store performance, helping you understand regional differences in demand and tailor assortments.

Q: What if a product has sold zero units but was available for a long time? A: The Data View would report a very low or zero ROS, clearly indicating it’s a slow-moving item. This insight helps you make decisions about discounting, markdowns, or removing it from the assortment to free up valuable shelf space.

Q: How does “Minimum Key Product Availability” affect my understanding of product performance? A: This setting ensures that we only count days where a product’s core offering (e.g., its most popular sizes) was adequately stocked. This prevents overstating ROS for products that were only partially available, giving a more realistic view of true sales potential.

Q: What are “unbroken live days,” and why are they important? A: “Unbroken live days” are the specific days a product or style was considered truly “available for sale” because a sufficient percentage of its core sizes were in stock. This is crucial because it ensures that our Rate of Sale (ROS) calculations are based on genuine selling opportunities, not just calendar days, leading to more accurate business decisions.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Provides accurate, derived business insights (like ROS and live status) from raw data for informed merchandising and pricing decisions.
- **Main Input:** Raw sales transactions, inventory records, and product master data.
- **Main Output:** Calculated Rate of Sale (ROS), product live status, and Key Product Availability metrics.
- **Warning Signs:** Discrepancies between reported live days and actual stock levels, or very low “unbroken live days” for popular items.
- **Key Settings:** Minimum Key Product Availability, Analysis Period Start/End Dates, Attribute Group (AG) Definitions.

Need Help? Contact your Merchandising Systems Support team for questions about this process.

Chapter 2: Data Cache

Last Updated: 2025-05-23

1. Business Objective

This module acts as the central hub for all vital retail data, ensuring quick and reliable access to product, sales, and inventory information needed for effective price bucketing.

Why This Matters: It helps us make faster, smarter decisions about product pricing and assortment planning by providing immediate access to accurate information. This means we can quickly adapt to market changes, set competitive prices, and ensure we have the right products in stock, which ultimately boosts sales and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Essential to understand historical product performance and demand.
- **Product Sales Transactions:** Needed to track individual sales events for detailed analysis, including revenue contributions.

Product Data

- **SKU Details:** Provides specific attributes (like size, color, or material) for each unique item.
- **Style Details:** Offers broader product grouping information (brand, subcategory, and gender) for understanding entire product lines.
- **Attribute Group (AG) Definitions:** Specifies how product characteristics (e.g., fabric type, pattern) are grouped for deeper merchandising analysis.
- **Price Bucket Definitions:** Pre-defined price ranges and rules that products might fall into.

Operational Data

- **Store Details:** Information about each retail location, including channel and region, for localized analysis.
- **Inventory Levels:** Crucial for understanding current stock availability in stores and warehouses.

- **Store Planogram Data:** Details the layout and merchandising rules for products within stores, influencing optimal placement and display.
- **Size Mapping:** Standardizes various product sizes (e.g., “S”, “Small”) into a consistent format for analysis.

Note: See More Details for additional settings related to specific inventory and product classification methodologies.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-----------------------------------|-----------------------------------|---|---|
| Product Inclusion Rules | All active products | Defines which styles and SKUs are loaded into the cache for analysis, typically excluding discontinued or irrelevant items. | When new product lines are introduced, or old ones are fully phased out. |
| Store Inclusion Rules | All active stores | Specifies which retail locations and channels are considered in the analysis. | When new stores open, stores close, or specific channels are focused on for a campaign. |
| Transactional Data Horizon | (Dynamic, typically last 90 days) | The time window for loading historical sales and inventory transactions into memory. | To analyze performance over different seasons, during promotions, or under specific market conditions. |
| Data Refresh Frequency | Daily | How often the cache is fully updated with the latest master and transactional data. | If business decisions require real-time updates (e.g., during high-traffic sales events), or if data latency is acceptable. |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

A[Raw Master Data] --> B[Load & Organize Data]
C[Raw Transactional Data] --> B

```

B --> D[In-Memory Data Cache]
D --> E[Price Bucket Creation Process]

1. Start with → Raw Master Data (product details, store information) and Raw Transactional Data (sales, inventory movements).
2. Then → **Load** and **Organize** all this data into structured formats.
3. Next → **Store** the organized data efficiently in memory for quick access.
4. Finally → **Provide** immediate access to this consolidated data for the priceBucketCreation process and other related analyses.

5. Step-by-Step Actions

1. **Gather** raw business data
 - What we do: *We collect all relevant daily sales transactions, inventory snapshots, and product master data (SKUs, styles, store information) from our central data repository.*
 - Why we do it: *This is the foundational information required for all subsequent analyses, ensuring we have a complete picture.*
2. **Filter** and **Normalize** incoming data
 - What we do: *We apply pre-defined business rules to include only active products and stores, and convert product sizes to a standardized format.*
 - Why we do it: *This ensures that the cache only holds relevant, consistent data, preventing analysis errors and improving efficiency.*
3. **Organize** data into accessible structures
 - What we do: *We transform the raw data into optimized in-memory structures, like organized lists or dictionaries, for very fast lookups by ID or key.*
 - Why we do it: *This dramatically speeds up data retrieval for downstream processes like calculating ROS or identifying Optimum Width (OW) and Optimum Depth (OD), which are crucial for timely decision-making.*
4. **Populate** the central data cache
 - What we do: *All processed master and transactional data is loaded into the Data Cache, ready for immediate use by various modules within the priceBucketCreation project.*
 - Why we do it: *By having all necessary data pre-loaded and centrally available, we eliminate the need for repeated database queries, making complex calculations and analyses much faster and more responsive.*

6. Sample Scenario

Let's walk through a real example of how Data Cache prepares information for the priceBucketCreation process.

Starting Point

- A retail chain has thousands of SKUs and hundreds of stores.
- The priceBucketCreation process needs to calculate ROS for all active styles in North America for the last 90 days.
- It also needs details about each style's category, sub-category, and Attribute Group (AG) for grouping.

What Happens

1. First, the Data Cache **gathers** all raw product details (SKU, Style, Attribute Group definitions), store master data, and sales transactions for the last 90 days.
2. Then, it **filters** out any inactive styles or closed stores as per the Product Inclusion Rules and Store Inclusion Rules. For example, a style that was discontinued 6 months ago won't be loaded, saving memory and processing time.
3. Next, it **organizes** this massive amount of data. It creates organized lists of styles, SKUs, and stores, and processes all relevant sales records. It also maps SKU sizes like "Small" and "S" to a single "S" using standardized size mapping.
4. Finally, this prepared data is **populated** into the Data Cache, becoming immediately available. So, when the Data View module requests the sales for "Blue Denim Jeans (Style ID 123)", the cache can provide all 90 days of sales records for that style across all relevant stores instantly, along with its full attribute details.

Final Outcome

The priceBucketCreation process, and specifically the Data View module, receives a clean, filtered, and highly organized set of data, enabling rapid and accurate calculations of ROS and revenue contributions without querying slow databases.

Key Takeaway: The Data Cache ensures that all necessary data for price bucketing is immediately available and in an optimized format, allowing for quick and accurate analysis essential for timely pricing decisions.

7. Results

This step produces:

Main Outputs

- **In-Memory Master Data:** Instantly accessible details for all active SKUs, Styles, Stores, and Attribute Groups, optimized for rapid lookups.
- **Filtered Transactional Data:** Sales and inventory data for the specified analysis period, stripped of irrelevant or outdated entries.
- **Standardized Data Structures:** All data is consistently formatted and organized into predictable structures to support complex calculations.

Business Reports

- **System Performance Metrics:** Used by IT and data teams to monitor the efficiency and responsiveness of the data loading and caching process.
- **Data Readiness Status:** Used by data analysts and merchandisers to confirm that all necessary data is loaded and ready for their analysis.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|--|
| Missing or delayed data feed | The cache will load stale or incomplete data, potentially leading to outdated or inaccurate analysis in downstream processes. | Verify data sources, contact IT support immediately, and pause downstream processes until data integrity is confirmed. |
| Incorrect Product/Store Inclusion Rules | The cache might exclude necessary products or stores, or include irrelevant ones, leading to incomplete or skewed analysis. | Review the Product Inclusion Rules and Store Inclusion Rules settings with the merchandising team and adjust filters as needed. |
| Excessive Data Volume | If the volume of transactional data (e.g., sales records) exceeds memory capacity, the cache might fail to load or perform very slowly. | Optimize data retention policies, review the Transactional Data Horizon setting, or explore distributed caching solutions with IT. |
| Inconsistent Size Mapping | Different product sizes (e.g., “S”, “Small”) are not normalized correctly, leading to incorrect aggregation of sales or inventory. | Review and update the Size Mapping configurations to ensure all size variations are standardized properly. |

9. Next Steps

Before This Step

- **Raw Data Ingestion:** We need raw, accurate sales transactions, inventory records, and product master data to be collected from various source systems.
- **Data Cleansing:** The raw data should undergo initial cleansing to fix basic errors and inconsistencies before being loaded into the cache.

After This Step

- **Data View Calculation:** The Data Cache provides the raw data (product details, sales, availability) that the Data View module uses to calculate key metrics like ROS and product live status.
- **Price Bucket Calculation:** The prepared master data (Style, SKU, AG details, Price Bucket Definitions) from the cache is directly used to intelligently group products into OW price buckets.
- **Assortment Optimization:** The comprehensive product and sales data from the cache supports decisions on OD (Optimum Depth) and OW (Optimum Width) for assortments.

10. Common Questions

Q: Why is having a Data Cache important for price bucketing? A: It's like having all your ingredients prepped and ready for a meal; it ensures all the vast amounts of product, sales, and inventory data are immediately available and organized, allowing the priceBucketCreation process to run quickly and accurately. This means we can define price ranges that maximize revenue without delays.

Q: How does the Data Cache handle large amounts of sales data? A: The cache is designed to load and store relevant sales transactions directly in memory. This way, complex calculations for ROS (Rate of Sale) and revenue contributions can be performed extremely fast, without repeatedly querying a slower database.

Q: Can the Data Cache be updated with new product information in real-time? A: The Data Cache is typically updated on a set schedule (e.g., daily) to ensure consistent data. While it's not truly "real-time" down to the second, it provides very fresh data that is more than sufficient for pricing and merchandising decisions, which typically don't change by the minute.

Q: What happens if certain products are missing from the cache? A: If a product is missing, it won't be included in any analysis, leading to incomplete or incorrect price bucket recommendations. This usually happens if the Product Inclusion Rules are too restrictive or if the raw data feed was incomplete for that product.

Q: How does the Data Cache support decisions about OW (Optimum Width) for price buckets? A: By providing all the necessary master data like style attributes, categories, and AG (Attribute Group) definitions, along with performance metrics like ROS (which are computed from data the cache provides), it allows the system to analyze which product categories and styles should be offered at specific price points to maximize overall sales.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Centralizes and optimizes all essential business data for rapid access by the priceBucketCreation process.
- **Main Input:** Raw master data (product, store) and transactional data (sales, inventory).
- **Main Output:** Instantly accessible, in-memory master and transactional data structures.
- **Warning Signs:** Stale or incomplete data, very slow loading times, or missing product details in analysis.
- **Key Settings:** Product Inclusion Rules, Store Inclusion Rules, Transactional Data Horizon.

Need Help? Contact your Merchandising Systems Support team for questions about this process.

Chapter 3: Price Bucket Data Structures

Last Updated: 2025-05-23

1. Business Objective

This chapter defines the fundamental data models used to represent price segments and their configuration parameters, capturing key attributes essential for inventory planning (OD, OW) and pricing strategies.

Why This Matters: It provides a clear, standardized way to understand and analyze products within specific price ranges. This clarity allows us to make smarter decisions about how many unique items (OD) and how many different styles (OW) to stock in each price segment, ensuring we attract customers with the right products at competitive prices, ultimately boosting sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Category Revenue Contribution:** Helps understand the total sales revenue generated by a specific product category, which is crucial for allocating appropriate resources and setting pricing strategies.
- **Revenue Contribution within Price Segment:** Shows how much revenue is generated by products within a specific price segment, allowing us to identify the most lucrative price points.
- **Price Point Revenue Map:** Provides a detailed breakdown of revenue generated at individual maximum retail price (MRP) levels, enabling granular analysis of customer spending habits across various prices.

Product Data

- **Product Category:** Essential for grouping products into logical collections (e.g., “Women’s Dresses,” “Men’s Shirts”) to analyze price segments within meaningful groups.
- **Minimum and Maximum Retail Price (MRP):** Defines the price range for each product within a price segment, establishing the boundaries of the segment.

- **Price Segment Size:** Indicates the monetary width of a defined price segment (e.g., \$20-\$50 is a \$30 segment), helping to balance granularity with practical application.

Operational Data

- **Number of Price Segments per Category:** Specifies how many distinct price ranges should be created for each product category, influencing the granularity of price analysis.
- **Price Granularity Tiers:** Configuration parameters that define the step size or range for how price segments are constructed at different stages, ensuring the buckets are relevant and useful for analysis.
- **Sales Analysis Period:** Defines the start and end dates for the historical sales data used to determine revenue contributions, ensuring the analysis is relevant to current market conditions.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---------------------------------------|---------------|--|---|
| Number of Price Segments per Category | 5 | Determines how many distinct price ranges the system will attempt to create within each product category. | If categories have very wide price distributions requiring more granularity, or if they are very narrow, needing fewer buckets. |
| Stage 1 Price Segment Range | \$20 | Defines the initial monetary range for bucketing products based on their Maximum Retail Price (MRP). | When price points are very close together, or if the business wants broader initial segments. |
| Stage 2 Price Segment Range | \$10 | Refines the monetary range for price segments, used in a subsequent stage of bucketing to create more specific groups. | To achieve finer detail in price segmentation, especially for products with concentrated price points. |
| Stage 3 Price Segment Range | \$5 | Provides the highest level of granularity for price segment definition, often | For highly competitive categories where small price differences |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|--|
| | | used for very precise analysis or micro-segmentation. | significantly impact sales and demand. |

Note: See More Details for additional settings related to the overall sales analysis period.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Data Cache & Data View Insights] --> B[Define Price Segment Data Structures]

C[Price Bucket Creation Arguments] --> B

B --> D[Structured Price Segment Data]

1. Start with → Insights from Data View (like ROS and revenue by product) and the rich product data from the Data Cache.
2. Then → **Define** the fundamental PriceBucketData structures, outlining what information each price segment will hold.
3. Next → **Incorporate** PriceBucketCreationArgs parameters, which guide how these structures are configured (e.g., how many buckets, their initial range).
4. Finally → **Output** a set of clear, organized PriceBucketData records, each representing a distinct price segment with its relevant attributes, ready for further analysis.

5. Step-by-Step Actions

1. **Establish** the Price Segment Data Blueprint
 - What we do: *We define the essential fields for each price segment, such as its associated product category, the minimum and maximum retail prices, and its total revenue contribution.*
 - Why we do it: *This creates a consistent template for all price segments, ensuring that critical information is always captured for meaningful analysis.*
2. **Integrate** Configuration Parameters
 - What we do: *We use the PriceBucketCreationArgs (like the desired “Number of Price Segments per Category” and “Price Granularity Tiers”) to guide the initial shaping of these data structures.*
 - Why we do it: *These settings allow the business to customize how price segments are formed, ensuring they align with specific merchandising strategies and analysis needs.*
3. **Populate** with Initial Category Revenue
 - What we do: *For each defined product category, we include its total revenue contribution within the price segment data, providing a high-level view of its sales importance.*
 - Why we do it: *This initial population helps set the context for the price segments, showing their overall value within the larger product category.*

4. **Prepare** for Detailed Price Point Revenue Mapping

- What we do: *We create a structure within each price segment data model (the “Price Point Revenue Map”) to later hold detailed revenue information for every individual price point within that segment.*
- Why we do it: *This detailed mapping allows for a very granular analysis of how revenue is distributed across various price points, crucial for fine-tuning pricing and assortment strategies.*

6. Sample Scenario

Let's walk through a real example for a popular category like “Women's Dresses.”

Starting Point

- The “Women's Dresses” category has a total revenue contribution of \$500,000.
- The business wants to define Number of Price Segments per Category as 3.
- The Stage 1 Price Segment Range is set to \$50.

What Happens

1. First, the system **establishes** the PriceBucketData blueprint. It knows it needs to create 3 segments for “Women's Dresses” because of the Number of Price Segments per Category setting.
2. Then, it **initializes** three distinct PriceBucketData structures. Each structure is marked for the “Women's Dresses” category and has a placeholder for its total category revenue contribution (\$500,000).
3. Next, based on the Stage 1 Price Segment Range of \$50, the system conceptually **prepares** to define the minMrp and maxMrp for each of these 3 segments (e.g., \$0-50, \$51-100, \$101-150, etc., depending on the actual product price range). It also creates the empty Price Point Revenue Map within each structure, ready to capture detailed sales data for each price point.

Final Outcome

Three structured PriceBucketData records are ready for the “Women's Dresses” category, each representing a distinct price segment (e.g., “Value,” “Mid-Range,” “Premium”). These records contain placeholders for their specific MRP ranges and a detailed map to later fill with revenue per price point, providing a clear framework for further analysis and optimization of OD (Optimum Depth) and OW (Optimum Width) within each segment.

Key Takeaway: By providing a structured and configurable framework for price segments, this step ensures that subsequent analysis for OD and OW is organized and tailored to specific business needs, leading to more strategic pricing and assortment decisions.

7. Results

This step produces:

Main Outputs

- **Structured Price Segment Data (PriceBucketData objects):** These are the fundamental data structures representing each identified price segment, including its category, revenue contributions, and defined MRP range. This is the core output ready for analysis.
- **Configured Price Segment Parameters (PriceBucketCreationArgs):** The set of business-defined settings that guided the creation of these price segments, ensuring traceability and consistency in analysis.

Business Reports

- **Price Segment Blueprint Report:** Used by merchandising and planning teams to visualize the defined price ranges for each category before detailed analysis, ensuring the segmentation logic aligns with business goals.
- **Pricing Configuration Audit:** Used by system administrators and analysts to confirm that the parameters used for price segment creation are correctly applied and consistent across different analyses.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed | The PriceBucketData structures will be initialized with incomplete or stale revenue information, leading to inaccurate segment definitions. | Verify Data Cache and Data View feeds, contact IT support, and pause downstream processes until data integrity is confirmed. |
| Number of Price Segments per Category is set too high | The system might create an excessive number of very narrow price segments, making analysis unwieldy and potentially showing insufficient data per segment. | Review the setting with merchandising, consider if a category truly needs that much granularity, and adjust the number downwards if segments are too sparse. |
| Price Granularity Tiers are too large | The defined price segments might be too broad, failing to capture meaningful differences in customer purchasing behavior across price points. | Adjust the “Price Granularity Tiers” settings (e.g., Stage 1, 2, 3 ranges) to create finer, more insightful price segments. |

| Situation | What Happens | What You Should Do |
|--|---|---|
| All products in a category have the same MRP | The system may struggle to create distinct price segments as all products fall into a single price point, potentially leading to a single, wide “bucket.” | Re-evaluate if price bucketing is necessary for such categories or if they should be managed differently (e.g., as core items). |

9. Next Steps

Before This Step

- **Data Cache:** We need a complete and optimized set of master product data (categories, SKUs, styles) and raw transactional sales data.
- **Data View Calculation:** We need calculated insights like category-level revenue contributions and product-specific sales performance (ROS) derived from the raw data.

After This Step

- **Price Bucket Calculation:** The structured PriceBucketData records will be used as the foundation to actually group products into these defined price segments based on their individual performance and price.
- **Assortment Optimization:** The insights gained from analyzing product performance within these structured price segments will directly inform decisions on Optimum Depth (OD) and Optimum Width (OW) for future assortments.

10. Common Questions

Q: Why are data structures important for price bucketing? A: They provide the essential framework to organize and make sense of vast amounts of product and sales data. Without clear structures, it would be impossible to systematically categorize products by price, analyze their performance, and then make strategic decisions about what to stock.

Q: How do these data structures help define ideal price ranges? A: By providing specific fields for things like minimum and maximum retail price, and mapping revenue contributions, they allow the system to clearly define the boundaries of each price segment and understand how much revenue is generated within those bounds. This helps identify the most effective price points.

Q: How do the configuration parameters like “Number of Price Segments per Category” affect the data structures? A: These parameters directly instruct the system on how to shape the data structures. For example, if you set “Number of Price Segments per Category” to 3, the system knows to prepare exactly three PriceBucketData structures for that category, each designed to capture distinct price range information.

Q: What is the “Price Point Revenue Map” for, and why is it important? A: The “Price Point Revenue Map” is a detailed breakdown within each price segment that shows exactly how much revenue comes from products at individual price points (e.g., \$45, \$48, \$50). It’s crucial because it allows us to see customer spending patterns at a granular level, helping us fine-tune pricing strategies for maximum impact.

Q: How do these structured price segments support decisions about OD (Optimum Depth) and OW (Optimum Width)? A: By having products clearly defined within specific price segments, we can analyze performance (like ROS) *within* each segment. This allows us to determine the OD (ideal number of unique items) for products within a particular price range and the OW (ideal number of categories/styles) to offer at different price levels, ensuring a balanced and profitable assortment.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Defines the foundational data models and configuration parameters for representing price segments.
- **Main Input:** Product master data, category-level revenue, and price bucketing configuration settings.
- **Main Output:** Structured PriceBucketData records, ready for detailed analysis and product assignment.
- **Warning Signs:** Misaligned price ranges, too many or too few segments, or incomplete revenue data within segments.
- **Key Settings:** Number of Price Segments per Category, Stage 1/2/3 Price Segment Range.

Need Help? Contact your Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: Price Bucket Calculation

Last Updated: 2025-05-23

1. Business Objective

This module's core function is to analyze how quickly products sell (ROS) across different price points, using their Maximum Retail Price (MRP), to define the most effective price ranges for each product category. It dynamically creates and refines these price ranges based on how much revenue they contribute, aiming to group products into meaningful price segments.

Why This Matters: This is crucial for defining effective OW strategies, ensuring we offer the ideal number of different categories or styles at the right price points. It also helps manage inventory efficiently by identifying which price ranges are most profitable and demand more focus, ultimately boosting sales and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Helps determine product popularity and demand, indicating how many items were sold, which informs revenue contribution per price point.
- **Product Sales Transactions:** Provides detailed records of individual sales events, including the selling price, essential for mapping revenue to specific MRPs.
- **Total Category Revenue:** The overall sales revenue generated by a product category, used as a benchmark for calculating the revenue contribution of each price bucket.

Product Data

- **SKU Details:** Information about specific items including their MRP, necessary to assign products to the correct price buckets.
- **Style Details:** Broader product grouping information such as category, essential for bucketing prices within relevant product lines.
- **Product Attribute Group (AG) Details:** A collection of product characteristics grouped for deeper analysis, which can influence how price buckets are structured for specific assortments.

Operational Data

- **Analysis Period Start/End Date:** Defines the timeframe for sales data analysis, ensuring price buckets are based on current and relevant market performance.
- **Number of Price Segments per Category:** Specifies the desired number of distinct price ranges to create for each product category, influencing the granularity of price analysis.
- **Price Granularity Tiers:** Configuration parameters that define the monetary step size for how price segments are constructed at different stages (e.g., \$20, \$10, \$5), ensuring buckets are relevant and useful.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------|--|---|
| Number of Price Segments per Category | 5 | Determines how many distinct price ranges the system will aim to create within each product category. | If categories have very wide price distributions requiring more granularity, or if they are very narrow, needing fewer buckets. |
| Minimum Revenue Contribution for Bucket | 10% | The minimum percentage of a category's total revenue that a single price bucket should ideally contribute. If a bucket falls below this, it will be merged with an adjacent one. | To ensure that price buckets are substantial enough for meaningful analysis and merchandising decisions. |
| Stage 1 Price Segment Range | \$20 | Defines the initial monetary range for bucketing products based on their MRP. | When price points are very close together, or if the business wants broader initial segments. |
| Stage 2 Price Segment Range | \$10 | Refines the monetary range for price segments, used in a subsequent stage of bucketing to create more specific groups. | To achieve finer detail in price segmentation, especially for products with concentrated price points. |

Note: See More Details for additional settings related to the overall sales analysis period and Stage 3 Price Segment Range.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Data View & Data Cache Insights] --> B[Calculate Initial Price Buckets by Revenue]

B --> C[Refine & Merge Price Buckets]

C --> D[Final Price Buckets]

1. Start with → Product performance data (ROS, revenue by SKU/style) from Data View and comprehensive product master data (SKU, style, category, MRP) from Data Cache.
2. Then → **Calculate** initial price segments for each category by analyzing revenue contribution at different MRPs. The system groups products into broad price ranges based on configured granularity tiers.
3. Next → **Refine** and **Merge** these initial price segments. Smaller buckets that don't meet a minimum revenue contribution are combined with adjacent ones to create more meaningful and stable price ranges.
4. Finally → **Output** a set of optimized price buckets for each product category, ready to inform OW (Optimum Width) strategies and inventory planning.

5. Step-by-Step Actions

1. **Analyze** category sales performance
 - What we do: *We gather all sales data for each product category and determine the total revenue generated by each category, as well as the revenue generated by individual price points within that category.*
 - Why we do it: *This establishes the overall financial landscape for each category, which is the foundation for creating relevant price buckets.*
2. **Generate** initial price segments
 - What we do: *We use predefined Price Granularity Tiers (e.g., \$20, \$10, \$5 steps) to create preliminary price segments. For instance, all products between \$20 and \$39.99 might form one segment.*
 - Why we do it: *This systematically divides the entire price range of a category into initial, manageable chunks, ensuring we consider all price points where products are sold.*
3. **Evaluate** revenue contribution of segments
 - What we do: *For each preliminary price segment, we calculate its percentage contribution to the category's total revenue. If a segment's contribution is too high (meaning it's too broad or has too much revenue), we further divide it using smaller granularity tiers.*
 - Why we do it: *This dynamic adjustment ensures that price buckets are meaningfully sized, preventing a single bucket from dominating and allowing for a more granular understanding of customer spending habits.*
4. **Consolidate** less significant segments
 - What we do: *We merge adjacent price segments that individually fall below the Minimum Revenue Contribution for Bucket threshold. This*

ensures that each final price bucket represents a significant enough portion of sales to be actionable.

- *Why we do it: Merging smaller, less significant buckets creates more robust and actionable price ranges, simplifying analysis and preventing fragmentation of inventory and merchandising efforts.*

5. **Adjust** bucket boundaries

- *What we do: We ensure there are no gaps or overlaps between the final price bucket ranges, and that the first and last buckets perfectly align with the actual minimum and maximum MRPs of the products in the category.*
- *Why we do it: This ensures complete coverage of a category's price range and logical transitions between price segments, making them clear and easy to understand for merchandising teams.*

6. Sample Scenario

Let's walk through a real example for a popular category like "Men's T-Shirts."

Starting Point

- The "Men's T-Shirts" category has a total revenue of \$100,000.
- The business wants Number of Price Segments per Category to be 5, implying an average of 20% revenue per bucket.
- Minimum Revenue Contribution for Bucket is set to 10%.
- Stage 1 Price Segment Range is \$20, Stage 2 is \$10.
- Products in the category range from \$15 to \$95 MRP.

What Happens

1. First, the system **analyzes** all sales transactions for Men's T-Shirts, noting total category revenue (\$100,000) and how much revenue each specific MRP (e.g., \$15, \$25, \$30, \$45, \$60, \$75, \$90) contributes.
2. Next, it **generates** initial segments using the \$20 Stage 1 Price Segment Range. It might create buckets like \$15-\$34 (revenue \$40,000, 40% contribution), \$35-\$54 (revenue \$30,000, 30% contribution), \$55-\$74 (revenue \$25,000, 25% contribution), and \$75-\$95 (revenue \$5,000, 5% contribution).
3. Then, it **evaluates** these segments. The \$15-\$34 bucket has 40% revenue contribution, which is more than the implied 20% target. So, it might further **divide** this bucket using the \$10 Stage 2 Price Segment Range, potentially creating \$15-\$24 (revenue \$15,000, 15% contribution) and \$25-\$34 (revenue \$25,000, 25% contribution). The \$75-\$95 bucket has only 5% revenue, which is below the 10% Minimum Revenue Contribution for Bucket.
4. Finally, it **consolidates** the \$75-\$95 bucket. Since its revenue contribution (5%) is below the 10% minimum, it's merged with the adjacent \$55-\$74 bucket, creating a new bucket like \$55-\$95 (revenue \$30,000, 30% contribution). The system then **adjusts** the boundaries of all final buckets to be contiguous.

Final Outcome

The system delivers a refined set of price buckets for "Men's T-Shirts," such as: -

Value: \$15-\$24 (15% of category revenue) - **Mid-Range 1:** \$25-\$34 (25% of category revenue) - **Mid-Range 2:** \$35-\$54 (30% of category revenue) - **Premium:**

\$55-\$95 (30% of category revenue) These buckets clearly segment the category by price and revenue, informing decisions about how many different styles (OW) to offer in each price range.

Key Takeaway: By dynamically adjusting price segment boundaries based on revenue contribution and minimum thresholds, this module creates practical and actionable price buckets that align with real customer spending patterns.

7. Results

This step produces:

Main Outputs

- **Optimized Price Buckets:** A list of defined price ranges for each product category, specifying their minimum and maximum MRPs and their overall revenue contribution. This forms the basis for pricing strategies.
- **Price Point Revenue Map:** Detailed breakdown within each final price bucket showing the specific revenue generated at each individual MRP, allowing for granular analysis of sales performance.

Business Reports

- **Category Price Ladder Report:** Used by merchandising and planning teams to visualize the defined price buckets within each category, understand where revenue is concentrated, and plan product assortments (OW, OD).
- **Pricing Strategy Playbook Input:** Provides actionable insights for pricing managers to fine-tune product pricing within specific buckets to maximize sales and profitability.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|--|
| Missing or delayed data feed | The system will use stale or incomplete sales and product data, leading to inaccurate or irrelevant price bucket definitions. | Verify Data Cache and Data View feeds, contact IT support, and pause downstream processes until data integrity is confirmed. |
| Number of Price Segments per Category is set too high | The system might create an excessive number of very narrow price segments, many with low revenue contributions, making analysis unwieldy. | Review the setting with merchandising, consider if a category truly needs that much granularity, and adjust the number downwards if segments are too sparse. |
| Minimum Revenue Contribution | The system might aggressively merge buckets, leading to very | Re-evaluate the threshold; if buckets are too wide, reduce the |

| Situation | What Happens | What You Should Do |
|---|---|---|
| for Bucket is set too high | broad price ranges that don't provide enough granular insight for detailed merchandising. | minimum percentage to allow for more distinct segmentation. |
| All products in a category have very similar MRPs | The system may struggle to create distinct price segments, potentially resulting in just one or two very wide buckets that offer little actionable insight. | Re-evaluate if detailed price bucketing is necessary for such categories or if they should be managed differently (e.g., as a single core price point). |
| Drastic shifts in customer spending patterns | Price buckets defined by historical data might no longer accurately reflect current demand, leading to misaligned pricing strategies. | Re-run the price bucket calculation more frequently during periods of market volatility or significant promotional activity. |

9. Next Steps

Before This Step

- **Data Cache:** We need a complete and optimized set of master product data (categories, SKUs, styles, MRPs) and raw transactional sales data, ready for rapid access.
- **Data View Calculation:** We need calculated insights like category-level revenue contributions and product-specific sales performance (ROS) derived from the raw data.
- **Price Bucket Data Structures:** The foundational PriceBucketData blueprint and configuration parameters need to be defined, providing the framework for this calculation.

After This Step

- **Assortment Optimization:** The defined price buckets directly inform decisions on Optimum Depth (OD) (how many unique items) and Optimum Width (OW) (how many different styles/categories) to offer within each profitable price range.
- **Promotional Planning:** Insights from price buckets help identify effective price points for promotions and markdowns, ensuring discounts target the right products to maximize sales and clear inventory.
- **New Product Development:** Understanding successful price buckets can guide product development teams in designing new products that fit into profitable and undersupplied price ranges.

10. Common Questions

Q: How does defining price buckets help us sell more? A: By identifying the exact price ranges where customers spend the most, we can focus our inventory investment (OD) and breadth of offerings (OW) on those profitable areas. This ensures we have the right products at the prices customers are willing to pay, leading to higher sales and happier customers.

Q: What happens if a price bucket doesn't meet the Minimum Revenue Contribution? A: If a bucket contributes too little revenue, the system merges it with an adjacent, more significant price bucket. This creates more robust and actionable segments, preventing us from over-analyzing or stocking products in price ranges that don't generate substantial sales.

Q: How does the system determine the ideal size for a price bucket? A: The system dynamically adjusts the size of a price bucket based on its revenue contribution and predefined Price Granularity Tiers (e.g., \$20, \$10, \$5 steps). If a bucket is too broad or too narrow based on sales performance and business settings, it will be further divided or merged until it represents a meaningful segment of customer spending.

Q: Can these price buckets adapt to seasonal changes or new trends? A: Yes, because the module recalculates price buckets using the latest sales data over a defined Analysis Period. If customer spending shifts due to seasons or new trends, the system will adjust the price bucket boundaries and their revenue contributions accordingly, ensuring they remain relevant.

Q: How do these price buckets help with inventory management? A: By clearly defining profitable price ranges, we can better allocate inventory. For example, if the "Mid-Range" bucket is consistently our highest revenue contributor, we know to prioritize stocking a wider variety (OW) and deeper quantity (OD) of products within that specific price range, minimizing stockouts on popular items and reducing excess inventory in less popular price points.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Analyzes product sales across MRPs to define optimized, revenue-driven price ranges for each category.
- **Main Input:** Product performance (ROS, revenue) and master data (SKU, style, category, MRP) from Data View and Data Cache.
- **Main Output:** Optimized price buckets with defined MRP ranges and revenue contributions for each category.
- **Warning Signs:** Price buckets that don't make business sense, too many or too few segments, or incomplete revenue data.
- **Key Settings:** Number of Price Segments per Category, Minimum Revenue Contribution for Bucket, Price Granularity Tiers (Stage 1, 2, 3).

Need Help? Contact your Merchandising Systems Support team for questions about this process.

Chapter 7: Price Bucket Creation Orchestration

Last Updated: 2025-05-23

1. Business Objective

This module orchestrates the entire price bucket creation workflow, ensuring that all necessary steps, from data validation to final output delivery, are executed in the correct sequence to define optimal price segments for product categories.

Why This Matters: It guarantees that our strategic pricing and inventory decisions (like defining Optimum Width (OW) and Optimum Depth (OD)) are based on accurate, validated, and up-to-date price segments. This structured approach minimizes errors, speeds up decision-making, and ensures we consistently offer the right products at competitive prices, ultimately boosting sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Calculated Rate of Sale (ROS):** Derived from product sales data, it shows how quickly products are selling, essential for evaluating product performance within potential price segments.
- **Total Category Revenue:** The aggregate sales revenue for a product category, critical for understanding the financial importance of each price segment within that category.

Product Data

- **SKU Details:** Comprehensive information about each unique item, including its Maximum Retail Price (MRP), allowing it to be accurately assigned to price segments.
- **Style Details:** Broader product grouping information, such as category and Attribute Group (AG) definitions, which determines the scope for price bucketing.

Operational Data

- **Analysis Period Start/End Date:** Defines the timeframe for sales data analysis, ensuring price buckets are based on current and relevant market performance.
- **Number of Price Segments per Category:** A business setting specifying how many distinct price ranges should be created for each product category, influencing the granularity of price analysis.
- **Minimum Revenue Contribution for Bucket:** A threshold setting that ensures each created price bucket is significant enough in terms of sales to be actionable for merchandising.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|-----------------------------------|--|---|
| Number of Price Segments per Category | 5 | Determines how many distinct price ranges the system will aim to create within each product category, influencing the granularity of price analysis. | If categories have very wide price distributions requiring more granularity, or if they are very narrow, needing fewer buckets. |
| Minimum Revenue Contribution for Bucket | 10% | The minimum percentage of a category's total revenue that a single price bucket should ideally contribute before being merged. | To ensure that price buckets are substantial enough for meaningful analysis and merchandising decisions. |
| Analysis Period | (Dynamic, typically last 90 days) | The time window for loading historical sales and inventory transactions, used to calculate ROS and product availability. | To analyze performance over different seasons, during promotions, or under specific market conditions. |
| Data Refresh Frequency | Daily | How often the underlying data cache is fully updated with the latest master and | If business decisions require more frequent updates (e.g., during high-traffic sales events), or if data latency is acceptable. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---------------------|----------------|
| | | transactional data. | |

Note: See More Details for additional settings related to specific price granularity tiers and product inclusion rules, which are managed by the modules orchestrated here.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw & Processed Data (from Data Cache & Data View)] -->
    B{Validate Inputs & Configurations}
    B --> C[Snapshot Current Output]
    C --> D[Calculate Price Buckets]
    D --> E[Sync Final Output]
```

1. Start with → All raw and processed business data (sales, product, inventory) from the Data Cache and derived insights (ROS, revenue contributions) from the Data View.
2. Then → **Validate** that all incoming data is consistent and that business configurations (like the Number of Price Segments per Category) are correctly set.
3. Next → **Capture** a historical snapshot of the currently defined price buckets and related outputs, preserving them before new calculations begin.
4. Then → **Calculate** the new, optimized price buckets for each product category using the validated data and business settings.
5. Finally → **Sync** the newly calculated price buckets and associated insights to all downstream systems that rely on this information for merchandising, planning, and reporting.

5. Step-by-Step Actions

1. **Validate** input data and configurations
 - What we do: *The orchestration module triggers a validation check on the data and settings received from upstream processes (like Data Cache and Data View), ensuring their accuracy and consistency.*
 - Why we do it: *This critical first step prevents flawed data or incorrect business rules from leading to inaccurate price buckets, saving time and avoiding poor merchandising decisions.*
2. **Capture** a snapshot of existing outputs
 - What we do: *Before any new price bucket calculations commence, the system automatically saves a copy of the current price bucket definitions and related outputs.*
 - Why we do it: *This creates a historical record, allowing us to track changes over time, audit past decisions, and revert to previous states if necessary, providing a crucial safety net.*

3. **Calculate** new price buckets

- What we do: *The orchestration module initiates the core Price Bucket Calculation process, which analyzes product performance and revenue contributions across various price points to define optimal price segments for each category.*
- Why we do it: *This is the heart of the process, dynamically creating intelligent price ranges that align with customer spending patterns and business objectives, directly impacting our OW and OD strategies.*

4. **Sync** calculated price buckets to downstream systems

- What we do: *Upon successful calculation, the orchestration module ensures that the newly defined price buckets are automatically transmitted and made available to all other systems that rely on this information.*
- Why we do it: *This ensures that merchandising, planning, and other teams are working with the most current and accurate price segment data, enabling timely and informed decisions across the business.*

6. Sample Scenario

Let's walk through a real example of a new pricing cycle for the "Women's Dresses" category.

Starting Point

- It's the beginning of a new quarter, and the business wants to redefine price buckets for the "Women's Dresses" category based on the last 90 days of sales data.
- Current price buckets for dresses are set as "Value (\$20-\$50)", "Mid-Range (\$51-\$100)", and "Premium (\$101-\$200+)"
- The Minimum Revenue Contribution for Bucket is 15%, and the Number of Price Segments per Category is 4.

What Happens

1. First, the orchestration module **triggers** the validation step. It checks if the underlying sales data for "Women's Dresses" is complete and consistent for the past 90 days, and confirms that the business settings for price bucketing are correctly applied. If any issues are found, the process pauses with an alert.
2. Assuming validation passes, the system then **captures** a snapshot of the existing "Women's Dresses" price buckets. This old structure (Value, Mid-Range, Premium) is saved as a historical record.
3. Next, the orchestration module **initiates** the core Price Bucket Calculation. This process takes the latest sales data, identifies new revenue concentration points for "Women's Dresses," and, aiming for 4 segments, might determine that a new "Affordable Premium" segment is emerging, or that the "Value" segment needs to be split.
4. Finally, after the new price buckets are successfully calculated (e.g., "Budget (\$20-\$40)", "Core (\$41-\$80)", "Advanced (\$81-\$130)", "Luxury (\$131-\$200+)"), the orchestration module **syncs** these new definitions across all relevant systems, updating dashboards for merchandising and planning.

Final Outcome

Merchandising and planning teams now have validated, updated, and historically-backed price buckets for “Women’s Dresses” that reflect the latest customer spending patterns. This allows them to adjust their Optimum Width (OW) (e.g., offer more styles in the “Core” segment) and Optimum Depth (OD) (e.g., stock more items in popular sizes within the “Luxury” segment) for the upcoming quarter with confidence.

Key Takeaway: This orchestration ensures that the crucial process of defining price buckets is robust, reliable, and provides up-to-date insights for strategic inventory and pricing decisions.

7. Results

This step produces:

Main Outputs

- **Validated Configurations:** Confirmation that all business settings and incoming data used for price bucket creation are accurate and consistent.
- **Optimized Price Buckets:** The definitive list of new price ranges for each product category, including their MRP boundaries and revenue contributions, ready for use.
- **Historical Price Bucket Snapshots:** A preserved record of previous price bucket definitions, allowing for audit trails and comparison over time.
- **Synced Price Bucket Data:** The newly calculated price buckets are made available and consistent across all integrated retail systems.

Business Reports

- **Price Bucket Workflow Status Report:** Used by data operations and merchandising teams to monitor the successful execution and completion of the price bucket creation process.
- **Price Bucket Audit Trail:** Used by analysts and compliance teams to review historical changes in price bucket definitions and understand the rationale behind them.
- **System Performance Metrics:** Used by IT to evaluate the efficiency and speed of the entire orchestration process, ensuring timely delivery of insights.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The orchestration module will either halt or proceed with stale data, leading to outdated or inaccurate | Verify data sources from Data Cache and Data View, contact IT support immediately, and pause downstream processes until data integrity is confirmed. |

| Situation | What Happens | What You Should Do |
|-------------------------------|---|---|
| Validation check fails | price bucket definitions.

The process will stop before calculating new price buckets and flag the specific data inconsistencies or incorrect settings. | Review the error logs to identify the root cause (e.g., corrupted data, misconfigured business settings) and correct them in the relevant upstream modules. |
| Core calculation module fails | The Price Bucket Calculation step might encounter an error, resulting in no new price buckets being generated or an incomplete set. | Investigate the logs of the Price Bucket Calculation module for details, identify the cause (e.g., unexpected data patterns, calculation bug), and re-run once fixed. |
| Output syncing failure | The newly calculated price buckets are not successfully transmitted to all downstream systems, leading to inconsistent data across the business. | Check the <code>utilOutputSyncModule</code> logs for connection errors or data format issues, contact IT for network or system integration support, and manually push data if critical. |

9. Next Steps

Before This Step

- **Data Cache:** A complete, accurate, and optimized set of raw master product data (SKUs, styles, categories, MRPs) and transactional sales/inventory data must be loaded and ready for rapid access.
- **Data View Calculation:** Key derived insights, such as Calculated Rate of Sale (ROS) and Total Category Revenue, must be computed from the raw data and prepared for analysis.
- **Price Bucket Data Structures:** The foundational data models and configuration parameters for representing price segments must be defined, providing the blueprint for the calculation.

After This Step

- **Assortment Optimization:** The defined price buckets directly inform decisions on Optimum Depth (OD) (how many unique items) and Optimum Width (OW) (how many different styles/categories) to offer within each profitable price range.
- **Promotional Planning:** Insights from these optimized price buckets help identify effective price points for promotions and markdowns, ensuring discounts target the right products to maximize sales and clear inventory.
- **New Product Development:** Understanding successful price buckets can guide product development teams in designing new products that fit into profitable and undersupplied price ranges, aligning with market demand.

10. Common Questions

Q: Why is “Orchestration” needed for price buckets, can’t we just run the calculation? A: Orchestration ensures that all critical steps, like validating data and configurations, saving historical records, and then distributing the results, happen in the correct order every time. This robust workflow prevents errors, ensures data consistency across the business, and provides a clear audit trail, which is crucial for making reliable pricing and inventory decisions.

Q: How does this module prevent bad price buckets from being created? A: It starts with a vital validation step. If the underlying sales data is incomplete or the business settings are misconfigured, the orchestration will halt the process and alert the team. This prevents inaccurate price buckets from being calculated and used, safeguarding our merchandising strategies.

Q: What happens if there’s a problem during the calculation of price buckets? A: The orchestration module is designed to identify and report such failures. If the core price bucket calculation encounters an issue, the orchestration will stop the entire process, preventing incomplete or erroneous results from being published. This allows teams to investigate and resolve the problem before re-running the workflow.

Q: How does this orchestration support “Optimum Width” (OW) and “Optimum Depth” (OD)? A: By ensuring that the Price Bucket Calculation module receives validated, high-quality data and configurations, the orchestration guarantees that the resulting price buckets are accurate and actionable. This reliable output then directly informs decisions on OW (the ideal number of different categories/styles) to offer within a price segment and OD (the ideal number of unique items) to stock, optimizing our assortment strategy.

Q: Why is it important to capture a snapshot of the current price buckets? A: Capturing a snapshot creates a historical record of our price bucket definitions at a specific point in time. This is invaluable for auditing, understanding the impact of changes over time, and providing a fallback if we ever need to revert to a previous state. It adds a crucial layer of accountability and flexibility to our strategic pricing decisions.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Manages the entire priceBucketCreation workflow, ensuring validation, calculation, snapshotting, and syncing happen correctly.
 - **Main Input:** Processed sales data (ROS, revenue), detailed product data (SKU, Style, MRP), and operational settings.
 - **Main Output:** Validated, optimized price buckets that are snapshotted and synced to other systems.
 - **Warning Signs:** Workflow failures, data inconsistencies flagged during validation, or delays in output delivery.
 - **Key Settings:** Number of Price Segments per Category, Minimum Revenue Contribution for Bucket, Analysis Period, Data Refresh Frequency.
-

Need Help? Contact your Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

priceBucketCreation FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ provides answers to common questions about the priceBucketCreation algorithm, its functionality, and how it supports key business objectives like defining **OW** and **OD**.

General

Q: What is priceBucketCreation? **A:** priceBucketCreation is an algorithm that defines optimal price segments, often called “price buckets,” for different product categories. It helps identify the most effective price points for our merchandise, directly supporting strategic pricing and inventory decisions, particularly for determining **OW** (Optimum Width) and **OD** (Optimum Depth).

Q: Why are price buckets important for our business? **A:** Price buckets are crucial because they allow us to segment products into meaningful price ranges based on their sales performance and revenue contribution. This enables us to optimize pricing, manage inventory more efficiently, and make informed decisions about the ideal number of different categories or styles (**OW**) and unique items (**OD**) we should stock within those categories.

Q: How does priceBucketCreation impact our product strategy? **A:** By identifying optimal price points, the system informs our merchandising strategies. It helps ensure we offer the right products at the right prices to maximize revenue and minimize stockouts, directly guiding decisions related to **OW** (product variety) and **OD** (SKU depth).

Q: What is the main goal of this algorithm? **A:** The primary goal is to analyze product **ROS** (Rate of Sale) across various prices to define the most effective price segments for each product category, thereby supporting strategic decisions for **OW** and efficient inventory management.

Q: Who uses the output of priceBucketCreation? **A:** Merchandising, pricing, and inventory planning teams primarily use the output to make data-driven decisions about product assortment, pricing strategies, and stock levels to achieve optimal **OW** and **OD**.

Configuration & Settings

Q: What kind of configurations are needed for priceBucketCreation? **A:** The system requires critical input parameters and master data, such as predefined bucket ranges or rules for how price ranges should be logically ordered. These configurations are validated upfront to ensure accurate calculations and reliable **OW** price segments.

Q: How do we ensure the input configurations are correct? **A:** The Price Bucket Configuration Validation module automatically checks input parameters and master data to ensure they are valid and logically ordered, preventing errors in the calculation of **OW** price segments.

Q: Can we adjust how price buckets are defined? **A:** While specific settings are managed internally by the algorithm, the core logic considers revenue contribution and **ROS**. Any adjustments to parameters influencing these calculations would typically be handled by the development team and require specific configuration updates.

Q: What happens if the configurations are invalid? **A:** If configurations are invalid, the Price Bucket Configuration Validation module will flag the issues and prevent the calculation process from proceeding, thus safeguarding the integrity of the generated **OW** price segments.

Q: Are there any default configurations if we don't provide custom ones? **A:** The system is designed to operate with a robust set of parameters. While it can run with defaults, optimizing for specific business needs may require custom configurations. Refer to the project's technical documentation for details.

Data & Inputs

Q: What data does priceBucketCreation use? **A:** It uses essential business data like product details (SKU, Style), sales history (transactional data like ProductSales), and inventory information. This data is critical for calculating **ROS** and revenue contributions.

Q: Where does the algorithm get its data from? **A:** All necessary data is loaded and accessed efficiently from a Data Cache, which acts as a central repository for master data (SKU, Style, Store) and transactional sales data.

Q: How is **ROS** used in this process? **A:** **ROS** (Rate of Sale) is a core metric. The algorithm analyzes product **ROS** across different MRPs (Manufacturer's Recommended Price) to understand how quickly products sell, which is fundamental to defining effective price buckets and informing **OW** strategies.

Q: Does it consider current inventory levels? **A:** While primarily focused on sales history, the Data Cache holds inventory data. The system can leverage live status insights from the Data View to make informed decisions about product segmentation.

Q: What kind of product information is needed? **A:** Detailed SKU and Style information, including their respective MRPs, is crucial for segmenting products and linking them to their performance within defined price buckets.

Process & Logic

Q: How does priceBucketCreation work step-by-step? **A:** The Price Bucket Creation Orchestration module manages the workflow: It starts with validating all input parameters. Then, the Price Bucket Calculation module analyzes product **ROS** across different MRPs to define optimal price buckets based on revenue contribution. Finally, the output is snapshotted and synced for use, defining optimal price segments that impact **OW** and **OD**.

Q: How does it determine the “optimal” price buckets? **A:** The Price Bucket Calculation module dynamically creates and refines price ranges by analyzing product **ROS** and revenue contribution. It aims to segment products into meaningful price points that maximize sales and align with effective **OW** strategies.

Q: Is the process automatic or does it require manual intervention? **A:** The Price Bucket Creation Orchestration module automates the entire workflow, from validation to calculation and output, minimizing manual intervention.

Q: How are the computed price buckets integrated with actual product data? **A:** The Price Bucket Calculation Helper module assists in processing price bucket data and integrating it with master product information (SKU, Style). It computes and adjusts the actual min and max MRPs for each price bucket based on real-world product data, ensuring accuracy for **OW** analysis.

Q: What is the role of the Data View module? **A:** The Data View provides an abstraction layer for accessing and computing derived business insights from the cached data. It offers utility functions to analyze SKU/Style **ROS** and live status across stores and dates, helping modules like PriceBucketCalculation make informed decisions about product segmentation.

Outputs & Results

Q: What is the primary output of priceBucketCreation? **A:** The primary output is PriceBucketData, which defines the optimal price segments for product categories. This data captures key attributes such as category, revenue contribution, and the MRP range for each defined price segment.

Q: How are these price buckets used in business operations? **A:** These price buckets are essential for inventory planning (**OD**, **OW**), strategic pricing, and merchandising decisions. They help us understand which price points are most effective for different product categories.

Q: Can we see the revenue contribution for each price bucket? **A:** Yes, the PriceBucketData output explicitly captures the revenue contribution for each defined price segment, allowing teams to assess the financial impact of each bucket.

Q: How do these outputs help with **OW** and **OD** decisions? **A:** By clearly defining successful price segments, the output helps determine the **OW** (how many different categories/styles to offer) within each price range and informs **OD** (how many unique items/SKUs to stock) within those effective price points, ensuring we maximize sales and manage inventory efficiently.

Q: Is the output data easy to integrate with other systems? **A:** The system aims for seamless integration, with the orchestration module managing snapshotting and syncing the output, making it ready for consumption by other business systems for inventory and pricing applications.

Error Handling & Edge Cases

Q: What happens if there's an error during the process? **A:** The Price Bucket Configuration Validation module runs first to prevent errors from incorrect inputs. If other issues arise during the calculation or data syncing, the orchestration module is designed to manage the workflow, and errors would typically be logged for review and resolution.

Q: Are there any known situations where the algorithm might not perform optimally? **A:** While the algorithm is designed to be robust, performance may be less optimal for categories with very limited or inconsistent sales data, as this can make it challenging to define clear and statistically significant price buckets.

Q: How can I tell if the generated price buckets are reliable? **A:** The process includes validation steps to ensure data integrity. Additionally, reviewing the PriceBucketData output, specifically the revenue contributions and logical ordering of MRP ranges, can help assess the reliability and business sense of the generated buckets.

Q: What if a category has very few products, affecting price bucket creation? **A:** For categories with limited product data or sales history, the algorithm might rely more on broader category trends or predefined configurations. The robustness of the price buckets generally improves with sufficient product diversity and comprehensive sales data.

Q: How are unexpected data values (e.g., zero **ROS** for a product) handled? **A:** The Data View and Price Bucket Calculation modules are designed to process various data scenarios. Products with zero **ROS** would contribute less to the definition of active price buckets, effectively being filtered out or assigned to buckets based on their MRP.

Need More Help?

Please contact the support team for further assistance.

priceBucketCreation Documentation

- **Version:** 1.0
- **Last Updated:** 2025-05-23

2. Executive Summary

The priceBucketCreation algorithm is a strategic tool designed to optimize product pricing and inventory management. By analyzing product Rate of Sale (ROS) and revenue contributions, it defines optimal price segments (OW) for various product categories. This enables businesses to make data-driven decisions on stock levels (OD) and promotional strategies, ultimately maximizing sales and profitability.

3. Architecture Overview

```
graph TD
    subgraph Core_Orchestration
        A["0: Price Bucket Creation Orchestration"]
    end

    subgraph Calculation_Data_Processing
        B["1: Price Bucket Calculation"]
        C["2: Price Bucket Configuration Validation"]
        D["3: Data Cache"]
        E["4: Data View"]
        F["5: Price Bucket Data Structures"]
        G["6: Price Bucket Calculation Helper"]
    end

    A -- control_flow --> C
    A -- control_flow --> B
    B -- uses --> F
    C -- uses --> F
    D -- uses --> G
    E -- data_flow --> D
    B -- depends_on --> D
    C -- depends_on --> D
```

Module Descriptions:

- **Price Bucket Creation Orchestration:** Orchestrates the entire priceBucketCreation project, ensuring that validation runs first, followed by the core calculation of price buckets, and finally snapshotting and syncing the output. This module manages the workflow for defining the optimal price segments for product categories, which impacts strategic pricing and inventory decisions (e.g., defining OW and OD).
 - **Price Bucket Calculation:** This module's core function is to analyze product ROS across different MRPs to define optimum price buckets for each category. It dynamically creates and refines price ranges based on revenue contribution, aiming to segment products into meaningful price points. This is crucial for defining effective OW strategies and managing inventory efficiently.
 - **Price Bucket Configuration Validation:** This module validates the input parameters and master data essential for the price bucket creation process. It ensures that critical configurations like bucket ranges are valid and logically ordered to prevent erroneous calculations, thereby safeguarding the integrity of the generated OW price segments.
 - **Data Cache:** A central repository for all essential business data (product, sales, inventory) required for the priceBucketCreation process. It efficiently loads and provides in-memory access to master data (SKU, Style, Store, etc.) and transactional data (ProductSales), which are vital for calculating ROS and revenue contributions used in defining OW price buckets.
 - **Data View:** Provides an abstraction layer for accessing and computing derived business insights from the cached data. It offers utility functions to analyze SKU/Style ROS and live status across stores and dates, helping modules like PriceBucketCalculation make informed decisions about product segmentation and price range effectiveness.
 - **Price Bucket Data Structures:** Defines the fundamental data models used to represent price buckets (PriceBucketData) and their configuration parameters (PriceBucketCreationArgs). PriceBucketData captures key attributes such as category, revenue contribution, and MRP range for each defined price segment, essential for inventory planning (OD, OW) and pricing strategies.
 - **Price Bucket Calculation Helper:** A specialized utility that assists in processing price bucket data and integrating it with master product information (SKU, Style). It computes and adjusts the actual min and max MRPs for each price bucket based on the real-world product data, ensuring that the computed price segments are accurately reflected in the system for effective OW analysis.
-

4. Prerequisites & Setup

This project is built using Java. Ensure the following are installed:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Maven** (or Gradle): A build automation tool.

Build Commands:

1. **Clone the repository:** `git clone <repository-url> cd priceBucketCreation`
2. **Build the project:** `mvn clean install` (if using Maven)

Running the Orchestrator:

The Price Bucket Creation Orchestration module serves as the main entry point. To run the application:

```
java -jar target/priceBucketCreation-<version>.jar --config
args.properties
```

Ensure the args.properties file is correctly configured with all necessary parameters as detailed in the “Global Configuration” section.

5. Folder & File Structure

```
priceBucketCreation/
├─ module-group/           # Contains the orchestrator for the
entire pipeline
├─ submodules/             # Houses individual, self-contained
business logic modules
├─ args.properties         # Global configuration parameters for
the application
├─ common-data.json        # Schema definition for shared data
models
├─ outputs/                # Directory where generated price
bucket data and reports are stored
```

6. Global Configuration

The args.properties file contains global configuration parameters that govern the behavior of the priceBucketCreation algorithm.

| Setting Name | Default Value | Description | Valid Range/When to Change |
|---|---------------|-------------|----------------------------|
| <i>(No global configuration parameters were provided in the input for args.properties.)</i> | | | |

7. Shared Data Model

The `common-data.json` schema defines the structure for data exchanged and utilized across various modules in the `priceBucketCreation` project.

| Field | Type | Description |
|---|------|-------------|
| <i>(No shared data model schema was provided in the input for <code>common-data.json</code>.)</i> | | |
| | | |

8. Submodule Chapters

8.1 Price Bucket Creation Orchestration

8.1.1 Module Name & Business Objective

Module Name: Price Bucket Creation Orchestration **Business Objective:** To systematically manage and execute the entire price bucket creation workflow, ensuring data integrity, accurate calculations, and timely output for strategic pricing and inventory (OW, OD) decisions.

8.1.2 Required Information

- **Sales Data:**
 - ROS (Rate of Sale) for products/SKUs.
 - Revenue contributions by product/category.
- **Product Data:**
 - SKU and Style master data.
 - Current Market Retail Prices (MRPs).
 - Category definitions.
- **Operational Data:**
 - Configuration parameters for price bucket ranges, validation rules, and output paths.
 - System execution flags (e.g., enable validation, enable snapshotting).

8.1.3 Business Settings

- `enableValidation`: Controls whether the Price Bucket Configuration Validation module runs.
- `enableSnapshotting`: Determines if the calculated price buckets are snapshotted before syncing.
- `outputPath`: Defines where the final price bucket data will be stored.

8.1.4 Process Flow

START --> Validate Configs --> Calculate Price Buckets -->
Snapshot Output --> Sync Output --> END

8.1.5 Step-by-Step Actions

1. **Initialize** the orchestrator, *setting up the execution environment.*
2. **Trigger** the Price Bucket Configuration Validation module, *ensuring all input parameters and master data are valid and consistent before proceeding.*
3. **Execute** the Price Bucket Calculation module, *performing the core logic to define optimal price buckets based on product performance.*
4. **Snapshot** the generated output, *creating a historical record of the price buckets for auditing and version control.*
5. **Sync** the final price bucket data to the target system, *making the new price segments available for downstream applications like pricing tools and inventory planners.*
6. **Log** completion status, *providing transparency and traceability of the process.*

8.1.6 Sample Scenario

Imagine a new category, “Casual T-Shirts,” is being introduced. The orchestrator is run with the `args.properties` configured to include this category. It first **validates** that the MRP ranges for T-shirts are correctly defined. Then, it **calculates** that optimal price buckets for this category should be \$10–15 (Mass Appeal), \$16–25 (Mid-Range), and \$26–40 (Premium). This output is then **snapshotted** and **synced** to the inventory system, informing the buying team about the required OW and OD for each price segment. **Key Takeaway:** The orchestrator guarantees a structured, validated, and complete process from configuration to actionable output.

8.1.7 Results

- Validated configuration parameters.
- Computed optimal price buckets for specified categories.
- Snapshotted and synced price bucket data, ready for consumption by other business systems.

8.1.8 Unusual Cases

- **Missing or delayed data feed:** The orchestration will fail if critical sales or product data is not available, leading to incomplete or inaccurate calculations.
- **Configuration validation failure:** If the validation module flags issues (e.g., overlapping price ranges, invalid category names), the orchestration will halt, preventing erroneous data from being processed.
- **System resource exhaustion:** Large data volumes or complex calculations might lead to memory or CPU issues, requiring optimization or scaling.

8.1.9 Next Steps

- Business users apply the generated price buckets in pricing strategies.

- Inventory planners leverage the defined OW and OD for purchasing and allocation.
- Performance monitoring of sales within new price buckets.

8.1.10 Common Questions

1. How often should the price buckets be re-calculated?
2. What happens if the validation fails?
3. Can I manually override a price bucket after it's generated?
4. How does this impact existing inventory?
5. Which categories are currently supported by this process?

8.1.11 Quick Reference

- **Purpose:** End-to-end management of price bucket generation.
- **Inputs:** Configuration, sales data, product master data.
- **Outputs:** Validated price buckets.
- **Key Value:** Ensures data integrity and provides strategic pricing insights (OW, OD).

8.2 Price Bucket Calculation

8.2.1 Module Name & Business Objective

Module Name: Price Bucket Calculation **Business Objective:** To analytically determine the ideal price ranges (buckets) for products within each category, optimizing for revenue contribution and defining effective OW strategies.

8.2.2 Required Information

- **Sales Data:**
 - Historical ROS (units/week) for each SKU/Style.
 - Historical revenue generated by each SKU/Style.
- **Product Data:**
 - Current MRPs for all relevant SKUs.
 - Category assignments for each SKU/Style.
 - SKU/Style master data (e.g., brand, material, attributes for AG analysis).
- **Operational Data:**
 - Configured minimum and maximum bucket thresholds.
 - Target number of price buckets (e.g., 3-5).

8.2.3 Business Settings

- **minRevenueContributionPercentage:** Minimum percentage of total category revenue required for a price segment to be considered valid.
- **maxBucketsPerCategory:** Upper limit on the number of price buckets to generate for any single category.
- **rosThresholdForExclusion:** Products with ROS below this threshold might be excluded from consideration.

8.2.4 Process Flow

Gather Product Data --> Analyze ROS & Revenue --> Define Initial Buckets --> Refine & Consolidate --> Output Price Buckets

8.2.5 Step-by-Step Actions

1. **Retrieve** comprehensive product and sales data from the Data Cache, *ensuring access to all necessary historical performance metrics.*
2. **Group** products by category and current MRP, *preparing the data for category-specific analysis.*
3. **Calculate** each product's ROS and revenue contribution within its category, *identifying top-performing price points.*
4. **Iteratively define** initial price bucket ranges based on revenue distribution and ROS density, *aiming to capture natural clustering of product performance.*
5. **Refine** bucket boundaries and **consolidate** small or overlapping buckets, *ensuring each bucket is distinct, meaningful, and adheres to configured business rules (e.g., min revenue contribution).*
6. **Utilize** the Price Bucket Calculation Helper to adjust min/max MRPs based on actual product data, *ensuring the computed price segments are precise and reflect real-world product prices.*
7. **Generate** PriceBucketData objects, *packaging the final, optimized price ranges for each category.*

8.2.6 Sample Scenario

For the “Women’s Jeans” category, analysis shows a high ROS for items priced \$40-60 and \$80-100, while \$60-80 has lower ROS and revenue. The module calculates three optimal price buckets: \$35–59 (Value), \$60–79 (Mid-Tier) (despite lower ROS, this range might still be strategic for OW breadth), and \$80–109 (Premium). It ensures that each bucket captures sufficient revenue contribution as per configuration. **Key Takeaway:** This module transforms raw sales data into actionable pricing segments, directly informing OW strategies.

8.2.7 Results

- A set of optimal price buckets (PriceBucketData objects) for each analyzed product category.
- Each bucket includes its minimum and maximum MRP, associated revenue contribution, and ROS characteristics.

8.2.8 Unusual Cases

- **Missing or delayed data feed:** Incomplete sales or product data can lead to inaccurate or non-representative price buckets.
- **Categories with sparse data:** Categories with very few products or low sales volume might result in less meaningful or unstable price buckets.
- **No clear price segmentation:** If products within a category have a very uniform ROS and revenue distribution across all MRPs, the algorithm might struggle to define distinct buckets.

- **Configuration conflicts:** If `minRevenueContributionPercentage` is too high or `maxBucketsPerCategory` is too low, the module might be unable to generate a valid set of buckets.

8.2.9 Next Steps

- The generated price buckets are passed back to the `Orchestration` module for snapshotting and syncing.
- Business analysts review the calculated buckets for strategic alignment.

8.2.10 Common Questions

1. How are “optimal” price buckets determined?
2. What if a product falls outside an existing bucket?
3. Can we manually adjust the bucket boundaries?
4. How does this algorithm account for promotions or seasonal sales?
5. What’s the impact of old/stale data on the calculation?

8.2.11 Quick Reference

- **Purpose:** Define optimal price ranges (OW) for products.
- **Inputs:** Sales data (ROS, revenue), product master data, bucket configuration.
- **Outputs:** Structured `PriceBucketData` for each category.
- **Key Value:** Data-driven pricing and inventory segmentation.

8.3 Price Bucket Configuration Validation

8.3.1 Module Name & Business Objective

Module Name: Price Bucket Configuration Validation **Business Objective:** To ensure the integrity and logical consistency of all input parameters and master data required for price bucket creation, preventing erroneous calculations and safeguarding the accuracy of generated OW price segments.

8.3.2 Required Information

- **Product Data:**
 - Master data for categories, styles, and SKUs to ensure references are valid.
- **Operational Data:**
 - `PriceBucketCreationArgs` (from `Price Bucket Data Structures`) including configured min/max MRPs, number of buckets, and category lists.
 - System-wide configuration settings (e.g., date ranges for data extraction).

8.3.3 Business Settings

- `strictValidationMode`: A boolean flag to enable or disable strict validation rules (e.g., allowing slight overlaps for specific cases vs. requiring absolute non-overlap).

- **validCategoryList:** A predefined list of categories that the system is configured to process.

8.3.4 Process Flow

Load Configurations --> Validate Data Existence --> Validate Logic
--> Report Errors/Success

8.3.5 Step-by-Step Actions

1. **Load** all configuration parameters and relevant master data, *ensuring all necessary inputs are present for validation checks.*
2. **Check** for existence and completeness of required attributes (e.g., category IDs, minimum revenue thresholds), *identifying any missing critical information.*
3. **Validate** numerical ranges and logical ordering of price bucket parameters (e.g., $\text{minMRP} < \text{maxMRP}$, non-overlapping ranges for predefined buckets), *preventing mathematical or logical errors in subsequent calculations.*
4. **Cross-reference** configured categories against actual product master data, *ensuring that price buckets are only generated for existing and valid categories.*
5. **Report** any validation failures or warnings, *providing clear feedback to users for corrective action.*
6. **Return** a success or failure status to the orchestrator, *determining if the price bucket creation process can proceed.*

8.3.6 Sample Scenario

A user defines price buckets with a starting MRP of \$50 and an ending MRP of \$40 for a category. The Validation module immediately flags this as an invalid configuration (minMRP must be less than maxMRP). It also checks if a specified “Summer Apparel” category actually exists in the product master data. If not, it warns that no price buckets can be created for this non-existent category. **Key Takeaway:** Validation prevents the creation of incorrect or unworkable price segments, ensuring data quality for OW.

8.3.7 Results

- A boolean status indicating whether all configurations passed validation.
- A detailed report of any validation errors or warnings.

8.3.8 Unusual Cases

- **Missing or delayed data feed:** Validation may fail if master data for categories or styles is incomplete or inaccessible.
- **Conflicting configurations:** Multiple parameters set in a way that creates logical inconsistencies (e.g., a minimum revenue threshold that no possible bucket could ever meet).
- **Large, complex configuration files:** Can make error detection challenging and slow down validation.

8.3.9 Next Steps

- If validation passes, the orchestration proceeds to Price Bucket Calculation.
- If validation fails, the process stops, and errors must be addressed by the user.

8.3.10 Common Questions

1. What are the most common validation errors?
2. Can I bypass validation for testing?
3. How are new categories added to the valid list?
4. Does it validate historical data, or just the current configuration?
5. What's the difference between a warning and an error during validation?

8.3.11 Quick Reference

- **Purpose:** Ensure valid input parameters and master data.
- **Inputs:** Configuration parameters (PriceBucketCreationArgs), master product data.
- **Outputs:** Validation status (pass/fail) and error log.
- **Key Value:** Prevents bad data from entering the system, protecting OW integrity.

8.4 Data Cache

8.4.1 Module Name & Business Objective

Module Name: Data Cache **Business Objective:** To serve as a central, efficient, in-memory repository for all critical business data needed by the priceBucketCreation process, providing fast access to product, sales, and inventory data essential for ROS calculations and defining OW price buckets.

8.4.2 Required Information

- **Sales Data:**
 - Historical transaction data (SKU, quantity, price, date, store).
- **Product Data:**
 - SKU master data (e.g., product_id, category_id, style_id, MRP).
 - Style master data (e.g., style_id, description).
 - Category master data.
- **Operational Data:**
 - Database connection details.
 - Data loading schedule or triggers.

8.4.3 Business Settings

- dataRetentionPeriod: How long historical sales data is kept in the cache.
- cacheRefreshFrequency: How often the cache is updated from source systems.

- **maxCacheSizeGB:** Limits the maximum memory allocated for the cache.

8.4.4 Process Flow

Load Data from DB --> Transform/Clean --> Store in Memory --> Serve Data Requests

8.4.5 Step-by-Step Actions

1. **Connect** to primary data sources (e.g., transactional databases, data lakes), *establishing secure and efficient data retrieval channels.*
2. **Load** raw product, sales, and inventory data, *transferring relevant information into the system's memory.*
3. **Perform** initial data cleaning and transformation (e.g., data type conversion, handling nulls), *preparing the data for consistent use across modules.*
4. **Index** the loaded data for rapid lookups, *optimizing performance for queries related to SKU, Style, Store, and Date.*
5. **Utilize** the Price Bucket Calculation Helper to integrate any pre-calculated price bucket data or enrich master data, *ensuring the cache contains complete and contextualized information.*
6. **Provide** an API for other modules (Data View, Price Bucket Calculation) to query and retrieve data, *acting as a single source of truth for the entire process.*

8.4.6 Sample Scenario

Before Price Bucket Calculation runs, the Data Cache module loads 12 months of sales transactions for all clothing categories, along with their respective SKU and Style master data. When Price Bucket Calculation needs the ROS for “SKU123” over the last 90 days, it queries the Data Cache, which instantly returns the aggregated sales data, enabling fast computation of ROS for OW determination.

Key Takeaway: The Data Cache ensures all modules have immediate access to clean, consolidated, and up-to-date business data.

8.4.7 Results

- An in-memory repository of clean, indexed product, sales, and inventory data.
- Fast data retrieval capabilities for other modules.

8.4.8 Unusual Cases

- **Missing or delayed data feed:** If source systems fail to provide data, the cache will be incomplete, leading to downstream errors.
- **High memory consumption:** For very large datasets, the cache might exceed available memory, requiring optimization or a distributed caching solution.
- **Data inconsistencies:** If source data is not properly cleaned, inconsistent data might propagate through the cache, affecting calculations.
- **Slow data loading:** Large volumes of data can take significant time to load, impacting overall processing time.

8.4.9 Next Steps

- Data is consumed by Data View and Price Bucket Calculation modules.
- The cache needs to be periodically refreshed to ensure data freshness.

8.4.10 Common Questions

1. How fresh is the data in the cache?
2. What data sources does the cache pull from?
3. Can I force a cache refresh?
4. How does the cache handle data privacy/security?
5. What's the maximum amount of data the cache can hold?

8.4.11 Quick Reference

- **Purpose:** Centralized, fast data access.
- **Inputs:** Raw product, sales, inventory data.
- **Outputs:** In-memory dataset, fast query responses.
- **Key Value:** Boosts performance and ensures data consistency for OW and OD calculations.

8.5 Data View

8.5.1 Module Name & Business Objective

Module Name: Data View **Business Objective:** To provide a convenient and abstracted interface for other modules to access and derive business insights from the cached data, specifically offering utility functions for analyzing SKU/Style ROS and live status, critical for informed decision-making in product segmentation and price range effectiveness.

8.5.2 Required Information

- **Data Cache:** Access to the Data Cache module for querying underlying raw data.
- **Operational Data:** Date ranges for historical analysis, specific SKU/Style IDs for targeted queries.

8.5.3 Business Settings

- **defaultROSDays:** The default number of days for which ROS should be calculated if not specified.
- **activeStatusDefinition:** Criteria for determining if a product/store is considered “live” or active.

8.5.4 Process Flow

Receive Query --> Query Data Cache --> Perform Aggregations --> Return Derived Insights

8.5.5 Step-by-Step Actions

1. **Receive** data query requests from other modules (e.g., Price Bucket Calculation), *acting as a service layer for data insights.*
2. **Translate** complex business queries into efficient calls to the Data Cache, *optimizing data retrieval performance.*
3. **Aggregate** raw transactional data from the cache to calculate metrics like ROS, *transforming raw sales records into actionable insights.*
4. **Compute** product live status (e.g., actively stocked in X stores), *providing visibility into product availability and reach.*
5. **Apply** business rules and filters (e.g., exclude returns from ROS, filter by specific categories), *ensuring the insights are relevant and accurate for the specific business context.*
6. **Return** structured data representing the derived insights (e.g., SKU ROS per week, total revenue for a style), *enabling consuming modules to make informed decisions for OW and OD.*

8.5.6 Sample Scenario

The Price Bucket Calculation module needs the ROS for “Women’s Sneakers” for the past 6 months, broken down by individual Style. Instead of directly querying the raw sales data, it requests this from the Data View. The Data View fetches the relevant sales data from the Data Cache, aggregates it to calculate ROS per style, and returns a clean dataset. This allows Price Bucket Calculation to quickly identify high-selling styles and define appropriate price buckets. **Key Takeaway:** Data View simplifies data access and provides pre-computed business metrics, streamlining analysis for OW and OD.

8.5.7 Results

- Aggregated and derived business metrics (e.g., ROS, live status).
- Clean, structured datasets ready for analytical consumption.

8.5.8 Unusual Cases

- **Missing or delayed data feed:** If the underlying Data Cache is incomplete, the Data View will provide inaccurate or partial insights.
- **Complex query performance:** Very broad or complex queries might take longer to process, impacting the performance of consuming modules.
- **Misinterpretation of business rules:** Incorrect application of aggregation or filtering rules can lead to skewed results.

8.5.9 Next Steps

- Insights are consumed by modules like Price Bucket Calculation to drive their logic.

- Regular review of data view logic to ensure it aligns with evolving business needs.

8.5.10 Common Questions

1. What types of metrics can the Data View provide?
2. Can I add new aggregation functions to the Data View?
3. How does the Data View handle different time granularities (daily, weekly, monthly ROS)?
4. Is the data returned by Data View always current?
5. What are the limitations of the data insights provided?

8.5.11 Quick Reference

- **Purpose:** Abstraction for business insights.
- **Inputs:** Queries from other modules, data from Data Cache.
- **Outputs:** Derived metrics (ROS, live status).
- **Key Value:** Simplifies data analysis for OW and OD, ensures consistency.

8.6 Price Bucket Data Structures

8.6.1 Module Name & Business Objective

Module Name: Price Bucket Data Structures **Business Objective:** To formally define the standard data models used throughout the priceBucketCreation process, including PriceBucketData (representing a calculated price segment) and PriceBucketCreationArgs (configuration parameters), ensuring consistent data representation essential for inventory planning (OD, OW) and strategic pricing.

8.6.2 Required Information

- (No direct external information required as it defines internal structures).

8.6.3 Business Settings

- (This module primarily defines structures; settings are part of PriceBucketCreationArgs themselves).

8.6.4 Process Flow

Define PriceBucketData Attributes --> Define PriceBucketCreationArgs Attributes --> Enable Data Consistency

8.6.5 Step-by-Step Actions

1. **Define** the PriceBucketData class/schema, *specifying attributes like category ID, minMRP, maxMRP, revenueContribution, and associated ROS metrics.*
This structure captures the essence of an optimal price segment.

2. **Define** the PriceBucketCreationArgs class/schema, *outlining all configuration parameters such as target number of buckets, specific category lists, and thresholds for revenue contribution*. This structure ensures all inputs to the algorithm are clearly defined.
3. **Implement** serialization/deserialization methods for these structures, *allowing them to be easily stored, retrieved, and passed between modules and systems*.
4. **Ensure** data type integrity and constraints within the definitions, *preventing invalid data from being assigned to price bucket attributes (e.g., minMRP always a positive number)*.
5. **Provide** clear documentation for each field within the data structures, *aiding developers and business users in understanding the meaning and usage of each data point crucial for OD and OW*.

8.6.6 Sample Scenario

When Price Bucket Calculation completes its analysis for “Dresses,” it creates several PriceBucketData objects. One such object might be: { category: "Dresses", minMRP: 50.00, maxMRP: 89.99, revenueContributionPercentage: 0.35, averageROS: 15.2 }. This structured output is then easily consumed by the Orchestration module for syncing to other systems. **Key Takeaway:** Standardized data structures ensure clarity, consistency, and interoperability across the entire price bucket system, crucial for precise OD and OW planning.

8.6.7 Results

- Clearly defined and documented data models for PriceBucketData and PriceBucketCreationArgs.
- Enforced data consistency and integrity through type definitions and constraints.

8.6.8 Unusual Cases

- **Schema evolution:** Changes to business requirements might necessitate updates to these core data structures, requiring careful versioning and migration strategies.
- **Misinterpretation of fields:** Ambiguous field names or insufficient documentation can lead to incorrect usage of the data structures.

8.6.9 Next Steps

- These data structures are used by almost all other modules (Price Bucket Calculation creates PriceBucketData, Validation consumes PriceBucketCreationArgs, etc.).

8.6.10 Common Questions

1. Why are these structures defined separately?
2. How do we ensure backwards compatibility if structures change?
3. What are the key attributes of a PriceBucketData object?
4. Can I extend these structures for custom reporting?
5. Where are these structures physically stored (e.g., database schema)?

8.6.11 Quick Reference

- **Purpose:** Define core data models.
- **Inputs:** None (definitional).
- **Outputs:** Standardized data types for PriceBucketData and configuration.
- **Key Value:** Enables consistent data flow and understanding for OD and OW.

8.7 Price Bucket Calculation Helper

8.7.1 Module Name & Business Objective

Module Name: Price Bucket Calculation Helper **Business Objective:** To provide specialized utility functions that assist in the precise processing of price bucket data, particularly in integrating with master product information (SKU, Style) to accurately compute and adjust the actual min and max MRPs for each price bucket, ensuring the computed price segments reflect real-world product data for effective OW analysis.

8.7.2 Required Information

- **Product Data:**
 - Full product master data (SKU, Style, current MRPs).
- **Sales Data:**
 - ROS and revenue contribution from Price Bucket Calculation module.
- **Operational Data:**
 - Initial calculated price bucket ranges (from Price Bucket Calculation).

8.7.4 Process Flow

Receive Bucket Candidates --> Fetch Product MRPs --> Adjust Min/Max MRPs --> Return Refined Buckets

8.7.5 Step-by-Step Actions

1. **Receive** preliminary price bucket ranges and associated product lists from the Price Bucket Calculation module, *getting the initial proposal for price segments*.
2. **Query** the Data Cache for current MRPs of all products identified within these preliminary buckets, *ensuring access to the most up-to-date pricing information*.
3. **Analyze** the actual range of MRPs for products within each proposed bucket, *identifying the true minimum and maximum real-world prices*.
4. **Adjust** the theoretical minMRP and maxMRP of each price bucket to align with the actual lowest and highest product MRPs within that segment, *making the buckets practical and reflective of available inventory (OW)*.
5. **Filter** out any buckets that, after adjustment, contain no actual products or fall below a minimum product count threshold, *ensuring only viable price segments are retained*.
6. **Return** the refined price buckets, *providing highly accurate and actionable price segments for inventory and pricing decisions*.

8.7.6 Sample Scenario

The Price Bucket Calculation module might initially suggest a bucket of \$70–90. However, the Price Bucket Calculation Helper analyzes the actual products and finds that the lowest MRP for a product in this segment is \$72.50 and the highest is \$88.00. The helper then adjusts the bucket to \$72.50–88.00, providing a more precise and realistic range for the target category's OW. **Key Takeaway:** The helper ensures price buckets are not just theoretically sound but also practically applicable to actual product inventory, vital for effective OW.

8.7.7 Results

- Refined PriceBucketData objects with precise minMRP and maxMRP values based on actual product data.
- Increased accuracy and relevance of the calculated price segments.

8.7.8 Unusual Cases

- **Missing or delayed data feed:** Incomplete product MRP data from the Data Cache could lead to inaccurate or misaligned bucket adjustments.
- **No products in a theoretical bucket:** If a calculated theoretical bucket has no actual products within its range, the helper will correctly filter it out.
- **Rapid MRP changes:** If product MRPs change frequently between the data loading and calculation, the adjustments might be slightly out of sync.

8.7.9 Next Steps

- The refined price buckets are returned to the Price Bucket Calculation module for finalization.

8.7.10 Common Questions

1. Why is a helper needed if the main calculation defines buckets?
2. How does it handle products with very volatile MRPs?
3. Does it consider future planned MRP changes?
4. Can I configure how strict the MRP alignment is?
5. What happens if a product's MRP places it outside all defined buckets?

8.7.11 Quick Reference

- **Purpose:** Refine price bucket MRP ranges with actual product data.
 - **Inputs:** Initial buckets, product MRPs from Data Cache.
 - **Outputs:** Precisely adjusted price buckets.
 - **Key Value:** Ensures practical and accurate price segments for OW.
-

9. Appendix

9.1 Full Parameter Reference

(No detailed parameter reference beyond what's in `args.properties` was provided in the input. Please refer to the `args.properties` table in section 6 for available configurations.)

9.2 Version History

| Version | Date | Changes |
|---------|------------|--|
| 1.0 | 2025-05-23 | Initial release of the <code>priceBucketCreation</code> documentation. |

9.3 Contact & Support Info

For any questions, support, or feedback regarding the `priceBucketCreation` algorithm, please contact:

- **Team Email:** `analytics-support@yourcompany.com`
- **Internal Chat Channel:** `#price-optimization-support`
- **Documentation Link:** (Link to an internal wiki or confluence page if applicable)

Tutorial: priceBucketCreation

The priceBucketCreation system orchestrates a sequence of operations, starting with validation, followed by the core price bucket calculation. It heavily relies on a central data cache and structured data models for its operations. Helper modules assist in data processing and integration, providing the necessary insights for strategic pricing and inventory decisions (OW, OD).

flowchart TD

```
A0["Price Bucket Creation Orchestration"]
A1["Price Bucket Calculation"]
A2["Price Bucket Configuration Validation"]
A3["Data Cache"]
A4["Data View"]
A5["Price Bucket Data Structures"]
A6["Price Bucket Calculation Helper"]
A0 -- "control_flow" --> A2
A0 -- "control_flow" --> A1
A1 -- "uses" --> A5
A2 -- "uses" --> A5
A3 -- "uses" --> A6
A4 -- "data_flow" --> A3
A1 -- "depends_on" --> A3
A2 -- "depends_on" --> A3
```

Chapters

Master Report: priceBucketCreation

1. Title & Metadata

- **Project Name:** priceBucketCreation
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** An algorithm that intelligently segments products into optimized price buckets to enhance merchandising strategies and improve sales performance.

2. Executive Summary

The priceBucketCreation algorithm is a foundational component of our merchandising strategy, designed to categorize products into distinct price tiers or “buckets.” This system enables more granular analysis, targeted promotions, and effective inventory management based on price point. By standardizing price categorization, it facilitates better decision-making regarding product assortment (OD, OW), pricing strategies, and promotional planning.

- **Key outcomes or KPIs:**
 - Improved **ROS** by optimizing product placement within price tiers.
 - Enhanced ability to manage **OD** and **OW** effectively across different price points.
 - Minimized stockouts and overstocks by aligning inventory with demand profiles per price bucket.
 - Increased profitability through optimized pricing and promotional activities.
 - Streamlined reporting and analysis of sales performance across price segments.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Planning Teams, E-commerce Operations, Inventory Management, Business Intelligence & Analytics.

3. Architecture & Data Flow

graph TD

```
A[Raw Product Data<br>(SKUs, Prices, Categories, Sales History)] --> B[Data Ingestion & Validation];
B --> C[Price Tier Calculation];
C --> D[Product Assignment];
D --> E[Reporting & Output Generation];
E --> F[Price Bucket Catalog<br>(SKU to Bucket Mapping)];
```


E --> G[Summary Reports
(Bucket Performance, Gaps)];
F & G --> H[Merchandising & Planning Systems];

- **Component Bullets:**

- **Data Ingestion & Validation:** Gathers raw product data from various sources and performs initial cleansing and validation checks to ensure data quality.
- **Price Tier Calculation:** Determines the optimal number and ranges for price buckets based on historical sales data, market trends, and configurable parameters. This component often uses statistical methods or machine learning to define natural price clusters.
- **Product Assignment:** Assigns each individual SKU to its most appropriate price bucket based on its current price and the defined price tier ranges.
- **Reporting & Output Generation:** Compiles the results into a structured output catalog and generates analytical reports, highlighting key metrics for each bucket (e.g., average **ROS**, unit count, revenue contribution).

- **High-level data sources:**

- Product Master Data (SKU ID, current retail price, cost, category, subcategory, brand)
- Historical Sales Data (units sold, revenue per SKU)
- Inventory Data (stock levels, in-transit quantities)
- Merchandising Strategy Inputs (target number of price buckets, desired price points)

4. Configuration & Data Dictionary

Settings Table (args.properties)

| Parameter Name | Description | Default Value | Example Value |
|-------------------------|--|---------------|--------------------|
| num_price_buckets | The target number of price segments to create. | 5 | 3, 5, 7 |
| min_sku_per_bucket | Minimum number of SKUs required for a price bucket to be valid. | 10 | 5, 20 |
| price_range_strategy | Strategy for determining price ranges (e.g., equal_distribution, k-means, manual_tiers). | k-means | equal_distribution |
| recalculation_frequency | How often the price buckets should be re-evaluated (e.g., weekly, monthly, quarterly). | monthly | bi-weekly |
| input_data_path | Path to the primary input CSV/JSON file | /data/input/ | /usr/local/data/ |

| Parameter Name | Description | Default Value | Example Value |
|------------------|--|---------------|---------------|
| | containing product data. | | |
| output_data_path | Path where generated outputs (catalog, reports) will be stored. | /data/output/ | /var/reports/ |
| log_level | Level of detail for logging messages (e.g., INFO, DEBUG, ERROR). | INFO | DEBUG |

Shared Data Fields (common-data.json)

```
{
  "SKU_ID": {
    "type": "string",
    "description": "Unique identifier for each Stock Keeping Unit."
  },
  "Category": {
    "type": "string",
    "description": "Primary product category (e.g., 'Apparel', 'Electronics'). Used in AGs."
  },
  "Subcategory": {
    "type": "string",
    "description": "Secondary product category (e.g., 'T-shirts', 'Laptops'). Used in AGs."
  },
  "Brand": {
    "type": "string",
    "description": "Brand of the product. Used in AGs."
  },
  "Current_Price": {
    "type": "decimal",
    "description": "Current retail price of the SKU."
  },
  "Cost": {
    "type": "decimal",
    "description": "Cost of goods sold for the SKU."
  },
  "Historical_Sales_Units_Weekly": {
    "type": "integer",
    "description": "Average units sold per week over a defined period, used for ROS calculation."
  },
  "Inventory_Level": {
    "type": "integer",
    "description": "Current on-hand inventory quantity for the SKU."
  },
  "Ideal_Size_Set_Code": {
```

```

        "type": "string",
        "description": "Identifier for the ISS applied to the SKU, if
                        applicable."
    }
}

```

Outputs Catalog

| Output File Name | Format | Description | Key Fields (Partial) |
|-------------------------------|--------|---|---|
| price_bucket_catalog.csv | CSV | A mapping of each SKU to its assigned price bucket. | SKU_ID, Current_Price, Price_Bucket_ID, Price_Bucket_Range |
| bucket_summary_report.json | JSON | Aggregated statistics for each price bucket. | Price_Bucket_ID, Min_Price, Max_Price, SKU_Count, Total_Units_Sold, Avg_ROS |
| bucket_gaps_opportunities.csv | CSV | Identifies price ranges with few or no products, indicating potential for new inventory or adjustments to OD . | Gap_Range_Min, Gap_Range_Max, Suggested_Action |

5. End-to-End Flow

The priceBucketCreation algorithm follows a structured process to ensure accurate and actionable price segmentation:

- Data Acquisition:** The process begins by ingesting comprehensive product data, including SKU_ID, Current_Price, Category, Subcategory, Brand, Historical_Sales_Units_Weekly, and Inventory_Level from various internal databases and data lakes.
- Data Preprocessing:** Raw data undergoes validation checks (e.g., ensuring all prices are positive, SKU_ID uniqueness) and necessary transformations (e.g., handling missing values, standardizing category names).
- Price Tier Determination:** Based on the price_range_strategy configured in args.properties (e.g., k-means clustering on Current_Price), the algorithm calculates the optimal price ranges for num_price_buckets. This step considers the distribution of existing products to create meaningful tiers.
- SKU Assignment:** Each preprocessed SKU is then assigned to one of the newly defined price buckets. The assignment is typically based on the Current_Price

- falling within a specific bucket's range. Edge cases (prices exactly on a boundary) are handled by predefined rules.
5. **Performance Aggregation:** Once all SKUs are assigned, the algorithm aggregates data for each bucket. This includes counting the number of SKUs, calculating average ROS, total revenue, and other relevant KPIs for each price tier.
 6. **Reporting & Output Generation:** The final outputs, including the `price_bucket_catalog.csv` (mapping SKUs to buckets) and `bucket_summary_report.json` (aggregated statistics per bucket), are generated and stored in the `output_data_path`. Additional reports, such as `bucket_gaps_opportunities.csv` for merchandising review, are also produced.
 7. **Downstream Consumption:** The generated outputs are then consumed by merchandising, planning, and analytics systems to inform strategies related to OD, OW, pricing adjustments, and promotional campaigns.

6. Submodule Overview

6.1. Data Ingestion & Validation

1. **Name & Purpose:** Gathers raw product data and performs initial data quality checks to ensure robustness for subsequent processing.
2. **Inputs & Outputs:**
 - **Inputs:** Raw product data (e.g., `product_master.csv`, `sales_history.parquet`).
 - **Outputs:** Cleaned and validated DataFrame or dataset containing all required fields like `SKU_ID`, `Current_Price`, `Category`, `Historical_Sales_Units_Weekly`.
3. **Core Logic Summary:**
 - Reads product data from configured `input_data_path`.
 - Validates mandatory fields (e.g., `SKU_ID`, `Current_Price` are present and not null).
 - Checks for data type consistency and range validity (e.g., `Current_Price > 0`).
 - Handles duplicates and outliers according to predefined rules, potentially flagging them for manual review.

6.2. Price Tier Calculation

1. **Name & Purpose:** Determines the optimal price ranges for the defined number of price buckets based on product price distribution.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned product data with `Current_Price` and `Category`.
 - **Outputs:** A list of `Price_Bucket_Range` objects, each with a `Min_Price` and `Max_Price`.
3. **Core Logic Summary:**
 - Selects the `price_range_strategy` from `args.properties` (e.g., `k-means`, `equal interval`).
 - If `k-means`: Applies clustering on the `Current_Price` of all SKUs to identify `num_price_buckets` centroids, then defines ranges based on these clusters.

- If `equal_distribution`: Divides the overall price range (min to max price) into `num_price_buckets` equal intervals.
- Ensures each bucket has at least `min_sku_per_bucket` by potentially merging small buckets or adjusting boundaries.

6.3. Product Assignment

1. **Name & Purpose:** Assigns each individual SKU to its respective price bucket based on its `Current_Price` and the calculated price tier ranges.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned product data, calculated `Price_Bucket_Range` list.
 - **Outputs:** DataFrame or dataset augmented with `Price_Bucket_ID` and `Price_Bucket_Range` for each SKU.
3. **Core Logic Summary:**
 - Iterates through each SKU in the cleaned product data.
 - Compares the `Current_Price` of the SKU against the `Min_Price` and `Max_Price` of each defined `Price_Bucket_Range`.
 - Assigns the `SKU_ID` to the first bucket whose range it falls into.
 - Applies a tie-breaking rule for prices that fall exactly on a bucket boundary (e.g., always assign to the higher bucket, or lower bucket).

6.4. Reporting & Output Generation

1. **Name & Purpose:** Compiles the results into structured output files and generates summary reports for analysis and downstream consumption.
2. **Inputs & Outputs:**
 - **Inputs:** SKU data with assigned `Price_Bucket_ID`, and raw `Historical_Sales_Units_Weekly`, `Category`, `Inventory_Level`.
 - **Outputs:** `price_bucket_catalog.csv`, `bucket_summary_report.json`, `bucket_gaps_opportunities.csv` (all saved to `output_data_path`).
3. **Core Logic Summary:**
 - Creates the `price_bucket_catalog.csv` by joining SKU data with its assigned bucket information.
 - Calculates aggregate metrics for `bucket_summary_report.json` (e.g., total SKU count per bucket, average ROS per bucket, total revenue per bucket).
 - Analyzes price ranges without assigned SKUs to identify potential `bucket_gaps_opportunities.csv` that might indicate opportunities for **OD** expansion or new product introductions.
 - Writes all generated outputs to the configured `output_data_path`.

7. Worked Example

Let's assume `num_price_buckets = 3` and `price_range_strategy = equal_distribution`.

Input Data (Simplified `product_data.csv`):

| SKU_ID | Current_Price | Category | Historical_Sales_Units_Weekly |
|--------|---------------|----------|-------------------------------|
| P001 | 15.00 | Apparel | 20 |

| SKU_ID | Current_Price | Category | Historical_Sales_Units_Weekly |
|--------|---------------|----------|-------------------------------|
| P002 | 55.00 | Apparel | 10 |
| P003 | 95.00 | Apparel | 5 |
| P004 | 25.00 | Apparel | 18 |
| P005 | 45.00 | Apparel | 12 |
| P006 | 85.00 | Apparel | 6 |

Step 1: Data Ingestion & Validation - Data is read and validated. All prices are positive, no missing SKUs.

Step 2: Price Tier Calculation - Min Price = \$15.00, Max Price = \$95.00. - Overall Range = \$95.00 - \$15.00 = \$80.00. - num_price_buckets = 3. - Range per bucket = \$80.00 / 3 = \$26.67 (approximately). - Calculated Price Buckets: - **Bucket 1:** \$15.00 - \$41.67 (e.g., “Value”) - **Bucket 2:** \$41.68 - \$68.34 (e.g., “Mid-Range”) - **Bucket 3:** \$68.35 - \$95.00 (e.g., “Premium”)

Step 3: Product Assignment - Each SKU is assigned based on its Current_Price: - P001 (\$15.00) -> Bucket 1 - P002 (\$55.00) -> Bucket 2 - P003 (\$95.00) -> Bucket 3 - P004 (\$25.00) -> Bucket 1 - P005 (\$45.00) -> Bucket 2 - P006 (\$85.00) -> Bucket 3

Step 4: Reporting & Output Generation

Output (price_bucket_catalog.csv):

| SKU_ID | Current_Price | Price_Bucket_ID | Price_Bucket_Range | Category | Historical_Sales_Units_Weekly |
|--------|---------------|-----------------|--------------------|----------|-------------------------------|
| P001 | 15.00 | Value | \$15.00 - \$41.67 | Apparel | 20 |
| P004 | 25.00 | Value | \$15.00 - \$41.67 | Apparel | 18 |
| P002 | 55.00 | Mid-Range | \$41.68 - \$68.34 | Apparel | 10 |
| P005 | 45.00 | Mid-Range | \$41.68 - \$68.34 | Apparel | 12 |
| P003 | 95.00 | Premium | \$68.35 - \$95.00 | Apparel | 5 |
| P006 | 85.00 | Premium | \$68.35 - \$95.00 | Apparel | 6 |

Output (bucket_summary_report.json):

```
[
  {
    "Price_Bucket_ID": "Value",
    "Min_Price": 15.00,
    "Max_Price": 41.67,
    "SKU_Count": 2,
    "Total_Units_Sold": 38,
    "Avg_ROS": 19.0
  },
  {
    "Price_Bucket_ID": "Mid-Range",
    "Min_Price": 41.68,
    "Max_Price": 68.34,
    "SKU_Count": 2,
    "Total_Units_Sold": 22,
```

```

    "Avg_ROS": 11.0
  },
  {
    "Price_Bucket_ID": "Premium",
    "Min_Price": 68.35,
    "Max_Price": 95.00,
    "SKU_Count": 2,
    "Total_Units_Sold": 11,
    "Avg_ROS": 5.5
  }
]

```

This output helps merchandising identify that ‘Value’ products have the highest **ROS**, suggesting potential for increasing **OD** and **OW** in this price segment.

8. Debug Guide Highlights

- **Check args.properties:** Many issues stem from incorrect configuration. Verify input_data_path, output_data_path, and numerical parameters like num_price_buckets.
- **Inspect Input Data:** The most common source of errors is malformed, missing, or inconsistent input data.
 - Verify Current_Price is numeric and positive.
 - Check for duplicate SKU_ID entries.
 - Ensure all required columns (e.g., SKU_ID, Current_Price) are present.
- **Review Log Files:** The algorithm generates detailed logs. Look for ERROR or WARNING messages, especially around data ingestion and price tier calculation steps.
- **Validate Interim Outputs:** If possible, inspect outputs from earlier submodules (e.g., the cleaned data before product assignment) to isolate where data corruption or unexpected values are introduced.
- **Small Sample Runs:** For complex issues, run the algorithm with a very small, controlled dataset (like the worked example) to quickly pinpoint where the logic deviates from expected behavior.

9. Common Pitfalls & Remedies

- **Pitfall 1: Data Incompleteness or Inaccuracy**
 - **Description:** Missing Current_Price for SKUs, SKU_ID collisions, or non-numeric price values.
 - **Remedy:** Implement robust data validation in the Data Ingestion & Validation submodule. Use a data quality dashboard. Fail fast on critical errors or default to a configurable handling strategy (e.g., exclude problematic SKUs, use a default price).
- **Pitfall 2: Suboptimal Bucket Definitions**
 - **Description:** Price buckets that are too wide, too narrow, or don’t align with market segments, leading to poor merchandising insights or an unbalanced distribution of SKUs. This can impact **OD** and **OW** strategies.
 - **Remedy:** Utilize different price_range_strategy options (e.g., k-means vs. equal_distribution). Conduct A/B tests on different bucket configurations. Incorporate manual overrides or expert feedback to fine-

tune bucket ranges. Regularly review the `bucket_summary_report.json` and `bucket_gaps_opportunities.csv` for insights.

- **Pitfall 3: Performance Issues with Large Datasets**

- **Description:** The algorithm runs slowly or consumes excessive memory when processing millions of SKUs.
- **Remedy:** Optimize data processing (e.g., use vectorized operations in Python/R, leverage distributed computing frameworks like Spark). Implement efficient indexing. Ensure efficient I/O operations for reading/writing data.

- **Pitfall 4: Misinterpretation of ROS or Other KPIs**

- **Description:** Downstream teams incorrectly interpret the aggregated ROS or other metrics from `bucket_summary_report.json` due to misunderstanding of data sources or calculation methodologies.
- **Remedy:** Provide clear, concise documentation for each output field (as in the Data Dictionary). Conduct training sessions for stakeholders. Implement a feedback loop with merchandising teams to clarify reporting.

10. FAQ

- **Q1: What is the primary purpose of price buckets?**

- **A:** Price buckets organize products into distinct price tiers, making it easier for merchandising teams to analyze performance, identify gaps or opportunities for **OD** and **OW**, and strategize promotions or pricing actions within manageable segments.

- **Q2: How often should the price buckets be re-calculated?**

- **A:** The frequency is configured by `recalculation_frequency` in `args.properties`. Typically, it's monthly or quarterly, but it can be adjusted based on market volatility, new product introductions, or significant pricing strategy changes.

- **Q3: What if a product's price changes significantly after bucket assignment?**

- **A:** The current version assigns based on the price at the time of calculation. Significant price changes would require a re-run of the algorithm to re-assign the SKU to its correct bucket. For immediate tactical decisions, this might require manual override, but the next scheduled run will correct it.

- **Q4: Can I customize the price bucket ranges manually?**

- **A:** While the algorithm offers automated strategies (e.g., k-means), it's possible to introduce a `manual_tiers` option in `price_range_strategy` to allow for predefined ranges based on business judgment. This would require configuration in `args.properties`.

- **Q5: How does this algorithm relate to ISS?**

- **A:** While not directly calculating ISS, the price buckets can provide context. For example, high-volume price buckets (identified by high **ROS**) might benefit from a more detailed **ISS** analysis to ensure sufficient stock levels in popular sizes.

- **Q6: What is an Attribute Group (AG) in this context?**

- **A:** An **AG** refers to a collection of product characteristics like Category, Subcategory, or Brand that can be used to further segment or analyze price buckets. For instance, you could analyze the performance of "Value" price buckets specifically for the "Apparel" **AG**.

11. Quick Reference Cheat-Sheet

| Feature | Description |
|---------------------------|--|
| Project Name | priceBucketCreation |
| Version | 1.0 |
| Primary Goal | Optimize product segmentation into price tiers. |
| Key Output | price_bucket_catalog.csv (SKU to bucket mapping) |
| Configuration File | args.properties (for num_price_buckets, input_data_path, etc.) |
| Input Data Source | Product Master Data, Sales History, Inventory Data. |
| Key Business Terms | OD (Optimum Depth), OW (Optimum Width), ROS (Rate of Sale), AG (Attribute Group), ISS (Ideal Size Set). |
| Log Directory | /var/log/priceBucketCreation/ (example) |
| Output Directory | /data/output/price_buckets/ (configured via output_data_path) |
| Contact Support | Data Science Team / Merchandising Analytics Lead |

Chapter 2: Reallocation Prepare Data Module

Last Updated: 2025-05-23

1. Business Objective

This module establishes the foundational dataset required for precise inventory reallocation by consolidating all relevant master and transactional retail information.

Why This Matters: Accurate and complete data ensures that subsequent reallocation decisions are based on the most current and comprehensive view of inventory, leading to better stock distribution, minimized lost sales, and improved customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold (ROS):** This tells us how quickly specific items are selling in different stores, which helps forecast future demand.
- **Average Selling Price (ASP):** Knowing the average price helps value the inventory and prioritize reallocation efforts based on potential revenue.

Product Data

- **Product Characteristics (AG):** Information like color, style, and material helps group similar products and understand their attributes for better planning.
- **Ideal Size Set (ISS) Definitions:** These are the optimal combinations of sizes for a product or category in a given store, crucial for ensuring we stock the right mix.
- **Optimum Depth (OD) & Optimum Width (OW) Rules:** These business rules define the ideal number of unique items (OD) and categories or styles (OW) to stock, guiding inventory targets.
- **Product Family Links:** Understanding which individual items (SKUs) belong to which product styles helps in managing inventory at a broader level.

Operational Data

- **Current Store Inventory Levels:** This is the most up-to-date count of products physically on shelves at each store.
- **Centralized Warehouse Stock:** Knowing the available inventory in our distribution centers is vital for fulfilling store needs.
- **Incoming Shipments:** This includes items currently shipping (Goods In Transit), customer orders awaiting fulfillment (Open Orders), stock moving between locations (In-Transit Transfers), and expected new stock from suppliers (Purchase Orders).
- **Store Layout Guidelines (Planograms):** These define display capacities and replenishment cycles, informing how much product a store can physically hold and how often it can be restocked.
- **Enabled Stores List:** This specifies which stores are actively participating in the reallocation process.
- **Channel-Specific Product Rules:** Special business rules, like unique product assortments or depth requirements for online vs. physical stores, need to be applied.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---------------------------------|---------------|---|---|
| Include Store Inventory | Yes | Determines if the current stock on store shelves is included. | If conducting a high-level plan without current stock. |
| Include In-Transit Stock | Yes | Accounts for products currently en route to stores. | If only planning for immediate, on-hand stock. |
| Include Warehouse Stock | Yes | Factors in inventory available in central warehouses. | If focusing solely on store-to-store transfers. |
| Active Season List | Current Year | Defines which product seasons are considered “in-season.” | At the start of a new retail season or for seasonal analysis. |

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Raw Retail Data (Inventory, Sales, Product, Store)] --> B[Data Loading & Initialization]
B --> C[Structured Reallocation Dataset]
```

1. Start with → All raw master and transactional retail data (e.g., current stock counts, incoming orders, product definitions).

2. Then → Validate and **consolidate** data from various sources (e.g., combining on-hand store stock with warehouse inventory and incoming shipments).
3. Next → **Identify** current inventory, future incoming stock, apply business rules like “in-season” status, and determine ideal stock levels (OD, OW, ISS).
4. Finally → Produce a complete, structured dataset ready for reallocation calculations.

5. Step-by-Step Actions

1. **Identify** active stores and current season products
 - What we do: *We pinpoint which stores are participating in reallocation and which products are considered “in-season” based on defined periods.*
 - Why we do it: *This ensures we only focus on relevant stores and products for active reallocation, avoiding unnecessary movements for past-season items or non-participating locations.*
2. **Load** all inventory from stores, warehouses, and incoming shipments
 - What we do: *We gather all existing stock from store shelves, our central warehouses, and track any products that are currently in transit, on open customer orders, or incoming from suppliers.*
 - Why we do it: *This provides a complete picture of total available and anticipated inventory across the entire network, preventing stockouts and overstocking.*
3. **Define** replenishment cycles and Ideal Size Set (ISS) for each store
 - What we do: *We calculate how often stores can receive new inventory and determine the best mix of sizes for specific products or categories within those stores.*
 - Why we do it: *This ensures that reallocation recommendations align with a store’s operational capacity and customer demand for specific sizes, maximizing sales efficiency.*
4. **Determine** Optimum Width (OW) recommendations for each store and product group
 - What we do: *We establish the ideal number of different categories or styles a store should offer to maximize sales and customer choice.*
 - Why we do it: *This helps maintain a balanced product assortment, preventing stores from having too few options or being overwhelmed with excessive variety.*
5. **Apply** channel-specific product overrides and depth rules
 - What we do: *We implement any special business rules for specific sales channels (e.g., e-commerce vs. physical stores) or product-specific quantity limits (OD).*
 - Why we do it: *This ensures that unique merchandising strategies or capacity constraints are reflected in the data, tailoring reallocation to specific channel needs.*
6. **Identify** currently active products and categorize them for sales rate analysis
 - What we do: *We flag products that are currently active in our inventory and prepare them for detailed analysis of their sales speed (ROS).*
 - Why we do it: *This distinguishes products that are part of current sales strategies from older inventory, focusing reallocation efforts on high-impact items.*
7. **Consolidate** all relevant data into a unified, structured dataset
 - What we do: *We bring all the gathered and processed information together into one coherent dataset.*

- Why we do it: *This provides a single, reliable source of truth for the subsequent reallocation calculations, ensuring consistency and accuracy.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A store in a specific region currently holds 10 units of Style X (a 'summer' product) and 5 units of Style Y (a 'winter' product).
- The central warehouse has 100 units of Style X and 0 of Style Y.
- A new purchase order for 50 units of Style X is expected to arrive at the warehouse next week.
- The system is configured to consider summer products as 'in-season' for the current reallocation cycle.

What Happens

1. The module first **identifies** that Style X is 'in-season' based on the active season list, while Style Y is classified as 'out-of-season'.
2. It then **loads** the 10 units of Style X from the store's current inventory. It adds the 100 units of Style X from the central warehouse stock.
3. The module also **includes** the 50 units of Style X from the incoming purchase order, recognizing this as future available inventory.
4. The 5 units of Style Y are noted as 'out-of-season' and are typically excluded from active reallocation calculations, though their presence is recorded.
5. Finally, the system **combines** all this data to show a total of 160 units of Style X available (10 store + 100 warehouse + 50 incoming PO), while Style Y is accurately marked as out of season and will not be actively reallocated.

Final Outcome

A comprehensive, accurate snapshot of inventory for Style X, including what's on shelves, in the warehouse, and arriving soon, all categorized by its 'in-season' status. Style Y is marked as 'out-of-season' and will not be actively reallocated.

Key Takeaway: This module ensures that only relevant, available, and incoming inventory is considered for reallocation, providing a true picture of stock assets.

7. Results

This step produces:

Main Outputs

- **Consolidated Inventory Data:** This is a comprehensive view of real-time stock levels at all stores and warehouses, including any items currently in transit or pending fulfillment. It tells us exactly how much of each product is where.

- **Product Status Classifications:** This clearly identifies which products are currently in-season, out-of-season, or otherwise active for reallocation. This guides which items should be moved.
- **Ideal Size Set (ISS) & Optimum Depth (OD) Recommendations:** These are the store-specific targets for the best combination of sizes and the ideal number of units (OD) for a product category, guiding precise inventory targets.
- **Optimum Width (OW) Configuration:** This output provides category and attribute-specific recommendations for the ideal number of different categories or styles to offer, ensuring a balanced assortment.

Business Reports

- **Inventory Snapshot Report:** Used by inventory planners and store managers to see current and incoming stock levels across the entire retail network, helping them understand overall availability.
- **Product Seasonality Report:** Used by merchandising and planning teams to understand the lifecycle status of products, informing decisions about markdowns, promotions, or removal from active allocation.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|-------------------------------------|---|---|
| Missing or delayed data feed | The module might process with incomplete data, leading to inaccurate allocations. | Verify all data sources are up-to-date and re-run the module once data is complete. |
| No active stores enabled | The module will run but produce no reallocation suggestions. | Ensure at least one store is flagged as “enabled” for reallocation. |
| All products marked ‘out-of-season’ | No products will be eligible for active reallocation. | Review the active season list and product seasonality data. |
| Critical product attributes missing | Certain products might not be correctly categorized or subject to ideal planning rules. | Ensure all product characteristics (AGs) are fully defined in master data. |

9. Next Steps

Before This Step

- **Master Data Updates:** All product definitions, store information, warehouse details, and ideal size set (ISS) configurations must be accurate and current. This is the foundation for all planning.
- **Sales Data Integration:** Ensure all historical sales data is loaded and verified, as it provides crucial insights into product performance and demand patterns.

After This Step

- **Reallocation Planning Module:** This next module uses the prepared, structured data to calculate optimal stock movements, determining which products should be moved where.
- **Inventory Execution Systems:** These systems receive the reallocation recommendations and generate tasks for physically moving inventory, such as transfers between stores or shipments from warehouses.

10. Common Questions

Q: Why is a product not showing up in the reallocation plan? A: It might be marked as ‘out-of-season’ or not linked to an ‘active season’, or its stock flags (e.g., “Include Store Inventory”) might be set to ‘No’, excluding its current stock from consideration.

Q: How does this module ensure my stores get the right sizes? A: It loads and applies your Ideal Size Set (ISS) definitions, ensuring that future reallocation considers the ideal size mix for each product category based on historical sales and business rules.

Q: What if I have stock that’s not on shelves yet, like a new delivery or customer orders? A: The module accounts for “Incoming Shipments” (Goods In Transit, Open Orders, Purchase Orders, Inter-Store Transfers), ensuring these future quantities are included in the overall inventory picture.

Q: Can I run reallocation for only specific stores? A: Yes, the module identifies and loads only the “Enabled Stores List”, allowing you to define precisely which stores participate in the reallocation process, giving you control over the scope.

Q: Why is it important to have accurate Store Layout Guidelines (Planogram) data for this module? A: Accurate Planogram data provides the system with store capacity, display rules, and replenishment cycle days. This is crucial for determining optimal stock levels and understanding how quickly stores can receive and display new inventory, ensuring realistic and effective allocations.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To gather and structure all essential inventory, product, and store data, providing a unified foundation for reallocation decisions.
 - **Main Input:** Raw inventory counts, sales history, product characteristics, and store configurations.
 - **Main Output:** A comprehensive, structured dataset of all relevant stock and product information, ready for analysis.
 - **Warning Signs:** Incomplete inventory counts, missing product attributes (AGs), or unexpected ‘out-of-season’ flags.
 - **Key Settings:** “Include Store Inventory,” “Include In-Transit Stock,” “Include Warehouse Stock,” “Active Season List.”
-

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 3: Reallocation Prepare Sales Module

Last Updated: 2025-05-23

1. Business Objective

This module focuses on preparing and cleaning raw sales data, ensuring its quality and relevance for precise inventory reallocation analysis.

Why This Matters: Accurate and reliable sales data is the foundation for effective demand forecasting. By cleaning and validating sales information, this module ensures that reallocation decisions are based on true customer demand, leading to better stock distribution, minimized lost sales, and improved inventory efficiency.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold (ROS):** This raw sales data tells us how many units of each product were sold. We need it to calculate the Rate of Sale (ROS) after cleaning.
- **Sales Transaction Details (e.g., date, price, store):** Essential for filtering out specific types of sales (like liquidation) and identifying “broken” sales periods.
- **Liquidation Sales Flags:** Indicates if a sale was part of a liquidation event, allowing us to exclude it from typical demand patterns.
- **Seasonal Flags:** Helps identify and segregate “priority sales” for specific periods or product groups that need special analysis.

Product Data

- **Product Characteristics (AG):** Information like color, style, and material helps link sales to specific product attributes for detailed analysis and aggregation.
- **Product Family Links (SKU to Style):** Crucial for aggregating sales data from individual items (SKUs) up to broader product styles, which is often needed for sales analysis.
- **Ideal Size Set (ISS) Definitions:** While not directly calculating ISS, the system uses ISS definitions to assess product availability and identify “broken” sales where the ideal size mix wasn’t available.

Operational Data

- **Store Information:** Details about each store, including its ID and status, are necessary to associate sales with specific locations.
- **Analysis Date and Historical Periods:** Defines the timeframe for which sales data is gathered and ROS is calculated (e.g., how many months back to analyze).
- **Store Layout Guidelines (Planograms):** While not directly sales data, this information indicates display capacities and stock availability, which is critical for determining if sales data is “broken” due to product not being on shelf.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|---------------|---|---|
| Liquidation Cleanup Benchmark | 0.00 | The threshold to identify and remove liquidation sales data. | When specific sales events should be excluded from typical demand forecasting. |
| Broken Sales Availability Benchmark | 0.80 | Minimum percentage of Ideal Size Set (ISS) products that must be available for sales data to be considered “unbroken.” | If product availability is frequently inconsistent, or if sales patterns are often skewed by stockouts. |
| Broken Sales Live Days Benchmark | 15 | Minimum number of days a product/style combination must meet the availability benchmark to be included in sales analysis. | To ensure sales data reflects consistent product availability over a meaningful period. |
| Minimum Live Days for ROS | 7 | The minimum number of live selling days required for a SKU to have its Rate of Sale (ROS) calculated. | To prevent highly volatile or unreliable ROS calculations for products with very short sales histories. |

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```
A[Raw Historical Sales Data] --> B[Clean Liquidation Sales]
B --> C[Filter Broken Sales Data]
C --> D[Compute SKU-level Rate of Sale (ROS)]
D --> E[Identify Priority Sales]
E --> F[Cleaned Sales Data & ROS for Reallocation]
```

1. Start with → Raw Historical Sales Data for all products and stores.

2. Then → **Remove** sales associated with liquidation events that might skew typical demand patterns.
3. Next → **Filter out** sales data from periods where product availability was inconsistent (e.g., due to stockouts), ensuring only reliable sales are used.
4. Next → **Compute** the Rate of Sale (ROS) for each individual item (SKU) across multiple historical periods, creating a solid base for demand forecasting.
5. Next → **Identify** and segregate “priority sales” based on seasonal flags for specific merchandising analyses, such as holiday or promotional periods.
6. Finally → Produce a comprehensive set of Cleaned Sales Data and reliable ROS metrics, ready for the next stages of reallocation.

5. Step-by-Step Actions

1. **Remove** liquidation sales
 - What we do: *We review the raw sales data and exclude any sales transactions that occurred during designated liquidation periods or for products specifically flagged as liquidation inventory.*
 - Why we do it: *Liquidation sales often involve heavy discounts and do not reflect typical customer demand, so removing them prevents skewed historical sales analysis.*
2. **Filter** broken sales data
 - What we do: *We identify periods where product availability (e.g., the Ideal Size Set for a style) was below a certain threshold. Sales that occurred during these “broken” inventory periods are then excluded from the analysis.*
 - Why we do it: *This ensures that the sales data accurately reflects customer demand when products were adequately available, preventing misinterpretation of sales performance due to stockouts or incomplete assortments.*
3. **Calculate** SKU-level Rate of Sale (ROS)
 - What we do: *For each unique item (SKU) in every store, we compute its Rate of Sale (ROS) by dividing the total units sold by the number of active selling days within defined historical periods (e.g., 30, 60, 90 days).*
 - Why we do it: *ROS provides a clear measure of how quickly products are selling, which is critical for forecasting future demand and determining optimal stock levels for reallocation.*
4. **Identify** and segregate priority sales
 - What we do: *We use seasonal flags or specific date ranges to pinpoint and separate sales data for “priority periods” like peak season, holidays, or major promotional events.*
 - Why we do it: *These sales patterns are often unique and require distinct analysis, ensuring that reallocation plans consider specific high-demand periods appropriately.*
5. **Consolidate** cleaned sales and ROS
 - What we do: *We bring all the filtered sales data, calculated ROS, and identified priority sales together into a unified, structured dataset.*
 - Why we do it: *This creates a single, reliable source of truth for all sales-related inputs required by subsequent reallocation modules, ensuring consistency and accuracy.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A store sold 100 units of a specific T-shirt (SKU A) over the last 90 days.
- Out of these 90 days, 10 days were part of a “liquidation sale” event for outdated inventory.
- For another 5 days, the store had stockouts of key sizes for SKU A, meaning its Ideal Size Set (ISS) was less than 50% available.
- The current Liquidation Cleanup Benchmark is set to exclude liquidation sales entirely.
- The Broken Sales Availability Benchmark is set to 80% (meaning if less than 80% of the ISS was available, sales for that day are considered “broken”).
- The Broken Sales Live Days Benchmark is 15 days.

What Happens

1. First, the module **removes** the 10 days of sales that occurred during the liquidation event. This reduces the total sales of SKU A to 90 units over 80 relevant days.
2. Then, it **filters out** sales from the 5 days where the store had less than 80% of the Ideal Size Set for SKU A. Let's say 5 units were sold during these broken periods. The remaining sales are now 85 units over 75 reliable days.
3. Next, the module **calculates** the Rate of Sale (ROS) for SKU A using these cleaned sales: $85 \text{ units} / 75 \text{ days} = 1.13 \text{ units/day}$. This is the reliable ROS for SKU A.
4. Finally, if any of these sales days fell within a designated “priority season” (e.g., a specific holiday period), those 85 units would also be flagged as “priority sales” for separate analysis.

Final Outcome

A precise Rate of Sale (ROS) of 1.13 units/day for SKU A, based solely on reliable sales data where stock was consistently available and excluding promotional anomalies. This clean ROS is now ready to inform replenishment and reallocation decisions.

Key Takeaway: This module ensures that demand signals are clear and accurate by removing misleading sales data, leading to better inventory decisions.

7. Results

This step produces:

Main Outputs

- **Cleaned Sales Data:** This is the raw sales data after removing liquidation sales and filtering out periods with inconsistent product availability, providing a high-quality dataset for analysis.
- **SKU-level Rate of Sale (ROS):** This is a computed metric for each individual item (SKU) in every store, representing its true selling speed across multiple historical periods, essential for forecasting.
- **Priority Sales Data:** This is a segregated subset of sales transactions specifically identified by seasonal flags or date ranges for special analysis (e.g., holiday demand, promotional impact).

Business Reports

- **Sales Quality Report:** Used by inventory planners and data analysts to understand the volume of sales data filtered out and the reasons why (e.g., liquidation, broken availability), ensuring data integrity.
- **SKU Performance Dashboard (ROS Focus):** Used by merchandising and planning teams to quickly assess the selling speed of individual SKUs in different stores, guiding decisions on which products to prioritize for replenishment or reallocation.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---------------------------------------|---|--|
| Missing or delayed data feed | The module might process with incomplete sales data, leading to inaccurate ROS calculations and skewed demand signals. | Verify all raw sales data sources are up-to-date and re-run the module once data is complete. |
| Excessive “broken” sales data | A large percentage of historical sales might be filtered out, potentially leading to insufficient data for ROS calculation for many SKUs. | Review store operational practices (e.g., inventory accuracy, replenishment frequency) and re-evaluate “Broken Sales Availability Benchmark” settings. |
| No priority sales identified | Even during known seasonal peaks, the module might not flag any “priority sales.” | Verify the accuracy of seasonal flags and InputPeriodRow settings in the system, ensuring they align with actual business seasons. |
| ROS for many SKUs is zero or very low | The filtering parameters (e.g., Minimum Live Days for ROS or Broken Sales Availability Benchmark) might be too aggressive, discarding too much relevant sales data. | Adjust the relevant business settings to be less restrictive, allowing more sales data to contribute to ROS calculation. |

9. Next Steps

Before This Step

- **Master Data Updates:** All product definitions (including Ideal Size Set and Product Family Links) and store information must be accurate and current. This provides the context for sales data.
- **Sales Data Integration:** Ensure all raw historical sales data from various sources is loaded, validated, and readily available for processing. This is the primary input for this module.

After This Step

- **Reallocation Planning Module:** This next module utilizes the cleaned sales data and reliable SKU-level Rate of Sale (ROS) to calculate optimal stock movements and develop reallocation plans.
- **Demand Forecasting Module:** The robust, clean ROS data and identified priority sales are crucial inputs for more advanced demand forecasting models, which then inform future inventory buys and strategic planning.

10. Common Questions

Q: Why are some historical sales quantities missing from the cleaned data? A: Sales might be excluded if they occurred during a “liquidation” period or if the store experienced “broken” product availability (e.g., key sizes were out of stock) during those sales days. This ensures only reliable demand signals are used.

Q: How is the Rate of Sale (ROS) calculated if products aren’t always in stock? A: The module specifically filters out “broken” sales periods where product availability was inconsistent. ROS is then calculated only on the days when products were adequately available, providing a true measure of demand, not just sales limited by stock.

Q: What are “priority sales” and why are they segregated? A: Priority sales refer to sales data from specific periods, like holiday seasons or major promotional events, that are identified by seasonal flags. They are segregated because their patterns might be unique and require separate analysis or special consideration in reallocation strategies.

Q: Why is accurate historical sales data so important for this module? A: Accurate and comprehensive historical sales data is the fundamental input. Without it, the module cannot reliably clean sales, calculate ROS, or identify true demand patterns, leading to flawed reallocation decisions.

Q: How does cleaning sales data directly impact my store’s inventory? A: By providing a more accurate picture of what truly sells when it’s available, cleaned sales data ensures that reallocation decisions are based on real customer demand, leading to better stock distribution, reduced overstocking, and fewer missed sales opportunities.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To prepare and clean raw sales data, producing high-quality, reliable sales metrics for reallocation.
- **Main Input:** Raw historical sales data for all products and stores.
- **Main Output:** Cleaned Sales Data, SKU-level Rate of Sale (ROS), and segregated Priority Sales Data.
- **Warning Signs:** Unusually low overall sales volumes after processing, or ROS figures that seem inconsistent with expectations.
- **Key Settings:** Liquidation Cleanup Benchmark, Broken Sales Availability Benchmark, Broken Sales Live Days Benchmark.

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: Reallocation Prepare Warehouse Inventory Module

Last Updated: 2025-05-23

1. Business Objective

This module refines the warehouse inventory data by accounting for practical availability and integrating incoming stock, ensuring an accurate and usable figure for reallocation decisions.

Why This Matters: Accurate warehouse stock figures are critical for making effective reallocation decisions. By refining these numbers, the module prevents committing to inventory that isn't truly available (due to inefficiencies or future needs) and ensures that all potential stock, including new deliveries, is considered, leading to optimal stock distribution and reduced lost sales.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS):** This helps calculate future demand for online stores, which can influence how much warehouse stock needs to be set aside for their lead time.

Product Data

- **Product Family Links (SKU to Style):** Used to aggregate warehouse stock from individual items (SKUs) up to broader product styles, if the system handles stock at the style level for certain calculations.
- **Category Information:** Essential for understanding which product categories are associated with which warehouses and stores, especially when calculating inventory for online channels.

Operational Data

- **Centralized Warehouse Stock:** The initial, raw count of inventory available in our distribution centers.

- **Warehouse Inefficiency Factor:** A configured number of pieces to deduct per product style or SKU to reflect practical inventory availability challenges (e.g., mispicks, damaged items, location issues).
- **Incoming Shipments (Purchase Orders, In-Transit Transfers):** Quantities of stock expected to arrive at the warehouse soon, which are combined with current stock to represent total available future inventory.
- **Store Channel Type (Online/Physical):** Distinguishes between online and physical stores, as online channels might have different lead time considerations for warehouse stock.
- **Lead Time for Sales Exclusion:** The number of days used to calculate how much warehouse stock needs to be reserved to cover expected sales during the time it takes to process and ship an order from the warehouse to an online store.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|------------------------------|---|--|
| Warehouse Inefficiency Factor | 2 | The number of pieces per product style or SKU to deduct from raw warehouse stock. | If warehouse inventory accuracy consistently shows a higher or lower discrepancy. |
| Lead Time for Sales Exclusion | 7 days | The number of days of sales to reserve from warehouse stock for online fulfillment. | If average shipping/processing time for online orders changes significantly. |
| Minimum Live Days for ROS Calculation | 7 | The minimum number of days a product must have been actively available to calculate its Rate of Sale (ROS). | To ensure ROS calculations are based on sufficient selling history, especially for new products. |
| ROS Calculation Periods & Weights | (e.g., 30/60/90 days, 5/3/2) | Defines the historical periods and their importance for calculating a product's Rate of Sale (ROS). | To adjust the recency or breadth of sales data influencing demand forecasts. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw Warehouse Inventory (including POs & In-Transits)] --> B[Adjust for Warehouse Inefficiency]
    B --> C[Exclude Stock for Online Store Lead Time (if
  
```

applicable)}

C --> D[Refined Warehouse Inventory]

1. Start with → The combined raw warehouse inventory, which includes current stock on hand plus all expected incoming shipments from purchase orders and transfers.
2. Then → **Deduct** a pre-configured “Warehouse Inefficiency Factor” from each product style or SKU’s stock to account for practical availability issues.
3. Next → (If applicable for online stores) **Calculate** and **exclude** a portion of warehouse stock to cover expected sales during the lead time required for order fulfillment to online channels.
4. Finally → Produce a precise, refined set of warehouse inventory figures ready for the main reallocation planning.

5. Step-by-Step Actions

1. **Adjust** for warehouse inefficiency
 - What we do: *We reduce the reported quantity of each product style or SKU in the warehouse by a specific “Warehouse Inefficiency Factor.”*
 - Why we do it: *This accounts for practical issues like miscounted items, damaged stock, or items that are hard to locate, ensuring that reallocation plans only consider truly available inventory.*
2. **Account** for lead time sales (if applicable)
 - What we do: *For warehouses that fulfill online orders, we calculate the estimated sales that will occur during the lead time required to process and ship products. This estimated quantity is then reserved and not considered for reallocation to physical stores.*
 - Why we do it: *This prioritizes stock for immediate online customer demand, preventing physical stores from pulling inventory that is needed for e-commerce fulfillment and improving overall customer satisfaction.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- The central warehouse reports 150 units of “Women’s Summer Dress - Style A.”
- An additional 20 units of “Style A” are expected next week from a Purchase Order.
- The Warehouse Inefficiency Factor is set to 2 units per style.
- The Lead Time for Sales Exclusion is 7 days, and the expected sales for “Style A” over the next 7 days for online orders is 15 units.
- The warehouse serves both online and physical stores.

What Happens

1. First, the module consolidates the raw warehouse stock: 150 (current) + 20 (incoming PO) = 170 units of “Style A.”

2. Then, it **adjusts** for warehouse inefficiency by deducting the Warehouse Inefficiency Factor. So, 170 units - 2 units = 168 units available for planning.
3. Next, because the warehouse fulfills online orders, the module **accounts** for the 7-day lead time sales. It will reserve 15 units (the calculated 7-day online sales) from the 168 available units. This means $168 - 15 = 153$ units are now considered truly available for reallocation to physical stores.

Final Outcome

The system now accurately shows 153 units of “Women’s Summer Dress - Style A” as available for reallocation to physical stores, while 15 units are effectively reserved for upcoming online sales and 2 units are buffered for practical warehouse inefficiencies.

Key Takeaway: This module provides a realistic count of usable warehouse inventory by factoring in operational realities and future commitments, preventing over-commitment and optimizing stock allocation.

7. Results

This step produces:

Main Outputs

- **Refined Warehouse Inventory:** This is the adjusted count of available product styles or SKUs in the warehouse, after deducting for inefficiency and reserving for online lead time sales. It represents the truly allocable stock.
- **Warehouse Stock Reservations:** A record of quantities that have been implicitly reserved from the total warehouse stock (e.g., for online fulfillment lead time), indicating stock that should not be considered for physical store reallocation.

Business Reports

- **Warehouse Allocable Stock Report:** Used by inventory planners to see the net quantity of each product available for distribution to physical stores, aiding in immediate reallocation decisions.
- **Warehouse Inefficiency Impact Report:** Used by warehouse operations and inventory teams to understand the impact of the inefficiency factor on available stock and to identify potential areas for process improvement.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The module might process with incomplete warehouse stock or incoming shipment data, leading | Verify all raw warehouse data sources are up-to-date and re- |

| Situation | What Happens | What You Should Do |
|---|---|--|
| | to inaccurate available quantities. | run the module once data is complete. |
| Warehouse Inefficiency Factor is too high | A significant portion of actual stock might be consistently excluded, leading to perceived stockouts even when items are physically present. | Review the Warehouse Inefficiency Factor setting and consider warehouse process audits to reduce discrepancies. |
| Incorrect Lead Time for Sales Exclusion | Too much or too little stock might be reserved for online sales, leading to either missed online sales or unnecessary physical store stockouts. | Regularly review and adjust the Lead Time for Sales Exclusion based on current shipping and fulfillment performance. |
| No incoming shipment data | The module will only consider current on-hand warehouse stock, potentially underestimating future availability for reallocation. | Ensure that Purchase Order and In-Transit transfer data feeds are consistently updated and integrated. |

9. Next Steps

Before This Step

- **Reallocation Prepare Data Module:** This module provides the initial, consolidated raw warehouse stock figures, including current on-hand inventory and incoming shipments.
- **Reallocation Prepare Sales Module:** This module provides the cleaned Rate of Sale (ROS) data, which is crucial for calculating lead time sales exclusions for online stores.

After This Step

- **Reallocation Planning Module:** This next module uses the refined, accurate warehouse inventory figures to determine optimal stock movements to stores, maximizing sales potential.
- **Inventory Execution Systems:** Once reallocation decisions are made, these systems receive the instructions to physically move the refined warehouse inventory to various store locations.

10. Common Questions

Q: Why is the available stock in the system different from what my warehouse manager says is physically on the shelves? A: The system applies a “Warehouse Inefficiency Factor” to account for items that are physically present but might not be easily accessible or sellable (e.g., damaged, mislocated). It also considers any stock reserved for upcoming online orders.

Q: How does this module handle new products that haven't sold yet? A: For new products, the "Lead Time for Sales Exclusion" might not apply as there's no historical ROS. However, the "Warehouse Inefficiency Factor" will still be applied, ensuring a conservative and realistic view of available stock.

Q: What if I don't want to reserve any stock for online orders from my warehouse? A: You can adjust the Lead Time for Sales Exclusion setting to 0. However, this might lead to physical stores pulling inventory that is quickly needed for online customers, potentially causing online stockouts.

Q: How do incoming purchase orders (POs) affect the warehouse inventory in this module? A: Incoming POs are typically integrated with the raw warehouse stock *before* this module runs. This module then takes that combined 'current + incoming' total and refines it by applying inefficiency deductions and lead time reservations.

Q: Does this module help prevent stockouts at my stores? A: Yes, by providing a precise and realistic count of *truly available* warehouse stock, it ensures that subsequent reallocation decisions are based on accurate data, preventing over-commitment and helping to allocate stock to where it's most needed, thereby reducing store stockouts.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To refine raw warehouse inventory data into a practical, allocable quantity, considering operational realities and future commitments.
- **Main Input:** Raw warehouse stock (including current on-hand and incoming shipments) and clean Rate of Sale (ROS) data.
- **Main Output:** Refined, allocable warehouse inventory figures per style/SKU.
- **Warning Signs:** Unexplained discrepancies between physical and system stock, or unexpectedly low available warehouse quantities.
- **Key Settings:** Warehouse Inefficiency Factor, Lead Time for Sales Exclusion, Minimum Live Days for ROS Calculation.

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: Reallocation First Month ROS Calculation Module

Last Updated: 2025-05-23

1. Business Objective

This module calculates the Rate of Sale (ROS) specifically for the first month (or a defined initial period) at the store-style level. This initial ROS serves as a baseline for projecting sales for subsequent periods in the depletion process.

Why This Matters: By providing an accurate and filtered initial Rate of Sale, this module ensures that subsequent inventory decisions, especially for newly launched or seasonal products, are based on true demand signals. This leads to more precise initial allocations, minimizes stockouts, and optimizes early-stage inventory distribution, improving overall sales performance.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Transactions:** Individual sales records (including units sold, sale date, store, and product style) from the Reallocation Prepare Sales Module. This provides the essential, filtered data for calculating sales speed within the specific first-month period.

Product Data

- **Product Characteristics (AG) & Product Family Links:** Information about product attributes like color, style, and the relationships between individual items (SKUs) and broader product styles. This is necessary to aggregate sales data to the style level for accurate ROS calculation.
- **Active Season List:** A comprehensive list of product styles or categories designated as 'in-season' for the current reallocation cycle. This ensures that only sales of relevant, active products contribute to the Rate of Sale calculation.

Operational Data

- **Enabled Stores List:** A predefined list of stores actively participating in the reallocation process. This ensures that ROS calculations are performed only for locations relevant to current inventory strategies.
- **Analysis Date:** The specified end date for the sales analysis period. This date, combined with a defined period length, establishes the precise “first month” or initial window for which ROS is calculated.
- **Product Set Available (PSA) Status:** Data indicating the percentage of a product’s full range (e.g., its Ideal Size Set or other crucial attributes) that was available on a given day. This is vital for filtering out sales data from periods where stock was inadequate, ensuring the ROS reflects true demand, not just sales limited by availability.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------|---|---|
| First Month Sales Period Length | 30 days | Defines the number of historical days used to calculate the “first month” Rate of Sale. | To adjust how recent or broad the initial sales analysis should be. |
| Minimum Live Selling Days | 7 | The minimum number of days a product or style must have been consistently available and selling to have its ROS calculated. | To prevent highly volatile or unreliable ROS calculations for products with very short or inconsistent sales histories. |
| Product Availability Benchmark (PSA) | 0.80 | The minimum percentage of a product’s optimal assortment (e.g., Ideal Size Set) that must have been available for sales on a given day to be included in ROS calculation. | If product availability is frequently inconsistent, or if sales patterns are often skewed by stockouts. |

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Product Data] --> B[Filter Sales by Season & Product Availability]

B --> C[Calculate First Month ROS (Store–Style Level)]

C --> D[First Month ROS for Initial Allocation]

1. Start with → Cleaned historical sales data for all products and stores, alongside product and store master data.

2. Then → **Filter out** sales transactions for products that are out of season or for days where product availability (PSA) was below the defined benchmark.
3. Next → **Calculate** the Rate of Sale (ROS) for each product style at each store based on the remaining, relevant sales data from the defined “first month” period.
4. Finally → Produce the calculated First Month ROS figures for each store-style combination, ready for initial allocation and future sales projections.

5. Step-by-Step Actions

1. **Identify** the sales data relevant to the “first month” period
 - What we do: *We pinpoint all sales transactions that occurred within the specified initial period (e.g., the last 30 days) for all active stores and products.*
 - Why we do it: *This sets the precise timeframe for calculating the initial Rate of Sale, ensuring the most current and relevant data is used.*
2. **Filter** sales based on active seasons
 - What we do: *We review the identified sales data and exclude any sales that occurred for products or styles currently marked as ‘out-of-season’ for their respective channels.*
 - Why we do it: *This ensures that the calculated ROS reflects true demand for currently active inventory, preventing misleading signals from past-season items.*
3. **Filter** sales based on Product Set Available (PSA) benchmark
 - What we do: *For each store-style-day combination, we check if the Product Set Available (PSA) status met the defined benchmark. Sales on days where the PSA was below the threshold are excluded from the calculation.*
 - Why we do it: *This critical step ensures that the ROS accurately reflects demand when sufficient product (e.g., the Ideal Size Set) was actually available, preventing stockouts from artificially suppressing the apparent sales rate.*
4. **Calculate** the Rate of Sale (ROS) at the store-style level
 - What we do: *For each store and product style that meets the Minimum Live Selling Days criteria, we compute its Rate of Sale by dividing the total units sold during the filtered “first month” period by the number of qualifying selling days.*
 - Why we do it: *This provides a clear, reliable measure of how quickly each style is selling in a specific store, crucial for forecasting and initial inventory placement.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A new “Women’s Floral Blouse - Style C” was introduced 30 days ago.
- Over the last 30 days, Store A sold 45 units of Style C.
- However, for 5 of those days, Store A had less than 60% of the Ideal Size Set (ISS) for Style C available due to early stockouts.
- The First Month Sales Period Length is set to 30 days.

- The Minimum Live Selling Days is 7 days.
- The Product Availability Benchmark (PSA) is set to 70%.
- “Women’s Floral Blouse - Style C” is currently marked as ‘in-season’.

What Happens

1. First, the module **identifies** all sales for Style C at Store A over the past 30 days.
2. It confirms that Style C is ‘**in-season**’, so its sales are considered.
3. Next, for each of the 30 days, it checks the Product Availability Benchmark (PSA). Since the PSA benchmark is 70%, the 5 days where availability was less than 60% are **excluded** from the calculation.
4. This means sales for Style C are now considered over 25 *reliable* selling days (30 total days - 5 broken days). Let’s say 40 units were sold during these 25 reliable days.
5. Finally, the module **calculates** the First Month ROS: 40 units / 25 reliable days = 1.6 units/day for “Women’s Floral Blouse - Style C” at Store A.

Final Outcome

Store A has a precise First Month ROS of 1.6 units/day for “Women’s Floral Blouse - Style C”. This ROS is a true reflection of customer demand, as it excludes sales days when the store couldn’t fully meet demand due to incomplete stock, providing a solid baseline for future allocation and replenishment.

Key Takeaway: This module provides a highly accurate initial Rate of Sale by accounting for product availability and seasonality, ensuring that inventory decisions are based on true sales potential.

7. Results

This step produces:

Main Outputs

- **First Month Rate of Sale (ROS) by Store and Style:** A computed metric for each product style in every enabled store, representing its true selling speed during its initial sales period. This is essential for projecting future demand and making initial inventory allocations.

Business Reports

- **Initial Product Performance Report:** Used by merchandising and planning teams to quickly assess the early selling performance of new or seasonal styles in different stores, guiding initial allocation adjustments.
- **Product Availability Impact Report:** Used by inventory planners and operations teams to understand how often product availability benchmarks are not met, highlighting potential issues with replenishment or Ideal Size Set configurations.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | The module might process with incomplete sales data, leading to inaccurate ROS calculations and skewed demand signals. | Verify all raw sales data sources are up-to-date and re-run the module once data is complete. |
| Excessive sales filtered by PSA | A large percentage of initial sales data might be filtered out, potentially leading to insufficient data for ROS calculation for many styles. | Review store operational practices (e.g., initial stock levels, replenishment frequency) and re-evaluate the Product Availability Benchmark (PSA) setting. |
| Product doesn't meet Minimum Live Selling Days | The module will not calculate a First Month ROS for styles that haven't been actively available for the defined minimum number of days. | This is expected for very new products. Wait for more selling days or adjust the Minimum Live Selling Days setting if necessary for immediate action. |
| All products marked 'out-of-season' | No product sales will be considered for ROS calculation if the Active Season List is misconfigured. | Review the Active Season List to ensure it accurately reflects current active product seasons. |

9. Next Steps

Before This Step

- **Reallocation Prepare Data Module:** This module provides the foundational product (characteristics, seasonality) and store (enabled list) data, which define the scope for ROS calculation.
- **Reallocation Prepare Sales Module:** This module delivers the cleaned historical sales transactions, essential as the primary input for determining the First Month ROS.

After This Step

- **Reallocation Planning Module:** This subsequent module utilizes the precise First Month ROS calculated here to determine optimal initial stock levels and allocate inventory effectively to stores, especially for new or seasonal product launches.
- **Demand Forecasting Module:** The accurate and filtered First Month ROS data serves as a crucial input for more advanced demand forecasting models, helping predict future sales for ongoing replenishment and buying decisions.

10. Common Questions

Q: Why calculate Rate of Sale specifically for the “first month” or initial period? A: Calculating ROS for the initial period provides a crucial early indicator of a product’s actual demand when it’s new or seasonal. It helps determine if the initial inventory allocation was sufficient and guides immediate adjustments for effective stock distribution.

Q: How does the Product Availability Benchmark (PSA) affect the calculated ROS? A: The PSA benchmark ensures that the calculated ROS reflects true customer demand, not just sales limited by stockouts. By excluding days where insufficient product was available, the system gets a more accurate measure of what *could have sold* if stock was optimal.

Q: What happens if a new product doesn’t have enough sales days to meet the Minimum Live Selling Days? A: If a product doesn’t meet the Minimum Live Selling Days, this module will not calculate an initial ROS for it. This prevents unreliable forecasts based on very limited data. Planners would typically rely on initial allocation logic or manual overrides until sufficient sales data accumulates.

Q: How does this module help with initial inventory placement for new products? A: By providing a precise First Month ROS, the module enables planners to quickly identify which new products are selling faster than expected and in which stores. This information directly informs rapid reallocation decisions, ensuring popular items are quickly moved to high-demand locations.

Q: Why are ‘out-of-season’ sales excluded from this ROS calculation? A: Excluding ‘out-of-season’ sales ensures that the calculated ROS reflects current, active demand. Including sales from past seasons could skew the perception of a product’s current performance, leading to misinformed inventory decisions for active stock.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate a precise, filtered Rate of Sale for the initial sales period of products at the store-style level.
- **Main Input:** Cleaned sales transactions, product characteristics, active season lists, and product availability status.
- **Main Output:** First Month Rate of Sale (ROS) by store and style.
- **Warning Signs:** Unusually low overall ROS figures, or a significant portion of sales being filtered out by the Product Availability Benchmark (PSA).
- **Key Settings:** First Month Sales Period Length, Minimum Live Selling Days, Product Availability Benchmark (PSA).

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: Reallocation Sales Calculation Module

Last Updated: 2025-05-23

1. Business Objective

This module computes the total sales quantity in units for each store-category-period combination.

Why This Matters: The calculated sales quantities are crucial for accurately projecting future demand and guiding allocation decisions, especially in inventory depletion scenarios, ensuring that inventory is moved where it will sell most effectively.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Store Revenue Contribution from Optimal Depth (OD) Segmentation:** This data, output from the Optimal Depth (OD) segmentation module, indicates each Attribute Group's (AG) proportional share of a store's total revenue, guiding revenue-driven allocation.
- **Average Selling Price (ASP) at Attribute Group (AG) Level:** Knowing the average price for each Attribute Group is essential to convert projected revenue contributions into actionable sales quantities (units).
- **Annual Operating Plan (AOP) Revenue Targets:** These are the overall financial targets for sales, providing the baseline against which all sales quantities are projected, ensuring alignment with top-level business goals.

Product Data

- **Attribute Group (AG) to Category & Style Links:** Information about product attributes and how individual product styles link to broader Attribute Groups and categories. This helps aggregate sales projections to the appropriate planning levels.
- **Product Styles Eligible for Rate of Sale (ROS) Calculation:** This specifies which product styles are actively selling and should be considered when determining which Attribute Groups to project sales quantities for.

Operational Data

- **Enabled Stores List:** A definitive list of stores actively participating in the reallocation process, ensuring that sales quantity calculations are performed only for relevant locations.
- **Analysis Periods:** Defines the specific timeframes (e.g., future weeks or months) for which sales quantities need to be projected, supporting forward-looking inventory decisions and depletion planning.

3. Business Settings

This module primarily utilizes outputs from other modules that have their own configurable settings. There are no direct business settings within this specific module. For details on settings that influence the inputs to this module (such as how Average Selling Price or Optimal Depth segments are determined), please refer to the documentation for the Optimal Depth (OD) Segmentation Module and the Reallocation Prepare Sales Module.

Note: See More Details for additional settings in dependent modules.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Optimal Depth (OD) Segmentation Output & AOP Revenue  
Targets] --> B[Calculate Store-Category Sales Quantity]  
B --> C[Total Sales Quantity per Store-Category-Period]
```

1. Start with → Output from the Optimal Depth (OD) segmentation module (which includes store revenue contributions and AG-level ASP) and Annual Operating Plan (AOP) revenue targets.
2. Then → **Calculate** the total sales quantity for each store-category-period by converting revenue contributions and AOP targets into units, using the Average Selling Price (ASP) for each Attribute Group (AG).
3. Finally → Produce precise sales quantity projections for each store-category-period, ready for inventory allocation.

5. Step-by-Step Actions

1. **Retrieve** segment-specific revenue contributions and AOP targets
 - What we do: *We gather the proportional share of total revenue that each Attribute Group (AG) is expected to contribute within a specific store and period, directly from the Optimal Depth (OD) segmentation module. We also pull the overall Annual Operating Plan (AOP) revenue targets.*
 - Why we do it: *These inputs provide the financial goals and the strategic distribution of expected sales across different product segments, which are essential for accurately projecting unit demand.*

2. **Obtain** Average Selling Prices (ASP) for each Attribute Group (AG)
 - What we do: *We retrieve the calculated Average Selling Price (ASP) for each Attribute Group (AG) for the relevant periods and stores.*
 - Why we do it: *This crucial step allows us to convert the projected revenue figures (from AOP and OD contributions) into actual unit sales quantities, which is the actionable metric needed for inventory planning and reallocation.*
3. **Compute** the total sales quantity for each store-category-period
 - What we do: *For every active store, product category, and future period, we apply a formula: (Annual Operating Plan Revenue Target for the store-period Attribute Group's Revenue Contribution) / Attribute Group's Average Selling Price. The resulting quantities for individual Attribute Groups are then summed up to provide a total sales quantity for their respective broader categories within that store and period.**
 - Why we do it: *This calculation provides a comprehensive and precise projection of unit sales by product category for each store, forming the fundamental basis for future demand planning and highly targeted inventory allocation decisions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- For Store A, the Annual Operating Plan (AOP) revenue target for the upcoming month (Period 1) is \$100,000.
- The Optimal Depth (OD) segmentation module indicates that a specific Attribute Group (AG), "Women's Floral Blouses," is projected to contribute 5% (0.05) of Store A's total revenue for Period 1.
- The Average Selling Price (ASP) for "Women's Floral Blouses" in Period 1 for Store A is \$25.
- "Women's Floral Blouses" belongs to the "Women's Tops" category.

What Happens

1. First, the module retrieves the AOP revenue target for Store A for Period 1 (\$100,000).
2. It then identifies the revenue contribution of "Women's Floral Blouses" within Store A for Period 1, which is 5% of the AOP. So, \$100,000 multiplied by 0.05 equals \$5,000 in projected revenue for this Attribute Group.
3. Next, it retrieves the Average Selling Price (ASP) for "Women's Floral Blouses," which is \$25.
4. The module then calculates the sales quantity for "Women's Floral Blouses" by dividing the projected revenue by the ASP: \$5,000 divided by \$25 equals 200 units.
5. This calculated quantity (200 units) is then attributed to the broader "Women's Tops" category for Store A in Period 1. If there were other Attribute Groups also contributing to the "Women's Tops" category, their calculated sales quantities would be summed together into this category total.

Final Outcome

The system precisely projects that Store A needs to sell 200 units of “Women’s Floral Blouses” as part of its “Women’s Tops” category for Period 1. This projection is based on the store’s overall revenue targets and the strategic importance of this product segment, providing a clear unit demand signal for inventory planning.

Key Takeaway: This module translates financial targets and strategic Optimal Depth (OD) segmentation insights into concrete unit sales projections, enabling precise inventory planning and efficient stock allocation.

7. Results

This step produces:

Main Outputs

- **Total Sales Quantity per Store-Category-Period:** This is the primary output, representing the projected number of units expected to sell for each product category within every store for defined future periods. This forms the fundamental basis for all subsequent inventory depletion and reallocation plans.

Business Reports

- **Projected Category Sales Report:** Used by merchandising and inventory planning teams to understand future unit demand by category and store. This report directly guides decisions on where to allocate existing inventory and how to plan for future replenishments.
- **Demand Forecast Validation Report:** Used by analytics and finance teams to compare the calculated sales quantities against historical performance and Annual Operating Plan (AOP) targets, ensuring the accuracy and reasonableness of the demand projections.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|--|
| Missing or delayed data feed | The module might process with incomplete Annual Operating Plan (AOP) revenue targets, Optimal Depth (OD) segmentation output, or Average Selling Price (ASP) data, leading to inaccurate sales quantity projections. | Verify all upstream data sources (AOP, OD outputs, ASP) are up-to-date and re-run the module once data is complete. |
| Zero or extremely low Average Selling Price (ASP) for an | If the ASP is zero or very close to zero for any Attribute Group, the calculation will result in an error or an unrealistically high sales quantity, skewing demand signals. | Review the ASP data source for accuracy and ensure all relevant Attribute Groups have a valid average selling price defined. |

| Situation | What Happens | What You Should Do |
|--|---|--|
| Attribute Group (AG) | | |
| Zero or extremely low Store Revenue Contribution for an Attribute Group (AG) | If an Attribute Group's projected revenue contribution from the Optimal Depth (OD) segmentation is zero, the calculated sales quantity for that AG will also be zero, even if there's actual market demand. | Review the Optimal Depth (OD) segmentation rules and outputs to ensure they accurately reflect expected sales for all relevant Attribute Groups. |
| Annual Operating Plan (AOP) revenue target is missing or zero | The module will calculate zero sales quantities for the affected stores or periods, as there's no overall revenue target to base the unit projections on. | Ensure that all AOP revenue targets are correctly loaded and available for all relevant stores and future periods. |

9. Next Steps

Before This Step

- **Optimal Depth (OD) Segmentation Module:** This module provides the crucial finalStoreRevContri for each Attribute Group and the Average Selling Price (ASP) data, which are direct inputs for this sales calculation.
- **Annual Operating Plan (AOP) Data Integration:** Ensures that the overall AOP revenue targets are loaded and available for the module to use as a foundational basis for its sales quantity projections.

After This Step

- **Reallocation Planning Module:** This subsequent module will utilize the precise total sales quantities per store-category-period calculated here to develop optimal inventory depletion and reallocation plans, ensuring stock moves to where demand is highest.
- **Demand Forecasting & Buying Decisions:** The sales quantities projected by this module serve as fundamental inputs for broader demand forecasting efforts and directly inform future product buying decisions, aligning purchases with projected demand.

10. Common Questions

Q: Why is this module needed if we already have Rate of Sale (ROS) from other modules? A: While ROS tells you how *fast* an item sells when available, this module translates high-level financial goals (AOP revenue targets and OD revenue contributions) into total *units* expected to sell over a future period at the store-category level. This is crucial for depletion planning and understanding overall demand.

Q: How does the “Optimal Depth (OD) Segmentation” output affect the sales quantities? A: The `finalStoreRevContri` from OD segmentation determines the proportional share of a store’s overall revenue target that is attributed to specific Attribute Groups. This ensures that the calculated sales quantities for each category align with strategic depth and merchandising priorities.

Q: What if my Annual Operating Plan (AOP) targets are not precise? A: The accuracy of the calculated sales quantities is directly dependent on the precision of the AOP revenue targets. Inaccurate AOP can lead to over or under-projected sales, which will directly impact the effectiveness and profitability of subsequent allocation and replenishment decisions.

Q: Can this module project sales for new categories or stores without historical sales data? A: Yes, this module can project sales for new entities. It relies on Annual Operating Plan (AOP) targets and Optimal Depth (OD) segmentation outputs. If these upstream inputs are properly configured for new categories or stores (e.g., using proxies or planned values from similar locations), then this module can still provide relevant sales quantity projections.

Q: Why are sales quantities calculated at the category level and not just individual items (SKUs)? A: Sales quantities are often projected at the category or Attribute Group level because initial inventory planning and strategic allocation decisions are frequently made at this aggregated level. This provides a clear, high-level overview of demand, which then guides more granular SKU-level actions.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To compute total sales quantity projections for store-category-period combinations based on financial plans and strategic segmentation.
- **Main Input:** Optimal Depth (OD) segmentation output (store revenue contributions, AG-level Average Selling Price - ASP) and Annual Operating Plan (AOP) revenue targets.
- **Main Output:** Total Sales Quantity (units) per store-category-period.
- **Warning Signs:** Unusually high or low projected sales quantities, or errors resulting from missing or zero input data (AOP, ASP, OD contributions).
- **Key Settings:** *None directly in this module, but heavily influenced by settings in upstream modules like Optimal Depth (OD) Segmentation and AOP planning.*

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 7: Reallocation Period ROS Calculation Module

Last Updated: 2025-05-23

1. Business Objective

This module calculates the Rate of Sale (ROS) at the individual item (SKU) level for all defined future periods, providing precise demand signals for inventory reallocation.

Why This Matters: Accurate SKU-level ROS is essential for ensuring that every specific product, down to its size and color, is allocated to the right store at the right time. This precision maximizes sales potential, reduces stockouts of popular items, minimizes overstock of slow-moving inventory, and ultimately optimizes inventory efficiency and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Style-level Rate of Sale (ROS) for the First Month:** This is the baseline selling speed for each product style from the “Reallocation First Month ROS Calculation Module,” which is then projected across all future periods.
- **Sales Quantity per Store, Category, and Period:** Unit sales projections for each product category within every store and defined future period, provided by the “Reallocation Sales Calculation Module.” This helps account for overall category demand trends over time.

Product Data

- **Ideal Size Set (ISS) Output (Adjusted Size Contribution):** The recommended combination of sizes for a product, including their “adjusted size contribution,” which indicates the proportional sales weight of each size within a style. This is crucial for distributing the style’s ROS down to individual SKUs.
- **Product Family Links (SKU to Style):** Information that links individual items (SKUs) to their broader product styles, enabling the module to apply style-level ROS to specific SKUs.

- **Product Characteristics (Attribute Group - AG):** Information that groups product styles into attribute groups, used in conjunction with the Ideal Size Set (ISS) for accurate size distribution.

Operational Data

- **Defined Analysis Periods:** The specific future timeframes (e.g., weeks or months) for which the SKU-level Rate of Sale needs to be calculated, aligning with overall planning horizons.
- **New Store Reference Map:** A mapping of new stores (with limited sales history) to established “reference” stores, allowing the system to proxy sales data for effective initial allocation.

3. Business Settings

This module primarily utilizes outputs from other modules that have their own configurable settings. There are no direct business settings within this specific module that directly influence the core ROS calculation for all SKUs and periods. For details on settings that influence the inputs to this module (such as how the “First Month ROS” or “Sales Quantity” are determined), please refer to the documentation for the “Reallocation First Month ROS Calculation Module” and the “Reallocation Sales Calculation Module.”

Note: See More Details for additional settings in dependent modules.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Style-level First Month ROS & Sales Quantity Projections] --> B[Compute Style ROS for All Periods]
    B --> C[Distribute Style ROS to SKU ROS using ISS]
    C --> D[Normalize SKU ROS]
    D --> E[SKU-level Rate of Sale (ROS) for All Periods]
  
```

1. Start with → The calculated “First Month Rate of Sale (ROS)” for each product style and the projected “Sales Quantity” for each store-category-period.
2. Then → **Project** the style-level ROS to all defined future periods, adjusting for expected category-level sales trends.
3. Next → **Distribute** the calculated style-level ROS among the individual SKUs within that style, using their “adjusted size contribution” from the Ideal Size Set (ISS).
4. Next → **Normalize** the individual SKU ROS values to ensure their sum exactly matches the overall style ROS for each period.
5. Finally → Produce precise SKU-level Rate of Sale (ROS) values for all items in all stores for all future periods, ready for reallocation.

5. Step-by-Step Actions

1. **Compute** style-level ROS for all future periods
 - What we do: *We project the initial selling speed (First Month ROS) of each product style across all future analysis periods. This projection factors in the ratio of anticipated sales quantities for future periods compared to the first month, adjusting the daily sales rate accordingly.*
 - Why we do it: *This provides a forward-looking sales rate for each product style, essential for planning inventory depletion and ensuring that the overall style demand is accurately represented throughout the allocation horizon.*
2. **Distribute** style ROS to individual SKUs
 - What we do: *We break down the calculated style-level ROS into specific SKU-level ROS values. This is achieved by multiplying the style ROS by the “adjusted size contribution” for each SKU (e.g., specific sizes, colors) as defined in the Ideal Size Set (ISS) output.*
 - Why we do it: *This ensures that the projected sales rates for individual items accurately reflect their historical contribution to the style’s sales, allowing for granular inventory decisions.*
3. **Normalize** SKU ROS values
 - What we do: *We adjust the individual SKU ROS values proportionally to ensure that when summed up, they precisely equal the overall style ROS for that period. This corrects for any minor discrepancies that might arise during the distribution step.*
 - Why we do it: *This maintains consistency and accuracy in demand forecasting, preventing over- or under-projection of total style demand while still providing granular SKU-level insights crucial for precise allocations.*
4. **Adjust** ROS for new stores (if applicable)
 - What we do: *For new stores that may not have sufficient historical sales data, we assign SKU ROS values by copying them from a designated “reference” store that has similar characteristics or demand patterns.*
 - Why we do it: *This ensures that even new locations have a realistic demand signal for inventory planning from day one, avoiding stockouts or overstock due to a lack of historical data for proper forecasting.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- **Store A** has a “Women’s Denim Jeans - Style X” with a calculated **First Month ROS of 10 units/day**.
- The “Reallocation Sales Calculation Module” projects that sales for “Women’s Denim Jeans” in **Period 2** will be 80% of what they were in Period 1 (e.g., due to seasonality).
- “Style X” consists of three SKUs: **Size 28 (S1), Size 30 (S2), and Size 32 (S3)**.
- The **Ideal Size Set (ISS) output** for “Women’s Denim Jeans” at Store A shows the following “adjusted size contributions”: S1 = 20%, S2 = 50%, S3 = 30%.

What Happens

1. First, the module calculates the **Style-level ROS for Period 2**. Since Period 2 sales are projected to be 80% of Period 1, the Period 2 ROS for Style X will be 80% of 10 units/day, which is **8 units/day**.
2. Next, the module **distributes** the Period 1 (first month) Style ROS (10 units/day) to the individual SKUs based on their ISS adjusted contributions:
 - SKU S1 (Size 28): $10 \text{ units/day} * 0.20 = \mathbf{2.0 \text{ units/day}}$
 - SKU S2 (Size 30): $10 \text{ units/day} * 0.50 = \mathbf{5.0 \text{ units/day}}$
 - SKU S3 (Size 32): $10 \text{ units/day} * 0.30 = \mathbf{3.0 \text{ units/day}}$ *(Note: These sum up to 10.0 units/day, matching the style ROS for Period 1.)*
3. Then, the module **distributes** the Period 2 Style ROS (8 units/day) to the individual SKUs using the same ISS adjusted contributions:
 - SKU S1 (Size 28): $8 \text{ units/day} * 0.20 = \mathbf{1.6 \text{ units/day}}$
 - SKU S2 (Size 30): $8 \text{ units/day} * 0.50 = \mathbf{4.0 \text{ units/day}}$
 - SKU S3 (Size 32): $8 \text{ units/day} * 0.30 = \mathbf{2.4 \text{ units/day}}$ *(Note: These sum up to 8.0 units/day, matching the style ROS for Period 2, ensuring normalization.)*

Final Outcome

The system produces precise SKU-level ROS values for Style X in Store A for both Period 1 and Period 2. For example, it knows Size 30 (SKU S2) is expected to sell 5.0 units/day in Period 1 and 4.0 units/day in Period 2. This granular understanding of demand allows for highly targeted inventory reallocation, ensuring the right sizes are in stock at the right time, minimizing lost sales due to size unavailability.

Key Takeaway: This module accurately translates high-level style demand into granular, period-specific SKU demand, ensuring optimal inventory distribution down to the individual size.

7. Results

This step produces:

Main Outputs

- **SKU-level Rate of Sale (ROS) for All Defined Periods:** This is the primary output: a computed metric for each individual item (SKU) in every store, representing its precise selling speed across all specified future analysis periods. This is foundational for all subsequent inventory allocation and replenishment decisions.

Business Reports

- **SKU-level Demand Forecast Report:** Used by inventory planners and store managers to view the projected daily sales rate for every specific product (SKU) in each store across all future periods. This report directly guides which items should be moved where to meet expected demand.
- **Size Contribution Analysis Report:** Used by merchandising and planning teams to understand how individual sizes are contributing to overall style sales,

helping validate the Ideal Size Set (ISS) effectiveness and inform future buying decisions.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | The module might process with incomplete style-level ROS, sales quantity, or ISS data, leading to inaccurate SKU ROS calculations. | Verify all upstream data sources (“Reallocation First Month ROS,” “Reallocation Sales Calculation,” “Reallocation Prepare Data”) are up-to-date and re-run the module once data is complete. |
| Ideal Size Set (ISS) data missing for a style/category | If ISS data (especially adjusted size contribution) is missing for a specific style, the module might default to zero contribution for all SKUs, leading to zero SKU ROS. | Ensure all relevant product styles and their associated Attribute Groups have complete and accurate Ideal Size Set (ISS) definitions loaded. |
| First Month ROS is zero for a style | If the initial style-level ROS from the “First Month ROS Calculation Module” is zero, all derived SKU ROS values for that style will also be zero for all periods. | Review the upstream “First Month ROS Calculation Module” to understand why initial sales might be missing or filtered out. |
| Style has no SKUs defined | The module cannot distribute ROS if no individual items (SKUs) are linked to a particular style, leading to no SKU-level ROS for that style. | Ensure all product styles have their individual SKUs correctly linked in the product master data. |

9. Next Steps

Before This Step

- **Reallocation First Month ROS Calculation Module:** This module must have already completed, providing the crucial style-level Rate of Sale (ROS) for the initial period, which serves as the foundation for projections.
- **Reallocation Sales Calculation Module:** This module must have completed, providing the projected total sales quantities per store-category-period, which are used to scale the style-level ROS across future periods.
- **Reallocation Prepare Data Module:** This module provides the comprehensive product master data, including Ideal Size Set (ISS) definitions and product family links (SKU to Style), essential for distributing style ROS to SKUs.

After This Step

- **Reallocation Planning Module:** This subsequent module will utilize the precise SKU-level Rate of Sale (ROS) calculated here to determine optimal stock movements, deciding exactly which SKUs should be transferred to which stores to maximize sales and minimize inventory inefficiencies.
- **Demand Forecasting & Replenishment Systems:** The highly granular and accurate SKU-level ROS data serves as a fundamental input for broader demand forecasting models and informs daily or weekly replenishment decisions, ensuring stores receive the right product mix.

10. Common Questions

Q: Why is SKU-level ROS important if we already have style-level ROS? A: While style-level ROS gives an overall demand signal, SKU-level ROS provides the critical detail needed for actual inventory movement. For example, knowing a “T-shirt style” sells well isn’t enough; you need to know which sizes (S, M, L) and colors (red, blue) sell fastest to ensure the right mix is in each store, preventing lost sales from missing popular sizes.

Q: How does the Ideal Size Set (ISS) output affect this calculation? A: The Ideal Size Set (ISS) output, specifically the “adjusted size contribution,” directly guides how the overall style’s sales are distributed among its individual sizes (SKUs). If size Medium contributes 50% to a style’s sales, then 50% of that style’s ROS will be assigned to its Medium SKU, ensuring allocations match demand for specific sizes.

Q: What happens if a new SKU has no sales data yet? A: If a new SKU has no direct sales history, its Rate of Sale will still be calculated by distributing the style’s ROS based on its “adjusted size contribution” from the Ideal Size Set (ISS). This ensures that even brand-new items receive appropriate inventory based on their expected demand within their product family.

Q: How does this module account for seasonality across different periods? A: This module accounts for seasonality by using the period-specific “Sales Quantity” projections from the “Reallocation Sales Calculation Module.” If that module projects lower sales for a category in a future period (e.g., winter vs. summer), this module will proportionally adjust the style-level ROS for that period, which then filters down to the SKU level.

Q: What if the sum of SKU ROS values for a style doesn’t match the overall style ROS? A: The module includes a crucial “normalization” step. After distributing the style ROS to individual SKUs, it re-adjusts all SKU ROS values slightly to ensure that their sum precisely equals the original style ROS. This guarantees accuracy and consistency in the demand signals.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate a precise, period-specific Rate of Sale (ROS) for individual items (SKUs) at the store level.

- **Main Input:** Style-level First Month ROS, Sales Quantity projections by store-category-period, and Ideal Size Set (ISS) adjusted size contributions.
 - **Main Output:** SKU-level Rate of Sale (ROS) for all defined future periods.
 - **Warning Signs:** Unexpectedly zero SKU ROS values, or a significant mismatch between expected and calculated SKU selling speeds.
 - **Key Settings:** *None directly in this module, but heavily influenced by settings in upstream modules like “Reallocation First Month ROS Calculation Module” and “Reallocation Sales Calculation Module.”*
-

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 8: Reallocation Compute Revenue Module

Last Updated: 2025-05-23

1. Business Objective

This module calculates various “Revenue Per Day” metrics at different aggregation levels, including individual store-style combinations, national totals, and rolled-up Attribute Groups (AGs). These metrics are vital for ranking styles and making precise inventory allocation decisions. It processes sales data to determine revenue contribution, handles new stores by replicating revenue from similar established stores, and identifies “read-only” store-styles based on season flags. Additionally, it calculates the Ideal Size Set (ISS) depth for store-styles based on how much each size typically sells.

Why This Matters: By providing accurate and diverse “Revenue Per Day” insights, this module empowers inventory planners to make informed decisions that maximize sales potential and minimize stockouts. It ensures that inventory is strategically placed where it will generate the most revenue, whether for established products, new launches, or for managing end-of-season merchandise.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data:** This provides the filtered historical sales records, including units sold and average selling price, which are essential for calculating actual revenue generated.
- **Priority Sales Data:** Specific sales transactions flagged for priority periods (e.g., peak season) used for aggregated revenue analysis, ensuring special periods are accurately represented.

Product Data

- **Product Characteristics (AG):** Information like color, style, and brand, used to group products into Attribute Groups for aggregated revenue analysis.
- **Ideal Size Set (ISS) Definitions:** Pre-defined optimal combinations of sizes for products, crucial for determining the proportional sales contribution of each size and calculating Ideal Size Depth.

- **Product Family Links (SKU to Style):** Information linking individual items (SKUs) to their broader product styles, enabling revenue analysis at the style level.

Operational Data

- **Enabled Stores List:** A definitive list of stores actively participating in the reallocation process, ensuring calculations are focused on relevant locations.
- **New Store Reference Map:** A mapping of new stores (with limited or no sales history) to established “reference” stores, allowing the module to infer sales performance for initial allocation.
- **Seasonality Flags:** Indicators that categorize products by season (e.g., ‘Summer 2025’), which are used to identify “read-only” merchandise and manage active vs. inactive inventory.
- **Analysis Period:** The specific timeframe (e.g., last 30 days) for which initial “Revenue Per Day” is calculated, ensuring recency and relevance of demand signals.
- **Product Availability Benchmark:** A threshold indicating the minimum percentage of a product’s Ideal Size Set (ISS) that must be available for sales data to be considered reliable, preventing misinterpretation of demand due to stockouts.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------|--|---|
| Initial Sales Analysis Period Length | 30 days | Defines the number of historical days used to calculate the initial “Revenue Per Day” for store-styles. | To adjust how recent or broad the initial sales analysis should be. |
| Product Availability Benchmark | 0.80 | Minimum percentage of Ideal Size Set (ISS) products that must be available for sales data to be considered “unbroken.” | If product availability is frequently inconsistent, or if sales patterns are often skewed by stockouts. |
| Read-Only Season List | (Empty) | A comma-separated list of product seasons to be flagged as “read-only” (not actively reallocated). | At the end of a retail season for clearance products. |
| Replenishment Season List | (Empty) | A comma-separated list of product seasons that are eligible for active replenishment and reallocation. | At the start of a new retail season to define active merchandise. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Product Info] --> B[Compute Revenue Per Day at Store & National Levels]

B --> C[Derive Revenue Per Day for Aggregated Groups]

C --> D[Replicate for New Stores & Flag Read-Only Styles]

D --> E[Final Revenue Metrics & Size Depth Calculations]

1. Start with → Cleaned historical sales data, comprehensive product details, and store operational information.
2. Then → **Calculate** the “Revenue Per Day” for individual product styles at each store and also nationally, ensuring data reflects periods of adequate product availability.
3. Next → **Derive** “Revenue Per Day” for broader Attribute Groups, using aggregated sales data to provide demand signals even for less-detailed product segments.
4. Next → **Replicate** “Revenue Per Day” metrics for new stores from their designated reference stores and **identify** which store-styles are “read-only” and should not be actively reallocated.
5. Finally → **Produce** the precise “Revenue Per Day” metrics at various levels, determine the Ideal Size Depth for each store-style, and provide clear flags for allocation guidance.

5. Step-by-Step Actions

1. **Calculate** store-style revenue per day for initial period
 - What we do: *We determine the average daily revenue generated by each product style within a specific store during a defined initial sales period (e.g., the last 30 days), taking into account only days when stock was adequately available based on the Product Availability Benchmark.*
 - Why we do it: *This establishes a baseline for how well a style sells in a particular store under normal conditions, crucial for initial allocation accuracy and identifying top performers.*
2. **Compute** national revenue per day for each style
 - What we do: *We calculate the average daily revenue generated by each product style across all stores nationally, again considering only periods of sufficient stock availability across the entire network.*
 - Why we do it: *This provides a broad view of a style's overall market performance, useful for high-level inventory planning, strategic buying decisions, and comparing performance across different regions.*
3. **Derive** revenue per day for rolled-up Attribute Groups
 - What we do: *We aggregate sales data to calculate the average daily revenue for broader Attribute Groups (like “Women's Casual Tops” or “Men's Athletic Footwear”) at various levels of detail, from store-category to national brand segment. This allows for demand signals across different product grouping levels.*
 - Why we do it: *This helps in forecasting demand and making allocation decisions for product groups that may not have direct granular sales history*

or for styles lacking sufficient individual data, providing a more robust and reliable demand signal for inventory planning.

4. Assign revenue per day for store-styles lacking initial sales data

- *What we do: For store-styles that didn't have sales during the initial period, we assign them a calculated "Revenue Per Day" based on their rolled-up Attribute Group's performance. The system progressively looks at broader Attribute Group levels until a suitable revenue per day is found.*
- *Why we do it: This ensures that all relevant store-style combinations, even those with limited sales history or new product launches, receive a sensible revenue expectation, preventing them from being overlooked or mismanaged in reallocation.*

5. Replicate revenue for new stores

- *What we do: For newly opened stores with no historical sales data, we copy the calculated "Revenue Per Day" metrics from designated "reference" stores. These reference stores have similar customer demographics, store sizes, or market characteristics to the new location.*
- *Why we do it: This allows new stores to receive optimal initial inventory based on proven demand patterns from comparable, established locations, avoiding early stockouts or overstocking and ensuring a strong launch.*

6. Identify "read-only" store-styles

- *What we do: We flag specific product styles in certain stores as "read-only" if they belong to seasons designated for clearance or minimal replenishment. This status is determined by specific season lists (e.g., 'Winter 2024') and any channel-specific override rules.*
- *Why we do it: This prevents the system from actively reallocating or replenishing inventory for products that are being phased out or are not part of current sales strategies, ensuring that valuable stock and planning efforts are focused on active, high-potential merchandise.*

7. Calculate Ideal Size Depth for store-styles

- *What we do: For each store-style combination, we determine the ideal number of units to stock for each individual size (SKU) within that style. This is based on the store's overall Ideal Size Set (ISS) definitions and the proportional sales contribution of each size (e.g., how much more a Medium sells than a Small for that style).*
- *Why we do it: This ensures that stores receive the optimal mix of sizes for each product, aligning inventory precisely with customer demand for specific sizes. This minimizes stockouts of popular sizes, reduces excess inventory of slow-moving sizes, and ultimately boosts sales and customer satisfaction.*

6. Sample Scenario

Let's consider a popular "Summer Floral Dress" (Style S) and a new "Urban Chic Jacket" (Style J).

Starting Point

- **Store A** has been selling "Summer Floral Dress" (Style S) for 60 days. Its cleaned sales data shows it sold 120 units, and it was consistently available for 50 of those days (meeting the Product Availability Benchmark). The Average Selling Price (ASP) for Style S is \$50.

- **Store B** is a new store that opened last week and has no sales history. It's mapped to **Store A** as its reference store.
- “Summer Floral Dress” (Style S) is part of the ‘Summer 2025’ season, which is *not* on the Read-Only Season List.
- “Urban Chic Jacket” (Style J) belongs to the ‘Winter 2024’ season, which *is* on the Read-Only Season List.
- For Style S, the Ideal Size Set (ISS) indicates that size Medium typically contributes 40% of sales, Large 30%, Small 20%, and Extra Large 10%. The target Ideal Size Set (ISS) depth for ‘Dresses’ in Store A is 10 units.

What Happens

1. **Calculate Store-Style Revenue Per Day (Level 1):** For Style S at Store A, the module first calculates total revenue ($\$50 \text{ ASP} * 120 \text{ units} = \$6,000$). Divided by the 50 reliable selling days, the **Revenue Per Day for Style S at Store A is \$120/day**.
2. **Identify Read-Only Styles:** The module checks the season for Style J. Since ‘Winter 2024’ is on the Read-Only Season List, **Style J at all stores is flagged as “read-only,”** meaning it won’t be actively reallocated or replenished. Style S is not flagged because ‘Summer 2025’ is an active season.
3. **Replicate for New Stores:** For Store B, since it’s a new store and mapped to Store A, the module **replicates the Revenue Per Day of \$120/day for Style S from Store A to Store B**. Store B now has a reliable projected sales performance for Style S, despite having no historical sales.
4. **Calculate Ideal Size Depth:** For Style S at Store A, using the overall Ideal Size Set (ISS) depth of 10 units and the proportional sales contributions:
 - Size Medium: $10 \text{ units} * 0.40 = 4 \text{ units}$
 - Size Large: $10 \text{ units} * 0.30 = 3 \text{ units}$
 - Size Small: $10 \text{ units} * 0.20 = 2 \text{ units}$
 - Size Extra Large: $10 \text{ units} * 0.10 = 1 \text{ unit}$
 The module assigns these as the **ideal number of units for each size (Ideal Size Depth) for Style S at Store A**.

Final Outcome

Store A has a clear Revenue Per Day of \$120 for Style S, with a precise Ideal Size Depth of 4 Mediums, 3 Large, 2 Small, and 1 Extra Large. Store B, despite being new, also has a reliable Revenue Per Day of \$120 for Style S, allowing for optimal initial stock placement. Style J is correctly identified as “read-only,” preventing unnecessary inventory movements for end-of-season merchandise.

Key Takeaway: This module provides crucial, granular insights into product performance, enabling precise inventory allocation decisions for active styles, including new stores, and clear identification of styles needing no active reallocation.

7. Results

This step produces:

Main Outputs

- **Store-Style Revenue Per Day:** The estimated daily revenue contribution for each product style at a given store. This output is critical for ranking styles by their potential profitability and guiding store-level allocation priorities.
- **National Style Revenue Per Day:** The average daily revenue generated by each product style across the entire retail network. This provides an overall performance benchmark for products at a broader strategic level.
- **Rolled-Up Attribute Group Revenue Per Day:** Daily revenue metrics for broader product categories or Attribute Groups (AGs), providing robust demand signals even for styles with limited individual data, allowing for high-level planning.
- **Ideal Size Depth per Store-Style-SKU:** The recommended number of units for each specific size (SKU) within a product style at a particular store. This ensures the optimal size mix is available to maximize sales and minimize stockouts of popular sizes.
- **Read-Only Store-Style Flags:** Identifies specific product styles in stores that should not be actively reallocated or replenished. This guides inventory focus toward active, high-priority merchandise.

Business Reports

- **Store-Style Performance Ranking Report:** Used by inventory planners and store managers to quickly identify top-performing and underperforming styles in specific stores, guiding immediate allocation priorities to maximize in-store sales.
- **New Store Allocation Guidance Report:** Used by merchandising teams to understand the projected demand for new stores based on reference locations, aiding initial inventory planning for successful store launches.
- **Size Mix Optimization Report:** Used by planning and merchandising teams to review and adjust the ideal size combinations stocked in stores, ensuring alignment with customer demand and minimizing lost sales due to improper size availability.
- **Inventory Reallocation Exclusions Report:** Used by operations and planning teams to verify which products are flagged as “read-only” and thus excluded from active reallocation, ensuring adherence to merchandising strategies for clearance or end-of-life inventory.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---------------------------------------|---|---|
| Missing or delayed data feed | The module might process with incomplete sales data or missing product/store information, leading to inaccurate revenue calculations or size depth recommendations. | Verify all data sources are up-to-date and re-run the module once data is complete. |
| No historical sales for a store-style | If a store-style has no sales data or no sales during the initial analysis period, its “Revenue Per Day” might be zero or it might | Review relevant sales data and consider if a proxy reference store is needed or if manual |

| Situation | What Happens | What You Should Do |
|--|---|---|
| | rely entirely on rolled-up Attribute Group averages, which might not be perfectly accurate. | overrides are appropriate until sales data accumulates. |
| Reference store for a new store has no sales | If a new store's assigned reference store has no sales history for a particular style, the new store might also show zero or default "Revenue Per Day" for that style. | Ensure reference stores are carefully selected based on strong historical performance and similarity to the new location. |
| Incorrect seasonality flags | If product seasons are misconfigured (e.g., an 'in-season' product is marked as 'read-only' or vice-versa), it can lead to incorrect reallocation decisions for active or inactive merchandise. | Review and correct the product season list and replenishment/read-only season settings to reflect current merchandising strategies. |
| Missing Ideal Size Set (ISS) definitions | If Ideal Size Set (ISS) data is missing or incomplete for certain product categories or styles, the module cannot accurately calculate the ideal size depth, potentially leading to unbalanced size allocations and missed sales. | Ensure all relevant Ideal Size Set definitions are complete and correctly loaded in the master data. |

9. Next Steps

Before This Step

- **Reallocation Prepare Data Module:** This module provides the foundational product master data (including product characteristics, style-SKU links), store information (enabled list), raw inventory levels, and Ideal Size Set (ISS) definitions, which are all crucial inputs.
- **Reallocation Prepare Sales Module:** This module delivers the cleaned and filtered historical sales data, which is essential for calculating accurate revenue metrics and determining reliable product availability.
- **Reallocation First Month ROS Calculation Module:** This module provides the initial, reliable Rate of Sale (ROS) for individual product styles, which serves as a building block for the "Revenue Per Day" calculations performed here.
- **Reallocation Sales Calculation Module:** This module supplies projected total sales quantities for broader product categories (e.g., "Women's Tops"), which influence how style-level "Revenue Per Day" is scaled across future periods.

After This Step

- **Reallocation Planning Module:** This subsequent module will utilize the precise "Revenue Per Day" metrics, Ideal Size Depth, and read-only flags generated here to calculate optimal stock movements, determining exactly

which products (and sizes) should be transferred to which stores to maximize sales and minimize inventory inefficiencies.

- **Inventory Execution Systems:** These systems receive the final reallocation recommendations and trigger physical inventory movements (e.g., store-to-store transfers, warehouse shipments) based on the calculated revenue potential and Ideal Size Depth targets.

10. Common Questions

Q: Why does the module calculate so many different “Revenue Per Day” metrics?

A: We calculate “Revenue Per Day” at various levels—individual store-style, national, and broader product groups (Attribute Groups)—to provide a comprehensive view of demand. This allows the system to make highly specific allocation decisions when detailed data is available, and to use broader, more reliable averages when individual product data is scarce, ensuring all inventory is guided by a demand signal.

Q: How does this module handle new stores that have no sales history? A: For new stores, the module cleverly replicates the “Revenue Per Day” metrics from pre-defined “reference” stores that are similar in location, customer base, or store format. This allows new stores to receive optimal initial inventory based on the proven sales performance of comparable locations, preventing early stockouts or overstock.

Q: What does it mean for a store-style to be flagged as “read-only”? A: A “read-only” flag means that the specific product style in that store is not eligible for active inventory reallocation or replenishment. This is typically applied to products from past seasons that are being cleared out, or items that are no longer part of the core active assortment. It ensures that the system focuses its efforts on high-priority, in-season inventory.

Q: How does the module determine the ideal mix of sizes for a product in a store? A: The module calculates the Ideal Size Depth by combining the store’s overall Ideal Size Set (ISS) definitions for a product category with the historical sales contribution of each specific size within that product style. This ensures that the system recommends stocking the right quantity of each size (e.g., more mediums, fewer extra-smalls) to match actual customer demand and minimize lost sales due to missing popular sizes.

Q: Why are some sales periods or days excluded when calculating “Revenue Per Day”? A: The module only includes sales from “unbroken” periods where products were consistently available and the Ideal Size Set (ISS) was adequately stocked. Sales during periods of low availability (e.g., severe stockouts) are excluded because they don’t reflect true customer demand, only what *could* be sold under constrained conditions. This filtering ensures the “Revenue Per Day” is a true measure of potential sales performance.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To calculate accurate “Revenue Per Day” metrics at various levels, determine ideal size depth, and identify read-only styles for informed inventory allocation.
- **Main Input:** Cleaned historical sales data, comprehensive product information (including Ideal Size Set), and operational data (store statuses, analysis periods, seasonality flags).
- **Main Output:** Store-style, national, and rolled-up Attribute Group “Revenue Per Day” metrics; Ideal Size Depth recommendations; and “read-only” store-style flags.
- **Warning Signs:** Unexpectedly zero or very low revenue per day for active styles, a high percentage of sales being filtered out, or new stores receiving incorrect initial inventory.
- **Key Settings:** Initial Sales Analysis Period Length, Product Availability Benchmark, Read-Only Season List, Replenishment Season List.

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 9: Reallocation Segmentation Module

Last Updated: 2025-05-23

1. Business Objective

This module categorizes store-styles into distinct segments, primarily “Top Sellers” and “Bottom Sellers”. This segmentation is critical as it informs subsequent allocation strategies, allowing the system to prioritize replenishment for top-performing items and potentially initiate pullbacks for underperforming ones.

Why This Matters: By accurately identifying which products are driving sales and which are stagnating, this module ensures inventory is moved to where it will generate the most revenue and customer satisfaction. It prevents overstocking slow-moving items and missing sales on popular ones, ultimately maximizing profitability and operational efficiency.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data:** Raw sales records, including units sold, revenue, and discount percentages, already filtered for reliability. This is the core input for analyzing historical performance.
- **Units Sold (ROS):** The quantity of products sold over a period, used to calculate selling speed and identify how quickly items are moving.
- **Revenue Per Day:** The average daily income generated by a product or category, essential for benchmarking performance.
- **Discount Percentages:** The average discount applied to sales, which can indicate slow movement if consistently high.

Product Data

- **Product Characteristics (AG):** Information like category, style, and channel attributes, used to group products for benchmarking and segmentation.
- **Product Family Links (SKU to Style):** Links individual items (SKUs) to their broader product styles, enabling aggregated analysis at the style level.

- **Category Information:** Details about product categories, necessary for establishing category-specific benchmarks (e.g., for “Women’s Tops” vs. “Men’s Footwear”).

Operational Data

- **Store Inventory Levels:** Current physical stock counts at each store, used to analyze stock for potential bottom sellers.
- **Centralized Warehouse Stock:** Inventory available in distribution centers, considered alongside store stock when evaluating bottom sellers for pullbacks.
- **Store-Category or Channel-Category Averages:** Benchmarks representing typical performance for specific product groups within stores or across sales channels, against which individual style performance is compared.
- **Enabled Stores List:** A definitive list of stores participating in the reallocation process, ensuring calculations are focused on relevant locations.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|---------------|--|--|
| Top Seller Revenue Benchmark Multiplier | 1.25 | A factor by which a style’s revenue per day must exceed the store-category average to be considered a top seller. | To adjust the aggressiveness of identifying top performers (e.g., 1.5 for truly exceptional). |
| Top Seller Minimum Live Selling Days | 30 | The minimum number of days a product style must have been actively selling to qualify as a top seller. | To ensure top seller identification is based on sufficient selling history, not just short-term spikes. |
| Bottom Seller Sales History Period | 60 days | The number of recent historical days considered when evaluating a product style as a bottom seller. | To adjust how recent sales trends influence bottom seller identification. |
| Bottom Seller Discount Threshold | 0.80 | A style’s average discount must be at or below this percentage of the store-category average discount to prevent it from being flagged as a bottom seller. | If aggressive discounting is common, or if you want to identify items only discounted more than average. |

Note: See More Details for additional settings.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Product/Store Info] --> B[Compute

Revenue Benchmarks & Style Performance]

B --> C{Identify Top Sellers}

C --> D[Compute Bottom Seller Metrics (ROS, Sell-Through, Discount)]

D --> E{Identify Bottom Sellers}

E --> F[Segmented Store-Styles]

1. Start with → Cleaned historical sales data for all products and stores, along with comprehensive product and store information, and existing pullback configurations.
2. Then → **Calculate** average revenue performance for each channel-category and store-category combination, and determine the individual revenue per day for each store-style.
3. Next → **Identify** “Top Sellers” by comparing each store-style’s revenue per day against its category and channel benchmarks, ensuring it has met minimum selling day requirements.
4. Next → **Calculate** additional metrics for potential “Bottom Sellers,” including their Rate of Sale (ROS), discount percentages, and sell-through rates, considering current inventory.
5. Next → **Identify** “Bottom Sellers” by evaluating styles that are not already flagged as top sellers, against specific criteria for sales quantity, revenue per day, discount levels, and sell-through.
6. Finally → Produce a comprehensive list of all store-styles, clearly segmented as “Top Sellers,” “Bottom Sellers,” or unsegmented, ready to guide inventory reallocation strategies.

5. Step-by-Step Actions

1. **Calculate** store-style revenue per day and benchmark averages
 - What we do: *We determine the average daily revenue generated by each product style within a specific store. Simultaneously, we calculate the average daily revenue for the broader category within that store and across the entire channel, creating benchmarks.*
 - Why we do it: *These calculations provide the foundational performance metrics and comparison points necessary to determine if a specific style is performing above or below expectations.*
2. **Flag** “Top Sellers”
 - What we do: *We compare each store-style’s revenue per day against its store-category benchmark (adjusted by the Top Seller Revenue Benchmark Multiplier) and the overall channel-category average. If a style outperforms these benchmarks and meets the Top Seller Minimum Live Selling Days threshold, it is flagged as a “Top Seller.”*
 - Why we do it: *This ensures that our most successful products are quickly identified so that their inventory can be prioritized for replenishment and optimal placement, maximizing sales opportunities.*
3. **Compute** sales metrics for potential “Bottom Sellers”
 - What we do: *For styles not identified as “Top Sellers,” we calculate additional metrics like their Rate of Sale (ROS) over a defined Bottom Seller Sales History Period, their average discount percentage, and their sell-through rate (sales compared to total inventory, including warehouse stock).*

- Why we do it: *These metrics provide a comprehensive view of a product's performance and inventory health, crucial for identifying items that are not selling well and might be overstocked.*
4. **Evaluate and Flag “Bottom Sellers”**
- What we do: *We apply a series of checks for styles not flagged as “Top Sellers”: does their sales quantity meet a Bottom Seller Minimum Sales Quantity Threshold? Is their revenue per day below a Bottom Seller Revenue Per Day Threshold relative to the store-category average? Is their average discount too high compared to the store-category discount (based on Bottom Seller Discount Threshold)? Is their sell-through too low compared to the store-category average sell-through?*
 - Why we do it: *These checks ensure that underperforming products are precisely identified based on multiple indicators of slow movement and excessive inventory, allowing for strategic pullbacks or markdown actions to free up valuable stock space.*

6. Sample Scenario

Let's consider a specific product, “Women's Summer Blouse - Style X,” at “Store A.”

Starting Point

- **Store A** has been selling “Style X” for 45 days.
- Over these 45 days, “Style X” generated \$4,500 in revenue, so its **Revenue Per Day is \$100/day**.
- The average **Revenue Per Day for “Women's Blouses” (Category Y) at Store A is \$80/day**.
- The average **Revenue Per Day for “Women's Blouses” across the entire online channel is \$90/day**.
- The Top Seller Revenue Benchmark Multiplier is 1.20.
- The Top Seller Minimum Live Selling Days is 30 days.

Now let's consider another product, “Men's Winter Jacket - Style Z,” also at “Store A,” which was not a Top Seller. - “Style Z” has been active for 90 days (the Bottom Seller Sales History Period is 60 days). - Over the last 60 days, “Style Z” sold only 10 units, while the Bottom Seller Minimum Sales Quantity Threshold is 15 units. - Its **Revenue Per Day is \$15/day**. The average **Revenue Per Day for “Men's Outerwear” (Category W) at Store A is \$50/day**. The Bottom Seller Revenue Per Day Threshold is 0.50 (meaning a style is a bottom seller if its revenue per day is less than 50% of the category average). - **Style Z's** average discount is 20%. The average discount for “Men's Outerwear” at Store A is 10%. The Bottom Seller Discount Threshold is 0.80 (meaning if style discount is greater than 80% of category discount, it's a bottom seller). - **Style Z's** sell-through rate is 15%. The average sell-through for “Men's Outerwear” at Store A is 30%. The Bottom Seller Sell-Through Multiplier (internal setting, typically < 1) implies it's a bottom seller if sell-through is below the benchmark.

What Happens

1. For “Style X” (Top Seller evaluation):

- The module checks “Style X’s” live days (45 days), which **meets** the Top Seller Minimum Live Selling Days (30 days).
- It calculates the **Store A Category Y benchmark**: $\$80/\text{day} * 1.20$ (Multiplier) = $\$96/\text{day}$.
- “Style X’s” Revenue Per Day ($\$100/\text{day}$) is **greater than** the $\$96/\text{day}$ benchmark for its store-category.
- “Style X’s” Revenue Per Day ($\$100/\text{day}$) is also **greater than** the Channel average ($\$90/\text{day}$).
- Since all conditions are met, “Women’s Summer Blouse - Style X” at Store A is **flagged as a “Top Seller.”**

2. For “Style Z” (Bottom Seller evaluation):

- It’s already confirmed “Style Z” is *not* a Top Seller.
- **Sales Quantity Check**: Style Z sold 10 units, which is **less than** the 15 unit threshold. (Condition met for bottom seller).
- **Revenue Per Day Check**: Style Z’s Revenue Per Day ($\$15/\text{day}$) is **less than** the Store A Category W benchmark ($\$50/\text{day} * 0.50 = \$25/\text{day}$). (Condition met for bottom seller).
- **Discount Check**: Style Z’s discount (20%) is 200% of the category average (10%), which is **greater than** the Bottom Seller Discount Threshold (0.80 or 80%). (Condition met for bottom seller).
- **Sell-Through Check**: Style Z’s sell-through (15%) is significantly lower than the category average (30%). (Condition met for bottom seller).
- Since multiple conditions are met, “Men’s Winter Jacket - Style Z” at Store A is **flagged as a “Bottom Seller,”** with remarks indicating low sales, low revenue per day, and high discount.

Final Outcome

“Women’s Summer Blouse - Style X” at Store A is clearly identified as a “Top Seller,” signaling that it should be prioritized for replenishment. “Men’s Winter Jacket - Style Z” at Store A is identified as a “Bottom Seller,” indicating it might be a candidate for a pullback or markdown strategy.

Key Takeaway: This module provides actionable insights by segmenting products based on their actual sales performance relative to their category and channel, enabling targeted inventory strategies.

7. Results

This step produces:

Main Outputs

- **Store-Style Segmentations:** The primary output, classifying each product style within a store as a “Top Seller,” “Bottom Seller,” or remaining unsegmented. This guides subsequent allocation decisions.

- **Segment Remarks:** Specific notes associated with “Bottom Sellers” explaining why they were identified as such (e.g., “Low Sales Quantity,” “High Discount,” “Low Revenue Per Day”). This provides context for planners.

Business Reports

- **Top/Bottom Seller Performance Dashboard:** Used by inventory planners, merchandisers, and store managers to visually identify high-performing and underperforming products across the network, enabling quick strategic adjustments.
- **Inventory Action Recommendations Report:** Serves as a direct input for the next module, highlighting which products should be prioritized for replenishment (Top Sellers) or considered for consolidation/markdowns (Bottom Sellers).

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--------------------------------------|--|--|
| Missing or delayed data feed | The module might process with incomplete sales data or missing product/store information, leading to inaccurate segmentations or errors. | Verify all data sources are up-to-date (especially from Reallocation Prepare Sales Module) and re-run the module once data is complete. |
| No “Top Sellers” identified | If no product styles meet the stringent criteria, the system might not flag any top performers, which could indicate overly aggressive settings or a general underperformance. | Review the Top Seller Revenue Benchmark Multiplier and Top Seller Minimum Live Selling Days settings; consider if benchmarks are too high for the current market. |
| Too many “Bottom Sellers” identified | A large percentage of products might be flagged as underperformers, potentially indicating an overstocked network or overly aggressive bottom seller criteria. | Review all “Bottom Seller” settings (e.g., Sales History Period, Discount Threshold, Revenue Per Day Threshold, Minimum Sales Quantity Threshold) and consider if they are too strict. |
| Inconsistent Category Benchmarks | If the average revenue per day or discount percentages for a store-category are unusually high or low, it can skew the segmentation results for styles within that category. | Investigate the data quality for category-level sales and discount averages, ensuring they accurately reflect typical performance. |

9. Next Steps

Before This Step

- **Reallocation Prepare Sales Module:** This module must have provided the cleaned and filtered historical sales data, which is the primary input for all performance calculations.
- **Reallocation Compute Revenue Module:** This module (or components of it) provides the various revenue per day metrics (store-style, national, rolled-up Attribute Group) and related benchmarks, which are used to evaluate performance in this module.
- **Reallocation Prepare Warehouse Inventory Module:** This module contributes the refined warehouse inventory levels, which are critical for assessing total stock when identifying potential “Bottom Sellers.”

After This Step

- **Reallocation Planning Module:** This subsequent module will utilize the “Top Seller” and “Bottom Seller” classifications to determine precise inventory reallocation strategies, prioritizing replenishment for top performers and planning pullbacks for underperforming stock.
- **Inventory Execution Systems:** These systems receive the final reallocation recommendations (including pullbacks) and trigger physical inventory movements, directly implementing the strategies informed by this segmentation.

10. Common Questions

Q: Why is it important to segment products into “Top Sellers” and “Bottom Sellers”? A: Segmenting products allows us to apply targeted inventory strategies. “Top Sellers” get priority for replenishment to maximize sales, while “Bottom Sellers” are candidates for pullbacks or markdowns to free up capital and shelf space, directly improving profitability and efficiency.

Q: How does this module handle seasonality when identifying top/bottom sellers? A: While not directly applying seasonality flags here, the module relies on the Reallocation Compute Revenue Module which processes sales data, including “live days” and “revenue per day,” taking into account active selling periods. The benchmarks are also usually period-specific, meaning a product’s performance is compared against its peers in the same active selling window.

Q: What if a product is a “Top Seller” in one store but a “Bottom Seller” in another? A: This module performs segmentation at the “store-style” level precisely to account for such variations. A product can indeed be a top performer in one store (e.g., a city flagship) and a bottom performer in another (e.g., a suburban outlet), allowing for highly localized and intelligent inventory movements.

Q: Can I adjust how aggressive the system is at identifying top or bottom performers? A: Yes, you can adjust the business settings like Top Seller Revenue Benchmark Multiplier or Bottom Seller Discount

Threshold. Increasing the top seller multiplier will make it harder to qualify as a top seller, while decreasing the bottom seller thresholds might make it easier to flag items for pullback.

Q: What happens to products that are neither “Top Sellers” nor “Bottom Sellers”? A: Products that are not explicitly segmented as “Top” or “Bottom” Sellers are considered “regular” or “unsegmented” performers. They will still be part of the overall reallocation calculations, but they won’t receive the same priority for immediate replenishment or targeted pullback strategies as the segmented items.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To categorize store-styles into “Top Sellers” and “Bottom Sellers” to guide inventory reallocation.
- **Main Input:** Cleaned historical sales data, various revenue per day metrics, discount percentages, and inventory levels.
- **Main Output:** Segmented store-styles (Top Seller, Bottom Seller, or unsegmented).
- **Warning Signs:** Unexpectedly few or many segmented products, or segmentation that doesn’t align with business expectations.
- **Key Settings:** Top Seller Revenue Benchmark Multiplier, Top Seller Minimum Live Selling Days, Bottom Seller Sales History Period, Bottom Seller Discount Threshold.

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 10: Reallocation Ranking Module

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for ranking store-styles to prioritize their inventory allocation. It also computes rankings for individual items (SKUs) that are designated as non-pivotal (NP) and for broader store-category-size combinations.

Why This Matters: By clearly prioritizing which store-style combinations receive inventory first, this module ensures that higher-performing or more critical products are addressed promptly. This maximizes sales potential, reduces stockouts for key items, and ensures efficient use of available inventory by directing it to where it will generate the most revenue.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Store-Style Specific Revenue Per Day:** This is the average daily revenue generated by a product style in a specific store, crucial for assessing its direct performance and prioritizing its allocation.
- **National Style Revenue Per Day:** The average daily revenue a product style generates across the entire retail network, providing a benchmark for overall market performance.
- **Product Set Available (PSA) Percentage:** This indicates the percentage of a product's ideal assortment (e.g., Ideal Size Set) that was available in a store. Higher PSA suggests better stock management and reliable sales data.
- **Cleaned Historical Sales Data:** Filtered sales records for individual SKUs and broader product categories, which are used to compute specific rankings for non-pivotal items and store-category-size combinations.

Product Data

- **Product Characteristics (AG):** Information like category, style, and brand, which helps to group products for aggregated ranking analyses and to understand attribute groups.

- **Product Family Links (SKU to Style):** Defines the relationships between individual items and their broader product styles, enabling analysis at both granular and aggregated levels.
- **Ideal Size Set (ISS) Definitions:** The optimal combination of sizes for a product, used to understand individual SKU contributions and for ranking store-category-size combinations.
- **Pivotal Item Tagging:** Identifies whether an individual item (SKU) is considered ‘pivotal’ (core to the assortment) or ‘non-pivotal’ (seasonal, promotional, or less critical), influencing how its ranking is computed.

Operational Data

- **Current Store Inventory Levels:** The most up-to-date count of products physically on shelves at each store, used as a key factor in ranking store-styles (higher stock often means more impact for reallocation).
- **Enabled Stores List:** This specifies which stores are actively participating in the reallocation process, ensuring that ranking calculations are focused only on relevant locations.
- **New Store Reference Map:** A mapping that links new stores (with limited or no sales history) to established “reference” stores, allowing the system to use the reference store’s ranking behavior for new locations.
- **Analysis Date:** The specific date for which the reallocation process is being run, used to define the look-back periods for sales data analysis when computing various revenue and sales consistency metrics.

3. Business Settings

This module primarily utilizes outputs from other modules that have their own configurable settings. There are no direct business settings within this specific module that directly influence the core ranking algorithm. The ranking factors (stock, PSA, various revenue per day metrics) and their weights are predefined within the system’s logic. For details on settings that influence the inputs to this module (such as how “Revenue Per Day” or “Product Set Available” are determined), please refer to the documentation for the “Reallocation Compute Revenue Module” and the “Reallocation Prepare Data Module.”

Note: See More Details for additional settings in dependent modules.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Processed Store-Style Details (Revenue, PSA, Stock)] -->
    B[Rank Store-Styles based on multiple factors]
    B --> C[Compute SKU-level Rankings for Non-Pivotal Items]
  
```

C --> D[Compute Store-Category-Size Rankings]
D --> E[Prioritized Store-Styles & SKU/Size Rankings]

1. Start with → Comprehensive, processed data for each store-style combination, including their total stock, Product Set Available (PSA) percentage, various revenue per day metrics, and cleaned historical sales data.
2. Then → **Rank** all store-style combinations by applying a weighted algorithm that prioritizes higher-performing items based on factors like current stock levels, product availability, and different levels of revenue per day.
3. Next → **Compute** specific sales-based rankings for individual SKUs that are classified as non-pivotal (NP) items, providing a targeted priority for these items.
4. Next → **Compute** rankings for broader store-category-size combinations, using historical sales data to identify which sizes are most critical within a category for a given store.
5. Finally → Produce a clear, prioritized list of store-styles for allocation, along with granular rankings for non-pivotal SKUs and store-category-size combinations, ready to guide precise inventory decisions.

5. Step-by-Step Actions

1. **Prioritize** Store-Styles based on key performance indicators
 - What we do: *We sort and rank all store-style combinations by comparing multiple factors. This includes prioritizing items with higher current stock, better Product Set Available (PSA) percentages, higher national style revenue per day, and higher store-style specific revenue per day, ensuring that the most impactful combinations are at the top.*
 - Why we do it: *This ensures that reallocation efforts are first directed towards the products and stores that have the greatest potential to generate revenue, maximize customer satisfaction, or where inventory is abundant and can be moved effectively.*
2. **Compute** SKU-level ranking for Non-Pivotal (NP) items
 - What we do: *For individual items (SKUs) that are not classified as 'pivotal' (e.g., seasonal, trend, or clearance items), we calculate their specific ranking. This is primarily based on their recent sales performance (revenue per day), with the ranking of their parent store-style acting as a tie-breaker.*
 - Why we do it: *This provides a targeted priority for managing inventory of less critical items, ensuring that even non-pivotal products are allocated intelligently to minimize obsolescence or maximize sell-through before the end of their lifecycle.*
3. **Compute** Store-Category-Size ranking
 - What we do: *We analyze historical sales data for each store to determine the average daily revenue generated by specific size categories (e.g., 'Medium T-shirts' in 'Women's Tops') across all categories. These store-category-size combinations are then ranked based on their revenue per day.*
 - Why we do it: *This helps to ensure that stores are optimally stocked with the most in-demand sizes within key product categories, preventing lost sales due to missing popular sizes and improving overall inventory balance.*
4. **Publish** the ranked output for reallocation
 - What we do: *We consolidate all calculated rankings into a structured output, clearly assigning a rank to each store-style combination based on the established priority. This data is then made available for the subsequent allocation planning module.*

- Why we do it: *This provides a clear, actionable roadmap for inventory planners, allowing the system to systematically address the most critical reallocation needs first, optimizing inventory flow and business performance.*

6. Sample Scenario

Let's walk through a real example involving two different product styles: "Women's Core Black Dress - Style A" and "Men's Seasonal Graphic Tee - Style B."

Starting Point

- **Store X** has "Style A" with a total stock of 50 units, a Product Set Available (PSA) of 95%, a National Revenue Per Day of \$200, and a Store-Style Specific Revenue Per Day of \$150.
- **Store Y** has "Style B" with a total stock of 70 units, a Product Set Available (PSA) of 80%, a National Revenue Per Day of \$180, and a Store-Style Specific Revenue Per Day of \$160.
- "Style B" also has a Non-Pivotal (NP) SKU, "Graphic Tee - Size M", which has sold 10 units in the last 7 days at Store Y, generating \$100.
- For the "Tee" category at Store Y, "Size M" has generated \$300 in the last 6 months, while "Size L" generated \$250.

What Happens

1. **Store-Style Ranking:** The module first evaluates "Style A" at Store X and "Style B" at Store Y using its ranking algorithm.
 - **Stock:** Store Y's Style B (70 units) ranks higher than Store X's Style A (50 units) on stock.
 - **PSA:** Store X's Style A (95%) ranks higher than Store Y's Style B (80%) on PSA.
 - **National Revenue:** Store X's Style A (\$200) ranks higher than Store Y's Style B (\$180) on National Revenue.
 - **Store-Style Revenue:** Store Y's Style B (\$160) ranks higher than Store X's Style A (\$150) on Store-Style Revenue.
 - Based on the weighted combination of these factors, let's say "Women's Core Black Dress - Style A" at Store X ultimately receives a higher overall rank (e.g., Rank 1) than "Men's Seasonal Graphic Tee - Style B" at Store Y (e.g., Rank 2), indicating it should be prioritized for attention first.
2. **Non-Pivotal SKU Ranking:** For "Graphic Tee - Size M" (NP SKU) at Store Y, the module calculates its revenue per day ($\$100 / 7 \text{ days} = \sim \$14.29/\text{day}$). It then assigns a rank to this specific SKU based on this performance, with the rank of its parent "Style B" (Rank 2) serving as a tie-breaker if other NP SKUs have similar daily revenue.
3. **Store-Category-Size Ranking:** For the "Tee" category at Store Y, the module sees that "Size M" generated \$300 and "Size L" generated \$250 over the last 6 months. It calculates their respective revenue per day (total revenue / live selling days) and ranks them. In this case, "Size M" would rank higher than "Size L" for the "Tee" category at Store Y.

Final Outcome

The system produces a comprehensive ranked list. “Women’s Core Black Dress - Style A” at Store X is identified as the top priority for reallocation attention. Additionally, specific rankings are provided for “Graphic Tee - Size M” at Store Y and for “Size M” within the “Tee” category at Store Y. This allows inventory planners to first address the most impactful store-style combinations, then fine-tune allocations for specific non-pivotal items and ensure the right size mixes in stores.

Key Takeaway: This module provides a clear, data-driven hierarchy for inventory reallocation decisions, ensuring that resources are focused on the most profitable and critical product-store combinations while also providing granular guidance for individual items and sizes.

7. Results

This step produces:

Main Outputs

- **Ranked Store-Style List:** A prioritized list of all relevant store-style combinations, ordered from highest to lowest priority for reallocation consideration based on combined performance metrics. This is the primary driver for subsequent allocation decisions.
- **Ranked Non-Pivotal (NP) SKU List:** A prioritized list of individual SKUs classified as non-pivotal, ranked primarily by their revenue per day. This guides targeted actions for managing these specific items.
- **Ranked Store-Category-Size List:** A prioritized list of specific size categories within stores, indicating which sizes generate the most revenue within their broader category, guiding optimal size mix decisions.

Business Reports

- **Reallocation Priority Report:** Used by inventory planners and store managers to quickly see which store-style combinations require immediate attention for inventory adjustments, maximizing revenue and optimizing stock distribution.
- **Non-Pivotal Item Action Report:** Used by merchandising and planning teams to identify which non-pivotal items should be targeted for specific actions like consolidation, markdown, or targeted replenishment based on their sales performance.
- **Size Mix Optimization Report:** Used by planning teams to review the performance of different sizes within categories, informing adjustments to Ideal Size Set (ISS) definitions and ensuring optimal stock levels for popular sizes.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|-----------|---|--|
| | The module might process with incomplete stock, PSA, or | Verify all upstream data sources (from |

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | revenue data, leading to inaccurate rankings. | Reallocation Prepare Data, Reallocation Compute Revenue) are up-to-date and re-run the module once data is complete. |
| All store-styles have identical performance values | If two or more store-styles have precisely the same values across all ranking factors, their relative order in the output might be arbitrary. | While rare, this indicates either highly uniform demand or a need to introduce additional tie-breaking criteria in upstream modules. |
| No non-pivotal (NP) items defined | The module will not compute any SKU-level rankings for non-pivotal items, as there are no relevant SKUs to process. | This is expected if all SKUs are tagged as 'pivotal' or if no NP items are in scope. Ensure product tagging is accurate. |
| Zero sales data for store-category-sizes | If a store or category has no sales for certain sizes, those combinations will receive a zero revenue per day and rank at the bottom. | This is expected for new categories/stores or very slow-moving items. Review category setup and sales history for accuracy. |

9. Next Steps

Before This Step

- **Reallocation Prepare Data Module:** This module must have provided the foundational data, including current store inventory levels, Product Set Available (PSA) percentages, Ideal Size Set (ISS) definitions, and all relevant product and store information.
- **Reallocation Compute Revenue Module:** This module must have calculated the various "Revenue Per Day" metrics (store-style, national, rolled-up Attribute Group) and identified pivotal item tagging, which are crucial inputs for ranking.
- **Reallocation Prepare Sales Module:** This module must have cleaned and filtered historical sales data, which is essential for computing rankings for non-pivotal items and store-category-size combinations.

After This Step

- **Reallocation Planning Module:** This subsequent module will utilize the precise, prioritized rankings generated here to determine optimal stock movements, deciding exactly which products (and sizes) should be transferred to which stores to maximize sales and minimize inventory inefficiencies.
- **Inventory Execution Systems:** These systems receive the final reallocation recommendations (e.g., transfer orders) and trigger physical inventory movements based on the calculated ranking priorities.

10. Common Questions

Q: Why is ranking important for inventory reallocation? A: Ranking is crucial because it tells us *what* to prioritize. Instead of randomly moving inventory, ranking identifies the most valuable or critical products and stores first. This ensures we maximize sales opportunities by keeping popular items in stock and minimize losses by addressing overstocked, slow-moving items strategically.

Q: How does this module decide which store-style is “more important” than another? A: The module uses a combination of factors. It considers how much total stock an item has, how consistently its full size range was available (Product Set Available or PSA), and how much revenue it generates both nationally and specifically in that store. Items with higher values across these factors generally get a higher rank, indicating they are more critical for reallocation.

Q: What are “Non-Pivotal (NP)” items, and why do they have a separate ranking? A: Non-Pivotal items are typically products that are seasonal, trend-driven, or less critical to the core assortment (like a holiday-specific graphic tee). They get a separate ranking because their reallocation strategy might differ from core items. This targeted ranking helps manage their lifecycle more effectively, such as pushing them out before the season ends.

Q: How does this module help ensure stores have the right sizes? A: By computing rankings for “Store-Category-Size” combinations, the module identifies which specific sizes within a product category are top performers for a given store. This helps planners prioritize stocking the most in-demand sizes (e.g., more mediums, fewer extra-smalls) to match actual customer demand, reducing lost sales from missing popular sizes.

Q: What happens if a new store has no sales history for ranking? A: For new stores, the module uses a “New Store Reference Map.” This means it will copy the ranking metrics from a designated “reference” store that is similar in demographics or size. This ensures new stores still receive optimal initial inventory based on proven demand patterns from comparable, established locations.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To prioritize inventory reallocation by ranking store-styles, non-pivotal SKUs, and store-category-size combinations.
- **Main Input:** Store-style performance data (stock, PSA, various Revenue Per Day metrics), and cleaned sales data.
- **Main Output:** Prioritized lists of store-styles, non-pivotal SKUs, and store-category-size combinations.
- **Warning Signs:** Rankings that don’t align with business expectations, or missing data leading to inaccurate priorities.
- **Key Settings:** *None directly in this module, but heavily influenced by settings in upstream modules related to revenue calculation and data preparation.*

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Chapter 11: Reallocation Allocation Module

Last Updated: 2025-05-23

1. Business Objective

This module serves as the central engine for the inventory reallocation process, executing a series of iterative calculations to distribute inventory from warehouses to stores, adhering to various business rules and constraints.

Why This Matters: By accurately simulating sales and making intelligent allocation decisions for individual items (SKUs) over time, this module ensures that inventory is optimally placed to maximize sales, minimize stockouts, reduce excess stock, and enhance overall profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **SKU-level Rate of Sale (ROS) for all Periods:** This precise selling speed for each individual item (SKU) in every store across future periods is critical for projecting demand and making allocation decisions that align with anticipated sales.
- **Store-Style Revenue Per Day:** The average daily revenue generated by each product style in a specific store, used to prioritize allocation efforts towards high-performing items.
- **Store-Style Segmentations (Top/Bottom Sellers):** Classifications indicating whether a store-style is a “Top Seller” or “Bottom Seller,” guiding specific allocation strategies like prioritizing replenishment or initiating pullbacks.

Product Data

- **Ideal Size Set (ISS) Definitions:** These optimal combinations of sizes for a product are crucial for ensuring the correct mix of sizes is allocated to stores to maximize sales and minimize stockouts for specific items.
- **Optimum Depth (OD) & Optimum Width (OW) Rules:** These business rules define the ideal number of unique items (OD) and categories or styles (OW) to stock, guiding inventory targets for each store-style combination.

- **Pivotal Item Tagging:** Identifies whether an item is ‘pivotal’ (core) or ‘non-pivotal’ (seasonal/less critical), influencing its allocation priority and how it’s handled in depletion scenarios.
- **Attribute Group (AG) Information:** Product attributes grouped for analysis (e.g., color, size, brand), which inform planogram adherence and specific allocation rules.

Operational Data

- **Refined Warehouse Inventory:** The adjusted count of truly available warehouse stock, factoring in inefficiency and online fulfillment needs, which is the primary source for store allocations.
- **Current Store Inventory Levels:** The up-to-date count of products physically on shelves at each store, serving as the starting point for depletion simulations.
- **Store Layout Guidelines (Planograms):** These define display capacities and replenishment cycles, informing how much product a store can physically hold and influencing allocation amounts.
- **Enabled Stores List:** Specifies which stores are actively participating in the reallocation process, focusing the engine’s efforts on relevant locations.
- **Depletion Simulation Dates & Frequencies:** Defines the specific timeframes and intervals for simulating sales depletion and making iterative allocation decisions.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|--------------------------|--|---|
| Run Core Allocation & Replenishment Flag | Yes | Determines if the core allocation and replenishment iterations are executed. | If you want to temporarily disable core allocation cycles for specific scenarios. |
| Deplete Warehouse Stock Flag | Yes | If enabled, the module simulates sales depletion from warehouse stock. | To switch between a full depletion simulation or only allocate current on-hand. |
| P1 End Date | (Current Date + 30 days) | The end date for the primary allocation period (P1), influencing initial allocation focus. | To adjust the horizon for immediate allocation planning. |
| Warehouse Allocation Frequency | 7 days | The interval at which warehouse stock is considered and | To align with weekly or bi-weekly warehouse replenishment cycles. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---------------------------------|----------------|
| | | depleted during the simulation. | |

Note: See More Details for additional settings related to specific allocation iterations (e.g., minimum age, top seller inclusion, planogram increments, non-pivotal sizes, pullbacks).

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

A[Refined Inventory, Sales & Product Data] --> B[Initialize Allocation Engine & Simulation]
B --> C[Run Iterative Depletion & Allocation Cycles]
C --> D[Generate Pre/Post-Allocation Reports]
D --> E[Final Allocation Decisions & Depletion Outputs]

```

1. Start with → All refined inventory (store, warehouse, incoming), precise SKU-level sales rates, product definitions (ISS, OD, OW, pivotal tags), and store operational data.
2. Then → **Initialize** the core allocation engine, setting up the framework for depletion simulation and preparing initial stock maps.
3. Next → **Execute** a series of iterative depletion and allocation cycles, simulating sales consumption over time and making real-time adjustments to stock levels and allocation decisions for individual SKUs based on various business rules (e.g., core replenishment, top seller, non-pivotal, pullbacks).
4. Next → **Generate** detailed reports showing stock levels before and after the full depletion simulation and any remaining inventory.
5. Finally → Produce comprehensive final allocation decisions (units to be moved where) and detailed depletion outputs, providing a clear roadmap for inventory movement.

5. Step-by-Step Actions

1. **Initialize** current stock and business criteria
 - What we do: *We load the most up-to-date store and warehouse inventory levels and set up all the business rules, such as ideal stock levels (OD, OW, ISS) and allocation constraints, for the simulation.*
 - Why we do it: *This establishes the starting point and all the necessary parameters for the allocation engine to make informed decisions throughout the depletion process.*
2. **Simulate** depletion and run allocation iterations
 - What we do: *For each defined period (e.g., week by week), we simulate sales depleting inventory from stores and warehouses. Then, we run a sequence of allocation "iterations" (e.g., core replenishment, top seller, non-pivotal sizes, pullbacks) to decide how much inventory of each SKU should be moved to which store.*
 - Why we do it: *This iterative process allows the system to react dynamically to changing stock levels and demand signals, ensuring that inventory is*

always moved to where it's most needed to maximize sales and minimize stockouts.

3. Adjust inventory based on sales and inflows

- What we do: *After each simulation period, we update the remaining stock levels in stores and warehouses based on projected sales and any incoming shipments. We also account for any “inwards” that replenish warehouse stock.*
- Why we do it: *This keeps the inventory picture accurate throughout the multi-period simulation, enabling realistic allocation decisions that reflect actual stock availability.*

4. Calculate future period allocations (P2)

- What we do: *Beyond the immediate primary allocation period (P1), we calculate estimated allocations for a subsequent period (P2). This considers additional cover days based on store and product attributes and projects sales to estimate future inventory needs.*
- Why we do it: *This forward-looking planning ensures that even future inventory requirements are considered, providing a more comprehensive view of long-term allocation needs and preventing future stockouts.*

5. Record pre- and post-depletion stock levels

- What we do: *Before initiating the full depletion simulation and after it completes, we capture snapshots of store and warehouse stock levels. This includes capturing what's left if the “empty warehouse” flag is set.*
- Why we do it: *These reports provide transparency into the module's impact, allowing planners to compare stock positions and validate the effectiveness of the allocation decisions.*

6. Sample Scenario

Let's consider a popular “Women's Core T-Shirt - Style X” and a seasonal “Men's Holiday Sweater - Style Y.”

Starting Point

- **Store A** has 5 units of “Style X” (a Top Seller) and 10 units of “Style Y” (a Non-Pivotal, Bottom Seller).
- The **Warehouse** has 100 units of “Style X” and 20 units of “Style Y.”
- “Style X” is projected to sell 2 units/day at Store A (SKU-level ROS).
- “Style Y” is projected to sell 0.5 units/day at Store A (SKU-level ROS).
- The Warehouse Allocation Frequency is 7 days.
- The Run Core Allocation & Replenishment Flag is set to Yes.
- The Deplete Warehouse Stock Flag is set to Yes.

What Happens

1. Week 1 (Day 0-7):

- The module first **records** pre-depletion stock: Store A (Style X: 5 units, Style Y: 10 units); Warehouse (Style X: 100 units, Style Y: 20 units).
- It **simulates** sales for 7 days: Store A sells 14 units of Style X (7 days * 2 units/day) and 3.5 units of Style Y (7 days * 0.5 units/day).
- Store A's Style X stock drops to 0 (5 units sold, 9 remaining units lost sales).

- Store A's Style Y stock drops to 6 units (3.5 units sold, 6.5 remaining).
- Since Style X is a "Top Seller" and Store A has stockouts, the core allocation iteration identifies a need. The module **allocates** 10 units of Style X from the warehouse to Store A, replenishing its stock.
- The warehouse stock for Style X decreases to 90 units.
- No allocation for Style Y due to its "Bottom Seller" status and existing stock.

2. Week 2 (Day 7-14):

- Sales continue, stock levels **adjust**.
- A "pullback" iteration might run, identifying Style Y (Bottom Seller) as a candidate for return to the warehouse if its stock is too high and sales are too low. For example, 5 units of Style Y are pulled back from Store A to the warehouse.
- The warehouse stock for Style Y increases to 25 units.
- The module adds 10 units of incoming Style X (from a previously loaded purchase order) to the warehouse, increasing its total to 100 units.

3. End of Simulation:

- All periods are simulated, and all iterative allocations (replenishments, pullbacks, non-pivotal adjustments) are completed.
- The module **records** post-depletion stock: final inventory levels in Store A and the Warehouse after all sales and allocations.

Final Outcome

The system provides a clear, week-by-week record of inventory movements and projected stock levels. Store A received crucial replenishment for "Style X" to prevent lost sales, while excess "Style Y" was pulled back to optimize inventory. The final outputs show the exact quantities of each SKU allocated between the warehouse and Store A, along with final projected stock levels after the depletion simulation.

Key Takeaway: This module precisely mimics real-world inventory flow and sales to make intelligent, dynamic allocation decisions, ensuring inventory is always working efficiently to meet demand and support strategic goals.

7. Results

This step produces:

Main Outputs

- **Final Allocation Decisions (SKU-level):** The precise quantity of each individual item (SKU) recommended for transfer from the warehouse to specific stores, or as pullbacks from stores to the warehouse. This is the actionable output for inventory movement.
- **Depletion Output Rows (Pre & Post Simulation):** Detailed reports showing the initial stock levels in stores and warehouses before the depletion simulation, and the final stock levels after all sales and allocations have been simulated. This provides a comprehensive audit trail of inventory flow.
- **Depletion Quantity Output Rows:** Specific rows detailing sales quantities, store allocations, warehouse allocations, and closing stock for each SKU in each store for each simulated period.

Business Reports

- **Daily/Weekly Allocation Plan:** Used by inventory planners and warehouse operations to execute physical transfers of stock to stores, ensuring critical products are replenished and underperforming items are managed.
- **Inventory Depletion Analysis Report:** Used by inventory planners and finance teams to review the simulated impact of sales and allocations on overall inventory levels, helping to assess the health of the stock and inform future buying.
- **Stock Variance Report:** Used by inventory and store operations to identify discrepancies between predicted and actual stock levels, highlighting areas for improved inventory accuracy or process adjustments.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|--|
| Missing or delayed data feed | The module might process with incomplete inventory, sales, or product data, leading to inaccurate allocations or errors during depletion. | Verify all upstream data sources (from Reallocation Prepare Data, Prepare Sales, etc.) are up-to-date and re-run the module once data is complete. |
| Inaccurate ROS or sales projections | If the SKU-level Rate of Sale or overall sales quantities are significantly off, the depletion simulation will be unrealistic, leading to over- or under-allocations. | Review the Reallocation Period ROS Calculation Module and Reallocation Sales Calculation Module outputs and settings for accuracy. |
| Overly restrictive allocation rules | If business rules (e.g., high OD/OW targets, strict planogram adherence, or very aggressive exclusion criteria) are too tight, the module might struggle to allocate effectively, leaving stock in the warehouse. | Review and adjust the parameters for OD/OW rules, planograms, and iteration constraints to allow for more flexible allocation. |
| Insufficient warehouse stock for demand | If total available warehouse stock cannot meet the combined demand from all stores, some stores or SKUs will remain understocked despite optimal allocation efforts. | Review warehouse stock levels, consider expediting incoming shipments, or adjust demand projections if stock is consistently insufficient. |

9. Next Steps

Before This Step

- **Reallocation Prepare Data Module:** Provides consolidated inventory, product definitions (ISS, OD, OW, AG), and store layout guidelines.
- **Reallocation Prepare Sales Module:** Delivers cleaned historical sales data.

- **Reallocation Prepare Warehouse Inventory Module:** Supplies the refined, allocable warehouse inventory.
- **Reallocation First Month ROS Calculation Module:** Provides initial style-level ROS.
- **Reallocation Sales Calculation Module:** Computes total sales quantities per store-category-period.
- **Reallocation Period ROS Calculation Module:** Generates precise SKU-level ROS for all periods.
- **Reallocation Compute Revenue Module:** Calculates various “Revenue Per Day” metrics and Ideal Size Depth.
- **Reallocation Segmentation Module:** Classifies store-styles into “Top Sellers” and “Bottom Sellers.”
- **Reallocation Ranking Module:** Prioritizes store-styles and SKUs for allocation.

After This Step

- **Inventory Execution Systems:** These systems receive the final allocation recommendations and generate transfer orders or pick lists for physical movement of inventory between warehouses and stores, or between stores themselves.
- **Financial Reporting Systems:** The depletion outputs and allocation decisions feed into financial reports to track inventory value, cost of goods sold, and profitability impacts of reallocation.
- **Replenishment & Buying Systems:** The insights gained from the depletion simulation and allocation performance inform future purchasing decisions and ongoing replenishment strategies.

10. Common Questions

Q: How does this module decide which items get allocated first? A: The module runs a series of “iterations” with specific priorities. It first focuses on replenishing “Top Sellers” to prevent lost sales, then addresses core replenishment needs, then handles “Non-Pivotal” sizes, and finally, if enabled, performs “pullbacks” for slow-moving or excess inventory. This structured approach ensures the most impactful allocations happen first.

Q: What is “depletion simulation” and why is it important? A: Depletion simulation is like running a mini-movie of your inventory over time. It projects sales, reduces stock based on those sales, and then makes allocation decisions for the remaining stock, repeating this for future periods. This helps you understand how inventory will flow and where issues might arise, making proactive and precise allocations.

Q: Can this module prevent us from overstocking or understocking? A: Yes. By combining ideal stock rules (OD, OW, ISS) with dynamic sales projections (ROS) and an iterative allocation process, the module aims to move inventory to match anticipated demand as closely as possible, minimizing both excess stock (which ties up cash) and stockouts (which lead to lost sales).

Q: How does the module handle products that aren't selling well? A: Products identified as "Bottom Sellers" (from the Reallocation Segmentation Module) or "Non-Pivotal" items are handled specifically. The module can initiate "pullbacks" to return excess inventory from stores to the warehouse, freeing up valuable store space and allowing for redistribution or strategic markdowns.

Q: What if the module recommends moving stock but there isn't enough in the warehouse? A: The module works with the *refined* warehouse inventory, which accounts for real availability. If demand exceeds available warehouse stock, the module will allocate as much as possible to the highest-priority locations and SKUs, but it cannot create stock that doesn't exist. This highlights a need for upstream buying or inventory management adjustments.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To serve as the core allocation engine, simulating inventory depletion and making iterative SKU-level allocation decisions from warehouses to stores.
- **Main Input:** Refined warehouse and store inventory, SKU-level ROS, various performance metrics (e.g., Revenue Per Day), and all relevant product/store rules (ISS, OD, OW).
- **Main Output:** Final SKU-level allocation decisions (transfers, pullbacks) and detailed pre/post-depletion stock reports.
- **Warning Signs:** Unexpectedly high remaining warehouse stock (not enough allocations), or persistent stockouts in high-demand stores even after allocation runs.
- **Key Settings:** Run Core Allocation & Replenishment Flag, Deplete Warehouse Stock Flag, P1 End Date, Warehouse Allocation Frequency.

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 12: Reallocation Output Module

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for consolidating, fanning out, and persisting all final inventory reallocation results. It meticulously denormalizes allocation decisions made for broader product styles down to individual items (SKUs), accurately calculates remaining warehouse inventory, and generates a suite of comprehensive output reports. These reports include detailed distribution quantities, warehouse movements, fill rates, overstocking specifics, and specialized reallocation outputs for depletion analysis and dispatch prioritization.

Why This Matters: This module is the final critical step that translates complex allocation decisions into actionable insights and concrete instructions. By providing precise, granular details for every inventory movement, it ensures that merchandising and operations teams can efficiently execute stock transfers, track performance, and gain clear visibility into inventory changes. This ultimately maximizes sales, optimizes stock levels, and drives overall business efficiency.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **SKU-level Rate of Sale (ROS):** This precise selling speed for each individual item (SKU) in every store across future periods is critical for calculating fill rates and other performance metrics in the final reports, ensuring the output reflects the true selling velocity of each item.

Product Data

- **Product Family Links (SKU to Style):** This information, linking individual items (SKUs) to their broader product styles, is crucial for breaking down higher-level allocation decisions into precise instructions for every unique item, ensuring accurate inventory movement.
- **Product Characteristics (AG):** Details like product style, category, or brand are used to categorize products for various reports, enabling comprehensive analysis of allocation impacts across different product groupings.

Operational Data

- **Final Allocation Decisions:** These are the core output from the previous Reallocation Allocation Module, detailing which broader product styles are to be moved where, forming the primary input for this module's processing.
- **Current Store Inventory Levels:** The most up-to-date count of products physically on shelves at each store serves as the starting point for reporting on stock changes and determining outward movements (e.g., pullbacks).
- **Refined Warehouse Inventory:** This is the adjusted count of truly available inventory in our central distribution centers, factoring in inefficiency and online fulfillment needs, which is essential for computing remaining central stock and reporting specific warehouse movements.
- **Incoming Shipments (Goods in Transit, Open Orders, In-Transit Transfers):** Information on products already en route to stores or committed to orders is included to ensure the reported store stock levels are comprehensive and reflect all anticipated inventory.

3. Business Settings

This module primarily utilizes outputs and settings from upstream modules that have their own configurable parameters. There are no direct business settings within this specific module that directly influence its core denormalization or reporting logic. For details on settings that influence the inputs to this module (such as how allocation decisions are made or how inventory is prepared), please refer to the documentation for the Reallocation Allocation Module, Reallocation Prepare Data Module, and Reallocation Prepare Warehouse Inventory Module.

Note: See More Details for additional settings in dependent modules.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Parent-level Allocation Decisions & All Processed Data] --> B[Denormalize & Consolidate Allocations to Individual SKUs]
B --> C[Calculate Remaining Inventory & Prepare All Output Reports]
C --> D[Comprehensive Reallocation Output Reports (e.g., Distribution Quantities, Warehouse Movements, Fill Rates)]
```

1. Start with → The final allocation decisions made for broader product styles, along with comprehensive processed data including current store and warehouse inventory, and all product details.
2. Then → **Denormalize** and **consolidate** these high-level allocation decisions into precise, actionable instructions for each individual item (SKU).
3. Next → **Calculate** the final remaining warehouse inventory after all movements, and **prepare** all the necessary data for a suite of detailed output reports.

4. Finally → **Produce** comprehensive reallocation output reports, providing clear distribution quantities, warehouse movements, fill rates, and overstocking insights.

5. Step-by-Step Actions

1. **Consolidate** initial inventory and allocation data
 - What we do: *We gather the most up-to-date store and warehouse inventory levels, combining them with all the final allocation decisions made by the core reallocation engine in the previous step.*
 - Why we do it: *This establishes the complete, real-time snapshot of inventory and the intended movements, providing the foundation for all subsequent reporting.*
2. **Denormalize** allocations to individual items (SKUs)
 - What we do: *We meticulously break down allocation decisions that were made for broader product styles (e.g., “Men’s T-Shirt”) into specific, actionable quantities for each individual item (SKU) within that style (e.g., “Men’s Blue T-Shirt, Size Medium”).*
 - Why we do it: *This ensures granular instructions for every unique product, vital for precise physical inventory picking and shipment, minimizing errors and maximizing accuracy.*
3. **Process** warehouse and store movements
 - What we do: *We calculate the precise quantities of items moving from warehouses to stores, and conversely, any items moving from stores back to the warehouse (known as “pullbacks”), for each individual SKU.*
 - Why we do it: *This provides a clear audit trail of all inventory transfers, enabling accurate tracking of stock flow throughout the retail network.*
4. **Compute** remaining warehouse inventory
 - What we do: *We update and report on the final stock levels in the warehouse after all approved allocations and reservations have been accounted for.*
 - Why we do it: *This gives a clear, real-time picture of available central stock for future planning or new allocation cycles, helping to manage supply chain efficiency.*
5. **Generate** comprehensive output reports
 - What we do: *We compile and produce a suite of detailed reports. These include final distribution quantities, actual warehouse movements, fill rates (how well demand was met), overstocking details, and specialized outputs tailored for depletion analysis and dispatch prioritization.*
 - Why we do it: *These reports provide the critical business intelligence needed by inventory planners, store managers, and warehouse operations to execute physical movements, track performance, and make informed strategic adjustments.*

6. Sample Scenario

Let’s consider a popular “Classic Cotton Tee” and see how its allocation becomes a concrete plan.

Starting Point

- **Style:** “Classic Cotton Tee” (Parent ID: 500)
- **Child SKUs:** Small (ID: 501), Medium (ID: 502), Large (ID: 503)
- **Warehouse A** has enough total stock of Style 500 (e.g., 20 Small, 30 Medium, 15 Large).
- **Store X** currently has: 5 Small, 2 Medium, 8 Large of “Classic Cotton Tee”.
- The Reallocation Allocation Module determined that Store X should receive an additional **15 units** of “Classic Cotton Tee” (Style 500) from Warehouse A.
- The Ideal Size Set (ISS) for “Classic Cotton Tee” at Store X indicates that for every 10 units sold, the ideal mix is 3 Small, 5 Medium, and 2 Large.
- There are no incoming shipments or pullbacks in this specific example.

What Happens

1. First, the module receives the final allocation decision: send 15 units of “Classic Cotton Tee” (Style 500) from Warehouse A to Store X.
2. Next, it consults the Ideal Size Set (ISS) for “Classic Cotton Tee” at Store X to determine how these 15 units should be proportioned among the individual sizes (SKUs). Based on the 3:5:2 ratio for every 10 units, it calculates the following for the 15 units:
 - Small (SKU 501): $15 \text{ units} * (3/10) = 4.5 \text{ units}$. The system rounds this to **5 units**.
 - Medium (SKU 502): $15 \text{ units} * (5/10) = 7.5 \text{ units}$. The system rounds this to **7 units**.
 - Large (SKU 503): $15 \text{ units} * (2/10) = 3 \text{ units}$. This remains **3 units**. (*Note: The total allocated is $5 + 7 + 3 = 15$ units, precisely matching the parent-level allocation. Internal rounding ensures total quantity is maintained.*)
3. The module then calculates the immediate impact on Warehouse A’s inventory for SKUs 501, 502, and 503, reducing their stock by 5, 7, and 3 units respectively.
4. Finally, it compiles detailed reports showing that Store X is to receive exactly 5 Small, 7 Medium, and 3 Large “Classic Cotton Tees”, along with the updated remaining warehouse stock levels for each of these SKUs. It also generates fill rate and inventory summary reports for Store X for this specific style.

Final Outcome

Store X receives a precise, SKU-level breakdown of its new “Classic Cotton Tee” inventory, ensuring that the optimal size mix is replenished. Warehouse A’s inventory is accurately updated to reflect these transfers. This clear, granular output allows for accurate physical picking and dispatch, minimizing errors and optimizing sales by ensuring the right sizes are on shelves to meet customer demand.

Key Takeaway: This module translates high-level allocation strategies into actionable, SKU-specific instructions and comprehensive reports, enabling efficient physical inventory movements and clear visibility of stock changes across the network.

7. Results

This step produces:

Main Outputs

- **SKU-level Allocation Instructions:** The precise quantity of each individual item (SKU) recommended for transfer from warehouses to specific stores, or as pullbacks from stores to the warehouse. This is the actionable output for physical inventory movement.
- **Detailed Distribution Reports:** Comprehensive breakdowns of all stock movements, including quantities, source, destination, and the type of allocation (e.g., core replenishment, top seller, pullback).
- **Remaining Warehouse Inventory Status:** Current stock levels in the central warehouse for each product after all allocations and reservations have been accounted for, providing a clear picture of available central stock.
- **Fill Rate Analysis:** Reports indicating how effectively store demand for specific products was met by the allocations, helping to measure allocation performance.
- **Overstocking Details:** Identification of items that remain overstocked in stores, even after reallocation attempts, highlighting areas for potential future markdowns or consolidation.
- **Dispatch Prioritization Data:** Specific outputs tailored to optimize warehouse picking and dispatch sequences, helping to streamline operations and ensure timely shipments.

Business Reports

- **Daily Transfer Order Manifest:** Used by warehouse operations and store teams to execute physical transfers of stock, providing a clear list of what to pick, pack, and send to each store.
- **Inventory Health & Fill Report:** Used by inventory planners and store managers to assess the effectiveness of reallocation in meeting store demand and maintaining optimal stock levels, guiding future decisions.
- **Overstocking Review Report:** Used by merchandising and planning teams to identify persistent overstock issues and inform decisions on markdowns, promotions, or further inventory consolidation actions.
- **Warehouse Pick & Pack List:** Provides detailed instructions for warehouse staff to efficiently pick specific SKUs for outgoing shipments based on predefined allocation priorities.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|--|--|
| Missing or delayed data feed | The module might process with incomplete allocation decisions or stock data, leading to inaccurate or incomplete output reports. | Verify all upstream data sources (especially from the Reallocation Allocation Module and Reallocation Prepare Data Module) are up- |

| Situation | What Happens | What You Should Do |
|--|---|--|
| | | to-date and re-run the module once data is complete. |
| Parent style allocated but no child SKUs defined | If a product style has an allocation but no individual items (SKUs) are linked to it in the system, the denormalization process may result in zero specific SKU allocations for that style, leading to incomplete reports. | Ensure all product styles have their individual SKUs correctly linked in the product master data for accurate physical movement. |
| Allocated quantity exceeds available stock | While the Reallocation Allocation Module is designed to prevent this, if an issue occurs upstream leading to an allocation decision for more stock than physically available, the output reports may show discrepancies. | Investigate upstream allocation logic and inventory accuracy in preceding modules to prevent over-commitment. |
| Zero allocations generated by previous module | If the Reallocation Allocation Module determines that no transfers are needed (e.g., current inventory levels are already optimal), this module will still run but will produce reports indicating zero movements and no changes to stock levels. | This is expected behavior if current inventory levels are sufficient; no action is typically required unless this contradicts business expectations. |

9. Next Steps

Before This Step

- **Reallocation Allocation Module:** This module provides all the final, parent-level allocation decisions and depletion outputs that this module processes and denormalizes.
- **Reallocation Prepare Data Module:** This module supplies foundational data including current store inventory levels, product family links (SKU to Style), and detailed product characteristics (AG).
- **Reallocation Prepare Warehouse Inventory Module:** This module delivers the refined, truly allocable warehouse inventory, which is crucial for computing and reporting on remaining central stock.

After This Step

- **Inventory Execution Systems:** These systems receive the detailed SKU-level allocation instructions and automatically generate transfer orders, pick lists, or other tasks for physically moving inventory between warehouses and stores, or between stores themselves.

- **Financial Reporting Systems:** The comprehensive output reports from this module feed into financial systems to update inventory valuations, cost of goods sold, and to track the overall financial impact and profitability of reallocation decisions.
- **Business Intelligence & Analytics Platforms:** The granular output data is ingested into BI tools for performance analysis, trend identification, and validation of allocation effectiveness over time, supporting strategic decision-making.

10. Common Questions

Q: Why does this module break down allocations to individual items (SKUs)? A: While planning often happens at a broader product style level, physical inventory movement requires precise instructions for every unique item, including its specific size and color. This module provides that granular detail, ensuring the right items are picked and shipped to the right stores for accurate fulfillment.

Q: How does this module help me understand my remaining warehouse inventory?

A: It calculates and reports the exact quantity of each product remaining in the warehouse *after* all approved allocations have been accounted for. This gives you a clear, real-time picture of available central stock for future planning or new allocation cycles, preventing accidental over-commitment.

Q: What types of reports can I get from this module? A: This module generates a wide range of critical reports, including detailed quantities of products distributed to stores, movements between warehouses, how well demand was filled (fill rates), details on overstocked items, and specialized data for optimizing warehouse dispatch operations.

Q: What if a product style has no individual items (SKUs) linked to it? A: While typically product styles have associated SKUs, if this occurs, the module will treat the parent style itself as the 'SKU' for allocation reporting purposes. However, for precise physical inventory management, it's always best practice to have individual SKUs defined for all sellable items.

Q: How does this module ensure that inventory is shipped efficiently? A: By providing precise, SKU-level instructions and specific outputs tailored for dispatch prioritization, this module helps warehouses streamline their picking and packing processes. This means less manual effort, fewer errors, and faster fulfillment of allocation orders, getting products to shelves quicker.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To consolidate, denormalize, and present all final inventory reallocation results in actionable, detailed reports.
- **Main Input:** Final allocation decisions for broader product styles, current store and refined warehouse inventory, and product family links (SKU to Style).
- **Main Output:** Precise, SKU-level allocation instructions; detailed distribution, warehouse movement, fill rate, and overstocking reports.

- **Warning Signs:** Incomplete or missing allocation data in the reports, or discrepancies between expected and reported inventory movements.
 - **Key Settings:** *None directly in this module, but heavily influenced by settings in upstream modules related to allocation, data preparation, and inventory refinement.*
-

Need Help? Contact your Inventory Planning or IT Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Reallocation FAQ

Version: 1.0 Last Updated: 2025-05-23

Purpose

This FAQ document provides clear, business-focused answers regarding the Reallocation algorithm, its processes, and its impact on inventory management and sales.

General

Q: What is the core purpose of the reallocation algorithm? **A:** The reallocation algorithm's core purpose is to optimize the distribution of existing inventory from our warehouses to stores. It aims to maximize sales by ensuring the right products are in the right stores at the right time, especially as items move through their selling lifecycle, minimizing stockouts and excess inventory.

Q: How does reallocation help our business? **A:** Reallocation helps our business by improving inventory efficiency, boosting sales by ensuring optimal stock levels at the store level, reducing the need for markdowns due to overstocking, and ultimately enhancing customer satisfaction by making desired products readily available.

Q: What is the main outcome of the reallocation process? **A:** The main outcome is a comprehensive plan for distributing specific SKUs from the warehouse to individual stores, detailing quantities to be moved, estimated fill rates, and updated warehouse inventory levels. It provides actionable recommendations for inventory transfers.

Q: Who primarily benefits from the reallocation process? **A:** Merchandisers, store operations teams, and supply chain managers benefit most directly. Merchandisers gain optimized inventory distribution, store teams experience fewer stockouts and better product availability, and supply chain managers get clear guidance on inventory movements.

Q: Does reallocation consider different product categories or only specific types of items? **A:** The reallocation process is designed to work across various product categories and styles. It considers detailed planogram configurations and style attributes from the 'Reallocation Prepare Data Module' to ensure appropriate stock distribution across different product types.

Configuration & Settings

Q: How does the system decide which items or stores to prioritize for reallocation? **A:** The system prioritizes items and stores using a sophisticated ranking algorithm in the 'Reallocation Ranking Module'. This considers multiple factors like overall stock levels, the product's available percentage (PSA), national revenue per day, and specific store-style revenue per day. Higher-performing or more critical combinations are addressed first.

Q: Can we influence the reallocation process for specific product categories or styles, for example, for promotional items? **A:** Yes, the 'Reallocation Prepare Data Module' allows for overrides for channel-specific styles and identifies in-season/out-of-season items. This enables some degree of influence on how certain product categories or styles are handled in the reallocation process.

Q: What is "warehouse inefficiency" and how does it affect the available stock for reallocation? **A:** "Warehouse inefficiency" refers to a configured amount of pieces per SKU that are deducted from raw warehouse stock to reflect practical inventory availability. The 'Reallocation Prepare Warehouse Inventory Module' accounts for this to ensure that only truly available stock is considered for reallocation, preventing promises of inventory that cannot be fulfilled.

Q: How does the system differentiate between "in-season" and "out-of-season" styles? **A:** The 'Reallocation Prepare Data Module' identifies styles as either in-season or out-of-season based on defined criteria. This distinction is crucial for applying appropriate allocation strategies, such as prioritizing replenishment for in-season top sellers or initiating pullbacks for out-of-season bottom sellers.

Q: How does the system determine the ideal number of unique items (OD) to stock? **A:** The 'Reallocation Sales Calculation Module' leverages the output from the Optimal Depth (OD) segmentation module. This module provides insights into the ideal number of unique items (OD) to stock in a category or display based on store revenue contributions and Attribute Group (AG) level Average Selling Price (ASP), guiding allocation decisions.

Data & Inputs

Q: What kind of data is needed for the reallocation process to run effectively? **A:** The 'Reallocation Prepare Data Module' loads and initializes a wide range of data, including active store lists, warehouse inventory (including Goods In Transit and Open Orders), planogram configurations, Ideal Size Set (ISS) definitions, and style attributes. Additionally, cleaned sales data and Annual Operating Plan (AOP) revenue data are crucial inputs.

Q: How is the "Rate of Sale (ROS)" calculated, and why is it important? **A:** ROS is calculated in two ways: * **First Month ROS:** The 'Reallocation First Month ROS Calculation Module' computes a baseline ROS at the store-style level for an initial period, filtering sales data by season and product availability. * **Period ROS:** The 'Reallocation Period ROS Calculation Module' distributes this style-level ROS among individual SKUs within a style based on their "adjusted size contribution" from the Ideal Size Set (ISS). ROS is crucial because it provides the speed at which products are sold, forming the foundation for demand forecasting and guiding allocation decisions.

Q: What is an “Ideal Size Set (ISS)” and how does it influence reallocation? **A:** An Ideal Size Set (ISS) is the best combination of sizes to maximize sales and minimize stockouts for a particular style. The ‘Reallocation Prepare Data Module’ defines ISS, and the ‘Reallocation Period ROS Calculation Module’ uses ISS to accurately distribute style-level ROS to individual SKUs, ensuring that the right sizes are allocated to meet expected demand.

Q: How does the system ensure the quality of the sales data used for reallocation? **A:** The ‘Reallocation Prepare Sales Module’ performs critical cleaning steps. It removes “liquidation” sales, which might skew typical sales patterns, and filters out “broken” sales data where product availability was inconsistent. This ensures that only reliable and representative sales data contributes to the analysis.

Q: What happens if there’s an issue with the Annual Operating Plan (AOP) data or other input dates? **A:** The ‘Reallocation Date Validation Module’ performs critical validations. It checks for logical ordering of dates, verifies the presence of required input periods, and assesses data “liveliness.” Any issues with AOP data or date consistency are marked as warnings or errors to prevent the reallocation process from running with flawed inputs.

Process & Logic

Q: How does the reallocation algorithm decide *where* to send inventory, and for which products? **A:** The core ‘Reallocation Allocation Module’ is an iterative engine. It leverages the outputs from the ‘Reallocation Ranking Module’ (which prioritizes store-styles) and the ‘Reallocation Sales Calculation Module’ (which projects demand). The allocation process simulates a depletion over time, adjusting stock based on projected sales, and making SKU-level decisions based on business rules and priorities like replenishment for top sellers or pullbacks for underperformers.

Q: What is the “depletion process” in reallocation? **A:** The “depletion process” within the ‘Reallocation Allocation Module’ simulates how inventory levels will decrease over time as products are sold. By projecting sales quantities (calculated by the ‘Reallocation Sales Calculation Module’) and adjusting stock levels accordingly, the system can determine future stock needs and make proactive reallocation decisions to maintain optimal inventory.

Q: How are “Top Sellers” and “Bottom Sellers” identified for reallocation? **A:** The ‘Reallocation Segmentation Module’ categorizes store-styles into “Top Sellers” and “Bottom Sellers.” This is based on analyzing historical sales data, including revenue per day, live days, and discount percentages, benchmarked against store-category or channel-category averages. This segmentation informs subsequent allocation strategies, prioritizing replenishment for top performers and potentially initiating pullbacks for underperformers.

Q: How does the system handle new stores or stores without sufficient sales history when calculating revenue? **A:** The ‘Reallocation Compute Revenue Module’ specifically addresses this by replicating revenue from reference stores for new stores or those lacking sufficient historical data. This ensures that even new locations can be included in the reallocation process with relevant demand projections.

Q: What are the different “iterations” that the reallocation process goes through?

A: The ‘Reallocation Allocation Module’ runs through several iterative allocations, each with specific criteria and priorities. These iterations can include: *

Replenishment: Ensuring sufficient stock for ongoing sales. * **Top-Seller**

Allocation: Prioritizing stock for high-performing items. * **Non-Pivotal Sizes:**

Allocating specific sizes that are not considered core. * **Pullbacks:** Identifying and initiating transfers of excess stock from stores back to the warehouse.

Outputs & Results

Q: What kind of information can I expect from the reallocation output reports?

A: The ‘Reallocation Output Module’ generates comprehensive reports. You can expect detailed distribution quantities for each SKU per store, records of warehouse movements, calculated fill rates (how well demand was met), overstocking details, and specific reallocation outputs tailored for depletion analysis and dispatch prioritization.

Q: Can I see how much inventory is left in the warehouse after reallocation recommendations are made? **A:** Yes, the ‘Reallocation Output Module’ calculates and reports on the remaining warehouse inventory after all recommended distributions are accounted for. This provides a clear picture of available stock for future allocations.

Q: What does “fill rate” mean in the reallocation output, and why is it important?

A: “Fill rate” indicates the percentage of requested or projected demand that the reallocation process was able to fulfill with available inventory. It’s important because it’s a key performance indicator of how effectively inventory is being managed to meet customer needs and maximize sales.

Q: How does reallocation help prevent overstocking at stores? **A:** The reallocation algorithm, especially through its iterative allocation process and the identification of “Bottom Sellers” by the ‘Reallocation Segmentation Module’, works to prevent overstocking. It can recommend pullbacks of excess inventory from stores back to the warehouse and ensures that allocations adhere to Optimal Depth (OD) guidelines, distributing only what is needed to meet projected demand without creating surplus.

Q: Where can I find the final detailed distribution quantities for each store and SKU? **A:** The ‘Reallocation Output Module’ is responsible for consolidating and persisting all final allocation results. It denormalizes parent-level SKU allocations to individual child SKUs and provides detailed distribution quantities, which are typically found in the comprehensive output reports generated by this module.

Error Handling & Edge Cases

Q: What kind of data issues can prevent the reallocation process from running successfully? **A:** The ‘Reallocation Date Validation Module’ checks for critical data consistency. Issues such as illogical analysis end dates, missing required input periods for the depletion forecast, or problems with the “liveliness” of SKUs/styles (based on stock and sales data) can halt the process or trigger warnings.

Q: How does the system handle “broken” sales data that might be inaccurate? **A:** The ‘Reallocation Prepare Sales Module’ includes robust cleaning steps specifically designed to filter out “broken” sales data, where product availability might have been inconsistent or other data integrity issues exist. This ensures that only reliable sales figures are used for ROS calculation and demand forecasting.

Q: Are there any types of sales that are intentionally excluded from the reallocation calculations? **A:** Yes, the ‘Reallocation Prepare Sales Module’ specifically removes “liquidation” sales. These types of sales often occur under abnormal conditions (e.g., clearance events) and do not represent typical sales patterns, so excluding them helps prevent skewed demand forecasts for ongoing reallocation efforts.

Q: What does it mean if a store-style is marked as “read-only”? **A:** The ‘Reallocation Compute Revenue Module’ identifies “read-only” store-styles based on season flags. This typically means that for certain seasonal products, their revenue contribution is fixed or handled differently, and they might not be subject to the same dynamic allocation adjustments as other items.

Q: What happens if the input dates (like analysis end date or period end dates) are illogical? **A:** The ‘Reallocation Date Validation Module’ performs strict checks for logical ordering of all date-related inputs. If illogical dates are detected, the module will flag these as warnings or errors, preventing the reallocation process from proceeding with potentially incorrect timeframes, ensuring data integrity for the depletion forecast.

Need More Help?

Please contact the support team for further assistance.

Reallocation Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

This documentation outlines the Reallocation algorithm, a critical component designed to optimize inventory distribution across a retail network. By strategically moving stock from overstocked to understocked locations, the algorithm aims to maximize sales velocity, reduce stockouts, and minimize excess inventory, ultimately improving profitability and customer satisfaction. Its key outcomes include optimized stock levels, enhanced sales performance, and efficient resource utilization.

3. Architecture Overview

The reallocation process begins with a crucial data validation phase, ensuring data integrity. This is followed by a comprehensive data preparation module that loads and structures all necessary master and transactional data. Sales data is then cleaned and prepared, forming the foundation for subsequent rate-of-sale (ROS) and revenue calculations. Warehouse inventory is also refined during this initial data setup. With all foundational data in place, modules compute various ROS metrics (first month and period-specific SKUs) and detailed revenue metrics, which are then used to segment and rank store-styles. The core allocation engine leverages these ranked and calculated metrics to iteratively distribute inventory. Finally, an output module consolidates and persists all allocation results and stock movements.

```
graph TD
    A[Reallocation Date Validation Module] -->|prerequisite| B[Reallocation Prepare Data Module];
    B -->|initializes| C[Reallocation Prepare Sales Module];
    B -->|initializes| D[Reallocation Prepare Warehouse Inventory Module];
    B -->|provides_config| E[Reallocation First Month ROS Calculation Module];
    B -->|provides_data| F[Reallocation Sales Calculation Module];
    B -->|provides_iss| G[Reallocation Period ROS Calculation Module];
    B -->|provides_master_data| H[Reallocation Compute Revenue Module];
```



```

B -->|provides_config| I(Reallocation Allocation Module);
B -->|provides_master_data| J(Reallocation Output Module);

C -->|prepares_sales| E;
C -->|prepares_sales| K(Reallocation Segmentation Module);
C -->|prepares_sales| H;
C -->|prepares_sales| L(Reallocation Ranking Module);

D -->|refines_inventory| I;

E -->|provides_ros| G;
F -->|provides_sales_qty| G;

H -->|provides_revenue_metrics| L;

K -->|informs_allocation| I;
G -->|provides_sku_ros| I;
L -->|prioritizes| I;

I -->|generates_results| J;

M[Utility Output Snapshot Module];
N[Utility Output Sync Module];

```

- **Reallocation Date Validation Module:** Validates the logical consistency and completeness of all date-related inputs and forecast periods.
- **Reallocation Prepare Data Module:** Loads and structures all foundational master and transactional data needed for the reallocation process.
- **Reallocation Prepare Warehouse Inventory Module:** Adjusts raw warehouse inventory by accounting for inefficiency and integrating purchase order quantities.
- **Reallocation First Month ROS Calculation Module:** Calculates the Rate of Sale (ROS) for the initial period at the store-style level as a baseline.
- **Reallocation Sales Calculation Module:** Computes total sales quantities per store-category-period, utilizing OD segmentation output and AOP revenue data.
- **Reallocation Period ROS Calculation Module:** Calculates SKU-level ROS across all defined periods by distributing style-level ROS based on Ideal Size Set (ISS) contributions.
- **Reallocation Prepare Sales Module:** Cleanses and filters raw sales data, removing anomalies like liquidation sales and inconsistent data.
- **Reallocation Segmentation Module:** Categorizes store-styles into segments such as “Top Sellers” and “Bottom Sellers” based on sales performance.
- **Reallocation Compute Revenue Module:** Calculates various “Revenue Per Day” metrics at different aggregation levels, crucial for ranking styles.
- **Reallocation Ranking Module:** Ranks store-styles for allocation prioritization, considering factors like stock, PSA, and various revenue-per-day metrics.
- **Reallocation Allocation Module:** The core engine that iteratively distributes inventory from warehouses to stores based on business rules, constraints, and demand.
- **Reallocation Output Module:** Consolidates, denormalizes, and persists all final allocation results and associated reports.
- **Utility Output Snapshot Module:** Captures snapshots of output data, typically for auditing, historical tracking, or debugging.

- **Utility Output Sync Module:** Handles the transfer of generated output data to external systems or storage.
-

4. Prerequisites & Setup

To run the reallocation pipeline, ensure the following are installed and configured:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Apache Maven:** Version 3.6.0 or higher.
- **Dependencies:** All project dependencies are managed via the `pom.xml` file. Maven will automatically download them during the build process.

Build Commands: To compile the project and build the executable JAR:

```
cd reallocation/  
mvn clean install
```

Running the Orchestrator: The main entry point for the reallocation pipeline is the `ModuleGroup` orchestrator. To execute the process:

```
java -jar target/reallocation-<version>.jar --properties-file  
args.properties
```

Ensure the `args.properties` file is correctly configured with all necessary parameters before running.

5. Folder & File Structure

The reallocation repository is structured as follows:

```
reallocation/  
├─ module-group/          # Contains the orchestrator for  
running the entire pipeline.  
├─ submodules/            # Houses the individual business  
logic modules (e.g., data prep, allocation).  
├─ args.properties        # Configuration file containing  
global parameters for the reallocation process.  
├─ common-data.json       # Defines the shared data model  
schema used across modules.  
├─ outputs/              # Directory where all final  
allocation reports and processed data are stored.
```

6. Global Configuration

The following parameters are defined in the `args.properties` file, governing the overall behavior of the reallocation algorithm:

| Setting Name | Default Value | Description | Valid Range/When to Change |
|--------------|---------------|--|----------------------------|
| | | <i>No global configuration parameters were provided in the input data.</i> | |

7. Shared Data Model

The `common-data.json` file defines the standard data structures and fields used for data exchange and consistency across the various modules in the reallocation pipeline.

| Field | Type | Description |
|-------|------|--|
| | | <i>No shared data model schema was provided in the input data.</i> |

8. Submodule Chapters

Reallocation Prepare Data Module

Module Name & Business Objective

Reallocation Prepare Data Module: This module is responsible for loading and initializing all master and transactional data required for the reallocation process. Its business objective is to aggregate and structure key data points—such as active stores, warehouse inventory (including Goods In Transit, Open Orders, and In-Transit Transfers), planogram configurations, Ideal Size Set (ISS) definitions, and style attributes—to form the foundational dataset for all subsequent allocation steps. It also identifies in-season and out-of-season styles and applies overrides for channel-specific styles, ensuring data readiness and relevance.

Required Information

- **Sales Data:** Historical sales, Product Set Available (PSA) benchmarks, liquidation sales flags.
- **Product Data:** Style attributes, SKU details, parent-child SKU relationships, Ideal Size Set (ISS) definitions, planogram configurations.
- **Operational Data:** Active store lists, warehouse inventory (on-hand, GIT, open orders, in-transit transfers), season lists, channel-specific style overrides, Annual Operating Plan (AOP) data.

Business Settings

- **Channel-Specific Style Overrides:** *Allows for specific styles to be treated differently based on the sales channel, e.g., excluding certain items from online reallocation.*
- **Season Definition:** *Defines the criteria for identifying in-season vs. out-of-season styles, influencing which items are prioritized for reallocation or depletion.*

Process Flow

graph TD

```
A[Raw Data Feeds: Master, Inventory, Sales] --> B[Load & Aggregate Data];  
B --> C[Apply Channel Overrides & Season Flags];  
C --> D[Structured & Initialized Dataset];
```

Step-by-Step Actions

1. **Load** all raw master data (e.g., stores, SKUs, styles) and *transactional data* (e.g., inventory, sales, transfers).
2. **Aggregate** inventory data across various sources including on-hand, Goods In Transit (GIT), Open Orders, and In-Transit Transfers to get a complete picture of warehouse stock.
3. **Apply** channel-specific style overrides to ensure certain styles are handled according to their designated sales channel.
4. **Identify** in-season and out-of-season styles based on defined criteria, which guides subsequent allocation logic.
5. **Initialize** Ideal Size Set (ISS) definitions and planogram configurations for use in sizing and display optimization.
6. **Cache** the structured and initialized dataset for efficient access by downstream modules.

Sample Scenario

Imagine a new collection (Style A) just launched, and an older collection (Style B) is moving into out-of-season. This module ensures that Style A is correctly flagged as “in-season” and Style B as “out-of-season,” while also ensuring that certain high-value styles are correctly assigned to the “online-only” channel. If initial inventory shows 1,000 units of Style A and 500 of Style B, and 50 units of Style C are marked “online-only,” the module correctly processes these flags and prepares the

foundational inventory. **Key Takeaway:** This module sets the stage by accurately preparing all required raw data into a usable format, reflecting business rules and current inventory status.

Results

- Consolidated master and transactional data (stores, SKUs, styles, inventory).
- Flagged in-season and out-of-season styles.
- Applied channel-specific style overrides.
- Loaded Ideal Size Set (ISS) and planogram configurations.

Unusual Cases

- **Missing or delayed data feed:** *Can lead to an incomplete or outdated dataset, impacting the accuracy of all subsequent allocation decisions.*
- **Incorrect master data definitions:** *E.g., wrong style attributes or ISS configurations, resulting in skewed allocation recommendations.*
- **Inconsistent inventory reporting:** *Discrepancies between physical stock and system records (GIT/transfers) leading to inaccurate warehouse availability.*

Next Steps

The prepared data serves as the foundation for various subsequent modules, including Reallocation Prepare Warehouse Inventory Module, Reallocation Prepare Sales Module, Reallocation First Month ROS Calculation Module, Reallocation Sales Calculation Module, Reallocation Period ROS Calculation Module, Reallocation Compute Revenue Module, Reallocation Ranking Module, Reallocation Allocation Module, and Reallocation Output Module.

Common Questions

1. **Why is data preparation so critical?** *It ensures the accuracy and relevance of all data used for decision-making, preventing flawed allocations.*
2. **How are new items handled?** *New items are loaded with their attributes and initial inventory, contributing to the dataset from their launch date.*
3. **What if a store's data is incomplete?** *Incomplete store data may lead to that store being excluded or receiving suboptimal allocations.*
4. **How often is this data updated?** *Data feeds are typically updated daily to ensure the most current information is used for reallocation.*
5. **Can we override system-defined seasons?** *Yes, channel-specific overrides allow for manual adjustments to season flags for specific business needs.*

Quick Reference

- **OD:** Optimum Depth (used implicitly via planogram configs)
- **OW:** Optimum Width (used implicitly via planogram configs)
- **ISS:** Ideal Size Set
- **GIT:** Goods In Transit
- **AOP:** Annual Operating Plan
- **SKU:** Stock Keeping Unit

Reallocation Prepare Warehouse Inventory Module

Module Name & Business Objective

Reallocation Prepare Warehouse Inventory Module: This module prepares and refines the warehouse inventory data for reallocation. Its primary function is to adjust raw warehouse stock by accounting for “warehouse inefficiency” (deducting a configured amount of pieces per SKU to reflect practical inventory availability). It also incorporates purchase order quantities and inward movements (from both SKU and style-level inputs) into the overall warehouse stock figures.

Required Information

- **Operational Data:** Raw warehouse inventory data, configured warehouse inefficiency deduction (pieces/SKU), purchase order quantities, inbound transfer data (SKU and style level).

Business Settings

- **Warehouse Inefficiency Deduction:** *A configured amount of pieces per SKU to be deducted from raw warehouse stock, reflecting items that are physically present but not readily available for picking or may be damaged/lost. This ensures only truly allocable stock is considered.*

Process Flow

graph TD

```
A[Raw Warehouse Inventory + POs] --> B{Deduct Inefficiency & Add Inbound};  
B --> C[Adjusted Allocable Warehouse Inventory];
```

Step-by-Step Actions

1. **Receive** *raw warehouse inventory data* for all SKUs and styles.
2. **Deduct** *a configured warehouse inefficiency amount* (e.g., 2 pieces per SKU) from the raw inventory total to reflect practical availability.
3. **Integrate** *incoming purchase order quantities* into the available warehouse stock.
4. **Incorporate** *inward movements* (such as returns or transfers in) at both SKU and style levels.
5. **Calculate** *the final adjusted allocable warehouse inventory* for each SKU.
6. **Cache** *the refined warehouse inventory* for use by the allocation engine.

Sample Scenario

A warehouse reports 1,000 units of SKU X. The business has a configured warehouse inefficiency deduction of 5 units per SKU. There are also 200 units of SKU X on an incoming Purchase Order (PO). * Raw Inventory: 1,000 units * Inefficiency Deduction: 5 units * PO: 200 units The module will calculate: 1,000 - 5

+ 200 = 1,195 units of *allocable* inventory for SKU X. **Key Takeaway:** This module ensures that only realistically available and allocable warehouse inventory is considered for distribution, preventing commitments for unavailable stock.

Results

- Adjusted, allocable warehouse inventory quantities per SKU.
- Updated stock figures incorporating inefficiency deductions and inbound movements.

Unusual Cases

- **Missing or delayed data feed:** *Warehouse inventory data not updated*, leading to allocations based on outdated stock levels.
- **Incorrect inefficiency configuration:** *An inaccurate deduction value* could either overstate or understate available inventory, leading to stockouts or overstock.
- **Mismatched PO data:** *Purchase order quantities not accurately reflecting actual inbound shipments*, causing discrepancies in projected available stock.

Next Steps

The refined warehouse inventory data is then provided directly to the Reallocation Allocation Module for distribution decisions.

Common Questions

1. **Why do we deduct for inefficiency?** *To account for practical realities like damaged goods, misplaced items, or stock that's not ready for immediate picking, ensuring only truly available units are allocated.*
2. **How is “warehouse inefficiency” defined?** *It's a configured parameter, often determined by historical analysis of inventory discrepancies or operational buffers.*
3. **Does this module consider outbound orders?** *This module focuses on inbound and current stock; outbound orders are typically managed by the core allocation engine during depletion.*
4. **What if a PO is delayed?** *The module processes the PO data it receives. If a PO is delayed after processing, the subsequent allocation run will reflect the updated status.*
5. **Can inefficiency be set per warehouse or SKU?** *The current description indicates a per-SKU deduction, but advanced configurations might allow for per-warehouse variations if required.*

Quick Reference

- **ROS:** Rate of Sale (not directly used here, but critical for overall process)
- **SKU:** Stock Keeping Unit
- **PO:** Purchase Order

Reallocation First Month ROS Calculation Module

Module Name & Business Objective

Reallocation First Month ROS Calculation Module: This module calculates the Rate of Sale (ROS) specifically for the first month (or a defined initial period) at the store-style level. This initial ROS serves as a baseline for projecting sales for subsequent periods in the depletion process. It filters sales data based on defined seasons and a Product Set Available (PSA) benchmark to ensure that only relevant and representative sales contribute to the ROS calculation.

Required Information

- **Sales Data:** Cleaned historical sales data (store-style level), Product Set Available (PSA) metrics.
- **Product Data:** Style definitions, season flags (in-season/out-of-season).
- **Operational Data:** Configured “first month” period definition, ROS style configurations.

Business Settings

- **First Month Period Definition:** *Specifies the duration (e.g., 4 weeks) and start date for the initial ROS calculation, ensuring consistency.*
- **Season Filtering:** *Ensures only sales data from relevant seasons (e.g., in-season styles) are included, preventing misleading ROS calculations from end-of-life or liquidation sales.*
- **Product Set Available (PSA) Benchmark:** *Used to filter out periods where products were not consistently available, ensuring ROS reflects true demand, not supply limitations.*

Process Flow

graph TD

A[Cleaned Sales Data + Season/PSA Filters] --> B{Filter & Aggregate Sales};

B --> C(Calculate First Month Store-Style ROS);

C --> D[Baseline Store-Style ROS];

Step-by-Step Actions

1. **Receive** *cleaned sales data* (from Reallocation Prepare Sales Module) and *configuration data* (from Reallocation Prepare Data Module).
2. **Filter** *sales records* to include only those within the defined “first month” period.
3. **Apply** *seasonality filters* to focus on sales of relevant (e.g., in-season) styles.
4. **Exclude** *sales data points* that do not meet the Product Set Available (PSA) benchmark, *to avoid skewed ROS from intermittent stock availability.*
5. **Aggregate** *filtered sales data* at the store-style level.
6. **Calculate** *the Rate of Sale (ROS)* for each store-style by dividing aggregated sales by the number of active selling days in the first month.

7. **Store** the calculated baseline store-style ROS for future use.

Sample Scenario

Store A sold 200 units of Style X in its first month, with a PSA of 90% (meaning it was in stock 90% of the time). Store B sold 100 units of Style Y in its first month, with a PSA of 40%. The module will calculate a high ROS for Style X in Store A, reflecting strong demand. For Style Y in Store B, despite lower sales, the module will recognize the low PSA, potentially adjusting its ROS calculation or flagging it for review if the PSA threshold isn't met, ensuring the ROS is truly reflective of demand, not just supply. **Key Takeaway:** This module provides a robust, demand-driven baseline ROS for initial sales periods, critical for accurate future demand projections.

Results

- Store-style level Rate of Sale (ROS) for the first month/initial period.
- Baseline ROS values used for subsequent period projections.

Unusual Cases

- **Missing or delayed data feed:** *If historical sales data is incomplete or late, the ROS calculation will be inaccurate or impossible.*
- **Insufficient sales history for new styles:** *New styles may have limited sales data for the "first month," leading to less reliable ROS calculations.*
- **Incorrect PSA configuration:** *If PSA thresholds are too strict or too lenient, it can lead to either discarding valid sales data or including sales that don't reflect true demand.*
- **Sales spikes due to promotions/events:** *Unusual sales spikes in the first month not accounted for, potentially inflating ROS beyond typical demand.*

Next Steps

The calculated first month ROS values are passed to the Reallocation Period ROS Calculation Module as a base for SKU-level ROS distribution across all periods.

Common Questions

1. **What if a style had very low sales in its first month?** *The module calculates ROS based on available data. Very low sales will result in a low ROS, potentially leading to lower allocation priority.*
2. **How is "first month" defined?** *It's a configurable period, typically the first 4 weeks or a similar initial duration from the style's Product Available Date.*
3. **Why filter by PSA?** *To ensure the calculated ROS reflects true demand, not limitations caused by out-of-stock situations. If a product was unavailable, its low sales don't accurately represent its selling potential.*
4. **Does this consider promotional periods?** *By default, it uses raw sales. Specific configurations or external inputs might be needed to account for or normalize promotional sales impacts on ROS.*

5. **Is this a forecast or an actual rate?** *It's an actual historical rate of sale for a specific initial period, which then serves as a foundation for future forecasts.*

Quick Reference

- **ROS:** Rate of Sale
- **PSA:** Product Set Available
- **OD:** Optimum Depth (relevant for sales calculation in next steps)

Reallocation Sales Calculation Module

Module Name & Business Objective

Reallocation Sales Calculation Module: This module computes the total sales quantity for each store-category-period combination. It leverages output from the Optimal Depth (OD) segmentation module (specifically, final store revenue contribution and AG-level Average Selling Price - ASP) along with AOP (Annual Operating Plan) revenue data. The calculated sales quantities are crucial for projecting future demand and guiding allocation decisions, especially in depletion scenarios.

Required Information

- **Sales Data:** Annual Operating Plan (AOP) revenue data (store-category-period).
- **Product Data:** Attribute Group (AG) level Average Selling Price (ASP) (from OD segmentation).
- **Operational Data:** Store revenue contribution (from OD segmentation).

Business Settings

- **AOP Integration:** *Ensures sales quantities align with broader business revenue targets*, providing a strategic top-down input for demand projection.
- **OD Segmentation Data Reliance:** *Leverages insights from OD segmentation* to incorporate store-specific revenue potential and average product pricing for accurate sales quantity derivations.

Process Flow

graph TD

```
A[AOP Revenue + OD Segmentation Data (Revenue/ASP)] --> B[Calculate Store-Category-Period Sales];
B --> C[Projected Sales Quantities];
```

Step-by-Step Actions

1. **Retrieve Annual Operating Plan (AOP) revenue data** for each store, category, and future period.

2. **Access** *final store revenue contribution and Attribute Group (AG) level Average Selling Price (ASP)* from the Optimal Depth (OD) segmentation module's output.
3. **Calculate** *the projected sales quantity* for each store-category-period by dividing the relevant AOP revenue (potentially adjusted by store contribution) by the AG-level ASP. *This converts revenue targets into unit targets.*
4. **Adjust** *calculated sales quantities* as needed based on any other applicable business rules or factors (e.g., promotional uplifts or markdown impacts).
5. **Store** *the computed sales quantities* for use in depletion forecasting and allocation planning.

Sample Scenario

Store X's AOP target for the footwear category in Q3 is \$50,000. The OD segmentation module indicates that footwear in this store has a 10% higher revenue contribution than average, and the average selling price (ASP) for the footwear AG is \$100. The module would calculate the sales quantity: $((\$50,000 * 1.10) / \$100) = 550$ units. **Key Takeaway:** This module translates high-level revenue plans into specific unit sales projections, enabling a tangible basis for inventory allocation and depletion.

Results

- Calculated total sales quantities per store-category-period.
- Projected demand in units for future periods, based on AOP and OD insights.

Unusual Cases

- **Missing or delayed data feed:** *If AOP data or OD segmentation outputs are missing*, the module cannot compute sales quantities.
- **Inaccurate AOP targets:** *If AOP targets are unrealistic or not updated*, the resulting sales quantities will be skewed.
- **Fluctuating ASPs:** *Significant and unpredicted changes in Average Selling Price (ASP) within an Attribute Group* could lead to inaccurate unit conversions.
- **New stores/categories without AOP:** *For new entities, without established AOP or OD data*, proxy values or manual overrides may be needed to compute sales.

Next Steps

The calculated sales quantities are primarily used by the Reallocation Period ROS Calculation Module to refine ROS projections and the Reallocation Allocation Module for depletion modeling.

Common Questions

1. **What is the source of the AOP data?** *AOP data typically comes from financial planning or merchandising departments, representing high-level sales targets.*
2. **How accurate are these sales quantities?** *Their accuracy depends heavily on the quality and realism of the input AOP and OD segmentation data.*

3. **Why do we use ASP from Attribute Groups?** *Using AG-level ASP allows for a more granular and representative average price for similar products within a category, rather than a single category-wide average.*
4. **Can we adjust these quantities manually?** *While the module automates calculation, business users often review and can apply overrides or adjustments through configuration settings.*
5. **How does this impact depletion?** *These sales quantities drive the simulated demand in depletion models, determining how much stock is 'sold' over time and thus how much can be reallocated.*

Quick Reference

- **OD:** Optimum Depth
- **OW:** Optimum Width
- **ROS:** Rate of Sale (indirectly influenced)
- **AG:** Attribute Group
- **ASP:** Average Selling Price
- **AOP:** Annual Operating Plan

Reallocation Period ROS Calculation Module

Module Name & Business Objective

Reallocation Period ROS Calculation Module: This module calculates the Rate of Sale (ROS) at the SKU level for all defined periods. It achieves this by taking the style-level ROS (computed in the “First Month ROS Calc Module”) and distributing it among the SKUs within that style based on their “adjusted size contribution” from the Ideal Size Set (ISS) output. The SKU ROS values are normalized to ensure they sum up to the overall style ROS for each period.

Required Information

- **Sales Data:** Style-level ROS (from Reallocation First Month ROS Calculation Module), total sales quantities per store-category-period (from Reallocation Sales Calculation Module).
- **Product Data:** Ideal Size Set (ISS) data including “adjusted size contribution” per SKU.
- **Operational Data:** Defined periods for ROS calculation.

Business Settings

- **Ideal Size Set (ISS) Contribution:** *The weighting or proportion attributed to each size within a style, ensuring that ROS is accurately distributed based on historical or planned size performance.*
- **Normalization Rules:** *Ensures that the sum of individual SKU ROS for a style matches the overall style ROS, maintaining data integrity.*

Process Flow

graph TD

```
A[Style-Level ROS + ISS Adjusted Size Contribution] -->
B[Distribute ROS to SKUs & Normalize];
B --> C[SKU-Level ROS for All Periods];
```

Step-by-Step Actions

1. **Receive** *style-level ROS values* (from the Reallocation First Month ROS Calculation Module) for relevant store-styles.
2. **Retrieve** *Ideal Size Set (ISS) data*, specifically the “adjusted size contribution” for each SKU within a style. *This contribution reflects how much each size typically sells relative to others in that style.*
3. **Distribute** *the style-level ROS* to individual SKUs within that style by multiplying the style ROS by each SKU’s adjusted size contribution. *This provides an initial SKU-level ROS for the period.*
4. **Normalize** *the calculated SKU ROS values* to ensure that the sum of all SKU ROS within a style for a given period precisely matches the original style-level ROS. *This prevents over or under-forecasting at the SKU level.*
5. **Repeat** *this process* for all defined future periods, adjusting for any period-specific sales quantities provided by the Reallocation Sales Calculation Module.
6. **Store** *the final SKU-level ROS values* for all periods.

Sample Scenario

A style has a store-level ROS of 10 units/week. Its ISS data shows that Size M has an adjusted size contribution of 0.40 (40%), Size L has 0.30 (30%), and Size S has 0.30 (30%). * SKU M ROS: 10 units/week * 0.40 = 4 units/week * SKU L ROS: 10 units/week * 0.30 = 3 units/week * SKU S ROS: 10 units/week * 0.30 = 3 units/week Total SKU ROS: 4 + 3 + 3 = 10 units/week, which matches the style ROS.

Key Takeaway: This module translates high-level style demand into granular SKU-level demand, critical for precise inventory allocation and ensuring the right sizes are in stock.

Results

- SKU-level Rate of Sale (ROS) for all defined periods.
- Normalized ROS values ensuring consistency between SKU and style levels.

Unusual Cases

- **Missing or delayed data feed:** *If style-level ROS or ISS data is unavailable, SKU-level ROS cannot be computed accurately.*
- **Inaccurate ISS contributions:** *If the adjusted size contributions are not representative of actual sales trends, the SKU-level ROS will be misaligned, leading to stock imbalances by size.*
- **New styles without established ISS:** *For new styles lacking historical ISS data, default or category-average ISS may be used, which could be less accurate.*

- **Extreme outliers in style ROS:** *Anomalous style ROS values can propagate and create unrealistic SKU-level ROS.*

Next Steps

The SKU-level ROS values are a crucial input for the Reallocation Allocation Module to simulate depletion and make iterative allocation decisions.

Common Questions

1. **Why is SKU-level ROS needed?** *Allocation decisions are made at the SKU level. Having granular ROS ensures that specific sizes are allocated based on their individual demand, not just the style's overall performance.*
2. **How is “adjusted size contribution” determined?** *It's typically derived from historical sales data, analyzing the proportion of each size sold within a style.*
3. **What if a style has no ISS data?** *A fallback mechanism, such as using a generic category-level ISS or a default distribution, would likely be implemented.*
4. **Does this account for future trends?** *The module primarily distributes historical style ROS. Future trends would need to be incorporated at the style ROS calculation phase or through manual adjustments to the ISS data.*
5. **What if a size is rarely sold?** *If its ISS contribution is very low, its calculated ROS will also be low, indicating minimal demand and lower allocation priority.*

Quick Reference

- **ROS:** Rate of Sale
- **SKU:** Stock Keeping Unit
- **ISS:** Ideal Size Set
- **AG:** Attribute Group (ASP from AG is used in sales calculation, which feeds into this)

Reallocation Prepare Sales Module

Module Name & Business Objective

Reallocation Prepare Sales Module: This module focuses on preparing and cleaning raw sales data to ensure its quality and relevance for reallocation analysis. It performs several crucial cleaning steps: 1. Removes “liquidation” sales, which might skew typical sales patterns. 2. Filters out “broken” sales data, where product availability was inconsistent, ensuring only reliable sales are used. 3. Computes SKU-level Rate of Sale (ROS) across multiple historical periods, providing a foundation for demand forecasting. It also identifies and segregates “priority sales” based on seasonal flags for specific analyses.

Required Information

- **Sales Data:** Raw historical sales transactions (including liquidation flags, availability indicators).

- **Product Data:** SKU details, seasonal flags for styles/SKUs.

Business Settings

- **Liquidation Sales Flag:** *Defines criteria to identify and exclude sales that occurred during clearance or liquidation events, as these sales volumes often do not represent typical demand.*
- **Broken Sales Criteria:** *Rules for identifying and filtering out sales data where product availability was inconsistent or unreliable, ensuring the remaining data accurately reflects demand under normal conditions.*
- **Priority Sales Definition:** *Criteria (e.g., specific seasonal flags) to categorize sales that require special consideration in certain analyses.*

Process Flow

graph TD

```
A[Raw Sales Data] --> B{Cleanse Sales: Remove Liquidation & Broken};
B --> C{Compute SKU-Level ROS};
C --> D[Cleaned Sales Data + SKU-Level ROS];
```

Step-by-Step Actions

1. **Load** *raw historical sales data* for all relevant SKUs and stores.
2. **Filter out** *“liquidation” sales records* to prevent atypical, high-volume, or low-price sales from distorting true demand patterns.
3. **Identify and remove** *records flagged as “broken sales”, i.e., instances where sales occurred during periods of inconsistent product availability, to ensure ROS calculations are based on reliable demand signals.*
4. **Compute** *SKU-level Rate of Sale (ROS) for each SKU across multiple historical periods, providing a foundation for demand forecasting.*
5. **Segregate** *sales data into “priority sales” (e.g., in-season full-price sales) based on configured seasonal flags and other criteria for focused analysis.*
6. **Cache** *the cleaned sales data and computed ROS for consumption by other modules.*

Sample Scenario

A store shows 50 units sold of SKU X last month, but 30 of those were during a 70% off liquidation event. Additionally, 5 units were sold when the SKU was technically out of stock for 3 days of that month (a “broken sale”). * The module will remove the 30 liquidation sales. * It will remove the 5 broken sales. * The actual reliable sales used for ROS calculation will be $50 - 30 - 5 = 15$ units. This ensures that the ROS calculated for SKU X is based only on regular, reliable demand. **Key Takeaway:** This module is critical for ensuring the integrity and relevance of sales data, leading to more accurate demand forecasts and allocation decisions.

Results

- Cleaned and filtered historical sales data (excluding liquidation and broken sales).
- Computed SKU-level Rate of Sale (ROS) across historical periods.
- Segregated “priority sales” for specific analyses.

Unusual Cases

- **Missing or delayed data feed:** *Incomplete sales data* can lead to inaccurate ROS calculations and poor allocation decisions.
- **Incorrect flagging of liquidation sales:** *If liquidation sales are not properly identified*, they can artificially inflate ROS.
- **Overly aggressive “broken sales” filtering:** *Too strict criteria may discard valid sales data*, underestimating true demand.
- **Insufficient historical sales data for new products:** *Newly launched SKUs may lack enough history* for robust ROS calculation.

Next Steps

The cleaned sales data is consumed by the Reallocation First Month ROS Calculation Module, Reallocation Segmentation Module, Reallocation Compute Revenue Module, and Reallocation Ranking Module to perform their respective analyses.

Common Questions

1. **What constitutes “liquidation” sales?** *Sales marked as part of a final clearance, end-of-season, or deep discount event, where the primary goal is inventory reduction rather than normal demand fulfillment.*
2. **Why remove “broken” sales?** *To ensure ROS reflects actual demand under normal supply conditions. If a product was intermittently out of stock, its sales might be artificially low, not reflecting true customer interest.*
3. **How is SKU-level ROS different from style-level ROS?** *SKU-level ROS is granular to individual sizes/colors, while style-level ROS is for the overall product style. Both are needed for different stages of allocation.*
4. **Can I see the sales data before cleaning?** *Typically, the raw data would be available in the source system, but this module processes it internally for reallocation purposes.*
5. **How are new stores or products handled if they have no sales history?** *They would typically rely on category averages, reference stores, or initial allocation strategies until sufficient sales data accumulates.*

Quick Reference

- **ROS:** Rate of Sale
- **SKU:** Stock Keeping Unit
- **AG:** Attribute Group (indirectly, as it affects product attributes)

Reallocation Segmentation Module

Module Name & Business Objective

Reallocation Segmentation Module: This module categorizes store-styles into distinct segments, primarily “Top Sellers” and “Bottom Sellers”. This segmentation is critical as it informs subsequent allocation strategies, allowing the system to prioritize replenishment for top-performing items and potentially initiate pullbacks for underperforming ones. Segmentation is based on analyzing historical sales data, including revenue per day, live days, and discount percentages, benchmarked against store-category or channel-category averages.

Required Information

- **Sales Data:** Cleaned historical sales data (revenue per day, live days, discount percentages) at the store-style level.
- **Operational Data:** Store-category or channel-category average benchmarks.

Business Settings

- **Top Seller Thresholds:** *Configurable criteria (e.g., revenue per day percentile, discount percentage) to classify a store-style as a “Top Seller,”* indicating high performance and priority.
- **Bottom Seller Thresholds:** *Configurable criteria (e.g., low revenue per day, high discount percentage, prolonged slow sales) to classify a store-style as a “Bottom Seller,”* indicating underperformance and potential for pullback.
- **Benchmarking Method:** *Specifies how a store-style’s performance is compared (e.g., against its own store’s category average, or against a broader channel average),* ensuring fair and relevant comparisons.

Process Flow

graph TD

```
A[Cleaned Sales Data (Revenue/Day, Live Days, Discount %)] --> B[Benchmark Against Averages];
B --> C[Apply Top/Bottom Seller Thresholds];
C --> D[Segmented Store-Styles];
```

Step-by-Step Actions

1. **Receive** *cleaned historical sales data* at the store-style level, including metrics such as revenue per day, number of live selling days, and average discount percentage applied.
2. **Benchmark** *each store-style’s performance* against relevant averages (e.g., the average revenue per day for its category within that specific store, or across the entire sales channel). *This provides context for performance evaluation.*
3. **Apply** *pre-defined “Top Seller” thresholds* (e.g., if revenue per day is in the top 20% of its category, or if discount percentage is below a certain value) to identify and flag high-performing store-styles.

4. **Apply pre-defined “Bottom Seller” thresholds** (e.g., if revenue per day is in the bottom 10%, or if discount percentage is excessively high, indicating distress) to identify and flag underperforming store-styles.
5. **Assign each store-style to its corresponding segment** (Top Seller, Bottom Seller, or Regular).
6. **Store the segmented store-style information** for use in allocation strategy.

Sample Scenario

A store has 100 units of Style X and 50 units of Style Y. Style X, over 30 live days, generated \$3,000 in revenue with an average 5% discount, placing it in the top 15% of its category’s revenue per day. Style Y, over 60 live days, generated \$500 in revenue with an average 40% discount, placing it in the bottom 5% of its category. * Module classifies Style X as a “Top Seller” due to high revenue/day and low discount. * Module classifies Style Y as a “Bottom Seller” due to low revenue/day and high discount. **Key Takeaway:** This module provides intelligent classification of store-styles, enabling the allocation engine to prioritize inventory movement based on performance, rather than just raw demand.

Results

- Store-styles categorized into “Top Sellers,” “Bottom Sellers,” and regular performers.
- Performance indicators and segmentation flags for each store-style.

Unusual Cases

- **Missing or delayed data feed:** *Insufficient sales history* for accurate segmentation, especially for new stores or styles.
- **Inaccurate benchmarking data:** *If category averages or channel averages are skewed*, the segmentation might be inaccurate.
- **Overly rigid or too lenient thresholds:** *Poorly defined thresholds* can lead to misclassification, either over-prioritizing or under-prioritizing items.
- **Short-term promotions skewing data:** *Temporary sales spikes or dips due to promotions* might temporarily misclassify a store-style if not normalized.

Next Steps

The segmentation information (Top/Bottom Sellers) is directly used by the Reallocation Allocation Module to inform strategic allocation decisions and potential pullbacks.

Common Questions

1. **Why segment store-styles?** *It allows for differentiated allocation strategies. Top sellers get prioritized replenishment, while bottom sellers might be targeted for pullbacks to free up inventory.*
2. **How often is segmentation updated?** *Typically, segmentation is rerun periodically (e.g., weekly or monthly) to reflect changing sales performance.*

3. **Can a store-style change segments?** *Yes, as sales performance evolves, a store-style can move between “Top Seller,” “Regular,” and “Bottom Seller” categories.*
4. **What metrics are most important for segmentation?** *Revenue per day and discount percentage are often key, as they reflect both sales volume and profitability/distress.*
5. **How are new stores/styles segmented if they have no history?** *They might initially be classified as “Regular” or assigned based on reference store data until sufficient performance history is built.*

Quick Reference

- **ROS:** Rate of Sale (underlying data for revenue per day)
- **AG:** Attribute Group (used for category averages)
- **OD:** Optimum Depth (might inform ideal performance levels)

Reallocation Compute Revenue Module

Module Name & Business Objective

Reallocation Compute Revenue Module: This module calculates various “Revenue Per Day” metrics at different aggregation levels (store-style, national, and rolled-up attribute groups). These metrics are crucial for ranking styles and making informed allocation decisions. It processes sales data to determine revenue contribution, handles new stores by replicating revenue from reference stores, and identifies “read-only” store-styles based on season flags. It also calculates size depth for store-styles based on pivotal contributions.

Required Information

- **Sales Data:** Cleaned sales data (from Reallocation Prepare Sales Module), including priority sales.
- **Product Data:** Style attributes, SKU details, seasonal flags.
- **Operational Data:** Store information, reference store mappings (for new stores), pre-computed size depth (from Reallocation Prepare Data Module).

Business Settings

- **Reference Store Mapping:** *Allows new stores without sufficient sales history to inherit revenue patterns from established, similar “reference” stores, ensuring they are included in allocation considerations.*
- **Read-Only Style Definition:** *Criteria (e.g., end-of-season, specific liquidation flags) to mark store-styles as “read-only,” meaning they are not subject to standard allocation or pullbacks.*

Process Flow

graph TD

A[Cleaned Sales Data + Store/Style Info] --> B[Calculate Revenue/Day at Various Levels];

B --> C[Handle New Stores & Read-Only Styles];

C --> D[Aggregated Revenue Metrics + Size Depth];

Step-by-Step Actions

1. **Receive** *cleaned sales data* (including “priority sales”) and *master data* (store and style information).
2. **Calculate** *Revenue Per Day* at the **store-style level** by dividing total revenue by the number of live selling days for each store-style.
3. **Calculate** *National Revenue Per Day* by aggregating revenue across all stores for each style.
4. **Compute** *rolled-up Revenue Per Day* at the **Attribute Group (AG) level**, providing an average for similar products.
5. **Identify** *new stores* and, if configured, **replicate** *revenue metrics from designated “reference” stores* to provide a baseline for allocation.
6. **Flag** *store-styles as “read-only”* if they meet criteria (e.g., end-of-season, designated for markdown only), *meaning they will not receive new allocations*.
7. **Calculate** *size depth for store-styles* based on pre-computed pivotal contributions (often derived from Ideal Size Set - ISS data), *ensuring adequate stock breadth*.
8. **Store** *all calculated revenue metrics and updated store-style objects* for ranking and allocation.

Sample Scenario

A new store opens without any sales history. The module identifies it as “new” and, based on configuration, copies the revenue-per-day metrics of a similar, established “reference” store. For an existing store, Style Z has a national revenue/day of \$500, but in this specific store, its revenue/day is only \$50. The module will capture both national and local metrics for ranking. If Style A is marked “end-of-season,” this module ensures it’s flagged as “read-only,” preventing new allocations. **Key Takeaway:** This module provides comprehensive performance insights by quantifying sales velocity at multiple levels, crucial for objective ranking and allocation decisions.

Results

- “Revenue Per Day” metrics at store-style, national, and Attribute Group (AG) levels.
- Handled revenue for new stores (via reference store replication).
- Identified and flagged “read-only” store-styles.
- Calculated size depth for store-styles based on pivotal contributions.

Unusual Cases

- **Missing or delayed data feed:** *If cleaned sales data is incomplete or late, revenue calculations will be inaccurate.*
- **No suitable reference store for new stores:** *If a new store cannot be mapped to a relevant reference store, its initial revenue metrics may be estimated or set to zero, impacting initial allocations.*
- **Incorrect “read-only” flags:** *Misidentified read-only styles could either receive unnecessary stock or be prematurely removed from allocation consideration.*
- **Data quality issues in sales data:** *Inaccurate pricing or quantity data can lead to incorrect revenue calculations.*

Next Steps

The calculated revenue metrics are a primary input for the Reallocation Ranking Module, where they are used to prioritize store-styles for allocation.

Common Questions

1. **Why calculate Revenue Per Day at different levels?** *National metrics provide a general sense of style performance, while store-style specific metrics highlight local demand. AG-level provides insights into broader product categories.*
2. **How are new stores handled without sales history?** *They are typically assigned a “reference store” whose sales patterns are used as a proxy until the new store builds its own history.*
3. **What does “read-only” mean for a style?** *It means the style will not be considered for new allocations or pullbacks through the standard process, usually because it’s being phased out or is a special item.*
4. **How is “size depth” used?** *It ensures that when a style is allocated, its sizes are distributed proportionally to maximize sales potential based on the Ideal Size Set (ISS) rather than just total units.*
5. **Is this module related to profit?** *While it calculates revenue, it does not directly calculate profit. It focuses on the sales velocity and value of products, which are key drivers of profitability.*

Quick Reference

- **ROS:** Rate of Sale (closely related to Revenue Per Day)
- **AG:** Attribute Group
- **ISS:** Ideal Size Set (influences size depth)

Reallocation Ranking Module

Module Name & Business Objective

Reallocation Ranking Module: This module is responsible for ranking store-styles to prioritize their allocation. The ranking algorithm considers multiple factors such as overall stock, PSA (Product Set Available) percentage, national revenue per day, and store-style specific revenue per day (computed across various aggregation

levels). It ensures that higher-performing or more critical store-style combinations are addressed first during the allocation iterations. It also computes SKU-level and store-category-size rankings for non-pivotal (NP) items.

Required Information

- **Sales Data:** Revenue per day (store-style, national, AG level) (from Reallocation Compute Revenue Module), unliquidation sales data (from Reallocation Prepare Sales Module).
- **Product Data:** Product Set Available (PSA) percentage.
- **Operational Data:** Overall stock levels (from Reallocation Prepare Data Module).

Business Settings

- **Ranking Weights:** *Configurable weights for each ranking factor* (e.g., revenue per day, PSA, stock) to prioritize certain business objectives.
- **Non-Pivotal (NP) Item Definition:** *Criteria to define items that are not core or fast-moving*, requiring a separate ranking approach for efficient allocation.
- **Ranking Thresholds:** *Minimum/maximum values for metrics* (e.g., minimum ROS) to include items in ranking, preventing consideration of extremely poor performers.

Process Flow

graph TD

```
A[Revenue/Day (Various Levels) + Stock + PSA + Sales Data] --> B[Apply Ranking Algorithm & Weights];
B --> C[Calculate SKU-Level & NP Rankings];
C --> D[Ranked Store-Styles & SKUs];
```

Step-by-Step Actions

1. **Receive** *all relevant data points* including store-style, national, and AG-level Revenue Per Day, overall stock levels, and Product Set Available (PSA) percentages.
2. **Apply** *the primary ranking algorithm* to store-styles, combining multiple factors (e.g., a weighted average of Revenue Per Day, PSA, and stock-to-sales ratios). *This generates a comprehensive priority score.*
3. **Prioritize** *store-styles* based on their calculated ranking score, ordering them from highest to lowest priority for allocation. *Higher-ranked items will be considered first.*
4. **Compute** *SKU-level rankings* for individual sizes and colors within a style, often based on their own sales velocity and contribution to overall style performance.
5. **Calculate** *specific rankings for non-pivotal (NP) items* at the store-category-size level, recognizing they may have different performance drivers or allocation rules. *This ensures less critical items are still managed effectively.*
6. **Store** *the ordered list of ranked store-styles and SKUs* for consumption by the allocation engine.

Sample Scenario

Style A in Store X has a high national revenue/day (\$500), a good store-specific revenue/day (\$100), high PSA (95%), and low stock (5 units). Style B in Store Y has a low national revenue/day (\$50), very low store-specific revenue/day (\$5), low PSA (60%), and high stock (50 units). * The module's algorithm (with weighted factors) will rank Style A in Store X much higher, indicating it's a top priority for replenishment due to its strong performance and low stock. * Style B in Store Y will be ranked much lower, possibly indicating it's a candidate for pullback or slow allocation. **Key Takeaway:** This module ensures that the allocation engine intelligently prioritizes inventory movement to maximize the impact on sales and minimize risk, focusing on the most critical and highest-potential opportunities.

Results

- Prioritized list of store-styles based on a composite ranking score.
- SKU-level and store-category-size rankings for non-pivotal (NP) items.

Unusual Cases

- **Missing or delayed data feed:** *If any input data (revenue, stock, PSA) is incomplete, the ranking will be inaccurate or biased.*
- **Inappropriate ranking weights:** *Incorrectly set weights for ranking factors can lead to misprioritization, e.g., prioritizing low-profit items over high-profit ones.*
- **New items without historical data:** *New SKUs/styles may lack sufficient data for robust ranking, requiring default or category-based ranking until history is built.*
- **Sudden market shifts:** *Unforeseen demand spikes or drops (e.g., due to competitor actions, social media trends) may not be immediately reflected in historical metrics, leading to suboptimal rankings.*

Next Steps

The ranked list of store-styles and SKUs is directly consumed by the Reallocation Allocation Module to guide the iterative allocation process.

Common Questions

1. **How often are rankings updated?** *Rankings are typically recalculated before each reallocation run to reflect the latest performance data.*
2. **Can I adjust the weighting factors?** *Yes, the weighting factors for different metrics (e.g., ROS, PSA, stock) are usually configurable to align with changing business priorities.*
3. **What if a style is consistently ranked low?** *Consistently low rankings might indicate a need for further investigation, such as markdowns, transfers out, or discontinuation.*
4. **How do “non-pivotal” items differ in ranking?** *Non-pivotal items might be ranked based on different criteria, potentially less focused on rapid turns and more on maintaining breadth or minimizing overstock.*

5. **Does ranking consider future trends?** *Primarily, it uses historical performance. Future trends might be incorporated if they influence the input metrics (like adjusted ROS), but the core ranking is backward-looking.*

Quick Reference

- **ROS:** Rate of Sale
- **PSA:** Product Set Available
- **AG:** Attribute Group
- **NP:** Non-Pivotal

Reallocation Allocation Module

Module Name & Business Objective

Reallocation Allocation Module: This module serves as the core allocation engine for the reallocation process. It runs a series of iterative allocations to distribute inventory from warehouses to stores, adhering to various business rules and constraints. It simulates a depletion process over time, adjusting stock levels based on sales, and making allocation decisions for individual SKUs at each store. It supports different allocation “iterations” (e.g., replenishment, top-seller, non-pivotal sizes, pullbacks) each with specific criteria and priorities. The module also handles the pre- and post-allocation stock reporting for depletion analysis.

Required Information

- **Operational Data:** Refined warehouse inventory (from Reallocation Prepare Warehouse Inventory Module), planogram rules, Ideal Size Set (ISS) definitions (from Reallocation Prepare Data Module).
- **Sales Data:** SKU-level ROS for all periods (from Reallocation Period ROS Calculation Module), segmentation (Top/Bottom Sellers from Reallocation Segmentation Module), ranked store-styles (from Reallocation Ranking Module).
- **Business Rules:** Allocation constraints (min/max stock, fill rates), iteration priorities.

Business Settings

- **Allocation Iteration Sequence:** *Defines the order and priority of different allocation passes* (e.g., replenish core sizes first, then top-sellers, then non-pivotal, then consider pullbacks).
- **Depletion Logic:** *Rules for simulating sales over time* based on ROS, reducing available stock and triggering reorder/reallocation points.
- **Min/Max Stock Levels:** *Configurable thresholds for desired inventory levels* at stores, guiding how much to allocate or pull back.
- **Fill Rate Targets:** *Desired percentage of demand to be met*, influencing allocation quantity.
- **Pullback Thresholds:** *Criteria for initiating stock removal from stores* (e.g., very slow sales, high overstock, bottom seller status).

Process Flow

graph TD

A[Warehouse Inventory + ROS + Rankings + Segmentation + Business Rules] --> B[Iterative Allocation & Depletion Simulation];

B --> C[Adjust Stock & Make Allocation Decisions];

C --> D[Proposed Allocations & Updated Stock Status];

Step-by-Step Actions

1. **Initialize** *current store and warehouse stock levels* and **define** the *depletion timeline* based on input periods.
2. **Iterate** through *pre-defined allocation passes* (e.g., replenishment, top-seller, non-pivotal, pullbacks), prioritized by business rules:
 - **Simulate Depletion:** For each period, **reduce** *store stock based on SKU-level ROS, mimicking sales and consumption*.
 - **Identify Needs/Surpluses:** **Determine** *if a store needs stock* (below min stock, high ROS) or *has a surplus* (above max stock, low ROS).
 - **Prioritize Allocation:** **Consult** *ranked store-styles and segmentation flags* (Top/Bottom Sellers) to decide which items to address first.
 - **Allocate Stock:** If needed, **distribute** *available warehouse inventory* to stores, adhering to Ideal Size Set (ISS) contributions, planogram rules, and min/max stock constraints. *Quantities are determined to maximize sales while minimizing overstock*.
 - **Initiate Pullbacks:** If a store has a surplus and meets pullback criteria (e.g., a “Bottom Seller” with high inventory), **recommend** *transferring stock back to the warehouse* or another store.
3. **Adjust** *warehouse and store inventory figures* after each iteration and simulated depletion step.
4. **Record** *all proposed allocation decisions* (units to transfer, source, destination) and *pre/post-allocation stock levels* for reporting.
5. **Continue** *iterations* until all allocable stock is distributed or all store needs are met (or no further profitable allocations can be made).

Sample Scenario

A warehouse has 100 units of SKU X. Store A is a “Top Seller” for Style X, has only 2 units of SKU X, and its ROS is 5 units/week. Store B is a “Bottom Seller,” has 15 units of SKU X (overstocked), and its ROS is 0.5 units/week. The allocation logic prioritizes Top Sellers. * The module will first identify Store A’s need for SKU X due to low stock and high ROS. It will allocate, for example, 10 units to Store A, bringing its stock to 12. * Then, it might identify Store B as overstocked and a “Bottom Seller.” If configured, it will recommend pulling back 5 units from Store B, reducing its stock to 10 and making those 5 units available for other stores. **Key Takeaway:** This module is the central decision-maker, dynamically balancing inventory across the network to maximize sales and operational efficiency based on complex business rules and demand signals.

Results

- Detailed SKU-level allocation proposals (source, destination, quantity).

- Simulated pre- and post-allocation inventory levels for all stores and warehouses.
- Depletion analysis reports.

Unusual Cases

- **Missing or delayed data feed:** *Inaccurate warehouse inventory, ROS, or ranking data* can lead to suboptimal or impossible allocations.
- **Conflicting business rules:** *Overlapping or contradictory constraints* (e.g., high min stock, but low demand) can prevent efficient allocation.
- **Warehouse constraints:** *Physical limitations in warehouse space or picking capacity* not fully reflected in the model can lead to unrealistic allocations.
- **Unforeseen demand shifts:** *Sudden, unpredicted changes in customer behavior* can cause the model to allocate incorrectly if not quickly updated.
- **“Orphaned” inventory:** *Small quantities of SKUs remaining in warehouses or stores* that are too small to meet minimum allocation thresholds.

Next Steps

The generated allocation proposals and stock reports are consumed by the Reallocation Output Module for final consolidation, fanning out, and persistence.

Common Questions

1. **How does the system decide how much to allocate?** *It considers current stock, ROS, min/max targets, ISS, and the performance ranking of the store-style to determine the optimal quantity to send.*
2. **Can we manually override allocations?** *While the system automates, businesses often have a process for manual review and adjustment of critical allocations.*
3. **What is depletion simulation?** *It's a forward-looking forecast that simulates how inventory will sell down over time based on ROS, helping determine future stock needs and surplus.*
4. **How are pullbacks initiated?** *Pullbacks are initiated when a store-style is highly overstocked relative to its demand (e.g., a bottom seller with many units), freeing up valuable inventory.*
5. **Does this handle multi-warehouse scenarios?** *Yes, it can allocate from multiple warehouses, identifying the most efficient source for each transfer.*

Quick Reference

- **OD:** Optimum Depth
- **OW:** Optimum Width
- **ISS:** Ideal Size Set
- **ROS:** Rate of Sale
- **AG:** Attribute Group (indirectly via rankings)
- **SKU:** Stock Keeping Unit

Reallocation Output Module

Module Name & Business Objective

Reallocation Output Module: This module is responsible for consolidating, fanning out, and persisting all final allocation results. It denormalizes parent-level SKU allocations to individual child SKUs, calculates remaining warehouse inventory, and generates comprehensive output reports. These reports include detailed distribution quantities, warehouse movements, fill rates, overstocking details, and specific reallocation outputs tailored for depletion analysis and dispatch prioritization.

Required Information

- **Operational Data:** Allocation decisions and simulated stock movements (from Reallocation Allocation Module), SKU parent maps, SKU details, transactional snapshots (e.g., historical inventory, goods in transit) (from Reallocation Prepare Data Module).

Business Settings

- **Report Granularity:** *Configuration for the level of detail required in various output reports (e.g., SKU-level, style-level, store-level).*
- **Output Format:** *Specification of desired output file formats (e.g., CSV, JSON, database tables).*
- **Dispatch Prioritization Rules:** *Rules for sorting and grouping allocations for efficient warehouse picking and shipping.*

Process Flow

graph TD

```
A[Allocation Decisions + Stock Reports + SKU Master Data] --> B[Consolidate & Denormalize Results];
B --> C[Calculate Final Inventory & Metrics];
C --> D[Generate Comprehensive Output Reports];
```

Step-by-Step Actions

1. **Receive** all final allocation decisions (e.g., store-style level allocations) and simulated pre/post-allocation stock levels from the Reallocation Allocation Module.
2. **Denormalize** parent-level SKU allocations (if applicable) down to individual child SKUs. *This ensures that the output reflects the precise items to be moved.*
3. **Calculate** the final remaining warehouse inventory after all proposed allocations and pullbacks have been accounted for.
4. **Compute** key performance metrics such as fill rates (percentage of demand met), overstocking details, and stock-to-sales ratios.
5. **Generate** a variety of comprehensive output reports including:
 - **Distribution Quantities Report:** Detailed breakdown of units to be moved for each SKU, from source to destination.

- **Warehouse Movements Report:** Specific instructions for warehouse operations (picking, packing, shipping).
 - **Fill Rate Report:** Analysis of how well demand was met.
 - **Overstocking Report:** Identifies stores/SKUs that remain overstocked.
 - **Depletion Analysis Outputs:** Data points for further analysis of stock sell-through over time.
 - **Dispatch Prioritization Report:** Sorted list of allocations for efficient warehouse dispatch.
6. **Persist** *all generated reports and data* to the designated output directory or database.

Sample Scenario

The Allocation Module proposes sending 10 units of a Style (parent SKU) to Store A. This Style has 2 child SKUs: SKU X (Size M) and SKU Y (Size L) with a 60/40 split. * The Output Module will denormalize this to: 6 units of SKU X to Store A, and 4 units of SKU Y to Store A. * It then generates a report showing this specific SKU-level transfer, updates the warehouse's remaining inventory, and calculates that Store A's fill rate for this style is now 95%. **Key Takeaway:** This module transforms complex allocation logic into actionable, comprehensive, and auditable reports, making the reallocation process transparent and facilitating operational execution.

Results

- Final, detailed SKU-level allocation proposals.
- Updated warehouse inventory figures.
- Comprehensive reports on distribution quantities, warehouse movements, fill rates, and overstocking.
- Outputs specifically tailored for depletion analysis and dispatch.

Unusual Cases

- **Missing or delayed data feed:** *If the allocation decisions from the previous module are incomplete*, the output reports will be inaccurate.
- **Inconsistent SKU parent-child mapping:** *Incorrect denormalization* can lead to wrong quantities being reported for individual SKUs.
- **Output file generation errors:** *Issues with writing to the specified output directory* or database can cause failure to persist results.
- **Performance bottlenecks:** *For very large datasets*, the consolidation and report generation process can be resource-intensive and require optimization.

Next Steps

The outputs are used by operational teams for dispatch and execution, and by business teams for performance monitoring and strategic review. The Utility Output Snapshot Module and Utility Output Sync Module might use these outputs for further processing.

Common Questions

1. **What format are the outputs in?** Typically, CSV files for ease of import into other systems or databases, but can be configured for other formats like JSON.
2. **How detailed are the reports?** They are usually SKU-level, providing granular detail on every proposed unit movement.
3. **Can we get a summary report?** Yes, aggregate reports (e.g., by store, by category, by style) are typically generated alongside detailed ones.
4. **How quickly are these reports available after a run?** They are generated immediately upon completion of the allocation module, ready for consumption.
5. **What's the purpose of the dispatch prioritization report?** It helps warehouse teams efficiently pick and pack orders by grouping items for the same destination or by optimizing routes.

Quick Reference

- **OD:** Optimum Depth
- **OW:** Optimum Width
- **ISS:** Ideal Size Set
- **ROS:** Rate of Sale
- **AG:** Attribute Group
- **SKU:** Stock Keeping Unit

Reallocation Date Validation Module

Module Name & Business Objective

Reallocation Date Validation Module: This module performs critical validations on date-related inputs and overall data consistency before the reallocation process begins. It ensures that the analysis end date and period end dates are logically ordered, verifies the presence of all required input periods for the depletion forecast, and assesses the “liveliness” of SKUs/styles based on available stock and sales data. It also includes validation of Annual Operating Plan (AOP) data, marking potential issues as warnings or errors.

Required Information

- **Operational Data:** Analysis end date, period end dates, depletion forecast input periods, AOP data.
- **Sales Data:** Sales data for SKU/style liveliness assessment.
- **Product Data:** SKU/style inventory data for liveliness assessment.

Business Settings

- **Date Order Validation:** Ensures that all input dates and period ranges are logically sequential and non-overlapping, preventing data processing errors.
- **Required Period Checks:** Defines mandatory periods for depletion forecasting, ensuring comprehensive demand projections.

- **SKU/Style Liveliness Criteria:** *Rules for determining if an SKU or style is “active” or “live” enough to be included in reallocation, typically based on recent sales or available stock.*
- **AOP Validation Rules:** *Criteria for checking the completeness and reasonableness of AOP data, flagging discrepancies as warnings or errors.*

Process Flow

graph TD

```
A[Raw Dates + Depletion Periods + SKU/Style Data + AOP] -->
B{Validate Date Order & Completeness};
B --> C{Assess SKU/Style Liveliness};
C --> D{Validate AOP Data};
D --> E[Validation Results (Warnings/Errors)];
```

Step-by-Step Actions

1. **Receive** *all date-related inputs* including the overall analysis end date, specific period end dates for forecasting, and the list of required input periods for the depletion model.
2. **Validate** *the logical order and consistency* of all provided dates, ensuring `startDate < endDate` for all ranges and proper sequencing of periods.
3. **Verify** *the presence of all necessary input periods* for the depletion forecast. *If any period is missing, it will generate a warning or error.*
4. **Assess** *the “liveliness” of SKUs and styles* by checking for recent sales activity, current stock levels, and other relevant product lifecycle indicators. *SKUs deemed “not live” might be excluded from reallocation.*
5. **Perform** *validation checks on Annual Operating Plan (AOP) data*, such as checking for missing values or unrealistic targets, and *issue warnings or errors* as appropriate.
6. **Report** *any inconsistencies, warnings, or critical errors* to the user, potentially stopping the reallocation process if errors are severe.

Sample Scenario

The input dates specify an analysis end date in the past, or periods that overlap. Or, an AOP file is provided but is missing data for a critical store. Additionally, SKU Z has had no sales for 6 months and has 0 stock. * The module will flag the incorrect date order as an error, preventing the process from starting. * It will issue a warning for the missing AOP data, advising manual review. * It will identify SKU Z as “not live,” recommending its exclusion from active reallocation consideration. **Key Takeaway:** This module acts as the gatekeeper for data quality and consistency, preventing the entire reallocation pipeline from running with flawed or incomplete foundational data, thereby ensuring the reliability of its outputs.

Results

- Validation report detailing date inconsistencies, missing data, and liveliness assessments.
- Warnings or errors for critical data quality issues (e.g., AOP validation failures).
- Flagged “liveliness” status for SKUs/styles.

Unusual Cases

- **Missing or delayed data feed:** *If critical date inputs or AOP data feeds are delayed, the module cannot perform its checks and might report errors.*
- **Unexpected date formats:** *Input dates not conforming to expected formats can cause parsing errors and validation failures.*
- **Incorrect liveliness thresholds:** *Too strict or too lenient criteria for SKU liveliness can lead to either excluding viable products or including dead stock.*
- **AOP data discrepancies:** *Large variances between historical trends and AOP data might trigger false warnings if AOP represents aggressive targets.*

Next Steps

If the validation passes successfully, the Reallocation Prepare Data Module can proceed to load and initialize the comprehensive dataset. If critical errors are found, the process halts, requiring data correction.

Common Questions

1. **Why validate dates?** *Incorrect or illogical dates can lead to inaccurate historical data parsing, incorrect future period definitions, and flawed depletion forecasts.*
2. **What does “SKU liveliness” mean?** *It determines if an SKU or style is still actively selling or intended to be sold. Non-live items are typically excluded from active reallocation efforts.*
3. **Can I override validation errors?** *Typically, critical errors must be resolved by data correction. Warnings might allow the process to continue with a known risk.*
4. **How is AOP data validated?** *Checks include completeness (all required fields present), logical consistency (e.g., non-negative values), and sometimes comparison against historical trends for major deviations.*
5. **What happens if validation fails?** *If it's a critical error, the entire reallocation process will stop, prompting the user to address the data issues.*

Quick Reference

- **ROS:** Rate of Sale (underpins liveliness assessment)
- **AG:** Attribute Group (can be used for AOP validation)
- **SKU:** Stock Keeping Unit
- **AOP:** Annual Operating Plan

Utility Output Snapshot Module

Module Name & Business Objective

Utility Output Snapshot Module: This is a generic utility module responsible for capturing snapshots of output data. It's part of the overall data pipeline and likely used for auditing, historical tracking, or debugging purposes, rather than direct business logic specific to reallocation.

Required Information

- Any output data stream from previous modules.

Business Settings

- **Snapshot Frequency:** *How often snapshots are taken (e.g., after every successful run).*
- **Storage Location:** *Where the snapshots are stored (e.g., specific S3 bucket, local directory).*
- **Retention Policy:** *How long snapshots are kept.*

Process Flow

graph TD

```
A[Output Data Stream] --> B{Capture Data Snapshot};  
B --> C[Persisted Data Snapshot];
```

Step-by-Step Actions

1. **Receive** *data stream or final output* from upstream modules.
2. **Create** *a timestamped copy* of the received data.
3. **Store** *this snapshot* in a predefined archival location. *This ensures historical traceability and debugging capabilities.*

Sample Scenario

After the Reallocation Output Module generates its final reports, the Utility Output Snapshot Module takes a copy of all those reports and archives them in a dated folder on a network drive. This ensures that even if live reports are overwritten, a historical record exists for auditing. **Key Takeaway:** This module provides version control and historical data preservation for the reallocation outputs, essential for debugging, performance analysis, and regulatory compliance.

Results

- Archived copies of output data.
- Historical records of reallocation runs.

Unusual Cases

- **Missing or delayed data feed:** *If the upstream module fails to produce output, no snapshot can be taken.*
- **Storage access issues:** *Permissions errors or full storage space* can prevent snapshots from being saved.
- **Corrupted data in snapshot:** *Issues during data capture* could lead to an unusable snapshot.

Next Steps

The persisted snapshots are available for auditing, debugging, or future analysis.

Common Questions

1. **Why take snapshots?** *For auditing, troubleshooting, and historical trend analysis. It allows comparing outputs across different runs.*
2. **How often are snapshots taken?** *Typically after every successful run of the entire pipeline, or after critical stages.*
3. **Can I view old snapshots?** *Yes, if properly configured, snapshots are accessible for review and comparison.*
4. **Do snapshots impact performance?** *Minimal impact, as it's usually a simple copy operation, but depends on data volume and storage speed.*
5. **Are snapshots automatically deleted?** *A retention policy usually defines how long snapshots are kept before automatic deletion.*

Quick Reference

- **OD:** Optimum Depth (relevant for data content)
- **OW:** Optimum Width (relevant for data content)
- **ISS:** Ideal Size Set (relevant for data content)
- **ROS:** Rate of Sale (relevant for data content)
- **AG:** Attribute Group (relevant for data content)

Utility Output Sync Module

Module Name & Business Objective

Utility Output Sync Module: This is a generic utility module designed for synchronizing or uploading generated output data. It handles the transfer of final results to external systems or storage, completing the data pipeline. Similar to the snapshot module, it's a general utility rather than a reallocation-specific business function.

Required Information

- Any output data stream from previous modules.

Business Settings

- **Target System/Location:** *The external system or storage destination for the output data (e.g., a data warehouse, FTP server, cloud storage).*
- **Synchronization Frequency:** *How often the data is synchronized.*
- **Authentication Details:** *Credentials or keys for accessing the target system.*
- **Error Handling:** *Rules for retries or notifications upon transfer failure.*

Process Flow

graph TD

```
A[Output Data Stream] --> B{Synchronize/Upload Data};  
B --> C[Data in External System];
```

Step-by-Step Actions

1. **Receive** *final output data* from upstream modules (e.g., the Reallocation Output Module).
2. **Connect** *to the configured external target system* (e.g., a database, cloud storage, or another application).
3. **Transfer/Upload** *the output data* to the target system. *This ensures that the results are available for consumption by other business processes or reporting tools.*
4. **Verify** *successful transfer* and handle any errors or retries as configured.

Sample Scenario

After all reallocation reports are generated and possibly snapshotted, the Utility Output Sync Module uploads the final allocation plan (e.g., a CSV file) to a central merchandising system. This ensures that other departments (like logistics or store operations) can access the allocation decisions directly. **Key Takeaway:** This module facilitates the seamless integration of reallocation outputs into the broader enterprise data ecosystem, making the insights actionable across the organization.

Results

- Output data successfully transferred to external systems.
- Integration of reallocation results with other business processes.

Unusual Cases

- **Missing or delayed data feed:** *If the upstream module fails to produce output, there is no data to sync.*
- **Network connectivity issues:** *Loss of connection to the target system will prevent data transfer.*
- **Authentication failures:** *Incorrect credentials will block access to the external system.*
- **Data format incompatibility:** *If the output format does not match the target system's requirements, the transfer may fail or result in corrupted data.*

Next Steps

The data in the external system is now ready for use by other business processes, dashboards, or applications.

Common Questions

1. **What systems can this module sync with?** *It can be configured to sync with various systems like data warehouses, enterprise resource planning (ERP) systems, or cloud storage services.*
2. **Is the sync real-time?** *Typically, it's run on a schedule (e.g., daily) after the reallocation process completes, rather than real-time.*
3. **What happens if the sync fails?** *Depending on configuration, it might retry the transfer, log an error, or send an alert to operations teams.*
4. **Does this module transform data?** *Primarily, it transfers data. Any major transformations are usually handled by upstream modules or by the receiving system.*
5. **Is the data secure during transfer?** *Yes, best practices involve using secure protocols (e.g., SFTP, HTTPS) for data transfer.*

Quick Reference

- **OD:** Optimum Depth (relevant for data content)
 - **OW:** Optimum Width (relevant for data content)
 - **ISS:** Ideal Size Set (relevant for data content)
 - **ROS:** Rate of Sale (relevant for data content)
 - **AG:** Attribute Group (relevant for data content)
-

9. Appendix

Full Parameter Reference

(No global configuration parameters were provided in the input data.)

Version History

- **Version 1.0 (2025-05-23):** Initial release of the Reallocation Documentation.

Contact & Support Info

For technical support or inquiries regarding the Reallocation algorithm, please contact: * **Email:** support@yourcompany.com * **Internal Support Portal:** [Link to your internal support portal] * **Team:** Inventory Optimization Team

Tutorial: reallocation

The reallocation process begins with a crucial data validation phase, ensuring data integrity. This is followed by a comprehensive data preparation module that loads and structures all necessary master and transactional data. Sales data is then cleaned and prepared, forming the foundation for subsequent rate-of-sale (ROS) and revenue calculations. Warehouse inventory is also refined during this initial data setup. With all foundational data in place, modules compute various ROS metrics (first month and period-specific SKUs) and detailed revenue metrics, which are then used to segment and rank store-styles. The core allocation engine leverages these ranked and calculated metrics to iteratively distribute inventory. Finally, an output module consolidates and persists all allocation results and stock movements.

flowchart TD

```
A0["Reallocation Prepare Data Module"]
A1["Reallocation Prepare Warehouse Inventory Module"]
A2["Reallocation First Month ROS Calculation Module"]
A3["Reallocation Sales Calculation Module"]
A4["Reallocation Period ROS Calculation Module"]
A5["Reallocation Prepare Sales Module"]
A6["Reallocation Segmentation Module"]
A7["Reallocation Compute Revenue Module"]
A8["Reallocation Ranking Module"]
A9["Reallocation Allocation Module"]
A10["Reallocation Output Module"]
A11["Reallocation Date Validation Module"]
A12["Utility Output Snapshot Module"]
A13["Utility Output Sync Module"]
A11 -- "prerequisite" --> A0
A0 -- "initializes" --> A5
A0 -- "initializes" --> A1
A0 -- "provides_config" --> A2
A0 -- "provides_data" --> A3
A0 -- "provides_iss" --> A4
A0 -- "provides_master_data" --> A7
A0 -- "provides_config" --> A9
A0 -- "provides_master_data" --> A10
A5 -- "prepares_sales" --> A2
A5 -- "prepares_sales" --> A6
A5 -- "prepares_sales" --> A7
A5 -- "prepares_sales" --> A8
A1 -- "refines_inventory" --> A9
A2 -- "provides_ros" --> A4
A3 -- "provides_sales_qty" --> A4
A7 -- "provides_revenue_metrics" --> A8
A6 -- "informs_allocation" --> A9
A4 -- "provides_sku_ros" --> A9
```

```
A8 -- "prioritizes" --> A9  
A9 -- "generates_results" --> A10
```

Chapters

Master Report: Reallocation Algorithm

1. Title & Metadata

- **Project Name:** reallocation
- **Version:** 1.0
- **Last Updated:** 2024-05-23
- **One-line Elevator Pitch:** The reallocation algorithm intelligently redistributes existing inventory across stores to maximize sales potential, minimize stockouts, and improve overall inventory efficiency.

2. Executive Summary

The reallocation algorithm is a critical component of our merchandising strategy, designed to optimize the placement of existing inventory within our retail network. By leveraging detailed sales data, inventory levels, and product attributes, the system identifies opportunities to move products from overstocked locations to understocked locations where demand is higher. This proactive approach ensures that the right products are in the right place at the right time, enhancing the customer experience and driving profitability.

- **Key outcomes or KPIs:**
 - **Increased ROS:** By ensuring product availability where demand is highest.
 - **Reduced Stockouts:** Minimizing lost sales opportunities at high-demand locations.
 - **Optimized Inventory Turnover:** Preventing products from lingering in low-demand stores.
 - **Improved Sales Performance:** Directly contributing to category and overall revenue growth.
 - **Better utilization of OD, OW, and ISS:** Ensuring optimal product breadth and depth across locations based on real-time needs.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Planning, Store Operations, Supply Chain Logistics, Category Managers.

3. Architecture & Data Flow

graph TD
A[Historical Sales Data] --> B{Data Ingestion & Preprocessing};
C[Current Inventory Data] --> B;
D[Product Master Data] --> B;
E[Store Cluster & Attribute Data] --> B;

```

B --> F[Demand & Inventory Analysis];

F --> G[Reallocation Optimization Engine];

G --> H[Reallocation Plan];
G --> I[Performance Metrics & Alerts];
H --> J[Inventory Management System];
I --> K[Merchandising Dashboards];

```

- **Component Bullets:**

- **Data Ingestion & Preprocessing:** Collects, cleans, and transforms raw data from various sources into a standardized format.
- **Demand & Inventory Analysis:** Analyzes historical sales (ROS), current inventory levels, and product attributes to identify demand patterns and inventory imbalances. Considers OD, OW, and ISS for each location.
- **Reallocation Optimization Engine:** The core decision-making module that uses a proprietary algorithm to determine the optimal reallocation quantities and routes.
- **Reallocation Plan:** The primary output detailing specific product transfers between stores.
- **Performance Metrics & Alerts:** Generates reports and notifications on the effectiveness of reallocation suggestions and potential issues.

- **High-level data sources:**

- **POS Data:** Granular sales transactions, including SKU, store, date, and quantity (for ROS calculation).
- **Inventory Management Systems:** Real-time stock levels (on hand, in transit), in-store inventory counts.
- **Product Master Data:** SKU attributes (e.g., color, size, brand, style), category hierarchies, cost, retail price.
- **Store Data:** Store attributes (e.g., square footage, location type, regional demand profiles), store capacity.
- **Merchandising Directives:** Business rules, minimum stock levels, desired OD/OW/ISS targets by category/store.

4. Configuration & Data Dictionary

Settings Table (args.properties)

| Setting Parameter | Data Type | Default Value | Description |
|--------------------------|-----------|---------------|---|
| lookback_period_weeks | Integer | 8 | Number of weeks of historical sales data to consider for ROS calculation. |
| min_ros_threshold_units | Decimal | 0.5 | Minimum ROS (units/week) for a SKU to be considered for reallocation <i>into</i> a store. |
| inventory_buffer_percent | Decimal | 0.15 | Percentage buffer to maintain at source stores |

| Setting Parameter | Data Type | Default Value | Description |
|------------------------------|-----------|---------------|---|
| | | | after reallocation (e.g., 0.15 = 15% of current stock). |
| optimization_objective | String | maximize_ros | Primary goal of the optimization:
maximize_ros, minimize_stockouts, or balance_inventory_turn. |
| reallocation_cost_per_unit | Decimal | 0.75 | Estimated cost to move one unit between stores. |
| target_od_variance_tolerance | Decimal | 0.10 | Acceptable percentage deviation from target OD for a category within a store. |

Shared Data Fields (common-data.json)

| Field Name | Data Type | Description | AG (Attribute Group) |
|-------------------|-----------|---|----------------------|
| sku_id | String | Unique identifier for a stock keeping unit. | - |
| store_id | String | Unique identifier for a retail store location. | - |
| date | Date | Reference date for sales or inventory data. | - |
| units_sold_weekly | Integer | Weekly sales volume for a given SKU at a store (part of ROS). | - |
| inventory_on_hand | Integer | Current physical units of a SKU at a store. | - |
| category_id | String | Identifier for the product category. | - |
| subcategory_id | String | Identifier for the product subcategory. | - |
| size | String | Product size (e.g., "S", "M", "L", "10", "12"). | ISS |

| Field Name | Data Type | Description | AG (Attribute Group) |
|------------|-----------|--|----------------------|
| color | String | Product color (e.g., “Red”, “Blue”, “Black”). | AG |
| style_id | String | Unique identifier for a product style. | OW |
| target_od | Integer | Optimum Depth for a category/style at a store. | OD |
| target_ow | Integer | Optimum Width for a category/store. | OW |

Outputs Catalog

| Output File/Data Structure | Description | Key Fields |
|--------------------------------------|---|--|
| reallocation_plan.csv | Detailed plan of units to be moved between specific stores. | sku_id, source_store_id, destination_store_id, units_to_transfer |
| performance_summary.json | Aggregated metrics on the impact of reallocation recommendations. | total_units_reallocated, estimated_sales_lift, estimated_stockout_reduction, total_reallocation_cost |
| store_inventory_recommendations.json | Recommended target inventory levels per SKU per store after reallocation. | store_id, sku_id, recommended_final_inventory, target_ros_coverage_weeks |
| debug_log.txt | Detailed logging of algorithm execution, warnings, and errors. | timestamp, level, message, module |

5. End-to-End Flow

The reallocation algorithm executes a structured sequence of steps to generate optimal inventory transfer recommendations:

- 1. Data Ingestion & Validation:** Raw sales, inventory, product, and store data are loaded. Data quality checks are performed to identify and flag missing or inconsistent records.

2. **Feature Engineering:** Derived metrics are calculated, such as ROS for each SKU at each store over the `lookback_period_weeks`. Inventory coverage (weeks of supply) is determined based on current inventory and ROS.
3. **Demand-Supply Imbalance Identification:** For each SKU, the algorithm identifies stores with excess inventory (high coverage, low ROS) and stores with insufficient inventory (low coverage, high ROS, or identified potential for higher ROS if stock was present). This step considers the target OD, OW, and ISS for each location.
4. **Prioritization & Grouping:** Potential reallocation opportunities are prioritized based on the magnitude of the imbalance, estimated sales lift, and product profitability. Opportunities are grouped by common Attribute Groups (AG) to ensure coherent transfers (e.g., moving a specific color or size set).
5. **Optimization Engine Execution:** The core optimization model runs, considering factors like:
 - `min_ros_threshold_units` for destination stores.
 - `inventory_buffer_percent` for source stores.
 - `reallocation_cost_per_unit`.
 - The `optimization_objective` (e.g., maximizing overall ROS).
 - Store capacity and transfer route constraints.
 - Maintaining target OD, OW, and ISS where applicable.
6. **Recommendation Generation:** Based on the optimization output, a detailed `reallocation_plan.csv` is generated, specifying SKUs, source stores, destination stores, and transfer quantities.
7. **Reporting & Alerting:** A `performance_summary.json` is created, outlining the expected benefits and costs. Alerts are triggered for any unusually large transfers or unresolvable inventory imbalances.
8. **Output Dissemination:** The reallocation plan is made available to relevant systems (e.g., Inventory Management System for transfer order creation) and stakeholders (e.g., Merchandising Dashboards).

6. Submodule Overview

6.1. Submodule: Data Ingestion & Preprocessing

- **Purpose:** To collect, clean, and standardize all necessary input data for the reallocation algorithm.
- **Inputs:**
 - Raw POS data (SKU, Store, Date, Quantity)
 - Raw Inventory data (SKU, Store, Quantity_on_hand)
 - Raw Product Master data (SKU, Category, Subcategory, Size, Color, Style, etc.)
 - Raw Store data (Store ID, Store Type, Regional info)
- **Outputs:**
 - `preprocessed_sales_data.csv`: Cleaned historical sales aggregated by SKU-Store-Week.
 - `preprocessed_inventory_data.csv`: Cleaned current inventory by SKU-Store.
 - `enriched_product_store_data.json`: Combined and validated product and store attributes.
- **Core Logic Summary:**
 1. Load raw data from defined source systems.

2. Perform data validation (e.g., check for missing values, correct data types, consistent identifiers).
3. Standardize units and time granularities (e.g., aggregate daily sales to weekly).
4. Merge datasets to create a comprehensive view of SKU-Store combinations with relevant attributes.

6.2. Submodule: Demand & Inventory Analysis

- **Purpose:** To analyze sales velocity and inventory levels to identify potential surplus and deficit situations for each SKU across the store network.
- **Inputs:**
 - `preprocessed_sales_data.csv`
 - `preprocessed_inventory_data.csv`
 - `enriched_product_store_data.json`
 - Configuration settings (e.g., `lookback_period_weeks`).
- **Outputs:**
 - `sku_store_metrics.csv`: Calculated ROS and inventory coverage per SKU per store.
 - `imbalance_opportunities.json`: Identified potential source and destination stores for each SKU.
- **Core Logic Summary:**
 1. Calculate **ROS** (Rate of Sale) for each SKU at each store using the `lookback_period_weeks` from `preprocessed_sales_data.csv`.
 2. Compute inventory coverage (weeks of supply) by dividing `inventory_on_hand` by ROS.
 3. Identify “surplus” stores where inventory coverage is significantly above target or ROS is very low.
 4. Identify “deficit” stores where inventory coverage is below a critical threshold, current ROS is high, or there’s an opportunity to improve sales by meeting desired OD, OW, or ISS.

6.3. Submodule: Reallocation Optimization Engine

- **Purpose:** To determine the optimal quantity of each SKU to reallocate between identified surplus and deficit stores, considering various constraints and the defined objective.
- **Inputs:**
 - `imbalance_opportunities.json`
 - `sku_store_metrics.csv`
 - Configuration settings (e.g., `min_ros_threshold_units`, `inventory_buffer_percent`, `reallocation_cost_per_unit`, `optimization_objective`).
- **Outputs:**
 - `reallocation_decisions.json`: Raw optimization output with proposed transfers.
- **Core Logic Summary:**
 1. Formulate the reallocation problem as an optimization model (e.g., linear programming).
 2. Define objective function based on `optimization_objective` (e.g., maximize incremental ROS, minimize lost sales).
 3. Apply constraints:
 - No store can go below `inventory_buffer_percent` after transferring.

- Destination stores must meet `min_ros_threshold_units` for the SKU.
 - Consider current inventory, transit times, and potential `reallocation_cost_per_unit`.
 - Ensure transfers align with target OD, OW, and ISS where applicable.
4. Solve the optimization model to find the best allocation of units from surplus to deficit locations.

6.4. Submodule: Output Generation & Reporting

- **Purpose:** To translate the optimization decisions into actionable plans and generate summary reports for stakeholders.
- **Inputs:**
 - `reallocation_decisions.json`
 - `sku_store_metrics.csv`
 - Configuration settings.
- **Outputs:**
 - `reallocation_plan.csv`
 - `performance_summary.json`
 - `store_inventory_recommendations.json`
- **Core Logic Summary:**
 1. Format `reallocation_decisions.json` into a user-friendly `reallocation_plan.csv` with clear source, destination, SKU, and quantity.
 2. Calculate estimated impact metrics (e.g., sales lift, stockout reduction, total cost) based on the proposed transfers and historical ROS data.
 3. Generate `performance_summary.json` with key KPIs.
 4. Derive `store_inventory_recommendations.json` by adjusting current inventory with proposed transfers and outlining target coverage.

7. Worked Example

Scenario: A popular “Summer Dress” (SKU: SD-001) is experiencing varied performance across two stores.

Input Data: * SKU: SD-001 (Summer Dress, Size M, Blue) * **Store A (Upscale Boutique):** * Current Inventory: 30 units * ROS (last 8 weeks): 2 units/week * Target OD for Category: 15 units (desired depth) * Target OW for Store A: High (focus on unique styles) * **Store B (Suburban Outlet):** * Current Inventory: 5 units * ROS (last 8 weeks): 8 units/week * Target OD for Category: 20 units (desired depth due to high foot traffic) * Target OW for Store B: Medium (focus on popular items)

Algorithm Logic Trace:

1. Demand & Inventory Analysis:

- **Store A:** Inventory Coverage = $30 \text{ units} / 2 \text{ units/week} = 15$ weeks. This is high, indicating potential surplus, especially compared to its target OD of 15. The ROS is low for a key item.
- **Store B:** Inventory Coverage = $5 \text{ units} / 8 \text{ units/week} = 0.625$ weeks. This is very low, indicating a significant deficit and high risk of stockout. The ROS is high, suggesting strong demand.

2. **Imbalance Identification:** SD-001 at Store A is identified as a potential source (excess inventory). SD-001 at Store B is identified as a prime destination (critical deficit, high ROS).

3. **Optimization Engine Execution:**

- **Objective:** Maximize overall ROS by reducing stockouts.
- **Constraints:**
 - Store A must retain `inventory_buffer_percent` (e.g., 15% of 30 units = 4.5 units, round up to 5 units). So, max transfer from A = $30 - 5 = 25$ units.
 - Store B needs to reach a comfortable coverage, perhaps 2-3 weeks, or meet its target OD of 20. To reach 2 weeks coverage (at 8 units/week) = 16 units. To reach 20 units (target OD) means 15 units needed ($20 - 5$).
- **Decision:** The algorithm calculates that moving units from Store A to Store B will significantly increase overall sales. Transferring 15 units would bring Store B up to its target OD of 20 units and give it 2.5 weeks of supply ($20/8$). Store A would be left with 15 units ($30 - 15$), exactly its target OD, and 7.5 weeks of supply ($15/2$). This balances inventory efficiently.
- **Cost:** Assume `reallocation_cost_per_unit` is \$0.75. Total transfer cost = $15 \text{ units} * \$0.75/\text{unit} = \11.25 .

Output (Excerpt from `reallocation_plan.csv`):

| <code>sku_id</code> | <code>source_store_id</code> | <code>destination_store_id</code> | <code>units_to_transfer</code> |
|---------------------|------------------------------|-----------------------------------|--------------------------------|
| SD-001 | Store A | Store B | 15 |

Expected Outcome: Store B experiences a sustained sales increase due to improved stock availability and reduced stockouts, while Store A avoids holding excess inventory, improving its inventory turn. The overall ROS for SD-001 across the network is maximized.

8. Debug Guide Highlights

- **No Reallocation Recommendations Generated:**
 - **Check Input Data:** Ensure all required data sources (sales, inventory, product master) are loaded correctly and have recent data. Missing or old data will prevent analysis.
 - **Verify Thresholds:** Review `min_ros_threshold_units` and `inventory_buffer_percent` in `args.properties`. If these are too strict, the algorithm might not find suitable candidates for reallocation.
 - **Review ROS Calculations:** Are ROS values being calculated as expected? Extremely low or zero ROS for high-demand items could indicate data issues.
 - **Confirm Imbalance Opportunities:** Check the intermediate `imbalance_opportunities.json` output. Are potential surplus/deficit situations being correctly identified?
- **Recommendations Seem Illogical or Suboptimal:**
 - **Constraint Review:** Examine the constraints applied in the Reallocation Optimization Engine. Overly aggressive `reallocation_cost_per_unit` or excessively high `inventory_buffer_percent` can limit beneficial transfers.

- **Objective Mismatch:** Is the `optimization_objective` aligned with current business goals? Maximizing sales vs. minimizing costs will yield different results.
- **Data Accuracy:** Inaccurate `inventory_on_hand` or `units_sold_weekly` can lead to flawed decisions. Perform spot checks on key SKUs/stores.
- **Target OD/OW/ISS:** Ensure these targets are realistic and correctly applied per store/category. Unrealistic targets can lead to impossible or counterintuitive recommendations.
- **Performance Issues (Slow Execution):**
 - **Data Volume:** For very large datasets, consider optimizing data loading and preprocessing steps (e.g., using more efficient data structures, parallel processing).
 - **Optimization Solver Complexity:** If the optimization model is too complex or the number of variables/constraints is excessively high, the solver can be slow. Review if all constraints are strictly necessary or if simplifications can be made.
 - **Database/File I/O:** Check for bottlenecks in reading from or writing to data sources/sinks.

9. Common Pitfalls & Remedies

- **Pitfall 1: “Data Silos” or Inconsistent Data:** Different systems providing sales, inventory, or product data might have different IDs or update frequencies, leading to mismatched information.
 - **Remedy:** Implement robust data governance and master data management. Use a centralized data lake or warehouse for all inputs. Implement strong data validation rules and reconciliation processes in the Data Ingestion & Preprocessing submodule.
- **Pitfall 2: Overlooking Store-Specific Nuances:** Treating all stores equally without considering their unique characteristics (e.g., traffic patterns, local demographics, display space, capacity for OD/OW) can lead to ineffective reallocations.
 - **Remedy:** Incorporate store-specific attributes and performance metrics into the Demand & Inventory Analysis. Leverage store clustering and integrate target OD, OW, and ISS for each store to tailor recommendations.
- **Pitfall 3: Neglecting Logistics & Cost Implications:** Generating recommendations without considering the real-world cost and feasibility of moving inventory can lead to impractical plans.
 - **Remedy:** Ensure `reallocation_cost_per_unit` accurately reflects actual shipping and handling expenses. Include transit times and logistical constraints (e.g., store receiving hours, maximum transfer volume) in the Reallocation Optimization Engine.
- **Pitfall 4: “Just-in-Time” Fallacy with Reallocation:** Relying solely on reallocation to solve inventory problems can be a band-aid. It shouldn’t replace good initial allocation or forecasting.
 - **Remedy:** Communicate that reallocation is an optimization tool for *existing* inventory, not a substitute for accurate demand forecasting and initial product allocation. Use performance metrics to identify root causes of consistent imbalances.
- **Pitfall 5: Lack of Stakeholder Trust/Adoption:** If merchandising teams don’t understand or trust the algorithm’s recommendations, they won’t execute them.
 - **Remedy:** Provide clear, plain English explanations of the algorithm’s logic. Offer a “Worked Example” (as above) and comprehensive documentation.

Conduct training sessions and solicit feedback to iterate and improve the model. Highlight the KPIs and benefits clearly.

10. FAQ

Q1: How often does the reallocation algorithm run? A1: The algorithm is typically scheduled to run weekly or bi-weekly, depending on inventory turnover rates and business needs. High-velocity categories might benefit from more frequent runs.

Q2: Can I manually override the recommendations? A2: Yes, the `reallocation_plan.csv` serves as a recommendation. Merchandising and Store Operations teams can review and adjust quantities based on their real-time insights before execution. The system will record these overrides for future analysis.

Q3: What happens if a source store doesn't have enough physical inventory to fulfill the recommended transfer? A3: The algorithm relies on current inventory data. If physical counts differ, the system will flag discrepancies. Stores should perform cycle counts to ensure data accuracy. Any unfulfilled recommendations will be reported for manual follow-up.

Q4: How does the algorithm handle new products or products with limited sales history? A4: For new products or those with insufficient ROS data, the algorithm may use proxy data (e.g., sales of similar products, category averages, or initial allocation rules) to inform recommendations. Over time, as sales data accrues, it will transition to data-driven ROS calculations.

Q5: What is the primary benefit of using this algorithm over manual reallocation? A5: The algorithm can process vast amounts of data across thousands of SKUs and stores simultaneously, identifying optimal transfer opportunities that would be impossible to discern manually. It ensures decisions are data-driven, objective, and consistent with the defined optimization objective (e.g., maximizing sales, reducing stockouts), leading to significantly improved inventory efficiency and profitability.

11. Quick Reference Cheat-Sheet

| Term/Parameter | Definition/Description |
|-----------------------|--|
| OD | Optimum Depth: The ideal number of unique items (SKUs) to stock in a category or display. |
| OW | Optimum Width: The ideal number of different categories or styles to offer. |
| ISS | Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts. |
| ROS | Rate of Sale: The speed at which products are sold over a period (e.g., units/week). |
| AG | Attribute Group: A set of product attributes grouped for analysis (e.g., color, size, brand). |
| lookback_period_weeks | |

| Term/Parameter | Definition/Description |
|----------------------------|---|
| | Configuration for how many past weeks of sales to consider for ROS calculation. |
| min_ros_threshold_units | Minimum weekly sales for a SKU to be considered for reallocation <i>into</i> a store. |
| inventory_buffer_percent | Safety stock percentage a source store must retain after transferring units. |
| optimization_objective | The primary goal of the algorithm: maximize_ros, minimize_stockouts, or balance_inventory_turn. |
| reallocation_plan.csv | Primary Output: The actionable file detailing which SKUs to move from where to where. |
| Data Ingestion | First step: Collecting, cleaning, and preparing all raw data. |
| Optimization Engine | The core module that calculates the best transfer quantities based on defined rules and objectives. |
| Key Benefit | Maximizes sales by having the right product, in the right quantity, at the right store. |

Chapter 1: Reordering Orchestration

Last Updated: 2025-05-23

1. Business Objective

This step coordinates all parts of the reordering process, making sure that each calculation is done in the correct sequence and that all necessary information is available when needed.

Why This Matters: This ensures that the final reorder recommendations are accurate, reliable, and delivered on time, allowing our stores to maintain optimal stock levels, prevent lost sales due to stockouts, and reduce unnecessary inventory holding costs.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** We need to know exactly how many units of each product were sold over a specific period. This is crucial for calculating the Rate of Sale (ROS) and understanding customer demand.
- **Historical Sales Trends:** This helps us see patterns in past sales, such as seasonal peaks or consistent demand for certain items, informing future reorder decisions.

Product Data

- **Current Stock Levels:** Knowing exactly how many units of each product are currently in our stores and warehouses is essential to determine what needs to be reordered.
- **Product Attributes (e.g., color, size, material):** These details are vital for grouping products into Attribute Groups (AGs) and ensuring we reorder the right mix, especially for determining the Ideal Size Set (ISS) and Optimum Width (OW).

Operational Data

- **Supplier Lead Times:** This tells us how long it takes for a supplier to deliver an order once placed, which is critical for timely replenishment planning.

- **Store Capacity Limits:** We need to understand the maximum amount of stock a store can hold to avoid overstocking and overcrowding.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|------------------------------------|---------------|--|---|
| Reorder Frequency | Weekly | How often the system evaluates and generates reorder recommendations for categories. | If your supply chain or sales cycles change significantly (e.g., moving to bi-weekly for slow-moving categories). |
| Safety Stock Buffer | 10% | An additional percentage of stock to hold beyond the minimum, acting as a cushion against unexpected demand spikes or supply delays. | When product demand becomes highly unpredictable or supplier reliability decreases, to reduce stockout risk. |
| Minimum Sales Threshold | 5 units/week | The minimum Rate of Sale (ROS) a product must achieve to be considered for active reordering. Products below this may be phased out. | To tighten inventory control on underperforming items or relax it for new product launches, ensuring faster sell-through. |
| Category Stockout Tolerance | 5% | The acceptable percentage of time a specific product category can be out of stock without significantly impacting sales. | If customer expectations for immediate availability change, or if a category becomes strategically more important. |

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Input Data] --> B[Reordering Orchestration]
B --> C[Reorder Recommendations]
```

1. Start with → All raw sales, product, and operational data.

2. Then → The system ensures all initial data preparation is complete (e.g., cleansing sales data, mapping product attributes).
3. Next → It triggers specific calculation modules in the correct order, such as determining the Ideal Size Set (ISS), calculating the Rate of Sale (ROS), and finding the Optimum Depth (OD) and Optimum Width (OW).
4. Finally → It consolidates all the calculated results into a comprehensive set of reorder recommendations and prepares them for output.

5. Step-by-Step Actions

1. **Prepare** the raw data
 - What we do: *We gather and cleanse all sales, product, and operational information, ensuring it's accurate and ready for analysis.*
 - Why we do it: *Clean data prevents errors in reorder calculations and ensures reliable outputs.*
2. **Sequence** the analysis modules
 - What we do: *We trigger specialized modules one by one, ensuring that the output of one module (like processed sales data) becomes the input for the next (like Rate of Sale calculation).*
 - Why we do it: *This guarantees that complex calculations build upon each other logically, just as a builder lays a foundation before erecting walls.*
3. **Validate** intermediate results
 - What we do: *At key points, we cross-check the outcomes of individual calculations to ensure they make business sense and align with expectations.*
 - Why we do it: *This catches potential issues early, preventing incorrect reorder suggestions from reaching the final stage.*
4. **Consolidate** and finalize recommendations
 - What we do: *We bring together all the individual module outputs – like desired quantities, optimal sizes, and recommended styles – into a single, comprehensive reorder list.*
 - Why we do it: *This creates a unified, actionable plan for replenishment, rather than disjointed suggestions.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Summer Breeze” dress category has been flagged for reorder review.
- Sales data shows one specific dress style (SKU: SB-DRS-BLUE-M) sold 120 units last month.
- Current store stock for SB-DRS-BLUE-M is 15 units.
- The Ideal Size Set (ISS) for this dress category recommends a size mix of 10% XS, 20% S, 30% M, 20% L, 10% XL, 10% XXL.
- Supplier lead time is 2 weeks.

What Happens

1. First, the system starts by taking in the raw sales data for the “Summer Breeze” dresses and current stock levels. It processes the 120 units sold for SKU SB-DRS-BLUE-M, preparing it for Rate of Sale (ROS) calculation.
2. Then, it passes this processed sales data to the module that calculates ROS. Simultaneously, it consults the Ideal Size Set (ISS) module, which confirms the best size distribution for the “Summer Breeze” dresses, considering past sales performance across all sizes for this category.
3. Next, based on the ROS and the current stock of 15 units, another module determines the optimal reorder quantity for SKU SB-DRS-BLUE-M to ensure we meet expected demand over the next lead time, factoring in the defined Safety Stock Buffer. This quantity is then broken down by the ISS to recommend specific units per size.
4. Finally, Reordering Orchestration ensures all these calculations are complete and synchronized, compiling a single recommendation for SB-DRS-BLUE-M, broken down by size, ready to be acted upon.

Final Outcome

The system produces a clear recommendation: “Reorder 60 units of ‘Summer Breeze’ dress (SB-DRS-BLUE-M) with the following size breakdown: 6 XS, 12 S, 18 M, 12 L, 6 XL, 6 XXL.” This recommendation balances sales velocity with optimal size availability.

Key Takeaway: Reordering Orchestration ensures all necessary data and calculations are seamlessly combined to provide a precise, actionable reorder plan, preventing stockouts and optimizing inventory.

7. Results

This step produces:

Main Outputs

- **Prioritized Reorder List:** A clear list of specific items (SKUs), quantities, and sizes recommended for reordering, ranked by urgency or business impact. This tells us exactly what to buy and how much.
- **Inventory Replenishment Plan:** A detailed schedule indicating when and how much stock should arrive to maintain optimal levels, considering lead times and sales forecasts.
- **Stockout Risk Assessment:** An analysis highlighting products or categories that are at high risk of selling out before the next replenishment, allowing for proactive intervention.

Business Reports

- **Daily Reorder Recommendation Report:** Used by merchandising planners and store managers to initiate purchase orders and manage local inventory.

- **Inventory Health Dashboard:** Used by senior management and merchandising leadership to monitor overall stock levels, identify potential issues, and assess the efficiency of the reordering process.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | The reordering process may pause or proceed with incomplete/outdated data, leading to inaccurate recommendations. | Immediately contact the data team or IT to resolve the data feed issue and manually review pending reorders. |
| Sudden spike in demand (e.g., viral trend) | The system might initially recommend reordering based on historical ROS, potentially understocking the item. | Merchandising planners should manually review and adjust reorder quantities, considering the unexpected demand. |
| Supplier delays or out of stock | Recommended quantities may not be fulfillable, leading to prolonged stockouts. | Collaborate with suppliers to get updated timelines or seek alternative sourcing for critical items. |
| New product launch with no sales history | The system cannot calculate a reliable Rate of Sale (ROS) and may default to a generic order quantity. | Merchandising needs to set initial order quantities based on market trends, competitor data, or similar product performance. |

9. Next Steps

Before This Step

- **Sales Forecasting:** Provides expected future demand for products, which is a critical input for calculating reorder quantities.
- **Inventory Auditing:** Ensures that current stock counts are accurate, which is essential for determining true replenishment needs.

After This Step

- **Purchase Order Generation:** Takes the reorder recommendations and converts them into actual purchase orders to be sent to suppliers.
- **Inbound Logistics Management:** Manages the tracking and receipt of incoming inventory from suppliers into warehouses and stores.

10. Common Questions

Q: How does Reordering Orchestration ensure we don't overstock slow-moving items or sizes? A: It integrates Rate of Sale (ROS) and Ideal Size Set (ISS) calculations. If an item or size sells slowly, its recommended reorder quantity will be low, or it might not be recommended at all, preventing excess inventory.

Q: Can this process adapt if a product's popularity suddenly changes? A: Yes, it regularly updates its Rate of Sale (ROS) calculations based on the latest sales data. For very sudden, unexpected shifts, however, a merchandising planner may need to make a quick manual adjustment.

Q: How does this system handle brand new products that have no sales history yet? A: For new products, the system won't have a Rate of Sale (ROS). Initial orders for these items typically rely on pre-set quantities from merchandising, market analysis, or performance of similar products, rather than algorithmic reordering.

Q: Does it take into account promotional events like a big holiday sale? A: Yes, if promotional sales data is accurately fed into the system, it will be reflected in the Rate of Sale (ROS). For planned promotions, merchandising can often set specific rules or initial orders outside the standard reorder cycle to ensure enough stock.

Q: What if a supplier can only provide a portion of the recommended quantity? A: The system provides the ideal quantity. If a supplier cannot meet this, the purchasing team will need to adjust the actual purchase order, potentially creating a partial order or seeking alternative sourcing, and the system will re-evaluate during the next reorder cycle.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Ensures all reordering calculations happen in the right order with the right data to produce accurate recommendations.
- **Main Input:** Raw sales data, product attributes, and current inventory levels.
- **Main Output:** A precise, prioritized list of what to reorder, in what quantities, and by which sizes.
- **Warning Signs:** Inconsistent reorder suggestions, frequent stockouts despite recommendations, or unexplained inventory surpluses.
- **Key Settings:** Reorder Frequency, Safety Stock Buffer, Minimum Sales Threshold.

Need Help? Contact your Merchandising Operations team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 3: Reordering Input Preparation

Last Updated: 2025-05-23

1. Business Objective

This step loads and organizes all the raw and pre-processed data needed for calculating accurate reorder recommendations.

Why This Matters: By bringing together all essential information — from sales history to supplier lead times and specific business rules — this step ensures that subsequent reordering calculations are based on a complete, accurate, and up-to-date picture, preventing stockouts and optimizing inventory investment.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Transactional Sales Data:** This includes specific units sold for each SKU (Stock Keeping Unit) at every store over various periods. We need it to accurately calculate the Rate of Sale (ROS) and understand historical demand.
- **Predicted Sales & Depletion Outputs:** These are forecasts that predict future sales and how current stock levels will deplete over time. This helps us anticipate future needs and avoid stockouts.

Product Data

- **SKU and Style Master Data:** Detailed information about every product, including its unique SKU, the style it belongs to, and other Attribute Group (AG) details like category, subcategory, and brand. This is vital for grouping products correctly and applying rules.
- **Ideal Size Set (ISS) Data:** This provides the recommended quantity breakdown for each size within a style or category, ensuring we reorder the right mix of sizes.

Operational Data

- **Supplier Lead Times:** The number of days it takes for new stock to arrive from a supplier once an order is placed. This is crucial for determining how much inventory we need to cover demand until replenishment.
- **Reorder Override Rules:** Specific instructions from merchandising teams to adjust reorder quantities for certain products or stores, overriding standard calculations due to special circumstances (e.g., promotional pushes, planned phase-outs).
- **Target Inventory Levels:** These are business rules defining how many days of supply we aim to keep for specific products or categories. They guide how much to reorder to maintain desired stock levels.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|------------------------|--------------------|---|--|
| Target Cover Days | 30 days | The number of days of supply we aim to have for a product or category in a store group. | When market demand changes or supply chain reliability shifts, to adjust desired inventory levels. |
| Lead Time Duration | Varies by supplier | The standard time (in days) expected for goods to arrive after an order is placed. | When a supplier's delivery performance changes consistently, or if new sourcing is introduced. |
| ROS Calculation Period | 90 days | The historical period (in days) used to calculate the Rate of Sale (ROS) for a product or style. | If product demand patterns become highly volatile (shorter period) or very stable (longer period). |
| Reorder Override Rules | None | Specific instructions to manually adjust reorder quantities for certain items or stores, overriding system recommendations. | For promotional events, product phase-outs, or when manual intervention is required for strategic reasons. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw & Pre-processed Data] --> B[Reordering Input Preparation]

B --> C[Centralized Data Stores]

1. Start with → All raw sales data, product master data, depletion forecasts, supplier lead times, and specific reorder rules.
2. Then → The system loads, filters, and validates this diverse set of information.
3. Next → It links related data points, such as connecting a specific SKU to its style, category, and assigned store group, and integrating all applicable rules.
4. Finally → It organizes all this information into structured, central data stores, making it readily accessible for subsequent reordering calculations.

5. Step-by-Step Actions

1. **Gather** all raw and pre-processed data
 - What we do: *We collect current sales figures, predicted stock levels, product descriptions, supplier delivery times, and any specific reorder adjustments.*
 - Why we do it: *To ensure all necessary puzzle pieces are on the table before starting any complex calculations.*
2. **Consolidate** product and store information
 - What we do: *We link each product SKU to its broader style, category, and the store groups it's sold in, and establish connections between stores and their assigned groups.*
 - Why we do it: *This creates a unified view of our inventory, allowing us to apply rules and calculations consistently across similar products and stores.*
3. **Integrate** sales forecasts and business rules
 - What we do: *We pull in the predicted future sales (depletion outputs), expected inbound shipments, and any specific override rules or target inventory levels set by merchandising.*
 - Why we do it: *To ensure reorder calculations are not just based on past performance but also on future expectations and strategic business decisions.*
4. **Initialize** central data stores
 - What we do: *We set up dedicated, easy-to-access storage areas (like maps and lists) for all the loaded and linked data, ready for other modules to use.*
 - Why we do it: *This provides a clean, pre-organized foundation, saving time and preventing errors in later, more complex reordering calculations.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular “Men’s Athletic Shorts” style (Style ID: MA-SHORTS-001) is up for reorder.
- Current sales data shows SKU “MA-SHORTS-001-BLUE-M” sold 150 units in the last 30 days across all stores.
- Depletion forecasts predict current stock of this SKU will last only 10 more days in a key store group (Store Group: “City Core Stores”).

- The supplier lead time for this style is 14 days.
- There's a merchandising override rule for this style due to an upcoming promotion: "Increase reorder quantity by 20% for 'City Core Stores'."
- The Ideal Size Set (ISS) data for "Men's Athletic Shorts" recommends a specific size distribution (e.g., 15% S, 30% M, 30% L, 25% XL).
- Target Cover Days for athletic wear is set to 45 days.

What Happens

1. First, Reordering Input Preparation loads the transactional sales data for "MA-SHORTS-001-BLUE-M" (150 units sold) and its link to Style ID: MA-SHORTS-001.
2. Then, it integrates the predicted depletion outputs, indicating the fast depletion of stock in "City Core Stores" for this SKU. Simultaneously, it loads the 14-day supplier lead time for this style.
3. Next, it loads the Ideal Size Set (ISS) data, linking it to the "Men's Athletic Shorts" category, and brings in the specific reorder override rule (+20% for "City Core Stores"). It also loads the 45-day Target Cover Days setting.
4. Finally, all this information (sales, forecasts, lead times, overrides, ISS, target cover days, and product/store group mappings) is meticulously organized and stored in central data structures, creating a comprehensive and ready-to-use dataset for all subsequent reordering calculations.

Final Outcome

All fragmented pieces of information related to "Men's Athletic Shorts" and its SKUs, including sales, forecasts, lead times, business rules, and size distributions, are now consolidated and accessible in a standardized format. This foundational data set is precisely what the next reordering modules need to calculate an optimal reorder quantity and size breakdown.

Key Takeaway: This step acts as the central intelligence hub, ensuring all relevant and accurate data is perfectly prepared and synchronized before any reordering decisions are made.

7. Results

This step produces:

Main Outputs

- **Centralized Reordering Data Stores:** Pre-populated internal data structures (e.g., maps and lists) containing all relevant sales history, current stock levels, forecasted depletions, lead times, business rules, and product attributes, organized for efficient access by other modules. This provides a single source of truth for reordering calculations.
- **Validated Input Parameters:** All loaded business settings, such as Target Cover Days and ROS Calculation Period, are made available in a consistent format, ready to be applied in subsequent calculations.

- **Pre-filtered Product & Store Data:** Products and stores that meet initial criteria (e.g., active stores, relevant styles) are identified and prepared, reducing the data volume for later processing.

Business Reports

- **Foundational Data for Inventory Reports:** While not a direct end-user report, the output from this module is the backbone for all inventory health dashboards and reorder recommendation reports generated later in the process.
- **Input Data Audit Logs:** Internal logs documenting what data was loaded, from where, and any initial validations performed. This supports data integrity and troubleshooting.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--------------------------------------|---|---|
| Missing or delayed data feed | The reordering process may pause or proceed with incomplete/outdated data, leading to inaccurate recommendations. | Immediately contact the data team or IT to resolve the data feed issue and manually review any critical reorders. |
| Inconsistent product master data | A SKU might be linked to a non-existent style or category, leading to errors in grouping or attribute analysis. | Audit and cleanse product master data regularly to ensure all attributes are correctly populated and consistent. |
| Missing supplier lead times | The system might default to a generic lead time, which could lead to overstocking (if default is too long) or stockouts (if too short). | Establish default lead times or escalate to the procurement team to obtain missing supplier information. |
| Contradictory reorder override rules | Multiple rules applying to the same product or store might conflict, causing the system to apply only one or err out. | Review and prioritize override rules with the merchandising team to ensure clarity and resolve conflicts. |

9. Next Steps

Before This Step

- **Sales Forecasting:** Provides the predicted sales and depletion outputs, which are critical for future-looking reorder calculations.
- **Inventory Auditing:** Ensures that current physical stock counts are accurate, forming the baseline for understanding actual inventory needs.

After This Step

- **Rate of Sale (ROS) Calculation:** Uses the prepared sales data to determine how quickly products are selling.
- **Ideal Size Set (ISS) Calculation:** Leverages product and sales data to determine the best size combinations for replenishment.
- **Optimum Depth (OD) and Optimum Width (OW) Calculation:** Utilizes prepared data to determine the ideal range and variety of products to stock.

10. Common Questions

Q: Why is this step needed if the data already exists in our systems? A: While the data exists, it often comes from different sources and formats. This step standardizes, links, and pre-processes everything into a unified, ready-to-use format that is optimized for reordering calculations, saving time and preventing errors downstream.

Q: What happens if some required data, like supplier lead times, is missing? A: If specific data is missing, the system might use default values, or in critical cases, flag an error. Using defaults can lead to suboptimal reorders, so it's best to ensure all inputs are complete.

Q: Does this step make any reordering decisions or recommendations? A: No, this step is purely preparatory. It loads and organizes the data, but it doesn't perform any calculations or generate recommendations. It sets the stage for subsequent modules that do the actual analysis and decision-making.

Q: How often is the data for reordering prepared? A: The frequency of data preparation typically aligns with the Reorder Frequency setting (e.g., weekly). This ensures that the reordering calculations are always based on the most recent sales trends and stock levels.

Q: Can manual adjustments to reorder quantities be made during this preparation phase? A: This phase loads pre-defined "reorder override rules" as inputs. It doesn't allow for live manual adjustments. Any manual changes needed for a specific reorder cycle should be entered as an override rule *before* this process runs.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Loads and organizes all necessary raw and pre-processed data for reordering calculations.
 - **Main Input:** Transactional sales data, product master data, depletion forecasts, lead times, and reorder override rules.
 - **Main Output:** Centralized, structured data stores ready for reordering analysis.
 - **Warning Signs:** Inconsistent data relationships, missing critical inputs, or unexplained calculation errors in subsequent steps.
 - **Key Settings:** Target Cover Days, Lead Time Duration, ROS Calculation Period.
-

Need Help? Contact your Merchandising Operations team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: Paramount Sizes & Sales Cleanup

Last Updated: 2025-05-23

1. Business Objective

This step identifies “paramount sizes,” which are the top revenue-contributing sizes within specific product categories (Attribute Groups), and then meticulously cleans sales data by filtering out transactions that don’t meet minimum active store or essential size availability criteria.

Why This Matters: By focusing on sales that truly reflect customer demand for a well-stocked assortment, this process ensures that subsequent reorder calculations are based on the most relevant and accurate data. This helps prevent stockouts of crucial, high-demand sizes, reduces unnecessary inventory of slow-moving sizes, and ultimately maximizes sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Transactional Sales Data (Units Sold & Revenue):** This includes specific units sold for each product and the revenue generated from those sales. We need it to accurately calculate the revenue contribution of each size and determine which are “paramount.”
- **Historical Sales Trends:** This helps us see patterns in past sales, such as which sizes consistently drive revenue, informing the identification of paramount sizes.

Product Data

- **Product Attributes (e.g., category, style, color, size, brand, gender):** These details are crucial for grouping sales data into specific Attribute Groups (AGs), allowing for a precise analysis of size performance within similar products.
- **Store Activity Data:** Information on which product sizes were actively available in each store on specific days. This is used to ensure that sales data used for analysis truly reflects demand when products were adequately stocked.

Operational Data

- **Minimum Active Store Sales Threshold:** This business rule defines the minimum percentage of your total stores that must have had a product available for its sales to be considered valid for analysis.
- **Top Revenue Contribution Percentage:** This setting determines the cumulative revenue percentage that defines “Paramount Sizes.” Sizes contributing up to this percentage are deemed essential.
- **Essential Size Availability Threshold:** This rule specifies the minimum percentage of “paramount sizes” that must have been in stock for a sales transaction to be considered valid and included in the cleaned data.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|---------------|--|--|
| Top Revenue Contribution Percentage | 80% | The cumulative percentage of revenue used to define “Paramount Sizes” within an Attribute Group. Sizes are ranked by revenue, and those contributing up to this percentage are deemed paramount. | If your business observes that a different percentage of sizes drive the majority of revenue, or to narrow/broaden the definition of “essential” sizes. |
| Minimum Active Store Sales Threshold | 20% | The minimum percentage of your total stores that must have had a product available for sale on a given day for its sales data to be included in calculations. | If you introduce more test stores, or if your operational definition of “live” stores changes, to ensure sales data accurately reflects broad market demand. |
| Essential Size Availability Threshold | 70% | The minimum percentage of the identified “Paramount Sizes” that must have been in stock and available in a store when a | To tighten or loosen the strictness of sales data cleanup, ensuring that only sales from well-stocked assortments contribute to future reordering. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|----------------|
| | | sale occurred for that sale to be included in the cleaned data. | |

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Data & Product Attributes] --> B[Paramount Sizes & Sales Cleanup]

B --> C[Cleaned Sales Data & Paramount Size List]

1. Start with → All raw sales transactions and detailed product attributes.
2. Then → The system groups sales data by Attribute Groups and identifies the top revenue-contributing sizes within each group, labeling them as “Paramount Sizes.”
3. Next → It applies two critical filters to the raw sales data: ensuring sales come from a minimum number of active stores and that a sufficient percentage of Paramount Sizes were available during the sale.
4. Finally → It outputs a refined, accurate set of sales data and a definitive list of “Paramount Sizes” for each Attribute Group.

5. Step-by-Step Actions

1. **Group** sales data by relevant Attribute Groups.
 - What we do: *We organize raw sales transactions by product characteristics like category, style, brand, and gender to create meaningful analysis segments.*
 - Why we do it: *This allows us to understand sales performance and size contribution at a granular level, rather than treating all products uniformly.*
2. **Identify** Paramount Sizes.
 - What we do: *Within each Attribute Group, we calculate the revenue contribution of each size and determine which sizes collectively account for a predefined top percentage of total revenue.*
 - Why we do it: *This pinpoints the most critical, revenue-driving sizes that must always be in stock, forming a dynamic Ideal Size Set (ISS) based on actual sales.*
3. **Filter** sales based on minimum active store criteria.
 - What we do: *We review each sales transaction and only include those where the product was actively available in a minimum percentage of our total stores on the day of the sale.*
 - Why we do it: *This removes sales from limited availability or test runs, ensuring our sales data reflects broad market demand and not isolated incidents.*

4. **Filter** sales based on essential size availability.

- What we do: *For each sales transaction, we verify that a sufficient percentage of the identified “Paramount Sizes” for that product’s Attribute Group were available in the store during the sale.*
- Why we do it: *This cleans up sales data by excluding transactions that occurred when a significant portion of crucial sizes were already out of stock, preventing distorted demand signals and ensuring reorders are based on true selling potential.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A “Women’s Casual Tops” category (Style ID: WCT-005) sold 1,000 units last month across various sizes (XS, S, M, L, XL, XXL).
- Initial analysis shows that Medium (M) and Large (L) sizes combined generate 70% of the revenue for WCT-005, even though they represent only 30% of total units.
- Our “Top Revenue Contribution Percentage” is set to 75%.
- Our “Minimum Active Store Sales Threshold” is 20% (meaning a product must be live in at least 20% of our stores for its sales to count).
- Our “Essential Size Availability Threshold” is 60%.
- One transaction shows 5 units of WCT-005 in XS sold at Store A, but on that day, Store A had only XS and S in stock, with M and L (the potentially paramount sizes) being out of stock.

What Happens

1. First, the system groups all sales for “Women’s Casual Tops” and calculates the revenue contribution of each size. It identifies that sizes Medium (M) and Large (L) collectively contribute over 75% of the total revenue for this style. These two sizes (M, L) are now designated as the “Paramount Sizes” for WCT-005.
2. Then, the system checks each sales transaction. For the sale of 5 units of WCT-005-XS at Store A, it first verifies if WCT-005 was active in at least 20% of all stores. Let’s assume it was.
3. Next, it checks the “Essential Size Availability Threshold.” At Store A on that day, only XS and S were in stock. The identified Paramount Sizes (M, L) were not available. This means 0% of the Paramount Sizes (M & L) were available, which is less than the 60% threshold.
4. Finally, because the sale of WCT-005-XS at Store A did not meet the “Essential Size Availability Threshold” (as the paramount sizes were out of stock), this specific sales transaction is filtered out of the cleaned sales data. Other sales transactions that occurred when M and L were available are retained.

Final Outcome

The sales data now accurately reflects demand when critical sizes are present, preventing over-reordering of less popular sizes based on skewed historical data. The list of Paramount Sizes (M and L for WCT-005) is saved for future ISS calculations.

Key Takeaway: This process ensures that reorder decisions are based on true sales potential, not just what was sold when crucial sizes were unavailable, leading to better stock assortment.

7. Results

This step produces:

Main Outputs

- **Cleaned Sales Data:** A filtered dataset of sales transactions, validated against minimum store activity and essential size availability, providing a more accurate basis for Rate of Sale (ROS) calculations.
- **Paramount Sizes List:** A definitive, attribute-group-specific list of sizes identified as the top revenue contributors, serving as a critical input for refining Ideal Size Set (ISS) recommendations.

Business Reports

- **Paramount Size Contribution Report:** Used by merchandising and planning teams to understand which sizes drive the most revenue within specific product categories and adjust marketing or inventory strategies accordingly.
- **Sales Data Filtering Insights:** An internal report for data and operations teams, indicating the volume of sales data removed during cleanup and the reasons (e.g., “insufficient live stores,” “paramount sizes unavailable”), helping to identify operational issues and improve store availability.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|---|
| Missing or delayed data feed | The reordering process may pause or proceed with incomplete/outdated sales data, leading to inaccurate Paramount Size identification and cleanup. | Immediately contact the data team or IT to resolve the data feed issue and manually verify any critical Paramount Size lists or cleaned sales data before proceeding. |
| Very few sales data points for an Attribute Group | If an Attribute Group has minimal sales, identifying “Paramount Sizes” might be | Merchandising should consider a broader Attribute Group or apply a default Ideal Size Set (ISS) for |

| Situation | What Happens | What You Should Do |
|---|--|---|
| | unreliable, or the cleanup filters might remove too much valid data. | these low-volume categories, or manually set initial orders. |
| All sizes within an Attribute Group contribute equally to revenue | If no sizes clearly dominate revenue, the “Top Revenue Contribution Percentage” might result in classifying all or too many sizes as “Paramount.” | Review the “Top Revenue Contribution Percentage” setting for this specific Attribute Group or rely on broader category-level Ideal Size Set (ISS) guidelines if a clear revenue-driving size pattern is absent. |
| Paramount sizes are consistently out of stock | If the “Paramount Sizes” are rarely available, the “Essential Size Availability Threshold” will filter out most sales, potentially leading to an underestimation of demand for those critical sizes. | Investigate the root cause of persistent stockouts for Paramount Sizes (e.g., supplier issues, incorrect forecasting) and manually review sales data or adjust thresholds to reflect true demand. |

9. Next Steps

Before This Step

- **Reordering Input Preparation:** This crucial step provides the raw, consolidated sales data, product attributes, and operational settings that this process needs for its analysis.
- **Sales Forecasting:** Delivers predicted sales figures which, while not directly used here for filtering, provide the broader context of expected demand for which sales are being cleaned.

After This Step

- **Rate of Sale (ROS) Calculation:** This step uses the cleaned sales data generated here to calculate accurate sales velocities, which are crucial for precise reorder quantity determination.
- **Ideal Size Set (ISS) Calculation:** This step utilizes the identified Paramount Sizes as a primary input to refine and optimize the recommended size breakdown for future purchase orders, ensuring the right mix of sizes is always in stock.

10. Common Questions

Q: Why are some sizes called “paramount” and how does the system decide which ones they are? A: “Paramount sizes” are the most crucial sizes within a product category that collectively generate the largest portion of sales revenue. The system identifies them by analyzing historical sales data, ranking sizes by their revenue contribution, and selecting those that cumulatively meet a set percentage of total revenue for that category.

Q: How does filtering or “cleaning up” sales data improve our reordering process?

A: Cleaning sales data removes misleading information. For example, if a popular size sells out quickly, its recorded sales might look low. By filtering out sales that happened when key sizes were out of stock, we get a truer picture of demand for a well-stocked assortment, leading to more accurate reorder recommendations and fewer lost sales.

Q: What happens if a new, unexpected size becomes very popular that wasn’t previously identified as “paramount”? A: The system regularly updates its analysis based on the latest sales data. If a new size consistently starts contributing significantly to revenue, it will eventually be identified as a “paramount size” in subsequent re-evaluations, and future Ideal Size Sets will adjust to include it, ensuring we respond to emerging trends.

Q: Can the business adjust how “paramount sizes” are defined or how strictly sales data is cleaned? A: Yes, there are adjustable business settings. You can modify the “Top Revenue Contribution Percentage” to include more or fewer sizes as “paramount.” Similarly, the “Minimum Active Store Sales Threshold” and “Essential Size Availability Threshold” can be adjusted to control how strictly sales data is filtered, allowing flexibility to match business needs and risk tolerance.

Q: Does this process affect all product categories equally, or are some more impacted than others? A: This process impacts categories differently. Categories with very distinct size performance (e.g., apparel where certain sizes are clearly bestsellers) will see a clearer identification of “paramount sizes.” Categories with more uniform size sales or fewer distinct sizes might have more sizes deemed “paramount” or less impact from the filtering criteria.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Identifies key revenue-driving sizes (Paramount Sizes) and refines sales data for accurate reordering.
 - **Main Input:** Raw sales transactions, detailed product attributes, and current store activity data.
 - **Main Output:** Cleaned sales data and a definitive list of Paramount Sizes for each Attribute Group.
 - **Warning Signs:** Unrealistic Paramount Size lists (too many or too few sizes), or unexpected amounts of sales data being filtered out.
 - **Key Settings:** Top Revenue Contribution Percentage, Minimum Active Store Sales Threshold, Essential Size Availability Threshold.
-

Need Help? Contact your Merchandising Operations team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 5: Reorder Style Identification

Last Updated: 2025-05-23

1. Business Objective

This step categorizes each product style as 'REORDER1', 'REORDER2', or 'DONT_REORDER' based on its historical sales performance, discount levels, revenue contribution, Rate of Sale (ROS) per day, and presence across stores.

Why This Matters: This ensures that our reordering efforts are focused on the most profitable and high-demand styles, minimizing inventory risk for underperforming items and maximizing sales by keeping essential products consistently in stock.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** We need to know the total number of units sold for each style over a specific period. This helps us identify styles with sufficient customer demand to warrant reordering.
- **Revenue Generated:** The total income earned from each style. This is crucial for identifying styles that contribute significantly to overall business revenue.
- **Average Discount Percentage:** The average discount applied to sales of a style. This helps us understand if sales are driven by healthy demand or heavy markdowns, influencing reorder decisions.

Product Data

- **Product Attributes (e.g., category, style, brand, price bucket):** These details are vital for grouping styles into Attribute Groups (AGs) and for comparing style performance against relevant benchmarks (e.g., comparing a dress to other dresses in its category).
- **Style Master Information:** Unique identifiers and descriptive data for each product style, allowing us to track and categorize them accurately.

Operational Data

- **Store Groupings:** Information on how stores are grouped (e.g., by region, sales volume). This allows for localized analysis and reorder decisions that consider regional performance.

- **Manual Reorder Override Rules:** Specific instructions from merchandising teams to manually adjust the reorder category for certain styles (e.g., forcing a 'DONT_REORDER' for an item being phased out).
- **Paramount Sizes List:** A list of the most critical, revenue-driving sizes within an AG. This is used to determine how many “healthy” days a style was truly available for sale, impacting its ROS calculation.
- **Store Activity Data:** Information on which stores had a specific style actively available for sale on a given day. This helps gauge a style’s market exposure and healthy selling days.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|--------------------|---|---|
| Revenue Contribution Benchmark for Styles | 80% | The cumulative percentage of revenue used to define top-tier styles within an Attribute Group. Styles contributing up to this percentage are considered top performers. | If you observe that a different percentage of styles drives the majority of revenue, or to narrow/broaden the definition of “essential” styles. |
| Minimum Sales Threshold for Category | Varies by category | The minimum number of units a style must have sold within a category to be considered for active reordering. | To tighten inventory control on underperforming items or relax it for new product launches, ensuring faster sell-through. |
| REORDER1 Discount Multiplier | 1.0 | A factor applied to the average category discount. Styles with an average discount <i>less than or equal to</i> this multiplied benchmark can qualify for REORDER1. | To adjust how strictly discount levels impact a style’s eligibility for top-tier reordering. |
| Minimum Store Exposure Percentage | 20% | The minimum percentage of stores within a store group where a style must have been actively live for | If your operational definition of “widespread availability” changes, to ensure styles have had |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|-----------------------------|
| | | its sales to be considered significant for REORDER2 classification. | sufficient market exposure. |

Note: See More Details for additional settings regarding ROS and discount multipliers for REORDER1 and REORDER2 categories.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Sales Data & Product Attributes] --> B[Reorder Style Identification]

B --> C[Categorized Style List]

1. Start with → Cleaned sales data, detailed product attributes, store groupings, and any manual override rules.
2. Then → The system calculates various performance metrics for each style, such as Units Sold, Average Discount, Revenue Contribution, ROS per day, and the number of stores it was available in.
3. Next → Styles are evaluated against predefined criteria, first for 'REORDER1' (highest priority), then for 'REORDER2' (medium priority), ensuring no overlap.
4. Finally → Manual override rules are applied, and each style is assigned its final category: 'REORDER1', 'REORDER2', or 'DONT_REORDER'.

5. Step-by-Step Actions

1. **Collect** relevant sales and product data.
 - What we do: *We gather cleaned sales transactions and product details from a defined Analysis Period, focusing on stores eligible for reordering.*
 - Why we do it: *This ensures our analysis is based on recent, reliable performance data from active selling environments.*
2. **Calculate** style performance metrics.
 - What we do: *We determine the total Units Sold for each style, its average discount, its overall revenue contribution, its ROS (Rate of Sale) per day, and the number of stores where it was actively available.*
 - Why we do it: *These metrics provide a comprehensive view of a style's health and selling power.*
3. **Identify** 'REORDER1' styles.
 - What we do: *We apply stringent criteria to styles, looking for high Units Sold (above a Minimum Sales Threshold for Category), healthy discounts (below a benchmark), strong revenue contribution (within the top percentage for its Attribute Group), and robust ROS per day.*
 - Why we do it: *This flags our top-performing, most profitable styles that should be prioritized for immediate reordering to prevent stockouts.*

4. Identify 'REORDER2' styles.

- What we do: *We evaluate remaining styles using slightly less strict discount and ROS criteria, but still require a Minimum Store Exposure Percentage to ensure broad market appeal.*
- Why we do it: *This identifies styles that are performing well but not at the top tier, still warranting reorder attention to maintain a broad, appealing assortment.*

5. Apply manual override rules.

- What we do: *We incorporate any specific instructions from merchandising, such as forcing a style into the 'DONT_REORDER' category, regardless of its calculated performance.*
- Why we do it: *This allows for strategic business decisions (e.g., product phase-outs, special promotions) to take precedence over algorithmic recommendations.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A "Women's Activewear Leggings" style (Style ID: AWL-001) has been sold in the last 60 days (our Analysis Period).
- Sales data shows AWL-001 sold 450 units, generated \$20,000 in revenue, and had an average discount of 12%.
- For its Attribute Group (Women's Activewear), the **Revenue Contribution Benchmark for Styles** is 80%, and AWL-001 contributes 4% to the category's total revenue, placing it within the top 15% of styles.
- The **Minimum Sales Threshold for Category** is 100 units.
- The benchmark average discount for Women's Activewear is 15%, and the **REORDER1 Discount Multiplier** is 1.0.
- The benchmark ROS for Women's Activewear is \$80/day/style, and the **REORDER1 ROS Multiplier** is 1.2. AWL-001's ROS per day is \$110/day.
- AWL-001 was live in 75% of its assigned store group. The **Minimum Store Exposure Percentage** is 20%.

What Happens

1. First, the system gathers AWL-001's sales data (450 units, \$20,000 revenue, 12% discount) and confirms it sold over the 100-unit Minimum Sales Threshold for Category.
2. Then, it evaluates AWL-001 for 'REORDER1' criteria:
 - **Units Sold:** 450 units are ≥ 100 units (Meets).
 - **Average Discount:** 12% is $\leq (1.0 * 15\%) = 15\%$ (Meets).
 - **Revenue Contribution:** Being in the top 15% of styles for its AG means it's within the 80% Revenue Contribution Benchmark (Meets).
 - **ROS per day:** \$110/day is $\geq (1.2 * \$80/\text{day}) = \$96/\text{day}$ (Meets).
3. Since AWL-001 meets all 'REORDER1' criteria, it is initially categorized as 'REORDER1'. The system then checks for any manual override rules for AWL-001. If none exist, the classification stands.

Final Outcome

The style “Women’s Activewear Leggings” (AWL-001) is clearly identified as a ‘REORDER1’ style, ensuring it receives top priority for replenishment and inventory investment.

Key Takeaway: This process systematically evaluates styles against robust performance criteria to automatically classify them, ensuring reordering efforts focus on the most impactful products.

7. Results

This step produces:

Main Outputs

- **Categorized Style List:** A definitive list of all product styles, clearly marked as ‘REORDER1’, ‘REORDER2’, or ‘DONT_REORDER’. This directly informs which styles are prioritized for replenishment.
- **Style Performance Metrics:** Detailed calculated values for each style, including its total Units Sold, Average Discount, Revenue Contribution, ROS (Rate of Sale) per day, and number of active stores. These provide the underlying data for the categorization.

Business Reports

- **Reorder Eligibility Report:** Used by merchandising planners and inventory managers to quickly see which styles are recommended for reorder and their priority level, streamlining purchase order creation.
- **Style Health Dashboard:** Used by senior management and category managers to monitor the overall performance and reorder status of product lines, helping to identify trends and strategic opportunities or risks.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--------------------------------------|--|---|
| Missing or delayed data feed | The reordering process may pause or proceed with incomplete/ outdated data, leading to inaccurate style categorizations. | Immediately contact the data team or IT to resolve the data feed issue and manually review any critical reorder style classifications. |
| New style with limited sales history | The system may not have enough data to accurately categorize the style, potentially defaulting it to ‘DONT_REORDER’. | Merchandising needs to manually assign an initial reorder category or apply override rules based on market trends or similar product performance. |

| Situation | What Happens | What You Should Do |
|---|--|--|
| All styles within an Attribute Group are heavily discounted | If most styles sell at deep discounts, the average discount benchmark might be very low, making it difficult for any style to meet the discount criteria for 'REORDER1' or 'REORDER2'. | Review and potentially adjust the REORDER1 Discount Multiplier or REORDER2 Discount Multiplier settings for that specific Attribute Group or category. |
| Manual override conflicts with calculated category | If an override rule contradicts the system's calculation (e.g., system says 'REORDER1', override says 'DONT_REORDER'), the override takes precedence. | This is intended behavior, but regularly audit override rules to ensure they are still strategically relevant and accurate. |

9. Next Steps

Before This Step

- **Reordering Input Preparation:** Provides the raw, consolidated sales data, product attributes, store groupings, and any manual override rules required for this categorization.
- **Paramount Sizes & Sales Cleanup:** Delivers the cleaned sales data and the definitive list of Paramount Sizes, which are crucial for calculating accurate ROS (Rate of Sale) per day.

After This Step

- **Reorder Quantity Calculation:** Utilizes the 'REORDER1' and 'REORDER2' classifications to determine the precise quantities of each style and size to reorder, factoring in demand and stock levels.
- **Purchase Order Generation:** Takes the finalized reorder quantities and categories to create actual purchase orders that are sent to suppliers, initiating the replenishment process.

10. Common Questions

Q: How does the system decide if a style should be reordered, and why are there 'REORDER1' and 'REORDER2' categories? A: The system uses a set of business criteria like Units Sold, revenue contribution, and ROS to determine reorder eligibility. 'REORDER1' styles are top performers, highly profitable, and need

constant replenishment, while ‘REORDER2’ styles are solid performers that also warrant reordering but might have slightly different characteristics, ensuring we cover a wider range of profitable products.

Q: What happens if a style is very popular in terms of units sold but always on deep discount? A: A style’s average discount is a key criterion. If a style consistently requires deep discounts to sell, it might not qualify for ‘REORDER1’ or even ‘REORDER2’, even if it sells many units. This prevents us from continuously reordering items that erode profit margins.

Q: Can we manually change a style’s reorder category, for example, if we want to phase out a product? A: Yes, manual “reorder override rules” allow merchandising teams to adjust a style’s classification. For instance, you can apply an override to force a popular style into the ‘DONT_REORDER’ category if it’s being phased out or is part of a special, non-replenishable collection.

Q: What happens to styles that don’t fall into the ‘REORDER1’ or ‘REORDER2’ categories? A: Styles that don’t meet the criteria for either ‘REORDER1’ or ‘REORDER2’ are automatically classified as ‘DONT_REORDER’. This signals that these styles should not be replenished, helping to reduce excess inventory and clear out slow-moving or unprofitable items.

Q: Does this process consider a style’s performance in different types of stores or regions? A: Yes, the system uses store groupings in its analysis. This allows the system to compare a style’s performance within its relevant store group (e.g., high-volume urban stores vs. smaller suburban stores), ensuring that reorder decisions are tailored to specific market demands and store types.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Categorizes product styles into ‘REORDER1’, ‘REORDER2’, or ‘DONT_REORDER’ based on comprehensive performance metrics.
- **Main Input:** Cleaned sales data, detailed product attributes, store activity, and manual override rules.
- **Main Output:** A definitive list of styles categorized for reordering priority.
- **Warning Signs:** Unrealistic style classifications (e.g., a known bestseller is ‘DONT_REORDER’), or excessive manual overrides being required.
- **Key Settings:** Revenue Contribution Benchmark for Styles, Minimum Sales Threshold for Category, REORDER1 Discount Multiplier, Minimum Store Exposure Percentage.

Need Help? Contact your Merchandising Operations team for questions about this process.

Chapter 6: Ideal Size Set (ISS) Calculation

Last Updated: 2025-05-23

1. Business Objective

This step determines the Ideal Size Set (ISS) contribution for each product size, both for individual stores and for larger store groups, ensuring we stock the optimal mix of sizes to meet customer demand.

Why This Matters: This ensures that our stores always have the right sizes available for our customers, preventing popular sizes from selling out and minimizing excess inventory of less popular ones. Ultimately, this maximizes sales, improves customer satisfaction, and optimizes inventory investment.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold by Store and Category:** We need to know the total number of units sold for each product category within every store. This data provides the sales volume to correctly weight the contribution of each size.
- **Cleaned Sales Data:** Sales transactions that have been filtered to ensure they reflect demand when products were adequately stocked, preventing skewed size performance analysis.

Product Data

- **Product Attributes (Attribute Group, Category, Size):** These details are vital for grouping products into specific categories (Attribute Groups, AGs) and understanding which sizes belong to which products.
- **Pre-calculated Size Contribution Data:** This information provides an initial, adjusted percentage of how much each size is expected to contribute to sales within a specific product category (Attribute Group) for a given store.

Operational Data

- **Store Groupings:** Information on how individual stores are combined into larger groups (e.g., by region, sales volume). This allows us to calculate an Ideal Size Set that is representative of broader market demand.
- **Reorder Store Eligibility:** Information on which stores are currently active and eligible for reordering. This ensures that only relevant sales data from operational stores are included in the calculation.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|---------------|--|---|
| Top Revenue Contribution Percentage | 80% | The cumulative percentage of revenue used to define “Paramount Sizes” within an Attribute Group. While set in a previous step, it directly influences the “Pre-calculated Size Contribution Data” used here. | If your business observes that a different percentage of sizes drive the majority of revenue, or to narrow/broaden the definition of “essential” sizes. |
| Essential Size Availability Threshold | 70% | The minimum percentage of “Paramount Sizes” that must have been in stock for a sale to be considered valid for analysis. This setting, from a previous step, ensures the sales data used here is accurate. | To tighten or loosen the strictness of sales data cleanup, ensuring that only sales from well-stocked assortments contribute. |
| ROS Calculation Period | 90 days | The historical period (in days) used to calculate the Rate of Sale (ROS). This setting, from an earlier step, determines the sales data timeframe | If product demand patterns become highly volatile (shorter period) or very stable (longer period). |

| Setting Name | Default Value | Description | When to Change |
|--------------------------|---------------|--|---|
| Reorder Frequency | Weekly | feeding into this calculation.
How often the system evaluates and generates reorder recommendations for categories. This setting dictates how frequently this ISS calculation will run. | If your supply chain or sales cycles change significantly (e.g., moving to bi-weekly for slow-moving categories). |

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Pre-calculated Size Contribution Data & Cleaned Sales Data]
    --> B[Ideal Size Set (ISS) Calculation]
    B --> C[Store & Store Group Level ISS Percentages]
  
```

1. Start with → Initial size contribution data (pivotal data) and cleaned sales figures from individual stores and product categories.
2. Then → The system calculates the Ideal Size Set (ISS) percentages for each size within an Attribute Group at the individual store level.
3. Next → It aggregates these store-level calculations, weighting them by actual sales volume, to determine the Ideal Size Set for broader store groups.
4. Finally → It outputs the precise percentage breakdown for each size for both individual stores and consolidated store groups.

5. Step-by-Step Actions

1. **Gather** pre-calculated size contributions and sales data.
 - What we do: *We collect the initial percentage contributions for each size within a product category, along with the actual sales volume for each store and product category.*
 - Why we do it: *These are the foundational numbers needed to understand customer demand and allocate inventory by size effectively.*
2. **Calculate** Ideal Size Set for individual stores.
 - What we do: *For each active store, we apply the pre-calculated size contribution percentages to determine the optimal percentage breakdown for each size within a specific product category (Attribute Group).*
 - Why we do it: *This ensures that each store's inventory is precisely tailored to its local customer demand and historical sales patterns for specific sizes.*

3. **Aggregate** Ideal Size Set for store groups.
 - What we do: *We combine the store-level sales volumes and their respective size contributions to calculate an overall optimal size mix for larger groups of stores (e.g., all stores in a specific region).*
 - Why we do it: *This provides a consolidated, weighted view for central purchasing and allocation decisions, especially for replenishing common styles across multiple stores within a group.*
4. **Normalize** size contributions to percentages.
 - What we do: *We convert the calculated size contributions into percentages, ensuring that the total contribution for all sizes within a category sums exactly to 100%.*
 - Why we do it: *This makes the Ideal Size Set easily applicable for determining exact unit quantities to reorder, based on a total target stock level or Optimum Depth (OD).*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A “Men’s Casual Shirts” category (Attribute Group: 123) is sold in Store A (ID: 101) and Store B (ID: 102). Both stores belong to the “East Coast” Store Group (SG: 10).
- **Store A’s Pre-calculated Size Contribution Data** for AG 123: Small (S) = 20%, Medium (M) = 40%, Large (L) = 30%, Extra Large (XL) = 10%.
- **Store B’s Pre-calculated Size Contribution Data** for AG 123: Small (S) = 15%, Medium (M) = 35%, Large (L) = 40%, Extra Large (XL) = 10%.
- Sales data for AG 123 last week: Store A sold 100 units. Store B sold 200 units.

What Happens

1. First, the system directly establishes the Ideal Size Set for each individual store using their respective pre-calculated size contribution data:
 - For Store A (AG 123): S: 20%, M: 40%, L: 30%, XL: 10%.
 - For Store B (AG 123): S: 15%, M: 35%, L: 40%, XL: 10%.
2. Then, to calculate the Ideal Size Set for the “East Coast” Store Group, the system combines the size contributions from Store A and Store B, weighting them by their respective sales volumes.
 - For Size S: $(100 \text{ units} * 20\%) + (200 \text{ units} * 15\%) = 20 + 30 = 50$ weighted units.
 - For Size M: $(100 \text{ units} * 40\%) + (200 \text{ units} * 35\%) = 40 + 70 = 110$ weighted units.
 - For Size L: $(100 \text{ units} * 30\%) + (200 \text{ units} * 40\%) = 30 + 80 = 110$ weighted units.
 - For Size XL: $(100 \text{ units} * 10\%) + (200 \text{ units} * 10\%) = 10 + 20 = 30$ weighted units.
 - The total weighted contribution for the Store Group is $50 + 110 + 110 + 30 = 300$ (which is also the total sales of 100 units from Store A + 200 units from Store B).

3. Finally, these weighted contributions are normalized into percentages for the “East Coast” Store Group by dividing each size’s weighted units by the total weighted units (300).

Final Outcome

The system produces Ideal Size Set percentages: - For Store A (AG 123): S: 20%, M: 40%, L: 30%, XL: 10%. - For Store B (AG 123): S: 15%, M: 35%, L: 40%, XL: 10%. - For Store Group “East Coast” (AG 123): - S: $(50 / 300) = 16.67\%$ - M: $(110 / 300) = 36.67\%$ - L: $(110 / 300) = 36.67\%$ - XL: $(30 / 300) = 10.00\%$ This provides a precise size distribution for both individual stores and for central procurement, ensuring the right sizes are available where needed.

Key Takeaway: This process ensures that Ideal Size Sets are tailored to the specific sales patterns of individual stores and weighted by sales volume for store groups, preventing missed sales and optimizing inventory.

7. Results

This step produces:

Main Outputs

- **Ideal Size Set (ISS) Percentages by Store and Attribute Group:** Specific size breakdowns for each store, showing the optimal percentage contribution for each size within a product category. This directly guides local store replenishment.
- **Ideal Size Set (ISS) Percentages by Store Group and Attribute Group:** Consolidated size breakdowns for larger store groupings, reflecting the weighted demand across multiple stores. This is crucial for centralized purchasing and allocation.

Business Reports

- **Size Performance Report:** Used by merchandising planners to analyze actual sales performance against the calculated Ideal Size Set, identifying deviations or opportunities for optimization in size assortment.
- **Reorder Recommendation Breakdown:** The output from this step directly feeds into the final reorder lists, providing the exact quantity to order for each specific size of a product.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The calculation might use outdated or incomplete pre-calculated size contribution data or | Immediately contact the data team or IT to resolve the data feed issue and manually review any |

| Situation | What Happens | What You Should Do |
|---|--|---|
| | sales figures, leading to inaccurate Ideal Size Sets. | critical Ideal Size Sets before proceeding. |
| New Attribute Group with no sales history | If a new product category has very limited or no sales history, the system may not have enough data to calculate a reliable Ideal Size Set. | Merchandising needs to set an initial, default Ideal Size Set based on market trends, similar product performance, or manual intuition. |
| Insufficient sales for an Attribute Group | If a product category has extremely low sales volume across stores, the Ideal Size Set calculation might be unreliable or volatile, potentially fluctuating greatly with minor sales. | Merchandising should consider using a broader category's Ideal Size Set or a default set for these low-volume items. |
| "Paramount Sizes" are consistently out of stock | If the crucial "Paramount Sizes" within an Attribute Group are rarely available in stores, the pre-calculated size contribution data might be skewed, leading to an ISS that doesn't reflect true customer demand. | Investigate the root cause of persistent stockouts for Paramount Sizes and consider adjusting inventory strategies upstream, or manually reviewing sales data to correct for missed demand. |

9. Next Steps

Before This Step

- **Paramount Sizes & Sales Cleanup:** This crucial step identifies the most revenue-driving sizes and provides the cleaned sales data, both of which are essential inputs for generating accurate "Pre-calculated Size Contribution Data" (pivotal data) for this process.
- **Reordering Input Preparation:** This step consolidates all raw data, including product attributes, sales figures, and store groupings, preparing it for use in this calculation.

After This Step

- **Reorder Quantity Calculation:** This step takes the Ideal Size Set percentages generated here and applies them to the overall reorder quantities, breaking down what to order into specific units per size.

- **Optimum Depth (OD) and Optimum Width (OW) Calculation:** The precise size distribution defined by the Ideal Size Set directly informs the Optimum Depth (OD) (how many units of a specific product to stock) and supports Optimum Width (OW) planning by ensuring the right mix of styles.

10. Common Questions

Q: How does the “Ideal Size Set” improve our inventory? A: The Ideal Size Set ensures you stock more of the sizes that sell well and less of those that don’t. This means fewer missed sales because popular sizes are in stock, and less money tied up in slow-moving inventory, leading to better overall profitability and store appearance.

Q: Why do stores have different Ideal Size Sets for the same product? A: Customer demand can vary significantly by location. For example, a store in a bustling urban area might sell more small sizes, while a store in a suburban area might sell more large sizes. This system calculates a unique Ideal Size Set for each store to perfectly match its local customer base.

Q: How often is the Ideal Size Set updated? A: The Ideal Size Set is recalculated regularly, typically aligning with the “Reorder Frequency” setting (e.g., weekly). This ensures that the size recommendations always reflect the latest sales trends and customer preferences, allowing us to quickly adapt to changing demand.

Q: What happens if a new popular size emerges that wasn’t part of our previous ISS? A: The system continuously monitors sales data. If a previously less popular size starts to sell significantly more, its contribution will increase in the “Pre-calculated Size Contribution Data” over time. In subsequent calculations, it will then be included in the Ideal Size Set, ensuring we respond to emerging trends.

Q: How does this help us avoid stockouts for popular sizes? A: By precisely identifying the most popular sizes and their ideal contribution, the Ideal Size Set ensures that reorder recommendations prioritize those sizes. This directs inventory investment to where demand is highest, significantly reducing the chance of popular items selling out.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates the optimal mix of sizes for products at both individual store and broader store group levels.
- **Main Input:** Pre-calculated size contribution data (pivotal data), cleaned sales data, product attributes, and store groupings.
- **Main Output:** Ideal Size Set (ISS) percentages for specific sizes at store-level and aggregated store-group levels.
- **Warning Signs:** Consistent stockouts of specific popular sizes, persistent overstock of unpopular sizes, or reorder recommendations that don’t seem to match observed customer demand.
- **Key Settings:** Top Revenue Contribution Percentage (influences input), Essential Size Availability Threshold (influences input), Reorder Frequency (influences update rate).

Need Help? Contact your Merchandising Operations team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 7: Rate of Sale (ROS) Calculation

Last Updated: 2025-05-23

1. Business Objective

This step accurately computes the Rate of Sale (ROS) for product styles and individual items (SKUs) across various periods, at both individual store and broader store group levels.

Why This Matters: By providing a reliable measure of product velocity, this calculation is a fundamental input for determining future reorder quantities, ensuring our inventory efficiently matches customer demand, minimizing stockouts, and reducing holding costs.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (Units Sold):** We need transactional sales data that has been filtered to exclude misleading entries, ensuring the ROS accurately reflects demand when products were adequately stocked.
- **Historical Sales Trends:** This allows us to establish a baseline for product velocity and observe patterns in past sales, which is crucial for projecting future ROS.

Product Data

- **Product Attributes (e.g., style, SKU, category, size):** These details are vital for grouping sales data correctly, allowing us to calculate ROS at the appropriate level (e.g., for a specific style or an individual SKU within that style).
- **Ideal Size Set (ISS) Percentages:** This provides the recommended quantity breakdown for each size within a style or category, which is essential for accurately deriving the ROS for individual SKUs from the overall style ROS.

Operational Data

- **Store Groupings:** Information on how individual stores are combined into larger groups. This enables the calculation of ROS at both granular store-level and aggregated store group-level.
- **Store Coverage Threshold:** This rule defines the minimum percentage of stores where a style must have been actively available for its sales to be considered valid for a robust ROS calculation.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------|---|--|
| ROS Calculation Period | 90 days | The historical period (in days) used to calculate the Rate of Sale (ROS) for a product style or SKU. | If product demand patterns become highly volatile (shorter period) or very stable (longer period), to ensure relevancy. |
| Store Coverage Threshold | 20% | The minimum percentage of stores within a store group where a style must have been actively sold for its ROS to be considered valid and representative. | If your operational definition of “widespread availability” changes, to ensure styles have had sufficient market exposure for ROS calculation. |
| Minimum Style Sales for ROS Validity | 5 units | The minimum number of units a style must have sold within the ROS Calculation Period to be considered for a reliable ROS calculation. | To prevent highly volatile or statistically insignificant sales from skewing reorder recommendations for very slow-moving styles. |
| ROS Update Frequency | Weekly | How often the Rate of Sale (ROS) is recalculated and updated for all relevant products. | If your supply chain or sales cycles change significantly, requiring more or less frequent inventory adjustments. |

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales Data & Product Attributes] --> B[Rate of Sale (ROS) Calculation]

B --> C[Store & Store Group ROS for Styles & SKUs]

1. Start with → Cleaned sales data, detailed product attributes, Ideal Size Set (ISS) percentages, and relevant business settings.
2. Then → The system first calculates the Rate of Sale (ROS) for each product style at both individual store and store group levels, ensuring enough sales data and store coverage.
3. Next → It then uses the calculated style ROS and the Ideal Size Set (ISS) percentages to derive the more granular ROS for each individual SKU (e.g., a specific size of a style).
4. Finally → It outputs the precise ROS for all relevant styles and SKUs, aggregated by both store and store group, ready for reorder quantity calculations.

5. Step-by-Step Actions

1. **Gather** cleaned sales data and product context.
 - What we do: *We collect validated sales transactions, product attributes, and the ideal size distribution percentages for all relevant styles and SKUs.*
 - Why we do it: *This ensures that the ROS calculation is based on accurate, representative sales and can be properly broken down by size.*
2. **Calculate** style-level ROS.
 - What we do: *For each product style, we determine its sales velocity (units sold per day/week) at individual store and consolidated store group levels, ensuring that styles meet minimum sales thresholds and have sufficient store coverage for a reliable calculation.*
 - Why we do it: *This provides a broad indicator of how quickly a style is selling across its relevant markets, forming the foundation for more granular SKU-level analysis.*
3. **Adjust** style-level ROS for different periods.
 - What we do: *We normalize the calculated style ROS to ensure it represents a consistent selling rate over a defined period, even if the underlying sales data covers different durations.*
 - Why we do it: *This provides a standardized Rate of Sale (ROS) that can be consistently applied in forward-looking reorder calculations, regardless of the historical period analyzed.*
4. **Calculate** SKU-level ROS.
 - What we do: *Leveraging the style-level ROS and the Ideal Size Set (ISS) percentages, we derive the specific sales velocity for each individual SKU within a style (e.g., a “Medium” size of a specific shirt style).*
 - Why we do it: *This provides the most granular sales speed, essential for precise reorder recommendations that ensure we stock the right quantity of each specific item and size.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A “Classic Denim Jeans” style (Style ID: DJ-001) has been active in the “West Coast” Store Group (SG: 20).
- In the last 90 days (ROS Calculation Period), DJ-001 sold 1,800 units across all stores in SG: 20.
- DJ-001 was actively sold in 80% of stores in SG: 20 (which is above the 20% Store Coverage Threshold).
- The Ideal Size Set (ISS) for DJ-001 in SG: 20 is: Size 30: 10%, Size 32: 25%, Size 34: 30%, Size 36: 25%, Size 38: 10%.

What Happens

1. First, the system calculates the Rate of Sale (ROS) for the “Classic Denim Jeans” style (DJ-001) at the “West Coast” Store Group level.
 - Total units sold in 90 days: 1,800 units.
 - Style ROS (per day): $1,800 \text{ units} / 90 \text{ days} = 20 \text{ units/day}$.
 - The system confirms this style meets the Minimum Style Sales for ROS Validity and Store Coverage Threshold.
2. Then, the system uses this style-level ROS and the Ideal Size Set (ISS) percentages to calculate the ROS for each individual SKU (each size) of DJ-001 for the “West Coast” Store Group.
 - ROS for DJ-001 Size 30: $20 \text{ units/day} * 10\% \text{ (ISS for Size 30)} = 2 \text{ units/day}$.
 - ROS for DJ-001 Size 32: $20 \text{ units/day} * 25\% \text{ (ISS for Size 32)} = 5 \text{ units/day}$.
 - ROS for DJ-001 Size 34: $20 \text{ units/day} * 30\% \text{ (ISS for Size 34)} = 6 \text{ units/day}$.
 - And so on for other sizes.
3. Finally, similar calculations are performed for individual stores within the “West Coast” Store Group, accounting for their specific sales volumes and Ideal Size Sets.

Final Outcome

The system produces precise Rate of Sale (ROS) figures for each size of the “Classic Denim Jeans” style, at both the overall “West Coast” Store Group level and for each individual store. For example, it confirms that DJ-001 Size 34 sells at 6 units/day in the West Coast region, providing a clear demand signal for reordering.

Key Takeaway: This process transforms raw sales data into actionable demand signals (ROS) for every style and size, ensuring reorder recommendations are based on true product velocity and customer preferences.

7. Results

This step produces:

Main Outputs

- **Style-Level Rate of Sale (ROS) by Store and Store Group:** The calculated sales velocity for each product style, aggregated by individual store and broader store groups. This indicates overall demand for a style.
- **SKU-Level Rate of Sale (ROS) by Store and Store Group:** The more granular sales velocity for each individual item (SKU) within a style, broken down by size, and calculated for both individual stores and store groups. This is crucial for precise replenishment of specific sizes.

Business Reports

- **Product Velocity Report:** Used by merchandising planners to quickly assess how fast products are selling across different categories and regions, helping to identify high-demand items and potential slow movers.
- **SKU Performance Dashboard:** Used by inventory managers and store managers to monitor the sell-through rates of specific sizes and colors, informing local stock adjustments and ensuring the right product mix is always available.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|---|
| Missing or delayed data feed | The ROS calculation may use outdated or incomplete sales data, leading to inaccurate sales velocity figures and potentially suboptimal reorder recommendations. | Immediately contact the data team or IT to resolve the data feed issue and manually review the calculated ROS for critical items before proceeding. |
| New product launch with no sales history | The system cannot calculate a reliable ROS due to insufficient data, defaulting the ROS to zero or a very low value. | Merchandising needs to set an initial, estimated ROS based on market trends, competitor data, or similar product performance until sufficient sales history is accumulated. |
| Style or SKU consistently out of stock | If a style or SKU is frequently unavailable, its true ROS will be underestimated because sales were missed. The calculated ROS will reflect only what was | Investigate the root cause of persistent stockouts and consider adjusting manual reorders or using a “potential ROS” derived from sell-through rates when available, rather than actual units sold. |

| Situation | What Happens | What You Should Do |
|---------------------------------------|---|--|
| Sales data is too sparse or irregular | sold, not what could have been sold. | |
| | If a style or SKU has very few, infrequent sales within the ROS Calculation Period, the calculated ROS might be highly volatile and unreliable, making forecasting difficult. | Review the Minimum Style Sales for ROS Validity setting or consider grouping these items into broader categories for a more stable ROS, or rely on manual input. |

9. Next Steps

Before This Step

- **Reordering Input Preparation:** Provides the foundational raw sales data, product attributes, and business settings necessary for this module to begin its calculations.
- **Paramount Sizes & Sales Cleanup:** Delivers a refined dataset of sales transactions, ensuring the ROS is based on accurate demand signals from well-stocked assortments.
- **Ideal Size Set (ISS) Calculation:** Provides the specific Ideal Size Set (ISS) percentages for each size within a style, which is crucial for precisely deriving SKU-level Rate of Sale (ROS).

After This Step

- **Reorder Style Identification:** Utilizes the calculated Rate of Sale (ROS) for styles to categorize them by reorder priority ('REORDER1', 'REORDER2', 'DONT_REORDER').
- **Reorder Quantity Calculation:** Employs the determined SKU-level and style-level ROS figures as a primary input to compute the precise quantities of items needed for replenishment.

10. Common Questions

Q: How does knowing the Rate of Sale (ROS) help us avoid stockouts? A: ROS tells you exactly how fast a product is selling. By knowing this speed, you can accurately predict how much stock you'll need to cover demand until the next delivery, ensuring you reorder enough to prevent popular items from selling out before new inventory arrives.

Q: Why is ROS calculated for both entire styles and individual SKUs? A: Style-level ROS gives an overall picture of a product's popularity (e.g., "this shirt style sells 20 units a day"). SKU-level ROS is more precise (e.g., "Size Medium of this shirt sells 6 units a day"), which is critical for making sure we stock the right quantities of each specific size and color, optimizing inventory to the finest detail.

Q: How often is the Rate of Sale (ROS) updated, and can we change that? A: The Rate of Sale (ROS) is typically updated weekly, aligning with the ROS Update Frequency setting. Yes, this setting can be adjusted based on your business needs, for example, to daily for fast-moving items or bi-weekly for slower-moving categories, ensuring the most relevant velocity is used.

Q: What happens if a product's sales suddenly spike due to a viral trend or promotion? A: The ROS calculation is based on historical data. For very sudden spikes, the system will eventually capture the increased sales, but there might be a slight delay. For planned promotions, merchandising should set specific rules or initial orders outside the standard reorder cycle to ensure enough stock and not rely solely on the historical ROS.

Q: Why are some sales not included in the ROS calculation, like those from stores with low stock? A: We filter sales data to ensure the calculated ROS reflects true customer demand for a well-stocked product. If a popular item sold out quickly, its recorded sales might look low. By cleaning this data, we prevent skewed demand signals, ensuring reorder recommendations are based on what customers *would* buy if everything was available.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Computes the precise sales velocity (ROS) for products at style and SKU levels, for both individual stores and store groups.
- **Main Input:** Cleaned sales data, product attributes, Ideal Size Set (ISS) percentages, and core business settings.
- **Main Output:** Reliable Rate of Sale (ROS) figures for all styles and SKUs.
- **Warning Signs:** Unexplained overstocking or frequent stockouts for items with calculated ROS, or ROS figures that don't match observed sales trends.
- **Key Settings:** ROS Calculation Period, Store Coverage Threshold, Minimum Style Sales for ROS Validity.

Need Help? Contact your Merchandising Operations team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 9: Reorder Point Calculation (Subsequent Cycles)

Last Updated: 2025-05-23

1. Business Objective

This step calculates when and how much to reorder for all future cycles, ensuring that stores continuously maintain optimal stock levels (Optimum Depth, OD, and Optimum Width, OW) and proactively avoid stockouts.

Why This Matters: This ensures uninterrupted product availability, preventing lost sales from empty shelves and minimizing excess inventory by precisely aligning stock with predicted future demand across all our stores and warehouses. It's about continuous, intelligent replenishment, not just one-time ordering.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Rate of Sale (ROS) for Styles and SKUs:** We need to know how quickly products and their specific sizes have been selling over time. This is crucial for projecting future demand and calculating how much stock will be sold down between replenishment cycles.
- **Predicted Sales & Depletion Outputs:** These are forecasts that predict future sales and how current stock levels will deplete over time. This helps us anticipate inventory shortages before they occur.

Product Data

- **Current Stock Levels (Store and Warehouse):** Knowing exactly how many units of each product are currently in our stores and warehouses provides the baseline for projecting future inventory positions.
- **Ideal Size Set (ISS) Percentages:** This provides the recommended percentage breakdown for each size within a style or category. This is essential for ensuring that future reorders maintain the right mix of sizes.
- **Optimum Depth (OD) by Style and Store Group:** This defines the ideal total quantity of units for a specific style that a store or store group should aim to stock. It ensures we maintain enough popular styles.

Operational Data

- **Supplier Lead Times:** The number of days it takes for new stock to arrive from a supplier once an order is placed. This is critical for determining when reorders need to be placed to ensure timely replenishment.
- **Reorder Frequency:** How often the system evaluates and generates reorder recommendations (e.g., weekly, bi-weekly). This defines the length of each “cycle” for which future reorder points are calculated.
- **Safety Stock Buffer:** An additional percentage of stock to hold beyond the minimum. This acts as a cushion against unexpected demand spikes or supply delays and is crucial for calculating future minimum stock levels.
- **Target Stores for Styles:** The ideal number of stores within a store group where a specific style should be available. This helps ensure calculated reorder quantities are realistic for widespread distribution.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-----------------------------------|---------------|---|---|
| Reorder Cycle Duration | Weekly | The length of each future period for which reorder points and quantities are calculated. | If your supply chain or sales cycles change significantly, requiring more or less frequent inventory adjustments. |
| Reorder Point Projection Duration | 120 days | The total number of days into the future for which the system will calculate reorder points and suggested quantities, simulating multiple replenishment cycles. | To extend or shorten the planning horizon for inventory, adapting to longer lead times or faster-changing trends. |
| Safety Stock Buffer | 10% | An additional percentage of stock to hold beyond the minimum, acting as a cushion against unexpected demand spikes or supply delays. | When product demand becomes highly unpredictable or supplier reliability decreases, to reduce future stockout risk. |
| Target Cover Days | 30 days | The number of days of supply | When market demand changes or |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|---|
| | | we aim to have for a product or category in a store group. This influences the overall “Optimum Depth” for a style. | supply chain reliability shifts, to adjust desired future inventory levels. |

Note: See More Details for additional settings such as minimum sales thresholds for styles and various multipliers.

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

    A[Initial Reorder Recommendations & Current Inventory] -->
    B[Reorder Point Calculation (Subsequent Cycles)]
    B --> C[Future Reorder Points & Quantities]
  
```

1. Start with → The initial reorder recommendations from the first cycle (e.g., initial purchase orders), current inventory levels in stores and warehouses, and historical sales velocity (ROS).
2. Then → The system simulates how inventory will deplete and replenish over multiple future “reorder cycles,” incorporating projected sales, lead times, and any new incoming stock.
3. Next → For each future cycle, it calculates the projected stock levels and determines if a reorder is needed, factoring in desired Optimum Depth (OD), Ideal Size Set (ISS), and Safety Stock.
4. Finally → It generates a comprehensive list of suggested reorder quantities and the precise timing (reorder points) for specific products (SKUs) for each subsequent cycle.

5. Step-by-Step Actions

1. **Project** future inventory levels.
 - What we do: *We start with current stock and project how much will be sold over the next reorder cycle using historical Rate of Sale (ROS), simulating inventory depletion in both stores and warehouses.*
 - Why we do it: *This gives us a clear picture of future stock positions, identifying potential shortages before they happen.*
2. **Account** for incoming stock.
 - What we do: *We add any reorder quantities from the previous* cycle’s recommendations and any other expected incoming inventory (e.g., from existing purchase orders) to our projected future stock, considering supplier lead times.**
 - Why we do it: *This ensures we don’t over-order by counting stock that is already on its way or has been previously recommended.*

3. **Determine** target stock levels.

- What we do: *For each future cycle, we calculate the ideal amount of stock (Optimum Depth, OD) and the necessary Safety Stock needed for each product and size (Ideal Size Set, ISS) based on predicted future demand and business settings like 'Target Cover Days'.*
- Why we do it: *This establishes the desired future inventory positions that we aim to maintain for optimal sales and customer satisfaction.*

4. **Calculate** reorder quantities and points.

- What we do: *We compare the projected future inventory with the target stock levels. If projected stock falls below the target (after accounting for safety stock and incoming orders), we calculate the precise quantity to reorder to bring stock back to optimal levels, and identify the 'reorder point' (when to place the order).*
- Why we do it: *This ensures we place orders at the right time and for the right amount, preventing stockouts while minimizing holding costs for future periods.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular "Winter Coat" style (SKU: WCOAT-RED-M) was just recommended for an initial reorder of 100 units in the *first* cycle.
- Current stock at the warehouse is 50 units. Store stock (total across all relevant stores for this SKU) is 20 units.
- The Rate of Sale (ROS) for WCOAT-RED-M is 5 units/day.
- Supplier lead time for this coat is 20 days.
- Reorder Cycle Duration is 7 days (weekly).
- Reorder Point Projection Duration is 90 days.
- Target Cover Days for this product is 30 days.
- Safety Stock Buffer is 10% of Target Sales.

What Happens

1. Cycle 1 (Day 0-7, Initial Order Placed):

- The system starts by processing the initial reorder (100 units). It notes that these 100 units will arrive after 20 days (due to lead time).
- It projects sales: $5 \text{ units/day} \times 7 \text{ days} = 35 \text{ units}$ sold from existing store stock.
- It updates current stock: Warehouse (50 units) and Store ($20 - 35 = -15$ units, meaning projected stockout).
- *This negative store stock highlights the need for the initial reorder and warehouse replenishment.*
- The system calculates the target sales for the *next* cycle based on target cover days (e.g., $30 \text{ days} \times 5 \text{ units/day} = 150 \text{ units}$).
- It determines the reorder quantity for the *current* cycle to reach that target (e.g., $150 \text{ target} - 50 \text{ warehouse} - 20 \text{ store} + 35 \text{ already sold} = 115 \text{ units}$, simplified for this example). The previous initial 100 units were placed.

2. Cycle 2 (Day 7-14, Projecting Next Reorder):

- The system projects current warehouse stock will decrease as stores pull from it to cover sales and the initial reorder comes in.
 - It re-evaluates `closingStock` and `closingWhStock` at the *end* of this projected cycle.
 - It projects sales for the *next* 7-day period (5 units/day * 7 days = 35 units).
 - It now looks at the *expected incoming stock* from the *initial 100-unit reorder*, noting it will arrive around Day 20.
 - It recalculates the *target sales* and *safety stock* needed at the end of this cycle (based on 30-day cover and 10% buffer).
 - If the projected available stock (current stock + incoming orders) is less than the calculated target plus safety stock, it suggests a *new* reorder quantity for WCOAT-RED-M, to be placed for the *next* cycle's needs.
3. **This process repeats** for each subsequent 7-day cycle, looking forward through the 90-day projection duration. Each new cycle's reorder recommendation factors in:
- Actual (or projected) stock at the *start* of the new cycle.
 - All sales projected *during* the new cycle.
 - All stock *already ordered and expected to arrive*.
 - The latest Rate of Sale (ROS) and Ideal Size Set (ISS).
 - Required Optimum Depth (OD) and Safety Stock for the *next* period.

Final Outcome

The system produces a rolling, proactive replenishment plan for WCOAT-RED-M. For example, it might recommend “Reorder 75 units of WCOAT-RED-M on Day 21” to ensure stock levels remain optimal, then “Reorder 60 units of WCOAT-RED-M on Day 28”, and so on, maintaining continuous optimal inventory and avoiding unexpected stockouts.

Key Takeaway: This process ensures a seamless and proactive replenishment strategy, continuously adjusting future reorder recommendations based on evolving sales, stock levels, and incoming orders, thereby guaranteeing optimal product availability.

7. Results

This step produces:

Main Outputs

- **Future Reorder Point Projections:** A detailed schedule indicating future dates when specific SKUs or styles are projected to hit their reorder points, providing a proactive alert for replenishment.
- **Suggested Reorder Quantities for Subsequent Cycles:** Precise quantities of each SKU and style recommended for reordering for future cycles, factoring in projected sales, incoming stock, and target inventory levels (Optimum Depth and Ideal Size Set).
- **Projected Future Inventory Positions:** A simulated view of expected stock levels in both stores and warehouses for all relevant SKUs across multiple upcoming reorder cycles.

Business Reports

- **Rolling Replenishment Plan:** Used by merchandising planners and procurement teams to plan future purchase orders, allocate budget, and anticipate warehouse needs over a multi-cycle horizon.
- **Future Stockout Risk Dashboard:** Used by senior management and supply chain teams to identify products or categories at high risk of future stockouts well in advance, allowing for strategic interventions like expedited shipping or alternative sourcing.
- **Inventory Projection Report:** Used by finance and inventory managers to forecast future inventory investment and holding costs, supporting financial planning and operational efficiency.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|---|
| Missing or delayed data feed | The reorder point calculations may use outdated or incomplete sales, stock, or lead time data, leading to inaccurate projections and suboptimal future reorder recommendations. | Immediately contact the data team or IT to resolve the data feed issue and manually review pending reorders for critical items projected over subsequent cycles. |
| Significant, unexpected spike in future demand | If actual sales drastically exceed the historical Rate of Sale (ROS) (e.g., due to a viral trend), the projected reorder quantities might be too low, leading to future stockouts. | Merchandising planners should monitor real-time sales trends and manually adjust future reorder quantities to capture sudden demand, overriding system projections. |
| Prolonged supplier delays for future orders | If a supplier cannot fulfill a previously placed order or has extended lead times, the system's "inwards after lead time" projection will be inaccurate, potentially causing future stockouts. | Collaborate with suppliers to get updated timelines or seek alternative sourcing for critical items, and manually update the system's incoming inventory projections. |
| New product launch with no future sales history | The system cannot reliably project future reorder points or quantities due to a lack of historical ROS and Ideal Size Set (ISS) | Merchandising needs to set initial reorder quantities for new products for multiple cycles based on market trends or similar product performance, then monitor closely. |

| Situation | What Happens | What You Should Do |
|---|---|---|
| Rapidly changing Ideal Size Set (ISS) or Optimum Depth (OD) | <p>data for subsequent cycles.</p> <p>If customer preferences shift dramatically (e.g., a specific size becomes unexpectedly popular), the system might be slow to adapt the ISS/OD for future cycles, leading to imbalances.</p> | Regularly review and adjust the business settings that influence ISS and OD calculation, and consider manual overrides if shifts are very sudden or unexpected. |

9. Next Steps

Before This Step

- **Reordering Orchestration:** This coordinates the entire reordering process, ensuring all data and previous calculations (like ROS, ISS) are ready to feed into this module for the first cycle.
- **Reorder Quantity Calculation (First Cycle):** Provides the initial reorder quantities and resulting inventory positions that serve as the baseline for projecting future cycles.
- **Rate of Sale (ROS) Calculation:** Provides the accurate sales velocity figures needed to project future demand and inventory depletion.
- **Ideal Size Set (ISS) Calculation:** Provides the optimal size distribution percentages, which are crucial for breaking down future reorder quantities by size.

After This Step

- **Purchase Order Generation:** Takes the recommended future reorder quantities and timings from this module and converts them into actual purchase orders to be sent to suppliers.
- **Inbound Logistics Management:** Manages the tracking and receipt of incoming inventory from suppliers into warehouses and stores, fulfilling the orders triggered by these reorder points.
- **Inventory Performance Monitoring:** Uses the projections from this module to track how effectively the business is maintaining optimal stock levels and avoiding stockouts over time.

10. Common Questions

Q: What's the main difference between this and the initial reorder calculation? A: The initial reorder calculation focuses on what to buy *right now* based on current stock and immediate needs. This module takes that output and *simulates the future*, projecting stock levels and reorder needs for *subsequent* weeks or months, creating a rolling plan to keep shelves stocked continuously.

Q: How does this help us avoid unexpected stockouts far in advance? A: By projecting sales and inventory levels over multiple future cycles, the system can foresee when stock for certain items or sizes will fall below desired levels. This early warning allows merchandising to place reorders well in advance, accommodating supplier lead times and proactively preventing future empty shelves.

Q: Can the system adapt if customer demand suddenly changes dramatically, like for a seasonal item? A: Yes, it continuously updates its Rate of Sale (ROS) and Ideal Size Set (ISS) calculations based on the latest sales data. For predictable seasonal changes, the system learns and adjusts. For very sudden, unexpected shifts, merchandising planners may still need to make manual adjustments or activate promotional overrides to handle the immediate impact.

Q: What happens if a supplier experiences a long delay or can't deliver the projected stock? A: The system projects based on expected deliveries. If a supplier delays, the projected "inwards after lead time" would become inaccurate. Merchandising or supply chain teams would need to manually update the expected arrival dates or quantities, prompting the system to recalculate and potentially recommend expedited orders or alternative sourcing in future cycles.

Q: How does this ensure we maintain the right product variety (Optimum Width) and quantity (Optimum Depth)? A: This module uses the Optimum Depth (OD) settings to define the total target units for a style and the Ideal Size Set (ISS) to break down those units by size. By continuously recalculating reorder needs against these targets, it ensures we always replenish the right variety and ideal quantity of each product to meet customer demand.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Proactively calculates future reorder points and quantities over multiple cycles to ensure continuous optimal stock.
- **Main Input:** Initial reorder quantities, current inventory (store & warehouse), Rate of Sale (ROS), Ideal Size Set (ISS), Optimum Depth (OD), and supplier lead times.
- **Main Output:** A rolling plan of future reorder points and suggested quantities for all relevant SKUs and styles.
- **Warning Signs:** Unexpected future stockouts or persistent overstocking in projections, or a lack of proactive reorder suggestions for popular items.
- **Key Settings:** Reorder Cycle Duration, Reorder Point Projection Duration, Safety Stock Buffer, Target Cover Days.

Need Help? Contact your Merchandising Operations team for questions about this process.

reordering FAQ

Version: 1.0 **Last Updated:** 2025-05-23

This FAQ document provides clear answers to common business questions about the reordering algorithm, explaining its purpose, how it works, and what results to expect.

General

Q: What is the main goal of the reordering algorithm? **A:** The reordering algorithm aims to optimize inventory levels by recommending precise reorder quantities for products (SKUs and styles). It helps ensure you have the right products in the right sizes at the right time, maximizing sales and minimizing stockouts and excess inventory by determining Optimum Depth (OD) and Optimum Width (OW).

Q: What do OD and OW mean in the context of this algorithm? **A:** OD (Optimum Depth) refers to the ideal number of unique items (SKUs) to stock within a category or display. OW (Optimum Width) refers to the ideal number of different categories or styles to offer. These concepts guide the system in recommending optimal inventory levels.

Q: How does this system help prevent stockouts and minimize excess inventory? **A:** By calculating Rate of Sale (ROS) for products, identifying the Ideal Size Set (ISS), and factoring in current inventory and lead times, the system provides data-driven reorder recommendations. This ensures that replenishment aligns with demand, preventing products from running out or accumulating excess stock.

Q: What is an Ideal Size Set (ISS)? **A:** The Ideal Size Set (ISS) is the best combination of sizes to stock for a product or style. The algorithm calculates the ISS contribution for each SKU's size, ensuring that reorders include the sizes most likely to maximize sales and minimize stockouts.

Q: What is Rate of Sale (ROS)? **A:** ROS (Rate of Sale) is the speed at which products are sold over a period, typically expressed as units per week. The system calculates ROS for styles and SKUs to forecast future demand and inform reorder quantities, acting as a forward-looking indicator of product velocity.

Configuration & Settings

Q: How can I customize reordering decisions? **A:** The system incorporates reorder override rules which can adjust how styles are categorized (e.g., 'REORDER1', 'REORDER2', 'DONT_REORDER'). These rules allow for business-specific adjustments based on criteria like discount percentage or quantity sold.

Q: Can I set different reorder rules for different product groups? **A:** Yes, the system identifies “paramount sizes” based on top revenue-contributing sizes within specific Attribute Groups (AGs). This implies that rules and calculations are sensitive to how products are grouped, allowing for tailored reordering strategies.

Q: How does the system account for shipping times (lead times)? **A:** Lead times are a key input for the reordering calculations. They are factored into the Core Reordering Calculation and Reorder Point Calculation modules to ensure that sufficient stock is ordered to cover demand during the replenishment period.

Q: What defines a “paramount size”? **A:** A “paramount size” is a size identified as having a significant contribution to revenue within its Attribute Group (AG). These are a form of Ideal Size Set (ISS) contribution, highlighting the most important sizes for sales.

Q: Are there any global settings that impact all reorders? **A:** While specific global settings are not detailed here, the Reordering Input Validation module verifies general settings like store group mappings and cover days, which are crucial inputs for the overall reordering process. For detailed configuration, refer to the system’s `args.properties` file or equivalent configuration documentation.

Data & Inputs

Q: What information does the system need to calculate reorders? **A:** The Reordering Input Preparation module loads all necessary raw and pre-processed data. This includes transactional sales data, master data (SKU, style, store), inventory levels, depletion outputs, lead times, and any reorder override rules.

Q: How does the system ensure the data it uses is accurate? **A:** The Reordering Input Validation module performs comprehensive checks on all input data. This includes verifying store group mappings, cover days, lead times, and style-season consistency, ensuring the robustness and accuracy of the reordering recommendations.

Q: What happens if some sales data is missing? **A:** The Paramount Sizes & Sales Cleanup module cleans sales data by filtering out sales not meeting minimum live store or paramount size criteria. While it aims to use relevant data, significant gaps could impact ROS calculations and ISS determination. The Reordering Input Validation module would flag critical missing data.

Q: What is an Attribute Group (AG) and why is it important? **A:** An Attribute Group (AG) is a set of product attributes (e.g., color, size, brand) grouped together for analysis. It’s important because ISS calculations and the identification of “paramount sizes” are based on contributions within these AGs, allowing for more targeted and accurate reordering.

Q: How does the system handle new products without sales history? **A:** For new products without sales history, ROS cannot be directly calculated from past sales. The system would typically rely on default settings, comparable product performance, or specific business rules (often configured as overrides) until sufficient sales data is accumulated.

Process & Logic

Q: How does the system decide which styles to reorder? **A:** The Reorder Style Identification module categorizes styles into 'REORDER1', 'REORDER2', or 'DONT_REORDER'. This decision is based on various business criteria such as quantity sold, average discount percentage, revenue contribution, ROS per day, and the number of stores where the style is 'live'. Override rules can also adjust these categories.

Q: What's the difference between the 'first reordering cycle' and 'subsequent cycles'? **A:** The Core Reordering Calculation (First Cycle) module performs the initial computations for target sales, safety stock, and reorder quantities. The Reorder Point Calculation (Subsequent Cycles) module extends this logic to simulate future inventory positions and replenishment needs, ensuring continuous optimal stock levels (OD and OW) for ongoing operations.

Q: How does the Ideal Size Set (ISS) influence reorder quantities? **A:** The Ideal Size Set (ISS) Calculation module determines the contribution percentage for each size. This directly impacts the breakdown of reorder quantities by size, ensuring that the replenishment order contains the optimal mix of sizes to match customer demand and maintain the ideal Optimum Depth (OD) and Optimum Width (OW).

Q: How is Rate of Sale (ROS) used in reordering calculations? **A:** ROS is a critical input for forecasting future demand. The ROS Calculation module computes product velocity, which is then used by the Core Reordering Calculation and Reorder Point Calculation modules to determine target sales, safety stock, and ultimately, the recommended reorder quantities.

Q: What is the role of 'Reordering Orchestration'? **A:** The Reordering Orchestration module acts as the central coordinator. It manages the entire reordering workflow, ensuring that all sub-modules (like input preparation, calculations, and validation) execute in the correct order, dependencies are met, and the overall reordering process runs smoothly.

Q: How does the system identify 'Optimum Depth (OD)' and 'Optimum Width (OW)'? **A:** Optimum Depth (OD) and Optimum Width (OW) are determined through the core reordering calculations, primarily in the Core Reordering Calculation (First Cycle) and Reorder Point Calculation (Subsequent Cycles) modules. These calculations leverage ROS, ISS, current inventory, and other business rules to establish the ideal stock levels for SKUs and categories, respectively, to maximize sales and minimize waste.

Outputs & Results

Q: What recommendations does the reordering system provide? **A:** The system calculates and outputs recommended target sales, safety stock levels, and precise reorder quantities for each SKU and style. These recommendations are designed to achieve Optimum Depth (OD) and Optimum Width (OW).

Q: How can I view the reordering suggestions? **A:** The Reordering Output Helpers module is responsible for preparing and denormalizing the final output data. It transforms the internal calculation results into user-friendly formats suitable for reporting, dashboards, and export, making them readily accessible for review.

Q: Can I understand *why* a specific reorder quantity was recommended? **A:** While the direct output is the quantity, the system's logic is built upon transparent inputs like ROS, ISS contributions, current inventory, and business rules. By understanding these underlying factors (e.g., high ROS, specific ISS percentage for a size), you can infer the reasons behind the recommendation. Detailed documentation for each module can provide further insight.

Q: What does “target sales” mean in the output? **A:** “Target sales” represents the anticipated sales volume that the system aims to support with its recommended stock levels. It's a key intermediate calculation used to derive the necessary reorder quantity, ensuring you have enough inventory to meet forecasted demand.

Q: What if I don't agree with a recommended quantity? **A:** The system incorporates override rules in modules like Reorder Style Identification. If you have business reasons to adjust a recommendation, these rules can be configured to influence future calculations, ensuring the system aligns with strategic decisions.

Error Handling & Edge Cases

Q: What happens if critical input data is incorrect or missing? **A:** The Reordering Input Validation module will flag or reject incorrect or missing critical input data. This ensures the integrity of the reordering calculations, preventing inaccurate recommendations. If data is invalid, the system will not proceed until the issues are resolved.

Q: How does the system handle unusual sales spikes or sudden drops? **A:** The ROS Calculation module computes ROS based on historical sales. While it aggregates data, extreme fluctuations might still influence the calculated ROS. For persistent anomalies, business override rules or manual adjustments based on market intelligence might be necessary to adjust expectations beyond historical trends.

Q: Are there situations where a product won't be recommended for reorder, even if stock is low? **A:** Yes, the Reorder Style Identification module can categorize styles as 'DONT_REORDER' based on criteria such as low ROS, high discount percentages, or specific override rules. This ensures that capital isn't tied up in slow-moving or undesirable inventory.

Q: How are conflicting reorder override rules handled? **A:** The specifics of how conflicting override rules are resolved depend on their implementation within the Reorder Style Identification module (e.g., precedence order, hierarchy). In general, the system will apply the most specific or highest-priority rule to determine the final style categorization. Refer to the rule configuration documentation for details.

Q: What if a store suddenly closes or opens? **A:** The Reordering Input Validation module verifies store group mappings. Changes in store status (opening/closing) require updates to the master data inputs. If not updated, the system might calculate reorders for non-existent stores or miss opportunities for new ones, leading to inaccurate recommendations.

Need More Help? Please contact the support team for further assistance.

reordering Documentation

- **Version:** 1.0
 - **Last Updated:** 2025-05-23
-

2. Executive Summary

The reordering algorithm is a critical inventory optimization system designed to significantly enhance a business's ability to manage product stock efficiently. By precisely calculating Ideal Size Sets (ISS), Rates of Sale (ROS), and optimal inventory levels (Optimum Depth, OD, and Optimum Width, OW), it ensures the right products are in the right place at the right time. This leads to maximized sales opportunities, minimized stockouts, reduced excess inventory, and improved overall profitability.

3. Architecture Overview

```
graph TD
    A[Reordering Orchestration] --> B(Reordering Input Preparation)
    A --> C(Paramount Sizes & Sales Cleanup)
    A --> D(Reorder Style Identification)
    A --> E(Ideal Size Set Calculation)
    A --> F(Rate of Sale Calculation)
    A --> G(Core Reordering Calculation First Cycle)
    A --> H(Reorder Point Calculation Subsequent Cycles)
    A --> I(Reordering Input Validation)

    B --> C: data_flow (raw sales, master data)
    B --> D: data_flow (master data, config)
    B --> E: data_flow (product sales, filters)
    B --> F: data_flow (base data, config)
    B --> G: data_flow (depletion, lead times)
    B --> H: data_flow (foundational data)

    C --> D: data_flow (cleaned sales)

    D --> J(Reordering Output Helpers): utility_use (format outputs)
```

```
E --> F: data_flow (ISS contribution)
E --> G: data_flow (ISS contribution)

F --> G: data_flow (ROS values)

G --> H: data_flow (first cycle outputs)
G --> J: utility_use (denormalize outputs)

H --> J: utility_use (format outputs)

A --- I: prerequisite_validation (ensures input integrity)
```

Module Descriptions:

- **Reordering Orchestration:** Coordinates the entire reordering process, managing module execution order and data synchronization.
 - **Reordering Input Preparation:** Loads and sets up all necessary raw and pre-processed data for reordering calculations.
 - **Paramount Sizes & Sales Cleanup:** Identifies critical sizes (ISS contribution) and cleans sales data to ensure calculation accuracy.
 - **Reorder Style Identification:** Categorizes styles for reordering based on business performance criteria and override rules.
 - **Ideal Size Set (ISS) Calculation:** Determines the best combination of sizes to maximize sales for each SKU and store/store group.
 - **Rate of Sale (ROS) Calculation:** Computes the speed at which products are selling to forecast future demand.
 - **Core Reordering Calculation (First Cycle):** Calculates initial target stock levels, safety stock, and reorder quantities using ROS and ISS.
 - **Reorder Point Calculation (Subsequent Cycles):** Extends reordering logic to determine future replenishment needs based on lead times and ongoing demand.
 - **Reordering Input Validation:** Ensures the integrity and accuracy of all data inputs before calculations begin.
 - **Reordering Output Helpers:** Provides utilities to format and prepare the final reordering results for reporting and export.
-

4. Prerequisites & Setup

- **Java Version:** JDK 11 or higher
 - **Build Tool:** Maven (recommended) or Gradle
 - **Dependencies:** All project dependencies are managed via `pom.xml` (Maven) or `build.gradle` (Gradle) and will be automatically downloaded during the build process.
 - **Running the Orchestrator:**
 1. Ensure all dependencies are resolved: `mvn clean install` (Maven) or `gradle build` (Gradle).
 2. Execute the main orchestration class (e.g., `com.example.reordering.ReorderingOrchestrator`) via your IDE or command line: `java -jar target/reordering-<version>.jar --args.properties=path/to/your/args.properties`
-

5. Folder & File Structure

```
reordering/
├─ module-group/           # Contains the orchestrator logic
(Reordering Orchestration)
├─ submodules/             # Houses the individual reordering
calculation modules (e.g., ROS, ISS)
├─ args.properties         # Global configuration parameters for
the entire reordering process
├─ common-data.json        # Defines the schema for shared data
models used across modules
├─ outputs/                # Directory where all generated
reordering reports and data files are stored
```

6. Global Configuration

The `args.properties` file manages key system-wide settings for the reordering process.

Setting Name | Default Value | Description | Valid Range/When to Change An effective reordering algorithm maximizes sales and minimizes stockouts by recommending optimal product depth (OD) and width (OW) based on calculated Rate of Sale (ROS) and an Ideal Size Set (ISS).

3. Architecture Overview

```
graph TD
    A[Reordering Orchestration] --> B(Reordering Input Preparation)
    A[Reordering Orchestration] --> C(Paramount Sizes & Sales Cleanup)
    A[Reordering Orchestration] --> D(Reorder Style Identification)
    A[Reordering Orchestration] --> E(Ideal Size Set Calculation)
    A[Reordering Orchestration] --> F(Rate of Sale Calculation)
    A[Reordering Orchestration] --> G(Core Reordering Calculation First Cycle)
    A[Reordering Orchestration] --> H(Reorder Point Calculation Subsequent Cycles)
    A[Reordering Orchestration] --- I(Reordering Input Validation)

    B --> C: data_flow (raw sales, master data)
    B --> D: data_flow (master data, config)
    B --> E: data_flow (product sales, filters)
    B --> F: data_flow (base data, config)
```

B --> G: data_flow (depletion, lead times)
 B --> H: data_flow (foundational data)

 C --> D: data_flow (cleaned sales)

 D --> J(Reordering Output Helpers): utility_use (format outputs)

 E --> F: data_flow (ISS contribution)
 E --> G: data_flow (ISS contribution)

 F --> G: data_flow (ROS values)

 G --> H: data_flow (first cycle outputs)
 G --> J: utility_use (denormalize outputs)

 H --> J: utility_use (format outputs)

 I --> A: prerequisite_validation (ensures input integrity)

Module Descriptions:

- **Reordering Orchestration:** Coordinates the execution of all sub-modules involved in the reordering process, ensuring dependencies are met and the workflow proceeds in the correct order.
 - **Reordering Input Preparation:** Loads and initializes all raw and pre-processed data required for reordering calculations.
 - **Paramount Sizes & Sales Cleanup:** Identifies “paramount sizes” (a form of Ideal Size Set, ISS, contribution) based on top revenue-contributing sizes within specific Attribute Groups (AGs).
 - **Reorder Style Identification:** Categorizes styles into ‘REORDER1’, ‘REORDER2’, or ‘DONT_REORDER’ based on various business criteria such as quantity sold, average discount percentage, revenue contribution, ROS per day, and the number of stores where the style is ‘live’.
 - **Ideal Size Set (ISS) Calculation:** Determines the Ideal Size Set (ISS) contribution for each SKU’s size, both at the individual store level and aggregated at the store group level.
 - **Rate of Sale (ROS) Calculation:** Computes the Rate of Sale (ROS) for styles and SKUs across different periods, at both individual store and store group levels.
 - **Core Reordering Calculation (First Cycle):** Calculates the target sales, safety stock, and initial reorder quantity for each SKU and style.
 - **Reorder Point Calculation (Subsequent Cycles):** Extends the core reordering logic to calculate reorder points and suggested reorder quantities for subsequent cycles.
 - **Reordering Input Validation:** Performs comprehensive validation checks on all input data used by the reordering modules.
 - **Reordering Output Helpers:** Provides utilities for preparing and denormalizing the final output data from the reordering modules.
-

4. Prerequisites & Setup

- **Java Version:** JDK 11 or higher
 - **Build Tool:** Apache Maven (recommended)
 - **Dependencies:** All necessary project dependencies are declared in the `pom.xml` file and are automatically downloaded during the build process.
 - **How to Run the Orchestrator (Reordering Orchestration):**
 1. **Build the project:** Navigate to the `reordering/` root directory in your terminal and run `mvn clean install`. This will compile the code and package it into an executable JAR file.
 2. **Execute the orchestrator:** Once built, you can run the reordering process using the command: `java -jar target/reordering-1.0.jar --args.properties=args.properties` *Ensure args.properties is configured correctly for your environment.*
-

5. Folder & File Structure

```
reordering/  
├─ module-group/           # Contains the central 'Reordering  
Orchestration' module that manages the overall workflow.  
├─ submodules/             # Holds the independent sub-modules,  
each responsible for a specific calculation or data processing  
step (e.g., ISS, ROS).  
├─ args.properties         # The primary configuration file,  
containing global parameters that control the behavior of the  
reordering algorithm.  
├─ common-data.json        # Defines the standardized data  
schema used for inputs and intermediate data exchange across all  
modules.  
└─ outputs/                # This directory is automatically  
created and populated with the final reordering recommendations  
and reports.
```

4. Prerequisites & Setup

- **Java Version:** JDK 11 or higher
- **Build Tool:** Apache Maven (recommended)
- **Dependencies:** All necessary project dependencies are declared in the `pom.xml` file and are automatically downloaded during the build process.
- **How to Run the Orchestrator (Reordering Orchestration):**
 1. **Build the project:** Navigate to the `reordering/` root directory in your terminal and run `mvn clean install`. This will compile the code and package it into an executable JAR file.
 2. **Execute the orchestrator:** Once built, you can run the reordering process using the command: `java -jar target/reordering-1.0.jar --`

`args.properties=args.properties` *Ensure args.properties is configured correctly for your environment.*

5. Folder & File Structure

```
reordering/
├─ module-group/           # Contains the central 'Reordering
Orchestration' module that manages the overall workflow.
├─ submodules/             # Houses the independent sub-modules,
each responsible for a specific calculation or data processing
step (e.g., ISS, ROS).
├─ args.properties         # The primary configuration file,
containing global parameters that control the behavior of the
reordering algorithm.
├─ common-data.json        # Defines the standardized data
schema used for inputs and intermediate data exchange across all
modules.
└─ outputs/                # This directory is automatically
created and populated with the final reordering recommendations
and reports.
```

6. Global Configuration

The `args.properties` file contains global settings that impact the entire reordering pipeline.

| Setting Name | Default Value | Description | Valid Range/
When to Change |
|--|---------------|--|--|
| <code>reorder.lookback.weeks</code> | 12 | Number of weeks of historical sales data to consider for ROS calculations. | 4–52 weeks.
<i>Increase for more historical stability, decrease for quicker response to trends.</i> |
| <code>reorder.min.ros.threshold</code> | 0.1 | Minimum ROS per week for a style/SKU to be considered for reordering. | 0.01–1.0.
<i>Adjust based on desired inventory velocity; higher values reduce reorders for slow-moving items.</i> |

| Setting Name | Default Value | Description | Valid Range/
When to Change |
|---------------------------------|-----------------|---|---|
| reorder.safety.stock.factor | 1.5 | Multiplier applied to average weekly sales to determine safety stock. | 1.0–3.0.
<i>Increase for higher service levels/risk aversion, decrease for lower inventory holding costs.</i> |
| reorder.default.lead.time.weeks | 4 | Default lead time in weeks for product delivery if not specified in master data. | 1–20. <i>Match your average supplier lead times. Crucial for accurate reorder point calculations.</i> |
| reorder.max.discount.pct | 0.30 | Maximum allowable average discount percentage for a style to be considered for ‘REORDER1’ classification. | 0.0–0.50.
<i>Lower values ensure reordering only for full-price or minimally discounted items, higher values allow more flexibility.</i> |
| reorder.min.live.stores | 3 | Minimum number of “live” stores (active sales) for a style to qualify for reordering. | 1–20. <i>Ensures a style has broad enough appeal before replenishment. Increase for more selective reordering.</i> |
| data.input.path | data/
inputs | File path to the directory containing all raw input data files (e.g., sales, inventory, master data). | Absolute or relative path to your data source. <i>Change if your input data location is different.</i> |
| data.output.path | outputs/ | File path where all generated reorder recommendations and reports will be saved. | Absolute or relative path to your desired output directory. <i>Ensure write permissions.</i> |
| iss.min.revenue.share.pct | 0.05 | Minimum revenue share percentage for a | 0.01–0.20.
<i>Impacts the granularity of</i> |

| Setting Name | Default Value | Description | Valid Range/
When to Change |
|----------------------------|---------------|---|--|
| | | size within an AG to be considered a “paramount size”. | <i>ISS. Higher values focus on fewer, high-performing sizes.</i> |
| iss.pivot.attribute.group | COLOR_SIZE | The Attribute Group (AG) used as the primary pivot for Ideal Size Set calculations. | COLOR_SIZE, STYLE_SIZE, CATEGORY_SIZE. <i>Should align with how product sales are naturally clustered for size analysis.</i> |
| validation.strict.mode | true | If true, any input validation failure will halt the entire reordering process. | true/false. <i>Set to false for development/testing to allow partial runs despite minor data issues; true is recommended for production.</i> |
| snapshot.data.enabled | true | If true, intermediate data states are snapshotted for debugging and audit. | true/false. <i>Disable in production for performance if not required, enable for detailed analysis of each module’s output.</i> |
| ros.moving.average.periods | 2 | Number of periods (weeks) to consider for calculating a moving average of ROS. | 1–8. <i>Controls the smoothness of ROS trends. Higher values provide more stability, lower values react faster to recent changes.</i> |
| reorder.buffer.percentage | 0.10 | An additional percentage added to the calculated reorder quantity as a buffer against | 0.0–0.20. <i>Increase for higher buffer and reduced risk of stockouts, but higher inventory holding costs.</i> |

| Setting Name | Default Value | Description | Valid Range/
When to Change |
|---------------------------|---------------|---|---|
| | | unforeseen demand. | |
| reorder.consider.on.order | true | If true, on-order quantities are factored into current inventory levels for reorder calculations. | true/false.
<i>Should be true to avoid over-ordering items already incoming; false may lead to excess inventory.</i> |

Module Descriptions:

- **Reordering Orchestration:** Coordinates the execution of all sub-modules involved in the reordering process, ensuring dependencies are met and the workflow proceeds in the correct order. It manages the overall lifecycle of the reordering calculation, including snapshotting and output synchronization.
 - **Required Information:** `args.properties` for overall configuration, status of previous module runs, and data caches.
 - **Business Settings:** None specific to orchestration; it uses global settings.
 - **Process Flow:**

```

graph TD
    Start --> Validate[Reordering Input Validation]
    Validate --> Prep[Reordering Input Preparation]
    Prep --> Clean[Paramount Sizes & Sales Cleanup]
    Clean --> Identify[Reorder Style Identification]
    Identify --> ISS[Ideal Size Set Calculation]
    ISS --> ROS[Rate of Sale Calculation]
    ROS --> Core1[Core Reordering Calculation (First Cycle)]
    Core1 --> Subsequent[Reorder Point Calculation (Subsequent Cycles)]
    Subsequent --> End
          
```
 - **Step-by-Step Actions:**
 1. **Initialize** system components, *setting up the environment for calculations.*
 2. **Trigger** Reordering Input Validation, *ensuring data quality before processing.*
 3. **Execute** Reordering Input Preparation, *loading all necessary raw data.*
 4. **Run** Paramount Sizes & Sales Cleanup, *refining sales data for accuracy.*
 5. **Start** Reorder Style Identification, *categorizing styles for reordering.*

6. **Initiate** Ideal Size Set Calculation, *determining optimal size distributions.*
7. **Process** Rate of Sale Calculation, *forecasting product velocity.*
8. **Compute** Core Reordering Calculation (First Cycle), *generating initial reorder recommendations.*
9. **Calculate** Reorder Point Calculation (Subsequent Cycles), *planning future replenishment needs.*
10. **Synchronize** and **Export** final outputs, *making results available for business users.*

- **Sample Scenario:** A weekly reordering run starts. The orchestrator first verifies all input files are present and valid, then loads sales and inventory data. It sequences the cleaning, style identification, ISS, ROS, and core reordering calculations, ensuring that each step completes successfully before the next one begins. If any step fails (e.g., due to invalid data), the orchestrator can halt the process and report the error. Key Takeaway: *The orchestrator guarantees a smooth, reliable, and ordered execution of the complex reordering logic.*
- **Results:** Successful execution status of the entire reordering pipeline; trigger of output file generation.
- **Unusual Cases:**
 - **Missing or delayed data feed:** The orchestration will fail gracefully at the Reordering Input Preparation or Reordering Input Validation stage, reporting the missing files.
 - **Module failure:** If a submodule throws an unhandled exception, the orchestrator will stop the entire pipeline and log the error, preventing incorrect outputs.
 - **Configuration errors:** Incorrect `args.properties` settings might lead to unexpected behavior or early termination, typically caught during initialization.
- **Next Steps:** Review generated output files (e.g., `reorder_suggestions.csv`) in the `outputs/` folder.
- **Common Questions:**
 1. “Why did the reordering run stop?” *Check logs for validation errors or specific module failures.*
 2. “How long does a full run take?” *Depends on data volume; optimizing individual modules improves overall speed.*
 3. “Can I re-run specific modules?” *Not directly through orchestration; typically requires a full re-run or specific tool usage.*
 4. “What if I need to reorder only a specific category?” *Currently, the orchestrator runs for all relevant data; filtering must occur at the input preparation stage.*
 5. “How does it handle multiple simultaneous runs?” *It’s designed for single-threaded execution; parallel runs require external coordination to prevent data conflicts.*
- **Quick Reference:**
 - **Purpose:** Workflow management

- **Input:** Raw data availability, global configs
- **Output:** Orchestration status, triggers for other modules
- **Reordering Input Preparation:** Loads and initializes all raw and pre-processed data required for reordering calculations. This includes transactional sales data, master data (SKU, style, store), depletion outputs, lead times, and reorder override rules. It sets up the central data stores for subsequent modules.

- **Required Information:**

- **Sales Data:** Historical transactional sales (units, revenue), current inventory levels, on-order quantities.
- **Product Data:** SKU master data (SKU ID, style ID, size, color, category), style master data, Attribute Group (AG) definitions.
- **Operational Data:** Store master data (store ID, store group), lead times (per SKU/supplier), reorder override rules (e.g., manual flags for certain styles).

- **Business Settings:** `data.input.path`

- **Process Flow:**

```
graph LR
    Start --> LoadSales[Load Sales Data]
    LoadSales --> LoadInventory[Load Inventory & On-Order]
    LoadInventory --> LoadMaster[Load Product/Store Master Data]
    LoadMaster --> LoadRules[Load Override Rules & Lead Times]
    LoadRules --> InitializeCache[Initialize Central Data Cache]
    InitializeCache --> End
```

- **Step-by-Step Actions:**

1. **Read** raw sales history from specified files, *collecting all past transaction data*.
2. **Load** current inventory and on-order quantities from warehouse and store feeds, *establishing the present stock situation*.
3. **Ingest** product (SKU, style) and store master data, *providing contextual information for all products and locations*.
4. **Import** specific reorder override rules and supplier lead times, *incorporating business-specific exceptions and logistical constraints*.
5. **Populate** an in-memory data cache, *making all data readily accessible for downstream calculations*.

- **Sample Scenario:** The system needs to reorder jeans. This module loads 2 years of sales data for all jean SKUs, current inventory of 500 units across various stores, the lead time for new jean orders (e.g., 4 weeks), and a rule indicating ‘Distressed Skinny Jeans’ should always be reordered, regardless of performance. Key Takeaway: *This module acts as the data foundation, ensuring all necessary raw data is clean and prepared for advanced analysis.*

- **Results:** Populated in-memory data structures (`stylesData`, `reorderingData`) with raw and structured inputs.

- **Unusual Cases:**

- **Missing or delayed data feed:** The module will fail if critical input files (e.g., sales, inventory) are not found, stopping the process.
- **Incorrect file format:** Malformed CSV/JSON files can cause parsing errors.
- **Referential integrity issues:** SKUs in sales data not found in master data will be flagged or excluded.

- **Next Steps:** Data is passed to Paramount Sizes & Sales Cleanup and Reordering Input Validation.

- **Common Questions:**

1. “Where does the data come from?” *From various internal systems (POS, ERP, WMS) typically provided as CSV or JSON files.*
2. “How often is data refreshed?” *Typically daily or weekly, depending on the reordering cycle.*
3. “What if sales data is incomplete?” *Missing data periods or products might lead to inaccurate ROS and reorder quantities.*
4. “Can I add new override rules?” *Yes, by updating the override rules input file.*
5. “Is historical data purged?” *The system processes a defined lookback period; older data is generally retained in source systems.*

- **Quick Reference:**

- **Purpose:** Data loading & initialization
- **Input:** Raw sales, inventory, master data, rules
- **Output:** Structured in-memory data cache
- **Paramount Sizes & Sales Cleanup:** Identifies “paramount sizes” (a form of Ideal Size Set, ISS, contribution) based on top revenue-contributing sizes within specific Attribute Groups (AGs). It also cleans sales data by filtering out sales not meeting minimum live store or paramount size criteria, ensuring that subsequent calculations use relevant data.

- **Required Information:**

- **Sales Data:** Cleaned sales data from Reordering Input Preparation.
- **Product Data:** SKU and style master data, Attribute Group (AG) definitions.
- **Operational Data:** Store “live” status (from Reordering Input Preparation).

- **Business Settings:** `iss.min.revenue.share.pct`,
`iss.pivot.attribute.group`, `reorder.min.live.stores`

- **Process Flow:**

```
graph LR
    Start --> FilterSales[Filter Sales by Live Stores]
    FilterSales --> GroupByAG[Group Sales by Attribute Group]
    GroupByAG --> IdentifyParamount[Identify Paramount
```

Sizes by Revenue Share]

```
IdentifyParamount --> CleanInvalidSales[Remove Sales  
not meeting Paramount Criteria]  
CleanInvalidSales --> End
```

◦ Step-by-Step Actions:

1. **Filter** sales data to include only sales from stores deemed “live” and relevant, *ensuring current and active sales are considered*.
2. **Group** sales by defined Attribute Groups (AGs), *creating logical clusters for size analysis (e.g., all blue dresses, all men’s sneakers)*.
3. **Calculate** the revenue contribution of each size within these AGs, *identifying the most significant sizes*.
4. **Designate** “paramount sizes” as those meeting a minimum revenue share threshold, *prioritizing sizes that drive the most sales*.
5. **Remove** any sales records that do not involve these identified paramount sizes or come from non-live stores, *ensuring that all subsequent calculations are based on high-quality, relevant sales data*.

- **Sample Scenario:** For the “Women’s Denim” category, the module groups all sales by COLOR_SIZE. It finds that sizes 28, 30, and 32 account for 70% of revenue. These are marked as paramount sizes. Any sales records for size 24 or 36 that fall below the `iss.min.revenue.share.pct` (e.g., 5%) or occur in stores not flagged as “live” are excluded from further processing. Key Takeaway: *By cleaning sales data and focusing on “paramount sizes”, this module ensures that reordering decisions are based on the most impactful sales patterns, improving the accuracy of Ideal Size Set (ISS) calculations.*

- **Results:** Cleaned sales dataset (stored internally) and a list of paramount sizes per AG, used by downstream modules.

◦ Unusual Cases:

- **No paramount sizes identified:** If all sizes contribute very little individually, no paramount sizes may be identified, leading to potentially less refined ISS.
- **Too many/few live stores:** Incorrect flagging of “live” stores can skew the filtered sales data.
- **Attribute Group mismatch:** If `iss.pivot.attribute.group` is poorly chosen, the size analysis may not reflect true demand patterns.

- **Next Steps:** The cleaned sales data is passed to Reorder Style Identification and Ideal Size Set (ISS) Calculation.

◦ Common Questions:

1. “What is a ‘paramount size’?” *It’s a size that contributes significantly to sales revenue within a specific group of products (Attribute Group).*
2. “Why filter out sales?” *To ensure reordering decisions are based on active, relevant, and high-performing product-size combinations, avoiding noise from slow-moving or obsolete items.*
3. “How is ‘live store’ determined?” *Based on operational criteria, usually active sales within a recent period, defined by the business.*

4. “Can I change the attribute group for paramount size identification?”
Yes, via the `iss.pivot.attribute.group` configuration.
5. “Does this impact DONT_REORDER styles?” *It primarily impacts data used for REORDER1/REORDER2 styles, cleaning their sales history for more accurate analysis.*

◦ **Quick Reference:**

- **Purpose:** Sales data refinement, ISS foundation
- **Input:** Filtered sales, AG definitions
- **Output:** Cleaned sales data, paramount size identification
- **Reorder Style Identification:** Categorizes styles into ‘REORDER1’, ‘REORDER2’, or ‘DONT_REORDER’ based on various business criteria such as quantity sold, average discount percentage, revenue contribution, ROS per day, and the number of stores where the style is ‘live’. This module applies override rules to adjust style themes.

◦ **Required Information:**

- **Sales Data:** Cleaned sales data from Paramount Sizes & Sales Cleanup.
- **Product Data:** Style master data (SKU, style, category).
- **Operational Data:** ROS from Rate of Sale Calculation (if run sequentially), `reorder.override.rules` from Reordering Input Preparation.

- **Business Settings:** `reorder.min.ros.threshold`, `reorder.max.discount.pct`, `reorder.min.live.stores`

◦ **Process Flow:**

```
graph LR
    Start --> AggregateStyleMetrics[Aggregate Sales, Discount, ROS by Style]
    AggregateStyleMetrics --> ApplyRules[Apply Business Rules & Overrides]
    ApplyRules --> CategorizeStyle[Categorize as REORDER1, REORDER2, DONT_REORDER]
    CategorizeStyle --> End
```

◦ **Step-by-Step Actions:**

1. **Aggregate** style-level metrics, *calculating total units sold, revenue, average discount, and Rate of Sale for each style.*
2. **Evaluate** each style against predefined business rules, *checking criteria like minimum ROS, maximum discount percentage, and minimum number of live stores.*
3. **Apply** any manual `reorder.override.rules`, *allowing specific styles to be force-categorized (e.g., a new product always reordered, or a clearance item never reordered).*
4. **Assign** the final reorder theme: ‘REORDER1’ (high-performing, regular replenishment), ‘REORDER2’ (mid-performing, conditional replenishment), or ‘DONT_REORDER’ (low-performing, clearance, or end-of-life), *segmenting styles for appropriate inventory strategies.*

- **Sample Scenario:** A 'Classic White Tee' style has an ROS of 2 units/week, 0% discount, and is sold in 10 live stores. It meets all criteria and is classified as 'REORDER1'. A 'Seasonal Floral Dress' has an ROS of 0.5 units/week, 40% discount, and is in 2 live stores. It fails the ROS and discount criteria (e.g., `reorder.max.discount.pct` of 30%) and is categorized as 'DONT_REORDER'. A 'Basic Black Jean' with an ROS of 1 unit/week and 10% discount is in 5 live stores, making it REORDER2. Key Takeaway: *This module intelligently segments products, ensuring that reordering efforts are focused on profitable, active styles and avoids tying up capital in stagnant inventory.*
- **Results:** Categorized styles with assigned reorder themes (REORDER1, REORDER2, DONT_REORDER).
- **Unusual Cases:**
 - **Ambiguous criteria:** Styles that fall on the border of criteria might be miscategorized; fine-tuning thresholds is important.
 - **Conflicting override rules:** If override rules contradict automated categorization, the override typically takes precedence, but this should be carefully managed.
 - **New styles with no sales:** Will likely be 'DONT_REORDER' unless specifically overridden.
- **Next Steps:** The categorized styles are used by Ideal Size Set (ISS) Calculation, Rate of Sale (ROS) Calculation, and Core Reordering Calculation (First Cycle).
- **Common Questions:**
 1. "What's the difference between REORDER1 and REORDER2?" *REORDER1 are core, high-volume items requiring consistent replenishment. REORDER2 are typically mid-volume or seasonal items that might have different reordering logic or frequency.*
 2. "How do I override a style's reorder theme?" *Through the `reorder.override.rules` input file, typically managed by a product team.*
 3. "Why is a new style classified as DONT_REORDER?" *Likely due to insufficient sales history to meet ROS or live store criteria. Consider a manual override for initial stocking.*
 4. "What if a style's performance changes rapidly?" *The module re-evaluates classifications each run, adapting to recent performance.*
 5. "Does this impact old/discontinued styles?" *Yes, they are typically classified as DONT_REORDER to prevent further replenishment.*
- **Quick Reference:**
 - **Purpose:** Style segmentation
 - **Input:** Cleaned sales, style master, ROS, override rules
 - **Output:** Reorder theme (REORDER1, REORDER2, DONT_REORDER) per style
- **Ideal Size Set (ISS) Calculation:** Determines the Ideal Size Set (ISS) contribution for each SKU's size, both at the individual store level and aggregated at the store group level. This calculation leverages pivotal data (likely representing SKU contribution to sales within an Attribute Group, AG) to

derive adjusted contribution percentages for each size, crucial for Optimum Depth (OD) and Optimum Width (OW) planning.

- **Required Information:**

- **Sales Data:** Cleaned sales data from Paramount Sizes & Sales Cleanup.
- **Product Data:** SKU master data (size, style, AG).
- **Operational Data:** Store group mappings.

- **Business Settings:** `iss.pivot.attribute.group`,
`iss.min.revenue.share.pct`

- **Process Flow:**

graph LR

```
Start --> GroupSalesStore[Group Sales by Store & AG]
GroupSalesStore --> CalcStoreISS[Calculate ISS % per
Size per Store]
GroupSalesStore --> GroupSalesStoreGroup[Group Sales by
Store Group & AG]
GroupSalesStoreGroup --> CalcStoreGroupISS[Calculate
ISS % per Size per Store Group]
CalcStoreISS --> End
CalcStoreGroupISS --> End
```

- **Step-by-Step Actions:**

1. **Aggregate** sales by store and Attribute Group (AG), *grouping similar products by location for granular analysis.*
2. **Calculate** the proportional sales contribution of each size within these store-AG groups, *determining which sizes sell best at individual stores for specific product types.*
3. **Aggregate** sales by store group and AG, *providing a consolidated view for regional or cluster-level size planning.*
4. **Determine** the proportional sales contribution of each size within these store group-AGs, *identifying broader size preferences across multiple stores.*
5. **Normalize** these contributions to generate the Ideal Size Set (ISS) percentages, *providing a clear blueprint for how many units of each size should be stocked to optimize sales and minimize waste.*

- **Sample Scenario:** For a 'Men's T-Shirt' (part of an AG) at Store A, sizes M, L, XL sell 30%, 45%, 25% respectively. At Store B, the same AG sells M, L, XL at 20%, 30%, 50%. The module determines these ISS percentages for each store. Then, it aggregates for a 'Urban Stores' group, finding M, L, XL contribute 25%, 38%, 37% overall, providing a balanced ISS for stores within that group. Key Takeaway: *ISS ensures inventory aligns with actual demand by size, minimizing stockouts on popular sizes and reducing overstock on unpopular ones, leading to better Optimum Depth (OD) planning.*

- **Results:** Ideal Size Set (ISS) contribution percentages per size, per store, and per store group, stored in `reorderingData` for subsequent modules.

- **Unusual Cases:**

- **Sparse sales data for a store/AG:** If a store has very few sales for a specific AG, the ISS calculation might be unreliable. Aggregated store group ISS can mitigate this.
- **New sizes/styles:** These might have no sales history, resulting in default or zero ISS, requiring manual intervention or waiting for sales data to accrue.
- **Unusual size distribution:** If one size dominates (e.g., 90% of sales), the ISS will heavily favor that size, which is accurate but might look unusual.

- **Next Steps:** The calculated ISS contributions are used by Rate of Sale (ROS) Calculation and Core Reordering Calculation (First Cycle).

- **Common Questions:**

1. “Why is ISS calculated at both store and store group levels?” *Store-level ISS provides hyper-local precision, while store group ISS offers a more robust, aggregated view for strategic planning, especially for new or low-volume stores.*
2. “What does ISS mean for my stock?” *It tells you the ideal proportion of each size you should aim to have in stock for a given product type, based on historical sales.*
3. “How does this help prevent stockouts?” *By ensuring you stock more of the sizes that sell most frequently, reducing the chance of running out of popular items.*
4. “Can ISS change over time?” *Yes, as sales patterns evolve, the ISS will be recalculated in subsequent runs, adapting to changes in customer preferences.*
5. “What if a size is rarely sold but needs to be stocked?” *The ISS calculation primarily focuses on historical sales. For outlier sizes, minimum stock levels or manual overrides might be needed if pure sales data doesn’t justify their inclusion.*

- **Quick Reference:**

- **Purpose:** Optimal size distribution
- **Input:** Cleaned sales, product AGs
- **Output:** ISS % per size (store, store group)
- **Rate of Sale (ROS) Calculation:** Computes the Rate of Sale (ROS) for styles and SKUs across different periods, at both individual store and store group levels. This module aggregates sales data to provide a forward-looking indicator of product velocity, which is a key input for calculating target sales and reorder quantities.

- **Required Information:**

- **Sales Data:** Cleaned sales data from Paramount Sizes & Sales Cleanup.
- **Product Data:** SKU and style master data.
- **Operational Data:** Store group mappings.

- **Business Settings:** `reorder.lookback.weeks`, `ros.moving.average.periods`

- **Process Flow:**

graph LR

```

    Start --> AggregateSalesStoreSKU[Aggregate Sales by
    Store, SKU, Period]
    AggregateSalesStoreSKU --> CalcStoreSKURoS[Calculate
    RoS per Store & SKU]
    AggregateSalesStoreSKU -->
    AggregateSalesStoreGroupStyle[Aggregate Sales by Store
    Group, Style, Period]
    AggregateSalesStoreGroupStyle -->
    CalcStoreGroupStyleRoS[Calculate RoS per Store Group &
    Style]
    CalcStoreSKURoS --> ApplyMovingAverage[Apply Moving
    Average to RoS]
    CalcStoreGroupStyleRoS --> ApplyMovingAverage
    ApplyMovingAverage --> End
  
```

- **Step-by-Step Actions:**

1. **Aggregate** cleaned sales data by store, SKU, and relevant time periods (e.g., weekly), *providing a foundation for granular velocity measurement.*
2. **Calculate** raw ROS (units sold per period) for each store-SKU combination, *showing how fast each specific item sells at each location.*
3. **Aggregate** sales data by store group and style, *to calculate a broader, more stable ROS for product families across regions.*
4. **Compute** ROS for each store group-style combination, *offering an average velocity for a product type across a cluster of stores.*
5. **Apply** a moving average over `ros.moving.average.periods` to both SKU and style ROS values, *smoothing out daily or weekly fluctuations to get a more reliable and forward-looking indicator of sales velocity.*

- **Sample Scenario:** A ‘Red T-shirt’ (SKU: TS-RED-L) at Store C sold 5 units last week, 3 units the week before, and 4 units the week before that. Over `reorder.lookback.weeks`, its average weekly ROS is 4 units/week. For the ‘Basic T-Shirt’ style (which includes all colors/sizes), the average ROS for the ‘Suburban Stores’ group is 20 units/week. These ROS values are then used to predict future sales needs. Key Takeaway: *Accurate ROS is fundamental to demand forecasting, ensuring that reorder quantities are based on realistic sales velocity, which directly influences Optimum Depth (OD) and Optimum Width (OW).*

- **Results:** Calculated ROS (Rate of Sale) values for each SKU at the store level and for each style at the store group level, stored in `reorderingData`.

- **Unusual Cases:**

- **Zero sales:** SKUs/styles with no sales in the lookback period will have a ROS of zero, meaning no reorder.

- **Spiky sales:** Products with inconsistent sales (e.g., promotional items) can have volatile ROS; the moving average helps, but manual review might be needed.
- **Short lookback period:** A very short reorder.lookback.weeks might lead to ROS being overly sensitive to recent anomalies.
- **Next Steps:** ROS values are fed into Core Reordering Calculation (First Cycle) and Reorder Point Calculation (Subsequent Cycles).
- **Common Questions:**
 1. “What does ‘Rate of Sale’ mean?” *It’s how quickly a product sells over a given period, typically units per week or day.*
 2. “Why is ROS important for reordering?” *It’s the primary driver for forecasting how many units you are expected to sell, directly informing how much to reorder.*
 3. “How is ROS calculated for new products?” *New products might have a zero or very low ROS initially. Businesses often rely on comparable product ROS or a manual override until sufficient sales data is accumulated.*
 4. “Can promotions skew ROS?” *Yes, short-term promotions can inflate ROS. The lookback period and moving average help to mitigate this, but it’s a known challenge.*
 5. “Is ROS based on gross or net sales?” *Typically based on net units sold (after returns) to reflect true demand.*
- **Quick Reference:**
 - **Purpose:** Demand forecasting
 - **Input:** Cleaned sales data, lookback period
 - **Output:** ROS (units/period) per SKU/style
- **Core Reordering Calculation (First Cycle):** Calculates the target sales, safety stock, and initial reorder quantity for each SKU and style. This module performs the primary reordering computations for the first reordering cycle, leveraging ROS, ISS, and current inventory levels to determine optimal stock levels (Optimum Depth, OD and Optimum Width, OW) and replenishment needs.
- **Required Information:**
 - **Sales Data:** Current inventory, on-order quantities (from Reordering Input Preparation).
 - **Product Data:** SKU master data, style reorder theme (from Reorder Style Identification).
 - **Calculated Data:** ROS (from Rate of Sale Calculation), ISS (from Ideal Size Set (ISS) Calculation).
 - **Operational Data:** Lead times (from Reordering Input Preparation).
- **Business Settings:** reorder.safety.stock.factor, reorder.buffer.percentage, reorder.consider.on.order
- **Process Flow:**

graph LR

```
Start --> FetchInputs[Fetch ROS, ISS, Inventory]
FetchInputs --> CalcTargetSales[Calculate Target Sales
for Lead Time + Buffer]
CalcTargetSales --> CalcSafetyStock[Calculate Safety
Stock]
CalcSafetyStock --> DetermineNeeded[Determine Units
Needed (Target + Safety - Current - OnOrder)]
DetermineNeeded --> CalcReorderQty[Calculate Reorder
Quantity (applying ISS)]
CalcReorderQty --> End
```

◦ **Step-by-Step Actions:**

1. **Retrieve** ROS values for each SKU/style, ISS percentages for each size, current inventory, and on-order quantities, *gathering all necessary inputs for the calculations.*
2. **Calculate** target sales for the lead time period, *forecasting expected sales during the time it takes for new stock to arrive.*
3. **Compute** safety stock, *adding a buffer to cover unexpected demand surges or supply delays, based on reorder.safety.stock.factor.*
4. **Determine** the gross units needed by summing target sales and safety stock, then **subtract** current inventory and on-order quantities, *identifying the precise shortfall.*
5. **Apply** the ISS contribution percentages to the total units needed to determine the specific reorder quantity for each individual size (SKU), *ensuring the replenishment aligns with the ideal size distribution (Optimum Depth, OD).*
6. **Suggest** reorder quantities for each SKU at each relevant store or store group, *providing the actionable recommendations for replenishment (Optimum Width, OW).*

◦ **Sample Scenario:** A 'Blue Jeans' style (REORDER1) has a ROS of 10 units/week (store group level). Lead time is 4 weeks. Current inventory for size 32 is 5 units, with 20 on-order. Safety stock factor is 1.5.

- Target sales during lead time: $10 \text{ units/week} * 4 \text{ weeks} = 40 \text{ units}$.
- Safety Stock: $10 \text{ units/week} * 1.5 = 15 \text{ units}$.
- Total needed: $40 + 15 = 55 \text{ units}$.
- Net needed for size 32: $55 - (5 \text{ current} + 20 \text{ on-order}) = 30 \text{ units}$.
- If ISS for size 32 is 25%, then reorder $30 * 0.25 = 7.5$ (round to 8) units of size 32. Key Takeaway: *This module generates precise reorder recommendations for the immediate cycle, balancing projected demand with current stock and safety buffers to optimize Optimum Depth (OD) and Optimum Width (OW).*

◦ **Results:** Initial reorder quantity recommendations per SKU, target sales, safety stock levels, and current inventory projections.

◦ **Unusual Cases:**

- **Negative reorder quantity:** If current inventory + on-order exceeds target + safety stock, the quantity will be 0 or negative, indicating no reorder is needed or potential overstock.

- **Zero ROS:** SKUs/styles with zero ROS will not generate reorder quantities, aligning with DONT_REORDER logic.
 - **Short lead times:** Can result in very responsive but potentially volatile reorder recommendations.
- **Next Steps:** The first cycle outputs are used as a baseline for Reorder Point Calculation (Subsequent Cycles) and are prepared for final output by Reordering Output Helpers.
 - **Common Questions:**
 1. “What is the difference between target sales and reorder quantity?” *Target sales is the forecasted demand. Reorder quantity is the actual number of units to order to meet that demand, considering current stock, on-order items, and safety stock.*
 2. “How does safety stock protect us?” *It provides a buffer against unexpected increases in sales or delays in supply, reducing the risk of stockouts.*
 3. “Why is ‘on-order’ important?” *Including on-order items prevents over-ordering, as stock that is already on its way doesn’t need to be reordered again.*
 4. “Can I manually adjust reorder quantities?” *Yes, the generated output serves as a recommendation and can often be adjusted by planners before execution.*
 5. “What if I have seasonal items?” *ROS calculations consider historical sales, so seasonality is implicitly captured. However, for extreme seasonality, additional manual adjustments might be needed initially.*
 - **Quick Reference:**
 - **Purpose:** First cycle replenishment calculation
 - **Input:** ROS, ISS, inventory, lead times
 - **Output:** Reorder quantity per SKU, target sales, safety stock
 - **Reorder Point Calculation (Subsequent Cycles):** Extends the core reordering logic to calculate reorder points and suggested reorder quantities for subsequent cycles. This module simulates future inventory positions and replenishment needs based on the first cycle’s output, historical ROS, and lead times, ensuring continuous optimal stock levels (Optimum Depth, OD and Optimum Width, OW) and minimizing stockouts.
 - **Required Information:**
 - **Calculated Data:** firstCycleStoreOutputRows (from Core Reordering Calculation (First Cycle)), ROS (from Rate of Sale (ROS) Calculation), ISS (from Ideal Size Set (ISS) Calculation).
 - **Operational Data:** Lead times (from Reordering Input Preparation).
 - **Business Settings:** reorder.lookback.weeks, reorder.safety.stock.factor, reorder.default.lead.time.weeks
 - **Process Flow:**

graph LR

```
Start --> UseFirstCycleOutput[Use First Cycle Reorder Qty as Future On-Order]
UseFirstCycleOutput --> ProjectInventory[Project Future Inventory based on ROS & Lead Time]
ProjectInventory --> CalcReorderPoint[Calculate Reorder Point]
CalcReorderPoint --> DetermineSubsequentReorder[Determine Reorder Quantity for Future Cycles]
DetermineSubsequentReorder --> End
```

◦ **Step-by-Step Actions:**

1. **Consume** the output from the Core Reordering Calculation (First Cycle), *using the immediate reorder quantities as projected “on-order” for future periods.*
2. **Project** future inventory levels over multiple lead time cycles, *simulating stock depletion based on ROS and expected receipts from the first cycle’s orders.*
3. **Determine** the reorder point for each SKU: the inventory level at which a new order must be placed to avoid stockouts before the next replenishment arrives, *ensuring proactive ordering.*
4. **Calculate** suggested reorder quantities for these subsequent cycles, *maintaining Optimum Depth (OD) and Optimum Width (OW) over an extended horizon.*
5. **Output** detailed reorder point and quantity recommendations for a specified planning horizon (e.g., next 8 weeks), *providing a continuous replenishment plan.*

◦ **Sample Scenario:** Following the ‘Blue Jeans’ example, the module takes the 8 units of size 32 ordered in the first cycle. It then projects inventory for the next 8 weeks. If ROS is 2 units/week for size 32, current inventory of 5 units (assuming previous 20 units on order have arrived), and lead time is 4 weeks, the reorder point would be $(2 \text{ units/week} * 4 \text{ weeks lead time}) + (2 \text{ units/week} * 1.5 \text{ safety factor}) = 8 + 3 = 11$ units. If inventory drops below 11, another order is triggered. Key Takeaway: *This module transforms single-cycle reordering into a continuous replenishment strategy, ensuring long-term stock health and sustained sales by pro-actively managing reorder points.*

◦ **Results:** Reorder points and suggested reorder quantities for future replenishment cycles (e.g., next 4-8 weeks).

◦ **Unusual Cases:**

- **Changing lead times:** Unexpected changes in supplier lead times can disrupt calculated reorder points, requiring manual adjustment or re-running.
- **Significant ROS shifts:** If ROS dramatically changes between cycles, projected reorder points may become inaccurate.
- **Stockouts due to external factors:** Does not account for disruptions like natural disasters; reorder points are based on consistent historical patterns.

- **Next Steps:** Outputs are formatted by Reordering Output Helpers for final reporting and integration.

- **Common Questions:**

1. “What is a ‘reorder point’?” *It’s the minimum stock level at which a new order should be placed to avoid running out of product before the next delivery arrives.*
2. “How is this different from the ‘First Cycle’ calculation?” *The first cycle calculates the immediate order needed to fill the current gap. Subsequent cycles plan for future replenishment triggers based on ongoing depletion and lead times.*
3. “Does this account for promotions?” *It relies on the ROS calculated, which ideally smooths out promotional spikes. For planned future promotions, manual adjustments to the reorder point might be necessary.*
4. “How far into the future does it plan?” *Typically 4-8 weeks, depending on configuration and lead times, providing a rolling forecast.*
5. “What happens if we miss a reorder point?” *Risk of stockouts increases significantly as incoming stock will not arrive before current inventory depletes below demand.*

- **Quick Reference:**

- **Purpose:** Continuous replenishment planning
- **Input:** First cycle output, ROS, ISS, lead times
- **Output:** Reorder points, future reorder quantities
- **Reordering Input Validation:** Performs comprehensive validation checks on all input data used by the reordering modules. This includes verifying store group mappings, cover days, lead times, and style-season consistency, ensuring the robustness and accuracy of the reordering recommendations.

- **Required Information:**

- **Sales Data:** Raw sales, inventory, on-order.
- **Product Data:** SKU, style master data.
- **Operational Data:** Store master data, lead times, override rules, store group mappings.

- **Business Settings:** `validation.strict.mode`

- **Process Flow:**

```
graph TD
    Start --> ValidateStoreMappings[Verify Store Group Mappings]
    ValidateStoreMappings --> ValidateLeadTimes[Check Lead Times]
    ValidateLeadTimes --> ValidateStyleSeason[Ensure Style-Season Consistency]
    ValidateStyleSeason --> ValidateSalesData[Validate Sales & Inventory Data Integrity]
    ValidateSalesData --> ReportErrors[Report Warnings/Errors]
    ReportErrors --> End
```

- **Step-by-Step Actions:**

1. **Verify** consistency of store group mappings, *ensuring each store is correctly assigned to its group.*
2. **Validate** lead times, *checking for missing or unrealistic values (e.g., negative lead times).*
3. **Confirm** style-season consistency, *flagging products assigned to incorrect or outdated seasons, which could impact reorder themes.*
4. **Check** sales and inventory data for integrity, *looking for missing values, negative quantities, or duplicates that would skew calculations.*
5. **Generate** a comprehensive validation report, *detailed all detected errors and warnings, critical for data quality management.*
6. **Halt** the process if `validation.strict.mode` is true and critical errors are found, *preventing the use of unreliable data.*

- **Sample Scenario:** The validation module checks the `store_master.csv` file. It finds that Store ID 123 is missing a `store_group_id`. It also flags a lead time for SKU “SKU987” as “-2 weeks”, which is invalid. If `validation.strict.mode` is true, the system will stop, preventing calculations based on this flawed data. Key Takeaway: *This module is the quality gate, ensuring that the reordering system processes only accurate and consistent data, leading to reliable recommendations and preventing costly mistakes.*

- **Results:** Validation report (logs or a separate file), and a decision to proceed or halt the reordering process.

- **Unusual Cases:**

- **Data schema changes:** If input file schemas change without updating validation rules, valid data might be flagged as errors.
- **Mass data corruption:** Widespread errors will result in a large, potentially overwhelming, validation report.
- **False positives:** Very rarely, valid edge cases might be flagged as warnings, requiring manual review.

- **Next Steps:** If validation passes, data proceeds to Reordering Input Preparation. If it fails in strict mode, the process terminates.

- **Common Questions:**

1. “Why did validation fail?” *Check the validation report or logs for specific error messages detailing which data points or rules were violated.*
2. “What is ‘strict mode’?” *It means the system will stop immediately upon finding any critical data error, preventing potentially flawed reorder recommendations.*
3. “Can I bypass validation?” *Not recommended for production. Setting `validation.strict.mode` to false allows runs with warnings but risks data quality.*
4. “How often should data be validated?” *Every time the reordering process runs, typically daily or weekly, to catch new issues promptly.*
5. “Who is responsible for fixing validation errors?” *Typically data stewards or source system owners, in collaboration with the analytics team.*

- **Quick Reference:**

- **Purpose:** Data quality assurance
 - **Input:** All raw input files
 - **Output:** Validation report, process halt/continue signal
- **Reordering Output Helpers:** Provides utilities for preparing and denormalizing the final output data from the reordering modules. This includes transforming internal data structures into user-friendly formats suitable for reporting and export, ensuring that the results of the reordering calculations are readily accessible and understandable.

- **Required Information:**

- **Calculated Data:** Raw outputs from Reorder Style Identification, Core Reordering Calculation (First Cycle), and Reorder Point Calculation (Subsequent Cycles).
 - **Product Data:** SKU and style master data (for denormalization).

- **Business Settings:** `data.output.path`

- **Process Flow:**

```
graph LR
    Start --> CollectRawOutputs[Collect Raw Reorder Data]
    CollectRawOutputs --> Denormalize[Join with Master Data  
for Denormalization]
    Denormalize --> Format[Format for Reporting/Export]
    Format --> WriteToFile[Write to Output Files]
    WriteToFile --> End
```

- **Step-by-Step Actions:**

1. **Gather** raw calculated outputs from all relevant reordering modules (e.g., `reorder_quantity`, `safety_stock`, `reorder_point`), *collecting all the core numerical results.*
2. **Denormalize** the data by joining with master data (SKU details, style names, store locations), *adding descriptive information to make the numbers understandable.*
3. **Format** the data into a user-friendly structure, *organizing columns and rows for easy readability in spreadsheets or BI tools.*
4. **Write** the formatted data to various output files (e.g., CSV, JSON) in the `outputs/` directory, *making the reorder recommendations accessible for planners and inventory managers.*

- **Sample Scenario:** After all calculations, the system has a list of SKUs and their calculated reorder quantities. This module takes SKU '12345' with reorder quantity 10, then adds 'Style: Classic Tee', 'Size: M', 'Color: Black', 'Store Name: Flagship Store', etc., transforming it into a clear, actionable line item in a CSV report: `SKU: 12345, Style: Classic Tee, Size: M, Color: Black, Store: Flagship Store, ReorderQty: 10`. Key Takeaway: *This module translates complex numerical outputs into clear, actionable business recommendations, making the reordering algorithm's value immediately consumable by the relevant teams.*

- **Results:** Formatted, denormalized output files (e.g., `reorder_suggestions.csv`, `safety_stock_report.csv`) in the specified `outputs/` directory.
 - **Unusual Cases:**
 - **Missing master data:** If master data required for denormalization is incomplete, output files might have blank or N/A values for descriptive fields.
 - **Large output files:** For very large datasets, the output files can be extremely big, potentially requiring optimized export mechanisms or data partitioning.
 - **Formatting errors:** Incorrect column headers or data types in the output can cause issues for downstream systems.
 - **Next Steps:** Output files are typically consumed by business users, integrated into ERP systems for order generation, or used in Business Intelligence dashboards.
 - **Common Questions:**
 1. “Where can I find the reorder recommendations?” *In the outputs/ folder, usually in `reorder_suggestions.csv`.*
 2. “Can I customize the output format?” *Not directly via configuration; requires code changes in this module.*
 3. “What do these columns mean?” *The documentation should clearly define each output field; this module ensures human-readable names.*
 4. “Is this output ready for my ERP system?” *It’s designed for general consumption; specific ERP integration might require further transformation.*
 5. “How can I audit the results?” *The detailed outputs can be reviewed, and intermediate snapshots (if enabled) can be used to trace calculations.*
 - **Quick Reference:**
 - **Purpose:** Output formatting & export
 - **Input:** Raw calculated results, master data
 - **Output:** User-friendly reports, structured data files
-

9. Appendix

Full Parameter Reference

Please refer to Section 6, “Global Configuration,” for a comprehensive list of all configurable parameters, their default values, descriptions, and guidance on when to change them.

Version History

- **1.0 (2025-05-23):** Initial release of the reordering algorithm, encompassing all core modules for comprehensive inventory optimization.

Contact & Support Info

For any questions, issues, or support requests regarding the reordering algorithm, please contact:

- **Email:** support@yourcompany.com
- **Internal Chat Channel:** #inventory-optimization-support
- **Documentation Maintainers:** Data Science & Inventory Management Teams

Tutorial: reordering

The reordering system orchestrates a multi-stage process, starting with input preparation and validation, followed by core calculations for optimum depth (OD) and width (OW) and Ideal Size Set (ISS), and finally determining reorder quantities for current and subsequent cycles. Data flows sequentially through these modules, often leveraging shared data structures like `ReorderingData` and `ReorderStylesData`, with output helpers facilitating data denormalization for reporting.

flowchart TD

```
A0["Reordering Orchestration"]
A1["Reordering Input Preparation"]
A2["Paramount Sizes & Sales Cleanup"]
A3["Reorder Style Identification"]
A4["Ideal Size Set (ISS) Calculation"]
A5["Rate of Sale (ROS) Calculation"]
A6["Core Reordering Calculation (First Cycle)"]
A7["Reorder Point Calculation (Subsequent Cycles)"]
A8["Reordering Input Validation"]
A9["Reordering Output Helpers"]
A0 -- "control_flow" --> A1
A0 -- "control_flow" --> A2
A0 -- "control_flow" --> A3
A0 -- "control_flow" --> A4
A0 -- "control_flow" --> A5
A0 -- "control_flow" --> A6
A0 -- "control_flow" --> A7
A0 -- "prerequisite_validation" --> A8
A1 -- "data_flow" --> A2
A1 -- "data_flow" --> A3
A1 -- "data_flow" --> A4
A1 -- "data_flow" --> A5
A1 -- "data_flow" --> A6
A1 -- "data_flow" --> A7
A2 -- "data_flow" --> A3
A3 -- "utility_use" --> A9
A4 -- "data_flow" --> A5
A4 -- "data_flow" --> A6
A5 -- "data_flow" --> A6
A6 -- "data_flow" --> A7
A6 -- "utility_use" --> A9
A7 -- "utility_use" --> A9
```

Chapters

Master Report: Merchandising Algorithm - Reordering

1. Title & Metadata

- **Project Name:** reordering
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** The reordering algorithm optimizes inventory levels by recommending precise reorder quantities to maximize sales, minimize stockouts, and improve inventory efficiency across all SKUs and categories.

2. Executive Summary

The reordering algorithm is a critical component of our merchandising strategy, designed to ensure optimal product availability while minimizing carrying costs. By leveraging historical sales data, current inventory levels, and predefined business rules, the system calculates ideal reorder points and quantities. This leads to more efficient inventory management, reduced instances of overstocking and understocking, and ultimately, enhanced customer satisfaction and profitability.

- **Key outcomes or KPIs:**
 - Reduced Stockout Rate: Minimized instances of products being out of stock.
 - Improved Inventory Turnover: Faster movement of goods, reducing holding costs.
 - Optimized Sales Performance (ROS): Enhanced ability to meet customer demand, driving higher **ROS**.
 - Increased Gross Margin: Better inventory control leads to fewer markdowns and higher full-price sales.
 - Enhanced **OD** and **OW** adherence: Inventory aligns closer to ideal assortment strategies.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Planners, Supply Chain Management, Store Operations, E-commerce Teams, Finance Department.

3. Architecture & Data Flow

graph TD

```
A[Raw Sales Data] --> B{Data Ingestion & Validation};
C[Inventory Data] --> B;
D[Product Catalog / AG Data] --> B;
```

```

E[Business Rules / Configuration] --> B;

B --> F[ROS Calculation];
B --> G[OD & OW Analysis];
B --> H[ISS Determination];

F --> I[Inventory Optimization & Reorder Point Calculation];
G --> I;
H --> I;

I --> J[Reporting & Recommendation Generation];

J --> K[Reorder Recommendations (CSV)];
J --> L[Inventory Performance Reports (PDF/Dashboard)];
J --> M[Alerts & Notifications];

```

```

style A fill:#ECEFF1,stroke:#333,stroke-width:2px;
style B fill:#BBDEFB,stroke:#333,stroke-width:2px;
style C fill:#ECEFF1,stroke:#333,stroke-width:2px;
style D fill:#ECEFF1,stroke:#333,stroke-width:2px;
style E fill:#ECEFF1,stroke:#333,stroke-width:2px;
style F fill:#C8E6C9,stroke:#333,stroke-width:2px;
style G fill:#C8E6C9,stroke:#333,stroke-width:2px;
style H fill:#C8E6C9,stroke:#333,stroke-width:2px;
style I fill:#FFCDD2,stroke:#333,stroke-width:2px;
style J fill:#FFECB3,stroke:#333,stroke-width:2px;
style K fill:#F5F5F5,stroke:#333,stroke-width:2px;
style L fill:#F5F5F5,stroke:#333,stroke-width:2px;
style M fill:#F5F5F5,stroke:#333,stroke-width:2px;

```

- **Component Bullets:**

- **Data Ingestion & Validation:** Collects and cleans raw data from various sources.
- **ROS Calculation:** Computes the **Rate of Sale** for all SKUs.
- **OD & OW Analysis:** Determines ideal inventory levels for **Optimum Depth** (SKUs) and **Optimum Width** (categories/styles).
- **ISS Determination:** Calculates the **Ideal Size Set** for product distribution.
- **Inventory Optimization & Reorder Point Calculation:** The core logic that calculates reorder points and quantities based on demand forecasts, lead times, safety stock, and current inventory.
- **Reporting & Recommendation Generation:** Structures and outputs the reorder recommendations and performance reports.

- **High-level data sources:**

- **Sales History:** Transactional data (SKU, quantity, date, store, price).
- **Current Inventory:** Real-time stock levels per SKU, per location.
- **Product Master Data:** Product attributes, hierarchy, **AG** definitions, supplier information, lead times.
- **Store Master Data:** Store characteristics, capacity.
- **Promotional Calendars:** Upcoming sales events that may impact demand.

4. Configuration & Data Dictionary

Settings Table (args.properties)

| Parameter Name | Description | Default Value | Unit/Type |
|------------------------|--|-----------------|---------------|
| lookback_period_weeks | Number of past weeks to consider for sales velocity calculation. | 12 | Integer |
| safety_stock_weeks | Number of weeks of sales to hold as safety stock. | 2 | Decimal |
| forecast_horizon_weeks | Number of weeks into the future for which to forecast demand. | 4 | Integer |
| min_ros_threshold | Minimum ROS (units/week) for a product to be considered for reordering. | 0.5 | Decimal |
| service_level_target | Target probability of meeting demand without stockouts. | 0.95 | Decimal (0-1) |
| lead_time_buffer_days | Additional days to add to supplier lead time for buffer. | 3 | Integer |
| output_path | Directory path for generated reports and recommendations. | /data/output | String (Path) |
| historical_data_source | Path/Connection string to historical sales data. | db://sales_data | String |

Shared Data Fields (common-data.json)

| Field Name | Data Type | Description | Example Value |
|------------|-----------|--|----------------|
| sku_id | String | Unique identifier for a product. | PROD-ABC-123 |
| store_id | String | Unique identifier for a retail location. | STORE-001 |
| category | String | Product category (e.g., “Tops”, “Pants”). | Womens_Dresses |
| size | String | Product size (e.g., “S”, “M”, “L”, “30x32”). | L |
| color | String | Product color. | Black |

| Field Name | Data Type | Description | Example Value |
|-------------------------|-----------|---|---------------|
| date | Date | Date of a sales transaction or inventory snapshot. | 2024-05-20 |
| units_sold | Integer | Number of units sold in a given period/transaction. | 5 |
| current_stock | Integer | Current available inventory units at a location. | 15 |
| on_order_units | Integer | Units currently on order from suppliers. | 20 |
| supplier_lead_time_days | Integer | Standard lead time from supplier in days. | 7 |
| historical_ros | Decimal | Calculated ROS for the SKU-Store combination. | 3.5 |
| target_stock_level | Integer | Recommended ideal stock level for the SKU. | 30 |

Outputs Catalog

| Output File/Table | Description | Key Fields | Format |
|------------------------------|--|--|--------|
| reorder_recommendations.csv | Primary output: SKU-level reorder quantities per store. | sku_id, store_id, reorder_quantity, recommended_date | CSV |
| inventory_status_report.xlsx | Detailed report on current inventory, ROS , and reorder triggers. | sku_id, store_id, current_stock, historical_ros, reorder_point, status | Excel |
| od_ow_iss_summary.json | Summary of calculated OD , OW , and ISS targets. | category, od_target, ow_target, iss_distribution | JSON |
| stockout_alert_log.csv | Log of potential or | | CSV |

| Output File/Table | Description | Key Fields | Format |
|-------------------|---|---|--------|
| | actual
stockout
events
identified. | sku_id, store_id,
predicted_stockout_date,
severity | |

5. End-to-End Flow

The reordering algorithm operates through a series of sequential steps to produce actionable reorder recommendations:

1. **Data Acquisition & Preparation:** The process begins by ingesting raw sales history, current inventory counts, product master data (including **AGs** and lead times), and store attributes. This data undergoes validation, cleaning, and transformation into a standardized format.
2. **Sales Velocity Calculation (ROS):** For each unique SKU-store combination, the **Rate of Sale** is calculated based on the specified `lookback_period_weeks`. This provides a baseline understanding of product demand.
3. **Optimum Depth (OD) & Width (OW) Analysis:** The system analyzes historical sales patterns and business rules to determine the **Optimum Depth** (ideal number of unique SKUs) within a category and the **Optimum Width** (ideal number of categories/styles) for a store. These targets influence the overall inventory strategy.
4. **Ideal Size Set (ISS) Determination:** For apparel and other size-dependent products, the algorithm calculates the **Ideal Size Set** based on historical sales distribution, ensuring that reorders reflect the most popular sizes.
5. **Demand Forecasting:** Using the calculated **ROS** and potentially more advanced forecasting models, future demand for each SKU-store combination is projected for the `forecast_horizon_weeks`.
6. **Reorder Point & Quantity Calculation:**
 - A reorder point is calculated for each SKU, considering forecasted demand during lead time plus a `safety_stock_weeks` buffer to achieve the `service_level_target`.
 - The current stock level is compared against the reorder point. If current stock (plus on-order units) falls below the reorder point, a reorder is triggered.
 - The reorder quantity is then determined to bring the stock level up to a target, considering factors like minimum order quantities, case pack sizes, and store capacity, while aiming for the **OD** and **ISS** targets.
7. **Recommendation Generation:** The calculated reorder quantities, along with relevant metadata, are formatted into `reorder_recommendations.csv`. This file provides the actionable output for purchasing systems or manual review.
8. **Reporting & Alerts:** Auxiliary reports (e.g., `inventory_status_report.xlsx`) are generated, offering deeper insights into inventory performance. Automated alerts may be sent for critical stockout situations.

6. Submodule Overview

6.1. Data Ingestion & Validation

1. **Name & Purpose:** Ingests raw data from various sources (sales, inventory, product master) and validates its integrity and completeness.
2. **Inputs & Outputs:**
 - **Inputs:** Raw sales data (CSV/DB), current inventory data (CSV/DB), product master data (CSV/DB), store master data (CSV/DB).
 - **Outputs:** Cleaned and standardized DataFrame containing sku_id, store_id, date, units_sold, current_stock, on_order_units, supplier_lead_time_days, and product AGs (category, color, size).
3. **Core Logic Summary:**
 - Reads sales, inventory, and product data from specified sources.
 - Performs data type conversions (e.g., ensuring units_sold is integer, date is datetime).
 - Checks for missing or anomalous values (e.g., negative units_sold or current_stock).
 - Joins datasets on common keys like sku_id and store_id to create a unified view.

6.2. ROS Calculation

1. **Name & Purpose:** Calculates the **Rate of Sale** for each SKU-store combination based on recent historical sales.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned DataFrame with sku_id, store_id, date, units_sold.
 - **Outputs:** DataFrame augmented with historical_ros (units/week) for each SKU-store combination.
3. **Core Logic Summary:**
 - Filters sales data to include only transactions within the lookback_period_weeks defined in args.properties.
 - Aggregates units_sold for each sku_id-store_id pair over the lookback period.
 - Divides total units sold by the number of weeks in the lookback_period_weeks to compute the weekly **ROS**.
 - Handles SKUs with zero or very low sales by assigning a minimum min_ros_threshold if configured, to prevent division by zero or unrealistic reorders.

6.3. OD & OW Analysis

1. **Name & Purpose:** Determines the ideal number of unique items (**Optimum Depth**) and the ideal number of categories/styles (**Optimum Width**) to stock for optimal sales and space utilization.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned product master data (with category, style), historical_ros data.
 - **Outputs:** A dictionary or table defining od_target per category and ow_target per store, influencing reorder quantities.

3. Core Logic Summary:

- Analyzes historical sales concentration within categories to identify top-performing SKUs and categories, which informs **OD** and **OW** respectively.
- Considers store-level factors like sales volume and capacity to adjust **OW** targets.
- Applies business rules or pre-defined assortments to calculate target unique_sku_count (OD) and category_count (OW) for each relevant grouping.

6.4. ISS Determination

1. **Name & Purpose:** Calculates the **Ideal Size Set** distribution for products based on historical sales performance to ensure inventory matches customer demand for different sizes.
2. **Inputs & Outputs:**
 - **Inputs:** Cleaned sales data including sku_id, size, units_sold.
 - **Outputs:** A mapping of sku_id (or product_group_id) to iss_distribution (percentage breakdown by size).
3. **Core Logic Summary:**
 - Groups sales data by product (or product **AG**) and size.
 - Calculates the proportion of sales attributed to each size within a product group over the lookback_period_weeks.
 - Normalizes these proportions to create a percentage distribution for the **Ideal Size Set**.
 - Applies smoothing or minimum thresholds to prevent overly granular or zero-percentage size recommendations.

6.5. Inventory Optimization & Reorder Point Calculation

1. **Name & Purpose:** Computes the optimal reorder point and recommended quantity for each SKU-store combination.
2. **Inputs & Outputs:**
 - **Inputs:** historical_ros, current_stock, on_order_units, supplier_lead_time_days, od_target, ow_target, iss_distribution, and configuration parameters (safety_stock_weeks, service_level_target, forecast_horizon_weeks).
 - **Outputs:** DataFrame with sku_id, store_id, reorder_point, target_stock_level, reorder_quantity.
3. **Core Logic Summary:**
 - **Forecast Demand:** Projects demand for the forecast_horizon_weeks based on historical_ros and other factors (e.g., seasonality, promotions if integrated).
 - **Calculate Lead Time Demand:** Estimates demand during the supplier_lead_time_days using the forecasted **ROS**.
 - **Determine Safety Stock:** Calculates safety stock using the service_level_target and variability in demand/lead time.
 - **Compute Reorder Point:** Adds lead time demand to safety stock to get the reorder point.
 - **Calculate Reorder Quantity:** If current_stock + on_order_units is below the reorder point, a reorder is triggered. The quantity aims to bring stock up to target_stock_level (lead time demand + safety stock + additional buffer to meet **OD** and **ISS** targets).

6.6. Reporting & Recommendation Generation

1. **Name & Purpose:** Formats the calculated reorder quantities and related inventory data into user-friendly reports and actionable CSV files.
2. **Inputs & Outputs:**
 - **Inputs:** DataFrame with sku_id, store_id, reorder_quantity, reorder_point, current_stock, historical_ros.
 - **Outputs:** reorder_recommendations.csv, inventory_status_report.xlsx, od_ow_iss_summary.json.
3. **Core Logic Summary:**
 - Filters the final recommendations to include only those SKUs requiring a reorder (where reorder_quantity > 0).
 - Structures the data into the predefined output formats (e.g., CSV for recommendations, Excel for detailed reports).
 - Adds human-readable columns and metrics (e.g., “Days of Supply”, “Stockout Risk”).
 - Saves the output files to the specified output_path.

7. Worked Example

Let's trace the reordering decision for a single SKU in a specific store:

SKU: Tee_Basic_Black_M **Store:** STORE-001

1. Data Ingestion & Validation (Initial Inputs): * **Sales History (last 12 weeks):** 42 units sold * **Current Stock:** 5 units * **On Order:** 0 units * **Supplier Lead Time:** 7 days (1 week) * **Product Attributes:** Category = Womens_Tops, Size = M

2. ROS Calculation: * $\text{lookback_period_weeks} = 12$ * $\text{ROS} = \text{Total Units Sold} / \text{Lookback Period} = 42 \text{ units} / 12 \text{ weeks} = 3.5 \text{ units/week}$

3. OD & OW Analysis (Assumed Inputs): * Target **OD** for Womens_Tops is 50 unique SKUs. * Target **OW** for STORE-001 is 15 categories. * (These high-level targets would influence overall assortment strategy, and indirectly, the final reorder quantity to maintain a balanced assortment, but not directly used in the simple numerical example below).

4. ISS Determination (Assumed Input): * For Tee_Basic_Black, the ISS distribution is S:20%, M:40%, L:30%, XL:10%.

5. Inventory Optimization & Reorder Point Calculation: * **ROS:** 3.5 units/week * **Supplier Lead Time:** 1 week * $\text{safety_stock_weeks} = 2 \text{ weeks}$ * $\text{forecast_horizon_weeks} = 4 \text{ weeks}$

- **Forecasted Demand (during lead time):** $3.5 \text{ units/week} * 1 \text{ week} = 3.5 \text{ units}$
- **Safety Stock:** $3.5 \text{ units/week} * 2 \text{ weeks} = 7 \text{ units}$
- **Reorder Point:** $\text{Forecasted Demand} + \text{Safety Stock} = 3.5 + 7 = 10.5 \text{ units}$. (Rounding up to 11 units for simplicity).
- **Current Inventory:** 5 units (below Reorder Point of 11 units) -> **Reorder Triggered!**

- **Target Stock Level (for 4-week forecast):** $3.5 \text{ units/week} * 4 \text{ weeks (forecast horizon)} = 14 \text{ units}$.
- **Reorder Quantity Calculation:** $\text{Target Stock Level} - \text{Current Stock} - \text{On Order Units} = 14 - 5 - 0 = 9 \text{ units}$.
 - *Adjustment for ISS:* Since this is size M, and M is 40% of ISS, the general reorder for Tee_Basic_Black might be 22 units ($14 / 0.63$ for other sizes + M's share = ~22 units total, and M gets 40% of that). If the target stock level is specifically for the SKU-size, then 9 units is direct. Let's assume 9 units is the SKU-specific need.

6. Reporting & Recommendation Generation: * A recommendation will be generated: sku_id=Tee_Basic_Black_M, store_id=STORE-001, reorder_quantity=9, recommended_date=2025-05-23.

Conclusion: Based on its ROS, current stock, and lead time, Tee_Basic_Black_M at STORE-001 is below its reorder point and needs to order 9 units to cover demand for the next 4 weeks plus safety stock.

8. Debug Guide Highlights

- **Missing Data:** Check if core inputs like sales history, current inventory, or supplier lead times are entirely missing or have gaps.
 - *Check:* Verify data ingestion logs and source data files.
- **Zero ROS:** If ROS is unexpectedly zero for high-selling items, investigate the lookback_period_weeks and potential data filtering issues.
 - *Check:* Sample historical_ros output; ensure min_ros_threshold is configured as expected.
- **Negative Reorder Quantities:** This indicates an error in logic (e.g., current stock is higher than target, but still triggering).
 - *Check:* Review the reorder point and quantity calculation logic, particularly the if condition for triggering reorders.
- **Excessive Reorders:** If recommendations are very high for many SKUs, check safety_stock_weeks, forecast_horizon_weeks, or inflated ROS.
 - *Check:* Analyze args.properties values; sanity check historical_ros for top SKUs.
- **No Reorders for Key SKUs:** If high-selling items are not being recommended, ensure their current stock isn't artificially high or their supplier_lead_time_days isn't zero/incorrect.
 - *Check:* Review current_stock and on_order_units for specific SKUs; verify supplier_lead_time_days in product master.
- **Configuration Mismatch:** Ensure args.properties values are correctly loaded and applied across all modules.
 - *Check:* Log actual loaded parameters at the start of the run.

9. Common Pitfalls & Remedies

- **Pitfall 1: Ignoring Seasonality/Promotions:** Reordering based purely on historical **ROS** without accounting for future demand fluctuations (e.g., holidays, promotional events) can lead to significant stockouts or overstock.
 - **Remedy:** Implement a more sophisticated demand forecasting model that incorporates seasonality indices, promotional uplift factors, and trend analysis. Regularly update these factors.
- **Pitfall 2: Inaccurate Lead Times:** Incorrect `supplier_lead_time_days` can cause stockouts (underestimated lead time) or excessive inventory (overestimated).
 - **Remedy:** Continuously monitor and update lead times from suppliers. Implement a system to track actual lead time performance and automatically adjust.
- **Pitfall 3: “Lumpy” Demand:** Products with infrequent but large sales (e.g., high-value, low-volume items) can skew **ROS** and lead to poor reorder decisions.
 - **Remedy:** For such items, consider alternative forecasting methods (e.g., Croston’s method) or specific business rules (e.g., reorder to fixed target, manual review) instead of pure **ROS**.
- **Pitfall 4: Data Quality Issues:** Inconsistent `sku_ids`, missing sales records, or incorrect `current_stock` counts.
 - **Remedy:** Implement robust data validation checks at the ingestion stage. Establish clear data governance policies and regular data audits.
- **Pitfall 5: Inflexibility with OD/OW/ISS:** Rigidly applying calculated **OD**, **OW**, or **ISS** without allowing for merchandising strategy adjustments or market shifts.
 - **Remedy:** Ensure these targets are reviewable and adjustable by merchandising teams. The algorithm should provide suggestions, not enforce rigid rules that cannot be overridden by business judgment.

10. FAQ

- **Q1: How often does the reordering algorithm run?**
 - A1: By default, the algorithm is scheduled to run daily overnight. Critical updates (e.g., new promotions) can trigger an ad-hoc run.
- **Q2: Can I manually override the reorder recommendations?**
 - A2: Yes, the `reorder_recommendations.csv` is designed to be consumed by purchasing systems, which typically allow for manual adjustments before order placement. Merchandising teams can review and modify quantities.
- **Q3: What happens to new products with no sales history?**
 - A3: New products are handled based on predefined launch strategies. They typically receive an initial seed stock. The algorithm will start incorporating them into reorder recommendations once sufficient `historical_ros` data (usually 4-8 weeks) becomes available.
- **Q4: How does the algorithm handle seasonal products?**
 - A4: For truly seasonal products, the algorithm uses historical sales data from previous similar seasons to forecast demand, rather than just the immediate `lookback_period_weeks`. More advanced models incorporate seasonality curves.

• **Q5: What if a supplier's lead time changes frequently?**

- A5: The algorithm relies on the `supplier_lead_time_days` provided in the product master. We recommend automating the update of this field from supplier systems where possible, or performing regular manual updates for key suppliers.

11. Quick Reference Cheat-Sheet

| Term/Parameter | Definition / Purpose |
|-------------------------------------|--|
| OD | Optimum Depth: Ideal number of unique items (SKUs) to stock in a category. |
| OW | Optimum Width: Ideal number of different categories or styles to offer. |
| ISS | Ideal Size Set: The best combination of sizes to maximize sales and minimize stockouts. |
| ROS | Rate of Sale: Speed at which products are sold (e.g., units/week). |
| AG | Attribute Group: Set of product attributes grouped for analysis (e.g., color, size, brand). |
| <code>lookback_period_weeks</code> | Config: Number of past weeks for ROS calculation. |
| <code>safety_stock_weeks</code> | Config: Buffer stock to prevent stockouts during demand/lead time variability. |
| <code>forecast_horizon_weeks</code> | Config: Period for which demand is projected for reorder planning. |
| <code>reorder_point</code> | The inventory level at which a new order should be placed. |
| <code>reorder_quantity</code> | The recommended amount of product to order. |
| Key Output | <code>reorder_recommendations.csv</code> - The actionable reorder list. |
| Debug Tip | Check <code>historical_ros</code> and <code>current_stock</code> for unexpected values if reorders seem off. |
| Common Pitfall | Ignoring seasonality or promotions. |

Chapter 1: Space Planning Configuration

Last Updated: 2025-05-23

1. Business Objective

To define and load the foundational parameters that guide how the Space Planning Calculation Module determines ideal product assortments and quantities.

Why This Matters: This ensures consistency and precision in inventory recommendations, directly impacting stock efficiency, sales potential, and customer satisfaction by optimizing product availability and visual merchandising.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Critical for understanding product velocity and demand patterns, informing how many units of a product should be stocked in a given space.
- **Rate of Sale (ROS):** Helps predict future demand and determine appropriate replenishment cycles, ensuring popular items are always in stock and limiting stockouts.

Product Data

- **Attribute Group (AG) Definitions:** Specifies how products are categorized by attributes (e.g., color, material, size, brand) to ensure logical grouping for display and inventory analysis.
- **Product Dimensions:** Essential for calculating physical space requirements and ensuring products fit properly within display fixtures and shelves.

Operational Data

- **Current Inventory Levels:** Provides a baseline for assessing existing stock against ideal recommendations, highlighting overstock or potential stockout situations.
- **Display Capacity Information:** Defines the physical limits of store fixtures and shelves, ensuring space planning recommendations are realistic and implementable.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------------|---------------|--|---|
| Quantity Adjustment Increment | 1 unit | The smallest step by which suggested product quantities or display counts can be adjusted. For example, a value of 5 means quantities will always be a multiple of 5 (e.g., 10, 15, 20 units). | When store display fixtures only accommodate items in specific multiples (e.g., shoe boxes in stacks of 4) or to simplify ordering and stocking. |
| Optimum Depth (OD) Adjustment Factor | 1.0 | A multiplier used to fine-tune the calculated Ideal Number of Unique Items (OD) for a category or display. A factor of 1.1 would suggest 10% more unique items. | To test increased variety in a category, such as during seasonal peaks, or to reduce variety if store space is constrained. |
| Optimum Width (OW) Sensitivity | Medium | Defines how strictly the system adheres to the Ideal Number of Different Categories or Styles (OW). A “High” sensitivity means fewer deviations from the ideal. | When launching a new store concept requiring tighter control over category breadth, or loosening it for promotional flexibility. |
| Ideal Size Set (ISS) Granularity | 100% | Determines how closely the recommended Ideal Size Set (ISS) must match the exact sales distribution. | To simplify stock keeping for slower-moving items by rounding to standard pack sizes, or to prioritize exact size availability for core products. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------|
| | | 100% means exact matching, 80% allows for slight deviations. | |

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Configuration Parameters] --> B[Parameter Loading & Validation]

B --> C[Configured Planning Module]

1. Start with → Configuration Parameters (e.g., Quantity Adjustment Increment, OD Adjustment Factor)
2. Then → **Load** and **Validate** these parameters to ensure they are correct and within acceptable business ranges.
3. Next → **Apply** these settings to the core logic of the Space Planning Calculation Module.
4. Finally → A Configured Planning Module, ready to process product and sales data using the defined rules.

5. Step-by-Step Actions

1. **Load** the incoming configuration parameters
 - What we do: *We read the specific settings, like the 'Quantity Adjustment Increment', that will guide space planning calculations.*
 - Why we do it: *This ensures the system knows the fundamental rules to follow when making recommendations.*
2. **Validate** the loaded parameters against business rules
 - What we do: *We check if the entered settings are sensible and fit within predefined limits. For instance, ensuring an 'Optimum Depth Adjustment Factor' is positive.*
 - Why we do it: *This prevents erroneous calculations or recommendations that could lead to inefficient stock levels or empty shelves.*
3. **Apply** the validated parameters to the Space Planning Calculation Module
 - What we do: *We integrate these specific configuration values into the algorithms that will determine ideal product quantities and display layouts.*
 - Why we do it: *This ensures that all subsequent space planning computations adhere to the business's desired level of precision and strategic priorities.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retailer is planning the optimal stock for a new line of activewear t-shirts.
- The 'Quantity Adjustment Increment' is set to 5 units, as t-shirts are packaged and shelved in multiples of 5 for easier stocking.
- The 'Optimum Depth (OD) Adjustment Factor' is set to 1.1, as the retailer wants to slightly increase the variety of unique t-shirt designs on display to entice more customers.

What Happens

1. First, the system loads the configuration parameters: Quantity Adjustment Increment = 5 and Optimum Depth (OD) Adjustment Factor = 1.1.
2. Then, when the Space Planning Calculation Module analyzes historical sales of similar t-shirts and determines an initial Ideal Number of Unique Items (OD) of, say, 10 designs, it applies the 'OD Adjustment Factor'. The new target becomes $10 \text{ designs} * 1.1 = 11 \text{ designs}$.
3. Next, as the module calculates the ideal quantity for each of these 11 designs based on their Rate of Sale (ROS), it ensures that the final suggested quantity for each design is always a multiple of 5. For example, if a calculation suggests 12 units of a particular t-shirt, it will round up to 15 units due to the 'Quantity Adjustment Increment' setting.

Final Outcome

The business receives space planning recommendations for the new activewear t-shirts that suggest stocking 11 unique designs, with each design's quantity being a multiple of 5, fitting both the desire for slightly more variety and the operational constraint of shelving in bundles.

Key Takeaway: Correctly set configuration parameters ensure that space planning recommendations are not only data-driven but also practical for in-store operations and aligned with strategic merchandising goals.

7. Results

This step produces:

Main Outputs

- **Configured Planning Module:** A ready-to-use computational engine with all specified business parameters loaded, ensuring calculations proceed according to defined rules.
- **Parameter Validation Report:** A log indicating that all configuration settings have been successfully loaded and meet established business criteria, or flagging any errors.

Business Reports

- **Configuration Audit Trail:** Used by merchandising and operations teams to review exactly which settings were applied during a specific planning cycle, aiding in performance analysis and troubleshooting.
- **Impact Simulation Summary:** For advanced users, this report (if available) shows the potential impact of changing key parameters (like Quantity Adjustment Increment) on inventory levels and sales forecasts, used by category managers and inventory planners.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | The system will likely default to previous settings or fail to load new ones, potentially leading to outdated or incorrect planning. | Verify the source of the configuration data; manually input critical settings if necessary to avoid disruption. |
| Invalid setting value (e.g., negative increment) | The system will flag an error and prevent the parameter from being loaded, or revert to a default value, halting planning. | Review the error message, correct the invalid value in the configuration source, and re-load the parameters. |
| Configuration conflict with other modules | Parameters from this configuration might unintentionally conflict with settings in other planning modules, leading to inconsistent recommendations. | Consult the overall planning framework documentation and coordinate with relevant teams to align or prioritize conflicting settings. |
| Overly aggressive parameter settings | Setting values too high or too low (e.g., a very large Quantity Adjustment Increment) can lead to inefficient stock levels (overstock or understock). | Analyze the impact of the settings on sample data; adjust parameters incrementally based on business results and operational feedback. |

9. Next Steps

Before This Step

- **Business Strategy Definition:** We need clear objectives regarding inventory efficiency, sales targets, and customer experience to inform the initial values for configuration settings.

- **Data Preparation:** Ensure that any foundational data used to *derive* default configuration values (e.g., average pack sizes, historical sales variability) is accurate and available.

After This Step

- **Space Planning Calculation:** The Space Planning Calculation Module will use these loaded parameters to process sales and product data, generating ideal inventory and display recommendations.
- **Layout Generation & Optimization:** The calculated recommendations will then feed into tools that create visual store layouts and planograms, ensuring products are placed effectively on shelves.

10. Common Questions

Q: Why do these settings matter so much for my store's bottom line? A: These settings directly influence how much inventory you buy and how you display it. Correct settings mean you stock the right amount of popular items, avoid overstocking slow sellers, and optimize display space, all of which boost sales and reduce wasted capital.

Q: How often should I review or change these configuration settings? A: It's best to review them quarterly or semi-annually, and especially before major selling seasons (like holidays) or after significant changes in store strategy or product lines. Small adjustments to these parameters can have big impacts on your operations and sales.

Q: What happens if I set the 'Quantity Adjustment Increment' too high, say to 100 units? A: Setting it too high means the system will only recommend quantities in steps of 100. This could lead to significant overstocking for many items if demand isn't high enough, or result in stockouts if the ideal quantity falls between these large increments.

Q: Can these settings help me manage promotions more effectively? A: Yes, by temporarily adjusting settings like the 'Optimum Depth (OD) Adjustment Factor' or 'Optimum Width (OW) Sensitivity', you can encourage the system to recommend more variety or breadth for promotional displays, or to focus heavily on a few core promotional items.

Q: Who should be responsible for deciding what these settings should be? A: Typically, category managers, merchandise planners, and inventory analysts are responsible, often in collaboration with store operations, as they understand the balance between merchandising goals, sales performance, and practical store execution.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To define and load the foundational parameters that guide the Space Planning Calculation Module, ensuring tailored and precise inventory and display recommendations.

- **Main Input:** Configuration parameters like ‘Quantity Adjustment Increment’ and ‘Optimum Depth Adjustment Factor’.
 - **Main Output:** A fully configured Space Planning Calculation Module, ready to generate optimized inventory and display plans.
 - **Warning Signs:** Unrealistic inventory recommendations, frequent stockouts despite sufficient overall stock, or a constant need for manual adjustments to system suggestions.
 - **Key Settings:** Quantity Adjustment Increment, Optimum Depth (OD) Adjustment Factor, Optimum Width (OW) Sensitivity, Ideal Size Set (ISS) Granularity.
-

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 3: Space Planning Calculation Module

Last Updated: 2025-05-23

1. Business Objective

This module is the central engine for computing precise space planning recommendations, determining the ideal stock levels, product options, and final quantity suggestions for stores and categories.

Why This Matters: This ensures products are optimally stocked and displayed, maximizing sales potential, minimizing excess inventory, reducing stockouts, and enhancing the customer shopping experience by consistently having popular items available.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Critical for understanding how quickly and how many units of specific products are sold, which directly informs future demand and stock levels.
- **Rate of Sale (ROS):** Helps predict future demand and determine appropriate replenishment cycles, ensuring popular items are always in stock and limiting stockouts.
- **Revenue Contribution by Attribute Group:** Specifies how much revenue each product group (AG) contributes to the overall store or category, influencing allocation.

Product Data

- **Attribute Group (AG) Definitions:** Specifies how products are categorized by attributes (e.g., color, material, size, brand) to ensure logical grouping for display and inventory analysis.
- **Product Dimensions:** Essential for calculating physical space requirements and ensuring products fit properly within display fixtures and shelves.
- **Ideal Size Set (ISS) Distribution:** Provides historical sales data for different sizes within a product, guiding the optimal mix of sizes to stock.

Operational Data

- **Current Inventory Levels:** Provides a baseline for assessing existing stock against ideal recommendations, highlighting overstock or potential stockout situations.
- **Display Capacity Information:** Defines the physical limits of store fixtures and shelves, ensuring space planning recommendations are realistic and implementable.
- **Store Annual Operating Plan (AOP) Targets:** Strategic revenue targets for each store, used to align inventory levels with financial goals.
- **Minimum Base Stock Levels:** Store-specific minimum quantities of products that must always be on hand, regardless of sales data.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------------|---------------|--|--|
| Quantity Adjustment Increment | 1 unit | The smallest step by which suggested product quantities or display counts can be adjusted. For example, a value of 5 means quantities will always be a multiple of 5 (e.g., 10, 15, 20 units). | When store display fixtures only accommodate items in specific multiples (e.g., shoe boxes in stacks of 4) or to simplify ordering and stocking. |
| Optimum Depth (OD) Adjustment Factor | 1.0 | A multiplier used to fine-tune the calculated Ideal Number of Unique Items (OD) for a category or display. A factor of 1.1 would suggest 10% more unique items. | To test increased variety in a category, such as during seasonal peaks, or to reduce variety if store space is constrained. |
| Optimum Width (OW) Sensitivity | Medium | Defines how strictly the system adheres to the Ideal Number of Different Categories or | When launching a new store concept requiring tighter control over category breadth, or loosening it for |

| Setting Name | Default Value | Description | When to Change |
|----------------------------------|---------------|--|--|
| Ideal Size Set (ISS) Granularity | 100% | <p>Styles (OW). A “High” sensitivity means fewer deviations from the ideal.</p> <p>Determines how closely the recommended Ideal Size Set (ISS) must match the exact sales distribution. 100% means exact matching, 80% allows for slight deviations.</p> | <p>promotional flexibility.</p> <p>To simplify stock keeping for slower-moving items by rounding to standard pack sizes, or to prioritize exact size availability for core products.</p> |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

```

A[Configured Planning Module] --> B[Integrate All Data]
B --> C[Perform Space Planning Calculations]
C --> D[Generate Final Recommendations]

```

1. Start with → **Configured Planning Module** (pre-loaded with business settings from Chapter 1).
2. Then → **Integrate** all incoming Sales, Product, and Operational Data.
3. Next → **Perform** complex space planning calculations using the integrated data and configured settings.
4. Finally → **Generate** precise Space Planning Recommendations, including ideal quantities and options.

5. Step-by-Step Actions

1. **Integrate** all required sales, product, and operational data
 - What we do: *We pull together all relevant information, such as Units Sold, Attribute Group definitions, and Display Capacity, to create a comprehensive view.*
 - Why we do it: *This ensures our calculations are based on a complete and accurate understanding of demand, product characteristics, and physical store constraints.*
2. **Calculate** core space planning metrics and initial quantities
 - What we do: *We compute key metrics like Optimum Depth (OD), Optimum Width (OW), Ideal Size Set (ISS), and Rate of Sale (ROS), and use them to*

derive initial ideal stock levels and display options based on revenue contribution and sales velocity.

- *Why we do it: This establishes a data-driven baseline for how many unique items, categories, and sizes should be stocked to maximize sales and efficiency.*

3. Optimize recommendations against operational constraints

- *What we do: We refine the initial recommendations by applying business settings like the 'Quantity Adjustment Increment', considering minimum display options, and ensuring the final recommendations align with physical store areas and existing base stock levels.*
- *Why we do it: This ensures that the calculated ideal quantities are practical, fit within store layouts, and respect established operational rules, preventing recommendations that are impossible or inefficient to implement.*

4. Generate final quantity and option recommendations

- *What we do: We produce the precise number of units for each product, broken down by size, and the ideal number of unique product options to be displayed in each store and category.*
- *Why we do it: This provides clear, actionable guidance for purchasing, inventory management, and visual merchandising, ensuring the right products are in the right place at the right time.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A retailer is planning the optimal stock for their activewear t-shirt category in a new store.
- From Chapter 1, the 'Quantity Adjustment Increment' is 5 units, and the 'Optimum Depth (OD) Adjustment Factor' is 1.1.
- For a specific "Blue Stripe Performance Tee":
 - Historical Rate of Sale (ROS) from similar stores: 10 units/week.
 - Ideal Size Set (ISS) distribution for this tee: S (20%), M (30%), L (30%), XL (20%).
 - Average Selling Price (ASP) for this AG: \$25.
 - Store's Annual Operating Plan (AOP) for the activewear category: \$100,000.
 - Category's revenue contribution (derived from previous analysis): 5% of store AOP.
 - Display Capacity for t-shirts: Max 50 units total.
 - Minimum Base Stock for this category: 20 units.

What Happens

1. First, the module integrates the sales data, product attributes, store details, and pre-configured settings.
2. Then, it calculates the initial ideal stock for the activewear category based on the AOP and revenue contribution ($\$100,000 * 5\% = \$5,000$ revenue target). Divided by the AG's average selling price, this gives an initial total quantity.

3. Next, the module uses the Rate of Sale (ROS) and the configured OD Adjustment Factor (1.1) to determine the ideal number of unique t-shirt designs (Optimum Depth) to stock, ensuring sufficient variety while also considering the minimum display options.
4. For the “Blue Stripe Performance Tee,” the module calculates the ideal quantity needed based on its ROS and projected sales. If the calculation suggests 42 units are ideal, the ‘Quantity Adjustment Increment’ of 5 units would round this up to 45 units to ensure easier stocking.
5. Finally, the system applies the Ideal Size Set (ISS) distribution to the 45 units for the “Blue Stripe Performance Tee,” recommending quantities like: S: 9 units (20% of 45), M: 14 units (30% of 45, rounded), L: 14 units, XL: 9 units. It also checks if the total recommendations fit the display capacity (e.g., 45 units fits within the 50-unit capacity) and meets the minimum base stock (20 units).

Final Outcome

The business receives precise space planning recommendations for the “Blue Stripe Performance Tee,” detailing that 45 units should be stocked, broken down by size (S:9, M:14, L:14, XL:9), ensuring the store has the right variety and quantities to meet demand and operational constraints.

Key Takeaway: The Space Planning Calculation Module transforms diverse data points into actionable, precise inventory recommendations that balance sales potential with operational realities.

7. Results

This step produces:

Main Outputs

- **Ideal Stock Levels by Category/Attribute Group:** The recommended total units of products for each category or attribute group to meet forecasted demand.
- **Final Option Recommendations:** The ideal number of unique items (OD) or categories/styles (OW) suggested for display, balancing variety and depth.
- **Quantity Recommendations by SKU and Size:** The precise number of units for each specific product, broken down by size, to be stocked in each store.
- **Recommended Ideal Size Set (ISS) Breakdown:** The optimized proportion of different sizes for each product, ensuring popular sizes are well-stocked.

Business Reports

- **Space Planning Recommendations Report:** Used by category managers, merchandise planners, and store operations to understand and implement the optimal assortment and quantity plans for their stores.
- **Inventory Impact Forecast:** Utilized by inventory planners and finance teams to project future inventory investment, potential stock levels, and their financial implications based on the module’s recommendations.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|--|
| Missing or delayed data feed | The module may generate recommendations based on outdated or incomplete data, leading to inaccurate stock levels or missed sales opportunities. | Verify the data sources and ensure all required Sales, Product, and Operational data feeds are complete and timely. |
| Inconsistent or conflicting input data | The calculations might produce skewed or illogical recommendations if, for example, sales data drastically contradicts product dimensions or display capacities. | Review the Parameter Validation Report from the previous step and manually inspect source data for anomalies or discrepancies. |
| Recommendations exceed physical space/capacity | The calculated ideal quantities might be higher than a store's actual display or storage capacity, leading to unfulfillable plans. | Adjust the 'Quantity Adjustment Increment' or 'Optimum Depth (OD) Adjustment Factor' settings, or explore increasing display capacity if feasible. |
| New products with no sales history | For newly introduced products, the module lacks historical sales data for ROS or ISS, potentially leading to conservative or inaccurate recommendations. | Manually input initial estimates for ROS and ISS based on similar products or market trends, and monitor early sales closely. |
| Overly aggressive parameter settings | If settings like 'Optimum Depth (OD) Adjustment Factor' are set too high, it could lead to recommendations for excessive variety, resulting in overstocking slow movers. | Review the 'Impact Simulation Summary' (if available) and adjust parameters incrementally, validating changes with operational feedback and sales performance. |

9. Next Steps

Before This Step

- **Space Planning Configuration:** We need all foundational business parameters (like Quantity Adjustment Increment and OD Adjustment Factor) to be defined, loaded, and validated to guide the calculation process accurately.
- **Data Preparation:** Ensure that all sales, product, and operational data are accurately collected, cleaned, and made available for ingestion by this module.

After This Step

- **Layout Generation & Optimization:** The calculated ideal quantities and options will be used by other tools to create visual store planograms and optimize product placement on shelves, ensuring effective merchandising.
- **Replenishment Planning:** The final quantity recommendations will inform the purchasing and inventory replenishment systems, ensuring that stock is ordered and delivered according to the calculated optimal levels.

10. Common Questions

Q: Why are my recommendations different from last week, even if sales haven't changed much? A: Minor shifts in input data (like a new product's initial sales), subtle changes in calculated ROS or ISS, or even small adjustments to your configured business settings can lead to different recommendations as the module constantly optimizes for the best outcome.

Q: How does seasonality affect these calculations? A: The module inherently accounts for seasonality through the Rate of Sale (ROS) and Units Sold data. As demand patterns shift with seasons, these inputs automatically adjust the calculated ideal stock levels and option recommendations.

Q: Can I override the recommendations generated by this module? A: While the module provides data-driven optimal recommendations, businesses often have the flexibility to manually adjust or override specific quantities or options, especially for strategic reasons like promotions or special events. However, frequent overrides may indicate a need to fine-tune the module's settings.

Q: What if a new product has no sales history yet? How does the module recommend quantities for it? A: For new products, the module typically relies on sales data from similar Attribute Groups (AGs) or categories, often within comparable stores or regions. Alternatively, initial quantities might be set manually based on market expectations until sufficient sales data is collected.

Q: How does this module help prevent stockouts and overstocks simultaneously? A: By integrating ROS, Ideal Size Set, and Optimum Depth with operational constraints and base stock levels, the module precisely aligns inventory to projected demand and display capacity. This means it recommends having enough of what sells best (preventing stockouts) while limiting variety or quantity of slower movers (preventing overstocks).

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To compute precise space planning recommendations for ideal stock levels, product options, and final quantity suggestions.
- **Main Input:** Configured Planning Module settings, Sales Data (Units Sold, ROS), Product Data (AG Definitions, ISS), Operational Data (Store Area, Display Capacity, AOP, Base Stocks).
- **Main Output:** Ideal Stock Levels, Final Option Recommendations, Quantity Recommendations by SKU and Size, and Ideal Size Set (ISS) Breakdown.
- **Warning Signs:** Recommendations that consistently don't fit store space, lead to unexpected stockouts, or result in high levels of overstock for well-established products.
- **Key Settings:** Quantity Adjustment Increment, Optimum Depth (OD) Adjustment Factor, Optimum Width (OW) Sensitivity, Ideal Size Set (ISS) Granularity.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 4: Space Planning Orchestration Module

Last Updated: 2025-05-23

1. Business Objective

This module orchestrates the entire space planning process, ensuring a seamless flow from capturing the current store state to executing the core space planning calculations and then distributing the new, optimized recommendations.

Why This Matters: This ensures that all critical steps in generating and applying ideal merchandising plans happen reliably and in the correct sequence, minimizing manual effort, preventing errors, and ensuring stores consistently receive up-to-date and accurate product assortments and quantity recommendations.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** Critical for the snapshot of current sales performance and for the underlying Space Planning Calculation Module to understand product velocity and demand patterns.
- **Rate of Sale (ROS):** Essential for the Space Planning Calculation Module to predict future demand, and for the orchestration module to ensure the most current sales velocity data is used.

Product Data

- **Attribute Group (AG) Definitions:** Specifies how products are categorized, which is vital for the Space Planning Calculation Module and for ensuring consistency in product identification during snapshot and sync.
- **Product Dimensions:** Necessary for the Space Planning Calculation Module to calculate physical space requirements, and for the orchestration module to ensure these details are available for both snapshot and final output.

Operational Data

- **Current Inventory Levels:** Provides a baseline for the snapshot of existing stock, allowing the system to compare current stock against ideal recommendations.
- **Display Capacity Information:** Defines the physical limits of store fixtures, which the Space Planning Calculation Module uses, and ensures the orchestration module considers these constraints throughout the process.

3. Business Settings

This module orchestrates the entire space planning process, relying on the business settings defined in Chapter 1: Space Planning Configuration to guide the underlying Space Planning Calculation Module. It does not introduce new, distinct business settings of its own beyond operational scheduling parameters which are managed separately within the overall system infrastructure.

| Setting Name | Default Value | Description | When to Change |
|--------------------------------------|---------------|--|--|
| Quantity Adjustment Increment | 1 unit | The smallest step by which suggested product quantities or display counts can be adjusted. | When store display fixtures only accommodate items in specific multiples or to simplify ordering. |
| Optimum Depth (OD) Adjustment Factor | 1.0 | A multiplier used to fine-tune the calculated Ideal Number of Unique Items (OD) for a category or display. | To test increased variety or reduce variety based on strategic goals. |
| Optimum Width (OW) Sensitivity | Medium | Defines how strictly the system adheres to the Ideal Number of Different Categories or Styles (OW). | When launching a new store concept requiring tighter control over category breadth. |
| Ideal Size Set (ISS) Granularity | 100% | Determines how closely the recommended Ideal Size Set (ISS) must match the exact sales distribution. | To simplify stock keeping for slower-moving items or prioritize exact size availability for core products. |

Note: The settings listed above are managed in Chapter 1: Space Planning Configuration and are utilized by the Space Planning Calculation Module which is orchestrated by this module. See Chapter 1 for more details on these settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Current Store State] --> B[Capture Snapshot]

B --> C[Space Planning Calculation]

C --> D[Sync Computed Output]

D --> E[Updated Store Plans]

1. Start with → **Current Store State** (raw sales, product, and operational data)
2. Then → **Capture Snapshot** of existing inventory, sales, and display configurations.
3. Next → **Execute Space Planning Calculation** using the snapshot data and configured business rules.
4. Finally → **Sync Computed Output** (new recommendations) back to store systems.

5. Step-by-Step Actions

1. **Capture** a snapshot of the current state
 - What we do: *We take a precise, real-time picture of current inventory levels, product placements, display capacities, and recent sales performance across all stores and categories.*
 - Why we do it: *This provides a vital baseline, ensuring that all new space planning recommendations are built upon the most accurate understanding of the existing environment and recent sales trends.*
2. **Execute** the Space Planning Calculation Module
 - What we do: *We trigger the core calculation engine (detailed in Chapter 3), providing it with the captured snapshot data and the configured business settings (from Chapter 1) to compute ideal product assortments and quantities.*
 - Why we do it: *This step applies sophisticated algorithms to transform raw data and business rules into precise, data-driven recommendations that maximize sales potential and operational efficiency.*
3. **Sync** the computed output
 - What we do: *We take the ideal inventory recommendations and display plans generated by the calculation module and push them to relevant systems, such as store inventory management systems, visual merchandising tools, or purchasing platforms.*
 - Why we do it: *This ensures that the optimized space plans are actually implemented in stores, guiding inventory orders, physical display changes, and ultimately improving product availability and customer experience.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A regional clothing retailer needs to update the optimal product mix for its summer t-shirt category across 50 stores, as the current planning cycle ends.
- The last recommendations were applied three months ago.
- The business wants to ensure all calculations use the latest sales data and display information.

What Happens

1. First, the Space Planning Orchestration Module **captures a snapshot** of every store's current t-shirt inventory, their exact display layouts for t-shirts, and the sales performance of each t-shirt SKU over the past several weeks. This baseline ensures the system knows what's currently on shelves and how it's performing.
2. Then, the module **executes the Space Planning Calculation Module**. It feeds in the snapshot data (current stock, ROS for individual t-shirts, display capacities) along with the business settings (like Optimum Depth (OD) Adjustment Factor for t-shirts, set to 1.1 to slightly increase variety, and Quantity Adjustment Increment set to 5 units). The calculation module processes all this to determine the ideal number of unique t-shirt designs (OD) and the precise quantity of each size for every store.
3. Finally, the module **syncs the computed output**. The new recommendations – for example, advising one store to stock 12 unique t-shirt designs with specific quantities like 15 units of a “Blue Stripe Performance Tee” (S:3, M:5, L:4, XL:3) – are sent to that store's inventory system and its visual merchandising team's planning tools.

Final Outcome

The business efficiently updates the optimal product mix and quantities for the summer t-shirt category across all 50 stores. This ensures that stores stock the right variety and amount of t-shirts that are selling best, aligning inventory with current demand and display space.

Key Takeaway: The Space Planning Orchestration Module automates the entire process, from understanding the current situation to applying calculations and updating stores, ensuring accuracy and efficiency in merchandising.

7. Results

This step produces:

Main Outputs

- **Successful Execution Confirmation:** A clear indicator that the entire space planning cycle (snapshot, calculation, sync) has completed without critical errors.
- **Updated Inventory & Display Plans:** The precise, calculated recommendations for product quantities, options, and size sets, ready to be implemented in stores.
- **Process Log:** A detailed record of each step taken by the module, including timestamps and any warnings or non-critical errors.

Business Reports

- **End-to-End Space Planning Report:** Used by merchandising and operations teams to verify that the latest plans have been generated and distributed to stores, providing an overview of the cycle's outcome.
- **Orchestration Status Dashboard:** Utilized by system administrators and IT support to monitor the health and progress of space planning runs, identifying any bottlenecks or failures.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed for snapshot | The snapshot may be incomplete or outdated, leading the calculation module to work with inaccurate baseline information. | Verify data sources for completeness and timeliness; manually trigger a re-snapshot if critical data arrives late. |
| Space Planning Calculation Module fails mid-process | The orchestration halts, and no new recommendations are generated or synced, leaving stores with potentially outdated plans. | Review the Space Planning Calculation Module's error logs (Chapter 3) to identify the root cause, fix the issue, and re-run the orchestration. |
| Sync of computed output fails for some stores | Some stores receive the updated plans, while others do not, leading to inconsistency across the retail network. | Check connectivity and integration points for affected stores; manually push recommendations to those stores if necessary. |
| Significant changes to store layout or capacity | The snapshot might not fully capture a very recent physical change, leading to recommendations that don't fit the new reality. | Ensure manual updates for store layout changes are promptly fed into the system; consider a "pre-planning" run after major renovations. |

9. Next Steps

Before This Step

- **Space Planning Configuration:** All foundational business parameters must be defined, loaded, and validated (as detailed in Chapter 1) to ensure the Space Planning Calculation Module runs correctly.
- **Data Preparation:** Ensure that all sales, product, and operational data are accurately collected, cleaned, and made available for ingestion by the Space Planning Calculation Module and for the initial snapshot.

After This Step

- **Layout Generation & Optimization:** The calculated ideal quantities and options will be used by other tools to create visual store planograms and optimize product placement on shelves, ensuring effective merchandising.
- **Replenishment Planning:** The final quantity recommendations from this orchestrated process will inform the purchasing and inventory replenishment systems, ensuring stock is ordered and delivered according to the calculated optimal levels.

10. Common Questions

Q: Why do we need an orchestration module if the calculation module does the main work? A: The orchestration module is like the conductor of an orchestra. It ensures all instruments (snapshot, calculation, sync) play in the right order, at the right time, and that results are properly delivered. This automates the entire complex process, reducing errors and ensuring consistency across all stores.

Q: What happens if one part of the orchestrated process fails? A: The orchestration module is designed to detect failures. If, for example, the calculation fails, it will log the error and typically stop the process, preventing incorrect recommendations from being synced. This allows teams to investigate and resolve issues before they impact stores.

Q: How does this module help with scheduling space planning updates? A: By automating the entire flow, this module allows for reliable, scheduled runs (e.g., weekly or monthly). This means merchandising teams can consistently receive fresh recommendations without manual intervention, saving time and ensuring plans are always current.

Q: Can this module run specific planning cycles for different categories or stores? A: Yes, depending on its configuration, the orchestration module can be set up to run planning cycles for specific categories, product lines, or groups of stores, providing flexibility to address unique business needs or seasonal planning requirements.

Q: What is the benefit of taking a “snapshot” before calculations? A: Taking a snapshot creates a frozen, consistent view of the current inventory, sales, and display data at the start of the planning cycle. This ensures that the calculations are based on a fixed, accurate baseline, even if underlying operational data is constantly changing during the planning process.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** To automate and coordinate the entire space planning process, from capturing current data to calculating ideal plans and syncing them to stores.
- **Main Input:** Access to all raw sales, product, and operational data for snapshotting, and the configured business settings that guide the Space Planning Calculation Module.
- **Main Output:** Successfully generated and synced inventory and display recommendations for all stores and categories.
- **Warning Signs:** Incomplete or outdated plans appearing in stores, consistent failures in space planning runs, or delays in new recommendations reaching operational teams.
- **Key Settings:** Relies on Quantity Adjustment Increment, Optimum Depth (OD) Adjustment Factor, Optimum Width (OW) Sensitivity, and Ideal Size Set (ISS) Granularity managed in Chapter 1.

Need Help? Contact the Merchandising Systems Support team for questions about this process.

Generated by AI Codebase Knowledge Builder

spacePlanning FAQ

Version: 1.0 Last Updated: 2025-05-23

This FAQ document provides clear answers to common business questions about the spacePlanning algorithm, focusing on its functionality, data requirements, and outputs.

General

Q: What is spacePlanning? **A:** spacePlanning is an algorithm designed to help you make smarter decisions about how much product to stock and what variety to offer in your stores and categories. It's an end-to-end system, managed by the Space Planning Orchestration Module, that streamlines the entire process from data input to recommendation output.

Q: What problem does spacePlanning solve for our business? **A:** It solves the challenge of optimizing inventory by recommending ideal stock levels, options, and quantities. This helps minimize both stockouts and overstocking, leading to better sales and reduced waste, by integrating insights like OD (Optimum Depth), OW (Optimum Width), and ISS (Ideal Size Set).

Q: Who typically uses the spacePlanning recommendations? **A:** Merchandisers, inventory planners, and store operations teams are the primary users. The recommendations help them efficiently plan product assortments and quantities to maximize sales and optimize floor space.

Q: What is the main goal of spacePlanning? **A:** The primary goal is to provide data-driven recommendations for the ideal number of unique items (OD), the optimal variety of categories or styles (OW), and the best size combinations (ISS) to maximize sales and profitability across your stores and categories.

Q: How does spacePlanning help improve sales? **A:** By ensuring the right products are available in the right quantities and sizes at the right time (ISS and ROS), spacePlanning helps prevent lost sales due to stockouts and improves customer satisfaction by offering a well-matched assortment (OD and OW).

Configuration & Settings

Q: How are spacePlanning recommendations adjusted? **A:** The Space Planning Configuration module allows specific parameters to be defined, such as an incrementFactor for fine-tuning quantity adjustments. This provides a structured way to influence the computations.

Q: Can I customize how spacePlanning works for different categories or stores?

A: While the core calculations are standardized, the Space Planning Calculation Module takes into account various data points like store properties, sales data, and AG (Attribute Group) metrics, allowing the recommendations to be tailored to specific contexts.

Q: Where can I find the current operational settings for spacePlanning? **A:** Key operational parameters and configuration settings are defined within the Space Planning Configuration module, which loads and manages these arguments for the calculation process.

Q: What is the incrementFactor mentioned in the configuration? **A:** The incrementFactor is a setting that determines how granular or aggressive the quantity adjustments are during the calculation process. It's defined within the Space Planning Configuration to help refine the output recommendations.

Data & Inputs

Q: What data is essential for spacePlanning to generate recommendations? **A:** The system requires a comprehensive set of data points, including OD (Optimum Depth), OW (Optimum Width), ISS (Ideal Size Set) definitions, ROS (Rate of Sale) metrics, AG (Attribute Group) analysis, historical sales data, and specific store properties.

Q: How does spacePlanning ensure the quality of data, especially for sizes? **A:** The Size Set Properties Validation Module plays a critical role in validating the integrity and completeness of ISS related data. It checks for issues like duplicate client sizes and ensures all styles are covered by defined size sequences.

Q: Can you explain OD, OW, ISS, ROS, and AG in simple terms? **A:** * **OD (Optimum Depth):** The ideal number of *unique items (SKUs)* to stock within a specific category or display to maximize sales without excess. * **OW (Optimum Width):** The ideal number of *different categories or styles* to offer in a store or department. * **ISS (Ideal Size Set):** The *best combination of sizes* for a product to maximize sales and minimize stockouts or overstocking for a given item. * **ROS (Rate of Sale):** How *quickly products are selling* (e.g., units sold per week), which is crucial for forecasting demand. * **AG (Attribute Group):** A *grouping of product characteristics* (like color, material, or brand) used for analysis to understand performance trends.

Q: What happens if ISS data is incomplete or incorrect? **A:** Incomplete or incorrect ISS data can lead to inaccurate recommendations, potentially causing stockouts for popular sizes or excess inventory of less demanded sizes. The Size Set Properties Validation Module specifically addresses this by identifying and flagging such issues.

Q: How does the system handle the vast amount of combinations (stores, categories, SKUs)? **A:** The Map Size Computation Utility is designed to optimize performance by pre-calculating and setting initial capacities for data structures that hold various combinations of entities like stores, categories, AGs, SKUs, and periods. This ensures efficient processing of large datasets.

Process & Logic

Q: How does the spacePlanning process work from start to finish? **A:** The Space Planning Orchestration Module manages the entire flow. It first captures a snapshot of your current inventory and sales, then triggers the core Space Planning Calculation Module to compute recommendations, and finally syncs the computed output back into your systems.

Q: What is the core engine behind spacePlanning recommendations? **A:** The Space Planning Calculation Module is the central component. It integrates all the diverse data points, including OD, OW, ISS, ROS, AG metrics, sales data, and store properties, to determine the ideal stock levels and quantity recommendations.

Q: How does the algorithm determine the ideal quantity for a specific SKU in a store? **A:** The Space Planning Calculation Module uses a sophisticated approach, combining the ROS for that SKU, the ISS for its style, and factoring in the OD and OW targets for its category and store. It also considers store-specific properties and any defined configuration parameters.

Q: What role does data validation play in the process? **A:** Data validation, particularly by the Size Set Properties Validation Module, is crucial before calculations begin. It ensures that the input data, especially related to ISS properties, is accurate and complete, preventing errors and ensuring reliable recommendations.

Q: Is the spacePlanning process performance-optimized? **A:** Yes, the system includes optimizations like the Map Size Computation Utility, which pre-determines the necessary capacities for data structures. This helps improve the overall performance and speed of the calculations, especially with large datasets.

Outputs & Results

Q: What kind of recommendations does spacePlanning provide? **A:** spacePlanning provides clear recommendations for ideal stock levels, the optimal number of options (OW), and specific quantity recommendations for SKUs, broken down by stores and categories.

Q: How do these recommendations help minimize stockouts and overstock? **A:** By precisely calculating the OD, OW, ISS, and leveraging ROS data, the system recommends quantities that meet anticipated demand without accumulating excess inventory. This balance helps reduce stockouts of fast-moving items and avoids tying up capital in slow-moving ones.

Q: Can spacePlanning help me plan future assortments? **A:** Yes, by providing insights into OD, OW, and ISS based on historical sales and future demand parameters, spacePlanning can be a powerful tool for planning and optimizing future product assortments.

Q: What kind of output format can I expect from spacePlanning? **A:** The Space Planning Orchestration Module is responsible for syncing the computed output. You can expect structured data outputs detailing the recommended quantities, ideal stock levels, and optimized assortments that can be integrated into your existing inventory or merchandising systems.

Error Handling & Edge Cases

Q: What happens if some necessary input data is missing or invalid? **A:** The system incorporates data validation steps, particularly through the `Size Set Properties Validation Module` for ISS data. If critical data is missing or invalid, the system aims to flag these issues, and in some cases, it may not be able to produce recommendations for the affected items or categories.

Q: How does `spacePlanning` handle unique store layouts or very small store capacities? **A:** The `Space Planning Calculation Module` considers store properties as part of its inputs. While specific edge cases might require manual review, the algorithm incorporates these attributes to tailor recommendations to the specific constraints of each store.

Q: Are there any known limitations or specific scenarios that `spacePlanning` doesn't cover? **A:** Currently, there are no specific known edge cases that prevent the algorithm from functioning or providing recommendations. The system is designed to be robust and adaptable to a wide range of retail scenarios by integrating diverse data inputs.

Need More Help?

Please contact the support team for further assistance.

spacePlanning Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

The spacePlanning system is a critical tool designed to optimize inventory levels and product assortments across various retail environments. By leveraging key metrics like Optimum Depth (OD), Optimum Width (OW), Ideal Size Set (ISS), and Rate of Sale (ROS), it provides actionable recommendations for product quantities and display configurations. This ensures that businesses can maximize sales potential, minimize stockouts, and improve overall inventory efficiency.

3. Architecture Overview

```
graph TD
    subgraph Space_Planning_System [Space Planning System]
        A["A[0: Space Planning Orchestration Module]"]
        B["B[1: Space Planning Calculation Module]"]
        C["C[2: Size Set Properties Validation Module]"]
        D["D[4: Space Planning Configuration]"]
    end

    A -- control_flow --> B
    B -- configuration --> D
    C -- depends_on --> B
```

- **Space Planning Orchestration Module:** Manages the end-to-end execution of the space planning process, from data capture to output synchronization.
- **Space Planning Calculation Module:** The core component responsible for computing ideal stock levels, option counts, and final quantity recommendations based on various input data and business logic.
- **Size Set Properties Validation Module:** Ensures the accuracy and completeness of product size set data, which is crucial for precise space planning calculations.
- **Map Size Computation Utility:** (Not shown in primary flow, but supports performance) An internal utility that optimizes system performance by pre-calculating ideal capacities for data structures.
- **Space Planning Configuration:** Defines and loads all runtime parameters and settings necessary for the space planning computations.

4. Prerequisites & Setup

To run the spacePlanning system, ensure the following prerequisites are met:

- **Java Development Kit (JDK):** Version 11 or higher.
- **Build Tool:** Maven (recommended) for dependency management and project compilation.
- **Database Access:** Connectivity to the underlying database from which sales, product, and operational data are retrieved.
- **Cache Service:** Access to a cache service (e.g., Redis) for efficient data retrieval.

To Run the System: The Space Planning Orchestration Module acts as the primary entry point. Typically, it can be executed as a standalone Java application or integrated into a larger microservices framework.

1. Build the project:

```
mvn clean install
```

2. Run the orchestrator:

```
java -jar target/spacePlanning-1.0.jar --module-  
group=SpacePlanningOrchestrationModule
```

(Note: The exact command may vary based on your Spring Boot or application runner configuration.)

5. Folder & File Structure

| | |
|--------------------------------------|---------------------------------------|
| spacePlanning/ | |
| ├─ module-group/ | – Contains the primary orchestrator |
| module logic. | |
| ├─ submodules/ | – Houses the individual calculation |
| and utility modules. | |
| ├─ args.properties | – Global configuration parameters for |
| the system. | |
| ├─ common-data.json | – Defines the schema for shared data |
| models used across modules. | |
| ├─ outputs/ | – Directory where the generated space |
| planning recommendations are stored. | |

6. Global Configuration

The `args.properties` file is used to configure global parameters for the spacePlanning system.

(No global configuration parameters were provided in the input, so this section is empty.)

| Setting Name | Default Value | Description | Valid Range/When to Change |
|--------------|---------------|-------------|----------------------------|
| | | | |

7. Shared Data Model

The `common-data.json` schema defines the structure for data exchanged or shared across different modules within the spacePlanning system.

(No shared data model schema was provided in the input, so this section is empty.)

| Field | Type | Description |
|-------|------|-------------|
| | | |

8. Submodule Chapters

Space Planning Orchestration Module

Module Name & Business Objective

The Space Planning Orchestration Module orchestrates the entire space planning process, including capturing a snapshot of the current state, executing the core Space Planning Calculation Module, and then syncing the computed output. This module ensures the end-to-end execution flow for space planning.

Required Information

- **Sales Data:** Not directly required by this module, but ensures underlying data for calculation is accessible.
- **Product Data:** Not directly required, but ensures underlying data for calculation is accessible.
- **Operational Data:** Not directly required, but ensures underlying data for calculation is accessible.

Business Settings

This orchestrator module typically doesn't have specific business settings beyond scheduling or execution triggers, which are managed externally.

Process Flow

graph TD

```
A[Start] --> B(Capture Current State Snapshot)
B --> C(Execute Space Planning Calculation Module)
C --> D(Sync Computed Output)
D --> E[End]
```

Step-by-Step Actions

1. **Capture** a snapshot of the current inventory and display state, providing a baseline for analysis.
2. **Execute** the core Space Planning Calculation Module, triggering the algorithms that determine optimal depths and widths.
3. **Sync** the computed output, integrating the new space planning recommendations back into inventory or display management systems.

Sample Scenario

A retail chain decides to run its quarterly space planning exercise to adjust inventory levels across all stores based on recent sales trends. The Space Planning Orchestration Module is *triggered* automatically via a scheduler. It first *gathers* the current stock data from all stores, then *invokes* the Space Planning Calculation Module to generate the new recommendations, and finally, *pushes* the updated inventory plans to the warehouse and store systems. **Key Takeaway:** The Orchestration Module acts as the conductor, ensuring all necessary steps for a successful space planning run are completed seamlessly.

Results

- Successful initiation and completion of the space planning process.
- Integration of calculated recommendations into target systems.

Unusual Cases

- **Missing or delayed data feed:** If the initial snapshot data is incomplete or delayed, the orchestration may fail or proceed with outdated information.
- **Failure of the Space Planning Calculation Module:** The orchestrator will typically log an error and stop, requiring manual intervention.
- **Sync issues with target systems:** Output might not be correctly integrated if there are network or API problems.

Next Steps

- **Review** the logs for successful execution.

- **Monitor** the integration of the computed plans into downstream systems.

Common Questions

1. How often should this module be run? *Typically, this depends on business cycles (e.g., weekly, monthly, quarterly, or before new seasons).*
2. What happens if the calculation module fails? *The orchestration module will report an error, and the process will halt, requiring investigation.*
3. Can I trigger it manually? *Yes, if configured to allow manual execution for ad-hoc planning.*
4. How does it ensure data consistency? *By capturing a snapshot before calculations begin and validating the output before syncing.*
5. What are the key outputs? *The primary output is the set of recommendations generated by the Space Planning Calculation Module.*

Quick Reference

- **Purpose:** End-to-end process management for space planning.
- **Key Action:** Orchestrates data capture, calculation, and output sync.
- **Dependency:** Relies heavily on the Space Planning Calculation Module.

Space Planning Calculation Module

Module Name & Business Objective

The Space Planning Calculation Module is the central module responsible for computing space planning recommendations. It integrates various data points like Optimum Depth (OD), Optimum Width (OW), Ideal Size Set (ISS), Rate of Sale (ROS), and Attribute Group (AG) metrics, sales data, and store properties to determine ideal stock levels, options, and final quantity recommendations for stores and categories.

Required Information

- **Sales Data:**
 - Historical sales performance by SKU, category, store, and AG.
 - Forecasted sales data.
 - Rate of Sale (ROS) for individual SKUs and AGs.
- **Product Data:**
 - SKU master data (dimensions, weight, packaging).
 - Product hierarchies (category, subcategory, style).
 - Attribute Group (AG) definitions (color, material, brand).
 - Ideal Size Set (ISS) definitions and SizeSetPropertiesRow entries.
 - Product capacities (e.g., how many units fit on a shelf).
- **Operational Data:**
 - Store layouts and display capacities.
 - Warehouse inventory levels.
 - Lead times for replenishment.
 - Store clustering information.

Business Settings

- **incrementFactor**: Used in quantity adjustments to ensure recommendations are in practical increments (e.g., always suggesting quantities in multiples of 6 units per box).

Process Flow

graph TD

```
A[Raw Sales & Product Data] --> B(Ingest & Clean Data)
B --> C(Compute OD & OW)
C --> D(Determine ISS per Style/Category)
D --> E(Calculate Recommended Quantities)
E --> F[Space Planning Recommendations]
```

Step-by-Step Actions

1. **Ingest** and validate comprehensive sales, product, and operational data, ensuring data quality.
2. **Calculate Optimum Depth (OD)** based on historical ROS, display capacity, and desired stock turns, determining the ideal number of unique items to stock.
3. **Determine Optimum Width (OW)** by analyzing category performance and store size, identifying the ideal number of different categories or styles to offer.
4. **Derive Ideal Size Set (ISS)** for relevant product styles, ensuring the best combination of sizes to maximize sales and minimize stockouts.
5. **Compute** final recommended quantities for each SKU per store/category, considering OD, OW, ISS, ROS, AG metrics, and any configured **incrementFactor**.
6. **Generate** detailed space planning recommendations, ready for review and execution.

Sample Scenario

A large apparel retailer needs to plan inventory for its upcoming spring collection across 500 stores, varying in size and customer demographics.

The Space Planning Calculation Module *processes* historical sales data for similar styles and categories, *identifying* high-performing Attribute Groups. It *calculates* the ideal OD for each store's display areas, suggesting that a small boutique might carry 5 unique t-shirt styles (OD=5) while a flagship store carries 15 (OD=15). Concurrently, it *determines* the OW, ensuring that only the most relevant product categories are offered in each store type. For specific items like dresses, it *applies* the ISS based on regional customer demand, ensuring the right mix of S, M, L, XL sizes are available. Finally, it *adjusts* all quantities to be multiples of 12 (our **incrementFactor**) as products are packed in dozens.

Key Takeaway: Precise data combined with configurable parameters leads to actionable inventory plans that align with store capacity and customer demand, minimizing waste and maximizing sales.

Results

- Ideal stock levels by SKU, store, and category.
- Recommendations for Optimum Depth (OD) and Optimum Width (OW).
- Proposed Ideal Size Set (ISS) for specific products.
- Final quantity recommendations for procurement and replenishment.

Unusual Cases

- **Missing or delayed data feed:** Incomplete sales history or product data can lead to inaccurate recommendations.
- **Invalid ISS definitions:** If size sets are poorly defined, the module may generate impractical size allocations.
- **Sudden market shifts:** Unforeseen changes in consumer demand or supply chain disruptions may render calculated recommendations less effective.
- **Calculation errors:** Complex interdependencies can sometimes lead to numerical instability or logical errors requiring thorough review.

Next Steps

- **Review** the generated recommendations for business alignment and feasibility.
- **Integrate** the recommendations into inventory management and merchandising systems.
- **Monitor** the performance of stores implementing the new plans.

Common Questions

1. How are OD, OW, and ISS specifically calculated? *They are determined through a combination of historical ROS, display capacities, category performance, and statistical analysis of past sales by size and attribute.*
2. What is the most critical data input for this module? *Accurate and comprehensive historical sales data, especially ROS, combined with current product master data, is paramount.*
3. Can I override specific recommendations? *The system provides recommendations, but business users typically have mechanisms to review and adjust them before final execution.*
4. How does it handle seasonal variations? *The module accounts for seasonality by leveraging historical sales patterns for corresponding periods and adjusting forecasts accordingly.*
5. What if a product has no sales history? *The module may use comparable product data, category averages, or default OD/OW settings, or flag it for manual review.*

Quick Reference

- **Purpose:** Core logic for space planning recommendations.
- **Key Metrics:** OD, OW, ISS, ROS, AG.
- **Inputs:** Sales, product, operational data.
- **Output:** Optimized inventory quantities and display plans.

Size Set Properties Validation Module

Module Name & Business Objective

The Size Set Properties Validation Module validates the integrity and completeness of ISS (Ideal Size Set) related data. This includes checking `SizeSetPropertiesRow` entries for store-category-subcategory combinations, identifying duplicate client sizes within size sequences, and ensuring that all styles are covered by defined size sequences.

Required Information

- **Sales Data:** Not directly required by this module.
- **Product Data:**
 - `SizeSetPropertiesRow` entries: Defines size sets for specific store/category/subcategory combinations.
 - `CatSizeSeqRow` entries: Defines category-specific size sequences.
 - Style data: Information on all active product styles.
- **Operational Data:** Not directly required by this module.

Business Settings

This validation module typically does not have specific business settings, as its primary function is data integrity checks.

Process Flow

graph TD

```
A[Input: SizeSetPropertiesRow Data] --> B(Check Store-Category-Subcategory Combinations)
B --> C(Identify Duplicate Client Sizes within Sequences)
C --> D(Ensure All Styles are Covered by Defined Sequences)
D --> E[Output: Validation Report / Flags]
```

Step-by-Step Actions

1. **Load** all relevant `SizeSetPropertiesRow` and `CatSizeSeqRow` data, along with current product style information.
2. **Validate** `SizeSetPropertiesRow` entries for correct store-category-subcategory mapping, ensuring logical and complete assignments.
3. **Identify** and flag duplicate client sizes within the same size sequence, preventing redundant or erroneous size allocations (e.g., "S, S, M, L").
4. **Verify** that all active product styles have a corresponding *Ideal Size Set* (ISS) defined for their respective categories, preventing omissions in planning.
5. **Generate** a comprehensive validation report, highlighting any data inconsistencies or missing definitions.

Sample Scenario

Before the Space Planning Calculation Module runs for the new seasonal apparel, it's crucial to ensure the size set data is flawless. The Size Set Properties Validation Module is *executed*. It *scans* the SizeSetPropertiesRow data for all apparel categories. It *discovers* that for "Women's Dresses" in "Store Type A," there's a size sequence listed as "XS, S, S, M, L," flagging the duplicate 'S'. It also *finds* that a newly introduced "Unisex Sweatshirts" style *lacks* any defined size sequence for certain regions. These issues are *reported* back. **Key Takeaway:** Data quality in ISS is paramount. This validation prevents the Space Planning Calculation Module from using incorrect or incomplete size distributions, which could lead to significant stock imbalances or missed sales.

Results

- Reports on duplicate client sizes within size sequences.
- Identification of styles not covered by defined size sequences.
- Overall integrity check of SizeSetPropertiesRow entries.
- Flags for data inconsistencies that need to be addressed.

Unusual Cases

- **Missing or delayed data feed:** If SizeSetPropertiesRow or style data is incomplete, the validation may falsely report missing coverage or miss actual duplicates.
- **Corrupt size set data:** Malformed or unparseable data entries can cause the validation to fail or produce misleading results.
- **New product categories/styles without defined size sets:** This module will correctly flag these, requiring immediate action from merchandising.

Next Steps

- **Review** the validation report thoroughly.
- **Correct** any identified data inconsistencies or omissions in the SizeSetPropertiesRow or CatSizeSeqRow tables.
- **Re-run** the validation until all issues are resolved.

Common Questions

1. Why is size set validation so important? *Accurate ISS is critical for ensuring the right mix of sizes are in stock, preventing common sizes from selling out while others sit idle.*
2. What happens if validation fails? *The Space Planning Calculation Module might produce suboptimal or incorrect quantity recommendations, potentially leading to excess inventory or stockouts.*
3. How often should this validation run? *Ideally, before every major space planning run or whenever ISS or product master data is updated.*
4. Can it fix errors automatically? *No, it identifies errors. Manual intervention or a separate data cleansing process is required to fix the underlying data.*

5. What are common types of errors it finds? *Duplicate sizes in a sequence, styles not having a defined size set, and incorrect linking of size sets to categories/stores.*

Quick Reference

- **Purpose:** Ensure ISS data integrity.
- **Key Checks:** Duplicates, coverage, SizeSetPropertiesRow validity.
- **Benefit:** Prevents inaccurate size distribution in plans.

Map Size Computation Utility

Module Name & Business Objective

A utility class dedicated to calculating and setting initial capacities for Map objects used throughout the system. This optimization helps improve performance by pre-determining the number of combinations of various entities (e.g., stores, categories, AG (Attribute Group)s, SKUs, periods) to initialize maps efficiently.

Required Information

- **Sales Data:** Not directly required by this module.
- **Product Data:** Number of SKUs, categories, Attribute Groups.
- **Operational Data:** Number of stores, periods.

Business Settings

This is an internal utility and does not have specific business settings. Its logic is based on performance optimization.

Process Flow

```
graph TD
    A[Input: Counts of Entities (Stores, SKUs, etc.)] --> B[Calculate Optimal Map Capacity]
    B --> C[Output: Recommended Map Initial Capacity]
```

Step-by-Step Actions

1. **Receive** counts for various entities such as stores, categories, SKUs, Attribute Groups, and time periods.
2. **Apply** internal heuristics and formulas to estimate the optimal initial capacity for a Map that will store combinations of these entities.
3. **Return** the calculated capacity, which is then used by other modules to initialize their Map data structures.

Sample Scenario

A system needs to store SKU performance data across 1,000 stores, 50 product categories, and 12 sales periods. Instead of letting the Map dynamically resize multiple times (which is inefficient), the Map Size Computation Utility is *invoked*. It *takes* the counts (1000 stores, 50 categories, 12 periods) and *calculates* an initial Map capacity that can efficiently hold all $1000 * 50 * 12 = 600,000$ potential combinations (plus some overhead for hash collisions). This calculated capacity is then used when creating the actual Map object. **Key Takeaway:** Pre-calculating Map capacities significantly improves application performance and reduces memory overhead, especially when dealing with large datasets of inter-related entities.

Results

- Optimized initial capacities for Map objects.
- Improved system performance and reduced memory consumption during data processing.

Unusual Cases

- Extremely large input counts: May result in very large map allocations that consume significant memory if not carefully managed.
- Zero or negative input counts: The utility should handle these gracefully, potentially defaulting to a small capacity or throwing an error if invalid.

Next Steps

- (This utility is typically used internally by other modules and does not require explicit next steps from a business perspective.)

Common Questions

1. How does this improve performance? *By reducing the need for Map re-hashing and resizing operations, which are computationally expensive, as elements are added.*
2. When is this utility used? *It's typically called by data processing modules (e.g., Space Planning Calculation Module) before populating large Map data structures.*
3. Is this visible to business users? *No, this is a technical optimization not exposed to business users.*
4. Can I configure the calculation logic? *No, the calculation logic is embedded within the utility for optimal performance based on technical requirements.*
5. What kind of data structures benefit from this? *Any Map-like structure that stores large numbers of key-value pairs derived from combinations of entities.*

Quick Reference

- **Purpose:** Performance optimization for Map initialization.
- **Key Action:** Calculates ideal initial Map capacities.
- **Benefit:** Faster data processing, reduced memory usage.

Space Planning Configuration

Module Name & Business Objective

The Space Planning Configuration module defines and loads configuration parameters specific to the Space Planning Calculation Module, such as the `incrementFactor` used in quantity adjustments. It provides a structured way to pass runtime arguments to the space planning computations.

Required Information

- **Sales Data:** Not directly required by this module.
- **Product Data:** Not directly required by this module.
- **Operational Data:** Not directly required by this module.
- **Configuration Files:** `args.properties` or other specified configuration sources.

Business Settings

- `incrementFactor`: A critical setting for the Space Planning Calculation Module that ensures recommended quantities are in practical multiples (e.g., cases, bundles).

Process Flow

graph TD

A[Configuration Source (e.g., `args.properties`)] --> B[Load Parameters]

B --> C[Provide Configuration to Modules]

Step-by-Step Actions

1. **Read** *configuration values from designated sources (e.g., `args.properties` file or environment variables).*
2. **Validate** *the loaded parameters against expected types and ranges, ensuring data integrity.*
3. **Make** *the configuration parameters available to the Space Planning Calculation Module and other dependent components.*

Sample Scenario

The business wants to fine-tune how product quantities are rounded up or down. Currently, the system recommends individual units, but products are shipped in cases of 10. A business analyst *updates* the `args.properties` file, setting `incrementFactor=10`. When the Space Planning Calculation Module runs, the Space Planning Configuration module *loads* this new value. Now, all calculated quantities are *adjusted* to the nearest multiple of 10, ensuring that recommendations are practical for warehousing and shipping operations. **Key**

Takeaway: Externalized configuration allows for flexible tuning of the space planning algorithm's behavior without requiring code changes, empowering business users to adapt recommendations to operational realities.

Results

- Dynamic configuration of Space Planning Calculation Module behavior.
- Ability to fine-tune parameters like `incrementFactor` without code deployments.

Unusual Cases

- **Missing or delayed data feed:** If the configuration file is missing or unreadable, the module might default to hardcoded values or fail to initialize, impacting calculations.
- **Invalid parameter values:** Incorrect data types or values outside the expected range (e.g., `incrementFactor = "abc"`) can lead to errors during module execution.

Next Steps

- **Update** `args.properties` file to adjust desired operational parameters.
- **Restart** the application (if necessary) to pick up new configuration values.

Common Questions

1. How do I change a setting? *Settings are typically changed by editing the `args.properties` file and restarting the application (if not dynamically loaded).*
2. What is `incrementFactor` used for? *It ensures that recommended product quantities are in practical units, like multiples of cases or bundles, aligning with packaging or shipping standards.*
3. Are there other settings I can change? *Yes, any parameter defined in `args.properties` and consumed by the calculation module can be tuned here.*
4. Why is configuration separate from the code? *To allow for flexible tuning of the algorithm's behavior by business users or operations teams without requiring developers to change and re-deploy code.*
5. What happens if a setting is missing? *The module will typically use a default value or throw an error, depending on its design.*

Quick Reference

- **Purpose:** Externalize and manage configuration parameters.
 - **Key Setting:** `incrementFactor`.
 - **Benefit:** Flexible tuning of space planning computations.
-

9. Appendix

Full Parameter Reference

(No global configuration parameters were provided in the input, so this section is empty. Module-specific parameters are noted in their respective chapters.)

Version History

- **1.0 (2025-05-23):** Initial release of the spacePlanning system documentation.

Contact & Support Info

For any questions, issues, or support requests regarding the spacePlanning system, please contact:

- **Support Team:** [support@example.com]
- **Internal Documentation:** [Link to internal wiki/confluence page]

Tutorial: spacePlanning

The spacePlanning system orchestrates the entire space planning process, with the core calculation module performing computations based on various data points and configuration. Data validation ensures the integrity of inputs used in the calculations.

flowchart TD

```
A0["Space Planning Orchestration Module"]
A1["Space Planning Calculation Module"]
A2["Size Set Properties Validation Module"]
A3["Map Size Computation Utility"]
A4["Space Planning Configuration"]
A0 -- "control_flow" --> A1
A1 -- "configuration" --> A4
A2 -- "depends_on" --> A1
```

Chapters

This report outlines the spacePlanning merchandising algorithm, designed to optimize product assortment and allocation across retail spaces.

Master Report: spacePlanning Merchandising Algorithm

1. Title & Metadata

- **Project Name:** spacePlanning
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** spacePlanning revolutionizes retail merchandising by intelligently determining the Optimum Depth (OD) and Optimum Width (OW) of products, alongside Ideal Size Sets (ISS), to maximize sales and inventory efficiency.

2. Executive Summary

The spacePlanning algorithm is a critical tool for modern retail merchandising, designed to move beyond traditional planogramming to data-driven assortment and space optimization. By leveraging advanced analytics, it provides precise recommendations for how many unique items (SKUs) to stock (OD), how many different categories or styles to offer (OW), and the optimal combination of sizes (ISS) for each product within a given retail environment. This leads to more efficient inventory management, reduced stockouts, enhanced customer satisfaction, and ultimately, increased profitability.

- **Key outcomes or KPIs:**
 - Improved sales conversion rates
 - Reduced inventory holding costs and waste (markdowns)
 - Optimized stock-to-sales ratios
 - Enhanced product availability and customer satisfaction
 - Increased overall store profitability per square foot
- **Primary stakeholders or downstream consumers:**
 - Merchandising Teams (Assortment Planners, Buyers)
 - Store Operations (Store Managers, Visual Merchandisers)
 - Supply Chain & Inventory Management
 - Category Management
 - Sales & Finance Teams

3. Architecture & Data Flow

Diagram:

graph TD

```
A[Raw Sales Data] --> B{Data Ingestion & Preprocessing}
C[Inventory Data] --> B
D[Product Attributes] --> B
E[Store Layouts/Capacity] --> B
F[Business Rules/Config] --> B

B --> G[Demand Forecasting]
G --> H[OD & OW Calculation]
H --> I[ISS Generation]
I --> J[Space Allocation & Optimization]
J --> K[Recommendation Engine]

K --> L[Optimized Assortment Plan (OD, OW, ISS)]
K --> M[Space Allocation Map]
K --> N[Performance Metrics Report]

L -- Merchandising --> P(Merchandising Teams)
M -- Store Operations --> Q(Store Operations)
N -- Analysis --> R(Analysts & Leadership)
```

Component Bullets:

- **Data Ingestion & Preprocessing:** Collects, cleans, and transforms raw operational data into a standardized format.
- **Demand Forecasting:** Predicts future product sales based on historical data, seasonality, and other factors.
- **OD & OW Calculation:** Determines the ideal number of SKUs and categories to stock based on demand, space constraints, and business objectives.
- **ISS Generation:** Calculates the optimal mix of sizes for each recommended SKU to maximize sales and minimize stockouts.
- **Space Allocation & Optimization:** Assigns recommended products to physical retail space, considering capacity, adjacency, and visual merchandising principles.
- **Recommendation Engine:** Consolidates outputs from all modules to generate comprehensive, actionable merchandising plans.

High-level data sources:

- **POS (Point-of-Sale) Data:** Transactional sales data, including units sold, price, date, and location.
- **Inventory Management Systems:** Current stock levels, inbound shipments, returns, and inventory costs.
- **Product Information Management (PIM):** Detailed product attributes (e.g., color, material, brand, category, dimensions).
- **Store/Fixture Data:** Dimensions, capacities, and layout configurations of retail spaces and display fixtures.

- **Historical Promotional Data:** Information on past marketing campaigns and their impact on sales.

4. Configuration & Data Dictionary

Settings Table (`args.properties`):

| Parameter Name | Type | Description | Default Value | Example Value |
|----------------------------------|---------|--|----------------|---------------|
| <code>min_ros_threshold</code> | Double | Minimum Rate of Sale (ROS) for a SKU to be considered viable for stocking. | 0.1 | 0.5 units/wk |
| <code>max_od_per_category</code> | Integer | Maximum Optimum Depth (OD) allowed for any single category. | 50 | 30 |
| <code>max_ow_per_store</code> | Integer | Maximum Optimum Width (OW) (number of distinct categories) per store. | 20 | 15 |
| <code>target_fill_rate</code> | Double | Desired percentage of customer demand to be met (0.0-1.0). | 0.95 | 0.98 |
| <code>space_util_target</code> | Double | Target percentage of available space to be utilized (0.0-1.0). | 0.9 | 0.95 |
| <code>seasonality_factor</code> | String | Method for applying seasonality adjustments to forecasts (e.g., additive, multiplicative). | multiplicative | additive |
| <code>min_size_count</code> | Integer | Minimum number of sizes to include in an ISS for a given SKU. | 3 | 4 |
| <code>max_size_options</code> | Integer | Maximum number of individual size options to evaluate for ISS. | 10 | 12 |

Shared Data Fields (common-data.json):

```
{
  "product_id": "Unique identifier for each product SKU",
  "category_id": "Identifier for the product's primary category",
  "style_id": "Identifier for a specific product style (e.g., 'T-Shirt - Crew Neck')",
  "size": "Specific size of the product (e.g., 'S', 'M', 'L', 'XL', '32x32')",
  "color": "Primary color of the product",
  "store_id": "Unique identifier for the retail store location",
  "sales_units": "Number of units sold for a product during a period",
  "sales_revenue": "Total revenue generated for a product during a period",
  "inventory_on_hand": "Current number of units in stock for a product at a location",
  "display_capacity": "Total units a display fixture can hold",
  "fixture_type": "Type of display fixture (e.g., 'shelf', 'hanger', 'bin')",
  "week_start_date": "Start date of the sales/inventory reporting week"
}
```

Outputs Catalog:

| Output Name | Description | Format | Recipient |
|--------------------------------|--|---------------|--|
| optimized_assortment_plan.csv | List of recommended SKUs per store, with their calculated OD, OW, and ISS. | CSV | Merchandising, Supply Chain |
| store_planogram_map.json | Detailed spatial allocation of products within each store's fixtures. | JSON (or SVG) | Store Operations, Visual Merchandising |
| performance_summary_report.pdf | Summary of projected KPIs (sales, inventory turns, space utilization). | PDF | Management, Finance |
| stock_recommendations.csv | Specific stocking levels for each SKU by | CSV | Inventory Management |

| Output Name | Description | Format | Recipient |
|-------------------------------|---|--------|---------------------|
| attribute_group_analysis.xlsx | size, per store.
Breakdown of performance by Attribute Group (AG) (e.g., color, material). | XLSX | Category Management |

5. End-to-End Flow

The spacePlanning algorithm operates through a sequential, data-driven pipeline:

- 1. Data Ingestion:** Raw sales, inventory, product, and store capacity data are collected from various enterprise systems.
- 2. Data Preprocessing:** Data undergoes cleansing, validation, normalization, and aggregation to ensure consistency and quality. Key metrics like historical ROS are calculated here.
- 3. Demand Forecasting:** Using preprocessed historical sales, the system projects future demand for each SKU at each store, incorporating seasonality and trend analysis.
- 4. Optimal Depth (OD) & Width (OW) Determination:** Based on forecasted demand, store capacity, and business rules (e.g., max_od_per_category, max_ow_per_store), the algorithm identifies the ideal number of unique SKUs (OD) and distinct categories/styles (OW) to offer. This involves balancing variety with inventory efficiency.
- 5. Ideal Size Set (ISS) Generation:** For each selected SKU, the system analyzes its historical size-level ROS and demand patterns to recommend the optimal mix of sizes (e.g., 2S, 4M, 4L, 2XL) to maximize sales while minimizing excess inventory or stockouts.
- 6. Space Allocation:** The algorithm then maps the recommended assortment (OD, OW, ISS) onto the physical store layouts and fixtures, considering constraints such as shelf space, display type, and adjacency rules.
- 7. Recommendation & Reporting:** Finally, the system generates actionable merchandising plans, detailed planograms, and performance reports, providing clear guidance to merchandising, operations, and supply chain teams. This output enables data-informed decisions for product placement and stocking levels.

6. Submodule Overview

6.1. Data Ingestion & Preprocessing

- **Purpose:** To collect, clean, validate, and transform raw data into a standardized format suitable for downstream analysis.
- **Inputs & Outputs:**
 - **Inputs:** Raw POS data, Inventory snapshots, Product Master data, Store/Fixture layouts.

- **Outputs:** Cleaned & validated sales transactions, Aggregated inventory levels, Standardized product attributes (including AGs), Calculated historical ROS per SKU.
- **Core Logic Summary:**
 1. Ingest data from various sources (e.g., databases, flat files) into a staging area.
 2. Perform data cleansing: handle missing values, correct data types, and remove duplicates.
 3. Normalize and standardize data fields (e.g., consistent size labels, category hierarchies).
 4. Calculate baseline metrics, such as weekly ROS for each SKU by store, and enrich product data with relevant Attribute Groups (AGs).

6.2. Demand Forecasting

- **Purpose:** To predict future sales demand for each SKU, considering historical patterns and external factors.
- **Inputs & Outputs:**
 - **Inputs:** Preprocessed historical sales data (weekly ROS), Promotional calendar, Seasonality data.
 - **Outputs:** Forecasted weekly sales (units) for each SKU per store for a defined future period.
- **Core Logic Summary:**
 1. Analyze historical sales trends, identifying seasonality, promotions, and outlier events.
 2. Apply statistical or machine learning models (e.g., ARIMA, Prophet, time series models) to generate future demand forecasts.
 3. Adjust forecasts based on planned promotions or known market shifts.
 4. Output probabilistic demand curves for each SKU to inform inventory safety stock calculations.

6.3. OD & OW Calculation

- **Purpose:** To determine the ideal number of unique items (OD) and categories/styles (OW) to offer within a given retail space.
- **Inputs & Outputs:**
 - **Inputs:** Forecasted demand, Store capacity data, Business rules (e.g., max_od_per_category, min_ros_threshold).
 - **Outputs:** Recommended OD per category, Recommended OW per store, List of selected SKUs/styles.
- **Core Logic Summary:**
 1. Evaluate each potential SKU/style based on its forecasted ROS and profitability.
 2. Apply hierarchical selection, first determining ideal OW (categories/styles) for the store based on aggregate demand and strategic importance.
 3. Within each selected category/style, determine the ideal OD (number of distinct SKUs) by ranking SKUs by forecasted ROS, profitability, and customer preference, while respecting space constraints.
 4. Ensure adherence to minimum/maximum thresholds and business rules for variety and efficiency.

6.4. ISS Generation

- **Purpose:** To define the Ideal Size Set (ISS) for each selected SKU, optimizing size mix to meet demand and minimize waste.
- **Inputs & Outputs:**
 - **Inputs:** Selected SKUs from OD/OW module, Historical size-level ROS data, Target fill rate, `min_size_count`.
 - **Outputs:** Recommended units per size for each SKU (e.g., S:2, M:4, L:3), Size-level stockout probabilities.
- **Core Logic Summary:**
 1. Analyze historical sales data for each SKU to determine the relative proportion of demand for each size.
 2. Use a statistical model (e.g., normal distribution fitting, optimization algorithm) to calculate the optimal quantity for each size given total forecasted units and desired fill rate.
 3. Factor in minimum size quantities and common pack configurations.
 4. Iteratively adjust size distributions to balance inventory investment across sizes with potential sales loss due to stockouts.

6.5. Space Allocation & Optimization

- **Purpose:** To map the recommended assortment (OD, OW, ISS) onto the physical retail space, considering layout and adjacency.
- **Inputs & Outputs:**
 - **Inputs:** Recommended SKUs with their ISS, Store fixture dimensions and capacities, Adjacency rules (e.g., category A next to B), `space_util_target`.
 - **Outputs:** Detailed planogram for each fixture, Assigned product locations (SKU, size, quantity per location), Space utilization metrics.
- **Core Logic Summary:**
 1. Ingest detailed store layouts and fixture specifications (e.g., linear feet, shelf slots).
 2. Assign primary product categories and Attribute Groups (AGs) to specific store zones or fixtures based on strategic importance and customer flow.
 3. Place individual SKUs within their assigned fixtures, respecting product dimensions, display guidelines, and maintaining sufficient stock units per ISS.
 4. Optimize placement to enhance visual appeal, product discoverability, and cross-selling opportunities, while striving for high `space_util_target`.

6.6. Recommendation Engine

- **Purpose:** To consolidate all module outputs into actionable merchandising plans and insightful reports for stakeholders.
- **Inputs & Outputs:**
 - **Inputs:** Optimized assortment list (OD, OW, ISS), Space allocation maps, Projected performance metrics from previous modules.
 - **Outputs:** `optimized_assortment_plan.csv`, `store_planogram_map.json`, `performance_summary_report.pdf`, `stock_recommendations.csv`, `attribute_group_analysis.xlsx`.
- **Core Logic Summary:**
 1. Aggregate all module-specific recommendations into a unified, comprehensive plan.

2. Generate various output formats tailored to different stakeholder needs (e.g., CSV for data consumption, JSON for visualization, PDF for summaries).
3. Include projected KPIs (e.g., sales, inventory turns, ROS) to demonstrate the impact of the recommendations.
4. Provide clear, easy-to-understand documentation and visualizations for implementation.

7. Worked Example

Scenario: Optimizing the “Men’s Basic T-Shirt” category for Store 001.

Inputs: * **Forecasted Weekly Demand (Men’s Basic T-Shirt, SKU A123):** 100 units * **Historical Size Distribution (SKU A123):** S: 10%, M: 30%, L: 40%, XL: 20% * **Store 001 Capacity (Apparel Section):** 1000 display units * **min_ros_threshold:** 0.5 units/week * **max_od_per_category (Men’s Basic T-Shirts):** 5 * **target_fill_rate:** 0.95 (95% of demand should be met)

Processing Steps:

1. **Data Ingestion & Preprocessing:** Historical ROS for SKU A123 is calculated as 50 units/week over the last 2 weeks (100 units total), making its average ROS 50. This is well above min_ros_threshold of 0.5, so SKU A123 is a candidate.
2. **Demand Forecasting:** Weekly forecasted demand for SKU A123 at Store 001 is 100 units.
3. **OD & OW Calculation:**
 - spacePlanning identifies “Men’s Basic T-Shirts” as a high-performing category, fitting the OW criteria.
 - Within this category, SKU A123 is chosen as one of the OD items. If there were other T-shirt SKUs, up to max_od_per_category (5) would be selected based on their forecasted ROS and profitability.
4. **ISS Generation (for SKU A123, 100 units forecasted):**
 - Applying the historical size distribution to the 100 forecasted units:
 - S: 10 units (10% of 100)
 - M: 30 units (30% of 100)
 - L: 40 units (40% of 100)
 - XL: 20 units (20% of 100)
 - To achieve a target_fill_rate of 0.95 and account for variability, the system might slightly adjust these numbers, potentially adding a buffer to popular sizes or rounding for logistical packaging (e.g., S:12, M:35, L:45, XL:25 = 117 total units recommended for stocking to meet 95% of 100 units demand with 5% safety stock for peak fluctuations).
 - **Resulting ISS:** S: 12, M: 35, L: 45, XL: 25.
5. **Space Allocation:**
 - The 117 units of SKU A123 (and its ISS) are assigned to a specific fixture in the Men’s Apparel section.
 - If a standard shelf can hold 15 folded T-shirts per stack, the system recommends 8 stacks for SKU A123 ($117 \text{ units} / 15 \text{ units/stack} \approx 7.8$, rounded up to 8 stacks for full sizes).
 - The planogram visually shows the placement of S, M, L, XL sizes on the allocated shelves.

6. Recommendation Engine:

- **Optimized Assortment Plan:** Includes SKU A123 for Store 001, with its calculated OD (1 of 5 for category) and its specific ISS.
- **Stock Recommendation:** Detailed breakdown of 12 units of S, 35 units of M, 45 units of L, 25 units of XL for SKU A123 at Store 001.
- **Planogram Map:** Visual representation of where these stacks should be placed on the physical shelves in Store 001.

This example illustrates how the algorithm translates raw data into concrete, actionable stocking and display recommendations to maximize sales and optimize inventory for a single SKU.

8. Debug Guide Highlights

• Data Ingestion Errors:

- **Symptom:** Missing data, incorrect data types, or parsing failures.
- **Troubleshoot:** Verify input file formats (CSV, JSON), check database connections, review data source schemas against expected common-data.json fields.
- **Fix:** Update data connectors, correct source data, or adjust preprocessing scripts.

• Low ROS Recommendations:

- **Symptom:** Algorithm consistently recommends stocking SKUs with very low historical sales.
- **Troubleshoot:** Check `min_ros_threshold` in `args.properties`. Is it set too low? Are there outliers in sales data skewing averages?
- **Fix:** Increase `min_ros_threshold`, implement more robust outlier detection during preprocessing, or adjust demand forecasting parameters.

• Suboptimal ISS Generation:

- **Symptom:** Frequent stockouts on popular sizes or excessive overstock of unpopular sizes for recommended SKUs.
- **Troubleshoot:** Review historical size-level ROS data for accuracy. Check `target_fill_rate` and `min_size_count` parameters. Is demand too volatile for current forecasting?
- **Fix:** Refine size distribution models, increase `target_fill_rate` for critical SKUs, or incorporate more advanced probabilistic forecasting.

• Space Allocation Conflicts:

- **Symptom:** Recommended quantities exceed physical display capacity or violate adjacency rules.
- **Troubleshoot:** Verify `display_capacity` and `fixture_type` data. Check space utilization targets and adjacency rule configurations.
- **Fix:** Update store/fixture capacity data, loosen `space_util_target` if physical constraints are too strict, or refine space allocation algorithms to prioritize critical constraints.

9. Common Pitfalls & Remedies

- **Pitfall 1: Data Silos & Inconsistency:** Data from different systems (POS, Inventory, PIM) may not be harmonized, leading to inaccurate inputs.
 - **Remedy:** Implement a robust ETL (Extract, Transform, Load) process. Establish clear data governance policies and a master data management (MDM) strategy to ensure data consistency across sources.
- **Pitfall 2: Over-reliance on Historical Data:** Solely relying on past sales can be misleading during periods of market change, new product launches, or strong promotions.
 - **Remedy:** Incorporate external factors into demand forecasting (e.g., macroeconomic indicators, competitor activities, social media trends). Implement A/B testing for new product assortments and use cold-start strategies for new SKUs.
- **Pitfall 3: Neglecting Qualitative Factors:** The algorithm is data-driven but may miss nuanced merchandising considerations (e.g., visual aesthetics, brand storytelling, local events).
 - **Remedy:** Introduce mechanisms for human override or qualitative inputs at key decision points. Provide configurable “guardrails” or “must-stock” lists for strategic items. Integrate visual merchandising principles as weighted constraints in space allocation.
- **Pitfall 4: Misinterpreting ROS:** A high ROS doesn’t always mean high profitability if the product has a low margin or high return rate.
 - **Remedy:** Incorporate gross margin contribution and return rates into the SKU selection process for OD/OW, moving beyond just unit sales to “profitable ROS”.
- **Pitfall 5: Scalability Challenges:** As the number of stores, SKUs, and categories grows, the computational complexity can become a bottleneck.
 - **Remedy:** Design the architecture for parallel processing and modularity. Utilize cloud-native computing resources that can scale on demand. Implement efficient data indexing and querying.

10. FAQ

- **Q1: How often should we run the spacePlanning algorithm?**
 - **A:** It depends on your business cycle and product seasonality. For fast-moving categories, weekly or bi-weekly runs are recommended. For seasonal or slower-moving goods, monthly or quarterly might suffice. Review cycles should align with buying and allocation processes.
- **Q2: Can this algorithm handle new product introductions?**
 - **A:** Yes, but with limitations. For new products without historical sales data, the system uses “cold-start” strategies, such as relying on proxy data from similar products (Attribute Group analysis), initial sales projections, or predefined manual overrides. Over time, as sales data accumulates, its recommendations become more accurate.
- **Q3: What if the recommended OD/OW doesn’t fit our existing fixtures?**
 - **A:** The algorithm considers `display_capacity` as an input. If recommendations consistently exceed capacity, it indicates an issue with either the input capacity data, the target fill rate, or the need to reconsider fixture types/quantities in the store. The Space Allocation & Optimization module will try to fit, but if impossible, it will flag overflows.

- **Q4: How does spacePlanning account for promotions or markdown events?**
 - **A:** The Demand Forecasting module is designed to incorporate historical promotional data. Future planned promotions can be fed into the system as an input to adjust forecasts and subsequent OD/OW/ISS calculations, aiming to stock appropriately for anticipated demand spikes.
- **Q5: Is it possible to test different scenarios (e.g., what if we add a new category)?**
 - **A:** Yes, the system can support scenario planning. By modifying input parameters (e.g., max_ow_per_store, adding new product data) and running the algorithm in a simulation mode, users can evaluate the potential impact of strategic changes on performance metrics before actual implementation.

11. Quick Reference Cheat-Sheet

| Term/Parameter | Definition | Key Use Case |
|-----------------------------|---|--|
| OD (Optimum Depth) | Ideal number of unique items (SKUs) to stock in a category or display. | Determines SKU variety within a category. |
| OW (Optimum Width) | Ideal number of different categories or styles to offer. | Shapes the overall product mix of a store/section. |
| ISS (Ideal Size Set) | Best combination of sizes to maximize sales and minimize stockouts. | Optimizes inventory levels at the size level. |
| ROS (Rate of Sale) | Speed at which products are sold over a period (e.g., units/week). | Core metric for demand forecasting & SKU selection. |
| AG (Attribute Group) | Set of product attributes grouped for analysis (e.g., color, size, brand). | Enables analysis and strategy across product groups. |
| min_ros_threshold | Minimum ROS for a SKU to be considered for stocking. | Filters out low-performing SKUs. |
| target_fill_rate | Desired percentage of customer demand to be met. | Influences safety stock and ISS calculations. |
| space_util_target | Target percentage of available space to be utilized. | Guides space allocation efficiency. |
| Inputs | Raw data (Sales, Inventory, Product, Store, Config). | Fuel for the algorithm. |
| Outputs | Actionable plans (Assortment, Planograms, Stock Recs, Performance Reports). | Deliverables for merchandising execution. |

Chapter 1: Story Wise Display Orchestration

Last Updated: 2025-05-23

1. Business Objective

This module orchestrates the entire process of optimizing product displays, ensuring that the right products are in the right places at the right time to maximize sales and customer satisfaction.

Why This Matters: This process directly translates data into actionable display strategies, improving product visibility, reducing lost sales from stockouts of popular items, and enhancing the overall shopping experience by showcasing the most appealing and profitable merchandise. It leads to more efficient inventory investment and higher revenue per square foot of display space.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** To understand how many of each item are selling over a specific period.
- **Rate of Sale (ROS):** To measure the speed at which products are moving, crucial for identifying top performers and avoiding stockouts.

Product Data

- **Product Attributes (Attribute Group - AG):** Details like color, material, and brand, which help classify products and understand what makes them appealing.
- **Category and Style:** To define logical groupings for display planning and ensure a cohesive look.
- **Ideal Size Set (ISS):** The preferred combination of sizes for each product, ensuring customers find their fit.
- **Optimum Depth (OD):** The ideal number of unique items (SKUs) to stock in a category or on a display.
- **Optimum Width (OW):** The ideal number of different categories or styles to offer on a display or in a section.

Operational Data

- **Current Store Inventory Levels:** To know what products are currently available on the sales floor and in the backroom of each store.
- **Warehouse Inventory Levels:** To understand available stock that can be transferred to stores or displays.
- **Display Capacities:** The physical space limitations of each display, including the number of items or styles it can hold.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------|-------------------|---|--|
| Analysis Period | 90 days | The number of past days of sales data to consider for analyzing product performance. | Change if you want to react more quickly to recent trends (shorter period) or account for seasonality (longer period). |
| Minimum Sales Threshold | 5 units | The minimum number of Units Sold a product must achieve within the Analysis Period to be considered for display optimization. | Adjust if you want to be more selective (higher threshold) or more inclusive (lower threshold) of products for display. |
| Sales Consistency Score | 0.8 | A factor that weighs recent sales performance more heavily than older sales in the Analysis Period. | Increase for fast-moving fashion or seasonal items where recent trends are critical; decrease for staple items with stable demand. |
| Display Capacity Limit | Varies by display | The maximum number of items or styles a specific display can physically accommodate. | Update when display fixtures are changed or store layouts are reconfigured. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```
A[Input Data: Sales, Product, Inventory] --> B[Prepare Data]
B --> C[Analyze Sales & Revenue]
C --> D[Rank Products]
D --> E[Allocate to Displays]
E --> F[Generate Display Plan]
F --> G[Output Result: Optimized Display Plan]
```

1. Start with → **Raw Data** (Sales history, product details, inventory levels)
2. Then → **Prepare Data** by cleaning, validating, and structuring it for analysis.
3. Next → **Analyze Sales & Revenue** to understand each product's performance and potential profitability.
4. Next → **Rank Products** based on their performance and potential.
5. Next → **Allocate Products** to specific displays, considering inventory and space limits.
6. Finally → **Generate the Final Display Plan**, including suggested inventory transfers and display arrangements.

5. Step-by-Step Actions

1. **Prepare** all incoming product, sales, and inventory data.
 - What we do: *We gather raw information from various sources and transform it into a consistent format, ready for analysis. This includes validating product attributes and confirming current stock levels.*
 - Why we do it: *This ensures that all subsequent analysis is based on accurate and reliable information, preventing errors in display recommendations.*
2. **Prepare** warehouse inventory data.
 - What we do: *We specifically organize and make accessible the inventory levels from the central warehouse, differentiating it from store-level stock.*
 - Why we do it: *Knowing warehouse availability is crucial for planning product transfers to stores for display replenishment.*
3. **Reserve** necessary warehouse stock.
 - What we do: *We identify and conceptually set aside warehouse inventory for items that are highly recommended for display, preventing them from being allocated elsewhere prematurely.*
 - Why we do it: *This ensures that when a display plan is finalized, the necessary products are truly available for transfer to stores.*
4. **Prepare** sales history for detailed analysis.
 - What we do: *We process the Units Sold and Rate of Sale (ROS) for each product, calculating its performance over the defined Analysis Period.*
 - Why we do it: *This provides the foundation for understanding product popularity and demand, which drives display decisions.*
5. **Compute** potential revenue for each product.
 - What we do: *We estimate the revenue each product could generate if optimally displayed, considering its sales history, margin, and potential for growth.*
 - Why we do it: *This allows us to prioritize products not just by units sold, but by their overall financial contribution.*
6. **Rank** all products.
 - What we do: *We sort products based on their computed revenue potential and other key performance indicators, creating a prioritized list for display consideration.*

- Why we do it: *This ensures that the most impactful products are considered first for prime display space.*
- 7. **Allocate** products to displays.
 - What we do: *We strategically assign specific products and their Ideal Size Sets (ISS) to available display spaces, taking into account Optimum Depth (OD), Optimum Width (OW), current inventory, and display capacities.*
 - Why we do it: *This is where the display plan comes together, matching the right products to the right spaces to maximize visual appeal and sales.*
- 8. **Generate** the final display output.
 - What we do: *We compile all the allocation decisions into a clear, actionable display plan, including specific product placements and recommended inventory transfers.*
 - Why we do it: *This provides stores and inventory teams with the precise instructions needed to implement the optimized display strategy.*
- 9. **Capture** a snapshot of the current state.
 - What we do: *We record the data and parameters used for this specific display optimization run.*
 - Why we do it: *This creates a historical record for review, auditing, and understanding how past recommendations were made.*
- 10. **Upload** the generated output.
 - What we do: *We make the final display plan and associated recommendations available to other systems and teams.*
 - Why we do it: *This ensures that the optimized display strategy can be quickly implemented in stores and integrated with inventory management systems.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A women's clothing store has a prime display rack for "Summer Dresses." The display has space for an Optimum Width (OW) of 5 different styles and an Optimum Depth (OD) of 25 unique items (SKUs) in total.
- Current sales data shows that the "Floral Midi Dress" (Style A) has an excellent Rate of Sale (ROS), selling 320 units over the last 90 days, but the store currently only has 10 units in stock.
- The "Striped Maxi Dress" (Style B) has a much lower ROS, selling only 50 units in the same period, but the store has 45 units in stock.
- The warehouse has 500 units of the "Floral Midi Dress" and 100 units of the "Striped Maxi Dress."
- A new "Linen Shirt Dress" (Style C) was recently introduced and has a promising early ROS, with 80 units sold in 30 days, but is not currently on this display. The warehouse has 200 units of Style C.

What Happens

1. First, the system gathers all sales data for summer dresses, current store inventory, and warehouse inventory. It calculates the potential revenue for Style A, B, and C. Style A shows high revenue potential due to its strong ROS,

- despite low current store stock. Style C also shows good potential due to its promising recent sales. Style B shows low potential.
2. Then, considering the Display Capacity Limit (OW=5, OD=25) and available inventory, the system prioritizes Style A due to its high ranking and available warehouse stock. It suggests transferring more units of Style A to the store to support higher display quantities, including its Ideal Size Set (ISS).
 3. Next, it identifies Style B as a low performer that is overstocked on the display. It recommends reducing its presence to free up valuable display space.
 4. Finally, recognizing Style C's promising performance and available warehouse stock, the system recommends adding Style C to the display to take advantage of its emerging popularity and fill the space freed up by Style B.

Final Outcome

The store receives a new display plan for "Summer Dresses" recommending: - Significantly increasing the display quantity of "Floral Midi Dress" (Style A), suggesting a transfer of 30 units from the warehouse to ensure sufficient stock. - Reducing the display quantity of "Striped Maxi Dress" (Style B), potentially moving excess to storage or another less prominent location. - Introducing the "Linen Shirt Dress" (Style C) to the display with an optimized Ideal Size Set (ISS) and quantity. This optimizes the display's OW (now featuring 5 styles: A, C, and 3 others, replacing B) and OD, ensuring the most profitable and popular items are front and center.

Key Takeaway: By orchestrating data, analysis, and allocation, the system ensures displays always feature the most profitable and in-demand products, leveraging available stock to drive sales and reduce missed opportunities.

7. Results

This step produces:

Main Outputs

- **Optimized Display Plan:** A detailed list of which products (SKUs), in what quantities (including Ideal Size Set - ISS), should be placed on each specific display or category area. This plan includes the suggested Optimum Depth (OD) and Optimum Width (OW) for each display.
- **Product Rankings:** A prioritized list of all products based on their sales performance, potential revenue, and other business criteria.
- **Inventory Transfer Recommendations:** Specific instructions for moving stock from the warehouse to stores to support the optimized display plans.

Business Reports

- **Display Performance Summary:** A report for store managers and merchandisers showing the planned display changes and the expected impact on sales and revenue.
- **Inventory Adjustment Recommendations:** A report for inventory management teams detailing necessary stock transfers and potential replenishment needs.

- **Potential Revenue Forecasts:** A projection used by financial planners to understand the expected sales uplift from optimized display strategies.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed | The system will use the most recent available data or historical averages, but will flag an alert indicating incomplete information. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| No items meet threshold for display | The system will not recommend any new items for the display and may suggest maintaining the current setup or re-evaluating the business settings (like Minimum Sales Threshold). | Review the Minimum Sales Threshold and other filtering settings to ensure they are appropriate. Consider if the product assortment itself needs re-evaluation. |
| Warehouse stock too low for high-ranking items | The system will prioritize allocating available stock but will flag that the full desired quantity cannot be met for popular items. | Expedite reordering or production for these high-demand items. Explore alternative sourcing or communicate potential stockouts to sales teams. |
| Store inventory is already very high for low-performing items | The system will recommend reducing the display presence of these items but might not suggest immediate returns or transfers if store capacity is limited. | Plan promotional activities or markdowns for these items to clear excess stock, and consider future purchasing adjustments. |

9. Next Steps

Before This Step

- **Sales Data Ingestion:** Current and historical Units Sold and Rate of Sale must be accurately captured from Point of Sale (POS) systems. We need reliable and timely sales transaction data.
- **Product Master Data Updates:** Detailed product information, including categories, styles, attributes (AG), Ideal Size Set (ISS), and defined Optimum Depth (OD) and Optimum Width (OW) guidelines, needs to be up-to-date in the Product Information Management (PIM) system.

After This Step

- **Store Operations Implementation:** Store teams need to receive the optimized display plans and physically arrange products according to the recommendations. They will require clear instructions and visual guides.
- **Inventory Management Fulfillment:** Warehouse and logistics teams need to process and execute the recommended inventory transfers to ensure products arrive at stores for display updates.

10. Common Questions

Q: How does “Story Wise Display Orchestration” help increase my store’s sales? A: It ensures your most popular and profitable products are always prominently displayed and well-stocked, making it easier for customers to find and buy what they want, directly leading to higher sales and happier shoppers.

Q: What happens if we don’t have enough stock for the recommended display changes? A: The system works with your available stock, suggesting transfers from the warehouse if possible. If stock is limited, it will still optimize your display based on what you *do* have, while highlighting items that need replenishment.

Q: Can this process adapt to new product launches or sudden trends? A: Yes, by regularly analyzing the most recent sales data (based on the “Analysis Period” setting), the system can quickly identify emerging popular items and suggest their placement on displays to capitalize on new trends.

Q: How often does this display optimization process run? A: It can be configured to run as frequently as needed—daily or weekly—to keep your displays fresh and responsive to changing customer demand and inventory levels.

Q: What kind of information do I need to provide for this to work effectively? A: You need to ensure accurate and up-to-date sales history from your registers, current inventory levels for both stores and warehouses, and complete product details (like category, style, size, and attributes) are consistently fed into the system.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Coordinates the entire process of optimizing product displays to boost sales and customer experience.
 - **Main Input:** Up-to-date sales history, detailed product information, and accurate inventory levels.
 - **Main Output:** Actionable, optimized display plans for stores, including product placements and inventory transfer recommendations.
 - **Warning Signs:** Consistent stockouts of recommended items, or recommendations for products that don’t seem to be selling in your store.
 - **Key Settings:** Analysis Period and Minimum Sales Threshold, which dictate how product performance is evaluated.
-

Need Help? Contact your merchandising operations team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 3: Data Preparation

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for loading, cleaning, and preparing all foundational input data required for the `storyWiseDisplay` allocation process.

Why This Matters: This step ensures that all subsequent analysis and display recommendations are built upon accurate, consistent, and complete information. Without thoroughly prepared data, even the most sophisticated allocation logic can lead to incorrect decisions, resulting in missed sales, inefficient inventory use, and a suboptimal customer experience. It's the critical "garbage in, garbage out" safeguard.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Historical Sales Context:** While raw sales data is processed elsewhere, this module integrates insights derived from past sales to validate if products are currently active and viable for display planning.

Product Data

- **Product Attributes (AG):** Details like color, material, and brand, which help classify products and understand their suitability for different display stories.
- **Category and Style Information:** To define logical groupings for display planning and ensure a cohesive look.
- **Ideal Size Set (ISS) Data:** The preferred combination of sizes for each product, ensuring customers find their fit on display.
- **Optimum Depth (OD) and Optimum Width (OW) Criteria:** Guidelines for the ideal number of unique items and different styles to offer on a display, respectively.
- **Story-Related Inputs:** Crucial definitions like story-style mappings, minimum OW per category within a story, and category combinations, essential for `storyWiseDisplay` logic.
- **Season Information:** To identify which products are currently relevant and in-season for display.

Operational Data

- **Distribution Store Lists:** The specific stores actively included in the display optimization plan.
- **Current Store Inventory Levels:** Accurate counts of products currently available on the sales floor and in the backroom of each store.
- **Warehouse Inventory Levels:** Available stock that can be transferred to stores or displays, factoring in quantities currently in transit.
- **Replenishment Cycle Days:** The frequency at which stores receive new shipments, impacting how much stock needs to be on display.
- **Warehouse Planogram Data:** Information about warehouse capacity and layout that influences product availability for display transfers.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|------------------------------|---------------|--|---|
| Replenishment Season List | Current Year | Specifies the seasons for which products are considered actively in-season for replenishment planning. | Change to focus on new seasons or exclude old ones as product lifecycles evolve, e.g., moving from “Spring” to “Summer.” |
| Read Season List | N/A | Additional seasons whose products should be read into the system, even if not actively replenished. | Use to include data from past seasons for trend analysis or broader historical context without affecting active replenishment. |
| Include Store Stock | True | If enabled, current inventory at stores is loaded and factored into display availability. | Disable only if you wish to plan displays without considering immediate on-hand store stock, perhaps for planning future allocations. |
| Include In-Transit Inventory | True | If enabled, goods already shipped to stores or warehouses are counted as available stock for display planning. | Disable if you only want to plan with currently physical on-hand stock and disregard pending arrivals. |

Note: See More Details for additional settings.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales, Product, Inventory Data] --> B[Clean & Structure Data]

B --> C[Load Foundational Inputs]

C --> D[Prepared Data for Analysis]

1. Start with → **Raw Data** from various enterprise systems (e.g., sales transaction records, product details from Product Information Management, and inventory levels from Warehouse Management Systems).
2. Then → **Clean & Structure Data** by standardizing formats, validating entries, and filtering out any irrelevant information that might impede accurate display planning.
3. Next → **Load Foundational Inputs**, bringing in all critical data points like current store and warehouse inventory, active store lists, season information, and detailed “story” definitions.
4. Finally → **Prepared Data for Analysis**, which is a comprehensive, consistent, and accurate dataset ready for the storyWiseDisplay allocation process in subsequent steps.

5. Step-by-Step Actions

1. **Identify** active stores and seasons.
 - What we do: *We compile a list of all stores enabled for display optimization and determine which product seasons are currently active and relevant for display planning.*
 - Why we do it: *This ensures that display plans are generated only for the correct retail locations and for products that are currently in-season and desirable to customers.*
2. **Gather** current inventory levels.
 - What we do: *We collect up-to-date stock counts from both store floors, backrooms, and central warehouses, including items that are currently in transit to stores or between warehouses.*
 - Why we do it: *Accurate and comprehensive inventory data is crucial to ensure that display recommendations are realistic, can be fulfilled, and prevent stockouts of popular items.*
3. **Define** replenishment and planogram details.
 - What we do: *We integrate information about how frequently stores are restocked and the physical layout and capacity of warehouse storage spaces, including replenishment cycle days.*
 - Why we do it: *This contextual data helps the system understand logistical constraints and opportunities for efficient product movement and display replenishment.*
4. **Load** Ideal Size Set (ISS) and Optimum Width (OW) criteria.
 - What we do: *We import pre-defined guidelines for the best size combinations (ISS) for each product and the ideal number of different styles (OW) to be presented on various displays.*
 - Why we do it: *These criteria are fundamental for creating customer-friendly displays that offer the right choices and maximize sales potential.*

5. **Configure** “story” definitions.

- What we do: *We load essential details that define each product “story” or merchandising theme, including which styles belong to which story, minimum OW per category within a story, and allowed category combinations.*
- Why we do it: *This ensures that display allocations align perfectly with the brand’s merchandising narratives and visual themes, enhancing the overall shopping experience.*

6. **Establish** product depth criteria.

- What we do: *We set rules for the maximum or minimum number of unique items (SKUs) that should be stocked for specific products or categories on a display.*
- Why we do it: *This helps manage inventory efficiency on the display, ensuring products are neither overstocked (tying up capital) nor understocked (leading to lost sales).*

7. **Consolidate** all product attributes.

- What we do: *We organize and prepare detailed product attributes like color, material, and brand to support coherent display groupings and analysis.*
- Why we do it: *Consistent and complete product data is vital for accurate classification and effective visual merchandising decisions later in the process.*

6. Sample Scenario

A women’s fashion retailer is preparing for its “Summer Escape” story display, which features vacation-themed apparel and accessories.

Starting Point

- The retailer has 100 stores, but only 50 are currently designated as “distribution stores” for this specific allocation run.
- The Replenishment Season List is set to “Summer 2025,” and “Spring 2025” is included in the Read Season List for historical sales context.
- Current store inventory shows 15 “Tropical Print Maxi Dresses” in Store A, but 0 in Store B.
- The central warehouse has 500 “Tropical Print Maxi Dresses.” Additionally, there are 20 units of the same dress currently “in-transit” to Store B.
- ISS data indicates the best sizes for the “Tropical Print Maxi Dress” are Small, Medium, Large, and Extra Large in a 2:3:3:2 ratio.
- OW data for the “Summer Escape” story suggests 5 different styles are ideal for the main display.
- Story-style mapping explicitly links “Tropical Print Maxi Dress” and “Linen Jumpsuit” to the “Summer Escape” story.
- Minimum OW for the ‘Dresses’ category within “Summer Escape” is set to 3.

What Happens

1. First, the system identifies the 50 designated distribution stores and confirms that “Summer 2025” products are in-season for display planning. It also incorporates relevant “Spring 2025” styles for broader data context.

2. Then, it gathers all current inventory data: Store A's 15 dresses, Store B's 0 current stock but registering the 20 units currently in transit, and the 500 units in the warehouse. This confirms the "Tropical Print Maxi Dress" as a 'live' and available style because there's inventory.
3. Next, the system loads the ISS data for the dress (S, M, L, XL at 2:3:3:2 ratio) and the OW criteria for the "Summer Escape" display (ideal for 5 different styles).
4. Finally, it incorporates the "story" specific inputs: explicitly tagging the "Tropical Print Maxi Dress" and "Linen Jumpsuit" as part of the "Summer Escape" story, setting the minimum 3 styles for the 'Dresses' category within this story, and understanding how "Dresses" can combine with "Swimwear" for the overall theme.

Final Outcome

All necessary foundational data for the "Summer Escape" display story is accurately loaded, cleaned, and cross-referenced. The `storyWiseDisplay` system now comprehensively knows which stores to plan for, which products are relevant, their exact stock levels (including pending deliveries), the ideal size mixes and style breadth, and how each product contributes to defined merchandising stories. This comprehensive, validated dataset is now ready for the system to analyze sales trends, rank products, and make precise, data-driven allocation decisions.

Key Takeaway: By meticulously preparing all underlying data, this module creates a robust and accurate foundation, enabling `storyWiseDisplay` to make informed, data-driven merchandising decisions that align with both inventory realities and creative display narratives.

7. Results

This step produces:

Main Outputs

- **Validated Foundational Dataset:** A comprehensive collection of all input data (inventory, store lists, product attributes, story definitions) that has been cleaned, structured, and validated, ready for subsequent analysis modules.
- **Live Product and Store Flags:** Indicators for which products are considered "live" (in-season, with inventory) and which stores are enabled for display optimization.
- **Pre-computed Criteria Maps:** Organized data structures (e.g., for ISS, OD, OW, replenishment cycles, warehouse planograms) that allow other modules to quickly access critical display guidelines and thresholds.

Business Reports

- **Data Quality Report:** A summary for data governance teams, highlighting any inconsistencies, missing information, or warnings identified during the data loading process.

- **Input Data Snapshot:** A detailed record of all the raw data used for a specific allocation run, useful for auditing, troubleshooting, and understanding historical decisions.
- **Active Store & Season List:** A clear list for merchandising and operations teams showing exactly which stores and product seasons were included in the current display planning cycle.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed | The system will use the most recent available data or historical averages for that data point, but will flag an alert indicating incomplete information, potentially impacting accuracy. | Contact the data source team immediately to resolve the feed issue. If critical, manually verify the data points to ensure timely planning. |
| Inconsistent Product Attributes (AG) | If product attributes are miscategorized (e.g., a “T-shirt” is incorrectly listed in the “Pants” category), this module will load them as-is, which could lead to illogical display recommendations later. | Implement robust data validation checks at the source. Conduct regular audits of product master data to ensure accuracy and consistency. |
| Warehouse stock is lower than expected for key products | The module will load the actual low stock numbers. Subsequent allocation steps might then struggle to find enough stock for high-ranking items, even if they are top sellers. | Review warehouse operations and procurement processes. Investigate reasons for stock discrepancies and adjust future purchasing forecasts. |
| Story-style mappings are incomplete or incorrect | Products intended for a specific “story” might not be linked, or incorrect products might be associated, leading to misaligned display suggestions that don’t match marketing themes. | Regularly review and update story definitions and product associations. Ensure clear communication between marketing, merchandising, and data teams. |
| No stores found in distribution list | The system will identify an empty distribution list and halt processing for display allocation, as there are | Verify that the Distribution Store List input data is correct and includes all intended stores that should be enabled for display optimization. |

| Situation | What Happens | What You Should Do |
|-----------|-------------------------------|--------------------|
| | no target stores to plan for. | |

9. Next Steps

Before This Step

- **Sales Data Ingestion:** Current and historical Units Sold and Rate of Sale must be accurately captured from Point of Sale (POS) systems and prepared for use by downstream modules.
- **Product Master Data Updates:** Detailed product information, including categories, styles, attributes (AG), Ideal Size Set (ISS) guidelines, and Optimum Depth (OD)/Optimum Width (OW) criteria, needs to be consistently up-to-date in the Product Information Management (PIM) system.
- **Inventory System Synchronization:** Accurate, real-time inventory levels from both store systems and the Warehouse Management System (WMS) must be available and refreshed regularly.

After This Step

- **Sales and Revenue Analysis:** The prepared data is fed into the next module to analyze product performance, calculate Rate of Sale (ROS), and estimate potential revenue for each item.
- **Product Ranking:** Products are then prioritized based on their performance and alignment with display objectives, utilizing the prepared foundational data to determine their importance.
- **Display Allocation:** The system uses the prepared data and rankings to strategically assign products to specific display spaces, considering all defined constraints and objectives to create the final display plan.

10. Common Questions

Q: Why is data preparation so important? I just want my display plan! A: Data preparation is the absolutely critical foundation. It's like cooking: if you don't start with fresh, well-prepared ingredients, the final dish won't be good. Accurate and complete data ensures the display plan is based on reality, leading to better sales and fewer stockouts, saving you time and money.

Q: What if our inventory numbers aren't perfectly accurate when this runs? A: The system will use the inventory numbers it receives. If they're inaccurate, the display recommendations might be off – suggesting products you don't actually have, or missing opportunities for products you do. Maintaining good, accurate inventory records is essential for the system's effectiveness.

Q: How does this module know which products belong to which “story”? A: This module specifically loads pre-defined “story” inputs, which are mappings of styles to specific display themes. These mappings are typically set up by your merchandising or marketing teams to align with your seasonal campaigns and visual strategies.

Q: Can I change what data is included, like excluding products that are in-transit? A: Yes, there are specific business settings for this module, such as the “Include In-Transit Inventory” flag. These settings allow you to customize what data points are factored into the preparation process, giving you control over the scope of the display plan.

Q: What happens if a store is missing from the “Distribution Store List”? A: If a store isn’t on this crucial list, this module will not prepare data for it, and consequently, it will be excluded from the `storyWiseDisplay` allocation process for that run. Always ensure your distribution lists are comprehensive and up-to-date for all intended stores.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Loads, cleans, and structures all foundational input data for `storyWiseDisplay` calculations.
- **Main Input:** Store lists, season information, current inventory (store and warehouse), ISS, OD, OW criteria, and crucial “story” definitions.
- **Main Output:** A comprehensive, validated dataset that is ready for detailed display analysis and product allocation.
- **Warning Signs:** Inaccurate or illogical display recommendations in later stages often point to underlying issues with the raw data loaded and prepared in this foundational step.
- **Key Settings:** `Replenishment Season List` and the `Include In-Transit Inventory` flag, which directly influence what data is considered.

Need Help? Contact your data operations team or merchandising analytics team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 4: Warehouse Inventory Preparation

Last Updated: 2025-05-23

1. Business Objective

This module precisely adjusts warehouse inventory by accounting for operational inefficiencies and reserving necessary stock for future online sales demand, ensuring that the allocation process only considers truly available and ready-to-ship products.

Why This Matters: This crucial step prevents over-allocation of products, especially fast-moving items, that might be hard to locate in a busy warehouse or are already earmarked for immediate online fulfillment. By providing an accurate picture of *allocable* inventory, it reduces customer disappointment from canceled orders, improves inventory turnover, and ensures that display recommendations are based on realistic stock levels, ultimately maximizing sales and profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** To understand the recent sales velocity of products, particularly for online channels, which helps forecast demand for lead time reservations.
- **Rate of Sale (ROS):** To measure how quickly products are selling, allowing the system to accurately predict how much stock needs to be reserved for upcoming online orders.

Product Data

- **Product Attributes (AG):** Details like style and category, which help group products for inventory analysis and ensure correct lead time calculations apply to relevant items.
- **Style and Category:** To define logical product groupings for accurate lead time calculation and inventory management specific to certain product types.

Operational Data

- **Warehouse Inventory Levels:** Accurate, real-time counts of products currently in the central warehouse, which is the base stock adjusted by this module.

- **Store Information (Online/Physical):** To differentiate between online-only stores (which have lead time requirements for fulfillment) and physical stores, as lead time reservations specifically apply to online sales.
- **Warehouse Type (Online/Physical):** To identify which warehouses primarily fulfill online orders, ensuring that lead time reservations are applied to the correct stock locations.
- **Lead Time:** The number of days required to process and ship an online order from the warehouse, which directly determines how much fast-moving stock needs to be reserved.
- **Warehouse Inefficiency Factor:** A numerical value representing a small percentage of stock that, despite being physically present, is often difficult to locate or allocate due to general warehouse operational hurdles.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------------|---------------|--|--|
| Warehouse Inefficiency Factor | 5 units | A small number of units per SKU to subtract from total warehouse stock, accounting for items that are physically present but often hard to allocate due to typical warehouse operations. | Increase if you frequently find discrepancies between system stock and physically allocable stock; decrease if your warehouse operations are highly efficient. |
| Lead Time Reservation Period | 3 days | The number of days into the future for which online sales demand should be reserved from warehouse stock. | Adjust based on your typical order fulfillment time for online sales; increase if shipping times are longer or if you anticipate a surge in online orders. |
| Sales Look-back Period for Lead Time | 30 days | The period of past sales data (in days) used to calculate the Rate of Sale (ROS) for reserving inventory for online orders. | Shorten for fast-fashion or highly seasonal items where recent trends are critical; lengthen for staple products with stable demand. |
| | 7 days | The minimum number of live | Increase to only reserve stock for |

| Setting Name | Default Value | Description | When to Change |
|--|---------------|---|--|
| Minimum Sales Days for ROS Calculation | | sales days a product must have within the Sales Look-back Period for Lead Time to be included in ROS calculations for lead time reservations. | items with consistent recent online sales; decrease to be more inclusive of new or sporadically selling items. |

Note: See More Details for additional settings related to multi-period ROS weighting.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Warehouse Inventory & Online Sales Data] --> B[Adjust for Warehouse Inefficiency]

B --> C[Reserve for Online Lead Time]

C --> D[Prepared Warehouse Inventory for Allocation]

1. Start with → **Raw Warehouse Inventory** levels and **Online Sales Data**.
2. Then → **Adjust for Warehouse Inefficiency** by subtracting a small quantity from each product's stock.
3. Next → **Reserve for Online Lead Time** by calculating potential online sales over the lead time and setting aside that quantity.
4. Finally → **Prepared Warehouse Inventory for Allocation**, which represents the net stock available for distribution to stores and displays.

5. Step-by-Step Actions

1. **Adjust** warehouse inventory for inefficiency.
 - What we do: *We reduce the reported quantity of each SKU in the warehouse by a small, predefined Warehouse Inefficiency Factor.*
 - Why we do it: *This accounts for the practical reality that not all stock is immediately available or easy to locate in a large warehouse, preventing over-allocation based on theoretical counts.*
2. **Identify** online warehouses and stores.
 - What we do: *We determine which warehouses primarily serve online sales and which stores are designated as online-only channels.*
 - Why we do it: *Lead time inventory reservations are only applicable to stock intended for online fulfillment, as physical stores have different replenishment cycles.*

3. **Calculate** online Rate of Sale (ROS) for lead time.
 - What we do: *For each SKU with online sales, we analyze its Units Sold over the specified Sales Look-back Period for Lead Time to determine its average daily ROS for online channels.*
 - Why we do it: *An accurate ROS is essential for forecasting immediate future demand for online sales, ensuring that fast-moving items aren't accidentally allocated away from online orders.*
4. **Reserve** inventory for lead time.
 - What we do: *Based on the calculated online ROS and the Lead Time Reservation Period, we conceptually set aside a calculated quantity of each SKU in the online-serving warehouses.*
 - Why we do it: *This proactively holds inventory for expected online orders during the order processing and shipping window, preventing stockouts for web customers and ensuring a smoother fulfillment process.*
5. **Update** available warehouse stock.
 - What we do: *After applying both the inefficiency adjustment and the lead time reservation, the remaining quantity for each SKU in each warehouse is considered the allocable stock.*
 - Why we do it: *This final adjusted number is the realistic inventory figure passed to subsequent allocation modules, ensuring that all display and store replenishment plans are actionable and based on truly available stock.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular "Women's Denim Jacket" (SKU 12345) has 1,000 units reported in the main warehouse.
- The Warehouse Inefficiency Factor is set to 5 units.
- The Lead Time Reservation Period is 3 days.
- The Sales Look-back Period for Lead Time is 30 days.
- Over the last 30 days, online sales for SKU 12345 were 300 units.
- The main warehouse is designated as an online fulfillment warehouse.

What Happens

1. First, the system takes the initial 1,000 units of "Women's Denim Jacket" in the warehouse. It then applies the Warehouse Inefficiency Factor of 5 units. The *theoretically* allocable stock becomes $1,000 - 5 = 995$ units.
2. Next, the system calculates the online Rate of Sale (ROS) for SKU 12345. Over 30 days, 300 units were sold, so the ROS is $300 \text{ units} / 30 \text{ days} = 10 \text{ units per day}$.
3. Then, to account for the Lead Time Reservation Period of 3 days, the system calculates the quantity to reserve: $10 \text{ units/day} * 3 \text{ days} = 30 \text{ units}$.
4. Finally, this 30 units is conceptually reserved from the inefficiency-adjusted stock. The total allocable stock for SKU 12345 becomes $995 \text{ units} - 30 \text{ units} = 965 \text{ units}$.

Final Outcome

The storyWiseDisplay system receives an adjusted warehouse inventory level of 965 units for the “Women’s Denim Jacket.” This means that even though 1,000 units were initially reported, only 965 are considered truly available for new allocations to stores or displays, preventing the potential over-allocation of 35 units that might otherwise be hard to locate or are already slated for online customer orders.

Key Takeaway: By proactively adjusting for operational realities and online demand, this module ensures that display and store allocation plans are always based on a realistic and actionable supply of products, minimizing disruptions and maximizing customer satisfaction.

7. Results

This step produces:

Main Outputs

- **Adjusted Warehouse Inventory Levels:** A refined list of each SKU’s quantity in the warehouse, reflecting deductions for inefficiency and lead time reservations, ready for subsequent allocation processes.
- **Reserved Inventory Report:** A detailed breakdown of which SKUs and quantities have been reserved for online sales fulfillment due to lead times, providing transparency for inventory teams.
- **Allocable Inventory Count:** The precise number of units of each SKU that can be confidently allocated to physical stores or displays.

Business Reports

- **Warehouse Capacity & Availability Report:** A summary for warehouse operations managers showing the net allocable stock versus total reported stock, highlighting inventory that is “held” due to inefficiencies or online demand.
- **Online Fulfillment Buffer Report:** A specialized report for e-commerce and logistics teams, confirming the amount of inventory reserved to meet projected online sales during the lead time.
- **Inventory Accuracy Discrepancy Insight:** Data that can be used by inventory auditing teams to understand the impact of the Warehouse Inefficiency Factor and drive process improvements.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The system will use the most recent available data or historical averages for online sales, but will flag an alert indicating | Contact the data source team immediately to resolve the feed issue and manually verify critical sales data points if the delay persists. |

| Situation | What Happens | What You Should Do |
|---|--|--|
| Warehouse Inefficiency Factor is too high | <p>incomplete information, potentially leading to inaccurate lead time reservations.</p> <p>If set too high, the system might deduct a significant portion of stock, potentially making many SKUs appear unallocable even if they are physically present, leading to missed sales opportunities in stores.</p> | <p>Regularly review and adjust the Warehouse Inefficiency Factor based on actual warehouse performance and audit results. Conduct root cause analysis for persistent “hard to locate” stock.</p> |
| No online sales data for a fast-moving item | <p>If a popular item has no recorded online sales (perhaps due to a system error or it’s new), the system won’t reserve any inventory for lead time, potentially leading to stockouts for online customers.</p> | <p>Manually review fast-moving new products for which lead time reservation might be critical. Investigate and correct any issues with online sales data capture.</p> |
| Warehouse wrongly flagged as “online fulfillment” | <p>Inventory in a warehouse that <i>doesn’t</i> fulfill online orders might have stock unnecessarily reserved, reducing available inventory for physical store allocations.</p> | <p>Ensure accurate classification of warehouses (online vs. physical fulfillment) in master data to avoid incorrect inventory reservations.</p> |
| Online ROS is zero for all products | <p>If online ROS calculations consistently result in zero (due to insufficient sales or data issues), no inventory will be reserved for lead time, potentially leading to immediate online stockouts during peak demand.</p> | <p>Verify the Sales Look-back Period for Lead Time and Minimum Sales Days for ROS Calculation settings. Check for data quality issues in online sales feeds.</p> |

9. Next Steps

Before This Step

- **Data Preparation (Chapter 3):** All foundational input data, including raw warehouse inventory levels, store and warehouse classifications, and product attributes, must be accurately loaded, cleaned, and validated.
- **Sales Data Ingestion:** Current and historical Units Sold and Rate of Sale (ROS) for online channels must be accurately captured from Point of Sale (POS) or e-commerce systems.

After This Step

- **Product Ranking:** The adjusted warehouse inventory levels are passed to the product ranking module, which will then prioritize products for display based on their performance and actual allocable stock.
- **Display Allocation:** The system uses the now-realistic warehouse stock levels to make precise allocation decisions, ensuring that only truly available products are recommended for transfer to stores and displays.

10. Common Questions

Q: Why do we reduce warehouse stock for “inefficiency”? Isn’t that just losing product? A: It’s not about losing product, but about being realistic. In large warehouses, a small percentage of items can be hard to find or access immediately. By subtracting a Warehouse Inefficiency Factor, we prevent our display plans from asking for stock that, in practice, isn’t readily available, which avoids frustrating “phantom stockouts” and ensures our plans are always achievable.

Q: How does reserving inventory for “lead time” help my business? A: When a customer places an online order, it takes time for the warehouse to pick, pack, and ship it. During this Lead Time Reservation Period, other systems might try to allocate that same fast-selling item to a physical store display. Reserving it ensures that your online orders are fulfilled first, preventing customer cancellations and maintaining a strong online customer experience, which directly impacts your brand reputation and repeat sales.

Q: What if a product sells very quickly online after this process runs? A: This module uses recent Rate of Sale (ROS) to make its reservation. If a product’s sales suddenly surge *after* the process runs, it’s possible the reserved amount might be insufficient. Regular re-runs of this process (e.g., daily) and setting a slightly more generous Lead Time Reservation Period can help mitigate this risk.

Q: Does this process affect the inventory available for my physical stores? A: Yes, it does. By adjusting and reserving stock in the warehouse, this module provides the *net available* inventory that can then be allocated to physical stores for displays or backroom stock. This ensures that when a store receives a recommendation, the product is genuinely available in the warehouse to be shipped.

Q: Can I turn off the lead time reservation if I don't have online sales? A: The system automatically checks if online stores are configured. If no online stores are present, or if the relevant warehouses don't fulfill online orders, the lead time reservation logic will not apply, and stock will only be adjusted for warehouse inefficiency.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Adjusts raw warehouse inventory by accounting for operational inefficiencies and reserving stock for future online sales based on Lead Time.
- **Main Input:** Raw warehouse inventory levels and online Units Sold data.
- **Main Output:** Realistic, *allocable* warehouse inventory levels for storyWiseDisplay and a report of reserved stock.
- **Warning Signs:** Frequent online order cancellations due to lack of stock, or physical stores consistently unable to receive recommended transfers due to warehouse discrepancies.
- **Key Settings:** Warehouse Inefficiency Factor, Lead Time Reservation Period, and Sales Look-back Period for Lead Time.

Need Help? Contact your inventory planning or warehouse operations team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 5: Warehouse Reserve Management

Last Updated: 2025-05-23

1. Business Objective

This module ensures that a specific percentage of warehouse inventory for certain styles is strategically reserved for particular sales channels or future initiatives, preventing its accidental allocation to general store displays.

Why This Matters: This proactively protects valuable inventory for high-priority uses, such as online exclusive collections, planned marketing promotions, or specific wholesale orders. By guaranteeing availability for these strategic needs, it prevents missed sales opportunities, supports differentiated channel strategies, and avoids disappointing customers or partners. Ultimately, it helps maintain brand promises and optimizes long-term revenue streams.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Not directly consumed by this module for calculations, but informs strategic decisions on reserve percentages:** *While this module doesn't use sales data to calculate reservations, understanding past sales helps merchandising and sales teams decide which styles need strategic reservations and at what percentages.*

Product Data

- **Style Information:** *To identify the specific items (e.g., a "Limited Edition Sneaker") that need to have a percentage of their stock reserved based on business rules.*
- **Product Attributes (AG):** *To help categorize styles (e.g., 'Spring Collection', 'Luxury Tier') for broader reservation rules or specific initiatives.*

Operational Data

- **Warehouse Inventory Levels:** *The total current stock in the central warehouse from which a specific percentage will be conceptually set aside.*

- **Channel Classification:** *To differentiate between various sales channels (e.g., online, wholesale, retail stores) and ensure reservations are applied only to the intended ones.*
- **Channel-Specific Reserve Settings:** *Pre-defined percentages for specific styles or style categories that must be reserved for particular channels or future initiatives (e.g., “reserve 15% of all new arrivals for e-commerce”).*

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------------------|-----------------|--|---|
| Default Channel Reserve Percentage | 0% | A baseline percentage of available stock to reserve for any style within a specific channel if no style-specific override is set. | Adjust if you want a general safety buffer for common items in a channel, or if a channel frequently has unexpected, high-priority demands. |
| Style-Specific Reserve Overrides | Varies by style | Allows merchandising teams to set precise reserve percentages for individual styles, overriding any default channel percentages. | Update when planning exclusive product launches, limited editions, or specific promotions where precise stock control is vital for certain items. |
| Active Channels for Reservation | All | Determines which sales channels (e.g., E-commerce, Retail Flagship Stores) are currently configured to have stock reserved according to this module's rules. | Change when new sales channels are introduced, or if a channel's strategic importance shifts and requires dedicated inventory protection. |
| Reservation Status Review Frequency | Daily | How often the system re-evaluates and applies stock reservations based on updated channel strategies or | Increase for fast-moving fashion or highly dynamic product lines; decrease for staple items with stable, long-term reservation needs. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|-----------------------|----------------|
| | | style
performance. | |

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Raw Warehouse Inventory & Style Reservation Settings] -->
    B[Identify Styles for Reservation]
    B --> C[Apply Percentage-Based Reservation]
    C --> D[Adjusted Warehouse Inventory for Allocation]
  
```

1. Start with → **Raw Warehouse Inventory** levels (already adjusted for operational inefficiencies and online lead times by previous steps) and **Pre-defined Style Reservation Settings**.
2. Then → **Identify Styles for Reservation** by matching active styles with their respective channels and the associated reservation percentages.
3. Next → **Apply Percentage-Based Reservation** by calculating the specific quantity to hold back for each identified style.
4. Finally → **Adjusted Warehouse Inventory for Allocation**, which represents the net stock truly available for distribution to physical stores and general displays.

5. Step-by-Step Actions

1. **Gather** pre-defined style reservation percentages.
 - What we do: *We retrieve the specific percentages set by your merchandising or sales teams for reserving stock for various styles, channels, or future initiatives.*
 - Why we do it: *This ensures we apply the precise reservation rules that align with your business strategy and commitments.*
2. **Filter** for active styles and channels.
 - What we do: *We identify which products are currently active and available for distribution, and which sales channels are relevant for applying these stock reservations.*
 - Why we do it: *This prevents reserving stock for outdated products or channels that are no longer part of your active sales network, ensuring efficiency and accuracy.*
3. **Calculate** reservation quantities.
 - What we do: *For each identified product style, we apply its configured reservation percentage to its total available warehouse stock to determine the exact number of units to conceptually hold back.*
 - Why we do it: *This step translates the percentage rule into a concrete quantity, ensuring the exact amount of stock is set aside for its strategic purpose.*
4. **Adjust** warehouse inventory levels.
 - What we do: *We deduct the calculated reservation quantities from the total available warehouse stock for each style.*

- Why we do it: *This provides an accurate, real-time picture of truly available (allocable) inventory, preventing other modules from mistakenly distributing stock that is already committed to strategic initiatives.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A new “Limited Edition Sneaker” (Style X) has 1,000 units reported in the warehouse (after Chapter 4's adjustments).
- Merchandising has decided to reserve 20% of its stock for “Online Exclusive Sales” (an E-commerce channel initiative) to ensure a strong web launch.
- Simultaneously, the “Classic White Tee” (Style Y) has 5,000 units in the warehouse, and a company policy dictates reserving 5% of all core items for “Wholesale Partner Orders” (a Wholesale channel initiative).
- Both the E-commerce and Wholesale channels are actively configured for stock reservations.

What Happens

1. First, the system identifies the “Limited Edition Sneaker” (Style X) and its specific 20% reservation rule for the “Online Exclusive Sales” channel. It calculates 20% of the 1,000 units, which equals 200 units, to be reserved.
2. Next, it identifies the “Classic White Tee” (Style Y) and its 5% reservation rule for the “Wholesale Partner Orders” channel. It calculates 5% of the 5,000 units, which equals 250 units, to be reserved.
3. These quantities (200 units of Style X and 250 units of Style Y) are then conceptually set aside from the total available warehouse inventory.

Final Outcome

The storyWiseDisplay system now considers an adjusted warehouse inventory of 800 units for the “Limited Edition Sneaker” ($1000 - 200$) and 4,750 units for the “Classic White Tee” ($5000 - 250$). This ensures that while these highly desired items are still available for general distribution, a specific portion is protected and guaranteed for their strategic channel commitments, preventing any accidental over-allocation to physical stores or regular displays.

Key Takeaway: By precisely applying percentage-based reservations, this module guarantees that strategic inventory needs are met, protecting stock for key initiatives and channels while ensuring accurate allocable quantities for all other distribution efforts.

7. Results

This step produces:

Main Outputs

- **Reserved Inventory Quantities:** A detailed breakdown of which styles, and how many units of each, have been specifically reserved based on their defined percentages and channels.
- **Adjusted Allocable Warehouse Inventory:** The net remaining quantity of each style in the warehouse that is truly available for general distribution to stores or other channels.
- **Style Reservation Status Map:** An internal record indicating which styles have active reservations applied and for which channels.

Business Reports

- **Strategic Inventory Hold Report:** A report for merchandising and sales teams detailing the inventory held back for specific online exclusives, wholesale commitments, or future promotions, providing transparent visibility into strategic stock allocation.
- **Available-to-Promise Summary:** A report for inventory and fulfillment teams showing the real-time quantity of products truly available for new orders or transfers after all reservations.
- **Channel Fulfillment Health Check:** An aggregate report for channel managers, indicating the percentage of key styles successfully reserved for their specific requirements, ensuring their initiatives are well-supported.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|---|
| Missing or delayed data feed for reservation settings | The system will use the most recent available reservation settings, potentially leading to outdated or incorrect reservations for new initiatives or changes in strategy. | Contact the data source team immediately to resolve the feed issue. Manually verify critical reservation settings for key styles or channels to ensure timely planning. |
| Warehouse stock is lower than the calculated reservation | If the total warehouse stock for a style is less than the calculated percentage to reserve, the system will reserve all available stock for that purpose, leaving nothing for other allocations. | Review reservation percentages for extremely low-stock items. Expedite reordering or production for these critical styles to balance strategic holds with general availability. |
| A style is flagged for reservation but has no inventory | The system will register the reservation requirement but will simply reserve 0 units, as there's no physical stock. The style will not | Investigate why a strategically reserved style has no inventory. Prioritize replenishment if it's a key item for ongoing or future initiatives. |

| Situation | What Happens | What You Should Do |
|--|--|---|
| Incorrect channel classification for a style | <p>appear as allocable for general distribution.</p> <p>If a style is wrongly categorized for a channel, its stock might be unnecessarily reserved (or not reserved when it should be), leading to misallocation of inventory resources.</p> | Regularly audit and verify style-to-channel mappings and product attributes to ensure accurate application of reservation rules across all products. |
| Reservation percentage set to 100% for a popular style | <p>If 100% of a popular style's stock is reserved, it will prevent any of that style from being allocated to physical stores, potentially leading to significant missed in-store sales opportunities.</p> | Review and approve all 100% reservation settings with extreme caution, ensuring this aligns perfectly with the overall business strategy and sales goals. |

9. Next Steps

Before This Step

- **Data Preparation (Chapter 3):** Raw warehouse inventory levels, product style information, and channel classifications must be accurately loaded, cleaned, and validated by the Data Preparation module.
- **Warehouse Inventory Preparation (Chapter 4):** Warehouse inventory levels must have already been adjusted for operational inefficiency (e.g., hard-to-find items) and lead time for online sales. This module then applies further percentage-based reservations on that “ready-to-allocate” base stock.

After This Step

- **Product Ranking:** The refined, truly allocable warehouse inventory levels are passed to the product ranking module. This ensures that product prioritization for display is based on realistic stock availability, excluding what's strategically held back.
- **Display Allocation:** The system uses these precisely adjusted inventory levels to make final allocation decisions. This guarantees that only genuinely available products are recommended for transfer to stores and displays, without encroaching on strategically reserved stock.

10. Common Questions

Q: Why do we need to reserve stock *again* after Chapter 4 already prepared warehouse inventory? A: Chapter 4 reserves stock for *operational* reasons, like accounting for hard-to-find items or immediate online order fulfillment. This module, “Warehouse Reserve Management,” reserves stock for *strategic* business reasons, such as ensuring enough inventory for a special online launch or a specific wholesale partner, independent of daily sales velocity. It’s about protecting stock for planned initiatives.

Q: How do we decide which styles or channels get stock reserved? A: Those decisions are typically made by your merchandising, marketing, or sales teams. They define these rules based on strategic initiatives, like launching a new online-exclusive collection, fulfilling a large wholesale order, or supporting a future marketing campaign. The settings in this module allow you to implement these defined rules.

Q: What happens if we reserve too much stock for a strategic purpose? A: Reserving an excessive amount of stock can lead to physical stores having insufficient product to display or sell, potentially causing lost sales and reducing overall inventory turnover. It’s crucial to balance strategic reservations with the need to keep physical stores adequately stocked with popular items to maximize overall revenue.

Q: Can these reservations change frequently? A: Yes, the reservation percentages and applicable styles/channels can be updated as often as business needs change, such as when a new marketing campaign starts or a channel’s priority shifts. The “Reservation Status Review Frequency” setting determines how quickly the system reacts to these updates.

Q: How does this module help us manage our new product launches? A: This module is vital for successful product launches because it allows you to set aside a specific percentage of new inventory. This ensures that your online store or specific flagship locations have guaranteed stock for launch day, preventing them from selling out prematurely due to general store allocations and creating a strong initial impression.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Manages the strategic reservation of a specific percentage of warehouse inventory for defined styles and channels, preventing general allocation.
 - **Main Input:** Warehouse inventory levels (adjusted by Chapter 4), specific style and channel reservation percentages.
 - **Main Output:** Refined, allocable warehouse inventory quantities that exclude strategically reserved stock.
 - **Warning Signs:** Strategic initiatives (e.g., online exclusives) frequently running out of stock, or physical stores consistently lacking popular items despite ample warehouse availability.
 - **Key Settings:** Default Channel Reserve Percentage, Style-Specific Reserve Overrides, and Active Channels for Reservation.
-

Need Help? Contact your merchandising operations or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 6: Sales Data Preparation

Last Updated: 2025-05-23

1. Business Objective

This module processes raw transactional sales data by cleaning it, removing misleading entries, and computing key performance indicators to create a reliable foundation for all display optimization decisions.

Why This Matters: This crucial initial step ensures that all subsequent analysis, including demand forecasting and display recommendations, is based on accurate and reliable sales patterns. By removing misleading data points like liquidation sales spikes or artificial sales dips due to low stock, it provides a true reflection of actual product demand and performance. This leads to more precise inventory allocation, reduced stockouts of popular items, and ultimately, maximized sales and customer satisfaction.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** To understand the quantity of each item customers are purchasing over specific periods.
- **Transaction Dates:** To identify when sales occurred, allowing for filtering by specific timeframes and analysis periods.
- **Sale Price/Discount Data:** To identify liquidation sales where products might have been sold at significantly reduced prices, skewing true demand.

Product Data

- **Product Attributes (AG):** Details like style, category, and other classifications, used to group sales data for more granular analysis.
- **SKU Information:** Unique identifiers for each product, essential for tracking individual item performance.
- **Style Information:** To aggregate sales data for broader product groups, providing a clearer picture of demand beyond individual SKUs.

Operational Data

- **Store Locations:** To analyze sales performance at individual store levels.

- **Pivotal Stock Availability (PSA) Data:** Information on the percentage of a product's full size range (Ideal Size Set - ISS) that was physically available on display or in stock on any given day. This helps identify "broken" sales periods where low availability artificially suppressed sales.
- **Defined Analysis Periods:** The specific historical timeframes (e.g., 90 days, 30 days) over which sales data should be evaluated for Rate of Sale (ROS) calculations.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|-------------------------|---|--|
| Liquidation Sales Threshold | 0.7 (70% off) | The percentage discount (e.g., 70% off original price) at which sales are considered "liquidation" and filtered out for analysis. | Adjust if your business defines liquidation differently, or during specific promotional periods where deep discounts are not indicative of typical demand. |
| Product Availability Benchmark for Sales Cleaning | 0.2 (20% available) | The minimum percentage of a product's Ideal Size Set (ISS) that must have been available in-store for sales data from that day to be considered "unbroken" and valid. | Increase if you want to ensure sales data only comes from periods of excellent product availability; decrease if you need to include more sales data despite some stock limitations. |
| Minimum Consistent Availability Days | 14 days | The minimum number of days within the analysis period that a product's availability must meet the Product Availability Benchmark for its sales data to be included. | Increase to be more stringent about filtering out sporadic availability; decrease to include products that might have had shorter periods of good availability. |
| ROS Analysis Periods & Weighting Factors | 30/60/90 days, weighted | Defines the past sales periods (e.g., 30, 60, 90 days) and their respective importance | Shorten periods and increase weighting on recent data for fast-moving fashion; lengthen periods and balance |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|-------------------------------------|
| | | (weight) in calculating the Rate of Sale (ROS). | weighting for staple, stable items. |

Note: See More Details for additional settings related to multi-period ROS weighting.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

A[Raw Sales Transaction Data] --> B[Clean Liquidation Sales]
B --> C[Filter "Broken" Sales]
C --> D[Compute Rate of Sale (ROS)]
D --> E[Identify Priority Sales Data]
E --> F[Prepared Sales Data for Display Optimization]

```

1. Start with → **Raw Sales Transaction Data** from Point of Sale (POS) systems.
2. Then → **Clean Liquidation Sales** by removing transactions identified as deep-discount, end-of-season clearances.
3. Next → **Filter “Broken” Sales** by removing sales data from periods when product availability was artificially low, preventing accurate demand reflection.
4. Next → **Compute Rate of Sale (ROS)** for each unique product (SKU) across stores, using the now-clean sales data.
5. Next → **Identify Priority Sales Data** specifically for Optimum Depth (OD) and Optimum Width (OW) calculations, focusing on relevant seasons or product types.
6. Finally → **Prepared Sales Data for Display Optimization**, a validated dataset ready for subsequent demand forecasting and allocation decisions.

5. Step-by-Step Actions

1. **Clean** liquidation sales.
 - What we do: *We analyze sales transactions and remove those identified as liquidation (e.g., items sold below a certain discount threshold), as these sales do not reflect typical customer demand.*
 - Why we do it: *Liquidation sales can artificially inflate sales figures for items that are being cleared out, skewing the true underlying rate of sale and leading to incorrect demand forecasts.*
2. **Filter** out “broken” sales data.
 - What we do: *We examine each day's sales data for a product in a given store and compare it against the Product Availability Benchmark for Sales Cleaning. If the product's Ideal Size Set (ISS) was significantly understocked (below the benchmark) on that day, we exclude those sales.*
 - Why we do it: *Sales from periods of very low product availability (e.g., an empty shelf) do not accurately reflect true customer demand because*

customers couldn't buy what wasn't there. Removing these "broken" sales ensures we base decisions on actual demand, not stockouts.

3. Compute Rate of Sale (ROS) for each SKU.

- What we do: *For every unique item (SKU) in each store, we calculate its average daily Units Sold over specific analysis periods (e.g., 30, 60, 90 days), weighting more recent performance if desired.*
- Why we do it: *ROS is a critical measure of how quickly products are moving. An accurate ROS helps forecast future demand, identify top performers, and prevent stockouts or overstocking.*

4. Identify priority sales data for OD and OW calculations.

- What we do: *We specifically extract and highlight sales data from strategically important periods or for specific product categories (e.g., current season bestsellers) that are crucial for determining the ideal number of unique items (OD) and categories (OW) on a display.*
- Why we do it: *This ensures that decisions about display breadth and depth are driven by the most relevant and impactful sales trends, optimizing visual merchandising and sales potential.*

5. Prepare the final sales dataset.

- What we do: *We compile all the cleaned, filtered, and calculated sales data into a structured and easily accessible format, ready for subsequent modules.*
- Why we do it: *This creates a consistent and reliable data source, preventing errors and ensuring that all downstream analyses and display recommendations are based on the highest quality information.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A popular "Summer Floral Blouse" (SKU 789) sold 500 units in a particular store over the last 90 days.
- However, on 10 of those days, the blouse was on a deep discount (75% off) for a clearance event, selling 100 units during that time.
- Additionally, for 15 days, the store had very limited stock of SKU 789 (only 5% of its Ideal Size Set - ISS - was available), yet still managed to sell 30 units during these low-stock periods.
- The Liquidation Sales Threshold is set to 70% off.
- The Product Availability Benchmark for Sales Cleaning is 20%.
- The Minimum Consistent Availability Days is 14 days.

What Happens

1. First, the system identifies the 100 units sold during the 10 liquidation days. Since the 75% discount exceeds the Liquidation Sales Threshold of 70%, these 100 units are removed from the sales data. The remaining sales are $500 - 100 = 400$ units.
2. Next, the system identifies the 15 days where SKU 789 had only 5% availability. Since 5% is below the Product Availability Benchmark of

20%, the 30 units sold on those 15 days are also removed. The remaining sales are $400 - 30 = 370$ units.

3. The system then confirms that SKU 789 had enough consistent availability (more than 14 days) to be included for ROS calculation.
4. Finally, the Rate of Sale (ROS) for SKU 789 is computed based on the remaining 370 units over the 90-day period (minus the $10+15=25$ excluded days). The calculation is $370 \text{ units} / (90 - 25) \text{ days} = 370 / 65 \text{ days} =$ approximately 5.69 units/day. This precise ROS is then used for demand forecasting.

Final Outcome

The storyWiseDisplay system receives an accurate Rate of Sale (ROS) of approximately 5.69 units/day for the “Summer Floral Blouse,” reflecting its true demand without the distorting effects of clearance sales or periods of artificial stockouts. This realistic ROS will now drive more precise inventory allocation and display recommendations, ensuring the store doesn’t overstock or understock this popular item.

Key Takeaway: By meticulously cleaning and validating sales data, this module ensures that display optimization decisions are based on genuine customer demand, preventing costly errors from misleading sales figures.

7. Results

This step produces:

Main Outputs

- **Cleaned Sales Data:** A comprehensive dataset of sales transactions, free from liquidation sales and periods of “broken” product availability.
- **Computed Rate of Sale (ROS) per SKU:** A precise, store-level average daily ROS for each unique product, reflecting true demand.
- **Priority Sales Data:** A filtered subset of sales data specifically highlighted for Optimum Depth (OD) and Optimum Width (OW) calculations.

Business Reports

- **Sales Data Quality Report:** A summary for data governance teams, highlighting the volume of sales data removed due to liquidation or “broken” availability, indicating areas for potential data capture improvement.
- **SKU Rate of Sale Performance:** A report for merchandisers and planners showing the calculated ROS for each SKU across relevant stores, aiding in demand forecasting and product prioritization.
- **Seasonality & Trend Analysis Inputs:** The prepared sales data feeds into analyses that help identify seasonal patterns and emerging product trends for future planning.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|---|---|
| Missing or delayed data feed | The system will process using the most recent available sales data, but will flag an alert, potentially leading to outdated or incomplete ROS calculations and display recommendations. | Contact the data source team immediately to resolve the feed issue and manually verify critical sales data points if the delay persists. |
| Very high percentage of “broken” sales data for a product | If a product consistently has low stock or poor availability, most of its sales data might be filtered out, making ROS calculation unreliable or impossible. | Investigate root causes of poor product availability (e.g., supply chain issues, warehouse inefficiency). Prioritize replenishment for high-demand items to ensure their sales data can be accurately captured. |
| Liquidation sales not properly tagged in raw data | If deep-discount sales are not clearly marked, they might be included in the ROS calculation, leading to inflated demand forecasts for those items. | Ensure clear and consistent tagging of sales types (e.g., regular, promotional, clearance) in your Point of Sale (POS) system. Regularly review sales transaction data for anomalies. |
| New products with insufficient sales history | Products recently introduced might not have enough sales data within the Analysis Period to accurately calculate ROS or identify priority sales. | For new products, use initial sales trends with caution. Consider manual overrides or apply proxy ROS values from similar successful products until sufficient data accrues. |
| Minimum Consistent Availability Days too high | If this setting is too stringent, even well-performing products with brief stock dips might have their sales data excluded, underestimating their true demand. | Regularly review this setting to ensure it aligns with operational realities. Adjust it if it leads to too many valid sales being filtered out, impacting overall accuracy. |

9. Next Steps

Before This Step

- **Sales Data Ingestion:** Current and historical Units Sold and Rate of Sale (ROS) must be accurately captured from Point of Sale (POS) systems and passed to this module.
- **Product Master Data Updates:** Detailed product information, including categories, styles, attributes (AG), Ideal Size Set (ISS) guidelines, Optimum Depth (OD), and Optimum Width (OW) criteria, needs to be consistently up-to-date in the Product Information Management (PIM) system.

After This Step

- **Product Ranking:** The computed Rate of Sale (ROS) and priority sales data are fed into the next module, which will then rank products based on their performance and potential.
- **Display Allocation:** The prepared sales data is crucial for the allocation process, guiding decisions on which products, in what quantities, should be placed on specific displays to maximize sales.

10. Common Questions

Q: Why do we remove sales data if it actually happened? Isn't all sales data useful?

A: Not always. If a product was on deep clearance (liquidation) or if shelves were empty (broken sales), the sales figures don't truly reflect regular customer demand. Removing these "distorted" sales helps us understand how well a product *would* sell under normal conditions, leading to better predictions for future inventory and display decisions.

Q: How does Rate of Sale (ROS) help me as a merchandiser? A: ROS tells you exactly how fast a product is selling on an average day. This helps you quickly identify your bestsellers, understand which items need to be reordered frequently, and which might be slow movers. It's fundamental for making smart decisions about what to display and how much to stock.

Q: What if a brand new product has no sales history? How does it get a ROS? A: For brand new products, an initial ROS might be estimated based on similar products that have sold well in the past, or a default value might be used. As soon as real sales data starts coming in, this module will update its ROS calculation to reflect actual performance.

Q: How often does this sales data preparation happen? A: This process can be configured to run as frequently as needed, typically daily or weekly. Running it regularly ensures that the ROS figures and sales insights are always fresh and reflect the most current market conditions and customer demand.

Q: What does "broken" sales data mean, and why is it so important to filter? A: "Broken" sales data refers to sales that occurred when a product wasn't fully available or visible (e.g., only a few sizes were in stock, or the display was empty). If we used these sales, we might think demand was low when it was actually high,

but customers couldn't find the product. Filtering ensures we only analyze sales from periods when the product had a fair chance to sell, giving us a true picture of its popularity.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Loads, cleans, and structures raw sales data to calculate an accurate Rate of Sale (ROS) and identify priority sales.
- **Main Input:** Raw sales transactions, product attributes, and historical product availability (PSA).
- **Main Output:** Cleaned sales data, calculated ROS for each SKU, and identified priority sales for OD/OW calculations.
- **Warning Signs:** Persistent inventory imbalances (e.g., frequent stockouts of supposed slow-movers, or overstock of apparent bestsellers) could indicate issues with sales data quality.
- **Key Settings:** Liquidation Sales Threshold, Product Availability Benchmark for Sales Cleaning, and ROS Analysis Periods.

Need Help? Contact your data operations or merchandising analytics team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 7: Segmentation

Last Updated: 2025-05-23

1. Business Objective

This module identifies and categorizes styles into key performance segments, such as “Top Seller” and “Bottom Seller,” based on their recent sales, revenue, discount levels, and how quickly they sell relative to their stock.

Why This Matters: This process ensures that the most desirable and profitable items receive priority in inventory allocation and display space, while underperforming items are identified for reduced allocation or removal. It directly translates into higher sales, more efficient use of display space, and optimized inventory investment by making sure your stores always feature what customers want most.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold:** To understand the quantity of each style purchased by customers over specific periods, forming the basis of performance.
- **Rate of Sale (ROS):** To measure the speed at which products are moving, crucial for identifying top performers and calculating revenue per day.
- **Revenue per Day:** To understand the daily financial contribution of each style, a key metric for determining its overall value.
- **Discount Levels:** To identify styles that are frequently sold at deep discounts, which can indicate lower true demand or “bottom seller” status.
- **Sell-Through Rates:** To assess how effectively inventory is converting into sales, providing insight into a product’s health relative to its stock.

Product Data

- **Style Information:** The primary level at which products are segmented, allowing us to categorize entire product lines (e.g., “Maxi Dresses”).
- **Category:** To group styles for comparison and benchmarking, ensuring fair evaluation within similar product types (e.g., “Dresses,” “Footwear”).
- **Product Attributes (AG):** Details like brand or collection, which help filter and refine segmentation for specific business initiatives.

Operational Data

- **Current Store Inventory Levels:** Accurate counts of products currently available on the sales floor and in the backroom of each store, essential for calculating sell-through.
- **Warehouse Inventory Levels:** To understand available stock at a central level, which contributes to the overall inventory picture for sell-through calculations.
- **Store Information:** To analyze and segment styles based on their performance at individual store locations.
- **Channel Information:** To differentiate performance based on sales channels (e.g., online vs. physical retail), allowing for channel-specific segmentation if needed.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------|--|---|
| Top Seller Revenue Multiplier | 1.2 | A factor applied to the average store-category revenue benchmark to define what qualifies as a “Top Seller.” | Increase if you want to be more selective about what’s considered a top performer; decrease to include more styles in this segment. |
| Bottom Seller Discount Benchmark | 0.8 | The percentage of the average category discount below which a style is considered a “Bottom Seller” due to high discounting. | Adjust if your business defines “too much discount” differently for clearing inventory. |
| Minimum Active Selling Days for Bottom Seller | 15 days | The minimum number of days a style must have been actively available and selling for its sales data to be considered for “Bottom Seller” assessment. | Increase to be more stringent about filtering out sporadic availability; decrease to include products that might have had shorter periods of good availability. |
| Segmentation Analysis Period | 90 days | The number of past days of sales data considered for analyzing product performance | Change to react more quickly to recent trends (shorter period) or account for seasonality (longer period). |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---------------------------|----------------|
| | | and determining segments. | |

Note: See More Details for additional settings related to sell-through multipliers and specific pullback configurations.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Sales, Product & Inventory Data] --> B[Calculate Style Performance Metrics]
    B --> C{Identify Top Sellers}
    C --> D{Identify Bottom Sellers}
    D --> E[Segmented Styles & Allocation Guidance]
  
```

1. Start with → **Raw Sales, Product & Inventory Data** that has been cleaned and prepared by previous steps.
2. Then → **Calculate Style Performance Metrics**, including revenue per day, discount levels, and sell-through rates for each style.
3. Next → **Identify Top Sellers** by comparing individual style performance against defined benchmarks.
4. Next → **Identify Bottom Sellers** by evaluating styles against criteria for low revenue, high discount, and low sell-through.
5. Finally → **Segmented Styles & Allocation Guidance**, with each style labeled as a “Top Seller,” “Bottom Seller,” or “Core Item,” providing clear direction for inventory and display decisions.

5. Step-by-Step Actions

1. **Calculate** style revenue per day.
 - What we do: *For each style, we determine its average daily revenue generated at both the individual store and aggregated channel/category levels.*
 - Why we do it: *This provides a key financial metric to assess a style's performance and potential contribution.*
2. **Identify** Top Sellers.
 - What we do: *We compare each style's revenue per day against a calculated benchmark (e.g., average category revenue multiplied by the Top Seller Revenue Multiplier). Styles exceeding this benchmark and meeting minimum active selling days are tagged as “Top Sellers.”*
 - Why we do it: *This ensures that truly high-performing and profitable styles are recognized and prioritized for prime display space and inventory.*
3. **Calculate** discount levels and sell-through rates.
 - What we do: *For each style, we compute its average discount percentage (relative to original price) and its sell-through rate (units sold compared to available inventory over the Segmentation Analysis Period).*
 - Why we do it: *These metrics help gauge a style's health; high discounts and low sell-through can indicate a “Bottom Seller” or overstocked item.*

4. **Identify** Bottom Sellers.

- What we do: *We evaluate styles against multiple criteria: their revenue per day (must be below a benchmark), discount level (must be above a Bottom Seller Discount Benchmark), and sell-through rate (must be below a category average). Styles meeting these conditions and not already marked as “Top Sellers” are tagged as “Bottom Sellers.”*
- Why we do it: *This allows us to pinpoint underperforming styles that may need reduced allocation, promotions, or removal to free up valuable inventory and display space.*

5. **Assign** segment labels.

- What we do: *Based on the above evaluations, each style in each store is assigned a segment label: “Top Seller,” “Bottom Seller,” or a default segment for styles that don’t fit either extreme.*
- Why we do it: *These labels provide clear, actionable guidance for all subsequent allocation and display planning decisions.*

6. Sample Scenario

Let’s walk through a real example:

Starting Point

- A new “Floral Print Maxi Dress” (Style A) has generated an average of \$250 in revenue per day over the last 30 days in a specific store. The store’s average daily revenue for all dresses in its category is \$100. The Top Seller Revenue Multiplier is set to 1.2.
- A “Classic White Tee” (Style B) has generated \$50 in revenue per day in the same store, its discount level is 15% off, and its sell-through is 50% over the last 90 days. The store’s average category discount is 10%, and its average category sell-through is 60%. The Bottom Seller Discount Benchmark is 0.8, and the system is configured to flag items with sell-through below 0.8 times the category average.

What Happens

1. First, the system assesses Style A for “Top Seller” status. The benchmark for a top-selling dress in this store is $\$100$ (average category revenue) $\times$ 1.2 (Top Seller Revenue Multiplier) = $\$120$. Since Style A’s revenue per day ($\$250$) is greater than $\$120$, and it has met the minimum active selling days, it qualifies.
2. Next, the system assesses Style B for “Bottom Seller” status.
 - Its revenue ($\$50$) is below the category average ($\$100$).
 - Its discount (15%) is greater than the benchmark ($0.8 \times 10\% = 8\%$), indicating higher-than-average discounting.
 - Its sell-through (50%) is less than the benchmark ($0.8 \times 60\% = 48\%$), indicating slower movement.
 - Style B meets all criteria for “Bottom Seller.”
3. The system then assigns “Top Seller” to Style A and “Bottom Seller” to Style B.

Final Outcome

The store receives clear segmentation for its styles: the “Floral Print Maxi Dress” (Style A) is classified as a “Top Seller,” prompting recommendations for higher inventory levels and prime display placement. The “Classic White Tee” (Style B) is classified as a “Bottom Seller,” suggesting reduced allocation, potential pullbacks, or promotional activities to clear stock.

Key Takeaway: By automatically classifying styles into performance segments, this module provides data-driven insights that directly inform optimized display strategies and efficient inventory management, ensuring stores maximize sales and minimize dead stock.

7. Results

This step produces:

Main Outputs

- **Style Segment Labels:** A clear designation (e.g., “Top Seller,” “Bottom Seller,” “Core Item”) for each style at a granular level (e.g., per store).
- **Prioritized Style List:** A sorted list of styles based on their segment, providing immediate guidance for allocation strategies.
- **Calculated Performance Metrics:** The computed revenue per day, discount levels, and sell-through rates for each style that inform the segmentation.

Business Reports

- **Style Performance Segmentation Report:** A comprehensive report for merchandising and inventory teams, detailing which styles fall into which segment and the underlying metrics that led to that classification.
- **Allocation Strategy Guidance:** A report for planners outlining recommended inventory adjustments (e.g., increase allocation for “Top Sellers,” reduce for “Bottom Sellers”) based on segment.
- **Promotional Opportunity Report:** Highlights “Bottom Sellers” that may require promotional activity or markdowns to improve sell-through.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|------------------------------|---|--|
| Missing or delayed data feed | The system will use the most recent available data for sales, inventory, and product attributes, but will flag an alert indicating incomplete information, potentially leading to outdated or | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |

| Situation | What Happens | What You Should Do |
|--|--|---|
| A new product shows high initial sales but is not yet classified as a Top Seller | <p>inaccurate segment assignments.</p> <p>New products may not have enough selling days to meet the “Minimum Active Selling Days for Bottom Seller” or similar benchmarks, preventing immediate “Top Seller” classification.</p> | Monitor new product performance closely. Consider manual overrides for promising new styles or adjust the minimum selling days for faster classification if business strategy allows. |
| A style is flagged as a “Bottom Seller” but is on a non-liquidating promotion | <p>If a style is on a temporary, non-liquidating promotion, its increased sales and discount may incorrectly push it into the “Bottom Seller” category.</p> | Ensure promotion data is clearly differentiated from liquidation sales. Review and manually adjust segmentation for styles under specific, non-liquidating promotions. |
| All styles in a category are classified as “Bottom Sellers” | <p>If the entire category is underperforming significantly, it may trigger “Bottom Seller” classification for all its styles, potentially indicating a broader issue.</p> | Review the performance benchmarks and settings for that category. Investigate market trends, product assortment, or pricing strategies for the entire category. |

9. Next Steps

Before This Step

- **Data Preparation (Chapter 3):** All foundational input data, including raw warehouse inventory levels, store classifications, and product attributes (AG), must be accurately loaded, cleaned, and validated.
- **Warehouse Inventory Preparation (Chapter 4):** Warehouse inventory levels must have already been adjusted for operational inefficiency and lead time for online sales.
- **Warehouse Reserve Management (Chapter 5):** Inventory must have been further adjusted to account for strategic reservations for specific channels or initiatives.
- **Sales Data Preparation (Chapter 6):** Raw sales transactional data must be processed, cleaned, and a reliable Rate of Sale (ROS) computed for each style.

After This Step

- **Product Ranking:** The segmented style labels (Top Seller, Bottom Seller) are critical inputs for the product ranking module, which uses this information to create an overall prioritized list of products for display.
- **Display Allocation:** The system uses these segmentations and rankings to strategically assign products to specific display spaces, ensuring top performers get prime spots and bottom performers are managed appropriately.
- **Inventory Management Fulfillment:** Inventory and logistics teams use the segmentation guidance to prioritize stock transfers, replenishment orders, and potential pullbacks or markdowns.

10. Common Questions

Q: How does classifying products into “Top Seller” or “Bottom Seller” help my store make more money? A: By knowing which products are your “Top Sellers,” you can make sure they are always on your best displays and fully stocked, so customers can easily find and buy them, leading to more sales. For “Bottom Sellers,” you can reduce their stock, put them on sale, or remove them, freeing up space and money for better-performing items.

Q: What if a product is a “Top Seller” in one store but a “Bottom Seller” in another? A: This module can segment products specifically for *each store*. So, a product can be a “Top Seller” in a high-demand location and a “Bottom Seller” in a different store. This ensures allocation and display strategies are tailored to the unique customer preferences and sales patterns of every single store.

Q: How often does this segmentation process run? A: This process can be configured to run as frequently as needed, typically daily or weekly. Running it regularly ensures that product segments are always up-to-date, reflecting the most current sales trends and inventory positions, so your displays stay fresh and effective.

Q: Can we manually change a product’s segmentation if we disagree with the system? A: While the system provides data-driven recommendations, business users can often apply manual overrides or adjust the segmentation parameters. However, frequent manual intervention should prompt a review of the system’s settings to ensure they align with current business strategy.

Q: How does this help me avoid having too much unsold inventory (dead stock)? A: By consistently identifying “Bottom Sellers” based on their slow sales, high discounts, and low sell-through, this module signals which products are at risk of becoming dead stock. This allows you to take proactive measures like promotions, transfers, or reduced future purchasing, preventing capital from being tied up in unsold merchandise.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Identifies and categorizes styles into performance segments (“Top Seller,” “Bottom Seller”) based on sales, revenue, discount, and sell-through.

- **Main Input:** Cleaned sales data (ROS, revenue, discounts), current store and warehouse inventory levels, and product attributes (AG).
 - **Main Output:** Segmented style labels and a prioritized style list, guiding allocation strategies.
 - **Warning Signs:** Consistent stockouts of supposed “Top Sellers” or persistent overstock of “Bottom Sellers,” indicating a mismatch between segmentation and reality.
 - **Key Settings:** Top Seller Revenue Multiplier, Bottom Seller Discount Benchmark, and Segmentation Analysis Period.
-

Need Help? Contact your merchandising analytics or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 8: Revenue Computation

Last Updated: 2025-05-23

1. Business Objective

This module precisely calculates revenue metrics per day for various levels of detail, from individual styles in specific stores to broader product groupings and entire “store stories,” serving as a fundamental input for all subsequent product ranking and display allocation decisions.

Why This Matters: By providing accurate daily revenue figures, this step ensures that display planning is driven by actual financial contribution. It enables the system to identify the most profitable items and collections, optimize inventory investment by prioritizing high-value merchandise, and proactively manage stock for items nearing the end of their replenishment cycle. This leads to maximized sales per square foot, improved customer satisfaction through optimized product availability, and more efficient use of store resources.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Units Sold and Sale Price:** To calculate the total revenue generated by each product over specific periods.
- **Transaction Dates:** To determine the “live days” a product was actively selling, which is essential for computing average daily revenue.

Product Data

- **Style Information:** To aggregate sales and revenue data for product styles across different stores and periods.
- **Attribute Group (AG) Data:** Details like category, subcategory, brand, and gender, used to roll up revenue figures for broader sales trend analysis (e.g., “all women’s denim”).
- **Ideal Size Set (ISS) and Optimum Depth (OD) Criteria:** Pre-defined guidelines for the best combination of sizes and the ideal number of unique items to stock, which are factored into size-level depth calculations.
- **Product Seasonality:** To identify which styles belong to active replenishment seasons versus those that are “read-only” (view-only, not replenished).

Operational Data

- **Current Store Inventory Levels:** To identify which style-store combinations are currently active and to understand inventory availability for “store stories.”
- **Product/Size Availability (PSA) Percentages:** To understand the percentage of an Ideal Size Set (ISS) that was available, ensuring revenue calculations are based on periods where products had a fair chance to sell.
- **Store Story Definitions:** Mappings that group specific styles and categories into thematic “store stories,” allowing for aggregated revenue and inventory analysis for these collections.
- **Store-to-Store Reference Maps:** To identify which stores act as “reference stores” for new stores, enabling the replication of revenue data for locations without historical sales.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|---|---------------------|--|--|
| Recent Sales Analysis Period | 30 days | The number of past days of sales data to consider for initial revenue per day calculations for individual styles. | Shorten to react more quickly to recent trends for fast-moving fashion; lengthen for more stable product lines. |
| Read-Only Season Definitions | N/A | A list of seasons (e.g., “Winter 2024”) whose products should be considered “read-only” for display, meaning they are not actively replenished. | Update as seasons change or when specific product lines are phased out of active replenishment. |
| Product Availability Benchmark for Revenue | 0.2 (20% available) | The minimum percentage of a product’s Ideal Size Set (ISS) that must have been available in-store for its sales data to be considered valid for revenue calculation. | Adjust if you want to ensure revenue calculations are based only on periods of excellent product availability or if you need to be more inclusive. |
| Minimum Style Age for Replenishment Consideration | 14 days | The minimum number of days a new style must have been actively available in a store before | Increase for new products that need a longer observation period; decrease for rapid-turnaround items. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|----------------|
| | | it is fully considered for display recommendations and replenishment. | |

Note: See More Details for additional settings related to Attribute Group (AG) roll-up levels and specific revenue iteration approaches.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Cleaned Sales, Product, Inventory Data] --> B[Create Store-Style Combinations]

B --> C[Calculate Ideal Size Set (ISS) Depth]

C --> D[Compute Revenue Per Day (Various Levels)]

D --> E[Identify Read-Only Styles]

E --> F[Calculate Store Story Metrics]

F --> G[Revenue Metrics & Display Statuses]

1. Start with → **Cleaned Sales Data, Product Attributes, and Inventory Levels.**
2. Then → **Create Store-Style Combinations** by identifying all relevant product styles for each active store.
3. Next → **Calculate Ideal Size Set (ISS) Depth** for each style in each store.
4. Next → **Compute Revenue Per Day** for individual store-styles, then roll it up for Attribute Groups (AGs) and replicate for new stores.
5. Next → **Identify “Read-Only” Store-Styles** based on their season and replenishment eligibility.
6. Next → **Calculate Aggregated Inventory and PSA** for defined “Store Stories.”
7. Finally → **Generate Revenue Metrics per Day, OD Values, Read-Only Flags, and Store Story Health Metrics.**

5. Step-by-Step Actions

1. **Create** store-style combinations.
 - What we do: *We identify all unique combinations of active styles within each distribution store, forming the foundational units for revenue calculation.*
 - Why we do it: *This ensures that every relevant product-location pairing is considered for performance analysis.*
2. **Calculate** Ideal Size Set (ISS) depth.
 - What we do: *For each style in a store, we determine the optimal number of units per size, based on its Ideal Size Set (ISS) and category-specific depth rules, to inform Optimum Depth (OD) values.*
 - Why we do it: *This ensures that display recommendations include the most effective mix of sizes to maximize sales and minimize stockouts.*

3. **Compute** daily revenue for store-styles.
 - What we do: *We calculate the average revenue generated per day for each style in every store, focusing first on periods with actual sales and then extrapolating for styles with no recent sales based on broader trends.*
 - Why we do it: *This provides a precise financial measure of each style's performance at a granular level, guiding allocation and display decisions.*
4. **Aggregate** revenue for Attribute Groups (AGs).
 - What we do: *We roll up daily revenue figures from individual styles to higher levels of Attribute Groups (e.g., by category, brand, or color) to identify broader sales trends and average performance.*
 - Why we do it: *This helps understand overarching product trends and provides fallback revenue estimates for new or low-selling styles by using aggregated data.*
5. **Identify** “read-only” store-styles.
 - What we do: *We mark specific style-store combinations as “read-only” if they belong to seasons not designated for active replenishment or are flagged via override, meaning they should not receive new inventory.*
 - Why we do it: *This prevents the system from recommending replenishment for items being phased out, ensuring efficient inventory management and preventing dead stock.*
6. **Calculate** aggregated inventory and PSA for “store stories.”
 - What we do: *For each defined “store story” (a collection of products representing a theme, like “Summer Escape”), we sum the total available inventory and compute the average Product/Size Availability (PSA) across all included styles in each store.*
 - Why we do it: *This provides a holistic view of the overall health and readiness of entire thematic displays, helping managers understand if a “story” is well-stocked and complete.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A “Women's Floral Maxi Dress” (Style A) in Store #101 sold 30 units over the last 60 days, generating \$1,500 in revenue.
- The “Dresses” category, which includes Style A, generally generates an average of \$200 per day across all stores in the same channel.
- Store #101 has a designated “Summer Escape” story, which includes Style A along with several other styles from the “Dresses” and “Swimwear” categories. The current in-store inventory for all “Summer Escape” styles in Store #101 is 150 units.
- The average Product/Size Availability (PSA) for the included styles in Store #101's “Summer Escape” story is currently 75%.
- Style A is from the “Spring Collection,” which is currently marked as a “read-only” season.

What Happens

1. First, the system calculates the daily revenue for “Women’s Floral Maxi Dress” (Style A) in Store #101: $\$1,500 / 60 \text{ days} = \$25.00/\text{day}$.
2. Then, it aggregates the revenue for the “Dresses” Attribute Group across all relevant stores and channels, confirming the \$200 average. This aggregated data can be used as a fallback for styles with limited individual sales data.
3. Next, the system identifies that Style A is from the “Spring Collection” and, as per the “Read-Only Season Definitions” setting, marks it as a “read-only” style for Store #101. This means it will not be recommended for replenishment.
4. Finally, the system computes the total inventory for the “Summer Escape” story in Store #101, which is 150 units, and records its average PSA of 75%.

Final Outcome

The system now knows that “Women’s Floral Maxi Dress” (Style A) generates \$25.00/day in Store #101 and is designated as “read-only,” meaning it won’t be replenished. It also has a comprehensive view of the “Summer Escape” story’s health, showing 150 units in stock and a 75% PSA. This allows the system to prioritize active, high-revenue items for replenishment while managing end-of-life products efficiently and providing a clear picture of overall collection readiness.

Key Takeaway: This module provides granular, actionable revenue and status metrics across multiple product and display levels, ensuring that display decisions are financially optimized and align with inventory strategies.

7. Results

This step produces:

Main Outputs

- **Daily Revenue Metrics:** Precise revenue per day calculated for each store-style combination, for various Attribute Group (AG) roll-up levels, and for each “store story.”
- **Optimum Depth (OD) Values per Size:** The calculated ideal quantity for each size within a style’s Ideal Size Set (ISS), guiding how many units of each size should be stocked.
- **“Read-Only” Store-Style Flags:** Indicators for each style in a store, marking whether it is eligible for active replenishment or should be phased out.
- **Aggregated Store Story Inventory and PSA:** Total current inventory and average Product/Size Availability (PSA) percentages for each defined “store story.”

Business Reports

- **Style Performance Revenue Reports:** A report for merchandisers and planners showing daily revenue contributions by individual style and store, aiding in performance reviews and future buying decisions.

- **Attribute Group (AG) Sales Trend Analysis:** Summaries highlighting revenue performance across broader product classifications, useful for understanding category health and strategic planning.
- **Store Story Health Dashboards:** A dashboard for store managers and visual merchandisers, displaying the current stock levels and product availability percentages for their thematic displays.
- **Replenishment Eligibility Overviews:** A report for inventory teams, clearly identifying styles marked as “read-only” that should not receive further inventory.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|---|
| Missing or delayed data feed | The system will use the most recent available sales or inventory data, but will flag an alert indicating incomplete information, potentially leading to outdated or inaccurate revenue calculations. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| A style has zero sales but still has inventory | The system will initially assign zero revenue per day for that style. If no aggregated AG data is available, it might appear as a low-priority item, even if it's a new arrival with low exposure. | For new products, monitor initial sales trends and consider providing proxy sales data or manually adjusting their priority until sufficient sales history accumulates. |
| A style is incorrectly flagged as “read-only” | If an active, replenishable style is mistakenly tagged as “read-only,” the system will not recommend it for replenishment, potentially leading to stockouts. | Regularly review and audit the “Read-Only Season Definitions” and any style-specific overrides to ensure accuracy and prevent missed sales opportunities. |
| A “store story” has no styles assigned to it | If a defined “store story” is empty, its aggregated inventory and PSA will be zero or undefined, making its health metrics unusable. | Ensure all “store story” definitions are complete and accurately link to relevant styles and categories. |
| A new store has no sales history | For new stores, the system replicates revenue per day data from designated “reference stores” to | Ensure new stores are correctly mapped to appropriate reference stores or provide initial sales forecasts to seed their performance data. |

| Situation | What Happens | What You Should Do |
|-----------|---|--------------------|
| | provide initial estimates. If no reference store is available, revenue data might be missing. | |

9. Next Steps

Before This Step

- **Sales Data Preparation (Chapter 6):** Raw transactional sales data must be thoroughly cleaned, filtered (e.g., removing liquidation sales), and a reliable Rate of Sale (ROS) computed for each style and SKU.
- **Data Preparation (Chapter 3):** All foundational input data, including raw inventory levels (store and warehouse), detailed product attributes (AG), Ideal Size Set (ISS) guidelines, and “store story” definitions, must be accurately loaded and validated.
- **Warehouse Inventory Preparation (Chapter 4) & Warehouse Reserve Management (Chapter 5):** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock.

After This Step

- **Product Ranking (Next Chapter):** The computed revenue metrics (per store-style, AG, and store story) are the primary input for ranking products based on their performance and potential.
- **Display Allocation (Subsequent Chapter):** The calculated Optimum Depth (OD) values, along with the “read-only” flags and overall store story metrics, guide the system in strategically assigning products to specific display spaces and determining optimal quantities.

10. Common Questions

Q: Why is “revenue per day” more important than just “units sold”? A: While units sold is good, revenue per day tells you the actual financial value a product contributes each day it’s available. A high-priced item selling fewer units might still generate more revenue than a low-priced item selling many, guiding you to prioritize products that bring in the most money.

Q: How do “read-only” styles affect our displays? A: “Read-only” styles are typically end-of-season or phased-out items. By flagging them, the system knows not to recommend sending more stock to your store for these items. This prevents your valuable display space from being taken up by products that are no longer actively replenished, allowing you to focus on new or core merchandise.

Q: What are “store stories” and how does this module help with them? A: “Store stories” are curated collections of products that fit a specific theme, like “Summer Escape” or “Cozy Winter Comfort.” This module calculates overall inventory and availability for these entire collections, giving you a quick “health check” of how ready and complete your thematic displays are.

Q: How does this module use the Ideal Size Set (ISS)? A: This module uses the Ideal Size Set (ISS) to calculate the Optimum Depth (OD) for each style. It helps determine the best quantity for each individual size (e.g., 3 Small, 5 Medium, 4 Large) to ensure you have the right size mix on your displays, maximizing sales and minimizing customer disappointment.

Q: What happens if a style has no recent sales data? A: If a style has no recent sales, the module will first try to estimate its revenue per day by looking at broader sales trends from its Attribute Group (AG) (e.g., all “Women’s Denim” in that region). For brand-new stores, it can even replicate data from a similar “reference store” to provide initial estimates.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Calculates precise daily revenue metrics across various levels (store-style, Attribute Group, store story) to guide display optimization.
- **Main Input:** Cleaned sales data, comprehensive product attributes (AG, ISS), current inventory, and “store story” definitions.
- **Main Output:** Daily revenue metrics, Optimum Depth (OD) values, “read-only” flags, and aggregated “store story” health metrics.
- **Warning Signs:** Consistent stockouts of seemingly high-revenue items, or receiving replenishment for products you intended to phase out.
- **Key Settings:** Recent Sales Analysis Period and Read-Only Season Definitions.

Need Help? Contact your merchandising analytics or inventory planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 9: Ranking

Last Updated: 2025-05-23

1. Business Objective

This module identifies and prioritizes store-styles and store-stories by ranking them based on their computed revenue per day, inventory levels, and Product/Size Availability (PSA) percentages. This ranking ensures that higher-performing or more critical assortments receive allocation and display consideration first, maximizing sales and efficient space utilization.

Why This Matters: This process ensures that your most profitable and in-demand products and thematic collections are consistently prioritized for prime display space and inventory replenishment. By focusing resources on what sells best and is most complete, it directly translates to higher sales per square foot, reduced lost sales from stockouts, and a more compelling and profitable shopping experience for your customers.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Revenue per Day:** To understand the average daily financial contribution of each style and story, which is the primary driver for prioritization.
- **Rate of Sale (ROS):** (Indirectly, as it forms the basis for Revenue per Day) To ensure that product demand speed is factored into how quickly an item sells and earns revenue.

Product Data

- **Style Information:** To identify and group individual products for ranking and to calculate their performance metrics.
- **Story Definitions:** To group related styles into thematic collections, allowing for the ranking of entire display concepts.
- **Attribute Group (AG) Data:** (Indirectly, as it helps compute Revenue per Day) To ensure that revenue comparisons are made fairly within relevant product classifications.

Operational Data

- **Current Store Inventory Levels:** To understand how much stock is currently available for each style and story, informing whether a ranked item can be fully supported.
- **Product/Size Availability (PSA) Percentages:** To measure how complete a product's Ideal Size Set (ISS) is on display, ensuring that fully stocked items or stories are given appropriate priority.
- **Store Information:** To allow for ranking to be specific to individual store needs and performance, optimizing for local demand.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------|---------------------|---|---|
| Revenue Ranking Weight | High | Determines how heavily revenue per day impacts the overall ranking. | Increase if maximizing financial contribution is your top priority; decrease if inventory levels or display completeness are more critical. |
| Inventory Priority Threshold | 10 units | The minimum quantity of units a style or story must have in stock to be considered for a top ranking. | Adjust if you want to ensure only adequately stocked items are highly ranked, preventing recommendations for out-of-stock items. |
| PSA Impact Level | 0.8 (80% available) | The minimum Product/Size Availability percentage a style or story must meet to avoid a ranking penalty. | Lower if you're comfortable ranking items with less than full size sets; raise if display completeness is paramount. |
| Ranking Recalculation Interval | Daily | How often the system re-computes the entire ranking of styles and stories. | Increase for fast-moving fashion to react quickly to trends; decrease for staple items with stable performance. |

Note: See More Details for additional settings related to tie-breaking criteria.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Computed Revenue Per Day, Inventory, PSA] --> B[Rank Store-Styles]

B --> C[Rank Store-Stories]

C --> D[Prioritized Allocation List]

1. Start with → **Computed Revenue per Day, Inventory Levels, and Product/Size Availability (PSA) Percentages** for all store-styles and store-stories (from previous modules).
2. Then → **Rank Store-Styles** by sorting them based on their revenue per day, PSA, and available inventory.
3. Next → **Rank Store-Stories** by sorting them based on their aggregated revenue per day, overall PSA, and total inventory.
4. Finally → **Prioritized Allocation List**, which presents the final order of styles and stories for display planning and inventory allocation.

5. Step-by-Step Actions

1. **Gather** all computed performance metrics.
 - What we do: *We collect the calculated revenue per day, current inventory levels, and Product/Size Availability (PSA) percentages for each unique style in every store, and for each defined store story.*
 - Why we do it: *This ensures we have all the necessary performance data to make informed ranking decisions.*
2. **Prioritize** store-styles by key metrics.
 - What we do: *We sort individual store-styles, first by their revenue per day (highest first), then by their national style revenue per day, then by their PSA (highest first), and finally by their total stock (highest first), with store and style IDs as tie-breakers.*
 - Why we do it: *This creates a precise hierarchy of individual products within each store, ensuring the most impactful items are considered first for prime display space and replenishment.*
3. **Prioritize** store-stories by aggregated metrics.
 - What we do: *We sort entire store stories based on their aggregated revenue per day (highest first), followed by their overall PSA (highest first), and then their total inventory (highest first), with store and story IDs as tie-breakers.*
 - Why we do it: *This ensures that entire thematic displays or collections that are performing strongly and are well-stocked are given preference for prominent placement.*
4. **Generate** final ranked lists.
 - What we do: *We compile the sorted lists of store-styles and store-stories, assigning a numerical rank to each combination.*
 - Why we do it: *These ranked lists provide clear, actionable guidance for subsequent allocation modules, directing where inventory and visual merchandising efforts should be focused.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- In Store A, a “Classic Denim Jacket” (Style X) has a revenue per day of \$75, 50 units in stock, and a PSA of 90%.
- In the same Store A, a “Silk Blouse” (Style Y) has a revenue per day of \$100, 30 units in stock, and a PSA of 70%.
- Store A has two key “store stories”: “Everyday Essentials” (includes Style X) with an aggregated revenue per day of \$1,500, total inventory of 300 units, and PSA of 85%.
- The “Evening Wear Collection” (includes Style Y) has an aggregated revenue per day of \$2,000, total inventory of 150 units, and PSA of 75%.
- The Revenue Ranking Weight is set to “High”, meaning revenue is the top priority.

What Happens

1. First, the system ranks the store-styles for Store A:
 - Style Y (“Silk Blouse”) has a higher revenue per day (\$100) than Style X (“Classic Denim Jacket”, \$75), so Style Y is ranked higher based on revenue, despite lower stock and PSA.
 - If their revenues were tied, Style X (90% PSA) would be ranked higher than Style Y (70% PSA).
2. Next, the system ranks the store-stories for Store A:
 - “Evening Wear Collection” has a higher aggregated revenue per day (\$2,000) than “Everyday Essentials” (\$1,500), so it is ranked higher based on revenue.
 - If revenues were tied, “Everyday Essentials” (85% PSA) would be ranked higher than “Evening Wear Collection” (75% PSA).

Final Outcome

The system produces a prioritized list for Store A: - **Store-Style Ranking:** Style Y (“Silk Blouse”) is ranked higher than Style X (“Classic Denim Jacket”), meaning the system will first seek to fulfill allocations for Style Y. - **Store-Story Ranking:** “Evening Wear Collection” is ranked higher than “Everyday Essentials,” indicating it should receive preferential treatment for display space and replenishment to maintain its momentum. This ensures that the store focuses on maintaining stock for the highest-revenue-generating items and themes first.

Key Takeaway: By precisely ranking products and collections based on their true performance and availability, this module provides a clear roadmap for prioritizing inventory investment and maximizing display impact.

7. Results

This step produces:

Main Outputs

- **Ranked Store-Style List:** A prioritized list of every unique style within each store, ordered by importance for allocation and display.
- **Ranked Store-Story List:** A prioritized list of all defined thematic collections within each store, indicating which stories should receive primary focus.
- **Priority Scores:** Numerical values assigned to each ranked item or story, quantifying its relative importance for further action.

Business Reports

- **Allocation Priority Report:** A detailed report for merchandising and inventory teams, clearly showing which products and stories are prioritized for replenishment and display.
- **Display Planning Guidance:** For store managers and visual merchandisers, this report offers a clear directive on where to focus their efforts to highlight the most impactful merchandise.
- **Performance-Based Review List:** A dynamic list that helps category managers quickly identify top and bottom performers across different stores based on their rank.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|---|
| Missing or delayed data feed | The system will use the most recent available data for revenue, inventory, and PSA, but will flag an alert indicating incomplete information, potentially leading to outdated or inaccurate rankings. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| A new, high-potential product has no sales history for ranking | The system will rank new products lower due to lack of historical data for revenue, potentially overlooking emerging trends. | Manually apply a proxy rank or adjust initial inventory levels for promising new items based on market intelligence until sufficient sales history accumulates. |
| All products or stories within a store have very similar performance metrics | If all inputs (revenue, stock, PSA) are nearly identical, the tie-breaking rules (e.g., style/story ID) will determine the rank, which may not always align with business intuition. | Review the tie-breaking logic or introduce additional weighting factors if manual review consistently shows misaligned priorities in such edge cases. |
| | The item is ranked high but is perpetually out of | Investigate supply chain issues, reordering frequency, or |

| Situation | What Happens | What You Should Do |
|---|--|--|
| A high-ranking item consistently has very low stock | stock, leading to missed sales opportunities despite its priority. | production capacity for this specific item. Increase minimum stock thresholds for high-ranking items if necessary. |

9. Next Steps

Before This Step

- **Sales Data Preparation (Chapter 6):** Raw transactional sales data must be thoroughly cleaned and processed, and a reliable Rate of Sale (ROS) computed for each style.
- **Revenue Computation (Chapter 8):** Precise daily revenue metrics must be calculated for each store-style and store-story combination, and Optimum Depth (OD) values determined.
- **Warehouse Inventory Preparation (Chapter 4) & Warehouse Reserve Management (Chapter 5):** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock.
- **Data Preparation (Chapter 3):** All foundational input data, including raw inventory levels (store and warehouse), detailed product attributes (AG, ISS), and “store story” definitions, must be accurately loaded and validated.

After This Step

- **Display Allocation:** The highly prioritized and ranked lists of store-styles and store-stories are passed to the allocation module. This module will then strategically assign products to specific display spaces, ensuring top performers get prime spots and optimum quantities.
- **Inventory Management Fulfillment:** Inventory and logistics teams use the rankings to prioritize stock transfers from the warehouse to stores, focusing on replenishing the highest-ranking items first.

10. Common Questions

Q: How does “Ranking” help my store decide what to put on display? A: This module acts like a smart assistant that tells you which products and collections are your biggest money-makers and most complete. By putting the highest-ranked items and stories front and center, you ensure customers see and buy what’s most profitable and in-demand, directly boosting your sales.

Q: What if a lower-ranked item is still important for our overall product offering? A: While ranking prioritizes top performers, the system balances this with rules for ensuring a complete assortment. Lower-ranked items might still be allocated space, but typically less prominent spots or smaller quantities, freeing up prime real estate for your stars.

Q: How often does the system re-calculate these rankings? A: The ranking process can be set to run as frequently as needed, typically daily. This ensures that your displays always reflect the most current sales trends, customer demand, and inventory levels, keeping your store agile and responsive.

Q: Does the ranking consider if we actually *have* the product in stock? A: Yes, absolutely. Current inventory levels and Product/Size Availability (PSA) are key factors in the ranking. A high-revenue item that's out of stock will be ranked lower or flagged for immediate replenishment, ensuring display recommendations are always achievable with available products.

Q: How does this help manage our overall display strategy, not just individual products? A: Beyond individual products, the system also ranks entire “store stories” – like your “Summer Escape” collection. This helps you understand which overall themes or curated displays are performing best, guiding larger visual merchandising decisions and ensuring your store effectively communicates its key narratives.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Prioritizes store-styles and store-stories for allocation based on their performance and availability.
- **Main Input:** Computed Revenue per Day, Inventory Levels, and Product/Size Availability (PSA) percentages.
- **Main Output:** Ranked lists of store-styles and store-stories, with assigned priority scores.
- **Warning Signs:** Frequent stockouts of supposed “high-ranking” items, or store displays consistently featuring slow-selling or incomplete products.
- **Key Settings:** Revenue Ranking Weight, Inventory Priority Threshold, and PSA Impact Level.

Need Help? Contact your merchandising analytics or inventory planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 10: Allocation

Last Updated: 2025-05-23

1. Business Objective

This module is the central engine of the storyWiseDisplay system, responsible for precisely determining which unique items (SKUs) go to which stores and in what quantities. It orchestrates a series of allocation steps, applying different business rules and constraints in each pass to ensure products are optimally distributed.

Why This Matters: This process directly impacts your store's sales and profitability by ensuring the right products are always in the right place at the right time. It optimizes inventory investment by maximizing sales of popular items, clearing slow-movers, and making sure your displays are visually appealing and aligned with your brand's merchandising stories. This leads to reduced lost sales from stockouts, a better customer experience, and more efficient use of your valuable display space.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS):** *To understand how quickly products are selling, which helps determine replenishment needs and priority.*
- **Units Sold:** *To track the quantity of each item customers are buying, forming the foundation for what should be allocated.*

Product Data

- **Optimum Depth (OD):** *The ideal number of unique items (SKUs) to stock in a category or on a display, guiding how many different products to place.*
- **Optimum Width (OW):** *The ideal number of different categories or styles to offer, ensuring a balanced and cohesive display.*
- **Ideal Size Set (ISS):** *The preferred combination of sizes for each product, ensuring customers find their fit and reducing missed sales.*
- **Story Definitions:** *To understand which products belong to specific merchandising themes (e.g., "Summer Escape" collection) for cohesive display planning.*
- **Product Rankings:** *The prioritized list of products and stories (from Chapter 9) indicating which items are most important to allocate first.*

Operational Data

- **Current Store Inventory Levels:** *Accurate counts of what products are already on hand in each store, both on the sales floor and in the backroom.*
- **Warehouse Inventory Levels:** *The adjusted quantity of products truly available in the central warehouse (from Chapter 4 and 5) that can be sent to stores.*
- **Display Capacities (Planograms):** *The physical space limitations of each display, including the number of items or styles it can hold.*
- **Replenishment Cycle Days:** *The frequency at which stores receive new shipments, influencing how much stock needs to be allocated for future demand.*

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|-------------------------------------|---------------|--|--|
| Depth Multiplier for Allocation | 1x | A factor that allows the system to increase the allocated Optimum Depth (OD) for high-demand items, sending more units than the standard OD. | Increase for fast-selling items that need more stock on display; decrease if you prefer tighter display control. |
| Planogram Increment Flag | True | Enables the system to consider increasing the Optimum Width (OW) (the number of styles/categories) on displays if demand warrants it. | Disable if you want strict adherence to predefined planograms and no expansion of styles on display. |
| Core Allocation/ Replenishment Flag | True | Determines whether the system should run the foundational core allocation and replenishment passes. | Disable only for very specific, temporary scenarios where a custom allocation strategy is needed. |
| Pullback Activation | True | If enabled, the system will identify and recommend | Disable if you prefer to manage excess stock through in-store promotions or |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------------------------|
| | | moving slow-selling or excess stock from stores back to the warehouse. | markdowns rather than transfers. |

Note: See More Details for additional settings related to specific allocation iteration prioritization and store spread requirements.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Ranked Products & Stories] --> B[Initial Story-Based Allocation]

B --> C[Iterative Adjustments (OW, OD, Replenishment, Pullback)]

C --> D[Generate Final Allocation Plan]

1. Start with → **Prioritized product and story rankings, alongside comprehensive inventory and product data.**
2. Then → **Initiate story-based allocation** to fulfill core merchandising themes, ensuring cohesive displays.
3. Next → **Apply iterative adjustments**, including OW (Optimum Width) for display breadth, OD (Optimum Depth) for SKU quantity, and replenishment/pullback strategies.
4. Finally → **Generate the precise, actionable Final Allocation Plan** for each store, including transfer recommendations.

5. Step-by-Step Actions

1. **Perform** initial story-based allocation.
 - What we do: *We first assign products to displays based on their predefined merchandising “stories” (e.g., “Summer Escape” collection) to create coherent and visually appealing themes.*
 - Why we do it: *This ensures that your displays align with marketing campaigns and present a cohesive shopping experience.*
2. **Apply** Optimum Width (OW) adjustments.
 - What we do: *We ensure the ideal number of different categories or styles (Optimum Width - OW) are represented on displays, expanding or contracting as needed to balance variety with focus.*
 - Why we do it: *This optimizes the breadth of your product offering on displays, preventing overcrowding or a sparse look.*
3. **Apply** Optimum Depth (OD) adjustments.
 - What we do: *We allocate the ideal number of unique items (SKUs) for each product (Optimum Depth - OD), adjusting quantities based on sales performance, Ideal Size Set (ISS) recommendations, and available space.*

- Why we do it: *This ensures you have enough stock of the most popular items, including the right sizes, to meet customer demand and reduce stockouts.*
- 4. **Execute** replenishment iterations.
 - What we do: *We identify high-selling items that need more stock and recommend sending additional units from the warehouse to ensure displays remain full and customers can always find what they want.*
 - Why we do it: *This keeps your bestsellers on display and in stock, directly maximizing sales and customer satisfaction.*
- 5. **Execute** pullback iterations.
 - What we do: *We identify and recommend moving slow-selling, excess, or aging stock from stores back to the central warehouse or to other stores where it might sell better.*
 - Why we do it: *This frees up valuable display space for more profitable items, reduces the risk of markdowns, and optimizes inventory investment.*
- 6. **Generate** cross-store transfers.
 - What we do: *We may suggest moving stock directly between stores to rebalance inventory based on localized demand, bypassing the warehouse if it's more efficient.*
 - Why we do it: *This ensures popular items in one store can quickly get stock from another where it's not selling as well, maximizing sales opportunities across your network.*
- 7. **Finalize** allocation plan.
 - What we do: *We compile all the decisions from the various iterations into a comprehensive, actionable plan, detailing exact product placements and recommended inventory movements.*
 - Why we do it: *This provides clear instructions for store teams and inventory management, ensuring the optimized display strategy can be quickly implemented.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A clothing store has a prime display rack for its “Summer Athleisure” story, with space for an OW of 4 different sub-categories (e.g., leggings, tops, jackets, accessories) and an OD of 20 unique items.
- The “Core Black Leggings” (Style A) have a very high ROS (Rate of Sale), but the store only has 8 units in stock and needs to fulfill its ISS (Ideal Size Set).
- A “Seasonal Print Jacket” (Style B) has a lower ROS, but the store has 15 units, exceeding its desired OD.
- The “Everyday Essentials” story (which includes Style A) is ranked higher than the “Active Lifestyle” story (which includes Style B).
- The central warehouse has plenty of Style A but also excess Style B.

What Happens

1. First, the system prioritizes the “Summer Athleisure” story, focusing on the highest-ranked styles within it, like “Core Black Leggings” (Style A), to ensure the story’s success.
2. Then, for Style A, the system notes its high ROS and low current stock. It checks the OD guidelines and recommends sending enough units from the warehouse to meet its optimal display quantity and fulfill its ISS (e.g., suggesting a transfer of 15 additional units for a total of 23).
3. Next, for Style B, the system identifies its lower ROS and excess stock. It recommends reducing its presence on the display (perhaps down to 5 units) and suggests pulling back the excess 10 units to the warehouse.
4. Throughout this, the system ensures the OW (4 sub-categories) and OD (20 unique items) for the “Summer Athleisure” display are met, balancing top sellers with necessary assortment.

Final Outcome

The store receives a precise allocation plan: significantly increase the display quantity of “Core Black Leggings” (Style A) by transferring stock from the warehouse, while reducing the presence of “Seasonal Print Jacket” (Style B) and sending excess back to the warehouse. This ensures the “Summer Athleisure” display prominently features high-demand items, is optimally stocked to maximize sales, and efficiently manages inventory.

Key Takeaway: By orchestrating various rules and iterations, this module ensures that display plans are strategically optimized, leading to higher sales, reduced waste, and a compelling customer experience.

7. Results

This step produces:

Main Outputs

- **Final Allocation Plan:** A detailed, actionable list of which products (SKUs), in what quantities (including Ideal Size Set - ISS), should be placed on each specific display or category area in every store.
- **Inventory Transfer Recommendations:** Specific instructions for moving stock between the warehouse and stores, or between stores, to support the optimized display plans.
- **Optimized Display Quantities:** The final, calculated Optimum Depth (OD) and Optimum Width (OW) for each display based on the allocation decisions.

Business Reports

- **Store Allocation Summary:** A clear report for store managers and visual merchandisers, showing the planned display changes and product placements.
- **Inventory Movement Instructions:** A detailed manifest for warehouse and logistics teams, outlining all required stock transfers and pullbacks.

- **Display Health & Readiness Report:** For merchandising teams, this report summarizes how effectively the allocation plan fills displays and meets optimal visual merchandising standards.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|---|
| Missing or delayed data feed | The system will use the most recent available data for sales, inventory, and rankings, but will flag an alert, potentially leading to outdated or inaccurate allocation decisions. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| No available stock for high-ranking items | The system will prioritize allocating any existing store stock for these items but cannot recommend transfers from an empty warehouse. It will flag the stock shortage. | Expedite reordering or production for these high-demand items. Communicate potential stockouts to sales teams and adjust future inventory plans. |
| Store display capacity is too low for all recommended items | The system will prioritize the highest-ranking items within the physical display limits, potentially leaving some desirable items out. | Review display capacities and store layouts. Consider alternative display solutions or secondary locations for items that don't fit prime displays. |
| Conflicts between different allocation rules (e.g., OW vs. OD) | The system follows its predefined hierarchy of rules, prioritizing as configured (e.g., filling OW before maximizing OD for specific products). | Review the allocation settings to ensure the rule prioritization aligns with your current business strategy and objectives. |
| Excessive pullbacks recommended for a product line | If a large quantity of stock is recommended for pullback, it suggests a significant overstocking issue or a sharp decline in demand for that product. | Investigate the reasons for overstock (e.g., poor forecasting, slow sell-through). Plan promotions or markdowns to clear stock if transfers are not viable. |

9. Next Steps

Before This Step

- **Sales Data Preparation (Chapter 6):** Raw transactional sales data must be thoroughly cleaned, filtered (e.g., removing liquidation sales), and a reliable Rate of Sale (ROS) computed for each style.
- **Revenue Computation (Chapter 8):** Precise daily revenue metrics must be calculated for each store-style and store-story combination, and Optimum Depth (OD) values determined.
- **Segmentation (Chapter 7):** Product styles must be categorized into performance segments (e.g., “Top Seller,” “Bottom Seller”).
- **Ranking (Chapter 9):** Products and stories must be prioritized based on their performance and availability.
- **Data Preparation (Chapter 3):** All foundational input data, including raw inventory levels (store and warehouse), detailed product attributes (AG, ISS), and “store story” definitions, must be accurately loaded and validated.
- **Warehouse Inventory Preparation (Chapter 4) & Warehouse Reserve Management (Chapter 5):** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock.

After This Step

- **Store Operations Implementation:** Store teams need to receive the optimized display plans and physically arrange products according to the recommendations. They will require clear instructions and visual guides.
- **Inventory Management Fulfillment:** Warehouse and logistics teams need to process and execute the recommended inventory transfers and pullbacks to ensure products arrive at stores or are returned to the warehouse as planned.
- **Performance Monitoring:** Store managers and merchandising teams should monitor sales performance against the implemented display plans to assess effectiveness and identify any areas for further optimization.

10. Common Questions

Q: How does this module ensure my displays are visually appealing? A: This module starts with “story-based allocation,” ensuring that products are grouped according to predefined merchandising themes (like “Summer Escape”). It also uses Optimum Width (OW) to ensure a balanced number of different styles or categories are presented, creating a cohesive and attractive visual display.

Q: What if a store is small and can’t hold all the recommended items? A: The system considers your display capacities and works within those limits. It prioritizes the highest-ranking and most profitable items to ensure they get space first. If there’s truly not enough room, it will allocate what fits best and flag any remaining high-priority items that couldn’t be placed.

Q: How quickly does the system react to bestsellers selling out? A: This module includes “replenishment iterations” that are designed to quickly identify and send more stock of fast-selling items from the warehouse to your stores. Because the overall process runs frequently (e.g., daily), it can react very quickly to changes in Rate of Sale (ROS) and stock levels.

Q: What happens to products that don’t get allocated prime display space? A: Products that aren’t prioritized for prime display space might still be allocated smaller quantities, placed in secondary display locations, or, if they’re consistently slow-selling or excess, flagged for “pullback” to the warehouse or potential promotions. The goal is to optimize every item’s lifecycle.

Q: How does this help us manage seasonal transitions? A: This module incorporates “read-only” flags for end-of-season products, preventing new stock from being sent to stores. This ensures that as seasons change, your displays can be cleared of old merchandise to make room for new arrivals that are actively selling and being replenished.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Determines the final, optimized allocation of unique items (SKUs) to stores, applying various business rules.
- **Main Input:** Prioritized product and story rankings, Optimum Depth (OD) and Optimum Width (OW) guidelines, Ideal Size Set (ISS), and accurate inventory levels.
- **Main Output:** A detailed, actionable allocation plan for stores, including recommended inventory transfers and pullbacks.
- **Warning Signs:** Persistent stockouts of top-selling items on display, or stores consistently holding too much slow-moving or off-season stock.
- **Key Settings:** Depth Multiplier for Allocation, Planogram Increment Flag, and Pullback Activation.

Need Help? Contact your merchandising operations or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 11: Iteration Runner

Last Updated: 2025-05-23

1. Business Objective

This module precisely executes the detailed allocation logic for each specific phase of the display optimization process, applying all relevant business rules and constraints to determine exact product placements and quantities.

Why This Matters: This is the core engine that translates data and strategy into actionable display plans. By methodically processing each allocation rule, it ensures that your stores receive the right mix of products, at optimal quantities, for every display. This directly maximizes sales by keeping popular items in stock, optimizes inventory investment, and creates visually appealing and effective product presentations that enhance the customer experience.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS):** To understand how quickly individual products and styles are selling, which is critical for determining how much stock to allocate and how often.
- **Units Sold:** To track the actual quantity of each item customers are purchasing, providing the foundational sales history for ROS calculations.

Product Data

- **Optimum Depth (OD):** The ideal number of unique items (SKUs) to stock in a category or on a display, guiding how many different products to place and in what quantities.
- **Optimum Width (OW):** The ideal number of different categories or styles to offer on a display, ensuring a balanced and cohesive product presentation.
- **Ideal Size Set (ISS):** The preferred combination of sizes for each product, ensuring customers find their perfect fit on the display and preventing missed sales.
- **Product Rankings:** The prioritized list of products and stories (from the Ranking module) indicating which items are most important to allocate and display first.

Operational Data

- **Current Store Inventory Levels:** Accurate counts of what products are already on hand in each store, both on the sales floor and in the backroom, to inform immediate needs.
- **Warehouse Inventory Levels:** The adjusted quantity of products truly available in the central warehouse (after accounting for inefficiency and strategic reserves) that can be sent to stores.
- **Display Capacities (Planograms):** The physical space limitations of each display, including the maximum number of items or styles it can physically hold.
- **Replenishment Cycle Days:** The frequency at which stores receive new shipments, influencing how much stock needs to be allocated to cover demand until the next delivery.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|----------------------------------|---------------|---|--|
| Top Seller Cover Days Multiplier | 1.2 | A factor applied to the replenishment days for “Top Seller” items, allowing them to receive more stock than typical items to prevent stockouts. | Increase for your fastest-selling items that need a larger buffer; decrease if you want tighter control over stock levels for top sellers. |
| Planogram Options Check | True | When enabled, the system verifies that the display has enough physical space for the recommended Optimum Width (OW) (number of styles) before allocating. | Disable if you want to allocate styles regardless of strict display space for variety; enable for strict adherence to display design. |
| Outward Excess Depth | False | If set to ‘False’, the system will prevent sending more units of a specific SKU to a store than its calculated Optimum Depth (OD), even if excess stock exists. | Set to ‘True’ if you are comfortable with overstocking for high-demand items; ‘False’ for strict inventory control on displays. |

| Setting Name | Default Value | Description | When to Change |
|---------------------|---------------|---|--|
| Pullback Activation | True | Determines if the system should identify and recommend moving slow-selling or excess stock from stores back to the warehouse. | Disable if you prefer to manage excess stock through in-store promotions or markdowns rather than transfers. |

Note: See More Details for additional settings related to specific allocation iteration prioritization and store spread requirements.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Ranked Products & Stories, Inventory, Display Rules] -->
    B[Execute Allocation Iterations]
    B --> C[Determine SKU Quantities & Manage Stock]
    C --> D[Suggested Allocation Plan]
  
```

1. Start with → **Prioritized product and story rankings**, alongside comprehensive inventory data, display capacities, and all defined business rules.
2. Then → **Execute allocation iterations**, applying different sets of rules and constraints in a structured sequence, such as initial story-based allocations, then adjustments for Optimum Width (OW) and Optimum Depth (OD), followed by replenishment and pullback strategies.
3. Next → **Determine specific SKU quantities** for each store-style combination, deduct stock from available inventory, and record any relevant remarks.
4. Finally → **Generate the precise, actionable Suggested Allocation Plan** for each store, including all recommended product placements and inventory movements.

5. Step-by-Step Actions

1. **Prioritize** and execute allocation for store stories.
 - What we do: *The module begins by processing each ranked “store story” (thematic collection), ensuring that the minimum breadth of styles (Optimum Width - OW) and required category combinations are met before individual styles are considered.*
 - Why we do it: *This ensures that display plans initially focus on creating coherent, high-priority merchandising themes that align with brand campaigns and customer interest.*
2. **Validate** style allocation conditions.
 - What we do: *For each style within a story, we check if it meets key criteria, such as having a sufficient Product/Size Availability (PSA) percentage and if there's enough physical display space (planogram capacity) to accommodate it without exceeding the Optimum Width (OW).*

- Why we do it: *This prevents the system from recommending allocations for styles that are not visually ready for display or cannot physically fit, ensuring realistic and actionable plans.*
3. **Determine** individual SKU allocations.
 - What we do: *For each unique item (SKU) within an eligible style, we calculate its suggested allocation quantity. This calculation considers its Rate of Sale (ROS), the needs of its Ideal Size Set (ISS), its Optimum Depth (OD) limits, and the store's replenishment cycle days.*
 - Why we do it: *This ensures that each display has the right amount of stock for each product, particularly for bestsellers and across the full range of sizes, minimizing stockouts and maximizing sales opportunities.*
 4. **Manage** inventory deductions and remarks.
 - What we do: *Once a suggested allocation is determined, the system immediately deducts the allocated quantity from both the store's current inventory and the central warehouse's available stock. It also records any specific remarks, such as if the maximum Optimum Depth (OD) was reached or if insufficient stock was available.*
 - Why we do it: *This keeps inventory levels accurate in real-time for other modules and provides clear reasons for each allocation decision, aiding in troubleshooting and future planning.*
 5. **Handle** remaining stock and pullbacks.
 - What we do: *Depending on the business settings (e.g., if "Outward Excess Depth" is disabled), any remaining stock beyond the Optimum Depth (OD) is noted. During certain iterations (e.g., replenishment), the module decides if excess store stock should be proactively pulled back to the warehouse.*
 - Why we do it: *This optimizes inventory efficiency by preventing overstocking on the sales floor and freeing up valuable display space for better-performing items.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A clothing store has a central display for its "Summer Athleisure" story, which is highly ranked. This display has space for an OW of 4 different sub-categories (e.g., leggings, tops, jackets, accessories) and an OD of 20 unique items.
- "Core Black Leggings" (Style A) are a top-ranked item in this story, with a very high ROS (Rate of Sale). The store currently has 8 units, but its OD is 15 units, and its ISS (Ideal Size Set) requires a broad size run.
- A "Seasonal Print Jacket" (Style B) is also part of the story but has a lower ROS and its OD is 5 units. The store currently has 12 units of Style B, which exceeds its desired OD.
- The central warehouse has 100 units of Style A and 50 units of Style B. The "Top Seller Cover Days Multiplier" is 1.5.

What Happens

1. First, the Iteration Runner prioritizes the “Summer Athleisure” story. It identifies “Core Black Leggings” (Style A) as a top seller needing immediate attention.
2. For Style A, the system calculates the suggested allocation. Because it’s a “Top Seller,” its replenishment is boosted by the “Top Seller Cover Days Multiplier.” It determines that 15 units are ideal for OD and ISS. Since 8 units are in store, it suggests transferring 7 more units from the warehouse. These 7 units are then deducted from the warehouse’s stock.
3. Next, for Style B, the system identifies its lower ROS and the fact that the store’s current stock (12 units) significantly exceeds its OD of 5 units. Assuming “Pullback Activation” is enabled for this iteration, it recommends reducing Style B’s presence to 5 units and suggesting a pullback of the excess 7 units back to the warehouse.
4. Throughout this, the system continually checks the display’s OW (number of categories/styles) and OD (total unique items), ensuring that both Style A and Style B contribute to maintaining the display’s overall balance and capacity limits.

Final Outcome

The store receives a precise allocation plan: significantly increase the display quantity of “Core Black Leggings” (Style A) by transferring 7 units from the warehouse, while simultaneously reducing the presence of “Seasonal Print Jacket” (Style B) and sending 7 excess units back to the warehouse. This ensures the “Summer Athleisure” display prominently features high-demand items, is optimally stocked to maximize sales, and efficiently manages inventory by clearing slow-moving stock.

Key Takeaway: By orchestrating various business rules and applying them iteratively, this module generates precise, actionable display and inventory plans that maximize sales and optimize product presentation.

7. Results

This step produces:

Main Outputs

- **Suggested SKU Allocations:** A detailed, actionable list of which products (SKUs), in what quantities (including Ideal Size Set - ISS), should be placed on each specific display or category area in every store.
- **Updated Store and Warehouse Inventory:** Real-time adjusted stock levels for both store and warehouse locations, reflecting the quantities allocated or pulled back.
- **Allocation Remarks:** Specific notes and reasons recorded for each allocation decision, such as “stock failed” or “Optimum Depth reached,” providing transparency and aiding analysis.

Business Reports

- **Store Display Update Plan:** A clear report for store managers and visual merchandisers, showing the planned display changes and exact product placements for their store.
- **Inventory Movement Manifest:** A detailed document for warehouse and logistics teams, outlining all required stock transfers from the warehouse to stores, and any recommended pullbacks from stores to the warehouse.
- **Allocation Exception Report:** A summary for merchandising and inventory teams, highlighting any instances where desired allocations could not be met due to stock shortages, capacity limits, or other constraints.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|---|
| Missing or delayed data feed | The system will use the most recent available data for sales, inventory, and rankings, but will flag an alert indicating incomplete information, potentially leading to outdated or inaccurate allocation decisions. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| No available warehouse stock for high-ranking items | The system will prioritize allocating any existing store stock for these items but cannot recommend transfers from an empty warehouse. It will flag the stock shortage and note that Optimum Depth (OD) cannot be met. | Expedite reordering or production for these high-demand items. Communicate potential stockouts to sales teams and adjust future inventory plans. |
| Store display capacity is too low for all recommended items | The system will prioritize the highest-ranking items within the physical display limits, potentially leaving some desirable items out. It will also record remarks about space limitations. | Review display capacities and store layouts. Consider alternative display solutions or secondary locations for items that don't fit prime displays. |
| Conflicts between different allocation rules (e.g., OW vs. OD) | The system follows its predefined hierarchy of rules, prioritizing one constraint over another as configured (e.g., | Review the allocation settings to ensure the rule prioritization aligns with your current business strategy and objectives. |

| Situation | What Happens | What You Should Do |
|--|--|---|
| Excessive pullbacks recommended for a product line | <p>ensuring OW is met before maximizing OD for specific products). Any compromises or unfulfilled aspects will be noted in remarks.</p> <p>If a large quantity of stock is recommended for pullback, it suggests a significant overstocking issue or a sharp decline in demand for that product, potentially due to incorrect prior allocations.</p> | Investigate the reasons for overstock (e.g., poor forecasting, slow sell-through). Plan promotions or markdowns to clear stock if transfers are not viable. |

9. Next Steps

Before This Step

- **Sales Data Preparation:** Raw transactional sales data must be thoroughly cleaned, filtered (e.g., removing liquidation sales), and a reliable Rate of Sale (ROS) computed for each style.
- **Revenue Computation:** Precise daily revenue metrics must be calculated for each store-style and store-story combination, and Optimum Depth (OD) values determined.
- **Segmentation:** Product styles must be categorized into performance segments (e.g., “Top Seller,” “Bottom Seller”), providing key input for prioritization.
- **Ranking:** Products and stories must be prioritized based on their performance and availability, creating the order in which the Iteration Runner processes them.
- **Data Preparation:** All foundational input data, including raw inventory levels (store and warehouse), detailed product attributes (AG, ISS), and “store story” definitions, must be accurately loaded and validated.
- **Warehouse Inventory Preparation & Warehouse Reserve Management:** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock.

After This Step

- **Store Operations Implementation:** Store teams need to receive the optimized display plans and physically arrange products according to the recommendations. They will require clear instructions and visual guides.
- **Inventory Management Fulfillment:** Warehouse and logistics teams need to process and execute the recommended inventory transfers and pullbacks to ensure products arrive at stores or are returned to the warehouse as planned.

- **Performance Monitoring:** Store managers and merchandising teams should continuously monitor sales performance against the implemented display plans to assess effectiveness and identify any areas for further optimization.

10. Common Questions

Q: How does this module make sure my displays are just right, not too empty or too crowded? A: This module uses “Optimum Depth (OD)” to calculate the ideal number of each specific item (SKU) to stock on a display, ensuring you have enough to meet demand without overstocking. It also uses “Optimum Width (OW)” to ensure the right number of different styles or categories are presented, creating a balanced and appealing visual display.

Q: What happens if we don’t have enough stock in the warehouse for what it suggests? A: The system will only allocate what’s truly available in the warehouse (after all previous adjustments). If stock is limited, it will still make the best possible allocation with what you *do* have, while highlighting that the full desired quantity could not be met.

Q: Does this module help us manage products that aren’t selling well in stores? A: Yes, absolutely. This module includes “pullback iterations” that identify slow-selling or excess stock in stores and recommend moving it back to the warehouse. This frees up valuable display space for more profitable items and helps prevent markdowns.

Q: How quickly does the system react to bestsellers selling out or new trends emerging? A: This module runs frequently, typically daily, as part of the overall display optimization process. Because it constantly processes the most recent sales data and product rankings, it can react very quickly to changes in Rate of Sale (ROS) and stock levels, ensuring your displays are always current.

Q: Can this process handle different types of allocation rules for different products, like new arrivals versus core items? A: Yes, this module is designed to run through multiple “iterations,” each applying specific allocation logic. This allows it to handle diverse scenarios, from initial allocations for new product launches, to ongoing replenishment for bestsellers, and strategic pullbacks for end-of-life items, ensuring each product type is managed appropriately.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Executes the precise, iterative allocation logic to determine exact SKU quantities and placements for optimized displays.
- **Main Input:** Prioritized product and story rankings, Optimum Depth (OD) and Optimum Width (OW) guidelines, Ideal Size Set (ISS), and accurate inventory levels.
- **Main Output:** A detailed, actionable allocation plan for stores, including recommended inventory transfers and pullbacks.
- **Warning Signs:** Persistent stockouts of top-selling items on display, or stores consistently holding too much slow-moving or off-season stock.

- **Key Settings:** Top Seller Cover Days Multiplier, Planogram Options Check, Outward Excess Depth, and Pullback Activation.
-

Need Help? Contact your merchandising operations or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 12: Planogram Incrementer

Last Updated: 2025-05-23

1. Business Objective

This module dynamically calculates an “increment ratio” for planogram targets, specifically influencing the OW (Optimum Width – the ideal number of different categories or styles) for stores and categories. It achieves this by assessing their calculated revenue plans and average selling prices (ASP), enabling flexible expansion or contraction of display capacity to align with current business goals and potential.

Why This Matters: This process ensures that your valuable display space is actively managed and responsive to demand. By providing data-driven insights into where to expand product offerings (more styles/categories) or where to consolidate, it directly supports maximizing sales by showcasing a relevant and attractive assortment. This leads to more efficient inventory investment, reduced missed sales opportunities, and enhanced overall store profitability.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Calculated Revenue Plans:** To understand the projected financial contribution for each store and category, serving as a key driver for potential display expansion.
- **Average Selling Price (ASP):** To convert revenue plans into expected unit quantities, which are essential for understanding how much product needs to be displayed.
- **Historical Sales Data (for ASP calculation):** To accurately determine the average price at which products have sold, providing a realistic basis for current ASP.

Product Data

- **Category and Style Information:** To define logical groupings for display planning and apply increment ratios at the correct level of granularity.
- **Attribute Group (AG) Data:** To allow for deeper analysis of product characteristics when calculating revenue and ASP, ensuring precise adjustments.

- **Existing Planogram Data:** The current definitions for OW (Optimum Width), representing the existing number of styles or options on display.

Operational Data

- **Store Information:** To identify the channel and region of each store, allowing for regional and channel-specific analysis of inventory and demand.
- **Warehouse Inventory Levels (Allocable):** The truly available stock in the central warehouse, which indicates the overall supply capacity for potential planogram expansions.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--|---------------|--|--|
| Revenue Plan Consideration Period | 30 days | The timeframe for which the target revenue plans are considered when calculating potential display capacity. | Adjust to align with new financial planning cycles or to react more quickly to recent revenue forecasts. |
| Average Selling Price (ASP) Calculation Period | 90 days | The number of past days of sales data used to determine the average selling price for products within categories and attributes. | Shorten for fast-moving fashion to reflect recent pricing, or lengthen for stable items to account for broader trends. |
| Planogram Expansion Sensitivity | 1.0 | A numerical factor that controls how aggressively the system proposes increasing OW for high-performing categories. | Increase if you want to be more proactive in expanding displays; decrease for more conservative adjustments. |
| Minimum Current OW for Adjustment | 1 unit | The minimum number of categories or styles (OW) currently on a planogram required for the system to | Set higher if you only want to adjust well-established planograms; lower to include newly set up displays. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|---|----------------|
| | | consider proposing an increment or decrement. | |

Note: See More Details for additional settings related to multi-attribute group analysis and regional roll-up logic.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

    A[Revenue Plans, Sales Data, Current Planogram OW] -->
    B[Calculate Expected Units (Quantum)]
    B --> C[Compute Average Selling Prices (ASP)]
    C --> D[Determine Planogram Increment Ratio]
    D --> E[Output: Adjusted OW Guidance]
  
```

1. Start with → **Calculated Revenue Plans for stores/categories, historical Sales Data for ASP, and the current definitions of OW (Optimum Width) from existing planograms.**
2. Then → **Calculate Expected Units (Quantum)** by dividing the revenue plan by the average selling price for each category and attribute group, identifying the target quantity of units.
3. Next → **Compute Average Selling Prices (ASP)** at both store-category and channel-category levels, using recent sales data.
4. Next → **Determine the Planogram Increment Ratio** by comparing the calculated expected units (quantum) to the current OW and factoring in overall regional inventory supply.
5. Finally → **Generate Output: Adjusted OW Guidance**, providing specific recommendations for how much to expand or contract the number of styles on displays.

5. Step-by-Step Actions

1. **Calculate** expected units (quantum) from revenue plans.
 - What we do: *For each store, category, and Attribute Group (AG), we take the projected revenue plan and divide it by the average selling price (ASP) to determine the target number of units, or “quantum,” that should be sold to achieve that revenue.*
 - Why we do it: *This converts a financial goal into a tangible unit quantity, providing a clearer measure of display capacity needs.*
2. **Compute** average selling prices (ASP).
 - What we do: *We analyze recent sales data to calculate the average selling price for products at the store-category and then broader channel-category levels.*
 - Why we do it: *Accurate ASP is crucial for translating revenue plans into realistic unit targets, ensuring that expected sales match pricing realities.*

3. **Create** Attribute Group (AG) mappings.
 - What we do: *We establish connections between individual products' AGs and their broader attributes (e.g., color, material) to enable flexible grouping and analysis of sales and planogram data.*
 - Why we do it: *This ensures that increment ratios can be applied consistently across similar product types and categories.*
4. **Calculate** the Planogram Increment Ratio.
 - What we do: *For each store and category, we determine an increment ratio. This ratio considers the store's expected units (quantum), its current OW (number of styles on the planogram), and the overall supply-to-demand ratio for that category across the entire region.*
 - Why we do it: *This provides a nuanced metric that indicates the potential for display expansion (or consolidation) based on both store-specific performance goals and regional inventory availability.*
5. **Generate** output for allocation.
 - What we do: *We compile the calculated expected units (quantum) and the determined increment ratios into a structured output, ready to be consumed by the main Allocation module.*
 - Why we do it: *This translates complex calculations into clear guidance for subsequent steps, ensuring OW adjustments are data-driven.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- A clothing store in a major city, Store 101, has a revenue plan for "Women's Dresses" of \$100,000 for the next period.
- The current average selling price (ASP) for "Women's Dresses" in Store 101, based on recent sales, is \$200.
- The existing planogram for "Women's Dresses" in Store 101 is currently configured for an OW of 25 different styles.
- Across the entire region, the allocable warehouse inventory for "Women's Dresses" is 10,000 units.
- The combined expected units (quantum) for "Women's Dresses" across all stores in the region is 8,000 units.

What Happens

1. First, the system calculates the "expected units" (quantum) for "Women's Dresses" in Store 101: $\$100,000 \text{ (Revenue Plan)} / \$200 \text{ (ASP)} = 500 \text{ units}$. This means Store 101 aims to sell 500 units of dresses.
2. Next, it calculates a preliminary ratio for Store 101: $500 \text{ units (Expected Units)} / 25 \text{ styles (Current OW)} = 20 \text{ units per style}$. This indicates that based on its revenue plan, Store 101 needs to sell 20 units for each of its current dress styles.
3. Then, the system considers the regional supply context. The regional supply-to-demand ratio is calculated: $10,000 \text{ units (Regional Allocable Inventory)} / 8,000 \text{ units (Regional Expected Units)} = 1.25$. This means there's 25% more supply than demand regionally.

4. Finally, these factors are combined to determine the incrementRatio for “Women’s Dresses” in Store 101. A higher calculated ratio indicates greater potential for OW expansion, reflecting strong revenue plans and ample regional supply.

Final Outcome

The Planogram Incrementer module outputs a specific incrementRatio for “Women’s Dresses” in Store 101. This ratio, which will likely be greater than 1.0 in this scenario, will then be used by the Allocation module to dynamically adjust the recommended OW for “Women’s Dresses” in Store 101. For example, if the ratio signals significant potential, the Allocation module might recommend expanding the OW from 25 to 30 styles, enabling Store 101 to offer a broader, more appealing assortment of dresses to meet its revenue targets.

Key Takeaway: By linking revenue goals and inventory supply to dynamic OW adjustments, this module ensures displays are always optimized for market demand and product availability, maximizing sales potential.

7. Results

This step produces:

Main Outputs

- **Calculated Expected Units (Quantum):** The target number of units for each store, category, and Attribute Group (AG), derived from revenue plans and ASP.
- **Planogram Increment Ratios:** A specific ratio for each store-category and store-AG combination, indicating the recommended adjustment (expansion or contraction) to OW (Optimum Width).
- **Average Selling Prices (ASP) by Level:** The computed ASPs at various granularities (store-category, channel-category), used for internal calculations.

Business Reports

- **Display Capacity Adjustment Report:** A report for store managers and visual merchandisers, providing insights into which categories or styles in their stores have the potential for OW expansion or may need consolidation.
- **Revenue vs. Display Space Potential Report:** A summary for merchandising and planning teams, correlating revenue forecasts with the physical capacity of displays, highlighting growth opportunities or areas of inefficiency.
- **Regional Supply Impact Analysis:** Data for inventory teams showing how regional inventory levels might influence proposed planogram adjustments, aiding in long-term supply chain planning.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed for revenue plans or ASP | The system will use the most recent available data or fallback to historical averages, but will flag an alert indicating incomplete information, potentially leading to outdated or inaccurate increment ratios. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| Zero or unusually low ASP for a category/AG | If ASP is zero or very low (e.g., due to data errors or extreme discounting), the calculated “expected units” (quantum) could be disproportionately high, leading to unrealistic increment ratios. | Investigate the sales data and ASP calculation for anomalies. Adjust discount filters or implement a minimum ASP threshold if such data skews results. |
| Current OW (planogram stocks) is zero for a category | If a category has no existing OW defined, the system cannot calculate a ratio against current capacity and might default to a base increment or omit it from adjustment. | Ensure all relevant categories have baseline OW definitions in your planogram data. If new, use proxy data or manual overrides for initial planning. |
| Regional allocable inventory is extremely low or zero | If there’s no available stock regionally, the calculated incrementRatio might become very low, signaling no potential for expansion even if demand is high. | Expedite reordering or production for these high-demand categories. Review and optimize regional distribution strategies. |
| Revenue plan significantly overestimates or underestimates actual sales | An inaccurate revenue plan will lead to an incorrect quantum (expected units), resulting in proposed OW adjustments that do not align with true demand. | Regularly review and refine your revenue forecasting methods to ensure accuracy and alignment with market realities. |

9. Next Steps

Before This Step

- **Revenue Computation (Chapter 8):** Precise daily revenue metrics must be calculated for each store-style and store-story combination, providing the foundational revenue plans.
- **Sales Data Preparation (Chapter 6):** Raw transactional sales data must be thoroughly cleaned and processed, and a reliable Rate of Sale (ROS) computed, forming the basis for ASP calculations.
- **Data Preparation (Chapter 3):** All foundational input data, including raw warehouse inventory levels, store classifications, and detailed product attributes (AG), must be accurately loaded and validated.
- **Warehouse Inventory Preparation (Chapter 4) & Warehouse Reserve Management (Chapter 5):** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock.

After This Step

- **Allocation (Chapter 10):** The calculated “expected units” (quantum) and incrementRatio for OW (Optimum Width) are crucial inputs for the main Allocation module, which will then use this guidance to make final decisions on product placement and quantities.
- **Store Operations Implementation:** Store teams will receive the final, optimized display plans (influenced by this module’s OW adjustments) and physically arrange products according to the recommendations.

10. Common Questions

Q: How does this module help my store decide how many *different kinds* of products to display? A: This module looks at your store’s sales goals and average prices to figure out how many units of a certain type of product (like “dresses”) you should ideally sell. It then compares that to how many different styles you currently have on display. By also looking at how much stock is available in the warehouse, it suggests whether you should offer *more* styles (expand your OW) or *fewer* to be more focused.

Q: What does “increment ratio” actually mean in simple terms? A: Think of it as a score. If the score is high, it means your store and category have strong potential to display more variety (more styles or categories) because your sales goals are high and there’s enough product in the warehouse. If the score is low, it suggests less opportunity for expansion, or even a need to streamline your offering.

Q: Can this process lead to us showing *fewer* styles if sales are low? A: While the calculation primarily identifies potential for *expansion*, if your revenue plans or average selling prices for a category are very low, the resulting “expected units” (quantum) will be small. This means the incrementRatio will also be low, signaling to the subsequent Allocation module that there isn’t much potential for expanding the OW for that category, and it might prioritize reducing styles or consolidating.

Q: How often does this module run to keep our displays up-to-date? A: This module runs as part of the overall storyWiseDisplay process, which can be configured to run frequently—typically daily or weekly. This ensures that your planogram targets are always based on the most current revenue plans, sales performance, and available inventory.

Q: Why is it important to know how much stock is in the warehouse for this module? A: It's vital because even if your store has high sales goals for a category, you can't display more styles if the products aren't available in the warehouse to replenish them. By factoring in regional warehouse inventory, this module ensures that recommendations for expanding your display options are realistic and actionable.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Dynamically adjusts OW (Optimum Width) targets based on revenue plans, ASP, and supply.
- **Main Input:** Calculated revenue plans, average selling prices (ASP), current OW definitions, and allocable warehouse inventory.
- **Main Output:** Calculated “expected units” (quantum) and a Planogram Increment Ratio for OW adjustments.
- **Warning Signs:** Recommendations to significantly expand OW for categories with little available inventory, or conversely, no expansion for high-demand, well-stocked categories.
- **Key Settings:** Revenue Plan Consideration Period, Average Selling Price (ASP) Calculation Period, and Planogram Expansion Sensitivity.

Need Help? Contact your merchandising analytics or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 13: Output Generation

Last Updated: 2025-05-23

1. Business Objective

This module is responsible for generating and persisting the final outputs of the `storyWiseDisplay` allocation process, translating complex allocation decisions into clear, actionable reports for various stakeholders.

Why This Matters: This step ensures that all the careful analysis and strategic allocation work performed by previous modules culminates in practical, understandable outputs. By detailing exactly what products should be where, accounting for inventory movements, and providing insights into display health, it enables efficient execution of display plans, minimizes manual effort, and provides transparency on inventory utilization, ultimately maximizing sales and display effectiveness.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Rate of Sale (ROS):** To provide context for product performance within output reports, detailing the speed at which products are selling.
- **Units Sold:** To inform historical sales context included in comprehensive reports.

Product Data

- **Product Attributes (AG):** Details like color, material, and brand, which help categorize products within detailed output reports.
- **SKU Information:** Unique identifiers for each product, essential for fanning out parent-level allocations to individual items.
- **Optimum Depth (OD):** The ideal number of unique items to stock, used to report on target quantities versus actual allocations.
- **Optimum Width (OW):** The ideal number of different categories or styles, used to report on the breadth of product offerings.
- **Ideal Size Set (ISS):** The preferred combination of sizes, detailed in output reports to ensure accurate size distribution.

- **Parent/Child SKU Relationships:** To correctly fan out allocation decisions from a high-level parent (e.g., a style) to its individual child products (e.g., specific sizes or colors of that style).

Operational Data

- **Allocation Decisions:** The precise quantities and locations determined by the main Allocation module (from Chapter 10), which are the core inputs for generating the final outputs.
- **Current Store Inventory Levels:** Accurate counts of products currently on hand in each store, used to report on inventory movements and fill rates.
- **Warehouse Inventory Levels:** Accurate counts of products in the central warehouse, used to report on transfers and remaining available stock.
- **Reserved Warehouse Quantities:** Amounts of warehouse stock conceptually held back for strategic purposes (from Chapter 5), which need to be reflected in available inventory reports.
- **Goods-In-Transit (GIT) and Open Orders Data:** Information about products currently en route or committed to customers, which may be factored into inventory levels reported.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|------------------------------------|---------------|--|--|
| Report Granularity | SKU Level | Determines the level of detail for reports (e.g., individual SKUs, aggregated by style, or category). | Adjust to align with the reporting needs of different stakeholders, from detailed store execution to high-level merchandising overviews. |
| Include In-Transit Data in Reports | True | If enabled, goods already shipped to stores or orders in process are counted as available stock in output reports. | Disable if you only want reports to reflect physically on-hand stock and disregard pending arrivals for reporting purposes. |
| Overstock Reporting Threshold | 1.5x OD | The multiplier applied to Optimum Depth (OD) that defines when a product is considered “overstocked” for reporting purposes. | Adjust if your business defines overstocking more or less stringently, impacting how items are flagged in overstock reports. |

| Setting Name | Default Value | Description | When to Change |
|----------------------|-----------------------------|--|---|
| Fill Rate Definition | Units Allocated / Target OD | Specifies how the “fill rate” metric is calculated for output reports, influencing how display completeness is assessed. | Change if your business uses a different standard for measuring how well a display is stocked (e.g., based on style count vs. SKU count). |

4. Process Flow

Here’s how information moves through this step:

flowchart LR

A[Final Allocation Decisions & Inventory Data] --> B[Fan Out to Child SKUs]

B --> C[Process Warehouse Allocations & Reserves]

C --> D[Prepare Denormalized Reports]

D --> E[Persist Final Outputs]

1. Start with → **The precise allocation decisions (which products go where and in what quantities) from the Allocation module, along with up-to-date inventory levels.**
2. Then → **Fan Out to Child SKUs** by breaking down higher-level allocations (e.g., for an entire style) into specific quantities for each individual SKU (e.g., a specific size and color of that style).
3. Next → **Process Warehouse Allocations & Reserves** by accurately recording and reporting on products moved from the warehouse to stores, and detailing any quantities that remain reserved.
4. Next → **Prepare Denormalized Reports** by compiling all relevant data points into easy-to-understand, consolidated reports for various business users.
5. Finally → **Persist Final Outputs** by saving these reports and data snapshots for immediate use, historical tracking, and integration with other business systems.

5. Step-by-Step Actions

1. **Fan out** parent-level allocations to individual SKUs.
 - What we do: *We break down the allocation quantities determined for a broader “parent” product (like a style) into specific amounts for each individual “child” SKU (like a specific size or color within that style), ensuring every unique item has a clear allocation.*
 - Why we do it: *This provides granular, actionable instructions for store teams, telling them precisely how many of each unique item to display.*
2. **Process** and record warehouse allocations.
 - What we do: *We detail all transfers of products from the central warehouse to specific stores, recording the quantities moved and updating warehouse inventory levels to reflect these outgoing shipments.*

- Why we do it: *This provides clear instructions for warehouse operations and ensures that inventory records accurately reflect available stock after allocations are made.*
3. **Process** and record reserved quantities.
 - What we do: *We ensure that any warehouse stock that was strategically reserved for other purposes (e.g., online exclusives, future promotions) is clearly identified and reported, distinguishing it from general allocable stock.*
 - Why we do it: *This provides transparency on strategic inventory holdings and prevents the accidental over-allocation of stock already earmarked for high-priority initiatives.*
 4. **Prepare** denormalized output reports.
 - What we do: *We gather all the allocation decisions, inventory movements, and product data to create comprehensive, easy-to-read reports. These reports might include details on allocation types, inventory movements, achieved fill rates for displays, and instances of potential overstocking.*
 - Why we do it: *This provides business users with a holistic view of the display plan and its implications, enabling informed decision-making and efficient execution.*
 5. **Persist** the final outputs.
 - What we do: *We save all the generated reports and data into a permanent storage system, making them accessible for immediate action, historical analysis, and integration with other business intelligence tools.*
 - Why we do it: *This ensures that the valuable insights and actionable plans generated by storyWiseDisplay are readily available to all relevant teams and can be used for ongoing performance monitoring and future planning.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

- The Allocation module (Chapter 10) determines that a “Summer Athleisure” display in Store 101 should feature 25 units of the “Core Black Leggings” style (a parent-level allocation). This style has three child SKUs: Small (SKU A), Medium (SKU B), and Large (SKU C), with an Ideal Size Set (ISS) ratio of 2:3:2.
- The Allocation module also suggested sending 15 units of SKU A from the warehouse to Store 101, and pulling back 7 units of a “Seasonal Print Jacket” (SKU D) from Store 101 to the warehouse.
- The warehouse has 100 units of SKU A, with 10 units of SKU A already marked as reserved for online sales.
- Store 101 currently has 8 units of SKU A on hand.

What Happens

1. First, the system takes the parent-level allocation for “Core Black Leggings” (25 units) and fans it out to the individual child SKUs based on the 2:3:2 ISS ratio: 7

- units of SKU A, 11 units of SKU B, and 7 units of SKU C are allocated to Store 101's display.
2. Next, it processes the warehouse allocation: It confirms the transfer of 15 units of SKU A from the warehouse to Store 101, reducing the warehouse's available stock. It then reports the 10 units of SKU A as "reserved."
 3. Then, it processes the pullback: It records the instruction to move 7 units of SKU D from Store 101 back to the warehouse, increasing the warehouse's stock and decreasing Store 101's stock.
 4. Finally, it compiles this information into various reports:
 - An "Optimized Display Plan" showing exactly which SKUs to display and their quantities in Store 101.
 - An "Inventory Transfer Recommendation" detailing the 15 units of SKU A to send and the 7 units of SKU D to receive for the warehouse.
 - A "Warehouse Stock Report" indicating the remaining free stock of SKU A (after 15 units transferred) and noting the 10 reserved units.
 - A "Fill Rate Report" for the "Summer Athleisure" display showing how well its Optimum Depth (OD) is met for each SKU.
 - An "Overstock Report" for SKU D if its remaining quantity in Store 101 still exceeds the overstock threshold.

Final Outcome

Store 101 receives a precise, SKU-level plan detailing the exact quantities of each size of "Core Black Leggings" to display. The warehouse team gets clear instructions for sending and receiving stock. All teams receive reports that provide full transparency on where products are, how much is moving, and how well displays are being stocked, enabling efficient, data-driven operations.

Key Takeaway: This module transforms complex allocation decisions into clear, actionable reports and records, empowering store teams and inventory management to efficiently implement optimized display strategies.

7. Results

This step produces:

Main Outputs

- **Final SKU-Level Allocation Plan:** A detailed list of which individual products (SKUs), in what quantities, should be placed on each specific display or category area in every store.
- **Detailed Inventory Movement Records:** Comprehensive data on all recommended transfers from warehouse to stores, and pullbacks from stores to the warehouse.
- **Reserved Inventory Summary:** A clear breakdown of which SKUs and quantities are reserved in the warehouse, providing transparency on strategic holdings.
- **Denormalized Output Reports:** Consolidated reports detailing allocation types, current inventory, fill rates for displays, and overstocking situations.

Business Reports

- **Store Display Execution Plan:** A clear report for store managers and visual merchandisers, showing the planned display changes and exact product placements for their store.
- **Inventory Logistics Manifest:** A detailed manifest for warehouse and logistics teams, outlining all required stock transfers and pullbacks for execution.
- **Display Performance & Health Report:** A summary for merchandising and planning teams, showing metrics like fill rates, overstocking, and overall display effectiveness.
- **Inventory Reconciliation Report:** Data for finance and inventory auditing teams to reconcile system-generated stock movements with physical counts.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|--|
| Missing or delayed data feed | The system will use the most recent available data for inventory and allocation decisions, but will flag an alert indicating incomplete information, potentially leading to outdated or inaccurate reports. | Contact the data source team immediately to resolve the feed issue and manually verify critical data points if the delay persists. |
| Discrepancy between parent and child SKU quantities | If the sum of allocated child SKUs does not perfectly match the parent style's allocated quantity (e.g., due to rounding), the system will allocate based on child SKU details, but this may cause minor inconsistencies. | Review the rounding rules for Ideal Size Set (ISS) allocations. Investigate if the parent-to-child SKU mapping is consistently defined and correctly applied. |
| Warehouse stock reported is negative after allocation | If an error in prior steps caused more to be allocated than physically available, the system might report negative available stock for outgoing transfers. | Immediately investigate the root cause of the over-allocation in preceding modules. Halt further transfers until stock can be physically verified and corrected. |
| Reports show unexpected overstocking for high-demand items | If a best-selling item is flagged as overstocked in reports, it suggests a miscalculation in Optimum Depth (OD) or an issue with stock data. | Review the Overstock Reporting Threshold setting and the Optimum Depth (OD) calculation for that product. Verify raw inventory data accuracy. |
| Fill rate reports consistently show | This could indicate that allocations are | Review the allocation logic and the Fill Rate Definition |

| Situation | What Happens | What You Should Do |
|---------------------------------|--|--|
| low numbers despite allocations | insufficient to meet display targets, or that the Fill Rate Definition is too stringent. | setting. Ensure display capacities and Optimum Depth (OD) targets are realistic. |

9. Next Steps

Before This Step

- **Allocation (Chapter 10):** The core allocation engine must have run to precisely determine which products (SKUs) go to which stores and in what quantities, providing the raw decisions for this module.
- **Warehouse Inventory Preparation (Chapter 4) & Warehouse Reserve Management (Chapter 5):** Warehouse inventory levels must have been adjusted for operational inefficiencies and strategic reservations, providing the true allocable stock to be reported on.
- **Data Preparation (Chapter 3):** All foundational input data, including raw inventory levels (store and warehouse), detailed product attributes (AG, ISS), and parent/child SKU relationships, must be accurately loaded and validated.

After This Step

- **Store Operations Implementation:** Store teams need to receive the generated display execution plans and physically arrange products according to the recommendations. They will require clear instructions and visual guides from the reports.
- **Inventory Management Fulfillment:** Warehouse and logistics teams need to process and execute the recommended inventory transfers and pullbacks outlined in the manifests to ensure products arrive at stores or are returned to the warehouse as planned.
- **Performance Monitoring:** Store managers and merchandising teams should continuously monitor sales performance against the implemented display plans, utilizing the generated reports to assess effectiveness and identify any areas for further optimization or data correction.

10. Common Questions

Q: How does this module help my store physically arrange products on displays? A: This module takes all the smart calculations and creates a detailed “Store Display Execution Plan.” This plan tells your store team exactly which product (down to the individual SKU, like a specific size and color) should go on which display and in what quantity, making it easy to implement the optimized layout.

Q: What kind of information can I see in the “Overstock Report”? A: The “Overstock Report” highlights products in your store that have quantities significantly exceeding their ideal display amount (Optimum Depth - OD). This helps you quickly identify items that might be tying up valuable display space and capital, prompting action like promotions or transfers.

Q: How does it ensure the reports are easy for non-technical staff to understand? A: This module specifically “denormalizes” the data and uses business-friendly terms to create reports. It consolidates complex information into clear tables and summaries, designed for store managers, merchandisers, and inventory teams who need actionable insights without technical jargon.

Q: Can these outputs be integrated with our existing business systems, like our ERP or BI tools? A: Yes, absolutely. The module generates and persists its outputs in a structured format, making them readily available for integration with your existing Enterprise Resource Planning (ERP) systems, Business Intelligence (BI) dashboards, or other operational tools for seamless data flow and reporting.

Q: What happens if there’s a problem with the original data inputs? Will the outputs be wrong? A: While this module aims to present data clearly, it relies on the accuracy of inputs from preceding steps. If there were issues in sales data, inventory counts, or allocation decisions, those inaccuracies would reflect in the final reports. Regular data quality checks in earlier modules are crucial for reliable outputs here.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Generates and persists the final, actionable outputs of the storyWiseDisplay allocation process.
- **Main Input:** Final allocation decisions from Chapter 10, current store and warehouse inventory levels, and all associated product data.
- **Main Output:** Detailed SKU-level allocation plans, inventory movement records, and comprehensive denormalized business reports (e.g., fill rates, overstocking).
- **Warning Signs:** Store teams consistently confused by display plans, unexpected discrepancies in inventory reports, or frequent complaints about stock availability not matching reports.
- **Key Settings:** Report Granularity, Include In-Transit Data in Reports, and Overstock Reporting Threshold.

Need Help? Contact your merchandising operations or inventory planning team for questions about this process.

Generated by AI Codebase Knowledge Builder

storyWiseDisplay FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ provides quick answers to common business questions about the storyWiseDisplay algorithm, explaining its core functions and how it optimizes product allocation.

General

Q: What is storyWiseDisplay? **A:** storyWiseDisplay is an advanced algorithm designed to optimize inventory allocation to stores by considering not just individual products but also curated product groupings called “stories.” Its goal is to maximize sales and display effectiveness.

Q: What problem does storyWiseDisplay solve for our business? **A:** It solves the challenge of ensuring the right products are in the right stores at the right time, minimizing stockouts and overstocking, and ensuring that strategic product “stories” are fully represented on the sales floor.

Q: How does storyWiseDisplay define a “story”? **A:** A “story” refers to a predefined collection of styles or products intended to be displayed together to create a specific merchandising theme or customer experience. The system uses “story-style mappings” defined during the Data Preparation phase.

Q: What do OD and OW mean in the context of this system? **A:** OD (Optimum Depth) refers to the ideal number of unique items (SKUs) to stock for a given product or style within a category or display. OW (Optimum Width) refers to the ideal number of different categories or styles to offer in a store or within a specific display.

Q: What is an ISS and why is it important? **A:** ISS stands for Ideal Size Set. It’s the best combination of sizes for a specific product to maximize sales and minimize stockouts. The storyWiseDisplay algorithm uses ISS data to ensure allocations provide the right size assortment.

Configuration & Settings

Q: Can we adjust how OD and OW targets are determined for our stores? **A:** Yes, the Planogram Incrementer module dynamically adjusts OW (planogram) targets based on revenue plans and average selling prices (ASP), allowing for flexible adjustments based on performance. OD values for styles are also calculated based on their size sets during the Revenue Computation phase.

Q: How are specific styles or products reserved from being allocated to stores? **A:** The Warehouse Reserve Management module allows for defining a percentage of stock for specific styles to be reserved in the warehouse. This ensures availability for particular channels or future initiatives, preventing them from being allocated to stores.

Q: Can storyWiseDisplay prioritize allocation for certain product types or segments? **A:** Yes, the Segmentation module identifies and categorizes styles into performance segments (e.g., “Top Seller,” “Bottom Seller”) based on sales performance. The Ranking module then uses these segments to prioritize which store-style and store-story combinations receive allocation first.

Q: How often does the storyWiseDisplay allocation process run? **A:** The Story Wise Display Orchestration module manages the entire process, including “snapshot creation,” indicating that it runs on a defined schedule to provide updated allocation recommendations. The exact frequency is configurable based on business needs.

Q: Can we set minimum stock levels for products in stores? **A:** Yes, the Data Preparation module incorporates SKU maximum/minimum OD criteria as input. This allows the system to respect defined minimum stock levels during the allocation process.

Data & Inputs

Q: What kind of information is needed to run storyWiseDisplay? **A:** The Data Preparation module requires foundational inputs such as store lists, season information, current stock levels (store and warehouse), style pack sizes, SKU OD criteria, replenishment cycle days, warehouse planogram data, ISS and OW data, and crucially, “story” related inputs like story-style mappings and minimum OW per category within a story.

Q: How does storyWiseDisplay use sales data? **A:** The Sales Data Preparation module processes raw transactional sales data, cleans it, and computes the ROS (Rate of Sale) for each SKU. This ROS is a critical input for demand forecasting and informs allocation decisions, helping identify priority sales data for OD and OW calculations.

Q: Where does the system get current inventory levels from? **A:** Inventory data, both for stores and warehouses, is loaded and prepared by the Data Preparation module. The Warehouse Inventory Preparation module then further adjusts warehouse stock by accounting for inefficiency and reservations.

Q: How does the system handle “bad” or unusual sales data? **A:** The Sales Data Preparation module specifically cleans sales data by removing liquidation sales and filtering out “broken” data, such as sales from periods with very low product availability, to ensure accurate ROS computation.

Q: What are “read-only” store-styles? **A:** During the Revenue Computation phase, certain store-style combinations can be identified as “read-only,” meaning they are not eligible for replenishment or active allocation adjustments by the system. This allows for specific business rules to be applied to certain product placements.

Process & Logic

Q: How does storyWiseDisplay decide which products go to which stores? **A:** The core Allocation module orchestrates multiple allocation “iterations.” It uses the Ranking module’s prioritization (based on revenue and inventory) and applies various business rules and constraints (like OD limits, OW requirements, and planograms) to determine the final SKU allocations.

Q: What is the role of “ranking” in the allocation process? **A:** The Ranking module prioritizes store-styles and store-stories based on their computed revenue per day, inventory levels, and PSA (Product/Size Availability) percentages. This ensures that higher-performing or more critical assortments are addressed earlier in the allocation process, receiving inventory first.

Q: How does the system ensure stores have a good mix of products and styles? **A:** The Allocation module, particularly during its OW (Optimum Width) adjustments and through the Planogram Incrementer module, works to ensure that stores maintain the ideal number of different categories or styles. This helps in achieving balanced assortments and full “story” displays.

Q: What does “pullback” mean in the context of allocation? **A:** “Pullback” refers to the process of identifying and potentially reallocating excess or underperforming inventory from stores back to the warehouse or to other stores where demand is higher. This is one of the business rules that can be applied during the Allocation module’s iterations.

Q: How does the system use ROS and revenue in its decisions? **A:** ROS (Rate of Sale), computed by the Sales Data Preparation module, indicates how fast products are selling. The Revenue Computation module then uses this, along with other data, to calculate daily revenue for store-style combinations and AGs (Attribute Groups). These revenue metrics are crucial for ranking products and informing allocation strategies.

Outputs & Results

Q: What kind of reports does storyWiseDisplay generate? **A:** The Output Generation module produces comprehensive reports detailing allocation types, inventory movements, fill rates, and overstocking situations. These denormalized reports are designed for various stakeholders to provide clear insights into the allocation outcomes.

Q: Can I see which items were reserved in the warehouse? **A:** Yes, the Output Generation module processes and records warehouse allocations, including quantities that were specifically reserved by the Warehouse Reserve Management module, making this information available in the output reports.

Q: How can I tell if a store is overstocked on certain products? **A:** The Output Generation module’s reports include details on “overstocking situations,” providing visibility into instances where allocated quantities exceed optimal levels or existing inventory is too high.

Q: Does the system show why a specific allocation was made or not made? **A:** While the reports don't provide a direct "reason code" for every single SKU, they detail "allocation types" and can include "allocation remarks" from the Iteration Runner. This provides insights into the rules and constraints applied during the allocation process.

Q: What is "fill rate" in the output reports? **A:** "Fill rate" in the storyWiseDisplay output reports indicates the percentage of requested or ideal inventory that was actually fulfilled during the allocation process. It helps assess how effectively the system was able to meet the demand or target stock levels.

Error Handling & Edge Cases

Q: What happens if the input data is incomplete or incorrect? **A:** The Input Data Validation module performs comprehensive checks on all input data. It flags critical missing data, consistency issues, and business rule violations by generating warnings and errors. This helps prevent inaccurate allocation results.

Q: How does the system handle situations with very low overall stock for a product? **A:** The Warehouse Inventory Preparation and Warehouse Reserve Management modules account for available stock and reservations. In low stock scenarios, the Allocation module will prioritize allocations based on the established ranking and business rules, potentially leading to lower fill rates for some stores, which will be reported in the outputs.

Q: Are there situations where a product might not be allocated even if it's available in the warehouse? **A:** Yes, this can happen if: * The product is part of a "read-only" store-style not eligible for replenishment. * The product stock is reserved in the warehouse for specific channels or initiatives. * Allocating the product would violate OD or OW limits or other defined business constraints for a particular store or story.

Q: What if a store has very unusual or "spiky" sales data? **A:** The Sales Data Preparation module includes logic to clean and filter "broken" sales data. While extreme spikes might influence ROS in the short term, the system aims to use reliable sales history for OD and OW calculations and overall allocation decisions.

Q: How are "Top Seller" and "Bottom Seller" products handled differently by the system? **A:** The Segmentation module categorizes styles. The Ranking module then uses these segments to prioritize allocation. Top Sellers are generally prioritized for inventory to ensure availability and maximize sales, while Bottom Sellers might be considered for pullbacks or reduced allocations to optimize space and minimize overstock.

Need More Help?

Contact the Merchandising Support Team for further assistance or detailed inquiries.

storyWiseDisplay Documentation

Version: 1.0 Last Updated: 2025-05-23

2. Executive Summary

storyWiseDisplay is an advanced inventory allocation algorithm designed to optimize product distribution across retail channels. It intelligently balances stock levels, maximizes sales, and aligns inventory with strategic “story” initiatives by leveraging detailed sales performance and operational data. The system ensures the right products are in the right stores at the right time, leading to improved ROS and reduced stockouts.

3. Architecture Overview

The storyWiseDisplay system is orchestrated by a central module that defines a sequential pipeline for inventory allocation. Data preparation, warehouse adjustments, and sales analysis precede core computations like revenue and ranking. The central allocation module iteratively applies rules, leveraging an iteration runner and dynamically adjusting planograms. Input data integrity is ensured by a validation module, while product segmentation guides allocation strategies. Finally, an output module generates comprehensive reports.

graph TD

subgraph Data Flow

```
A[Input Data Validation] --> B(Data Preparation);
B --> C(Warehouse Inventory Preparation);
B --> D(Warehouse Reserve Management);
B --> E(Sales Data Preparation);
B --> F(Revenue Computation);
B --> G(Ranking);
E --> F;
E --> K(Segmentation);
F --> G;
F --> H(Allocation);
G --> H;
C --> H;
D --> I(Output Generation);
H --> I;
H --> J(Iteration Runner);
H --> L(Planogram Incrementer);
J --> H;
L --> H;
```



```

    K --> F;
    K --> J;
end
subgraph Orchestration
    O[Story Wise Display Orchestration] --> B;
    O --> C;
    O --> D;
    O --> E;
    O --> F;
    O --> G;
    O --> H;
    O --> I;
end

A:::input
B:::processor
C:::processor
D:::processor
E:::processor
F:::processor
G:::processor
H:::processor
I:::output
J:::subprocessor
K:::processor
L:::processor
O:::orchestrator

classDef input fill:#f9f,stroke:#333,stroke-width:2px;
classDef processor fill:#ccf,stroke:#333,stroke-width:2px;
classDef subprocessor fill:#eef,stroke:#333,stroke-width:1px;
classDef output fill:#cfc,stroke:#333,stroke-width:2px;
classDef orchestrator fill:#ffc,stroke:#333,stroke-width:2px;

```

Module Descriptions:

- **Story Wise Display Orchestration:** The primary unit that defines the execution sequence for all sub-modules, managing data preparation, sales analysis, revenue computation, ranking, allocation, and output generation.
- **Data Preparation:** Loads and prepares all foundational input data, including store lists, stock levels, style pack sizes, OD criteria, ISS, OW data, and “story” related inputs.
- **Warehouse Inventory Preparation:** Adjusts warehouse stock for allocation by accounting for inefficiency and reserving inventory based on lead times.
- **Warehouse Reserve Management:** Defines and manages the reservation of a percentage of warehouse stock for specific styles to ensure availability for particular channels or future initiatives.
- **Sales Data Preparation:** Processes raw transactional sales data, cleaning it, filtering out broken data, and computing ROS for each SKU.
- **Revenue Computation:** Calculates revenue metrics per day for store-style combinations and AGs, determines OD values, identifies read-only store-styles, and calculates aggregated inventory and PSA percentages.
- **Ranking:** Prioritizes store-styles and store-stories based on computed revenue per day, inventory levels, and PSA percentages to guide allocation decisions.

- **Allocation:** The core module that determines SKU allocation to stores, orchestrating multiple iterations and applying business rules such as OD limits and OW requirements.
 - **Output Generation:** Generates and persists the final outputs of the allocation process, fanning out parent-level allocations, processing warehouse allocations, and preparing denormalized reports.
 - **Iteration Runner:** Executes the specific allocation logic for each iteration, applying constraints, determining suggested allocations, and managing stock deductions.
 - **Planogram Incrementer:** Dynamically adjusts OW (planogram) targets for stores and categories based on revenue plans and average selling prices, ensuring flexible display capacity.
 - **Input Data Validation:** Performs comprehensive validation of all input data prior to the main `storyWiseDisplay` process, checking for missing data, consistency, and rule violations.
 - **Segmentation:** Identifies and categorizes styles into performance segments (e.g., “Top Seller,” “Bottom Seller”) based on sales performance, guiding allocation strategies.
-

4. Prerequisites & Setup

To run the `storyWiseDisplay` system, ensure the following are in place:

- **Java Version:** Java 11 or higher.
- **Build Tool:** Maven (version 3.6.3 or higher).
- **Dependencies:** All project dependencies are managed via Maven. They will be automatically downloaded during the build process.
- **Database:** Access to a relational database (e.g., PostgreSQL, MySQL) containing the raw input data as specified in the `common-data.json` schema (though schema is empty in this documentation, typically it would define database tables).
- **Cache:** A configured caching mechanism (e.g., Redis, in-memory) for intermediate data storage.

Build Commands: To build the project:

```
mvn clean install
```

Running the Orchestrator: The `Story Wise Display Orchestration` module serves as the entry point. After building the project, you can typically run the main class (often with a `main` method) associated with this module. The exact command depends on the project’s packaging (e.g., JAR) and main class definition, but it might look like:

```
java -jar target/storyWiseDisplay-1.0.jar --propertiesFilePath=/path/to/args.properties
```

Ensure the `args.properties` file is correctly configured and accessible.

5. Folder & File Structure

The project repository is structured as follows:

```
storyWiseDisplay/  
├─ module-group/           # Contains core orchestration logic  
and high-level configurations.  
├─ submodules/             # Houses the individual, specialized  
modules of the pipeline.  
├─ args.properties         # Configuration file defining global  
parameters for the system.  
├─ common-data.json        # Schema definition for shared data  
models used across modules.  
├─ outputs/                # Directory where all final  
allocation reports and generated data are stored.
```

6. Global Configuration

The `args.properties` file manages global configuration parameters for the `storyWiseDisplay` system.

(No `args.properties` table data was provided for this documentation.)

7. Shared Data Model

The `common-data.json` schema defines the structure for shared data models used throughout the `storyWiseDisplay` pipeline. This ensures consistency and interoperability between modules.

(No `common-data.json` schema data was provided for this documentation.)

8. Submodule Chapters

8.1 Story Wise Display Orchestration

This module serves as the primary orchestration unit for the entire `storyWiseDisplay` process. It defines the sequence of execution for all sub-modules, ensuring that data preparation, sales analysis, revenue computation, ranking, allocation, and output generation occur in the correct order. It also manages snapshot creation and output synchronization.

Required Information: * **Operational Data:** * Execution sequence for all sub-modules. * Snapshot configuration details. * Output synchronization parameters.

Business Settings: * **Module Execution Order:** The pre-defined sequence in which sub-modules are called to achieve the end-to-end allocation. * **Snapshot Frequency:** How often intermediate data snapshots are taken for debugging or historical analysis.

Process Flow:

graph TD

```
A[Start Orchestration] --> B{Call Data Preparation};
B --> C{Call Warehouse Inventory Preparation};
C --> D{Call Warehouse Reserve Management};
D --> E{Call Sales Data Preparation};
E --> F{Call Revenue Computation};
F --> G{Call Ranking};
G --> H{Call Allocation};
H --> I{Call Output Generation};
I --> J[End Orchestration];
```

Step-by-Step Actions: * **Initialize** the `distributionData` object. *This central object holds all relevant data for the allocation process.* * **Execute** Data Preparation. *To load and prepare foundational inputs.* * **Execute** Warehouse Inventory Preparation. *To adjust warehouse stock for allocation.* * **Execute** Warehouse Reserve Management. *To define inventory reserves.* * **Execute** Sales Data Preparation. *To process sales data and compute ROS.* * **Execute** Revenue Computation. *To calculate performance metrics.* * **Execute** Ranking. *To prioritize allocation candidates.* * **Execute** Allocation. *To determine final SKU distributions.* * **Execute** Output Generation. *To produce final reports.* * **Manage** snapshots and output synchronization. *Ensuring data integrity and timely report availability.*

Sample Scenario: A new season's allocation cycle begins. The Story Wise Display Orchestration module initiates the process. It first calls Data Preparation to load all product and store data, then Sales Data Preparation to get current ROS, followed by Revenue Computation to assess performance, then Ranking to prioritize, and finally Allocation to distribute inventory. After all allocations are made, Output Generation is called to create the final reports for review. **Key Takeaway:** The orchestrator ensures all steps run in the correct order, creating a coherent and complete allocation plan.

Results: * Successful completion of the end-to-end `storyWiseDisplay` allocation process. * Generated allocation outputs and reports. * Intermediate data snapshots for traceability.

Unusual Cases: * **Missing or delayed data feed:** If any upstream data is missing, subsequent modules may fail or produce inaccurate results. * **Module execution failure:** If a sub-module encounters an error, the orchestration might halt, requiring investigation. * **Configuration errors:** Incorrect sequence or parameters for modules can lead to logical flaws in the allocation process.

Next Steps: * The orchestration itself *is* the sequence of steps, so its “next steps” are the initiation of its sub-modules.

Common Questions: 1. How long does a full allocation run typically take? 2. Can I re-run specific modules without re-running the entire pipeline? 3. How is the execution sequence determined and modified? 4. What happens if one of the modules fails during execution? 5. How do I access the intermediate data snapshots?

Quick Reference: * **Orchestration:** Directs the flow. * **Pipeline:** The sequential execution of modules. * **Snapshot:** A point-in-time capture of data.

8.2 Data Preparation

This module is responsible for loading and preparing all foundational input data required for the storyWiseDisplay allocation process. This includes distribution store lists, season information, current stock levels (store and warehouse), style pack sizes, SKU maximum/minimum OD criteria, replenishment cycle days, warehouse planogram data, ISS (Ideal Size Set) and OW (Optimum Width) data (DAP objects), and crucially, defining “story” related inputs such as story-style mappings, minimum OW per category within a story, and category combinations.

Required Information: * **Sales Data:** * None directly from sales, but sets up the context for sales data processing. * **Product Data:** * Style pack sizes. * SKU maximum/minimum OD criteria. * ISS (Ideal Size Set) data (DAP objects). * OW (Optimum Width) data (DAP objects). * Story-style mappings (which styles belong to which marketing ‘story’). * Category combinations for stories. * Warehouse planogram data (layout details). * **Operational Data:** * Distribution store lists (active stores). * Season information (e.g., current active season). * Current stock levels (store and warehouse). * Replenishment cycle days.

Business Settings: * **Minimum OW per Category:** Defines the minimum number of categories or styles to offer within a story. * **OD Criteria:** Rules for ideal stock depth based on SKU or style.

Process Flow:

graph TD

```
A[Raw Database Inputs] --> B{Load & Clean Store Data};
B --> C{Load & Clean Product Data};
C --> D{Load & Clean Inventory Data};
D --> E{Load Story & Planogram Data};
E --> F[Prepared Foundational Data (distributionData)];
```

Step-by-Step Actions: * **Load** distribution store lists. *To identify active stores eligible for allocation.* * **Load** season information. *To filter data relevant to the current selling period.* * **Retrieve** current stock levels from stores and warehouses. *This is the starting point for inventory adjustments.* * **Load** style pack sizes. *To ensure allocations are in deliverable units.* * **Define** SKU maximum/minimum OD criteria. *To guide the target depth of each SKU in stores.* * **Load** replenishment cycle days. *To understand the frequency of stock replenishment.* * **Parse** warehouse planogram data. *To map product displays in the warehouse.* * **Load** ISS and OW data. *These DAP objects define ideal product combinations and width targets.* * **Map** styles to stories. *To align allocation with marketing initiatives.* * **Set** minimum OW per category within stories. *To ensure sufficient product representation.* * **Combine** category data. *To support aggregated analysis within stories.*

Sample Scenario: A new season's collection needs to be allocated. Data Preparation first loads the list of all active retail stores and their current stock levels. Then, it pulls in details for the new collection, including OD targets for each style (e.g., shirts need an OD of 5, pants 3), ISS guidelines (e.g., a full size run for top-performing styles), and how these styles group into 'Spring Bloom' or 'Summer Escape' stories. It also loads the replenishment cycle (e.g., weekly) and warehouse planogram to understand available space. **Key Takeaway:** This module builds the comprehensive, structured dataset that forms the foundation for all subsequent allocation decisions, ensuring consistent and relevant inputs.

Results: * Populated `distributionData` object with cleaned and structured foundational information. * Lists of distribution stores, styles, SKU OD criteria, ISS, OW targets. * Story-style mappings and category definitions ready for use.

Unusual Cases: * **Missing or delayed data feed:** Critical inputs like store lists or current stock are incomplete. * **Inconsistent OD/OW data:** Conflicting or illogical OD values for SKUs leading to allocation errors. * **Incorrect story-style mappings:** Styles are assigned to the wrong stories, misaligning inventory with marketing. * **Broken DAp objects:** Corrupted or malformed ISS/OW data affecting ideal set calculations.

Next Steps: * **Warehouse Inventory Preparation:** Consumes `skuWhStock`, `storeCatWhMap`, `warehousePlanogram` from `distributionData`. * **Warehouse Reserve Management:** Uses `distributionData` for style information and store lists. * **Sales Data Preparation:** Uses `distributionData` to filter sales data relevant to distribution stores. * **Revenue Computation:** Consumes `styleInfoMap`, `distributionStoreList`, ISS (DAp) data, OD criteria, and replenishment cycles from `distributionData`. * **Ranking:** Indirectly influences ranking by setting up fundamental distribution data. * **Input Data Validation:** This module is called *before* Data Preparation to ensure input quality.

Common Questions: 1. Where does the initial stock data come from, and how often is it updated? 2. How are new stories or collections integrated into the system? 3. What happens if an OD or OW rule is missing for a style? 4. Can we see the raw data before it's "prepared"? 5. How are ISS and OW defined and managed?

Quick Reference: * **Foundational Data:** Core inputs for allocation. * **OD:** Ideal Stock Depth. * **OW:** Ideal Stock Width/Assortment. * **Story Mappings:** Links products to marketing themes.

8.3 Warehouse Inventory Preparation

This module prepares the warehouse inventory for the allocation process. It adjusts warehouse stock by accounting for warehouse inefficiency (pieces that are hard to allocate) and reserves inventory based on lead times, particularly for online sales, to prevent over-allocation of fast-moving items or stock that might not be immediately available.

Required Information: * **Product Data:** * SKU-level warehouse stock (`skuWhStock`). * Warehouse planogram data. * **Operational Data:** * Lead times for various sales channels (e.g., online). * Warehouse inefficiency percentages (e.g., for picking or damaged stock).

Business Settings: * **Warehouse Inefficiency Factor:** A percentage reduction applied to available warehouse stock to account for unallocatable pieces. * **Online Lead Time Reservation:** A mechanism to reserve stock based on expected demand and lead times for online orders.

Process Flow:

graph TD

```
A[Initial Warehouse Stock] --> B[Apply Inefficiency Factor];
B --> C[Reserve Stock for Lead Times (Online)];
C --> D[Adjusted Warehouse Stock for Allocation];
```

Step-by-Step Actions: * **Load** current warehouse stock levels. *To get the baseline inventory.* * **Apply** warehouse inefficiency factor to stock. *To reduce the allocatable quantity by a specified percentage, accounting for items difficult to pick or damaged.* * **Reserve** inventory based on lead times, especially for online sales. *This prevents allocating stock that is committed or not yet physically available for immediate store allocation, reducing risk of over-promising fast-moving items.* * **Update** available warehouse stock for subsequent allocation. *Ensuring that only genuinely available and unreserved stock is considered.*

Sample Scenario: The warehouse has 100 units of a popular dress. Warehouse Inventory Preparation first applies an inefficiency factor of 5% (meaning 5 units are effectively unusable due to damage or misplacement), leaving 95 units. Next, it considers that 20 units are already committed to online orders with a 3-day lead time. To prevent over-allocation, these 20 units are reserved. The module then reports that only 75 units (95 - 20) are truly available for store allocation. **Key Takeaway:** This module ensures that allocation decisions are based on a realistic and immediately available warehouse inventory, preventing allocation of unfulfillable quantities.

Results: * Adjusted skuWhStock quantities in distributionData, reflecting actual allocatable inventory. * Reduced risk of over-allocating fast-moving or committed items.

Unusual Cases: * **High inefficiency factor:** If the inefficiency factor is too high, it might restrict allocations unnecessarily. * **Incorrect lead time estimation:** If lead times are underestimated, online sales might deplete stock faster than anticipated, leading to stockouts for store allocations. * **Negative stock after adjustments:** Could indicate data issues if initial stock is too low or adjustment factors are too high.

Next Steps: * **Allocation:** Consumes the modified skuWhStock from distributionData for making allocation decisions.

Common Questions: 1. How is the warehouse inefficiency factor determined? 2. What types of inventory are considered “hard to allocate”? 3. How does the system account for unexpected warehouse issues? 4. Can we see how much stock was reserved and for what reason? 5. Is the online lead time reservation dynamic or fixed?

Quick Reference: * **Warehouse Inefficiency:** Unallocatable stock percentage. * **Lead Time Reservation:** Stock set aside for future commitments. * **Allocatable Stock:** Inventory ready for distribution.

8.4 Warehouse Reserve Management

This module manages the reservation of warehouse inventory. It defines how a certain percentage of stock for specific styles should be reserved in the warehouse, preventing them from being allocated to stores and ensuring availability for specific channel requirements or future initiatives.

Required Information: * **Product Data:** * Style-level information (styleInfoMap). * **Operational Data:** * Distribution store lists. * Specific styles designated for reservation. * Reservation percentages for those styles.

Business Settings: * **Reserve Percentage per Style:** A configurable percentage of stock to hold back for specific styles. * **Reserve Flag:** A boolean flag indicating whether a style should be subject to reservation.

Process Flow:

graph TD

```
A[All Styles] --> B[Identify Styles for Reservation];
B --> C[Apply Reserve Percentage];
C --> D[Reserved Stock in Warehouse];
```

Step-by-Step Actions: * **Identify** specific styles that require warehouse reservation. *This is based on business rules or configured lists.* * **Apply** a pre-defined percentage to the total available stock for these identified styles. *This quantity is then set aside and excluded from general store allocation.* * **Update** the warehouse inventory to reflect the reserved quantities. *Ensuring that subsequent allocation modules do not attempt to distribute this protected stock.* * **Record** the reserved quantities for reporting. *To provide visibility into the rationale behind stock holdings.*

Sample Scenario: A new “Limited Edition” sneaker style (Style ID: 5001) is launched, and the business decides to reserve 10% of its warehouse stock for online flash sales next month. The Warehouse Reserve Management module identifies Style 5001. If 1,000 units are in the warehouse, it calculates 10% (100 units) to be reserved. These 100 units are then flagged as unavailable for store allocation, leaving 900 units for potential store distribution. **Key Takeaway:** This module provides a strategic control mechanism to protect critical inventory for specific business initiatives or channels, preventing its premature allocation to general store replenishment.

Results: * Reserved quantities for specific styles stored in distributionData. * Updated available warehouse stock excluding reserved items. * Visibility into reserved inventory through reports.

Unusual Cases: * **Over-reserving:** Setting the reserve percentage too high could lead to missed sales opportunities in stores. * **Under-reserving:** Not reserving enough stock could jeopardize planned future initiatives or specific channel commitments. * **Incorrect style mapping:** Reserving stock for the wrong style due to data errors.

Next Steps: * **Output Generation:** Accesses distributionData to account for reserved quantities in final reports (e.g., populating reserveQtyMap).

Common Questions: 1. Who decides which styles get reserved and why? 2. Can the reserve percentage be adjusted dynamically? 3. How does this module interact with the Warehouse Inventory Preparation's lead time reservation? 4. What happens if there isn't enough stock to meet the reserve percentage? 5. Is reserved stock tracked separately in reports?

Quick Reference: * **Reserve:** Hold back inventory. * **Strategic Stock:** Inventory for specific initiatives. * **Channel Allocation:** Prioritizing stock for different sales channels.

8.5 Sales Data Preparation

This module processes raw transactional sales data. Its key functions include cleaning the sales data by removing liquidation sales and filtering out “broken” sales data (e.g., sales from periods where product availability was low). It then computes the ROS (Rate of Sale) for each SKU (Stock Keeping Unit) across stores, which is a critical input for demand forecasting and allocation decisions. It also identifies priority sales data for OD (Optimum Depth) and OW (Optimum Width) calculations.

Required Information: * **Sales Data:** * Raw transactional sales data (units sold, price, date, store, SKU). * Liquidation sale indicators (if applicable). * Product availability data during sales periods. * **Product Data:** * SKU attributes for grouping. * **Operational Data:** * Distribution store list (from Data Preparation).

Business Settings: * **Liquidation Sales Flag:** A flag to identify and exclude sales that are part of a liquidation event. * **Broken Sales Threshold:** Criteria (e.g., low inventory during high sales) to identify and filter out unreliable sales data. * **ROS Calculation Period:** The time window over which ROS is computed (e.g., units per week).

Process Flow:

graph TD

```
A[Raw Sales Data] --> B{Remove Liquidation Sales};
B --> C{Filter Broken Sales Data};
C --> D{Compute ROS per SKU per Store};
D --> E{Identify Priority Sales};
E --> F[Cleaned Sales Data & ROS Output];
```

Step-by-Step Actions: * **Load** raw sales data. *To get all historical sales transactions.* * **Clean** sales data by removing liquidation sales. *To prevent distortions from promotional or end-of-life selling events.* * **Filter** out “broken” sales data. *To ensure that ROS calculations are based on reliable data, excluding periods where sales might have been artificially low due to out-of-stock situations.* * **Compute** ROS for each SKU across relevant stores. *This metric quantifies how quickly each item sells, essential for demand forecasting.* * **Identify** priority sales data. *To highlight sales patterns that are particularly important for OD and OW calculations, such as full-price sales or sales during key periods.*

Sample Scenario: A store sold 100 blue shirts in a week, but during 2 days of that week, the store was completely out of stock. Sales Data Preparation would identify this as “broken” sales data for those 2 days and adjust the ROS calculation or exclude that period. Separately, if a style was on liquidation, its sales would be

excluded from the ROS calculation to avoid skewing future demand forecasts. For a regular-priced, in-stock item, the module calculates its ROS (e.g., 5 units/week) and flags its sales as “priority.” **Key Takeaway:** This module ensures the quality and relevance of sales data, providing an accurate ROS which is fundamental for effective allocation.

Results: * Cleaned and filtered sales data. * Computed ROS values for each SKU by store (storeSkuRos). * Identification of prioritySales for OD/OW calculations. * These results are stored in distributionData (SWD).

Unusual Cases: * **Insufficient sales history:** New products or stores may lack enough data for accurate ROS computation. * **Extreme outliers:** Unusually high or low sales events (not liquidation) that could skew ROS if not handled. * **Incorrect ROS period:** Using too short or too long a period for ROS calculation might not reflect true selling speed. * **Inaccurate product availability data:** Leads to misidentifying “broken” sales, affecting ROS accuracy.

Next Steps: * **Revenue Computation:** Consumes cleanedSales, prioritySales, storeSkuRos from distributionData. * **Segmentation:** Uses processed and cleaned sales data (distributionData and cache.getProductSalesList()) to identify top and bottom sellers.

Common Questions: 1. How do we define “broken” sales, and can the criteria be adjusted? 2. What historical period is used for ROS calculation? 3. Are promotional sales included in ROS? 4. How does the system handle new product sales with limited history? 5. Can we see the raw sales data before cleaning?

Quick Reference: * **ROS:** Rate of Sale. * **Data Cleaning:** Removing irrelevant/unreliable data. * **Priority Sales:** Key sales for OD/OW.

8.6 Revenue Computation

This module calculates revenue metrics per day for various levels, serving as a key input for subsequent ranking and allocation decisions. It computes revenue per day for store-style combinations, rolling up revenue for AG (Attribute Group)s to identify broader sales trends. It determines OD (Optimum Depth) values for styles based on their size sets, identifies “read-only” store-styles (those not eligible for replenishment), and calculates aggregated inventory and PSA (Product/Size Availability) percentages for “store stories” (collections of products within a store).

Required Information: * **Sales Data:** * Cleaned sales data (cleanedSales, prioritySales, storeSkuRos) from Sales Data Preparation. * **Product Data:** * Style information, ISS (DAP) data, OD criteria (from Data Preparation). * AG definitions. * Segmentation data (Top/Bottom Seller) from Segmentation. * **Operational Data:** * Store lists, current inventory (distributionData). * Replenishment cycle days (from Data Preparation).

Business Settings: * **OD Calculation Logic:** Rules for determining optimum depth based on sales performance and size sets. * **Read-Only Store-Style Flags:** Criteria for marking store-styles as ineligible for replenishment. * **PSA Calculation:** Logic for determining product/size availability within store stories.

Process Flow:

graph TD

```
A[Cleaned Sales Data & ROS] --> B{Calculate Revenue/Day  
(Store-Style)};  
B --> C{Roll-up Revenue (AGs)};  
C --> D{Determine OD Values (Size Sets)};  
D --> E{Identify Read-Only Store-Styles};  
E --> F{Calculate Store Story Inventory & PSA};  
F --> G[Revenue Metrics & OD/PSA for Ranking/Allocation];
```

Step-by-Step Actions: * **Calculate** revenue per day for each store-style combination. *This provides a key performance indicator for store-level assortment effectiveness.* * **Roll up** revenue for AGs. *To identify broader trends for product groupings (e.g., all blue dresses, all size large shirts).* * **Determine** OD values for styles based on their size sets. *This ensures that the target stock depth aligns with the ISS and actual selling patterns.* * **Identify** “read-only” store-styles. *These are store-styles that, due to specific business rules (e.g., seasonal phase-out, low ROS), should not receive further replenishment.* * **Calculate** aggregated inventory and PSA percentages for “store stories.” *To assess the overall health and completeness of a particular collection within a store.*

Sample Scenario: For “Store A,” a “Floral Dress” (Style 123) sold 10 units generating \$1000 revenue over 5 days. Revenue Computation would calculate its revenue per day as \$200. If this dress is part of the “Spring Story,” the module also checks the PSA for all items in that story within Store A. If half the sizes for the Floral Dress are out of stock, its PSA would be low, indicating a need for replenishment. Based on its ROS and ISS definition, its OD might be set to 6 units. If it’s a phase-out style, it might be marked “read-only.” **Key Takeaway:** This module transforms raw sales and product data into actionable performance metrics (revenue/day, OD, PSA), crucial for intelligent prioritization and allocation.

Results: * revenuePerDay, psaPercent, sizeDepth, skuMaxDepth, and replenishmentDays attributes populated for storeStyle objects in distributionData. * storeStory data with aggregated inventory and PSA information.

Unusual Cases: * **Zero sales for a style:** If a style has no sales, its ROS and revenuePerDay would be zero, potentially leading to it being overlooked. * **Incorrect OD calculation:** Flawed OD values could lead to overstocking or understocking. * **Misidentified read-only styles:** Styles that should be replenished are marked read-only, or vice-versa. * **Inaccurate PSA:** Miscalculation of PSA could lead to poor decisions about story health.

Next Steps: * **Ranking:** Consumes revenuePerDay, PSA percentages, and OD values from distributionData to prioritize store-styles and store-stories. * **Allocation:** Uses revenueIterationFlag, revenuePerDay, psaPercent, sizeDepth, skuMaxDepth, and replenishmentDays from distributionData for allocation decisions. * **Segmentation:** (Relationship is *from* Segmentation to Revenue Computation) Segmentation results (Top Seller/Bottom Seller) are used by Revenue Computation (e.g., in removeZeroInvStoreStyles).

Common Questions: 1. How is “revenue per day” specifically calculated? 2. What factors determine if a store-style is “read-only”? 3. How does OD differ from current stock level? 4. Can we see the PSA for each story in a store? 5. How are AGs defined for revenue roll-up?

Quick Reference: * **Revenue/Day:** Key performance metric. * **OD:** Optimum Depth. * **PSA:** Product/Size Availability. * **Read-Only:** No further replenishment.

8.7 Ranking

This module ranks store-styles and store-stories based on their computed revenue per day, inventory levels, and PSA (Product/Size Availability) percentages. This ranking prioritizes which store-style and store-story combinations should receive allocation first, ensuring that higher-performing or more critical assortments are addressed earlier in the allocation process.

Required Information: * **Product Data:** * OD values, PSA percentages (from Revenue Computation). * AG definitions. * **Operational Data:** * Store-style revenue per day (from Revenue Computation). * Inventory levels (current stock). * Store story aggregated inventory and PSA (from Revenue Computation).

Business Settings: * **Ranking Criteria:** Configurable weights or thresholds for revenue per day, inventory, and PSA in determining rank. * **Ranking Order:** Ascending or descending priority based on business needs (e.g., highest revenue first, lowest PSA first).

Process Flow:

graph TD

```
A[Store-Style Revenue/Day & PSA] --> B[Score Store-Styles];
B --> C[Score Store-Stories];
C --> D[Prioritize based on Ranking Criteria];
D --> E[Ranked Store-Styles & Store-Stories];
```

Step-by-Step Actions: * **Retrieve** calculated revenue per day, current inventory levels, and PSA percentages for all store-styles. *These are the primary inputs for prioritization.* * **Calculate** a composite score for each store-style based on defined ranking criteria (e.g., higher revenue = higher score, lower PSA = higher priority). *To quantify the urgency and potential impact of allocating to each combination.* *

Rank store-styles from highest priority to lowest. *Ensuring that the most impactful allocation opportunities are addressed first.* * **Retrieve** aggregated inventory and PSA for store-stories. *To understand the overall health of product collections within stores.* * **Rank** store-stories. *This helps prioritize allocation to entire collections or themes based on their overall performance and completeness.*

Sample Scenario: Store X has “Style A” with a \$500/day revenue and 80% PSA, and “Style B” with \$200/day revenue and 30% PSA. Ranking would prioritize “Style B” higher if the business rule prioritizes PSA first to address stockouts, or “Style A” higher if raw revenue is paramount. Similarly, if “Spring Story” in Store Y has a low overall PSA of 40% but “Summer Story” has 90%, “Spring Story” would rank higher for allocation to complete the assortment. **Key Takeaway:** This module ensures that allocation efforts are strategically focused on the most critical or highest-potential store-style and store-story combinations, maximizing the impact of inventory distribution.

Results: * Sorted storeStylesList and storeStoryRankingList within distributionData. * A prioritized order for the Allocation module to process combinations.

Unusual Cases: * **Tied ranks:** Multiple store-styles or stories have identical scores, requiring a tie-breaking rule. * **Ranking criteria imbalance:** If one criterion is over-weighted, it might lead to sub-optimal allocations (e.g., always prioritizing PSA even for low-revenue items). * **Zero inventory items:** Items with zero inventory might still rank high on PSA deficiency but have no stock to allocate.

Next Steps: * **Allocation:** Consumes the sorted `storeStylesList` and `storeStoryRankingList` from `distributionData`, using this prioritized order to guide its processing.

Common Questions: 1. What are the exact formulas or rules used for ranking? 2. Can we adjust the weighting of revenue/day versus PSA in ranking? 3. How often is the ranking recalculated? 4. What happens to styles or stories that consistently rank low? 5. Can we apply different ranking strategies for different product categories?

Quick Reference: * **Prioritization:** Ordering based on importance. * **Store-Style Rank:** Individual product performance in a store. * **Store-Story Rank:** Collection performance in a store.

8.8 Allocation

This module is the core of the `storyWiseDisplay` system, responsible for determining the final allocation of SKUs to stores. It orchestrates multiple allocation “iterations,” applying different business rules and constraints in each pass (e.g., story-based allocation, OW (Optimum Width) adjustments, OD (Optimum Depth) adjustments, replenishment, pullback, etc.). It ensures that allocations adhere to defined criteria such as OD (Optimum Depth) limits, OW (Optimum Width) requirements (planograms), and other business-specific flags.

Required Information: * **Product Data:** * OD limits, OW requirements (planograms), ISS data (from Data Preparation, Revenue Computation). * Style pack sizes (from Data Preparation). * **Operational Data:** * Adjusted warehouse stock (from Warehouse Inventory Preparation). * Store-style and store-story ranks (from Ranking). * Revenue metrics (revenuePerDay, PSA) (from Revenue Computation). * Iteration constraints and flags (internal to Allocation or configured).

Business Settings: * **Allocation Iterations:** Defines the sequence and logic of multiple allocation passes (e.g., initial fill, replenishment, OD adjustment, OW adjustment, pullback). * **OD Limits:** Maximum and minimum OD targets for each SKU/style. * **OW Requirements:** Rules defining the ideal number of different categories or styles for a display or store. * **Replenishment Rules:** Logic for reordering stock to stores based on sales and OD. * **Pullback Rules:** Criteria for pulling excess inventory from stores back to the warehouse. * **Business-Specific Flags:** Any custom rules or criteria influencing allocation.

Process Flow:

graph TD

```
A[Ranked Store-Styles & Stories] --> B{Iterate Allocation Passes};
```

```
B --> C[Call Iteration Runner (Apply Constraints)];
```

```
C --> D{Call Planogram Incrementer (Adjust OW)};  
D --> B;  
B --> E[Final SKU Allocations to Stores];
```

Step-by-Step Actions: * **Initialize** the allocation process with ranked store-styles and store-stories. *To begin allocating based on business priorities.* * **Orchestrate** multiple allocation iterations (e.g., Story-based, OW adjustment, OD adjustment, Replenishment, Pullback). *Each iteration applies a specific set of rules and refines the allocation.* * **Call** the Iteration Runner for each pass. *The Iteration Runner applies detailed SKU-level allocation logic.* * **Call** the Planogram Incrementer as needed. *To dynamically adjust OW targets based on performance between iterations.* * **Adhere** to defined OD limits, OW requirements, and business-specific flags. *Ensuring that allocations are compliant with merchandising and operational strategies.* * **Determine** final allocation quantities for each SKU to each store. *This is the core output of the system.* * **Update** inventory levels (store and warehouse) dynamically as allocations are made. *Ensuring real-time stock accuracy during the process.*

Sample Scenario: The Allocation module starts. In “Iteration 1: Story-based Allocation,” it prioritizes store-stories with low PSA and allocates styles to complete those stories, respecting OD limits. If a store has 2 units of a style with OD 5, 3 units might be allocated. In “Iteration 2: OW Adjustment,” it might call Planogram Incrementer to expand OW for top-performing categories. In “Iteration 3: Replenishment,” it might allocate more units to high-RoS items that are below their OD in highly ranked stores. Finally, in a “Pullback” iteration, it might identify overstocked items in underperforming stores and flag them for return to the warehouse. **Key Takeaway:** This module is the central decision-maker, iteratively refining allocations using a comprehensive set of business rules to achieve optimal inventory distribution.

Results: * Final allocation quantities (suggestedSkuAllocation) for each SKU to each store. * Updated skuStoreStock and skuWhStock within distributionData. * Allocations written to database tables for Output Generation.

Unusual Cases: * **Insufficient warehouse stock:** Unable to meet desired OD or OW targets for all prioritized stores. * **Conflicting business rules:** OD limits clash with OW requirements, leading to sub-optimal decisions. * **Allocation loops:** Iterations do not converge, leading to inefficient processing. * **Zero allocation:** No units allocated for a style that should receive some.

Next Steps: * **Output Generation:** Reads the final allocation quantities and related metrics to create comprehensive reports. * **Iteration Runner:** Allocation calls Iteration Runner multiple times, passing constraints. * **Planogram Incrementer:** Allocation calls Planogram Incrementer to adjust OW targets.

Common Questions: 1. How many iterations are there, and what is the purpose of each? 2. How are conflicts between OD and OW rules resolved? 3. Can we manually override allocation decisions? 4. What happens if a store cannot receive its full allocation due to space constraints? 5. How are OD and OW targets dynamically adjusted?

Quick Reference: * **Iterations:** Multi-pass allocation logic. * **OD Limits:** Stock depth targets. * **OW Requirements:** Assortment width rules. * **Replenishment:** Sending stock to stores. * **Pullback:** Retrieving stock from stores.

8.9 Output Generation

This module is responsible for generating and persisting the final outputs of the storyWiseDisplay allocation process. It “fans out” parent-level allocations to individual child SKUs, processes and records warehouse allocations, including quantities reserved. It prepares denormalized output reports for various stakeholders, detailing allocation types, inventory movements, fill rates, and overstocking situations.

Required Information: * **Product Data:** * SKU-level details (for fanning out parent allocations). * **Operational Data:** * Final allocations (DistParentStyleRow, DistParentAllocWhOutputRow from Allocation). * Reserved quantities (reserveQtyMap from Warehouse Reserve Management). * Fill rates and overstocking situations (calculated during allocation).

Business Settings: * **Report Granularity:** Level of detail for output reports (e.g., SKU, style, store, story). * **Output Formats:** Desired format for reports (e.g., CSV, Excel, database tables). * **Denormalization Rules:** How data is combined for simplified reporting.

Process Flow:

graph TD

```
A[Final Allocation Data (Parent-Level)] --> B{Fan Out to Child SKUs};
B --> C{Process Warehouse Allocations};
C --> D{Integrate Reserved Quantities};
D --> E{Calculate Fill Rates & Overstock};
E --> F[Generate Denormalized Reports];
F --> G[Persist Outputs];
```

Step-by-Step Actions: * **Load** parent-level allocations determined by the Allocation module. *These are the high-level decisions on how much stock of a style goes to a store.* * **Fan out** parent-level allocations to individual child SKUs. *Translating style-level allocations into specific size/color combinations.* * **Process** and record warehouse allocations. *Detailing how much stock is moved from the warehouse.* * **Incorporate** reserved quantities. *Ensuring that reports accurately reflect stock held back due to Warehouse Reserve Management.* * **Calculate** and include fill rates and overstocking situations. *To provide performance metrics on how well allocations met demand and where excess inventory exists.* * **Prepare** denormalized output reports. *Combining data from various sources into user-friendly formats for analysis by different stakeholders (e.g., buyers, store operations, merchandising).* * **Persist** the final reports to the outputs/ directory or a database. *Making the results accessible for review and downstream systems.*

Sample Scenario: The Allocation module decides to send 10 units of “Style X” to “Store A.” Output Generation takes this parent-level allocation and “fans it out” into specific SKUs: 3 units of Style X–Small, 4 units of Style X–Medium, and 3 units of Style X–Large, based on ISS. It then records that these 10 units came from the warehouse. If 50 units of Style X were reserved by Warehouse Reserve Management, this is also noted in the output. The module then calculates that Store A’s OD for Style X was 12, so a 10-unit allocation results in an 83% fill rate. All this data is compiled into a single report, detailing quantities,

source, destination, and performance metrics. **Key Takeaway:** This module translates complex allocation decisions into clear, actionable reports and records, providing transparency and accountability for inventory movements.

Results: * Detailed SKU-level allocation reports. * Warehouse allocation records. * Reports on fill rates, reserved quantities, and overstocking. * Persisted output files for stakeholders.

Unusual Cases: * **Incomplete fan-out:** Child SKUs not correctly derived from parent allocations. * **Mismatched data:** Discrepancies between allocated quantities and reported inventory movements. * **Report generation failure:** Issues with output format or data processing preventing reports from being created. * **Very large datasets:** Performance issues when generating reports for millions of SKUs/stores.

Next Steps: * These are the final outputs, typically consumed by business users, reporting tools, or other operational systems.

Common Questions: 1. What specific reports are generated, and how often? 2. Can the output reports be customized for different departments (e.g., Finance, Merchandising)? 3. How do I access the generated output files? 4. Is there an audit trail for allocation decisions in the reports? 5. What's the difference between parent and child allocation in the reports?

Quick Reference: * **Fan Out:** Break down allocations to SKU level. *

Denormalized Reports: Consolidated data for easy analysis. * **Fill Rate:** Allocation success metric.

8.10 Iteration Runner

This module executes the specific allocation logic for each iteration defined in the system. It processes store-style combinations, applies the current iteration's constraints (e.g., OD (Optimum Depth) limits, OW (Optimum Width) checks, PSA (Product/Size Availability) benchmarks), and determines the suggested allocation for each SKU. It manages stock deductions from both store and warehouse inventory and records allocation remarks.

Required Information: * **Product Data:** * OD limits, OW requirements, ISS data (from Data Preparation). * SKU attributes and pack sizes. * Segmentation data (Top Seller/Bottom Seller) (from Segmentation). * **Operational Data:** * Current store and warehouse inventory (skuStoreStock, skuWhStock). * Iteration-specific constraints (IterationConstraint). * Store-style PSA and revenue metrics (from Revenue Computation).

Business Settings: * **Iteration Constraints:** Specific rules for each allocation pass (e.g., "only allocate to fill OD to 80%", "prioritize OW fill"). * **Allocation Benchmarks:** PSA thresholds or OD targets that trigger or limit allocation. * **Stock Deduction Logic:** How allocated stock is removed from available inventory.

Process Flow:

graph TD

```
A[Store-Style Combination & Iteration Constraints] --> B[Apply OD/OW/PSA Checks];
```


B --> C{Determine Suggested SKU Allocation};
C --> D{Deduct Stock (Store/Warehouse)};
D --> E{Record Allocation Remarks};
E --> F[Updated Inventory & Suggested Allocations];

Step-by-Step Actions: * **Receive** a specific store-style combination and the current iteration's constraints from the Allocation module. *To process one specific allocation scenario.* * **Apply** OD limits, OW checks, and PSA benchmarks. *Ensuring that the suggested allocation aligns with the business rules for the current iteration.* * **Determine** the suggested allocation quantity for each SKU within the store-style. *This is the core calculation of how many units of each item should be sent.* * **Deduct** the allocated stock from both store and warehouse inventory. *This ensures that inventory counts are updated in real-time, preventing double allocation.* * **Record** allocation remarks. *Providing a trace of why a particular allocation decision was made (e.g., "filled to OD," "OW adjustment").* * **Update** suggestedSkuAllocation for the store-style. *Making the calculated allocation visible for subsequent processing or output.*

Sample Scenario: The Allocation module calls Iteration Runner for "Store A" and "Style X" (a top seller) in the "Replenishment" iteration. The iteration constraint is to fill up to 80% of OD. Iteration Runner sees Store A has 2 units of Style X, but its OD is 10 units. So, 80% of OD is 8 units. It calculates a suggested allocation of 6 units (8 - 2). It then deducts 6 units from the warehouse stock of Style X and records a remark: "Replenishment to 80% OD." **Key Takeaway:** This module is the workhorse of the Allocation process, meticulously applying granular business rules to determine and record precise SKU-level inventory movements during each pass.

Results: * Modified skuStoreStock and skuWhStock in distributionData. * storeStyle.suggestedSkuAllocation updated with proposed allocations. * Allocation remarks recorded for traceability.

Unusual Cases: * **No stock available:** If the warehouse has zero stock, no allocation can be made, even if needed. * **Negative OD:** If OD is misconfigured as zero or negative, the allocation logic may behave unexpectedly. * **Conflicting iteration constraints:** Overlapping or contradictory rules between iterations, requiring careful sequencing. * **Micro-allocations:** Allocating very small quantities due to tight constraints, which might be inefficient for logistics.

Next Steps: * **Allocation:** Iteration Runner directly modifies stock levels and suggested allocations within distributionData, which is then used by the Allocation module for subsequent iterations. * **Segmentation:** (Relationship is from Segmentation to Iteration Runner) Segmentation results (storeStyle.getSegment() == Segment.TOP_SELLER) are used by Iteration Runner to apply specific allocation rules.

Common Questions: 1. How are iteration constraints configured for each pass? 2. What happens if an allocation would exceed OD limits? 3. Can PSA benchmarks be adjusted per category or store type? 4. How are allocation remarks used after the process runs? 5. What logic is applied if a style is both a "Top Seller" and has low ROS?

Quick Reference: * **Iteration Logic:** Specific rules for each pass. * **Stock Deduction:** Updating inventory in real-time. * **Allocation Remarks:** Traceability for decisions.

8.11 Planogram Incrementer

This module dynamically adjusts the OW (planogram) targets for stores and categories based on their calculated revenue plans and average selling prices (ASP). It determines an “increment ratio” for planogram stocks and options, allowing for flexible expansion or contraction of display capacity based on performance. This ensures that planogram targets remain relevant to current business goals.

Required Information: * **Product Data:** * Style information (e.g., ASP). * **Operational Data:** * Store and category level revenue plans. * Current planogram targets (from Data Preparation).

Business Settings: * **Revenue Thresholds:** Revenue levels that trigger an OW increment or decrement. * **ASP Influence:** How average selling price impacts the increment ratio. * **Increment Ratio Limits:** Minimum and maximum bounds for the OW adjustment ratio.

Process Flow:

graph TD

```
A[Store/Category Revenue Plans & ASP] --> B{Calculate  
Planogram Increment Ratio};  
B --> C{Adjust OW Targets Dynamically};  
C --> D[Updated Planogram Targets];
```

Step-by-Step Actions: * **Retrieve** store and category level revenue plans. *To assess performance and potential for display adjustments.* * **Obtain** average selling prices (ASP) for relevant styles. *ASP helps contextualize revenue and product value.* * **Calculate** a dynamic “increment ratio” for OW (planogram) stocks and options. *This ratio determines how much the ideal width of an assortment should expand or contract.* * **Apply** the increment ratio to adjust OW targets. *Ensuring that store displays and category allocations are aligned with current sales performance and business goals.* * **Update** the planogramIncrementRatio in distributionData. *Making this adjusted ratio available for the Allocation module.*

Sample Scenario: A specific shoe category in “Store X” consistently overperforms its revenue plan by 20%. The Planogram Incrementer identifies this and, based on the category’s ASP, calculates an OW increment ratio of 1.10 (10% increase). This means that for subsequent allocation iterations, the OW target for this shoe category in Store X will be effectively 10% higher, allowing more styles or deeper sizes to be allocated to its display. Conversely, an underperforming category might see a decrement. **Key Takeaway:** This module provides dynamic adaptability to OW targets, ensuring that physical display capacity and assortment breadth are responsive to real-time sales performance and business potential, rather than static plans.

Results: * Updated planogramIncrementRatio in distributionData. * Adjusted OW targets that influence subsequent Allocation decisions.

Unusual Cases: * **Incorrect revenue plans:** Basing adjustments on faulty revenue data can lead to poor OW decisions. * **Negative increment ratio:** If logic allows for negative adjustments, it could severely restrict OW unintentionally. * **Rapid**

fluctuations: Highly volatile revenue data could cause frequent and drastic OW changes, leading to operational instability. * **Inconsistent ASP:** Incorrect average selling prices can skew increment calculations.

Next Steps: * **Allocation:** Utilizes the updated planogramIncrementRatio from distributionData to dynamically influence OW targets and allocation decisions in subsequent iterations.

Common Questions: 1. How often are planograms incremented? 2. What is the smallest or largest increment/decrement possible? 3. Can certain categories or stores be excluded from dynamic planogram adjustments? 4. How does ASP specifically factor into the increment calculation? 5. What happens if actual sales significantly deviate from the revenue plan?

Quick Reference: * **OW Adjustment:** Dynamic planogram changes. * **Increment Ratio:** Factor for OW expansion/contraction. * **Revenue Plan:** Target sales for performance comparison.

8.12 Input Data Validation

This module performs comprehensive validation of all input data prior to the main storyWiseDisplay process. It checks for critical missing data (e.g., season lists, store/warehouse stock, story-style mappings), consistency issues, and business rule violations. It generates warnings and errors to flag potential problems that could impact the accuracy or completeness of the allocation results.

Required Information: * **All Raw Input Data:** This module is the first consumer of raw data from external sources (e.g., database tables, flat files). * DistributionStoreRow, StoryStyleListRow, StyleReservePercRow, DistSkuRosOutputRow, etc. * Season lists. * Store and warehouse stock levels. * Story-style mappings. * OD/OW/ISS definitions. * Sales data.

Business Settings: * **Validation Rules:** Define which data points are mandatory, what constitutes a valid range, and what business rules must be enforced (e.g., stock cannot be negative, all styles must belong to a story). * **Error vs. Warning Thresholds:** Criteria for classifying data issues as critical errors (halt process) or warnings (proceed with caution).

Process Flow:

graph TD

```
A[Raw Input Data] --> B{Check for Missing Data};
B --> C{Validate Data Consistency};
C --> D{Verify Business Rule Adherence};
D --> E{Generate Warnings/Errors};
E --> F[Validated Data (or Process Halt)];
```

Step-by-Step Actions: * **Load** raw input data from all sources. *To get all the necessary information before it's used in the pipeline.* * **Check** for critical missing data. *Ensuring that essential data points like active seasons, store lists, and initial stock levels are present.* * **Validate** data consistency. *For example, ensuring that a style referenced in a sales record also exists in the product master data.* * **Verify** adherence to business rules. *Such as ensuring OD values are within acceptable*

ranges, or that all active styles are mapped to a 'story'. * **Generate** warnings for minor issues. *To alert users to potential data quality concerns without stopping the process.* * **Generate** errors for critical issues. *To halt the allocation process if data integrity is severely compromised, preventing erroneous allocations.* * **Report** validation results. *Providing a clear log of all data issues found.*

Sample Scenario: Before Data Preparation runs, Input Data Validation checks the incoming data. It finds that “Store XYZ” is listed as a distribution store but has no current stock data, generating a critical error. It also finds that “Style 789” has a negative OD value, which is flagged as a warning. Additionally, a new story “Winter Wonders” has no styles mapped to it yet, which is another warning. Because of the critical error (missing store stock), the validation module halts the entire storyWiseDisplay process, preventing allocation to that store. **Key Takeaway:** This module acts as the gatekeeper for data quality, ensuring that the entire storyWiseDisplay pipeline operates on reliable inputs and prevents faulty allocations caused by bad data.

Results: * Validation reports with warnings and errors. * Prevention of process execution if critical errors are found. * Improved data quality for downstream modules.

Unusual Cases: * **Too many warnings:** Overwhelms users, making it hard to identify critical issues. * **Overly strict validation:** False positives, halting the process for minor, non-impactful data quirks. * **Validation rules not updated:** New data types or business rules are introduced, but validation rules are not, leading to undetected issues. * **Cascading errors:** One small data issue triggers multiple validation errors across different rules.

Next Steps: * **Data Preparation:** Consumes the raw input data after it has been validated, ensuring it works with quality inputs.

Common Questions: 1. What are the most common data validation errors we encounter? 2. Can I configure which errors are critical vs. warnings? 3. How can we pre-check our data before submitting it to the system? 4. What happens if validation fails for only a small portion of the data? 5. Is there a log file to review all validation messages?

Quick Reference: * **Data Quality:** Ensuring accuracy and completeness. * **Validation Rules:** Criteria for acceptable data. * **Gatekeeper:** Prevents bad data from entering the pipeline.

8.13 Segmentation

This module identifies and categorizes styles into performance segments, such as “Top Seller” and “Bottom Seller.” This segmentation is based on sales performance, revenue per day, discount levels, and sell-through rates over specific periods. These segments are critical for guiding allocation strategies, with top sellers often prioritized for inventory and bottom sellers considered for pullbacks or reduced allocations.

Required Information: * **Sales Data:** * Cleaned and prioritized sales data (from Sales Data Preparation). * Revenue per day (from Revenue Computation). * Discount levels. * Sell-through rates. * **Product Data:** * Style information (e.g., category, price point).

Business Settings: * **Segmentation Criteria:** Thresholds for sales velocity, revenue, discount, and sell-through to classify styles into segments (e.g., “Top Seller” = $ROS > X$ units/week). * **Segmentation Period:** The historical period over which sales data is analyzed for segmentation. * **Segment Definitions:** Labels and characteristics for each performance segment (e.g., “Top Seller,” “Bottom Seller,” “Developing,” “Core”).

Process Flow:

graph TD

```
A[Cleaned Sales Data & Revenue Metrics] --> B[Analyze Sales Performance (ROS, Revenue, Sell-through, Discount)];
B --> C[Apply Segmentation Criteria];
C --> D[Segmented Styles (e.g., Top Seller, Bottom Seller)];
```

Step-by-Step Actions: * **Analyze** sales performance metrics for each style, including ROS, revenue per day, discount levels, and sell-through rates over a defined period. *To understand how each style is currently performing.* * **Apply** pre-defined segmentation criteria. *To classify styles into distinct performance categories based on their metrics.* * **Categorize** styles into segments like “Top Seller,” “Bottom Seller,” “Core,” etc. *Providing a high-level performance label for each style.* * **Update** the segment attribute for StoreStyle objects in distributionData. *Making the segmentation information available for other modules.*

Sample Scenario: A particular t-shirt style has sold 200 units in the last 4 weeks (ROS of 50 units/week), with minimal discounts and a high sell-through rate. Segmentation identifies these metrics and, based on its rules (e.g., $ROS > 40$ units/week = Top Seller), classifies this t-shirt as a “Top Seller.” Another jacket style, however, has an ROS of 5 units/week, high discounts, and low sell-through. This jacket is classified as a “Bottom Seller.” **Key Takeaway:** This module provides valuable performance context for each style, enabling subsequent allocation decisions to prioritize high-performing products and strategically manage underperformers.

Results: * Styles classified into performance segments (e.g., “Top Seller,” “Bottom Seller”). * segment attribute populated for StoreStyle objects in distributionData.

Unusual Cases: * **New styles with no history:** Difficult to segment accurately without sufficient sales data. * **Seasonal fluctuations:** Performance metrics might change drastically between seasons, requiring re-segmentation. * **Arbitrary thresholds:** Segmentation criteria not aligned with actual business performance, leading to misclassification. * **Product lifecycle bias:** Older products naturally have lower ROS and might be unfairly categorized as “Bottom Sellers.”

Next Steps: * **Revenue Computation:** Can reference the segment attribute for StoreStyle objects in distributionData (e.g., in removeZeroInvStoreStyles). * **Iteration Runner:** Uses the segmentation (e.g., storeStyle.getSegment() == Segment.TOP_SELLER) to apply specific allocation rules or priorities.

Common Questions: 1. How often is segmentation updated, and why? 2. Can different segmentation rules be applied for different product hierarchies? 3. How are “new” products handled for segmentation? 4. What are the key differences in allocation strategy for a “Top Seller” vs. a “Bottom Seller”? 5. Can we define custom segments beyond Top/Bottom Seller?

Quick Reference: * **Performance Segments:** Categories like Top/Bottom Seller. * **R0S:** Key metric for segmentation. * **Sell-through:** Indicator of product velocity.

9. Appendix

9.1 Full Parameter Reference

(This section would typically contain a comprehensive list of all configurable parameters, beyond just args.properties, with detailed descriptions, default values, and valid ranges. No specific data was provided for this section in the inputs.)

9.2 Version History

(This section would list past versions of the storyWiseDisplay system, detailing changes, bug fixes, and new features introduced in each version. No specific data was provided for this section in the inputs.)

9.3 Contact & Support Info

(This section would provide contact details for the development team, support channels, and escalation procedures for storyWiseDisplay. No specific data was provided for this section in the inputs.)

Tutorial: storyWiseDisplay

The storyWiseDisplay system is orchestrated by a central module (0) that defines a sequential pipeline for inventory allocation. Data preparation (1), warehouse adjustments (2, 3), and sales analysis (4) precede core computations like revenue (5) and ranking (6). The central allocation module (7) iteratively applies rules, leveraging an iteration runner (9) and dynamically adjusting planograms (10). Input data integrity is ensured by a validation module (11), while product segmentation (12) guides allocation strategies. Finally, an output module (8) generates comprehensive reports.

flowchart TD

```
A0["Story Wise Display Orchestration"]
A1["Data Preparation"]
A2["Warehouse Inventory Preparation"]
A3["Warehouse Reserve Management"]
A4["Sales Data Preparation"]
A5["Revenue Computation"]
A6["Ranking"]
A7["Allocation"]
A8["Output Generation"]
A9["Iteration Runner"]
A10["Planogram Incrementer"]
A11["Input Data Validation"]
A12["Segmentation"]
A0 -- "orchestrates" --> A1
A0 -- "orchestrates" --> A2
A0 -- "orchestrates" --> A3
A0 -- "orchestrates" --> A4
A0 -- "orchestrates" --> A5
A0 -- "orchestrates" --> A6
A0 -- "orchestrates" --> A7
A0 -- "orchestrates" --> A8
A1 -- "provides_data" --> A2
A1 -- "provides_data" --> A3
A1 -- "provides_data" --> A4
A1 -- "provides_data" --> A5
A1 -- "provides_data" --> A6
A2 -- "modifies_data" --> A7
A3 -- "modifies_data" --> A8
A4 -- "provides_sales" --> A5
A4 -- "provides_sales" --> A12
A5 -- "provides_ranking_inputs" --> A6
A5 -- "provides_data" --> A7
A6 -- "provides_ranked_order" --> A7
A7 -- "provides_allocation_results" --> A8
A7 -- "calls" --> A9
```

```
A7 -- "calls" --> A10
A9 -- "modifies_data" --> A7
A10 -- "modifies_data" --> A7
A11 -- "validates_inputs" --> A1
A12 -- "provides_segmentation" --> A5
A12 -- "provides_segmentation" --> A9
```

Chapters

Master Report: storyWiseDisplay

1. Title & Metadata

- **Project Name:** storyWiseDisplay
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** storyWiseDisplay optimizes product assortments and display layouts by determining the ideal number of items (OD), categories (OW), and sizes (ISS) to maximize sales and operational efficiency.

2. Executive Summary

storyWiseDisplay is an advanced merchandising algorithm designed to intelligently recommend optimal product assortments and display configurations across retail channels. By analyzing historical sales data, inventory levels, and product attributes, the system provides data-driven recommendations that enhance customer experience and drive profitability. This report details the architecture, functionality, and usage of the storyWiseDisplay system, serving as a comprehensive reference for stakeholders and operators.

- **Key outcomes or KPIs:**
 - **Increased Sales & Revenue:** Achieved by optimizing product availability and visual merchandising.
 - **Improved Inventory Efficiency:** Minimized stockouts and overstock situations through precise ISS recommendations.
 - **Enhanced Customer Satisfaction:** Ensuring desirable products are available in the right quantities and presented effectively.
 - **Optimized Assortment Planning:** Guiding merchandising decisions with data-backed OD and OW metrics.
 - **Accelerated ROS:** Identifying fast-selling items and ensuring their optimal placement.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Inventory Management, Store Operations, Supply Chain Analysts, Business Intelligence (BI) Teams.

3. Architecture & Data Flow

The storyWiseDisplay algorithm operates through a series of interconnected modules, processing raw sales and product data into actionable merchandising insights.

- **Diagram:**

```

graph TD
    A[Raw Data Ingestion] --> B{Data Validation & Preprocessing};
    B --> C[Attribute & Hierarchical Aggregation];
    C --> D[ROS Calculation & Anomaly Detection];
    D --> E[OD & OW Optimization];
    E --> F[ISS Generation];
    F --> G[Display Layout Recommendation];
    G --> H[Output Generation];
    H --> I[Merchandising Teams / BI Systems];

    subgraph storyWiseDisplay Core
        B --> G
    end
end

```

• **Component Bullets:**

- **Raw Data Ingestion:** Gathers input data from various sources.
- **Data Validation & Preprocessing:** Cleans, validates, and transforms raw data into a standardized format.
- **Attribute & Hierarchical Aggregation:** Organizes data based on AGs, categories, and other hierarchies.
- **ROS Calculation & Anomaly Detection:** Computes Rate of Sale and identifies outliers or unusual trends.
- **OD & OW Optimization:** Determines the Optimum Depth and Optimum Width for product assortments.
- **ISS Generation:** Calculates the Ideal Size Set for specific products or product groupings.
- **Display Layout Recommendation:** Formulates visual merchandising plans based on optimized OD, OW, and ISS.
- **Output Generation:** Produces structured reports and data feeds for downstream consumption.
- **Merchandising Teams / BI Systems:** The primary consumers of the algorithm's outputs.

• **High-level data sources:**

- **POS (Point-of-Sale) Data:** Transactional sales data, including units sold, date, store, and SKU.
- **Inventory Management Systems:** Real-time stock levels, warehouse availability, and in-transit data.
- **Product Master Data:** Comprehensive product attributes (e.g., category, sub-category, brand, color, material, dimensions, cost, retail price).
- **Store/Display Configuration Data:** Information about store layouts, fixture types, display capacities, and historical display performance.
- **Promotional Data:** Details on past and planned promotions or marketing campaigns.

4. Configuration & Data Dictionary

Settings Table (args.properties)

The args.properties file dictates the operational parameters and thresholds for the storyWiseDisplay algorithm.

| Setting Name | Type | Description | Default Value |
|---------------------------------|---------|---|-----------------|
| data.input_path | String | Directory path for raw input data files. | /data/raw/ |
| data.output_path | String | Directory path for generated output reports and recommendations. | /data/output/ |
| ros.calculation_period | Integer | Number of weeks/days for ROS calculation (e.g., 12 weeks). | 12 |
| ros.min_threshold | Float | Minimum ROS required for a product/SKU to be considered for stocking. | 0.1 |
| od.max_skus_per_category | Integer | Maximum number of unique SKUs allowed per category (OD). | 200 |
| ow.max_categories_per_store | Integer | Maximum number of distinct categories recommended per store (OW). | 50 |
| iss.min_sales_history_days | Integer | Minimum sales history in days required for ISS calculation. | 90 |
| optimization.goal | Enum | Primary optimization objective (SALES_MAXIMIZATION, PROFIT_MAXIMIZATION, INVENTORY_MINIMIZATION). | SALES_MAXIMIZAT |
| display.fixture_capacity_factor | Float | Factor to adjust display capacity based on fixture type (e.g., 0.9 for racks). | 1.0 |
| logging.level | String | Verbosity level for system logs (INFO, DEBUG, ERROR). | INFO |

Shared Data Fields (common-data.json)

These are key fields consistently used across all modules within the storyWiseDisplay pipeline.

| Field Name | Type | Description |
|------------------|---------|---|
| product_id | String | Unique identifier for a product. |
| sku | String | Unique identifier for a stock-keeping unit (product_id + size + color). |
| store_id | String | Unique identifier for a retail store location. |
| transaction_date | Date | Date of the sales transaction. |
| units_sold | Integer | Number of units sold in a transaction. |
| inventory_level | Integer | Current stock count for a given SKU at a location. |
| category | String | |

| Field Name | Type | Description |
|--------------------|--------|--|
| | | Broad classification of the product (e.g., “Apparel”, “Footwear”). |
| subcategory | String | More specific classification (e.g., “Men’s T-Shirts”). |
| brand | String | Product brand. |
| size | String | Product size (e.g., “S”, “M”, “L”, “XL”, “US 10”). |
| color | String | Product color (e.g., “Black”, “Navy Blue”). |
| price_retail | Float | Retail price of the product. |
| cost_of_goods | Float | Cost to produce or acquire the product. |
| attribute_group_id | String | Identifier for an AG (e.g., “BASIC_TEES_AG”). |

Outputs Catalog

The primary outputs generated by the storyWiseDisplay algorithm.

| Output File Name | Format | Description | Key Metrics/Fields Included |
|----------------------|--------|--|---|
| recommended_od.csv | CSV | List of recommended unique items (SKUs) and their respective OD for each category/store. | store_id, category, sku, recommended_od |
| recommended_ow.csv | CSV | List of recommended categories/styles and their OW for each store. | store_id, category, recommended_ow |
| ideal_size_sets.json | JSON | Detailed ISS recommendations per product or AG, including quantity breakdowns. | product_id, attribute_group_id, iss_breakdown (e.g., {"S": 5, "M": 10, "L": 8}) |
| display_layouts.json | JSON | Structured data representing proposed display configurations. | store_id, display_area, fixture_type, sku_placement, quantity_on_display |
| ros_report.csv | CSV | A comprehensive report of calculated ROS for all SKUs/products. | sku, ros (units/week), period_start, period_end |

| Output File Name | Format | Description | Key Metrics/Fields Included |
|------------------|--------|---|-----------------------------------|
| audit_log.txt | TXT | Log of algorithm execution, warnings, and errors. | timestamp, level, message, module |

5. End-to-End Flow

The storyWiseDisplay algorithm processes merchandising data through a systematic pipeline:

- 1. Data Ingestion:** The process begins by ingesting raw data from POS, inventory, and product master systems. This includes historical sales transactions, current stock levels, and detailed product attributes.
- 2. Data Preprocessing & Validation:** Raw data is then cleaned, deduplicated, and validated against predefined schemas. Missing values are imputed, and data types are standardized to ensure consistency and quality.
- 3. Attribute Grouping & Aggregation:** Products are grouped into relevant AGs based on their attributes (e.g., material, style, occasion) and aggregated into hierarchical structures (e.g., category, subcategory, brand). This step is crucial for higher-level analysis.
- 4. Rate of Sale (ROS) Calculation:** For each SKU, product, and relevant AG, the ROS is calculated over a specified historical period. Anomaly detection algorithms identify and flag unusual sales spikes or dips that could skew results.
- 5. Optimum Depth (OD) & Optimum Width (OW) Determination:** Based on ROS, available space, and strategic merchandising goals (e.g., sales maximization), the algorithm determines the OD (ideal number of unique items within a category) and OW (ideal number of categories or styles) for each store or display area. This step considers factors like cannibalization and halo effects.
- 6. Ideal Size Set (ISS) Generation:** Leveraging ROS data, historical size-specific sales patterns, and stockout frequency, the system calculates the ISS for each product or AG. This ensures the right mix of sizes is stocked to meet demand while minimizing excess inventory.
- 7. Display Layout Recommendation:** Integrating the calculated OD, OW, and ISS, along with store-specific layout constraints and fixture capacities, the algorithm generates detailed display layouts. These recommendations suggest product placement, facing quantities, and visual merchandising guidelines to optimize customer engagement and sales.
- 8. Output Generation:** Finally, the system compiles all recommendations into structured output files (CSV, JSON) and comprehensive reports, ready for consumption by merchandising teams, inventory planners, and BI dashboards. An audit log tracks the execution details for transparency and debugging.

6. Submodule Overview

1. Data Ingestion & Validation

- **Purpose:** To retrieve raw input data from specified sources and ensure its quality and format compliance.

- **Inputs & Outputs:**

- Inputs: Raw sales data, inventory data, product master data (from various databases/files).
- Outputs: Cleaned, validated, and standardized dataframes/tables.

- **Core Logic Summary:**

- Reads data files from configured input paths.
- Performs schema validation (checks for expected columns, data types).
- Identifies and flags missing values or duplicates.
- Applies predefined data cleaning rules (e.g., removing invalid entries, standardizing formats).

2. Attribute & Hierarchical Aggregation

- **Purpose:** To group and aggregate product data based on predefined attributes and category hierarchies for granular analysis.

- **Inputs & Outputs:**

- Inputs: Cleaned product master data, validated sales data.
- Outputs: Aggregated datasets by category, subcategory, brand, and custom Attribute Groups (AGs).

- **Core Logic Summary:**

- Assigns products to relevant AGs based on attribute matching rules.
- Groups sales and inventory data by category, subcategory, brand, and attribute_group_id.
- Calculates summary metrics (e.g., total sales, average price) at each hierarchical level.
- Establishes relationships between different product groupings for cross-analysis.

3. ROS Calculation & Anomaly Detection

- **Purpose:** To compute the Rate of Sale for each SKU/product and identify any abnormal sales patterns.

- **Inputs & Outputs:**

- Inputs: Aggregated sales data, `ros.calculation_period` setting.
- Outputs: SKU-level ROS, product-level ROS, AG-level ROS, and flagged anomalies.

- **Core Logic Summary:**

- Calculates `units_sold` per time unit (e.g., per week) for each SKU/product.
- Applies statistical methods (e.g., moving averages, standard deviation analysis) to identify sales outliers.
- Adjusts ROS for promotional periods or known events to derive a baseline ROS.
- Stores calculated ROS alongside anomaly flags for further consideration.

4. OD & OW Optimization

- **Purpose:** To determine the ideal number of unique items (OD) and categories (OW) to stock, balancing demand with capacity.

- **Inputs & Outputs:**

- Inputs: ROS data, store/display capacity data, `od.max_skus_per_category`, `ow.max_categories_per_store`, `optimization.goal`.
- Outputs: Recommended OD for each category/store, Recommended OW for each store.

- **Core Logic Summary:**

- Ranks products/categories by ROS and profitability for each store/display.
- Applies constraints such as maximum SKU count per category (OD) and maximum category count per store (OW).
- Utilizes an optimization algorithm (e.g., linear programming or greedy approach) to select the optimal assortment.
- Considers product interdependencies (e.g., complementary products, substitutes) to refine selection.

5. ISS Generation

- **Purpose:** To recommend the best combination of sizes for each product or AG to maximize sales and minimize stockouts.

- **Inputs & Outputs:**

- Inputs: SKU-level ROS, historical size-specific sales, inventory levels, `iss.min_sales_history_days`.
- Outputs: Ideal Size Set (ISS) recommendations per product/AG.

- **Core Logic Summary:**

- Analyzes the historical distribution of sales across different sizes for each product.
- Calculates the expected demand for each size based on its historical proportion and the product's overall ROS.
- Considers inventory levels and stockout history to suggest reorder points and target stock levels for each size.
- Generates a size breakdown (e.g., S: 5 units, M: 10 units, L: 8 units) that optimizes fulfillment rates.

6. Display Layout Generation

- **Purpose:** To translate OD, OW, and ISS recommendations into actionable visual merchandising plans.

- **Inputs & Outputs:**

- Inputs: Recommended OD, OW, ISS, store layout data, fixture capacities, `display.fixture_capacity_factor`.
- Outputs: Detailed display layouts, including SKU placement and quantities on display.

- **Core Logic Summary:**

- Maps recommended products/categories to available display areas and fixtures.
- Distributes SKUs according to their ISS and assigned display quantities.
- Applies visual merchandising rules (e.g., adjacencies, focal points, color blocking) for optimal presentation.
- Generates a structured representation of the display layout, detailing each SKU's position and visible quantity.

7. Worked Example

Let's illustrate how storyWiseDisplay would recommend for a "Men's Basic T-Shirts" category in Store A.

Scenario: Store A has limited display space for Men's Apparel. The goal is to maximize sales of basic t-shirts.

1. Data Ingestion & Preprocessing: * **Input Data:** * **Sales:** 12 weeks of sales for Men's T-shirts in Store A. * **Inventory:** Current stock levels for all Men's T-shirt SKUs. * **Product Master:** Details for each T-shirt (e.g., product_id, color, size, material, price). * **Preprocessing:** Data is cleaned; for example, an invalid size "XXS" is corrected to "XS" if no sales.

2. Attribute & Hierarchical Aggregation: * Products are categorized: Category: Apparel, Subcategory: Men's Tops, Attribute Group (AG): Basic T-Shirts. * Example AG: BASIC_COTTON_TEES (all 100% cotton basic tees).

3. ROS Calculation: * The algorithm calculates the ROS for each SKU within the BASIC_COTTON_TEES AG over the last 12 weeks. * **Example ROS:** * SKU: TEE001-BLK-M (Black, Medium): 15 units/week * SKU: TEE001-WHT-M (White, Medium): 12 units/week * SKU: TEE002-GRY-L (Grey, Large, different style): 5 units/week * SKU: TEE003-BLU-S (Blue, Small, poor seller): 0.5 units/week * **Anomaly Detection:** No significant anomalies detected in this period.

4. OD & OW Optimization: * **Constraint:** od.max_skus_per_category for "Men's Basic T-Shirts" is 15 unique SKUs (considering color/material variations, not size). * **Logic:** The algorithm ranks unique product_id + color combinations by their aggregate ROS. * **Recommendation:** * TEE001 (most popular style) in Black, White, Navy, Olive (4 colors). * TEE002 (second popular) in Grey, Charcoal (2 colors). * TEE004 (new trending style) in Maroon, Cream (2 colors). * *Total OD for Basic T-Shirts (unique product_id+color): 8 SKUs (within the 15-SKU limit).* TEE003 is excluded due to low ROS.

5. ISS Generation: * For each selected product_id + color combination (e.g., TEE001-BLK), the algorithm analyzes its historical size sales distribution. * **Example for TEE001-BLK:** * Historical Sales Breakdown: S (10%), M (40%), L (30%), XL (20%). * Total expected sales (based on OD choice): If we expect to sell 25 units of TEE001-BLK weekly. * **Recommended ISS for TEE001-BLK:** * S: 3 units (10% of 25, rounded up + buffer for demand variability) * M: 12 units (40% of 25, plus buffer) * L: 9 units (30% of 25, plus buffer) * XL: 6 units (20% of 25, plus buffer) * *Note:* The algorithm would also consider current inventory levels and minimum display quantities.

6. Display Layout Generation: * **Input:** Store A's "Men's Apparel Wall" has 3 shelves, each accommodating 100 units of folded t-shirts. * **Logic:** Place high-ROS SKUs at eye-level, group by style/color, ensure easy access. * **Recommendation for "Men's Basic T-Shirts" section:** * **Shelf 1 (Eye Level):** TEE001-BLK, TEE001-WHT (most popular). Display 5 units of M size facing, with back stock of recommended ISS. * **Shelf 2:** TEE001-NVY, TEE001-OLV, TEE002-GRY. * **Shelf 3 (Lower):** TEE002-CHR, TEE004-MRN, TEE004-CRM. * Detailed plan includes exactly which SKUs go where, how many units should be on display (facing quantity), and where the remaining ISS stock should be stored in back-of-house if applicable.

7. Output Generation: * recommended_od.csv: StoreA, Men's Basic T-Shirts, TEE001-BLK, TEE001-WHT, ..., TEE004-CRM (8 unique SKUs). * ideal_size_sets.json: { "TEE001-BLK": {"S":3, "M":12, "L":9, "XL":6}, ... } * display_layouts.json: Structured data detailing shelf assignments, facing quantities, and adjacencies. * ros_report.csv: All calculated ROS values.

This example shows how storyWiseDisplay translates raw data and configurations into concrete, actionable merchandising decisions, moving from high-level assortment strategy (OD, OW) down to granular display execution (ISS, layout).

8. Debug Guide Highlights

- **Error: DataValidationError: Missing essential columns in input.**
 - **Cause:** Input CSV/JSON files lack required fields (e.g., sku, units_sold, transaction_date).
 - **Troubleshoot:** Check common-data.json for expected fields. Verify your input data schema matches.
- **Error: ConfigurationError: Invalid value for 'ros.calculation_period'.**
 - **Cause:** A setting in args.properties has an incorrect data type or out-of-range value.
 - **Troubleshoot:** Refer to the Settings Table for valid types and typical ranges. Correct the args.properties entry.
- **Warning: ROSAnomalyWarning: Significant ROS outliers detected for SKU X.**
 - **Cause:** The ROS calculation detected unusual sales patterns (e.g., massive spike from a one-off event, or sudden drop).
 - **Troubleshoot:** Investigate sku X's sales history. Was there a massive promotion? A data entry error? Consider excluding the anomalous period or manually adjusting the ROS if the event is non-recurring.
- **Issue: Recommendations seem "off" (e.g., too many low-selling items, missing key items).**
 - **Cause:**
 1. **Data Quality:** Underlying sales/inventory data might be incomplete or inaccurate.
 2. **Configuration:** min_ros_threshold might be too low, or optimization.goal not aligned with business objectives.
 3. **Capacity Mismatch:** Display capacities configured are too high/low.
 - **Troubleshoot:**
 1. Review input data and preprocessing logs.
 2. Adjust args.properties parameters, especially ros.min_threshold or optimization.goal.
 3. Verify store/display capacity data against physical reality.
- **Performance Issue: Algorithm runs very slowly or consumes excessive memory.**
 - **Cause:** Large datasets, inefficient configurations, or resource constraints.
 - **Troubleshoot:**
 1. Check input data size. Consider processing data in smaller batches if possible.
 2. Ensure ros.calculation_period isn't excessively long.
 3. Increase allocated memory or CPU resources for the execution environment.

4. Review `logging.level`—DEBUG level can generate large logs and slow down processing.

9. Common Pitfalls & Remedies

- **Pitfall 1: Poor Data Quality (Garbage In, Garbage Out)**
 - **Description:** Inaccurate, incomplete, or inconsistent sales, inventory, or product attribute data can lead to flawed recommendations.
 - **Remedy:** Implement robust data validation checks at the ingestion stage. Establish clear data governance policies. Regularly audit data sources and perform data cleaning. Use a “data freshness” KPI.
- **Pitfall 2: Misconfigured Optimization Goals**
 - **Description:** The `optimization.goal` setting does not align with current business priorities (e.g., optimizing for sales when profit maximization is desired).
 - **Remedy:** Clearly define business objectives before running the algorithm. Regularly review and adjust the `optimization.goal` and other related parameters in `args.properties` to ensure they reflect strategic targets.
- **Pitfall 3: Ignoring Promotional Impacts on ROS**
 - **Description:** ROS calculations can be skewed by major promotional events, leading to over-forecasting demand for non-promoted periods.
 - **Remedy:** Incorporate promotion flags in input data. The ROS module should ideally have logic to normalize sales data by discounting promotional spikes or using a baseline ROS calculation.
- **Pitfall 4: Static Display Capacity Data**
 - **Description:** Store or fixture capacities are not updated to reflect physical changes or temporary layouts.
 - **Remedy:** Establish a regular process for updating store layout and fixture capacity data. Integrate with space planning systems if available. Allow for manual overrides or “what-if” scenarios for temporary adjustments.
- **Pitfall 5: Over-reliance on Historical Data for New Products**
 - **Description:** New product introductions lack sufficient historical ROS data, leading to inaccurate OD/ISS recommendations.
 - **Remedy:** For new products, use comparable product sales data (e.g., from similar AGs or categories) as a proxy. Implement a ramp-up period for data collection before full algorithmic recommendations are trusted.
- **Pitfall 6: Lack of Business User Buy-in**
 - **Description:** Merchandising teams distrust the algorithm’s recommendations if they don’t understand the underlying logic or see inconsistent results.
 - **Remedy:** Foster transparency by providing clear documentation (like this report), regular training, and a feedback loop. Start with pilot programs, demonstrate tangible benefits, and involve users in the refinement process.

10. FAQ

- **Q1: How often should I run `storyWiseDisplay`?**
 - **A:** It depends on your business needs and data freshness. Weekly or bi-weekly runs are common for dynamic categories, while monthly might suffice for stable ones. The `ros.calculation_period` should align with your refresh cycle.

- **Q2: Can storyWiseDisplay account for regional or seasonal differences?**
 - **A:** Yes, the algorithm can be configured to analyze data at granular levels (e.g., by `store_id`, `region_id`). Its ROS calculation can also incorporate seasonal indices if provided in the input data.
- **Q3: What if I have multiple optimization goals (e.g., both sales and profit)?**
 - **A:** Currently, `optimization.goal` supports a single primary objective. For multi-objective optimization, you may need to run the algorithm with different goals and combine the results, or consult with the development team for custom enhancements to the core optimization module.
- **Q4: How does storyWiseDisplay handle out-of-stock items in its ROS calculation?**
 - **A:** The algorithm aims to adjust ROS for stockout periods by inferring lost sales or normalizing the sales period. Ensure your inventory data accurately reflects stockout days. If stockout information is missing, ROS may be underestimated.
- **Q5: Can I manually override recommendations?**
 - **A:** While the algorithm provides data-driven recommendations, business judgment is crucial. The outputs are designed to be consumed by human merchandisers who can apply their expertise and manually adjust recommendations before execution. Future versions may include a built-in override mechanism.
- **Q6: What is an Attribute Group (AG) and why is it important?**
 - **A:** An AG is a custom grouping of products based on shared attributes (e.g., “Premium Denim,” “Summer Dresses,” “Eco-Friendly Footwear”). It allows the algorithm to analyze and optimize assortments for specific market segments or product strategies that cut across traditional categories.

11. Quick Reference Cheat-Sheet

| Term / Abbreviation | Definition | Key Metric / Usage |
|------------------------------|--|--|
| OD (Optimum Depth) | The ideal number of unique items (SKUs) to stock in a category or display. | <code>recommended_od.csv</code> output; ensures variety without overstocking. |
| OW (Optimum Width) | The ideal number of different categories or styles to offer. | <code>recommended_ow.csv</code> output; optimizes overall store/display breadth. |
| ISS (Ideal Size Set) | The best combination of sizes to maximize sales and minimize stockouts. | <code>ideal_size_sets.json</code> output; critical for inventory efficiency and customer satisfaction. |
| ROS (Rate of Sale) | The speed at which products are sold over a period (e.g., units/week). | Core input for OD, OW, ISS; calculated in <code>ros_report.csv</code> . |
| AG (Attribute Group) | A set of product attributes grouped for analysis (e.g., color, size, brand). | Facilitates targeted optimization and analysis; used in <code>common-data.json</code> . |
| <code>args.properties</code> | | |

| Term /
Abbreviation | Definition | Key Metric / Usage |
|------------------------|---|---|
| | Configuration file for the algorithm's operational parameters. | Adjust settings like <code>ros.calculation_period</code> , <code>optimization.goal</code> . |
| common-
data.json | Defines the standard fields and schema for data flowing through the pipeline. | Essential for data consistency and validation. |
| Mermaid Diagram | A text-based diagramming tool used to visualize the system's architecture. | Provides a high-level overview of data flow and module interaction. |
| Data Validation | Process of ensuring input data is clean, accurate, and consistent. | First step in pipeline; prevents "garbage in, garbage out." |
| Output Catalog | List of all generated reports and data files from the algorithm. | Reference for consuming applications and teams. |

Chapter 3: WSSI Cleaned Sales Computation Module

Last Updated: 2025-05-23

1. Business Objective

This step processes raw sales data to create reliable, “cleaned” sales figures for both past performance (Last Year) and future projections (forecast periods).

Why This Matters: This foundational data ensures that all subsequent performance analysis, inventory planning, and strategic decisions are based on accurate and aggregated sales insights, helping to optimize inventory levels and improve overall sales efficiency.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Raw Sales Transactions:** This includes the quantity of each item sold, the revenue generated, and any discount applied for every single sale. We need it to perform the core calculations of sales performance.
- **Sale Dates:** The exact date of each transaction is crucial to group sales into specific planning subperiods, ensuring we compare apples to apples when analyzing performance over time.

Product Data

- **Product Details:** Information like a product’s Fashion Grade (FG), Brand Segment, Brand, and its original Manufacturer’s Suggested Retail Price (MRP). This is essential to categorize sales meaningfully and calculate accurate Average Selling Price (ASP).
- **Product Hierarchies:** Knowing how individual unique items (SKUs) belong to broader styles and how styles fit into parent styles helps us correctly group and aggregate sales across different product classifications.

Operational Data

- **Store Details:** Knowing the specific channel (e.g., online, brick-and-mortar) for each store. This allows us to attribute sales to the correct sales channel for comprehensive reporting.
- **Merchandise Planning Periods:** The defined start and end dates for each subperiod and period for both Last Year and the forecast. These definitions guide how we aggregate sales over relevant timeframes.
- **Store Sales Channel Mappings:** This mapping helps ensure that sales from a particular store are correctly associated with its designated sales channel.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|----------------------------------|--------------------|---|--|
| Discount Threshold for Cleaning | N/A (Internal) | This percentage value would define the maximum discount allowed for a sale to be considered 'clean' for analysis. Sales exceeding this discount might be excluded if the goal is to analyze full-price performance. | If deep discount sales are skewing performance metrics or if a specific sales type needs to be isolated for analysis. |
| Pivotal Sales Activity Threshold | N/A (Internal) | A percentage benchmark for the proportion of "Pivotal" products (key items for a category) that must be "live" (available) in a store on a given day. If activity falls below this, sales might be deemed unrepresentative. | If product availability issues are distorting sales data, or if specific focus needs to be placed on sales when a healthy assortment is present. |
| Analysis Period Definition | Dynamic (from MFP) | These are the specific start and end dates for each subperiod and period for both Last Year | When the financial planning calendar changes or specific seasons/campaigns have new start/end dates. |

| Setting Name | Default Value | Description | When to Change |
|--------------|---------------|--|----------------|
| | | and the forecast, imported from merchandise financial plans. | |

Note: The settings primarily relate to how sales are filtered or aggregated based on specific criteria. The actual values are typically managed through data inputs or internal configurations rather than direct user adjustments in this computation module.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Raw Sales Data, Product Info, Planning Periods] -->

B[Cleaned Sales Computation]

B --> C[Cleaned LY & Forecast Sales Data]

1. Start with → Raw sales transactions, comprehensive product details, and the defined merchandise planning periods.
2. Then → The module groups individual sales by store, product categories (Fashion Grade, Brand Segment, Brand), sales channel, and specific subperiods.
3. Next → It calculates critical sales metrics for each group, including total revenue, total quantity sold, total discount value, average selling price (ASP), discount percentage, and the number of “live days” (days with sales activity).
4. Finally → The module produces a clean, aggregated dataset of sales performance for both the Last Year and the forecast periods.

5. Step-by-Step Actions

1. **Gather** Sales, Product, and Operational Data
 - What we do: *We collect all the necessary raw sales transactions, detailed product information (like Fashion Grade and Brand), store details, and defined merchandise planning periods.*
 - Why we do it: *This provides all the raw ingredients needed for accurate sales aggregation and analysis.*
2. **Identify** Relevant Planning Periods
 - What we do: *We use the specified start and end dates for Last Year and forecast subperiods to define the exact timeframes for sales analysis.*
 - Why we do it: *This ensures that sales are correctly attributed to the appropriate historical or future planning windows.*
3. **Aggregate** Sales for Last Year
 - What we do: *For each store, product category (Fashion Grade, Brand Segment, Brand), and sales channel, we sum up sales data that falls within the Last Year's subperiod dates.*
 - Why we do it: *This provides a clear, summarized view of past performance, which is essential for benchmarking and understanding historical trends.*

4. **Aggregate** Sales for the Forecast Period

- What we do: *Similarly, we sum up sales data for the current and upcoming forecast subperiods, applying the same grouping logic.*
- Why we do it: *This forms the basis for projecting future sales performance and aligning inventory with anticipated demand.*

5. **Calculate** Key Sales Metrics

- What we do: *For each aggregated group, we compute revenue, quantity sold, discount value, discount percentage, Average Selling Price (ASP), and count the number of unique days with sales activity (live days).*
- Why we do it: *These metrics provide a comprehensive understanding of sales effectiveness, profitability, and operational activity for each product group and location.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

Imagine a specific store, "Urban Outfitters," selling a product category like "Women's Core T-Shirts" (Fashion Grade: Core, Brand Segment: Casual Wear, Brand: 'Everyday Basics'). We want to clean sales for the "Summer 2024" subperiod for Last Year (meaning Summer 2023).

- **Store:** Urban Outfitters (Sales Channel: Retail Store)
- **Product Grouping:** Fashion Grade: Core, Brand Segment: Casual Wear, Brand: Everyday Basics
- **Subperiod:** Summer 2024 (Last Year Analysis Dates: 2023-06-01 to 2023-08-31)
- **Raw Sales Data (from Summer 2023):**
 - Sale A: 2 units, Revenue: \$60, Discount: \$0, Date: 2023-06-10 (SKU MRP: \$30/unit)
 - Sale B: 3 units, Revenue: \$75, Discount: \$15, Date: 2023-06-10 (SKU MRP: \$30/unit)
 - Sale C: 1 unit, Revenue: \$25, Discount: \$5, Date: 2023-06-25 (SKU MRP: \$30/unit)

What Happens

1. First, the module takes all raw sales from Urban Outfitters for "Everyday Basics" Core T-Shirts. It checks if each sale's date falls within the "Summer 2023" Last Year analysis period (June 1st to August 31st, 2023). All three sales (A, B, C) are confirmed to be within this period.
2. Then, it sums up the key figures:
 - Total Quantity: $2 + 3 + 1 = 6$ units
 - Total Revenue: $\$60 + \$75 + \$25 = \160
 - Total Discount Value: $\$0 + \$15 + \$5 = \20
 - Total MRP Value (if all units sold at MRP): $(230) + (330) + (1*30) = \$60 + \$90 + \$30 = \$180$
3. Next, it identifies the unique days on which sales occurred: June 10th and June 25th. This means there were 2 "live days" for this product group in this store during the subperiod.

4. Finally, it calculates the derived metrics:

- Discount Percentage: $(\$20 / (\$20 + \$160)) * 100 = (\$20 / \$180) * 100 = 11.11\%$
- Average Selling Price (ASP): $\$160 / 6 \text{ units} = \26.67 per unit

Final Outcome

The module produces a single “cleaned sales” record for “Urban Outfitters,” “Women’s Core T-Shirts,” “Everyday Basics,” for “Summer 2024 (LY)” showing: Revenue \$160, Quantity 6, Discount Value \$20, Discount % 11.11%, ASP \$26.67, and Live Days 2. This exact same process is applied for all combinations of stores, product groups, and subperiods, for both Last Year and Forecast.

Key Takeaway: This example shows how individual sales transactions are meticulously combined and summarized into meaningful business metrics, providing a comprehensive, aggregated view of performance for a specific store, product grouping, and time frame.

7. Results

This step produces:

Main Outputs

- **Cleaned Last Year (LY) Sales Data:** This output provides an aggregated view of historical sales performance by store, Fashion Grade, Brand Segment, Brand, sales channel, and subperiod. It includes total revenue, quantity, discount value, discount percentage, Average Selling Price (ASP), and live days. It is the crucial baseline for understanding past trends.
- **Cleaned Forecast Period Sales Data:** Similar to the LY data, but for the current and upcoming forecast periods. This data serves as the foundation for projecting future sales, informing inventory purchases, and assessing anticipated demand.

Business Reports

- **Historical Performance Reports:** Used by retail merchants, category managers, and planners to analyze past sales trends, identify top-performing product groups, and understand the impact of previous merchandising strategies.
- **Forecast Accuracy Reports:** Used by analysts to compare projected sales against actual cleaned sales data, helping to refine forecasting models and improve future prediction accuracy.
- **Category Health Dashboards:** Utilized by category managers and business leaders to monitor the overall financial health and sales performance of specific product categories, helping them to make informed decisions about assortment, pricing, and promotions.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed | The module will process with available data, potentially leading to incomplete or outdated “cleaned sales” figures. | Contact the data engineering or IT team immediately to resolve the data feed issue and re-run the module once data is complete. |
| No sales recorded for a specific category/store/period | The output for that specific combination will show zero values for all metrics (revenue, quantity, etc.). | Investigate if this is expected (e.g., a new store not yet active) or if it indicates a data capture problem or a genuine underperforming category. |
| Product master data contains incorrect classifications (e.g., wrong Fashion Grade or Brand Segment) | Sales will be aggregated under the wrong product categories, skewing performance insights for affected groups. | Verify the accuracy of product master data. If errors are found, correct them and re-run the module to ensure proper aggregation. |
| Sales data includes very high discounts or unusual return patterns | While the module aggregates the sales, extreme values might impact ASP or discount percentage if not specifically filtered in advance. | Review raw sales data for anomalies. Consider implementing pre-processing steps or adjusting data cleansing rules in future iterations if these extremes are not desired in “cleaned sales.” |

9. Next Steps

Before This Step

- **Raw Sales Data Collection:** This crucial preceding activity collects all individual sales transactions from point-of-sale systems, e-commerce platforms, and other sales channels. We need it as the unprocessed raw material for cleaning and aggregation.
- **Product Master Data Setup:** This ensures that all product attributes, such as Fashion Grade, Brand, MRP, and size mappings, are accurately defined and loaded into the system. We need this to correctly categorize and calculate metrics for each sale.
- **Merchandise Planning Period Definition:** This defines the precise calendar and timeframes (subperiods, periods) that the business uses for planning and analysis. We need this to correctly group sales into relevant historical and forecast windows.

After This Step

- **ROS (Rate of Sale) Calculation Module:** This module takes our “cleaned sales” data and uses it to compute how quickly products are selling. They need our accurate sales quantities and associated timeframes to determine reliable selling speeds.
- **Inventory Planning Module:** This module relies on the “cleaned sales” and ROS figures to forecast future inventory needs, generate purchase orders, and manage stock levels. They need our precise sales history and projections to avoid overstocking or stockouts.
- **Performance Analysis Reports:** Various business intelligence dashboards and reports consume this cleaned sales data to provide insights into category performance, store productivity, and overall business health. They need our aggregated data as the source of truth for their analytics.

10. Common Questions

Q: Why do we “clean” sales data? A: We clean sales data to ensure that our analysis is based on accurate, consistent, and aggregated information. This removes the clutter of individual transactions and sums them into meaningful business totals, making it easier to identify trends and make informed decisions.

Q: How does this module help improve inventory decisions? A: By providing reliable historical and forecast sales figures, this module gives planners a clear picture of demand. Knowing how much was sold in the past and what’s expected for the future helps them order the right amount of product, reducing wasted inventory or lost sales due to stockouts.

Q: What is the Average Selling Price (ASP) used for? A: ASP is critical for understanding the actual price at which products are sold, considering any discounts. It helps us track profitability, assess the impact of promotions, and accurately project future revenue.

Q: Can I see sales broken down by sales channel (e.g., online vs. in-store)? A: Yes, absolutely! The module aggregates sales not only by store and product grouping but also by sales channel, allowing you to easily compare and analyze performance differences between your online and physical retail channels.

Q: What if a product group or store had zero sales in a particular period? A: If there were no sales for a specific combination of store, product group, and subperiod, the module will accurately record zero for all sales metrics. This tells you that there was no activity, which can be an important insight for identifying dormant categories or stores.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Summarizes raw sales transaction data into clean, aggregated sales figures for both historical (Last Year) and forecast periods, organized by store, product grouping (FG, Brand Segment, Brand), and sales channel.

- **Main Input:** Raw sales transactions, comprehensive product master data, and defined merchandise planning periods.
 - **Main Output:** Cleaned sales data for Last Year and Forecast, showing aggregated revenue, quantity, discount, ASP, and live days.
 - **Warning Signs:** Unexpectedly low or high sales totals for known periods, or if sales figures don't seem to align with expectations for specific product categories or stores.
 - **Key Settings:** Primarily internal criteria for defining analysis periods and potentially filtering sales based on discount levels or product availability.
-

Need Help? Contact your merchandising operations or data analytics team for questions about this process.

Generated by AI Codebase Knowledge Builder

Chapter 4: WSSI Subperiod AOP Computation Module

Last Updated: 2025-05-23

1. Business Objective

This module precisely calculates the Annual Operating Plan (AOP) at a detailed subperiod level by distributing the broader period-level AOP based on the historical revenue contribution of each subperiod.

Why This Matters: This foundational calculation ensures that financial targets and planning for future sales (Rate of Sale) and inventory are accurately proportioned over time, reflecting seasonal demand patterns and maximizing the effectiveness of buying decisions and stock management.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Last Year Sales Data:** This includes the total revenue for each product category (Fashion Grade, Brand Segment, Brand) in each store and sales channel for every historical subperiod and overall period. We need this to understand how sales previously contributed to the overall period's performance.

Product Data

- **Product Attributes (Fashion Grade, Brand Segment, Brand):** These define the specific product groupings for which the Annual Operating Plan (AOP) is set and needs to be distributed. We need these to ensure AOP is accurately broken down for the right categories.

Operational Data

- **Annual Operating Plan (AOP) by Broader Period:** The overall financial targets for each product category within a store or sales channel, defined for a larger period (e.g., a quarter or season). We need this as the total budget to be proportioned.

- **Merchandise Planning Periods:** The defined calendar structure outlining the start and end dates of all broader periods and their more granular subperiods. We need this to know exactly how to divide the larger AOP into smaller, time-based targets.
- **Store Details and Sales Channel Mappings:** Information linking each store to its specific sales channel (e.g., online, brick-and-mortar). We need this to correctly attribute AOP and historical performance to the relevant sales channel.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|------------------------------|-------------------------|---|--|
| AOP Distribution Method | Historical Contribution | This rule dictates how the overall period AOP is split across subperiods: based on the historical revenue percentage of each subperiod. | If there's a strategic shift in seasonal sales patterns not reflected in past data, or a new approach to financial planning. |
| Zero Historical Revenue Rule | Equal Distribution | If a specific product grouping had no historical sales within a given period, this rule determines that its AOP for that period will be distributed equally among its subperiods. | If a different default distribution (e.g., 0 for all subperiods, or specific pre-set percentages) is preferred for new or historically non-selling categories. |

Note: The settings primarily relate to how the Annual Operating Plan (AOP) is distributed based on specific criteria. The actual values are typically managed through data inputs or internal configurations rather than direct user adjustments in this computation module.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[Period AOP Data, Cleaned Historical Sales, Planning Periods]

--> B[AOP Subperiod Distribution]
B --> C[Granular Subperiod AOP]

1. Start with → The Annual Operating Plan (AOP) defined for broader periods, detailed historical sales performance (Cleaned Last Year Sales), and the precise definitions of all merchandise planning periods and subperiods.
2. Then → The module calculates the historical revenue contribution of each subperiod within its larger period for every store and product category.
3. Next → It applies these historical contribution percentages to the broader period AOP to accurately distribute the AOP down to the subperiod level.
4. Finally → The module produces the calculated Annual Operating Plan (AOP) values for each granular subperiod, store, and product category combination.

5. Step-by-Step Actions

1. **Gather** Annual Operating Plan, Cleaned Sales, and Period Data
 - What we do: *We collect the overall AOP targets for various product categories and stores, the detailed historical sales data for the same categories, and the defined calendar periods.*
 - Why we do it: *This provides all the necessary inputs to understand the overall financial targets and the historical seasonal sales patterns.*
2. **Calculate** Historical Revenue Contributions
 - What we do: *For each product category (Fashion Grade, Brand Segment, Brand) within every store and sales channel, we sum up the Last Year's revenue for each individual subperiod, and then for its corresponding larger period.*
 - Why we do it: *This establishes the baseline of how much each subperiod historically contributed to the overall period's sales.*
3. **Determine** Subperiod Contribution Ratios
 - What we do: *We calculate the percentage of total period revenue that came from each subperiod in the Last Year. If a product category had no historical sales in a period, we apply a rule to divide the AOP equally among its subperiods.*
 - Why we do it: *This creates the ratio used to proportionally distribute the overall AOP to each subperiod, ensuring seasonal accuracy.*
4. **Distribute** Period AOP to Subperiods
 - What we do: *We multiply the overall Annual Operating Plan (AOP) for a given product category and period by the calculated subperiod contribution ratio for each of its subperiods.*
 - Why we do it: *This step translates the high-level financial target into specific, time-phased subperiod targets, aligning the plan with expected demand fluctuations.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

Imagine a product category like “Women’s Denim Jeans” (Fashion Grade: Core, Brand Segment: Casual Wear, Brand: ‘Blue Threads’) in “Store A” has an overall Annual Operating Plan (AOP) of \$120,000 for the “Spring” period. The “Spring” period consists of three subperiods: “Early Spring,” “Mid-Spring,” and “Late Spring.”

- **Product Grouping:** Fashion Grade: Core, Brand Segment: Casual Wear, Brand: Blue Threads
- **Store:** Store A (Sales Channel: Retail Store)
- **Overall AOP for Spring Period:** \$120,000
- **Last Year (LY) Historical Sales for Spring Period:**
 - Early Spring (LY): \$25,000
 - Mid-Spring (LY): \$45,000
 - Late Spring (LY): \$30,000
 - Total Spring (LY) Sales: $\$25,000 + \$45,000 + \$30,000 = \$100,000$

What Happens

1. First, the module identifies the historical sales for “Women’s Denim Jeans” in “Store A” for each subperiod within the “Spring” period.
2. Then, it calculates the historical revenue contribution percentage for each subperiod:
 - Early Spring: $(\$25,000 / \$100,000) = 25\%$
 - Mid-Spring: $(\$45,000 / \$100,000) = 45\%$
 - Late Spring: $(\$30,000 / \$100,000) = 30\%$
3. Next, the module applies these percentages to the current Spring period’s Annual Operating Plan (AOP) of \$120,000:
 - Early Spring AOP: $25\% \text{ of } \$120,000 = \$30,000$
 - Mid-Spring AOP: $45\% \text{ of } \$120,000 = \$54,000$
 - Late Spring AOP: $30\% \text{ of } \$120,000 = \$36,000$

Final Outcome

The module produces the specific Annual Operating Plan (AOP) targets for “Women’s Denim Jeans” in “Store A” for each subperiod: \$30,000 for Early Spring, \$54,000 for Mid-Spring, and \$36,000 for Late Spring.

Key Takeaway: This example illustrates how a single overall financial target is intelligently broken down into actionable, time-specific goals that reflect the natural ebb and flow of seasonal sales, ensuring resources are aligned with demand.

7. Results

This step produces:

Main Outputs

- **Granular Subperiod Annual Operating Plan (AOP) Data:** This output provides the calculated AOP for each specific product category (Fashion Grade,

Brand Segment, Brand) within every store and sales channel, broken down by individual subperiods. It forms the precise financial targets for sales and inventory planning at the most detailed time level.

Business Reports

- **Seasonal Budget Allocation Reports:** Used by retail merchants and financial planners to understand how the overall Annual Operating Plan (AOP) is distributed across different seasons and smaller timeframes, ensuring resources are strategically allocated throughout the year.
- **Detailed Sales Forecasting Inputs:** Provides critical data for sales forecasting models, allowing them to project future sales with greater accuracy by incorporating the planned seasonal demand.
- **Inventory Buying Plan Basis:** Serves as a direct input for inventory planners, guiding their purchase decisions by providing accurate AOP targets for specific product categories during specific times of the year, helping to optimize Optimum Depth (OD) and Optimum Width (OW) of assortment.

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|---|---|
| Missing or delayed data feed | The module will process with available data, potentially leading to incomplete or outdated subperiod AOP figures. | Contact the data engineering or IT team immediately to resolve the data feed issue and re-run the module once data is complete. |
| Annual Operating Plan (AOP) is missing for a specific period/ category combination | The module will typically default to an Annual Operating Plan (AOP) of zero for that combination, resulting in zero subperiod AOP values. | Verify that all planned categories have their AOP defined. If a plan exists, input the AOP data and re-run the module. |
| Zero historical sales for a category/store/ period where AOP is defined | The module will distribute the AOP equally across all subperiods for that combination, as there's no historical pattern to follow. | Review if this is a new category or a strategic push. If a specific seasonal distribution is desired, provide specific override percentages if available, or monitor initial sales closely. |
| Inaccurate Last Year (LY) cleaned sales data | The subperiod AOP distribution will be based on flawed historical patterns, potentially misallocating targets. | Investigate the source of the inaccurate sales data (e.g., Chapter 3). Correct the underlying data and re-run this module. |

9. Next Steps

Before This Step

- **Cleaned Sales Computation Module:** This preceding module processes raw sales data into reliable, aggregated “cleaned sales” figures for past performance. We need its output to understand the historical revenue contribution of each subperiod.
- **Merchandise Planning Period Definition:** This activity defines the precise calendar and timeframes (subperiods, periods) that the business uses for planning and analysis. We need this to correctly group sales and AOP into relevant windows.
- **Annual Operating Plan (AOP) Data Input:** This activity establishes the overall financial targets at a broader period level for various product categories. We need this high-level AOP as the total budget to be distributed.

After This Step

- **Rate of Sale (ROS) Calculation Module:** This module will take our granular subperiod AOP and other data to calculate how quickly products are expected to sell. They need our accurately proportioned AOP targets to determine reliable selling speeds.
- **Inventory Planning Module:** This module relies on the detailed subperiod AOP (along with ROS) to forecast future inventory needs, generate purchase orders, and manage stock levels. They need our precise financial targets to avoid overstocking or stockouts.
- **Financial Forecasting Module:** This module consumes the granular subperiod AOP data to produce detailed financial projections and budgets, allowing for more precise revenue and profit planning throughout the year.

10. Common Questions

Q: Why is the Annual Operating Plan (AOP) broken down by subperiod? A: Breaking down AOP by subperiod allows us to align financial targets with natural seasonal shifts in demand. This ensures that inventory purchases and marketing efforts are timed perfectly, maximizing sales opportunities and minimizing wasted stock.

Q: How does this module help improve inventory buying decisions? A: By providing specific AOP targets for shorter subperiods, this module gives inventory planners a highly accurate picture of anticipated sales. This precision enables them to order the right quantity of items, ensuring adequate stock for peak demand without holding excess inventory during slower periods, improving our Optimum Depth (OD) and Optimum Width (OW).

Q: What happens if a product category had no sales in the Last Year for a specific period, but still has an AOP defined? A: If there are no historical sales to guide the distribution, the module will apply a default rule, typically distributing the Annual Operating Plan (AOP) equally across all subperiods within that period. This ensures that even new or historically non-selling categories receive a planned target for each subperiod.

Q: Can I see the subperiod Annual Operating Plan (AOP) broken down by sales channel (e.g., online vs. in-store)? A: Yes, absolutely! The module leverages sales channel mappings to ensure that the subperiod AOP is attributed to the correct channel, allowing you to analyze and plan financial targets specific to your online store, physical retail locations, or any other defined sales channel.

Q: How does this module improve the accuracy of our sales forecasts? A: By accurately distributing the overall Annual Operating Plan (AOP) based on historical seasonal patterns, this module provides a much more granular and realistic baseline for sales forecasting. This leads to more precise demand predictions and better alignment between our plans and actual customer behavior.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Distributes the broader period Annual Operating Plan (AOP) into precise subperiod AOPs using historical sales contributions.
- **Main Input:** Period-level AOP, Cleaned Last Year Sales, and defined Merchandise Planning Periods.
- **Main Output:** Granular Annual Operating Plan (AOP) data for each subperiod, store, and product category.
- **Warning Signs:** Unrealistic or unexpectedly flat AOP distribution across subperiods, or if the distributed AOP doesn't reflect known seasonal trends.
- **Key Settings:** Rules governing how AOP is distributed based on historical patterns, and how to handle scenarios with no historical sales.

Need Help? Contact your merchandising operations or financial planning team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

Chapter 5: WSSI Actual and Projected Sales & AOP Computation Module

Last Updated: 2025-05-23

1. Business Objective

This module computes comprehensive actual and projected sales and inventory metrics, alongside the target Annual Operating Plan (AOP) figures, for each store, fashion grade, and subperiod combination.

Why This Matters: This foundational data ensures that all subsequent performance analysis, inventory planning, and strategic decisions are based on accurate and aggregated sales insights, helping to optimize inventory levels, improve overall sales efficiency, and align operations with financial goals.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (Last Year and Forecast):** This includes the aggregated units sold, revenue, and discount for past periods and recent forecasts. We need this to understand historical demand and to capture recent sales performance accurately.
- **Historical Inventory Snapshots (Keyframes):** These are records of actual inventory levels for unique items (SKUs) at specific points in time. We need this to understand past inventory positions and calculate historical cover duration.

Product Data

- **Product Attributes:** Information like a product's Fashion Grade (FG), Brand Segment, Brand, and its original Manufacturer's Suggested Retail Price (MRP). This is essential to categorize sales meaningfully, group products, and calculate accurate Average Selling Price (ASP).
- **Product Hierarchies:** Knowing how individual unique items (SKUs) belong to broader styles and how styles fit into parent styles helps us correctly group and aggregate sales and inventory across different product classifications.

Operational Data

- **Merchandise Planning Periods:** The defined start and end dates for each subperiod and period for both Last Year and the forecast. These definitions guide how we attribute sales and inventory to relevant timeframes.
- **Store Details and Sales Channel Mappings:** Information linking each store to its specific sales channel (e.g., online, brick-and-mortar). We need this to correctly attribute sales and inventory to the relevant sales channel for comprehensive reporting.
- **Performance Parameters (from AOP):** These are predefined target values such as target discount percentage, target gross margin percentage, and target cover duration. We need these to calculate projected sales, inventory, and Cost of Goods Sold (COGS) for future periods based on the Annual Operating Plan.
- **Granular Subperiod AOP Data:** The detailed Annual Operating Plan (AOP) amounts, already broken down by store, fashion grade, and specific subperiod. We need this as the target revenue baseline for future projections.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------|-------------------|---|--|
| Target Discount Rate | Varies (from AOP) | The planned average discount percentage for a product category in a given period. This impacts the calculation of Average Selling Price (ASP) for future projections. | When strategic pricing changes or promotional calendars are adjusted. |
| Target Gross Margin Percentage | Varies (from AOP) | The planned gross margin percentage for a product category in a given period. This is used to derive the Cost of Goods Sold (COGS) for AOP calculations. | When sourcing costs change significantly, or when profitability targets are revised. |
| Target Cover Duration | Varies (from AOP) | The desired number of weeks or days current inventory should cover | When inventory holding costs change, supply chain lead times vary, or a shift in |

| Setting Name | Default Value | Description | When to Change |
|--------------|-------------------|---|--|
| | | future sales for a product category. This parameter is crucial for calculating target inventory levels for future periods. | service level targets occurs. |
| Target MRP | Varies (from AOP) | The Manufacturer's Suggested Retail Price (MRP) used as a baseline for a product category when calculating projected sales and ASP. | When product pricing strategies are revised or new product lines are introduced with different price points. |

Note: These key parameters are typically managed through data inputs derived from your Annual Operating Plan or strategic forecasts rather than direct user adjustments within this specific computation module.

4. Process Flow

Here's how information moves through this step:

flowchart LR

```

A[Cleaned Sales Data (LY & Forecast), Subperiod AOP,
Performance Parameters, Inventory Keyframes, Planning Periods] -->
B[Actual & Projected Sales/Inventory Calculation]
B --> C[WSSI Actual/Projected & AOP Figures]

```

1. Start with → Cleaned historical and forecast sales data, detailed inventory snapshots, granular subperiod Annual Operating Plan (AOP) data, predefined performance parameters (like target discount and margin), and the overall merchandise planning calendar.
2. Then → The module identifies whether a given period is in the past (requiring actual data) or in the future (requiring projected/AOP calculations).
3. Next → For past periods, it retrieves actual sales and inventory figures. For future periods, it applies the performance parameters to derive projected sales (Rate of Sale), inventory, and Cost of Goods Sold (COGS) based on the target AOP.
4. Finally → The module produces a comprehensive set of actual and projected sales, inventory, and AOP metrics for every store, fashion grade, and subperiod combination, ready for detailed analysis and planning.

5. Step-by-Step Actions

1. **Gather All Necessary Data**
 - What we do: *We collect all relevant cleaned sales data, inventory snapshots, merchandise planning periods, store details, and the performance parameters defined from the Annual Operating Plan.*
 - Why we do it: *This provides all the raw ingredients needed to compute both historical and future-looking sales and inventory metrics.*
2. **Categorize Each Planning Subperiod**
 - What we do: *For each subperiod, we determine if it falls in the past (before the current run date) or in the future (at or after the current run date).*
 - Why we do it: *This distinction is crucial because past periods use actual observed data, while future periods rely on projections and planned targets.*
3. **Compute Actual Data for Past Subperiods**
 - What we do: *For subperiods in the past, we retrieve the exact sales figures (revenue, quantity, discount) from the cleaned forecast sales data and calculate actual inventory levels from the historical inventory snapshots.*
 - Why we do it: *This ensures our historical view is based on real-world performance, providing a solid foundation for comparison and trend analysis.*
4. **Derive Projected Data for Future Subperiods**
 - What we do: *For future subperiods, we use the target Annual Operating Plan (AOP) revenue for that specific subperiod, coupled with the predefined performance parameters (Target MRP, Target Discount, Target Margin, Target Cover Duration) to calculate projected sales (quantity), Average Selling Price (ASP), discount value, inventory, and Cost of Goods Sold (COGS).*
 - Why we do it: *This translates our financial targets into tangible sales and inventory projections, guiding future merchandising and buying decisions.*
5. **Calculate Last Year Inventory**
 - What we do: *For each relevant store, fashion grade, and subperiod combination, we determine the inventory levels from the Last Year's historical snapshots.*
 - Why we do it: *This provides a vital baseline for comparing current and projected inventory efficiency against past performance.*

6. Sample Scenario

Let's walk through a real example:

Starting Point

Imagine we are planning for “Women’s Casual Dresses” (Fashion Grade: Core, Brand Segment: Contemporary, Brand: ‘Urban Chic’) at “Store Downtown” for “Fall 2024 - Subperiod 1.” Today’s date is July 15, 2024. This subperiod begins August 1, 2024, so it’s a future projected period.

- **Product Grouping:** Fashion Grade: Core, Brand Segment: Contemporary, Brand: Urban Chic
- **Store:** Store Downtown (Sales Channel: Retail Store)
- **Subperiod:** Fall 2024 - Subperiod 1 (Forecast Start Date: 2024-08-01, End Date: 2024-08-31)

- **Subperiod AOP Target (Revenue) for Fall 2024 - Subperiod 1:** \$50,000
- **Performance Parameters (from AOP for Fall 2024 Period):**
 - Target MRP: \$100 per unit
 - Target Discount Rate: 10%
 - Target Gross Margin: 40%
 - Target Cover Duration: 4 weeks

What Happens

1. First, the module identifies that “Fall 2024 - Subperiod 1” is a future period relative to today’s date (July 15, 2024). This means it will use projected calculations based on AOP and performance parameters, not actual historical sales.
2. Then, it calculates the projected Average Selling Price (ASP) for “Urban Chic” casual dresses in “Store Downtown” using the Target MRP and Target Discount:
 - $ASP = \text{Target MRP} - (\text{Target Discount Rate} / 100 * \text{Target MRP})$
 - $ASP = \$100 - (10 / 100 * \$100) = \$100 - \$10 = \$90$ per unit
3. Next, it calculates the projected sales quantity (Units Sold) using the AOP Revenue Target and the calculated ASP:
 - $\text{Projected Quantity Sold} = \text{AOP Revenue Target} / \text{ASP}$
 - $\text{Projected Quantity Sold} = \$50,000 / \$90 = 556$ units (rounded)
4. It calculates the projected Discount Value:
 - $\text{Discount Value} = \text{AOP Revenue Target} * (\text{Target Discount Rate} / 100)$
 - $\text{Discount Value} = \$50,000 * (10 / 100) = \$5,000$
5. It calculates the projected Cost of Goods Sold (COGS) using the ASP and Target Gross Margin:
 - $\text{COGS} = \text{ASP} * (1 - (\text{Target Gross Margin} / 100))$
 - $\text{COGS} = \$90 * (1 - (40 / 100)) = \$90 * 0.60 = \$54$ per unit
6. Finally, it calculates the projected target inventory for the subperiod using the Projected Quantity Sold and Target Cover Duration (assuming sales per week = total subperiod sales / number of weeks in subperiod, roughly 4 weeks for a month):
 - $\text{Projected Inventory} = \text{Projected Quantity Sold (for subperiod)} * (\text{Target Cover Duration} / \text{Weeks in Subperiod})$
 - $\text{Projected Inventory} = 556 \text{ units} * (4 \text{ weeks} / 4 \text{ weeks}) = 556$ units (simplified for monthly subperiod). (Note: The code (`r.coverDuration * r.salesQty`) simplifies this for the output, implying `r.salesQty` might be a rate or `coverDuration` is a multiplier based on it). Let’s assume `coverDuration` is in terms of sales quantity units for simplicity in this example.
 - $\text{Projected Inventory} = \text{Target Cover Duration (in sales quantity equivalent)} * \text{Projected Quantity Sold (per period unit of measure)}$
 - $\text{Projected Inventory} = 4 * 556 \text{ units} = 2,224$ units (This assumes cover duration is 4 * total subperiod sales, which is unlikely. Let’s adjust to be clearer)
 - Let’s assume `coverDuration` is defined in *weeks of supply*, and the subperiod is 4 weeks. Then
 $\text{ROS} = \text{Projected Quantity Sold} / \text{Weeks in Subperiod}$.
 $\text{Target Inventory} = \text{ROS} * \text{Target Cover Duration}$.
 - $\text{Projected ROS (units/week)} = 556 \text{ units} / 4 \text{ weeks} = 139 \text{ units/week}$
 - $\text{Projected Target Inventory} = 139 \text{ units/week} * 4 \text{ weeks} = 556$ units. (This matches $\text{inventory} = \text{coverDuration} * \text{salesQty}$ if `salesQty` is for one week and `coverDuration` is in weeks). The code shows `r.inventory = (int) (r.coverDuration * r.salesQty)`. This implies `salesQty` is

- either per week or coverDuration refers to total *period sales*. Given the problem description says “indicating how long current inventory can cover sales”, let’s assume salesQty here implicitly means ROS. Let’s simplify.
- Projected Inventory = Projected Quantity Sold (for the subperiod) * Target Cover Duration (expressed as a multiplier of subperiod sales). Let’s assume coverDuration means “how many times over” the subperiod’s sales the inventory should be. This seems more plausible from the code logic. If coverDuration = 4, then inventory is 4 times the salesQty for the period.
 - Projected Inventory = 4 * 560 units (approx) = 2,240 units (using 560 as a round number for 556 to make sense with a target of 4x sales)

Final Outcome

The module produces a WSSI Projected record for “Store Downtown,” “Women’s Casual Dresses,” for “Fall 2024 - Subperiod 1” with: Projected Revenue \$50,000, Projected Quantity 556 units, Projected ASP \$90, Discount % 10%, Projected Inventory 2,224 units, and Projected COGS per unit \$54. It also outputs an AOP record with these same values.

Key Takeaway: This example shows how financial targets and strategic parameters are meticulously translated into detailed sales and inventory projections, providing clear, actionable goals for future periods.

7. Results

This step produces:

Main Outputs

- **WSSI Actual and Projected Sales & Inventory Data:** This comprehensive output combines both historical (actual) and future (projected) performance metrics for each store, fashion grade, and subperiod. It includes sales revenue, quantity sold, Average Selling Price (ASP), discount value, inventory levels, and calculated cover duration. This provides a single, unified view of performance over time.
- **WSSI Annual Operating Plan (AOP) Data:** This output details the target sales revenue, quantity, Average Selling Price (ASP), inventory, and Cost of Goods Sold (COGS) for each store, fashion grade, and subperiod, aligning with the strategic financial plan.
- **Updated Cleaned Last Year (LY) Sales Data (with inventory):** This output enhances the previously generated Last Year sales data by incorporating the corresponding historical inventory levels, providing a more complete picture of past performance efficiency.

Business Reports

- **Performance vs. Plan Dashboards:** Used by retail merchants and inventory planners to compare actual sales and inventory against projected targets and the Annual Operating Plan (AOP), identifying areas of over- or under-performance.

- **Inventory Health Reports:** Utilized by inventory managers to assess current and projected stock levels, cover duration, and potential stockouts or overstocks, informing decisions about replenishment and markdown strategies.
- **Financial Projections and Profitability Analysis:** Used by financial planners and business leaders to understand expected revenue, Cost of Goods Sold (COGS), and gross margin contributions across categories and timeframes, guiding financial strategy.
- **Open-to-Buy Reconciliation:** Provides the necessary data for buyers to reconcile their purchasing plans against both actual performance and projected targets, helping them manage their budget effectively and ensure Optimum Depth (OD) and Optimum Width (OW).

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|---|--|--|
| Missing or delayed data feed for Cleaned Sales, AOP, or Keyframes | The module will process with available data, potentially leading to incomplete or outdated actual/projected figures. Projected calculations will use default or zero values if AOP/performance parameters are missing. | Contact the data engineering or IT team immediately to resolve the data feed issue and re-run the module once data is complete. |
| Annual Operating Plan (AOP) or Performance Parameters are missing for a specific future period/category combination | The module will typically default to an Annual Operating Plan (AOP) of zero for that combination, leading to zero projected sales and inventory. Default values for MRP, discount, margin, and cover duration will be used (often zero). | Verify that all planned categories have their AOP and corresponding performance parameters defined. If a plan exists, input the data and re-run the module. |
| Inconsistent or unrealistic Performance Parameters (e.g., extremely high discount, very low margin) | The calculated Average Selling Price (ASP), projected sales quantity, inventory, or COGS may appear illogical or unrealistic, impacting the reliability of the projections. | Review the input performance parameters from the Annual Operating Plan. Collaborate with finance and merchandising teams to ensure these targets are realistic and aligned with business strategy. |
| Historical Inventory Snapshots are sparse or inconsistent | The calculation of Last Year's inventory and cover duration for actual periods may be inaccurate or incomplete, leading to a | Ensure that inventory keyframe data is collected regularly and accurately. Investigate data capture issues and improve data quality processes for inventory tracking. |

| Situation | What Happens | What You Should Do |
|-----------|-------------------------------------|--------------------|
| | distorted view of past performance. | |

9. Next Steps

Before This Step

- **WSSI Cleaned Sales Computation Module:** This crucial preceding module processes raw sales data into reliable, aggregated “cleaned sales” figures for both past performance and recent forecasts. We need its output to capture actual sales for historical periods.
- **WSSI Subperiod AOP Computation Module:** This module precisely calculates the Annual Operating Plan (AOP) at a detailed subperiod level by distributing broader period-level AOP. We need its output as the target revenue baseline for future projections.
- **Inventory Keyframe Data Collection:** This activity captures snapshots of inventory levels for all unique items (SKUs) at regular intervals. We need this data to determine actual historical inventory for performance analysis.
- **Performance Parameters Definition/Input:** This activity defines the target discount, margin, cover duration, and MRP for various product categories and periods, usually as part of the broader Annual Operating Plan. We need these values to derive projected sales and inventory for future periods.

After This Step

- **Rate of Sale (ROS) Calculation Module:** This module takes our actual and projected sales quantities to calculate how quickly products are selling. They need our accurate units sold over time to determine reliable selling speeds, which is essential for Ideal Size Set (ISS) and Optimum Depth (OD) planning.
- **Inventory Planning Module:** This module heavily relies on the actual and projected sales, inventory, and AOP figures (including COGS) to forecast future inventory needs, generate purchase orders, and manage stock levels. They need our precise data to avoid overstocking or stockouts and ensure Optimum Depth (OD) and Optimum Width (OW) are met.
- **Financial Reporting and Analysis:** Various business intelligence dashboards and reports consume this detailed actual and projected data to provide insights into category performance, store productivity, gross margin, and overall business health. They need our aggregated data as the source of truth for their analytics and decision-making.

10. Common Questions

Q: Why do we have separate “Actual and Projected” and “AOP” outputs if they often show the same numbers for future periods? A: While they may align for future periods, the “Actual and Projected” output is a dynamic view that becomes “actual” as time passes. The “AOP” output always represents the fixed, strategic plan. This distinction allows us to track performance against the plan and see how projections evolve with real-world data.

Q: How does this module help improve inventory buying decisions? A: By providing specific target inventory levels and expected sales quantities for future subperiods, this module gives inventory planners a highly accurate picture of anticipated demand. This precision enables them to order the right quantity of items, ensuring adequate stock for peak demand without holding excess inventory during slower periods, improving our Optimum Depth (OD) and Optimum Width (OW).

Q: What is “Cover Duration” and why is it important for my planning? A: Cover Duration indicates how many weeks or days your current inventory can cover future sales at the projected Rate of Sale (ROS). It’s a critical metric for inventory efficiency, helping you avoid both stockouts (too low cover duration) and overstock (too high cover duration), which impacts Ideal Size Set (ISS) and overall inventory health.

Q: What if a product group or store had zero actual sales in a historical period, but still has a projected AOP? A: For actual periods, the module will accurately record zero sales. For future periods, if there’s an AOP defined, the module will proceed with calculating projected sales and inventory based on that AOP and the performance parameters, ensuring that even new or historically non-selling categories receive a planned target.

Q: How does this module contribute to understanding profitability? A: By calculating Cost of Goods Sold (COGS) for both actual and projected periods, this module provides the foundational data needed to determine gross margin at a detailed level. This allows merchandisers and finance teams to assess the profitability of different product categories and make informed pricing and sourcing decisions.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Computes a comprehensive view of actual and projected sales, inventory, and Annual Operating Plan (AOP) figures for detailed planning.
- **Main Input:** Cleaned historical and forecast sales, subperiod AOP targets, inventory snapshots, and key performance parameters.
- **Main Output:** Unified WSSI Actual & Projected Data, detailed WSSI AOP Data, and updated Last Year Cleaned Sales with inventory.
- **Warning Signs:** Unrealistic or missing projected figures, discrepancies between actuals and projections that are not explainable by market shifts, or illogical inventory levels.
- **Key Settings:** Target Discount Rate, Target Gross Margin Percentage, Target Cover Duration, and Target MRP, which are inputs from the Annual Operating Plan.

Need Help? Contact your merchandising operations or data analytics team for questions about this process.

Chapter 6: WSSI Output Generation Module

Last Updated: 2025-05-23

1. Business Objective

This module consolidates all computed WSSI metrics from upstream modules, along with current inventory and Goods Received Note (GRN) data, to generate the final, comprehensive WSSI output at both store and channel levels. It also plays a key role in tagging styles to appropriate sales channels based on inventory bucket flow logic and GRN dates, influencing how inventory is managed and tracked across different sales channels.

Why This Matters: This foundational module provides a holistic view of actual performance, Annual Operating Plan (AOP) targets, and Last Year (LY) comparisons for sales revenue, quantity, inventory, cover duration (related to Rate of Sale or ROS), discount, and Average Selling Price (ASP). This comprehensive view is essential for making informed buying decisions, optimizing inventory levels (Optimum Depth - OD and Optimum Width - OW), and ensuring that stock is available in the right channels at the right time.

2. Required Information

We need these pieces of information to do our job:

Sales Data

- **Cleaned Sales Data (Last Year and Forecast):** Aggregated sales figures including units sold, revenue, and discount for past and recent forecast periods. We need this to understand historical demand and to capture recent sales performance accurately.
- **Warehouse and Store Inwards Data:** Records of quantities of new stock received at warehouses and individual stores on specific dates. We need this to calculate total incoming inventory for each period.
- **Historical Inventory Snapshots (Keyframes):** Records of actual inventory levels for unique items (SKUs) at specific points in time at stores. We need this to understand past inventory positions and calculate historical cover duration.

Product Data

- **Product Attributes:** Information like a product's Fashion Grade, Brand Segment, Brand, and unique Style IDs. This is essential to categorize sales meaningfully and group products for reporting.
- **Style-SKU Mappings & Product Hierarchies:** Details on how individual unique items (SKUs) belong to broader styles and how styles fit into parent styles. We need this to correctly aggregate sales and inventory across different product classifications.

Operational Data

- **Merchandise Planning Periods:** The defined start and end dates for each subperiod and period for Last Year and the forecast. We need these definitions to correctly attribute sales and inventory to relevant timeframes.
- **Store Details and Sales Channel Mappings:** Information linking each store to its specific sales channel (e.g., online, brick-and-mortar). We need this to correctly attribute sales and inventory to the relevant sales channel for comprehensive reporting.
- **Inventory Bucket Flow Rules:** Pre-defined logic that dictates how styles transition between different inventory buckets (e.g., "New Arrivals," "Core," "Seasonal Clearance") and their associated sales channels over time, based on factors like shelf life and lead time. We need this to dynamically tag styles to their most appropriate sales channels.
- **Style Inventory Bucket Assignment:** Which specific inventory bucket each style is initially assigned to. We need this as the starting point for applying the inventory bucket flow rules.
- **Goods Received Note (GRN) Dates for Styles:** The earliest date a style was received into inventory across the network. This is crucial for determining the starting point of its journey through the inventory bucket flow logic.
- **Annual Operating Plan (AOP) Data (subperiod level):** Detailed financial targets (revenue, quantity, inventory, discount, ASP, Cost of Goods Sold) broken down by store, fashion grade, and specific subperiod. We need this as the target baseline for future projections and performance comparison.

3. Business Settings

| Setting Name | Default Value | Description | When to Change |
|--------------------------------|------------------------|---|---|
| Inventory Bucket Flow Logic | N/A (Defined by Rules) | This dictates how a style's inventory is categorized and flows through sales channels based on its age (linked to GRN date), shelf life, and lead time. | When there are strategic changes to product lifecycle management, promotion strategies, or inventory disposition processes. |
| Sales Channel Contribution AOP | N/A (Derived from AOP) | This represents the target sales revenue | When overall financial targets for sales channels |

| Setting Name | Default Value | Description | When to Change |
|--------------------------------|----------------|--|---|
| Style Channel Assignment Logic | N/A (Internal) | <p>contribution of each sales channel within a broader category. It's used to proportionally attribute inventory and inwards if a style is assigned to multiple channels.</p> <p>This rule determines which sales channels a style is tagged to based on its GRN date and the defined inventory bucket flow.</p> | <p>change, or if strategic focus shifts between online vs. physical retail.</p> <p>This logic is generally hard-coded but may be reviewed if the fundamental approach to inventory channel allocation needs revision.</p> |

Note: The settings primarily relate to how inventory is attributed and flows through sales channels, and how aggregate metrics are calculated. These are typically managed through data inputs or internal configurations from upstream modules and master data rather than direct user adjustments in this computation module.

4. Process Flow

Here's how information moves through this step:

flowchart LR

A[WSSI Metrics (Actual/Projected, AOP), Cleaned LY Sales, Inventory (Keyframes, WH Stock, GRN), Inwards, GRN Dates, Inventory Bucket Flow Rules] --> B[WSSI Output Generation & Style Channel Tagging]

B --> C[Final WSSI Output (Store & Channel), Style Sales Channel Tags]

1. Start with → All computed WSSI metrics (Actual and Projected Sales, AOP targets, Last Year Sales) from upstream modules, detailed inventory snapshots, Goods Received Note (GRN) dates, and the pre-defined inventory bucket flow rules.
2. Then → The module determines the earliest Goods Received Note (GRN) date for each individual style.
3. Next → It dynamically tags each style to its most appropriate sales channel(s) based on its GRN date and the specified inventory bucket flow logic (e.g., after X weeks, move from “New Arrivals” to “Core” bucket, associated with different channels).

4. Next → It consolidates all computed WSSI metrics (sales, inventory, cover duration, discount, ASP) for each store, attributing sales and inventory according to the newly tagged sales channels.
5. Finally → The module aggregates these store-level metrics to generate comprehensive, channel-level WSSI output, along with detailed reports on how styles were tagged to channels.

5. Step-by-Step Actions

1. **Gather** All Necessary Data Inputs
 - What we do: *We collect comprehensive cleaned sales data, inventory snapshots, GRN records, subperiod AOP data, and all relevant master data like style-to-SKU mappings and inventory bucket flow rules.*
 - Why we do it: *This provides all the raw ingredients needed to produce a complete and accurate WSSI output.*
2. **Determine** Style-Specific Goods Received Note (GRN) Dates
 - What we do: *For each style, we identify its earliest Goods Received Note (GRN) date across all stores. This pinpoints when the style first entered the network.*
 - Why we do it: *The GRN date is crucial as it acts as the starting point for applying the inventory bucket flow logic, which determines a style's optimal sales channel over its lifecycle.*
3. **Tag** Styles to Sales Channels
 - What we do: *Based on each style's earliest GRN date, its assigned inventory bucket, and the defined inventory bucket flow rules (which specify shelf life and lead times for transitions), we dynamically assign the style to its current optimal sales channel(s).*
 - Why we do it: *This ensures that inventory is tracked and attributed to the channels where it is most likely to sell given its age and lifecycle stage, optimizing sales opportunities.*
4. **Compute** Store-Level WSSI Output
 - What we do: *For each store, we consolidate actual and projected sales figures, current inventory levels (adjusted by channel tagging), cover duration, discount percentages, and Average Selling Prices, presenting them alongside AOP targets and Last Year comparisons.*
 - Why we do it: *This provides a granular, store-specific view of performance, enabling local inventory management and performance analysis.*
5. **Aggregate** Channel-Level WSSI Output
 - What we do: *We sum up the store-level WSSI metrics by sales channel to generate a holistic view of performance and inventory at the channel level. This includes combining current inventory levels from both stores and warehouses, considering channel attribution.*
 - Why we do it: *This provides a strategic, overarching view for channel managers and merchandisers to assess the health of their entire online or brick-and-mortar business, guiding high-level inventory deployment and sales strategies.*

6. Sample Scenario

Let's walk through a real example of how a style is tagged to a sales channel:

Starting Point

Imagine a new product, the “Classic Wool Coat” (Style ID: 12345, Fashion Grade: Winter Wear, Brand: Arctic). - Its earliest **Goods Received Note (GRN) Date** is August 1, 2024. - It’s initially assigned to the “**Seasonal Core**” inventory bucket. - Today’s **Run Date** for the WSSI process is October 15, 2024. - The **Merchandise Planning Periods** relevant are: - Subperiod: Aug-Sep 2024 - Subperiod: Oct-Nov 2024 (this is the current subperiod for our run date) - Subperiod: Dec-Jan 2025 - The **Inventory Bucket Flow Rules** for “Seasonal Core” are defined as: - **Flow 1**: fromSalesChannel = “Retail Stores Only”, shelfLife = 1 subperiod, leadTime = 0. - **Flow 2**: fromSalesChannel = “Online and Outlet”, shelfLife = 2 subperiods, leadTime = 0.

What Happens

1. First, the module identifies that the “Classic Wool Coat” (Style ID: 12345) has a GRN date of August 1, 2024, which falls within the Aug-Sep 2024 subperiod.
2. Next, it applies the “Seasonal Core” inventory bucket flow rules. For “Flow 1,” the shelfLife is 1 subperiod. This means after one full subperiod following its GRN date, the product is expected to transition.
3. Since the GRN was in Aug-Sep 2024, one subperiod of “shelf life” means the effective period for Flow 1 is roughly from the end of Aug-Sep until the end of Oct-Nov.
4. The current WSSI run date is October 15, 2024, which falls within the Oct-Nov 2024 subperiod. This period is still within the shelfLife specified for Flow 1 (i.e., it hasn’t yet transitioned to Flow 2).
5. Therefore, the module tags “Classic Wool Coat” as belonging to the “Retail Stores Only” sales channel based on the rules for Flow 1 of the “Seasonal Core” bucket.
6. Finally, any sales or inventory for “Classic Wool Coat” (Style ID: 12345) will now be attributed to the “Retail Stores Only” sales channel within the WSSI output for the current period. This process is repeated for all other styles based on their individual GRN dates and inventory bucket assignments.

Final Outcome

The “Classic Wool Coat” is now officially tagged as a “Retail Stores Only” style for the current WSSI cycle. This means its sales performance and inventory levels are clearly attributed to physical stores in the WSSI output, allowing merchandisers to focus their efforts on store-level sales and inventory management for this particular style.

Key Takeaway: This example illustrates how inventory bucket flow logic dynamically assigns styles to specific sales channels based on their lifecycle, ensuring accurate reporting and optimized sales strategies for each product.

7. Results

This step produces:

Main Outputs

- **Comprehensive WSSI Output (Store-Level):** This detailed output provides a complete view of actual performance, AOP targets, and Last Year comparisons for sales revenue, quantity, inventory, cover duration, discount, and Average Selling Price (ASP) for every combination of store, fashion grade, brand segment, brand, and sales channel.
- **Comprehensive WSSI Output (Channel-Level):** An aggregated view of all the above WSSI metrics, rolled up from individual stores to total sales channel performance (e.g., “Online,” “Retail Stores,” “Outlet”). This includes combined inventory from both stores and warehouses attributed to each channel.
- **Style Sales Channel Tagging Report:** A report detailing which sales channel(s) each style has been dynamically assigned to based on the inventory bucket flow logic.
- **Style First GRN Dates Report:** A report listing the earliest Goods Received Note (GRN) date for each style, critical for understanding the start of its lifecycle journey.

Business Reports

- **Overall WSSI Performance Dashboards:** Used by retail merchants, inventory planners, and leadership to monitor the health of the entire business, identify trends, compare performance against plan, and make strategic adjustments to buying and merchandising.
- **Inventory Deployment and Movement Reports:** Utilized by supply chain managers and inventory teams to understand where inventory is located (stores vs. warehouses), how it’s moving through different sales channels, and to optimize logistics and stock transfers.
- **Channel Profitability and Strategy Reviews:** Used by channel heads and financial analysts to assess the financial performance of each sales channel, informing strategic decisions about channel-specific assortments, promotions, and investments.
- **Open-to-Buy and Replenishment Insights:** Provides critical inputs for buyers to reconcile purchasing plans against actual performance and projected targets, ensuring that inventory purchases are aligned with demand and channel strategy, thereby supporting Optimum Depth (OD) and Optimum Width (OW).

8. Unusual Cases

| Situation | What Happens | What You Should Do |
|--|--|---|
| Missing or delayed data feed (e.g., GRN, Inventory Keyframes, AOP) | The module will process with available data, potentially leading to incomplete or outdated WSSI figures. Outputs may show zeros or “not found” for missing data. | Contact the data engineering or IT team immediately to resolve the data feed issue and re-run the module once data is complete. |
| Style has no Goods Received Note (GRN) date | The module may default to an initial sales channel or tag the style | Investigate why the GRN date is missing or out of range. Correct |

| Situation | What Happens | What You Should Do |
|---|--|---|
| or GRN date falls outside defined periods | as
“GRN_NOT_FOUND,”
potentially misattributing its inventory and sales. | master data or historical records and re-run the module. |
| Inventory bucket flow logic not defined for a specific style’s bucket | The module may not be able to dynamically tag the style to an appropriate sales channel, potentially leaving it untagged or defaulting to a general channel. | Define or update the inventory bucket flow rules for the affected inventory buckets to ensure all styles can be accurately categorized. |
| No sales data for a specific style/store/channel but inventory exists | The WSSI output will accurately show inventory with zero sales, resulting in an undefined or very high cover duration. | Investigate the reason for no sales (e.g., new product, merchandising issue, stock not live). This provides a critical insight into potential issues. |

9. Next Steps

Before This Step

- **WSSI Cleaned Sales Computation Module:** This crucial preceding module processes raw sales data into reliable, aggregated “cleaned sales” figures for both past performance and recent forecasts. We need its output to capture actual sales for historical periods.
- **WSSI Subperiod AOP Computation Module:** This module precisely calculates the Annual Operating Plan (AOP) at a detailed subperiod level by distributing broader period-level AOP. We need its output as the target revenue baseline for future projections.
- **WSSI Actual and Projected Sales & AOP Computation Module:** This module computes comprehensive actual and projected sales and inventory metrics, along with AOP figures. We need its output as the primary source of WSSI metrics.
- **Inventory Keyframe and Goods Received Note (GRN) Data Collection:** This activity captures snapshots of inventory levels and records GRN dates for all unique items (SKUs) at regular intervals. We need this data for accurate inventory tracking and channel tagging.
- **Inventory Bucket Flow Rule Definition:** This activity establishes the rules for how inventory transitions between different sales channels over its lifecycle. We need these rules to dynamically tag styles to channels.

After This Step

- **Business Intelligence (BI) Dashboards and Reporting Tools:** These tools consume the final WSSI output to create interactive dashboards and reports for

retail merchants, planners, and leadership. They need our comprehensive data as the single source of truth for performance monitoring.

- **Assortment Planning and Optimization Systems:** These systems use the WSSI output (especially channel-level performance and inventory data) to make decisions about future product assortments, ensuring Optimum Depth (OD) and Optimum Width (OW) are achieved. They need our aggregated data to align purchasing with channel-specific demand.
- **Financial Reporting and Forecasting:** Financial teams leverage the WSSI output to generate detailed financial statements, forecast future revenue and profitability, and assess the financial health of product categories and sales channels. They need our consolidated data for accurate financial planning.
- **Replenishment and Stock Transfer Systems:** These systems use the WSSI's inventory and sales performance data to trigger replenishment orders, initiate stock transfers between stores or channels, and manage overall inventory flow. They need our output to ensure optimal stock levels and reduce stockouts.

10. Common Questions

Q: Why are store and channel levels important in the WSSI output? A: Reporting at both store and channel levels provides a complete picture. Store-level data helps local managers optimize their specific inventory and sales efforts, while channel-level data allows corporate teams to understand broader performance trends, compare online vs. in-store effectiveness, and allocate resources strategically across the entire business.

Q: How does the Goods Received Note (GRN) date affect inventory tracking? A: The GRN date is like a product's "birth date" in your inventory. This module uses it as the starting point for applying inventory bucket flow rules. These rules dictate how long a product typically stays in its initial sales channel (e.g., "new arrivals") before transitioning to other channels (e.g., "clearance"), ensuring inventory is always tracked where it's most likely to sell.

Q: What is "inventory bucket flow logic" and why is it important? A: Inventory bucket flow logic defines a systematic way to move products through different sales channels over their lifecycle. For example, a new dress might start in "full-price retail stores," then move to "online only" after a certain period, and finally to "outlet stores" as clearance. This logic is crucial because it ensures that inventory is proactively managed, reducing holding costs and maximizing sales opportunities by getting products to the right customer at the right time.

Q: How does this module help optimize Optimum Depth (OD) and Optimum Width (OW)? A: By providing a holistic view of actual and projected sales, inventory, and GRN-driven channel tagging, this module helps planners understand demand patterns across channels. This data is critical for making informed decisions on how many unique items (OD) and how many different categories (OW) to stock, ensuring you have the right product mix where it's needed most, minimizing stockouts and excess inventory.

Q: Why is it important to see actual, AOP, and Last Year (LY) data together? A: Seeing actual performance alongside AOP targets and Last Year's results provides context and a comprehensive understanding. Actuals show what truly happened,

AOP shows what was planned, and LY shows historical trends. This comparison helps identify gaps or opportunities, evaluate the accuracy of forecasts, and make data-driven adjustments to future merchandising and inventory strategies.

11. Quick Reference

Key things to remember about this step:

- **Purpose:** Consolidates all WSSI metrics, inventory, and GRN data to produce final, comprehensive WSSI output at store and channel levels, and dynamically tags styles to sales channels based on inventory flow logic.
- **Main Input:** WSSI Actual/Projected Data, WSSI AOP Data, Cleaned Last Year Sales, Inventory Keyframes, Warehouse and Store Inwards, Goods Received Note (GRN) dates, and Inventory Bucket Flow Rules.
- **Main Output:** Comprehensive WSSI output (Store and Channel level), Style Sales Channel Tagging Report, Style First GRN Dates Report.
- **Warning Signs:** Unexplained discrepancies between actuals and plans, missing or illogical inventory numbers for specific channels, or styles tagged to unexpected sales channels.
- **Key Settings:** The pre-defined Inventory Bucket Flow Logic and the Sales Channel Contribution derived from the Annual Operating Plan (AOP).

Need Help? Contact your merchandising operations or data analytics team for questions about this process.

Generated by [AI Codebase Knowledge Builder](#)

WSSI FAQ

Version: 1.0 **Last Updated:** 2025-05-23

Purpose: This FAQ document provides answers to common questions about the WSSI (Wholesale Store Stock Initiative) algorithm, its processes, and the insights it delivers for inventory management and sales planning.

General

Q: What is WSSI? **A:** WSSI stands for Wholesale Store Stock Initiative. It is a comprehensive computation pipeline designed to provide a holistic view of actual performance, Annual Operating Plan (AOP) targets, and Last Year (LY) comparisons for sales revenue, quantity, inventory, and other key business metrics.

Q: What business problem does the WSSI algorithm solve? **A:** The WSSI algorithm helps businesses optimize their inventory by providing insights into sales performance, forecasting future demand, and suggesting ideal stock levels (like OD and ISS) to maximize sales and minimize stockouts or overstock.

Q: What is the overall flow of the WSSI system? **A:** The WSSI system processes sales and inventory data through a sequential pipeline. It starts with data validation, followed by cleaning sales data, distributing the Annual Operating Plan (AOP) at a granular level, computing actual and projected sales and inventory, and finally, generating a comprehensive output for analysis.

Q: What do the terms OD, OW, ISS, ROS, and AG mean in WSSI? **A:** * **OD (Optimum Depth):** The ideal number of unique items (SKUs) to stock in a category or display. * **OW (Optimum Width):** The ideal number of different categories or styles to offer. * **ISS (Ideal Size Set):** The best combination of sizes to maximize sales and minimize stockouts. * **ROS (Rate of Sale):** The speed at which products are sold over a period (e.g., units/week). * **AG (Attribute Group):** A set of product attributes grouped for analysis (e.g., color, size, brand).

Q: Who typically uses the insights from WSSI? **A:** Merchandisers, inventory planners, store managers, and financial analysts use WSSI insights to make informed decisions about purchasing, stock allocation, sales forecasting, and performance evaluation.

Configuration & Settings

Q: How does WSSI determine ideal stock levels (OD, ISS)? **A:** WSSI uses a `coverDuration` metric, defined by business parameters, to calculate how long current or projected inventory can cover future sales (ROS). This `coverDuration` helps in determining the Optimum Depth (OD) and Ideal Size Set (ISS) needed to meet demand without excessive stock.

Q: Can we adjust the business parameters for future projections? **A:** Yes, the WSSI algorithm uses predefined performance parameters, such as target discount, margin, and `coverDuration`, to derive projected sales (ROS), inventory, and Cost of Goods Sold (COGS) for future periods. These parameters are configurable inputs to the system.

Q: How are sales targets incorporated into WSSI? **A:** Sales targets are integrated through the Annual Operating Plan (AOP). The `WSSI Subperiod AOP Computation Module` takes broader AOP data and distributes it across granular subperiods based on historical revenue contribution, ensuring targets are accurately proportioned over time for planning.

Data & Inputs

Q: What kind of data does WSSI require as input? **A:** WSSI primarily uses raw sales data, inventory figures, Goods Received Note (GRN) data, and Annual Operating Plan (AOP) targets. These inputs are processed at various levels, including store, fashion grade (FG), brand segment, and subperiod.

Q: How does WSSI handle raw sales data? **A:** The `WSSI Cleaned Sales Computation Module` processes raw sales data to generate 'cleaned' figures. This involves aggregating sales, revenue, quantity, discount value, discount percentage, and Average Selling Price (ASP) to ensure data accuracy for analysis.

Q: Where does the Annual Operating Plan (AOP) data originate for WSSI? **A:** The AOP data is provided to the `WSSI Subperiod AOP Computation Module` at a broader period level and is then systematically distributed to more granular subperiod levels based on historical revenue contribution patterns.

Q: Does WSSI account for incoming inventory (GRN)? **A:** Yes, the `WSSI Output Generation Module` consolidates inventory and Goods Received Note (GRN) data along with other computed WSSI metrics to provide a complete picture of stock levels and future incoming goods.

Process & Logic

Q: How does WSSI calculate the Rate of Sale (ROS) for future periods? **A:** The `WSSI Actual and Projected Sales & AOP Computation Module` derives projected sales (ROS) for future periods by using historical cleaned sales data and applying predefined performance parameters like target discount, margin, and `cover duration`, along with the AOP targets.

Q: What validations are performed on the input data? **A:** The WSSI Validation Module performs critical checks on input data. For example, it ensures that the specified run date falls within a recognized subperiod, which is essential for accurate historical and forecasted calculations, preventing downstream errors.

Q: How is the Annual Operating Plan (AOP) distributed across different time periods? **A:** The WSSI Subperiod AOP Computation Module takes AOP data defined at a broader period level and intelligently distributes it across granular subperiods. This distribution is based on the historical revenue contribution of each subperiod, ensuring time-accurate planning.

Q: How does WSSI assign styles to sales channels? **A:** The WSSI Output Generation Module plays a key role in tagging styles to appropriate sales channels. This is done based on inventory bucket flow logic and Goods Received Note (GRN) dates, which influences how inventory is managed and tracked across different sales channels.

Outputs & Results

Q: What kind of insights can I gain from the WSSI output? **A:** The WSSI output provides a holistic view of actual performance, AOP targets, and Last Year (LY) comparisons. You can see detailed metrics for sales revenue, quantity, inventory, cover duration (related to ROS and OD), discount, and Average Selling Price (ASP).

Q: At what level of detail are the WSSI outputs generated? **A:** The WSSI output is generated at both store and channel levels, providing granular insights that support detailed planning and analysis for specific locations and sales approaches.

Q: How does WSSI help identify potential stockouts or overstock situations? **A:** By providing cover duration metrics and ideal stock levels (OD, ISS), WSSI helps identify categories or items where inventory might be insufficient to cover projected sales (potential stockouts) or excessive relative to demand (overstock).

Q: What is 'cover duration' in the WSSI output and why is it important? **A:** 'Cover duration' indicates how long current inventory can cover projected sales (ROS). It's a crucial metric for Ideal Size Set (ISS) and Optimum Depth (OD) planning, helping you understand if you have enough stock to meet demand for a specified period.

Error Handling & Edge Cases

Q: What happens if the run date for WSSI is invalid or outside a recognized period? **A:** The WSSI Validation Module will detect this. It performs checks to ensure the specified run date falls within a recognized subperiod. If it's invalid, the system will prevent further computations to maintain data integrity.

Q: How does WSSI handle potential data quality issues in raw sales figures? **A:** The WSSI Cleaned Sales Computation Module is designed to process raw sales data into 'cleaned' figures. While specific handling of *missing* or *corrupt* data isn't detailed, the "cleaning" implies aggregation and validation steps to ensure the data used for analysis is reliable.

Q: What if there are significant, unforecasted events (e.g., a major promotion or unexpected store closure)? **A:** While the WSSI system relies on historical patterns for AOP distribution and cleaned sales, significant deviations would impact future ROS and inventory projections. Reviewing the cleaned sales and AOP inputs would be crucial to ensure they reflect such events, or manual adjustments may be needed.

Need More Help? Please contact your internal support team for further assistance.

wssi Documentation

Version: 1.0

Last Updated: 2025-05-23

2. Executive Summary

The WSSI (Wholesale Store Stock Initiative) algorithm is a crucial business intelligence tool designed to optimize inventory management and sales planning across retail channels. It processes historical and forecasted sales data, integrates with Annual Operating Plans (AOP), and provides actionable insights on inventory levels, sales performance, and profitability metrics. The key outcome is improved stock efficiency, reduced stockouts, maximized sales, and enhanced strategic decision-making for merchandise planning, contributing to better financial performance.

3. Architecture Overview

```mermaid

graph TD

subgraph Core Orchestration

WVM[WSSI Validation Module]

WGM[WSSI Group Module (Orchestration)]

end

subgraph Data Processing

WCSC[WSSI Cleaned Sales Computation Module]

WSAC[WSSI Subperiod AOP Computation Module]

WAAP[WSSI Actual and Projected Sales & AOP Computation Module]

WOGM[WSSI Output Generation Module]

end

WVM -- precedes --> WGM

WGM -- executes --> WCSC

WGM -- executes --> WSAC

WGM -- executes --> WAAP

WGM -- executes --> WOGM

WCSC -- data\_flow: WssiLyCleanedSalesRow --> WSAC

WCSC -- data\_flow: WssiCleanedSalesRow --> WAAP

WSAC -- data\_flow: WssiStoreFgSegSubperiodAopRow --> WAAP

WAAP -- data\_flow: WssiActProjRow, WssiAopRow --> WOGM

WCSC -- data\_flow: WssiLyCleanedSalesRow (updated by WAAP) --> WOGM

WSAC -- data\_flow: WssiStoreFgSegSubperiodAopRow --> WOGM

## Module Descriptions:

- **WSSI Group Module (Orchestration):** This module orchestrates the entire WSSI computation pipeline, ensuring correct data flow and managing pre/post-computation tasks.
- **WSSI Validation Module:** This module performs critical checks on input data, ensuring integrity and preventing errors in subsequent computations.
- **WSSI Cleaned Sales Computation Module:** This module processes raw sales data to generate cleaned sales figures for Last Year (LY) and forecast periods, calculating key metrics.
- **WSSI Subperiod AOP Computation Module:** This module calculates the Annual Operating Plan (AOP) at a granular subperiod level by distributing broader period AOP based on historical revenue.
- **WSSI Actual and Projected Sales & AOP Computation Module:** This module computes comprehensive actual and projected sales, inventory metrics, and AOP figures, deriving metrics like Rate of Sale (ROS) and cover duration (related to OD/ISS).
- **WSSI Output Generation Module:** This module consolidates all computed WSSI metrics, inventory, and GRN data to generate final, comprehensive WSSI outputs at store and channel levels.

## 4. Prerequisites & Setup

To run the WSSI pipeline, ensure the following are in place:

- **Java Version:** Java 11 or higher.
- **Build Tool:** Maven (preferred) or Gradle.
- **Dependencies:** All project dependencies are managed via `pom.xml` (for Maven) or `build.gradle` (for Gradle).
- **Database:** Access to the configured database containing raw sales, inventory, AOP, and other relevant data.
- **Build Command:** To build the project, navigate to the `<project_name>` root directory and run `mvn clean install` (for Maven) or `gradle build` (for Gradle).
- **Running the Orchestrator:** Execute the WSSI pipeline by running the main class of the `module-group` orchestrator, typically via `java -jar target/<project_name>-<version>.jar` with the `args.properties` file specified as a program argument.

## 5. Folder & File Structure

```
wssi/
├─ module-group/ # Contains the orchestrator module
for running the entire pipeline.
├─ submodules/ # Contains individual computational
modules for specific WSSI steps.
├─ args.properties # Global configuration parameters for
the WSSI application.
├─ common-data.json # Defines the shared data model
```

schema used across modules.  
└─ outputs/ # Directory where the final WSSI  
output reports are generated.

## 6. Global Configuration

This section details the configuration parameters found in `args.properties`. These settings control the overall behavior of the WSSI pipeline.

| Setting Name       | Default Value                            | Description                                                                                                               | Valid Range/<br>When to Change                                                                                             |
|--------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| run.date           | YYYY-MM-DD (current date)                | The date for which the WSSI computation is performed. All historical and forecast calculations are relative to this date. | Must be a valid date within recognized subperiods. Change for historical runs or specific planning cycles.                 |
| data.source.db.url | jdbc:postgresql://localhost:5432/wssi_db | JDBC URL for the input/output database.                                                                                   | Update for different database environments (e.g., dev, staging, production).                                               |
| output.file.path   | outputs/wssi_results.csv                 | The file path where the final WSSI consolidated report will be saved.                                                     | Adjust to change the output location or file name.                                                                         |
| enable.validation  | true                                     | Controls whether the WSSI Validation Module runs at the beginning of the pipeline.                                        | Set to false only during development or for debugging data issues if validation is known to fail and needs to be bypassed. |
| snapshot.enabled   | false                                    | If true, intermediate data                                                                                                | Enable for detailed debugging or                                                                                           |

| Setting Name | Default Value | Description                                                  | Valid Range/<br>When to Change                           |
|--------------|---------------|--------------------------------------------------------------|----------------------------------------------------------|
|              |               | snapshots are taken at key stages for debugging or recovery. | long-running production processes requiring checkpoints. |

## 7. Shared Data Model

The `common-data.json` schema defines the structure of data exchanged and persisted across various WSSI modules. This ensures data consistency and interoperability.

| Field                            | Type    | Description                                                                                        |
|----------------------------------|---------|----------------------------------------------------------------------------------------------------|
| <code>subperiodId</code>         | Integer | Unique identifier for a financial subperiod (e.g., week, month).                                   |
| <code>storeId</code>             | String  | Unique identifier for a retail store.                                                              |
| <code>fashionGrade</code>        | String  | Categorization of products based on their fashion lifecycle (e.g., ‘Core’, ‘Fashion’, ‘Seasonal’). |
| <code>brandSegment</code>        | String  | Segment of the brand (e.g., ‘Luxury’, ‘Value’, ‘Mainstream’).                                      |
| <code>revenue</code>             | Double  | Total sales revenue for a given period.                                                            |
| <code>quantity</code>            | Integer | Total number of units sold.                                                                        |
| <code>discountValue</code>       | Double  | Total monetary value of discounts applied.                                                         |
| <code>discountPercentage</code>  | Double  | Average discount percentage on sales.                                                              |
| <code>averageSellingPrice</code> | Double  | Average price at which a product is sold.                                                          |
| <code>inventoryUnits</code>      | Integer | Number of units currently in stock.                                                                |
| <code>grnUnits</code>            | Integer | Number of units in Goods Received Notes (incoming stock).                                          |
| <code>rateOfSale</code>          | Double  | <b>ROS:</b> Speed at which units are sold per period (e.g., units/week).                           |
| <code>optimumDepth</code>        | Integer | <b>OD:</b> The ideal number of unique items (SKUs) to stock in a category or display.              |
| <code>optimumWidth</code>        | Integer | <b>OW:</b> The ideal number of different categories or styles to offer.                            |
| <code>idealSizeSet</code>        | String  | <b>ISS:</b> The best combination of sizes to maximize sales and minimize stockouts.                |

| Field           | Type    | Description                                                               |
|-----------------|---------|---------------------------------------------------------------------------|
| coverDuration   | Double  | Number of periods current inventory can cover based on ROS.               |
| aopRevenue      | Double  | Annual Operating Plan (AOP) target for revenue.                           |
| aopQuantity     | Integer | Annual Operating Plan (AOP) target for quantity.                          |
| costOfGoodsSold | Double  | Cost incurred in selling products (COGS).                                 |
| channelTag      | String  | Tag indicating the sales channel (e.g., 'Online', 'Retail', 'Clearance'). |

## 8. Submodule Chapters

### 8.1 WSSI Group Module (Orchestration)

#### 1. Module Name & Business Objective

**WSSI Group Module (Orchestration):** This module orchestrates the entire WSSI (Wholesale Store Stock Initiative) computation pipeline. It defines the sequence of execution for various sub-modules, ensuring that data flows correctly between different stages of the WSSI process. It also manages pre-computation snapshots and post-computation output synchronization. The business objective is to provide a robust, automated, and error-resistant framework for generating WSSI insights by coordinating complex data processing steps efficiently.

#### 2. Required Information

- **Operational Data:**
  - `run.date`: The specific date for which the WSSI computation should be run.

#### 3. Business Settings

- `snapshot.enabled`: Controls whether intermediate data snapshots are taken.
- `output.synchronization.enabled`: Determines if final outputs are synchronized to external systems.

#### 4. Process Flow

```
graph TD
 A[Start Orchestration] --> B{Validate Data?};
 B -- Yes --> C[Run Validation Module];
 B -- No --> D[Begin Computation Pipeline];
 C --> D;
```

```
D --> E[Execute Cleaned Sales Comp.];
E --> F[Execute Subperiod AOP Comp.];
F --> G[Execute Actual/Projected Comp.];
G --> H[Execute Output Generation];
H --> I[Synchronize Outputs];
I --> J[End Orchestration];
```

## 5. Step-by-Step Actions

1. **Initialize Pipeline:** *The orchestrator prepares the environment and loads global configurations.*
2. **Validate Inputs (Optional): Checks** `run.date` and other critical parameters to ensure data integrity before heavy computation begins.
3. **Execute Cleaned Sales Computation: Triggers** the WSSI Cleaned Sales Computation Module to process raw sales data into a cleaned format.
4. **Execute Subperiod AOP Computation: Activates** the WSSI Subperiod AOP Computation Module to distribute the annual operating plan into granular subperiod targets.
5. **Execute Actual and Projected Sales & AOP Computation: Launches** the WSSI Actual and Projected Sales & AOP Computation Module to derive comprehensive actual and future sales and inventory metrics, including ROS and cover duration.
6. **Execute Output Generation: Initiates** the WSSI Output Generation Module to consolidate all processed data into final reports.
7. **Synchronize Outputs: Transfers** the final WSSI output to specified output locations or downstream systems.

## 6. Sample Scenario

Imagine your business runs WSSI weekly. The Group Module is configured to run every Monday. - **Action:** On Monday, 2025-05-27, the WSSI Group Module is executed. - **Process:** It first calls the Validation Module to check if 2025-05-27 is a valid run date within the defined subperiods. Upon validation, it proceeds to sequentially trigger the Cleaned Sales, AOP, Actual/Projected, and Output Generation modules. - **Outcome:** The entire WSSI pipeline completes successfully, generating the latest inventory and sales insights for the week. **Key Takeaway:** The Group Module ensures a consistent and automated flow of data through all WSSI processing steps, guaranteeing timely insights.

## 7. Results

The successful completion of the WSSI pipeline, with all sub-modules executed in the correct sequence, and final consolidated outputs synchronized to the specified location.

## 8. Unusual Cases

- **Missing or delayed data feed:** If any upstream data source (e.g., raw sales, inventory) is delayed, the orchestration may fail due to missing inputs for sub-modules.

- **Module failure:** If a sub-module encounters an error (e.g., database connection issue, data type mismatch), the orchestration will halt, requiring manual intervention.

## 9. Next Steps

- Business users consume the generated WSSI reports for merchandise planning, inventory rebalancing, and sales forecasting.
- Integration with BI dashboards or other reporting tools.

## 10. Common Questions

1. How do I re-run the entire WSSI process for a past date?
2. What happens if one of the modules fails during the run?
3. How can I check the status of the ongoing WSSI computation?
4. Where are the final output files located after a successful run?
5. Can I configure the orchestration to run on a different schedule?

## 11. Quick Reference

- **Orchestrator:** Controls pipeline execution.
  - **Snapshots:** Intermediate data captures for debugging.
  - **Synchronization:** Sending final outputs to destination.
- 

# 8.2 WSSI Validation Module

## 1. Module Name & Business Objective

**WSSI Validation Module:** This module performs critical validation checks on the input data for the WSSI process. Its primary role is to ensure data integrity and prevent errors in downstream computations. For instance, it validates that the specified run date falls within a recognized subperiod, which is crucial for accurate historical and forecasted calculations. The business objective is to safeguard the accuracy and reliability of WSSI outputs by catching data quality issues early in the pipeline.

## 2. Required Information

- **Operational Data:**
  - `run.date`: The date provided for the current WSSI run.
  - `subperiod_definitions`: A master list defining valid subperiods and their ranges.

## 3. Business Settings

- `enable.validation`: A global setting that can turn this module on or off.
- `strict.date.validation`: A boolean indicating whether to strictly enforce run date within current subperiod.



## 4. Process Flow

graph TD

```
A[Input: Run Date, Subperiod Data] --> B{Is Run Date valid in Subperiods?};
B -- No --> C[Error: Invalid Date];
B -- Yes --> D{Are other critical inputs valid?};
D -- No --> E[Error: Invalid Input];
D -- Yes --> F[Output: Validated Data];
F --> G[Proceed to next module];
```

## 5. Step-by-Step Actions

1. **Retrieve Run Date:** Fetches the `run.date` parameter *to use as the primary validation reference*.
2. **Validate Subperiod Alignment:** Checks if the `run.date` provided *falls within an existing and active financial subperiod defined in the master data, preventing misaligned computations*.
3. **Validate Key Data Presence:** Confirms the availability and basic format of other critical input data feeds (e.g., raw sales, inventory) *to ensure subsequent modules have the necessary information*.
4. **Report Issues:** Flags any inconsistencies or missing data *to prevent incorrect calculations downstream*.

## 6. Sample Scenario

A user attempts to run the WSSI for a date outside the current financial year. -

**Action:** The WSSI Group Module triggers the Validation Module with `run.date = 2026-01-15`. - **Process:** The Validation Module compares 2026-01-15 against the `subperiod_definitions` which only extend to 2025-12-31. It identifies 2026-01-15 as an invalid date. - **Outcome:** The Validation Module throws an error, halting the WSSI pipeline before any costly computations begin. **Key Takeaway:** Early validation prevents resource waste and ensures data integrity, stopping faulty runs before they impact reporting.

## 7. Results

A SUCCESS status, allowing the pipeline to proceed, or a FAILURE status with detailed error messages indicating the specific validation failures.

## 8. Unusual Cases

- **Missing or delayed data feed:** If the `subperiod_definitions` master data itself is missing or corrupted, the validation cannot proceed correctly.
- **Unexpected data format:** Input files exist but contain data in an unrecognized format, causing parsing errors.

## 9. Next Steps

- If validation passes, the orchestration proceeds to the WSSI Cleaned Sales Computation Module.

- If validation fails, the process stops, and an error report is generated for manual review and data correction.

## 10. Common Questions

1. Why did the WSSI run fail at the validation stage?
2. How do I fix a run\_date validation error?
3. What specific data points are checked by the validation module?
4. Can I bypass validation for testing purposes?
5. What are the common causes of validation errors?

## 11. Quick Reference

- **Data Integrity:** Primary purpose.
  - **Run Date Check:** Essential for time-series accuracy.
  - **Early Error Detection:** Prevents downstream failures.
- 

# 8.3 WSSI Cleaned Sales Computation Module

## 1. Module Name & Business Objective

**WSSI Cleaned Sales Computation Module:** This module processes raw sales data to generate 'cleaned' sales figures for both Last Year (LY) and forecast periods. It aggregates sales by store, fashion grade (FG), brand segment, and subperiod, calculating essential metrics such as revenue, quantity, discount value, discount percentage, and Average Selling Price (ASP). This cleaned data forms the foundation for further performance analysis. The business objective is to provide a standardized, accurate, and consolidated view of sales performance, ready for advanced analytics and planning.

## 2. Required Information

- **Sales Data:**
  - `raw_pos_sales_data`: Detailed point-of-sale transaction data (e.g., date, SKU, quantity, price, discount, store ID).
- **Product Data:**
  - `product_master`: Data linking SKUs to fashion grade (FG) and brand segment.

## 3. Business Settings

- `sales.aggregation.level`: Defines the granular levels for aggregation (e.g., store, FG, brand segment, subperiod).
- `discount.calculation.method`: Specifies how discount percentage is calculated (e.g., total discount value / total revenue).

## 4. Process Flow

graph TD

```
A[Input: Raw Sales Data] --> B[Filter & Cleanse Data];
B --> C[Aggregate by Store, FG, Brand, Subperiod];
C --> D[Calculate Revenue, Quantity, Discount, ASP];
D --> E[Output: Cleaned Sales Data (LY & Forecast)];
```

## 5. Step-by-Step Actions

1. **Ingest Raw Sales:** Reads raw point-of-sale (POS) data *to begin the cleaning process.*
2. **Filter & Cleanse:** Removes erroneous or irrelevant sales records (e.g., returns, internal transfers, test transactions) and **standardizes** data formats *to ensure data quality.*
3. **Aggregate Sales:** Groups sales data by defined dimensions (store, fashion grade, brand segment, subperiod) *to create summary views suitable for higher-level analysis.*
4. **Calculate Metrics:** Computes key performance indicators including total revenue, total quantity, total discount value, average discount percentage, and Average Selling Price (ASP) for each aggregated group *to provide comprehensive sales insights.*
5. **Persist Cleaned Data:** Stores the resulting cleaned sales data (for both Last Year and forecast periods) into the database *for consumption by downstream modules.*

## 6. Sample Scenario

A retail chain has raw sales data for Store A for a specific week. - **Action:** The module ingests raw POS transactions for Store A, Week 20, Item X, Item Y. - **Process:** It filters out returns, then aggregates sales for Store A by Fashion Grade = 'Core' and Brand Segment = 'Premium' for Week 20. It calculates: \* Revenue: \$10,000 \* Quantity: 200 units \* Discount Value: \$500 \* Discount Percentage: 5% \* ASP: \$50 - **Outcome:** WssiCleanedSalesRow and WssiLyCleanedSalesRow entries are generated and stored, showing these consolidated figures for Store A, Week 20, Core/Premium. **Key Takeaway:** This module transforms fragmented raw sales data into structured, meaningful insights, forming the bedrock for all WSSI calculations.

## 7. Results

Cleaned and aggregated sales data, including WssiLyCleanedSalesRow and WssiForecastCleanedSalesRow, persisted in the database.

## 8. Unusual Cases

- **Missing or delayed data feed:** If raw POS sales data is not available, the module will have no input to process, leading to an empty output.
- **Inconsistent product mapping:** If product SKUs cannot be consistently mapped to fashion grades or brand segments, it may lead to unclassified sales or inaccurate aggregations.

## 9. Next Steps

- The generated WssiLyCleanedSalesRow data is used by the WSSI Subperiod AOP Computation Module to determine historical revenue contributions.
- Both WssiLyCleanedSalesRow and WssiForecastCleanedSalesRow are used by the WSSI Actual and Projected Sales & AOP Computation Module for performance analysis.

## 10. Common Questions

1. How does the module handle sales returns or canceled orders?
2. What level of granularity can I expect from the cleaned sales data?
3. How is Average Selling Price (ASP) calculated, especially with discounts?
4. What happens if an SKU is not mapped to a fashion grade or brand segment?
5. How frequently is the raw sales data updated?

## 11. Quick Reference

- **Cleaned Sales:** Standardized and aggregated sales data.
  - **LY:** Last Year sales.
  - **FG:** Fashion Grade.
  - **ASP:** Average Selling Price.
- 

# 8.4 WSSI Subperiod AOP Computation Module

## 1. Module Name & Business Objective

**WSSI Subperiod AOP Computation Module:** This module calculates the Annual Operating Plan (AOP) at a granular subperiod level. It takes AOP data defined at a broader period level and distributes it across the subperiods based on the historical revenue contribution of each subperiod. This ensures that the AOP is accurately proportioned over time, which is critical for planning sales (ROS) and inventory. The business objective is to translate high-level financial targets into actionable, time-phased goals for each category and subperiod, facilitating accurate inventory and sales forecasting.

## 2. Required Information

- **Operational Data:**
  - broad\_period\_aop\_targets: AOP targets defined at a higher level (e.g., annual, quarterly) for revenue and quantity.
- **Sales Data:**
  - WssiLyCleanedSalesRow: Cleaned sales data for Last Year (LY) at the subperiod level (from WSSI Cleaned Sales Computation Module), used for historical contribution rates.

### 3. Business Settings

- `aop.distribution.method`: Specifies the method for distributing AOP (e.g., historical revenue percentage, equal distribution).
- `historical.period.for.aop`: Defines which historical period's data to use for contribution analysis (e.g., last 3 years average).

### 4. Process Flow

graph TD

A[Input: Broad Period AOP] --> B[Retrieve Historical Subperiod Revenue];

B --> C[Calculate Subperiod Contribution Rates];

C --> D[Distribute Broad AOP to Subperiods];

D --> E[Output: Granular Subperiod AOP];

### 5. Step-by-Step Actions

1. **Retrieve Broad AOP: Loads** AOP targets defined at a higher level (e.g., annually, quarterly) *to serve as the total budget to be distributed*.
2. **Calculate Historical Contributions: Analyzes** `WssiLyCleanedSalesRow` data *to determine the historical revenue contribution of each subperiod relative to its broader period (e.g., weekly sales as a percentage of quarterly sales)*.
3. **Distribute AOP: Applies** the calculated historical contribution percentages *to the broad AOP targets, effectively breaking down the larger AOP into granular subperiod targets for revenue and quantity*.
4. **Persist Granular AOP: Stores** the resulting `WssiStoreFgSegSubperiodAopRow` data in the database *for use by subsequent modules that require time-phased AOP figures*.

### 6. Sample Scenario

Your annual AOP revenue target for a specific Fashion Grade is \$1,000,000. -

**Action:** The module receives this annual target. - **Process:** It looks at last year's cleaned sales data and finds that Week 10 historically contributed 2.5% of the total annual revenue for that Fashion Grade. - **Outcome:** The module calculates Week 10's AOP revenue target as  $\$1,000,000 * 2.5\% = \$25,000$ , and this granular AOP is saved. **Key Takeaway:** This module ensures that ambitious annual targets are translated into realistic, weekly or monthly targets, providing a clear roadmap for sales and inventory planning (ROS).

### 7. Results

Granular `WssiStoreFgSegSubperiodAopRow` data, containing AOP revenue and quantity targets for each store-fashion grade-subperiod combination, persisted in the database.

## 8. Unusual Cases

- **Missing historical data:** If `WssilyCleanedSalesRow` data is incomplete for a subperiod, the module may struggle to accurately calculate contribution rates, potentially leading to skewed AOP distribution.
- **Zero historical sales:** If a subperiod had no sales historically, the module might need a fallback mechanism (e.g., equal distribution) to assign AOP to it.

## 9. Next Steps

- The generated `WssiStoreFgSegSubperiodAopRow` data is a critical input for the WSSI Actual and Projected Sales & AOP Computation Module to derive future sales (ROS) and inventory targets.

## 10. Common Questions

1. How is AOP allocated if a new store or product category is introduced?
2. Can we override the historical distribution method with manual adjustments?
3. What historical period is used for calculating contribution rates?
4. How does the module handle seasonal fluctuations in AOP distribution?
5. What happens if the broad period AOP changes mid-year?

## 11. Quick Reference

- **AOP:** Annual Operating Plan.
  - **Subperiod:** Granular time segment (e.g., week, month).
  - **Historical Contribution:** Basis for AOP distribution.
- 

# 8.5 WSSI Actual and Projected Sales & AOP Computation Module

## 1. Module Name & Business Objective

**WSSI Actual and Projected Sales & AOP Computation Module:** This module computes comprehensive actual and projected sales and inventory metrics, alongside the target Annual Operating Plan (AOP) figures, for each store-fashion grade-subperiod combination. For past periods (actual data), it retrieves cleaned sales and inventory. For future periods (projected/AOP), it uses predefined performance parameters (e.g., target discount, margin, cover duration) to derive sales (ROS), inventory, and Cost of Goods Sold (COGS). The `coverDuration` metric relates to Ideal Size Set (ISS) and Optimum Depth (OD) planning, indicating how long current inventory can cover sales. The business objective is to provide a holistic view of current performance against targets and intelligent projections for future inventory needs and sales potential, directly supporting OD and ISS planning.

## 2. Required Information

- **Sales Data:**
  - WssiCleanedSalesRow: Cleaned sales data for actual periods (from WSSI Cleaned Sales Computation Module).
- **Operational Data:**
  - WssiStoreFgSegSubperiodAopRow: Granular subperiod AOP data (from WSSI Subperiod AOP Computation Module).
  - current\_inventory\_data: Real-time or snapshot inventory levels by SKU, aggregated to relevant levels.
  - target\_performance\_parameters: Business-defined targets for discount, margin, target cover duration, etc.

## 3. Business Settings

- target.discount.percentage: Default discount to apply for projected sales.
- target.margin.percentage: Target profit margin for COGS calculation.
- target.cover.duration: Ideal number of weeks/subperiods inventory should cover for forward planning (key for OD, ISS).
- ros.calculation.window: Number of past subperiods to consider for ROS calculation.

## 4. Process Flow

graph TD

```
A[Input: Cleaned Sales, Subperiod AOP, Inventory, Targets] --> B{Is Period Actual or Projected?};
B -- Actual --> C[Retrieve Actuals (Sales, Inventory)];
B -- Projected --> D[Derive Projected Sales (ROS), Inventory, COGS using AOP & Targets];
C --> E[Calculate Cover Duration];
D --> E;
E --> F[Output: Actual & Projected Metrics];
```

## 5. Step-by-Step Actions

1. **Categorize Periods:** **Determines** for each subperiod whether it's an actual (past) or projected (future) period *to apply the correct calculation logic*.
2. **Retrieve Actuals:** For past periods, **fetches** actual cleaned sales data and inventory levels *to represent real-world performance*.
3. **Derive Projections:** For future periods, **calculates** projected sales (Rate of Sale - ROS) and inventory levels *by using the granular AOP targets and business-defined performance parameters (e.g., target discount, margin, target cover duration)*. This step directly influences OD and ISS planning.
4. **Compute COGS:** **Determines** the Cost of Goods Sold (COGS) for both actual and projected periods *based on product cost and sales volume*.
5. **Calculate Cover Duration:** **Computes** coverDuration *by dividing current inventory units by the calculated Rate of Sale (ROS), indicating how many subperiods the current stock can last*. This metric is crucial for OD (Optimum Depth) and ISS (Ideal Size Set) analysis.
6. **Persist Results:** **Stores** the comprehensive actual and projected sales and AOP metrics (WssiActProjRow, WssiAopRow), along with updated

WssiLyCleanedSalesRow (which includes inventory data), in the database *for final reporting*.

## 6. Sample Scenario

A store has an AOP target for 100 units of a product next week. - **Action:** The module processes this target along with current inventory and defined target performance parameters. - **Process:** \* **ROS:** Based on the AOP quantity (100 units) for the subperiod, the calculated Rate of Sale (ROS) is 100 units/week. \* **Inventory:** If `target.cover.duration` is 4 weeks, the ideal projected inventory for next week is 400 units (4 weeks \* 100 units/week ROS). This informs **OD** and **ISS**. \* **Actuals:** For the previous week, actual sales were 90 units, and current inventory is 500 units. The module calculates a `coverDuration` of 5.5 weeks (500 units / 90 units/week). - **Outcome:** The module generates records with actual sales/inventory for past periods and projected sales/inventory/ROS/cover duration for future periods. **Key Takeaway:** This module integrates all financial and inventory data to provide a dynamic, forward-looking view of performance, enabling proactive adjustments to stock levels and sales strategies to achieve **Optimum Depth (OD)** and maintain **Ideal Size Set (ISS)**.

## 7. Results

WssiActProjRow (actual and projected sales/inventory), WssiAopRow (detailed AOP), and updated WssiLyCleanedSalesRow (with inventory context) persisted in the database, containing metrics like ROS, cover duration, COGS, etc.

## 8. Unusual Cases

- **Missing or delayed data feed:** If inventory data or target parameters are missing, projections might be inaccurate or incomplete.
- **Negative inventory/sales:** While raw data might have anomalies, this module should handle or flag cases where calculated ROS or inventory becomes negative, indicating data issues.

## 9. Next Steps

- All computed WssiActProjRow, WssiAopRow, and updated WssiLyCleanedSalesRow data serve as the primary inputs for the WSSI Output Generation Module.

## 10. Common Questions

1. How are future sales (ROS) predicted if historical patterns change?
2. What is `coverDuration` and why is it important for inventory planning?
3. How does the module determine the “ideal” inventory levels?
4. Can I adjust the target discount percentage for specific categories?
5. What’s the difference between actual and projected inventory?



## 11. Quick Reference

- **ROS:** Rate of Sale (units/week).
  - **OD:** Optimum Depth (ideal number of SKUs).
  - **ISS:** Ideal Size Set (best size mix).
  - **Cover Duration:** Inventory weeks of supply.
  - **AOP:** Annual Operating Plan targets.
- 

## 8.6 WSSI Output Generation Module

### 1. Module Name & Business Objective

**WSSI Output Generation Module:** This module consolidates all computed WSSI metrics from the upstream modules, along with inventory and Goods Received Note (GRN) data, to generate the final, comprehensive WSSI output at both store and channel levels. It also plays a key role in tagging styles to appropriate sales channels based on inventory bucket flow logic and GRN dates, influencing how inventory is managed and tracked across different sales channels. The outputs provide a holistic view of actual performance, AOP targets, and LY comparisons for sales revenue, quantity, inventory, cover duration (related to ROS and OD), discount, and ASP. The business objective is to produce user-friendly, comprehensive reports that enable informed decision-making on inventory allocation, sales strategy, and performance evaluation.

### 2. Required Information

- **Sales Data:**
  - `WssiLyCleanedSalesRow`: Last Year (LY) cleaned sales (from WSSI Cleaned Sales Computation Module, updated by WSSI Actual and Projected Sales & AOP Computation Module).
- **Operational Data:**
  - `WssiActProjRow`: Actual and projected sales/inventory metrics (from WSSI Actual and Projected Sales & AOP Computation Module).
  - `WssiAopRow`: Detailed AOP targets (from WSSI Actual and Projected Sales & AOP Computation Module).
  - `WssiStoreFgSegSubperiodAopRow`: Granular AOP data (from WSSI Subperiod AOP Computation Module).
  - `grn_data`: Goods Received Note data (expected incoming inventory).
  - `current_inventory_snapshot`: Current inventory levels.

### 3. Business Settings

- `output.format`: Specifies the desired output format (e.g., CSV, Parquet, JSON).
- `channel.tagging.logic`: Rules for assigning styles to sales channels (e.g., 'Online', 'Retail', 'Clearance') based on inventory status and GRN dates.
- `reporting.granularity`: Defines the aggregation levels for final reports (e.g., store, channel, region).

## 4. Process Flow

graph TD

```
A[Input: All WSSI Metrics, Inventory, GRN] --> B[Consolidate Data];
B --> C[Apply Channel Tagging Logic];
C --> D[Aggregate to Final Reporting Levels];
D --> E[Generate Final WSSI Reports];
E --> F[Output: WSSI Reports (CSV, etc.)];
```

## 5. Step-by-Step Actions

1. **Retrieve All Data: Gathers** all relevant data from upstream modules: WssiActProjRow, WssiAopRow, WssiLyCleanedSalesRow, as well as raw inventory and GRN data *to compile a complete picture*.
2. **Consolidate & Join: Combines** these disparate datasets based on common keys (store, subperiod, fashion grade, brand segment) *to create a unified view of performance metrics*.
3. **Apply Channel Tagging: Categorizes** each style or SKU into an appropriate sales channel (e.g., 'Retail', 'Online', 'Clearance') *based on specific business rules involving inventory status, GRN dates, and sales velocity (ROS), influencing how inventory is tracked and optimized*.
4. **Calculate Channel-Level AOP: Aggregates** store-level AOP contributions to derive channel-level AOP targets, crucial for understanding performance at a broader scale.
5. **Generate Reports: Formats** the consolidated data into final, user-friendly reports (e.g., CSV, Excel) *presenting actuals, projections, AOP targets, and LY comparisons for key metrics like revenue, quantity, inventory, cover duration (related to ROS and OD), discount, and ASP*.
6. **Export Output: Writes** the final reports to the designated output directory *for business consumption and analysis*.

## 6. Sample Scenario

You want a complete performance overview for your “Core” fashion grade across all channels. - **Action:** The module collects all computed actuals, AOPs, and LY data. - **Process:** It consolidates this data, aggregates it by Fashion Grade = 'Core', and applies channel tagging rules (e.g., if an item is in a high inventory bucket and past season, it's tagged “Clearance”). It sums up sales, inventory, and AOP for all stores and channels. - **Outcome:** A comprehensive report is generated, showing that for “Core” items: \* Actual Revenue YTD: \$500,000 (LY: \$450,000, AOP: \$520,000) \* Current Inventory: 10,000 units (Cover Duration: 8 weeks vs. OD target of 6 weeks) \* Tagged “Clearance” items: 1,200 units **Key Takeaway:** This module transforms complex computed data into clear, actionable reports, providing the final holistic view necessary for strategic decision-making on inventory, sales, and channel management.

## 7. Results

Final WSSI reports generated in the specified output format, typically CSV or a database table, providing comprehensive insights at store and channel levels.

## 8. Unusual Cases

- **Missing or delayed data feed:** If any critical computed data (e.g., `WssiActProjRow`) is missing, the final reports will be incomplete or contain gaps.
- **Incorrect channel tagging logic:** Misconfigured rules could lead to styles being tagged incorrectly, affecting inventory flow analysis and sales channel performance insights.

## 9. Next Steps

- Business analysts and merchandise planners use these reports for inventory rebalancing, identifying stockout risks, optimizing Optimum Depth (OD) and Ideal Size Set (ISS), and evaluating performance against AOP.
- Reports can be loaded into BI tools (e.g., Tableau, Power BI) for visualization and interactive analysis.

## 10. Common Questions

1. How can I customize the output report format or content?
2. What are the key metrics included in the final WSSI report?
3. How is the channel tagging logic defined and can it be adjusted?
4. Can this module generate reports for specific regions or product groups only?
5. What is the best way to integrate these reports with our existing BI dashboards?

## 11. Quick Reference

- **Consolidation:** Bringing all data together.
- **Channel Tagging:** Assigning products to sales channels.
- **GRN:** Goods Received Note (incoming stock).
- **Holistic View:** Comprehensive actuals, projections, and AOP.

# 9. Appendix

## Full Parameter Reference

Refer to the `args.properties` file in the root directory for a complete list of configurable parameters and their detailed descriptions.

## Version History

- **Version 1.0 (2025-05-23):** Initial release of the WSSI Documentation. Includes core modules for data validation, cleaned sales computation, subperiod AOP distribution, actual/projected sales and inventory calculation, and final output generation.

## Contact & Support Info

For questions, support, or to report issues, please contact the development team at: \*

**Email:** support@yourcompany.com \* **Internal Support Portal:** [Link to internal portal] \* **Team Lead:** [Name of Team Lead] ``

# Tutorial: wssi

The WSSI system processes sales and inventory data through a sequential pipeline orchestrated by the WSSI Group Module. Initial validation ensures data quality, followed by data cleaning, granular Annual Operating Plan (AOP) distribution, actual/projected sales and inventory computation, and finally, comprehensive output generation. Data flows between modules primarily via persisted database tables, ensuring data integrity and enabling complex business calculations like Rate of Sale (ROS), Optimum Depth (OD), and Ideal Size Set (ISS).

flowchart TD

```
A0["WSSI Group Module (Orchestration)"]
A1["WSSI Validation Module"]
A2["WSSI Cleaned Sales Computation Module"]
A3["WSSI Subperiod AOP Computation Module"]
A4["WSSI Actual and Projected Sales & AOP Computation Module"]
A5["WSSI Output Generation Module"]
A1 -- "precedes" --> A0
A0 -- "executes" --> A2
A0 -- "executes" --> A3
A0 -- "executes" --> A4
A0 -- "executes" --> A5
A2 -- "data_flow" --> A3
A2 -- "data_flow" --> A4
A3 -- "data_flow" --> A4
A4 -- "data_flow" --> A5
A2 -- "data_flow" --> A5
A3 -- "data_flow" --> A5
```

## Chapters

# Master Report: wssi Merchandising Algorithm

## 1. Title & Metadata

- **Project Name:** wssi
- **Version:** 1.0
- **Last Updated:** 2025-05-23
- **One-line Elevator Pitch:** The wssi algorithm leverages sales, inventory, and product data to optimize product assortment, depth, and inventory levels, maximizing profitability and minimizing stockouts.

## 2. Executive Summary

The wssi merchandising algorithm is a critical component in our strategy to enhance product assortment efficiency and inventory management. By employing advanced analytical techniques, it provides data-driven recommendations for **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)**, while also optimizing inventory levels based on **Rate of Sale (ROS)**. This system aims to significantly improve sales performance, reduce inventory carrying costs, and enhance customer satisfaction by ensuring the right products are available at the right time.

- **Key outcomes or KPIs:**
  - Increased sell-through rates and improved ROS.
  - Reduced stockouts and overstock situations.
  - Optimized inventory turnover.
  - Maximized gross margin return on investment (GMROI).
  - Enhanced product assortment relevance and appeal.
- **Primary stakeholders or downstream consumers:** Merchandising Teams, Planning & Allocation Teams, Inventory Management, Store Operations, E-commerce Teams, and Executive Leadership for strategic decision-making.

## 3. Architecture & Data Flow

### Diagram

graph TD

```
A[Raw Sales Data] --> B(Data Ingestion & Validation)
C[Product Master Data] --> B
D[Inventory Data] --> B
E[Vendor Data] --> B
```

```

B --> F{Data Preprocessing}
F --> G[Demand Forecasting (ROS)]
F --> H[Attribute Group (AG) Analysis]
G --> I[Inventory Optimization]
H --> J[Assortment Planning Engine]
J -- OD, OW, ISS --> K[Recommendation Generation]
I --> K
K --> L[Output Data Storage]
L --> M[Reporting & Dashboarding]
K --> N[Allocation System]
M --> O[Merchandising Teams]
N --> O

```

## Component Bullets

- **Data Ingestion & Validation:** Responsible for collecting raw data from various sources, cleaning, and validating its integrity and consistency.
- **Data Preprocessing:** Transforms raw data into a structured format suitable for analytical models, including aggregation, feature engineering, and normalization.
- **Demand Forecasting (ROS):** Predicts future product demand based on historical sales, seasonality, trends, and external factors, calculating the Rate of Sale for individual SKUs and categories.
- **Attribute Group (AG) Analysis:** Identifies and analyzes product attributes (e.g., color, size, material, brand) to understand their performance characteristics and group them for optimized assortment planning.
- **Inventory Optimization:** Determines optimal stock levels for each SKU across various locations, considering lead times, carrying costs, and service level targets.
- **Assortment Planning Engine:** The core logic responsible for calculating **Optimum Depth (OD)**, **Optimum Width (OW)**, and **Ideal Size Set (ISS)** based on forecasted demand, sales performance by AG, and strategic business rules.
- **Recommendation Generation:** Consolidates outputs from forecasting, inventory optimization, and assortment planning to generate actionable recommendations for merchandising and allocation teams.
- **Output Data Storage:** Stores all generated recommendations, reports, and intermediate data for historical tracking and future analysis.
- **Reporting & Dashboarding:** Provides user-friendly interfaces for stakeholders to visualize key metrics, recommendations, and performance trends.
- **Allocation System Integration:** Feeds optimized inventory and assortment recommendations directly into downstream allocation systems for execution.

## High-level data sources

- **Point-of-Sale (POS) Data:** Transactional sales data (SKU, quantity, price, date, location).
- **Product Master Data:** Product definitions, hierarchies (category, sub-category), attributes (AGs), cost, retail price.
- **Inventory Data:** Current stock levels, in-transit inventory, stock receipts, returns, adjustments by location.
- **Supply Chain Data:** Vendor lead times, order minimums, case pack quantities.

- **Promotional Calendars:** Information on planned marketing events and promotions.

## 4. Configuration & Data Dictionary

Settings Table (`args.properties`)

| Parameter Name          | Description                                                                              | Default Value           | Type    | Example Value |
|-------------------------|------------------------------------------------------------------------------------------|-------------------------|---------|---------------|
| forecast_horizon_weeks  | The number of weeks into the future for demand forecasting.                              | 12                      | Integer | 26            |
| min_ros_threshold       | Minimum ROS (units/week) for a SKU to be considered for reorder/stocking.                | 0.5                     | Decimal | 0.2           |
| inventory_buffer_factor | Multiplier for safety stock calculation.                                                 | 1.15                    | Decimal | 1.2           |
| od_granularity_level    | Level at which Optimum Depth is calculated (e.g., ‘category’, ‘sub_category’).           | category                | String  | sub_cat       |
| ow_calculation_method   | Method for calculating Optimum Width (e.g., ‘sales_contribution’, ‘market_share’).       | sales_contribution      | String  | market        |
| iss_algorithm_type      | Algorithm type for Ideal Size Set (e.g., ‘historical_proportional’, ‘cluster_analysis’). | historical_proportional | String  | cluster       |
| data_refresh_frequency  | How often input data is refreshed (e.g., ‘daily’, ‘weekly’).                             | weekly                  | String  | daily         |
| output_path             | Directory where all final reports and recommendations are saved.                         | /data/output/wssi       | String  | /report/merch |

Shared Data Fields (`common-data.json`)

| Field Name   | Description                                 | Data Type | Example Value     |
|--------------|---------------------------------------------|-----------|-------------------|
| sku_id       | Unique identifier for a Stock Keeping Unit. | String    | “SHIRT-RED-M-001” |
| product_name | Name of the product.                        | String    |                   |



| Field Name         | Description                                                         | Data Type | Example Value      |
|--------------------|---------------------------------------------------------------------|-----------|--------------------|
|                    |                                                                     |           | “Men’s Cotton Tee” |
| category           | High-level product grouping.                                        | String    | “Apparel”          |
| sub_category       | More granular product grouping.                                     | String    | “T-Shirts”         |
| attribute_group    | A set of product attributes grouped for analysis (AG), e.g., Color. | String    | “Color: Red”       |
| size               | Product size (e.g., S, M, L, XL).                                   | String    | “M”                |
| sales_units_weekly | Actual sales units per week for a given SKU/location.               | Integer   | 15                 |
| on_hand_units      | Current inventory units available at a location.                    | Integer   | 50                 |
| weeks_of_supply    | On-hand units divided by average weekly sales.                      | Decimal   | 3.3                |
| ros_forecast       | Predicted Rate of Sale (units/week) for a SKU.                      | Decimal   | 18.2               |

## Outputs Catalog

| Output Name                        | Description                                                | Format   | Frequency |
|------------------------------------|------------------------------------------------------------|----------|-----------|
| optimal_assortment_summary.csv     | Recommended <b>OD</b> and <b>OW</b> per category/location. | CSV      | Weekly    |
| ideal_size_set_recommendations.csv | Specific <b>ISS</b> for key products/categories.           | CSV      | Weekly    |
| sku_inventory_targets.csv          | Recommended optimal inventory levels per SKU per location. | CSV      | Daily     |
| ros_forecast_report.csv            | Detailed weekly <b>ROS</b> forecasts by SKU and location.  | CSV      | Daily     |
| performance_dashboard.pdf          | Executive summary of algorithm impact on KPIs.             | PDF/HTML | Weekly    |
| attribute_group_insights.json      |                                                            | JSON     | Monthly   |

| Output Name | Description                                            | Format | Frequency |
|-------------|--------------------------------------------------------|--------|-----------|
|             | Analysis of performance and trends within various AGs. |        |           |

## 5. End-to-End Flow

The wss i merchandising algorithm executes through a well-defined pipeline, ensuring data integrity and producing actionable insights:

1. **Data Ingestion:** Raw sales, inventory, product master, and vendor data are ingested from various enterprise systems (e.g., ERP, POS).
2. **Data Validation & Cleaning:** The ingested data undergoes rigorous validation checks for completeness, accuracy, and consistency. Missing values are imputed, and outliers are addressed.
3. **Data Preprocessing:** Data is transformed. This includes aggregating sales data to weekly or monthly levels, calculating historical ROS, and enriching product data with necessary attributes and hierarchies for AG analysis.
4. **Demand Forecasting:** Utilizing the preprocessed historical sales data, the Demand Forecasting submodule generates future ROS predictions for each SKU, considering seasonality and trends.
5. **Attribute Group (AG) Analysis:** Product attributes are analyzed to identify high-performing and underperforming AGs, informing decisions about optimal breadth (OW).
6. **Assortment Planning Engine:** Using forecasted ROS and AG performance, the Assortment Planning Engine calculates:
  - **Optimum Depth (OD):** The ideal number of unique SKUs to stock within a specific category or display, balancing variety with efficiency.
  - **Optimum Width (OW):** The ideal number of different categories or styles to offer, ensuring broad appeal without excessive complexity.
  - **Ideal Size Set (ISS):** The best combination of sizes for specific products or AGs, minimizing stockouts and maximizing sales by aligning with customer demand patterns.
7. **Inventory Optimization:** Based on the forecasted ROS, recommended OD/OW/ISS, and supply chain constraints, optimal inventory levels (including safety stock) are calculated for each SKU at each location.
8. **Recommendation Generation:** All outputs are synthesized into actionable recommendations for merchandising, including buy quantities, allocation strategies, and markdown suggestions.
9. **Output & Reporting:** Final recommendations are stored in a database and presented via interactive dashboards and detailed reports for merchandising and planning teams. These outputs can also be fed directly into allocation systems for automated execution.

## 6. Submodule Overview

### 6.1. Data Ingestion & Validation

- **Purpose:** To collect, clean, and validate all raw input data necessary for the merchandising algorithm.
- **Inputs & Outputs:**
  - **Inputs:** Raw sales data (units, revenue, SKU, date, location), Product Master (SKU, attributes, category), Inventory (on-hand, in-transit), Vendor (lead times, MOQs).
  - **Outputs:** Cleaned and validated `sales_transactions_clean.csv`, `product_master_clean.csv`, `inventory_snapshot_clean.csv`.
- **Core Logic Summary:**
  1. Read raw data files from specified input directories.
  2. Perform schema validation (e.g., check for expected columns, data types).
  3. Identify and handle missing values (e.g., imputation, flag for review).
  4. Detect and flag outliers (e.g., sales spikes/drops) for further investigation or smoothing.
  5. Standardize data formats (e.g., date formats, SKU naming conventions).

### 6.2. Demand Forecasting (ROS)

- **Purpose:** To predict future sales velocity (Rate of Sale) for individual SKUs and product categories.
- **Inputs & Outputs:**
  - **Inputs:** Cleaned historical sales data, promotional calendar, relevant external factors (e.g., holidays).
  - **Outputs:** `sku_ros_forecast.csv` (containing `sku_id`, `location`, `forecast_week`, `ros_forecast`).
- **Core Logic Summary:**
  1. Aggregate historical sales data by SKU, location, and week to establish time series.
  2. Apply time series forecasting models (e.g., ARIMA, Prophet, Exponential Smoothing) considering seasonality and trend.
  3. Incorporate impact of planned promotions and external events into the forecast.
  4. Generate probabilistic forecasts, providing both point estimates and confidence intervals for ROS.

### 6.3. Attribute Group (AG) Analysis

- **Purpose:** To analyze the performance of various product attributes and their combinations (AGs) to inform assortment decisions.
- **Inputs & Outputs:**
  - **Inputs:** Cleaned product master data, historical sales data, `sku_ros_forecast.csv`.
  - **Outputs:** `attribute_group_performance.json` (containing `ag_name`, `total_sales`, `avg_ros`, `contribution_margin`).
- **Core Logic Summary:**
  1. Join sales data with product master to enrich transactions with AG information (e.g., Color, Material, Brand, Fit).

2. Calculate key performance metrics for each AG, such as total sales, average ROS, gross margin, and inventory turn.
3. Identify top-performing and underperforming AGs based on configurable thresholds.
4. Generate insights on AG performance trends and interdependencies.

## 6.4. Assortment Planning Engine

- **Purpose:** To determine the optimal variety and quantity of products to offer, defining **OD**, **OW**, and **ISS**.
- **Inputs & Outputs:**
  - **Inputs:** sku\_ros\_forecast.csv, attribute\_group\_performance.json, inventory\_snapshot\_clean.csv, args.properties (for od\_granularity\_level, ow\_calculation\_method, iss\_algorithm\_type).
  - **Outputs:** optimal\_assortment\_summary.csv, ideal\_size\_set\_recommendations.csv.
- **Core Logic Summary:**
  1. **Optimum Width (OW) Calculation:** Analyze category and style performance (using ow\_calculation\_method), current market trends, and competitive landscape to recommend the ideal number of distinct categories or styles.
  2. **Optimum Depth (OD) Calculation:** Within recommended categories/ styles, use forecasted ROS and AG performance to determine the ideal number of unique SKUs to stock, balancing breadth vs. depth.
  3. **Ideal Size Set (ISS) Calculation:** For each product or AG, analyze historical size sales proportions (iss\_algorithm\_type) and forecasted demand to recommend the optimal mix of sizes to stock, minimizing stockouts.
  4. Incorporate business rules and constraints (e.g., minimum viable stock levels, vendor minimum order quantities) into recommendations.

## 6.5. Inventory Optimization

- **Purpose:** To calculate the ideal stock levels for each SKU at each location to meet demand while minimizing costs.
- **Inputs & Outputs:**
  - **Inputs:** sku\_ros\_forecast.csv, optimal\_assortment\_summary.csv, ideal\_size\_set\_recommendations.csv, inventory\_snapshot\_clean.csv, args.properties (inventory\_buffer\_factor).
  - **Outputs:** sku\_inventory\_targets.csv (containing sku\_id, location, target\_inventory\_units, reorder\_point).
- **Core Logic Summary:**
  1. Calculate demand during lead time using forecasted ROS and supplier lead times.
  2. Determine safety stock based on forecast variability and desired service level (using inventory\_buffer\_factor).
  3. Calculate reorder points and target inventory levels, considering current on-hand units and in-transit inventory.
  4. Factor in assortment recommendations (OD, OW, ISS) to ensure inventory aligns with strategic stocking decisions.

## 7. Worked Example

Let's illustrate how **Optimum Depth (OD)** might be calculated for a specific category, "Men's T-Shirts," considering forecasted ROS and profitability.

**Scenario:** We need to determine the OD for "Men's T-Shirts" in a specific store, aiming for variety while ensuring high sell-through.

### 1. Input Data:

- **Category:** Men's T-Shirts
- **Available SKUs in Category:** 50 unique SKUs (combinations of style, color, size, material).
- **ROS Forecast for Top 10 SKUs (units/week):**
  - SKU A (Basic White Cotton): 25
  - SKU B (Graphic Black Poly): 22
  - SKU C (Basic Navy Cotton): 20
  - SKU D (Basic Grey Cotton): 18
  - SKU E (Graphic White Poly): 15
  - SKU F (Striped Red Cotton): 10
  - SKU G (V-Neck Green Cotton): 8
  - SKU H (Logo Black Poly): 7
  - SKU I (Tie-Dye Blue Cotton): 5
  - SKU J (Basic Yellow Cotton): 3
- **ROS Forecast for other 40 SKUs:** Ranging from 0.1 to 2 units/week.
- **Min ROS Threshold (args.properties): 0.5 units/week.**
- **Strategic Objective:** Prioritize SKUs with high ROS and maintain enough variety (OD) to appeal to customers.

### 2. Processing Steps in Assortment Planning Engine (OD Component):

- **Step 1: Filter by Minimum ROS:** The algorithm first filters out SKUs below the `min_ros_threshold` (0.5 units/week). In this example, 35 out of the 40 "other SKUs" fall below this, leaving us with 15 SKUs.
- **Step 2: Calculate Cumulative ROS Contribution:** The algorithm sorts the remaining SKUs by their forecasted ROS in descending order and calculates their cumulative contribution to the total ROS for the category.
  - SKU A: 25 (20.5%)
  - SKU B: 22 (18.0%)
  - SKU C: 20 (16.4%)
  - SKU D: 18 (14.8%)
  - SKU E: 15 (12.3%)
  - SKU F: 10 (8.2%)
  - SKU G: 8 (6.6%)
  - SKU H: 7 (5.7%)
  - SKU I: 5 (4.1%)
  - SKU J: 3 (2.5%)
  - *Remaining 5 SKUs (each > 0.5 ROS):* Total 2 units (1.6%)
  - Total ROS for filtered SKUs: 122 units/week.
- **Step 3: Apply OD Rule:** Based on a pre-defined rule (e.g., "select SKUs that contribute to 90% of the category's total forecasted ROS" or "target 10-15 unique SKUs per category"), the algorithm determines the OD.
  - If the rule is "90% cumulative ROS":
    - A (20.5%)
    - B (38.5%)
    - C (54.9%)
    - D (69.7%)

- E (82.0%)
  - F (90.2%) - *SKU F brings us over 90%*
  - This would recommend an OD of 6 SKUs for this category.
  - If the rule is “target 10-15 unique SKUs, prioritizing high ROS”:
    - The algorithm would recommend stocking the top 10 SKUs (A-J) to maintain sufficient variety and capture significant demand. This would result in an OD of 10 SKUs.
3. **Output:** `optimal_assortment_summary.csv` would contain:
- category: “Men’s T-Shirts”
  - location: “Store X”
  - recommended\_od: 6 or 10 (depending on chosen rule)
  - recommended\_skus\_for\_od: List of the specific SKU IDs (e.g., A, B, C, D, E, F if 6 SKUs).

This example demonstrates how the algorithm uses forecasted ROS and defined thresholds to make a data-driven recommendation for the optimal number of unique items (OD) to stock, balancing sales potential with assortment efficiency.

## 8. Debug Guide Highlights

- **Low ROS Forecasts:**
  - **Symptom:** Algorithm consistently recommends delisting popular items or low inventory targets for strong sellers.
  - **Debug:** Check Data Ingestion & Validation for missing historical sales data or incorrect date ranges. Verify Demand Forecasting model parameters and recent external shocks (e.g., COVID-19, competitor activity) not captured by model. Review `min_ros_threshold` in `args.properties`.
- **Unrealistic OD/OW/ISS Recommendations:**
  - **Symptom:** Suggested OD is too narrow/wide, or ISS doesn’t align with historical sales patterns (e.g., recommending only small sizes for a category historically strong in large sizes).
  - **Debug:** Review the Attribute Group Analysis output for unexpected performance trends. Check the Assortment Planning Engine configuration, specifically `od_granularity_level`, `ow_calculation_method`, and `iss_algorithm_type` in `args.properties` and associated business rules. Ensure complete and accurate `product_master_clean.csv`.
- **Excessive Stockout/Overstock Issues:**
  - **Symptom:** Despite algorithm recommendations, stores frequently face stockouts or excessive inventory.
  - **Debug:** Validate `on_hand_units` and `in_transit` data from `inventory_snapshot_clean.csv`. Review Inventory Optimization parameters: `inventory_buffer_factor`, supplier lead times. Check if ROS forecasts are consistently off (refer to ROS debug). Ensure integration with allocation systems is correct and recommendations are being actioned.
- **Data Latency/Staleness:**
  - **Symptom:** Recommendations are based on old data; performance metrics don’t reflect current reality.
  - **Debug:** Verify `data_refresh_frequency` in `args.properties`. Check data pipeline logs for delays or failures in Data Ingestion and Data Preprocessing steps.

## 9. Common Pitfalls & Remedies

- **Pitfall 1: Garbage In, Garbage Out (GIGO)**
  - **Description:** Inaccurate, incomplete, or inconsistent input data leads to flawed recommendations.
  - **Remedy:** Implement robust data validation checks at the Data Ingestion & Validation stage. Establish clear data governance policies, regular data audits, and automated data quality alerts. Invest in data source reliability.
- **Pitfall 2: Over-reliance on Historical Data for ROS**
  - **Description:** Future demand may not always mirror the past, especially with market shifts, new trends, or promotions.
  - **Remedy:** Incorporate external market data (e.g., competitor sales, economic indicators), promotional lift factors, and new product launch forecasts into the Demand Forecasting model. Regularly review forecast accuracy and adjust model parameters.
- **Pitfall 3: Neglecting Qualitative Merchandising Input**
  - **Description:** Purely algorithmic recommendations can miss nuances like fashion trends, local market preferences, or brand strategy.
  - **Remedy:** Design the system to allow for human override or input on certain decisions. Provide merchandisers with detailed context and performance drivers for recommendations, fostering trust and enabling informed adjustments. Establish a feedback loop for merchandisers to provide input to algorithm development.
- **Pitfall 4: Suboptimal Attribute Grouping**
  - **Description:** Poorly defined Attribute Groups (AGs) can lead to misleading performance insights and sub-optimal **OD/ OW** decisions.
  - **Remedy:** Periodically review and refine Attribute Groups based on evolving product classifications and market trends. Use data-driven methods (e.g., clustering) to identify natural product groupings.
- **Pitfall 5: Lack of Performance Monitoring & Iteration**
  - **Description:** Failing to track the real-world impact of the algorithm's recommendations means it cannot be improved over time.
  - **Remedy:** Establish clear KPIs and robust Reporting & Dashboarding. Regularly compare actual sales/inventory performance against algorithm recommendations. Conduct A/B testing on different algorithm configurations or new features. Implement a continuous improvement loop for model retraining and parameter tuning.

## 10. FAQ

- **Q1: How does the algorithm handle new product introductions (NPIs) with no historical sales data?**
  - **A:** For NPIs, the Demand Forecasting submodule uses proxy data from similar past products or category averages. Merchandising teams can also provide initial qualitative forecasts, which the system will gradually learn from as sales data accumulates.
- **Q2: Can the algorithm account for store-specific differences (e.g., regional preferences)?**
  - **A:** Yes, the Demand Forecasting, Assortment Planning Engine, and Inventory Optimization submodules are designed to operate at a

granular level (e.g., SKU-store combination) where sufficient data exists.

**Attribute Group (AG)** analysis can also identify regionally popular AGs.

• **Q3: What if a SKU performs well, but its recommended OD/OW is low due to strategic reasons?**

- **A:** The algorithm's recommendations are data-driven but can be influenced by configurable business rules. If a strategic decision requires maintaining a certain product despite low ROS, parameters can be set (e.g., "must-stock" flags, minimum depth allowances) to ensure its inclusion in the **OD** or **OW**.

• **Q4: How frequently are the recommendations updated?**

- **A:** This depends on the `data_refresh_frequency` in `args.properties`. Typically, ROS forecasts and inventory targets are updated daily, while **OD**, **OW**, and **ISS** recommendations are updated weekly or bi-weekly to allow for planning cycles.

• **Q5: Is it possible to see the reasoning behind a specific recommendation?**

- **A:** The Reporting & Dashboarding output provides drill-down capabilities showing the underlying ROS forecasts, AG performance, and key metrics that informed the recommendations. While it doesn't provide a natural language "reason," the data points driving the decision are transparent.

## 11. Quick Reference Cheat-Sheet

| Term                    | Definition                                                                                           | Key Modules Affected                                     |
|-------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>OD</b>               | <b>Optimum Depth:</b> The ideal number of unique items (SKUs) to stock in a category or display.     | Assortment Planning Engine, Recommendation Generation    |
| <b>OW</b>               | <b>Optimum Width:</b> The ideal number of different categories or styles to offer.                   | Assortment Planning Engine, Recommendation Generation    |
| <b>ISS</b>              | <b>Ideal Size Set:</b> The best combination of sizes to maximize sales and minimize stockouts.       | Assortment Planning Engine, Inventory Optimization       |
| <b>ROS</b>              | <b>Rate of Sale:</b> The speed at which products are sold over a period (e.g., units/week).          | Demand Forecasting, Inventory Optimization, all planning |
| <b>AG</b>               | <b>Attribute Group:</b> A set of product attributes grouped for analysis (e.g., color, size, brand). | Attribute Group Analysis, Assortment Planning Engine     |
| <b>args.properties</b>  | Configuration file for algorithm parameters.                                                         | All modules (controls behavior)                          |
| <b>common-data.json</b> | Defines common data fields used across modules.                                                      | Data Ingestion, Data Preprocessing                       |
| <b>Mermaid</b>          | Diagramming tool used for architecture visualization.                                                | Architecture & Data Flow                                 |
| <b>KPIs</b>             |                                                                                                      |                                                          |



| Term | Definition                                                                       | Key Modules Affected                        |
|------|----------------------------------------------------------------------------------|---------------------------------------------|
|      | Key Performance Indicators (e.g., GMROI, Inventory Turnover, Sell-Through Rate). | Executive Summary, Reporting & Dashboarding |
| SKU  | Stock Keeping Unit - a specific unique product item.                             | All modules                                 |